The Weil Conjectures
A Lean 4 Formalization Proof Blueprint

Development version: strict mathlib foundation boundary

4 August 2026

Development status. This file is not yet a proof of the Weil conjectures. It contains only the topologically initial declarations whose proofs have been expanded below the Nagata granularity threshold. The exact unresolved geometric and cohomological statements are recorded in `weil_open_obligations.md`. No unresolved statement is treated as a theorem in this file.

Contents

1 Global data and normalization **1**

2 Nagata compactification **4**
2.1 Compactifications and scheme-theoretic images . . . 4
2.2 Separated gluing . . . 7
2.3 Construction of blow-ups from Rees algebras . . . 8
2.3.1 The graded Rees algebra . . . 8
2.3.2 Affine blow-ups and their standard charts . . . 10
2.3.3 Localization compatibility and coherence . . . 26
2.3.4 Global blow-up by gluing . . . 28
2.3.5 Complement, properness, and principalization . . . 33
2.3.6 Universal property, flat base change, and products of ideals . . . 36
2.3.7 An arbitrary composition of two blow-ups . . . 39
2.4 Admissible blow-ups and strict transforms . . . 41
2.4.1 Power torsion and strict-transform charts . . . 41
2.4.2 Atomic flattening and elimination of indeterminacy . . . 57
2.5 Separating boundary pieces . . . 71
2.6 Common modifications and graph extension . . . 75
2.7 The dense-source valuative criterion . . . 78
2.8 The two-open compactification theorem . . . 87
2.9 The Noetherian case . . . 93
2.10 Finite-presentation reduction and approximation . . . 102
2.10.1 Finite data and absolute Noetherian models . . . 102
2.10.2 Embedding a finite-type scheme into a finite-presentation model . . . 117
2.10.3 Descent of relative finite-presentation data . . . 122
2.10.4 Eventual separatedness of the descended model . . . 132
2.11 Lean interfaces and implementation order . . . 133
2.12 Explicit final assembly . . . 134

3 Zeta functions and finite Frobenius point counting **137**
3.1 Coefficientwise exponential and logarithm . . . 137
3.2 Locally finite sums and products . . . 143
3.3 Single Euler factors . . . 148
3.4 Finite geometric-Frobenius orbits . . . 152
3.5 Orbit Euler products and the counting zeta series . . . 158
3.6 Abstract finite-extension and closed-point interfaces . . . 163
3.7 Integral Euler-family packages for the literal lift . . . 169
3.8 Literal integral power-series realization . . . 172

4 The present mathlib ℓ-adic endpoint and the Frobenius interface **182**
4.1 Exactly verified big and small sites . . . 182
4.2 The exact integral endpoint currently present in mathlib . . . 183
4.3 Conditional scalar extension, with no comparison theorem assumed . . . 184
4.4 Typed field, local, and scheme Frobenius data . . . 185
4.4.1 Comparison with ordinary categorical powers . . . 188
4.5 Abstract Weil data and ordered cohomological iterates . . . 190
4.6 Tate-object targets without an existence assertion . . . 193
4.7 Conditional finite-dimensional Frobenius-module interface . . . 195
4.8 Typed conditional context for the six operations . . . 196
4.9 Insertion-stable Module 02 merge certificate . . . 199

5 From traces to determinants and conditional rationality **202**
5.1 Formal-series group laws used by the determinant calculation . . . 202
5.2 Polynomial and matrix coordinates . . . 204
5.3 Rebasing and independence of the displayed basis . . . 210
5.4 Scalar extension and the root–eigenvalue dictionary . . . 211
5.5 Finite graded families and alternating determinants . . . 213
5.5.1 Formal compact-support Frobenius-family adapter . . . 213
5.6 Normalized rational germs and Taylor expansion at zero . . . 217
5.7 Normalized rational germs and descent from a coefficient extension . . . 223
5.8 Conditional trace formula and cohomological factorization . . . 224

6 Duality and the functional equation **228**
6.1 Perfect pairings with an explicit inverse . . . 228
6.2 The Frobenius similitude calculation . . . 230
6.3 Conditional geometric construction of the Poincare pairing . . . 236
6.4 A bounded graded duality certificate . . . 243
6.5 Assembly of the rational-function identity . . . 247
6.6 Normalization regression tests . . . 249

7 Weights, the Deligne–Laumon estimate, and the Riemann hypothesis **250**
7.1 Scalar weights and their elementary algebra . . . 250
7.2 Spectra, Galois stability, and weight filtrations . . . 253
7.3 The PUR chain of Weil II 1.3–1.5 . . . 259
7.4 Nilpotent monodromy and the MON package . . . 267
7.4.1 The intrinsic nilpotent weight filtration . . . 270
7.4.2 Intrinsic characterization and uniqueness . . . 273
7.4.3 Typed primitive strings from a selected Jordan decomposition . . . 278
7.4.4 Typed signed dual datum and conditional assembly . . . 285
7.5 Artin–Schreier and global Fourier–Deligne constructions . . . 292
7.6 Local Fourier transform and stationary phase . . . 296
7.7 Laumon's curve estimate . . . 300
7.8 Deligne's direct-image devissage . . . 308
7.8.1 The ten components of the relative-curve tame reduction . . . 320
7.9 Smooth proper purity and the Riemann-hypothesis modulus . . . 329

8 Betti degrees, integral factors, and independence of ℓ **335**
8.1 Three weight predicates and the realization-field interface . . . 335
8.2 Conditional arithmetic recovery input and degree computation . . . 358
8.3 Scalar descent of rationality by Hankel rank . . . 360
8.4 The integral Fatou lattice and a reduced integral fraction . . . 366

8.5 No cancellation and identification with the reduced fraction . . . 375
8.6 Rational and integral individual factors . . . 378
8.7 Compatible realizations and the unique weight strata . . . 383
8.8 Degrees and realization-independent Betti numbers . . . 386
8.9 Recovery and independence of functional-equation constants . . . 386
8.10 Conditional comparison with singular Betti numbers . . . 390
8.11 Bridge from the weight module to arithmetic recovery . . . 394

9 Final conditional assembly of the Weil conjectures 395
9.1 The classical scheme and its two point-counting presentations . . . 395
9.2 The conditional cohomological master package . . . 402
9.2.1 Supplied proper-support adapters . . . 404
9.3 Degree zero and the normalized top trace . . . 407
9.4 Common integral factors, degrees, and rationality over the rationals . . . 411
9.5 Functional equation and the two root variables . . . 413
9.6 Optional characteristic-zero lift and terminal assembly . . . 416
9.7 Finite torsion constructible derived categories . . . 417
9.7.1 Universes, sites, and coefficient sheaves . . . 417
9.7.2 Constructible sheaves and elementary closure . . . 419
9.7.3 Etale–pro-etale comparison at finite level . . . 421
9.7.4 Bounded derived localization and the standard t-structure . . . 424
9.7.5 Tensor and pullback at derived level . . . 427
9.7.6 A1 realization and the A2 finite-level rebind . . . 428

10 Exceptional direct image from compactifications 431
10.1 Finite torsion input and open extension by zero . . . 431
10.2 Proper pushforward and the torsion comparison certificate . . . 438
10.3 Finite-level local data . . . 439
10.4 Assembly of the finite comparison . . . 439
10.5 Coherence export to the A1 interface . . . 440
10.6 Compactification data and refinement comparisons . . . 441
10.7 Common refinements and compactification independence . . . 443
10.8 Definition and composition of $Rf_!$ . . . 445
10.9 Compact-support base change . . . 447
10.10 The pinned integral coefficient ring . . . 449
10.11 Module sheaves and the ambient derived category . . . 452
10.12 Atomic integral scalar action and its cohomology action . . . 452
10.13 K-flat models and finite reduction . . . 457
10.14 General unbounded derived functors . . . 461
10.15 A K-flat/K-injective construction on unbounded ringed-topos derived categories . . . 470
10.15.1 Weil coefficient specializations . . . 477
10.15.2 Selected unbounded replacement discharges . . . 479
10.16 Derived inverse limits and completion . . . 480
10.17 Lisse and constructible complete complexes . . . 482
10.18 Rational coefficients by explicit linearization . . . 486
10.19 Finite extensions and their transition functors . . . 489
10.20 The explicit filtered bicategorical colimit for algebraically closed coefficients . . . 491
10.21 Compatible finite-level systems and reconstruction . . . 494
10.22 The exact proper-complete interface used by pro-etale base change . . . 497
10.23 Coefficient-change compatibility interfaces . . . 501

11 Stacks route: comparison through the strictly henselian projective line 503
11.1 S0. Comparison data and injective acyclicity . . . 503
11.2 S1. Finite morphisms . . . 511
11.3 S2. Composition and proper-surjective cancellation . . . 512
11.4 S3. Reduction to the projective line . . . 513
11.5 S4. The projective line over a strictly henselian local base . . . 514
11.6 S5. The projective line over an arbitrary base . . . 521
11.6.1 S5.1. Stalks through strict henselization . . . 521
11.6.2 S5.2. Proper curves under separably closed extension . . . 524
11.6.3 S5.3. Stalkwise projective-line base change . . . 532
11.7 S6. Finite-torsion proper base change . . . 533
11.8 S7. Bounded-below finite-torsion complexes . . . 534
11.9 S8. Etale-to-pro-etale finite-level transport . . . 537
11.10 S9. Derived completion and inverse limits . . . 550
11.11 S10. Complete pro-etale proper base change . . . 560
11.12 Exceptional inverse image and Verdier duality . . . 573
11.12.1 Coefficient, Frobenius, and twist conventions . . . 573
11.12.2 Finite-level construction of the exceptional inverse image . . . 574
11.12.3 The bounded-below coproduct calculation and the unbounded gap . . . 620
11.12.4 Underived and derived internal Hom . . . 641
11.12.5 Adic reconstruction and coefficient change . . . 647
11.12.6 Composition, units, counits, and projection . . . 650
11.12.7 Smooth purity and the trace normalization . . . 654
11.12.8 Verdier duality and the proper case . . . 656
11.12.9 Poincare pairing and Frobenius similitude . . . 660
11.12.10 Coefficient and base-change compatibility rectangles . . . 664

12 A5. Smooth base change and compact-support transport 666
12.1 A5.1. Smooth squares and coefficient data . . . 666
12.2 A5.2. Audit repair for finite-level etale/pro-etale comparison . . . 667
12.3 A5.3. Local systems and the smooth stalk calculation . . . 669
12.4 A5.4. Constructible devissage and derived smooth base change . . . 672
12.5 A5.5. Compact-support base change at finite level . . . 674
12.6 A5.6. Compactification refinement . . . 676
12.7 A5.7. Constructibility preservation for $Rf_!$ . . . 676
12.8 A5.8. Inherited obligations and coefficient passage . . . 679

13 Trace and arithmetic core 681
13.1 Arithmetic input and Frobenius conventions . . . 681
13.2 Closed-point products and point-counting series . . . 683
13.3 Local terms and the cohomological trace formula . . . 685
13.4 Determinants, trace–log identities, and rationality . . . 691
13.5 Weight route from the supplied PDF . . . 693
13.5.1 Conventions and the foundational boundary . . . 693
13.5.2 W0: Frobenius modules and derived weights . . . 694
13.5.3 BASE ownership and unresolved interface . . . 696
13.5.4 Fourier line . . . 696
13.6 The Artin–Schreier finite etale cover . . . 697
13.7 A conditional Artin–Schreier character summand . . . 701
13.7.1 Objects and arrow candidates . . . 701
13.7.2 Finite algebra . . . 704
13.7.3 Image and direct sum . . . 707

13.7.4 Frobenius arrows . . . 708
13.7.5 Torsor model, rank, and trace . . . 709
13.7.6 Rigidification and addition . . . 712
13.7.7 Pairing and inverse-character duality . . . 714
13.7.8 Conditional closure and exact OPEN boundary . . . 715
13.7.9 The affine H_c^1 estimate . . . 718
13.7.10 Projective and compact-support curve layers . . . 721
13.7.11 Controlled tame relative curves . . . 724
13.7.12 D1–D2: support, normalization, and radicial descent . . . 726
13.7.13 D3–D4: coefficients and target localization . . . 729
13.7.14 D5–D6: composition and relative dimension zero . . . 731
13.7.15 Fixed-degree direct-image assembly . . . 733
13.7.16 Derived upper bound for one bounded constructible input . . . 754
13.7.17 Fixed Poincare–Verdier upper-to-lower argument . . . 756
13.7.18 Strict conditional WPDF atoms . . . 758
13.7.19 Fixed RH endpoint: eigenvalues and reciprocal-root coordinates . . . 761
13.7.20 Zeta and global Weil extensions not entered . . . 762
13.7.21 Forbidden branch . . . 762

14 Trace-core follow-up: fixed points and determinant series 763
14.1 Finite-field Frobenius and fixed loci . . . 763
14.2 One factor: finite matrix algebra and formal series . . . 765
14.3 A4 adic exceptional-transition cluster . . . 766
14.3.1 Finite-level transition datum . . . 766
14.3.2 Tower and Frobenius structure . . . 769
14.3.3 Adic inverse limit . . . 770
14.4 A4 finite exceptional-pullback arrow cluster . . . 770
14.4.1 Arrow construction and formal identities . . . 770
14.4.2 Frobenius and coefficient reduction . . . 772
14.5 Compact-support pushforward from a dense compactification . . . 775
14.5.1 A refinement is the identity over the source open . . . 776
14.5.2 Comparison along a dense refinement . . . 777
14.5.3 Common refinements and compactification independence . . . 785
14.6 Units, composition, and associativity of compact support . . . 787
14.7 Open–closed localization and its compact-support image . . . 788

15 Locally finite Frobenius orbit dictionary 794
15.1 Locally finite periodic input . . . 794
15.2 The unrestricted orbit quotient . . . 795
15.3 Minimal periods and orbit degree . . . 796
15.4 Explicit enumeration of one orbit . . . 797
15.5 Finite orbit sets attached to one iterate . . . 798
15.6 The finite-type scheme instance . . . 800
15.7 Closed points, residue embeddings, and the two explicit maps . . . 802
15.8 Finite-extension points and the fixed-locus equivalence . . . 809
15.9 Scheme Frobenius after finite base-field extension . . . 812
15.10 Scheme-map fixed points and finite-extension points . . . 814
15.11 EFI closed-point finite-field and geometric-point adapters . . . 818
15.12 Exact ZPC interface instances supplied by TraceCore3 . . . 822
15.12.1 The concrete finite-extension point interface for Module 03 . . . 823
15.13 Canonical finite-field towers and the final point realization . . . 828
15.13.1 The canonical fixed-subfield tower . . . 828

15.13.2 Point transitions and fixed-locus transitions . . . 829
15.13.3 The two literal Final equivalences . . . 830
15.13.4 Closed points and finite structures . . . 833
15.13.5 The actual Final candidate and certificate . . . 835
15.13.6 The remaining finite-field presentation boundary . . . 836
15.13.7 External finite-field towers and the precise independence boundary . . . 836

16 Atomic terminal assembly of the Weil conjectures 846
16.1 The already constructed geometric input . . . 846
16.2 Frobenius factors and the trace formula . . . 848
16.3 Integral factors, degrees, and realization independence . . . 850
16.4 Poincare duality and the functional equation . . . 852
16.5 Purity, reciprocal roots, and the Riemann hypothesis . . . 852
16.6 Optional singular comparison and terminal theorem . . . 854
16.7 Typed construction of scheme Frobenius . . . 855
16.7.1 Affine construction and localization coherence . . . 855
16.7.2 Global gluing and the descended model . . . 856
16.7.3 Affine formula, uniqueness, and functoriality . . . 858
16.7.4 Absolute and semilinear factorization . . . 862
16.7.5 Universal-homeomorphism certificates . . . 864
16.7.6 Finite-type finiteness and properness . . . 866

Chapter 1

Global data and normalization

Definition 1.0.1 (Prime-power datum)**.** Uses:

Lean: `Weil.PrimePowerData`

A prime-power datum is a triple (p, a, q) consisting of a prime number p, a positive integer a, and the specified equality $q = p^a$.

Definition 1.0.2 (Finite-field datum)**.** Uses: `def:weil-prime-power-data`

Lean: `Weil.FiniteFieldData`

For a prime-power datum, a finite-field datum consists of a finite field k_0, a specified equality $\#k_0 = q$, a field k with a selected k_0-algebra structure, a proof that k/k_0 is algebraic, and a proof that k is algebraically closed. Thus k is a selected algebraic closure of k_0 with its structural embedding $k_0 \hookrightarrow k$. We write $k_0 = \mathbf{F}_q$.

Definition 1.0.3 (Arithmetic field Frobenius)**.** Uses: `def:weil-prime-power-data,def:weil-finite-field-data`

Lean: `Weil.arithmeticFieldFrobenius`

Define the arithmetic field Frobenius $\varphi_q \in \mathrm{Aut}_{k_0}(k)$ to be `FiniteField.frobeniusAlgEquivOfAlgebraic` k_0 k.

Library status: verified only after the pinned-contract audit.

Lemma 1.0.4 (Arithmetic Frobenius is an automorphism)**.** Uses: `def:weil-arithmetic-field-frobenius`

Lean: `Weil.arithmeticFieldFrobenius_bijective`

The map φ_q is a field automorphism of k fixing k_0 pointwise.

Proof. Uses: `def:weil-arithmetic-field-frobenius`

The defining mathlib object has type $k \simeq_{k_0} k$. Its underlying equivalence is therefore bijective, its underlying map is a field homomorphism, and the algebra-equivalence commutation law says that it fixes the image of every element of k_0. □

Lemma 1.0.5 (Power formula for arithmetic field Frobenius)**.** Uses: `def:weil-prime-power-data,def:weil-finite-field-data,def:weil-arithmetic-field-frobenius`

Lean: `Weil.arithmeticFieldFrobenius_apply`

For every $x \in k$, one has $\varphi_q(x) = x^q$.

Proof. Uses: `def:weil-finite-field-data,def:weil-arithmetic-field-frobenius`

Apply `FiniteField.coe_frobeniusAlgEquivOfAlgebraic` with $K = k_0$ and $L = k$. It gives $\varphi_q(x) = x^{\#k_0}$; rewrite the exponent using the equality $\#k_0 = q$ stored in the finite-field datum. □

Definition 1.0.6 (Geometric field Frobenius)**.** Uses: `def:weil-arithmetic-field-frobenius`

Lean: `Weil.geometricFieldFrobenius`

Define geometric field Frobenius to be $F_k := \varphi_q^{-1} : k \to k$.

Lemma 1.0.7 (Frobenius inverse identities)**.** Uses: `def:weil-arithmetic-field-frobenius,def:weil-geometric-field-frobenius`

Lean: `Weil.geometricFieldFrobenius_inverse`

One has $F_k \circ \varphi_q = \mathrm{id}_k$ and $\varphi_q \circ F_k = \mathrm{id}_k$.

Proof. Uses: `def:weil-geometric-field-frobenius,lemma:weil-arithmetic-frobenius-automorphism`
These are the two inverse equations in the definition of the inverse of the automorphism φ_q. □

Definition 1.0.8 (Coefficient algebraic-closure datum)**.** Uses: `def:weil-prime-power-data`

Lean: `Weil.CoefficientAlgebraicClosureData`

A coefficient algebraic-closure datum consists of a prime $\ell \neq p$, the field $E = \mathbf{Q}_\ell$, a field $\overline{E}$ with a specified embedding $E \hookrightarrow \overline{E}$, a proof that $\overline{E}/E$ is algebraic, and a specified algebraically closed field structure on $\overline{E}$. These are algebraic-closure data only; they contain no map to $\mathbf{C}$.

Definition 1.0.9 (Fixed complex-realization datum)**.** Uses: `def:weil-coefficient-algebraic-closure-data`

Lean: `Weil.FixedComplexRealizationData`

For a coefficient algebraic-closure datum, a fixed complex-realization datum is a specified field embedding

$$\iota_0 : \overline{E} \hookrightarrow \mathbf{C}.$$

It is not required to be surjective. In particular it is not a witness for the global field-isomorphism type quantified over by universal-realization purity.

Definition 1.0.10 (Coefficient datum)**.** Uses: `def:weil-coefficient-algebraic-closure-data,def:weil-fixed-complex-realization-data`

Lean: `Weil.CoefficientData`

A coefficient datum consists of a coefficient algebraic-closure datum and a fixed complex-realization datum for it. The selected $\overline{E}$ is thus an algebraic closure of $E = \mathbf{Q}_\ell$, while the distinguished embedding ι_0 is used only by fixed-realization predicates, where it is also denoted by ι. A theorem which varies the complex realization takes a separate global complex-realization datum introduced later.

Definition 1.0.11 (Finite-dimensional Frobenius module)**.** Uses: `def:weil-coefficient-data`

Lean: `Weil.FrobeniusModule`

A Frobenius module consists of an E-vector space V, a specified finite-dimensionality instance, and a specified E-linear equivalence

$$F_V : V \simeq_E V.$$

We use the same symbol for its forward endomorphism and write F_V^{-1} for the inverse field of this equivalence. Thus neither an inverse nor either inverse equation is chosen later inside a proof.

Definition 1.0.12 (Tate line)**.** Uses: `def:weil-prime-power-data,def:weil-coefficient-data,def:weil-frobenius-module`

Lean: `Weil.tateLine`

Define $E(1)$ on the concrete one-dimensional carrier E with selected basis vector $e_1 = 1$. Its forward Frobenius is

$$F_{E(1)}(x) = q^{-1}x,$$

and the inverse field of the linear equivalence is $x \mapsto qx$. Their two composites are the identity because $q \neq 0$ in E. For $n \in \mathbf{Z}$, define $E(n)$ as the n-fold tensor power of this concrete line when $n \geq 0$ and as the $(-n)$-fold tensor power of its specified dual when $n < 0$.

Definition 1.0.13 (Tate twist)**.** Uses: `def:weil-frobenius-module,def:weil-tate-line`

Lean: `Weil.FrobeniusModule.tateTwist`

For a Frobenius module V, define its n-th Tate twist by $V(n) := V \otimes_E E(n)$ with the tensor-product Frobenius.

Definition 1.0.14 (ι-weight of a scalar)**.** Uses: `def:weil-prime-power-data,def:weil-coefficient-data`

Lean: `Weil.IsIotaPureScalar`

For $m \geq 1$, $\alpha \in \overline{E}^\times$, and $w \in \mathbf{R}$, say that α is ι-pure of weight w relative to q^m if

$$|\iota(\alpha)| = (q^m)^{w/2}.$$

Definition 1.0.15 (Universal arithmetic weight)**.** Uses: `def:weil-prime-power-data`

Lean: `Weil.IsUniversallyPureScalar`

For an algebraic number $\alpha \in \overline{\mathbf{Q}}^\times$, say that α is universally pure of weight w relative to q^m if every field embedding $\sigma : \mathbf{Q}(\alpha) \hookrightarrow \mathbf{C}$ satisfies $|\sigma(\alpha)| = (q^m)^{w/2}$.

Lemma 1.0.16 (Tate twists shift weights)**.** *Uses:* def:weil-iota-scalar-weight, def:weil-tate-twist

Lean: Weil.IsIotaPureScalar.tateTwist

If α is ι-pure of weight w relative to q^m, then $q^{-mn}\alpha$ is ι-pure of weight $w-2n$ relative to q^m.

Proof. Uses: def:weil-iota-scalar-weight

Compute

$$|\iota(q^{-mn}\alpha)| = q^{-mn}|\iota(\alpha)| = q^{-mn}(q^m)^{w/2} = (q^m)^{(w-2n)/2}.$$

This is exactly the defining equality for weight $w-2n$. □

Definition 1.0.17 (Reciprocal characteristic polynomial)**.** Uses: def:weil-frobenius-module

Lean: Weil.FrobeniusModule.reciprocalCharpoly

For a Frobenius module (V, F_V), define

$$P_V(T) := \det(1 - TF_V) \in E[T].$$

An eigenvalue α of F_V is called a reciprocal root of P_V.

Chapter 2

Nagata compactification

This chapter gives a formalization blueprint for Nagata's compactification theorem for schemes. The key result is

1. the terminal theorem below, asserting that every separated finite type morphism $X \to S$ with S quasi-compact and quasi-separated factors as a quasi-compact open immersion followed by a proper morphism.

The proof follows the scheme-theoretic proof in the Stacks Project. The intermediate statements are deliberately finer than the printed proof: every change of ambient scheme, every use of a scheme-theoretic image, every admissible blow-up, and every valuative lifting step is given its own labelled interface. This is the level at which the objects and equalities will have to be supplied to Lean.

Throughout this chapter, "finite type" includes quasi-compactness, and a compactification always has a quasi-compact open immersion as its first map. Fiber products and all morphisms are taken in the category of schemes.

2.1 Compactifications and scheme-theoretic images

Definition 2.1.1 (Compactification)**.** Uses:

Lean: `AlgebraicGeometry.Compactification`

Let $f\colon X \to S$ be a morphism of schemes. A *compactification* of f consists of a scheme $\overline{X}$, morphisms

$$X \xrightarrow{j} \overline{X} \xrightarrow{p} S,$$

and an equality $j;p = f$, such that j is a quasi-compact open immersion and p is proper.

Definition 2.1.2 (Compactifiability)**.** Uses: `def:nagata-compactification`

Lean: `AlgebraicGeometry.Compactifiable`

A morphism $f\colon X \to S$ is *compactifiable* if the type of compactifications of f is nonempty.

Definition 2.1.3 (Scheme-theoretically dense compactification)**.** Uses: `def:nagata-compactification`

Lean: `AlgebraicGeometry.Compactification.IsSchemeTheoreticallyDense`

A compactification $X \xrightarrow{j} \overline{X} \to S$ is *scheme-theoretically dense* if

$$\mathcal{O}_{\overline{X}} \longrightarrow j_*\mathcal{O}_X$$

is injective. Equivalently, the scheme-theoretic image of j is all of $\overline{X}$.

Lemma 2.1.4 (Quasi-coherent kernel for a quasi-compact image)**.** *Uses:*

Lean: *`SchemeHom.schemeTheoreticImage.kernelQuasiCoherent`*

For a quasi-compact morphism $g\colon T \to Y$, the kernel

$$\mathcal{I}_g := \ker(\mathcal{O}_Y \longrightarrow g_*\mathcal{O}_T)$$

is a quasi-coherent ideal sheaf on Y and determines a closed subscheme of Y.

Proof. Uses:

The assertion is local on Y, so assume $Y = \operatorname{Spec} A$ is affine. Since g is quasi-compact, choose a finite affine open cover $T = \bigcup_{r=1}^{n} T_r$ and form the affine coproduct $T' = \coprod_r T_r$. The map $T' \to T \to Y$ gives an injection $g_*\mathcal{O}_T \hookrightarrow g'_*\mathcal{O}_{T'}$ on sections over affine opens, because regular functions on T are exactly compatible families of regular functions on the finite affine cover. Hence

$$\ker(\mathcal{O}_Y \to g_*\mathcal{O}_T) = \ker(\mathcal{O}_Y \to g'_*\mathcal{O}_{T'}).$$

The morphism g' is affine, so $g'_*\mathcal{O}_{T'}$ is quasi-coherent; kernels of morphisms of quasi-coherent modules on an affine scheme are quasi-coherent, and the local constructions agree on overlaps. The closed subscheme cut out by this ideal is the scheme-theoretic image. This is the quasi-compact part of Stacks Project, Lemma 29.6.3 (Tag 01R8). □

Definition 2.1.5 (Scheme-theoretic image)**.** Uses:

Lean: `Scheme.Hom.schemeTheoreticImage`

Constructed by: `lemma:nagata-scheme-image-kernel-quasicoherent`

Let $g\colon T \to Y$ be a quasi-compact morphism. The kernel of $\mathcal{O}_Y \to g_*\mathcal{O}_T$ is the quasi-coherent ideal supplied by the preceding kernel construction. Define $\operatorname{SIm}(g) \hookrightarrow Y$ to be the closed subscheme cut out by the kernel of $\mathcal{O}_Y \to g_*\mathcal{O}_T$.

Definition 2.1.6 (Canonical factor through a scheme-theoretic image)**.** Uses: `def:nagata-scheme-theoretic-image`

Lean: `Scheme.Hom.toSchemeTheoreticImage`

Define

$$g^{\mathrm{im}}\colon T \longrightarrow \operatorname{SIm}(g)$$

to be the morphism induced by the fact that the kernel defining $\operatorname{SIm}(g)$ acts trivially on $g_*\mathcal{O}_T$.

Definition 2.1.7 (Comparison map for a closed factorization)**.** Uses: `def:nagata-scheme-theoretic-image`

Lean: `Scheme.Hom.schemeTheoreticImageComparison`

Suppose g is supplied with a factorization $T \xrightarrow{h} Z \xrightarrow{z} Y$, where z is a closed immersion. Define

$$c_{g,Z}\colon \operatorname{SIm}(g) \longrightarrow Z$$

from the containment of the ideal defining Z in the kernel ideal defining $\operatorname{SIm}(g)$.

Lemma 2.1.8 (Universal property of the scheme-theoretic image)**.** *Uses:* *def:nagata-scheme-image-factor, def:nagata-scheme-image-comparison*

Lean: *Scheme.Hom.schemeTheoreticImage_lift*

The composite of g^{im} with $\operatorname{SIm}(g) \hookrightarrow Y$ is g. For every closed factorization through Z, the constructed map $c_{g,Z}$ is a closed immersion compatible with the maps to Y, and it is the unique compatible morphism $\operatorname{SIm}(g) \to Z$.

Proof. Uses: `lemma:nagata-scheme-image-kernel-quasicoherent`

The factorization equality is the defining equality of g^{im}. For the second assertion, the defining ideal of Z is contained in the kernel defining $\operatorname{SIm}(g)$; the resulting quotient map makes $c_{g,Z}$ a closed immersion. Uniqueness follows because a closed immersion is a monomorphism. □

Lemma 2.1.9 (Restriction of a scheme-theoretic image to an open target)**.** *Uses:* *def:nagata-scheme-theoretic-image*

Lean: *Scheme.Hom.schemeTheoreticImage_restrict*

Let $W \subseteq Y$ be open, let $T_W = T \times_Y W$, and let $g_W\colon T_W \to W$ be the base change of g. Then

$$\operatorname{SIm}(g_W) = \operatorname{SIm}(g) \times_Y W$$

as closed subschemes of W.

Proof. Uses: `lemma:nagata-scheme-image-kernel-quasicoherent`

Restriction to an open commutes with the map $\mathcal{O}_Y \to g_*\mathcal{O}_T$ and with its kernel. Hence both closed subschemes are defined on W by the restriction of the same quasi-coherent ideal. □

Lemma 2.1.10 ([1, Tag 01RG])**.** *Uses:* *def:nagata-scheme-image-factor*

Lean: *Scheme.Hom.isOpenImmersion_toSchemeTheoreticImage*

Let $g\colon T \to Y$ be a quasi-compact immersion. The constructed map $g^{\mathrm{im}}\colon T \to \operatorname{SIm}(g)$ is an open immersion, and its image is topologically and scheme-theoretically dense.

Proof. Uses: lemma:nagata-scheme-image-kernel-quasicoherent,lemma:nagata-scheme-image-restrict-open

The assertion is local on Y. Factor the quasi-compact immersion as an open immersion followed by a closed immersion after replacing Y by the closed subscheme defined by the kernel ideal; this is the construction of Stacks Project, Lemma 29.3.2 (Tag 01QV). On an affine open where the immersion is closed in an open subscheme, the kernel ideal is exactly the closed-immersion ideal on that open, so the induced map to $\mathrm{SIm}(g)$ identifies T with an open subscheme. The open-restriction lemma shows that every open of $\mathrm{SIm}(g)$ meets this image scheme-theoretically through the zero kernel, hence the image is scheme-theoretically dense. Topological density follows from the quasi-compact image theorem, Stacks Project, Lemma 29.6.3 (Tag 01R8). This is the quasi-compact case of Stacks Project, Lemma 29.7.7 (Tag 01RG). □

Theorem 2.1.11 (Scheme-theoretic image package)**.** Uses: def:nagata-scheme-theoretic-image

Lean: Scheme.Hom.schemeTheoreticImagePackage

The scheme-theoretic image construction for quasi-compact morphisms supplies the closed subscheme, canonical factor map, universal closed-factor comparison, open-restriction compatibility, and the open dense factorization for quasi-compact immersions used in the Nagata construction.

Proof. Uses: lemma:nagata-scheme-image-kernel-quasicoherent,lemma:nagata-scheme-image-universal,lemma:nagata-scheme-image-restrict-open,lemma:nagata-immersion-open-in-scheme-image

The kernel-quasi-coherence lemma constructs the closed subscheme. The definition of g^{im} gives the canonical factor map through that closed subscheme. The universal lemma gives the comparison map to any closed factorization and proves its uniqueness. The restriction lemma identifies the construction after base change to an open of the target. Finally, the immersion lemma identifies the factor of a quasi-compact immersion with a scheme-theoretically dense open subscheme of its scheme-theoretic image. These are exactly the fields required of the scheme-theoretic-image package. □

Definition 2.1.12 (Compactification constructed from an immersion)**.** Uses: def:nagata-compactification,def:nagata-scheme-image-factor

Lean: AlgebraicGeometry.Compactification.ofImmersionConstruction

Let $g\colon X \to Y$ be a quasi-compact immersion over S, with $Y \to S$ proper. Define $\mathsf{C}(g)$ to be the factorization

$$X \xrightarrow{g^{\mathrm{im}}} \mathrm{SIm}(g) \hookrightarrow Y \longrightarrow S.$$

Lemma 2.1.13 (The immersion construction is a compactification)**.** Uses: def:nagata-immersion-compactification

Lean: AlgebraicGeometry.Compactification.ofImmersion

The constructed factorization $\mathsf{C}(g)$ is a compactification of $X \to S$.

Proof. Uses: lemma:nagata-immersion-open-in-scheme-image

The first map is a quasi-compact open immersion by Lemma 2.1.10. The second map is the composite of a closed immersion with the proper map $Y \to S$, hence is proper. □

Definition 2.1.14 (Base-change construction for a compactification)**.** Uses: def:nagata-compactification

Lean: AlgebraicGeometry.Compactification.baseChangeConstruction

Given a compactification $C = (X \to \overline{X} \to S)$ and $S' \to S$, define $C_{S'}$ to be

$$X \times_S S' \longrightarrow \overline{X} \times_S S' \longrightarrow S'.$$

Lemma 2.1.15 (Base change of a compactification)**.** Uses: def:nagata-compactification-base-change

Lean: AlgebraicGeometry.Compactification.baseChange

The constructed factorization $C_{S'}$ is a compactification of $X \times_S S' \to S'$.

Proof. Uses:

Open immersions, quasi-compact morphisms, and proper morphisms are stable under arbitrary base change. The factorization equality follows from the universal property of the two fibre products. □

Definition 2.1.16 (Dense-refinement construction)**.** Uses: `def:nagata-compactification`,`def:nagata-scheme-image-factor`

Lean: `AlgebraicGeometry.Compactification.denseRefinementConstruction`

For a compactification $C = (X \xrightarrow{j} \overline{X} \to S)$, define $\mathsf{Dense}(C)$ by replacing $\overline{X}$ with $\mathrm{SIm}(j)$ and using j^{im} as its first map.

Proposition 2.1.17 (Properties of the dense-refinement construction)**.** *Uses:* `def:nagata-dense-compactification`, `def:nagata-dense-refinement`

Lean: `AlgebraicGeometry.Compactification.denseRefinement`

The object $\mathsf{Dense}(C)$ *is a scheme-theoretically dense compactification of the same morphism as* C*.*

Proof. Uses: `lemma:nagata-immersion-open-in-scheme-image`

The induced map from X is a quasi-compact open immersion and is scheme-theoretically dense by Lemma 2.1.10. The new ambient scheme is closed in $\overline{X}$, so it remains proper over S. □

Definition 2.1.18 (Closed-subscheme compactification construction)**.** Uses: `def:nagata-compactification`,`def:nagata-scheme-theoretic-image`,`def:nagata-scheme-image-factor`

Lean: `AlgebraicGeometry.Compactification.closedSubschemeConstruction`

Let $i\colon X \hookrightarrow X'$ be closed over S and fix a compactification $C' = (X' \to \overline{X}' \to S)$. Define $\mathsf{Closed}(i, C')$ by taking the scheme-theoretic image $\overline{X} = \mathrm{SIm}(X \to \overline{X}')$ and the factorization

$$X \longrightarrow \overline{X} \hookrightarrow \overline{X}' \longrightarrow S.$$

Proposition 2.1.19 (The closed-subscheme construction is a compactification)**.** *Uses:* `def:nagata-closed-subscheme-compactification`

Lean: `AlgebraicGeometry.CompactifiableOfClosedImmersion`

The constructed factorization $\mathsf{Closed}(i, C')$ *is a compactification of* $X \to S$*.*

Proof. Uses: `lemma:nagata-scheme-image-restrict-open`,`lemma:nagata-immersion-open-in-scheme-image`

Its ambient scheme $\overline{X}$ is closed in the proper scheme $\overline{X}'$, hence is proper over S. By Lemma 2.1.9, its pullback to the open $X' \subseteq \overline{X}'$ is the scheme-theoretic image of $X \hookrightarrow X'$, namely X. Therefore

$$X = \overline{X} \times_{\overline{X}'} X',$$

so $X \to \overline{X}$ is the base change of the quasi-compact open immersion $X' \to \overline{X}'$. □

2.2 Separated gluing

Lemma 2.2.1 (The diagonal on a mixed open)**.** *Uses:*

Lean: `Scheme.diagonal_pullback_openPair`

Let $X = U \cup V$ *be an open cover over* S*. The pullback of* $\Delta_{X/S}$ *to* $U \times_S V$ *is canonically the map*

$$U \cap V \longrightarrow U \times_S V.$$

The analogous assertion holds over $V \times_S U$*.*

Proof. Uses:

Both schemes represent triples consisting of a point of U, a point of V, and an equality of their images in X. The canonical isomorphism is obtained from the universal property of the pullback. □

Lemma 2.2.2 ([1, Tag 0F3U])**.** *Uses:*

Lean: `Scheme.IsSeparated.of_two_open_cover`

Let $X = U \cup V$ *be an open cover over* S*. Assume that* $U \to S$ *and* $V \to S$ *are separated and that*

$$U \cap V \longrightarrow U \times_S V$$

is a closed immersion. Then $X \to S$ *is separated.*

Proof. Uses: `lemma:nagata-diagonal-mixed-open`

The target of $\Delta_{X/S}$ is covered by

$$U \times_S U, \quad U \times_S V, \quad V \times_S U, \quad V \times_S V.$$

On the two unmixed pieces the diagonal is closed by separatedness of U/S and V/S. On the mixed pieces use Lemma 2.2.1. Closed immersions are local on the target, so $\Delta_{X/S}$ is closed. □

2.3 Construction of blow-ups from Rees algebras

This section expands the blow-up package down to the algebraic and gluing lemmas needed by Lean. The later Nagata argument only uses the public geometric interfaces, but those interfaces should not be marked complete until the constructions in this section are available.

2.3.1 The graded Rees algebra

Definition 2.3.1 (Rees algebra). Uses:

Lean: `ReesAlgebra`

Let A be a ring and $I \subseteq A$ an ideal. The Rees algebra is

$$\mathcal{R}_A(I) = \bigoplus_{n \geq 0} I^n t^n \subseteq A[t].$$

Its underlying ring is the subring of $A[t]$ consisting of polynomials whose coefficient of t^n belongs to I^n.

Definition 2.3.2 (Homogeneous piece of the Rees algebra). Uses: `def:nagata-rees-algebra`

Lean: `ReesAlgebra.piece`

For $n \in \mathbb{N}$, let $\mathcal{R}_A(I)_n$ be the A-submodule of Rees elements whose underlying polynomial is supported in degree n.

Definition 2.3.3 (Coefficient witness in one Rees piece). Uses: `def:nagata-rees-piece`

Lean: `ReesAlgebra.pieceCoefficientWitness`

For $F \in \mathcal{R}_A(I)_n$, define $a_n(F) \in I^n$ to be the degree-n coefficient of the underlying polynomial, equipped with the ideal-membership proof stored by the Rees algebra subtype.

Lemma 2.3.4 (The selected Rees coefficient belongs to the ideal power). *Uses:* `def:nagata-rees-piece`, `def:nagata-rees-piece-coefficient-witness`

Lean: `ReesAlgebra.pieceCoefficientWitness_mem`

For $F \in \mathcal{R}_A(I)_n$, the coefficient $a_n(F)$, regarded in A, belongs to I^n.

Proof. Uses: `def:nagata-rees-piece-coefficient-witness`

Unfold $a_n(F)$. Its second subtype projection is a proof of the displayed membership. □

Lemma 2.3.5 (Underlying polynomial of the selected Rees coefficient). *Uses:* `def:nagata-rees-piece`, `def:nagata-rees-piece-coefficient-witness`

Lean: `ReesAlgebra.pieceCoefficientWitness_polynomial`

For $F \in \mathcal{R}_A(I)_n$, the underlying polynomial of F equals $a_n(F)t^n$.

Proof. Uses: `def:nagata-rees-piece`, `def:nagata-rees-piece-coefficient-witness`

Apply polynomial coefficient extensionality. In degree n, the equality is the first projection of the definition of $a_n(F)$. In every degree $r \neq n$, membership of F in the degree-n piece makes the left coefficient zero, and the monomial coefficient formula makes the right coefficient zero. □

Lemma 2.3.6 (A monomial with coefficient in an ideal power lies in the Rees piece). *Uses:* `def:nagata-rees-piece`

Lean: `ReesAlgebra.monomial_mem_piece`

If $a \in I^n$, then $at^n \in \mathcal{R}_A(I)_n$.

Proof. Uses: `def:nagata-rees-piece`

The coefficient of at^n in degree n is $a \in I^n$; every other coefficient is zero and hence belongs to the corresponding ideal power. Its polynomial support is therefore contained in the singleton $\{n\}$, which is the defining degree-n support condition. □

Lemma 2.3.7 (Membership in one Rees piece). *Uses:* `def:nagata-rees-piece`

Lean: `ReesAlgebra.mem_piece_iff`

Constructed by: `def:nagata-rees-piece-coefficient-witness`

An element $F \in \mathcal{R}_A(I)$ lies in $\mathcal{R}_A(I)_n$ if and only if there is $a \in I^n$ such that the underlying polynomial of F is at^n.

Proof. Uses: lemma:nagata-rees-piece-coefficient-witness-mem,lemma:nagata-rees-piece-coefficient-witness-polynomial,lemma:nagata-rees-monomial-mem-piece
For the forward implication, regard the membership proof as an element of the degree-n piece and take the coefficient selected in Lemma 2.3.4. The required polynomial equality is Lemma 2.3.5. For the reverse implication, instantiate Lemma 2.3.6 with the displayed coefficient. □

Lemma 2.3.8 (Multiplication of Rees pieces)**.** *Uses:*

Lean: `ReesAlgebra.piece_mul_piece`
Multiplication maps

$$\mathcal{R}_A(I)_m \cdot \mathcal{R}_A(I)_n \subseteq \mathcal{R}_A(I)_{m+n}.$$

Proof. Uses: lemma:nagata-rees-piece-membership
Write the two elements as at^m and bt^n with $a \in I^m$ and $b \in I^n$. Their product is $(ab)t^{m+n}$ and $ab \in I^m I^n = I^{m+n}$. □

Lemma 2.3.9 (Finite support of the homogeneous decomposition)**.** *Uses:* `def:nagata-rees-piece`

Lean: `ReesAlgebra.componentFiniteSupport`
For $F \in \mathcal{R}_A(I)$, only finitely many homogeneous components $F_n t^n$ are nonzero.

Proof. Uses:
The components are indexed by the support of the underlying polynomial, which is finite. □

Lemma 2.3.10 (Reconstruction from homogeneous components)**.** *Uses:*

Lean: `ReesAlgebra.sum_components`
Every $F \in \mathcal{R}_A(I)$ is the finite sum of the monomials $F_n t^n$ regarded as elements of $\mathcal{R}_A(I)_n$.

Proof. Uses: lemma:nagata-rees-component-finite-support,lemma:nagata-rees-piece-membership
Sum over the finite polynomial support. Equality follows by comparing the coefficient in every degree. □

Lemma 2.3.11 (Uniqueness of the homogeneous decomposition)**.** *Uses:*

Lean: `ReesAlgebra.components_unique`
If a finite sum of elements $F_n \in \mathcal{R}_A(I)_n$ is zero, then every F_n is zero.

Proof. Uses: lemma:nagata-rees-piece-membership
Apply the coefficient map in degree n. Every summand of degree different from n has zero nth coefficient. □

Proposition 2.3.12 (Graded-ring structure on the Rees algebra)**.** *Uses:*

Lean: `ReesAlgebra.instGradedRingPiece`
The family $n \mapsto \mathcal{R}_A(I)_n$ defines an $\mathbb{N}$-graded-ring structure on $\mathcal{R}_A(I)$.

Proof. Uses: lemma:nagata-rees-piece-mul,lemma:nagata-rees-sum-components,lemma:nagata-rees-components-unique
Use Lemma 2.3.8 for graded multiplication, Lemma 2.3.10 for existence of the direct-sum decomposition, and Lemma 2.3.11 for uniqueness. □

Lemma 2.3.13 (The degree-zero piece)**.** *Uses:*

Lean: `ReesAlgebra.pieceZeroEquiv`
Taking the constant coefficient gives a ring equivalence

$$\mathcal{R}_A(I)_0 \cong A.$$

Proof. Uses: lemma:nagata-rees-piece-membership
Since $I^0 = A$, a degree-zero Rees element is exactly a constant polynomial. The inverse sends a to the constant monomial at^0. The two composites are identities by coefficient extensionality. □

Lemma 2.3.14 (Degree-one element associated to an ideal element)**.** *Uses:*

Lean: `ReesAlgebra.degreeOneElement`
Every $a \in I$ determines a homogeneous element at of degree one. Its underlying polynomial and its coefficient in degree one are recorded by simp lemmas.

Proof. Uses: `lemma:nagata-rees-piece-membership`

Since $a \in I = I^1$, the monomial at belongs to the degree-one piece by Lemma 2.3.7. The underlying-polynomial and coefficient formulas follow from the corresponding formulas for a polynomial monomial. □

Proposition 2.3.15 (Finite generation over degree zero)**.** *Uses:*

Lean: `ReesAlgebra.finiteType_pieceZero`

If $I = (f_0, \dots, f_r)$, then $\mathcal{R}_A(I)$ is generated as an algebra over its degree-zero piece by $f_0 t, \dots, f_r t$. In particular it is of finite type over $\mathcal{R}_A(I)_0$.

Proof. Uses: `lemma:nagata-rees-degree-zero,lemma:nagata-rees-degree-one-element,lemma:nagata-rees-sum-components`

A homogeneous coefficient in I^n is a linear combination of degree-n monomials in the f_i. Hence every homogeneous Rees element belongs to the algebra generated by the $f_i t$, and Lemma 2.3.10 treats an arbitrary element. Transport finite generation along Lemma 2.3.13. □

2.3.2 Affine blow-ups and their standard charts

Definition 2.3.16 (Affine blow-up)**.** Uses: `def:nagata-rees-algebra`

Lean: `AlgebraicGeometry.affineBlowup`

For an ideal $I \subseteq A$, define

$$\mathrm{Bl}_I(\operatorname{Spec} A) = \operatorname{Proj}(\mathcal{R}_A(I)).$$

Its structure morphism is induced by the canonical degree-zero map $A \to \mathcal{R}_A(I)_0$.

Definition 2.3.17 (Affine blow-up chart algebra)**.** Uses:

Lean: `ReesAlgebra.blowupChartRing`

Let $a \in I$. Define $A[I/a]$ to be the A-subalgebra of A_a generated by the fractions x/a with $x \in I$.

Lemma 2.3.18 (Normal form in a chart algebra)**.** *Uses:* `def:nagata-blowup-chart-ring`

Lean: `ReesAlgebra.mem_blowupChartRing_iff`

Every element of $A[I/a]$ is represented in A_a by x/a^n for some $n \geq 0$ and $x \in I^n$.

Proof. Uses:

The generators have the required form with $n = 1$. The set of such fractions contains the image of A, and is closed under addition and multiplication after replacing two denominators by a common power of a. It therefore contains the generated subalgebra. Conversely, $x \in I^n$ is a sum of products of n elements of I, so x/a^n belongs to the generated algebra. □

Lemma 2.3.19 (The degree-zero chart fraction belongs to the chart ring)**.** *Uses:* `def:nagata-blowup-chart-ring`

Lean: `ReesAlgebra.chartFraction_mem_zero`

Let C be a commutative ring, let $K \subseteq C$ be an ideal, let $c \in K$, and let $x \in K^0$. Then x/c^0 belongs to $C[K/c]$.

Proof. Uses: `def:nagata-blowup-chart-ring`

Rewrite $K^0 = \top$ and $c^0 = 1$. The fraction $x/1$ is the image of x under $C \to C_c$. Apply the exact mathlib leaf `Subalgebra.algebraMap_mem` to the subalgebra $C[K/c]$. □

Lemma 2.3.20 (The product branch for a successor chart fraction)**.** *Uses:* `def:nagata-blowup-chart-ring`

Lean: `ReesAlgebra.chartFraction_mem_succ_product`

Let $y \in K$, let $z \in K^n$, and suppose $z/c^n \in C[K/c]$. Then

$$\frac{yz}{c^{n+1}} \in C[K/c].$$

Proof. Uses: `def:nagata-blowup-chart-ring`

By the defining generator inclusion for $C[K/c]$, one has $y/c \in C[K/c]$. Apply the exact mathlib leaf `Subalgebra.mul_mem` to y/c and z/c^n. The localization-ring calculation

$$(y/c)(z/c^n) = yz/c^{n+1}$$

identifies its conclusion with the displayed target. □

Lemma 2.3.21 (The additive branch for a fixed-denominator chart fraction). *Uses:* def:nagata-blowup-chart-ring

Lean: ReesAlgebra.chartFraction_mem_add

If u/c^n and v/c^n belong to $C[K/c]$, then $(u+v)/c^n$ belongs to $C[K/c]$.

Proof. Uses: def:nagata-blowup-chart-ring

Apply the exact mathlib leaf `Subalgebra.add_mem`. Addition in C_c gives $u/c^n + v/c^n = (u+v)/c^n$. □

Lemma 2.3.22 (Membership of a power fraction in a chart ring). *Uses:* def:nagata-blowup-chart-ring

Lean: ReesAlgebra.chartFraction_mem

Let C be a commutative ring, let $K \subseteq C$ be an ideal, let $c \in K$, and let $n \in \mathbf{N}$. If $x \in K^n$, then the localization fraction $x/c^n \in C_c$ belongs to the subalgebra $C[K/c]$.

Proof. Uses: lemma:nagata-chart-ring-fraction-mem-zero, lemma:nagata-chart-ring-fraction-mem-succ-product, lemma:nagata-chart-ring-fraction-mem-add

Induct on n. The zero case is Lemma 2.3.19. In the successor case, rewrite the membership hypothesis with the exact mathlib identity `Ideal.IsTwoSided.pow_succ`,

$$K^{n+1} = K\,K^n.$$

Apply `Submodule.mul_induction_on` with motive $P(w) :\Longleftrightarrow w/c^{n+1} \in C[K/c]$. Its product branch is Lemma 2.3.20, using the induction hypothesis for the K^n factor. Its addition branch is Lemma 2.3.21. These are precisely the two fields required by `Submodule.mul_induction_on`; no unrecorded finite-sum decomposition is selected. □

Lemma 2.3.23 (Equality criterion for chart fractions over an arbitrary base ring). *Uses:* def:nagata-blowup-chart-ring

Lean: ReesAlgebra.chartFraction_eq_iff

Let C be a commutative ring, let $K \subseteq C$ be an ideal, let $c \in K$, and let $n, m \in \mathbf{N}$. For $x \in K^n$ and $y \in K^m$, the following are equivalent:

$$\frac{x}{c^n} = \frac{y}{c^m} \quad \text{in } C[K/c], \qquad \exists r \in \mathbf{N}, \quad c^{r+m}x = c^{r+n}y.$$

All rings, ideals, elements, exponents, and ideal-power membership proofs in this statement are explicit parameters. In particular the statement may later be instantiated with $(C, K, c) = (B, IB, b)$.

Proof. Uses: def:nagata-blowup-chart-ring

Equality in the subring $C[K/c] \subseteq C_c$ is equality in C_c. The localization equality criterion for the denominators c^n and c^m says exactly that there is an r such that

$$c^r(c^m x - c^n y) = 0.$$

Distribute c^r and use $c^r c^m = c^{r+m}$ and $c^r c^n = c^{r+n}$. This gives the displayed coefficient equality. The same calculation in reverse supplies the localization-equality witness. No cancellation by c is used. □

Definition 2.3.24 (Representatives for the degree-zero homogeneous chart). Uses: def:nagata-rees-algebra

Lean: ReesAlgebra.HomogeneousAwayZero.Representative

For $a \in I$, let HRep_a be the type of pairs (n, x) with $n \in \mathbf{N}$ and $x \in I^n$. The pair (n, x) denotes the raw homogeneous fraction $(xt^n)/(at)^n$. Define a binary relation ρ_a on representatives by

$$(n, x)\ \rho_a\ (m, y) \quad \Longleftrightarrow \quad \exists r \in \mathbf{N}, \quad a^{r+m}x = a^{r+n}y.$$

At this stage ρ_a is only a binary relation.

Lemma 2.3.25 (Reflexivity of the homogeneous-chart representative relation). *Uses:* def:nagata-homogeneous-away-chart-representatives

Lean: ReesAlgebra.HomogeneousAwayZero.relation_refl

For every $u \in \mathrm{HRep}_a$, one has $u\ \rho_a\ u$.

Proof. Uses: def:nagata-homogeneous-away-chart-representatives

Write $u = (n, x)$ and take $r = 0$. The required equality is $a^n x = a^n x$. □

Lemma 2.3.26 (Symmetry of the homogeneous-chart representative relation)**.** *Uses:* `def:nagata-homogeneous-away-chart-representatives`

Lean: `ReesAlgebra.HomogeneousAwayZero.relation_symm`

If $u\ \rho_a\ v$, then $v\ \rho_a\ u$.

Proof. Uses: `def:nagata-homogeneous-away-chart-representatives`

Write $u = (n, x)$ and $v = (m, y)$. A witness r gives $a^{r+m}x = a^{r+n}y$; its symmetric equality, with the same r, is the defining relation from v to u. □

Lemma 2.3.27 (Transitivity of the homogeneous-chart representative relation)**.** *Uses:* `def:nagata-homogeneous-away-chart-representatives`

Lean: `ReesAlgebra.HomogeneousAwayZero.relation_trans`

If $u\ \rho_a\ v$ and $v\ \rho_a\ w$, then $u\ \rho_a\ w$.

Proof. Uses: `def:nagata-homogeneous-away-chart-representatives`

Write $u = (n, x)$, $v = (m, y)$, and $w = (k, z)$. Let r and s satisfy

$$a^{r+m}x = a^{r+n}y, \qquad a^{s+k}y = a^{s+m}z.$$

Multiply the first equality by a^{s+k} and the second by a^{r+n}. Their middle terms agree, hence

$$a^{r+s+m+k}x = a^{r+s+m+n}z.$$

This is $u\ \rho_a\ w$ with witness $r + s + m$. □

Definition 2.3.28 (Setoid of homogeneous-chart representatives)**.** Uses: `def:nagata-homogeneous-away-chart-representatives`

Lean: `ReesAlgebra.HomogeneousAwayZero.representativeSetoid`

Constructed by: `lemma:nagata-homogeneous-away-chart-relation-reflexive`, `lemma:nagata-homogeneous-away-chart-relation-symmetric`, `lemma:nagata-homogeneous-away-chart-relation-transitive`

Define $\mathsf{HSetoid}_a$ to have carrier relation ρ_a and equivalence fields supplied by the preceding three lemmas.

Definition 2.3.29 (Quotient type of homogeneous-chart representatives)**.** Uses: `def:nagata-homogeneous-away-chart-representative-setoid`

Lean: `ReesAlgebra.HomogeneousAwayZero.Quotient`

Define the bare quotient type

$$\mathrm{HChart}_a = \mathrm{Quotient}(\mathsf{HSetoid}_a).$$

This Definition supplies no constants and no algebraic operations.

Definition 2.3.30 (Raw zero homogeneous-chart representative)**.** Uses: `def:nagata-homogeneous-away-chart-representatives`

Lean: `ReesAlgebra.HomogeneousAwayZero.rawZero`

Define $0_{\mathrm{raw}} = (0, 0) \in \mathrm{HRep}_a$. The coefficient belongs to I^0 because every ideal contains zero.

Definition 2.3.31 (Raw one homogeneous-chart representative)**.** Uses: `def:nagata-homogeneous-away-chart-representatives`

Lean: `ReesAlgebra.HomogeneousAwayZero.rawOne`

Define $1_{\mathrm{raw}} = (0, 1) \in \mathrm{HRep}_a$, using $I^0 = \top$.

Definition 2.3.32 (Raw negation of homogeneous-chart representatives)**.** Uses: `def:nagata-homogeneous-away-chart-representatives`

Lean: `ReesAlgebra.HomogeneousAwayZero.rawNeg`

Define

$$\mathrm{neg}_{\mathrm{raw}}(n, x) = (n, -x).$$

The output membership proof is closure of I^n under negation.

Definition 2.3.33 (Raw addition of homogeneous-chart representatives)**.** Uses: `def:nagata-homogeneous-away-chart-representatives`

Lean: `ReesAlgebra.HomogeneousAwayZero.rawAdd`

For $u = (n, x)$ and $v = (m, y)$, define

$$\mathrm{add}_{\mathrm{raw}}(u, v) = \big(n + m,\ a^m x + a^n y\big).$$

Since $a^m \in I^m$, one has $a^m x \in I^{m+n}$. Independently, $a^n \in I^n$ and $y \in I^m$ give $a^n y \in I^{n+m}$. Transport the first membership across $m + n = n + m$, then add inside I^{n+m} to obtain the output subtype field.

Definition 2.3.34 (Raw multiplication of homogeneous-chart representatives)**.** Uses: `def:nagata-homogeneous-away-chart-representatives`

Lean: `ReesAlgebra.HomogeneousAwayZero.rawMul`

For $u = (n, x)$ and $v = (m, y)$, define

$$\mathrm{mul}_{\mathrm{raw}}(u, v) = (n + m, xy).$$

The output membership is the ideal-power product inclusion $I^n I^m \subseteq I^{n+m}$.

Lemma 2.3.35 (Raw negation respects the representative relation)**.** Uses: `def:nagata-homogeneous-away-chart-raw-neg`

Lean: `ReesAlgebra.HomogeneousAwayZero.rawNeg_respects`

If $u \ \rho_a \ v$, then $\mathrm{neg}_{\mathrm{raw}}(u) \ \rho_a \ \mathrm{neg}_{\mathrm{raw}}(v)$.

Proof. Uses: `def:nagata-homogeneous-away-chart-representatives,def:nagata-homogeneous-away-chart-raw-neg`

Write $u = (n, x)$ and $v = (m, y)$ and retain a witness r. Negating $a^{r+m}x = a^{r+n}y$ gives $a^{r+m}(-x) = a^{r+n}(-y)$. □

Lemma 2.3.36 (Raw addition respects the representative relation)**.** Uses: `def:nagata-homogeneous-away-chart-raw-add`

Lean: `ReesAlgebra.HomogeneousAwayZero.rawAdd_respects`

If $u \ \rho_a \ u'$ and $v \ \rho_a \ v'$, then

$$\mathrm{add}_{\mathrm{raw}}(u, v) \ \rho_a \ \mathrm{add}_{\mathrm{raw}}(u', v').$$

Proof. Uses: `def:nagata-homogeneous-away-chart-representatives,def:nagata-homogeneous-away-chart-raw-add`

Write $u = (n, x)$, $u' = (n', x')$, $v = (m, y)$, and $v' = (m', y')$. Let r, s be the witnesses in the two relation hypotheses, so that

$$a^{r+n'}x = a^{r+n}x', \qquad a^{s+m'}y = a^{s+m}y'.$$

Use $r + s$ as the witness for the two raw sums. After distributing, its left side is

$$a^{r+s+n'+m'+m}x + a^{r+s+n'+m'+n}y,$$

and its right side is

$$a^{r+s+n+m+m'}x' + a^{r+s+n+m+n'}y'.$$

The first pair of summands is equal after multiplying the first witness by $a^{s+m+m'}$; the second pair is equal after multiplying the second witness by $a^{r+n+n'}$. Add those two equalities. □

Lemma 2.3.37 (Raw multiplication respects the representative relation)**.** Uses: `def:nagata-homogeneous-away-chart-raw-mul`

Lean: `ReesAlgebra.HomogeneousAwayZero.rawMul_respects`

If $u \ \rho_a \ u'$ and $v \ \rho_a \ v'$, then

$$\mathrm{mul}_{\mathrm{raw}}(u, v) \ \rho_a \ \mathrm{mul}_{\mathrm{raw}}(u', v').$$

Proof. Uses: `def:nagata-homogeneous-away-chart-representatives,def:nagata-homogeneous-away-chart-raw-mul`

With the notation and witnesses r, s from the preceding proof, use $r + s$. Multiplying the two witness equalities gives

$$a^{r+s+n'+m'}xy = a^{r+s+n+m}x'y',$$

which is precisely the required relation between the two product representatives. □

Definition 2.3.38 (Zero operation on the homogeneous-chart quotient)**.** Uses: `def:nagata-homogeneous-away-chart-quotient-type,def:nagata-homogeneous-away-chart-raw-zero`

Lean: `ReesAlgebra.HomogeneousAwayZero.quotientZero`

Define $\mathsf{zero}_H = [0_{\mathrm{raw}}] \in \mathrm{HChart}_a$.

Definition 2.3.39 (One operation on the homogeneous-chart quotient)**.** Uses: `def:nagata-homogeneous-away-chart-quotient-type,def:nagata-homogeneous-away-chart-raw-one`

Lean: `ReesAlgebra.HomogeneousAwayZero.quotientOne`

Define $\mathsf{one}_H = [1_{\mathrm{raw}}] \in \mathrm{HChart}_a$.

Definition 2.3.40 (Negation operation on the homogeneous-chart quotient)**.** Uses: `def:nagata-homogeneous-away-chart-quotient-type`,`def:nagata-homogeneous-away-chart-raw-neg`

Lean: `ReesAlgebra.HomogeneousAwayZero.quotientNeg`

Constructed by: `lemma:nagata-homogeneous-away-chart-raw-neg-respects`

Define $\mathsf{neg}_H\colon \mathrm{HChart}_a \to \mathrm{HChart}_a$ by quotient lift of $\mathrm{neg}_{\mathrm{raw}}$.

Definition 2.3.41 (Addition operation on the homogeneous-chart quotient)**.** Uses: `def:nagata-homogeneous-away-chart-quotient-type`,`def:nagata-homogeneous-away-chart-raw-add`

Lean: `ReesAlgebra.HomogeneousAwayZero.quotientAdd`

Constructed by: `lemma:nagata-homogeneous-away-chart-raw-add-respects`

Define $\mathsf{add}_H\colon \mathrm{HChart}_a \to \mathrm{HChart}_a \to \mathrm{HChart}_a$ by the binary quotient lift of $\mathrm{add}_{\mathrm{raw}}$.

Definition 2.3.42 (Multiplication operation on the homogeneous-chart quotient)**.** Uses: `def:nagata-homogeneous-away-chart-quotient-type`,`def:nagata-homogeneous-away-chart-raw-mul`

Lean: `ReesAlgebra.HomogeneousAwayZero.quotientMul`

Constructed by: `lemma:nagata-homogeneous-away-chart-raw-mul-respects`

Define $\mathsf{mul}_H\colon \mathrm{HChart}_a \to \mathrm{HChart}_a \to \mathrm{HChart}_a$ by the binary quotient lift of $\mathrm{mul}_{\mathrm{raw}}$.

In the next ring-law statements only, write $0_H = \mathsf{zero}_H$, $1_H = \mathsf{one}_H$, $-_H = \mathsf{neg}_H$, $\boxplus = \mathsf{add}_H$, and $\boxtimes = \mathsf{mul}_H$. These symbols refer to the preceding explicit functions, not to a ring instance.

Lemma 2.3.43 (Associativity of quotient addition)**.** *Uses:* *`def:nagata-homogeneous-away-chart-quotient-add-op`*

Lean: *`ReesAlgebra.HomogeneousAwayZero.quotientAdd_assoc`*

For $u, v, w \in \mathrm{HChart}_a$, $(u \boxplus v) \boxplus w = u \boxplus (v \boxplus w)$.

Proof. Uses: `def:nagata-homogeneous-away-chart-raw-add`,`def:nagata-homogeneous-away-chart-quotient-add-op`

Apply quotient induction to representatives (n, x), (m, y), and (k, z). Both sides have degree $n + m + k$ and their numerators reduce, by two distributivity steps and associativity, to $a^{m+k}x + a^{n+k}y + a^{n+m}z$. Hence the raw representatives are equal, so their quotient classes are equal. □

Lemma 2.3.44 (Left zero law for quotient addition)**.** *Uses:* *`def:nagata-homogeneous-away-chart-quotient-zero-op`*, *`def:nagata-homogeneous-away-chart-quotient-add-op`*

Lean: *`ReesAlgebra.HomogeneousAwayZero.quotientZero_add`*

For $u \in \mathrm{HChart}_a$, $0_H \boxplus u = u$.

Proof. Uses: `def:nagata-homogeneous-away-chart-raw-zero`,`def:nagata-homogeneous-away-chart-raw-add`,`def:nagata-homogeneous-away-chart-quotient-zero-op`,`def:nagata-homogeneous-away-chart-quotient-add-op`

Quotient induction reduces to $u = [(n, x)]$. The left representative is $(n, a^n 0 + a^0 x) = (n, x)$. □

Lemma 2.3.45 (Right zero law for quotient addition)**.** *Uses:* *`def:nagata-homogeneous-away-chart-quotient-zero-op`*, *`def:nagata-homogeneous-away-chart-quotient-add-op`*

Lean: *`ReesAlgebra.HomogeneousAwayZero.quotientAdd_zero`*

For $u \in \mathrm{HChart}_a$, $u \boxplus 0_H = u$.

Proof. Uses: `def:nagata-homogeneous-away-chart-raw-zero`,`def:nagata-homogeneous-away-chart-raw-add`,`def:nagata-homogeneous-away-chart-quotient-zero-op`,`def:nagata-homogeneous-away-chart-quotient-add-op`

Quotient induction reduces to (n, x). The left representative is $(n, a^0 x + a^n 0) = (n, x)$. □

Lemma 2.3.46 (Commutativity of quotient addition)**.** *Uses:* *`def:nagata-homogeneous-away-chart-quotient-add-op`*

Lean: *`ReesAlgebra.HomogeneousAwayZero.quotientAdd_comm`*

For $u, v \in \mathrm{HChart}_a$, $u \boxplus v = v \boxplus u$.

Proof. Uses: `def:nagata-homogeneous-away-chart-raw-add`,`def:nagata-homogeneous-away-chart-quotient-add-op`

On (n, x) and (m, y), both representatives have degree $n + m$ after transporting $m + n = n + m$, and their numerators are equal by commutativity of addition. □

Lemma 2.3.47 (Additive inverse law on the quotient). *Uses:* *def:nagata-homogeneous-away-chart-quotient-zero-op, def:nagata-homogeneous-away-chart-quotient-neg-op, def:nagata-homogeneous-away-chart-quotient-add-op*

Lean: *ReesAlgebra.HomogeneousAwayZero.quotientAdd_neg*

For $u \in \mathrm{HChart}_a$, $u \boxplus (\mathsf{neg}_H(u)) = 0_H$.

Proof. Uses: `def:nagata-homogeneous-away-chart-representatives`, `def:nagata-homogeneous-away-chart-raw-zero`, `def:nagata-homogeneous-away-chart-raw-neg`, `def:nagata-homogeneous-away-chart-raw-add`, `def:nagata-homogeneous-away-chart-quotient-zero-op`, `def:nagata-homogeneous-away-chart-quotient-neg-op`, `def:nagata-homogeneous-away-chart-quotient-add-op`

For $u = [(n, x)]$, the left representative is $(2n, a^n x + a^n(-x)) = (2n, 0)$. It is ρ_a-related to $(0, 0)$ with witness 0, since both sides of the defining equality are zero. Quotient soundness gives the result. □

Lemma 2.3.48 (Left additive inverse law on the quotient). *Uses:* *def:nagata-homogeneous-away-chart-quotient-zero-op, def:nagata-homogeneous-away-chart-quotient-neg-op, def:nagata-homogeneous-away-chart-quotient-add-op*

Lean: *ReesAlgebra.HomogeneousAwayZero.quotientNeg_add*

For $u \in \mathrm{HChart}_a$, $\mathsf{neg}_H(u) \boxplus u = 0_H$.

Proof. Uses: `lemma:nagata-homogeneous-away-chart-quotient-add-comm`, `lemma:nagata-homogeneous-away-chart-quotient-add-neg`

Commute the two summands and apply the right additive inverse law. □

Lemma 2.3.49 (Associativity of quotient multiplication). *Uses:* *def:nagata-homogeneous-away-chart-quotient-mul-op*

Lean: *ReesAlgebra.HomogeneousAwayZero.quotientMul_assoc*

For $u, v, w \in \mathrm{HChart}_a$, $(u \boxtimes v) \boxtimes w = u \boxtimes (v \boxtimes w)$.

Proof. Uses: `def:nagata-homogeneous-away-chart-raw-mul`, `def:nagata-homogeneous-away-chart-quotient-mul-op`

Quotient induction gives representatives of degree $n+m+k$ and numerators $(xy)z$ and $x(yz)$; they are equal by associativity in A. □

Lemma 2.3.50 (Left unit law for quotient multiplication). *Uses:* *def:nagata-homogeneous-away-chart-quotient-one-op, def:nagata-homogeneous-away-chart-quotient-mul-op*

Lean: *ReesAlgebra.HomogeneousAwayZero.quotientOne_mul*

For $u \in \mathrm{HChart}_a$, $1_H \boxtimes u = u$.

Proof. Uses: `def:nagata-homogeneous-away-chart-raw-one`, `def:nagata-homogeneous-away-chart-raw-mul`, `def:nagata-homogeneous-away-chart-quotient-one-op`, `def:nagata-homogeneous-away-chart-quotient-mul-op`

On (n, x) the left representative is $(n, 1x) = (n, x)$. □

Lemma 2.3.51 (Right unit law for quotient multiplication). *Uses:* *def:nagata-homogeneous-away-chart-quotient-one-op, def:nagata-homogeneous-away-chart-quotient-mul-op*

Lean: *ReesAlgebra.HomogeneousAwayZero.quotientMul_one*

For $u \in \mathrm{HChart}_a$, $u \boxtimes 1_H = u$.

Proof. Uses: `def:nagata-homogeneous-away-chart-raw-one`, `def:nagata-homogeneous-away-chart-raw-mul`, `def:nagata-homogeneous-away-chart-quotient-one-op`, `def:nagata-homogeneous-away-chart-quotient-mul-op`

On (n, x) the left representative is $(n, x1) = (n, x)$. □

Lemma 2.3.52 (Commutativity of quotient multiplication). *Uses:* *def:nagata-homogeneous-away-chart-quotient-mul-op*

Lean: *ReesAlgebra.HomogeneousAwayZero.quotientMul_comm*

For $u, v \in \mathrm{HChart}_a$, $u \boxtimes v = v \boxtimes u$.

Proof. Uses: `def:nagata-homogeneous-away-chart-raw-mul`, `def:nagata-homogeneous-away-chart-quotient-mul-op`

On (n, x) and (m, y), transport the second representative across $m + n = n + m$ and use $xy = yx$. □

Lemma 2.3.53 (Left distributivity on the quotient). *Uses:* *def:nagata-homogeneous-away-chart-quotient-add-op*, *def:nagata-homogeneous-away-chart-quotient-mul-op*

Lean: *ReesAlgebra.HomogeneousAwayZero.quotient_left_distrib*

For $u, v, w \in \mathrm{HChart}_a$, $u \boxtimes (v \boxplus w) = (u \boxtimes v) \boxplus (u \boxtimes w)$.

Proof. Uses: `def:nagata-homogeneous-away-chart-representatives`, `def:nagata-homogeneous-away-chart-raw-add`, `def:nagata-homogeneous-away-chart-raw-mul`, `def:nagata-homogeneous-away-chart-quotient-add-op`, `def:nagata-homogeneous-away-chart-quotient-mul-op`

Use representatives (n, x), (m, y), and (k, z). The left side has degree $n+m+k$ and numerator $x(a^k y + a^m z)$. The right side has degree $2n+m+k$ and numerator $a^{n+k}xy + a^{n+m}xz = a^n x(a^k y + a^m z)$. These representatives are ρ_a-related with witness 0; after multiplying the first numerator by a^{2n+m+k} and the second by a^{n+m+k}, the two coefficients are identical. □

Lemma 2.3.54 (Right distributivity on the quotient). *Uses:* *def:nagata-homogeneous-away-chart-quotient-add-op*, *def:nagata-homogeneous-away-chart-quotient-mul-op*

Lean: *ReesAlgebra.HomogeneousAwayZero.quotient_right_distrib*

For $u, v, w \in \mathrm{HChart}_a$, $(u \boxplus v) \boxtimes w = (u \boxtimes w) \boxplus (v \boxtimes w)$.

Proof. Uses: `lemma:nagata-homogeneous-away-chart-quotient-left-distrib`, `lemma:nagata-homogeneous-away-chart-quotient-mul-comm`

Commute the two factors in each of the three products, apply left distributivity once, and commute them back. □

Lemma 2.3.55 (Left absorption of zero on the quotient). *Uses:* *def:nagata-homogeneous-away-chart-quotient-zero-op*, *def:nagata-homogeneous-away-chart-quotient-mul-op*

Lean: *ReesAlgebra.HomogeneousAwayZero.quotient_zero_mul*

For $u \in \mathrm{HChart}_a$, $0_H \boxtimes u = 0_H$.

Proof. Uses: `def:nagata-homogeneous-away-chart-representatives`, `def:nagata-homogeneous-away-chart-raw-zero`, `def:nagata-homogeneous-away-chart-raw-mul`, `def:nagata-homogeneous-away-chart-quotient-zero-op`, `def:nagata-homogeneous-away-chart-quotient-mul-op`

On (n, x) the product representative is $(n, 0x) = (n, 0)$, which is ρ_a-related to $(0, 0)$ with witness 0. □

Lemma 2.3.56 (Right absorption of zero on the quotient). *Uses:* *def:nagata-homogeneous-away-chart-quotient-zero-op*, *def:nagata-homogeneous-away-chart-quotient-mul-op*

Lean: *ReesAlgebra.HomogeneousAwayZero.quotient_mul_zero*

For $u \in \mathrm{HChart}_a$, $u \boxtimes 0_H = 0_H$.

Proof. Uses: `lemma:nagata-homogeneous-away-chart-quotient-zero-mul`, `lemma:nagata-homogeneous-away-chart-quotient-mul-comm`

Commute the two factors and apply left absorption. □

Definition 2.3.57 (Commutative-ring structure on the homogeneous-chart quotient). Uses: `def:nagata-homogeneous-away-chart-quotient-type`, `def:nagata-homogeneous-away-chart-quotient-zero-op`, `def:nagata-homogeneous-away-chart-quotient-one-op`, `def:nagata-homogeneous-away-chart-quotient-neg-op`, `def:nagata-homogeneous-away-chart-quotient-add-op`, `def:nagata-homogeneous-away-chart-quotient-mul-op`

Lean: `ReesAlgebra.HomogeneousAwayZero.instCommRing`

Constructed by: `lemma:nagata-homogeneous-away-chart-quotient-add-assoc`, `lemma:nagata-homogeneous-away-chart-quotient-zero-add`, `lemma:nagata-homogeneous-away-chart-quotient-neg-add`, `lemma:nagata-homogeneous-away-chart-quotient-mul-assoc`, `lemma:nagata-homogeneous-away-chart-quotient-mul-comm`, `lemma:nagata-homogeneous-away-chart-quotient-one-mul`, `lemma:nagata-homogeneous-away-chart-quotient-left-distrib`

Invoke the exact mathlib constructor `CommRing.ofMinimalAxioms`. Its seven arguments, in constructor order, are supplied by

`add_assoc`	the preceding add-associativity lemma
`zero_add`	the preceding zero-add lemma
`neg_add_cancel`	the preceding neg-add-cancel lemma
`mul_assoc`	the preceding multiplication-associativity lemma
`mul_comm`	the preceding multiplication-commutativity lemma
`one_mul`	the preceding one-mul lemma
`left_distrib`	the preceding left-distributivity lemma.

The other seven previously proved equations are retained as independent consistency lemmas, but they are not falsely listed as constructor fields.

Definition 2.3.58 (Actual degree-zero homogeneous localization used by Proj)**.** Uses: `def:nagata-rees-algebra,def:nagata-rees-piece`

Lean: `HomogeneousLocalization.Away`

Constructed by: `lemma:nagata-rees-degree-one-element`

Let $\mathcal{A}_n = \mathcal{R}_A(I)_n$ be the grading from the preceding Rees-piece Definition, and let $h_{at} : at \in \mathcal{A}_1$ be the previously certified degree-one membership proof. Define

$$\mathcal{L}_a^{\mathrm{hom}} := \mathrm{HomogeneousLocalization.\,Away}(n \mapsto \mathcal{A}_n)(at).$$

This is literally the coordinate ring occurring in `AlgebraicGeometry.Proj.basicOpenIsoSpec`; it is not an ordinary `Localization.Away` range subring.

Definition 2.3.59 (Actual homogeneous-away element represented by a pair)**.** Uses: `def:nagata-homogeneous-away-chart-representatives,def:nagata-rees-away-localization`

Lean: `HomogeneousLocalization.Away.mk`

For $u = (n, x) \in \mathrm{HRep}_a$, let h_x be the proof that the Rees monomial xt^n lies in $\mathcal{A}_{n\cdot 1}$. Define

$$\lambda_a^{\mathrm{hom}}(n, x) := \mathrm{HomogeneousLocalization.\,Away.\,mk}(n \mapsto \mathcal{A}_n)\ h_{at}\ n\ (xt^n)\ h_x \in \mathcal{L}_a^{\mathrm{hom}}.$$

Every argument of `Away.mk`, including the degree-one and numerator membership proofs, is displayed.

Lemma 2.3.60 (Actual homogeneous-away elements detect the representative relation)**.** *Uses:* *`def:nagata-homogeneous-away-ambient-fraction`*

Lean: *`ReesAlgebra.HomogeneousAwayZero.actualFraction_eq_iff`*

For $u, v \in \mathrm{HRep}_a$,

$$\lambda_a^{\mathrm{hom}}(u) = \lambda_a^{\mathrm{hom}}(v) \quad \Longleftrightarrow \quad u\ \rho_a\ v.$$

Proof. Uses: `def:nagata-homogeneous-away-chart-representatives,def:nagata-homogeneous-away-ambient-fraction,lemma:nagata-rees-piece-membership`

Write $u = (n, x)$ and $v = (m, y)$. Apply the exact mathlib equivalence `ext_iff_val` (in the `HomogeneousLocalization` namespace); the goal becomes equality of the underlying ordinary localization fractions. Its equality criterion gives

$$\exists r, \quad (at)^r (at)^m (xt^n) = (at)^r (at)^n (yt^m).$$

Both sides lie in degree $r + m + n$. One use of Lemma 2.3.7 compares the coefficients and yields $a^{r+m}x = a^{r+n}y$. Reversing these three steps proves the converse. □

Definition 2.3.61 (Proj standard degree-zero homogeneous localization)**.** Uses: `def:nagata-rees-away-localization`

Lean: `HomogeneousLocalization.Away`

Set

$$\mathcal{R}_A(I)_{(at),0} := \mathcal{L}_a^{\mathrm{hom}}.$$

This notation is definitionally the actual `HomogeneousLocalization.Away` used by `Proj.basicOpenIsoSpec`.

Definition 2.3.62 (Raw map to the actual homogeneous localization)**.** Uses: `def:nagata-homogeneous-away-chart-representatives,def:nagata-homogeneous-away-ambient-fraction,def:nagata-degree-zero-homogeneous-localization`

Lean: `ReesAlgebra.HomogeneousAwayZero.toActualAwayRaw`

Define

$$\eta_a^{\mathrm{raw}}(u) := \lambda_a^{\mathrm{hom}}(u) \in \mathcal{R}_A(I)_{(at),0}.$$

Lemma 2.3.63 (The raw actual-localization map respects representatives)**.** *Uses:* *`def:nagata-homogeneous-quotient-to-standard-raw-map`*

Lean: *`ReesAlgebra.HomogeneousAwayZero.toActualAwayRaw_respects`*

If $u\ \rho_a\ v$, then $\eta_a^{\mathrm{raw}}(u) = \eta_a^{\mathrm{raw}}(v)$.

Proof. Uses: `def:nagata-homogeneous-quotient-to-standard-raw-map,lemma:nagata-homogeneous-away-ambient-fraction-equality`

This is the forward implication of Lemma 2.3.60. □

Definition 2.3.64 (Quotient function to the actual homogeneous localization). Uses: def:nagata-homogeneous-away-chart-quotient-type, def:nagata-homogeneous-quotient-to-standard-raw-map, def:nagata-degree-zero-homogeneous-localization

Lean: ReesAlgebra.HomogeneousAwayZero.toActualAwayFunction

Constructed by: lemma:nagata-homogeneous-quotient-to-standard-raw-map-respects

Define

$$\eta_a \colon \mathrm{HChart}_a \longrightarrow \mathcal{R}_A(I)_{(at),0}$$

by Quotient.lift of η_a^{raw} and the displayed respect proof.

Lemma 2.3.65 (The actual-localization quotient function preserves zero). Uses: def:nagata-homogeneous-quotient-to-standard-function, def:nagata-homogeneous-away-chart-quotient-comm-ring

Lean: ReesAlgebra.HomogeneousAwayZero.toActualAwayFunction_zero

One has $\eta_a(0) = 0$.

Proof. Uses: def:nagata-homogeneous-away-chart-raw-zero, def:nagata-homogeneous-away-ambient-fraction, def:nagata-homogeneous-quotient-to-standard-raw-map, def:nagata-homogeneous-quotient-to-standard-function

The quotient computation rule reduces the assertion to HomogeneousLocalization.Away.mk with numerator zero and exponent zero, which is zero by equality of underlying localization values and ext_iff_val (in the HomogeneousLocalization namespace). □

Lemma 2.3.66 (The actual-localization quotient function preserves one). Uses: def:nagata-homogeneous-quotient-to-standard-function, def:nagata-homogeneous-away-chart-quotient-comm-ring

Lean: ReesAlgebra.HomogeneousAwayZero.toActualAwayFunction_one

One has $\eta_a(1) = 1$.

Proof. Uses: def:nagata-homogeneous-away-chart-raw-one, def:nagata-homogeneous-away-ambient-fraction, def:nagata-homogeneous-quotient-to-standard-raw-map, def:nagata-homogeneous-quotient-to-standard-function

Evaluate on $(0, 1)$ and apply ext_iff_val (in the HomogeneousLocalization namespace); both values are $1/1$. □

Lemma 2.3.67 (The actual-localization quotient function preserves addition). Uses: def:nagata-homogeneous-quotient-to-standard-function, def:nagata-homogeneous-away-chart-quotient-comm-ring

Lean: ReesAlgebra.HomogeneousAwayZero.toActualAwayFunction_add

For $u, v \in \mathrm{HChart}_a$, $\eta_a(u + v) = \eta_a(u) + \eta_a(v)$.

Proof. Uses: def:nagata-homogeneous-away-chart-raw-add, def:nagata-homogeneous-away-ambient-fraction, def:nagata-homogeneous-quotient-to-standard-raw-map, def:nagata-homogeneous-quotient-to-standard-function, def:nagata-homogeneous-away-chart-quotient-add-op

Quotient induction reduces to (n, x) and (m, y). Apply ext_iff_val (in the HomogeneousLocalization namespace); both underlying fractions have numerator $(a^m x + a^n y)t^{n+m}$ and denominator $(at)^{n+m}$. □

Lemma 2.3.68 (The actual-localization quotient function preserves multiplication). Uses: def:nagata-homogeneous-quotient-to-standard-function, def:nagata-homogeneous-away-chart-quotient-comm-ring

Lean: ReesAlgebra.HomogeneousAwayZero.toActualAwayFunction_mul

For $u, v \in \mathrm{HChart}_a$, $\eta_a(uv) = \eta_a(u)\eta_a(v)$.

Proof. Uses: def:nagata-homogeneous-away-chart-raw-mul, def:nagata-homogeneous-away-ambient-fraction, def:nagata-homogeneous-quotient-to-standard-raw-map, def:nagata-homogeneous-quotient-to-standard-function, def:nagata-homogeneous-away-chart-quotient-mul-op

Quotient induction reduces to two representatives. After applying ext_iff_val; both values are the fraction $xyt^{n+m}/(at)^{n+m}$. □

Definition 2.3.69 (Ring homomorphism from the quotient to the actual localization)**.** Uses: `def:nagata-homogeneous-quotient-to-standard-function`, `def:nagata-homogeneous-away-chart-quotient-comm-ring`, `def:nagata-degree-zero-homogeneous-localization`

Lean: `ReesAlgebra.HomogeneousAwayZero.toActualAway`

Constructed by: `lemma:nagata-homogeneous-quotient-to-standard-zero`, `lemma:nagata-homogeneous-quotient-to-standard-one`, `lemma:nagata-homogeneous-quotient-to-standard-add`, `lemma:nagata-homogeneous-quotient-to-standard-mul`

Package η_a and the preceding four preservation proofs as a ring homomorphism $\mathrm{HChart}_a \to \mathcal{R}_A(I)_{(at),0}$.

Lemma 2.3.70 (Surjectivity onto the actual homogeneous localization)**.** *Uses:* *def:nagata-homogeneous-quotient-to-standard-map*

Lean: *ReesAlgebra.HomogeneousAwayZero.toActualAway_surjective*

The ring homomorphism η_a is surjective.

Proof. Uses: `def:nagata-homogeneous-quotient-to-standard-raw-map`, `def:nagata-homogeneous-quotient-to-standard-function`, `def:nagata-homogeneous-quotient-to-standard-map`, `lemma:nagata-rees-piece-membership`

Let $z \in$ HomogeneousLocalization. Away $\mathcal{A}(at)$. Instantiate the exact mathlib theorem `Away.mk_surjective` (in the `HomogeneousLocalization` namespace) with $(\mathcal{A}, h_{at}, z)$. It gives n, a numerator F, a proof $F \in \mathcal{A}_{n\cdot 1}$, and Away. $\mathrm{mk}(h_{at}, n, F) = z$. By Lemma 2.3.7, write $F = xt^n$ with $x \in I^n$. Then the quotient class $[(n, x)]$ maps to z by the computation rule for `Quotient.lift`. □

Lemma 2.3.71 (Injectivity of the quotient-to-actual-localization map)**.** *Uses:* *def:nagata-homogeneous-quotient-to-standard-map*

Lean: *ReesAlgebra.HomogeneousAwayZero.toActualAway_injective*

The ring homomorphism η_a is injective.

Proof. Uses: `def:nagata-homogeneous-quotient-to-standard-raw-map`, `def:nagata-homogeneous-quotient-to-standard-function`, `def:nagata-homogeneous-quotient-to-standard-map`, `lemma:nagata-homogeneous-away-ambient-fraction-equality`

Apply quotient induction to classes $[u]$ and $[v]$. Equality of their images and the reverse implication of Lemma 2.3.60 give $u \rho_a v$; `Quotient.sound` then gives $[u] = [v]$. □

Definition 2.3.72 (Equivalence with the actual Proj coordinate ring)**.** Uses: `def:nagata-homogeneous-quotient-to-standard-map`, `def:nagata-homogeneous-away-chart-quotient-comm-ring`, `def:nagata-degree-zero-homogeneous-localization`

Lean: `ReesAlgebra.HomogeneousAwayZero.quotientEquivActualAway`

Constructed by: `lemma:nagata-homogeneous-quotient-to-standard-surjective`, `lemma:nagata-homogeneous-quotient-to-standard-injective`

Apply the exact mathlib constructor `RingEquiv.ofBijective` to η_a and its two preceding property lemmas. Denote the result by

$$\Psi_a \colon \mathrm{HChart}_a \xrightarrow{\sim} \text{HomogeneousLocalization. Away } \mathcal{A}(at).$$

Definition 2.3.73 (Raw representative function to the affine chart ring)**.** Uses: `def:nagata-homogeneous-away-chart-representatives`, `def:nagata-blowup-chart-ring`

Lean: `ReesAlgebra.HomogeneousAwayZero.representativeToChart`

Constructed by: `lemma:nagata-chart-ring-fraction-mem`

Define

$$\vartheta_a(n, x) = \left\langle \frac{x}{a^n}, \mathrm{chartFraction_mem}(x \in I^n) \right\rangle \in A[I/a].$$

The second displayed component is the explicit chart-subring membership certificate recorded by `\constructedby`.

Lemma 2.3.74 (The raw affine-chart function respects representatives)**.** *Uses:* *def:nagata-homogeneous-away-chart-representatives*, *def:nagata-homogeneous-away-chart-representative-function*

Lean: *ReesAlgebra.HomogeneousAwayZero.representativeToChart_respects*

If $(n, x)\ \rho_a\ (m, y)$, then $\vartheta_a(n, x) = \vartheta_a(m, y)$ in $A[I/a]$.

Proof. Uses: `lemma:nagata-chart-ring-fraction-equality`

The defining witness for ρ_a is exactly the right-hand side of Lemma 2.3.23 instantiated with $(C, K, c) = (A, I, a)$. That lemma gives equality of the underlying fractions; subtype extensionality gives equality in $A[I/a]$. □

Definition 2.3.75 (Quotient function to the affine chart ring). Uses: `def:nagata-homogeneous-away-chart-quotient-type`, `def:nagata-homogeneous-away-chart-representative-function`, `def:nagata-blowup-chart-ring`

Lean: `ReesAlgebra.HomogeneousAwayZero.toChartFunction`

Constructed by: `lemma:nagata-homogeneous-away-chart-representative-respects`

Define

$$\overline{\vartheta}_a \colon \mathrm{HChart}_a \longrightarrow A[I/a]$$

by quotient lift of ϑ_a.

Lemma 2.3.76 (The quotient chart function preserves zero). Uses: *def:nagata-homogeneous-away-chart-quotient-function*, *def:nagata-homogeneous-away-chart-quotient-comm-ring*

Lean: *ReesAlgebra.HomogeneousAwayZero.toChartFunction_zero*

One has $\overline{\vartheta}_a(0) = 0$.

Proof. Uses: `def:nagata-homogeneous-away-chart-raw-zero`, `def:nagata-homogeneous-away-chart-representative-function`, `def:nagata-homogeneous-away-chart-quotient-function`

Evaluate the quotient lift on $(0, 0)$; the underlying fraction is $0/a^0 = 0$. Apply subtype extensionality. □

Lemma 2.3.77 (The quotient chart function preserves one). Uses: *def:nagata-homogeneous-away-chart-quotient-function*, *def:nagata-homogeneous-away-chart-quotient-comm-ring*

Lean: *ReesAlgebra.HomogeneousAwayZero.toChartFunction_one*

One has $\overline{\vartheta}_a(1) = 1$.

Proof. Uses: `def:nagata-homogeneous-away-chart-raw-one`, `def:nagata-homogeneous-away-chart-representative-function`, `def:nagata-homogeneous-away-chart-quotient-function`

Evaluate on $(0, 1)$; the fraction is $1/a^0 = 1$. □

Lemma 2.3.78 (The quotient chart function preserves addition). Uses: *def:nagata-homogeneous-away-chart-quotient-function*, *def:nagata-homogeneous-away-chart-quotient-comm-ring*

Lean: *ReesAlgebra.HomogeneousAwayZero.toChartFunction_add*

For $u, v \in \mathrm{HChart}_a$, $\overline{\vartheta}_a(u + v) = \overline{\vartheta}_a(u) + \overline{\vartheta}_a(v)$.

Proof. Uses: `def:nagata-homogeneous-away-chart-raw-add`, `def:nagata-homogeneous-away-chart-representative-function`, `def:nagata-homogeneous-away-chart-quotient-function`, `def:nagata-homogeneous-away-chart-quotient-add-op`

Quotient induction reduces to (n, x) and (m, y). Both sides have underlying localization element $(a^m x + a^n y)/a^{n+m}$. Apply subtype extensionality. □

Lemma 2.3.79 (The quotient chart function preserves multiplication). Uses: *def:nagata-homogeneous-away-chart-quotient-function*, *def:nagata-homogeneous-away-chart-quotient-comm-ring*

Lean: *ReesAlgebra.HomogeneousAwayZero.toChartFunction_mul*

For $u, v \in \mathrm{HChart}_a$, $\overline{\vartheta}_a(uv) = \overline{\vartheta}_a(u)\overline{\vartheta}_a(v)$.

Proof. Uses: `def:nagata-homogeneous-away-chart-raw-mul`, `def:nagata-homogeneous-away-chart-representative-function`, `def:nagata-homogeneous-away-chart-quotient-function`, `def:nagata-homogeneous-away-chart-quotient-mul-op`

Quotient induction reduces to (n, x) and (m, y); both sides have underlying fraction xy/a^{n+m}. □

Definition 2.3.80 (Ring homomorphism from the quotient to the affine chart). Uses: `def:nagata-homogeneous-away-chart-quotient-function`, `def:nagata-homogeneous-away-chart-quotient-comm-ring`, `def:nagata-blowup-chart-ring`

Lean: `ReesAlgebra.HomogeneousAwayZero.quotientToChart`

Constructed by: `lemma:nagata-homogeneous-away-chart-quotient-zero`, `lemma:nagata-homogeneous-away-chart-quotient-one`, `lemma:nagata-homogeneous-away-chart-quotient-add`, `lemma:nagata-homogeneous-away-chart-quotient-mul`

Package $\overline{\vartheta}_a$ and its four preservation lemmas as a ring homomorphism $\varphi_a \colon \mathrm{HChart}_a \to A[I/a]$.

Lemma 2.3.81 (Formula for the quotient-to-chart ring homomorphism). *Uses:* def:nagata-homogeneous-quotient-to-chart-map

Lean: ReesAlgebra.HomogeneousAwayZero.quotientToChart_apply

For $n \in \mathbf{N}$ and $x \in I^n$,

$$\varphi_a([(n,x)]) = x/a^n.$$

Proof. Uses: def:nagata-homogeneous-away-chart-representative-function, def:nagata-homogeneous-away-chart-quotient-function, def:nagata-homogeneous-quotient-to-chart-map

This is the computation rule of the quotient lift, followed by the defining formula for ϑ_a. □

Lemma 2.3.82 (Surjectivity of the quotient-to-chart map). *Uses:* def:nagata-homogeneous-quotient-to-chart-map

Lean: ReesAlgebra.HomogeneousAwayZero.quotientToChart_surjective

The ring homomorphism φ_a is surjective.

Proof. Uses: lemma:nagata-chart-ring-normal-form, lemma:nagata-homogeneous-quotient-to-chart-map-apply

Given $z \in A[I/a]$, use Lemma 2.3.18 to choose n and $x \in I^n$ with $z = x/a^n$. Then $[(n,x)]$ maps to z by the displayed formula. □

Lemma 2.3.83 (Injectivity of the quotient-to-chart map). *Uses:* def:nagata-homogeneous-quotient-to-chart-map

Lean: ReesAlgebra.HomogeneousAwayZero.quotientToChart_injective

The ring homomorphism φ_a is injective.

Proof. Uses: lemma:nagata-chart-ring-fraction-equality, lemma:nagata-homogeneous-quotient-to-chart-map-apply

By quotient induction it suffices to compare $[(n,x)]$ and $[(m,y)]$. Equality of their images is equality of x/a^n and y/a^m. Lemma 2.3.23, instantiated with (A, I, a), gives an r with $a^{r+m}x = a^{r+n}y$. Thus the representatives are ρ_a-related, and quotient soundness proves equality of their classes. □

Definition 2.3.84 (Quotient-to-affine-chart ring equivalence). Uses: def:nagata-homogeneous-quotient-to-chart-map, def:nagata-homogeneous-away-chart-quotient-comm-ring, def:nagata-blowup-chart-ring

Lean: ReesAlgebra.HomogeneousAwayZero.quotientEquivChart

Constructed by: lemma:nagata-homogeneous-quotient-to-chart-surjective, lemma:nagata-homogeneous-quotient-to-chart-injective

Define

$$\Phi_a \colon \mathrm{HChart}_a \xrightarrow{\sim} A[I/a]$$

from the bijective ring homomorphism φ_a.

Lemma 2.3.85 (Inverse formula for the quotient-to-chart equivalence). *Uses:* def:nagata-homogeneous-quotient-chart-ring-equivalence

Lean: ReesAlgebra.HomogeneousAwayZero.quotientEquivChart_symm_apply

For $n \in \mathbf{N}$ and $x \in I^n$,

$$\Phi_a^{-1}(x/a^n) = [(n,x)].$$

Proof. Uses: def:nagata-homogeneous-quotient-chart-ring-equivalence, lemma:nagata-homogeneous-quotient-to-chart-map-apply

Apply the injective forward map Φ_a. The left side becomes x/a^n by the inverse law, and the right side becomes x/a^n by Lemma 2.3.81. □

Definition 2.3.86 (Homogeneous-localization chart map). Uses: def:nagata-homogeneous-quotient-standard-ring-equivalence, def:nagata-homogeneous-quotient-chart-ring-equivalence, def:nagata-degree-zero-homogeneous-localization, def:nagata-blowup-chart-ring

Lean: ReesAlgebra.homogeneousAwayToChart

Define the public ring homomorphism

$$\theta_a \colon \mathcal{R}_A(I)_{(at),0} \longrightarrow A[I/a]$$

as the composite

$$\mathcal{R}_A(I)_{(at),0} \xrightarrow{\Psi_a^{-1}} \mathrm{HChart}_a \xrightarrow{\Phi_a} A[I/a].$$

Thus the public domain is the independently constructed standard degree-zero homogeneous localization, not the bare quotient type.

Lemma 2.3.87 (Formula for the homogeneous-localization chart map)**.** *Uses:* def:nagata-homogeneous-away-to-chart, def:nagata-homogeneous-quotient-standard-ring-equivalence

Lean: ReesAlgebra.homogeneousAwayToChart_apply

For $n \in \mathbf{N}$ and $x \in I^n$,

$$\theta_a\big(\eta_a^{\mathrm{raw}}(n,x)\big) = \frac{x}{a^n}.$$

The input is the subtype element whose underlying ambient localization fraction is $(xt^n)/(at)^n$.

Proof. Uses: def:nagata-homogeneous-away-to-chart, def:nagata-homogeneous-quotient-standard-ring-equivalence, def:nagata-homogeneous-quotient-to-standard-function, def:nagata-homogeneous-quotient-to-standard-raw-map, def:nagata-homogeneous-quotient-chart-ring-equivalence, lemma:nagata-homogeneous-quotient-to-chart-map-apply

The forward map of Ψ_a sends $[(n,x)]$ to $\eta_a^{\mathrm{raw}}(n,x)$ by its quotient-lift computation rule. Hence Ψ_a^{-1} sends the displayed input to $[(n,x)]$. The formula for Φ_a is Lemma 2.3.81. □

Lemma 2.3.88 (Surjectivity of the homogeneous-localization chart map)**.** *Uses:* def:nagata-homogeneous-away-to-chart

Lean: ReesAlgebra.homogeneousAwayToChart_surjective

The ring homomorphism θ_a is surjective.

Proof. Uses: def:nagata-homogeneous-away-to-chart, lemma:nagata-homogeneous-quotient-to-chart-surjective

The map Ψ_a^{-1} is surjective because it is an equivalence, and Φ_a is surjective by Lemma 2.3.82. The composite of these two surjective functions is surjective. □

Lemma 2.3.89 (Injectivity of the homogeneous-localization chart map)**.** *Uses:* def:nagata-homogeneous-away-to-chart

Lean: ReesAlgebra.homogeneousAwayToChart_injective

The ring homomorphism θ_a is injective.

Proof. Uses: def:nagata-homogeneous-away-to-chart, lemma:nagata-homogeneous-quotient-to-chart-injective

The inverse equivalence Ψ_a^{-1} is injective, and Φ_a is injective by Lemma 2.3.83. Their composite is injective. □

Definition 2.3.90 (Ring equivalence between the standard homogeneous and affine chart rings)**.** Uses: def:nagata-homogeneous-away-to-chart, def:nagata-degree-zero-homogeneous-localization, def:nagata-blowup-chart-ring

Lean: ReesAlgebra.homogeneousAwayChartRingEquiv

Constructed by: lemma:nagata-homogeneous-away-to-chart-surjective, lemma:nagata-homogeneous-away-to-chart-injective

Define

$$\Theta_a \colon \mathcal{R}_A(I)_{(at),0} \xrightarrow{\sim} A[I/a]$$

from the bijective public ring homomorphism θ_a. Its forward map is definitionally θ_a.

Lemma 2.3.91 (Inverse formula for the public homogeneous-chart equivalence)**.** *Uses:* def:nagata-homogeneous-away-chart-ring-equivalence, def:nagata-homogeneous-quotient-to-standard-raw-map

Lean: ReesAlgebra.homogeneousAwayChartRingEquiv_symm_apply

For $n \in \mathbf{N}$ and $x \in I^n$,

$$\Theta_a^{-1}(x/a^n) = \eta_a^{\mathrm{raw}}(n,x).$$

Proof. Uses: def:nagata-homogeneous-away-chart-ring-equivalence, lemma:nagata-homogeneous-away-to-chart

Apply the injective forward map Θ_a. The left side becomes x/a^n by the inverse law; the right side becomes x/a^n by Lemma 2.3.87. □

Definition 2.3.92 (Base-ring map into the actual homogeneous localization)**.** Uses: `def:nagata-degree-zero-homogeneous-localization`,`def:nagata-rees-piece`

Lean: `ReesAlgebra.actualAwayBaseRingMap`

Constructed by: `lemma:nagata-rees-degree-zero`

Let

$$\epsilon_0 \colon \mathcal{R}_A(I)_0 \xrightarrow{\sim} A$$

be the previously certified coefficient equivalence. Define the actual-Away base-ring homomorphism

$$\iota_a^{\mathrm{away}} \colon A \longrightarrow \text{HomogeneousLocalization.Away } \mathcal{A}(at)$$

by the explicit composite

$$A \xrightarrow{\epsilon_0^{-1}} \mathcal{R}_A(I)_0 \xrightarrow{\text{fromZeroRingHom}} \text{HomogeneousLocalization.Away } \mathcal{A}(at).$$

Lemma 2.3.93 (Formula for the actual-Away base-ring map)**.** *Uses:* `def:nagata-actual-away-base-ring-map`,`def:nagata-homogeneous-quotient-to-standard-raw-map`

Lean: `ReesAlgebra.actualAwayBaseRingMap_apply`

For $r \in A$, let $\widetilde{r}_0 \in I^0$ be the subtype element with underlying value r, using $I^0 = \top$. Then

$$\iota_a^{\mathrm{away}}(r) = \eta_a^{\mathrm{raw}}(0, \widetilde{r}_0).$$

Proof. Uses: `def:nagata-actual-away-base-ring-map`,`def:nagata-homogeneous-quotient-to-standard-raw-map`,`lemma:nagata-rees-degree-zero`

Unfold both maps. The inverse of ϵ_0 sends r to the constant degree-zero Rees monomial rt^0. The exact map `HomogeneousLocalization.fromZeroRingHom` sends it to the homogeneous fraction with numerator rt^0 and denominator $(at)^0$. This is precisely the defining `HomogeneousLocalization.Away.mk` expression for $\eta_a^{\mathrm{raw}}(0, \widetilde{r}_0)$. □

Definition 2.3.94 (Base-ring map into the affine chart ring)**.** Uses: `def:nagata-blowup-chart-ring`

Lean: `ReesAlgebra.affineChartBaseRingMap`

Define

$$\iota_a^{\mathrm{chart}} \colon A \longrightarrow A[I/a]$$

to be the algebra-structure map of the A-subalgebra $A[I/a] \subseteq A_a$.

Lemma 2.3.95 (Formula for the affine-chart base-ring map)**.** *Uses:* `def:nagata-affine-chart-base-ring-map`

Lean: `ReesAlgebra.affineChartBaseRingMap_apply`

For every $r \in A$,

$$\iota_a^{\mathrm{chart}}(r) = \frac{r}{1} = \frac{r}{a^0}.$$

Proof. Uses: `def:nagata-affine-chart-base-ring-map`,`def:nagata-blowup-chart-ring`

This is the defining formula for the algebra map into the localization, restricted to the chart subalgebra, followed by the equality $a^0 = 1$. □

Lemma 2.3.96 (The homogeneous-chart equivalence is compatible with the base ring)**.** *Uses:* `def:nagata-actual-away-base-ring-map`,`def:nagata-affine-chart-base-ring-map`,`def:nagata-homogeneous-away-chart-ring-equivalence`

Lean: `ReesAlgebra.homogeneousAwayChartRingEquiv_base`

The two ring homomorphisms from A to $A[I/a]$ are equal:

$$\Theta_a.\mathit{toRingHom} \circ \iota_a^{\mathrm{away}} = \iota_a^{\mathrm{chart}}.$$

Proof. Uses: `def:nagata-homogeneous-away-chart-ring-equivalence`,`lemma:nagata-actual-away-base-ring-map-apply`,`lemma:nagata-affine-chart-base-ring-map-apply`,`lemma:nagata-homogeneous-away-to-chart`

Apply ring-homomorphism extensionality and fix $r \in A$. Replace $\iota_a^{\mathrm{away}}(r)$ by $\eta_a^{\mathrm{raw}}(0, \widetilde{r}_0)$ using Lemma 2.3.93. The forward map of Θ_a is definitionally θ_a, so Lemma 2.3.87, with $n = 0$, gives

$$\Theta_a(\iota_a^{\mathrm{away}}(r)) = r/a^0 = r/1.$$

This equals $\iota_a^{\mathrm{chart}}(r)$ by Lemma 2.3.95. □

Lemma 2.3.97 (Inverse base-ring compatibility). *Uses:* *def:nagata-actual-away-base-ring-map, def:nagata-affine-chart-base-ring-map, def:nagata-homogeneous-away-chart-ring-equivalence*

Lean: *ReesAlgebra.homogeneousAwayChartRingEquiv_symm_base*

The inverse equivalence satisfies

$$\Theta_a^{-1}.toRingHom \circ \iota_a^{\mathrm{chart}} = \iota_a^{\mathrm{away}}.$$

Proof. Uses: `lemma:nagata-homogeneous-away-chart-ring-equivalence-base-compatible`

Apply ring-homomorphism extensionality and fix $r \in A$. Apply the injective function Θ_a to the desired equality. The left side reduces by the inverse law to $\iota_a^{\mathrm{chart}}(r)$; the right side is the same element by Lemma 2.3.96. □

Definition 2.3.98 (Structure morphism on the Proj basic open). Uses: `def:nagata-affine-blowup,def:nagata-rees-piece`

Lean: `AlgebraicGeometry.affineBlowupBasicOpenStructure`

Constructed by: `lemma:nagata-rees-degree-zero`

With $\epsilon_0 \colon \mathcal{R}_A(I)_0 \simeq A$ as above, define

$$\pi_a^{\mathrm{away}} \colon D_+(at) \longrightarrow \mathrm{Spec}(A)$$

to be the composite

$$D_+(at) \xrightarrow{\iota} \mathrm{Proj}(\mathcal{R}_A(I)) \xrightarrow{\text{Proj.toSpecZero}} \mathrm{Spec}(\mathcal{R}_A(I)_0) \xrightarrow{\text{Spec.map}(\epsilon_0^{-1})} \mathrm{Spec}(A).$$

Thus this is literally the restriction of the affine blow-up structure morphism to the basic open.

Definition 2.3.99 (Structure morphism on the affine chart spectrum). Uses: `def:nagata-affine-chart-base-ring-map`

Lean: `AlgebraicGeometry.affineBlowupChartStructure`

Define

$$\pi_a^{\mathrm{chart}} \colon \mathrm{Spec}(A[I/a]) \longrightarrow \mathrm{Spec}(A)$$

by

$$\pi_a^{\mathrm{chart}} := \mathrm{Spec.\,map}(\iota_a^{\mathrm{chart}}).$$

Lemma 2.3.100 (The Proj basic-open structure morphism through actual Away). *Uses:* *def:nagata-affine-blowup-basic-open-structure-morphism, def:nagata-actual-away-base-ring-map*

Lean: *AlgebraicGeometry.affineBlowupBasicOpenStructure_viaAway*

One has

$$\mathrm{Proj.\,basicOpenIsoSpec}(\mathcal{A}, at, h_{at}, h_1).\mathrm{hom} \gg \mathrm{Spec.\,map}(\iota_a^{\mathrm{away}}) = \pi_a^{\mathrm{away}}.$$

Proof. Uses: `def:nagata-affine-blowup-basic-open-structure-morphism,def:nagata-actual-away-base-ring-map,lemma:nagata-rees-degree-zero`

Apply the exact mathlib identity

`AlgebraicGeometry.Proj.awayι_toSpecZero`.

It identifies Spec. map(fromZeroRingHom) with

$$\mathrm{Proj.\,away}\,\iota \gg \mathrm{Proj.\,toSpecZero}\,.$$

Precompose it with Proj. basicOpenIsoSpec .hom and unfold Proj. away ι = Proj. basicOpenIsoSpec .inv $\gg \iota$. The adjacent isomorphism maps cancel. Postcompose with Spec. map(ϵ_0^{-1}) and use contravariant Spec functoriality. Since $\iota_a^{\mathrm{away}} = \mathrm{fromZeroRingHom} \circ \epsilon_0^{-1}$, the left side is the displayed composite and the right side is the Definition of π_a^{away}. □

Definition 2.3.101 (Canonical affine blow-up chart isomorphism). Uses: `def:nagata-affine-blowup,def:nagata-homogeneous-away-chart-ring-equivalence`

Lean: `AlgebraicGeometry.affineBlowup_basicOpenIsoSpec`

Let $h_{at} \colon at \in \mathcal{A}_1$ and $h_1 \colon 0 < 1$ be the displayed degree certificates. Define

$$\Xi_a := \mathrm{Proj.\,basicOpenIsoSpec}(\mathcal{A}, at, h_{at}, h_1) \gg\!\!\!\sim \mathrm{Scheme.Spec.\,mapIso}(\Theta_a.\mathrm{toCommRingCatIso}^{-1}).$$

The source is $D_+(at)$ and the target is Spec$(A[I/a])$. This is one composition of the exact mathlib isomorphisms `Proj.basicOpenIsoSpec` and `Scheme.Spec.mapIso`.

Definition 2.3.102 (Canonical affine blow-up chart morphism)**.** Uses: `def:nagata-affine-blowup-chart-isomorphism`

Lean: `AlgebraicGeometry.affineBlowup_basicOpenToSpec`

Define $\chi_a := \Xi_a.\mathrm{hom}$. Expanding the preceding Definition,

$$\chi_a = \mathrm{Proj.\,basicOpenIsoSpec}(\mathcal{A}, at, h_{at}, h_1).\mathrm{hom} \gg \mathrm{Spec.\,map}(\mathrm{CommRingCat.\,ofHom}(\Theta_a^{-1})).$$

The ring map is $\Theta_a^{-1}\colon A[I/a] \to \mathrm{HomogeneousLocalization.\,Away}\,\mathcal{A}(at)$ because `Spec.map` is contravariant.

Lemma 2.3.103 (The affine chart triangle over the base spectrum commutes)**.** Uses: `def:nagata-affine-blowup-chart-map`, `def:nagata-affine-blowup-basic-open-structure-morphism`, `def:nagata-affine-blowup-chart-structure-morphism`

Lean: `AlgebraicGeometry.affineBlowupChart_triangle`

The explicitly defined structure morphisms satisfy

$$\chi_a \gg \pi_a^{\mathrm{chart}} = \pi_a^{\mathrm{away}}.$$

Proof. Uses: `def:nagata-affine-blowup-chart-isomorphism`, `def:nagata-affine-blowup-chart-map`, `def:nagata-affine-blowup-chart-structure-morphism`, `lemma:nagata-homogeneous-away-chart-ring-equivalence-sym-base-compatible`, `lemma:nagata-affine-blowup-basic-open-structure-via-away`

Expand χ_a and π_a^{chart}. Apply the contravariant functor Spec to Lemma 2.3.97. The exact functoriality identity `Spec.map_comp` gives

$$\mathrm{Spec.\,map}(\Theta_a^{-1}) \gg \mathrm{Spec.\,map}(\iota_a^{\mathrm{chart}}) = \mathrm{Spec.\,map}(\iota_a^{\mathrm{away}}).$$

Precompose this equality with Proj. basicOpenIsoSpec .hom. The left side is $\chi_a \gg \pi_a^{\mathrm{chart}}$ by the two Definitions, and the right side is π_a^{away} by Lemma 2.3.100. □

Proposition 2.3.104 (Affine chart identification)**.** Uses: `def:nagata-affine-blowup-chart-map`, `def:nagata-affine-blowup-basic-open-structure-morphism`, `def:nagata-affine-blowup-chart-structure-morphism`

Lean: `AlgebraicGeometry.affineBlowup_basicOpenIsoSpec_hom_isIso`

The constructed morphism χ_a is an isomorphism and satisfies

$$\chi_a \gg \pi_a^{\mathrm{chart}} = \pi_a^{\mathrm{away}}.$$

Hence it is an isomorphism over $\mathrm{Spec}(A)$.

Proof. Uses: `def:nagata-affine-blowup-chart-isomorphism`, `def:nagata-affine-blowup-chart-map`, `lemma:nagata-affine-blowup-chart-triangle`

By definition, χ_a is the forward morphism of the already constructed scheme isomorphism Ξ_a, so its underlying morphism is an isomorphism. The required equality of structure morphisms is exactly Lemma 2.3.103. These two facts are the underlying-isomorphism and base-triangle components of the assertion that χ_a is an isomorphism over $\mathrm{Spec}(A)$. □

Lemma 2.3.105 (Degree-one generators cover Proj)**.** Uses:

Lean: `AlgebraicGeometry.affineBlowup_iSup_basicOpen_eq_top`

If $I = (f_0, \ldots, f_r)$, then the standard opens $D_+(f_i t)$ cover $\mathrm{Proj}(\mathcal{R}_A(I))$.

Proof. Uses: `prop:nagata-rees-finite-generation`

The elements $f_i t$ generate the positive-degree algebra and therefore the irrelevant ideal. A homogeneous prime in Proj cannot contain all of them. □

Proposition 2.3.106 (Finite affine chart cover)**.** Uses:

Lean: `AlgebraicGeometry.affineBlowup_affineOpenCover`

If $I = (f_0, \ldots, f_r)$, the affine opens

$$\mathrm{Spec}(A[I/f_i])$$

form a finite open cover of $\mathrm{Bl}_I(\mathrm{Spec}\,A)$.

Proof. Uses: `prop:nagata-affine-blowup-chart`, `lemma:nagata-rees-generators-cover-proj`

The opens $D_+(f_i t)$ cover by Lemma 2.3.105. Replace each one by the affine spectrum supplied by Proposition 2.3.104. The compatibility of the open embeddings with the structure map is part of that chart isomorphism. □

Lemma 2.3.107 (Blow-up of the unit ideal)**.** *Uses:* def:nagata-affine-blowup

Lean: AlgebraicGeometry.affineBlowup_top_iso

The blow-up of $\operatorname{Spec}(A)$ *in the unit ideal is canonically isomorphic to* $\operatorname{Spec}(A)$ *over* $\operatorname{Spec}(A)$*.*

Proof. Uses: prop:nagata-affine-blowup-chart

The Rees algebra of the unit ideal is $A[t]$. The single standard open $D_+(t)$ covers Proj, and its chart algebra $A[A/1]$ is A. □

Lemma 2.3.108 (Blow-up of the zero ideal)**.** *Uses:* def:nagata-affine-blowup

Lean: AlgebraicGeometry.affineBlowup_bot_isEmpty

The blow-up in the zero ideal is empty: the positive-degree part of its Rees algebra vanishes, so the irrelevant ideal is zero and Proj has no points.

Proof. Uses:

If $I = 0$, then $I^n = 0$ for every $n > 0$. Hence the irrelevant ideal of $\mathcal{R}_A(0)$ is zero. A point of Proj would be a homogeneous prime not containing the irrelevant ideal, which is impossible because every prime contains zero. □

2.3.3 Localization compatibility and coherence

Lemma 2.3.109 (Localization commutes with powers of an ideal)**.** *Uses:*

Lean: Ideal.map_localization_pow

Let $s \in A$*. For every* n *the following equality holds*

$$(I^n)A_s = (IA_s)^n.$$

Proof. Uses:

Extension of ideals along a ring homomorphism preserves products and the unit ideal. Induct on n. □

Lemma 2.3.110 (Localization of each Rees piece)**.** *Uses:*

Lean: ReesAlgebra.localization_piece_equiv

Flatness of $A \to A_s$ *gives an* A_s*-linear equivalence*

$$A_s \otimes_A \mathcal{R}_A(I)_n \cong \mathcal{R}_{A_s}(IA_s)_n$$

for every n*.*

Proof. Uses: lemma:nagata-localization-ideal-pow, lemma:nagata-rees-piece-membership

Identify the two modules with $A_s \otimes_A I^n$ and $(IA_s)^n$, then use exactness of localization and Lemma 2.3.109. □

Definition 2.3.111 (Coefficientwise Rees-localization map)**.** Uses: def:nagata-rees-algebra

Lean: ReesAlgebra.localizationHom

Define

$$\rho_s \colon A_s \otimes_A \mathcal{R}_A(I) \longrightarrow \mathcal{R}_{A_s}(IA_s)$$

degreewise by sending a coefficient $x \in I^n$ to $x/1 \in (IA_s)^n$.

Proposition 2.3.112 (Graded localization of the Rees algebra)**.** *Uses:* def:nagata-rees-localization-map

Lean: ReesAlgebra.localizationEquiv

The map ρ_s *is a graded ring equivalence.*

Proof. Uses: lemma:nagata-localization-rees-piece, lemma:nagata-localization-ideal-pow

Multiplicativity follows from convolution of coefficients and Lemma 2.3.109. The inverse is obtained from the universal property of localization. The two composites are identities in every homogeneous degree by Lemma 2.3.110. □

Lemma 2.3.113 (Compatibility with degree zero)**.** *Uses:*

Lean: ReesAlgebra.localizationEquiv_pieceZero

On degree zero, the constructed coefficientwise localization map is the standard ring equivalence $A_s \otimes_A A \cong A_s$*.*

Proof. Uses: prop:nagata-rees-localization, lemma:nagata-rees-degree-zero

Evaluate both maps on a constant monomial and use ring-homomorphism extensionality. □

Lemma 2.3.114 (Compatibility with degree-one generators)**.** *Uses:*

Lean: `ReesAlgebra.localizationEquiv_degreeOneElement`

The element at maps to the degree-one element $(a/1)t$ of the localized Rees algebra.

Proof. Uses: `prop:nagata-rees-localization,lemma:nagata-rees-degree-one-element`

Unfold the degreewise map in Proposition 2.3.112. In degree one it is extension of the coefficient map $I \to IA_s$, so it sends a to $a/1$ and therefore sends at to $(a/1)t$. □

Lemma 2.3.115 (Localization at one)**.** *Uses:*

Lean: `ReesAlgebra.localizationEquiv_one`

After the canonical identifications $A_1 \cong A$, the Rees localization equivalence at 1 is the identity.

Proof. Uses: `prop:nagata-rees-localization,lemma:nagata-rees-sum-components`

Both the localization equivalence and the identity send each homogeneous monomial at^n to itself under $A_1 \cong A$. Use Lemma 2.3.10 and ring-homomorphism extensionality. □

Lemma 2.3.116 (Iterated localization coherence)**.** *Uses:*

Lean: `ReesAlgebra.localizationEquiv_mul`

Localizing first at s and then at t gives the same graded equivalence as localizing at st, after the canonical equivalence $(A_s)_t \cong A_{st}$.

Proof. Uses: `prop:nagata-rees-localization`

Both maps send a homogeneous coefficient $x \in I^n$ to $x/1$ and commute with the inclusions of the localized base ring. Graded ring-homomorphism extensionality gives equality. □

Lemma 2.3.117 (Naturality under a ring isomorphism)**.** *Uses:*

Lean: `ReesAlgebra.localizationEquiv_natural`

The Rees localization equivalence commutes with transport of I and s along a ring isomorphism.

Proof. Uses: `prop:nagata-rees-localization`

It suffices to compare the maps on homogeneous monomials. Both routes send at^n to the monomial with coefficient obtained by transporting a and then localizing it. Naturality of localization of rings gives equality of those coefficients. □

Definition 2.3.118 (Affine blow-up restriction morphism)**.** Uses: `def:nagata-affine-blowup,def:nagata-rees-localization-map`

Lean: `AlgebraicGeometry.affineBlowup_localizationMap`

Define

$$r_s \colon \mathrm{Bl}_{IA_s}(\operatorname{Spec} A_s) \longrightarrow \mathrm{Bl}_I(\operatorname{Spec} A) \times_{\operatorname{Spec} A} \operatorname{Spec} A_s$$

by applying relative Proj to ρ_s and the canonical degree-zero identifications.

Proposition 2.3.119 (Restriction of an affine blow-up)**.** *Uses:* `def:nagata-affine-blowup-restriction-map`

Lean: `AlgebraicGeometry.affineBlowup_localizationIso`

The constructed morphism r_s is an isomorphism over $\operatorname{Spec}(A_s)$.

Proof. Uses: `prop:nagata-rees-localization,lemma:nagata-rees-localization-degree-zero`

By Proposition 2.3.112, the localized graded algebra is the Rees algebra of the localized ideal. Taking Proj gives the comparison. The degree-zero calculation identifies the base spectra, and the structure maps agree on every element of A. □

Lemma 2.3.120 (Identity and cocycle laws for affine restriction)**.** *Uses:*

Lean: `AlgebraicGeometry.affineBlowup_localizationIso_cocycle`

The restriction isomorphism is the identity for $s = 1$, and its two possible composites on an iterated basic open $D(st)$ agree literally after the chosen canonical associativity isomorphisms.

Proof. Uses: `lemma:nagata-rees-localization-one,lemma:nagata-rees-localization-mul,prop:nagata-affine-blowup-restrict`

Apply Proj to Lemma 2.3.115 and Lemma 2.3.116. Record the resulting equality as a named lemma rather than relying on simplification through several chosen isomorphisms. □

2.3.4 Global blow-up by gluing

Definition 2.3.121 (Local blow-up model on an affine open)**.** Uses: def:nagata-affine-blowup

Lean: Scheme.IdealSheafData.localBlowup

Let $\mathcal{I} \subseteq \mathcal{O}_X$ be quasi-coherent and let $V = \operatorname{Spec}(A) \subseteq X$ be affine. Put $I_V = \Gamma(V, \mathcal{I}) \subseteq A$ and define the local model

$$\mathsf{B}_V(\mathcal{I}) = \operatorname{Bl}_{I_V}(V) \longrightarrow V.$$

Definition 2.3.122 (Basic-overlap comparison map)**.** Uses: def:nagata-global-blowup-local-model,def:nagata-affine-blowup-restriction-map

Lean: Scheme.IdealSheafData.localBlowup_overlapBasicMap

If a basic affine open $D(s)$ lies in $V_i \cap V_j$, define

$$\omega_{ij,s} \colon \mathsf{B}_{V_i}(\mathcal{I})|_{D(s)} \longrightarrow \mathsf{B}_{V_j}(\mathcal{I})|_{D(s)}$$

by composing the restriction maps r_s for the two affine presentations of the same localized ideal.

Lemma 2.3.123 (The basic-overlap comparison is an isomorphism)**.** *Uses:* def:nagata-global-blowup-overlap-basic-map

Lean: Scheme.IdealSheafData.localBlowup_overlapBasicIso

The constructed map $\omega_{ij,s}$ is an isomorphism over $D(s)$.

Proof. Uses: prop:nagata-affine-blowup-restrict

Quasi-coherence identifies both restricted ideals with the localization of either affine ideal. Each restriction map is an isomorphism by Proposition 2.3.119; hence their composite is an isomorphism. □

Lemma 2.3.124 (Independence of the basic refinement)**.** *Uses:*

Lean: Scheme.IdealSheafData.localBlowup_overlapBasicIso_independent

On the intersection of two chosen basic refinements, the two overlap constructed maps $\omega_{ij,s}$ agree.

Proof. Uses: lemma:nagata-affine-blowup-restrict-cocycle,lemma:nagata-global-blowup-overlap-basic

Refine once more to a basic open contained in both. Both maps become the canonical iterated-localization map, and equality follows from Lemma 2.3.120. Equality on a cover gives equality of the original morphisms. □

Definition 2.3.125 (Raw global blow-up gluing datum)**.** Uses: def:nagata-global-blowup-local-model,def:nagata-global-blowup-overlap-basic-map

Constructed by: lemma:nagata-global-blowup-overlap-independent

Lean: Scheme.IdealSheafData.blowupRawGlueData

Fix an affine cover $(V_i)_i$ of X. On the inverse image of $V_i \cap V_j$, glue the basic-open maps $\omega_{ij,s}$ along the equality supplied on every pairwise basic refinement. The resulting raw datum $\mathsf{G}(X, \mathcal{I}, (V_i))$ consists of the local schemes $\mathsf{B}_{V_i}(\mathcal{I})$, the inverse-image overlap opens, and these glued overlap maps. No isomorphism or cocycle field is included yet.

Lemma 2.3.126 (The glued overlap map is an isomorphism)**.** *Uses:* def:nagata-global-blowup-glue-data

Lean: Scheme.IdealSheafData.localBlowup_overlapIso_isIso

Every overlap map in $\mathsf{G}(X, \mathcal{I}, (V_i))$ is an isomorphism.

Proof. Uses: lemma:nagata-global-blowup-overlap-basic,lemma:nagata-global-blowup-overlap-independent

The source overlap has the selected basic affine cover used in the raw datum. On each member the map is $\omega_{ij,s}$, which is an isomorphism. The local inverses agree after a common basic refinement because the forward maps agree there. They therefore glue to an inverse of the full overlap map; both inverse equations can be checked on the same cover. □

Lemma 2.3.127 (Identity law for global blow-up overlaps)**.** *Uses:* def:nagata-global-blowup-glue-data

Lean: Scheme.IdealSheafData.localBlowup_overlapIso_refl

The overlap map from chart i to itself is the identity.

Proof. Uses: lemma:nagata-affine-blowup-restrict-cocycle,lemma:nagata-global-blowup-overlap-independent

Restrict to a basic affine member of the overlap cover. Both the self-overlap map and the identity are obtained from localization at 1; the identity part of the affine restriction cocycle makes them equal. Equality of scheme morphisms is local on the source, so the equality holds on the entire overlap. □

Lemma 2.3.128 (Symmetry law for global blow-up overlaps)**.** Uses: def:nagata-global-blowup-glue-data

Lean: Scheme.IdealSheafData.localBlowup_overlapIso_symm

The (j,i) overlap map is inverse to the (i,j) overlap map.

Proof. Uses: lemma:nagata-affine-blowup-restrict-cocycle,lemma:nagata-global-blowup-overlap-independent,lemma:nagata-global-blowup-overlap-map-is-iso

On a common basic affine refinement the two composites are the two localization comparisons followed in opposite orders. The iterated localization equation reduces each composite to localization at 1, hence to the identity. The equalities glue because the basic refinements cover the overlap. The isomorphism certificate identifies the reversed map with the inverse of the forward map. □

Lemma 2.3.129 (Identity and symmetry of overlap isomorphisms)**.** Uses: def:nagata-global-blowup-glue-data

Lean: Scheme.IdealSheafData.localBlowup_overlapIso_refl_symm

The overlap isomorphisms satisfy the identity and symmetry laws.

Proof. Uses: lemma:nagata-global-blowup-overlap-identity,lemma:nagata-global-blowup-overlap-symmetry

The two conjuncts are the two cited atomic laws. □

Lemma 2.3.130 (Triple-overlap cocycle)**.** Uses: def:nagata-global-blowup-glue-data

Lean: Scheme.IdealSheafData.localBlowup_overlapIso_cocycle

On every triple overlap, the composite of the (i,j) and (j,k) overlap isomorphisms equals the (i,k) overlap isomorphism.

Proof. Uses: lemma:nagata-affine-blowup-restrict-cocycle,lemma:nagata-global-blowup-overlap-independent

Cover the triple overlap by basic affine opens refining all three selected covers. Each route is induced by localization from the same Rees algebra. The two routes agree by the iterated-localization equation. Equality on this cover gives equality on the full triple overlap. □

Lemma 2.3.131 (Validity of the global blow-up gluing datum)**.** Uses: def:nagata-global-blowup-glue-data

Lean: Scheme.IdealSheafData.blowupRawGlueData_isGlueData

The raw datum has isomorphism-valued overlap maps satisfying identity, symmetry, and the triple-overlap cocycle.

Proof. Uses: lemma:nagata-global-blowup-overlap-map-is-iso,lemma:nagata-global-blowup-overlap-identity,lemma:nagata-global-blowup-overlap-symmetry,lemma:nagata-global-blowup-overlap-cocycle

Insert the four cited results into the four validity fields, in the order in which they occur in the gluing-data record. □

Definition 2.3.132 (Certified global blow-up gluing datum)**.** Uses: def:nagata-global-blowup-glue-data

Constructed by: lemma:nagata-global-blowup-overlap-map-is-iso,lemma:nagata-global-blowup-overlap-identity,lemma:nagata-global-blowup-overlap-symmetry,lemma:nagata-global-blowup-overlap-cocycle

Lean: Scheme.IdealSheafData.certifiedBlowupGlueData

Define $\mathsf{G}^{\mathrm{cert}}(X,\mathcal{I},(V_i))$ to be the raw datum together with the isomorphism, identity, symmetry, and triple-cocycle proof fields supplied by the four preceding atomic lemmas.

Theorem 2.3.133 (N02 gluing certificate). *Uses:* *def:nagata-certified-global-blowup-glue-data*

Lean: *Scheme.IdealSheafData.n02Certificate*
The displayed certified datum is the typed N02 scheme-gluing certificate.

Proof. Uses: lemma:nagata-global-blowup-glue-data-valid
The validity lemma supplies exactly the proof fields retained by the certified datum. □

Definition 2.3.134 (Global blow-up). Uses: def:nagata-certified-global-blowup-glue-data

Lean: Scheme.IdealSheafData.blowup
Define $\mathrm{Bl}_{\mathcal{I}}(X)$ by applying the scheme-gluing constructor to $\mathsf{G}^{\mathrm{cert}}(X, \mathcal{I}, (V_i))$.

Lemma 2.3.135 (Compatibility of the local structure maps). *Uses:* *def:nagata-global-blowup-overlap-basic-map*

Lean: *Scheme.IdealSheafData.localBlowupMap_compatible*
On every overlap, the two local structure maps to X agree after applying the constructed overlap map ω_{ij}.

Proof. Uses: prop:nagata-affine-blowup-restrict
Check on basic affine refinements. Compatibility is part of Proposition 2.3.119; equality on the chosen basic cover gives equality on the full overlap. □

Definition 2.3.136 (Global blow-up morphism). Uses: def:nagata-global-blowup,def:nagata-global-blowup-local-model

Lean: Scheme.IdealSheafData.blowupMap
Define

$$b_{X,\mathcal{I}} \colon \mathrm{Bl}_{\mathcal{I}}(X) \longrightarrow X$$

by gluing the local structure maps $\mathsf{B}_{V_i}(\mathcal{I}) \to V_i \to X$.

Definition 2.3.137 (Affine-restriction comparison for the global blow-up). Uses: def:nagata-global-blowup-map,def:nagata-global-blowup-local-model

Lean: Scheme.IdealSheafData.preimageAffineMap
If $V = \operatorname{Spec}(A) \subseteq X$ and I_V represents $\mathcal{I}|_V$, define

$$q_V : b_{X,\mathcal{I}}^{-1}(V) \longrightarrow \mathrm{Bl}_{I_V}(\operatorname{Spec} A)$$

by gluing the affine restriction maps on the basic intersections of V with the chosen affine cover of X.

Proposition 2.3.138 (Affine restriction of the global blow-up). *Uses:* *def:nagata-global-blowup-affine-restriction-map*

Lean: *Scheme.IdealSheafData.preimageAffineIso*
The constructed comparison q_V is an isomorphism over V.

Proof. Uses: prop:nagata-affine-blowup-restrict,lemma:nagata-global-blowup-overlap-cocycle,lemma:nagata-global-blowup-local-maps-agree
For affine opens in the chosen gluing cover this is immediate. For an arbitrary affine V, cover its intersections with the chosen charts by basic opens. Each local comparison is an isomorphism by Proposition 2.3.119; they agree by the cocycle and local-map compatibility lemmas, hence glue to an isomorphism. □

Definition 2.3.139 (Canonical common affine refinement). Uses:

Lean: Scheme.AffineCover.commonRefinement
For affine covers $(V_i)_i$ and $(W_j)_j$ of X, let $\mathcal{R}(V, W)$ be the family of all affine opens $R \subseteq X$ for which $R \subseteq V_i \cap W_j$ for some pair (i, j). The datum retains R, one witnessing pair, and the two open-immersion factorizations.

Lemma 2.3.140 (The canonical common refinement covers). *Uses:* *def:nagata-affine-cover-common-refinement*

Lean: *Scheme.AffineCover.commonRefinement_iSup_eq_top*
The members of $\mathcal{R}(V, W)$ cover X.

Proof. Uses:
Given $x \in X$, choose indices i, j with $x \in V_i \cap W_j$. Affine opens form a basis for a scheme, so there is an affine neighbourhood R of x contained in this intersection. This R is a member of the displayed family. Thus every point lies in its union. □

Lemma 2.3.141 (Transitivity of affine restriction comparisons). Uses: `def:nagata-global-blowup-affine-restriction-map`

Lean: `Scheme.IdealSheafData.blowupAffineRestrict_trans`

If $T \subseteq R$ are affine opens and

$$r_{T,R} : \mathrm{Bl}_{I_R}(R)|_T \xrightarrow{\sim} \mathrm{Bl}_{I_T}(T)$$

is the affine restriction isomorphism, then for either affine cover C one has

$$q_T^C = r_{T,R}\big(q_R^C|_T\big).$$

Proof. Uses: `prop:nagata-global-blowup-affine-restrict,lemma:nagata-affine-blowup-restrict-cocycle,lemma:nagata-global-blowup-overlap-independent`

Cover T by opens simultaneously basic in T and R. On every such open, both displayed routes are the same iterated localization map of the Rees algebra. The affine restriction cocycle identifies the two composites, and independence removes the selected basic refinement. Equality of morphisms is local on the source, so the equation holds on all of T. □

Definition 2.3.142 (Local family for the affine-cover comparison). Uses: `def:nagata-affine-cover-common-refinement,def:nagata-global-blowup-affine-restriction-map`

Lean: `Scheme.IdealSheafData.blowupCoverComparisonFamily`

Let B_V and B_W be the two global blow-ups. For $R \in \mathcal{R}(V, W)$ let $q_R^V : B_V|_R \xrightarrow{\sim} \mathrm{Bl}_{I_R}(R)$ and $q_R^W : B_W|_R \xrightarrow{\sim} \mathrm{Bl}_{I_R}(R)$ be the affine restriction comparisons, and set

$$c_R = (q_R^W)^{-1} q_R^V : B_V|_R \longrightarrow B_W|_R.$$

Lemma 2.3.143 (The local comparison is over the target). Uses: `def:nagata-global-blowup-cover-comparison-family`

Lean: `Scheme.IdealSheafData.blowupCoverComparisonFamily_over`

For each R, the two composites of c_R with the structure map to R are equal.

Proof. Uses: `prop:nagata-global-blowup-affine-restrict`

Both affine restriction comparisons are morphisms over R. Composing the equality for q_R^W with its inverse and then with q_R^V gives the asserted equation for c_R. □

Lemma 2.3.144 (Agreement of local cover comparisons). Uses: `def:nagata-global-blowup-cover-comparison-family`

Lean: `Scheme.IdealSheafData.blowupCoverComparisonFamily_overlap`

For $R, S \in \mathcal{R}(V, W)$, the restrictions of c_R and c_S to $R \cap S$ agree.

Proof. Uses: `lemma:nagata-global-blowup-affine-restriction-transitive`

Cover $R \cap S$ by affine opens T contained in both. On T, expand c_R and c_S using their two affine restriction comparisons. The restriction cocycle identifies both expansions with $(q_T^W)^{-1} q_T^V$. Hence they agree on every T, and therefore on the whole intersection. □

Definition 2.3.145 (Certified affine-cover comparison cocone). Uses: `def:nagata-global-blowup-cover-comparison-family`

Constructed by: `lemma:nagata-affine-cover-common-refinement-covers,lemma:nagata-global-blowup-cover-comparison-family-over,lemma:nagata-global-blowup-cover-comparison-family-overlap`

Lean: `Scheme.IdealSheafData.blowupCoverComparisonCocone`

Bundle the local family with its covering proof, overlap equations, and equations over X. This is the cocone accepted by the morphism-gluing constructor.

Definition 2.3.146 (Comparison between two affine-cover constructions). Uses: `def:nagata-global-blowup-cover-comparison-cocone,def:nagata-global-blowup`

Lean: `Scheme.IdealSheafData.blowupCoverComparison`

Define $c_{V,W} : B_V \to B_W$ by gluing the certified local family $(c_R)_{R \in \mathcal{R}(V,W)}$.

Lemma 2.3.147 (The global comparison is over the target). *Uses:* def:nagata-global-blowup-cover-comparison

Lean: Scheme.IdealSheafData.blowupCoverComparison_over

The equality $b_W c_{V,W} = b_V$ holds as morphisms to X.

Proof. Uses: lemma:nagata-global-blowup-cover-comparison-family-over,lemma:nagata-affine-cover-common-refinement-covers

Restrict both sides to the inverse image of each R in the common refinement. The desired equality is the local over-target equation. These opens cover the source, so the global morphisms are equal. □

Lemma 2.3.148 (Restriction formula for the global comparison). *Uses:* def:nagata-global-blowup-cover-comparison

Lean: Scheme.IdealSheafData.blowupCoverComparison_restrict

On every $R \in \mathcal{R}(V,W)$ one has

$$q_R^W (c_{V,W}|_R) = q_R^V.$$

Thus the comparison becomes the identity after identifying both sides with the same affine blow-up.

Proof. Uses: lemma:nagata-global-blowup-cover-comparison-family-overlap

The defining equation of the morphism-gluing constructor says that the restriction is $c_R = (q_R^W)^{-1} q_R^V$. Post-compose with q_R^W and use its inverse equation. □

Lemma 2.3.149 (Inverse equations for affine-cover comparisons). *Uses:* def:nagata-global-blowup-cover-comparison

Lean: Scheme.IdealSheafData.blowupCoverComparison_hom_inv_id

The comparisons obtained after exchanging the two covers satisfy

$$c_{W,V} c_{V,W} = \mathrm{id}_{B_V}, \qquad c_{V,W} c_{W,V} = \mathrm{id}_{B_W}.$$

Proof. Uses: lemma:nagata-global-blowup-cover-comparison-restrict,lemma:nagata-affine-cover-common-refinement-covers

On $R \in \mathcal{R}(V,W)$ the first composite is $(q_R^V)^{-1} q_R^W (q_R^W)^{-1} q_R^V$, hence the identity. The second equation is the same calculation with V, W exchanged. Both equations globalize because the refinement covers. □

Lemma 2.3.150 (The affine-cover comparison is an isomorphism). *Uses:* def:nagata-global-blowup-cover-comparison

Lean: Scheme.IdealSheafData.blowupCoverComparison_isIso

The morphism $c_{V,W}$ is an isomorphism with inverse $c_{W,V}$.

Proof. Uses: lemma:nagata-global-blowup-cover-comparison-inverses

The two inverse equations supply an explicit inverse in the category of schemes. □

Lemma 2.3.151 (Uniqueness of the affine-cover comparison). *Uses:* def:nagata-global-blowup-cover-comparison

Lean: Scheme.IdealSheafData.blowupCoverComparison_unique

If $h : B_V \to B_W$ is over X and satisfies $q_R^W (h|_R) = q_R^V$ on every member of any common affine refinement, then $h = c_{V,W}$.

Proof. Uses: lemma:nagata-global-blowup-cover-comparison-restrict

On each member of the stated refinement, both morphisms equal $(q_R^W)^{-1} q_R^V$. The corresponding inverse-image opens cover B_V. Equality of morphisms is local on the source, so $h = c_{V,W}$. □

Lemma 2.3.152 (Identity and composition of affine-cover comparisons). *Uses:* def:nagata-global-blowup-cover-comparison

Lean: Scheme.IdealSheafData.blowupCoverComparison_id_comp

One has $c_{V,V} = \mathrm{id}_{B_V}$ and, for a third affine cover T, one has $c_{W,T} c_{V,W} = c_{V,T}$.

Proof. Uses: lemma:nagata-global-blowup-cover-comparison-unique,lemma:nagata-global-blowup-cover-comparison-over,lemma:nagata-global-blowup-cover-comparison-restrict

The identity is over X and has the required affine restriction formula, so uniqueness gives the first equation. On an affine open refining all three covers, the second composite reduces to $(q_R^T)^{-1} q_R^W (q_R^W)^{-1} q_R^V = (q_R^T)^{-1} q_R^V$. It is over X by the over-target equations; uniqueness gives the second equation. □

Proposition 2.3.153 (Independence of the affine cover). *Uses:* def:nagata-global-blowup-cover-comparison

Lean: Scheme.IdealSheafData.blowupIso_of_cover

The constructed comparison is the unique isomorphism over X restricting to the identity comparison after affine restriction. It is independent of a cofinal common affine refinement and satisfies identity and composition.

Proof. Uses: lemma:nagata-global-blowup-cover-comparison-over, lemma:nagata-global-blowup-cover-comparison-restrict, lemma:nagata-global-blowup-cover-comparison-is-iso, lemma:nagata-global-blowup-cover-comparison-unique, lemma:nagata-global-blowup-cover-comparison-id-comp

The five cited atomic results give, respectively, the over-target equation, the common-refinement formula, the explicit inverse, uniqueness, and the two coherence equations. Any cofinal refinement has the same local formula, so uniqueness also gives independence of that refinement. □

Theorem 2.3.154 (Global blow-up gluing package). *Uses:* def:nagata-global-blowup-cover-comparison

Lean: Scheme.IdealSheafData.blowupGluingPackage

The affine local blow-up models and their overlap maps assemble to a scheme, a structure morphism to X, and a cover-independent global blow-up construction.

Proof. Uses: def:nagata-global-blowup-glue-data, lemma:nagata-global-blowup-glue-data-valid, def:nagata-global-blowup, lemma:nagata-global-blowup-local-maps-agree, def:nagata-global-blowup-map, def:nagata-global-blowup-cover-comparison, prop:nagata-global-blowup-cover-independent

The raw datum is the finite affine-cover family of affine blow-up schemes together with the overlap isomorphisms constructed on basic affine refinements. The gluing-validity lemma proves that these isomorphisms satisfy identity, inverse, and triple-cocycle laws, so the scheme-gluing constructor applies and gives $\mathrm{Bl}_{\mathcal{I}}(X)$. Compatibility of the local affine structure maps supplies the cocone required to glue them to $b_{X,\mathcal{I}} : \mathrm{Bl}_{\mathcal{I}}(X) \to X$. For two affine covers, the comparison is glued from the identity maps on a common affine refinement; affine restriction identifies these local maps with isomorphisms, and the cover-independence proposition proves uniqueness over X. These are exactly the raw datum, gluing certificate, assembled scheme, structure cocone, assembled structure morphism, comparison cocone, and assembled cover-comparison fields. □

2.3.5 Complement, properness, and principalization

Lemma 2.3.155 (The center ideal is the unit ideal off its support). *Uses:*

Lean: Scheme.IdealSheafData.restrict_compl_support_eq_top

If $Z = V(\mathcal{I})$ and $U = X \setminus Z$, then $\mathcal{I}|_U = \mathcal{O}_U$.

Proof. Uses:

At a point outside $V(\mathcal{I})$, the stalk $\mathcal{I}_x$ is not contained in the maximal ideal of $\mathcal{O}_{X,x}$, so it contains a unit and equals $\mathcal{O}_{X,x}$. Equality of quasi-coherent ideals can be checked on stalks. □

Lemma 2.3.156 (Affine complement-of-center isomorphism). *Uses:*

Lean: AlgebraicGeometry.affineBlowup_compl_zeroLocus_iso

For $Z = V(I) \subseteq \mathrm{Spec}(A)$, the affine blow-up morphism restricts to an isomorphism over $\mathrm{Spec}(A) \setminus Z$.

Proof. Uses: lemma:nagata-affine-blowup-top, prop:nagata-affine-blowup-restrict, lemma:nagata-affine-blowup-restrict-cocycle

Cover the complement by basic opens $D(a)$ with $a \in I$. On $D(a)$ the localized ideal is the unit ideal. Apply Proposition 2.3.119 and Lemma 2.3.107. The local isomorphisms agree on overlaps by Lemma 2.3.120. □

Theorem 2.3.157 (Complement of the center). *Uses:*

Lean: Scheme.IdealSheafData.blowup_compl_support_iso

Let $U = X \setminus V(\mathcal{I})$. The restriction

$$b_{X,\mathcal{I}}^{-1}(U) \longrightarrow U$$

is an isomorphism.

Proof. Uses: prop:nagata-global-blowup-affine-restrict, lemma:nagata-affine-blowup-complement

Cover X by affine opens. The assertion becomes Lemma 2.3.156 under the affine restriction isomorphism. Isomorphisms are local on source and target. □

Definition 2.3.158 (Homogeneous polynomial map to the Rees algebra)**.** Uses: `def:nagata-rees-algebra`

Lean: `ReesAlgebra.gradedPolynomialMap`

If $I = (f_0, \dots, f_r)$, define the A-algebra map

$$\rho_I \colon A[T_0, \dots, T_r] \longrightarrow \mathcal{R}_A(I)$$

by $\rho_I(T_i) = f_i t$, with each T_i placed in degree one.

Lemma 2.3.159 (Properties of the homogeneous polynomial map)**.** *Uses:* *def:nagata-rees-polynomial-map*

Lean: *ReesAlgebra.gradedPolynomial_surjective*

The constructed map ρ_I is graded and surjective.

Proof. Uses: `lemma:nagata-rees-degree-one-element,prop:nagata-rees-finite-generation`

The assignment is homogeneous because $f_i t$ lies in degree one by Lemma 2.3.14. A monomial $T_{i_1} \cdots T_{i_n}$ maps to $(f_{i_1} \cdots f_{i_n}) t^n$. Such products generate I^n, so every degree component is hit. Every Rees element is a finite sum of homogeneous components, proving surjectivity. □

Definition 2.3.160 (Projective-space map of an affine blow-up)**.** Uses: `def:nagata-affine-blowup,def:nagata-rees-polynomial-map`

Lean: `AlgebraicGeometry.affineBlowupToProjectiveSpace`

Define

$$e_I \colon \operatorname{Bl}_I(\operatorname{Spec} A) \longrightarrow \mathbf{P}^r_A$$

to be the contravariant Proj morphism induced by ρ_I.

Lemma 2.3.161 (Closed immersion into relative projective space)**.** *Uses:* *def:nagata-affine-blowup-projective-embedding*

Lean: *AlgebraicGeometry.affineBlowup_toProjectiveSpace_isClosedImmersion*

The constructed morphism e_I is a closed immersion over $\operatorname{Spec}(A)$.

Proof. Uses: `lemma:nagata-rees-polynomial-surjection`

A surjective graded algebra map induces a closed immersion in the opposite direction on Proj. The source of ρ_I has Proj $\mathbf{P}^r_A$, and its degree-zero map is the identity of A, so the closed immersion is over $\operatorname{Spec}(A)$. □

Proposition 2.3.162 (Projectivity of the affine blow-up)**.** *Uses:*

Lean: *AlgebraicGeometry.affineBlowup_isProjective*

If I is finitely generated, the affine blow-up morphism

$$\operatorname{Bl}_I(\operatorname{Spec}(A)) \longrightarrow \operatorname{Spec}(A)$$

is projective.

Proof. Uses: `lemma:nagata-affine-blowup-projective-embedding`

Choose finitely many generators of I. By Lemma 2.3.161, the blow-up is a closed subscheme of a finite-dimensional projective space over $\operatorname{Spec}(A)$. The composite of a closed immersion with $\mathbf{P}^r_A \to \operatorname{Spec}(A)$ is projective. □

Lemma 2.3.163 (Properness of the affine blow-up)**.** *Uses:*

Lean: *AlgebraicGeometry.affineBlowup_isProper*

If I is finitely generated, the affine blow-up morphism is proper.

Proof. Uses: `prop:nagata-affine-blowup-projective`

It is projective by Proposition 2.3.162, and every projective morphism is proper. □

Lemma 2.3.164 (Local properness of the global blow-up)**.** *Uses:*

Lean: *Scheme.IdealSheafData.blowupMap_restrict_isProper*

If $\mathcal{I}$ is of finite type, the restriction of $b_{X,\mathcal{I}}$ over every affine open of X is proper.

Proof. Uses: `prop:nagata-global-blowup-affine-restrict,lemma:nagata-affine-blowup-proper`

Finite type of $\mathcal{I}$ gives finite generation of the corresponding ideal on the affine open. Transfer Lemma 2.3.163 through Proposition 2.3.138. □

Theorem 2.3.165 (Projectivity of the global blow-up). *Uses:*

Lean: `Scheme.IdealSheafData.blowupMap_isProjective`

If $\mathcal{I}$ is of finite type, then $b_{X,\mathcal{I}}\colon \mathrm{Bl}_{\mathcal{I}}(X) \to X$ is projective.

Proof. Uses: `prop:nagata-global-blowup-affine-restrict,prop:nagata-affine-blowup-projective`

On an affine open $W = \mathrm{Spec}(A) \subseteq X$, finite type of $\mathcal{I}$ gives a finite generating set for $\mathcal{I}(W)$. The restriction of the global blow-up is the affine blow-up by Proposition 2.3.138, and is projective by Proposition 2.3.162. Projectivity is local on the target, so these local projective structures prove the global assertion. □

Theorem 2.3.166 (Properness of the global blow-up). *Uses:*

Lean: `Scheme.IdealSheafData.blowupMap_isProper`

If $\mathcal{I}$ is of finite type, then $b_{X,\mathcal{I}}$ is proper.

Proof. Uses: `thm:nagata-global-blowup-projective`

By Theorem 2.3.165, the morphism is projective. Every projective morphism is proper. □

Lemma 2.3.167 (Principalization on one affine chart). *Uses:* `def:nagata-blowup-chart-ring`

Lean: `ReesAlgebra.map_ideal_blowupChartRing_eq_span`

In $A[I/a]$, the extended ideal satisfies

$$I\,A[I/a] = (a).$$

Proof. Uses:

For $x \in I$, the identity $x = a(x/a)$ gives $IA[I/a] \subseteq (a)$. Since $a \in I$, the reverse inclusion is immediate. □

Lemma 2.3.168 (Regularity of the chart generator). *Uses:* `def:nagata-blowup-chart-ring`

Lean: `ReesAlgebra.isRegular_algebraMap_blowupChartRing`

Multiplication by a on $A[I/a]$ is injective.

Proof. Uses:

The chart algebra is a subring of A_a. The image of a is a unit in A_a, hence multiplication by it is injective on the subring. □

Lemma 2.3.169 (Transition between local exceptional generators). *Uses:*

Lean: `ReesAlgebra.chartGeneratorsAssociated_on_overlap`

On the overlap of the a-chart and the b-chart, the two generators a and b of the extended ideal differ by a unit.

Proof. Uses: `lemma:nagata-chart-extended-ideal-principal`

On the overlap, both (a) and (b) equal the same extended ideal. More explicitly, $b = a(b/a)$ and $a = b(a/b)$, and $(a/b)(b/a) = 1$. □

Theorem 2.3.170 (Principalization of the center ideal). *Uses:*

Lean: `Scheme.IdealSheafData.extendedToBlowup_isInvertible`

On $\mathrm{Bl}_{\mathcal{I}}(X)$, the extended ideal $\mathcal{I}\mathcal{O}_{\mathrm{Bl}_{\mathcal{I}}(X)}$ is locally principal, generated locally by a non-zero-divisor, and hence is an invertible ideal defining an effective Cartier divisor.

Proof. Uses: `prop:nagata-affine-blowup-chart-cover,lemma:nagata-chart-extended-ideal-principal,lemma:nagata-chart-generator-regular,lemma:nagata-chart-generators-associated`

Work on the finite affine chart cover associated to local generators of $\mathcal{I}$. Use Lemma 2.3.167 and Lemma 2.3.168. The local principal ideals glue because their generators differ by units by Lemma 2.3.169. □

2.3.6 Universal property, flat base change, and products of ideals

Definition 2.3.171 (Graded map induced by an invertible quotient)**.** Uses:

Lean: `Scheme.IdealSheafData.reesToTensorPowersConstruction`

Let $f\colon Y \to X$, and let $f^*\mathcal{I} \twoheadrightarrow \mathcal{L}$ be the image quotient onto an invertible ideal. Define α_f degreewise by the tensor powers of this quotient:

$$\alpha_f\colon f^*\Big(\bigoplus_{n\geq 0}\mathcal{I}^n\Big) \longrightarrow \bigoplus_{n\geq 0}\mathcal{L}^{\otimes n}.$$

Lemma 2.3.172 (The quotient construction is a graded algebra map)**.** Uses: *def:nagata-blowup-universal-graded-map*

Lean: *Scheme.IdealSheafData.reesToTensorPowers*

The degreewise map α_f respects the unit and multiplication and is a graded algebra morphism.

Proof. Uses:

Tensor the quotient map n times and factor through the image $f^*\mathcal{I}^n$. Compatibility with multiplication follows from associativity of tensor products and multiplication of ideal powers. □

Definition 2.3.173 (Universal lift construction)**.** Uses: `def:nagata-global-blowup`, `def:nagata-blowup-universal-graded-map`

Lean: `Scheme.IdealSheafData.liftToBlowupConstruction`

Define

$$\widetilde{f}\colon Y \longrightarrow \mathrm{Bl}_{\mathcal{I}}(X)$$

by applying the local Proj construction to α_f on affine opens and gluing the resulting local maps.

Lemma 2.3.174 (The constructed lift lies over X)**.** Uses: *def:nagata-blowup-universal-lift*

Lean: *Scheme.IdealSheafData.lift_blowup_fac*

The constructed map $\widetilde{f}$ is an X-morphism: its composite with $b_{X,\mathcal{I}}$ is f.

Proof. Uses: `lemma:nagata-blowup-universal-graded-map`, `lemma:nagata-global-blowup-glue-data-valid`, `lemma:nagata-global-blowup-local-maps-agree`

The local Proj maps are defined by the graded morphism of Lemma 2.3.172. They agree on affine overlaps and glue by the validity and local-map compatibility of the global blow-up datum. The factorization to X holds on every affine member of the cover and hence globally. □

Lemma 2.3.175 (Uniqueness of the universal lift)**.** Uses: *def:nagata-blowup-universal-lift*

Lean: *Scheme.IdealSheafData.lift_blowup_unique*

Every X-morphism $h\colon Y \to \mathrm{Bl}_{\mathcal{I}}(X)$ inducing the chosen invertible quotient equals the constructed map $\widetilde{f}$.

Proof. Uses:

Affine-locally, both maps to Proj are represented by the same homogeneous quotient, so they agree on every standard chart. The chart cover is jointly surjective, hence the local equalities imply $h = \widetilde{f}$. □

Theorem 2.3.176 ([1, Tag 0806])**.** Uses: *def:nagata-blowup-universal-lift*

Lean: *Scheme.IdealSheafData.existsUnique_lift_blowup*

The constructed map $\widetilde{f}$ is the unique X-morphism from Y to $\mathrm{Bl}_{\mathcal{I}}(X)$ inducing the chosen invertible quotient. Thus the blow-up is final among such X-schemes.

Proof. Uses: `lemma:nagata-blowup-universal-factorization`, `lemma:nagata-blowup-universal-unique`

The factorization property is Lemma 2.3.174; uniqueness is Lemma 2.3.175. □

Corollary 2.3.177 (Blow-up of an effective Cartier center)**.** Uses:

Lean: *Scheme.IdealSheafData.blowupMap_isIso_of_isInvertible*

If $\mathcal{I}$ is already invertible on X, then $b_{X,\mathcal{I}}$ is an isomorphism.

Proof. Uses: `thm:nagata-blowup-principalizes-center`, `thm:nagata-blowup-universal-property`

The identity of X lifts to the blow-up by the universal property. Compose this lift with the blow-up map in both orders and use uniqueness. □

Lemma 2.3.178 (Flat pullback commutes with ideal powers)**.** *Uses:*

Lean: `IdealSheafData.flat_pullback_pow`

If $g\colon Y \to X$ is flat, then the image of $g^*(\mathcal{I}^n)$ in $\mathcal{O}_Y$ is $(\mathcal{I}\mathcal{O}_Y)^n$ for every n.

Proof. Uses:

Flatness preserves the injections $\mathcal{I}^n \hookrightarrow \mathcal{O}_X$. Tensor products commute with multiplication of the image ideals. Check the statement on affine opens. □

Definition 2.3.179 (Flat pullback map for Rees algebras)**.** Uses: `def:nagata-rees-algebra`

Lean: `IdealSheafData.flatPullbackReesMap`

For flat $g\colon Y \to X$, define β_g degreewise by the canonical maps

$$g^*(\mathcal{I}^n) \longrightarrow (\mathcal{I}\mathcal{O}_Y)^n.$$

Thus

$$\beta_g\colon g^*\mathcal{R}_X(\mathcal{I}) \longrightarrow \mathcal{R}_Y(\mathcal{I}\mathcal{O}_Y).$$

Lemma 2.3.180 (Flat pullback of the Rees algebra)**.** *Uses:* `def:nagata-flat-pullback-rees-map`

Lean: `IdealSheafData.flat_pullback_rees_equiv`

The map β_g is a graded algebra equivalence.

Proof. Uses: `lemma:nagata-flat-pullback-ideal-pow`

Apply Lemma 2.3.178 in every degree and check compatibility with multiplication. The degreewise isomorphisms assemble to the claimed graded equivalence. □

Definition 2.3.181 (Flat base-change comparison for blow-ups)**.** Uses: `def:nagata-global-blowup,def:nagata-flat-pullback-rees-map`

Lean: `Scheme.IdealSheafData.blowup_flatBaseChangeMap`

Define

$$\mathrm{bc}_g\colon \mathrm{Bl}_{\mathcal{I}\mathcal{O}_Y}(Y) \longrightarrow Y \times_X \mathrm{Bl}_{\mathcal{I}}(X)$$

by applying relative Proj to β_g on affine opens and gluing.

Theorem 2.3.182 (Flat base change of blow-ups)**.** *Uses:* `def:nagata-blowup-flat-base-change-map`

Lean: `Scheme.IdealSheafData.blowup_flatBaseChangeIso`

The constructed comparison bc_g is an isomorphism over Y.

Proof. Uses: `lemma:nagata-flat-pullback-rees,lemma:nagata-global-blowup-overlap-cocycle`

Relative Proj sends the graded equivalence of Lemma 2.3.180 to an affine-local isomorphism. Compatibility with degree zero identifies the structure maps, and the localization cocycle glues the local isomorphisms. □

Lemma 2.3.183 (Product of invertible ideals is invertible)**.** *Uses:*

Lean: `Scheme.IdealSheafData.IsInvertible.mul`

The product of two invertible ideal sheaves is invertible.

Proof. Uses:

On an open on which the two ideals are generated by regular elements a and b, their product is generated by ab, which is regular. □

Lemma 2.3.184 (Affine factorization of an invertible product)**.** *Uses:*

Lean: `Ideal.exists_basicCover_isPrincipal_regular_of_mul_eq_span_regular`

Let A be a ring, let $I, J \subseteq A$ be ideals, and suppose $IJ = (f)$ for a non-zero-divisor $f \in A$. There are finite families $a_r \in I$, $b_r \in J$, and $c_r \in A$ such that

$$f = \sum_{r=1}^{N} a_r b_r, \qquad a_r b_r = c_r f, \qquad \sum_{r=1}^{N} c_r = 1.$$

Consequently the basic opens $D(c_r)$ cover $\mathrm{Spec}(A)$, and on A_{c_r} one has

$$I_{c_r} = (a_r), \qquad J_{c_r} = (b_r),$$

with both a_r and b_r non-zero-divisors.

Proof. Uses:

Membership of f in the product ideal gives a finite expression $f = \sum_r a_r b_r$ with $a_r \in I$ and $b_r \in J$. Since every $a_r b_r$ belongs to $IJ = (f)$, choose c_r with $a_r b_r = c_r f$. Then

$$\left(1 - \sum_r c_r\right) f = 0.$$

Regularity of f gives $\sum_r c_r = 1$, so the $D(c_r)$ cover the spectrum.

Fix r and localize at c_r. The element c_r is now a unit, hence $(a_r b_r) = (f) = I_{c_r} J_{c_r}$. In particular $a_r b_r$ is regular, so each factor is regular. For $x \in I_{c_r}$ we have $x b_r \in I_{c_r} J_{c_r} = (a_r b_r)$; write $x b_r = d a_r b_r$ and cancel the regular element b_r to obtain $x = d a_r$. Thus $I_{c_r} = (a_r)$. Interchanging the factors gives $J_{c_r} = (b_r)$. □

Lemma 2.3.185 (Transition-unit cocycle for the two factors)**.** Uses:

Lean: `Ideal.regularGeneratorTransitionCocycle`

Let A be a ring, let $I, J \subseteq A$ be ideals, and let a_r, b_r, c_r be finite families such that the $D(c_r)$ cover $\operatorname{Spec}(A)$ *and, on A_{c_r},*

$$I_{c_r} = (a_r), \qquad J_{c_r} = (b_r),$$

with a_r and b_r regular. On every overlap $D(c_r c_s)$ there are unique units $u_{rs}, v_{rs} \in A_{c_r c_s}^{\times}$ satisfying

$$a_r = u_{rs} a_s, \qquad b_r = v_{rs} b_s.$$

They satisfy the identity and triple-overlap equations

$$u_{rr} = v_{rr} = 1, \qquad u_{rt} = u_{rs} u_{st}, \qquad v_{rt} = v_{rs} v_{st}.$$

Proof. Uses:

On the overlap, a_r and a_s are regular generators of the same ideal. Write $a_r = u_{rs} a_s$ and $a_s = u_{sr} a_r$. Cancellation of the regular generator gives $u_{rs} u_{sr} = 1$, so u_{rs} is a unit; regularity also makes it unique. The same argument gives v_{rs}. On a triple overlap, the two expressions $a_r = u_{rt} a_t$ and $a_r = u_{rs} u_{st} a_t$ are equal, and cancellation of a_t proves the cocycle equation. The remaining equations are identical. □

Lemma 2.3.186 ([1, Tag 07ZV])**.** Uses:

Lean: `Scheme.IdealSheafData.IsInvertible.of_mul`

If the product $\mathcal{I}\mathcal{J}$ defines an effective Cartier divisor, then each of $\mathcal{I}$ and $\mathcal{J}$ defines an effective Cartier divisor.

Proof. Uses: `lemma:nagata-affine-invertible-product-factor,lemma:nagata-invertible-product-transition-cocycle`

Work on an affine open $V = \operatorname{Spec}(A)$ on which $\mathcal{I}\mathcal{J} = (f)$ with f regular. Apply Lemma 2.3.184. The resulting finite basic cover $V = \bigcup_r D(c_r)$ has regular generators a_r for $\mathcal{I}$ and b_r for $\mathcal{J}$. Hence multiplication by a_r and b_r identifies each restricted ideal with the structure sheaf.

By Lemma 2.3.185, these trivializations differ on overlaps by units satisfying the identity and triple cocycles. They therefore glue to invertible ideal sheaves, and the regularity of every local generator identifies both with effective Cartier divisor ideals. Refining the affine cover on which $\mathcal{I}\mathcal{J}$ is Cartier proves the assertion globally. □

Definition 2.3.187 (Comparison maps for the product-ideal blow-up)**.** Uses: `def:nagata-blowup-universal-lift,def:nagata-global-blowup`

Lean: `Scheme.IdealSheafData.iteratedBlowupComparison`

Let $X_{\mathcal{I},\mathcal{J}}$ be the scheme obtained by first blowing up $\mathcal{I}$ and then the extended ideal $\mathcal{J}$. On this iterated blow-up the two extended ideals are invertible; define

$$\Phi_{\mathcal{I},\mathcal{J}} \colon X_{\mathcal{I},\mathcal{J}} \longrightarrow \operatorname{Bl}_{\mathcal{I}\mathcal{J}}(X)$$

by the universal-lift construction for their product. On $\operatorname{Bl}_{\mathcal{I}\mathcal{J}}(X)$, factor the invertible product first through the $\mathcal{I}$-blow-up and then through the extended $\mathcal{J}$-blow-up, and define the resulting map to be

$$\Psi_{\mathcal{I},\mathcal{J}} \colon \operatorname{Bl}_{\mathcal{I}\mathcal{J}}(X) \longrightarrow X_{\mathcal{I},\mathcal{J}}.$$

Theorem 2.3.188 ([1, Tag 080A])**.** Uses: `def:nagata-blowup-product-comparison`

Lean: `Scheme.IdealSheafData.iteratedBlowupIso_mul`

The constructed maps $\Phi_{\mathcal{I},\mathcal{J}}$ and $\Psi_{\mathcal{I},\mathcal{J}}$ are inverse over X. Thus the iterated blow-up is identified with the blow-up of $\mathcal{I}\mathcal{J}$.

Proof. Uses: thm:nagata-blowup-universal-property,lemma:nagata-invertible-ideal-mul,lemma:nagata-invertible-product-factor

The product of the two extended ideals on the iterated blow-up is invertible, so Φ is defined. Conversely, invertibility of $\mathcal{I}\mathcal{J}$ on its blow-up implies invertibility of each factor by Lemma 2.3.186, which defines Ψ. Both composites are morphisms over X pulling back the relevant universal invertible quotients to the same quotients. Uniqueness in the blow-up universal property makes the composites identities. □

Definition 2.3.189 (Domination maps from the product blow-up)**.** Uses: def:nagata-blowup-product-comparison

Lean: Scheme.IdealSheafData.blowupMulDominationMaps

Define

$$d_{\mathcal{I}} \colon \mathrm{Bl}_{\mathcal{I}\mathcal{J}}(X) \longrightarrow \mathrm{Bl}_{\mathcal{I}}(X)$$

as $\Psi_{\mathcal{I},\mathcal{J}}$ followed by the first blow-up projection. Define $d_{\mathcal{J}}$ symmetrically after interchanging $\mathcal{I}$ and $\mathcal{J}$.

Corollary 2.3.190 (Common refinement by the product ideal)**.** *Uses:* def:nagata-blowup-product-domination-maps

Lean: Scheme.IdealSheafData.blowupMulToLeft

The constructed $d_{\mathcal{I}}$ and $d_{\mathcal{J}}$ are morphisms over X. If both ideals restrict to the unit ideal on an open U, the product blow-up and both maps restrict to the identity over U.

Proof. Uses: thm:nagata-blowup-product-ideal,thm:nagata-global-blowup-complement

The maps are composites over X by definition. On U, all three ideals are the unit ideal, so all three blow-ups are identified with U by the complement theorem. The universal maps are unique over U, hence become the identity under these identifications. □

2.3.7 An arbitrary composition of two blow-ups

The product-ideal theorem treats a second center pulled back from the original scheme. Stability of admissible blow-ups under composition needs the more general statement in which the second center is an arbitrary finite-presentation closed subscheme of the first blow-up.

Definition 2.3.191 (Finite-degree data attached to a closed center in Proj)**.** Uses:

Lean: AlgebraicGeometry.Proj.finiteDegreeCenterData

Let $\mathcal{A} = \bigoplus_{n\geq 0} \mathcal{I}^n$, let $X' = \mathrm{Proj}_X(\mathcal{A})$, and let $Z' \hookrightarrow X'$ be of finite presentation. On a fixed finite standard affine cover, choose finite homogeneous generating lists for the ideal of Z'. Let d_0 be a common positive multiple of their degrees, homogenize every generator to degree d_0, and let $\mathcal{F}_0 \subseteq \mathcal{I}^{d_0}$ be the finite type submodule of degree-d_0 coefficients. Let d be the least positive multiple of d_0 for which the finite type quotient outside the desired support vanishes, and define $\mathcal{F} \subseteq \mathcal{I}^d$ by multiplying $\mathcal{F}_0$ by $\mathcal{I}^{d-d_0}$. Denote this pair by $\mathsf{FD}(Z') = (d(Z'), \mathcal{F}(Z'))$.

Lemma 2.3.192 (Properties of the finite-degree center data)**.** *Uses:* def:nagata-proj-closed-center-finite-degree

Lean: AlgebraicGeometry.Proj.exists_finiteDegree_submodule_defining_closed

The selected integer $d(Z')$ is positive, $\mathcal{F}(Z')$ is a finite type submodule of $\mathcal{I}^{d(Z')}$, and

$$Z' = \mathrm{Proj}_X\big(\mathcal{A}/\mathcal{F}(Z')\mathcal{A}\big).$$

Moreover the support of $\mathcal{I}^{d(Z')}/\mathcal{F}(Z')$ lies over the image of Z' together with the first center.

Proof. Uses:

On each chosen standard chart, homogenization only multiplies an equation by a power of the invertible chart coordinate and hence does not change its zero scheme. The finite homogeneous generator lists therefore cut out the same closed subscheme after passage to the common degree. Finite type is preserved under multiplication by the finite type ideal power. The final selected multiple is precisely one at which the finite type quotient vanishes off the specified closed support, giving the support assertion. □

Theorem 2.3.193 (Finite-degree center-data package)**.** *Uses:* def:nagata-proj-closed-center-finite-degree

Lean: AlgebraicGeometry.Proj.finiteDegreeCenterPackage

The finite-degree Proj center data provides the selected degree, the finite type submodule defining the closed center, the equality with the Proj closed subscheme, and the support bound used by the composite blow-up theorem.

Proof. Uses: lemma:nagata-proj-closed-center-finite-degree
The lemma already proves positivity of the degree, finite type of the selected submodule, the Proj equality, and the support bound. The theorem is the typed packaging of exactly those four assertions into the composite center interface. □

Definition 2.3.194 (Center ideal representing a second blow-up)**.** Uses: def:nagata-proj-closed-center-finite-degree

Lean: AlgebraicGeometry.Blowup.compositionCenterIdealConstruction
With $\mathsf{FD}(Z') = (d, \mathcal{F})$, define

$$\mathcal{J}(Z') = \mathcal{F} \subseteq \mathcal{I}^d \subseteq \mathcal{O}_X, \qquad \mathcal{K}(Z') = \mathcal{I}\mathcal{J}(Z'), \qquad Y(Z') = V(\mathcal{K}(Z')).$$

Lemma 2.3.195 (Support of the constructed composition center)**.** *Uses:* def:nagata-composition-center-ideal

Lean: *AlgebraicGeometry.Blowup.compositionCenterIdeal*
The ideal $\mathcal{K}(Z')$ is finite type and

$$|Y(Z')| = |V(\mathcal{I})| \cup b(|Z'|).$$

Proof. Uses: lemma:nagata-proj-closed-center-finite-degree,thm:nagata-global-blowup-complement
Outside $V(\mathcal{I})$, the first blow-up is an isomorphism and the finite-degree Proj description says that $\mathcal{J}(Z')$ cuts out exactly the image of Z'. Since $\mathcal{J}(Z') \subseteq \mathcal{I}^d$, its vanishing set contains the first center. Thus $V(\mathcal{J}(Z')) = V(\mathcal{I}) \cup b(|Z'|)$; multiplication by $\mathcal{I}$ does not change this union. Finite type is preserved under finite products. □

Lemma 2.3.196 (Comparison of the two second-center ideals)**.** *Uses:*

Lean: *AlgebraicGeometry.Blowup.extended_compositionCenterIdeal*
Let E be the exceptional divisor of the first blow-up and let $\mathcal{I}_{Z'}$ define the second center. On X' one has

$$b^{-1}\mathcal{J}\,\mathcal{O}_{X'} = \mathcal{I}_E^d \mathcal{I}_{Z'}.$$

Proof. Uses: lemma:nagata-composition-center-ideal,lemma:nagata-chart-extended-ideal-principal
Work on the affine blow-up chart $A[I/a]$. If $m \in \mathcal{F}$ maps to $f \in J$, the degree-d equation for Z' is m/a^d. In the chart ring,

$$f = a^d(m/a^d).$$

The factor (a^d) is the dth power of the exceptional ideal by Lemma 2.3.167; the remaining factors generate the ideal of Z'. These equalities agree on chart overlaps. □

Definition 2.3.197 (Comparison for a composite of arbitrary blow-ups)**.** Uses: def:nagata-composition-center-ideal,def:nagata-blowup-product-comparison,def:nagata-global-blowup

Lean: AlgebraicGeometry.Blowup.compComparisonBlowup
Let $X' \to X$ be the blow-up of $\mathcal{I}$, and let the second blow-up be centred at $Z' \hookrightarrow X'$. Form the explicit ideal $\mathcal{K}(Z') = \mathcal{I}\mathcal{J}(Z')$ on X. The extended ideal on X' is $\mathcal{I}_E^d \mathcal{I}_{Z'}$. Define

$$\Omega_{Z,Z'} \colon \mathrm{Bl}_{\mathcal{I}_{Z'}}(X') \longrightarrow \mathrm{Bl}_{\mathcal{K}(Z')}(X)$$

as the product-blow-up comparison followed by the comparison which removes the invertible factor $\mathcal{I}_E^d$. Define the reverse map by the corresponding universal lifts.

Theorem 2.3.198 ([1, Tag 080B])**.** *Uses:* def:nagata-composition-of-blowups-comparison

Lean: *AlgebraicGeometry.Blowup.compIsoBlowup*
The two constructed comparison maps are inverse over X. Hence the composite is the blow-up of the finite-presentation center $Y(Z') = V(\mathcal{K}(Z')) \hookrightarrow X$, whose support satisfies

$$|Y(Z')| = |Z| \cup b(|Z'|).$$

Proof. Uses: lemma:nagata-composition-center-ideal,lemma:nagata-composition-extended-ideal-equality,thm:nagata-blowup-product-ideal,cor:nagata-blowup-cartier-iso
By Lemma 2.3.196, the pullback of $\mathcal{J}(Z')$ is $\mathcal{I}_E^d \mathcal{I}_{Z'}$. The factor $\mathcal{I}_E^d$ is invertible, and Corollary 2.3.177 identifies blowing up this product with blowing up $\mathcal{I}_{Z'}$. The product-ideal comparison then identifies the resulting iterated blow-up with the blow-up of $\mathcal{I}\mathcal{J}(Z') = \mathcal{K}(Z')$. At every step the explicitly defined reverse map is the universal inverse, so uniqueness proves the two composites are identities. The support formula is Lemma 2.3.195. □

2.4 Admissible blow-ups and strict transforms

Definition 2.4.1 (U-admissible blow-up, [1, Tag 080K]). Uses:

Lean: `AlgebraicGeometry.AdmissibleBlowup`

Let $U \hookrightarrow X$ be a quasi-compact open immersion. A morphism $\pi\colon X' \to X$ is a *U-admissible blow-up* if there is a closed immersion $C \hookrightarrow X$ of finite presentation, disjoint from U, such that X' is the blow-up of X in C. Equivalently, the center is defined by a finite type quasi-coherent ideal $\mathcal{I}$ satisfying $\mathcal{I}|_U = \mathcal{O}_U$. The bundled data should contain this ideal, its finite type proof, a chosen lift $U \to X'$, the factorization equality, and the assertion that $X' \times_X U \to U$ is an isomorphism.

Definition 2.4.2 (Strict transform). Uses: `def:nagata-scheme-theoretic-image`

Lean: `AlgebraicGeometry.StrictTransform`

Let $Z \to X$ be a morphism and let $X' \to X$ be the blow-up of a center $C \subseteq X$. Put $U = X \setminus C$. The *strict transform* of Z is the scheme-theoretic closure of

$$(Z \times_X U) \times_U (X' \times_X U)$$

in $Z \times_X X'$. It is not enough to use only the topological closure.

2.4.1 Power torsion and strict-transform charts

The strict transform is the correction term for the failure of blow-ups to commute with non-flat base change. The following algebraic lemmas make that correction explicit before it is sheafified and glued.

Definition 2.4.3 (Power-torsion submodule). Uses:

Lean: `Submodule.powTorsion`

Let R be a ring, let M be an R-module, and let $a \in R$. Define

$$\Gamma_{(a)}(M) = \{m \in M \mid \text{there is } n \geq 0 \text{ with } a^n m = 0\}.$$

Lemma 2.4.4 (Power torsion is a submodule). Uses: `def:nagata-power-torsion`

Lean: `Submodule.powTorsion_isSubmodule`

The subset $\Gamma_{(a)}(M)$ contains zero and is closed under addition and scalar multiplication.

Proof. Uses:

Zero is killed by a^0. If $a^m m_1 = 0$ and $a^n m_2 = 0$, then $a^{m+n}(m_1 + m_2) = 0$. If $r \in R$, then $a^m(rm_1) = r(a^m m_1) = 0$. □

Definition 2.4.5 (Selected power-torsion exponent). Uses: `def:nagata-power-torsion`

Lean: `Submodule.powTorsionExponent`

For $m \in M$ and $h : m \in \Gamma_{(a)}(M)$, define

$$\nu_a(m, h) := \mathrm{Nat.find}(h) \in \mathbb{N},$$

where h is unfolded to the nonempty type of exponents n satisfying $a^n m = 0$. Thus the selected exponent is explicit dependent data.

Lemma 2.4.6 (Specification of the selected power-torsion exponent). Uses: `def:nagata-power-torsion`, `def:nagata-power-torsion-exponent`

Lean: `Submodule.powTorsionExponent_spec`

For $h : m \in \Gamma_{(a)}(M)$, one has $a^{\nu_a(m,h)} m = 0$.

Proof. Uses: `def:nagata-power-torsion-exponent`

Unfold $\nu_a(m, h)$. The target is exactly $\mathrm{Nat.find_{spec}}(h)$. □

Lemma 2.4.7 (An annihilating exponent gives power torsion). Uses: `def:nagata-power-torsion`

Lean: `Submodule.mem_powTorsion_of_pow_smul_eq_zero`

If $a^n m = 0$, then $m \in \Gamma_{(a)}(M)$.

Proof. Uses: `def:nagata-power-torsion`
Unfold membership in $\Gamma_{(a)}(M)$ and use n as the witness; the remaining field is the supplied equality $a^n m = 0$. □

Lemma 2.4.8 (Membership in power torsion)**.** *Uses:* *def: nagata-power-torsion*

Lean: *Submodule. mem_ powTorsion_ iff*

Constructed by: *def: nagata-power-torsion-exponent*
One has

$$m \in \Gamma_{(a)}(M) \iff \exists n,\ a^n m = 0.$$

The right-hand side should be the simp-normal form for membership.

Proof. Uses: `def:nagata-power-torsion-exponent,lemma:nagata-power-torsion-exponent-spec,lemma:nagata-power-torsion-of-exponent`
For the forward implication, use the selected exponent $\nu_a(m, h)$ from Definition 2.4.5; its required equality is Lemma 2.4.6. For the reverse implication, instantiate Lemma 2.4.7 with the supplied exponent and equality. □

Lemma 2.4.9 (Power torsion as a localization kernel)**.** *Uses:* *def: nagata-power-torsion*

Lean: *Submodule. powTorsion_ eq_ ker_ localization*
The power-torsion submodule is the kernel of the localization map

$$M \longrightarrow M_a.$$

Proof. Uses:
An element maps to zero in localization away from the powers of a exactly when it is killed by one element of that multiplicative system, namely by a power of a. □

Lemma 2.4.10 (Localization of power torsion)**.** *Uses:*

Lean: *Submodule. map_ powTorsion_ localization*
Let $s \in R$. Under localization at s, the image of $\Gamma_{(a)}(M)$ is $\Gamma_{(a/1)}(M_s)$.

Proof. Uses: `lemma:nagata-power-torsion-kernel-localization`
Localize the kernel description in Lemma 2.4.9. Localization is flat, so it preserves this kernel. The iterated localization $(M_s)_{a/1}$ is canonically M_{sa}. □

Lemma 2.4.11 (The power-torsion quotient has no exceptional torsion)**.** *Uses:* *def: nagata-power-torsion*

Lean: *Submodule. powTorsion_ quotient_ torsionfree*
Multiplication by a on $M/\Gamma_{(a)}(M)$ is injective. More generally, no nonzero class in the quotient is killed by a power of a.

Proof. Uses:
If $a^n[m] = 0$, then $a^n m$ belongs to the power-torsion submodule, so some further power of a kills m. Thus m already represents zero in the quotient. □

Definition 2.4.12 (Affine base-change chart ring)**.** Uses: `def:nagata-blowup-chart-ring`

Lean: `AlgebraicGeometry.strictTransformBaseChangeChartRing`
Let $A \to B$ be a ring map, let $I \subseteq A$, and let $a \in I$. Define

$$C_a(A, B, I) = B \otimes_A A[I/a].$$

We regard a as an element of $C_a(A, B, I)$ through the structural map $A \to C_a(A, B, I)$.

Lemma 2.4.13 (Ordinary base-change chart)**.** *Uses:* *def: nagata-strict-transform-basechange-chart-ring*

Lean: *AlgebraicGeometry. strictTransform_ baseChangeChart*
Over the a-chart of $\mathrm{Bl}_I(\operatorname{Spec} A)$, the ordinary base change

$$\operatorname{Spec}(B) \times_{\operatorname{Spec} A} \mathrm{Bl}_I(\operatorname{Spec} A)$$

is canonically $\operatorname{Spec}(C_a(A, B, I))$.

Proof. Uses: `prop:nagata-affine-blowup-chart`, `def:nagata-strict-transform-basechange-chart-ring`
Apply the affine fibre-product formula to the constructed chart identification of Proposition 2.3.104. The tensor product appearing in that formula is precisely Definition 2.4.12. □

Definition 2.4.14 (Affine strict-transform chart ring)**.** Uses: `def:nagata-power-torsion`, `def:nagata-strict-transform-basechange-chart-ring`

Lean: `AlgebraicGeometry.strictTransformChartRing`
Define

$$C_a^{\mathrm{st}}(A,B,I) = C_a(A,B,I)/\Gamma_{(a)}(C_a(A,B,I)).$$

Lemma 2.4.15 (Closedness of the strict-transform chart)**.** *Uses:* `def:nagata-strict-transform-chart-ring`

Lean: `AlgebraicGeometry.strictTransformChart_closedImmersion`
The quotient map $C_a(A,B,I) \to C_a^{\mathrm{st}}(A,B,I)$ defines a closed immersion of the strict-transform chart into the ordinary base-change chart.

Proof. Uses: `lemma:nagata-power-torsion-submodule`
The power-torsion subset is an ideal by Lemma 2.4.4, applied to the regular module $C_a(A,B,I)$. The spectrum of a quotient ring is a closed subscheme. □

Definition 2.4.16 (Extended ideal in the new base ring)**.** Uses:

Lean: `AlgebraicGeometry.extendedBlowupChartIdeal`
Let $\phi\colon A \to B$ be a ring homomorphism and let $I \subseteq A$ be an ideal. Define

$$J = \mathrm{Ideal.map}(\phi, I) \subseteq B.$$

Thus J is the extension IB, with its construction recorded as an ideal map rather than as informal notation.

Definition 2.4.17 (Image of the distinguished chart element)**.** Uses:

Lean: `AlgebraicGeometry.extendedBlowupChartElement`
For $a \in I$, define $b = \phi(a) \in B$.

Lemma 2.4.18 (The image chart element belongs to the extended ideal)**.** *Uses:* `def:nagata-extended-blowup-chart-ideal`, `def:nagata-extended-blowup-chart-element`

Lean: `AlgebraicGeometry.extendedBlowupChartElement_mem`
The element b belongs to J.

Proof. Uses: `def:nagata-extended-blowup-chart-ideal`, `def:nagata-extended-blowup-chart-element`
Apply the canonical constructor $a \in I \Rightarrow \phi(a) \in \mathrm{Ideal.map}(\phi, I)$. □

Lemma 2.4.19 (Ideal-power membership is preserved by the base-ring map)**.** *Uses:* `def:nagata-extended-blowup-chart-ideal`

Lean: `AlgebraicGeometry.extendedBlowupChart_map_mem_pow`
For $n \in \mathbf{N}$ and $x \in I^n$, one has $\phi(x) \in J^n$.

Proof. Uses: `def:nagata-extended-blowup-chart-ideal`
The ideal-map operation preserves products, hence preserves powers by induction on n:

$$\mathrm{Ideal.map}(\phi, I^n) = (\mathrm{Ideal.map}(\phi, I))^n = J^n.$$

The canonical map-membership constructor sends $x \in I^n$ to $\phi(x) \in \mathrm{Ideal.map}(\phi, I^n)$; transport along the displayed equality. □

Definition 2.4.20 (Canonical factor map from the extended base ring)**.** Uses: `def:nagata-extended-blowup-chart-ideal`, `def:nagata-extended-blowup-chart-element`, `def:nagata-blowup-chart-ring`

Lean: `AlgebraicGeometry.extendedBlowupChartBFactor`
Constructed by: `lemma:nagata-extended-blowup-chart-element-mem`
Using $b \in J$, form the chart ring $B[J/b]$ and define

$$\iota_B\colon B \longrightarrow B[J/b]$$

to be its canonical B-algebra structure map.

Definition 2.4.21 (Raw representative map from the original chart factor). Uses: def:nagata-homogeneous-away-chart-representatives, def:nagata-extended-blowup-chart-ideal, def:nagata-extended-blowup-chart-element, def:nagata-blowup-chart-ring

Lean: AlgebraicGeometry.extendedBlowupChartFactorRepresentative

Constructed by: lemma:nagata-extended-blowup-chart-map-power-mem, lemma:nagata-chart-ring-fraction-mem

For $(n, x) \in \mathrm{HRep}_a$, define

$$\tau_a^{\mathrm{raw}}(n, x) = \left\langle \frac{\phi(x)}{b^n}, \mathrm{chartFraction_mem}(\phi(x) \in J^n) \right\rangle \in B[J/b].$$

The power-membership and chart-membership certificates are the two constructors recorded by \constructedby.

Lemma 2.4.22 (The original-chart factor map respects localization). Uses: def:nagata-homogeneous-away-chart-representatives, def:nagata-extended-blowup-chart-factor-representative-map

Lean: AlgebraicGeometry.extendedBlowupChartFactorRepresentative_respects

If $(n, x)\ \rho_a\ (m, y)$, then $\tau_a^{\mathrm{raw}}(n, x) = \tau_a^{\mathrm{raw}}(m, y)$ in $B[J/b]$.

Proof. Uses: def:nagata-extended-blowup-chart-element, lemma:nagata-extended-blowup-chart-element-mem, lemma:nagata-extended-blowup-chart-map-power-mem, lemma:nagata-chart-ring-fraction-equality

A relation witness satisfies $a^{r+m}x = a^{r+n}y$. Apply ϕ and use $\phi(a) = b$ to obtain $b^{r+m}\phi(x) = b^{r+n}\phi(y)$. Instantiate Lemma 2.3.23 with the explicit data $(C, K, c) = (B, J, b)$; its hypotheses $b \in J$, $\phi(x) \in J^n$, and $\phi(y) \in J^m$ are respectively Lemma 2.4.18 and two instances of Lemma 2.4.19. The result is equality of the underlying fractions, and subtype extensionality gives the stated equality in $B[J/b]$. □

Definition 2.4.23 (Quotient function from original-chart representatives). Uses: def:nagata-homogeneous-away-chart-quotient-type, def:nagata-extended-blowup-chart-factor-representative-map, def:nagata-blowup-chart-ring

Lean: AlgebraicGeometry.extendedBlowupChartFactorFunction

Constructed by: lemma:nagata-extended-blowup-chart-factor-respects

Define

$$\overline{\tau}_a \colon \mathrm{HChart}_a \longrightarrow B[J/b]$$

by quotient lift of τ_a^{raw}.

Lemma 2.4.24 (The original-chart quotient function preserves zero). Uses: def:nagata-extended-blowup-chart-factor-function, def:nagata-homogeneous-away-chart-quotient-comm-ring

Lean: AlgebraicGeometry.extendedBlowupChartFactorFunction_zero

One has $\overline{\tau}_a(0) = 0$.

Proof. Uses: def:nagata-homogeneous-away-chart-raw-zero, def:nagata-extended-blowup-chart-factor-function, def:nagata-extended-blowup-chart-factor-representative-map

Evaluate the quotient lift on $(0, 0)$. Its underlying fraction is $\phi(0)/b^0 = 0$; subtype extensionality gives the equality. □

Lemma 2.4.25 (The original-chart quotient function preserves one). Uses: def:nagata-extended-blowup-chart-factor-function, def:nagata-homogeneous-away-chart-quotient-comm-ring

Lean: AlgebraicGeometry.extendedBlowupChartFactorFunction_one

One has $\overline{\tau}_a(1) = 1$.

Proof. Uses: def:nagata-homogeneous-away-chart-raw-one, def:nagata-extended-blowup-chart-factor-function, def:nagata-extended-blowup-chart-factor-representative-map

Evaluate on $(0, 1)$. The underlying fraction is $\phi(1)/b^0 = 1$. □

Lemma 2.4.26 (The original-chart quotient function preserves addition). Uses: def:nagata-extended-blowup-chart-factor-function, def:nagata-homogeneous-away-chart-quotient-comm-ring

Lean: AlgebraicGeometry.extendedBlowupChartFactorFunction_add

For all $u, v \in \mathrm{HChart}_a$, $\overline{\tau}_a(u + v) = \overline{\tau}_a(u) + \overline{\tau}_a(v)$.

Proof. Uses: def:nagata-homogeneous-away-chart-raw-add, def:nagata-homogeneous-away-chart-quotient-add-op, def:nagata-extended-blowup-chart-factor-function, def:nagata-extended-blowup-chart-factor-representative-map

Quotient induction reduces to (n, x) and (m, y). Since $\phi(a) = b$, both sides have underlying fraction

$$\frac{b^m \phi(x) + b^n \phi(y)}{b^{n+m}}.$$

Apply subtype extensionality. □

Lemma 2.4.27 (The original-chart quotient function preserves multiplication)**.** *Uses:* def: nagata-extended-blowup-chart-factor-function, def: nagata-homogeneous-away-chart-quotient-comm-ring

Lean: AlgebraicGeometry.extendedBlowupChartFactorFunction_mul

For all $u, v \in \mathrm{HChart}_a$, $\overline{\tau}_a(uv) = \overline{\tau}_a(u)\overline{\tau}_a(v)$.

Proof. Uses: def:nagata-homogeneous-away-chart-raw-mul, def:nagata-homogeneous-away-chart-quotient-mul-op, def:nagata-extended-blowup-chart-factor-function, def:nagata-extended-blowup-chart-factor-representative-map

Quotient induction reduces to (n, x) and (m, y); both underlying fractions are $\phi(xy)/b^{n+m}$. □

Definition 2.4.28 (Original-chart factor ring homomorphism)**.** Uses: def:nagata-extended-blowup-chart-factor-function, def:nagata-homogeneous-away-chart-quotient-comm-ring, def:nagata-blowup-chart-ring

Lean: AlgebraicGeometry.extendedBlowupChartFactorQuotient

Constructed by: lemma:nagata-extended-blowup-chart-factor-zero, lemma:nagata-extended-blowup-chart-factor-one, lemma:nagata-extended-blowup-chart-factor-add, lemma:nagata-extended-blowup-chart-factor-mul

Package $\overline{\tau}_a$ and the preceding four preservation lemmas as a ring homomorphism $\mathrm{HChart}_a \to B[J/b]$.

Definition 2.4.29 (Map from the original affine chart factor)**.** Uses: def:nagata-extended-blowup-chart-factor-quotient-map, def:nagata-homogeneous-quotient-chart-ring-equivalence, def:nagata-blowup-chart-ring

Lean: AlgebraicGeometry.extendedBlowupChartFactor

Define

$$\tau_a \colon A[I/a] \longrightarrow B[J/b]$$

by the composite

$$A[I/a] \xrightarrow{\Phi_a^{-1}} \mathrm{HChart}_a \xrightarrow{\overline{\tau}_a} B[J/b].$$

Lemma 2.4.30 (Compatibility of the two factor maps over the base ring)**.** *Uses:* def: nagata-extended-blowup-chart-b-factor-map, def: nagata-extended-blowup-chart-factor-map

Lean: AlgebraicGeometry.extendedBlowupChartFactors_comm

For every $r \in A$, the two images of r agree:

$$\iota_B(\phi(r)) = \tau_a(r/1).$$

Proof. Uses: def:nagata-extended-blowup-chart-b-factor-map, def:nagata-extended-blowup-chart-factor-map, def:nagata-extended-blowup-chart-factor-quotient-map, def:nagata-extended-blowup-chart-factor-function, def:nagata-extended-blowup-chart-factor-representative-map, lemma:nagata-homogeneous-quotient-chart-ring-equivalence-sym-apply

By Lemma 2.3.85 with $n = 0$, Φ_a^{-1} sends $r/1$ to $[(0, r)]$. The quotient-lift computation rule sends this class to $\phi(r)/b^0 = \phi(r)$, which is $\iota_B(\phi(r))$. □

Definition 2.4.31 (Raw balanced pair map for the strict-transform chart)**.** Uses: def:nagata-extended-blowup-chart-b-factor-map, def:nagata-extended-blowup-chart-factor-map

Lean: AlgebraicGeometry.strictTransformChartRawPairMap

Define

$$\mu_a \colon B \times A[I/a] \longrightarrow B[J/b], \qquad \mu_a(y, z) = \iota_B(y)\tau_a(z).$$

This is only a function of two variables.

Lemma 2.4.32 (Zero law for the raw pair map in the first variable)**.** *Uses:* *def:nagata-strict-chart-raw-pair-map*

Lean: *AlgebraicGeometry.StrictTransformChartRawPairMap_zero_left*
For $z \in A[I/a]$, $\mu_a(0, z) = 0$.

Proof. Uses: def:nagata-strict-chart-raw-pair-map
Expand μ_a, use $\iota_B(0) = 0$, and simplify $0 \cdot \tau_a(z) = 0$. □

Lemma 2.4.33 (Additivity of the raw pair map in the first variable)**.** *Uses:* *def:nagata-strict-chart-raw-pair-map*

Lean: *AlgebraicGeometry.StrictTransformChartRawPairMap_add_left*
For $y, y' \in B$ and $z \in A[I/a]$, $\mu_a(y + y', z) = \mu_a(y, z) + \mu_a(y', z)$.

Proof. Uses: def:nagata-strict-chart-raw-pair-map
Expand μ_a, use additivity of ι_B, and distribute once. □

Lemma 2.4.34 (Zero law for the raw pair map in the second variable)**.** *Uses:* *def:nagata-strict-chart-raw-pair-map*

Lean: *AlgebraicGeometry.StrictTransformChartRawPairMap_zero_right*
For $y \in B$, $\mu_a(y, 0) = 0$.

Proof. Uses: def:nagata-strict-chart-raw-pair-map,def:nagata-extended-blowup-chart-factor-map
Expand μ_a, use $\tau_a(0) = 0$, and simplify $\iota_B(y) \cdot 0 = 0$. □

Lemma 2.4.35 (Additivity of the raw pair map in the second variable)**.** *Uses:* *def:nagata-strict-chart-raw-pair-map*

Lean: *AlgebraicGeometry.StrictTransformChartRawPairMap_add_right*
For $y \in B$ and $z, z' \in A[I/a]$, $\mu_a(y, z + z') = \mu_a(y, z) + \mu_a(y, z')$.

Proof. Uses: def:nagata-strict-chart-raw-pair-map,def:nagata-extended-blowup-chart-factor-map
Expand μ_a, use additivity of τ_a, and distribute once. □

Definition 2.4.36 (Right-variable additive homomorphism for the pair map)**.** Uses: def:nagata-strict-chart-raw-pair-map

Lean: AlgebraicGeometry.strictTransformChartPairRightAddHom

Constructed by: lemma:nagata-strict-chart-raw-pair-zero-right,lemma:nagata-strict-chart-raw-pair-add-right
For $y \in B$, package the function $z \mapsto \mu_a(y, z)$ as

$$\mu^R_{a,y} \colon A[I/a] \longrightarrow +B[J/b]$$

using AddMonoidHom.mk' and the preceding zero and addition laws.

Lemma 2.4.37 (The right-variable additive homomorphism sends zero in the first variable to zero)**.** *Uses:* *def:nagata-strict-chart-pair-right-add-hom*

Lean: *AlgebraicGeometry.StrictTransformChartPairRightAddHom_zero*
One has $\mu^R_{a,0} = 0$ as additive homomorphisms $A[I/a] \to +B[J/b]$.

Proof. Uses: def:nagata-strict-chart-pair-right-add-hom,lemma:nagata-strict-chart-raw-pair-zero-left
Apply extensionality for additive homomorphisms. At z, the desired equality is exactly $\mu_a(0, z) = 0$. □

Lemma 2.4.38 (The right-variable additive homomorphism is additive in the first variable)**.** *Uses:* *def:nagata-strict-chart-pair-right-add-hom*

Lean: *AlgebraicGeometry.StrictTransformChartPairRightAddHom_add*
For $y, y' \in B$, $\mu^R_{a,y+y'} = \mu^R_{a,y} + \mu^R_{a,y'}$.

Proof. Uses: def:nagata-strict-chart-pair-right-add-hom,lemma:nagata-strict-chart-raw-pair-add-left
Apply additive-homomorphism extensionality. Evaluation at z reduces the goal to $\mu_a(y + y', z) = \mu_a(y, z) + \mu_a(y', z)$. □

Definition 2.4.39 (Nested additive homomorphism for the tensor constructor)**.** Uses: `def:nagata-strict-chart-pair-right-add-hom`

Lean: `AlgebraicGeometry.strictTransformChartNestedAddHom`

Constructed by: `lemma:nagata-strict-chart-pair-right-add-hom-zero,lemma:nagata-strict-chart-pair-right-add-hom-add`

Package $y \mapsto \mu^R_{a,y}$ as the nested additive homomorphism

$$\mu_a^{++} \colon B \longrightarrow +\big(A[I/a] \longrightarrow +B[J/b]\big).$$

Its evaluation formula is $\mu_a^{++}(y)(z) = \mu_a(y,z)$.

Lemma 2.4.40 (Balancing equation for the nested additive homomorphism)**.** *Uses:* *def:nagata-strict-chart-nested-add-hom*

Lean: *AlgebraicGeometry.strictTransformChartRawPairMap_balanced*

For $r \in A$, $y \in B$, and $z \in A[I/a]$,

$$\mu_a^{++}(r \cdot y)(z) = \mu_a^{++}(y)(r \cdot z),$$

where $r \cdot y = \phi(r)y$ and $r \cdot z = (r/1)z$ are the two A-module structures.

Proof. Uses: `def:nagata-strict-chart-nested-add-hom,def:nagata-strict-chart-pair-right-add-hom,def:nagata-strict-chart-raw-pair-map,def:nagata-extended-blowup-chart-factor-map,lemma:nagata-extended-blowup-chart-factor-base-compatible`

Expand the nested homomorphism and μ_a. Replace $\iota_B(\phi(r))$ by $\tau_a(r/1)$ using Lemma 2.4.30. Replace $\tau_a((r/1)z)$ by $\tau_a(r/1)\tau_a(z)$ and reassociate the three factors. Both sides are $\iota_B(y)\tau_a(r/1)\tau_a(z)$ after commutativity. □

Definition 2.4.41 (Tensor-product lift of the strict-chart pair map)**.** Uses: `def:nagata-strict-transform-basechange-chart-ring,def:nagata-strict-chart-nested-add-hom,def:nagata-blowup-chart-ring`

Lean: `AlgebraicGeometry.strictTransformChartTensorLift`

Constructed by: `lemma:nagata-strict-chart-raw-pair-balanced`

Instantiate the mathlib constructor with the displayed arguments:

$$\widetilde{\sigma}_a = \mathrm{TensorProduct.liftAddHom}\big(\mu_a^{++}, h_{\mathrm{bal}}\big).$$

This defines an additive homomorphism

$$\widetilde{\sigma}_a \colon B \otimes_A A[I/a] \longrightarrow +B[J/b]$$

satisfying $\widetilde{\sigma}_a(y \otimes z) = \mu_a^{++}(y)(z) = \mu_a(y,z)$. The constructor used here has exact input type $B \to +(A[I/a] \to +B[J/b])$ together with the balancing certificate h_{bal} recorded by `\constructedby`.

Lemma 2.4.42 (The tensor-product lift preserves one)**.** *Uses:* *def:nagata-strict-chart-tensor-lift*

Lean: *AlgebraicGeometry.strictTransformChartTensorLift_one*

One has $\widetilde{\sigma}_a(1) = 1$.

Proof. Uses: `def:nagata-strict-chart-tensor-lift,def:nagata-strict-chart-raw-pair-map`

The unit is the pure tensor $1 \otimes 1$. The computation rule for the tensor lift gives $\iota_B(1)\tau_a(1) = 1$. □

Lemma 2.4.43 (Multiplication formula on two pure tensors)**.** *Uses:* *def:nagata-strict-chart-tensor-lift*

Lean: *AlgebraicGeometry.strictTransformChartTensorLift_mul_pure*

For $y, y' \in B$ and $z, z' \in A[I/a]$,

$$\widetilde{\sigma}_a\big((y \otimes z)(y' \otimes z')\big) = \widetilde{\sigma}_a(y \otimes z)\widetilde{\sigma}_a(y' \otimes z').$$

Proof. Uses: `def:nagata-strict-chart-tensor-lift,def:nagata-strict-chart-nested-add-hom,def:nagata-strict-chart-pair-right-add-hom,def:nagata-strict-chart-raw-pair-map`

The pure-tensor multiplication rule gives $(y \otimes z)(y' \otimes z') = yy' \otimes zz'$. Evaluate $\widetilde{\sigma}_a$ on the three pure tensors. The left side is $\iota_B(yy')\tau_a(zz')$; the right side is $\iota_B(y)\tau_a(z)\iota_B(y')\tau_a(z')$. Use multiplicativity of ι_B and τ_a, commute the middle two factors, and reassociate. □

Lemma 2.4.44 (Multiplication formula with a pure tensor on the left)**.** *Uses:* `def:nagata-strict-chart-tensor-lift`

Lean: `AlgebraicGeometry.strictTransformChartTensorLift_mul_pure_left`

For $y \in B$, $z \in A[I/a]$, and $v \in B \otimes_A A[I/a]$,

$$\widetilde{\sigma}_a((y \otimes z)v) = \widetilde{\sigma}_a(y \otimes z)\widetilde{\sigma}_a(v).$$

Proof. Uses: `lemma:nagata-strict-chart-tensor-lift-mul-pure`

Apply tensor-product induction to v. In the zero branch, simplify $(y \otimes z)0 = 0$ and use that the additive homomorphism $\widetilde{\sigma}_a$ sends zero to zero. The pure-tensor branch is Lemma 2.4.43. In the addition branch, distribute $(y \otimes z)(v_1 + v_2)$, use additivity of $\widetilde{\sigma}_a$, apply the two induction hypotheses, and distribute $\widetilde{\sigma}_a(y \otimes z)$ over the resulting sum. □

Lemma 2.4.45 (The tensor-product lift preserves multiplication)**.** *Uses:* `def:nagata-strict-chart-tensor-lift`

Lean: `AlgebraicGeometry.strictTransformChartTensorLift_mul`

For $u, v \in B \otimes_A A[I/a]$,

$$\widetilde{\sigma}_a(uv) = \widetilde{\sigma}_a(u)\widetilde{\sigma}_a(v).$$

Proof. Uses: `lemma:nagata-strict-chart-tensor-lift-mul-pure-left`

Fix v and apply tensor-product induction to u. In the zero branch, simplify $0v = 0$ and use preservation of zero by the additive homomorphism. The pure-tensor branch is Lemma 2.4.44. In the addition branch, distribute $(u_1 + u_2)v$, use additivity of $\widetilde{\sigma}_a$, apply the two induction hypotheses, and distribute multiplication by $\widetilde{\sigma}_a(v)$ over the sum. □

Lemma 2.4.46 (The tensor-product lift preserves the (B)-algebra structure)**.** *Uses:* `def:nagata-strict-chart-tensor-lift`, `def:nagata-extended-blowup-chart-b-factor-map`

Lean: `AlgebraicGeometry.strictTransformChartTensorLift_algebraMap`

For every $y \in B$, $\widetilde{\sigma}_a(y \otimes 1) = \iota_B(y)$.

Proof. Uses: `def:nagata-strict-chart-tensor-lift`, `def:nagata-strict-chart-raw-pair-map`

Apply the pure-tensor computation rule and simplify $\mu_a(y, 1) = \iota_B(y)\tau_a(1) = \iota_B(y)$. □

Definition 2.4.47 (Map to the blow-up chart of the extended ideal)**.** Uses: `def:nagata-strict-transform-basechange-chart-ring`, `def:nagata-strict-chart-tensor-lift`, `def:nagata-blowup-chart-ring`

Lean: `AlgebraicGeometry.strictTransformChartToExtendedBlowupChart`

Constructed by: `lemma:nagata-strict-chart-tensor-lift-one`, `lemma:nagata-strict-chart-tensor-lift-mul`, `lemma:nagata-strict-chart-tensor-lift-b-algebra`

Define σ_a to be the B-algebra homomorphism whose underlying additive map is $\widetilde{\sigma}_a$ and whose multiplicative and B-algebra fields are the three preceding lemmas.

Lemma 2.4.48 (Pure-tensor formula for the extended-ideal chart map)**.** *Uses:* `def:nagata-strict-chart-to-extended-blowup-chart`

Lean: `AlgebraicGeometry.strictTransformChartToExtendedBlowupChart_apply`

For $y \in B$, $n \geq 0$, and $x \in I^n$,

$$\sigma_a\big(y \otimes (x/a^n)\big) = y\,\overline{x}/b^n.$$

Proof. Uses: `def:nagata-strict-chart-to-extended-blowup-chart`, `def:nagata-strict-chart-tensor-lift`, `def:nagata-strict-chart-raw-pair-map`, `def:nagata-extended-blowup-chart-factor-map`, `def:nagata-extended-blowup-chart-factor-quotient-map`, `def:nagata-extended-blowup-chart-factor-function`, `def:nagata-extended-blowup-chart-factor-representative-map`, `lemma:nagata-homogeneous-quotient-chart-ring-equivalence-sym-apply`

The tensor-lift computation rule gives $\sigma_a(y \otimes (x/a^n)) = \iota_B(y)\tau_a(x/a^n)$. By Lemma 2.3.85, $\Phi_a^{-1}(x/a^n) = [(n, x)]$. The quotient-lift computation rule therefore gives $\tau_a(x/a^n) = \overline{x}/b^n$; multiplying yields the formula. □

Lemma 2.4.49 (Surjectivity onto the extended-ideal chart)**.** *Uses:* `def:nagata-strict-chart-to-extended-blowup-chart`

Lean: `AlgebraicGeometry.strictTransformChartToExtended_surjective`

The constructed homomorphism σ_a is surjective.

Proof. Uses: lemma:nagata-strict-chart-to-extended-blowup-chart,lemma:nagata-chart-ring-normal-form

By Lemma 2.3.18, the target is generated as a B-algebra by the fractions z/b with $z \in J$. Write $z = \sum_k y_k \overline{x_k}$ with $x_k \in I$. Then

$$z/b = \sum_k \sigma_a\big(y_k \otimes (x_k/a)\big),$$

proving surjectivity. □

Lemma 2.4.50 (Kernel of the map to the extended-ideal chart)**.** *Uses:* def:nagata-power-torsion, def:nagata-strict-chart-to-extended-blowup-chart

Lean: AlgebraicGeometry.ker_strictTransformChartToExtended

The kernel of σ_a is exactly $\Gamma_{(a)}(C_a(A, B, I))$.

Proof. Uses: lemma:nagata-strict-chart-map-surjective,lemma:nagata-power-torsion-kernel-localization

Both source and target map to B_b. The target $B[J/b]$ is the subalgebra generated there by the fractions from J, and σ_a has exactly that image by Lemma 2.4.49. Thus its kernel is the kernel of localization at b. Under the A-action this is the kernel of localization at a, which equals the a-power torsion by Lemma 2.4.9. □

Definition 2.4.51 (Strict-transform chart comparison)**.** Uses: def:nagata-strict-transform-chart-ring,def:nagata-strict-chart-to-extended-blowup-chart

Lean: AlgebraicGeometry.strictTransformChartComparison

Define

$$\overline{\sigma}_a \colon C_a^{\mathrm{st}}(A, B, I) \longrightarrow B[IB/b]$$

to be the map induced by σ_a on the power-torsion quotient.

Proposition 2.4.52 (Affine strict-transform chart formula)**.** *Uses:* def:nagata-strict-transform-chart-comparison

Lean: AlgebraicGeometry.strictTransformChartIsoExtendedBlowupChart

The constructed comparison $\overline{\sigma}_a$ is a ring equivalence.

Proof. Uses: lemma:nagata-strict-chart-map-surjective,lemma:nagata-strict-chart-map-kernel

The map σ_a is surjective and its kernel is precisely the ideal by which the source was quotiented. The first isomorphism theorem therefore makes $\overline{\sigma}_a$ bijective and identifies its inverse ring structure, so it is a ring equivalence. □

Definition 2.4.53 (Localized strict-transform chart comparison)**.** Uses: def:nagata-strict-transform-chart-comparison

Lean: AlgebraicGeometry.strictTransformChartComparisonLocalization

After localizing the base ring or a chart ring, define $(\overline{\sigma}_a)_s$ by localizing the source and target of $\overline{\sigma}_a$ and transporting along the canonical tensor-product and iterated-localization associativity maps.

Lemma 2.4.54 (Strict-transform chart formula localizes)**.** *Uses:* def:nagata-strict-transform-chart-comparison-localized

Lean: AlgebraicGeometry.strictTransformChartIso_localization

The constructed localized comparison $(\overline{\sigma}_a)_s$ equals the strict-transform chart comparison constructed directly from the localized data.

Proof. Uses: lemma:nagata-power-torsion-localization,prop:nagata-strict-transform-chart-iso

Localization commutes with the tensor product and with the power-torsion quotient by Lemma 2.4.10. Both maps send every pure tensor and every generator x/a to the same fraction. Ring-map extensionality gives the equality, and Proposition 2.4.52 supplies the equivalence property on both sides. □

Lemma 2.4.55 (Cocycle for strict-transform charts)**.** *Uses:* def:nagata-strict-transform-chart-comparison-localized

Lean: AlgebraicGeometry.strictTransformChartIso_cocycle

The localized strict-transform chart comparisons agree on double localizations and satisfy the cocycle equality on triple chart overlaps.

Proof. Uses: lemma:nagata-strict-transform-chart-localization,lemma:nagata-affine-blowup-restrict-cocycle

All maps are induced by the same homomorphism into a common localization of B. The equality on double overlaps follows from Lemma 2.4.54; on triple overlaps it reduces to the affine blow-up localization cocycle. Equality holds on the images of B and on all fractions from I, hence everywhere. □

Definition 2.4.56 (Strict transform of a quasi-coherent module)**.** Uses: def:nagata-power-torsion

Lean: QuasiCoherent.Sheaf.strictTransform

For a quasi-coherent $\mathcal{O}_Y$-module $\mathcal{F}$ and a blow-up $X' \to X$, define the strict transform of $\mathcal{F}$ to be the pullback to $Y \times_X X'$ modulo the subsheaf of sections killed locally by a power of the exceptional ideal.

Lemma 2.4.57 (Affine formula for a module strict transform)**.** *Uses: def:nagata-module-strict-transform, def:nagata-power-torsion*

Lean: QuasiCoherent.Sheaf.strictTransform_restrict_chart

If $\mathcal{F}$ corresponds to a B-module M, then on the a-chart its strict transform corresponds to

$$\frac{M \otimes_A A[I/a]}{a\text{-power torsion}}.$$

Proof. Uses:

Pullback of a quasi-coherent module on an affine morphism is tensor product. Sections supported on the exceptional closed subset are exactly the elements killed by a power of its local equation a. □

Lemma 2.4.58 (Quasi-coherence of the module strict transform)**.** *Uses: def:nagata-module-strict-transform*

Lean: QuasiCoherent.Sheaf.strictTransform_isQuasiCoherent

The module strict transform is quasi-coherent.

Proof. Uses: lemma:nagata-module-strict-transform-chart,lemma:nagata-power-torsion-localization

On affine charts it is associated to the quotient module in Lemma 2.4.57. These modules localize correctly by Lemma 2.4.10, so their associated sheaves glue. □

Definition 2.4.59 (Closed chart model of the global strict transform)**.** Uses: def:nagata-strict-transform-chart-ring

Lean: AlgebraicGeometry.Hom.strictTransformClosedChartModel

For $f\colon Y \to X$ and a blow-up $X' \to X$, define $Y^{\mathrm{st}}_{\mathrm{chart}}$ by gluing the affine schemes $\operatorname{Spec}(C_a^{\mathrm{st}})$ and their quotient closed immersions into the corresponding charts of $Y \times_X X'$.

Proposition 2.4.60 (Global strict transform is a closed subscheme)**.** *Uses: def:nagata-strict-transform-closed-chart-model, def:nagata-strict-transform*

Lean: AlgebraicGeometry.Hom.strictTransform_iota

The constructed chart model $Y^{\mathrm{st}}_{\mathrm{chart}}$ is a closed subscheme of $Y \times_X X'$ and agrees with the strict transform of Definition 2.4.2.

Proof. Uses: lemma:nagata-strict-transform-chart-closed,lemma:nagata-strict-transform-chart-cocycle

The chartwise closed immersions agree on overlaps by Lemma 2.4.55; therefore their ideals glue to a global quasi-coherent ideal. On each chart the quotient removes exactly the sections supported on the exceptional divisor, so the resulting closed subscheme is the scheme-theoretic closure of the preserved open and hence is the strict transform. □

Definition 2.4.61 (Comparison from the strict transform to the extended-ideal blow-up)**.** Uses: def:nagata-strict-transform-closed-chart-model,def:nagata-strict-transform-chart-comparison,def:nagata-global-blowup

Lean: AlgebraicGeometry.Hom.strictTransformToBlowup

Define

$$\Theta_{f,\mathcal{I}}\colon Y^{\mathrm{st}} \longrightarrow \mathrm{Bl}_{\mathcal{I}\mathcal{O}_Y}(Y)$$

by gluing the affine morphisms $\operatorname{Spec}(\overline{\sigma}_a^{-1})$ on the standard blow-up charts.

Theorem 2.4.62 ([1, Tag 080E]). *Uses:* def:nagata-strict-transform-blowup-comparison

Lean: AlgebraicGeometry.Hom.strictTransformIsoBlowup

The constructed morphism $\Theta_{f,\mathcal{I}}$ is an isomorphism over Y and identifies the strict transform with the blow-up of Y in the extended ideal $\mathcal{I}\mathcal{O}_Y$.

Proof. Uses: prop:nagata-strict-transform-chart-iso,prop:nagata-strict-transform-closed-fiber-product,prop:nagata-global-blowup-affine-restrict,lemma:nagata-strict-transform-chart-cocycle

On every standard affine chart, $\Theta_{f,\mathcal{I}}$ is induced by the ring equivalence $\overline{\sigma}_a$ of Proposition 2.4.52; hence it is an isomorphism there. The charts cover the target by Proposition 2.3.138, and their comparison maps agree on overlaps by Lemma 2.4.55. Thus the local inverses glue. Their formulas preserve the structural maps to Y, so the global isomorphism is over Y. □

Lemma 2.4.63 (Scheme-theoretic closure characterization). *Uses:*

Lean: AlgebraicGeometry.Hom.strictTransform_eq_schemeTheoreticClosure

Let $U = X \setminus V(\mathcal{I})$. The strict transform is the smallest closed subscheme of $Y \times_X X'$ through which

$$Y \times_X U \longrightarrow Y \times_X X'$$

factors. In particular it is the scheme-theoretic closure of this open part.

Proof. Uses: lemma:nagata-power-torsion-quotient-torsionfree,prop:nagata-strict-transform-closed-fiber-product

On an a-chart, a further quotient that agrees after inverting a has kernel killed locally by a power of a. The quotient C_a^{st} has no such nonzero torsion by Lemma 2.4.11. Therefore no smaller closed subscheme can contain the preserved open. Glue this affine statement. □

Lemma 2.4.64 (Flatness kills the strict-transform torsion). *Uses:* def:nagata-power-torsion

Lean: AlgebraicGeometry.strictTransform_powTorsion_eq_bot_of_flat

If B is flat over A, then the a-power-torsion in $B \otimes_A A[I/a]$ is zero.

Proof. Uses: lemma:nagata-chart-generator-regular

Multiplication by a is injective on $A[I/a]$ by Lemma 2.3.168. Tensor this injection with the flat A-module B; multiplication by a remains injective, and therefore no nonzero element is killed by a power of a. □

Corollary 2.4.65 (Flat strict transform equals ordinary base change). *Uses:*

Lean: AlgebraicGeometry.Hom.strictTransformIsoBaseChange_of_flat

If $Y \to X$ is flat, the strict transform is all of $Y \times_X \mathrm{Bl}_{\mathcal{I}}(X)$.

Proof. Uses: lemma:nagata-flat-strict-transform-no-torsion,prop:nagata-strict-transform-closed-fiber-product

On every affine blow-up chart the quotient is by the zero ideal, so the closed immersion in Proposition 2.4.60 is an isomorphism. □

Lemma 2.4.66 (Strict transform on the preserved open). *Uses:*

Lean: AlgebraicGeometry.Hom.strictTransform_restrict_compl

Over $U = X \setminus V(\mathcal{I})$, the strict transform is canonically $Y \times_X U$, and its map to Y is the ordinary pullback isomorphism.

Proof. Uses: thm:nagata-global-blowup-complement,lemma:nagata-strict-transform-closure-characterization

The blow-up is the identity over U by Theorem 2.3.157. No exceptional torsion remains there, so the strict-transform quotient is unchanged. □

Lemma 2.4.67 (Strict transform under a dominating blow-up). *Uses:*

Lean: AlgebraicGeometry.Hom.strictTransform_comp_dominatingBlowup

Suppose $X'' \to X' \to X$ are blow-ups which are isomorphisms over the same open U. The strict transform of Y in X'' is canonically the strict transform, along $X'' \to X'$, of the strict transform of Y in X'.

Proof. Uses: lemma:nagata-strict-transform-closure-characterization

Both closed subschemes of $Y \times_X X''$ are the scheme-theoretic closure of the same open $Y \times_X U$. Apply uniqueness in Lemma 2.4.63. □

Lemma 2.4.68 (Disjoint strict transforms stay disjoint)**.** *Uses:*

Lean: `AlgebraicGeometry.StrictTransform.disjoint_of_dominates`

If two strict transforms in X' are disjoint, their strict transforms after a further dominating blow-up $X'' \to X'$ are disjoint.

Proof. Uses: `lemma:nagata-strict-transform-dominating`

Each new strict transform is contained in the inverse image of the corresponding old strict transform. Inverse images of disjoint closed subschemes are disjoint. □

Lemma 2.4.69 (Composition of admissible blow-ups, [1, Tag 080L])**.** *Uses:* `def:nagata-admissible-blowup`

Lean: `AlgebraicGeometry.AdmissibleBlowup.comp`

Let X be quasi-compact and quasi-separated and let $U \subseteq X$ be quasi-compact. A composition of two U-admissible blow-ups is a U-admissible blow-up. The chosen isomorphism over U is the composite of the chosen pullback isomorphisms.

Proof. Uses: `thm:nagata-composition-of-blowups`

Apply Theorem 2.3.198. It writes the composite as the blow-up of a finite-presentation center $Y \hookrightarrow X$ whose support is the union of the first center and the image of the second center. Both sets are disjoint from U, because the first blow-up is the identity over U and the second center is disjoint from its inverse image. Thus $Y \cap U = \varnothing$, so the resulting blow-up is U-admissible. Pulling the composite back to U identifies it with the composite of the two chosen isomorphisms. □

Lemma 2.4.70 (Properness of a blow-up)**.** *Uses:*

Lean: `AlgebraicGeometry.IsBlowup.isProper`

The blow-up of a finite type quasi-coherent ideal is projective and hence proper. In particular, every admissible blow-up used in this chapter is proper.

Proof. Uses: `def:nagata-global-blowup,thm:nagata-global-blowup-proper`

A finite type quasi-coherent ideal defines the global blow-up of Definition 2.3.134. Its structure map is proper by Theorem 2.3.166. An admissible blow-up is a blow-up of precisely this form. □

Lemma 2.4.71 (Strict transform and restriction)**.** *Uses:* `def:nagata-strict-transform`

Lean: `AlgebraicGeometry.StrictTransform.restrict_isIso_locus`

Over the open on which a blow-up is an isomorphism, the strict transform is canonically the ordinary pullback. This identification is compatible with the projection to the original scheme.

Proof. Uses: `lemma:nagata-strict-transform-over-complement`

Over the isomorphism locus there is no exceptional closed subset, so the local equation of the exceptional divisor is a unit and its power-torsion submodule is zero. Equivalently, apply Lemma 2.4.66. Compatibility with the projection follows from the defining fiber-product square. □

Lemma 2.4.72 ([1, Tag 080E])**.** *Uses:* `def:nagata-strict-transform`

Lean: `AlgebraicGeometry.StrictTransform.of_closedImmersion`

Let $Z \hookrightarrow X$ be a closed immersion and let $X' \to X$ be a blow-up. Blowing up Z in the pulled-back center gives the strict transform of Z in X'. Strict transforms computed inside Z and inside X agree under the induced closed immersion.

Proof. Uses: `thm:nagata-strict-transform-is-blowup,prop:nagata-strict-transform-chart-iso`

By Theorem 2.4.62, the strict transform is the blow-up of Z in the extended center ideal. The universal property gives its canonical map to $Z \times_X X'$, and the affine chart formula in Proposition 2.4.52 identifies this map with the closed immersion defining the strict transform. Thus computing first in Z or directly inside X gives the same closed subscheme. □

Theorem 2.4.73 (Relative Proj and blow-up package)**.** *Uses:* `def:nagata-global-blowup`

Lean: `AlgebraicGeometry.relativeProjBlowupPackage`

The Rees-algebra relative-Proj construction supplies the affine and global blow-up, standard affine charts, chart restriction and cocycle maps, cover independence, complement-of-center isomorphism, universal property, projectivity and properness, flat base change, product-ideal comparison, and strict-transform chart comparison used in the Nagata chapter.

Proof. Uses: prop:nagata-rees-graded, prop:nagata-affine-blowup-chart, prop:nagata-affine-blowup-chart-cover, prop:nagata-affine-blowup-restrict, lemma:nagata-affine-blowup-restrict-cocycle, thm:nagata-global-blowup-gluing-package, prop:nagata-global-blowup-affine-restrict, prop:nagata-global-blowup-cover-independent, thm:nagata-global-blowup-complement, thm:nagata-global-blowup-projective, thm:nagata-global-blowup-proper, thm:nagata-blowup-universal-property, cor:nagata-blowup-cartier-iso, thm:nagata-blowup-flat-base-change, thm:nagata-blowup-product-ideal, cor:nagata-blowup-product-common-refinement, prop:nagata-strict-transform-chart-iso, lemma:nagata-strict-transform-chart-cocycle, thm:nagata-strict-transform-is-blowup, lemma:nagata-strict-transform-ambient-closed

The Rees algebra is graded by Proposition 2.3.12. Relative Proj of that graded algebra is the affine blow-up on affine targets; the standard charts, their cover, and their structure maps are Propositions 2.3.104 and 2.3.106. Localization of the Rees algebra gives the restriction maps and the identity/triple cocycle by Proposition 2.3.119 and Lemma 2.3.120. The global construction is obtained by gluing these affine models, with cover independence supplied by Theorem 2.3.154 and Propositions 2.3.138 and 2.3.153. The complement, projectivity, properness, universal property, Cartier specialization, flat base change, and product-ideal comparison are the corresponding cited theorems. Finally, Proposition 2.4.52, Lemmas 2.4.55 and 2.4.72, and Theorem 2.4.62 identify strict transforms chartwise and prove compatibility with ambient closed subschemes. These components exhaust the blow-up interfaces used by the rest of the proof. □

Definition 2.4.74 (Maximal extension of a quasi-coherent submodule)**.** Uses:

Lean: `Scheme.QuasiCoherent.maximalSubmoduleExtension`

Let $j\colon W \hookrightarrow X$ be a quasi-compact open immersion, let $\mathcal{M}$ be quasi-coherent on the qcqs scheme X, and let $\mathcal{F} \subseteq \mathcal{M}|_W$ be quasi-coherent. Define

$$\mathcal{M}^{\max}_{\mathcal{F}} = \ker\bigl(\mathcal{M} \longrightarrow j_*\bigl((\mathcal{M}|_W)/\mathcal{F}\bigr)\bigr).$$

Lemma 2.4.75 (Properties of the maximal submodule extension)**.** *Uses:* *def:nagata-maximal-submodule-extension*

Lean: *Scheme.QuasiCoherent.maximalSubmoduleExtension_spec*

The sheaf $\mathcal{M}^{\max}_{\mathcal{F}}$ is a quasi-coherent submodule of $\mathcal{M}$, its restriction to W is $\mathcal{F}$, and it contains every quasi-coherent submodule $\mathcal{N} \subseteq \mathcal{M}$ satisfying $\mathcal{N}|_W \subseteq \mathcal{F}$.

Proof. Uses:

A quasi-compact open immersion into a quasi-separated scheme has quasi-coherent direct image. Thus the quotient, its direct image, and the displayed kernel are quasi-coherent. Restriction to W is exact and the counit $j^*j_* \to \mathrm{id}$ is an isomorphism for an open immersion, so the restricted kernel is

$$\ker\bigl(\mathcal{M}|_W \to (\mathcal{M}|_W)/\mathcal{F}\bigr) = \mathcal{F}.$$

If $\mathcal{N}|_W \subseteq \mathcal{F}$, the composite from $\mathcal{N}$ to the displayed quotient vanishes on every open subset of W and hence, by adjunction, vanishes as a map to its direct image. The universal property of the kernel gives the asserted factorization. □

Lemma 2.4.76 (Affine cleared-denominator extension)**.** *Uses:*

Lean: *Module.QuasiCoherent.affineClearedDenominatorExtension*

Let A be a ring, let M be an A-module, let $W = \bigcup_{i=1}^{r} D(f_i) \subseteq \mathrm{Spec}(A)$, and let $\mathcal{F} \subseteq \widetilde{M}|_W$ be a finite type quasi-coherent submodule. There is a finitely generated submodule $N \subseteq M$ such that

$$\widetilde{N}|_W = \mathcal{F}.$$

It can be obtained from finite generating lists on the $D(f_i)$ by one finite, uniform denominator-clearing operation.

Proof. Uses:

Choose generators of $\mathcal{F}|_{D(f_i)}$ and write them, after changing by powers of the unit f_i, as localizations of elements $m_{ij} \in M$. For fixed i, j, k, the image of m_{ij} on $D(f_i f_k)$ belongs to $\mathcal{F}|_{D(f_i f_k)}$. Since the selected $m_{k\ell}$ generate on $D(f_k)$, localization membership supplies an exponent e_{ijk} such that

$$f_i^{e_{ijk}} m_{ij} \in \sum_{\ell} A_{f_k} m_{k\ell} \subseteq M_{f_k}.$$

Take one exponent e_i at least all e_{ijk} for the finitely many pairs (j,k), and put $m'_{ij} = f_i^{e_i} m_{ij}$. Multiplication by $f_i^{e_i}$ does not change the generated module on $D(f_i)$, while the displayed containments show that every m'_{ij} belongs to $\mathcal{F}|_{D(f_k)}$ for every k.

Let N be generated by the finite family of all m'_{ij}. On $D(f_k)$, all its generators belong to $\mathcal{F}$, proving $\widetilde{N}|_{D(f_k)} \subseteq \mathcal{F}|_{D(f_k)}$. The subfamily $m'_{k\ell}$ still generates $\mathcal{F}|_{D(f_k)}$, proving the reverse containment. The equalities agree on overlaps because both sides are submodules of $\widetilde{M}$ and all restriction maps are localization. □

Lemma 2.4.77 (Finite-type submodule extension from a quasi-compact open)**.** *Uses:* `def:nagata-maximal-submodule-extension`

Lean: `Scheme.QuasiCoherent.exists_finiteType_submodule_restrict_eq`

Under the hypotheses of Definition 2.4.74, if $\mathcal{F}$ is of finite type and W is quasi-compact, there is a finite type quasi-coherent submodule $\mathcal{N} \subseteq \mathcal{M}$ with $\mathcal{N}|_W = \mathcal{F}$.

Proof. Uses: `lemma:nagata-maximal-submodule-extension,lemma:nagata-affine-cleared-denominator-extension`

First replace $\mathcal{M}$ by $\mathcal{M}^{\max}_{\mathcal{F}}$. This does not change the desired restriction and ensures that every submodule constructed below restricts into $\mathcal{F}$.

Choose affine opens $V_1, \dots, V_n$ with $X = W \cup V_1 \cup \cdots \cup V_n$, taking n minimal. We induct on n. There is nothing to prove when $n = 0$. Suppose a finite type submodule has been constructed on $X_{n-1} = W \cup V_1 \cup \cdots \cup V_{n-1}$ and restricts to $\mathcal{F}$ on W. The overlap $X_{n-1} \cap V_n$ is quasi-compact by quasi-separatedness. Since V_n is affine, write that overlap as a finite union of principal opens and apply Lemma 2.4.76 to extend the already constructed finite type submodule on the overlap to a finite type submodule on V_n.

The two submodules are literally equal on the overlap, so the identity comparison glues them on $X_n = X_{n-1} \cup V_n$. Identity comparisons and associativity of restriction give the identity and triple cocycles. Repeating the finite induction reaches X. Containment in the maximal extension gives the inclusion after restriction to W, while the initial equality on W gives the reverse inclusion. □

Definition 2.4.78 (Finite-type ideal-extension witness predicate)**.** Uses:

Lean: `Scheme.QuasiCoherent.IsFiniteTypeIdealExtension`

Let X be qcqs, let $W \subseteq X$ be a quasi-compact open, and let $\mathcal{J} \subseteq \mathcal{O}_W$ be a finite-type quasi-coherent ideal. For a submodule $\mathcal{I} \subseteq \mathcal{O}_X$, let $\mathsf{IsExt}_{X,W}(\mathcal{J}, \mathcal{I})$ mean that $\mathcal{I}$ is a quasi-coherent ideal of finite type and is equipped with mutually inverse maps of subideals over $\mathcal{O}_W$

$$\rho : \mathcal{I}|_W \longrightarrow \mathcal{J}, \qquad \sigma : \mathcal{J} \longrightarrow \mathcal{I}|_W.$$

Lemma 2.4.79 (Existence of a finite-type ideal-extension witness, [1, Tag 01PF])**.** *Uses:* `def:nagata-finite-type-ideal-extension-predicate`

Lean: `Scheme.QuasiCoherent.exists_finiteTypeIdealExtension`

There exists $\mathcal{I} \subseteq \mathcal{O}_X$ satisfying $\mathsf{IsExt}_{X,W}(\mathcal{J}, \mathcal{I})$.

Proof. Uses: `lemma:nagata-qcqs-finite-type-submodule-extension`

Apply the finite-type submodule-extension lemma to $\mathcal{M} = \mathcal{O}_X$ and $\mathcal{F} = \mathcal{J}$. Its finite affine induction constructs a finite-type quasi-coherent submodule $\mathcal{I} \subseteq \mathcal{O}_X$, hence an ideal, together with the equality $\mathcal{I}|_W = \mathcal{J}$. Take the equality map and its inverse as ρ and σ; their inverse equations are equality induction. □

Definition 2.4.80 (Selected finite-type ideal-extension witness)**.** Uses: `def:nagata-finite-type-ideal-extension-predicate`

Constructed by: `lemma:nagata-finite-type-ideal-extension-exists`

Lean: `Scheme.QuasiCoherent.selectedFiniteTypeIdealExtension`

Select the first witness in the fixed well-order and denote its ideal by $\mathcal{I}^{\mathrm{sel}}_{X,W}(\mathcal{J})$. Retain its ideal inclusion, quasi-coherence, finite-type, restriction maps, and inverse equations as fields of the selected witness.

Definition 2.4.81 (Raw localization datum for the selected extension)**.** Uses: `def:nagata-selected-finite-type-ideal-extension`

Lean: `Scheme.QuasiCoherent.finiteTypeIdealExtensionRawLocalData`

Choose a finite affine cover of X used by the extension induction and, on each overlap with W, a finite basic cover $D(f_{ri})$. The raw datum consists of the localized subideals $\mathcal{I}^{\mathrm{sel}}(D(f_{ri})) \subseteq \mathcal{O}_W(D(f_{ri}))$, the ideals $\mathcal{J}(D(f_{ri}))$, and the restriction maps induced by the selected ρ and σ.

Lemma 2.4.82 (Forward localization containment)**.** *Uses:* `def:nagata-finite-type-ideal-extension-local-data`

Lean: `Scheme.QuasiCoherent.finiteTypeIdealExtension_localization_le`

On each selected $D(f_{ri})$, the localized extension is contained in $\mathcal{J}(D(f_{ri}))$.

Proof. Uses: `def:nagata-selected-finite-type-ideal-extension`

The selected map ρ commutes with the two inclusions into $\mathcal{O}_W$. Localizing it on $D(f_{ri})$ therefore identifies every element of the localized extension with an element of $\mathcal{J}(D(f_{ri}))$. □

Lemma 2.4.83 (Reverse localization containment). *Uses:* def:nagata-finite-type-ideal-extension-local-data

Lean: Scheme.QuasiCoherent.finiteTypeIdealExtension_le_localization
On each selected $D(f_{ri})$, $\mathcal{J}(D(f_{ri}))$ is contained in the localized extension.

Proof. Uses: def:nagata-selected-finite-type-ideal-extension
The inverse map σ is also a map of subideals over $\mathcal{O}_W$. After localization it sends every element of $\mathcal{J}(D(f_{ri}))$ to the same ambient section in the localized extension, giving the reverse containment. □

Lemma 2.4.84 (Equality on every selected localization). *Uses:* def:nagata-finite-type-ideal-extension-local-data

Lean: Scheme.QuasiCoherent.finiteTypeIdealExtension_localization_eq
The two localized ideals are equal on every selected basic open.

Proof. Uses: lemma:nagata-finite-type-ideal-extension-localization-forward,lemma:nagata-finite-type-ideal-extension-localization-reverse
Apply antisymmetry to the two containments. □

Lemma 2.4.85 (Overlap equality for localization comparisons). *Uses:* def:nagata-finite-type-ideal-extension-local-data

Lean: Scheme.QuasiCoherent.finiteTypeIdealExtension_overlap_eq
On the intersection of two selected basic opens, the two restrictions of the localization equality coincide.

Proof. Uses: lemma:nagata-finite-type-ideal-extension-localization-equality
Both maps are the equality between the same two submodules after localization in $\mathcal{O}_W(D(f_{ri}f_{sj}))$. Localization of equality is equality, so the two maps coincide. □

Lemma 2.4.86 (Identity law for extension overlaps). *Uses:* def:nagata-finite-type-ideal-extension-local-data

Lean: Scheme.QuasiCoherent.finiteTypeIdealExtension_overlap_refl
A self-overlap comparison is the identity.

Proof. Uses: lemma:nagata-finite-type-ideal-extension-overlap-equality
Both restrictions are localization along the same identity map. □

Lemma 2.4.87 (Triple cocycle for extension overlaps). *Uses:* def:nagata-finite-type-ideal-extension-local-data

Lean: Scheme.QuasiCoherent.finiteTypeIdealExtension_overlap_cocycle
On a triple overlap, the composite of the first two comparisons is the direct comparison.

Proof. Uses: lemma:nagata-finite-type-ideal-extension-overlap-equality
All three maps are identities between the same localized subideal. Associativity of localization identifies their composite with the direct identity. □

Lemma 2.4.88 (Quasi-coherence field for the selected extension). *Uses:* def:nagata-selected-finite-type-ideal-extension

Lean: Scheme.QuasiCoherent.finiteTypeIdealExtension_quasiCoherent
The selected ideal is quasi-coherent.

Proof. Uses: lemma:nagata-finite-type-ideal-extension-exists
This is a field of the selected exact extension witness. □

Lemma 2.4.89 (Finite-type field for the selected extension). *Uses:* def:nagata-selected-finite-type-ideal-extension

Lean: Scheme.QuasiCoherent.finiteTypeIdealExtension_finiteType
The selected ideal is of finite type.

Proof. Uses: lemma:nagata-finite-type-ideal-extension-exists
This is a field of the selected exact extension witness. □

Definition 2.4.90 (Certified localization datum for ideal extension)**.** Uses: `def:nagata-finite-type-ideal-extension-local-data`

Constructed by: `lemma:nagata-finite-type-ideal-extension-localization-forward,lemma:nagata-finite-type-ideal-extension-localization-reverse,lemma:nagata-finite-type-ideal-extension-overlap-equality,lemma:nagata-finite-type-ideal-extension-overlap-identity,lemma:nagata-finite-type-ideal-extension-overlap-cocycle,lemma:nagata-finite-type-ideal-extension-quasi-coherent,lemma:nagata-finite-type-ideal-extension-finite-type`

Lean: `Scheme.QuasiCoherent.certifiedFiniteTypeIdealExtensionGlueData`

Bundle the raw localization data with the two containment fields, overlap equality, identity, triple cocycle, quasi-coherence, and finite-type fields.

Definition 2.4.91 (Finite-type extension of an ideal from a quasi-compact open)**.** Uses: `def:nagata-certified-finite-type-ideal-extension-glue-data`

Lean: `Scheme.QuasiCoherent.finiteTypeIdealExtension`

Define $\mathrm{Ext}_{X,W}(\mathcal{J})$ to be the underlying ideal of the certified selected datum.

Definition 2.4.92 (Restriction comparison for the extended ideal)**.** Uses: `def:nagata-finite-type-ideal-extension`

Lean: `Scheme.QuasiCoherent.finiteTypeIdealExtensionRestrictionIso`

Let $\rho : \mathrm{Ext}_{X,W}(\mathcal{J})|_W \to \mathcal{J}$ and $\sigma : \mathcal{J} \to \mathrm{Ext}_{X,W}(\mathcal{J})|_W$ be the two maps retained by the selected witness.

Lemma 2.4.93 (Left inverse equation for the restriction comparison)**.** Uses: `def:nagata-finite-type-ideal-extension-restriction-map`

Lean: `Scheme.QuasiCoherent.finiteTypeIdealExtensionRestrictionIso_hom_inv_id`

One has $\sigma\rho = 1$ on the restricted extension.

Proof. Uses: `lemma:nagata-finite-type-ideal-extension-localization-equality`

Check on the selected basic cover of W; there both maps are identity equalities of the same localized subideal. □

Lemma 2.4.94 (Right inverse equation for the restriction comparison)**.** Uses: `def:nagata-finite-type-ideal-extension-restriction-map`

Lean: `Scheme.QuasiCoherent.finiteTypeIdealExtensionRestrictionIso_inv_hom_id`

One has $\rho\sigma = 1$ on $\mathcal{J}$.

Proof. Uses: `lemma:nagata-finite-type-ideal-extension-localization-equality`

The same basic-open calculation gives the other inverse equation. □

Lemma 2.4.95 (Localization and cocycle certificate for the extended ideal)**.** Uses: `def:nagata-finite-type-ideal-extension`

Lean: `Scheme.QuasiCoherent.finiteTypeIdealExtension_localization`

The extension has the selected localization equalities, overlap equality, identity, triple cocycle, quasi-coherence, and finite-type certificates.

Proof. Uses: `lemma:nagata-finite-type-ideal-extension-localization-equality,lemma:nagata-finite-type-ideal-extension-overlap-equality,lemma:nagata-finite-type-ideal-extension-overlap-identity,lemma:nagata-finite-type-ideal-extension-overlap-cocycle,lemma:nagata-finite-type-ideal-extension-quasi-coherent,lemma:nagata-finite-type-ideal-extension-finite-type`

These are exactly the six displayed certificate fields. □

Lemma 2.4.96 (Finite-type ideal extension, [1, Tag 01PF])**.** Uses: `def:nagata-finite-type-ideal-extension`

Lean: `Scheme.QuasiCoherent.extend_finiteType_ideal`

The sheaf $\mathrm{Ext}_{X,W}(\mathcal{J})$ is a finite-type quasi-coherent ideal and its restriction to W is $\mathcal{J}$.

Proof. Uses: `lemma:nagata-finite-type-ideal-extension-quasi-coherent,lemma:nagata-finite-type-ideal-extension-finite-type,lemma:nagata-finite-type-ideal-extension-restriction-left-inverse,lemma:nagata-finite-type-ideal-extension-restriction-right-inverse`

The first two lemmas give the sheaf fields; the last two make the displayed restriction comparison an isomorphism. □

Lemma 2.4.97 (Independence after restriction to the original open)**.** *Uses:*

Lean: `Scheme.QuasiCoherent.finiteTypeIdealExtension_restrictIso_unique`

If $\mathcal{I}$ and $\mathcal{I}'$ are two finite-type extensions of $\mathcal{J}$, their Rees algebras and blow-ups after restriction to W are canonically identical. These comparisons are transitive and compatible with restriction to smaller opens.

Proof. Uses: `thm:nagata-blowup-flat-base-change`

In degree n, both restricted Rees pieces are $\mathcal{J}^n$. The degreewise identity preserves multiplication and degree zero; relative Proj gives the identity comparison of restricted blow-ups. Composites and further restrictions of identity maps are identities. No equality of the two global ideals is asserted. □

Theorem 2.4.98 (Finite-type ideal extension package)**.** *Uses:* `def:nagata-finite-type-ideal-extension`

Lean: `Scheme.QuasiCoherent.finiteTypeIdealExtensionPackage`

The certified local data, the extended ideal, its restriction isomorphism, and the restricted Rees/blow-up comparison form the extension package.

Proof. Uses: `lemma:nagata-finite-type-ideal-extension-localization,lemma:nagata-extend-finite-type-ideal,lemma:nagata-finite-type-ideal-extension-independent`

Assemble the three cited interfaces. The independence field is explicitly restricted to W. □

Definition 2.4.99 (Extension construction for an admissible blow-up)**.** Uses: `def:nagata-admissible-blowup,def:nagata-finite-type-ideal-extension,def:nagata-global-blowup`

Lean: `AlgebraicGeometry.AdmissibleBlowup.extensionConstruction`

Let X be quasi-compact and quasi-separated, let $W \subseteq X$ and $U \subseteq W$ be quasi-compact opens, and write a given U-admissible blow-up of W as $\mathrm{Bl}_{\mathcal{J}}(W) \to W$. Glue $\mathcal{J}$ on W with the unit ideal on U to an ideal $\mathcal{J}^+$ on $U \cup W$, set

$$\mathcal{I}_X(\mathcal{J}) = \mathrm{Ext}_{X,U\cup W}(\mathcal{J}^+),$$

and define

$$\mathsf{ExtBl}_{X,W,U}(\mathcal{J}) = \mathrm{Bl}_{\mathcal{I}_X(\mathcal{J})}(X) \longrightarrow X.$$

Lemma 2.4.100 (Property of the extended admissible blow-up, [1, Tag 080M])**.** *Uses:* `def:nagata-extended-admissible-blowup`

Lean: `AlgebraicGeometry.AdmissibleBlowup.extend_from_open`

The constructed morphism $\mathsf{ExtBl}_{X,W,U}(\mathcal{J}) \to X$ is U-admissible, and its restriction to W is canonically the given blow-up $\mathrm{Bl}_{\mathcal{J}}(W) \to W$.

Proof. Uses: `lemma:nagata-extend-finite-type-ideal,thm:nagata-blowup-flat-base-change`

By Lemma 2.4.96, the ideal $\mathcal{I}_X(\mathcal{J})$ is finite type, restricts to $\mathcal{J}$ on W, and restricts to the unit ideal on U. Its blow-up is therefore U-admissible. The open immersion $W \to X$ is flat, so Theorem 2.3.182 identifies the pullback of the constructed blow-up with $\mathrm{Bl}_{\mathcal{J}}(W)$; the chart formula shows that this identification is the identity over U. □

Lemma 2.4.101 ([1, Tag 080A])**.** *Uses:*

Lean: `AlgebraicGeometry.Blowup.productIdeal_dominates`

Let $\mathcal{I}_1$ and $\mathcal{I}_2$ be finite type quasi-coherent ideals on X. The blow-up of $\mathcal{I}_1\mathcal{I}_2$ canonically dominates the blow-ups of $\mathcal{I}_1$ and of $\mathcal{I}_2$. If both ideals restrict to the unit ideal on U, all three blow-ups are U-admissible.

Proof. Uses: `cor:nagata-blowup-product-common-refinement,thm:nagata-global-blowup-complement`

The two domination maps are Corollary 2.3.190. If the two ideals are the unit ideal on U, so is their product; therefore the product blow-up is unchanged over U by Theorem 2.3.157. □

2.4.2 Atomic flattening and elimination of indeterminacy

The public elimination theorem uses two different modifications. The first one makes the boundary Cartier and hence makes the preserved open scheme-theoretically dense. The second one is the flattening modification. Keeping these modifications separate until the final composition prevents the flattening centre on the first blow-up from being confused with the eventual centre on the original scheme.

Lemma 2.4.102 (Finite-type ideal cutting out the boundary, [1, Tag 01PH]). *Uses:*

Lean: `AlgebraicGeometry.exists_finiteTypeIdealSheafWithBoundary`

Let X be qcqs and let $U \subseteq X$ be a quasi-compact open. There is a finite-type quasi-coherent ideal $\mathcal{I}_0 \subseteq \mathcal{O}_X$ such that

$$\mathcal{I}_0|_U = \mathcal{O}_U, \qquad V(\mathcal{I}_0) = X \setminus U$$

set-theoretically.

Proof. Uses: `lemma:nagata-qcqs-finite-type-submodule-extension`

Let $\mathcal{J} \subseteq \mathcal{O}_X$ be the quasi-coherent radical ideal defining the reduced closed complement $X \setminus U$. Its restriction to U is the unit ideal. Apply Lemma 2.4.77 to

$$\mathcal{O}_U = \mathcal{J}|_U \subseteq \mathcal{J}|_U.$$

It gives a finite-type quasi-coherent submodule $\mathcal{I}_0 \subseteq \mathcal{J}$ with $\mathcal{I}_0|_U = \mathcal{O}_U$. Since it is a submodule of the ideal $\mathcal{J}$, it is an ideal. The restriction equality shows that $V(\mathcal{I}_0)$ is disjoint from U, while $\mathcal{I}_0 \subseteq \mathcal{J}$ gives $V(\mathcal{J}) \subseteq V(\mathcal{I}_0)$. These two inclusions give the displayed equality of closed sets. □

Definition 2.4.103 (Selected boundary ideal and first blow-up). Uses: `def:nagata-global-blowup`

Constructed by: `lemma:nagata-flattening-boundary-ideal-exists`

Lean: `AlgebraicGeometry.flatteningFirstBlowup`

Select the first ideal supplied by the preceding boundary-ideal existence statement, denote it by $\mathcal{I}_0(X, U)$, and define

$$b_0 \colon X_0 = \mathrm{Bl}_{\mathcal{I}_0}(X) \longrightarrow X.$$

Retain the finite-type, restriction, and zero-locus fields of the selected witness.

Lemma 2.4.104 (The first blow-up is the identity over the preserved open). *Uses:* `def:nagata-flattening-first-blowup`

Lean: `AlgebraicGeometry.flatteningFirstBlowup_over_open`

The base change of b_0 to U is an isomorphism. In particular b_0 is U-admissible.

Proof. Uses: `def:nagata-blowup-chart-ring`

On an affine open $\mathrm{Spec}(A) \subseteq U$, the selected ideal is the unit ideal. Using 1 as a generator, its only needed standard chart is $A[A/1] = A$; equivalently its Rees algebra is $A[t]$ and its relative Proj is $\mathrm{Spec}(A)$. These affine identities commute with restriction and give $b_0^{-1}(U) \cong U$. Its centre is of finite presentation because its defining quasi-coherent ideal is of finite type, and its support is $X \setminus U$, which gives the asserted admissibility. □

Lemma 2.4.105 (The first exceptional inverse image is effective Cartier). *Uses:* `def:nagata-flattening-first-blowup`

Lean: `AlgebraicGeometry.flatteningFirstBlowup_exceptionalCartier`

The extended ideal $\mathcal{I}_0\mathcal{O}_{X_0}$ is invertible and defines an effective Cartier divisor $E_0 \subseteq X_0$ with

$$|E_0| = X_0 \setminus b_0^{-1}(U).$$

Proof. Uses: `def:nagata-blowup-chart-ring`

On the standard chart $A[I/a]$, the extended centre is (a) and a is a non-zero-divisor because $A[I/a] \subseteq A_a$. The transition ratios b/a and a/b are inverse units on chart overlaps, so these local regular equations glue to an effective Cartier divisor. Its support is the inverse image of $V(\mathcal{I}_0)$. The selected zero-locus equality gives $b_0^{-1}(V(\mathcal{I}_0)) = b_0^{-1}(X \setminus U) = X_0 \setminus b_0^{-1}(U)$. □

Lemma 2.4.106 (Density of an effective-Cartier complement, [1, Tag 07ZU]). *Uses:*

Lean: `Scheme.EffectiveCartierDivisor.compl_schematically_dense`

If $D \subseteq T$ is an effective Cartier divisor, then $T \setminus D$ is retrocompact and scheme-theoretically dense in T.

Proof. Uses:

Both assertions are local on T. On an affine open $\mathrm{Spec}(A)$, the divisor is cut out by a non-zero-divisor $a \in A$ and its complement is $D(a)$. This principal open is quasi-compact. Moreover, if $x/1 = 0$ in A_a, then $a^n x = 0$ for some n, hence $x = 0$ because a is a non-zero-divisor. Thus $A \to A_a$ is injective, which is the affine criterion for schematic density. The local statements glue. □

Lemma 2.4.107 (Density after the boundary blow-up). *Uses:* `def:nagata-flattening-first-blowup`

Lean: `AlgebraicGeometry.flatteningFirstBlowup_open_dense`

Via the chosen isomorphism over U, the open $U \subseteq X_0$ is retrocompact and scheme-theoretically dense.

Proof. Uses: lemma:nagata-flattening-first-blowup-over-U,lemma:nagata-flattening-first-exceptional-cartier,lemma:nagata-effective-cartier-complement-dense

The preceding two lemmas identify U with $X_0 \setminus E_0$. Apply the effective-Cartier complement lemma. □

Lemma 2.4.108 (The first blow-up is qcqs)**.** Uses: def:nagata-flattening-first-blowup

Lean: AlgebraicGeometry.flatteningFirstBlowup_qcqs

If X is qcqs, then X_0 is qcqs.

Proof. Uses: def:nagata-blowup-chart-ring

Choose a finite affine cover of X. On one member $\mathrm{Spec}(A)$, choose finite generators $a_1, \dots, a_m$ of the boundary ideal. Its inverse image is covered by the finitely many affine charts $\mathrm{Spec}(A[I/a_i])$, proving quasi-compactness. The intersection of the ith and jth charts is the principal open where a_j/a_i is invertible, and the two coordinate rings identify by the corresponding localizations. The tensor-product map onto this intersection ring is surjective because the jth chart supplies the inverse a_i/a_j. The affine diagonal criterion therefore makes the blow-up separated over X. A quasi-compact separated morphism over a qcqs scheme has qcqs source, which gives the claim. □

Definition 2.4.109 (First strict-transform projection)**.** Open obligation: OBL-NAG-FLAT-STRICT-TRANSFORM-CONSTRUCTOR

Uses: def:nagata-flattening-first-blowup,def:nagata-strict-transform

Lean: AlgebraicGeometry.flatteningFirstStrictTransform

For $q\colon Z \to X$, define

$$Z_0 = \mathrm{ST}_{b_0}(Z), \qquad q_0\colon Z_0 \longrightarrow X_0$$

using the chartwise power-torsion quotient and its certified gluing. The open token records that the current scheme-theoretic-image and global strict-transform constructors still lie in nonimportable umbrella nodes; it is not an assertion of their existence by choice.

Lemma 2.4.110 (Finite type of the first strict-transform projection)**.** Open obligation: OBL-NAG-FLAT-FIRST-STRICT-FINITE-TYPE

Uses: def:nagata-flattening-first-strict-transform

Lean: AlgebraicGeometry.flatteningFirstStrictTransform_finiteType

If q is of finite type, then q_0 is of finite type.

Proof. Uses: prop:nagata-strict-transform-closed-fiber-product

The strict transform is a closed subscheme of $Z \times_X X_0$. The projection of the fibre product is the base change of q, hence is of finite type; composing it with the closed immersion gives the claim. The open token remains until the cited global strict-transform closed immersion has an importable atomic constructor. □

Lemma 2.4.111 (Separatedness of the first strict-transform projection)**.** Open obligation: OBL-NAG-FLAT-FIRST-STRICT-SEPARATED

Uses: def:nagata-flattening-first-strict-transform

Lean: AlgebraicGeometry.flatteningFirstStrictTransform_separated

If q is separated, then q_0 is separated.

Proof. Uses: prop:nagata-strict-transform-closed-fiber-product

The base change $Z \times_X X_0 \to X_0$ is separated. A closed immersion is separated, and separated morphisms are stable under composition. The same constructor boundary as in the preceding lemma is recorded explicitly. □

Lemma 2.4.112 (Quasi-separatedness of the first strict-transform projection)**.** Open obligation: OBL-NAG-FLAT-FIRST-STRICT-QUASI-SEPARATED

Uses: def:nagata-flattening-first-strict-transform

Lean: AlgebraicGeometry.flatteningFirstStrictTransform_quasiSeparated

If q is separated, then q_0 is quasi-separated.

Proof. Uses: lemma:nagata-flattening-first-strict-transform-separated

A separated morphism has closed diagonal; every closed immersion is quasi-compact, so the diagonal is quasi-compact. □

Lemma 2.4.113 (The first strict transform is unchanged over U). *Open obligation:* `OBL-NAGATA-FLAT-FIRST-STRICT-OVER-U`

Uses: `def:nagata-flattening-first-strict-transform`

Lean: `AlgebraicGeometry.flatteningFirstStrictTransform_over_open`

If $q^{-1}(U) \to U$ is the chosen isomorphism, its pullback identifies $q_0^{-1}(U) \to U$ with that same isomorphism.

Proof. Uses: `lemma:nagata-flattening-first-blowup-over-U,lemma:nagata-strict-transform-restrict`

Over U the first blow-up is the identity. The strict-transform restriction comparison has no exceptional power torsion on this open and is compatible with the projection. Composing it with the given $q^{-1}(U) \cong U$ gives the stated identification. □

Definition 2.4.114 (Raynaud–Gruson module-flattening witness). Uses:

Lean: `AlgebraicGeometry.IsRGModuleFlatteningWitness`

Let S be qcqs, let $Y \to S$ be quasi-compact and locally of finite presentation, let $U \subseteq S$ be retrocompact and scheme-theoretically dense, and let $\mathcal{F}$ be a finite-type quasi-coherent module whose restriction over U is flat and of finite presentation. A Raynaud–Gruson witness is a finite sequence of U-admissible blow-ups together with the iterated strict transform $\mathcal{F}'$ and certificates that $\mathcal{F}'$ is flat over the final base and finitely presented on the final base change.

Lemma 2.4.115 (Projective-algebra splitting used by the coefficient ideal, [1, Tag 05FT]). *Open obligation:* `OBL-NAG-FLAT-RG-PROJECTIVE-ALGEBRA`

Uses:

Lean: `conditional-interface:Module.rgProjectiveAlgebraSplitting`

If $R \to A$ is flat and of finite presentation with geometrically integral fibres, there are a set Λ, an R-module C, and mutually inverse maps exhibiting

$$\bigoplus_{\lambda \in \Lambda} R \cong A \oplus C.$$

Proof. Uses:

This is the first irreducible Raynaud–Gruson foundation leaf in this route. Its proof requires spreading the algebra to a Noetherian finitely-generated $\mathbf{Z}$-model, the adic-completion Mittag–Leffler criterion, and faithfully flat descent of projectivity. Those constructions are not present in the current blueprint, so the statement is deliberately not closed by citing Tag 05FT. □

Definition 2.4.116 (Coefficient ideal of a Fitting ideal). Uses:

Lean: `Module.rgCoefficientIdeal`

Fix explicit splitting maps $\bigoplus_{\lambda \in \Lambda} R \rightleftarrows A \oplus C$. For a finitely generated ideal $J = (h_1, \dots, h_m) \subseteq A$, write each h_i in the free summand and define $c_R(J) \subseteq R$ to be the ideal generated by the finitely many nonzero coordinates of the h_i.

Lemma 2.4.117 (The coefficient ideal is finite and independent of generators). *Uses:* `def:nagata-rg-coefficient-ideal`

Lean: `Module.rgCoefficientIdeal_finite_independent`

The ideal $c_R(J)$ is finitely generated and equals the ideal generated by the coordinates of all elements of J; hence it is independent of the displayed finite generating list.

Proof. Uses: `def:nagata-rg-coefficient-ideal`

Every coordinate projection on the free module is R-linear. If $h = \sum_i a_i h_i$ with $a_i \in A$, multiplication by a_i is an R-linear endomorphism of the direct summand A. After composing with the fixed inclusion, projection, and free-coordinate maps, every coordinate of $a_i h_i$ is an R-linear combination of coordinates of h_i. Consequently all coordinates of all elements of J lie in the ideal generated by the finite coordinate family of the h_i. The reverse containment is immediate. □

Lemma 2.4.118 (Support of the Raynaud–Gruson coefficient centre). *Uses:* `def:nagata-rg-coefficient-ideal`

Lean: `Module.rgCoefficientIdeal_zeroLocus`

Assume that $R \to A$ is flat and finitely presented with geometrically integral fibres, and fix the splitting used in the coefficient-ideal definition. Let $f \in R$, let M be a finitely presented A-module, and suppose that for every prime $\mathfrak{p} \not\ni f$ the localization at the prime $\mathfrak{p}A$ is free of rank r. Put

$$J = \mathrm{Fit}_r(M), \qquad I = f\, c_R(J).$$

Then $V(I) = V(f)$ in $\mathrm{Spec}(R)$.

Proof. Uses: `lemma:nagata-rg-coefficient-ideal-finite`

The inclusion $V(f) \subseteq V(I)$ is immediate. Conversely, fix $\mathfrak{p} \not\supseteq f$. Rank-$r$ freeness at $\mathfrak{p}A$ says that $\mathrm{Fit}_r(M)$ is not contained in $\mathfrak{p}A$. Choose $h \in J$ whose image in $A \otimes_R \kappa(\mathfrak{p})$ is nonzero. If every coefficient of h lay in $\mathfrak{p}$, its image in that fibre would vanish because the fixed free-summand inclusion remains injective after tensoring. Thus some coefficient avoids $\mathfrak{p}$, so $c_R(J) \not\subseteq \mathfrak{p}$ and $I \not\subseteq \mathfrak{p}$. □

Lemma 2.4.119 (Fitting calculation on a coefficient blow-up chart, [1, Tag 0810]). *Open obligation:* `OBL-NAG-FLAT-RG-COEFFICIENT-CHART`

Uses: `def:nagata-rg-coefficient-ideal`

Lean: `conditional-interface:Module.rgCoefficientBlowupChart_free`

Assume that $R \to A$ is flat and finitely presented with geometrically integral fibres. Let M be a finite A-module such that M_f is finitely presented and $M_{\mathfrak{p}A}$ is free of rank r whenever $f \notin \mathfrak{p}$. There is a finitely generated ideal $I \subseteq R$ with $V(I) = V(f)$ such that, for every $a \in I$, if $R' = R[I/a]$ and

$$A' = A \otimes_R R', \qquad M' = (M \otimes_R R')/(a\text{-power torsion}),$$

and every $\mathfrak{p}' \in \mathrm{Spec}(R')$, there is $g \in A' \setminus \mathfrak{p}'A'$ such that M'_g is free of rank r.

Proof. Uses: `lemma:nagata-rg-projective-algebra-splitting,lemma:nagata-rg-coefficient-ideal-finite,lemma:nagata-rg-coefficient-centre-support`

Choose a finite free surjection $A^n \twoheadrightarrow M$. Since M_f is finitely presented, a finite submodule of the kernel gives a finitely presented module M_1 and a surjection $M_1 \twoheadrightarrow M$ which is an isomorphism after inverting f.

Use the projective-algebra splitting and put $J = \mathrm{Fit}_r(M_1)$ and $I = fc_R(J)$. The coefficient-ideal finiteness lemma makes I finitely generated, and the support lemma gives $V(I) = V(f)$. Fix $a \in I$, write $a = fa'$, and set $R' = R[I/a]$. The induced map of strict transforms $M_1' \twoheadrightarrow M'$ is an isomorphism: it is an isomorphism after inverting a, and its kernel is a-power torsion inside the a-torsion-free module M_1'. It therefore suffices to treat the finitely presented module M_1, which we rename M.

In R' one has $c_R(J)R' = a'R'$, and a' is a non-zero-divisor. Every coefficient of JA' is divisible by a'. Expressing the generator a' of $c_R(J)R'$ as a finite R'-linear combination of coefficients gives elements $h_j \in J$ such that not all h_j/a' lie in $\mathfrak{q}' = \mathfrak{p}'A'$; otherwise all those normalized coefficients would lie in $\mathfrak{p}'$, contradicting that their combination is 1. Since geometrically integral fibres make $\mathfrak{p}'A'$ prime, one h_j/a' is a unit in $A'_{\mathfrak{q}'}$. Hence $JA'_{\mathfrak{q}'} = a'A'_{\mathfrak{q}'}$. The Fitting criterion gives r generators for $M'_{\mathfrak{q}'}$; after shrinking by $g \notin \mathfrak{q}'$, choose a surjection $(A'_g)^r \twoheadrightarrow M'_g$.

Let $J' = \mathrm{Fit}_{r-1}(M')$. For every $\mathfrak{p}'' \in \mathrm{Spec}(R'_a)$, the rank condition over $R'_a = R_f$ makes the image of J'_a vanish in $(A'_a)_{\mathfrak{p}''A'}$. The localization map $A'_a \to \prod_{\mathfrak{p}''}(A'_a)_{\mathfrak{p}''A'}$ is injective, so $J'_a = 0$. The element a is a non-zero-divisor in R', and A' is flat over R', so multiplication by a is injective on A'. Since every element of J' is killed by a power of a, this gives $J' = 0$. The Fitting criterion now upgrades the displayed surjection to an isomorphism. The open token remains solely because the projective-algebra splitting has not been constructed. □

Lemma 2.4.120 (Pointwise finite presentation of a flat finite-type module, [1, Tag 05I7]). *Open obligation:* `OBL-NAG-FLAT-MODULE-POINTWISE-FP`

Uses:

Lean: `conditional-interface:Module.FinitePresentation.at_flat_point`

Let $Y \to S$ be locally of finite presentation and let $\mathcal{F}$ be a finite-type quasi-coherent module. If $\mathcal{F}$ is flat over S at $y \in Y$, then $\mathcal{F}_y$ is a finitely presented $\mathcal{O}_{Y,y}$-module.

Proof. Uses:

After passing to affine neighbourhoods, write the morphism as a finitely presented ring map $R \to A$ and the sheaf as a finite A-module M. The missing algebraic input is the pointwise flat finite-type module theorem: after an elementary etale neighbourhood of the image of y, it produces a neighbourhood of the lifted point on which M is finitely presented and base-flat. The local ring map back to $\mathcal{O}_{Y,y}$ is faithfully flat, so finite presentation descends. Expanding that pointwise theorem requires the complete-devissage induction from the earlier flat finite-type module chapter; it is not proved in the current manuscript. This exact theorem, rather than Tag 081N, is therefore recorded as the irreducible foundation leaf. □

Lemma 2.4.121 (Finite-presentation extension across a dense open, [1, Tag 081N]). *Open obligation:* `OBL-NAG-FLAT-RG-FINITE-PRESENTATION-EXTENSION`

Uses:

Lean: `Module.finitePresentation_of_flat_denseOpen`

Let $Y \to S$ be locally of finite presentation, let $U \subseteq S$ be retrocompact and scheme-theoretically dense, and let $\mathcal{F}$ be a finite-type quasi-coherent module, flat over S, whose restriction over U is finitely presented. Then $\mathcal{F}$ is finitely presented.

Proof. Uses: `lemma:nagata-flat-finite-type-module-pointwise-fp`

The question is affine local. Write $S = \operatorname{Spec}(R)$, $Y = \operatorname{Spec}(A)$, $U = \bigcup_{i=1}^{n} D(f_i)$, and let M be the finite A-module corresponding to $\mathcal{F}$. Schematic density is the injectivity of $R \to \prod_i R_{f_i}$.

Fix $\mathfrak{q} \in \operatorname{Spec}(A)$. The preceding pointwise theorem applies because M is R-flat, and gives that $M_{\mathfrak{q}}$ is finitely presented. Choose a finite free surjection $A^m \twoheadrightarrow M$. Because $M_{\mathfrak{q}}$ and each M_{f_i} are finitely presented, finitely many relations can be selected so that the quotient $M' \twoheadrightarrow M$ is an isomorphism after localization at $\mathfrak{q}$ and at every f_i. Openness of flatness for the finitely presented module M' gives $g \notin \mathfrak{q}$ for which M'_g is R-flat. Tensoring the injective map $R \to \prod_i R_{f_i}$ with M'_g gives an injection

$$M'_g \longrightarrow \prod_i M'_{gf_i}.$$

In the commutative square with the corresponding map for M_g, the bottom arrow is an isomorphism and the top arrow is already surjective. The left injection forces its kernel to vanish, so $M'_g \cong M_g$. Thus M is finitely presented near every prime $\mathfrak{q}$. Finite presentation is local on $\operatorname{Spec}(A)$, so these neighbourhoods prove the claim. The open token is inherited only from the pointwise theorem isolated above. □

Lemma 2.4.122 (Cartier factors do not change strict transform, [1, Tag 080H])**.** *Uses:*

Lean: `AlgebraicGeometry.strictTransform_mul_effectiveCartier`

Let $C, D \subseteq T$ be closed subschemes, with D an effective Cartier divisor, and let $\mathcal{G}$ be a quasi-coherent module on a T-scheme having no nonzero local section supported over D. Under the canonical isomorphism

$$\operatorname{Bl}_{C \cdot D}(T) \cong \operatorname{Bl}_C(T),$$

the strict transforms of $\mathcal{G}$ for the two displayed centres agree.

Proof. Uses: `thm:nagata-blowup-product-ideal,cor:nagata-blowup-cartier-iso,lemma:nagata-module-strict-transform-chart,lemma:nagata-power-torsion-localization`

The assertion is local both on T and on the T-scheme carrying $\mathcal{G}$. Work with $T = \operatorname{Spec}(A)$ and an affine source, let C be cut out by I, and write $D = V(x)$ with x a non-zero-divisor. On the chart indexed by $a \in I$, cancellation of x identifies

$$A[xI/(xa)] = A[I/a].$$

If M represents $\mathcal{G}$, its two strict-transform modules on this common chart are the quotient of $N = M \otimes_A A[I/a]$ by its a-power torsion and by its (xa)-power torsion. The first torsion submodule is contained in the second. Conversely, if $(xa)^n y = 0$, then after inverting a this says $x^n y = 0$ in $N_a \cong M_a$. The hypothesis makes M_a x-power-torsion-free, so $y/1 = 0$ and $a^m y = 0$ for some m. Thus the two torsion submodules agree. The chart identifications commute with localization and hence glue. □

Lemma 2.4.123 (Strict transform through an exact composite blow-up, [1, Tag 080I])**.** *Uses:*

Lean: `AlgebraicGeometry.strictTransform_comp_exactBlowup`

Let $T_1 \to T$ be the blow-up of $C \subseteq T$ and let $T_2 \to T_1$ be the blow-up of $C' \subseteq T_1$. Suppose that a closed subscheme $K \subseteq T$ satisfies

$$|K| = |C| \cup \operatorname{image}(|C'|)$$

and that the composite $T_2 \to T$ is identified with $\operatorname{Bl}_K(T)$. Given a T-scheme W, this identification canonically identifies its direct strict transform for K with its iterated strict transform for C and then C'.

Proof. Uses: `lemma:nagata-flattening-cartier-factor-strict-transform,lemma:nagata-power-torsion-quotient-torsionfree,prop:nagata-strict-transform-closed-fiber-product`

Let $E \subseteq T_1$ be the effective Cartier inverse image of C. The first strict-transform structure sheaf has no nonzero local section supported over E. By the preceding Cartier-factor lemma, its second strict transform may equally be computed after replacing the centre C' by $C' \cdot E$. The resulting quotient has no nonzero local section supported over $|C'| \cup |E|$.

Let $p : T_2 \to T$ and $p_1 : T_2 \to T_1$ denote the structure maps. Because $T_1 \to T$ is an isomorphism away from $|C|$, the equality defining K gives

$$p^{-1}(|K|) = p^{-1}(|C|) \cup p_1^{-1}(|C'|).$$

Hence, on $W \times_T T_2$, both the direct and iterated strict transforms are quotients of the pulled-back structure sheaf. The kernel of the iterated quotient is supported over the inverse image of $|K|$, while its quotient has no local section supported there. The affine power-torsion characterization therefore gives the same kernel as the direct strict-transform quotient. These equalities commute with chart localization, so they glue to the asserted canonical scheme isomorphism. □

Lemma 2.4.124 (Etale-local spreading of a blow-up centre)**.** *Open obligation:* `OBL-NAG-FLAT-RG-ETALE-CENTRE-SPREADING`

Uses:

Lean: `conditional-interface:AlgebraicGeometry.rgEtaleCentreSpreading`

Let $A \to B$ be etale, let $a \in A$ be a non-zero-divisor, and let $J \subseteq B$ be a finite-type ideal supported over $V(a)$. Locally near every prime of B, there are finite-type ideals $I \subseteq A$ and $J' \subseteq B$ with $V(I) \subseteq V(a)$ and

$$IB = JJ'.$$

Proof. Uses:

After one principal localization, present B as a localization of $C = A[x]/(F)$ with F monic. Lift finite generators $h_1, \dots, h_m$ of J to C. Since J is supported over $V(a)$, enlarge the list by an equality expressing a power a^N in their ideal. Define

$$I = \mathrm{Fit}_0\big(C/(h_1, \dots, h_m, a^N)\big)$$

as an A-module Fitting ideal. The quotient is finite over A and killed by a^N, so a power of a belongs to I. The multiplication section of $C \to C \otimes_A C$ splits the latter etale-locally into the diagonal factor and a complementary factor. Multiplicativity of zeroth Fitting ideals on this direct sum gives $IB = JJ'$ for the zeroth Fitting ideal of the complementary quotient. The missing token records the still unexpanded standard-etale presentation and complementary-factor construction; no global flattening conclusion is being imported here. □

Lemma 2.4.125 (Etale descent of finitely many local flattening blow-ups)**.** *Open obligation:* `OBL-NAG-FLAT-RG-ETALE-DESCENT`

Uses:

Lean: `conditional-interface:AlgebraicGeometry.rgEtaleFlatteningDescent`

For a finite quasi-compact etale cover of the base and a finite etale cover of the source, suppose each local pullback has a flattening U-admissible blow-up. Then one global U-admissible blow-up dominates all local ones and its strict transform is flat.

Proof. Uses: `lemma:nagata-rg-etale-centre-spreading,cor:nagata-blowup-product-common-refinement,thm:nagata-blowup-product-ideal,lemma:nagata-flattening-strict-transform-composition,cor:nagata-flat-strict-transform-base-change`

First pull every local centre through any preceding global blow-up. The etale spreading lemma gives, after a finite refinement of the etale cover, a finite-type ideal $\mathcal{I}_i$ on the global base whose pullback is the local centre times an auxiliary ideal. Blow up the product $\prod_i \mathcal{I}_i$. Its base change dominates each local blow-up by the product-ideal universal maps. For each factor, the product-ideal comparison presents the common refinement as a further blow-up of the local modification. The exact two-stage strict-transform comparison reduces its transform to the second transform of the locally flat module; the flat strict-transform corollary then identifies this with ordinary pullback. Flatness descends from the jointly surjective etale source cover. The token remains because the centre-spreading and descent data have not been constructed; the exact two-stage comparison is the preceding lemma. □

Lemma 2.4.126 (Fibre-local structure for the Raynaud–Gruson induction)**.** *Open obligation:* `OBL-NAG-FLAT-RG-FIBRE-LOCAL-STRUCTURE`

Uses:

Lean: `conditional-interface:AlgebraicGeometry.rgFibreLocalStructure`

Let $Y \to S$ be locally of finite type, let $\mathcal{F}$ be finite type, and let $y \in Y$ have fibre-support dimension d. There are elementary etale neighbourhoods $Y' \to Y$ and $S' \to S$, affine schemes Z' and P' over S', a finitely presented closed immersion $Z' \hookrightarrow Y'$, a finite map $Z' \to P'$, and a finite-type module $\mathcal{G}$ on Z' whose pushforward is the pullback of $\mathcal{F}$. The morphism $P' \to S'$ is smooth with geometrically integral fibres of dimension d.

Proof. Uses:

This is the second irreducible Raynaud–Gruson foundation leaf. Proving it requires the etale-local geometric presentation and fibre-dimension theory developed before Tag 081R. The current long manuscript contains only the invocation, not the construction of the etale neighbourhoods, the finite projection, or their comparison maps, so the node remains genuinely open. □

Lemma 2.4.127 (Dimension drop for the cokernel induction)**.** *Open obligation:* `OBL-NAG-FLAT-RG-COKERNEL-DIMENSION-DROP`

Uses:

Lean: `conditional-interface:Module.rgCokernel_supportDimension_lt`

In the smooth geometrically integral affine situation of the preceding local-structure lemma, after the generic-freeness blow-up choose $\alpha\colon \mathcal{O}_Y^r \to \mathcal{F}$ which is an isomorphism near the generic point of every maximal-dimensional fibre. Then α is universally injective and the fibre support of $\mathcal{H} = \operatorname{coker}(\alpha)$ has dimension strictly smaller than that of $\mathcal{F}$.

Proof. Uses: `lemma:nagata-rg-coefficient-chart-free`

Local freeness on an open meeting every fibre supplies the generic basis. At each fibre-generic point the local flatness criterion identifies universal injectivity with injectivity after the residue-field base change; the chosen generic basis gives the latter. Universal injectivity can be checked at all base primes, so it holds globally. The cokernel vanishes on a neighbourhood of every generic point of every support component of maximal fibre dimension. Its fibre support is therefore a proper closed subset of each such component and has strictly smaller dimension. The token remains because the generic-freeness chart theorem still depends on the open projective-algebra splitting. □

Lemma 2.4.128 (Exactness of strict transforms in the induction)**.** *Open obligation:* `OBL-NAG-FLAT-RG-STRICT-TRANSFORM-EXACT`

Uses:

Lean: `conditional-interface: Module.rgStrictTransform_exact`

If $0 \to \mathcal{O}_Y^r \xrightarrow{\alpha} \mathcal{F} \to \mathcal{H} \to 0$ has universally injective first map, then after every admissible blow-up the strict transforms fit into an exact sequence

$$0 \longrightarrow \mathcal{O}_{Y'}^r \longrightarrow \mathcal{F}' \longrightarrow \mathcal{H}' \longrightarrow 0.$$

Proof. Uses: `def:nagata-power-torsion,lemma:nagata-mem-power-torsion`

Work on a chart with exceptional equation a. Pullback preserves the first injection because universal injectivity is precisely injectivity after arbitrary tensor product. Quotient the middle and right modules by their a-power-torsion. If the image of $x \in \mathcal{F} \otimes R'$ becomes a-power torsion in the cokernel, then $a^n x$ lies in the free submodule, say it equals y. Universal injectivity remains injectivity after tensoring with $R'/(a^n)$; hence $y \in a^n \mathcal{O}_{Y'}^r$. Write $y = a^n z$. Then $x - z$ is a-power torsion and vanishes in the middle strict transform, which proves exactness there. Right exactness is immediate. The chart maps commute with localization, hence the sequences glue. The token records the unresolved global strict-transform constructor, not an omitted algebraic step. □

Lemma 2.4.129 (Raynaud–Gruson module flattening by dimension induction)**.** *Open obligation:* `OBL-NAG-FLAT-RG-MODULE-FLATTENING`

Uses: `def:nagata-rg-module-flattening-witness`

Lean: `conditional-interface: AlgebraicGeometry.exists_rgModuleFlatteningWitness`

Under the hypotheses of Definition 2.4.114, its witness type is nonempty.

Proof. Uses: `lemma:nagata-rg-fibre-local-structure,lemma:nagata-rg-coefficient-chart-free,lemma:nagata-rg-etale-flattening-descent,lemma:nagata-rg-cokernel-dimension-drop,lemma:nagata-rg-strict-transform-exact-sequence,lemma:nagata-rg-finite-presentation-extension`

Use etale descent to work affine locally. Induct on the maximum dimension of the fibre supports. The fibre-local structure lemma reduces a maximal-dimensional point to a finite module on a smooth affine scheme with geometrically integral fibres. The coefficient-ideal blow-up makes that module free on an open meeting every fibre. Choose the generic basis and form its cokernel. The dimension-drop lemma puts the cokernel under the induction hypothesis. The strict-transform exact sequence says that flattening the cokernel also flattens the original module. The etale descent lemma combines the finitely many local modifications. Finally, because the witness hypothesis makes U schematically dense, the finite-presentation extension lemma upgrades the flat finite-type strict transform to a finitely presented one. The listed open inputs are exactly the unresolved foundation leaves and their reductions; Tag 081R is not used as a black box. □

Lemma 2.4.130 (Collapse of a finite admissible blow-up chain)**.** *Open obligation:* `OBL-NAG-FLAT-RG-CHAIN-COLLAPSE`

Uses:

Lean: `conditional-interface: AlgebraicGeometry.rgFlatteningChainCollapse`

Let

$$S_n \longrightarrow S_{n-1} \longrightarrow \cdots \longrightarrow S_0 = S$$

be a finite chain of U-admissible blow-ups in finite-type ideals. Its composite is isomorphic over S to the blow-up of one finite-type ideal $\mathcal{I} \subseteq \mathcal{O}_S$ satisfying $\mathcal{I}|_U = \mathcal{O}_U$. The support $V(\mathcal{I})$ is the union of the set-theoretic images in S of the selected centres in the chain.

Proof. Uses: `thm:nagata-composition-of-blowups`

Induct on the length of the chain. The empty and one-arrow cases are immediate. For the induction step, apply the two-blow-up composition theorem to the ideal supplied for the first $n-1$ arrows and the last centre. The constructed centre has finite presentation. The selected centres are disjoint from U by admissibility. The support formula for the constructed centre is their union, hence is disjoint from U, so its ideal restricts to the unit ideal. The token records precisely the still-open finite-degree construction in 'OBL-NAG-COMPOSITE-CENTER'. □

Lemma 2.4.131 (Strict transform under collapse of a blow-up chain)**.** *Open obligation:* `OBL-NAG-FLAT-RG-CHAIN-STRICT-TRANSFORM`

Uses:

Lean: `conditional-interface: AlgebraicGeometry.rgFlatteningChainStrictTransformIso`

Under the comparison of the preceding lemma, the strict transform for the single blow-up is canonically isomorphic to the iterated strict transform along the finite chain, compatibly with the maps to S_n and to the original source.

Proof. Uses: `lemma:nagata-rg-flattening-chain-collapse,lemma:nagata-flattening-strict-transform-composition`

Induct on the chain length. At the last step, the collapse lemma supplies the finite-type composite centre, its support-union equality, and the comparison of the composite with its single blow-up. Apply the exact composite strict-transform lemma and then the induction hypothesis. The canonical comparisons preserve the structural maps at every step. The token is inherited exactly from the composite-centre construction. □

Lemma 2.4.132 (Finite common refinement of admissible blow-ups)**.** *Uses:*

Lean: `AlgebraicGeometry.finiteAdmissibleBlowups_commonRefinement`

Let S be qcqs and let $U \subseteq S$ be quasi-compact. For finitely many U-admissible blow-ups $S_i = \mathrm{Bl}_{\mathcal{I}_i}(S) \to S$, the blow-up of $\prod_i \mathcal{I}_i$ is U-admissible and has an S-morphism to every S_i; each comparison morphism is itself a U-admissible blow-up.

Proof. Uses: `thm:nagata-blowup-product-ideal,cor:nagata-blowup-product-common-refinement,lemma:nagata-product-ideal-common-blowup`

The empty family is handled by the identity modification. Otherwise set $\mathcal{I} = \prod_j \mathcal{I}_j$. This is finite type and is the unit ideal on U, so $\mathrm{Bl}_{\mathcal{I}}(S) \to S$ is U-admissible. For a fixed i, write $\mathcal{I} = \mathcal{I}_i\mathcal{J}_i$ with $\mathcal{J}_i = \prod_{j\neq i} \mathcal{I}_j$. The product-ideal theorem identifies $\mathrm{Bl}_{\mathcal{I}}(S)$ with the blow-up of $\mathcal{J}_i\mathcal{O}_{S_i}$ on S_i. This extended ideal is finite type and is the unit ideal over U; hence the comparison to S_i is a U-admissible blow-up. The common-refinement corollary identifies this comparison with the canonical domination map. Since i was arbitrary, all asserted comparisons follow. □

Lemma 2.4.133 (Strict transform commutes with affine pushforward, [1, Tag 080G])**.** *Uses:*

Lean: `QuasiCoherent.Sheaf.strictTransform_affinePushforward`

Let $g\colon W \to V$ be an affine morphism of schemes over S, let $S' \to S$ be a blow-up, and let g' be the base change of g. For a quasi-coherent module $\mathcal{G}$ on W, there is a canonical equality

$$g'_*\big(\mathcal{G}^{\mathrm{st}}\big) = \big(g_*\mathcal{G}\big)^{\mathrm{st}}.$$

Proof. Uses: `lemma:nagata-module-strict-transform-chart,lemma:nagata-power-torsion-localization`

The assertion is local on affine opens $\mathrm{Spec}(B) \subseteq V$ lying over $\mathrm{Spec}(A) \subseteq S$. Since g is affine, its inverse image is affine; let M be the module representing $\mathcal{G}$ there. On the a-chart of the blow-up, both sides are the $B \otimes_A A[I/a]$-module

$$\big(M \otimes_A A[I/a]\big)/(a\text{-power torsion}).$$

Affine pushforward is exact, and power torsion commutes with localization, so these equalities agree on overlaps and glue. □

Lemma 2.4.134 (Strict transform commutes with restriction on the source)**.** *Uses:*

Lean: `AlgebraicGeometry.StrictTransform.restrict_sourceOpen`

Let $W \subseteq Y$ be an open subscheme of an S-scheme and let $S' \to S$ be a blow-up. The strict transform of W is canonically the inverse image of W in the strict transform of Y.

Proof. Uses: `lemma:nagata-power-torsion-localization,prop:nagata-strict-transform-chart-iso,lemma:nagata-strict-transform-chart-cocycle`

Work on an affine open $\mathrm{Spec}(B) \subseteq Y$ over $\mathrm{Spec}(A) \subseteq S$. The principal opens $D(f)$ contained in W form a basis. If

$$C_a = (B \otimes_A A[I/a])/(a\text{-power torsion})$$

is the strict-transform ring on the a-chart, then localization of power torsion gives the strict-transform ring of $D(f)$ as $(C_a)_f$. Thus its transform is exactly the inverse-image principal open $D(f) \subseteq \mathrm{Spec}(C_a)$. These identifications agree on chart overlaps; gluing over the principal basis proves the assertion. □

Lemma 2.4.135 (Flat strict transforms remain flat under domination)**.** *Uses:*

Lean: `AlgebraicGeometry.flatStrictTransform_of_productDomination`

Let $S_i = \mathrm{Bl}_{\mathcal{I}_i}(S)$ and let

$$S' = \mathrm{Bl}_{\mathcal{J}_i\mathcal{O}_{S_i}}(S_i) \cong \mathrm{Bl}_{\mathcal{I}_i\mathcal{J}_i}(S).$$

If the strict transform of $W \to S$ on S_i is flat and locally of finite presentation, then its direct strict transform on S' has the same two properties.

Proof. Uses: `thm:nagata-blowup-product-ideal,lemma:nagata-flattening-strict-transform-composition,cor:nagata-flat-strict-transform-base-change`

The equality $V(\mathcal{I}_i\mathcal{J}_i) = V(\mathcal{I}_i) \cup V(\mathcal{J}_i)$, together with the fact that $S_i \to S$ is an isomorphism away from $V(\mathcal{I}_i)$, identifies this support with the union of the first centre and the image of the second centre. The product-ideal comparison and the exact composite strict-transform lemma identify the direct transform on S' with the second strict transform of the transform on S_i. Since that intermediate transform is flat over S_i, its second strict transform is its ordinary base change. Flatness and local finite presentation are stable under base change. □

Lemma 2.4.136 (Flattening the first strict-transform scheme, [1, Tag 081R])**.** *Open obligation:* `OBL-NAGATA-FLAT-081R-SCHEME`

Uses: `def:nagata-flattening-first-strict-transform`

Lean: `conditional-interface: AlgebraicGeometry.exists_flattening_Blowup`

There is a U-admissible blow-up $b_1 \colon X_1 \to X_0$ for which the iterated strict-transform projection $q_1 \colon Z_1 \to X_1$ is flat and of finite presentation.

Proof. Uses: `lemma:nagata-flattening-first-boundary-dense,lemma:nagata-flattening-first-blowup-qcqs,lemma:nagata-flattening-first-strict-transform-finite-type,lemma:nagata-flattening-first-strict-transform-quasi-separated,lemma:nagata-flattening-first-strict-transform-over-U,lemma:nagata-rg-module-flattening-exists,lemma:nagata-rg-flattening-chain-collapse,lemma:nagata-rg-flattening-chain-strict-transform,lemma:nagata-flattening-finite-common-refinement,lemma:nagata-flattening-flat-under-domination,lemma:nagata-flattening-affine-pushforward,lemma:nagata-flattening-strict-transform-source-open,lemma:nagata-strict-transform-ambient-closed`

The first blow-up is qcqs. Since q_0 is finite type and quasi-separated, Z_0 is qcqs. Cover Z_0 by finitely many affine opens. For one member W_i, choose an immersion into affine space over X_0 and a quasi-compact open V_i on which it is a closed immersion $i_i \colon W_i \hookrightarrow V_i$. Then V_i is quasi-compact and locally of finite presentation over X_0, while $(i_i)_*\mathcal{O}_{W_i}$ is a finite-type module.

The open $q_0^{-1}(U) \cong U$ is quasi-compact in the quasi-separated scheme Z_0, so $W_i \cap U$ is quasi-compact. Its map to U is a quasi-compact open immersion and hence is flat and of finite presentation. After base change to U, the closed immersion i_i is of finite presentation: both its source and ambient are locally of finite presentation over U, and its source is quasi-compact. Consequently $((i_i)_*\mathcal{O}_{W_i})|_U$ is flat over U and finitely presented on $V_i \times_{X_0} U$. Thus all hypotheses of the module-flattening theorem are satisfied.

Apply that theorem to $(i_i)_*\mathcal{O}_{W_i}$. The affine-pushforward comparison identifies its strict transform with the pushforward of the structure sheaf of the strict-transform scheme W_i^{st}. Base-flatness of this module is exactly flatness of W_i^{st} over the modified base. Its finite presentation makes $W_i^{\mathrm{st}} \hookrightarrow V_i'$ a closed immersion of finite presentation; since V_i' is locally of finite presentation over the modified base, W_i^{st} is locally of finite presentation there. For each i this first gives a finite admissible chain. Collapse each chain to one blow-up and transport its iterated strict transform through the chain comparison.

Blow up the product of these finitely many centres. The finite common refinement lemma supplies maps to every local flattening blow-up. The domination lemma makes the strict transform of every affine member flat and locally finitely presented on this common blow-up. The source-open restriction lemma identifies these transforms with the inverse images of the W_i inside Z_1. They therefore cover Z_1, so flatness and local finite presentation hold for q_1. Finally, the closed-fibre-product description makes q_1 finite type and quasi-separated, because q_0 has those properties. Thus q_1 is quasi-compact, quasi-separated, and locally of finite presentation, hence is of finite presentation. This reproduces every reduction in Tag 081R; the node remains unresolved only because its listed atomic inputs remain unresolved. □

Lemma 2.4.137 (Separatedness after the flattening blow-up). *Open obligation:* `OBL-NAG-FLAT-SECOND-STRICT-SEPARATED`

Uses:

Lean: `conditional-interface: AlgebraicGeometry.flatteningSecondStrictTransform_separated`

For a witness $b_1 \colon X_1 \to X_0$ and $q_1 \colon Z_1 \to X_1$ supplied by the preceding flattening statement, the morphism q_1 is separated.

Proof. Uses: `lemma:nagata-flattening-first-strict-transform-separated,lemma:nagata-flattening-strict-transform-exists,lemma:nagata-strict-transform-ambient-closed`

The fibre-product projection $Z_0 \times_{X_0} X_1 \to X_1$ is separated by base change from q_0. The iterated strict transform is a closed subscheme of this fibre product. A closed immersion is separated, and separated morphisms are stable under composition. The token is inherited from the global strict-transform constructor and witness existence. □

Lemma 2.4.138 (The second strict transform is unchanged over U). *Open obligation:* `OBL-NAG-FLAT-SECOND-STRICT-OVER-U`

Uses:

Lean: `conditional-interface: AlgebraicGeometry.flatteningSecondStrictTransform_over_open`

For a witness b_1, q_1 supplied by the flattening statement, the restriction $(q_1)_U$ is the originally chosen isomorphism $q^{-1}(U) \cong U$.

Proof. Uses: `lemma:nagata-flattening-first-strict-transform-over-U,lemma:nagata-flattening-strict-transform-exists,lemma:nagata-strict-transform-restrict`

Admissibility makes b_1 the identity over U. Strict-transform restriction therefore identifies $(Z_1)_U$ with $(Z_0)_U$ and its projection with $(q_0)_U$. The first comparison over U identifies the latter with the originally chosen isomorphism. The token is inherited from the exact global restriction comparison and witness existence. □

Lemma 2.4.139 (Pullback of an effective Cartier divisor by a blow-up, [1, Tag 0809]). *Uses:*

Lean: `AlgebraicGeometry.Blowup.pullback_effectiveCartierDivisor`

If $D \subseteq T$ is an effective Cartier divisor and $b \colon T' \to T$ is a blow-up, then $b^{-1}D$ is an effective Cartier divisor on T'.

Proof. Uses: `def:nagata-blowup-chart-ring`

Work on a standard chart $A[I/a] \subseteq A_a$. If $x \in A$ is a regular local equation for D and $x(y/a^n) = 0$ in $A[I/a]$, then for some m one has $a^m xy = 0$ in A. Regularity of x gives $a^m y = 0$, hence $y/a^n = 0$ in the chart ring. Thus the image of x is again a non-zero-divisor. Pullbacks of two local equations differ by the pullback of a unit, so the local effective Cartier divisors glue. □

Lemma 2.4.140 (Support of the pulled-back boundary divisor). *Uses:*

Lean: `AlgebraicGeometry.flatteningSecondBoundary_support`

Let $b_1 \colon X_1 \to X_0$ be a U-admissible blow-up and let E_0 have support $X_0 \setminus U$. Then

$$|b_1^{-1}E_0| = X_1 \setminus U.$$

Proof. Uses:

Set-theoretic inverse image commutes with complements. Since b_1 is an isomorphism over U, the inverse image of U is the selected copy of U in X_1. Therefore $b_1^{-1}(X_0 \setminus U) = X_1 \setminus U$, which is the displayed support. □

Lemma 2.4.141 (Density after the flattening blow-up). *Uses:*

Lean: `AlgebraicGeometry.flatteningSecondBlowup_open_dense`

In the situation of the preceding lemma, $U \subseteq X_1$ is retrocompact and scheme-theoretically dense.

Proof. Uses: `lemma:nagata-blowup-pullback-effective-cartier,lemma:nagata-flattening-second-boundary-support,lemma:nagata-effective-cartier-complement-dense`

The pullback $b_1^{-1}E_0$ is effective Cartier and its complement is U. Apply the effective-Cartier complement lemma. □

Lemma 2.4.142 (Flat pullback preserves schematic density, [1, Tag 081H]). *Uses:*

Lean: `AlgebraicGeometry.Scheme.Hom.schematicallyDense_preimage_of_flat`

If $p\colon T \to S$ is flat and $U \subseteq S$ is retrocompact and scheme-theoretically dense, then $p^{-1}(U)$ is scheme-theoretically dense in T.

Proof. Uses:

It suffices to work with affine opens $\operatorname{Spec}(B) \subseteq T$ mapping into $\operatorname{Spec}(A) \subseteq S$. Write $U \cap \operatorname{Spec}(A) = \bigcup_{i=1}^{n} D(f_i)$. Schematic density is the injectivity of $A \to \prod_i A_{f_i}$. Tensor this injection with the flat A-module B to obtain

$$B \longrightarrow \prod_i B_{f_i}.$$

This is exactly the affine schematic-density criterion for the inverse image of U. □

Lemma 2.4.143 (The diagonal is an isomorphism from dense-open agreement). *Uses:*

Lean: `AlgebraicGeometry.Scheme.Hom.diagonal_iso_of_flat_dense_open_iso`

Let $f\colon Y \to S$ be separated and flat. If a retrocompact schematically dense open $U \subseteq S$ pulls back isomorphically, then $\Delta_f\colon Y \to Y \times_S Y$ is an isomorphism; hence f is a monomorphism.

Proof. Uses: `lemma:nagata-flat-pullback-schematic-density`

The morphism $Y \times_S Y \to S$ is flat, so the inverse image $W = (Y \times_S Y)_U$ is scheme-theoretically dense. Separatedness makes Δ_f a closed immersion, say with quasi-coherent ideal $\mathcal{K}$. Over U, the map f_U is an isomorphism, so $(\Delta_f)_U$ is an isomorphism and $\mathcal{K}|_W = 0$. Schematic density says that $\mathcal{O}_{Y\times_S Y} \to j_*\mathcal{O}_W$ is injective. Since the composite $\mathcal{K} \hookrightarrow \mathcal{O}_{Y\times_S Y} \to j_*\mathcal{O}_W$ is zero, this injection forces $\mathcal{K} = 0$. Thus the closed immersion is the identity closed subscheme. A morphism whose diagonal is an isomorphism is a monomorphism. □

Lemma 2.4.144 (Finite presentation extends for a flat morphism, [1, Tag 081P]). *Open obligation:* `OBL-NAG-FLAT-DENSE-OPEN-LFP`

Uses:

Lean: `conditional-interface: AlgebraicGeometry.Scheme.Hom.locallyFinitePresentation_of_flat_denseOpen`

Let $f\colon Y \to S$ be flat and locally of finite type. If $U \subseteq S$ is retrocompact and scheme-theoretically dense and f_U is locally of finite presentation, then f is locally of finite presentation.

Proof. Uses: `lemma:nagata-rg-finite-presentation-extension`

The assertion is local on source and target. For affine Y and S, use the finite-type generators to factor f as a closed immersion

$$i\colon Y \hookrightarrow \mathbf{A}^n_S$$

followed by the affine-space projection. As a module on $\mathbf{A}^n_S$, the sheaf $i_*\mathcal{O}_Y$ is finite type and flat over S. Over U it is finitely presented: the assumed finite presentation of f_U says exactly that the kernel of the chosen polynomial presentation is locally finitely generated. Apply the dense-open module extension lemma to conclude that $i_*\mathcal{O}_Y$ is finitely presented. Hence i is a finitely presented closed immersion, and its composite with $\mathbf{A}^n_S \to S$ is locally of finite presentation. The open token is inherited solely from the pointwise module theorem beneath Tag 081N. □

Lemma 2.4.145 (A flat finitely presented monomorphism is open, [1, Tag 025G]). *Uses:*

Lean: `AlgebraicGeometry.Scheme.Hom.isOpenImmersion_of_flat_monomorphism`

A flat monomorphism locally of finite presentation is an open immersion.

Proof. Uses:

A flat morphism locally of finite presentation is open. Its image is therefore an open subscheme $V \subseteq S$. After replacing the target by V, the map is surjective. A monomorphism is universally injective, so the resulting open surjection is bijective on points and hence a homeomorphism; in particular it is quasi-compact. It is now an fpqc cover. After base change along itself, the projection $Y \times_V Y \to Y$ is an isomorphism because the diagonal is an isomorphism. Being an isomorphism descends along an fpqc cover, so $Y \to V$ is an isomorphism. Composing with $V \hookrightarrow S$ proves that the original map is an open immersion. □

Lemma 2.4.146 (Separated flat finitely-presented dense-open criterion). *Uses:*

Lean: `AlgebraicGeometry.Scheme.Hom.isOpenImmersion_of_flat_fp_denseOpen`

Let $f\colon Y \to S$ be separated, flat, and locally of finite presentation. If a retrocompact schematically dense open $U \subseteq S$ pulls back isomorphically, then f is an open immersion.

Proof. Uses: `lemma:nagata-flat-dense-diagonal-is-iso,lemma:nagata-flat-fp-monomorphism-open`
Dense-open agreement and separatedness make the diagonal an isomorphism, so f is a monomorphism. The flat finitely-presented monomorphism criterion then makes f an open immersion. No form of Tag 081M or of the dense-open finite-presentation extension is imported here. □

Lemma 2.4.147 (Separated flat dense-open criterion, [1, Tag 081M]). *Open obligation:* `OBL-NAG-FLAT-081M-DENSE-OPEN`

Uses:

Lean: `Scheme.Hom.isOpenImmersion_of_flat_dense_open`
Let $f\colon Y \to S$ be separated, flat, and locally of finite type. If a retrocompact schematically dense open $U \subseteq S$ pulls back isomorphically, then f is an open immersion.

Proof. Uses: `lemma:nagata-flat-dense-open-lfp,lemma:nagata-flat-fp-dense-open-is-open-immersion`
The dense-open finite-presentation extension gives local finite presentation. Apply the preceding finitely-presented criterion. Thus Tag 081M is reduced to the displayed atomic chain, not imported as a terminal citation; its only inherited foundation leaf is the pointwise flat-module theorem. □

Definition 2.4.148 (Elimination witness predicate). Uses:

Lean: `AlgebraicGeometry.IsEliminationIndeterminacyWitness`
For $q\colon Z \to X$ and $U \subseteq X$, an elimination witness consists of a finite-type quasi-coherent ideal $\mathcal{F} \subseteq \mathcal{O}_X$, an isomorphism $\mathcal{F}|_U \cong \mathcal{O}_U$, and the blow-up $b\colon \mathrm{Bl}_{\mathcal{F}}(X) \to X$, together with an open immersion from the strict transform of Z into that blow-up and a comparison over U whose forward map, inverse, and two inverse equations agree with the originally chosen isomorphism $q^{-1}(U) \cong U$.

Lemma 2.4.149 (Collapse of the two modifications to one admissible blow-up). *Open obligation:* `OBL-NAG-FLAT-COMPOSITE-CENTRE`

Uses:

Lean: `conditional-interface:AlgebraicGeometry.flatteningCollapseTwoBlowups`
The composite $b_0 \circ b_1\colon X_1 \to X$ is isomorphic over X to the blow-up of one finite-type ideal $\mathcal{F}_U(q) \subseteq \mathcal{O}_X$. If $C_1 \subseteq X_0$ is the selected centre of b_1, then

$$|V(\mathcal{F}_U(q))| = |V(\mathcal{I}_0)| \cup b_0(|C_1|).$$

This ideal is the unit ideal on U.

Proof. Uses: `lemma:nagata-admissible-blowup-comp,thm:nagata-composition-of-blowups`
The composition theorem constructs a finite-presentation centre on X whose blow-up is the displayed composite. Its support is the union of the first centre and the image of the second centre. Both selected centres are disjoint from U by admissibility. Therefore its finite-type defining ideal restricts to the unit ideal on U. The token is retained because the exact finite-degree centre used by the composition theorem is still open in 'OBL-NAG-COMPOSITE-CENTER'. □

Lemma 2.4.150 (Strict transform comparison for the collapsed blow-up). *Open obligation:* `OBL-NAG-FLAT-COMPOSITE-STRICT-TRANSFORM`

Uses:

Lean: `conditional-interface:AlgebraicGeometry.flatteningCollapsedStrictTransformIso`
Under the preceding comparison, the strict transform for the single blow-up is canonically isomorphic to the iterated strict transform Z_1, and the isomorphism commutes with the projections to X_1 and Z.

Proof. Uses: `lemma:nagata-flattening-collapse-two-blowups,lemma:nagata-flattening-strict-transform-composition`
The collapse lemma supplies the finite-type composite centre, the equality of its support with the union of the two selected centres, and the comparison of its blow-up with the composite. Apply the exact composite strict-transform lemma. Its canonical comparison commutes with both structural projections. The token is inherited only from the unresolved composite-centre construction. □

Lemma 2.4.151 (Comparison of the collapsed construction over U). *Open obligation:* `OBL-NAG-FLAT-COMPARISON-OVER-U`

Uses:

Lean: `conditional-interface:AlgebraicGeometry.flatteningCollapsedComparisonOverOpen`
The restriction of the collapsed blow-up and its strict transform over U is the originally chosen isomorphism $q^{-1}(U) \cong U$; the selected forward and inverse maps satisfy both inverse equations.

Proof. Uses: lemma:nagata-flattening-second-strict-transform-over-U,lemma:nagata-flattening-collapsed-strict-transform

The second-transform comparison identifies the iterated transform over U with $q^{-1}(U)$ and its projection with $q|_{q^{-1}(U)}$. Transport these maps across the collapsed strict-transform isomorphism. The two inverse equations are preserved by transport, so they are exactly the inverse equations of the original chosen isomorphism. The node remains unresolved only through the two comparison constructors explicitly listed above. □

Lemma 2.4.152 (Existence of an elimination witness)**.** *Open obligation:* OBL-NAG-FLAT-EXISTS-ELIMINATION-WITNESS

Uses: def:nagata-elimination-indeterminacy-predicate

Lean: conditional-interface:AlgebraicGeometry.exists_eliminationIndeterminacyWitness

Let $q\colon Z \to X$ be separated and of finite type, let X be qcqs, and let $U \subseteq X$ be quasi-compact with a chosen isomorphism $q^{-1}(U) \cong U$. The type of elimination witnesses is nonempty.

Proof. Uses: def:nagata-flattening-first-blowup,def:nagata-flattening-first-strict-transform,lemma:nagata-flattening-first-strict-transform-finite-type,lemma:nagata-flattening-first-strict-transform-quasi-separated,lemma:nagata-flattening-first-strict-transform-over-U,lemma:nagata-flattening-strict-transform-exists,lemma:nagata-flattening-second-strict-transform-separated,lemma:nagata-flattening-second-strict-transform-over-U,lemma:nagata-flattening-second-boundary-dense,lemma:nagata-flat-fp-dense-open-is-open-immersion,lemma:nagata-flattening-collapse-two-blowups,lemma:nagata-flattening-collapsed-strict-transform,lemma:nagata-flattening-collapsed-comparison-over-U

The first modification makes U retrocompact and schematically dense. Its strict-transform projection remains separated, finite type, and unchanged over U. The expanded Raynaud–Gruson chain supplies the second U-admissible blow-up and makes the iterated strict transform flat and finitely presented. The two separate comparison lemmas make this second projection separated and unchanged over U. Pullback of the first exceptional divisor makes U schematically dense after the second blow-up. The closed finite-presentation form of the 081M chain then makes the iterated strict-transform projection an open immersion. Finally, collapse the two modifications to one finite-type centre on X, transport the strict transform, and use the explicit comparison over U. These are exactly the fields of the witness predicate. The open token is inherited from the three genuine construction leaves, not from an appeal to 081S. □

Definition 2.4.153 (Selected elimination-of-indeterminacy construction)**.** Open obligation: OBL-NAG-FLAT-SELECT-ELIMINATION-WITNESS

Uses: def:nagata-elimination-indeterminacy-predicate

Constructed by: lemma:nagata-elimination-indeterminacy-exists

Lean: AlgebraicGeometry.eliminationIndeterminacyConstruction

Select the first elimination witness in the fixed well-order. Denote its ideal by $\mathcal{F}_U(q)$, its blow-up by $X^{\mathrm{el}}(q,U)$, its strict transform by $Z^{\mathrm{el}}(q,U)$, and its strict-transform projection by q^{el}. The token prevents this selection from being imported before witness nonemptiness is closed.

Lemma 2.4.154 (Elimination of indeterminacy, [1, Tag 081S])**.** *Open obligation:* OBL-NAG-FLAT-ELIMINATION-ASSEMBLY

Uses: def:nagata-elimination-indeterminacy

Lean: AlgebraicGeometry.exists_admissibleBlowup_strictTransform_open

The morphism $X^{\mathrm{el}}(q,U) \to X$ is U-admissible, the constructed projection $q^{\mathrm{el}}\colon Z^{\mathrm{el}}(q,U) \to X^{\mathrm{el}}(q,U)$ is an open immersion, and its restriction over U is the originally chosen isomorphism $q^{-1}(U) \cong U$.

Proof. Uses: lemma:nagata-elimination-indeterminacy-exists

These are the admissibility, open-immersion, and comparison fields of the selected witness. The existence proof above constructs those fields by the boundary blow-up, the expanded Raynaud–Gruson chain, the atomic dense-open criterion, and the composite-centre comparison. Thus the final statement performs record projection only; its token is inherited from witness nonemptiness. □

Lemma 2.4.155 (A proper dense open immersion is an isomorphism)**.** *Uses:*

Lean: Scheme.Hom.isIso_of_isProper_isOpenImmersion_dense

Let $q\colon Z \to X$ be both proper and an open immersion. If its image contains a scheme-theoretically dense open subscheme of X, then q is an isomorphism.

Proof. Uses:

A proper immersion is a closed immersion. Thus the image is both open and closed. Its closed complement has structure sheaf supported away from the dense open; scheme-theoretic density forces that complement to be empty. □

2.5 Separating boundary pieces

Lemma 2.5.1 (Finite-presentation closed immersions and finite type ideals)**.** *Uses:*

Lean: `Scheme.Hom.closedImmersion_finitePresentation_iff_ideal_finiteType`

A closed immersion $Z \hookrightarrow X$ is of finite presentation if and only if its defining quasi-coherent ideal sheaf is of finite type.

Proof. Uses:

The assertion is local on X. On $\operatorname{Spec}(A)$ the closed immersion is $\operatorname{Spec}(A/I) \to \operatorname{Spec}(A)$. The quotient A/I is a finitely presented A-algebra exactly when I is finitely generated as an ideal. Translate this equivalence back to the associated quasi-coherent ideal sheaf. □

Definition 2.5.2 (Radical ideal sheaf of a closed complement)**.** Uses:

Lean: `Scheme.closedComplementRadicalIdeal`

Let $W \subseteq Y$ be open and put $Z = Y \setminus W$. On an affine open $V = \operatorname{Spec}(A)$ define

$$I_Z(V) = \bigcap_{\mathfrak{p} \in Z \cap V} \mathfrak{p} \subseteq A.$$

Localization of these radical ideals supplies the restriction maps; denote the resulting ideal sheaf by $\mathcal{I}_Z^{\mathrm{rad}}$.

Lemma 2.5.3 (Properties of the closed-complement radical ideal)**.** *Uses:* `def:nagata-closed-complement-radical-ideal`

Lean: `Scheme.closedComplementRadicalIdeal_spec`

The affine ideals defining $\mathcal{I}_Z^{\mathrm{rad}}$ agree under localization, so this is a quasi-coherent radical ideal sheaf, and

$$V(\mathcal{I}_Z^{\mathrm{rad}}) = Z, \qquad \mathcal{I}_Z^{\mathrm{rad}}|_W = \mathcal{O}_W.$$

Proof. Uses:

If $V = \operatorname{Spec}(A)$ and $D(s) \subseteq V$, then

$$Z \cap D(s) = V(I_Z(V)A_s)$$

inside $\operatorname{Spec}(A_s)$. Taking the intersection of the primes containing an ideal is its radical, and radical commutes with localization; hence $I_Z(D(s)) = I_Z(V)A_s$. Principal opens form a basis, so these equalities give the identity and triple localization cocycles and glue the affine modules to a quasi-coherent radical ideal.

On each affine V, the zero set of the intersection of the primes in $Z \cap V$ is exactly the closed subset $Z \cap V$. Thus the global zero set is Z. Its intersection with W is empty, which is equivalent to the restricted radical ideal being the unit ideal. □

Definition 2.5.4 (Finite-type ideal attached to a quasi-compact open)**.** Uses: `def:nagata-closed-complement-radical-ideal`

Lean: `Scheme.qcOpenComplementIdeal`

Let $W \subseteq Y$ be a quasi-compact open in a qcqs scheme. On a finite affine cover $Y = \bigcup Y_r$, write $W \cap Y_r = \bigcup_k D(f_{rk})$. Since every $D(f_{rk})$ is disjoint from $Z = Y \setminus W$, the corresponding local section of a suitable positive power of f_{rk} belongs to $\mathcal{I}_Z^{\mathrm{rad}}$. Apply the finite affine extension induction to the submodule $\mathcal{O}_W = \mathcal{I}_Z^{\mathrm{rad}}|_W$, clearing the selected denominators at each newly adjoined affine chart. Define $\mathcal{I}_Y(W) \subseteq \mathcal{I}_Z^{\mathrm{rad}}$ to be the finite type quasi-coherent ideal generated by the synchronized numerators.

Lemma 2.5.5 (Vanishing set of the open-complement ideal)**.** *Uses:* `def:nagata-qc-open-complement-ideal`

Lean: `Scheme.zeroLocus_qcOpenComplementIdeal`

The ideal $\mathcal{I}_Y(W)$ is finite type and quasi-coherent, and

$$V(\mathcal{I}_Y(W)) = Y \setminus W.$$

Proof. Uses: `lemma:nagata-closed-complement-radical-ideal,lemma:nagata-affine-cleared-denominator-extension,lemma:nagata-qcqs-finite-type-submodule-extension`

First verify the selected local generators. On $Y_r = \operatorname{Spec}(A_r)$, containment $D(f_{rk}) \subseteq W$ gives

$$V(I_Z(Y_r)) \subseteq V(f_{rk}),$$

hence $f_{rk} \in \sqrt{I_Z(Y_r)} = I_Z(Y_r)$. Allowing a positive power is the equivalent cleared-denominator form used on a refinement. Apply Lemma 2.4.77 to $\mathcal{M} = \mathcal{I}_Z^{\mathrm{rad}}$ and $\mathcal{F} = \mathcal{O}_W$. The affine induction in its proof is exactly the

selected construction above, so it produces a finite type quasi-coherent subideal whose restriction to W is the unit ideal.

Since $\mathcal{I}_Y(W) \subseteq \mathcal{I}_Z^{\mathrm{rad}}$, its zero set contains Z. On the other hand, the selected f_{rk} generate the unit ideal at every point of $\bigcup_k D(f_{rk}) = W \cap Y_r$, so the zero set is disjoint from W. Therefore $V(\mathcal{I}_Y(W)) = Z = Y \setminus W$. Pairwise overlap maps are equality inside $\mathcal{I}_Z^{\mathrm{rad}}$; identity and associativity of localization give the identity and triple cocycles. The finite affine induction gives finite type and quasi-coherence. □

Lemma 2.5.6 (Chart separation for a zero product)**.** *Uses:*

Lean: `AlgebraicGeometry.Blowup.zeroProduct_chartSeparation`

Let A be a ring, let $I = (a_1, \dots, a_r)$ and $J = (b_1, \dots, b_s)$ be finite type ideals with $IJ = 0$, and put $K = I + J$. In $\mathrm{Bl}_K(\mathrm{Spec}(A))$, the standard charts indexed by the a_i are disjoint from those indexed by the b_j. On every a_i-chart the extended ideal J is zero, and on every b_j-chart the extended ideal I is zero. The two chart families together cover the blow-up.

Proof. Uses: `prop:nagata-affine-blowup-chart,prop:nagata-affine-blowup-chart-cover`

On the a_i-chart the extended center ideal $K\mathcal{O}$ is generated by a_i and is invertible. Thus multiplication by a_i is injective. For every $b \in J$, the equality $a_i b = 0$ therefore gives $b = 0$ on that chart; hence $J\mathcal{O} = 0$. The symmetric argument gives $I\mathcal{O} = 0$ on every b_j-chart.

If an a_i-chart met a b_j-chart, then on the intersection both a_i and b_j would generate the same invertible center ideal. The first chart makes $b_j = 0$, while the second makes it a regular generator, forcing the intersection to be empty. Finally the standard-chart cover theorem says that the charts indexed by the chosen generators of $K = I + J$ cover the blow-up. □

Lemma 2.5.7 (Chinese-remainder decomposition of the product center over U)**.** *Uses:*

Lean: `AlgebraicGeometry.productCenter_restrict_disjointUnion`

Let X be qcqs, let $U \subseteq X$ be quasi-compact, let $Z_i = V(\mathcal{I}_i) \hookrightarrow X$ be closed immersions of finite presentation, and assume $Z_1 \cap Z_2 \cap U = \varnothing$. For $Z = V(\mathcal{I}_1\mathcal{I}_2)$, restriction to U gives a canonical scheme isomorphism

$$Z \times_X U \cong (Z_1 \times_X U) \amalg (Z_2 \times_X U).$$

After composition with the open immersion $Z \times_X U \hookrightarrow Z$, the two summands are disjoint quasi-compact open subschemes of Z; each is open and closed inside $Z \times_X U$.

Proof. Uses:

The empty intersection says $(\mathcal{I}_1 + \mathcal{I}_2)|_U = \mathcal{O}_U$. For comaximal ideals, $\mathcal{I}_1 \cap \mathcal{I}_2 = \mathcal{I}_1\mathcal{I}_2$, and the Chinese remainder map is an isomorphism

$$\mathcal{O}_U/(\mathcal{I}_1\mathcal{I}_2) \xrightarrow{\sim} \mathcal{O}_U/\mathcal{I}_1 \times \mathcal{O}_U/\mathcal{I}_2.$$

Taking relative spectra gives the displayed disjoint union. The two summand immersions are open and closed in $Z \times_X U$, hence open in Z. They are quasi-compact because U is quasi-compact, X is quasi-separated, and the $Z_i \to X$ are closed immersions of finite presentation. □

Definition 2.5.8 (Closed-boundary separation blow-up)**.** Uses: `def:nagata-qc-open-complement-ideal,def:nagata-global-blowup,def:nagata-strict-transform`

Lean: `AlgebraicGeometry.closedBoundarySeparationBlowup`

Let X be qcqs, let $U \subseteq X$ be quasi-compact, and let $Z_i = V(\mathcal{I}_i) \hookrightarrow X$ be closed immersions of finite presentation with $Z_1 \cap Z_2 \cap U = \varnothing$. Put

$$Z = V(\mathcal{I}_1\mathcal{I}_2), \qquad P_i = Z_i \cap U \subseteq Z.$$

The Chinese-remainder decomposition makes the P_i disjoint quasi-compact open subschemes of Z, each open and closed in $Z \times_X U$, with $Z \times_X U = P_1 \amalg P_2$. Set $\mathcal{A}_i = \mathcal{I}_Z(P_i)$. Since $V(\mathcal{A}_1\mathcal{A}_2) = Z$, choose the least $n > 0$ for which $\mathcal{A}_1^n\mathcal{A}_2^n = 0$, and define

$$\mathcal{K} = \mathcal{A}_1^n + \mathcal{A}_2^n, \qquad Y = V(\mathcal{K}) \hookrightarrow Z \hookrightarrow X.$$

Define $\mathsf{SepBl}_U(Z_1, Z_2) \to X$ to be the blow-up of this explicitly constructed composite center $Y \hookrightarrow X$.

Lemma 2.5.9 (Finite-presentation certificate for the boundary-separation center)**.** *Uses:* `def:nagata-separated-closed-subschemes`

Lean: `AlgebraicGeometry.closedBoundarySeparationCenterIdeal_spec`

In the notation of Definition 2.5.8, the ideals $\mathcal{A}_1$ and $\mathcal{A}_2$ are finite type, the set

$$\{m > 0 \mid \mathcal{A}_1^m\mathcal{A}_2^m = 0\}$$

is nonempty, and its least element n has the following properties:

$$\mathcal{K} = \mathcal{A}_1^n + \mathcal{A}_2^n \quad \text{is finite type}, \qquad V(\mathcal{K}) = Z \setminus (P_1 \amalg P_2).$$

Consequently the composite center $Y = V(\mathcal{K}) \hookrightarrow Z \hookrightarrow X$ is a closed immersion of finite presentation.

Proof. Uses: lemma:nagata-product-center-open-disjoint-union,lemma:nagata-qc-open-complement-ideal,lemma:nagata-closed-fp-ideal-finite-type

By Lemma 2.5.7, $Z \times_X U = P_1 \amalg P_2$ as schemes, and the two summands are disjoint quasi-compact opens of Z. By Lemma 2.5.5, each $\mathcal{A}_i$ is a finite-type quasi-coherent ideal and $V(\mathcal{A}_i) = Z \setminus P_i$. Since $P_1 \cap P_2 = \varnothing$,

$$V(\mathcal{A}_1\mathcal{A}_2) = V(\mathcal{A}_1) \cup V(\mathcal{A}_2) = Z.$$

Thus $\mathcal{L} = \mathcal{A}_1\mathcal{A}_2$ is locally nilpotent. More explicitly, on an affine chart $\operatorname{Spec}(A) \subseteq Z$ the finitely generated ideal $L = \mathcal{L}(\operatorname{Spec}(A))$ lies in $\sqrt{(0)}$. Choose nilpotence exponents for a finite generating set of L and take their sum; the pigeonhole argument for a product of that many generators gives $L^e = 0$. A finite affine cover of the quasi-compact scheme Z and the maximum of its exponents give one $e > 0$ with $\mathcal{L}^e = 0$. Hence the displayed set of positive integers is nonempty, and well-ordering supplies its least element n; moreover $\mathcal{A}_1^n\mathcal{A}_2^n = (\mathcal{A}_1\mathcal{A}_2)^n = 0$.

Powers and finite sums of finite-type ideals are finite type, so $\mathcal{K}$ is finite type. Moreover

$$V(\mathcal{K}) = V(\mathcal{A}_1) \cap V(\mathcal{A}_2) = (Z \setminus P_1) \cap (Z \setminus P_2) = Z \setminus (P_1 \amalg P_2).$$

In particular the center is disjoint from $Z \times_X U = P_1 \amalg P_2$.

Finally $\mathcal{I}_1\mathcal{I}_2$ is finite type, so $Z \hookrightarrow X$ is of finite presentation by Lemma 2.5.1; the same lemma applies to $Y = V(\mathcal{K}) \hookrightarrow Z$. Closed immersions of finite presentation are stable under composition, proving the last assertion. □

Lemma 2.5.10 (Properties of the closed-boundary separation blow-up, [1, Tag 0F3V])**.** Uses: def:nagata-separate-closed-subschemes

Lean: AlgebraicGeometry.exists_admissibleBlowup_disjoint_strictTransforms

The morphism $\mathsf{SepBl}_U(Z_1, Z_2) \to X$ *is* U*-admissible, and the strict transforms of* Z_1 *and* Z_2 *in it are disjoint.*

Proof. Uses: lemma:nagata-boundary-separation-center-ideal,lemma:nagata-zero-product-blowup-chart-separation,lemma:nagata-strict-transform-ambient-closed

By Lemma 2.5.9, the selected center is a finite-presentation closed subscheme disjoint from U. The complement-of-center isomorphism for the blow-up therefore makes the structure morphism U-admissible.

First work in Z. The strict transform of Z in the ambient blow-up is canonically

$$Z' = \mathrm{Bl}_{\mathcal{K}}(Z).$$

Put $I = \mathcal{A}_1^n$ and $J = \mathcal{A}_2^n$. They are finite type, $IJ = 0$, and $\mathcal{K} = I + J$. Applying Lemma 2.5.6 on a finite affine cover of Z glues the I-charts and J-charts to a disjoint open-and-closed decomposition

$$Z' = Z_1' \amalg Z_2'.$$

The first part contains the inverse image of P_1 and the second contains the inverse image of P_2: on P_i the ideal $\mathcal{A}_i$ is the unit ideal, while the other chart family is empty there.

The center has underlying closed set $Z \setminus (P_1 \amalg P_2)$. Hence outside the center, Z_1 meets Z in exactly P_1 and Z_2 meets it in exactly P_2. Their strict transforms in Z' are therefore the respective closures of the inverse images of P_1 and P_2. Since Z_1' and Z_2' are also closed, these closures remain in the two disjoint components. Finally Lemma 2.4.72 identifies these strict transforms with those computed in the ambient blow-up of X. □

Theorem 2.5.11 (Closed-boundary separation construction package)**.** Uses: def:nagata-separate-closed-subschemes

Lean: AlgebraicGeometry.closedBoundarySeparationPackage

The N07 construction consists of the certified finite-presentation center $Y \hookrightarrow X$*, its blow-up, the selected identification over* U*, and the two strict transforms; these data form a* U*-admissible blow-up in which the two strict transforms are disjoint.*

Proof. Uses: lemma:nagata-boundary-separation-center-ideal,lemma:nagata-separate-closed-subschemes

The first lemma supplies the center, its finite-presentation certificate, and its disjointness from U. Form its global blow-up and take the two strict transforms. The second lemma supplies the chosen isomorphism over U and the chartwise disjointness certificate. These are exactly the fields of the asserted construction package. □

Definition 2.5.12 (Ideal whose support is a constructible closed subset)**.** Uses: def:nagata-qc-open-complement-ideal

Lean: Scheme.constructibleClosedSupportIdeal

Let $T \subseteq U$ be constructible and closed, with U qcqs. Its complement $W = U \setminus T$ is a quasi-compact open. Define

$$\mathcal{J}_U(T) = \mathcal{I}_U(W).$$

Lemma 2.5.13 (Property of the constructible-support ideal, [1, Tag 01PH]). Uses: def:nagata-constructible-closed-support

Lean: Scheme.exists_finiteType_ideal_zeroLocus_eq

The ideal $\mathcal{J}_U(T)$ is finite type and quasi-coherent, and its vanishing set is exactly T.

Proof. Uses: lemma:nagata-qc-open-complement-ideal

An open constructible subset of a spectral space is quasi-compact. Apply Lemma 2.5.5 to $W = U \setminus T$; its vanishing set is $U \setminus W = T$. □

Definition 2.5.14 (Constructible-boundary separation blow-up). Uses: def:nagata-constructible-closed-support,def:nagata-finite-type-ideal-extension,def:nagata-separate-closed-subschemes

Lean: AlgebraicGeometry.constructibleBoundarySeparationBlowup

Let $T_1, T_2 \subseteq U$ be disjoint constructible subsets closed in U. Define

$$\mathcal{J}_i = \mathcal{J}_U(T_i), \qquad \mathcal{I}_i = \mathrm{Ext}_{X,U}(\mathcal{J}_i), \qquad Z_i = V(\mathcal{I}_i),$$

and set

$$\mathsf{CSepBl}_U(T_1, T_2) = \mathsf{SepBl}_U(Z_1, Z_2).$$

Lemma 2.5.15 (Properties of constructible-boundary separation). Uses: def:nagata-separate-constructible-closed

Lean: AlgebraicGeometry.exists_admissibleBlowup_disjoint_closures

The constructed blow-up $\mathsf{CSepBl}_U(T_1, T_2) \to X$ is U-admissible, and the closures of the inverse images of T_1 and T_2 are disjoint.

Proof. Uses: lemma:nagata-constructible-closed-support,lemma:nagata-extend-finite-type-ideal,lemma:nagata-separate-closed-subschemes

The support and extension lemmas show that $Z_i \cap U$ has underlying closed set T_i. Hence $Z_1 \cap Z_2 \cap U = \varnothing$. Apply Lemma 2.5.10; the closure of each inverse image of T_i is contained in the support of the corresponding strict transform, so the two closures are disjoint. □

Definition 2.5.16 (Selected quasi-compact separating opens in a spectral space). Uses:

Lean: TopologicalSpace.Spectral.qcOpenSeparatingClosed

Let V be a spectral space and let $T, T' \subseteq V$ be disjoint closed subsets. For every $t' \in T'$, select a quasi-compact basic open $B_{t'}$ with $t' \in B_{t'} \subseteq V \setminus T$, select a finite subcover of T', and let W be its union. Next, for every $x \in V \setminus W$, select a quasi-compact basic open B'_x with $x \in B'_x \subseteq V \setminus T'$, select a finite subcover of $V \setminus W$, and let W' be its union. Denote this selected pair by $(W(T,T'), W'(T,T'))$.

Lemma 2.5.17 (Quasi-compact separating opens in a spectral space). Uses: def:nagata-spectral-qc-separating-opens

Lean: TopologicalSpace.Spectral.qcOpenSeparatingClosed_spec

Let V be a spectral space and let $T, T' \subseteq V$ be disjoint closed subsets. The selected opens $W = W(T,T')$ and $W' = W'(T,T')$ are quasi-compact and satisfy

$$T' \subseteq W \subseteq V \setminus T, \qquad V \setminus W \subseteq W' \subseteq V \setminus T'.$$

Proof. Uses:

A spectral space is quasi-compact and has a basis of quasi-compact opens. The closed subset T' is therefore quasi-compact. For every $t' \in T'$, choose a quasi-compact basic neighbourhood contained in the open $V \setminus T$; a finite subcover has quasi-compact open union W, giving $T' \subseteq W \subseteq V \setminus T$. The closed subset $V \setminus W$ is quasi-compact and is contained in $V \setminus T'$. Cover it by basic quasi-compact opens contained in $V \setminus T'$ and again take a finite subcover. Its union is the required W'. □

Definition 2.5.18 (Constructible separation data in a spectral space). Uses: def:nagata-spectral-qc-separating-opens

Lean: TopologicalSpace.Spectral.constructibleClosedSeparation

Let $T, T' \subseteq V$ be disjoint closed subsets of a spectral space. Use the selected pair $(W, W') = (W(T,T'), W'(T,T'))$ of Definition 2.5.16 and define

$$T_c = V \setminus W, \qquad T'_c = V \setminus W'.$$

Lemma 2.5.19 (Properties of the constructible separation data). *Uses:* *def:nagata-spectral-constructible-separation*

Lean: *TopologicalSpace.Spectral.exists_disjoint_constructibleClosed_supersets*

The constructed subsets T_c, T_c' are constructible and closed in V, and they satisfy

$$T \subseteq T_c, \qquad T' \subseteq T_c', \qquad T_c \cap T_c' = \varnothing.$$

Proof. Uses: lemma:nagata-spectral-qc-separating-opens

Complements of quasi-compact opens in a spectral space are constructible closed subsets. The choice of W gives $T \subseteq T_c$, the choice of W' gives $T' \subseteq T_c'$, and $V \setminus W \subseteq W'$ gives $T_c \cap T_c' = \varnothing$. □

Theorem 2.5.20 (Spectral constructible-separation package). *Uses:* *def:nagata-spectral-constructible-separation*

Lean: *TopologicalSpace.Spectral.constructibleClosedSeparationPackage*

The selected quasi-compact opens (W, W'), their complements (T_c, T_c'), the constructible-closed certificates, the two containment maps, and the disjointness proof assemble into the N08 separation record.

Proof. Uses: lemma:nagata-spectral-qc-separating-opens, lemma:nagata-spectral-constructible-separation

Choose (W, W') by the first lemma and define the two complements. The second lemma supplies constructible closedness, containment of T and T', and disjointness. Packaging these selected objects and certificates gives precisely the asserted record. □

Definition 2.5.21 (Modification making a proper map an isomorphism near a closure). Uses: def:nagata-spectral-constructible-separation, def:nagata-separate-constructible-closed, def:nagata-elimination-indeterminacy, def:nagata-extended-admissible-blowup

Lean: AlgebraicGeometry.isomorphismNearClosureConstruction

Let $q\colon Z \to X$ be proper, let $V \subseteq X$ be quasi-compact, set $U = q^{-1}(V)$, and let $T \subseteq V$ be closed. Let $E \subseteq V$ be the complement of the maximal open over which $U \to V$ is an isomorphism. Apply Definition 2.5.18 to (T, E) and denote the output by (T_c, E_c). Perform the explicitly constructed V-admissible separation blow-up

$$X_1 = \mathsf{CSepBl}_V(T_c, E_c).$$

Let $X_0 = X_1 \setminus \overline{E}_c$ and let $q_0\colon Z_0 \to X_0$ be the restriction of the strict transform of q. Form $X_0^{\mathrm{el}}(q_0, V \setminus E_c)$ and extend this admissible blow-up to X_1 by Definition 2.4.99. Define $\mathsf{IsoBl}_{V,T}(q) \to X$ to be the composite of these two V-admissible blow-ups, represented as one blow-up by the composition construction.

Lemma 2.5.22 (Property of the isomorphism-near-closure construction, [1, Tag 0F3W]). *Uses:* *def:nagata-isomorphism-near-closure*

Lean: *AlgebraicGeometry.exists_admissibleBlowup_isIso_near_closure*

If $U \to V$ is an isomorphism over an open neighbourhood of T, then $\mathsf{IsoBl}_{V,T}(q) \to X$ is V-admissible and the strict transform of q is an isomorphism over an open neighbourhood of the closure of T in $\mathsf{IsoBl}_{V,T}(q)$.

Proof. Uses: lemma:nagata-spectral-constructible-separation, lemma:nagata-separate-constructible-closed, lemma:nagata-elimination-indeterminacy, lemma:nagata-proper-open-dense-iso, lemma:nagata-extend-admissible-blowup, lemma:nagata-admissible-blowup-comp

The bad locus E is disjoint from T. The two separation steps make $\overline{T}_c$ and $\overline{E}_c$ disjoint in X_1, so X_0 contains an open neighbourhood of $\overline{T}_c$, hence of $\overline{T}$. On $V \setminus E_c$ the restricted proper map is already an isomorphism.

By Lemma 2.4.154, its strict transform after the constructed blow-up of X_0 is an open immersion. It remains proper, and the preserved open is scheme-theoretically dense, so Lemma 2.4.155 makes it an isomorphism. The extension property transports this blow-up to X_1 without changing its restriction to X_0. Finally Lemma 2.4.69 identifies the composite with a single V-admissible blow-up, while the neighbourhood of $\overline{T}$ stays inside X_0. □

2.6 Common modifications and graph extension

Lemma 2.6.1 (Scheme-theoretic image of a partial section). *Uses:*

Lean: *Scheme.Hom.schemeTheoreticImage_partialSection_preimage*

Let $U \hookrightarrow X$ be open, let $Y \to S$ be separated, and let $U \to X \times_S Y$ be a morphism whose first projection is the given open immersion. If Z is its scheme-theoretic image, then the projection $Z \to X$ has inverse image U over the open $U \subseteq X$, and this inverse image maps isomorphically to U.

Proof. Uses: `lemma:nagata-scheme-image-restrict-open`

By Lemma 2.1.9, restriction to $U \times_S Y$ commutes with taking the scheme-theoretic image. On this open target the original map is a section of $U \times_S Y \to U$. That projection is separated, so its section is a closed immersion and hence equals its own scheme-theoretic image. □

Lemma 2.6.2 ([1, Tag 01W6]). *Uses:*

Lean: `Scheme.Hom.IsProper.to_separated_target`

In a commutative triangle $T \to Y \to B$, if $T \to B$ is proper and $Y \to B$ is separated, then $T \to Y$ is proper.

Proof. Uses:

Factor $T \to Y$ through the graph $T \to T \times_B Y$. The graph is closed because $Y \to B$ is separated, and the projection $T \times_B Y \to Y$ is the base change of the proper morphism $T \to B$. □

Lemma 2.6.3 (Finite type over a separated target). *Uses:*

Lean: `Scheme.Hom.FiniteType.to_separated_target`

In a commutative triangle $T \to Y \to B$, if $T \to B$ is of finite type and $Y \to B$ is separated, then $T \to Y$ is of finite type.

Proof. Uses:

Factor $T \to Y$ as its graph

$$T \longrightarrow T \times_B Y \longrightarrow Y.$$

The graph is a closed immersion because $Y \to B$ is separated, hence is of finite type. The second map is the base change of the finite-type morphism $T \to B$. Their composite is therefore of finite type. □

Corollary 2.6.4 (Quasi-compactness of an open map to a separated target). *Uses:*

Lean: `Scheme.Hom.IsOpenImmersion.quasiCompact_of_finiteType_over`

In the preceding triangle, if $T \to Y$ is an open immersion, then it is a quasi-compact open immersion.

Proof. Uses: `lemma:nagata-finite-type-to-separated-target`

By Lemma 2.6.3, the open immersion is of finite type. An open immersion is locally of finite type, and it is of finite type exactly when it is quasi-compact. □

Definition 2.6.5 (Common-modification construction). Uses: `def:nagata-scheme-theoretic-image,def:nagata-elimination-indeterminacy,def:nagata-global-blowup,def:nagata-extended-admissible-blowup`

Lean: `AlgebraicGeometry.commonModificationConstruction`

Let $U \hookrightarrow X_i$ be open immersions over S, with U, X_1, X_2 separated and of finite type over the qcqs scheme S. Construct the following objects in order.

1. Let
$$X_{12} = \mathrm{SIm}\big(U \longrightarrow X_1 \times_S X_2\big)$$
and write $p_i \colon X_{12} \to X_i$ for the projections.
2. Apply Definition 2.4.153 to (p_i, U). Denote the resulting blow-ups by $X_i^i = X^{\mathrm{el}}(p_i, U)$, their center ideals by $\mathcal{I}_i$, and the open strict transforms by $X_{12}^i \subseteq X_i^i$.
3. On X_{12} define
$$\mathcal{K}_{12} = p_1^{-1}\mathcal{I}_1\mathcal{O}_{X_{12}}\, p_2^{-1}\mathcal{I}_2\mathcal{O}_{X_{12}}, \qquad X_{12}' = \mathrm{Bl}_{\mathcal{K}_{12}}(X_{12}).$$
4. The product-center blow-up maps to each X_{12}^i. Extend $X_{12}' \to X_{12}^i$ across the open $X_{12}^i \subseteq X_i^i$ by Definition 2.4.99; call the resulting ambient schemes X_i'.
5. Define
$$\mathsf{CM}(X_1, X_2; U) = X_1' \amalg_{X_{12}'} X_2'$$
using the two constructed open immersions of X_{12}'.

The construction includes the maps $X_i' \to X_i$, the two open maps $X_i' \to \mathsf{CM}(X_1, X_2; U)$, and the identified copy of U.

Lemma 2.6.6 (Properties of the common modification, [1, Tag 0F3X]). *Uses:* `def:nagata-common-modification`

Lean: `AlgebraicGeometry.existsCommonModification`

Each constructed map $X_i' \to X_i$ is a U-admissible blow-up. Each $X_i' \to \mathsf{CM}(X_1, X_2; U)$ is an open immersion, their two copies of U agree, and $\mathsf{CM}(X_1, X_2; U)$ is separated and of finite type over S.

Proof. Uses: `lemma:nagata-scheme-image-partial-section,lemma:nagata-elimination-indeterminacy,lemma:nagata-product-ideal-common-blowup,lemma:nagata-extend-admissible-blowup,lemma:nagata-proper-to-separated-target,lemma:nagata-separated-two-open-cover,lemma:nagata-admissible-blowup-comp`

By Lemma 2.6.1, each projection p_i is an isomorphism over U. The elimination property therefore makes $X_i^i \to X_i$ U-admissible and identifies X_{12}^i with an open subscheme of X_i^i. The product-ideal property gives domination maps $X_{12}' \to X_{12}^i$, all unchanged over U. The extension property makes the constructed $X_i' \to X_i^i$ admissible with restriction $X_{12}' \to X_{12}^i$; composition remains U-admissible.

It remains to verify that the gluing is separated. The map

$$X_{12}' \longrightarrow X_1' \times_S X_2'$$

is an immersion. Its composite to $X_1 \times_S X_2$ is proper because $X_{12}' \to X_{12}$ is proper and X_{12} is closed in the old product. The new product is separated over the old product, so Lemma 2.6.2 makes the displayed immersion proper, hence closed. The separated two-open gluing lemma now proves separatedness. Finite type is local on the two open pieces, and the construction identifies their copies of U at every step. □

Lemma 2.6.7 ([1, Tag 0F3Y])**.** *Uses:* `def:nagata-scheme-theoretic-image,def:nagata-strict-transform`

Lean: `Scheme.Hom.schemeTheoreticImage_afterAdmissibleBlowup`

Let $V \to X \times_S Y$ be quasi-compact, and assume its projection to X factors through an open $U \subseteq X$. Let Z be its scheme-theoretic image. If $X' \to X$ is a U-admissible blow-up, then the scheme-theoretic image of the lifted map $V \to X' \times_S Y$ is the strict transform of Z.

Proof. Uses: `lemma:nagata-scheme-image-restrict-open`

Let Z' be the strict transform and $Z'' \subseteq Z'$ the scheme-theoretic image of V. Over U, the two closed subschemes agree by Lemma 2.1.9. Thus the kernel of $\mathcal{O}_{Z'} \to \mathcal{O}_{Z''}$ is supported on the exceptional divisor. By the saturation characterization of strict transform, $\mathcal{O}_{Z'}$ has no nonzero subsheaf supported there. Hence the kernel vanishes and $Z'' = Z'$. □

Definition 2.6.8 (Graph-extension construction)**.** Uses: `def:nagata-scheme-theoretic-image,def:nagata-elimination-indeterminacy`

Lean: `AlgebraicGeometry.Compactification.properExtensionConstruction`

Let $U \to S$ be separated and of finite type, let $V \subseteq U$ be quasi-compact, and fix a compactification $V \hookrightarrow Y \to S$. Define

$$Z_V = \mathrm{SIm}\big(V \xrightarrow{(j,\,\mathrm{incl})} Y \times_S U\big).$$

Apply Definition 2.4.153 to the first projection $Z_V \to Y$ over V. Denote the resulting blow-up by $Y^{\mathrm{gr}} \to Y$ and the open strict transform by $Z_V^{\mathrm{st}} \hookrightarrow Y^{\mathrm{gr}}$. Define

$$V^{\mathrm{gr}} = \mathrm{im}(Z_V^{\mathrm{st}} \hookrightarrow Y^{\mathrm{gr}}), \qquad \psi^{\mathrm{gr}} \colon V^{\mathrm{gr}} \longrightarrow U$$

by transporting the second projection $Z_V^{\mathrm{st}} \to U$ across the open-immersion identification.

Lemma 2.6.9 (Properties of the graph-extension construction, [1, Tag 0F3Z])**.** *Uses:* `def:nagata-graph-extension`

Lean: `AlgebraicGeometry.Compactification.exists_properExtension`

The map $Y^{\mathrm{gr}} \to Y$ is V-admissible, $V \subseteq V^{\mathrm{gr}} \subseteq Y^{\mathrm{gr}}$ is open, and $\psi^{\mathrm{gr}} \colon V^{\mathrm{gr}} \to U$ is proper. Its restriction to V is the original inclusion, its inverse image of V is exactly V, and this restriction is an isomorphism.

Proof. Uses: `lemma:nagata-scheme-image-partial-section,lemma:nagata-scheme-image-strict-transform,lemma:nagata-elimination-indeterminacy`

By Lemma 2.6.1, the first projection $Z_V \to Y$ is an isomorphism over V. Under the constructed admissible blow-up, the new scheme-theoretic image is its strict transform by Lemma 2.6.7. The elimination property makes that strict transform open in Y^{gr} and leaves it unchanged over V.

The scheme Z_V^{st} is closed in $Y^{\mathrm{gr}} \times_S U$. Since $Y^{\mathrm{gr}} \to S$ is proper, the projection of this product to U is proper, and hence so is $Z_V^{\mathrm{st}} \to U$. Transporting across the open identification proves properness of ψ^{gr}. Restriction of the original graph image to the open $V \subseteq Y$ gives all three final assertions. □

2.7 The dense-source valuative criterion

Lemma 2.7.1 (Factorization through a closed subscheme over a valuation ring)**.** *Uses:*

Lean: `Scheme.Spec.factor_closed_of_fractionField_factor`

Let A be a valuation ring with fraction field K, let $Z \hookrightarrow Y$ be a closed immersion, and let $g\colon \operatorname{Spec}(A) \to Y$. If the restriction of g to $\operatorname{Spec}(K)$ factors through Z, then g factors through Z.

Proof. Uses:

Pull the defining ideal of Z back to A. It becomes zero after applying the injective map $A \hookrightarrow K$, so it was already zero in A. □

Lemma 2.7.2 (Flat pullback of a quasi-compact schematically dense map, [1, Tag 081I])**.** *Uses:*

Lean: `Scheme.Hom.schemeDense_baseChange_of_flat`

Let $h\colon U \to X$ be quasi-compact and have scheme-theoretic image X, and let $q\colon P \to X$ be flat. Then the base change

$$h_P\colon U \times_X P \longrightarrow P$$

is quasi-compact and has scheme-theoretic image P.

Proof. Uses: `lemma:nagata-scheme-image-restrict-open`

Quasi-compactness is preserved by base change. The assertion about the image is local on P. Pull back an affine open cover of X and then cover every inverse image by affine opens. This gives a cover of P by affine opens $W = \operatorname{Spec} B$ for which there is an affine $V = \operatorname{Spec} R \subseteq X$ with $W \subseteq q^{-1}(V)$. Fix one such pair. Since h is quasi-compact, $h^{-1}(V)$ admits a finite affine open cover

$$h^{-1}(V) = \bigcup_{j=1}^{n} \operatorname{Spec} C_j.$$

By open restriction of scheme-theoretic images, the map $h^{-1}(V) \to V$ has image V. Consequently

$$R \longrightarrow \prod_{j=1}^{n} C_j$$

is injective: it factors through $R \to \Gamma(h^{-1}(V), \mathcal{O})$, which is injective, and restriction to an open cover detects the zero section.

Flatness of q makes B a flat R-algebra. Tensor the displayed injection with B. Exactness of flat tensor product and commutation with the finite product give an injection

$$B \longrightarrow \prod_{j=1}^{n} (B \otimes_R C_j).$$

The affine schemes $\operatorname{Spec}(B \otimes_R C_j)$ cover $(U \times_X P)|_W$, so this injection says exactly that the scheme-theoretic image over W is all of W. These W's cover P, proving the claim. □

Lemma 2.7.3 (Finite affine detection of a schematically dense source)**.** *Uses:*

Lean: `AlgebraicGeometry.schemeDenseValuationProbe.finiteAffineCover`

Let $h\colon T \to P = \operatorname{Spec} R$ be quasi-compact and have scheme-theoretic image P. There is a finite affine open cover

$$T = \bigcup_{i=1}^{n} U_i, \qquad U_i = \operatorname{Spec} B_i,$$

for which the product map $R \to \prod_i B_i$ is injective.

Proof. Uses: `lemma:nagata-scheme-image-kernel-quasicoherent`

Because the target is affine and h is quasi-compact, T is quasi-compact and therefore has a finite affine open cover as displayed. The hypothesis on the scheme-theoretic image says that

$$R \longrightarrow \Gamma(T, \mathcal{O}_T)$$

is injective. The sheaf axiom makes $\Gamma(T, \mathcal{O}_T) \to \prod_i \Gamma(U_i, \mathcal{O}_{U_i})$ injective as well. Its composite is the asserted map $R \to \prod_i B_i$. □

Lemma 2.7.4 (A nonzero localization above the prescribed point)**.** *Uses:*

Lean: `AlgebraicGeometry.schemeDensEvaluationProbe.exists_nonzero_localization`

Let $R \to \prod_{i=1}^{n} B_i$ be injective and let $\mathfrak{p} \in \operatorname{Spec} R$. Then for some i,

$$(B_i)_{\mathfrak{p}} := (R \setminus \mathfrak{p})^{-1} B_i$$

is nonzero.

Proof. Uses:

Localizing the injection at $S = R \setminus \mathfrak{p}$ gives

$$R_{\mathfrak{p}} \hookrightarrow S^{-1}\left(\prod_i B_i\right) \cong \prod_i S^{-1} B_i;$$

the last isomorphism uses finiteness of the product. The source is a nonzero local ring because $\mathfrak{p}$ is a prime ideal. Hence the product on the right, and therefore at least one of its factors, is nonzero. □

Lemma 2.7.5 (A prime above a generalization of the prescribed point)**.** *Uses:*

Lean: `AlgebraicGeometry.schemeDensEvaluationProbe.exists_prime_over_generalization`

Let $R \to B$ be a ring map, let $\mathfrak{p} \in \operatorname{Spec} R$, and suppose that $(R \setminus \mathfrak{p})^{-1} B \neq 0$. There is a prime $\mathfrak{q} \in \operatorname{Spec} B$ whose contraction $\mathfrak{p}_0 = \mathfrak{q} \cap R$ satisfies $\mathfrak{p}_0 \subseteq \mathfrak{p}$.

Proof. Uses:

Put $S = R \setminus \mathfrak{p}$. The nonzero ring $S^{-1}B$ has a prime ideal $\mathfrak{Q}$. Let $\mathfrak{q}$ be its inverse image in B. Then $\mathfrak{q}$ is prime and is disjoint from the image of S. Thus its contraction $\mathfrak{p}_0$ to R is disjoint from $R \setminus \mathfrak{p}$, which is equivalent to $\mathfrak{p}_0 \subseteq \mathfrak{p}$. □

Lemma 2.7.6 (The residue field and its local source subring)**.** *Uses:*

Lean: `AlgebraicGeometry.schemeDensEvaluationProbe.residueField_localSubring`

Let $R \to B$ be a ring map, let $\mathfrak{p} \in \operatorname{Spec} R$, and let $\mathfrak{q} \in \operatorname{Spec} B$ have contraction $\mathfrak{p}_0 = \mathfrak{q} \cap R \subseteq \mathfrak{p}$. Set

$$K' = \kappa(\mathfrak{q}) = \operatorname{Frac}(B/\mathfrak{q}) \quad \text{and} \quad D = \operatorname{im}(R_{\mathfrak{p}} \longrightarrow K').$$

The map $R_{\mathfrak{p}} \to K'$ has kernel $\mathfrak{p}_0 R_{\mathfrak{p}}$, and hence

$$D \cong R_{\mathfrak{p}} / \mathfrak{p}_0 R_{\mathfrak{p}}.$$

In particular $D \subseteq K'$ is a local domain, and its maximal ideal is the image of $\mathfrak{p} R_{\mathfrak{p}}$.

Proof. Uses:

The kernel of $R \to B \to \kappa(\mathfrak{q})$ is precisely $\mathfrak{q} \cap R = \mathfrak{p}_0$. Since $\mathfrak{p}_0 \subseteq \mathfrak{p}$, the map extends across $R_{\mathfrak{p}}$; exactness of localization identifies its kernel with $\mathfrak{p}_0 R_{\mathfrak{p}}$. The first isomorphism theorem gives the displayed description of D. It is a domain because it is a subring of the field K', and it is local because it is a quotient of the local ring $R_{\mathfrak{p}}$. Its maximal ideal is therefore the indicated image of $\mathfrak{p} R_{\mathfrak{p}}$. □

Lemma 2.7.7 (Zorn construction of a dominating valuation ring, [1, Tag 00IA])**.** *Uses:*

Lean: `ValuationRing.exists_dominating_localSubring`

Let $D \subseteq K$ be a local subring of a field. There is a valuation ring $A' \subseteq K$ which dominates D and whose fraction field is K.

Proof. Uses:

Consider local subrings of K dominating D, ordered by domination. This partially ordered set is nonempty because it contains D. For a chain $\{D_\lambda\}$, put

$$E = \bigcup_\lambda D_\lambda, \qquad \mathfrak{m}_E = \bigcup_\lambda \mathfrak{m}_{D_\lambda}.$$

The domination ordering ensures that membership in a maximal ideal is preserved on passing up the chain. Thus every element of E outside $\mathfrak{m}_E$ already has its inverse in one D_λ, whereas no element of $\mathfrak{m}_E$ becomes a unit later in the chain. Hence E is local with maximal ideal $\mathfrak{m}_E$, and it dominates every D_λ. Zorn's lemma supplies a maximal member A'.

It remains to prove $\operatorname{Frac}(A') = K$. Write $L = \operatorname{Frac}(A')$ and suppose $L \neq K$; choose $t \in K \setminus L$. If t is transcendental over L, then

$$A'[t]_{(\mathfrak{m}_{A'}, t)}$$

is a local subring of K, strictly larger than A', and dominates A', contradicting maximality. If t is algebraic over L, write

$$t^n + c_{n-1}t^{n-1} + \cdots + c_0 = 0 \qquad (c_i \in L)$$

and choose a common nonzero denominator $a \in A'$ with $ac_i \in A'$ for every i. After multiplication by a^n, the element $u = at$ satisfies

$$u^n + (ac_{n-1})u^{n-1} + (a^2c_{n-2})u^{n-2} + \cdots + a^n c_0 = 0.$$

Thus at is integral over A'. The finite integral algebra $A'[at]$ has, by lying over, a prime $\mathfrak{m}'$ above $\mathfrak{m}_{A'}$. Then $A'[at]_{\mathfrak{m}'}$ dominates A'. It is strictly larger, since equality would put at in A' and hence $t = (at)/a$ in L, again a contradiction. Therefore $L = K$. Maximality for domination among local subrings of its fraction field now says exactly that A' is a valuation ring. □

Lemma 2.7.8 (The selected valuation is centred at the prescribed point)**.** *Uses:*

Lean: `AlgebraicGeometry.schemeDensevaluationProbe.closedPoint_center`

Let $\mathfrak{p}_0 \subseteq \mathfrak{p}$ be primes of R, let $D \subseteq K'$ be the image of a map $R_{\mathfrak{p}} \to K'$ with kernel $\mathfrak{p}_0 R_{\mathfrak{p}}$, and let $A' \subseteq K'$ dominate D. For the composite $R \to D \hookrightarrow A'$, the inverse image of $\mathfrak{m}_{A'}$ is $\mathfrak{p}$. Equivalently, the closed point of $\operatorname{Spec} A'$ maps to the point $z = \mathfrak{p}$ of $\operatorname{Spec} R$.

Proof. Uses:

The local domain $D \cong R_{\mathfrak{p}}/\mathfrak{p}_0 R_{\mathfrak{p}}$ has maximal ideal equal to the image of $\mathfrak{p}R_{\mathfrak{p}}$. Since A' dominates D,

$$\mathfrak{m}_{A'} \cap D = \mathfrak{m}_D.$$

Pulling this equality back first to $R_{\mathfrak{p}}$ and then to R gives $\mathfrak{p}R_{\mathfrak{p}}$ and $\mathfrak{p}$, respectively. This is exactly the claimed closed-point image. □

Lemma 2.7.9 (Commutativity of the valuation-probe square)**.** *Uses:*

Lean: `AlgebraicGeometry.schemeDensevaluationProbe.square_commutes`

Let $P = \operatorname{Spec} R$, let $U = \operatorname{Spec} B \subseteq T$ be an affine open, and suppose the ring map $R \to B$ represents $h|_U$. Let $\mathfrak{p} \in \operatorname{Spec} R$, choose $\mathfrak{q} \in \operatorname{Spec} B$ whose contraction is contained in $\mathfrak{p}$, set $K' = \kappa(\mathfrak{q})$, and put $D = \operatorname{im}(R_{\mathfrak{p}} \to K')$. Let $A' \subseteq K'$ contain D. Define

$$\eta\colon \operatorname{Spec} K' \longrightarrow U = \operatorname{Spec} B \longrightarrow T$$

by the residue-field map at $\mathfrak{q}$, and define $\alpha\colon \operatorname{Spec} A' \to \operatorname{Spec} R$ by $R \to D \hookrightarrow A'$. Then

$$\begin{array}{ccc} \operatorname{Spec} K' & \xrightarrow{\eta} & T \\ \downarrow & & \downarrow h \\ \operatorname{Spec} A' & \xrightarrow{\alpha} & \operatorname{Spec} R \end{array}$$

commutes.

Proof. Uses:

Both composites are affine-locally opposite to the same ring map

$$R \longrightarrow B \longrightarrow \kappa(\mathfrak{q}) = K'.$$

Indeed, the lower-then-left composite is $R \to D \hookrightarrow A' \hookrightarrow K'$, and D was defined as the image of the localization of the displayed map. Equality on every $r \in R$ proves equality of the scheme morphisms. □

Lemma 2.7.10 (Existence of a dense valuation probe, [1, Tag 02JQ])**.** *Uses:*

Lean: `AlgebraicGeometry.exists_valuationRing_through_schemeDense`

Let $h\colon T \to P$ be quasi-compact and have scheme-theoretic image P, assume P is affine, and let $z \in P$. There exist a field K', a valuation subring $A' \subseteq K'$ with fraction field K', and morphisms

$$\eta\colon \operatorname{Spec} K' \to T, \qquad \alpha\colon \operatorname{Spec} A' \to P$$

such that the valuation-probe square commutes and the closed point of $\operatorname{Spec} A'$ maps to z.

Proof. Uses: `lemma:nagata-valuation-probe-finite-affine-cover,lemma:nagata-valuation-probe-nonzero-localization,lemma:nagata-valuation-probe-prime,lemma:nagata-valuation-probe-residue-local-domain,lemma:nagata-zorn-valuation-domination,lemma:nagata-valuation-probe-closed-point-center,lemma:nagata-valuation-probe-square-commutes`

Write $P = \operatorname{Spec} R$ and let $\mathfrak{p} \subset R$ correspond to z. Choose the finite affine cover $T = \bigcup_i U_i$, $U_i = \operatorname{Spec} B_i$, supplied by Lemma 2.7.3. Its product map is injective, so Lemma 2.7.4 supplies an i with $(B_i)_{\mathfrak{p}} \neq 0$. By Lemma 2.7.5, choose $\mathfrak{q} \in \operatorname{Spec} B_i$ whose contraction $\mathfrak{p}_0$ is contained in $\mathfrak{p}$.

Put $K' = \kappa(\mathfrak{q})$ and $D = \mathrm{im}(R_{\mathfrak{p}} \to K')$. By Lemma 2.7.6, D is a local domain inside K', with maximal ideal induced by $\mathfrak{p}R_{\mathfrak{p}}$. Apply Lemma 2.7.7 to obtain a valuation subring $A' \subseteq K'$, with fraction field K', dominating D.

The residue-field morphism at $\mathfrak{q}$, followed by the open immersion $U_i \hookrightarrow T$, defines η. The composite $R \to D \hookrightarrow A'$ defines α. The closed-point assertion is Lemma 2.7.8, and the equality $h\eta = \alpha|_{\mathrm{Spec}\, K'}$ is Lemma 2.7.9. These are all the required properties. □

Definition 2.7.11 (Valuation probe through a schematically dense source)**.** Uses:

Lean: `AlgebraicGeometry.schemeDenseValuationProbe`

Constructed by: `lemma:nagata-dense-valuation-probe-exists`

For a quasi-compact morphism $h\colon T \to P$ whose scheme-theoretic image is the affine scheme P, and for $z \in P$, let $\mathcal{W}(h, z)$ be the type of tuples

$$(A', K', \eta, \alpha)$$

in which $A' \subseteq K'$ is a valuation ring with fraction field K', the map $\eta\colon \mathrm{Spec}\, K' \to T$ is a generic lift, $\alpha\colon \mathrm{Spec}\, A' \to P$ completes the commutative square, and its closed point maps to z. The preceding existence construction proves $\mathcal{W}(h, z)$ is inhabited. Select one element by classical choice and denote it

$$\mathsf{VP}(h, z) = (A'_{h,z}, K'_{h,z}, \eta_{h,z}, \alpha_{h,z}).$$

Lemma 2.7.12 (Properties of the valuation probe)**.** *Uses:* *def:nagata-dense-valuation-probe*

Lean: *AlgebraicGeometry.schemeDenseValuationProbe.spec*

Let $h\colon T \to P$ and $z \in P$ satisfy the hypotheses of Definition 2.7.11. Then $A'_{h,z}$ is a valuation subring of $K'_{h,z}$ with fraction field $K'_{h,z}$, the map $\eta_{h,z}\colon \mathrm{Spec}\, K'_{h,z} \to T$ is a generic lift, the maps form a commutative diagram

$$\begin{array}{ccc} \mathrm{Spec}(K'_{h,z}) & \xrightarrow{\eta_{h,z}} & T \\ \downarrow & & \downarrow h \\ \mathrm{Spec}(A'_{h,z}) & \xrightarrow{\alpha_{h,z}} & P, \end{array}$$

and the closed point of $\mathrm{Spec}(A'_{h,z})$ maps to z.

Proof. Uses: `lemma:nagata-dense-valuation-probe-exists`

By definition, $\mathsf{VP}(h, z)$ is selected from the witness type $\mathcal{W}(h, z)$. Membership in that type is precisely the conjunction of the valuation-ring, fraction-field, generic-lift, commutativity, and closed-point assertions listed here. The specification of classical choice therefore gives all five conclusions. □

Definition 2.7.13 (Dense-source valuative lifting property)**.** Uses:

Lean: `AlgebraicGeometry.ValuativeCriterion.DenseSourceLifting`

Let $f\colon X \to S$ and $h\colon U \to X$ be morphisms. Write $\mathsf{DVL}(f, h)$ for the following property. For every valuation ring A with fraction field K, every pair

$$u_K\colon \mathrm{Spec}\, K \longrightarrow U, \qquad s_A\colon \mathrm{Spec}\, A \longrightarrow S$$

satisfying

$$f \circ h \circ u_K = s_A|_{\mathrm{Spec}\, K},$$

admits a unique morphism $x_A\colon \mathrm{Spec}\, A \to X$ such that

$$f \circ x_A = s_A, \qquad x_A|_{\mathrm{Spec}\, K} = h \circ u_K.$$

Lemma 2.7.14 (Reduction invariance of dense-source valuation squares, [1, Tag 0356])**.** *Uses:* *def:nagata-dense-valuative-lifting-property*

Lean: *AlgebraicGeometry.ValuativeCriterion.denseSourceLifting_reduction*

Let $f\colon X \to S$ and $h\colon U \to X$ be morphisms, and let

$$f_{\mathrm{red}}\colon X_{\mathrm{red}} \to S_{\mathrm{red}}, \qquad h_{\mathrm{red}}\colon U_{\mathrm{red}} \to X_{\mathrm{red}}$$

be the induced maps. Then

$$\mathsf{DVL}(f, h) \iff \mathsf{DVL}(f_{\mathrm{red}}, h_{\mathrm{red}}).$$

Moreover, every valuation square for f is the composite with the reduction immersions of its unique valuation square for f_{red}, and a lift of the latter gives a lift of the former.

Proof. Uses: `def:nagata-dense-valuative-lifting-property`

A valuation ring and its fraction field are reduced. Hence every map from either of their spectra to a scheme Y factors uniquely through Y_{red}. Apply this to the maps into U, X, S. Uniqueness of factorization through reductions transports the commutativity equations in both directions. It also identifies lifts into X with lifts into X_{red}, because the source of a lift is again reduced. This proves both implications and the final assertion. □

Lemma 2.7.15 (Dense reduced source has full scheme-theoretic image, [1, Tag 056B])**.** *Uses:*

Lean: `AlgebraicGeometry.Scheme.Hom.schemeImage_eq_top_of_reduction_dense`

Let $h\colon U \to X$ be quasi-compact with dense set-theoretic image. Then $h_{\mathrm{red}}\colon U_{\mathrm{red}} \to X_{\mathrm{red}}$ is quasi-compact, has dense set-theoretic image, and has scheme-theoretic image all of X_{red}.

Proof. Uses:

Reduction does not change the underlying topological spaces, so density is unchanged; quasi-compactness is unchanged as well. The source U_{red} is reduced. Its scheme-theoretic image is therefore the reduced induced subscheme on the closure of its image. That closure is all of the underlying space of X_{red}, so the reduced induced subscheme is X_{red} itself. □

Lemma 2.7.16 (Affine base neighbourhood for a valuation map)**.** *Uses:*

Lean: `AlgebraicGeometry.ValuationRing.Spec.factor_affineNeighbourhood`

Let A be a valuation ring and $s_A\colon \operatorname{Spec} A \to S$. If $S_0 \subseteq S$ is an open neighbourhood of the image of the closed point of $\operatorname{Spec} A$, then s_A factors uniquely through S_0. In particular one may choose S_0 affine.

Proof. Uses:

Every prime of a valuation ring is contained in its maximal ideal, so every point of $\operatorname{Spec} A$ is a generalization of the closed point. An open subset is stable under generalization. Thus $s_A^{-1}(S_0)$ contains every point and s_A factors through the open subscheme S_0. The factor is unique because an open immersion is a monomorphism. Every point of a scheme has an affine open neighbourhood. □

Lemma 2.7.17 (Restriction of dense-source lifting to an open base)**.** *Uses:* `def:nagata-dense-valuative-lifting-property`

Lean: `AlgebraicGeometry.ValuativeCriterion.denseSourceLifting_restrictBase`

Let $f\colon X \to S$ and $h\colon U \to X$ be quasi-compact, with $h(U)$ dense in X, and assume $\mathsf{DVL}(f, h)$. For an open subscheme $S_0 \subseteq S$, put

$$X_0 = X \times_S S_0, \qquad U_0 = U \times_X X_0.$$

Then the restricted maps $f_0\colon X_0 \to S_0$ and $h_0\colon U_0 \to X_0$ are quasi-compact, $h_0(U_0)$ is dense in X_0, and $\mathsf{DVL}(f_0, h_0)$ holds.

Proof. Uses: `def:nagata-dense-valuative-lifting-property`

Both restricted maps are base changes of quasi-compact maps. Moreover $h_0(U_0) = h(U) \cap X_0$, which is dense in the open subspace X_0. A valuation square over S_0 becomes one over S, so $\mathsf{DVL}(f, h)$ supplies a unique lift to X. Its composite to S factors through S_0, hence the lift factors uniquely through the inverse image X_0. This gives existence and uniqueness for the restricted square. □

Lemma 2.7.18 (Flat polynomial probe through a field-valued point)**.** *Uses:*

Lean: `AlgebraicGeometry.ValuativeCriterion.exists_flatPolynomialProbe`

Let $x_K\colon \operatorname{Spec} K \to X$ be a field-valued point and choose an affine open $V = \operatorname{Spec} B \subseteq X$ through which it factors. There are a polynomial B-algebra P, a surjective B-algebra map $e\colon P \twoheadrightarrow K$, and a factorization

$$\operatorname{Spec} K \xrightarrow{z} \operatorname{Spec} P \xrightarrow{q} X$$

of x_K, with q flat. The point z has residue field K.

Proof. Uses:

Regard K as a B-algebra through x_K, take one indeterminate T_a for each element $a \in K$, and set

$$P = B[T_a \mid a \in K].$$

Evaluation $T_a \mapsto a$ is a surjective B-algebra map $e\colon P \to K$. A polynomial algebra is free, hence flat, over B. Thus $\operatorname{Spec} P \to \operatorname{Spec} B$ is flat, and so is its composite with the open immersion $\operatorname{Spec} B \hookrightarrow X$. The kernel of e defines z, its residue field is K, and the displayed composite is the original point by construction. □

Lemma 2.7.19 (Residue-field extension of a valuation, [1, Tag 00IA]). *Uses:*

Lean: `ValuationRing.exists_dominating_fieldExtension`

Let $A \subseteq K$ be a valuation ring with fraction field K, and let $\iota\colon K \hookrightarrow k'$ be a field embedding. There exists a valuation subring $A'' \subseteq k'$, with fraction field k', which dominates the local subring $\iota(A) \subseteq k'$. In particular, $\iota(A) \subseteq A''$ and $\mathfrak{m}_{A''} \cap \iota(A) = \iota(\mathfrak{m}_A)$.

Proof. Uses: `lemma:nagata-zorn-valuation-domination`

The embedding ι identifies A with the local subring $\iota(A)$ of k'; its maximal ideal is $\iota(\mathfrak{m}_A)$. Apply the dominating-valuation construction to this local subring of the field k'. It produces a valuation subring with fraction field k', and domination gives the asserted contraction of maximal ideals. □

Lemma 2.7.20 (Residue map at the centre of a valuation probe). *Uses:*

Lean: `AlgebraicGeometry.schemeDensEvaluationProbe.residueFieldMap`

Let $e\colon P \twoheadrightarrow K$ define a point $z \in \operatorname{Spec} P$, and let $a'\colon \operatorname{Spec} A' \to \operatorname{Spec} P$ carry the closed point of the valuation ring A' to z. If $k' = A'/\mathfrak{m}_{A'}$, then the residue map of a' is a field embedding

$$\iota\colon K = \kappa(z) \hookrightarrow k'$$

and the composite $P \to A' \to k'$ is $P \xrightarrow{e} K \xrightarrow{\iota} k'$.

Proof. Uses:

The condition on the closed point says that the inverse image of $\mathfrak{m}_{A'}$ under $P \to A'$ is $\ker(e)$. The induced injective map on residue fields is therefore $\operatorname{Frac}(P/\ker(e)) = K \to k'$. Its definition gives the stated factorization of $P \to k'$. □

Definition 2.7.21 (Composite valuation subring). Uses:

Lean: `ValuationRing.compositeSubring`

Let A' be a valuation ring with maximal ideal $\mathfrak{m}'$, residue field $k' = A'/\mathfrak{m}'$, and residue map $\rho\colon A' \twoheadrightarrow k'$. For a valuation subring $A'' \subseteq k'$, define

$$C(A', A'') := \rho^{-1}(A'') = \{a \in A' \mid \rho(a) \in A''\}.$$

Lemma 2.7.22 (Composite of two valuation rings, [1, Tag 088Z]). *Uses:* `def:nagata-composite-valuation-subring`

Lean: `ValuationRing.comap_residue_valuationRing`

In the situation of Definition 2.7.21, assume A'' has fraction field k'. Then $C = C(A', A'')$ is a valuation ring, its fraction field is $\operatorname{Frac}(A')$, and

$$\mathfrak{m}' \subseteq C, \qquad C/\mathfrak{m}' \cong A''.$$

The inclusions and quotient induce a commutative square

$$\begin{array}{ccc} \operatorname{Spec} k' & \longrightarrow & \operatorname{Spec} A' \\ \downarrow & & \downarrow \\ \operatorname{Spec} A'' & \longrightarrow & \operatorname{Spec} C. \end{array}$$

Proof. Uses: `def:nagata-composite-valuation-subring`

Let $x \neq 0$ lie in $\operatorname{Frac}(A')$. The valuation property of A' puts x or x^{-1} in A'. If one of these lies in $\mathfrak{m}'$, its residue is zero and it belongs to C. Otherwise x is a unit of A', so both x and x^{-1} lie in A' and have inverse nonzero residues in k'. The valuation property of A'' puts one residue in A'', hence puts x or x^{-1} in C. Thus C is a valuation ring.

The equality $\rho^{-1}(0) = \mathfrak{m}'$ gives $\mathfrak{m}' \subseteq C$, and restriction of ρ to C is surjective onto A'' with kernel $\mathfrak{m}'$. This proves the quotient formula. The preceding inverse argument also shows that every element of A' lies in $\operatorname{Frac}(C)$; hence $\operatorname{Frac}(C) = \operatorname{Frac}(A')$. Finally, the two routes in the square are opposite to the same ring map $C \to k'$, so it commutes. □

Lemma 2.7.23 (Base map factors through the composite valuation). *Uses:* `def:nagata-composite-valuation-subring`

Lean: `ValuationRing.compositeSubring_lift`

With $C = C(A', A'')$ as above, let $\beta\colon R \to A'$ be a ring map. If the image of $\rho \circ \beta\colon R \to k'$ is contained in A'', then there is a unique map $\beta_C\colon R \to C$ whose composite with $C \hookrightarrow A'$ is β.

Proof. Uses: `def:nagata-composite-valuation-subring`

The containment hypothesis says exactly that every $\beta(r)$ belongs to the inverse image $\rho^{-1}(A'') = C$. Corestricting β therefore gives β_C. The inclusion $C \hookrightarrow A'$ is injective, so this corestriction is unique. □

Lemma 2.7.24 (Lift over the composite valuation)**.** *Uses:* `def:nagata-dense-valuative-lifting-property`

Lean: `AlgebraicGeometry.ValuativeCriterion.denseSource_compositeLift`

Assume $\mathsf{DVL}(f,h)$. *Let* C *be a valuation ring with fraction field* K', *let* $u'\colon \operatorname{Spec} K' \to U$, *and let* $s_C\colon \operatorname{Spec} C \to S$ *satisfy* $fhu' = s_C|_{\operatorname{Spec} K'}$. *Then there is a unique* $g_C\colon \operatorname{Spec} C \to X$ *over* S *whose generic restriction is* hu'.

Proof. Uses: `def:nagata-dense-valuative-lifting-property`

This is the defining lifting property applied to the valuation ring C and the displayed commutative square. □

Lemma 2.7.25 (Comparison of the two maps from the first valuation)**.** *Uses:* `def:nagata-dense-valuative-lifting-property`

Lean: `AlgebraicGeometry.ValuativeCriterion.denseSource_firstValuation_eq`

Assume $\mathsf{DVL}(f,h)$. *Let* $C \subseteq A'$ *be valuation rings with the same fraction field* K', *let* $j\colon \operatorname{Spec} A' \to \operatorname{Spec} C$ *be induced by the inclusion, and let* $g_C\colon \operatorname{Spec} C \to X$. *Suppose* $g'\colon \operatorname{Spec} A' \to X$ *and* $g_C \circ j$ *lie over the same map to* S, *and their restrictions to* $\operatorname{Spec} K'$ *are the same map through* U. *Then*

$$g' = g_C \circ j.$$

Proof. Uses: `def:nagata-dense-valuative-lifting-property`

Both maps are dotted arrows for the same dense-source valuation square with valuation ring A', fraction field K', and the specified generic map through U. The uniqueness clause of $\mathsf{DVL}(f,h)$ identifies them. □

Lemma 2.7.26 (Generic comparison after restricting the composite lift)**.** *Uses:* `def:nagata-composite-valuation-subring`

Lean: `AlgebraicGeometry.ValuativeCriterion.compositeLift_residue_generic`

Let $C = C(A', A'')$, *write* $k' = A'/\mathfrak{m}_{A'}$, *and let*

$$j\colon \operatorname{Spec} A' \to \operatorname{Spec} C, \qquad i''\colon \operatorname{Spec} A'' \to \operatorname{Spec} C$$

be the maps induced by $C \hookrightarrow A'$ *and* $C \twoheadrightarrow A''$. *Let* $g_C\colon \operatorname{Spec} C \to X$ *and* $g'\colon \operatorname{Spec} A' \to X$ *satisfy* $g_C j = g'$. *Suppose that the restriction of* g' *to the closed point* $\operatorname{Spec} k' \to \operatorname{Spec} A'$ *equals*

$$\operatorname{Spec} k' \longrightarrow \operatorname{Spec} K \xrightarrow{x_K} X$$

for a field embedding $K \hookrightarrow k'$. *Then* $g_{A''} := g_C i''\colon \operatorname{Spec} A'' \to X$ *has generic restriction equal to the same base extension of* x_K.

Proof. Uses: `lemma:nagata-composite-valuation-ring`

There is no map $\operatorname{Spec} A'' \to \operatorname{Spec} A'$ in general. Instead compare the two maps from $\operatorname{Spec} k'$ to $\operatorname{Spec} C$. The first is

$$\operatorname{Spec} k' \to \operatorname{Spec} A' \xrightarrow{j} \operatorname{Spec} C,$$

and the second is

$$\operatorname{Spec} k' \to \operatorname{Spec} A'' \xrightarrow{i''} \operatorname{Spec} C.$$

Both are opposite to the same ring map $C \to C/\mathfrak{m}_{A'} = A'' \hookrightarrow k'$, so they are equal. After composing with g_C, the first route is the closed-point restriction of $g' = g_C j$, while the second is the generic restriction of $g_{A''}$. The assumed description of the first route proves the claim. □

Lemma 2.7.27 (Intersection with a dominating valuation ring)**.** *Uses:*

Lean: `ValuationRing.inter_eq_of_dominates`

Let $A \subseteq K$ *be a valuation ring with fraction field* K, *let* $K \hookrightarrow L$ *be a field extension, and let* $B \subseteq L$ *be a valuation ring dominating the image of* A. *Then, after identifying* K *with its image in* L,

$$K \cap B = A.$$

Proof. Uses:

The inclusion $A \subseteq K \cap B$ is immediate. Conversely, let $x \in K \setminus A$. The valuation property of A gives $x^{-1} \in \mathfrak{m}_A$: it lies in A, and it cannot be a unit because that would put x in A. Domination gives $x^{-1} \in \mathfrak{m}_B$. Hence $x \notin B$, since otherwise $1 = xx^{-1} \in \mathfrak{m}_B$. Thus no element of $K \setminus A$ lies in B, proving the reverse inclusion. □

Lemma 2.7.28 (The local image of the residue lift lies in the old fraction field)**.** *Uses:*

Lean: `AlgebraicGeometry.ValuativeCriterion.residueLift_localImage`

Let $K \hookrightarrow k'$ be a field extension, let $A'' \subseteq k'$ be a valuation ring with fraction field k', and let $g_{A''} \colon \operatorname{Spec} A'' \to X$. Suppose its generic restriction is the base extension of a map $x_K \colon \operatorname{Spec} K \to X$. If $x \in X$ is the image of the closed point of $\operatorname{Spec} A''$, then the image of

$$\theta \colon \mathcal{O}_{X,x} \longrightarrow A'' \hookrightarrow k'$$

is contained in the subfield $K \subseteq k'$.

Proof. Uses:

The generic point of $\operatorname{Spec} A''$ maps to the image of x_K, a generalization of x. Passing from the closed point to the generic point carries θ to the composite

$$\mathcal{O}_{X,x} \longrightarrow K \hookrightarrow k'$$

induced by x_K and the field embedding. The map $A'' \hookrightarrow k'$ is injective, so this equality says element by element that the image of θ is contained in K. □

Lemma 2.7.29 (Intersection descent from the residue valuation to the original one)**.** *Uses:*

Lean: `AlgebraicGeometry.ValuativeCriterion.residueLift_descends`

Let $A \subseteq K$ be a valuation ring, let $K \hookrightarrow k'$ be a field extension, and let $A'' \subseteq k'$ be a valuation ring dominating A. Let $g_{A''} \colon \operatorname{Spec} A'' \to X$ lie over the base change of $s_A \colon \operatorname{Spec} A \to S$, and suppose its generic restriction is the base change of $x_K \colon \operatorname{Spec} K \to X$. Then $g_{A''}$ factors through a morphism

$$x_A \colon \operatorname{Spec} A \longrightarrow X$$

which lies over s_A and restricts to x_K.

Proof. Uses: `lemma:nagata-valuation-intersection,lemma:nagata-dense-valuative-local-image-in-field`

Let x be the image of the closed point of $\operatorname{Spec} A''$. By the local-image lemma, the local map $\theta \colon \mathcal{O}_{X,x} \to A'' \subseteq k'$ has image in K. Since it also has image in A'', the intersection formula gives

$$\theta(\mathcal{O}_{X,x}) \subseteq K \cap A'' = A.$$

Corestrict θ to $\theta_A \colon \mathcal{O}_{X,x} \to A$. It is local because $\mathfrak{m}_{A''} \cap A = \mathfrak{m}_A$. The standard morphism $\operatorname{Spec} A \to \operatorname{Spec} \mathcal{O}_{X,x} \to X$ defined by θ_A is x_A, and its pullback along $A \hookrightarrow A''$ is $g_{A''}$, because both morphisms send the closed point to x and induce the same local map $\mathcal{O}_{X,x} \to A''$.

For the base equation, let s be the image of the closed point in S. The two maps $\mathcal{O}_{S,s} \rightrightarrows A$ induced by fx_A and s_A become equal after the injection $A \hookrightarrow A''$, since $g_{A''}$ lies over the base change of s_A. They were therefore equal already in A, so $fx_A = s_A$. On generic points the same argument uses the injection $K \hookrightarrow k'$: the two K-valued maps become the assumed equal k'-valued map, and hence are equal. Thus $x_A|_{\operatorname{Spec} K} = x_K$, completing the descent. □

Lemma 2.7.30 (Existence in the valuative criterion from a dense source)**.** *Uses:* `def:nagata-dense-valuative-lifting-property`

Lean: `AlgebraicGeometry.ValuativeCriterion.exists_of_denseSourceLifting`

Let $f \colon X \to S$ and $h \colon U \to X$ be quasi-compact, assume $h(U)$ is dense in X, and assume DVL(f,h)*. Then every commutative valuation square*

$$\begin{array}{ccc} \operatorname{Spec} K & \xrightarrow{x_K} & X \\ \downarrow & & \downarrow f \\ \operatorname{Spec} A & \xrightarrow{s_A} & S \end{array}$$

admits a lift $x_A \colon \operatorname{Spec} A \to X$.

Proof. Uses: `lemma:nagata-dense-valuative-reduction-invariance,lemma:nagata-dense-valuative-reduced-schematic-image,lemma:nagata-valuation-map-affine-base-neighbourhood,lemma:nagata-dense-valuative-open-base-restriction,lemma:nagata-dense-valuative-polynomial-probe,lemma:nagata-dense-flat-pullback,lemma:nagata-dense-valuation-probe,lemma:nagata-dense-valuative-probe-residue-map,lemma:nagata-dense-valuative-residue-extension,def:nagata-composite-valuation-subring,lemma:nagata-composite-valuation-ring,lemma:nagata-composite-valuation-base-factorization,lemma:nagata-dense-valuative-composite-lift,lemma:nagata-dense-valuative-Aprime-comparison,lemma:nagata-dense-valuative-composite-generic-comparison,lemma:nagata-dense-valuative-intersection-descent`

By reduction invariance, the valuation square and the dense-source lifting property factor through $U_{\mathrm{red}}, X_{\mathrm{red}}, S_{\mathrm{red}}$, and a lift there gives a lift of the original square. We therefore replace U, X, S by their reductions. The reduced-source lemma now says that the scheme-theoretic image of h is all of X.

The affine-neighbourhood lemma factors $\operatorname{Spec} A \to S$ through an affine open S_0 containing the image of its closed point. Apply the open-base restriction lemma and replace S, X, U by $S_0, X \times_S S_0, U \times_S S_0$. Write the restricted base as $S = \operatorname{Spec} R$. Choose an affine open $V = \operatorname{Spec} B \subseteq X$ through which x_K factors. The polynomial-probe lemma supplies a surjection $P \twoheadrightarrow K$, a point $z = \operatorname{Spec} K \to \operatorname{Spec} P$, and a flat map $q\colon \operatorname{Spec} P \to X$ whose composite with z is x_K. Flat pullback gives a quasi-compact schematically dense map

$$h_P\colon U \times_X \operatorname{Spec} P \longrightarrow \operatorname{Spec} P.$$

Apply the dense valuation probe to h_P and z. It gives a valuation ring A' with fraction field K', a generic map

$$u'\colon \operatorname{Spec} K' \longrightarrow U \times_X \operatorname{Spec} P,$$

and a map $a'\colon \operatorname{Spec} A' \to \operatorname{Spec} P$ whose closed point maps to z. Put $k' = A'/\mathfrak{m}_{A'}$. The closed-point residue lemma gives a field embedding $\iota\colon K \hookrightarrow k'$ and identifies the residue of the ring map $P \to A'$ with $P \twoheadrightarrow K \hookrightarrow k'$. Choose, by the residue-extension lemma, a valuation ring $A'' \subseteq k'$ with fraction field k' dominating $\iota(A)$.

Let $C = C(A', A'')$. The composite-valuation lemma makes C a valuation ring with fraction field K', gives $C/\mathfrak{m}_{A'} = A''$, and gives the two comparison maps from $\operatorname{Spec} A'$ and $\operatorname{Spec} A''$ to $\operatorname{Spec} C$. The residue of the structural ring map $R \to P \to A'$ is

$$R \longrightarrow A \longrightarrow K \xrightarrow{\iota} k',$$

because the original valuation square commutes and the closed point of the probe is z. Its image lies in A'', so the base-factorization lemma corestricts $R \to A'$ to $R \to C$. The generic probe map projected to U, together with $\operatorname{Spec} C \to S$, is therefore a commutative dense-source valuation square. The composite-lift lemma gives $g_C\colon \operatorname{Spec} C \to X$.

Let $g'\colon \operatorname{Spec} A' \to \operatorname{Spec} P \to X$ be the map from the probe. Both g' and the restriction of g_C along $\operatorname{Spec} A' \to \operatorname{Spec} C$ lie over the same map to S, and their generic restrictions are the projection of u' through U. The A'-comparison lemma gives their equality. Now restrict g_C, not g', along the closed immersion

$$\operatorname{Spec} A'' = \operatorname{Spec}(C/\mathfrak{m}_{A'}) \longrightarrow \operatorname{Spec} C$$

and call the result $g_{A''}$. The composite generic-comparison lemma shows that its restriction to $\operatorname{Spec} k'$ is $\operatorname{Spec} k' \to \operatorname{Spec} K \xrightarrow{x_K} X$. Finally, intersection descent applied to $A \subseteq A''$ produces a map $x_A\colon \operatorname{Spec} A \to X$ over S whose generic restriction is x_K. This is the required lift. □

Lemma 2.7.31 (Quasi-compact valuative existence implies universal closedness, [1, Tag 01KF]). *Uses:*

Lean: `AlgebraicGeometry.ValuativeCriterion.universallyClosed_of_exists`

Let $f\colon X \to S$ be quasi-compact. If every valuation square for f admits a lift, then f is universally closed.

Proof. Uses:

This is the existence part of the valuative criterion for universal closedness. Quasi-compactness is exactly the additional hypothesis needed for the implication from valuative existence to universal closedness. □

Lemma 2.7.32 (Universal closedness from dense-source lifting). *Uses:* `def:nagata-dense-valuative-lifting-property`

Lean: `AlgebraicGeometry.ValuativeCriterion.universallyClosed_of_denseSourceLifting`

Let $f\colon X \to S$ and $h\colon U \to X$ be quasi-compact, assume $h(U)$ is dense in X, and assume $\mathsf{DVL}(f, h)$. Then f is universally closed.

Proof. Uses: `lemma:nagata-dense-source-valuative-existence,lemma:nagata-qc-valuative-existence-universally-closed`

The dense-source construction supplies a lift for every valuation square of f. Since f is quasi-compact, the valuative criterion for universal closedness applies. □

Lemma 2.7.33 (The diagonal inherits the dense-source lifting property). *Uses:* `def:nagata-dense-valuative-lifting-property`

Lean: `AlgebraicGeometry.ValuativeCriterion.denseSourceLifting_diagonal`

If $\mathsf{DVL}(f, h)$, then

$$\mathsf{DVL}(\Delta_f, h)$$

for $\Delta_f\colon X \to X \times_S X$, where $h\colon U \to X$ is used as the dense-source map into the source of the diagonal.

Proof. Uses: def:nagata-dense-valuative-lifting-property

A map $\operatorname{Spec} A \to X \times_S X$ is a pair (a_1, a_2) of maps to X with the same composite to S. In a dense-source valuation square for Δ_f, both generic restrictions equal the given map $\operatorname{Spec} K \to U \to X$. Thus a_1 and a_2 are two lifts of one valuation square for f. The uniqueness clause of $\mathsf{DVL}(f, h)$ gives $a_1 = a_2$, so the pair factors through Δ_f. This factor is unique because every diagonal is a monomorphism. □

Lemma 2.7.34 (A universally closed diagonal is a closed immersion)**.** *Uses:*

Lean: *AlgebraicGeometry.separated_of_universallyClosed_diagonal*

Let $f\colon X \to S$ be a morphism. If its diagonal $\Delta_f\colon X \to X \times_S X$ is universally closed, then Δ_f is a closed immersion and f is separated.

Proof. Uses:

Every diagonal of schemes is an immersion. A universally closed morphism is closed, so the image of Δ_f is closed in $X \times_S X$. An immersion with closed image is a closed immersion. The defining criterion for separatedness now gives that f is separated. □

Lemma 2.7.35 ([1, Tag 0894])**.** *Uses:* def:nagata-dense-valuative-lifting-property

Lean: *AlgebraicGeometry.ValuativeCriterion.universallyClosed_of_dense*

Let $f\colon X \to S$ and $h\colon U \to X$ be quasi-compact morphisms, and assume $h(U)$ is dense in X. Suppose that for every valuation ring A with fraction field K, every commutative diagram

$$\operatorname{Spec} K \longrightarrow U \longrightarrow X, \qquad \operatorname{Spec} A \longrightarrow S$$

has a unique lift $\operatorname{Spec} A \to X$. Then f is universally closed. If f is quasi-separated, it is also separated.

Proof. Uses: lemma:nagata-dense-source-universally-closed, lemma:nagata-dense-valuative-diagonal-lifting, lemma:nagata-universally-closed-diagonal-separated

The hypothesis is precisely $\mathsf{DVL}(f, h)$, so Lemma 2.7.32 makes f universally closed.

Suppose now that f is quasi-separated. Then $\Delta_f\colon X \to X \times_S X$ is quasi-compact. The map h remains quasi-compact and has dense image in the source X of the diagonal, and Lemma 2.7.33 gives $\mathsf{DVL}(\Delta_f, h)$. Applying Lemma 2.7.32 to Δ_f makes the diagonal universally closed. It is consequently a closed immersion by Lemma 2.7.34, and f is separated. □

2.8 The two-open compactification theorem

Throughout this section let S be Noetherian, let $U \to S$ be separated and of finite type, and fix an open cover $U = U_1 \cup U_2$ such that $U_1 \cap U_2$ is dense in U. Assume each $U_i \to S$ is compactifiable.

Definition 2.8.1 (Chosen dense compactifications of the two opens)**.** Uses: def:nagata-compactifiable, def:nagata-dense-refinement

Lean: AlgebraicGeometry.Compactifiable.twoOpenDenseCompactifications

Let C_i^0 be the compactification selected by classical choice from the nonempty type witnessing compactifiability of $U_i \to S$. Define

$$C_i = \mathsf{Dense}(C_i^0) = \big(U_i \hookrightarrow X_i \longrightarrow S\big).$$

Lemma 2.8.2 (Properties of the chosen dense compactifications)**.** *Uses:* def:nagata-two-open-dense-compactifications

Lean: *AlgebraicGeometry.Compactifiable.twoOpenDenseCompactifications_spec*

For $i = 1, 2$, the constructed C_i is a scheme-theoretically dense compactification of $U_i \to S$; in particular $X_i \to S$ is proper and $U_i \hookrightarrow X_i$ is a quasi-compact open immersion.

Proof. Uses: prop:nagata-dense-refinement

This is exactly Proposition 2.1.17, applied to each selected compactification C_i^0. □

Definition 2.8.3 (Chosen graph extensions)**.** Uses: def:nagata-two-open-dense-compactifications, def:nagata-graph-extension

Lean: AlgebraicGeometry.Compactifiable.twoOpenGraphExtensions

Apply the graph-extension construction to $U_i \subseteq U$ and C_i. Denote its output by

$$X_i^{\mathrm{gr}} \longrightarrow X_i, \qquad U_i \subseteq V_i \subseteq X_i^{\mathrm{gr}}, \qquad \psi_i\colon V_i \longrightarrow U.$$

From now on replace X_i by X_i^{gr} and retain the notation X_i for the resulting proper ambient scheme.

Lemma 2.8.4 (Properties of the chosen graph extensions). *Uses:* *def:nagata-two-open-graph-extensions*

Lean: *AlgebraicGeometry.Compactifiable.twoOpenGraphExtensions_spec*

Each replacement $X_i \to X_i^{\mathrm{old}}$ is U_i-admissible; ψ_i is proper, extends the inclusion $U_i \hookrightarrow U$, satisfies $\psi_i^{-1}(U_i) = U_i$, and is an isomorphism over U_i.

Proof. Uses: lemma:nagata-graph-extension

These are the properties of the constructed graph extension in Lemma 2.6.9. Properness of the new $X_i \to S$ follows by composing the proper admissible blow-up with the original proper compactification map. □

Definition 2.8.5 (The four boundary loci). Uses: def:nagata-two-open-graph-extensions

Lean: AlgebraicGeometry.Compactifiable.boundaryLociConstruction

For $\{i,j\} = \{1,2\}$, define

$$Z_i = U \setminus U_j \subseteq U_i, \qquad Z_{i,k} = \psi_i^{-1}(Z_k) \subseteq V_i \quad (k = 1,2).$$

Lemma 2.8.6 (Closedness and disjointness of the four boundary loci). *Uses:*

Lean: *AlgebraicGeometry.Compactifiable.boundaryLoci*

The subsets Z_i are closed in U. Each $Z_{i,k}$ is closed in V_i, and $Z_{i,i} \cap Z_{i,j} = \varnothing$. Moreover ψ_i is an isomorphism over an open neighbourhood of Z_i.

Proof. Uses: lemma:nagata-graph-extension

Since $Z_i = U \setminus U_j$, it is closed in U and contained in U_i. Proper morphisms are continuous, so the inverse images $Z_{i,k}$ are closed. Their disjointness is the inverse image of $Z_i \cap Z_j = \varnothing$. Finally $Z_i \subseteq U_i$ and $\psi_i^{-1}(U_i) \to U_i$ is the isomorphism supplied by Lemma 2.6.9. □

Definition 2.8.7 (Prepared ambient schemes with separated boundary closures). Uses: def:nagata-two-open-boundary-loci,def:nagata-separate-constructible-closed

Lean: AlgebraicGeometry.Compactifiable.preparedBoundaryConstruction

For $\{i,j\} = \{1,2\}$, define

$$X_i^{\mathrm{prep}} = \mathsf{CSepBl}_{V_i}(Z_{i,i}, Z_{i,j})$$

and transport V_i, U_i, and ψ_i along the chosen isomorphism over V_i. Replace X_i by X_i^{prep} and let $\overline{Z}_{i,k}$ denote the closure of the transported $Z_{i,k}$ in the new X_i.

Lemma 2.8.8 (Properties of the prepared boundary closures). *Uses:* *def:nagata-two-open-prepared-boundaries*

Lean: *AlgebraicGeometry.Compactifiable.preparedBoundaryClosures*

The constructed replacement $X_i^{\mathrm{prep}} \to X_i^{\mathrm{old}}$ is V_i-admissible, the closures $\overline{Z}_{i,i}$ and $\overline{Z}_{i,j}$ are disjoint, and the open V_i together with the proper map ψ_i is unchanged.

Proof. Uses: lemma:nagata-boundary-loci,lemma:nagata-separate-constructible-closed

By Lemma 2.8.6, the subsets $Z_{i,i}$ and $Z_{i,j}$ are disjoint constructible subsets closed in V_i. Apply the property of the explicitly constructed separation blow-up. It is V_i-admissible, so the selected isomorphism over V_i transports ψ_i without changing it. The conclusion of Lemma 2.5.15 is exactly the disjointness of the two closures. □

Lemma 2.8.9 (Disjoint boundary closures persist). *Uses:*

Lean: *AlgebraicGeometry.StrictTransform.disjoint_of_disjoint*

If $\overline{Z}_{i,i}$ and $\overline{Z}_{i,j}$ are disjoint, then their strict transforms remain disjoint after every further V_i-admissible blow-up.

Proof. Uses: lemma:nagata-strict-transform-restrict

The two strict transforms are contained in the inverse images of the two original closed subsets. Those inverse images are disjoint. □

Definition 2.8.10 (The two-open comparison space). Uses: def:nagata-two-open-prepared-boundaries,def:nagata-scheme-theoretic-image

Lean: AlgebraicGeometry.Compactifiable.comparisonSpace

Define

$$V_{12} = V_1 \times_U V_2.$$

The separatedness of $U \to S$ gives a closed immersion $V_{12} \to V_1 \times_S V_2$, hence an immersion $V_{12} \to X_1 \times_S X_2$. Define

$$X_{12} = \mathrm{SIm}\big(V_{12} \longrightarrow X_1 \times_S X_2\big)$$

and let $p_i \colon X_{12} \to X_i$ be the restrictions of the two product projections.

Lemma 2.8.11 (Properness of the comparison projections)**.** *Uses:*

Lean: `AlgebraicGeometry.Compactifiable.comparisonProjection_isProper`

Both projections $p_i\colon X_{12} \to X_i$ are proper.

Proof. Uses:

The scheme X_{12} is closed in $X_1 \times_S X_2$. The projection from this product to X_i is the base change of the proper morphism $X_j \to S$. Compose the closed immersion with that proper projection. □

Definition 2.8.12 (Distinguished comparison boundary)**.** Uses: `def:nagata-two-open-boundary-loci,def:nagata-two-open-comparison-space`

Lean: `AlgebraicGeometry.Compactifiable.comparisonBoundary`

Define

$$Z_{12,2} = (p_1|_{V_{12}})^{-1}(Z_{1,2}).$$

Equivalently, by the fibre-product identities, this is $(p_2|_{V_{12}})^{-1}(Z_{2,2})$.

Lemma 2.8.13 (Restriction of the comparison image over the proper-extension open)**.** *Uses:*

Lean: `AlgebraicGeometry.Compactifiable.comparison_preimage_properExtension`

For $\{i,j\} = \{1,2\}$, the canonical morphism

$$V_{12} = V_1 \times_U V_2 \longrightarrow p_i^{-1}(V_i)$$

is an isomorphism. Under this identification, $p_i^{-1}(V_i) \to V_i$ is the base change of $\psi_j\colon V_j \to U$ along $\psi_i\colon V_i \to U$.

Proof. Uses: `lemma:nagata-scheme-image-restrict-open,lemma:nagata-proper-to-separated-target`

Restrict the scheme-theoretic image defining X_{12} to the open target $V_i \times_S X_j \subseteq X_i \times_S X_j$. The inverse image of this open in V_{12} is all of V_{12}, so Lemma 2.1.9 identifies $p_i^{-1}(V_i)$ with the scheme-theoretic image of

$$V_{12} \longrightarrow V_i \times_S X_j.$$

The map $V_{12} \to V_i \times_S V_j$ is the pullback of the diagonal $U \to U \times_S U$; it is therefore a closed immersion because $U \to S$ is separated. Composing with the open immersion $V_i \times_S V_j \to V_i \times_S X_j$ shows that the displayed map is an immersion. It is also proper: its composite to V_i is the base change of the proper morphism ψ_j, while $V_i \times_S X_j \to V_i$ is separated; apply Lemma 2.6.2. A proper immersion is a closed immersion, so its scheme-theoretic image is V_{12} itself. The description of the projection follows directly from the defining fibre square for V_{12}. □

Definition 2.8.14 (Improved first comparison projection)**.** Uses: `def:nagata-two-open-comparison-space,def:nagata-two-open-comparison-boundary,def:nagata-isomorphism-near-closure,def:nagata-strict-transform`

Lean: `AlgebraicGeometry.Compactifiable.improvedComparisonConstruction`

Let $N_2 \subseteq U$ be the maximal open over which $\psi_2\colon V_2 \to U$ is an isomorphism. Apply $\mathsf{IsoBl}_{V_1,Z_{1,2}}(p_1)$ to the proper projection $p_1\colon X_{12} \to X_1$. Define X_1^{cmp} to be this constructed blow-up and define X_{12}^{cmp} to be the strict transform of X_{12}. Transport V_1, all four boundary loci, and the projection maps along the selected isomorphism over V_1, and thereafter retain the notation X_1, X_{12}, p_i for the transported data.

Lemma 2.8.15 (Properties of the improved first comparison projection)**.** *Uses:* `def:nagata-two-open-improved-comparison`

Lean: `AlgebraicGeometry.Compactifiable.prepareComparisonProjection`

The constructed map $X_1^{\mathrm{cmp}} \to X_1^{\mathrm{old}}$ is V_1-admissible, the transported p_1 is an isomorphism over an open neighbourhood of $\overline{Z}_{1,2}$, and all previously established boundary-disjointness relations still hold.

Proof. Uses: `lemma:nagata-boundary-loci,lemma:nagata-comparison-projections-proper,lemma:nagata-comparison-restrict-proper-open,lemma:nagata-isomorphism-near-closure,lemma:nagata-scheme-image-strict-transform,lemma:nagata-boundary-disjoint-persists`

By Lemma 2.8.13, the restriction of p_1 over V_1 is the base change $V_1 \times_U V_2 \to V_1$ of ψ_2. The maximal isomorphism open N_2 contains Z_2 by Lemma 2.8.6; hence p_1 is an isomorphism over $\psi_1^{-1}(N_2)$, an open containing $Z_{1,2}$.

The subset $Z_{1,2}$ is closed in V_1, and p_1 is proper by Lemma 2.8.11. Therefore the property of $\mathsf{IsoBl}_{V_1,Z_{1,2}}(p_1)$ gives the asserted isomorphism near the closure. The new comparison scheme is the strict transform by Lemma 2.6.7. Finally Lemma 2.8.9 preserves every earlier disjointness relation under this further V_1-admissible blow-up. □

Lemma 2.8.16 (Density over the prepared boundary). *Uses:*

Lean: `AlgebraicGeometry.Compactifiable.comparisonBoundary_dense`

The subset $Z_{12,2}$ is dense in $p_1^{-1}(\overline{Z}_{1,2})$.

Proof. Uses: `lemma:nagata-comparison-isIso-near-boundary`

Near $\overline{Z}_{1,2}$, the map p_1 is an isomorphism. Under this identification, $Z_{12,2}$ is identified with $Z_{1,2}$, which is dense in its closure $\overline{Z}_{1,2}$. □

Lemma 2.8.17 (Key boundary disjointness). *Uses:*

Lean: `AlgebraicGeometry.Compactifiable.keyBoundaryDisjointness`

Inside $X_1 \times_S X_2$ one has

$$X_{12} \cap (\overline{Z}_{1,2} \times_S \overline{Z}_{2,1}) = \varnothing.$$

Proof. Uses: `lemma:nagata-prepared-boundaries,lemma:nagata-comparison-boundary-density`

By Lemma 2.8.16, the dense subset $Z_{12,2}$ of $p_1^{-1}(\overline{Z}_{1,2})$ maps under p_2 into $Z_{2,2}$. Continuity therefore maps the whole closure into $\overline{Z}_{2,2}$. This is disjoint from $\overline{Z}_{2,1}$ by Lemma 2.8.8. □

Definition 2.8.18 (Intermediate two-open schemes). Uses: `def:nagata-two-open-improved-comparison,def:nagata-two-open-prepared-boundaries`

Lean: `AlgebraicGeometry.Compactifiable.intermediateSchemes`

For $\{i, j\} = \{1, 2\}$, define

$$X_i^\circ = X_i \setminus \overline{Z}_{i,j}$$

and glue the open subschemes $U_i \subseteq U$ and $U_i \subseteq X_i^\circ$ by the identity. Denote the result by

$$W_i = U \amalg_{U_i} X_i^\circ.$$

The construction includes the two open immersions $U \hookrightarrow W_i$ and $X_i^\circ \hookrightarrow W_i$ and the induced structural map $W_i \to S$.

Lemma 2.8.19 (Closed image of the proper extension graph). *Uses:*

Lean: `AlgebraicGeometry.Compactifiable.properExtensionGraph_isClosed`

The morphism

$$\gamma_i = (\psi_i, \mathrm{incl}) \colon V_i \longrightarrow U \times_S X_i$$

is proper. In particular, its image is closed.

Proof. Uses: `lemma:nagata-proper-to-separated-target`

Regard both schemes as schemes over U. The source $V_i \to U$ is proper. The target $U \times_S X_i \to U$ is separated because it is the base change of the separated morphism $X_i \to S$. Apply Lemma 2.6.2 to the commutative triangle $V_i \to U \times_S X_i \to U$. A proper morphism is closed, so its image is closed. □

Lemma 2.8.20 (Constructibility of the overlap image). *Uses:*

Lean: `AlgebraicGeometry.Compactifiable.intermediateOverlap_constructible`

The image of

$$U_i \longrightarrow U \times_S (X_i \setminus \overline{Z}_{i,j})$$

is constructible.

Proof. Uses:

The source and target are of finite type over the Noetherian scheme S. The displayed morphism is an immersion and hence of finite type. Chevalley's theorem makes its image constructible. □

Lemma 2.8.21 (A specialization leaving U_i meets the forbidden boundary). *Uses:*

Lean: `AlgebraicGeometry.Compactifiable.intermediateSpecialization_mem_boundary`

Let a point in the image of U_i specialize in $U \times_S X_i$ to a point (u, x). If $u \notin U_i$, then $x \in \overline{Z}_{i,j}$.

Proof. Uses: `lemma:nagata-proper-extension-graph-closed,lemma:nagata-boundary-loci`

The image of U_i is contained in the image of $\gamma_i \colon V_i \to U \times_S X_i$. The latter image is closed by Lemma 2.8.19; hence the specialization (u, x) is $\gamma_i(v)$ for some $v \in V_i$. Thus $\psi_i(v) = u$ and the image of v in X_i is x. If $u \notin U_i$, then $u \in Z_j$, so $v \in Z_{i,j} = \psi_i^{-1}(Z_j)$. Therefore $x \in Z_{i,j} \subseteq \overline{Z}_{i,j}$. □

Lemma 2.8.22 (Specialization stability inside the boundary complement). *Uses:*

Lean: `AlgebraicGeometry.Compactifiable.intermediateOverlap_specializationClosed`

The image of U_i in $U \times_S (X_i \setminus \overline{Z}_{i,j})$ is stable under specialization.

Proof. Uses: `lemma:nagata-intermediate-specialization-boundary`

Let a point of the image specialize to (u, x) in the displayed target. If $u \notin U_i$, then Lemma 2.8.21 gives $x \in \overline{Z}_{i,j}$, contradicting the definition of the target. Hence $u \in U_i$. Over U_i, the proper extension ψ_i is the chosen isomorphism with inverse the open inclusion, so (u, x) is again the image of u. □

Lemma 2.8.23 (Closedness of the intermediate overlap). *Uses:*

Lean: `AlgebraicGeometry.Compactifiable.intermediateOverlap_isClosedImmersion`

The canonical immersion

$$U_i \longrightarrow U \times_S (X_i \setminus \overline{Z}_{i,j})$$

is a closed immersion.

Proof. Uses: `lemma:nagata-intermediate-overlap-constructible,lemma:nagata-intermediate-overlap-specialization-closed`

By Lemma 2.8.20, the image is constructible. By Lemma 2.8.22, it is stable under specialization. The target is Noetherian, so the image is closed. An immersion with closed image is a closed immersion. □

Lemma 2.8.24 (Separatedness of the intermediate schemes). *Uses:*

Lean: `AlgebraicGeometry.Compactifiable.intermediate_isSeparated`

Each $W_i \to S$ is separated and of finite type.

Proof. Uses: `lemma:nagata-intermediate-overlap-closed,lemma:nagata-separated-two-open-cover`

Finite type is local on the source. Apply Lemma 2.2.2 to the two open pieces U and $X_i \setminus \overline{Z}_{i,j}$, using Lemma 2.8.23 for the mixed overlap. □

Lemma 2.8.25 (The pair of valuation extensions factors through X_{12}). *Uses:*

Lean: `AlgebraicGeometry.Compactifiable.valuationPair_factorsComparison`

Let A be a valuation ring with fraction field K. Suppose $\mathrm{Spec}(K) \to U_1 \cap U_2$ is an S-morphism and let $g_i \colon \mathrm{Spec}(A) \to X_i$ be extensions. Then the pair

$$(g_1, g_2) \colon \mathrm{Spec}(A) \to X_1 \times_S X_2$$

factors through X_{12}.

Proof. Uses: `lemma:nagata-valuation-factor-closed`

The generic restriction factors through V_{12} and hence through its scheme-theoretic image X_{12}. Apply Lemma 2.7.1 to the closed immersion $X_{12} \hookrightarrow X_1 \times_S X_2$. □

Lemma 2.8.26 (One valuation extension avoids the forbidden boundary). *Uses:* `def:nagata-two-open-comparison-space`, `def:nagata-two-open-intermediate-schemes`

Lean: `AlgebraicGeometry.Compactifiable.exists_valuationExtension_avoidingBoundary`

Let A be a valuation ring with fraction field K. Let $\mathrm{Spec}(K) \to U_1 \cap U_2$ be an S-morphism and let $g_i \colon \mathrm{Spec}(A) \to X_i$ be extensions. At least one g_i factors through $X_i \setminus \overline{Z}_{i,j}$, and hence determines a map $\mathrm{Spec}(A) \to W_i$ extending the given generic map.

Proof. Uses: `lemma:nagata-key-boundary-disjointness,lemma:nagata-valuation-pair-factors-comparison`

Otherwise the closed point would map into $\overline{Z}_{1,2}$ under g_1 and into $\overline{Z}_{2,1}$ under g_2. By Lemma 2.8.25, the pair factors through X_{12}, contradicting Lemma 2.8.17. An open subset of $\mathrm{Spec}(A)$ containing the closed point is all of $\mathrm{Spec}(A)$, so avoiding the boundary at the closed point gives the asserted factorization. □

Definition 2.8.27 (Final common modification for the two-open construction). Uses: `def:nagata-two-open-intermediate-schemes,def:nagata-common-modification`

Lean: `AlgebraicGeometry.Compactifiable.twoOpenFinalModification`

Apply the common-modification construction to the two open immersions $U \hookrightarrow W_1$ and $U \hookrightarrow W_2$. Denote its output by

$$W_i' \longrightarrow W_i, \qquad W = \mathsf{CM}(W_1, W_2; U),$$

with the two constructed open immersions $W_i' \hookrightarrow W$ and their common open subscheme U.

Lemma 2.8.28 (Structural properties of the final common modification). *Uses:* `def:nagata-two-open-final-modification`

Lean: `AlgebraicGeometry.Compactifiable.twoOpenFinalModification_spec`

Each $W_i' \to W_i$ is U-admissible; the maps $W_i' \hookrightarrow W$ are open immersions covering W; their copies of U agree; and $W \to S$ is separated and of finite type. In particular the induced map $j_W \colon U \to W$ is an open immersion.

Proof. Uses: lemma:nagata-intermediate-separated,lemma:nagata-common-modification
The schemes W_i are separated and of finite type by Lemma 2.8.24. Thus all hypotheses of the common-modification property apply, and Lemma 2.6.6 gives every stated assertion. The map j_W is the composite of $U \hookrightarrow W_i'$ with either open immersion $W_i' \hookrightarrow W$; the two composites coincide by the gluing construction. □

Definition 2.8.29 (Two-open compactification candidate)**.** Uses: def:nagata-two-open-final-modification

Lean: AlgebraicGeometry.Compactifiable.twoOpenCandidate
Define $\mathsf{C}_{12}(U/S)$ to be the constructed factorization

$$U \xrightarrow{j_W} W \longrightarrow S.$$

Lemma 2.8.30 (Density after the final common modification)**.** *Uses:* def:nagata-two-open-compactification-candidate

Lean: AlgebraicGeometry.Compactifiable.dense_intersection_in_finalGlue
The constructed map $U_1 \cap U_2 \to W$ is quasi-compact and has dense image.

Proof. Uses: lemma:nagata-two-open-dense-compactifications,lemma:nagata-two-open-graph-extensions,lemma:nagata-two-open-final-modification
The intersection is dense in U by hypothesis. Each U_i is scheme-theoretically dense in its selected X_i, and all later admissible blow-ups preserve the relevant open. Consequently U is dense in both modified pieces W_i'. These two opens cover W, so $U_1 \cap U_2$ is dense in W. The map is quasi-compact because every open and blow-up in the finite construction is quasi-compact. □

Lemma 2.8.31 (Valuative lifting for the two-open candidate)**.** *Uses:* def:nagata-two-open-compactification-candidate

Lean: AlgebraicGeometry.Compactifiable.twoOpenCandidate_valuative
Let A be a valuation ring with fraction field K. Every commutative diagram whose generic map $\operatorname{Spec}(K) \to W$ factors through $U_1 \cap U_2$ has a unique lift $\operatorname{Spec}(A) \to W$ over S.

Proof. Uses: lemma:nagata-one-valuation-avoids-boundary,lemma:nagata-blowup-proper,lemma:nagata-two-open-final-modification
Properness of the selected compactifications $X_i \to S$ extends the generic map to maps $g_i\colon \operatorname{Spec}(A) \to X_i$. By Lemma 2.8.26, one extension factors through $X_i \setminus \overline{Z}_{i,j}$ and hence through W_i. The generic map already lifts to the preserved open $U \subseteq W_i'$. Properness of the admissible blow-up $W_i' \to W_i$ supplies a lift to W_i', and its open map to W gives the desired lift. Uniqueness follows from separatedness of $W \to S$ in Lemma 2.8.28. □

Lemma 2.8.32 (Properness of the two-open candidate)**.** *Uses:* def:nagata-two-open-compactification-candidate

Lean: AlgebraicGeometry.Compactifiable.twoOpenCandidate_isProper
The constructed structural morphism $W \to S$ is proper.

Proof. Uses: lemma:nagata-final-density,lemma:nagata-two-open-valuative-lifting,lemma:nagata-two-open-final-modification,lemma:nagata-dense-source-valuative
The morphism $U_1 \cap U_2 \to W$ is quasi-compact and dense by Lemma 2.8.30, and its valuation diagrams admit unique lifts by Lemma 2.8.31. The dense-source valuative criterion makes $W \to S$ universally closed. It is separated and of finite type by Lemma 2.8.28; hence it is proper. □

Theorem 2.8.33 ([1, Tag 0F40])**.** *Uses:* def:nagata-compactification, def:nagata-two-open-compactification-candidate

Lean: AlgebraicGeometry.Compactifiable.union_of_two
The constructed factorization $\mathsf{C}_{12}(U/S)$ is a compactification of $U \to S$. Consequently $U \to S$ is compactifiable.

Proof. Uses: lemma:nagata-two-open-final-modification,lemma:nagata-two-open-candidate-proper,cor:nagata-open-immersion-qc-over-separated-target,def:nagata-compactification
By Lemma 2.8.28, the first map $j_W\colon U \to W$ is an open immersion and $W \to S$ is separated and of finite type. It is proper by Lemma 2.8.32. The open immersion j_W is quasi-compact by Corollary 2.6.4, applied to the triangle $U \to W \to S$. The defining gluing maps give the factorization equality, so all fields of Definition 2.1.1 are present. □

2.9 The Noetherian case

Definition 2.9.1 (A finite affine base-cover witness for Tag 04II)**.** Uses:

Lean: `AlgebraicGeometry.QuasiAffineImmersionData.baseCoverWitness`

Let X be absolutely quasi-affine and let $\pi\colon X \to S$ be locally of finite type. The inverse images of affine opens of S cover X. Choose a finite subcover, using the quasi-compactness carried by absolute quasi-affineness, reindex it by a finite type I, and retain its affine and cover certificates. Denote this record by $\mathsf{B}_{\mathrm{base}}(X/S)$.

Lemma 2.9.2 (Existence of finite affine base-cover data for Tag 04II)**.** *Uses:* *def: nagata-quasi-affine-base-cover-witness*

Lean: *AlgebraicGeometry.QuasiAffineImmersionData.exists_baseCover*

Constructed by: *def: nagata-quasi-affine-base-cover-witness*

Let X be an absolutely quasi-affine scheme and let $\pi\colon X \to S$ be locally of finite type. The candidate type of a finite type I and affine opens $(V_i)_{i\in I}$ of S satisfying $X = \bigcup_{i\in I} \pi^{-1}(V_i)$ is nonempty.

Proof. Uses: `def:nagata-quasi-affine-base-cover-witness`

The record $\mathsf{B}_{\mathrm{base}}(X/S)$ constructed in Definition 2.9.1 is an inhabitant of the displayed candidate type. Local finite type is retained for the later chart rings but is not needed for this finite-subcover step. □

Definition 2.9.3 (Selected finite affine base-cover data for Tag 04II)**.** Uses: `def:nagata-quasi-affine-base-cover-witness`

Lean: `AlgebraicGeometry.QuasiAffineImmersionData.baseCover`

Constructed by: `def:nagata-quasi-affine-base-cover-witness`

Define $(I, (V_i)_{i\in I})$ to be the data in $\mathsf{B}_{\mathrm{base}}(X/S)$. Retain in the record the finite instance for I, affineness of every V_i, and the displayed cover equality.

Lemma 2.9.4 (The selected base-cover index is finite)**.** *Uses:* *def: nagata-quasi-affine-base-cover-data*

Lean: *AlgebraicGeometry.QuasiAffineImmersionData.baseCover_finite*

The index type I of the selected base-cover data is finite.

Proof. Uses: `def:nagata-quasi-affine-base-cover-witness`

This is the finite-instance field of $\mathsf{B}_{\mathrm{base}}(X/S)$ constructed in Definition 2.9.1 and retained verbatim by the selected record. □

Lemma 2.9.5 (The selected base opens cover the image)**.** *Uses:* *def: nagata-quasi-affine-base-cover-data*

Lean: *AlgebraicGeometry.QuasiAffineImmersionData.baseCover_covers*

The opens $\pi^{-1}(V_i)$ cover X.

Proof. Uses: `def:nagata-quasi-affine-base-cover-witness`

This is the cover-equality field of $\mathsf{B}_{\mathrm{base}}(X/S)$ from Definition 2.9.1. □

Definition 2.9.6 (A subordinate affine principal-cover witness)**.** Uses: `def:nagata-quasi-affine-base-cover-data`

Lean: `AlgebraicGeometry.QuasiAffineImmersionData.principalCoverWitness`

Put $A = \Gamma(X, \mathcal{O}_X)$. Refine the selected inverse-image cover by affine basic opens X_f, choose a finite subcover by quasi-compactness, and record for every chosen member a containing base-chart index. More precisely, for a quasi-affine scheme those principal opens X_f which are affine form a basis; no assertion is made that every principal open is affine. For each selected affine member, the quasi-compact, quasi-separated basic-open localization theorem records $A_f \simeq \Gamma(X_f, \mathcal{O})$. Denote the resulting finite record by $\mathsf{B}_{\mathrm{prin}}(X/S)$.

Lemma 2.9.7 (Existence of a finite subordinate affine principal cover)**.** *Uses:* *def: nagata-quasi-affine-principal-cover-witness*

Lean: *AlgebraicGeometry.QuasiAffineImmersionData.exists_principalCover*

Constructed by: *def: nagata-quasi-affine-principal-cover-witness*

Put $A = \Gamma(X, \mathcal{O}_X)$. The candidate type of a finite type J, sections $f_j \in A$, and a map $\kappa\colon J \to I$ satisfying, for $U_j = X_{f_j}$,

$$X = \bigcup_{j\in J} U_j, \qquad U_j \subseteq \pi^{-1}(V_{\kappa(j)}),$$

with every U_j affine and restriction identifying $A_{f_j} \cong \Gamma(U_j, \mathcal{O}_{U_j})$, is nonempty.

Proof. Uses: def:nagata-quasi-affine-principal-cover-witness
The record $\mathsf{B}_{\mathrm{prin}}(X/S)$ from Definition 2.9.6 is an inhabitant of the displayed candidate type. □

Definition 2.9.8 (Selected subordinate principal cover). Uses: def:nagata-quasi-affine-base-cover-data, def:nagata-quasi-affine-principal-cover-witness

Lean: AlgebraicGeometry.QuasiAffineImmersionData.principalCover

Constructed by: def:nagata-quasi-affine-principal-cover-witness
Define $(J, (f_j)_j, \kappa)$ to be the data in $\mathsf{B}_{\mathrm{prin}}(X/S)$, and write $U_j = X_{f_j}$. The selected record includes finiteness, coverage, subordination, affineness, and the localization isomorphism for every j.

Lemma 2.9.9 (The selected principal-cover index is finite). Uses: def:nagata-quasi-affine-principal-cover-data

Lean: AlgebraicGeometry.QuasiAffineImmersionData.principalCover_finite
The index type J is finite.

Proof. Uses: def:nagata-quasi-affine-principal-cover-witness
Project the finite-instance field of $\mathsf{B}_{\mathrm{prin}}(X/S)$ from Definition 2.9.6. □

Lemma 2.9.10 (The selected principal opens cover the source). Uses: def:nagata-quasi-affine-principal-cover-data

Lean: AlgebraicGeometry.QuasiAffineImmersionData.principalCover_covers
One has $X = \bigcup_{j \in J} U_j$.

Proof. Uses: def:nagata-quasi-affine-principal-cover-witness
Project the cover-equality field of $\mathsf{B}_{\mathrm{prin}}(X/S)$ from Definition 2.9.6. □

Lemma 2.9.11 (Subordination of the selected principal cover). Uses: def:nagata-quasi-affine-base-cover-data, def:nagata-quasi-affine-principal-cover-data

Lean: AlgebraicGeometry.QuasiAffineImmersionData.principalCover_subordinate
For every $j \in J$, one has $U_j \subseteq \pi^{-1}(V_{\kappa(j)})$.

Proof. Uses: def:nagata-quasi-affine-principal-cover-witness
This is the subordinate-containment field of $\mathsf{B}_{\mathrm{prin}}(X/S)$ from Definition 2.9.6. □

Lemma 2.9.12 (Affineness of the selected principal opens). Uses: def:nagata-quasi-affine-principal-cover-data

Lean: AlgebraicGeometry.QuasiAffineImmersionData.principalCover_isAffine
Every $U_j = X_{f_j}$ is affine.

Proof. Uses: def:nagata-quasi-affine-principal-cover-witness
This is the affine field of $\mathsf{B}_{\mathrm{prin}}(X/S)$ from Definition 2.9.6. □

Lemma 2.9.13 (Localization on the selected principal opens). Uses: def:nagata-quasi-affine-principal-cover-data

Lean: AlgebraicGeometry.QuasiAffineImmersionData.principalCover_isLocalization
Restriction induces an isomorphism

$$A_{f_j} \xrightarrow{\sim} C_j := \Gamma(U_j, \mathcal{O}_{U_j})$$

for every j.

Proof. Uses: def:nagata-quasi-affine-principal-cover-witness
This is the localization field of $\mathsf{B}_{\mathrm{prin}}(X/S)$ from Definition 2.9.6. Concretely it is the quasi-compact, quasi-separated basic-open localization theorem applied to f_j. □

Definition 2.9.14 (Finite local algebra-generator witnesses). Uses: def:nagata-quasi-affine-base-cover-data, def:nagata-quasi-affine-principal-cover-data

Lean: AlgebraicGeometry.QuasiAffineImmersionData.localGeneratorsWitness
For every selected principal chart $U_j \to V_{\kappa(j)}$, local finite type and affineness make $\Gamma(V_{\kappa(j)}, \mathcal{O}) \to \Gamma(U_j, \mathcal{O})$ a finite-type ring map. Choose a finite algebra-generating family, reindex it by a finite type K_j, and retain the generation equality. Denote the resulting dependent record by $\mathsf{B}_{\mathrm{gen}}(X/S)$.

Lemma 2.9.15 (Existence of finite local algebra generators). *Uses:* *def: nagata-quasi-affine-local-generators-witness*

Lean: *AlgebraicGeometry.QuasiAffineImmersionData.exists_localGenerators*

Constructed by: *def: nagata-quasi-affine-local-generators-witness*

Write $R_j = \Gamma(V_{\kappa(j)}, \mathcal{O}_{V_{\kappa(j)}})$ and $C_j = \Gamma(U_j, \mathcal{O}_{U_j})$. For every j, the candidate type of a finite type K_j and elements $(g_{jk})_{k \in K_j}$ which generate C_j as an R_j-algebra is nonempty.

Proof. Uses: `def:nagata-quasi-affine-local-generators-witness`
The dependent record $\mathsf{B}_{\mathrm{gen}}(X/S)$ from Definition 2.9.14 supplies an inhabitant for every j. □

Definition 2.9.16 (Selected local algebra generators). Uses: `def:nagata-quasi-affine-base-cover-data`, `def:nagata-quasi-affine-principal-cover-data`, `def:nagata-quasi-affine-local-generators-witness`

Lean: `AlgebraicGeometry.QuasiAffineImmersionData.localGenerators`

Constructed by: `def:nagata-quasi-affine-local-generators-witness`
For every j, take the finite type K_j and the family $(g_{jk})_{k \in K_j} \subseteq C_j$ from $\mathsf{B}_{\mathrm{gen}}(X/S)$. Retain the finite instance and the algebra-generation equality in the record.

Lemma 2.9.17 (Each selected local generator family is finite). *Uses:* *def: nagata-quasi-affine-local-generator-data*

Lean: *AlgebraicGeometry.QuasiAffineImmersionData.localGenerators_finite*
Every index type K_j is finite.

Proof. Uses: `def:nagata-quasi-affine-local-generators-witness`
This is the finite-instance field of $\mathsf{B}_{\mathrm{gen}}(X/S)$ from Definition 2.9.14. □

Lemma 2.9.18 (The selected local families generate). *Uses:* *def: nagata-quasi-affine-local-generator-data*

Lean: *AlgebraicGeometry.QuasiAffineImmersionData.localGenerators_generate*
For each j, the family $(g_{jk})_{k \in K_j}$ generates C_j as an R_j-algebra.

Proof. Uses: `def:nagata-quasi-affine-local-generators-witness`
This is the generation field of $\mathsf{B}_{\mathrm{gen}}(X/S)$ from Definition 2.9.14. □

Definition 2.9.19 (Simultaneous numerator-lift witness). Uses: `def:nagata-quasi-affine-principal-cover-data`, `def:nagata-quasi-affine-local-generator-data`

Lean: `AlgebraicGeometry.QuasiAffineImmersionData.sectionLiftsWitness`
Under the selected localization isomorphism $\Gamma(X, \mathcal{O})_{f_j} \simeq \Gamma(U_j, \mathcal{O})$, represent every selected generator g_{jk} by a fraction $a_{jk}/f_j^{e_{jk}}$ and retain the cleared-denominator equality. Dependent choice over the finite family of pairs gives the record $\mathsf{B}_{\mathrm{lift}}(X/S)$.

Lemma 2.9.20 (Existence of simultaneous numerator lifts). *Uses:* *def: nagata-quasi-affine-section-lifts-witness*

Lean: *AlgebraicGeometry.QuasiAffineImmersionData.exists_sectionLifts*

Constructed by: *def: nagata-quasi-affine-section-lifts-witness*
For every (j,k), the candidate type of $e_{jk} \in \mathbf{N}$ and $a_{jk} \in A$ satisfying, for the restriction map $\rho_j \colon A \to C_j$,

$$\rho_j(a_{jk}) = g_{jk}\rho_j(f_j)^{e_{jk}}$$

is nonempty.

Proof. Uses: `def:nagata-quasi-affine-section-lifts-witness`
The record $\mathsf{B}_{\mathrm{lift}}(X/S)$ constructed in Definition 2.9.19 is an inhabitant of the displayed candidate type. □

Definition 2.9.21 (Selected global-section lifts of the local generators). Uses: `def:nagata-quasi-affine-principal-cover-data`, `def:nagata-quasi-affine-local-generator-data`, `def:nagata-quasi-affine-section-lifts-witness`

Lean: `AlgebraicGeometry.QuasiAffineImmersionData.sectionLifts`

Constructed by: `def:nagata-quasi-affine-section-lifts-witness`
Take (e_{jk}, a_{jk}) from $\mathsf{B}_{\mathrm{lift}}(X/S)$ and retain the displayed restriction equality for each pair (j,k).

Lemma 2.9.22 (Specification of the selected section lifts). *Uses:* *def:nagata-quasi-affine-section-lift-data*

Lean: *AlgebraicGeometry.QuasiAffineImmersionData.sectionLifts_spec*

For every (j,k), $\rho_j(a_{jk}) = g_{jk}\rho_j(f_j)^{e_{jk}}$ in C_j.

Proof. Uses: def:nagata-quasi-affine-section-lifts-witness

This is the equality field of $\mathsf{B}_{\mathrm{lift}}(X/S)$ from Definition 2.9.19. □

Definition 2.9.23 (Affine-space map attached to an absolutely quasi-affine scheme). Uses: def:nagata-quasi-affine-principal-cover-data,def:nagata-quasi-affine-section-lift-data

Lean: AlgebraicGeometry.quasiAffineImmersionAffineSpace

Let $\mathsf{Coord} = J \amalg \coprod_{j\in J} K_j$ and define global sections

$$s_{\mathrm{inl}(j)} = f_j, \qquad s_{\mathrm{inr}(j,k)} = a_{jk}.$$

This finite family defines the S-morphism

$$\iota_X \colon X \longrightarrow Y := \mathbb{A}_S^{\mathsf{Coord}}.$$

A fixed equivalence $\mathsf{Coord} \simeq \{1,\dots,N\}$ identifies Y with $\mathbb{A}_S^N$.

Lemma 2.9.24 (Preimage of each selected standard target open). *Uses:* *def:nagata-quasi-affine-principal-cover-data,def:nagata-quasi-affine-immersion-affine-space*

Lean: *AlgebraicGeometry.quasiAffineImmersionAffineSpace_preimage*

If $D_j = D(x_{\mathrm{inl}(j)}) \subseteq Y$, then $\iota_X^{-1}(D_j) = U_j = X_{f_j}$.

Proof. Uses:

The defining coordinate equation for the section map sends $x_{\mathrm{inl}(j)}$ to f_j. Pullback of a principal open is the principal open of the pulled-back section, giving the asserted equality. □

Definition 2.9.25 (Relative affine target chart). Uses: def:nagata-quasi-affine-base-cover-data,def:nagata-quasi-affine-principal-cover-data,def:nagata-quasi-affine-immersion-affine-space

Lean: AlgebraicGeometry.QuasiAffineImmersionData.relativeStandardChart

For the projection $p\colon Y \to S$, define

$$T_j = D_j \cap p^{-1}(V_{\kappa(j)}).$$

The restriction of ι_X to U_j factors through a morphism $c_j \colon U_j \to T_j$.

Lemma 2.9.26 (Affineness and ring of the relative target chart). *Uses:* *def:nagata-quasi-affine-relative-standard-chart*

Lean: *AlgebraicGeometry.QuasiAffineImmersionData.relativeStandardChart_isAffine*

The scheme T_j is affine and

$$\Gamma(T_j, \mathcal{O}_{T_j}) \cong R_j[(x_c)_{c\in\mathsf{Coord}}]_{x_{\mathrm{inl}(j)}}.$$

Proof. Uses:

Over the affine open $V_{\kappa(j)} = \mathrm{Spec}(R_j)$, relative affine space is the spectrum of the displayed polynomial ring. Intersecting with D_j localizes that ring at $x_{\mathrm{inl}(j)}$. □

Lemma 2.9.27 (Surjectivity of the relative chart coordinate-ring map). *Uses:* *def:nagata-quasi-affine-local-generator-data,def:nagata-quasi-affine-section-lift-data,def:nagata-quasi-affine-relative-standard-chart*

Lean: *AlgebraicGeometry.quasiAffineImmersionAffineSpace_chart_surjective*

The ring map $\Phi_j \colon \Gamma(T_j, \mathcal{O}_{T_j}) \to C_j$ induced by c_j is surjective.

Proof. Uses: lemma:nagata-quasi-affine-relative-standard-chart-affine,lemma:nagata-quasi-affine-local-generators-generate,lemma:nagata-quasi-affine-section-lifts

Under the ring description in Lemma 2.9.26, the map Φ_j restricts on coefficients to the structural map $R_j \to C_j$, sends $x_{\mathrm{inl}(j)}$ to $\rho_j(f_j)$, and sends $x_{\mathrm{inr}(j,k)}$ to $\rho_j(a_{jk})$. Thus

$$\Phi_j\left(x_{\mathrm{inr}(j,k)} x_{\mathrm{inl}(j)}^{-e_{jk}}\right) = g_{jk}$$

by Lemma 2.9.22. The image contains the image of R_j and every algebra generator from Lemma 2.9.18, so it is all of C_j. □

Lemma 2.9.28 (Closed immersion into each relative target chart). *Uses:* `def:nagata-quasi-affine-relative-standard-chart`

Lean: `AlgebraicGeometry.quasiAffineImmersionAffineSpace_chart_closedImmersion`

The morphism $c_j : U_j \to T_j$ is a closed immersion.

Proof. Uses: `lemma:nagata-quasi-affine-principal-cover-affine,lemma:nagata-quasi-affine-relative-standard-chart-affine,lemma:nagata-quasi-affine-chart-ring-surjective`

The source is affine by Lemma 2.9.12 and the target is affine by Lemma 2.9.26. The corresponding ring map is surjective by Lemma 2.9.27; the affine closed-immersion criterion applies. □

Lemma 2.9.29 (Immersion on each standard target open). *Uses:* `def:nagata-quasi-affine-immersion-affine-space,def:nagata-quasi-affine-relative-standard-chart`

Lean: `AlgebraicGeometry.quasiAffineImmersionAffineSpace_chart_immersion`

The restriction $\iota_X^{-1}(D_j) = U_j \to D_j$ is an immersion.

Proof. Uses: `lemma:nagata-quasi-affine-map-standard-open-preimage,lemma:nagata-quasi-affine-chart-closed-immersion`

The source equality is Lemma 2.9.24. The map is the composite of the closed immersion $U_j \to T_j$ from Lemma 2.9.28 and the open immersion $T_j \hookrightarrow D_j$. This composite is an immersion. □

Lemma 2.9.30 (Compatibility of the local chart restrictions). *Uses:* `def:nagata-quasi-affine-immersion-affine-space,def:nagata-quasi-affine-relative-standard-chart`

Lean: `AlgebraicGeometry.quasiAffineImmersionAffineSpace_chart_compat`

For the open immersion $u_j : U_j \hookrightarrow X$ and $q_j : T_j \hookrightarrow Y$, the square identity $\iota_X \circ u_j = q_j \circ c_j$ holds. Consequently the two restrictions to $U_j \cap U_{j'}$ agree as morphisms to Y.

Proof. Uses:

The morphism c_j was defined by factorizing the restriction of the single global section map ι_X through T_j, so the square identity is definitional. Restricting that identity for j and j' to the overlap gives two copies of the same global morphism. Equivalently, every affine-space coordinate pulls back to the same named global section. □

Lemma 2.9.31 (Property of the absolutely quasi-affine affine-space map, [1, Tag 04II]). *Uses:* `def:nagata-quasi-affine-immersion-affine-space`

Lean: `AlgebraicGeometry.existsImmersionAffineSpace_of_quasiAffine`

If X is absolutely quasi-affine and $\pi : X \to S$ is locally of finite type, the constructed map $\iota_X : X \to \mathbb{A}^N_S$ is an immersion.

Proof. Uses: `lemma:nagata-quasi-affine-principal-cover-covers,lemma:nagata-quasi-affine-map-standard-open-preimage,lemma:nagata-quasi-affine-chart-restriction-immersion,lemma:nagata-quasi-affine-local-maps-compatible`

Let $D = \bigcup_{j \in J} D_j$. The cover equality in Lemma 2.9.10 and the preimage formula in Lemma 2.9.24 show that $\iota_X^{-1}(D) = X$, so ι_X factors through the open $D \subseteq Y$. The D_j cover D, and over each D_j the map is an immersion by Lemma 2.9.29. These local maps are the restrictions of one global map by Lemma 2.9.30. Since immersion is local on the target, the map $X \to D$ is an immersion; composition with the open immersion $D \hookrightarrow Y$ proves the assertion. □

Corollary 2.9.32 (Finite-dimensional affine-space immersion for a quasi-affine source). *Uses:* `def:nagata-quasi-affine-immersion-affine-space`

Lean: `AlgebraicGeometry.existsFiniteImmersion_affineSpace_of_quasiAffine`

Constructed by: `def:nagata-quasi-affine-immersion-affine-space`

If X is absolutely quasi-affine and $\pi : X \to S$ is locally of finite type, there is a finite natural number N and an immersion $X \to \mathbb{A}^N_S$.

Proof. Uses: `def:nagata-quasi-affine-immersion-affine-space,lemma:nagata-quasi-affine-immersion-affine-space`

Take the finite coordinate type and the morphism in Definition 2.9.23. This constructed morphism is an immersion by Lemma 2.9.31. □

Lemma 2.9.33 (Relative quasi-affineness gives absolute quasi-affineness over an affine base). *Uses:*

Lean: `AlgebraicGeometry.IsQuasiAffine.quasiAffine_source_of_affine_target`

Let $\pi : X \to S$ be a quasi-affine morphism. If S is affine, then X is a quasi-affine scheme.

Proof. Uses:
By the definition of a quasi-affine morphism, the inverse image of every affine open of S is a quasi-affine scheme. Apply this to the affine open $S \subseteq S$ itself; its inverse image is X. □

Lemma 2.9.34 (An affine source is absolutely quasi-affine)**.** *Uses:*

Lean: *AlgebraicGeometry.IsAffine.isQuasiAffine*

Every affine scheme is a quasi-affine scheme.

Proof. Uses:
An affine scheme is an open subscheme of itself by the identity open immersion, which is the defining absolute quasi-affineness certificate. □

Definition 2.9.35 (Affine compactification construction)**.** Uses: `def:nagata-quasi-affine-immersion-affine-space`, `def:nagata-immersion-compactification`

Lean: `AlgebraicGeometry.Compactification.ofAffineConstruction`

Constructed by: `lemma:nagata-affine-source-is-quasi-affine`

Let $U \to S$ be separated and of finite type, with U affine. Compose the constructed map $\iota_U \colon U \to \mathbb{A}^n_S$ with the standard open immersion $\mathbb{A}^n_S \hookrightarrow \mathbb{P}^n_S$, and define

$$\mathsf{C}_{\mathrm{aff}}(U/S) = \mathsf{C}\big(U \longrightarrow \mathbb{P}^n_S\big)$$

using Definition 2.1.12.

Proposition 2.9.36 (Property of the affine compactification construction)**.** *Uses:* *def:nagata-affine-compactification*

Lean: *AlgebraicGeometry.CompactifiableOver.of_isAffine*

The constructed factorization $\mathsf{C}_{\mathrm{aff}}(U/S)$ is a compactification of $U \to S$.

Proof. Uses: `lemma:nagata-affine-source-is-quasi-affine`, `lemma:nagata-quasi-affine-immersion-affine-space`, `lemma:nagata-immersion-into-proper`
The first map to affine space is an immersion by Lemma 2.9.31, and its composite with the standard open immersion into projective space is again an immersion. Relative projective space is proper over S. The assertion is therefore Lemma 2.1.13 applied to this explicit composite. □

Definition 2.9.37 (Disjoint affine component-piece witness)**.** Uses:

Lean: `Scheme.affineComponentPieceWitness`

Let $X = \bigcup_{i=1}^{r} Z_i$ be its finite irreducible-component decomposition and fix $x \in X$. Inside the open obtained by deleting the components not containing x, choose an affine neighbourhood W of x. For every remaining component generic point not already assigned to W, choose an affine neighbourhood V_i inside the complement of all other components. Generalization and component exhaustivity give the required memberships and pairwise disjointness. Retain these fields in $\mathsf{B}_{\mathrm{piece}}(X, x)$.

Lemma 2.9.38 (Existence of disjoint affine component pieces)**.** *Uses:* *def:nagata-affine-neighbourhood-pieces-witness*

Lean: *Scheme.exists_disjoint_affine_componentPieces*

Constructed by: *def:nagata-affine-neighbourhood-pieces-witness*

Let $X = Z_1 \cup \cdots \cup Z_r$ be the finite exhaustive decomposition into irreducible components, with generic point η_i of Z_i, and let $x \in X$. Put $I_x = \{i \mid x \in Z_i\}$. The candidate type of affine opens W and V_i for $i \notin I_x$ satisfying

$$x \in W \subseteq X \setminus \bigcup_{i \notin I_x} Z_i, \qquad \eta_i \in V_i \subseteq X \setminus \bigcup_{k \neq i} Z_k.$$

together with pairwise disjointness of W and the V_i and membership of every η_i with $i \in I_x$ in W, is nonempty.

Proof. Uses: `def:nagata-affine-neighbourhood-pieces-witness`
The record $\mathsf{B}_{\mathrm{piece}}(X, x)$ from Definition 2.9.37 is an inhabitant of the displayed candidate type. □

Definition 2.9.39 (Selected disjoint affine component pieces)**.** Uses: `def:nagata-affine-neighbourhood-pieces-witness`

Lean: `Scheme.affineComponentPieceData`

Constructed by: `def:nagata-affine-neighbourhood-pieces-witness`

Define $\mathsf{P}(X, x; (Z_i, \eta_i)_i)$ to be $\mathsf{B}_{\mathrm{piece}}(X, x)$. Its record contains the affine certificates, all displayed containments, and pairwise disjointness.

Lemma 2.9.40 (Specifications of the selected component pieces). *Uses:* def:nagata-affine-neighbourhood-piece-data

Lean: Scheme.affineComponentPieceData_spec

The selected W is affine, contains x and every η_i with $i \in I_x$; each selected V_i is affine and contains η_i; and all selected pieces are pairwise disjoint open subschemes of X.

Proof. Uses: def:nagata-affine-neighbourhood-pieces-witness

These are precisely the fields of $\mathsf{B}_{\mathrm{piece}}(X, x)$ constructed in Definition 2.9.37 and retained by the selected record. □

Lemma 2.9.41 (Affine field of the selected component-piece record). *Uses:* def:nagata-affine-neighbourhood-piece-data

Lean: Scheme.affineComponentPieceData_isAffine

Every component piece in the selected record is affine.

Proof. Uses: lemma:nagata-affine-neighbourhood-piece-data-spec

Project the affine field of Lemma 2.9.40. □

Lemma 2.9.42 (Disjointness field of the selected component-piece record). *Uses:* def:nagata-affine-neighbourhood-piece-data

Lean: Scheme.affineComponentPieceData_pairwiseDisjoint

Distinct component pieces in the selected record are disjoint.

Proof. Uses: lemma:nagata-affine-neighbourhood-piece-data-spec

Project the pairwise-disjointness field of Lemma 2.9.40. □

Lemma 2.9.43 (Membership field of the selected component-piece record). *Uses:* def:nagata-affine-neighbourhood-piece-data

Lean: Scheme.affineComponentPieceData_mem

The distinguished point and each component generic point belong to their assigned selected piece.

Proof. Uses: lemma:nagata-affine-neighbourhood-piece-data-spec

Project the membership field of Lemma 2.9.40. □

Definition 2.9.44 (Affine neighbourhood containing all component generics). Uses: def:nagata-affine-neighbourhood-piece-data

Lean: Scheme.affineOpen_mem_and_genericPoints

Define the open subscheme

$$A(x; \eta_1, \dots, \eta_r) = W \amalg \coprod_{i \notin I_x} V_i \subseteq X$$

as the union of the selected pairwise disjoint component pieces.

Lemma 2.9.45 (Openness of the assembled component neighbourhood). *Uses:* def:nagata-affine-neighbourhood-generic-points

Lean: Scheme.affineOpen_mem_and_genericPoints_isOpen

The subset $A(x; \eta_1, \dots, \eta_r)$ is open in X.

Proof. Uses: lemma:nagata-affine-neighbourhood-piece-data-spec

It is a finite union of the open pieces supplied by Lemma 2.9.40. □

Lemma 2.9.46 (Affineness of the assembled component neighbourhood). *Uses:* def:nagata-affine-neighbourhood-generic-points

Lean: Scheme.affineOpen_mem_and_genericPoints_isAffine

The open subscheme $A(x; \eta_1, \dots, \eta_r)$ is affine.

Proof. Uses: lemma:nagata-affine-neighbourhood-piece-data-spec

The pieces are affine and pairwise disjoint by Lemma 2.9.40. Their union is finite, and a finite disjoint union $\coprod_{q \in Q} \operatorname{Spec}(B_q)$ is $\operatorname{Spec}(\prod_{q \in Q} B_q)$. This identifies the displayed open subscheme with an affine scheme. □

Lemma 2.9.47 (Membership in the assembled component neighbourhood). *Uses:* `def:nagata-affine-neighbourhood-generic-points`

Lean: `Scheme.affineOpen_mem_and_genericPoints_mem`

The open $A(x;\eta_1,\dots,\eta_r)$ contains x and every η_i.

Proof. Uses: `lemma:nagata-affine-neighbourhood-piece-data-spec`

The point x and the generic points with $i \in I_x$ lie in W; for $i \notin I_x$, the point η_i lies in V_i. These are the membership fields in Lemma 2.9.40. □

Lemma 2.9.48 (Property of the selected affine neighbourhood, [1, Tag 01ZX]). *Uses:* `def:nagata-affine-neighbourhood-generic-points`

Lean: `Scheme.exists_affineOpen_mem_and_genericPoints`

The constructed open $A(x;\eta_1,\dots,\eta_r)$ is affine and contains x and every η_i.

Proof. Uses: `lemma:nagata-affine-neighbourhood-generic-points-affine,lemma:nagata-affine-neighbourhood-generic-points-membership`

Affineness is Lemma 2.9.46, and all asserted memberships are Lemma 2.9.47. □

Definition 2.9.49 (Finite affine dense-cover witness). Uses: `def:nagata-affine-neighbourhood-generic-points`

Lean: `Scheme.finiteAffineOpenCoverDenseIInterWitness`

For nonempty quasi-compact X, take a finite subcover from the affine neighbourhoods $A(x;\eta_1,\dots,\eta_r)$; every member contains every component generic point. Reindex the nonempty subcover by $\{1,\dots,n\}$ with $n>0$. For empty X, take $n=1$ and the affine open $\varnothing$. Retain the cover, affine, generic-membership, and positivity fields in $\mathsf{B}_{\mathrm{dense}}(X)$.

Lemma 2.9.50 (Existence of a finite affine cover with dense total intersection). *Uses:* `def:nagata-dense-affine-cover-witness`

Lean: `Scheme.exists_finite_affineOpen_cover_genericPoints`

Constructed by: `def:nagata-dense-affine-cover-witness`

Let X be quasi-compact and suppose its irreducible components are exactly the finite family $(Z_i,\eta_i)_{i\in I}$. The candidate type of an integer $n>0$ and affine opens $U_1,\dots,U_n$ which cover X and contain every η_i in every U_j is nonempty.

Proof. Uses: `def:nagata-dense-affine-cover-witness`

The record $\mathsf{B}_{\mathrm{dense}}(X)$ defined in Definition 2.9.49 inhabits the displayed candidate type in both the empty and nonempty cases. □

Definition 2.9.51 (Finite affine cover with selected dense intersection). Uses: `def:nagata-affine-neighbourhood-generic-points,def:nagata-dense-affine-cover-witness`

Lean: `Scheme.finiteAffineOpenCoverDenseIInter`

Constructed by: `def:nagata-dense-affine-cover-witness`

Take the tuple $(n,(U_j)_{1\le j\le n})$ from $\mathsf{B}_{\mathrm{dense}}(X)$ and define

$$\mathcal{U}_X=(U_1,\dots,U_n),\qquad D_X=\bigcap_{j=1}^{n}U_j.$$

The positivity field $n>0$ is retained in the selected record.

Lemma 2.9.52 (Finiteness and positivity of the selected dense cover). *Uses:* `def:nagata-dense-affine-cover`

Lean: `Scheme.finiteAffineOpenCoverDenseIInter_finite`

The selected cover is indexed by the finite type $\{1,\dots,n\}$ and $n>0$.

Proof. Uses: `def:nagata-dense-affine-cover-witness`

These are the index and positivity fields of $\mathsf{B}_{\mathrm{dense}}(X)$ from Definition 2.9.49. □

Lemma 2.9.53 (Affineness of every selected dense-cover member). *Uses:* `def:nagata-dense-affine-cover`

Lean: `Scheme.finiteAffineOpenCoverDenseIInter_isAffine`

Every U_j in $\mathcal{U}_X$ is affine.

Proof. Uses: def:nagata-dense-affine-cover-witness
This is the affine field of $\mathsf{B}_{\mathrm{dense}}(X)$ from Definition 2.9.49. □

Lemma 2.9.54 (Cover property of the selected dense cover). *Uses:* def:nagata-dense-affine-cover

Lean: Scheme.finiteAffineOpenCoverDenseIInter_iSup_eq_top
One has $X = \bigcup_{j=1}^{n} U_j$.

Proof. Uses: def:nagata-dense-affine-cover-witness
This is the cover field of $\mathsf{B}_{\mathrm{dense}}(X)$ from Definition 2.9.49. □

Lemma 2.9.55 (Every cover member contains every component generic point). *Uses:* def:nagata-dense-affine-cover

Lean: Scheme.finiteAffineOpenCoverDenseIInter_generic_mem
For every component index i *and cover index* j, *one has* $\eta_i \in U_j$.

Proof. Uses: def:nagata-dense-affine-cover-witness
This is the common-generic-point field of $\mathsf{B}_{\mathrm{dense}}(X)$ from Definition 2.9.49. □

Lemma 2.9.56 (Density of the selected total intersection). *Uses:* def:nagata-dense-affine-cover

Lean: Scheme.finiteAffineOpenCoverDenseIInter_dense
The subset $D_X = \bigcap_{j=1}^{n} U_j$ *is dense in* X.

Proof. Uses: lemma:nagata-dense-affine-cover-contains-generics
By Lemma 2.9.55, D_X contains every η_i. Hence its closure contains $\overline{\{\eta_i\}} = Z_i$ for every i. The Z_i exhaust X, so the closure of D_X is all of X. This also covers the empty case. □

Lemma 2.9.57 (Properties of the selected dense affine cover). *Uses:* def:nagata-dense-affine-cover

Lean: Scheme.exists_finite_affineOpen_cover_dense_iInter
The U_i *form a finite nonempty affine open cover of* X, *and* D_X *is dense in* X.

Proof. Uses: lemma:nagata-dense-affine-cover-finite, lemma:nagata-dense-affine-cover-affine, lemma:nagata-dense-affine-cover-covers, lemma:nagata-dense-affine-cover-intersection-dense
Finiteness and positivity are Lemma 2.9.52; affineness is Lemma 2.9.53; coverage is Lemma 2.9.54; and density is Lemma 2.9.56. □

Definition 2.9.58 (Partial unions of the selected affine cover). Uses: def:nagata-dense-affine-cover

Lean: Scheme.partialAffineOpenUnions
For $1 \le r \le n$, define

$$X_r = U_1 \cup \cdots \cup U_r.$$

Lemma 2.9.59 (Density needed for induction on the affine cover). *Uses:* def:nagata-partial-affine-unions

Lean: TopologicalSpace.dense_inter_of_dense_total_inter
For $r > 1$, *the overlap* $X_{r-1} \cap U_r$ *is dense in* X_r.

Proof. Uses: lemma:nagata-dense-affine-cover
The overlap contains D_X. Since X_r is open in X and D_X is dense in X, it is dense in X_r; therefore the larger subset $X_{r-1} \cap U_r$ is dense in X_r. □

Definition 2.9.60 (Recursive Noetherian compactification construction). Uses: def:nagata-affine-compactification, def:nagata-partial-affine-unions, def:nagata-two-open-compactification-candidate

Lean: AlgebraicGeometry.noetherianCompactificationConstruction
Let S be Noetherian and let $X \to S$ be separated and of finite type. Use the selected cover $\mathcal{U}_X = (U_1, \dots, U_n)$. Define compactification candidates C_r recursively by

$$\mathsf{C}_1 = \mathsf{C}_{\mathrm{aff}}(U_1/S),$$

and, for $r > 1$, let C_r be the two-open candidate for

$$X_r = X_{r-1} \cup U_r$$

formed from C_{r-1} and $\mathsf{C}_{\mathrm{aff}}(U_r/S)$. Define

$$\mathsf{C}_{\mathrm{Noeth}}(X/S) = \mathsf{C}_n.$$

Theorem 2.9.61 (Nagata compactification over a Noetherian base). *Uses:* `def:nagata-compactification`, `def:nagata-noetherian-compactification`

Lean: `AlgebraicGeometry.nagataCompactification_of_isNoetherian`

The recursively constructed factorization $\mathsf{C}_{\mathrm{Noeth}}(X/S)$ is a compactification of $X \to S$. Consequently $X \to S$ is compactifiable.

Proof. Uses: `prop:nagata-affine-compactification`, `lemma:nagata-dense-affine-cover`, `lemma:nagata-partial-union-overlap-dense`, `thm:nagata-two-open`

Since X is finite type over the Noetherian scheme S, it is Noetherian; thus the selected finite affine cover and its dense total intersection are available. We prove by induction that the explicitly defined C_r is a compactification of $X_r \to S$. The initial case is Proposition 2.9.36. At stage $r > 1$, the induction hypothesis and the affine construction compactify the two opens X_{r-1} and U_r, while Lemma 2.9.59 supplies the density hypothesis. The property theorem for the two-open construction makes C_r a compactification. At $r = n$ one has $X_n = X$, giving the assertion. □

2.10 Finite-presentation reduction and approximation

2.10.1 Finite data and absolute Noetherian models

Definition 2.10.1 (Selected upper bound of a finite set). Uses:

Lean: `Order.Directed.finsetUpperBound`

Let I be a nonempty directed preorder. Fix a base point $i_\varnothing$ and, for each pair (i, j), fix a selected common upper bound $d(i, j)$. Define $u(F) \in I$ recursively on a finite set by

$$u(\varnothing) = i_\varnothing, \qquad u(F \cup \{i\}) = d(u(F), i).$$

Lemma 2.10.2 (Property of the selected finite upper bound). *Uses:* `def:nagata-directed-finite-upper-bound`

Lean: `Order.Directed.exists_ge_finset`

Every $i \in F$ satisfies $i \le u(F)$.

Proof. Uses:

Induct on F. The empty case is vacuous. At an insertion step, the selected $d(u(F), i)$ bounds both $u(F)$ and i; transitivity and the induction hypothesis prove the assertion for all old and new elements. □

Lemma 2.10.3 (A ring is the filtered colimit of its finite-type $\mathbf{Z}$-subalgebras). *Uses:*

Lean: `Algebra.filteredColimit_fgSubalgebra_int`

For every ring A, its finitely generated $\mathbf{Z}$-subalgebras form a directed system and the canonical map

$$\varinjlim_{A_0 \subseteq A,\ A_0/\mathbf{Z} \text{ finite type}} A_0 \longrightarrow A$$

is an isomorphism. In particular every finite set of elements of A is contained in one such subalgebra.

Proof. Uses:

The subalgebra generated by the union of two finite generating sets contains both corresponding subalgebras. Every element of A lies in the subalgebra generated by that element, proving surjectivity. Since all transition maps are inclusions, two representatives with equal image agree in a common over-algebra, proving injectivity. A finite set is contained in the subalgebra it generates. □

Definition 2.10.4 (Finite affine-cover witness for a quasi-compact scheme). Uses:

Lean: `Scheme.finiteAffineOpenCoverWitnessOfCompactSpace`

Cover a quasi-compact scheme S by affine basis opens, take a finite subcover, and reindex it by a finite type A. Retain the affine and cover certificates in $\mathsf{B}_{\mathrm{aff}}(S)$.

Lemma 2.10.5 (Existence of a finite affine cover of a quasi-compact scheme)**.** *Uses:* `def:nagata-qc-finite-affine-cover-witness`

Lean: `Scheme.exists_finite_affineOpen_cover_of_compactSpace`

Constructed by: `def:nagata-qc-finite-affine-cover-witness`

If S is quasi-compact, the candidate type of a finite type A and affine opens $(U_a)_{a\in A}$ with $S=\bigcup_{a\in A}U_a$ is nonempty.

Proof. Uses: `def:nagata-qc-finite-affine-cover-witness`

The record $\mathsf{B}_{\mathrm{aff}}(S)$ from Definition 2.10.4 is an inhabitant of the displayed candidate type. □

Definition 2.10.6 (Selected finite affine cover for the gluing presentation)**.** Uses: `def:nagata-qc-finite-affine-cover-witness`

Lean: `Scheme.FiniteAffineGlueData.affineCover`

Constructed by: `def:nagata-qc-finite-affine-cover-witness`

For a quasi-compact scheme S, take the finite type A and affine opens $(U_a)_{a\in A}$ from $\mathsf{B}_{\mathrm{aff}}(S)$. Write $A_a=\Gamma(U_a,\mathcal{O}_{U_a})$ and retain affineness and the cover equality in the selected record.

Lemma 2.10.7 (Specifications of the selected finite affine cover)**.** *Uses:* `def:nagata-finite-affine-cover-data`

Lean: `Scheme.FiniteAffineGlueData.affineCover_spec`

The index type A is finite, every U_a is affine, and $S=\bigcup_{a\in A}U_a$.

Proof. Uses: `def:nagata-qc-finite-affine-cover-witness`

These are exactly the fields of $\mathsf{B}_{\mathrm{aff}}(S)$ constructed in Definition 2.10.4 and retained in the selected record. □

Lemma 2.10.8 (Finite index field of the selected affine cover)**.** *Uses:* `def:nagata-finite-affine-cover-data`

Lean: `Scheme.FiniteAffineGlueData.affineCover_finite`

The selected affine-cover index type is finite.

Proof. Uses: `lemma:nagata-finite-affine-cover-data-spec`

Project the finite-index field of Lemma 2.10.7. □

Lemma 2.10.9 (Affine field of the selected affine cover)**.** *Uses:* `def:nagata-finite-affine-cover-data`

Lean: `Scheme.FiniteAffineGlueData.affineCover_isAffine`

Each selected chart is affine.

Proof. Uses: `lemma:nagata-finite-affine-cover-data-spec`

Project the affine field of Lemma 2.10.7. □

Lemma 2.10.10 (Cover field of the selected affine cover)**.** *Uses:* `def:nagata-finite-affine-cover-data`

Lean: `Scheme.FiniteAffineGlueData.affineCover_covers`

The selected affine charts cover the source scheme.

Proof. Uses: `lemma:nagata-finite-affine-cover-data-spec`

Project the cover field of Lemma 2.10.7. □

Lemma 2.10.11 (Quasi-compactness of pairwise affine-chart intersections)**.** *Uses:* `def:nagata-finite-affine-cover-data`

Lean: `Scheme.FiniteAffineGlueData.affineCover_inter_isCompact`

If S is quasi-separated, then $U_a\cap U_b$ is quasi-compact for every $a,b\in A$.

Proof. Uses:

Quasi-separatedness says that the diagonal of S is quasi-compact. The intersection $U_a\cap U_b$ is the inverse image of $U_a\times U_b$ under that diagonal. Since both affine charts are quasi-compact, the inverse image is quasi-compact. □

Definition 2.10.12 (Normalized finite affine overlap-cover witness)**.** Uses: `def:nagata-finite-affine-cover-data`

Lean: `Scheme.FiniteAffineGlueData.overlapAffineCoverWitness`

For each unordered pair of selected affine charts, quasi-separatedness makes their intersection quasi-compact. Choose a finite affine subcover, use the same record after reversing the pair, and use the singleton chart cover on the diagonal. Denote the normalized dependent record by $\mathsf{B}_{\mathrm{overlap}}(S)$.

Lemma 2.10.13 (Existence of finite affine overlap covers)**.** *Uses:* *def:nagata-finite-affine-overlap-cover-witness*

Lean: *Scheme.FiniteAffineGlueData.existsOverlapAffineCover*

Constructed by: *def:nagata-finite-affine-overlap-cover-witness*

If S is quasi-separated, then for every pair (a, b) the candidate type of a finite type R_{ab} and affine opens $(W_{ab,r})_{r \in R_{ab}}$ covering $O_{ab} := U_a \cap U_b$ is nonempty. Its members may be normalized so that R_{aa} is a singleton with $W_{aa,*} = U_a$, and so that $R_{ba} = R_{ab}$ and $W_{ba,r} = W_{ab,r}$ under the symmetry $O_{ba} = O_{ab}$.

Proof. Uses: `def:nagata-finite-affine-overlap-cover-witness`

The record $\mathsf{B}_{\mathrm{overlap}}(S)$ from Definition 2.10.12 is an inhabitant of every displayed candidate type and carries the stated normalizations. □

Definition 2.10.14 (Selected finite affine overlap covers)**.** Uses: `def:nagata-finite-affine-cover-data,def:nagata-finite-affine-overlap-cover-witness`

Lean: `Scheme.FiniteAffineGlueData.overlapAffineCover`

Constructed by: `def:nagata-finite-affine-overlap-cover-witness`

Take $(R_{ab}, (W_{ab,r})_r)$ from $\mathsf{B}_{\mathrm{overlap}}(S)$, including its transported reverse-order data and prescribed singleton on the diagonal. Put $B_{ab,r} = \Gamma(W_{ab,r}, \mathcal{O}_{W_{ab,r}})$ and retain both open immersions $W_{ab,r} \hookrightarrow U_a, U_b$.

Lemma 2.10.15 (Specifications of the selected overlap covers)**.** *Uses:* *def:nagata-finite-affine-overlap-cover-data*

Lean: *Scheme.FiniteAffineGlueData.overlapAffineCover_spec*

Every R_{ab} is finite, every $W_{ab,r}$ is affine and contained in O_{ab}, and $O_{ab} = \bigcup_r W_{ab,r}$. On the diagonal the selected cover is the singleton U_a, and reversing (a, b) leaves the selected cells and their inclusions into S unchanged.

Proof. Uses: `def:nagata-finite-affine-overlap-cover-witness`

These are the finite, affine, containment, and cover fields of $\mathsf{B}_{\mathrm{overlap}}(S)$ from Definition 2.10.12. □

Lemma 2.10.16 (Finite index field of the selected overlap covers)**.** *Uses:* *def:nagata-finite-affine-overlap-cover-data*

Lean: *Scheme.FiniteAffineGlueData.overlapAffineCover_finite*

Each selected overlap-cover index type is finite.

Proof. Uses: `lemma:nagata-finite-affine-overlap-cover-data-spec`

Project the finite-index field of Lemma 2.10.15. □

Lemma 2.10.17 (Affine field of the selected overlap covers)**.** *Uses:* *def:nagata-finite-affine-overlap-cover-data*

Lean: *Scheme.FiniteAffineGlueData.overlapAffineCover_isAffine*

Each selected overlap cell is affine.

Proof. Uses: `lemma:nagata-finite-affine-overlap-cover-data-spec`

Project the affine field of Lemma 2.10.15. □

Lemma 2.10.18 (Cover field of the selected overlap covers)**.** *Uses:* *def:nagata-finite-affine-overlap-cover-data*

Lean: *Scheme.FiniteAffineGlueData.overlapAffineCover_covers*

The selected cells cover each pairwise chart overlap.

Proof. Uses: `lemma:nagata-finite-affine-overlap-cover-data-spec`

Project the cover field of Lemma 2.10.15. □

Lemma 2.10.19 (Normalization field of the selected overlap covers). *Uses:* def:nagata-finite-affine-overlap-cover-data

Lean: Scheme.FiniteAffineGlueData.overlapAffineCover_normalized

Diagonal overlap data are literal singleton chart data, and reversal preserves the selected cell record.

Proof. Uses: lemma:nagata-finite-affine-overlap-cover-data-spec

Project the normalization field of Lemma 2.10.15. □

Definition 2.10.20 (Finite two-sided basic-presentation witness). Uses: def:nagata-finite-affine-cover-data, def:nagata-finite-affine-overlap-cover-data

Lean: Scheme.FiniteAffineGlueData.twoSidedBasicRefinementWitness

Each selected affine overlap cell is a quasi-compact open in either incident affine chart. A quasi-compact open immersion is of finite presentation [1, Tag 01TU]. Since source and target are affine, each induced chart-ring map $A_a \to B_{ab,r}$ and $A_b \to B_{ab,r}$ is therefore a finitely presented algebra map. Choose finite polynomial generators and a finite relation list presenting each map, and use the resulting quotient-ring isomorphisms. A quasi-compact open of an affine scheme also has a finite principal-open cover, so choose such covers on both sides. Choose once for each unordered pair, exchange the presentations under reversal, and choose the same presentation twice on a diagonal cell. Retain these fields in $\mathsf{B}_{\mathrm{basic}}(S)$.

Lemma 2.10.21 (Existence of finite two-sided basic presentation data). *Uses:* def:nagata-finite-affine-basic-refinement-witness

Lean: Scheme.FiniteAffineGlueData.exists_twoSidedBasicRefinement

Constructed by: def:nagata-finite-affine-basic-refinement-witness

For every selected affine overlap cell $W_{ab,r} = \mathrm{Spec}(B_{ab,r})$, the candidate type of finite principal-open covers in U_a and U_b, finite generator and relation types, and quotient-presentation isomorphisms

$$\theta^a_{ab,r}\colon A_a[(X_\ell)_{\ell\in L^a_{ab,r}}]/(q^a_m)_{m\in M^a_{ab,r}} \xrightarrow{\sim} B_{ab,r}, \qquad \theta^b_{ab,r}\colon A_b[(Y_\ell)_{\ell\in L^b_{ab,r}}]/(q^b_m)_{m\in M^b_{ab,r}} \xrightarrow{\sim} B_{ab,r}.$$

is nonempty. Its members may be normalized so that the two presentations on a diagonal cell are equal and reversal of (a,b) exchanges the two presentations.

Proof. Uses: def:nagata-finite-affine-basic-refinement-witness

The record $\mathsf{B}_{\mathrm{basic}}(S)$ from Definition 2.10.20 supplies an inhabitant with the displayed quotient isomorphisms and normalizations. □

Definition 2.10.22 (Selected two-sided basic presentation data). Uses: def:nagata-finite-affine-cover-data, def:nagata-finite-affine-overlap-cover-data, def:nagata-finite-affine-basic-refinement-witness

Lean: Scheme.FiniteAffineGlueData.twoSidedBasicRefinement

Constructed by: def:nagata-finite-affine-basic-refinement-witness

Take from $\mathsf{B}_{\mathrm{basic}}(S)$ the two finite principal-open covers, the finite types L^a, L^b, M^a, M^b, the relation polynomials, and the displayed quotient-presentation isomorphisms. Denote their quotient rings by $P^a_{ab,r}$ and $P^b_{ab,r}$. The selected record retains the cover, presentation, diagonal, and reversal certificates separately.

Lemma 2.10.23 (Specifications of the selected two-sided presentation data). *Uses:* def:nagata-finite-affine-basic-refinement-data

Lean: Scheme.FiniteAffineGlueData.twoSidedBasicRefinement_spec

Every selected principal-open, generator, and relation index type is finite; the principal opens cover the named cell on each side; and $\theta^a_{ab,r}$ and $\theta^b_{ab,r}$ are isomorphisms onto the same ring $B_{ab,r}$. The diagonal and reversal normalizations hold.

Proof. Uses: def:nagata-finite-affine-basic-refinement-witness

These are the individual fields of $\mathsf{B}_{\mathrm{basic}}(S)$ from Definition 2.10.20 and retained by the selected record. □

Lemma 2.10.24 (Finite index field of the two-sided presentations). *Uses:* def:nagata-finite-affine-basic-refinement-data

Lean: Scheme.FiniteAffineGlueData.twoSidedBasicRefinement_finite

Each generator, relation, and principal-open index type in the selected two-sided presentations is finite.

Proof. Uses: lemma:nagata-finite-affine-basic-refinement-data-spec

Project the finite-index field of Lemma 2.10.23. □

Lemma 2.10.25 (Cover field of the two-sided presentations). *Uses:* def:nagata-finite-affine-basic-refinement-data

Lean: Scheme.FiniteAffineGlueData.twoSidedBasicRefinement_covers

The selected principal opens cover the named affine cell from each incident chart side.

Proof. Uses: lemma:nagata-finite-affine-basic-refinement-data-spec

Project the cover field of Lemma 2.10.23. □

Lemma 2.10.26 (Quotient-isomorphism field of the two-sided presentations). *Uses:* def:nagata-finite-affine-basic-refinement-data

Lean: Scheme.FiniteAffineGlueData.twoSidedBasicRefinement_isomorphisms

Each selected quotient model is isomorphic to the coordinate ring of its literal affine cell.

Proof. Uses: lemma:nagata-finite-affine-basic-refinement-data-spec

Project the quotient-isomorphism field of Lemma 2.10.23. □

Lemma 2.10.27 (Normalization field of the two-sided presentations). *Uses:* def:nagata-finite-affine-basic-refinement-data

Lean: Scheme.FiniteAffineGlueData.twoSidedBasicRefinement_normalized

Diagonal presentations are literally equal, and reversal exchanges the two selected presentation sides.

Proof. Uses: lemma:nagata-finite-affine-basic-refinement-data-spec

Project the normalization field of Lemma 2.10.23. □

Definition 2.10.28 (Finite affine triple-refinement witness). Uses: def:nagata-finite-affine-overlap-cover-data, def:nagata-finite-affine-basic-refinement-data

Lean: Scheme.FiniteAffineGlueData.tripleRefinementWitness

Each finite intersection of three selected affine pair cells is a quasi-compact open of the quasi-separated scheme S. Choose a finite affine subcover. For every chosen member Q put $C_Q = \Gamma(Q, \mathcal{O}_Q)$. For each $d \in \{a, b, c\}$ choose a finite quotient presentation P_Q^d of C_Q as an A_d-algebra together with an A_d-algebra isomorphism

$$\theta_Q^d \colon P_Q^d \xrightarrow{\sim} C_Q.$$

Indeed, Q is a quasi-compact affine open in every incident affine chart and pair cell. Tag 01TU makes each such open immersion finitely presented; because both schemes are affine, every induced ring map to C_Q is a finitely presented algebra map and hence admits the asserted finite generator-and-relation quotient presentation. Also choose the corresponding finite presentations over the three incident pair-cell rings. Retain comparison isomorphisms making the restriction squares commute: restricting either side of the selected pair-cell presentation to Q agrees with the appropriate (P_Q^d, θ_Q^d). Thus all three chart models have the same named target ring C_Q; no two quotient rings belonging to different pair cells are identified without this common refinement. Store the cover, containment, presentation, and restriction-comparison fields in $\mathsf{B}_{\mathrm{triple}}(S)$.

Lemma 2.10.29 (Existence of finite affine triple-overlap refinements). *Uses:* def:nagata-finite-affine-triple-refinement-witness

Lean: Scheme.FiniteAffineGlueData.existsTripleRefinement

Constructed by: def:nagata-finite-affine-triple-refinement-witness

For every a, b, c and cells $W_{ab,r}, W_{bc,s}, W_{ac,t}$, the candidate type of a finite affine cover of the common open

$$Z_{abc,rst} = W_{ab,r} \cap W_{bc,s} \cap W_{ac,t}$$

together with the three chart-relative presentations $\theta_Q^d : P_Q^d \simeq C_Q$, the incident pair-cell presentations, and their commuting restriction comparisons is nonempty.

Proof. Uses: def:nagata-finite-affine-triple-refinement-witness

The record $\mathsf{B}_{\mathrm{triple}}(S)$ from Definition 2.10.28 is an inhabitant with the displayed affine cover and presentation fields. □

Definition 2.10.30 (Selected finite affine triple-overlap refinements). Uses: def:nagata-finite-affine-overlap-cover-data, def:nagata-finite-affine-triple-refinement-witness

Lean: Scheme.FiniteAffineGlueData.tripleRefinement

Constructed by: def:nagata-finite-affine-triple-refinement-witness

Take from $\mathsf{B}_{\mathrm{triple}}(S)$, for every finite tuple (a, b, c, r, s, t), the finite affine cover $(Q_{abc,rst,u})_u$ of $Z_{abc,rst}$, together with finite quotient presentations of C_Q over all incident cell and chart rings. In particular retain the three models P_Q^a, P_Q^b, P_Q^c, their isomorphisms $\theta_Q^a, \theta_Q^b, \theta_Q^c$ to the single ring C_Q, and the comparison squares identifying them with the restrictions of the pair-cell models. Retain all containment and cover equalities in the record.

Lemma 2.10.31 (Specifications of the selected triple-overlap refinements). *Uses:* *def:nagata-finite-affine-triple-refinement-data*

Lean: *Scheme.FiniteAffineGlueData.tripleRefinement_spec*

Every triple-refinement index, generator type, and relation type is finite; the $Q_{abc,rst,u}$ cover $Z_{abc,rst}$; and for every Q the three isomorphisms $\theta_Q^d : P_Q^d \simeq C_Q$ and the incident pair-cell comparison squares identify restrictions with the same literal affine open subscheme $Q \subseteq S$.

Proof. Uses: `def:nagata-finite-affine-triple-refinement-witness`

Project the finite, affine, cover, containment, and presentation fields of $\mathsf{B}_{\mathrm{triple}}(S)$ from Definition 2.10.28. □

Lemma 2.10.32 (Finite index field of the triple refinements). *Uses:* *def:nagata-finite-affine-triple-refinement-data*

Lean: *Scheme.FiniteAffineGlueData.tripleRefinement_finite*

Each triple-refinement cover, generator, and relation index type is finite.

Proof. Uses: `lemma:nagata-finite-affine-triple-refinement-data-spec`

Project the finite-index field of Lemma 2.10.31. □

Lemma 2.10.33 (Cover field of the triple refinements). *Uses:* *def:nagata-finite-affine-triple-refinement-data*

Lean: *Scheme.FiniteAffineGlueData.tripleRefinement_covers*

The selected affine triple-refinement cells cover each named triple intersection.

Proof. Uses: `lemma:nagata-finite-affine-triple-refinement-data-spec,lemma:nagata-finite-affine-overlap-cover-data-covers`

For fixed pair-cell indices (r, s, t), the cover field in Lemma 2.10.31 says that the selected $Q_{abc,rst,u}$ cover $Z_{abc,rst}$. The three instances of Lemma 2.10.18 say that the selected pair cells cover O_{ab}, O_{bc}, and O_{ac}; hence their intersections $Z_{abc,rst}$ cover $U_a \cap U_b \cap U_c$. Combining these two finite covers proves the asserted triple-intersection cover. □

Lemma 2.10.34 (Presentation field of the triple refinements). *Uses:* *def:nagata-finite-affine-triple-refinement-data*

Lean: *Scheme.FiniteAffineGlueData.tripleRefinement_presentations*

Each selected triple-refinement cell Q has the recorded finite quotient presentation over every incident ring. In particular $\theta_Q^d : P_Q^d \simeq C_Q$ is recorded for $d = a, b, c$, and the comparison squares identify the restriction of each pair-cell presentation with these three common-refinement models.

Proof. Uses: `lemma:nagata-finite-affine-triple-refinement-data-spec`

Project the presentation field of Lemma 2.10.31. □

Lemma 2.10.35 (Restriction-compatibility field of the triple refinements). *Uses:* *def:nagata-finite-affine-triple-refinement-data*

Lean: *Scheme.FiniteAffineGlueData.tripleRefinement_restrictions*

For every ordered incident pair (d, e) and every selected triple cell Q, the covariant change of presentation obtained by restricting the pair-cell models to Q is

$$\varphi_Q^{de} := (\theta_Q^e)^{-1} \circ \theta_Q^d \colon P_Q^d \xrightarrow{\sim} P_Q^e.$$

In particular both its source and target are models of the same ring C_Q.

Proof. Uses: `lemma:nagata-finite-affine-triple-refinement-data-spec`

The commuting restriction-comparison squares in Lemma 2.10.31 identify the two restricted pair-cell presentations with θ_Q^d and θ_Q^e. Their change-of-presentation map is therefore the displayed composite through the common ring C_Q. □

Definition 2.10.36 (Canonical global transition on a pairwise overlap). Uses: `def:nagata-finite-affine-cover-data`

Lean: `Scheme.FiniteAffineGlueData.overlapTransition`

Regard $O_{ab} = U_a \cap U_b$ once as an open subscheme of U_a and once as an open subscheme of U_b. Both copies are the pullback of the same two open immersions into S. Define

$$\tau_{ab} \colon O_{ab}^{(a)} \xrightarrow{\sim} O_{ba}^{(b)}$$

to be the canonical isomorphism induced by the equality of these literal open subfunctors of S.

Lemma 2.10.37 (Cell formula for the global overlap transition). *Uses:* def:nagata-finite-affine-overlap-transition, def:nagata-finite-affine-overlap-cover-data, def:nagata-finite-affine-basic-refinement-data

Lean: Scheme.FiniteAffineGlueData.overlapTransition_restrict

On the selected cell $W_{ab,r}$, put

$$\varphi_r^{ab} = (\theta_{ab,r}^b)^{-1} \circ \theta_{ab,r}^a \colon P_{ab,r}^a \xrightarrow{\sim} P_{ab,r}^b.$$

With the source modeled by $\operatorname{Spec}(P_{ab,r}^a)$ and the target by $\operatorname{Spec}(P_{ab,r}^b)$, the restricted scheme transition is

$$\tau_{ab}|_{W_{ab,r}} = \operatorname{Spec}\big((\varphi_r^{ab})^{-1}\big).$$

Proof. Uses: lemma:nagata-finite-affine-overlap-cover-data-spec, lemma:nagata-finite-affine-basic-refinement-data-spec

Both θ maps identify their quotient model with the coordinate ring of the same literal cell $W_{ab,r} \subseteq S$. Restricting τ_{ab} therefore changes covariantly from the a-presentation to that common ring and then to the b-presentation, which is φ_r^{ab}. Affine spectrum reverses arrows, so the scheme map from the a-model to the b-model is induced by $(\varphi_r^{ab})^{-1}$. The selected cells cover O_{ab} by the overlap-cover specification. □

Definition 2.10.38 (Raw finite affine gluing presentation). Uses: def:nagata-finite-affine-cover-data, def:nagata-finite-affine-overlap-cover-data, def:nagata-finite-affine-basic-refinement-data, def:nagata-finite-affine-triple-refinement-data, def:nagata-finite-affine-overlap-transition

Lean: Scheme.FiniteAffineGlueData.Raw

Define $\mathsf{G}_S^{\mathrm{raw}}$ to contain the finite chart type and chart rings, the symmetric finite affine-cell presentations of every overlap, the global transition τ_{ab}, and the finite triple refinements. On a pair cell its covariant ring comparison isomorphism is

$$\varphi_r^{ab} = (\theta_{ab,r}^b)^{-1} \circ \theta_{ab,r}^a \colon P_{ab,r}^a \xrightarrow{\sim} P_{ab,r}^b.$$

This is well typed because each θ is a quotient-presentation *isomorphism*, not merely the polynomial-ring quotient map. The scheme transition from the a-model to the b-model is $\operatorname{Spec}((\varphi_r^{ab})^{-1})$, the restriction of τ_{ab} by the preceding cell formula. On a selected triple cell Q, record instead the well-typed common-refinement comparisons

$$\varphi_Q^{de} = (\theta_Q^e)^{-1}\theta_Q^d \colon P_Q^d \xrightarrow{\sim} P_Q^e \qquad (d, e \in \{a, b, c\}),$$

and retain slots for the restriction-comparison certificates asserting that these maps represent the corresponding pair transitions on Q. Record the finite relative-generator lists on which the identity, two inverse, and triple-cocycle equations will be tested. This Definition supplies only the raw candidate data; the next lemma certifies its transition fields.

Lemma 2.10.39 (Validity of the raw transition fields). *Uses:* def:nagata-finite-affine-gluing-raw-presentation

Lean: Scheme.FiniteAffineGlueData.Raw.transition_fields

The pair-cell transition stored in $\mathsf{G}_S^{\mathrm{raw}}$ is $\operatorname{Spec}((\varphi_r^{ab})^{-1})$, and on every triple cell Q its restriction is represented by the typed common-refinement map $\varphi_Q^{ab} : P_Q^a \simeq P_Q^b$.

Proof. Uses: lemma:nagata-finite-affine-overlap-transition-cell, lemma:nagata-finite-affine-triple-refinement-data-restrictions

The pair-cell formula is Lemma 2.10.37. The commuting restriction squares and the typed formula through C_Q are exactly Lemma 2.10.35. □

Lemma 2.10.40 (Finiteness of the raw gluing record). *Uses:* def:nagata-finite-affine-gluing-raw-presentation

Lean: Scheme.FiniteAffineGlueData.Raw.finite

Every chart, cell, refinement, generator, relation, and equation index type occurring in $\mathsf{G}_S^{\mathrm{raw}}$ is finite.

Proof. Uses: lemma:nagata-finite-affine-cover-data-spec, lemma:nagata-finite-affine-overlap-cover-data-spec, lemma:nagata-finite-affine-basic-refinement-data-spec, lemma:nagata-finite-affine-triple-refinement-data-spec

The four cited specification lemmas give finiteness of the chart, cell, two-sided presentation, and triple-refinement indices. Finite sums and products preserve finiteness. Every equality to be certified compares algebra maps relative to the same chart or cell coefficient map: their restrictions to the coefficient ring already agree because both encode the same literal-open inclusion. Equality is therefore tested only on the finite list of selected polynomial generators of the presented quotient. No assertion is made that an arbitrary chart ring A_a, cell ring $B_{ab,r}$, or coefficient ring is a finite set or a finitely generated ring; only the chosen combinatorial and relative presentation data are finite. □

Lemma 2.10.41 (Identity equations for the raw gluing maps)**.** *Uses:* def:nagata-finite-affine-gluing-raw-presentation

Lean: Scheme.FiniteAffineGlueData.Raw.identity

The transition on $O_{aa} = U_a$ is the identity, both as an open-subscheme isomorphism and on every selected quotient-presentation generator.

Proof. Uses: lemma:nagata-finite-affine-overlap-cover-data-spec,lemma:nagata-finite-affine-basic-refinement-data-spec

The diagonal overlap is the singleton cell U_a by Lemma 2.10.15, and the two presentations were chosen equal by Lemma 2.10.23. Hence $(\theta^a)^{-1}\theta^a$ is the identity and fixes every generator. □

Lemma 2.10.42 (Left inverse equations for the raw gluing maps)**.** *Uses:* def:nagata-finite-affine-gluing-raw-presentation

Lean: Scheme.FiniteAffineGlueData.Raw.inverse_left

Globally on O_{ab} one has $\tau_{ba} \circ \tau_{ab} = \mathrm{id}$; on every selected pair cell this is represented by $\varphi_r^{ba} \circ \varphi_r^{ab} = \mathrm{id}$.

Proof. Uses: lemma:nagata-finite-affine-overlap-cover-data-spec,lemma:nagata-finite-affine-basic-refinement-data-spec,lemma:nagata-finite-affine-overlap-transition-cell

Reversal exchanges θ^a and θ^b by Lemma 2.10.23. Substitution gives $(\theta^a)^{-1}\theta^b(\theta^b)^{-1}\theta^a = \mathrm{id}$ on each cell, i.e. $\varphi^{ba}\varphi^{ab} = \mathrm{id}_{P^a}$. Because $\tau_{ab} = \mathrm{Spec}(\varphi^{ba})$ and $\tau_{ba} = \mathrm{Spec}(\varphi^{ab})$, contravariance identifies the restricted scheme composite with $\mathrm{Spec}(\varphi^{ba}\varphi^{ab})$. The cells cover O_{ab}, so open-cover extensionality gives the global equation. □

Lemma 2.10.43 (Right inverse equations for the raw gluing maps)**.** *Uses:* def:nagata-finite-affine-gluing-raw-presentation

Lean: Scheme.FiniteAffineGlueData.Raw.inverse_right

Globally on O_{ba} one has $\tau_{ab} \circ \tau_{ba} = \mathrm{id}$; on every selected pair cell this is represented by $\varphi_r^{ab} \circ \varphi_r^{ba} = \mathrm{id}$.

Proof. Uses: lemma:nagata-finite-affine-overlap-cover-data-spec,lemma:nagata-finite-affine-basic-refinement-data-spec,lemma:nagata-finite-affine-overlap-transition-cell

The reverse composite is $(\theta^b)^{-1}\theta^a(\theta^a)^{-1}\theta^b = \mathrm{id}$. Equality on the finite selected generators records the required cell equation list. Thus $\varphi^{ab}\varphi^{ba} = \mathrm{id}_{P^b}$, whose affine spectrum is the restricted composite $\tau_{ab}\tau_{ba}$. The cell formula and cover equality promote it to the displayed global equation. □

Lemma 2.10.44 (Triple-cocycle equations for the raw gluing maps)**.** *Uses:* def:nagata-finite-affine-gluing-raw-presentation

Lean: Scheme.FiniteAffineGlueData.Raw.triple_cocycle

Globally on $U_a \cap U_b \cap U_c$, the restricted transitions satisfy $\tau_{bc} \circ \tau_{ab} = \tau_{ac}$. On every selected affine triple-refinement cell this is represented by

$$\varphi_Q^{bc} \circ \varphi_Q^{ab} = \varphi_Q^{ac}, \qquad P_Q^a \longrightarrow P_Q^b \longrightarrow P_Q^c.$$

Proof. Uses: lemma:nagata-finite-affine-triple-refinement-data-spec,lemma:nagata-finite-affine-triple-refinement-data-restrictions,lemma:nagata-finite-affine-overlap-cover-data-covers,lemma:nagata-finite-affine-overlap-transition-cell

Fix a selected triple cell Q and put $C_Q = \Gamma(Q, \mathcal{O}_Q)$. By the triple-refinement specification, the three maps $\theta_Q^d : P_Q^d \xrightarrow{\sim} C_Q$ have this same named codomain. Hence the displayed composite is well typed and cancellation in C_Q gives

$$\varphi_Q^{bc} \circ \varphi_Q^{ab} = (\theta_Q^c)^{-1}\theta_Q^b(\theta_Q^b)^{-1}\theta_Q^a = (\theta_Q^c)^{-1}\theta_Q^a = \varphi_Q^{ac}.$$

By Lemmas 2.10.35 and 2.10.37, these are precisely the restrictions to Q of the three pair-transition presentations; no quotient rings attached to distinct pair cells are directly composed.

The composite scheme transition is induced by the reverse ring map

$$(\varphi_Q^{ab})^{-1} \circ (\varphi_Q^{bc})^{-1} = (\varphi_Q^{bc} \circ \varphi_Q^{ab})^{-1} = (\varphi_Q^{ac})^{-1},$$

hence is τ_{ac}. All three ring maps are algebra maps over the same coefficient morphism attached to the literal inclusion $Q \hookrightarrow S$, so their coefficient restrictions already agree. The selected finite relative presentation of $\Gamma(Q, \mathcal{O})$ therefore makes equality on its finite polynomial-generator list a finite certificate. The selected pair cells cover O_{ab}, O_{bc}, and O_{ac} by Lemma 2.10.18, so their common refinements $Z_{abc,rst}$ cover $U_a \cap U_b \cap U_c$; the selected Q cover every such $Z_{abc,rst}$. Thus the cell equations imply the asserted global cocycle by open-cover extensionality. □

Definition 2.10.45 (Selected finite affine gluing presentation)**.** Uses: `def:nagata-finite-affine-gluing-raw-presentation`

Constructed by: `lemma:nagata-finite-affine-gluing-raw-presentation-valid,lemma:nagata-finite-affine-gluing-raw-presentation-finite,lemma:nagata-finite-affine-gluing-identity,lemma:nagata-finite-affine-gluing-inverse-left,lemma:nagata-finite-affine-gluing-inverse-right,lemma:nagata-finite-affine-gluing-triple-cocycle`

Lean: `Scheme.finiteAffineGlueData`

Define the certified datum G_S to be $\mathsf{G}_S^{\mathrm{raw}}$ together with its pair/triple restriction, raw-finiteness, identity, left-inverse, right-inverse, and triple-cocycle proof fields supplied by the six preceding atomic lemmas. Its transition maps are the canonical maps between the two chart copies of the same literal overlap in S. Thus $\mathrm{Glue}(\mathsf{G}_S)$ is typed using an actual certified gluing datum, not a candidate with empty slots.

Lemma 2.10.46 (Validity of the selected finite affine gluing presentation)**.** *Uses:* *def:nagata-finite-affine-gluing-presentation*

Lean: *Scheme.FiniteAffineGlueData.valid*

The certified datum G_S is a finite valid affine gluing presentation: its transition fields have the stated pair/triple restrictions and satisfy identity, both inverse equations, and the triple cocycle.

Proof. Uses: `lemma:nagata-finite-affine-gluing-raw-presentation-valid,lemma:nagata-finite-affine-gluing-raw-presentation-finite,lemma:nagata-finite-affine-gluing-identity,lemma:nagata-finite-affine-gluing-inverse-left,lemma:nagata-finite-affine-gluing-inverse-right,lemma:nagata-finite-affine-gluing-triple-cocycle`

The raw transition and finiteness fields are the first two cited lemmas. The remaining four lemmas give, respectively, the identity, left-inverse, right-inverse, and triple-cocycle fields. These are exactly the fields retained by the certified Definition. □

Definition 2.10.47 (Canonical morphism from the glued presentation)**.** Uses: `def:nagata-finite-affine-gluing-presentation`

Lean: `Scheme.FiniteAffineGlueData.recoveryMap`

Since G_S is certified, let $G_S = \mathrm{Glue}(\mathsf{G}_S)$, let $\iota_a \colon U_a \to G_S$ be its chart morphisms, and let $j_a \colon U_a \hookrightarrow S$ be the original open immersions. Since the j_a agree through the selected transition maps, the gluing universal property defines

$$\varepsilon_S \colon G_S \longrightarrow S.$$

Lemma 2.10.48 (Chart equation for the canonical recovery morphism)**.** *Uses:* *def:nagata-finite-affine-gluing-recovery-map*

Lean: *Scheme.FiniteAffineGlueData.recoveryMap_comp_chart*

For every chart index a, $\varepsilon_S \circ \iota_a = j_a$.

Proof. Uses: `lemma:nagata-finite-affine-overlap-cover-data-spec,lemma:nagata-finite-affine-gluing-triple-cocycle`

The selected transition maps are induced by equality of the two inclusions of the literal overlap into S. The finite affine cells cover every overlap, so the j_a form a cocone. The displayed equation is the chart component equation in the universal property defining ε_S. □

Definition 2.10.49 (Canonical morphism back to the glued presentation)**.** Uses: `def:nagata-finite-affine-gluing-presentation`

Lean: `Scheme.FiniteAffineGlueData.corecoveryMap`

Since G_S is certified, on each member U_a of the selected open cover of S, use the gluing chart map $\iota_a \colon U_a \to G_S$. The inverse and cocycle fields of G_S make these maps agree on every pairwise overlap. Open-cover descent for morphisms therefore defines

$$\eta_S \colon S \longrightarrow G_S.$$

Lemma 2.10.50 (Chart equation for the canonical corecovery morphism)**.** *Uses:* *def:nagata-finite-affine-gluing-corecovery-map*

Lean: *Scheme.FiniteAffineGlueData.corecoveryMap_comp_chart*

For every chart index a, $\eta_S \circ j_a = \iota_a$.

Proof. Uses: `lemma:nagata-finite-affine-gluing-inverse-left,lemma:nagata-finite-affine-gluing-inverse-right,lemma:nagata-finite-affine-gluing-triple-cocycle`

This is the local component equation of the morphism obtained by gluing the ι_a. The cited inverse and cocycle equations are precisely their agreement certificates on double and triple overlaps. □

Lemma 2.10.51 (Left inverse equation for the recovery maps). *Uses:* *def: nagata-finite-affine-gluing-recovery-map*, *def: nagata-finite-affine-gluing-corecovery-map*

Lean: *Scheme.FiniteAffineGlueData.recovery_hom_inv_id*

One has $\varepsilon_S \circ \eta_S = \mathrm{id}_S$.

Proof. Uses: lemma:nagata-finite-affine-cover-data-spec, lemma:nagata-finite-affine-gluing-recovery-map-chart, lemma:nagata-finite-affine-gluing-corecovery-map-chart

After precomposition with every covering open immersion j_a, the left side becomes $\varepsilon_S \circ \iota_a = j_a$, by the two chart equations. The j_a cover S by Lemma 2.10.7, so morphism extensionality on an open cover gives the asserted equality. □

Lemma 2.10.52 (Right inverse equation for the recovery maps). *Uses:* *def: nagata-finite-affine-gluing-recovery-map*, *def: nagata-finite-affine-gluing-corecovery-map*

Lean: *Scheme.FiniteAffineGlueData.recovery_inv_hom_id*

One has $\eta_S \circ \varepsilon_S = \mathrm{id}_{G_S}$.

Proof. Uses: lemma:nagata-finite-affine-gluing-recovery-map-chart, lemma:nagata-finite-affine-gluing-corecovery-map-chart

Precomposition with each gluing chart ι_a turns the left side into $\eta_S \circ j_a = \iota_a$. The images of the ι_a cover G_S by the gluing construction, so open-cover extensionality proves the equality. □

Definition 2.10.53 (Canonical recovery isomorphism). Uses: def:nagata-finite-affine-gluing-recovery-map, def:nagata-finite-affine-gluing-corecovery-map

Constructed by: lemma:nagata-finite-affine-gluing-recovery-left-inverse, lemma:nagata-finite-affine-gluing-recovery-right-inverse

Lean: Scheme.FiniteAffineGlueData.recoveryIso

Define the isomorphism

$$\mathrm{rec}_S \colon G_S \xrightarrow{\sim} S$$

with forward map ε_S, inverse map η_S, and inverse fields supplied by the two preceding atomic lemmas.

Lemma 2.10.54 (Validity of the canonical recovery isomorphism). *Uses:* *def: nagata-finite-affine-gluing-recovery-isomorphism*

Lean: *Scheme.FiniteAffineGlueData.recoveryIso_valid*

The defined isomorphism $\mathrm{rec}_S : G_S \xrightarrow{\sim} S$ *has forward map* ε_S *and inverse map* η_S.

Proof. Uses: lemma:nagata-finite-affine-gluing-recovery-left-inverse, lemma:nagata-finite-affine-gluing-recovery-right-inverse

This is the projection of the forward, inverse, and two inverse fields retained in the preceding Definition. □

Lemma 2.10.55 (Recovery of the scheme from the certified presentation). *Uses:* *def: nagata-finite-affine-gluing-recovery-isomorphism*

Lean: *Scheme.FiniteAffineGlueData.glue_iso_source*

The isomorphism $\mathrm{rec}_S \colon G_S \simeq S$ *satisfies* $\mathrm{rec}_S.hom \circ \iota_a = j_a$ *on every selected chart.*

Proof. Uses: lemma:nagata-finite-affine-gluing-recovery-isomorphism-valid, lemma:nagata-finite-affine-gluing-recovery-map-chart

The validity lemma supplies both inverse fields of rec_S, and Lemma 2.10.48 is exactly the displayed component equation for its forward map. □

Lemma 2.10.56 (Properties of the selected finite affine presentation). *Uses:* *def: nagata-finite-affine-gluing-presentation*

Lean: *Scheme.exists_finiteAffineGlueData_of_qcqs*

Every overlap in G_S *is quasi-compact and represented by finitely many affine cells with finite two-sided ring presentations; all gluing identities have finite generator-indexed certificates; and gluing* G_S *recovers* S.

Proof. Uses: lemma:nagata-finite-affine-gluing-presentation-valid, lemma:nagata-finite-affine-cover-overlap-qc, lemma:nagata-finite-affine-overlap-cover-data-spec, lemma:nagata-finite-affine-basic-refinement-data-spec, lemma:nagata-finite-affine-triple-refinement-data-spec, lemma:nagata-finite-affine-gluing-raw-presentation-finite, lemma:nagata-finite-affine-gluing-recovery

Pairwise overlaps are quasi-compact by Lemma 2.10.11; their finite cell covers and two-sided presentations are the next two cited specification lemmas; triple refinements are Lemma 2.10.31; finiteness of every certificate list is Lemma 2.10.40; and the recovery isomorphism is Lemma 2.10.55. □

Definition 2.10.57 (One-affine extension coefficient datum)**.** Uses: `def:nagata-finite-affine-gluing-presentation`

Lean: `Scheme.AffineGlueData.coefficientStage`

Let T be qcqs and let $T = V \cup U$ with $V \subseteq T$ quasi-compact open and $U = \mathrm{Spec}(B)$ affine. Thus $W = U \cap V$ is quasi-compact. Suppose a selected system $V = \varprojlim_i V_i$ has affine transitions and finite-type $\mathbf{Z}$-stages. On a cofinal tail select quasi-compact opens $W_i \subseteq V_i$ which recover W, are preserved by transition inverse image, and are quasi-affine. Set

$$R = \Gamma(W, \mathcal{O}_W), \qquad R_i = \Gamma(W_i, \mathcal{O}_{W_i}), \qquad B_i = B \times_R R_i.$$

Here $B \to R$ is restriction and $R_i \to R$ is the limit map. Select a finite basic cover $W = \bigcup_{\ell=1}^{q} D(g_\ell)$ inside $U = \mathrm{Spec}(B)$ and, at one common stage, compatible lifts $g_{\ell,i} \in R_i$ of the restrictions of g_ℓ. Name

$$h_{\ell,i} = (g_\ell, g_{\ell,i}) \in B_i.$$

After restricting the index set once more, store $W_i = \bigcup_\ell D(h_{\ell,i})$ as an open of $\mathrm{Spec}(B_i)$. Compatibility of the $g_{\ell,i}$ gives the literal equality

$$W_{i'} = W_i \times_{\mathrm{Spec}(B_i)} \mathrm{Spec}(B_{i'})$$

for every $i' \geq i$.

Lemma 2.10.58 (The one-affine coefficient datum has the required colimits)**.** *Uses:* `def:nagata-affine-gluing-coefficient-stage`

Lean: `Scheme.AffineGlueData.exists_finiteTypeModel`

On a selected cofinal tail,

$$R \cong \varinjlim_i R_i, \qquad B \cong \varinjlim_i B_i,$$

and $W_i \to \mathrm{Spec}(B_i)$ is the open immersion represented by the compatible finite basic family $(h_{\ell,i})_\ell$. For every finite subset $F \subseteq B_i$ there is a finite-type subalgebra $A \subseteq B_i$ containing F and all $h_{\ell,i}$ such that

$$A_{h_{\ell,i}} \xrightarrow{\sim} (B_i)_{h_{\ell,i}} \quad (1 \leq \ell \leq q).$$

Proof. Uses: `lemma:nagata-ring-finite-type-subalgebras`

Tags 01Z4 and 01Z5 give the transition-stable W_i and their eventual quasi-affineness; Tag 01Z0 gives $R = \varinjlim_i R_i$. Filtered colimits commute with the displayed fibre products because the map $B \to R$ is fixed, so $B = \varinjlim_i (B \times_R R_i)$. For each ℓ one has

$$B_{g_\ell} \cong R_{s(g_\ell)}.$$

After increasing the common stage, the opens $D(g_{\ell,i})$ lie in and cover W_i. Localizing the cartesian square defining B_i gives $(B_i)_{(g_\ell, g_{\ell,i})} \cong (R_i)_{g_{\ell,i}}$, which is the open-immersion certificate. If $\beta_{ii'} \colon B_i \to B_{i'}$ is a transition, then

$$W_i \times_{\mathrm{Spec}(B_i)} \mathrm{Spec}(B_{i'}) = \bigcup_\ell D(\beta_{ii'}(h_{\ell,i})) = \bigcup_\ell D(h_{\ell,i'}) = W_{i'},$$

by base change of basic opens and compatibility of the selected lifts. For a prescribed finite $F \subseteq B_i$, choose finite $\mathbf{Z}$-algebra generators of every $(B_i)_{h_{\ell,i}}$ and clear their denominators to elements $y_{\ell r} \in B_i$; these localized rings are finite type because they are coordinate rings of the affine finite-type opens $D(h_{\ell,i}) \subseteq W_i \subseteq V_i$. The subalgebra generated by F, the $h_{\ell,i}$, and the $y_{\ell r}$ has the asserted localization isomorphisms. This is the explicit construction in [1, Tag 01Z7 and Tag 01Z9]. □

Definition 2.10.59 (Strict coefficient record for a one-affine stage)**.** Uses: `def:nagata-affine-gluing-coefficient-stage`

Lean: `Scheme.AffineGlueData.strictCoefficientRecord`

For i let P_i consist of records

$$p = (A_p \subseteq B_i,\ (h_{\ell,i} \in A_p)_\ell, (\rho_{p\ell})_\ell), \qquad \rho_{p\ell} \colon (A_p)_{h_{\ell,i}} \xrightarrow{\sim} (B_i)_{h_{\ell,i}},$$

where A_p is a finite-type $\mathbf{Z}$-algebra and every $\rho_{p\ell}$ is induced by the displayed inclusion. Thus the finite union

$$W_{i,p} = \bigcup_\ell D(h_{\ell,i}) \subseteq \mathrm{Spec}(A_p)$$

is identified with W_i, and the square with $\mathrm{Spec}(B_i) \to \mathrm{Spec}(A_p)$ is cartesian. This cartesian identification, including the named basic cover, is a field of the record; an algebra which only admits an open factorization is not in P_i.

Definition 2.10.60 (Index preorder for strict one-affine stages)**.** Uses: `def:nagata-strict-one-affine-stage-record`

Lean: `Scheme.AffineGlueData.strictCoefficientIndex`

Define

$$E(T/V) = \coprod_i P_i, \qquad \alpha\colon E(T/V) \longrightarrow I, \quad (i,p) \longmapsto i.$$

Put $(i,p) \le (i',p')$ when $i \le i'$, the map $B_i \to B_{i'}$ carries A_p into $A_{p'}$, and the stored square

$$\begin{array}{ccc} W_{i'} & \hookrightarrow & \operatorname{Spec}(A_{p'}) \\ \downarrow & & \downarrow \\ W_i & \hookrightarrow & \operatorname{Spec}(A_p) \end{array}$$

is cartesian. The projection α is monotone for this relation.

Lemma 2.10.61 (Finite incoming strict records have a common extension)**.** *Uses:* *def:nagata-strict-one-affine-stage-index*

Lean: *Scheme.AffineGlueData.strictCoefficientRecord_finite_extension*

Let K be finite, let $p_k \in P_{i_k}$, choose $i \ge i_k$ for all $k \in K$, and let $F \subseteq B_i$ be finite. There is $p \in P_i$ such that $F \subseteq A_p$, every $B_{i_k} \to B_i$ carries A_{p_k} into A_p, and, for every k, the square

$$\begin{array}{ccc} W_i & \hookrightarrow & \operatorname{Spec}(A_p) \\ \downarrow & & \downarrow \\ W_{i_k} & \hookrightarrow & \operatorname{Spec}(A_{p_k}) \end{array}$$

is cartesian.

Proof. Uses: `lemma:nagata-affine-gluing-coefficients-descend`

Put into one finite set F, the images of finite algebra generators of all A_{p_k}, and the compatible elements $h_{\ell,i}$. The strict transition equality for the B_i says that the inverse image of $\bigcup_\ell D(h_{\ell,i_k})$ in $\operatorname{Spec}(B_i)$ is precisely $W_i = \bigcup_\ell D(h_{\ell,i})$. Adjoin denominator-cleared generators of each $(B_i)_{h_{\ell,i}}$ as in the preceding proof and call the resulting finite-type subalgebra A_p. Then $(A_p)_{h_{\ell,i}} \cong (B_i)_{h_{\ell,i}}$ for every ℓ, so $p \in P_i$. Base changing the basic-open presentation of each W_{i_k} along $A_{p_k} \to A_p$ gives the same union $\bigcup_\ell D(h_{\ell,i}) = W_i$. This proves all the displayed cartesian squares simultaneously. □

Definition 2.10.62 (Cocycle-valid one-affine extension stage)**.** Uses: `def:nagata-strict-one-affine-stage-index`

Lean: `Scheme.AffineGlueData.validStage`

For $e = (i,p)$, with coefficient algebra A_p, the selected raw datum is

$$\mathsf{G}_e = \big(V_i, \operatorname{Spec}(A_p), W_i, \mathrm{id}_{W_i}\big),$$

where the two copies of W_i are the selected opens in V_i and in $\operatorname{Spec}(A_p)$. This tuple is the stage-and-datum field selected for N13.

Lemma 2.10.63 (Certificate for the one-affine gluing datum)**.** *Uses:* *def:nagata-affine-gluing-valid-stage*

Lean: *Scheme.AffineGlueData.eventually_cocycle*

Every G_e is certified scheme-gluing data.

Proof. Uses: `lemma:nagata-affine-gluing-coefficients-descend`

Both displayed maps from W_i are open immersions. The overlap isomorphism is the identity of W_i; its selected inverse is itself. The two identity equations, the inverse equations, and every triple-overlap cocycle therefore reduce to $\mathrm{id} \circ \mathrm{id} = \mathrm{id}$. These equations give the certificate field of the N13 datum, rather than merely asserting that a later stage exists. □

Definition 2.10.64 (Finite-type stage of a one-affine extension)**.** Uses: `def:nagata-affine-gluing-valid-stage`

Lean: `Scheme.AffineGlueData.noetherianStage`

For $e = (i,p) \in E(T/V)$, define

$$T_e = V_i \amalg_{W_i} \operatorname{Spec}(A_p)$$

by applying scheme gluing to the certified datum G_e. Store the two open immersions and their equality on W_i as part of this selected stage.

Definition 2.10.65 (Raw transition morphism between strict stages). Uses: def:nagata-noetherian-stage,def:nagata-strict-one-affine-stage-index

Lean: AlgebraicGeometry.noetherianStageTransition

For $(i,p) \le (i',p')$ in the selected relation, the stored strict-pullback certificate identifies $W_{i'} = W_i \times_{\mathrm{Spec}(A_p)} \mathrm{Spec}(A_{p'})$. Define the raw transition $T_{(i',p')} \to T_{(i,p)}$ by gluing $V_{i'} \to V_i$ and $\mathrm{Spec}(A_{p'}) \to \mathrm{Spec}(A_p)$; their restrictions to $W_{i'}$ agree by construction.

Lemma 2.10.66 (The strict-stage preorder is directed). Uses: def:nagata-noetherian-transition-morphism

Lean: AlgebraicGeometry.noetherianStageIndex_directed

The displayed relation on $E(T/V)$ is a nonempty directed preorder.

Proof. Uses: lemma:nagata-strict-one-affine-stage-extension,lemma:nagata-directed-finite-upper-bound

Reflexivity uses the identity cartesian square in each record. Transitivity follows by pasting the two stored cartesian squares. For finitely many records, choose a common upper i for their indices and apply Lemma 2.10.61; the resulting record supplies all required algebra inclusions and strict squares at once. □

Definition 2.10.67 (Selected Noetherian inverse system). Uses: def:nagata-noetherian-transition-morphism

Lean: AlgebraicGeometry.noetherianInverseSystem

For a finite affine cover $S = U_1 \cup \cdots \cup U_n$ from G_S, start with the system of spectra of finite-type $\mathbf{Z}$-subalgebras of $\Gamma(U_1, \mathcal{O}_S)$. Recursively apply the preceding one-affine extension to

$$U_1 \cup \cdots \cup U_{r-1} \subseteq U_1 \cup \cdots \cup U_r.$$

The final recursively selected diagram is

$$\mathsf{N}(S) = (I_S, (S_\lambda)_\lambda, (t_{\mu\lambda})_{\lambda \le \mu}).$$

Choose the affine cover nonempty; when $S = \varnothing$, use the one-member cover $U_1 = \varnothing = \mathrm{Spec}(0)$.

Definition 2.10.68 (Diagram certificate for the Noetherian inverse system). Uses: def:nagata-noetherian-inverse-system

Lean: AlgebraicGeometry.noetherianInverseSystemDiagramCertificate

For every $e \in I_S$, store the unique-gluing equality $t_{ee} = \mathrm{id}_{S_e}$. For every $e \le e' \le e''$, store

$$t_{e''e} = t_{e'e} \circ t_{e''e'}.$$

On the two selected opens these are respectively the identity and composition equations of the V-transition and affine ring maps; uniqueness in scheme gluing proves the displayed global equalities. The finite-cover recursion transports these equalities at each extension step. Their family is the selected N23 diagram certificate.

Lemma 2.10.69 (Properties of the finite-type stages). Uses: def:nagata-noetherian-inverse-system,def:nagata-noetherian-diagram-certificate

Lean: Scheme.AffineGlueData.glue_finiteType_over_int

The preorder I_S is directed, every S_λ is of finite type over $\mathbf{Z}$, and every transition morphism $t_{\mu\lambda}$ is affine.

Proof. Uses: lemma:nagata-affine-gluing-identities-eventually,lemma:nagata-noetherian-stage-index-directed,lemma:nagata-directed-finite-upper-bound

Directedness is Lemma 2.10.66, transported through the finite-cover recursion. Each one-affine stage is covered by the finite-type scheme V_i and the affine finite-type scheme $\mathrm{Spec}(A_p)$, hence is finite type over $\mathbf{Z}$. The inverse image of the target open V_i is $V_{i'}$ by the stored cartesian square, and the inverse image of $\mathrm{Spec}(A_p)$ is $\mathrm{Spec}(A_{p'})$. Both restricted transition maps are affine, so the glued transition is affine. Induction over the selected finite affine cover proves all three assertions for $\mathsf{N}(S)$. □

Lemma 2.10.70 (Cofinality of the strict-stage projection). Uses: def:nagata-strict-one-affine-stage-index

Lean: Scheme.AffineGlueData.strictCoefficientIndex_cofinal

The monotone projection $\alpha\colon E(T/V) \to I$ is cofinal.

Proof. Uses: `lemma:nagata-strict-one-affine-stage-extension,lemma:nagata-noetherian-stage-index-directed`

For $i_0 \in I$, choose $i \geq i_0$ and apply the extension lemma with $K = \varnothing$ and $F = \varnothing$ to obtain (i, p) above i_0. Given two objects above i_0, choose a common upper index and apply the extension lemma to their two records with $F = \varnothing$. The resulting record receives both objects over i_0. Thus every comma preorder $(i_0 \downarrow \alpha)$ is nonempty and directed, hence connected. □

Lemma 2.10.71 (Coefficient colimit over the strict-stage index). Uses: `def:nagata-strict-one-affine-stage-index`

Lean: `Scheme.AffineGlueData.strictCoefficientIndex_colimit`

The canonical inclusions induce an isomorphism

$$\varinjlim_{(i,p)\in E(T/V)} A_p \xrightarrow{\sim} \varinjlim_{i} B_i \xrightarrow{\sim} B.$$

Proof. Uses: `lemma:nagata-strict-one-affine-stage-index-cofinal,lemma:nagata-strict-one-affine-stage-extension,lemma:nagata-affine-gluing-coefficients-descend`

The second map is the selected coefficient-colimit isomorphism. For surjectivity of the first composite, represent an element in some B_i, apply the extension lemma with that singleton as F, and use the resulting A_p. If two representatives have the same image in B, move them to a common B_i where they are equal and apply the extension lemma to the two incoming records with both images in F. They then agree in one A_p, proving injectivity. The preceding cofinality lemma certifies that these representatives and equality witnesses are compatible with the selected projection to I. □

Definition 2.10.72 (Affine-transition certificate for the Noetherian system). Uses: `def:nagata-noetherian-inverse-system,def:nagata-noetherian-diagram-certificate`

Lean: `AlgebraicGeometry.noetherianInverseSystemAffineCertificate`

For every $e \leq e'$, store the two inverse-image identities

$$t_{e'e}^{-1}(V_e) = V_{e'}, \qquad t_{e'e}^{-1}(U_e) = U_{e'},$$

where V_e and $U_e = \operatorname{Spec}(A_e)$ are the two named opens at the current one-affine recursion step. Store the given affine certificate on $V_{e'} \to V_e$ and the affine-spectrum certificate on $U_{e'} \to U_e$. Affineness local on the target then certifies $t_{e'e}$ affine. The strict identities and certificates are transported through every later finite one-affine extension.

Definition 2.10.73 (Explicit inverse limit of the Noetherian system). Uses: `def:nagata-noetherian-affine-transition-certificate,def:nagata-noetherian-diagram-certificate`

Lean: `AlgebraicGeometry.noetherianInverseSystemLimit`

Select $e_0 \in I_S$ and work on the cofinal directed tail $I_{\geq e_0}$. For $e \geq e_0$ put $a_e = t_{ee_0}$. Since a_e is affine, set

$$\mathcal{A}_e = (a_e)_*\mathcal{O}_{S_e}, \qquad \mathcal{A}_\infty = \varinjlim_{e\geq e_0} \mathcal{A}_e, \qquad L_S = \operatorname*{Spec}_{S_{e_0}}(\mathcal{A}_\infty).$$

The affine-transition certificate supplies canonical isomorphisms $S_e \cong \operatorname{Spec}_{S_{e_0}}(\mathcal{A}_e)$, and the diagram certificate supplies the directed algebra maps $\mathcal{A}_e \to \mathcal{A}_{e'}$. Their filtered colimit is quasi-coherent. The maps $\mathcal{A}_e \to \mathcal{A}_\infty$ define compatible projections $\pi_e \colon L_S \to S_e$ on the tail, hence on all of I_S by cofinality. If $(q_e \colon Q \to S_e)_e$ is a compatible family, relative-Spec adjunction gives compatible $\mathcal{O}_{S_{e_0}}$-algebra maps

$$\mathcal{A}_e \longrightarrow (q_{e_0})_*\mathcal{O}_Q,$$

hence a unique map $\mathcal{A}_\infty \to (q_{e_0})_*\mathcal{O}_Q$ and a unique corresponding $Q \to L_S$. Its composites with the π_e are the q_e. This is the explicit affine-transition limit construction of [1, Tag 01YX].

Definition 2.10.74 (Raw family of stage comparisons). Uses: `def:nagata-noetherian-inverse-system`

Lean: `Scheme.AffineGlueData.limitComparisonFamily`

At a one-affine extension stage, the maps $V \to V_i$ and $U = \operatorname{Spec}(B) \to \operatorname{Spec}(A_p)$ agree on W and glue to $T \to T_{(i,p)}$. Iterating over the finite cover gives, for each $e \in I_S$, a selected stage map $\kappa_e \colon S \to S_e$. At this point this is only a raw family; no map to the inverse limit is yet defined.

Definition 2.10.75 (Comparison cocone certificate)**.** Uses: `def:nagata-glued-limit-comparison-family`,`def:nagata-noetherian-diagram-certificate`

Lean: `Scheme.AffineGlueData.limitComparisonCocone`

For every $e \le e'$, store the equality

$$t_{e'e} \circ \kappa_{e'} = \kappa_e \colon S \longrightarrow S_e.$$

On the selected V-open this is the projection-cocone equality of the original inverse system; on the affine open it is the inclusion $A_p \to A_{p'} \to B$ equalling $A_p \to B$. These two equalities agree on W and unique gluing proves the displayed global equality. This certificate, not merely the functor laws for the transitions, induces κ_S into the inverse limit.

Definition 2.10.76 (Comparison from the original scheme to the inverse limit)**.** Uses: `def:nagata-glued-limit-comparison-cocone`,`def:nagata-noetherian-inverse-limit`

Lean: `Scheme.AffineGlueData.limitComparison`

Apply the stored universal property of L_S to the certified cocone $(\kappa_e)_e$ and define the unique comparison

$$\kappa_S \colon S \longrightarrow L_S$$

whose composite with every limit projection is κ_e.

Definition 2.10.77 (Recovery map from the explicit glued limit)**.** Uses: `def:nagata-glued-limit-comparison`,`def:nagata-noetherian-affine-transition-certificate`,`def:nagata-noetherian-inverse-limit`

Constructed by: `lemma:nagata-strict-one-affine-stage-coefficient-colimit`

Lean: `Scheme.AffineGlueData.limitRecoveryMap`

For the initial affine open, recovery is the identity $\operatorname{Spec}(\varinjlim_A A) = \operatorname{Spec}(B)$. At a one-affine extension $T = V \cup U$, define

$$L_V = \pi_{e_0}^{-1}(V_{e_0}), \qquad L_U = \pi_{e_0}^{-1}(U_{e_0}), \qquad L_W = L_V \cap L_U = \pi_{e_0}^{-1}(W_{e_0}).$$

The strict transition certificate identifies these with the inverse images of V_e, U_e, W_e for every $e \ge e_0$, and $L_V \cup L_U = L_T$ because $V_{e_0} \cup U_{e_0} = T_{e_0}$. Restricting the relative-Spec limit of Definition 2.10.73 gives the three stored identifications

$$L_V \cong \varprojlim_{e \ge e_0} V_e \xrightarrow{\sim} V, \qquad L_U \cong \varprojlim_{e \ge e_0} U_e \cong \operatorname{Spec}\left(\varinjlim_{(i,p)} A_p\right) \xrightarrow{\sim} \operatorname{Spec}(B) = U, \qquad L_W \cong \varprojlim_{e \ge e_0} W_e \xrightarrow{\sim} W.$$

The first is the previously selected recovery map for V; the second uses the independently certified strict-stage coefficient-colimit isomorphism. The restriction statement is the open-limit calculation of [1, Tag 01YX(2)]. The first two identifications restrict on L_W to the same map because both have the selected projection cone to every W_e. Glue them and define $\delta_T \colon L_T \to T$. Finite induction over the affine cover defines the selected global recovery map $\delta_S \colon L_S \to S$.

Lemma 2.10.78 (Recovery of the glued limit)**.** *Uses:* `def:nagata-glued-limit-recovery-map`

Lean: `Scheme.AffineGlueData.limit_iso`

The maps κ_S and δ_S are inverse isomorphisms.

Proof. Uses: `lemma:nagata-affine-gluing-coefficients-descend`,`lemma:nagata-strict-one-affine-stage-coefficient-colimit`

On L_V and L_U, the composite $\kappa_S\delta_S$ is the identity by the two colimit identifications used to define δ_S; these opens cover L_S. On V and U, the composite $\delta_S\kappa_S$ is likewise the identity, and these opens cover S. Equality of scheme morphisms is local on the source, so both global composites are identities. □

Proposition 2.10.79 ([1, Tag 01ZA])**.** *Uses:* `def:nagata-glued-limit-comparison`,`def:nagata-noetherian-inverse-limit`,`def:nagata-noetherian-inverse-system`

Lean: `AlgebraicGeometry.exists_noetherian_inverseSystem`

The selected system $\mathsf{N}(S)$ has Noetherian finite-type $\mathbf{Z}$-stages and affine transitions, and κ_S identifies S with its inverse limit.

Proof. Uses: `lemma:nagata-noetherian-stage-properties`,`lemma:nagata-glued-limit-recovery`

The preceding construction is the finite induction of [1, Tag 07RN] starting from an affine open. Finite type over the Noetherian ring $\mathbf{Z}$ makes every stage Noetherian. The transition and recovery certificates are exactly Lemmas 2.10.69 and 2.10.78. □

2.10.2 Embedding a finite-type scheme into a finite-presentation model

Definition 2.10.80 (Selected affine data for the immersion model)**.** Uses:

Lean: `AlgebraicGeometry.finitePresentationImmersionAffineCoverData`

Let $f\colon X \to S$ be locally of finite type with X qcqs. Select a finite affine cover $X = \bigcup_a V_a$, affine opens $T_a \subseteq S$ with $f(V_a) \subseteq T_a$, and a finite family $h_{a1}, \dots, h_{am_a} \in \mathcal{O}_X(V_a)$ generating $\mathcal{O}_X(V_a)$ over $\mathcal{O}_S(T_a)$. The record stores the cover, subordination, and generator equations as separate fields.

Lemma 2.10.81 (Specification of the selected immersion affine data)**.** *Uses:* *def:nagata-immersion-affine-cover-data*

Lean: *AlgebraicGeometry.finitePresentationImmersionAffineCoverData_spec*

The selected V_a are affine and cover X, every restriction $V_a \to S$ factors through the named affine T_a, and the named finite family generates its coordinate algebra over $\mathcal{O}_S(T_a)$.

Proof. Uses:

Choose a finite affine cover of the quasi-compact scheme X. Local finite type gives finite algebra generators after shrinking each member so that its image lies in an affine open of S; quasi-compactness extracts a finite refinement. These choices are exactly the fields of the preceding record. □

Definition 2.10.82 (Affine-cover persistence stage for the immersion model)**.** Uses: `def:nagata-immersion-affine-cover-data`,`def:nagata-noetherian-inverse-system`,`def:nagata-directed-finite-upper-bound`

Lean: `AlgebraicGeometry.finitePresentationImmersionAffineCoverStage`

Apply $\mathsf{N}(-)$ to X. For every V_a, select its Tag 01Z4 quasi-compact-open descent stage, its Tag 01Z6 affineness stage, and the Tag 01Z4(3) equality stage at which the finite union of the descended opens is all of X_λ. Let λ_{cov} be their common upper bound and denote the transported opens by $V_{a,\lambda_{\mathrm{cov}}} \subseteq X_{\lambda_{\mathrm{cov}}}$.

Lemma 2.10.83 (Specification of the immersion affine-cover stage)**.** *Uses:* *def:nagata-immersion-affine-cover-stage*

Lean: *AlgebraicGeometry.finitePresentationImmersionAffineCoverStage_spec*

The $V_{a,\lambda_{\mathrm{cov}}}$ are affine, cover $X_{\lambda_{\mathrm{cov}}}$, and satisfy

$$V_a = X \times_{X_{\lambda_{\mathrm{cov}}}} V_{a,\lambda_{\mathrm{cov}}}.$$

These three properties persist after every later transition.

Proof. Uses: `lemma:nagata-directed-finite-upper-bound`

The selected open-descent, affineness, and cover-equality certificates hold at their common upper bound. Pullback preserves the open and cover equalities, while inverse images of affine opens under the affine transition maps are affine. □

Definition 2.10.84 (Generator-lift stage for the immersion model)**.** Uses: `def:nagata-immersion-affine-cover-stage`,`def:nagata-directed-finite-upper-bound`

Lean: `AlgebraicGeometry.finitePresentationImmersionGeneratorLiftStage`

The affine-limit equality identifies

$$\mathcal{O}_X(V_a) = \varinjlim_{\lambda \geq \lambda_{\mathrm{cov}}} \mathcal{O}_{X_\lambda}(V_{a,\lambda}).$$

Select a finite-stage representative of every h_{ab} and define λ_X as the common upper bound of λ_{cov} and all representative stages. Write the transported representatives as h_{ab,λ_X}.

Lemma 2.10.85 (Specification of the immersion generator lifts)**.** *Uses:* *def:nagata-immersion-generator-lift-stage*

Lean: *AlgebraicGeometry.finitePresentationImmersionGeneratorLiftStage_spec*

Every h_{ab,λ_X} restricts to h_{ab} on V_a, and the affine opens V_{a,λ_X} still cover X_{λ_X} and recover the V_a.

Proof. Uses: `lemma:nagata-immersion-affine-cover-stage-spec`

This is the representative specification at the selected common upper bound, together with persistence of the affine-cover certificate. □

Definition 2.10.86 (Finite-presentation immersion stage scheme). Uses: def:nagata-immersion-generator-lift-stage

Lean: AlgebraicGeometry.finitePresentationImmersionStageScheme

Define only the N14 stage-scheme field by

$$Y^{\mathrm{imm}}(X/S) = X_{\lambda_X} \times_{\operatorname{Spec}\mathbf{Z}} S.$$

Definition 2.10.87 (Structure morphism of the immersion stage). Uses: def:nagata-immersion-stage-scheme

Lean: AlgebraicGeometry.finitePresentationImmersionStageStructureMap

Define the N14 structure-map field to be the second projection

$$p_Y : Y^{\mathrm{imm}}(X/S) \longrightarrow S.$$

Definition 2.10.88 (Base-change comparison of the immersion stage). Uses: def:nagata-immersion-stage-structure-map

Lean: AlgebraicGeometry.finitePresentationImmersionStageBaseChange

Store the fibre-product symmetry isomorphism

$$\eta_Y : Y^{\mathrm{imm}}(X/S) \xrightarrow{\sim} S \times_{\operatorname{Spec}\mathbf{Z}} X_{\lambda_X}.$$

Define $i^{\mathrm{imm}} : X \to Y^{\mathrm{imm}}(X/S)$ by the pair consisting of the limit projection $X \to X_{\lambda_X}$ and $f : X \to S$. For every a define the affine target open

$$W_a = V_{a,\lambda_X} \times_{\operatorname{Spec}\mathbf{Z}} T_a \subseteq Y^{\mathrm{imm}}(X/S).$$

Store the typed inverse-image equality $(i^{\mathrm{imm}})^{-1}(W_a) = V_a$. On coordinate rings the restriction to $V_a \to W_a$ is the map induced by the transported h_{ab,λ_X} and the $\mathcal{O}_S(T_a)$-structure. The equations with the two fibre-product projections and these inverse-image identities are the selected N14 base-change data. Surjectivity and closedness are not fields of this Definition; they are certified by the separate chartwise lemma below.

Definition 2.10.89 (Finite-presentation immersion model). Uses: def:nagata-immersion-stage-scheme,def:nagata-immersion-stage-structure-map,def:nagata-immersion-stage-base-change

Lean: AlgebraicGeometry.finitePresentationImmersionModel

Define the selected N14 record to be $(\lambda_X, Y^{\mathrm{imm}}, p_Y, \eta_Y, i^{\mathrm{imm}})$ with the three fields and their equations constructed above.

Lemma 2.10.90 (Finite presentation of the immersion-stage target). Uses: def:nagata-immersion-stage-structure-map

Lean: AlgebraicGeometry.finitePresentationImmersionStage_finitePresentation

The morphism $p_Y : Y^{\mathrm{imm}}(X/S) \to S$ is of finite presentation.

Proof. Uses: prop:nagata-noetherian-inverse-system

The stage X_{λ_X} is finite type over the Noetherian ring $\mathbf{Z}$, hence finite presentation over $\mathbf{Z}$; the assertion is its base change to S. □

Lemma 2.10.91 (Chartwise closedness of the immersion-stage map). Uses: def:nagata-immersion-stage-base-change

Lean: AlgebraicGeometry.finitePresentationImmersionStage_chartwiseClosed

For every a, the inverse image of W_a is exactly V_a and the restriction $V_a \to W_a$ is a closed immersion.

Proof. Uses: lemma:nagata-immersion-generator-lift-stage-spec,lemma:nagata-immersion-affine-cover-data-spec

The first assertion is the stored inverse-image equation. On coordinate rings, the lifted h_{ab,λ_X} together with $\mathcal{O}_S(T_a)$ map to the selected generators of $\mathcal{O}_X(V_a)$, so the ring map is surjective. □

Lemma 2.10.92 (The chart targets cover a neighbourhood of the image). Uses: def:nagata-immersion-stage-base-change

Lean: AlgebraicGeometry.finitePresentationImmersionStage_imageCover

The open $W = \bigcup_a W_a$ contains $i^{\mathrm{imm}}(X)$.

Proof. Uses: lemma:nagata-immersion-generator-lift-stage-spec

The V_a cover X and equal the inverse images of the corresponding W_a, so their finite union has inverse image all of X. □

Lemma 2.10.93 (Properties of the finite-presentation immersion model, [1, Tag 01ZE]). *Uses:* `def:nagata-immersion-into-fp-model`

Lean: `AlgebraicGeometry.existsImmersionFinitePresentationModel`

The morphism $Y^{\mathrm{imm}}(X/S) \to S$ is of finite presentation, and the constructed map i^{imm} is an immersion over S.

Proof. Uses: `lemma:nagata-immersion-stage-finite-presentation`, `lemma:nagata-immersion-stage-chartwise-closed`, `lemma:nagata-immersion-stage-image-cover`

The first lemma gives finite presentation. The latter two exhibit an open neighbourhood of the image on which the map is target-locally a closed immersion, which is exactly an immersion over S. □

Definition 2.10.94 (Closed-neighbourhood predicate for an immersion). Uses:

Lean: `AlgebraicGeometry.Immersion.closedNeighbourhoodPredicate`

Let $i\colon X \to Y$ be an immersion with X quasi-compact and Y quasi-separated. For $x \in X$ and an affine open $W \subseteq Y$, define $\mathsf{ClosedNbhd}_i(x, W)$ to mean that $i(x) \in W$, the inverse image $i^{-1}(W)$ is an open neighbourhood of x, and $i^{-1}(W) \to W$ is a closed immersion.

Lemma 2.10.95 (Every image point has a closed affine neighbourhood). *Uses:* `def:nagata-immersion-closed-neighbourhood-predicate`

Lean: `AlgebraicGeometry.Immersion.closedNeighbourhoodPredicate_inhabited`

For every $x \in X$ there is an affine $W_x \subseteq Y$ satisfying $\mathsf{ClosedNbhd}_i(x, W_x)$.

Proof. Uses:

By the definition of an immersion, locally on the target it factors as a closed immersion into an open subscheme. Shrink that target neighbourhood around $i(x)$ to an affine open and restrict the closed immersion. □

Definition 2.10.96 (Finite closed-neighbourhood selection). Uses: `def:nagata-immersion-closed-neighbourhood-predicate`

Lean: `AlgebraicGeometry.Immersion.finiteClosedNeighbourhoodSelection`

Select one W_x for every $x \in X$. The opens $i^{-1}(W_x)$ cover X; quasi-compactness selects points $x_1, \dots, x_r$ whose inverse images still cover. Denote the corresponding finite target list by $W_1, \dots, W_r$ and store $X = \bigcup_{a=1}^{r} i^{-1}(W_a)$.

Definition 2.10.97 (Quasi-compact open on which an immersion is closed). Uses: `def:nagata-immersion-closed-neighbourhood-finite-selection`

Lean: `AlgebraicGeometry.Immersion.qcClosedNeighbourhood`

Define only the selected union

$$Y^{\mathrm{cl}}(i) = W_1 \cup \cdots \cup W_r.$$

Concretely this is the open subscheme represented by the union of the selected open subfunctors; the inclusions $W_a \hookrightarrow Y$ glue by their literal equality on pairwise intersections. Restrict i through this union using the factorization supplied by $i(X) \subseteq \bigcup_a W_a$.

Lemma 2.10.98 (The selected closed neighbourhood is quasi-compact open). *Uses:* `def:nagata-immersion-shrink-to-closed`

Lean: `AlgebraicGeometry.Immersion.qcClosedNeighbourhood_qcOpen`

The subscheme $Y^{\mathrm{cl}}(i)$ is a quasi-compact open of Y, and its open immersion into the quasi-separated scheme Y is quasi-compact.

Proof. Uses:

It is a finite union of affine opens. A quasi-compact open immersion into a quasi-separated scheme is quasi-compact. □

Lemma 2.10.99 (The selected closed neighbourhood contains the image). *Uses:* `def:nagata-immersion-shrink-to-closed`

Lean: `AlgebraicGeometry.Immersion.qcClosedNeighbourhood_contains`

The selected factorization of i lands in $Y^{\mathrm{cl}}(i)$.

Proof. Uses: `def:nagata-immersion-closed-neighbourhood-finite-selection`

This is the stored equality $X = \bigcup_a i^{-1}(W_a)$ for the finite selected family. □

Lemma 2.10.100 (Closedness on the selected neighbourhood). *Uses:* `def:nagata-immersion-shrink-to-closed`

Lean: `AlgebraicGeometry.Immersion.qcClosedNeighbourhood_closed`

The induced morphism $X \to Y^{\mathrm{cl}}(i)$ is a closed immersion.

Proof. Uses: `def:nagata-immersion-closed-neighbourhood-finite-selection`

Closedness is local on the target and holds over every selected W_a by its $\mathsf{ClosedNbhd}_i$ certificate. □

Lemma 2.10.101 (Properties of the closed-immersion neighbourhood, [1, Tag 01ZG]). *Uses:* `def:nagata-immersion-shrink-to-closed`

Lean: `AlgebraicGeometry.Immersion.exists_qcOpen_closedImmersion`

The constructed $Y^{\mathrm{cl}}(i)$ is a quasi-compact open containing $i(X)$, and $X \to Y^{\mathrm{cl}}(i)$ is a closed immersion.

Proof. Uses: `lemma:nagata-immersion-closed-neighbourhood-qc-open,lemma:nagata-immersion-closed-neighbourhood-contains,lemma:nagata-immersion-closed-neighbourhood-restriction`

These three lemmas are exactly the asserted properties. □

Definition 2.10.102 (Finite-presentation closed-subscheme system). Uses:

Lean: `AlgebraicGeometry.ClosedImmersion.finitePresentationSystem`

Let $X = V(\mathcal{I}) \hookrightarrow Y$ be closed with Y qcqs. Let $A_{X/Y}$ be the directed set of finite type quasi-coherent subideals $\mathcal{I}_\alpha \subseteq \mathcal{I}$, ordered by inclusion. Define

$$X_\alpha = V(\mathcal{I}_\alpha) \hookrightarrow Y,$$

and for $\alpha \leq \beta$ define the transition $X_\beta \hookrightarrow X_\alpha$ from $\mathcal{I}_\alpha \subseteq \mathcal{I}_\beta$.

Lemma 2.10.103 (Properties of the closed-subscheme system, [1, Tag 09ZP]). *Uses:* `def:nagata-closed-subscheme-fp-system`

Lean: `AlgebraicGeometry.ClosedImmersion.limit_finitePresentation`

Every $X_\alpha \hookrightarrow Y$ is of finite presentation, all transitions are closed immersions, and the canonical map $X \to \varprojlim_\alpha X_\alpha$ is an isomorphism.

Proof. Uses:

A quasi-coherent ideal on a qcqs scheme is the filtered union of its finite type quasi-coherent subideals. Finite type of $\mathcal{I}_\alpha$ is equivalent to finite presentation of $X_\alpha \hookrightarrow Y$. On an affine open, the coordinate ring of the limit is the quotient by $\varinjlim \mathcal{I}_\alpha = \mathcal{I}$, so the limit is X; these affine identifications glue. □

Definition 2.10.104 (Finite-presentation model system of a finite-type morphism). Uses: `def:nagata-immersion-into-fp-model,def:nagata-immersion-shrink-to-closed,def:nagata-closed-subscheme-fp-system`

Lean: `AlgebraicGeometry.finiteTypeFinitePresentationModelSystem`

For $X \to S$ locally of finite type with X qcqs and S quasi-separated, form $i^{\mathrm{imm}} \colon X \to Y^{\mathrm{imm}}$, set $Y^\circ = Y^{\mathrm{cl}}(i^{\mathrm{imm}})$, and apply Definition 2.10.102 to $X \hookrightarrow Y^\circ$. Denote the resulting system by

$$\mathsf{FP}(X/S) = (A_{X/S}, (X_\alpha)_\alpha),$$

with structural maps $i_\alpha \colon X \to X_\alpha$.

Lemma 2.10.105 (Properties of the finite-presentation model system, [1, Tag 09ZQ]). *Uses:* `def:nagata-finite-type-fp-model-system`

Lean: `AlgebraicGeometry.finiteType_limit_finitePresentation_models`

Every $X_\alpha \to S$ is of finite presentation, every transition map is a closed immersion, every i_α is a closed immersion, and $X \cong \varprojlim_\alpha X_\alpha$ over S.

Proof. Uses: `lemma:nagata-immersion-into-fp-model,lemma:nagata-immersion-shrink-to-closed,lemma:nagata-closed-subscheme-fp-system`

The immersion model is of finite presentation over S. Since S is quasi-separated, its source is quasi-separated; the selected open Y° is quasi-compact and remains of finite presentation over S. The closed-subscheme system then has closed transitions and limit X. Composition of each finite-presentation closed immersion $X_\alpha \hookrightarrow Y^\circ$ with $Y^\circ \to S$ is of finite presentation. The canonical maps from the limit are closed immersions because the transition maps are closed immersions. □

Definition 2.10.106 (Affine chart data for fixed-base separatedness). Uses: `def:nagata-directed-finite-upper-bound`

Lean: `AlgebraicGeometry.Limits.separatedChartDataOverFixedBase`

Let $X = \varprojlim_{\alpha} X_\alpha$ over a quasi-separated base S, with affine transitions, qcqs terms, and every $X_\alpha \to S$ of finite presentation; assume $X \to S$ separated. Fix one stage α_0 and affine opens $U_1, \dots, U_r \subseteq S$ covering the image of X_{α_0}. Put $X_{\alpha,a} = X_\alpha \times_S U_a$. For every a, choose a finite affine cover $(P_{\alpha_0,a,p})_p$ of $X_{\alpha_0,a}$. For $\alpha \geq \alpha_0$ and on the limit define, respectively,

$$P_{\alpha,a,p} = P_{\alpha_0,a,p} \times_{X_{\alpha_0,a}} X_{\alpha,a}, \qquad P_{a,p} = P_{\alpha_0,a,p} \times_{X_{\alpha_0,a}} X_a.$$

These are affine because the transition and limit projections are affine, and the displayed families cover the corresponding schemes.

Definition 2.10.107 (Affine-intersection stage for fixed-base separatedness). Uses: `def:nagata-fixed-base-separated-chart-data`, `def:nagata-directed-finite-upper-bound`

Lean: `AlgebraicGeometry.Limits.separatedAffineIntersectionStage`

Separatedness of $X_a \to U_a$ makes every $P_{a,p} \cap P_{a,q}$ affine. Apply Tag 01Z6 to the finitely many inverse systems $(P_{\alpha,a,p} \cap P_{\alpha,a,q})_\alpha$ and select a common stage α_{aff} at which all these intersections are affine. Store the cartesian transition squares for the intersections; they imply that affineness persists at every $\alpha \geq \alpha_{\mathrm{aff}}$.

Definition 2.10.108 (Surjectivity-witness stage for fixed-base separatedness). Uses: `def:nagata-fixed-base-affine-intersection-stage`, `def:nagata-directed-finite-upper-bound`

Lean: `AlgebraicGeometry.Limits.separatedSurjectivityStage`

Write $U_a = \mathrm{Spec}(R_a)$, $P_{\alpha,a,p} = \mathrm{Spec}(B_{\alpha,a,p})$, and $P_{\alpha,a,p} \cap P_{\alpha,a,q} = \mathrm{Spec}(C_{\alpha,a,pq})$ above α_{aff}. Since the diagonal of $X_{\alpha_{\mathrm{aff}},a} \to U_a$ is locally of finite type, select a finite family $(c_{a,pq,t})_t$ generating $C_{\alpha_{\mathrm{aff}},a,pq}$ over $B_{\alpha_{\mathrm{aff}},a,p} \otimes_{R_a} B_{\alpha_{\mathrm{aff}},a,q}$. The cartesian intersection squares imply that their images generate every later $C_{\alpha,a,pq}$ over the corresponding tensor product. In the limit the multiplication map is surjective because $X_a \to U_a$ is separated. For every image of $c_{a,pq,t}$ select a preimage in the filtered colimit of the tensor-product rings, a stage carrying that preimage, and a further stage carrying the equality with the named generator. Let α_{surj} be the common upper bound of α_{aff} and all preimage and equality stages. At that stage the maps

$$B_{\alpha_{\mathrm{surj}},a,p} \otimes_{R_a} B_{\alpha_{\mathrm{surj}},a,q} \longrightarrow C_{\alpha_{\mathrm{surj}},a,pq}$$

are surjective.

Definition 2.10.109 (Selected stage for fixed-base separatedness). Uses: `def:nagata-fixed-base-surjectivity-stage`

Lean: `AlgebraicGeometry.Limits.separatedStageOverFixedBase`

Define the selected common stage by $\alpha_{\mathrm{sep}} = \alpha_{\mathrm{surj}}$ and retain the finite affine covers, affine-intersection certificates, generator lists, preimages, and equality witnesses constructed above.

Lemma 2.10.110 (Separatedness at the selected fixed-base stage). *Uses:* `def:nagata-fixed-base-separated-stage`

Lean: `AlgebraicGeometry.Limits.separated_at_selected_stage`

The morphism $X_{\alpha_{\mathrm{sep}}} \to S$ is separated.

Proof. Uses: `def:nagata-fixed-base-affine-intersection-stage`, `def:nagata-fixed-base-surjectivity-stage`

Over each U_a, the selected $P_{\alpha_{\mathrm{sep}},a,p}$ form an affine cover with affine pairwise intersections. Every multiplication map to an intersection ring is surjective because the selected preimages hit the finite transported generator list. The affine-cover criterion therefore makes the diagonal closed over each U_a, and target-locality glues these closed diagonals over S. □

Lemma 2.10.111 (Persistence of fixed-base separatedness). *Uses:* `def:nagata-fixed-base-separated-stage`

Lean: `AlgebraicGeometry.Limits.separated_selected_stage_persists`

For every $\alpha \geq \alpha_{\mathrm{sep}}$, the morphism $X_\alpha \to S$ is separated.

Proof. Uses:

The transition $X_\alpha \to X_{\alpha_{\mathrm{sep}}}$ is affine, hence separated, and its composite with the selected separated structure morphism is separated. □

Lemma 2.10.112 (Eventual separatedness over a fixed base, [1, Tag 01ZH]). *Uses:* `def:nagata-fixed-base-separated-stage`

Lean: `AlgebraicGeometry.Limits.separated_eventually_over_fixedBase`

For every $\alpha \geq \alpha_{\mathrm{sep}}$, the morphism $X_\alpha \to S$ is separated.

Proof. Uses: `lemma:nagata-fixed-base-separated-persistence`

This is the persistence lemma, whose selected-stage proof is the witness-level argument of [1, Tag 086Q and Tag 01ZH]. □

Definition 2.10.113 (Separated finite-presentation envelope). Uses: `def:nagata-finite-type-fp-model-system,def:nagata-fixed-base-separated-stage`

Lean: `AlgebraicGeometry.closedImmersionFinitePresentationEnvelope`

Let S be qcqs and let $X \to S$ be separated and of finite type. Form

$$\mathsf{FP}(X/S) = (A_{X/S}, (X_\alpha)_\alpha).$$

Apply Definition 2.10.109 to this system and define

$$\alpha_{\mathrm{sep}}^{X/S} = \alpha_{\mathrm{sep}}, \qquad X^{\mathrm{fp}} = X_{\alpha_{\mathrm{sep}}^{X/S}}, \qquad i_{X/S}^{\mathrm{fp}} = i_{\alpha_{\mathrm{sep}}^{X/S}} \colon X \hookrightarrow X^{\mathrm{fp}}.$$

Denote the resulting explicit pair by $\mathsf{Env}_{\mathrm{fp}}(X/S) = (X^{\mathrm{fp}}, i_{X/S}^{\mathrm{fp}})$.

Proposition 2.10.114 (Properties of the separated finite-presentation envelope, [1, Tag 01ZJ]). *Uses:* `def:nagata-finite-type-closed-in-fp`

Lean: `AlgebraicGeometry.exists_closedImmersion_finitePresentation_model`

The constructed morphism $X^{\mathrm{fp}} \to S$ is separated and of finite presentation, and the constructed map $i_{X/S}^{\mathrm{fp}} \colon X \to X^{\mathrm{fp}}$ is a closed immersion over S.

Proof. Uses: `lemma:nagata-finite-type-fp-model-system,lemma:nagata-fixed-base-separated-eventually`

A finite-type morphism over a quasi-compact base has quasi-compact source, and separatedness makes that source quasi-separated, so the model system is available. Its property lemma makes every term finite presentation and every structural map i_α a closed immersion. At the explicitly selected stage $\alpha_{\mathrm{sep}}^{X/S}$, separatedness follows from Lemma 2.10.112. These are exactly the three assertions for the named envelope. □

2.10.3 Descent of relative finite-presentation data

In this subsection fix a directed inverse system $S = \varprojlim_{\lambda \in I} S_\lambda$ with qcqs terms and affine transition morphisms.

Definition 2.10.115 (Basic-open data for a quasi-compact open). Uses:

Lean: `AlgebraicGeometry.Limits.qcOpenBasicData`

Let Y_{λ_0} be qcqs and of finite presentation over S_{λ_0}. Put $Y = S \times_{S_{\lambda_0}} Y_{\lambda_0}$, and let $W \subseteq Y$ be a quasi-compact open. Select a finite affine cover $Y_{\lambda_0} = \bigcup_a Q_{a,\lambda_0}$. Its inverse images $Q_a \subseteq Y$ are affine, and quasi-separatedness makes $W \cap Q_a$ quasi-compact. Choose presentations

$$W \cap Q_a = \bigcup_{k=1}^{m_a} D(f_{ak})$$

and store the finiteness and cover equations.

Definition 2.10.116 (Function representatives for a quasi-compact open). Uses: `def:nagata-qc-open-basic-data`

Lean: `AlgebraicGeometry.Limits.qcOpenFunctionLifts`

For every f_{ak} select a finite-stage lift $f_{ak,\lambda_{ak}}$ and store the equation that its pullback to Q_a is f_{ak}.

Definition 2.10.117 (Common stage for a quasi-compact open). Uses: `def:nagata-qc-open-function-lifts,def:nagata-directed-finite-upper-bound`

Lean: `AlgebraicGeometry.Limits.qcOpenDescentStage`

Define the N17 common-stage field by

$$\lambda_W = u(\{\lambda_0\} \cup \{\lambda_{ak}\}_{a,k}).$$

Transport every already selected function and its pullback equation to this common stage.

Definition 2.10.118 (Finite-stage model of a quasi-compact open)**.** Uses: `def:nagata-qc-open-descent-stage`

Lean: `AlgebraicGeometry.Limits.qcOpenDescentConstruction`

Transport every lifted function to λ_W and define the N17 descended-open field to be the actual finite union

$$W_{\lambda_W} = \bigcup_{a,k} D(f_{ak,\lambda_W}) \subseteq Y_{\lambda_W}.$$

Lemma 2.10.119 (Quasi-compactness of the descended open)**.** *Uses:* `def:nagata-qc-open-descent`

Lean: `AlgebraicGeometry.Limits.qcOpenDescentConstruction_qc`

The open $W_{\lambda_W} \subseteq Y_{\lambda_W}$ is quasi-compact.

Proof. Uses:

It is a finite union of basic opens in the selected finite affine cover. □

Lemma 2.10.120 (Base-change equation for the descended open)**.** *Uses:* `def:nagata-qc-open-descent`

Lean: `AlgebraicGeometry.Limits.qcOpenDescentConstruction_baseChange`

The inverse image of W_{λ_W} in Y is exactly W.

Proof. Uses: `def:nagata-qc-open-function-lifts`

Pullback sends each $D(f_{ak,\lambda_W})$ to $D(f_{ak})$; the selected finite unions agree with W on every Q_a and hence globally. □

Lemma 2.10.121 (Property of the descended quasi-compact open)**.** *Uses:* `def:nagata-qc-open-descent`

Lean: `AlgebraicGeometry.Limits.qcOpen_descends`

The constructed W_{λ_W} is quasi-compact, and its inverse image in Y is exactly W.

Proof. Uses: `lemma:nagata-qc-open-descent-qc`, `lemma:nagata-qc-open-descent-base-change`

These are the two asserted fields. □

Definition 2.10.122 (Basic presentations for equality of quasi-compact opens)**.** Uses:

Lean: `AlgebraicGeometry.Limits.qcOpenEqualityBasicData`

Let Y_{λ_0} be qcqs, put $Y = S \times_{S_{\lambda_0}} Y_{\lambda_0}$, and let $W_{1,\lambda_0}, W_{2,\lambda_0} \subseteq Y_{\lambda_0}$ be quasi-compact opens. Denote their inverse images in Y by W_1, W_2 and assume $W_1 = W_2$. Choose a finite affine cover $Y_{\lambda_0} = \bigcup_a Q_{a,\lambda_0}$, with $Q_{a,\lambda_0} = \operatorname{Spec}(A_{a,\lambda_0})$. Since the two stage opens are quasi-compact, select finite basic presentations

$$W_{1,\lambda_0} \cap Q_{a,\lambda_0} = \bigcup_r D(f_{ar,\lambda_0}), \qquad W_{2,\lambda_0} \cap Q_{a,\lambda_0} = \bigcup_s D(g_{as,\lambda_0})$$

with $f_{ar,\lambda_0}, g_{as,\lambda_0} \in A_{a,\lambda_0}$. Store these two finite-stage cover equations. Denote their pullbacks to the limiting affine chart by f_{ar}, g_{as}; the pullback equations present W_1 and W_2 there. We suppress the chart index a below.

Definition 2.10.123 (Radical-containment witnesses for open equality)**.** Uses: `def:nagata-qc-open-equality-basic-data`

Lean: `AlgebraicGeometry.Limits.qcOpenRadicalWitnesses`

Equality after pullback says, in each limiting chart ring,

$$f_r^{N_r} = \sum_s c_{rs} g_s, \qquad g_s^{M_s} = \sum_r d_{sr} f_r$$

Select positive exponents and coefficients in the limiting chart ring and store each displayed identity separately.

Definition 2.10.124 (Finite-stage representatives of radical witnesses)**.** Uses: `def:nagata-qc-open-radical-witnesses`

Lean: `AlgebraicGeometry.Limits.qcOpenRadicalWitnessStages`

The elements f_r, g_s already come from the named λ_0-stage basic presentations. Choose later stages carrying every c_{rs}, d_{sr} and then stages at which each of the finitely many displayed equations, with the transported f_r, g_s, holds. Store the representative and equality-stage specification for every coefficient witness.

Definition 2.10.125 (Stage of equality for descended quasi-compact opens)**.** Uses: def:nagata-qc-open-radical-witness-stages,def:nagata-directed-finite-upper-bound

Lean: AlgebraicGeometry.Limits.qcOpenEqualityStage

Define $\lambda_{W_1=W_2}$ to be the selected upper bound of λ_0 together with these coefficient and equality stages.

Lemma 2.10.126 (First inclusion for descended open equality)**.** *Uses:* def:nagata-qc-open-equality-stage

Lean: AlgebraicGeometry.Limits.qcOpenEquality_firstInclusion

At every $\lambda \geq \lambda_{W_1=W_2}$, the pullback of W_{1,λ_0} is contained in the pullback of W_{2,λ_0}.

Proof. Uses: def:nagata-qc-open-radical-witness-stages

The equations $f_r^{N_r} = \sum_s c_{rs} g_s$ give the inclusion on each affine chart, and their transported finite-stage certificates persist after base change. □

Lemma 2.10.127 (Second inclusion for descended open equality)**.** *Uses:* def:nagata-qc-open-equality-stage

Lean: AlgebraicGeometry.Limits.qcOpenEquality_secondInclusion

At every $\lambda \geq \lambda_{W_1=W_2}$, the pullback of W_{2,λ_0} is contained in the pullback of W_{1,λ_0}.

Proof. Uses: def:nagata-qc-open-radical-witness-stages

The equations $g_s^{M_s} = \sum_r d_{sr} f_r$ give the reverse inclusion on each affine chart and persist after base change. □

Lemma 2.10.128 (Eventual equality of descended quasi-compact opens)**.** *Uses:* def:nagata-qc-open-equality-stage

Lean: AlgebraicGeometry.Limits.qcOpen_eq_eventually

For every $\lambda \geq \lambda_{W_1=W_2}$, the inverse images of W_{1,λ_0} and W_{2,λ_0} in Y_λ are equal.

Proof. Uses: lemma:nagata-qc-open-equality-first-inclusion,lemma:nagata-qc-open-equality-second-inclusion

Antisymmetry combines the two separately proved inclusions. This is the explicit radical-containment certificate used in the equality step of [1, Tag 01ZM]. □

Definition 2.10.129 (Coefficient representatives for an affine presentation)**.** Uses:

Lean: AlgebraicGeometry.Limits.affineFinitePresentationCoefficientLifts

Suppose $S_{\lambda_0} = \mathrm{Spec}(A_{\lambda_0})$, $S = \mathrm{Spec}(A)$, and fix a finite presentation

$$B = A[x_1, \ldots, x_n]/(r_1, \ldots, r_m).$$

Select a finite-stage lift of every coefficient of every r_j and store its coefficientwise pullback equation.

Definition 2.10.130 (Coefficient stage of a finitely presented affine algebra)**.** Uses: def:nagata-affine-fp-coefficient-lifts,def:nagata-directed-finite-upper-bound

Lean: AlgebraicGeometry.Limits.affineFinitePresentationCoefficientStage

Let λ_B be the selected upper bound of λ_0 and all coefficient-representative stages. Transport every lift and specification to λ_B. This is the N19 common-stage field.

Definition 2.10.131 (Lifted relation list)**.** Uses: def:nagata-affine-fp-coefficient-stage

Lean: AlgebraicGeometry.Limits.affineFinitePresentationLiftedPolynomials

Transport every selected coefficient lift to A_{λ_B} and let r_{j,λ_B} be the lifted polynomials, and define $(r_{1,\lambda_B}, \ldots, r_{m,\lambda_B})$ to be the N19 lifted-relation field. Each polynomial maps to the corresponding r_j coefficientwise.

Definition 2.10.132 (Finite-stage quotient algebra)**.** Uses: def:nagata-affine-fp-lifted-polynomials

Lean: AlgebraicGeometry.Limits.affineFinitePresentationQuotient

Define the N19 quotient-algebra field by

$$B_{\lambda_B} = A_{\lambda_B}[x_1, \ldots, x_n]/(r_{1,\lambda_B}, \ldots, r_{m,\lambda_B}).$$

Definition 2.10.133 (Forward comparison for affine finite-presentation descent)**.** Uses: `def:nagata-affine-fp-quotient-algebra`

Lean: `AlgebraicGeometry.Limits.affineFinitePresentationForwardMap`

Define β_B to be the comparison from $B_{\lambda_B} \otimes_{A_{\lambda_B}} A$ to B induced by the presentation. It sends each polynomial generator to its named class and the coefficient copy of A identically to $A \subseteq B$.

Definition 2.10.134 (Reverse comparison for affine finite-presentation descent)**.** Uses: `def:nagata-affine-fp-forward-map`

Lean: `AlgebraicGeometry.Limits.affineFinitePresentationReverseMap`

Define γ_B in the other direction by sending the class of each x_i in B to $x_i \otimes 1$ and the image of $a \in A$ to $1 \otimes a$. The coefficientwise lift equations for the relations make this map well-defined.

Lemma 2.10.135 (Left inverse equation for affine descent)**.** Uses: *def:nagata-affine-fp-forward-map*, *def:nagata-affine-fp-reverse-map*

Lean: *AlgebraicGeometry.Limits.affineFinitePresentation_leftInverse*

The selected maps satisfy $\beta_B \gamma_B = \mathrm{id}_B$.

Proof. Uses:

The composite fixes the coefficient algebra and every named polynomial generator of the chosen presentation of B, hence is the identity. □

Lemma 2.10.136 (Right inverse equation for affine descent)**.** Uses: *def:nagata-affine-fp-forward-map*, *def:nagata-affine-fp-reverse-map*

Lean: *AlgebraicGeometry.Limits.affineFinitePresentation_rightInverse*

The selected maps satisfy

$$\gamma_B \beta_B = \mathrm{id}_{B_{\lambda_B} \otimes_{A_{\lambda_B}} A}.$$

Proof. Uses:

The composite fixes the coefficient copy of A and every polynomial generator $x_i \otimes 1$, which generate the displayed tensor-product quotient. □

Definition 2.10.137 (Finite-stage model of a finitely presented affine algebra)**.** Uses: `def:nagata-affine-fp-forward-map`,`def:nagata-affine-fp-reverse-map`

Lean: `AlgebraicGeometry.Limits.affineFinitePresentationModel`

Package B_{λ_B}, β_B, γ_B, and the two separately proved inverse equations as the selected N19 affine descent record.

Lemma 2.10.138 (Finite presentation of the descended affine algebra)**.** Uses: *def:nagata-affine-fp-quotient-algebra*

Lean: *AlgebraicGeometry.Limits.affineFinitePresentationQuotient_finitePresentation*

The algebra B_{λ_B} *is finitely presented over* A_{λ_B}.

Proof. Uses:

It is a polynomial algebra on a finite generator list modulo the finite selected relation list. □

Lemma 2.10.139 (Isomorphism of the affine comparison)**.** Uses: *def:nagata-affine-fp-descent*

Lean: *AlgebraicGeometry.Limits.affineFinitePresentationComparison_iso*

The map β_B *is an isomorphism with inverse* γ_B.

Proof. Uses: `lemma:nagata-affine-fp-left-inverse`,`lemma:nagata-affine-fp-right-inverse`

The two displayed inverse equations are the isomorphism certificate. □

Lemma 2.10.140 (Property of affine finite-presentation descent)**.** Uses: *def:nagata-affine-fp-descent*

Lean: *AlgebraicGeometry.Limits.affine_finitePresentation_descends*

The algebra B_{λ_B} *is finitely presented over* A_{λ_B}*, and the constructed map* β_B *is an isomorphism.*

Proof. Uses: `lemma:nagata-affine-fp-quotient-finite-presentation`,`lemma:nagata-affine-fp-comparison-isomorphism`

These are the two asserted fields. □

Definition 2.10.141 (Stage of equality for two descended morphisms)**.** Uses: `def:nagata-qc-open-equality-stage,def:nagata-directed-finite-upper-bound`

Lean: `AlgebraicGeometry.Limits.morphismEqualityStage`

Let X_{λ_0} and Y_{λ_0} be of finite presentation over S_{λ_0}, and let $\varphi_{\lambda_0}, \psi_{\lambda_0} \colon X_{\lambda_0} \to Y_{\lambda_0}$ be S_{λ_0}-morphisms with equal base change to S. Choose a finite affine cover $(V_{j,\lambda_0})_j$ of Y_{λ_0}, with every member mapping into an affine open of S_{λ_0}. The inverse-image opens $U^{\varphi}_{j,\lambda_0}$ and U^{ψ}_{j,λ_0} have the same pullback to X_S. Apply the N18 radical-containment construction to every pair and take a common stage λ_{open} at which

$$U^{\varphi}_{j,\lambda_{\mathrm{open}}} = U^{\psi}_{j,\lambda_{\mathrm{open}}} =: U_{j,\lambda_{\mathrm{open}}}.$$

Choose a finite affine cover $(W_{jk})_k$ of every common U_j. On each W_{jk} the two maps have common affine target V_j; equality after base change gives equality of the two ring maps on a finite presentation of $\mathcal{O}(V_j)$. Select the finitely many coefficient-equality stages and define $\lambda_{\varphi=\psi}$ to be their upper bound together with λ_{open}.

Lemma 2.10.142 (Eventual equality of morphisms, [1, Tag 01ZM(3)])**.** Uses: `def:nagata-morphism-equality-stage`

Lean: `AlgebraicGeometry.Limits.hom_eq_eventually`

For every $\lambda \geq \lambda_{\varphi=\psi}$, the base changes of φ_{λ_0} and ψ_{λ_0} to S_λ are equal.

Proof. Uses: `lemma:nagata-qc-open-eq-eventually,lemma:nagata-directed-finite-upper-bound`

Above the selected stage, the two maps have equal inverse images of every target chart. On a finite affine cover of each common inverse image, their ring maps agree on a finite generating set and hence agree. The local equalities glue to the asserted global equality and persist at later stages. □

Definition 2.10.143 (Cover data for descent of a morphism)**.** Uses: `def:nagata-qc-open-descent,def:nagata-qc-open-equality-stage,def:nagata-directed-finite-upper-bound`

Lean: `AlgebraicGeometry.Limits.morphismDescentCoverData`

Let $X_{\lambda_0}, Y_{\lambda_0}$ be of finite presentation over S_{λ_0} and let $\varphi \colon X_{\lambda_0,S} \to Y_{\lambda_0,S}$ be an S-morphism. Choose a finite affine cover $(V_{j,\lambda_0})_j$ of Y_{λ_0}, with every member mapping into an affine open T_{j,λ_0} of S_{λ_0}. Apply N17 inside the restricted system $X_\lambda \times_{S_\lambda} T_{j,\lambda}$ and descend the quasi-compact opens $U_j = \varphi^{-1}(V_{j,S})$ to opens U_{j,λ_1} over T_{j,λ_1}. Their union and X_{λ_1} have the same pullback to S. Apply N18 to these two quasi-compact opens and select a cover-equality stage λ_{cov} at which

$$X_{\lambda_{\mathrm{cov}}} = \bigcup_j U_{j,\lambda_{\mathrm{cov}}}.$$

Choose finite affine covers $U_{j,\lambda_{\mathrm{cov}}} = \bigcup_k W_{jk,\lambda_{\mathrm{cov}}}$. Store the target subordination, descended-open base-change equations, cover equality, and affine source cover as separate fields.

Definition 2.10.144 (Raw local maps for descent of a morphism)**.** Uses: `def:nagata-morphism-descent-cover-data,def:nagata-affine-fp-descent,def:nagata-directed-finite-upper-bound`

Lean: `AlgebraicGeometry.Limits.morphismDescentLocalMaps`

On the limit, the restricted map is an $\mathcal{O}(T_{j,S})$-algebra map

$$\mathcal{O}(V_{j,S}) \longrightarrow \mathcal{O}(W_{jk,S}).$$

Lift the images of a finite presentation of the target algebra and the finitely many relation equalities as $\mathcal{O}(T_{j,\lambda})$-algebra data to obtain maps $\varphi_{jk,\lambda_2} \colon W_{jk,\lambda_2} \to Y_{\lambda_2}$ whose image factors through V_{j,λ_2}. Store the resulting equality of its composite to S_{λ_2} with the source structure map. For every (j,k) this gives a typed raw local S_{λ_2}-morphism whose base change is the named restriction of φ.

Definition 2.10.145 (Overlap-equality stages for descended local maps)**.** Uses: `def:nagata-morphism-descent-local-maps,def:nagata-morphism-equality-stage,def:nagata-directed-finite-upper-bound`

Lean: `AlgebraicGeometry.Limits.morphismDescentOverlapStages`

For every pair $a = (j,k)$, $b = (j',k')$, put $D_{ab,\lambda_2} = W_{a,\lambda_2} \cap W_{b,\lambda_2}$. The two restrictions $D_{ab,\lambda_2} \rightrightarrows Y_{\lambda_2}$ have equal base change; apply the preceding morphism-equality construction and store its witness stage and typed equality certificate.

Definition 2.10.146 (Common stage for descent of a morphism)**.** Uses: `def:nagata-morphism-descent-overlap-stages,def:nagata-directed-finite-upper-bound`

Lean: `AlgebraicGeometry.Limits.morphismDescentStage`

Define λ_φ to be the upper bound of λ_{cov}, all local-map stages, and all finitely many typed overlap-equality stages. Transport the already selected cover, maps, base-map equations, and equality certificates to this common stage. This is the N20 common-stage field.

Definition 2.10.147 (Cocone certificate for descended local morphisms)**.** Uses: `def:nagata-morphism-descent-stage`

Lean: `AlgebraicGeometry.Limits.morphismDescentCocone`

For each pair a, b, transport the equality certificate to λ_φ and store

$$\varphi_{a,\lambda_\varphi} \circ \iota^{a}_{ab,\lambda_\varphi} = \varphi_{b,\lambda_\varphi} \circ \iota^{b}_{ab,\lambda_\varphi} : D_{ab,\lambda_\varphi} \longrightarrow Y_{\lambda_\varphi},$$

where both ι^{a}_{ab} and ι^{b}_{ab} are the named open inclusions of the same intersection. Also store the local S_{λ_φ}-map equations transported from the $\mathcal{O}(T_j)$-linear lifts. Together with the proved cover equality, these typed equations are the N20 morphism-cocone certificate in schemes over S_{λ_φ}.

Definition 2.10.148 (Finite-stage model of a morphism)**.** Uses: `def:nagata-morphism-descent-cocone`

Lean: `AlgebraicGeometry.Limits.morphismDescentConstruction`

Glue the certified maps on the cover $(W_{jk,\lambda_\varphi})_{j,k}$ and define the N20 global-morphism field

$$\varphi_{\lambda_\varphi} : X_{\lambda_\varphi} \longrightarrow Y_{\lambda_\varphi}.$$

Lemma 2.10.149 (Property of the descended morphism, [1, Tag 01ZM(2)])**.** *Uses:* `def:nagata-morphism-descent`

Lean: `AlgebraicGeometry.Limits.hom_extends`

The base change of the constructed $\varphi_{\lambda_\varphi}$ to S is φ.

Proof. Uses: `lemma:nagata-qc-open-descends,lemma:nagata-affine-fp-descends,lemma:nagata-morphism-eq-eventually`

The N18 cover-equality certificate says the descended source opens cover the whole finite-stage source. Each local ring map pulls back to the original restriction, and the typed N20 cocone permits gluing. After base change the glued map and φ agree on every $W_{jk,S}$, hence agree globally. This is the construction in part (2) of [1, Tag 01ZM]. □

Definition 2.10.150 (Finite-stage base affine cover for relative descent)**.** Uses:

Lean: `AlgebraicGeometry.Limits.relativeBaseAffineCover`

Fix λ_0 and select a finite affine cover $S_{\lambda_0} = \bigcup_a T_{a,\lambda_0}$. For $\lambda \geq \lambda_0$ put $T_{a,\lambda} = T_{a,\lambda_0} \times_{S_{\lambda_0}} S_\lambda$ and put $T_a = T_{a,\lambda_0} \times_{S_{\lambda_0}} S$. The transition and limit projections are affine, so these are finite affine covers at every stage and $T_a = \varprojlim_\lambda T_{a,\lambda}$.

Definition 2.10.151 (Relative chart-presentation data)**.** Uses: `def:nagata-relative-base-affine-cover`

Lean: `AlgebraicGeometry.Limits.relativeChartPresentationData`

Let $X \to S$ be of finite presentation. Select finite affine covers $X \times_S T_a = \bigcup_b U_{ab}$ and record finite presentations

$$\mathcal{O}_X(U_{ab}) = \mathcal{O}_S(T_a)[x_{ab,1}, \ldots, x_{ab,n_{ab}}]/(r_{ab,1}, \ldots, r_{ab,m_{ab}}).$$

together with their finite generator, relation, and coefficient index types.

Definition 2.10.152 (Relative overlap-presentation data)**.** Uses: `def:nagata-relative-chart-presentation-data`

Lean: `AlgebraicGeometry.Limits.relativeOverlapPresentationData`

Quasi-separatedness of X makes every $U_{ab} \cap U_{a'b'}$ quasi-compact; choose finite basic affine covers of these intersections in both charts and record both coordinate-ring maps for the identity isomorphism. On triple intersections record the restrictions of those maps, their proposed inverses, and the finite identity, inverse, structural, and cocycle equation index types.

Definition 2.10.153 (Selected finite relative affine presentation)**.** Uses: `def:nagata-relative-base-affine-cover,def:nagata-relative-chart-presentation-data,def:nagata-relative-overlap-presentation-data`

Lean: `AlgebraicGeometry.Limits.relativeFiniteAffinePresentation`

Package the previously selected fields. The raw record R_X consists of the finite index types, finite-stage base opens, rings, polynomials, basic-open elements, and maps. It performs no new selection; all lists are finite by their defining records.

Lemma 2.10.154 (Inverse equations in the relative affine presentation)**.** *Uses:* def:nagata-relative-finite-affine-presentation

Lean: AlgebraicGeometry.Limits.relativeFiniteAffinePresentation_inverse

For every ordered pair p, q, the selected maps $\phi_{pq}\colon W^p_{pq} \to W^q_{qp}$ and ϕ_{qp} have the named domains and satisfy both inverse equations.

Proof. Uses:

Both maps are restrictions of the identity identification of $U_p \cap U_q$. On every finite basic cell their coordinate maps are the two selected localization maps, whose composites fix the coefficient ring and all localized generators. The cellwise identities glue. □

Lemma 2.10.155 (Triple cocycle in the relative affine presentation)**.** *Uses:* def:nagata-relative-finite-affine-presentation

Lean: AlgebraicGeometry.Limits.relativeFiniteAffinePresentation_cocycle

On every named triple-overlap domain, $\phi_{pr} = \phi_{qr}\phi_{pq}$.

Proof. Uses:

After the selected common basic refinement, both sides are restrictions of the identity identification of $U_p \cap U_q \cap U_r$; equality holds on the finite ring generators and therefore on each cell, then glues. □

Lemma 2.10.156 (Structural compatibility in the relative affine presentation)**.** *Uses:* def:nagata-relative-finite-affine-presentation

Lean: AlgebraicGeometry.Limits.relativeFiniteAffinePresentation_overBase

The two maps from every overlap to S agree after composition with ϕ_{pq}.

Proof. Uses:

Both composites are restrictions of the given structural map $X \to S$. □

Definition 2.10.157 (Certificate for the finite relative affine presentation)**.** Uses: def:nagata-relative-finite-affine-presentation

Lean: AlgebraicGeometry.Limits.relativeFiniteAffinePresentationCertificate

Package the named overlap opens, the selected ϕ_{pq}, both inverse lemmas, the triple-cocycle lemma on its typed common domains, and the structural compatibility lemma. This finite family is the N21 certificate; the Definition performs no new proof or selection.

Definition 2.10.158 (Chart transport isomorphisms for relative descent)**.** Uses: def:nagata-relative-finite-affine-presentation-certificate,def:nagata-affine-fp-descent

Lean: AlgebraicGeometry.Limits.relativeChartTransportIsomorphism

Apply N19 to each chart algebra and obtain $U_{p,\lambda_p} \to T_{a(p),\lambda_p}$. If β_p and γ_p are the two N19 ring maps, define their contravariant scheme maps

$$\epsilon_p = \mathrm{Spec}(\gamma_p)\colon U_{p,\lambda_p,S} \longrightarrow U_p, \qquad \delta_p = \mathrm{Spec}(\beta_p)\colon U_p \longrightarrow U_{p,\lambda_p,S}.$$

Store separately the equations $\epsilon_p\delta_p = \mathrm{id}_{U_p}$ and $\delta_p\epsilon_p = \mathrm{id}_{U_{p,\lambda_p,S}}$.

Definition 2.10.159 (Common starting stage for transported relative charts)**.** Uses: def:nagata-relative-chart-transport-isomorphism,def:nagata-directed-finite-upper-bound

Lean: AlgebraicGeometry.Limits.relativeChartCommonStage

There are finitely many chart stages λ_p. Define λ_{chart} to be their selected common upper bound. For every p, transport U_{p,λ_p} to

$$U_{p,\lambda_{\mathrm{chart}}} = U_{p,\lambda_p} \times_{S_{\lambda_p}} S_{\lambda_{\mathrm{chart}}}.$$

Base change the two chart maps and retain the notation

$$\epsilon_p\colon U_{p,\lambda_{\mathrm{chart}},S} \to U_p, \qquad \delta_p\colon U_p \to U_{p,\lambda_{\mathrm{chart}},S}.$$

Their two inverse equations persist by base change. Thus every transported chart, overlap, and map used below belongs to a system on the single cofinal tail $I_{\geq\lambda_{\mathrm{chart}}}$.

Definition 2.10.160 (Typed transport of overlap opens and transition maps)**.** Uses: `def:nagata-relative-chart-common-stage`,`def:nagata-relative-finite-affine-presentation-certificate`

Lean: `AlgebraicGeometry.Limits.relativeOverlapTransport`

Before applying an open- or morphism-descent constructor, transport every original overlap into the base change of its finite-stage ambient chart:

$$\widetilde{W}^p_{pq} = \epsilon_p^{-1}(W^p_{pq}) \subseteq U_{p,\lambda_{\mathrm{chart}},S}, \qquad \widetilde{W}^q_{qp} = \epsilon_q^{-1}(W^q_{qp}) \subseteq U_{q,\lambda_{\mathrm{chart}},S}.$$

Restrict the chart isomorphisms and define the typed transported map

$$\widetilde{\phi}_{pq} = \delta_q \circ \phi_{pq} \circ \epsilon_p \colon \widetilde{W}^p_{pq} \xrightarrow{\sim} \widetilde{W}^q_{qp}.$$

Its target lies in $\widetilde{W}^q_{qp}$ because $\epsilon_q\delta_q = \mathrm{id}$. Store the typed identities

$$\epsilon_q\widetilde{\phi}_{pq} = \phi_{pq}\epsilon_p, \qquad \widetilde{\phi}_{qp}\widetilde{\phi}_{pq} = \mathrm{id},$$

their reverse counterparts, and equality of the two structure maps to S. Thus the transported opens and maps, not objects living in the original ambient charts, are the inputs to N17 and N20.

Definition 2.10.161 (Common stage for descended overlap models)**.** Uses: `def:nagata-relative-overlap-transport`,`def:nagata-qc-open-descent`,`def:nagata-directed-finite-upper-bound`

Lean: `AlgebraicGeometry.Limits.relativeOverlapCommonStage`

For every ordered pair (p,q), apply N17 in the two restricted ambient systems to $\widetilde{W}^p_{pq}$ and $\widetilde{W}^q_{qp}$. Denote the resulting open-model stages by $\lambda^{p,\mathrm{op}}_{pq}$ and $\lambda^{q,\mathrm{op}}_{qp}$. Since there are finitely many pairs, select

$$\lambda_{\mathrm{open}} = u\big(\{\lambda_{\mathrm{chart}}\} \cup \{\lambda^{p,\mathrm{op}}_{pq}, \lambda^{q,\mathrm{op}}_{qp}\}_{p,q}\big)\,.$$

Transport every ambient chart, descended open, and N17 base-change certificate to this stage. Write the transported opens as

$$\widetilde{W}^p_{pq,\lambda_{\mathrm{open}}} \subseteq U_{p,\lambda_{\mathrm{open}}}, \qquad \widetilde{W}^q_{qp,\lambda_{\mathrm{open}}} \subseteq U_{q,\lambda_{\mathrm{open}}},$$

and store the isomorphisms

$$\theta^p_{pq} \colon (\widetilde{W}^p_{pq,\lambda_{\mathrm{open}}})_S \xrightarrow{\sim} \widetilde{W}^p_{pq}, \qquad \theta^q_{qp} \colon (\widetilde{W}^q_{qp,\lambda_{\mathrm{open}}})_S \xrightarrow{\sim} \widetilde{W}^q_{qp}.$$

Each descended open is of finite presentation over $S_{\lambda_{\mathrm{open}}}$: its open immersion into a finite-presentation chart is quasi-compact, hence of finite presentation. The conjugate

$$\widehat{\phi}_{pq,S} = (\theta^q_{qp})^{-1} \circ \widetilde{\phi}_{pq} \circ \theta^p_{pq} \colon (\widetilde{W}^p_{pq,\lambda_{\mathrm{open}}})_S \longrightarrow (\widetilde{W}^q_{qp,\lambda_{\mathrm{open}}})_S$$

is therefore a typed input to N20 with common starting stage λ_{open}.

Definition 2.10.162 (Finite-stage raw relative gluing datum)**.** Uses: `def:nagata-relative-overlap-common-stage`,`def:nagata-morphism-descent`,`def:nagata-directed-finite-upper-bound`

Lean: `AlgebraicGeometry.Limits.relativeGlueDataConstruction`

For every base member use the already selected affine system $T_a = \varprojlim T_{a,\lambda}$. The finite-stage charts and their overlap models and their base-change identifications now all live at λ_{open}. For every ordered pair apply N20 with

$$X_{\lambda_{\mathrm{open}}} = \widetilde{W}^p_{pq,\lambda_{\mathrm{open}}}, \qquad Y_{\lambda_{\mathrm{open}}} = \widetilde{W}^q_{qp,\lambda_{\mathrm{open}}}, \qquad \varphi = \widehat{\phi}_{pq,S}.$$

This produces a map stage $\lambda^{\mathrm{map}}_{pq}$ and a descended transition morphism. Define $\lambda_{\mathrm{raw}}(X/S)$ to be the upper bound of λ_{open} and the finitely many map stages $\lambda^{\mathrm{map}}_{pq}$. Transport all domains and codomains to that stage and denote the resulting typed raw datum by $\mathsf{R}^{\mathrm{raw}}_{X,\lambda_{\mathrm{raw}}}$.

Lemma 2.10.163 (Property of the descended finite affine presentation)**.** *Uses:* `def:nagata-relative-gluing-datum-descent`

Lean: `AlgebraicGeometry.Limits.finiteAffinePresentation.descends`

Every chart in the constructed raw datum base changes to its named original chart by ϵ_p. Every descended overlap and overlap map base changes to $\widetilde{W}^p_{pq}$ and $\widetilde{\phi}_{pq}$, and conjugating by the stored ϵ_p, δ_p recovers W^p_{pq} and ϕ_{pq} in R_X.

Proof. Uses: `lemma:nagata-qc-open-descends`,`lemma:nagata-affine-fp-descends`,`lemma:nagata-morphism-descends`

Apply the corresponding property lemma to each transported quasi-compact overlap and transported overlap morphism, then use the two chart inverse equations and $\epsilon_q\widetilde{\phi}_{pq} = \phi_{pq}\epsilon_p$. There are only finitely many objects, and the common upper bound transports all of them to one stage without changing their base changes. □

Definition 2.10.164 (Equation-witness stages for the relative gluing datum)**.** Uses: `def:nagata-relative-gluing-datum-descent`,`def:nagata-qc-open-equality-stage`,`def:nagata-morphism-equality-stage`,`def:nagata-directed-finite-upper-bound`

Lean: `AlgebraicGeometry.Limits.relativeGlueDataEquationStages`

For the raw datum, form the finite list of required typed equalities. For each p, q, apply morphism equality to $\phi_{qp}\phi_{pq}$ and $\mathrm{id}_{W^p_{pq}}$. For every triple define

$$D^p_{pqr} = W^p_{pq} \cap W^p_{pr}, \qquad E^p_{pqr} = W^p_{pq} \cap \phi^{-1}_{pq}(W^q_{qr}).$$

Their base changes are the same triple overlap; apply N18 to identify them at a finite stage. On that single domain apply morphism equality to the two maps ϕ_{pr} and $\phi_{qr}\phi_{pq}$ into U_r. Finally apply morphism equality to the two structural maps $W^p_{pq} \rightrightarrows S_\lambda$. Attach to each equation its selected witness stage and its typed equality specification.

Definition 2.10.165 (Cocycle-valid stage for the relative datum)**.** Uses: `def:nagata-relative-gluing-equation-stages`,`def:nagata-directed-finite-upper-bound`

Lean: `AlgebraicGeometry.Limits.relativeGlueDataValidStage`

Define $\lambda_{\mathrm{rel}}(X/S)$ to be the common upper bound of all equation-witness stages together with $\lambda_{\mathrm{raw}}(X/S)$. Let $\mathsf{R}_{X,\lambda_{\mathrm{rel}}}$ be the raw datum transported to this stage, with every previously proved typed equality transported alongside it.

Lemma 2.10.166 (The relative gluing axioms hold at the selected stage)**.** *Uses:* `def:nagata-relative-gluing-valid-stage`

Lean: `AlgebraicGeometry.Limits.finiteAffinePresentation.eventually_glueData`

In $\mathsf{R}_{X,\lambda_{\mathrm{rel}}}$ *every* ϕ_{pq} *has the named source and target, the two directions are inverse, the structural maps agree, and all triple-overlap cocycle identities hold on the identified typed domains.*

Proof. Uses: `lemma:nagata-relative-gluing-datum-descends`,`lemma:nagata-qc-open-eq-eventually`,`lemma:nagata-morphism-eq-eventually`,`lemma:nagata-directed-finite-upper-bound`

Every required equality holds after base change to S. Equality of opens and equality of morphisms therefore hold above their selected witness stages. The common upper bound includes all finitely many pair and triple conditions, so they hold simultaneously in the named datum. □

Definition 2.10.167 (Relative gluing certificate)**.** Uses: `def:nagata-relative-gluing-valid-stage`

Lean: `AlgebraicGeometry.Limits.relativeGlueDataCertificate`

Package the already proved open-domain identifications, inverse equations, structural-map equalities, and triple-cocycle equations at λ_{rel} as the scheme-gluing certificate $\mathsf{C}_{X,\lambda_{\mathrm{rel}}}$. Every proof field is the independently certified equation at the displayed common domain and codomain; the Definition performs no new proof.

Definition 2.10.168 (Descended finite-presentation scheme)**.** Uses: `def:nagata-relative-gluing-certificate`

Lean: `AlgebraicGeometry.Limits.finitePresentationDescendedScheme`

Apply scheme gluing to $\mathsf{C}_{X,\lambda_{\mathrm{rel}}}$ and define

$$X^{\mathrm{desc}}_{\lambda_{\mathrm{rel}}}$$

together with chart open immersions $j_{p,\lambda} : U_{p,\lambda} \hookrightarrow X^{\mathrm{desc}}_{\lambda_{\mathrm{rel}}}$. The structural-map equalities glue the chart maps into $X^{\mathrm{desc}}_{\lambda_{\mathrm{rel}}} \to S_{\lambda_{\mathrm{rel}}}$.

Definition 2.10.169 (Forward chartwise comparison cocone)**.** Uses: `def:nagata-finite-presentation-descended-scheme`,`def:nagata-relative-chart-transport-isomorphism`,`def:nagata-relative-overlap-transport`

Lean: `AlgebraicGeometry.Limits.finitePresentationForwardCocone`

Use the already defined chart maps $\epsilon_p = \mathrm{Spec}(\gamma_p)$ and $\delta_p = \mathrm{Spec}(\beta_p)$, with their two stored inverse equations. Let $i_p : U_p \hookrightarrow X$ be the original chart inclusion. On $W^p_{pq,\lambda,S}$ store the typed equality

$$i_p\epsilon_p\iota^p_{pq,S} = i_q\epsilon_q\iota^q_{qp,S}(\phi_{pq})_S : W^p_{pq,\lambda,S} \longrightarrow X.$$

Here $\iota^p_{pq,S}$ and $\iota^q_{qp,S}$ are the descended inclusions whose base changes are the transported opens. The equality is the stored $\epsilon_q\widetilde{\phi}_{pq} = \phi_{pq}\epsilon_p$, followed by the original chart-gluing equality, so the two sides have literally the same source and target. This finite family is the forward chartwise cocone.

Definition 2.10.170 (Inverse chartwise comparison cocone)**.** Uses: def:nagata-finite-presentation-descended-scheme,def:nagata-relative-chart-transport-isomorphism,def:nagata-relative-overlap-transport

Lean: AlgebraicGeometry.Limits.finitePresentationInverseCocone

On every original overlap W^p_{pq} store the typed equality

$$j_{p,\lambda,S}\delta_p\iota^p_{pq} = j_{q,\lambda,S}\delta_q\iota^q_{qp}\phi_{pq}\colon W^p_{pq} \longrightarrow X^{\mathrm{desc}}_{\lambda_{\mathrm{rel}},S}.$$

It follows by conjugating the descended overlap equality with the stored chart inverse maps. This finite family is the inverse chartwise cocone.

Definition 2.10.171 (Typed base-change comparison cocone)**.** Uses: def:nagata-finite-presentation-forward-cocone,def:nagata-finite-presentation-inverse-cocone

Lean: AlgebraicGeometry.Limits.finitePresentationComparisonCocone

Package the separately constructed forward and inverse chartwise cocones as the N24 comparison-cocone certificate.

Definition 2.10.172 (Finite-presentation descent construction)**.** Uses: def:nagata-finite-presentation-comparison-cocone

Lean: AlgebraicGeometry.Limits.finitePresentationDescentConstruction

Glue the maps $i_p\epsilon_p$ using the typed forward cocone and define the N24 global base-change comparison

$$\epsilon_X\colon X^{\mathrm{desc}}_{\lambda_{\mathrm{rel}}} \times_{S_{\lambda_{\mathrm{rel}}}} S \longrightarrow X.$$

Definition 2.10.173 (Inverse of the global base-change comparison)**.** Uses: def:nagata-finite-presentation-descent

Lean: AlgebraicGeometry.Limits.finitePresentationDescentInverse

Glue the maps $j_{p,\lambda,S}\delta_p$ using the typed inverse cocone and define $\delta_X\colon X \to X^{\mathrm{desc}}_{\lambda_{\mathrm{rel}}} \times_{S_{\lambda_{\mathrm{rel}}}} S$.

Lemma 2.10.174 (Left inverse for the global base-change comparison)**.** *Uses:* def:nagata-finite-presentation-descent, def:nagata-finite-presentation-descent-inverse

Lean: AlgebraicGeometry.Limits.finitePresentationComparison_leftInverse

The constructed maps satisfy $\epsilon_X\delta_X = \mathrm{id}_X$.

Proof. Uses: lemma:nagata-affine-fp-left-inverse

On every original chart U_p, the composite is $\epsilon_p\delta_p = \mathrm{id}_{U_p}$. These charts cover X, so source-local equality proves the global equation. □

Lemma 2.10.175 (Right inverse for the global base-change comparison)**.** *Uses:* def:nagata-finite-presentation-descent, def:nagata-finite-presentation-descent-inverse

Lean: AlgebraicGeometry.Limits.finitePresentationComparison_rightInverse

The constructed maps satisfy

$$\delta_X\epsilon_X = \mathrm{id}_{X^{\mathrm{desc}}_{\lambda_{\mathrm{rel}}} \times_{S_{\lambda_{\mathrm{rel}}}} S}.$$

Proof. Uses: lemma:nagata-affine-fp-right-inverse

On every base-changed descended chart, the composite is $\delta_p\epsilon_p = \mathrm{id}$. Those charts cover the source, so the local equations glue to the displayed identity. □

Lemma 2.10.176 (Property of finite-presentation descent, [1, Tag 01ZM(1)])**.** *Uses:* def:nagata-finite-presentation-descent, def:nagata-finite-presentation-descent-inverse

Lean: AlgebraicGeometry.Limits.finitePresentation_descends

The constructed $X^{\mathrm{desc}}_{\lambda_{\mathrm{rel}}} \to S_{\lambda_{\mathrm{rel}}}$ *is of finite presentation, and* ϵ_X *is an isomorphism over* S.

Proof. Uses: lemma:nagata-relative-gluing-datum-descends,lemma:nagata-relative-gluing-axioms-eventually,lemma:nagata-finite-presentation-comparison-left-inverse,lemma:nagata-finite-presentation-comparison-right-inverse

The valid-stage property permits gluing. Finitely many finitely presented affine charts cover the result and the overlaps are quasi-compact, so the structural morphism is of finite presentation. After base change to S, the datum becomes the selected datum R_X. The chartwise comparisons are isomorphisms and satisfy the stored comparison cocone. The two separate inverse lemmas make ϵ_X an isomorphism with the explicitly assembled inverse δ_X. □

2.10.4 Eventual separatedness of the descended model

Lemma 2.10.177 (Diagonal commutes with base change)**.** *Uses:*

Lean: `AlgebraicGeometry.diagonal_baseChange_iso`

For $X_\lambda \to S_\lambda$ and $S \to S_\lambda$, the base change of $\Delta_{X_\lambda/S_\lambda}$ is canonically identified with $\Delta_{X_\lambda \times_{S_\lambda} S/S}$.

Proof. Uses:

Base change the defining fibre square of the diagonal. Associativity of fibre products identifies the result with the map sending a point to its pair with itself, which is the diagonal of the base-changed morphism. □

Definition 2.10.178 (Affine stage for a prospective closed immersion)**.** Uses: `def:nagata-directed-finite-upper-bound`

Lean: `AlgebraicGeometry.Limits.closedImmersionAffineStage`

Let $g_\lambda \colon Z_\lambda \to Y_\lambda$ be of finite presentation between finite-presentation S_λ-schemes, and assume its base change to S is a closed immersion. Since that base change is affine, first choose, by Tag 01ZN, a stage μ_{aff} at which g is affine. On a selected finite affine cover $(Q_a)_a$ of Y_λ, the systems $Z_{a,\mu} = g_\mu^{-1}(Q_{a,\mu})$ have affine transitions and affine limits. Select the Tag 01Z6 affineness stage for each system and take their finite common upper bound as part of μ_{aff}. Thus above μ_{aff} write

$$g^{-1}(Q_{a,\mu}) = \mathrm{Spec}(C_{a,\mu}), \qquad Q_{a,\mu} = \mathrm{Spec}(B_{a,\mu}).$$

Store the finite cover, its cartesian transports, and all affineness certificates.

Definition 2.10.179 (Generator data for a prospective closed immersion)**.** Uses: `def:nagata-closed-immersion-affine-stage`

Lean: `AlgebraicGeometry.Limits.closedImmersionGeneratorData`

Select finite $B_{a,\mu_{\mathrm{aff}}}$-algebra generators $(z_{ab})_b$ of $C_{a,\mu_{\mathrm{aff}}}$. Such a list exists because an affine finite-presentation morphism gives a finitely presented coordinate algebra. The cartesian transport squares ensure that the images of this same list generate every later $C_{a,\mu}$ over $B_{a,\mu}$.

Definition 2.10.180 (Preimage and equality stages for closedness)**.** Uses: `def:nagata-closed-immersion-generator-data,def:nagata-directed-finite-upper-bound`

Lean: `AlgebraicGeometry.Limits.closedImmersionPreimageStage`

The limiting ring map has direction $B_{a,S} \to C_{a,S}$ and is surjective. Thus every $1 \otimes z_{ab}$ has a selected preimage $b_{ab} \in B_{a,S}$. Since $B_{a,S} = \varinjlim_{\mu \geq \mu_{\mathrm{aff}}} B_{a,\mu}$, choose stages carrying these finitely many preimages and further stages at which their images equal $1 \otimes z_{ab}$. Store each preimage representative and its equality certificate.

Definition 2.10.181 (Stage at which a finite-presentation map is closed)**.** Uses: `def:nagata-closed-immersion-preimage-stage,def:nagata-directed-finite-upper-bound`

Lean: `AlgebraicGeometry.Limits.closedImmersionStage`

Define $\mu_{\mathrm{cl}}(g)$ to be the selected upper bound of μ_{aff} and all finitely many preimage and equality stages. Transport the already selected affine cover, generator lists, preimages, and equality certificates to this common stage.

Lemma 2.10.182 (A finite-presentation closed immersion descends at the selected stage)**.** *Uses:* *def:nagata-closed-immersion-stage*

Lean: `AlgebraicGeometry.Limits.closedImmersion_eventually`

For every $\mu \geq \mu_{\mathrm{cl}}(g)$, the base change g_μ is a closed immersion.

Proof. Uses:

Above μ_{aff} all selected inverse images are affine. On each piece the chosen elements generate the target of the coordinate-ring map, while the stored preimages show that every generator lies in its image. Hence the map is surjective at $\mu_{\mathrm{cl}}(g)$ and remains surjective after further base change. The affine morphism is therefore a closed immersion over each Q_a, and closed immersions are local on the target. This supplies the missing affine-descent step and is exactly the construction of [1, Tag 01ZN, Tag 07RH, and Tag 01ZP]. □

Definition 2.10.183 (Separated stage of a descended finite-presentation model)**.** Uses: `def:nagata-finite-presentation-descent,def:nagata-closed-immersion-stage`

Lean: `AlgebraicGeometry.Limits.separatedDescentStage`

Let $X \to S$ be separated and of finite presentation, and start from its constructed finite-stage model $X^{\mathrm{desc}}_{\lambda_{\mathrm{rel}}} \to S_{\lambda_{\mathrm{rel}}}$. Apply Definition 2.10.181 to its diagonal, whose base change is identified with $\Delta_{X/S}$. Define

$$\lambda_{\mathrm{sep}}(X/S) = \mu_{\mathrm{cl}}(\Delta_{X^{\mathrm{desc}}_{\lambda_{\mathrm{rel}}}/S_{\lambda_{\mathrm{rel}}}}),$$

and let $X^{\mathrm{sep}}_{\lambda_{\mathrm{sep}}(X/S)}$ be the base change of the descended model to this stage.

Lemma 2.10.184 (Property of the selected separated descent stage, [1, Tag 01ZQ]). *Uses:* *def: nagata-separated-descent-stage*

Lean: *AlgebraicGeometry.Limits.separated_eventually*

The constructed morphism $X^{\mathrm{sep}}_{\lambda_{\mathrm{sep}}(X/S)} \to S_{\lambda_{\mathrm{sep}}(X/S)}$ is separated and of finite presentation, and its base change to S is canonically $X \to S$.

Proof. Uses: `lemma:nagata-finite-presentation-descends`, `lemma:nagata-diagonal-base-change`, `lemma:nagata-closed-immersion-descends`

The initial descended model is finite presentation and has base change X. The diagonal base-change lemma identifies the base change of its diagonal with the closed immersion $\Delta_{X/S}$. At the explicitly selected closed-immersion stage, the actual diagonal is closed, hence the model is separated. Finite presentation and the base-change identification persist under increasing the stage. □

2.11 Lean interfaces and implementation order

The basic bundled structure should expose all construction data rather than only a proposition.

```
structure Compactification {X S : Scheme} (f : X -> S) where
  Xbar : Scheme
  j : X -> Xbar
  p : Xbar -> S
  fac : j >> p = f
  isOpenImmersion_j : IsOpenImmersion j
  quasiCompact_j : QuasiCompact j
  isProper_p : IsProper p

def Compactifiable {X S : Scheme} (f : X -> S) : Prop :=
  Nonempty (Compactification f)
```

The implementation order forced by the dependency graph is:

1. compactifications, scheme-theoretic images, restriction to opens, and closed-subscheme inheritance;
2. admissible blow-ups, strict transforms, extension of centers, and domination by product ideals;
3. boundary separation and elimination of indeterminacy;
4. common modifications, graph extension, and separated gluing;
5. the dense-source valuative criterion and the two-open theorem;
6. affine compactification and the Noetherian theorem;
7. finite-presentation reduction, Noetherian approximation, and the final assembly.

Before any declaration receives a `\leanok` marker, the following points must be checked in Lean:

1. every displayed composite equality is stated using the actual chosen pullback and gluing morphisms;
2. every scheme-theoretic image is formed under the required quasi-compactness hypotheses;
3. restriction of a scheme-theoretic image to an open target is available as an equality or a specified isomorphism of closed subschemes;

4. every admissible blow-up carries a chosen lift and chosen isomorphism over the preserved open;
5. the boundary-separation blow-ups preserve V_i, not only U_i;
6. the common-modification proof uses properness of $X'_{12} \to X_1 \times_S X_2$ and separatedness of the new product over the old product; it does not incorrectly assume the projections $X_{12} \to X_i$ are proper in the general common-modification lemma;
7. the dense-source valuative criterion is invoked with uniqueness, not with existence alone;
8. the pair of valuation extensions factors scheme-theoretically through X_{12} by injectivity of $A \hookrightarrow K$;
9. the finite affine cover has dense total intersection, and the induction uses density of each partial overlap;
10. finite presentation, the base-change identification, and separatedness all descend at one common sufficiently large index.

2.12 Explicit final assembly

In this final section let S be quasi-compact and quasi-separated and let $f\colon X \to S$ be separated and of finite type. Every object used below is the output of a previously defined construction; no new existential choice is made inside the final theorem.

Definition 2.12.1 (Finite-presentation envelope used in the final assembly)**.** Uses: `def:nagata-finite-type-closed-in-fp`

Lean: `AlgebraicGeometry.NagataAssembly.fpEnvelope`

Define

$$\mathsf{E}_f = (X_f^{\mathrm{fp}}, i_f^{\mathrm{fp}}) = \mathsf{Env}_{\mathrm{fp}}(X/S).$$

Lemma 2.12.2 (Properties of the final finite-presentation envelope)**.** *Uses:* `def:nagata-final-fp-envelope`

Lean: `AlgebraicGeometry.NagataAssembly.fpEnvelope_spec`

The map $i_f^{\mathrm{fp}}\colon X \hookrightarrow X_f^{\mathrm{fp}}$ is a closed immersion over S, and $X_f^{\mathrm{fp}} \to S$ is separated and of finite presentation.

Proof. Uses: `prop:nagata-finite-type-closed-in-fp`

These are precisely the three properties established for the constructed envelope in Proposition 2.10.114. □

Definition 2.12.3 (Noetherian base system used in the final assembly)**.** Uses: `def:nagata-noetherian-inverse-system`

Lean: `AlgebraicGeometry.NagataAssembly.baseSystem`

Define

$$\mathsf{N}_f = \mathsf{N}(S) = (I_S, (S_\lambda)_\lambda, (t_{\mu\lambda})_{\lambda\le\mu})$$

and retain the constructed limit comparison $\kappa_S\colon S \to \varprojlim_\lambda S_\lambda$.

Lemma 2.12.4 (Properties of the final Noetherian base system)**.** *Uses:* `def:nagata-final-noetherian-base-system`

Lean: `AlgebraicGeometry.NagataAssembly.baseSystem_spec`

Every S_λ is Noetherian and of finite type over $\mathbf{Z}$, every transition map is affine, and κ_S is an isomorphism.

Proof. Uses: `prop:nagata-noetherian-inverse-system`

Apply Proposition 2.10.79 to the explicitly selected system $\mathsf{N}(S)$. □

Definition 2.12.5 (Separated descended model used in the final assembly). Uses: def:nagata-final-fp-envelope,def:nagata-final-noetherian-base-system,def:nagata-separated-descent-stage

Lean: AlgebraicGeometry.NagataAssembly.descendedModel

Apply the separated-descent construction to $X_f^{\mathrm{fp}} \to S$ relative to N_f. Define

$$\lambda_f = \lambda_{\mathrm{sep}}(X_f^{\mathrm{fp}}/S), \qquad X_{f,\lambda_f}^{\mathrm{fp}} = (X_f^{\mathrm{fp}})_{\lambda_f}^{\mathrm{sep}},$$

and let

$$\epsilon_f \colon X_{f,\lambda_f}^{\mathrm{fp}} \times_{S_{\lambda_f}} S \longrightarrow X_f^{\mathrm{fp}}$$

be the constructed base-change comparison.

Lemma 2.12.6 (Properties of the final descended model). Uses: def:nagata-final-descended-model

Lean: AlgebraicGeometry.NagataAssembly.descendedModel_spec

The morphism $X_{f,\lambda_f}^{\mathrm{fp}} \to S_{\lambda_f}$ is separated and of finite presentation, S_{λ_f} is Noetherian, and ϵ_f is an isomorphism over S.

Proof. Uses: lemma:nagata-separated-eventually,lemma:nagata-final-noetherian-base-system

The first and third assertions are the property of the constructed separated descent stage. The selected base stage belongs to N_f, so it is Noetherian by Lemma 2.12.4. □

Definition 2.12.7 (Noetherian compactification used in the final assembly). Uses: def:nagata-final-descended-model,def:nagata-noetherian-compactification

Lean: AlgebraicGeometry.NagataAssembly.noetherianCompactification

Define

$$C_{f,\lambda_f} = \mathsf{C}_{\mathrm{Noeth}}(X_{f,\lambda_f}^{\mathrm{fp}}/S_{\lambda_f}).$$

Lemma 2.12.8 (Property of the selected Noetherian compactification). Uses: def:nagata-final-noetherian-compactification

Lean: AlgebraicGeometry.NagataAssembly.noetherianCompactification_spec

The constructed C_{f,λ_f} is a compactification of $X_{f,\lambda_f}^{\mathrm{fp}} \to S_{\lambda_f}$.

Proof. Uses: lemma:nagata-final-descended-model,thm:nagata-noetherian

The base is Noetherian and the descended morphism is separated and of finite type. Therefore Theorem 2.9.61 applies to the explicitly selected recursive construction and gives the assertion. □

Definition 2.12.9 (Base-changed compactification of the finite-presentation envelope). Uses: def:nagata-compactification-base-change,def:nagata-final-noetherian-compactification,def:nagata-final-descended-model

Lean: AlgebraicGeometry.NagataAssembly.baseChangedCompactification

Base change C_{f,λ_f} along $S \to S_{\lambda_f}$ and transport its source across ϵ_f. Define the resulting factorization to be

$$C_f^{\mathrm{fp}} = \big(X_f^{\mathrm{fp}} \hookrightarrow \overline{X}_f^{\mathrm{fp}} \longrightarrow S\big).$$

Lemma 2.12.10 (Property of the base-changed compactification). Uses: def:nagata-final-basechanged-compactification

Lean: AlgebraicGeometry.NagataAssembly.baseChangedCompactification_spec

The constructed C_f^{fp} is a compactification of $X_f^{\mathrm{fp}} \to S$.

Proof. Uses: lemma:nagata-final-noetherian-compactification,lemma:nagata-compactification-base-change,lemma:nagata-final-descended-model

Base change preserves every compactification field by Lemma 2.1.15. Transporting the source across the isomorphism ϵ_f preserves the open-immersion, quasi-compactness, properness, and factorization equalities. □

Definition 2.12.11 (Final Nagata compactification candidate). Uses: def:nagata-closed-subscheme-compactification,def:nagata-final-fp-envelope,def:nagata-final-basechanged-compactification

Lean: AlgebraicGeometry.nagataCompactificationConstruction

Define

$$\mathsf{C}_{\mathrm{Nag}}(f) = \mathsf{Closed}(i_f^{\mathrm{fp}}, C_f^{\mathrm{fp}}) = \big(X \xrightarrow{j_f} \overline{X}_f \xrightarrow{p_f} S\big).$$

Thus $\overline{X}_f$ is the scheme-theoretic image of $X \to X_f^{\mathrm{fp}} \to \overline{X}_f^{\mathrm{fp}}$, while j_f and p_f are the already constructed scheme-image factor and restricted proper map.

Lemma 2.12.12 (The final ambient morphism is proper). *Uses:* def:nagata-final-compactification

Lean: AlgebraicGeometry.nagataCompactificationConstruction_isProper

The constructed morphism $p_f\colon \overline{X}_f \to S$ is proper.

Proof. Uses: lemma:nagata-final-basechanged-compactification,prop:nagata-closed-subscheme-compactifiable

The ambient scheme $\overline{X}_f$ is closed in the proper ambient scheme of C_f^{fp}. A closed immersion followed by a proper morphism is proper; this is the properness part of the closed-subscheme construction. □

Lemma 2.12.13 (The final source map is a quasi-compact open immersion). *Uses:* def:nagata-final-compactification

Lean: AlgebraicGeometry.nagataCompactificationConstruction_isOpenImmersion

The constructed map $j_f\colon X \to \overline{X}_f$ is a quasi-compact open immersion.

Proof. Uses: lemma:nagata-final-fp-envelope,lemma:nagata-final-basechanged-compactification,prop:nagata-closed-subscheme-compactifiable

Restrict the scheme-theoretic image defining $\overline{X}_f$ to the open X_f^{fp} in the ambient compactification. It is the scheme-theoretic image of the closed immersion $X \hookrightarrow X_f^{\mathrm{fp}}$, hence equals X. Therefore j_f is the base change of the quasi-compact open immersion in C_f^{fp}. □

Lemma 2.12.14 (Factorization equality for the final candidate). *Uses:* def:nagata-final-compactification

Lean: AlgebraicGeometry.nagataCompactificationConstruction_fac

The constructed maps satisfy $j_f; p_f = f$.

Proof. Uses:

The map p_f is the restriction of the ambient structural morphism, and j_f is the factor of the original composite through its scheme-theoretic image. The defining factorization equality for the scheme-image map, followed by the equality for the finite-presentation envelope, gives $j_f; p_f = f$. □

Theorem 2.12.15 (Nagata compactification, [1, Tag 0F3T]). *Uses:* def:nagata-compactification, def:nagata-final-compactification

Lean: AlgebraicGeometry.nagataCompactification

The explicitly constructed factorization $\mathsf{C}_{\mathrm{Nag}}(f)$ is a compactification of f. In particular, every separated finite type morphism to a quasi-compact and quasi-separated scheme is compactifiable.

Proof. Uses: lemma:nagata-final-ambient-proper,lemma:nagata-final-open-immersion,lemma:nagata-final-factorization,def:nagata-compactification

The first map is a quasi-compact open immersion by Lemma 2.12.13; the second is proper by Lemma 2.12.12; and their composite is f by Lemma 2.12.14. These are exactly the fields of Definition 2.1.1. Hence the named construction is a compactification, and its existence witnesses compactifiability. □

Chapter 3

Zeta functions and finite Frobenius point counting

Unless a later scope paragraph explicitly strengthens the hypotheses, let R be an arbitrary commutative ring. We write $R[[T]]$ for the formal power-series ring, $[T^n]A$ for the coefficient of T^n in A, and $c(A) = [T^0]A$ for its constant coefficient. No topology or analytic convergence is used: every construction below is coefficientwise finite. The coefficient-ring hypothesis of a declaration is the most recent explicit scope paragraph above it; each reset below is therefore part of the formal statement envelope and is not merely expository prose.

3.1 Coefficientwise exponential and logarithm

Definition 3.1.1 (Positive-order formal series)**.** Uses:

Lean: `Weil.PositiveSeries`

A series $U \in R[[T]]$ is a *positive-order series* if $c(U) = 0$. We write $TR[[T]]$ for the subtype of such series.

Lemma 3.1.2 (A positive-order series admits formal substitution)**.** *Uses:* *def:weil-positive-series*

Lean: *Weil.PositiveSeries.hasSubst*

If $U \in TR[[T]]$, then every series $A \in R[[X]]$ may be substituted at $X = U$ in the sense of *PowerSeries.subst*.

Proof. Uses: `def:weil-positive-series`

The hypothesis on U is exactly $c(U) = 0$. Apply `PowerSeries.HasSubst.of_constantCoeff_zero'` with $S = R$ and $a = U$. Its conclusion is `PowerSeries.HasSubst U`, which is the required substitution certificate. □

Rational coefficient scope. For the remaining declarations of this section, strengthen the standing hypothesis: R is a commutative $\mathbf{Q}$-algebra. This is the precise scope in which division by positive integers, formal exponential and logarithm, and the logarithmic-derivative uniqueness arguments are used.

Definition 3.1.3 (Coefficientwise formal exponential)**.** Uses: `def:weil-positive-series`

Lean: `Weil.formalExp`

For $U \in TR[[T]]$, define

$$\mathrm{Exp}(U) := \big(\texttt{PowerSeries.exp}\ R\big)\big|_{X=U} \in R[[T]].$$

Thus the fixed mathlib series $\sum_{r\geq 0} X^r/r!$ is substituted at U; this is not notation for an analytic exponential.

Lemma 3.1.4 (Finite coefficient formula for the formal exponential)**.** *Uses:* *def:weil-positive-series, def:weil-formal-exp*

Lean: *Weil.formalExp_coeff*

For $U \in TR[[T]]$ and $N \geq 0$,

$$[T^N]\operatorname{Exp}(U) = \sum_{0 \leq r \leq N} \frac{[T^N]U^r}{r!}.$$

In particular the coefficient is a finite sum.

Proof. Uses: def:weil-positive-series,def:weil-formal-exp,lemma:weil-positive-series-has-subst
Apply `PowerSeries.coeff_subst'` to the substitution certificate from Lemma 3.1.2, with outer series `PowerSeries.exp` R, inner series U, and coefficient N. The resulting summand at r is

$$\texttt{PowerSeries.coeff}\ r\,(\texttt{PowerSeries.exp}\ R)\,[T^N]U^r.$$

By `PowerSeries.coeff_exp`, the first factor is $1/r!$. By `PowerSeries.HasSubst.eventually_coeff_pow_eq_zero`, applied with $f = U$ and $n = N$, only finitely many powers can contribute. The explicit bound is $r \leq N$: because $c(U) = 0$, the convolution formula `PowerSeries.coeff_pow` shows that $[T^N]U^r = 0$ for $r > N$: every nonzero convolution term would require r positive indices summing to N. Removing these zero summands gives exactly the displayed finite sum. □

Lemma 3.1.5 (Constant coefficient of the formal exponential)**.** Uses: def:weil-formal-exp
Lean: `Weil.formalExp_constantCoeff`
For every positive-order U, one has $c(\operatorname{Exp}(U)) = 1$.

Proof. Uses: def:weil-formal-exp,lemma:weil-formal-exp-coeff
Put $N = 0$ in Lemma 3.1.4. Only $r = 0$ remains, and $U^0 = 1$ has constant coefficient 1. Hence $c(\operatorname{Exp}(U)) = 1/0! = 1$. □

Definition 3.1.6 (Coefficientwise formal logarithm)**.** Uses: def:weil-positive-series
Lean: `Weil.formalLog`
For a series $A \in R[[T]]$ with $c(A) = 1$, its reduced part $A - 1$ has constant coefficient zero. Define

$$\operatorname{Log}(A) := \big(\texttt{PowerSeries.log}\ R\big)\big|_{X=A-1} \in R[[T]].$$

Here `PowerSeries.log` is the fixed mathlib series $\sum_{r\geq 1}(-1)^{r+1}X^r/r$.

Lemma 3.1.7 (Finite coefficient formula for the formal logarithm)**.** Uses: def:weil-formal-log
Lean: `Weil.formalLog_coeff`
If $c(A) = 1$, then $[T^0]\operatorname{Log}(A) = 0$, and for $N \geq 1$,

$$[T^N]\operatorname{Log}(A) = \sum_{1 \leq r \leq N} \frac{(-1)^{r+1}}{r}[T^N](A-1)^r.$$

Proof. Uses: def:weil-positive-series,def:weil-formal-log,lemma:weil-positive-series-has-subst
The equality $c(A) = 1$ gives $c(A-1) = 0$, so Lemma 3.1.2 supplies the substitution certificate for $A - 1$. Apply `PowerSeries.coeff_subst'` with outer series `PowerSeries.log`. The exact coefficient theorem `PowerSeries.coeff_log` gives zero at $r = 0$ and $(-1)^{r+1}/r$ at $r > 0$. Since $c(A-1) = 0$, `PowerSeries.coeff_pow` makes the coefficient of $(A-1)^r$ vanish when $r > N$. This leaves precisely the stated finite sum. At $N = 0$, every $r > 0$ term has zero constant coefficient, so the result is zero. □

Definition 3.1.8 (Formal derivative)**.** Uses:
Lean: `Weil.formalDerivative`
For $A = \sum_{n\geq 0} a_n T^n$, define

$$DA := \sum_{n \geq 0}(n+1)a_{n+1}T^n.$$

This is the map `PowerSeries.derivative R`.

Lemma 3.1.9 (Coefficient formula and product rule for the formal derivative)**.** Uses: def:weil-formal-derivative
Lean: `Weil.formalDerivative_rules`
For $A, B \in R[[T]]$ and $n \geq 0$,

$$[T^n]DA = (n+1)[T^{n+1}]A, \qquad D(AB) = (DA)B + A(DB).$$

Proof. Uses: `def:weil-formal-derivative`
The coefficient equality is `PowerSeries.coeff_derivative` with coefficient index n and series A, after commuting the two scalar factors. The second equality is the Leibniz field of the derivation `PowerSeries.derivative R`; equivalently it is `PowerSeries.derivativeFun_mul` followed by `smul_eq_mul` and commutativity of $R[[T]]$. □

Lemma 3.1.10 (The formal derivative preserves addition). *Uses:* `def:weil-formal-derivative`

Lean: `Weil.formalDerivative_add`
For $A, B \in R[[T]]$,

$$D(A+B) = DA + DB.$$

Proof. Uses: `def:weil-formal-derivative`
Fix $n \geq 0$. Unfolding the coefficient formula for D on the two sides gives $(n+1)([T^{n+1}]A + [T^{n+1}]B)$ and $(n+1)[T^{n+1}]A + (n+1)[T^{n+1}]B$. Distributivity identifies these coefficients. Apply `PowerSeries.ext`. □

Lemma 3.1.11 (Derivative and constant coefficient determine a series). *Uses:* `def:weil-formal-derivative`

Lean: `Weil.formalDerivative_ext`
If $DA = DB$ and $c(A) = c(B)$, then $A = B$.

Proof. Uses: `def:weil-formal-derivative`
Apply `PowerSeries.derivative.ext` with the present A, B. Its typeclass hypothesis `IsAddTorsionFree R` follows from the $\mathbf{Q}$-algebra structure: multiplication by a positive integer is multiplication by a nonzero unit coming from $\mathbf{Q}$. The two explicit arguments of that theorem are exactly $DA = DB$ and $c(A) = c(B)$. □

Lemma 3.1.12 (A normalized series with zero derivative is one). *Uses:* `def:weil-formal-derivative`

Lean: `Weil.eq_one_of_derivative_eq_zero_of_constantCoeff_eq_one`
If $DA = 0$ and $c(A) = 1$, then $A = 1$.

Proof. Uses: `def:weil-formal-derivative,lemma:weil-formal-derivative-ext`
The definition of D gives $D(1) = 0$: every coefficient $(n+1)[T^{n+1}]1$ vanishes. Thus $DA = D(1)$, while the second hypothesis is $c(A) = c(1)$. Instantiate Lemma 3.1.11 with the pair $(A, 1)$. □

Definition 3.1.13 (Geometric inverse of a one-series). Uses: `def:weil-positive-series`

Lean: `Weil.geometricInverse`
If $c(A) = 1$, put $U = A - 1$ and define

$$\mathrm{GeomInv}(A) := \big(\texttt{PowerSeries.mk}\ (\lambda n, 1)\big)\big|_{X=-U}.$$

Coefficientwise this is the formal geometric sum $\sum_{r\geq 0}(-U)^r$.

Lemma 3.1.14 (The geometric inverse is a two-sided inverse). *Uses:* `def:weil-geometric-inverse`

Lean: `Weil.geometricInverse_mul`
If $c(A) = 1$, then

$$A\,\mathrm{GeomInv}(A) = 1 = \mathrm{GeomInv}(A)A.$$

Proof. Uses: `def:weil-positive-series,def:weil-geometric-inverse,lemma:weil-positive-series-has-subst`
Write $U = A - 1$. Since $c(U) = 0$, the inner series $-U$ has zero constant coefficient and Lemma 3.1.2 applies. Start from the exact mathlib identity

$$\texttt{PowerSeries.mk_one_mul_one_sub_eq_one}\ R: \quad (\texttt{PowerSeries.mk}\ 1)(1-X) = 1.$$

Apply the algebra homomorphism `PowerSeries.substAlgHom` attached to $-U$. The first factor becomes $\mathrm{GeomInv}(A)$, and the second becomes $1 - (-U) = 1 + U = A$. Thus $\mathrm{GeomInv}(A)A = 1$. Commutativity gives the other order. □

Lemma 3.1.15 (Constant coefficient of the geometric inverse). *Uses:* `def:weil-geometric-inverse`

Lean: `Weil.formalGeometricInverse_constantCoeff`
If $c(A) = 1$, then $c(\mathrm{GeomInv}(A)) = 1$.

Proof. Uses: `def:weil-geometric-inverse`
Put $U = A - 1$. In degree zero, `PowerSeries.coeff_subst'` for the substitution at $-U$ has only the $r = 0$ contribution, because $c(-U) = 0$. The outer geometric series has coefficient one in degree zero, and $(-U)^0 = 1$. Therefore the resulting constant coefficient is one. □

Lemma 3.1.16 (Derivative of the geometric inverse). *Uses:* *def:weil-geometric-inverse, def:weil-formal-derivative*

Lean: *Weil.geometricInverse_derivative*
If $c(A) = 1$, then

$$D\,\mathrm{GeomInv}(A) = -\,\mathrm{GeomInv}(A)^2\,DA.$$

Proof. Uses: `lemma:weil-formal-derivative-rules,lemma:weil-geometric-inverse-mul`
Differentiate the equality $A\,\mathrm{GeomInv}(A) = 1$. The product rule gives

$$(DA)\,\mathrm{GeomInv}(A) + A\,D\,\mathrm{GeomInv}(A) = 0.$$

Multiply by $\mathrm{GeomInv}(A)$ and use $A\,\mathrm{GeomInv}(A) = 1$. Moving the first term to the other side and commuting the factors yields the displayed identity. □

Definition 3.1.17 (Geometric comparison series). Uses: `def:weil-geometric-inverse`

Lean: `Weil.geometricComparisonSeries`
For $X, Y \in R[[T]]$ with $c(Y) = 1$, define

$$\mathrm{Cmp}(X, Y) := X\,\mathrm{GeomInv}(Y).$$

Lemma 3.1.18 (Derivative of the geometric comparison series). *Uses:* *def:weil-geometric-comparison-series, def:weil-formal-derivative*

Lean: *Weil.geometricComparisonSeries_derivative*
If $c(Y) = 1$, then

$$D\,\mathrm{Cmp}(X, Y) = (DX)\,\mathrm{GeomInv}(Y) - X\,\mathrm{GeomInv}(Y)^2(DY).$$

Proof. Uses: `def:weil-geometric-comparison-series,lemma:weil-formal-derivative-rules,lemma:weil-geometric-inverse-derivative`
Apply the product rule to $X\,\mathrm{GeomInv}(Y)$. Replace the derivative of the second factor by $-\,\mathrm{GeomInv}(Y)^2 DY$ using Lemma 3.1.16. Distribute the multiplication by X over the minus sign; this is the displayed formula. □

Lemma 3.1.19 (Constant coefficient of a normalized comparison series). *Uses:* *def:weil-geometric-comparison-series*

Lean: *Weil.geometricComparisonSeries_constantCoeff*
If $c(X) = c(Y) = 1$, then $c(\mathrm{Cmp}(X, Y)) = 1$.

Proof. Uses: `def:weil-geometric-comparison-series,lemma:weil-formal-geometric-inverse-constant`
The degree-zero instance of `PowerSeries.coeff_mul` says that the constant coefficient of a product is the product of the two constant coefficients. These are 1 and 1 by the hypothesis on X and Lemma 3.1.15 applied to Y. □

Lemma 3.1.20 (A common logarithmic derivative kills the comparison). *Uses:* *def:weil-geometric-comparison-series, def:weil-formal-derivative*

Lean: *Weil.geometricComparisonSeries_derivative_eq_zero*
Let $H, X, Y \in R[[T]]$ with $c(Y) = 1$. If $DX = HX$ and $DY = HY$, then

$$D\,\mathrm{Cmp}(X, Y) = 0.$$

Proof. Uses: `lemma:weil-geometric-comparison-derivative,lemma:weil-geometric-inverse-mul`
Write $G = \mathrm{GeomInv}(Y)$. Substitute $DX = HX$ and $DY = HY$ into Lemma 3.1.18; it gives

$$HXG - XG^2HY.$$

Commutativity and associativity rewrite the second term as $HXG(GY)$. Lemma 3.1.14 gives $GY = 1$, so the two terms are equal and their difference is zero. □

Lemma 3.1.21 (A normalized common-ODE comparison is one). *Uses:* def:weil-geometric-comparison-series, def:weil-formal-derivative

Lean: WeilGeometricComparisonSeries_eq_one

Let $H, X, Y \in R[[T]]$ with $c(X) = c(Y) = 1$. If $DX = HX$ and $DY = HY$, then $\mathrm{Cmp}(X, Y) = 1$.

Proof. Uses: lemma:weil-geometric-comparison-derivative-zero, lemma:weil-geometric-comparison-constant, lemma:weil-formal-derivative-zero-constant-one

The first cited lemma gives $D\,\mathrm{Cmp}(X, Y) = 0$, and the second gives $c(\mathrm{Cmp}(X, Y)) = 1$. Instantiate Lemma 3.1.12 with $A = \mathrm{Cmp}(X, Y)$. □

Lemma 3.1.22 (Uniqueness for normalized formal logarithmic ODEs). *Uses:* def:weil-formal-derivative

Lean: Weil.eq_of_same_logarithmicDerivative

Let $H, X, Y \in R[[T]]$ with $c(X) = c(Y) = 1$. If $DX = HX$ and $DY = HY$, then $X = Y$.

Proof. Uses: def:weil-geometric-comparison-series, lemma:weil-geometric-comparison-eq-one, lemma:weil-geometric-inverse-mul

The comparison lemma gives $X\,\mathrm{GeomInv}(Y) = 1$. Multiply this equality on the right by Y. Associativity and $\mathrm{GeomInv}(Y)Y = 1$ reduce the left side to X and the right side to Y. □

Lemma 3.1.23 (Chain rule for the formal exponential). *Uses:* def:weil-formal-exp, def:weil-formal-derivative

Lean: Weil.formalExp_derivative

For $U \in TR[[T]]$,

$$D\,\mathrm{Exp}(U) = (DU)\,\mathrm{Exp}(U).$$

Proof. Uses: def:weil-formal-exp, lemma:weil-positive-series-has-subst

Apply `PowerSeries.derivative_subst` with outer series `PowerSeries.exp` R, inner series U, and the certificate from Lemma 3.1.2. Replace the derivative of the outer series using `PowerSeries.derivative_exp`. The substituted outer derivative is then $\mathrm{Exp}(U)$. The chain rule reads $D\,\mathrm{Exp}(U) = \mathrm{Exp}(U)DU$; commute the factors. □

Lemma 3.1.24 (Chain rule for the formal logarithm). *Uses:* def:weil-formal-log, def:weil-geometric-inverse, def:weil-formal-derivative

Lean: Weil.formalLog_derivative

If $c(A) = 1$, then

$$D\,\mathrm{Log}(A) = (DA)\,\mathrm{GeomInv}(A).$$

Proof. Uses: def:weil-formal-log, def:weil-geometric-inverse, lemma:weil-positive-series-has-subst

Put $U = A - 1$. Apply `PowerSeries.derivative_subst` to the outer series `PowerSeries.log` and the inner series U. The exact theorem `PowerSeries.deriv_log` identifies the outer derivative with the series whose r-th coefficient is $(-1)^r$. After substitution at U, this becomes $\sum_{r\ge 0}(-U)^r = \mathrm{GeomInv}(A)$: applying `PowerSeries.coeff_subst'` on both sides gives the same finite sum $\sum_r (-1)^r [T^N]U^r$ in each degree N. Finally, $DU = DA$ because the derivative of the constant series 1 is zero. Substitution into the chain rule gives the asserted formula. □

Lemma 3.1.25 (Derivative of the exponential of a sum). *Uses:* def:weil-formal-exp, def:weil-formal-derivative

Lean: Weil.formalExp_sum_derivative

For positive-order series U, V,

$$D\,\mathrm{Exp}(U + V) = (DU + DV)\,\mathrm{Exp}(U + V).$$

Proof. Uses: lemma:weil-formal-exp-derivative, lemma:weil-formal-derivative-add

Apply Lemma 3.1.23 to the positive-order series $U + V$. Replace $D(U + V)$ by $DU + DV$ using Lemma 3.1.10. □

Lemma 3.1.26 (Derivative of a product of two formal exponentials). *Uses:* def:weil-formal-exp, def:weil-formal-derivative

Lean: Weil.formalExp_product_derivative

For positive-order series U, V,

$$D\big(\mathrm{Exp}(U)\,\mathrm{Exp}(V)\big) = (DU + DV)\,\mathrm{Exp}(U)\,\mathrm{Exp}(V).$$

Proof. Uses: lemma:weil-formal-exp-derivative,lemma:weil-formal-derivative-rules
The product rule makes the left side

$$(DU)\operatorname{Exp}(U)\operatorname{Exp}(V)+\operatorname{Exp}(U)(DV)\operatorname{Exp}(V).$$

Commutativity moves DV to the front of the second summand, and distributivity factors the common product $\operatorname{Exp}(U)\operatorname{Exp}(V)$. □

Lemma 3.1.27 (The formal exponential converts sums to products). *Uses:* def:weil-formal-exp

Lean: Weil.formalExp_add
For positive-order series U, V,

$$\operatorname{Exp}(U+V)=\operatorname{Exp}(U)\operatorname{Exp}(V).$$

Proof. Uses: lemma:weil-formal-exp-constant,lemma:weil-formal-exp-sum-derivative,lemma:weil-formal-exp-product-derivative,lemma:weil-common-logarithmic-derivative-unique
Set

$$X=\operatorname{Exp}(U+V),\quad Y=\operatorname{Exp}(U)\operatorname{Exp}(V),\quad H=DU+DV.$$

The two preceding derivative lemmas give $DX=HX$ and $DY=HY$. Lemma 3.1.5 gives $c(X)=1$ and gives constant coefficient one for both factors of Y; the degree-zero coefficient product is therefore $c(Y)=1$. Apply Lemma 3.1.22. □

Lemma 3.1.28 (Logarithm after exponential). *Uses:* def:weil-formal-exp, def:weil-formal-log

Lean: Weil.formalLog_formalExp
For every $U\in TR[[T]]$,

$$\operatorname{Log}(\operatorname{Exp}(U))=U.$$

Proof. Uses: lemma:weil-formal-exp-constant,lemma:weil-formal-exp-derivative,lemma:weil-formal-log-derivative,lemma:weil-geometric-inverse-mul,lemma:weil-formal-derivative-ext,lemma:weil-formal-log-coeff
Put $A=\operatorname{Exp}(U)$; its constant coefficient is one. The two chain rules and the geometric-inverse identity give

$$D\operatorname{Log}(A)=(DA)\operatorname{GeomInv}(A)=(DU)A\operatorname{GeomInv}(A)=DU.$$

Lemma 3.1.7 gives constant coefficient zero for $\operatorname{Log}(A)$, and $c(U)=0$ by hypothesis. Apply Lemma 3.1.11. □

Lemma 3.1.29 (Derivative of the exponential of a formal logarithm). *Uses:* def:weil-formal-exp, def:weil-formal-log, def:weil-geometric-inverse, def:weil-formal-derivative

Lean: Weil.formalExp_logarithm_derivative
If $c(A)=1$, then

$$D\operatorname{Exp}(\operatorname{Log}(A))=\big((DA)\operatorname{GeomInv}(A)\big)\operatorname{Exp}(\operatorname{Log}(A)).$$

Proof. Uses: lemma:weil-formal-log-coeff,lemma:weil-formal-exp-derivative,lemma:weil-formal-log-derivative
Lemma 3.1.7 supplies $c(\operatorname{Log}(A))=0$, so the exponential derivative lemma applies to $\operatorname{Log}(A)$. It gives $D\operatorname{Exp}(\operatorname{Log}(A))=(D\operatorname{Log}(A))\operatorname{Exp}(\operatorname{Log}(A))$. Replace the first factor using Lemma 3.1.24. □

Lemma 3.1.30 (A one-series satisfies its logarithmic ODE). *Uses:* def:weil-geometric-inverse, def:weil-formal-derivative

Lean: Weil.oneSeries_logarithmicODE
If $c(A)=1$, then

$$DA=\big((DA)\operatorname{GeomInv}(A)\big)A.$$

Proof. Uses: lemma:weil-geometric-inverse-mul
Commute and associate the factors on the right to obtain $(DA)(\operatorname{GeomInv}(A)A)$. Replace the parenthesized factor by 1 using Lemma 3.1.14. □

Lemma 3.1.31 (Exponential after logarithm). *Uses:* def:weil-formal-exp, def:weil-formal-log

Lean: Weil.formalExp_formalLog
If $c(A)=1$, then

$$\operatorname{Exp}(\operatorname{Log}(A))=A.$$

Proof. Uses: lemma:weil-formal-log-coeff,lemma:weil-formal-exp-constant,lemma:weil-formal-exp-logarithm-derivative,lemma:weil-one-series-logarithmic-ode,lemma:weil-common-logarithmic-derivative-unique

Put

$$X = \operatorname{Exp}(\operatorname{Log}(A)), \quad Y = A, \quad H = (DA)\operatorname{GeomInv}(A).$$

The two preceding ODE lemmas give $DX = HX$ and $DY = HY$. Lemma 3.1.7 gives $c(\operatorname{Log}(A)) = 0$, so Lemma 3.1.5 gives $c(X) = 1$; the hypothesis gives $c(Y) = 1$. Apply Lemma 3.1.22. □

Proposition 3.1.32 (Formal exponential and logarithm are inverse)**.** Uses: def:weil-positive-series, def:weil-formal-exp, def:weil-formal-log

Lean: Weil.formalExpLogEquiv

Formal exponential and logarithm are inverse bijections

$$TR[[T]] \underset{\operatorname{Log}}{\overset{\operatorname{Exp}}{\rightleftarrows}} \{A \in R[[T]] \mid c(A) = 1\}.$$

Proof. Uses: lemma:weil-formal-exp-constant,lemma:weil-formal-log-coeff,lemma:weil-formal-log-exp,lemma:weil-formal-exp-log

Lemma 3.1.5 shows that exponential lands in the right-hand subtype, and Lemma 3.1.7 shows that logarithm lands in the left-hand subtype. The two composite maps are the identity by Lemmas 3.1.28 and 3.1.31. □

Lemma 3.1.33 (Formal logarithm converts a product to a sum)**.** Uses: def:weil-formal-exp, def:weil-formal-log

Lean: Weil.formalLog_mul

If $c(A) = c(B) = 1$, then

$$\operatorname{Log}(AB) = \operatorname{Log}(A) + \operatorname{Log}(B).$$

Proof. Uses: lemma:weil-formal-log-coeff,lemma:weil-formal-exp-add,lemma:weil-formal-exp-log,lemma:weil-formal-log-exp

Both logarithms on the right have zero constant coefficient. Apply Exp to their sum and use Lemma 3.1.27; the result is

$$\operatorname{Exp}(\operatorname{Log}(A))\operatorname{Exp}(\operatorname{Log}(B)) = AB.$$

Apply Log to this equality. The left side becomes the sum by Lemma 3.1.28, while the right side becomes $\operatorname{Log}(AB)$. This gives the stated equality. □

3.2 Locally finite sums and products

Generic coefficient-ring scope. Reset R to an arbitrary commutative ring. Agreement through a degree, positive-order locally finite families, active finsets, truncated products, and coefficientwise locally finite products below use only this hypothesis. In particular, all of these Definitions and their product-compatibility proofs are available for $R = \mathbf{Z}$; no $\mathbf{Q}$-algebra structure is present or used in this scope.

Definition 3.2.1 (Agreement through a degree)**.** Uses:

Lean: Weil.PowerSeries.AgreeThrough

For $A, B \in R[[T]]$ and $N \in \mathbf{N}$, write $A \equiv_N B$ if $[T^j]A = [T^j]B$ for every $0 \le j \le N$.

Lemma 3.2.2 (Agreement is preserved by addition and multiplication)**.** Uses: def:weil-series-agree-through

Lean: Weil.PowerSeries.AgreeThrough.add_mul

If $A \equiv_N A'$ and $B \equiv_N B'$, then $A + B \equiv_N A' + B'$ and $AB \equiv_N A'B'$.

Proof. Uses: def:weil-series-agree-through

The addition assertion follows coefficient by coefficient. For the product assertion, fix $j \le N$ and apply PowerSeries.coeff_mul to both products. Every pair (r, s) in Finset.antidiagonal j satisfies $r, s \le j \le N$, so the two hypotheses identify both factors in every summand. The finite sums are therefore equal. □

Lemma 3.2.3 (Agreement is preserved by subtraction). *Uses:* def:weil-series-agree-through

Lean: Weil.PowerSeries.AgreeThrough.sub

If $A \equiv_N A'$ and $B \equiv_N B'$, then

$$A - B \equiv_N A' - B'.$$

Proof. Uses: def:weil-series-agree-through

Fix $j \le N$. The coefficient of $A - B$ in degree j is $[T^j]A - [T^j]B$, and the corresponding coefficient of $A' - B'$ is $[T^j]A' - [T^j]B'$. Substitute the two agreement hypotheses. □

Lemma 3.2.4 (Agreement is preserved by natural powers). *Uses:* def:weil-series-agree-through

Lean: Weil.PowerSeries.AgreeThrough.pow

If $A \equiv_N B$, then for every $r \in \mathbf{N}$,

$$A^r \equiv_N B^r.$$

Proof. Uses: lemma:weil-series-agree-through-operations

Induct on r. At $r = 0$, both powers are 1. At the successor step, rewrite $A^{r+1} = A^r A$ and $B^{r+1} = B^r B$, and apply the multiplicative agreement lemma to the induction hypothesis and $A \equiv_N B$. □

Definition 3.2.5 (T-adically locally finite family). Uses: def:weil-positive-series, def:weil-series-agree-through

Lean: Weil.TAdicallyLocallyFiniteFamily

For a commutative ring R, define a T-adically locally finite family to be a record with four fields: an index type I; a raw component map $u : I \to R[[T]]$; a positive-order certificate $h_{\mathrm{pos}} : \forall i,\ c(u_i) = 0$; and, writing

$$\mathrm{Act}_N(u) := \{i \in I \mid u_i \not\equiv_N 0\},$$

an active-set certificate $h_{\mathrm{fin}} : \forall N,\ \mathrm{Act}_N(u)$ is finite. Thus every packaged family must receive both certificates as constructor fields; this Definition does not synthesize either proof from the component formula.

Definition 3.2.6 (Active-index finset). Uses: def:weil-t-adic-locally-finite-family

Lean: Weil.TAdicallyLocallyFiniteFamily.activeFinset

For locally finite u and $N \ge 0$, define $A_N(u)$ to be the finset obtained from the finite set $\mathrm{Act}_N(u)$ by Set.Finite.toFinset.

Lemma 3.2.7 (Membership in the active-index finset). *Uses:* def:weil-series-agree-through, def:weil-t-adic-locally-finite-family, def:weil-active-index-finset

Lean: Weil.TAdicallyLocallyFiniteFamily.mem_activeFinset

One has $i \in A_N(u)$ if and only if $u_i \not\equiv_N 0$.

Proof. Uses: def:weil-active-index-finset

This is Set.Finite.mem_toFinset applied to the finite set $\mathrm{Act}_N(u)$ used in the definition. □

Lemma 3.2.8 (Monotonicity of active-index finsets). *Uses:* def:weil-active-index-finset

Lean: Weil.TAdicallyLocallyFiniteFamily.activeFinset_mono

If $M \le N$, then $A_M(u) \subseteq A_N(u)$.

Proof. Uses: def:weil-series-agree-through, lemma:weil-active-index-finset-mem

If $i \notin A_N(u)$, then $[T^j]u_i = 0$ for every $j \le N$. This includes every $j \le M$, so $i \notin A_M(u)$. Taking the contrapositive gives the inclusion. □

Definition 3.2.9 (Finite product at a truncation level). Uses: def:weil-active-index-finset

Lean: Weil.TAdicallyLocallyFiniteFamily.truncatedProduct

For a locally finite family u, define

$$\Pi_N(u) := \prod_{i \in A_N(u)} (1 + u_i) \in R[[T]].$$

Lemma 3.2.10 (Compatibility of finite truncated products). *Uses:* def:weil-series-agree-through, def:weil-t-adic-locally-finite-family, def:weil-truncated-product

Lean: Weil.TAdicallyLocallyFiniteFamily.truncatedProduct_agree

If $M \le N$, then $\Pi_N(u) \equiv_M \Pi_M(u)$.

Proof. Uses: `lemma:weil-series-agree-through-operations`, `lemma:weil-active-index-finset-mem`, `lemma:weil-active-index-finset-mono`

By Lemma 3.2.8, split the finite product over $A_N(u)$ into the factors indexed by $A_M(u)$ and those indexed by $A_N(u) \setminus A_M(u)$. For an index in the second set, Lemma 3.2.7 gives $u_i \equiv_M 0$, hence $1 + u_i \equiv_M 1$. Repeatedly apply the multiplicative part of Lemma 3.2.2 to replace these finitely many factors by 1. The remaining product is $\Pi_M(u)$. □

Definition 3.2.11 (Coefficientwise locally finite product). Uses: `def:weil-truncated-product`

Lean: `Weil.TAdicallyLocallyFiniteFamily.product`

Define the locally finite product explicitly by

$$\prod_{i \in I}^{\mathrm{lf}} (1 + u_i) := \texttt{PowerSeries.mk}\big(N \longmapsto [T^N]\Pi_N(u)\big).$$

Thus no inverse-limit object and no unspecified choice of a compatible lift occurs in the definition.

Lemma 3.2.12 (Coefficient formula for the locally finite product). Uses: *def:weil-locally-finite-product*, *def:weil-truncated-product*

Lean: *Weil.TAdicallyLocallyFiniteFamily.coeff_product*

For every N,

$$[T^N] \prod_{i \in I}^{\mathrm{lf}} (1 + u_i) = [T^N]\Pi_N(u).$$

Proof. Uses: `def:weil-locally-finite-product`

Apply `PowerSeries.coeff_mk` at the index N. Its right-hand side is exactly the coefficient function used in the definition. □

Lemma 3.2.13 (Constant coefficient of a locally finite product). Uses: *def:weil-locally-finite-product*

Lean: *Weil.TAdicallyLocallyFiniteFamily.product_constantCoeff*

For every T-adically locally finite family u,

$$c\left(\prod_{i}^{\mathrm{lf}} (1 + u_i)\right) = 1.$$

Proof. Uses: `def:weil-truncated-product`, `lemma:weil-locally-finite-product-coeff`

The coefficient formula at degree zero identifies the left side with $[T^0]\Pi_0(u)$. Every factor $1 + u_i$ in this finite product has constant coefficient 1, because $c(u_i) = 0$ is a field of the locally finite family. Repeated use of the degree-zero instance of `PowerSeries.coeff_mul` makes the constant coefficient of the finite product equal to the finite product of ones, hence to one. □

Lemma 3.2.14 (The locally finite product has the prescribed truncations). Uses: *def:weil-series-agree-through*, *def:weil-locally-finite-product*, *def:weil-truncated-product*

Lean: *Weil.TAdicallyLocallyFiniteFamily.product_agree*

For every N,

$$\prod_{i \in I}^{\mathrm{lf}} (1 + u_i) \equiv_N \Pi_N(u).$$

Proof. Uses: `lemma:weil-truncated-products-compatible`, `lemma:weil-locally-finite-product-coeff`

Fix $j \le N$. The coefficient formula at level j identifies the left coefficient with $[T^j]\Pi_j(u)$. Compatibility with $M = j$ identifies that coefficient with $[T^j]\Pi_N(u)$. This holds for every $j \le N$, which is the definition of agreement through N. □

Lemma 3.2.15 (A prescribed finite superset gives the same truncation). Uses: *def:weil-series-agree-through*, *def:weil-tadic-locally-finite-family*, *def:weil-active-index-finset*, *def:weil-truncated-product*

Lean: *Weil.TAdicallyLocallyFiniteFamily.truncatedProduct_agree_finset*

Let S be a finset of indices. If $A_N(u) \subseteq S$, then

$$\Pi_N(u) \equiv_N \prod_{i \in S} (1 + u_i).$$

Proof. Uses: `def:weil-truncated-product`, `lemma:weil-active-index-finset-mem`, `lemma:weil-series-agree-through-operations`

Use `Finset.prod_sdiff` with $A_N(u) \subseteq S$ to split the product over S into the product over $A_N(u)$ and the factors indexed by $S \setminus A_N(u)$. For i in the latter difference, Lemma 3.2.7 gives $u_i \equiv_N 0$, hence $1 + u_i \equiv_N 1$ by the additive agreement lemma. Induction on the finite difference, using the multiplicative agreement lemma at the induction step, replaces its product by 1. The remaining product is $\Pi_N(u)$. □

Lemma 3.2.16 (A locally finite product agrees with a prescribed truncation)**.** *Uses:* *def:weil-series-agree-through*, *def:weil-t-adic-locally-finite-family*, *def:weil-locally-finite-product*

Lean: *Weil.TAdicallyLocallyFiniteFamily.product_agree_finset*

Let S be a finset of indices. If $A_N(u) \subseteq S$, then

$$\prod_{i}^{\mathrm{lf}} (1 + u_i) \equiv_N \prod_{i \in S} (1 + u_i).$$

Proof. Uses: `lemma:weil-locally-finite-product-agree`, `lemma:weil-truncated-product-agree-prescribed-finset`

Fix $j \leq N$. The first cited lemma identifies the degree-j coefficient of the locally finite product with that of $\Pi_N(u)$; the second identifies the latter with the coefficient of the product over S. This is exactly agreement through N. □

Rational logarithm scope. For the logarithmic subchain beginning here, strengthen R again to a commutative $\mathbf{Q}$-algebra. The already constructed active finsets, truncated products, and locally finite product are used through restriction of their preceding arbitrary-commutative-ring statements.

Definition 3.2.17 (Locally finite logarithm sum)**.** Uses: `def:weil-active-index-finset`, `def:weil-formal-log`

Lean: `Weil.TAdicallyLocallyFiniteFamily.logSum`

For a locally finite family u, define

$$\sum_{i \in I}^{\mathrm{lf}} \mathrm{Log}(1 + u_i) := \texttt{PowerSeries.mk}\left(N \longmapsto \sum_{i \in A_N(u)} [T^N] \mathrm{Log}(1 + u_i) \right).$$

Lemma 3.2.18 (An inactive factor has inactive logarithm)**.** *Uses:* *def:weil-series-agree-through*, *def:weil-formal-log*

Lean: *Weil.formalLog_agree_zero*

If $u \equiv_N 0$ and $c(u) = 0$, then $\mathrm{Log}(1 + u) \equiv_N 0$.

Proof. Uses: `lemma:weil-formal-log-coeff`

Fix $j \leq N$. In the finite coefficient formula for $[T^j] \mathrm{Log}(1 + u)$, every summand contains a coefficient of u^r with $r \geq 1$. Expand it using `PowerSeries.coeff_pow`. Every convolution index is at most $j \leq N$, and at least one factor occurs, so every product contains a coefficient of u that vanishes by $u \equiv_N 0$. Thus every summand is zero. □

Lemma 3.2.19 (Formal logarithm respects finite-degree agreement)**.** *Uses:* *def:weil-series-agree-through*, *def:weil-formal-log*

Lean: *Weil.formalLog_agreeThrough*

If $A \equiv_N B$ and $c(A) = c(B) = 1$, then

$$\mathrm{Log}(A) \equiv_N \mathrm{Log}(B).$$

Proof. Uses: `lemma:weil-series-agree-through-sub`, `lemma:weil-series-agree-through-pow`, `lemma:weil-formal-log-coeff`

Subtracting the identical constant series 1 gives $A - 1 \equiv_N B - 1$. Hence, for every r, the power-agreement lemma gives $(A-1)^r \equiv_N (B-1)^r$. Fix $j \leq N$. If $j = 0$, both logarithmic coefficients are zero. If $j \geq 1$, apply the finite coefficient formula for formal logarithm at degree j. Every summand has $1 \leq r \leq j$, and power agreement identifies its degree-j coefficient on the two sides. The two finite sums are equal. □

Lemma 3.2.20 (Formal logarithm of one is zero). *Uses:* def:weil-formal-log

Lean: Weil.formalLog_one

One has $\mathrm{Log}(1) = 0$.

Proof. *Uses:* lemma:weil-formal-log-coeff

The constant coefficient is zero by the logarithm coefficient lemma. In every positive degree, that lemma writes the coefficient as a finite sum of coefficients of $(1-1)^r = 0^r$ with $r \geq 1$; every summand is zero. Apply `PowerSeries.ext`. □

Lemma 3.2.21 (A finite product turns into a finite logarithm sum). *Uses:* def:weil-formal-log

Lean: Weil.formalLog_finsetProd

Let S be a finset and let $A_i \in R[[T]]$ satisfy $c(A_i) = 1$ for every $i \in S$. Then

$$\mathrm{Log}\left(\prod_{i\in S} A_i\right) = \sum_{i\in S} \mathrm{Log}(A_i).$$

Proof. *Uses:* lemma:weil-formal-log-one, lemma:weil-formal-log-mul

Induct on S. For the empty finset, both sides are zero by Lemma 3.2.20. At an insertion $S \cup \{a\}$ with $a \notin S$, the degree-zero coefficient formula for a finite product and the hypotheses give $c(\prod_{i\in S} A_i) = 1$. Apply Lemma 3.1.33 to $A_a \prod_{i\in S} A_i$, then substitute the induction hypothesis and the `Finset.sum_insert` identity. □

Lemma 3.2.22 (Logarithms agree with the active finite truncation). *Uses:* def:weil-series-agree-through, def:weil-locally-finite-product, def:weil-formal-log, def:weil-truncated-product

Lean: Weil.TAdicallyLocallyFiniteFamily.log_agree_truncatedProduct

For every $N \geq 0$,

$$\mathrm{Log}\left(\prod_{i}^{\mathrm{lf}}(1+u_i)\right) \equiv_N \mathrm{Log}(\Pi_N(u)).$$

Proof. *Uses:* lemma:weil-locally-finite-product-constant, lemma:weil-locally-finite-product-agree, lemma:weil-formal-log-agree-through

The locally finite product has constant coefficient one. The finite product $\Pi_N(u)$ also has constant coefficient one, because every factor $1+u_i$ does. The locally finite product agrees with $\Pi_N(u)$ through N. Instantiate the logarithm-agreement lemma with these two constant-coefficient certificates and this agreement. □

Lemma 3.2.23 (The truncated logarithm has the prescribed coefficient). *Uses:* def:weil-locally-finite-log-sum, def:weil-formal-log, def:weil-truncated-product

Lean: Weil.TAdicallyLocallyFiniteFamily.coeff_log_truncatedProduct

For every $N \geq 0$,

$$[T^N]\,\mathrm{Log}(\Pi_N(u)) = [T^N] \sum_{i}^{\mathrm{lf}} \mathrm{Log}(1+u_i).$$

Proof. *Uses:* def:weil-locally-finite-log-sum, lemma:weil-formal-log-finset-product

Apply the finite-product logarithm lemma to $\Pi_N(u) = \prod_{i\in A_N(u)}(1+u_i)$; each factor has constant coefficient one. Taking degree N gives $\sum_{i\in A_N(u)}[T^N]\,\mathrm{Log}(1+u_i)$, which is the defining coefficient of the locally finite logarithm sum. □

Lemma 3.2.24 (Coefficient formula for the logarithm of a locally finite product). *Uses:* def:weil-locally-finite-product, def:weil-locally-finite-log-sum, def:weil-formal-log

Lean: Weil.TAdicallyLocallyFiniteFamily.coeff_log_product

For every $N \geq 0$,

$$[T^N]\,\mathrm{Log}\left(\prod_{i}^{\mathrm{lf}}(1+u_i)\right) = [T^N] \sum_{i}^{\mathrm{lf}} \mathrm{Log}(1+u_i).$$

Proof. *Uses:* lemma:weil-locally-finite-log-agree-truncation, lemma:weil-truncated-log-coeff

The first cited lemma identifies the degree-N coefficient on the left with $[T^N]\,\mathrm{Log}(\Pi_N(u))$. The second cited lemma identifies that coefficient with the right side. □

Proposition 3.2.25 (Logarithm of a locally finite product). *Uses:* *def:weil-locally-finite-product, def:weil-locally-finite-log-sum, def:weil-formal-log*

Lean: *Weil.TAdicallyLocallyFiniteFamily.log_product*

For every T-adically locally finite family,

$$\mathrm{Log}\left(\prod_{i\in I}^{\mathrm{lf}}(1+u_i)\right)=\sum_{i\in I}^{\mathrm{lf}}\mathrm{Log}(1+u_i).$$

Proof. Uses: `lemma:weil-locally-finite-log-coeff`

Apply `PowerSeries.ext`. At every coefficient degree N, the required equality is Lemma 3.2.24. □

Generic coefficient-ring reset. Reset R to an arbitrary commutative ring. The following coefficient-subring predicate and preservation lemma concern only finite coefficient formulas for the already constructed locally finite product.

Definition 3.2.26 (Coefficientwise membership in a subring). Uses:

Lean: `Weil.PowerSeries.HasCoefficientsIn`

If $R_0 \subseteq R$ is a subring, say that $A \in R[[T]]$ has coefficients in R_0 if $[T^n]A \in R_0$ for every $n \geq 0$.

Lemma 3.2.27 (A locally finite product preserves a coefficient subring). *Uses:* *def:weil-locally-finite-product, def:weil-power-series-over-subring*

Lean: *Weil.TAdicallyLocallyFiniteFamily.product_mem_subring*

Let $R_0 \subseteq R$ be a subring. If every u_i has coefficients in R_0, then $\prod_i^{\mathrm{lf}}(1+u_i)$ has coefficients in R_0.

Proof. Uses: `def:weil-truncated-product,lemma:weil-locally-finite-product-coeff`

Fix N. The coefficient formula computes the desired coefficient inside the finite product $\Pi_N(u)$. Each factor $1+u_i$ has coefficients in R_0. Repeated use of `PowerSeries.coeff_mul` expresses the coefficient as a finite sum of finite products of elements of R_0. Closure of the subring under $0, 1, +$, and multiplication puts the result in R_0. □

3.3 Single Euler factors

Generic Euler-factor scope. Retain the arbitrary-commutative-ring hypothesis. The inverse Euler factor is first defined in $\mathbf{Z}[[T]]$ and is mapped to $R[[T]]$ along the canonical ring homomorphism. Its coefficient, inverse, constant-coefficient, positive-part, and initial-vanishing declarations below require no rational scalars.

Definition 3.3.1 (Raw integral inverse Euler factor). Uses:

Lean: `Weil.integralInverseEulerFactor`

For a natural number $m \geq 1$, define

$$E_m^{\mathbf{Z}}(T) := \texttt{PowerSeries.mk}\left(n \longmapsto \begin{cases}1, & m \mid n,\\ 0, & m \nmid n\end{cases}\right) \in \mathbf{Z}[[T]].$$

This Definition has exactly one output object: the integral power series.

Lemma 3.3.2 (Coefficient formula for the raw integral inverse Euler factor). *Uses:* *def:weil-integral-inverse-euler-factor*

Lean: *Weil.coeff_integralInverseEulerFactor*

For $m \geq 1$ and $n \geq 0$,

$$[T^n]E_m^{\mathbf{Z}}(T) = \begin{cases}1, & m \mid n,\\ 0, & m \nmid n.\end{cases}$$

Proof. Uses: `def:weil-integral-inverse-euler-factor`

Unfold $E_m^{\mathbf{Z}}$. The theorem `PowerSeries.coeff_mk`, instantiated with the displayed function and degree n, returns that function's value at n, which is exactly the stated divisibility split. □

Lemma 3.3.3 (Constant coefficient of the raw integral inverse Euler factor). *Uses:* def:weil-integral-inverse-euler-factor

Lean: Weil.integralInverseEulerFactor_constantCoeff

For $m \geq 1$, one has $c(E_m^{\mathbf{Z}}) = 1$.

Proof. Uses: lemma:weil-integral-inverse-euler-factor-coeff

Instantiate the raw integral coefficient formula at $n = 0$. Since $m \mid 0$, its first branch is 1. □

Lemma 3.3.4 (Initial vanishing of the raw integral positive part). *Uses:* def:weil-integral-inverse-euler-factor, def:weil-series-agree-through

Lean: Weil.integralInverseEulerFactor_sub_one_agree_zero

The series $E_m^{\mathbf{Z}} - 1$ has constant coefficient zero. If $N < m$, then $E_m^{\mathbf{Z}} - 1 \equiv_N 0$.

Proof. Uses: def:weil-series-agree-through, lemma:weil-integral-inverse-euler-factor-coeff

Instantiate `map_sub` for the additive homomorphism `PowerSeries.coeff (R := Int) n`. At $n = 0$, the raw integral coefficient formula and `PowerSeries.coeff_one` give $[T^0](E_m^{\mathbf{Z}} - 1) = 1 - 1 = 0$. Now assume $1 \leq n \leq N < m$. Then $m \nmid n$, so the raw coefficient formula gives $[T^n]E_m^{\mathbf{Z}} = 0$; since $n \neq 0$, `PowerSeries.coeff_one` gives $[T^n]1 = 0$. Thus the degree-n coefficient of the difference is zero. Unfolding agreement through N yields the second assertion. □

Definition 3.3.5 (Scalar extension of an inverse Euler factor). Uses: def:weil-integral-inverse-euler-factor

Lean: Weil.inverseEulerFactor

For the current commutative coefficient ring R and $m \geq 1$, define

$$E_{m,R}(T) := \texttt{PowerSeries.map}\big(\texttt{Int.castRingHom}\ R\big)\big(E_m^{\mathbf{Z}}(T)\big) \in R[[T]].$$

When R is fixed by the surrounding scope, write this series as $E_m(T)$. This Definition has exactly one output object, the scalar extension of the previously defined integral series.

Lemma 3.3.6 (Coefficient formula for a scalar-extended inverse Euler factor). *Uses:* def:weil-inverse-euler-factor

Lean: Weil.coeff_inverseEulerFactor

For $m \geq 1$ and $n \geq 0$,

$$[T^n]E_{m,R}(T) = \begin{cases} 1, & m \mid n, \\ 0, & m \nmid n. \end{cases}$$

Equivalently, $E_{m,R}(T) = \sum_{r \geq 0} T^{mr}$ coefficientwise.

Proof. Uses: def:weil-integral-inverse-euler-factor, def:weil-inverse-euler-factor

Unfold $E_{m,R}$ and apply `PowerSeries.coeff_map`. Its degree-n coefficient is

$$\big(\texttt{Int.castRingHom}\ R\big)\big([T^n]E_m^{\mathbf{Z}}\big).$$

Unfold $E_m^{\mathbf{Z}}$ and apply `PowerSeries.coeff_mk`; the inner integral coefficient is 1 when $m \mid n$ and 0 otherwise. The ring homomorphism sends these values to 1_R and 0_R by `map_one` and `map_zero`. The second description follows because $n = mr$ for a unique $r \in \mathbf{N}$ exactly when $m \mid n$. □

Lemma 3.3.7 (Constant coefficient of an inverse Euler factor). *Uses:* def:weil-inverse-euler-factor

Lean: Weil.inverseEulerFactor_constantCoeff

For $m \geq 1$, one has $c(E_m) = 1$.

Proof. Uses: lemma:weil-inverse-euler-factor-coeff

Instantiate Lemma 3.3.6 at $n = 0$. Since every positive m divides 0, its first branch gives $[T^0]E_m = 1$. □

Lemma 3.3.8 (Inverse identity for a single Euler factor). *Uses:* def:weil-inverse-euler-factor

Lean: Weil.one_sub_X_pow_mul_inverseEulerFactor

For $m \geq 1$,

$$(1 - T^m)E_m(T) = 1 = E_m(T)(1 - T^m).$$

Proof. Uses: `lemma:weil-inverse-euler-factor-coeff`
The constant coefficient of the left product is 1. For $n \geq 1$, use `PowerSeries.coeff_mul`. Since $1 - T^m$ has nonzero coefficients only in degrees 0 and m, the result is

$$[T^n]E_m - \begin{cases} [T^{n-m}]E_m, & m \leq n, \\ 0, & n < m. \end{cases}$$

If $m \nmid n$, both terms are zero; if $m \mid n$ and $n > 0$, then $m \leq n$ and both terms are one. Hence every positive coefficient is zero. Equality of coefficients gives the first identity, and commutativity gives the second. □

Definition 3.3.9 (Positive part of an inverse Euler factor)**.** Uses: `def:weil-inverse-euler-factor`
Lean: `Weil.inverseEulerFactorPositivePart`
For $m \geq 1$, define $e_m(T) := E_m(T) - 1$.

Lemma 3.3.10 (Initial vanishing of an inverse Euler factor)**.** *Uses:* `def:weil-inverse-euler-factor-positive-part, def:weil-series-agree-through`
Lean: `Weil.inverseEulerFactorPositivePart_agree_zero`
The series e_m has constant coefficient zero. If $N < m$, then $e_m \equiv_N 0$.

Proof. Uses: `def:weil-inverse-euler-factor-positive-part, def:weil-series-agree-through, lemma:weil-inverse-euler-factor-coeff`
The coefficient of E_m in degree zero is one, so $c(e_m) = 0$. For $1 \leq n < m$, the positive integer m does not divide n, hence the coefficient of E_m, and therefore of e_m, is zero. This proves agreement through every $N < m$. □

Lemma 3.3.11 (Activation of an inverse Euler factor over a nontrivial ring)**.** *Uses:* `def:weil-inverse-euler-factor-positive-part, def:weil-series-agree-through`
Lean: `Weil.inverseEulerFactorPositivePart_not_agree_zero`
Assume that the commutative ring R is nontrivial. If $N \geq m$, then $e_m \not\equiv_N 0$.

Proof. Uses: `def:weil-inverse-euler-factor-positive-part, def:weil-series-agree-through, lemma:weil-inverse-euler-factor-coeff`
Unfold $e_m = E_m - 1$ and instantiate `map_sub` for the additive homomorphism `PowerSeries.coeff (R := R) m`; this gives $[T^m](E_m - 1) = [T^m]E_m - [T^m]1$. Since $m \mid m$, the inverse Euler-factor coefficient formula gives $[T^m]E_m = 1$. Since $m \geq 1$, one has $m \neq 0$, and `PowerSeries.coeff_one` gives $[T^m]1 = 0$. Hence $[T^m]e_m = 1 - 0 = 1$. Nontriviality of R gives $1 \neq 0$, so this differs from the degree-m coefficient of the zero series. Because $m \leq N$, agreement through N fails. □

Rational Euler-logarithm scope. For the remaining declarations of this section, work over a commutative $\mathbf{Q}$-algebra (and specialize to $\mathbf{Q}[[T]]$ where displayed). This stronger hypothesis is used only for division, formal logarithm, exponential, and the differential equations.

Definition 3.3.12 (Logarithm series of a single Euler factor)**.** Uses:
Lean: `Weil.eulerLogSeries`
For $m \geq 1$, define $L_m(T) \in \mathbf{Q}[[T]]$ by

$$[T^n]L_m(T) = \begin{cases} 1/(n/m), & n > 0 \text{ and } m \mid n, \\ 0, & \text{otherwise.} \end{cases}$$

Equivalently, $L_m(T) = \sum_{r \geq 1} T^{mr}/r$.

Lemma 3.3.13 (Constant coefficient and derivative of the Euler logarithm)**.** *Uses:* `def:weil-euler-log-series, def:weil-formal-derivative, def:weil-inverse-euler-factor`
Lean: `Weil.eulerLogSeries_derivative`
One has $c(L_m) = 0$ and

$$DL_m = mT^{m-1}E_m(T).$$

Proof. Uses: `def:weil-euler-log-series, lemma:weil-formal-derivative-rules, lemma:weil-inverse-euler-factor-coeff`
The constant coefficient is zero by definition. Fix $n \geq 0$. The coefficient of DL_m is $(n+1)[T^{n+1}]L_m$. It vanishes unless $n + 1 = mr$ with $r \geq 1$; in that case it equals $(mr)/r = m$. By `PowerSeries.coeff_mul_X_pow'`, the coefficient of $mT^{m-1}E_m$ is also zero unless $m - 1 \leq n$ and $m \mid n - (m - 1)$, equivalently $m \mid n + 1$; in that case it is m. The coefficients agree in every degree. □

Lemma 3.3.14 (Derivative of an inverse Euler factor). *Uses:* *def:weil-inverse-euler-factor, def:weil-formal-derivative*

Lean: *Weil.inverseEulerFactor_derivative*

For $m \geq 1$,

$$DE_m = mT^{m-1}E_m^2.$$

Proof. Uses: lemma:weil-inverse-euler-factor-mul, lemma:weil-formal-derivative-rules

Differentiate $(1 - T^m)E_m = 1$. The product rule and the direct monomial derivative $D(T^m) = mT^{m-1}$ give

$$-mT^{m-1}E_m + (1 - T^m)DE_m = 0.$$

Multiply by E_m and use $(1 - T^m)E_m = 1$. The result is the displayed identity. □

Lemma 3.3.15 (The exponential Euler series satisfies the Euler ODE). *Uses:* *def:weil-formal-exp, def:weil-euler-log-series, def:weil-inverse-euler-factor, def:weil-formal-derivative*

Lean: *Weil.formalExp_eulerLogSeries_ode*

For $m \geq 1$,

$$D\operatorname{Exp}(L_m) = (mT^{m-1}E_m)\operatorname{Exp}(L_m).$$

Proof. Uses: lemma:weil-euler-log-series-derivative, lemma:weil-formal-exp-derivative

The first cited lemma gives $c(L_m) = 0$ and $DL_m = mT^{m-1}E_m$. Apply the exponential derivative lemma to L_m and substitute this displayed value of DL_m. □

Lemma 3.3.16 (The inverse Euler factor satisfies the Euler ODE). *Uses:* *def:weil-inverse-euler-factor, def:weil-formal-derivative*

Lean: *Weil.inverseEulerFactor_ode*

For $m \geq 1$,

$$DE_m = (mT^{m-1}E_m)E_m.$$

Proof. Uses: lemma:weil-inverse-euler-factor-derivative

Lemma 3.3.14 gives $DE_m = mT^{m-1}E_m^2$. Unfold the square as E_mE_m and reassociate the three factors. □

Lemma 3.3.17 (Exponential of the Euler logarithm). *Uses:* *def:weil-formal-exp, def:weil-euler-log-series, def:weil-inverse-euler-factor*

Lean: *Weil.formalExp_eulerLogSeries*

For $m \geq 1$,

$$\operatorname{Exp}(L_m) = E_m.$$

Proof. Uses: lemma:weil-euler-log-series-derivative, lemma:weil-formal-exp-constant, lemma:weil-inverse-euler-factor-constant, lemma:weil-exp-euler-log-series-ode, lemma:weil-inverse-euler-factor-ode, lemma:weil-common-logarithmic-derivative-unique

Set $X = \operatorname{Exp}(L_m)$, $Y = E_m$, and $H = mT^{m-1}E_m$. The two preceding ODE lemmas give $DX = HX$ and $DY = HY$. Lemma 3.3.13 gives $c(L_m) = 0$, so the formal-exponential constant lemma gives $c(X) = 1$; the inverse Euler constant lemma gives $c(Y) = 1$. Apply Lemma 3.1.22. □

Lemma 3.3.18 (Logarithm of an inverse Euler factor). *Uses:* *def:weil-formal-log, def:weil-inverse-euler-factor, def:weil-euler-log-series*

Lean: *Weil.formalLog_inverseEulerFactor*

For $m \geq 1$,

$$\operatorname{Log}(E_m(T)) = L_m(T) = \sum_{r \geq 1} \frac{T^{mr}}{r}.$$

Proof. Uses: lemma:weil-exp-euler-log-series, lemma:weil-formal-log-exp, lemma:weil-euler-log-series-derivative, def:weil-euler-log-series

Apply Log to $\operatorname{Exp}(L_m) = E_m$. The series L_m has zero constant coefficient by Lemma 3.3.13, so Lemma 3.1.28 identifies the left side with L_m. The coefficient description on the right is part of Definition 3.3.12. □

3.4 Finite geometric-Frobenius orbits

No coefficient ring is in scope in the combinatorial declarations of this section. When Euler products resume later, their rational coefficient ring is stated there explicitly.

The letter F in this section denotes a permutation modeling *geometric* Frobenius. This convention agrees with `weil_conventions.md`: no arithmetic Frobenius occurs in this module.

Definition 3.4.1 (Finite-orbit geometric-Frobenius datum)**.** Uses:

Lean: `Weil.FiniteOrbitFrobeniusData`

A finite-orbit geometric-Frobenius datum D consists of:

(i) a type Ω and a permutation $F\colon \Omega \simeq \Omega$;

(ii) for each $x \in \Omega$, a selected integer $r_x \geq 1$ and an equality $F^{r_x}(x) = x$;

(iii) for each $n \geq 1$, a selected finset $\mathrm{Fix}_D(n)$ and a proof, for every x, of the equivalence

$$x \in \mathrm{Fix}_D(n) \quad \Longleftrightarrow \quad F^n(x) = x.$$

Iteration is natural-number iteration of the underlying function of F.

Definition 3.4.2 (Frobenius orbit length)**.** Uses: `def:weil-finite-orbit-frobenius-datum`

Lean: `Weil.FiniteOrbitFrobeniusData.orbitLength`

For $x \in \Omega$, define

$$d_F(x) := \texttt{Function.minimalPeriod}\ F\, x.$$

Lemma 3.4.3 (Positivity of Frobenius orbit length)**.** *Uses:* *def:weil-finite-orbit-frobenius-datum, def:weil-frobenius-orbit-length*

Lean: *Weil.FiniteOrbitFrobeniusData.orbitLength_pos*

For every $x \in \Omega$, one has $d_F(x) > 0$.

Proof. Uses: `def:weil-finite-orbit-frobenius-datum,def:weil-frobenius-orbit-length`

The pair $(r_x, F^{r_x}x = x)$ stored in the Frobenius datum, together with $r_x > 0$, proves that $x \in$ `Function.periodicPts` F. Apply `Function.minimalPeriod_pos_of_mem_periodicPts` with $f = F$ and the point x. □

Lemma 3.4.4 (Fixed-point criterion from the minimal period)**.** *Uses:* *def:weil-frobenius-orbit-length*

Lean: *Weil.FiniteOrbitFrobeniusData.isPeriodicPt_iff_dvd*

For $n \geq 1$ and $x \in \Omega$,

$$F^n(x) = x \quad \Longleftrightarrow \quad d_F(x) \mid n.$$

Proof. Uses: `def:weil-frobenius-orbit-length`

The left side is `Function.IsPeriodicPtFnx`. Apply the exact mathlib theorem `Function.isPeriodicPt_iff_minimalPeriod_dvd` with $f = F$, period n, and point x. Replace `Function.minimalPeriodFx` by $d_F(x)$ using the definition. □

Definition 3.4.5 (Cyclic group generated by Frobenius)**.** Uses: `def:weil-finite-orbit-frobenius-datum`

Lean: `Weil.FiniteOrbitFrobeniusData.cyclicSubgroup`

Regard F as an element of the group Equiv.Perm(Ω). Define

$$\Gamma_F := \texttt{Subgroup.zpowers}\ F.$$

It acts on Ω by evaluation of permutations.

Definition 3.4.6 (Type of Frobenius orbits)**.** Uses: `def:weil-frobenius-cyclic-subgroup`

Lean: `Weil.FiniteOrbitFrobeniusData.Orbit`

Define the type of Frobenius orbits by

$$\mathrm{Orb}_F(\Omega) := \texttt{MulAction.orbitRel.Quotient}\ \Gamma_F\, \Omega.$$

For $x \in \Omega$, write $[x]_F$ for its quotient class and, for an orbit O, write $|O| \subseteq \Omega$ for `MulAction.orbitRel.Quotient.orbitO`.

Definition 3.4.7 (Selected representative of a Frobenius orbit). Uses: `def:weil-frobenius-orbit-type`

Lean: `Weil.FiniteOrbitFrobeniusData.Orbit.representative`

For $O \in \mathrm{Orb}_F(\Omega)$, define

$$\mathrm{rep}(O) := \texttt{Quotient.out}\ O \in \Omega.$$

Lemma 3.4.8 (The selected representative represents its orbit). *Uses:* *def:weil-frobenius-orbit-type, def:weil-frobenius-orbit-representative*

Lean: *Weil.FiniteOrbitFrobeniusData.Orbit.mk_representative*

For every orbit O, one has $[\mathrm{rep}(O)]_F = O$ and $\mathrm{rep}(O) \in |O|$.

Proof. Uses: `def:weil-frobenius-orbit-representative`

The first equality is the exact theorem `Quotient.out_eq'` for the setoid `MulAction.orbitRel` Γ_F Ω. Substitute this equality into `MulAction.orbitRel.Quotient.mem_orbit`; that theorem states that $x \in |O|$ if and only if $[x]_F = O$, and gives the second assertion. □

Definition 3.4.9 (Intrinsic degree of a Frobenius orbit). Uses: `def:weil-frobenius-orbit-type`

Lean: `Weil.FiniteOrbitFrobeniusData.Orbit.degree`

For $O \in \mathrm{Orb}_F(\Omega)$, define

$$\deg_F(O) := \mathrm{ncard}(|O|).$$

The finiteness and positivity of $|O|$ are proved below rather than included silently in this definition.

Lemma 3.4.10 (A Frobenius orbit is finite). *Uses:* *def:weil-finite-orbit-frobenius-datum, def:weil-frobenius-cyclic-subgroup, def:weil-frobenius-orbit-type*

Lean: *Weil.FiniteOrbitFrobeniusData.Orbit.finite*

For every $O \in \mathrm{Orb}_F(\Omega)$, the set $|O|$ is finite.

Proof. Uses: `def:weil-frobenius-orbit-representative,lemma:weil-frobenius-orbit-length-positive,lemma:weil-frobenius-orbit-representative-class`

Put $x = \mathrm{rep}(O)$ and $d = d_F(x) > 0$. The exact equivalence `MulAction.orbitZPowersEquivFx` identifies `MulAction.orbit(Subgroup.zpowersF)x` with $\mathbf{Z}/d\mathbf{Z}$ = `ZMod d`. Positivity of d supplies the `NeZero d` instance, so `ZMod d` is finite. Transport finiteness across the equivalence. Finally, `MulAction.orbitRel.Quotient.orbit_eq_orbit_out`, with `Quotient.out_eq'`, identifies that orbit with $|O|$. □

Lemma 3.4.11 (Orbit degree equals the minimal period of a representative). *Uses:* *def:weil-frobenius-orbit-degree, def:weil-frobenius-orbit-representative, def:weil-frobenius-orbit-length*

Lean: *Weil.FiniteOrbitFrobeniusData.Orbit.degree_eq_minimalPeriod*

For every orbit O,

$$\deg_F(O) = d_F(\mathrm{rep}(O)).$$

Proof. Uses: `lemma:weil-frobenius-orbit-finite,lemma:weil-frobenius-orbit-representative-class`

Install the finite type structure on $|O|$ obtained from Lemma 3.4.10. Rewrite $|O|$ as the orbit of $x = \mathrm{rep}(O)$ using `MulAction.orbitRel.Quotient.orbit_eq_orbit_out`. Apply `MulAction.minimalPeriod_eq_card` with the group element $a = F$ and point $b = x$. It gives

$$d_F(x) = \mathrm{card}(\texttt{MulAction.orbit (Subgroup.zpowers F) x}).$$

For a finite set, `Set.ncard` is that subtype cardinal. Reverse the displayed equality to match the definition of $\deg_F(O)$. □

Lemma 3.4.12 (Orbit degree is positive). *Uses:* *def:weil-frobenius-orbit-degree*

Lean: *Weil.FiniteOrbitFrobeniusData.Orbit.degree_pos*

For every Frobenius orbit O, one has $\deg_F(O) > 0$.

Proof. Uses: `lemma:weil-frobenius-orbit-degree-representative,lemma:weil-frobenius-orbit-length-positive`

Replace $\deg_F(O)$ by the minimal period of its representative and apply Lemma 3.4.3. □

Lemma 3.4.13 (Degree of the orbit containing a point). *Uses:* *def:weil-frobenius-orbit-type, def:weil-frobenius-orbit-degree, def:weil-frobenius-orbit-length*

Lean: *Weil.FiniteOrbitFrobeniusData.degree_mk*

For every $x \in \Omega$,

$$\deg_F([x]_F) = d_F(x).$$

Proof. Uses: lemma:weil-frobenius-orbit-finite, lemma:weil-frobenius-orbit-degree-representative

The quotient-orbit theorem `MulAction.orbitRel.Quotient.orbit_mk` identifies $|[x]_F|$ with `MulAction.orbit(Subgroup.zpowersF)x`. Install its finite type structure using Lemma 3.4.10 and apply `MulAction.minimalPeriod_eq_card` with $a = F, b = x$. Converting the cardinal to `Set.ncard` gives the displayed equality directly. □

Lemma 3.4.14 (Fixed-point criterion in terms of intrinsic orbit degree). *Uses:* *def:weil-frobenius-orbit-type, def:weil-frobenius-orbit-degree*

Lean: *Weil.FiniteOrbitFrobeniusData.fixed_iff_degree_dvd*

For $n \geq 1$ and $x \in \Omega$,

$$F^n(x) = x \quad \Longleftrightarrow \quad \deg_F([x]_F) \mid n.$$

Proof. Uses: lemma:weil-frobenius-fixed-iff-orbit-length-divides, lemma:weil-frobenius-orbit-degree-point

Replace $\deg_F([x]_F)$ with $d_F(x)$ by Lemma 3.4.13. The result is exactly Lemma 3.4.4. □

Definition 3.4.15 (Explicit finset of points in an orbit). Uses: def:weil-frobenius-orbit-representative, def:weil-frobenius-orbit-length

Lean: `Weil.FiniteOrbitFrobeniusData.Orbit.pointFinset`

For an orbit O, put $x = \mathrm{rep}(O)$ and $d = d_F(x)$, and define

$$\mathcal{P}(O) := \big(\mathrm{Finset.range}\, d\big).\mathrm{image}\big(j \longmapsto F^j(x)\big).$$

Lemma 3.4.16 (The explicit orbit finset has the correct members). *Uses:* *def:weil-frobenius-orbit-point-finset, def:weil-frobenius-orbit-type*

Lean: *Weil.FiniteOrbitFrobeniusData.Orbit.mem_pointFinset*

For $y \in \Omega$ and an orbit O,

$$y \in \mathcal{P}(O) \quad \Longleftrightarrow \quad y \in |O|.$$

Proof. Uses: lemma:weil-frobenius-orbit-length-positive, lemma:weil-frobenius-orbit-representative-class

Put $x = \mathrm{rep}(O)$ and $d = d_F(x)$. A member of $\mathcal{P}(O)$ has the form $F^j(x)$, so it lies in the Subgroup.zpowers(F)-orbit of x and hence in $|O|$.

Conversely, rewrite $|O|$ as the orbit of x by `MulAction.orbitRel.Quotient.orbit_eq_orbit_out`. A point in this orbit has the form $F^z(x)$ for $z \in \mathbf{Z}$. The exact theorem `MulAction.zpow_smul_mod_minimalPeriod`, with group element F and point x, replaces z by its residue j modulo d. Positivity of d gives $0 \leq j < d$, so $y = F^j(x)$ belongs to the image of Finset.range d. □

Lemma 3.4.17 (Cardinality of the explicit orbit finset). *Uses:* *def:weil-frobenius-orbit-point-finset, def:weil-frobenius-orbit-degree*

Lean: *Weil.FiniteOrbitFrobeniusData.Orbit.card_pointFinset*

For every orbit O,

$$\#\mathcal{P}(O) = \deg_F(O).$$

Proof. Uses: lemma:weil-frobenius-orbit-point-finset-mem, lemma:weil-frobenius-orbit-finite

Lemma 3.4.16 identifies the coercion of $\mathcal{P}(O)$ to a set with $|O|$. For a finite set, the cardinal of any finset having exactly that membership predicate is its `Set.ncard`. The latter is $\deg_F(O)$ by definition. □

Lemma 3.4.18 (Distinct orbits have disjoint point finsets). *Uses:* *def:weil-frobenius-orbit-point-finset*

Lean: *Weil.FiniteOrbitFrobeniusData.Orbit.pointFinset_disjoint*

If $O \neq O'$, then $\mathrm{Disjoint}(\mathcal{P}(O), \mathcal{P}(O'))$.

Proof. Uses: lemma:weil-frobenius-orbit-point-finset-mem

Suppose y belonged to both finsets. Then $y \in |O|$ and $y \in |O'|$. The exact equivalence `MulAction.orbitRel.Quotient.mem_orbit` turns these memberships into $[y]_F = O$ and $[y]_F = O'$. Transitivity gives $O = O'$, a contradiction. □

Definition 3.4.19 (Orbit class of a point). Uses: `def:weil-frobenius-orbit-type`

Lean: `Weil.FiniteOrbitFrobeniusData.orbitClass`

Define $\mathrm{cl}_F(x) := [x]_F$.

Definition 3.4.20 (Finset of orbits of bounded degree). Uses: `def:weil-finite-orbit-frobenius-datum`, `def:weil-frobenius-orbit-class`, `def:weil-frobenius-orbit-degree`

Lean: `Weil.FiniteOrbitFrobeniusData.boundedOrbitFinset`

For $r \geq 1$, define

$$\mathcal{O}_{\leq r} := \left(\mathrm{Fix}_D(r!).\mathrm{image}(\mathrm{cl}_F)\right).\mathrm{filter}\big(O \longmapsto \deg_F(O) \leq r\big).$$

Lemma 3.4.21 (Membership criterion for bounded orbit finsets). *Uses:* `def:weil-bounded-frobenius-orbit-finset`

Lean: `Weil.FiniteOrbitFrobeniusData.mem_boundedOrbitFinset`

If $r \geq 1$, then

$$O \in \mathcal{O}_{\leq r} \quad \Longleftrightarrow \quad \deg_F(O) \leq r.$$

Proof. Uses: `lemma:weil-frobenius-orbit-degree-positive`, `lemma:weil-frobenius-orbit-representative-class`, `lemma:weil-frobenius-fixed-iff-orbit-degree-divides`

Forward implication is the filter condition. Conversely assume $1 \leq \deg_F(O) \leq r$. The exact arithmetic theorem `Nat.dvd_factorial` gives $\deg_F(O) \mid r!$. Apply Lemma 3.4.14 to $x = \mathrm{rep}(O)$ and $n = r!$; use $[\mathrm{rep}(O)]_F = O$. Thus the representative belongs to $\mathrm{Fix}_D(r!)$. Its image under cl_F is O, so O belongs to the image finset, and the assumed inequality puts it in the filter. □

Definition 3.4.22 (Finset of orbits of one degree). Uses: `def:weil-bounded-frobenius-orbit-finset`, `def:weil-frobenius-orbit-degree`

Lean: `Weil.FiniteOrbitFrobeniusData.degreeOrbitFinset`

For $m \geq 1$, define

$$\mathcal{O}_m := \mathcal{O}_{\leq m}.\mathrm{filter}\big(O \longmapsto \deg_F(O) = m\big).$$

Lemma 3.4.23 (Membership criterion for one-degree orbit finsets). *Uses:* `def:weil-degree-frobenius-orbit-finset`

Lean: `Weil.FiniteOrbitFrobeniusData.mem_degreeOrbitFinset`

If $m \geq 1$, then

$$O \in \mathcal{O}_m \quad \Longleftrightarrow \quad \deg_F(O) = m.$$

Proof. Uses: `lemma:weil-bounded-frobenius-orbit-finset-mem`

Forward implication is the second filter condition. Conversely, $\deg_F(O) = m$ implies $\deg_F(O) \leq m$, so Lemma 3.4.21 puts O in $\mathcal{O}_{\leq m}$; the equality then puts it in the filter. □

Definition 3.4.24 (Number of Frobenius orbits of one degree). Uses: `def:weil-degree-frobenius-orbit-finset`

Lean: `Weil.FiniteOrbitFrobeniusData.orbitCount`

For $m \geq 1$, define

$$a_m(D) := \#\mathcal{O}_m.$$

Definition 3.4.25 (Relevant orbits for a Frobenius power). Uses: `def:weil-bounded-frobenius-orbit-finset`, `def:weil-frobenius-orbit-degree`

Lean: `Weil.FiniteOrbitFrobeniusData.relevantOrbitFinset`

For $n \geq 1$, define

$$\mathcal{O}(n) := \mathcal{O}_{\leq n}.\mathrm{filter}\big(O \longmapsto \deg_F(O) \mid n\big).$$

Lemma 3.4.26 (Membership criterion for relevant orbit finsets). *Uses:* `def:weil-relevant-frobenius-orbit-finset`

Lean: `Weil.FiniteOrbitFrobeniusData.mem_relevantOrbitFinset`

If $n \geq 1$, then

$$O \in \mathcal{O}(n) \quad \Longleftrightarrow \quad \deg_F(O) \mid n.$$

Proof. Uses: `lemma:weil-bounded-frobenius-orbit-finset-mem`, `lemma:weil-frobenius-orbit-degree-positive`

Forward implication is the filter condition. Conversely, if $\deg_F(O) \mid n$, positivity of the degree and $n \geq 1$ imply $\deg_F(O) \leq n$. The bounded-orbit membership lemma puts O in $\mathcal{O}_{\leq n}$, and divisibility puts it in the filter. □

Definition 3.4.27 (Orbitwise union of fixed points). Uses: `def:weil-relevant-frobenius-orbit-finset,def:weil-frobenius-orbit-point-finset`

Lean: `Weil.FiniteOrbitFrobeniusData.fixedPointOrbitUnion`

For $n \geq 1$, define the finset

$$U_n := \mathcal{O}(n).\,\mathrm{biUnion}\big(O \longmapsto \mathcal{P}(O)\big).$$

Lemma 3.4.28 (The orbitwise union is the fixed-point finset). Uses: `def:weil-finite-orbit-frobenius-datum,def:weil-fixed-point-orbit-union`

Lean: `Weil.FiniteOrbitFrobeniusData.fixedPointOrbitUnion_eq`

For every $n \geq 1$,

$$U_n = \mathrm{Fix}_D(n).$$

Proof. Uses: `lemma:weil-relevant-frobenius-orbit-finset-mem,lemma:weil-frobenius-orbit-point-finset-mem,lemma:weil-frobenius-fixed-iff-orbit-degree-divides`

Use finset extensionality and fix $x \in \Omega$. Membership in U_n means that there is an $O \in \mathcal{O}(n)$ with $x \in \mathcal{P}(O)$. The point membership lemma forces $O = [x]_F$, and relevant-orbit membership is then $\deg_F([x]_F) \mid n$. By the fixed-point criterion this is equivalent to $F^n(x) = x$, which is equivalent to $x \in \mathrm{Fix}_D(n)$ by the membership field of the datum. □

Definition 3.4.29 (Number of fixed points). Uses: `def:weil-finite-orbit-frobenius-datum`

Lean: `Weil.FiniteOrbitFrobeniusData.fixedPointCount`

For $n \geq 1$, define

$$N_n(D) := \#\,\mathrm{Fix}_D(n).$$

Lemma 3.4.30 (Fixed points counted by relevant orbits). Uses: `def:weil-frobenius-fixed-point-count,def:weil-relevant-frobenius-orbit-finset,def:weil-frobenius-orbit-degree`

Lean: `Weil.FiniteOrbitFrobeniusData.fixedPointCount_eq_sum_orbits`

For $n \geq 1$,

$$N_n(D) = \sum_{O \in \mathcal{O}(n)} \deg_F(O).$$

Proof. Uses: `lemma:weil-fixed-point-orbit-union-eq,lemma:weil-frobenius-orbit-point-finsets-disjoint,lemma:weil-frobenius-orbit-point-finset-card`

Replace $\mathrm{Fix}_D(n)$ by U_n. The family $O \mapsto \mathcal{P}(O)$ is pairwise disjoint on $\mathcal{O}(n)$ by Lemma 3.4.18. Apply the exact theorem `Finset.card_biUnion`; its right side is $\sum_{O \in \mathcal{O}(n)} \#\mathcal{P}(O)$. Replace each cardinal by $\deg_F(O)$ using Lemma 3.4.17. □

Lemma 3.4.31 (Regrouping relevant orbits by degree). Uses: `def:weil-relevant-frobenius-orbit-finset,def:weil-frobenius-orbit-count`

Lean: `Weil.FiniteOrbitFrobeniusData.sum_relevant_eq_sum_divisors`

For $n \geq 1$,

$$\sum_{O \in \mathcal{O}(n)} \deg_F(O) = \sum_{m \mid n} m\, a_m(D).$$

The sum on the right is over the positive divisors of n.

Proof. Uses: `lemma:weil-relevant-frobenius-orbit-finset-mem,lemma:weil-degree-frobenius-orbit-finset-mem`

Partition the finite set $\mathcal{O}(n)$ by the value $m = \deg_F(O)$. Relevant-orbit membership says precisely $m \mid n$. For a fixed positive divisor m, the fiber is

$$\{O \in \mathcal{O}(n) \mid \deg_F(O) = m\} = \mathcal{O}_m :$$

the forward inclusion uses the one-degree membership criterion; the reverse inclusion uses $m \mid n$ and the relevant-orbit criterion. The summand $\deg_F(O)$ is constant with value m on this fiber, so its fiber sum is $m\#\mathcal{O}_m = m a_m(D)$. The finite fiberwise sum formula, obtained by expanding both sides as sums over the disjoint pairs (m, O) with $m \mid n$ and $\deg_F(O) = m$, gives the displayed equality. □

Proposition 3.4.32 (Fixed points counted orbit by orbit)**.** *Uses:* *def:weil-frobenius-fixed-point-count, def:weil-frobenius-orbit-count*

Lean: *Weil.FiniteOrbitFrobeniusData.fixedPointCount_eq_divisorSum*

For every $n \geq 1$,

$$N_n(D) = \sum_{m \mid n} m\, a_m(D).$$

Proof. *Uses:* `lemma:weil-fixed-point-count-relevant-orbits`, `lemma:weil-relevant-orbits-regroup-by-degree`

Substitute the right side of Lemma 3.4.30 into Lemma 3.4.31. □

Definition 3.4.33 (Möbius numerator of an orbit count)**.** *Uses:* `def:weil-frobenius-fixed-point-count`

Lean: `Weil.FiniteOrbitFrobeniusData.mobiusNumerator`

For $m \geq 1$, define the integer

$$M_m(D) := \sum_{r \mid m} \mu(r)\, N_{m/r}(D),$$

where $\mu =$ `ArithmeticFunction.moebius`.

Lemma 3.4.34 (Möbius numerator equals degree times orbit count)**.** *Uses:* *def:weil-orbit-count-mobius-numerator, def:weil-frobenius-orbit-count*

Lean: *Weil.FiniteOrbitFrobeniusData.mobiusNumerator_eq*

For $m \geq 1$,

$$M_m(D) = m\, a_m(D)$$

as an equality of integers.

Proof. *Uses:* `prop:weil-fixed-points-orbit-count`

Apply the forward direction of `ArithmeticFunction.sum_eq_iff_sum_mul_moebius_eq` with coefficient ring $\mathbf{Z}$,

$$f(i) = i\, a_i(D), \qquad g(n) = N_n(D).$$

Its hypothesis $\sum_{i \in n.\mathtt{divisors}} f(i) = g(n)$ is exactly Proposition 3.4.32. Its conclusion is

$$\sum_{(r,s) \in n.\mathtt{divisorsAntidiagonal}} \mu(r) g(s) = f(n).$$

Reindex the antidiagonal by $r \mid n$, where $s = n/r$. At $n = m$ this is the defining sum for $M_m(D)$ on the left and $m a_m(D)$ on the right. □

Corollary 3.4.35 (Möbius recovery of orbit counts)**.** *Uses:* *def:weil-frobenius-orbit-count, def:weil-frobenius-fixed-point-count*

Lean: *Weil.FiniteOrbitFrobeniusData.orbitCount_eq_mobius*

For $m \geq 1$,

$$a_m(D) = \frac{1}{m} \sum_{r \mid m} \mu(r) N_{m/r}(D)$$

in $\mathbf{Q}$.

Proof. *Uses:* `def:weil-orbit-count-mobius-numerator`, `lemma:weil-orbit-count-mobius-numerator-eq`

Embed the integer equality $M_m(D) = m a_m(D)$ into $\mathbf{Q}$ and multiply by m^{-1}; it exists because $m \geq 1$. Expand $M_m(D)$ by its definition. □

Corollary 3.4.36 (Divisibility and nonnegativity of Möbius recovery)**.** *Uses:* *def:weil-frobenius-fixed-point-count*

Lean: *Weil.FiniteOrbitFrobeniusData.mobius_divisible_nonnegative*

For $m \geq 1$, the integer $\sum_{r \mid m} \mu(r) N_{m/r}(D)$ is divisible by m, and its quotient by m is nonnegative.

Proof. *Uses:* `def:weil-orbit-count-mobius-numerator`, `lemma:weil-orbit-count-mobius-numerator-eq`

The preceding integer equality writes the numerator as $m a_m(D)$, proving divisibility. Its quotient is the natural number $a_m(D)$ embedded in $\mathbf{Z}$, so it is nonnegative. □

3.5 Orbit Euler products and the counting zeta series

From now on, all orbit Euler factors are mapped from $\mathbf{Z}[[T]]$ to $\mathbf{Q}[[T]]$ before logarithms are taken.

Definition 3.5.1 (Raw orbit Euler-factor family). Uses: `def:weil-frobenius-orbit-type`, `def:weil-frobenius-orbit-degree`, `def:weil-inverse-euler-factor-positive-part`

Lean: `Weil.FiniteOrbitFrobeniusData.orbitEulerFactorFamily`

For $O \in \mathrm{Orb}_F(\Omega)$, define

$$u_O(T) := E_{\deg_F(O)}(T) - 1.$$

Lemma 3.5.2 (Orbit Euler factors have positive order). *Uses:* `def:weil-orbit-euler-factor-family`, `def:weil-positive-series`

Lean: `Weil.FiniteOrbitFrobeniusData.orbitEulerFactorFamily_positive`

Every u_O has constant coefficient zero.

Proof. Uses: `lemma:weil-frobenius-orbit-degree-positive`, `lemma:weil-inverse-euler-factor-initial-vanishing`

The degree $\deg_F(O)$ is positive. Apply Lemma 3.3.10 with $m = \deg_F(O)$; its first conclusion is $c(E_m - 1) = 0$. □

Lemma 3.5.3 (Local finiteness of the orbit Euler-factor family). *Uses:* `def:weil-orbit-euler-factor-family`, `def:weil-series-agree-through`

Lean: `Weil.FiniteOrbitFrobeniusData.orbitEulerFactorFamily_locallyFinite`

For every $N \geq 0$, only finitely many orbits O satisfy $u_O \not\equiv_N 0$.

Proof. Uses: `lemma:weil-inverse-euler-factor-initial-vanishing`, `lemma:weil-bounded-frobenius-orbit-finset-mem`

By the initial-vanishing lemma, $u_O \not\equiv_N 0$ implies $\deg_F(O) \leq N$. If $N = 0$, this is impossible because orbit degrees are positive. If $N \geq 1$, the bounded orbit membership criterion identifies these orbits with members of the explicit finset $\mathcal{O}_{\leq N}$. Thus the active set is finite. □

Definition 3.5.4 (Packaged locally finite orbit Euler family). Uses: `def:weil-orbit-euler-factor-family`, `def:weil-t-adic-locally-finite-family`

Lean: `Weil.FiniteOrbitFrobeniusData.orbitEulerFamily`

Constructed by: `lemma:weil-orbit-euler-factor-family-positive`, `lemma:weil-orbit-euler-factor-family-locally-finite`

Define $\mathcal{E}_D$ to be the T-adically locally finite family with index type $\mathrm{Orb}_F(\Omega)$, component $u_O = E_{\deg_F(O)} - 1$, and the previously constructed positive-order and active-set-finiteness certificates recorded by the constructor metadata above.

Definition 3.5.5 (Orbit Euler product). Uses: `def:weil-orbit-euler-locally-finite-family`, `def:weil-locally-finite-product`

Lean: `Weil.FiniteOrbitFrobeniusData.orbitEulerProduct`

Define

$$Z_{\mathrm{orb},D}(T) := \prod_{O \in \mathrm{Orb}_F(\Omega)}^{\mathrm{lf}} E_{\deg_F(O)}(T).$$

Lemma 3.5.6 (Active orbit Euler factors are exactly bounded-degree orbits). *Uses:* `def:weil-active-index-finset`, `def:weil-orbit-euler-locally-finite-family`, `def:weil-bounded-frobenius-orbit-finset`

Lean: `Weil.FiniteOrbitFrobeniusData.orbitEulerFamily_activeFinset`

For $N \geq 1$, the active-index finset $A_N(\mathcal{E}_D)$ equals $\mathcal{O}_{\leq N}$. At $N = 0$ it is empty.

Proof. Uses: `lemma:weil-active-index-finset-mem`, `lemma:weil-inverse-euler-factor-initial-vanishing`, `lemma:weil-inverse-euler-factor-activation`, `lemma:weil-bounded-frobenius-orbit-finset-mem`, `lemma:weil-frobenius-orbit-degree-positive`

For $N \geq 1$, active-index membership is $E_{\deg_F(O)} - 1 \not\equiv_N 0$. The initial-vanishing and activation lemmas say this is equivalent to $\deg_F(O) \leq N$, and the bounded-orbit membership lemma says the latter is equivalent to $O \in \mathcal{O}_{\leq N}$. Finset extensionality gives equality. For $N = 0$, positivity of $\deg_F(O)$ and the same initial-vanishing lemma show that no index is active. □

Lemma 3.5.7 (Constant coefficient of the orbit Euler product). *Uses:* def:weil-orbit-euler-product

Lean: Weil.FiniteOrbitFrobeniusData.orbitEulerProduct_constantCoeff

One has $c(Z_{\mathrm{orb},D}) = 1$.

Proof. Uses: lemma:weil-locally-finite-product-coeff,lemma:weil-orbit-euler-active-finset

At truncation level zero the active finset is empty, so the truncated product is the empty product 1. The locally finite coefficient formula therefore gives constant coefficient one. □

Definition 3.5.8 (Counting logarithm series). Uses: def:weil-frobenius-fixed-point-count

Lean: Weil.FiniteOrbitFrobeniusData.countingLogSeries

Define $S_D(T) \in \mathbf{Q}[[T]]$ by

$$[T^0]S_D = 0, \qquad [T^n]S_D = \frac{N_n(D)}{n} \quad (n \geq 1).$$

Lemma 3.5.9 (The counting logarithm has positive order). *Uses:* def:weil-counting-log-series,def:weil-positive-series

Lean: Weil.FiniteOrbitFrobeniusData.countingLogSeries_positive

One has $c(S_D) = 0$.

Proof. Uses: def:weil-counting-log-series

This is the coefficient specified at degree zero in the definition. □

Definition 3.5.10 (Exponential counting zeta series). Uses: def:weil-counting-log-series,def:weil-formal-exp

Lean: Weil.FiniteOrbitFrobeniusData.countingZetaSeries

Define

$$Z_{\mathrm{cnt},D}(T) := \mathrm{Exp}(S_D(T)) = \mathrm{Exp}\left(\sum_{n\geq 1} N_n(D)\frac{T^n}{n}\right).$$

Lemma 3.5.11 (Coefficient of the logarithm of the orbit Euler product). *Uses:* def:weil-orbit-euler-product,def:weil-counting-log-series

Lean: Weil.FiniteOrbitFrobeniusData.coeff_log_orbitEulerProduct

For every $n \geq 1$,

$$[T^n]\,\mathrm{Log}(Z_{\mathrm{orb},D}) = \frac{1}{n}\sum_{O\in\mathcal{O}(n)} \deg_F(O).$$

Proof. Uses: prop:weil-log-locally-finite-product,lemma:weil-orbit-euler-active-finset,lemma:weil-log-inverse-euler-factor,lemma:weil-relevant-frobenius-orbit-finset-mem

The locally finite logarithm formula turns the left side into the finite sum over $A_n(\mathcal{E}_D) = \mathcal{O}_{\leq n}$ of $[T^n]\,\mathrm{Log}(E_{\deg_F(O)})$. By Lemma 3.3.18, the summand is zero unless $\deg_F(O) \mid n$; when it divides n, the summand is

$$\frac{1}{n/\deg_F(O)} = \frac{\deg_F(O)}{n}.$$

Filtering $\mathcal{O}_{\leq n}$ by this divisibility condition gives $\mathcal{O}(n)$ by Lemma 3.4.26. Pulling out the common scalar $1/n$ gives the formula. □

Proposition 3.5.12 (Logarithm of the orbit Euler product is the counting logarithm). *Uses:* def:weil-orbit-euler-product,def:weil-counting-log-series

Lean: Weil.FiniteOrbitFrobeniusData.log_orbitEulerProduct

One has

$$\mathrm{Log}(Z_{\mathrm{orb},D}) = S_D = \sum_{n\geq 1} N_n(D)\frac{T^n}{n}.$$

Proof. Uses: lemma:weil-orbit-euler-product-constant,lemma:weil-log-orbit-euler-product-coeff,lemma:weil-fixed-point-count-relevant-orbits

In degree zero, both sides vanish: formal logarithm has zero constant coefficient, and S_D does by definition. For $n \geq 1$, the preceding coefficient lemma gives

$$[T^n]\,\mathrm{Log}(Z_{\mathrm{orb},D}) = \frac{1}{n}\sum_{O\in\mathcal{O}(n)} \deg_F(O) = \frac{N_n(D)}{n},$$

where the second equality is Lemma 3.4.30. This is the defining coefficient of S_D. Apply PowerSeries.ext. □

Theorem 3.5.13 (Euler product equals the exponential counting series). *Uses:* def:weil-orbit-euler-product, def:weil-counting-zeta-series

Lean: Weil.FiniteOrbitFrobeniusData.orbitEulerProduct_eq_countingZeta

For every finite-orbit geometric-Frobenius datum,

$$Z_{\mathrm{cnt},D}(T) = Z_{\mathrm{orb},D}(T) = \prod_{O \in \mathrm{Orb}_F(\Omega)} (1 - T^{\deg_F(O)})^{-1}$$

as formal series in $1 + T\mathbf{Q}[[T]]$.

Proof. Uses: def:weil-counting-zeta-series, lemma:weil-orbit-euler-product-constant, prop:weil-log-orbit-euler-product, lemma:weil-formal-exp-log, lemma:weil-inverse-euler-factor-mul

Apply Exp to $\mathrm{Log}(Z_{\mathrm{orb},D}) = S_D$. Since the orbit product has constant coefficient one, Lemma 3.1.31 makes the left side $Z_{\mathrm{orb},D}$. The right side is $Z_{\mathrm{cnt},D}$ by definition. The last displayed product is the expansion of the orbit Euler product definition, because its factor $1 + u_O$ is $E_{\deg_F(O)} = (1 - T^{\deg_F(O)})^{-1}$ by Lemma 3.3.8. □

Corollary 3.5.14 (Integrality of the counting zeta series). *Uses:* def:weil-counting-zeta-series

Lean: Weil.FiniteOrbitFrobeniusData.countingZetaSeries_integral

The coefficients of $Z_{\mathrm{cnt},D}(T)$ *are integers.*

Proof. Uses: lemma:weil-inverse-euler-factor-coeff, lemma:weil-locally-finite-product-subring, thm:weil-orbit-euler-exponential

Every $u_O = E_{\deg_F(O)} - 1$ has integer coefficients by the inverse Euler factor coefficient formula. Apply Lemma 3.2.27 to the subring $\mathbf{Z} \subseteq \mathbf{Q}$; it gives integer coefficients for $Z_{\mathrm{orb},D}$. Transfer them to $Z_{\mathrm{cnt},D}$ using Theorem 3.5.13. □

Definition 3.5.15 (Positive natural degree indices). *Uses:*

Lean: Weil.PositiveDegreeIndex

Define the index type

$$\mathbf{N}_{>0} := \{m : \mathbf{N} \mid 1 \le m\}.$$

Its coercion to $\mathbf{N}$ is the subtype projection.

Definition 3.5.16 (Raw degree-grouped Euler-factor family). *Uses:* def:weil-positive-natural-degree-index, def:weil-frobenius-orbit-count, def:weil-inverse-euler-factor

Lean: Weil.FiniteOrbitFrobeniusData.degreeEulerFactorFamily

For $m \in \mathbf{N}_{>0}$, using its subtype projection whenever a natural-number degree or exponent is required, define

$$v_m(T) := E_m(T)^{a_m(D)} - 1.$$

Lemma 3.5.17 (Degree-grouped Euler factors have positive order). *Uses:* def:weil-degree-euler-factor-family, def:weil-positive-series

Lean: Weil.FiniteOrbitFrobeniusData.degreeEulerFactorFamily_positive

For every $m \in \mathbf{N}_{>0}$, *one has* $c(v_m) = 0$.

Proof. Uses: def:weil-degree-euler-factor-family, lemma:weil-inverse-euler-factor-constant

The constant coefficient of E_m is one. Repeated use of the degree-zero coefficient formula for multiplication gives $c(E_m^{a_m(D)}) = 1^{a_m(D)} = 1$. Subtracting the constant series 1 gives $c(v_m) = 0$. □

Lemma 3.5.18 (Local finiteness of the degree-grouped Euler family). *Uses:* def:weil-degree-euler-factor-family, def:weil-series-agree-through

Lean: Weil.FiniteOrbitFrobeniusData.degreeEulerFactorFamily_locallyFinite

For every $N \ge 0$, *only finitely many* $m \in \mathbf{N}_{>0}$ *satisfy* $v_m \not\equiv_N 0$.

Proof. Uses: lemma:weil-inverse-euler-factor-initial-vanishing, lemma:weil-series-agree-through-operations

Fix N. If $m > N$, then $E_m \equiv_N 1$. Repeated application of the multiplicative agreement lemma gives $E_m^{a_m(D)} \equiv_N 1^{a_m(D)} = 1$, so $v_m \equiv_N 0$. Thus every active index lies in the finite set $\{m \in \mathbf{N} \mid 1 \le m \le N\}$. □

Definition 3.5.19 (Packaged degree-grouped Euler family). Uses: `def:weil-positive-natural-degree-index`, `def:weil-degree-euler-factor-family`, `def:weil-t-adic-locally-finite-family`

Constructed by: `lemma:weil-degree-euler-factor-family-positive`, `lemma:weil-degree-euler-factor-family-locally-finite`

Lean: `Weil.FiniteOrbitFrobeniusData.degreeEulerFamily`

Define $\mathcal{G}_D$ to be the locally finite family indexed by positive integers, with component $v_m = E_m^{a_m(D)} - 1$. Its positive-order and local-finiteness fields are the two previously constructed certificates recorded by the constructor metadata above.

Definition 3.5.20 (Degree-grouped Euler product). Uses: `def:weil-degree-euler-locally-finite-family`, `def:weil-locally-finite-product`

Lean: `Weil.FiniteOrbitFrobeniusData.degreeEulerProduct`

Define

$$Z_{\deg,D}(T) := \prod_{m\geq 1}^{\mathrm{lf}} E_m(T)^{a_m(D)} = \prod_{m\geq 1}^{\mathrm{lf}} (1-T^m)^{-a_m(D)}.$$

Definition 3.5.21 (Finite positive-degree index set). Uses: `def:weil-positive-natural-degree-index`

Lean: `Weil.positiveDegreeIndexFinset`

For $N \in \mathbf{N}$, define the finset

$$I_N^+ := \{m \in \mathbf{N}_{>0} \mid (m:\mathbf{N}) \leq N\}.$$

Concretely, this is obtained by mapping `Finset.Icc 1 N` into $\mathbf{N}_{>0}$ with the subtype constructor and its lower-bound proof.

Lemma 3.5.22 (Membership in the finite positive-degree index set). *Uses:* *`def:weil-positive-degree-index-finset`*

Lean: *`Weil.mem_positiveDegreeIndexFinset`*

For $m \in \mathbf{N}_{>0}$,

$$m \in I_N^+ \quad\Longleftrightarrow\quad (m:\mathbf{N}) \leq N.$$

Proof. Uses: `def:weil-positive-degree-index-finset`

Unfold I_N^+. The membership theorem for the mapped finset reduces the left side to $1 \leq (m:\mathbf{N}) \leq N$. The lower inequality is the subtype field of m; deleting it leaves exactly $(m:\mathbf{N}) \leq N$. □

Lemma 3.5.23 (A bounded-orbit degree fiber is the one-degree orbit finset). *Uses:* *`def:weil-positive-degree-index-finset`, `def:weil-bounded-frobenius-orbit-finset`, `def:weil-degree-frobenius-orbit-finset`, `def:weil-frobenius-orbit-degree`*

Lean: *`Weil.FiniteOrbitFrobeniusData.boundedOrbit_degreeFiber`*

If $N \geq 1$ and $m \in I_N^+$, then

$$\mathcal{O}_{\leq N}.\,\mathrm{filter}(O \mapsto \deg_F(O) = (m:\mathbf{N})) = \mathcal{O}_m.$$

Proof. Uses: `lemma:weil-positive-degree-index-finset-mem`, `lemma:weil-bounded-frobenius-orbit-finset-mem`, `lemma:weil-degree-frobenius-orbit-finset-mem`

Apply `Finset.ext` and fix an orbit O. Membership on the left is equivalent to $\deg_F(O) \leq N$ and $\deg_F(O) = m$. Membership on the right is equivalent to $\deg_F(O) = m$. The forward implication discards the first conjunct. For the reverse implication, substitute $\deg_F(O) = m$ and use $m \leq N$, obtained from Lemma 3.5.22. □

Lemma 3.5.24 (Finite bounded-orbit products regroup by degree). *Uses:* *`def:weil-positive-degree-index-finset`, `def:weil-bounded-frobenius-orbit-finset`, `def:weil-frobenius-orbit-degree`, `def:weil-frobenius-orbit-count`, `def:weil-inverse-euler-factor`*

Lean: *`Weil.FiniteOrbitFrobeniusData.boundedOrbitProduct_regroup_degree`*

If $N \geq 1$, then

$$\prod_{O\in\mathcal{O}_{\leq N}} E_{\deg_F(O)} = \prod_{m\in I_N^+} E_m^{a_m(D)}.$$

Proof. Uses: `def:weil-frobenius-orbit-count`, `lemma:weil-frobenius-orbit-degree-positive`, `lemma:weil-positive-degree-index-finset-mem`, `lemma:weil-bounded-frobenius-orbit-finset-mem`, `lemma:weil-bounded-orbit-degree-fiber`

Apply `Finset.prod_fiberwise_of_maps_to'` with

$$s = \mathcal{O}_{\le N}, \quad t = I_N^+, \quad g(O) = \langle \deg_F(O), 1 \le \deg_F(O) \rangle, \quad f(m) = E_m.$$

The positivity certificate in g is Lemma 3.4.12. If $O \in s$, the bounded-orbit membership lemma gives $\deg_F(O) \le N$; the positive index membership lemma therefore proves $g(O) \in t$, which is the exact `MapsTo` argument of `Finset.prod_fiberwise_of_maps_to'`. For each $m \in t$, replace its filtered fiber by $\mathcal{O}_m$ using Lemma 3.5.23. The exact identity `Finset.prod_const` turns the product of the constant factor E_m over this fiber into $E_m^{\#\mathcal{O}_m}$. Unfold Definition 3.4.24 to replace the exponent by $a_m(D)$, and reverse the fiberwise-product equality. □

Lemma 3.5.25 (Active degree indices are bounded by the truncation degree). *Uses:* `def:weil-active-index-finset`, `def:weil-degree-euler-factor-family`, `def:weil-positive-degree-index-finset`

Lean: `Weil.FiniteOrbitFrobeniusData.degreeEulerFamily_activeFinset_subset`

For every $N \ge 0$,

$$A_N(\mathcal{G}_D) \subseteq I_N^+.$$

Proof. Uses: `def:weil-degree-euler-factor-family`, `lemma:weil-active-index-finset-mem`, `lemma:weil-inverse-euler-factor-initial-vanishing`, `lemma:weil-series-agree-through-operations`, `lemma:weil-positive-degree-index-finset-mem`

Let $m \in A_N(\mathcal{G}_D)$ and suppose $m > N$. The initial-vanishing lemma gives $E_m \equiv_N 1$. Induction on the natural exponent $a_m(D)$, using the multiplicative agreement lemma at the successor step, gives $E_m^{a_m(D)} \equiv_N 1$. Subtracting one gives $v_m = E_m^{a_m(D)} - 1 \equiv_N 0$, contradicting the active-index membership criterion. Hence $m \le N$, which is membership in I_N^+ by Lemma 3.5.22. □

Lemma 3.5.26 (The degree Euler product has the bounded-degree truncation). *Uses:* `def:weil-series-agree-through`, `def:weil-degree-euler-product`, `def:weil-positive-degree-index-finset`

Lean: `Weil.FiniteOrbitFrobeniusData.degreeEulerProduct_agree_boundedProduct`

For every $N \ge 0$,

$$Z_{\deg,D} \equiv_N \prod_{m \in I_N^+} E_m^{a_m(D)}.$$

Proof. Uses: `def:weil-degree-euler-factor-family`, `lemma:weil-degree-euler-active-finset-subset`, `lemma:weil-locally-finite-product-agree-prescribed-finset`

Apply Lemma 3.2.16 to the locally finite family $\mathcal{G}_D$ and the prescribed finset I_N^+. Its inclusion hypothesis is Lemma 3.5.25. Unfold the component $v_m = E_m^{a_m(D)} - 1$; then $1 + v_m = E_m^{a_m(D)}$ in every factor. □

Lemma 3.5.27 (The orbit Euler product has the bounded-orbit truncation). *Uses:* `def:weil-series-agree-through`, `def:weil-orbit-euler-product`, `def:weil-bounded-frobenius-orbit-finset`

Lean: `Weil.FiniteOrbitFrobeniusData.orbitEulerProduct_agree_boundedProduct`

If $N \ge 1$, then

$$Z_{\mathrm{orb},D} \equiv_N \prod_{O \in \mathcal{O}_{\le N}} E_{\deg_F(O)}.$$

Proof. Uses: `def:weil-orbit-euler-factor-family`, `def:weil-truncated-product`, `lemma:weil-orbit-euler-active-finset`, `lemma:weil-locally-finite-product-agree`

The active-orbit lemma identifies $A_N(\mathcal{E}_D)$ with $\mathcal{O}_{\le N}$. Substitute this equality into the prescribed-truncation lemma for $\mathcal{E}_D$. Unfold $u_O = E_{\deg_F(O)} - 1$ in the finite product, so that its factor $1 + u_O$ is $E_{\deg_F(O)}$. □

Proposition 3.5.28 (Regrouping the orbit Euler product by degree). *Uses:* `def:weil-orbit-euler-product`, `def:weil-degree-euler-product`

Lean: `Weil.FiniteOrbitFrobeniusData.orbitEulerProduct_eq_degreeEulerProduct`

One has

$$Z_{\mathrm{orb},D}(T) = Z_{\deg,D}(T).$$

Proof. Uses: `lemma:weil-orbit-euler-product-constant,lemma:weil-positive-degree-index-finset-mem,lemma:weil-degree-euler-product-agree-bounded-product,lemma:weil-orbit-euler-product-agree-bounded-product,lemma:weil-bounded-orbit-product-regroup-by-degree`

Apply `PowerSeries.ext` and fix a coefficient degree N. If $N = 0$, the orbit product has constant coefficient one by Lemma 3.5.7. The degree-product agreement lemma at $N = 0$ identifies the other constant coefficient with that of the empty product over I_0^+, hence also with one. Now assume $N \geq 1$. The orbit-product agreement lemma identifies the degree-N coefficient of $Z_{\mathrm{orb},D}$ with that of $\prod_{O \in \mathcal{O}_{\leq N}} E_{\deg_F(O)}$. The finite regrouping lemma replaces this series by $\prod_{m \in I_N^+} E_m^{a_m(D)}$. Finally, the degree-product agreement lemma identifies its degree-N coefficient with that of $Z_{\deg,D}$. Thus the two degree-N coefficients agree in both cases. □

Corollary 3.5.29 (Three equivalent forms of the orbit zeta series)**.** Uses: *def:weil-counting-zeta-series, def:weil-orbit-euler-product, def:weil-degree-euler-product, def:weil-power-series-over-subring*

Lean: *Weil.FiniteOrbitFrobeniusData.zeta_three_forms*

For every finite-orbit datum,

$$\operatorname{Exp}\left(\sum_{n\geq 1} N_n(D)\frac{T^n}{n}\right) = \prod_{O}(1-T^{\deg_F(O)})^{-1} = \prod_{m\geq 1}(1-T^m)^{-a_m(D)}.$$

All equalities and all locally finite products in this statement are taken in $\mathbf{Q}[[T]]$*. Moreover,*

$$c(Z_{\mathrm{cnt},D}) = 1, \qquad \mathit{HasCoefficientsIn}\big((\mathrm{algebraMap}_{\mathbf{Z},\mathbf{Q}}).\mathrm{range}, Z_{\mathrm{cnt},D}\big).$$

Thus the conclusion presently asserts the integer-coefficient predicate inside $\mathbf{Q}[[T]]$*; it does not assert a typed lift in* $\mathbf{Z}[[T]]$*.*

Proof. Uses: `def:weil-counting-zeta-series,def:weil-power-series-over-subring,thm:weil-orbit-euler-exponential,prop:weil-orbit-euler-product-grouped,cor:weil-counting-zeta-integral,lemma:weil-counting-log-series-positive,lemma:weil-formal-exp-constant`

The first equality is Theorem 3.5.13; the second is Proposition 3.5.28. After unfolding Definition 3.2.26, the required coefficient predicate is exactly Corollary 3.5.14. Finally, $Z_{\mathrm{cnt},D} = \operatorname{Exp}(S_D)$ by definition, and Lemma 3.5.9 gives $c(S_D) = 0$. Hence Lemma 3.1.5 gives $c(Z_{\mathrm{cnt},D}) = 1$. □

3.6 Abstract finite-extension and closed-point interfaces

The next definitions isolate the exact combinatorial output required from scheme theory. They do not assert that a scheme supplies this output.

> **Status.** Exact geometric obligations exported by this abstract module. The interfaces below make no scheme-theoretic assertion by themselves. The later TraceCore3 scheme module constructs their finite-type-scheme instances and discharges the four separately indexed geometric interface obligations; no field of an abstract interface is used there as a substitute for a geometric proof.

Definition 3.6.1 (Finite-extension point interface)**.** Uses: `def:weil-finite-orbit-frobenius-datum`

Lean: `Weil.FiniteExtensionPointInterface`

A finite-extension point interface P over a finite-orbit datum D consists, for every $n \geq 1$, of a finite type $\mathrm{Rat}_P(n)$ and a selected equivalence

$$\rho_n : \mathrm{Rat}_P(n) \xrightarrow{\sim} \{x \in \Omega \mid x \in \mathrm{Fix}_D(n)\}.$$

The intended later instance is $\mathrm{Rat}_P(n) = X_0(\mathbf{F}_{q^n})$.

Definition 3.6.2 (Finite-extension point count)**.** Uses: `def:weil-finite-extension-point-interface`

Lean: `Weil.FiniteExtensionPointInterface.pointCount`

For $n \geq 1$, define

$$N_n(P) := \mathrm{Fintype.card}(\mathrm{Rat}_P(n)).$$

Lemma 3.6.3 (Finite-extension point count equals fixed-point count). *Uses:* *def:weil-finite-extension-point-interface*, *def:weil-finite-extension-point-count*, *def:weil-frobenius-fixed-point-count*

Lean: *Weil.FiniteExtensionPointInterface.pointCount_eq_fixedPointCount*

For every $n \geq 1$,

$$N_n(P) = N_n(D).$$

Proof. Uses: `def:weil-finite-extension-point-interface`, `def:weil-finite-extension-point-count`, `def:weil-frobenius-fixed-point-count`

Apply `Fintype.card_congr` to the equivalence ρ_n stored in the interface. The target subtype has cardinal $\#\mathrm{Fix}_D(n)$ by `Fintype.card_coe`; this cardinal is $N_n(D)$ by definition. □

Definition 3.6.4 (Finite-extension counting logarithm). Uses: `def:weil-finite-extension-point-count`

Lean: `Weil.FiniteExtensionPointInterface.countingLogSeries`

Define $S_P(T) \in \mathbf{Q}[[T]]$ by

$$[T^0]S_P = 0, \qquad [T^n]S_P = \frac{N_n(P)}{n} \quad (n \geq 1).$$

Definition 3.6.5 (Finite-extension counting zeta series). Uses: `def:weil-finite-extension-counting-log-series`, `def:weil-formal-exp`

Lean: `Weil.FiniteExtensionPointInterface.countingZetaSeries`

Define

$$Z_P(T) := \mathrm{Exp}(S_P(T)) = \mathrm{Exp}\left(\sum_{n\geq 1} N_n(P)\frac{T^n}{n}\right).$$

Proposition 3.6.6 (Finite-extension and orbit counting zeta series agree). *Uses:* *def:weil-finite-extension-counting-zeta-series*, *def:weil-counting-zeta-series*

Lean: *Weil.FiniteExtensionPointInterface.countingZetaSeries_eq*

One has

$$Z_P(T) = Z_{\mathrm{cnt},D}(T).$$

Proof. Uses: `lemma:weil-finite-extension-point-count-eq-fixed`

The coefficient of degree zero in S_P and S_D is zero. In every positive degree n, the preceding lemma gives $N_n(P)/n = N_n(D)/n$. Thus $S_P = S_D$ by `PowerSeries.ext`. Apply the same function Exp to this equality. □

Definition 3.6.7 (Closed-point–orbit interface). Uses: `def:weil-frobenius-orbit-type`, `def:weil-frobenius-orbit-degree`

Lean: `Weil.ClosedPointOrbitInterface`

A closed-point–orbit interface C over D consists of:

(i) a type $\mathrm{ClPt}(C)$;

(ii) a function $\deg_C : \mathrm{ClPt}(C) \to \mathbf{N}$ and a proof $\deg_C(c) \geq 1$ for every c;

(iii) a selected equivalence

$$\theta_C : \mathrm{ClPt}(C) \xrightarrow{\sim} \mathrm{Orb}_F(\Omega);$$

(iv) for every c, the equality

$$\deg_C(c) = \deg_F(\theta_C(c)).$$

The intended later source type is the type of closed points of a finite-type $\mathbf{F}_q$-scheme.

Definition 3.6.8 (Finset of closed points of one degree). Uses: `def:weil-closed-point-orbit-interface`, `def:weil-degree-frobenius-orbit-finset`

Lean: `Weil.ClosedPointOrbitInterface.degreeClosedPointFinset`

For $m \geq 1$, define

$$\mathcal{C}_m := \mathcal{O}_m.\,\mathrm{preimage}(\theta_C)\big(\theta_C \text{ is injective}\big).$$

Equivalently, this is the finset transported from $\mathcal{O}_m$ through the inverse equivalence θ_C^{-1}.

Lemma 3.6.9 (Membership in the one-degree closed-point finset). *Uses:* *def:weil-degree-closed-point-finset*

Lean: *Weil.ClosedPointOrbitInterface.mem_degreeClosedPointFinset*

For $m \geq 1$,

$$c \in \mathcal{C}_m \quad \Longleftrightarrow \quad \deg_C(c) = m.$$

Proof. Uses: def:weil-closed-point-orbit-interface,lemma:weil-degree-frobenius-orbit-finset-mem

By the finset preimage membership theorem, $c \in \mathcal{C}_m$ if and only if $\theta_C(c) \in \mathcal{O}_m$. The orbit membership criterion turns this into $\deg_F(\theta_C(c)) = m$. Replace this degree by $\deg_C(c)$ using the compatibility field of the interface. □

Definition 3.6.10 (Number of closed points of one degree). Uses: def:weil-degree-closed-point-finset

Lean: Weil.ClosedPointOrbitInterface.closedPointCount

For $m \geq 1$, define $b_m(C) := \#\mathcal{C}_m$.

Definition 3.6.11 (Forward candidate on a fixed closed-point degree). Uses: def:weil-closed-point-orbit-interface,def:weil-degree-closed-point-finset

Lean: Weil.ClosedPointOrbitInterface.degreeForwardCandidate

For $m \geq 1$, define

$$f_m : \{c \mid c \in \mathcal{C}_m\} \longrightarrow \mathrm{Orb}_F(\Omega), \qquad f_m(c) := \theta_C(c).$$

Lemma 3.6.12 (The fixed-degree forward candidate lands in the orbit fiber). *Uses:* *def:weil-degree-closed-point-forward-candidate,def:weil-degree-frobenius-orbit-finset*

Lean: *Weil.ClosedPointOrbitInterface.degreeForwardCandidate_mem*

If $c \in \mathcal{C}_m$, then $f_m(c) \in \mathcal{O}_m$.

Proof. Uses: def:weil-degree-closed-point-forward-candidate,def:weil-degree-closed-point-finset

Membership in $\mathcal{C}_m$ is, by its preimage definition, exactly membership of $\theta_C(c) = f_m(c)$ in $\mathcal{O}_m$; unfolding the forward candidate identifies its value with $\theta_C(c)$. □

Definition 3.6.13 (Forward map on a fixed closed-point degree). Uses: def:weil-degree-closed-point-forward-candidate,def:weil-degree-frobenius-orbit-finset

Constructed by: lemma:weil-degree-closed-point-forward-candidate-mem

Lean: Weil.ClosedPointOrbitInterface.degreeForwardMap

Define

$$\bar{f}_m : \{c \mid c \in \mathcal{C}_m\} \longrightarrow \{O \mid O \in \mathcal{O}_m\}$$

by pairing $f_m(c)$ with the landing certificate recorded above.

Definition 3.6.14 (Inverse candidate on a fixed orbit degree). Uses: def:weil-closed-point-orbit-interface,def:weil-degree-frobenius-orbit-finset

Lean: Weil.ClosedPointOrbitInterface.degreeInverseCandidate

For $m \geq 1$, define

$$g_m : \{O \mid O \in \mathcal{O}_m\} \longrightarrow \mathrm{ClPt}(C), \qquad g_m(O) := \theta_C^{-1}(O).$$

Lemma 3.6.15 (The fixed-degree inverse candidate lands in the closed-point fiber). *Uses:* *def:weil-degree-closed-point-inverse-candidate,def:weil-degree-closed-point-finset*

Lean: *Weil.ClosedPointOrbitInterface.degreeInverseCandidate_mem*

If $O \in \mathcal{O}_m$, then $g_m(O) \in \mathcal{C}_m$.

Proof. Uses: def:weil-degree-closed-point-inverse-candidate,def:weil-degree-closed-point-finset

Unfolding the inverse candidate gives $g_m(O) = \theta_C^{-1}(O)$. Membership in $\mathcal{C}_m$ therefore means $\theta_C(\theta_C^{-1}(O)) \in \mathcal{O}_m$. The right-inverse identity of θ_C reduces this to the given membership $O \in \mathcal{O}_m$. □

Definition 3.6.16 (Inverse map on a fixed orbit degree). Uses: `def:weil-degree-closed-point-inverse-candidate`, `def:weil-degree-closed-point-finset`

Constructed by: `lemma:weil-degree-closed-point-inverse-candidate-mem`

Lean: `Weil.ClosedPointOrbitInterface.degreeInverseMap`

Define

$$\bar{g}_m : \{O \mid O \in \mathcal{O}_m\} \longrightarrow \{c \mid c \in \mathcal{C}_m\}$$

by pairing $g_m(O)$ with the landing certificate recorded above.

Lemma 3.6.17 (The inverse map is a left inverse of the forward map). *Uses:* *def:weil-degree-closed-point-forward-map*, *def:weil-degree-closed-point-inverse-map*

Lean: *Weil.ClosedPointOrbitInterface.degreeMaps_leftInverse*

For every $c \in \mathcal{C}_m$, one has $\bar{g}_m(\bar{f}_m(c)) = c$.

Proof. Uses: `def:weil-closed-point-orbit-interface`, `def:weil-degree-closed-point-forward-map`, `def:weil-degree-closed-point-inverse-map`

Unfold the forward and inverse maps, then apply subtype extensionality. The underlying equality is $\theta_C^{-1}(\theta_C(c)) = c$, which is the left-inverse field of the equivalence θ_C. □

Lemma 3.6.18 (The inverse map is a right inverse of the forward map). *Uses:* *def:weil-degree-closed-point-forward-map*, *def:weil-degree-closed-point-inverse-map*

Lean: *Weil.ClosedPointOrbitInterface.degreeMaps_rightInverse*

For every $O \in \mathcal{O}_m$, one has $\bar{f}_m(\bar{g}_m(O)) = O$.

Proof. Uses: `def:weil-closed-point-orbit-interface`, `def:weil-degree-closed-point-forward-map`, `def:weil-degree-closed-point-inverse-map`

Unfold the forward and inverse maps, then apply subtype extensionality. The underlying equality is $\theta_C(\theta_C^{-1}(O)) = O$, which is the right-inverse field of the equivalence θ_C. □

Definition 3.6.19 (Equivalence of fixed-degree closed points and orbits). Uses: `def:weil-degree-closed-point-forward-map`, `def:weil-degree-closed-point-inverse-map`

Constructed by: `lemma:weil-degree-closed-point-maps-left-inverse`, `lemma:weil-degree-closed-point-maps-right-inverse`

Lean: `Weil.ClosedPointOrbitInterface.degreeEquiv`

Define

$$\bar{\theta}_{C,m} : \{c \mid c \in \mathcal{C}_m\} \simeq \{O \mid O \in \mathcal{O}_m\}$$

with forward map $\bar{f}_m$, inverse map $\bar{g}_m$, and the two inverse certificates recorded above.

Lemma 3.6.20 (Closed-point degree counts equal orbit degree counts). *Uses:* *def:weil-closed-point-count*, *def:weil-frobenius-orbit-count*

Lean: *Weil.ClosedPointOrbitInterface.closedPointCount_eq_orbitCount*

For $m \geq 1$,

$$b_m(C) = a_m(D).$$

Proof. Uses: `def:weil-closed-point-count`, `def:weil-frobenius-orbit-count`, `def:weil-degree-closed-point-orbit-equiv`

Apply `Fintype.card_congr` to $\bar{\theta}_{C,m}$. Unfolding the two count definitions identifies its source and target cardinals with $b_m(C)$ and $a_m(D)$. □

Definition 3.6.21 (Raw closed-point Euler-factor family). Uses: `def:weil-closed-point-orbit-interface`, `def:weil-inverse-euler-factor-positive-part`

Lean: `Weil.ClosedPointOrbitInterface.eulerFactorFamily`

For a closed point c, define

$$w_c(T) := E_{\deg_C(c)}(T) - 1.$$

Lemma 3.6.22 (Closed-point Euler factors have positive order). *Uses:* *def:weil-closed-point-euler-factor-family*, *def:weil-positive-series*

Lean: *Weil.ClosedPointOrbitInterface.eulerFactorFamily_positive*

Every w_c has constant coefficient zero.

Proof. Uses: `def:weil-closed-point-orbit-interface,lemma:weil-inverse-euler-factor-constant`
The interface gives $\deg_C(c) \geq 1$, so the inverse Euler-factor constant lemma gives $c(E_{\deg_C(c)}) = 1$. Subtracting one gives $c(w_c) = 0$. □

Lemma 3.6.23 (Closed-point components transport to orbit components). Uses: *def:weil-closed-point-orbit-interface, def:weil-orbit-euler-factor-family, def:weil-closed-point-euler-factor-family*

Lean: *Weil.ClosedPointOrbitInterface.eulerFactorFamily_component_transport*
For every closed point c,

$$u_{\theta_C(c)} = w_c \qquad \text{in } \mathbf{Q}[[T]].$$

Proof. Uses: `def:weil-closed-point-orbit-interface,def:weil-orbit-euler-factor-family,def:weil-closed-point-euler-factor-family`
Unfold the two raw component Definitions. The left side is $E_{\deg_F(\theta_C(c))} - 1$ and the right side is $E_{\deg_C(c)} - 1$. Rewrite the former with the degree-compatibility field $\deg_F(\theta_C(c)) = \deg_C(c)$ of the interface. The two displayed power series are then definitionally identical. □

Lemma 3.6.24 (Active closed points map into active orbits). Uses: *def:weil-series-agree-through, def:weil-closed-point-orbit-interface, def:weil-orbit-euler-factor-family, def:weil-closed-point-euler-factor-family*

Lean: *Weil.ClosedPointOrbitInterface.activeClosed_image_subset_activeOrbit*
For every $N \geq 0$,

$$\theta_C''\{c \mid w_c \not\equiv_N 0\} \subseteq \{O \mid u_O \not\equiv_N 0\}.$$

Proof. Uses: `def:weil-series-agree-through,lemma:weil-closed-point-euler-component-transport`
Take O in the image. By image membership there is a closed point c with $w_c \not\equiv_N 0$ and $O = \theta_C(c)$. Substitute this equality and then substitute $u_{\theta_C(c)} = w_c$ from the component-transport lemma. The target non-agreement is exactly the source non-agreement. □

Lemma 3.6.25 (The active closed-point image is finite). Uses: *def:weil-series-agree-through, def:weil-closed-point-orbit-interface, def:weil-orbit-euler-factor-family, def:weil-closed-point-euler-factor-family*

Lean: *Weil.ClosedPointOrbitInterface.activeClosed_image_finite*
For every $N \geq 0$, the set

$$\theta_C''\{c \mid w_c \not\equiv_N 0\}$$

is finite.

Proof. Uses: `lemma:weil-closed-point-active-image-subset,lemma:weil-orbit-euler-factor-family-locally-finite`
The orbit-family local-finiteness lemma makes $\{O \mid u_O \not\equiv_N 0\}$ finite. Apply `Set.Finite.subset` once, with the subset relation supplied by the preceding active-image lemma. □

Lemma 3.6.26 (Local finiteness of the closed-point Euler family). Uses: *def:weil-closed-point-euler-factor-family, def:weil-series-agree-through*

Lean: *Weil.ClosedPointOrbitInterface.eulerFactorFamily_locallyFinite*
For every $N \geq 0$, only finitely many $c \in \mathrm{ClPt}(C)$ satisfy $w_c \not\equiv_N 0$.

Proof. Uses: `def:weil-closed-point-orbit-interface,lemma:weil-closed-point-active-image-finite`
Apply `Set.Finite.of_finite_image`. Its first argument is the finiteness of $\theta_C''\{c \mid w_c \not\equiv_N 0\}$ supplied by the preceding active-image-finiteness lemma. Its second argument is the restricted injectivity proof θ_C`.injective.injOn` supplied by the equivalence field. □

Definition 3.6.27 (Packaged closed-point Euler family). Uses: `def:weil-closed-point-euler-factor-family,def:weil-t-adic-locally-finite-family`

Constructed by: `lemma:weil-closed-point-euler-factor-family-positive,lemma:weil-closed-point-euler-factor-family-locally-finite`

Lean: `Weil.ClosedPointOrbitInterface.eulerFamily`
Define $\mathcal{E}_C$ to be the locally finite family indexed by $\mathrm{ClPt}(C)$, with component w_c. Its positive-order and local-finiteness fields are the two previously constructed certificates recorded by the constructor metadata above.

Definition 3.6.28 (Closed-point Euler product). Uses: `def:weil-closed-point-euler-locally-finite-family`,`def:weil-locally-finite-product`

Lean: `Weil.ClosedPointOrbitInterface.eulerProduct`

Define

$$Z_{\mathrm{cl},C}(T) := \prod_{c\in \mathrm{ClPt}(C)}^{\mathrm{lf}} E_{\deg_C(c)}(T) = \prod_{c}^{\mathrm{lf}} (1-T^{\deg_C(c)})^{-1}.$$

Lemma 3.6.29 (Active closed points correspond exactly to active orbits). Uses: *def:weil-active-index-finset*, *def:weil-closed-point-euler-locally-finite-family*, *def:weil-orbit-euler-locally-finite-family*

Lean: *Weil.ClosedPointOrbitInterface.mem_activeFinset_iff*

For every $N \geq 0$ and every $c \in \mathrm{ClPt}(C)$,

$$c \in A_N(\mathcal{E}_C) \quad \Longleftrightarrow \quad \theta_C(c) \in A_N(\mathcal{E}_D).$$

Proof. Uses: `def:weil-closed-point-orbit-interface`,`def:weil-closed-point-euler-factor-family`, `def:weil-orbit-euler-factor-family`,`lemma:weil-active-index-finset-mem`

Active membership on the left is $E_{\deg_C(c)}-1 \not\equiv_N 0$, while active membership on the right is $E_{\deg_F(\theta_C(c))}-1 \not\equiv_N 0$. The degree-compatibility field of C identifies the two series, so the two predicates are equivalent. □

Lemma 3.6.30 (Closed-point and orbit truncated products agree). Uses: *def:weil-closed-point-orbit-interface*, *def:weil-closed-point-euler-locally-finite-family*, *def:weil-orbit-euler-locally-finite-family*, *def:weil-truncated-product*

Lean: *Weil.ClosedPointOrbitInterface.truncatedProduct_eq*

For every $N \geq 0$,

$$\Pi_N(\mathcal{E}_C) = \Pi_N(\mathcal{E}_D).$$

Proof. Uses: `def:weil-closed-point-orbit-interface`,`def:weil-closed-point-euler-factor-family`, `def:weil-orbit-euler-factor-family`,`def:weil-truncated-product`,`lemma:weil-closed-point-active-iff-orbit-active`

Unfold the two truncated products. Apply `Finset.prod_equiv` with the equivalence $\theta_C : \mathrm{ClPt}(C) \simeq \mathrm{Orb}_F(\Omega)$, source finset $A_N(\mathcal{E}_C)$, and target finset $A_N(\mathcal{E}_D)$. Its membership-equivalence argument is the preceding active-membership lemma. For a source member c, the two factors are $E_{\deg_C(c)}$ and $E_{\deg_F(\theta_C(c))}$; the degree-compatibility field of C identifies them. □

Lemma 3.6.31 (Closed-point and orbit Euler products agree coefficientwise). Uses: *def:weil-closed-point-euler-product*, *def:weil-orbit-euler-product*

Lean: *Weil.ClosedPointOrbitInterface.coeff_eulerProduct_eq*

For every $N \geq 0$,

$$[T^N]Z_{\mathrm{cl},C}(T) = [T^N]Z_{\mathrm{orb},D}(T).$$

Proof. Uses: `lemma:weil-locally-finite-product-coeff`,`lemma:weil-closed-point-truncated-product-reindex`

The locally finite product coefficient formula identifies the two displayed coefficients with the degree-N coefficients of $\Pi_N(\mathcal{E}_C)$ and $\Pi_N(\mathcal{E}_D)$, respectively. Replace the first truncated product by the second using the preceding reindexing lemma. □

Proposition 3.6.32 (Reindexing the closed-point product by Frobenius orbits). Uses: *def:weil-closed-point-euler-product*, *def:weil-orbit-euler-product*

Lean: *Weil.ClosedPointOrbitInterface.eulerProduct_eq_orbitEulerProduct*

One has

$$Z_{\mathrm{cl},C}(T) = Z_{\mathrm{orb},D}(T).$$

Proof. Uses: `lemma:weil-closed-point-euler-product-coeff-reindex`

Apply `PowerSeries.ext`. The required equality in every degree is the preceding coefficient-reindexing lemma. □

Theorem 3.6.33 (Closed-point Euler product and finite-extension point counts). *Uses:* *def:weil-finite-extension-counting-zeta-series, def:weil-closed-point-euler-product, def:weil-power-series-over-subring*

Lean: *Weil.ClosedPointOrbitInterface.eulerProduct_eq_countingZeta*

Given both a finite-extension point interface P and a closed-point–orbit interface C over the same datum D,

$$\operatorname{Exp}\left(\sum_{n\geq 1} \#\operatorname{Rat}_P(n)\frac{T^n}{n}\right) = \prod_{c\in \operatorname{ClPt}(C)} (1-T^{\deg_C(c)})^{-1}$$

as an equality in $\mathbf{Q}[[T]]$. In addition,

$$c(Z_P) = 1, \qquad \mathit{HasCoefficientsIn}\big((\text{algebraMap}_{\mathbf{Z},\mathbf{Q}}).\text{range}, Z_P\big).$$

Thus the common rational series has constant coefficient one and integer coefficients; no typed object of $\mathbf{Z}[[T]]$ is asserted here.

Proof. Uses: `def:weil-power-series-over-subring,prop:weil-finite-extension-zeta-eq-orbit-zeta,thm:weil-orbit-euler-exponential,prop:weil-closed-point-euler-product-reindex,cor:weil-counting-zeta-integral,lemma:weil-orbit-euler-product-constant`

Proposition 3.6.6 identifies Z_P with $Z_{\mathrm{cnt},D}$. Theorem 3.5.13 identifies it with $Z_{\mathrm{orb},D}$, and Proposition 3.6.32 identifies that with the closed-point product. After unfolding Definition 3.2.26, the integer-coefficient predicate transfers from Corollary 3.5.14 across these equalities. The common value equals $Z_{\mathrm{orb},D}$, whose constant coefficient is one by Lemma 3.5.7; hence $c(Z_P) = 1$. □

Corollary 3.6.34 (Closed points recovered from finite-extension point counts). *Uses:* *def:weil-finite-extension-point-count, def:weil-closed-point-count*

Lean: *Weil.ClosedPointOrbitInterface.closedPointCount_eq_mobius*

For $m \geq 1$,

$$b_m(C) = \frac{1}{m}\sum_{r\mid m} \mu(r) N_{m/r}(P).$$

Proof. Uses: `lemma:weil-finite-extension-point-count-eq-fixed,lemma:weil-closed-point-count-eq-orbit-count,cor:weil-orbit-count-mobius-recovery`

Replace $b_m(C)$ by $a_m(D)$, apply the orbit Möbius recovery formula, and replace every $N_{m/r}(D)$ by $N_{m/r}(P)$ using the finite-extension point-count equality. □

3.7 Integral Euler-family packages for the literal lift

Throughout this section the coefficient ring is exactly $\mathbf{Z}$. These declarations instantiate only the coefficient-ring-generic definitions and lemmas above; no formal logarithm, division by a positive integer, or $\mathbf{Q}$-algebra structure is used.

Definition 3.7.1 (Raw integer orbit Euler-factor family). Uses: `def:weil-frobenius-orbit-type, def:weil-frobenius-orbit-degree, def:weil-integral-inverse-euler-factor`

Lean: `Weil.FiniteOrbitFrobeniusData.integerOrbitEulerFactorFamily`

For $O \in \operatorname{Orb}_F(\Omega)$, define the series

$$u_O^{\mathbf{Z}}(T) := E^{\mathbf{Z}}_{\deg_F(O)}(T) - 1 \in \mathbf{Z}[[T]],$$

where the superscript records that the inverse Euler factor is formed over $\mathbf{Z}$.

Lemma 3.7.2 (Positive order of every integer orbit component). *Uses:* *def:weil-integer-orbit-euler-factor-family, def:weil-positive-series*

Lean: *Weil.FiniteOrbitFrobeniusData.integerOrbitEulerFactorFamily_positive*

For every orbit O, one has $c(u_O^{\mathbf{Z}}) = 0$.

Proof. Uses: `def:weil-integer-orbit-euler-factor-family,lemma:weil-frobenius-orbit-degree-positive,lemma:weil-integral-inverse-euler-factor-initial-vanishing`

The orbit degree $m = \deg_F(O)$ is positive. Instantiate the constant- coefficient conclusion of the raw integral initial-vanishing lemma at this m. Its series $E_m^{\mathbf{Z}} - 1$ is exactly $u_O^{\mathbf{Z}}$ after unfolding the raw component Definition. □

Lemma 3.7.3 (Local finiteness of the integer orbit components). *Uses:* *def:weil-integer-orbit-euler-factor-family, def:weil-series-agree-through*

Lean: *Weil.FiniteOrbitFrobeniusData.integerOrbitEulerFactorFamily_locallyFinite*

For every $N \geq 0$, the set

$$\left\{O \in \mathrm{Orb}_F(\Omega) \mid u_O^{\mathbf{Z}} \not\equiv_N 0\right\}$$

is finite.

Proof. Uses: def:weil-integer-orbit-euler-factor-family,lemma:weil-frobenius-orbit-degree-positive,lemma:weil-integral-inverse-euler-factor-initial-vanishing,lemma:weil-bounded-frobenius-orbit-finset-mem

If $N < \deg_F(O)$, the raw integral initial-vanishing lemma gives $u_O^{\mathbf{Z}} \equiv_N 0$. Hence every active orbit has degree at most N. For $N = 0$ there is no such orbit because orbit degrees are positive. For $N \geq 1$, the bounded-orbit membership lemma places every active orbit in the explicit finite finset $\mathcal{O}_{\leq N}$. □

Definition 3.7.4 (Packaged integer orbit Euler family). Uses: def:weil-integer-orbit-euler-factor-family,def:weil-t-adic-locally-finite-family

Constructed by: lemma:weil-integer-orbit-euler-factor-family-positive,lemma:weil-integer-orbit-euler-factor-family-locally-finite

Lean: Weil.FiniteOrbitFrobeniusData.integerOrbitEulerFamily

Define $\mathcal{E}_D^{\mathbf{Z}}$ by assigning the record fields

$$\begin{aligned} I &= \mathrm{Orb}_F(\Omega), \\ u(O) &= u_O^{\mathbf{Z}}, \\ h_{\mathrm{pos}} &= \text{the preceding positive-order proof}, \\ h_{\mathrm{fin}} &= \text{the preceding local-finiteness proof}. \end{aligned}$$

No certificate field is proved inside this Definition.

Definition 3.7.5 (Raw integer degree-grouped Euler-factor family). Uses: def:weil-positive-natural-degree-index,def:weil-frobenius-orbit-count,def:weil-integral-inverse-euler-factor

Lean: Weil.FiniteOrbitFrobeniusData.integerDegreeEulerFactorFamily

For $m \in \mathbf{N}_{>0}$, define

$$v_m^{\mathbf{Z}}(T) := \left(E_m^{\mathbf{Z}}(T)\right)^{a_m(D)} - 1 \in \mathbf{Z}[[T]].$$

Lemma 3.7.6 (Positive order of every integer degree component). *Uses:* *def:weil-integer-degree-euler-factor-family, def:weil-positive-series*

Lean: *Weil.FiniteOrbitFrobeniusData.integerDegreeEulerFactorFamily_positive*

For every $m \in \mathbf{N}_{>0}$, one has $c(v_m^{\mathbf{Z}}) = 0$.

Proof. Uses: def:weil-integer-degree-euler-factor-family,lemma:weil-integral-inverse-euler-factor-constant

The raw integral constant-coefficient lemma gives $c(E_m^{\mathbf{Z}}) = 1$. The degree-zero multiplication formula, iterated $a_m(D)$ times, gives $c((E_m^{\mathbf{Z}})^{a_m(D)}) = 1^{a_m(D)} = 1$. Subtracting the constant series one gives $c(v_m^{\mathbf{Z}}) = 0$. □

Lemma 3.7.7 (Local finiteness of the integer degree components). *Uses:* *def:weil-integer-degree-euler-factor-family, def:weil-series-agree-through*

Lean: *Weil.FiniteOrbitFrobeniusData.integerDegreeEulerFactorFamily_locallyFinite*

For every $N \geq 0$, the set

$$\left\{m \in \mathbf{N}_{>0} \mid v_m^{\mathbf{Z}} \not\equiv_N 0\right\}$$

is finite.

Proof. Uses: def:weil-integer-degree-euler-factor-family,lemma:weil-integral-inverse-euler-factor-initial-vanishing,lemma:weil-series-agree-through-operations,lemma:weil-series-agree-through-pow,lemma:weil-series-agree-through-sub

Fix N and let $m > N$. Raw integral initial vanishing gives $E_m^{\mathbf{Z}} - 1 \equiv_N 0$. Add one to both sides to obtain $E_m^{\mathbf{Z}} \equiv_N 1$, apply preservation under the natural power $a_m(D)$, and subtract one. The result is $v_m^{\mathbf{Z}} \equiv_N 0$. Thus every active positive degree lies in the finite interval $\{m \mid 1 \leq m \leq N\}$. □

Definition 3.7.8 (Packaged integer degree-grouped Euler family). Uses: def:weil-positive-natural-degree-index,def:weil-integer-degree-euler-factor-family,def:weil-t-adic-locally-finite-family

Constructed by: lemma:weil-integer-degree-euler-factor-family-positive,lemma:weil-integer-degree-euler-factor-family-locally-finite

Lean: Weil.FiniteOrbitFrobeniusData.integerDegreeEulerFamily

Define $\mathcal{G}_D^{\mathbf{Z}}$ by assigning index type $\mathbf{N}_{>0}$, component $v_m^{\mathbf{Z}}$, positive-order field the preceding positive-order proof, and active-set-finiteness field the preceding local-finiteness proof. No certificate field is proved inside this Definition.

Definition 3.7.9 (Raw integer closed-point Euler-factor family). Uses: def:weil-closed-point-orbit-interface,def:weil-integral-inverse-euler-factor

Lean: Weil.ClosedPointOrbitInterface.integerEulerFactorFamily

For $c \in \mathrm{ClPt}(C)$, define

$$w_c^{\mathbf{Z}}(T) := E^{\mathbf{Z}}_{\deg_C(c)}(T) - 1 \in \mathbf{Z}[[T]].$$

Lemma 3.7.10 (Positive order of every integer closed-point component). Uses: def:weil-integer-closed-point-euler-factor-family, def:weil-positive-series

Lean: Weil.ClosedPointOrbitInterface.integerEulerFactorFamily_positive

For every closed point c, one has $c(w_c^{\mathbf{Z}}) = 0$.

Proof. Uses: def:weil-closed-point-orbit-interface,def:weil-integer-closed-point-euler-factor-family,lemma:weil-integral-inverse-euler-factor-initial-vanishing

The interface supplies $\deg_C(c) \geq 1$. Instantiate the constant- coefficient conclusion of raw integral initial vanishing at $m = \deg_C(c)$; the displayed difference is $w_c^{\mathbf{Z}}$ after unfolding the raw component Definition. □

Lemma 3.7.11 (Integer closed-point components transport to orbit components). Uses: def:weil-closed-point-orbit-interface, def:weil-integer-orbit-euler-factor-family, def:weil-integer-closed-point-euler-factor-family

Lean: Weil.ClosedPointOrbitInterface.integerEulerFactorFamily_component_transport

For every closed point c,

$$u^{\mathbf{Z}}_{\theta_C(c)} = w_c^{\mathbf{Z}} \qquad \text{in } \mathbf{Z}[[T]].$$

Proof. Uses: def:weil-closed-point-orbit-interface,def:weil-integer-orbit-euler-factor-family,def:weil-integer-closed-point-euler-factor-family

Unfold both raw integer component Definitions. The left side is $E^{\mathbf{Z}}_{\deg_F(\theta_C(c))} - 1$ and the right side is $E^{\mathbf{Z}}_{\deg_C(c)} - 1$. Rewrite the former using $\deg_F(\theta_C(c)) = \deg_C(c)$ from the interface. The resulting integral power series are definitionally identical. □

Lemma 3.7.12 (Active integer closed points map into active integer orbits). Uses: def:weil-series-agree-through, def:weil-closed-point-orbit-interface, def:weil-integer-orbit-euler-factor-family, def:weil-integer-closed-point-euler-factor-family

Lean: Weil.ClosedPointOrbitInterface.integerActiveClosed_image_subset_activeOrbit

For every $N \geq 0$,

$$\theta_C''\{c \mid w_c^{\mathbf{Z}} \not\equiv_N 0\} \subseteq \{O \mid u_O^{\mathbf{Z}} \not\equiv_N 0\}.$$

Proof. Uses: def:weil-series-agree-through,lemma:weil-integer-closed-point-euler-component-transport

Take O in the displayed image. Its membership supplies a closed point c with $w_c^{\mathbf{Z}} \not\equiv_N 0$ and $O = \theta_C(c)$. Substitute $O = \theta_C(c)$ and then $u^{\mathbf{Z}}_{\theta_C(c)} = w_c^{\mathbf{Z}}$ from the preceding component-transport lemma. The target non-agreement is the given source non-agreement. □

Lemma 3.7.13 (The active integer closed-point image is finite). Uses: def:weil-series-agree-through, def:weil-closed-point-orbit-interface, def:weil-integer-orbit-euler-factor-family, def:weil-integer-closed-point-euler-factor-family

Lean: Weil.ClosedPointOrbitInterface.integerActiveClosed_image_finite

For every $N \geq 0$, the set

$$\theta_C''\{c \mid w_c^{\mathbf{Z}} \not\equiv_N 0\}$$

is finite.

Proof. Uses: `lemma:weil-integer-closed-point-active-image-subset`, `lemma:weil-integer-orbit-euler-factor-family-locally-finite`

The integer-orbit local-finiteness lemma makes $\{O \mid u_O^{\mathbf{Z}} \not\equiv_N 0\}$ finite. Apply `Set.Finite.subset` once, using the preceding active-image inclusion. □

Lemma 3.7.14 (Local finiteness of the integer closed-point components). Uses: *def:weil-integer-closed-point-euler-factor-family*, *def:weil-series-agree-through*

Lean: *Weil.ClosedPointOrbitInterface.integerEulerFactorFamily_locallyFinite*

For every $N \geq 0$, the set

$$\left\{c \in \mathrm{ClPt}(C) \mid w_c^{\mathbf{Z}} \not\equiv_N 0\right\}$$

is finite.

Proof. Uses: `def:weil-closed-point-orbit-interface`, `lemma:weil-integer-closed-point-active-image-finite`

Apply `Set.Finite.of_finite_image`. Its first argument is the preceding lemma's finiteness proof for $\theta_C''\{c \mid w_c^{\mathbf{Z}} \not\equiv_N 0\}$. Its second argument is θ_C`.injective.injOn`, the restriction of the equivalence's injectivity field. □

Definition 3.7.15 (Packaged integer closed-point Euler family). Uses: `def:weil-integer-closed-point-euler-factor-family`, `def:weil-t-adic-locally-finite-family`

Constructed by: `lemma:weil-integer-closed-point-euler-factor-family-positive`, `lemma:weil-integer-closed-point-euler-factor-family-locally-finite`

Lean: `Weil.ClosedPointOrbitInterface.integerEulerFamily`

Define $\mathcal{E}_C^{\mathbf{Z}}$ by assigning index type $\mathrm{ClPt}(C)$, component $w_c^{\mathbf{Z}}$, positive-order field the preceding positive-order proof, and active-set-finiteness field the preceding local-finiteness proof. No certificate field is proved inside this Definition.

3.8 Literal integral power-series realization

> **Status.** This promoted block is dependency-closed over the atomized integer Euler families constructed earlier in this chapter. Every coefficient-ring comparison points from $\mathbf{Z}[[T]]$ to $\mathbf{Q}[[T]]$; no reverse ring homomorphism or coefficientwise choice of integral lifts is used.

Definition 3.8.1 (Integer-to-rational power-series map). Uses:

Lean: `Weil.powerSeriesIntegerToRational`

Define the ring homomorphism

$$\iota_*\colon \mathbf{Z}[[T]] \longrightarrow \mathbf{Q}[[T]], \qquad \iota_* := \texttt{PowerSeries.map}(\mathrm{algebraMap}_{\mathbf{Z},\mathbf{Q}}).$$

Lemma 3.8.2 (Coefficient formula for integer-to-rational scalar extension). Uses: *def:weil-power-series-integer-to-rational*

Lean: *Weil.powerSeriesIntegerToRational_coeff*

For $A \in \mathbf{Z}[[T]]$ and $n \geq 0$,

$$[T^n]\iota_*(A) = \mathrm{algebraMap}_{\mathbf{Z},\mathbf{Q}}([T^n]A).$$

Proof. Uses: `def:weil-power-series-integer-to-rational`

Apply the exact theorem `PowerSeries.coeff_map` with $R = \mathbf{Z}$, $S = \mathbf{Q}$, $f = \mathrm{algebraMap}_{\mathbf{Z},\mathbf{Q}}$, degree n, and series A. Its conclusion is the displayed equality. □

Lemma 3.8.3 (Ring-homomorphism type of integer-to-rational scalar extension). Uses: *def:weil-power-series-integer-to-rational*

Lean: *Weil.powerSeriesIntegerToRational_isRingHom*

The object ι_* has type

$$\mathbf{Z}[[T]] \longrightarrow + * \mathbf{Q}[[T]].$$

Proof. Uses: `def:weil-power-series-integer-to-rational`

Unfold ι_*. The exact declaration `PowerSeries.map` takes a ring homomorphism $R \to + * S$ and returns a ring homomorphism $R[[T]] \to + * S[[T]]$. Instantiate it with the canonical ring homomorphism $\mathbf{Z} \to + * \mathbf{Q}$. The resulting type is the displayed type. □

Lemma 3.8.4 (Injectivity of integer-to-rational scalar extension)**.** *Uses:* `def:weil-power-series-integer-to-rational`

Lean: `Weil.powerSeriesIntegerToRational_injective`

The map ι_* is injective.

Proof. *Uses:* `lemma:weil-power-series-integer-to-rational-coeff`

Let $\iota_*(A) = \iota_*(B)$. Apply `PowerSeries.coeff` n for an arbitrary n. The coefficient formula rewrites the resulting equality as

$$([T^n]A : \mathbf{Q}) = ([T^n]B : \mathbf{Q}).$$

The exact theorem `Int.cast_injective` gives $[T^n]A = [T^n]B$. Apply `PowerSeries.ext`. □

Definition 3.8.5 (Raw rational components of an integral locally finite family)**.** *Uses:* `def:weil-t-adic-locally-finite-family`, `def:weil-power-series-integer-to-rational`

Lean: `Weil.TAdicallyLocallyFiniteFamily.rationalComponents`

Let u be a T-adically locally finite family over $\mathbf{Z}$, with index type I and components $u_i \in \mathbf{Z}[[T]]$. Define its raw rational component family by

$$u_i^{\mathbf{Q}} := \iota_*(u_i) \in \mathbf{Q}[[T]] \qquad (i \in I).$$

Lemma 3.8.6 (Scalar extension reflects finite-degree agreement)**.** *Uses:* `def:weil-power-series-integer-to-rational`, `def:weil-series-agree-through`

Lean: `Weil.powerSeriesIntegerToRational_agreeThrough`

For $A, B \in \mathbf{Z}[[T]]$ and $N \geq 0$,

$$A \equiv_N B \quad \Longleftrightarrow \quad \iota_*(A) \equiv_N \iota_*(B).$$

Proof. *Uses:* `def:weil-series-agree-through`, `lemma:weil-power-series-integer-to-rational-coeff`

Suppose first that $A \equiv_N B$, and fix $j \leq N$. Rewrite both mapped coefficients with the coefficient formula and map the equality $[T^j]A = [T^j]B$ into $\mathbf{Q}$. Conversely, suppose that the mapped series agree through N, and fix $j \leq N$. Rewrite their degree-j coefficient equality with the same coefficient formula. Apply `Int.cast_injective` to recover $[T^j]A = [T^j]B$. These are the two implications in the definition of agreement through N. □

Lemma 3.8.7 (Coefficient formula for a mapped component)**.** *Uses:* `def:weil-integer-family-rational-components`

Lean: `Weil.TAdicallyLocallyFiniteFamily.coeff_rationalComponents`

For every $i \in I$ and $n \geq 0$,

$$[T^n]u_i^{\mathbf{Q}} = \mathrm{algebraMap}_{\mathbf{Z},\mathbf{Q}}([T^n]u_i).$$

Proof. *Uses:* `def:weil-integer-family-rational-components`, `lemma:weil-power-series-integer-to-rational-coeff`

Unfold $u_i^{\mathbf{Q}}$ and apply the scalar-extension coefficient formula to u_i at degree n. □

Lemma 3.8.8 (Mapped components retain positive order)**.** *Uses:* `def:weil-integer-family-rational-components`, `def:weil-positive-series`

Lean: `Weil.TAdicallyLocallyFiniteFamily.rationalComponents_positive`

For every $i \in I$, one has $c(u_i^{\mathbf{Q}}) = 0$.

Proof. *Uses:* `def:weil-t-adic-locally-finite-family`, `lemma:weil-integer-family-rational-components-coeff`

Put $n = 0$ in the mapped-component coefficient formula. The integral family stores $c(u_i) = 0$, so the right side is the image of 0, hence is 0. □

Lemma 3.8.9 (Activity is preserved by integer-to-rational scalar extension)**.** *Uses:* `def:weil-integer-family-rational-components`, `def:weil-series-agree-through`

Lean: `Weil.TAdicallyLocallyFiniteFamily.rationalComponents_active_iff`

For every $i \in I$ and $N \geq 0$,

$$u_i \not\equiv_N 0 \quad \Longleftrightarrow \quad u_i^{\mathbf{Q}} \not\equiv_N 0.$$

Proof. *Uses:* `def:weil-integer-family-rational-components`, `lemma:weil-power-series-integer-to-rational-ring-hom`, `lemma:weil-power-series-integer-to-rational-agree-through`

Apply the agreement-reflection lemma to $A = u_i$ and $B = 0$. The right side it supplies is $\iota_*(u_i) \equiv_N \iota_*(0)$. Because ι_* is a ring homomorphism, $\iota_*(0) = 0$; unfold $u_i^{\mathbf{Q}}$. Negating the resulting equivalence gives the displayed active-index equivalence. □

Lemma 3.8.10 (Equality of active index sets before and after scalar extension). *Uses:* def:weil-integer-family-rational-components, def:weil-series-agree-through

Lean: Weil.TAdicallyLocallyFiniteFamily.rationalComponents_activeSet

For every $N \geq 0$,

$$\{i \in I \mid u_i \not\equiv_N 0\} = \{i \in I \mid u_i^{\mathbf{Q}} \not\equiv_N 0\}.$$

Proof. Uses: lemma:weil-integer-family-rational-components-active-iff

Apply `Set.ext` and fix i. Membership in the two displayed sets is exactly the two sides of the active-index equivalence. □

Lemma 3.8.11 (Local finiteness after integer-to-rational scalar extension). *Uses:* def:weil-integer-family-rational-components

Lean: Weil.TAdicallyLocallyFiniteFamily.rationalComponents_locallyFinite

For every $N \geq 0$, the set

$$\{i \in I \mid u_i^{\mathbf{Q}} \not\equiv_N 0\}$$

is finite.

Proof. Uses: def:weil-t-adic-locally-finite-family, lemma:weil-integer-family-rational-components-active-set

The integral locally finite family stores finiteness of $\{i \mid u_i \not\equiv_N 0\}$. Rewrite that set using the preceding active-set equality. The rewritten finiteness proof has exactly the displayed target. □

Definition 3.8.12 (Packaged rational image of an integral locally finite family). Uses: def:weil-integer-family-rational-components, def:weil-t-adic-locally-finite-family

Constructed by: lemma:weil-integer-family-rational-components-positive, lemma:weil-integer-family-rational-components-locally-finite

Lean: Weil.TAdicallyLocallyFiniteFamily.rationalization

Define $u^{\mathbf{Q}}$ to be the T-adically locally finite family over $\mathbf{Q}$ with index type I and component $u_i^{\mathbf{Q}}$. Its positive-order and active-set-finiteness fields are the two preceding certificates.

Lemma 3.8.13 (Active-finset equality after scalar extension). *Uses:* def:weil-integer-family-rational-locally-finite-family, def:weil-active-index-finset

Lean: Weil.TAdicallyLocallyFiniteFamily.activeFinset_rationalization

For every $N \geq 0$,

$$A_N(u) = A_N(u^{\mathbf{Q}}).$$

Proof. Uses: lemma:weil-active-index-finset-mem, lemma:weil-integer-family-rational-components-active-iff

Apply `Finset.ext` and fix i. The active-finset membership lemma rewrites membership on each side as failure of agreement with zero through degree N. The mapped-component active equivalence identifies those two predicates. □

Lemma 3.8.14 (Scalar extension of a finite power-series product). *Uses:* def:weil-power-series-integer-to-rational

Lean: Weil.powerSeriesIntegerToRational_finsetProduct

Let S be a finset and let $A_i \in \mathbf{Z}[[T]]$ for $i \in S$. Then

$$\iota_*\left(\prod_{i \in S} A_i\right) = \prod_{i \in S} \iota_*(A_i).$$

Proof. Uses: lemma:weil-power-series-integer-to-rational-ring-hom

Apply the exact theorem `map_prod` with the monoid homomorphism underlying the ring homomorphism ι_*, the family $i \mapsto A_i$, and the finset S. Its conclusion is the displayed identity. □

Lemma 3.8.15 (Scalar extension of an integral inverse Euler factor). *Uses:* def:weil-inverse-euler-factor, def:weil-power-series-integer-to-rational

Lean: Weil.powerSeriesIntegerToRational_inverseEulerFactor

For every $m \geq 1$,

$$\iota_*(E_m^{\mathbf{Z}}) = E_m^{\mathbf{Q}}.$$

Proof. Uses: `lemma:weil-inverse-euler-factor-coeff`, `lemma:weil-power-series-integer-to-rational-coeff`

Apply `PowerSeries.ext` and fix $n \geq 0$. The scalar-extension coefficient formula identifies the coefficient on the left with the image in $\mathbf{Q}$ of $[T^n]E_m^{\mathbf{Z}}$. The integral and rational instances of the inverse-Euler-factor coefficient formula identify both sides with 1 if $m \mid n$ and with 0 otherwise. □

Lemma 3.8.16 (Scalar extension of the positive Euler part). Uses: *def: weil-inverse-euler-factor-positive-part*, *def: weil-power-series-integer-to-rational*

Lean: *Weil.powerSeriesIntegerToRational_inverseEulerFactorPositivePart*

For every $m \geq 1$,

$$\iota_*(E_m^{\mathbf{Z}} - 1) = E_m^{\mathbf{Q}} - 1.$$

Proof. Uses: `lemma:weil-power-series-integer-to-rational-ring-hom`, `lemma:weil-inverse-euler-factor-integer-to-rational`

Because ι_* is a ring homomorphism, it preserves subtraction and sends 1 to 1. Apply these two map laws and then substitute the preceding inverse-Euler-factor comparison. □

Lemma 3.8.17 (Component projection of the packaged integral orbit family). Uses: *def: weil-integer-orbit-euler-factor-family*, *def: weil-integer-orbit-euler-locally-finite-family*

Lean: *Weil.FiniteOrbitFrobeniusData.integerOrbitEulerFamily_component*

For every orbit O, the O-component of $\mathcal{E}_D^{\mathbf{Z}}$ is $u_O^{\mathbf{Z}}$.

Proof. Uses: `def:weil-integer-orbit-euler-locally-finite-family`

Unfold the record constructor. Its component projection is definitionally the raw component function, so the assertion is reflexivity. □

Lemma 3.8.18 (Rational realization of an integral orbit component). Uses: *def: weil-integer-family-rational-components*, *def: weil-integer-orbit-euler-locally-finite-family*, *def: weil-orbit-euler-factor-family*

Lean: *Weil.FiniteOrbitFrobeniusData.integerOrbitEulerFamily_rationalComponent*

For every orbit O, the O-component of the generic rationalization $(\mathcal{E}_D^{\mathbf{Z}})^{\mathbf{Q}}$ is the previously defined rational component u_O.

Proof. Uses: `def:weil-integer-family-rational-components`, `def:weil-orbit-euler-factor-family`, `lemma:weil-integer-orbit-euler-family-component`, `lemma:weil-inverse-euler-positive-part-integer-to-rational`

Unfold the generic rational-component Definition and use the component projection lemma. The resulting equality is precisely scalar extension of the positive Euler part with $m = \deg_F(O)$. □

Lemma 3.8.19 (Component projection of the packaged integral degree family). Uses: *def: weil-integer-degree-euler-factor-family*, *def: weil-integer-degree-euler-locally-finite-family*

Lean: *Weil.FiniteOrbitFrobeniusData.integerDegreeEulerFamily_component*

For every $m \in \mathbf{N}_{>0}$, the m-component of $\mathcal{G}_D^{\mathbf{Z}}$ is $v_m^{\mathbf{Z}}$.

Proof. Uses: `def:weil-integer-degree-euler-locally-finite-family`

Unfold the record constructor. The component projection reduces definitionally to the raw component function. □

Lemma 3.8.20 (Rational realization of an integral degree component). Uses: *def: weil-integer-family-rational-components*, *def: weil-integer-degree-euler-locally-finite-family*, *def: weil-degree-euler-factor-family*

Lean: *Weil.FiniteOrbitFrobeniusData.integerDegreeEulerFamily_rationalComponent*

For every $m \in \mathbf{N}_{>0}$, the m-component of $(\mathcal{G}_D^{\mathbf{Z}})^{\mathbf{Q}}$ is the previously defined rational component v_m.

Proof. Uses: `def:weil-integer-family-rational-components`, `def:weil-degree-euler-factor-family`, `lemma:weil-integer-degree-euler-family-component`, `lemma:weil-power-series-integer-to-rational-ring-hom`, `lemma:weil-inverse-euler-factor-integer-to-rational`

Unfold the generic rational component and the two raw degree-component Definitions. The ring-homomorphism laws give

$$\iota_*\big((E_m^{\mathbf{Z}})^{a_m(D)} - 1\big) = \iota_*(E_m^{\mathbf{Z}})^{a_m(D)} - 1.$$

Replace $\iota_*(E_m^{\mathbf{Z}})$ by $E_m^{\mathbf{Q}}$ using the inverse-Euler-factor comparison. The result is v_m. □

Lemma 3.8.21 (Component projection of the packaged integral closed-point family)**.** *Uses: def:weil-integer-closed-point-euler-factor-family, def:weil-integer-closed-point-euler-locally-finite-family*

Lean: WeilConjecture.ClosedPointOrbitInterface.integerEulerFamily_component

For every closed point c, the c-component of $\mathcal{E}_C^{\mathbf{Z}}$ is $w_c^{\mathbf{Z}}$.

Proof. Uses: def:weil-integer-closed-point-euler-locally-finite-family

Unfold the record constructor. The component projection reduces definitionally to the raw component function. □

Lemma 3.8.22 (Rational realization of an integral closed-point component)**.** *Uses: def:weil-integer-family-rational-components, def:weil-integer-closed-point-euler-locally-finite-family, def:weil-closed-point-euler-factor-family*

Lean: WeilConjecture.ClosedPointOrbitInterface.integerEulerFamily_rationalComponent

For every closed point c, the c-component of $(\mathcal{E}_C^{\mathbf{Z}})^{\mathbf{Q}}$ is the previously defined rational component w_c.

Proof. Uses: def:weil-integer-family-rational-components,def:weil-closed-point-euler-factor-family,lemma:weil-integer-closed-point-euler-family-component,lemma:weil-inverse-euler-positive-part-integer-to-rational

Unfold the generic rational-component Definition and use the component projection lemma. The remaining equality is scalar extension of the positive Euler part with $m = \deg_C(c)$. □

Lemma 3.8.23 (Scalar extension of a locally finite product)**.** *Uses: def:weil-power-series-integer-to-rational, def:weil-integer-family-rational-locally-finite-family, def:weil-locally-finite-product*

Lean: WeilConjecture.TAdicallyLocallyFiniteFamily.product_rationalization

Let u be a T-adically locally finite family over $\mathbf{Z}$, with index type I and components $u_i \in \mathbf{Z}[[T]]$. For its packaged rational image $u^{\mathbf{Q}}$, one has

$$\iota_*\left(\prod_{i\in I}^{\mathrm{lf}}(1+u_i)\right) = \prod_{i\in I}^{\mathrm{lf}}(1+u_i^{\mathbf{Q}}) \qquad \text{in } \mathbf{Q}[[T]].$$

Proof. Uses: def:weil-integer-family-rational-components,def:weil-truncated-product,lemma:weil-power-series-integer-to-rational-coeff,lemma:weil-power-series-integer-to-rational-ring-hom,lemma:weil-locally-finite-product-coeff,lemma:weil-integer-family-rational-active-finset,lemma:weil-power-series-integer-to-rational-finset-product

Apply `PowerSeries.ext` and fix $N \geq 0$. Write $\iota = \mathrm{algebraMap}_{\mathbf{Z},\mathbf{Q}}$, and denote the two locally finite products in the statement by $P_{\mathbf{Z}}$ and $P_{\mathbf{Q}}$. Then

$$\begin{aligned}
[T^N]\iota_*(P_{\mathbf{Z}}) &= \iota\big([T^N]P_{\mathbf{Z}}\big)\\
&= \iota\big([T^N]\Pi_N(u)\big)\\
&= [T^N]\iota_*\big(\Pi_N(u)\big)\\
&= [T^N]\prod_{i\in A_N(u)}\iota_*(1+u_i)\\
&= [T^N]\prod_{i\in A_N(u)}(1+u_i^{\mathbf{Q}})\\
&= [T^N]\prod_{i\in A_N(u^{\mathbf{Q}})}(1+u_i^{\mathbf{Q}})\\
&= [T^N]\Pi_N(u^{\mathbf{Q}})\\
&= [T^N]P_{\mathbf{Q}}.
\end{aligned}$$

The first and third equalities are the scalar-extension coefficient formula. The second and last are the locally finite product coefficient formula, first over $\mathbf{Z}$ and then over $\mathbf{Q}$. After unfolding $\Pi_N(u)$, the fourth equality is the finite-product map lemma. The ring-homomorphism laws and the definition $u_i^{\mathbf{Q}} = \iota_*(u_i)$ give $\iota_*(1+u_i) = 1+u_i^{\mathbf{Q}}$, proving the fifth equality factor by factor. The active-finset equality gives the sixth, and the definition of the truncated product gives the seventh. Thus the degree-N coefficients agree; extensionality proves the claim. □

Definition 3.8.24 (Integer orbit Euler product). Uses: def:weil-integer-orbit-euler-locally-finite-family,def:weil-locally-finite-product

Lean: Weil.FiniteOrbitFrobeniusData.integerOrbitEulerProduct

Let $\mathcal{E}_D^{\mathbf{Z}}$ be the already constructed integral orbit family. Define its integer orbit Euler product by

$$Z^{\mathbf{Z}}_{\mathrm{orb},D}(T) := \prod_{O\in \mathrm{Orb}_F(\Omega)}^{\mathrm{lf}} \big(1+u_O^{\mathbf{Z}}(T)\big) \in \mathbf{Z}[[T]].$$

The notation means the coefficientwise locally finite product of Definition 3.2.11, applied to the family $\mathcal{E}_D^{\mathbf{Z}}$.

Lemma 3.8.25 (Active finsets after rationalizing the integral orbit family). Uses: def:weil-integer-orbit-euler-locally-finite-family,def:weil-integer-family-rational-locally-finite-family,def:weil-orbit-euler-locally-finite-family,def:weil-active-index-finset

Lean: Weil.FiniteOrbitFrobeniusData.integerOrbitEulerFamily_rationalization_activeFinset

For every $N \geq 0$, the active finsets of the rationalized integral family and of the previously defined rational orbit family coincide:

$$A_N\big((\mathcal{E}_D^{\mathbf{Z}})^{\mathbf{Q}}\big) = A_N(\mathcal{E}_D).$$

Proof. Uses: def:weil-active-index-finset,lemma:weil-active-index-finset-mem,lemma:weil-integer-orbit-euler-family-rational-component

Apply `Finset.ext` and fix an orbit O. By the active-finset membership lemma, membership on the left is equivalent to

$$((\mathcal{E}_D^{\mathbf{Z}})^{\mathbf{Q}})_O \not\equiv_N 0,$$

while membership on the right is equivalent to $u_O \not\equiv_N 0$. The rational-component lemma identifies $((\mathcal{E}_D^{\mathbf{Z}})^{\mathbf{Q}})_O$ with u_O; substituting this equality makes the two predicates identical. Hence the two finset membership propositions agree for every O, and extensionality gives the displayed equality. □

Lemma 3.8.26 (Rationalized integral orbit product equals the rational orbit product). Uses: def:weil-integer-orbit-euler-locally-finite-family,def:weil-integer-family-rational-locally-finite-family,def:weil-orbit-euler-locally-finite-family,def:weil-orbit-euler-product,def:weil-locally-finite-product

Lean: Weil.FiniteOrbitFrobeniusData.integerOrbitEulerProduct_rationalization

One has the equality in $\mathbf{Q}[[T]]$

$$\prod_{O}^{\mathrm{lf}} \big(1+((\mathcal{E}_D^{\mathbf{Z}})^{\mathbf{Q}})_O\big) = Z_{\mathrm{orb},D}(T).$$

Proof. Uses: def:weil-locally-finite-product,def:weil-truncated-product,lemma:weil-locally-finite-product-coeff,lemma:weil-integer-orbit-euler-family-rationalization-active-finset,lemma:weil-integer-orbit-euler-family-rational-component,def:weil-orbit-euler-product

Apply `PowerSeries.ext` and fix $N \geq 0$. The coefficient formula for the locally finite product rewrites the coefficient on the left as

$$[T^N] \prod_{O\in A_N((\mathcal{E}_D^{\mathbf{Z}})^{\mathbf{Q}})} \big(1+((\mathcal{E}_D^{\mathbf{Z}})^{\mathbf{Q}})_O\big),$$

and rewrites the coefficient of the right-hand product as

$$[T^N] \prod_{O\in A_N(\mathcal{E}_D)} (1+u_O).$$

Replace the first active finset by the second using the preceding active-finset equality. For each member O of this common finset, the rational-component lemma gives $((\mathcal{E}_D^{\mathbf{Z}})^{\mathbf{Q}})_O = u_O$; adding 1 to both sides gives equality of the corresponding factors. Apply the exact theorem `Finset.prod_congr` to the common active finset. Its finset-equality argument is reflexivity, and its pointwise factor-equality argument is the equality just obtained. The theorem therefore identifies the finite products, and hence their degree-N coefficients. Since N was arbitrary, `PowerSeries.ext` proves the asserted series equality. □

Lemma 3.8.27 (Scalar image of the integer orbit Euler product). Uses: def:weil-power-series-integer-to-rational,def:weil-integer-orbit-euler-product,def:weil-orbit-euler-product

Lean: Weil.FiniteOrbitFrobeniusData.integerOrbitEulerProduct_map

For the canonical coefficient map $\iota_* : \mathbf{Z}[[T]] \to \mathbf{Q}[[T]]$,

$$\iota_*\big(Z^{\mathbf{Z}}_{\mathrm{orb},D}(T)\big) = Z_{\mathrm{orb},D}(T).$$

Proof. Uses: `def:weil-integer-orbit-euler-locally-finite-family`, `def:weil-integer-orbit-euler-product`, `lemma:weil-power-series-integer-to-rational-locally-finite-product`, `lemma:weil-integer-orbit-euler-product-rationalization`

Instantiate the scalar-extension lemma for the integral family $u = \mathcal{E}_D^{\mathbf{Z}}$. By the definition of the integer orbit product, its left-hand side is $\iota_*(Z_{\mathrm{orb},D}^{\mathbf{Z}})$, and the lemma identifies it with

$$\prod_O{}^{\mathrm{lf}} \bigl(1 + ((\mathcal{E}_D^{\mathbf{Z}})^{\mathbf{Q}})_O\bigr).$$

The rationalization-product lemma identifies this last series with $Z_{\mathrm{orb},D}$. Substitution gives the displayed equality. □

Lemma 3.8.28 (Constant coefficient of the integer orbit Euler product). Uses: *def:weil-integer-orbit-euler-product*

Lean: *Weil.FiniteOrbitFrobeniusData.integerOrbitEulerProduct_constantCoeff*

The integer orbit Euler product has constant coefficient one:

$$[T^0]Z_{\mathrm{orb},D}^{\mathbf{Z}}(T) = 1 \quad \text{in } \mathbf{Z}.$$

Proof. Uses: `def:weil-integer-orbit-euler-product`, `lemma:weil-locally-finite-product-constant`

Apply the generic constant-coefficient lemma for a locally finite product to the family $\mathcal{E}_D^{\mathbf{Z}}$. The product in that lemma is exactly the product used in the Definition of $Z_{\mathrm{orb},D}^{\mathbf{Z}}$, so its coefficient at degree zero is 1. □

Proposition 3.8.29 (Integral lift of the counting zeta series). Uses: *def:weil-power-series-integer-to-rational*, *def:weil-integer-orbit-euler-product*, *def:weil-counting-zeta-series*

Lean: *Weil.FiniteOrbitFrobeniusData.countingZetaSeries_integralLift*

The counting zeta series is the scalar image of the integer orbit product:

$$\iota_*\bigl(Z_{\mathrm{orb},D}^{\mathbf{Z}}(T)\bigr) = Z_{\mathrm{cnt},D}(T).$$

Proof. Uses: `lemma:weil-integer-orbit-euler-product-map`, `thm:weil-orbit-euler-exponential`

The scalar-map lemma gives

$$\iota_*(Z_{\mathrm{orb},D}^{\mathbf{Z}}) = Z_{\mathrm{orb},D}.$$

The orbit Euler exponential theorem gives $Z_{\mathrm{cnt},D} = Z_{\mathrm{orb},D}$; reversing that equality gives $Z_{\mathrm{orb},D} = Z_{\mathrm{cnt},D}$. Substituting this second equality into the first yields the displayed integral-lift identity. □

Definition 3.8.30 (Integer degree-grouped Euler product). Uses: `def:weil-integer-degree-euler-locally-finite-family`, `def:weil-locally-finite-product`

Lean: `Weil.FiniteOrbitFrobeniusData.integerDegreeEulerProduct`

Let $\mathcal{G}_D^{\mathbf{Z}}$ be the previously constructed integral degree-grouped family. Define

$$Z_{\mathrm{deg},D}^{\mathbf{Z}}(T) := \prod_{m \in \mathbf{N}_{>0}}{}^{\mathrm{lf}} \bigl(1 + v_m^{\mathbf{Z}}(T)\bigr) \in \mathbf{Z}[[T]].$$

This is the coefficientwise locally finite product of Definition 3.2.11, applied to $\mathcal{G}_D^{\mathbf{Z}}$.

Lemma 3.8.31 (Active finsets after rationalizing the integral degree family). Uses: *def:weil-integer-family-rational-locally-finite-family*, *def:weil-degree-euler-locally-finite-family*, *def:weil-active-index-finset*

Lean: *Weil.FiniteOrbitFrobeniusData.integerDegree_rationalizedActiveFinset*

For every $N \geq 0$,

$$A_N\bigl((\mathcal{G}_D^{\mathbf{Z}})^{\mathbf{Q}}\bigr) = A_N(\mathcal{G}_D).$$

Proof. Uses: `lemma:weil-active-index-finset-mem`, `lemma:weil-integer-degree-euler-family-rational-component`, `def:weil-degree-euler-factor-family`

Apply `Finset.ext` and fix $m \in \mathbf{N}_{>0}$. The active-finset membership lemma rewrites membership on the left as

$$((\mathcal{G}_D^{\mathbf{Z}})^{\mathbf{Q}})_m \not\equiv_N 0$$

and membership on the right as $v_m \not\equiv_N 0$, after unfolding the generic degree-family component. The rational-component lemma gives $((\mathcal{G}_D^{\mathbf{Z}})^{\mathbf{Q}})_m = v_m$. Substitution makes the two membership predicates identical. Finset extensionality proves the displayed equality. □

Lemma 3.8.32 (Rationalized and generic degree truncations agree). *Uses:* *def: weil-integer-family-rational-locally-finite-family, def: weil-degree-euler-locally-finite-family, def: weil-truncated-product*

Lean: *Weil.FiniteOrbitFrobeniusData.integerDegree_rationalizedTruncatedProduct*

For every $N \geq 0$,

$$\Pi_N\big((\mathcal{G}_D^{\mathbf{Z}})^{\mathbf{Q}}\big) = \Pi_N(\mathcal{G}_D) \quad \textit{in } \mathbf{Q}[[T]].$$

Proof. Uses: `def:weil-truncated-product`, `def:weil-degree-euler-factor-family`, `lemma:weil-integer-degree-rationalized-active-finset`, `lemma:weil-integer-degree-euler-family-rational-component`

Replace the active finset in the left-hand truncated product by the equal active finset from the preceding lemma. On this common finite index set, the rational-component lemma gives, for every m,

$$1 + ((\mathcal{G}_D^{\mathbf{Z}})^{\mathbf{Q}})_m = 1 + v_m.$$

The generic degree-family component is v_m by its raw component Definition. Apply `Finset.prod_congr` to the displayed factor equality. After unfolding the two truncated products, the conclusion of `Finset.prod_congr` is precisely the asserted equality. □

Lemma 3.8.33 (Scalar image of the integer degree-grouped Euler product). *Uses:* *def: weil-integer-degree-euler-product, def : weil-degree-euler-product*

Lean: *Weil.FiniteOrbitFrobeniusData.integerDegreeEulerProduct_map*

For the integer-to-rational coefficient map ι_*,

$$\iota_*\big(Z^{\mathbf{Z}}_{\deg,D}(T)\big) = Z_{\deg,D}(T) \quad \textit{in } \mathbf{Q}[[T]].$$

Proof. Uses: `def:weil-integer-degree-euler-product`, `def:weil-degree-euler-product`, `lemma:weil-power-series-integer-to-rational-locally-finite-product`, `lemma:weil-locally-finite-product-coeff`, `lemma:weil-integer-degree-rationalized-truncated-product`

Instantiate the scalar-extension theorem for locally finite products at the integral family $\mathcal{G}_D^{\mathbf{Z}}$. It gives

$$\iota_*\big(Z^{\mathbf{Z}}_{\deg,D}\big) = \prod_m{}^{\mathrm{lf}}\big(1 + ((\mathcal{G}_D^{\mathbf{Z}})^{\mathbf{Q}})_m\big).$$

It remains to identify the product on the right with $Z_{\deg,D}$. Apply `PowerSeries.ext` and fix $N \geq 0$. The locally finite product coefficient formula changes their degree-N coefficients into

$$[T^N]\Pi_N((\mathcal{G}_D^{\mathbf{Z}})^{\mathbf{Q}}) \quad \text{and} \quad [T^N]\Pi_N(\mathcal{G}_D),$$

respectively. These two truncated products are equal by the preceding lemma. Hence the two coefficients agree for every N; extensionality gives the desired product equality, and substitution into the first displayed equality proves the scalar-map formula. □

Proposition 3.8.34 (The integer orbit product regrouped by degree). *Uses:* *def: weil-integer-orbit-euler-product, def : weil-integer-degree-euler-product*

Lean: *Weil.FiniteOrbitFrobeniusData.integerOrbitEulerProduct_eq_integerDegreeEulerProduct*

One has the equality in $\mathbf{Z}[[T]]$

$$Z^{\mathbf{Z}}_{\mathrm{orb},D}(T) = Z^{\mathbf{Z}}_{\deg,D}(T).$$

Proof. Uses: `lemma:weil-power-series-integer-to-rational-injective`, `lemma:weil-integer-orbit-euler-product-map`, `lemma:weil-integer-degree-euler-product-map`, `prop:weil-orbit-euler-product-grouped`

Apply injectivity of $\iota_* : \mathbf{Z}[[T]] \to \mathbf{Q}[[T]]$ to the desired equality. The orbit scalar-map lemma changes the image of the left side into $Z_{\mathrm{orb},D}$, and the degree scalar-map lemma changes the image of the right side into $Z_{\deg,D}$. The rational regrouping proposition proves $Z_{\mathrm{orb},D} = Z_{\deg,D}$. Thus the mapped series are equal, and injectivity gives the equality over $\mathbf{Z}$. □

Definition 3.8.35 (Integral closed-point Euler product). Uses: `def:weil-integer-closed-point-euler-locally-finite-family`, `def:weil-integer-closed-point-euler-factor-family`, `def:weil-locally-finite-product`

Lean: `Weil.ClosedPointOrbitInterface.integerEulerProduct`

Let $\mathcal{E}_C^{\mathbf{Z}}$ be the already constructed integral closed-point Euler family. Define its coefficientwise locally finite product by

$$Z_{\mathrm{cl},C}^{\mathbf{Z}}(T) := \prod_{c\in \mathrm{ClPt}(C)}^{\mathrm{lf}} \big(1 + w_c^{\mathbf{Z}}(T)\big) \in \mathbf{Z}[[T]],$$

where $w_c^{\mathbf{Z}}$ is the component of $\mathcal{E}_C^{\mathbf{Z}}$. Thus the factor at c is literally $E^{\mathbf{Z}}_{\deg_C(c)}$, but the Definition uses the already constructed locally finite-family product and makes no new infinite multiplication operation.

Lemma 3.8.36 (Active finsets after rationalizing the integral closed-point family)**.** *Uses:* `def:weil-integer-family-rational-locally-finite-family`, `def:weil-integer-closed-point-euler-locally-finite-family`, `def:weil-active-index-finset`

Lean: `Weil.ClosedPointOrbitInterface.integerEulerFamily_rationalActiveFinset`

For every $N \geq 0$, the active finset of the rationalized integral closed-point family is the active finset of the previously defined rational closed-point family:

$$A_N\big((\mathcal{E}_C^{\mathbf{Z}})^{\mathbf{Q}}\big) = A_N(\mathcal{E}_C).$$

Proof. Uses: `lemma:weil-active-index-finset-mem`, `lemma:weil-integer-closed-point-euler-family-rational-component`

Apply `Finset.ext` and fix a closed point c. By the active- finset membership lemma, membership on the left is equivalent to $(\mathcal{E}_C^{\mathbf{Z}})_c^{\mathbf{Q}} \not\equiv_N 0$, while membership on the right is equivalent to $w_c \not\equiv_N 0$. The preceding rational-component lemma gives the equality $(\mathcal{E}_C^{\mathbf{Z}})_c^{\mathbf{Q}} = w_c$. Substitute this equality into the first predicate; both membership propositions are then identical, which is the required finset extensionality statement. □

Lemma 3.8.37 (The rationalized integral closed-point product is the rational closed-point product)**.** *Uses:* `def:weil-integer-family-rational-locally-finite-family`, `def:weil-integer-closed-point-euler-locally-finite-family`, `def:weil-closed-point-euler-locally-finite-family`, `def:weil-closed-point-euler-product`, `def:weil-locally-finite-product`

Lean: `Weil.ClosedPointOrbitInterface.rationalizedIntegerEulerProduct_eq`

In $\mathbf{Q}[[T]]$, one has

$$\prod_{c\in \mathrm{ClPt}(C)}^{\mathrm{lf}} \big(1 + (\mathcal{E}_C^{\mathbf{Z}})_c^{\mathbf{Q}}\big) = Z_{\mathrm{cl},C}(T).$$

Proof. Uses: `def:weil-integer-family-rational-locally-finite-family`, `def:weil-closed-point-euler-locally-finite-family`, `def:weil-closed-point-euler-product`, `def:weil-closed-point-euler-factor-family`, `def:weil-truncated-product`, `lemma:weil-locally-finite-product-coeff`, `lemma:weil-integer-closed-point-rational-active-finset`, `lemma:weil-integer-closed-point-euler-family-rational-component`

Apply `PowerSeries.ext` and fix $N \geq 0$. Denote the left product by P^{rat} and the right product by $Z_{\mathrm{cl},C}$. The locally finite-product coefficient formula gives

$$[T^N]P^{\mathrm{rat}} = [T^N]\Pi_N\big((\mathcal{E}_C^{\mathbf{Z}})^{\mathbf{Q}}\big), \qquad [T^N]Z_{\mathrm{cl},C} = [T^N]\Pi_N(\mathcal{E}_C).$$

Unfold both truncated products and the rational closed-point product Definition. The raw rational closed-point factor Definition says that its component is $w_c = E_{\deg_C(c)} - 1$; hence each right-hand factor is $1 + w_c = 1 + (E_{\deg_C(c)} - 1) = E_{\deg_C(c)}$, by the additive group laws. The active-finset equality from the preceding lemma replaces the first finite index set by the second. On this common finset, the component equality $(\mathcal{E}_C^{\mathbf{Z}})_c^{\mathbf{Q}} = w_c$ gives, for each c,

$$1 + (\mathcal{E}_C^{\mathbf{Z}})_c^{\mathbf{Q}} = 1 + w_c.$$

Apply `Finset.prod_congr` to replace every factor in the finite product. The two degree-N coefficients are equal, and `PowerSeries.ext` yields the displayed equality of series. □

Lemma 3.8.38 (Scalar extension of the integral closed-point Euler product)**.** *Uses:* `def:weil-integer-closed-point-euler-product`, `def:weil-closed-point-euler-product`, `def:weil-power-series-integer-to-rational`

Lean: `Weil.ClosedPointOrbitInterface.integerEulerProduct_map`

The canonical coefficient map carries the integral closed-point product to the rational closed-point product:

$$\iota_*\big(Z_{\mathrm{cl},C}^{\mathbf{Z}}(T)\big) = Z_{\mathrm{cl},C}(T) \qquad \text{in } \mathbf{Q}[[T]].$$

Proof. Uses: def:weil-integer-closed-point-euler-product,def:weil-integer-closed-point-euler-locally-finite-family,def:weil-integer-family-rational-locally-finite-family,def:weil-closed-point-euler-product,lemma:weil-power-series-integer-to-rational-locally-finite-product,lemma:weil-integer-closed-point-rationalized-product

Unfold the integral-product Definition, so the left side is $\iota_*(\prod_c^{\mathrm{lf}}(1+w_c^{\mathbf{Z}}))$. Instantiate the scalar-extension theorem for locally finite products with coefficient rings $\mathbf{Z}$ and $\mathbf{Q}$, the canonical map ι_*, and the family $u=\mathcal{E}_C^{\mathbf{Z}}$. Its conclusion is the explicit equality

$$\iota_*\big(Z_{\mathrm{cl},C}^{\mathbf{Z}}\big)=\prod_c^{\mathrm{lf}}\big(1+(\mathcal{E}_C^{\mathbf{Z}})_c^{\mathbf{Q}}\big).$$

The rationalized-product comparison lemma identifies the right-hand side with $Z_{\mathrm{cl},C}$. Transitivity of equality proves the claim. □

Lemma 3.8.39 (The two integral products have the same rational image). Uses: def:weil-power-series-integer-to-rational,def:weil-integer-closed-point-euler-product,def:weil-integer-orbit-euler-product

Lean: Weil.ClosedPointOrbitInterface.integerProducts_rationalImage_eq

Their scalar extensions satisfy

$$\iota_*(Z_{\mathrm{cl},C}^{\mathbf{Z}})=\iota_*(Z_{\mathrm{orb},D}^{\mathbf{Z}})\quad \textit{in } \mathbf{Q}[[T]].$$

Proof. Uses: lemma:weil-integer-closed-point-euler-product-map,lemma:weil-integer-orbit-euler-product-map,prop:weil-closed-point-euler-product-reindex

The three preceding equalities form the explicit chain

$$\iota_*(Z_{\mathrm{cl},C}^{\mathbf{Z}})=Z_{\mathrm{cl},C}=Z_{\mathrm{orb},D}=\iota_*(Z_{\mathrm{orb},D}^{\mathbf{Z}}).$$

The first equality is the closed-point scalar-map lemma. The second is the already proved rational closed-point/orbit reindexing proposition, whose proof reindexes the finite active products. The last equality is the symmetry of the orbit scalar-map lemma. Transitivity gives the required equality in $\mathbf{Q}[[T]]$. □

Proposition 3.8.40 (Integral closed-point/orbit Euler-product reindexing). Uses: def:weil-integer-closed-point-euler-product,def:weil-integer-orbit-euler-product

Lean: Weil.ClosedPointOrbitInterface.integerEulerProduct_eq_orbitEulerProduct

One has

$$Z_{\mathrm{cl},C}^{\mathbf{Z}}(T)=Z_{\mathrm{orb},D}^{\mathbf{Z}}(T)\quad \textit{in } \mathbf{Z}[[T]].$$

Proof. Uses: lemma:weil-integer-closed-point-orbit-rational-image,lemma:weil-power-series-integer-to-rational-injective

Let $A=Z_{\mathrm{cl},C}^{\mathbf{Z}}$ and $B=Z_{\mathrm{orb},D}^{\mathbf{Z}}$. The preceding lemma gives $\iota_*(A)=\iota_*(B)$. The injectivity lemma for $\iota_*:\mathbf{Z}[[T]]\to\mathbf{Q}[[T]]$ therefore gives $A=B$. This is the required return from the equality over $\mathbf{Q}[[T]]$ to the literal equality over $\mathbf{Z}[[T]]$. The finite active-product reindexing is the already proved rational proposition used in the preceding lemma; the only new operation here is the injective descent of that equality. □

Definition 3.8.41 (Literal integral zeta certificate). Uses: def:weil-power-series-integer-to-rational,def:weil-counting-zeta-series,def:weil-degree-euler-product,def:weil-closed-point-euler-product,def:weil-integer-orbit-euler-product,def:weil-integer-degree-euler-product,def:weil-integer-closed-point-euler-product

Constructed by: prop:weil-counting-zeta-integral-lift,lemma:weil-integer-degree-euler-product-map,prop:weil-integer-orbit-euler-product-grouped,lemma:weil-integer-closed-point-euler-product-map,prop:weil-integer-closed-point-euler-product-reindex,lemma:weil-integer-orbit-euler-product-constant

Lean: Weil.FiniteOrbitFrobeniusData.LiteralIntegralZetaCertificate

For the fixed compatible data (C,D), define $\mathfrak{Z}_{C,D}^{\mathbf{Z}}$ to be the explicitly recorded tuple consisting of

$$Z_{\mathrm{orb},D}^{\mathbf{Z}}\in\mathbf{Z}[[T]],\qquad Z_{\mathrm{deg},D}^{\mathbf{Z}}\in\mathbf{Z}[[T]],\qquad Z_{\mathrm{cl},C}^{\mathbf{Z}}\in\mathbf{Z}[[T]],$$
$$\iota_*(Z_{\mathrm{orb},D}^{\mathbf{Z}})=Z_{\mathrm{cnt},D},\quad \iota_*(Z_{\mathrm{deg},D}^{\mathbf{Z}})=Z_{\mathrm{deg},D},\quad Z_{\mathrm{orb},D}^{\mathbf{Z}}=Z_{\mathrm{deg},D}^{\mathbf{Z}},$$
$$\iota_*(Z_{\mathrm{cl},C}^{\mathbf{Z}})=Z_{\mathrm{cl},C},\quad Z_{\mathrm{cl},C}^{\mathbf{Z}}=Z_{\mathrm{orb},D}^{\mathbf{Z}},\quad [T^0]Z_{\mathrm{orb},D}^{\mathbf{Z}}=1.$$

The first three entries are the already defined typed products. Each of the six displayed equations is inserted as its named proof field, in the order listed in the constructedby metadata. Thus the Definition constructs a concrete certificate by record assembly; it does not assert an unproved existence or replace any equation by a coefficient-membership slogan.

Chapter 4

The present mathlib ℓ-adic endpoint and the Frobenius interface

Status. The first twelve declarations in this chapter are exactly the locally compiled class-A declarations of the pinned mathlib audit. Everything after them is either an elementary conditional construction or a specification predicate. In particular this chapter does not construct a constructible adic category, rational ℓ-adic cohomology, compact support, a scheme or cohomological Frobenius action, a Tate object, or a finite-dimensional cohomology group. The separately numbered obligations in the module manifest must be discharged before any such target can be selected.

4.1 Exactly verified big and small sites

Definition 4.1.1 (Big etale precoverage)**.** Uses:

Lean: `AlgebraicGeometry.Scheme.etalePrecoverage`

For the category Scheme, define the big etale precoverage to consist of the covering families of etale scheme morphisms used by the MorphismProperty precoverage construction.

Library status: verified only after the pinned-contract audit.

Definition 4.1.2 (Big etale pretopology)**.** Uses: `def:weil-big-etale-precoverage`

Lean: `AlgebraicGeometry.Scheme.etalePretopology`

Define the big etale pretopology on Scheme to be the MorphismProperty pretopology generated by etale morphisms.

Library status: verified only after the pinned-contract audit.

Definition 4.1.3 (Big etale topology)**.** Uses: `def:weil-big-etale-pretopology`

Lean: `AlgebraicGeometry.Scheme.etaleTopology`

Define the big etale Grothendieck topology to be the topology generated by the big etale pretopology.

Library status: verified only after the pinned-contract audit.

Definition 4.1.4 (Small etale topology)**.** Uses: `def:weil-big-etale-topology`

Lean: `AlgebraicGeometry.Scheme.smallEtaleTopology`

For a scheme X, let X_{et} denote the category X.Etale equipped with the Grothendieck topology induced from the big etale topology.

Library status: verified only after the pinned-contract audit.

Definition 4.1.5 (Small etale pretopology)**.** Uses: `def:weil-big-etale-pretopology`, `def:weil-small-etale-topology`

Lean: `AlgebraicGeometry.Scheme.smallEtalePretopology`

For a scheme X, define the generating pretopology on X.Etale by applying mathlib's small-pretopology construction with both morphism properties equal to etale morphisms.

Library status: verified only after the pinned-contract audit.

Formalization note. The exact import for the preceding five declarations is `Mathlib.AlgebraicGeometry.Sites.ElladicCohomology`. The defining source is `Sites/Etale.lean`, lines 32–54 at mathlib commit `81a5d257c8e410db227a6665ed08f64fea08e997`.

Definition 4.1.6 (Small pro-etale category)**.** Uses:

Lean: `AlgebraicGeometry.Scheme.ProEt`

For a scheme S, define S.ProEt to be the category of weakly etale scheme morphisms to S, represented by mathlib's MorphismProperty over-category.

Library status: verified only after the pinned-contract audit.

Definition 4.1.7 (Forgetful functor from the small pro-etale category)**.** Uses: `def:weil-small-proetale-category`

Lean: `AlgebraicGeometry.Scheme.ProEt.forget`

Define

$$\mathrm{forg}_S : S.\mathrm{ProEt} \longrightarrow \mathrm{Over}(S)$$

by forgetting the proof that the structural morphism is weakly etale.

Library status: verified only after the pinned-contract audit.

Definition 4.1.8 (Small pro-etale precoverage)**.** Uses: `def:weil-small-proetale-category`, `def:weil-small-proetale-forgetful-functor`

Lean: `AlgebraicGeometry.Scheme.ProEt.precoverage`

Define the precoverage on S.ProEt by pulling the big pro-etale precoverage back along forg_S followed by the forgetful functor $\mathrm{Over}(S) \to \mathrm{Scheme}$.

Library status: verified only after the pinned-contract audit.

Definition 4.1.9 (Small pro-etale topology)**.** Uses: `def:weil-small-proetale-precoverage`

Lean: `AlgebraicGeometry.Scheme.ProEt.topology`

Define the small pro-etale Grothendieck topology to be the topology generated by the small pro-etale precoverage.

Library status: verified only after the pinned-contract audit.

Formalization note. These four definitions are `Sites/Proetale.lean`, lines 97, 116, 140, and 144 at the pinned commit. The mathematical source named in that file is Bhatt–Scholze, Definition 4.1.1.

4.2 The exact integral endpoint currently present in mathlib

Definition 4.2.1 (Integral pro-etale coefficient sheaf)**.** Uses: `def:weil-small-proetale-category`, `def:weil-small-proetale-forgetful-functor`, `def:weil-small-proetale-topology`

Lean: `AlgebraicGeometry.Scheme.ellAdicSheaf`

Let X be a scheme and let ℓ be a prime. Define $\mathcal{Z}_{\ell,X}$ to be the sheaf of abelian groups on X_{proet} whose value before sheafification is the group of continuous maps

$$U \longmapsto C\big(|U|, \mathbf{Z}_\ell\big).$$

This is exactly the object $X.\mathrm{ellAdicSheaf}(\ell)$ in mathlib.

Library status: verified only after the pinned-contract audit.

Definition 4.2.2 (Integral pro-etale cohomology)**.** Uses: `def:weil-small-proetale-topology`, `def:weil-integral-proetale-coefficient-sheaf`

Lean: `AlgebraicGeometry.Scheme.EllAdicCohomology`

For $n \in \mathbf{N}$, define

$$H^n_{\mathrm{proet}}(X, \mathbf{Z}_\ell) := H^n(X_{\mathrm{proet}}, \mathcal{Z}_{\ell,X})$$

using mathlib's generic sheaf cohomology after applying its universe-lifting functor from abelian groups. The resulting type lies in universe $u+1$ and has the additive commutative group structure furnished by the same module.

Library status: verified only after the pinned-contract audit.

Lemma 4.2.3 (The integral coefficient sheaf on the empty scheme is zero). Uses: `def:weil-integral-proetale-coefficient-sheaf`

Lean: `AlgebraicGeometry.Scheme.isZero_ellAdicSheaf_of_isEmpty`

Let X : Scheme^u, let ℓ : $\mathbf{N}$, assume Fact(ℓ *is prime*)*, and assume that X is empty. Then $\mathcal{Z}_{\ell,X}$ is a zero object in the category of abelian-group-valued sheaves on X_{proet}.*

Proof. Uses: `def:weil-integral-proetale-coefficient-sheaf`

Under the exact import `Mathlib.AlgebraicGeometry.Sites.ElladicCohomology`, instantiate `AlgebraicGeometry.Scheme.isZero_ellAdicSheaf_of_isEmpty` with the parameters X and ℓ. Its hypotheses are exactly the displayed `Fact` and `IsEmpty X` instances, and its conclusion is definitionally IsZero(X.ellAdicSheaf(ℓ)), which is the claimed target. □

> **Formalization note.** The two definitions and the lemma are `Sites/ElladicCohomology.lean`, lines 63–76 at the pinned commit; the coefficient-sheaf comment cites Bhatt–Scholze, Definition 6.8.1. The module's own documentation explicitly says that comparison with the classical etale definition is future work. No rational, constructible, compact-support, or Frobenius conclusion is inferred here.

4.3 Conditional scalar extension, with no comparison theorem assumed

Definition 4.3.1 (Coefficient-extension datum). Uses: `def:weil-coefficient-data`

Lean: `Weil.Etale.CoefficientExtensionData`

For the fixed prime ℓ, put $\Lambda = \mathbf{Z}_\ell$, $E = \mathbf{Q}_\ell$, and retain the specified injective ring map $\lambda : \Lambda \hookrightarrow E$. A coefficient-extension datum records these three typed objects and this map; it does not assert that any cohomology group is a Λ-module.

Definition 4.3.2 (Integral cohomology module candidate). Uses: `def:weil-integral-proetale-cohomology,def:weil-coefficient-extension-data`

Lean: `Weil.Etale.IntegralCohomologyModuleCandidate`

For fixed X, ℓ, n, an integral cohomology module candidate is a specified Λ-module structure on the additive group $H^n_{\mathrm{proet}}(X, \mathbf{Z}_\ell)$. This is an input type; no inhabitant is selected by this definition.

Definition 4.3.3 (Raw rationalization of integral cohomology). Uses: `def:weil-coefficient-extension-data,def:weil-integral-cohomology-module-candidate`

Lean: `Weil.Etale.rawRationalizedCohomology`

Given an integral cohomology module candidate M^n_X, define its raw scalar extension by

$$V^{n,\mathrm{raw}}_{X,\ell} := E \otimes_\Lambda M^n_X.$$

This is an E-vector space by the left scalar action on the first tensor factor. The adjective "raw" records that no comparison with classical constructible $\mathbf{Q}_\ell$-cohomology has been proved.

Definition 4.3.4 (Rational-cohomology comparison predicate). Uses: `def:weil-raw-rationalized-cohomology`

Lean: `Weil.Etale.IsRationalCohomologyComparison`

Let $V^n_{X,\ell}$ be a proposed classical rational cohomology space and let $c^n_X : V^{n,\mathrm{raw}}_{X,\ell} \to V^n_{X,\ell}$ be E-linear. Define IsRatComparison(c^n_X) to mean that c^n_X is bijective. This predicate does not construct $V^n_{X,\ell}$ or c^n_X.

Definition 4.3.5 (Functorial rational-comparison predicate). Uses: `def:weil-rational-cohomology-comparison-predicate`

Lean: `Weil.Etale.IsRationalComparisonNatural`

For a family of proposed comparison maps c^n_X and a specified class of scheme or coefficient morphisms f, define IsRatComparisonNatural to mean that the square made from c^n_X, c^n_Y, the raw scalar-extended map, and the proposed classical map commutes for every f. This is a property predicate, not a naturality theorem.

Definition 4.3.6 (Scalar extension of a linear endomorphism). Uses: `def:weil-coefficient-extension-data,def:weil-raw-rationalized-cohomology`

Lean: `Weil.Etale.scalarExtensionEndomorphism`

Let M be a Λ-module and let $u : M \to M$ be Λ-linear. Define

$$u_E := \mathrm{id}_E \otimes u : E \otimes_\Lambda M \longrightarrow E \otimes_\Lambda M$$

by the tensor-product lift of the balanced map $(a, m) \mapsto a \otimes u(m)$.

Lemma 4.3.7 (Scalar extension on pure tensors). *Uses:* *def:weil-scalar-extension-endomorphism*

Lean: *Weil.Etale.scalarExtensionEndomorphism_tmul*

For $a \in E$ and $m \in M$, one has

$$u_E(a \otimes m) = a \otimes u(m).$$

Proof. Uses: def:weil-scalar-extension-endomorphism

The scalar-extension map was defined by the tensor-product lift of the balanced map $(a, m) \mapsto a \otimes u(m)$. Evaluating that lift on a pure tensor gives the displayed equality by the defining computation rule of the lift. □

Definition 4.3.8 (Scalar-extended inverse candidate). Uses: def:weil-scalar-extension-endomorphism

Lean: Weil.Etale.scalarExtensionInverseCandidate

Suppose $v : M \to M$ is Λ-linear and satisfies $v \circ u = \mathrm{id}_M = u \circ v$. Define the inverse candidate for u_E to be $v_E = \mathrm{id}_E \otimes v$.

Lemma 4.3.9 (The scalar-extended inverse candidate is a left inverse). *Uses:* *def:weil-scalar-extension-endomorphism, def:weil-scalar-extension-inverse-candidate*

Lean: *Weil.Etale.scalarExtensionInverseCandidate_left*

Under the two inverse equations in Definition 4.3.8, $v_E \circ u_E = \mathrm{id}$.

Proof. Uses: def:weil-scalar-extension-endomorphism,def:weil-scalar-extension-inverse-candidate,lemma:weil-scalar-extension-pure-tensor

On a pure tensor,

$$(v_E \circ u_E)(a \otimes m) = v_E(a \otimes u(m)) = a \otimes v(u(m)) = a \otimes m.$$

The tensor-product induction principle says that two linear maps agreeing on all pure tensors are equal. Hence the composite is the identity. □

Lemma 4.3.10 (The scalar-extended inverse candidate is a right inverse). *Uses:* *def:weil-scalar-extension-endomorphism, def:weil-scalar-extension-inverse-candidate*

Lean: *Weil.Etale.scalarExtensionInverseCandidate_right*

Under the two inverse equations in Definition 4.3.8, $u_E \circ v_E = \mathrm{id}$.

Proof. Uses: def:weil-scalar-extension-endomorphism,def:weil-scalar-extension-inverse-candidate,lemma:weil-scalar-extension-pure-tensor

For every pure tensor,

$$(u_E \circ v_E)(a \otimes m) = u_E(a \otimes v(m)) = a \otimes u(v(m)) = a \otimes m.$$

Tensor-product induction identifies the two linear maps on the whole tensor product. □

Proposition 4.3.11 (Conditional preservation of invertibility under scalar extension). *Uses:* *def:weil-scalar-extension-endomorphism, def:weil-scalar-extension-inverse-candidate*

Lean: *Weil.Etale.scalarExtensionEndomorphism_isUnit*

If u is supplied together with a two-sided Λ-linear inverse v, then u_E is invertible with inverse v_E.

Proof. Uses: def:weil-scalar-extension-inverse-candidate,lemma:weil-scalar-extension-left-inverse,lemma:weil-scalar-extension-right-inverse

The two preceding lemmas are exactly the left- and right-inverse fields required to package v_E as the inverse of u_E. □

4.4 Typed field, local, and scheme Frobenius data

Definition 4.4.1 (Descended scheme datum). Uses: def:weil-finite-field-data

Lean: Weil.Etale.DescendedSchemeData

A descended scheme datum consists of a k_0-scheme X_0, a k-scheme X, a projection $\pi : X \to X_0$, and a specified cartesian square identifying X with $X_0 \times_{\mathrm{Spec}\, k_0} \mathrm{Spec}\, k$. The cartesian square is part of the input data.

Definition 4.4.2 (Closed-point arithmetic datum). Uses: def:weil-prime-power-data,def:weil-finite-field-data,def:weil-descended-scheme-data

Lean: Weil.Etale.ClosedPointArithmeticData

For a descended finite-type scheme X_0, a closed-point arithmetic datum consists of a closed point x, a positive integer e, a specified k_0-algebra isomorphism $\kappa(x) \simeq \mathbf{F}_{q^e}$, an algebraic closure $\overline{\kappa(x)}$, and its embedding of $\kappa(x)$. Write $\deg(x) = e$ and $q_x = q^e$. This is an input type. The residue-field finiteness and degree fields are discharged independently. Its selected finite-field identification and geometric embedding are supplied by the explicit TraceCore3 adapter below.

Definition 4.4.3 (Local arithmetic field Frobenius). Uses: def:weil-arithmetic-field-frobenius,def:weil-closed-point-arithmetic-data

Lean: Weil.Etale.localArithmeticFieldFrobenius

For a closed-point arithmetic datum, define

$$\varphi_x : \overline{\kappa(x)} \longrightarrow \overline{\kappa(x)}, \qquad u \longmapsto u^{q_x},$$

by instantiating the arithmetic field Frobenius at the prime power q_x.

Definition 4.4.4 (Local geometric field Frobenius). Uses: def:weil-geometric-field-frobenius,def:weil-closed-point-arithmetic-data,def:weil-local-arithmetic-field-frobenius

Lean: Weil.Etale.localGeometricFieldFrobenius

Define

$$F_x^{\mathrm{fld}} := \varphi_x^{-1}$$

by specializing the previously defined geometric field Frobenius. This is the inverse field automorphism and not an inverse scheme morphism.

Lemma 4.4.5 (Local field Frobenius inverse equations). *Uses: def:weil-local-arithmetic-field-frobenius, def:weil-local-geometric-field-frobenius*

Lean: Weil.Etale.localGeometricFieldFrobenius_inverse

One has

$$F_x^{\mathrm{fld}} \circ \varphi_x = \mathrm{id}, \qquad \varphi_x \circ F_x^{\mathrm{fld}} = \mathrm{id}\,.$$

Proof. Uses: def:weil-local-arithmetic-field-frobenius,def:weil-local-geometric-field-frobenius,lemma:weil-frobenius-inverse-identities

Instantiate Lemma 1.0.7 with the finite field $\kappa(x)$, its algebraic closure, and the prime power $q_x = q^e$. Its two conclusions are the two displayed equations. □

Definition 4.4.6 (Local decomposition-group Frobenius predicate). Uses: def:weil-closed-point-arithmetic-data,def:weil-local-geometric-field-frobenius

Lean: Weil.Etale.IsLocalDecompositionFrobenius

Given a geometric point $\bar{x}$ over x, a proposed decomposition group $D_{\bar{x}}$, and $g \in D_{\bar{x}}$, define IsLocalFrobenius(g) to mean that the induced automorphism of $\overline{\kappa(x)}$ equals F_x^{fld}. The decomposition group, the element, and the induced action are inputs to this predicate.

Definition 4.4.7 (Scheme Frobenius candidate). Uses: def:weil-descended-scheme-data

Lean: Weil.Etale.SchemeFrobeniusCandidate

A scheme Frobenius candidate is a specified scheme endomorphism

$$F_X^{\mathrm{sch}} : X \longrightarrow X.$$

No affine formula or geometric property is included in the candidate.

Definition 4.4.8 (Typed affine q-power algebra endomorphism). Lean: conditional-interface: def:weil-affine-q-power-ring-endomorphism

Uses: def:weil-prime-power-data,def:weil-finite-field-data

For an $\mathbf{F}_q$-algebra A, define the bundled $\mathbf{F}_q$-algebra endomorphism

$$\varphi_{A,q} : A \longrightarrow_{\mathbf{F}_q}^{\mathrm{alg}} A, \qquad a \longmapsto a^q.$$

If $q = p^a$, iteration of $(x+y)^p = x^p + y^p$ supplies the additive field; $0^q = 0$, $1^q = 1$, and $(xy)^q = x^q y^q$ supply the other ring fields. The identity $c^q = c$ on $\mathbf{F}_q$ supplies the algebra-homomorphism field.

Definition 4.4.9 (Canonical pullback model). Lean: `conditional-interface:def:weil-descended-canonical-pullback-model`

Uses: `def:weil-descended-scheme-data`

For a descended datum define the categorical pullback

$$P_X := X_0 \times_{\operatorname{Spec} \mathbf{F}_q} \operatorname{Spec} k$$

with its two selected projections.

Definition 4.4.10 (Canonical pullback comparison for the chosen X). Lean: `conditional-interface:def:weil-descended-scheme-pullback-comparison`

Uses: `def:weil-descended-scheme-data,def:weil-descended-canonical-pullback-model`

The cartesian square stored in the descended datum induces the selected comparison isomorphism

$$\chi_X : X \xrightarrow{\sim} P_X$$

whose composites with the pullback projections are the two supplied structural maps.

Definition 4.4.11 (Chosen descended affine pullback coordinate). Lean: `conditional-interface:def:weil-descended-affine-pullback-coordinate`

Uses: `def:weil-descended-scheme-data,def:weil-descended-scheme-pullback-comparison`

For a descended affine $U_0 = \operatorname{Spec} A_0 \subseteq X_0$, put $U = \pi^{-1}(U_0)$. Restriction of χ_X and the canonical affine pullback identification define

$$\theta_{U_0} : U \xrightarrow{\sim} \operatorname{Spec}(A_0 \otimes_{\mathbf{F}_q} k).$$

The two projection equations are fields of this coordinate datum.

Definition 4.4.12 (Chosen descended affine coordinate-ring equivalence). Lean: `conditional-interface:def:weil-descended-affine-coordinate-ring-equivalence`

Uses: `def:weil-descended-affine-pullback-coordinate`

Let $\theta_{U_0}^{\#}$ be the algebra equivalence from the tensor algebra to global sections induced contravariantly by θ_{U_0}. Define

$$\Theta_{U_0} := (\theta_{U_0}^{\#})^{-1} : \Gamma(U, \mathcal{O}_U) \xrightarrow{\sim}{}^{\mathrm{alg}}_{\mathbf{F}_q} A_0 \otimes_{\mathbf{F}_q} k.$$

This orientation is fixed for every conjugation formula below.

Definition 4.4.13 (Descended affine tensor endomorphism). Lean: `conditional-interface:def:weil-descended-affine-frobenius-tensor-map`

Uses: `def:weil-affine-q-power-ring-endomorphism,def:weil-descended-affine-pullback-coordinate`

Define

$$\psi_{A_0,k} := \varphi_{A_0,q} \otimes \mathrm{id}_k : A_0 \otimes_{\mathbf{F}_q} k \longrightarrow^{\mathrm{alg}}_{\mathbf{F}_q} A_0 \otimes_{\mathbf{F}_q} k$$

by the tensor-product map of the two displayed algebra homomorphisms.

Definition 4.4.14 (Typed descended affine Frobenius predicate). Lean: `conditional-interface:def:weil-typed-descended-affine-frobenius`

Uses: `def:weil-descended-scheme-data,def:weil-scheme-frobenius-candidate,def:weil-descended-affine-pullback-coordinate,def:weil-descended-affine-coordinate-ring-equivalence,def:weil-descended-affine-frobenius-tensor-map`

For a candidate $F : X \to X$, define IsTypedDescendedAffineFrobenius(F) to consist, for each $U_0 = \operatorname{Spec} A_0 \subseteq X_0$, of the following typed fields:

$$F^{-1}(U) = U, \qquad F_U : U \longrightarrow U, \qquad j_U \circ F_U = F \circ j_U,$$

where $U = \pi^{-1}(U_0)$ and $j_U : U \hookrightarrow X$, together with

$$\Theta_{U_0} \circ F_U^{\#} \circ \Theta_{U_0}^{-1} = \psi_{A_0,k}.$$

The restricted map F_U is the restriction selected by the stability field; the displayed square records its typing rather than treating a partial map as an endomorphism.

Definition 4.4.15 (Descended affine Frobenius formula predicate). Uses: `def:weil-typed-descended-affine-frobenius`

Lean: `Weil.Etale.IsDescendedAffineFrobenius`

Define IsDescendedAffineFrobenius(F) to mean IsTypedDescendedAffineFrobenius(F). Thus the legacy name is a typed alias whose stability, restriction, chosen-coordinate, and conjugacy fields are exactly those in the preceding record.

Definition 4.4.16 (Canonical semilinear field morphism). Lean: `conditional-interface:def:weil-canonical-semilinear-field-morphism`

Uses: `def:weil-geometric-field-frobenius`, `def:weil-descended-canonical-pullback-model`

On P_X, define

$$S_{P_X} := \mathrm{id}_{X_0} \times \mathrm{Spec}(F_k) : P_X \longrightarrow P_X$$

by the pullback universal property. Its pullback on the k-coordinate is the fixed ring automorphism $F_k : k \to k$.

Definition 4.4.17 (Semilinear field morphism on the chosen X). Lean: `conditional-interface:def:weil-semilinear-field-morphism-on-geometric-scheme`

Uses: `def:weil-canonical-semilinear-field-morphism`, `def:weil-descended-scheme-pullback-comparison`

Define the transported semilinear morphism

$$S_X := \chi_X^{-1} \circ S_{P_X} \circ \chi_X : X \longrightarrow X.$$

Definition 4.4.18 (Field–scheme Frobenius comparison predicate). Uses: `def:weil-descended-scheme-data`, `def:weil-scheme-frobenius-candidate`, `def:weil-semilinear-field-morphism-on-geometric-scheme`

Lean: `Weil.Etale.IsFieldSchemeFrobeniusComparison`

For a proposed absolute q-power map $\mathrm{Fr}^{\mathrm{abs}}_{X,q} : X \to X$, define the comparison predicate on the supplied cartesian model to be

$$F^{\mathrm{sch}}_X = \mathrm{Fr}^{\mathrm{abs}}_{X,q} \circ S_X.$$

The factor S_X is already transported to X; no morphism on P_X is composed directly with a morphism on the chosen model.

Definition 4.4.19 (Iterate of a scheme Frobenius candidate). Uses: `def:weil-scheme-frobenius-candidate`

Lean: `Weil.Etale.SchemeFrobeniusCandidate.iterate`

Define $(F^{\mathrm{sch}}_X)^{[0]} = \mathrm{id}_X$ and recursively

$$(F^{\mathrm{sch}}_X)^{[m+1]} := F^{\mathrm{sch}}_X \circ (F^{\mathrm{sch}}_X)^{[m]}.$$

The brackets distinguish a scheme-map iterate from a pullback functor and from a cohomological operator.

Lemma 4.4.20 (Zeroth scheme Frobenius iterate). *Uses:* *`def:weil-scheme-frobenius-iterate`*

Lean: *`Weil.Etale.SchemeFrobeniusCandidate.iterate_zero`*

One has $(F^{\mathrm{sch}}_X)^{[0]} = \mathrm{id}_X$.

Proof. Uses: `def:weil-scheme-frobenius-iterate`

This is the zero clause of the recursive definition. □

Lemma 4.4.21 (Successor scheme Frobenius iterate). *Uses:* *`def:weil-scheme-frobenius-iterate`*

Lean: *`Weil.Etale.SchemeFrobeniusCandidate.iterate_succ`*

For every m,

$$(F^{\mathrm{sch}}_X)^{[m+1]} = F^{\mathrm{sch}}_X \circ (F^{\mathrm{sch}}_X)^{[m]}.$$

Proof. Uses: `def:weil-scheme-frobenius-iterate`

This is the successor clause of the recursive definition. □

4.4.1 Comparison with ordinary categorical powers

Definition 4.4.22 (Ordinary categorical power of a scheme endomorphism). Uses: `def:weil-scheme-frobenius-candidate`, `def:weil-scheme-frobenius-iterate`

Lean: `Weil.Etale.schemeEndomorphismOrdinaryPower`

For a scheme Frobenius candidate $F = F^{\mathrm{sch}}_X : X \to X$ and $m \in \mathbf{N}$, define $F^{\langle m \rangle}$ to be the composite of the length-m constant string

$$X \xrightarrow{F} X \xrightarrow{F} \cdots \xrightarrow{F} X$$

in the category of schemes. The empty string has composite id_X. Thus this definition uses the ordinary composition and identity operations in the endomorphism monoid $\mathrm{End}_{\mathrm{Sch}}(X)$; it is independent of the recursive bracket notation in Definition 4.4.19.

Lemma 4.4.23 (Zero ordinary categorical power)**.** *Uses:* def:weil-scheme-endomorphism-ordinary-power

Lean: `Weil.Etale.schemeEndomorphismOrdinaryPower_zero`

One has $F^{\langle 0\rangle} = \mathrm{id}_X$.

Proof. Uses: def:weil-scheme-endomorphism-ordinary-power

The indexing ordinal for the length-zero string is empty. By the definition of composition of an empty finite string in a category, its composite is the identity of its unique endpoint, here id_X. □

Lemma 4.4.24 (Successor ordinary categorical power)**.** *Uses:* def:weil-scheme-endomorphism-ordinary-power

Lean: `Weil.Etale.schemeEndomorphismOrdinaryPower_succ`

For every $m \in \mathbf{N}$,

$$F^{\langle m+1\rangle} = F \circ F^{\langle m\rangle}.$$

Proof. Uses: def:weil-scheme-endomorphism-ordinary-power

Split the ordered string of $m+1$ copies of F into its final displayed arrow and the preceding string of m copies. Functorial associativity of finite composition identifies its composite with $F \circ F^{\langle m\rangle}$. For $m = 0$, the right unit law reduces this to $F \circ \mathrm{id}_X = F$, so the formula also covers the first successor. □

Lemma 4.4.25 (Addition law for ordinary categorical powers)**.** *Uses:* def:weil-scheme-endomorphism-ordinary-power

Lean: `Weil.Etale.schemeEndomorphismOrdinaryPower_add`

For all $m, n \in \mathbf{N}$,

$$F^{\langle m+n\rangle} = F^{\langle m\rangle} \circ F^{\langle n\rangle}.$$

Proof. Uses: lemma:weil-scheme-endomorphism-ordinary-power-zero, lemma:weil-scheme-endomorphism-ordinary-power-succ

Induct on m. At $m = 0$, the assertion is the left unit equation $\mathrm{id}_X \circ F^{\langle n\rangle} = F^{\langle n\rangle}$. For the successor step, apply the successor formula to the left side and to the first factor on the right, substitute the induction hypothesis, and use associativity:

$$F^{\langle (m+1)+n\rangle} = F \circ F^{\langle m+n\rangle} = F \circ (F^{\langle m\rangle} \circ F^{\langle n\rangle}) = (F \circ F^{\langle m\rangle}) \circ F^{\langle n\rangle}.$$

The last expression is $F^{\langle m+1\rangle} \circ F^{\langle n\rangle}$. □

Lemma 4.4.26 (Uniqueness of the ordinary-power recursion)**.** *Uses:* def:weil-scheme-endomorphism-ordinary-power

Lean: `Weil.Etale.schemeEndomorphismOrdinaryPower_unique`

Let $P : \mathbf{N} \to \mathrm{End}_{\mathrm{Sch}}(X)$ *satisfy*

$$P(0) = \mathrm{id}_X, \qquad P(m+1) = F \circ P(m)$$

for every m. *Then* $P(m) = F^{\langle m\rangle}$ *for every* m.

Proof. Uses: lemma:weil-scheme-endomorphism-ordinary-power-zero, lemma:weil-scheme-endomorphism-ordinary-power-succ

Induct on m. The zero equation for P, followed by the zero ordinary power lemma, proves the base case. In the successor step, use the recursion for P, substitute the induction hypothesis, and apply the successor ordinary-power lemma:

$$P(m+1) = F \circ P(m) = F \circ F^{\langle m\rangle} = F^{\langle m+1\rangle}.$$

No commutativity of endomorphisms is used, so the order agrees with the scheme-map composition convention fixed in the bracket recursion. □

Proposition 4.4.27 (Bracket iterates are ordinary scheme-map powers)**.** *Uses:* def:weil-scheme-frobenius-iterate, def:weil-scheme-endomorphism-ordinary-power

Lean: `Weil.Etale.schemeFrobeniusIterate_eq_ordinaryPower`

For every scheme Frobenius candidate F_X^{sch} *and every* $m \in \mathbf{N}$,

$$(F_X^{\mathrm{sch}})^{[m]} = (F_X^{\mathrm{sch}})^{\langle m\rangle}.$$

Consequently the categorical recursion used by the blueprint is exactly the ordinary m*-fold scheme-map composite, including the empty and successor cases.*

Proof. Uses: `lemma:weil-scheme-frobenius-iterate-zero`, `lemma:weil-scheme-frobenius-iterate-succ`, `lemma:weil-scheme-endomorphism-ordinary-power-unique`

Apply the uniqueness lemma to $P(m) := (F_X^{\mathrm{sch}})^{[m]}$. Its zero hypothesis is Lemma 4.4.20; its successor hypothesis is Lemma 4.4.21. The resulting equality is the displayed comparison for every m. □

Theorem 4.4.28 (SCHFROB–12 discharge). Uses: *`def:weil-scheme-frobenius-iterate`, `def:weil-scheme-endomorphism-ordinary-power`*

Lean: *`Weil.Etale.schFrob12`*

The recursively defined scheme Frobenius iterate is identified, in the category of schemes and with the fixed composition order, with every ordinary scheme-map power required by the scheme-iterate comparison interface.

Proof. Uses: `prop:weil-scheme-frobenius-iterate-ordinary-power`, `lemma:weil-scheme-endomorphism-ordinary-power-add`

The proposition gives the comparison in every degree. The addition lemma shows that these comparisons respect concatenation of finite strings, so the identification is one of categorical powers rather than only a pointwise family of equal endomorphisms. The zero and successor cases, and hence the chosen order of composition, were proved separately above. These are all clauses of the stated obligation. □

Definition 4.4.29 (Local/global iterate comparison predicate). Uses: `def:weil-closed-point-arithmetic-data`, `def:weil-local-decomposition-frobenius-predicate`, `def:weil-scheme-frobenius-iterate`

Lean: `Weil.Etale.IsLocalGlobalFrobeniusIterate`

Let $m \geq 1$, let $e = \deg(x)$ divide m, and let y be a geometric fixed point above x. Define the local/global iterate predicate to be

$$\Phi_y^{[m]} \sim \mathrm{Frob}_x^{m/e},$$

where $\sim$ means equality after a specified stalk transport and means conjugacy when that transport is omitted. The two operators and the transport are inputs; this definition proves no comparison.

> **Status.** The source for the typed dictionary is Deligne, Weil II, Sections 1.1.8–1.1.14 (article pp. 151–153; PDF pp. 16–18), especially Definition 1.1.10 and paragraph 1.1.14. The normalization control is the supplied Weil notes, Section 5.3 (printed p. 29; PDF p. 39). Every property of the scheme and local candidates remains OPEN in the manifest.

4.5 Abstract Weil data and ordered cohomological iterates

Definition 4.5.1 (Pullback-endofunctor context). Uses:

Lean: `Weil.Etale.PullbackEndofunctorContext`

A pullback-endofunctor context consists of a category $\mathscr{C}$ and a specified endofunctor $G : \mathscr{C} \to \mathscr{C}$. In the intended instance $G = (F_X^{\mathrm{sch}})^*$, but this abstract definition assumes no geometric construction.

Definition 4.5.2 (Abstract Weil datum). Uses: `def:weil-pullback-endofunctor-context`

Lean: `Weil.Etale.AbstractWeilDatum`

For a pullback-endofunctor context, an abstract Weil datum consists of an object $K \in \mathscr{C}$ and a specified isomorphism

$$\phi : G(K) \xrightarrow{\sim} K.$$

This is conditional input data.

Definition 4.5.3 (Iterated pullback object). Uses: `def:weil-pullback-endofunctor-context`, `def:weil-abstract-weil-datum`

Lean: `Weil.Etale.AbstractWeilDatum.iteratedObject`

Define $K_{[0]} := K$ and recursively $K_{[m+1]} := G(K_{[m]})$.

Definition 4.5.4 (Ordered iterate of a Weil map). Uses: `def:weil-pullback-endofunctor-context`, `def:weil-abstract-weil-datum`, `def:weil-iterated-pullback-object`

Lean: `Weil.Etale.AbstractWeilDatum.iteratedIso`

Define $\phi^{[m]} : K_{[m]} \xrightarrow{\sim} K$ recursively by

$$\phi^{[0]} := \mathrm{id}_K, \qquad \phi^{[m+1]} := \phi \circ G(\phi^{[m]}).$$

The rightmost map is applied first.

Lemma 4.5.5 (Zeroth Weil-map iterate). *Uses:* `def:weil-iterated-pullback-object`, `def:weil-weil-map-iterate`

Lean: `Weil.Etale.AbstractWeilDatum.iteratedIso_zero`

The map $\phi^{[0]} : K_{[0]} \to K$ *is* id_K.

Proof. *Uses:* `def:weil-iterated-pullback-object`, `def:weil-weil-map-iterate`

The zero clause gives $K_{[0]} = K$ and $\phi^{[0]} = \mathrm{id}_K$. □

Lemma 4.5.6 (Successor Weil-map iterate). *Uses:* `def:weil-weil-map-iterate`

Lean: `Weil.Etale.AbstractWeilDatum.iteratedIso_succ`

For every m,

$$\phi^{[m+1]} = \phi \circ G(\phi^{[m]}).$$

Proof. *Uses:* `def:weil-weil-map-iterate`

This is the successor clause of the recursive definition. □

Definition 4.5.7 (Cohomology pullback-comparison datum). *Uses:* `def:weil-coefficient-data`, `def:weil-pullback-endofunctor-context`

Lean: `Weil.Etale.CohomologyPullbackComparisonData`

Let Vect_E be the category of E-vector spaces. A cohomology pullback-comparison datum consists of a functor $H : \mathscr{C} \to \mathrm{Vect}_E$ and a specified natural isomorphism

$$\tau : H \xRightarrow{\sim} H \circ G.$$

No etale cohomology functor or topological-invariance map is constructed here.

Lemma 4.5.8 (Naturality of the pullback comparison). *Uses:* `def:weil-cohomology-pullback-comparison-data`

Lean: `Weil.Etale.CohomologyPullbackComparisonData.naturality`

For every morphism $f : A \to B$ *in* $\mathscr{C}$,

$$H(Gf) \circ \tau_A = \tau_B \circ H(f).$$

Proof. *Uses:* `def:weil-cohomology-pullback-comparison-data`

This is the naturality field of τ, instantiated once at f. □

Definition 4.5.9 (Iterated cohomology pullback comparison). *Uses:* `def:weil-iterated-pullback-object`, `def:weil-cohomology-pullback-comparison-data`

Lean: `Weil.Etale.CohomologyPullbackComparisonData.iterate`

Define $\tau_K^{[m]} : H(K) \xrightarrow{\sim} H(K_{[m]})$ recursively by

$$\tau_K^{[0]} := \mathrm{id}_{H(K)}, \qquad \tau_K^{[m+1]} := \tau_{K_{[m]}} \circ \tau_K^{[m]}.$$

Lemma 4.5.10 (Zeroth iterated pullback comparison). *Uses:* `def:weil-cohomology-pullback-comparison-iterate`

Lean: `Weil.Etale.CohomologyPullbackComparisonData.iterate_zero`

One has $\tau_K^{[0]} = \mathrm{id}_{H(K)}$.

Proof. *Uses:* `def:weil-cohomology-pullback-comparison-iterate`

This is the zero clause of the recursive definition. □

Lemma 4.5.11 (Successor iterated pullback comparison). *Uses:* `def:weil-cohomology-pullback-comparison-iterate`

Lean: `Weil.Etale.CohomologyPullbackComparisonData.iterate_succ`

For every m,

$$\tau_K^{[m+1]} = \tau_{K_{[m]}} \circ \tau_K^{[m]}.$$

Proof. *Uses:* `def:weil-cohomology-pullback-comparison-iterate`

This is the successor clause of the recursive definition. □

Definition 4.5.12 (Induced cohomological Frobenius endomorphism)**.** Uses: `def:weil-abstract-weil-datum`, `def:weil-cohomology-pullback-comparison-data`

Lean: `Weil.Etale.inducedCohomologyFrobenius`

For an abstract Weil datum (K, ϕ), define

$$\Phi_K := H(\phi) \circ \tau_K : H(K) \longrightarrow H(K).$$

This definition records only the linear endomorphism.

Definition 4.5.13 (Inverse candidate for induced cohomological Frobenius)**.** Uses: `def:weil-abstract-weil-datum`, `def:weil-cohomology-pullback-comparison-data`, `def:weil-induced-cohomology-frobenius`

Lean: `Weil.Etale.inducedCohomologyFrobeniusInverseCandidate`

Define

$$\Psi_K := \tau_K^{-1} \circ H(\phi^{-1}) : H(K) \longrightarrow H(K).$$

Lemma 4.5.14 (The cohomological inverse candidate is a left inverse)**.** *Uses:* *def:weil-induced-cohomology-frobenius*, *def:weil-induced-cohomology-frobenius-inverse-candidate*

Lean: *Weil.Etale.inducedCohomologyFrobeniusInverseCandidate_left*

One has $\Psi_K \circ \Phi_K = \mathrm{id}_{H(K)}$.

Proof. Uses: `def:weil-induced-cohomology-frobenius`, `def:weil-induced-cohomology-frobenius-inverse-candidate`

Expand the two definitions and use functoriality once:

$$\Psi_K \circ \Phi_K = \tau_K^{-1} \circ H(\phi^{-1}) \circ H(\phi) \circ \tau_K = \tau_K^{-1} \circ H(\phi^{-1} \circ \phi) \circ \tau_K = \tau_K^{-1} \circ \tau_K.$$

The last composite is the identity by the inverse equation for τ_K. □

Lemma 4.5.15 (The cohomological inverse candidate is a right inverse)**.** *Uses:* *def:weil-induced-cohomology-frobenius*, *def:weil-induced-cohomology-frobenius-inverse-candidate*

Lean: *Weil.Etale.inducedCohomologyFrobeniusInverseCandidate_right*

One has $\Phi_K \circ \Psi_K = \mathrm{id}_{H(K)}$.

Proof. Uses: `def:weil-induced-cohomology-frobenius`, `def:weil-induced-cohomology-frobenius-inverse-candidate`

Expand and cancel the adjacent inverse comparison maps:

$$\Phi_K \circ \Psi_K = H(\phi) \circ \tau_K \circ \tau_K^{-1} \circ H(\phi^{-1}) = H(\phi) \circ H(\phi^{-1}) = H(\phi \circ \phi^{-1}) = \mathrm{id}_{H(K)}.$$

□

Proposition 4.5.16 (Induced cohomological Frobenius is invertible)**.** *Uses:* *def:weil-induced-cohomology-frobenius*, *def:weil-induced-cohomology-frobenius-inverse-candidate*

Lean: *Weil.Etale.inducedCohomologyFrobenius_isUnit*

The endomorphism Φ_K *is invertible with inverse* Ψ_K.

Proof. Uses: `lemma:weil-induced-cohomology-frobenius-left-inverse`, `lemma:weil-induced-cohomology-frobenius-right-inverse`

The two cited lemmas supply the two inverse equations required for an invertible endomorphism. □

Definition 4.5.17 (Cohomological operator of an ordered Weil iterate)**.** Uses: `def:weil-weil-map-iterate`, `def:weil-cohomology-pullback-comparison-iterate`, `def:weil-induced-cohomology-frobenius`

Lean: `Weil.Etale.inducedCohomologyFrobeniusIterate`

Define

$$\Phi_K^{[m]} := H(\phi^{[m]}) \circ \tau_K^{[m]} : H(K) \longrightarrow H(K).$$

Lemma 4.5.18 (Zeroth cohomological Frobenius iterate)**.** *Uses:* *def:weil-induced-cohomology-frobenius-iterate*

Lean: *Weil.Etale.inducedCohomologyFrobeniusIterate_zero*

One has $\Phi_K^{[0]} = \mathrm{id}_{H(K)}$.

Proof. Uses: def:weil-induced-cohomology-frobenius-iterate,lemma:weil-weil-map-iterate-zero,lemma:weil-cohomology-pullback-comparison-iterate-zero

At $m = 0$, the two factors in the defining composite are identities, so their composite is the identity. □

Lemma 4.5.19 (Successor cohomological Frobenius iterate). Uses: def:weil-induced-cohomology-frobenius, def:weil-induced-cohomology-frobenius-iterate

Lean: Weil.Etale.inducedCohomologyFrobeniusIterate_succ

For every m,

$$\Phi_K^{[m+1]} = \Phi_K \circ \Phi_K^{[m]}.$$

Proof. Uses: def:weil-induced-cohomology-frobenius,def:weil-induced-cohomology-frobenius-iterate,lemma:weil-weil-map-iterate-succ,lemma:weil-cohomology-pullback-comparison-iterate-succ,lemma:weil-cohomology-pullback-comparison-naturality

The two successor definitions give

$$\Phi_K^{[m+1]} = H(\phi) \circ H(G\phi^{[m]}) \circ \tau_{K_{[m]}} \circ \tau_K^{[m]}.$$

Apply naturality once to $\phi^{[m]} : K_{[m]} \to K$, replacing the middle composite by $\tau_K \circ H(\phi^{[m]})$. Regrouping the four maps gives $\Phi_K \circ \Phi_K^{[m]}$. □

Proposition 4.5.20 (Ordered Weil iterates induce ordinary powers). Uses: def:weil-induced-cohomology-frobenius, def:weil-induced-cohomology-frobenius-iterate

Lean: Weil.Etale.inducedCohomologyFrobeniusIterate_eq_pow

For every $m \in \mathbf{N}$, $\Phi_K^{[m]} = \Phi_K^m$.

Proof. Uses: lemma:weil-induced-cohomology-frobenius-iterate-zero,lemma:weil-induced-cohomology-frobenius-iterate-succ

Induct on m. The zero case is the preceding zero lemma. For the successor, substitute the induction hypothesis into the successor lemma and use the successor definition $\Phi_K^{m+1} = \Phi_K \circ \Phi_K^m$. □

> **Formalization note.** The ordered map agrees with the supplied Weil notes, Definition 5.9 (printed p. 30; PDF p. 40). The generic argument above is category theory. Constructing its geometric instance requires the OPEN topological-invariance, rational-coefficient, and descent nodes in the manifest.

4.6 Tate-object targets without an existence assertion

Definition 4.6.1 (Kummer-level candidate). Uses: def:weil-small-etale-topology,def:weil-coefficient-data

Lean: Weil.Etale.KummerLevelCandidate

For $r \geq 1$, a Kummer-level candidate consists of an abelian sheaf $\mathcal{M}_r$ on X_{et} and a morphism $i_r : \mathcal{M}_r \to h_{\mathbf{G}_m}$. No kernel property is included.

Definition 4.6.2 (Kummer-kernel predicate). Uses: def:weil-kummer-level-candidate

Lean: Weil.Etale.IsKummerKernel

Define IsKummerKernel$(\mathcal{M}_r, i_r)$ to mean that i_r exhibits $\mathcal{M}_r$ as the kernel sheaf of $[ell^r] : h_{\mathbf{G}_m} \to h_{\mathbf{G}_m}$.

Definition 4.6.3 (Kummer transition candidate). Uses: def:weil-kummer-level-candidate

Lean: Weil.Etale.KummerTransitionCandidate

For Kummer-level candidates in all positive levels, a transition candidate is a family of sheaf morphisms $\rho_{r+1,r} : \mathcal{M}_{r+1} \to \mathcal{M}_r$.

Definition 4.6.4 (Kummer transition formula predicate). Uses: def:weil-kummer-level-candidate,def:weil-kummer-transition-candidate

Lean: Weil.Etale.IsKummerTransition

Define IsKummerTransition(ρ) to mean that

$$i_r \circ \rho_{r+1,r} = [\ell] \circ i_{r+1}$$

for every r. Thus, on geometric sections represented by roots of unity, the transition is $\zeta \mapsto \zeta^\ell$.

Definition 4.6.5 (Integral Tate cone candidate). Uses: `def:weil-kummer-level-candidate`, `def:weil-kummer-transition-candidate`

Lean: `Weil.Etale.IntegralTateConeCandidate`

An integral Tate cone candidate consists of an abelian sheaf L and maps $p_r : L \to \mathcal{M}_r$ such that $\rho_{r+1,r} \circ p_{r+1} = p_r$ for each r.

Definition 4.6.6 (Integral Tate limit predicate). Uses: `def:weil-integral-tate-cone-candidate`

Lean: `Weil.Etale.IsIntegralTateLimit`

Define IsIntegralTateLimit$(L, (p_r))$ to mean that for every compatible cone $(A, (a_r))$ there is exactly one map $A \to L$ whose composite with p_r is a_r for every r. This is the limit predicate; it does not provide such an L.

Definition 4.6.7 (Adic rationalization context). Uses: `def:weil-coefficient-extension-data`

Lean: `Weil.Etale.AdicRationalizationContext`

An adic rationalization context consists of an integral coefficient category $\mathscr{A}_\Lambda(X)$, a rational coefficient category $\mathscr{A}_E(X)$, and a specified functor $R_E : \mathscr{A}_\Lambda(X) \to \mathscr{A}_E(X)$. No normalized adic category or functor is constructed by this definition.

Definition 4.6.8 (Integral Tate-object candidate). Uses: `def:weil-integral-tate-cone-candidate`, `def:weil-integral-tate-limit-predicate`, `def:weil-adic-rationalization-context`

Lean: `Weil.Etale.IntegralTateObjectCandidate`

Given an adic rationalization context, an integral Tate-object candidate is an object $\Lambda(1) \in \mathscr{A}_\Lambda(X)$ equipped with comparison maps to the Kummer-level cone. The limit and normalization properties are separate predicates and are not fields of this candidate.

Definition 4.6.9 (Rational Tate-object candidate). Uses: `def:weil-adic-rationalization-context`, `def:weil-integral-tate-object-candidate`

Lean: `Weil.Etale.RationalTateObjectCandidate`

For an integral Tate-object candidate define the rational candidate by

$$E(1)_{\mathrm{cand}} := R_E\big(\Lambda(1)\big).$$

This is a conditional application of the supplied rationalization functor.

Definition 4.6.10 (Etale Tate normalization predicate). Uses: `def:weil-prime-power-data`, `def:weil-tate-line`, `def:weil-rational-tate-object-candidate`, `def:weil-abstract-weil-datum`, `def:weil-induced-cohomology-frobenius`

Lean: `Weil.Etale.IsTateNormalized`

For a Weil datum on $E(1)_{\mathrm{cand}}$ and a supplied geometric-stalk realization, define IsTateNormalized to mean that the realized one-dimensional Frobenius module is isomorphic to the previously defined Tate line $E(1)$; equivalently its geometric Frobenius operator is $q^{-1}\,\mathrm{id}$. This is a predicate, not a normalization theorem.

Definition 4.6.11 (Degree- e local Tate normalization predicate). Uses: `def:weil-closed-point-arithmetic-data`, `def:weil-rational-tate-object-candidate`, `def:weil-etale-tate-normalization-predicate`

Lean: `Weil.Etale.IsLocalTateNormalized`

At a closed point of degree e, define the local Tate normalization predicate to be the equality

$$\mathrm{Frob}_x \mid E(1)_{\bar{x}} = q_x^{-1}\,\mathrm{id} = q^{-e}\,\mathrm{id}\,.$$

The equality $q_x^{-1} = q^{-e}$ uses the defining equation $q_x = q^e$; the Frobenius assertion remains a target.

> **Status.** The target construction is $\mathbf{Z}_\ell(1) = \varprojlim_r \mu_{\ell^r}$ and $\mathbf{Q}_\ell(1) = \mathbf{Z}_\ell(1) \otimes \mathbf{Q}_\ell$, as in the supplied Weil notes, Definition 5.7 and Proposition 5.8 (printed p. 30; PDF p. 40). None of the Kummer, limit, normalized-adic, or Frobenius-normalization predicates is proved in this chapter.

4.7 Conditional finite-dimensional Frobenius-module interface

Definition 4.7.1 (Cohomological finite-dimensionality predicate)**.** Uses: `def:weil-coefficient-data`, `def:weil-induced-cohomology-frobenius`

Lean: `Weil.Etale.IsFiniteDimensionalCohomology`

Define IsFiniteDimensionalCohomology(H, K) to mean that the E-vector space $H(K)$ has finite dimension.

Definition 4.7.2 (Cohomological Frobenius-module input)**.** Uses: `def:weil-induced-cohomology-frobenius`, `def:weil-induced-cohomology-frobenius-inverse-candidate`, `def:weil-cohomology-finite-dimensional-predicate`

Lean: `Weil.Etale.CohomologyFrobeniusModuleInput`

A cohomological Frobenius-module input consists of a proof of IsFiniteDimensionalCohomology(H, K) and the two displayed inverse equations for Φ_K and Ψ_K. This definition packages certificates already supplied as inputs and asserts no finiteness theorem.

Definition 4.7.3 (Conditional cohomological Frobenius module)**.** Uses: `def:weil-frobenius-module`, `def:weil-induced-cohomology-frobenius`, `def:weil-cohomology-frobenius-module-input`

Lean: `Weil.Etale.cohomologyFrobeniusModule`

Given a cohomological Frobenius-module input, define

$$\mathsf{FrobMod}(H, K) := (H(K), \Phi_K)$$

as a finite-dimensional Frobenius module in the sense of Definition 1.0.11.

Lemma 4.7.4 (Carrier of the conditional cohomological Frobenius module)**.** *Uses:* `def:weil-conditional-cohomology-frobenius-module`

Lean: `Weil.Etale.cohomologyFrobeniusModule_carrier`

The carrier of $\mathsf{FrobMod}(H, K)$ *is* $H(K)$.

Proof. Uses: `def:weil-conditional-cohomology-frobenius-module`

This is the first field of the defining ordered pair. □

Lemma 4.7.5 (Operator of the conditional cohomological Frobenius module)**.** *Uses:* `def:weil-conditional-cohomology-frobenius-module`

Lean: `Weil.Etale.cohomologyFrobeniusModule_operator`

The Frobenius operator of $\mathsf{FrobMod}(H, K)$ *is* Φ_K.

Proof. Uses: `def:weil-conditional-cohomology-frobenius-module`

This is the second field of the defining ordered pair. □

Definition 4.7.6 (Actual cohomology realization predicate)**.** Uses: `def:weil-raw-rationalized-cohomology`, `def:weil-rational-cohomology-comparison-predicate`, `def:weil-induced-cohomology-frobenius`, `def:weil-conditional-cohomology-frobenius-module`

Lean: `Weil.Etale.IsActualCohomologyRealization`

For a proposed instance $(\mathscr{C}, G, K, H, \tau, \phi)$, define IsActualCohomologyRealization to mean that its vector space is identified by a supplied rational comparison with $V_{X,\ell}^{n,\mathrm{raw}}$, and that this comparison intertwines the scalar-extended integral action with Φ_K. This is a property predicate; it does not construct the comparison or prove its bijectivity.

> **Status.** The supplied Weil notes, Theorem 5.6 (printed p. 29; PDF p. 39), is the source target for bounded finite-dimensional cohomology, but its proof capsule depends on the entire finite-coefficient and normalized-adic finiteness chain. Accordingly this chapter exports only a conditional constructor. It does not prove finite-dimensionality, compact-support finiteness, or the existence of an actual cohomological Frobenius module.

4.8 Typed conditional context for the six operations

Definition 4.8.1 (Typed constructible-derived context)**.** Uses: `def:weil-coefficient-data`

Lean: `Weil.Etale.ConstructibleDerivedTypedContext`

A typed constructible-derived context over the coefficient field E consists of the following data, with no coherence assertion included.

(i) For every scheme X in the stated finite-type universe, a category $\mathsf{Sh}_c(X,E)$ of constructible-sheaf candidates and a category $\mathcal{D}_c^b(X,E)$ of bounded constructible-derived candidates.

(ii) A functor $\iota_X : \mathsf{Sh}_c(X,E) \to \mathcal{D}_c^b(X,E)$, a sheaf $\underline{E}_X \in \mathsf{Sh}_c(X,E)$, and the abbreviation $E_X := \iota_X \underline{E}_X \in \mathcal{D}_c^b(X,E)$.

(iii) For every integer r, a shift endofunctor $[r]_X : \mathcal{D}_c^b(X,E) \to \mathcal{D}_c^b(X,E)$.

(iv) A bifunctor $\otimes_X^L : \mathcal{D}_c^b(X,E) \times \mathcal{D}_c^b(X,E) \to \mathcal{D}_c^b(X,E)$ and an internal-Hom bifunctor

$$R\mathcal{H}om_X : \mathcal{D}_c^b(X,E)^{\mathrm{op}} \times \mathcal{D}_c^b(X,E) \longrightarrow \mathcal{D}_c^b(X,E).$$

(v) For every integer i, a sheaf-valued functor $\mathcal{H}_X^i : \mathcal{D}_c^b(X,E) \to \mathsf{Sh}_c(X,E)$.

Exactness or full faithfulness of ι_X, and the triangulated, monoidal, boundedness, and t-structure axioms, are not fields of this raw typed context. Their construction remains part of OBL-ETALE-CONSTRUCTIBLE and OBL-SIX-FUNCTOR-ADIC.

Definition 4.8.2 (Typed geometric-point and point-base realizations)**.** Uses: `def:weil-finite-field-data`, `def:weil-constructible-derived-typed-context`

Lean: `Weil.Etale.PointRealizationTypedContext`

For every geometric point $\bar{x} \to X$, supply a functor

$$\mathrm{st}_{\bar{x}} : \mathsf{Sh}_c(X,E) \longrightarrow \mathrm{Vect}_E\,.$$

Put $S := \operatorname{Spec} k$. Supply also a point-base realization $\rho_S : \mathsf{Sh}_c(S,E) \to \mathrm{Vect}_E$, and, for $K \in \mathcal{D}_c^b(S,E)$, define

$$\mathbf{H}_S^i(K) := \rho_S\big(\mathcal{H}_S^i(K)\big).$$

No exactness, stalk theorem, or comparison between ρ_S and a selected geometric stalk is asserted.

Definition 4.8.3 (Typed derived Tate objects and twists)**.** Uses: `def:weil-constructible-derived-typed-context`

Lean: `Weil.Etale.DerivedTateTwistTypedContext`

For every scheme X and integer n, supply an endofunctor

$$(n)_X : \mathcal{D}_c^b(X,E) \longrightarrow \mathcal{D}_c^b(X,E).$$

For $K \in \mathcal{D}_c^b(X,E)$, define $K(n) := (n)_X K$, and define $E_X(n) := (n)_X(E_X)$. Tensor realization, positive tensor powers, negative dual powers, inverse equations, and Frobenius normalization are not asserted by this input type; they remain under OBL-SIX-FUNCTOR-ADIC and the Tate-object obligations of the etale interface.

Definition 4.8.4 (Typed four-operation context)**.** Uses: `def:weil-constructible-derived-typed-context`

Lean: `Weil.Etale.FourOperationsTypedContext`

A typed four-operation context assigns to every morphism $f : X \to Y$ the functors

$$Lf^* : \mathcal{D}_c^b(Y,E) \to \mathcal{D}_c^b(X,E), \qquad Rf_* : \mathcal{D}_c^b(X,E) \to \mathcal{D}_c^b(Y,E),$$

and, when f is supplied together with proofs that it is separated and of finite type, the functors

$$Rf_! : \mathcal{D}_c^b(X,E) \to \mathcal{D}_c^b(Y,E), \qquad Rf^! : \mathcal{D}_c^b(Y,E) \to \mathcal{D}_c^b(X,E).$$

The morphism, separation proof, and finite-type proof are explicit indices of the last two fields. Identity, composition, base-change, and adjunction laws are deliberately absent. Exceptional inverse image is always denoted $Rf^!$; for $f = a_X$ the literal specialization is $Ra_X^!$, and no bare $f^!$ notation is used below.

Definition 4.8.5 (Cohomology-object abbreviations for derived images)**.** Uses: `def:weil-constructible-derived-typed-context`, `def:weil-four-operations-typed-context`

Lean: `Weil.Etale.FourOperationCohomologyObjectAbbreviations`

For $\mathcal{F} \in \mathsf{Sh}_c(X,E)$, define

$$R^i f_* \mathcal{F} := \mathcal{H}_Y^i\big(Rf_*(\iota_X \mathcal{F})\big) \in \mathsf{Sh}_c(Y,E).$$

When f is supplied with separated finite-type proofs, also define

$$R^i f_! \mathcal{F} := \mathcal{H}_Y^i\big(Rf_!(\iota_X \mathcal{F})\big) \in \mathsf{Sh}_c(Y,E).$$

These are notation Definitions only. They assert neither constructibility nor a comparison with any independently proposed direct-image sheaf.

Definition 4.8.6 (Typed adjunction-map candidates)**.** Uses: `def:weil-four-operations-typed-context`

Lean: `Weil.Etale.SixFunctorAdjunctionMapCandidates`

For each indexed morphism $f : X \to Y$, typed adjunction-map candidates are natural transformations with components of the exact types

$$\eta_f^* : \mathrm{id}_{\mathcal{D}_c^b(Y,E)} \longrightarrow Rf_* Lf^*, \qquad \epsilon_{f,*} : Lf^* Rf_* \longrightarrow \mathrm{id}_{\mathcal{D}_c^b(X,E)},$$

and, in the separated finite-type case,

$$\eta_{f,!} : \mathrm{id}_{\mathcal{D}_c^b(X,E)} \longrightarrow Rf^! Rf_!, \qquad \epsilon_f^! : Rf_! Rf^! \longrightarrow \mathrm{id}_{\mathcal{D}_c^b(Y,E)}.$$

No triangle identity is included in this data type.

Definition 4.8.7 (Point-constant realization candidates)**.** Uses: `def:weil-point-realization-typed-context`

Lean: `Weil.Etale.PointConstantRealizationCandidate`

For $S = \operatorname{Spec} k$, supply maps

$$\epsilon_E : \mathbf{H}^0_S(E_S) \longrightarrow E, \qquad \eta_E : E \longrightarrow \mathbf{H}^0_S(E_S).$$

These are candidates only; the equations identifying degree-zero cohomology of the constant point object with E are not built into their types.

Definition 4.8.8 (Point-constant left-inverse predicate)**.** Uses: `def:weil-point-constant-realization-candidate`

Lean: `Weil.Etale.IsPointConstantLeftInverse`

Define the predicate by $\epsilon_E \circ \eta_E = \mathrm{id}_E$.

Definition 4.8.9 (Point-constant right-inverse predicate)**.** Uses: `def:weil-point-constant-realization-candidate`

Lean: `Weil.Etale.IsPointConstantRightInverse`

Define the predicate by $\eta_E \circ \epsilon_E = \mathrm{id}_{\mathbf{H}^0_S(E_S)}$.

Definition 4.8.10 (Point-constant realization certificate)**.** Uses: `def:weil-point-constant-realization-candidate,def:weil-point-constant-left-inverse-predicate,def:weil-point-constant-right-inverse-predicate`

Lean: `Weil.Etale.PointConstantRealizationCertificate`

A point-constant realization certificate is a candidate satisfying both preceding inverse predicates.

Open obligation: `OBL-ETALE-CONSTRUCTIBLE`

Definition 4.8.11 (Typed derived-global-section context)**.** Uses: `def:weil-four-operations-typed-context,def:weil-point-realization-typed-context`

Lean: `Weil.Etale.DerivedGlobalSectionTypedContext`

For every structural morphism $a_X : X \to S := \operatorname{Spec} k$, define

$$R\Gamma(X,K) := Ra_{X*}K, \qquad R\Gamma_c(X,K) := Ra_{X!}K$$

inside $\mathcal{D}_c^b(S,E)$. For every integer i, define

$$H^i(X,K) := \mathbf{H}^i_S(R\Gamma(X,K)), \qquad H^i_c(X,K) := \mathbf{H}^i_S(R\Gamma_c(X,K)).$$

The second abbreviation is indexed by explicit proofs that a_X is separated and of finite type. This definition supplies types only; it does not assert finite-dimensionality or comparison with a previously selected cohomology theory.

Definition 4.8.12 (Typed Cartesian base-change index)**.** Uses: `def:weil-four-operations-typed-context`

Lean: `Weil.Etale.CartesianBaseChangeIndex`

A typed Cartesian base-change index is a diagram

$$\begin{array}{ccc} X' & \xrightarrow{g'} & X \\ {\scriptstyle f'}\downarrow & & \downarrow{\scriptstyle f} \\ Y' & \xrightarrow{g} & Y \end{array}$$

in the fixed scheme universe, together with the equality $f \circ g' = g \circ f'$ and a proof that the square is Cartesian. A compact-support index additionally stores separated finite-type proofs for both f and f'; these proofs are data, not consequences silently invoked from stability under base change.

Definition 4.8.13 (Typed six-operation comparison-map candidates)**.** Uses: `def:weil-constructible-derived-typed-context`,`def:weil-derived-tate-twist-typed-context`,`def:weil-four-operations-typed-context`,`def:weil-six-functor-cartesian-base-change-index`

Lean: `Weil.Etale.SixFunctorComparisonMapCandidates`

Comparison-map candidates consist of the following separately typed natural transformations.

(i) For every separated finite-type f, a forget-supports map

$$\kappa_f : Rf_! \longrightarrow Rf_*$$

of functors $\mathcal{D}_c^b(X,E) \to \mathcal{D}_c^b(Y,E)$.

(ii) For every typed Cartesian base-change index

$$\begin{array}{ccc} X' & \xrightarrow{g'} & X \\ {\scriptstyle f'}\downarrow & & \downarrow{\scriptstyle f} \\ Y' & \xrightarrow{g} & Y, \end{array}$$

an ordinary exchange map, a compact-support exchange map when the stored separated finite-type proofs are present, and, under the same proofs, an exceptional-pullback mate:

$$\mathrm{BC}_{*,\square} : Lg^* Rf_* \longrightarrow Rf'_* Lg'^*,$$

$$\mathrm{BC}_{!,\square} : Lg^* Rf_! \longrightarrow Rf'_! Lg'^*, \qquad \mathrm{BC}^{!,\square} : Lg'^* Rf^! \longrightarrow Rf'^! Lg^*.$$

The first two sides are functors $\mathcal{D}_c^b(X,E) \to \mathcal{D}_c^b(Y',E)$; the two sides of the exceptional mate are functors $\mathcal{D}_c^b(Y,E) \to \mathcal{D}_c^b(X',E)$.

(iii) For $K \in \mathcal{D}_c^b(X,E)$ and $L \in \mathcal{D}_c^b(Y,E)$, the exact component types

$$\mathrm{PF}_{!,f,K,L} : (Rf_!K) \otimes_Y^L L \longrightarrow Rf_!\big(K \otimes_X^L Lf^*L\big),$$

$$\mathrm{PF}_{*,f,K,L} : (Rf_*K) \otimes_Y^L L \longrightarrow Rf_*\big(K \otimes_X^L Lf^*L\big).$$

The compactly supported component is indexed by the separated finite-type proofs for f.

(iv) For $K_1, K_2 \in \mathcal{D}_c^b(X,E)$, the exact ordinary and mixed-support cup components

$$\mathrm{Cup}_{*,f,K_1,K_2} : Rf_*K_1 \otimes_Y^L Rf_*K_2 \longrightarrow Rf_*(K_1 \otimes_X^L K_2),$$

$$\mathrm{Cup}_{!,*,f,K_1,K_2} : Rf_!K_1 \otimes_Y^L Rf_*K_2 \longrightarrow Rf_!(K_1 \otimes_X^L K_2).$$

The mixed-support component is indexed by the separated finite-type proofs for f. For $f = a_X$ and $K_1 = K_2 = E_X$, the second component has exactly the source and target required by the compact–ordinary cup construction in Module 04.

(v) For $n \in \mathbf{Z}$, $K \in \mathcal{D}_c^b(Y,E)$, and $f : X \to Y$, a pullback–twist comparison component

$$\mathrm{TwBC}_{f,n,K} : Lf^*(K(n)) \longrightarrow (Lf^*K)(n).$$

Naturality, monoidal-unit and associativity laws, isomorphism, inverse, base-change cocycle, projection-formula, twist coherence, and Frobenius compatibility are separate predicates and are not asserted here.

Definition 4.8.14 (Complete typed six-functor input context)**.** Uses: `def:weil-constructible-derived-typed-context`,`def:weil-point-realization-typed-context`,`def:weil-derived-tate-twist-typed-context`,`def:weil-four-operations-typed-context`,`def:weil-six-functor-adjunction-map-candidates`,`def:weil-point-constant-realization-candidate`,`def:weil-derived-global-section-typed-context`,`def:weil-six-functor-comparison-map-candidates`

Lean: `Weil.Etale.SixFunctorTypedContext`

A complete typed six-functor input context is the ordered tuple of the eight input structures named in the displayed dependency list. The cohomology-object abbreviations and Cartesian-square index are derived and indexing Definitions, not additional tuple fields. Require that all scheme, morphism, category, functor, point-realization, twist, and coefficient indices agree definitionally. It is only an input type. In particular, this definition neither constructs an inhabitant nor promotes any six-operation theorem to the foundation contract.

4.9 Insertion-stable Module 02 merge certificate

Status. The phrase "literal concatenation" applies to the Module 02 source merge unit: the prefix bytes are followed by the tail bytes. It does not mean that the evolved canonical blueprint still contains that entire byte string as one contiguous substring. The canonical file normalizes CRLF to LF and contains the later SCHFROB–12 block between two complete tail declarations. The certificate below proves the honest invariant: every original declaration unit occurs once, with normalized text unchanged and with the original order unchanged.

Definition 4.9.1 (Pinned Module 02 source merge datum)**.** Uses: `def:weil-big-etale-precoverage`,`def:weil-six-functor-typed-context`

Lean: `document-certificate:Weil.Etale.Module02SourceMergeDatum`

The pinned source merge datum consists of the three files

$$P = \texttt{02_etale_frobenius_interface.tex},$$
$$T = \texttt{02_etale_frobenius_interface_tail.tex},$$
$$M = \texttt{02_etale_frobenius_interface_merge.tex},$$

with SHA–256 values

$$h(P) = \texttt{c84a3d310029c6043521c754cca1446f42def2fa91560cf30da876c5e8648b51},$$
$$h(T) = \texttt{a4719b076e672ca91a380a9026b84cc0e563ba835b857fc6716d4f70888a95f4},$$
$$h(M) = \texttt{ba40ab76ccf6c97fabffb6eb4b9e1d75cc5daf4d4c0a42cfb8d9ad2265c42f50}.$$

The literal byte concatenation is $P \,\|\, T$; its length is 56324 bytes and its SHA–256 value is

$$\texttt{da5aa5c59a1983dafe84c552179334a6694b6ae2aa28da1b1611118a5718dff2}.$$

Let L_P and L_T be the label lists obtained by scanning complete declaration environments in file order, and put $L_{\mathrm{src}} := L_P \mathbin{+\!\!+} L_T$. A declaration unit is its statement environment followed, for every non-Definition, by its immediate proof environment.

Lemma 4.9.2 (The merge driver has literal prefix–tail order)**.** *Uses:* *`def:weil-efi-integrate01-source-merge-datum`*

Lean: *`document-certificate:Weil.Etale.module02MergeDriver_order`*

The two input commands in M, counted with multiplicity and in file order, are exactly the command importing P followed by the command importing T. Thus expansion of the source merge unit has literal prefix–tail order.

Proof. Uses: `def:weil-efi-integrate01-source-merge-datum`

The hash of M fixes its contents. Parsing its two `input` commands gives the ordered list

(`modules/02_etale_frobenius_interface.tex`, `modules/02_etale_frobenius_interface_tail.tex`).

There is no third input command and neither displayed command is repeated. □

Lemma 4.9.3 (Source declaration and label census)**.** *Uses:* *`def:weil-efi-integrate01-source-merge-datum`*

Lean: *`document-certificate:Weil.Etale.module02SourceLabelCensus`*

The prefix has 23 *declarations and the tail has* 64*. Every one of the* 87 *declaration environments carries exactly one label, and these labels are pairwise distinct. The aggregate consists of* 66 *Definitions and* 21 *proof-bearing declarations:* 18 *Lemmas and* 3 *Propositions.*

Proof. Uses: `def:weil-efi-integrate01-source-merge-datum`

Scan the two hash-pinned files for the five admitted declaration environment kinds and require one label in every match. The prefix kind counts are $(18, 4, 1)$ and the tail kind counts are $(48, 14, 2)$, in the order (Definition, Lemma, Proposition). Their componentwise sum is $(66, 18, 3)$. The finite duplicate table for the resulting 87 labels is empty, and every non-Definition is followed immediately by one proof environment. □

Definition 4.9.4 (Pinned pre-integration canonical embedding audit datum)**.** Uses: `def:weil-efi-integrate01-source-merge-datum`

Constructed by: `lemma:weil-efi-integrate01-source-label-census`

Lean: `document-certificate:Weil.Etale.Module02CanonicalEmbeddingDatum`

Pin the pre-integration canonical TeX and obligation-ledger baselines by

$$h(\mathcal{C}_{\mathrm{tex}}) = \texttt{7ac5ed5d9688f8481fa519e3960c2200392c7d17017724bf365b36de0cf58927},$$
$$h(\mathcal{C}_{\mathrm{obl}}) = \texttt{c1f467702d3e753da6f6efc779ca6d369f427d85c4d13169198ce28dc9b55196}.$$

Normalize CRLF and CR line endings to LF in $P, T, \mathcal{C}_{\mathrm{tex}}$. For every entry of L_{src}, record its unique canonical label index, the character offset of its complete declaration unit, and the SHA–256 value of that normalized unit. The full 87-row table is the `source_units` field of `tmp/efi_integrate_01/merge_audit.json`.

Lemma 4.9.5 (Normalized declaration units occur uniquely). *Uses:* *def:weil-efi-integrate01-canonical-embedding-datum*

Lean: *document-certificate:Weil.Etale.module02NormalizedUnit_unique*

For every one of the 87 source labels, the label, normalized statement environment, and normalized complete declaration unit each occur exactly once in the hash-pinned pre-integration canonical TeX.

Proof. Uses: `def:weil-efi-integrate01-canonical-embedding-datum`

For each row of the finite source list, search the pinned canonical text for the label token, statement text, and complete unit text. All three occurrence-count columns are identically 1. The row hashes prevent a same-label but textually altered declaration from satisfying the unit check. □

Lemma 4.9.6 (The eight external dependencies precede the module). *Uses:* *def:weil-prime-power-data*, *def:weil-finite-field-data*, *def:weil-arithmetic-field-frobenius*, *def:weil-geometric-field-frobenius*, *def:weil-coefficient-data*, *def:weil-tate-line*, *def:weil-frobenius-module*, *def:weil-efi-integrate01-canonical-embedding-datum*

Lean: *document-certificate:Weil.Etale.module02ExternalDependencies_precede*

The set of labels occurring in a source `uses` *field but not in* L_{src} *is exactly*

$$\left\{\begin{array}{c} \textit{def:weil-prime-power-data}, \\ \textit{def:weil-finite-field-data}, \\ \textit{def:weil-arithmetic-field-frobenius}, \\ \textit{def:weil-geometric-field-frobenius}, \\ \textit{lemma:weil-frobenius-inverse-identities}, \\ \textit{def:weil-coefficient-data}, \\ \textit{def:weil-tate-line}, \\ \textit{def:weil-frobenius-module} \end{array}\right\}.$$

Each occurs exactly once in the pre-integration canonical label sequence and strictly before the first source label.

Proof. Uses: `def:weil-efi-integrate01-canonical-embedding-datum`

The eight canonical label indices, in the displayed order, are $(0, 1, 2, 5, 6, 7, 9, 8)$; the first source label has index 740. Hence every external dependency is unique and earlier. Direct set subtraction on all source `uses` fields produces no ninth external label. □

Lemma 4.9.7 (Original declaration order is unchanged). *Uses:* *def:weil-efi-integrate01-canonical-embedding-datum*

Lean: *document-certificate:Weil.Etale.module02OriginalOrder_preserved*

Both the 87 canonical label indices and the 87 canonical declaration- unit offsets are strictly increasing in the order L_{src}. *Consequently no original declaration has been reordered.*

Proof. Uses: `lemma:weil-efi-integrate01-normalized-unit-uniqueness`

Compare each adjacent pair in the two finite integer columns of the embedding table. All 86 label-index inequalities and all 86 unit-offset inequalities are strict. Uniqueness of units then turns these inequalities into the asserted order-preserving embedding. □

Lemma 4.9.8 (Complete accounting of later insertions inside the source span). *Uses:* *def:weil-efi-integrate01-canonical-embedding-datum*

Lean: *document-certificate:Weil.Etale.module02PostMergeInsertionAccounting*

There is exactly one gap between adjacent entries of L_{src} *that contains later canonical labels. Its adjacent source anchors are*

lemma:weil-scheme-frobenius-iterate-succ,

def:weil-local-global-frobenius-iterate-predicate.

The gap contains exactly the following SCHFROB–12 labels:

1. *sec:weil-schfrob12-ordinary-powers;*
2. *def:weil-scheme-endomorphism-ordinary-power;*
3. *lemma:weil-scheme-endomorphism-ordinary-power-zero;*
4. *lemma:weil-scheme-endomorphism-ordinary-power-succ;*
5. *lemma:weil-scheme-endomorphism-ordinary-power-add;*
6. *lemma:weil-scheme-endomorphism-ordinary-power-unique;*
7. *prop:weil-scheme-frobenius-iterate-ordinary-power;*
8. *thm:weil-efi-schfrob12-discharge.*

Proof. Uses: `def:weil-efi-integrate01-canonical-embedding-datum`
For each adjacent source-label pair, subtract the source-label set from the intervening canonical-label slice. Eight labels remain in the displayed gap and every other slice is empty. This proves both completeness of the accounting and the failure of a contiguous-substring interpretation; it does not alter the original declaration order. □

Lemma 4.9.9 (The certificate extension is disjoint from the source sequence)**.** *Uses:* `def:weil-efi-integrate01-canonical-embedding-datum`

Lean: `document-certificate:Weil.Etale.module02CertificateExtension_disjoint`

The integration generator inserts this certificate immediately after `def:weil-six-functor-typed-context`*, which is the 87-th and final source label. No label in the certificate belongs to* L_{src}*. Therefore the extension changes neither the unique source-unit occurrences nor their order or internal insertion gaps.*

Proof. Uses: `lemma:weil-efi-integrate01-source-label-census,def:weil-efi-integrate01-canonical-embedding-datum`
The final row of the hash-pinned source table is the displayed anchor. The generator requires that anchor to occur exactly once inside a Definition and inserts only after its matching end token. Direct intersection of the 87 source labels with the fresh certificate labels is empty. Removing the inserted certificate and its two boundary line breaks recovers the pinned canonical TeX byte for byte, so all three asserted invariants persist. □

Definition 4.9.10 (Insertion-stable Module 02 merge certificate)**.** Uses: `def:weil-efi-integrate01-source-merge-datum,def:weil-efi-integrate01-canonical-embedding-datum`

Constructed by: `lemma:weil-efi-integrate01-merge-driver-order,lemma:weil-efi-integrate01-source-label-census,lemma:weil-efi-integrate01-normalized-unit-uniqueness,lemma:weil-efi-integrate01-external-dependencies-precede,lemma:weil-efi-integrate01-original-order-preserved,lemma:weil-efi-integrate01-post-merge-insertion-accounting,lemma:weil-efi-integrate01-candidate-extension-disjoint`

Lean: `document-certificate:Weil.Etale.Module02InsertionStableMergeCertificate`

The certificate packages: literal prefix–tail expansion of the source merge driver; the 87-label source census; normalized unique occurrence of every complete source declaration unit; precedence of exactly eight external dependencies; strict preservation of the two source order columns; and the complete account of the single later insertion gap. It also records that the certificate extension is disjoint from the source sequence.

Theorem 4.9.11 (EFI INTEGRATE–01 discharge)**.** *Uses:* `def:weil-efi-integrate01-merge-certificate`

Lean: `document-certificate:Weil.Etale.efiIntegrate01`

The Module 02 prefix and tail are faithfully integrated in the evolved canonical blueprint: their source merge unit has literal prefix–tail order, and their 87 *original labelled declaration units occur, after line-ending normalization, exactly once and in the same order. Later complete declaration insertions are permitted and do not count as reordering the source units.*

Proof. Uses: `lemma:weil-efi-integrate01-merge-driver-order,lemma:weil-efi-integrate01-source-label-census,lemma:weil-efi-integrate01-normalized-unit-uniqueness,lemma:weil-efi-integrate01-external-dependencies-precede,lemma:weil-efi-integrate01-original-order-preserved,lemma:weil-efi-integrate01-post-merge-insertion-accounting,lemma:weil-efi-integrate01-candidate-extension-disjoint`
These seven atomic leaves are precisely the fields of the insertion-stable merge certificate. Packaging them gives the stated faithful integration theorem. The theorem intentionally makes no false claim that the current canonical file contains $P \parallel T$ as a contiguous byte substring. □

> **Status.** Constructing an inhabitant of Definition 4.8.14 from the actual etale categories is blocked by OBL-ETALE-CONSTRUCTIBLE, OBL-COMPACT-SUPPORT, OBL-SIX-FUNCTOR-ADIC, and OBL-EXCEPTIONAL-PULLBACK. The present section repairs type formation only; none of those obligations is discharged.

Chapter 5

From traces to determinants and conditional rationality

Status. This chapter has two logically different parts. The matrix, formal-series, scalar-extension, and normalized-fraction statements are proved below. The final section is conditional on an explicitly supplied finite cohomological trace datum. The existence of such a datum for a separated finite-type scheme is not asserted: it remains blocked by OBL-FINITENESS, OBL-FROBENIUS, and OBL-TRACE-FORMULA.

Throughout the algebraic sections, K is a characteristic-zero field. All matrices are indexed by a displayed finite type, and every basis used in a coordinate calculation is a field of an earlier Definition.

5.1 Formal-series group laws used by the determinant calculation

Lemma 5.1.1 (Constant coefficient of the geometric inverse). *Uses:* `def:weil-geometric-inverse`

Lean: `Weil.geometricInverse_constantCoeff`

If $c(A) = 1$, then

$$c(\mathrm{GeomInv}(A)) = 1.$$

Proof. *Uses:* `def:weil-geometric-inverse`

Write $U = A - 1$. By definition, $\mathrm{GeomInv}(A) = \sum_{r \geq 0} (-U)^r$, interpreted by substitution at the positive-order series $-U$. The term $r = 0$ has constant coefficient 1. Every term with $r > 0$ has constant coefficient $c(-U)^r = 0$. Hence the total constant coefficient is 1. □

Lemma 5.1.2 (Uniqueness of the inverse of a one-series). *Uses:* `def:weil-geometric-inverse`

Lean: `Weil.geometricInverse_unique`

Suppose $c(A) = 1$. If $AB = 1$, then

$$B = \mathrm{GeomInv}(A).$$

Proof. *Uses:* `def:weil-geometric-inverse,lemma:weil-geometric-inverse-mul`

Put $G = \mathrm{GeomInv}(A)$. The previously proved inverse identity gives $AG = 1$. Since $K[[T]]$ is commutative,

$$B = B(AG) = (BA)G = 1 \cdot G = G.$$

This is the asserted equality. □

Lemma 5.1.3 (The geometric inverse is involutive). *Uses:* `def:weil-geometric-inverse`

Lean: `Weil.geometricInverse_involutive`

If $c(A) = 1$, then

$$\mathrm{GeomInv}(\mathrm{GeomInv}(A)) = A.$$

Proof. Uses: `lemma:weil-geometric-inverse-constant,lemma:weil-geometric-inverse-mul,lemma:weil-geometric-inverse-unique`

The inner geometric inverse has constant coefficient 1. Moreover $\mathrm{GeomInv}(A)A = 1$. Apply the inverse-uniqueness lemma with $\mathrm{GeomInv}(A)$ in place of A and A in place of B. □

Lemma 5.1.4 (Formal exponential at zero). *Uses:* `def:weil-formal-exp`

Lean: `Weil.formalExp_zero`

One has

$$\mathrm{Exp}(0) = 1.$$

Proof. Uses: `def:weil-formal-exp,lemma:weil-formal-exp-coeff`

In degree N, the finite coefficient formula is $\sum_{0\le r\le N}[T^N]0^r/r!$. For $N = 0$, the sole nonzero term is $r = 0$, equal to 1. For $N > 0$, every term is zero. These are the coefficients of the constant series 1. □

Lemma 5.1.5 (Formal exponential of a negative). *Uses:* `def:weil-formal-exp, def:weil-geometric-inverse`

Lean: `Weil.formalExp_neg`

For a positive-order series U,

$$\mathrm{Exp}(-U) = \mathrm{GeomInv}(\mathrm{Exp}(U)).$$

Proof. Uses: `lemma:weil-formal-exp-add,lemma:weil-formal-exp-zero,lemma:weil-formal-exp-constant,lemma:weil-geometric-inverse-unique`

The exponential addition law gives

$$\mathrm{Exp}(U)\,\mathrm{Exp}(-U) = \mathrm{Exp}(U-U) = \mathrm{Exp}(0) = 1.$$

The first factor has constant coefficient 1. Inverse uniqueness now identifies the second factor with its geometric inverse. □

Lemma 5.1.6 (Formal exponential of a finite sum). *Uses:* `def:weil-formal-exp`

Lean: `Weil.formalExp_finsetSum`

Let I be a finite type and let U_i be positive-order series. Then

$$\mathrm{Exp}\left(\sum_{i\in I} U_i\right) = \prod_{i\in I}\mathrm{Exp}(U_i).$$

Proof. Uses: `lemma:weil-formal-exp-zero,lemma:weil-formal-exp-add`

Induct on the finite index set. For the empty set, both sides are 1 by the zero formula. Inserting one index replaces the sum by $U + V$ and the product by $\mathrm{Exp}(U)\,\mathrm{Exp}(V)$; the required step is exactly the exponential addition law. □

Lemma 5.1.7 (Geometric inverse of a finite product). *Uses:* `def:weil-geometric-inverse`

Lean: `Weil.geometricInverse_finsetProduct`

Let I be finite and suppose $c(A_i) = 1$ for every i. Then

$$\mathrm{GeomInv}\left(\prod_{i\in I} A_i\right) = \prod_{i\in I}\mathrm{GeomInv}(A_i).$$

Proof. Uses: `lemma:weil-geometric-inverse-mul,lemma:weil-geometric-inverse-unique`

The finite product $\prod_i A_i$ has constant coefficient 1. Commutativity and the individual inverse identities give

$$\left(\prod_i A_i\right)\left(\prod_i \mathrm{GeomInv}(A_i)\right) = \prod_i \big(A_i\,\mathrm{GeomInv}(A_i)\big) = 1.$$

Apply inverse uniqueness to this displayed right inverse. □

5.2 Polynomial and matrix coordinates

Definition 5.2.1 (Polynomial-to-power-series inclusion)**.** Uses:

Lean: `Polynomial.toPowerSeries`

For $P = \sum_{n=0}^{d} p_n T^n \in K[T]$, define

$$\jmath(P) \in K[[T]], \qquad [T^n]\jmath(P) := p_n.$$

This is the map `Polynomial.toPowerSeries`.

Lemma 5.2.2 (Coefficient formula for the polynomial inclusion)**.** *Uses:* *def:weil-polynomial-power-series-inclusion*

Lean: *Polynomial.coeff_coe*

For every n,

$$[T^n]\jmath(P) = P.\mathrm{coeff}(n).$$

Proof. Uses: `def:weil-polynomial-power-series-inclusion`

Instantiate `Polynomial.coeff_coe` with coefficient ring K, polynomial P, and index n. Its conclusion is the displayed coefficient equality. □

Lemma 5.2.3 (The polynomial inclusion preserves products)**.** *Uses:* *def:weil-polynomial-power-series-inclusion*

Lean: *Polynomial.coe_mul*

For $P, Q \in K[T]$,

$$\jmath(PQ) = \jmath(P)\jmath(Q).$$

Proof. Uses: `def:weil-polynomial-power-series-inclusion`

Instantiate `Polynomial.coe_mul` with $R = K$, $\varphi = P$, and $\psi = Q$. The coercions in that theorem are precisely $\jmath$. □

Lemma 5.2.4 (The polynomial inclusion is injective)**.** *Uses:* *def:weil-polynomial-power-series-inclusion*

Lean: *Polynomial.coe_injective*

If $\jmath(P) = \jmath(Q)$, then $P = Q$.

Proof. Uses: `def:weil-polynomial-power-series-inclusion`

Apply `Polynomial.coe_injective` with coefficient semiring K to the displayed equality. □

Lemma 5.2.5 (The polynomial inclusion commutes with differentiation)**.** *Uses:* *def:weil-polynomial-power-series-inclusion, def:weil-formal-derivative*

Lean: *PowerSeries.derivative_coe*

For $P \in K[T]$,

$$D\jmath(P) = \jmath(P').$$

Proof. Uses: `def:weil-polynomial-power-series-inclusion,def:weil-formal-derivative`

Instantiate `PowerSeries.derivative_coe` with $R = K$ and $f = P$. Its two derivatives are the formal derivative D and the polynomial derivative P', respectively. □

Definition 5.2.6 (Based finite-dimensional endomorphism datum)**.** Uses:

Lean: `Weil.LinearAlgebra.BasedEndomorphismData`

A based endomorphism datum over K consists of:

(i) a natural number r;

(ii) a K-vector space V;

(iii) a displayed basis $b : \mathrm{Basis}(\mathrm{Fin}(r), K, V)$;

(iv) a K-linear endomorphism $u : V \to V$.

The basis is part of the input data; no basis is introduced inside a proof.

Definition 5.2.7 (Coordinate matrix of a based endomorphism)**.** Uses: `def:weil-based-linear-endomorphism-datum`

Lean: `Weil.LinearAlgebra.BasedEndomorphismData.matrix`

For a based datum $D = (r, V, b, u)$, define

$$M_D := \mathrm{toMatrix}(b, b, u) \in \mathrm{Mat}_r(K), \qquad (M_D)_{ij} := (b.\,\mathrm{repr}(u(b_j)))_i.$$

Definition 5.2.8 (Based automorphism datum)**.** Uses: `def:weil-based-linear-endomorphism-datum`

Lean: `Weil.LinearAlgebra.BasedAutomorphismData`

A based automorphism datum is a based endomorphism datum together with a displayed linear map $v : V \to V$ and the two equations

$$v \circ u = \mathrm{id}_V, \qquad u \circ v = \mathrm{id}_V .$$

Definition 5.2.9 (Explicit matrix trace)**.** Uses:

Lean: `Matrix.trace`

For a finite square matrix $A = (a_{ij})$ over a commutative ring, define

$$\mathrm{tr}_\Sigma(A) := \sum_i a_{ii}.$$

Lemma 5.2.10 (Cyclicity of the explicit matrix trace)**.** *Uses:* `def:weil-explicit-matrix-trace`

Lean: `Matrix.trace_mul_comm`

For a finite $m \times n$ matrix A and a finite $n \times m$ matrix B over a commutative ring,

$$\mathrm{tr}_\Sigma(AB) = \mathrm{tr}_\Sigma(BA).$$

Proof. Uses: `def:weil-explicit-matrix-trace`

Apply `Matrix.trace_mul_comm` with index types m, n, coefficient ring the displayed commutative ring, and matrices A, B. Expanding both sides gives the same finite double sum $\sum_{i,j} A_{ij} B_{ji}$, with the order of summation interchanged. □

Definition 5.2.11 (Reciprocal polynomial matrix)**.** Uses: `def:weil-coordinate-endomorphism-matrix`

Lean: `Weil.LinearAlgebra.BasedEndomorphismData.reciprocalMatrix`

For $D = (r, V, b, u)$, let $C(M_D)$ denote entrywise inclusion of M_D as a constant matrix over $K[T]$. Define

$$A_D(T) := I_r - T\, C(M_D) \in \mathrm{Mat}_r(K[T]).$$

Definition 5.2.12 (Based reciprocal determinant polynomial)**.** Uses: `def:weil-reciprocal-polynomial-matrix`

Lean: `Weil.LinearAlgebra.BasedEndomorphismData.reciprocalDet`

Define

$$P_D(T) := \det_\Sigma(A_D(T)) = \sum_{\sigma \in S_r} \mathrm{sgn}(\sigma) \prod_{i \in \mathrm{Fin}(r)} A_D(T)_{\sigma(i),i}.$$

Lemma 5.2.13 (Constant coefficient of the reciprocal determinant)**.** *Uses:* `def:weil-based-reciprocal-determinant`, `def:weil-polynomial-power-series-inclusion`

Lean: `Weil.LinearAlgebra.BasedEndomorphismData.reciprocalDet_coeff_zero`

One has

$$P_D(0) = 1, \qquad c(\jmath(P_D)) = 1.$$

Proof. Uses: `def:weil-reciprocal-polynomial-matrix`, `def:weil-based-reciprocal-determinant`, `def:weil-polynomial-power-series-inclusion`, `lemma:weil-polynomial-series-coeff`

Evaluation at $T = 0$ turns $A_D(T)$ into I_r. In the displayed permutation formula for its determinant, the identity permutation contributes 1; every other permutation has an off-diagonal identity entry and contributes 0. Thus $P_D(0) = 1$. The coefficient formula for $\jmath$ identifies its constant coefficient with $P_D(0)$. □

Definition 5.2.14 (Explicit reciprocal adjugate)**.** Uses: `def:weil-reciprocal-polynomial-matrix`, `def:weil-based-linear-endomorphism-datum`

Lean: `Weil.LinearAlgebra.BasedEndomorphismData.reciprocalAdjugate`

Define

$$\mathrm{Adj}_D(T) := \mathrm{adjugate}(A_D(T)).$$

Its (i,j)-entry is the determinant of the matrix obtained from $A_D(T)$ by replacing row j by the coordinate row having 1 in column i and 0 elsewhere. All indices refer to the displayed basis in Definition 5.2.6.

Lemma 5.2.15 (Entry formula for the explicit reciprocal adjugate)**.** Uses: *def:weil-explicit-reciprocal-adjugate*

Lean: *Matrix.adjugate_apply*

For $i, j \in \mathrm{Fin}(r)$,

$$\mathrm{Adj}_D(T)_{ij} = \det_{\Sigma}\big(A_D(T).\,\mathrm{updateRow}\big(j, \mathrm{single}(i,1)\big)\big)\,.$$

Proof. Uses: `def:weil-explicit-reciprocal-adjugate`

Instantiate `Matrix.adjugate_apply` with coefficient ring $K[T]$, index type $\mathrm{Fin}(r)$, matrix $A_D(T)$, and indices i, j. Its right side is exactly the displayed updated-row determinant. □

Lemma 5.2.16 (Left adjugate identity)**.** Uses: *def:weil-explicit-reciprocal-adjugate*, *def:weil-reciprocal-polynomial-matrix*, *def:weil-based-reciprocal-determinant*

Lean: *Matrix.mul_adjugate*

One has

$$A_D(T)\,\mathrm{Adj}_D(T) = P_D(T) I_r.$$

Proof. Uses: `def:weil-explicit-reciprocal-adjugate`, `def:weil-based-reciprocal-determinant`

Apply `Matrix.mul_adjugate` to the matrix $A_D(T)$. Its right side is $A_D(T).\det$ times the identity matrix, which is $P_D(T) I_r$ by definition. □

Lemma 5.2.17 (Right adjugate identity)**.** Uses: *def:weil-explicit-reciprocal-adjugate*, *def:weil-reciprocal-polynomial-matrix*, *def:weil-based-reciprocal-determinant*

Lean: *Matrix.adjugate_mul*

One has

$$\mathrm{Adj}_D(T) A_D(T) = P_D(T) I_r.$$

Proof. Uses: `def:weil-explicit-reciprocal-adjugate`, `def:weil-based-reciprocal-determinant`

Apply `Matrix.adjugate_mul` to $A_D(T)$ and unfold $P_D(T) = A_D(T).\det$. □

Lemma 5.2.18 (Derivative of the explicit reciprocal determinant)**.** Uses: *def:weil-coordinate-endomorphism-matrix*, *def:weil-based-reciprocal-determinant*, *def:weil-explicit-reciprocal-adjugate*, *def:weil-explicit-matrix-trace*

Lean: *Weil.LinearAlgebra.BasedEndomorphismData.reciprocalDet_derivative*

In $K[T]$,

$$P_D'(T) = -\,\mathrm{tr}_{\Sigma}(\mathrm{Adj}_D(T)\, C(M_D))\,.$$

Proof. Uses: `def:weil-reciprocal-polynomial-matrix`, `def:weil-based-reciprocal-determinant`, `def:weil-explicit-reciprocal-adjugate`, `def:weil-explicit-matrix-trace`, `lemma:weil-explicit-reciprocal-adjugate-entry`

Expand P_D by the finite permutation formula `Matrix.det_apply`. Differentiate each finite product. A summand is specified by a permutation σ and a unique differentiated column j; the derivative of $\delta_{\sigma(j),j} - T(M_D)_{\sigma(j),j}$ is $-(M_D)_{\sigma(j),j}$. For fixed row and column, summing the remaining products with their signs is the updated-row determinant in Lemma 5.2.15. Regrouping the finite sum by that row and column gives $-\sum_i (\mathrm{Adj}_D\, C(M_D))_{ii}$, which is the displayed explicit trace. □

Definition 5.2.19 (Entrywise polynomial-series inclusion for matrices)**.** Uses: `def:weil-polynomial-power-series-inclusion`

Lean: `Weil.LinearAlgebra.matrixPolynomialToPowerSeries`

For a finite matrix B over $K[T]$, define

$$\widehat{B}_{ij} := \jmath(B_{ij}).$$

Lemma 5.2.20 (Matrix polynomial-series inclusion preserves products). *Uses:* `def:weil-matrix-polynomial-series-inclusion`

Lean: `Weil.LinearAlgebra.matrixPolynomialToPowerSeries_mul`

For finite compatible matrices B, C over $K[T]$,

$$\widehat{BC} = \widehat{B}\,\widehat{C}.$$

Proof. Uses: `def:weil-matrix-polynomial-series-inclusion,lemma:weil-polynomial-series-mul`

The (i,j)-entry on the left is $\jmath(\sum_k B_{ik}C_{kj})$. The polynomial-series inclusion is additive coefficientwise and preserves each product by Lemma 5.2.3; hence this entry is $\sum_k \jmath(B_{ik})\jmath(C_{kj})$, the (i,j)-entry on the right. □

Definition 5.2.21 (Formal matrix resolvent). Uses: `def:weil-coordinate-endomorphism-matrix`

Lean: `Weil.LinearAlgebra.BasedEndomorphismData.resolvent`

Define $R_D(T) \in \mathrm{Mat}_r(K[[T]])$ coefficientwise by

$$[T^n]R_D(T) := M_D^n \qquad (n \geq 0).$$

Equivalently, $R_D(T) = \sum_{n\geq 0} T^n M_D^n$.

Lemma 5.2.22 (Left resolvent inverse). *Uses:* `def:weil-formal-matrix-resolvent,def:weil-reciprocal-polynomial-matrix,def:weil-matrix-polynomial-series-inclusion`

Lean: `Weil.LinearAlgebra.BasedEndomorphismData.reciprocalMatrix_mul_resolvent`

One has

$$\widehat{A_D(T)}\,R_D(T) = I_r.$$

Proof. Uses: `def:weil-formal-matrix-resolvent,def:weil-reciprocal-polynomial-matrix,def:weil-matrix-polynomial-series-inclusion`

The constant coefficient of the product is $I_r M_D^0 = I_r$. For $n \geq 1$, its coefficient is $M_D^n - M_D M_D^{n-1} = 0$. Equality of all matrix entries in every coefficient gives the displayed identity. □

Lemma 5.2.23 (Right resolvent inverse). *Uses:* `def:weil-formal-matrix-resolvent,def:weil-reciprocal-polynomial-matrix,def:weil-matrix-polynomial-series-inclusion`

Lean: `Weil.LinearAlgebra.BasedEndomorphismData.resolvent_mul_reciprocalMatrix`

One has

$$R_D(T)\,\widehat{A_D(T)} = I_r.$$

Proof. Uses: `def:weil-formal-matrix-resolvent,def:weil-reciprocal-polynomial-matrix,def:weil-matrix-polynomial-series-inclusion`

The constant coefficient is $M_D^0 I_r = I_r$. In degree $n \geq 1$, the coefficient is $M_D^n - M_D^{n-1}M_D = 0$. Thus every coefficient matrix equals the corresponding coefficient of I_r. □

Lemma 5.2.24 (Adjugate expressed through the formal resolvent). *Uses:* `def:weil-explicit-reciprocal-adjugate,def:weil-based-reciprocal-determinant,def:weil-formal-matrix-resolvent,def:weil-matrix-polynomial-series-inclusion,def:weil-polynomial-power-series-inclusion`

Lean: `Weil.LinearAlgebra.BasedEndomorphismData.adjugate_eq_det_mul_resolvent`

In $\mathrm{Mat}_r(K[[T]])$,

$$\widehat{\mathrm{Adj}_D(T)} = \jmath(P_D(T))\,R_D(T).$$

Proof. Uses: `lemma:weil-explicit-reciprocal-adjugate-left,lemma:weil-matrix-polynomial-series-inclusion-mul,lemma:weil-formal-matrix-resolvent-right`

Apply entrywise polynomial-series inclusion to the left adjugate identity. It gives $\widehat{A}_D\,\widehat{\mathrm{Adj}}_D = \jmath(P_D)I_r$. Multiply on the left by R_D. The right resolvent inverse reduces $R_D\widehat{A}_D$ to I_r, leaving $\widehat{\mathrm{Adj}}_D = \jmath(P_D)R_D$. □

Lemma 5.2.25 (Trace of the resolvent times the matrix). *Uses:* `def:weil-formal-matrix-resolvent,def:weil-coordinate-endomorphism-matrix,def:weil-explicit-matrix-trace`

Lean: `Weil.LinearAlgebra.BasedEndomorphismData.trace_resolvent_mul`

One has

$$\mathrm{tr}_\Sigma(R_D(T)M_D) = \sum_{n\geq 1} \mathrm{tr}_\Sigma(M_D^n)T^{n-1}.$$

Proof. Uses: def:weil-formal-matrix-resolvent,def:weil-explicit-matrix-trace
The coefficient of T^m on the left is the trace of $M_D^m M_D = M_D^{m+1}$, because matrix multiplication and the finite diagonal sum commute with coefficient extraction. Put $n = m+1$ to obtain the coefficient on the right. □

Definition 5.2.26 (Trace logarithm of a based endomorphism)**.** Uses: def:weil-coordinate-endomorphism-matrix,def:weil-explicit-matrix-trace

Lean: Weil.LinearAlgebra.BasedEndomorphismData.traceLogSeries
Define $S_D(T) \in TK[[T]]$ by

$$[T^0]S_D = 0, \qquad [T^n]S_D = \frac{\mathrm{tr}_\Sigma(M_D^n)}{n} \quad (n \geq 1).$$

Lemma 5.2.27 (Constant coefficient of the trace logarithm)**.** *Uses:* def:weil-based-trace-log-series

Lean: Weil.LinearAlgebra.BasedEndomorphismData.traceLogSeries_constantCoeff
One has

$$c(S_D) = 0.$$

Proof. Uses: def:weil-based-trace-log-series
This is the degree-zero field in the coefficientwise Definition of S_D. □

Lemma 5.2.28 (Derivative of the trace logarithm)**.** *Uses:* def:weil-based-trace-log-series,def:weil-formal-derivative

Lean: Weil.LinearAlgebra.BasedEndomorphismData.traceLogSeries_derivative
One has

$$DS_D(T) = \sum_{n \geq 1} \mathrm{tr}_\Sigma(M_D^n) T^{n-1}.$$

Proof. Uses: def:weil-based-trace-log-series,def:weil-formal-derivative
For $m \geq 0$, the coefficient of T^m in DS_D is

$$(m+1)[T^{m+1}]S_D = (m+1)\frac{\mathrm{tr}_\Sigma(M_D^{m+1})}{m+1}.$$

The nonzero integer $m+1$ is invertible in the characteristic-zero field K, so this coefficient is $\mathrm{tr}_\Sigma(M_D^{m+1})$, as required. □

Lemma 5.2.29 (Derivative factorization through the formal resolvent)**.** *Uses:* def:weil-based-reciprocal-determinant,def:weil-formal-matrix-resolvent,def:weil-coordinate-endomorphism-matrix,def:weil-explicit-matrix-trace,def:weil-polynomial-series-inclusion,def:weil-formal-derivative

Lean: Weil.LinearAlgebra.BasedEndomorphismData.reciprocalDet_derivative_resolvent
In $K[[T]]$,

$$D\jmath(P_D) = -\jmath(P_D)\ \mathrm{tr}_\Sigma(R_D(T)M_D).$$

Proof. Uses: def:weil-explicit-matrix-trace,lemma:weil-polynomial-series-derivative,lemma:weil-based-reciprocal-determinant-derivative,lemma:weil-explicit-adjugate-resolvent
Embed the polynomial derivative identity into $K[[T]]$ and use the polynomial–series derivative lemma. This gives

$$D\jmath(P_D) = -\,\mathrm{tr}_\Sigma(\widehat{\mathrm{Adj}_D(T)}\, M_D).$$

Replace the embedded adjugate by $\jmath(P_D)R_D(T)$. Expanding the finite diagonal sum in the Definition of tr_Σ moves the scalar $\jmath(P_D)$ outside the trace, yielding exactly the displayed factorization. □

Lemma 5.2.30 (Logarithmic derivative of the reciprocal determinant)**.** *Uses:* def:weil-based-reciprocal-determinant,def:weil-coordinate-endomorphism-matrix,def:weil-explicit-matrix-trace,def:weil-polynomial-series-inclusion,def:weil-geometric-inverse,def:weil-formal-derivative

Lean: Weil.LinearAlgebra.BasedEndomorphismData.reciprocalDet_logDerivative
One has

$$-D\jmath(P_D)\ \mathrm{GeomInv}(\jmath(P_D)) = \sum_{n \geq 1} \mathrm{tr}_\Sigma(M_D^n) T^{n-1}.$$

Proof. Uses: `lemma:weil-based-reciprocal-determinant-constant,lemma:weil-based-reciprocal-determinant-derivative-resolvent,lemma:weil-formal-resolvent-trace-series,lemma:weil-geometric-inverse-mul`

Multiply the resolvent factorization by $-\operatorname{GeomInv}(\jmath(P_D))$. The two minus signs cancel. Commutativity and the constant-one inverse identity reduce $\jmath(P_D)\operatorname{GeomInv}(\jmath(P_D))$ to 1. The result is $\operatorname{tr}_\Sigma(R_D M_D)$, which the resolvent trace-series lemma identifies coefficientwise with the displayed right side. □

Lemma 5.2.31 (Derivative agreement for the negative logarithm). *Uses:* `def:weil-based-reciprocal-determinant, def:weil-based-trace-log-series, def:weil-formal-log, def:weil-polynomial-power-series-inclusion, def:weil-formal-derivative`

Lean: `Weil.LinearAlgebra.BasedEndomorphismData.neg_log_det_derivative`

One has

$$D\big(-\operatorname{Log}(\jmath(P_D))\big) = DS_D.$$

Proof. Uses: `def:weil-formal-derivative,lemma:weil-based-reciprocal-determinant-constant,lemma:weil-formal-log-derivative,lemma:weil-based-reciprocal-log-derivative,lemma:weil-based-trace-log-series-derivative`

The coefficient formula for the formal derivative gives $D(-A) = -DA$. Apply this to $A = \operatorname{Log}(\jmath(P_D))$. The formal-log derivative formula and the reciprocal logarithmic-derivative lemma identify the resulting series with $\sum_{n\geq 1} \operatorname{tr}_\Sigma(M_D^n)T^{n-1}$. The trace-log derivative lemma identifies the latter series with DS_D. □

Lemma 5.2.32 (Negative logarithm equals the trace-log series). *Uses:* `def:weil-based-reciprocal-determinant, def:weil-based-trace-log-series, def:weil-formal-log, def:weil-polynomial-power-series-inclusion`

Lean: `Weil.LinearAlgebra.BasedEndomorphismData.neg_log_det_eq_traceLogSeries`

For every based finite-dimensional endomorphism datum,

$$-\operatorname{Log}(\jmath(P_D)) = S_D.$$

Proof. Uses: `lemma:weil-based-reciprocal-determinant-constant,lemma:weil-based-trace-log-series-constant,lemma:weil-formal-log-coeff,lemma:weil-based-negative-log-derivative-agreement,lemma:weil-formal-derivative-ext`

The derivative equality is Lemma 5.2.31. The trace-log series has constant coefficient zero. The reciprocal determinant has constant coefficient one, so the degree-zero formal-log formula gives $c(-\operatorname{Log}(\jmath(P_D))) = 0$. Apply formal derivative extensionality once. □

Lemma 5.2.33 (Exponential of the negative logarithm). *Uses:* `def:weil-formal-exp, def:weil-formal-log, def:weil-geometric-inverse`

Lean: `Weil.formalExp_neg_formalLog`

If $c(A) = 1$, then

$$\operatorname{Exp}(-\operatorname{Log}(A)) = \operatorname{GeomInv}(A).$$

Proof. Uses: `lemma:weil-formal-log-coeff,lemma:weil-formal-exp-neg,lemma:weil-formal-exp-log`

The formal-log coefficient formula says that $\operatorname{Log}(A)$ has positive order. Hence

$$\operatorname{Exp}(-\operatorname{Log}(A)) = \operatorname{GeomInv}(\operatorname{Exp}(\operatorname{Log}(A))) = \operatorname{GeomInv}(A),$$

where the two equalities are respectively the exponential-of-a-negative lemma and the exponential-after-log lemma. □

Theorem 5.2.34 (Determinant as the exponential of traces, based form). *Uses:* `def:weil-based-reciprocal-determinant, def:weil-based-trace-log-series, def:weil-formal-exp, def:weil-geometric-inverse, def:weil-polynomial-power-series-inclusion`

Lean: `Weil.LinearAlgebra.BasedEndomorphismData.det_one_sub_eq_exp_trace`

For every based finite-dimensional endomorphism datum,

$$\operatorname{GeomInv}(\jmath(P_D)) = \operatorname{Exp}(S_D) = \operatorname{Exp}\left(\sum_{n\geq 1} \operatorname{tr}_\Sigma(M_D^n)\frac{T^n}{n}\right).$$

Proof. Uses: `def:weil-based-trace-log-series,lemma:weil-based-reciprocal-determinant-constant,lemma:weil-based-negative-log-eq-trace-series,lemma:weil-formal-exp-negative-log`

Apply Exp to the negative-log equality and use the preceding exponential-of-the-negative-log lemma. This gives the first equality, with its sides in the displayed order. The second equality is the coefficientwise Definition of S_D. □

5.3 Rebasing and independence of the displayed basis

Definition 5.3.1 (Two-basis endomorphism datum)**.** Uses:

Lean: `Weil.LinearAlgebra.TwoBasisEndomorphismData`

A two-basis datum consists of a natural number r, a K-vector space V, two displayed bases

$$b, c : \mathrm{Basis}(\mathrm{Fin}(r), K, V),$$

and one K-linear endomorphism $u : V \to V$.

Definition 5.3.2 (Transition matrices between the two displayed bases)**.** Uses: `def:weil-two-basis-endomorphism-datum`

Lean: `Weil.LinearAlgebra.TwoBasisEndomorphismData.transitionMatrices`

For a two-basis datum, define

$$S := \mathrm{toMatrix}(c, b, \mathrm{id}_V), \qquad R := \mathrm{toMatrix}(b, c, \mathrm{id}_V).$$

Thus S converts c-coordinates to b-coordinates and R converts b-coordinates to c-coordinates.

Lemma 5.3.3 (Transition matrices are mutual inverses)**.** *Uses:* *def:weil-two-basis-transition-matrices*

Lean: *Weil.LinearAlgebra.TwoBasisEndomorphismData.transitionMatrices_inverse*

One has

$$RS = I_r, \qquad SR = I_r.$$

Proof. Uses: `def:weil-two-basis-transition-matrices`

Apply both composites to a coordinate column. The map S sends the column to the b-coordinates of the vector represented in the c-basis; R sends those coordinates back to the original c-coordinates. Hence RS fixes every coordinate column. The same calculation with b and c interchanged proves $SR = I_r$. □

Lemma 5.3.4 (Coordinate matrices are conjugate under rebasing)**.** *Uses:* *def:weil-two-basis-endomorphism-datum*, *def:weil-two-basis-transition-matrices*

Lean: *Weil.LinearAlgebra.TwoBasisEndomorphismData.matrix_conjugate*

Let $M_b = \mathrm{toMatrix}(b, b, u)$ *and* $M_c = \mathrm{toMatrix}(c, c, u)$*. Then*

$$M_c = R M_b S.$$

Proof. Uses: `def:weil-two-basis-transition-matrices`

Start with a c-coordinate column. Multiplication by S converts it to b-coordinates. Multiplication by M_b applies u in those coordinates. Multiplication by R returns to c-coordinates. This composite is, by the coordinate-matrix definition, multiplication by M_c. Equality on every coordinate column gives the matrix equality. □

Theorem 5.3.5 (Reciprocal determinant is independent of the displayed basis)**.** *Uses:* *def:weil-two-basis-endomorphism-datum*, *def:weil-based-reciprocal-determinant*

Lean: *Weil.LinearAlgebra.TwoBasisEndomorphismData.reciprocalDet_eq*

For the two based endomorphism data obtained from b *and* c*,*

$$P_{(V,b,u)}(T) = P_{(V,c,u)}(T).$$

Proof. Uses: `def:weil-reciprocal-polynomial-matrix`, `lemma:weil-two-basis-transition-inverses`, `lemma:weil-two-basis-coordinate-conjugacy`

Embed R, S as constant matrices over $K[T]$. The inverse and conjugacy equations give

$$I - TM_c = R(I - TM_b)S.$$

Apply `Matrix.det_mul` twice. Since $\det(R)\det(S) = \det(RS) = \det(I) = 1$, the determinant of the right side is the determinant of $I - TM_b$. These two determinants are the two reciprocal polynomials by definition. □

5.4 Scalar extension and the root–eigenvalue dictionary

Definition 5.4.1 (Based scalar-extension datum)**.** Uses: `def:weil-based-linear-automorphism-datum`

Lean: `Weil.LinearAlgebra.BasedScalarExtensionData`

A based scalar-extension datum consists of a based automorphism datum $D = (r, V, b, u, v)$, a field L, and a displayed K-algebra structure on L. Define

$$V_L := L \otimes_K V.$$

Definition 5.4.2 (Scalar-extended endomorphism and inverse)**.** Uses: `def:weil-based-scalar-extension-datum`

Lean: `Weil.LinearAlgebra.BasedScalarExtensionData.endomorphism`

Define

$$u_L := \mathrm{id}_L \otimes u, \qquad v_L := \mathrm{id}_L \otimes v$$

using `LinearMap.baseChange`.

Definition 5.4.3 (Scalar-extended displayed basis)**.** Uses: `def:weil-based-scalar-extension-datum`

Lean: `Weil.LinearAlgebra.BasedScalarExtensionData.basis`

Define the displayed L-basis

$$b_L := b.\,\mathrm{baseChange}(L) : \mathrm{Fin}(r) \to V_L.$$

Its j-th vector is $1 \otimes b_j$.

Lemma 5.4.4 (Scalar extension on pure tensors)**.** *Uses:* `def:weil-based-scalar-extended-endomorphism`

Lean: `LinearMap.baseChange_tmul`

For $a \in L$ and $x \in V$,

$$u_L(a \otimes x) = a \otimes u(x), \qquad v_L(a \otimes x) = a \otimes v(x).$$

Proof. Uses: `def:weil-based-scalar-extended-endomorphism`

Apply `LinearMap.baseChange_tmul` first to u and then to v, with scalar a and vector x. □

Lemma 5.4.5 (The scalar-extended maps are mutual inverses)**.** *Uses:* `def:weil-based-scalar-extended-endomorphism`

Lean: `Weil.LinearAlgebra.BasedScalarExtensionData.inverse`

One has

$$v_L u_L = \mathrm{id}_{V_L}, \qquad u_L v_L = \mathrm{id}_{V_L}.$$

Proof. Uses: `def:weil-based-linear-automorphism-datum`, `lemma:weil-based-scalar-extension-pure-tensor`

On a pure tensor $a \otimes x$, the first composite gives $a \otimes v(u(x)) = a \otimes x$, and the second gives $a \otimes u(v(x)) = a \otimes x$. Apply `TensorProduct.induction_on`; its zero, pure-tensor, and additive cases identify each composite with the identity on all of V_L. □

Lemma 5.4.6 (Coordinate matrix after scalar extension)**.** *Uses:* `def:weil-coordinate-endomorphism-matrix`, `def:weil-based-scalar-extended-endomorphism`, `def:weil-based-scalar-extended-basis`

Lean: `Weil.LinearAlgebra.BasedScalarExtensionData.toMatrix`

The matrix of u_L in b_L is the entrywise image of M_D:

$$\mathrm{toMatrix}(b_L, b_L, u_L) = (\mathrm{algebraMap}_{K,L})(M_D).$$

Proof. Uses: `lemma:weil-based-scalar-extension-pure-tensor`

Fix i, j. By `Module.Basis.baseChange_apply`, $(b_L)_j = 1 \otimes b_j$. The pure-tensor formula sends this vector to $1 \otimes u(b_j)$. Apply `Module.Basis.baseChange_repr_tmul`; its i-th coordinate is the image in L of $(b.\,\mathrm{repr}(u(b_j)))_i = (M_D)_{ij}$. Equality of every entry proves the matrix identity. □

Definition 5.4.7 (Base-changed reciprocal polynomial)**.** Uses: `def:weil-based-reciprocal-determinant,def:weil-based-scalar-extension-datum`

Lean: `Weil.LinearAlgebra.BasedScalarExtensionData.reciprocalDet`

Define

$$P_{D,L}(T) := P_D(T).\,\mathrm{map}(\mathrm{algebraMap}_{K,L}) \in L[T].$$

Lemma 5.4.8 (Determinant base change for the reciprocal polynomial)**.** *Uses: def:weil-based-reciprocal-determinant-base-change, def:weil-based-scalar-extended-endomorphism, def:weil-based-basis, def:weil-based-reciprocal-determinant*

Lean: *Weil.LinearAlgebra.BasedScalarExtensionData.reciprocalDet_eq*

The polynomial $P_{D,L}$ equals the reciprocal determinant computed from (V_L, b_L, u_L):

$$P_{D,L}(T) = \det(I - T\,\mathrm{toMatrix}(b_L, b_L, u_L))\,.$$

Proof. Uses: `def:weil-reciprocal-polynomial-matrix,lemma:weil-based-scalar-extension-coordinate-matrix`

Apply `RingHom.map_det` to the matrix $I - TM_D$. Entrywise coefficient mapping sends it to $I - T\ \mathrm{map}(M_D)$. Replace the mapped matrix by the coordinate matrix of u_L using the preceding lemma. The left side is the coefficientwise map of P_D, namely $P_{D,L}$. □

Lemma 5.4.9 (Evaluation of the base-changed reciprocal polynomial)**.** *Uses: def:weil-based-reciprocal-determinant-base-change, def:weil-based-scalar-extended-endomorphism, def:weil-based-basis*

Lean: *Weil.LinearAlgebra.BasedScalarExtensionData.reciprocalDet_eval*

For $\beta \in L$,

$$P_{D,L}(\beta) = \det(I - \beta\,\mathrm{toMatrix}(b_L, b_L, u_L)) = \det_L(\mathrm{id}_{V_L} - \beta u_L).$$

Proof. Uses: `lemma:weil-based-reciprocal-determinant-base-change`

Evaluate the polynomial matrix entrywise at $T = \beta$. The compatibility `Polynomial.eval₂_eq_eval_map` identifies evaluation of a coefficientwise mapped polynomial at β with $\mathrm{eval}_2(\mathrm{algebraMap}_{K,L}, \beta)$, and `RingHom.map_det` moves evaluation through the finite determinant. For the second equality, apply `LinearMap.det_toMatrix` to the displayed basis b_L and the endomorphism $\mathrm{id}_{V_L} - \beta u_L$; its coordinate matrix is $I - \beta\,\mathrm{toMatrix}(b_L, b_L, u_L)$. □

Definition 5.4.10 (Scalar-extended eigenspace)**.** Uses: `def:weil-based-scalar-extended-endomorphism`

Lean: `Weil.LinearAlgebra.BasedScalarExtensionData.eigenspace`

For $\alpha \in L$, define

$$E_\alpha(u_L) := \ker(u_L - \alpha\,\mathrm{id}_{V_L}).$$

Say that α is an eigenvalue if $E_\alpha(u_L) \neq \bot$.

Lemma 5.4.11 (Zero is not an eigenvalue of the scalar-extended automorphism)**.** *Uses: def:weil-based-scalar-extended-eigenspace*

Lean: *Weil.LinearAlgebra.BasedScalarExtensionData.zero_not_eigenvalue*

One has

$$E_0(u_L) = \bot.$$

Proof. Uses: `def:weil-based-scalar-extended-eigenspace,lemma:weil-based-scalar-extension-inverses`

If $u_L x = 0$, apply v_L and use $v_L u_L = \mathrm{id}$ to obtain $x = 0$. Thus $\ker(u_L) = \bot$, which is exactly $E_0(u_L) = \bot$. □

Lemma 5.4.12 (Kernel rescaling for reciprocal parameters)**.** *Uses: def:weil-based-scalar-extended-endomorphism, def:weil-based-scalar-extended-eigenspace*

Lean: *Weil.LinearAlgebra.BasedScalarExtensionData.ker_one_sub_eq_eigenspace*

If $\beta \neq 0$, then

$$\ker(\mathrm{id}_{V_L} - \beta u_L) = E_{\beta^{-1}}(u_L).$$

Proof. Uses: `def:weil-based-scalar-extended-eigenspace`

The two endomorphisms satisfy

$$\mathrm{id} - \beta u_L = -\beta\big(u_L - \beta^{-1}\,\mathrm{id}\big).$$

Multiplication by the nonzero scalar $-\beta$ has zero kernel. Hence a vector is killed by the left endomorphism exactly when it is killed by $u_L - \beta^{-1}\,\mathrm{id}$, giving equality of the two kernels. □

Theorem 5.4.13 (Ordinary roots are inverses of scalar-extended eigenvalues). *Uses:* *def:weil-based-reciprocal-determinant-base-change, def:weil-based-scalar-extended-eigenspace*

Lean: *Weil.LinearAlgebra.BasedScalarExtensionData.root_iff_inverse_eigenvalue*

For every $\beta \in L$,

$$P_{D,L}(\beta) = 0 \quad \Longleftrightarrow \quad E_{\beta^{-1}}(u_L) \neq \bot.$$

Thus an ordinary root β is the inverse of the reciprocal eigenvalue $\alpha = \beta^{-1}$, and this eigenvalue lives on $V_L = L \otimes_K V$, not on V unless it already belongs to K.

Proof. Uses: `lemma:weil-based-reciprocal-determinant-evaluation,lemma:weil-based-scalar-extension-reciprocal-kernel,lemma:weil-based-scalar-extension-zero-not-eigenvalue`

By the evaluation lemma and the exact pinned declaration `LinearMap.det_eq_zero_iff_ker_ne_bot`, the left side is equivalent to

$$\ker(\mathrm{id}_{V_L} - \beta u_L) \neq \bot.$$

The displayed basis b_L supplies the finite free module instances required by that declaration, and the field L supplies its domain hypothesis. If $P_{D,L}(\beta) = 0$, then $\beta \neq 0$: the evaluation lemma at $\beta = 0$ gives $P_{D,L}(0) = \det(\mathrm{id}_{V_L}) = 1$. The reciprocal-kernel lemma now changes this kernel into $E_{\beta^{-1}}(u_L)$.

Conversely, if $E_{\beta^{-1}}(u_L) \neq \bot$, then $\beta \neq 0$. Indeed $\beta = 0$ would give $\beta^{-1} = 0$, contrary to the zero-eigenspace lemma. Apply the reciprocal-kernel equality in the reverse direction and then `LinearMap.det_eq_zero_iff_ker_ne_bot` to obtain $P_{D,L}(\beta) = 0$. □

5.5 Finite graded families and alternating determinants

Definition 5.5.1 (Bounded graded based automorphism family). Uses: `def:weil-based-linear-automorphism-datum`

Lean: `Weil.LinearAlgebra.BoundedGradedBasedAutomorphismFamily`

A bounded graded based automorphism family consists of a natural number B and, for every $i \in \mathrm{Fin}(B+1)$, a based automorphism datum D_i. The integer B is part of the data and records that no degree above B occurs.

Definition 5.5.2 (Degreewise reciprocal factor). Uses: `def:weil-bounded-graded-based-automorphism-family,def:weil-based-reciprocal-determinant`

Lean: `Weil.LinearAlgebra.BoundedGradedBasedAutomorphismFamily.factor`

For $i \leq B$, define

$$P_{G,i}(T) := P_{D_i}(T) \in K[T].$$

5.5.1 Formal compact-support Frobenius-family adapter

The following adapter is deliberately conditional. It turns separately supplied geometric certificates into the exact bounded based family used by this chapter. It does not construct a six-functor context, compact-support cohomology, a vanishing theorem, or geometric Frobenius.

Definition 5.5.3 (Supplied compact-support Frobenius-family input). Uses: `def:weil-derived-global-section-typed-context,def:weil-induced-cohomology-frobenius,def:weil-induced-cohomology-frobenius-inverse-candidate,def:weil-actual-cohomology-realization-predicate`

Lean: `Weil.Trace.TRR03SuppliedCompactSupportFrobeniusFamily`

Fix supplied typed contexts, a separated finite-type structural morphism $a_X : X \to S$, and the constant coefficient object E_X. Put

$$V_j := H^j_c(X, E_X).$$

A supplied TRR-03 family consists of:

(i) a natural number B;

(ii) for every $j < 0$ or $B < j$, a displayed E-linear equivalence $z_j : V_j \simeq_E 0$;

(iii) for every $i \in \mathrm{Fin}(B+1)$, a specified finite-dimensionality instance on V_i;

(iv) for every such i, an E-linear equivalence

$$F^{\mathrm{geom}}_{c,i} : V_i \simeq_E V_i$$

together with a supplied actual-realization certificate and two displayed equalities identifying its forward map with the induced geometric cohomological Frobenius Φ_i and its inverse map with the inverse candidate Ψ_i.

The four classes of fields are inputs, not conclusions of this Definition.

Open obligation: OBL-TRR-03-COMPACT-CARRIER

Open obligation: OBL-TRR-03-VANISHING-BOUND

Open obligation: OBL-TRR-03-FINITE-DIMENSIONAL

Open obligation: OBL-TRR-03-GEOMETRIC-FROBENIUS

Lemma 5.5.4 (Inverse laws stored by the supplied geometric Frobenius). *Uses:* def:weil-trr03-supplied-compact-support-frobenius-family

Lean: Weil.Trace.TRR03SuppliedCompactSupportFrobeniusFamily.inverse_laws

For every $i \in \mathrm{Fin}(B+1)$, writing F_i and F_i^{-1} for the forward and inverse maps of $F_{c,i}^{\mathrm{geom}}$,

$$F_i^{-1} F_i = \mathrm{id}_{V_i}, \qquad F_i F_i^{-1} = \mathrm{id}_{V_i}.$$

Proof. Uses: def:weil-trr03-supplied-compact-support-frobenius-family

These are the left- and right-inverse fields of the supplied linear equivalence $F_{c,i}^{\mathrm{geom}}$. No geometric theorem is invoked. □

Definition 5.5.5 (Canonical displayed basis of a supplied degree). Uses: def:weil-trr03-supplied-compact-support-frobenius-family

Lean: Weil.Trace.TRR03SuppliedCompactSupportFrobeniusFamily.basis

For $i \in \mathrm{Fin}(B+1)$, define

$$r_i := \mathrm{finrank}_E(V_i), \qquad b_i := \texttt{Module.finBasis}\ E\ V_i : \mathrm{Basis}(\mathrm{Fin}(r_i), E, V_i).$$

This is the pinned noncomputable basis selector applied to the supplied finite-dimensionality instance.

Definition 5.5.6 (Based automorphism attached to one supplied degree). Uses: def:weil-based-linear-automorphism-datum, def:weil-trr03-supplied-compact-support-frobenius-family, def:weil-trr03-canonical-compact-support-basis

Constructed by: lemma:weil-trr03-supplied-geometric-frobenius-inverses

Lean: Weil.Trace.TRR03SuppliedCompactSupportFrobeniusFamily.degreeData

For $i \in \mathrm{Fin}(B+1)$, define D_i^c to have rank r_i, carrier $V_i = H_c^i(X, E_X)$, displayed basis b_i, forward endomorphism F_i, and inverse F_i^{-1}. Use the preceding two inverse equations for the proof fields of the based automorphism datum.

Definition 5.5.7 (Bounded based family from the supplied degrees). Uses: def:weil-bounded-graded-based-automorphism-family, def:weil-trr03-supplied-compact-support-frobenius-family, def:weil-trr03-degree-based-automorphism

Lean: Weil.Trace.TRR03SuppliedCompactSupportFrobeniusFamily.boundedFamily

Define G_c to have stored bound B and degree- i datum D_i^c for every $i \in \mathrm{Fin}(B+1)$.

Lemma 5.5.8 (Carrier projection of the TRR-03 formal adapter). *Uses:* def:weil-trr03-bounded-based-compact-support-family

Lean: Weil.Trace.TRR03SuppliedCompactSupportFrobeniusFamily.boundedFamily_carrier

For every $i \in \mathrm{Fin}(B+1)$, the carrier of the degree- i datum in G_c is literally $H_c^i(X, E_X)$.

Proof. Uses: def:weil-trr03-bounded-based-compact-support-family, def:weil-trr03-degree-based-automorphism

Unfold the family and then its degree datum. The carrier field is V_i, which was defined to be $H_c^i(X, E_X)$. □

Lemma 5.5.9 (Forward-operator projection of the TRR-03 formal adapter). *Uses:* def:weil-trr03-bounded-based-compact-support-family

Lean: Weil.Trace.TRR03SuppliedCompactSupportFrobeniusFamily.boundedFamily_forward

For every $i \in \mathrm{Fin}(B+1)$, the forward endomorphism stored in G_c is the forward linear map F_i of $F_{c,i}^{\mathrm{geom}}$, hence is the supplied induced geometric Frobenius on $H_c^i(X, E_X)$.

Proof. Uses: def:weil-trr03-bounded-based-compact-support-family, def:weil-trr03-degree-based-automorphism, def:weil-trr03-supplied-compact-support-frobenius-family

The first equality is the forward-map field of D_i^c. Its geometric interpretation is exactly the actual-realization equality stored in the supplied input; no converse inference from a trace identity is made. □

Lemma 5.5.10 (Inverse-operator projection of the TRR-03 formal adapter). *Uses:* `def:weil-trr03-bounded-based-compact-support-family`

Lean: `Weil.Trace.TRR03SuppliedCompactSupportFrobeniusFamily.boundedFamily_inverse`

For every $i \in \mathrm{Fin}(B+1)$, the inverse map stored in the degree-i datum is the supplied geometric-Frobenius inverse F_i^{-1}.

Proof. Uses: `def:weil-trr03-bounded-based-compact-support-family`, `def:weil-trr03-degree-based-automorphism`

This is the inverse-map field of the displayed degree datum. □

Lemma 5.5.11 (Stored bound of the TRR-03 formal adapter). *Uses:* `def:weil-trr03-bounded-based-compact-support-family`

Lean: `Weil.Trace.TRR03SuppliedCompactSupportFrobeniusFamily.boundedFamily_bound`

The natural-number bound stored in G_c is exactly the supplied B.

Proof. Uses: `def:weil-trr03-bounded-based-compact-support-family`

This is the first field of the defining pair. □

Lemma 5.5.12 (Out-of-range compact-support vanishing remains explicit). *Uses:* `def:weil-trr03-supplied-compact-support-frobenius-family`, `def:weil-trr03-bounded-based-compact-support-family`

Lean: `Weil.Trace.TRR03SuppliedCompactSupportFrobeniusFamily.vanishesOutside`

For every integer j with $j < 0$ or $B < j$, the supplied equivalence z_j identifies $H_c^j(X, E_X)$ with the zero E-vector space.

Proof. Uses: `def:weil-trr03-supplied-compact-support-frobenius-family`

Project the out-of-range vanishing field. The bounded-family constructor does not manufacture this certificate by omitting those degrees. □

Lemma 5.5.13 (Coordinate traces of iterates are basis independent). *Uses:* `def:weil-trr03-degree-based-automorphism`, `def:weil-explicit-matrix-trace`, `def:weil-two-basis-endomorphism-datum`

Lean: `Weil.Trace.TRR03SuppliedCompactSupportFrobeniusFamily.trace_pow_independent`

Fix $i \le B$, let c_i be any second displayed basis of V_i indexed by $\mathrm{Fin}(r_i)$, and let M_b, M_c be the matrices of F_i in b_i, c_i. For every $n \in \mathbf{N}$,

$$\mathrm{tr}_\Sigma(M_b^n) = \mathrm{tr}_\Sigma(M_c^n) \in E.$$

Thus every trace summand used by Module 03 has the coefficient-field type E and does not depend on the displayed basis.

Proof. Uses: `lemma:weil-two-basis-transition-inverses`, `lemma:weil-two-basis-coordinate-conjugacy`, `lemma:weil-explicit-matrix-trace-cyclic`

Let S, R be the transition matrices, so $M_c = RM_bS$, $SR = I$, and $RS = I$. Induction on n gives $M_c^n = RM_b^nS$: in the successor step the adjacent product SR cancels. Cyclicity of the explicit matrix trace then gives

$$\mathrm{tr}_\Sigma(RM_b^nS) = \mathrm{tr}_\Sigma(SRM_b^n) = \mathrm{tr}_\Sigma(M_b^n).$$

Both traces are finite sums of diagonal entries of matrices over E, so the equality has exactly the coefficient-field type asserted. □

Definition 5.5.14 (Intrinsic reciprocal factor of a supplied degree). Uses: `def:weil-based-reciprocal-determinant`, `def:weil-trr03-degree-based-automorphism`

Lean: `Weil.Trace.TRR03SuppliedCompactSupportFrobeniusFamily.intrinsicFactor`

For $i \le B$, define

$$\mathcal{P}_{c,i}(T) := P_{D_i^c}(T) \in E[T].$$

After the next independence theorem, this common polynomial is denoted $\det(1 - TF_i \mid H_c^i(X, E_X))$.

Theorem 5.5.15 (The compact-support reciprocal factor is basis independent). *Uses:* `def:weil-trr03-intrinsic-compact-support-factor`, `def:weil-two-basis-endomorphism-datum`

Lean: `Weil.Trace.TRR03SuppliedCompactSupportFrobeniusFamily.intrinsicFactor_eq`

For any second displayed basis c_i of V_i, indexed by $\mathrm{Fin}(r_i)$, the reciprocal determinant computed from (V_i, c_i, F_i) equals $\mathcal{P}_{c,i}(T)$.

Proof. Uses: def:weil-trr03-intrinsic-compact-support-factor,def:weil-trr03-degree-based-automorphism,thm:weil-based-reciprocal-determinant-independent

Apply the existing two-basis theorem to b_i, c_i and the single endomorphism F_i. Its first reciprocal polynomial is $\mathcal{P}_{c,i}$ by definition, and its second is the polynomial computed in c_i. Reverse the equality if necessary. □

Lemma 5.5.16 (The graded factor is the intrinsic compact-support determinant). *Uses: def:weil-trr03-bounded-based-compact-support-family, def:weil-trr03-intrinsic-compact-support-factor, def:weil-graded-reciprocal-factor*

Lean: *Weil.Trace.TRR03SuppliedCompactSupportFrobeniusFamily.factor_eq_det*

For every $i \le B$,

$$P_{G_c,i}(T) = \mathcal{P}_{c,i}(T) = \det(1 - TF_i \mid H^i_c(X, E_X)).$$

The last expression denotes the basis-independent common polynomial proved in the preceding theorem.

Proof. Uses: def:weil-trr03-bounded-based-compact-support-family,def:weil-trr03-degree-based-automorphism,def:weil-trr03-intrinsic-compact-support-factor,def:weil-graded-reciprocal-factor,thm:weil-trr03-compact-support-factor-basis-independent

Unfold the graded family and the degreewise factor once. Both the first and second expressions become $P_{D_i^c}$. The final notation is well defined by the basis-independence theorem, which identifies every alternate coordinate computation with this common polynomial. □

Lemma 5.5.17 (Degreewise reciprocal factors have constant coefficient one). *Uses: def:weil-graded-reciprocal-factor, def:weil-polynomial-power-series-inclusion*

Lean: *Weil.LinearAlgebra.BoundedGradedBasedAutomorphismFamily.factor_coeff_zero*

For every $i \le B$,

$$P_{G,i}(0) = 1, \qquad c(\jmath(P_{G,i})) = 1.$$

Proof. Uses: def:weil-graded-reciprocal-factor,lemma:weil-based-reciprocal-determinant-constant

Apply the based reciprocal-determinant constant-coefficient lemma to the datum D_i, then unfold $P_{G,i} = P_{D_i}$. □

Definition 5.5.18 (Signed degree factor as a formal series). Uses: def:weil-graded-reciprocal-factor,def:weil-geometric-inverse,def:weil-polynomial-power-series-inclusion

Lean: Weil.LinearAlgebra.BoundedGradedBasedAutomorphismFamily.signedFactor

For $i \le B$, define

$$Q_{G,i}(T) := \begin{cases} \mathrm{GeomInv}(\jmath(P_{G,i})), & i \text{ even}, \\ \jmath(P_{G,i}), & i \text{ odd}. \end{cases}$$

This is the formal-series meaning of $P_{G,i}^{(-1)^{i+1}}$.

Definition 5.5.19 (Alternating determinant product). Uses: def:weil-bounded-graded-based-automorphism-family,def:weil-signed-graded-determinant-factor

Lean: Weil.LinearAlgebra.BoundedGradedBasedAutomorphismFamily.alternatingDet

Define the finite formal product

$$\Delta_G(T) := \prod_{i=0}^{B} Q_{G,i}(T).$$

Definition 5.5.20 (Alternating trace logarithm). Uses: def:weil-bounded-graded-based-automorphism-family,def:weil-coordinate-endomorphism-matrix,def:weil-explicit-matrix-trace

Lean: Weil.LinearAlgebra.BoundedGradedBasedAutomorphismFamily.traceLogSeries

Define $S_G(T) \in TK[[T]]$ by

$$[T^0]S_G = 0, \qquad [T^n]S_G = \frac{1}{n}\sum_{i=0}^{B}(-1)^i \operatorname{tr}_\Sigma(M_{D_i}^n) \quad (n \ge 1).$$

Lemma 5.5.21 (Alternating trace logarithm is a finite signed sum). *Uses: def:weil-alternating-trace-log-series, def:weil-based-trace-log-series*

Lean: *Weil.LinearAlgebra.BoundedGradedBasedAutomorphismFamily.traceLogSeries_eq_sum*

One has

$$S_G = \sum_{i=0}^{B}(-1)^i S_{D_i}.$$

Proof. Uses: `def:weil-alternating-trace-log-series`, `def:weil-based-trace-log-series`
Both sides have constant coefficient zero. In positive degree n, the right side has coefficient

$$\sum_{i=0}^{B}(-1)^i\frac{\mathrm{tr}_\Sigma(M_{D_i}^n)}{n},$$

which is the defining coefficient of the left side after factoring out $1/n$. Equality in every coefficient proves the series equality. □

Lemma 5.5.22 (Signed degree factor as an exponential). *Uses:* `def:weil-signed-graded-determinant-factor`, `def:weil-based-trace-log-series`, `def:weil-formal-exp`

Lean: `Weil.LinearAlgebra.BoundedGradedBasedAutomorphismFamily.signedFactor_eq_exp`
For $i \le B$,

$$Q_{G,i} = \mathrm{Exp}\big((-1)^i S_{D_i}\big).$$

Proof. Uses: `def:weil-signed-graded-determinant-factor`, `thm:weil-based-trace-log-det`, `lemma:weil-formal-exp-neg`, `lemma:weil-geometric-inverse-involutive`
If i is even, then $(-1)^i = 1$, and the statement is exactly the based determinant–trace theorem. If i is odd, then $(-1)^i = -1$. The exponential-of-a-negative lemma turns the right side into $\mathrm{GeomInv}(\mathrm{Exp}(S_{D_i}))$. The determinant theorem replaces the inner exponential by $\mathrm{GeomInv}(\jmath(P_{G,i}))$, and involutivity reduces the result to $\jmath(P_{G,i})$, the odd signed factor. □

Corollary 5.5.23 (Finite alternating determinant–trace identity). *Uses:* `def:weil-alternating-determinant-product`, `def:weil-alternating-trace-log-series`, `def:weil-formal-exp`

Lean: `Weil.LinearAlgebra.BoundedGradedBasedAutomorphismFamily.alternatingDet_eq_exp`
One has

$$\Delta_G(T) = \mathrm{Exp}(S_G(T)) = \mathrm{Exp}\left(\sum_{n\ge1}\left(\sum_{i=0}^{B}(-1)^i\,\mathrm{tr}_\Sigma(M_{D_i}^n)\right)\frac{T^n}{n}\right).$$

Proof. Uses: `def:weil-alternating-determinant-product`, `lemma:weil-signed-graded-factor-exp`, `lemma:weil-formal-exp-finset-sum`, `lemma:weil-alternating-trace-log-finset-sum`
Replace each factor of Δ_G by the exponential supplied by the signed-factor lemma. The finite exponential-sum formula changes their product into $\mathrm{Exp}(\sum_i(-1)^i S_{D_i})$. The preceding finite-sum identity identifies its exponent with S_G. □

5.6 Normalized rational germs and Taylor expansion at zero

Definition 5.6.1 (Normalized polynomial fraction datum). *Uses:*

Lean: `Weil.Algebra.NormalizedPolynomialFraction`
A normalized polynomial fraction over K is a pair $\mathcal{F} = (N_\mathcal{F}, D_\mathcal{F}) \in K[T]^2$ together with the displayed equality

$$D_\mathcal{F}(0) = 1.$$

Definition 5.6.2 (Equivalence of normalized polynomial fractions). *Uses:* `def:weil-normalized-polynomial-fraction-datum`

Lean: `Weil.Algebra.NormalizedPolynomialFraction.Rel`
For normalized fractions $\mathcal{F}, \mathcal{G}$, define

$$\mathcal{F} \sim \mathcal{G} \quad :\Longleftrightarrow \quad N_\mathcal{F} D_\mathcal{G} = N_\mathcal{G} D_\mathcal{F} \quad \text{in } K[T].$$

Lemma 5.6.3 (Reflexivity of normalized-fraction equivalence). *Uses:* `def:weil-normalized-polynomial-fraction-relation`

Lean: `Weil.Algebra.NormalizedPolynomialFraction.rel_refl`
For every normalized fraction $\mathcal{F}$, $\mathcal{F} \sim \mathcal{F}$.

Proof. Uses: `def:weil-normalized-polynomial-fraction-relation`
Both cross-products are the same polynomial $N_\mathcal{F} D_\mathcal{F}$. □

Lemma 5.6.4 (Symmetry of normalized-fraction equivalence). *Uses:* def:weil-normalized-polynomial-fraction-relation

Lean: Weil.Algebra.NormalizedPolynomialFraction.rel_symm

If $\mathcal{F} \sim \mathcal{G}$, then $\mathcal{G} \sim \mathcal{F}$.

Proof. Uses: def:weil-normalized-polynomial-fraction-relation

The defining polynomial equality remains true after exchanging its two sides. □

Lemma 5.6.5 (A normalized denominator is nonzero). *Uses:* def:weil-normalized-polynomial-fraction-datum

Lean: Weil.Algebra.NormalizedPolynomialFraction.denom_ne_zero

For every normalized fraction $\mathcal{F}$,

$$D_{\mathcal{F}} \neq 0.$$

Proof. Uses: def:weil-normalized-polynomial-fraction-datum

The zero polynomial evaluates to 0 at $T = 0$, whereas $D_{\mathcal{F}}(0) = 1$. Since $0 \neq 1$ in the field K, the denominator cannot be the zero polynomial. □

Lemma 5.6.6 (Transitivity of normalized-fraction equivalence). *Uses:* def:weil-normalized-polynomial-fraction-relation

Lean: Weil.Algebra.NormalizedPolynomialFraction.rel_trans

If $\mathcal{F} \sim \mathcal{G}$ and $\mathcal{G} \sim \mathcal{H}$, then $\mathcal{F} \sim \mathcal{H}$.

Proof. Uses: def:weil-normalized-polynomial-fraction-relation,lemma:weil-normalized-fraction-denominator-ne-zero

The two hypotheses are

$$N_{\mathcal{F}} D_{\mathcal{G}} = N_{\mathcal{G}} D_{\mathcal{F}}, \qquad N_{\mathcal{G}} D_{\mathcal{H}} = N_{\mathcal{H}} D_{\mathcal{G}}.$$

Multiply the first by $D_{\mathcal{H}}$, the second by $D_{\mathcal{F}}$, and substitute. After commuting factors,

$$D_{\mathcal{G}}(N_{\mathcal{F}} D_{\mathcal{H}} - N_{\mathcal{H}} D_{\mathcal{F}}) = 0.$$

Rewrite the right side as $D_{\mathcal{G}} \cdot 0$. Since the polynomial ring over a field is an integral domain and $D_{\mathcal{G}} \neq 0$, apply the exact cancellation theorem $\texttt{mul_left_cancel}_0$. It gives $N_{\mathcal{F}} D_{\mathcal{H}} - N_{\mathcal{H}} D_{\mathcal{F}} = 0$, hence the cross-product equality defining $\mathcal{F} \sim \mathcal{H}$. □

Proposition 5.6.7 (Normalized-fraction equivalence relation). *Uses:* def:weil-normalized-polynomial-fraction-relation

Lean: Weil.Algebra.NormalizedPolynomialFraction.relEquivalence

The relation $\sim$ is an equivalence relation.

Proof. Uses: lemma:weil-normalized-fraction-relation-refl,lemma:weil-normalized-fraction-relation-symm,lemma:weil-normalized-fraction-relation-trans

The three preceding lemmas are respectively the reflexive, symmetric, and transitive fields of an equivalence relation. □

Definition 5.6.8 (Normalized-fraction setoid). Uses: def:weil-normalized-polynomial-fraction-relation

Constructed by: prop:weil-normalized-fraction-equivalence-relation

Lean: Weil.Algebra.NormalizedPolynomialFraction.setoid

Define $\mathcal{S}_K$ to be the `Setoid` on normalized polynomial fractions whose relation field is $\mathcal{F} \sim \mathcal{G}$. Its reflexive, symmetric, and transitive fields are supplied by the previously proved equivalence-relation certificate recorded by the constructor metadata.

Definition 5.6.9 (Regular rational germ at zero). Uses: def:weil-normalized-polynomial-fraction-datum,def:weil-normalized-fraction-setoid

Lean: Weil.Algebra.RegularRationalGerm

Define

$$\mathrm{RegRat}_0(K) := \mathrm{Quotient}(\mathcal{S}_K).$$

Definition 5.6.10 (Rational-function value of a normalized representative)**.** Uses: `def:weil-normalized-polynomial-fraction-datum`

Lean: `Weil.Algebra.NormalizedPolynomialFraction.ratFuncValue`

For $\mathcal{F} = (N_{\mathcal{F}}, D_{\mathcal{F}})$, define

$$\mathrm{rv}(\mathcal{F}) := \mathrm{RatFunc.mk}(N_{\mathcal{F}}, D_{\mathcal{F}}) \in K(T).$$

Lemma 5.6.11 (Rational-function value respects normalized equivalence)**.** Uses: *def:weil-normalized-polynomial-fraction-relation*, *def:weil-normalized-fraction-rational-value*

Lean: *Weil.Algebra.NormalizedPolynomialFraction.ratFuncValue_eq*

If $\mathcal{F} \sim \mathcal{G}$, then

$$\mathrm{rv}(\mathcal{F}) = \mathrm{rv}(\mathcal{G}).$$

Proof. Uses: `def:weil-normalized-polynomial-fraction-relation`, `def:weil-normalized-fraction-rational-value`, `lemma:weil-normalized-fraction-denominator-ne-zero`

Both denominators are nonzero. Apply the reverse direction of the exact theorem `RatFunc.mk_eq_mk`; its required cross-product equality is exactly $\mathcal{F} \sim \mathcal{G}$. □

Lemma 5.6.12 (Equality of representative rational values recovers the relation)**.** Uses: *def:weil-normalized-polynomial-fraction-relation*, *def:weil-normalized-fraction-rational-value*

Lean: *Weil.Algebra.NormalizedPolynomialFraction.rel_of_ratFuncValue_eq*

If $\mathrm{rv}(\mathcal{F}) = \mathrm{rv}(\mathcal{G})$, then $\mathcal{F} \sim \mathcal{G}$.

Proof. Uses: `def:weil-normalized-polynomial-fraction-relation`, `def:weil-normalized-fraction-rational-value`, `lemma:weil-normalized-fraction-denominator-ne-zero`

Apply the forward direction of `RatFunc.mk_eq_mk`, with the two denominator-nonzero certificates supplied by the preceding lemma. Its conclusion is exactly the cross-product equality defining $\mathcal{F} \sim \mathcal{G}$. □

Definition 5.6.13 (Rational-function realization of regular rational germs)**.** Uses: `def:weil-regular-rational-germ`, `def:weil-normalized-fraction-rational-value`

Constructed by: `lemma:weil-normalized-fraction-rational-value-respects`

Lean: `Weil.Algebra.RegularRationalGerm.toRatFunc`

Define

$$\rho_K : \mathrm{RegRat}_0(K) \longrightarrow K(T)$$

by `Quotient.lift`, using the preceding respect lemma as its relation-respect argument.

Lemma 5.6.14 (Rational realization on a quotient representative)**.** Uses: *def:weil-regular-rational-germ-ratfunc-map*, *def:weil-normalized-fraction-rational-value*

Lean: *Weil.Algebra.RegularRationalGerm.toRatFunc_mk*

For every normalized fraction $\mathcal{F}$,

$$\rho_K([\mathcal{F}]) = \mathrm{rv}(\mathcal{F}).$$

Proof. Uses: `def:weil-regular-rational-germ-ratfunc-map`

This is the computation rule for the displayed `Quotient.lift`; it reduces definitionally to reflexivity. □

Lemma 5.6.15 (The rational-function realization is injective)**.** Uses: *def:weil-regular-rational-germ-ratfunc-map*

Lean: *Weil.Algebra.RegularRationalGerm.toRatFunc_injective*

The map ρ_K is injective.

Proof. Uses: `lemma:weil-normalized-fraction-relation-of-rational-value-eq`, `lemma:weil-regular-rational-germ-ratfunc-map-mk`

Apply `Quotient.inductionOn₂`. On representatives, rewrite both realizations with the preceding computation lemma, apply the representative-level reverse implication, and finish with one application of `Quotient.sound`. □

Lemma 5.6.16 (A normalized denominator series has constant coefficient one)**.** Uses: *def:weil-normalized-polynomial-fraction-datum*, *def:weil-polynomial-power-series-inclusion*

Lean: *Weil.Algebra.NormalizedPolynomialFraction.denomSeries_constantCoeff*

For every normalized fraction $\mathcal{F}$,

$$c(\jmath(D_{\mathcal{F}})) = 1.$$

Proof. Uses: def:weil-normalized-polynomial-fraction-datum,lemma:weil-polynomial-series-coeff
The coefficient formula at degree zero gives $c(\jmath(D_{\mathcal{F}})) = D_{\mathcal{F}}.\mathrm{coeff}(0)$. The exact mathlib identity `Polynomial.coeff_zero_eq_eval_zero` rewrites the latter as $D_{\mathcal{F}}(0)$, which is 1 by the normalization field. □

Definition 5.6.17 (Taylor series of a normalized representative)**.** Uses: def:weil-normalized-polynomial-fraction-datum,def:weil-polynomial-power-series-inclusion,def:weil-geometric-inverse

Constructed by: lemma:weil-normalized-fraction-denominator-series-constant

Lean: `Weil.Algebra.NormalizedPolynomialFraction.taylor`
Define

$$\tau_{\mathrm{rep}}(\mathcal{F}) := \jmath(N_{\mathcal{F}})\ \mathrm{GeomInv}(\jmath(D_{\mathcal{F}})) \in K[[T]].$$

Lemma 5.6.18 (Taylor series respects normalized equivalence)**.** *Uses:* def:weil-normalized-polynomial-fraction-relation,def:weil-normalized-fraction-taylor-series

Lean: `Weil.Algebra.NormalizedPolynomialFraction.taylor_eq`
If $\mathcal{F} \sim \mathcal{G}$, then

$$\tau_{\mathrm{rep}}(\mathcal{F}) = \tau_{\mathrm{rep}}(\mathcal{G}).$$

Proof. Uses: def:weil-normalized-polynomial-fraction-relation,def:weil-normalized-fraction-taylor-series,lemma:weil-polynomial-series-mul,lemma:weil-normalized-fraction-denominator-series-constant,lemma:weil-geometric-inverse-mul
The preceding constant-coefficient lemma supplies the two hypotheses needed by the geometric-inverse identities. Embed the cross-product equality into $K[[T]]$, then multiply it by $\mathrm{GeomInv}(\jmath(D_{\mathcal{F}}))\,\mathrm{GeomInv}(\jmath(D_{\mathcal{G}}))$. Both denominator factors cancel by their geometric-inverse identities, leaving precisely the equality of the two Taylor representatives. □

Lemma 5.6.19 (Equality of representative Taylor series recovers the relation)**.** *Uses:* def:weil-normalized-polynomial-fraction-relation,def:weil-normalized-fraction-taylor-series

Lean: `Weil.Algebra.NormalizedPolynomialFraction.rel_of_taylor_eq`
If $\tau_{\mathrm{rep}}(\mathcal{F}) = \tau_{\mathrm{rep}}(\mathcal{G})$, then $\mathcal{F} \sim \mathcal{G}$.

Proof. Uses: def:weil-normalized-polynomial-fraction-relation,def:weil-normalized-fraction-taylor-series,lemma:weil-normalized-fraction-denominator-series-constant,lemma:weil-geometric-inverse-mul,lemma:weil-polynomial-series-mul,lemma:weil-polynomial-series-injective
Multiply the assumed equality by $\jmath(D_{\mathcal{F}})\jmath(D_{\mathcal{G}})$. The two named constant-one certificates permit the inverse identities, which reduce the result to

$$\jmath(N_{\mathcal{F}} D_{\mathcal{G}}) = \jmath(N_{\mathcal{G}} D_{\mathcal{F}}).$$

Injectivity of $\jmath$ gives the cross-product equality defining the relation. □

Definition 5.6.20 (Taylor map for regular rational germs)**.** Uses: def:weil-regular-rational-germ,def:weil-normalized-fraction-taylor-series

Constructed by: lemma:weil-normalized-fraction-taylor-respects

Lean: `Weil.Algebra.RegularRationalGerm.taylor`
Define

$$\tau_K : \mathrm{RegRat}_0(K) \longrightarrow K[[T]]$$

by `Quotient.lift`, using the preceding Taylor-respect lemma as its relation-respect argument.

Lemma 5.6.21 (Taylor realization on a quotient representative)**.** *Uses:* def:weil-regular-rational-germ-taylor-map, def:weil-normalized-fraction-taylor-series

Lean: `Weil.Algebra.RegularRationalGerm.taylor_mk`
For every normalized fraction $\mathcal{F}$,

$$\tau_K([\mathcal{F}]) = \tau_{\mathrm{rep}}(\mathcal{F}).$$

Proof. Uses: def:weil-regular-rational-germ-taylor-map
This is the computation rule for the displayed `Quotient.lift`; it reduces definitionally to reflexivity. □

Lemma 5.6.22 (The Taylor map for regular rational germs is injective)**.** *Uses:* def:weil-regular-rational-germ-taylor-map

Lean: `Weil.Algebra.RegularRationalGerm.taylor_injective`
The map τ_K is injective.

Proof. Uses: lemma:weil-normalized-fraction-relation-of-taylor-eq,lemma:weil-regular-rational-germ-taylor-map-mk

Apply `Quotient.inductionOn₂`. On representatives, rewrite both Taylor maps with the preceding computation lemma, apply the representative-level reverse implication, and finish with one application of `Quotient.sound`. □

Proposition 5.6.23 (Two faithful realizations of a regular rational germ). Uses: def:weil-regular-rational-germ-ratfunc-map, def:weil-regular-rational-germ-taylor-map, def:weil-normalized-polynomial-fraction-datum, def:weil-polynomial-power-series-inclusion, def:weil-geometric-inverse

Lean: `Weil.Algebra.RegularRationalGerm.realizations`

The two previously defined maps are injective:

$$\mathrm{RegRat}_0(K) \xhookrightarrow{\rho_K} K(T), \qquad \mathrm{RegRat}_0(K) \xhookrightarrow{\tau_K} K[[T]],$$

and for a representative (N, D) the two images are respectively N/D and $\jmath(N)\,\mathrm{GeomInv}(\jmath(D))$.

Proof. Uses: def:weil-regular-rational-germ-ratfunc-map,def:weil-regular-rational-germ-taylor-map,lemma:weil-regular-rational-germ-ratfunc-injective,lemma:weil-regular-rational-germ-taylor-injective,lemma:weil-regular-rational-germ-ratfunc-map-mk,lemma:weil-regular-rational-germ-taylor-map-mk

The two injectivity assertions are the preceding two injectivity lemmas. The two representative formulas are the preceding two quotient-lift computation lemmas. □

Definition 5.6.24 (Rational formal series at zero). Uses: def:weil-regular-rational-germ-taylor-map

Lean: `Weil.Algebra.IsRationalAtZero`

A series $A \in K[[T]]$ is rational at zero if

$$A \in \mathrm{range}(\tau_K).$$

Definition 5.6.25 (Odd-degree numerator). Uses: def:weil-bounded-graded-based-automorphism-family,def:weil-graded-reciprocal-factor

Lean: `Weil.LinearAlgebra.BoundedGradedBasedAutomorphismFamily.numerator`

Define

$$N_G(T) := \prod_{\substack{0 \le i \le B \\ i \text{ odd}}} P_{G,i}(T) \in K[T].$$

Definition 5.6.26 (Even-degree denominator). Uses: def:weil-bounded-graded-based-automorphism-family,def:weil-graded-reciprocal-factor

Lean: `Weil.LinearAlgebra.BoundedGradedBasedAutomorphismFamily.denominator`

Define

$$D_G(T) := \prod_{\substack{0 \le i \le B \\ i \text{ even}}} P_{G,i}(T) \in K[T].$$

Lemma 5.6.27 (The odd-degree numerator is normalized). Uses: def:weil-alternating-rational-numerator

Lean: `Weil.LinearAlgebra.BoundedGradedBasedAutomorphismFamily.numerator_coeff_zero`

One has

$$N_G(0) = 1.$$

Proof. Uses: def:weil-alternating-rational-numerator,lemma:weil-graded-reciprocal-factor-constant

Evaluation at zero commutes with the displayed finite product. Every odd factor evaluates to 1, so their finite product is 1. □

Lemma 5.6.28 (The even-degree denominator is normalized). Uses: def:weil-alternating-rational-denominator

Lean: `Weil.LinearAlgebra.BoundedGradedBasedAutomorphismFamily.denominator_coeff_zero`

One has

$$D_G(0) = 1.$$

Proof. Uses: def:weil-alternating-rational-denominator,lemma:weil-graded-reciprocal-factor-constant

Evaluation at zero commutes with the finite product, and every even factor has value 1. Hence the product has value 1. □

Lemma 5.6.29 (Alternating determinant as a normalized fraction series). *Uses:* def:weil-alternating-determinant-product, def:weil-alternating-rational-numerator, def:weil-alternating-rational-denominator, def:weil-polynomial-power-series-inclusion, def:weil-geometric-inverse

Lean: Weil.LinearAlgebra.BoundedGradedBasedAutomorphismFamily.alternatingDet_fraction

One has

$$\Delta_G = \jmath(N_G)\,\mathrm{GeomInv}(\jmath(D_G)).$$

Proof. Uses: def:weil-signed-graded-determinant-factor,lemma:weil-polynomial-series-mul,lemma:weil-graded-reciprocal-factor-constant,lemma:weil-geometric-inverse-finset-product

Partition the finite product defining Δ_G into odd and even indices. The odd subproduct is $\jmath(N_G)$, because $\jmath$ preserves finite products. The even subproduct is the product of the geometric inverses of $\jmath(P_{G,i})$. Every such factor has constant coefficient one; the finite-product inverse lemma identifies their product with $\mathrm{GeomInv}(\jmath(D_G))$. Multiplying the two subproducts gives the displayed formula. □

Definition 5.6.30 (Canonical normalized fraction of a graded family). Uses: def:weil-normalized-polynomial-fraction-datum,def:weil-alternating-rational-numerator,def:weil-alternating-rational-denominator

Constructed by: lemma:weil-alternating-rational-denominator-constant

Lean: Weil.LinearAlgebra.BoundedGradedBasedAutomorphismFamily.normalizedFraction

Define $\mathcal{F}_G$ to be the normalized polynomial fraction with numerator N_G, denominator D_G, and normalization field $D_G(0) = 1$.

Definition 5.6.31 (Canonical regular rational germ of a graded family). Uses: def:weil-graded-canonical-normalized-fraction,def:weil-regular-rational-germ

Lean: Weil.LinearAlgebra.BoundedGradedBasedAutomorphismFamily.rationalGerm

Define

$$\mathfrak{z}_G := [\mathcal{F}_G] \in \mathrm{RegRat}_0(K).$$

Lemma 5.6.32 (Taylor series of the canonical graded germ). *Uses:* def:weil-graded-canonical-rational-germ, def:weil-regular-rational-germ-taylor-map, def:weil-alternating-determinant-product

Lean: Weil.LinearAlgebra.BoundedGradedBasedAutomorphismFamily.taylor_rationalGerm

One has

$$\tau_K(\mathfrak{z}_G) = \Delta_G.$$

Proof. Uses: def:weil-graded-canonical-normalized-fraction,lemma:weil-regular-rational-germ-taylor-map-mk,lemma:weil-alternating-determinant-normalized-fraction

Apply the Taylor computation lemma to the representative $\mathcal{F}_G = (N_G, D_G)$. Its right side is the normalized-fraction series, which the alternating-determinant fraction lemma identifies with Δ_G. □

Corollary 5.6.33 (The alternating determinant series is rational over K). *Uses:* def:weil-alternating-determinant-product, def:weil-rational-formal-series-at-zero, def:weil-graded-canonical-rational-germ, def:weil-regular-rational-germ-ratfunc-map, def:weil-alternating-rational-numerator, def:weil-alternating-rational-denominator

Lean: Weil.LinearAlgebra.BoundedGradedBasedAutomorphismFamily.alternatingDet_rational

The series Δ_G is rational at zero. Its explicitly constructed rational function is

$$\rho_K(\mathfrak{z}_G) = \frac{N_G(T)}{D_G(T)} \in K(T).$$

Proof. Uses: def:weil-rational-formal-series-at-zero,def:weil-graded-canonical-rational-germ,def:weil-regular-rational-germ-ratfunc-map,lemma:weil-graded-canonical-rational-germ-taylor,lemma:weil-regular-rational-germ-ratfunc-map-mk

The defining range witness is the previously defined germ $\mathfrak{z}_G$, whose Taylor series is Δ_G. Apply the rational realization computation lemma to the representative $\mathcal{F}_G = (N_G, D_G)$ for the displayed rational-function formula. □

5.7 Normalized rational germs and descent from a coefficient extension

Definition 5.7.1 (Normalized rational germ)**.** Uses:

Lean: `Weil.TraceCore4.NormalizedGerm`

For a field F, a normalized rational germ is a pair $(N, D) \in F[T]^2$ with $D(0) = 1$, modulo the relation

$$(N, D) \sim (N', D') \iff ND' = N'D.$$

Its Taylor series is

$$\tau_F[N, D] := \jmath(N)\,\mathrm{GeomInv}(\jmath(D)) \in F[[T]].$$

Lemma 5.7.2 (Taylor map is well-defined and injective)**.** *Uses:* *def: tracecore4-normalized-germ*

Lean: *Weil.TraceCore4.NormalizedGerm.taylor_injective*

The Taylor assignment respects $\sim$. If two normalized germs have equal Taylor series, then their cross products are equal, hence they represent the same rational germ.

Proof. Uses: `def:tracecore4-normalized-germ`

Since $D(0) = D'(0) = 1$, both denominator series are units. Multiply the Taylor equality by the two denominators and use multiplicativity of the power-series inclusion to obtain $\jmath(ND') = \jmath(N'D)$. Coefficient extensionality followed by injectivity of the polynomial-to-series inclusion gives $ND' = N'D$. The same calculation with the relation as input proves well-definedness. □

Definition 5.7.3 (Hankel rows of a rational-coefficient series)**.** Uses:

Lean: `Weil.TraceCore4.HankelRows`

For $A = \sum_{n\ge 0} a_n T^n \in \mathbf{Q}[[T]]$, $e, d \ge 0$, and $n > e$, set

$$h_n^{(d)} := (a_n, a_{n-1}, \dots, a_{n-d}) \in \mathbf{Q}^{d+1}, \qquad a_j := 0 \ (j < 0).$$

Lemma 5.7.4 (Finite Hankel kernel descent)**.** *Uses:* *def: tracecore4-hankel-rows*

Lean: *Weil.TraceCore4.HankelKernelDescent*

Suppose $K/\mathbf{Q}$ is a field extension and $q = (q_0, \dots, q_d) \in K^{d+1}$ is nonzero with

$$\sum_{j=0}^{d} q_j a_{n-j} = 0 \qquad (n > e).$$

Then there are $s \le d$ and $\tilde{q}_0, \dots, \tilde{q}_{d-s} \in \mathbf{Q}$, with $\tilde{q}_0 = 1$, such that

$$\sum_{j=0}^{d-s} \tilde{q}_j a_{n-j} = 0 \qquad (n > e - s).$$

Proof. Uses: `def:tracecore4-hankel-rows`

Let W be the $mathbf{Q}$-span of all rows $h_n^{(d)}$ with $n > e$. Choose a maximal linearly independent finite list of rows; it has at most $d+1$ entries and therefore is finite. Every row is a $mathbf{Q}$-linear combination of that list, so the same list spans after extension of scalars to K. The displayed q lies in the K-orthogonal complement of the extended row space. Hence $\dim_K(W \otimes_{\mathbf{Q}} K) \le d$, and the $\mathbf{Q}$-orthogonal complement of W is nonzero. Gaussian elimination over $\mathbf{Q}$ produces a nonzero vector $u = (u_0, \dots, u_d) \in \mathbf{Q}^{d+1}$ annihilating every row. Let s be the least index with $u_s \ne 0$. Divide by u_s and put $\tilde{q}_j = u_{s+j}/u_s$. Replacing n by $n + s$ in the row equations gives the asserted recurrence for $n > e - s$. □

Lemma 5.7.5 (Normalized denominator and numerator truncation)**.** *Uses:* *def: tracecore4-hankel-rows*

Lean: *Weil.TraceCore4.normalizedQDenominator*

Under the hypotheses of the preceding lemma, define

$$D(T) := \sum_{j=0}^{d-s} \tilde{q}_j T^j, \qquad N(T) := \sum_{n=0}^{\max(0,e-s)} [T^n](AD)\,T^n.$$

Then $D(0) = 1$, $N, D \in \mathbf{Q}[T]$, and

$$A = \jmath(N)\,\mathrm{GeomInv}(\jmath(D)).$$

Proof. Uses: def:tracecore4-hankel-rows,lemma:tracecore4-hankel-kernel-descent
The normalization gives $D(0) = \tilde{q}_0 = 1$. The coefficient of T^n in AD is $\sum_j \tilde{q}_j a_{n-j}$, which vanishes for $n > e - s$ by the descended recurrence. Thus $AD = \jmath(N)$ by coefficient extensionality. Multiply by the geometric inverse of $\jmath(D)$ to obtain the displayed equality. □

Theorem 5.7.6 (Formal-series to rational-function passage). *Uses:* def:tracecore4-normalized-germ, def:tracecore4-hankel-rows

Lean: Weil.TraceCore4.rationalDescent
Let $A \in \mathbf{Q}[[T]]$ and let $K/\mathbf{Q}$ be a field extension. If the scalar extension of A is the Taylor series of a normalized germ (N_K, D_K) over K, then A is the Taylor series of a normalized germ (N, D) over $\mathbf{Q}$. The germ (N, D) is unique.

Proof. Uses: def:tracecore4-normalized-germ,lemma:tracecore4-hankel-kernel-descent,lemma:tracecore4-normalized-q-denominator,lemma:tracecore4-germ-taylor-injective
Write $e = \deg N_K$ and $d = \deg D_K$. Equality of the scalar-extended Taylor series and multiplication by D_K give $\sum_{j=0}^{d} (D_K)_j a_{n-j} = 0$ for every $n > e$; the coefficient vector of D_K is nonzero. Apply finite Hankel kernel descent and then normalized denominator/numerator truncation to obtain (N, D) over $\mathbf{Q}$. The constructed Taylor series is A. If (N', D') is another normalized pair with Taylor series A, the injectivity lemma gives $ND' = N'D$, so the two pairs define the same normalized germ. □

5.8 Conditional trace formula and cohomological factorization

Status. Convention gate for later scheme instantiation. The operator in degree i must be the fixed geometric Frobenius on compactly supported cohomology; its factor is $P_i(T) = \det(1 - TF_{\mathrm{geom}} \mid H^i_c)$. The variable is the same zeta variable T, not its reciprocal. Thus a root β of P_i corresponds to the Frobenius eigenvalue β^{-1}. A geometric instantiation is permitted only for a separated finite-type structural morphism, so that $R\Gamma_c = Ra_!$ is defined, and only after each D_i has been identified with geometric Frobenius on H^i_c. No arithmetic-Frobenius variant may be inserted without first proving and recording the inverse-convention transport.

Definition 5.8.1 (Counting logarithm over a coefficient field). Uses: def:weil-finite-extension-point-count

Lean: Weil.Trace.CoefficientCountingLogSeries
Let P be a finite-extension point interface and let K be a characteristic-zero field. Define $S_{P,K}(T) \in TK[[T]]$ by

$$[T^0]S_{P,K} = 0, \qquad [T^n]S_{P,K} = \frac{(N_n(P) : K)}{(n : K)} \quad (n \geq 1),$$

where both natural numbers are mapped by the canonical characteristic-zero homomorphism $\mathbf{N} \to K$.

Definition 5.8.2 (Counting zeta series over a coefficient field). Uses: def:weil-coefficient-counting-log-series,def:weil-formal-exp

Lean: Weil.Trace.CoefficientCountingZetaSeries
Define

$$Z_{P,K}(T) := \mathrm{Exp}(S_{P,K}(T)).$$

Definition 5.8.3 (Coefficientwise map of rational power series). Uses:

Lean: Weil.Trace.mapRationalPowerSeries
Let $\iota_{\mathbf{Q},K} : \mathbf{Q} \to + * K$ be the unique field homomorphism. Define the power-series ring homomorphism

$$\mathrm{map}_{\mathbf{Q},K} := \mathrm{PowerSeries.map}(\iota_{\mathbf{Q},K}) : \mathbf{Q}[[T]] \longrightarrow + * K[[T]].$$

Thus $[T^n]\,\mathrm{map}_{\mathbf{Q},K}(A) = \iota_{\mathbf{Q},K}([T^n]A)$ for every n.

Lemma 5.8.4 (Scalar extension of the counting logarithm). *Uses:* def:weil-rational-power-series-scalar-extension, def:weil-finite-extension-counting-log-series, def:weil-coefficient-counting-log-series

Lean: Weil.Trace.mapRationalPowerSeries_countingLog
One has

$$\mathrm{map}_{\mathbf{Q},K}(S_P) = S_{P,K}.$$

Proof. Uses: def:weil-rational-power-series-scalar-extension,def:weil-finite-extension-counting-log-series,def:weil-coefficient-counting-log-series

The degree-zero coefficients are zero. In degree $n \geq 1$, the coefficient on both sides is $\iota_{\mathbf{Q},K}(N_n(P)/n) = (N_n(P) : K)/(n : K)$. Coefficient extensionality proves the equality. □

Lemma 5.8.5 (Coefficientwise scalar extension commutes with formal exponential). *Uses:* def:weil-rational-power-series-scalar-extension, def:weil-formal-exp

Lean: Weil.Trace.mapRationalPowerSeries_exp

For every positive-order $U \in \mathbf{Q}[[T]]$,

$$\mathrm{map}_{\mathbf{Q},K}(\mathrm{Exp}(U)) = \mathrm{Exp}(\mathrm{map}_{\mathbf{Q},K}(U)).$$

Proof. Uses: def:weil-rational-power-series-scalar-extension,lemma:weil-formal-exp-coeff

Fix degree N. The finite exponential coefficient formula expresses each side as a sum over $0 \leq r \leq N$. The field map preserves the finite sum, products, powers, and the rational scalar $1/r!$. Corresponding summands are therefore equal in degree N. Equality for every N proves the series identity. □

Lemma 5.8.6 (The coefficient-field zeta is the scalar extension of the counting zeta). *Uses:* def:weil-coefficient-counting-zeta-series, def:weil-finite-extension-counting-zeta-series, def:weil-rational-power-series-scalar-extension

Lean: Weil.Trace.coefficientCountingZetaSeries_eq_map

One has

$$Z_{P,K} = \mathrm{map}_{\mathbf{Q},K}(Z_P).$$

Proof. Uses: lemma:weil-rational-power-series-scalar-extension-counting-log,lemma:weil-rational-power-series-scalar-extension-exp

Unfold the two zeta definitions, commute scalar extension with formal exponential, and then apply the preceding counting-logarithm equality. □

Definition 5.8.7 (Trace-formula predicate at one iterate). Uses: def:weil-finite-extension-point-count,def:weil-bounded-graded-based-automorphism-family,def:weil-coordinate-endomorphism-matrix,def:weil-explicit-matrix-trace

Lean: Weil.Trace.TraceFormulaAt

For $n \geq 1$, a point interface P, and a graded family G, define TraceFormulaAt(P, G, n) to be the equality

$$(N_n(P) : K) = \sum_{i=0}^{B} (-1)^i \,\mathrm{tr}_\Sigma(M_{D_i}^n) \quad \text{in } K.$$

Definition 5.8.8 (Finite cohomological trace datum). Uses: def:weil-finite-extension-point-interface,def:weil-bounded-graded-based-automorphism-family,def:weil-trace-formula-at-iterate

Lean: Weil.Trace.FiniteCohomologicalTraceData

A finite cohomological trace datum over K consists of:

(i) a finite-extension point interface P;

(ii) a bounded graded based automorphism family $G = (D_i)_{0 \leq i \leq B}$;

(iii) for every $n \geq 1$, a displayed proof term

$$t_n : \mathrm{TraceFormulaAt}(P, G, n).$$

The third field is input data. This Definition neither constructs it nor asserts that a scheme supplies it.

> **Status.** This record is algebraic and does not itself certify geometric provenance. For a scheme, constructing it requires separately: a separated finite-type structural morphism; the compact-support objects H_c^i; finite-dimensionality and a finite vanishing bound; geometric Frobenius automorphisms with displayed bases; and the trace-formula proof for every positive iterate. These are exactly the still-open obligations OBL-TRR-03–OBL-TRR-05; none is a field silently constructed in this chapter. The concrete point-interface obligation is supplied by the TraceCore3 adapter below.

Lemma 5.8.9 (Every-iterate trace identity supplied by the datum). *Uses:* def:weil-finite-cohomological-trace-datum, def:weil-trace-formula-at-iterate

Lean: Weil.Trace.FiniteCohomologicalTraceData.traceFormulaAt

For every $n \geq 1$,

$$(N_n(P) : K) = \sum_{i=0}^{B} (-1)^i \operatorname{tr}_\Sigma(M_{D_i}^n).$$

Proof. Uses: def:weil-finite-cohomological-trace-datum, def:weil-trace-formula-at-iterate

Project the field t_n from the supplied finite cohomological trace datum and unfold TraceFormulaAt(P, G, n). □

Lemma 5.8.10 (Counting logarithm equals the alternating trace logarithm). *Uses:* def:weil-finite-cohomological-trace-datum, def:weil-coefficient-counting-log-series, def:weil-alternating-trace-log-series

Lean: Weil.Trace.FiniteCohomologicalTraceData.countingLog_eq_traceLog

For a finite cohomological trace datum,

$$S_{P,K} = S_G.$$

Proof. Uses: def:weil-coefficient-counting-log-series, def:weil-alternating-trace-log-series, lemma:weil-finite-cohomological-trace-at-iterate

The constant coefficients are both zero. For $n \geq 1$, divide the every-iterate trace identity by the nonzero scalar $n \in K$. The two resulting sides are exactly the degree-n coefficients of $S_{P,K}$ and S_G. Coefficient extensionality proves the series equality. □

Lemma 5.8.11 (Counting zeta equals the alternating determinant). *Uses:* def:weil-finite-cohomological-trace-datum, def:weil-coefficient-counting-zeta-series, def:weil-alternating-determinant-product

Lean: Weil.Trace.FiniteCohomologicalTraceData.zeta_eq_alternatingDet

For every finite cohomological trace datum,

$$Z_{P,K}(T) = \Delta_G(T).$$

Proof. Uses: def:weil-coefficient-counting-zeta-series, lemma:weil-counting-log-eq-alternating-trace-log, cor:weil-alternating-determinant-trace

Unfold $Z_{P,K} = \operatorname{Exp}(S_{P,K})$, replace its exponent by S_G, and apply the finite alternating determinant–trace identity. □

Proposition 5.8.12 (Cohomological factorization from a supplied trace datum). *Uses:* def:weil-finite-cohomological-trace-datum, def:weil-coefficient-counting-zeta-series, def:weil-alternating-determinant-product, def:weil-graded-reciprocal-factor

Lean: Weil.Trace.FiniteCohomologicalTraceData.zeta_factorization

For every finite cohomological trace datum,

$$Z_{P,K}(T) = \Delta_G(T) = \prod_{i=0}^{B} P_{G,i}(T)^{(-1)^{i+1}} \quad \text{in } K[[T]],$$

where a negative exponent means the geometric inverse of the corresponding constant-one polynomial series.

Proof. Uses: def:weil-alternating-determinant-product, def:weil-signed-graded-determinant-factor, lemma:weil-zeta-eq-alternating-determinant-from-trace-datum

The first equality is the preceding lemma. Unfold the finite product Δ_G and its signed degree factors once; even indices give the geometric inverse and odd indices give the polynomial series, which is the displayed exponent notation. □

Lemma 5.8.13 (Counting zeta is the Taylor series of the canonical germ). *Uses:* def:weil-finite-cohomological-trace-datum, def:weil-coefficient-counting-zeta-series, def:weil-graded-canonical-rational-germ, def:weil-regular-rational-germ-taylor-map

Lean: Weil.Trace.FiniteCohomologicalTraceData.zeta_eq_taylor

For every finite cohomological trace datum,

$$Z_{P,K} = \tau_K(\mathfrak{z}_G).$$

Proof. Uses: `lemma:weil-zeta-eq-alternating-determinant-from-trace-datum`, `lemma:weil-graded-canonical-rational-germ-taylor`

Both sides are equal to the same previously defined series Δ_G; compose those two equalities. □

Corollary 5.8.14 (Conditional cohomological rationality over the coefficient field)**.** *Uses:* *def: weil-finite-cohomological-trace-datum*, *def: weil-coefficient-counting-zeta-series*, *def: weil-rational-formal-series-at-zero*, *def: weil-graded-canonical-rational-germ*, *def: weil-regular-rational-germ-taylor-map*, *def: weil-alternating-rational-numerator*, *def: weil-alternating-rational-denominator*

Lean: *Weil.Trace.FiniteCohomologicalTraceData.zeta_rational*

For every finite cohomological trace datum,

$$Z_{P,K} = \tau_K(\mathfrak{z}_G), \qquad \rho_K(\mathfrak{z}_G) = \frac{N_G(T)}{D_G(T)} \in K(T).$$

Consequently $Z_{P,K}$ is rational at zero over K.

Proof. Uses: `def:weil-rational-formal-series-at-zero`, `lemma:weil-zeta-eq-canonical-germ-taylor-from-trace-datum`, `cor:weil-alternating-determinant-rational`

The preceding lemma supplies the displayed Taylor equality and therefore the explicit range witness required by rationality at zero. The rational function value is the one already computed for the same canonical germ in the alternating-determinant rationality corollary. □

Corollary 5.8.15 (Compatibility with the original point-counting zeta series)**.** *Uses:* *def: weil-finite-cohomological-trace-datum*, *def: weil-finite-extension-counting-zeta-series*, *def: weil-rational-power-series-scalar-extension*, *def: weil-graded-canonical-rational-germ*, *def: weil-regular-rational-germ-taylor-map*, *def: weil-graded-reciprocal-factor*

Lean: *Weil.Trace.FiniteCohomologicalTraceData.map_countingZeta_eq_taylor*

For every finite cohomological trace datum,

$$\mathrm{map}_{\mathbf{Q},K}(Z_P) = \tau_K(\mathfrak{z}_G) = \prod_{i=0}^{B} P_{G,i}^{(-1)^{i+1}}.$$

Proof. Uses: `lemma:weil-coefficient-counting-zeta-scalar-extension`, `prop:weil-zeta-cohomological-factorization-from-trace-datum`, `lemma:weil-graded-canonical-rational-germ-taylor`

The scalar-extension lemma identifies $\mathrm{map}_{\mathbf{Q},K}(Z_P)$ with $Z_{P,K}$. The factorization proposition identifies this with Δ_G, and the Taylor lemma identifies Δ_G with $\tau_K(\mathfrak{z}_G)$. The same factorization proposition supplies the final alternating product. □

> **Status.** Dependency barrier. The preceding three conditional statements may be instantiated only after a finite cohomological trace datum has actually been constructed. At the current foundation boundary, no such datum is available for a nonempty separated finite-type scheme. In particular, OBL-TRACE-FORMULA is not cited as a theorem, and rationality over $\mathbf{Q}$ is not inferred from rationality over $K = \mathbf{Q}_\ell$. Scalar descent belongs to the later module A05/I00; integrality and independence of the individual $P_{G,i}$ belong to I04–I06.

Chapter 6

Duality and the functional equation

Status. Geometric Poincare duality is not present in the pinned mathlib foundation. This chapter therefore separates two layers. First it proves the complete finite-dimensional linear algebra of a perfect Frobenius similitude pairing. It then defines, in order, the exceptional-adjunction, purity, cup-product, trace, compact-support, and proper-support candidates whose eventual construction would instantiate that algebra. Every statement about those candidates is conditional on explicit inverse and equivariance certificates. The open geometric obligations are itemized in the module manifest; none is a usable theorem here.

6.1 Perfect pairings with an explicit inverse

Throughout this section $E = \mathbf{Q}_\ell$. All vector spaces are finite-dimensional over E, all displayed endomorphisms are invertible, and $V^\vee = \mathrm{Hom}_E(V, E)$.

Definition 6.1.1 (Bilinear pairing datum). Uses: `def:weil-coefficient-data`, `def:weil-frobenius-module`

Lean: `Weil.Duality.BilinearPairingData`

A bilinear pairing datum consists of Frobenius modules (V, u), (W, v), and an E-bilinear map

$$B : V \times W \longrightarrow E.$$

No nondegeneracy or Frobenius compatibility is included in this definition.

Definition 6.1.2 (Right adjoint map of a pairing). Uses: `def:weil-duality-bilinear-pairing-data`

Lean: `Weil.Duality.BilinearPairingData.adjointMap`

For a bilinear pairing datum, define the E-linear map

$$\theta_B : W \longrightarrow V^\vee, \qquad \theta_B(w)(x) := B(x, w).$$

Lemma 6.1.3 (Evaluation formula for the pairing adjoint). *Uses:* *`def:weil-duality-bilinear-pairing-data`, `def:weil-duality-pairing-adjoint-map`*

Lean: *`Weil.Duality.BilinearPairingData.adjointMap_apply`*

For $w \in W$ and $x \in V$, one has

$$\theta_B(w)(x) = B(x, w).$$

Proof. Uses: `def:weil-duality-pairing-adjoint-map`

This is the evaluation rule in the definition of θ_B. □

Definition 6.1.4 (Inverse candidate for the pairing adjoint). Uses: `def:weil-duality-pairing-adjoint-map`

Lean: `Weil.Duality.BilinearPairingData.AdjointInverseCandidate`

An inverse candidate for θ_B is a specified E-linear map

$$s_B : V^\vee \longrightarrow W.$$

This is input data and does not assert that an inverse exists.

Definition 6.1.5 (Left-inverse predicate for a pairing adjoint)**.** Uses: `def:weil-duality-pairing-adjoint-map`,`def:weil-duality-pairing-adjoint-inverse-candidate`

Lean: `Weil.Duality.BilinearPairingData.IsAdjointLeftInverse`

Define AdjLeftInv(s_B) to be the equation

$$s_B \circ \theta_B = \mathrm{id}_W .$$

Definition 6.1.6 (Right-inverse predicate for a pairing adjoint)**.** Uses: `def:weil-duality-pairing-adjoint-map`,`def:weil-duality-pairing-adjoint-inverse-candidate`

Lean: `Weil.Duality.BilinearPairingData.IsAdjointRightInverse`

Define AdjRightInv(s_B) to be the equation

$$\theta_B \circ s_B = \mathrm{id}_{V^\vee} .$$

Definition 6.1.7 (Perfectness certificate for a bilinear pairing)**.** Uses: `def:weil-duality-pairing-adjoint-inverse-candidate`,`def:weil-duality-pairing-adjoint-left-inverse-predicate`,`def:weil-duality-pairing-adjoint-right-inverse-predicate`

Lean: `Weil.Duality.BilinearPairingData.PerfectCertificate`

A perfectness certificate consists of an inverse candidate s_B, a proof of AdjLeftInv(s_B), and a proof of AdjRightInv(s_B). Thus the inverse map and both inverse equations are stored separately from the pairing.

Definition 6.1.8 (Linear equivalence constructed from a perfectness certificate)**.** Uses: `def:weil-duality-pairing-adjoint-map`,`def:weil-duality-perfect-pairing-certificate`

Lean: `Weil.Duality.BilinearPairingData.adjointEquiv`

Given a perfectness certificate, define

$$e_B : W \xrightarrow{\sim} V^\vee$$

to have forward map θ_B, inverse map s_B, and inverse fields the two equations stored in the certificate.

Lemma 6.1.9 (Forward map of the adjoint equivalence)**.** *Uses:* *`def:weil-duality-pairing-adjoint-equivalence`*

Lean: *`Weil.Duality.BilinearPairingData.adjointEquiv_apply`*

The forward linear map underlying e_B is θ_B.

Proof. Uses: `def:weil-duality-pairing-adjoint-equivalence`

The forward-map field of the constructed equivalence was defined to be θ_B. □

Lemma 6.1.10 (Inverse map of the adjoint equivalence)**.** *Uses:* *`def:weil-duality-pairing-adjoint-equivalence`*

Lean: *`Weil.Duality.BilinearPairingData.adjointEquiv_symm_apply`*

The inverse linear map underlying e_B is s_B.

Proof. Uses: `def:weil-duality-pairing-adjoint-equivalence`

The inverse-map field of the constructed equivalence was defined to be s_B. □

Lemma 6.1.11 (Dimension equality from perfectness)**.** *Uses:* *`def:weil-duality-bilinear-pairing-data`,`def:weil-duality-perfect-pairing-certificate`*

Lean: *`Weil.Duality.BilinearPairingData.finrank_eq`*

For a bilinear pairing supplied with a perfectness certificate,

$$\dim_E W = \dim_E V.$$

Proof. Uses: `def:weil-duality-pairing-adjoint-equivalence`,`lemma:weil-duality-pairing-adjoint-equivalence-forward`

The equivalence $e_B : W \simeq V^\vee$ preserves finite dimension. Mathlib's finite-dual dimension theorem gives $\dim_E(V^\vee) = \dim_E V$. Combining these two equalities gives the displayed formula. □

6.2 The Frobenius similitude calculation

Definition 6.2.1 (Scaled inverse-dual operator)**.** Uses: `def:weil-duality-bilinear-pairing-data`

Lean: `Weil.Duality.BilinearPairingData.scaledInverseDual`

For $\mu \in E^\times$, define

$$A_{u,\mu} : V^\vee \longrightarrow V^\vee, \qquad A_{u,\mu} := \mu\,(u^{-1})^\vee.$$

Definition 6.2.2 (Frobenius similitude predicate)**.** Uses: `def:weil-duality-bilinear-pairing-data`

Lean: `Weil.Duality.BilinearPairingData.IsSimilitude`

For $\mu \in E^\times$, define $\mathrm{IsSim}_\mu(B)$ to mean

$$B(ux, vw) = \mu B(x, w)$$

for every $x \in V$ and $w \in W$.

Lemma 6.2.3 (Intertwining formula for the adjoint map)**.** *Uses:* `def:weil-duality-pairing-adjoint-map`, `def:weil-duality-scaled-inverse-dual-operator`, `def:weil-duality-frobenius-similitude-predicate`

Lean: `Weil.Duality.BilinearPairingData.adjointMap_comp_operator`

If $\mathrm{IsSim}_\mu(B)$, *then*

$$\theta_B \circ v = A_{u,\mu} \circ \theta_B.$$

Proof. Uses: `def:weil-duality-pairing-adjoint-map`, `def:weil-duality-scaled-inverse-dual-operator`, `def:weil-duality-frobenius-similitude-predicate`, `lemma:weil-duality-pairing-adjoint-apply`

Evaluate both sides at $w \in W$ and then at $x \in V$. The left side is $B(x, vw)$. In the similitude equation replace its first input by $u^{-1}x$. Since $u(u^{-1}x) = x$, that equation becomes

$$B(x, vw) = \mu B(u^{-1}x, w) = \big(A_{u,\mu}\theta_B(w)\big)(x).$$

Equality for every x, w is equality of the two linear maps. □

Definition 6.2.4 (Conjugate operator candidate attached to a perfect pairing)**.** Uses: `def:weil-duality-pairing-adjoint-map`, `def:weil-duality-perfect-pairing-certificate`, `def:weil-duality-scaled-inverse-dual-operator`

Lean: `Weil.Duality.BilinearPairingData.pairedConjugateOperator`

Given a perfectness certificate, define the endomorphism of W

$$c_{B,\mu} := s_B \circ A_{u,\mu} \circ \theta_B.$$

Lemma 6.2.5 (The paired operator is the constructed conjugate)**.** *Uses:* `def:weil-duality-perfect-pairing-certificate`, `def:weil-duality-frobenius-similitude-predicate`, `def:weil-duality-paired-conjugate-operator`

Lean: `Weil.Duality.BilinearPairingData.operator_eq_pairedConjugate`

If $\mathrm{IsSim}_\mu(B)$, *then*

$$v = c_{B,\mu} = e_B^{-1} A_{u,\mu} e_B.$$

Proof. Uses: `def:weil-duality-pairing-adjoint-left-inverse-predicate`, `def:weil-duality-perfect-pairing-certificate`, `def:weil-duality-paired-conjugate-operator`, `lemma:weil-duality-adjoint-intertwining`, `lemma:weil-duality-pairing-adjoint-equivalence-forward`, `lemma:weil-duality-pairing-adjoint-equivalence-inverse`

Compose the equality $\theta_B v = A_{u,\mu}\theta_B$ on the left with s_B. The stored left-inverse equation reduces the left side to v, while the right side is the defining composite for $c_{B,\mu}$. The two projection lemmas for e_B rewrite that composite as $e_B^{-1} A_{u,\mu} e_B$. □

Lemma 6.2.6 (Determinant of a dual endomorphism)**.** *Uses:* `def:weil-frobenius-module`

Lean: `LinearMap.det_dualMap`

For an endomorphism f of a finite-dimensional E-vector space,

$$\det(f^\vee) = \det(f).$$

Library status: verified only after the pinned-contract audit.

Proof. Uses: `def:weil-frobenius-module`
Instantiate `LinearMap.det_dualMap` with the coefficient field E, the finite-dimensional space, and f. Its conclusion is exactly the displayed equality. □

Lemma 6.2.7 (Determinant of an inverse endomorphism). *Uses:* `def:weil-frobenius-module`
Lean: `LinearEquiv.det_coe_symm`
For an invertible endomorphism f,

$$\det(f^{-1}) = \det(f)^{-1}.$$

Library status: verified only after the pinned-contract audit.

Proof. Uses: `def:weil-frobenius-module`
Regard f as a linear equivalence and instantiate `LinearEquiv.det_coe_symm`. The underlying map of the symmetric linear equivalence is f^{-1}, so the theorem's two sides are the two displayed determinants. □

Lemma 6.2.8 (Determinant of a scalar multiple). *Uses:* `def:weil-frobenius-module`
Lean: `LinearMap.det_smul`
If f acts on an r-dimensional E-vector space and $a \in E$, then

$$\det(af) = a^r \det(f).$$

Library status: verified only after the pinned-contract audit.

Proof. Uses: `def:weil-frobenius-module`
Apply `LinearMap.det_smul` to a and f. Its exponent is the finite rank of the module, which is r by the displayed dimension hypothesis. □

Lemma 6.2.9 (Determinant is invariant under the adjoint equivalence). *Uses:* `def:weil-duality-pairing-adjoint-equivalence`, `def:weil-duality-paired-conjugate-operator`
Lean: `LinearMap.det_conj`
The determinant of $c_{B,\mu}$ equals the determinant of $A_{u,\mu}$.

Proof. Uses: `def:weil-duality-paired-conjugate-operator`, `lemma:weil-duality-pairing-adjoint-equivalence-forward`, `lemma:weil-duality-pairing-adjoint-equivalence-inverse`
The defining expression is $c_{B,\mu} = e_B^{-1} \circ A_{u,\mu} \circ e_B$. In `LinearMap.det_conj`, take $f = A_{u,\mu}$ on $V^\vee$ and take the displayed equivalence parameter to be $e = e_B^{-1} : V^\vee \simeq W$. Its left-hand side is then exactly $\det(c_{B,\mu})$, and its right-hand side is $\det(A_{u,\mu})$. □

Proposition 6.2.10 (Determinants in a perfect similitude pair). *Uses:* `def:weil-duality-bilinear-pairing-data`, `def:weil-duality-perfect-pairing-certificate`, `def:weil-duality-frobenius-similitude-predicate`
Lean: `Weil.Duality.BilinearPairingData.det_mul_det`
Let $r = \dim_E V$. If the pairing has a perfectness certificate and satisfies $\mathrm{IsSim}_\mu(B)$, then

$$\det(u)\det(v) = \mu^r.$$

Proof. Uses: `lemma:weil-duality-perfect-pairing-dimension`, `lemma:weil-duality-paired-operator-conjugacy`, `lemma:weil-duality-determinant-dual`, `lemma:weil-duality-determinant-inverse`, `lemma:weil-duality-determinant-scalar`, `lemma:weil-duality-determinant-conjugacy`
Conjugacy gives

$$\det(v) = \det\big(\mu(u^{-1})^\vee\big).$$

The scalar determinant formula contributes μ^r; the dual formula removes the dual, and the inverse formula replaces the remaining determinant by $\det(u)^{-1}$. Thus $\det(v) = \mu^r \det(u)^{-1}$. Multiplication by the nonzero scalar $\det(u)$ gives the result. □

Definition 6.2.11 (Pinned finite basis). Uses: `def:weil-frobenius-module`
Lean: `Module.finBasis`
For a finite-dimensional Frobenius module (V, u), define

$$\beta_V := \texttt{Module.finBasis}\ E\ V : \mathrm{Basis}(\mathrm{Fin}(\dim_E V), E, V).$$

This is the explicit noncomputable basis value returned by the pinned mathlib definition `Module.finBasis`; later proofs only unfold this value.

Definition 6.2.12 (Coordinate matrix in the pinned basis). Uses: def:weil-duality-canonical-coordinate-basis

Lean: Weil.Duality.coordinateMatrix

For an endomorphism $f : V \to V$, define

$$[f]_{\beta_V} := \texttt{LinearMap.toMatrix}\ \beta_V\ \beta_V\ f.$$

Lemma 6.2.13 (Coordinate matrix respects composition). Uses: def:weil-duality-coordinate-matrix

Lean: LinearMap.toMatrix_comp

For endomorphisms f, g of V,

$$[f \circ g]_{\beta_V} = [f]_{\beta_V}[g]_{\beta_V}.$$

Proof. Uses: def:weil-duality-canonical-coordinate-basis, def:weil-duality-coordinate-matrix

Instantiate LinearMap.toMatrix_comp with all three bases equal to β_V, first map f, and second map g. Its conclusion is the displayed matrix equality. □

Lemma 6.2.14 (Coordinate determinant agrees with intrinsic determinant). Uses: def:weil-duality-coordinate-matrix

Lean: LinearMap.det_toMatrix

For every endomorphism f of V,

$$\det([f]_{\beta_V}) = \det(f).$$

Proof. Uses: def:weil-duality-canonical-coordinate-basis, def:weil-duality-coordinate-matrix

Instantiate LinearMap.det_toMatrix with $b = \beta_V$ and the endomorphism f. This is exactly the displayed equality. □

Definition 6.2.15 (Laurent coefficient ring). Uses: def:weil-coefficient-data

Lean: LaurentPolynomial

Define

$$R_E := \mathrm{LaurentPolynomial}(E) = E[T, T^{-1}].$$

Definition 6.2.16 (Laurent coefficient embedding). Uses: def:weil-duality-laurent-coefficient-ring

Lean: LaurentPolynomial.C

Define the coefficient ring homomorphism

$$\iota_E : E \longrightarrow R_E, \qquad \iota_E := \texttt{LaurentPolynomial.C}.$$

Definition 6.2.17 (Laurent variable). Uses: def:weil-duality-laurent-coefficient-ring

Lean: LaurentPolynomial.T

Define the Laurent variable and its displayed inverse candidate by

$$\tau := \texttt{LaurentPolynomial.T}(1), \qquad \tau^{-1,\mathrm{cand}} := \texttt{LaurentPolynomial.T}(-1).$$

Lemma 6.2.18 (The displayed Laurent inverse is two-sided). Uses: def:weil-duality-laurent-variable

Lean: LaurentPolynomial.isUnit_T

One has

$$\tau\tau^{-1,\mathrm{cand}} = 1, \qquad \tau^{-1,\mathrm{cand}}\tau = 1.$$

Proof. Uses: def:weil-duality-laurent-variable

The two exact mathlib constants are displayed separately:

$$\texttt{LaurentPolynomial.T_add},$$
$$\texttt{LaurentPolynomial.T_zero}.$$

Abbreviate them by T_add and T_zero. Their exact instantiations are

$$\begin{aligned} \tau\tau^{-1,\mathrm{cand}} &= \texttt{T}(1)\texttt{T}(-1) = \texttt{T}(1 + (-1)) && \text{by } (\texttt{T_add}\ 1\ -1)^{-1} \\ &= \texttt{T}(0) = 1 && \text{by } \texttt{T_zero}, \end{aligned}$$

and

$$\begin{aligned}\tau^{-1,\mathrm{cand}}\tau = \mathtt{T}(-1)\mathtt{T}(1) = \mathtt{T}((-1)+1) \quad & \text{by } (\mathtt{T_add}\ -1\ 1)^{-1}\\ = \mathtt{T}(0) = 1 \quad & \text{by } \mathtt{T_zero}.\end{aligned}$$

□

Definition 6.2.19 (Laurent extension of a coordinate matrix)**.** Uses: `def:weil-duality-coordinate-matrix`, `def:weil-duality-laurent-coefficient-embedding`

Lean: `Weil.Duality.laurentCoordinateMatrix`

For an endomorphism f, define

$$[f]_{\beta_V,R_E} := [f]_{\beta_V}.\mathrm{map}(\iota_E).$$

Lemma 6.2.20 (Determinant after Laurent coefficient extension)**.** *Uses:* *def:weil-duality-laurent-coordinate-matrix*

Lean: *RingHom.map_det*

For every endomorphism f,

$$\det([f]_{\beta_V,R_E}) = \iota_E(\det f).$$

Proof. Uses: `def:weil-duality-laurent-coefficient-embedding`, `def:weil-duality-laurent-coordinate-matrix`, `lemma:weil-duality-coordinate-matrix-determinant`

Apply `RingHom.map_det` to ι_E and $[f]_{\beta_V}$. Its right side is the left side displayed above. Its left side is $\iota_E(\det([f]_{\beta_V}))$, which the coordinate determinant lemma rewrites to $\iota_E(\det f)$. □

Lemma 6.2.21 (Inverse reciprocal-determinant identity)**.** *Uses:* *def:weil-frobenius-module*, *def:weil-reciprocal-characteristic-polynomial*

Lean: *Weil.Duality.inverse_reciprocalCharpoly*

If u acts on an r-dimensional space and $D = \det(u)$, then in $E[T,T^{-1}]$

$$\det(1 - T^{-1}u^{-1}) = (-1)^r T^{-r} D^{-1} \det(1 - Tu).$$

Proof. Uses: `def:weil-reciprocal-characteristic-polynomial`, `def:weil-duality-laurent-variable`, `def:weil-duality-laurent-coordinate-matrix`, `lemma:weil-duality-coordinate-matrix-composition`, `lemma:weil-duality-laurent-variable-inverse`, `lemma:weil-duality-laurent-coordinate-determinant`

Apply the coordinate-composition lemma to $u \circ u^{-1} = 1$ and to $u^{-1} \circ u = 1$, then apply the Laurent coefficient embedding entrywise. The two resulting equations say that $[u^{-1}]_{\beta_V,R_E}$ is a two-sided inverse of $[u]_{\beta_V,R_E}$. Write these already defined matrices as M^- and M, respectively. Entrywise multiplication, using the two explicit inverse equations for τ, gives

$$I - \tau M = (-\tau M)(I - \tau^{-1,\mathrm{cand}} M^-).$$

Apply `Matrix.det_mul` to this equality. Apply `Matrix.det_smul` with matrix argument M and scalar argument $-\tau$. The result is

$$\det(I - \tau M) = (-\tau)^r \det(M) \det(I - \tau^{-1,\mathrm{cand}} M^-).$$

The Laurent-coordinate determinant lemma replaces $\det(M)$ by $\iota_E(D)$. Unfolding the reciprocal-characteristic-polynomial definition and applying `RingHom.map_det` identifies the other two determinants with the Laurent images under ι_E of $\det(1-Tu)$ and $\det(1-T^{-1}u^{-1})$, respectively. The displayed inverse equations make τ a unit, and D is a unit because u is invertible. Multiply by their explicit inverses. Since $(-\tau)^{-r} = (-1)^r \tau^{-r}$, the result is exactly the asserted Laurent-polynomial identity. □

Proposition 6.2.22 (Degreewise reciprocal characteristic identity)**.** *Uses:* *def:weil-duality-bilinear-pairing-data*, *def:weil-duality-perfect-pairing-certificate*, *def:weil-duality-frobenius-similitude-predicate*, *def:weil-reciprocal-characteristic-polynomial*

Lean: *Weil.Duality.BilinearPairingData.reciprocalCharpoly_pair*

Let $r = \dim_E V$, $D = \det(u)$, and $P_V(T) = \det(1 - Tu)$, $P_W(T) = \det(1 - Tv)$. Under perfectness and $\mathrm{IsSim}_\mu(B)$*,*

$$P_W(\mu^{-1}T^{-1}) = (-1)^r T^{-r} D^{-1} P_V(T)$$

in $E[T,T^{-1}]$.

Proof. Uses: lemma:weil-duality-paired-operator-conjugacy,lemma:weil-duality-determinant-dual, lemma:weil-duality-determinant-conjugacy,lemma:weil-duality-inverse-reciprocal-determinant

Replace v by its conjugate $\mu(u^{-1})^{\vee}$. Determinant invariance under conjugacy gives

$$\begin{aligned} P_W(\mu^{-1}T^{-1}) &= \det\bigl(1 - T^{-1}(u^{-1})^{\vee}\bigr) \\ &= \det(1 - T^{-1}u^{-1}). \end{aligned}$$

The second equality is the determinant-of-dual identity applied to $1 - T^{-1}u^{-1}$. The inverse reciprocal-determinant lemma evaluates the last expression as the required Laurent polynomial. □

Definition 6.2.23 (Reciprocal-eigenvalue presentation candidate)**.** Uses: def:weil-coefficient-data,def:weil-frobenius-module

Lean: Weil.Duality.ReciprocalEigenvaluePresentationCandidate

After scalar extension along the fixed embedding $E \hookrightarrow \overline{E}$, a reciprocal-eigenvalue presentation candidate for u is an explicitly supplied finite multiset

$$\mathcal{A}_u \in \mathrm{Multiset}(\overline{E}^{\times}).$$

Membership in $\overline{E}^{\times}$, rather than merely $\overline{E}$, stores a nonzero proof for every entry. This definition neither chooses roots nor asserts a factorization.

Definition 6.2.24 (Cardinality predicate for an eigenvalue presentation)**.** Uses: def:weil-duality-reciprocal-eigenvalue-presentation-candidate

Lean: Weil.Duality.ReciprocalEigenvaluePresentationCandidate.HasCardinality

Define HasEigCard($\mathcal{A}_u$) to be the equation

$$\mathrm{card}(\mathcal{A}_u) = \dim_E V.$$

Definition 6.2.25 (Factorization predicate for an eigenvalue presentation)**.** Uses: def:weil-reciprocal-characteristic-polynomial,def:weil-duality-reciprocal-eigenvalue-presentation-candidate

Lean: Weil.Duality.ReciprocalEigenvaluePresentationCandidate.Factors

Let $P_{V,\overline{E}}$ be the coefficientwise image of P_V in $\overline{E}[T]$. Define FactorsEig($\mathcal{A}_u$) to be the polynomial equation

$$P_{V,\overline{E}}(T) = \prod_{\alpha \in \mathcal{A}_u} (1 - \alpha T),$$

where a unit α is coerced to its value in $\overline{E}$.

Definition 6.2.26 (Eigenvalue-presentation certificate)**.** Uses: def:weil-duality-reciprocal-eigenvalue-presentation-candidate,def:weil-duality-eigenvalue-presentation-cardinality-predicate,def:weil-duality-eigenvalue-presentation-factorization-predicate

Lean: Weil.Duality.ReciprocalEigenvaluePresentationCandidate.Certificate

An eigenvalue-presentation certificate consists of a proof of HasEigCard($\mathcal{A}_u$) and a proof of FactorsEig($\mathcal{A}_u$). The candidate, cardinality equation, and factorization equation are therefore separate fields; no existence assertion occurs in this certificate definition.

Definition 6.2.27 (Paired eigenvalue-presentation candidate)**.** Uses: def:weil-duality-bilinear-pairing-data,def:weil-duality-reciprocal-eigenvalue-presentation-candidate

Lean: Weil.Duality.pairedEigenvaluePresentationCandidate

For $\mu \in E^{\times}$ and a candidate $\mathcal{A}_u$, define the explicit multiset

$$\mathcal{A}_{B,\mu}^{\mathrm{pair}} := \mathcal{A}_u.\,\mathrm{map}\bigl(\alpha \longmapsto \mu/\alpha\bigr) \in \mathrm{Multiset}(\overline{E}^{\times}).$$

The division takes place in the unit group, so nonzeroness of every output is part of the constructed output type.

Lemma 6.2.28 (Cardinality of the paired candidate)**.** *Uses:* *def:weil-duality-reciprocal-eigenvalue-presentation-candidate, def:weil-duality-paired-eigenvalue-presentation-candidate*

Lean: *Weil.Duality.pairedEigenvaluePresentationCandidate_card*

One has

$$\mathrm{card}(\mathcal{A}_{B,\mu}^{\mathrm{pair}}) = \mathrm{card}(\mathcal{A}_u).$$

Proof. Uses: `def:weil-duality-paired-eigenvalue-presentation-candidate`

Unfold the mapped candidate and instantiate `Multiset.card_map` with the function $\alpha \mapsto \mu/\alpha$. Its conclusion is the displayed equation. □

Lemma 6.2.29 (Product in a certified eigenvalue presentation). *Uses:* *def:weil-frobenius-module, def:weil-duality-reciprocal-eigenvalue-presentation-candidate, def:weil-duality-eigenvalue-presentation-certificate*

Lean: *Weil.Duality.product_certifiedReciprocalEigenvalues*

For a certified candidate, in $\overline{E}$,

$$\prod_{\alpha \in \mathcal{A}_u} \alpha = \det(u).$$

Proof. Uses: `def:weil-reciprocal-characteristic-polynomial,def:weil-duality-eigenvalue-presentation-cardinality-predicate,def:weil-duality-eigenvalue-presentation-factorization-predicate,def:weil-duality-eigenvalue-presentation-certificate,def:weil-duality-canonical-coordinate-basis,def:weil-duality-coordinate-matrix,lemma:weil-duality-coordinate-matrix-determinant`

Put $r = \dim_E V$. The stored cardinality equation says that the product has exactly r factors. In the stored factorization, the coefficient of T^r on the product side is $(-1)^r \prod_{\alpha \in \mathcal{A}_u} \alpha$: the degree-$r$ term is obtained by taking the linear term from every factor.

Compute the same coefficient from $P_V(T) = \det(I - Tu)$ using the already defined coordinate matrix. In the Leibniz determinant formula, a term has degree r only when every row contributes its $-T[u]_{\beta_V}$-entry. The sum of these terms is $\det(-[u]_{\beta_V}) = (-1)^r \det([u]_{\beta_V})$ by `Matrix.det_smul`; the coordinate-determinant lemma changes the last factor to $\det(u)$. Equality of the stored polynomial factorization's top coefficients and cancellation of the unit $(-1)^r$ prove the formula. □

Lemma 6.2.30 (Cardinality predicate for the paired presentation). *Uses:* *def:weil-duality-perfect-pairing-certificate, def:weil-duality-reciprocal-eigenvalue-presentation-candidate, def:weil-duality-eigenvalue-presentation-certificate, def:weil-duality-paired-eigenvalue-presentation-candidate*

Lean: *Weil.Duality.pairedEigenvaluePresentationCandidate_hasCardinality*

If $\mathcal{A}_u$ has a certificate, then

$$\operatorname{card}(\mathcal{A}^{\mathrm{pair}}_{B,\mu}) = \dim_E W.$$

Proof. Uses: `def:weil-duality-eigenvalue-presentation-cardinality-predicate,def:weil-duality-eigenvalue-presentation-certificate,lemma:weil-duality-paired-eigenvalue-candidate-card,lemma:weil-duality-perfect-pairing-dimension`

The map-cardinality lemma gives the cardinality of $\mathcal{A}_u$. The stored cardinality field changes this to $\dim_E V$, and perfectness changes $\dim_E V$ to $\dim_E W$. □

Lemma 6.2.31 (Factorization predicate for the paired presentation). *Uses:* *def:weil-duality-perfect-pairing-certificate, def:weil-duality-frobenius-similitude-predicate, def:weil-duality-eigenvalue-presentation-certificate, def:weil-duality-paired-eigenvalue-presentation-candidate*

Lean: *Weil.Duality.pairedEigenvaluePresentationCandidate_factors*

Under perfectness and $\mathrm{IsSim}_\mu(B)$, if $\mathcal{A}_u$ has a certificate, then

$$P_{W,\overline{E}}(S) = \prod_{\alpha \in \mathcal{A}_u} \bigl(1 - (\mu/\alpha)S\bigr).$$

Proof. Uses: `def:weil-duality-eigenvalue-presentation-factorization-predicate,def:weil-duality-eigenvalue-presentation-cardinality-predicate,def:weil-duality-eigenvalue-presentation-certificate,def:weil-duality-paired-eigenvalue-presentation-candidate,lemma:weil-duality-eigenvalue-product-determinant,prop:weil-duality-paired-reciprocal-characteristic`

Let $r = \operatorname{card}(\mathcal{A}_u)$ and $D = \det(u)$. The stored cardinality field identifies this r with $\dim_E V$. The stored factorization and the product lemma then turn the paired reciprocal-characteristic identity into

$$P_W(\mu^{-1}T^{-1}) = (-1)^r T^{-r} \left(\prod_{\alpha \in \mathcal{A}_u} \alpha \right)^{-1} \prod_{\alpha \in \mathcal{A}_u} (1 - \alpha T).$$

For each unit α, factor $1-\alpha T = -\alpha T(1-\alpha^{-1}T^{-1})$. Apply `Multiset.map_congr` to these pointwise equalities and then apply `congrArg Multiset.prod`. Rewrite the product of the two pointwise factors with `Multiset.prod_map_mul`. Rewrite each constant-valued map with `Multiset.map_const'` and its product with `Multiset.prod_replicate`. The stored cardinality equation changes the resulting exponent to r. Commutativity then cancels the factors $(-1)^r T^r \prod \alpha$ and gives

$$P_W(\mu^{-1}T^{-1}) = \prod_{\alpha \in \mathcal{A}_u} (1-\alpha^{-1}T^{-1}).$$

Set $S = \mu^{-1}T^{-1}$; equivalently $T^{-1} = \mu S$. Replacing the right side term by term gives $1-\alpha^{-1}T^{-1} = 1-(\mu/\alpha)S$, proving exactly the displayed factorization. This proves a polynomial equation for the explicitly mapped candidate; it does not infer a multiset merely from equality of polynomials. □

Proposition 6.2.32 (Certified paired reciprocal-eigenvalue presentation). *Uses:* *def:weil-duality-perfect-pairing-certificate, def:weil-duality-frobenius-similitude-predicate, def:weil-duality-eigenvalue-presentation-certificate, def:weil-duality-paired-eigenvalue-presentation-candidate*

Lean: *Weil.Duality.pairedEigenvaluePresentationCandidate_certificate*

Under the hypotheses of the preceding lemma, the explicitly defined $\mathcal{A}_{B,\mu}^{\mathrm{pair}}$ satisfies both its cardinality predicate for W and its factorization predicate for P_W.

Proof. Uses: `lemma:weil-duality-paired-eigenvalue-cardinality,lemma:weil-duality-paired-eigenvalue-factorization`

The first cited lemma is the required cardinality field. The second cited lemma is the required factorization field. Pair these two proof terms; this operation constructs no additional data. □

Definition 6.2.33 (Swap-sign predicate for a self-pairing). Uses: `def:weil-duality-bilinear-pairing-data`

Lean: `Weil.Duality.BilinearPairingData.HasSwapSign`

When $W = V$, and $s \in \{1,-1\}$, define

$$\mathrm{HasSwapSign}_s(B) \quad \Longleftrightarrow \quad B(y,x) = sB(x,y)$$

for every $x, y \in V$.

Lemma 6.2.34 (Odd skew-perfect spaces have even dimension). *Uses:* *def:weil-duality-perfect-pairing-certificate, def:weil-duality-pairing-swap-sign-predicate*

Lean: *Weil.Duality.BilinearPairingData.even_finrank_of_skew*

If B has a perfectness certificate and $\mathrm{HasSwapSign}_{-1}(B)$, then $\dim_E V$ is even.

Proof. Uses: `def:weil-duality-pairing-adjoint-equivalence,def:weil-duality-pairing-swap-sign-predicate`

Let $r = \dim_E V$ and represent θ_B in the same finite basis on the source and the dual basis on the target. The swap-sign equation says that the representing matrix A satisfies $A^{\mathrm{t}} = -A$. The adjoint equivalence makes A invertible, hence $\det A \neq 0$. On the other hand,

$$\det A = \det(A^{\mathrm{t}}) = \det(-A) = (-1)^r \det A.$$

The first equality is `Matrix.det_transpose`; the last is `Matrix.det_smul`. If r were odd, subtraction would give $2\det A = 0$, impossible in the characteristic-zero field E. Therefore r is even. □

6.3 Conditional geometric construction of the Poincare pairing

The definitions in this section expose the maps which the geometric proof must construct. A word such as "candidate" means that the object is supplied as input; it does not certify the corresponding open obligation.

Definition 6.3.1 (Exceptional adjunction candidate for a structural morphism). Uses: `def:weil-coefficient-data,def:weil-six-functor-typed-context`

Lean: `Weil.Duality.ExceptionalAdjunctionCandidate`

Fix a supplied typed six-functor context, and let $a_X : X \to \operatorname{Spec} k$ be a separated finite-type structural morphism. An exceptional adjunction candidate consists of functors

$$Ra_{X!} : D_c^b(X,E) \longrightarrow D_c^b(k,E), \qquad Ra_X^! : D_c^b(k,E) \longrightarrow D_c^b(X,E),$$

together with specified unit and counit maps, including

$$\epsilon_{a_X} : Ra_{X!}Ra_X^! E \longrightarrow E.$$

It is a data type: adjunction, triangle identities, constructibility, and Frobenius compatibility are separate predicates and obligations.

Definition 6.3.2 (Dualizing-object candidate). Uses: `def:weil-duality-exceptional-adjunction-candidate`

Lean: `Weil.Duality.dualizingObjectCandidate`

For an exceptional adjunction candidate, define

$$\omega_X^{\mathrm{cand}} := Ra_X^! E.$$

Definition 6.3.3 (Smooth-purity comparison candidate). Uses: `def:weil-tate-twist,def:weil-duality-dualizing-object-candidate`

Lean: `Weil.Duality.SmoothPurityComparisonCandidate`

If X is smooth of pure dimension d, a smooth-purity comparison candidate is a specified morphism

$$\mathrm{pur}_X : E_X(d)[2d] \longrightarrow \omega_X^{\mathrm{cand}}.$$

This definition supplies the morphism but does not assert that it is an isomorphism.

Definition 6.3.4 (Smooth-purity inverse candidate). Uses: `def:weil-duality-smooth-purity-comparison-candidate`

Lean: `Weil.Duality.SmoothPurityInverseCandidate`

A smooth-purity inverse candidate is a specified morphism

$$\mathrm{pur}_X^{-1,\mathrm{cand}} : \omega_X^{\mathrm{cand}} \longrightarrow E_X(d)[2d].$$

This definition supplies only a map in the reverse direction.

Definition 6.3.5 (Left-inverse predicate for smooth purity). Uses: `def:weil-duality-smooth-purity-comparison-candidate,def:weil-duality-smooth-purity-inverse-candidate`

Lean: `Weil.Duality.IsSmoothPurityLeftInverse`

Define $\mathrm{PurLeftInv}_X$ to be the equation

$$\mathrm{pur}_X^{-1,\mathrm{cand}} \circ \mathrm{pur}_X = \mathrm{id}_{E_X(d)[2d]}.$$

Definition 6.3.6 (Right-inverse predicate for smooth purity). Uses: `def:weil-duality-smooth-purity-comparison-candidate,def:weil-duality-smooth-purity-inverse-candidate`

Lean: `Weil.Duality.IsSmoothPurityRightInverse`

Define $\mathrm{PurRightInv}_X$ to be the equation

$$\mathrm{pur}_X \circ \mathrm{pur}_X^{-1,\mathrm{cand}} = \mathrm{id}_{\omega_X^{\mathrm{cand}}}.$$

Definition 6.3.7 (Smooth-purity isomorphism predicate). Uses: `def:weil-duality-smooth-purity-inverse-candidate,def:weil-duality-smooth-purity-left-inverse-predicate,def:weil-duality-smooth-purity-right-inverse-predicate`

Lean: `Weil.Duality.IsSmoothPurityIsomorphism`

Define $\mathrm{IsSmoothPurityIso}(\mathrm{pur}_X)$ to mean that the supplied inverse candidate satisfies $\mathrm{PurLeftInv}_X$ and $\mathrm{PurRightInv}_X$. Thus neither the reverse map nor either inverse equation is hidden inside an existential proposition.

Definition 6.3.8 (Derived trace candidate). Uses: `def:weil-duality-exceptional-adjunction-candidate,def:weil-duality-smooth-purity-comparison-candidate`

Lean: `Weil.Duality.derivedTraceCandidate`

Define the derived trace candidate to be the composite

$$Ra_{X!}\big(E_X(d)[2d]\big) \xrightarrow{Ra_{X!}(\mathrm{pur}_X)} Ra_{X!}Ra_X^! E \xrightarrow{\epsilon_{a_X}} E.$$

Definition 6.3.9 (Top-degree twisted trace candidate)**.** Uses: `def:weil-six-functor-typed-context,def:weil-duality-derived-trace-candidate`

Lean: `Weil.Duality.topTwistedTraceCandidate`

A top-degree twisted trace candidate contains the following separately specified data:

$$t_X^0 : H^0(Ra_{X!}(E_X(d)[2d])) \longrightarrow H^0(E) = E,$$

declared to be the degree-zero cohomology map of the derived trace candidate, and a shift-comparison map

$$s_X^{2d} : H_c^{2d}(X, E(d)) \longrightarrow H^0(Ra_{X!}(E_X(d)[2d])).$$

Define the output map by the literal composite

$$\mathrm{tr}_X^{(d)} := t_X^0 \circ s_X^{2d} : H_c^{2d}(X, E(d)) \longrightarrow E.$$

The construction and functoriality of t_X^0, and the construction, inverse, and shift-sign convention for s_X^{2d}, are the independent obligations OBL-DFE-TRACE-HZERO and OBL-DFE-TRACE-SHIFT; this definition does not assert either obligation.

Definition 6.3.10 (Untwisted top trace candidate)**.** Uses: `def:weil-tate-line,def:weil-tate-twist,def:weil-duality-top-twisted-trace-candidate`

Lean: `Weil.Duality.untwistedTopTraceCandidate`

An untwisted top trace candidate contains a specified cohomological twist equivalence

$$c_X^{(d)} : H_c^{2d}(X, E) \otimes_E E(d) \xrightarrow{\sim} H_c^{2d}(X, E(d))$$

and a specified Tate-line duality equivalence

$$\delta_d : \mathrm{Hom}_E(E(d), E) \xrightarrow{\sim} E(-d).$$

For $z \in H_c^{2d}(X, E)$, define

$$\mathrm{tr}_X(z) := \delta_d\Big(e \longmapsto \mathrm{tr}_X^{(d)}\big(c_X^{(d)}(z \otimes e)\big)\Big).$$

Thus the resulting map has the explicit type $\mathrm{tr}_X : H_c^{2d}(X, E) \to E(-d)$. The construction and inverse equations for $c_X^{(d)}$, and the evaluation/coevaluation construction and normalization of δ_d, are the independent obligations OBL-DFE-TRACE-TWIST and OBL-DFE-TRACE-TATE-DUAL.

Definition 6.3.11 (Compact–ordinary cup-product candidate)**.** Uses: `def:weil-coefficient-data,def:weil-six-functor-typed-context`

Lean: `Weil.Duality.compactOrdinaryCupCandidate`

A compact–ordinary cup-product candidate first contains the separately specified derived comparison and multiplication maps

$$\kappa_X^{\smile} : R\Gamma_c(X, E) \otimes_E^L R\Gamma(X, E) \longrightarrow R\Gamma_c(X, E \otimes_E^L E),$$

$$m_X^{\smile} : R\Gamma_c(X, E \otimes_E^L E) \longrightarrow R\Gamma_c(X, E).$$

For every $0 \le i \le 2d$, it also contains a specified degree-component map

$$\pi_{i,2d-i} : H_c^i(X, E) \otimes_E H^{2d-i}(X, E) \longrightarrow H^{2d}\big(R\Gamma_c(X, E) \otimes_E^L R\Gamma(X, E)\big).$$

Applying this map and then $H^{2d}(m_X^{\smile} \circ \kappa_X^{\smile})$ defines

$$\smile_i : H_c^i(X, E) \times H^{2d-i}(X, E) \longrightarrow H_c^{2d}(X, E).$$

The construction and coherence of $\kappa_X^{\smile}$ and $m_X^{\smile}$ are obligation OBL-DFE-CUP-DERIVED; construction of $\pi_{i,2d-i}$, including the Kunneth comparison, totalization, and Koszul convention, is the separate obligation OBL-DFE-CUP-DEGREE. Neither is asserted here.

Definition 6.3.12 (Compact Poincare pairing candidate)**.** Uses: `def:weil-duality-untwisted-top-trace-candidate,def:weil-duality-compact-ordinary-cup-candidate`

Lean: `Weil.Duality.compactPoincarePairingCandidate`

Define

$$B_{c,i} : H_c^i(X, E) \times H^{2d-i}(X, E) \longrightarrow E(-d), \qquad B_{c,i}(x, y) := \mathrm{tr}_X(x \smile_i y).$$

Lemma 6.3.13 (Formula for the compact Poincare pairing). *Uses:* def:weil-duality-compact-poincare-pairing-candidate

Lean: Weil.Duality.compactPoincarePairingCandidate_apply

For $x \in H^i_c(X, E)$ and $y \in H^{2d-i}(X, E)$,

$$B_{c,i}(x, y) = \mathrm{tr}_X(x \smile_i y).$$

Proof. Uses: def:weil-duality-compact-poincare-pairing-candidate

This is the defining composite for $B_{c,i}$. □

Lemma 6.3.14 (Bilinearity of the compact Poincare pairing). *Uses:* def:weil-duality-compact-poincare-pairing-candidate

Lean: Weil.Duality.compactPoincarePairingCandidate_bilinear

If the compact cup candidate is bilinear and the trace candidate is linear, then $B_{c,i}$ is E-bilinear.

Proof. Uses: def:weil-duality-compact-poincare-pairing-candidate,lemma:weil-duality-compact-poincare-pairing-formula

For fixed y, the map $x \mapsto x \smile_i y$ is linear by the supplied cup-product field, and composition with the linear trace remains linear. The same computation with x fixed proves linearity in y. □

Definition 6.3.15 (Tate-power coordinate map candidate). Uses: def:weil-tate-line,def:weil-tate-twist

Lean: Weil.Duality.TatePowerCoordinateMapCandidate

For the fixed integer d, a Tate-power coordinate map candidate is a specified E-linear map

$$c_{-d} : E(-d) \longrightarrow E.$$

This is input data; the definition does not assert that the tensor/dual power $E(-d)$ has been trivialized.

Definition 6.3.16 (Inverse candidate for a Tate-power coordinate map). Uses: def:weil-duality-tate-power-coordinate-map-candidate

Lean: Weil.Duality.TatePowerCoordinateInverseCandidate

An inverse candidate is a specified E-linear map

$$b_{-d} : E \longrightarrow E(-d).$$

Definition 6.3.17 (Source inverse predicate for a Tate-power coordinate map). Uses: def:weil-duality-tate-power-coordinate-map-candidate,def:weil-duality-tate-power-coordinate-inverse-candidate

Lean: Weil.Duality.IsTatePowerCoordinateSourceInverse

Define TateCoordSrcInv$_d$ to be the equation

$$b_{-d} \circ c_{-d} = \mathrm{id}_{E(-d)}\,.$$

Definition 6.3.18 (Target inverse predicate for a Tate-power coordinate map). Uses: def:weil-duality-tate-power-coordinate-map-candidate,def:weil-duality-tate-power-coordinate-inverse-candidate

Lean: Weil.Duality.IsTatePowerCoordinateTargetInverse

Define TateCoordTgtInv$_d$ to be the equation

$$c_{-d} \circ b_{-d} = \mathrm{id}_E\,.$$

Definition 6.3.19 (Tate-power trivialization certificate). Uses: def:weil-duality-tate-power-coordinate-inverse-candidate,def:weil-duality-tate-power-coordinate-source-inverse-predicate,def:weil-duality-tate-power-coordinate-target-inverse-predicate

Lean: Weil.Duality.TatePowerTrivializationCertificate

A Tate-power trivialization certificate contains b_{-d}, a proof of TateCoordSrcInv$_d$, and a proof of TateCoordTgtInv$_d$. Hence the reverse map and the two inverse equations are separate fields, with no existence claim.

Definition 6.3.20 (Compact Poincare adjoint map). Uses: `def:weil-duality-compact-poincare-pairing-candidate`, `def:weil-duality-tate-power-coordinate-map-candidate`

Lean: `Weil.Duality.compactPoincareAdjointMap`

For the explicitly supplied coordinate map $c_{-d} : E(-d) \to E$, define

$$\Theta_{c,i} : H^{2d-i}(X,E) \longrightarrow H^i_c(X,E)^\vee, \qquad \Theta_{c,i}(y)(x) := c_{-d}\big(B_{c,i}(x,y)\big).$$

No unnamed underlying-vector-space identification occurs in this formula. Constructing c_{-d}, its reverse map, and their two inverse equations from the tensor/dual-power definition of the Tate twist is the independent obligation OBL-DFE-TATE-TRIVIALIZATION.

Definition 6.3.21 (Compact Poincare inverse candidate). Uses: `def:weil-duality-compact-poincare-adjoint-map`

Lean: `Weil.Duality.CompactPoincareInverseCandidate`

A compact Poincare inverse candidate in degree i is a specified linear map

$$S_{c,i} : H^i_c(X,E)^\vee \longrightarrow H^{2d-i}(X,E).$$

Definition 6.3.22 (Compact Poincare perfectness certificate). Uses: `def:weil-duality-compact-poincare-adjoint-map`, `def:weil-duality-compact-poincare-inverse-candidate`

Lean: `Weil.Duality.CompactPoincarePerfectCertificate`

A compact Poincare perfectness certificate consists of $S_{c,i}$ and the two specified equations

$$S_{c,i}\Theta_{c,i} = \mathrm{id}, \qquad \Theta_{c,i}S_{c,i} = \mathrm{id}\,.$$

Definition 6.3.23 (Cup-product Frobenius predicate). Uses: `def:weil-duality-compact-ordinary-cup-candidate`

Lean: `Weil.Duality.IsCupFrobeniusEquivariant`

Define $\mathrm{CupFrob}_i$ to mean

$$F_{c,2d}(x \smile_i y) = F_{c,i}(x) \smile_i F_{2d-i}(y)$$

for all x, y in the indicated degrees.

Definition 6.3.24 (Trace Frobenius predicate). Uses: `def:weil-prime-power-data`, `def:weil-duality-untwisted-top-trace-candidate`

Lean: `Weil.Duality.IsTraceFrobeniusEquivariant`

Define $\mathrm{TraceFrob}_X$ to mean

$$\mathrm{tr}_X(F_{c,2d}z) = q^d\,\mathrm{tr}_X(z)$$

for every $z \in H^{2d}_c(X,E)$. The multiplier is q^d because the target is $E(-d)$, on which geometric Frobenius acts by q^d.

Lemma 6.3.25 (Frobenius similitude formula for the compact pairing). *Uses:* `def:weil-duality-compact-poincare-pairing-candidate`, `def:weil-duality-cup-frobenius-predicate`, `def:weil-duality-trace-frobenius-predicate`

Lean: *`Weil.Duality.compactPoincarePairing_frobenius`*

If $\mathrm{CupFrob}_i$ *and* $\mathrm{TraceFrob}_X$ *hold, then*

$$B_{c,i}(F_{c,i}x, F_{2d-i}y) = q^d B_{c,i}(x,y).$$

Proof. Uses: `def:weil-duality-cup-frobenius-predicate`, `def:weil-duality-trace-frobenius-predicate`, `lemma:weil-duality-compact-poincare-pairing-formula`

The cup predicate rewrites the left side as $\mathrm{tr}_X(F_{c,2d}(x \smile_i y))$. The trace predicate rewrites this expression as $q^d\,\mathrm{tr}_X(x \smile_i y) = q^d B_{c,i}(x,y)$. □

Definition 6.3.26 (Proper support-comparison candidate). Uses: `def:weil-coefficient-data`

Lean: `Weil.Duality.ProperSupportComparisonCandidate`

For a proper X/k, a proper support-comparison candidate contains a specified derived component

$$\kappa_X^{\mathrm{der}} : Ra_{X!}E_X \longrightarrow Ra_{X*}E_X$$

of the proposed natural transformation $Ra_{X!} \to Ra_{X*}$. For every integer i, define the degree map by

$$\kappa_i := H^i(\kappa_X^{\mathrm{der}}) : H_c^i(X,E) \longrightarrow H^i(X,E).$$

Construction and naturality of the derived component is obligation OBL-DFE-PROP-NAT; functorial passage to H^i, including compatibility with the fixed models of compact and ordinary cohomology, is the separate obligation OBL-DFE-PROP-HI. This definition asserts neither.

Definition 6.3.27 (Proper support inverse candidate). Uses: `def:weil-duality-proper-support-comparison-candidate`

Lean: `Weil.Duality.ProperSupportInverseCandidate`

An inverse candidate for κ_i is a specified linear map

$$\lambda_i : H^i(X,E) \longrightarrow H_c^i(X,E).$$

Definition 6.3.28 (Proper support isomorphism certificate). Uses: `def:weil-duality-proper-support-comparison-candidate`, `def:weil-duality-proper-support-inverse-candidate`

Lean: `Weil.Duality.ProperSupportIsoCertificate`

A proper support isomorphism certificate consists of the two specified equations

$$\lambda_i\kappa_i = \mathrm{id}_{H_c^i}, \qquad \kappa_i\lambda_i = \mathrm{id}_{H^i}\,.$$

Definition 6.3.29 (Frobenius predicate for the proper support inverse). Uses: `def:weil-duality-proper-support-inverse-candidate`

Lean: `Weil.Duality.IsProperSupportInverseFrobeniusEquivariant`

Define $\mathrm{SupportFrob}_i$ to mean

$$\lambda_i F_i = F_{c,i}\lambda_i.$$

Definition 6.3.30 (Ordinary Poincare pairing candidate for a proper scheme). Uses: `def:weil-duality-compact-poincare-pairing-candidate`, `def:weil-duality-proper-support-inverse-candidate`

Lean: `Weil.Duality.properPoincarePairingCandidate`

Define

$$B_i : H^i(X,E) \times H^{2d-i}(X,E) \longrightarrow E(-d), \qquad B_i(x,y) := B_{c,i}(\lambda_i x, y).$$

Lemma 6.3.31 (Formula for the proper Poincare pairing). *Uses:* `def:weil-duality-proper-poincare-pairing-candidate`

Lean: `Weil.Duality.properPoincarePairingCandidate_apply`

For $x \in H^i(X,E)$ and $y \in H^{2d-i}(X,E)$,

$$B_i(x,y) = B_{c,i}(\lambda_i x, y).$$

Proof. Uses: `def:weil-duality-proper-poincare-pairing-candidate`

This is the defining formula for B_i. □

Definition 6.3.32 (Inverse candidate for the proper Poincare adjoint). Uses: `def:weil-duality-compact-poincare-inverse-candidate`, `def:weil-duality-proper-support-comparison-candidate`

Lean: `Weil.Duality.properPoincareInverseCandidate`

Let $\kappa_i^\vee : H^i(X,E)^\vee \to H_c^i(X,E)^\vee$ be the dual map. Define

$$S_i := S_{c,i} \circ \kappa_i^\vee : H^i(X,E)^\vee \longrightarrow H^{2d-i}(X,E).$$

Lemma 6.3.33 (Left inverse for the proper Poincare adjoint). *Uses:* `def:weil-duality-compact-poincare-perfectness-certificate`, `def:weil-duality-proper-support-isomorphism-certificate`, `def:weil-duality-proper-poincare-pairing-candidate`, `def:weil-duality-proper-poincare-inverse-candidate`

Lean: `Weil.Duality.properPoincareInverseCandidate_left`

Under the two displayed certificates, $S_i\Theta_i = \mathrm{id}$, where Θ_i is the adjoint of the proper pairing B_i.

Proof. Uses: def:weil-duality-compact-poincare-adjoint-map, def:weil-duality-compact-poincare-perfectness-certificate, def:weil-duality-proper-support-isomorphism-certificate, def:weil-duality-proper-poincare-inverse-candidate, lemma:weil-duality-proper-poincare-pairing-formula

The formula for B_i gives $\Theta_i = \lambda_i^\vee \Theta_{c,i}$. Hence

$$S_i \Theta_i = S_{c,i} \kappa_i^\vee \lambda_i^\vee \Theta_{c,i} = S_{c,i} (\lambda_i \kappa_i)^\vee \Theta_{c,i}.$$

The support certificate replaces $\lambda_i \kappa_i$ by the identity, and the compact perfectness certificate replaces $S_{c,i}\Theta_{c,i}$ by the identity. □

Lemma 6.3.34 (Right inverse for the proper Poincare adjoint). Uses: def:weil-duality-compact-poincare-perfectness-certificate, def:weil-duality-proper-support-isomorphism-certificate, def:weil-duality-proper-poincare-pairing-candidate, def:weil-duality-proper-poincare-inverse-candidate

Lean: Weil.Duality.properPoincareInverseCandidate_right

Under the same certificates, $\Theta_i S_i = \mathrm{id}_{H^i(X,E)^\vee}$.

Proof. Uses: def:weil-duality-compact-poincare-adjoint-map, def:weil-duality-compact-poincare-perfectness-certificate, def:weil-duality-proper-support-isomorphism-certificate, def:weil-duality-proper-poincare-inverse-candidate, lemma:weil-duality-proper-poincare-pairing-formula

Again $\Theta_i = \lambda_i^\vee \Theta_{c,i}$. Therefore

$$\Theta_i S_i = \lambda_i^\vee \Theta_{c,i} S_{c,i} \kappa_i^\vee = \lambda_i^\vee \kappa_i^\vee = (\kappa_i \lambda_i)^\vee.$$

Compact perfectness gives the second equality; the support certificate turns the last map into the identity. □

Proposition 6.3.35 (Conditional perfectness of the proper Poincare pairing). Uses: def:weil-duality-compact-poincare-perfectness-certificate, def:weil-duality-proper-support-isomorphism-certificate, def:weil-duality-proper-poincare-pairing-candidate, def:weil-duality-proper-poincare-inverse-candidate

Lean: Weil.Duality.properPoincarePairing_perfect

Under the compact perfectness and proper support certificates, the explicitly constructed S_i is a two-sided inverse to the adjoint of B_i.

Proof. Uses: lemma:weil-duality-proper-poincare-adjoint-left-inverse, lemma:weil-duality-proper-poincare-adjoint-right-inverse

The two cited lemmas are exactly the left- and right-inverse fields required for a perfectness certificate. □

Lemma 6.3.36 (Conditional Frobenius similitude for the proper pairing). Uses: def:weil-duality-cup-frobenius-predicate, def:weil-duality-trace-frobenius-predicate, def:weil-duality-proper-support-frobenius-predicate, def:weil-duality-proper-poincare-pairing-candidate

Lean: Weil.Duality.properPoincarePairing_frobenius

If the cup, trace, and support Frobenius predicates hold, then

$$B_i(F_i x, F_{2d-i} y) = q^d B_i(x, y).$$

Proof. Uses: def:weil-duality-proper-support-frobenius-predicate, lemma:weil-duality-compact-pairing-frobenius-similitude, lemma:weil-duality-proper-poincare-pairing-formula

Use support equivariance to rewrite $\lambda_i F_i x = F_{c,i} \lambda_i x$. The compact-pairing similitude lemma then gives

$$B_{c,i}(F_{c,i}\lambda_i x, F_{2d-i} y) = q^d B_{c,i}(\lambda_i x, y),$$

which is the asserted formula after unfolding the proper pairing once. □

Definition 6.3.37 (Graded-commutativity predicate for the proper pairing). Uses: def:weil-duality-proper-poincare-pairing-candidate

Lean: Weil.Duality.IsProperPairingGradedCommutative

Define ProperGradedComm$_i$ to mean

$$B_{2d-i}(y, x) = (-1)^{i(2d-i)} B_i(x, y)$$

for all $x \in H^i(X, E)$ and $y \in H^{2d-i}(X, E)$.

Lemma 6.3.38 (Middle-degree swap sign). *Uses:* *def:weil-duality-proper-pairing-graded-commutativity-predicate*

Lean: *Weil.Duality.properMiddlePairing_swap*

If ProperGradedComm$_d$ *holds, then*

$$B_d(y,x) = (-1)^d B_d(x,y).$$

Proof. Uses: def:weil-duality-proper-pairing-graded-commutativity-predicate

At $i = d$, the exponent in the defining sign is $d(2d-d) = d^2$. Since $d^2 - d = d(d-1)$ is even, $(-1)^{d^2} = (-1)^d$. Substitute this equality into the predicate. □

6.4 A bounded graded duality certificate

Definition 6.4.1 (Bounded graded perfect-similitude system). Uses: def:weil-prime-power-data, def:weil-coefficient-data, def:weil-frobenius-module, def:weil-duality-bilinear-pairing-data, def:weil-duality-perfect-pairing-certificate, def:weil-duality-frobenius-similitude-predicate, def:weil-duality-pairing-swap-sign-predicate

Lean: Weil.Duality.GradedSimilitudeSystem

A bounded graded perfect-similitude system of dimension parameter $d \in \mathbf{N}$ consists of the following explicit data.

1. For every $0 \le i \le 2d$, a Frobenius module (V_i, F_i) over E.
2. For every such i, a bilinear pairing $B_i : V_i \times V_{2d-i} \to E$, an explicit perfectness certificate, and a proof of $\mathrm{IsSim}_{q^d}(B_i)$.
3. For $i = d$, the proof $B_d(y,x) = (-1)^d B_d(x,y)$.

The scalar-valued form is the coordinate form of a pairing into $E(-d)$ with respect to the fixed basis datum of the Tate line. The data type has no claim that etale cohomology supplies an inhabitant.

Definition 6.4.2 (Graded Betti dimensions). Uses: def:weil-duality-graded-similitude-system

Lean: Weil.Duality.GradedSimilitudeSystem.betti

For $0 \le i \le 2d$, define

$$b_i := \dim_E V_i.$$

Definition 6.4.3 (Graded Frobenius determinants). Uses: def:weil-duality-graded-similitude-system

Lean: Weil.Duality.GradedSimilitudeSystem.detFrob

For $0 \le i \le 2d$, define

$$D_i := \det(F_i) \in E^\times.$$

Definition 6.4.4 (Graded Frobenius factors). Uses: def:weil-reciprocal-characteristic-polynomial, def:weil-duality-graded-similitude-system

Lean: Weil.Duality.GradedSimilitudeSystem.factor

For $0 \le i \le 2d$, define

$$P_i(T) := \det(1 - TF_i \mid V_i).$$

Definition 6.4.5 (Graded Euler characteristic). Uses: def:weil-duality-graded-betti-dimensions

Lean: Weil.Duality.GradedSimilitudeSystem.eulerCharacteristic

Define the integer

$$\chi := \sum_{i=0}^{2d} (-1)^i b_i.$$

Definition 6.4.6 (Alternating Frobenius determinant). Uses: `def:weil-duality-graded-frobenius-determinants`

Lean: `Weil.Duality.GradedSimilitudeSystem.alternatingFrobeniusDet`

Define

$$\Delta := \prod_{i=0}^{2d} D_i^{(-1)^i} \in E^\times.$$

An exponent $(-1)^i = -1$ means inversion in $E^\times$.

Definition 6.4.7 (Alternating graded determinant function). Uses: `def:weil-duality-graded-frobenius-factors`

Lean: `Weil.Duality.GradedSimilitudeSystem.zetaDeterminant`

Define the rational function

$$Z_F(T) := \prod_{i=0}^{2d} P_i(T)^{(-1)^{i+1}} \in E(T).$$

Lemma 6.4.8 (Dimensions in complementary degrees). *Uses:* *def:weil-duality-graded-similitude-system, def:weil-duality-graded-betti-dimensions*

Lean: *Weil.Duality.GradedSimilitudeSystem.betti_symm*

For every $0 \le i \le 2d$,

$$b_{2d-i} = b_i.$$

Proof. Uses: `lemma:weil-duality-perfect-pairing-dimension`

Apply the dimension equality for a perfect pairing to the stored pairing $B_i : V_i \times V_{2d-i} \to E$. Its two dimensions are exactly b_i and b_{2d-i}. □

Lemma 6.4.9 (Determinants in complementary degrees). *Uses:* *def:weil-duality-graded-similitude-system, def:weil-duality-graded-betti-dimensions, def:weil-duality-graded-frobenius-determinants*

Lean: *Weil.Duality.GradedSimilitudeSystem.det_pair*

For every $0 \le i \le 2d$,

$$D_i D_{2d-i} = q^{db_i}.$$

Proof. Uses: `prop:weil-duality-paired-determinant-product`

Apply the paired-determinant proposition to B_i, whose stored multiplier is $\mu = q^d$ and whose first dimension is b_i. Its right side is $(q^d)^{b_i} = q^{db_i}$. □

Lemma 6.4.10 (Certified reciprocal-eigenvalue presentation in complementary degrees). *Uses:* *def:weil-duality-graded-similitude-system, def:weil-duality-reciprocal-eigenvalue-presentation-candidate, def:weil-duality-eigenvalue-presentation-certificate, def:weil-duality-paired-eigenvalue-presentation-candidate*

Lean: *Weil.Duality.GradedSimilitudeSystem.reciprocalEigenvalues_pair*

Given an explicit certified presentation $\mathcal{A}_i$ *for* F_i, *the explicitly defined multiset*

$$\mathcal{A}_{2d-i}^{\mathrm{pair}} := \mathcal{A}_i.\mathrm{map}\big(\alpha \longmapsto q^d/\alpha\big)$$

has cardinality b_{2d-i} *and gives the factorization* $P_{2d-i}(T) = \prod_{\beta \in \mathcal{A}_{2d-i}^{\mathrm{pair}}} (1 - \beta T)$ *over* $\overline{E}$.

Proof. Uses: `def:weil-duality-paired-eigenvalue-presentation-candidate,prop:weil-duality-paired-reciprocal-eigenvalue-presentation`

Instantiate the certified paired-presentation proposition with the stored pairing B_i, its stored perfectness and similitude fields, and $\mu = q^d$. Its two conclusions are exactly the displayed cardinality and factorization statements. □

Lemma 6.4.11 (Degreewise reciprocal factor identity). *Uses:* *def:weil-duality-graded-similitude-system, def:weil-duality-graded-betti-dimensions, def:weil-duality-graded-frobenius-determinants, def:weil-duality-graded-frobenius-factors*

Lean: *Weil.Duality.GradedSimilitudeSystem.factor_reciprocal*

For every $0 \le i \le 2d$, in $E[T, T^{-1}]$,

$$P_{2d-i}(q^{-d}T^{-1}) = (-1)^{b_i} T^{-b_i} D_i^{-1} P_i(T).$$

Proof. Uses: prop:weil-duality-paired-reciprocal-characteristic

Apply the paired reciprocal-characteristic proposition to B_i with $\mu = q^d$, $r = b_i$, and $D = D_i$. Since $\mu^{-1} = q^{-d}$, its formula is the displayed formula. □

Lemma 6.4.12 (Middle pairing is alternating in odd dimension parameter). *Uses:* *def:weil-duality-graded-similitude-system*

Lean: *Weil.Duality.GradedSimilitudeSystem.middle_skew_of_odd*

If d is odd, then

$$B_d(y, x) = -B_d(x, y).$$

Proof. Uses: def:weil-duality-graded-similitude-system

The system stores $B_d(y, x) = (-1)^d B_d(x, y)$. For odd d, the scalar $(-1)^d$ is -1; substitution gives the result. □

Lemma 6.4.13 (Parity of the middle Betti dimension). *Uses:* *def:weil-duality-graded-similitude-system, def:weil-duality-graded-betti-dimensions*

Lean: *Weil.Duality.GradedSimilitudeSystem.middle_betti_even*

If d is odd, then b_d is even.

Proof. Uses: lemma:weil-duality-graded-middle-alternating, lemma:weil-duality-alternating-perfect-even-dimension

The preceding lemma supplies swap sign -1 for the stored perfect middle pairing. Apply the even-dimension lemma to that pairing. Its dimension is b_d. □

Lemma 6.4.14 (Parity of the Euler characteristic when d is odd). *Uses:* *def:weil-duality-graded-similitude-system, def:weil-duality-graded-betti-dimensions, def:weil-duality-graded-euler-characteristic*

Lean: *Weil.Duality.GradedSimilitudeSystem.euler_even_of_d_odd*

If d is odd, then χ is even.

Proof. Uses: lemma:weil-duality-graded-dimension-symmetry, lemma:weil-duality-graded-middle-betti-even

Split the defining sum for χ into the pairs $i, 2d-i$ with $i < d$, and the middle index d. The two signs in a pair agree because $2d$ is even, and their dimensions agree by dimension symmetry. Each pair therefore contributes $2(-1)^i b_i$, an even integer. The middle contribution $(-1)^d b_d$ is even by the middle-parity lemma. The whole sum is even. □

Lemma 6.4.15 (Integrality of the functional-equation exponent). *Uses:* *def:weil-duality-graded-similitude-system, def:weil-duality-graded-euler-characteristic*

Lean: *Weil.Duality.GradedSimilitudeSystem.mul_euler_even*

The integer $d\chi$ is even.

Proof. Uses: lemma:weil-duality-graded-euler-even-of-d-odd

If d is even, $d\chi$ is a multiple of 2. If d is odd, the preceding lemma writes $\chi = 2m$ for an integer m, so $d\chi = 2(dm)$. These two cases exhaust the parity of d. □

Definition 6.4.16 (Half functional-equation exponent). Uses: def:weil-duality-graded-similitude-system, def:weil-duality-graded-euler-characteristic

Lean: Weil.Duality.GradedSimilitudeSystem.halfExponent

Define the integer

$$h := \frac{d\chi}{2}$$

using integer Euclidean division by 2.

Lemma 6.4.17 (Twice the half exponent). *Uses:* *def:weil-duality-half-functional-equation-exponent*

Lean: *Weil.Duality.GradedSimilitudeSystem.two_mul_halfExponent*

One has $2h = d\chi$.

Proof. Uses: def:weil-duality-half-functional-equation-exponent,lemma:weil-duality-functional-equation-exponent-even

The parity lemma gives an integer m with $d\chi = 2m$. Euclidean division of $2m$ by 2 returns m, so $h = m$ and $2h = d\chi$. □

Lemma 6.4.18 (Square of the alternating Frobenius determinant). *Uses:* *def:weil-duality-graded-similitude-system*, *def:weil-duality-graded-euler-characteristic*, *def:weil-duality-alternating-frobenius-determinant*

Lean: *Weil.Duality.GradedSimilitudeSystem.alternatingFrobeniusDet_sq*

One has

$$\Delta^2 = q^{d\chi}.$$

Proof. Uses: lemma:weil-duality-graded-dimension-symmetry,lemma:weil-duality-graded-determinant-pair

Split the finite product defining Δ into complementary pairs and the middle degree:

$$\Delta = \left(\prod_{i<d} (D_i D_{2d-i})^{(-1)^i}\right) D_d^{(-1)^d}.$$

After squaring, replace every paired product by q^{db_i}. In middle degree the determinant-pair identity reads $D_d^2 = q^{db_d}$; raising it to the exponent $(-1)^d$ treats both parity cases. Thus

$$\Delta^2 = q^{d\left(2\sum_{i<d}(-1)^i b_i + (-1)^d b_d\right)}.$$

Dimension symmetry rewrites the integer in parentheses as the defining alternating sum χ. □

Definition 6.4.19 (Functional-equation sign candidate). Uses: def:weil-duality-graded-euler-characteristic,def:weil-duality-alternating-frobenius-determinant,def:weil-duality-half-functional-equation-exponent

Lean: Weil.Duality.GradedSimilitudeSystem.epsilon

Define

$$\epsilon := (-1)^\chi q^{-h} \Delta \in E^\times.$$

Lemma 6.4.20 (Square of the functional-equation sign). *Uses:* *def:weil-duality-functional-equation-sign-candidate*

Lean: *Weil.Duality.GradedSimilitudeSystem.epsilon_sq*

One has $\epsilon^2 = 1$.

Proof. Uses: def:weil-duality-functional-equation-sign-candidate,lemma:weil-duality-twice-half-functional-equation-exponent,lemma:weil-duality-alternating-frobenius-determinant-square

Squaring the definition gives

$$\epsilon^2 = (-1)^{2\chi} q^{-2h} \Delta^2 = q^{-2h} q^{d\chi}.$$

The twice-half-exponent identity says $2h = d\chi$, so the two powers of q cancel and leave 1. □

Lemma 6.4.21 (The sign candidate is a sign). *Uses:* *def:weil-duality-functional-equation-sign-candidate*

Lean: *Weil.Duality.GradedSimilitudeSystem.epsilon_eq_one_or_neg_one*

One has $\epsilon = 1$ *or* $\epsilon = -1$.

Proof. Uses: lemma:weil-duality-functional-equation-sign-square

The square identity gives

$$0 = \epsilon^2 - 1 = (\epsilon - 1)(\epsilon + 1).$$

The coefficient field E has no zero divisors. Hence one factor is zero, which gives the two alternatives. □

6.5 Assembly of the rational-function identity

Definition 6.5.1 (Transformed graded determinant function). Uses: `def:weil-prime-power-data`, `def:weil-duality-graded-determinant-function`

Lean: `Weil.Duality.GradedSimilitudeSystem.transformedZetaDeterminant`

Define

$$Z_F^\vee(T) := Z_F(q^{-d}T^{-1}) \in E(T).$$

Lemma 6.5.2 (Reindexing the transformed degree product). *Uses:* *def:weil-duality-graded-frobenius-factors*, *def:weil-duality-transformed-graded-determinant-function*

Lean: *Weil.Duality.GradedSimilitudeSystem.transformed_reindex*

One has

$$Z_F^\vee(T) = \prod_{i=0}^{2d} P_{2d-i}(q^{-d}T^{-1})^{(-1)^{i+1}}.$$

Proof. Uses: `def:weil-duality-graded-determinant-function`, `def:weil-duality-transformed-graded-determinant-function`

Start with the defining product for $Z_F(q^{-d}T^{-1})$ and apply the bijection $i \mapsto 2d - i$ of the finite index set. Its square is the identity. Since $2d$ is even, $(-1)^{2d-i+1} = (-1)^{i+1}$, giving the displayed exponents. □

Lemma 6.5.3 (Reassembly of the characteristic factors). *Uses:* *def:weil-duality-graded-frobenius-factors*, *def:weil-duality-graded-determinant-function*

Lean: *Weil.Duality.GradedSimilitudeSystem.factor_product_reassembles*

The product of the $P_i(T)$-terms obtained by substituting the degreewise reciprocal identities is $Z_F(T)$.

Proof. Uses: `def:weil-duality-graded-determinant-function`

The degree-i reciprocal identity contributes $P_i(T)^{(-1)^{i+1}}$. Multiplying over $0 \le i \le 2d$ is exactly the finite product defining $Z_F(T)$. □

Lemma 6.5.4 (Exponent of T in the transformed product). *Uses:* *def:weil-duality-graded-betti-dimensions*, *def:weil-duality-graded-euler-characteristic*

Lean: *Weil.Duality.GradedSimilitudeSystem.transformed_t_exponent*

The total exponent of T after degreewise substitution is χ:

$$\sum_{i=0}^{2d}(-b_i)(-1)^{i+1} = \sum_{i=0}^{2d}(-1)^i b_i = \chi.$$

Proof. Uses: `def:weil-duality-graded-euler-characteristic`

For every i, $(-b_i)(-1)^{i+1} = (-1)^i b_i$. Sum this equality over the finite index set and unfold the definition of χ. □

Lemma 6.5.5 (Sign in the transformed product). *Uses:* *def:weil-duality-graded-betti-dimensions*, *def:weil-duality-graded-euler-characteristic*

Lean: *Weil.Duality.GradedSimilitudeSystem.transformed_parity_sign*

The product of the degreewise signs is $(-1)^\chi$.

Proof. Uses: `def:weil-duality-graded-euler-characteristic`

Inverting a sign leaves it unchanged, so the degree-i contribution is $(-1)^{b_i}$ for either value of $(-1)^{i+1}$. The full product is $(-1)^{\sum_i b_i}$. Modulo 2, the integers $\sum_i b_i$ and $\sum_i (-1)^i b_i = \chi$ agree because $(-1)^i \equiv 1 \pmod 2$. Their powers of -1 are equal. □

Lemma 6.5.6 (Determinant scalar in the transformed product). *Uses:* *def:weil-duality-graded-frobenius-determinants*, *def:weil-duality-alternating-frobenius-determinant*

Lean: *Weil.Duality.GradedSimilitudeSystem.transformed_det_scalar*

The product of the degreewise determinant scalars is Δ.

Proof. Uses: def:weil-duality-alternating-frobenius-determinant

In degree i, the reciprocal identity contains D_i^{-1}, raised to the exponent $(-1)^{i+1}$. Its resulting exponent on D_i is $-(-1)^{i+1} = (-1)^i$. Multiplication over i is the definition of Δ. □

Proposition 6.5.7 (Raw graded functional equation)**.** *Uses:* def:weil-duality-graded-euler-characteristic, def:weil-duality-alternating-frobenius-determinant, def:weil-duality-graded-determinant-function, def:weil-duality-transformed-graded-determinant-function

Lean: Weil.Duality.GradedSimilitudeSystem.rawFunctionalEquation

In $E(T)$,

$$Z_F(q^{-d}T^{-1}) = (-1)^{\chi}\Delta T^{\chi} Z_F(T).$$

Proof. Uses: lemma:weil-duality-graded-reciprocal-factor, lemma:weil-duality-transformed-product-reindex, lemma:weil-duality-transformed-characteristic-factor-product, lemma:weil-duality-transformed-t-exponent, lemma:weil-duality-transformed-parity-sign, lemma:weil-duality-transformed-determinant-scalar

Insert the degreewise reciprocal-factor identity into the reindexed product. The characteristic-factor lemma gives $Z_F(T)$. The exponent lemma gives T^{χ}, the parity lemma gives $(-1)^{\chi}$, and the determinant lemma gives Δ. Multiplying these four independently computed factors proves the formula. □

Proposition 6.5.8 (Weil-normalized graded functional equation)**.** *Uses:* def:weil-duality-graded-euler-characteristic, def:weil-duality-graded-determinant-function, def:weil-duality-half-functional-equation-exponent, def:weil-duality-functional-equation-sign-candidate

Lean: Weil.Duality.GradedSimilitudeSystem.functionalEquation

The sign candidate belongs to $\{1, -1\}$, *and*

$$Z_F(q^{-d}T^{-1}) = \epsilon q^h T^{\chi} Z_F(T), \qquad 2h = d\chi.$$

Proof. Uses: def:weil-duality-functional-equation-sign-candidate, lemma:weil-duality-twice-half-functional-equation-exponent, lemma:weil-duality-functional-equation-sign-dichotomy, prop:weil-duality-raw-graded-functional-equation

The definition of ϵ rearranges to $(-1)^{\chi}\Delta = \epsilon q^h$. Substitute this equality into the raw functional equation. The twice-half-exponent lemma supplies $2h = d\chi$, and the sign-dichotomy lemma supplies membership in $\{1, -1\}$. □

Definition 6.5.9 (Cohomological factorization comparison predicate)**.** Uses: def:weil-duality-graded-determinant-function

Lean: Weil.Duality.IsZetaFactorizationComparison

For a rational zeta function $Z_X(T) \in E(T)$, define

$$\mathrm{IsZetaFactorization}(Z_X, Z_F) \iff Z_X(T) = Z_F(T).$$

This is a predicate; the trace-formula module must prove it before the next proposition can be instantiated with a geometric zeta function.

Proposition 6.5.10 (Transfer of the functional equation to a zeta function)**.** *Uses:* def:weil-duality-graded-euler-characteristic, def:weil-duality-half-functional-equation-exponent, def:weil-duality-functional-equation-sign-candidate, def:weil-duality-zeta-factorization-comparison-predicate

Lean: Weil.Duality.transferFunctionalEquationToZeta

If $\mathrm{IsZetaFactorization}(Z_X, Z_F)$, *then*

$$Z_X(q^{-d}T^{-1}) = \epsilon q^h T^{\chi} Z_X(T), \qquad \epsilon \in \{1, -1\}, \qquad 2h = d\chi.$$

Proof. Uses: def:weil-duality-zeta-factorization-comparison-predicate, prop:weil-duality-weil-normalized-functional-equation

Rewrite Z_X as Z_F on both sides using the supplied comparison equation. Apply the Weil-normalized graded functional equation, including its sign and exponent fields, and rewrite Z_F back to Z_X. □

6.6 Normalization regression tests

Lemma 6.6.1 (Point sign test). *Uses:* *def:weil-duality-functional-equation-sign-candidate, def:weil-duality-graded-determinant-function*

Lean: *Weil.Duality.functionalEquation_point_test*

For the degree-zero system $d = 0$, $V_0 = E$, and $F_0 = 1$,

$$\chi = 1, \qquad \Delta = 1, \qquad \epsilon = -1, \qquad Z_F(T) = (1-T)^{-1},$$

and the functional equation is $Z_F(T^{-1}) = -TZ_F(T)$.

Proof. Uses: def:weil-duality-graded-betti-dimensions, def:weil-duality-graded-frobenius-determinants, def:weil-duality-graded-frobenius-factors, def:weil-duality-graded-euler-characteristic, def:weil-duality-alternating-frobenius-determinant, def:weil-duality-graded-determinant-function, def:weil-duality-half-functional-equation-exponent, def:weil-duality-functional-equation-sign-candidate

There is one degree. Direct evaluation gives $b_0 = 1, D_0 = 1, P_0 = 1 - T, \chi = 1, \Delta = 1$, and $h = 0$. Thus $\epsilon = (-1)^1 q^0 = -1$ and

$$Z_F(T^{-1}) = (1-T^{-1})^{-1} = -T(1-T)^{-1} = -TZ_F(T).$$

□

Lemma 6.6.2 (Projective-line sign test). *Uses:* *def:weil-duality-functional-equation-sign-candidate, def:weil-duality-graded-determinant-function*

Lean: *Weil.Duality.functionalEquation_projectiveLine_test*

For $d = 1$, $V_0 = E$ with $F_0 = 1$, $V_1 = 0$, and $V_2 = E(-1)$ with $F_2 = q$,

$$\chi = 2, \qquad \Delta = q, \qquad \epsilon = 1, \qquad Z_F(T) = \frac{1}{(1-T)(1-qT)},$$

and

$$Z_F(q^{-1}T^{-1}) = qT^2 Z_F(T).$$

Proof. Uses: def:weil-duality-graded-betti-dimensions, def:weil-duality-graded-frobenius-determinants, def:weil-duality-graded-frobenius-factors, def:weil-duality-graded-euler-characteristic, def:weil-duality-alternating-frobenius-determinant, def:weil-duality-graded-determinant-function, def:weil-duality-half-functional-equation-exponent, def:weil-duality-functional-equation-sign-candidate

The three degrees give

$$(b_0, b_1, b_2) = (1, 0, 1), \quad (D_0, D_2) = (1, q), \quad (P_0, P_2) = (1 - T, 1 - qT).$$

Hence $\chi = 2$, $\Delta = q$, and $h = d\chi/2 = 1$. The definition gives $\epsilon = (-1)^2 q^{-1} q = 1$. Finally,

$$\begin{aligned} Z_F(q^{-1}T^{-1}) &= \frac{1}{(1-q^{-1}T^{-1})(1-T^{-1})} \\ &= \frac{qT^2}{(1-qT)(1-T)} = qT^2 Z_F(T). \end{aligned}$$

□

Chapter 7

Weights, the Deligne–Laumon estimate, and the Riemann hypothesis

Status. This chapter uses cohomological geometric Frobenius. On $E(1)$ it acts by q^{-1}, hence a twist (n) lowers weight by $2n$. The characteristic factor is $P_i(T) = \det(1 - TF_i)$: its reciprocal roots are Frobenius eigenvalues, while its ordinary roots are their inverses. Deligne's PUR, local monodromy MON, the six operations, trace, duality, local Fourier transform, and the ten-part TAME reduction lie outside the pinned mathlib foundation. They are therefore decomposed below into constructions and property certificates. Only implications from supplied certificates and finite algebraic or absolute-value calculations are proved. Primary locators are Deligne, *Weil II*, printed pp. 150–177 and 204–207 (PDF pp. 15–42 and 69–72), and Laumon, Sections 1–4, especially printed pp. 204–208 (PDF pp. 75–79).

7.1 Scalar weights and their elementary algebra

Definition 7.1.1 (Local Frobenius scalar datum)**.** Uses: `def:weil-prime-power-data`, `def:weil-coefficient-data`

Lean: `Weil.Weight.LocalScalarData`

A local Frobenius scalar datum is a positive integer e, the local cardinality $q_e := q^e$, and a nonzero scalar $\alpha \in \overline{E}^{\times}$. It models an eigenvalue of geometric Frobenius at a closed point of degree e.

Definition 7.1.2 (Fixed-embedding local purity)**.** Uses: `def:weil-weight-local-scalar-data`, `def:weil-iota-scalar-weight`

Lean: `Weil.Weight.LocalScalarData.IsIotaPure`

For $w \in \mathbf{R}$, define

$$\mathrm{Pure}_{\iota,w}(e,\alpha) \quad :\Longleftrightarrow \quad |\iota(\alpha)| = q_e^{w/2}.$$

This is the fixed-ι convention of Deligne (1.2.2), printed p. 152 (PDF p. 17).

Lemma 7.1.3 (Local and global fixed-embedding conventions agree)**.** *Uses:* *`def:weil-weight-local-scalar-data`, `def:weil-weight-local-iota-pure-predicate`, `def:weil-iota-scalar-weight`*

Lean: *`Weil.Weight.LocalScalarData.isIotaPure_iff`*

The local predicate of weight w is the global scalar predicate relative to q^e.

Proof. Uses: `def:weil-weight-local-scalar-data`, `def:weil-weight-local-iota-pure-predicate`, `def:weil-iota-scalar-weight`

Both predicates reduce to the same equality $|\iota(\alpha)| = (q^e)^{w/2}$. □

Definition 7.1.4 (Universal local purity)**.** Uses: `def:weil-prime-power-data`, `def:weil-universal-scalar-weight`

Lean: `Weil.Weight.IsUniversallyPureLocalScalar`

For an algebraic number $a \in \overline{\mathbf{Q}}^{\times}$, degree $e > 0$, and $w \in \mathbf{R}$, define universal local purity by

$$\forall \sigma : \mathbf{Q}(a) \hookrightarrow \mathbf{C}, \qquad |\sigma(a)| = (q^e)^{w/2}.$$

Definition 7.1.5 (Algebraic realization of an ell-adic scalar). Uses: `def:weil-coefficient-data`, `def:weil-weight-local-scalar-data`

Lean: `Weil.Weight.AlgebraicRealizationData`

An algebraic realization of (e, α) consists of an algebraic number $a \in \overline{\mathbf{Q}}^{\times}$, an embedding $\rho : \overline{\mathbf{Q}} \hookrightarrow \overline{E}$, and the displayed identity $\rho(a) = \alpha$. The number, embedding, and identity are data; their construction is not implicit.

Definition 7.1.6 (Universal purity certificate for an ell-adic scalar). Uses: `def:weil-weight-algebraic-realization-data`, `def:weil-weight-local-universal-pure-predicate`

Lean: `Weil.Weight.UniversalRealizationCertificate`

A universal purity certificate of weight w is an algebraic realization $(a, \rho, \rho(a) = \alpha)$ together with the universal local purity predicate for a.

Definition 7.1.7 (Compatibility of the selected complex realization). Uses: `def:weil-coefficient-data`, `def:weil-weight-algebraic-realization-data`

Lean: `Weil.Weight.IsSelectedRealizationCompatible`

Define selected-realization compatibility to mean that the restricted complex embedding $\iota \circ \rho : \mathbf{Q}(a) \hookrightarrow \mathbf{C}$ is one of the embeddings quantified over in universal purity.

Lemma 7.1.8 (Universal purity evaluates at the selected embedding). *Uses:* `def:weil-weight-local-iota-pure-predicate`, `def:weil-weight-universal-realization-certificate`, `def:weil-weight-realization-compatibility-predicate`

Lean: `Weil.Weight.UniversalRealizationCertificate.isIotaPure`

If selected-realization compatibility holds, a universally pure scalar of weight w is ι-pure of weight w.

Proof. Uses: `def:weil-weight-local-iota-pure-predicate`, `def:weil-weight-universal-realization-certificate`, `def:weil-weight-realization-compatibility-predicate`

Evaluate the universal modulus equality at the displayed embedding $\iota \circ \rho$, and rewrite $\rho(a)$ as α. The resulting identity is $|\iota(\alpha)| = (q^e)^{w/2}$. □

Lemma 7.1.9 (Product of pure scalars). *Uses:* `def:weil-weight-local-iota-pure-predicate`

Lean: `Weil.Weight.isIotaPure_mul`

For local scalars of the same degree e, purity of α in weight u and purity of β in weight v imply purity of $\alpha\beta$ in weight $u + v$.

Proof. Uses: `def:weil-weight-local-iota-pure-predicate`

Multiply the two defining equalities and use

$$|\iota(\alpha\beta)| = |\iota(\alpha)|\,|\iota(\beta)|, \qquad q_e^{u/2} q_e^{v/2} = q_e^{(u+v)/2}.$$

□

Lemma 7.1.10 (Inverse of a pure scalar). *Uses:* `def:weil-weight-local-iota-pure-predicate`

Lean: `Weil.Weight.isIotaPure_inv`

Purity of α in weight w implies purity of α^{-1} in weight $-w$.

Proof. Uses: `def:weil-weight-local-scalar-data`, `def:weil-weight-local-iota-pure-predicate`

Nonvanishing gives

$$|\iota(\alpha^{-1})| = |\iota(\alpha)|^{-1} = (q_e^{w/2})^{-1} = q_e^{-w/2}.$$

□

Lemma 7.1.11 (Integral powers of a pure scalar). *Uses:* `def:weil-weight-local-iota-pure-predicate`

Lean: `Weil.Weight.isIotaPure_zpow`

For $n \in \mathbf{Z}$, purity of α in weight w implies purity of α^n in weight nw.

Proof. Uses: lemma:weil-weight-iota-pure-product,lemma:weil-weight-iota-pure-inverse
For $n \geq 0$, induction on n repeatedly applies the product lemma, starting from $|1| = q_e^0$. For $n < 0$, write $\alpha^n = (\alpha^{-1})^{-n}$, apply the inverse lemma, and apply the nonnegative case to $-n$. □

Lemma 7.1.12 (Local Tate-twist weight shift). *Uses:* def:weil-weight-local-iota-pure-predicate, def:weil-tate-twist

Lean: Weil.Weight.isIotaPure_tateTwist
If α has local weight w, then the degree-e twist scalar $q^{-en}\alpha$ has local weight $w - 2n$.

Proof. Uses: def:weil-weight-local-scalar-data,def:weil-weight-local-iota-pure-predicate,def:weil-tate-line,def:weil-tate-twist
The degree-e geometric Frobenius on the n-fold Tate line is multiplication by $(q^{-n})^e = q^{-en}$. Therefore

$$|\iota(q^{-en}\alpha)| = q^{-en}(q^e)^{w/2} = (q^e)^{(w-2n)/2}.$$

This is the local purity equation in weight $w - 2n$. □

Definition 7.1.13 (Fixed-embedding scalar upper bound). Uses: def:weil-weight-local-scalar-data

Lean: Weil.Weight.LocalScalarData.IsIotaWeightAtMost
Define $\mathrm{Wt}_\iota(\alpha) \leq w$ by

$$|\iota(\alpha)| \leq q_e^{w/2}.$$

Definition 7.1.14 (Fixed-embedding scalar lower bound). Uses: def:weil-weight-local-scalar-data

Lean: Weil.Weight.LocalScalarData.IsIotaWeightAtLeast
Define $\mathrm{Wt}_\iota(\alpha) \geq w$ by

$$q_e^{w/2} \leq |\iota(\alpha)|.$$

Lemma 7.1.15 (Purity gives the upper bound). *Uses:* def:weil-weight-local-iota-pure-predicate, def:weil-weight-iota-at-most-predicate

Lean: Weil.Weight.IsIotaPure.isWeightAtMost
Purity of weight w implies weight at most w.

Proof. Uses: def:weil-weight-local-iota-pure-predicate,def:weil-weight-iota-at-most-predicate
Replace the left side of the required inequality by the defining equality; the target becomes reflexivity. □

Lemma 7.1.16 (Purity gives the lower bound). *Uses:* def:weil-weight-local-iota-pure-predicate, def:weil-weight-iota-at-least-predicate

Lean: Weil.Weight.IsIotaPure.isWeightAtLeast
Purity of weight w implies weight at least w.

Proof. Uses: def:weil-weight-local-iota-pure-predicate,def:weil-weight-iota-at-least-predicate
Replace the right side of the required inequality by the defining equality; the target becomes reflexivity. □

Lemma 7.1.17 (The prime-power base is greater than one). *Uses:* def:weil-prime-power-data

Lean: Weil.Weight.one_lt_q
The real number q satisfies $1 < q$.

Proof. Uses: def:weil-prime-power-data
A prime natural number satisfies $2 \leq p$, and the exponent a is positive. Hence $2 \leq p \leq p^a = q$. The natural-number inequality embeds into $\mathbf{R}$, giving $1 < q$. □

Lemma 7.1.18 (Uniqueness of a fixed-embedding weight). *Uses:* def:weil-weight-local-iota-pure-predicate

Lean: Weil.Weight.isIotaPure_weight_unique
If the same nonzero scalar at degree e is pure of weights u and v, then $u = v$.

Proof. Uses: def:weil-weight-local-scalar-data,def:weil-weight-local-iota-pure-predicate,lemma:weil-weight-q-greater-than-one

The two purity equations give $(q^e)^{u/2} = (q^e)^{v/2}$. Since $q^e > 1$, the real exponential map $t \mapsto (q^e)^t$ is injective. Thus $u/2 = v/2$, hence $u = v$. □

Lemma 7.1.19 (Opposite bounds give purity)**.** Uses: def:weil-weight-local-iota-pure-predicate,def:weil-weight-iota-at-most-predicate,def:weil-weight-iota-at-least-predicate

Lean: Weil.Weight.isIotaPure_of_bounds

Weight at most w together with weight at least w implies purity of weight w.

Proof. Uses: def:weil-weight-local-iota-pure-predicate,def:weil-weight-iota-at-most-predicate,def:weil-weight-iota-at-least-predicate

Antisymmetry applied to the two displayed inequalities yields $|\iota(\alpha)| = q_e^{w/2}$, which is the purity predicate. □

7.2 Spectra, Galois stability, and weight filtrations

Definition 7.2.1 (Finite Frobenius spectrum datum)**.** Uses: def:weil-coefficient-data

Lean: Weil.Weight.FiniteSpectrumData

A finite Frobenius spectrum datum is a finite multiset S of elements of $\overline{E}^\times$. Multiplicity in S is part of the datum.

Definition 7.2.2 (Characteristic-factor realization predicate)**.** Uses: def:weil-weight-finite-spectrum-data

Lean: Weil.Weight.FiniteSpectrumData.IsCharacteristicFactor

For $P(T) \in \overline{E}[T]$, define

$$\mathrm{RealizesChar}(S, P) \quad :\Longleftrightarrow \quad P(T) = \prod_{\alpha \in S} (1 - \alpha T),$$

where the product uses multiset multiplicities.

Definition 7.2.3 (Galois action on a finite spectrum candidate)**.** Uses: def:weil-weight-finite-spectrum-data

Lean: Weil.Weight.SpectrumGaloisActionCandidate

A Galois-action candidate assigns to every $\sigma \in \mathrm{Aut}_E(\overline{E})$ a permutation $\pi_\sigma : S \simeq S$. No compatibility equation is included.

Definition 7.2.4 (Galois evaluation compatibility predicate)**.** Uses: def:weil-weight-finite-spectrum-data,def:weil-weight-spectrum-galois-action-candidate

Lean: Weil.Weight.IsSpectrumGaloisEvaluationCompatible

Define evaluation compatibility by

$$\mathrm{val}(\pi_\sigma(s)) = \sigma(\mathrm{val}(s))$$

for every σ and every occurrence s of the multiset S.

Definition 7.2.5 (Galois identity predicate)**.** Uses: def:weil-weight-spectrum-galois-action-candidate

Lean: Weil.Weight.IsSpectrumGaloisIdentity

Define the identity predicate by $\pi_{\mathrm{id}} = \mathrm{id}_S$.

Definition 7.2.6 (Galois cocycle predicate)**.** Uses: def:weil-weight-spectrum-galois-action-candidate

Lean: Weil.Weight.IsSpectrumGaloisCocycle

Define the cocycle predicate by $\pi_{\sigma\tau} = \pi_\sigma \circ \pi_\tau$ for all σ, τ.

Definition 7.2.7 (Galois-stable spectrum certificate)**.** Uses: def:weil-weight-spectrum-galois-action-candidate,def:weil-weight-spectrum-galois-evaluation-predicate,def:weil-weight-spectrum-galois-identity-predicate,def:weil-weight-spectrum-galois-cocycle-predicate

Lean: Weil.Weight.GaloisStableSpectrumCertificate

A Galois-stable spectrum certificate consists of an action candidate and proofs of evaluation compatibility, the identity equation, and the cocycle equation. Thus Galois stability is never inferred merely from the phrase "the spectrum is algebraic."

Definition 7.2.8 (Pure finite spectrum)**.** Uses: `def:weil-weight-finite-spectrum-data`, `def:weil-weight-local-iota-pure-predicate`

Lean: `Weil.Weight.FiniteSpectrumData.IsIotaPure`

At degree e, define S to be ι-pure of weight w when every occurrence $\alpha \in S$ satisfies $|\iota(\alpha)| = (q^e)^{w/2}$.

Definition 7.2.9 (Upper-bounded finite spectrum)**.** Uses: `def:weil-weight-finite-spectrum-data`, `def:weil-weight-iota-at-most-predicate`

Lean: `Weil.Weight.FiniteSpectrumData.IsIotaWeightAtMost`

At degree e, define S to have weights at most w when every occurrence in S satisfies the scalar upper-bound predicate.

Definition 7.2.10 (Lower-bounded finite spectrum)**.** Uses: `def:weil-weight-finite-spectrum-data`, `def:weil-weight-iota-at-least-predicate`

Lean: `Weil.Weight.FiniteSpectrumData.IsIotaWeightAtLeast`

At degree e, define S to have weights at least w when every occurrence in S satisfies the scalar lower-bound predicate.

Definition 7.2.11 (Direct-sum spectrum)**.** Uses: `def:weil-weight-finite-spectrum-data`

Lean: `Weil.Weight.FiniteSpectrumData.directSum`

For spectra S and T, define their direct-sum spectrum to be the multiset union $S \sqcup T$.

Lemma 7.2.12 (Purity of a direct-sum spectrum)**.** *Uses:* *`def:weil-weight-pure-spectrum-predicate`, `def:weil-weight-spectrum-direct-sum`*

Lean: *`Weil.Weight.FiniteSpectrumData.directSum_isIotaPure`*

If S and T are pure of the same weight w, then $S \sqcup T$ is pure of weight w.

Proof. Uses: `def:weil-weight-pure-spectrum-predicate`, `def:weil-weight-spectrum-direct-sum`

An occurrence in the multiset union is an occurrence of S or an occurrence of T. Apply the corresponding input equality. □

Definition 7.2.13 (Tensor-product spectrum)**.** Uses: `def:weil-weight-finite-spectrum-data`

Lean: `Weil.Weight.FiniteSpectrumData.tensorProduct`

Define

$$S \otimes T := \{\alpha\beta : (\alpha, \beta) \in S \times T\}$$

with product multiplicities.

Lemma 7.2.14 (Purity of a tensor-product spectrum)**.** *Uses:* *`def:weil-weight-pure-spectrum-predicate`, `def:weil-weight-spectrum-tensor-product`*

Lean: *`Weil.Weight.FiniteSpectrumData.tensorProduct_isIotaPure`*

If S is pure of weight u and T is pure of weight v, then $S \otimes T$ is pure of weight $u + v$.

Proof. Uses: `def:weil-weight-pure-spectrum-predicate`, `def:weil-weight-spectrum-tensor-product`, `lemma:weil-weight-iota-pure-product`

Every occurrence of $S \otimes T$ has the displayed form $\alpha\beta$. Apply the scalar product lemma to the purity equations for α and β. □

Definition 7.2.15 (Dual spectrum)**.** Uses: `def:weil-weight-finite-spectrum-data`

Lean: `Weil.Weight.FiniteSpectrumData.dual`

Define $S^\vee$ by replacing each occurrence α by α^{-1}, preserving its multiplicity.

Lemma 7.2.16 (Purity of the dual spectrum)**.** *Uses:* *`def:weil-weight-pure-spectrum-predicate`, `def:weil-weight-spectrum-dual`*

Lean: *`Weil.Weight.FiniteSpectrumData.dual_isIotaPure`*

If S is pure of weight w, then $S^\vee$ is pure of weight $-w$.

Proof. Uses: `def:weil-weight-pure-spectrum-predicate`, `def:weil-weight-spectrum-dual`, `lemma:weil-weight-iota-pure-inverse`

Each dual occurrence is α^{-1} for an occurrence α of S. Apply the scalar inverse lemma. □

Definition 7.2.17 (Tate-twisted spectrum)**.** Uses: `def:weil-weight-finite-spectrum-data`,`def:weil-tate-twist`

Lean: `Weil.Weight.FiniteSpectrumData.tateTwist`

At degree e, define $S(n)$ by replacing every occurrence α with $q^{-en}\alpha$.

Lemma 7.2.18 (Purity of a Tate-twisted spectrum)**.** *Uses:* *def:weil-weight-pure-spectrum-predicate*, *def:weil-weight-spectrum-tate-twist*

Lean: *Weil.Weight.FiniteSpectrumData.tateTwist_isIotaPure*

If S is pure of weight w, then $S(n)$ is pure of weight $w - 2n$.

Proof. Uses: `def:weil-weight-pure-spectrum-predicate`,`def:weil-weight-spectrum-tate-twist`,`lemma:weil-weight-local-tate-shift`

Apply the local Tate-shift lemma to every occurrence of S. □

Definition 7.2.19 (Subquotient spectrum certificate)**.** Uses: `def:weil-weight-finite-spectrum-data`

Lean: `Weil.Weight.SubquotientSpectrumCertificate`

A subquotient spectrum certificate from S to T is a multiplicity function m_T on $\overline{E}^{\times}$, together with the pointwise inequalities $m_T(\alpha) \le m_S(\alpha)$. This explicit certificate records the invariant-subspace characteristic-polynomial divisibility that must be proved in a geometric application.

Lemma 7.2.20 (Upper weights pass to a certified subquotient)**.** *Uses:* *def:weil-weight-spectrum-at-most-predicate*, *def:weil-weight-subquotient-spectrum-certificate*

Lean: *Weil.Weight.SubquotientSpectrumCertificate.isWeightAtMost*

If S has weights at most w and T is a certified subquotient spectrum of S, then T has weights at most w.

Proof. Uses: `def:weil-weight-spectrum-at-most-predicate`,`def:weil-weight-subquotient-spectrum-certificate`

An occurrence of α in T has positive multiplicity there. The multiplicity inequality makes its multiplicity in S positive. Apply the upper-bound predicate for S to that occurrence. □

Lemma 7.2.21 (Lower weights pass to a certified subquotient)**.** *Uses:* *def:weil-weight-spectrum-at-least-predicate*, *def:weil-weight-subquotient-spectrum-certificate*

Lean: *Weil.Weight.SubquotientSpectrumCertificate.isWeightAtLeast*

If S has weights at least w and T is a certified subquotient spectrum of S, then T has weights at least w.

Proof. Uses: `def:weil-weight-spectrum-at-least-predicate`,`def:weil-weight-subquotient-spectrum-certificate`

The multiplicity inequality sends every occurrence of T to an occurrence of S. Apply the lower-bound predicate for S. □

Definition 7.2.22 (Exact-spectrum decomposition certificate)**.** Uses: `def:weil-weight-finite-spectrum-data`

Lean: `Weil.Weight.ExactSpectrumDecompositionCertificate`

For a Frobenius-equivariant short exact sequence $0 \to A \to B \to C \to 0$, an exact-spectrum certificate is the displayed multiset identity $S_B = S_A \sqcup S_C$. It is a separate input from exactness because it requires characteristic-polynomial multiplicativity.

Lemma 7.2.23 (Upper weights are extension-closed)**.** *Uses:* *def:weil-weight-spectrum-at-most-predicate*, *def:weil-weight-exact-spectrum-decomposition-certificate*

Lean: *Weil.Weight.ExactSpectrumDecompositionCertificate.isWeightAtMost*

If S_A and S_C have weights at most w, then a certified extension spectrum S_B has weights at most w.

Proof. Uses: `def:weil-weight-spectrum-at-most-predicate`,`def:weil-weight-exact-spectrum-decomposition-certificate`

Rewrite S_B as $S_A \sqcup S_C$. Each occurrence belongs to one of the two input spectra and obeys its upper bound. □

Lemma 7.2.24 (Lower weights are extension-closed)**.** *Uses:* *def:weil-weight-spectrum-at-least-predicate*, *def:weil-weight-exact-spectrum-decomposition-certificate*

Lean: *Weil.Weight.ExactSpectrumDecompositionCertificate.isWeightAtLeast*

If S_A and S_C have weights at least w, then a certified extension spectrum S_B has weights at least w.

Proof. Uses: `def:weil-weight-spectrum-at-least-predicate,def:weil-weight-exact-spectrum-decomposition-certificate`

Use the certified multiset identity and apply the appropriate lower bound to each occurrence in its two summands. □

Definition 7.2.25 (Closed-point Frobenius spectrum family)**.** Uses: `def:weil-weight-finite-spectrum-data`

Lean: `Weil.Weight.ClosedPointSpectrumFamily`

For a finite-type k_0-scheme X and a proposed Weil sheaf $\mathcal{F}$, a closed-point spectrum family assigns to every closed point $x \in |X|$ a finite spectrum S_x, with local degree $e_x = [k(x) : k_0]$. The construction of stalks and their Frobenius actions is not contained in this definition.

Definition 7.2.26 (Pointwise purity)**.** Uses: `def:weil-weight-closed-point-spectrum-family,def:weil-weight-pure-spectrum-predicate`

Lean: `Weil.Weight.IsPointwiseIotaPure`

Define $\mathcal{F}$ to be pointwise ι-pure of weight w when S_x is pure of weight w relative to q^{e_x} for every closed point x.

Definition 7.2.27 (Pointwise upper weight bound)**.** Uses: `def:weil-weight-closed-point-spectrum-family,def:weil-weight-spectrum-at-most-predicate`

Lean: `Weil.Weight.IsPointwiseIotaWeightAtMost`

Define pointwise weights at most w by requiring the corresponding bound on every S_x.

Definition 7.2.28 (Pointwise lower weight bound)**.** Uses: `def:weil-weight-closed-point-spectrum-family,def:weil-weight-spectrum-at-least-predicate`

Lean: `Weil.Weight.IsPointwiseIotaWeightAtLeast`

Define pointwise weights at least w by requiring the corresponding bound on every S_x.

Definition 7.2.29 (Raw typed geometric-sheaf operations context)**.** Uses: `def:weil-constructible-derived-typed-context,def:weil-point-realization-typed-context,def:weil-derived-tate-twist-typed-context,def:weil-six-functor-typed-context`

Lean: `Weil.Weight.GeometricOperationsTypedContext`

Fix the same schemes, coefficient field, constructible hearts $\mathsf{Sh}_c(X, E)$, derived categories $\mathcal{D}^b_c(X, E)$, heart embeddings ι_X, cohomology-object functors $\mathcal{H}^i_X$, and six operations as in the fixed Module-02 context. A raw typed geometric-sheaf operations context consists of the following candidate data.

(i) In every $\mathsf{Sh}_c(X, E)$, a zero object candidate, finite biproduct candidates, and, for every arrow, kernel, cokernel, image, and coimage object candidates with all structure arrows. It also supplies types of subobject, quotient, and composable-pair candidates and a short-exact-sequence predicate whose arguments have literal sources and targets in this same heart.

(ii) In every $\mathcal{D}^b_c(X, E)$, a zero object candidate and, for every arrow $u : K \to L$, a cone object $\mathrm{Cone}(u)$, the three triangle arrows $K \to L \to \mathrm{Cone}(u) \to K[1]$, and proposed rotation and inverse candidates. These data use the already fixed shift functors.

(iii) A contravariant Verdier-dual candidate

$$\mathbb{D}_X : \mathcal{D}^b_c(X, E)^{\mathrm{op}} \longrightarrow \mathcal{D}^b_c(X, E),$$

together with biduality, evaluation, and coevaluation map candidates. This is distinct from both the lisse dual and finite-dimensional linear dual.

(iv) For every K, closed-support candidates for $\mathcal{H}^i_X(K)$, dimension-bound candidates for those supports and their Verdier-dual cosupports, perverse-support and perverse-cosupport predicate candidates, perverse truncation functor candidates ${}^p\tau_{\le n}, {}^p\tau_{\ge n}$, their natural maps and triangle candidates, and perverse-cohomology object candidates ${}^p\mathcal{H}^n(K)$.

This Definition supplies only objects, types, functors, and arrows. It does not assert an abelian structure, kernel universal properties, exactness, triangulated or stable axioms, support correctness, Verdier biduality, or the perverse t-structure axioms.

Definition 7.2.30 (Geometric-sheaf operations realization certificate)**.** Uses: `def:weil-weight-geometric-operations-typed-context`

Lean: `Weil.Weight.GeometricOperationsRealizationCertificate`

A realization certificate supplies, for exactly the preceding raw context: the additive and abelian-heart axioms; the kernel, cokernel, image, coimage, and exactness universal properties; the bounded triangulated or stable

axioms and the theorem that an arrow with zero cone is invertible; the Verdier-duality inverse, naturality, and triangle compatibilities; correctness of all support and dimension candidates; and the orthogonality, truncation, heart, boundedness, and Verdier-dual compatibility axioms of the perverse t-structure. Each axiom is indexed by the same X, E, objects, and arrows as the raw context.

Open obligation: `OBL-WGT-TYPED-GEOMETRIC-OPERATIONS`

> **Status.** The preceding context is a type-formation boundary, not a mathlib theorem and not a construction of geometric sheaf theory. Every later occurrence of a zero object, kernel, image, quotient, cone, Verdier dual, support dimension, or perverse truncation is a projection of this raw context or of a more specialized candidate. Any proof using its categorical properties must receive the matching realization certificate.

Definition 7.2.31 (Finite weight-filtration candidate)**.** Uses: `def:weil-weight-closed-point-spectrum-family`,`def:weil-weight-geometric-operations-typed-context`

Lean: `Weil.Weight.FiltrationCandidate`

A finite weight-filtration candidate on $\mathcal{F}$ consists of an integer interval $[a, b]$, subsheaf candidates $W_r\mathcal{F}$ for $a \le r \le b$, inclusion candidates $W_r\mathcal{F} \to W_{r+1}\mathcal{F}$, endpoint candidates $W_{a-1} = 0$, $W_b = \mathcal{F}$, and quotient candidates $\mathrm{Gr}_r^W \mathcal{F}$. No exactness or purity is included.

Definition 7.2.32 (Weight-filtration exactness predicate)**.** Uses: `def:weil-weight-filtration-candidate`,`def:weil-weight-geometric-operations-typed-context`

Lean: `Weil.Weight.IsFiltrationExact`

Define exactness by requiring, for every r, that the displayed sequence

$$0 \longrightarrow W_{r-1}\mathcal{F} \longrightarrow W_r\mathcal{F} \longrightarrow \mathrm{Gr}_r^W \mathcal{F} \longrightarrow 0$$

is exact and Frobenius-equivariant.

Definition 7.2.33 (Weight-filtration purity predicate)**.** Uses: `def:weil-weight-filtration-candidate`,`def:weil-weight-pointwise-pure-predicate`

Lean: `Weil.Weight.IsFiltrationGradedPure`

Define graded purity by requiring $\mathrm{Gr}_r^W \mathcal{F}$ to be pointwise pure of weight r for each index r.

Definition 7.2.34 (Mixedness certificate)**.** Uses: `def:weil-weight-filtration-candidate`,`def:weil-weight-filtration-exactness-predicate`,`def:weil-weight-filtration-purity-predicate`

Lean: `Weil.Weight.MixedCertificate`

A mixedness certificate consists of a finite filtration candidate, its exactness proof, and its graded-purity proof. A certificate has weights at most n when all nonzero graded indices satisfy $r \le n$, and weights at least n when they satisfy $n \le r$.

Definition 7.2.35 (The explicit two-step weight-filtration candidate)**.** Uses: `def:weil-weight-filtration-candidate`

Lean: `Weil.Weight.twoStepFiltrationCandidate`

For a sheaf $\mathcal{F}$ and an integer w, define the filtration candidate with index interval $[w, w]$ by

$$W_{w-1}\mathcal{F} := 0, \qquad W_w\mathcal{F} := \mathcal{F}, \qquad \mathrm{Gr}_w^W \mathcal{F} := \mathcal{F}.$$

Its only inclusion is the zero map $0 \to \mathcal{F}$, its quotient map is $\mathrm{id}_{\mathcal{F}}$, and its two endpoint identifications are the identity maps of 0 and $\mathcal{F}$, respectively. This is the specific witness formerly hidden in the proof of the pure-to-mixed lemma.

Lemma 7.2.36 (Left endpoint of the two-step filtration)**.** *Uses:* `def:weil-weight-two-step-filtration-candidate`

Lean: `Weil.Weight.twoStepFiltrationCandidate_leftEndpoint`

The left endpoint of the explicit candidate is $W_{w-1}\mathcal{F} = 0$.

Proof. Uses: `def:weil-weight-two-step-filtration-candidate`

Unfold the left-endpoint field of the explicit candidate. □

Lemma 7.2.37 (Right endpoint of the two-step filtration)**.** *Uses:* `def:weil-weight-two-step-filtration-candidate`

Lean: `Weil.Weight.twoStepFiltrationCandidate_rightEndpoint`

The right endpoint of the explicit candidate is $W_w\mathcal{F} = \mathcal{F}$.

Proof. Uses: def:weil-weight-two-step-filtration-candidate
Unfold the right-endpoint field of the explicit candidate. □

Lemma 7.2.38 (Graded piece of the two-step filtration). *Uses: def:weil-weight-two-step-filtration-candidate*

Lean: *Weil.Weight.twoStepFiltrationCandidate_gradedPiece*
The sole graded piece of the explicit candidate is $\mathrm{Gr}_w^W \mathcal{F} = \mathcal{F}$*, with comparison map the identity.*

Proof. Uses: def:weil-weight-two-step-filtration-candidate
Unfold the graded-piece and quotient-map fields of the explicit candidate. □

Lemma 7.2.39 (Exactness of the two-step filtration). *Uses: def:weil-weight-two-step-filtration-candidate, def:weil-weight-filtration-exactness-predicate, def:weil-weight-geometric-operations-realization-certificate*

Lean: *Weil.Weight.twoStepFiltrationCandidate_isExact*
The explicit two-step candidate satisfies the weight-filtration exactness predicate.

Proof. Uses: def:weil-weight-two-step-filtration-candidate,def:weil-weight-filtration-exactness-predicate,def:weil-weight-geometric-operations-realization-certificate,lemma:weil-weight-two-step-left-endpoint,lemma:weil-weight-two-step-right-endpoint,lemma:weil-weight-two-step-graded-piece
After the three displayed rewrites, the only sequence is

$$0 \longrightarrow 0 \longrightarrow \mathcal{F} \xrightarrow{\mathrm{id}_{\mathcal{F}}} \mathcal{F} \longrightarrow 0.$$

Its image–kernel equalities are $0 = 0$ at the first two terms and $\mathcal{F} = \mathcal{F}$ at the last two terms. All Frobenius squares contain only a zero map or an identity map and hence commute by composition with zero and identity. □

Lemma 7.2.40 (Graded purity of the two-step filtration). *Uses: def:weil-weight-two-step-filtration-candidate, def:weil-weight-pointwise-pure-predicate, def:weil-weight-filtration-purity-predicate*

Lean: *Weil.Weight.twoStepFiltrationCandidate_isGradedPure*
If $\mathcal{F}$ *is pointwise pure of weight* w*, then the explicit two-step candidate satisfies the graded-purity predicate.*

Proof. Uses: def:weil-weight-filtration-purity-predicate,lemma:weil-weight-two-step-graded-piece
Rewrite the sole graded piece by the preceding lemma. The remaining goal is exactly the input pointwise-purity predicate for $\mathcal{F}$. □

Definition 7.2.41 (The explicit two-step mixedness certificate). Uses: def:weil-weight-two-step-filtration-candidate,def:weil-weight-filtration-exactness-predicate,def:weil-weight-geometric-operations-realization-certificate,def:weil-weight-filtration-purity-predicate,def:weil-weight-mixed-certificate

Lean: Weil.Weight.twoStepMixedCertificate
For a pointwise pure $\mathcal{F}$ of integral weight w, define its two-step mixedness certificate to have the explicit two-step filtration as its underlying filtration, the immediately preceding exactness proof as its exactness field, and the immediately preceding graded-purity proof as its graded-purity field. Its lower and upper nonzero weight indices are both w.

Constructed by: lemma:weil-weight-two-step-filtration-exact,lemma:weil-weight-two-step-filtration-graded-pure

Lemma 7.2.42 (A pure sheaf has the explicit two-step mixedness certificate). *Uses: def:weil-weight-pointwise-pure-predicate, def:weil-weight-two-step-mixed-certificate*

Lean: *Weil.Weight.MixedCertificate.ofPure*
The explicitly defined certificate is concentrated in weight w*, hence has weights both at most and at least* w*.*

Proof. Uses: def:weil-weight-two-step-mixed-certificate,def:weil-weight-mixed-certificate
Unfold the two stored endpoint indices. The only nonzero graded index is w, so the two required inequalities reduce to $w \le w$. □

7.3 The PUR chain of Weil II 1.3–1.5

Definition 7.3.1 (ι-real local characteristic factor)**.** Uses: `def:weil-weight-spectrum-characteristic-factor-predicate`

Lean: `Weil.Weight.PUR.IsIotaRealLocalFactor`

For a finite spectrum S_x, define ι-reality at x to mean that every coefficient of

$$\iota\left(\prod_{\alpha\in S_x}(1-\alpha T)\right)$$

is real. This is Deligne Variant (1.2.13), printed p. 156 (PDF p. 21), expressed without suppressing the selected embedding.

Definition 7.3.2 (ι-real Weil-sheaf predicate)**.** Uses: `def:weil-weight-closed-point-spectrum-family,def:weil-pur-iota-real-local-factor-predicate`

Lean: `Weil.Weight.PUR.IsIotaRealSheaf`

A closed-point spectrum family is ι-real when its local characteristic factor is ι-real at every closed point.

Definition 7.3.3 (Paired closed-point spectrum-family candidate)**.** Uses: `def:weil-weight-closed-point-spectrum-family,def:weil-weight-spectrum-direct-sum,def:weil-weight-spectrum-dual,def:weil-weight-spectrum-tate-twist`

Lean: `Weil.Weight.PUR.pairedSpectrumFamilyCandidate`

For a closed-point spectrum family $(S_x)_x$ and an integer w, define the paired spectrum at a degree-e closed point by

$$S_x^{\mathrm{pair}} := S_x \sqcup q^{ew} S_x^{\vee}.$$

This definition is finite-multiset algebra only; it does not assert that an actual sheaf has this stalk spectrum.

Definition 7.3.4 (Raw typed lisse-heart operations context)**.** Uses: `def:weil-constructible-derived-typed-context,def:weil-point-realization-typed-context,def:weil-derived-tate-twist-typed-context,def:weil-six-functor-typed-context,def:weil-weight-geometric-operations-typed-context`

Lean: `Weil.Weight.LisseHeartOperationsTypedContext`

Fix one complete typed six-functor context. A raw typed lisse-heart operations context over exactly the same schemes, coefficient field, constructible hearts, and derived categories consists of the following data.

(i) For every scheme X, a category $\mathsf{Lis}(X,E)$ and a carrier functor

$$U_X : \mathsf{Lis}(X,E) \longrightarrow \mathsf{Sh}_c(X,E), \qquad \jmath_X := \iota_X \circ U_X : \mathsf{Lis}(X,E) \longrightarrow \mathcal{D}_c^b(X,E).$$

No full-faithfulness or essential-image predicate is asserted.

(ii) For every finite type A and family $(\mathcal{L}_a)_{a\in A}$ in $\mathsf{Lis}(X,E)$, an object $\bigoplus^{\mathrm{lis}}_{a\in A}\mathcal{L}_a$, inclusion candidates, and projection candidates. Supply in parallel finite-direct-sum candidate objects in $\mathsf{Sh}_c(X,E)$ and $\mathcal{D}_c^b(X,E)$, and forward and reverse comparison-map candidates from the carried lisse sum to those two sums. Write $\mathcal{L}\boxplus\mathcal{M}$ for the binary lisse candidate. The biproduct equations and universal property are not fields.

(iii) A contravariant functor candidate

$$\mathbb{D}_X^{\mathrm{lis}} : \mathsf{Lis}(X,E)^{\mathrm{op}} \longrightarrow \mathsf{Lis}(X,E), \qquad \mathcal{L}^{\vee_{\mathrm{lis}}} := \mathbb{D}_X^{\mathrm{lis}}(\mathcal{L}),$$

called the lisse dual. It is not the Verdier dual and not the dual of a realized finite-dimensional Frobenius module.

(iv) For every $n\in\mathbf{Z}$, an endofunctor candidate

$$(n)_X^{\mathrm{lis}} : \mathsf{Lis}(X,E) \longrightarrow \mathsf{Lis}(X,E), \qquad \mathcal{L}(n)_{\mathrm{lis}} := (n)_X^{\mathrm{lis}}\mathcal{L}.$$

Supply comparison-map candidates in $\mathcal{D}_c^b(X,E)$

$$\jmath_X\big(\mathcal{L}(n)_{\mathrm{lis}}\big) \rightleftarrows \big(\jmath_X\mathcal{L}\big)(n)$$

and

$$\jmath_X(\mathcal{L}^{\vee_{\mathrm{lis}}}) \rightleftarrows R\mathcal{H}om_X(\jmath_X\mathcal{L}, E_X).$$

These are typed candidates only.

All displayed categories, functors, finite indices, and comparison maps must use definitionally identical $X, E, \mathsf{Sh}_c(X,E)$, and $\mathcal{D}_c^b(X,E)$ indices from the fixed six-functor context. No inhabitant, naturality, exactness, biproduct identity, duality identity, twist coherence, or geometric-stalk compatibility is asserted by this Definition. The actual paired-sheaf direct-sum, dual, twist, and stalk comparisons remain precisely OBL-WGT-PUR-REAL.

Definition 7.3.5 (Typed paired lisse-sheaf candidate)**.** Uses: `def:weil-lisse-heart-operations-typed-context,def:weil-weight-closed-point-spectrum-family`

Lean: `Weil.Weight.PUR.pairedSheafCandidate`

For $\mathcal{F} \in \mathsf{Lis}(X, E)$, use the fixed raw lisse-heart operations context to define, in order,

$$\mathcal{F}^{\vee_{\mathrm{lis}}} := \mathbb{D}_X^{\mathrm{lis}}(\mathcal{F}), \qquad \mathcal{H} := \mathcal{F}^{\vee_{\mathrm{lis}}}(-w)_{\mathrm{lis}},$$

$$\mathcal{G}_{\mathrm{lis}} := \mathcal{F} \boxplus \mathcal{H} \in \mathsf{Lis}(X, E), \qquad \mathcal{G} := U_X(\mathcal{G}_{\mathrm{lis}}) \in \mathsf{Sh}_c(X, E), \qquad \mathcal{G}^D := \iota_X \mathcal{G} \in \mathcal{D}_c^b(X, E).$$

This Definition selects only typed candidate objects. It does not use a heart operation absent from the context and does not assert that their geometric-stalk spectra equal the proposed paired spectrum.

Definition 7.3.6 (Paired closed-point stalk Frobenius candidate data)**.** Uses: `def:weil-pur-paired-sheaf-candidate,def:weil-point-realization-typed-context,def:weil-weight-finite-spectrum-data`

Lean: `Weil.Weight.PUR.PairedStalkFrobeniusCandidateData`

For every closed point x, and separately for the carried sheaves $U_X\mathcal{F}$, $U_X\mathcal{F}^{\vee_{\mathrm{lis}}}$, $U_X\mathcal{H}$, and $\mathcal{G}$, select the geometric-stalk vector space by the fixed functor $\mathrm{st}_{\bar{x}}$. For

$$A \in \{\mathcal{F}, \mathcal{F}^{\vee_{\mathrm{lis}}}, \mathcal{H}, \mathcal{G}\}$$

select an independent finite index type $J_{x,A}$, forward and reverse linear comparison-map candidates with $E^{J_{x,A}}$, a geometric-Frobenius linear endomorphism candidate and a proposed inverse, and a finite-spectrum candidate for that endomorphism. The four finite types need not have equal cardinality. All four families use the same geometric point, residue degree, coefficient field, and chosen descent Frobenius. No basis inverse equation, finite-dimensionality proof, automorphism equation, or characteristic-factor identity is part of this raw datum.

Definition 7.3.7 (Paired sheaf-to-spectrum comparison candidate)**.** Uses: `def:weil-pur-paired-spectrum-family-candidate,def:weil-pur-paired-sheaf-candidate,def:weil-pur-paired-stalk-frobenius-candidate-data`

Lean: `Weil.Weight.PUR.PairedSheafSpectrumComparisonCandidate`

For every closed point x, define forward and reverse occurrence-map candidates between the finite-spectrum candidate selected for $\mathrm{st}_{\bar{x}}(\mathcal{G})$ and S_x^{pair}. The source is not called the actual geometric stalk spectrum: its finite-dimensional realization, Frobenius automorphism, and characteristic-factor identity are still only proposed data. No inverse equation or multiplicity statement is built into this raw comparison.

Definition 7.3.8 (Paired sheaf-to-spectrum realization certificate)**.** Uses: `def:weil-pur-paired-sheaf-spectrum-comparison-candidate`

Lean: `Weil.Weight.PUR.PairedSheafSpectrumRealizationCertificate`

A realization certificate first proves the forward and reverse finite-model maps to be inverse, hence proves finite-dimensionality; proves each proposed Frobenius endomorphism and inverse to be mutually inverse; and identifies each selected finite spectrum with the corresponding characteristic factor. It then supplies both inverse occurrence equations, equality of multiplicities, the stalk/direct-sum comparison, the lisse-dual versus inverse-transpose comparison, the degree-e Tate multiplier q^{-en}, and every geometric-Frobenius commutative square. This is the only geometric sheaf-to-spectrum realization part of paired reality left open.

Open obligation: `OBL-WGT-PUR-REAL`

Definition 7.3.9 (Pure paired real-sheaf input datum)**.** Uses: `def:weil-pur-paired-spectrum-family-candidate,def:weil-pur-paired-sheaf-candidate,def:weil-pur-paired-stalk-frobenius-candidate-data,def:weil-pur-paired-sheaf-spectrum-comparison-candidate,def:weil-weight-pointwise-pure-predicate`

Lean: `Weil.Weight.PUR.PairedRealSheafInputData`

A pure paired input fixes $\mathcal{F}$, an integer w, its closed-point spectrum family, a proof that this family is pointwise ι-pure of weight w, the typed sheaf $\mathcal{G}$, and the preceding proposed stalk-spectrum comparisons. Thus pointwise purity is an input to paired reality, not a consequence of the real-sheaf theorem.

Definition 7.3.10 (Paired occurrence conjugation witness)**.** Uses: `def:weil-pur-paired-real-sheaf-candidate`

Lean: `Weil.Weight.PUR.pairedOccurrenceConjugationWitness`

For a degree-e point and an occurrence $\alpha \in S_x$, define its paired occurrence by the explicit scalar

$$\alpha^{\mathrm{pair}} := q^{ew}\alpha^{-1}.$$

Nonvanishing of α is part of the finite-spectrum datum, so the inverse is defined without a choice.

Lemma 7.3.11 (Squared modulus of a pure occurrence). *Uses:* def:weil-pur-paired-real-sheaf-candidate

Lean: Weil.Weight.PUR.pureOccurrence_modulusSq

For every $\alpha \in S_x$ at a degree-e point,

$$|\iota(\alpha)|^2 = q^{ew}.$$

Proof. Uses: def:weil-weight-pointwise-pure-predicate,def:weil-weight-pure-spectrum-predicate

The input purity equation is $|\iota(\alpha)| = (q^e)^{w/2}$. Square both sides and use $((q^e)^{w/2})^2 = q^{ew}$. □

Lemma 7.3.12 (The paired occurrence is the complex conjugate occurrence). *Uses:* def:weil-pur-paired-real-sheaf-candidate, def:weil-pur-paired-occurrence-conjugation-witness

Lean: Weil.Weight.PUR.pairedOccurrence_isConjugate

For every occurrence α,

$$\iota(\alpha^{\mathrm{pair}}) = \overline{\iota(\alpha)}.$$

Proof. Uses: def:weil-pur-paired-occurrence-conjugation-witness,lemma:weil-pur-pure-occurrence-modulus-square

Put $z = \iota(\alpha)$. Since $z \neq 0$, $\overline{z} = |z|^2 z^{-1}$. Replace $|z|^2$ by q^{ew}, and use that the coefficient embedding fixes the rational scalar q^{ew} and sends inverses to inverses. The result is $\overline{z} = \iota(q^{ew}\alpha^{-1})$. □

Lemma 7.3.13 (The dual-twist factor is coefficientwise conjugate). *Uses:* def:weil-pur-paired-real-sheaf-candidate, def:weil-pur-iota-real-local-factor-predicate

Lean: Weil.Weight.PUR.dualTwistFactor_isConjugate

At every closed point, the characteristic factor of $q^{ew}S_x^\vee$, after applying ι, is the coefficientwise complex conjugate of the characteristic factor of S_x.

Proof. Uses: def:weil-weight-spectrum-characteristic-factor-predicate,lemma:weil-pur-paired-occurrence-is-conjugate

Apply the occurrence conjugation identity to each factor $1-\alpha T$. Complex conjugation is a ring automorphism and therefore commutes with the finite product indexed by the occurrence multiset, including multiplicities. □

Lemma 7.3.14 (The paired spectrum factor is real). *Uses:* def:weil-pur-paired-real-sheaf-candidate, def:weil-pur-iota-real-local-factor-predicate

Lean: Weil.Weight.PUR.pairedSpectrumFactor_isReal

The characteristic factor of S_x^{pair} has real coefficients after applying ι.

Proof. Uses: def:weil-weight-spectrum-direct-sum,lemma:weil-pur-dual-twist-factor-is-conjugate

The direct-sum factor is $A_x(T)\overline{A_x(T)}$. Its conjugate is $\overline{A_x(T)}A_x(T)$, equal to the original polynomial by commutativity. Hence every coefficient is fixed by conjugation and is real. □

Definition 7.3.15 (Explicit paired-factor reality certificate). Uses: def:weil-pur-paired-real-sheaf-candidate,def:weil-pur-paired-sheaf-spectrum-realization-certificate,def:weil-pur-iota-real-sheaf-predicate

Lean: Weil.Weight.PUR.pairedFactorRealityCertificateOfPure

Define the paired-factor reality certificate by combining the supplied actual-sheaf spectrum-realization certificate with the explicit occurrence map $\alpha \mapsto q^{ew}\alpha^{-1}$, its coefficientwise factor identity, and the real-product proof. No invocation of the PUR real-sheaf endpoint occurs in this construction.

Constructed by: lemma:weil-pur-paired-occurrence-is-conjugate,lemma:weil-pur-dual-twist-factor-is-conjugate,lemma:weil-pur-paired-factor-product-real

Lemma 7.3.16 (The pure paired candidate is real). *Uses:* def:weil-pur-paired-real-sheaf-candidate, def:weil-pur-paired-factor-reality-certificate, def:weil-pur-iota-real-sheaf-predicate

Lean: Weil.Weight.PUR.pairedRealShеafCandidate_isIotaReal

The actual paired sheaf $\mathcal{G}$ is ι-real.

Proof. Uses: def:weil-pur-paired-sheaf-spectrum-realization-certificate,lemma:weil-pur-paired-factor-product-real

At each closed point, use the realization certificate to replace the actual stalk factor by the paired spectrum factor, and then apply the preceding real-product lemma. □

Definition 7.3.17 (Curve class-field reciprocity candidate). Uses: `def:weil-weight-closed-point-spectrum-family`

Lean: `Weil.Weight.PUR.ClassFieldReciprocityCandidate`

For a smooth geometrically connected curve U/k_0, a class-field reciprocity candidate consists of the degree map on divisors, the abelianized geometric fundamental-group quotient, and a proposed reciprocity homomorphism from degree-zero divisor classes to that quotient. These objects are listed separately; no reciprocity property is asserted.

Definition 7.3.18 (Principal-divisor compatibility predicate). Uses: `def:weil-pur-class-field-reciprocity-candidate`

Lean: `Weil.Weight.PUR.IsPrincipalDivisorCompatible`

Define principal-divisor compatibility by requiring every principal degree-zero divisor to map to the identity.

Definition 7.3.19 (Closed-point Frobenius reciprocity predicate). Uses: `def:weil-pur-class-field-reciprocity-candidate`

Lean: `Weil.Weight.PUR.IsClosedPointFrobeniusCompatible`

Define Frobenius reciprocity by requiring the class of a closed point x to map to its geometric Frobenius class, after subtracting the selected degree-correcting divisor.

Definition 7.3.20 (Finite degree-zero divisor-class predicate). Uses: `def:weil-pur-class-field-reciprocity-candidate`

Lean: `Weil.Weight.PUR.IsDegreeZeroClassGroupFinite`

Define the finiteness predicate by requiring the degree-zero divisor-class group in the candidate to be finite.

Definition 7.3.21 (Rank-one geometric-monodromy decomposition candidate). Uses: `def:weil-pur-class-field-reciprocity-candidate,def:weil-lisse-heart-operations-typed-context`

Lean: `Weil.Weight.PUR.RankOneDecompositionCandidate`

For a rank-one lisse Weil sheaf, a decomposition candidate consists of a finite-order geometric character χ_{fin}, a scalar $c \in \overline{E}^{\times}$ assigned to degree one, and a proposed factorization

$$\alpha_x = \chi_{\mathrm{fin}}(\mathrm{Frob}_x)c^{\deg(x)}$$

for every closed point x.

Definition 7.3.22 (Rank-one decomposition certificate). Uses: `def:weil-pur-class-field-principal-predicate,def:weil-pur-class-field-frobenius-predicate,def:weil-pur-class-field-degree-zero-finite-predicate,def:weil-pur-rank-one-decomposition-candidate`

Lean: `Weil.Weight.PUR.RankOneDecompositionCertificate`

A rank-one decomposition certificate consists of the three class-field predicates, the pointwise factorization equations, and proofs that every value of χ_{fin} is a root of unity. Constructing this certificate is the class-field-theory obligation in Deligne Theorem (1.3.1), printed pp. 156–157 (PDF pp. 21–22).

Open obligation: `OBL-WGT-PUR-CLASSFIELD`

Lemma 7.3.23 (Rank-one local modulus from a decomposition certificate). *Uses:* `def:weil-pur-rank-one-decomposition-certificate,def:weil-weight-local-iota-pure-predicate`

Lean: `Weil.Weight.PUR.RankOneDecompositionCertificate.localModulus`

If $|\iota(c)| = q^{w/2}$, then every local eigenvalue in the certified rank-one decomposition is pure of weight w.

Proof. Uses: `def:weil-pur-rank-one-decomposition-candidate,def:weil-pur-rank-one-decomposition-certificate,def:weil-weight-local-iota-pure-predicate`

At degree e, the factorization is $\alpha_x = \zeta_x c^e$, where ζ_x is a root of unity. Its complex absolute value is one. Therefore

$$|\iota(\alpha_x)| = |\iota(c)|^e = (q^{w/2})^e = (q^e)^{w/2}.$$

□

Definition 7.3.24 (Determinant spectrum). Uses: `def:weil-weight-finite-spectrum-data`

Lean: `Weil.Weight.PUR.FiniteSpectrumData.determinant`

For $S = [\alpha_1, \dots, \alpha_r]$, define the determinant spectrum to be the singleton multiset

$$\det(S) := [\alpha_1 \cdots \alpha_r].$$

Definition 7.3.25 (Determinantal weight datum). Uses: `def:weil-weight-closed-point-spectrum-family`, `def:weil-pur-determinant-spectrum`

Lean: `Weil.Weight.PUR.DeterminantalWeightData`

For a nonzero rank r, a determinantal weight datum consists of a real number d and, at every closed point x, the predicate that the product of the r local eigenvalues is pure of weight rd. The normalized number d is Deligne's determinantal weight from Definition (1.3.5), printed p. 158 (PDF p. 23).

Lemma 7.3.26 (Determinant of a pure spectrum). *Uses:* `def:weil-weight-pure-spectrum-predicate`, `def:weil-pur-determinant-spectrum`

Lean: `Weil.Weight.PUR.FiniteSpectrumData.determinant_isIotaPure`

If a rank-r spectrum is pure of weight w, then its determinant spectrum is pure of weight rw.

Proof. Uses: `def:weil-pur-determinant-spectrum`, `lemma:weil-weight-iota-pure-product`

Induct on the finite multiset. The empty product has weight 0. At the successor step, multiply the new weight-w scalar by the product of the previous r scalars and apply the product lemma. The weight changes from rw to $(r+1)w$. □

Definition 7.3.27 (Algebraic monodromy group candidate). Uses: `def:weil-weight-closed-point-spectrum-family`

Lean: `Weil.Weight.PUR.AlgebraicMonodromyGroupCandidate`

For a lisse representation $\rho : \pi_1(U, \bar{u}) \to \mathrm{GL}(V)$, define the algebraic monodromy candidate G to be the specified Zariski closure of $\rho(\pi_1(U, \bar{u}))$, together with its geometric subgroup candidate G^{geom} and component-group candidate. Closure, subgroup, and component comparison properties remain separate.

Definition 7.3.28 (Zariski-density predicate for monodromy). Uses: `def:weil-pur-algebraic-monodromy-group-candidate`

Lean: `Weil.Weight.PUR.IsMonodromyZariskiDense`

Define density by requiring every regular function on G vanishing on the representation image to vanish on G.

Definition 7.3.29 (Unipotent-radical triviality predicate). Uses: `def:weil-pur-algebraic-monodromy-group-candidate`

Lean: `Weil.Weight.PUR.IsUnipotentRadicalTrivialOnSemisimplification`

Define the predicate by requiring the unipotent radical of G^{geom} to act trivially on the selected semisimplification of V.

Definition 7.3.30 (Reductive-quotient comparison candidate). Uses: `def:weil-pur-algebraic-monodromy-group-candidate`

Lean: `Weil.Weight.PUR.ReductiveQuotientComparisonCandidate`

A reductive-quotient comparison candidate consists of a proposed quotient $G^{\mathrm{geom}} \to G^{\mathrm{red}}$, a representation of G^{red} on the semisimplification, and forward and inverse comparison maps to the original action.

Definition 7.3.31 (Reductive comparison certificate). Uses: `def:weil-pur-monodromy-density-predicate`, `def:weil-pur-unipotent-radical-triviality-predicate`, `def:weil-pur-reductive-quotient-comparison-candidate`

Lean: `Weil.Weight.PUR.ReductiveComparisonCertificate`

A reductive comparison certificate supplies the density and radical predicates, the two inverse equations for the comparison maps, and equivariance with the G^{red}-action. Its construction is the group-theoretic content of Deligne (1.3.7)–(1.3.8), printed pp. 158–160 (PDF pp. 23–25), including the cited SGA 3 reductive-group input.

Open obligation: `OBL-WGT-PUR-MONODROMY-GROUP`

Definition 7.3.32 (Central positive-degree element candidate). Uses: `def:weil-pur-algebraic-monodromy-group-candidate`

Lean: `Weil.Weight.PUR.CentralDegreeElementCandidate`

A central positive-degree element candidate is a specified $z \in G(\overline{E})$, a positive integer m, and the proposed properties that z is central and projects to geometric Frobenius degree m.

Definition 7.3.33 (Central weight-detection predicate). Uses: `def:weil-pur-central-degree-element-candidate`, `def:weil-pur-determinantal-weight-data`

Lean: `Weil.Weight.PUR.IsCentralWeightDetecting`

Define central weight detection by requiring every irreducible G^{red}-constituent of determinantal weight d to have z-eigenvalues of complex modulus $q^{md/2}$.

Definition 7.3.34 (Central weight-detection certificate)**.** Uses: `def:weil-pur-reductive-comparison-certificate`, `def:weil-pur-central-degree-element-candidate`, `def:weil-pur-central-weight-detection-predicate`

Lean: `Weil.Weight.PUR.CentralWeightDetectionCertificate`

A central weight-detection certificate consists of the reductive comparison, the centrality and degree equations, and the weight-detection predicate. This separates Deligne Lemma (1.3.10) and Corollaries (1.3.11)–(1.3.12), printed pp. 160–161 (PDF pp. 25–26).

Open obligation: `OBL-WGT-PUR-CENTRAL`

Definition 7.3.35 (Exterior-power spectrum)**.** Uses: `def:weil-weight-finite-spectrum-data`

Lean: `Weil.Weight.PUR.FiniteSpectrumData.exteriorPower`

For $0 \le s \le r$, define $\bigwedge^s S$ to be the multiset of products $\alpha_{i_1} \cdots \alpha_{i_s}$ over strictly increasing s-tuples, with the induced multiplicities.

Lemma 7.3.36 (Purity of an exterior-power spectrum)**.** *Uses:* `def:weil-weight-pure-spectrum-predicate`, `def:weil-pur-exterior-power-spectrum`

Lean: `Weil.Weight.PUR.FiniteSpectrumData.exteriorPower_isIotaPure`

If S is pure of weight w, then $\bigwedge^s S$ is pure of weight sw.

Proof. Uses: `def:weil-weight-pure-spectrum-predicate`, `def:weil-pur-exterior-power-spectrum`, `lemma:weil-weight-iota-pure-product`

Each exterior occurrence is a product of exactly s occurrences of S. Induction on s and the product lemma give weight sw. □

Definition 7.3.37 (Low-degree curve cohomology candidates)**.** Uses: `def:weil-weight-closed-point-spectrum-family`

Lean: `Weil.Weight.PUR.LowDegreeCurveCohomologyCandidates`

For a smooth curve U and lisse $\mathcal{F}$, define candidates for

$$H^0(U_{\bar{k}}, \mathcal{F}), \quad H^0_c(U_{\bar{k}}, \mathcal{F}), \quad H^2_c(U_{\bar{k}}, \mathcal{F}),$$

their geometric Frobenius maps, and their finite spectra. Kernels, compact-support maps, and the duality identification for H^2_c are listed as maps, not asserted as isomorphisms.

Definition 7.3.38 (Low-degree curve calculation certificate)**.** Uses: `def:weil-pur-low-degree-curve-cohomology-candidates`, `def:weil-weight-spectrum-at-most-predicate`, `def:weil-duality-perfect-pairing-certificate`

Lean: `Weil.Weight.PUR.LowDegreeCurveCertificate`

A low-degree certificate supplies the invariant description of H^0, the support vanishing of H^0_c on a nonproper connected curve, the Poincare-dual description of H^2_c with twist (-1), and the induced Frobenius-equivariant spectral bounds. It expands Deligne (1.4.1)–(1.4.4), printed pp. 162–163 (PDF pp. 27–28).

Open obligation: `OBL-WGT-PUR-LOW-DEGREE`

Definition 7.3.39 (Cohomological L-function candidate)**.** Uses: `def:weil-weight-closed-point-spectrum-family`, `def:weil-pur-low-degree-curve-cohomology-candidates`

Lean: `Weil.Weight.PUR.CohomologicalLFunctionCandidate`

Define the Euler candidate

$$L(U, \mathcal{F}, T) := \prod_{x \in |U|} \prod_{\alpha \in S_x} (1 - \alpha T^{\deg x})^{-1}$$

and the cohomological determinant candidate

$$\prod_{i=0}^{2} \det(1 - TF \mid H^i_c(U_{\bar{k}}, \mathcal{F}))^{(-1)^{i+1}}.$$

The infinite product and determinant expression are separate objects.

Definition 7.3.40 (Trace–L-function comparison certificate)**.** Uses: `def:weil-pur-cohomological-l-function-candidate`

Lean: `Weil.Weight.PUR.TraceLFunctionComparisonCertificate`

A trace–L-function certificate supplies coefficientwise convergence in formal power series and equality of the two candidates, obtained from the Grothendieck–Lefschetz trace formula for every extension degree. It is the input used at Deligne (1.4.5.1), printed p. 163 (PDF p. 28), and is not a foundation leaf.

Open obligation: `OBL-WGT-PUR-TRACE-L`

Definition 7.3.41 (Point-count growth stratification candidate). Uses: `def:weil-pur-cohomological-l-function-candidate`

Lean: `Weil.Weight.PUR.PointGrowthStratificationCandidate`

A point-growth stratification candidate is a finite locally closed stratification of U, a quasi-finite map from each stratum to an affine space, and constants proposed to bound the number of degree-n points on each stratum.

Definition 7.3.42 (Euler-product convergence certificate). Uses: `def:weil-pur-cohomological-l-function-candidate,def:weil-pur-point-growth-stratification-candidate`

Lean: `Weil.Weight.PUR.EulerConvergenceCertificate`

A convergence certificate supplies the quasi-finite point-count bounds, absolute convergence of the logarithmic Euler series on the claimed open disk, and uniform convergence on each smaller closed disk. This is the expanded input of Deligne Proposition (1.4.6), printed p. 163 (PDF p. 28).

Open obligation: `OBL-WGT-PUR-CONVERGENCE`

Definition 7.3.43 (Pole-detection circle candidate). Uses: `def:weil-pur-cohomological-l-function-candidate`

Lean: `Weil.Weight.PUR.PoleDetectionCircleCandidate`

For a proposed maximal eigenvalue modulus R, define the candidate circle $|T| = R^{-1}$, the numerator zero multiset, and the denominator pole multiset of the rational cohomological L-function.

Definition 7.3.44 (Pole-detection certificate). Uses: `def:weil-pur-pole-detection-circle-candidate,def:weil-pur-euler-convergence-certificate,def:weil-pur-trace-l-function-certificate`

Lean: `Weil.Weight.PUR.PoleDetectionCertificate`

A pole-detection certificate supplies: absence of denominator poles in the convergence disk; the cancellation multiplicity comparison on its boundary; reality of the local Euler coefficients; and the sign argument preventing the selected extremal pole from cancelling. These are the analytic inputs of Deligne (1.4.6)–(1.4.7) and (1.5.2).

Open obligation: `OBL-WGT-PUR-POLE-DETECTION`

Definition 7.3.45 (Numerical ι-weight of a local scalar). Uses: `def:weil-weight-local-scalar-data`

Lean: `Weil.Weight.PUR.LocalScalarData.numericalWeight`

Define

$$\nu_\iota(e,\alpha) := \frac{2\log|\iota(\alpha)|}{\log(q^e)} \in \mathbf{R}.$$

The denominator is nonzero because $q^e > 1$.

Lemma 7.3.46 (A scalar is pure of its numerical weight). *Uses:* `def:weil-pur-local-numerical-weight,def:weil-weight-local-iota-pure-predicate`

Lean: `Weil.Weight.PUR.LocalScalarData.isIotaPure_numericalWeight`

Every local scalar is ι-pure of weight $\nu_\iota(e,\alpha)$.

Proof. Uses: `def:weil-weight-local-scalar-data,def:weil-pur-local-numerical-weight,def:weil-weight-local-iota-pure-predicate,lemma:weil-weight-q-greater-than-one`

Positivity of $|\iota(\alpha)|$ and $q^e > 1$ permits logarithms. Compute

$$(q^e)^{\nu_\iota(e,\alpha)/2} = \exp\left(\log(q^e)\frac{\log|\iota(\alpha)|}{\log(q^e)}\right) = |\iota(\alpha)|.$$

□

Definition 7.3.47 (Tensor-power limiting-inequality certificate). Uses: `def:weil-pur-local-numerical-weight,def:weil-pur-determinantal-weight-data,def:weil-pur-pole-detection-certificate`

Lean: `Weil.Weight.PUR.TensorPowerLimitCertificate`

A tensor-power limit certificate for an irreducible constituent of determinantal weight d assigns, to every local eigenvalue α and every positive integer k, the inequality

$$\nu_\iota(e,\alpha) \le d + \frac{1}{k}.$$

It also records that this inequality is obtained from the k-fold tensor power, central weight detection, low-degree cohomology, and pole detection. This separates all inputs of Deligne Lemma (1.5.2), printed p. 164 (PDF p. 29).

Open obligation: `OBL-WGT-PUR-TENSOR-LIMIT`

Lemma 7.3.48 (Removing the tensor-power error). *Uses:* *def:weil-pur-tensor-power-limit-certificate*, *def:weil-pur-local-numerical-weight*

Lean: *Weil.Weight.PUR.TensorPowerLimitCertificate.numericalWeight_le*

The tensor-power inequalities imply $\nu_\iota(e,\alpha) \le d$ for every local eigenvalue.

Proof. Uses: def:weil-pur-tensor-power-limit-certificate

Assume $d < \nu_\iota(e,\alpha)$ and put $\varepsilon = \nu_\iota(e,\alpha) - d > 0$. The Archimedean property supplies a positive integer k with $1/k < \varepsilon$. The certified inequality gives $\nu_\iota(e,\alpha) \le d + 1/k < d + \varepsilon = \nu_\iota(e,\alpha)$, a contradiction. □

Definition 7.3.49 (Determinant product identity certificate). Uses: def:weil-pur-determinantal-weight-data, def:weil-pur-local-numerical-weight

Lean: Weil.Weight.PUR.DeterminantProductIdentityCertificate

For a rank-r local spectrum $[\alpha_1, \dots, \alpha_r]$, a determinant product identity certificate stores the equality

$$\alpha_1 \cdots \alpha_r = \delta_x$$

and the determinantal purity equation $|\iota(\delta_x)| = (q^e)^{rd/2}$.

Lemma 7.3.50 (Sum of numerical weights equals determinant weight). *Uses:* *def:weil-pur-local-numerical-weight*, *def:weil-pur-determinant-product-identity-certificate*

Lean: *Weil.Weight.PUR.DeterminantProductIdentityCertificate.sum_numericalWeight*

For the certified rank-r spectrum,

$$\sum_{j=1}^{r} \nu_\iota(e,\alpha_j) = rd.$$

Proof. Uses: def:weil-pur-local-numerical-weight, def:weil-pur-determinant-product-identity-certificate

Take complex absolute values in the product identity and then take logarithms:

$$\sum_j \log|\iota(\alpha_j)| = \log|\iota(\delta_x)| = \frac{rd}{2}\log(q^e).$$

Multiply by $2/\log(q^e)$. □

Lemma 7.3.51 (Finite upper-bound saturation). *Uses:*

Lean: *Weil.Weight.PUR.finite_upper_bound_saturation*

Let $r > 0$ and $a_1, \dots, a_r, d \in \mathbf{R}$. If $a_j \le d$ for every j and $\sum_j a_j = rd$, then $a_j = d$ for every j.

Proof. Uses:

Fix j. If $a_j < d$, then

$$\sum_m a_m = a_j + \sum_{m \ne j} a_m < d + (r-1)d = rd,$$

contradicting the sum identity. Thus $d \le a_j$; combine this with $a_j \le d$. □

Definition 7.3.52 (Exterior-power constituent-selection candidate). Uses: def:weil-pur-exterior-power-spectrum, def:weil-pur-algebraic-monodromy-group-candidate

Lean: Weil.Weight.PUR.ExteriorConstituentSelectionCandidate

For an irreducible rank-r constituent, an exterior-selection candidate specifies, for each $1 \le s \le r$, the irreducible monodromy constituent of $\bigwedge^s V$ containing a selected product of s eigenlines, along with its inclusion and projection maps.

Definition 7.3.53 (Exterior-power selection certificate). Uses: def:weil-pur-exterior-constituent-selection-candidate, def:weil-pur-central-weight-detection-certificate

Lean: Weil.Weight.PUR.ExteriorSelectionCertificate

An exterior-selection certificate supplies both inverse equations on the selected direct summand, monodromy equivariance, determinant-weight compatibility, and the characteristic-spectrum submultiset certificate. This isolates Deligne Proposition (1.3.13)(iii), printed p. 161 (PDF p. 26), and the exterior-power step in (1.5.3).

Open obligation: OBL-WGT-PUR-EXTERIOR

Definition 7.3.54 (Complete PUR antecedent certificate). Uses: def:weil-pur-rank-one-decomposition-certificate,def:weil-pur-central-weight-detection-certificate,def:weil-pur-low-degree-curve-certificate,def:weil-pur-trace-l-function-certificate,def:weil-pur-euler-convergence-certificate,def:weil-pur-pole-detection-certificate,def:weil-pur-tensor-power-limit-certificate,def:weil-pur-determinant-product-identity-certificate,def:weil-pur-exterior-selection-certificate,def:weil-lisse-heart-operations-typed-context

Lean: Weil.Weight.PUR.CompleteAntecedentCertificate

A complete PUR antecedent certificate consists of the listed class-field, monodromy-group, low-degree, trace, convergence, pole-detection, tensor-power, determinant-product, and exterior-selection certificates for every irreducible lisse constituent and every closed point. It contains no field asserting that the constituent is pure.

Proposition 7.3.55 (Conditional PUR endpoint). *Uses: def:weil-pur-complete-antecedent-certificate, def:weil-pur-determinantal-weight-data, def:weil-weight-pointwise-pure-predicate*

Lean: Weil.Weight.PUR.irreducibleConstituent_isPointwiseIotaPure

An irreducible lisse constituent equipped with a complete PUR antecedent certificate and determinantal weight d is pointwise ι-pure of weight d.

Proof. Uses: lemma:weil-pur-numerical-weight-realizes-modulus,lemma:weil-pur-limit-removes-reciprocal-error,lemma:weil-pur-sum-numerical-weights,lemma:weil-pur-finite-upper-bound-saturation

Fix a closed point and enumerate its rank-r spectrum. The tensor-power limit gives $\nu_\iota(e,\alpha_j) \le d$ for every j. The determinant product identity gives $\sum_j \nu_\iota(e,\alpha_j) = rd$. Finite upper-bound saturation gives $\nu_\iota(e,\alpha_j) = d$ for every j. The numerical-weight modulus lemma turns each equality into $|\iota(\alpha_j)| = (q^e)^{d/2}$. Since the point was arbitrary, this is pointwise purity. □

7.4 Nilpotent monodromy and the MON package

Definition 7.4.1 (Nilpotent operator datum). Uses: def:weil-coefficient-data

Lean: Weil.Weight.MON.NilpotentOperatorData

A nilpotent operator datum consists of a finite-dimensional $\overline{E}$-vector space V, an endomorphism $N : V \to V$, a natural number r, and the displayed equation $N^{r+1} = 0$.

Definition 7.4.2 (Monodromy-filtration candidate). Uses: def:weil-mon-nilpotent-operator-data

Lean: Weil.Weight.MON.FiltrationCandidate

A monodromy-filtration candidate is a family of subspaces $(M_jV)_{-r-1\le j\le r}$, together with inclusion maps $M_jV \to M_{j+1}V$, and endpoint data $M_{-r-1}V = 0$, $M_rV = V$.

Definition 7.4.3 (Monodromy-filtration increasing predicate). Uses: def:weil-mon-filtration-candidate

Lean: Weil.Weight.MON.IsIncreasing

Define the increasing predicate by $M_jV \subseteq M_{j+1}V$ for every allowed j, with the specified inclusion equal to the subtype inclusion.

Definition 7.4.4 (Monodromy operator shift predicate). Uses: def:weil-mon-nilpotent-operator-data,def:weil-mon-filtration-candidate

Lean: Weil.Weight.MON.IsOperatorShift

Define the shift predicate by

$$N(M_jV) \subseteq M_{j-2}V$$

for every j, using the zero endpoint outside the displayed interval.

Definition 7.4.5 (Monodromy graded-piece candidate). Uses: def:weil-mon-filtration-candidate

Lean: Weil.Weight.MON.gradedPieceCandidate

For every j, define the graded-piece candidate

$$\mathrm{Gr}^M_j V := M_jV/M_{j-1}V.$$

Definition 7.4.6 (Induced monodromy-power map candidate)**.** Uses: `def:weil-mon-nilpotent-operator-data`,`def:weil-mon-operator-shift-predicate`,`def:weil-mon-graded-piece-candidate`

Lean: `Weil.Weight.MON.inducedPowerMapCandidate`

For $j \geq 0$, define the proposed induced map

$$\overline{N}^{\,j} : \mathrm{Gr}^M_j V \longrightarrow \mathrm{Gr}^M_{-j} V$$

by applying N^j to a representative. The shift predicate places $N^j(M_j)$ in M_{-j}; representative independence is a separate predicate.

Definition 7.4.7 (Representative-independence predicate)**.** Uses: `def:weil-mon-induced-power-map-candidate`

Lean: `Weil.Weight.MON.IsInducedPowerWellDefined`

Define representative independence by requiring two representatives differing by M_{j-1} to have N^j-images differing by M_{-j-1}.

Definition 7.4.8 (Inverse candidate for an induced monodromy power)**.** Uses: `def:weil-mon-induced-power-map-candidate`

Lean: `Weil.Weight.MON.InducedPowerInverseCandidate`

For $j \geq 0$, an inverse candidate is a specified linear map

$$s_j : \mathrm{Gr}^M_{-j} V \longrightarrow \mathrm{Gr}^M_j V.$$

Definition 7.4.9 (Left-inverse predicate for monodromy powers)**.** Uses: `def:weil-mon-induced-power-map-candidate`,`def:weil-mon-induced-power-inverse-candidate`

Lean: `Weil.Weight.MON.IsInducedPowerLeftInverse`

Define the left-inverse predicate by $s_j \overline{N}^{\,j} = \mathrm{id}_{\mathrm{Gr}^M_j V}$.

Definition 7.4.10 (Right-inverse predicate for monodromy powers)**.** Uses: `def:weil-mon-induced-power-map-candidate`,`def:weil-mon-induced-power-inverse-candidate`

Lean: `Weil.Weight.MON.IsInducedPowerRightInverse`

Define the right-inverse predicate by $\overline{N}^{\,j} s_j = \mathrm{id}_{\mathrm{Gr}^M_{-j} V}$.

Definition 7.4.11 (Monodromy-filtration certificate)**.** Uses: `def:weil-mon-filtration-increasing-predicate`,`def:weil-mon-operator-shift-predicate`,`def:weil-mon-induced-power-well-defined-predicate`,`def:weil-mon-induced-power-inverse-candidate`,`def:weil-mon-induced-power-left-inverse-predicate`,`def:weil-mon-induced-power-right-inverse-predicate`

Lean: `Weil.Weight.MON.FiltrationCertificate`

A monodromy-filtration certificate consists of the increasing and shift predicates and, for every $j \geq 0$, representative independence, a specified inverse s_j, and both inverse equations. This is the full property list of Deligne Proposition (1.6.1), printed p. 165 (PDF p. 30).

Open obligation: `OBL-WGT-MON-FILTRATION-EXISTENCE`

Definition 7.4.12 (Comparison map between two monodromy filtrations)**.** Uses: `def:weil-mon-filtration-candidate`

Lean: `Weil.Weight.MON.FiltrationComparisonMapCandidate`

For two filtration candidates on the same (V, N), define a comparison candidate to be the identity map on V, equipped at each index j with a proposed restricted map $M_jV \to M'_jV$.

Definition 7.4.13 (Comparison inclusion predicate)**.** Uses: `def:weil-mon-filtration-comparison-map-candidate`

Lean: `Weil.Weight.MON.IsFiltrationComparisonInclusion`

Define comparison inclusion by requiring the restricted map at every j to land in M'_jV and to equal the identity after both subspace inclusions into V.

Definition 7.4.14 (Monodromy-filtration uniqueness certificate)**.** Uses: `def:weil-mon-filtration-certificate`,`def:weil-mon-filtration-comparison-map-candidate`,`def:weil-mon-filtration-comparison-inclusion-predicate`

Lean: `Weil.Weight.MON.FiltrationUniquenessCertificate`

For two certified filtrations, a uniqueness certificate supplies comparison candidates in both directions and their inclusion predicates.

Open obligation: `OBL-WGT-MON-FILTRATION-UNIQUENESS`

Lemma 7.4.15 (Equality from a monodromy-filtration uniqueness certificate). *Uses:* *def:weil-mon-filtration-uniqueness-certificate*

Lean: *Weil.Weight.MON.FiltrationUniquenessCertificate.eq*

Two filtrations equipped with a uniqueness certificate have $M_jV = M'_jV$ for every j.

Proof. Uses: def:weil-mon-filtration-comparison-inclusion-predicate,def:weil-mon-filtration-uniqueness-certificate

The forward comparison gives $M_jV \subseteq M'_jV$, and the reverse comparison gives $M'_jV \subseteq M_jV$. Subspace antisymmetry gives the equality for each j. □

Definition 7.4.16 (Primitive-piece candidate). Uses: def:weil-mon-induced-power-map-candidate,def:weil-mon-graded-piece-candidate

Lean: Weil.Weight.MON.primitivePieceCandidate

For $j \geq 0$, define

$$P_j(V,N) := \ker\left(\overline{N}^{\,j+1} : \mathrm{Gr}^M_j V \to \mathrm{Gr}^M_{-j-2} V\right).$$

Definition 7.4.17 (Primitive-decomposition map candidate). Uses: def:weil-mon-primitive-piece-candidate

Lean: Weil.Weight.MON.primitiveDecompositionMapCandidate

For each j, define the proposed map

$$\bigoplus_{a\geq 0} N^a P_{j+2a}(V,N) \longrightarrow \mathrm{Gr}^M_j V$$

by summing the displayed inclusions and powers of N.

Definition 7.4.18 (Primitive-decomposition certificate). Uses: def:weil-mon-primitive-decomposition-map-candidate

Lean: Weil.Weight.MON.PrimitiveDecompositionCertificate

A primitive-decomposition certificate supplies a proposed inverse, both inverse equations, equivariance for the one-dimensional Lie algebra generated by N, and independence from the auxiliary $\mathfrak{sl}_2$-splitting. It expands Deligne (1.6.14.2)–(1.6.14.5), printed pp. 169–170 (PDF pp. 34–35).

Open obligation: OBL-WGT-MON-PRIMITIVE

Definition 7.4.19 (Tensor monodromy operator). Uses: def:weil-mon-nilpotent-operator-data

Lean: Weil.Weight.MON.tensorOperator

For nilpotent data (V, N_V) and (W, N_W), define

$$N_{V\otimes W} := N_V \otimes 1 + 1 \otimes N_W.$$

Definition 7.4.20 (Tensor monodromy-filtration candidate). Uses: def:weil-mon-filtration-candidate,def:weil-mon-tensor-operator

Lean: Weil.Weight.MON.tensorFiltrationCandidate

Define

$$M_n(V\otimes W) := \sum_{a+b=n} M_aV \otimes M_bW.$$

The finite sum is taken over the displayed filtration ranges.

Definition 7.4.21 (Tensor-filtration comparison certificate). Uses: def:weil-mon-filtration-certificate,def:weil-mon-tensor-filtration-candidate

Lean: Weil.Weight.MON.TensorFiltrationCertificate

A tensor-filtration certificate supplies the induced graded-piece decomposition, the monodromy-power inverse maps, and their two inverse equations for the tensor candidate. This isolates the tensor part of Deligne Proposition (1.6.9), printed p. 167 (PDF p. 32).

Open obligation: OBL-WGT-MON-TENSOR

Definition 7.4.22 (Dual monodromy operator). Uses: def:weil-mon-nilpotent-operator-data

Lean: Weil.Weight.MON.dualOperator

Define the dual nilpotent operator by

$$(N^\vee\lambda)(v) := -\lambda(Nv).$$

Definition 7.4.23 (Dual monodromy-filtration candidate). Uses: `def:weil-mon-filtration-candidate,def:weil-mon-dual-operator`

Lean: `Weil.Weight.MON.dualFiltrationCandidate`

Define

$$M_j(V^\vee) := \{\lambda : \lambda(M_{-j-1}V) = 0\}.$$

Definition 7.4.24 (Dual-filtration comparison certificate). Uses: `def:weil-mon-filtration-certificate,def:weil-mon-dual-filtration-candidate`

Lean: `Weil.Weight.MON.DualFiltrationCertificate`

A dual-filtration certificate supplies the graded duality maps

$$\mathrm{Gr}^M_j(V^\vee) \longrightarrow (\mathrm{Gr}^M_{-j} V)^\vee,$$

their inverses, both inverse equations, and compatibility with N. This is the dual part of Deligne Proposition (1.6.9).

Open obligation: `OBL-WGT-MON-DUAL`

7.4.1 The intrinsic nilpotent weight filtration

Lemma 7.4.25 (Finite-dimensional ambient space of a nilpotent datum). Uses: `def:weil-mon-nilpotent-operator-data`

Lean: `Weil.Weight.MON.underlying_finiteDimensional`

For a nilpotent operator datum (V, N, r), the underlying $\overline{E}$-vector space V is finite-dimensional.

Proof. Uses: `def:weil-mon-nilpotent-operator-data`

This is the finite-dimensionality field of the datum, not a consequence of nilpotence. In particular it remains an explicit hypothesis in every foundation lemma below that uses a finite basis or algebraic duality. □

Lemma 7.4.26 (Finite-dimensionality of the algebraic dual). Uses: `def:weil-mon-nilpotent-operator-data`

Lean: `Weil.Weight.MON.dual_finiteDimensional`

For a nilpotent operator datum (V, N, r), the algebraic dual $V^\vee = \mathrm{Hom}_{\overline{E}}(V, \overline{E})$ is finite-dimensional.

Proof. Uses: `lemma:weil-mon-underlying-finite-dimensional`

Choose a finite basis $(e_i)_{i\in I}$ of V. Evaluation on this basis identifies $V^\vee$ with $\overline{E}^I$; the coordinate functionals $(e_i^*)_{i\in I}$ are a finite basis. Thus finite-dimensionality is an explicitly supplied field for the dual datum, rather than an implicit consequence of its nilpotence equation. □

Lemma 7.4.27 (Signed dual nilpotent datum with the same bound). Uses: `def:weil-mon-nilpotent-operator-data,def:weil-mon-dual-operator`

Lean: `Weil.Weight.MON.dualNilpotentOperatorData`

For a nilpotent datum (V, N, r), the finite dual $V^\vee$, the signed transpose $N^\vee = -N^*$, and the same natural number r form a nilpotent operator datum.

Proof. Uses: `def:weil-mon-nilpotent-operator-data,def:weil-mon-dual-operator,lemma:weil-mon-dual-finite-dimensional`

The preceding lemma supplies the finite-dimensionality field. Induction on the natural number m gives

$$((N^\vee)^m\lambda)(v) = (-1)^m\lambda(N^m v).$$

At the natural-number endpoint $m = r + 1$, the right side is zero by the datum equation $N^{r+1} = 0$. Hence $(N^\vee)^{r+1} = 0$, as required. □

Definition 7.4.28 (Polynomial-evaluation module of a nilpotent operator). Uses: `def:weil-mon-nilpotent-operator-data`

Lean: `Weil.Weight.MON.polynomialEvaluationModule`

Give V its $\overline{E}[T]$-module structure by

$$p(T)\cdot v := p(N)v = \sum_i p_i N^i v.$$

The module laws follow from the finite coefficient sums and $N^iN^j = N^{i+j}$. In particular, the action of T is exactly N.

Lemma 7.4.29 (Finite generation and the polynomial torsion bound). *Uses:* def:weil-mon-polynomial-evaluation-module

Lean: Weil.Weight.MON.polynomialEvaluationModule_finite_torsion

The polynomial-evaluation module is finitely generated over $\overline{E}[T]$, and T^{r+1} annihilates it.

Proof. Uses: def:weil-mon-nilpotent-operator-data, def:weil-mon-polynomial-evaluation-module, lemma:weil-mon-underlying-finite-dimensional

Choose a finite $\overline{E}$-basis B of V. Constant polynomials already produce every $\overline{E}$-linear combination of B, so the same finite set generates V over $\overline{E}[T]$. Since T acts as N, the datum equation $N^{r+1} = 0$ is precisely $T^{r+1}V = 0$. □

Theorem 7.4.30 (Atomic elementary-divisor input for a polynomial PID). *Uses:*

Lean: Weil.Weight.MON.polynomialPID_elementaryDivisors

Open obligation: OBL-WGT-MON-PID-ELEMENTARY-DIVISORS

Let K be a field, let M be a finitely generated $K[T]$-module, and suppose $T^{r+1}M = 0$. Then there are a finite set A, integers $1 \le d_a \le r+1$, and a $K[T]$-linear equivalence

$$\Phi : \bigoplus_{a \in A} K[T]/(T^{d_a}) \xrightarrow{\sim} M.$$

This is the sole unverified PID structure-theorem input in this v3 fragment.

Proof. Uses:

No proof is asserted in this candidate: the displayed theorem is exactly the atomic open obligation recorded in the ledger. Downstream proofs are marked unresolved whenever they depend on this input. □

Definition 7.4.31 (Jordan-chain decomposition datum). Uses: def:weil-mon-nilpotent-operator-data

Lean: Weil.Weight.MON.JordanChainDecomposition

For nilpotent data (V, N, r), a Jordan-chain decomposition consists of a finite index set A, positive integers $d_a \le r+1$, and a basis

$$\{e_{a,t} : a \in A,\ 0 \le t < d_a\}$$

such that

$$Ne_{a,0} = 0, \qquad Ne_{a,t} = e_{a,t-1} \quad (1 \le t < d_a).$$

Attach to a basis vector the integer

$$\mathrm{wt}(a,t) := 2t - (d_a - 1).$$

Lemma 7.4.32 (Transport of cyclic polynomial blocks to Jordan chains). *Uses:* def:weil-mon-jordan-chain-decomposition, def:weil-mon-polynomial-evaluation-module

Lean: Weil.Weight.MON.cyclicBlocks_to_jordanChains

Suppose a finite family $(d_a)_{a \in A}$, with $1 \le d_a \le r+1$, and a polynomial-module equivalence

$$\Phi : \bigoplus_{a \in A} \overline{E}[T]/(T^{d_a}) \xrightarrow{\sim} V$$

are given. Transporting reversed monomial classes through Φ produces a Jordan-chain decomposition of (V, N, r).

Proof. Uses: def:weil-mon-jordan-chain-decomposition, def:weil-mon-polynomial-evaluation-module

For a fixed a, division by the monic polynomial T^{d_a} gives a unique representative of degree less than d_a. Consequently the classes of $1, T, \dots, T^{d_a-1}$ form a basis of the cyclic quotient. Put

$$e_{a,t} := \Phi([T^{d_a-1-t}]) \qquad (0 \le t < d_a).$$

The direct-sum union of these vectors is a basis because Φ is an equivalence. Polynomial-linearity and the definition of the evaluation action give

$$Ne_{a,t} = \Phi([T^{d_a-t}]) = e_{a,t-1} \quad (t > 0), \qquad Ne_{a,0} = \Phi([T^{d_a}]) = 0.$$

These are exactly the fields of the Jordan-chain datum. □

Lemma 7.4.33 (Existence of a bounded Jordan-chain decomposition). *Uses:* def:weil-mon-jordan-chain-decomposition

Lean: Weil.Weight.MON.jordanChainDecomposition_exists

Every nilpotent datum (V, N, r) has a Jordan-chain decomposition, conditional only on the atomic elementary-divisor input above.

Proof. Uses: lemma:weil-mon-polynomial-module-finite-torsion,thm:weil-mon-polynomial-pid-elementary-divisors,lemma:weil-mon-cyclic-block-transport

Apply the elementary-divisor theorem to the finite polynomial-evaluation module. It supplies the finite family (d_a), the bounds $1 \le d_a \le r+1$, and the module equivalence required by the block transport lemma. That lemma supplies the claimed Jordan-chain datum. □

Definition 7.4.34 (Endpoint extension of a filtration candidate)**.** Uses: def:weil-mon-filtration-candidate

Lean: Weil.Weight.MON.FiltrationCandidate.endpointExtension

For a filtration candidate M, define its integer-indexed endpoint extension by

$$M_k^{\mathrm{ext}} = \begin{cases} 0, & k \le -r-1, \\ M_k, & -r \le k \le r-1, \\ V, & k \ge r. \end{cases}$$

The two omitted supplied endpoint steps agree with this definition. Below, an integer-indexed expression M_k always means this endpoint extension.

Definition 7.4.35 (Jordan weight filtration)**.** Uses: def:weil-mon-filtration-candidate,def:weil-mon-jordan-chain-decomposition

Lean: Weil.Weight.MON.jordanWeightFiltration

For a selected Jordan-chain decomposition define, for every integer k,

$$W_k^{\mathrm{J}} V := \mathrm{span}\{e_{a,t} : \mathrm{wt}(a,t) \le k\}.$$

Restricting to $-r-1 \le k \le r$ defines a filtration candidate. Its endpoint extension is 0 for $k \le -r-1$ and V for $k \ge r$, because every chain weight lies in $[-r, r]$.

Lemma 7.4.36 (Endpoint, monotonicity, and shift for the Jordan filtration)**.** *Uses:* def:weil-mon-jordan-weight-filtration

Lean: Weil.Weight.MON.jordanWeightFiltration_shift

The Jordan filtration has endpoints $W^{\mathrm{J}}_{-r-1} = 0$ and $W^{\mathrm{J}}_r = V$, is increasing, and satisfies

$$N(W_k^{\mathrm{J}}) \subseteq W_{k-2}^{\mathrm{J}} \qquad (k \in \mathbf{Z}).$$

Proof. Uses: def:weil-mon-jordan-chain-decomposition,def:weil-mon-jordan-weight-filtration

Since $d_a \le r+1$, each chain weight lies between $-r$ and r, which gives the endpoints. Inclusion of indexing subsets gives monotonicity. For $t > 0$,

$$\mathrm{wt}(a, t-1) = \mathrm{wt}(a,t) - 2,$$

whereas $e_{a,0}$ is killed by N. This calculation also covers every outer integer index through the endpoint extension. □

Lemma 7.4.37 (Coordinate quotient-basis lemma)**.** *Uses:*

Lean: Weil.Weight.MON.coordinateSpan_quotientBasis

Let $(v_i)_{i \in I}$ be a finite basis and let $I_0 \subseteq I_1 \subseteq I$. The residue classes of the vectors indexed by $I_1 \setminus I_0$ form a basis of

$$\mathrm{span}\{v_i : i \in I_1\} / \mathrm{span}\{v_i : i \in I_0\}.$$

Proof. Uses:

Every vector in the numerator has a unique coordinate expansion on I_1. After subtracting its coordinates on I_0, its quotient class is a linear combination of the displayed residue classes, proving spanning. If such a combination maps to zero, it is also a combination of the I_0-vectors. Uniqueness of coordinates in the full basis forces every coefficient on the disjoint set $I_1 \setminus I_0$ to vanish. Thus the displayed family is linearly independent. □

Lemma 7.4.38 (Basis of each Jordan graded piece)**.** *Uses:* def:weil-mon-graded-piece-candidate,def:weil-mon-jordan-weight-filtration

Lean: Weil.Weight.MON.jordanGradedBasis

For every integer k, the residue classes of

$$\{e_{a,t} : \mathrm{wt}(a,t) = k\}$$

form a basis of $\mathrm{Gr}_k^{W^{\mathrm{J}}} V$.

Proof. Uses: def:weil-mon-jordan-chain-decomposition,def:weil-mon-jordan-weight-filtration,lemma:weil-mon-coordinate-quotient-basis

Apply the coordinate quotient-basis lemma to the nested subsets of Jordan vectors having weight at most $k-1$ and at most k. Their difference is the set of exact-weight k vectors. For outer indices both subsets are simultaneously empty or simultaneously the full basis, so the same argument also gives the unique basis of the zero graded piece. □

Lemma 7.4.39 (Explicit inverse to every graded monodromy power). *Uses:* def:weil-mon-induced-power-inverse-candidate, def:weil-mon-jordan-weight-filtration, def:weil-mon-graded-piece-candidate

Lean: Weil.Weight.MON.jordanGradedPowerEquiv

For every $j \geq 0$, the induced map

$$\overline{N}^{\,j} : \mathrm{Gr}_j^{W^{\mathrm{J}}} V \longrightarrow \mathrm{Gr}_{-j}^{W^{\mathrm{J}}} V$$

is well-defined and has a specified linear inverse satisfying both inverse equations.

Proof. Uses: lemma:weil-mon-jordan-weight-filtration-shift,lemma:weil-mon-jordan-graded-basis

Repeated use of the shift sends W^{J}_{j-1} into W^{J}_{-j-1}, so the map is representative-independent. If a Jordan vector has exact weight j, then

$$t - j = d_a - 1 - t \geq 0, \qquad N^j e_{a,t} = e_{a,t-j}, \qquad \mathrm{wt}(a, t-j) = -j.$$

Thus $(a,t) \mapsto (a,t-j)$ bijects the exact-weight j basis with the exact-weight $-j$ basis, with inverse $(a,u) \mapsto (a,u+j)$. Linear extension gives both inverse equations. When $j > r$, both graded pieces are zero and the unique zero-space map is the required equivalence. □

Proposition 7.4.40 (Existence of the monodromy-filtration certificate). *Uses:* def:weil-mon-filtration-certificate, def:weil-mon-jordan-weight-filtration

Lean: Weil.Weight.MON.filtrationCertificate_exists

Conditional on the elementary-divisor leaf, every nilpotent operator datum has a monodromy-filtration candidate with a complete certificate.

Proof. Uses: lemma:weil-mon-jordan-chain-decomposition-exists,lemma:weil-mon-jordan-weight-filtration-shift,lemma:weil-mon-jordan-graded-power-isomorphism

Choose the Jordan decomposition supplied by the existence lemma and its weight filtration. The endpoint, increasing, and shift fields come from the shift lemma. At every nonnegative index install the induced map, its representative-independence proof, the inverse, and the two inverse equations from the graded-power lemma. □

7.4.2 Intrinsic characterization and uniqueness

Lemma 7.4.41 (Blockwise kernel-image coordinate formula). *Uses:* def:weil-mon-jordan-chain-decomposition

Lean: Weil.Weight.MON.jordanBlock_kernel_inf_range

For a selected Jordan decomposition and nonnegative a, b,

$$\ker N^{a+1} \cap \mathrm{im}\, N^b = \mathrm{span}\{e_{c,t} : t \leq a \text{ and } t + b \leq d_c - 1\}.$$

In particular kernels, images, their intersections, and finite sums of these intersections decompose coordinatewise over the Jordan blocks.

Proof. Uses: def:weil-mon-jordan-chain-decomposition

Induction on m gives

$$N^m e_{c,t} = e_{c,t-m} \quad (m \leq t), \qquad N^m e_{c,t} = 0 \quad (m > t).$$

Hence the kernel is the coordinate span of the vectors with $t \leq a$, and the image is the coordinate span of those with $t + b \leq d_c - 1$. The intersection of two coordinate spans in a fixed basis is the span of the intersection of their indexing subsets: coefficient uniqueness proves both inclusions. The displayed formula and its blockwise consequences follow. □

Definition 7.4.42 (Intrinsic kernel-image filtration)**.** Uses: `def:weil-mon-nilpotent-operator-data`

Lean: `Weil.Weight.MON.intrinsicKernelImageFiltration`

For $-r-1 \le k \le r$, define

$$W_k^{\mathrm{int}}(N) := \sum_{\substack{0\le a,b\le r\\ a-b=k}} \bigl(\ker N^{a+1} \cap \operatorname{im} N^b\bigr).$$

Extend this family by 0 for $k < -r-1$ and by V for $k > r$. At the endpoints the formula already gives 0 and V, respectively.

Lemma 7.4.43 (The intrinsic and Jordan filtrations agree)**.** Uses: `def:weil-mon-intrinsic-kernel-image-filtration, def:weil-mon-jordan-weight-filtration`

Lean: `Weil.Weight.MON.intrinsicKernelImageFiltration_eq_jordan`

For every selected Jordan decomposition and every integer k,

$$W_k^{\mathrm{int}}(N) = W_k^{\mathrm{J}} V.$$

Thus the Jordan filtration is independent of the selected decomposition whenever a decomposition exists.

Proof. Uses: `lemma:weil-mon-jordan-blockwise-kernel-image, def:weil-mon-intrinsic-kernel-image-filtration, def:weil-mon-jordan-weight-filtration`

First dispose of the boundary ranges. If $k \le -r-1$, then no pair $0 \le a, b \le r$ has $a-b=k$ at the endpoint, and below it the intrinsic extension is zero. Every Jordan weight is at least $-r$, so the Jordan step is also zero. If $k \ge r$, the endpoint summand $(a,b) = (r,0)$ is $\ker N^{r+1} = V$, and above the endpoint the intrinsic extension remains V. Every Jordan weight is at most r, so the Jordan step is also V.

It remains to assume $-r \le k < r$. By the blockwise coordinate lemma, a Jordan vector $e_{c,t}$ belongs to the (a,b)-summand exactly when

$$t \le a, \qquad t + b \le d_c - 1.$$

Together with $a-b=k$, these inequalities imply $2t - (d_c-1) \le k$. Conversely, suppose this weight inequality holds. If $k \ge t$, set $(a,b) = (k,0)$. Here $0 \le a = k < r$, so both indices are in range. If $k < t$, set $(a,b) = (t, t-k)$. Then

$$0 \le a = t \le d_c - 1 \le r, \qquad 0 \le b = t-k \le d_c - 1 - t \le r.$$

In either case the vector occurs in a summand if and only if its weight is at most k. The two sides are therefore the same coordinate span. □

Lemma 7.4.44 (Every certified filtration contains the intrinsic filtration)**.** Uses: `def:weil-mon-filtration-certificate, def:weil-mon-intrinsic-kernel-image-filtration`

Lean: `Weil.Weight.MON.intrinsicKernelImageFiltration_le`

If M is any certified monodromy filtration, then, using its endpoint extension,

$$W_k^{\mathrm{int}}(N) \subseteq M_k \qquad (k \in \mathbf{Z}).$$

Proof. Uses: `def:weil-mon-filtration-endpoint-extension, def:weil-mon-filtration-increasing-predicate, def:weil-mon-operator-shift-predicate, def:weil-mon-filtration-certificate`

If $k \le -r-1$, the source is zero. If $k \ge r$, the target endpoint extension is V. Thus assume $-r \le k < r$, where the finite kernel-image sum is the defining formula. It is enough by linearity to take

$$x \in \ker N^{a+1} \cap \operatorname{im} N^b, \qquad 0 \le a, b \le r, \qquad a - b = k.$$

The supplied increasing predicate and the endpoint equations make the endpoint extension increasing on all of $\mathbf{Z}$: the only new boundary inclusions are from 0 and into V. If $x = 0$, the conclusion is immediate. Otherwise choose y with $x = N^b y$; then $y \neq 0$. The set

$$S = \{q \in \mathbf{Z} : y \in M_q\}$$

contains r, and it contains no integer at most $-r-1$. Its least element is therefore the least element of the nonempty finite set $S \cap [-r, r]$; call it q.

If $q > a + b$, then $q \ge 1$ and

$$N^q y = N^{q-a-b-1} N^{a+b+1} y = N^{q-a-b-1} N^{a+1} x = 0.$$

The certified isomorphism $N^q : \mathrm{Gr}_q^M \to \mathrm{Gr}_{-q}^M$ sends the class of y to zero. Injectivity puts y in M_{q-1}, contrary to the minimality of q. Hence $q \le a + b$. Applying the shift b times and then monotonicity gives

$$x = N^b y \in M_{q-2b} \subseteq M_{a-b} = M_k.$$

Taking the finite sum proves the inner-range assertion, and the two boundary branches prove the assertion for every integer. □

Lemma 7.4.45 (Dual annihilator formula for the intrinsic filtration)**.** *Uses:* `def:weil-mon-dual-operator`, `def:weil-mon-intrinsic-kernel-image-filtration`

Lean: `Weil.Weight.MON.intrinsicDual_annihilator`

For every integer k and $N^\vee = -N^*$,

$$W_k^{\mathrm{int}}(N^\vee) = \mathrm{Ann}\big(W_{-k-1}^{\mathrm{int}}(N)\big).$$

Proof. Uses: `lemma:weil-mon-signed-dual-nilpotent-datum`, `lemma:weil-mon-jordan-chain-decomposition-exists`, `lemma:weil-mon-intrinsic-eq-jordan-filtration`

Choose a Jordan basis $e_{a,t}$ and its dual basis. For a block of length d_a, put

$$g_{a,u} = (-1)^u e^*_{a,d_a-1-u}.$$

Direct evaluation gives $N^\vee g_{a,0} = 0$ and $N^\vee g_{a,u} = g_{a,u-1}$. The dual-chain weight of $g_{a,u}$ is the negative of the weight of its paired primal vector. A coordinate functional annihilates the span of primal vectors of weight at most $-k-1$ exactly when its paired primal weight is at least $-k$, equivalently when its dual weight is at most k. This coordinate statement remains valid in both constant outer ranges. Apply intrinsic–Jordan equality to the primal and dual Jordan bases. □

Definition 7.4.46 (Basis datum adapted to two nested subspaces)**.** Uses: `def:weil-mon-nilpotent-operator-data`

Lean: `Weil.Weight.MON.AnnihilatorAdaptedBasisDatum`

For subspaces $A \subseteq B \subseteq V$, an adapted-basis datum consists of finite families

$$(a_i)_{i\in I_A} \subseteq A, \quad (b_u)_{u\in I_{B/A}} \subseteq B, \quad (c_w)_{w\in I_{V/B}} \subseteq V$$

such that (a_i) is a basis of A, the classes $([b_u])$ are a basis of B/A, the classes $([c_w])$ are a basis of V/B, and the concatenated family (a_i, b_u, c_w) is a basis of V.

Lemma 7.4.47 (Existence of an adapted basis)**.** *Uses:* `def:weil-mon-nilpotent-operator-data`, `def:weil-mon-annihilator-adapted-basis-datum`

Lean: `Weil.Weight.MON.annihilatorAdaptedBasisDatum_exists`

For a nilpotent datum on V and subspaces $A \subseteq B \subseteq V$, an adapted-basis datum exists.

Proof. Uses: `lemma:weil-mon-underlying-finite-dimensional`

Choose a basis of A, a basis of B/A, and a basis of V/B, and choose representatives $b_u \in B$ and $c_w \in V$ of the latter two bases. The family (a_i, b_u) spans B: subtract from any $b \in B$ the combination of the b_u's representing its quotient class, leaving an element of A. It is independent: a relation first descends to a relation among the basis classes in B/A, and then to one among the a_i's. The same quotient argument, now for V/B, proves that (a_i, b_u, c_w) spans V and is independent. This constructs every field of the adapted-basis datum without applying a coordinate-span lemma before a common ambient basis has been built. □

Lemma 7.4.48 (Annihilators reverse inclusions)**.** *Uses:*

Lean: `Weil.Weight.MON.annihilator_mono`

For subspaces $A \subseteq B \subseteq V$, one has

$$\mathrm{Ann}(B) \subseteq \mathrm{Ann}(A).$$

Proof. Uses:

If λ vanishes on B, then it vanishes on the subspace $A \subseteq B$. This inclusion is the denominator typing required to form $\mathrm{Ann}(A)/\mathrm{Ann}(B)$. □

Definition 7.4.49 (Typed inclusion of nested annihilators)**.** Uses:

Constructed by: `lemma:weil-mon-annihilator-reverses-inclusion`

Lean: `Weil.Weight.MON.annihilatorInclusionMap`

For $A \subseteq B \subseteq V$, let

$$\iota_{B,A} : \mathrm{Ann}(B) \longrightarrow \mathrm{Ann}(A)$$

be the subtype inclusion supplied by $\mathrm{Ann}(B) \subseteq \mathrm{Ann}(A)$.

Lemma 7.4.50 (Representative independence for restriction to B/A)**.** *Uses:*

Lean: `Weil.Weight.MON.annihilatorRestriction_repIndependent`

For $A \subseteq B$, $\lambda \in \mathrm{Ann}(A)$, and $b, b' \in B$ with $b - b' \in A$, one has

$$\lambda(b) = \lambda(b').$$

Proof. Uses:
Since λ annihilates A, the hypothesis gives $\lambda(b - b') = 0$, hence the displayed equality. □

Definition 7.4.51 (Restriction to an annihilator)**.** Uses:

Constructed by: `lemma:weil-mon-annihilator-restriction-representatives`

Lean: `Weil.Weight.MON.annihilatorRestrictionMap`
For $A \subseteq B \subseteq V$, define

$$R_{A,B} : \mathrm{Ann}(A) \longrightarrow (B/A)^\vee, \qquad R_{A,B}(\lambda)([b]) = \lambda(b).$$

The preceding representative lemma supplies the quotient lift in the B/A variable.

Lemma 7.4.52 (Restriction vanishes on the larger annihilator)**.** Uses: *def:weil-mon-annihilator-restriction-map, def:weil-mon-annihilator-inclusion-map*

Lean: *Weil.Weight.MON.annihilatorRestrictionMap_vanishes*
The typed restriction of $R_{A,B}$ to $\mathrm{Ann}(B)$ is zero:

$$R_{A,B} \circ \iota_{B,A} = 0 : \mathrm{Ann}(B) \longrightarrow (B/A)^\vee.$$

Proof. Uses: `def:weil-mon-annihilator-restriction-map,def:weil-mon-annihilator-inclusion-map`
If λ annihilates B, then $R_{A,B}(\iota_{B,A}\lambda)([b]) = \lambda(b) = 0$ for every $b \in B$. Extensionality on B/A proves the zero-map equation. □

Definition 7.4.53 (Forward annihilator-quotient map)**.** Uses: `def:weil-mon-annihilator-restriction-map,def:weil-mon-annihilator-inclusion-map`

Constructed by: `lemma:weil-mon-annihilator-restriction-vanishes`

Lean: `Weil.Weight.MON.annihilatorQuotientForward`
Use $\mathrm{Ann}(B) \subseteq \mathrm{Ann}(A)$ to form the quotient and lift $R_{A,B}$ through its zero restriction. This defines

$$\Theta_{A,B} : \mathrm{Ann}(A)/\mathrm{Ann}(B) \longrightarrow (B/A)^\vee, \qquad \Theta_{A,B}([\lambda])([b]) = \lambda(b).$$

Definition 7.4.54 (Functional extension along an adapted basis)**.** Uses: `def:weil-mon-annihilator-adapted-basis-datum`

Lean: `Weil.Weight.MON.annihilatorFunctionalExtension`
Fix an adapted-basis datum $\mathcal{B} = (a_i, b_u, c_w)$. For $\psi \in (B/A)^\vee$, define $E_{\mathcal{B}}(\psi) \in V^\vee$ on the adapted basis by

$$E_{\mathcal{B}}(\psi)(a_i) = 0, \qquad E_{\mathcal{B}}(\psi)(b_u) = \psi([b_u]), \qquad E_{\mathcal{B}}(\psi)(c_w) = 0,$$

and extend linearly. The first equations put the result in $\mathrm{Ann}(A)$, so $E_{\mathcal{B}}$ is a linear map into that annihilator.

Lemma 7.4.55 (The adapted functional extends the quotient functional)**.** Uses: *def:weil-mon-annihilator-functional-extension, def:weil-mon-annihilator-restriction-map*

Lean: *Weil.Weight.MON.annihilatorFunctionalExtension_restricts*
For every $\psi \in (B/A)^\vee$,

$$R_{A,B}(E_{\mathcal{B}}(\psi)) = \psi.$$

Proof. Uses: `def:weil-mon-annihilator-adapted-basis-datum,def:weil-mon-annihilator-functional-extension,def:weil-mon-annihilator-restriction-map`
The classes $([b_u])$ are a basis of B/A. Both sides have value $\psi([b_u])$ on each basis vector, by the defining equation for $E_{\mathcal{B}}$. Linear-map extensionality on that quotient basis gives the displayed equality. □

Definition 7.4.56 (Adapted-basis inverse to annihilator restriction)**.** Uses: `def:weil-mon-annihilator-functional-extension,def:weil-mon-annihilator-inclusion-map`

Lean: `Weil.Weight.MON.annihilatorRestrictionInverse`
For an adapted-basis datum $\mathcal{B}$, define

$$\Xi_{\mathcal{B}} : (B/A)^\vee \longrightarrow \mathrm{Ann}(A)/\mathrm{Ann}(B), \qquad \Xi_{\mathcal{B}}(\psi) := [E_{\mathcal{B}}(\psi)].$$

Lemma 7.4.57 (Right inverse for annihilator restriction)**.** Uses: *def:weil-mon-annihilator-quotient-forward, def:weil-mon-annihilator-restriction-inverse*

Lean: *Weil.Weight.MON.annihilatorRestrictionInverse_right*
For every adapted-basis datum,

$$\Theta_{A,B} \circ \Xi_{\mathcal{B}} = 1_{(B/A)^\vee}.$$

Proof. Uses: `lemma:weil-mon-annihilator-functional-extension-restricts`
This is the functional-extension restriction equation, pointwise in ψ. □

Lemma 7.4.58 (Left inverse for annihilator restriction)**.** *Uses:* `def:weil-mon-annihilator-quotient-forward`, `def:weil-mon-annihilator-restriction-inverse`

Lean: `Weil.Weight.MON.annihilatorRestrictionInverse_left`
For every adapted-basis datum,

$$\Xi_{\mathcal{B}} \circ \Theta_{A,B} = 1_{\mathrm{Ann}(A)/\mathrm{Ann}(B)}.$$

Proof. Uses: `def:weil-mon-annihilator-functional-extension`, `def:weil-mon-annihilator-quotient-forward`, `def:weil-mon-annihilator-restriction-inverse`
Represent a source class by $\lambda \in \mathrm{Ann}(A)$. On every a_i both $E_{\mathcal{B}}(\Theta[\lambda])$ and λ vanish; on every b_u they agree by definition of Θ and $E_{\mathcal{B}}$. Since (a_i, b_u) is a basis of B, their difference vanishes on B, hence belongs to $\mathrm{Ann}(B)$. The two representatives therefore define the same quotient class. □

Lemma 7.4.59 (Adapted-basis independence of the quotient inverse)**.** *Uses:* `def:weil-mon-annihilator-restriction-inverse`

Lean: `Weil.Weight.MON.annihilatorRestrictionInverse_independent`
If $\mathcal{B}$ and $\mathcal{B}'$ are two adapted-basis data, then $\Xi_{\mathcal{B}} = \Xi_{\mathcal{B}'}$.

Proof. Uses: `lemma:weil-mon-annihilator-restriction-inverse-left`, `lemma:weil-mon-annihilator-restriction-inverse-right`
Both maps are two-sided inverses of the same canonical map $\Theta_{A,B}$. Thus $\Xi_{\mathcal{B}} = \Xi_{\mathcal{B}}\Theta\Xi_{\mathcal{B}'} = \Xi_{\mathcal{B}'}$. □

Lemma 7.4.60 (Annihilator-quotient duality)**.** *Uses:* `def:weil-mon-annihilator-quotient-forward`

Lean: `Weil.Weight.MON.annihilatorQuotientEquiv`
For subspaces $A \subseteq B \subseteq V$, the canonical restriction map $\Theta_{A,B}$ is a linear equivalence

$$\mathrm{Ann}(A)/\mathrm{Ann}(B) \xrightarrow{\sim} (B/A)^{\vee}.$$

Its inverse is the basis-independent quotient class represented by any adapted functional extension.

Proof. Uses: `lemma:weil-mon-annihilator-adapted-basis-exists`, `def:weil-mon-annihilator-restriction-inverse`, `lemma:weil-mon-annihilator-restriction-inverse-left`, `lemma:weil-mon-annihilator-restriction-inverse-right`, `lemma:weil-mon-annihilator-restriction-inverse-independent`
Choose an adapted basis, install $\Xi_{\mathcal{B}}$ as the inverse, and use the two inverse equations. Independence shows that the resulting inverse, and hence the equivalence, is canonical. □

Lemma 7.4.61 (Signed transpose on annihilator graded quotients)**.** *Uses:* `def:weil-mon-dual-operator`, `def:weil-mon-filtration-certificate`

Lean: `Weil.Weight.MON.annihilatorGraded_signedTranspose`
For a certified filtration M and $j \in \mathbf{N}$, identify

$$\mathrm{Gr}^{M^{\vee}}_{(j:\mathbf{Z})}(V^{\vee}) \simeq (\mathrm{Gr}^{M}_{-(j:\mathbf{Z})} V)^{\vee}, \qquad \mathrm{Gr}^{M^{\vee}}_{-(j:\mathbf{Z})}(V^{\vee}) \simeq (\mathrm{Gr}^{M}_{(j:\mathbf{Z})} V)^{\vee}$$

by annihilator-quotient duality. Under these equivalences the natural power induced by $(N^{\vee})^j$ is $(-1)^j$ times the transpose of the natural power induced by N^j.

Proof. Uses: `def:weil-mon-dual-operator`, `def:weil-mon-operator-shift-predicate`, `lemma:weil-mon-annihilator-quotient-duality`
Induction on j gives, for representatives,

$$((N^{\vee})^j\lambda)(v) = (-1)^j\lambda(N^j v).$$

The shift sends the two denominator filtration steps into the appropriate denominators, so both sides descend to the displayed quotients. Evaluating the descended equality on every quotient class is precisely the signed transpose identity. This same calculation supplies representative independence before any inverse is introduced. □

Lemma 7.4.62 (Finite-dimensional double-annihilator separation)**.** *Uses:*

Lean: `Weil.Weight.MON.doubleAnnihilator_eq`
For every subspace $S \subseteq V$,

$$\{v \in V : \lambda(v) = 0 \text{ for every } \lambda \in \mathrm{Ann}(S)\} = S.$$

Equivalently, evaluation identifies the double annihilator of S with S itself.

Proof. Uses: lemma:weil-mon-underlying-finite-dimensional

Every vector of S is killed by every functional in $\mathrm{Ann}(S)$, giving one inclusion. For the reverse, choose a basis of S and extend it to a basis of V. If $v \notin S$, some coordinate of v on the complementary basis vectors is nonzero. The corresponding coordinate functional vanishes on S but not on v. Thus v is not in the displayed left side. □

7.4.3 Typed primitive strings from a selected Jordan decomposition

Lemma 7.4.63 (One-step shift for the endpoint extension). Uses: def:weil-mon-filtration-endpoint-extension, def:weil-mon-filtration-certificate

Lean: Weil.Weight.MON.endpointExtension_oneStepShift

For every integer m, a certified filtration satisfies

$$N\left(M_m^{\mathrm{ext}}\right) \subseteq M_{m-2}^{\mathrm{ext}}.$$

Proof. Uses: def:weil-mon-filtration-endpoint-extension,def:weil-mon-filtration-increasing-predicate,def:weil-mon-operator-shift-predicate,def:weil-mon-filtration-certificate

If $m \le -r-1$, the source is zero. If $-r \le m \le r$, this is the stored shift certificate. If $m \ge r+2$, the target is the full space. The remaining upper-boundary case is $m = r+1$: here $M_{r+1}^{\mathrm{ext}} = V = M_r$, so the stored shift at r gives

$$N(V) \subseteq M_{r-2} \subseteq M_{r-1} = M_{(r+1)-2}^{\mathrm{ext}}.$$

If $r \ge 1$, the middle inclusion is the stored increasingness certificate. If $r = 0$, both middle terms are zero by the lower endpoint extension, so the same inclusion is the unique zero-space inclusion. This is the step that the constant upper-extension argument alone does not supply. □

Lemma 7.4.64 (Iterated shift on the endpoint extension). Uses: def:weil-mon-filtration-endpoint-extension, def:weil-mon-filtration-certificate

Lean: Weil.Weight.MON.endpointExtension_iterateShift

For every integer m and natural number a, the endpoint-extended filtration of a certified candidate satisfies

$$N^a\left(M_m^{\mathrm{ext}}\right) \subseteq M_{m-2a}^{\mathrm{ext}}.$$

The same statement with $m-1$ in place of m also holds.

Proof. Uses: lemma:weil-mon-endpoint-extension-one-step-shift

Induct on a. The zero case is the identity inclusion. For the successor step, apply the induction hypothesis and then the one-step lemma at the integer index $m-2a$. The endpoint cases, including the exceptional index $r+1$, were discharged in that lemma, so the induction is valid for every integer index. □

Lemma 7.4.65 (Representative difference for a general graded power). Uses: def:weil-mon-filtration-endpoint-extension, def:weil-mon-filtration-certificate

Lean: Weil.Weight.MON.generalGradedPower_repDifference

For a certified filtration, if $x, x' \in M_m^{\mathrm{ext}}$ have $x - x' \in M_{m-1}^{\mathrm{ext}}$, then

$$N^a x - N^a x' \in M_{m-2a-1}^{\mathrm{ext}}.$$

Proof. Uses: lemma:weil-mon-endpoint-extension-iterate-shift

Linearity gives $N^a x - N^a x' = N^a(x - x')$. Apply the iterated-shift lemma to the step indexed by $m-1$. □

Definition 7.4.66 (General induced graded power). Uses: def:weil-mon-graded-piece-candidate,def:weil-mon-filtration-endpoint-extension,def:weil-mon-filtration-certificate

Constructed by: lemma:weil-mon-general-graded-power-representative-difference

Lean: Weil.Weight.MON.generalGradedPower

For a certified filtration candidate, $m \in \mathbf{Z}$, and $a \in \mathbf{N}$, define

$$\overline{N}_m^a : \mathrm{Gr}_m^M V \longrightarrow \mathrm{Gr}_{m-2a}^M V, \qquad [x] \longmapsto [N^a x].$$

The construction uses the endpoint extension for indices outside the stored range. Its quotient lift is the representative-difference certificate.

Lemma 7.4.67 (Zero and successor laws for general graded powers). *Uses:* def:weil-mon-general-graded-power

Lean: Weil.Weight.MON.generalGradedPower_zero_succ

For every integer m,

$$\overline{N}_m^{0} = 1, \qquad \overline{N}_m^{a+1} = \overline{N}_{m-2a}^{1} \circ \overline{N}_m^{a}.$$

Proof. Uses: def:weil-mon-general-graded-power

Both equations hold on representatives by $N^0 = 1$ and $N^{a+1} = N \circ N^a$. Quotient extensionality gives the asserted linear map equalities. □

Lemma 7.4.68 (Additivity law for general graded powers). *Uses:* def:weil-mon-general-graded-power

Lean: Weil.Weight.MON.generalGradedPower_add

For $a, b \in \mathbf{N}$,

$$\overline{N}_m^{a+b} = \overline{N}_{m-2a}^{b} \circ \overline{N}_m^{a}.$$

Proof. Uses: def:weil-mon-general-graded-power

On representatives this is $N^{a+b} = N^b N^a$. The two sides have the same source and target because $(m-2a)-2b = m-2(a+b)$. □

Definition 7.4.69 (Natural lift of a nonnegative graded degree). Uses:

Lean: Weil.Weight.MON.gradedDegreeNat

For $m \in \mathbf{Z}$, put

$$m_{\mathbf{N}} := \mathrm{Int.toNat}(m) \in \mathbf{N}.$$

This definition is total; it will be used as an exponent only in a branch carrying a proof that $0 \le m$.

Lemma 7.4.70 (Cast and target identities for a nonnegative degree). *Uses:* def:weil-mon-graded-degree-natural-lift

Lean: Weil.Weight.MON.gradedDegreeNat_cast

If $0 \le m$, then

$$(m_{\mathbf{N}} : \mathbf{Z}) = m, \qquad m - 2\big((m_{\mathbf{N}} + 1) : \mathbf{Z}\big) = -m - 2.$$

Proof. Uses: def:weil-mon-graded-degree-natural-lift

The first equality is the defining cast identity for Int.toNat on a nonnegative integer. Substitute it into the left side of the second equality and normalize in $\mathbf{Z}$. □

Definition 7.4.71 (All-integer primitive piece). Uses: def:weil-mon-general-graded-power, def:weil-mon-graded-degree-natural-lift, def:weil-mon-nilpotent-operator-data, def:weil-mon-filtration-certificate

Constructed by: lemma:weil-mon-graded-degree-natural-lift-cast

Lean: Weil.Weight.MON.extendedPrimitivePiece

For a certified filtration and $m \in \mathbf{Z}$, put

$$P_m := \begin{cases} \ker\big(\overline{N}_m^{\,m_{\mathbf{N}}+1} : \mathrm{Gr}_m^M V \to \mathrm{Gr}_{-m-2}^M V\big), & 0 \le m \le r, \\ 0, & \text{otherwise.} \end{cases}$$

The exponent is a natural number. In the nonzero branch the cast-and-target lemma identifies the actual target index $m-2((m_{\mathbf{N}}+1) : \mathbf{Z})$ with $-m-2$. Thus every occurrence of P_m is typed, including negative and out-of-range indices.

Definition 7.4.72 (Finite primitive source index). Uses: def:weil-mon-nilpotent-operator-data

Lean: Weil.Weight.MON.primitiveSourceIndex

For $j \in \mathbf{Z}$, define the finite subtype

$$A_j := \{a \in \mathrm{Fin}(r+1) \mid (a.\mathrm{val} : \mathbf{Z}) \le j + 2(a.\mathrm{val} : \mathbf{Z}), \quad 0 \le j + 2(a.\mathrm{val} : \mathbf{Z}) \le (r : \mathbf{Z})\}.$$

It carries the inherited Fintype instance from $\mathrm{Fin}(r+1)$. Writing $m = j + 2(a.\mathrm{val} : \mathbf{Z})$, its predicate is the exact typed string condition $0 \le (a.\mathrm{val} : \mathbf{Z}) \le m \le (r : \mathbf{Z})$; no comparison mixes natural and integer terms.

Definition 7.4.73 (Primitive image summand). Uses: def:weil-mon-extended-primitive-piece,def:weil-mon-general-graded-power,def:weil-mon-primitive-source-index

Lean: Weil.Weight.MON.primitiveImageSummand

For $a \in A_j$, put $m = j + 2(a.\,\mathrm{val} : \mathbf{Z})$ and define

$$I_{j,a} := \mathrm{range}\Big(\overline{N}_m^{\,a.\,\mathrm{val}}\big|_{P_m} : P_m \longrightarrow \mathrm{Gr}_j^M V\Big).$$

This is the typed meaning of the summand $N^a P_{j+2a}$ in the canonical primitive-decomposition source.

Definition 7.4.74 (Finite primitive source). Uses: def:weil-mon-primitive-image-summand,def:weil-mon-primitive-source-index

Lean: Weil.Weight.MON.primitiveSource

For $j \in \mathbf{Z}$, define

$$D_j := \bigoplus_{a \in A_j} I_{j,a}.$$

This is the finite direct sum $\bigoplus_a N^a P_{j+2a}$ over an explicit finite subtype.

Definition 7.4.75 (All-index primitive image summand). Uses: def:weil-mon-extended-primitive-piece,def:weil-mon-general-graded-power

Lean: Weil.Weight.MON.primitiveAllIndexSummand

For $a \in \mathbf{N}$, set $m = j + 2(a : \mathbf{Z})$ and define

$$J_{j,a} := \mathrm{range}\Big(\overline{N}_m^{\,a}\big|_{P_m} : P_m \longrightarrow \mathrm{Gr}_j^M V\Big).$$

Definition 7.4.76 (Canonical all-index primitive source). Uses: def:weil-mon-primitive-all-index-summand,def:weil-mon-primitive-decomposition-map-candidate

Lean: Weil.Weight.MON.primitiveAllIndexSource

Define

$$C_j := \bigoplus_{a \in \mathbf{N}} J_{j,a}.$$

This is the all-natural-number direct-sum source occurring literally in the canonical primitive-decomposition candidate.

Lemma 7.4.77 (Every excluded all-index summand is zero). *Uses:* def:weil-mon-primitive-all-index-summand,def:weil-mon-primitive-source-index

Lean: Weil.Weight.MON.primitiveAllIndexSummand_eq_zero

Let $a \in \mathbf{N}$ and $m = j + 2(a : \mathbf{Z})$. If a does not determine an element of A_j, then $J_{j,a} = 0$.

Proof. Uses: def:weil-mon-extended-primitive-piece,lemma:weil-mon-graded-degree-natural-lift-cast,lemma:weil-mon-general-graded-power-add

If $m < 0$ or $m > (r : \mathbf{Z})$, then $P_m = 0$ by definition, so its image is zero. In the remaining branch $0 \le m \le (r : \mathbf{Z})$, failure of the A_j-predicate means $(a : \mathbf{Z}) > m$. The cast lemma writes $m = (m_{\mathbf{N}} : \mathbf{Z})$, hence $a = m_{\mathbf{N}} + 1 + c$ for some $c \in \mathbf{N}$. The additivity law factors

$$\overline{N}_m^{\,a} = \overline{N}_{-m-2}^{\,c} \circ \overline{N}_m^{\,m_{\mathbf{N}}+1}.$$

The second factor vanishes on P_m by its kernel definition. Thus the restricted map, and therefore its range $J_{j,a}$, is zero. □

Definition 7.4.78 (Finite-to-canonical primitive reindexing). Uses: def:weil-mon-primitive-source,def:weil-mon-primitive-all-index-source

Lean: Weil.Weight.MON.primitiveFiniteToCanonical

Define $\rho_j : D_j \to C_j$ by sending the coordinate indexed by $a \in A_j$ identically to the all-index coordinate $a.\,\mathrm{val} \in \mathbf{N}$.

Definition 7.4.79 (Canonical-to-finite primitive compression). Uses: def:weil-mon-primitive-source,def:weil-mon-primitive-all-index-source

Constructed by: lemma:weil-mon-primitive-excluded-summand-zero

Lean: Weil.Weight.MON.primitiveCanonicalToFinite

Define $\pi_j : C_j \to D_j$ coordinatewise. If the natural index a satisfies the A_j-predicate, send its coordinate identically to the corresponding subtype coordinate; otherwise use the certified equality $J_{j,a} = 0$ to send that coordinate to zero.

Lemma 7.4.80 (Finite compression after reindexing). *Uses:* *def:weil-mon-primitive-finite-to-canonical*, *def:weil-mon-primitive-canonical-to-finite*

Lean: *Weil.Weight.MON.primitiveReindex_left*

For every j, $\pi_j \circ \rho_j = 1_{D_j}$.

Proof. Uses: def:weil-mon-primitive-finite-to-canonical,def:weil-mon-primitive-canonical-to-finite

On every subtype coordinate, compression takes the true predicate branch and returns the same coordinate. Direct-sum extensionality proves the equation. □

Lemma 7.4.81 (Reindexing after finite compression). *Uses:* *def:weil-mon-primitive-finite-to-canonical*, *def:weil-mon-primitive-canonical-to-finite*

Lean: *Weil.Weight.MON.primitiveReindex_right*

For every j, $\rho_j \circ \pi_j = 1_{C_j}$.

Proof. Uses: lemma:weil-mon-primitive-excluded-summand-zero

Check an all-index coordinate a. On a valid coordinate both maps are the identity. On an excluded coordinate the summand is the zero space, so both the composite and the identity are its unique linear endomorphism. Direct-sum extensionality gives the equality. □

Definition 7.4.82 (Finite/canonical primitive-source equivalence). Uses: def:weil-mon-primitive-finite-to-canonical,def:weil-mon-primitive-canonical-to-finite

Constructed by: lemma:weil-mon-primitive-reindex-left,lemma:weil-mon-primitive-reindex-right

Lean: Weil.Weight.MON.primitiveSourceEquiv

The maps ρ_j and π_j, with the two displayed inverse equations, define a linear equivalence $D_j \simeq C_j$.

Definition 7.4.83 (Finite primitive forward map). Uses: def:weil-mon-primitive-source

Lean: Weil.Weight.MON.primitiveFiniteForwardMap

Define $\mu_j : D_j \to \mathrm{Gr}_j^M V$ by $\mu_j|_{I_{j,a}} = \iota_{I_{j,a}}$.

Definition 7.4.84 (Canonical primitive forward map). Uses: def:weil-mon-primitive-all-index-source,def:weil-mon-primitive-decomposition-map-candidate

Lean: Weil.Weight.MON.canonicalPrimitiveForwardMap

Let $\kappa_j : C_j \to \mathrm{Gr}_j^M V$ be the canonical primitive-decomposition candidate: on the a-th summand it is the image subspace inclusion $J_{j,a} \hookrightarrow \mathrm{Gr}_j^M V$.

Lemma 7.4.85 (Finite and canonical forward maps agree under reindexing). *Uses:* *def:weil-mon-primitive-forward-map*, *def:weil-mon-canonical-primitive-forward-map*, *def:weil-mon-primitive-finite-to-canonical*

Lean: *Weil.Weight.MON.primitiveForwardMap_reindex*

For every integer j,

$$\kappa_j \circ \rho_j = \mu_j.$$

Proof. Uses: def:weil-mon-primitive-image-summand,def:weil-mon-primitive-all-index-summand

On the coordinate $a \in A_j$, both sides are the inclusion of the same range subspace, because reindexing replaces a only by $a.\mathrm{val}$. Direct-sum extensionality proves equality. This is the explicit adapter between the finite presentation and the literal all-a map in the historical contract. □

Lemma 7.4.86 (Block formula for the finite primitive forward map). *Uses:* *def:weil-mon-jordan-chain-decomposition*, *def:weil-mon-primitive-forward-map*

Lean: *Weil.Weight.MON.primitiveFiniteForwardMap_onJordanBlock*

Fix selected Jordan data. On a block of length d, the only nonzero primitive piece is in degree $d-1$, generated by the top class. Every degree-j block vector has the unique natural distance $a = (d-1-j)/2$, and that distance, viewed in $\mathrm{Fin}(r+1)$, lies in A_j. The corresponding summand maps to the same graded basis vector.

Proof. Uses: lemma:weil-mon-jordan-graded-basis,lemma:weil-mon-graded-degree-natural-lift-cast

Write the block as $e_0, \dots, e_{d-1}$, with $Ne_u = e_{u-1}$. Its top class has degree $d-1$ and is killed by the natural power N^d. A smaller nonnegative degree $m < d-1$ is not primitive because $N^{m_{\mathbf{N}}+1}$ sends its exact-weight basis class to a nonzero basis class. The degree equation $j = d-1-2(a : \mathbf{Z})$, together with $d-1 \le r$, proves every casted inequality in the definition of A_j. The component formula is $N^a[e_{d-1}]$. □

Lemma 7.4.87 (Finite primitive forward map is bijective conditionally). *Uses:* *def: weil-mon-jordan-chain-decomposition, def: weil-mon-primitive-forward-map*

Lean: *Weil.Weight.MON.primitiveFiniteForwardMap_bijective*

Given selected Jordan data and a specified stepwise identification of the certified filtration with its Jordan filtration, μ_j is bijective for every integer j.

Proof. Uses: lemma:weil-mon-primitive-forward-map-on-jordan-block,lemma:weil-mon-coordinate-quotient-basis

Transport all graded pieces and image summands along the supplied stepwise identification. The block formula maps the basis of D_j, indexed by a block top and its unique natural distance, bijectively to the exact-weight j Jordan basis. Empty degrees give the unique equivalence between zero spaces. □

Definition 7.4.88 (Finite primitive inverse selected from Jordan data). Uses: def:weil-mon-primitive-forward-map

Constructed by: lemma:weil-mon-primitive-forward-map-bijective

Lean: Weil.Weight.MON.primitiveFiniteInverse

For the selected data, let $\nu_j^{\mathrm{fin}} : \mathrm{Gr}_j^M V \to D_j$ be the inverse of μ_j.

Lemma 7.4.89 (Finite primitive inverse left equation). *Uses:* *def: weil-mon-primitive-forward-map, def: weil-mon-primitive-finite-inverse*

Lean: *Weil.Weight.MON.primitiveFiniteInverse_left*

For every j, $\nu_j^{\mathrm{fin}}\mu_j = 1_{D_j}$.

Proof. Uses: def:weil-mon-primitive-finite-inverse

This is the left-inverse equation of the selected finite-source equivalence. □

Lemma 7.4.90 (Finite primitive inverse right equation). *Uses:* *def: weil-mon-primitive-forward-map, def: weil-mon-primitive-finite-inverse*

Lean: *Weil.Weight.MON.primitiveFiniteInverse_right*

For every j, $\mu_j\nu_j^{\mathrm{fin}} = 1_{\mathrm{Gr}_j^M V}$.

Proof. Uses: def:weil-mon-primitive-finite-inverse

This is the right-inverse equation of the selected finite-source equivalence. □

Definition 7.4.91 (Canonical primitive inverse). Uses: def:weil-mon-primitive-finite-inverse,def:weil-mon-primitive-finite-to-canonical

Lean: Weil.Weight.MON.primitiveInverse

Define the inverse with the source required by the historical contract by

$$\nu_j := \rho_j \circ \nu_j^{\mathrm{fin}} : \mathrm{Gr}_j^M V \longrightarrow C_j.$$

Lemma 7.4.92 (Canonical primitive inverse left equation). *Uses:* *def: weil-mon-canonical-primitive-forward-map, def: weil-mon-primitive-inverse*

Lean: *Weil.Weight.MON.primitiveInverse_left*

For every j, $\nu_j\kappa_j = 1_{C_j}$.

Proof. Uses: lemma:weil-mon-primitive-forward-map-reindex,lemma:weil-mon-primitive-reindex-right,lemma:weil-mon-primitive-finite-inverse-left

Insert $1_{C_j} = \rho_j\pi_j$, then calculate

$$\nu_j\kappa_j = \rho_j\nu_j^{\mathrm{fin}}\kappa_j\rho_j\pi_j = \rho_j\nu_j^{\mathrm{fin}}\mu_j\pi_j = \rho_j\pi_j = 1_{C_j}.$$

□

Lemma 7.4.93 (Canonical primitive inverse right equation). *Uses:* *def: weil-mon-canonical-primitive-forward-map, def: weil-mon-primitive-inverse*

Lean: *Weil.Weight.MON.primitiveInverse_right*

For every j, $\kappa_j\nu_j = 1_{\mathrm{Gr}_j^M V}$.

Proof. Uses: lemma:weil-mon-primitive-forward-map-reindex,lemma:weil-mon-primitive-finite-inverse-right

By definition and the reindex equation, $\kappa_j\nu_j = \kappa_j\rho_j\nu_j^{\mathrm{fin}} = \mu_j\nu_j^{\mathrm{fin}} = 1$. □

Definition 7.4.94 (Lowering map on the finite primitive source). Uses: `def:weil-mon-primitive-source`, `def:weil-mon-general-graded-power`, `def:weil-mon-extended-primitive-piece`

Constructed by: `lemma:weil-mon-general-graded-power-zero-succ`, `lemma:weil-mon-graded-degree-natural-lift-cast`

Lean: `Weil.Weight.MON.primitiveFiniteSourceLowering`

For $a \in A_j$, put $m = j + 2(a.\mathrm{val} : \mathbf{Z})$. If $(a.\mathrm{val} : \mathbf{Z}) < m$, apply one graded N into the coordinate of A_{j-2} with value $a.\mathrm{val}+1$; the inequality $a.\mathrm{val}+1 \le m \le r$ supplies its $\mathrm{Fin}(r+1)$ bound and its predicate. If $(a.\mathrm{val} : \mathbf{Z}) = m$, use the zero component. These components define $\delta_j : D_j \to D_{j-2}$.

Lemma 7.4.95 (Finite forward map intertwines lowering). *Uses:* *def:weil-mon-primitive-forward-map*, *def:weil-mon-primitive-source-lowering*

Lean: *Weil.Weight.MON.primitiveFiniteForwardMap_equivariant*

For every j, $\mu_{j-2}\delta_j = \overline{N}_j^{1}\mu_j$.

Proof. Uses: `lemma:weil-mon-general-graded-power-zero-succ`, `lemma:weil-mon-graded-degree-natural-lift-cast`

On a non-bottom coordinate this is the successor law, with the same source degree because $(j-2)+2(a+1) = j+2a$ after integer coercion. On a bottom coordinate $(a : \mathbf{Z}) = m$, the cast lemma identifies $a = m_{\mathbf{N}}$, and primitivity says $\overline{N}_m^{m_{\mathbf{N}}+1} = 0$ on P_m. Direct-sum extensionality proves the equation. □

Lemma 7.4.96 (Finite inverse intertwines lowering). *Uses:* *def:weil-mon-primitive-finite-inverse*, *def:weil-mon-primitive-source-lowering*

Lean: *Weil.Weight.MON.primitiveFiniteInverse_equivariant*

For every j, $\delta_j\nu_j^{\mathrm{fin}} = \nu_{j-2}^{\mathrm{fin}}\overline{N}_j^{1}$.

Proof. Uses: `lemma:weil-mon-primitive-forward-equivariant`, `lemma:weil-mon-primitive-finite-inverse-left`, `lemma:weil-mon-primitive-finite-inverse-right`

Postcompose both sides with the injective map μ_{j-2}; forward equivariance and the two finite inverse equations make the composites equal. □

Definition 7.4.97 (Canonical primitive-source lowering). Uses: `def:weil-mon-primitive-source-lowering`, `def:weil-mon-primitive-finite-to-canonical`, `def:weil-mon-primitive-canonical-to-finite`

Lean: `Weil.Weight.MON.canonicalPrimitiveSourceLowering`

Define

$$\delta_j^{\mathrm{can}} := \rho_{j-2} \circ \delta_j \circ \pi_j : C_j \longrightarrow C_{j-2}.$$

Lemma 7.4.98 (Canonical forward lowering equivariance). *Uses:* *def:weil-mon-canonical-primitive-forward-map*, *def:weil-mon-canonical-primitive-source-lowering*

Lean: *Weil.Weight.MON.canonicalPrimitiveForwardMap_equivariant*

For every j, $\kappa_{j-2}\delta_j^{\mathrm{can}} = \overline{N}_j^{1}\kappa_j$.

Proof. Uses: `lemma:weil-mon-primitive-forward-map-reindex`, `lemma:weil-mon-primitive-forward-equivariant`, `lemma:weil-mon-primitive-reindex-right`

Expand δ^{can}, replace $\kappa\rho$ by μ, use finite forward equivariance, and finish with $\rho\pi = 1_C$. □

Lemma 7.4.99 (Canonical inverse lowering equivariance). *Uses:* *def:weil-mon-primitive-inverse*, *def:weil-mon-canonical-primitive-source-lowering*

Lean: *Weil.Weight.MON.primitiveInverse_equivariant*

For every j, $\delta_j^{\mathrm{can}}\nu_j = \nu_{j-2}\overline{N}_j^{1}$.

Proof. Uses: `lemma:weil-mon-primitive-finite-inverse-equivariant`, `lemma:weil-mon-primitive-reindex-left`

Expand δ^{can} and ν. The middle composite $\pi_j\rho_j$ is the identity on D_j; the remaining equality is finite inverse equivariance followed by ρ_{j-2}. □

Lemma 7.4.100 (Uniqueness of the canonical primitive inverse). *Uses:* *def:weil-mon-canonical-primitive-forward-map*, *def:weil-mon-primitive-inverse*

Lean: *Weil.Weight.MON.primitiveInverse_unique*

If $\nu'_j : \mathrm{Gr}_j^M V \to C_j$ is a two-sided inverse to κ_j, then $\nu'_j = \nu_j$.

Proof. Uses: lemma:weil-mon-primitive-inverse-left,lemma:weil-mon-primitive-inverse-right
Compute $\nu_j' = \nu_j'(\kappa_j\nu_j) = (\nu_j'\kappa_j)\nu_j = \nu_j$. □

Definition 7.4.101 (Total graded lowering data). Uses: def:weil-mon-general-graded-power,def:weil-mon-filtration-endpoint-extension

Lean: Weil.Weight.MON.totalGradedLoweringData
For a certified filtration put

$$\mathcal{G}_M := \bigoplus_{-r\le j\le r} \mathrm{Gr}_j^M V.$$

Let F have component $\overline{N}_j^1 : \mathrm{Gr}_j^M V \to \mathrm{Gr}_{j-2}^M V$, using zero outside this finite range, and let H act on the degree-j component by the image of $j \in \mathbf{Z}$ in the characteristic-zero field $\overline{E}$.

Definition 7.4.102 (Raw auxiliary sl2 splitting datum). Uses: def:weil-mon-total-graded-lowering-data

Lean: Weil.Weight.MON.AuxiliarySl2SplittingDatum
A raw auxiliary splitting is an endomorphism $E : \mathcal{G}_M \to \mathcal{G}_M$ raising degree by two and satisfying

$$[H,E] = 2E, \qquad [H,F] = -2F, \qquad [E,F] = H$$

in $\mathrm{End}_{\overline{E}}(\mathcal{G}_M)$. No primitive inverse or inverse equation is a field of this raw datum.

Definition 7.4.103 (Primitive inverse candidate attached to a splitting). Uses: def:weil-mon-auxiliary-sl2-splitting-datum,def:weil-mon-primitive-all-index-source

Lean: Weil.Weight.MON.Sl2PrimitiveInverseCandidate
For a raw splitting $\mathcal{S}$, an inverse candidate is a family

$$\sigma_j^{\mathcal{S}} : \mathrm{Gr}_j^M V \longrightarrow C_j \qquad (j \in \mathbf{Z}).$$

No inverse equation is included.

Definition 7.4.104 (Certified splitting-inverse predicate). Uses: def:weil-mon-sl2-primitive-inverse-candidate,def:weil-mon-canonical-primitive-forward-map

Lean: Weil.Weight.MON.IsCertifiedSl2PrimitiveInverse
Define IsSl2Inverse$(\mathcal{S},\sigma)$ by requiring, for every j,

$$\sigma_j^{\mathcal{S}}\kappa_j = 1_{C_j}, \qquad \kappa_j\sigma_j^{\mathcal{S}} = 1_{\mathrm{Gr}_j^M V}.$$

Definition 7.4.105 (Certified auxiliary sl2 primitive splitting). Uses: def:weil-mon-auxiliary-sl2-splitting-datum,def:weil-mon-sl2-primitive-inverse-candidate,def:weil-mon-certified-sl2-primitive-inverse-predicate

Lean: Weil.Weight.MON.CertifiedAuxiliarySl2PrimitiveSplitting
A certified auxiliary splitting consists of a raw datum $\mathcal{S}$, an inverse candidate $\sigma^{\mathcal{S}}$, and a witness of IsSl2Inverse$(\mathcal{S},\sigma^{\mathcal{S}})$.

Lemma 7.4.106 (Certified sl2 inverse left equation). *Uses:* def:weil-mon-certified-auxiliary-sl2-splitting

Lean: Weil.Weight.MON.CertifiedAuxiliarySl2PrimitiveSplitting.left
For every certified auxiliary splitting and every j, $\sigma_j^{\mathcal{S}}\kappa_j = 1_{C_j}$.

Proof. Uses: def:weil-mon-certified-auxiliary-sl2-splitting
This is the left-equation field of the certified predicate. □

Lemma 7.4.107 (Certified sl2 inverse right equation). *Uses:* def:weil-mon-certified-auxiliary-sl2-splitting

Lean: Weil.Weight.MON.CertifiedAuxiliarySl2PrimitiveSplitting.right
For every certified auxiliary splitting and every j, $\kappa_j\sigma_j^{\mathcal{S}} = 1_{\mathrm{Gr}_j^M V}$.

Proof. Uses: def:weil-mon-certified-auxiliary-sl2-splitting
This is the right-equation field of the certified predicate. □

Lemma 7.4.108 (Independence of certified auxiliary sl2 splittings). *Uses:* *def:weil-mon-certified-auxiliary-sl2-splitting*

Lean: *Weil.Weight.MON.primitiveInverse_splittingIndependent*

If $\mathcal{S}$ and $\mathcal{T}$ are certified auxiliary splittings, then $\sigma_j^{\mathcal{S}} = \sigma_j^{\mathcal{T}}$ for every j.

Proof. Uses: `lemma:weil-mon-certified-sl2-inverse-left`, `lemma:weil-mon-certified-sl2-inverse-right`

All maps are typed against the same κ_j, and

$$\sigma_j^{\mathcal{S}} = \sigma_j^{\mathcal{S}}(\kappa_j \sigma_j^{\mathcal{T}}) = (\sigma_j^{\mathcal{S}} \kappa_j)\sigma_j^{\mathcal{T}} = \sigma_j^{\mathcal{T}}.$$

□

Lemma 7.4.109 (Every certified splitting inverse is canonical). *Uses:* *def:weil-mon-certified-auxiliary-sl2-splitting*, *def:weil-mon-primitive-inverse*

Lean: *Weil.Weight.MON.sl2PrimitiveInverse_eq_canonical*

For every certified auxiliary splitting and every j, $\sigma_j^{\mathcal{S}} = \nu_j$.

Proof. Uses: `lemma:weil-mon-certified-sl2-inverse-left`, `lemma:weil-mon-certified-sl2-inverse-right`, `lemma:weil-mon-primitive-inverse-unique`

The two projected fields make $\sigma_j^{\mathcal{S}}$ a two-sided inverse to κ_j. Apply uniqueness of the canonical primitive inverse. □

Theorem 7.4.110 (Inverse construction from raw sl2 relations). *Uses:* *def:weil-mon-auxiliary-sl2-splitting-datum*, *def:weil-mon-certified-auxiliary-sl2-splitting*

Lean: *Weil.Weight.MON.sl2InverseFromRelations*

Open obligation: *OBL-WGT-MON-SL2-INVERSE-FROM-RELATIONS*

Every raw auxiliary $\mathfrak{sl}_2$-splitting datum admits an inverse candidate and a certified splitting-inverse predicate.

Proof. Uses:

This is the finite-dimensional $\mathfrak{sl}_2$-representation theorem. No proof is asserted in this candidate; conditional independence above needs only a supplied certified inverse and does not import this open existence leaf. □

Proposition 7.4.111 (Conditional primitive-decomposition certificate). *Uses:* *def:weil-mon-jordan-chain-decomposition*, *def:weil-mon-primitive-decomposition-certificate*

Lean: *Weil.Weight.MON.primitiveCertificate_ofJordanData*

A certified monodromy filtration together with selected Jordan data and a specified stepwise identification with the Jordan filtration has a primitive-decomposition certificate on the literal canonical source $C_j = \bigoplus_{a\geq 0} N^a P_{j+2a}$.

Proof. Uses: `def:weil-mon-primitive-source-equivalence`, `def:weil-mon-canonical-primitive-forward-map`, `def:weil-mon-primitive-inverse`, `lemma:weil-mon-primitive-inverse-left`, `lemma:weil-mon-primitive-inverse-right`, `def:weil-mon-canonical-primitive-source-lowering`, `lemma:weil-mon-canonical-primitive-forward-equivariant`, `lemma:weil-mon-primitive-inverse-equivariant`, `lemma:weil-mon-primitive-splitting-independent`, `lemma:weil-mon-sl2-inverse-eq-canonical`

Install κ_j, its canonical inverse ν_j, and the transported lowering maps. The source equivalence records the finite/all-index adapter. The cited leaves supply both inverse equations, forward and inverse lowering equivariance, and equality with every typed auxiliary $\mathfrak{sl}_2$-splitting inverse. The finite/all-index adapter already proved that κ_j is the historical candidate map. No unconditional Jordan or PID existence is asserted. □

7.4.4 Typed signed dual datum and conditional assembly

Definition 7.4.112 (Signed-transpose inverse on dual graded pieces). Uses: `def:weil-mon-filtration-certificate`, `def:weil-mon-dual-filtration-candidate`

Constructed by: `lemma:weil-mon-annihilator-quotient-duality`

Lean: `Weil.Weight.MON.signedTransposeInverse`

For $j \in \mathbf{N}$, let

$$p_j : \mathrm{Gr}^M_{(j:\mathbf{Z})} V \longrightarrow \mathrm{Gr}^M_{-(j:\mathbf{Z})} V, \qquad s_j : \mathrm{Gr}^M_{-(j:\mathbf{Z})} V \longrightarrow \mathrm{Gr}^M_{(j:\mathbf{Z})} V$$

be the certified natural graded power and its supplied inverse. Write

$$\Theta_{(j:\mathbf{Z})} : \mathrm{Gr}^{M^\vee}_{(j:\mathbf{Z})}(V^\vee) \xrightarrow{\sim} (\mathrm{Gr}^M_{-(j:\mathbf{Z})} V)^\vee, \quad \Theta_{-(j:\mathbf{Z})} : \mathrm{Gr}^{M^\vee}_{-(j:\mathbf{Z})}(V^\vee) \xrightarrow{\sim} (\mathrm{Gr}^M_{(j:\mathbf{Z})} V)^\vee$$

for the annihilator-quotient equivalences. Define the proposed inverse to the dual graded power by the typed composite

$$t_j := \Theta^{-1}_{(j:\mathbf{Z})} \circ \left((-1)^j s_j^*\right) \circ \Theta_{-(j:\mathbf{Z})} : \mathrm{Gr}^{M^\vee}_{-(j:\mathbf{Z})}(V^\vee) \longrightarrow \mathrm{Gr}^{M^\vee}_{(j:\mathbf{Z})}(V^\vee).$$

Lemma 7.4.113 (Left equation for the signed-transpose inverse). *Uses:* `def:weil-mon-signed-transpose-inverse`

Lean: `Weil.Weight.MON.signedTransposeInverse_left`

For $j \in \mathbf{N}$, if q_j is the natural graded power induced by $(N^\vee)^j$, then

$$t_j \circ q_j = 1_{\mathrm{Gr}^{M^\vee}_{(j:\mathbf{Z})}(V^\vee)}.$$

Proof. Uses: `lemma:weil-mon-annihilator-graded-signed-transpose,def:weil-mon-induced-power-right-inverse-predicate`

Conjugate by $\Theta_{(j:\mathbf{Z})}$. The signed-transpose lemma identifies q_j with $(-1)^j p_j^*$, while the definition identifies t_j with $(-1)^j s_j^*$. Their composite is

$$(-1)^{2j} s_j^* p_j^* = (p_j s_j)^* = 1,$$

using the supplied primal right-inverse equation. Conjugation by an equivalence reflects equality. □

Lemma 7.4.114 (Right equation for the signed-transpose inverse). *Uses:* `def:weil-mon-signed-transpose-inverse`

Lean: `Weil.Weight.MON.signedTransposeInverse_right`

For $j \in \mathbf{N}$, if q_j is the natural graded power induced by $(N^\vee)^j$, then

$$q_j \circ t_j = 1_{\mathrm{Gr}^{M^\vee}_{-(j:\mathbf{Z})}(V^\vee)}.$$

Proof. Uses: `lemma:weil-mon-annihilator-graded-signed-transpose,def:weil-mon-induced-power-left-inverse-predicate`

Conjugate by $\Theta_{-(j:\mathbf{Z})}$. The composite becomes

$$(-1)^{2j} p_j^* s_j^* = (s_j p_j)^* = 1,$$

using the supplied primal left-inverse equation. This proves the second inverse equation without combining it with the construction of t_j. □

Lemma 7.4.115 (The annihilator filtration of a certificate is certified). *Uses:* `def:weil-mon-dual-filtration-candidate,def:weil-mon-filtration-certificate`

Lean: `Weil.Weight.MON.dualFiltration_isCertified`

For every certified filtration M, the annihilator filtration

$$M_k^\vee = \mathrm{Ann}(M_{-k-1})$$

is a certified filtration for $N^\vee = -N^*$.

Proof. Uses: `def:weil-mon-filtration-endpoint-extension,def:weil-mon-dual-operator,def:weil-mon-dual-filtration-candidate,lemma:weil-mon-signed-dual-nilpotent-datum,lemma:weil-mon-annihilator-quotient-duality,lemma:weil-mon-annihilator-graded-signed-transpose,def:weil-mon-signed-transpose-inverse,lemma:weil-mon-signed-transpose-inverse-left,lemma:weil-mon-signed-transpose-inverse-right`

Annihilators reverse inclusions, so index reversal makes $M^\vee$ increasing. The endpoints are $\mathrm{Ann}(V) = 0$ and $\mathrm{Ann}(0) = V^\vee$. If λ annihilates M_{-k-1}, then $-\lambda \circ N$ annihilates M_{-k+1}, proving the shift.

For $j \in \mathbf{N}$, signed transpose supplies quotient typing, representative independence, and the formula for the natural dual power. Install the separately defined t_j; its left and right lemmas supply the two certificate fields. Together with the preceding filtration fields, this gives a certificate, including the zero-space cases $j > r$. □

Theorem 7.4.116 (Intrinsic characterization of every certified filtration). *Uses:* `def:weil-mon-filtration-certificate,def:weil-mon-intrinsic-kernel-image-filtration`

Lean: `Weil.Weight.MON.certifiedFiltration_eq_intrinsic`

Conditional on the elementary-divisor leaf, every certified monodromy filtration equals the intrinsic kernel-image filtration at every integer index.

Proof. Uses: `lemma:weil-mon-intrinsic-contained-in-certified,lemma:weil-mon-intrinsic-dual-annihilator,lemma:weil-mon-certified-dual-filtration,lemma:weil-mon-finite-dimensional-double-annihilator`

Containment gives $W_k^{\mathrm{int}}(N) \subseteq M_k$. Apply it to the certified dual filtration:

$$W_k^{\mathrm{int}}(N^\vee) \subseteq \mathrm{Ann}(M_{-k-1}).$$

The dual-annihilator formula identifies the left side with $\mathrm{Ann}(W_{-k-1}^{\mathrm{int}}(N))$. Taking annihilators and using finite-dimensional double-annihilator separation gives $M_{-k-1} \subseteq W_{-k-1}^{\mathrm{int}}(N)$. Reindexing supplies the reverse containment in every degree, including the constant outer ranges. □

Proposition 7.4.117 (Monodromy-filtration uniqueness certificate). *Uses:* *def:weil-mon-filtration-uniqueness-certificate*

Lean: *Weil.Weight.MON.filtrationUniquenessCertificate_exists*

Conditional on the elementary-divisor leaf, any two certified monodromy filtrations on the same nilpotent datum admit a uniqueness certificate.

Proof. Uses: thm:weil-mon-certified-filtration-intrinsic,def:weil-mon-filtration-comparison-map-candidate,def:weil-mon-filtration-comparison-inclusion-predicate

Both filtrations equal $W^{\mathrm{int}}(N)$ at every step. Restrict the identity of V along these equalities in both directions. Each restriction lands in the target step and becomes the identity after inclusion into V, which supplies the comparison maps and predicates. □

Proposition 7.4.118 (Dual-filtration certificate from atomic duality data). *Uses:* *def:weil-mon-filtration-certificate*, *def:weil-mon-dual-filtration-certificate*

Lean: *Weil.Weight.MON.dualCertificate_ofAtomicData*

Every certified monodromy filtration has a dual-filtration comparison certificate for the signed dual nilpotent datum.

Proof. Uses: lemma:weil-mon-signed-dual-nilpotent-datum,lemma:weil-mon-annihilator-quotient-duality,lemma:weil-mon-certified-dual-filtration,lemma:weil-mon-annihilator-graded-signed-transpose,def:weil-mon-signed-transpose-inverse,lemma:weil-mon-signed-transpose-inverse-left,lemma:weil-mon-signed-transpose-inverse-right

The annihilator-quotient equivalence and its explicitly constructed inverse give the comparison and inverse maps and their two inverse equations. The certified-dual lemma gives increasingness, endpoints, shift, and quotient well-definedness. The signed-transpose formula gives compatibility with the operator. Install t_j as the specified inverse and use its separate left and right equations. The signed-dual datum supplies both its explicit finite-dimensionality field and its nilpotence field, so these atoms assemble the required certificate with the same bound r. □

Definition 7.4.119 (Local Weil-representation datum). Uses: def:weil-coefficient-data

Lean: Weil.Weight.MON.LocalWeilRepresentationData

At a closed point s, a local Weil-representation datum consists of a finite-dimensional $\overline{E}$-space V, an inertia action $\rho_I : I_s \to \mathrm{GL}(V)$, a selected geometric Frobenius lift ϕ_s, and the conjugation equation

$$\phi_s \rho_I(\tau) \phi_s^{-1} = \rho_I(\tau^{q_s^{-1}})$$

in the tame quotient convention.

Definition 7.4.120 (Quasi-unipotence exponent candidate). Uses: def:weil-mon-local-weil-representation-data

Lean: Weil.Weight.MON.QuasiUnipotenceExponentCandidate

A quasi-unipotence exponent candidate is a positive integer m and a finite-index subgroup $I_s' \subseteq I_s$ on which every $\rho_I(\tau)^m$ is proposed to be unipotent.

Definition 7.4.121 (Quasi-unipotence certificate). Uses: def:weil-mon-quasi-unipotence-exponent-candidate

Lean: Weil.Weight.MON.QuasiUnipotenceCertificate

A quasi-unipotence certificate supplies the finite-index proof and, for every $\tau \in I_s'$, a nilpotence exponent for $\rho_I(\tau)^m - 1$. Its construction is Grothendieck's local monodromy theorem, cited by Deligne (1.7.2)–(1.7.3), printed pp. 170–171 (PDF pp. 35–36).

Open obligation: OBL-WGT-MON-QUASI-UNIPOTENCE

Definition 7.4.122 (Finite logarithm of unipotent inertia). Uses: def:weil-mon-quasi-unipotence-certificate

Lean: Weil.Weight.MON.unipotentLogarithm

For $U = \rho_I(\tau)^m$ and a supplied exponent $(U-1)^{r+1} = 0$, define

$$\log(U) := \sum_{a=1}^{r} \frac{(-1)^{a+1}}{a} (U-1)^a.$$

This is a finite sum over the characteristic-zero coefficient field.

Lemma 7.4.123 (The finite logarithm is nilpotent). *Uses:* *def:weil-mon-unipotent-logarithm*

Lean: *Weil.Weight.MON.unipotentLogarithm_isNilpotent*

The endomorphism $\log(U)$ *is nilpotent.*

Proof. Uses: def:weil-mon-unipotent-logarithm

Factor the finite polynomial as

$$\log(U) = (U-1)\sum_{a=1}^{r} \frac{(-1)^{a+1}}{a}(U-1)^{a-1}.$$

Both factors are polynomials in $U-1$, so they commute. Raising to the power $r+1$ produces a factor $(U-1)^{r+1} = 0$. □

Definition 7.4.124 (Normalized local nilpotent operator candidate). Uses: def:weil-tate-line,def:weil-mon-unipotent-logarithm

Lean: Weil.Weight.MON.normalizedLocalNilpotentCandidate

Using a selected tame character $t_\ell : I'_s \to \mathbf{Z}_\ell(1)$, define the candidate

$$N : V(1) \longrightarrow V$$

by the equation $\rho_I(\tau) = \exp(t_\ell(\tau)N)$ on the chosen finite-index subgroup. The equation, normalization, and independence of τ are separate predicates.

Definition 7.4.125 (Normalized logarithm compatibility predicate). Uses: def:weil-mon-normalized-local-nilpotent-candidate

Lean: Weil.Weight.MON.IsNormalizedLogarithmCompatible

Define compatibility by requiring

$$\log(\rho_I(\tau)^m) = m\, t_\ell(\tau)N$$

for each allowed τ, together with the geometric-Frobenius relation $\phi_s N \phi_s^{-1} = q_s^{-1}N$.

Definition 7.4.126 (Local Frobenius-lift comparison candidate). Uses: def:weil-mon-local-weil-representation-data

Lean: Weil.Weight.MON.FrobeniusLiftComparisonCandidate

For two geometric Frobenius lifts ϕ_s, ϕ'_s, a comparison candidate is an inertia element τ and the proposed equation $\phi'_s = \rho_I(\tau)\phi_s$.

Definition 7.4.127 (Frobenius-lift root-of-unity certificate). Uses: def:weil-mon-frobenius-lift-comparison-candidate,def:weil-mon-quasi-unipotence-certificate

Lean: Weil.Weight.MON.FrobeniusLiftRootUnityCertificate

A lift certificate supplies the comparison equation and a bijection between the two semisimplified spectra whose paired eigenvalues differ by roots of unity. This expands Deligne Lemma (1.7.4), printed p. 171 (PDF p. 36).

Open obligation: OBL-WGT-MON-FROBENIUS-LIFT

Lemma 7.4.128 (Local weights are independent of a certified Frobenius lift). *Uses:* *def:weil-mon-frobenius-lift-root-unity-certificate,def:weil-weight-local-iota-pure-predicate*

Lean: *Weil.Weight.MON.FrobeniusLiftRootUnityCertificate.isIotaPure_iff*

Paired eigenvalues under a lift certificate have the same fixed-ι *weight.*

Proof. Uses: def:weil-mon-frobenius-lift-root-unity-certificate,def:weil-weight-local-iota-pure-predicate

For a paired eigenvalue relation $\alpha' = \zeta\alpha$, the certified root of unity satisfies $|\iota(\zeta)| = 1$. Hence $|\iota(\alpha')| = |\iota(\alpha)|$, and the two purity equations are equivalent. □

Definition 7.4.129 (Local monodromy weight-filtration candidate). Uses: def:weil-mon-normalized-local-nilpotent-candidate,def:weil-mon-filtration-candidate

Lean: Weil.Weight.MON.LocalWeightFiltrationCandidate

For the normalized $N : V(1) \to V$, define a local weight-filtration candidate by the monodromy candidate $M_\bullet V$, together with proposed Frobenius actions on every $\mathrm{Gr}^M_j V$.

Definition 7.4.130 (Local weight-monodromy graded-purity predicate)**.** Uses: def:weil-mon-local-weight-filtration-candidate,def:weil-weight-pure-spectrum-predicate

Lean: Weil.Weight.MON.IsGradedPure

For an ambient weight w, define graded purity by requiring

$$\mathrm{Gr}_j^M V \quad \text{to be pure of weight } w+j$$

for every j. The sign agrees with Deligne Theorem (1.8.4), printed pp. 175–176 (PDF pp. 40–41).

Definition 7.4.131 (Weight-monodromy antecedent certificate)**.** Uses: def:weil-mon-filtration-certificate,def:weil-mon-primitive-decomposition-certificate,def:weil-mon-tensor-filtration-certificate,def:weil-mon-dual-filtration-certificate,def:weil-mon-quasi-unipotence-certificate,def:weil-mon-normalized-logarithm-compatibility-predicate,def:weil-pur-complete-antecedent-certificate

Lean: Weil.Weight.MON.WeightMonodromyAntecedentCertificate

A weight-monodromy antecedent certificate packages the filtration, primitive, tensor, dual, quasi-unipotence, normalized-logarithm, finite-cover comparison, and PUR certificates used in Deligne (1.8.4). It contains no graded-purity field.

Open obligation: OBL-WGT-MON-FINITE-COVER

Definition 7.4.132 (Primitive tensor-bound certificate)**.** Uses: def:weil-mon-weight-monodromy-antecedent-certificate,def:weil-mon-primitive-piece-candidate,def:weil-pur-local-numerical-weight

Lean: Weil.Weight.MON.PrimitiveTensorBoundCertificate

For every primitive-piece eigenvalue α, every positive k, and its graded index j, a primitive tensor-bound certificate supplies both

$$w+j-\frac{1}{k} \le \nu_\iota(e,\alpha) \quad \text{and} \quad \nu_\iota(e,\alpha) \le w+j+\frac{1}{k}.$$

It records the finite-cover pullback, tensor-square, dual, and descent maps that produce these inequalities.

Open obligation: OBL-WGT-MON-PRIMITIVE-BOUNDS

Lemma 7.4.133 (Primitive tensor bounds give the exact graded weight)**.** *Uses:* def:weil-mon-primitive-tensor-bound-certificate,def:weil-weight-local-iota-pure-predicate

Lean: Weil.Weight.MON.PrimitiveTensorBoundCertificate.isIotaPure

Every eigenvalue on $P_j(V,N)$ is pure of weight $w+j$.

Proof. Uses: def:weil-mon-primitive-tensor-bound-certificate,lemma:weil-pur-numerical-weight-realizes-modulus

If $\nu_\iota(e,\alpha) < w+j$, the Archimedean property supplies $k>0$ with $1/k < w+j-\nu_\iota(e,\alpha)$, contradicting the certified lower inequality. The same argument contradicts the upper inequality when $w+j < \nu_\iota(e,\alpha)$. Thus the numerical weight equals $w+j$, and the numerical-weight modulus lemma gives the purity equation. □

Proposition 7.4.134 (Conditional weight-monodromy theorem)**.** *Uses:* def:weil-mon-local-weight-filtration-candidate,def:weil-mon-graded-purity-predicate,def:weil-mon-primitive-decomposition-certificate,def:weil-mon-primitive-tensor-bound-certificate

Lean: Weil.Weight.MON.gradedPure

If all primitive pieces carry primitive tensor-bound certificates, then the local monodromy filtration is graded-pure with Gr_j^M of weight $w+j$.

Proof. Uses: def:weil-mon-primitive-decomposition-certificate,lemma:weil-mon-primitive-bounds-give-weight,lemma:weil-weight-spectrum-tate-twist-pure,lemma:weil-weight-spectrum-direct-sum-pure

The primitive decomposition writes each graded piece as the finite direct sum of the appropriate $N^a P_{j+2a}$. The relation $N : V(1) \to V$ changes an N^a-image by the exact Tate normalization encoded in the graded index. Apply the primitive weight lemma to each primitive summand, the Tate-shift lemma to each N^a-image, and the direct-sum lemma. Every summand in Gr_j^M has weight $w+j$. □

Definition 7.4.135 (Boundary inertia-invariant candidate)**.** Uses: def:weil-mon-local-weil-representation-data

Lean: Weil.Weight.MON.boundaryInvariantCandidate

For $j : U \hookrightarrow X$ and $s \in X \setminus U$, define the candidate

$$(j_*\mathcal{F})_s := V^{I_s}$$

as the kernel of every $\rho_I(\tau)-1$, with its induced geometric Frobenius candidate.

Definition 7.4.136 (Boundary tensor-limit certificate). Uses: def:weil-mon-boundary-invariant-candidate,def:weil-pur-trace-l-function-certificate,def:weil-pur-euler-convergence-certificate,def:weil-lisse-heart-operations-typed-context

Lean: Weil.Weight.MON.BoundaryTensorLimitCertificate

For a pure lisse sheaf of weight w, a boundary tensor-limit certificate assigns to each invariant eigenvalue α and positive integer k the inequality

$$\nu_\iota(e_s, \alpha) \le w + \frac{1}{k}.$$

It records the trace, convergence, low-degree, and tensor-power maps used in Deligne Lemma (1.8.1), printed p. 175 (PDF p. 40).

Open obligation: OBL-WGT-MON-BOUNDARY-UPPER

Proposition 7.4.137 (Boundary invariants have weights at most w). Uses: def:weil-mon-boundary-tensor-limit-certificate, def:weil-weight-spectrum-at-most-predicate

Lean: Weil.Weight.MON.boundaryInvariant_weightAtMost

Every eigenvalue on $(j_*\mathcal{F})_s$ has weight at most w.

Proof. Uses: def:weil-mon-boundary-tensor-limit-certificate,def:weil-pur-local-numerical-weight,def:weil-weight-iota-at-most-predicate

Fix an eigenvalue. The same Archimedean contradiction used for the PUR tensor limit changes the inequalities $\nu_\iota \le w + 1/k$ for all $k > 0$ into $\nu_\iota \le w$. Exponentiating after multiplying by $\log(q_s)/2 > 0$ gives $|\iota(\alpha)| \le q_s^{w/2}$. □

Definition 7.4.138 (Local $R^1 j_*$ candidate). Uses: def:weil-mon-local-weil-representation-data

Lean: Weil.Weight.MON.localR1PushforwardCandidate

Define the local $R^1 j_*$-candidate at s to be the proposed continuous group cohomology space $H^1(I_s, V)$, equipped with the action induced by the selected geometric Frobenius lift.

Definition 7.4.139 (Local-duality pairing candidate). Uses: def:weil-mon-boundary-invariant-candidate,def:weil-mon-local-r1-pushforward-candidate,def:weil-tate-line,def:weil-lisse-heart-operations-typed-context

Lean: Weil.Weight.MON.localDualityPairingCandidate

Define the proposed local-duality pairing

$$(j_*\mathcal{F})_s \otimes (R^1 j_* \mathcal{F}^{\vee_{\mathrm{lis}}})_s \longrightarrow E(-1).$$

The target is $E(-1)$, on which degree-e_s geometric Frobenius acts by q_s. This is the exact twist in Laumon Lemma (4.2.2.1), printed p. 205 (PDF p. 76).

Definition 7.4.140 (Local-duality perfectness certificate). Uses: def:weil-mon-local-duality-pairing-candidate

Lean: Weil.Weight.MON.LocalDualityPerfectnessCertificate

A local-duality perfectness certificate supplies the two adjoint maps, specified inverses, all four inverse equations, and the Frobenius equivariance equation with multiplier q_s.

Open obligation: OBL-WGT-MON-LOCAL-DUALITY

Definition 7.4.141 (Local-duality eigenvalue pairing certificate). Uses: def:weil-mon-local-duality-perfectness-certificate,def:weil-weight-finite-spectrum-data,def:weil-lisse-heart-operations-typed-context

Lean: Weil.Weight.MON.LocalDualityEigenvaluePairingCertificate

An eigenvalue pairing certificate is a multiplicity-preserving bijection between the spectrum on $R^1 j_*\mathcal{F}$ and the inverse spectrum on $(j_*\mathcal{F}^{\vee_{\mathrm{lis}}})_s$, with paired scalars α, γ satisfying

$$\alpha\gamma = q_s.$$

Proposition 7.4.142 (The exact $R^1 j_*$ lower bound). Uses: def:weil-mon-local-duality-eigenvalue-pairing-certificate, def:weil-weight-spectrum-at-most-predicate, def:weil-weight-spectrum-at-least-predicate, def:weil-lisse-heart-operations-typed-context

Lean: Weil.Weight.MON.localR1PushforwardAtLeast

If $(j_*\mathcal{F}^{\vee_{\mathrm{lis}}})_s$ has weights at most $-w$, then $(R^1 j_*\mathcal{F})_s$ has weights at least $w + 2$.

Proof. Uses: def:weil-mon-local-duality-eigenvalue-pairing-certificate,def:weil-weight-spectrum-at-most-predicate,def:weil-weight-spectrum-at-least-predicate

For paired eigenvalues $\alpha\gamma = q_s$, the input upper bound gives $|\iota(\gamma)| \le q_s^{-w/2}$. Since $\gamma \neq 0$,

$$|\iota(\alpha)| = q_s|\iota(\gamma)|^{-1} \ge q_s q_s^{w/2} = q_s^{(w+2)/2}.$$

This is the lower-bound predicate in weight $w+2$. □

Definition 7.4.143 (Length-two monodromy quotient candidate). Uses: def:weil-mon-boundary-invariant-candidate,def:weil-mon-local-weight-filtration-candidate

Lean: Weil.Weight.MON.lengthTwoQuotientCandidate

When inertia is unipotent of level two, define the quotient candidate

$$Q_2(V) := V/V^{I_s}$$

together with its proposed identification with the unique nonzero off-central monodromy graded piece. The quotient map and the graded comparison map are separately specified.

Definition 7.4.144 (Length-two graded comparison certificate). Uses: def:weil-mon-length-two-quotient-candidate,def:weil-mon-filtration-certificate

Lean: Weil.Weight.MON.LengthTwoGradedComparisonCertificate

A length-two comparison certificate supplies a proposed inverse to the graded comparison, both inverse equations, and Frobenius equivariance. It is the specialization of Deligne (1.8.4) used by Laumon Lemma (4.2.2.2), printed p. 205 (PDF p. 76).

Open obligation: OBL-WGT-MON-LENGTH-TWO

Proposition 7.4.145 (The exact length-two quotient weight). *Uses:* def:weil-mon-length-two-graded-comparison-certificate, def:weil-mon-graded-purity-predicate, def:weil-weight-pure-spectrum-predicate

Lean: Weil.Weight.MON.lengthTwoQuotient_isIotaPure

For a pure local representation of weight w, the certified length-two quotient $Q_2(V)$ is pure of weight $w+1$.

Proof. Uses: def:weil-mon-length-two-graded-comparison-certificate,def:weil-mon-graded-purity-predicate

The comparison certificate identifies the quotient spectrum, with multiplicity and Frobenius action, with the degree-1 monodromy graded piece. The graded-purity predicate assigns weight $w+1$ to that piece. Transport each modulus equality through the certified bijection. □

Definition 7.4.146 (Boundary primitive reciprocal-pairing certificate). Uses: def:weil-mon-primitive-decomposition-certificate,def:weil-mon-graded-purity-predicate,def:weil-pur-paired-real-sheaf-candidate

Lean: Weil.Weight.MON.BoundaryPrimitivePairingCertificate

For each primitive eigenvalue α of index j, a boundary primitive-pairing certificate supplies the matching eigenvalue

$$q_s^{w-j}/\alpha$$

on the paired sheaf, its multiplicity, and the coefficientwise conjugation identity. This expands Laumon Lemma (4.2.2.3), printed pp. 205–206 (PDF pp. 76–77), including Deligne (1.6.14).

Open obligation: OBL-WGT-MON-BOUNDARY-REALITY

Lemma 7.4.147 (Boundary characteristic factors are real). *Uses:* def:weil-mon-boundary-primitive-pairing-certificate, def:weil-pur-iota-real-local-factor-predicate

Lean: Weil.Weight.MON.BoundaryPrimitivePairingCertificate.isIotaReal

The local characteristic factor of the already defined paired sheaf $\mathcal{G}$ at the boundary is ι-real.

Proof. Uses: def:weil-mon-boundary-primitive-pairing-certificate,def:weil-pur-iota-real-local-factor-predicate

Partition the spectrum by primitive index. The certificate pairs every factor $1-\alpha T$ with the factor whose ι-image is its complex conjugate after the $(-w)$-twist. Multiplying each pair gives a conjugation-fixed polynomial. The finite product over primitive indices is also fixed by conjugation. □

7.5 Artin–Schreier and global Fourier–Deligne constructions

Definition 7.5.1 (Additive-character datum). Uses: `def:weil-prime-power-data`, `def:weil-coefficient-data`

Lean: `Weil.Weight.Fourier.AdditiveCharacterData`

An additive-character datum is a nontrivial homomorphism

$$\psi : (\mathbf{F}_p, +) \longrightarrow \overline{E}^{\times}$$

whose image has order p.

Definition 7.5.2 (Lang covering candidate). Uses: `def:weil-fourier-additive-character-data`

Lean: `Weil.Weight.Fourier.LangCoveringCandidate`

Define the Lang-map candidate on $\mathbf{A}^1$ by

$$L(t) = t^p - t.$$

The source, target, translation action of $\mathbf{F}_p$, and quotient map are stored as separate data.

Definition 7.5.3 (Lang torsor predicate). Uses: `def:weil-fourier-lang-covering-candidate`

Lean: `Weil.Weight.Fourier.IsLangTorsor`

Define the Lang torsor predicate by requiring finite etaleness, freeness and transitivity of the $\mathbf{F}_p$-action on every geometric fiber, and the identity between the categorical quotient and the target.

Definition 7.5.4 (Artin–Schreier idempotent candidate). Uses: `def:weil-fourier-additive-character-data`, `def:weil-fourier-lang-covering-candidate`

Lean: `Weil.Weight.Fourier.artinSchreierIdempotentCandidate`

Define the proposed character projector

$$e_\psi := \frac{1}{p} \sum_{a \in \mathbf{F}_p} \psi(a)^{-1} [a]$$

on the finite pushforward of the constant sheaf along the Lang map.

Definition 7.5.5 (Artin–Schreier sheaf candidate). Uses: `def:weil-fourier-artin-schreier-idempotent-candidate`, `def:weil-weight-geometric-operations-typed-context`

Lean: `Weil.Weight.Fourier.artinSchreierSheafCandidate`

Define $\mathcal{L}_\psi$ to be the proposed image of the idempotent e_ψ, equipped with its inclusion and projection candidates.

Definition 7.5.6 (Artin–Schreier image certificate). Uses: `def:weil-fourier-lang-torsor-predicate`, `def:weil-fourier-artin-schreier-sheaf-candidate`

Lean: `Weil.Weight.Fourier.ArtinSchreierImageCertificate`

An image certificate supplies idempotence of e_ψ, an object realizing its image, inclusion and projection maps, and both splitting equations.

Open obligation: `OBL-WGT-AS-IMAGE`

Definition 7.5.7 (Artin–Schreier rigidification candidate). Uses: `def:weil-fourier-artin-schreier-sheaf-candidate`

Lean: `Weil.Weight.Fourier.ArtinSchreierRigidificationCandidate`

A rigidification candidate is a specified isomorphism between the stalk of $\mathcal{L}_\psi$ at 0 and the constant rank-one coefficient object, together with a proposed inverse.

Definition 7.5.8 (Artin–Schreier additivity comparison candidate). Uses: `def:weil-fourier-artin-schreier-sheaf-candidate`

Lean: `Weil.Weight.Fourier.ArtinSchreierAdditivityCandidate`

For addition $m : \mathbf{A}^1 \times \mathbf{A}^1 \to \mathbf{A}^1$, define the comparison candidate

$$m^* \mathcal{L}_\psi \longrightarrow \mathrm{pr}_1^* \mathcal{L}_\psi \otimes \mathrm{pr}_2^* \mathcal{L}_\psi$$

and a proposed inverse.

Definition 7.5.9 (Artin–Schreier additivity certificate). Uses: `def:weil-fourier-artin-schreier-rigidification-candidate`, `def:weil-fourier-artin-schreier-additivity-candidate`

Lean: `Weil.Weight.Fourier.ArtinSchreierAdditivityCertificate`

An additivity certificate supplies the two inverse equations, compatibility with the rigidification, associativity on $(\mathbf{A}^1)^3$, symmetry, and the unit equation at 0.

Open obligation: `OBL-WGT-AS-ADDITIVITY`

Definition 7.5.10 (Artin–Schreier trace-function predicate). Uses: `def:weil-fourier-artin-schreier-sheaf-candidate`

Lean: `Weil.Weight.Fourier.IsArtinSchreierTraceFunction`

Define trace compatibility by

$$\mathrm{Tr}(\mathrm{Frob}_x \mid (\mathcal{L}_\psi)_{\bar{x}}) = \psi\big(\mathrm{Tr}_{k(x)/\mathbf{F}_p}(x)\big)$$

for every closed point x.

Definition 7.5.11 (Relative Artin–Schreier orthogonality candidate). Uses: `def:weil-fourier-artin-schreier-sheaf-candidate`

Lean: `Weil.Weight.Fourier.ArtinSchreierOrthogonalityCandidate`

For a line bundle $E \to S$ and a section $\lambda \in E^\vee(S)$, define

$$R\pi_!\mathcal{L}_\psi(\langle e, \lambda\rangle)$$

and the zero-locus inclusion candidate $i_\lambda : Z(\lambda) \hookrightarrow S$.

Definition 7.5.12 (Relative orthogonality certificate). Uses: `def:weil-fourier-artin-schreier-orthogonality-candidate`

Lean: `Weil.Weight.Fourier.ArtinSchreierOrthogonalityCertificate`

An orthogonality certificate supplies the comparison with the zero-locus-supported constant object, its inverse, both inverse equations, the exact shift and twist, base-change compatibility, and vanishing over the complement of $Z(\lambda)$. This expands Laumon (1.1.3.1)–(1.1.3.4), printed pp. 139–140 (PDF pp. 10–11).

Open obligation: `OBL-WGT-AS-ORTHOGONALITY`

Definition 7.5.13 (Rank-one Fourier geometry datum). Uses: `def:weil-fourier-artin-schreier-sheaf-candidate`

Lean: `Weil.Weight.Fourier.RankOneGeometryData`

Let A be a one-dimensional vector space over k_0 and A' its dual. Define the projections

$$p : A \times A' \to A, \qquad p' : A \times A' \to A',$$

the evaluation map $\langle\ ,\ \rangle : A \times A' \to \mathbf{A}^1$, and the Fourier kernel $\mathcal{K}_\psi := \langle\ ,\ \rangle^*\mathcal{L}_\psi$.

Definition 7.5.14 (Global Fourier–Deligne transform candidate). Uses: `def:weil-six-functor-typed-context`, `def:weil-fourier-rank-one-geometry-data`

Lean: `Weil.Weight.Fourier.globalTransformCandidate`

For a constructible complex K on A, define the candidate

$$\mathrm{FT}_\psi(K) := Rp'_!\big(p^*K \otimes \mathcal{K}_\psi\big)[1].$$

The shift is exactly [1], the rank-one specialization of Laumon Definition (1.2.1.1), printed p. 141 (PDF p. 12).

Definition 7.5.15 (Fourier trace-function comparison candidate). Uses: `def:weil-fourier-global-transform-candidate`, `def:weil-fourier-artin-schreier-trace-predicate`

Lean: `Weil.Weight.Fourier.TraceFunctionComparisonCandidate`

A trace-function comparison candidate is the proposed equality between the trace function of $\mathrm{FT}_\psi(K)$ and the finite Fourier transform of the trace function of K, including the rank-one sign (-1) contributed by the shift [1].

Definition 7.5.16 (Fourier trace-function certificate). Uses: `def:weil-fourier-trace-function-comparison-candidate`, `def:weil-pur-trace-l-function-certificate`

Lean: `Weil.Weight.Fourier.TraceFunctionCertificate`

A trace-function certificate supplies the trace formula for the projection, the pointwise kernel trace calculation, interchange of the finite sums, and the final sign equation. It expands Laumon Theorem (1.2.1.2), printed p. 141 (PDF p. 12).

Open obligation: `OBL-WGT-FOURIER-TRACE`

Definition 7.5.17 (Double-Fourier convolution kernel candidate)**.** Uses: `def:weil-fourier-global-transform-candidate`

Lean: `Weil.Weight.Fourier.doubleKernelCandidate`

On $A \times A''$, define the double-kernel candidate by integrating over A' the tensor product of the two evaluation kernels. The addition isomorphism rewrites the phase as $\langle a + a'', a' \rangle$.

Definition 7.5.18 (Negative diagonal candidate)**.** Uses: `def:weil-fourier-rank-one-geometry-data`

Lean: `Weil.Weight.Fourier.negativeDiagonalCandidate`

Define

$$\Delta_- : A \hookrightarrow A \times A'', \qquad a \longmapsto (a, -a),$$

and its projection to each factor.

Definition 7.5.19 (Delta-kernel comparison candidate)**.** Uses: `def:weil-fourier-double-kernel-candidate,def:weil-fourier-negative-diagonal-candidate`

Lean: `Weil.Weight.Fourier.DeltaKernelComparisonCandidate`

Define the proposed comparison from the double convolution kernel to $\Delta_{-,!}E(-1)[-2]$, and define a proposed inverse.

Definition 7.5.20 (Delta-kernel certificate)**.** Uses: `def:weil-fourier-artin-schreier-orthogonality-certificate,def:weil-fourier-delta-kernel-comparison-candidate`

Lean: `Weil.Weight.Fourier.DeltaKernelCertificate`

A delta-kernel certificate supplies the two inverse equations, the off-negative-diagonal vanishing, the fiber calculation on the negative diagonal, proper-base-change compatibility, and the exact twist (-1) and shift $[-2]$. It expands Laumon Proposition (1.2.2.2), printed pp. 142–143 (PDF pp. 13–14).

Open obligation: `OBL-WGT-FOURIER-DELTA`

Definition 7.5.21 (Fourier inversion comparison candidate)**.** Uses: `def:weil-fourier-global-transform-candidate,def:weil-fourier-negative-diagonal-candidate`

Lean: `Weil.Weight.Fourier.InversionComparisonCandidate`

For every K, define the proposed comparison

$$\mathrm{FT}_\psi \mathrm{FT}_\psi(K) \longrightarrow [-1]^* K(-1)$$

and a proposed inverse. No residual cohomological shift occurs after the two rank-one shifts $[1]$ are combined with the delta-kernel shift $[-2]$.

Definition 7.5.22 (Fourier inversion certificate)**.** Uses: `def:weil-fourier-delta-kernel-certificate,def:weil-fourier-inversion-comparison-candidate`

Lean: `Weil.Weight.Fourier.InversionCertificate`

An inversion certificate supplies both inverse equations, naturality in K, compatibility with shifts and triangles, and base-change compatibility. This is the atomized rank-one form of Laumon Theorem (1.2.2.1), printed pp. 141–142 (PDF pp. 12–13).

Open obligation: `OBL-WGT-FOURIER-INVERSION`

Proposition 7.5.23 (Conditional Fourier inversion)**.** *Uses:* *`def:weil-fourier-inversion-certificate`*

Lean: *`Weil.Weight.Fourier.inversion`*

An inversion certificate yields a natural isomorphism

$$\mathrm{FT}_\psi \mathrm{FT}_\psi(K) \simeq [-1]^* K(-1).$$

Proof. Uses: `def:weil-fourier-inversion-comparison-candidate,def:weil-fourier-inversion-certificate`

Use the forward comparison as the isomorphism and its specified inverse as the inverse morphism. The two inverse fields of the certificate are exactly the two isomorphism axioms. □

Definition 7.5.24 (Forget-supports Fourier comparison candidate)**.** Uses: `def:weil-six-functor-typed-context,def:weil-six-functor-comparison-map-candidates,def:weil-fourier-global-transform-candidate`

Lean: `Weil.Weight.Fourier.ForgetSupportsCandidate`

Define the comparison candidate to be the (p')-component of the supplied typed forget-supports transformation,

$$Rp'_!\big(p^* K \otimes \mathcal{K}_\psi\big) \longrightarrow Rp'_*\big(p^* K \otimes \mathcal{K}_\psi\big).$$

Definition 7.5.25 (Universal strong-acyclicity candidate)**.** Uses: `def:weil-fourier-forget-supports-candidate,def:weil-weight-geometric-operations-typed-context`

Lean: `Weil.Weight.Fourier.UniversalAcyclicityCandidate`

After compactifying A to $\mathbf{P}^1$, define the boundary cone of the forget-supports comparison and its pullback along every base change $S' \to S$.

Definition 7.5.26 (Fourier universal-acyclicity certificate)**.** Uses: `def:weil-fourier-universal-acyclicity-candidate,def:weil-weight-geometric-operations-realization-certificate`

Lean: `Weil.Weight.Fourier.UniversalAcyclicityCertificate`

A universal-acyclicity certificate supplies vanishing of every pulled-back boundary cone and compatibility of these vanishings with iterated base change. It expands the Katz–Laumon input quoted in Laumon Theorems (1.3.1.1)–(1.3.1.2), printed pp. 146–147 (PDF pp. 17–18).

Open obligation: `OBL-WGT-FOURIER-ULA`

Lemma 7.5.27 (Forget supports is invertible under universal acyclicity)**.** *Uses:* *`def:weil-fourier-forget-supports-candidate,def:weil-fourier-universal-acyclicity-certificate`*

Lean: *`Weil.Weight.Fourier.forgetSupports_isIso`*

The forget-supports comparison has zero cone and hence is invertible.

Proof. Uses: `def:weil-fourier-universal-acyclicity-candidate,def:weil-fourier-universal-acyclicity-certificate,def:weil-weight-geometric-operations-realization-certificate`

Apply the certificate to the identity base change. Its vanishing field says that the cone of the comparison is zero. In a triangulated category, a morphism with zero cone is an isomorphism; use the specified zero-object identification to construct its inverse. □

Definition 7.5.28 (Perverse-heart candidate on the affine line)**.** Uses: `def:weil-fourier-rank-one-geometry-data,def:weil-weight-geometric-operations-typed-context`

Lean: `Weil.Weight.Fourier.PerverseHeartCandidate`

Define a candidate full subcategory by the support and cosupport inequalities

$$\dim \operatorname{Supp} \mathcal{H}^{-i}(K) \le i, \qquad \dim \operatorname{Supp} \mathcal{H}^{-i}(DK) \le i.$$

Truncation functors and heart axioms are separate candidates.

Definition 7.5.29 (Perverse truncation candidate)**.** Uses: `def:weil-fourier-perverse-heart-candidate,def:weil-weight-geometric-operations-typed-context`

Lean: `Weil.Weight.Fourier.PerverseTruncationCandidate`

A perverse truncation candidate consists of functors ${}^{p}\tau_{\le 0}$, ${}^{p}\tau_{\ge 0}$, their natural maps, and proposed distinguished triangles for every constructible complex.

Definition 7.5.30 (Perverse t-structure certificate)**.** Uses: `def:weil-fourier-perverse-heart-candidate,def:weil-fourier-perverse-truncation-candidate,def:weil-weight-geometric-operations-realization-certificate`

Lean: `Weil.Weight.Fourier.PerverseTStructureCertificate`

A perverse t-structure certificate supplies orthogonality, the truncation triangles, shift closure, heart identification, recollement for open–closed decompositions, and Artin–Gabber affine exactness.

Open obligation: `OBL-WGT-PERVERSE-T-STRUCTURE`

Definition 7.5.31 (Fourier perversity certificate)**.** Uses: `def:weil-fourier-global-transform-candidate,def:weil-fourier-inversion-certificate,def:weil-fourier-universal-acyclicity-certificate,def:weil-fourier-perverse-t-structure-certificate`

Lean: `Weil.Weight.Fourier.PerversityCertificate`

A Fourier perversity certificate supplies the support and cosupport inequalities for the transform of every perverse object and for its inverse, and exactness on every short exact sequence in the perverse heart. It expands Laumon Theorem (1.3.2.3), printed p. 147 (PDF p. 18).

Open obligation: `OBL-WGT-FOURIER-PERVERSE`

Definition 7.5.32 (Simple perverse transform-shape candidate). Uses: `def:weil-fourier-global-transform-candidate,def:weil-fourier-perverse-heart-candidate,def:weil-lisse-heart-operations-typed-context`

Lean: `Weil.Weight.Fourier.SimpleTransformShapeCandidate`

For a simple middle extension $j_*\mathcal{F}[1]$, a transform-shape candidate consists of a dense open $j' : U' \hookrightarrow A'$, an irreducible lisse $\mathcal{F}'$ on U', and comparison maps

$$\mathrm{FT}_\psi(j_*\mathcal{F}[1]) \rightleftarrows j'_*\mathcal{F}'[1].$$

The punctual and Artin–Schreier exceptional cases are recorded as separate disjoint alternatives.

Definition 7.5.33 (Simple transform-shape certificate). Uses: `def:weil-fourier-perversity-certificate,def:weil-fourier-simple-transform-shape-candidate`

Lean: `Weil.Weight.Fourier.SimpleTransformShapeCertificate`

A shape certificate supplies both inverse equations, simplicity and lissity of $\mathcal{F}'$, exclusion of the punctual and Artin–Schreier alternatives by explicit support and ramification predicates, and uniqueness up to the specified isomorphism. It expands Laumon Theorem (1.4.2.1), printed p. 149 (PDF p. 20).

Open obligation: `OBL-WGT-FOURIER-SHAPE`

7.6 Local Fourier transform and stationary phase

Definition 7.6.1 (Henselian local Fourier coordinate datum). Uses: `def:weil-fourier-rank-one-geometry-data`

Lean: `Weil.Weight.LFT.HenselianCoordinateData`

A local coordinate datum consists of henselian traits at $0, \infty, 0', \infty'$, selected uniformizers $x, 1/x, x', 1/x'$, geometric generic points, inertia groups, and specialization maps.

Definition 7.6.2 (Break-decomposition candidate). Uses: `def:weil-lft-henselian-coordinate-data,def:weil-mon-local-weil-representation-data`

Lean: `Weil.Weight.LFT.BreakDecompositionCandidate`

For a local representation V, a break candidate is a finite direct-sum decomposition $V = \bigoplus_{\lambda \in \mathbf{Q}_{\geq 0}} V(\lambda)$, with inclusion and projection maps and a proposed slope λ on each summand.

Definition 7.6.3 (Break-decomposition certificate). Uses: `def:weil-lft-break-decomposition-candidate`

Lean: `Weil.Weight.LFT.BreakDecompositionCertificate`

A break certificate supplies both splitting equations, inertia stability, the ramification-subgroup characterization of each slope, rationality of all slopes, and functoriality. It expands Laumon Theorem (2.1.2.6), printed p. 153 (PDF p. 24).

Open obligation: `OBL-WGT-LFT-BREAKS`

Definition 7.6.4 (Swan conductor candidate). Uses: `def:weil-lft-break-decomposition-candidate`

Lean: `Weil.Weight.LFT.swanConductorCandidate`

Define

$$\mathrm{Swan}(V) := \sum_{\lambda} \lambda \, \dim V(\lambda).$$

Integrality is a separate predicate.

Definition 7.6.5 (GOS formula certificate). Uses: `def:weil-lft-swan-conductor-candidate`

Lean: `Weil.Weight.LFT.GOSCertificate`

A Grothendieck–Ogg–Shafarevich certificate supplies the Euler characteristic formula with rank, tame defect, and Swan terms, including compact supports and Frobenius equivariance. It expands Laumon Theorem (2.2.1.2), printed p. 156 (PDF p. 27).

Open obligation: `OBL-WGT-LFT-GOS`

Definition 7.6.6 (Gabber–Katz extension candidate). Uses: `def:weil-lft-henselian-coordinate-data,def:weil-mon-local-weil-representation-data,def:weil-lisse-heart-operations-typed-context`

Lean: `Weil.Weight.LFT.GabberKatzExtensionCandidate`

For a local representation at 0, a Gabber–Katz candidate consists of a lisse sheaf on $\mathbf{G}_m$, a local comparison at 0, a proposed tame structure at ∞, and an extension to the affine line.

Definition 7.6.7 (Gabber–Katz extension certificate)**.** Uses: `def:weil-lft-gabber-katz-extension-candidate`

Lean: `Weil.Weight.LFT.GabberKatzExtensionCertificate`

An extension certificate supplies both local-comparison inverse equations, tameness at infinity, prescribed wild inertia at zero, functoriality, uniqueness, and independence of the global coordinate. It expands Laumon Theorem (2.2.2.2), printed p. 157 (PDF p. 28).

Open obligation: `OBL-WGT-LFT-GABBER-KATZ`

Definition 7.6.8 (Nearby-cycles candidate)**.** Uses: `def:weil-lft-henselian-coordinate-data`

Lean: `Weil.Weight.LFT.nearbyCyclesCandidate`

For a map to a henselian trait, define a proposed nearby-cycles object $R\Psi(K)$, its inertia action, geometric Frobenius action, and specialization map from the special-fiber pullback.

Definition 7.6.9 (Vanishing-cycles candidate)**.** Uses: `def:weil-lft-nearby-cycles-candidate,def:weil-weight-geometric-operations-typed-context`

Lean: `Weil.Weight.LFT.vanishingCyclesCandidate`

Define $R\Phi(K)$ as a proposed cone of the specialization map, together with its distinguished-triangle maps and inherited inertia and Frobenius actions.

Definition 7.6.10 (Vanishing-cycles triangle certificate)**.** Uses: `def:weil-lft-vanishing-cycles-candidate,def:weil-weight-geometric-operations-realization-certificate`

Lean: `Weil.Weight.LFT.VanishingCyclesTriangleCertificate`

A triangle certificate supplies the distinguished-triangle predicate, construction independence, base-change maps and their inverses, inertia and Frobenius equivariance, and compatibility with Verdier duality.

Open obligation: `OBL-WGT-LFT-VANISHING-CYCLES`

Definition 7.6.11 (Three local kernel candidates)**.** Uses: `def:weil-lft-henselian-coordinate-data,def:weil-fourier-artin-schreier-sheaf-candidate`

Lean: `Weil.Weight.LFT.ThreeKernelCandidates`

Define the three pulled-back local kernels with phases

$$xx', \qquad x/x', \qquad 1/(xx')$$

for the coordinate pairs $(0,\infty')$, $(\infty,0')$, and (∞,∞'), including Laumon's shifts inside the corresponding vanishing-cycle complexes.

Definition 7.6.12 (Global-to-local kernel comparison candidate)**.** Uses: `def:weil-fourier-global-transform-candidate,def:weil-lft-three-kernel-candidates,def:weil-lft-vanishing-cycles-candidate`

Lean: `Weil.Weight.LFT.GlobalLocalComparisonCandidate`

For each of the three coordinate pairs, define the proposed comparison from the restriction of the global Fourier vanishing-cycle complex to the corresponding local kernel complex, and define a proposed inverse.

Definition 7.6.13 (Global-to-local comparison certificate)**.** Uses: `def:weil-lft-global-local-comparison-candidate,def:weil-lft-vanishing-cycles-triangle-certificate`

Lean: `Weil.Weight.LFT.GlobalLocalComparisonCertificate`

A comparison certificate supplies both inverse equations in all three coordinates, compatibility with inertia and Frobenius, base change, coordinate changes, and the selected Artin–Schreier rigidification. It expands Laumon Lemma (2.4.2.1), printed p. 162 (PDF p. 33).

Open obligation: `OBL-WGT-LFT-GLOBAL-LOCAL`

Definition 7.6.14 (Local concentration predicate)**.** Uses: `def:weil-lft-three-kernel-candidates,def:weil-lft-vanishing-cycles-candidate`

Lean: `Weil.Weight.LFT.IsConcentratedInDegreeOne`

Define concentration by requiring support at the closed point, vanishing of all cohomology objects except degree 1, and constructibility of that degree-one object.

Definition 7.6.15 (Local concentration certificate)**.** Uses: `def:weil-lft-global-local-comparison-certificate,def:weil-lft-concentration-predicate,def:weil-lft-gabber-katz-extension-certificate`

Lean: `Weil.Weight.LFT.ConcentrationCertificate`

A concentration certificate supplies the three concentration predicates and their compatibility with the global comparisons. It expands Laumon Proposition (2.4.2.2), printed p. 163 (PDF p. 34).

Open obligation: `OBL-WGT-LFT-CONCENTRATION`

Definition 7.6.16 (Three local Fourier functors). Uses: `def:weil-lft-concentration-certificate`

Lean: `Weil.Weight.LFT.threeLocalFourierFunctors`

Define

$$\mathrm{FT}^{(0,\infty')}(V), \quad \mathrm{FT}^{(\infty,0')}(V), \quad \mathrm{FT}^{(\infty,\infty')}(V)$$

to be the degree-one cohomology representations of the three concentrated vanishing-cycle complexes. This is Laumon Definition (2.4.2.3), printed p. 163 (PDF p. 34).

Definition 7.6.17 (Local Fourier exactness certificate). Uses: `def:weil-lft-three-local-fourier-functors,def:weil-weight-geometric-operations-typed-context,def:weil-weight-geometric-operations-realization-certificate`

Lean: `Weil.Weight.LFT.ExactnessCertificate`

An exactness certificate supplies, for each local functor, the image, kernel, and cokernel comparisons for every short exact sequence, plus all naturality and Frobenius-equivariance squares.

Open obligation: `OBL-WGT-LFT-EXACTNESS`

Definition 7.6.18 (Local Fourier rank-and-slope certificate). Uses: `def:weil-lft-three-local-fourier-functors,def:weil-lft-break-decomposition-certificate,def:weil-lft-gos-certificate`

Lean: `Weil.Weight.LFT.RankSlopeCertificate`

A rank-and-slope certificate supplies Laumon's conductor/rank formulas, the allowed source and target slope intervals, lissity, and the multiplicity-preserving break transformations from Theorem (2.4.3), printed pp. 163–164 (PDF pp. 34–35).

Open obligation: `OBL-WGT-LFT-RANK-SLOPE`

Definition 7.6.19 (Local Fourier inverse comparison candidate). Uses: `def:weil-lft-three-local-fourier-functors`

Lean: `Weil.Weight.LFT.InverseComparisonCandidate`

For each inverse pair of local Fourier functors, define the proposed comparison of their composite with the original representation, including the sign pullback and Tate twist (1) or (-1) dictated by the chosen direction, and define a proposed inverse.

Definition 7.6.20 (Local Fourier inversion certificate). Uses: `def:weil-lft-inverse-comparison-candidate,def:weil-fourier-inversion-certificate`

Lean: `Weil.Weight.LFT.InversionCertificate`

A local inversion certificate supplies both inverse equations, coordinate independence, and compatibility with the global Fourier inversion $\mathrm{FT}^2 = [-1]^*(-1)$. It expands Laumon (2.5.6), printed p. 169 (PDF p. 40).

Open obligation: `OBL-WGT-LFT-INVERSION`

Definition 7.6.21 (Local Fourier duality comparison candidate). Uses: `def:weil-lft-three-local-fourier-functors`

Lean: `Weil.Weight.LFT.DualityComparisonCandidate`

Define the proposed comparison between the dual of each local Fourier transform and the appropriate transform of the dual representation, including the precise sign pullback and Tate twist.

Definition 7.6.22 (Local Fourier duality certificate). Uses: `def:weil-lft-duality-comparison-candidate,def:weil-mon-local-duality-perfectness-certificate`

Lean: `Weil.Weight.LFT.DualityCertificate`

A duality certificate supplies inverse maps, both inverse equations, evaluation-pairing compatibility, Frobenius equivariance, and independence of local coordinates. It fills the argument left unstated in Laumon (2.5.7), printed p. 169 (PDF p. 40).

Open obligation: `OBL-WGT-LFT-DUALITY`

Definition 7.6.23 (Stationary-phase input datum). Uses: `def:weil-fourier-simple-transform-shape-candidate,def:weil-lft-three-local-fourier-functors,def:weil-lisse-heart-operations-typed-context`

Lean: `Weil.Weight.LFT.StationaryPhaseInputData`

For irreducible lisse $\mathcal{F}$ on $U \subset \mathbf{A}^1$, nonconstant and unramified at infinity, a stationary-phase input consists of the transform $j'_*\mathcal{F}'[1]$, its local representation at $0'$, the space $H^1_c(\mathbf{A}^1_{\bar{k}}, j_*\mathcal{F})$, and the infinity stalk $\mathcal{F}_\infty(-1)$.

Definition 7.6.24 (Stationary-phase inclusion candidate)**.** Uses: `def:weil-lft-stationary-phase-input-data`

Lean: `Weil.Weight.LFT.stationaryPhaseInclusionCandidate`

Define the proposed map

$$a : H^1_c(\mathbf{A}^1_{\bar{k}}, j_*\mathcal{F}) \longrightarrow \mathcal{F}'_{\eta_{0'}}$$

from the global-to-local specialization and degree-one vanishing-cycle comparison.

Definition 7.6.25 (Stationary-phase quotient candidate)**.** Uses: `def:weil-lft-stationary-phase-input-data`

Lean: `Weil.Weight.LFT.stationaryPhaseQuotientCandidate`

Define the proposed map

$$b : \mathcal{F}'_{\eta_{0'}} \longrightarrow \mathcal{F}_\infty(-1)$$

from the local $(\infty, 0')$ Fourier comparison.

Definition 7.6.26 (Stationary-phase zero-composite predicate)**.** Uses: `def:weil-lft-stationary-phase-inclusion-candidate,def:weil-lft-stationary-phase-quotient-candidate`

Lean: `Weil.Weight.LFT.IsStationaryPhaseZeroComposite`

Define the predicate by $b \circ a = 0$.

Definition 7.6.27 (Stationary-phase injectivity predicate)**.** Uses: `def:weil-lft-stationary-phase-inclusion-candidate`

Lean: `Weil.Weight.LFT.IsStationaryPhaseInjective`

Define injectivity by $\ker(a) = 0$.

Definition 7.6.28 (Stationary-phase surjectivity predicate)**.** Uses: `def:weil-lft-stationary-phase-quotient-candidate`

Lean: `Weil.Weight.LFT.IsStationaryPhaseSurjective`

Define surjectivity by $\mathrm{range}(b) = \mathcal{F}_\infty(-1)$.

Definition 7.6.29 (Stationary-phase middle-exactness predicate)**.** Uses: `def:weil-lft-stationary-phase-inclusion-candidate,def:weil-lft-stationary-phase-quotient-candidate`

Lean: `Weil.Weight.LFT.IsStationaryPhaseMiddleExact`

Define middle exactness by $\mathrm{range}(a) = \ker(b)$.

Definition 7.6.30 (Stationary-phase inertia predicate)**.** Uses: `def:weil-lft-stationary-phase-input-data,def:weil-lft-stationary-phase-inclusion-candidate,def:weil-lft-stationary-phase-quotient-candidate`

Lean: `Weil.Weight.LFT.IsStationaryPhaseInertiaCompatible`

Define inertia compatibility by requiring a and b to be $I_{0'}$-equivariant and requiring inertia to act trivially on the outer terms.

Definition 7.6.31 (Stationary-phase Frobenius predicate)**.** Uses: `def:weil-lft-stationary-phase-input-data,def:weil-lft-stationary-phase-inclusion-candidate,def:weil-lft-stationary-phase-quotient-candidate`

Lean: `Weil.Weight.LFT.IsStationaryPhaseFrobeniusCompatible`

Define Frobenius compatibility by the two commuting squares for geometric Frobenius. On the quotient term, the operator is $q\,F_{\mathcal{F}_\infty}$, because the term is $\mathcal{F}_\infty(-1)$.

Definition 7.6.32 (Stationary-phase invariant-identification candidate)**.** Uses: `def:weil-lft-stationary-phase-input-data,def:weil-lft-stationary-phase-inclusion-candidate`

Lean: `Weil.Weight.LFT.StationaryPhaseInvariantComparisonCandidate`

Define the proposed comparison

$$H^1_c(\mathbf{A}^1_{\bar{k}}, j_*\mathcal{F}) \longrightarrow (\mathcal{F}'_{\eta_{0'}})^{I_{0'}}$$

by factoring a through inertia invariants, and define a proposed inverse.

Definition 7.6.33 (Stationary-phase exact-sequence certificate). Uses: def:weil-lft-global-local-comparison-certificate, def:weil-lft-exactness-certificate, def:weil-lft-stationary-phase-zero-composite-predicate, def:weil-lft-stationary-phase-injective-predicate, def:weil-lft-stationary-phase-surjective-predicate, def:weil-lft-stationary-phase-middle-exact-predicate, def:weil-lft-stationary-phase-inertia-predicate, def:weil-lft-stationary-phase-frobenius-predicate, def:weil-lft-stationary-phase-invariant-comparison-candidate

Lean: Weil.Weight.LFT.StationaryPhaseExactSequenceCertificate

A stationary-phase certificate consists of all four exactness predicates, the inertia and Frobenius predicates, a specified inverse for the invariant comparison with both inverse equations, naturality in $\mathcal{F}$, and coordinate independence. It proves no weight statement. Its target sequence is exactly Laumon (4.4), printed p. 207 (PDF p. 78):

$$0 \to H^1_c(\mathbf{A}^1_{\bar{k}}, j_*\mathcal{F}) \xrightarrow{a} \mathcal{F}'_{\eta_{0'}} \xrightarrow{b} \mathcal{F}_\infty(-1) \to 0.$$

Open obligation: OBL-WGT-LFT-STATIONARY-PHASE

Proposition 7.6.34 (Conditional stationary-phase short exact sequence). *Uses: def:weil-lft-stationary-phase-exact-sequence-certificate*

Lean: Weil.Weight.LFT.stationaryPhase_shortExact

The displayed stationary-phase sequence is short exact, $I_{0'}$-equivariant, and geometric-Frobenius-equivariant; its invariant subspace is identified with the first term.

Proof. Uses: def:weil-lft-stationary-phase-zero-composite-predicate, def:weil-lft-stationary-phase-injective-predicate, def:weil-lft-stationary-phase-surjective-predicate, def:weil-lft-stationary-phase-middle-exact-predicate, def:weil-lft-stationary-phase-inertia-predicate, def:weil-lft-stationary-phase-frobenius-predicate, def:weil-lft-stationary-phase-invariant-comparison-candidate, def:weil-lft-stationary-phase-exact-sequence-certificate

The injectivity, middle-exactness, and surjectivity fields are precisely the three short-exactness conditions; the zero-composite field makes the maps a complex. The inertia and Frobenius fields give equivariance. The specified inverse and its two equations make the invariant comparison an isomorphism. □

7.7 Laumon's curve estimate

Definition 7.7.1 (Affine curve-estimate input datum). Uses: def:weil-weight-pointwise-pure-predicate, def:weil-fourier-simple-transform-shape-candidate, def:weil-lisse-heart-operations-typed-context

Lean: Weil.Weight.Curve.AffineEstimateInputData

An affine input consists of a dense open $j : U \hookrightarrow A = \mathbf{A}^1_{k_0}$, an irreducible lisse sheaf $\mathcal{F}$ on U, an integral weight w, pointwise purity of weight w, geometric nonconstancy, and an explicit unramified extension across infinity.

Definition 7.7.2 (No Artin–Schreier constituent predicate). Uses: def:weil-curve-affine-estimate-input-data, def:weil-fourier-artin-schreier-sheaf-candidate

Lean: Weil.Weight.Curve.HasNoArtinSchreierConstituent

Define the predicate by requiring every morphism from a translated Artin–Schreier rank-one sheaf to $\mathcal{F}$, and every morphism in the reverse direction, to be zero.

Definition 7.7.3 (Ramification exclusion certificate). Uses: def:weil-curve-no-artin-schreier-predicate, def:weil-lft-break-decomposition-certificate

Lean: Weil.Weight.Curve.RamificationExclusionCertificate

A ramification exclusion certificate supplies the break of every translated Artin–Schreier sheaf at infinity, the zero break of $\mathcal{F}$ there, and the calculation showing a nonzero morphism would preserve incompatible breaks.

Open obligation: OBL-WGT-CURVE-NO-AS

Definition 7.7.4 (Affine transform datum). Uses: def:weil-curve-affine-estimate-input-data, def:weil-fourier-simple-transform-shape-candidate

Lean: Weil.Weight.Curve.AffineTransformData

An affine transform datum specifies the simple transform-shape candidate

$$\mathrm{FT}_\psi(j_*\mathcal{F}[1]) \rightleftarrows j'_*\mathcal{F}'[1], \qquad U' = A' \setminus \{0'\},$$

including $\mathcal{F}'$, both comparison maps, and its local representation at $0'$.

Definition 7.7.5 (Paired Fourier transform candidate)**.** Uses: `def:weil-pur-paired-real-sheaf-candidate,def:weil-fourier-global-transform-candidate`

Lean: `Weil.Weight.Curve.pairedTransformCandidate`

Let $\mathcal{G} \in \mathsf{Sh}_c(U, E)$ be the carried paired lisse object already defined from the raw lisse-heart context. Define

$$\mathcal{G}' := \mathcal{H}^0 \,\mathrm{FT}_\psi\big(j_*\mathcal{G}[1]\big)\,\big|_{A'-\{0'\}}\,.$$

Its proposed inclusion and projection of $\mathcal{F}'$ are part of the candidate. No second direct sum, lisse dual, or heart Tate twist is selected inside this Definition.

Definition 7.7.6 (Closed-point product-formula candidate for the transform)**.** Uses: `def:weil-curve-paired-transform-candidate,def:weil-pur-cohomological-l-function-candidate,def:weil-t-adic-locally-finite-family,def:weil-locally-finite-product`

Lean: `Weil.Weight.Curve.TransformProductFormulaCandidate`

For every $x' \in |A'| - \{0'\}$, define the proposed formal Euler-product identity for $\det(1 - T\,\mathrm{Frob}_{x'} \mid \mathcal{G}'_{\bar{x}'})$ in terms of the local factors indexed by all closed points $x \in |A|$,

$$\det(1 - T\,\mathrm{Frob}_x \mid j_*\mathcal{G}_{\bar{x}})$$

and the Artin–Schreier trace $\psi(\mathrm{Tr}(xx'))$. The product is T-adically locally finite: equality is first a coefficientwise identity in the formal-series ring and only then is transported to the supplied rational-function realization.

Definition 7.7.7 (Transform product-formula certificate)**.** Uses: `def:weil-curve-transform-product-formula-candidate,def:weil-fourier-trace-function-certificate,def:weil-pur-trace-l-function-certificate,def:weil-pur-paired-factor-reality-certificate,def:weil-mon-boundary-primitive-pairing-certificate`

Lean: `Weil.Weight.Curve.TransformProductFormulaCertificate`

A product-formula certificate supplies equality of every coefficient, the local-finiteness certificate, comparison with the rational realization, Frobenius-equivariance, the explicit interior paired-factor reality certificate, the separate boundary-reality certificate, and coefficientwise preservation of reality under the stabilized finite truncations. In particular, interior reality is imported from the preceding pure paired construction rather than re-assumed as a field. It expands the last paragraph of Laumon (4.4), printed p. 208 (PDF p. 79).

Open obligation: `OBL-WGT-CURVE-TRANSFORM-PRODUCT`

Lemma 7.7.8 (The paired Fourier transform is ι-real)**.** *Uses:* *`def:weil-curve-paired-transform-candidate,def:weil-curve-transform-product-formula-certificate,def:weil-pur-iota-real-sheaf-predicate`*

Lean: *`Weil.Weight.Curve.pairedTransform_isIotaReal`*

The transform product-formula certificate makes $\mathcal{G}'$ ι-real on $A' - \{0'\}$.

Proof. Uses: `def:weil-pur-iota-real-local-factor-predicate,def:weil-curve-transform-product-formula-candidate,def:weil-curve-transform-product-formula-certificate,lemma:weil-locally-finite-product-subring`

At a closed point x', replace the transform factor by the certified formal Euler product. Every finite truncation has real coefficients after applying ι. The locally finite product subring lemma shows that each coefficient stabilizes inside the real subfield, so the full formal product, and hence its certified rational realization, has real coefficients. Apply this calculation to every x'. □

Definition 7.7.9 (Transform direct-summand certificate)**.** Uses: `def:weil-curve-affine-transform-data,def:weil-curve-paired-transform-candidate`

Lean: `Weil.Weight.Curve.TransformDirectSummandCertificate`

A transform direct-summand certificate supplies inclusion and projection maps $\mathcal{F}' \rightleftarrows \mathcal{G}'$, both splitting equations, lissity, irreducibility of $\mathcal{F}'$, and Frobenius equivariance.

Open obligation: `OBL-WGT-CURVE-TRANSFORM-SUMMAND`

Definition 7.7.10 (Irreducible constituent in a typed real ambient sheaf)**.** Uses: `def:weil-pur-iota-real-sheaf-predicate,def:weil-curve-transform-direct-summand-certificate,def:weil-lisse-heart-operations-typed-context`

Lean: `Weil.Weight.PUR.IrreducibleRealAmbientData`

An irreducible-real-ambient datum consists of an actual lisse Weil sheaf $\mathcal{G}$, a proof that it is ι-real, an irreducible lisse sheaf $\mathcal{L}$, inclusion and projection maps $\mathcal{L} \rightleftarrows \mathcal{G}$, the retraction equation, and all Frobenius commutative squares. Every object and map is indexed by the same base, coefficient field, and fixed complex realization.

Definition 7.7.11 (The transformed real-ambient datum). Uses: def:weil-pur-irreducible-real-ambient-data,def:weil-curve-paired-transform-candidate,def:weil-curve-transform-product-formula-certificate,def:weil-curve-transform-direct-summand-certificate

Lean: Weil.Weight.Curve.transformRealAmbientData

Define the transformed real ambient datum by taking $\mathcal{G} = \mathcal{G}'$, $\mathcal{L} = \mathcal{F}'$, the inclusion and projection stored in the direct-summand certificate, and the immediately preceding pointwise reality proof for the paired transform.

Constructed by: lemma:weil-curve-paired-transform-real

Definition 7.7.12 (PUR data for the transformed constituent). Uses: def:weil-curve-transform-real-ambient-data,def:weil-pur-complete-antecedent-certificate,def:weil-pur-determinantal-weight-data

Lean: Weil.Weight.Curve.TransformPURData

Transform PUR data consist of the explicit transformed real-ambient datum, a complete PUR antecedent certificate indexed by its constituent $\mathcal{F}'$, and a determinantal weight w'. The real number w' is stored data, not a value inferred in this definition.

Proposition 7.7.13 (Conditional purity of the transformed sheaf). *Uses: def:weil-curve-transform-pur-data,def:weil-weight-pointwise-pure-predicate*

Lean: Weil.Weight.Curve.transform_isPointwiseIotaPure

The transformed irreducible sheaf $\mathcal{F}'$ is pointwise pure of weight w'.

Proof. Uses: prop:weil-pur-irreducible-constituent-pure

Apply the conditional PUR endpoint to the complete antecedent certificate and determinantal weight stored in the transform PUR data. □

Definition 7.7.14 (Stationary-phase weight-comparison datum). Uses: def:weil-curve-affine-estimate-input-data,def:weil-curve-transform-pur-data,def:weil-lft-stationary-phase-exact-sequence-certificate,def:weil-mon-length-two-graded-comparison-certificate

Lean: Weil.Weight.Curve.StationaryWeightComparisonData

A stationary weight-comparison datum consists of the stationary-phase exact sequence, a nonzero quotient $\mathcal{F}_\infty(-1)$, the level-two local monodromy certificate for $\mathcal{F}'_{\eta_{0'}}$, and an occurrence δ in the quotient spectrum. The corresponding occurrence in the level-two graded quotient is explicitly identified.

Lemma 7.7.15 (The stationary quotient has weight $w+2$). *Uses: def:weil-curve-stationary-weight-comparison-data,def:weil-weight-pure-spectrum-predicate*

Lean: Weil.Weight.Curve.stationaryQuotient_sourceWeight

The occurrence δ in $\mathcal{F}_\infty(-1)$ is pure of weight $w+2$.

Proof. Uses: def:weil-curve-affine-estimate-input-data,lemma:weil-weight-spectrum-tate-twist-pure

The unramified extension at infinity identifies $\mathcal{F}_\infty$ with a local spectrum of the weight-w sheaf. Apply the spectrum Tate-shift lemma with $n = -1$. The weight changes from w to $w - 2(-1) = w + 2$. □

Lemma 7.7.16 (The stationary quotient has weight $w'+1$). *Uses: def:weil-curve-stationary-weight-comparison-data,def:weil-weight-pure-spectrum-predicate*

Lean: Weil.Weight.Curve.stationaryQuotient_transformWeight

The same occurrence δ, viewed through the level-two comparison, is pure of weight $w'+1$.

Proof. Uses: prop:weil-curve-transform-pure,prop:weil-mon-length-two-quotient-pure

The transform-purity proposition makes the local representation pure of weight w'. Apply the length-two quotient proposition; it assigns weight $w'+1$ to the quotient spectrum. Transport that equation through the stored occurrence identification. □

Lemma 7.7.17 (The transformed weight is $w+1$). *Uses: def:weil-curve-stationary-weight-comparison-data*

Lean: Weil.Weight.Curve.transformWeight_eq

One has

$$w' = w + 1.$$

Proof. Uses: lemma:weil-curve-stationary-quotient-source-weight,lemma:weil-curve-stationary-quotient-transform-weight,lemma:weil-weight-iota-pure-weight-unique

The nonzero occurrence δ is pure of weights $w+2$ and $w'+1$. Weight uniqueness gives $w+2 = w'+1$. Subtract one from both sides. □

Definition 7.7.18 (Transformed boundary-upper certificate). Uses: def:weil-curve-transform-pur-data,def:weil-mon-boundary-tensor-limit-certificate

Lean: Weil.Weight.Curve.TransformBoundaryUpperCertificate

A transformed boundary-upper certificate is the boundary tensor-limit certificate for $\mathcal{F}'$ at $0'$, with ambient weight w'.

Lemma 7.7.19 (Weight bound on the transformed inertia invariants). Uses: def:weil-curve-transform-boundary-upper-certificate, def:weil-weight-spectrum-at-most-predicate

Lean: Weil.Weight.Curve.transformInvariants_weightAtMost

The spectrum of $(\mathcal{F}'_{\eta_{0'}})^{I_{0'}}$ has weights at most $w+1$.

Proof. Uses: prop:weil-mon-boundary-invariant-upper,lemma:weil-curve-transform-weight-successor

Apply the boundary-invariant upper proposition in weight w', then replace w' by $w+1$. □

Definition 7.7.20 (Stationary invariant-transfer certificate). Uses: def:weil-lft-stationary-phase-exact-sequence-certificate,def:weil-weight-subquotient-spectrum-certificate

Lean: Weil.Weight.Curve.StationaryInvariantTransferCertificate

An invariant-transfer certificate supplies the Frobenius-equivariant isomorphism

$$H^1_c(A_{\bar{k}}, j_*\mathcal{F}) \simeq (\mathcal{F}'_{\eta_{0'}})^{I_{0'}}$$

and the induced equality of spectra with multiplicity.

Definition 7.7.21 (Complete typed affine-estimate certificate). Uses: def:weil-curve-affine-estimate-input-data,def:weil-curve-ramification-exclusion-certificate,def:weil-curve-affine-transform-data,def:weil-pur-paired-factor-reality-certificate,def:weil-curve-transform-product-formula-certificate,def:weil-curve-transform-direct-summand-certificate,def:weil-curve-transform-real-ambient-data,def:weil-curve-transform-pur-data,def:weil-curve-stationary-weight-comparison-data,def:weil-curve-transform-boundary-upper-certificate,def:weil-curve-stationary-invariant-transfer-certificate

Lean: Weil.Weight.Curve.CompleteAffineEstimateCertificate

A complete affine-estimate certificate fixes one common affine input, additive character, Fourier context, transformed sheaf, and all eleven listed certificates over exactly those indices. It includes the no–Artin–Schreier ramification exclusion and the pure paired interior reality certificate. Its compatibility fields identify every occurrence and every Frobenius operator used by stationary phase and invariant transfer. It contains no upper-weight conclusion.

Proposition 7.7.22 (One certified irreducible affine estimate). Uses: def:weil-curve-complete-affine-estimate-certificate, def:weil-weight-spectrum-at-most-predicate

Lean: Weil.Weight.Curve.affineHc1_weightAtMost

For the single irreducible sheaf indexed by the complete affine certificate, every geometric-Frobenius eigenvalue on $H^1_c(A_{\bar{k}}, j_*\mathcal{F})$ has weight at most $w+1$.

Proof. Uses: lemma:weil-curve-transform-invariants-upper,def:weil-curve-complete-affine-estimate-certificate,def:weil-curve-stationary-invariant-transfer-certificate

Project the transformed invariant bound and the spectrum equality from the same complete certificate. Transport the former through the latter. □

Definition 7.7.23 (Projective curve theorem input datum). Uses: def:weil-weight-pointwise-pure-predicate,def:weil-lisse-heart-operations-typed-context

Lean: Weil.Weight.Curve.ProjectiveInputData

A projective input consists of a smooth projective geometrically connected curve X/k_0, a dense open $j : U \hookrightarrow X$, a lisse sheaf $\mathcal{F}$ on U pointwise pure of integral weight w, and candidates for $H^i(X_{\bar{k}}, j_*\mathcal{F})$ with geometric Frobenius for $i = 0, 1, 2$.

Definition 7.7.24 (Finite map to the projective line candidate)**.** Uses: `def:weil-curve-projective-input-data`

Lean: `Weil.Weight.Curve.FiniteMapCandidate`

A finite-map candidate consists of a nonconstant rational function $f \in k_0(X)$, its proposed extension $f : X \to \mathbf{P}^1$, and the induced restriction on a dense affine open. The function and morphism are explicit data.

Definition 7.7.25 (Finite-map morphism certificate)**.** Uses: `def:weil-curve-finite-map-candidate`

Lean: `Weil.Weight.Curve.FiniteMapCertificate`

A finite-map certificate supplies regularity of the extended map, finiteness, surjectivity, compatibility with the affine restrictions, and the fiber-dimension calculation.

Open obligation: `OBL-WGT-CURVE-FINITE-MAP`

Definition 7.7.26 (Finite constant-field extension candidate)**.** Uses: `def:weil-curve-projective-input-data`

Lean: `Weil.Weight.Curve.ConstantFieldExtensionCandidate`

A constant-field extension candidate is a positive integer m, the field $k_m = \mathbf{F}_{q^m}$, the base-changed curve and sheaf, and a rational point above the selected point at infinity.

Definition 7.7.27 (Constant-field purity-descent certificate)**.** Uses: `def:weil-curve-constant-field-extension-candidate`

Lean: `Weil.Weight.Curve.ConstantFieldDescentCertificate`

A descent certificate supplies the identity $\mathrm{Frob}_{q^m} = \mathrm{Frob}_q^m$ on every cohomology space, the multiplicity comparison of spectra, and the implication from modulus relative to q^m to modulus relative to q.

Open obligation: `OBL-WGT-CURVE-CONSTANT-DESCENT`

Definition 7.7.28 (Finite pushforward comparison candidate)**.** Uses: `def:weil-curve-finite-map-candidate,def:weil-curve-projective-input-data`

Lean: `Weil.Weight.Curve.FinitePushforwardComparisonCandidate`

Define comparison candidates

$$f_* j_* \mathcal{F} \rightleftarrows j'_* f_{U,*} \mathcal{F}$$

and

$$H^i(X_{\bar{k}}, j_* \mathcal{F}) \rightleftarrows H^i(\mathbf{P}^1_{\bar{k}}, f_* j_* \mathcal{F})$$

for every i.

Definition 7.7.29 (Finite pushforward comparison certificate)**.** Uses: `def:weil-curve-finite-map-certificate,def:weil-curve-finite-pushforward-comparison-candidate`

Lean: `Weil.Weight.Curve.FinitePushforwardComparisonCertificate`

A comparison certificate supplies all inverse equations, exactness of finite pushforward, vanishing of higher direct images, geometric-Frobenius equivariance, and compatibility with j_*.

Open obligation: `OBL-WGT-CURVE-FINITE-PUSHFORWARD`

Definition 7.7.30 (Punctual–lisse decomposition candidate)**.** Uses: `def:weil-curve-finite-pushforward-comparison-candidate,def:weil-lisse-heart-operations-typed-context`

Lean: `Weil.Weight.Curve.PunctualLisseDecompositionCandidate`

For the finite pushforward, define its maximal punctual subobject candidate, its quotient candidate, and the proposed decomposition of the quotient into irreducible lisse constituents on a dense open.

Definition 7.7.31 (Punctual–lisse decomposition certificate)**.** Uses: `def:weil-curve-punctual-lisse-decomposition-candidate`

Lean: `Weil.Weight.Curve.PunctualLisseDecompositionCertificate`

A decomposition certificate supplies exactness of the punctual quotient sequence, construction of every irreducible filtration factor, all Frobenius actions, spectrum decomposition, and termination of the finite filtration.

Open obligation: `OBL-WGT-CURVE-PUNCTUAL-LISSE`

Definition 7.7.32 (Open–closed long exact sequence candidate)**.** Uses: `def:weil-curve-projective-input-data`

Lean: `Weil.Weight.Curve.OpenClosedLongExactCandidate`

For the affine complement of infinity, define the restriction, boundary, and connecting maps in the proposed long exact sequence

$$\cdots \to H^i_c(A, j_* \mathcal{F}) \to H^i(\mathbf{P}^1, j_* \mathcal{F}) \to H^i(\{\infty\}, i^* j_* \mathcal{F}) \to \cdots .$$

Definition 7.7.33 (Open–closed exactness certificate). Uses: `def:weil-curve-open-closed-long-exact-candidate`

Lean: `Weil.Weight.Curve.OpenClosedExactnessCertificate`

An exactness certificate supplies every adjacent zero composite, image–kernel identity, endpoint vanishing, and geometric-Frobenius commutative square.

Open obligation: `OBL-WGT-CURVE-OPEN-CLOSED`

Definition 7.7.34 (Irreducible devissage filtration candidate). Uses: `def:weil-curve-punctual-lisse-decomposition-candidate`

Lean: `Weil.Weight.Curve.IrreducibleDevissageCandidate`

An irreducible devissage candidate is a finite filtration of the lisse quotient by subsheaves whose successive quotient candidates are irreducible, together with every inclusion and quotient map.

Definition 7.7.35 (Devissage connecting-map vanishing certificate). Uses: `def:weil-curve-irreducible-devissage-candidate,def:weil-mon-boundary-tensor-limit-certificate`

Lean: `Weil.Weight.Curve.ConnectingVanishingCertificate`

A connecting-map certificate supplies the long exact sequences, boundary weight bounds on their source and target, disjointness of the relevant weights, and the deduction that every connecting map is zero.

Open obligation: `OBL-WGT-CURVE-CONNECTING-ZERO`

Definition 7.7.36 (Geometrically constant cohomology certificate). Uses: `def:weil-curve-projective-input-data,def:weil-tate-line`

Lean: `Weil.Weight.Curve.GeometricallyConstantCohomologyCertificate`

For a geometrically constant constituent with value M, a certificate supplies

$$H_c^i(A_{\bar{k}}, M) = 0 \ (i \neq 2), \qquad H_c^2(A_{\bar{k}}, M) \simeq M(-1),$$

including inverse maps and geometric-Frobenius equivariance.

Open obligation: `OBL-WGT-CURVE-CONSTANT-COHOMOLOGY`

Definition 7.7.37 (Curve Poincare lower-bound certificate). Uses: `def:weil-curve-projective-input-data,def:weil-duality-perfect-pairing-certificate,def:weil-duality-frobenius-similitude-predicate,def:weil-lisse-heart-operations-typed-context`

Lean: `Weil.Weight.Curve.PoincareLowerBoundCertificate`

A curve lower-bound certificate supplies the perfect pairings

$$H^i(X_{\bar{k}}, j_*\mathcal{F}) \otimes H^{2-i}(X_{\bar{k}}, j_*\mathcal{F}^{\vee_{\mathrm{lis}}}) \longrightarrow E(-1),$$

the multiplier q, and the reciprocal eigenvalue bijection.

Open obligation: `OBL-WGT-CURVE-POINCARE`

Definition 7.7.38 (Complete reverse-reduction certificate). Uses: `def:weil-curve-complete-affine-estimate-certificate,def:weil-curve-finite-pushforward-comparison-certificate,def:weil-curve-constant-field-descent-certificate,def:weil-curve-punctual-lisse-decomposition-certificate,def:weil-curve-open-closed-exactness-certificate,def:weil-curve-irreducible-devissage-candidate,def:weil-curve-connecting-vanishing-certificate,def:weil-curve-geometrically-constant-certificate,def:weil-curve-poincare-lower-certificate`

Lean: `Weil.Weight.Curve.ReverseReductionCertificate`

A reverse-reduction certificate contains a finite index type of irreducible lisse constituents, a complete affine-estimate certificate for every nonconstant constituent, the punctual and geometrically constant branches, the finite devissage filtration with its termination proof, finite pushforward and constant-field comparisons, and the open–closed comparison. Every spectrum transfer is indexed by its source constituent, target degree, and common Frobenius operator. It contains no projective upper or lower weight conclusion. This expands the reductions in Laumon Proposition (4.3.2.1), printed pp. 206–207 (PDF pp. 77–78).

Definition 7.7.39 (Dual projective input datum). Uses: `def:weil-curve-projective-input-data,def:weil-weight-spectrum-dual,def:weil-lisse-heart-operations-typed-context`

Lean: `Weil.Weight.Curve.dualProjectiveInputData`

From the projective input define the dual input on the same X and U with the typed lisse dual sheaf $\mathcal{F}^{\vee_{\mathrm{lis}}}$, declared weight $-w$, and cohomology degree indexed independently. The stalk spectra are the inverse multisets of the original stalk spectra.

Lemma 7.7.40 (Pointwise purity of the dual curve input). *Uses:* *def:weil-curve-dual-projective-input-data*, *def:weil-weight-pointwise-pure-predicate*

Lean: *Weil.Weight.Curve.dualInput_isPointwisePure*

The dual input is pointwise pure of weight $-w$.

Proof. Uses: def:weil-curve-dual-projective-input-data,lemma:weil-weight-spectrum-dual-pure

At every closed point, instantiate the finite-spectrum dual-purity lemma with the original pure stalk spectrum. □

Definition 7.7.41 (Dual reverse-reduction certificate). Uses: def:weil-curve-dual-projective-input-data,def:weil-curve-reverse-reduction-certificate

Lean: Weil.Weight.Curve.DualReverseReductionCertificate

A dual reverse-reduction certificate is a separately indexed complete reverse-reduction certificate for $\mathcal{F}^{\vee_{\mathrm{lis}}}$. It includes a dual complete affine-estimate certificate for every dual constituent and explicit compatibility with the original duality maps. Its construction is not inferred from the primal certificate.

Open obligation: OBL-WGT-CURVE-DUAL-REDUCTION

Lemma 7.7.42 (Finite pushforward transports an upper bound). *Uses:* *def:weil-curve-finite-pushforward-comparison-certificate*, *def:weil-weight-spectrum-at-most-predicate*

Lean: *Weil.Weight.Curve.finitePushforward_transportsUpper*

An upper bound on the finite-pushforward cohomology spectrum transports to the original curve cohomology spectrum in the same degree.

Proof. Uses: def:weil-curve-finite-pushforward-comparison-certificate

Use the certified multiplicity-preserving Frobenius-spectrum bijection and rewrite each occurrence modulus along that bijection. □

Lemma 7.7.43 (Upper bound for a punctual constituent). *Uses:* *def:weil-curve-punctual-lisse-decomposition-certificate*, *def:weil-weight-spectrum-at-most-predicate*

Lean: *Weil.Weight.Curve.punctualPiece_upper*

Every punctual constituent has the upper bound stored in the decomposition certificate in its only nonzero cohomological degree.

Proof. Uses: def:weil-curve-punctual-lisse-decomposition-certificate

Project the punctual stalk spectrum and its sole-degree cohomology spectrum from the certificate; they are equal with multiplicity and Frobenius action. □

Lemma 7.7.44 (Upper bound for a geometrically constant constituent). *Uses:* *def:weil-curve-geometrically-constant-certificate*, *def:weil-weight-spectrum-at-most-predicate*

Lean: *Weil.Weight.Curve.constantPiece_upper*

The compactly supported cohomology of a geometrically constant constituent has the required degree-shifted upper bound.

Proof. Uses: def:weil-curve-geometrically-constant-certificate,lemma:weil-weight-spectrum-tate-twist-pure

Use the certified vanishing outside degree two. In degree two rewrite by $M(-1)$ and apply the Tate-twist weight formula once. □

Lemma 7.7.45 (One irreducible devissage step preserves an upper bound). *Uses:* *def:weil-curve-connecting-vanishing-certificate*, *def:weil-weight-spectrum-at-most-predicate*

Lean: *Weil.Weight.Curve.oneDevissageStep_upper*

If the subobject and quotient of one certified devissage step have the required upper bound, then the middle object has that bound.

Proof. Uses: def:weil-curve-connecting-vanishing-certificate,lemma:weil-weight-extension-at-most

The vanishing connecting map turns the relevant cohomology segment into the certified short exact sequence. Apply extension closure once. □

Lemma 7.7.46 (Finite irreducible devissage preserves an upper bound). *Uses:* def:weil-curve-irreducible-devissage-candidate, def:weil-curve-reverse-reduction-certificate, def:weil-weight-spectrum-at-most-predicate

Lean: Weil.Weight.Curve.finiteDevissage_upper

If every successive constituent of the stored finite filtration has the required upper bound, then the filtered lisse quotient has that bound.

Proof. Uses: def:weil-curve-irreducible-devissage-candidate, lemma:weil-curve-one-devissage-step-upper

Induct on the explicitly stored finite filtration length. The zero-length case is the zero spectrum; the successor case is one application of the preceding one-step lemma. □

Lemma 7.7.47 (Open–closed transport of the curve upper bound). *Uses:* def:weil-curve-open-closed-exactness-certificate, def:weil-weight-spectrum-at-most-predicate

Lean: Weil.Weight.Curve.openClosed_transportsUpper

If the affine and boundary terms in the certified open–closed sequence have the required upper bound, then the projective cohomology term has that bound.

Proof. Uses: def:weil-curve-open-closed-exactness-certificate, lemma:weil-weight-subquotient-at-most, lemma:weil-weight-extension-at-most

Exactness expresses the projective spectrum as one extension of a quotient of the affine spectrum by a subobject of the boundary spectrum. Apply the subquotient lemma and then extension closure. □

Proposition 7.7.48 (Projective curve upper bound). *Uses:* def:weil-curve-projective-input-data, def:weil-curve-reverse-reduction-certificate, def:weil-weight-spectrum-at-most-predicate

Lean: Weil.Weight.Curve.projectiveCohomology_weightAtMost

For $i = 0, 1, 2$, the Frobenius spectrum on $H^i(X_{\bar{k}}, j_*\mathcal{F})$ has weights at most $w + i$.

Proof. Uses: prop:weil-curve-affine-hc1-upper, lemma:weil-curve-punctual-piece-upper, lemma:weil-curve-constant-piece-upper, lemma:weil-curve-finite-devissage-upper, lemma:weil-curve-open-closed-transports-upper, lemma:weil-curve-finite-pushforward-transports-upper, def:weil-curve-reverse-reduction-certificate

The reverse-reduction certificate supplies the hypotheses of the finite devissage lemma from the separately certified affine, punctual, and constant pieces. Apply open–closed transport and then the single finite-pushforward spectrum comparison. □

Proposition 7.7.49 (Dual projective curve upper bound). *Uses:* def:weil-curve-dual-projective-input-data, def:weil-curve-dual-reverse-reduction-certificate, def:weil-weight-spectrum-at-most-predicate, def:weil-lisse-heart-operations-typed-context

Lean: Weil.Weight.Curve.dualProjectiveCohomology_weightAtMost

The spectrum on $H^{2-i}(X_{\bar{k}}, j_*\mathcal{F}^{\vee_{\mathrm{lis}}})$ has weights at most $-w + 2 - i$.

Proof. Uses: lemma:weil-curve-dual-input-pure-minus-weight, prop:weil-curve-projective-upper

Instantiate the projective upper proposition for the dual input in degree $2 - i$. □

Lemma 7.7.50 (The scalar Poincare upper-to-lower calculation). *Uses:* def:weil-weight-spectrum-at-most-predicate, def:weil-weight-spectrum-at-least-predicate

Lean: Weil.Weight.Curve.poincareScalarUpperToLower

If nonzero scalars α, γ satisfy $\alpha\gamma = q$ and γ has weight at most $-w + 2 - i$, then α has weight at least $w + i$.

Proof. Uses: def:weil-weight-spectrum-at-most-predicate, def:weil-weight-spectrum-at-least-predicate

The upper bound is $|\iota(\gamma)| \le q^{(-w+2-i)/2}$. Since $|\iota(\alpha)| = q|\iota(\gamma)|^{-1}$, inversion reverses the positive inequality and gives

$$|\iota(\alpha)| \ge q^{1-(-w+2-i)/2} = q^{(w+i)/2}.$$

□

Lemma 7.7.51 (Poincare pairing transports the dual upper bound). *Uses:* def:weil-curve-poincare-lower-certificate, def:weil-weight-spectrum-at-most-predicate, def:weil-weight-spectrum-at-least-predicate

Lean: Weil.Weight.Curve.pairingUpperToLower

The dual upper bound in degree $2 - i$ implies the original lower bound $w + i$ in degree i.

Proof. Uses: def:weil-curve-poincare-lower-certificate,lemma:weil-curve-poincare-scalar-upper-to-lower

For each original occurrence, take its multiplicity-matched dual occurrence from the reciprocal eigenvalue bijection. The pairing multiplier is q; apply the scalar calculation to that pair. □

Proposition 7.7.52 (Projective curve lower bound)**.** *Uses: def:weil-curve-projective-input-data, def:weil-curve-dual-reverse-reduction-certificate, def:weil-curve-poincare-lower-certificate, def:weil-curve-weight-spectrum-at-least-predicate*

Lean: Weil.Weight.Curve.projectiveCohomology_weightAtLeast

For $i = 0, 1, 2$, the spectrum on $H^i(X_{\bar{k}}, j_*\mathcal{F})$ has weights at least $w + i$.

Proof. Uses: prop:weil-curve-dual-projective-upper,lemma:weil-curve-pairing-upper-to-lower

Apply the pairing transport lemma to the dual upper bound. □

Lemma 7.7.53 (Opposite projective bounds give degree purity)**.** *Uses: def:weil-weight-spectrum-at-most-predicate, def:weil-weight-spectrum-at-least-predicate, def:weil-weight-pure-spectrum-predicate*

Lean: Weil.Weight.Curve.projectiveCohomology_degreePure

A spectrum bounded both above and below by $w + i$ is pure of weight $w + i$.

Proof. Uses: lemma:weil-weight-bounds-give-purity

Instantiate the bounds-to-purity lemma with the common integer $w + i$. □

Theorem 7.7.54 (Conditional projective curve purity)**.** *Uses: def:weil-curve-projective-input-data, def:weil-curve-reverse-reduction-certificate, def:weil-curve-dual-reverse-reduction-certificate, def:weil-weight-pure-spectrum-predicate*

Lean: Weil.Weight.Curve.projectiveCohomology_isIotaPure

For $i = 0, 1, 2$, the geometric-Frobenius spectrum on $H^i(X_{\bar{k}}, j_*\mathcal{F})$ is pure of weight $w + i$.

Proof. Uses: prop:weil-curve-projective-upper,prop:weil-curve-projective-lower,lemma:weil-curve-projective-degree-pure

Fix $i \in \{0, 1, 2\}$. Supply the preceding upper and lower propositions to the degree-purity lemma. □

7.8 Deligne's direct-image devissage

Definition 7.8.1 (Bare finite-type morphism datum for dimension bounds)**.** Uses:

Lean: Weil.Weight.DirectImage.BareMorphismData

A bare direct-image morphism datum consists of finite-type k_0-schemes X, Y, a morphism $f : X \to Y$, and explicit proofs that f is separated and of finite type. It contains no compactification, six-functor context, or candidate dimension bound.

Lemma 7.8.2 (A nonzero prime quotient lowers transcendence degree)**.** *Uses:*

Lean: Weil.Weight.DirectImage.Dimension.primeQuotientTranscendenceDrop

Let A be a nonzero finitely generated domain over a field K, and let $0 \neq \mathfrak{p} \subset A$ be prime. Then

$$\operatorname{trdeg}_K \operatorname{Frac}(A/\mathfrak{p}) + 1 \leq \operatorname{trdeg}_K \operatorname{Frac}(A).$$

Proof. Uses:

Take a finite transcendence basis $\bar{z}_1, \dots, \bar{z}_r$ of $\operatorname{Frac}(A/\mathfrak{p})$, write each member as a quotient with denominator outside $\mathfrak{p}$, and lift those quotients to $z_1, \dots, z_r \in \operatorname{Frac}(A)$. Any polynomial relation among the lifts lies in the localization of A at $A \setminus \mathfrak{p}$ and reduces there to the same relation among the barred elements in $\operatorname{Frac}(A/\mathfrak{p})$; hence the lifts are algebraically independent.

Choose $0 \neq a \in \mathfrak{p}$. If a were algebraic over $K(z_1, \dots, z_r)$, clear the denominators in a dependence of least degree and remove a common power of a. This gives

$$c_0(z) + c_1(z)a + \cdots + c_s(z)a^s = 0, \qquad c_0(z) \neq 0,$$

with the c_i polynomials over K. Reduction modulo $\mathfrak{p}$ would give $c_0(\bar{z}) = 0$, contrary to algebraic independence. Thus $z_1, \dots, z_r, a$ are algebraically independent, which is the displayed inequality. □

Lemma 7.8.3 (Krull dimension of a polynomial algebra over a field)**.** *Uses:*

Lean: `Weil.Weight.DirectImage.Dimension.polynomialKrullDimension`

For a field K and $m \in \mathbf{N}$,

$$\dim K[T_1, \dots, T_m] = m.$$

Consequently every quotient of this polynomial algebra has Krull dimension at most m.

Proof. Uses: `lemma:weil-direct-image-prime-quotient-transcendence-drop`

The chain

$$(0) \subsetneq (T_1) \subsetneq (T_1, T_2) \subsetneq \cdots \subsetneq (T_1, \dots, T_m)$$

gives the lower bound. For the upper bound, let $\mathfrak{p}_0 \subsetneq \cdots \subsetneq \mathfrak{p}_s$ be a strict prime chain in $P = K[T_1, \dots, T_m]$. Starting with the finitely generated domain $P/\mathfrak{p}_0$, apply the transcendence-drop lemma successively to the nonzero primes $\mathfrak{p}_i/\mathfrak{p}_{i-1}$. It gives

$$s \le \operatorname{trdeg}_K \operatorname{Frac}(P/\mathfrak{p}_0) \le m,$$

where the last inequality holds because that fraction field is generated as a field by the images of the m polynomial variables. Hence $\dim P = m$. Finally, contraction along a surjection embeds each strict prime chain in the quotient into one upstairs, so quotient dimension cannot increase. □

Lemma 7.8.4 (Dimension bound from a finite algebra generating list)**.** *Uses:* `def:weil-direct-image-bare-morphism-data`

Lean: `Weil.Weight.DirectImage.Dimension.finiteTypeAlgebraDimensionBound`

Let A be a K-algebra generated by a displayed list of m elements. Then

$$\dim A \le m.$$

Proof. Uses: `lemma:weil-direct-image-polynomial-krull-dimension`

The displayed generators give a surjective homomorphism

$$K[T_1, \dots, T_m] \twoheadrightarrow A.$$

The quotient clause of the polynomial-dimension lemma sends every strict prime chain of A to one of the same length in the polynomial ring. Hence $\dim A \le m$. □

Proposition 7.8.5 (Finite-type schemes over a field have finite dimension)**.** *Uses:* `def:weil-direct-image-bare-morphism-data`

Lean: `Weil.Weight.DirectImage.Dimension.finiteTypeSchemeDimensionBound`

For a finite-type K-scheme Z, there is an explicit natural number d_Z such that

$$\dim Z \le d_Z.$$

For the empty scheme take $d_Z = 0$.

Proof. Uses: `lemma:weil-direct-image-finite-type-algebra-dimension-bound`

Finite type over a field implies quasi-compactness. Choose a finite affine open cover

$$Z = \bigcup_{a=1}^{r} \operatorname{Spec} A_a.$$

For each a, unpack finite type and select a finite algebra generating list of length m_a. Put

$$d_Z := \max(0, m_1, \dots, m_r).$$

The algebra lemma gives $\dim \operatorname{Spec} A_a \le m_a \le d_Z$.

It remains to pass from the cover to Z. Given a strict chain $Z_0 \subsetneq \cdots \subsetneq Z_s$ of irreducible closed subsets, choose a cover member meeting Z_0. Every inclusion remains strict after intersection with that member: in the irreducible space Z_{i+1}, the chosen open and the nonempty open $Z_{i+1} \setminus Z_i$ meet. The intersections therefore form a strict chain in one affine cover member, so $s \le d_Z$. This proves the global bound. □

Definition 7.8.6 (Natural-valued source dimension)**.** Uses: `def:weil-direct-image-bare-morphism-data`

Lean: `Weil.Weight.DirectImage.Dimension.naturalSourceDimension`

Let $\mathcal{S}_X \subseteq \mathbf{N}$ be the set of natural numbers n such that every strict chain of irreducible closed subsets of X has length at most n. Define

$$\dim_0 X := \begin{cases} \min \mathcal{S}_X, & \mathcal{S}_X \neq \varnothing, \\ 0, & \mathcal{S}_X = \varnothing. \end{cases}$$

For an empty scheme every natural number is a bound, so this definition gives $\dim_0 X = 0$. No conversion from an extended-valued dimension to a natural number is used. Formally the inhabited branch is the value returned by `Nat.find` from a witness of $\exists n,\ n \in \mathcal{S}_X$.

Lemma 7.8.7 (The natural source dimension bounds the source). *Uses:* `def:weil-direct-image-natural-source-dimension`

Lean: `Weil.Weight.DirectImage.Dimension.naturalSourceDimension_spec`

One has

$$\dim X \le \dim_0 X.$$

Proof. *Uses:* `prop:weil-direct-image-finite-type-scheme-dimension-bound`

The finite-type scheme proposition gives a natural number $d_X \in \mathcal{S}_X$, so the inhabited branch of the definition applies. Well-ordering of $\mathbf{N}$ places $\min \mathcal{S}_X$ in $\mathcal{S}_X$. Expanding that membership says that every strict irreducible chain in X has length at most $\dim_0 X$, which is precisely the displayed dimension inequality. □

Lemma 7.8.8 (Affine ordinary fibers do not exceed source dimension). *Uses:* `def:weil-direct-image-bare-morphism-data`, `def:weil-direct-image-natural-source-dimension`

Lean: `Weil.Weight.DirectImage.Dimension.affineOrdinaryFiberDimension`

Let $V = \operatorname{Spec} B \subseteq Y$, let $U = \operatorname{Spec} A \subseteq f^{-1}(V)$, and let $y \in V$ correspond to $\mathfrak{q} \subset B$. Then

$$\dim(U \times_Y \operatorname{Spec} \kappa(y)) \le \dim_0 X.$$

Proof. *Uses:* `lemma:weil-direct-image-natural-source-dimension-spec`

Put $S = B \setminus \mathfrak{q}$. The ordinary-fiber chart has ring

$$A \otimes_B \kappa(\mathfrak{q}) \simeq S^{-1}A/\mathfrak{q}S^{-1}A.$$

Its prime ideals correspond to primes of A contracting to $\mathfrak{q}$, and localization and quotient preserve strict inclusions under this correspondence. Thus every strict prime chain in the fiber chart is a strict prime chain in A. Consequently

$$\dim(U_y) \le \dim A = \dim U \le \dim X \le \dim_0 X,$$

where the last inequality is the source-dimension specification. For the open-subscheme inequality, close a strict irreducible chain from U in X. Intersecting each closure back with U recovers the original chain, so distinct members cannot acquire the same closure. □

Proposition 7.8.9 (Uniform bound for all ordinary fibers). *Uses:* `def:weil-direct-image-bare-morphism-data`, `def:weil-direct-image-natural-source-dimension`

Lean: `Weil.Weight.DirectImage.Dimension.ordinaryFiberDimensionBound`

For every point $y \in Y$,

$$\dim X_y \le \dim_0 X.$$

Proof. *Uses:* `lemma:weil-direct-image-affine-ordinary-fiber-dimension`

Choose an affine neighborhood V of y. Since f is of finite type, X_V is quasi-compact and has a finite affine open cover $U_1, \dots, U_r$. Their ordinary fibers form a finite affine open cover of X_y, and the affine lemma bounds every member by $\dim_0 X$. The finite-open-cover chain argument in the finite-type scheme proposition then bounds the whole fiber by the same natural number. □

Lemma 7.8.10 (Triangular Noether-normalization step). *Uses:*

Lean: `Weil.Weight.DirectImage.Dimension.noetherTriangularStep`

Let $m \ge 1$ and $0 \ne P \in K[X_1, \dots, X_m]$. There is a triangular polynomial automorphism after which P, multiplied by a nonzero scalar, is monic in the last variable.

Proof. *Uses:*

Write the finite support of P as exponent vectors $\alpha = (\alpha_1, \dots, \alpha_m)$. Choose $B > 1 + \max_{\alpha,i} \alpha_i$ and put $N_i = B^i$ for $i < m$. Under

$$X_i \longmapsto X_i + X_m^{N_i} \quad (i < m), \qquad X_m \longmapsto X_m,$$

the largest X_m-power contributed by X^α has exponent

$$w(\alpha) = \alpha_m + \sum_{i<m} \alpha_i N_i.$$

These base-B numerals are distinct, so exactly one term has maximal weight and its coefficient is nonzero. Dividing by that coefficient makes the transformed relation monic. The substitution is triangular and its inverse subtracts the same powers, so it is an automorphism. □

Lemma 7.8.11 (A monic relation removes one algebra generator)**.** *Uses:*

Lean: `Weil.Weight.DirectImage.Dimension.noetherMonicFiniteStep`

If an algebra generated by $x_1, \dots, x_m$ has a monic relation of degree s for x_m over $B = K[x_1, \dots, x_{m-1}]$, then it is generated as a B-module by

$$1, x_m, \dots, x_m^{s-1}.$$

Proof. Uses:

The monic relation expresses x_m^s as a B-linear combination of lower powers. Multiplication by x_m^t and induction on t reduces every higher power to the displayed finite list. Every polynomial in the algebra generators therefore has such an expression. In particular the extension $B \to A = B[x_m]$ is finite; it is integral because x_m satisfies the displayed monic relation. □

Proposition 7.8.12 (Finite injective Noether normalization)**.** *Uses:*

Lean: `Weil.Weight.DirectImage.Dimension.noetherNormalization`

For every nonzero finitely generated K-algebra A, there are $e \in \mathbf{N}$ and an injective finite homomorphism

$$K[T_1, \dots, T_e] \hookrightarrow A.$$

Proof. Uses: `lemma:weil-direct-image-noether-triangular-step`, `lemma:weil-direct-image-noether-monic-finite-step`

Choose a surjection $K[X_1, \dots, X_m] \twoheadrightarrow A$ and induct on m. If its kernel is zero, take $e = m$, the identity polynomial subalgebra, and module generator 1. If the kernel is nonzero, select a nonzero relation. Then $m \geq 1$, because a nonzero quotient of the field K has zero kernel. The triangular lemma changes generators so that the relation is monic in X_m; the monic-finite lemma makes A finite over the subalgebra B generated by the other $m - 1$ elements. Apply induction to B. If B is finite over the injected polynomial algebra and (b_i) and (c_j) are the two finite module generating lists, then $(b_i c_j)$ generates A over that polynomial algebra. Injectivity is preserved through the inclusions. This completes the induction. □

Lemma 7.8.13 (Lying over for an injective integral extension)**.** *Uses:*

Lean: `Weil.Weight.DirectImage.Dimension.integralLyingOver`

If $A \hookrightarrow B$ is injective and integral, then every prime $\mathfrak{p} \subset A$ is the contraction of a prime of B.

Proof. Uses:

Put $S = A \setminus \mathfrak{p}$. Localization gives an injective integral map $A_{\mathfrak{p}} \hookrightarrow S^{-1}B$; the target is nonzero because the map is injective. Choose a maximal ideal $\mathfrak{m}$ of $S^{-1}B$. Its contraction is maximal in $A_{\mathfrak{p}}$: after quotienting, $S^{-1}B/\mathfrak{m}$ is a field integral over the contracted domain. A domain over which a field is integral is itself a field, since an integral equation for a^{-1} expresses a^{-1} in the domain for each nonzero a. The only maximal ideal of $A_{\mathfrak{p}}$ is $\mathfrak{p}A_{\mathfrak{p}}$. Pulling $\mathfrak{m}$ back to B therefore gives a prime whose contraction is $\mathfrak{p}$. □

Lemma 7.8.14 (Incomparability for an integral extension)**.** *Uses:*

Lean: `Weil.Weight.DirectImage.Dimension.integralIncomparability`

For an integral map $A \to B$, comparable primes of B with the same contraction are equal.

Proof. Uses:

Suppose $\mathfrak{q} \subseteq \mathfrak{q}'$ have common contraction $\mathfrak{p}$. Quotient by $\mathfrak{q}$ and localize at the nonzero elements of $A/\mathfrak{p}$. The resulting domain is integral over the field $K = \mathrm{Frac}(A/\mathfrak{p})$, hence is itself a field: for a nonzero integral element x, an integral equation of least degree has nonzero constant term and expresses x^{-1} as a polynomial in x. The localization of $\mathfrak{q}'/\mathfrak{q}$ is therefore zero. If $b \in \mathfrak{q}'$, some $s \in A \setminus \mathfrak{p}$ has $sb \in \mathfrak{q}$. Since $s \notin \mathfrak{q}$ and $\mathfrak{q}$ is prime, $b \in \mathfrak{q}$. Thus $\mathfrak{q}' = \mathfrak{q}$. □

Lemma 7.8.15 (Going up for an integral extension)**.** *Uses:*

Lean: `Weil.Weight.DirectImage.Dimension.integralGoingUp`

Let $A \to B$ be integral, let $\mathfrak{p} \subseteq \mathfrak{p}'$ be primes of A, and let $\mathfrak{q} \subset B$ lie over $\mathfrak{p}$. There is a prime $\mathfrak{q}' \supseteq \mathfrak{q}$ lying over $\mathfrak{p}'$.

Proof. Uses: `lemma:weil-direct-image-integral-lying-over`

The induced map $A/\mathfrak{p} \hookrightarrow B/\mathfrak{q}$ is injective and integral. Localize it at the complement of $\mathfrak{p}'/\mathfrak{p}$. Apply lying over to the maximal ideal of the localized source. Pulling the resulting prime first back to $B/\mathfrak{q}$ and then to B gives $\mathfrak{q}'$; localization shows that its contraction is exactly $\mathfrak{p}'$, and the quotient construction gives $\mathfrak{q} \subseteq \mathfrak{q}'$. □

Lemma 7.8.16 (Injective integral extensions preserve Krull dimension)**.** *Uses:*

Lean: `Weil.Weight.DirectImage.Dimension.integralExtensionDimension`

If $A \hookrightarrow B$ is an injective integral homomorphism of commutative rings, then

$$\dim B = \dim A.$$

Proof. Uses: `lemma:weil-direct-image-integral-lying-over`, `lemma:weil-direct-image-integral-incomparability`, `lemma:weil-direct-image-integral-going-up`

Contraction sends a strict chain of primes of B to a strict chain of primes of A: if two comparable primes have the same contraction, incomparability for integral extensions makes them equal. Hence $\dim B \le \dim A$.

Conversely, lying over supplies a prime of B above the first member of any finite strict prime chain in A. Repeated going up extends it, one member at a time, to a chain lying over the whole chain. Distinct contractions force the lifted inclusions to be strict. Thus every finite chain downstairs lifts to one of the same length upstairs, giving $\dim A \le \dim B$. □

Proposition 7.8.17 (Field extension preserves finite-type Krull dimension)**.** *Uses:*

Lean: `Weil.Weight.DirectImage.Dimension.fieldExtensionDimension`

For every field extension L/K and finitely generated K-algebra A,

$$\dim(A \otimes_K L) = \dim A.$$

Proof. Uses: `prop:weil-direct-image-noether-normalization`, `lemma:weil-direct-image-integral-extension-dimension`, `lemma:weil-direct-image-polynomial-krull-dimension`

The zero algebra is immediate. Otherwise choose a finite injective Noether normalization

$$K[T_1, \dots, T_e] \hookrightarrow A.$$

The integral-extension lemma and the polynomial-dimension lemma give $\dim A = e$. Tensoring with L preserves injectivity because a field extension is flat. Concretely, after choosing a K-basis of L, tensoring a K-linear injection is the direct sum of copies of that injection. The same finite module generators remain generators after base change. Thus

$$L[T_1, \dots, T_e] \hookrightarrow A \otimes_K L$$

is again injective and finite, hence integral. For the last implication, let $v_1, \dots, v_t$ generate the finite module and write multiplication by an element b as $bv_i = \sum_j a_{ij} v_j$. The adjugate identity for $bI - (a_{ij})$ shows that its monic determinant annihilates every v_i, and hence annihilates 1, because 1 lies in their span. Thus b is integral. Applying the same two lemmas over L gives $\dim(A \otimes_K L) = e = \dim A$. □

Proposition 7.8.18 (Uniform bound for all geometric fibers)**.** *Uses:* `def:weil-direct-image-bare-morphism-data`, `def:weil-direct-image-natural-source-dimension`

Lean: `Weil.Weight.DirectImage.Dimension.geometricFiberDimensionBound`

For every geometric point $\bar{y} = \operatorname{Spec} \Omega \to Y$,

$$\dim X_{\bar{y}} \le \dim_0 X.$$

Proof. Uses: `prop:weil-direct-image-ordinary-fiber-dimension-bound`, `prop:weil-direct-image-field-extension-dimension`

Let y be the image of $\bar{y}$. The ordinary fiber X_y is of finite type over $\kappa(y)$. On every member $\operatorname{Spec} C_a$ of a finite affine cover, the corresponding geometric-fiber chart has ring $C_a \otimes_{\kappa(y)} \Omega$, of the same dimension by the field-extension proposition. The finite-cover chain argument therefore gives

$$\dim X_{\bar{y}} = \dim X_y \le \dim_0 X.$$

□

Definition 7.8.19 (Least ordinary-fiber dimension bound)**.** Uses: `def:weil-direct-image-bare-morphism-data`

Lean: `Weil.Weight.DirectImage.Dimension.leastOrdinaryFiberBound`

Let

$$\mathcal{B}_f = \{\, n \in \mathbf{N} : \dim X_y \le n \text{ for every } y \in Y \,\}.$$

Define b_f^{rel} to be $\min \mathcal{B}_f$ when this set is inhabited and zero otherwise. The inhabited branch is implemented by `Nat.find` on a witness of $\exists n,\ n \in \mathcal{B}_f$.

Lemma 7.8.20 (Least ordinary-fiber bound specification). *Uses:* def:weil-direct-image-least-ordinary-fiber-bound, def:weil-direct-image-natural-source-dimension

Lean: Weil.Weight.DirectImage.Dimension.leastOrdinaryFiberBound_spec

The set $\mathcal{B}_f$ is inhabited, $b_f^{\mathrm{rel}} \in \mathcal{B}_f$, and therefore

$$\dim X_y \le b_f^{\mathrm{rel}} \qquad (y \in Y).$$

Proof. Uses: prop:weil-direct-image-ordinary-fiber-dimension-bound

The ordinary-fiber proposition gives $\dim_0 X \in \mathcal{B}_f$, so the inhabited branch applies. Well-ordering of $\mathbf{N}$ says that the least element of an inhabited subset belongs to it. Expanding membership gives the displayed bound. □

Lemma 7.8.21 (Least ordinary bound also controls geometric fibers). *Uses:* def:weil-direct-image-least-ordinary-fiber-bound

Lean: Weil.Weight.DirectImage.Dimension.leastGeometricFiberBound_spec

For every geometric point $\bar{y} \to Y$,

$$\dim X_{\bar{y}} \le b_f^{\mathrm{rel}}.$$

Proof. Uses: lemma:weil-direct-image-least-ordinary-fiber-bound-spec, prop:weil-direct-image-field-extension-dimension

Apply the ordinary specification to the image y, then use the field-extension proposition on a finite affine cover of X_y to identify $\dim X_{\bar{y}} = \dim X_y$. □

Definition 7.8.22 (Pure dimension-bound candidate over bare morphism data). Uses: def:weil-direct-image-bare-morphism-data

Lean: Weil.Weight.DirectImage.PureDimensionBoundCandidate

A pure dimension-bound candidate consists of a bare morphism datum and two natural numbers $b_{\mathrm{rel}}, b_{\mathrm{src}}$. It contains no compactification or six-functor data.

Definition 7.8.23 (Pure dimension-bound predicate). Uses: def:weil-direct-image-pure-dimension-bound-candidate

Lean: Weil.Weight.DirectImage.IsPureDimensionBound

The pure bound predicate is

$$\bigl(\forall y \in Y,\ \dim X_y \le b_{\mathrm{rel}}\bigr) \quad \text{and} \quad \dim X \le b_{\mathrm{src}},$$

with the empty-scheme convention equal to zero.

Definition 7.8.24 (Pure dimension-bound certificate). Uses: def:weil-direct-image-pure-dimension-bound-candidate, def:weil-direct-image-pure-dimension-bound-predicate

Lean: Weil.Weight.DirectImage.PureDimensionBoundCertificate

A pure certificate is a pure candidate together with a proof of its bound predicate.

Proposition 7.8.25 (Canonical pure dimension-bound certificate). *Uses:* def:weil-direct-image-pure-dimension-bound-certificate, def:weil-direct-image-natural-source-dimension, def:weil-direct-image-least-ordinary-fiber-bound

Lean: Weil.Weight.DirectImage.pureDimensionBoundCertificate

Every bare morphism datum has a pure dimension-bound certificate with

$$b_{\mathrm{rel}} = b_f^{\mathrm{rel}}, \qquad b_{\mathrm{src}} = \dim_0 X.$$

Proof. Uses: lemma:weil-direct-image-least-ordinary-fiber-bound-spec, lemma:weil-direct-image-natural-source-dimension-spec, def:weil-direct-image-pure-dimension-bound-predicate

Install the two displayed natural numbers. The least-bound lemma proves the first conjunct of the pure predicate, and the source-dimension lemma proves the second. Pair the candidate with these two proofs. □

Definition 7.8.26 (Geometric data for a compactly supported direct image). Uses: def:nagata-compactification, def:weil-six-functor-typed-context

Lean: Weil.Weight.DirectImage.GeometricMorphismData

A geometric direct-image datum consists of finite-type schemes X, Y over k_0, a morphism $f : X \to Y$, explicit proofs that f is separated and of finite type, a selected compactification of f, a typed six-functor context containing the indexed functor $Rf_!$, and two natural numbers b_{rel} and b_{src}. The two numbers are candidate bounds only; their bounding properties are not fields of this raw datum.

Definition 7.8.27 (Fiber- and source-dimension bound predicate)**.** Uses: `def:weil-direct-image-geometric-morphism-data`

Lean: `Weil.Weight.DirectImage.IsDimensionBound`

Define the dimension-bound predicate to be the conjunction

$$\big(\forall y \in Y,\ \dim X_y \le b_{\mathrm{rel}}\big) \quad \text{and} \quad \dim X \le b_{\mathrm{src}}.$$

For an empty scheme the dimension entering this predicate is defined to be zero. Thus neither a maximum over an empty type nor an unproved attainment claim occurs.

Definition 7.8.28 (Certified direct-image dimension bounds)**.** Uses: `def:weil-direct-image-geometric-morphism-data`, `def:weil-direct-image-dimension-bound-predicate`

Lean: `Weil.Weight.DirectImage.DimensionBoundCertificate`

A dimension-bound certificate is a geometric direct-image datum together with a proof of the preceding bound predicate. Assembly over supplied compatible geometric and pure dimension data is proved below. Constructing the geometric datum from a bare separated finite-type morphism remains the separate packaging obligation recorded there.

Definition 7.8.29 (Bare datum underlying geometric direct-image data)**.** Uses: `def:weil-direct-image-bare-morphism-data`, `def:weil-direct-image-geometric-morphism-data`

Lean: `Weil.Weight.DirectImage.GeometricMorphismData.toBare`

Forget the compactification, typed six-functor context, and candidate bounds from a geometric direct-image datum, retaining its schemes, morphism, and separated finite-type proofs as a bare morphism datum.

Definition 7.8.30 (Compatibility of geometric and pure dimension data)**.** Uses: `def:weil-direct-image-geometric-to-bare`, `def:weil-direct-image-pure-dimension-bound-certificate`

Lean: `Weil.Weight.DirectImage.GeometricPureDimensionCompatibility`

A geometric datum and a pure dimension certificate are compatible when the entire bare record obtained from the geometric datum equals the bare record stored in the pure certificate. This single equality includes the transports for the dependent morphism and proof fields.

Proposition 7.8.31 (Dimension-certificate adapter over supplied geometry)**.** *Uses:* *`def:weil-direct-image-geometric-morphism-data`, `def:weil-direct-image-dimension-bound-certificate`, `def:weil-direct-image-pure-dimension-bound-certificate`, `def:weil-direct-image-geometric-pure-dimension-compatibility`*

Lean: *`Weil.Weight.DirectImage.dimensionCertificateAdapter`*

Given compatible geometric and pure dimension data, retain the compactification and typed six-functor fields, replace the two candidate bounds by those of the pure certificate, and obtain a direct-image dimension-bound certificate.

Proof. Uses: `def:weil-direct-image-pure-dimension-bound-predicate`, `def:weil-direct-image-dimension-bound-predicate`

Form the record update on the geometric datum. Compatibility identifies its underlying bare record with the one in the pure certificate. Eliminate that record equality; after equality induction the schemes, morphism, and separated finite-type proofs agree definitionally. Retain the geometric compactification and typed context, replace its two numeric fields by the pure bounds, and use the two pure-predicate conjuncts as the geometric dimension-bound predicate. Pair the updated datum with that proof. □

Proposition 7.8.32 (Geometric direct-image data from a bare morphism)**.** *Uses:* *`def:weil-direct-image-bare-morphism-data`, `def:weil-direct-image-geometric-morphism-data`, `def:weil-direct-image-geometric-to-bare`*

Lean: *`Weil.Weight.DirectImage.geometricDataPackaging`*

For every bare morphism datum D, construct a geometric direct-image datum G together with an equality

$$G.\mathrm{toBare} = D.$$

Thus the selected compactification and typed six-functor context are attached without an untyped comparison of dependent morphism fields.

Open obligation: *`OBL-WGT-DI-GEOMETRIC-DATA-PACKAGING`*

Proof. Uses: `def:weil-direct-image-geometric-morphism-data`, `def:weil-direct-image-geometric-to-bare`

No proof is claimed. The selected compactification, complete typed six-functor context, and equality to the input bare record are exactly the data isolated by the displayed OPEN. □

Proposition 7.8.33 (Dimension-bound certificate from a bare morphism). Uses: *def:weil-direct-image-bare-morphism-data, def:weil-direct-image-dimension-bound-certificate*

Lean: *Weil.Weight.DirectImage.dimensionBoundCertificateFromBare*

For every bare morphism datum, construct a direct-image dimension-bound certificate.

Open obligation: *OBL-WGT-DI-FIBER-BOUND*

Proof. Uses: `prop:weil-direct-image-pure-dimension-bound-certificate, prop:weil-direct-image-dimension-certificate-adapter, prop:weil-direct-image-geometric-data-packaging`

The pure certificate constructed from the input stores that input bare record. The packaging OPEN would supply G with $G.\mathrm{toBare} = D$, which is the compatibility equality required by the adapter. No proof of the proposition is claimed until that packaging datum exists. □

Definition 7.8.34 (Direct-image weight input datum). Uses: `def:weil-weight-mixed-certificate, def:weil-direct-image-dimension-bound-certificate`

Lean: `Weil.Weight.DirectImage.InputData`

A direct-image input consists of certified geometric morphism and dimension data, a constructible Weil sheaf $\mathcal{F}$ on X, an integer n, and a mixedness certificate showing weights at most n. In particular, separatedness, compactifiability, a selected compactification, finite type of both source and target over k_0, and the typed functor $Rf_!$ are explicit input fields.

Definition 7.8.35 (Exceptional direct-image cohomology candidate). Uses: `def:weil-direct-image-weight-input-data`

Lean: `Weil.Weight.DirectImage.cohomologyCandidate`

For every integer i, define the proposed sheaf

$$R^i f_! \mathcal{F} := \mathcal{H}^i(Rf_! \mathcal{F}),$$

its stalk candidates, geometric-Frobenius actions, and closed-point spectra. No six-operation construction or finiteness statement is included.

Definition 7.8.36 (Direct-image upper mixedness candidate datum). Uses: `def:weil-direct-image-cohomology-candidate, def:weil-weight-filtration-candidate, def:weil-weight-geometric-operations-typed-context`

Lean: `Weil.Weight.DirectImage.UpperMixedCertificateData`

In degree i, an upper mixedness candidate datum C_i consists of a finite filtration candidate $W(C_i)$, a candidate underlying sheaf $|C_i|$, forward and reverse comparison maps

$$|C_i| \rightleftarrows R^i f_! \mathcal{F},$$

and proposed stalkwise geometric-Frobenius comparison maps on every filtration step and graded piece. It contains no inverse equation, exactness proof, purity proof, or weight-index inequality.

Definition 7.8.37 (Direct-image upper-weight predicate). Uses: `def:weil-direct-image-upper-mixed-certificate-data, def:weil-weight-filtration-exactness-predicate, def:weil-weight-filtration-purity-predicate`

Lean: `Weil.Weight.DirectImage.IsUpperWeightBound`

For a supplied candidate C_i, define IsUpperWeightBound(C_i) as a record with exactly the following named fields:

(i) `underlying`: a specified object equality $|C_i| = R^i f_! \mathcal{F}$, both inverse equations for the displayed comparison maps, and the equations identifying those maps with transport along this equality;

(ii) `exact`: $W(C_i)$ satisfies the filtration-exactness predicate;

(iii) `gradedPure`: every graded piece satisfies the pointwise purity predicate at its declared index;

(iv) `indexBound`: every nonzero graded index r satisfies $r \leq n + i$;

(v) `frobenius`: every underlying, filtration, quotient, and graded comparison square commutes with the selected geometric Frobenius.

No one of these fields is part of the raw candidate datum.

Definition 7.8.38 (Coefficient long-exact-sequence candidate). Uses: `def:weil-direct-image-cohomology-candidate, def:weil-weight-geometric-operations-typed-context`

Lean: `Weil.Weight.DirectImage.CoefficientLESCandidate`

For a short exact coefficient sequence $0 \to \mathcal{F}' \to \mathcal{F} \to \mathcal{F}'' \to 0$, define all terms and connecting maps in the proposed long exact sequence of $R^i f_!$.

Definition 7.8.39 (Coefficient devissage certificate). Uses: def:weil-direct-image-coefficient-les-candidate,def:weil-weight-exact-spectrum-decomposition-certificate

Lean: Weil.Weight.DirectImage.CoefficientDevissageCertificate

A coefficient certificate supplies every zero composite, image–kernel identity, Frobenius square, spectrum subquotient certificate, and direct-summand retraction used in Deligne step (a), printed p. 204 (PDF p. 69).

Open obligation: OBL-WGT-DI-COEFFICIENT

Lemma 7.8.40 (Coefficient devissage preserves the target upper bound). *Uses: def:weil-direct-image-coefficient-devissage-certificate, def:weil-direct-image-upper-weight-predicate*

Lean: Weil.Weight.DirectImage.coefficientDevissage

If the adjacent coefficient terms satisfy the target upper bounds, then so does the middle term; a Frobenius-equivariant direct summand inherits the bound.

Proof. Uses: def:weil-direct-image-coefficient-devissage-certificate,lemma:weil-weight-extension-at-most,lemma:weil-weight-subquotient-at-most

At each degree, exactness identifies the middle spectrum as an extension of a quotient of one adjacent spectrum by a subobject of the other. Apply the subquotient upper-bound lemma twice and then extension closure. For a direct summand, apply the stored spectrum submultiset certificate. □

Definition 7.8.41 (Source open–closed localization candidate). Uses: def:weil-direct-image-weight-input-data,def:weil-weight-geometric-operations-typed-context

Lean: Weil.Weight.DirectImage.SourceLocalizationCandidate

For $j : U \hookrightarrow X$ open with closed complement $i : Z \hookrightarrow X$, define the proposed short exact sequence

$$0 \to j_! j^* \mathcal{F} \to \mathcal{F} \to i_* i^* \mathcal{F} \to 0$$

and its three structural maps.

Definition 7.8.42 (Source localization certificate). Uses: def:weil-direct-image-source-localization-candidate

Lean: Weil.Weight.DirectImage.SourceLocalizationCertificate

A source certificate supplies exactness, functoriality, Frobenius equivariance, the identities $f_! j_! = (fj)_!$ and $f_! i_* = (fi)_!$, and all associativity maps. It expands Deligne step (b), printed p. 204 (PDF p. 69).

Open obligation: OBL-WGT-DI-SOURCE

Lemma 7.8.43 (Source devissage preserves the target upper bound). *Uses: def:weil-direct-image-source-localization-certificate, def:weil-direct-image-upper-weight-predicate*

Lean: Weil.Weight.DirectImage.sourceDevissage

If the target upper bound holds for the open and closed restrictions, then it holds for $\mathcal{F}$.

Proof. Uses: def:weil-direct-image-source-localization-certificate,lemma:weil-direct-image-coefficient-devissage

Apply coefficient devissage to the certified open–closed short exact sequence and rewrite its two exceptional direct images using the stored composition identities. □

Definition 7.8.44 (Target open–closed base-change candidate). Uses: def:weil-direct-image-weight-input-data

Lean: Weil.Weight.DirectImage.TargetBaseChangeCandidate

For $j : V \hookrightarrow Y$ open with closed complement $i : T \hookrightarrow Y$, define the two exchange candidates comparing pullback of $Rf_!\mathcal{F}$ with exceptional direct image after the open and closed base changes.

Definition 7.8.45 (Target base-change certificate). Uses: def:weil-direct-image-target-base-change-candidate

Lean: Weil.Weight.DirectImage.TargetBaseChangeCertificate

A target certificate supplies inverse exchange maps, all four inverse equations, geometric-Frobenius equivariance, and constructible gluing of the two local weight filtrations. It expands Deligne step (c), printed p. 204 (PDF p. 69).

Open obligation: OBL-WGT-DI-TARGET

Lemma 7.8.46 (Target devissage detects the upper bound). *Uses:* *def:weil-direct-image-target-base-change-certificate, def:weil-direct-image-upper-weight-predicate*

Lean: *Weil.Weight.DirectImage.targetDevissage*

If the base-changed sheaves over V and T satisfy the target upper bound, then $R^i f_! \mathcal{F}$ satisfies it on Y.

Proof. Uses: def:weil-direct-image-target-base-change-certificate

Transport the two weight filtrations through the exchange isomorphisms. The gluing field of the certificate supplies a filtration on Y; its graded pieces restrict to pure pieces of the same allowed weights on the open and closed parts. This is the target upper-weight predicate. □

Definition 7.8.47 (Leray page candidate for exceptional composition). Uses: def:weil-direct-image-cohomology-candidate

Lean: Weil.Weight.DirectImage.LerayPageCandidate

For $X \xrightarrow{h} Z \xrightarrow{g} Y$, define the proposed cohomological spectral-sequence pages with

$$E_2^{a,b} := R^a g_! \, R^b h_! \mathcal{F}$$

and proposed differentials $d_r : E_r^{a,b} \to E_r^{a+r,b-r+1}$.

Definition 7.8.48 (Leray differential-square predicate). Uses: def:weil-direct-image-leray-page-candidate

Lean: Weil.Weight.DirectImage.IsLerayDifferential

Define the differential predicate by $d_r^2 = 0$ and by identifying $E_{r+1}^{a,b}$ with the homology of d_r at $E_r^{a,b}$.

Definition 7.8.49 (Leray boundedness predicate). Uses: def:weil-direct-image-leray-page-candidate

Lean: Weil.Weight.DirectImage.IsLerayBounded

Define boundedness by a finite rectangle outside which every page term vanishes.

Definition 7.8.50 (Leray convergence-filtration candidate). Uses: def:weil-direct-image-leray-page-candidate

Lean: Weil.Weight.DirectImage.LerayConvergenceCandidate

For every total degree i, define a finite filtration candidate on $R^i(gh)_!\mathcal{F}$ and proposed identifications of its graded pieces with $E_\infty^{a,i-a}$.

Definition 7.8.51 (Leray Frobenius-equivariance predicate). Uses: def:weil-direct-image-leray-page-candidate, def:weil-direct-image-leray-convergence-candidate

Lean: Weil.Weight.DirectImage.IsLerayFrobeniusEquivariant

Define Frobenius equivariance by requiring every differential, page transition, convergence filtration map, and graded comparison to commute with geometric Frobenius.

Definition 7.8.52 (Leray convergence certificate). Uses: def:weil-direct-image-leray-page-candidate, def:weil-direct-image-leray-differential-predicate, def:weil-direct-image-leray-boundedness-predicate, def:weil-direct-image-leray-convergence-candidate, def:weil-direct-image-leray-frobenius-predicate

Lean: Weil.Weight.DirectImage.LerayConvergenceCertificate

A Leray certificate has named fields `page`, `differential`, `boundedness`, `convergence`, and `frobenius`, where every predicate is instantiated on the literal `page` field and `convergence` is the selected convergence candidate on that page. Its `compositionComparison` field supplies forward and reverse maps $R(gh)_! \rightleftarrows Rg_!Rh_!$ and both inverse equations. It expands Deligne step (d), printed p. 204 (PDF p. 69).

Open obligation: OBL-WGT-DI-LERAY

Lemma 7.8.53 (Composition devissage preserves upper weights). *Uses:* *def:weil-direct-image-leray-convergence-certificate, def:weil-direct-image-upper-weight-predicate*

Lean: *Weil.Weight.DirectImage.compositionDevissage*

If every $R^a g_! R^b h_! \mathcal{F}$ has weights at most $n + a + b$, then $R^i(gh)_!\mathcal{F}$ has weights at most $n + i$.

Proof. Uses: def:weil-direct-image-leray-convergence-certificate, lemma:weil-weight-subquotient-at-most, lemma:weil-weight-extension-at-most

Every $E_\infty^{a,b}$ is a subquotient of $E_2^{a,b}$, so its weights are at most $n + a + b$. When $a + b = i$, this is $n + i$. The finite convergence filtration builds the abutment from these graded pieces. Repeated extension closure gives the same bound on the abutment. □

Definition 7.8.54 (Universal-homeomorphism pullback candidate)**.** Uses: `def:weil-direct-image-weight-input-data`

Lean: `Weil.Weight.DirectImage.UniversalHomeomorphismCandidate`

For a universal homeomorphism $u : Y' \to Y$, define pullback and pushforward functor candidates on constructible etale sheaves and unit and counit comparison candidates.

Definition 7.8.55 (Universal-homeomorphism equivalence certificate)**.** Uses: `def:weil-direct-image-universal-homeomorphism-candidate`

Lean: `Weil.Weight.DirectImage.UniversalHomeomorphismCertificate`

An equivalence certificate supplies both triangle identities, equivalence on etale topoi, compatibility with $Rf_!$, stalk comparisons, and geometric-Frobenius equivariance. It expands Deligne step (e), printed p. 204 (PDF p. 69).

Open obligation: `OBL-WGT-DI-UNIVERSAL-HOMEO`

Lemma 7.8.56 (Universal-homeomorphism invariance of upper weights)**.** *Uses:* *`def:weil-direct-image-universal-homeomorphism-certificate`, `def:weil-direct-image-upper-weight-predicate`*

Lean: *`Weil.Weight.DirectImage.universalHomeomorphism_invariant`*

The target upper bound holds before the universal-homeomorphism base change exactly when it holds after that base change.

Proof. Uses: `def:weil-direct-image-universal-homeomorphism-certificate`

The certificate identifies corresponding stalk spectra and their geometric Frobenius actions. The pointwise modulus conditions and every graded piece of a mixedness filtration are therefore identical under the equivalence. Use the unit and counit equations for the two implications. □

Definition 7.8.57 (Relative-dimension-zero factorization candidate)**.** Uses: `def:weil-direct-image-weight-input-data`

Lean: `Weil.Weight.DirectImage.RelativeZeroFactorizationCandidate`

For a quasi-finite separated f, a dimension-zero factorization candidate consists of an open immersion $j : X \hookrightarrow \bar{X}$, a finite morphism $\bar{f} : \bar{X} \to Y$, and the equation $f = \bar{f} \circ j$. This is the Zariski-main construction required in Deligne step (f).

Definition 7.8.58 (Relative-zero exactness certificate)**.** Uses: `def:weil-direct-image-relative-zero-factorization-candidate`

Lean: `Weil.Weight.DirectImage.RelativeZeroExactnessCertificate`

A relative-zero certificate supplies the open and finite morphism properties, exactness of $j_!$ and $\bar{f}_*$, vanishing of all positive higher direct images, residue-field decomposition of each stalk, and the formula identifying local geometric Frobenius powers.

Open obligation: `OBL-WGT-DI-RELATIVE-ZERO`

Lemma 7.8.59 (Relative dimension zero preserves pointwise weights)**.** *Uses:* *`def:weil-direct-image-relative-zero-exactness-certificate`, `def:weil-direct-image-upper-weight-predicate`*

Lean: *`Weil.Weight.DirectImage.relativeZero`*

For relative dimension zero, $R^i f_! \mathcal{F} = 0$ when $i \neq 0$, and $f_! \mathcal{F}$ preserves the pointwise upper weight bound.

Proof. Uses: `def:weil-direct-image-relative-zero-exactness-certificate`

The vanishing field gives the assertion for $i \neq 0$. In degree zero, each target stalk is a finite direct sum of source stalks over residue-field extensions. The Frobenius-power formula preserves the normalized weight: an eigenvalue of weight w for $q^{e'}$ has the same w after it is viewed at degree e'. Finite direct sums preserve the upper bound. □

Definition 7.8.60 (Geometric-complexity diagnostic)**.** Uses: `def:weil-direct-image-weight-input-data`, `def:weil-direct-image-dimension-bound-certificate`

Lean: `Weil.Weight.DirectImage.geometricComplexity`

Define the certified geometric complexity

$$\mu(f) := (b_{\mathrm{rel}}, b_{\mathrm{src}}) \in \mathbf{N} \times \mathbf{N}.$$

It is ordered lexicographically, but it is not the recursion argument for coefficient, source, target, or universal-homeomorphism devissage. It is used only to certify edges explicitly tagged as geometric dimension reduction.

Definition 7.8.61 (Relative-dimension branch classifier)**.** Uses: `def:weil-direct-image-dimension-bound-certificate`

Lean: `Weil.Weight.DirectImage.relativeDimensionBranch`

Classify a certified input as zero when $b_{\mathrm{rel}} = 0$, as curve when $b_{\mathrm{rel}} = 1$, and as higher when $2 \leq b_{\mathrm{rel}}$. These three constructors are exhaustive and disjoint by decidable comparison in $\mathbf{N}$; no comparison of a pair with the natural number 1 occurs.

Definition 7.8.62 (Direct-image reduction branch tag)**.** Uses: `def:weil-direct-image-relative-dimension-branch-classifier`

Lean: `Weil.Weight.DirectImage.ReductionBranchTag`

Define the finite branch type with constructors

coefficient, source, target, composition, universalHomeomorphism, geometricReduction, zeroLeaf, curveLeaf.

Definition 7.8.63 (Reduction-edge kind)**.** Uses: `def:weil-direct-image-reduction-branch-tag`

Lean: `Weil.Weight.DirectImage.ReductionEdgeKind`

Define two edge kinds. A transport edge is a finite coefficient/support/base-change devissage edge and need not change geometric dimension. A geometric edge is produced by a genuine dimension-reduction factorization and must carry a strict complexity proof.

Definition 7.8.64 (Dependent reduction-node data)**.** Uses: `def:weil-direct-image-weight-input-data`, `def:weil-direct-image-reduction-branch-tag`, `def:weil-direct-image-reduction-edge-kind`

Lean: `Weil.Weight.DirectImage.ReductionNodeData`

A node datum contains a parent direct-image input D_v, a branch tag, a finite child index type I_v, a dependent family of child inputs $D_{v,a}$ for $a \in I_v$, an edge kind for every child, and parent and child cohomological degrees and weight indices. It contains no branch-specific factorization or comparison map. Those maps are fields of the corresponding branch candidate, so data from one tag cannot be used at another tag. The child dimension-bound certificate is a field of $D_{v,a}$; it is not reused from the parent by definitional equality.

Definition 7.8.65 (Geometric-child decrease predicate)**.** Uses: `def:weil-direct-image-reduction-node-data`, `def:weil-direct-image-induction-measure`

Lean: `Weil.Weight.DirectImage.IsGeometricChildDecrease`

For every child tagged geometric, require $\mu(D_{v,a}) <_{\mathrm{lex}} \mu(D_v)$. For a transport edge the predicate is True. Consequently no same-dimension coefficient or base-change step is forced to satisfy a false geometric inequality.

Definition 7.8.66 (Geometric-reduction factorization candidate)**.** Uses: `def:weil-direct-image-reduction-node-data`, `def:weil-direct-image-source-localization-candidate`, `def:weil-direct-image-leray-page-candidate`, `def:weil-direct-image-leray-convergence-candidate`

Lean: `Weil.Weight.DirectImage.GeometricReductionCandidate`

For a node proposed to have tag geometricReduction, select a dense-open candidate $j : U \hookrightarrow X$, its closed-complement candidate $i : Z \hookrightarrow X$, an intermediate scheme M, and morphism candidates

$$U \xrightarrow{h} M \xrightarrow{g} Y.$$

Select two distinct child indices $a_{\mathrm{open}}, a_{\mathrm{closed}}$ and an equivalence $I_v \simeq \{\mathsf{open}, \mathsf{closed}\}$. The open child input is typed over the composite $g \circ h$ with coefficient $j^*\mathcal{F}$; the closed child input is typed over $f \circ i$ with coefficient $i^*\mathcal{F}$. Also select the source-localization candidate for (j, i), a named Leray-page field `lerayPage` attached to (h, g), and a named convergence field `lerayConvergence` proposed for the root of the open child tree, together with forward and reverse cohomology comparisons between $(f \circ j)_!$ and $(g \circ h)_!$, and forward and reverse comparison maps on every proposed filtration step and graded piece. No equality or inverse property is included.

Definition 7.8.67 (Geometric-reduction factorization predicate)**.** Uses: `def:weil-direct-image-geometric-reduction-candidate`, `def:weil-direct-image-reduction-child-measure-predicate`

Lean: `Weil.Weight.DirectImage.IsGeometricReduction`

The geometric-reduction predicate is the conjunction of the following named fields.

(i) `tag`: the node tag is geometricReduction.

(ii) `factorization`: j is an open immersion with dense image, i is the scheme-theoretic closed complement, their images are disjoint and cover X, and the literal morphism equality $f \circ j = g \circ h$ holds with all domains and codomains displayed above.

(iii) `children`: the two selected child indices are exhaustive; the corresponding dependent child inputs are definitionally the typed open composite and closed-complement inputs, with the displayed coefficient sheaves, degrees, weight indices, and dimension-bound certificates. The open edge is a transport edge and the closed edge is a geometric edge satisfying the stored decrease predicate. Decrease fields for descendants of the open composition child belong to those descendant node certificates, not to this node-local predicate.

(iv) `lerayTyping`: the page underlying `lerayConvergence` is definitionally the field `lerayPage`; all its schemes, arrows, coefficient objects, degrees, and Frobenius operators are the selected $(h, g, j^*\mathcal{F})$ data.

(v) `comparisonInverse`: the two cohomology comparison maps and each pair of filtration and graded comparison maps are mutual inverses.

(vi) `frobenius`: every one of those maps commutes with the selected geometric Frobenius at every closed point.

(vii) `filtration`: the comparisons preserve the filtration maps, quotient maps, declared graded indices, and the spectra with multiplicity.

In particular, the morphism equality and child-input identifications are proof fields, not proposed maps inherited from raw node data.

Definition 7.8.68 (Geometric-reduction factorization certificate)**.** Uses: `def:weil-direct-image-geometric-reduction-candidate`, `def:weil-direct-image-geometric-reduction-predicate`, `def:weil-direct-image-source-localization-certificate`

Lean: `Weil.Weight.DirectImage.GeometricReductionCertificate`

A geometric-reduction certificate consists of one factorization candidate, a proof of its complete predicate, and a source-localization certificate whose underlying localization candidate is definitionally `candidate.localization`. In particular, its j, i, coefficient sheaves, localization injection and quotient, and all triangle or exact-sequence maps are the corresponding localization fields. The factorization cohomology and filtration comparisons remain separate named fields of the geometric candidate and are not identified with localization triangle maps. The validity certificate of the surrounding reduction tree must identify the root tag of the designated open child with composition; that child node, rather than this certificate, stores the matching Leray convergence certificate. The geometric certificate contains no child or parent upper-weight conclusion.

Open obligation: `OBL-WGT-DI-GEOMETRIC-REDUCTION`

Definition 7.8.69 (Finite dependent reduction-tree candidate)**.** Uses: `def:weil-direct-image-reduction-node-data`

Lean: `Weil.Weight.DirectImage.ReductionTreeCandidate`

Define an inductive finite rooted tree: a node consists of node data and, for every $a \in I_v$, a child tree whose root is proposed to carry $D_{v,a}$. Finiteness is structural; it is not deduced from the two-dimensional geometric complexity.

Definition 7.8.70 (Relative-curve reduction candidate)**.** Uses: `def:weil-direct-image-reduction-tree-candidate`, `def:weil-direct-image-relative-dimension-branch-classifier`

Lean: `Weil.Weight.DirectImage.RelativeCurveReductionCandidate`

For an input classified as higher, define a selected finite reduction-tree candidate whose proposed terminal vertices are zero or curve leaves. Dense-open restrictions and factorizations occur as explicitly tagged internal nodes with dependent child inputs.

7.8.1 The ten components of the relative-curve tame reduction

Definition 7.8.71 (TAME-1 smooth-locus candidate)**.** Uses: `def:weil-direct-image-relative-curve-reduction-candidate`

Lean: `Weil.Weight.TAME.SmoothLocusCandidate`

At a generic point η of the target, define an open subcurve $C^\circ \subset X_\eta$ and the proposed smoothness and density data.

Definition 7.8.72 (TAME-2 finite curve-cover candidate)**.** Uses: `def:weil-tame-smooth-locus-candidate`

Lean: `Weil.Weight.TAME.FiniteCoverCandidate`

Define a finite surjective cover candidate $u : D^\circ \to C^\circ$, together with a smooth projective compactification D of D°.

Definition 7.8.73 (TAME-3 finite etale boundary candidate)**.** Uses: `def:weil-tame-finite-cover-candidate`

Lean: `Weil.Weight.TAME.FiniteEtaleBoundaryCandidate`

Define the boundary candidate $E = D \setminus D^\circ$, a finite purely inseparable base extension candidate, and the proposed finite-etale map from E to the extended generic base.

Definition 7.8.74 (TAME-4 tame pullback predicate)**.** Uses: `def:weil-tame-finite-cover-candidate,def:weil-tame-finite-etale-boundary-candidate`

Lean: `Weil.Weight.TAME.IsPullbackTame`

Define tameness by requiring every wild inertia subgroup along E to act trivially on $u^*\mathcal{F}$.

Definition 7.8.75 (TAME-5 trace-retraction candidate)**.** Uses: `def:weil-tame-finite-cover-candidate`

Lean: `Weil.Weight.TAME.TraceRetractionCandidate`

Define the adjunction map $\mathcal{F} \to u_* u^* \mathcal{F}$, the trace map in the reverse direction, and the degree-normalized proposed retraction.

Definition 7.8.76 (TAME-5 trace-retraction certificate)**.** Uses: `def:weil-tame-trace-retraction-candidate`

Lean: `Weil.Weight.TAME.TraceRetractionCertificate`

A trace-retraction certificate supplies the trace–unit composite formula, invertibility of the cover degree in the coefficient field, the retraction equation, Frobenius equivariance, and the resulting spectrum subquotient certificate.

Open obligation: `OBL-WGT-TAME-TRACE-RETRACTION`

Definition 7.8.77 (TAME-6 common spreading-stage candidate)**.** Uses: `def:weil-tame-smooth-locus-candidate,def:weil-tame-finite-cover-candidate,def:weil-tame-finite-etale-boundary-candidate,def:weil-tame-trace-retraction-candidate`

Lean: `Weil.Weight.TAME.CommonSpreadingStageCandidate`

A common spreading-stage candidate is one dense open $V \subseteq Y$ and models over V of every generic object, morphism, inverse equation, smoothness datum, projectivity datum, finite-etale boundary, tame action, and trace retraction.

Definition 7.8.78 (TAME-6 spreading compatibility predicate)**.** Uses: `def:weil-tame-common-spreading-stage-candidate`

Lean: `Weil.Weight.TAME.IsSpreadingCompatible`

Define compatibility by requiring generic-fiber recovery of every model, preservation of all displayed composites and identities, and compatibility under further restriction of V.

Definition 7.8.79 (TAME-7 radicial comparison candidate)**.** Uses: `def:weil-tame-finite-etale-boundary-candidate,def:weil-direct-image-universal-homeomorphism-candidate`

Lean: `Weil.Weight.TAME.RadicialComparisonCandidate`

Define unit and counit comparisons across the finite purely inseparable base extension for the sheaf, the cover, the boundary, and every exceptional direct image.

Definition 7.8.80 (TAME-7 radicial invariance certificate)**.** Uses: `def:weil-tame-radicial-comparison-candidate,def:weil-direct-image-universal-homeomorphism-certificate`

Lean: `Weil.Weight.TAME.RadicialInvarianceCertificate`

A radicial certificate supplies all inverse equations and identifies the geometric-Frobenius spectra before and after the extension.

Open obligation: `OBL-WGT-TAME-RADICIAL`

Definition 7.8.81 (TAME-8 relative monodromy-filtration candidate)**.** Uses: `def:weil-tame-common-spreading-stage-candidate,def:weil-mon-filtration-candidate`

Lean: `Weil.Weight.TAME.RelativeMonodromyFiltrationCandidate`

Along the finite-etale relative boundary, define subobject candidates M_j of the relative local representation and relative versions of every monodromy-power map.

Definition 7.8.82 (TAME-8 relative monodromy certificate). Uses: `def:weil-tame-relative-monodromy-filtration-candidate`, `def:weil-mon-filtration-certificate`, `def:weil-mon-graded-purity-predicate`

Lean: `Weil.Weight.TAME.RelativeMonodromyCertificate`

A relative certificate supplies all filtration properties over the base, graded-purity predicates, and constructibility of every graded object.

Open obligation: `OBL-WGT-TAME-RELATIVE-MONODROMY`

Definition 7.8.83 (TAME-9 fiber comparison candidate). Uses: `def:weil-tame-relative-monodromy-filtration-candidate`, `def:weil-direct-image-cohomology-candidate`

Lean: `Weil.Weight.TAME.FiberComparisonCandidate`

For every geometric point $\bar{y} \to V$, define base-change maps from the relative filtration and $R^i f_!$ to the corresponding constructions on the fiber, together with proposed inverses.

Definition 7.8.84 (TAME-9 fiber compatibility certificate). Uses: `def:weil-tame-fiber-comparison-candidate`, `def:weil-tame-relative-monodromy-certificate`

Lean: `Weil.Weight.TAME.FiberCompatibilityCertificate`

A fiber certificate supplies all inverse equations, preservation of each filtration step and graded piece, Frobenius equivariance, and compatibility with the boundary exact sequence.

Open obligation: `OBL-WGT-TAME-FIBERS`

Definition 7.8.85 (TAME-10 descent datum). Uses: `def:weil-tame-trace-retraction-certificate`, `def:weil-tame-radicial-invariance-certificate`, `def:weil-tame-fiber-compatibility-certificate`

Lean: `Weil.Weight.TAME.WeightDescentData`

A weight-descent datum consists of all spectrum inclusions and equalities needed to transfer an upper bound through the cover, finite pushforward, direct summand, radicial comparison, generic open, and closed complement.

Definition 7.8.86 (TAME-10 descent certificate). Uses: `def:weil-tame-weight-descent-data`, `def:weil-weight-subquotient-spectrum-certificate`, `def:weil-weight-exact-spectrum-decomposition-certificate`

Lean: `Weil.Weight.TAME.WeightDescentCertificate`

A descent certificate supplies the direction of every spectral transfer, its multiplicity inequality or equality, and its compatibility with the mixedness filtrations.

Open obligation: `OBL-WGT-TAME-DESCENT`

Definition 7.8.87 (Complete TAME certificate). Uses: `def:weil-tame-smooth-locus-candidate`, `def:weil-tame-finite-cover-candidate`, `def:weil-tame-finite-etale-boundary-candidate`, `def:weil-tame-pullback-predicate`, `def:weil-tame-trace-retraction-certificate`, `def:weil-tame-spreading-compatibility-predicate`, `def:weil-tame-radicial-invariance-certificate`, `def:weil-tame-relative-monodromy-certificate`, `def:weil-tame-fiber-compatibility-certificate`, `def:weil-tame-weight-descent-certificate`

Lean: `Weil.Weight.TAME.CompleteCertificate`

A complete TAME certificate is the ordered tuple of the ten preceding components. Deligne's proof of Theorem (3.3.1), printed p. 205 (PDF p. 70), states this reduction in prose and provides no single theorem that constructs this tuple.

Definition 7.8.88 (Relative compactification localization candidate). Uses: `def:weil-tame-complete-certificate`

Lean: `Weil.Weight.DirectImage.RelativeLocalizationCandidate`

For the smooth projective relative curve $\bar{f} : \bar{X} \to Y$, open $j : X \hookrightarrow \bar{X}$, and finite-etale boundary $i : D \hookrightarrow \bar{X}$, define the proposed exact sequence

$$0 \to j_! \mathcal{F} \to j_* \mathcal{F} \to i_* i^* j_* \mathcal{F} \to 0$$

and the filtration induced on the boundary term.

Definition 7.8.89 (Fiberwise projective-curve input family). Uses: `def:weil-direct-image-relative-localization-candidate`, `def:weil-curve-projective-input-data`, `def:weil-curve-reverse-reduction-certificate`, `def:weil-curve-dual-reverse-reduction-certificate`

Lean: `Weil.Weight.DirectImage.RelativeCurveFiberFamily`

For every geometric point $\bar{y} \to Y$, define a projective-curve input datum for the fiber, a primal and a dual reverse-reduction certificate, the fiber Frobenius, and comparison candidates from the fiber cohomology to the stalk of the relative direct image. All four items are indexed by the same $\bar{y}$, degree, coefficient field, and realization.

Definition 7.8.90 (Fiberwise projective comparison certificate)**.** Uses: `def:weil-direct-image-relative-curve-fiber-family`,`def:weil-tame-fiber-compatibility-certificate`

Lean: `Weil.Weight.DirectImage.RelativeCurveFiberCertificate`

A fiber certificate supplies inverse base-change maps, equality of Frobenius actions and spectra with multiplicity, compatibility of the primal and dual pairings, and the boundary-filtration comparison for every geometric point.

Open obligation: `OBL-WGT-DI-RELATIVE-CURVE-FIBERS`

Definition 7.8.91 (Relative localization certificate)**.** Uses: `def:weil-direct-image-relative-localization-candidate`,`def:weil-direct-image-relative-curve-fiber-certificate`

Lean: `Weil.Weight.DirectImage.RelativeLocalizationCertificate`

A relative localization certificate has a named field `fiberCertificate` containing a fiberwise projective comparison certificate for the same typed fiber family. Its remaining fields supply exactness of the relative $j_!$-to-j_* sequence, base change to every indexed fiber, Frobenius equivariance, and the monodromy graded comparison for the finite boundary. Thus every later use of projective-curve purity has the literal projection path `localization.fiberCertificate`; no separately chosen fiber datum is silently substituted.

Open obligation: `OBL-WGT-DI-RELATIVE-CURVE-LOCALIZATION`

Lemma 7.8.92 (Fiberwise interior purity)**.** *Uses:* *`def:weil-direct-image-relative-curve-fiber-certificate`, `def:weil-weight-pure-spectrum-predicate`*

Lean: *`Weil.Weight.DirectImage.relativeCurveFiber_interiorPure`*

For every indexed geometric fiber, the interior projective-curve cohomology spectrum is pure in the required degree-shifted weight.

Proof. Uses: `thm:weil-curve-projective-purity`,`def:weil-direct-image-relative-curve-fiber-family`,`def:weil-direct-image-relative-curve-fiber-certificate`

At the chosen fiber and degree, instantiate the projective-curve theorem with the primal and dual certificates stored at exactly that index, then transport the spectrum through the certified base-change bijection. □

Lemma 7.8.93 (Relative-curve boundary upper bound)**.** *Uses:* *`def:weil-direct-image-relative-localization-certificate`, `def:weil-direct-image-upper-weight-predicate`*

Lean: *`Weil.Weight.DirectImage.relativeCurveBoundary_upper`*

The finite boundary term in the relative localization sequence has the required upper mixedness bound.

Proof. Uses: `prop:weil-mon-graded-purity`,`lemma:weil-direct-image-relative-zero`

Apply graded monodromy purity to each stored boundary graded piece, then apply the relative-zero lemma to its finite-etale pushforward. □

Lemma 7.8.94 (Relative localization assembles the covered upper bound)**.** *Uses:* *`def:weil-direct-image-relative-localization-certificate`, `def:weil-direct-image-upper-weight-predicate`*

Lean: *`Weil.Weight.DirectImage.relativeLocalization_upper`*

The interior fiberwise bound and the finite-boundary bound give the upper bound on the covered relative-curve model.

Proof. Uses: `lemma:weil-direct-image-relative-curve-fiber-interior-pure`,`lemma:weil-direct-image-relative-curve-boundary-upper`,`lemma:weil-direct-image-coefficient-devissage`

Instantiate coefficient devissage on the certified relative localization sequence with the two preceding term bounds. □

Proposition 7.8.95 (Conditional relative-curve upper bound)**.** *Uses:* *`def:weil-direct-image-relative-localization-certificate`, `def:weil-tame-complete-certificate`, `def:weil-direct-image-upper-weight-predicate`*

Lean: *`Weil.Weight.DirectImage.relativeCurve_upperWeight`*

For the certified relative-curve model, the supplied upper mixedness datum on every $R^i f_! \mathcal{F}$ satisfies the bound $n+i$.

Proof. Uses: `lemma:weil-direct-image-relative-localization-upper`,`def:weil-tame-weight-descent-certificate`

Apply the TAME descent spectrum transfer once to the covered-model bound of the preceding lemma. □

Definition 7.8.96 (Relative-curve leaf terminal certificate)**.** Uses: `def:weil-direct-image-reduction-node-data`, `def:weil-tame-complete-certificate`, `def:weil-direct-image-relative-localization-certificate`

Lean: `Weil.Weight.DirectImage.CurveLeafTerminalCertificate`

For a node tagged curveLeaf, a terminal certificate supplies: the proof that its child index type is empty; the curve branch-classifier equation; a complete TAME certificate on the node's parent input; and a relative-localization certificate on that same input. It selects no second fiber certificate: the only fiber certificate is the projection `localization.fiberCertificate`. Parent input, degree, coefficient field, Frobenius realization, and filtration candidate are required to be literally identical along this projection path. No upper-weight conclusion is a field.

Definition 7.8.97 (Reduction-node certificate)**.** Uses: `def:weil-direct-image-reduction-node-data`, `def:weil-direct-image-reduction-child-measure-predicate`, `def:weil-direct-image-coefficient-devissage-certificate`, `def:weil-direct-image-source-localization-certificate`, `def:weil-direct-image-target-base-change-certificate`, `def:weil-direct-image-leray-convergence-certificate`, `def:weil-direct-image-universal-homeomorphism-certificate`, `def:weil-direct-image-geometric-reduction-certificate`, `def:weil-direct-image-relative-zero-exactness-certificate`, `def:weil-direct-image-curve-leaf-terminal-certificate`

Lean: `Weil.Weight.DirectImage.ReductionNodeCertificate`

A node certificate is a dependent branch record. After matching on the stored tag, its branch field has exactly one of the following types: coefficient devissage, source localization, target base change, Leray convergence, universal-homeomorphism equivalence, geometric-reduction factorization, relative-zero terminal, or relative-curve terminal. The geometric branch therefore stores a `GeometricReductionCertificate`; it cannot call a comparison map from raw node data. The record also supplies the dependent parent–child input equalities, degree and weight-index equations, branch exhaustiveness, and geometric-child decrease proof. No parent upper-weight conclusion is a field.

Definition 7.8.98 (Reduction-tree validity certificate)**.** Uses: `def:weil-direct-image-reduction-tree-candidate`, `def:weil-direct-image-reduction-node-certificate`

Lean: `Weil.Weight.DirectImage.ReductionTreeValidityCertificate`

A validity certificate supplies the root input equality, a node certificate at every vertex, equality between every declared child input and the root input of its child tree, absence of dangling children, empty child types at leaves, and tag/classifier compatibility at zero and curve leaves. At a geometric-reduction vertex it additionally supplies the literal input equality

$$\texttt{openChild.rootInput} = D(g \circ h, j^{*}\mathcal{F}),$$

identifies the designated open child root tag with composition, and writes L_{open} for that child's named Leray branch and G_v for the geometric candidate. It then supplies the two precisely typed field equalities

$$L_{\mathrm{open}}.\texttt{page} = G_v.\texttt{lerayPage}, \qquad L_{\mathrm{open}}.\texttt{convergence} = G_v.\texttt{lerayConvergence}.$$

It identifies the other child root input with the certified closed complement. Thus neither side is the whole geometric candidate; L_{open} is definitionally the projection through the open child's node certificate and its Leray branch.

Definition 7.8.99 (Relative-curve termination certificate)**.** Uses: `def:weil-direct-image-relative-curve-reduction-candidate`, `def:weil-direct-image-reduction-tree-validity-certificate`

Lean: `Weil.Weight.DirectImage.RelativeCurveTerminationCertificate`

A termination certificate is a validity certificate for the selected finite tree, including terminal-classifier proofs and every geometric-edge decrease proof. Termination itself follows from structural finiteness of the supplied tree; constructing this complete tree remains open.

Open obligation: `OBL-WGT-DI-TERMINATION`

Definition 7.8.100 (Complete certified direct-image reduction tree)**.** Uses: `def:weil-direct-image-weight-input-data`, `def:weil-direct-image-reduction-tree-candidate`, `def:weil-direct-image-reduction-tree-validity-certificate`, `def:weil-direct-image-relative-curve-termination-certificate`, `def:weil-direct-image-curve-leaf-terminal-certificate`

Lean: `Weil.Weight.DirectImage.CompleteAntecedentCertificate`

A complete antecedent certificate contains a selected finite reduction tree whose root is definitionally identified with the requested input and its full validity certificate. Each internal node certificate contains its own branch-specific devissage certificate and dependent child inputs. At a curve leaf the fiber certificate is not a second root-level choice: it is obtained only by `validity.node.branch.curveTerminal.localization.fiberCertificate`. There is no flat root-level list whose children lack antecedent certificates, and no final upper-weight field.

Definition 7.8.101 (Geometric open-child transport candidate)**.** Uses: `def:weil-direct-image-geometric-reduction-certificate`,`def:weil-direct-image-upper-mixed-certificate-data`

Lean: `Weil.Weight.DirectImage.geometricOpenTransportCandidate`

Given the upper mixedness candidate returned by the designated open composition child, define the transported candidate's underlying sheaf to be literally the open-term cohomology candidate and take its two underlying comparison maps to be identities. Copy the child's filtration-step objects, graded objects, inclusion maps, quotient maps, and declared indices definitionally; only the terminal filtration map and the Frobenius actions are conjugated by the forward and reverse factorization comparisons. No validity property is asserted by this Definition.

Definition 7.8.102 (Tag-specific node output candidate)**.** Uses: `def:weil-direct-image-reduction-node-data`,`def:weil-direct-image-reduction-node-certificate`,`def:weil-direct-image-geometric-open-transport-candidate`,`def:weil-direct-image-curve-leaf-terminal-certificate`,`def:weil-direct-image-upper-mixed-certificate-data`

Lean: `Weil.Weight.DirectImage.nodeOutputCandidate`

Given a node and a dependent family of already constructed child upper mixedness data, define the parent candidate by cases on the branch tag: extension filtration for coefficient and source, glued filtration for target, Leray filtration for composition, transported filtration for universalHomeomorphism, the source-localization extension of the geometric open-transport candidate by the designated closed child for geometricReduction, and the stored terminal filtration for a zero or curve leaf. Every case uses only maps reached through the matching dependent branch of the node certificate.

Lemma 7.8.103 (Coefficient node assembly)**.** *Uses:* *def:weil-direct-image-node-output-candidate*, *def:weil-direct-image-upper-weight-predicate*

Lean: *Weil.Weight.DirectImage.coefficientNode_assembles*

At a coefficient node, valid child upper mixedness data make the tag-specific parent output satisfy the upper-weight predicate.

Proof. Uses: `lemma:weil-direct-image-coefficient-devissage`

Instantiate coefficient devissage with the child certificates and the short exact sequence stored at this node. □

Lemma 7.8.104 (Source node assembly)**.** *Uses:* *def:weil-direct-image-node-output-candidate*, *def:weil-direct-image-upper-weight-predicate*

Lean: *Weil.Weight.DirectImage.sourceNode_assembles*

At a source-localization node, valid child data make the parent output satisfy the upper-weight predicate.

Proof. Uses: `lemma:weil-direct-image-source-devissage`

Instantiate the source-devissage lemma with the open and closed child data. □

Lemma 7.8.105 (Target node assembly)**.** *Uses:* *def:weil-direct-image-node-output-candidate*, *def:weil-direct-image-upper-weight-predicate*

Lean: *Weil.Weight.DirectImage.targetNode_assembles*

At a target-localization node, valid child data make the glued parent output satisfy the upper-weight predicate.

Proof. Uses: `lemma:weil-direct-image-target-devissage`

Instantiate target devissage with the open and closed base-change child data. □

Lemma 7.8.106 (Composition node assembly)**.** *Uses:* *def:weil-direct-image-node-output-candidate*, *def:weil-direct-image-upper-weight-predicate*

Lean: *Weil.Weight.DirectImage.compositionNode_assembles*

At a composition node, valid page-term child data make the Leray parent output satisfy the upper-weight predicate.

Proof. Uses: `lemma:weil-direct-image-composition-devissage`

Instantiate composition devissage with the dependent (a, b)-indexed child family stored at this node. □

Lemma 7.8.107 (Universal-homeomorphism node assembly)**.** *Uses:* *def:weil-direct-image-node-output-candidate*, *def:weil-direct-image-upper-weight-predicate*

Lean: *Weil.Weight.DirectImage.universalHomeomorphismNode_assembles*

At a universal-homeomorphism node, the transported child output satisfies the parent upper-weight predicate.

Proof. Uses: `lemma:weil-direct-image-universal-homeomorphism-invariance`
Apply the certified equivalence in the direction selected by the node tag. □

Lemma 7.8.108 (Underlying field of geometric open transport)**.** *Uses:* *def:weil-direct-image-geometric-open-transport-candidate*, *def:weil-direct-image-geometric-reduction-certificate*, *def:weil-direct-image-upper-weight-predicate*

Lean: *Weil.Weight.DirectImage.geometricOpenTransport_underlying*
The transported candidate satisfies the named `underlying` *field.*

Proof. Uses: `def:weil-direct-image-geometric-open-transport-candidate`
Unfold the transported underlying sheaf and its two identity comparisons. The object equality and both inverse equations are reflexive. □

Lemma 7.8.109 (Exactness field of geometric open transport)**.** *Uses:* *def:weil-direct-image-geometric-open-transport-candidate*, *def:weil-direct-image-geometric-reduction-certificate*, *def:weil-direct-image-upper-weight-predicate*

Lean: *Weil.Weight.DirectImage.geometricOpenTransport_exact*
If the open child satisfies the upper-weight predicate, the transported candidate satisfies the named `exact` *field.*

Proof. Uses: `def:weil-direct-image-geometric-open-transport-candidate`, `def:weil-direct-image-upper-weight-predicate`
The filtration-step objects, inclusions, and quotients are copied definitionally. Project the child's `exact` field. □

Lemma 7.8.110 (Graded-purity field of geometric open transport)**.** *Uses:* *def:weil-direct-image-geometric-open-transport-candidate*, *def:weil-direct-image-geometric-reduction-certificate*, *def:weil-direct-image-upper-weight-predicate*

Lean: *Weil.Weight.DirectImage.geometricOpenTransport_gradedPure*
If the open child satisfies the upper-weight predicate, the transported candidate satisfies the named `gradedPure` *field.*

Proof. Uses: `def:weil-direct-image-geometric-open-transport-candidate`, `def:weil-direct-image-upper-weight-predicate`
The graded objects and spectra are copied definitionally. Project the child's `gradedPure` field. □

Lemma 7.8.111 (Index field of geometric open transport)**.** *Uses:* *def:weil-direct-image-geometric-open-transport-candidate*, *def:weil-direct-image-geometric-reduction-certificate*, *def:weil-direct-image-upper-weight-predicate*

Lean: *Weil.Weight.DirectImage.geometricOpenTransport_indexBound*
If the open child satisfies the upper-weight predicate, the transported candidate satisfies the named `indexBound` *field.*

Proof. Uses: `def:weil-direct-image-geometric-open-transport-candidate`, `def:weil-direct-image-upper-weight-predicate`
The declared indices and their nonzero predicates are copied definitionally. Project the child's `indexBound` field. □

Lemma 7.8.112 (Frobenius field of geometric open transport)**.** *Uses:* *def:weil-direct-image-geometric-open-transport-candidate*, *def:weil-direct-image-geometric-reduction-certificate*, *def:weil-direct-image-upper-weight-predicate*

Lean: *Weil.Weight.DirectImage.geometricOpenTransport_frobenius*
If the open child satisfies the upper-weight predicate, the transported candidate satisfies the named `frobenius` *field.*

Proof. Uses: `def:weil-direct-image-geometric-open-transport-candidate`, `def:weil-direct-image-geometric-reduction-predicate`, `def:weil-direct-image-upper-weight-predicate`
For each filtration or graded map, substitute its displayed conjugate definition. Use the child's `frobenius` square and the stored factorization `frobenius` square once; cancel the adjacent forward and reverse maps by `comparisonInverse`. □

Lemma 7.8.113 (Geometric open-child transport preserves validity). *Uses:* *def:weil-direct-image-geometric-open-transport-candidate*, *def:weil-direct-image-geometric-reduction-certificate*, *def:weil-direct-image-upper-weight-predicate*

Lean: *Weil.Weight.DirectImage.geometricOpenTransport_valid*

If the designated open composition child satisfies the upper-weight predicate, then its transported open-term candidate satisfies the same predicate with the indices required by source localization.

Proof. Uses: lemma:weil-direct-image-geometric-open-transport-underlying, lemma:weil-direct-image-geometric-open-transport-exact, lemma:weil-direct-image-geometric-open-transport-graded-pure, lemma:weil-direct-image-geometric-open-transport-index-bound, lemma:weil-direct-image-geometric-open-transport-frobenius

Apply the constructor of the five-field upper-weight predicate and supply, in field order, the five preceding lemmas. □

Lemma 7.8.114 (Geometric source extension assembles). *Uses:* *def:weil-direct-image-node-output-candidate*, *def:weil-direct-image-geometric-reduction-certificate*, *def:weil-direct-image-upper-weight-predicate*

Lean: *Weil.Weight.DirectImage.geometricSourceExtension_assembles*

If the transported open candidate and the designated closed child candidate satisfy their upper-weight predicates, then the geometric node output satisfies the parent predicate.

Proof. Uses: def:weil-direct-image-geometric-reduction-certificate, lemma:weil-direct-image-source-devissage

Instantiate the already proved source-devissage lemma once with the matching source-localization certificate and the two supplied predicate proofs. □

Lemma 7.8.115 (Geometric-reduction node assembly). *Uses:* *def:weil-direct-image-node-output-candidate*, *def:weil-direct-image-geometric-reduction-certificate*, *def:weil-direct-image-upper-weight-predicate*

Lean: *Weil.Weight.DirectImage.geometricReductionNode_assembles*

At a geometric-reduction node, valid open-composition and closed-complement child data make the parent output satisfy the upper-weight predicate.

Proof. Uses: lemma:weil-direct-image-geometric-open-transport-valid, lemma:weil-direct-image-geometric-source-extension-assembles

Apply the transport lemma to the designated open child and use its result as the open hypothesis of the source-extension lemma; its closed hypothesis is the designated closed-child validity proof. The Leray devissage was already performed by the open child's separately certified composition node. □

Lemma 7.8.116 (Relative-zero leaf assembly). *Uses:* *def:weil-direct-image-node-output-candidate*, *def:weil-direct-image-upper-weight-predicate*

Lean: *Weil.Weight.DirectImage.zeroLeaf_assembles*

The output at a zero leaf satisfies the upper-weight predicate.

Proof. Uses: lemma:weil-direct-image-relative-zero

Instantiate the relative-zero lemma with the terminal certificate stored at the leaf. □

Lemma 7.8.117 (Relative-curve leaf assembly). *Uses:* *def:weil-direct-image-node-output-candidate*, *def:weil-direct-image-upper-weight-predicate*

Lean: *Weil.Weight.DirectImage.curveLeaf_assembles*

The output at a curve leaf satisfies the upper-weight predicate.

Proof. Uses: def:weil-direct-image-curve-leaf-terminal-certificate, def:weil-direct-image-relative-localization-certificate, prop:weil-direct-image-relative-curve-upper

Project the matching curve-terminal branch from the node certificate, then project its `localization` field and the named `localization.fiberCertificate` field. Instantiate the relative-curve proposition with this localization certificate and the TAME certificate stored in the same terminal record. All indices agree by the terminal record's definitional-equality fields. □

Definition 7.8.118 (Upper mixedness candidate constructed from the finite tree)**.** Uses: `def:weil-direct-image-complete-antecedent-certificate`,`def:weil-direct-image-node-output-candidate`,`def:weil-direct-image-upper-mixed-certificate-data`

Lean: `Weil.Weight.DirectImage.upperMixedCertificateFromTree`

Define raw upper mixedness candidate data at every node by structural recursion on the supplied finite tree: recursively construct the dependent child family and then apply the tag-specific node-output constructor. The requested datum is the value at the root. Structural recursion on the tree, not induction on $(b_{\mathrm{rel}}, b_{\mathrm{src}})$, is the termination argument.

Lemma 7.8.119 (Validity of the recursively constructed root datum)**.** *Uses:* *def:weil-direct-image-upper-mixed-certificate-from-tree*, *def:weil-direct-image-upper-weight-predicate*

Lean: *Weil.Weight.DirectImage.upperMixedCertificateFromTree_valid*

The recursively constructed root datum satisfies the direct-image upper-weight predicate.

Proof. Uses: `def:weil-direct-image-reduction-tree-validity-certificate`,`lemma:weil-direct-image-coefficient-node-assembles`,`lemma:weil-direct-image-source-node-assembles`,`lemma:weil-direct-image-target-node-assembles`,`lemma:weil-direct-image-composition-node-assembles`,`lemma:weil-direct-image-universal-homeomorphism-node-assembles`,`lemma:weil-direct-image-geometric-reduction-node-assembles`,`lemma:weil-direct-image-zero-leaf-assembles`,`lemma:weil-direct-image-curve-leaf-assembles`

Perform structural induction on the certified finite tree. In each node constructor, the induction hypotheses are exactly the dependent child validity fields; dispatch on the stored branch tag and apply the one matching assembly lemma. □

Lemma 7.8.120 (Underlying sheaf of the tree output)**.** *Uses:* *def:weil-direct-image-upper-mixed-certificate-from-tree*, *def:weil-direct-image-cohomology-candidate*

Lean: *Weil.Weight.DirectImage.upperMixedCertificateFromTree_underlying*

The underlying sheaf of the root datum is $R^i f_! \mathcal{F}$.

Proof. Uses: `def:weil-direct-image-upper-weight-predicate`,`lemma:weil-direct-image-tree-output-valid`

Project the named `underlying` field from the root proof of IsUpperWeightBound. □

Lemma 7.8.121 (Exactness of the tree-output filtration)**.** *Uses:* *def:weil-direct-image-upper-mixed-certificate-from-tree*, *def:weil-weight-filtration-exactness-predicate*

Lean: *Weil.Weight.DirectImage.upperMixedCertificateFromTree_exact*

The filtration stored in the root datum is exact and Frobenius-equivariant.

Proof. Uses: `def:weil-direct-image-upper-weight-predicate`,`lemma:weil-direct-image-tree-output-valid`

Project the named `exact` field from the root proof of IsUpperWeightBound. □

Lemma 7.8.122 (Graded purity of the tree output)**.** *Uses:* *def:weil-direct-image-upper-mixed-certificate-from-tree*, *def:weil-weight-filtration-purity-predicate*

Lean: *Weil.Weight.DirectImage.upperMixedCertificateFromTree_gradedPure*

Every graded piece of the root filtration is pointwise pure of its declared weight.

Proof. Uses: `def:weil-direct-image-upper-weight-predicate`,`lemma:weil-direct-image-tree-output-valid`

Project the named `gradedPure` field from the root proof of IsUpperWeightBound. □

Lemma 7.8.123 (Weight-index bound of the tree output)**.** *Uses:* *def:weil-direct-image-upper-mixed-certificate-from-tree*

Lean: *Weil.Weight.DirectImage.upperMixedCertificateFromTree_indexBound*

Every nonzero graded index in the root filtration is at most $n + i$.

Proof. Uses: `def:weil-direct-image-upper-weight-predicate`,`lemma:weil-direct-image-tree-output-valid`

Project the named `indexBound` field from the root proof of IsUpperWeightBound. □

Theorem 7.8.124 (Conditional direct-image upper-weight theorem). *Uses:* *def:weil-direct-image-weight-input-data, def:weil-direct-image-complete-antecedent-certificate, def:weil-direct-image-upper-mixed-certificate-from-tree, def:weil-direct-image-upper-weight-predicate*

Lean: *Weil.Weight.DirectImage.upperWeight*

For every integer i, the explicitly constructed raw tree output satisfies IsUpperWeightBound*; its named fields make it a mixedness certificate on $R^i f_! \mathcal{F}$ of weights at most $n + i$.*

Proof. Uses: `lemma:weil-direct-image-tree-output-valid`

Apply the root-validity lemma to the supplied complete certified tree. □

7.9 Smooth proper purity and the Riemann-hypothesis modulus

Definition 7.9.1 (Constant-sheaf closed-point spectrum). Uses: `def:weil-weight-closed-point-spectrum-family`

Lean: `Weil.Weight.constantSheafSpectrumFamily`

For the constant coefficient sheaf E_X, define its spectrum at every closed point to be the singleton multiset $[1]$.

Lemma 7.9.2 (The constant sheaf is pointwise pure of weight zero). *Uses:* *def:weil-weight-constant-sheaf-spectrum-family, def:weil-weight-pointwise-pure-predicate*

Lean: *Weil.Weight.constantSheaf_isPointwiseIotaPure*

The constant sheaf spectrum family is pointwise pure of weight 0.

Proof. Uses: `def:weil-weight-constant-sheaf-spectrum-family`, `def:weil-weight-pointwise-pure-predicate`, `def:weil-weight-pure-spectrum-predicate`

At every degree e, the sole eigenvalue is 1, and

$$|\iota(1)| = 1 = (q^e)^0.$$

□

Definition 7.9.3 (Smooth proper scheme datum). Uses: `def:weil-finite-field-data`

Lean: `Weil.Weight.SmoothProperSchemeData`

A smooth proper scheme datum consists of a finite-type k_0-scheme X, a nonnegative integer d, proofs that X is smooth and proper over k_0, and a proof that every irreducible component has dimension d. Projectivity and geometric connectedness may be appended for the final classical Weil statement, but neither is used in the weight calculation.

Definition 7.9.4 (Smooth proper compact-support cohomology datum). Uses: `def:weil-weight-smooth-proper-scheme-data`, `def:weil-direct-image-cohomology-candidate`, `def:weil-bounded-graded-based-automorphism-family`

Lean: `Weil.Weight.SmoothProperCompactCohomologyData`

For the structure morphism $a : X \to \mathrm{Spec}(k_0)$, a compact cohomology datum consists of candidates $H^i_c(X_{\bar{k}}, E)$, their geometric Frobenius automorphisms, a bound B, displayed bases, and a bounded graded based automorphism family $G_c = (D_{c,i})_{0 \le i \le B}$ realizing these spaces and operators.

Definition 7.9.5 (Smooth proper ordinary cohomology datum). Uses: `def:weil-weight-smooth-proper-scheme-data`, `def:weil-conditional-cohomology-frobenius-module`, `def:weil-bounded-graded-based-automorphism-family`

Lean: `Weil.Weight.SmoothProperOrdinaryCohomologyData`

An ordinary cohomology datum consists of candidates $H^i(X_{\bar{k}}, E)$, their geometric Frobenius automorphisms, displayed bases, and a bounded graded based automorphism family $G = (D_i)_{0 \le i \le B}$ realizing those spaces and operators.

Definition 7.9.6 (Cohomology realization comparison candidate). Uses: `def:weil-weight-smooth-proper-compact-cohomology-data`, `def:weil-weight-smooth-proper-ordinary-cohomology-data`

Lean: `Weil.Weight.CohomologyRealizationComparisonCandidate`

For every degree i, define a proposed linear map

$$c_i : H^i_c(X_{\bar{k}}, E) \longrightarrow H^i(X_{\bar{k}}, E)$$

and a proposed inverse s_i.

Definition 7.9.7 (Proper support-comparison certificate). Uses: def:weil-weight-cohomology-realization-comparison-candidate,def:weil-duality-proper-support-isomorphism-certificate

Lean: Weil.Weight.ProperSupportComparisonCertificate

A proper support-comparison certificate supplies both inverse equations for c_i, s_i, geometric-Frobenius equivariance, compatibility with the actual $Ra_! \to Ra_*$ map, and equality of the two spectra with multiplicity in every degree.

Open obligation: OBL-WGT-RH-PROPER-SUPPORT

Definition 7.9.8 (Structure-morphism direct-image certificate). Uses: def:weil-weight-smooth-proper-compact-cohomology-data,def:weil-weight-constant-sheaf-spectrum-family,def:weil-direct-image-complete-antecedent-certificate

Lean: Weil.Weight.StructureDirectImageCertificate

A structure-morphism certificate specializes the complete direct-image antecedent certificate to $a : X \to \mathrm{Spec}(k_0)$, the constant sheaf, $n = 0$, and every degree $0 \le i \le B$. It also supplies the identification of $R^i a_! E_X$ with the compact cohomology datum.

Lemma 7.9.9 (Compact-support upper weight bound). Uses: def:weil-weight-structure-direct-image-certificate, def:weil-weight-spectrum-at-most-predicate

Lean: Weil.Weight.smoothProper_compact_weightAtMost

The Frobenius spectrum on $H^i_c(X_{\bar{k}}, E)$ has weights at most i.

Proof. Uses: lemma:weil-weight-constant-sheaf-pure-zero,lemma:weil-weight-pure-gives-mixed-certificate,thm:weil-direct-image-upper-weight,def:weil-weight-structure-direct-image-certificate

The constant sheaf has weight zero and hence the two-step mixedness certificate concentrated at zero. Apply the direct-image upper-weight theorem to the structure morphism with $n = 0$. Its degree-i conclusion is weights at most i; transport it through the compact-cohomology identification stored in the structure certificate. □

Lemma 7.9.10 (Ordinary upper weight bound under properness). Uses: def:weil-weight-proper-support-comparison-certificate, def:weil-weight-spectrum-at-most-predicate

Lean: Weil.Weight.smoothProper_ordinary_weightAtMost

The Frobenius spectrum on $H^i(X_{\bar{k}}, E)$ has weights at most i.

Proof. Uses: lemma:weil-weight-smooth-proper-compact-upper,def:weil-weight-proper-support-comparison-certificate

The support-comparison certificate identifies the ordinary spectrum with the compact-support spectrum, preserving multiplicities and Frobenius. Transport the compact-support upper bound through that identity. □

Definition 7.9.11 (Poincare spectrum-pairing candidate). Uses: def:weil-weight-smooth-proper-scheme-data,def:weil-weight-smooth-proper-ordinary-cohomology-data,def:weil-duality-proper-poincare-pairing-candidate

Lean: Weil.Weight.PoincareSpectrumPairingCandidate

For every $0 \le i \le 2d$, define the proposed perfect pairing

$$B_i : H^i(X_{\bar{k}}, E) \otimes H^{2d-i}(X_{\bar{k}}, E) \longrightarrow E(-d)$$

and a multiplicity-preserving proposed bijection between their spectra.

Definition 7.9.12 (Poincare spectrum-pairing certificate). Uses: def:weil-weight-poincare-spectrum-pairing-candidate,def:weil-duality-perfect-pairing-certificate,def:weil-duality-frobenius-similitude-predicate

Lean: Weil.Weight.PoincareSpectrumPairingCertificate

A Poincare spectrum-pairing certificate supplies the adjoint inverse maps, all inverse equations, cup and trace Frobenius equivariance, and the spectral relation

$$\alpha\,\gamma = q^d$$

for every paired occurrence $\alpha \in H^i$, $\gamma \in H^{2d-i}$. The scalar q^d is geometric Frobenius on $E(-d)$.

Open obligation: OBL-WGT-RH-POINCARE-SPECTRA

Lemma 7.9.13 (Poincare duality converts the upper bound into a lower bound). Uses: def:weil-weight-poincare-spectrum-pairing-certificate, def:weil-weight-spectrum-at-least-predicate

Lean: Weil.Weight.smoothProper_ordinary_weightAtLeast

The Frobenius spectrum on $H^i(X_{\bar{k}}, E)$ has weights at least i.

Proof. Uses: lemma:weil-weight-smooth-proper-ordinary-upper, def:weil-weight-poincare-spectrum-pairing-certificate

For an eigenvalue occurrence α in degree i, let γ be its paired occurrence in degree $2d - i$, as stored in the pairing certificate. The ordinary upper bound gives

$$|\iota(\gamma)| \le q^{(2d-i)/2}.$$

The pairing relation and nonvanishing give

$$|\iota(\alpha)| = q^d |\iota(\gamma)|^{-1} \ge q^{d-(2d-i)/2} = q^{i/2}.$$

This is the degree-i lower-weight predicate. □

Definition 7.9.14 (Fixed-realization smooth proper antecedent certificate). Uses: def:weil-weight-smooth-proper-scheme-data, def:weil-weight-smooth-proper-compact-cohomology-data, def:weil-weight-smooth-proper-ordinary-cohomology-data, def:weil-weight-structure-direct-image-certificate, def:weil-weight-proper-support-comparison-certificate, def:weil-weight-poincare-spectrum-pairing-certificate

Lean: Weil.Weight.SmoothProperIotaCertificate

A fixed-realization antecedent certificate is the ordered tuple of the scheme, compact and ordinary cohomology, direct-image, proper-support, and Poincare spectral certificates for the embedding ι in the coefficient datum. It contains no purity field.

Theorem 7.9.15 (Conditional fixed-realization smooth proper purity). *Uses: def:weil-weight-smooth-proper-iota-certificate, def:weil-weight-pure-spectrum-predicate*

Lean: Weil.Weight.smoothProper_isIotaPure

For every degree i, geometric Frobenius on $H^i(X_{\bar{k}}, E)$ is ι-pure of weight i.

Proof. Uses: lemma:weil-weight-smooth-proper-ordinary-upper, lemma:weil-weight-smooth-proper-ordinary-lower, lemma:weil-weight-bounds-give-purity

Fix an eigenvalue occurrence. The upper and lower lemmas give opposite inequalities with common endpoint $q^{i/2}$. Apply the scalar opposite-bounds lemma. Perform this for every occurrence in the finite spectrum. □

Definition 7.9.16 (Universal ell-adic eigenvalue-purity predicate). Uses: def:weil-prime-power-data, def:weil-coefficient-data

Lean: Weil.Weight.IsUniversallyPureElladicEigenvalue

For $\alpha \in \overline{E}^\times$ and $i \in \mathbf{Z}$, define

$$\mathrm{EllPure}_q(\alpha, i) \quad :\Longleftrightarrow \quad \forall \jmath : \overline{E} \xrightarrow{\sim} \mathbf{C}, \qquad |\jmath(\alpha)| = q^{i/2}.$$

This predicate is formulated to match the realization interface used in the arithmetic-factor module; it does not assert integrality or ℓ-independence.

Definition 7.9.17 (Graded universal eigenvalue-purity predicate). Uses: def:weil-bounded-graded-based-automorphism-family, def:weil-based-scalar-extension-datum, def:weil-based-scalar-extended-eigenspace, def:weil-weight-universal-elladic-eigenvalue-purity-predicate

Lean: Weil.Weight.IsUniversallyPureGradedEigenvalues

For a bounded graded based automorphism family $G = (D_i)$, define UniversalEigenPurity(G) to mean: for every degree i and every $\alpha \in \overline{E}^\times$ whose scalar-extended eigenspace for D_i is nonzero, $\mathrm{EllPure}_q(\alpha, i)$.

Definition 7.9.18 (Realization-indexed antecedent family). Uses: def:weil-weight-smooth-proper-scheme-data, def:weil-weight-smooth-proper-ordinary-cohomology-data, def:weil-weight-smooth-proper-iota-certificate

Lean: Weil.Weight.RealizationIndexedCertificateFamily

A realization-indexed antecedent family assigns to every field isomorphism $\jmath : \overline{E} \xrightarrow{\sim} \mathbf{C}$ a fixed-realization smooth proper antecedent certificate whose complex embedding field is $\jmath$, together with identities showing that the underlying scheme, cohomology spaces, based Frobenius operators, and eigenvalue occurrences are independent of $\jmath$.

Open obligation: OBL-WGT-RH-ALL-REALIZATIONS

Definition 7.9.19 (Smooth proper RH input datum). Uses: def:weil-weight-smooth-proper-scheme-data, def:weil-weight-smooth-proper-ordinary-cohomology-data, def:weil-weight-realization-indexed-certificate-family

Lean: Weil.Weight.SmoothProperRHInputData

A smooth proper RH input datum consists of the scheme datum, the bounded graded ordinary cohomology family G, and a realization-indexed antecedent family. It includes explicit comparisons between the scalar-extended eigenspaces used by G and the spectra in every fixed-realization certificate.

Theorem 7.9.20 (Conditional smooth proper universal purity). Uses: def:weil-weight-smooth-proper-rh-input-data, def:weil-weight-graded-universal-purity-predicate

Lean: Weil.Weight.smoothProper_universalPurity

The graded geometric-Frobenius family G satisfies UniversalEigenPurity(G)*: every eigenvalue in degree i has absolute value $q^{i/2}$ under every complex realization.*

Proof. Uses: def:weil-weight-realization-indexed-certificate-family, thm:weil-weight-smooth-proper-iota-purity

Fix a degree i, a scalar-extended eigenvalue occurrence α, and a field isomorphism $\jmath : \overline{E} \simeq \mathbf{C}$. Use the antecedent certificate indexed by $\jmath$. Its occurrence-comparison field places α in the degree-i spectrum. Apply the fixed-realization purity theorem with complex embedding $\jmath$, obtaining $|\jmath(\alpha)| = q^{i/2}$. The three inputs were arbitrary. □

Corollary 7.9.21 (Smooth proper eigenvalue absolute value). Uses: def:weil-weight-smooth-proper-rh-input-data, def:weil-weight-universal-elladic-eigenvalue-purity-predicate

Lean: Weil.Weight.smoothProper_eigenvalue_modulus

For every degree-i geometric-Frobenius eigenvalue $\alpha \in \overline{E}^{\times}$ and every $\jmath : \overline{E} \simeq \mathbf{C}$,

$$|\jmath(\alpha)| = q^{i/2}.$$

Proof. Uses: thm:weil-weight-smooth-proper-universal-purity, def:weil-weight-graded-universal-purity-predicate, def:weil-weight-universal-elladic-eigenvalue-purity-predicate

Apply universal purity to the degree i, eigenvalue α, and realization $\jmath$. □

Definition 7.9.22 (Complex root-realization datum). Uses: def:weil-weight-smooth-proper-rh-input-data, def:weil-based-scalar-extension-datum, def:weil-based-reciprocal-determinant-base-change

Lean: Weil.Weight.ComplexRootRealizationData

For a degree i and $\jmath : \overline{E} \simeq \mathbf{C}$, a complex root-realization datum consists of the based scalar extension of D_i to $\mathbf{C}$, a scalar $\beta \in \mathbf{C}$, and the equation

$$P_{D_i,\mathbf{C}}(\beta) = 0.$$

Definition 7.9.23 (Ell-adic pullback of a complex root). Uses: def:weil-weight-complex-root-realization-data, def:weil-based-scalar-extension-datum, def:weil-based-reciprocal-determinant-base-change

Lean: Weil.Weight.ElladicRootPullbackData

Define

$$\gamma := \jmath^{-1}(\beta) \in \overline{E}$$

and define the based scalar-extension datum for D_i from E to $\overline{E}$, with reciprocal polynomial $P_{D_i,\overline{E}}$. These are explicit inverse-image and base-change constructions.

Lemma 7.9.24 (The ell-adic pullback is an ordinary root). Uses: def:weil-weight-elladic-root-pullback-data

Lean: Weil.Weight.ElladicRootPullbackData.isRoot

One has

$$P_{D_i,\overline{E}}(\gamma) = 0.$$

Proof. Uses: def:weil-weight-elladic-root-pullback-data, def:weil-weight-complex-root-realization-data, lemma:weil-based-reciprocal-determinant-base-change

Map the displayed value to $\mathbf{C}$ through $\jmath$. Determinant base change and polynomial evaluation give

$$\jmath(P_{D_i,\overline{E}}(\gamma)) = P_{D_i,\mathbf{C}}(\jmath(\gamma)) = P_{D_i,\mathbf{C}}(\beta) = 0.$$

Injectivity of the field isomorphism $\jmath$ gives the asserted equality. □

Lemma 7.9.25 (The ell-adic pullback root is nonzero). *Uses:* def:weil-weight-elladic-root-pullback-data

Lean: Weil.Weight.ElladicRootPullbackData.root_ne_zero

One has $\gamma \neq 0$.

Proof. Uses: def:weil-weight-elladic-root-pullback-data,def:weil-weight-complex-root-realization-data,lemma:weil-based-reciprocal-determinant-constant

If $\gamma = 0$, then $\beta = \jmath(\gamma) = 0$. The reciprocal determinant has constant coefficient 1, so its value at zero is one, contradicting the root equation in the complex root-realization datum. □

Definition 7.9.26 (Ell-adic reciprocal eigenvalue occurrence). Uses: def:weil-weight-elladic-root-pullback-data,def:weil-based-scalar-extended-eigenspace

Lean: Weil.Weight.ElladicRootPullbackData.reciprocalEigenvalueOccurrence

Define

$$\alpha := \gamma^{-1} \in \overline{E}^{\times}$$

and its scalar-extended eigenspace $E_\alpha((F_i)_{\overline{E}})$.

Lemma 7.9.27 (The reciprocal pullback is an ell-adic eigenvalue). *Uses:* def:weil-weight-elladic-root-pullback-data, def:weil-weight-elladic-reciprocal-eigenvalue-occurrence

Lean: Weil.Weight.ElladicRootPullbackData.reciprocal_isEigenvalue

The eigenspace $E_\alpha((F_i)_{\overline{E}})$ is nonzero.

Proof. Uses: lemma:weil-weight-elladic-pullback-is-root,thm:weil-root-reciprocal-conversion-after-scalar-extension

Apply the determinant module's scalar-extension root theorem over $\overline{E}$ to the equation $P_{D_i,\overline{E}}(\gamma) = 0$. Its conclusion is nonvanishing of the eigenspace at $\gamma^{-1} = \alpha$. □

Corollary 7.9.28 (Smooth proper ordinary-root modulus). *Uses:* def:weil-weight-smooth-proper-rh-input-data, def:weil-weight-complex-root-realization-data, def:weil-weight-elladic-root-pullback-data

Lean: Weil.Weight.smoothProper_ordinaryRoot_modulus

Every ordinary complex root β of the degree-i factor $\det(1 - TF_i)$ satisfies

$$|\beta| = q^{-i/2}.$$

Proof. Uses: def:weil-weight-elladic-reciprocal-eigenvalue-occurrence,lemma:weil-weight-elladic-reciprocal-is-eigenvalue,cor:weil-weight-smooth-proper-eigenvalue-modulus

The eigenvalue modulus corollary applied to $\alpha = \gamma^{-1}$ gives $|\jmath(\alpha)| = q^{i/2}$. From $\jmath(\gamma) = \beta$ and $\gamma \neq 0$,

$$|\jmath(\alpha)| = |\jmath(\gamma^{-1})| = |\beta|^{-1}.$$

Invert the positive equality to obtain $|\beta| = q^{-i/2}$. □

Definition 7.9.29 (Riemann-hypothesis output certificate). Uses: def:weil-weight-smooth-proper-rh-input-data,def:weil-weight-graded-universal-purity-predicate

Lean: Weil.Weight.RHOutputCertificate

Define the RH output certificate to be the universal eigenvalue-purity predicate for G, together with the ordinary-root modulus predicate for every degree, every complex realization, and every root-realization datum. It deliberately contains no integrality, no no-cancellation statement, and no ℓ-independence statement.

Proposition 7.9.30 (Conditional Riemann hypothesis for the cohomological factors). *Uses:* def:weil-weight-smooth-proper-rh-input-data, def:weil-weight-rh-output-certificate

Lean: Weil.Weight.cohomologicalRiemannHypothesis

The smooth proper RH input datum determines an RH output certificate: geometric-Frobenius eigenvalues in degree i have universal modulus $q^{i/2}$, and ordinary roots of $\det(1 - TF_i)$ have modulus $q^{-i/2}$.

Proof. Uses: thm:weil-weight-smooth-proper-universal-purity,cor:weil-weight-smooth-proper-ordinary-root-modulus,def:weil-weight-rh-output-certificate

Use universal purity as the first field of the output certificate. For the second field, apply the ordinary-root modulus corollary to each root-realization datum. □

Status. The proof order in this chapter is

$$\begin{aligned}\mathrm{PUR} &\longrightarrow \mathrm{MON} \longrightarrow \text{global/local Fourier} \longrightarrow \text{curve estimate}\\ &\longrightarrow \text{direct image} \longrightarrow \text{duality lower bound} \longrightarrow \text{smooth proper purity}.\end{aligned}$$

No no-cancellation, individual-factor integrality, or ℓ-independence result is used anywhere in that chain. The archimedean conclusion is conditional until every open certificate listed in the module manifest has been constructed.

Chapter 8

Betti degrees, integral factors, and independence of ℓ

Status. This chapter never uses individual integrality or ℓ-independence in order to prove purity. Its input is the already constructed point-counting series, a supplied finite cohomological trace datum, and universal-realization purity for the reciprocal roots. The order is strictly

$$\text{purity} \Rightarrow \text{no cancellation} \Rightarrow \text{algebraicity} \Rightarrow \mathbf{Q}\text{-descent} \Rightarrow \mathbf{Z}\text{-integrality} \Rightarrow \ell\text{-independence}.$$

Thus the first assertion about an individual factor having rational coefficients occurs only after no cancellation has been proved. The geometric inputs required to inhabit the conditional data remain separately numbered, nonimportable obligations in the module manifest.

8.1 Three weight predicates and the realization-field interface

Definition 8.1.1 (Global complex realization candidate). Uses: `def:weil-coefficient-algebraic-closure-data`

Lean: `Weil.Arithmetic.GlobalComplexRealizationCandidate`

For a coefficient algebraic-closure datum $(E, \overline{E})$, a global complex realization candidate is a specified field isomorphism

$$\jmath : \overline{E} \xrightarrow{\sim} \mathbf{C}.$$

It is a type of candidate data, not an assertion that such an isomorphism has been constructed. Unlike the fixed embedding ι_0, a candidate is required to be surjective.

Formalization note. In the cardinal statements below, cardinalities of types living in different universes are compared only after applying `Cardinal.lift` to a common universe. Thus an equality of basis cardinals is converted to an actual equivalence of the two basis index types with `Cardinal.lift_mk_eq'`, rather than by silently identifying universe levels.

Definition 8.1.2 (Binary partial sums in the ell-adic field). Uses: `def:weil-coefficient-algebraic-closure-data`

Lean: `Weil.Arithmetic.padicBinaryPartialSums`

For a binary sequence $a\colon \mathbf{N} \to \{0,1\}$ and $N \in \mathbf{N}$, put

$$s_N(a) = \sum_{0 \le n < N} a(n)\ell^n \in E = \mathbf{Q}_\ell.$$

Lemma 8.1.3 (The binary partial sums are ell-adic Cauchy). *Uses:* `def:weil-padic-binary-partial-sums`

Lean: `Weil.Arithmetic.padicBinaryPartialSums_cauchy`

For every binary sequence a, the sequence $N \mapsto s_N(a)$ is Cauchy for the ℓ-adic norm.

Proof. Uses: def:weil-padic-binary-partial-sums
If $M \geq N$, the ultrametric inequality gives

$$|s_M(a) - s_N(a)|_\ell \leq \max_{N \leq n < M} |\ell^n|_\ell \leq \ell^{-N}.$$

The bound tends to zero because the selected ℓ is prime, hence $\ell \geq 2$. □

Definition 8.1.4 (Binary ell-adic limit). Uses: def:weil-padic-binary-partial-sums

Lean: Weil.Arithmetic.padicBinaryLimit

Constructed by: lemma:weil-padic-binary-partial-sums-cauchy
Define $x_a \in E$ to be the limit of $N \mapsto s_N(a)$ in the complete field $E = \mathbf{Q}_\ell$.

Lemma 8.1.5 (The binary ell-adic limit is integral). Uses: def:weil-padic-binary-limit

Lean: Weil.Arithmetic.padicBinaryLimit_mem_padicInt
For every a, one has $|x_a|_\ell \leq 1$, so x_a belongs to the valuation ring $\mathbf{Z}_\ell \subset E$.

Proof. Uses: def:weil-padic-binary-partial-sums,def:weil-padic-binary-limit
Every partial sum has norm at most one by the ultrametric inequality. The closed unit ball is closed, so its limit x_a also has norm at most one. □

Definition 8.1.6 (Integral binary ell-adic limit). Uses: def:weil-padic-binary-limit

Lean: Weil.Arithmetic.padicBinaryIntegralLimit

Constructed by: lemma:weil-padic-binary-limit-integral
Package x_a with its unit-ball proof as an element $z_a \in \mathbf{Z}_\ell$.

Lemma 8.1.7 (Residues of the binary ell-adic limit). Uses: def:weil-padic-binary-integral-limit

Lean: Weil.Arithmetic.padicBinaryLimit_mod_pow
For $r \geq 1$, reduction in $\mathbf{Z}_\ell/\ell^r\mathbf{Z}_\ell$ gives

$$x_a \equiv \sum_{0 \leq n < r} a(n)\ell^n \pmod{\ell^r}.$$

Proof. Uses: def:weil-padic-binary-partial-sums,lemma:weil-padic-binary-partial-sums-cauchy,lemma:weil-padic-binary-limit-integral
For every $N \geq r$, the difference between $s_N(a)$ and the displayed finite sum is divisible by ℓ^r. The ideal $\ell^r\mathbf{Z}_\ell$ is closed, so the same congruence holds after taking the limit. □

Lemma 8.1.8 (The first differing binary digit survives). Uses: def:weil-padic-binary-integral-limit

Lean: Weil.Arithmetic.padicBinaryLimit_firstDifference
If $a \neq b$ and m is the least index with $a(m) \neq b(m)$, then

$$x_a - x_b \equiv (a(m) - b(m))\ell^m \not\equiv 0 \pmod{\ell^{m+1}}.$$

Proof. Uses: lemma:weil-padic-binary-limit-residue
Reduce both limits modulo ℓ^{m+1}. Terms below m cancel by the minimality of m, and terms above m vanish. The remaining coefficient is 1 or -1, which is a unit modulo the prime ℓ, so $\pm\ell^m$ is nonzero modulo ℓ^{m+1}. □

Lemma 8.1.9 (Binary ell-adic expansion is injective). Uses: def:weil-padic-binary-limit

Lean: Weil.Arithmetic.padicBinaryLimit_injective
The map $a \mapsto x_a$ from $\{0,1\}^{\mathbf{N}}$ to $E = \mathbf{Q}_\ell$ is injective.

Proof. Uses: lemma:weil-padic-binary-limit-first-difference
If $a \neq b$, choose their least differing index. The preceding lemma shows that $x_a - x_b$ has a nonzero residue, and therefore $x_a \neq x_b$. □

Lemma 8.1.10 (Continuum lower bound for the ell-adic field). Uses: def:weil-coefficient-algebraic-closure-data, def:weil-padic-binary-limit

Lean: Weil.Arithmetic.continuum_le_mk_padic
One has $\mathfrak{c} \leq \#E$.

Proof. Uses: `lemma:weil-padic-binary-limit-injective`
The source of the displayed injection has cardinality $2^{\aleph_0} = \mathfrak{c}$. □

Lemma 8.1.11 (Cardinality of the ell-adic Cauchy-sequence carrier). *Uses:* `def:weil-coefficient-algebraic-closure-data`

Lean: `Weil.Arithmetic.mk_padicSeq_le_continuum`
The type of ℓ-adic Cauchy sequences of rational numbers has cardinality at most $\mathfrak{c}$.

Proof. Uses: `def:weil-coefficient-algebraic-closure-data`
Forgetting the Cauchy proof injects this type into $\mathbf{Q}^{\mathbf{N}}$. Hence its cardinality is at most

$$(\#\mathbf{Q})^{\#\mathbf{N}} = \aleph_0^{\aleph_0} = \mathfrak{c}.$$

□

Lemma 8.1.12 (Continuum upper bound for the ell-adic field). *Uses:* `def:weil-coefficient-algebraic-closure-data`

Lean: `Weil.Arithmetic.mk_padic_le_continuum`
One has $\#E \le \mathfrak{c}$.

Proof. Uses: `lemma:weil-padic-cauchy-carrier-cardinal-upper`
At the pinned definition, $\mathbf{Q}_\ell$ is the quotient of the type of ℓ-adic rational Cauchy sequences by the zero-distance relation. The quotient map is surjective, so a quotient cannot have larger cardinality than its carrier. □

Proposition 8.1.13 (Cardinality of the ell-adic field). *Uses:* `def:weil-coefficient-algebraic-closure-data`

Lean: `Weil.Arithmetic.mk_padic_eq_continuum`
The field $E = \mathbf{Q}_\ell$ has cardinality $\mathfrak{c}$.

Proof. Uses: `lemma:weil-padic-cardinal-continuum-lower,lemma:weil-padic-cardinal-continuum-upper`
Apply antisymmetry to the two cardinal inequalities. □

Lemma 8.1.14 (The ell-adic field is uncountable). *Uses:* `def:weil-coefficient-algebraic-closure-data`

Lean: `Weil.Arithmetic.aleph0_lt_mk_padic`
One has $\aleph_0 < \#E$.

Proof. Uses: `prop:weil-padic-cardinal-continuum`
Rewrite $\#E$ as $\mathfrak{c}$ and use $\aleph_0 < \mathfrak{c}$. □

Lemma 8.1.15 (Coefficient-closure cardinal lower bound). *Uses:* `def:weil-coefficient-algebraic-closure-data`

Lean: `Weil.Arithmetic.continuum_le_mk_coefficientClosure`
One has $\mathfrak{c} \le \#\overline{E}$.

Proof. Uses: `def:weil-coefficient-algebraic-closure-data,prop:weil-padic-cardinal-continuum`
The specified algebra map $E \hookrightarrow \overline{E}$ is injective because it is a map of fields. Transport the equality $\#E = \mathfrak{c}$ along this injection. □

Lemma 8.1.16 (Coefficient-closure cardinal upper bound). *Uses:* `def:weil-coefficient-algebraic-closure-data`

Lean: `Weil.Arithmetic.mk_coefficientClosure_le_continuum`
One has $\#\overline{E} \le \mathfrak{c}$.

Proof. Uses: `def:weil-coefficient-algebraic-closure-data,prop:weil-padic-cardinal-continuum`
For an algebraic extension of domains, every element is a root of a nonzero polynomial over the base; the union of the finite root sets gives

$$\#\overline{E} \le \max(\#E, \aleph_0).$$

This is the exact content of `Algebra.IsAlgebraic.cardinalMk_le_max`. Substitute $\#E = \mathfrak{c}$ and $\aleph_0 \le \mathfrak{c}$. □

Proposition 8.1.17 (Cardinality of the coefficient closure). *Uses:* def:weil-coefficient-algebraic-closure-data

Lean: Weil.Arithmetic.mk_coefficientClosure_eq_continuum

The selected algebraic closure $\overline{E}$ has cardinality $\mathfrak{c}$.

Proof. Uses: lemma:weil-coefficient-closure-cardinal-lower,lemma:weil-coefficient-closure-cardinal-upper

Apply antisymmetry. □

Lemma 8.1.18 (The coefficient closure is uncountable). *Uses:* def:weil-coefficient-algebraic-closure-data

Lean: Weil.Arithmetic.aleph0_lt_mk_coefficientClosure

One has $\aleph_0 < \#\overline{E}$.

Proof. Uses: prop:weil-coefficient-closure-cardinal-continuum

Rewrite by the preceding equality and use $\aleph_0 < \mathfrak{c}$. □

Lemma 8.1.19 (Existence of an ell-adic transcendence basis). *Uses:* def:weil-coefficient-algebraic-closure-data

Lean: Weil.Arithmetic.exists_padic_transcendenceBasis

There is a set $B_\ell \subset E$ whose inclusion is a transcendence basis of $E/\mathbf{Q}$.

Proof. Uses: def:weil-coefficient-algebraic-closure-data

Apply the maximal algebraically independent set theorem, equivalently the pinned declaration exists_isTranscendenceBasis, to $\mathbf{Q} \to E$. □

Definition 8.1.20 (Selected ell-adic transcendence basis). Uses: def:weil-coefficient-algebraic-closure-data

Lean: Weil.Arithmetic.padicTranscendenceBasis

Constructed by: lemma:weil-padic-transcendence-basis-exists

Choose once and for all one basis B_ℓ supplied by the preceding existence theorem, together with its basis certificate.

Lemma 8.1.21 (Cardinality of the ell-adic transcendence basis). *Uses:* def:weil-padic-transcendence-basis

Lean: Weil.Arithmetic.mk_padicTranscendenceBasis_eq_continuum

The selected basis satisfies $\#B_\ell = \mathfrak{c}$.

Proof. Uses: def:weil-padic-transcendence-basis,prop:weil-padic-cardinal-continuum,lemma:weil-padic-uncountable

The field E is algebraic over the rational-function field $\mathbf{Q}(B_\ell)$. Polynomial and fraction-field cardinality, followed by the algebraic-extension bound, gives

$$\#E \le \max(\aleph_0, \#B_\ell).$$

Also $\#B_\ell \le \#E$. If $\#B_\ell \le \aleph_0$, the first inequality would make E countable, contradicting the preceding uncountability lemma. Thus the displayed maximum equals $\#B_\ell$, and $\#E = \mathfrak{c}$ forces $\#B_\ell = \mathfrak{c}$. The pinned generic formal counterpart is the cardinality theorem for an IsTranscendenceBasis. □

Proposition 8.1.22 (Transcendence degree of the ell-adic field). *Uses:* def:weil-coefficient-algebraic-closure-data, def:weil-padic-transcendence-basis

Lean: Weil.Arithmetic.trdeg_rat_padic_eq_continuum

One has $\mathrm{trdeg}_{\mathbf{Q}} E = \mathfrak{c}$.

Proof. Uses: def:weil-padic-transcendence-basis,lemma:weil-padic-transcendence-basis-cardinal

Compute transcendence degree with the displayed certified basis. □

Definition 8.1.23 (The ell-adic basis inside the coefficient closure). Uses: def:weil-coefficient-algebraic-closure-data,def:weil-padic-transcendence-basis

Lean: Weil.Arithmetic.coefficientClosureTranscendenceFamily

Map the selected family $B_\ell \hookrightarrow E$ into $\overline{E}$ by the specified algebra map, obtaining a family $\overline{B}_\ell \colon B_\ell \to \overline{E}$.

Lemma 8.1.24 (The transported ell-adic basis is a basis of the closure). *Uses:* *def:weil-coefficient-closure-transcendence-family*

Lean: *Weil.Arithmetic.coefficientClosureTranscendenceFamily_isBasis*

The family $\overline{B}_\ell$ is a transcendence basis of $\overline{E}/\mathbf{Q}$.

Proof. Uses: def:weil-coefficient-algebraic-closure-data,def:weil-padic-transcendence-basis,def:weil-coefficient-closure-transcendence-family

Injectivity of $E \hookrightarrow \overline{E}$ preserves algebraic independence. The field E is algebraic over $\mathbf{Q}(B_\ell)$, and $\overline{E}$ is algebraic over E by the stored coefficient-closure datum. Transitivity of algebraicity therefore makes $\overline{E}$ algebraic over $\mathbf{Q}(B_\ell)$, which is exactly the maximality condition for the transported family. □

Proposition 8.1.25 (Transcendence degree of the coefficient closure). *Uses:* *def:weil-coefficient-algebraic-closure-data, def:weil-coefficient-closure-transcendence-family*

Lean: *Weil.Arithmetic.trdeg_rat_coefficientClosure_eq_continuum*

One has $\mathrm{trdeg}_{\mathbf{Q}}\,\overline{E} = \mathfrak{c}$.

Proof. Uses: prop:weil-padic-transcendence-degree-continuum,lemma:weil-coefficient-closure-transcendence-basis,prop:weil-coefficient-closure-cardinal-continuum,lemma:weil-coefficient-closure-uncountable

Algebraicity of $\overline{E}/E$ preserves the displayed $\mathbf{Q}$-transcendence basis, so the ell-adic transcendence-degree calculation gives $\mathfrak{c}$. In the pinned cardinal formulation, the same conclusion follows by applying the uncountable-field transcendence-basis cardinal theorem to the displayed basis and rewriting $\#\overline{E} = \mathfrak{c}$. □

Lemma 8.1.26 (Cardinality of the complex field). *Uses:*

Lean: *Cardinal.mk_complex*

The complex field has cardinality $\#\mathbf{C} = \mathfrak{c}$.

Proof. Uses:

Identify $\mathbf{C}$ with $\mathbf{R}^2$ and use $\#\mathbf{R} = \mathfrak{c}$ and $\mathfrak{c} \cdot \mathfrak{c} = \mathfrak{c}$. This is the pinned theorem `Cardinal.mk_complex`. □

Lemma 8.1.27 (The complex field is uncountable). *Uses:*

Lean: *Weil.Arithmetic.aleph0_lt_mk_complex*

One has $\aleph_0 < \#\mathbf{C}$.

Proof. Uses: lemma:weil-complex-cardinal-continuum

Rewrite by $\#\mathbf{C} = \mathfrak{c}$ and use $\aleph_0 < \mathfrak{c}$. □

Lemma 8.1.28 (Existence of a complex transcendence basis). *Uses:*

Lean: *Weil.Arithmetic.exists_complex_transcendenceBasis*

There is a set $B_{\mathbf{C}} \subset \mathbf{C}$ whose inclusion is a transcendence basis of $\mathbf{C}/\mathbf{Q}$.

Proof. Uses:

Apply `exists_isTranscendenceBasis` to $\mathbf{Q} \to \mathbf{C}$. □

Definition 8.1.29 (Selected complex transcendence basis). Uses:

Lean: `Weil.Arithmetic.complexTranscendenceBasis`

Constructed by: `lemma:weil-complex-transcendence-basis-exists`

Choose one such set $B_{\mathbf{C}}$ together with its basis certificate.

Lemma 8.1.30 (Cardinality of the complex transcendence basis). *Uses:* *def:weil-complex-transcendence-basis*

Lean: *Weil.Arithmetic.mk_complexTranscendenceBasis_eq_continuum*

The selected basis satisfies $\#B_{\mathbf{C}} = \mathfrak{c}$.

Proof. Uses: def:weil-complex-transcendence-basis,lemma:weil-complex-cardinal-continuum,lemma:weil-complex-uncountable

Repeat the cardinal argument for the algebraic extension $\mathbf{C}/\mathbf{Q}(B_{\mathbf{C}})$: the rational-function field has cardinality $\max(\aleph_0, \#B_{\mathbf{C}})$, algebraic extension does not increase that maximum, and uncountability rules out a countable basis. □

Proposition 8.1.31 (Transcendence degree of the complex field). *Uses:* *def:weil-complex-transcendence-basis*

Lean: *Weil.Arithmetic.trdeg_rat_complex_eq_continuum*

One has $\operatorname{trdeg}_{\mathbf{Q}} \mathbf{C} = \mathfrak{c}$.

Proof. Uses: def:weil-complex-transcendence-basis,lemma:weil-complex-transcendence-basis-cardinal

Compute transcendence degree with the selected certified basis. □

Lemma 8.1.32 (Equipotence of the two transcendence bases). *Uses:* *def:weil-padic-transcendence-basis*, *def:weil-complex-transcendence-basis*

Lean: *Weil.Arithmetic.nonempty_padicBasis_equiv_complexBasis*

There exists an equivalence $B_\ell \simeq B_{\mathbf{C}}$.

Proof. Uses: prop:weil-coefficient-closure-transcendence-degree-continuum,prop:weil-complex-transcendence-degree-continuum,lemma:weil-coefficient-closure-transcendence-basis,def:weil-complex-transcendence-basis

The two certified bases compute transcendence degrees which are both $\mathfrak{c}$. Therefore their lifted cardinalities agree. Apply `Cardinal.lift_mk_eq'` to obtain an equivalence of the actual index types. □

Definition 8.1.33 (Selected equivalence of transcendence bases). Uses: def:weil-padic-transcendence-basis,def:weil-complex-transcendence-basis

Lean: `Weil.Arithmetic.padicBasisEquivComplexBasis`

Constructed by: lemma:weil-realization-transcendence-bases-equipotent

Choose an equivalence $e_B \colon B_\ell \xrightarrow{\sim} B_{\mathbf{C}}$.

Definition 8.1.34 (Polynomial-adjoin equivalence from the basis bijection). Uses: def:weil-realization-transcendence-basis-equivalence,def:weil-coefficient-closure-transcendence-family,def:weil-complex-transcendence-basis

Lean: `Weil.Arithmetic.realizationPolynomialAdjoinEquiv`

Constructed by: lemma:weil-coefficient-closure-transcendence-basis

Let

$$A_\ell = \mathbf{Q}[\overline{B}_\ell] \subset \overline{E}, \qquad A_{\mathbf{C}} = \mathbf{Q}[B_{\mathbf{C}}] \subset \mathbf{C}.$$

Rename variables by e_B in the multivariable polynomial ring and conjugate by the two evaluation equivalences. This defines a $\mathbf{Q}$-algebra equivalence $A_\ell \simeq A_{\mathbf{C}}$ sending each basis element to its e_B-mate.

Definition 8.1.35 (Rational-function-field equivalence). Uses: def:weil-realization-polynomial-adjoin-equivalence

Lean: `Weil.Arithmetic.realizationRationalFunctionFieldEquiv`

The preceding equivalence maps nonzero elements to nonzero elements, so the universal property of fraction fields extends it uniquely to

$$e_{\mathrm{rat}} \colon \mathbf{Q}(\overline{B}_\ell) = \operatorname{Frac}(A_\ell) \xrightarrow{\sim} \operatorname{Frac}(A_{\mathbf{C}}) = \mathbf{Q}(B_{\mathbf{C}}).$$

Lemma 8.1.36 (The coefficient closure is algebraic over its rational-function subfield). *Uses:* *def:weil-coefficient-closure-transcendence-family*, *def:weil-realization-rational-function-field-equivalence*

Lean: *Weil.Arithmetic.coefficientClosure_isAlgebraic_rationalFunctionField*

The extension $\overline{E}/\mathbf{Q}(\overline{B}_\ell)$ *is algebraic.*

Proof. Uses: lemma:weil-coefficient-closure-transcendence-basis

This is the algebraicity clause of the transcendence-basis certificate, after identifying the intermediate field generated by the basis with the fraction field by `AlgebraicIndependent.aevalEquivField`. □

Lemma 8.1.37 (The complex field is algebraic over its rational-function subfield). *Uses:* *def:weil-complex-transcendence-basis*, *def:weil-realization-rational-function-field-equivalence*

Lean: *Weil.Arithmetic.complex_isAlgebraic_rationalFunctionField*

The extension $\mathbf{C}/\mathbf{Q}(B_{\mathbf{C}})$ *is algebraic.*

Proof. Uses: `def:weil-complex-transcendence-basis`
This is the same algebraicity clause for the selected complex transcendence basis. □

Definition 8.1.38 (Constructed global complex realization)**.** Uses: `def:weil-global-complex-realization-candidate,def:weil-realization-rational-function-field-equivalence`

Lean: `Weil.Arithmetic.globalComplexRealization`

Constructed by: `lemma:weil-coefficient-closure-algebraic-over-rational-function-field,lemma:weil-complex-algebraic-over-rational-function-field`

Regard $\overline{E}$ and $\mathbf{C}$ as algebraic closures of their displayed rational-function subfields: each is algebraic over its subfield by the two preceding lemmas and algebraically closed by its specified field structure. Extend e_{rat} across these algebraic closures and choose the resulting field isomorphism

$$\jmath_0 \colon \overline{E} \xrightarrow{\sim} \mathbf{C}.$$

This is an inhabitant of GlobalComplexRealizationCandidate. No compatibility with the separately fixed embedding ι_0 is asserted or required. At the pinned revision, the explicit endpoint is `IsAlgClosure.equivOfEquiv`. Equivalently, the compressed classification theorem `IsAlgClosed.equivOfTranscendenceBasis` performs the polynomial rename and algebraic-closure extension in one definition. The resulting ring equivalence is a field isomorphism; in characteristic zero it fixes the prime field $\mathbf{Q}$ automatically.

Definition 8.1.39 (Raw universal-realization purity)**.** Uses: `def:weil-prime-power-data,def:weil-coefficient-algebraic-closure-data,def:weil-global-complex-realization-candidate`

Lean: `Weil.Arithmetic.IsRawUniversallyPure`

For $\alpha \in \overline{E}^{\times}$ and $w \in \mathbf{Z}$, define $\mathrm{RawPure}_q(\alpha, w)$ to mean

$$\forall\, \jmath : \overline{E} \xrightarrow{\sim} \mathbf{C}, \qquad |\jmath(\alpha)| = q^{w/2}.$$

This predicate does not assert that α is algebraic over $\mathbf{Q}$.

Definition 8.1.40 (Finite-subfield embedding extension candidate)**.** Uses: `def:weil-coefficient-algebraic-closure-data,def:weil-global-complex-realization-candidate`

Lean: `Weil.Arithmetic.FiniteSubfieldEmbeddingExtensionCandidate`

Let $L \subseteq \overline{E}$ be finite over $\mathbf{Q}$ and let $\tau : L \hookrightarrow \mathbf{C}$ be a $\mathbf{Q}$-embedding. An extension candidate is a global realization $\widetilde{\tau}$.

Definition 8.1.41 (Finite-subfield embedding extension predicate)**.** Uses: `def:weil-finite-subfield-embedding-extension-candidate`

Lean: `Weil.Arithmetic.ExtendsFiniteSubfieldEmbedding`

Define $\mathrm{Extends}(\widetilde{\tau}, \tau)$ to mean that the restriction of $\widetilde{\tau}$ to L equals τ.

Definition 8.1.42 (Finite-subfield complex-extension selection)**.** Uses: `def:weil-finite-subfield-embedding-extension-candidate`

Lean: `Weil.Arithmetic.FiniteSubfieldComplexExtensionSelection`

A finite-subfield complex-extension selection assigns to each finite $\mathbf{Q}$-subfield $L \subseteq \overline{E}$ and each $\mathbf{Q}$-embedding $\tau : L \hookrightarrow \mathbf{C}$ a candidate $\widetilde{\tau} : \overline{E} \simeq \mathbf{C}$. No restriction equation is included in this raw selection.

Definition 8.1.43 (Finite-subfield complex-extension selection predicate)**.** Uses: `def:weil-finite-subfield-complex-extension-selection,def:weil-finite-subfield-embedding-extension-predicate`

Lean: `Weil.Arithmetic.IsFiniteSubfieldComplexExtensionSelection`

For a selection $e_{\mathbf{C}}$, define $\mathrm{IsComplexExtensionSelection}(e_{\mathbf{C}})$ to mean that $\mathrm{Extends}(e_{\mathbf{C}}(L, \tau), \tau)$ for every admissible pair (L, τ).

Definition 8.1.44 (Target algebra structure prescribed by a finite embedding)**.** Uses: `def:weil-finite-subfield-embedding-extension-candidate`

Lean: `Weil.Arithmetic.finiteSubfieldTargetAlgebra`

For an admissible pair (L, τ), regard $\mathbf{C}$ as an L-algebra, denoted $\mathbf{C}_\tau$, through

$$\mathrm{algebraMap}_{L,\mathbf{C}_\tau} = \tau.$$

The source $\overline{E}$ carries its canonical L-algebra structure through the inclusion $L \subseteq \overline{E}$. Since τ is a $\mathbf{Q}$-algebra embedding, both L-algebra structures form scalar towers over the standard $\mathbf{Q}$-algebra structures. At the pinned revision the target structure is `RingHom.toAlgebra` applied to `tau.toRingHom`; the scalar-tower identity is the pointwise `AlgHom.commutes` equation.

Lemma 8.1.45 (A finite coefficient subfield is algebraic over the rationals). *Uses:* *def:weil-finite-subfield-embedding-extension-candidate*

Lean: *Weil.Arithmetic.finiteSubfield_isAlgebraic_rat*

For every admissible finite subfield $L \subseteq \overline{E}$, the extension $L/\mathbf{Q}$ is algebraic.

Proof. Uses: def:weil-finite-subfield-embedding-extension-candidate

The admissibility datum contains finite-dimensionality of $L/\mathbf{Q}$. Apply the pinned instance `Algebra.IsAlgebraic.of_finite`. □

Lemma 8.1.46 (The coefficient-closure basis remains a basis over the finite subfield). *Uses:* *def:weil-finite-subfield-target-algebra, def:weil-coefficient-closure-transcendence-family*

Lean: *Weil.Arithmetic.coefficientClosureTranscendenceFamily_isBasis_finiteSubfield*

The already selected family $\overline{B}_\ell \colon B_\ell \to \overline{E}$ is a transcendence basis of $\overline{E}/L$.

Proof. Uses: lemma:weil-finite-subfield-algebraic-over-rationals, lemma:weil-coefficient-closure-transcendence-basis

Use the scalar tower $\mathbf{Q} \subseteq L \subseteq \overline{E}$. Algebraic independence persists after the algebraic scalar extension $L/\mathbf{Q}$, and the algebraicity clause in the transcendence-basis certificate is unchanged. This is exactly the forward implication of the pinned equivalence `Algebra.IsAlgebraic.isTranscendenceBasis_iff`. □

Lemma 8.1.47 (The complex basis remains a basis for the prescribed target algebra). *Uses:* *def:weil-finite-subfield-target-algebra, def:weil-complex-transcendence-basis*

Lean: *Weil.Arithmetic.complexTranscendenceBasis_isBasis_finiteSubfield*

The selected family $B_{\mathbf{C}} \to \mathbf{C}_\tau$ is a transcendence basis of $\mathbf{C}_\tau/L$.

Proof. Uses: lemma:weil-finite-subfield-algebraic-over-rationals, def:weil-complex-transcendence-basis

The map τ makes $\mathbf{Q} \to L \to \mathbf{C}_\tau$ a scalar tower. Apply the same pinned equivalence `Algebra.IsAlgebraic.isTranscendenceBasis_iff` to the certified $\mathbf{Q}$-basis $B_{\mathbf{C}}$ and the algebraic extension $L/\mathbf{Q}$. □

Definition 8.1.48 (Finite-subfield adjoin equivalence). Uses: def:weil-realization-transcendence-basis-equivalence, def:weil-coefficient-closure-transcendence-family, def:weil-complex-transcendence-basis, def:weil-finite-subfield-target-algebra

Lean: `Weil.Arithmetic.finiteSubfieldAdjoinEquiv`

Constructed by: lemma:weil-finite-subfield-source-transcendence-basis, lemma:weil-finite-subfield-target-transcendence-basis

For an admissible (L, τ), put

$$A_L = L[\overline{B}_\ell] \subseteq \overline{E}, \qquad A_\tau = L[B_{\mathbf{C}}] \subseteq \mathbf{C}_\tau.$$

Rename polynomial variables with the selected equivalence $e_B : B_\ell \simeq B_{\mathbf{C}}$ and conjugate by the two evaluation equivalences. The result is an L-algebra equivalence

$$e_{L,\tau} : A_L \xrightarrow{\sim} A_\tau.$$

In particular, for every $x \in L$,

$$e_{L,\tau}(\mathrm{algebraMap}_{L,A_L}(x)) = \mathrm{algebraMap}_{L,A_\tau}(x).$$

The pinned ingredients are `MvPolynomial.rename`, `AlgebraicIndependent.aevalEquiv`, and `AlgEquiv.commutes`.

Lemma 8.1.49 (The coefficient closure is an algebraic closure of the source adjoin). *Uses:* *def:weil-finite-subfield-adjoin-equivalence*

Lean: *Weil.Arithmetic.coefficientClosure_isAlgClosure_finiteSubfieldAdjoin*

The field $\overline{E}$ is an algebraic closure of A_L.

Proof. Uses: lemma:weil-finite-subfield-source-transcendence-basis

The source basis certificate says that $\overline{E}$ is algebraic over A_L, and $\overline{E}$ is algebraically closed by the coefficient-closure datum. Apply the pinned theorem `IsAlgClosed.isAlgClosure_of_transcendence_basis`. □

Lemma 8.1.50 (The complex field is an algebraic closure of the target adjoin)**.** *Uses:* *def:weil-finite-subfield-adjoin-equivalence*

Lean: *Weil.Arithmetic.complex_isAlgClosure_finiteSubfieldAdjoin*

The field $\mathbf{C}_\tau$ *is an algebraic closure of* A_τ*.*

Proof. Uses: `lemma:weil-finite-subfield-target-transcendence-basis`

The target basis certificate supplies algebraicity over A_τ, while $\mathbf{C}$ is algebraically closed. Apply `IsAlgClosed.isAlgClosure_of_transcendence_basis`. □

Definition 8.1.51 (Global realization prescribed on a finite subfield)**.** Uses: `def:weil-finite-subfield-embedding-extension-candidate,def:weil-finite-subfield-adjoin-equivalence`

Lean: `Weil.Arithmetic.prescribedFiniteSubfieldGlobalRealization`

Constructed by: `lemma:weil-finite-subfield-source-is-algebraic-closure,lemma:weil-finite-subfield-target-is-algebraic-closure`

Extend $e_{L,\tau}$ across the two algebraic closures by setting

$$\widetilde{\tau}_{L,\tau} := \mathrm{IsAlgClosure.equivOfEquiv}(\overline{E}, \mathbf{C}, e_{L,\tau}) : \overline{E} \xrightarrow{\sim} \mathbf{C}.$$

This is a finite-subfield embedding extension candidate. It is selected noncomputably by the pinned algebraic-closure equivalence and is not asserted to equal the separately fixed embedding ι_0 or the earlier global choice $\jmath_0$.

Lemma 8.1.52 (Restriction of the prescribed global realization)**.** *Uses:* *def:weil-prescribed-finite-subfield-global-realization, def:weil-finite-subfield-embedding-extension-predicate*

Lean: *Weil.Arithmetic.prescribedFiniteSubfieldGlobalRealization_restrict*

For every admissible (L,τ) *and every* $x \in L$*,*

$$\widetilde{\tau}_{L,\tau}(x) = \tau(x).$$

Consequently $\mathrm{Extends}(\widetilde{\tau}_{L,\tau}, \tau)$*.*

Proof. Uses: `def:weil-finite-subfield-target-algebra,def:weil-finite-subfield-adjoin-equivalence,def:weil-prescribed-finite-subfield-global-realization`

The adjoin equivalence commutes with the L-algebra maps. Apply the pinned equation `IsAlgClosure.equivOfEquiv_algebraMap` to the image of x in A_L and then rewrite with `AlgEquiv.commutes`. The target algebra map $L \to \mathbf{C}_\tau$ is definitionally τ, giving the displayed equality. Equivalently, $\widetilde{\tau}_{L,\tau} \circ \jmath_0^{-1}$ is an automorphism of $\mathbf{C}$ whose restriction to $\jmath_0(L)$ is $\tau \circ (\jmath_0|_L)^{-1}$. □

Definition 8.1.53 (Selected finite-subfield complex extension)**.** Uses: `def:weil-finite-subfield-complex-extension-selection,def:weil-prescribed-finite-subfield-global-realization`

Lean: `Weil.Arithmetic.selectedFiniteSubfieldComplexExtension`

Define the dependent selection by

$$e_{\mathbf{C}}(L,\tau) := \widetilde{\tau}_{L,\tau}.$$

Each output is raw candidate data; the restriction certificate remains a separate proposition.

Lemma 8.1.54 (The selected finite-subfield extensions have the prescribed restrictions)**.** *Uses:* *def:weil-selected-finite-subfield-complex-extension, def:weil-finite-subfield-complex-extension-selection-predicate*

Lean: *Weil.Arithmetic.selectedFiniteSubfieldComplexExtension_spec*

The selected family satisfies

$$\mathrm{IsComplexExtensionSelection}(e_{\mathbf{C}}).$$

Proof. Uses: `lemma:weil-prescribed-finite-subfield-global-realization-restrict`

Fix an admissible pair (L,τ) and unfold the selected family. Its sole required field is exactly the pointwise restriction equation proved in the preceding lemma. □

Definition 8.1.55 (Varying complex-realization extension interface)**.** Uses: `def:weil-global-complex-realization-candidate,def:weil-finite-subfield-complex-extension-selection,def:weil-finite-subfield-complex-extension-selection-predicate,def:weil-global-complex-realization,def:weil-selected-finite-subfield-complex-extension`

Lean: `Weil.Arithmetic.VaryingComplexRealizationExtensionInterface`

Constructed by: `lemma:weil-selected-finite-subfield-complex-extension-spec`

A varying complex-realization extension interface consists of one global realization $\jmath_0$, a finite-subfield complex-extension selection $e_{\mathbf{C}}$, and a proof of IsComplexExtensionSelection($e_{\mathbf{C}}$). The preceding constructions supply these fields by the already selected global realization, the selected family $e_{\mathbf{C}}(L,\tau) = \widetilde{\tau}_{L,\tau}$, and its restriction certificate.

Definition 8.1.56 (Finite-subfield automorphism extension candidate). Uses: `def:weil-coefficient-algebraic-closure-data`

Lean: `Weil.Arithmetic.FiniteSubfieldAutomorphismExtensionCandidate`

For finite $\mathbf{Q}$-subfields $L, L' \subseteq \overline{E}$ and a $\mathbf{Q}$-embedding $s : L \hookrightarrow L'$, an automorphism-extension candidate is a specified $\sigma \in \mathrm{Aut}(\overline{E}/\mathbf{Q})$.

Definition 8.1.57 (Finite-subfield automorphism extension predicate). Uses: `def:weil-finite-subfield-automorphism-extension-candidate`

Lean: `Weil.Arithmetic.ExtendsFiniteSubfieldEmbeddingByAutomorphism`

Define AutExtends(σ, s) to mean that $\sigma|_L = s$ after the displayed inclusion $L' \subseteq \overline{E}$.

Definition 8.1.58 (Finite-subfield automorphism-extension selection). Uses: `def:weil-finite-subfield-automorphism-extension-candidate`

Lean: `Weil.Arithmetic.FiniteSubfieldAutomorphismExtensionSelection`

An automorphism-extension selection assigns to every admissible $s : L \hookrightarrow L'$ a candidate $e_{\mathrm{aut}}(L, L', s) \in \mathrm{Aut}(\overline{E}/\mathbf{Q})$. It contains no extension equation.

Definition 8.1.59 (Finite-subfield automorphism-extension selection predicate). Uses: `def:weil-finite-subfield-automorphism-extension-selection,def:weil-finite-subfield-automorphism-extension-predicate`

Lean: `Weil.Arithmetic.IsFiniteSubfieldAutomorphismExtensionSelection`

For a selection e_{aut}, define IsAutomorphismExtensionSelection(e_{aut}) to mean that

$$\mathrm{AutExtends}(e_{\mathrm{aut}}(L, L', s), s)$$

for every admissible triple (L, L', s).

Definition 8.1.60 (Reference complex embedding of the target finite subfield). Uses: `def:weil-global-complex-realization,def:weil-finite-subfield-automorphism-extension-candidate`

Lean: `Weil.Arithmetic.automorphismReferenceComplexEmbedding`

For an admissible triple (L, L', s), restrict the selected global realization $\jmath_0$ to L' and regard the result as a $\mathbf{Q}$-algebra embedding

$$\tau_{L'} : L' \hookrightarrow \mathbf{C}, \qquad \tau_{L'}(y) = \jmath_0(y).$$

At the pinned revision this is the composite of `IntermediateField.val` with `jmath_0.toRingHom`, followed by `RingHom.toRatAlgHom`. No new embedding is chosen.

Definition 8.1.61 (Reference complex embedding pulled back along the finite map). Uses: `def:weil-automorphism-reference-complex-embedding`

Lean: `Weil.Arithmetic.automorphismPulledComplexEmbedding`

Define the prescribed source embedding by

$$\tau_s := \tau_{L'} \circ s : L \hookrightarrow \mathbf{C}.$$

It is a $\mathbf{Q}$-algebra embedding because both factors are $\mathbf{Q}$-algebra embeddings.

Definition 8.1.62 (Source realization for the automorphism comparison). Uses: `def:weil-selected-finite-subfield-complex-extension,def:weil-automorphism-pulled-complex-embedding`

Lean: `Weil.Arithmetic.automorphismSourceComplexRealization`

Set

$$J_s := e_{\mathbf{C}}(L, \tau_s) : \overline{E} \xrightarrow{\sim} \mathbf{C}.$$

Lemma 8.1.63 (Source realization has the pulled-back restriction). Uses: `def:weil-automorphism-source-complex-realization`

Lean: `Weil.Arithmetic.automorphismSourceComplexRealization_restrict`

For every $x \in L$,

$$J_s(x) = \tau_s(x) = \tau_{L'}(s(x)).$$

Proof. Uses: `lemma:weil-selected-finite-subfield-complex-extension-spec`, `def:weil-automorphism-pulled-complex-embedding`

Specialize the selected finite-subfield complex-extension certificate to the admissible pair (L, τ_s), then unfold τ_s. □

Definition 8.1.64 (Target realization for the automorphism comparison). Uses: `def:weil-selected-finite-subfield-complex-extension`, `def:weil-automorphism-reference-complex-embedding`

Lean: `Weil.Arithmetic.automorphismTargetComplexRealization`

Set

$$J_{L'} := e_{\mathbf{C}}(L', \tau_{L'}) : \overline{E} \xrightarrow{\sim} \mathbf{C}.$$

Lemma 8.1.65 (Target realization has the reference restriction). *Uses:* *def:weil-automorphism-target-complex-realization*

Lean: *Weil.Arithmetic.automorphismTargetComplexRealization_restrict*

For every $y \in L'$,

$$J_{L'}(y) = \tau_{L'}(y).$$

Proof. Uses: `lemma:weil-selected-finite-subfield-complex-extension-spec`

This is the same certificate specialized to the admissible pair $(L', \tau_{L'})$. □

Definition 8.1.66 (Comparison ring automorphism of the coefficient closure). Uses: `def:weil-automorphism-source-complex-realization`, `def:weil-automorphism-target-complex-realization`

Lean: `Weil.Arithmetic.finiteSubfieldComparisonRingAutomorphism`

Define the ring equivalence

$$\rho_{L,L',s} := J_{L'}^{-1} \circ J_s : \overline{E} \xrightarrow{\sim} \overline{E}.$$

In Lean this is `J_s.trans J_L'.symm`; the order agrees with the displayed functional composite.

Lemma 8.1.67 (The comparison ring automorphism fixes the rational image). *Uses:* *def:weil-finite-subfield-comparison-ring-automorphism*

Lean: *Weil.Arithmetic.finiteSubfieldComparisonRingAutomorphism_fixesRat*

For every $r \in \mathbf{Q}$,

$$\rho_{L,L',s}(\mathrm{algebraMap}_{\mathbf{Q},\overline{E}}(r)) = \mathrm{algebraMap}_{\mathbf{Q},\overline{E}}(r).$$

Proof. Uses: `def:weil-finite-subfield-comparison-ring-automorphism`

Every ring homomorphism between characteristic-zero fields maps rational casts to rational casts. Apply the pinned equation `RingHom.map_rat_algebraMap` to the underlying ring homomorphism of $\rho_{L,L',s}$. □

Definition 8.1.68 (Prescribed finite-subfield automorphism). Uses: `def:weil-finite-subfield-automorphism-extension-candidate`, `def:weil-finite-subfield-comparison-ring-automorphism`

Lean: `Weil.Arithmetic.prescribedFiniteSubfieldAutomorphism`

Constructed by: `lemma:weil-finite-subfield-comparison-ring-automorphism-fixes-rationals`

Package $\rho_{L,L',s}$ with its rational-image equation by `AlgEquiv.ofRingEquiv`, obtaining

$$\sigma_{L,L',s} \in \mathrm{Aut}(\overline{E}/\mathbf{Q}).$$

Its underlying ring equivalence is definitionally $\rho_{L,L',s}$.

Lemma 8.1.69 (Restriction of the prescribed finite-subfield automorphism). *Uses:* *def:weil-prescribed-finite-subfield-automorphism*, *def:weil-finite-subfield-automorphism-extension-predicate*

Lean: *Weil.Arithmetic.prescribedFiniteSubfieldAutomorphism_restrict*

For every $x \in L$,

$$\sigma_{L,L',s}(x) = s(x)$$

after the inclusion $L' \subseteq \overline{E}$. *Hence* $\mathrm{AutExtends}(\sigma_{L,L',s}, s)$.

Proof. Uses: `lemma:weil-automorphism-source-complex-realization-restrict`, `lemma:weil-automorphism-target-complex-realization-restrict`, `def:weil-finite-subfield-comparison-ring-automorphism`

For $x \in L$, the two restriction certificates give

$$J_s(x) = \tau_{L'}(s(x)) = J_{L'}(s(x)).$$

Apply $J_{L'}^{-1}$ and use its inverse equation. The underlying map of $\sigma_{L,L',s}$ is $J_{L'}^{-1} \circ J_s$, so the resulting equality is exactly the displayed restriction equation. □

Definition 8.1.70 (Selected finite-subfield automorphism extension)**.** Uses: `def:weil-finite-subfield-automorphism-extension-selection`,`def:weil-prescribed-finite-subfield-automorphism`

Lean: `Weil.Arithmetic.selectedFiniteSubfieldAutomorphismExtension`

Define the dependent family

$$e_{\mathrm{aut}}(L, L', s) := \sigma_{L,L',s}.$$

It contains only candidate data; its restriction certificate remains a separate proposition.

Lemma 8.1.71 (The selected automorphisms have the prescribed restrictions)**.** *Uses:* *def:weil-selected-finite-subfield-automorphism-extension*, *def:weil-finite-subfield-automorphism-extension-selection-predicate*

Lean: *Weil.Arithmetic.selectedFiniteSubfieldAutomorphismExtension_spec*

The selected family satisfies

$$\mathrm{IsAutomorphismExtensionSelection}(e_{\mathrm{aut}}).$$

Proof. Uses: `lemma:weil-prescribed-finite-subfield-automorphism-restrict`

Fix an admissible triple and unfold the selected family. Its required predicate is precisely the preceding pointwise restriction theorem. □

Definition 8.1.72 (Finite-subfield automorphism-extension interface)**.** Uses: `def:weil-finite-subfield-automorphism-extension-selection`,`def:weil-finite-subfield-automorphism-extension-selection-predicate`,`def:weil-selected-finite-subfield-automorphism-extension`

Lean: `Weil.Arithmetic.FiniteSubfieldAutomorphismExtensionInterface`

Constructed by: `lemma:weil-selected-finite-subfield-automorphism-extension-spec`

A finite-subfield automorphism-extension interface consists of a selection e_{aut} and a proof of $\mathrm{IsAutomorphismExtensionSelection}(e_{\mathrm{aut}})$. The preceding definitions supply the selected comparison automorphism and its exact restriction certificate for every admissible triple.

Lemma 8.1.73 (A transcendental singleton is algebraically independent)**.** *Uses:* *def:weil-coefficient-algebraic-closure-data*

Lean: *Weil.Arithmetic.transcendentalSingleton_algebraicIndependent*

If $x \in \overline{E}$ is not algebraic over $\mathbf{Q}$, then the singleton set $\{x\}$, with its subtype inclusion into $\overline{E}$, is algebraically independent over $\mathbf{Q}$.

Proof. Uses: `def:weil-coefficient-algebraic-closure-data`

Rewrite nonalgebraicity as transcendence and apply the pinned equivalence `algebraicIndependent_singleton_iff`. □

Lemma 8.1.74 (Existence of a transcendence basis containing a prescribed element)**.** *Uses:* *def:weil-coefficient-algebraic-closure-data*

Lean: *Weil.Arithmetic.existsTranscendenceBasisContaining*

For $x \in \overline{E}$ transcendental over $\mathbf{Q}$, there is a set $B_x \subseteq \overline{E}$ such that $x \in B_x$ and the subtype inclusion

$$v_x : B_x \longrightarrow \overline{E}$$

is a transcendence basis of $\overline{E}/\mathbf{Q}$.

Proof. Uses: `lemma:weil-transcendental-singleton-algebraically-independent`

Apply `exists_isTranscendenceBasis_superset` to the algebraically independent singleton $\{x\}$. Its subset certificate supplies $x \in B_x$, while the second output is the required basis certificate. □

Definition 8.1.75 (Selected transcendence basis containing the input)**.** Uses: `def:weil-coefficient-algebraic-closure-data`

Lean: `Weil.Arithmetic.transcendenceBasisContaining`

Constructed by: `lemma:weil-transcendental-basis-containing-exists`

For $x \in \overline{E}$ and a proof h_x that x is transcendental over $\mathbf{Q}$, choose the set B_x from the preceding existence theorem. The choice depends on both x and h_x and is not the fixed basis $\overline{B}_\ell$ selected earlier.

Lemma 8.1.76 (The selected pointed basis contains its input)**.** *Uses:* *def:weil-transcendental-basis-containing*

Lean: *Weil.Arithmetic.mem_transcendenceBasisContaining*

The selected set satisfies $x \in B_x$.

Proof. Uses: lemma:weil-transcendental-basis-containing-exists
This is the first projection of the selected existence witness. □

Lemma 8.1.77 (The selected pointed family is a transcendence basis). *Uses:* def:weil-transcendental-basis-containing

Lean: Weil.Arithmetic.transcendenceBasisContaining_isBasis
The subtype inclusion $v_x : B_x \to \overline{E}$ is a transcendence basis of $\overline{E}/\mathbf{Q}$.

Proof. Uses: lemma:weil-transcendental-basis-containing-exists
This is the second projection of the selected existence witness. □

Definition 8.1.78 (Distinguished coordinate in the pointed basis). Uses: def:weil-transcendental-basis-containing

Lean: Weil.Arithmetic.transcendenceBasisDistinguishedCoordinate

Constructed by: lemma:weil-transcendental-basis-containing-mem
Let

$$x_B := \langle x,\ x \in B_x \rangle : B_x.$$

Then $v_x(x_B) = x$ definitionally.

Lemma 8.1.79 (Cardinality of a pointed transcendence basis). *Uses:* def:weil-transcendental-basis-containing

Lean: Weil.Arithmetic.mk_transcendenceBasisContaining_eq_continuum
The selected pointed basis has cardinality

$$\#B_x = \mathfrak{c}.$$

Proof. Uses: lemma:weil-transcendental-basis-containing-is-basis,prop:weil-coefficient-closure-transcendence-degree-continuum
Apply IsTranscendenceBasis.cardinalMk_eq_trdeg to v_x and rewrite the transcendence degree by the established continuum calculation. □

Lemma 8.1.80 (A pointed basis has a second coordinate). *Uses:* def:weil-transcendental-basis-distinguished-coordinate

Lean: Weil.Arithmetic.exists_transcendenceBasisSecondCoordinate
There is $b_B \in B_x$ with $b_B \neq x_B$.

Proof. Uses: lemma:weil-transcendental-basis-containing-cardinal
Rewrite $\#B_x = \mathfrak{c}$ in Cardinal.nat_lt_continuum 1. The resulting inequality $1 < \#B_x$ gives a Nontrivial instance through Cardinal.one_lt_iff_nontrivial; now apply exists_ne to x_B. □

Definition 8.1.81 (Selected second coordinate). Uses: def:weil-transcendental-basis-containing,def:weil-transcendental-basis-distinguished-coordinate

Lean: Weil.Arithmetic.transcendenceBasisSecondCoordinate

Constructed by: lemma:weil-transcendental-basis-second-coordinate-exists
Choose one $b_B \in B_x$ different from x_B.

Lemma 8.1.82 (The selected second coordinate is distinct). *Uses:* def:weil-transcendental-basis-second-coordinate

Lean: Weil.Arithmetic.transcendenceBasisSecondCoordinate_ne_distinguished
One has $b_B \neq x_B$.

Proof. Uses: lemma:weil-transcendental-basis-second-coordinate-exists
This is the certificate retained by the classical choice. □

Definition 8.1.83 (Ambient value of the selected second coordinate). Uses: def:weil-transcendental-basis-second-coordinate

Lean: Weil.Arithmetic.transcendenceBasisSecondValue
Set $b := v_x(b_B) \in \overline{E}$.

Lemma 8.1.84 (The selected second value differs from the input). *Uses:* *def:weil-transcendental-basis-second-value*, *def:weil-transcendental-basis-distinguished-coordinate*

Lean: *Weil.Arithmetic.transcendenceBasisSecondValue_ne*

One has $b \neq x$.

Proof. Uses: `lemma:weil-transcendental-basis-second-coordinate-certificate`

This follows from $b_B \neq x_B$ and injectivity of the subtype inclusion v_x. □

Definition 8.1.85 (Swap of the two pointed-basis coordinates). Uses: `def:weil-transcendental-basis-distinguished-coordinate`, `def:weil-transcendental-basis-second-coordinate`

Lean: `Weil.Arithmetic.transcendenceBasisSwap`

Using classical decidable equality on B_x, define

$$\pi_x := \mathrm{Equiv.swap}(x_B, b_B) : B_x \simeq B_x.$$

Lemma 8.1.86 (Exact equation for the distinguished swapped coordinate). *Uses:* *def:weil-transcendental-basis-swap*

Lean: *Weil.Arithmetic.transcendenceBasisSwap_apply_distinguished*

The transposition satisfies $\pi_x(x_B) = b_B$.

Proof. Uses: `def:weil-transcendental-basis-swap`

Apply `Equiv.swap_apply_left`. □

Definition 8.1.87 (Reindexed pointed-basis family). Uses: `def:weil-transcendental-basis-swap`, `def:weil-transcendental-basis-containing`

Lean: `Weil.Arithmetic.transcendenceBasisReindexedFamily`

Define

$$w_x := v_x \circ \pi_x : B_x \longrightarrow \overline{E}.$$

Lemma 8.1.88 (The reindexed family is algebraically independent). *Uses:* *def:weil-transcendental-reindexed-basis-family*

Lean: *Weil.Arithmetic.transcendenceBasisReindexedFamily_algebraicIndependent*

The family w_x is algebraically independent over $\mathbf{Q}$.

Proof. Uses: `lemma:weil-transcendental-basis-containing-is-basis`, `def:weil-transcendental-basis-swap`

Compose the algebraic-independence component of the basis certificate with the injective map π_x, using `AlgebraicIndependent.comp`. □

Lemma 8.1.89 (Reindexing does not change the generated field). *Uses:* *def:weil-transcendental-reindexed-basis-family*

Lean: *Weil.Arithmetic.range_transcendenceBasisReindexedFamily*

The two families have identical ranges:

$$\mathrm{range}(w_x) = \mathrm{range}(v_x).$$

Proof. Uses: `def:weil-transcendental-basis-swap`

The permutation π_x is surjective; apply the elementary range-of-a-surjective-composite equality. □

Lemma 8.1.90 (The reindexed family generates the same intermediate field). *Uses:* *def:weil-transcendental-reindexed-basis-family*, *def:weil-transcendental-basis-containing*

Lean: *Weil.Arithmetic.adjoin_transcendenceBasisReindexedFamily*

The generated intermediate fields agree:

$$\mathrm{IntermediateField.adjoin}_{\overline{E}}(\mathbf{Q}, \mathrm{range}(w_x)) = \mathrm{IntermediateField.adjoin}_{\overline{E}}(\mathbf{Q}, \mathrm{range}(v_x)).$$

Proof. Uses: `lemma:weil-transcendental-reindexed-basis-range`

Apply congruence of `IntermediateField.adjoin` to the range equality. □

Definition 8.1.91 (Rational-function field of the pointed basis)**.** Uses: `def:weil-transcendental-basis-containing`

Lean: `Weil.Arithmetic.transcendenceBasisRationalFunctionField`

Set

$$F_x := \mathbf{Q}(B_x) := \mathrm{IntermediateField.adjoin}_{\overline{E}}\big(\mathbf{Q}, \mathrm{range}(v_x)\big).$$

Definition 8.1.92 (Distinguished coordinate in the rational-function field)**.** Uses: `def:weil-transcendental-basis-rational-function-field,def:weil-transcendental-basis-distinguished-coordinate`

Lean: `Weil.Arithmetic.transcendenceBasisRationalFunctionXCoordinate`

Let $x_F \in F_x$ be the element whose ambient value is x.

Definition 8.1.93 (Second coordinate in the rational-function field)**.** Uses: `def:weil-transcendental-basis-rational-function-field,def:weil-transcendental-basis-second-coordinate,def:weil-transcendental-basis-second-value`

Lean: `Weil.Arithmetic.transcendenceBasisRationalFunctionBCoordinate`

Let $b_F \in F_x$ be the element whose ambient value is b.

Definition 8.1.94 (Rational-function equivalence induced by the swap)**.** Uses: `def:weil-transcendental-basis-rational-function-field,def:weil-transcendental-reindexed-basis-family`

Lean: `Weil.Arithmetic.transcendenceBasisRationalFunctionSwap`

Constructed by: `lemma:weil-transcendental-reindexed-basis-independent,lemma:weil-transcendental-reindexed-basis-adjoin`

Let A_v and A_w be the canonical equivalences from $\mathrm{Frac}(\mathbf{Q}[X_i \mid i \in B_x])$ to the intermediate fields generated by v_x and w_x, respectively, supplied by `AlgebraicIndependent.aevalEquivField`. Transport the target of A_w to F_x using `IntermediateField.equivOfEq` and the preceding range equality. Define

$$e_{\pi,x} := A_v^{-1}\,;\, A_w\,;\, \mathrm{equivOfEq} : F_x \xrightarrow{\sim}_{\mathbf{Q}} F_x.$$

Lemma 8.1.95 (Exact coordinate equation for the rational-function swap)**.** *Uses:* *def:weil-transcendental-basis-rational-function-swap, def:weil-transcendental-basis-rational-function-x-coordinate, def:weil-transcendental-basis-rational-function-b-coordinate*

Lean: *Weil.Arithmetic.transcendenceBasisRationalFunctionSwap_apply_distinguished*

The induced equivalence has the exact value

$$e_{\pi,x}(x_F) = b_F.$$

Proof. Uses: `lemma:weil-transcendental-basis-swap-apply-distinguished,def:weil-transcendental-reindexed-basis-family`

Represent x_F by the rational-function coordinate X_{x_B}. Apply `AlgebraicIndependent.aevalEquivField_algebraMap_apply_coe` first to A_v and then to A_w. The target evaluation is $w_x(x_B) = v_x(\pi_x(x_B)) = v_x(b_B)$ by the exact swap equation; `IntermediateField.equivOfEq_apply` leaves the underlying element unchanged. □

Lemma 8.1.96 (The coefficient closure is an algebraic closure of the pointed rational-function field)**.** *Uses:* *def:weil-transcendental-basis-rational-function-field*

Lean: *Weil.Arithmetic.coefficientClosure_isAlgClosure_transcendenceBasisRationalFunctionField*

The field $\overline{E}$ is an algebraic closure of F_x.

Proof. Uses: `lemma:weil-transcendental-basis-containing-is-basis,def:weil-coefficient-algebraic-closure-data`

The pinned theorem `IsTranscendenceBasis.isAlgebraic_field` says that $\overline{E}/F_x$ is algebraic. The coefficient-closure datum supplies that $\overline{E}$ is algebraically closed. These are precisely the two fields of the `IsAlgClosure` instance. □

Definition 8.1.97 (Ring automorphism extending the pointed-basis swap)**.** Uses: `def:weil-transcendental-basis-rational-function-swap,def:weil-transcendental-basis-rational-function-field`

Lean: `Weil.Arithmetic.transcendenceBasisRingAutomorphism`

Constructed by: `lemma:weil-transcendental-basis-rational-function-is-algebraic-closure`

Apply algebraic-closure uniqueness to the same source and target closure:

$$\rho_x := \mathrm{IsAlgClosure.equivOfEquiv}(\overline{E}, \overline{E}, e_{\pi,x}) : \overline{E} \xrightarrow{\sim} \overline{E}.$$

This is a ring equivalence whose restriction to F_x is $e_{\pi,x}$; it is not an F_x-algebra equivalence.

Lemma 8.1.98 (Exact movement equation for the extended ring automorphism). *Uses:* def:weil-transcendental-basis-ring-automorphism

Lean: Weil.Arithmetic.transcendenceBasisRingAutomorphism_apply_distinguished

The extension satisfies

$$\rho_x(x) = b.$$

Proof. Uses: lemma:weil-transcendental-basis-rational-function-swap-coordinate

Write $x = \mathrm{algebraMap}_{F_x,\overline{E}}(x_F)$. The pinned equation `IsAlgClosure.equivOfEquiv_algebraMap` rewrites its image as $\mathrm{algebraMap}_{F_x,\overline{E}}(e_{\pi,x}(x_F))$. Substitute the exact rational-function coordinate equation to obtain b. □

Lemma 8.1.99 (The extended ring automorphism fixes rational scalars). *Uses:* def:weil-transcendental-basis-ring-automorphism

Lean: Weil.Arithmetic.transcendenceBasisRingAutomorphism_fixesRat

For every $r \in \mathbf{Q}$,

$$\rho_x\big(\mathrm{algebraMap}_{\mathbf{Q},\overline{E}}(r)\big) = \mathrm{algebraMap}_{\mathbf{Q},\overline{E}}(r).$$

Proof. Uses: def:weil-transcendental-basis-ring-automorphism

Apply `RingHom.map_rat_algebraMap` to the underlying ring homomorphism of ρ_x. □

Definition 8.1.100 (Rational automorphism moving the transcendental input). Uses: def:weil-transcendental-basis-ring-automorphism

Lean: Weil.Arithmetic.transcendenceBasisQAutomorphism

Constructed by: lemma:weil-transcendental-basis-ring-automorphism-fixes-rationals

Package ρ_x with its rational-image equation using `AlgEquiv.ofRingEquiv`. This gives

$$\sigma_x \in \mathrm{Aut}(\overline{E}/\mathbf{Q})$$

with underlying map definitionally equal to ρ_x.

Lemma 8.1.101 (Exact coordinate equation for the selected rational automorphism). *Uses:* def:weil-transcendental-basis-q-automorphism

Lean: Weil.Arithmetic.transcendenceBasisQAutomorphism_apply

The selected automorphism satisfies the exact equation

$$\sigma_x(x) = b.$$

Proof. Uses: lemma:weil-transcendental-basis-ring-automorphism-coordinate

This is the ring-automorphism coordinate equation because `AlgEquiv.ofRingEquiv` preserves the underlying map. □

Lemma 8.1.102 (The selected rational automorphism moves the input). *Uses:* def:weil-transcendental-basis-q-automorphism

Lean: Weil.Arithmetic.transcendenceBasisQAutomorphism_moves

The selected automorphism satisfies $\sigma_x(x) \neq x$.

Proof. Uses: lemma:weil-transcendental-basis-q-automorphism-coordinate, lemma:weil-transcendental-basis-second-value-ne

Substitute the exact coordinate equation and apply $b \neq x$. □

Theorem 8.1.103 (Every transcendental scalar is moved by a rational automorphism). *Uses:* def:weil-coefficient-algebraic-closure-data

Lean: Weil.Arithmetic.exists_qAlgAutomorphism_ne_of_not_isAlgebraic

For every $x \in \overline{E}$,

$$\neg\,\mathrm{IsAlgebraic}_{\mathbf{Q}}(x) \quad\Longrightarrow\quad \exists\,\sigma \in \mathrm{Aut}(\overline{E}/\mathbf{Q}),\ \sigma(x) \neq x.$$

Proof. Uses: def:weil-transcendental-basis-q-automorphism, lemma:weil-transcendental-basis-q-automorphism-moves

Given the nonalgebraicity certificate, take the selected automorphism σ_x. Its movement lemma supplies the required inequality. □

Lemma 8.1.104 (An algebraic scalar is integral). *Uses:* `def:weil-coefficient-algebraic-closure-data`

Lean: `Weil.Arithmetic.isIntegral_of_isAlgebraic`

For $x \in \overline{E}$, a certificate $\mathrm{IsAlgebraic}_{\mathbf{Q}}(x)$ *gives* $\mathrm{IsIntegral}_{\mathbf{Q}}(x)$.

Proof. Uses: `def:weil-coefficient-algebraic-closure-data`

Apply the forward implication of the pinned equivalence `isAlgebraic_iff_isIntegral`, equivalently the method `IsAlgebraic.isIntegral`. □

Lemma 8.1.105 (A nonrational scalar is outside the bottom intermediate field). *Uses:* `def:weil-coefficient-algebraic-closure-data`

Lean: `Weil.Arithmetic.not_mem_bot_of_not_mem_rationalImage`

If there is no $r \in \mathbf{Q}$ with $\mathrm{algebraMap}_{\mathbf{Q},\overline{E}}(r) = x$, *then*

$$x \notin (\bot : \mathrm{IntermediateField}_{\mathbf{Q}}(\overline{E})).$$

Proof. Uses: `def:weil-coefficient-algebraic-closure-data`

The pinned theorem `IntermediateField.mem_bot` identifies membership in the bottom intermediate field with membership in the range of $\mathrm{algebraMap}_{\mathbf{Q},\overline{E}}$. Unfold range membership and apply the supplied negation. □

Lemma 8.1.106 (The nonrational algebraic minimal polynomial has degree at least two). *Uses:* `def:weil-coefficient-algebraic-closure-data`

Lean: `Weil.Arithmetic.two_le_natDegree_minpoly_of_algebraic_not_rational`

If x is algebraic over $\mathbf{Q}$ but is not in the rational image, then

$$2 \le \mathrm{natDegree}(\mathrm{minpoly}_{\mathbf{Q}}(x)).$$

Proof. Uses: `lemma:weil-algebraic-scalar-integral`, `lemma:weil-nonrational-scalar-not-mem-bottom`

Use the integral certificate in the exact equivalence `minpoly.two_le_natDegree_iff`. Its right side is nonmembership in the range of $\mathrm{algebraMap}_{\mathbf{Q},\overline{E}}$, which is the preceding nonrationality statement. □

Lemma 8.1.107 (The algebraic minimal polynomial is separable). *Uses:* `def:weil-coefficient-algebraic-closure-data`

Lean: `Weil.Arithmetic.isSeparable_of_isAlgebraic_rat`

If x is algebraic over $\mathbf{Q}$, then $\mathrm{minpoly}_{\mathbf{Q}}(x)$ *is separable; in other words,* $\mathrm{IsSeparable}_{\mathbf{Q}}(x)$.

Proof. Uses: `lemma:weil-algebraic-scalar-integral`

The field $\mathbf{Q}$ has characteristic zero, hence carries the pinned `PerfectField.ofCharZero` instance. Apply `PerfectField.separable_of_irreducible` to `minpoly.irreducible` of the integral certificate. □

Lemma 8.1.108 (The algebraic minimal polynomial splits in the coefficient closure). *Uses:* `def:weil-coefficient-algebraic-closure-data`

Lean: `Weil.Arithmetic.minpoly_map_splits_coefficientClosure`

For algebraic $x \in \overline{E}$, the polynomial

$$\mathrm{minpoly}_{\mathbf{Q}}(x)\ .\ \mathrm{map}\ \mathrm{algebraMap}_{\mathbf{Q},\overline{E}}$$

splits over $\overline{E}$.

Proof. Uses: `def:weil-coefficient-algebraic-closure-data`

The coefficient-closure datum supplies `IsAlgClosed` for $\overline{E}$. Apply `IsAlgClosed.splits` to the displayed mapped polynomial. □

Lemma 8.1.109 (Root-set cardinality equals the minimal-polynomial degree). *Uses:* `def:weil-coefficient-algebraic-closure-data`

Lean: `Weil.Arithmetic.card_rootSet_minpoly_eq_natDegree`

For algebraic $x \in \overline{E}$,

$$\#\,\mathrm{rootSet}_{\overline{E}}(\mathrm{minpoly}_{\mathbf{Q}}(x)) = \mathrm{natDegree}(\mathrm{minpoly}_{\mathbf{Q}}(x)).$$

Proof. Uses: `lemma:weil-algebraic-scalar-minpoly-separable`, `lemma:weil-algebraic-scalar-minpoly-splits`

Apply the pinned equality `Polynomial.card_rootSet_eq_natDegree` to the separability and splitting certificates. □

Lemma 8.1.110 (The root set is nontrivial). *Uses:* `def:weil-coefficient-algebraic-closure-data`

Lean: `Weil.Arithmetic.nontrivial_rootSet_minpoly_of_algebraic_not_rational`

For algebraic nonrational x, the finite type $\mathrm{rootSet}_{\overline{E}}(\mathrm{minpoly}_{\mathbf{Q}}(x))$ is nontrivial.

Proof. Uses: `lemma:weil-algebraic-nonrational-minpoly-degree`, `lemma:weil-algebraic-scalar-rootset-cardinality`

Rewrite the root-set cardinality by the preceding equality. The lower bound $2 \le \mathrm{natDegree}$ gives cardinality strictly larger than one; apply `Fintype.one_lt_card_iff_nontrivial`. □

Lemma 8.1.111 (The original scalar is in its minimal-polynomial root set). *Uses:* `def:weil-coefficient-algebraic-closure-data`

Lean: `Weil.Arithmetic.mem_rootSet_minpoly_self`

For algebraic $x \in \overline{E}$,

$$x \in \mathrm{rootSet}_{\overline{E}}(\mathrm{minpoly}_{\mathbf{Q}}(x)).$$

Proof. Uses: `lemma:weil-algebraic-scalar-integral`

Apply `Polynomial.mem_rootSet.mpr` to `minpoly.ne_zero` and `minpoly.aeval`. □

Lemma 8.1.112 (Existence of a distinct root-set coordinate). *Uses:* `def:weil-coefficient-algebraic-closure-data`

Lean: `Weil.Arithmetic.exists_ne_rootSetCoordinate_minpoly`

For algebraic nonrational x, there is

$$\widehat{y} \in \mathrm{rootSet}_{\overline{E}}(\mathrm{minpoly}_{\mathbf{Q}}(x)) \quad \textit{with} \quad (\widehat{y} : \overline{E}) \neq x.$$

Proof. Uses: `lemma:weil-algebraic-nonrational-rootset-nontrivial`, `lemma:weil-algebraic-scalar-rootset-self`

Regard x as the root-set subtype element supplied by the self-root lemma. Apply `nontrivial_iff_exists_ne` at that element and transport subtype inequality with `Subtype.coe_ne_coe`. □

Definition 8.1.113 (Selected distinct conjugate root coordinate). Uses: `def:weil-coefficient-algebraic-closure-data`

Lean: `Weil.Arithmetic.distinctRootSetCoordinate`

Constructed by: `lemma:weil-algebraic-nonrational-distinct-root-exists`

For algebraic nonrational x, choose a root-set coordinate $\widehat{y}_x$ satisfying the preceding existence statement.

Definition 8.1.114 (Selected ambient conjugate root). Uses: `def:weil-algebraic-nonrational-distinct-root-coordinate`

Lean: `Weil.Arithmetic.distinctConjugateRoot`

Define

$$y_x := (\widehat{y}_x : \overline{E}).$$

Lemma 8.1.115 (The selected ambient root is distinct). *Uses:* `def:weil-algebraic-nonrational-distinct-root`

Lean: `Weil.Arithmetic.distinctConjugateRoot_ne`

The selected root satisfies $y_x \neq x$.

Proof. Uses: `lemma:weil-algebraic-nonrational-distinct-root-exists`

This is the inequality component of the selected witness, after unfolding y_x. □

Lemma 8.1.116 (The selected root is a conjugate of the input). *Uses:* `def:weil-algebraic-nonrational-distinct-root`

Lean: `Weil.Arithmetic.distinctConjugateRoot_isConjRoot`

The selected root satisfies

$$\mathrm{IsConjRoot}_{\mathbf{Q}}(x, y_x).$$

Proof. Uses: lemma:weil-algebraic-scalar-integral, def:weil-algebraic-nonrational-distinct-root-coordinate

The subtype membership of $\widehat{y_x}$ says that y_x lies in the minimal-polynomial root set. Apply the reverse implication of `isConjRoot_iff_mem_minpoly_rootSet` with the integral certificate. □

Definition 8.1.117 (Simple source field of the algebraic input). Uses: def:weil-coefficient-algebraic-closure-data

Lean: `Weil.Arithmetic.algebraicMoverSourceField`

For algebraic nonrational x, set

$$L_x := \mathbf{Q}\langle x\rangle = \mathrm{IntermediateField.adjoin}_{\overline{E}}(\mathbf{Q}, \{x\}).$$

Lemma 8.1.118 (The simple source field is finite). *Uses: def:weil-algebraic-nonrational-source-field*

Lean: `Weil.Arithmetic.algebraicMoverSourceField_finiteDimensional`

The extension $L_x/\mathbf{Q}$ is finite-dimensional.

Proof. Uses: lemma:weil-algebraic-scalar-integral

Apply `IntermediateField.adjoin.finiteDimensional` to the integral certificate. □

Definition 8.1.119 (Ambient simple-field embedding at the selected root). Uses: def:weil-algebraic-nonrational-source-field, def:weil-algebraic-nonrational-distinct-root

Lean: `Weil.Arithmetic.algebraicMoverAmbientEmbedding`

Constructed by: lemma:weil-algebraic-nonrational-distinct-root-conjugate

The conjugacy certificate puts y_x in $\mathrm{aroots}_{\overline{E}}(\mathrm{minpoly}_{\mathbf{Q}}(x))$ via `isConjRoot_iff_mem_minpoly_aroots`. Define

$$f_x := \big(\mathrm{algHomAdjoinIntegralEquiv}(\mathbf{Q}, h_x)\big)^{-1}(y_x) : L_x \longrightarrow_{\mathbf{Q}} \overline{E},$$

where h_x is the integral certificate and y_x carries that root membership proof.

Lemma 8.1.120 (Exact generator equation for the ambient embedding). *Uses: def:weil-algebraic-nonrational-ambient-embedding*

Lean: `Weil.Arithmetic.algebraicMoverAmbientEmbedding_gen`

For the canonical generator $x_L := \mathrm{AdjoinSimple.gen}(\mathbf{Q}, x)$ of L_x,

$$f_x(x_L) = y_x.$$

Proof. Uses: def:weil-algebraic-nonrational-ambient-embedding

This is the exact pinned equation `algHomAdjoinIntegralEquiv_symm_apply_gen`. □

Definition 8.1.121 (Finite target field of the algebraic embedding). Uses: def:weil-algebraic-nonrational-ambient-embedding

Lean: `Weil.Arithmetic.algebraicMoverTargetField`

Set

$$L'_x := \mathrm{AlgHom.fieldRange}(f_x) \subseteq \overline{E}.$$

Lemma 8.1.122 (The target field is finite). *Uses: def:weil-algebraic-nonrational-target-field*

Lean: `Weil.Arithmetic.algebraicMoverTargetField_finiteDimensional`

The extension $L'_x/\mathbf{Q}$ is finite-dimensional.

Proof. Uses: lemma:weil-algebraic-nonrational-source-field-finite

Apply the pinned instance `f_x.toLinearMap.finiteDimensional_range`. The generated theorem `AlgHom.fieldRange_toSubalgebra` identifies the underlying subalgebra of L'_x with `f_x.range`; transport the finite-dimensional module instance across that identification. This is the same typed source pattern used in the pinned construction of finite-dimensional normal closures. □

Definition 8.1.123 (Embedding between the two finite subfields). Uses: def:weil-algebraic-nonrational-ambient-embedding, def:weil-algebraic-nonrational-target-field

Lean: `Weil.Arithmetic.algebraicMoverFiniteFieldEmbedding`

Define the range-restricted embedding

$$s_x := \mathrm{AlgHom.equivFieldRange}(f_x).\mathrm{toAlgHom} : L_x \longrightarrow_{\mathbf{Q}} L'_x.$$

Lemma 8.1.124 (Exact generator equation for the finite-field embedding). *Uses:* *def:weil-algebraic-nonrational-finite-field-embedding*

Lean: *Weil.Arithmetic.algebraicMoverFiniteFieldEmbedding_gen*

After the inclusion $L'_x \subseteq \overline{E}$, the generator equation is

$$(s_x(x_L) : \overline{E}) = y_x.$$

Proof. Uses: lemma:weil-algebraic-nonrational-ambient-embedding-generator

Use `AlgHom.equivFieldRange_apply_coe` and substitute the exact ambient-embedding generator equation. □

Definition 8.1.125 (Selected automorphism for the algebraic input). Uses: def:weil-selected-finite-subfield-automorphism-extension,def:weil-algebraic-nonrational-finite-field-embedding

Lean: `Weil.Arithmetic.algebraicMoverQAutomorphism`

Constructed by: lemma:weil-algebraic-nonrational-source-field-finite,lemma:weil-algebraic-nonrational-target-field-finite

Apply the already constructed finite-subfield selection to the admissible triple (L_x, L'_x, s_x) and define

$$\sigma_x^{\mathrm{alg}} := e_{\mathrm{aut}}(L_x, L'_x, s_x) \in \mathrm{Aut}(\overline{E}/\mathbf{Q}).$$

Lemma 8.1.126 (Exact movement equation for the algebraic automorphism). *Uses:* *def:weil-algebraic-nonrational-q-automorphism*

Lean: *Weil.Arithmetic.algebraicMoverQAutomorphism_apply*

The selected automorphism satisfies

$$\sigma_x^{\mathrm{alg}}(x) = y_x.$$

Proof. Uses: lemma:weil-selected-finite-subfield-automorphism-extension-spec,lemma:weil-algebraic-nonrational-finite-field-embedding-generator

Specialize the finite-subfield restriction certificate to the generator $x_L \in L_x$. Rewrite its ambient value by `AdjoinSimple.coe_gen` and rewrite the target value by the exact finite-field embedding equation. □

Lemma 8.1.127 (The selected algebraic automorphism moves the input). *Uses:* *def:weil-algebraic-nonrational-q-automorphism*

Lean: *Weil.Arithmetic.algebraicMoverQAutomorphism_moves*

The selected automorphism satisfies $\sigma_x^{\mathrm{alg}}(x) \neq x$.

Proof. Uses: lemma:weil-algebraic-nonrational-q-automorphism-coordinate,lemma:weil-algebraic-nonrational-distinct-root-ne

Substitute the exact movement equation and apply $y_x \neq x$. □

Theorem 8.1.128 (Every nonrational algebraic scalar is moved). *Uses:* *def:weil-coefficient-algebraic-closure-data*

Lean: *Weil.Arithmetic.exists_qAlgAutomorphism_ne_of_isAlgebraic_not_rational*

For every $x \in \overline{E}$, if x is algebraic over $\mathbf{Q}$ and is not in the image of $\mathbf{Q}$, then

$$\exists\, \sigma \in \mathrm{Aut}(\overline{E}/\mathbf{Q}), \qquad \sigma(x) \neq x.$$

Proof. Uses: def:weil-algebraic-nonrational-q-automorphism,lemma:weil-algebraic-nonrational-q-automorphism-moves

Take σ_x^{alg} from the selected finite-subfield extension and apply its movement lemma. □

Definition 8.1.129 (Rational fixed-field predicate). Uses: def:weil-coefficient-algebraic-closure-data

Lean: `Weil.Arithmetic.HasRationalAbsoluteFixedField`

Define RatFixed$(\overline{E})$ to mean

$$\big(\forall \sigma \in \mathrm{Aut}(\overline{E}/\mathbf{Q}),\ \sigma(x) = x\big) \quad \Longleftrightarrow \quad x \in \mathrm{im}(\mathbf{Q} \hookrightarrow \overline{E})$$

for every $x \in \overline{E}$.

Definition 8.1.130 (Fixed-to-rational direction)**.** Uses: `def:weil-coefficient-algebraic-closure-data`

Lean: `Weil.Arithmetic.FixedToRational`

Define FixedToRat($\overline{E}$) to mean that for every $x \in \overline{E}$,

$$\big(\forall \sigma \in \mathrm{Aut}(\overline{E}/\mathbf{Q}),\ \sigma(x) = x\big) \quad \Longrightarrow \quad x \in \mathrm{im}(\mathbf{Q} \hookrightarrow \overline{E}).$$

This is only the difficult implication in RatFixed($\overline{E}$).

Lemma 8.1.131 (Rational scalars are fixed by absolute automorphisms)**.** *Uses:* *def:weil-coefficient-algebraic-closure-data*

Lean: *Weil.Arithmetic.rationalImage_fixed*

For $r \in \mathbf{Q}$ and $\sigma \in \mathrm{Aut}(\overline{E}/\mathbf{Q})$,

$$\sigma\big(\mathrm{map}_{\mathbf{Q},\overline{E}}(r)\big) = \mathrm{map}_{\mathbf{Q},\overline{E}}(r).$$

Proof. Uses: `def:weil-coefficient-algebraic-closure-data`

This is the base-field compatibility equation for a $\mathbf{Q}$-algebra automorphism, namely the algebra-equivalence law `commutes`. □

Theorem 8.1.132 (Every absolutely fixed scalar is rational)**.** *Uses:* *def:weil-fixed-to-rational-predicate*

Lean: *Weil.Arithmetic.fixedToRational_coefficientClosure*

The coefficient closure satisfies FixedToRat($\overline{E}$).

Proof. Uses: `thm:weil-transcendental-scalar-moved-by-rational-automorphism`, `thm:weil-algebraic-nonrational-scalar-moved-by-rational-automorphism`

Fix $x \in \overline{E}$ and suppose that every $\mathbf{Q}$-algebra automorphism fixes x. Assume for contradiction that x is not in the rational image, and split classically on $\mathrm{IsAlgebraic}_{\mathbf{Q}}(x)$.

In the algebraic branch, the algebraic nonrational mover supplies σ with $\sigma(x) \neq x$, contradicting the fixedness hypothesis. In the nonalgebraic branch, the transcendental mover supplies the same contradiction. Hence x lies in the image of $\mathrm{algebraMap}_{\mathbf{Q},\overline{E}}$, exactly as required by FixedToRat($\overline{E}$). □

Lemma 8.1.133 (The two fixed-field directions assemble)**.** *Uses:* *def:weil-fixed-to-rational-predicate*, *def:weil-rational-fixed-field-predicate*

Lean: *Weil.Arithmetic.fixedToRational_iff_rationalFixedField*

If FixedToRat($\overline{E}$) *is supplied, then* RatFixed($\overline{E}$) *holds.*

Proof. Uses: `def:weil-fixed-to-rational-predicate`, `lemma:weil-rational-image-fixed-by-absolute-automorphisms`

The forward implication is the supplied fixed-to-rational direction. For the reverse implication, write x as the image of $r \in \mathbf{Q}$ and apply the rational-image fixed lemma to every automorphism. □

Definition 8.1.134 (Rational fixed-scalar retraction candidate)**.** Uses: `def:weil-coefficient-algebraic-closure-data`

Lean: `Weil.Arithmetic.RationalFixedScalarRetractionCandidate`

A rational fixed-scalar retraction candidate is a dependent function

$$\mathrm{rat}_0 : \prod_{x \in \overline{E}} \big(\forall \sigma \in \mathrm{Aut}(\overline{E}/\mathbf{Q}), \sigma(x) = x\big) \longrightarrow \mathbf{Q}.$$

Definition 8.1.135 (Rational fixed-scalar retraction predicate)**.** Uses: `def:weil-rational-fixed-scalar-retraction-candidate`

Lean: `Weil.Arithmetic.IsRationalFixedScalarRetraction`

Define IsRatRetraction(rat_0) to mean

$$\mathrm{map}_{\mathbf{Q},\overline{E}}\big(\mathrm{rat}_0(x, h_x)\big) = x$$

for every fixed scalar x and every invariance certificate h_x.

Definition 8.1.136 (Canonical retraction from the fixed-to-rational direction)**.** Uses: `def:weil-fixed-to-rational-predicate`, `def:weil-rational-fixed-scalar-retraction-candidate`

Lean: `Weil.Arithmetic.rationalFixedScalarRetractionOfFixedToRational`

Given h_{FR} : FixedToRat($\overline{E}$), define $\mathrm{rat}_{h_{\mathrm{FR}}}(x, h_x)$ by applying h_{FR} to (x, h_x) and selecting the exhibited $r \in \mathbf{Q}$ with $\mathrm{map}_{\mathbf{Q},\overline{E}}(r) = x$.

Lemma 8.1.137 (Canonical fixed-scalar retraction specification). *Uses:* *def:weil-rational-fixed-scalar-retraction-from-fixed-to-rational*, *def:weil-rational-fixed-scalar-retraction-predicate*

Lean: *Weil.Arithmetic.rationalFixedScalarRetractionOfFixedToRational_spec*

The candidate $\mathrm{rat}_{h_{\mathrm{FR}}}$ *satisfies* $\mathrm{IsRatRetraction}(\mathrm{rat}_{h_{\mathrm{FR}}})$.

Proof. Uses: `def:weil-rational-fixed-scalar-retraction-from-fixed-to-rational`

This is the witness equation returned by the selected range certificate. □

Lemma 8.1.138 (Canonical fixed-scalar retraction is proof-independent). *Uses:* *def:weil-rational-fixed-scalar-retraction-from-fixed-to-rational*

Lean: *Weil.Arithmetic.rationalFixedScalarRetractionOfFixedToRational_proof_irrel*

For two fixed-to-rational certificates $h_{\mathrm{FR}}, h'_{\mathrm{FR}}$ *and two invariance certificates* h_x, h'_x *on the same scalar* x,

$$\mathrm{rat}_{h_{\mathrm{FR}}}(x, h_x) = \mathrm{rat}_{h'_{\mathrm{FR}}}(x, h'_x).$$

Proof. Uses: `lemma:weil-rational-fixed-scalar-retraction-from-fixed-to-rational-spec`

Map both sides to $\overline{E}$. Both images are x by the specification; injectivity of the field embedding $\mathbf{Q} \hookrightarrow \overline{E}$ reflects the equality. □

Lemma 8.1.139 (Canonical fixed-scalar retraction preserves addition). *Uses:* *def:weil-rational-fixed-scalar-retraction-from-fixed-to-rational*

Lean: *Weil.Arithmetic.rationalFixedScalarRetractionOfFixedToRational_add*

For fixed scalars x, y *and any induced fixedness certificate* h_{x+y},

$$\mathrm{rat}_{h_{\mathrm{FR}}}(x + y, h_{x+y}) = \mathrm{rat}_{h_{\mathrm{FR}}}(x, h_x) + \mathrm{rat}_{h_{\mathrm{FR}}}(y, h_y).$$

Proof. Uses: `lemma:weil-rational-fixed-scalar-retraction-from-fixed-to-rational-spec`

After applying the injective map $\mathbf{Q} \hookrightarrow \overline{E}$, the left side is $x + y$ by the specification and the right side is also $x + y$ by the two specifications and preservation of addition. □

Lemma 8.1.140 (Canonical fixed-scalar retraction is rational-linear). *Uses:* *def:weil-rational-fixed-scalar-retraction-from-fixed-to-rational*

Lean: *Weil.Arithmetic.rationalFixedScalarRetractionOfFixedToRational_smul*

For $r \in \mathbf{Q}$, *a fixed scalar* x, *and any induced certificate* h_{rx},

$$\mathrm{rat}_{h_{\mathrm{FR}}}(\mathrm{map}_{\mathbf{Q},\overline{E}}(r)x, h_{rx}) = r\ \mathrm{rat}_{h_{\mathrm{FR}}}(x, h_x).$$

Proof. Uses: `lemma:weil-rational-fixed-scalar-retraction-from-fixed-to-rational-spec`

Apply the injective map $\mathbf{Q} \hookrightarrow \overline{E}$. The specification identifies the left image with $\mathrm{map}(r)x$ and the right image with the same scalar after preservation of multiplication. □

Lemma 8.1.141 (Canonical fixed-scalar retraction retracts rational scalars). *Uses:* *def:weil-rational-fixed-scalar-retraction-from-fixed-to-rational*

Lean: *Weil.Arithmetic.rationalFixedScalarRetractionOfFixedToRational_algebraMap*

For $r \in \mathbf{Q}$ *and the canonical fixedness certificate* h_r *of its image,*

$$\mathrm{rat}_{h_{\mathrm{FR}}}(\mathrm{map}_{\mathbf{Q},\overline{E}}(r), h_r) = r.$$

Proof. Uses: `lemma:weil-rational-image-fixed-by-absolute-automorphisms`, `lemma:weil-rational-fixed-scalar-retraction-from-fixed-to-rational-spec`

The rational-image fixed lemma supplies h_r. Mapping the displayed equality to $\overline{E}$, the specification makes both sides equal to the image of r; injectivity reflects the equality. □

Definition 8.1.142 (Realization-field interface). Uses: `def:weil-coefficient-algebraic-closure-data`, `def:weil-varying-complex-realization-extension-interface`, `def:weil-finite-subfield-automorphism-extension-interface`, `def:weil-fixed-to-rational-predicate`, `def:weil-rational-fixed-scalar-retraction-from-fixed-to-rational`

Lean: `Weil.Arithmetic.RealizationFieldInterface`

A realization-field interface consists of:

(i) a varying complex-realization extension interface;

(ii) a finite-subfield automorphism-extension interface;

(iii) a certificate h_{FR} of FixedToRat($\overline{E}$).

The proof of RatFixed($\overline{E}$) and the rational fixed-scalar retraction with its specification are derived canonically from field (iii) by the preceding atomic constructions; they are not additional independent assumptions.

Definition 8.1.143 (Constructed realization-field interface). Uses: `def:weil-realization-field-interface`, `def:weil-global-complex-realization`, `def:weil-selected-finite-subfield-complex-extension`, `def:weil-selected-finite-subfield-automorphism-extension`

Lean: `Weil.Arithmetic.constructedRealizationFieldInterface`

Constructed by: `lemma:weil-selected-finite-subfield-complex-extension-spec`, `lemma:weil-selected-finite-subfield-automorphism-extension-spec`, `thm:weil-fixed-scalars-are-rational`

Define

$$\mathcal{R}_{\overline{E}} : \mathrm{RealizationFieldInterface}(\overline{E})$$

by the following nested record terms. Its first field is the varying complex-realization record

$$(\jmath_0, e_{\mathbf{C}}, \mathrm{IsComplexExtensionSelection}(e_{\mathbf{C}})),$$

where the third component is the selected restriction theorem. Its second field is

$$(e_{\mathrm{aut}}, \mathrm{IsAutomorphismExtensionSelection}(e_{\mathrm{aut}})),$$

using the selected automorphism restriction theorem. Its third field is the preceding certificate FixedToRat($\overline{E}$). Thus every constructor field is supplied with the exact type required by Definition 8.1.142.

Lemma 8.1.144 (The prime power is larger than one). Uses: `def:weil-prime-power-data`

Lean: `Weil.PrimePowerData.one_lt_q`

For every prime-power datum, $1 < q$.

Proof. Uses: `def:weil-prime-power-data`

The prime field of the datum satisfies $1 < p$ by the exact mathlib theorem `Nat.Prime.one_lt`. The exponent a is positive. Apply the positive-exponent power inequality and rewrite p^a as q using the stored equality in the datum. □

Lemma 8.1.145 (A supplied realization evaluates raw purity). Uses: `def:weil-global-complex-realization-candidate`, `def:weil-raw-universal-realization-purity`

Lean: `Weil.Arithmetic.IsRawUniversallyPure.apply`

If $\mathrm{RawPure}_q(\alpha, w)$*, then every supplied global realization $\jmath$ satisfies $|\jmath(\alpha)| = q^{w/2}$.*

Proof. Uses: `def:weil-raw-universal-realization-purity`

Unfold the universal quantifier in the definition and instantiate it once at the displayed $\jmath$. □

Lemma 8.1.146 (Raw purity is invariant under rational automorphisms). Uses: `def:weil-raw-universal-realization-purity`

Lean: `Weil.Arithmetic.IsRawUniversallyPure.aut`

If $\mathrm{RawPure}_q(\alpha, w)$ *and* $\sigma \in \mathrm{Aut}(\overline{E}/\mathbf{Q})$*, then* $\mathrm{RawPure}_q(\sigma(\alpha), w)$.

Proof. Uses: `def:weil-raw-universal-realization-purity`

Fix $\jmath : \overline{E} \simeq \mathbf{C}$. The composite $\jmath \circ \sigma$ is another field isomorphism of the quantified type. Applying raw purity to this composite gives

$$|\jmath(\sigma(\alpha))| = |(\jmath \circ \sigma)(\alpha)| = q^{w/2}.$$

Since $\jmath$ was arbitrary, this is the defining universal statement. □

Lemma 8.1.147 (Distinct raw weights are disjoint). Uses: `def:weil-global-complex-realization-candidate`, `def:weil-raw-universal-realization-purity`

Lean: `Weil.Arithmetic.rawPure_weight_unique`

Let a global realization candidate be supplied. If nonzero α is raw universally pure of weights i and j, then $i = j$.

Proof. Uses: `lemma:weil-prime-power-gt-one`, `lemma:weil-raw-purity-evaluate-realization`

Evaluate the two purity equalities at the supplied realization $\jmath_0$. They give $q^{i/2} = q^{j/2}$. The real logarithm is injective on positive numbers and $\log q \neq 0$ because $q > 1$; taking logarithms and cancelling $\frac{1}{2}\log q$ gives $i = j$. This is a single ordered-field calculation. □

Lemma 8.1.148 (Raw purity gives the classical all-embeddings modulus). Uses: def:weil-realization-field-interface, def:weil-raw-universal-realization-purity, def:weil-universal-scalar-weight

Lean: Weil.Arithmetic.rawPure_to_classicalPure

Suppose $\alpha \in \overline{E}^{\times}$ is algebraic over $\mathbf{Q}$ and raw universally pure of weight i. Then, after identifying $\mathbf{Q}(\alpha)$ with its copy in $\overline{E}$, every embedding $\tau : \mathbf{Q}(\alpha) \hookrightarrow \mathbf{C}$ satisfies $|\tau(\alpha)| = q^{i/2}$.

Proof. Uses: def:weil-realization-field-interface,def:weil-finite-subfield-embedding-extension-predicate,lemma:weil-raw-purity-evaluate-realization

The finite extension field $L = \mathbf{Q}(\alpha)$ and τ are admissible inputs to the finite-subfield selection in field (i) of the realization interface. Project its explicitly supplied $\widetilde{\tau} : \overline{E} \simeq \mathbf{C}$ and the equality $\widetilde{\tau}|_L = \tau$. Raw purity evaluated at $\widetilde{\tau}$ gives $|\widetilde{\tau}(\alpha)| = q^{i/2}$; rewriting with the restriction equality gives the stated formula for τ. □

> **Formalization note.** The field-interface atoms implement the set-theoretic argument in the supplied notes, Chapter 14, *Extension and fixed fields* (printed p. 74, physical PDF p. 84). The global field isomorphism is constructed above from the explicit ell-adic cardinal calculation, matched transcendence bases, rational-function fields, and algebraic-closure uniqueness. The prescribed finite-subfield embeddings and automorphisms now have exact restriction equations, while the two mover constructions prove that every nonrational scalar is changed by some rational automorphism. The varying-realization interface concerns only field isomorphisms and their restriction equations. It does not supply the realization-indexed geometric antecedents or their independence equations; those remain the separate OBL-WGT-RH-ALL-REALIZATIONS.

8.2 Conditional arithmetic recovery input and degree computation

Definition 8.2.1 (Universal purity predicate for a graded factor family). Uses: def:weil-bounded-graded-based-automorphism-family,def:weil-graded-reciprocal-factor,def:weil-raw-universal-realization-purity

Lean: Weil.Arithmetic.IsUniversallyPureGradedFactorFamily

For a bounded graded based automorphism family $G = (D_i)_{0 \le i \le B}$, define PureFactors($G$) to mean: for every $i \le B$ and every reciprocal root $\alpha \in \overline{E}^{\times}$ of $P_{G,i}$, $\mathrm{RawPure}_q(\alpha, i)$.

Definition 8.2.2 (Arithmetic factor-recovery candidate). Uses: def:weil-finite-cohomological-trace-datum,def:weil-realization-field-interface

Lean: Weil.Arithmetic.FactorRecoveryCandidate

An arithmetic factor-recovery candidate consists of a finite cohomological trace datum $\mathcal{T} = (P, G, (t_n))$ over E and a realization-field interface for $\overline{E}$.

Definition 8.2.3 (Explicit integer coefficient sequence of a recovery candidate). Uses: def:weil-arithmetic-factor-recovery-candidate,def:weil-orbit-euler-product

Lean: Weil.Arithmetic.recoveryCountingZetaIntegerSequence

Let D be the finite-orbit datum underlying the point interface P. For $n \in \mathbf{N}$, define

$$\widehat{z}_{\mathcal{R},n} := [T^n] Z_{\mathrm{orb},D}(T) \in \mathbf{Z}.$$

Definition 8.2.4 (Integer coefficient comparison predicate). Uses: def:weil-recovery-counting-zeta-integer-sequence,def:weil-finite-extension-counting-zeta-series

Lean: Weil.Arithmetic.IsRecoveryCountingZetaIntegerSequence

Define IsIntegerZetaSequence($\mathcal{R}$) to mean

$$\mathrm{map}_{\mathbf{Z},\mathbf{Q}}(\widehat{z}_{\mathcal{R},n}) = [T^n] Z_P \qquad (n \in \mathbf{N}).$$

Definition 8.2.5 (Arithmetic factor-recovery input). Uses: def:weil-arithmetic-factor-recovery-candidate,def:weil-graded-factor-universal-purity-predicate,def:weil-recovery-counting-zeta-integer-sequence-predicate

Lean: Weil.Arithmetic.FactorRecoveryInput

An arithmetic factor-recovery input is a candidate $\mathcal{R}$ together with stored proofs of PureFactors(G) and IsIntegerZetaSequence($\mathcal{R}$). This Definition packages the terminal outputs expected from the point-counting, trace, and weight modules; it does not construct those outputs for a scheme.

Definition 8.2.6 (Arithmetic factor-recovery input constructor)**.** Uses: `def:weil-arithmetic-factor-recovery-candidate`, `def:weil-graded-factor-universal-purity-predicate`, `def:weil-recovery-counting-zeta-integer-sequence-predicate`, `def:weil-arithmetic-factor-recovery-input`

Lean: `Weil.Arithmetic.mkFactorRecoveryInput`

For a candidate $\mathcal{R}$, take the two explicit certificates

$$h_{\mathrm{pur}} : \mathrm{PureFactors}(G), \qquad h_{\mathbf{Z}} : \mathrm{IsIntegerZetaSequence}(\mathcal{R}).$$

Define

$$\mathrm{mkRecovery}(\mathcal{R}, h_{\mathrm{pur}}, h_{\mathbf{Z}}) := \langle \mathcal{R}, h_{\mathrm{pur}}, h_{\mathbf{Z}} \rangle.$$

Definition 8.2.7 (The ℓ-adic cohomological Betti number)**.** Uses: `def:weil-arithmetic-factor-recovery-candidate`, `def:weil-bounded-graded-based-automorphism-family`

Lean: `Weil.Arithmetic.ellAdicBettiNumber`

For $i \le B$, define

$$b_{i,\ell} := \dim_E(V_i),$$

where V_i is the finite-dimensional carrier in the based automorphism datum D_i.

Lemma 8.2.8 (Leading coefficient of a reciprocal determinant)**.** Uses: `def:weil-based-linear-automorphism-datum`, `def:weil-based-reciprocal-determinant`

Lean: `Weil.LinearAlgebra.reciprocalDet_leadingCoeff`

If $D = (V, u, b)$ has dimension r, then the coefficient of T^r in $P_D(T) = \det(I_r - TM_D)$ is $(-1)^r \det(M_D)$.

Proof. Uses: `def:weil-coordinate-endomorphism-matrix`, `def:weil-reciprocal-polynomial-matrix`, `def:weil-based-reciprocal-determinant`

Use the permutation expansion of the determinant. A term contributing degree r must contribute the degree-one entry $-T(M_D)_{a,\sigma(a)}$ in every row; using any Kronecker-delta entry lowers the degree. Summing the degree-r terms over $\sigma \in S_r$ gives

$$(-1)^r \sum_{\sigma} \mathrm{sgn}(\sigma) \prod_{a} (M_D)_{a,\sigma(a)} = (-1)^r \det(M_D).$$

□

Lemma 8.2.9 (The automorphism determinant is nonzero)**.** Uses: `def:weil-based-linear-automorphism-datum`, `def:weil-coordinate-endomorphism-matrix`

Lean: `Weil.LinearAlgebra.automorphismMatrix_det_ne_zero`

For a based linear automorphism datum, $\det(M_D) \neq 0$.

Proof. Uses: `def:weil-based-linear-automorphism-datum`, `def:weil-coordinate-endomorphism-matrix`

The inverse automorphism has coordinate matrix N satisfying $M_D N = I_r$; this follows by evaluating the two inverse equations on the basis vectors. Taking determinants gives $\det(M_D)\det(N) = 1$. Therefore $\det(M_D)$ is a unit, hence nonzero in the field E. □

Proposition 8.2.10 (Degree of a reciprocal factor)**.** Uses: `def:weil-elladic-betti-number`, `def:weil-graded-reciprocal-factor`

Lean: `Weil.Arithmetic.factor_natDegree_eq_betti`

For every $i \le B$,

$$\deg P_{G,i} = b_{i,\ell}.$$

Proof. Uses: `def:weil-elladic-betti-number`, `def:weil-graded-reciprocal-factor`, `lemma:weil-reciprocal-determinant-leading-coefficient`, `lemma:weil-based-automorphism-determinant-nonzero`

Let $r = b_{i,\ell}$. The determinant expansion has degree at most r because each Leibniz term contains one entry from each of the r rows. Its coefficient in degree r is $(-1)^r \det(M_{D_i})$, which is nonzero by the preceding lemma. Therefore its degree is exactly r. □

> **Status.** Proposition 8.2.10 is pure linear algebra. It neither says that $b_{i,\ell}$ is independent of ℓ nor identifies it with a singular Betti number. Those are later, logically separate conclusions.

8.3 Scalar descent of rationality by Hankel rank

Definition 8.3.1 (Sequence left shift)**.** Uses:

Lean: `Weil.Arithmetic.sequenceShift`

For a field K and a sequence $a : \mathbf{N} \to K$, define

$$(Sa)_n := a_{n+1}.$$

Definition 8.3.2 (Shift span)**.** Uses: `def:weil-sequence-left-shift`

Lean: `Weil.Arithmetic.shiftSpan`

Define

$$\mathrm{Sh}_K(a) := \mathrm{span}_K\{S^m a \mid m \in \mathbf{N}\} \subseteq K^{\mathbf{N}}.$$

Definition 8.3.3 (Eventual linear recurrence)**.** Uses:

Lean: `Weil.Arithmetic.EventualLinearRecurrence`

For $a : \mathbf{N} \to K$, integers $r, N \geq 0$, and coefficients $c_1, \dots, c_r \in K$, define $\mathrm{Rec}(a, r, N, c)$ to mean

$$a_n + c_1 a_{n-1} + \cdots + c_r a_{n-r} = 0 \qquad (n \geq \max\{N, r\}).$$

Definition 8.3.4 (Hankel matrix of a sequence)**.** Uses:

Lean: `Weil.Arithmetic.hankelMatrix`

For $a : \mathbf{N} \to K$, define the infinite Hankel matrix

$$H(a)_{u,v} := a_{u+v}.$$

Definition 8.3.5 (Hankel rank bound)**.** Uses: `def:weil-hankel-matrix`

Lean: `Weil.Arithmetic.HankelRankAtMost`

For $r \in \mathbf{N}$, define $\mathrm{Hrk}(a) \leq r$ to mean that for every two index maps $u, v : \mathrm{Fin}(r+1) \to \mathbf{N}$, the determinant

$$\det\big(a_{u(x)+v(y)}\big)_{x,y \in \mathrm{Fin}(r+1)}$$

vanishes.

Definition 8.3.6 (Scalar rationality-descent datum)**.** Uses: `def:weil-normalized-polynomial-fraction-datum`, `def:weil-normalized-fraction-taylor-series`

Lean: `Weil.Arithmetic.ScalarRationalityDescentData`

A scalar rationality-descent datum consists of a field extension $K \subseteq L$, a series $f = \sum a_n T^n \in K[[T]]$, a normalized fraction $\mathcal{F} = (N, D)$ over L, and the displayed equality

$$\mathrm{map}_{K,L}(f) = \tau_{\mathrm{rep}}(\mathcal{F}) \quad \text{in } L[[T]].$$

The fraction and equality are input fields; no rationality conclusion is included in this Definition.

Lemma 8.3.7 (A normalized fraction gives an eventual recurrence)**.** *Uses:* *`def:weil-normalized-polynomial-fraction-datum`, `def:weil-normalized-fraction-taylor-series`, `def:weil-eventual-linear-recurrence`*

Lean: *`Weil.Arithmetic.NormalizedFraction.eventualRecurrence`*

Let $\mathcal{F} = (N, D)$ with $D = 1 + d_1 T + \cdots + d_r T^r$. If $g = \tau_{\mathrm{rep}}(\mathcal{F}) = \sum b_n T^n$, then

$$b_n + d_1 b_{n-1} + \cdots + d_r b_{n-r} = 0 \qquad (n > \deg N,\ n \geq r).$$

Proof. Uses: `def:weil-normalized-fraction-taylor-series`, `lemma:weil-geometric-inverse-mul`

Multiply the equality $g = \jmath(N)\,\mathrm{GeomInv}(\jmath(D))$ by $\jmath(D)$. The geometric-inverse identity gives $\jmath(D)g = \jmath(N)$. The coefficient of T^n on the left is the displayed convolution; the coefficient on the right is zero when $n > \deg N$. This is exactly the stated recurrence. □

Lemma 8.3.8 (An eventual recurrence bounds the shift span)**.** *Uses:* *`def:weil-sequence-left-shift`, `def:weil-sequence-shift-span`, `def:weil-eventual-linear-recurrence`*

Lean: *`Weil.Arithmetic.EventualLinearRecurrence.finiteDimensional_shiftSpan`*

If $\mathrm{Rec}(a, r, N, c)$, then $\mathrm{Sh}_K(a)$ is spanned by

$$a, Sa, \dots, S^{N+r}a \quad \text{and the coordinate vectors} \quad e_0, \dots, e_{N+r-1}.$$

In particular the shift span is finite-dimensional.

Proof. Uses: def:weil-sequence-left-shift,def:weil-sequence-shift-span,def:weil-eventual-linear-recurrence

For $m > N$, apply the recurrence at every coordinate to express the tail of $S^{m+r}a$ as $-c_1S^{m+r-1}a - \cdots - c_rS^m a$. Their difference is supported in the first $N+r$ coordinates, hence is a linear combination of $e_0, \ldots, e_{N+r-1}$. Induction on m places every later shift in the displayed finite span. Earlier shifts are already listed. □

Definition 8.3.9 (Shift restriction endomorphism)**.** Uses: def:weil-sequence-left-shift,def:weil-sequence-shift-span

Lean: Weil.Arithmetic.shiftRestriction

On $\mathrm{Sh}_K(a)$ define

$$S_a : \mathrm{Sh}_K(a) \longrightarrow \mathrm{Sh}_K(a)$$

by restricting the sequence shift. It maps the span into itself because $S(S^m a) = S^{m+1}a$ on every spanning generator.

Definition 8.3.10 (Shift characteristic polynomial)**.** Uses: def:weil-sequence-shift-span,def:weil-shift-restriction-endomorphism

Lean: Weil.Arithmetic.shiftCharpoly

When $\mathrm{Sh}_K(a)$ is finite-dimensional, define

$$C_a(U) := \mathrm{charpoly}(S_a) = U^s + c_{s-1}U^{s-1} + \cdots + c_0.$$

Lemma 8.3.11 (Cayley–Hamilton recurrence for the shift span)**.** *Uses:* def:weil-sequence-shift-span,def:weil-shift-characteristic-polynomial

Lean: Weil.Arithmetic.shiftCharpoly_recurrence

For finite-dimensional $\mathrm{Sh}_K(a)$*,*

$$a_{n+s} + c_{s-1}a_{n+s-1} + \cdots + c_0 a_n = 0 \qquad (n \geq 0).$$

Proof. Uses: def:weil-sequence-shift-span,def:weil-shift-restriction-endomorphism,def:weil-shift-characteristic-polynomial

Apply the exact Cayley–Hamilton identity `LinearMap.aeval_self_charpoly` to S_a. Evaluate the resulting zero endomorphism at the generator $a = S^0 a$, and then evaluate the resulting zero sequence in coordinate n. Since $(S_a^j a)_n = a_{n+j}$, the displayed coordinate recurrence is obtained. □

Definition 8.3.12 (Denominator constructed from the shift characteristic polynomial)**.** Uses: def:weil-shift-characteristic-polynomial

Lean: Weil.Arithmetic.shiftRecurrenceDenominator

For $C_a(U) = U^s + c_{s-1}U^{s-1} + \cdots + c_0$, define

$$D_a(T) := 1 + c_{s-1}T + \cdots + c_0 T^s.$$

Definition 8.3.13 (Numerator constructed from the shift recurrence)**.** Uses: def:weil-shift-recurrence-denominator

Lean: Weil.Arithmetic.shiftRecurrenceNumerator

For $f_a(T) = \sum_{n\geq 0} a_n T^n$, define N_a to be the truncation through degree $s-1$ of the product $D_a(T)f_a(T)$.

Lemma 8.3.14 (The constructed recurrence fraction has the prescribed series)**.** *Uses:* def:weil-shift-recurrence-denominator,def:weil-shift-recurrence-numerator

Lean: Weil.Arithmetic.shiftRecurrenceFraction_taylor

If the shift span is finite-dimensional, then

$$D_a(T)f_a(T) = N_a(T), \qquad D_a(0) = 1.$$

Proof. Uses: def:weil-shift-recurrence-denominator,def:weil-shift-recurrence-numerator,lemma:weil-shift-cayley-hamilton-recurrence

The constant coefficient of D_a is 1 by construction. For $m \geq s$, the coefficient of T^m in $D_a f_a$ is the Cayley–Hamilton recurrence with $n = m - s$, hence zero. Thus the product has degree below s, where by definition it equals its truncation N_a. □

Proposition 8.3.15 (Finite shift span gives an explicit rational fraction)**.** *Uses:* def:weil-sequence-shift-span, def:weil-shift-recurrence-denominator, def:weil-shift-recurrence-numerator

Lean: Weil.Arithmetic.finiteShiftSpan_rational

If $\mathrm{Sh}_K(a)$ *is finite-dimensional, then the explicitly constructed normalized fraction* (N_a, D_a) *has Taylor series* $f_a = \sum a_n T^n$.

Proof. Uses: def:weil-normalized-fraction-taylor-series, lemma:weil-shift-recurrence-fraction-taylor, lemma:weil-geometric-inverse-mul

The preceding product equality says $D_a f_a = N_a$, and $D_a(0) = 1$ gives its formal inverse. Multiply by that inverse to obtain $f_a = \jmath(N_a)\,\mathrm{GeomInv}(\jmath(D_a))$, exactly the Taylor series of the constructed normalized fraction. □

Definition 8.3.16 (First nonzero coordinate, with a total default)**.** Uses:

Lean: Weil.Arithmetic.firstNonzeroSequenceCoordinate

For $v : \mathbf{N} \to K$, define

$$\mu(v) := \begin{cases} \mathrm{Nat.find}(h), & h : \{n \mid v(n) \neq 0\} \neq \varnothing, \\ 0, & \{n \mid v(n) \neq 0\} = \varnothing. \end{cases}$$

The decidable conditional makes μ a total function; the first branch uses the least natural number certified by h.

Lemma 8.3.17 (The first nonzero coordinate is nonzero)**.** *Uses:* def:weil-first-nonzero-sequence-coordinate

Lean: Weil.Arithmetic.firstNonzeroSequenceCoordinate_spec

If $v \neq 0$, *then* $v(\mu(v)) \neq 0$.

Proof. Uses: def:weil-first-nonzero-sequence-coordinate

If the first branch condition failed, every coordinate of v would be zero, and function extensionality would give $v = 0$. Hence the first branch condition holds. Unfold μ and apply `Nat.find_spec` to its stored nonemptiness certificate. □

Definition 8.3.18 (Finite sequence-evaluation matrix)**.** Uses:

Lean: Weil.Arithmetic.sequenceEvaluationMatrix

For $v : \mathrm{Fin}\, m \to (\mathbf{N} \to K)$ and $u : \mathrm{Fin}\, m \to \mathbf{N}$, define

$$\mathrm{Ev}(v, u)_{x,y} := v_y(u_x) \qquad (x, y : \mathrm{Fin}\, m).$$

Definition 8.3.19 (Elimination residual at the last sequence)**.** Uses: def:weil-finite-sequence-evaluation-matrix

Lean: Weil.Arithmetic.independentSequenceEliminationResidual

Let $v : \mathrm{Fin}(m+1) \to (\mathbf{N} \to K)$ and let $u^- : \mathrm{Fin}\, m \to \mathbf{N}$. Put

$$A := \mathrm{Ev}(v|_{\mathrm{Fin}\, m}, u^-), \qquad b_x := v_m(u^-_x), \qquad c := A^{-1} b,$$

where matrix inversion is the total matrix inverse over K. Define

$$w(v, u^-) := v_m - \sum_{y : \mathrm{Fin}\, m} c_y v_y \in K^{\mathbf{N}}.$$

Definition 8.3.20 (Canonical finite-coordinate witness)**.** Uses: def:weil-first-nonzero-sequence-coordinate, def:weil-independent-sequence-elimination-residual

Lean: Weil.Arithmetic.independentSequenceCoordinateWitness

Define

$$\mathrm{coord}(v) : \mathrm{Fin}\, m \longrightarrow \mathbf{N}$$

recursively on m. For $m = 0$ it is the empty function. For $v : \mathrm{Fin}(m+1) \to K^{\mathbf{N}}$, its first m values are $\mathrm{coord}(v|_{\mathrm{Fin}\, m})$, and its last value is

$$\mu\big(w\big(v, \mathrm{coord}(v|_{\mathrm{Fin}\, m})\big)\big).$$

This is a total recursive Definition and does not contain an unspecified coordinate tuple.

Lemma 8.3.21 (Restriction preserves linear independence)**.** *Uses:*

Lean: Weil.Arithmetic.linearIndependent_finRestriction

If $v : \mathrm{Fin}(m+1) \to K^{\mathbf{N}}$ *is linearly independent, then* $v|_{\mathrm{Fin}\, m}$ *is linearly independent.*

Proof. Uses:
A linear relation on the first m vectors extends to a relation on all $m+1$ vectors by assigning coefficient 0 to the last vector. Linear independence of v makes every extended coefficient zero, hence all original coefficients are zero. □

Lemma 8.3.22 (The elimination residual vanishes on the old coordinates). *Uses:* *def: weil-independent-sequence-elimination-residual, def: weil-finite-sequence-evaluation-matrix*

Lean: *Weil.Arithmetic.eliminationResidual_oldCoordinate*

If

$$\det \mathrm{Ev}(v|_{\mathrm{Fin}\, m}, u^-) \neq 0,$$

then for every $x : \mathrm{Fin}\, m$,

$$w(v, u^-)(u^-_x) = 0.$$

Proof. Uses: `def:weil-independent-sequence-elimination-residual`,`def:weil-finite-sequence-evaluation-matrix`
Write $A = \mathrm{Ev}(v|_{\mathrm{Fin}\, m}, u^-)$. The determinant hypothesis makes A invertible, so $A(A^{-1}b) = b$. The x-th coordinate of this matrix-vector equality is

$$\sum_y (A^{-1}b)_y v_y(u^-_x) = v_m(u^-_x).$$

Move the sum to the right and unfold $w(v, u^-)$. □

Lemma 8.3.23 (The elimination residual is nonzero). *Uses:* *def: weil-independent-sequence-elimination-residual*

Lean: *Weil.Arithmetic.eliminationResidual_ne_zero*

If $v : \mathrm{Fin}(m+1) \to K^{\mathbf{N}}$ *is linearly independent, then* $w(v, u^-) \neq 0$ *for every* u^-.

Proof. Uses: `def:weil-independent-sequence-elimination-residual`
If $w(v, u^-) = 0$, its defining equation rewrites as

$$v_m - \sum_{y : \mathrm{Fin}\, m} c_y v_y = 0.$$

This is a linear relation on v whose coefficient at the last vector is 1. Linear independence would force that coefficient to equal 0, contradicting $1 \neq 0$ in the field K. □

Lemma 8.3.24 (One elimination step updates the evaluation determinant). *Uses:* *def: weil-independent-sequences-finite-coordinate-witness, def: weil-independent-sequence-elimination-residual, def: weil-finite-sequence-evaluation-matrix*

Lean: *Weil.Arithmetic.sequenceEvaluationDeterminant_succ*

Let $u = \mathrm{coord}(v)$, *let* u^- *be its first* m *entries, and let* $w = w(v, u^-)$. *If* $\det \mathrm{Ev}(v|_{\mathrm{Fin}\, m}, u^-) \neq 0$, *then*

$$\det \mathrm{Ev}(v, u) = \det \mathrm{Ev}(v|_{\mathrm{Fin}\, m}, u^-) \cdot w(u_m).$$

Proof. Uses: `def:weil-independent-sequences-finite-coordinate-witness`,`def:weil-independent-sequence-elimination-residual`,`lemma:weil-independent-sequence-residual-old-coordinates`
In the last column of $\mathrm{Ev}(v, u)$, subtract the linear combination of the first m columns with coefficients c_y. Determinant invariance under this column operation changes the last column to the evaluations of w. Its first m entries are zero by the old-coordinate lemma and its last entry is $w(u_m)$. Laplace expansion along this last column gives the displayed product; the last cofactor has sign $(-1)^{2m} = 1$. □

Lemma 8.3.25 (Finite coordinates detect finitely many independent sequences). *Uses:* *def: weil-independent-sequences-finite-coordinate-witness, def: weil-finite-sequence-evaluation-matrix*

Lean: *Weil.Arithmetic.linearIndependent_finite_coordinate_witness*

If $v : \mathrm{Fin}\, m \to K^{\mathbf{N}}$ *is linearly independent, then the explicitly defined coordinates satisfy*

$$\det \mathrm{Ev}(v, \mathrm{coord}(v)) \neq 0.$$

Proof. Uses: `def:weil-independent-sequences-finite-coordinate-witness`, `lemma:weil-first-nonzero-sequence-coordinate-spec`, `lemma:weil-independent-sequences-restriction`, `lemma:weil-independent-sequence-residual-nonzero`, `lemma:weil-independent-sequence-evaluation-determinant-step`

Induct on m. For $m = 0$, the determinant of the empty matrix is 1. For $m + 1$, the induction hypothesis applied to the restricted independent family makes the old determinant nonzero. The residual-nonzero lemma and the first-nonzero-coordinate specification make its value at the last defined coordinate nonzero. The determinant-step lemma expresses the new determinant as the product of these two nonzero scalars. □

Lemma 8.3.26 (A Hankel rank bound controls the shift span). *Uses:* *def:weil-sequence-shift-span*, *def:weil-hankel-rank-bound*

Lean: *Weil.Arithmetic.hankelRankAtMost_finiteShiftSpan*

If $\mathrm{Hrk}(a) \le r$, *then* $\dim_K \mathrm{Sh}_K(a) \le r$.

Proof. Uses: `def:weil-hankel-matrix`, `def:weil-hankel-rank-bound`, `lemma:weil-independent-sequences-finite-coordinate-witness`

If the dimension exceeded r, then $r + 1$ shifts $S^{v_0}a, \dots, S^{v_r}a$ would be linearly independent. The finite-coordinate witness would supply indices u_x for which

$$\det\big((S^{v_y}a)_{u_x}\big) = \det\big(a_{u_x+v_y}\big) \neq 0,$$

contradicting the defining Hankel rank bound. □

Lemma 8.3.27 (An eventual recurrence gives a Hankel rank bound). *Uses:* *def:weil-eventual-linear-recurrence*, *def:weil-hankel-rank-bound*

Lean: *Weil.Arithmetic.EventualLinearRecurrence.hankelRankBound*

If $\mathrm{Rec}(a, r, N, c)$, *then* $\mathrm{Hrk}(a) \le N + 2r + 1$.

Proof. Uses: `lemma:weil-eventual-recurrence-finite-shift-span`, `lemma:weil-independent-sequences-finite-coordinate-witness`

The explicit spanning list in Lemma 8.3.8 has at most $N + 2r + 1$ entries. Any square minor larger than that span dimension has dependent columns, so its determinant vanishes. □

Lemma 8.3.28 (A finite determinant detects scalar extension). *Uses:*

Lean: *Weil.Arithmetic.det_map_eq_zero_iff*

For an injective field map $K \hookrightarrow L$ *and a finite square matrix* M *over* K,

$$\det(\mathrm{map}_{K,L}\, M) = 0 \quad \Longleftrightarrow \quad \det(M) = 0.$$

Proof. Uses:

The determinant is a finite signed sum of finite products, so mapping entries maps the determinant. Injectivity of the field map reflects zero. These are the two exact steps `RingHom.map_det` and `map_eq_zero_iff`. □

Lemma 8.3.29 (Hankel rank bounds descend along field extensions). *Uses:* *def:weil-hankel-rank-bound*

Lean: *Weil.Arithmetic.hankelRankAtMost_restrictScalars*

Let $K \hookrightarrow L$ *be injective. If the coefficientwise image of* $a : \mathbf{N} \to K$ *has Hankel rank at most* r *over* L, *then* a *has Hankel rank at most* r *over* K.

Proof. Uses: `def:weil-hankel-rank-bound`, `lemma:weil-finite-determinant-scalar-extension`

Fix the two finite index maps in the rank-bound definition. The determinant over L is the image of the corresponding determinant over K. Its vanishing is reflected by the preceding finite-determinant lemma. Since the index maps were arbitrary, every required K-minor vanishes. □

Theorem 8.3.30 (Constructive Hankel rationality descent). *Uses:* *def:weil-scalar-rationality-descent-datum*, *def:weil-shift-recurrence-denominator*, *def:weil-shift-recurrence-numerator*

Lean: *Weil.Arithmetic.hankelRationalityDescent*

For every scalar rationality-descent datum, the series $f \in K[[T]]$ *is the Taylor series of the explicitly constructed normalized* K*-fraction obtained from the characteristic polynomial of its finite shift span.*

Proof. Uses: def:weil-scalar-rationality-descent-datum, lemma:weil-normalized-fraction-eventual-recurrence, lemma:weil-eventual-recurrence-hankel-bound, lemma:weil-hankel-rank-bound-descends, lemma:weil-hankel-rank-bound-finite-shift-span, prop:weil-finite-shift-span-rational

The supplied L-fraction gives an eventual recurrence for the mapped coefficient sequence. The recurrence supplies a finite Hankel rank bound over L; scalar descent reflects every determinant and gives the same bound over K. Hence the K-shift span is finite-dimensional. Apply the explicit shift-characteristic-polynomial construction and the preceding proposition. Each arrow is one of the cited lemmas; no denominator is averaged over field automorphisms. □

Definition 8.3.31 (Counting-zeta scalar-descent candidate). Uses: def:weil-arithmetic-factor-recovery-candidate, def:weil-finite-extension-counting-zeta-series, def:weil-graded-canonical-normalized-fraction, def:weil-scalar-rationality-descent-datum

Lean: Weil.Arithmetic.countingZetaScalarDescentCandidate

For an arithmetic recovery candidate, take $K = \mathbf{Q}$, $L = E$, $f = Z_P$, and the displayed E-fraction $\mathcal{F}_G$. This defines all object fields of a scalar rationality-descent datum except its Taylor-series equality.

Definition 8.3.32 (Counting-zeta descent compatibility predicate). Uses: def:weil-counting-zeta-scalar-descent-candidate, def:weil-rational-power-series-scalar-extension, def:weil-normalized-fraction-taylor-series

Lean: Weil.Arithmetic.IsCountingZetaDescentCompatible

Define compatibility to mean

$$\mathrm{map}_{\mathbf{Q},E}(Z_P) = \tau_{\mathrm{rep}}(\mathcal{F}_G).$$

Proposition 8.3.33 (The trace datum supplies counting-zeta descent compatibility). *Uses: def:weil-arithmetic-factor-recovery-candidate, def:weil-counting-zeta-descent-compatibility-predicate*

Lean: Weil.Arithmetic.countingZetaDescentCompatible

The counting-zeta scalar-descent candidate is compatible.

Proof. Uses: def:weil-counting-zeta-descent-compatibility-predicate, def:weil-graded-canonical-normalized-fraction, cor:weil-zeta-factorization-after-coefficient-extension, lemma:weil-alternating-determinant-normalized-fraction

The coefficient-extension factorization identifies the left side with the alternating determinant product Δ_G. The normalized-fraction lemma identifies Δ_G with $\tau_{\mathrm{rep}}(\mathcal{F}_G)$. Transitivity gives exactly the defined compatibility predicate. □

Definition 8.3.34 (Rational counting-zeta fraction over $\mathbf{Q}$). Uses: def:weil-counting-zeta-scalar-descent-candidate, def:weil-counting-zeta-descent-compatibility-predicate, def:weil-shift-recurrence-denominator, def:weil-shift-recurrence-numerator

Lean: Weil.Arithmetic.rationalCountingZetaFraction

Given an explicit proof h of counting-zeta descent compatibility, define $\mathcal{F}_{P,\mathbf{Q}}$ to be the normalized $\mathbf{Q}$-fraction constructed by the shift-characteristic-polynomial procedure in the preceding section.

Proposition 8.3.35 (The rational counting-zeta fraction has Taylor series Z_P). *Uses: def:weil-rational-counting-zeta-fraction, def:weil-finite-extension-counting-zeta-series, def:weil-normalized-fraction-taylor-series*

Lean: Weil.Arithmetic.rationalCountingZetaFraction_taylor

With h instantiated by the proved compatibility,

$$\tau_{\mathrm{rep}}(\mathcal{F}_{P,\mathbf{Q}}) = Z_P.$$

Proof. Uses: prop:weil-counting-zeta-descent-compatible, thm:weil-hankel-rationality-descent

Insert the proved compatibility as the last field of the scalar rationality-descent datum. The conclusion of the constructive Hankel descent theorem is precisely the displayed Taylor equality for the fraction just defined. □

> **Formalization note.** The generic algebra follows the supplied notes, Chapter 2, *Rational series and eventual recurrences* through *Hankel descent* (printed pp. 10–12, physical PDF pp. 20–22), but every choice of a denominator has been replaced by the shift endomorphism and its characteristic polynomial. The finite-minor proof is compatible with the pinned mathlib matrix determinant API.

8.4 The integral Fatou lattice and a reduced integral fraction

Definition 8.4.1 (Integral rational-series datum). Uses: `def:weil-normalized-polynomial-fraction-datum`, `def:weil-normalized-fraction-taylor-series`, `def:weil-power-series-over-subring`

Lean: `Weil.Arithmetic.IntegralRationalSeriesData`

An integral rational-series datum consists of a series $f = \sum a_n T^n \in \mathbf{Q}[[T]]$, an explicit integer sequence $\widehat{a} : \mathbf{N} \to \mathbf{Z}$, coefficient identities

$$\mathrm{map}_{\mathbf{Z},\mathbf{Q}}(\widehat{a}_n) = a_n \quad (n \in \mathbf{N}),$$

a normalized rational fraction $\mathcal{F} = (N, D)$ over $\mathbf{Q}$, and an equality $\tau_{\mathrm{rep}}(\mathcal{F}) = f$.

Definition 8.4.2 (Integral coefficient sequence). Uses: `def:weil-integral-rational-series-datum`

Lean: `Weil.Arithmetic.IntegralRationalSeriesData.integerSequence`

Define $\widehat{a}_n$ to be the n-th value of the explicit integer sequence stored in the integral rational-series datum.

Lemma 8.4.3 (Integral coefficient sequence maps back to the rational sequence). *Uses:* `def:weil-integral-coefficient-sequence`

Lean: `Weil.Arithmetic.IntegralRationalSeriesData.coe_integerSequence`

For every n, the rational image of $\widehat{a}_n$ is a_n.

Proof. Uses: `def:weil-integral-rational-series-datum`, `def:weil-integral-coefficient-sequence`

Unfold the projection in Definition 8.4.2; the target equality is the coefficient-identity field of the datum, evaluated at n. □

Definition 8.4.4 (Finite coordinate projection). Uses: `def:weil-sequence-shift-span`

Lean: `Weil.Arithmetic.coordinateProjection`

For $M \in \mathbf{N}$, define

$$\pi_M : \mathrm{Sh}_{\mathbf{Q}}(a) \longrightarrow \mathbf{Q}^{\mathrm{Fin}\, M}, \qquad \pi_M(v) = (v_0, \ldots, v_{M-1}).$$

Definition 8.4.5 (Finite coordinate kernel). Uses: `def:weil-finite-coordinate-projection`

Lean: `Weil.Arithmetic.coordinateKernel`

Define $K_M := \ker(\pi_M)$.

Lemma 8.4.6 (Coordinate kernels descend). *Uses:* `def:weil-finite-coordinate-kernel`

Lean: `Weil.Arithmetic.coordinateKernel_anti`

For every M, $K_{M+1} \subseteq K_M$.

Proof. Uses: `def:weil-finite-coordinate-projection`, `def:weil-finite-coordinate-kernel`

If the first $M+1$ coordinates vanish, then in particular the first M coordinates vanish. This is precisely the displayed subspace inclusion. □

Definition 8.4.7 (Minimal coordinate-kernel dimension). Uses: `def:weil-finite-coordinate-kernel`

Lean: `Weil.Arithmetic.minCoordinateKernelDimension`

Assuming the shift span is finite-dimensional, define

$$\delta := \min\{\dim_{\mathbf{Q}} K_M \mid M \in \mathbf{N}\}.$$

The set is nonempty because it contains $\dim K_0$, and the minimum is the well-order minimum in $\mathbf{N}$.

Definition 8.4.8 (Coordinate stabilization index). Uses: `def:weil-finite-coordinate-kernel`, `def:weil-minimal-coordinate-kernel-dimension`

Lean: `Weil.Arithmetic.coordinateStabilizationIndex`

Define M_* to be the least M such that $\dim_{\mathbf{Q}} K_M = \delta$.

Lemma 8.4.9 (Coordinate kernels stabilize at the least index). *Uses:* `def:weil-finite-coordinate-kernel`, `def:weil-coordinate-stabilization-index`

Lean: `Weil.Arithmetic.coordinateKernel_stabilizes`

For every $M \geq M_*$, $K_M = K_{M_*}$.

Proof. Uses: def:weil-minimal-coordinate-kernel-dimension,def:weil-coordinate-stabilization-index,lemma:weil-coordinate-kernels-descend

Repeatedly applying the descending-kernel lemma gives $K_M \subseteq K_{M_*}$. Minimality of δ gives $\delta \le \dim K_M$, while inclusion gives $\dim K_M \le \dim K_{M_*} = \delta$. Thus the two finite-dimensional subspaces have equal dimension and one contains the other; the exact finite-dimensional equality criterion gives $K_M = K_{M_*}$. □

Lemma 8.4.10 (The intersection of all coordinate kernels is zero). Uses: def:weil-finite-coordinate-kernel

Lean: Weil.Arithmetic.iInf_coordinateKernel_eq_bot

One has $\bigcap_M K_M = \{0\}$.

Proof. Uses: def:weil-finite-coordinate-projection,def:weil-finite-coordinate-kernel

If v lies in every K_M, then for any coordinate n take $M = n + 1$. Membership in K_{n+1} gives $v_n = 0$. Thus every coordinate is zero, and function extensionality gives $v = 0$. Conversely the zero sequence lies in every kernel. □

Lemma 8.4.11 (The least-stage finite coordinates detect the shift span). Uses: def:weil-coordinate-stabilization-index,def:weil-finite-coordinate-projection

Lean: Weil.Arithmetic.coordinateProjection_injective

The map π_{M_*} *is injective.*

Proof. Uses: def:weil-finite-coordinate-kernel,lemma:weil-coordinate-kernels-stabilize,lemma:weil-coordinate-kernel-intersection-zero

By stabilization, K_{M_*} equals every K_M for $M \ge M_*$ and contains every earlier K_M by the descending inclusions. Hence it equals the intersection of all K_M, which is zero. A linear map has zero kernel exactly when it is injective. □

Definition 8.4.12 (Integral shift lattice). Uses: def:weil-integral-coefficient-sequence,def:weil-sequence-left-shift

Lean: Weil.Arithmetic.integralShiftLattice

Define the $\mathbf{Z}$-submodule

$$\mathcal{M}_f := \sum_{m \ge 0} \mathbf{Z}\, S^m \widehat{a} \subseteq \mathbf{Z}^{\mathbf{N}}.$$

This is the span of the explicitly defined integral shifts, not an arbitrary an arbitrary lattice in an ambient rational vector space.

Definition 8.4.13 (Integral finite-coordinate map). Uses: def:weil-integral-shift-lattice,def:weil-coordinate-stabilization-index

Lean: Weil.Arithmetic.integralCoordinateMap

Define

$$\widehat{\pi}_{M_*} : \mathcal{M}_f \longrightarrow \mathbf{Z}^{\mathrm{Fin}\, M_*}$$

by retaining the first M_* integer coordinates.

Lemma 8.4.14 (The integral finite-coordinate map is injective). Uses: def:weil-integral-finite-coordinate-map

Lean: Weil.Arithmetic.integralCoordinateMap_injective

The map $\widehat{\pi}_{M_*}$ *is injective.*

Proof. Uses: def:weil-integral-shift-lattice,def:weil-integral-finite-coordinate-map,lemma:weil-integral-coefficient-sequence-map,lemma:weil-coordinate-stabilization-kernel-zero

If an element maps to zero, then after coefficientwise inclusion $\mathbf{Z} \hookrightarrow \mathbf{Q}$ its rational sequence lies in $\ker \pi_{M_*}$. The preceding lemma makes that rational sequence zero. Injectivity of $\mathbf{Z} \hookrightarrow \mathbf{Q}$ then makes every integral coordinate zero. □

Lemma 8.4.15 (The integral shift lattice is finitely generated). Uses: def:weil-integral-shift-lattice,def:weil-integral-finite-coordinate-map

Lean: Weil.Arithmetic.integralShiftLattice_moduleFinite

The $\mathbf{Z}$*-module* $\mathcal{M}_f$ *is finitely generated.*

Proof. Uses: `lemma:weil-integral-finite-coordinate-map-injective`
The image of $\widehat{\pi}_{M_*}$ is a submodule of the finite free module $\mathbf{Z}^{\mathrm{Fin}\, M_*}$. Instantiate `IsNoetherian.noetherian` for that submodule: it is finitely generated. Pull a finite generating family back along the proved injection; the preimages generate $\mathcal{M}_f$ because the map identifies it with its image. □

Lemma 8.4.16 (The integral shift lattice is torsion-free). Uses: `def:weil-integral-shift-lattice`

Lean: `Weil.Arithmetic.integralShiftLattice_noZeroSMulDivisors`
The module $\mathcal{M}_f$ is torsion-free over $\mathbf{Z}$.

Proof. Uses: `def:weil-integral-shift-lattice`
If $n \neq 0$ and $nv = 0$, then coordinatewise $nv_j = 0$ in $\mathbf{Z}$. The integers have no zero divisors, so every $v_j = 0$, hence $v = 0$. □

Proposition 8.4.17 (The integral shift lattice is finite free). Uses: `def:weil-integral-shift-lattice`

Lean: `Weil.Arithmetic.integralShiftLattice_finiteFree`
The module $\mathcal{M}_f$ is finite free over $\mathbf{Z}$.

Proof. Uses: `lemma:weil-integral-shift-lattice-finite,lemma:weil-integral-shift-lattice-torsion-free`
A finitely generated torsion-free module over the principal ideal domain $\mathbf{Z}$ is free of finite rank. This is the exact pinned mathlib PID-module structure theorem specialized to the two preceding certificates. □

Definition 8.4.18 (Integral lattice shift endomorphism). Uses: `def:weil-integral-shift-lattice,def:weil-sequence-left-shift`

Lean: `Weil.Arithmetic.integralShiftEndomorphism`
Define $S_{\mathcal{M}} : \mathcal{M}_f \to \mathcal{M}_f$ by sequence shift. It preserves the lattice because it maps the generator $S^m\widehat{a}$ to $S^{m+1}\widehat{a}$.

Definition 8.4.19 (Integral shift characteristic polynomial). Uses: `def:weil-integral-lattice-shift-endomorphism`

Lean: `Weil.Arithmetic.integralShiftCharpoly`
Using the finite-free structure, define

$$C_f(U) := \mathrm{charpoly}(S_{\mathcal{M}}) = U^r + c_{r-1}U^{r-1} + \cdots + c_0 \in \mathbf{Z}[U].$$

Lemma 8.4.20 (Integral Cayley–Hamilton recurrence). Uses: `def:weil-integral-shift-characteristic-polynomial`

Lean: `Weil.Arithmetic.integralShiftCharpoly_recurrence`
For every $n \geq 0$,

$$\widehat{a}_{n+r} + c_{r-1}\widehat{a}_{n+r-1} + \cdots + c_0\widehat{a}_n = 0.$$

Proof. Uses: `def:weil-integral-lattice-shift-endomorphism,def:weil-integral-shift-characteristic-polynomial,prop:weil-integral-shift-lattice-finite-free`
Apply `LinearMap.aeval_self_charpoly` to the integral lattice shift, evaluate at $\widehat{a}$, and then at coordinate n. Every power of the shift advances the coordinate by that power, giving the displayed equality. □

Definition 8.4.21 (Integral Fatou denominator). Uses: `def:weil-integral-shift-characteristic-polynomial`

Lean: `Weil.Arithmetic.integralFatouDenominator`
Define

$$B_f(T) := 1 + c_{r-1}T + \cdots + c_0T^r \in \mathbf{Z}[T].$$

Definition 8.4.22 (Integral Fatou numerator). Uses: `def:weil-integral-rational-series-datum,def:weil-integral-fatou-denominator`

Lean: `Weil.Arithmetic.integralFatouNumerator`
Define $A_f(T)$ to be the truncation through degree $r-1$ of $B_f(T)f(T)$, using the integral coefficient witnesses of f.

Lemma 8.4.23 (Integral Fatou product identity). *Uses:* *def:weil-integral-fatou-denominator, def:weil-integral-fatou-numerator*

Lean: *Weil.Arithmetic.integralFatouDenominator_mul*

One has

$$B_f(T)f(T) = A_f(T) \quad \text{in } \mathbf{Z}[[T]].$$

Proof. Uses: `def:weil-integral-fatou-denominator`, `def:weil-integral-fatou-numerator`, `lemma:weil-integral-shift-cayley-hamilton`, `lemma:weil-integral-coefficient-sequence-map`

For degrees below r this is the defining truncation. In degree $n \geq r$, the coefficient of the product is the integral Cayley–Hamilton recurrence with index $n-r$, so it is zero. Equality of every coefficient proves the series identity over $\mathbf{Z}$; mapping to $\mathbf{Q}$ gives the original series identity. □

Theorem 8.4.24 (Constructive integral Fatou lemma). *Uses:* *def:weil-integral-rational-series-datum, def:weil-integral-fatou-denominator, def:weil-integral-fatou-numerator*

Lean: *Weil.Arithmetic.constructiveIntegralFatou*

For an integral rational-series datum,

$$f(T) = \frac{A_f(T)}{B_f(T)}, \qquad A_f, B_f \in \mathbf{Z}[T], \qquad B_f(0) = 1.$$

If $f(0) = 1$, then also $A_f(0) = 1$.

Proof. Uses: `lemma:weil-integral-fatou-product`

The product identity and $B_f(0) = 1$ give the displayed normalized fraction. Both polynomials are integral by their Definitions. At $T = 0$ the product identity reads $A_f(0) = B_f(0)f(0) = f(0)$, proving the last assertion. □

> **Formalization note.** This is the supplied notes, Integral Fatou lemma, Chapter 2 (printed pp. 12–13, physical PDF pp. 22–23), split into coordinate kernel stabilization, the explicit integral shift lattice, noetherian finite generation, PID freeness, integral Cayley–Hamilton, and coefficient recovery. The lattice is the explicit span from its Definition.

Definition 8.4.25 (Denominator-cleared integral polynomial). *Uses:*

Lean: `Weil.Arithmetic.integerNormalizationPolynomial`

Put $M_{\mathbf{Z}} :=$ nonZeroDivisors($\mathbf{Z}$). For every $P \in \mathbf{Q}[T]$, define

$$I_P := \mathrm{IsLocalization.integerNormalization}(M_{\mathbf{Z}}, P) \in \mathbf{Z}[T].$$

This is the total noncomputable Definition supplied by the pinned localization API.

Definition 8.4.26 (Clearing-denominator witness). *Uses:* `def:weil-rational-polynomial-integer-normalization`

Lean: `Weil.Arithmetic.integerNormalizationWitness`

For $P \in \mathbf{Q}[T]$, let

$$e_P := \mathrm{IsLocalization.integerNormalization_spec}(M_{\mathbf{Z}}, P).$$

Thus e_P has type

$$\exists b \in M_{\mathbf{Z}},\ \mathrm{map}_{\mathbf{Z},\mathbf{Q}}(I_P) = b \cdot P.$$

Define the witness

$$\delta_P := \left\langle e_P.\mathrm{choose}, e_P.\mathrm{choose_spec}.1 \right\rangle \in M_{\mathbf{Z}}.$$

This Definition isolates the only existential projection in the normalization construction.

Lemma 8.4.27 (The clearing-denominator witness is nonzero). *Uses:* *def:weil-rational-polynomial-integer-normalization-witness*

Lean: *Weil.Arithmetic.integerNormalizationWitness_ne_zero*

The underlying integer of δ_P is nonzero.

Proof. Uses: `def:weil-rational-polynomial-integer-normalization-witness`

The second coordinate of δ_P is a proof that its first coordinate belongs to nonZeroDivisors($\mathbf{Z}$). Unfold membership in that submonoid. Since $\mathbf{Z}$ is a domain, a non-zero-divisor is nonzero. This is a direct projection and definition reduction. □

Lemma 8.4.28 (Exact integer-normalization equation). *Uses:* *def:weil-rational-polynomial-integer-normalization*, *def:weil-rational-polynomial-integer-normalization-witness*

Lean: *Weil.Arithmetic.integerNormalizationPolynomial_map*

For every $P \in \mathbf{Q}[T]$,

$$\mathrm{map}_{\mathbf{Z},\mathbf{Q}}(I_P) = (\delta_P : \mathbf{Z}) \cdot P.$$

Proof. Uses: `def:weil-rational-polynomial-integer-normalization`, `def:weil-rational-polynomial-integer-normalization-witness`

Unfold δ_P. Apply the second proof component of e_P:

$$e_P.\mathrm{choose_spec.2}.$$

This is the displayed instance of `IsLocalization.integerNormalization_spec`. □

Lemma 8.4.29 (The denominator-cleared polynomial is nonzero). *Uses:* *def:weil-rational-polynomial-integer-normalization*

Lean: *Weil.Arithmetic.integerNormalizationPolynomial_ne_zero*

If $P \neq 0$, *then* $I_P \neq 0$.

Proof. Uses: `def:weil-rational-polynomial-integer-normalization`

Apply the equivalence $I_P = 0 \iff P = 0$ obtained from `IsLocalization.integerNormalization_eq_zero_iff` with

$$R = \mathbf{Z}, \qquad S = \mathbf{Q}, \qquad M = \mathrm{nonZeroDivisors}(\mathbf{Z}).$$

Its submonoid-inclusion argument is `le_rfl`. Negating the equivalence sends the hypothesis $P \neq 0$ to the target $I_P \neq 0$. □

Definition 8.4.30 (Primitive part of the denominator-cleared polynomial). Uses: `def:weil-rational-polynomial-integer-normalization`

Lean: `Weil.Arithmetic.rawPrimitivePart`

For $P \in \mathbf{Q}[T]$, define

$$B_P := \mathrm{Polynomial.primPart}(I_P) \in \mathbf{Z}[T].$$

The pinned mathlib convention is $\mathrm{Polynomial.primPart}(0) = 1$; consequently B_P is nonzero even in the total zero-polynomial branch.

Lemma 8.4.31 (Raw content factorization). *Uses:* *def:weil-rational-polynomial-integer-normalization*, *def:weil-rational-polynomial-raw-primitive-part*

Lean: *Weil.Arithmetic.integerNormalization_eq_content_mul_primPart*

For every P,

$$I_P = C(\mathrm{content}(I_P))\, B_P.$$

Proof. Uses: `def:weil-rational-polynomial-integer-normalization`, `def:weil-rational-polynomial-raw-primitive-part`

This is the single exact instance `Polynomial.eq_C_content_mul_primPart` applied to I_P. □

Lemma 8.4.32 (The raw primitive part has content one). *Uses:* *def:weil-rational-polynomial-raw-primitive-part*

Lean: *Weil.Arithmetic.rawPrimitivePart_content*

For every P, $\mathrm{content}(B_P) = 1$.

Proof. Uses: `def:weil-rational-polynomial-raw-primitive-part`

This is the single exact instance `Polynomial.content_primPart` applied to I_P. □

Lemma 8.4.33 (The raw primitive part is nonzero). *Uses:* *def:weil-rational-polynomial-raw-primitive-part*

Lean: *Weil.Arithmetic.rawPrimitivePart_ne_zero*

For every P, $B_P \neq 0$.

Proof. Uses: `def:weil-rational-polynomial-raw-primitive-part`

This is the single exact instance `Polynomial.primPart_ne_zero` applied to I_P. □

Definition 8.4.34 (Sign normalizer). Uses: def:weil-rational-polynomial-raw-primitive-part

Lean: Weil.Arithmetic.primitiveSign

Define

$$\sigma_P := \begin{cases} -1, & \text{trailingCoeff}(B_P) < 0, \\ 1, & \text{trailingCoeff}(B_P) \geq 0. \end{cases} \in \mathbf{Z}.$$

Because $B_P \neq 0$, its trailing coefficient is its first nonzero coefficient.

Lemma 8.4.35 (The sign normalizer is a unit sign). Uses: def:weil-rational-polynomial-primitive-sign

Lean: Weil.Arithmetic.primitiveSign_eq_one_or_neg_one

One has $\sigma_P = 1$ or $\sigma_P = -1$, and in particular $\sigma_P \neq 0$.

Proof. Uses: def:weil-rational-polynomial-primitive-sign

Split on the decidable inequality $\text{trailingCoeff}(B_P) < 0$ and unfold the two branches of the Definition. Each branch is an integer normalization. □

Definition 8.4.36 (Signed primitive integral polynomial). Uses: def:weil-rational-polynomial-raw-primitive-part,def:weil-rational-polynomial-primitive-sign

Lean: Weil.Arithmetic.signedPrimitivePart

Define

$$N_P := C(\sigma_P) B_P \in \mathbf{Z}[T].$$

Lemma 8.4.37 (The signed primitive polynomial has positive first coefficient). Uses: def:weil-rational-polynomial-signed-primitive-part

Lean: Weil.Arithmetic.signedPrimitivePart_trailingCoeff_pos

For every P,

$$0 < \text{trailingCoeff}(N_P).$$

Proof. Uses: def:weil-rational-polynomial-raw-primitive-part,def:weil-rational-polynomial-primitive-sign,def:weil-rational-polynomial-signed-primitive-part,lemma:weil-rational-polynomial-raw-primitive-part-nonzero

By Polynomial.coeff_natTrailingDegree_ne_zero, $\text{trailingCoeff}(B_P) \neq 0$. Split on whether this integer is negative. In the negative branch $N_P = -B_P$, so its trailing coefficient is the negative of a negative integer. In the other branch $N_P = B_P$; nonnegativity together with nonvanishing makes its trailing coefficient positive. □

Lemma 8.4.38 (The signed primitive polynomial has content one). Uses: def:weil-rational-polynomial-signed-primitive-part

Lean: Weil.Arithmetic.signedPrimitivePart_content

For every P, $\text{content}(N_P) = 1$.

Proof. Uses: def:weil-rational-polynomial-raw-primitive-part,def:weil-rational-polynomial-primitive-sign,def:weil-rational-polynomial-signed-primitive-part,lemma:weil-rational-polynomial-raw-primitive-part-content,lemma:weil-rational-polynomial-primitive-sign-dichotomy

Apply Polynomial.content_C_mul. In the two possible sign cases, $\text{normalize}(\sigma_P) = 1$. Substitute $\text{content}(B_P) = 1$; the resulting product is 1. □

Definition 8.4.39 (Primitive-normalization scalar). Uses: def:weil-rational-polynomial-integer-normalization,def:weil-rational-polynomial-integer-normalization-witness,def:weil-rational-polynomial-primitive-sign

Lean: Weil.Arithmetic.primitiveNormalizationScalar

Define

$$\lambda_P := \frac{\sigma_P \ \text{content}(I_P)}{(\delta_P : \mathbf{Z})} \in \mathbf{Q},$$

where numerator and denominator are mapped from $\mathbf{Z}$ to $\mathbf{Q}$.

Lemma 8.4.40 (The primitive-normalization scalar is nonzero). Uses: def:weil-rational-polynomial-primitive-scalar

Lean: Weil.Arithmetic.primitiveNormalizationScalar_ne_zero

If $P \neq 0$, then $\lambda_P \neq 0$.

Proof. Uses: def:weil-rational-polynomial-integer-normalization, def:weil-rational-polynomial-integer-normalization-witness, def:weil-rational-polynomial-primitive-sign, def:weil-rational-polynomial-primitive-scalar, lemma:weil-rational-polynomial-integer-normalization-witness-nonzero, lemma:weil-rational-polynomial-integer-normalization-nonzero, lemma:weil-rational-polynomial-primitive-sign-dichotomy

The denominator is nonzero by the witness-nonzero lemma. The sign is nonzero by its dichotomy. By `Polynomial.content_eq_zero_iff`, the content of I_P is nonzero because $I_P \neq 0$. The quotient of the resulting nonzero rational numerator by the nonzero rational denominator is nonzero. □

Definition 8.4.41 (Primitive integral normalization datum). Uses: def:weil-rational-polynomial-signed-primitive-part, def:weil-rational-polynomial-primitive-scalar

Lean: `Weil.Arithmetic.primitiveNormalization`

For every $P \in \mathbf{Q}[T]$, define

$$\mathcal{N}_{\mathbf{Z}}(P) := (N_P, \lambda_P) \in \mathbf{Z}[T] \times \mathbf{Q}, \qquad \mathrm{prim}_{\mathbf{Z}}(P) := N_P.$$

No property is bundled into this Definition.

Lemma 8.4.42 (Primitive-normalization factorization). Uses: def:weil-rational-polynomial-primitive-normalization

Lean: `Weil.Arithmetic.primitiveNormalization_factorization`

If $P \neq 0$, then

$$P = \lambda_P \, \mathrm{map}_{\mathbf{Z},\mathbf{Q}}(\mathrm{prim}_{\mathbf{Z}}(P)).$$

Proof. Uses: def:weil-rational-polynomial-integer-normalization, def:weil-rational-polynomial-integer-normalization-witness, def:weil-rational-polynomial-raw-primitive-part, def:weil-rational-polynomial-primitive-sign, def:weil-rational-polynomial-signed-primitive-part, def:weil-rational-polynomial-primitive-scalar, def:weil-rational-polynomial-primitive-normalization, lemma:weil-rational-polynomial-integer-normalization-witness-nonzero, lemma:weil-rational-polynomial-integer-normalization-equation, lemma:weil-rational-polynomial-raw-content-factorization, lemma:weil-rational-polynomial-primitive-sign-dichotomy

Map the raw content factorization to $\mathbf{Q}[T]$ and substitute the integer-normalization equation:

$$(\delta_P : \mathbf{Q})P = \mathrm{content}(I_P)\,\mathrm{map}(B_P).$$

The denominator is nonzero. In either sign case, $\mathrm{map}(N_P) = \sigma_P \,\mathrm{map}(B_P)$ and $\sigma_P^2 = 1$. Divide the displayed equality by δ_P, insert $\sigma_P^2 = 1$, and unfold λ_P. This finite field calculation is the asserted factorization. □

Lemma 8.4.43 (Primitive normalization identity). Uses: def:weil-rational-polynomial-primitive-normalization

Lean: `Weil.Arithmetic.primitiveNormalization_spec`

For nonzero P,

$$P = \lambda_P \, \mathrm{map}(\mathrm{prim}_{\mathbf{Z}}(P)), \qquad \mathrm{content}(\mathrm{prim}_{\mathbf{Z}}(P)) = 1, \qquad \lambda_P \neq 0,$$

and the first nonzero coefficient of $\mathrm{prim}_{\mathbf{Z}}(P)$ is positive.

Proof. Uses: def:weil-rational-polynomial-primitive-normalization, lemma:weil-rational-polynomial-signed-primitive-part-positive, lemma:weil-rational-polynomial-signed-primitive-part-content, lemma:weil-rational-polynomial-primitive-scalar-nonzero, lemma:weil-rational-polynomial-primitive-normalization-factorization

The four conjuncts are exactly the four cited single-property lemmas after unfolding the ordered pair $\mathcal{N}_{\mathbf{Z}}(P)$. □

Lemma 8.4.44 (Product of primitive integer polynomials is primitive). Uses:

Lean: `Weil.Arithmetic.IsPrimitive.mul`

If $P_0, Q_0 \in \mathbf{Z}[T]$ both have content 1, then P_0Q_0 has content 1.

Proof. Uses:

Instantiate mathlib's polynomial content-multiplication theorem over the GCD domain $\mathbf{Z}$:

$$\mathrm{content}(P_0Q_0) \sim \mathrm{content}(P_0)\,\mathrm{content}(Q_0) = 1.$$

Normalized contents that are associated to 1 equal 1. □

Definition 8.4.45 (Explicit normalized Gauss lift)**.** Uses: `def:weil-rational-polynomial-primitive-normalization`

Lean: `Weil.Arithmetic.normalizedGaussIntegralLift`

For nonzero $P \in \mathbf{Q}[T]$, put

$$c_P := \mathrm{prim}_{\mathbf{Z}}(P)(0) \in \mathbf{Z}$$

and define, without making a choice,

$$\mathcal{G}_{\mathbf{Z}}(P) := \begin{cases} \mathrm{prim}_{\mathbf{Z}}(P), & c_P = 1, \\ -\mathrm{prim}_{\mathbf{Z}}(P), & c_P \neq 1. \end{cases}$$

The second branch is deliberately total: the normalized Gauss lemma below proves that, under its hypotheses, $c_P \in \{1, -1\}$.

Lemma 8.4.46 (Normalized rational factors of an integral polynomial are integral)**.** *Uses:* *def:weil-normalized-gauss-integral-lift*

Lean: *Weil.Arithmetic.normalizedGaussFactor*

Let $C \in \mathbf{Z}[T]$ satisfy $C(0) = 1$. Suppose

$$\mathrm{map}(C) = PQ, \qquad P, Q \in \mathbf{Q}[T], \qquad P(0) = Q(0) = 1.$$

Then the explicitly defined polynomials $\mathcal{G}_{\mathbf{Z}}(P), \mathcal{G}_{\mathbf{Z}}(Q)$ map to P, Q, respectively. Moreover, each is the unique integral polynomial with the corresponding image.

Proof. Uses: `def:weil-rational-polynomial-primitive-normalization`, `def:weil-normalized-gauss-integral-lift`, `lemma:weil-rational-polynomial-primitive-normalization-identity`, `lemma:weil-primitive-integer-product`

Neither P nor Q is zero because its constant term is 1. Write $P = \lambda_P P_0$ and $Q = \lambda_Q Q_0$ with primitive integral P_0, Q_0. Then

$$\mathrm{map}(C) = \lambda_P \lambda_Q \,\mathrm{map}(P_0 Q_0).$$

Both C and $P_0 Q_0$ are primitive: their contents are 1, the first because $C(0) = 1$ and the second by the preceding lemma. Comparing normalized contents after clearing the nonzero denominator of $\lambda_P \lambda_Q$ gives $\lambda_P \lambda_Q = \pm 1$. Write $c_P = P_0(0)$ and $c_Q = Q_0(0)$. Evaluating at zero gives

$$\lambda_P c_P = 1, \qquad \lambda_Q c_Q = 1,$$

hence $c_P c_Q = \pm 1$. The two integer factors are therefore units ± 1, so $\lambda_P = c_P$ and $\lambda_Q = c_Q$ also belong to $\{\pm 1\}$. Unfolding $\mathcal{G}_{\mathbf{Z}}$ in the two possible cases proves the two mapping identities. Injectivity of $\mathbf{Z}[T] \to \mathbf{Q}[T]$ gives uniqueness. □

Definition 8.4.47 (Normalized polynomial gcd)**.** Uses:

Lean: `Weil.Arithmetic.normalizedPolynomialGCD`

For $A, B \in \mathbf{Z}[T]$ with $A(0) = B(0) = 1$, let $G_0 = \gcd(A, B)$ in the normalized-GCD monoid $\mathbf{Z}[T]$. Since $G_0(0)$ divides 1, it is a unit. Define G by multiplying G_0 by the unique sign for which $G(0) = 1$.

Definition 8.4.48 (Quotients by the normalized gcd)**.** Uses: `def:weil-normalized-polynomial-gcd`

Lean: `Weil.Arithmetic.normalizedGCDQuotients`

Define $A_{\mathrm{red}}, B_{\mathrm{red}} \in \mathbf{Z}[T]$ to be the exact quotients defined by the stored divisibility witnesses

$$A = G A_{\mathrm{red}}, \qquad B = G B_{\mathrm{red}}.$$

Lemma 8.4.49 (Normalized gcd factor identities)**.** *Uses:* *def:weil-normalized-polynomial-gcd*, *def:weil-normalized-gcd-quotients*

Lean: *Weil.Arithmetic.normalizedGCDQuotients_spec*

The defined quotients satisfy

$$A = G A_{\mathrm{red}}, \quad B = G B_{\mathrm{red}}, \quad A_{\mathrm{red}}(0) = B_{\mathrm{red}}(0) = 1.$$

Proof. Uses: `def:weil-normalized-polynomial-gcd`, `def:weil-normalized-gcd-quotients`

The first two equalities are the specifications of the exact quotient witnesses. Evaluate them at zero. Since $A(0) = B(0) = G(0) = 1$, cancellation in $\mathbf{Z}$ gives both quotient values equal to 1. □

Lemma 8.4.50 (The normalized gcd quotients are coprime). *Uses:* def:weil-normalized-gcd-quotients

Lean: Weil.Arithmetic.normalizedGCDQuotients_isCoprime

The polynomials A_{red} and B_{red} are coprime in $\mathbf{Z}[T]$ and remain coprime after scalar extension to every field of characteristic zero.

Proof. Uses: def:weil-normalized-polynomial-gcd,lemma:weil-normalized-gcd-factor-identities

If a nonunit divided both quotients, then its product with G would be a common divisor of A and B strictly larger than their gcd, contradicting the gcd universal property. Thus their gcd is a unit. Bezout coprimality over $\mathbf{Q}[T]$ follows from the PID structure, and the Bezout identity maps to any characteristic-zero field, proving preservation after scalar extension. □

Definition 8.4.51 (Reduced integral fraction). Uses: def:weil-normalized-gcd-quotients,def:weil-normalized-polynomial-fraction-datum

Lean: Weil.Arithmetic.reducedIntegralFraction

For an integral Fatou pair (A_f, B_f), define

$$\mathcal{F}_f^{\mathrm{red}} := (A_{\mathrm{red}}, B_{\mathrm{red}})$$

using the normalized gcd quotients, regarded also as a normalized fraction over $\mathbf{Q}$ by coefficient inclusion.

Proposition 8.4.52 (Reduced integral presentation). *Uses:* def:weil-integral-rational-series-datum,def:weil-reduced-integral-fraction

Lean: Weil.Arithmetic.reducedIntegralFraction_spec

The reduced fraction has coprime integral numerator and denominator, both with constant term 1, and its Taylor series is the original f.

Proof. Uses: lemma:weil-integral-fatou-product,lemma:weil-normalized-gcd-factor-identities,lemma:weil-normalized-gcd-quotients-coprime

Substitute $A_f = GA_{\mathrm{red}}$ and $B_f = GB_{\mathrm{red}}$ into $B_f f = A_f$. The polynomial G is nonzero because $G(0) = 1$; its image in $\mathbf{Q}[[T]]$ can be cancelled in the integral domain, yielding $B_{\mathrm{red}} f = A_{\mathrm{red}}$. The constant terms and coprimality are the two preceding lemmas. Multiplication by the formal inverse of the constant-one denominator gives the Taylor formula. □

Definition 8.4.53 (Counting-zeta integral-rational candidate). Uses: def:weil-arithmetic-factor-recovery-input,def:weil-rational-counting-zeta-fraction,def:weil-integral-rational-series-datum,def:weil-recovery-counting-zeta-integer-sequence

Lean: Weil.Arithmetic.countingZetaIntegralRationalCandidate

Take $f = Z_P$, the explicit integer sequence $\widehat{z}_{\mathcal{R},n}$, and the explicitly constructed rational fraction $\mathcal{F}_{P,\mathbf{Q}}$. These objects are the data fields of the required integral rational-series datum.

Proposition 8.4.54 (The explicit integer sequence maps to the counting zeta). *Uses:* def:weil-arithmetic-factor-recovery-candidate,def:weil-recovery-counting-zeta-integer-sequence-predicate

Lean: Weil.Arithmetic.finiteExtensionCountingZeta_integral

Every arithmetic factor-recovery candidate satisfies IsIntegerZetaSequence($\mathcal{R}$).

Proof. Uses: def:weil-recovery-counting-zeta-integer-sequence,def:weil-recovery-counting-zeta-integer-sequence-predicate,prop:weil-finite-extension-zeta-eq-orbit-zeta

The module01 equality identifies Z_P with the coefficientwise rational image of $Z_{\mathrm{orb},D}$. Apply the coefficient map $[T^n]$ to that series equality and unfold $\widehat{z}_{\mathcal{R},n}$. □

Definition 8.4.55 (Counting-zeta integral-rational datum). Uses: def:weil-arithmetic-factor-recovery-input,def:weil-counting-zeta-integral-rational-candidate,def:weil-integral-rational-series-datum

Lean: Weil.Arithmetic.countingZetaIntegralRationalData

Define the counting-zeta integral-rational datum from the data fields of its candidate and the integer-sequence comparison proof stored in the arithmetic factor-recovery input.

Definition 8.4.56 (Reduced integral numerator and denominator of the counting zeta)**.** Uses: def:weil-counting-zeta-integral-rational-datum,def:weil-reduced-integral-fraction

Lean: Weil.Arithmetic.countingZetaReducedIntegralFraction

Instantiate the constructive Fatou and normalized-gcd constructions and write

$$\mathcal{F}_P^{\mathrm{red}} = (A_P, B_P) \in \mathbf{Z}[T]^2.$$

Proposition 8.4.57 (Properties of the reduced integral counting-zeta fraction)**.** *Uses:* def:weil-counting-zeta-reduced-integral-fraction,def:weil-finite-extension-counting-zeta-series

Lean: Weil.Arithmetic.countingZetaReducedIntegralFraction_spec

One has

$$A_P(0) = B_P(0) = 1, \qquad \gcd(A_P, B_P) = 1, \qquad Z_P(T) = \frac{A_P(T)}{B_P(T)}$$

as a formal Taylor identity and as a rational-function identity.

Proof. Uses: prop:weil-counting-zeta-descent-compatible,prop:weil-rational-counting-zeta-fraction-taylor,prop:weil-finite-extension-counting-zeta-integral,thm:weil-constructive-integral-fatou,prop:weil-reduced-integral-presentation,prop:weil-regular-rational-germ-realizations

The first three cited results supply the valid integral rational-series datum. Constructive Fatou gives an integral pair; reduced presentation cancels its normalized gcd and proves the Taylor identity. Injectivity of the Taylor realization of regular rational germs converts the same equality into the rational-function identity. Its remaining fields are exactly the normalization and coprimality conclusions of reduced presentation. □

> **Formalization note.** The primitive normalization is backed by `Mathlib.RingTheory.Polynomial.GaussLemma`; the relevant exact local objects are `IsLocalization.integerNormalization` and its `integerNormalization_spec`, `Polynomial.primPart`, `Polynomial.content_primPart`, polynomial content multiplication, and the normalized-GCD instance. The mathematical source is the supplied notes, Chapter 2, *Normalized Gauss factors* (printed p. 13, physical PDF p. 23).

8.5 No cancellation and identification with the reduced fraction

Definition 8.5.1 (Algebraic-closure extension of a graded factor)**.** Uses: def:weil-graded-reciprocal-factor,def:weil-coefficient-data

Lean: Weil.Arithmetic.factorOverAlgebraicClosure

For $i \le B$, define

$$\overline{P}_{G,i} := \mathrm{map}_{E,\overline{E}}(P_{G,i}) \in \overline{E}[T].$$

Definition 8.5.2 (Ordinary-root multiset of a graded factor)**.** Uses: def:weil-graded-factor-algebraic-closure

Lean: Weil.Arithmetic.factorOrdinaryRoots

Define $\mathcal{R}_{G,i}$ to be the finite multiset of roots of $\overline{P}_{G,i}$ in the algebraically closed field $\overline{E}$, with multiplicities.

Lemma 8.5.3 (Zero is not an ordinary root of a graded factor)**.** *Uses:* def:weil-graded-factor-algebraic-closure

Lean: Weil.Arithmetic.zero_notMem_factorRoots

For every $i \le B$, $0 \notin \mathcal{R}_{G,i}$.

Proof. Uses: def:weil-graded-factor-ordinary-root-multiset,lemma:weil-graded-reciprocal-factor-constant

The constant coefficient of $P_{G,i}$ is 1, and scalar extension preserves it. Thus $\overline{P}_{G,i}(0) = 1 \neq 0$, so zero is not a root. □

Definition 8.5.4 (Reciprocal-root multiset of a graded factor)**.** Uses: def:weil-graded-factor-ordinary-root-multiset

Lean: Weil.Arithmetic.factorReciprocalRoots

Define

$$\mathcal{A}_{G,i} := \mathrm{map}(\rho \mapsto \rho^{-1})\mathcal{R}_{G,i}.$$

The preceding nonzero-root property will justify every inverse used later.

Lemma 8.5.5 (Reciprocal-root support is the eigenvalue support). *Uses:* def:weil-graded-factor-reciprocal-root-multiset

Lean: WeilArithmetic.mem_factorReciprocalRoots_iff_eigenspace_ne_bot

For every $\alpha \in \overline{E}^{\times}$,

$$\alpha \in \mathcal{A}_{G,i} \quad \Longleftrightarrow \quad E_{\alpha}\big((u_i)_{\overline{E}}\big) \neq \bot.$$

This is an equality of supports only; no assertion about algebraic multiplicity or generalized eigenspaces is made.

Proof. Uses: def:weil-graded-factor-reciprocal-root-multiset,lemma:weil-graded-factor-zero-not-root,thm:weil-root-reciprocal-conversion-after-scalar-extension

Membership in the mapped multiset means that $\alpha = \rho^{-1}$ for some $\rho \in \mathcal{R}_{G,i}$; the nonzero-root lemma makes this inversion legitimate. Membership in the root multiset is equivalent to $\overline{P}_{G,i}(\rho) = 0$. Apply the cited scalar-extension theorem with $\rho = \alpha^{-1}$ to identify this equation with nonvanishing of $E_{\alpha}((u_i)_{\overline{E}})$. The same equivalence in reverse supplies the converse membership. Nothing in this argument compares root order with the dimension of an eigenspace. □

Lemma 8.5.6 (Different degrees have disjoint ordinary roots). *Uses:* def:weil-arithmetic-factor-recovery-input, def:weil-graded-factor-ordinary-root-multiset

Lean: WeilArithmetic.factorRoots_disjoint_of_ne

If $i \neq j$, *then* $\mathcal{R}_{G,i}$ *and* $\mathcal{R}_{G,j}$ *are disjoint.*

Proof. Uses: def:weil-graded-factor-universal-purity-predicate,def:weil-graded-factor-reciprocal-root-multiset,lemma:weil-graded-factor-zero-not-root,lemma:weil-raw-purity-disjoint-weights

Suppose ρ belonged to both ordinary-root multisets. It is nonzero. Then $\alpha = \rho^{-1}$ belongs to both reciprocal-root multisets. The purity field of the recovery input therefore says that α is raw universally pure of weights i and j. The supplied global realization and the disjoint-weight lemma force $i = j$, contradicting the hypothesis. □

Lemma 8.5.7 (Different graded factors are coprime over the algebraic closure). *Uses:* def:weil-arithmetic-factor-recovery-input, def:weil-graded-factor-algebraic-closure

Lean: WeilArithmetic.factors_isCoprime_of_ne

If $i \neq j$, *then* $\overline{P}_{G,i}$ *and* $\overline{P}_{G,j}$ *are coprime in* $\overline{E}[T]$.

Proof. Uses: lemma:weil-different-degrees-disjoint-roots,lemma:weil-graded-reciprocal-factor-constant

If the gcd were nonconstant, it would have a root because $\overline{E}$ is algebraically closed. That root would occur in both factor root multisets, contradicting their disjointness. Hence the gcd is constant. Since both polynomials have constant coefficient 1, neither is zero, and a constant gcd is a unit; this is coprimality. □

Lemma 8.5.8 (Odd and even cohomological products are coprime). *Uses:* def:weil-arithmetic-factor-recovery-input, def:weil-alternating-rational-numerator, def:weil-alternating-rational-denominator

Lean: WeilArithmetic.parityProducts_isCoprime

The scalar extensions of N_G *and* D_G *to* $\overline{E}[T]$ *are coprime.*

Proof. Uses: def:weil-alternating-rational-numerator,def:weil-alternating-rational-denominator,lemma:weil-different-graded-factors-coprime

Every factor of N_G has odd degree index and every factor of D_G has even degree index, so no index occurs on both sides. The pairwise coprimality lemma applies to every cross pair. Induction over the two finite products, using that a product is coprime to Q exactly when each factor is coprime to Q, gives coprimality of the two products. □

Lemma 8.5.9 (The reduced integral factors remain coprime after scalar extension). *Uses:* def:weil-counting-zeta-reduced-integral-fraction, def:weil-coefficient-data

Lean: WeilArithmetic.reducedCountingFactors_isCoprime_map

The images of A_P *and* B_P *in* $\overline{E}[T]$ *are coprime.*

Proof. Uses: prop:weil-counting-zeta-reduced-integral-fraction

The reduced-fraction proposition provides a Bezout identity over $\mathbf{Q}[T]$. Map that finite polynomial identity through $\mathbf{Q} \hookrightarrow \overline{E}$. The image identity exhibits the two extended polynomials as coprime. □

Lemma 8.5.10 (Cross-product identity for the two zeta presentations). *Uses:* def:weil-arithmetic-factor-recovery-input, def:weil-alternating-rational-numerator, def:weil-alternating-rational-denominator, def:weil-counting-zeta-reduced-integral-fraction

Lean: Weil.Arithmetic.parityReduced_crossProduct

Inside $\overline{E}[T]$,

$$\overline{N}_G\,\overline{B}_P = \overline{A}_P\,\overline{D}_G.$$

Proof. Uses: cor:weil-zeta-factorization-after-coefficient-extension, lemma:weil-alternating-determinant-normalized-fraction, prop:weil-counting-zeta-reduced-integral-fraction, prop:weil-regular-rational-germ-realizations

The cohomological factorization and normalized-fraction formula represent the coefficient extension of Z_P by N_G/D_G. The reduced integral presentation represents the same series by A_P/B_P. Injectivity of the Taylor realization of normalized rational germs makes their rational functions equal. Apply the cross-multiplication criterion for nonzero denominators, then extend coefficients to $\overline{E}$. □

Lemma 8.5.11 (A reduced cross-product forces numerator divisibility). *Uses:*

Lean: Weil.Arithmetic.isCoprime_crossProduct_dvd

Let K be a field. If $\gcd(N, D) = 1$, $\gcd(A, B) = 1$, and $NB = AD$, then $N \mid A$ and $A \mid N$.

Proof. Uses:

From $NB = AD$, one has $N \mid AD$. Euclid's lemma and $\gcd(N, D) = 1$ give $N \mid A$. Reversing the equality gives $A \mid NB$; Euclid's lemma and $\gcd(A, B) = 1$ give $A \mid N$. □

Lemma 8.5.12 (Normalized mutually divisible polynomials are equal). *Uses:*

Lean: Weil.Arithmetic.eq_of_dvd_dvd_coeffZero_one

Let K be a field. If $N \mid A$, $A \mid N$, and $N(0) = A(0) = 1$, then $N = A$.

Proof. Uses:

Mutual divisibility in the polynomial domain makes N and A associates, so $N = cA$ for a unit constant $c \in K^\times$. Evaluating at zero gives $1 = c \cdot 1$, hence $c = 1$ and $N = A$. □

Proposition 8.5.13 (Cohomological parity products are the reduced integral factors). *Uses:* def:weil-arithmetic-factor-recovery-input, def:weil-alternating-rational-numerator, def:weil-alternating-rational-denominator, def:weil-counting-zeta-reduced-integral-fraction

Lean: Weil.Arithmetic.parityProducts_eq_reducedFactors

Inside $\overline{E}[T]$,

$$\overline{N}_G = \overline{A}_P, \qquad \overline{D}_G = \overline{B}_P.$$

Proof. Uses: lemma:weil-cohomological-parity-products-coprime, lemma:weil-reduced-counting-zeta-factors-coprime-over-closure, lemma:weil-parity-reduced-cross-product, lemma:weil-reduced-cross-product-numerator-divides, lemma:weil-normalized-mutual-divisibility-equality, lemma:weil-alternating-rational-numerator-constant, lemma:weil-alternating-rational-denominator-constant, prop:weil-counting-zeta-reduced-integral-fraction

Apply the reduced cross-product divisibility lemma to the two coprime pairs. The numerator polynomials divide one another, and all their constant coefficients are 1; normalized mutual divisibility gives $\overline{N}_G = \overline{A}_P$. Apply the same argument to the denominators, or substitute the numerator equality back into the cross-product and cancel the nonzero numerator, to obtain $\overline{D}_G = \overline{B}_P$. □

Lemma 8.5.14 (The odd product is the reduced numerator). *Uses:* def:weil-arithmetic-factor-recovery-input, def:weil-alternating-rational-numerator, def:weil-counting-zeta-reduced-integral-fraction

Lean: Weil.Arithmetic.oddParityProduct_eq_reducedNumerator

Inside $\overline{E}[T]$, one has $\overline{N}_G = \overline{A}_P$.

Proof. Uses: prop:weil-parity-products-equal-reduced-integral-factors

Project the numerator equality from the parity-product proposition. □

Lemma 8.5.15 (The even product is the reduced denominator). *Uses:* def:weil-arithmetic-factor-recovery-input, def:weil-alternating-rational-denominator, def:weil-counting-zeta-reduced-integral-fraction

Lean: Weil.Arithmetic.evenParityProduct_eq_reducedDenominator

Inside $\overline{E}[T]$, one has $\overline{D}_G = \overline{B}_P$.

Proof. Uses: prop:weil-parity-products-equal-reduced-integral-factors
Project the denominator equality from the parity-product proposition. □

Corollary 8.5.16 (Arithmetic algebraicity of every reciprocal root). Uses: def:weil-arithmetic-factor-recovery-input, def:weil-graded-factor-reciprocal-root-multiset

Lean: Weil.Arithmetic.factorReciprocalRoot_isAlgebraic
Every element of every $\mathcal{A}_{G,i}$ is algebraic over $\mathbf{Q}$.

Proof. Uses: def:weil-alternating-rational-numerator,def:weil-alternating-rational-denominator,lemma:weil-graded-factor-zero-not-root,prop:weil-parity-products-equal-reduced-integral-factors
An ordinary root ρ of $P_{G,i}$ is a root of N_G or D_G according to the parity of i. The parity-product proposition makes it a root of the integer polynomial A_P or B_P, so ρ is algebraic over $\mathbf{Q}$. It is nonzero, and the inverse of an algebraic element is algebraic. Thus the reciprocal root ρ^{-1} is algebraic over $\mathbf{Q}$. □

> **Status.** This corollary is the first place where arithmetic algebraicity is available. The proof of pairwise root disjointness used raw universal realizations and did not use this corollary, rational coefficients of individual factors, their integrality, or independence of ℓ.

> **Formalization note.** This is the no-cancellation argument of the supplied notes, Chapter 14 (printed pp. 74–75, physical PDF pp. 84–85), which expands the reference in Deligne, Weil II, Corollary (3.3.9), article p. 207/PDF p. 72, to Weil I, implication (1.7) to (1.6). SGA 7 XXI (5.2.2) is not imported here: the reduced-fraction comparison is proved explicitly.

8.6 Rational and integral individual factors

Definition 8.6.1 (Galois conjugate of a polynomial). Uses: def:weil-coefficient-data

Lean: Weil.Arithmetic.galoisConjugatePolynomial
For $Q = \sum c_n T^n \in \overline{E}[T]$ and $\sigma \in \mathrm{Aut}(\overline{E}/\mathbf{Q})$, define

$$\sigma_* Q := \sum \sigma(c_n) T^n.$$

Lemma 8.6.2 (Rational polynomials are fixed by polynomial conjugation). Uses: def:weil-polynomial-galois-conjugate

Lean: Weil.Arithmetic.galoisConjugate_mapRat
For $C \in \mathbf{Q}[T]$, $\sigma_*(\mathrm{map}\, C) = \mathrm{map}\, C$.

Proof. Uses: def:weil-polynomial-galois-conjugate
Every coefficient of $\mathrm{map}\, C$ lies in the image of $\mathbf{Q}$, and σ fixes that image pointwise. Coefficient extensionality proves the equality. □

Lemma 8.6.3 (Polynomial conjugation transports roots with multiplicity). Uses: def:weil-polynomial-galois-conjugate

Lean: Weil.Arithmetic.roots_galoisConjugate
For nonzero $Q \in \overline{E}[T]$, the root multiset of $\sigma_* Q$ is the image under σ of the root multiset of Q.

Proof. Uses: def:weil-polynomial-galois-conjugate
For each ρ, evaluation commutes with coefficient conjugation:

$$(\sigma_* Q)(\sigma\rho) = \sigma(Q(\rho)).$$

Thus roots correspond bijectively. For multiplicity m, divisibility by $(T-\rho)^m$ is carried by the polynomial automorphism to divisibility by $(T-\sigma\rho)^m$, and the failure of divisibility at $m+1$ is reflected by its inverse. Hence multiplicities agree. □

Lemma 8.6.4 (The reduced parity roots are classified by weight). Uses: def:weil-arithmetic-factor-recovery-input, def:weil-counting-zeta-reduced-integral-fraction, def:weil-graded-factor-reciprocal-root-multiset

Lean: Weil.Arithmetic.reducedParityRoots_weightClassification
Fix $i \le B$. In the parity polynomial A_P for odd i, or B_P for even i, the multiset of nonzero ordinary roots ρ for which ρ^{-1} is raw universally pure of weight i is exactly $\mathcal{R}_{G,i}$.

Proof. Uses: `def:weil-alternating-rational-numerator`,`def:weil-alternating-rational-denominator`,`def:weil-graded-factor-universal-purity-predicate`,`lemma:weil-different-degrees-disjoint-roots`,`lemma:weil-raw-purity-disjoint-weights`,`prop:weil-parity-products-equal-reduced-integral-factors`

The parity-product equality writes the relevant reduced polynomial as the finite product of all $P_{G,j}$ of the specified parity. Its root multiset is the multiset sum of their root multisets. Pairwise disjointness says each root occurs in exactly one degree j, with the same multiplicity as in that factor. The purity certificate gives weight j to its reciprocal. If that reciprocal also has weight i, weight disjointness gives $j = i$. Conversely every root of $P_{G,i}$ has reciprocal weight i. This proves equality, including multiplicity. □

Definition 8.6.5 (Raw weight stratum of the reduced parity factor). Uses: `def:weil-arithmetic-factor-recovery-input`,`def:weil-counting-zeta-reduced-integral-fraction`,`def:weil-raw-universal-realization-purity`

Lean: `Weil.Arithmetic.recoveryRawWeightStratum`

For $i \le B$, let $\overline{C}_{G,i} \in \overline{E}[T]$ be the scalar extension of A_P when i is odd and of B_P when i is even. Define $\mathscr{S}_{G,i}$ to be the submultiset of $\mathrm{roots}(\overline{C}_{G,i})$ consisting of those ρ such that $\rho \neq 0$ and $\mathrm{RawPure}_q(\langle \rho^{-1}, \mathrm{inv_ne_zero}(h_\rho)\rangle, i)$, where $h_\rho : \rho \neq 0$ is the first field. Use classical decidability for this dependent multiset filter. Multiplicities are inherited from the root multiset of $\overline{C}_{G,i}$.

Definition 8.6.6 (Complete normalized raw weight-stratum factor). Uses: `def:weil-recovery-raw-weight-stratum`

Lean: `Weil.Arithmetic.IsCompleteRawWeightStratumFactor`

For $Q \in \overline{E}[T]$, define $\mathrm{CompleteRawWeightFactor}_{G,i}(Q)$ to mean

$$Q(0) = 1 \quad \text{and} \quad \mathrm{roots}(Q) = \mathscr{S}_{G,i}$$

as multisets.

Lemma 8.6.7 (The degree-factor roots are exactly the raw weight stratum). Uses: *def:weil-recovery-raw-weight-stratum*, *def:weil-graded-factor-algebraic-closure*

Lean: *Weil.Arithmetic.factorRoots_eq_recoveryRawWeightStratum*

For every $i \le B$,

$$\mathrm{roots}(\overline{P}_{G,i}) = \mathscr{S}_{G,i}.$$

Proof. Uses: `lemma:weil-reduced-parity-roots-classified-by-weight`

This is the reduced-parity classification lemma after unfolding $\mathscr{S}_{G,i}$. □

Lemma 8.6.8 (The scalar-extended degree factor is normalized). Uses: *def:weil-graded-factor-algebraic-closure*

Lean: *Weil.Arithmetic.factorOverAlgebraicClosure_coeffZero*

For every $i \le B$, one has $\overline{P}_{G,i}(0) = 1$.

Proof. Uses: `lemma:weil-graded-reciprocal-factor-constant`

The coefficient map $E \to \overline{E}$ sends the constant coefficient $P_{G,i}(0) = 1$ to one. □

Lemma 8.6.9 (The degree factor is the complete raw weight stratum). Uses: *def:weil-complete-raw-weight-stratum-factor-predicate*, *def:weil-graded-factor-algebraic-closure*

Lean: *Weil.Arithmetic.factor_isCompleteRawWeightStratum*

For every $i \le B$,

$$\mathrm{CompleteRawWeightFactor}_{G,i}(\overline{P}_{G,i}).$$

Proof. Uses: `lemma:weil-graded-factor-roots-equal-raw-weight-stratum`,`lemma:weil-graded-factor-algebraic-closure-constant`

Use the preceding root-multiset equality and normalization lemma as the two fields of the complete-factor predicate. □

Lemma 8.6.10 (A normalized raw weight-stratum factor is unique). Uses: *def:weil-complete-raw-weight-stratum-factor-predicate*

Lean: *Weil.Arithmetic.completeRawWeightStratumFactor_unique*

If $Q, Q' \in \overline{E}[T]$ both satisfy $\mathrm{CompleteRawWeightFactor}_{G,i}$, *then $Q = Q'$.*

Proof. Uses: def:weil-coefficient-data,def:weil-complete-raw-weight-stratum-factor-predicate
Both polynomials are nonzero because their value at zero is one. Over the algebraically closed field $\overline{E}$ selected by the coefficient datum, equality of root multisets with multiplicity makes them nonzero scalar multiples of one another. Evaluating at zero shows that the scalar is one. □

Lemma 8.6.11 (The degree factor is recovered uniquely over the coefficient field)**.** Uses: def:weil-graded-reciprocal-factor,def:weil-graded-factor-algebraic-closure,def:weil-complete-raw-weight-stratum-factor-predicate

Lean: Weil.Arithmetic.factor_eq_of_map_isCompleteRawWeightStratum
Let $Q \in E[T]$. If its scalar extension $\overline{Q}$ satisfies CompleteRawWeightFactor$_{G,i}$*, then $Q = P_{G,i}$.*

Proof. Uses: lemma:weil-graded-factor-complete-raw-weight-stratum,lemma:weil-complete-raw-weight-stratum-factor-unique
The uniqueness lemma gives $\overline{Q} = \overline{P}_{G,i}$. The coefficient map $E[T] \to \overline{E}[T]$ is injective, so it reflects this equality. □

Definition 8.6.12 (No-cancellation and weight-recovery certificate)**.** Uses: def:weil-arithmetic-factor-recovery-input,def:weil-complete-raw-weight-stratum-factor-predicate

Lean: Weil.Arithmetic.NoCancellationRecoveryCertificate
A no-cancellation and weight-recovery certificate consists of the following already typed fields:

(i) the scalar extensions of N_G and D_G are coprime;

(ii) they equal the scalar extensions of the reduced numerator A_P and denominator B_P, respectively;

(iii) every $\overline{P}_{G,i}$ is a complete normalized raw weight-stratum factor;

(iv) every $Q \in E[T]$ whose scalar extension is that complete weight stratum equals $P_{G,i}$.

Theorem 8.6.13 (No cancellation and recovery by weight)**.** Uses: def:weil-no-cancellation-recovery-certificate

Lean: Weil.Arithmetic.noCancelationRecovery
Every arithmetic factor-recovery input admits a Definition 8.6.12.

Proof. Uses: lemma:weil-cohomological-parity-products-coprime,prop:weil-parity-products-equal-reduced-integral-factors,lemma:weil-graded-factor-complete-raw-weight-stratum,lemma:weil-graded-factor-recovered-uniquely-by-weight
Use the four displayed results for fields (i)–(iv), in that order. This is record assembly only: all cancellation, classification, normalization, and injectivity arguments occur in the cited atomic lemmas. □

Lemma 8.6.14 (The degree-i root multiset is Galois stable)**.** Uses: def:weil-arithmetic-factor-recovery-input,def:weil-graded-factor-ordinary-root-multiset

Lean: Weil.Arithmetic.factorRoots_galoisStable
For every $\sigma \in \mathrm{Aut}(\overline{E}/\mathbf{Q})$,

$$\mathrm{map}(\sigma)\mathcal{R}_{G,i} = \mathcal{R}_{G,i}.$$

Proof. Uses: lemma:weil-rational-polynomial-galois-fixed,lemma:weil-polynomial-galois-roots,lemma:weil-raw-purity-galois-invariant,lemma:weil-reduced-parity-roots-classified-by-weight,prop:weil-counting-zeta-reduced-integral-fraction
The relevant reduced parity polynomial has rational coefficients, so it is fixed by σ and its full root multiset is permuted by σ. The classification lemma identifies $\mathcal{R}_{G,i}$ as the roots whose inverses have weight i. Since $\sigma(\rho)^{-1} = \sigma(\rho^{-1})$, Galois invariance of raw purity shows that this submultiset is preserved. Applying the same argument to σ^{-1} gives equality rather than only inclusion. □

Lemma 8.6.15 (The individual factor is fixed coefficientwise)**.** Uses: def:weil-arithmetic-factor-recovery-input,def:weil-graded-factor-algebraic-closure,def:weil-polynomial-galois-conjugate

Lean: Weil.Arithmetic.factor_galoisFixed
For every $\sigma \in \mathrm{Aut}(\overline{E}/\mathbf{Q})$,

$$\sigma_* \overline{P}_{G,i} = \overline{P}_{G,i}.$$

Proof. Uses: lemma:weil-polynomial-galois-roots,lemma:weil-individual-factor-root-multiset-galois-stable,lemma:weil-graded-reciprocal-factor-constant
The two polynomials have the same root multiset by the root-transport lemma and Galois stability. Therefore they differ by a nonzero scalar. Polynomial conjugation preserves the constant coefficient 1, so evaluating at zero makes that scalar equal to 1. □

Definition 8.6.16 (Rational descent of a fixed scalar). Uses: `def:weil-realization-field-interface,def:weil-rational-fixed-scalar-retraction-from-fixed-to-rational`

Lean: `Weil.Arithmetic.rationalDescentOfFixedScalar`

For $x \in \overline{E}$ and an explicit certificate $h_x : \forall \sigma, \sigma(x) = x$, define

$$\mathrm{rat}(x, h_x) := \mathrm{rat}_{h_{\mathrm{FR}}}(x, h_x) \in \mathbf{Q},$$

where h_{FR} is field (iii) of the realization-field interface and $\mathrm{rat}_{h_{\mathrm{FR}}}$ is its canonical derived retraction.

Lemma 8.6.17 (Rational descent maps to the fixed scalar). Uses: `def:weil-rational-descent-of-fixed-scalar`

Lean: `Weil.Arithmetic.coe_rationalDescentOfFixedScalar`

One has

$$\mathrm{map}_{\mathbf{Q},\overline{E}}(\mathrm{rat}(x, h_x)) = x.$$

Proof. Uses: `def:weil-realization-field-interface,def:weil-rational-descent-of-fixed-scalar,lemma:weil-rational-fixed-scalar-retraction-from-fixed-to-rational-spec`

Unfold the rational-descent Definition. The result is the retraction specification derived from field (iii) of the realization-field interface, instantiated at (x, h_x). □

Definition 8.6.18 (Rational individual factor). Uses: `def:weil-arithmetic-factor-recovery-input,def:weil-graded-factor-algebraic-closure,def:weil-rational-descent-of-fixed-scalar`

Lean: `Weil.Arithmetic.rationalIndividualFactor`

Given the explicit invariance certificate for $\overline{P}_{G,i}$, define

$$P_{i,\mathbf{Q}}(T) := \sum_{n=0}^{b_{i,\ell}} \mathrm{rat}\big([T^n]\overline{P}_{G,i}, h_{i,n}\big) T^n \in \mathbf{Q}[T],$$

where $h_{i,n}$ is the coefficientwise consequence of the supplied invariance certificate.

Lemma 8.6.19 (Scalar extension of the rational individual factor). Uses: `def:weil-rational-individual-factor,def:weil-graded-factor-algebraic-closure`

Lean: `Weil.Arithmetic.map_rationalIndividualFactor`

The coefficient extension of $P_{i,\mathbf{Q}}$ to $\overline{E}[T]$ equals $\overline{P}_{G,i}$.

Proof. Uses: `def:weil-rational-individual-factor,lemma:weil-rational-descent-of-fixed-scalar-map,prop:weil-reciprocal-factor-degree-equals-dimension`

For $n \le b_{i,\ell}$, the coefficient equality is the scalar-descent specification. For $n > b_{i,\ell}$, the coefficient of $\overline{P}_{G,i}$ vanishes because its degree is $b_{i,\ell}$, and the finite sum defining $P_{i,\mathbf{Q}}$ has no such term. Coefficient extensionality proves the polynomial equality. □

Theorem 8.6.20 (Rationality of every individual factor). Uses: `def:weil-arithmetic-factor-recovery-input,def:weil-rational-individual-factor`

Lean: `Weil.Arithmetic.individualFactor_rational`

For each $i \le B$, the explicitly constructed $P_{i,\mathbf{Q}} \in \mathbf{Q}[T]$ has constant coefficient 1 and maps to $P_{G,i}$ over E.

Proof. Uses: `lemma:weil-individual-factor-galois-fixed,lemma:weil-rational-individual-factor-map,lemma:weil-graded-reciprocal-factor-constant`

Instantiate the rational-factor Definition with the proved Galois invariance certificate. The map equality over $\overline{E}$ is the preceding lemma. Faithfulness of $E \hookrightarrow \overline{E}$ reflects equality with $P_{G,i}$ over E. The constant coefficient maps to 1, hence is 1 in $\mathbf{Q}$. □

Corollary 8.6.21 (Classical all-embeddings weight of reciprocal roots). Uses: `def:weil-arithmetic-factor-recovery-input,def:weil-rational-individual-factor,def:weil-universal-scalar-weight`

Lean: `Weil.Arithmetic.individualFactor_classicalWeight`

Every reciprocal root α of $P_{i,\mathbf{Q}}$ is an algebraic number, and every embedding $\tau : \mathbf{Q}(\alpha) \hookrightarrow \mathbf{C}$ satisfies

$$|\tau(\alpha)| = q^{i/2}.$$

Proof. Uses: cor:weil-reciprocal-roots-algebraic-over-q, lemma:weil-rational-individual-factor-map, lemma:weil-raw-purity-to-classical-weight, def:weil-graded-factor-universal-purity-predicate

The scalar-extension equality identifies the reciprocal roots with those of $P_{G,i}$. Their algebraicity is the no-cancellation corollary, and their raw weight i is the purity field of the recovery input. Apply the raw-to-classical weight lemma to each root. □

Definition 8.6.22 (Same-parity complementary rational factor)**.** Uses: def:weil-rational-individual-factor, def:weil-bounded-graded-based-automorphism-family

Lean: Weil.Arithmetic.sameParityComplementaryFactor

For $i \le B$, define

$$Q_{i,\mathbf{Q}}(T) := \prod_{\substack{0 \le j \le B \\ j \equiv i \pmod 2 \\ j \neq i}} P_{j,\mathbf{Q}}(T).$$

Lemma 8.6.23 (Integral parity polynomial factors through the degree-i factor)**.** *Uses: def:weil-counting-zeta-reduced-integral-fraction, def:weil-rational-individual-factor, def:weil-same-parity-complementary-factor*

Lean: *Weil.Arithmetic.reducedParity_factorization*

If i is odd, then in $\mathbf{Q}[T]$

$$A_P = P_{i,\mathbf{Q}} Q_{i,\mathbf{Q}}.$$

If i is even, then

$$B_P = P_{i,\mathbf{Q}} Q_{i,\mathbf{Q}}.$$

Every factor displayed on the right has constant coefficient 1.

Proof. Uses: def:weil-alternating-rational-numerator, def:weil-alternating-rational-denominator, def:weil-same-parity-complementary-factor, prop:weil-parity-products-equal-reduced-integral-factors, lemma:weil-rational-individual-factor-map, lemma:weil-alternating-rational-numerator-constant, lemma:weil-alternating-rational-denominator-constant

After scalar extension to $\overline{E}$, the claimed equality is exactly the definition of the relevant parity product together with the parity-product identification. Injectivity of $\mathbf{Q}[T] \to \overline{E}[T]$ reflects the equality to $\mathbf{Q}[T]$. Each individual constant coefficient is 1, so their finite complementary product also has value 1 at zero. □

Definition 8.6.24 (Integral individual-factor candidate)**.** Uses: def:weil-rational-individual-factor

Lean: Weil.Arithmetic.IntegralIndividualFactorCandidate

For a rational individual factor, an integral candidate is a polynomial $\widetilde{P}_i \in \mathbf{Z}[T]$ whose coefficientwise image in $\mathbf{Q}[T]$ is $P_{i,\mathbf{Q}}$.

Definition 8.6.25 (Integral individual factor)**.** Uses: def:weil-rational-individual-factor, def:weil-normalized-gauss-integral-lift

Lean: Weil.Arithmetic.integralIndividualFactor

For $i \le B$, define the integral polynomial

$$P_{i,\mathbf{Z}}(T) := \mathcal{G}_{\mathbf{Z}}\big(P_{i,\mathbf{Q}}\big) \in \mathbf{Z}[T].$$

The input of $\mathcal{G}_{\mathbf{Z}}$ is nonzero because $P_{i,\mathbf{Q}}(0) = 1$ by its defining determinant formula.

Theorem 8.6.26 (Integrality of every individual factor)**.** *Uses: def:weil-arithmetic-factor-recovery-input, def:weil-integral-individual-factor, def:weil-integral-individual-factor-candidate*

Lean: *Weil.Arithmetic.individualFactor_integral*

For every $i \le B$, the explicitly defined $P_{i,\mathbf{Z}}$ maps to $P_{i,\mathbf{Q}}$ over $\mathbf{Q}$, has constant coefficient 1*, and maps to $P_{G,i}$ over E. It is the unique Definition 8.6.24.*

Proof. Uses: def:weil-integral-individual-factor, lemma:weil-integral-parity-polynomial-factorization, lemma:weil-normalized-gauss-factor, thm:weil-individual-factor-rational

Use $C = A_P$ in odd degree and $C = B_P$ in even degree. The preceding factorization writes map C as $P_{i,\mathbf{Q}} Q_{i,\mathbf{Q}}$, with all three constant coefficients equal to 1. The normalized Gauss-factor lemma says precisely that the Definition $\mathcal{G}_{\mathbf{Z}}(P_{i,\mathbf{Q}}) = P_{i,\mathbf{Z}}$ maps to $P_{i,\mathbf{Q}}$ and is the unique such integral polynomial. Its constant term maps to 1, so injectivity of $\mathbf{Z} \to \mathbf{Q}$ makes that constant term equal to 1. Composing coefficient maps with the rational-factor theorem gives the equality with $P_{G,i}$ over E. □

Corollary 8.6.27 (Frobenius reciprocal roots are algebraic integers). *Uses:* def:weil-arithmetic-factor-recovery-input, def:weil-integral-individual-factor

Lean: Weil.Arithmetic.factorReciprocalRoot_isIntegral

Every reciprocal root α of $P_{G,i}$ is an algebraic integer.

Proof. Uses: thm:weil-individual-factor-integral, prop:weil-reciprocal-factor-degree-equals-dimension, lemma:weil-graded-factor-zero-not-root

Write $P_{i,\mathbf{Z}}(T) = 1 + c_1 T + \cdots + c_b T^b$. If α is a reciprocal root, then $\alpha \neq 0$ and $P_{i,\mathbf{Z}}(\alpha^{-1}) = 0$. Multiplication by α^b gives

$$\alpha^b + c_1 \alpha^{b-1} + \cdots + c_b = 0.$$

This is a monic polynomial over $\mathbf{Z}$, so it is the defining certificate that α is integral over $\mathbf{Z}$. □

> **Formalization note.** The order in this section is exactly the supplied notes, Chapter 14, printed pp. 75–76/physical PDF pp. 85–86. Deligne, Weil II, Corollary (3.3.9) states the integral and ℓ-independent conclusion but refers the arithmetic part back to Weil I; it is not used as a proof leaf. Deligne's SGA 7 XXI (5.2.2) integrality input in the proof of Weil II, Corollary (3.3.3), is likewise not imported.

8.7 Compatible realizations and the unique weight strata

Definition 8.7.1 (Compatible arithmetic-realization family candidate). Uses: def:weil-arithmetic-factor-recovery-input, def:weil-global-complex-realization-candidate

Lean: Weil.Arithmetic.CompatibleRealizationFamilyCandidate

Fix one prime-power datum and one cohomological bound B. A compatible arithmetic-realization family candidate consists of the following stored data:

1. a type Λ of realizations and a base index $\lambda_* : \Lambda$;
2. a global embedding $\iota_{\overline{\mathbf{Q}}} : \overline{\mathbf{Q}} \hookrightarrow \mathbf{C}$;
3. for every $\lambda : \Lambda$, a coefficient datum $(E_\lambda, \overline{E}_\lambda)$, a coefficient prime $\ell_\lambda \neq p$, and an arithmetic factor-recovery input $\mathcal{R}_\lambda$ of bound B;
4. one common finite-extension point-counting datum $\mathcal{P}$;
5. for every λ, identifications of the prime-power datum, the bound, and the point-counting component of $\mathcal{R}_\lambda$ with the three common objects.

No assertion that such a family comes from a scheme is part of this Definition.

Definition 8.7.2 (Compatible arithmetic-realization predicate). Uses: def:weil-compatible-realization-family-candidate, def:weil-finite-extension-counting-zeta-series

Lean: Weil.Arithmetic.IsCompatibleRealizationFamily

For a candidate $\mathfrak{R}$, define Compatible($\mathfrak{R}$) to mean that, after transport along its stored identifications, for every λ and every $n \geq 1$, the trace datum of $\mathcal{R}_\lambda$ has the same point number $N_n = \#X(\mathbf{F}_{q^n})$, the same series $Z_{\mathcal{P}}$, and the same scalar-extension map $\mathbf{Q} \hookrightarrow E_\lambda$.

Definition 8.7.3 (Compatible arithmetic-realization family). Uses: def:weil-compatible-realization-family-candidate, def:weil-compatible-realization-predicate

Lean: Weil.Arithmetic.CompatibleRealizationFamily

A compatible arithmetic-realization family is a candidate together with a stored proof of Compatible($\mathfrak{R}$).

Definition 8.7.4 (Common reduced counting-zeta fraction). Uses: def:weil-compatible-realization-family, def:weil-counting-zeta-reduced-integral-fraction

Lean: Weil.Arithmetic.commonReducedCountingZetaFraction

For a compatible family, apply Definition 8.4.56 to the base realization λ_* and define

$$(A_{\mathfrak{R}}, B_{\mathfrak{R}}) := (A_{\mathcal{P},\lambda_*}, B_{\mathcal{P},\lambda_*}) \in \mathbf{Z}[T]^2.$$

Lemma 8.7.5 (The reduced fraction depends only on the point sequence). *Uses:* def:weil-compatible-realization-family, def:weil-family-common-reduced-fraction, def:weil-counting-zeta-reduced-integral-fraction

Lean: Weil.Arithmetic.reducedCountingZetaFraction_eq_common

For every $\lambda : \Lambda$,

$$A_{\mathcal{P},\lambda} = A_{\mathfrak{R}}, \qquad B_{\mathcal{P},\lambda} = B_{\mathfrak{R}}.$$

Proof. Uses: def:weil-compatible-realization-predicate, def:weil-family-common-reduced-fraction, def:weil-counting-zeta-integral-rational-datum, def:weil-reduced-integral-fraction

Transport the point sequence of $\mathcal{R}_\lambda$ along the equality stored in the compatibility proof. The definitions of the coefficient sequence, shift span, least stabilization index, integral shift lattice, characteristic polynomial, numerator, normalized gcd, and normalized quotients then have identical inputs. Congruence for each Definition gives identical outputs. Proof irrelevance removes the certificate fields. Applying the same calculation to λ_* gives the two displayed equalities. □

Definition 8.7.6 (Common parity polynomial). Uses: def:weil-family-common-reduced-fraction

Lean: Weil.Arithmetic.commonParityPolynomial

For $i \le B$, define

$$C_{\mathfrak{R},i} := \begin{cases} A_{\mathfrak{R}}, & i \text{ odd}, \\ B_{\mathfrak{R}}, & i \text{ even}. \end{cases}$$

Definition 8.7.7 (Universal weight-i stratum of a rational polynomial). Uses: def:weil-prime-power-data, def:weil-universal-scalar-weight

Lean: Weil.Arithmetic.polynomialUniversalWeightStratum

For $C \in \mathbf{Q}[T]$ with $C(0) = 1$, extend C to $\overline{\mathbf{Q}}[T]$. Define $\mathscr{W}_i(C)$ to be the submultiset of its ordinary root multiset consisting of those ρ for which $\rho \neq 0$ and ρ^{-1} is universally pure of weight i relative to q. Multiplicity is inherited from the root multiset of C.

Definition 8.7.8 (Complete normalized weight-stratum factor predicate). Uses: def:weil-polynomial-universal-weight-stratum

Lean: Weil.Arithmetic.IsCompleteWeightStratumFactor

For $P, C \in \mathbf{Q}[T]$, define $\mathrm{CompleteWeightFactor}_i(P, C)$ to mean

$$P(0) = 1 \quad \text{and} \quad \mathrm{roots}\big(\mathrm{map}_{\mathbf{Q},\overline{\mathbf{Q}}}\, P\big) = \mathscr{W}_i(C)$$

as multisets in $\overline{\mathbf{Q}}$.

Lemma 8.7.9 (A nonzero universally pure scalar has only one weight). Uses: def:weil-compatible-realization-family, def:weil-universal-scalar-weight

Lean: Weil.Arithmetic.universalScalarWeight_unique

Let $a \in \overline{\mathbf{Q}}^\times$. If a is universally pure of weights i and j relative to q, then $i = j$.

Proof. Uses: def:weil-compatible-realization-family-candidate, def:weil-universal-scalar-weight, lemma:weil-prime-power-gt-one

Restrict the stored map $\iota_{\overline{\mathbf{Q}}} : \overline{\mathbf{Q}} \hookrightarrow \mathbf{C}$ to $\mathbf{Q}(a)$. The two universal statements evaluated at this map give $q^{i/2} = q^{j/2}$. Since $q > 1$, the map $x \mapsto q^{x/2}$ is injective on $\mathbf{R}$; hence $i = j$. □

Lemma 8.7.10 (Every recovered rational factor is its complete weight stratum). Uses: def:weil-compatible-realization-family, def:weil-family-common-parity-polynomial, def:weil-complete-weight-stratum-factor-predicate, def:weil-rational-individual-factor

Lean: Weil.Arithmetic.rationalFactor_isCompleteWeightStratum

For every $\lambda : \Lambda$ and $i \le B$,

$$\mathrm{CompleteWeightFactor}_i\big(P^{(\lambda)}_{i,\mathbf{Q}}, C_{\mathfrak{R},i}\big).$$

Proof. Uses: lemma:weil-family-reduced-fraction-compatible, lemma:weil-integral-parity-polynomial-factorization, cor:weil-individual-factor-classical-weight, lemma:weil-universal-scalar-weight-unique, lemma:weil-rational-individual-factor-map, lemma:weil-polynomial-galois-roots

The factorization of the parity polynomial for $\mathcal{R}_\lambda$, transported by the reduced-fraction compatibility lemma, writes $C_{\mathfrak{R},i}$ as the product of the rational factors $P^{(\lambda)}_{j,\mathbf{Q}}$ with $j \equiv i \pmod 2$. The root multiset of a finite product is the multiset sum of the factor root multisets. Every reciprocal root belonging to the j-th summand is universally pure of weight j. If it also has weight i, the weight-uniqueness lemma gives $j = i$. Conversely every reciprocal root of the i-th factor has weight i. Thus the root multiset of the i-th factor is exactly $\mathscr{W}_i(C_{\mathfrak{R},i})$, including multiplicity. Its constant coefficient is 1 by the rational-factor map lemma and the determinant constant-coefficient lemma. □

Lemma 8.7.11 (A complete normalized weight-stratum factor is unique). Uses: def:weil-complete-weight-stratum-factor-predicate

Lean: Weil.Arithmetic.completeWeightStratumFactor_unique

If $P, Q \in \mathbf{Q}[T]$ both satisfy $\mathrm{CompleteWeightFactor}_i(-, C)$, then $P = Q$.

Proof. Uses: def:weil-complete-weight-stratum-factor-predicate

After extension to $\overline{\mathbf{Q}}$, the two polynomials have the same root multiset. Neither polynomial is zero because its value at zero is 1. Factorization over the algebraically closed field shows that one is a nonzero scalar multiple of the other. Evaluating at zero makes that scalar equal to 1. Injectivity of $\mathbf{Q}[T] \to \overline{\mathbf{Q}}[T]$ reflects the equality. □

Definition 8.7.12 (Common rational Frobenius factor)**.** Uses: def:weil-compatible-realization-family,def:weil-rational-individual-factor

Lean: Weil.Arithmetic.commonRationalFrobeniusFactor

For $i \le B$, define

$$P_{i,\mathbf{Q}}^{\mathrm{com}} := P_{i,\mathbf{Q}}^{(\lambda_*)} \in \mathbf{Q}[T].$$

Definition 8.7.13 (Common integral Frobenius factor)**.** Uses: def:weil-compatible-realization-family,def:weil-integral-individual-factor

Lean: Weil.Arithmetic.commonIntegralFrobeniusFactor

For $i \le B$, define

$$P_{i,\mathbf{Z}}^{\mathrm{com}} := P_{i,\mathbf{Z}}^{(\lambda_*)} \in \mathbf{Z}[T].$$

Lemma 8.7.14 (Rational factors are realization-independent)**.** *Uses:* def:weil-compatible-realization-family,def:weil-family-common-rational-factor,def:weil-rational-individual-factor

Lean: Weil.Arithmetic.rationalFactor_eq_common

For every $\lambda : \Lambda$ and $i \le B$,

$$P_{i,\mathbf{Q}}^{(\lambda)} = P_{i,\mathbf{Q}}^{\mathrm{com}}.$$

Proof. Uses: lemma:weil-family-rational-factor-complete-weight-stratum,lemma:weil-complete-weight-stratum-factor-unique

The two polynomials satisfy the complete normalized weight-i stratum predicate for the same common parity polynomial. Apply the uniqueness lemma once. □

Lemma 8.7.15 (Integral factors are realization-independent)**.** *Uses:* def:weil-compatible-realization-family,def:weil-family-common-integral-factor,def:weil-integral-individual-factor

Lean: Weil.Arithmetic.integralFactor_eq_common

For every $\lambda : \Lambda$ and $i \le B$,

$$P_{i,\mathbf{Z}}^{(\lambda)} = P_{i,\mathbf{Z}}^{\mathrm{com}}.$$

Proof. Uses: lemma:weil-rational-factor-realization-independent,thm:weil-individual-factor-integral

The two integral polynomials have the same image in $\mathbf{Q}[T]$: both images are the common rational factor. The coefficient map $\mathbf{Z}[T] \to \mathbf{Q}[T]$ is injective, so the polynomials are equal. □

Theorem 8.7.16 (Integral Frobenius factors are realization-independent)**.** *Uses:* def:weil-compatible-realization-family,def:weil-family-common-rational-factor,def:weil-family-common-integral-factor

Lean: Weil.Arithmetic.integralFrobeniusFactor_independent

For every $i \le B$, the common polynomial $P_{i,\mathbf{Z}}^{\mathrm{com}}$ has constant coefficient 1; for every λ, its images in $\mathbf{Q}[T]$ and $E_\lambda[T]$ are $P_{i,\mathbf{Q}}^{(\lambda)}$ and $P_{G_\lambda,i}$, respectively. Thus the individual integral factor, not merely the alternating zeta quotient, is independent of ℓ_λ and of the realization.

Proof. Uses: lemma:weil-integral-factor-realization-independent,lemma:weil-rational-factor-realization-independent,thm:weil-individual-factor-integral

Use integral-factor independence to replace the factor of $\mathcal{R}_\lambda$ by the common factor. The three asserted properties then are the three conclusions of individual-factor integrality for $\mathcal{R}_\lambda$. □

8.8 Degrees and realization-independent Betti numbers

Definition 8.8.1 (Realization-independent cohomological degree)**.** Uses: `def:weil-family-common-integral-factor`

Lean: `Weil.Arithmetic.commonBettiNumber`

For $i \le B$, define

$$b_i^{\mathrm{com}} := \mathrm{natDegree}\big(P_{i,\mathbf{Z}}^{\mathrm{com}}\big) \in \mathbf{N}.$$

Lemma 8.8.2 (Degree survives the two coefficient extensions)**.** *Uses:* *def:weil-compatible-realization-family, def:weil-family-common-integral-factor, def:weil-realization-independent-betti-number*

Lean: *Weil.Arithmetic.commonFactor_natDegree_map*

For every λ and $i \le B$,

$$\mathrm{natDegree}\, P_{i,\mathbf{Z}}^{\mathrm{com}} = \mathrm{natDegree}\, P_{i,\mathbf{Q}}^{(\lambda)} = \mathrm{natDegree}\, P_{G_\lambda,i}.$$

Proof. Uses: `thm:weil-integral-factor-realization-independence`

The common integral factor has constant coefficient 1, hence is nonzero. For an injective coefficient homomorphism, the leading coefficient of a nonzero polynomial remains nonzero and all coefficients above its degree remain zero. Therefore `Polynomial.natDegree_map_eq_of_injective` applies first to $\mathbf{Z} \hookrightarrow \mathbf{Q}$ and then to $\mathbf{Q} \hookrightarrow E_\lambda$. The factor-independence theorem identifies the two images. □

Theorem 8.8.3 (Cohomological dimensions are realization-independent)**.** *Uses:* *def:weil-compatible-realization-family, def:weil-ell-adic-betti-number, def:weil-realization-independent-betti-number*

Lean: *Weil.Arithmetic.ellAdicBettiNumber_independent*

For every $\lambda : \Lambda$ and every $i \le B$,

$$\dim_{E_\lambda} V_{\lambda,i} = b_{i,\ell_\lambda} = b_i^{\mathrm{com}}.$$

In particular the cohomological dimension is independent of the auxiliary prime and the coefficient realization.

Proof. Uses: `lemma:weil-common-factor-degree-after-extension,prop:weil-reciprocal-factor-degree-equals-dimension`

The degree proposition for $\mathcal{R}_\lambda$ identifies $\dim_{E_\lambda} V_{\lambda,i}$ with $\mathrm{natDegree}\, P_{G_\lambda,i}$. The coefficient-extension degree lemma identifies the latter with the Definition of b_i^{com}. This is a transitive equality. □

Formalization note. The implication

$$\text{common point counts} + \text{universal purity} \Longrightarrow P_{i,\mathbf{Z}} \text{ and } b_i \text{ independent}$$

is Deligne's arithmetic recovery argument as expanded in the supplied notes, Chapter 14, printed pp. 75–77/physical PDF pp. 85–87. Weil II, Corollary (3.3.9), article p. 207/PDF p. 72, records the conclusion but sends the arithmetic implication to Weil I, (1.7)⇒(1.6); neither citation is used as an atomic proof leaf here.

8.9 Recovery and independence of functional-equation constants

Definition 8.9.1 (Compatible duality-realization family)**.** Uses: `def:weil-compatible-realization-family,def:weil-duality-graded-similitude-system`

Lean: `Weil.Arithmetic.CompatibleDualityRealizationFamily`

A compatible duality-realization family consists of a compatible arithmetic-realization family, a natural number d, a stored equality $B = 2d$, and, for every $\lambda : \Lambda$, a Definition 6.4.1 $\mathcal{S}_\lambda$ of dimension d. It also stores degreewise identifications of the vector spaces and geometric-Frobenius automorphisms in $\mathcal{S}_\lambda$ with those in the recovery input $\mathcal{R}_\lambda$. Under these identifications its $b_{\lambda,i}$, $D_{\lambda,i}$, and $P_{\lambda,i}$ are, respectively,

$$\dim_{E_\lambda} V_{\lambda,i}, \qquad \det(F_{\lambda,i}), \qquad \det(1 - TF_{\lambda,i}).$$

Definition 8.9.2 (Common Frobenius determinant). Uses: def:weil-compatible-duality-realization-family,def:weil-family-common-integral-factor,def:weil-realization-independent-betti-number

Lean: Weil.Arithmetic.commonFrobeniusDeterminant

For $0 \le i \le 2d$, define

$$D_i^{\mathrm{com}} := (-1)^{b_i^{\mathrm{com}}} \operatorname{leadingCoeff}\left(P_{i,\mathbf{Z}}^{\mathrm{com}}\right) \in \mathbf{Z} \subset \mathbf{Q}.$$

Lemma 8.9.3 (The common determinant is nonzero). Uses: def:weil-compatible-duality-realization-family, def:weil-family-common-frobenius-determinant

Lean: Weil.Arithmetic.commonFrobeniusDeterminant_ne_zero

For every $i \le 2d$, $D_i^{\mathrm{com}} \neq 0$ in $\mathbf{Q}$.

Proof. Uses: def:weil-family-common-frobenius-determinant,lemma:weil-common-factor-degree-after-extension,lemma:weil-based-automorphism-determinant-nonzero,lemma:weil-reciprocal-determinant-leading-coefficient,thm:weil-betti-number-realization-independence,thm:weil-integral-factor-realization-independence

Map the displayed integer to the coefficient field of the base realization. The common-factor theorem identifies the polynomial image with $P_{G_{\lambda_*},i}$, and Betti independence identifies its degree with b_i^{com}. The reciprocal-determinant leading-coefficient formula changes the image into

$$(-1)^{b_i^{\mathrm{com}}}(-1)^{b_i^{\mathrm{com}}} \det(F_{\lambda_*,i}) = \det(F_{\lambda_*,i}).$$

This determinant is nonzero because $F_{\lambda_*,i}$ is an automorphism. An injective field map reflects nonvanishing. □

Lemma 8.9.4 (Leading coefficients recover every realization determinant). Uses: def:weil-compatible-duality-realization-family, def:weil-family-common-frobenius-determinant, def:weil-duality-graded-frobenius-determinants

Lean: Weil.Arithmetic.commonFrobeniusDeterminant_map

For every λ and $i \le 2d$, the coefficient map $\mathbf{Q} \hookrightarrow E_\lambda$ sends

$$D_i^{\mathrm{com}} \longmapsto D_{\lambda,i} = \det(F_{\lambda,i}).$$

Proof. Uses: def:weil-family-common-frobenius-determinant,lemma:weil-reciprocal-determinant-leading-coefficient,thm:weil-betti-number-realization-independence,thm:weil-integral-factor-realization-independence

The integral-factor theorem identifies the image of $P_{i,\mathbf{Z}}^{\mathrm{com}}$ with $\det(1 - TF_{\lambda,i})$. Injectivity of the coefficient map preserves degree and therefore sends leading coefficient to leading coefficient. Betti independence replaces b_i^{com} by $\dim V_{\lambda,i}$. The reciprocal-determinant leading-coefficient formula then gives

$$(-1)^{b_{\lambda,i}} \operatorname{leadingCoeff} \det(1 - TF_{\lambda,i}) = (-1)^{2b_{\lambda,i}} \det(F_{\lambda,i}) = D_{\lambda,i}.$$

□

Definition 8.9.5 (Common Euler characteristic). Uses: def:weil-compatible-duality-realization-family,def:weil-realization-independent-betti-number

Lean: Weil.Arithmetic.commonEulerCharacteristic

Define

$$\chi^{\mathrm{com}} := \sum_{i=0}^{2d} \begin{cases} b_i^{\mathrm{com}}, & i \text{ even}, \\ -b_i^{\mathrm{com}}, & i \text{ odd} \end{cases} \in \mathbf{Z}.$$

Lemma 8.9.6 (The common Euler characteristic equals every realization Euler characteristic). Uses: def:weil-compatible-duality-realization-family, def:weil-family-common-euler-characteristic, def:weil-duality-graded-euler-characteristic

Lean: Weil.Arithmetic.commonEulerCharacteristic_eq

For every λ,

$$\chi^{\mathrm{com}} = \chi_\lambda.$$

Proof. Uses: def:weil-family-common-euler-characteristic,def:weil-duality-graded-euler-characteristic,thm:weil-betti-number-realization-independence

Expand both finite sums. In each summand, Betti independence replaces b_i^{com} by $b_{\lambda,i}$; the parity sign is unchanged. Finite-sum congruence proves the equality. □

Definition 8.9.7 (Common alternating Frobenius determinant). Uses: `def:weil-compatible-duality-realization-family`, `def:weil-family-common-frobenius-determinant`

Lean: `Weil.Arithmetic.commonAlternatingFrobeniusDeterminant`

Define

$$\Delta^{\mathrm{com}} := \prod_{i=0}^{2d} \begin{cases} D_i^{\mathrm{com}}, & i \text{ even}, \\ (D_i^{\mathrm{com}})^{-1}, & i \text{ odd} \end{cases} \in \mathbf{Q}.$$

This is the explicit meaning of $\prod_i (D_i^{\mathrm{com}})^{(-1)^i}$.

Lemma 8.9.8 (The common alternating determinant is nonzero). Uses: *def:weil-family-common-alternating-determinant*

Lean: *Weil.Arithmetic.commonAlternatingFrobeniusDeterminant_ne_zero*

One has $\Delta^{\mathrm{com}} \neq 0$.

Proof. Uses: `def:weil-family-common-alternating-determinant`, `lemma:weil-family-common-frobenius-determinant-nonzero`

Every factor in the finite product is nonzero, and the inverse of a nonzero field element is nonzero. Apply `Finset.prod_ne_zero_iff.mpr` to the pointwise nonvanishing proof. □

Lemma 8.9.9 (The common alternating determinant realizes every duality determinant). Uses: *def:weil-compatible-duality-realization-family*, *def:weil-family-common-alternating-determinant*, *def:weil-duality-alternating-frobenius-determinant*

Lean: *Weil.Arithmetic.commonAlternatingFrobeniusDeterminant_map*

For every λ, *the map* $\mathbf{Q} \hookrightarrow E_\lambda$ *sends*

$$\Delta^{\mathrm{com}} \longmapsto \Delta_\lambda.$$

Proof. Uses: `def:weil-family-common-alternating-determinant`, `def:weil-duality-alternating-frobenius-determinant`, `lemma:weil-family-common-frobenius-determinant-nonzero`, `lemma:weil-family-common-determinant-maps-to-realization`

A field homomorphism preserves finite products. In even degree use the determinant-map lemma directly. In odd degree, nonvanishing permits $\mathrm{map}(D_i^{-1}) = \mathrm{map}(D_i)^{-1}$. Finite-product congruence yields precisely the defining product for Δ_λ. □

Definition 8.9.10 (Common half functional-equation exponent). Uses: `def:weil-compatible-duality-realization-family`, `def:weil-family-common-euler-characteristic`, `def:weil-duality-half-functional-equation-exponent`

Lean: `Weil.Arithmetic.commonHalfFunctionalEquationExponent`

Define

$$h^{\mathrm{com}} := \left(d\,\chi^{\mathrm{com}}\right) \operatorname{ediv} 2 \in \mathbf{Z},$$

using the same integer-division operation as Definition 6.4.16.

Lemma 8.9.11 (The common half exponent realizes the duality half exponent). Uses: *def:weil-compatible-duality-realization-family*, *def:weil-family-common-half-functional-equation-exponent*, *def:weil-duality-half-functional-equation-exponent*

Lean: *Weil.Arithmetic.commonHalfFunctionalEquationExponent_eq*

For every λ,

$$h^{\mathrm{com}} = h_\lambda \qquad \textit{and} \qquad 2h^{\mathrm{com}} = d\chi^{\mathrm{com}}.$$

Proof. Uses: `def:weil-family-common-half-functional-equation-exponent`, `def:weil-duality-half-functional-equation-exponent`, `lemma:weil-family-common-euler-characteristic-realization`, `lemma:weil-duality-twice-half-functional-equation-exponent`

The Euler-characteristic lemma makes the inputs to the two identical integer-division Definitions equal, proving the first equality by congruence. Rewrite the module04 twice-half identity with that equality and with $\chi_\lambda = \chi^{\mathrm{com}}$; this gives the second equality. □

Definition 8.9.12 (Realization-independent functional-equation sign). Uses: `def:weil-compatible-duality-realization-family`, `def:weil-family-common-euler-characteristic`, `def:weil-family-common-alternating-determinant`, `def:weil-family-common-half-functional-equation-exponent`, `def:weil-duality-functional-equation-sign-candidate`

Lean: `Weil.Arithmetic.commonFunctionalEquationSign`

Define

$$\varepsilon^{\mathrm{com}} := (-1)^{\chi^{\mathrm{com}}} q^{-h^{\mathrm{com}}} \Delta^{\mathrm{com}} \in \mathbf{Q}.$$

The Frobenius is geometric, so the power and determinant agree with the normalization in Definition 6.4.19.

Lemma 8.9.13 (The common sign agrees with the duality sign candidate). *Uses:* `def:weil-compatible-duality-realization-family`, `def:weil-family-common-functional-equation-sign`, `def:weil-duality-functional-equation-sign-candidate`

Lean: `Weil.Arithmetic.commonFunctionalEquationSign_map`

For every λ, the coefficient embedding $\mathbf{Q} \hookrightarrow E_\lambda$ sends

$$\varepsilon^{\mathrm{com}} \longmapsto \varepsilon_\lambda,$$

where the right side is exactly Definition 6.4.19 for $\mathcal{S}_\lambda$.

Proof. Uses: `def:weil-family-common-functional-equation-sign`, `def:weil-duality-functional-equation-sign-candidate`, `lemma:weil-family-common-euler-characteristic-realization`, `lemma:weil-family-common-alternating-determinant-realization`, `lemma:weil-family-common-half-exponent-realization`

Map the three factors in the Definition of $\varepsilon^{\mathrm{com}}$. The Euler-characteristic and half-exponent lemmas identify the two integer exponents, while the alternating-determinant lemma identifies the last factor. The image is $(-1)^{\chi_\lambda} q^{-h_\lambda} \Delta_\lambda$, which is the duality sign candidate by Definition. □

Lemma 8.9.14 (The common functional-equation sign squares to one). *Uses:* `def:weil-compatible-duality-realization-family`, `def:weil-family-common-functional-equation-sign`

Lean: `Weil.Arithmetic.commonFunctionalEquationSign_sq`

One has

$$(\varepsilon^{\mathrm{com}})^2 = 1 \quad \text{in } \mathbf{Q}.$$

Proof. Uses: `lemma:weil-family-common-sign-agrees-with-duality-candidate`, `lemma:weil-duality-functional-equation-sign-square`

Map the target equality to the coefficient field of the base realization. The sign-agreement lemma changes the left side to $\varepsilon_{\lambda_*}^2$, which equals 1 by the module04 sign-square lemma. Injectivity of $\mathbf{Q} \hookrightarrow E_{\lambda_*}$ reflects the equality. □

Lemma 8.9.15 (The common sign belongs to the two-element sign set). *Uses:* `def:weil-compatible-duality-realization-family`, `def:weil-family-common-functional-equation-sign`

Lean: `Weil.Arithmetic.commonFunctionalEquationSign_eq_one_or_neg_one`

One has

$$\varepsilon^{\mathrm{com}} = 1 \quad \text{or} \quad \varepsilon^{\mathrm{com}} = -1.$$

Proof. Uses: `lemma:weil-family-common-functional-equation-sign-square`

Factor the proved equality as

$$0 = (\varepsilon^{\mathrm{com}})^2 - 1 = (\varepsilon^{\mathrm{com}} - 1)(\varepsilon^{\mathrm{com}} + 1).$$

The field $\mathbf{Q}$ has no zero divisors, so one of the two factors vanishes. The first case gives $\varepsilon^{\mathrm{com}} = 1$; the second gives $\varepsilon^{\mathrm{com}} = -1$. □

Theorem 8.9.16 (Functional-equation constants are realization-independent). *Uses:* `def:weil-compatible-duality-realization-family`, `def:weil-family-common-frobenius-determinant`, `def:weil-family-common-alternating-determinant`, `def:weil-family-common-functional-equation-sign`, `def:weil-duality-graded-frobenius-determinants`, `def:weil-duality-alternating-frobenius-determinant`, `def:weil-duality-functional-equation-sign-candidate`

Lean: `Weil.Arithmetic.functionalEquationConstants_independent`

For every realization λ, the images of D_i^{com}, Δ^{com}, and $\varepsilon^{\mathrm{com}}$ are respectively $D_{\lambda,i}$, Δ_λ, and ε_λ. Moreover

$$\varepsilon^{\mathrm{com}} \in \{1, -1\}.$$

Thus every functional-equation constant is recovered from the common integral factors and is independent of ℓ_λ.

Proof. Uses: `lemma:weil-family-common-determinant-maps-to-realization`, `lemma:weil-family-common-alternating-determinant-realization`, `lemma:weil-family-common-sign-agrees-with-duality-candidate`, `lemma:weil-family-common-functional-equation-sign-dichotomy`

The first three assertions are exactly the three realization lemmas. The dichotomy lemma is the asserted membership in $\{1, -1\}$. □

> **Status.** The endpoint identities $P_0(T) = 1 - T$ and $P_{2d}(T) = 1 - q^d T$ are intentionally assigned to the final assembly module. Their proof requires the geometric-connectedness computation of H^0, the geometric-Frobenius action on constants, and the top-degree duality identification. None of those endpoint inputs is part of arithmetic recovery, and no endpoint identity is used above.

8.10 Conditional comparison with singular Betti numbers

Definition 8.10.1 (Mixed-characteristic trait datum). Uses: `def:weil-prime-power-data`

Lean: `Weil.Arithmetic.MixedCharacteristicTraitData`

A mixed-characteristic trait datum consists of a discrete valuation ring R, its fraction field K of characteristic 0, an identification of its residue field with $\mathbf{F}_q$, geometric points $\bar{s} \to \operatorname{Spec} R$ and $\bar{\eta} \to \operatorname{Spec} R$, a geometric specialization $\bar{\eta} \rightsquigarrow \bar{s}$, and a field embedding $\kappa : K \hookrightarrow \mathbf{C}$. All maps and identifications are stored data.

Definition 8.10.2 (Smooth proper lifting candidate). Uses: `def:weil-mixed-characteristic-trait-datum`

Lean: `Weil.Arithmetic.SmoothProperLiftingCandidate`

For $X_0/\mathbf{F}_q$, a lifting candidate consists of a finite-type R-scheme $\mathscr{X}$, a morphism $f : \mathscr{X} \to \operatorname{Spec} R$, an isomorphism from its special fiber to X_0, and the generic and complex fibers

$$X_{\bar{\eta}} := \mathscr{X} \times_R \bar{K}, \qquad X_{\mathbf{C}} := \mathscr{X} \times_{R,\kappa} \mathbf{C}.$$

Definition 8.10.3 (Smooth proper lifting predicate). Uses: `def:weil-smooth-proper-lifting-candidate`

Lean: `Weil.Arithmetic.IsSmoothProperLifting`

Define IsSmoothProperLift$(\mathscr{X}, f)$ to mean that f is smooth and proper, the stored special-fiber arrow and the displayed fiber squares are isomorphisms, and all structural maps in those squares commute.

Definition 8.10.4 (Smooth proper lifting datum). Uses: `def:weil-smooth-proper-lifting-candidate,def:weil-smooth-proper-lifting-predicate`

Lean: `Weil.Arithmetic.SmoothProperLiftingData`

A smooth proper lifting datum is a lifting candidate together with a stored proof of its smooth proper lifting predicate.

Definition 8.10.5 (Finite-coefficient specialization system). Uses: `def:weil-smooth-proper-lifting-datum`

Lean: `Weil.Arithmetic.FiniteCoefficientSpecializationSystem`

Fix $\ell \neq p$ and $i \geq 0$. For every $r \geq 1$, set $\Lambda_r = \mathbf{Z}/\ell^r\mathbf{Z}$ and record the typed cohomology groups

$$H^i_{s,r} := H^i_{\text{et}}(X_{\bar{s}}, \Lambda_r), \qquad H^i_{\eta,r} := H^i_{\text{et}}(X_{\bar{\eta}}, \Lambda_r),$$

together with their coefficient-reduction maps from level $r+1$ to level r.

Definition 8.10.6 (Finite specialization map). Uses: `def:weil-finite-coefficient-specialization-system`

Lean: `Weil.Arithmetic.finiteSpecializationMap`

For every $r \geq 1$, define

$$\mathrm{sp}_{i,r} : H^i_{s,r} \longrightarrow H^i_{\eta,r}$$

to be the stalk map of $R^i f_* \Lambda_r$ induced by the stored geometric specialization $\bar{\eta} \rightsquigarrow \bar{s}$, transported through the proper-base-change stalk identifications

$$(R^i f_* \Lambda_r)_{\bar{s}} \cong H^i_{s,r}, \qquad (R^i f_* \Lambda_r)_{\bar{\eta}} \cong H^i_{\eta,r}.$$

This Definition is conditional on those typed stalk identifications.

Definition 8.10.7 (Finite specialization certificate). Uses: `def:weil-finite-specialization-map,def:weil-finite-coefficient-specialization-system`

Lean: `Weil.Arithmetic.IsFiniteSpecializationSystem`

A finite specialization certificate consists of proofs that:

1. every $\mathrm{sp}_{i,r}$ is Λ_r-linear and bijective;
2. for every r, specialization commutes with reduction $\Lambda_{r+1} \to \Lambda_r$;
3. the levelwise inverse maps commute with the same reductions;
4. identity specializations give identity maps, and composition of geometric specializations gives composition of the corresponding maps.

Definition 8.10.8 (Integral adic specialization map)**.** Uses: `def:weil-finite-specialization-map`,`def:weil-finite-specialization-certificate`

Lean: `Weil.Arithmetic.integralAdicSpecializationMap`

Define

$$\mathrm{sp}_{i,\mathbf{Z}_\ell} : \varprojlim_r H^i_{s,r} \longrightarrow \varprojlim_r H^i_{\eta,r}$$

coordinatewise by

$$(x_r)_r \longmapsto \big(\mathrm{sp}_{i,r}(x_r)\big)_r.$$

The transition compatibility stored in the certificate supplies membership of the displayed family in the target inverse limit.

Definition 8.10.9 (Inverse integral adic specialization candidate)**.** Uses: `def:weil-finite-specialization-certificate`

Lean: `Weil.Arithmetic.integralAdicSpecializationInverse`

Define

$$\mathrm{sp}^{-1}_{i,\mathbf{Z}_\ell} : \varprojlim_r H^i_{\eta,r} \longrightarrow \varprojlim_r H^i_{s,r}$$

coordinatewise by the inverses $\mathrm{sp}^{-1}_{i,r}$. Compatibility of these inverses is the third field of the finite specialization certificate.

Lemma 8.10.10 (Integral adic specialization is well-defined)**.** *Uses:* *def:weil-integral-adic-specialization-map*

Lean: *Weil.Arithmetic.integralAdicSpecializationMap_mem_limit*

The coordinate formula for $\mathrm{sp}_{i,\mathbf{Z}_\ell}$ *preserves every inverse-limit compatibility equation.*

Proof. Uses: `def:weil-finite-specialization-certificate`,`def:weil-integral-adic-specialization-map`

For an adjacent pair $r+1 \to r$, expand the target compatibility equation. It becomes the specialization–reduction commutative square stored in the certificate, applied to the source compatibility equation. Function extensionality over r proves membership in the inverse limit. □

Lemma 8.10.11 (The two integral adic specialization maps are inverse)**.** *Uses:* *def:weil-integral-adic-specialization-map*, *def:weil-integral-adic-specialization-inverse*

Lean: *Weil.Arithmetic.integralAdicSpecializationMap_inverse*

The composites in both orders are the corresponding identity maps.

Proof. Uses: `def:weil-integral-adic-specialization-map`,`def:weil-integral-adic-specialization-inverse`,`def:weil-finite-specialization-certificate`

Evaluate either composite at level r. The coordinate is $\mathrm{sp}^{-1}_{i,r}\,\mathrm{sp}_{i,r}$ or $\mathrm{sp}_{i,r}\,\mathrm{sp}^{-1}_{i,r}$, hence the original coordinate. Extensionality for inverse-limit elements proves both identities. □

Definition 8.10.12 (Rational adic specialization equivalence)**.** Uses: `def:weil-integral-adic-specialization-map`,`def:weil-integral-adic-specialization-inverse`,`def:weil-raw-rationalized-cohomology`

Lean: `Weil.Arithmetic.rationalAdicSpecializationEquiv`

Tensor the two integral maps with $\mathbf{Q}_\ell$ over $\mathbf{Z}_\ell$. Define

$$\mathrm{sp}_{i,\mathbf{Q}_\ell} : H^i_{\mathrm{et}}(X_{\bar{s}}, \mathbf{Q}_\ell) \xrightarrow{\sim} H^i_{\mathrm{et}}(X_{\bar{\eta}}, \mathbf{Q}_\ell)$$

to be the resulting pair of mutually inverse linear maps, transported through the rational-cohomology comparison data.

Lemma 8.10.13 (Rational adic specialization preserves dimension)**.** *Uses:* *def:weil-rational-adic-specialization-equivalence*

Lean: *Weil.Arithmetic.rationalAdicSpecialization_finrank*

If either side is finite-dimensional, then both sides are and

$$\dim_{\mathbf{Q}_\ell} H^i_{\mathrm{et}}(X_{\bar{s}}, \mathbf{Q}_\ell) = \dim_{\mathbf{Q}_\ell} H^i_{\mathrm{et}}(X_{\bar{\eta}}, \mathbf{Q}_\ell).$$

Proof. Uses: `lemma:weil-integral-adic-specialization-inverse-identities`,`def:weil-rational-adic-specialization-equivalence`

Tensoring preserves the two inverse identities. Thus the rational map is a linear equivalence. Apply mathlib's `LinearEquiv.finrank_eq` in the direction matching the displayed equality. □

Definition 8.10.14 (Geometric generic-to-complex comparison map)**.** Uses: `def:weil-smooth-proper-lifting-datum`

Lean: `Weil.Arithmetic.genericComplexEtaleComparisonMap`

For an algebraically closed overfield $\Omega \subset \mathbf{C}$ containing the image of $\bar{K}$, define the pullback map

$$c_{i,r}^{\mathrm{geom}} : H^i_{\mathrm{et}}(X_{\bar{\eta}}, \Lambda_r) \longrightarrow H^i_{\mathrm{et}}(X_{\mathbf{C}}, \Lambda_r)$$

from the base-change morphism $X_{\mathbf{C}} \to X_{\bar{\eta}}$.

Definition 8.10.15 (Geometric field-extension comparison certificate)**.** Uses: `def:weil-generic-complex-etale-comparison-map`

Lean: `Weil.Arithmetic.IsGenericComplexEtaleComparison`

The certificate asserts that every $c_{i,r}^{\mathrm{geom}}$ is an isomorphism, that it commutes with coefficient reduction, and that the induced inverse maps commute with reduction. It also records identity and composition compatibility for towers of algebraically closed overfields.

Definition 8.10.16 (Finite Artin comparison map)**.** Uses: `def:weil-smooth-proper-lifting-datum`

Lean: `Weil.Arithmetic.finiteArtinComparisonMap`

For every $r \geq 1$, define the comparison morphism

$$a_{i,r} : H^i_{\mathrm{sing}}\big(X_{\mathbf{C}}(\mathbf{C}), \Lambda_r\big) \longrightarrow H^i_{\mathrm{et}}(X_{\mathbf{C}}, \Lambda_r)$$

from the canonical morphism of sites from the analytic topology to the etale topology.

Definition 8.10.17 (Finite Artin comparison certificate)**.** Uses: `def:weil-finite-artin-comparison-map`

Lean: `Weil.Arithmetic.IsFiniteArtinComparisonSystem`

The certificate consists of proofs that each $a_{i,r}$ is an isomorphism, that the comparison square for $\Lambda_{r+1} \to \Lambda_r$ commutes, and that the inverse comparison maps also commute with coefficient reduction. It additionally records functoriality in morphisms of complex algebraic varieties.

Definition 8.10.18 (Adic Artin comparison equivalence)**.** Uses: `def:weil-finite-artin-comparison-map,def:weil-finite-artin-comparison-certificate`

Lean: `Weil.Arithmetic.adicArtinComparisonEquiv`

Apply the finite comparison maps and their inverses coordinatewise on the two inverse limits, then tensor with $\mathbf{Q}_\ell$. Define the resulting linear equivalence

$$a_{i,\mathbf{Q}_\ell} : H^i_{\mathrm{sing}}\big(X_{\mathbf{C}}(\mathbf{C}), \mathbf{Q}_\ell\big) \xrightarrow{\sim} H^i_{\mathrm{et}}(X_{\mathbf{C}}, \mathbf{Q}_\ell).$$

Lemma 8.10.19 (The adic Artin maps are inverse)**.** *Uses:* `def:weil-adic-artin-comparison-equivalence`

Lean: `Weil.Arithmetic.adicArtinComparison_inverse`

The two maps stored in $a_{i,\mathbf{Q}_\ell}$ compose to the identities.

Proof. Uses: `def:weil-finite-artin-comparison-certificate,def:weil-adic-artin-comparison-equivalence`

At level r, each composite is the identity by the bijectivity field of the finite certificate. Inverse-limit extensionality gives the integral identities, and tensoring with $\mathbf{Q}_\ell$ preserves composition and identity. These are the two required linear-equivalence fields. □

Definition 8.10.20 (Singular rational coefficient-extension map)**.** Uses: `def:weil-smooth-proper-lifting-datum`

Lean: `Weil.Arithmetic.singularRationalCoefficientExtensionMap`

Define the canonical map

$$u_i : \mathbf{Q}_\ell \otimes_{\mathbf{Q}} H^i_{\mathrm{sing}}\big(X_{\mathbf{C}}(\mathbf{C}), \mathbf{Q}\big) \longrightarrow H^i_{\mathrm{sing}}\big(X_{\mathbf{C}}(\mathbf{C}), \mathbf{Q}_\ell\big)$$

by $a \otimes [c] \mapsto [a\,c]$ on cochain classes.

Definition 8.10.21 (Singular coefficient-extension certificate)**.** Uses: `def:weil-singular-rational-coefficient-extension-map`

Lean: `Weil.Arithmetic.IsSingularRationalCoefficientExtension`

The certificate asserts that the singular cochain complex is represented by a degreewise finite free complex over $\mathbf{Z}$, that u_i is well-defined on cohomology, and that u_i is a $\mathbf{Q}_\ell$-linear isomorphism. It records compatibility with the usual coboundary and with extension of coefficient fields.

Definition 8.10.22 (Lift-comparison interface). Uses: `def:weil-compatible-realization-family,def:weil-smooth-proper-lifting-datum,def:weil-finite-specialization-certificate,def:weil-generic-complex-etale-comparison-certificate,def:weil-finite-artin-comparison-certificate,def:weil-singular-coefficient-extension-certificate`

Lean: `Weil.Arithmetic.LiftComparisonInterface`

For an index $\lambda : \Lambda$, a lift-comparison interface consists of the four displayed certificates and identifications of the special-fiber $\mathbf{Q}_{\ell_\lambda}$-cohomology with the vector spaces $V_{\lambda,i}$ in $\mathcal{R}_\lambda$. It stores all comparison maps; it does not assert that the certificates have been constructed from the current mathlib library.

Definition 8.10.23 (Special-fiber to singular comparison equivalence). Uses: `def:weil-lift-comparison-interface,def:weil-rational-adic-specialization-equivalence,def:weil-generic-complex-etale-comparison-map,def:weil-adic-artin-comparison-equivalence,def:weil-singular-rational-coefficient-extension-map`

Lean: `Weil.Arithmetic.liftBettiComparisonEquiv`

Define $\Theta_{i,\lambda}$ as the composite of:

1. the stored identification $V_{\lambda,i} \simeq H^i_{\text{ét}}(X_{\bar{s}}, \mathbf{Q}_{\ell_\lambda})$;
2. rational adic specialization to $X_{\bar{\eta}}$;
3. geometric algebraically-closed field extension to $X_{\mathbf{C}}$;
4. the inverse adic Artin comparison;
5. the inverse singular coefficient-extension map.

Thus

$$\Theta_{i,\lambda} : V_{\lambda,i} \xrightarrow{\sim} \mathbf{Q}_{\ell_\lambda} \otimes_{\mathbf{Q}} H^i_{\text{sing}}(X_{\mathbf{C}}(\mathbf{C}), \mathbf{Q}).$$

Lemma 8.10.24 (The lift-comparison composite has the displayed source and target). *Uses: def:weil-lift-betti-comparison-equivalence*

Lean: Weil.Arithmetic.liftBettiComparisonEquiv_typed

The source of $\Theta_{i,\lambda}$ is $V_{\lambda,i}$, its target is the displayed scalar extension of singular cohomology, and both composites with its stored inverse are identities.

Proof. Uses: `def:weil-lift-comparison-interface,def:weil-lift-betti-comparison-equivalence,lemma:weil-integral-adic-specialization-inverse-identities,lemma:weil-adic-artin-comparison-inverse-identities`

The target of each numbered arrow is definitionally the source of the next arrow after applying the interface identifications. Associativity of linear-map composition gives the displayed source and target. Compose the stored inverse arrows in reverse order; the two-adic inverse lemmas and the inverse fields of the geometric-extension and singular-coefficient certificates cancel adjacent pairs. The remaining composite is the identity. □

Definition 8.10.25 (Singular Betti number of a complex lift). Uses: `def:weil-smooth-proper-lifting-datum`

Lean: `Weil.Arithmetic.singularBettiNumber`

Define

$$b_i^{\text{sing}}(X_{\mathbf{C}}) := \dim_{\mathbf{Q}} H^i_{\text{sing}}\big(X_{\mathbf{C}}(\mathbf{C}), \mathbf{Q}\big).$$

Lemma 8.10.26 (Finite-dimensional scalar extension preserves dimension). *Uses:*

Lean: Weil.Arithmetic.finrank_scalarExtension

Let $K \to L$ be an injective field map and let V be a finite-dimensional K-vector space. Then

$$\dim_L(L \otimes_K V) = \dim_K V.$$

Proof. Uses:

Instantiate the existing mathlib theorem

$$\texttt{Module.finrank_baseChange} : \operatorname{finrank}_L(L \otimes_K V) = \operatorname{finrank}_K(V)$$

with $S = K$, $R = L$, and $M' = V$. Fields satisfy `StrongRankCondition`; their modules are free by the division-ring module instance; the field map supplies Algebra KL and the required scalar tower. The instantiated theorem is exactly the target. □

Theorem 8.10.27 (Betti comparison for a supplied smooth proper lift). *Uses:* *def:weil-compatible-realization-family*, *def:weil-realization-independent-betti-number*, *def:weil-lift-comparison-interface*, *def:weil-singular-betti-number*

Lean: *Weil.Arithmetic.commonBettiNumber_eq_singular*

For every lift-comparison interface and every $i \le B$,

$$b_i^{\mathrm{com}} = \dim_{\mathbf{Q}} H^i_{\mathrm{sing}}\big(X_{\mathbf{C}}(\mathbf{C}), \mathbf{Q}\big) = b_i^{\mathrm{sing}}(X_{\mathbf{C}}).$$

Proof. Uses: `def:weil-singular-betti-number`, `lemma:weil-lift-betti-comparison-equivalence-typed`, `lemma:weil-field-extension-finrank`, `thm:weil-betti-number-realization-independence`

Betti independence identifies b_i^{com} with $\dim_{\mathbf{Q}_{\ell_\lambda}} V_{\lambda,i}$. The comparison equivalence identifies that dimension with the finrank of

$$\mathbf{Q}_{\ell_\lambda} \otimes_{\mathbf{Q}} H^i_{\mathrm{sing}}(X_{\mathbf{C}}(\mathbf{C}), \mathbf{Q}).$$

The scalar-extension finrank lemma reduces this to the rational singular dimension. The last equality is the Definition of $b_i^{\mathrm{sing}}(X_{\mathbf{C}})$. □

> **Status.** The preceding theorem is conditional, not an import of smooth proper base change or Artin comparison. Its interface has four separate nonimportable obligations: OBL-M05-SPBC-FINITE constructs $\mathrm{sp}_{i,r}$, proves the four fields of Definition 8.10.7, and targets SGA 4, Exposé XII, Théorème 5.1 together with Exposé XVI, Théorème 1.1 and §2; OBL-M05-SPBC-ADIC proves that the finite systems model the module02 integral pro-etale objects and that inverse limits followed by $\mathbf{Q}_\ell$-rationalization give the displayed rational cohomology; OBL-M05-GEOMETRIC-EXTENSION constructs the algebraically closed field-extension comparison and all fields of its certificate; and OBL-M05-ARTIN-COMPARISON constructs the site morphism, the finite comparison maps, the inverse-limit passage, and the singular coefficient-extension certificate, targeting SGA 4, Exposé XVI, Théorème 4.1. The supplied notes only state the composite argument in Chapter 14, printed p. 76/physical PDF p. 86.

8.11 Bridge from the weight module to arithmetic recovery

Definition 8.11.1 (Smooth-proper recovery bridge data). Uses: `def:weil-arithmetic-factor-recovery-candidate`, `def:weil-weight-smooth-proper-rh-input-data`

Lean: `Weil.Arithmetic.SmoothProperRecoveryBridgeData`

A smooth-proper recovery bridge datum consists of an arithmetic factor-recovery candidate $\mathcal{R}_0$, a Definition 7.9.19 $\mathcal{W}$, and a stored identification of the bounded graded based automorphism family G in $\mathcal{R}_0$ with the ordinary cohomology family in $\mathcal{W}$. In particular the vector spaces, geometric Frobenius operators, coefficient field, algebraic closure, prime-power datum, and degree indices agree after transport.

Lemma 8.11.2 (Smooth-proper universal purity supplies factor purity). *Uses:* *def:weil-smooth-proper-recovery-bridge-data*, *def:weil-graded-factor-universal-purity-predicate*, *def:weil-weight-graded-universal-purity-predicate*

Lean: *Weil.Arithmetic.weightPurity_to_factorPurity*

Every smooth-proper recovery bridge datum satisfies PureFactors(G).

Proof. Uses: `def:weil-graded-factor-universal-purity-predicate`, `def:weil-raw-universal-realization-purity`, `def:weil-weight-graded-universal-purity-predicate`, `def:weil-weight-universal-elladic-eigenvalue-purity-predicate`, `lemma:weil-graded-factor-reciprocal-roots-eigenvalues`, `thm:weil-weight-smooth-proper-universal-purity`, `def:weil-smooth-proper-recovery-bridge-data`

Fix $i \le B$ and a reciprocal root α of $P_{G,i}$. The reciprocal-root support lemma turns this membership into a nonzero ordinary eigenspace for α on the scalar extension of V_i. Apply Theorem 7.9.20 to the identified module06 input and this eigenspace. Its conclusion is

$$\forall \jmath : \overline{E} \xrightarrow{\sim} \mathbf{C}, \qquad |\jmath(\alpha)| = q^{i/2}.$$

After unfolding the two displayed purity predicates, this is exactly $\mathrm{RawPure}_q(\alpha, i)$. Since i and α were arbitrary, the defining universal quantifiers of PureFactors(G) hold. □

> **Status.** This bridge fixes the topological order: module06 proves universal smooth-proper purity first; module05 then performs no-cancellation, rational descent, integral recovery, and realization independence. No statement in module05 is an input to Theorem 7.9.20.

Chapter 9

Final conditional assembly of the Weil conjectures

Status. This chapter is a terminal assembly theorem for explicitly supplied data. It is not an unconditional construction of the required etale, trace, duality, or weight certificates from a scheme. The missing constructions are displayed as separate final-module obligations obligations. Geometric Frobenius is used throughout, it acts on $E(1)$ by q^{-1}, and every factor is $P_i(T) = \det(1 - TF_i)$.

9.1 The classical scheme and its two point-counting presentations

Definition 9.1.1 (Finite-type structural-morphism predicate)**.** Uses: `def:weil-finite-field-data`

Lean: `Weil.Final.IsFiniteTypeStructureMorphism`

For a morphism $a : Y \to \operatorname{Spec} k_0$, define

$$\mathrm{IsFiniteType}(a) := \mathrm{LocallyOfFiniteType}(a) \ \wedge \ \mathrm{QuasiCompact}(a).$$

This is the scheme-theoretic finite-type predicate; it is strictly stronger than the first conjunct alone.

Definition 9.1.2 (Classical smooth scheme core datum)**.** Uses: `def:weil-finite-field-data,def:weil-final-finite-type-structure-predicate`

Lean: `Weil.Final.ClassicalSchemeCoreData`

A classical smooth scheme core datum $\mathcal{X}^\circ$ consists of a scheme X_0, a morphism

$$a_0 : X_0 \longrightarrow \operatorname{Spec} k_0,$$

a nonnegative integer d, a proof of $\mathrm{IsFiniteType}(a_0)$, a separate proof that a_0 is smooth, a proof that every irreducible component of X_0 has dimension d, and a proof that

$$X := X_0 \times_{\operatorname{Spec} k_0} \operatorname{Spec} k$$

is connected. This datum contains no projectivity or properness field.

Lemma 9.1.3 (The core structure morphism is locally of finite type)**.** *Uses:* `def:weil-final-classical-scheme-core-data`

Lean: `Weil.Final.core_locallyOfFiniteType`

The structural morphism a_0 is locally of finite type.

Proof. Uses: `def:weil-final-classical-scheme-core-data,def:weil-final-finite-type-structure-predicate`

Project the finite-type certificate from the core datum, unfold IsFiniteType, and project its first conjunct. □

Lemma 9.1.4 (The core structure morphism is quasi-compact)**.** *Uses:* `def:weil-final-classical-scheme-core-data`

Lean: `Weil.Final.core_quasiCompact`

The structural morphism a_0 is quasi-compact.

Proof. Uses: def:weil-final-classical-scheme-core-data, def:weil-final-finite-type-structure-predicate

Project the finite-type certificate from the core datum, unfold IsFiniteType, and project its second conjunct. □

Definition 9.1.5 (Standard homogeneous polynomial ring). Uses: def:weil-finite-field-data

Lean: Weil.Final.standardHomogeneousRing

For $N : \mathbf{N}$, define

$$S_N := \mathrm{MvPolynomial}(\mathrm{Fin}(N+1), k_0).$$

Definition 9.1.6 (Standard homogeneous grading). Uses: def:weil-final-standard-homogeneous-ring

Lean: Weil.Final.standardHomogeneousGrading

For $n : \mathbf{N}$, define

$$\mathscr{G}_N(n) := \mathrm{MvPolynomial.\,homogeneousSubmodule}(\mathrm{Fin}(N+1), k_0, n) \subseteq S_N.$$

Definition 9.1.7 (Installed standard graded-algebra structure). Uses: def:weil-final-standard-homogeneous-grading

Lean: Weil.Final.standardGradedAlgebra

Define the graded-algebra structure on $\mathscr{G}_N$ to be the exact mathlib object

$$\mathrm{MvPolynomial.\,gradedAlgebra}\,.$$

Install this object as the local GradedAlgebra-instance whenever $\mathrm{Proj}(\mathscr{G}_N)$ is formed.

Definition 9.1.8 (Standard scalar-tower certificate). Uses: def:weil-final-standard-homogeneous-grading, def:weil-final-standard-graded-algebra-structure

Lean: Weil.Final.standardScalarTower

For the natural algebra structures, define

$$\tau_N : \mathrm{IsScalarTower}\big(k_0, \mathscr{G}_N(0), S_N\big)$$

by the exact instance IsScalarTower.of_algebraMap_eq'. Its required ring-homomorphism equality is definitional: the composite $k_0 \to \mathscr{G}_N(0) \to S_N$ is the multivariate-polynomial constant map $C : k_0 \to S_N$.

Definition 9.1.9 (Standard finite-type certificate). Uses: def:weil-final-standard-homogeneous-ring, def:weil-final-standard-homogeneous-grading, def:weil-final-standard-scalar-tower

Lean: Weil.Final.standardFiniteType

Define

$$\mathrm{FT}_N : \mathrm{Algebra.\,FiniteType}\big(\mathscr{G}_N(0), S_N\big)$$

by applying Algebra.FiniteType.of_restrictScalars_finiteType to the standard finite-type k_0-algebra S_N and to τ_N.

Definition 9.1.10 (Degree-zero coefficient homomorphism). Uses: def:weil-final-standard-homogeneous-grading

Lean: Weil.Final.degreeZeroCoeffHom

Define the ring homomorphism

$$e_N : \mathscr{G}_N(0) \longrightarrow k_0, \qquad e_N(f) := \mathrm{coeff}(0, f).$$

The zero, one and addition fields are coefficient simplifications. The multiplication field is the specialization of MvPolynomial.coeff_mul at the zero multi-index.

Definition 9.1.11 (Degree-zero constant homomorphism). Uses: def:weil-final-standard-homogeneous-grading

Lean: Weil.Final.degreeZeroConstHom

Define the ring homomorphism

$$c_N : k_0 \longrightarrow \mathscr{G}_N(0), \qquad c_N(r) := \big(C(r), \mathrm{MvPolynomial.\,isHomogeneous_C}(r)\big).$$

Its ring-homomorphism fields follow by subtype extensionality and the four corresponding simplification rules for C.

Lemma 9.1.12 (Coefficient after the constant map). *Uses:* *def:weil-final-degree-zero-coefficient-map*, *def:weil-final-degree-zero-constant-map*

Lean: *Weil.Final.degreeZeroCoeffConst*

For every $r \in k_0$, $e_N(c_N(r)) = r$.

Proof. Uses: def:weil-final-degree-zero-coefficient-map,def:weil-final-degree-zero-constant-map

Unfold both maps and apply the coefficient-of-C simplification at the zero multi-index. □

Lemma 9.1.13 (The constant map recovers every degree-zero element). *Uses:* *def:weil-final-degree-zero-coefficient-map*, *def:weil-final-degree-zero-constant-map*, *def:weil-final-standard-homogeneous-grading*

Lean: *Weil.Final.degreeZeroConstCoeff*

For every $f \in \mathscr{G}_N(0)$, $c_N(e_N(f)) = f$.

Proof. Uses: def:weil-final-degree-zero-coefficient-map,def:weil-final-degree-zero-constant-map,def:weil-final-standard-homogeneous-grading

Apply subtype extensionality. By MvPolynomial.homogeneousComponent_zero, the underlying polynomial of the left side is the degree-zero homogeneous component of f. By MvPolynomial.homogeneousComponent_of_mem, that component is f, because the membership proof stored in f has degree zero.

□

Definition 9.1.14 (Degree-zero ring equivalence). Uses: def:weil-final-degree-zero-coefficient-map,def:weil-final-degree-zero-constant-map

Constructed by: lemma:weil-final-degree-zero-coefficient-constant,lemma:weil-final-degree-zero-constant-coefficient

Lean: Weil.Final.degreeZeroRingEquiv

Define

$$\epsilon_N : \mathscr{G}_N(0) \simeq_{\mathrm{Ring}} k_0$$

by RingEquiv.ofRingHom(e_N, c_N), using the two preceding inverse equations as equalities of ring homomorphisms.

Definition 9.1.15 (Degree-zero commutative-ring isomorphism). Uses: def:weil-final-degree-zero-ring-equivalence

Lean: Weil.Final.degreeZeroCommRingIso

Define

$$\epsilon_N^{\mathrm{CRing}} : \mathrm{CommRingCat.of}(\mathscr{G}_N(0)) \simeq \mathrm{CommRingCat.of}(k_0)$$

to be ϵ_N.toCommRingCatIso.

Definition 9.1.16 (Degree-zero affine-scheme isomorphism). Uses: def:weil-final-degree-zero-commring-isomorphism

Lean: Weil.Final.degreeZeroSpecIso

Contravariantly define

$$s_N : \mathrm{Spec}\,\mathscr{G}_N(0) \simeq \mathrm{Spec}\,k_0$$

by

$$s_N := \mathrm{Scheme.Spec.mapIso}\big((\epsilon_N^{\mathrm{CRing}})^{-1,\mathrm{op}}\big).$$

The inverse and the opposite are both explicit; no definitional identification of the two affine schemes is used.

Definition 9.1.17 (Standard projective scheme). Uses: def:weil-final-standard-homogeneous-grading,def:weil-final-standard-graded-algebra-structure

Lean: Weil.Final.standardProjectiveScheme

Define

$$\mathbf{P}^N_{k_0} := \mathrm{Proj}(\mathscr{G}_N).$$

Definition 9.1.18 (Standard projective structural morphism). Uses: def:weil-final-standard-homogeneous-grading,def:weil-final-degree-zero-spec-isomorphism,def:weil-final-standard-projective-scheme

Lean: Weil.Final.standardProjectiveStructureMap

Define

$$\pi_N : \mathbf{P}^N_{k_0} \longrightarrow \operatorname{Spec} k_0$$

to be the displayed composite

$$\operatorname{Proj}(\mathscr{G}_N) \xrightarrow{\operatorname{Proj.toSpecZero}(\mathscr{G}_N)} \operatorname{Spec} \mathscr{G}_N(0) \xrightarrow{s_N^{\mathrm{hom}}} \operatorname{Spec} k_0.$$

Lemma 9.1.19 (The standard projective structural morphism is proper). Uses: def:weil-final-standard-projective-structure-map

Lean: Weil.Final.standardProjectiveStructureMap_isProper

The morphism π_N is proper.

Proof. Uses: def:weil-final-standard-graded-algebra-structure,def:weil-final-standard-finite-type-certificate,def:weil-final-degree-zero-spec-isomorphism,def:weil-final-standard-projective-structure-map

Instantiate Proj.instIsProperToSpecZeroOfFiniteTypeSubtypeMemOfNatNat with $\mathscr{G}_N$ and FT_N. The morphism s_N^{hom} is an isomorphism. Apply the exact generic leaf

$$\text{CategoryTheory.MorphismProperty.of_isIso(@IsProper)}\ s_N^{\mathrm{hom}}$$

to obtain its properness. Apply IsProper.instCompScheme once to the two proper morphisms. This is exactly the target checked by the pinned projective-base harness. □

Definition 9.1.20 (Projective-presentation candidate). Uses: def:weil-final-classical-scheme-core-data,def:weil-final-standard-projective-scheme

Lean: Weil.Final.ProjectivePresentationCandidate

A projective-presentation candidate for $\mathscr{X}^\circ$ consists of a natural number N and a morphism

$$j : X_0 \longrightarrow \mathbf{P}^N_{k_0}.$$

No closed-immersion or over-base property is imposed in this Definition.

Definition 9.1.21 (Projective-presentation closed-immersion predicate). Uses: def:weil-final-projective-presentation-candidate

Lean: Weil.Final.IsProjectivePresentationClosed

Define IsPresentationClosed(N, j) to mean IsClosedImmersion(j).

Definition 9.1.22 (Projective-presentation over-base predicate). Uses: def:weil-final-classical-scheme-core-data,def:weil-final-projective-presentation-candidate,def:weil-final-standard-projective-structure-map

Lean: Weil.Final.IsProjectivePresentationOverBase

Define IsPresentationOverBase(N, j) to be the equality

$$j \gg \pi_N = a_0$$

in the hom-set $X_0 \longrightarrow \operatorname{Spec} k_0$.

Definition 9.1.23 (Certified projective presentation). Uses: def:weil-final-projective-presentation-candidate,def:weil-final-projective-presentation-closed-predicate,def:weil-final-projective-presentation-over-base-predicate

Lean: Weil.Final.ProjectivePresentationCertificate

A certified projective presentation is a candidate (N, j), a proof of IsPresentationClosed(N, j), and a proof of IsPresentationOverBase(N, j).

Definition 9.1.24 (Classical smooth projective input datum). Uses: def:weil-final-classical-scheme-core-data,def:weil-final-projective-presentation-certificate

Lean: Weil.Final.ClassicalSchemeData

A classical smooth projective scheme datum $\mathscr{X}$ is a classical smooth scheme core datum together with a certified projective presentation. Thus projectivity is finite data (N, j) plus two separately named proof fields, not an undefined predicate.

Definition 9.1.25 (Projective-presentation composite). Uses: `def:weil-final-classical-scheme-data,def:weil-final-standard-projective-structure-map`

Lean: `Weil.Final.projectivePresentationComposite`

For $\mathcal{X}$, define

$$p_{\mathcal{X}} := j \gg \pi_N : X_0 \longrightarrow \operatorname{Spec} k_0,$$

using the N and j projected from its presentation certificate.

Lemma 9.1.26 (The chosen projective embedding is closed). *Uses:* `def:weil-final-classical-scheme-data`

Lean: `Weil.Final.projectivePresentation_isClosedImmersion`

The chosen morphism $j : X_0 \to \mathbf{P}^N_{k_0}$ is a closed immersion.

Proof. Uses: `def:weil-final-classical-scheme-data,def:weil-final-projective-presentation-certificate,def:weil-final-projective-presentation-closed-predicate`

Project the closed-immersion proof field from the presentation certificate and unfold its predicate. □

Lemma 9.1.27 (The projective-presentation composite is the structure map). *Uses:* `def:weil-final-classical-scheme-data,def:weil-final-projective-presentation-composite`

Lean: `Weil.Final.projectivePresentationComposite_eq_structureMap`

One has $p_{\mathcal{X}} = a_0$.

Proof. Uses: `def:weil-final-projective-presentation-composite,def:weil-final-projective-presentation-certificate,def:weil-final-projective-presentation-over-base-predicate`

Project the over-base equality from the presentation certificate and unfold the composite and its predicate. □

Lemma 9.1.28 (The projective-presentation composite is proper). *Uses:* `def:weil-final-classical-scheme-data,def:weil-final-projective-presentation-composite`

Lean: `Weil.Final.projectivePresentationComposite_isProper`

The morphism $p_{\mathcal{X}}$ is proper.

Proof. Uses: `def:weil-final-projective-presentation-composite,lemma:weil-final-projective-presentation-closed,lemma:weil-final-standard-projective-structure-map-proper`

By IsFinite.instOfIsClosedImmersion, j is finite; by IsProper.instOfIsFinite, it is proper. Compose its properness with the properness of π_N using IsProper.instCompScheme. The resulting inferred instance has target IsProper($p_{\mathcal{X}}$). □

Definition 9.1.29 (Projective-to-proper target predicate). Uses: `def:weil-final-classical-scheme-data`

Lean: `Weil.Final.IsProjectiveProperTarget`

Define ProjectiveProperTarget($\mathcal{X}$) to mean that the stored structural morphism $a_0 : X_0 \to \operatorname{Spec} k_0$ is proper.

Definition 9.1.30 (Projective-to-proper certificate). Uses: `def:weil-final-classical-scheme-data,def:weil-final-projective-proper-predicate`

Lean: `Weil.Final.ProjectiveProperCertificate`

A projective-to-proper certificate for $\mathcal{X}$ is an inhabitant of ProjectiveProperTarget($\mathcal{X}$).

Definition 9.1.31 (Presentation-induced projective-to-proper certificate). Uses: `def:weil-final-classical-scheme-data,def:weil-final-projective-proper-certificate`

Constructed by: `lemma:weil-final-projective-presentation-factorization,lemma:weil-final-projective-presentation-composite-proper`

Lean: `Weil.Final.projectiveProperCertificateFromPresentation`

Define the presentation-induced projective-to-proper certificate by transporting IsProper($p_{\mathcal{X}}$) along $p_{\mathcal{X}} = a_0$. No independence from the selected presentation is asserted or required.

Definition 9.1.32 (Smooth-proper weight datum constructed from projectivity). Uses: `def:weil-final-classical-scheme-data,def:weil-final-projective-proper-certificate-from-presentation,def:weil-weight-smooth-proper-scheme-data`

Lean: `Weil.Final.smoothProperWeightDatum`

From $\mathcal{X}$, define the Definition 7.9.3 with scheme X_0 and dimension d. Its finite-type field is the stored IsFiniteType(a_0)-certificate (not merely its locally finite-type conjunct); its smooth and pure-dimension fields come from the core datum, and its properness field is the presentation-induced certificate Definition 9.1.31.

Lemma 9.1.33 (Projectivity supplies the properness input). *Uses:* def:weil-final-classical-scheme-data

Lean: Weil.Final.projective_implies_proper

For every classical smooth projective scheme datum, the structural morphism a_0 is proper.

Proof. Uses: def:weil-final-projective-proper-certificate-from-presentation,def:weil-final-projective-proper-predicate

Unfold the target predicate and apply the presentation-induced certificate constructed from the explicit projective presentation. □

> **Status.** Hypothesis routing is fixed as follows. The explicit finite-type certificate IsFiniteType(a_0) is used for finite point sets and the trace-formula scope. Projectivity is used only to supply the preceding properness certificate. Properness then supplies compact-support comparison and ordinary Poincare duality. Smoothness and pure dimension d supply the duality shift, Tate twist, and weight bounds. Geometric connectedness is used only in the degree-zero certificate and the endpoint factors. A characteristic-zero lift is used only in the optional singular-Betti statement.

Definition 9.1.34 (Closed-point type of the final scheme). Uses: def:weil-final-classical-scheme-data

Lean: Weil.Final.SchemeClosedPoint

Define

$$\mathrm{ClPt}(X_0) := \{x \in X_0 \mid \{x\} \text{ is closed in the underlying topological space of } X_0\}.$$

This subtype, rather than the entire topological carrier, is the target of the closed-orbit comparison and the index type of the scheme Euler product.

Definition 9.1.35 (Scheme point-realization candidate). Uses: def:weil-final-classical-scheme-data,def:weil-final-scheme-closed-point-type,def:weil-finite-orbit-frobenius-datum,def:weil-finite-extension-point-interface,def:weil-closed-point-orbit-interface

Lean: Weil.Final.PointRealizationCandidate

A point-realization candidate for $\mathcal{X}$ consists of:

(i) a finite-orbit geometric-Frobenius datum D;

(ii) a finite-extension point interface P over D;

(iii) a closed-point–orbit interface C over D;

(iv) for every $n \geq 1$, maps

$$r_n : \mathrm{Rat}_P(n) \longrightarrow X_0(\mathbf{F}_{q^n}), \qquad s_n : X_0(\mathbf{F}_{q^n}) \longrightarrow \mathrm{Rat}_P(n);$$

(v) maps

$$r_{\mathrm{cl}} : \mathrm{ClPt}(C) \longrightarrow \mathrm{ClPt}(X_0), \qquad s_{\mathrm{cl}} : \mathrm{ClPt}(X_0) \longrightarrow \mathrm{ClPt}(C).$$

The displayed maps are candidate data; no inverse or degree property is included here.

Definition 9.1.36 (Point-realization compatibility predicate). Uses: def:weil-final-point-realization-candidate,def:weil-finite-extension-point-count,def:weil-closed-point-count

Lean: Weil.Final.IsPointRealization

Define IsPointRealization($\mathcal{P}_0$) to mean the conjunction of the following equations.

(i) For every $n \geq 1$, $s_n r_n = \mathrm{id}$ and $r_n s_n = \mathrm{id}$.

(ii) The maps $r_{\mathrm{cl}}, s_{\mathrm{cl}}$ satisfy both inverse equations.

(iii) For every $c \in \mathrm{ClPt}(C)$, $\deg_C(c) = [\kappa(r_{\mathrm{cl}} c) : k_0]$.

(iv) The finite type structures used by $N_n(P)$ and $b_m(C)$ are the ones transported through these inverse pairs.

Definition 9.1.37 (Certified scheme point realization). Uses: def:weil-final-point-realization-candidate,def:weil-final-point-realization-predicate

Lean: Weil.Final.PointRealizationCertificate

A certified point realization is a candidate $\mathcal{P}_0$ together with a proof of IsPointRealization($\mathcal{P}_0$). The canonical TraceCore3 inhabitant of this type is constructed below from the literal fixed subfields k_n, the ZPC scheme interfaces, and two explicit equivalences. Comparison with arbitrary external presentations of the finite fields is a separate presentation-independence problem and is not a field of this certificate.

Definition 9.1.38 (Finite-extension point number of the scheme). Uses: `def:weil-final-point-realization-certificate`, `def:weil-finite-extension-point-count`

Lean: `Weil.Final.pointNumber`

For $n \geq 1$, define

$$N_n(X_0) := N_n(P) = \#\,\mathrm{Rat}_P(n).$$

Definition 9.1.39 (Point-counting zeta series of the scheme). Uses: `def:weil-final-point-realization-certificate`, `def:weil-finite-extension-counting-zeta-series`

Lean: `Weil.Final.zetaSeries`

Define

$$Z_X(T) := Z_P(T) \in \mathbf{Q}[[T]].$$

Definition 9.1.40 (Closed-point Euler product of the scheme). Uses: `def:weil-final-point-realization-certificate`, `def:weil-final-scheme-closed-point-type`, `def:weil-closed-point-euler-product`

Lean: `Weil.Final.closedPointEulerProduct`

Define

$$Z_{X,\mathrm{cl}}(T) := Z_{\mathrm{cl},C}(T) = \prod_{x \in \mathrm{ClPt}(X_0)}^{\mathrm{lf}} (1 - T^{\deg x})^{-1}.$$

The last expression is read through the degree-preserving inverse pair in the point-realization certificate.

Lemma 9.1.41 (The interface point number is the geometric point number). Uses: `def:weil-final-point-realization-certificate`, `def:weil-final-scheme-point-number`

Lean: `Weil.Final.pointNumber_eq_card`

For every $n \geq 1$,

$$N_n(X_0) = \#X_0(\mathbf{F}_{q^n}).$$

Proof. Uses: `def:weil-final-point-realization-predicate`, `def:weil-final-scheme-point-number`

Project the two inverse equations for r_n, s_n from the certificate and apply `Fintype.card_congr` to the resulting equivalence. Unfolding $N_n(X_0)$ gives the displayed equality. □

Lemma 9.1.42 (Exponential point-count formula). Uses: `def:weil-final-scheme-point-number`, `def:weil-final-scheme-zeta-series`

Lean: `Weil.Final.zetaSeries_eq_exp`

One has

$$Z_X(T) = \mathrm{Exp}\left(\sum_{n \geq 1} N_n(X_0)\frac{T^n}{n}\right).$$

Proof. Uses: `def:weil-final-scheme-point-number`, `def:weil-final-scheme-zeta-series`, `def:weil-finite-extension-counting-log-series`, `def:weil-finite-extension-counting-zeta-series`

Unfold the four Definitions. Both sides are the formal exponential of the series whose constant coefficient is zero and whose degree-n coefficient is $N_n(P)/n$. □

Proposition 9.1.43 (Equality of the exponential and closed-point definitions). Uses: `def:weil-final-scheme-zeta-series`, `def:weil-final-scheme-closed-point-type`, `def:weil-final-scheme-closed-euler-product`

Lean: `Weil.Final.zetaSeries_eq_closedPointEulerProduct`

One has

$$Z_X(T) = Z_{X,\mathrm{cl}}(T) = \prod_{x \in \mathrm{ClPt}(X_0)}^{\mathrm{lf}} (1 - T^{\deg x})^{-1}.$$

Proof. Uses: `def:weil-final-scheme-zeta-series`, `def:weil-final-scheme-closed-euler-product`, `thm:weil-closed-point-euler-exponential`

Apply Theorem 3.6.33 to the point interface P and closed-point interface C over their common orbit datum D. The two sides become the two Definitions in the statement. The degree-preservation field of the certified point realization rewrites the index and exponent as the scheme closed point and its residue degree. □

Corollary 9.1.44 (Integral coefficients of the scheme zeta series). *Uses:* def:weil-final-scheme-zeta-series

Lean: Weil.Final.zetaSeries_mem_intPowerSeries

Every coefficient of $Z_X(T)$ belongs to $\mathbf{Z} \subset \mathbf{Q}$.

Proof. Uses: def:weil-final-scheme-zeta-series,cor:weil-counting-zeta-integral

The scheme zeta series is Z_P, which equals the orbit counting series by the point interface. Apply the integral-coefficient corollary for that orbit series in every degree. □

9.2 The conditional cohomological master package

Definition 9.2.1 (Final cohomological core candidate). Uses: def:weil-final-point-realization-certificate,def:weil-final-projective-proper-certificate,def:weil-compatible-duality-realization-family,def:weil-weight-smooth-proper-rh-input-data

Lean: Weil.Final.CohomologicalCoreCandidate

A final cohomological core candidate consists of:

(i) a certified scheme point realization $\mathcal{P}_X$;

(ii) a projective-to-proper certificate for its classical scheme datum;

(iii) a compatible duality-realization family $\mathfrak{D}$ of dimension d, with underlying realization type Λ, base index λ_*, common bound B, recovery inputs $\mathcal{R}_\lambda$, graded families G_λ, and duality systems $\mathcal{S}_\lambda$;

(iv) for every $\lambda : \Lambda$, a smooth proper RH input datum $\mathcal{W}_\lambda$;

(v) explicit transport maps between the scheme, point interface, cohomology spaces, bases, and geometric-Frobenius operators occurring in $\mathcal{P}_X$, $\mathcal{R}_\lambda$, $\mathcal{S}_\lambda$, and $\mathcal{W}_\lambda$.

No compatibility equation is imposed in this candidate Definition.

Definition 9.2.2 (Literal Module 02 rebind as a based automorphism). Uses: def:weil-cohomology-frobenius-module-input,def:weil-conditional-cohomology-frobenius-module,def:weil-based-linear-automorphism-datum

Lean: Weil.Final.module02BasedRebind

Fix a Module 02 cohomological Frobenius-module input for (H, K), a natural number r, and a displayed basis

$$b : \mathrm{Basis}(\mathrm{Fin}(r), E, H(K)).$$

Define $D^{\mathrm{et}}(H, K, b)$ to be the based automorphism datum whose carrier is $H(K)$, whose basis is b, whose forward operator is Φ_K, and whose displayed inverse is Ψ_K. Its two inverse fields are exactly the two fields of the supplied Frobenius-module input. This is a rebind of already supplied data; it does not construct cohomology, finite-dimensionality, a basis, or geometric Frobenius.

Lemma 9.2.3 (Carrier of the literal Module 02 rebind). *Uses:* def:weil-final-module02-based-rebind

Lean: Weil.Final.module02BasedRebind_carrier

The carrier of $D^{\mathrm{et}}(H, K, b)$ is definitionally $H(K)$.

Proof. Uses: def:weil-final-module02-based-rebind

Unfold the rebind Definition and project its carrier field. □

Lemma 9.2.4 (Basis of the literal Module 02 rebind). *Uses:* def:weil-final-module02-based-rebind

Lean: Weil.Final.module02BasedRebind_basis

After reducing the carrier field to $H(K)$, the displayed basis of $D^{\mathrm{et}}(H, K, b)$ is b.

Proof. Uses: def:weil-final-module02-based-rebind

Unfold the rebind Definition and project its basis field. No equality recursor or change of carrier is involved. □

Lemma 9.2.5 (Forward operator of the literal Module 02 rebind). *Uses:* def:weil-final-module02-based-rebind

Lean: Weil.Final.module02BasedRebind_operator

The forward operator of $D^{\mathrm{et}}(H, K, b)$ is Φ_K.

Proof. Uses: def:weil-final-module02-based-rebind,lemma:weil-conditional-cohomology-frobenius-module-operator

Unfold the rebind and use the operator projection of the conditional cohomological Frobenius module. □

Lemma 9.2.6 (Inverse operator of the literal Module 02 rebind). *Uses:* def:weil-final-module02-based-rebind

Lean: Weil.Final.module02BasedRebind_inverse

The displayed inverse operator of $D^{\mathrm{et}}(H,K,b)$ is Ψ_K.

Proof. Uses: def:weil-final-module02-based-rebind,def:weil-conditional-cohomology-frobenius-module,def:weil-induced-cohomology-frobenius-inverse-candidate

Unfold the two Definitions. The inverse field of the constructed linear equivalence is the supplied inverse candidate Ψ_K. □

Definition 9.2.7 (Literal transport to the common based carrier). Uses: def:weil-final-module02-based-rebind

Lean: Weil.Final.module02LiteralTransport

Define

$$e^{\mathrm{lit}}_{H,K,b} : H(K) \simeq_E D^{\mathrm{et}}(H,K,b).V$$

to be the identity linear equivalence. The target reduces definitionally to $H(K)$ by the rebind Definition, so this map contains no cast along a propositional equality.

Definition 9.2.8 (Inverse literal transport from the common based carrier). Uses: def:weil-final-module02-literal-transport

Lean: Weil.Final.module02LiteralTransportInverse

Define

$$s^{\mathrm{lit}}_{H,K,b} : D^{\mathrm{et}}(H,K,b).V \simeq_E H(K)$$

to be the inverse linear equivalence $(e^{\mathrm{lit}}_{H,K,b})^{-1}$.

Lemma 9.2.9 (The literal transport is the identity equivalence). *Uses:* def:weil-final-module02-literal-transport

Lean: Weil.Final.module02LiteralTransport_eq_refl

After definitional reduction of its target, one has

$$e^{\mathrm{lit}}_{H,K,b} = \mathrm{id}_{H(K)}$$

as an E-linear equivalence.

Proof. Uses: def:weil-final-module02-literal-transport,lemma:weil-final-module02-based-rebind-carrier

Unfold the transport Definition. Both its forward and inverse functions are the identity, so linear-equivalence extensionality gives the equality. □

Lemma 9.2.10 (Left inverse equation for the literal transport). *Uses:* def:weil-final-module02-literal-transport, def:weil-final-module02-literal-transport-inverse

Lean: Weil.Final.module02LiteralTransport_leftInverse

For every $x \in H(K)$,

$$s^{\mathrm{lit}}_{H,K,b}\big(e^{\mathrm{lit}}_{H,K,b}(x)\big) = x.$$

Proof. Uses: def:weil-final-module02-literal-transport-inverse,lemma:weil-final-module02-literal-transport-identity

The inverse of the identity linear equivalence is the identity; evaluate the resulting composite at x. □

Lemma 9.2.11 (Right inverse equation for the literal transport). *Uses:* def:weil-final-module02-literal-transport, def:weil-final-module02-literal-transport-inverse

Lean: Weil.Final.module02LiteralTransport_rightInverse

For every $y \in D^{\mathrm{et}}(H,K,b).V$,

$$e^{\mathrm{lit}}_{H,K,b}\big(s^{\mathrm{lit}}_{H,K,b}(y)\big) = y.$$

Proof. Uses: def:weil-final-module02-literal-transport-inverse,lemma:weil-final-module02-literal-transport-identity

After reducing the carrier, both maps are identities. Evaluate their composite at y. □

Definition 9.2.12 (Basis transported to the literal based carrier). Uses: def:weil-final-module02-literal-transport

Lean: Weil.Final.module02LiteralTransportedBasis

Define b^{lit} to be the basis obtained from b by Basis.map along $e^{\mathrm{lit}}_{H,K,b}$. Thus both sides of the later basis comparison inhabit the basis type of the target carrier; they are not compared across unequal module types.

Lemma 9.2.13 (Basis compatibility of the literal transport). Uses: def:weil-final-module02-literal-transported-basis, def:weil-final-module02-based-rebind

Lean: Weil.Final.module02LiteralTransportedBasis_eq

The transported basis b^{lit} equals the displayed basis of $D^{\mathrm{et}}(H,K,b)$.

Proof. Uses: def:weil-final-module02-literal-transported-basis, lemma:weil-final-module02-literal-transport-identity, lemma:weil-final-module02-based-rebind-basis

Mapping a basis along the identity linear equivalence leaves it unchanged. The target basis is b by the rebind basis projection. □

Lemma 9.2.14 (Frobenius compatibility of the literal transport). Uses: def:weil-final-module02-literal-transport, def:weil-final-module02-based-rebind

Lean: Weil.Final.module02LiteralTransport_frobenius

As linear maps from $H(K)$ to $D^{\mathrm{et}}(H,K,b).V$,

$$e^{\mathrm{lit}}_{H,K,b} \circ \Phi_K = D^{\mathrm{et}}(H,K,b).u \circ e^{\mathrm{lit}}_{H,K,b}.$$

Proof. Uses: lemma:weil-final-module02-literal-transport-identity, lemma:weil-final-module02-based-rebind-operator

Replace the transport by the identity and the target operator by Φ_K. Both sides then reduce to Φ_K. □

9.2.1 Supplied proper-support adapters

> **Status.** The declarations in this subsection are deliberately conditional. They project and repackage a supplied proper-support comparison candidate, a supplied inverse candidate, and supplied equation certificates. They do not construct the geometric map $Ra_! \to Ra_*$, identify its cohomology with the project's selected carriers, prove proper base change, or produce any of the supplied inputs.

Definition 9.2.15 (Supplied derived proper-support projection). Uses: def:weil-duality-proper-support-comparison-candidate

Lean: Weil.Final.suppliedProperSupportDerivedComparison

For a supplied proper-support comparison candidate $\mathcal{C}$, define

$$\kappa^{\mathrm{der}}_X(\mathcal{C}) := \mathcal{C}.\kappa^{\mathrm{der}}_X : Ra_{X!}E_X \longrightarrow Ra_{X*}E_X$$

by projecting its displayed derived component. This Definition does not assert that the type of such candidates is inhabited.

Definition 9.2.16 (Supplied degree proper-support projection). Uses: def:weil-final-supplied-proper-support-derived-comparison, def:weil-duality-proper-support-comparison-candidate

Lean: Weil.Final.suppliedProperSupportDegreeComparison

For an integer i, project the degree component of $\mathcal{C}$ and denote it by

$$\kappa_i(\mathcal{C}) : H^i_c(X,E) \longrightarrow H^i(X,E).$$

Within the supplied candidate this is the named component $H^i(\kappa^{\mathrm{der}}_X(\mathcal{C}))$. No comparison with an independently selected cohomology model is made here.

Definition 9.2.17 (Supplied degree inverse projection). Uses: def:weil-duality-proper-support-inverse-candidate, def:weil-final-supplied-proper-support-degree-comparison

Lean: Weil.Final.suppliedProperSupportDegreeInverse

For a supplied inverse candidate $\mathcal{L}$ for $\mathcal{C}$, define

$$\lambda_i(\mathcal{L}) : H^i(X,E) \longrightarrow H^i_c(X,E)$$

to be its displayed linear map.

Lemma 9.2.18 (Left inverse projected from a supplied support certificate)**.** *Uses:* *def:weil-duality-proper-support-isomorphism-certificate, def:weil-final-supplied-proper-support-degree-comparison, def:weil-final-supplied-proper-support-degree-inverse*

Lean: *Weil.Final.suppliedProperSupport_leftInverse*

For a supplied isomorphism certificate $\mathcal{I}$ for $(\mathcal{C}, \mathcal{L})$,

$$\lambda_i(\mathcal{L}) \circ \kappa_i(\mathcal{C}) = \mathrm{id}_{H^i_c(X,E)}.$$

Proof. Uses: `def:weil-duality-proper-support-isomorphism-certificate`

Project the first equation field of $\mathcal{I}$. The assertion is one of the supplied fields, not a proof that $\mathcal{I}$ exists. □

Lemma 9.2.19 (Right inverse projected from a supplied support certificate)**.** *Uses:* *def:weil-duality-proper-support-isomorphism-certificate, def:weil-final-supplied-proper-support-degree-comparison, def:weil-final-supplied-proper-support-degree-inverse*

Lean: *Weil.Final.suppliedProperSupport_rightInverse*

For the same supplied certificate,

$$\kappa_i(\mathcal{C}) \circ \lambda_i(\mathcal{L}) = \mathrm{id}_{H^i(X,E)}.$$

Proof. Uses: `def:weil-duality-proper-support-isomorphism-certificate`

Project the second equation field of $\mathcal{I}$. □

Definition 9.2.20 (Linear equivalence from a supplied support certificate)**.** Uses: `def:weil-final-supplied-proper-support-degree-comparison`, `def:weil-final-supplied-proper-support-degree-inverse`, `def:weil-duality-proper-support-isomorphism-certificate`

Lean: `Weil.Final.suppliedProperSupportDegreeEquiv`

Constructed by: `lemma:weil-final-supplied-proper-support-left-inverse`, `lemma:weil-final-supplied-proper-support-right-inverse`

Define

$$e_i(\mathcal{C}, \mathcal{L}, \mathcal{I}) : H^i_c(X, E) \simeq_E H^i(X, E)$$

with forward linear map $\kappa_i(\mathcal{C})$, inverse linear map $\lambda_i(\mathcal{L})$, and inverse fields given by the preceding two lemmas.

Lemma 9.2.21 (Forward map of the supplied support equivalence)**.** *Uses:* *def:weil-final-supplied-proper-support-degree-equivalence*

Lean: *Weil.Final.suppliedProperSupportDegreeEquiv_apply*

For every $x \in H^i_c(X, E)$,

$$e_i(\mathcal{C}, \mathcal{L}, \mathcal{I})(x) = \kappa_i(\mathcal{C})(x).$$

Proof. Uses: `def:weil-final-supplied-proper-support-degree-equivalence`

Unfold the linear-equivalence Definition and project its forward map. □

Lemma 9.2.22 (Inverse map of the supplied support equivalence)**.** *Uses:* *def:weil-final-supplied-proper-support-degree-equivalence*

Lean: *Weil.Final.suppliedProperSupportDegreeEquiv_symm_apply*

For every $y \in H^i(X, E)$,

$$e_i(\mathcal{C}, \mathcal{L}, \mathcal{I})^{-1}(y) = \lambda_i(\mathcal{L})(y).$$

Proof. Uses: `def:weil-final-supplied-proper-support-degree-equivalence`

Unfold the linear-equivalence Definition and project its inverse map. □

Lemma 9.2.23 (Inverse Frobenius equation from the supplied predicate)**.** *Uses:* *def:weil-duality-proper-support-frobenius-predicate, def:weil-final-supplied-proper-support-degree-inverse*

Lean: *Weil.Final.suppliedProperSupport_inverseFrobenius*

If a supplied proof $\mathcal{P}$ satisfies $\mathrm{SupportFrob}_i(\mathcal{L})$*, then*

$$\lambda_i(\mathcal{L}) \circ F_i = F_{c,i} \circ \lambda_i(\mathcal{L}).$$

Proof. Uses: `def:weil-duality-proper-support-frobenius-predicate`

Unfold $\mathrm{SupportFrob}_i$ and use the supplied proof $\mathcal{P}$. □

Lemma 9.2.24 (Forward Frobenius equation transferred through a supplied inverse)**.** *Uses: def:weil-duality-proper-support-isomorphism-certificate, def:weil-duality-proper-support-frobenius-predicate, def:weil-final-supplied-proper-support-degree-comparison, def:weil-final-supplied-proper-support-degree-inverse*

Lean: *Weil.Final.suppliedProperSupport_forwardFrobenius*

Under the supplied isomorphism and inverse-Frobenius certificates,

$$\kappa_i(\mathcal{C}) \circ F_{c,i} = F_i \circ \kappa_i(\mathcal{C}).$$

Proof. Uses: lemma:weil-final-supplied-proper-support-left-inverse,lemma:weil-final-supplied-proper-support-right-inverse,lemma:weil-final-supplied-proper-support-inverse-frobenius

Associativity of composition and the three cited equations give

$$\begin{aligned}\kappa_i F_{c,i} &= \kappa_i F_{c,i}(\lambda_i\kappa_i) = \kappa_i(F_{c,i}\lambda_i)\kappa_i \\ &= \kappa_i(\lambda_i F_i)\kappa_i = (\kappa_i\lambda_i)F_i\kappa_i = F_i\kappa_i.\end{aligned}$$

All maps in the chain have source $H^i_c(X,E)$ and target $H^i(X,E)$. □

Definition 9.2.25 (Final cohomological compatibility predicate)**.** Uses: def:weil-final-cohomological-core-candidate,def:weil-final-smooth-proper-weight-datum,def:weil-finite-cohomological-trace-datum,def:weil-weight-smooth-proper-ordinary-cohomology-data,def:weil-duality-graded-similitude-system

Lean: Weil.Final.IsCohomologicalCore

Define IsCohomologicalCore($\mathcal{C}_0$) to mean:

(i) the scheme datum in every $\mathcal{W}_\lambda$ is the explicitly constructed Definition 9.1.32;

(ii) the point interface in every trace datum is the point interface of $\mathcal{P}_X$;

(iii) its degree-i based automorphism is geometric Frobenius on $H^i(X,E_\lambda)$, and the trace-datum proof field is the Grothendieck–Lefschetz equality for every positive iterate;

(iv) the ordinary cohomology family in $\mathcal{W}_\lambda$ is G_λ, including bases and operators;

(v) the carriers and operators of $\mathcal{S}_\lambda$ are those of G_λ, and its pairings are the cup–trace Poincare pairings into $E_\lambda(-d)$;

(vi) all transport maps commute with geometric Frobenius and with change of the realization index.

Definition 9.2.26 (Certified final cohomological core)**.** Uses: def:weil-final-cohomological-core-candidate,def:weil-final-cohomological-core-predicate

Lean: Weil.Final.CohomologicalCoreCertificate

A certified final cohomological core is a candidate together with a proof of IsCohomologicalCore.

Open obligation: OBL-FINAL-COH-TRACE

Open obligation: OBL-FINAL-COH-REALIZATION

Open obligation: OBL-FINAL-DUALITY-REALIZATION

Open obligation: OBL-FINAL-WEIGHT-REALIZATION

Open obligation: OBL-FINAL-PROPER-SUPPORT

Definition 9.2.27 (Cohomological truncation bound)**.** Uses: def:weil-final-cohomological-core-certificate

Lean: Weil.Final.cohomologicalBound

Define $B_{\mathcal{M}}$ to be the natural-number bound B stored in the compatible realization family of the core.

Definition 9.2.28 (Geometric twice-dimension bound)**.** Uses: def:weil-final-classical-scheme-data

Lean: Weil.Final.twiceDimensionBound

Define $B_X := 2d$.

Lemma 9.2.29 (The cohomological bound is twice the dimension)**.** *Uses: def:weil-final-cohomological-core-certificate, def:weil-final-cohomological-bound, def:weil-final-twice-dimension-bound*

Lean: *Weil.Final.cohomologicalBound_eq_twiceDimension*

One has

$$B_{\mathcal{M}} = B_X = 2d.$$

Proof. Uses: def:weil-final-cohomological-bound,def:weil-final-twice-dimension-bound,def:weil-compatible-duality-realization-family

The compatible duality-realization family stores the equality $B = 2d$. The two Definitions name its left and right sides, so rewriting by this field gives both displayed equalities. □

9.3 Degree zero and the normalized top trace

Definition 9.3.1 (Degree-zero unit-map candidate)**.** Uses: `def:weil-final-cohomological-core-certificate`

Lean: `Weil.Final.BottomUnitMapCandidate`

For the base realization, let V_0 be the degree-zero carrier. A degree-zero unit-map candidate is an E_{λ_*}-linear map

$$\eta_0 : E_{\lambda_*} \longrightarrow V_0$$

together with the specified element $1_X := \eta_0(1)$.

Definition 9.3.2 (Degree-zero inverse-map candidate)**.** Uses: `def:weil-final-bottom-unit-map-candidate`

Lean: `Weil.Final.BottomInverseMapCandidate`

An inverse-map candidate for η_0 is an E_{λ_*}-linear map

$$\epsilon_0 : V_0 \longrightarrow E_{\lambda_*}.$$

Definition 9.3.3 (Degree-zero source-inverse predicate)**.** Uses: `def:weil-final-bottom-unit-map-candidate,def:weil-final-bottom-inverse-map-candidate`

Lean: `Weil.Final.IsBottomSourceInverse`

Define BottomSourceInv(η_0, ϵ_0) to be

$$\epsilon_0 \circ \eta_0 = \mathrm{id}_{E_{\lambda_*}}.$$

Definition 9.3.4 (Degree-zero target-inverse predicate)**.** Uses: `def:weil-final-bottom-unit-map-candidate,def:weil-final-bottom-inverse-map-candidate`

Lean: `Weil.Final.IsBottomTargetInverse`

Define BottomTargetInv(η_0, ϵ_0) to be

$$\eta_0 \circ \epsilon_0 = \mathrm{id}_{V_0}.$$

Definition 9.3.5 (Degree-zero Frobenius-normalization predicate)**.** Uses: `def:weil-final-bottom-unit-map-candidate`

Lean: `Weil.Final.IsBottomFrobeniusNormalized`

Define BottomFrob(η_0) to mean

$$F_0 \circ \eta_0 = \eta_0.$$

Thus the scalar operator on the source is the identity, as required for geometric Frobenius on constant sections.

Definition 9.3.6 (Geometric-constant comparison predicate)**.** Uses: `def:weil-final-classical-scheme-data,def:weil-final-bottom-unit-map-candidate,def:weil-final-bottom-inverse-map-candidate`

Lean: `Weil.Final.IsBottomGeometricConstantComparison`

Define BottomConstant(η_0, ϵ_0) to mean that $\eta_0(a)$ is the constant etale class with value a, and that ϵ_0 evaluates this constant on the single connected component of X. This predicate records the exact use of geometric connectedness.

Definition 9.3.7 (Degree-zero connectedness certificate)**.** Uses: `def:weil-final-bottom-unit-map-candidate,def:weil-final-bottom-inverse-map-candidate,def:weil-final-bottom-source-inverse-predicate,def:weil-final-bottom-target-inverse-predicate,def:weil-final-bottom-frobenius-predicate,def:weil-final-bottom-geometric-constant-predicate`

Lean: `Weil.Final.BottomConnectednessCertificate`

A degree-zero connectedness certificate consists of η_0, ϵ_0 and proofs of the four preceding predicates.

Open obligation: `OBL-FINAL-HZERO-UNIT`

Open obligation: `OBL-FINAL-HZERO-CONNECTED`

Open obligation: `OBL-FINAL-HZERO-FROBENIUS`

Definition 9.3.8 (Top trace-coordinate map)**.** Uses: `def:weil-final-cohomological-core-certificate,def:weil-final-bottom-connectedness-certificate`

Lean: `Weil.Final.topTraceCoordinateMap`

Let $B_0 : V_0 \times V_{2d} \to E_{\lambda_*}$ be the degree-zero pairing in $\mathcal{S}_{\lambda_*}$. Define

$$\tau_{2d} : V_{2d} \longrightarrow E_{\lambda_*}, \qquad \tau_{2d}(y) := B_0(1_X, y).$$

Definition 9.3.9 (Normalized top generator). Uses: def:weil-final-cohomological-core-certificate,def:weil-final-bottom-inverse-map-candidate,def:weil-duality-pairing-adjoint-inverse-candidate

Lean: Weil.Final.topGenerator

Let $s_{B_0} : V_0^\vee \to V_{2d}$ be the inverse candidate stored in the perfectness certificate for B_0. Regard ϵ_0 as an element of $V_0^\vee$ and define

$$\omega_X := s_{B_0}(\epsilon_0) \in V_{2d}.$$

Definition 9.3.10 (Inverse candidate for the top trace coordinate). Uses: def:weil-final-top-generator

Lean: Weil.Final.topTraceCoordinateInverse

Define

$$\kappa_{2d} : E_{\lambda_*} \longrightarrow V_{2d}, \qquad \kappa_{2d}(a) := a\omega_X.$$

Definition 9.3.11 (Top trace-normalization predicate). Uses: def:weil-final-top-trace-coordinate-map,def:weil-final-top-generator

Lean: Weil.Final.IsTopTraceNormalized

Define TopTraceNormalized to be the equation

$$\tau_{2d}(\omega_X) = 1.$$

Definition 9.3.12 (Geometric top-trace comparison candidate). Uses: def:weil-final-cohomological-core-certificate,def:weil-final-top-trace-coordinate-map,def:weil-duality-untwisted-top-trace-candidate,def:weil-duality-tate-power-coordinate-map-candidate

Lean: Weil.Final.TopTraceComparisonCandidate

A geometric top-trace comparison candidate consists of the module04 untwisted trace candidate, the coordinate map $E_{\lambda_*}(-d) \to E_{\lambda_*}$, and the proposed equality between their composite on $H^{2d}(X, E_{\lambda_*})$ and τ_{2d}.

Definition 9.3.13 (Geometric top-trace comparison predicate). Uses: def:weil-final-top-trace-comparison-candidate,def:weil-final-top-trace-normalization-predicate

Lean: Weil.Final.IsTopTraceComparison

Define IsTopTraceComparison to mean that the proposed equality holds, the two Tate-coordinate inverse equations hold, the trace is geometric-Frobenius equivariant with scalar q^d, and TopTraceNormalized holds.

Definition 9.3.14 (Certified geometric top trace). Uses: def:weil-final-top-trace-comparison-candidate,def:weil-final-top-trace-comparison-predicate

Lean: Weil.Final.TopTraceComparisonCertificate

A certified geometric top trace is a comparison candidate together with a proof of IsTopTraceComparison.

Open obligation: OBL-FINAL-TOP-TRACE

Open obligation: OBL-FINAL-TOP-TWIST

Open obligation: OBL-FINAL-TOP-PAIRING

Definition 9.3.15 (Final Weil master input datum). Uses: def:weil-final-cohomological-core-certificate,def:weil-final-bottom-connectedness-certificate,def:weil-final-top-trace-comparison-certificate

Lean: Weil.Final.MasterInputData

A final Weil master input datum $\mathcal{M}$ consists of a certified cohomological core, a degree-zero connectedness certificate for its base realization, and a certified geometric top trace for the same realization. All scheme, point, coefficient, cohomology, Frobenius, pairing, and weight objects are required to be literally the transported objects stored in the core certificate.

Lemma 9.3.16 (First inverse equation in degree zero). *Uses:* def:weil-final-master-input-data,def:weil-final-bottom-unit-map-candidate,def:weil-final-bottom-inverse-map-candidate

Lean: Weil.Final.bottom_sourceInverse

One has $\epsilon_0\eta_0 = \mathrm{id}_{E_{\lambda_*}}$.

Proof. Uses: def:weil-final-master-input-data,def:weil-final-bottom-connectedness-certificate,def:weil-final-bottom-source-inverse-predicate

Project the source-inverse proof field and unfold its predicate. □

Lemma 9.3.17 (Second inverse equation in degree zero). *Uses:* def:weil-final-master-input-data, def:weil-final-bottom-unit-map-candidate, def:weil-final-bottom-inverse-map-candidate

Lean: Weil.Final.bottom_targetInverse

One has $\eta_0\epsilon_0 = \mathrm{id}_{V_0}$.

Proof. Uses: def:weil-final-master-input-data,def:weil-final-bottom-connectedness-certificate,def:weil-final-bottom-target-inverse-predicate

Project the target-inverse proof field and unfold its predicate. □

Lemma 9.3.18 (Dimension of degree-zero cohomology). *Uses:* def:weil-final-master-input-data

Lean: Weil.Final.bottom_finrank

One has $\dim_{E_{\lambda_*}} V_0 = 1$.

Proof. Uses: lemma:weil-final-bottom-source-inverse,lemma:weil-final-bottom-target-inverse

The two inverse equations make η_0 a linear equivalence $E_{\lambda_*} \simeq V_0$. Finite dimension is invariant under this equivalence, and the coefficient field has dimension one over itself. □

Lemma 9.3.19 (Geometric Frobenius is the identity in degree zero). *Uses:* def:weil-final-master-input-data

Lean: Weil.Final.bottom_frobenius_eq_id

As an endomorphism of V_0, $F_0 = \mathrm{id}_{V_0}$.

Proof. Uses: def:weil-final-bottom-frobenius-predicate,lemma:weil-final-bottom-target-inverse

For $x \in V_0$, the target-inverse equation writes $x = \eta_0(\epsilon_0 x)$. The Frobenius-normalization field gives

$$F_0 x = F_0\eta_0(\epsilon_0 x) = \eta_0(\epsilon_0 x) = x.$$

Extensionality proves equality of endomorphisms. □

Lemma 9.3.20 (The base-realization degree-zero factor). *Uses:* def:weil-final-master-input-data, def:weil-graded-reciprocal-factor

Lean: Weil.Final.bottom_factor_base

For the base realization,

$$P_{G_{\lambda_*},0}(T) = 1 - T.$$

Proof. Uses: def:weil-graded-reciprocal-factor,def:weil-based-reciprocal-determinant,lemma:weil-final-bottom-dimension-one,lemma:weil-final-bottom-frobenius-identity

Use the basis $\eta_0(1)$ supplied by the two inverse equations. In this one-element basis the matrix of F_0 is $[1]$, so $\det([1] - T[1]) = 1 - T$. Basis independence identifies this determinant with the displayed graded reciprocal factor. □

Lemma 9.3.21 (The normalized top generator has trace one). *Uses:* def:weil-final-master-input-data, def:weil-final-top-trace-normalization-predicate

Lean: Weil.Final.topTrace_normalized

One has $\tau_{2d}(\omega_X) = 1$.

Proof. Uses: def:weil-final-master-input-data,def:weil-final-top-trace-comparison-certificate,def:weil-final-top-trace-comparison-predicate,def:weil-final-top-trace-normalization-predicate

Project the normalization field of the certified top-trace comparison and unfold its predicate. □

Lemma 9.3.22 (First inverse equation for the top trace). *Uses:* def:weil-final-master-input-data, def:weil-final-top-trace-coordinate-map, def:weil-final-top-trace-inverse-map

Lean: Weil.Final.topTrace_sourceInverse

One has

$$\tau_{2d} \circ \kappa_{2d} = \mathrm{id}_{E_{\lambda_*}}.$$

Proof. Uses: def:weil-final-top-trace-inverse-map,lemma:weil-final-top-trace-normalized
For $a \in E_{\lambda_*}$, linearity and normalization give

$$\tau_{2d}(\kappa_{2d}(a)) = \tau_{2d}(a\omega_X) = a\tau_{2d}(\omega_X) = a.$$

Extensionality gives the map equality. □

Lemma 9.3.23 (Second inverse equation for the top trace). *Uses:* def:weil-final-master-input-data, def:weil-final-top-trace-coordinate-map, def:weil-final-top-trace-inverse-map
Lean: Weil.Final.topTrace_targetInverse
One has

$$\kappa_{2d} \circ \tau_{2d} = \mathrm{id}_{V_{2d}}.$$

Proof. Uses: def:weil-final-top-trace-coordinate-map,def:weil-final-top-generator,def:weil-final-top-trace-inverse-map,lemma:weil-duality-pairing-adjoint-apply,lemma:weil-final-bottom-target-inverse
Let $y \in V_{2d}$. Apply the injective adjoint map $\theta_{B_0} : V_{2d} \to V_0^\vee$; injectivity is one inverse equation in the perfectness certificate. For $x \in V_0$, the degree-zero inverse equation gives $x = \eta_0(\epsilon_0 x)$, whence bilinearity gives

$$B_0(x, y) = \epsilon_0(x)B_0(1_X, y) = \epsilon_0(x)\tau_{2d}(y).$$

The Definition of $\omega_X = s_{B_0}(\epsilon_0)$ and the other perfectness inverse equation give $B_0(x, \omega_X) = \epsilon_0(x)$. Multiplying by $\tau_{2d}(y)$ shows that θ_{B_0} has the same value on $\kappa_{2d}(\tau_{2d}y)$ and on y. Injectivity yields the required identity. □

Lemma 9.3.24 (Dimension of top cohomology). *Uses:* def:weil-final-master-input-data
Lean: Weil.Final.top_finrank
One has $\dim_{E_{\lambda_*}} V_{2d} = 1$.

Proof. Uses: lemma:weil-final-top-trace-source-inverse,lemma:weil-final-top-trace-target-inverse
The two top-trace inverse equations define a linear equivalence $V_{2d} \simeq E_{\lambda_*}$. Its target has dimension one. □

Lemma 9.3.25 (Top trace is Frobenius equivariant with multiplier q^d). *Uses:* def:weil-final-master-input-data, def:weil-final-top-trace-coordinate-map
Lean: Weil.Final.topTrace_frobenius
For every $y \in V_{2d}$,

$$\tau_{2d}(F_{2d}y) = q^d\tau_{2d}(y).$$

Proof. Uses: def:weil-final-top-trace-coordinate-map,def:weil-duality-frobenius-similitude-predicate,lemma:weil-final-bottom-frobenius-identity
The similitude field of B_0 says $B_0(F_0x, F_{2d}y) = q^d B_0(x, y)$. Substitute $x = 1_X$ and rewrite $F_0(1_X) = 1_X$ by the degree-zero Frobenius identity. The two sides are the asserted coordinate values by Definition. □

Lemma 9.3.26 (Geometric Frobenius is q^d in top degree). *Uses:* def:weil-final-master-input-data
Lean: Weil.Final.top_frobenius_eq_smul
As an endomorphism of V_{2d},

$$F_{2d} = q^d\,\mathrm{id}_{V_{2d}}.$$

Proof. Uses: lemma:weil-final-top-trace-frobenius-coordinate,lemma:weil-final-top-trace-target-inverse
For $y \in V_{2d}$, use $y = \kappa_{2d}(\tau_{2d}y)$. The coordinate identity gives

$$F_{2d}y = \kappa_{2d}(\tau_{2d}(F_{2d}y)) = \kappa_{2d}(q^d\tau_{2d}y) = q^d y.$$

Extensionality proves the endomorphism equality. □

Lemma 9.3.27 (The base-realization top factor). *Uses:* def:weil-final-master-input-data, def:weil-graded-reciprocal-factor
Lean: Weil.Final.top_factor_base
For the base realization,

$$P_{G_{\lambda_*},2d}(T) = 1 - q^d T.$$

Proof. Uses: def:weil-graded-reciprocal-factor,def:weil-based-reciprocal-determinant,lemma:weil-final-top-dimension-one,lemma:weil-final-top-frobenius-scalar
Use the one-element basis ω_X. The top Frobenius matrix is $[q^d]$, hence the reciprocal determinant is $\det([1] - T[q^d]) = 1 - q^d T$. □

9.4 Common integral factors, degrees, and rationality over the rationals

Definition 9.4.1 (Final common integral Frobenius factor). Uses: `def:weil-final-master-input-data,def:weil-family-common-integral-factor`

Lean: `Weil.Final.commonFrobeniusFactor`

For $0 \le i \le 2d$, define

$$P_i(T) := P_{i,\mathbf{Z}}^{\mathrm{com}}(T) \in \mathbf{Z}[T].$$

Definition 9.4.2 (Final common Betti number). Uses: `def:weil-final-master-input-data,def:weil-realization-independent-betti-number`

Lean: `Weil.Final.commonBettiNumber`

For $0 \le i \le 2d$, define

$$b_i := b_i^{\mathrm{com}} = \operatorname{natDegree} P_i.$$

Set $b_i = 0$ for indices outside $[0, 2d]$.

Definition 9.4.3 (Final Euler characteristic). Uses: `def:weil-final-master-input-data,def:weil-family-common-euler-characteristic`

Lean: `Weil.Final.eulerCharacteristic`

Define

$$\chi := \chi^{\mathrm{com}} = \sum_{i=0}^{2d} (-1)^i b_i.$$

Definition 9.4.4 (Final half functional-equation exponent). Uses: `def:weil-final-master-input-data,def:weil-family-common-half-functional-equation-exponent`

Lean: `Weil.Final.halfFunctionalEquationExponent`

Define $h := h^{\mathrm{com}} \in \mathbf{Z}$.

Definition 9.4.5 (Final functional-equation sign). Uses: `def:weil-final-master-input-data,def:weil-family-common-functional-equation-sign`

Lean: `Weil.Final.functionalEquationSign`

Define

$$\varepsilon := \varepsilon^{\mathrm{com}} = (-1)^{\chi} q^{-h} \Delta^{\mathrm{com}} \in \mathbf{Q}.$$

Definition 9.4.6 (Final rational zeta function). Uses: `def:weil-final-common-frobenius-factor`

Lean: `Weil.Final.rationalZetaFunction`

Define

$$Z_X^{\mathrm{rat}}(T) := \prod_{i=0}^{2d} P_i(T)^{(-1)^{i+1}} \in \mathbf{Q}(T),$$

where the even-degree factors occur in the denominator and the odd-degree factors occur in the numerator.

Lemma 9.4.7 (The common factor realizes every cohomological factor). *Uses:* `def:weil-final-master-input-data,def:weil-final-common-frobenius-factor,def:weil-graded-reciprocal-factor`

Lean: `Weil.Final.commonFactor_map`

For every $\lambda : \Lambda$ and $0 \le i \le 2d$, the coefficient map $\mathbf{Z}[T] \to E_\lambda[T]$ sends

$$P_i(T) \longmapsto P_{G_\lambda,i}(T) = \det(1 - TF_{\lambda,i} \mid V_{\lambda,i}).$$

Proof. Uses: `def:weil-final-common-frobenius-factor,def:weil-final-cohomological-core-predicate,thm:weil-integral-factor-realization-independence`

Apply the realization-independence theorem to the underlying compatible family. Its coefficient-field image is $P_{G_\lambda,i}$. The core compatibility field identifies that based automorphism with geometric Frobenius on $H^i(X, E_\lambda)$. □

Proposition 9.4.8 (Integrality and normalization of every factor). *Uses:* `def:weil-final-master-input-data,def:weil-final-common-frobenius-factor`

Lean: `Weil.Final.factor_integral_normalized`

For every $0 \le i \le 2d$,

$$P_i(T) \in \mathbf{Z}[T], \qquad P_i(0) = 1.$$

Proof. Uses: def:weil-final-common-frobenius-factor,thm:weil-integral-factor-realization-independence

Membership in $\mathbf{Z}[T]$ is the output type of the Definition. The constant-coefficient assertion is the normalization field of the cited realization-independence theorem. □

Proposition 9.4.9 (Degrees equal all ℓ-adic Betti dimensions). *Uses:* def:weil-final-master-input-data, def:weil-final-common-frobenius-factor, def:weil-final-common-betti-number

Lean: Weil.Final.factor_degree_eq_betti

For every $\lambda : \Lambda$ and $0 \le i \le 2d$,

$$\deg P_i = b_i = \dim_{E_\lambda} H^i(X, E_\lambda).$$

The right side is independent of ℓ_λ, and $b_i = 0$ outside $[0, 2d]$.

Proof. Uses: def:weil-final-common-frobenius-factor,def:weil-final-common-betti-number,def:weil-final-cohomological-core-predicate,thm:weil-betti-number-realization-independence

The first equality is the Definition of b_i. The cited theorem identifies b_i^{com} with the dimension of $V_{\lambda,i}$. The core certificate identifies that carrier with $H^i(X, E_\lambda)$. The outside-range assertion is the second clause of the common-Betti Definition. □

Lemma 9.4.10 (Poincare symmetry of the common Betti numbers). *Uses:* def:weil-final-master-input-data, def:weil-final-common-betti-number

Lean: Weil.Final.commonBetti_symm

For $0 \le i \le 2d$,

$$b_{2d-i} = b_i.$$

Proof. Uses: lemma:weil-duality-graded-dimension-symmetry,thm:weil-betti-number-realization-independence

Apply dimension symmetry to $\mathcal{S}_{\lambda_*}$. Replace both dimensions by their common Betti numbers using realization independence. □

Lemma 9.4.11 (The common degree-zero factor). *Uses:* def:weil-final-master-input-data, def:weil-final-common-frobenius-factor

Lean: Weil.Final.commonBottomFactor

One has $P_0(T) = 1 - T$.

Proof. Uses: lemma:weil-final-common-factor-realization,lemma:weil-final-bottom-factor-base

Map both polynomials to $E_{\lambda_*}[T]$. The common-factor realization lemma turns the image of P_0 into the base factor, and the base-factor lemma computes it as $1 - T$. Injectivity of $\mathbf{Z}[T] \to E_{\lambda_*}[T]$ reflects the equality. □

Lemma 9.4.12 (The common bottom Betti number). *Uses:* def:weil-final-master-input-data, def:weil-final-common-betti-number

Lean: Weil.Final.commonBottomBetti

One has $b_0 = 1$.

Proof. Uses: prop:weil-final-factor-degree-and-betti-independence,lemma:weil-final-bottom-dimension-one

Use the degree–dimension proposition at λ_* and $i = 0$, then apply the computed dimension of V_0. □

Lemma 9.4.13 (The common top factor). *Uses:* def:weil-final-master-input-data, def:weil-final-common-frobenius-factor

Lean: Weil.Final.commonTopFactor

One has $P_{2d}(T) = 1 - q^d T$.

Proof. Uses: lemma:weil-final-common-factor-realization,lemma:weil-final-top-factor-base

Map to $E_{\lambda_*}[T]$. The two cited lemmas identify the images of the two sides. The coefficient embedding is injective, so polynomial extensionality reflects the equality over $\mathbf{Z}$. □

Lemma 9.4.14 (The common top Betti number). *Uses:* def:weil-final-master-input-data, def:weil-final-common-betti-number

Lean: Weil.Final.commonTopBetti

One has $b_{2d} = 1$.

Proof. Uses: prop:weil-final-factor-degree-and-betti-independence,lemma:weil-final-top-dimension-one

Use the degree–dimension proposition at λ_* and $i = 2d$, then apply the computed top dimension. □

Lemma 9.4.15 (The rational zeta function realizes the duality determinant). *Uses:* def:weil-final-master-input-data, def:weil-final-rational-zeta-function, def:weil-duality-graded-determinant-function

Lean: Weil.Final.rationalZeta_map

For every λ, coefficient extension $\mathbf{Q}(T) \to E_\lambda(T)$ sends

$$Z_X^{\mathrm{rat}}(T) \longmapsto Z_{F_\lambda}(T) = \prod_{i=0}^{2d} P_{G_\lambda,i}(T)^{(-1)^{i+1}}.$$

Proof. Uses: def:weil-final-rational-zeta-function,def:weil-duality-graded-determinant-function,lemma:weil-final-common-factor-realization

The coefficient homomorphism preserves finite products and sends inverses of nonzero factors to inverses. Apply the common-factor realization lemma in each degree and use the stored equality $B = 2d$. The result is the defining product for Z_{F_λ}. □

Proposition 9.4.16 (Cohomological factorization over $\mathbf{Q}$). *Uses:* def:weil-final-master-input-data, def:weil-final-scheme-zeta-series, def:weil-final-common-frobenius-factor, def:weil-final-rational-zeta-function

Lean: Weil.Final.zeta_cohomologicalFactorization

The Taylor expansion at zero of Z_X^{rat} is Z_X, and

$$Z_X(T) = \prod_{i=0}^{2d} P_i(T)^{(-1)^{i+1}} \quad \text{in } \mathbf{Q}[[T]].$$

Proof. Uses: def:weil-final-cohomological-core-predicate,def:weil-final-rational-zeta-function,lemma:weil-final-common-factor-realization,cor:weil-zeta-factorization-after-coefficient-extension

Map both formal series coefficientwise to the base field E_{λ_*}. The core compatibility field identifies the trace datum's point interface with the one defining Z_X. The cited coefficient-extension factorization computes the image of Z_X as the alternating product of $P_{G_{\lambda_*},i}$. Degreewise common-factor realization computes the image of the right side as the same finite product. The field embedding $\mathbf{Q} \hookrightarrow E_{\lambda_*}$ is injective; coefficient extensionality therefore reflects the formal-series equality. Unfolding the rational-function Definition identifies that series with its normalized Taylor expansion. □

Corollary 9.4.17 (Rationality over $\mathbf{Q}$). *Uses:* def:weil-final-master-input-data, def:weil-final-scheme-zeta-series, def:weil-final-rational-zeta-function

Lean: Weil.Final.zeta_rational

The point-counting series $Z_X(T)$ is the expansion of the rational function $Z_X^{\mathrm{rat}}(T) \in \mathbf{Q}(T)$.

Proof. Uses: prop:weil-final-zeta-cohomological-factorization

This is the first assertion of the cohomological-factorization proposition. □

9.5 Functional equation and the two root variables

Lemma 9.5.1 (Normalization of the functional-equation exponent). *Uses:* def:weil-final-master-input-data, def:weil-final-euler-characteristic, def:weil-final-half-functional-equation-exponent

Lean: Weil.Final.two_mul_halfExponent

One has

$$2h = d\chi.$$

Proof. Uses: def:weil-final-euler-characteristic,def:weil-final-half-functional-equation-exponent,lemma:weil-family-common-half-exponent-realization

Unfold the two final aliases and apply the common half-exponent equality. □

Lemma 9.5.2 (The final sign is a sign). *Uses:* *def:weil-final-master-input-data, def:weil-final-functional-equation-sign*

Lean: *Weil.Final.functionalEquationSign_eq_one_or_neg_one*

One has $\varepsilon = 1$ or $\varepsilon = -1$.

Proof. Uses: def:weil-final-functional-equation-sign, lemma:weil-family-common-functional-equation-sign-dichotomy

Unfold the final sign alias and apply the common-sign dichotomy. □

Proposition 9.5.3 (Functional equation with geometric Frobenius). *Uses:* *def:weil-final-master-input-data, def:weil-final-rational-zeta-function, def:weil-final-euler-characteristic, def:weil-final-half-functional-equation-exponent, def:weil-final-functional-equation-sign*

Lean: *Weil.Final.zeta_functionalEquation*

In $\mathbf{Q}(T)$,

$$Z_X^{\mathrm{rat}}(q^{-d}T^{-1}) = \varepsilon q^{h} T^{\chi} Z_X^{\mathrm{rat}}(T), \qquad \varepsilon \in \{1,-1\}, \qquad 2h = d\chi.$$

Equivalently, the power of q is $d\chi/2$.

Proof. Uses: lemma:weil-final-rational-zeta-realization, lemma:weil-final-functional-equation-exponent, lemma:weil-final-functional-equation-sign-dichotomy, lemma:weil-family-common-euler-characteristic-realization, lemma:weil-family-common-half-exponent-realization, lemma:weil-family-common-sign-agrees-with-duality-candidate, prop:weil-duality-weil-normalized-functional-equation

Map the desired equality to $E_{\lambda_*}(T)$. The rational-zeta realization lemma identifies both zeta functions with $Z_{F_{\lambda_*}}$. The three common-realization lemmas identify χ, h, ε with the corresponding duality-system constants. Apply the module04 normalized functional equation. Injectivity of $\mathbf{Q}(T) \to E_{\lambda_*}(T)$ reflects the equality. The last two assertions are the two preceding lemmas. □

Definition 9.5.4 (Common reciprocal-root datum). Uses: def:weil-final-master-input-data, def:weil-final-common-frobenius-factor

Lean: Weil.Final.CommonReciprocalRootData

For $0 \le i \le 2d$, a common reciprocal-root datum consists of a nonzero algebraic scalar $\alpha \in \overline{\mathbf{Q}}^{\times}$ and the equation

$$P_i(\alpha^{-1}) = 0$$

after coefficient extension to $\overline{\mathbf{Q}}$.

Definition 9.5.5 (Common ordinary-root datum). Uses: def:weil-final-master-input-data, def:weil-final-common-frobenius-factor

Lean: Weil.Final.CommonOrdinaryRootData

For $0 \le i \le 2d$, a common ordinary-root datum consists of $\beta \in \overline{\mathbf{Q}}$ and the equation

$$P_i(\beta) = 0.$$

Definition 9.5.6 (Reciprocal scalar attached to an ordinary root). Uses: def:weil-final-common-ordinary-root-data

Lean: Weil.Final.CommonOrdinaryRootData.reciprocal

Define

$$\alpha_\beta := \beta^{-1} \in \overline{\mathbf{Q}}.$$

Lemma 9.5.7 (An ordinary root is nonzero). *Uses:* *def:weil-final-common-ordinary-root-data*

Lean: *Weil.Final.CommonOrdinaryRootData.ne_zero*

Every common ordinary root β is nonzero.

Proof. Uses: prop:weil-final-factor-integrality

If $\beta = 0$, the root equation reads $P_i(0) = 0$, whereas factor normalization gives $P_i(0) = 1$. This contradiction proves nonvanishing. □

Lemma 9.5.8 (The reciprocal of an ordinary root is a reciprocal root). *Uses:* *def:weil-final-common-ordinary-root-data, def:weil-final-ordinary-root-reciprocal-scalar, def:weil-final-common-reciprocal-root-data*

Lean: *Weil.Final.CommonOrdinaryRootData.reciprocal_isRoot*

The scalar α_β with the nonzero proof from the preceding lemma defines a common reciprocal-root datum.

Proof. Uses: def:weil-final-common-ordinary-root-data,def:weil-final-ordinary-root-reciprocal-scalar,def:weil-final-common-reciprocal-root-data,lemma:weil-final-ordinary-root-nonzero

Nonvanishing makes inversion involutive, so $(\alpha_\beta)^{-1} = \beta$. Substitute this equality into the stored ordinary-root equation $P_i(\beta) = 0$. □

Lemma 9.5.9 (Reciprocal roots are algebraic integers). Uses: def:weil-final-master-input-data,def:weil-final-common-reciprocal-root-data

Lean: Weil.Final.reciprocalRoot_isIntegral

Every common reciprocal root α is an algebraic integer.

Proof. Uses: lemma:weil-integral-factor-realization-independent,cor:weil-frobenius-roots-algebraic-integers

Use integral-factor independence at the base realization to identify P_i with the integral factor attached to $\mathcal{R}_{\lambda_*}$. The stored reciprocal-root equation therefore has the form required by the algebraic-integrality corollary; apply it to α. □

Lemma 9.5.10 (Classical weight of a common reciprocal root). Uses: def:weil-final-master-input-data,def:weil-final-common-reciprocal-root-data

Lean: Weil.Final.reciprocalRoot_classicalWeight

For every embedding $\sigma : \mathbf{Q}(\alpha) \hookrightarrow \mathbf{C}$,

$$|\sigma(\alpha)| = q^{i/2}.$$

Proof. Uses: lemma:weil-rational-factor-realization-independent,cor:weil-individual-factor-classical-weight

At the base realization, rational-factor independence identifies the rational image of P_i with the factor in the recovery input. The reciprocal-root equation transports along that equality. Apply the classical all-embeddings weight corollary in degree i. □

Definition 9.5.11 (Base-realization eigenvalue occurrence attached to a reciprocal root). Uses: def:weil-final-master-input-data,def:weil-final-common-reciprocal-root-data,def:weil-based-scalar-extension-datum,def:weil-based-scalar-extended-eigenspace

Lean: Weil.Final.ReciprocalRootEigenvalueOccurrence

Map α to $\overline{E}_{\lambda_*}$ through the realization field comparison stored in the compatible family. Define its occurrence to be the scalar-extended eigenspace

$$E_\alpha((F_{\lambda_*,i})_{\overline{E}_{\lambda_*}}).$$

Lemma 9.5.12 (A common reciprocal root occurs as a Frobenius eigenvalue). Uses: def:weil-final-master-input-data,def:weil-final-common-reciprocal-root-data,def:weil-final-reciprocal-root-eigenvalue-occurrence

Lean: Weil.Final.reciprocalRoot_isEigenvalue

The eigenspace in the preceding Definition is nonzero.

Proof. Uses: lemma:weil-final-common-factor-realization,thm:weil-root-reciprocal-conversion-after-scalar-extension

The common-factor realization equality sends the equation $P_i(\alpha^{-1}) = 0$ to the base realization factor. Apply the module03 root–eigenvalue theorem with ordinary root α^{-1}. Its conclusion is nonvanishing of the scalar-extended eigenspace with eigenvalue $(\alpha^{-1})^{-1} = \alpha$. □

Lemma 9.5.13 (Universal modulus of a cohomological eigenvalue). Uses: def:weil-final-master-input-data,def:weil-final-reciprocal-root-eigenvalue-occurrence

Lean: Weil.Final.eigenvalue_universalModulus

For every field isomorphism $\jmath : \overline{E}_{\lambda_*} \xrightarrow{\sim} \mathbf{C}$,

$$|\jmath(\alpha)| = q^{i/2}.$$

Proof. Uses: def:weil-final-cohomological-core-predicate,lemma:weil-final-reciprocal-root-is-eigenvalue,cor:weil-weight-smooth-proper-eigenvalue-modulus

The core compatibility field identifies the base graded family with the ordinary family in $\mathcal{W}_{\lambda_*}$. The preceding lemma gives the required nonzero eigenspace. Apply the smooth-proper eigenvalue-modulus corollary for $\mathcal{W}_{\lambda_*}$, degree i, and $\jmath$. □

Lemma 9.5.14 (Modulus of an ordinary root). *Uses:* *def:weil-final-master-input-data*, *def:weil-final-common-ordinary-root-data*

Lean: *Weil.Final.ordinaryRoot_modulus*

For every embedding $\sigma : \mathbf{Q}(\beta) \hookrightarrow \mathbf{C}$,

$$|\sigma(\beta)| = q^{-i/2}.$$

Proof. Uses: def:weil-final-ordinary-root-reciprocal-scalar, lemma:weil-final-ordinary-root-nonzero, lemma:weil-final-ordinary-root-reciprocal-is-root, lemma:weil-final-reciprocal-root-classical-weight

Put $\alpha_\beta = \beta^{-1}$. Since inversion stays inside $\mathbf{Q}(\beta)$, the same embedding σ evaluates both β and α_β. The reciprocal-root weight lemma gives $|\sigma(\beta^{-1})| = q^{i/2}$. Since $\beta \neq 0$, taking positive real inverses gives $|\sigma(\beta)| = q^{-i/2}$. □

Proposition 9.5.15 (Riemann hypothesis for the common factors). *Uses:* *def:weil-final-master-input-data*, *def:weil-final-common-reciprocal-root-data*, *def:weil-final-common-ordinary-root-data*

Lean: *Weil.Final.riemannHypothesis*

Every reciprocal root α of P_i is an algebraic integer and has absolute value $q^{i/2}$ under every complex embedding of its number field. Every ordinary root β of $P_i(T)$ has absolute value $q^{-i/2}$. Under coefficient extension, α occurs as a geometric Frobenius eigenvalue and $\beta = \alpha^{-1}$.

Proof. Uses: lemma:weil-final-reciprocal-root-algebraic-integer, lemma:weil-final-reciprocal-root-classical-weight, lemma:weil-final-reciprocal-root-is-eigenvalue, lemma:weil-final-ordinary-root-reciprocal-is-root, lemma:weil-final-ordinary-root-modulus

The algebraic-integrality, reciprocal-root modulus, eigenvalue occurrence, and ordinary-root modulus are the four cited lemmas. The equality between the two variables is the Definition of α_β together with inversion involutivity from the ordinary-root nonzero lemma. □

9.6 Optional characteristic-zero lift and terminal assembly

Definition 9.6.1 (Characteristic-zero lift enhancement). Uses: def:weil-final-master-input-data, def:weil-lift-comparison-interface

Lean: Weil.Final.LiftEnhancementData

A lift enhancement of $\mathcal{M}$ is a supplied Definition 8.10.22 whose special fiber, compatible realization family, and common Betti numbers are identified with those of $\mathcal{M}$. It is additional input; this Definition does not assert that X_0 lifts to characteristic zero.

Theorem 9.6.2 (Singular Betti interpretation for a supplied lift). *Uses:* *def:weil-final-lift-enhancement-data*, *def:weil-final-common-betti-number*, *def:weil-singular-betti-number*

Lean: *Weil.Final.commonBetti_eq_singular*

For every lift enhancement and every $0 \le i \le 2d$,

$$b_i = \dim_{\mathbf{Q}} H^i_{\mathrm{sing}}(X_{\mathbf{C}}(\mathbf{C}), \mathbf{Q}) = b_i^{\mathrm{sing}}(X_{\mathbf{C}}).$$

Proof. Uses: def:weil-final-common-betti-number, def:weil-final-lift-enhancement-data, thm:weil-betti-comparison-for-lift

Project the supplied lift-comparison interface and apply the module05 Betti comparison theorem. The enhancement identifications rewrite its common Betti number and complex fiber as the objects in the statement. □

Theorem 9.6.3 (Weil conjectures: conditional master-data form). *Uses:* *def:weil-final-master-input-data*, *def:weil-final-scheme-point-number*, *def:weil-final-scheme-zeta-series*, *def:weil-final-scheme-closed-point-type*, *def:weil-final-scheme-closed-euler-product*, *def:weil-final-common-frobenius-factor*, *def:weil-final-common-betti-number*, *def:weil-final-rational-zeta-function*, *def:weil-final-euler-characteristic*, *def:weil-final-half-functional-equation-exponent*, *def:weil-final-functional-equation-sign*, *def:weil-final-lift-enhancement-data*

Lean: *Weil.Final.weilConjectures*

Constructed by: *def:weil-final-common-frobenius-factor*

For every final Weil master input datum attached to a smooth, projective, geometrically connected $X_0/\mathbf{F}_q$ of pure dimension d, all of the following hold.

(i)
$$Z_X(T) = \operatorname{Exp}\left(\sum_{n\geq 1} \#X_0(\mathbf{F}_{q^n})\frac{T^n}{n}\right) = \prod_{x\in \mathrm{ClPt}(X_0)}^{\mathrm{lf}} (1-T^{\deg x})^{-1}.$$

(ii) *There are already defined polynomials* $P_i \in \mathbf{Z}[T]$, $0 \leq i \leq 2d$, *with* $P_i(0) = 1$, *such that*
$$Z_X(T) = \prod_{i=0}^{2d} P_i(T)^{(-1)^{i+1}}, \qquad Z_X^{\mathrm{rat}} \in \mathbf{Q}(T).$$
Their images are $\det(1 - TF_{\mathrm{geom}} \mid H^i(X, E_\lambda))$ *for every realization.*

(iii)
$$\deg P_i = b_i = \dim_{E_\lambda} H^i(X, E_\lambda), \qquad b_i = b_{2d-i},$$
and these numbers are independent of ℓ_λ, *with zero value outside* $[0, 2d]$.

(iv)
$$P_0 = 1 - T, \quad b_0 = 1, \qquad P_{2d} = 1 - q^d T, \quad b_{2d} = 1.$$

(v) *Every reciprocal root in degree* i *is an algebraic integer and has absolute value* $q^{i/2}$ *under every complex embedding; every ordinary root has absolute value* $q^{-i/2}$.

(vi)
$$Z_X^{\mathrm{rat}}(q^{-d}T^{-1}) = \varepsilon q^h T^\chi Z_X^{\mathrm{rat}}(T), \qquad \varepsilon \in \{1, -1\}, \qquad 2h = d\chi.$$

(vii) *For every supplied characteristic-zero lift enhancement,*
$$b_i = \dim_{\mathbf{Q}} H^i_{\mathrm{sing}}(X_{\mathbf{C}}(\mathbf{C}), \mathbf{Q}).$$

Proof. Uses: `lemma:weil-final-scheme-point-number-cardinality`, `lemma:weil-final-zeta-exponential-formula`, `prop:weil-final-zeta-euler-product`, `cor:weil-final-zeta-integral-coefficients`, `prop:weil-final-factor-integrality`, `lemma:weil-final-common-factor-realization`, `prop:weil-final-factor-degree-and-betti-independence`, `lemma:weil-final-common-betti-symmetry`, `lemma:weil-final-common-bottom-factor`, `lemma:weil-final-common-bottom-betti`, `lemma:weil-final-common-top-factor`, `lemma:weil-final-common-top-betti`, `prop:weil-final-zeta-cohomological-factorization`, `cor:weil-final-zeta-rationality`, `prop:weil-final-zeta-functional-equation`, `prop:weil-final-riemann-hypothesis`, `thm:weil-final-singular-betti-interpretation`

For (i), rewrite the exponential formula with the point-cardinality lemma and then apply the Euler-product proposition; integral coefficients are the cited corollary. For (ii), use factor integrality, common-factor realization, cohomological factorization, and rationality. For (iii), use the degree–Betti proposition and Betti symmetry. The four assertions in (iv) are the four endpoint lemmas. Assertion (v) is the Riemann-hypothesis proposition. Assertion (vi) is the normalized functional equation, including its sign and exponent conclusions. For (vii), instantiate the supplied-lift theorem with the enhancement. Each clause is therefore one direct specialization of a preceding result. □

9.7 Finite torsion constructible derived categories

Status. This section treats only a finite commutative torsion ring Λ and the small etale and pro-etale sites of a noetherian scheme. It does not form $\mathbf{Z}_\ell$, $\mathbf{Q}_\ell$, or $\overline{\mathbf{Q}}_\ell$ coefficients, and it does not construct $Rf_!$, duality, or a trace map. Published references in the source index are verification anchors; the closure arguments needed here are decomposed below and are not imported as black-box mathlib theorems.

9.7.1 Universes, sites, and coefficient sheaves

Definition 9.7.1 (Universe convention)**.** Uses:

Lean: `conditional-interface:def:weil-ftdc-universe-convention`

Fix Grothendieck universes $\mathcal{U} \in \mathcal{V}$. The scheme X, its affine coordinate rings, the finite ring Λ, and the selected site objects belong to $\mathcal{U}$. Site categories are chosen $\mathcal{U}$-small, sheaves take values in $\mathcal{U}$-small Λ-modules, and categories of such sheaves and their localizations are regarded as $\mathcal{V}$-categories. Every Hom is a $\mathcal{V}$-set. Replacing a site by its selected small skeleton is part of this convention, so no proper-class localization occurs below.

Definition 9.7.2 (Finite torsion coefficient datum). Lean: `conditional-interface:def:weil-ftdc-finite-torsion-coefficient`

Uses: `def:weil-ftdc-universe-convention`

Fix a noetherian $\mathcal{U}$-scheme X, a finite commutative ring Λ, and an integer $N > 0$ such that $N\Lambda = 0$. Require that the image of N in every local ring $\mathcal{O}_{X,x}$ is a unit, equivalently that $X \to \operatorname{Spec} \mathbf{Z}$ factors through $\operatorname{Spec} \mathbf{Z}[1/N]$. We say that Λ is invertible on X. The categorical constructions below use finiteness (hence noetherianity) of Λ; invertibility is retained for the later etale cohomological operations.

Lemma 9.7.3 (The coefficient ring is noetherian). Lean: *conditional-interface:lemma:weil-ftdc-coefficient-noetherian*

Uses: *def:weil-ftdc-finite-torsion-coefficient*

Every ideal and every submodule of a finite Λ-module is finite and finitely generated. In particular, Λ is noetherian.

Proof. Uses:

Every ascending chain of subsets of a finite set stabilizes. Apply this to ideals of Λ and submodules of a finite module. A maximal member in the finite set of finitely generated submodules must be the whole module. □

Definition 9.7.4 (Selected small etale site). Lean: `conditional-interface:def:weil-ftdc-small-etale-site`

Uses: `def:weil-ftdc-universe-convention`

Let $X^{\mathcal{U}}_{\mathrm{et}}$ be the selected $\mathcal{U}$-small skeleton of etale X-schemes in $\mathcal{U}$. It is closed under finite fiber products and contains X. Coverings are jointly surjective families of etale morphisms. Write $\tau = \mathrm{et}$ for this topology.

Definition 9.7.5 (Selected small pro-etale site). Lean: `conditional-interface:def:weil-ftdc-small-proetale-site`

Uses: `def:weil-ftdc-universe-convention`

Let $X^{\mathcal{U}}_{\mathrm{proet}}$ be the selected $\mathcal{U}$-small skeleton of weakly etale X-schemes in $\mathcal{U}$, closed under finite fiber products and containing X. A family $\{U_i \to U\}$ is a covering when it is weakly etale and, for every affine open $V \subset U$, finitely many affine opens in the U_i have images covering V. Write $\tau = \mathrm{proet}$ for this topology.

Lemma 9.7.6 (Finite site limits and subcanonicity). Lean: *conditional-interface:lemma:weil-ftdc-sites-finite-limits-subcanonical*

Uses: *def:weil-ftdc-small-etale-site, def:weil-ftdc-small-proetale-site*

Both selected sites have a final object and finite fiber products, their coverings are stable under base change and composition, and representable presheaves are sheaves.

Proof. Uses:

The final object is X. Etale and weakly etale morphisms are stable under composition and base change. Joint surjectivity and the affine finite subcover condition are preserved by these operations. Etale descent and fpqc descent for morphisms show subcanonicity; a pro-etale covering is fpqc with the displayed local finite-subcover condition. □

Definition 9.7.7 (Finite constant coefficient sheaf). Lean: `conditional-interface:def:weil-ftdc-coefficient-sheaf`

Uses: `def:weil-ftdc-finite-torsion-coefficient, def:weil-ftdc-small-etale-site, def:weil-ftdc-small-proetale-site`

For $\tau \in \{\mathrm{et}, \mathrm{proet}\}$, let $\underline{\Lambda}_{X,\tau}$ be the sheafification of the constant Λ-presheaf on $X^{\mathcal{U}}_{\tau}$. Thus its sections are locally constant maps to the finite discrete ring Λ; on a pro-etale object U these are the continuous maps $|U| \to \Lambda$.

Definition 9.7.8 (Ambient module-sheaf category). Lean: `conditional-interface:def:weil-ftdc-module-sheaves`

Uses: `def:weil-ftdc-coefficient-sheaf`

Define

$$\mathsf{A}_\tau(X, \Lambda) := \mathrm{Mod}(X^{\mathcal{U}}_{\tau}, \underline{\Lambda}_{X,\tau}) = \texttt{SheafOfModules}(\underline{\Lambda}_{X,\tau}).$$

Morphisms are $\underline{\Lambda}_{X,\tau}$-linear sheaf morphisms.

Theorem 9.7.9 (The ambient module-sheaf category is abelian). Lean: *conditional-interface:thm:weil-ftdc-ambient-abelian*

Uses: *def:weil-ftdc-module-sheaves*

For either τ, $\mathsf{A}_\tau(X, \Lambda)$ is an abelian category. Kernels are computed sectionwise and cokernels are obtained by module-sheafification of the sectionwise cokernel.

Proof. Uses: `lemma:weil-ftdc-sites-finite-limits-subcanonical`

The exact pinned mathlib leaf is `SheafOfModules.instAbelian`; its hypotheses are `HasSheafify` and `WEqualsLocallyBijective` for the selected small site. The generic small-site instances provide these hypotheses. No Grothendieck-category, injective-resolution, or constructibility claim is supplied by this leaf. □

9.7.2 Constructible sheaves and elementary closure

Definition 9.7.10 (Finite locally constant module sheaf). Lean: `conditional-interface:def:weil-ftdc-finite-locally-constant`

Uses: `def:weil-ftdc-module-sheaves`

A sheaf $F \in \mathsf{A}_\tau(X, \Lambda)$ is finite locally constant if there is a covering $\{U_a \to X\}$ and finite Λ-modules M_a with $F|_{U_a} \simeq \underline{M_a}$.

Lemma 9.7.11 (A finite diagram has a common trivializing cover). Lean: `conditional-interface:lemma:weil-ftdc-local-systems-common-cover`

Uses: `def:weil-ftdc-finite-locally-constant`

For a finite diagram of finite locally constant sheaves, there is one cover on which every object is constant and every arrow is induced by a homomorphism of finite Λ-modules.

Proof. Uses:

Take the finite fiber-product refinement of covers trivializing all objects. On this refinement an arrow is a section of the finite locally constant sheaf $\underline{\mathrm{Hom}_\Lambda(M, M')}$. Refine once more by the inverse images of its finitely many values. Finiteness of the diagram keeps the resulting refinement a cover in the selected universe. □

Lemma 9.7.12 (Local systems are closed under kernels and cokernels). Lean: `conditional-interface:lemma:weil-ftdc-local-systems-kernel-cokernel`

Uses:

If $u : F \to G$ is a morphism of finite locally constant sheaves, then $\ker u$ *and* $\operatorname{coker} u$ *are finite locally constant.*

Proof. Uses: `lemma:weil-ftdc-local-systems-common-cover,lemma:weil-ftdc-coefficient-noetherian`

On the common cover, u is a fixed homomorphism $M \to M'$ of finite Λ-modules. Exact inverse image identifies the two sheaf operations with the constant sheaves attached to its kernel and cokernel. Those modules are finite, so the local descriptions descend to the assertion. □

Lemma 9.7.13 (Local systems are closed under extensions). Lean: `conditional-interface:lemma:weil-ftdc-local-systems-extension`

Uses: `def:weil-ftdc-finite-locally-constant`

In an exact sequence $0 \to F \to E \to G \to 0$, if F and G are finite locally constant, then E is finite locally constant.

Proof. Uses: `lemma:weil-ftdc-coefficient-noetherian`

Work on a cover where $F = \underline{M}$ and $G = \underline{Q}$. Choose a finite presentation $0 \to R \to \Lambda^m \to Q \to 0$. After a further cover, the images of the standard generators of Q lift to sections of E. The extension is then the pushout of constant sheaves along a map $\underline{R} \to \underline{M}$. Refine once more so that this map is a constant module homomorphism. The pushout is the constant sheaf attached to the corresponding finite module extension. The module R is finite by noetherianity, so every object in the construction is finite. □

Definition 9.7.14 (Finite constructible stratification). Lean: `conditional-interface:def:weil-ftdc-finite-stratification`

Uses: `def:weil-ftdc-finite-torsion-coefficient`

A finite constructible stratification of X is a finite partition $X = \coprod_{a \in A} X_a$ by reduced locally closed subschemes whose underlying subsets are constructible. Restrictions to X_a are taken on the site of X_a of the same type τ.

Lemma 9.7.15 (Common refinement of finitely many stratifications). Lean: `conditional-interface:lemma:weil-ftdc-stratification-common-refinement`

Uses: `def:weil-ftdc-finite-stratification`

Finitely many finite constructible stratifications admit a finite common refinement by nonempty intersections of their strata.

Proof. Uses:
Finite intersections of constructible locally closed subsets are constructible. Intersect one stratum from each partition, discard empty intersections, and give each intersection its reduced locally closed subscheme structure. The resulting finite family is disjoint and covers X. □

Definition 9.7.16 (Constructible finite-torsion sheaf)**.** Lean: `conditional-interface:def:weil-ftdc-constructible-sheaf`

Uses: `def:weil-ftdc-finite-locally-constant,def:weil-ftdc-finite-stratification`
A sheaf $F \in \mathsf{A}_\tau(X,\Lambda)$ is constructible if some finite constructible stratification $X = \coprod X_a$ makes every $F|_{X_a}$ finite locally constant. Let $\mathsf{Cons}_\tau(X,\Lambda) \subset \mathsf{A}_\tau(X,\Lambda)$ be the full subcategory on constructible sheaves.

Lemma 9.7.17 (Constructible kernels)**.** Lean: `conditional-interface:lemma:weil-ftdc-constructible-kernel`

Uses: `def:weil-ftdc-constructible-sheaf`
The kernel in A_τ of a morphism between constructible sheaves is constructible.

Proof. Uses: `lemma:weil-ftdc-stratification-common-refinement,lemma:weil-ftdc-local-systems-kernel-cokernel`
Pass to a common refinement of stratifications for source and target. Exact restriction identifies the restricted kernel with the kernel of the restricted morphism. Apply local-system kernel closure on every stratum. □

Lemma 9.7.18 (Constructible cokernels)**.** Lean: `conditional-interface:lemma:weil-ftdc-constructible-cokernel`

Uses: `def:weil-ftdc-constructible-sheaf`
The cokernel in A_τ of a morphism between constructible sheaves is constructible.

Proof. Uses: `lemma:weil-ftdc-stratification-common-refinement,lemma:weil-ftdc-local-systems-kernel-cokernel`
Use the same common refinement. Restriction is exact, and local-system cokernel closure applies on each stratum. □

Lemma 9.7.19 (Constructible extensions)**.** Lean: `conditional-interface:lemma:weil-ftdc-constructible-extension`

Uses: `def:weil-ftdc-constructible-sheaf`
If $0 \to F \to E \to G \to 0$ is exact in A_τ and F, G are constructible, then E is constructible.

Proof. Uses: `lemma:weil-ftdc-stratification-common-refinement,lemma:weil-ftdc-local-systems-extension`
Choose a common refinement for F and G. Restriction preserves the short exact sequence, and the middle term is finite locally constant on every stratum by the local extension lemma. □

Theorem 9.7.20 (The constructible heart is an abelian full subcategory)**.** Lean: `conditional-interface:thm:weil-ftdc-constructible-abelian`

Uses:
The full subcategory $\mathsf{Cons}_\tau(X,\Lambda)$ is abelian, and its inclusion in $\mathsf{A}_\tau(X,\Lambda)$ is exact and extension closed.

Proof. Uses: `thm:weil-ftdc-ambient-abelian,lemma:weil-ftdc-constructible-kernel,lemma:weil-ftdc-constructible-cokernel,lemma:weil-ftdc-constructible-extension`
The zero sheaf and finite biproducts are constructible. The kernel and cokernel lemmas compute both operations in the ambient abelian category. Thus the full subcategory is abelian and the inclusion is exact. The extension lemma is precisely extension closure. □

Lemma 9.7.21 (Finite limits and colimits in the constructible heart)**.** Lean: `conditional-interface:lemma:weil-ftdc-constructible-finite-limits-colimits`

Uses:
Every finite diagram in $\mathsf{Cons}_\tau(X,\Lambda)$ has its ambient finite limit and colimit in $\mathsf{Cons}_\tau(X,\Lambda)$.

Proof. Uses: `thm:weil-ftdc-constructible-abelian`
Finite products and coproducts are finite biproducts. Equalizers are kernels of differences and coequalizers are cokernels of differences. The abelian closure theorem therefore constructs all finite limits and colimits and shows that the inclusion preserves them. □

Lemma 9.7.22 (Underived tensor closure). Lean: `conditional-interface:lemma:weil-ftdc-constructible-tensor`

Uses: `def:weil-ftdc-constructible-sheaf`

For constructible sheaves F, G, the sheaf tensor product $F \otimes_{\underline{\Lambda}} G$ is constructible.

Proof. Uses: `lemma:weil-ftdc-stratification-common-refinement`

On a common refinement, trivialize F and G as finite modules M and N. Pullback commutes with tensor, and the restriction of the tensor is $\underline{M \otimes_{\Lambda} N}$, again finite locally constant. □

Definition 9.7.23 (Same-coefficient pullback). Lean: `conditional-interface:def:weil-ftdc-scheme-pullback`

Uses: `def:weil-ftdc-coefficient-sheaf`, `def:weil-ftdc-module-sheaves`

For a morphism $f : Y \to X$ of noetherian $\mathcal{U}$-schemes and either site, define

$$f_\tau^* F := f_\tau^{-1} F \otimes_{f_\tau^{-1}\underline{\Lambda}_{X,\tau}} \underline{\Lambda}_{Y,\tau}.$$

The canonical coefficient map is an isomorphism because the coefficient ring is the same finite constant ring, so $f_\tau^* = f_\tau^{-1}$.

Lemma 9.7.24 (Pullback is exact). Lean: `conditional-interface:lemma:weil-ftdc-pullback-exact`

Uses: `def:weil-ftdc-scheme-pullback`

The functor f_τ^* is exact and commutes with all finite limits and all colimits.

Proof. Uses:

The inverse-image functor of a geometric morphism preserves finite limits and all colimits. It is therefore exact on abelian sheaves. The displayed coefficient morphism is an isomorphism, so scalar extension adds no Tor term. □

Lemma 9.7.25 (Pullback preserves constructibility). Lean: `conditional-interface:lemma:weil-ftdc-pullback-constructible`

Uses: `def:weil-ftdc-constructible-sheaf`, `def:weil-ftdc-scheme-pullback`

For every $f : Y \to X$, the functor f_τ^* carries constructible sheaves to constructible sheaves.

Proof. Uses:

Pull back an adapted finite stratification of X. Base change of a constructible locally closed immersion is constructible locally closed, and the resulting nonempty inverse images form a finite partition of Y. Pullback of a finite locally constant sheaf is finite locally constant, with the same finite stalk module. □

9.7.3 Etale–pro-etale comparison at finite level

> **Status.** This subsection expands the finite-level etale–pro-etale comparison at the same granularity as the surrounding blueprint. The section formula for inverse image, finite-lisse descent, open and closed immersion comparison maps, stratum induction, and bounded derived-amplitude comparison are written as local proof leaves below rather than as external status capsules.

Definition 9.7.26 (Comparison geometric morphism). Lean: `conditional-interface:def:weil-ftdc-etale-proetale-morphism`

Uses: `def:weil-ftdc-small-etale-site`, `def:weil-ftdc-small-proetale-site`

The inclusion sending an etale X-scheme to the same weakly etale X-scheme induces

$$\epsilon_X : \mathrm{Sh}(X^{\mathcal{U}}_{\mathrm{proet}}) \longrightarrow \mathrm{Sh}(X^{\mathcal{U}}_{\mathrm{et}}).$$

Write ϵ_X^{-1} for its exact inverse-image functor.

Lemma 9.7.27 (Fpqc descent for pulled-back etale sections). Lean: `conditional-interface:lemma:weil-ftdc-fpqc-section-descent`

Uses: `def:weil-ftdc-etale-proetale-morphism`

Let F be a sheaf of sets on X_{et}, let $f : Y \to X$ be an X-scheme, and put

$$A_F(Y) = \Gamma(Y_{\mathrm{et}}, f_{\mathrm{et}}^{-1} F).$$

For every fpqc covering $(Y_i \to Y)_{i\in I}$ the sequence

$$A_F(Y) \longrightarrow \prod_i A_F(Y_i) \rightrightarrows \prod_{i,j} A_F(Y_i \times_Y Y_j)$$

is an equalizer.

Proof. Uses: `def:weil-ftdc-etale-proetale-morphism`

Apply the fpqc descent theorem for sections of the inverse image of an etale sheaf on the big etale site: for $\tau = \mathrm{et}$, the presheaf

$$T \longmapsto \Gamma(T_{\mathrm{et}}, g_{\mathrm{et}}^{-1}F)$$

attached to a sheaf on the small etale site satisfies the sheaf condition for fpqc coverings. In the Stacks Project this is Etale Cohomology, Lemma 59.39.2 (Tag 09XN). Its proof constructs, for any comparison section, the largest agreement open; Descent, Lemma 35.13.6 descends those opens from the fpqc cover; the resulting etale covering of Y then applies the ordinary sheaf equalizer axiom for F. The uniqueness part of the same equalizer gives separatedness, and the surjectivity part gives gluing. □

Proposition 9.7.28 (Section formula for etale inverse image). Lean: `conditional-interface:prop:weil-ftdc-epsilon-section-formula`

Uses: `def:weil-ftdc-etale-proetale-morphism`

For every sheaf F on X_{et}, the inverse image $\epsilon_X^{-1}F$ is the pro-etale sheaf

$$(Y \xrightarrow{f} X) \longmapsto \Gamma(Y_{\mathrm{et}}, f_{\mathrm{et}}^{-1}F).$$

On an etale object $U \to X$, this formula identifies $(\epsilon_X^{-1}F)(U)$ with $F(U)$.

Proof. Uses: `lemma:weil-ftdc-fpqc-section-descent`

The previous lemma proves that the displayed rule is already a sheaf for the pro-etale topology, since every pro-etale covering is fpqc. Denote this sheaf by $a(F)$. For a pro-etale sheaf G, a morphism $a(F) \to G$ restricts on etale objects to a morphism $F \to \epsilon_{X,*}G$. Conversely, a map $F \to \epsilon_{X,*}G$ pulls back to each Y_{et} and then evaluates on sections, producing a map $a(F)(Y) \to G(Y)$ compatible with pro-etale restriction. The two constructions are inverse by the identity and composition laws for inverse image. Thus a is left adjoint to $\epsilon_{X,*}$, hence $a = \epsilon_X^{-1}$. This is the proof of Stacks Project, Lemma 61.19.1 (Tag 0GLZ), with the fpqc sheaf step made explicit. If U is etale over X, then $U_{\mathrm{et}} \to X_{\mathrm{et}}$ is the restriction morphism and

$$\Gamma(U_{\mathrm{et}}, F|_U) = F(U),$$

so the displayed formula gives the stated identification. □

Lemma 9.7.29 (Full faithfulness on etale sheaves). Lean: `conditional-interface:lemma:weil-ftdc-epsilon-fully-faithful`

Uses: `def:weil-ftdc-etale-proetale-morphism`

The functor ϵ_X^{-1} is fully faithful on sheaves coming from the etale site, and its unit $F \to \epsilon_{X,*}\epsilon_X^{-1}F$ is an isomorphism.

Proof. Uses: `prop:weil-ftdc-epsilon-section-formula`

Evaluate the unit on an etale object $U \to X$. By Proposition 9.7.28,

$$(\epsilon_{X,*}\epsilon_X^{-1}F)(U) = (\epsilon_X^{-1}F)(U) = \Gamma(U_{\mathrm{et}}, F|_U) = F(U).$$

Under this identification the unit is the identity restriction map. Hence the unit is an isomorphism of etale sheaves. For sheaves F, G on X_{et}, the adjunction bijection gives

$$\mathrm{Hom}_{X_{\mathrm{proet}}}(\epsilon_X^{-1}F, \epsilon_X^{-1}G) \cong \mathrm{Hom}_{X_{\mathrm{et}}}(F, \epsilon_{X,*}\epsilon_X^{-1}G),$$

and the unit isomorphism for G identifies the right side with $\mathrm{Hom}_{X_{\mathrm{et}}}(F, G)$. The composite is the usual pullback map on morphisms, so ϵ_X^{-1} is fully faithful. This is Stacks Project, Lemma 61.19.2 (Tag 099T), expanded at the unit level. □

Theorem 9.7.30 (Finite-lisse descent). Lean: `conditional-interface:thm:weil-ftdc-finite-lisse-descent`

Uses: `def:weil-ftdc-constructible-sheaf`

The functor ϵ_X^{-1} restricts to an equivalence between finite locally constant Λ-module sheaves on X_{et} and finite locally constant Λ-module sheaves on X_{proet}.

Proof. Uses: `lemma:weil-ftdc-epsilon-fully-faithful`

Because Λ is finite, finite locally constant Λ-module sheaves are locally constant sheaves of finite presentation over the Noetherian ring Λ. Stacks Project, Lemma 61.19.9 (Tag 099Y), gives an equivalence between locally constant finite-presentation sheaves on the etale and pro-etale sites. Its construction is the finite automorphism torsor descent argument: choose a pro-etale cover trivializing the pro-etale sheaf, group the covering members by the finite module M on which the restriction is constant, use fpqc descent of opens to obtain an etale open decomposition, and descend the finite $\mathrm{Aut}_\Lambda(M)$-torsor by equality of etale and pro-etale H^1 for pulled-back group sheaves. Full faithfulness on the etale side is the preceding lemma, so the descended etale sheaf is unique up to the unique isomorphism whose pullback gives the chosen pro-etale identification. □

Lemma 9.7.31 (Open-immersion extension comparison)**.** Lean: `conditional-interface: lemma: weil-ftdc-open-extension-comparison`

Uses: `def: weil-ftdc-etale-proetale-morphism`

For an etale open immersion $j : U \hookrightarrow X$ and an abelian sheaf G on U_{et}, the natural comparison

$$\epsilon_X^{-1} j_!^{\mathrm{et}} G \longrightarrow j_!^{\mathrm{proet}} \epsilon_U^{-1} G$$

is an isomorphism.

Proof. Uses: `lemma:weil-ftdc-epsilon-fully-faithful`

Both sides are left adjoint to the same restriction functor. Indeed, restriction to U commutes with ϵ_* because an object etale over U is equivalently an etale object over X whose structure map factors through U. Thus

$$j_{\mathrm{proet}}^{-1} \epsilon_{X,*} = \epsilon_{U,*} j_{\mathrm{et}}^{-1}.$$

The left adjoint of this functor is therefore both $\epsilon_X^{-1} j_!^{\mathrm{et}}$ and $j_!^{\mathrm{proet}} \epsilon_U^{-1}$. The comparison is the unique isomorphism between these two left adjoints compatible with the adjunction units. This is Stacks Project, Lemma 61.26.2 (Tag 09AF). □

Lemma 9.7.32 (Closed-immersion direct-image comparison)**.** Lean: `conditional-interface: lemma: weil-ftdc-closed-pushforward-comparison`

Uses: `def: weil-ftdc-etale-proetale-morphism`

For a finitely presented closed immersion $i : Z \hookrightarrow X$ and an abelian sheaf G on Z_{et}, the natural comparison

$$\epsilon_X^{-1} i_*^{\mathrm{et}} G \longrightarrow i_*^{\mathrm{proet}} \epsilon_Z^{-1} G$$

is an isomorphism.

Proof. Uses: `prop:weil-ftdc-epsilon-section-formula`

A finitely presented closed immersion is quasi-compact and quasi-separated. For any qcqs morphism f, Stacks Project, Lemma 61.23.1 (Tag 09A6), gives

$$f_{\mathrm{proet},*} \epsilon^{-1} G = \epsilon^{-1} f_{\mathrm{et},*} G$$

for sheaves of sets, and the same equality for abelian sheaves. Applying this to $f = i$ gives exactly the displayed comparison. Objectwise, after a Zariski localization on X, a pro-etale object is covered by affine inverse limits of etale objects; the section formula of Proposition 9.7.28 reduces both sides to the same filtered colimit of etale sections on $Z \times_X V_\alpha$. □

Theorem 9.7.33 (Equivalence of constructible hearts)**.** Lean: `conditional-interface: thm: weil-ftdc-constructible-site-equivalence`

Uses: `def: weil-ftdc-constructible-sheaf`

Inverse image restricts to an equivalence

$$\epsilon_X^{-1} : \mathsf{Cons}_{\mathrm{et}}(X, \Lambda) \xrightarrow{\sim} \mathsf{Cons}_{\mathrm{proet}}(X, \Lambda).$$

Proof. Uses: `lemma:weil-ftdc-epsilon-fully-faithful,thm:weil-ftdc-finite-lisse-descent,lemma:weil-ftdc-open-extension-comparison,lemma:weil-ftdc-closed-pushforward-comparison,lemma:weil-ftdc-constructible-extension`

It preserves finite locally constant sheaves and restrictions to strata, so it preserves constructibility. Full faithfulness on constructible sheaves is the full faithfulness of the unit on all etale sheaves proved above.

For essential surjectivity, use the induction of Stacks Project, Lemma 61.27.2 (Tag 09AI). Work Zariski locally on X, choose a finite constructible stratification of the pro-etale sheaf F, and induct on the number of strata. The one-stratum case is Theorem 9.7.30. For the induction step, choose a point of a stratum and shrink

Zariski locally so that this stratum is closed. Let $i : Z \hookrightarrow X$ be that closed stratum and $j : U \hookrightarrow X$ its open complement. The open–closed localization sequence is

$$0 \to j_! j^{-1} F \to F \to i_* i^{-1} F \to 0.$$

By induction the two outer terms are inverse images of constructible etale sheaves, say $j^{-1}F \simeq \epsilon_U^{-1} B$ and $i^{-1}F \simeq \epsilon_Z^{-1} A$. The open and closed comparison lemmas give

$$j_! j^{-1} F \simeq \epsilon_X^{-1} j_! B, \qquad i_* i^{-1} F \simeq \epsilon_X^{-1} i_* A.$$

Functoriality yields the commutative diagram obtained by applying $\epsilon_X^{-1}\epsilon_{X,*}$ to the localization sequence and comparing it with the original sequence. The two outer vertical arrows are isomorphisms by the displayed identifications and by the unit isomorphism for etale sheaves. The five lemma makes the middle vertical arrow an isomorphism. Thus F is locally in the essential image; the local etale preimages glue by full faithfulness, so F is globally in the essential image. □

Lemma 9.7.34 (The heart equivalence is exact, monoidal, and pullback natural)**.** Lean: `conditional-interface:lemma:weil-ftdc-heart-equivalence-properties`

Uses:

The equivalence preserves kernels, cokernels, extensions, finite limits/colimits, tensor products, and same-coefficient pullback. For every $f : Y \to X$ there is a natural equality up to the canonical site isomorphism

$$f^*_{\mathrm{proet}} \epsilon_X^{-1} \simeq \epsilon_Y^{-1} f^*_{\mathrm{et}}.$$

Proof. Uses: `thm:weil-ftdc-constructible-site-equivalence,lemma:weil-ftdc-pullback-exact,lemma:weil-ftdc-constructible-tensor,lemma:weil-ftdc-pullback-constructible`

Inverse image is exact and symmetric monoidal for the constant coefficient ring. Both composites in the displayed formula are sheafified pullback along the same base-change functor on objects. Their units and restrictions therefore agree, and all stated properties follow from exactness and the corresponding closure lemmas. □

9.7.4 Bounded derived localization and the standard t-structure

Definition 9.7.35 (Bounded cochain complexes)**.** Lean: `conditional-interface:def:weil-ftdc-bounded-complexes`

Uses: `def:weil-ftdc-module-sheaves`

Let $\mathsf{C}^b_\tau(X, \Lambda)$ be the category of cochain complexes in $\mathsf{A}_\tau(X, \Lambda)$ with only finitely many nonzero terms.

Definition 9.7.36 (Bounded homotopy category)**.** Lean: `conditional-interface:def:weil-ftdc-bounded-homotopy-category`

Uses: `def:weil-ftdc-bounded-complexes`

Let $\mathsf{K}^b_\tau(X, \Lambda)$ have the same objects as C^b_τ and morphisms equal to cochain maps modulo cochain homotopy. Shifts and mapping cones are the standard cochain constructions.

Definition 9.7.37 (Bounded quasi-isomorphisms)**.** Lean: `conditional-interface:def:weil-ftdc-quasi-isomorphism`

Uses: `def:weil-ftdc-bounded-homotopy-category`

A morphism $q : K \to L$ in K^b_τ is a quasi-isomorphism when $H^i(q)$ is an isomorphism in A_τ for every $i \in \mathbf{Z}$. Let W_τ denote this class.

Lemma 9.7.38 (The quasi-isomorphisms form a saturated triangulated system)**.** Lean: `conditional-interface:lemma:weil-ftdc-quasi-isomorphism-system`

Uses: `def:weil-ftdc-quasi-isomorphism`

The class W_τ contains identities, is closed under composition, has two-out-of-three, is invariant under shift, and a morphism belongs to W_τ exactly when its cone is acyclic.

Proof. Uses: `thm:weil-ftdc-ambient-abelian`

The first four assertions follow degreewise from the cohomology functors. The long exact cohomology sequence of the mapping-cone triangle identifies $H^i(q)$ as an isomorphism for all i exactly when every cohomology object of $\mathrm{Cone}(q)$ vanishes. □

Definition 9.7.39 (Bounded derived category)**.** Lean: `conditional-interface:def:weil-ftdc-bounded-derived-category`

Uses: `def:weil-ftdc-bounded-homotopy-category`, `def:weil-ftdc-quasi-isomorphism`

Define the Verdier localization

$$D^b_\tau(X,\Lambda) := \mathsf{K}^b_\tau(X,\Lambda)[W_\tau^{-1}],$$

and write $Q_\tau : \mathsf{K}^b_\tau \to D^b_\tau$ for its localization functor.

Theorem 9.7.40 (Existence and triangulated structure of the localization)**.** *Lean:* `conditional-interface:thm:weil-ftdc-derived-localization-exists`

Uses: `def:weil-ftdc-bounded-derived-category`

The localization is a $\mathcal{V}$-locally small triangulated category; Q_τ is exact, and the distinguished triangles are those isomorphic to images of mapping-cone triangles.

Proof. Uses: `lemma:weil-ftdc-quasi-isomorphism-system`, `thm:weil-ftdc-ambient-abelian`

Verdier localization of the triangulated category K^b_τ by the thick subcategory of acyclic complexes gives the stated category. The universe convention makes the roof collections $\mathcal{V}$-sets. At the pinned mathlib level, `HasDerivedCategory.standard`, `DerivedCategory.Qh`, and `DerivedCategory.mappingCone_triangle_distinguished` provide the generic ambient localization and cone theorem; boundedness is imposed as the full subcategory cut out by cohomological vanishing.

□

Definition 9.7.41 (Cohomology objects)**.** Lean: `conditional-interface:def:weil-ftdc-cohomology-objects`

Uses: `def:weil-ftdc-bounded-derived-category`

For $i \in \mathbf{Z}$, define $\mathcal{H}^i_\tau : D^b_\tau(X,\Lambda) \to \mathsf{A}_\tau(X,\Lambda)$ by $\mathcal{H}^i_\tau(Q_\tau K) = H^i(K)$. Quasi-isomorphism invariance makes this independent of the representative.

Definition 9.7.42 (Bounded constructible derived category)**.** Lean: `conditional-interface:def:weil-ftdc-constructible-derived-category`

Uses: `def:weil-ftdc-cohomology-objects`, `def:weil-ftdc-constructible-sheaf`

Define

$$D^b_{c,\tau}(X,\Lambda) := \{K \in D^b_\tau(X,\Lambda) : \mathcal{H}^i_\tau(K) \in \mathsf{Cons}_\tau(X,\Lambda) \text{ for all } i\}$$

as a strictly full subcategory, with the ambient morphisms. Boundedness is already part of D^b_τ.

Definition 9.7.43 (Restricted standard t-structure)**.** Lean: `conditional-interface:def:weil-ftdc-standard-t-structure`

Uses: `def:weil-ftdc-constructible-derived-category`

Set

$$D^{\le 0}_{c,\tau} := \{K : \mathcal{H}^i(K) = 0 \ (i > 0)\}, \qquad D^{\ge 0}_{c,\tau} := \{K : \mathcal{H}^i(K) = 0 \ (i < 0)\}.$$

The truncation candidates are the usual good truncations $\tau_{\le 0}K$ and $\tau_{\ge 1}K$ of a representative complex.

Lemma 9.7.44 (Good truncation preserves constructible cohomology)**.** *Lean:* `conditional-interface:lemma:weil-ftdc-truncation-constructible`

Uses: `def:weil-ftdc-standard-t-structure`, `def:weil-ftdc-constructible-derived-category`

For $K \in D^b_{c,\tau}$ and any n, both $\tau_{\le n}K$ and $\tau_{\ge n}K$ lie in $D^b_{c,\tau}$.

Proof. Uses:

Good truncation retains the original cohomology sheaves on one side of n and replaces those on the other side by zero. The zero sheaf and every $\mathcal{H}^i(K)$ are constructible, and boundedness is preserved. □

Proposition 9.7.45 (The standard t-structure restricts)**.** *Lean:* `conditional-interface:prop:weil-ftdc-standard-t-structure`

Uses: `def:weil-ftdc-standard-t-structure`

The displayed pair is a bounded t-structure on $D^b_{c,\tau}(X,\Lambda)$. Its heart is canonically equivalent to $\mathsf{Cons}_\tau(X,\Lambda)$ by $F \mapsto F[0]$, with inverse $\mathcal{H}^0_\tau$.

Proof. Uses: `lemma:weil-ftdc-truncation-constructible`

The orthogonality and shift inclusions are inherited from the standard t-structure on D^b_τ. The good-truncation triangle lies inside the constructible subcategory by the preceding lemma, proving the decomposition axiom. Boundedness is inherited from D^b. An object of the heart has only $\mathcal{H}^0$ nonzero and is canonically isomorphic to that sheaf in degree zero. □

Lemma 9.7.46 (Shift closure). *Lean:* `conditional-interface:lemma:weil-ftdc-shift-closure`

Uses: `def:weil-ftdc-constructible-derived-category`

If $K \in D^b_{c,\tau}$ and $m \in \mathbf{Z}$, then $K[m]$ lies in $D^b_{c,\tau}$.

Proof. Uses:

$\mathcal{H}^i(K[m]) = \mathcal{H}^{i+m}(K)$. This reindexes a finite family of constructible sheaves. □

Lemma 9.7.47 (Cohomology of a cone). *Lean:* `conditional-interface:lemma:weil-ftdc-cone-cohomology-short-exact`

Uses: `def:weil-ftdc-cohomology-objects`

For $u : K \to L$ and $C = \mathrm{Cone}(u)$, every i has a natural short exact sequence

$$0 \to \operatorname{coker}\bigl(\mathcal{H}^i(u)\bigr) \to \mathcal{H}^i(C) \to \ker\bigl(\mathcal{H}^{i+1}(u)\bigr) \to 0.$$

Proof. Uses:

Extract the displayed sequence from the segment $\mathcal{H}^i(K) \to \mathcal{H}^i(L) \to \mathcal{H}^i(C) \to \mathcal{H}^{i+1}(K) \to \mathcal{H}^{i+1}(L)$ of the long exact cohomology sequence. □

Lemma 9.7.48 (Cone closure). *Lean:* `conditional-interface:lemma:weil-ftdc-cone-closure`

Uses:

If $K, L \in D^b_{c,\tau}$, then the cone of every morphism $K \to L$ lies in $D^b_{c,\tau}$.

Proof. Uses: `lemma:weil-ftdc-cone-cohomology-short-exact`, `lemma:weil-ftdc-constructible-kernel`, `lemma:weil-ftdc-constructible-cokernel`, `lemma:weil-ftdc-constructible-extension`

The kernel and cokernel in the cone sequence are constructible, and their extension is constructible. Outside the union of the finite cohomological supports of K and L (shifted by one), the same sequence makes $\mathcal{H}^i(C)$ zero, so the cone is bounded. □

Proposition 9.7.49 (Triangulated constructible subcategory). *Lean:* `conditional-interface:prop:weil-ftdc-constructible-triangulated`

Uses:

The category $D^b_{c,\tau}(X, \Lambda)$ is a strictly full triangulated subcategory of $D^b_\tau(X, \Lambda)$, with the inherited zero object, shifts, and distinguished triangles.

Proof. Uses: `lemma:weil-ftdc-shift-closure`, `lemma:weil-ftdc-cone-closure`

It contains zero, is shift closed, and is closed under a third vertex of a mapping-cone triangle. Every distinguished triangle is isomorphic to such a triangle, and strict fullness transports membership across isomorphisms. □

Definition 9.7.50 (Stable enhancement). Lean: `conditional-interface:def:weil-ftdc-stable-enhancement`

Uses: `def:weil-ftdc-bounded-complexes`, `def:weil-ftdc-quasi-isomorphism`

Let $\mathcal{D}^b_\tau(X, \Lambda)$ be the stable localization of the dg nerve of bounded complexes at quasi-isomorphisms, and let $\mathcal{D}^b_{c,\tau}$ be its full subcategory on objects with constructible cohomology. Its homotopy category is the Verdier category defined above.

Lemma 9.7.51 (Finite limits and colimits in the stable enhancement). *Lean:* `conditional-interface:lemma:weil-ftdc-derived-finite-limits-colimits`

Uses: `def:weil-ftdc-stable-enhancement`

The full stable subcategory $\mathcal{D}^b_{c,\tau}$ is closed under finite limits and finite colimits in $\mathcal{D}^b_\tau$.

Proof. Uses: `lemma:weil-ftdc-shift-closure`, `lemma:weil-ftdc-cone-closure`

In a stable category it suffices to have zero objects and to be closed under fibers and cofibers. A cofiber is a cone, and a fiber is its shifted cone. The zero, cone, and shift closure lemmas therefore give both assertions. This statement concerns the stable enhancement; it does not falsely assert that a triangulated homotopy category has categorical kernels of all maps. □

9.7.5 Tensor and pullback at derived level

Definition 9.7.52 (Finite Tor amplitude)**.** Lean: `conditional-interface:def:weil-ftdc-finite-tor-amplitude`

Uses: `def:weil-ftdc-bounded-derived-category`

An object $P \in D^b_\tau(X,\Lambda)$ has finite Tor amplitude $[a,b]$ if $P \otimes^L_{\underline{\Lambda}} M$ has cohomology only in $[a,b]$ for every module sheaf M placed in degree zero. A perfect object has finite Tor amplitude locally and, on a finite constructible stratification, uniformly.

Lemma 9.7.53 (Constructibility of derived tensor cohomology)**.** *Lean:* `conditional-interface:lemma:weil-ftdc-derived-tensor-constructible-minus`

Uses:

If K, L are bounded-above complexes with constructible cohomology, then $K \otimes^L_{\underline{\Lambda}} L$ is bounded above and all its cohomology sheaves are constructible.

Proof. Uses: `lemma:weil-ftdc-constructible-tensor,lemma:weil-ftdc-constructible-extension`

For a fixed total degree, replace both inputs by sufficiently high good truncations; only finitely many cohomology sheaves contribute. The finite hyper-Tor filtration has graded pieces $\mathrm{Tor}^{\underline{\Lambda}}_p(\mathcal{H}^q K, \mathcal{H}^r L)$. On a common stratification these are the constant sheaves attached to Tor of finite Λ-modules, hence are constructible. Extension closure reconstructs the abutment. The total degree bound gives bounded-above vanishing. □

Lemma 9.7.54 (Bounded tensor closure with a finite-Tor factor)**.** *Lean:* `conditional-interface:lemma:weil-ftdc-derived-tensor-bounded`

Uses: `def:weil-ftdc-finite-tor-amplitude`

If $K, L \in D^b_{c,\tau}$ and one factor has finite Tor amplitude, then $K \otimes^L_{\underline{\Lambda}} L$ lies in $D^b_{c,\tau}$. In particular, tensor with a perfect constructible complex preserves D^b_c.

Proof. Uses: `lemma:weil-ftdc-derived-tensor-constructible-minus`

The preceding lemma gives constructible cohomology and an upper bound. If K has amplitude $[a,b]$ and L has cohomology in $[r,s]$, then the tensor has no cohomology outside $[a+r, b+s]$. The argument is symmetric in the two factors. □

Proposition 9.7.55 (Unrestricted bounded tensor closure is false)**.** *Lean:* `conditional-interface:prop:weil-ftdc-unrestricted-tensor-counterexample`

Uses: `def:weil-ftdc-constructible-derived-category`

For a general finite torsion ring, two objects of D^b_c need not have bounded derived tensor product. Hence no unrestricted monoidal closure of $D^b_c(X,\Lambda)$ is asserted.

Proof. Uses:

Take X to be a geometric point, $\Lambda = \mathbf{Z}/\ell^2$, and $M = \Lambda/\ell$ in degree zero. The periodic free resolution

$$\cdots \xrightarrow{\ell} \Lambda \xrightarrow{\ell} \Lambda \xrightarrow{\ell} \Lambda \longrightarrow M \longrightarrow 0$$

is exact because $\ker(\ell) = \ell\Lambda = \mathrm{im}(\ell)$. After tensoring with M, every differential is zero, so $\mathrm{Tor}^{\Lambda}_i(M,M) \simeq M$ for every $i \geq 0$. Thus $M \otimes^L_\Lambda M$ is unbounded below although both inputs are bounded constructible. □

Definition 9.7.56 (Derived same-coefficient pullback)**.** Lean: `conditional-interface:def:weil-ftdc-derived-pullback`

Uses: `def:weil-ftdc-scheme-pullback,def:weil-ftdc-bounded-derived-category`

Define $Lf^*_\tau : D^b_\tau(X,\Lambda) \to D^b_\tau(Y,\Lambda)$ by applying f^*_τ termwise to a bounded representative.

Lemma 9.7.57 (Derived pullback is represented termwise)**.** *Lean:* `conditional-interface:lemma:weil-ftdc-derived-pullback-termwise`

Uses: `def:weil-ftdc-derived-pullback`

Termwise pullback preserves quasi-isomorphisms, and $\mathcal{H}^i(Lf^*_\tau K) \simeq f^*_\tau \mathcal{H}^i(K)$ naturally.

Proof. Uses: `lemma:weil-ftdc-pullback-exact`

Exactness of f^*_τ makes it commute with kernels, images, and cokernels, hence with cohomology. It therefore sends acyclic cones to acyclic cones and descends through the Verdier localization. □

Lemma 9.7.58 (Derived pullback preserves bounded constructibility). *Lean:* `conditional-interface:lemma:weil-ftdc-derived-pullback-constructible`

Uses:

For every morphism $f : Y \to X$, Lf_τ^* restricts to $D^b_{c,\tau}(X,\Lambda) \to D^b_{c,\tau}(Y,\Lambda)$ and is t-exact.

Proof. Uses: `lemma:weil-ftdc-derived-pullback-termwise,lemma:weil-ftdc-pullback-constructible`

The cohomology formula and constructible pullback lemma give membership. The formula preserves the inequalities defining the standard t-structure. □

Theorem 9.7.59 (Finite-level derived site comparison). *Lean:* `conditional-interface:thm:weil-ftdc-derived-site-equivalence`

Uses: `def:weil-ftdc-constructible-derived-category`

Exact inverse image induces a t-exact triangulated equivalence

$$\epsilon_X^{-1} : D^b_{c,\mathrm{et}}(X,\Lambda) \xrightarrow{\sim} D^b_{c,\mathrm{proet}}(X,\Lambda).$$

It commutes with cohomology, shifts, cones, finite stable limits/colimits, same-coefficient pullback, and derived tensor whenever the latter remains in the bounded finite-Tor range.

Proof. Uses: `thm:weil-ftdc-constructible-site-equivalence,lemma:weil-ftdc-heart-equivalence-properties,lemma:weil-ftdc-truncation-constructible,lemma:weil-ftdc-cone-closure`

Exactness gives the cohomology identity $\mathcal{H}^i(\epsilon_X^{-1}K) = \epsilon_X^{-1}\mathcal{H}^i(K)$. The heart equivalence proves full faithfulness and essential surjectivity on objects with a single nonzero cohomology sheaf. For full faithfulness, induct on

$$\mathrm{amp}(K) + \mathrm{amp}(L)$$

for two bounded constructible complexes. If one object has positive amplitude, cut it by the good truncation triangle at its top nonzero cohomological degree. The two outer Hom groups in the resulting long exact sequence have smaller total amplitude; the amplitude-zero case is the heart equivalence together with the usual Ext comparison. Exactness of ϵ_X^{-1} transports the truncation triangles and their connecting maps, so the two long exact Hom sequences form a commutative diagram and the five lemma gives bijectivity for the middle Hom group.

For essential surjectivity, induct on the amplitude of a bounded constructible pro-etale complex M. The amplitude-zero case is the heart equivalence. In the induction step, write the truncation triangle

$$\tau_{\le n-1}M \longrightarrow M \longrightarrow \mathcal{H}^n(M)[-n] \xrightarrow{\delta} \tau_{\le n-1}M[1].$$

By induction and by the heart equivalence the two outer objects lift to the etale constructible derived category. Full faithfulness lifts the connecting map δ uniquely. Completing the lifted map to a cone gives an etale object whose inverse image is isomorphic to M by the triangulated five lemma. This is the bounded specialization of Stacks Project, Lemma 61.27.4 (Tag 09AI), which combines the constructible-heart equivalence with the derived comparison theorem for ϵ^{-1}.

T-exactness is the cohomology identity, and compatibility with shifts, cones, finite stable limits and colimits follows because the functor is represented termwise and is exact. Pullback naturality is the site-level base-change identity from the heart comparison. Tensor compatibility holds in the stated bounded finite-Tor range because the tensor product is computed by the same finite Tor filtration before and after the exact equivalence of hearts. □

9.7.6 A1 realization and the A2 finite-level rebind

Definition 9.7.60 (A1 finite-torsion category realization). *Lean:* `conditional-interface:def:weil-ftdc-a1-torsion-category-realization`

Uses: `def:weil-ftdc-constructible-derived-category`

For the A1 specialization $\Lambda_n = \mathbf{Z}/\ell^n$ with ℓ invertible on X, define the category denoted there by $\mathcal{D}_n(X)$ to be

$$\mathcal{D}_n^{\mathrm{real}}(X) := D^b_{c,\mathrm{proet}}(X, \mathbf{Z}/\ell^n).$$

This is a definition of the requested object, not a bundled certificate.

Lemma 9.7.61 (The A1 realization has the requested operations). *Lean:* `conditional-interface:lemma:weil-ftdc-a1-realization-properties`

Uses: `def:weil-ftdc-a1-torsion-category-realization`

The realization has a zero object, cochain shifts, distinguished cone triangles, cohomology objects $\mathcal{H}^r_X$ in the constructible heart, and the bounded standard t-structure. Its stable enhancement has finite limits and colimits.

Proof. Uses: `prop:weil-ftdc-standard-t-structure,prop:weil-ftdc-constructible-triangulated,lemma:weil-ftdc-derived-finite-limits-colimits`

These are respectively the zero complex, localized cochain shift, localized mapping-cone triangles, the cohomology functors, the restricted t-structure, and the stable finite-(co)limit result proved above. □

Theorem 9.7.62 (Discharge of the A1 torsion derived-category obligation)**.** Lean: `conditional-interface:thm:weil-ftdc-a1-torsion-derived-category-discharge`

Uses:

The construction and properties above discharge the finite-torsion derived-category obligation recorded in the project ledger for every noetherian X and every $\Lambda_n = \mathbf{Z}/\ell^n$ with ℓ invertible on X.

Proof. Uses: `lemma:weil-ftdc-a1-realization-properties,thm:weil-ftdc-constructible-abelian,thm:weil-ftdc-derived-localization-exists`

The obligation asks for the typed category, zero, shift, distinguished triangles, and cohomology sheaves. The realization defines the category as an actual Verdier localization, the constructible heart is abelian, and the preceding properties lemma supplies every requested operation. No A1 or A2 OPEN declaration occurs in this dependency chain. This discharge is only for the pro-etale finite-torsion category and does not discharge any of the etale/pro-etale comparison obligations displayed above. □

Definition 9.7.63 (A2 finite-torsion rebind functor)**.** Lean: `conditional-interface:def:weil-ftdc-a2-torsion-rebind`

Uses: `def:weil-ftdc-a1-torsion-category-realization`

Use the coefficient-sheaf identification $\underline{\mathbf{Z}/\ell^n}_{X,\mathrm{proet}} \simeq \Lambda_{n,X}$ to define

$$\mathfrak{r}_{X,n} : \mathcal{D}_n^{\mathrm{real}}(X) \longrightarrow D_c^b(X_{\mathrm{proet}}, \mathbf{Z}/\ell^n)_{\mathrm{A2}}$$

as the identity on the underlying complexes and morphisms.

Lemma 9.7.64 (Intrinsic constructibility equals the A2 witness predicate)**.** Lean: `conditional-interface:lemma:weil-ftdc-a2-witness-equivalence`

Uses: `def:weil-ftdc-a2-torsion-rebind`

For a bounded finite-level complex K, the following are equivalent:

1. *every $\mathcal{H}^i(K)$ is constructible in the intrinsic sense;*
2. *K admits the finite interval, one finite stratification, and finite locally constant restrictions required by the A2 finite-torsion witness.*

Proof. Uses: `lemma:weil-ftdc-stratification-common-refinement`

The second condition immediately implies the first. Conversely, only finitely many cohomology sheaves are nonzero. Choose one adapted finite stratification for each nonzero cohomology sheaf and take their finite common refinement. Take the minimum and maximum of the finite cohomological support. Exact locally closed pullback identifies the restricted cohomology with the cohomology of the restriction, giving every A2 witness field separately. □

Theorem 9.7.65 (The A1 and A2 finite-torsion categories agree)**.** Lean: `conditional-interface:thm:weil-ftdc-a1-a2-torsion-rebind-equivalence`

Uses: `def:weil-ftdc-a2-torsion-rebind`

The functor $\mathfrak{r}_{X,n}$ is an equivalence, with inverse the identity on underlying complexes and morphisms. It preserves zero, cohomology, shifts, cones, the standard t-structure, and every finite-Tor tensor and same-coefficient pullback defined above.

Proof. Uses: `lemma:weil-ftdc-a2-witness-equivalence`

The witness-equivalence lemma says that source and target have exactly the same objects inside the same ambient derived category. By definition they have the same morphisms. Thus both composites with the identity inverse are identities. Every listed operation is computed on the same representative complex, so its two displayed aliases coincide naturally. □

Theorem 9.7.66 (Quarantine root removed at finite level)**.** Lean: `conditional-interface:thm:weil-ftdc-a3-quarantine-root-removed`

Uses:

After replacing the A1 typed placeholder by $\mathcal{D}_n^{\mathrm{real}}(X)$, the five A3 A2-rebind labels quarantined solely through the finite-level derived-category obligation no longer have that OPEN ancestor. This theorem does not assert closure of any adic completed category, any unrestricted bounded derived tensor product, or any etale/pro-etale site-comparison obligation.

Proof. Uses: `thm:weil-ftdc-a1-torsion-derived-category-discharge,thm:weil-ftdc-a1-a2-torsion-rebind-equivalence`

The discharge theorem removes the unique OPEN category root, and the rebind equivalence identifies the A1 and A2 finite-torsion object predicates and operations used by those labels. The counterexample above explains why the transport is restricted to the finite-Tor tensor operations actually used by the finite reduction objects. No assertion concerning completion, $Rf_!$, duality, or trace enters this proof. □

Chapter 10

Exceptional direct image from compactifications

Status. The constructions in this chapter are made at finite torsion coefficient level. The pro-etale constructible derived categories and the finite-level proper-base-change and extension-by-zero theorems are not mathlib leaves. They therefore enter only through explicitly named input certificates. No OPEN certificate is used as a proved theorem elsewhere.

10.1 Finite torsion input and open extension by zero

Definition 10.1.1 (Finite torsion coefficient datum)**.** Uses: `def:weil-prime-power-data,def:weil-coefficient-data`

Lean: `Weil.A1.TorsionCoefficientDatum`

Fix a prime ℓ invertible on every scheme under consideration and an integer $n \geq 1$. Put

$$\Lambda_n := \mathbf{Z}/\ell^n\mathbf{Z}.$$

The datum records the quotient map $\mathbf{Z}_\ell \twoheadrightarrow \Lambda_n$, the ring structure on Λ_n, and the convention that all sheaves in this section are sheaves of Λ_n-modules on the selected small pro-etale sites.

Definition 10.1.2 (Finite torsion constructible derived category)**.** Uses: `def:weil-a1-torsion-coefficient-datum,def:weil-ftdc-a1-torsion-category-realization`

Lean: `Weil.A1.TorsionDerivedCategory`

For a scheme X and the datum of Definition 10.1.1, define

$$\mathcal{D}_n(X) := \mathcal{D}_n^{\mathrm{real}}(X) = D^b_{c,\mathrm{proet}}(X, \mathbf{Z}/\ell^n\mathbf{Z}).$$

Its zero object, cochain shifts, distinguished cone triangles, bounded standard t-structure, and cohomology sheaves $\mathcal{H}^r_X$ are the structures inherited from the realized finite-torsion category.

Proof. Uses: `thm:weil-ftdc-a1-torsion-derived-category-discharge,thm:weil-ftdc-a1-a2-torsion-rebind-equivalence`

The first cited theorem supplies the category and every operation required by the former finite-torsion derived-category obligation. The second identifies this realization with the A2 finite-torsion witness category by an equivalence that is the identity on underlying complexes and morphisms. This is only a finite-level rebind: it does not construct an adic category and does not assert existence or constructibility preservation for $Rf_!$. □

Definition 10.1.3 (Open–closed complement datum)**.** Uses: `def:weil-a1-torsion-derived-category`

Lean: `Weil.A1.OpenClosedComplement`

An open–closed complement datum on X consists of an open immersion $j : U \hookrightarrow X$, a closed immersion $i : Z \hookrightarrow X$, an identification $Z = X \setminus U$, and the equality of the two pullback composites with the empty scheme. The identification is part of the datum, not a later appeal to a complement.

Definition 10.1.4 (Raw extension-by-zero presheaf)**.** Uses: `def:weil-a1-open-closed-complement`

Lean: `Weil.A1.ExtensionZeroPresheaf`

Let $\mathcal{F}$ be a Λ_n-module sheaf on U_{proet}. For $V \to X$ in X_{proet}, define

$$P_{j,\mathcal{F}}(V) := \begin{cases} \mathcal{F}(V \times_X U), & V \times_X U \to V \text{ is an isomorphism}, \\ 0, & V \times_X U \text{ is empty}, \\ \{s \in \mathcal{F}(V \times_X U) : \mathrm{supp}_V(s) \subseteq V \text{ is closed and contained in } V \times_X U\}, & \text{otherwise}. \end{cases}$$

Here support is defined as the complement of the largest pro-etale open on which the restriction of s is zero. Restriction maps are restriction of sections and preserve the displayed support condition.

Lemma 10.1.5 (Restriction preserves the support condition)**.** *Uses:* `def:weil-a1-extension-zero-presheaf`

Lean: `Weil.A1.ExtensionZeroPresheaf.restrict`

For an arrow $V' \to V$ in X_{proet} and $s \in P_{j,\mathcal{F}}(V)$, the restriction $s|_{V'}$ belongs to $P_{j,\mathcal{F}}(V')$.

Proof. Uses: `def:weil-a1-extension-zero-presheaf`

The inverse image of the largest open on which s vanishes is an open on which $s|_{V'}$ vanishes. Therefore $\mathrm{supp}_{V'}(s|_{V'})$ is contained in the inverse image of the closed set $\mathrm{supp}_V(s)$. It is consequently closed and is contained in $V' \times_X U$, which is the required condition. □

Definition 10.1.6 (Sheaf extension by zero)**.** Uses: `def:weil-a1-extension-zero-presheaf`

Lean: `Weil.A1.ExtensionZeroSheaf`

Define $j_!\mathcal{F}$ to be the sheafification of $P_{j,\mathcal{F}}$ in Λ_n-modules. For a morphism of sheaves $a : \mathcal{F} \to \mathcal{G}$, define $j_!a$ by applying a to every local representative and then sheafifying.

Lemma 10.1.7 (Functoriality of extension by zero)**.** *Uses:* `def:weil-a1-extension-zero-sheaf`

Lean: `Weil.A1.ExtensionZeroSheaf.map_id_comp`

The assignments $\mathcal{F} \mapsto j_!\mathcal{F}$ and $a \mapsto j_!a$ preserve identities and composition.

Proof. Uses: `def:weil-a1-extension-zero-sheaf`

Before sheafification, both assertions hold because restriction and the zero section are computed pointwise. Sheafification is a functor, so it preserves the two equalities. □

Definition 10.1.8 (Extension unit)**.** Uses: `def:weil-a1-extension-zero-sheaf`

Lean: `Weil.A1.ExtensionZeroSheaf.unit`

For $\mathcal{F}$ on U, define

$$\upsilon_{j,\mathcal{F}} : \mathcal{F} \longrightarrow j^* j_!\mathcal{F}$$

by sending a section to the same local section on the first case of $P_{j,\mathcal{F}}$. On a general object of U_{proet}, use the canonical identification with its pullback to X and the sheafification unit.

Definition 10.1.9 (Restriction of a target section)**.** Uses: `def:weil-a1-extension-zero-sheaf`

Lean: `Weil.A1.ExtensionZeroSheaf.adjunctionForward`

For $\mathcal{G}$ on X, define

$$\mathrm{res}_{j,\mathcal{F},\mathcal{G}} : \mathrm{Hom}_X(j_!\mathcal{F}, \mathcal{G}) \longrightarrow \mathrm{Hom}_U(\mathcal{F}, j^*\mathcal{G})$$

by restricting a morphism along $\upsilon_{j,\mathcal{F}}$ and then applying the pullback functor j^*.

Definition 10.1.10 (Extension of a restricted target section)**.** Uses: `def:weil-a1-extension-zero-sheaf`

Lean: `Weil.A1.ExtensionZeroSheaf.adjunctionBackward`

For $b : \mathcal{F} \to j^*\mathcal{G}$, define

$$\mathrm{ext}_{j,\mathcal{F},\mathcal{G}}(b) : j_!\mathcal{F} \to \mathcal{G}$$

on the presheaf $P_{j,\mathcal{F}}$ by applying b to a local representative on $V \times_X U$ and extending it by zero on the complement. To check this is a section of $\mathcal{G}(V)$, refine V by the open where the support is closed and by its complement; on the overlap both sections are zero. Glue the two pieces and pass through the sheafification universal property.

Lemma 10.1.11 (Extension followed by restriction)**.** *Uses:* `def:weil-a1-extension-unit`, `def:weil-a1-extension-adjunction-forward`, `def:weil-a1-extension-adjunction-backward`

Lean: `Weil.A1.ExtensionZeroSheaf.adjunction_left`

For every $b : \mathcal{F} \to j^*\mathcal{G}$,

$$\mathrm{res}(\mathrm{ext}(b)) = b.$$

Proof. Uses: def:weil-a1-extension-unit,def:weil-a1-extension-adjunction-forward,def:weil-a1-extension-adjunction-backward

Evaluate on an object $W \to U$. The pullback of W to X lies entirely in the first case of the presheaf definition. The extension sends $b(s)$ to the same section, and restriction sends it back to $b(s)$. Equality on all local sections gives equality of sheaf morphisms. □

Lemma 10.1.12 (Restriction followed by extension). Uses: def:weil-a1-extension-unit, def:weil-a1-extension-adjunction-forward, def:weil-a1-extension-adjunction-backward

Lean: WeilA1.ExtensionZeroSheaf.adjunction_right

For every $a : j_!\mathcal{F} \to \mathcal{G}$,

$$\mathrm{ext}(\mathrm{res}(a)) = a.$$

Proof. Uses: def:weil-a1-extension-unit,def:weil-a1-extension-adjunction-forward,def:weil-a1-extension-adjunction-backward

It suffices to compare the two maps after the sheafification epimorphism $P_{j,\mathcal{F}} \to j_!\mathcal{F}$. For a representative s on V, the map a is local on V. On the support-open, both maps equal the restriction of a; on the complementary open, both maps are zero by the defining support condition. Their restrictions agree on the overlap, so the sheaf gluing axiom gives equality on V and hence after sheafification. □

Proposition 10.1.13 (Extension by zero adjunction). Uses: def:weil-a1-extension-zero-sheaf, def:weil-a1-extension-adjunction-forward, def:weil-a1-extension-adjunction-backward

Lean: WeilA1.ExtensionZeroSheaf.adjunction

The maps res *and* ext *are mutually inverse, naturally in $\mathcal{F}$ and $\mathcal{G}$; thus $j_! \dashv j^*$.*

Proof. Uses: lemma:weil-a1-extension-adjunction-left-inverse,lemma:weil-a1-extension-adjunction-right-inverse

The two preceding inverse equations prove bijectivity of the Hom map. Naturality follows by evaluating both composites on a local representative and using functoriality of restriction. □

Lemma 10.1.14 (Restriction of extension to the open). Uses: def:weil-a1-extension-zero-sheaf, def:weil-a1-extension-unit

Lean: WeilA1.ExtensionZeroSheaf.restrict_iso

The unit $\upsilon_{j,\mathcal{F}} : \mathcal{F} \to j^* j_!\mathcal{F}$ is an isomorphism.

Proof. Uses: def:weil-a1-extension-zero-sheaf,def:weil-a1-extension-unit

On every object of U_{proet}, the pullback to X is in the first case of the presheaf definition, so the unit is the identity on sections. The inverse is induced by the same identity calculation and sheafification. □

Lemma 10.1.15 (Extension has zero closed restriction). Uses: def:weil-a1-open-closed-complement, def:weil-a1-extension-zero-sheaf

Lean: WeilA1.ExtensionZeroSheaf.closed_zero

There is a canonical zero morphism $i^* j_!\mathcal{F} \to 0$ and the induced object $i^* j_!\mathcal{F}$ is zero.

Proof. Uses: def:weil-a1-open-closed-complement,def:weil-a1-extension-zero-sheaf

Every object over Z has empty pullback to U. The presheaf value is thus zero, and sheafification preserves the zero sheaf. □

Lemma 10.1.16 (Geometric stalks of extension by zero). Uses: def:weil-a1-extension-zero-sheaf, def:weil-a1-open-closed-complement

Lean: WeilA1.ExtensionZeroSheaf.stalk_formula

For every geometric point $\bar{x} \to X$,

$$(j_!\mathcal{F})_{\bar{x}} \cong \begin{cases} \mathcal{F}_{\bar{x}}, & \bar{x} \text{ factors through } U, \\ 0, & \bar{x} \text{ factors through } Z. \end{cases}$$

The isomorphism in the first case is the stalk of $\upsilon_{j,\mathcal{F}}$.

Proof. Uses: def:weil-a1-extension-zero-sheaf,def:weil-a1-open-closed-complement,lemma:weil-a1-extension-zero-open-restriction,lemma:weil-a1-extension-zero-closed-restriction

The first case follows from the open-restriction isomorphism. In the second case every neighborhood has a refinement whose pullback to U is empty; the filtered colimit defining the stalk is therefore the zero group. □

Lemma 10.1.17 (Module sheafification preserves kernels). *Lean:* `blueprint-target:lemma:weil-a1-module-sheafification-kernel-v1`

Uses: `def:weil-ftdc-module-sheaves`

Let a be module sheafification on either selected small site. For every morphism $\varphi : P \to Q$ of presheaves of modules, the canonical map

$$a(\ker \varphi) \longrightarrow \ker(a\varphi)$$

is an isomorphism.

Proof. Uses: `thm:weil-ftdc-ambient-abelian`

A section of aP is locally represented by a presheaf section. If its image in aQ is zero, refine the representing cover until the image of each representative is already zero in Q. The refined representatives then lie in $\ker \varphi$ and give a local preimage from $a(\ker \varphi)$. Conversely, two local kernel representatives with equal images in aP agree after a common refinement, hence already agree in the sheafification of $\ker \varphi$. The canonical map is therefore locally surjective and locally injective, so it is an isomorphism. This is the usual left exactness of associated module sheafification and uses no point of the topos. □

Lemma 10.1.18 (Additivity of extension by zero). *Lean:* `blueprint-target:lemma:weil-a1-extension-zero-additive-v1`

Uses: `def:weil-a1-extension-zero-sheaf`

For morphisms $a, b : F \to G$ of Λ_n-module sheaves on U, one has

$$j_!(a+b) = j_!a + j_!b, \qquad j_!(0_{F,G}) = 0_{j_!F, j_!G}.$$

Proof. Uses: `def:weil-a1-extension-zero-presheaf,def:weil-a1-extension-zero-sheaf`

On every support-closed local representative s, the two sides of the first equality have value $a(s) + b(s)$, while both sides of the second have value zero. Addition preserves the support condition because $\operatorname{supp}(s+t)$ is contained in the union of the two closed supports. The equalities are compatible with restriction and therefore survive module sheafification. □

Lemma 10.1.19 (The raw extension presheaf preserves kernels). *Lean:* `blueprint-target:lemma:weil-a1-extension-zero-raw-kernel-v1`

Uses: `def:weil-a1-extension-zero-presheaf,def:weil-ftdc-module-sheaves`

For a morphism $c : F \to G$, the canonical map of presheaves

$$P_{j,\ker c} \longrightarrow \ker\bigl(P_{j,F} \longrightarrow P_{j,G}\bigr)$$

is an isomorphism.

Proof. Uses: `thm:weil-ftdc-ambient-abelian`

Kernels of module sheaves are computed sectionwise. If a test object lies over U, both sides are $\ker(F(V) \to G(V))$; if its pullback to U is empty, both sides are zero. In the remaining case, a section belongs to the right-hand kernel precisely when it is a section of $\ker c$ on $V \times_X U$. Its support as a kernel section equals its support after the monomorphism $\ker c \hookrightarrow F$, so the same closed-support condition occurs on both sides. These identifications commute with restriction. □

Lemma 10.1.20 (Exactness of extension by zero). *Uses:* `def:weil-a1-extension-zero-sheaf` *Lean:* `Weil.A1.ExtensionZeroSheaf.exact`

For a short exact sequence $0 \to \mathcal{F}' \to \mathcal{F} \to \mathcal{F}'' \to 0$ on U, its image under $j_!$ is short exact on X.

Proof. Uses: `lemma:weil-a1-module-sheafification-kernel-v1,lemma:weil-a1-extension-zero-additive-v1,lemma:weil-a1-extension-zero-raw-kernel-v1,prop:weil-a1-extension-zero-adjunction,thm:weil-ftdc-ambient-abelian`

The raw-kernel lemma and kernel preservation by module sheafification give

$$j_!(\ker c) \xrightarrow{\sim} \ker(j_!c)$$

for every sheaf morphism c. The previously proved adjunction $j_! \dashv j^*$ makes $j_!$ a left adjoint, so it preserves cokernels. Together with additivity, preservation of kernels and cokernels makes $j_!$ an exact functor between the two abelian module-sheaf categories. Applying this to the displayed short exact sequence proves the assertion. This argument replaces the former geometric-stalk criterion and uses no enough-points hypothesis. □

Lemma 10.1.21 (Open extension preserves finite constructibility). *Lean:* `blueprint-target:lemma:weil-a1-extension-zero-constructible-v1`

Uses: `def:weil-a1-extension-zero-sheaf`, `def:weil-ftdc-constructible-sheaf`, `def:weil-a1-open-closed-complement`

If F is a constructible finite Λ_n-module sheaf on U, then $j_!F$ is a constructible finite Λ_n-module sheaf on X.

Proof. Uses: `lemma:weil-a1-extension-zero-open-restriction`, `lemma:weil-a1-extension-zero-closed-restriction`

Choose a finite locally closed stratification of U on which F is finite locally constant. Each of its strata remains locally closed in X because U is open. Adjoin the closed complement Z as one further stratum. On the old strata, the open-restriction isomorphism identifies $j_!F$ with F; on Z, the closed-restriction lemma identifies it with the zero sheaf. This finite stratification proves constructibility and finiteness. □

Lemma 10.1.22 (Termwise open extension is a bounded complex). *Lean:* `blueprint-target:lemma:weil-a1-extension-zero-termwise-complex-v1`

Uses: `def:weil-a1-extension-zero-sheaf`, `def:weil-ftdc-bounded-complexes`

For a bounded cochain complex $K = (K^r, d^r)$ on U, the assignments

$$(J_jK)^r = j_!K^r, \qquad d^r_{J_jK} = j_!(d^r)$$

form a bounded cochain complex on X.

Proof. Uses: `lemma:weil-a1-extension-zero-additive-v1`, `lemma:weil-a1-extension-zero-functorial`

Functoriality and additivity give

$$j_!(d^{r+1})\, j_!(d^r) = j_!(d^{r+1}d^r) = j_!0 = 0.$$

If K^r is a zero object, then $1_{j_!K^r} = j_!(1_{K^r}) = j_!0 = 0$; hence $j_!K^r$ is a zero object. Therefore the same finite interval that bounds K bounds J_jK. □

Lemma 10.1.23 (Termwise open extension preserves chain maps). *Lean:* `blueprint-target:lemma:weil-a1-extension-zero-chain-map-v1`

Uses: `def:weil-a1-extension-zero-sheaf`, `def:weil-ftdc-bounded-complexes`

For a cochain map $f : K \to L$, the degreewise maps $(J_jf)^r = j_!(f^r)$ form a cochain map $J_jK \to J_jL$ and preserve identity and composition.

Proof. Uses: `lemma:weil-a1-extension-zero-functorial`

Applying functoriality to $d^r_L f^r = f^{r+1} d^r_K$ gives

$$j_!(d^r_L) j_!(f^r) = j_!(f^{r+1}) j_!(d^r_K),$$

which is the cochain-map equation. The identity and composition equations hold in every degree by the same functoriality lemma. □

Lemma 10.1.24 (Termwise open extension preserves cochain homotopies). *Lean:* `blueprint-target:lemma:weil-a1-extension-zero-chain-homotopy-v1`

Uses: `def:weil-a1-extension-zero-sheaf`, `def:weil-ftdc-bounded-homotopy-category`

If h is a cochain homotopy from f to g, then the degreewise family $j_!h$ is a cochain homotopy from J_jf to J_jg.

Proof. Uses: `lemma:weil-a1-extension-zero-additive-v1`, `lemma:weil-a1-extension-zero-functorial`

Apply $j_!$ to the homotopy equation $f - g = d_Lh + hd_K$. Additivity transports the sum and difference, and functoriality transports both composites, giving

$$J_jf - J_jg = d_{J_jL}(j_!h) + (j_!h) d_{J_jK}.$$

□

Definition 10.1.25 (Termwise extension on bounded homotopy categories). Lean: `blueprint-target:def:weil-a1-extension-zero-homotopy-functor-v1`

Uses: `def:weil-a1-extension-zero-sheaf`, `def:weil-ftdc-bounded-homotopy-category`

Define

$$J_j : \mathsf{K}^b_{\mathrm{pro\acute{e}t}}(U, \Lambda_n) \longrightarrow \mathsf{K}^b_{\mathrm{pro\acute{e}t}}(X, \Lambda_n)$$

by termwise extension on objects and cochain maps.

Constructed by: `lemma:weil-a1-extension-zero-termwise-complex-v1`, `lemma:weil-a1-extension-zero-chain-map-v1`, `lemma:weil-a1-extension-zero-chain-homotopy-v1`

Lemma 10.1.26 (Cohomology of termwise open extension). *Lean:* `blueprint-target:lemma:weil-a1-extension-zero-cohomology-v1`

Uses: `def:weil-a1-extension-zero-homotopy-functor-v1, def:weil-ftdc-cohomology-objects`

For every bounded complex K and every $r \in \mathbf{Z}$, there is a natural isomorphism

$$H^r(J_j K) \xrightarrow{\sim} j_! H^r(K).$$

Proof. Uses: `lemma:weil-a1-extension-zero-exact`

Exactness makes $j_!$ commute with the kernels and images defining $H^r = \ker(d^r)/\operatorname{im}(d^{r-1})$. Applying these two comparison isomorphisms and the induced quotient isomorphism gives the displayed map. Every construction is induced by the differential and is therefore natural in cochain maps. □

Lemma 10.1.27 (Termwise open extension preserves quasi-isomorphisms). *Lean:* `blueprint-target:lemma:weil-a1-extension-zero-qis-v1`

Uses: `def:weil-a1-extension-zero-homotopy-functor-v1, def:weil-ftdc-quasi-isomorphism`

The functor J_j sends every bounded quasi-isomorphism on U to a bounded quasi-isomorphism on X.

Proof. Uses: `lemma:weil-a1-extension-zero-cohomology-v1`

For a quasi-isomorphism $q : K \to L$, naturality of the cohomology comparison identifies $H^r(J_j q)$ with $j_! H^r(q)$. Each $H^r(q)$ is an isomorphism, and every functor preserves isomorphisms. Hence $H^r(J_j q)$ is an isomorphism for all r. □

Lemma 10.1.28 (Termwise open extension preserves constructible cohomology). *Lean:* `blueprint-target:lemma:weil-a1-extension-zero-derived-constructible-v1`

Uses: `def:weil-a1-extension-zero-homotopy-functor-v1, def:weil-ftdc-constructible-derived-category`

If K has bounded constructible finite-torsion cohomology on U, then $J_j K$ has bounded constructible finite-torsion cohomology on X.

Proof. Uses: `lemma:weil-a1-extension-zero-cohomology-v1, lemma:weil-a1-extension-zero-constructible-v1`

The cohomology comparison identifies $H^r(J_j K)$ with $j_! H^r(K)$. The latter is constructible and finite by the constructibility lemma. The termwise-complex lemma retains a finite bound, so the output belongs to the bounded constructible subcategory. □

Definition 10.1.29 (Derived extension by zero). Uses: `def:weil-a1-extension-zero-homotopy-functor-v1, def:weil-a1-torsion-derived-category, def:weil-ftdc-bounded-derived-category, def:weil-ftdc-constructible-derived-category`

Lean: `Weil.A1.DerivedExtensionZero`

Let $\mathsf{K}_c^b(U, \Lambda_n)$ be the full subcategory of bounded complexes whose cohomology is constructible and finite, and define $\mathsf{K}_c^b(X, \Lambda_n)$ similarly. Write Q_U^c and Q_X^c for the restrictions of the two Verdier localization functors. Define

$$Rj_! : \mathcal{D}_n(U) \longrightarrow \mathcal{D}_n(X)$$

as the constructible restriction of the localization factor of J_j, with its factorization isomorphism

$$\eta_j : (Rj_!) Q_U^c \xRightarrow{\sim} Q_X^c \big(J_j|_{\mathsf{K}_c^b(U,\Lambda_n)}\big).$$

Constructed by: `lemma:weil-a1-extension-zero-qis-v1, lemma:weil-a1-extension-zero-derived-constructible-v1, thm:weil-ftdc-derived-localization-exists`

No injective, K-injective, K-flat, or K-projective replacement is selected.

Lemma 10.1.30 (Representative independence for derived extension). *Lean:* `blueprint-target:lemma:weil-a1-derived-extension-zero-independence-v1`

Uses: `def:weil-a1-derived-extension-zero, def:weil-ftdc-quasi-isomorphism`

For a quasi-isomorphism $q : K \to L$ in $\mathsf{K}_c^b(U, \Lambda_n)$, the morphism $Rj_!(Q_U^c q)$ is an isomorphism and, through η_j, equals $Q_X^c(J_j q)$. Consequently the value of $Rj_!$ is independent of a bounded constructible-cohomology representative and of a quasi-isomorphic refinement.

Proof. Uses: lemma:weil-a1-extension-zero-qis-v1

The quasi-isomorphism lemma makes $J_j q$ a quasi-isomorphism, so its image under Q_X^c is invertible. Naturality of the factorization isomorphism η_j identifies this image with $Rj_!(Q_U^c q)$. A roof comparison is generated by such inverted arrows, and refinement of two roofs gives the same localized morphism by the universal property. □

Lemma 10.1.31 (Uniqueness of the derived-extension factor). *Lean:* blueprint-target:lemma:weil-a1-derived-extension-factorization-uniqueness-v1

Uses: def:weil-a1-derived-extension-zero, def:weil-ftdc-bounded-derived-category

Let $T : \mathcal{D}_n(U) \to \mathcal{D}_n(X)$ carry a natural isomorphism

$$\theta : TQ_U^c \xRightarrow{\sim} Q_X^c\big(J_j|_{\mathsf{K}^b_c(U,\Lambda_n)}\big).$$

There is a unique natural isomorphism $\alpha : T \xRightarrow{\sim} Rj_!$ satisfying

$$\eta_j \circ (\alpha Q_U^c) = \theta.$$

Proof. Uses: thm:weil-ftdc-derived-localization-exists, lemma:weil-a1-extension-zero-qis-v1

The restricted functor $Q_X^c J_j$ inverts the quasi-isomorphisms by the cited lemma. Existence and uniqueness of α are exactly the functor-factorization part of the Verdier localization universal property. Applying the same property to the inverse factorization gives an inverse to α, so the unique comparison is a natural isomorphism. □

Lemma 10.1.32 (Triangulated exactness of derived extension). *Lean:* blueprint-target:lemma:weil-a1-derived-extension-triangulated-v1

Uses: def:weil-a1-derived-extension-zero, def:weil-ftdc-bounded-derived-category

The functor $Rj_!$ is additive and triangulated exact. It commutes with cochain shifts and mapping-cone triangles, and its cohomology comparison is

$$\mathcal{H}^r_X(Rj_!K) \xrightarrow{\sim} j_!\mathcal{H}^r_U(K).$$

Proof. Uses: lemma:weil-a1-extension-zero-additive-v1, lemma:weil-a1-extension-zero-cohomology-v1, lemma:weil-a1-derived-extension-factorization-uniqueness-v1, thm:weil-ftdc-derived-localization-exists

Termwise J_j commutes with shifts. Its mapping-cone differential is the image of the usual block differential; additivity and functoriality identify that image with the cone of J_j applied to the original map. The localization functors are exact, so the factorization isomorphism transports these shift and cone comparisons to $Rj_!$. Factorization uniqueness gives their naturality and coherence. Localizing the termwise cohomology comparison gives the displayed formula. □

Lemma 10.1.33 (Well-definedness certificate for def:weil-a1-derived-extension-zero). *Lean:* blueprint-target:lemma:weil-sf-well-defined-def-weil-a1-derived-extension-zero

Uses: def:weil-a1-derived-extension-zero

The functor $Rj_!$ has a typed termwise model, preserves bounded constructible finite-torsion objects, descends through quasi-isomorphisms, is independent of representatives and localization-factor choices, and is triangulated exact with the displayed cohomology comparison.

Proof. Uses: lemma:weil-a1-extension-zero-termwise-complex-v1, lemma:weil-a1-extension-zero-chain-map-v1, lemma:weil-a1-extension-zero-chain-homotopy-v1, lemma:weil-a1-extension-zero-qis-v1, lemma:weil-a1-extension-zero-derived-constructible-v1, lemma:weil-a1-derived-extension-representative-independence-v1, lemma:weil-a1-derived-extension-factorization-uniqueness-v1, lemma:weil-a1-derived-extension-triangulated-v1

The first three leaves assemble the additive functor J_j on bounded homotopy categories. Quasi-isomorphism preservation supplies its Verdier factor, while constructible-cohomology preservation restricts that factor to $\mathcal{D}_n$. The two independence leaves give representative and factor independence. The final leaf supplies shift, cone, and cohomology coherence. These are all fields of the stated contract. No proper base-change theorem and no resolution-existence statement enters this dependency chain. □

Lemma 10.1.34 (Degree-zero compatibility of derived extension). *Uses:* def:weil-a1-derived-extension-zero

Lean: Weil.A1.DerivedExtensionZero.degree_zero

For a sheaf $\mathcal{F}$ in degree zero, $\mathcal{H}^0_X(Rj_!\mathcal{F}) \cong j_!\mathcal{F}$ and all other cohomology sheaves vanish.

Proof. Uses: lemma:weil-a1-derived-extension-triangulated-v1

Apply the cohomology comparison for $Rj_!$ to $\mathcal{F}[0]$. Its only nonzero cohomology sheaf is $\mathcal{F}$ in degree zero, which gives the claimed isomorphism and vanishing in every other degree. □

10.2 Proper pushforward and the torsion comparison certificate

Definition 10.2.1 (Proper direct-image input)**.** Uses: `def:weil-a1-torsion-derived-category`

Lean: `Weil.A1.ProperDirectImageInput`

For a proper morphism $p : T \to Y$, a proper direct-image input consists of the left-exact sheaf functor $p_* : \mathrm{Mod}(T_{\mathrm{proet}}, \Lambda_n) \to \mathrm{Mod}(Y_{\mathrm{proet}}, \Lambda_n)$, a selected functorial injective (or K-injective) resolution I_T with a comparison map from every complex to I_T, and the total complex

$$Rp_*K := \mathrm{Tot}\big(p_* I_T(K)\big).$$

The input includes the assertion that quasi-isomorphisms are sent to quasi-isomorphisms, but does not include base change.

Open obligation: `OBL-A1-PROPER-DERIVED-PUSHFORWARD`

Lemma 10.2.2 (Proper pushforward preserves identities and composition)**.** *Uses:* `def:weil-a1-proper-direct-image-input`

Lean: `Weil.A1.ProperDirectImageInput.functorial`

For proper maps $p : T \to Y$ and $r : Y \to S$, the resolved construction gives natural maps

$$Rp_*\, Rp_*? \quad \textit{(with the second } Rp_* \textit{ replaced by } Rr_*\textit{)}$$

and a canonical comparison $Rr_*Rp_* \to R(rp)_*$, compatible with identity maps.

Proof. Uses: `def:weil-a1-proper-direct-image-input`

Apply the composition map of the chosen resolution functors to a complex, then totalize the resulting bicomplex. The identity comparison is the identity on each injective term. Associativity is equality of the two maps between the corresponding triple total complexes.

Open obligation: `OBL-A1-PROPER-DERIVED-COMPOSITION-COHERENCE`

□

Definition 10.2.3 (Proper base-change square)**.** Uses: `def:weil-a1-proper-direct-image-input`

Lean: `Weil.A1.ProperBaseChangeSquare`

A proper base-change square is a Cartesian diagram

$$\begin{array}{ccc} T' & \xrightarrow{g'} & T \\ {\scriptstyle p'}\downarrow & & \downarrow{\scriptstyle p} \\ Y' & \xrightarrow{g} & Y \end{array}$$

with p proper and with the chosen proper direct-image inputs for p and p'.

Definition 10.2.4 (Proper base-change morphism)**.** Uses: `def:weil-a1-proper-base-change-square`

Lean: `Weil.A1.ProperBaseChangeMap`

For $K \in \mathcal{D}_n(T)$ define

$$\mathrm{BC}_{p,g}(K) : Lg^* Rp_* K \longrightarrow Rp'_* Lg'^* K$$

as the composite of the unit of (Lg'^*, Rg'_*), the equality $pg' = gp'$, and the counit of (Lg^*, Rg_*). The map is first defined on resolved complexes and then localized.

Definition 10.2.5 (Proper base-change certificate)**.** Uses: `def:weil-a1-proper-base-change-square`, `def:weil-a1-proper-base-change-map`

Lean: `Weil.A1.ProperBaseChangeCertificate`

A proper base-change certificate assigns to every object K an inverse $\mathrm{BC}^{-1}_{p,g}(K)$ and records separately the two equations

$$\mathrm{BC}^{-1} \circ \mathrm{BC} = \mathrm{id}, \qquad \mathrm{BC} \circ \mathrm{BC}^{-1} = \mathrm{id},$$

as well as naturality in K, identity-square compatibility, and horizontal and vertical composition compatibility.

Open obligation: `OBL-A1-PROPER-BASE-CHANGE-FINITE-TORSION`

Lemma 10.2.6 (Proper base-change isomorphism from its certificate)**.** *Uses:* `def:weil-a1-proper-base-change-map`, `def:weil-a1-proper-base-change-certificate`

Lean: `Weil.A1.ProperBaseChangeMap.isIso`

Under a proper base-change certificate, $\mathrm{BC}_{p,g}(K)$ is an isomorphism for every K.

Proof. Uses: `def:weil-a1-proper-base-change-map`, `def:weil-a1-proper-base-change-certificate`

The two inverse equations in the certificate are exactly the categorical left- and right-inverse equations; package the inverse candidate and these equations as an isomorphism. □

10.3 Finite-level local data

Definition 10.3.1 (Finite proper base-change comparison datum). Lean: `conditional-interface:def:weil-pbc-followup-finite-comparison-datum`

Uses: `def:weil-a1-proper-base-change-map`

Fix a finite torsion level $n \geq 1$ and a proper Cartesian square $\square = (p, p', g, g', \alpha)$. A finite comparison datum consists of the functorial map

$$\beta_{n,K} : Lg^* Rp_* K \longrightarrow Rp'_* Lg'^* K, \qquad K \in \mathcal{D}_n(T),$$

obtained from the unit–counit formula for the chosen Cartesian identification α. The datum includes its chain-level representative, the localization map, and the equality identifying the two pullback paths. We write β_n for this family of maps.

Definition 10.3.2 (Strict-henselian finite proper certificate). Lean: `conditional-interface:def:weil-pbc-followup-local-certificate`

Uses: `def:weil-pbc-followup-finite-comparison-datum`

For the datum above, a strict-henselian finite proper certificate consists of the following fields for every geometric point $\bar{y} \to Y'$ and every $K \in \mathcal{D}_n(T)$:

1. a selected strict-henselian neighbourhood $Y'_{(\bar{y})}$ and its fibre $T'_{\bar{y}}$;
2. the induced cohomology-stalk map $s_{\bar{y},K} : H^i(\beta_{n,K})_{\bar{y}}$ in every degree i;
3. maps $t_{\bar{y},K} : H^i(Rp'_* Lg'^* K)_{\bar{y}} \to H^i(Lg^* Rp_* K)_{\bar{y}}$;
4. the two displayed equations $t_{\bar{y},K} s_{\bar{y},K} = 1$ and $s_{\bar{y},K} t_{\bar{y},K} = 1$, together with naturality in K;
5. specialization equations saying that the $t_{\bar{y},K}$ commute with every chosen specialization map between geometric points.

The record is a certificate input, not an existence assertion.

Definition 10.3.3 (Finite stalk-descent certificate). Lean: `conditional-interface:def:weil-pbc-followup-stalk-descent`

Uses: `def:weil-pbc-followup-local-certificate`

A finite stalk-descent certificate for Y' assigns to every specialization- compatible family of maps $t_{\bar{y},K}$ a chosen derived morphism

$$\widetilde{t}_K : Rp'_* Lg'^* K \longrightarrow Lg^* Rp_* K$$

whose induced cohomology-stalk map at $\bar{y}$ is $t_{\bar{y},K}$. It also records (a) naturality in K, (b) compatibility with restriction in i, and (c) derived stalk detection: two maps of the bounded complexes are equal when all their induced cohomology-stalk maps agree. Thus the certificate contains the chain-level descent and the detection principle used below. The certificate is the exact descent input needed below; no general claim about all sheaf categories is made.

Lemma 10.3.4 (Local inverse equations are degreewise). Lean: `conditional-interface:lemma:weil-pbc-followup-local-inverse-equations`

Uses: `def:weil-pbc-followup-local-certificate`

For every $\bar{y}$, K, and degree i, the local maps in the certificate satisfy

$$t_{\bar{y},K}\, s_{\bar{y},K} = \mathrm{id}, \qquad s_{\bar{y},K}\, t_{\bar{y},K} = \mathrm{id}\,.$$

Proof. Uses: `def:weil-pbc-followup-local-certificate`

These are fields (4) of the strict-henselian certificate, with the degree index made explicit. No passage between different degrees is used. □

10.4 Assembly of the finite comparison

Definition 10.4.1 (Assembled finite proper certificate). Lean: `conditional-interface:def:weil-pbc-followup-assembled-certificate`

Uses: `def:weil-pbc-followup-finite-comparison-datum,def:weil-pbc-followup-local-certificate,def:weil-pbc-followup-stalk-descent`

An assembled finite proper certificate is a finite comparison datum equipped with both certificates above and, for every K, a chosen inverse candidate $\beta_{n,K}^{\langle -1 \rangle}$ in the derived category. The candidate is part of the record together with the equality

$$H^i(\beta_{n,K}^{\langle -1 \rangle})_{\bar{y}} = t_{\bar{y},K}$$

for every $\bar{y}$ and i.

Lemma 10.4.2 (The assembled inverse is a left inverse). *Lean:* `conditional-interface:lemma:weil-pbc-followup-assembled-left-inverse`

Uses: `def:weil-pbc-followup-assembled-certificate`

For every $K \in \mathcal{D}_n(T)$,

$$\beta_{n,K}^{\langle -1\rangle} \circ \beta_{n,K} = \mathrm{id}_{Lg^* Rp_* K}.$$

Proof. Uses: `def:weil-pbc-followup-assembled-certificate`

Apply H^i and then the geometric stalk at $\bar{y}$. The resulting map is $t_{\bar{y},K} s_{\bar{y},K} = 1$ by the local inverse equation. The derived stalk-detection field of the descent certificate then identifies these stalk equalities with equality of the two derived morphisms. □

Lemma 10.4.3 (The assembled inverse is a right inverse). *Lean:* `conditional-interface:lemma:weil-pbc-followup-assembled-right-inverse`

Uses: `def:weil-pbc-followup-assembled-certificate`

For every $K \in \mathcal{D}_n(T)$,

$$\beta_{n,K} \circ \beta_{n,K}^{\langle -1\rangle} = \mathrm{id}_{Rp'_* Lg'^* K}.$$

Proof. Uses: `def:weil-pbc-followup-assembled-certificate`

The same stalk calculation now gives $s_{\bar{y},K} t_{\bar{y},K} = 1$. Derived stalk detection gives the displayed equality in the derived category. □

Proposition 10.4.4 (Finite proper base change from the assembled certificate). *Lean:* `conditional-interface:prop:weil-pbc-followup-finite-proper-isomorphism`

Uses: `def:weil-pbc-followup-assembled-certificate`

Under an assembled finite proper certificate, $\beta_{n,K}$ *is an isomorphism for every bounded constructible* K*, with inverse* $\beta_{n,K}^{\langle -1\rangle}$.

Proof. Uses: `lemma:weil-pbc-followup-assembled-left-inverse,lemma:weil-pbc-followup-assembled-right-inverse`

The two preceding lemmas are the categorical left- and right-inverse equations. Package the inverse candidate and these equations as an isomorphism. □

10.5 Coherence export to the A1 interface

Definition 10.5.1 (Coherent finite proper certificate family). Lean: `conditional-interface:def:weil-pbc-followup-coherent-family`

Uses: `def:weil-pbc-followup-assembled-certificate`

A coherent finite proper certificate family consists of an assembled certificate at every level n, together with explicit equations for:

1. reduction from level n to level $m \le n$;
2. the identity Cartesian square;
3. horizontal composition of Cartesian squares; and
4. vertical composition of Cartesian squares.

Each equation is an equality of the displayed comparison maps and of their inverse candidates after applying the corresponding reduction functor.

Lemma 10.5.2 (Coherent family supplies A1 naturality). *Lean:* `conditional-interface:lemma:weil-pbc-followup-a1-export-naturality`

Uses: `def:weil-pbc-followup-coherent-family`

The coherent family supplies the naturality field required by `def:weil-a1-proper-base-change-certificate`.

Proof. Uses: `def:weil-pbc-followup-coherent-family`

Use the local naturality equation and the derived stalk-detection field. □

Lemma 10.5.3 (Coherent family supplies A1 identity compatibility). *Lean:* `conditional-interface:lemma:weil-pbc-followup-a1-export-identity`

Uses: `def:weil-pbc-followup-coherent-family`

The coherent family supplies the identity-square compatibility field required by `def:weil-a1-proper-base-change-certificate`.

Proof. Uses: def:weil-pbc-followup-coherent-family
This is the identity equation included in the coherent-family record. □

Lemma 10.5.4 (Coherent family supplies A1 horizontal composition). Lean: conditional-interface:lemma:weil-pbc-followup-a1-export-horizontal

Uses: def:weil-pbc-followup-coherent-family
The coherent family supplies the horizontal-composition field required by def:weil-a1-proper-base-change-certificate.

Proof. Uses: def:weil-pbc-followup-coherent-family
This is the horizontal composition equation included in the record. □

Lemma 10.5.5 (Coherent family supplies A1 vertical composition). Lean: conditional-interface:lemma:weil-pbc-followup-a1-export-vertical

Uses: def:weil-pbc-followup-coherent-family
The coherent family supplies the vertical-composition field required by def:weil-a1-proper-base-change-certificate.

Proof. Uses: def:weil-pbc-followup-coherent-family
This is the vertical composition equation included in the record. □

Proposition 10.5.6 (Exact unresolved finite proper-base-change fragment). Lean: conditional-interface:prop:weil-pbc-followup-open-finite-proper

Uses: def:weil-pbc-followup-local-certificate, def:weil-pbc-followup-stalk-descent
The preceding formal assembly is conditional. The unresolved fragment is the construction of the strict-henselian maps $s_{\bar{y},K}$ and their inverses for all finite constructible K, together with the stated specialization equations and the finite stalk-descent certificate.

Open obligation: OBL-A1-PROPER-BASE-CHANGE-FINITE-TORSION

Open obligation: OBL-PBC-FINITE-STALK-DESCENT

Proof. Uses: def:weil-pbc-followup-local-certificate, def:weil-pbc-followup-stalk-descent
No proof is claimed: these are precisely the two certificate records left as inputs in the definitions. Once they are supplied, the assembly and isomorphism propositions above apply verbatim. □

10.6 Compactification data and refinement comparisons

Definition 10.6.1 (Compactification datum for a separated finite-type map). Uses: def:nagata-compactification, def:weil-a1-torsion-derived-category

Lean: Weil.A1.CompactificationDatum
For $f : X \to Y$, define a compactification datum to be an object

$$C = (\overline{X}, j, p, \alpha), \qquad X \xrightarrow{j} \overline{X} \xrightarrow{p} Y,$$

where j is a quasi-compact open immersion, p is proper, and $\alpha : p \circ j = f$ is an equality. This is the typed projection of the frozen Nagata compactification structure. No existence claim is made here.

Definition 10.6.2 (Selected compactification index). Uses: def:weil-a1-compactification-datum

Lean: Weil.A1.SelectedCompactification
A selected compactification index is the tuple (f, C) with an explicit compactification datum C as in the preceding definition. The selection is an input field; it is never introduced by an unexpanded "choose" in a proof.

Definition 10.6.3 (Compactification candidate at finite level). Uses: def:weil-a1-selected-compactification, def:weil-a1-derived-extension-zero, def:weil-a1-proper-direct-image-input

Lean: Weil.A1.CompactificationCandidate
For $K \in \mathcal{D}_n(X)$ and $C = (\overline{X}, j, p, \alpha)$ define

$$\mathsf{C}_C(K) := Rp_* \, Rj_! K \in \mathcal{D}_n(Y).$$

On a morphism $u : K \to L$, define $\mathsf{C}_C(u) := Rp_* Rj_!(u)$.

Lemma 10.6.4 (Functoriality of the compactification candidate). *Uses:* *def:weil-a1-compactification-candidate*

Lean: *Weil.A1.CompactificationCandidate.functorial*

The assignment $K \mapsto \mathsf{C}_C(K)$ is a functor and sends identity and composition to identity and composition.

Proof. Uses: def:weil-a1-compactification-candidate,lemma:weil-a1-extension-zero-functorial,lemma:weil-a1-proper-direct-image-functorial

Compose the two functors $Rj_!$ and Rp_*. Their identity and composition equations give the claimed equations term by term. □

Definition 10.6.5 (Morphism of compactifications). Uses: def:weil-a1-compactification-datum

Lean: Weil.A1.CompactificationMorphism

For compactifications $C_a = (\overline{X}_a, j_a, p_a, \alpha_a)$ of the same f, a morphism $q : C_1 \to C_2$ consists of a proper map $q : \overline{X}_1 \to \overline{X}_2$ and equalities

$$q \circ j_1 = j_2, \qquad p_2 \circ q = p_1,$$

whose two equalities are stored as fields.

Definition 10.6.6 (Proper-modification extension comparison). Uses: def:weil-a1-compactification-morphism,def:weil-a1-derived-extension-zero,def:weil-a1-proper-direct-image-input

Lean: Weil.A1.ModificationExtensionComparison

For $q : C_1 \to C_2$ define the sheaf-level map

$$\gamma_{q,K} : Rq_* Rj_{1!} K \longrightarrow Rj_{2!} K$$

as follows. On a resolved representative, a local section of $j_{1!}K$ has a support-closed representative in $\overline{X}_1 \times_{\overline{X}_2} V$. Push it along q; the equality $qj_1 = j_2$ makes its support lie in the open part of V, and extension by zero gives a section of $j_{2!}K$. The local maps agree after restriction, hence sheafify, and the chosen derived resolution totalizes them.

Lemma 10.6.7 (Modification comparison restriction). *Uses:* *def:weil-a1-modification-extension-comparison*, *def:weil-a1-compactification-morphism*

Lean: *Weil.A1.ModificationExtensionComparison.restrict*

After applying j_2^*, the map $\gamma_{q,K}$ is the proper direct-image map induced by the identity $q|_X = \mathrm{id}_X$.

Proof. Uses: def:weil-a1-modification-extension-comparison,def:weil-a1-compactification-morphism

On an object over U, both $j_{1!}K$ and $j_{2!}K$ restrict to K by the open-restriction lemma. The equation $qj_1 = j_2$ identifies the two pullbacks, so the local formula is the identity on K. □

Lemma 10.6.8 (Modification comparison on the boundary). *Uses:* *def:weil-a1-modification-extension-comparison*, *def:weil-a1-open-closed-complement*

Lean: *Weil.A1.ModificationExtensionComparison.boundary*

For a geometric point $\bar{y} \to \overline{X}_2$ outside X, the stalk of $Rq_* Rj_{1!}K$ at $\bar{y}$ is the cohomology of the proper fiber $\overline{X}_{1,\bar{y}}$ with coefficients in the restriction of $Rj_{1!}K$, and that restriction is zero.

Proof. Uses: def:weil-a1-modification-extension-comparison,def:weil-a1-open-closed-complement

The proper-base-change stalk formula identifies the stalk with the derived sections of the fiber. Since $q^{-1}(X) = X$ by $qj_1 = j_2$, the fiber lies in the closed complement of U; the stalk formula for extension by zero makes the coefficient complex zero on that fiber.

Open obligation: OBL-A1-MODIFICATION-BOUNDARY-STALK-PROPER-BASE-CHANGE

□

Lemma 10.6.9 (Modification comparison isomorphism). *Uses:* *def:weil-a1-modification-extension-comparison*

Lean: *Weil.A1.ModificationExtensionComparison.isIso*

The map $\gamma_{q,K}$ is an isomorphism.

Proof. Uses: lemma:weil-a1-modification-extension-restricts,lemma:weil-a1-modification-extension-boundary-zero

At points in X the map is the identity on stalks. At points in the boundary both source and target stalks are zero by the preceding lemma. The enough-geometric-points criterion therefore gives an isomorphism of cohomology sheaves, hence an isomorphism in the bounded derived category.

Open obligation: OBL-A1-MODIFICATION-COMPARISON-ISOMORPHISM

□

Definition 10.6.10 (Compactification comparison map). Uses: def:weil-a1-compactification-candidate, def:weil-a1-modification-extension-comparison, def:weil-a1-compactification-morphism

Lean: Weil.A1.CompactificationComparison

For $q : C_1 \to C_2$ define

$$c_q(K) : \mathsf{C}_{C_1}(K) = Rp_{1*}Rj_{1!}K \longrightarrow Rp_{2*}Rj_{2!}K = \mathsf{C}_{C_2}(K)$$

by the equality $p_1 = p_2 q$, the composition comparison for proper pushforward, and $Rp_{2*}(\gamma_{q,K})$.

Lemma 10.6.11 (Compactification comparison isomorphism). Uses: def:weil-a1-compactification-comparison-map

Lean: Weil.A1.CompactificationComparison.isIso

The map $c_q(K)$ is an isomorphism.

Proof. Uses: def:weil-a1-compactification-comparison-map, lemma:weil-a1-modification-extension-isomorphism

The composition comparison identifies $c_q(K)$ with $Rp_{2*}(\gamma_{q,K})$. The latter is an isomorphism because $\gamma_{q,K}$ is an isomorphism and Rp_{2*} is a functor. □

Lemma 10.6.12 (Identity comparison). Uses: def:weil-a1-compactification-comparison-map, def:weil-a1-compactification-morphism

Lean: Weil.A1.CompactificationComparison.id

For the identity morphism of a compactification, c_{id} is the identity natural transformation.

Proof. Uses: def:weil-a1-compactification-comparison-map, def:weil-a1-compactification-morphism

All maps in the defining composite are identity maps: $p \circ \mathrm{id} = p$ and the local extension comparison is the identity on every support-closed representative. □

Lemma 10.6.13 (Composition of refinement comparisons). Uses: def:weil-a1-compactification-comparison-map, def:weil-a1-compactification-morphism

Lean: Weil.A1.CompactificationComparison.comp

For $C_1 \xrightarrow{q_{12}} C_2 \xrightarrow{q_{23}} C_3$,

$$c_{q_{23}q_{12}} = c_{q_{23}} \circ c_{q_{12}}.$$

Proof. Uses: def:weil-a1-compactification-comparison-map, lemma:weil-a1-proper-direct-image-functorial

Expand both sides as composites of Rp_*, the proper composition comparison, and γ. On a local support-closed representative the two maps both push the same section first along q_{12} and then along q_{23}. Equality before sheafification and functoriality of totalization give the displayed equality. □

10.7 Common refinements and compactification independence

Definition 10.7.1 (Product-closure refinement candidate). Uses: def:weil-a1-compactification-datum

Lean: Weil.A1.ProductClosureRefinement

For compactifications C_1, C_2 of f, form $\overline{X}_1 \times_Y \overline{X}_2$. Let $\widetilde{X}$ be the scheme-theoretic closure of the image of

$$X \xrightarrow{(j_1,j_2)} \overline{X}_1 \times_Y \overline{X}_2.$$

Let $q_a : \widetilde{X} \to \overline{X}_a$ be the restricted projections and let $j_{12} : X \to \widetilde{X}$ be the induced open map candidate. All three maps and the closure ideal are part of this candidate.

Lemma 10.7.2 (Product-closure restriction to the source). Uses: def:weil-a1-product-closure-refinement

Lean: Weil.A1.ProductClosureRefinement.source

The equations $q_a \circ j_{12} = j_a$ and $p_a \circ q_a = \widetilde{p}$ hold, where $\widetilde{p} := p_1 q_1 = p_2 q_2$.

Proof. Uses: def:weil-a1-product-closure-refinement

The equations hold on the dense image of X by construction of the graph map. The closure is defined as the smallest closed subscheme through which the graph factors, so each projection restricts to the displayed map.

Open obligation: OBL-A1-PRODUCT-CLOSURE-MAP-ASSEMBLY

□

Lemma 10.7.3 (Product-closure open and proper properties). *Uses:* def:weil-a1-product-closure-refinement

Lean: Weil.A1.ProductClosureRefinement.isCompactification

The map j_{12} is a quasi-compact open immersion and $\widetilde{p}\colon \widetilde{X} \to Y$ is proper. The maps q_1, q_2 are proper.

Proof. Uses: def:weil-a1-product-closure-refinement,lemma:weil-a1-product-closure-source

The product $\overline{X}_1 \times_Y \overline{X}_2 \to Y$ is proper as a base change and composition of proper morphisms. A closed subscheme of a proper scheme over Y is proper, giving $\widetilde{p}$. Each q_a is the composite of the closed immersion with a projection, hence proper. The assertion that the graph remains an open immersion is the scheme-theoretic closure calculation on the open locus; quasi-compactness is inherited from j_a.

Open obligation: OBL-A1-PRODUCT-CLOSURE-OPEN-IMMERSION

□

Proposition 10.7.4 (Compactifications are cofiltered). *Uses:* def:weil-a1-product-closure-refinement

Lean: Weil.A1.Compactifications.cofiltered

Any two compactifications of f admit a compactification C_{12} and refinement morphisms $C_{12} \to C_1$ and $C_{12} \to C_2$.

Proof. Uses: def:weil-a1-product-closure-refinement,lemma:weil-a1-product-closure-source,lemma:weil-a1-product-closure-properties

Take the product-closure candidate. The source and properness equations are the preceding lemmas, and the two projections supply the refinement morphisms.

Open obligation: OBL-A1-COMPACTIFICATIONS-COFILTERED

□

Lemma 10.7.5 (Independence of the common refinement). *Uses:* def:weil-a1-compactification-morphism,def:weil-a1-compactification-comparison-map

Lean: Weil.A1.Compactifications.common_refinement_independent

If D and D' both dominate C_1 and C_2, then the two composites

$$\mathsf{C}_{C_1}(K) \to \mathsf{C}_D(K) \to \mathsf{C}_{C_2}(K), \qquad \mathsf{C}_{C_1}(K) \to \mathsf{C}_{D'}(K) \to \mathsf{C}_{C_2}(K)$$

are equal.

Proof. Uses: def:weil-a1-compactification-morphism,def:weil-a1-compactification-comparison-map,prop:weil-a1-compactifications-cofiltered,lemma:weil-a1-compactification-comparison-composition

Choose a third refinement E dominating D and D' by the cofilteredness proposition. By the comparison-composition lemma, both displayed composites become the same composite through E. Since all comparison maps are isomorphisms, cancel the common postcomposition and obtain equality already between the two displayed maps.

Open obligation: OBL-A1-COMMON-REFINEMENT-CANCELLATION

□

Definition 10.7.6 (Canonical comparison family). Uses: def:weil-a1-compactification-morphism,def:weil-a1-compactification-comparison-map

Lean: Weil.A1.CanonicalComparisonFamily

For compactifications C_1, C_2 define $\kappa_{C_1,C_2}(K) : \mathsf{C}_{C_1}(K) \to \mathsf{C}_{C_2}(K)$ by choosing the product-closure refinement D and setting

$$\kappa_{C_1,C_2}(K) := c_{D\to C_2}(K) \circ c_{D\to C_1}(K)^{-1}.$$

The inverse in this formula is the inverse supplied by the proved comparison-isomorphism property, and the product-closure candidate is the explicit selected construction of the preceding definition.

Lemma 10.7.7 (Well-definedness certificate for def:weil-a1-canonical-comparison-family). *Lean:* conditional-interface:lemma:weil-sf-well-defined-def-weil-a1-canonical-comparison-family

Uses: def:weil-a1-canonical-comparison-family

The construction in Definition 10.7.6 is a typed raw datum. Its theorem-valued naturality, comparison, descent, or coherence fields are isolated in this property lemma.

Proof. Uses: `prop:weil-a1-compactifications-cofiltered`

The displayed fields are well-defined precisely when the cited dependencies supply the corresponding equations; assembling those equations into the stated property is the remaining leaf.

Open obligation: `OBL-SF-WELLDEFINED-DEF-WEIL-A1-CANONICAL-COMPARISON-FAMILY`

□

Lemma 10.7.8 (Canonical comparison is independent of the chosen refinement)**.** *Uses:* `def:weil-a1-canonical-comparison-family`

Lean: `Weil.A1.CanonicalComparisonFamily.independent`

The map $\kappa_{C_1,C_2}(K)$ is independent of the selected common refinement, and is natural in K.

Proof. Uses: `def:weil-a1-canonical-comparison-family,lemma:weil-a1-common-refinement-independence`

The two choices become equal after passing to a common third refinement. The comparison-composition lemma then gives the same composite; cancellation by the isomorphism from the first refinement proves equality. Naturality follows by applying each comparison to a morphism $K \to L$.

Open obligation: `OBL-A1-CANONICAL-COMPARISON-INDEPENDENCE`

□

Lemma 10.7.9 (Canonical comparison identity and cocycle)**.** *Uses:* `def:weil-a1-canonical-comparison-family`

Lean: `Weil.A1.CanonicalComparisonFamily.cocycle`

For all compactifications C_1, C_2, C_3,

$$\kappa_{C,C} = \mathrm{id}, \qquad \kappa_{C_2,C_3} \circ \kappa_{C_1,C_2} = \kappa_{C_1,C_3}.$$

Proof. Uses: `def:weil-a1-canonical-comparison-family,lemma:weil-a1-canonical-comparison-independent,lemma:weil-a1-compactification-comparison-identity,lemma:weil-a1-compactification-comparison-composition`

Compute all three maps through one common refinement dominating the three compactifications. The identity case reduces to the identity comparison; the cocycle case is cancellation in the chain of three comparison isomorphisms. □

10.8 Definition and composition of $Rf_!$

Definition 10.8.1 (Exceptional direct image at a selected compactification)**.** Uses: `def:weil-a1-selected-compactification,def:weil-a1-compactification-candidate`

Lean: `Weil.A1.RfShriekSelected`

For a selected index (f, C) define

$$Rf_{!,C} := \mathsf{C}_C : \mathcal{D}_n(X) \longrightarrow \mathcal{D}_n(Y).$$

Definition 10.8.2 (Choice comparison for exceptional direct image)**.** Uses: `def:weil-a1-rfshriek-selected,def:weil-a1-canonical-comparison-family`

Lean: `Weil.A1.RfShriekChoiceComparison`

For two selected compactifications C, C' of f, define the natural isomorphism

$$\kappa_{C,C'} : Rf_{!,C} \xrightarrow{\sim} Rf_{!,C'}$$

whose component at K is the canonical comparison family.

Lemma 10.8.3 (Choice comparison identity and cocycle)**.** *Uses:* `def:weil-a1-rfshriek-choice-comparison`

Lean: `Weil.A1.RfShriekChoiceComparison.cocycle`

For selected compactifications C_1, C_2, C_3,

$$\kappa_{C,C} = \mathrm{id}, \qquad \kappa_{C_2,C_3} \circ \kappa_{C_1,C_2} = \kappa_{C_1,C_3}.$$

Proof. Uses: `def:weil-a1-rfshriek-choice-comparison,lemma:weil-a1-canonical-comparison-coherence`

Evaluate at K. The assertion is exactly the identity and cocycle equations for $\kappa_{C_i,C_j}(K)$, and naturality follows componentwise. □

Proposition 10.8.4 (Compactification independence). *Uses:* *def:weil-a1-rfshriek-selected, def:weil-a1-rfshriek-choice-comparison*

Lean: *Weil.A1.RfShriek.independent*

The family $\{Rf_{!,C}\}_C$ together with the maps $\kappa_{C,C'}$ is a canonical pseudofunctorial object: changing the selected compactification changes the result by a unique coherent natural isomorphism.

Proof. Uses: def:weil-a1-rfshriek-choice-comparison,lemma:weil-a1-rfshriek-choice-comparison-cocycle

The two inverse equations for each κ are supplied by the comparison isomorphism. The preceding cocycle is precisely the coherence condition for the family, and uniqueness follows by cancellation after a common refinement.

Open obligation: OBL-A1-RFSHRIEK-INDEPENDENCE

□

Definition 10.8.5 (Successive compactification for a composite). Uses: def:weil-a1-compactification-datum

Lean: Weil.A1.SuccessiveCompositeCompactification

For $X \xrightarrow{f} Y \xrightarrow{g} Z$ and compactifications $C_f = (\overline{X}, j_f, p_f)$, $C_g = (\overline{Y}, j_g, p_g)$, define the successive compactification candidate by first applying j_f and then the open immersion j_g after the proper map p_f. Its ambient scheme is the closure of X in $\overline{X} \times_Y \overline{Y}$, with structure map $p_g p_f$ and source map induced by the diagonal graph.

Lemma 10.8.6 (Successive candidate is a compactification). *Uses:* *def:weil-a1-successive-composite-compactification, def:weil-a1-compactification-datum*

Lean: *Weil.A1.SuccessiveCompositeCompactification.valid*

The successive candidate has an open source map and a proper structure map, and its composite with the source map equals $g \circ f$.

Proof. Uses: def:weil-a1-successive-composite-compactification,def:weil-a1-compactification-datum

The source equality follows by substitution of $p_f j_f = f$ and $p_g j_g = g$. Properness is stable under composition and base change; the source map is the product-closure open map. The remaining open assertion is exactly the product-closure obligation from the common-refinement construction.

Open obligation: OBL-A1-SUCCESSIVE-COMPOSITE-OPEN-PROPER

□

Definition 10.8.7 (Exceptional composition morphism). Uses: def:weil-a1-rfshriek-selected, def:weil-a1-successive-composite-compactification,def:weil-a1-compactification-candidate

Lean: Weil.A1.RfShriekCompositionMap

For selected compactifications of f and g, and a selected compactification C_{gf} of gf, define

$$\mathrm{comp}_{g,f} : Rg_{!,C_g} Rf_{!,C_f} \longrightarrow R(gf)_{!,C_{gf}}$$

by identifying the left side with the candidate attached to the successive compactification, using the composition comparison for proper pushforwards, and then applying the canonical comparison from that candidate to C_{gf}.

Definition 10.8.8 (Exceptional composition inverse candidate). Uses: def:weil-a1-rfshriek-composition-map

Lean: Weil.A1.RfShriekCompositionInverseCandidate

Define the reverse candidate by using the inverse of the canonical comparison from C_{gf} to the successive compactification and the inverse of the proper-pushforward composition comparison. All inverse maps are attached to the corresponding explicit isomorphism certificates; no inverse is selected inside a proof.

Lemma 10.8.9 (Exceptional composition left inverse). *Uses:* *def:weil-a1-rfshriek-composition-map, def:weil-a1-rfshriek-composition-inverse-candidate*

Lean: *Weil.A1.RfShriekComposition.left_inverse*

The reverse candidate composed after $\mathrm{comp}_{g,f}$ is the identity.

Proof. Uses: def:weil-a1-rfshriek-composition-map,def:weil-a1-rfshriek-composition-inverse-candidate,lemma:weil-a1-canonical-comparison-coherence,lemma:weil-a1-proper-direct-image-functorial

Expand the two composites. Adjacent comparison isomorphisms cancel, and the two proper composition maps cancel by their inverse equations. The remaining map is the identity on the successive candidate.

Open obligation: OBL-A1-RFSHRIEK-COMPOSITION-LEFT-INVERSE

□

Lemma 10.8.10 (Exceptional composition right inverse). *Uses:* *def:weil-a1-rfshriek-composition-map*, *def:weil-a1-rfshriek-composition-inverse-candidate*

Lean: *Weil.A1.RfShriekComposition.right_inverse*

The composition of $\mathrm{comp}_{g,f}$ *after the reverse candidate is the identity.*

Proof. Uses: def:weil-a1-rfshriek-composition-map, def:weil-a1-rfshriek-composition-inverse-candidate, lemma:weil-a1-canonical-comparison-coherence, lemma:weil-a1-proper-direct-image-functorial

The same expansion, in the opposite order, leaves the identity on the composite candidate. This is a separate equation and does not follow merely from the left-inverse calculation.

Open obligation: OBL-A1-RFSHRIEK-COMPOSITION-RIGHT-INVERSE

□

Proposition 10.8.11 (Exceptional composition isomorphism). *Uses:* *def:weil-a1-rfshriek-composition-map*, *def:weil-a1-rfshriek-composition-inverse-candidate*

Lean: *Weil.A1.RfShriekComposition.isIso*

The map $\mathrm{comp}_{g,f}$ *is a natural isomorphism.*

Proof. Uses: lemma:weil-a1-rfshriek-composition-left-inverse, lemma:weil-a1-rfshriek-composition-right-inverse

Package the two displayed inverse equations as the isomorphism data. □

Lemma 10.8.12 (Unit for the identity morphism). *Uses:* *def:weil-a1-rfshriek-selected*, *def:weil-a1-rfshriek-choice-comparison*

Lean: *Weil.A1.RfShriek.unit*

For the identity morphism 1_X*, select the compactification* $X \xrightarrow{\mathrm{id}} X \xrightarrow{\mathrm{id}} X$*. The resulting* $R(1_X)_!$ *is naturally isomorphic to* $\mathrm{id}_{\mathcal{D}_n(X)}$*.*

Proof. Uses: def:weil-a1-rfshriek-selected, def:weil-a1-compactification-candidate, lemma:weil-a1-extension-zero-open-restriction

The open extension for the identity is the identity functor and the proper pushforward for the identity is the identity. Their composite is the identity on every object and arrow. □

Lemma 10.8.13 (Associativity of exceptional composition). *Uses:* *def:weil-a1-rfshriek-composition-map*

Lean: *Weil.A1.RfShriek.associativity*

For composable f, g, h*, the two natural transformations from* $Rh_!Rg_!Rf_!$ *to* $R(hgf)_!$ *obtained by parenthesizing the composite in the two possible ways are equal.*

Proof. Uses: def:weil-a1-rfshriek-composition-map, lemma:weil-a1-compactification-comparison-composition, lemma:weil-a1-proper-direct-image-functorial, prop:weil-a1-rfshriek-composition-isomorphism

Choose one compactification of each of f, g, h and one common refinement of all induced composite compactifications. Expand both transformations into proper composition maps and refinement comparisons. Proper composition is associative on the triple total complex, and all refinement comparisons agree by the cocycle lemma; hence the two expansions are equal.

Open obligation: OBL-A1-RFSHRIEK-ASSOCIATIVITY

□

10.9 Compact-support base change

Definition 10.9.1 (Base-change index for a compactifiable morphism). Uses: def:weil-a1-compactification-datum

Lean: Weil.A1.CompactSupportBaseChangeIndex

Define an index to be a Cartesian square

$$\begin{array}{ccc} X' & \xrightarrow{g'} & X \\ {\scriptstyle f'}\downarrow & & \downarrow{\scriptstyle f} \\ Y' & \xrightarrow{g} & Y \end{array}$$

with selected compactifications C_f and $C_{f'}$ and an explicit identification of $C_{f'}$ with the base change of C_f.

Definition 10.9.2 (Compact-support base-change map). Uses: def:weil-a1-compact-support-base-change-index,def:weil-a1-rfshriek-selected,def:weil-a1-proper-base-change-map

Lean: Weil.A1.CompactSupportBaseChangeMap

For $K \in \mathcal{D}_n(X)$ define

$$\mathrm{BC}_{!,f,g}(K) : Lg^* Rf_{!,C_f} K \longrightarrow Rf'_{!,C_{f'}} Lg'^* K$$

as the composite

$$Lg^* Rp_* Rj_! K \xrightarrow{\mathrm{BC}_{p,g}(Rj_! K)} Rp'_* Lg_X^* Rj_! K \xrightarrow{Rp'_* \beta_{j,g}} Rp'_* Rj'_! Lg'^* K,$$

where $\beta_{j,g}$ is the open-extension base-change map induced by the defining presheaf and the chosen identification of the pulled-back open.

Definition 10.9.3 (Compact-support base-change inverse candidate). Uses: def:weil-a1-compact-support-base-change-map,def:weil-a1-proper-base-change-certificate

Lean: Weil.A1.CompactSupportBaseChangeInverseCandidate

Define the inverse candidate as the composite of the inverse of $Rp'_*\beta_{j,g}$ and the inverse proper-base-change morphism, with the source and target exchanged. The inverse maps are fields of the respective comparison certificates.

Lemma 10.9.4 (Compact-support base-change left inverse). Uses: def:weil-a1-compact-support-base-change-map, def:weil-a1-compact-support-base-change-inverse

Lean: Weil.A1.CompactSupportBaseChange.left_inverse

The inverse candidate followed by $\mathrm{BC}_{!,f,g}$ *is the identity.*

Proof. Uses: def:weil-a1-compact-support-base-change-map,def:weil-a1-compact-support-base-change-inverse,lemma:weil-a1-proper-base-change-isomorphism

Substitute the two composites and cancel the proper base-change map with its inverse. The open-extension base-change map cancels with its inverse on each presheaf section; sheafification and totalization preserve the equality.

Open obligation: OBL-A1-COMPACT-SUPPORT-BASE-CHANGE-LEFT

□

Lemma 10.9.5 (Compact-support base-change right inverse). Uses: def:weil-a1-compact-support-base-change-map, def:weil-a1-compact-support-base-change-inverse

Lean: Weil.A1.CompactSupportBaseChange.right_inverse

The composite of $\mathrm{BC}_{!,f,g}$ *after the inverse candidate is the identity.*

Proof. Uses: def:weil-a1-compact-support-base-change-map,def:weil-a1-compact-support-base-change-inverse,lemma:weil-a1-proper-base-change-isomorphism

The opposite cancellation is checked separately on a local representative and then on the proper derived resolution. This proves the second inverse equation and does not rely on the first one.

Open obligation: OBL-A1-COMPACT-SUPPORT-BASE-CHANGE-RIGHT

□

Proposition 10.9.6 (Compact-support base-change isomorphism). Uses: def:weil-a1-compact-support-base-change-map, def:weil-a1-compact-support-base-change-inverse

Lean: Weil.A1.CompactSupportBaseChange.isIso

The compact-support base-change map is a natural isomorphism.

Proof. Uses: lemma:weil-a1-compact-support-base-change-left-inverse,lemma:weil-a1-compact-support-base-change-right-inverse

Package the two inverse equations into the isomorphism structure. □

Lemma 10.9.7 (Base-change compatibility with refinement comparisons). Uses: def:weil-a1-canonical-comparison-family, def:weil-a1-compact-support-base-change-map

Lean: Weil.A1.CompactSupportBaseChange.refinement_naturality

For a morphism of compactifications $q : C_1 \to C_2$*, the square formed by* κ_{C_1,C_2} *and the two compact-support base-change maps commutes.*

Proof. Uses: def:weil-a1-canonical-comparison-family,def:weil-a1-compact-support-base-change-map,lemma:weil-a1-proper-base-change-isomorphism,prop:weil-a1-compact-support-base-change-isomorphism

Expand the square into the proper base-change exchange maps and the extension-by-zero exchange maps. Both paths are the same map on a local support-closed representative; the proper base-change cocycle and the extension base-change cocycle then identify the derived maps.

Open obligation: OBL-A1-BASE-CHANGE-REFINEMENT-COHERENCE

□

Proposition 10.9.8 (Canonical compact-support base change). *Uses: def:weil-a1-compact-support-base-change-map*

Lean: Weil.A1.RfShriek.base_change

The base-change isomorphism is independent of the selected compactifications and therefore defines a canonical natural isomorphism

$$Lg^{*}Rf_{!} \xrightarrow{\sim} Rf'_{!}Lg'^{*}.$$

Proof. Uses: prop:weil-a1-rfshriek-independent,lemma:weil-a1-base-change-refinement-compatibility,prop:weil-a1-compact-support-base-change-isomorphism

Replace either selected compactification by a common refinement. The refinement-naturality square shows that the resulting base-change maps agree; the identity and cocycle equations make the agreement transitive. Thus the map descends from selected indices to the canonical $Rf_{!}$ family.

Open obligation: OBL-A1-RFSHRIEK-CANONICAL-BASE-CHANGE

□

Proposition 10.9.9 (A1 compact-support interface). *Uses: def:weil-a1-rfshriek-selected*

Lean: Weil.A1.LowerShriekInterface

The preceding constructions provide, subject exactly to the listed OPEN obligations, a compact-support operation $Rf_{!}$ on finite torsion constructible complexes, its compactification-independent comparison family, composition and unit isomorphisms, associativity, and Cartesian base change.

Proof. Uses: def:weil-a1-rfshriek-selected,prop:weil-a1-rfshriek-independent,prop:weil-a1-rfshriek-composition-isomorphism,lemma:weil-a1-rfshriek-unit,lemma:weil-a1-rfshriek-associativity,prop:weil-a1-rfshriek-base-change

This is a field-by-field assembly: the operation is Definition 10.8.1; independence is the preceding proposition; composition, unit, associativity, and base change are supplied by their separate results. No additional property is introduced in this final assembly. □

10.10 The pinned integral coefficient ring

Definition 10.10.1 (Pinned additive coefficient sheaf). Lean: conditional-interface:def:weil-a2-pinned-additive-sheaf

Uses: def:weil-integral-proetale-coefficient-sheaf

For a scheme X and a prime ℓ, write

$$\mathcal{Z}_{\ell,X} := \texttt{AlgebraicGeometry.Scheme.ellAdicSheaf}(X,\ell).$$

Write $\Lambda := \mathbf{Z}_{\ell}$.

Definition 10.10.2 (Continuous-function ring presheaf). Lean: conditional-interface:def:weil-a2-continuous-ring-presheaf

Uses: def:weil-small-proetale-category,def:weil-small-proetale-forgetful-functor,def:weil-coefficient-data

For $U \in X_{\mathrm{proet}}$, define

$$\Lambda_X^{\mathrm{pre}}(U) := C(|U|,\Lambda)$$

with pointwise $0, 1, +, -, \cdot$. For $v : V \to U$, define

$$v^{\sharp}(a) := a \circ |v|.$$

Lemma 10.10.3 (Restrictions preserve multiplication). *Lean: conditional-interface:lemma:weil-a2-continuous-ring-restriction-mul*

Uses: def:weil-a2-continuous-ring-presheaf

For $a, b \in \Lambda_X^{\mathrm{pre}}(U)$, $v^{\sharp}(ab) = v^{\sharp}(a)v^{\sharp}(b)$.

Proof. Uses: def:weil-a2-continuous-ring-presheaf
At $x \in |V|$, both sides have value $a(|v|(x))b(|v|(x))$. □

Lemma 10.10.4 (Restrictions preserve zero and one). Lean: conditional-interface:lemma:weil-a2-continuous-ring-restriction-zero-one

Uses: def:weil-a2-continuous-ring-presheaf
One has $v^\sharp(0) = 0$ and $v^\sharp(1) = 1$.

Proof. Uses: def:weil-a2-continuous-ring-presheaf
Precomposition preserves constant functions, once for the constant 0 and once for the constant 1. □

Lemma 10.10.5 (Restrictions preserve addition and negation). Lean: conditional-interface:lemma:weil-a2-continuous-ring-restriction-add-neg

Uses: def:weil-a2-continuous-ring-presheaf
For $a, b \in \Lambda_X^{\mathrm{pre}}(U)$,

$$v^\sharp(a+b) = v^\sharp(a) + v^\sharp(b), \qquad v^\sharp(-a) = -v^\sharp(a).$$

Proof. Uses: def:weil-a2-continuous-ring-presheaf
Evaluate both equations at $x \in |V|$. They reduce respectively to addition and negation in $\mathbf{Z}_\ell$. □

Lemma 10.10.6 (Restriction identity law). Lean: conditional-interface:lemma:weil-a2-continuous-ring-restriction-id

Uses: def:weil-a2-continuous-ring-presheaf
The restriction along id_U is the identity.

Proof. Uses: def:weil-a2-continuous-ring-presheaf
For a section a, $a \circ |\,\mathrm{id}_U\,| = a$. □

Lemma 10.10.7 (Restriction composition law). Lean: conditional-interface:lemma:weil-a2-continuous-ring-restriction-comp

Uses: def:weil-a2-continuous-ring-presheaf
For $W \xrightarrow{w} V \xrightarrow{v} U$, $(v \circ w)^\sharp = w^\sharp v^\sharp$.

Proof. Uses: def:weil-a2-continuous-ring-presheaf
Both sides send a to $a \circ |v| \circ |w|$. □

Proposition 10.10.8 (Continuous functions form a ring presheaf). Lean: conditional-interface:prop:weil-a2-continuous-ring-presheaf

Uses: def:weil-a2-continuous-ring-presheaf
The preceding data define a presheaf of commutative rings.

Proof. Uses: lemma:weil-a2-continuous-ring-restriction-mul, lemma:weil-a2-continuous-ring-restriction-zero-one, lemma:weil-a2-continuous-ring-restriction-add-neg, lemma:weil-a2-continuous-ring-restriction-id, lemma:weil-a2-continuous-ring-restriction-comp
The first three lemmas say each restriction is a unital ring homomorphism. The last two are the contravariant functor laws. □

Definition 10.10.9 (Ring-valued continuous-map sheaf on the source site). Lean: conditional-interface:def:weil-a2-continuous-ring-source-sheaf

Uses: def:weil-a2-continuous-ring-presheaf
Let $\mathcal{C}_\Lambda^{\mathrm{ring}}$ be continuousMapPresheaf(Λ), equipped with the CRing-valued sheaf structure obtained by the same fpqc descent proof as the additive object.

Proposition 10.10.10 (The ring-valued continuous presheaf is a sheaf). Lean: conditional-interface:prop:weil-a2-continuous-ring-source-is-sheaf

Uses: def:weil-a2-continuous-ring-source-sheaf
The presheaf $\mathcal{C}_\Lambda^{\mathrm{ring}}$ satisfies the fpqc sheaf condition, hence also the topology used by sheafPushforwardContinuous.

Proof. Uses: `prop:weil-a2-continuous-ring-presheaf`

The pinned mathlib theorem `isSheaf_fpqcTopology_continuousMapPresheafAb` proves the matching and unique-gluing statements for the underlying additive maps. For a matching family, the glued additive section is pointwise the unique glued continuous function. Multiplication and the unit are determined pointwise, so their restrictions agree with the corresponding operations on every member of the cover. Additive-sheaf uniqueness therefore forces the glued section to preserve multiplication and 1. This supplies the ring sheaf property without a second sheafification. □

Definition 10.10.11 (Integral ring sheaf on the pro-etale site)**.** Lean: `conditional-interface:def:weil-a2-integral-ring-sheaf`

Uses: `def:weil-a2-continuous-ring-source-sheaf`, `def:weil-small-proetale-forgetful-functor`

Define

$$\Lambda_X := \Big((\mathrm{ProEt.forget}\, X \circ \mathrm{Over.forget}).\texttt{sheafPushforwardContinuous}\Big)\big(\mathcal{C}_\Lambda^{\mathrm{ring}}\big).$$

The topology and continuity arguments are the same ones appearing in the pinned definition of $\mathcal{Z}_{\ell,X}$.

Definition 10.10.12 (Underlying-additive comparison candidate)**.** Lean: `conditional-interface:def:weil-a2-underlying-additive-comparison`

Uses: `def:weil-a2-integral-ring-sheaf`, `def:weil-a2-pinned-additive-sheaf`

Apply the natural isomorphism `continuousMapPresheafAbForgetIso` before pushforward and then push it forward. Define the resulting morphism

$$c_X : \mathrm{Forget}_{\mathrm{CRing}\to\mathrm{Ab}}(\Lambda_X) \longrightarrow \mathcal{Z}_{\ell,X}.$$

Define d_X by pushing forward its inverse.

Lemma 10.10.13 (Comparison object components)**.** Lean: *`conditional-interface:lemma:weil-a2-underlying-comparison-components`*

Uses: *`def:weil-a2-underlying-additive-comparison`*

On $U \in X_{\mathrm{proet}}$, the component $c_{X,U}$ sends a bundled continuous function to the identical bundled additive continuous function, and $d_{X,U}$ performs the inverse rebundling.

Proof. Uses: `def:weil-a2-underlying-additive-comparison`

Unfold the exact source definitions `Scheme.ellAdicSheaf`, `continuousMapPresheaf_obj`, and `continuousMapPresheafAbForgetIso`. The functor `sheafPushforwardContinuous` acts on object components by precomposition, so it does not alter the bundled function. □

Lemma 10.10.14 (Comparison naturality)**.** Lean: *`conditional-interface:lemma:weil-a2-underlying-comparison-naturality`*

Uses: *`def:weil-a2-underlying-additive-comparison`*

For $v : V \to U$, the squares for c_X and d_X commute.

Proof. Uses: `lemma:weil-a2-underlying-comparison-components`

Both paths in either square send a continuous function a to $a \circ |v|$. This is the exact simp equation `continuousMapPresheaf_map`. □

Lemma 10.10.15 (First comparison inverse equation)**.** Lean: *`conditional-interface:lemma:weil-a2-underlying-comparison-left-inverse`*

Uses: *`def:weil-a2-underlying-additive-comparison`*

One has $d_X c_X = \mathrm{id}$.

Proof. Uses: `lemma:weil-a2-underlying-comparison-components`

Objectwise, the composite forgets and then restores the same ring bundle. The resulting function and structure are unchanged. □

Lemma 10.10.16 (Second comparison inverse equation)**.** Lean: *`conditional-interface:lemma:weil-a2-underlying-comparison-right-inverse`*

Uses: *`def:weil-a2-underlying-additive-comparison`*

One has $c_X d_X = \mathrm{id}$.

Proof. Uses: `lemma:weil-a2-underlying-comparison-components`

Objectwise, the composite restores and then forgets the same ring bundle. □

Proposition 10.10.17 (Exact comparison with the pinned additive sheaf). Lean: `conditional-interface:prop:weil-a2-underlying-additive-comparison`

Uses: `def:weil-a2-underlying-additive-comparison`

The maps c_X, d_X exhibit the underlying additive sheaf of Λ_X as $\mathcal{Z}_{\ell,X}$.

Proof. Uses: `lemma:weil-a2-underlying-comparison-naturality,lemma:weil-a2-underlying-comparison-left-inverse,lemma:weil-a2-underlying-comparison-right-inverse`

The first lemma makes both families sheaf morphisms. The last two are their inverse equations. □

10.11 Module sheaves and the ambient derived category

Definition 10.11.1 (Integral module-sheaf category). Lean: `conditional-interface:def:weil-a2-module-category`

Uses: `def:weil-a2-integral-ring-sheaf`

Define

$$\mathsf{Mod}(X,\Lambda) := \texttt{SheafOfModules}(\Lambda_X).$$

An object is a sheaf with a Λ_X-action and a morphism is a restriction-compatible linear map.

Theorem 10.11.2 (The module-sheaf category is abelian). Lean: `conditional-interface:thm:weil-a2-module-category-abelian`

Uses: `def:weil-a2-module-category`

The category $\mathsf{Mod}(X,\Lambda)$ is abelian.

Proof. Uses: `prop:weil-a2-continuous-ring-source-is-sheaf`

Instantiate the exact mathlib instance `SheafOfModules.instAbelian` from `Mathlib.Algebra.Category.ModuleCat.Sheaf.Abelian`. Its hypotheses are `HasSheafify` and `WEqualsLocallyBijective` for the small pro-etale site. The generic small-site instances supply both hypotheses. The ring object is Λ_X, so the instance conclusion is exactly `Abelian (SheafOfModules` Λ_X`)`. □

10.12 Atomic integral scalar action and its cohomology action

Definition 10.12.1 (Scalar action on the pinned integral coefficient sheaf). Lean: `conditional-interface:def:weil-integral-proetale-section-scalar-action`

Uses: `def:weil-a2-integral-ring-sheaf,def:weil-a2-underlying-additive-comparison`

For $U \in X_{\mathrm{proet}}$, $\lambda \in \mathbf{Z}_\ell$, and $s \in \mathcal{Z}_{\ell,X}(U)$, let $\underline{\lambda}_U$ be the constant section of $\Lambda_X(U)$ and define

$$\lambda \bullet_U s := c_{X,U}\big(\underline{\lambda}_U\, d_{X,U}(s)\big).$$

Here c_X and d_X are the two maps in the pinned underlying-additive comparison.

Lemma 10.12.2 (The pointwise scalar multiple is continuous). Lean: `conditional-interface:lemma:weil-efi-coeff-section-scalar-continuous`

Uses: `def:weil-integral-proetale-section-scalar-action`

The section $\lambda \bullet_U s$ is represented by the continuous map $x \mapsto \lambda s(x)$.

Proof. Uses: `lemma:weil-a2-underlying-comparison-components`

The maps $c_{X,U}$ and $d_{X,U}$ only forget and restore the ring bundle, so the displayed definition is pointwise multiplication by the constant map λ. Multiplication in the topological ring $\mathbf{Z}_\ell$ is continuous, hence its composite with the continuous map s is continuous. This asserts closure of continuous sections; it does not install an unmentioned topology on the section group. □

Lemma 10.12.3 (Zero scalar equation on sections). Lean: `conditional-interface:lemma:weil-efi-coeff-section-scalar-zero`

Uses: `def:weil-integral-proetale-section-scalar-action`

For every section s, $0 \bullet_U s = 0$.

Proof. Uses: `lemma:weil-a2-underlying-comparison-components`

At every $x \in |U|$ the two sides have values $0\,s(x) = 0$. □

Lemma 10.12.4 (Unit scalar equation on sections). Lean: `conditional-interface:lemma:weil-efi-coeff-section-scalar-one`

Uses: `def:weil-integral-proetale-section-scalar-action`

For every section s, $1 \bullet_U s = s$.

Proof. Uses: `lemma:weil-a2-underlying-comparison-components`

Evaluate at $x \in |U|$ and use $1s(x) = s(x)$ in $\mathbf{Z}_\ell$. □

Lemma 10.12.5 (Scalar-addition equation on sections). Lean: `conditional-interface:lemma:weil-efi-coeff-section-scalar-add`

Uses: `def:weil-integral-proetale-section-scalar-action`

For $\lambda, \mu \in \mathbf{Z}_\ell$ and a section s,

$$(\lambda + \mu) \bullet_U s = \lambda \bullet_U s + \mu \bullet_U s.$$

Proof. Uses: `lemma:weil-a2-underlying-comparison-components`

At $x \in |U|$ this is the distributive equation $(\lambda + \mu)s(x) = \lambda s(x) + \mu s(x)$. □

Lemma 10.12.6 (Scalar-multiplication equation on sections). Lean: `conditional-interface:lemma:weil-efi-coeff-section-scalar-mul`

Uses: `def:weil-integral-proetale-section-scalar-action`

For $\lambda, \mu \in \mathbf{Z}_\ell$ and a section s,

$$(\lambda\mu) \bullet_U s = \lambda \bullet_U (\mu \bullet_U s).$$

Proof. Uses: `lemma:weil-a2-underlying-comparison-components`

Evaluation at x reduces the claim to $(\lambda\mu)s(x) = \lambda(\mu s(x))$. □

Lemma 10.12.7 (Section-addition distributivity). Lean: `conditional-interface:lemma:weil-efi-coeff-section-scalar-distrib`

Uses: `def:weil-integral-proetale-section-scalar-action`

For $\lambda \in \mathbf{Z}_\ell$ and sections s, t,

$$\lambda \bullet_U (s + t) = \lambda \bullet_U s + \lambda \bullet_U t.$$

Proof. Uses: `lemma:weil-a2-underlying-comparison-components`

At $x \in |U|$ the equality is $\lambda(s(x) + t(x)) = \lambda s(x) + \lambda t(x)$. □

Definition 10.12.8 (Sectionwise module-law predicate). Lean: `conditional-interface:def:weil-efi-coeff-section-module-certificate`

Uses: `def:weil-integral-proetale-section-scalar-action`

Let $\mathsf{SectionModuleLaw}(X, \ell)$ mean that the displayed zero, unit, scalar-addition, scalar-multiplication, and section-addition equations hold for every pro-etale object and every well-typed input.

Proposition 10.12.9 (Sectionwise module-law certificate). Lean: `conditional-interface:prop:weil-efi-coeff-section-module-certificate`

Uses: `def:weil-efi-coeff-section-module-certificate`

The predicate $\mathsf{SectionModuleLaw}(X, \ell)$ holds.

Proof. Uses: `lemma:weil-efi-coeff-section-scalar-zero`, `lemma:weil-efi-coeff-section-scalar-one`, `lemma:weil-efi-coeff-section-scalar-add`, `lemma:weil-efi-coeff-section-scalar-mul`, `lemma:weil-efi-coeff-section-scalar-distrib`

The five cited equations fill the five fields of the predicate in their displayed order. □

Lemma 10.12.10 (Restriction is linear for the section action). Lean: `conditional-interface:lemma:weil-efi-coeff-section-restriction-linear`

Uses: `def:weil-integral-proetale-section-scalar-action`

For $v : V \to U$ and $s \in \mathcal{Z}_{\ell,X}(U)$,

$$v^*(\lambda \bullet_U s) = \lambda \bullet_V v^* s.$$

Proof. Uses: `lemma:weil-a2-underlying-comparison-naturality`, `lemma:weil-a2-continuous-ring-restriction-mul`

Naturality moves c_X and d_X through restriction. Restriction sends $\underline{\lambda}_U$ to $\underline{\lambda}_V$ and preserves the product, so both sides are represented by $x \mapsto \lambda s(|v|(x))$. □

Definition 10.12.11 (Integral module-sheaf candidate on the pinned additive sheaf)**.** Lean: `conditional-interface:def:weil-integral-proetale-module-sheaf-candidate`

Uses: `def:weil-a2-module-category`, `def:weil-a2-underlying-additive-comparison`, `def:weil-integral-proetale-section-scalar-action`

Constructed by: `prop:weil-efi-coeff-section-module-certificate`, `lemma:weil-efi-coeff-section-restriction-linear`

Let $\mathcal{M}_{\ell,X}$ be the regular left Λ_X-module sheaf, equipped with the additive comparison

$$c_X : \mathrm{Forget}(\mathcal{M}_{\ell,X}) \xrightarrow{\sim} \mathcal{Z}_{\ell,X}.$$

Transporting constant scalars through c_X gives the action $\bullet_U$ above.

Lemma 10.12.12 (Underlying additive sheaf of the module candidate)**.** Lean: *conditional-interface:lemma:weil-efi-coeff-module-sheaf-underlying*

Uses: *def:weil-integral-proetale-module-sheaf-candidate*

The displayed map c_X is an isomorphism from the underlying additive sheaf of $\mathcal{M}_{\ell,X}$ to the pinned sheaf $\mathcal{Z}_{\ell,X}$.

Proof. Uses: `prop:weil-a2-underlying-additive-comparison`

This is exactly the previously proved underlying-additive comparison, now retained as the comparison field of the regular module sheaf. □

Proposition 10.12.13 (The candidate packages the pinned sheaf as a module sheaf)**.** Lean: *conditional-interface:prop:weil-efi-coeff-module-sheaf-package*

Uses: *def:weil-integral-proetale-module-sheaf-candidate*

The object $\mathcal{M}_{\ell,X}$ is a sheaf of Λ_X-modules whose underlying additive sheaf is identified with $\mathcal{Z}_{\ell,X}$.

Proof. Uses: `prop:weil-efi-coeff-section-module-certificate`, `lemma:weil-efi-coeff-section-restriction-linear`, `lemma:weil-efi-coeff-module-sheaf-underlying`

The first result supplies the objectwise module equations, the second makes the restriction maps linear, and the last supplies the specified underlying additive comparison. □

Definition 10.12.14 (Scalar sheaf endomorphism)**.** Lean: `conditional-interface:def:weil-efi-coeff-scalar-sheaf-endomorphism`

Uses: `def:weil-integral-proetale-section-scalar-action`

Constructed by: `lemma:weil-efi-coeff-section-restriction-linear`

For $\lambda \in \mathbf{Z}_\ell$, define

$$m_\lambda : \mathcal{Z}_{\ell,X} \longrightarrow \mathcal{Z}_{\ell,X}, \qquad (m_\lambda)_U(s) = \lambda \bullet_U s.$$

Lemma 10.12.15 (Zero scalar endomorphism)**.** Lean: *conditional-interface:lemma:weil-efi-coeff-scalar-endomorphism-zero*

Uses: *def:weil-efi-coeff-scalar-sheaf-endomorphism*

One has $m_0 = 0$.

Proof. Uses: `lemma:weil-efi-coeff-section-scalar-zero`

The equality holds on every section by the cited zero-scalar equation. □

Lemma 10.12.16 (Unit scalar endomorphism)**.** Lean: *conditional-interface:lemma:weil-efi-coeff-scalar-endomorphism-one*

Uses: *def:weil-efi-coeff-scalar-sheaf-endomorphism*

One has $m_1 = \mathrm{id}$.

Proof. Uses: `lemma:weil-efi-coeff-section-scalar-one`

The equality holds on every section by the cited unit-scalar equation. □

Lemma 10.12.17 (Addition of scalar endomorphisms)**.** Lean: *conditional-interface:lemma:weil-efi-coeff-scalar-endomorphism-add*

Uses: *def:weil-efi-coeff-scalar-sheaf-endomorphism*

One has $m_{\lambda+\mu} = m_\lambda + m_\mu$.

Proof. Uses: `lemma:weil-efi-coeff-section-scalar-add`

Evaluate the two sheaf morphisms on each section and apply the cited equation. □

Lemma 10.12.18 (Composition of scalar endomorphisms). *Lean:* `conditional-interface:lemma:weil-efi-coeff-scalar-endomorphism-mul`

Uses: `def:weil-efi-coeff-scalar-sheaf-endomorphism`
One has $m_{\lambda\mu} = m_\lambda m_\mu$.

Proof. *Uses:* `lemma:weil-efi-coeff-section-scalar-mul`
Evaluate on a section and use the cited scalar-multiplication equation. □

Definition 10.12.19 (Scalar action on integral pro-etale cohomology). *Lean:* `conditional-interface:def:weil-efi-coeff-integral-cohomology-scalar-action`

Uses: `def:weil-integral-proetale-cohomology`, `def:weil-efi-coeff-scalar-sheaf-endomorphism`
For $\alpha \in H^n_{\mathrm{proet}}(X, \mathbf{Z}_\ell)$ define

$$\lambda \star \alpha := H^n(m_\lambda)(\alpha),$$

using functoriality of the pinned generic sheaf-cohomology construction.

Lemma 10.12.20 (Zero scalar equation on cohomology). *Lean:* `conditional-interface:lemma:weil-efi-coeff-cohomology-scalar-zero`

Uses: `def:weil-efi-coeff-integral-cohomology-scalar-action`
For every α, $0 \star \alpha = 0$.

Proof. *Uses:* `lemma:weil-efi-coeff-scalar-endomorphism-zero`
The generic cohomology functor is additive and therefore sends the zero sheaf morphism to the zero homomorphism. □

Lemma 10.12.21 (Unit scalar equation on cohomology). *Lean:* `conditional-interface:lemma:weil-efi-coeff-cohomology-scalar-one`

Uses: `def:weil-efi-coeff-integral-cohomology-scalar-action`
For every α, $1 \star \alpha = \alpha$.

Proof. *Uses:* `lemma:weil-efi-coeff-scalar-endomorphism-one`
Functoriality sends the identity sheaf morphism to the identity on cohomology. □

Lemma 10.12.22 (Scalar-addition equation on cohomology). *Lean:* `conditional-interface:lemma:weil-efi-coeff-cohomology-scalar-add`

Uses: `def:weil-efi-coeff-integral-cohomology-scalar-action`
For $\lambda, \mu \in \mathbf{Z}_\ell$,

$$(\lambda + \mu) \star \alpha = \lambda \star \alpha + \mu \star \alpha.$$

Proof. *Uses:* `lemma:weil-efi-coeff-scalar-endomorphism-add`
Additivity gives $H^n(m_\lambda + m_\mu) = H^n(m_\lambda) + H^n(m_\mu)$; evaluate at α. □

Lemma 10.12.23 (Scalar-multiplication equation on cohomology). *Lean:* `conditional-interface:lemma:weil-efi-coeff-cohomology-scalar-mul`

Uses: `def:weil-efi-coeff-integral-cohomology-scalar-action`
For $\lambda, \mu \in \mathbf{Z}_\ell$,

$$(\lambda\mu) \star \alpha = \lambda \star (\mu \star \alpha).$$

Proof. *Uses:* `lemma:weil-efi-coeff-scalar-endomorphism-mul`
Functoriality identifies $H^n(m_\lambda m_\mu)$ with $H^n(m_\lambda)H^n(m_\mu)$; evaluate at α. □

Lemma 10.12.24 (Cohomology-addition distributivity). *Lean:* `conditional-interface:lemma:weil-efi-coeff-cohomology-scalar-distrib`

Uses: `def:weil-efi-coeff-integral-cohomology-scalar-action`
For cohomology classes α, β,

$$\lambda \star (\alpha + \beta) = \lambda \star \alpha + \lambda \star \beta.$$

Proof. *Uses:* `def:weil-efi-coeff-scalar-sheaf-endomorphism`
The map $H^n(m_\lambda)$ is a homomorphism of additive groups. □

Definition 10.12.25 (Cohomology module-law predicate)**.** Lean: `conditional-interface:def:weil-efi-coeff-cohomology-module-certificate`

Uses: `def:weil-efi-coeff-integral-cohomology-scalar-action`

Let $\mathsf{CohomologyModuleLaw}(X, \ell, n)$ mean that the displayed zero, unit, scalar-addition, scalar-multiplication, and cohomology-addition equations hold.

Proposition 10.12.26 (Cohomology module-law certificate)**.** *Lean:* *`conditional-interface:prop:weil-efi-coeff-cohomology-module-certificate`*

Uses: *`def:weil-efi-coeff-cohomology-module-certificate`*

The predicate $\mathsf{CohomologyModuleLaw}(X, \ell, n)$ *holds.*

Proof. Uses: `lemma:weil-efi-coeff-cohomology-scalar-zero,lemma:weil-efi-coeff-cohomology-scalar-one,lemma:weil-efi-coeff-cohomology-scalar-add,lemma:weil-efi-coeff-cohomology-scalar-mul,lemma:weil-efi-coeff-cohomology-scalar-distrib`

The five cited equations fill the five fields of the predicate. □

Definition 10.12.27 (Constructed integral cohomology module)**.** Lean: `conditional-interface:def:weil-efi-coeff-integral-cohomology-module`

Uses: `def:weil-integral-cohomology-module-candidate,def:weil-efi-coeff-integral-cohomology-scalar-action`

Constructed by: `prop:weil-efi-coeff-cohomology-module-certificate`

Equip the existing additive group $H^n_{\mathrm{proet}}(X, \mathbf{Z}_\ell)$ with the action $\star$. This is an inhabitant of the integral cohomology module-candidate type, rather than an assumed inhabitant of that type.

Lemma 10.12.28 (The constructed module retains the existing additive group)**.** *Lean:* *`conditional-interface:lemma:weil-efi-coeff-integral-cohomology-underlying-additive`*

Uses: *`def:weil-efi-coeff-integral-cohomology-module`*

Forgetting the $\mathbf{Z}_\ell$*-action gives the original additive group* $H^n_{\mathrm{proet}}(X, \mathbf{Z}_\ell)$.

Proof. Uses: `def:weil-efi-coeff-integral-cohomology-module`

The construction changes only the scalar-action field; its carrier, zero, addition, and negation are the fields of the existing cohomology group. □

Definition 10.12.29 (Scalar-extension comparison-square predicate)**.** Lean: `conditional-interface:def:weil-efi-coeff-scalar-extension-square-predicate`

Uses: `def:weil-rational-cohomology-comparison-predicate,def:weil-scalar-extension-endomorphism`

For a proposed comparison $c : E \otimes_\Lambda M \to V$, an integral endomorphism $u : M \to M$, and a proposed rational endomorphism $w : V \to V$, let $\mathsf{ScalarSquare}(c, u, w)$ denote the single equation

$$c\, u_E = w\, c.$$

This definition supplies only the type of the compatibility equation.

Definition 10.12.30 (Complex category)**.** Lean: `conditional-interface:def:weil-a2-complex-category`

Uses: `def:weil-a2-module-category`

Let $\mathsf{C}(X, \Lambda)$ be the category of unbounded cochain complexes in $\mathsf{Mod}(X, \Lambda)$.

Definition 10.12.31 (Quasi-isomorphisms)**.** Lean: `conditional-interface:def:weil-a2-quasi-isomorphism`

Uses: `def:weil-a2-complex-category`

A cochain map $q : C \to D$ is a quasi-isomorphism when $H^i(q)$ is an isomorphism for every $i \in \mathbf{Z}$.

Definition 10.12.32 (Ambient derived category)**.** Lean: `conditional-interface:def:weil-a2-derived-category`

Uses: `def:weil-a2-complex-category,def:weil-a2-quasi-isomorphism`

Define

$$\mathcal{D}(X, \Lambda) := \mathsf{C}(X, \Lambda)[\mathrm{qis}^{-1}]$$

using mathlib's derived-category localization, and write $Q_X : \mathsf{C}(X, \Lambda) \to \mathcal{D}(X, \Lambda)$ for the localization.

Proposition 10.12.33 (Triangulated structure). Lean: `conditional-interface:prop:weil-a2-derived-category-triangulated`

Uses: `def:weil-a2-derived-category`

The category $\mathcal{D}(X,\Lambda)$ has the standard shift and distinguished mapping-cone triangles.

Proof. Uses: `thm:weil-a2-module-category-abelian`

Apply the generic mathlib derived-category theorem to the abelian category $\mathsf{Mod}(X,\Lambda)$. Its shift and distinguished triangles are defined by localization of cochain shifts and mapping-cone triangles, which are exactly those in the statement. □

Definition 10.12.34 (Sheaf tensor product). Lean: `conditional-interface:def:weil-a2-sheaf-tensor`

Uses: `def:weil-a2-module-category`

For $M, N \in \mathsf{Mod}(X,\Lambda)$, define $M \otimes_{\Lambda_X} N$ by the tensor product in $\mathtt{SheafOfModules}(\Lambda_X)$.

Proposition 10.12.35 (Tensor universal property). Lean: `conditional-interface:prop:weil-a2-sheaf-tensor-universal`

Uses: `def:weil-a2-sheaf-tensor`

Maps $M \otimes_{\Lambda_X} N \to P$ correspond naturally to Λ_X-balanced bilinear sheaf maps $M \times N \to P$.

Proof. Uses: `def:weil-a2-sheaf-tensor`

Instantiate the generic tensor-product universal property in `SheafOfModules`. Its source ring object and all three module arguments are respectively Λ_X, M, N, P; this gives the displayed equivalence without a new sheafification argument. □

10.13 K-flat models and finite reduction

Definition 10.13.1 (Total tensor complex). Lean: `conditional-interface:def:weil-a2-total-tensor-complex`

Uses: `def:weil-a2-complex-category,def:weil-a2-sheaf-tensor`

For complexes C, D, define

$$(C \otimes D)^n := \bigoplus_{p+q=n} C^p \otimes_{\Lambda_X} D^q, \qquad d(c \otimes d) := d_C c \otimes d + (-1)^p c \otimes d_D d$$

for $c \in C^p$.

Lemma 10.13.2 (The total differential squares to zero). Lean: `conditional-interface:lemma:weil-a2-total-tensor-square-zero`

Uses: `def:weil-a2-total-tensor-complex`

The total differential satisfies $d^2 = 0$.

Proof. Uses: `def:weil-a2-total-tensor-complex`

On a pure tensor, the two pure terms vanish by $d_C^2 = d_D^2 = 0$. The two mixed terms have coefficients $(-1)^{p+1}$ and $(-1)^p$, so they cancel. □

Definition 10.13.3 (K-flat complex). Lean: `conditional-interface:def:weil-a2-kflat-complex`

Uses: `def:weil-a2-total-tensor-complex`

A complex P is K-flat if $P \otimes A$ is acyclic for every acyclic complex A.

Definition 10.13.4 (Functorial K-flat package). Lean: `conditional-interface:def:weil-a2-kflat-package`

Uses: `def:weil-a2-complex-category,def:weil-a2-derived-category,def:weil-a2-kflat-complex`

A functorial K-flat package consists of

$$\mathfrak{P} : \mathcal{D}(X,\Lambda) \longrightarrow \mathsf{C}(X,\Lambda), \qquad \varepsilon : Q_X \mathfrak{P} \Longrightarrow \mathrm{id},$$

such that every $\mathfrak{P}(K)$ is K-flat and every ε_K is an isomorphism. Retain the identity and composition equations for $\mathfrak{P}$.

Proposition 10.13.5 (Functorial K-flat package existence). *Lean:* `conditional-interface:prop:weil-a2-kflat-package-existence-open`

Uses: `def:weil-a2-kflat-package`

There exists a functorial K-flat package for $\mathsf{Mod}(X, \Lambda)$.

Proof. Uses: `thm:weil-a2-module-category-abelian,prop:weil-a2-sheaf-tensor-universal`

OPEN obligation A2-KFLAT-PACKAGE. The exact target is a functorial unbounded K-flat replacement for complexes of Λ_X-module sheaves. The proof must construct bounded-above flat replacements, the transfinite Spaltenstein tower, its successor maps and limit, the quasi-isomorphism to the input, K-flatness of the limit, and identity/composition homotopies. This proposition is not promoted to the foundation contract. □

Definition 10.13.6 (Fixed K-flat package parameter). Lean: `conditional-interface:def:weil-a2-fixed-kflat-package`

Uses: `def:weil-a2-kflat-package`

Fix an explicit input κ_X of type *functorial K-flat package*. Every construction below retains this parameter until package-independence is proved.

Definition 10.13.7 (Two-term coefficient resolution). Lean: `conditional-interface:def:weil-a2-finite-coefficient-complex`

Uses: `def:weil-a2-complex-category,def:weil-a2-integral-ring-sheaf`

For $n \geq 1$, define L_n by

$$L_n^{-1} = \Lambda_X, \qquad L_n^0 = \Lambda_X, \qquad d^{-1} = \ell^n,$$

and $L_n^i = 0$ otherwise.

Lemma 10.13.8 (Cohomology of the coefficient resolution). *Lean:* `conditional-interface:lemma:weil-a2-finite-coefficient-cohomology`

Uses: `def:weil-a2-finite-coefficient-complex`

One has

$$H^{-1}(L_n) = 0, \qquad H^0(L_n) = \Lambda_X/\ell^n, \qquad H^i(L_n) = 0 \quad (i \neq -1, 0).$$

Proof. Uses: `def:weil-a2-finite-coefficient-complex,prop:weil-a2-underlying-additive-comparison`

Multiplication by ℓ^n is injective on continuous $\mathbf{Z}_\ell$-valued functions because it is injective pointwise. Its cokernel is Λ_X/ℓ^n. The remaining terms vanish. □

Definition 10.13.9 (Finite torsion coefficient ring sheaf). Lean: `conditional-interface:def:weil-a2-torsion-ring-sheaf`

Uses: `def:weil-a2-integral-ring-sheaf`

For $n \geq 1$, define

$$\Lambda_{n,X} := \operatorname{coker}\bigl(\Lambda_X \xrightarrow{\ell^n} \Lambda_X\bigr)$$

in sheaves of commutative rings; equivalently it is the sheaf of continuous $\mathbf{Z}/\ell^n$-valued functions.

Proposition 10.13.10 (Finite torsion coefficient identification). *Lean:* `conditional-interface:prop:weil-a2-torsion-ring-identification`

Uses: `def:weil-a2-torsion-ring-sheaf`

The quotient map induces an isomorphism

$$\Lambda_X/\ell^n \xrightarrow{\sim} \Lambda_{n,X}.$$

Proof. Uses: `prop:weil-a2-underlying-additive-comparison`

Objectwise reduction sends a continuous $\mathbf{Z}_\ell$-valued function to a continuous $\mathbf{Z}/\ell^n$-valued function. Such a function is locally constant; on each clopen fiber choose its least lift in $\{0, \dots, \ell^n - 1\}$, obtaining a continuous lift. Thus the map is locally surjective, and its kernel is pointwise $\ell^n\Lambda$. The sheaf quotient therefore gives the asserted isomorphism. □

Definition 10.13.11 (Finite torsion derived category). Lean: `conditional-interface:def:weil-a2-torsion-derived-category`

Uses: `def:weil-a2-torsion-ring-sheaf`

Define

$$\mathcal{D}(X, \Lambda_n) := D\bigl(\texttt{SheafOfModules}(\Lambda_{n,X})\bigr).$$

Proposition 10.13.12 (The finite torsion module category is abelian). *Lean:* `conditional-interface:prop:weil-a2-torsion-module-abelian`

Uses: `def:weil-a2-torsion-ring-sheaf`
The category `SheafOfModules`$(\Lambda_{n,X})$ *is abelian.*

Proof. Uses: `thm:weil-a2-module-category-abelian`
Apply the same exact mathlib instance `SheafOfModules.instAbelian`. Its site hypotheses are unchanged; only the ring-sheaf argument is replaced by $\Lambda_{n,X}$. □

Lemma 10.13.13 (The coefficient resolution is K-flat). *Lean:* `conditional-interface:lemma:weil-a2-finite-coefficient-kflat`

Uses: `def:weil-a2-finite-coefficient-complex`, `def:weil-a2-kflat-complex`
For every $n \geq 1$, L_n is K-flat.

Proof. Uses: `def:weil-a2-finite-coefficient-complex`,`def:weil-a2-kflat-complex`
Its two nonzero terms are free rank-one Λ_X-modules. Filter $L_n \otimes A$ by its two columns. The associated graded complexes are shifts of A. If A is acyclic, the long exact sequence for the two-step filtration makes the total complex acyclic. □

Definition 10.13.14 (Finite reduction). Lean: `conditional-interface:def:weil-a2-finite-reduction`

Uses: `def:weil-a2-fixed-kflat-package`,`def:weil-a2-finite-coefficient-complex`,`def:weil-a2-total-tensor-complex`,`def:weil-a2-torsion-derived-category`
For $K \in \mathcal{D}(X, \Lambda)$, define

$$\mathrm{Red}_n^{\kappa_X}(K) := Q_X\big(\mathfrak{P}_{\kappa_X}(K) \otimes_{\Lambda_X} L_n\big).$$

The action factors through $\Lambda_{n,X}$, so the output lies in $\mathcal{D}(X, \Lambda_n)$.

Definition 10.13.15 (Tensor transport). Lean: `conditional-interface:def:weil-a2-finite-reduction-transport`

Uses: `def:weil-a2-total-tensor-complex`,`def:weil-a2-finite-coefficient-complex`
For $q : P \to P'$, define

$$q \otimes \mathrm{id}_{L_n} : P \otimes L_n \to P' \otimes L_n, \qquad p \otimes x \longmapsto q(p) \otimes x.$$

Lemma 10.13.16 (Tensor transport is a cochain map). *Lean:* `conditional-interface:lemma:weil-a2-finite-reduction-transport-chain-map`

Uses: `def:weil-a2-finite-reduction-transport`
The displayed map commutes with total differentials.

Proof. Uses: `def:weil-a2-finite-reduction-transport`,`def:weil-a2-total-tensor-complex`
On a pure tensor, the terms involving the first differential agree by the cochain-map equation for q, and the terms involving the second differential are identical. □

Lemma 10.13.17 (Tensor with L_n preserves quasi-isomorphisms). *Lean:* `conditional-interface:lemma:weil-a2-finite-reduction-transport-qis`

Uses: `def:weil-a2-finite-reduction-transport`, `def:weil-a2-quasi-isomorphism`
If q is a quasi-isomorphism, then $q \otimes \mathrm{id}_{L_n}$ is a quasi-isomorphism.

Proof. Uses: `lemma:weil-a2-finite-coefficient-kflat`
The cone of q is acyclic. Tensoring it with the K-flat complex L_n remains acyclic. Direct expansion identifies the result with the cone of $q \otimes \mathrm{id}_{L_n}$. Apply the cone criterion. □

Definition 10.13.18 (Comparison roof between two K-flat packages). Lean: `conditional-interface:def:weil-a2-kflat-package-comparison-roof`

Uses: `def:weil-a2-kflat-package`,`def:weil-a2-derived-category`
For packages κ, κ' and K, a comparison roof is a complex R_K with quasi-isomorphisms

$$R_K \longrightarrow \mathfrak{P}_\kappa(K), \qquad R_K \longrightarrow \mathfrak{P}_{\kappa'}(K)$$

whose localized composites to K agree.

Lemma 10.13.19 (Comparison roofs exist). Lean: `conditional-interface:lemma:weil-a2-kflat-package-comparison-roof-exists`

Uses: `def:weil-a2-kflat-package-comparison-roof`

For each K, a comparison roof exists.

Proof. Uses: `def:weil-a2-derived-category,def:weil-a2-quasi-isomorphism`

The two localized complexes are isomorphic to K. Represent the resulting derived isomorphism by a finite zigzag. Replace adjacent backward arrows by mapping-cylinder roofs. Their new projections are quasi-isomorphisms by the cone criterion. Induction on the zigzag length gives R_K. □

Lemma 10.13.20 (Finite reduction is package-independent). Lean: `conditional-interface:lemma:weil-a2-finite-reduction-package-independent`

Uses: `def:weil-a2-finite-reduction,def:weil-a2-kflat-package-comparison-roof`

Every comparison roof induces an isomorphism

$$\mathrm{Red}_n^{\kappa}(K) \simeq \mathrm{Red}_n^{\kappa'}(K),$$

independent of the roof.

Proof. Uses: `lemma:weil-a2-kflat-package-comparison-roof-exists,lemma:weil-a2-finite-reduction-transport-qis`

Tensor both roof arrows with L_n and localize. They become isomorphisms. Two roofs admit a common refinement by the first cited lemma. The two comparison maps agree after precomposition by the same isomorphism, so cancellation gives equality. □

Proposition 10.13.21 (Finite reduction functor). Lean: `conditional-interface:prop:weil-a2-finite-reduction-functor`

Uses: `def:weil-a2-finite-reduction`

For each $n \geq 1$, finite reduction is a functor

$$\mathrm{Red}_n^{\kappa_X} : \mathcal{D}(X, \Lambda) \longrightarrow \mathcal{D}(X, \Lambda/\ell^n),$$

and the package-independence comparisons are natural.

Proof. Uses: `lemma:weil-a2-finite-reduction-transport-chain-map,lemma:weil-a2-finite-reduction-package-independent`

Apply $\mathfrak{P}_{\kappa_X}$ to a derived morphism, tensor the resulting cochain map with L_n, and localize. Tensoring preserves identities and composition on pure tensors. For package naturality, place a morphism and the two package roofs over one common refinement; both paths then localize the same cochain map. □

Definition 10.13.22 (Reduction transition on coefficient complexes). Lean: `conditional-interface:def:weil-a2-reduction-transition-complex`

Uses: `def:weil-a2-finite-coefficient-complex`

Define $\rho_{n+1,n} : L_{n+1} \to L_n$ as multiplication by ℓ in degree -1 and the identity in degree 0.

Lemma 10.13.23 (The reduction transition is a cochain map). Lean: `conditional-interface:lemma:weil-a2-reduction-transition-chain-map`

Uses: `def:weil-a2-reduction-transition-complex`

The map $\rho_{n+1,n}$ commutes with differentials.

Proof. Uses: `def:weil-a2-reduction-transition-complex`

The unique square is $\ell^n \circ \ell = \mathrm{id} \circ \ell^{n+1}$; both sides are multiplication by ℓ^{n+1}. □

Definition 10.13.24 (Reduction transition). Lean: `conditional-interface:def:weil-a2-reduction-transition`

Uses: `def:weil-a2-finite-reduction,def:weil-a2-reduction-transition-complex`

Define

$$r_{n+1,n}(K) : \mathrm{Red}_{n+1}(K) \longrightarrow \mathrm{Red}_n(K)$$

by localizing $\mathrm{id}_{\mathfrak{P}(K)} \otimes \rho_{n+1,n}$.

Lemma 10.13.25 (Reduction transition naturality). Lean: `conditional-interface:lemma:weil-a2-reduction-transition-natural`

Uses: `def:weil-a2-reduction-transition`

For $f : K \to K'$, the square formed by $r_{n+1,n}$ and $\mathrm{Red}(f)$ commutes.

Proof. Uses: `def:weil-a2-reduction-transition`

Before localization both paths are the same cochain map $\mathfrak{P}(f) \otimes \rho_{n+1,n}$. □

Definition 10.13.26 (Total Hom complex for all-integer complexes). Lean: `conditional-interface:def:weil-unbounded-ki-hom-complex`

Uses: `def:weil-a2-complex-category`

For complexes $A^\bullet, I^\bullet$ in the fixed abelian category, define

$$\underline{\mathrm{Hom}}^r(A, I) := \prod_{n \in \mathbf{Z}} \mathrm{Hom}_{\mathcal{A}}(A^n, I^{n+r}), \qquad (dh)^n = d_I^{n+r} h^n - (-1)^r h^{n+1} d_A^n$$

for a homogeneous family h of degree r.

Lemma 10.13.27 (The total Hom differential squares to zero). Lean: `conditional-interface:lemma:weil-unbounded-ki-hom-square`

Uses: `def:weil-unbounded-ki-hom-complex`

The differential in Definition 10.13.26 satisfies $d^2 = 0$.

Proof. Uses: `def:weil-unbounded-ki-hom-complex`

For a homogeneous h of degree r, the degree-n component of $d^2 h$ is

$$d_I^{n+r+1} d_I^{n+r} h^n - (-1)^r d_I^{n+r+1} h^{n+1} d_A^n - (-1)^{r+1} d_I^{n+r+1} h^{n+1} d_A^n + (-1)^{2r+1} h^{n+2} d_A^{n+1} d_A^n.$$

The first and last terms vanish, and the two middle terms cancel. □

10.14 General unbounded derived functors

Definition 10.14.1 (Unbounded derived localization). Lean: `conditional-interface:def:weil-general-unbounded-derived-localization-v1`

Uses: `def:weil-a2-complex-category,def:weil-a2-quasi-isomorphism`

For an abelian category $\mathcal{A}$, write

$$\mathrm{Ch}(\mathcal{A}) = \mathrm{CochainComplex}(\mathcal{A}, \mathbf{Z}), \qquad W_{\mathcal{A}} = \{\text{quasi-isomorphisms}\},$$

and

$$K(\mathcal{A}) := \mathrm{Hot}(\mathrm{Ch}(\mathcal{A})), \qquad \overline{W}_{\mathcal{A}} := \{[u]_K : u \in W_{\mathcal{A}}\}, \qquad D(\mathcal{A}) := K(\mathcal{A})[\overline{W}_{\mathcal{A}}^{-1}].$$

Let $Q_{\mathcal{A}}$ denote the composite of the projection to $K(\mathcal{A})$ with the localization at $\overline{W}_{\mathcal{A}}$. All complexes and all products in this section are indexed by $\mathbf{Z}$.

Definition 10.14.2 (K-injective, K-projective, and K-flat). Lean: `conditional-interface:def:weil-general-unbounded-k-models-v1`

Uses: `def:weil-general-unbounded-derived-localization-v1,def:weil-unbounded-ki-hom-complex,def:weil-a2-total-tensor-complex`

For $I, P \in \mathrm{Ch}(\mathcal{A})$, use the previously fixed total Hom complex (and its displayed differential) and define

$$I \text{ is K-injective} \iff \mathrm{Hom}^\bullet(A, I) \text{ is acyclic for every acyclic } A,$$
$$P \text{ is K-projective} \iff \mathrm{Hom}^\bullet(P, A) \text{ is acyclic for every acyclic } A.$$

If $\mathcal{A}$ is a chosen monoidal abelian category for which the displayed countable coproducts and tensor-totalization maps are supplied, let the total tensor be fixed by

$$(P \otimes A)^n = \bigoplus_{i+j=n} P^i \otimes A^j, \qquad d(p \otimes a) = d_P(p) \otimes a + (-1)^i p \otimes d_A(a) \quad (p \in P^i).$$

For the concrete A2 coefficient category this input is the previously defined totalization in Definition 10.13.1. Then P is K-flat when $P \otimes A$ is acyclic for every acyclic A.

Definition 10.14.3 (Functorial K-injective replacement certificate). Lean: `conditional-interface:def:weil-general-unbounded-kinjective-replacement-v1`

Uses: `def:weil-general-unbounded-derived-localization-v1,def:weil-general-unbounded-k-models-v1`

A functorial K-injective replacement certificate on $\mathcal{A}$ is an endofunctor $I_{\mathcal{A}} : \mathrm{Ch}(\mathcal{A}) \to \mathrm{Ch}(\mathcal{A})$ and a natural transformation $\iota : \mathrm{id} \Rightarrow I_{\mathcal{A}}$ such that every ι_C is a quasi-isomorphism and every $I_{\mathcal{A}}(C)$ is K-injective. The certificate includes the identity and composition equations for $I_{\mathcal{A}}$ and the naturality equations for ι.

Definition 10.14.4 (Functorial K-projective replacement certificate). Lean: `conditional-interface:def:weil-general-unbounded-kprojective-replacement-v1`

Uses: `def:weil-general-unbounded-derived-localization-v1,def:weil-general-unbounded-k-models-v1`

A functorial K-projective replacement certificate is an endofunctor $P_{\mathcal{A}} : \mathrm{Ch}(\mathcal{A}) \to \mathrm{Ch}(\mathcal{A})$ and a natural transformation $\epsilon : P_{\mathcal{A}} \Rightarrow \mathrm{id}$ such that every ϵ_C is a quasi-isomorphism and every $P_{\mathcal{A}}(C)$ is K-projective. The identity and composition equations for $P_{\mathcal{A}}$ and the naturality equations for ϵ are part of the certificate.

Definition 10.14.5 (Functorial K-flat replacement certificate). Lean: `conditional-interface:def:weil-general-unbounded-kflat-replacement-v1`

Uses: `def:weil-general-unbounded-derived-localization-v1,def:weil-general-unbounded-k-models-v1`

For a tensor abelian category $(\mathcal{A}, \otimes)$, a functorial K-flat replacement certificate is an endofunctor $P^{\mathrm{fl}}_{\mathcal{A}}$ and a natural quasi-isomorphism $\epsilon^{\mathrm{fl}} : P^{\mathrm{fl}}_{\mathcal{A}} \Rightarrow \mathrm{id}$ whose values are K-flat, together with the identity and composition equations. This certificate is used for derived tensor and scalar-extension functors; K-flatness alone is not a replacement theorem for an arbitrary additive functor.

Definition 10.14.6 (Objectwise replacement data). Lean: `conditional-interface:def:weil-general-unbounded-objectwise-replacement-v1`

Uses: `def:weil-general-unbounded-k-models-v1`

For one all-integer complex M, write

$$\mathrm{KIRep}(M) = (I_M,\ M \xrightarrow{\iota_M} I_M,\ \kappa_M),$$
$$\mathrm{KPRep}(M) = (P_M,\ P_M \xrightarrow{\pi_M} M,\ \kappa'_M),$$
$$\mathrm{KFRep}(M) = (P^{\flat}_M,\ P^{\flat}_M \xrightarrow{\pi^{\flat}_M} M,\ \kappa^{\flat}_M),$$

where the displayed arrows are quasi-isomorphisms and the final fields prove, respectively, K-injective, K-projective, and K-flat. These are independent objectwise data; no map between the choices for different M is included.

Proposition 10.14.7 (Objectwise data extracted from a functorial package). Lean: *conditional-interface:prop:weil-general-unbounded-objectwise-from-package-v1*

Uses: *def:weil-general-unbounded-objectwise-replacement-v1*

Each of the three functorial replacement certificates supplies the corresponding objectwise datum for every complex M.

Proof. Uses: `def:weil-general-unbounded-objectwise-replacement-v1,def:weil-general-unbounded-kinjective-replacement-v1,def:weil-general-unbounded-kprojective-replacement-v1,def:weil-general-unbounded-kflat-replacement-v1`

Evaluate the endofunctor and its natural quasi-isomorphism at M and use the model predicate field of the certificate. This gives the displayed triple for each M. The argument is objectwise and uses no boundedness. □

Proposition 10.14.8 (Objectwise replacement status). Lean: *conditional-interface:prop:weil-general-unbounded-objectwise-existence-open-v1*

Uses: *def:weil-general-unbounded-objectwise-replacement-v1*

The ringed-site specialization below constructs objectwise K-injective and termwise-flat K-flat data for every all-integer complex in the selected Weil coefficient categories. It does not construct K-projective replacements for module sheaves, so that unused generic branch remains conditional.

Open obligation: *OBL-GENERAL-UNBOUNDED-KPROJECTIVE-OBJECTWISE*

Proof. Uses: `def:weil-general-unbounded-objectwise-replacement-v1`

The selected K-injective and K-flat theorems occur after the generic definitions because their hypotheses use the concrete ringed sites. Their objectwise triples provide exactly the fields defined above. No theorem in that specialization supplies a projective generator or a dg-projective model for sheaf categories, so the displayed K-projective marker is retained. □

Lemma 10.14.9 (Replacement functors preserve quasi-isomorphisms)**.** *Lean:* `conditional-interface:lemma:weil-general-unbounded-replacement-preserves-qis-v1`

Uses: `def:weil-general-unbounded-kinjective-replacement-v1`, `def:weil-general-unbounded-kprojective-replacement-v1`, `def:weil-general-unbounded-kflat-replacement-v1`

If $u : C \to C'$ is a quasi-isomorphism, then $I_{\mathcal{A}}(u)$, $P_{\mathcal{A}}(u)$, and $P^{\mathrm{fl}}_{\mathcal{A}}(u)$ are quasi-isomorphisms whenever the corresponding replacement certificates are fixed.

Proof. Uses: `def:weil-general-unbounded-kinjective-replacement-v1`, `def:weil-general-unbounded-kprojective-replacement-v1`, `def:weil-general-unbounded-kflat-replacement-v1`

Naturality gives $I(u)\iota_C = \iota_{C'}u$ and $\epsilon_{C'}P(u) = u\epsilon_C$, together with $(\epsilon^{\mathrm{fl}}_{C'})P^{\mathrm{fl}}(u) = u\epsilon^{\mathrm{fl}}_C$. Two-out-of-three for quasi-isomorphisms proves all three assertions. □

Lemma 10.14.10 (Quasi-isomorphisms between K-models are homotopy equivalences)**.** *Lean:* `conditional-interface:lemma:weil-general-unbounded-kmodel-qis-homotopy-v1`

Uses: `def:weil-general-unbounded-k-models-v1`

If $v : I \to I'$ is a quasi-isomorphism between K-injective complexes, or a quasi-isomorphism between K-projective complexes, then v is a homotopy equivalence.

Proof. Uses: `def:weil-general-unbounded-k-models-v1`

Put $C = \mathrm{Cone}(v)$. The cone criterion gives that C is acyclic. In the K-injective case, apply the long exact Hom sequence of the cone triangle $I \xrightarrow{v} I' \to C \to I[1]$ to the target I. The groups $\mathrm{Hom}_{K(\mathcal{A})}(C, I[r])$ vanish for all r by K-injectivity, so the identity of I lifts to a map $s : I' \to I$ with $sv = \mathrm{id}_I$ in $K(\mathcal{A})$. Applying the same exact sequence to the target I' shows that the difference $vs - \mathrm{id}_{I'}$, whose composite with v is zero, factors through a vanishing Hom group; hence $vs = \mathrm{id}_{I'}$. The two equalities in $K(\mathcal{A})$ are precisely cochain homotopies. For K-projectives apply the covariant Hom long exact sequence instead, with $\mathrm{Hom}_{K(\mathcal{A})}(P, C[r]) = 0$, and obtain the dual homotopy inverse. This proof uses the all-integer cone triangle and does not invoke a bounded truncation. □

Proposition 10.14.11 (Remaining functorial replacement interfaces)**.** *Lean:* `conditional-interface:prop:weil-general-unbounded-replacement-existence-v1`

Uses: `def:weil-general-unbounded-kinjective-replacement-v1`, `def:weil-general-unbounded-kprojective-replacement-v1`, `def:weil-general-unbounded-kflat-replacement-v1`

The mathematical 079P construction gives functorial K-injectives in a Grothendieck category, but its concrete universe adapter is still required as a typed Lean term. A K-projective replacement additionally requires a suitable supply of projective objects and is not automatic for a sheaf category. For K-flat pullback, the later 06YY theorem supplies the allowed choice-independent localization alternative, so no strict K-flat endofunctor is asserted or required here.

Open obligation: `OBL-GENERAL-UNBOUNDED-KINJECTIVE-EXISTENCE`

Open obligation: `OBL-GENERAL-UNBOUNDED-KPROJECTIVE-EXISTENCE`

Proof. Uses: `def:weil-general-unbounded-kinjective-replacement-v1`, `def:weil-general-unbounded-kprojective-replacement-v1`, `def:weil-general-unbounded-kflat-replacement-v1`

The first marker records the still-missing concrete Lean export of the functorial 079P construction, not a missing mathematical existence theorem. The second records the genuinely unselected K-projective branch. The 06YS adapted objects and 06YY localization theorem below satisfy the alternative explicitly permitted by the K-flat obligation without manufacturing a strict replacement endofunctor. □

Definition 10.14.12 (Right derived model datum from K-injectives)**.** Lean: `conditional-interface:def:weil-general-unbounded-right-derived-v1`

Uses: `def:weil-general-unbounded-derived-localization-v1`, `def:weil-general-unbounded-kinjective-replacement-v1`

Let $G : \mathcal{A} \to \mathcal{B}$ be additive and fix a functorial K-injective replacement certificate on $\mathcal{A}$. Put

$$G_{\mathrm{res}} := G_{\mathrm{cw}} \circ I_{\mathcal{A}} : \mathrm{Ch}(\mathcal{A}) \to \mathrm{Ch}(\mathcal{B}).$$

The right-derived input condition is

$$G_{\mathrm{res}}(W_{\mathcal{A}}) \subseteq W_{\mathcal{B}}. \tag{R-W}$$

A right-derived output is a functor $RG : \mathsf{D}(\mathcal{A}) \to \mathsf{D}(\mathcal{B})$ together with a natural isomorphism

$$\rho : Q_{\mathcal{A}} \gg RG \xRightarrow{\sim} G_{\mathrm{res}} \gg Q_{\mathcal{B}}.$$

The notation RG is reserved for a chosen output; neither the replacement certificate nor (R-W) is implicit in the symbol.

Lemma 10.14.13 (The K-injective resolved functor preserves quasi-isomorphisms). *Lean:* `conditional-interface:lemma:weil-general-unbounded-right-resolved-preserves-qis-v1`

Uses: `def:weil-general-unbounded-right-derived-v1`

For an additive G and a fixed K-injective replacement certificate, $G_{\mathrm{res}} = G_{\mathrm{cw}} I_{\mathcal{A}}$ sends every arrow of $W_{\mathcal{A}}$ to a quasi-isomorphism. Thus (R-W) is a consequence of the replacement package, not an additional boundedness assumption.

Proof. Uses: `def:weil-general-unbounded-right-derived-v1,lemma:weil-general-unbounded-replacement-preserves-qis-v1,lemma:weil-general-unbounded-kmodel-qis-homotopy-v1`

For $u \in W_{\mathcal{A}}$, the replacement Lemma makes $I(u)$ a quasi-isomorphism, and the K-model Lemma makes it a homotopy equivalence because both endpoints are K-injective. An additive degreewise functor sends a homotopy inverse and its two homotopies to a homotopy inverse and homotopies. Therefore $G_{\mathrm{cw}} I(u)$ is a homotopy equivalence and hence a quasi-isomorphism. □

Proposition 10.14.14 (Conditional construction of the right derived functor). *Lean:* `conditional-interface:prop:weil-general-unbounded-right-derived-existence-v1`

Uses: `def:weil-general-unbounded-right-derived-v1,def:weil-general-unbounded-derived-localization-v1`

If (R-W) holds, a right-derived output exists by the universal property of the localization $Q_{\mathcal{A}}$.

Constructed by: `def:weil-general-unbounded-right-derived-v1`

Proof. Uses: `def:weil-general-unbounded-right-derived-v1,def:weil-general-unbounded-derived-localization-v1,lemma:weil-general-unbounded-right-resolved-preserves-qis-v1`

The composite $G_{\mathrm{res}} \gg Q_{\mathcal{B}}$ sends every arrow of $W_{\mathcal{A}}$ to an isomorphism by (R-W). The localization universal property gives RG and the displayed factorization isomorphism. The preceding K-injective-resolved Lemma supplies (R-W) for every additive G. The localization universal property then applies directly. □

Definition 10.14.15 (Left derived model datum from K-projectives). Lean: `conditional-interface:def:weil-general-unbounded-left-derived-v1`

Uses: `def:weil-general-unbounded-derived-localization-v1,def:weil-general-unbounded-kprojective-replacement-v1`

Let $F : \mathcal{A} \to \mathcal{B}$ be additive and fix a functorial K-projective replacement certificate on $\mathcal{A}$. Put

$$F_{\mathrm{res}} := F_{\mathrm{cw}} \circ P_{\mathcal{A}}.$$

The left-derived input condition is

$$F_{\mathrm{res}}(W_{\mathcal{A}}) \subseteq W_{\mathcal{B}}. \tag{L-W}$$

A left-derived output is a functor $LF : \mathsf{D}(\mathcal{A}) \to \mathsf{D}(\mathcal{B})$ and a natural isomorphism

$$\lambda : Q_{\mathcal{A}} \gg LF \xRightarrow{\sim} F_{\mathrm{res}} \gg Q_{\mathcal{B}}.$$

The output is not defined until the K-projective package and (L-W) are supplied.

Lemma 10.14.16 (The K-projective resolved functor preserves quasi-isomorphisms). *Lean:* `conditional-interface:lemma:weil-general-unbounded-left-resolved-preserves-qis-v1`

Uses: `def:weil-general-unbounded-left-derived-v1`

For an additive F and a fixed K-projective replacement certificate, $F_{\mathrm{res}} = F_{\mathrm{cw}} P_{\mathcal{A}}$ sends every arrow of $W_{\mathcal{A}}$ to a quasi-isomorphism. Hence the left input condition is automatic for K-projective models.

Proof. Uses: `def:weil-general-unbounded-left-derived-v1,lemma:weil-general-unbounded-replacement-preserves-qis-v1,lemma:weil-general-unbounded-kmodel-qis-homotopy-v1`

For $u \in W_{\mathcal{A}}$, $P(u)$ is a quasi-isomorphism by the replacement Lemma and is a homotopy equivalence by the K-projective case of the K-model Lemma. Applying the additive functor F_{cw} to its inverse and homotopies shows that $F_{\mathrm{res}}(u)$ is a homotopy equivalence, hence a quasi-isomorphism. □

Proposition 10.14.17 (Conditional construction of the left derived functor). *Lean:* `conditional-interface:prop:weil-general-unbounded-left-derived-existence-v1`

Uses: `def:weil-general-unbounded-left-derived-v1,def:weil-general-unbounded-derived-localization-v1`

If (L-W) holds, a left-derived output exists by the universal property of the localization $Q_{\mathcal{A}}$.

Constructed by: `def:weil-general-unbounded-left-derived-v1`

Proof. Uses: def:weil-general-unbounded-left-derived-v1,def:weil-general-unbounded-derived-localization-v1,lemma:weil-general-unbounded-left-resolved-preserves-qis-v1

The composite $F_{\mathrm{res}} \gg Q_{\mathcal{B}}$ inverts $W_{\mathcal{A}}$ by (L-W). Localization therefore supplies LF and the factorization isomorphism λ. The proof is on the full unbounded category and does not invoke a bounded-above replacement. □

Definition 10.14.18 (Left derived model datum from K-flat models)**.** Lean: conditional-interface:def:weil-general-unbounded-left-derived-kflat-v2

Uses: def:weil-general-unbounded-derived-localization-v1,def:weil-general-unbounded-kflat-replacement-v1

Let $F : \mathcal{A} \to \mathcal{B}$ be additive and fix a functorial K-flat replacement certificate on the tensor source. Put

$$F^{\flat}_{\mathrm{res}} := F_{\mathrm{cw}} \circ P^{\mathrm{fl}}_{\mathcal{A}}.$$

The K-flat left input condition is the explicit field

$$F^{\flat}_{\mathrm{res}}(W_{\mathcal{A}}) \subseteq W_{\mathcal{B}}. \tag{L-W$^{\flat}$}$$

When this field is supplied, a K-flat left-derived output is a functor $LF^{\flat} : D(\mathcal{A}) \to D(\mathcal{B})$ with a natural factorization isomorphism

$$Q_{\mathcal{A}} \gg LF^{\flat} \xRightarrow{\sim} F^{\flat}_{\mathrm{res}} \gg Q_{\mathcal{B}}.$$

K-flatness by itself does not assert this field for an arbitrary additive functor; it is automatic for the fixed K-flat tensor factor treated below.

Proposition 10.14.19 (Conditional construction from K-flat models)**.** Lean: conditional-interface:prop:weil-general-unbounded-left-derived-kflat-existence-v2

Uses: def:weil-general-unbounded-left-derived-kflat-v2, def:weil-general-unbounded-derived-localization-v1

If (L-W$^{\flat}$) holds, the K-flat left-derived output exists by the localization universal property. It is a construction on the full all-integer derived category and does not use D^{-}.

Constructed by: def:weil-general-unbounded-left-derived-kflat-v2

Proof. Uses: def:weil-general-unbounded-left-derived-kflat-v2,def:weil-general-unbounded-derived-localization-v1

The composite $F^{\flat}_{\mathrm{res}} \gg Q_{\mathcal{B}}$ inverts $W_{\mathcal{A}}$ by (L-W$^{\flat}$). Localization supplies the factor functor and its factorization isomorphism. □

Definition 10.14.20 (Left derived tensor from K-flat models)**.** Lean: conditional-interface:def:weil-general-unbounded-derived-tensor-v1

Uses: def:weil-general-unbounded-k-models-v1,def:weil-general-unbounded-kflat-replacement-v1,def:weil-general-unbounded-derived-localization-v1

For a tensor abelian category $(\mathcal{A}, \otimes)$ and a fixed K-flat complex $S \in \mathrm{Ch}(\mathcal{A})$, the tensor functor $T_S(C) = S \otimes C$ has the K-flat model definition

$$S \otimes^{\mathbf{L}} Q_{\mathcal{A}}(C) := Q_{\mathcal{A}}\big(S \otimes P^{\mathrm{fl}}_{\mathcal{A}}(C)\big) \qquad (C \in \mathrm{Ch}(\mathcal{A})).$$

The K-flat condition on S is the field that makes $S \otimes -$ preserve quasi-isomorphisms; it is what makes this independent of the selected replacement of C. If S is not K-flat, both tensor factors must instead be replaced and a separate comparison certificate is required. Thus K-flatness is used here only for tensor/scalar-extension functors, not as a replacement theorem for an arbitrary additive functor.

Lemma 10.14.21 (K-flat tensoring preserves quasi-isomorphisms)**.** Lean: conditional-interface:lemma:weil-general-unbounded-kflat-tensor-qis-v1

Uses: def:weil-general-unbounded-k-models-v1

If S is K-flat and $u : M \to N$ is a quasi-isomorphism, then $1_S \otimes u : S \otimes M \to S \otimes N$ is a quasi-isomorphism.

Proof. Uses: def:weil-general-unbounded-k-models-v1

The cone of u is acyclic. With the total tensor differential fixed in Definition 10.14.2, the identity on the two direct summands gives a sign-preserving isomorphism

$$\mathrm{Cone}(1_S \otimes u) \cong S \otimes \mathrm{Cone}(u).$$

The right side is acyclic by K-flatness, and the cone criterion gives the claim. □

Proposition 10.14.22 (K-flat tensor model satisfies the left input condition)**.** Lean: `conditional-interface:prop:weil-general-unbounded-kflat-tensor-input-v1`

Uses: `def:weil-general-unbounded-derived-tensor-v1`

For the tensor functor T_S in Definition 10.14.20, the left input condition (L-W$^\flat$) holds for every chosen K-flat replacement package.

Proof. Uses: `def:weil-general-unbounded-derived-tensor-v1,lemma:weil-general-unbounded-kflat-tensor-qis-v1`

Apply the preceding Lemma to S and the image under the replacement functor of any quasi-isomorphism. Naturality of the replacement map makes the resulting tensor map exactly the map defining the left input condition. □

Definition 10.14.23 (Comparison datum for two right replacement packages)**.** Lean: `conditional-interface:def:weil-general-unbounded-right-model-comparison-v1`

Uses: `def:weil-general-unbounded-right-derived-v1,def:weil-general-unbounded-kinjective-replacement-v1`

For two K-injective packages I and I' on $\mathcal{A}$, a right-model comparison datum is a natural transformation $c : I \Rightarrow I'$ such that each c_M and each $G_{\mathrm{cw}}(c_M)$ is a quasi-isomorphism. Given right-derived outputs (RG_I, ρ_I) and $(RG_{I'}, \rho_{I'})$, the comparison isomorphism is oriented by

$$\rho_{I',M} \circ \chi_{Q_{\mathcal{A}}M} = Q_{\mathcal{B}}(G_{\mathrm{cw}}c_M) \circ \rho_{I,M}. \tag{R-C}$$

The comparison datum is extra input and is not inferred from the two replacement certificates.

Proposition 10.14.24 (Right-derived model independence, conditional)**.** Lean: `conditional-interface:prop:weil-general-unbounded-right-model-independence-v1`

Uses: `def:weil-general-unbounded-right-model-comparison-v1`

Given the right-model comparison datum, there is a unique natural isomorphism $\chi : RG_I \Rightarrow RG_{I'}$ satisfying (R-C).

Constructed by: `def:weil-general-unbounded-right-model-comparison-v1`

Proof. Uses: `def:weil-general-unbounded-right-model-comparison-v1,def:weil-general-unbounded-right-derived-v1`

Each $Q_{\mathcal{B}}(G_{\mathrm{cw}}c_M)$ is invertible. Equation (R-C) therefore defines the component of χ on every object represented by M; the naturality of c, ρ_I, and $\rho_{I'}$ makes these components compatible with roofs. The localization universal property extends them uniquely to all objects of $\mathsf{D}(\mathcal{A})$. Applying the same construction to the inverse of each localized comparison proves that χ is an isomorphism. □

Definition 10.14.25 (Comparison datum for two left replacement packages)**.** Lean: `conditional-interface:def:weil-general-unbounded-left-model-comparison-v1`

Uses: `def:weil-general-unbounded-left-derived-v1,def:weil-general-unbounded-kprojective-replacement-v1,def:weil-general-unbounded-kflat-replacement-v1`

For two selected left model packages P and P', a left-model comparison datum is a natural transformation $c : P \Rightarrow P'$ such that each c_M and each $F_{\mathrm{cw}}(c_M)$ is a quasi-isomorphism. For left-derived outputs (LF_P, λ_P) and $(LF_{P'}, \lambda_{P'})$, the comparison is oriented by

$$\lambda_{P',M} \circ \chi_{Q_{\mathcal{A}}M} = Q_{\mathcal{B}}(F_{\mathrm{cw}}c_M) \circ \lambda_{P,M}. \tag{L-C}$$

Proposition 10.14.26 (Left-derived model independence, conditional)**.** Lean: `conditional-interface:prop:weil-general-unbounded-left-model-independence-v1`

Uses: `def:weil-general-unbounded-left-model-comparison-v1`

Given the left-model comparison datum, there is a unique natural isomorphism $\chi : LF_P \Rightarrow LF_{P'}$ satisfying (L-C). In particular, a K-projective and a K-flat construction agree only after their comparison datum has been supplied.

Constructed by: `def:weil-general-unbounded-left-model-comparison-v1`

Proof. Uses: `def:weil-general-unbounded-left-model-comparison-v1,def:weil-general-unbounded-left-derived-v1`

The localized maps $Q_{\mathcal{B}}(F_{\mathrm{cw}}c_M)$ are isomorphisms. The factorization equations define χ on the essential image of $Q_{\mathcal{A}}$, and the localization universal property extends it naturally and uniquely. The inverse comparison gives the inverse. No existence of a comparison between arbitrary concrete packages is asserted. □

Definition 10.14.27 (Exact-functor special case). Lean: `conditional-interface:def:weil-general-unbounded-exact-left-derived-v1`

Uses: `def:weil-general-unbounded-derived-localization-v1`, `def:weil-general-unbounded-left-derived-v1`

If $F : \mathcal{A} \to \mathcal{B}$ is additive and exact, its termwise complex functor is declared to have the exact derived output

$$LF_{\mathrm{ex}} : D(\mathcal{A}) \to D(\mathcal{B}),$$

characterized by $Q_{\mathcal{A}} \gg LF_{\mathrm{ex}} \cong F_{\mathrm{cw}} \gg Q_{\mathcal{B}}$. This definition does not choose a K-projective or K-flat replacement; its existence is the exactness lemma below.

Lemma 10.14.28 (Exact termwise functors preserve quasi-isomorphisms). Lean: *conditional-interface:lemma:weil-general-unbounded-exact-termwise-qis-v1*

Uses: *def:weil-general-unbounded-exact-left-derived-v1*, *def:weil-general-unbounded-derived-localization-v1*

If F is exact, then F_{cw} sends every quasi-isomorphism to a quasi-isomorphism.

Proof. Uses: `def:weil-general-unbounded-derived-localization-v1`

For a quasi-isomorphism u, its mapping cone is acyclic. Exactness preserves the kernels and images defining each cohomology object and sends the cone direct-sum differential to the corresponding cone differential. Hence $F_{\mathrm{cw}}\,\mathrm{Cone}(u)$ is acyclic, so the cone criterion gives that $F_{\mathrm{cw}}u$ is a quasi-isomorphism. □

Proposition 10.14.29 (Exact left derived output exists without models). Lean: *conditional-interface:prop:weil-general-unbounded-exact-left-existence-v1*

Uses: *def:weil-general-unbounded-exact-left-derived-v1*, *def:weil-general-unbounded-derived-localization-v1*

The exact derived output in Definition 10.14.27 exists by localization. Whenever a K-projective or K-flat left model output also exists for the same functor, it is uniquely naturally isomorphic to LF_{ex}.

Constructed by: *def:weil-general-unbounded-exact-left-derived-v1*

Proof. Uses: `def:weil-general-unbounded-exact-left-derived-v1`, `lemma:weil-general-unbounded-exact-termwise-qis-v1`, `def:weil-general-unbounded-left-derived-v1`

The Lemma supplies the W-preservation field required by localization, giving LF_{ex}. Compare its factorization with any model output on the image of $Q_{\mathcal{A}}$; the uniqueness part of the localization universal property extends the comparison uniquely to all derived objects. □

Proposition 10.14.30 (Exact right derived output exists without models). Lean: *conditional-interface:prop:weil-general-unbounded-exact-right-existence-v1*

Uses: *def:weil-general-unbounded-derived-localization-v1*, *def:weil-general-unbounded-right-derived-v1*, *def:weil-general-unbounded-exact-left-derived-v1*

If $G : \mathcal{A} \to \mathcal{B}$ is exact, then termwise G_{cw} defines a right-derived output by localization, without a K-injective replacement. In the nonexact case the K-injective input of Definition 10.14.12 remains necessary.

Constructed by: *def:weil-general-unbounded-exact-left-derived-v1*

Proof. Uses: `def:weil-general-unbounded-derived-localization-v1`, `def:weil-general-unbounded-right-derived-v1`, `lemma:weil-general-unbounded-exact-termwise-qis-v1`

Apply the exact-termwise Lemma to G. Thus G_{cw} inverts $W_{\mathcal{A}}$, and localization supplies the factor functor and its factorization isomorphism. Without exactness no such W-preservation field is available from the underived additive functor alone. □

Lemma 10.14.31 (Exact left adjoints preserve K-injective targets). Lean: *conditional-interface:lemma:weil-general-unbounded-right-adjoint-preserves-kinjective-v1*

Uses: *def:weil-general-unbounded-k-models-v1*

Suppose $F \dashv G$ underived and F is exact. If J is K-injective in $\mathrm{Ch}(\mathcal{B})$, then $G_{\mathrm{cw}}J$ is K-injective in $\mathrm{Ch}(\mathcal{A})$.

Proof. Uses: `def:weil-general-unbounded-k-models-v1`

For acyclic C, exactness makes $F_{\mathrm{cw}}C$ acyclic. The underived adjunction gives a natural isomorphism of Hom complexes

$$\mathrm{Hom}^{\bullet}(C, G_{\mathrm{cw}}J) \cong \mathrm{Hom}^{\bullet}(F_{\mathrm{cw}}C, J),$$

which is acyclic because J is K-injective. □

Lemma 10.14.32 (Underived adjunction induces a Hom-complex isomorphism)**.** *Lean:* `conditional-interface:lemma:weil-general-unbounded-adjunction-hom-complex-v1`

Uses: `def:weil-general-unbounded-k-models-v1`

Suppose $F \dashv G$ underived with Hom-set bijections $\varphi_{A,B}$. For all complexes C and J, the degreewise formula

$$(\Phi^r h)^n := \varphi_{C^n, J^{n+r}}(h^n)$$

defines an isomorphism of total Hom complexes

$$\Phi_{C,J} : \mathrm{Hom}^\bullet(F_{\mathrm{cw}} C, J) \xrightarrow{\sim} \mathrm{Hom}^\bullet(C, G_{\mathrm{cw}} J).$$

Proof. Uses: `def:weil-general-unbounded-k-models-v1`

The inverse is obtained by applying φ^{-1} in every degree. For a homogeneous h of degree r, naturality in the target and source gives

$$\varphi(d_J h^n) - (-1)^r \varphi(h^{n+1} F(d_C^n)) = G(d_J)\varphi(h^n) - (-1)^r \varphi(h^{n+1}) d_C^n,$$

which is exactly $\Phi^{r+1}(dh) = d\Phi^r(h)$. The same calculation for φ^{-1} proves that the two complex maps are inverse. □

Definition 10.14.33 (K-model localization Hom comparison certificate)**.** Lean: `conditional-interface:def:weil-general-unbounded-kmodel-localization-hom-v2`

Uses: `def:weil-general-unbounded-derived-localization-v1,def:weil-general-unbounded-k-models-v1`

A K-model localization Hom comparison certificate consists of the following two natural bijections, induced by the localization functor. For every K-projective P and every complex Y, it contains

$$\mathrm{Hom}_{K(\mathcal{A})}(P, Y) \xrightarrow{\sim} \mathrm{Hom}_{D(\mathcal{A})}(Q_{\mathcal{A}} P, Q_{\mathcal{A}} Y), \tag{KP-Hom}$$

and for every complex X and every K-injective I, it contains

$$\mathrm{Hom}_{K(\mathcal{A})}(X, I) \xrightarrow{\sim} \mathrm{Hom}_{D(\mathcal{A})}(Q_{\mathcal{A}} X, Q_{\mathcal{A}} I). \tag{KI-Hom}$$

The fields include naturality in both variables and compatibility with composition and identities. This is a localization theorem, not part of the definition of a K-projective or K-injective complex.

Proposition 10.14.34 (Generic K-model localization Hom interface)**.** *Lean:* `conditional-interface:prop:weil-general-unbounded-kmodel-localization-hom-open-v2`

Uses: `def:weil-general-unbounded-kmodel-localization-hom-v2`

The preceding certificate is a generic interface: a concrete coefficient category must derive it from its localization and adapted replacement classes before using either K-model adjunction theorem. The ringed-topos specialization below supplies that derivation from the everywhere-defined left and right derived functors.

Proof. Uses: `def:weil-general-unbounded-kmodel-localization-hom-v2`

For a quasi-isomorphism u, the cone of u is acyclic. K-projectivity and K-injectivity make the corresponding total Hom complex against that cone acyclic, so precomposition or postcomposition by u is a bijection on homotopy classes. Completing the roof calculation, its naturality, and the two composition laws is exactly the recorded localization obligation; it is not silently imported from the bounded-below API. □

Theorem 10.14.35 (Unbounded derived adjunction from K-projective and K-injective models)**.** *Lean:* `conditional-interface:thm:weil-general-unbounded-kprojective-kinjective-adjunction-v2`

Uses: `def:weil-general-unbounded-left-derived-v1,def:weil-general-unbounded-right-derived-v1,def:weil-general-unbounded-kmodel-localization-hom-v2`

Let $F : \mathcal{A} \rightleftarrows \mathcal{B} : G$ be an additive underived adjunction. Fix a functorial K-projective replacement on $\mathcal{A}$, a functorial K-injective replacement on $\mathcal{B}$, their localization outputs LF and RG, and K-model localization Hom comparison certificates for $\mathcal{A}$ and $\mathcal{B}$. Then, on the full all-integer derived categories,

$$LF : D(\mathcal{A}) \rightleftarrows D(\mathcal{B}) : RG, \qquad LF \dashv RG.$$

No exactness of F or G is required in this K-projective/K-injective model.

Proof. Uses: `def:weil-general-unbounded-left-derived-v1,def:weil-general-unbounded-right-derived-v1,def:weil-general-unbounded-kmodel-localization-hom-v2,lemma:weil-general-unbounded-adjunction-hom-complex-v1,prop:weil-general-unbounded-left-derived-existence-v1,prop:weil-general-unbounded-right-derived-existence-v1`

For $X \in \mathrm{Ch}(\mathcal{A})$ and $Y \in \mathrm{Ch}(\mathcal{B})$, put $P = P_{\mathcal{A}}X$ and $I = I_{\mathcal{B}}Y$. The two comparison fields and the Hom-complex adjunction give the natural chain

$$\begin{aligned}\mathrm{Hom}_{D(\mathcal{B})}(LF(Q_{\mathcal{A}}X), Q_{\mathcal{B}}Y) &\cong \mathrm{Hom}_{K(\mathcal{B})}(F_{\mathrm{cw}}P, I)\\ &\cong \mathrm{Hom}_{K(\mathcal{A})}(P, G_{\mathrm{cw}}I)\\ &\cong \mathrm{Hom}_{D(\mathcal{A})}(Q_{\mathcal{A}}X, RG(Q_{\mathcal{B}}Y)).\end{aligned}$$

The first line uses the K-injective-target comparison and the localized replacement of Y; the last uses the K-projective-source comparison and the localized replacement of X. Naturality supplies the adjunction. Defining the unit and counit as the images of identity arrows makes both triangle identities the usual inverse-bijection identities. □

Theorem 10.14.36 (Unbounded derived adjunction). *Lean:* `conditional-interface:thm:weil-general-unbounded-derived-adjunction-v1`

Uses: `def:weil-general-unbounded-exact-left-derived-v1`, `def:weil-general-unbounded-right-derived-v1`, `def:weil-general-unbounded-kmodel-localization-hom-v2`

Suppose $F \dashv G$ underived, F is exact, and a functorial K-injective replacement certificate on $\mathcal{B}$ and K-model localization Hom comparison certificates for $\mathcal{A}$ and $\mathcal{B}$ have been supplied. Then the exact-functor construction LF is left adjoint to RG on the full unbounded derived categories:

$$LF_{\mathrm{ex}} : D(\mathcal{A}) \rightleftarrows D(\mathcal{B}) : RG, \qquad LF_{\mathrm{ex}} \dashv RG.$$

Proof. Uses: `def:weil-general-unbounded-exact-left-derived-v1`, `def:weil-general-unbounded-right-derived-v1`, `def:weil-general-unbounded-kmodel-localization-hom-v2`, `lemma:weil-general-unbounded-right-adjoint-preserves-kinjective-v1`, `lemma:weil-general-unbounded-adjunction-hom-complex-v1`, `lemma:weil-general-unbounded-kmodel-qis-homotopy-v1`

Replace a target complex B by its K-injective model $J = I(B)$. Localization against a K-injective target identifies derived morphisms with homotopy classes, and the termwise adjunction gives

$$\mathrm{Hom}_{K(\mathcal{B})}(F_{\mathrm{cw}}A, J) \cong \mathrm{Hom}_{K(\mathcal{A})}(A, G_{\mathrm{cw}}J).$$

The preceding K-injective-target Lemma makes $G_{\mathrm{cw}}J$ K-injective, so the right-hand side is the Hom set in $D(\mathcal{A})$ and the left-hand side is the Hom set from $LF(Q_{\mathcal{A}}A)$ to $Q_{\mathcal{B}}B$. Naturality is inherited from the chain-level Hom-complex isomorphism; the unit and counit are the images of the identity maps and satisfy the two triangle identities. □

Proposition 10.14.37 (Restriction after the unbounded construction). *Lean:* `conditional-interface:prop:weil-general-unbounded-derived-restrictions-v1`

Uses: `def:weil-general-unbounded-derived-localization-v1`

The categories $D^+(\mathcal{A})$ and $D^b_c(X, \Lambda)$ are full subcategory restrictions of the already defined $D(\mathcal{A})$. Restricting the adjunction requires separate lower-amplitude, upper-amplitude, and constructibility/boundedness preservation certificates. No D^- is needed in the exact same-coefficient case.

Proof. Uses: `def:weil-general-unbounded-derived-localization-v1`, `thm:weil-general-unbounded-derived-adjunction-v1`

The subcategories in the statement are full by construction, so their Hom sets are the same Hom sets as in the unbounded categories. Restriction certificates ensure that both functors land in the indicated subcategories; restricting the unbounded Hom-bijection and its unit/counit then preserves the triangle identities. No D^- enters this operation. □

Proposition 10.14.38 (Crosswalk to the same-coefficient pullback case). *Lean:* `conditional-interface:prop:weil-general-unbounded-same-coefficient-crosswalk-v1`

Uses: `def:weil-general-unbounded-exact-left-derived-v1`, `def:weil-general-unbounded-right-derived-v1`

For a same-coefficient inverse-image functor f^* that is exact, the general left-derived construction reduces to the termwise localization LF_{ex} and therefore needs neither K-projective nor K-flat replacements. The direct-image functor f_* is a separate right-derived problem: unless it is exact, its unbounded Rf_* requires the K-injective replacement package and the W-preservation proof in the right model datum. Thus the exact f^* simplification does not close the right-derived construction.

Proof. Uses: `def:weil-general-unbounded-exact-left-derived-v1`, `prop:weil-general-unbounded-exact-left-existence-v1`, `def:weil-general-unbounded-right-derived-v1`

Apply the exact-termwise Lemma and the exact-left existence Proposition to f^*. For f_*, Definition 10.14.12 explicitly requires the K-injective package and (R-W); no exactness of f^* supplies either field for f_*. □

Proposition 10.14.39 (General model-layer status). *Lean:* `conditional-interface:prop:weil-general-unbounded-model-layer-status-v1`

Uses: `def:weil-general-unbounded-objectwise-replacement-v1`, `def:weil-general-unbounded-kinjective-replacement-v1`, `def:weil-general-unbounded-kprojective-replacement-v1`, `def:weil-general-unbounded-kflat-replacement-v1`, `def:weil-general-unbounded-right-derived-v1`, `def:weil-general-unbounded-left-derived-v1`, `def:weil-general-unbounded-kmodel-localization-hom-v2`

This section supplies the generic definitions on the full unbounded derived categories. The ringed-site specialization below constructs the selected objectwise K-injective and K-flat models. It does not construct the unused K-projective branch or import any derived Hom comparison beyond explicitly proved hypotheses.

Open obligation: `OBL-GENERAL-UNBOUNDED-KPROJECTIVE-OBJECTWISE`

Proof. Uses: `def:weil-general-unbounded-objectwise-replacement-v1`, `def:weil-general-unbounded-right-derived-v1`, `def:weil-general-unbounded-left-derived-v1`, `def:weil-general-unbounded-kmodel-localization-hom-v2`

Every derived-functor output above is obtained only after its replacement, W-preservation, and, for adjunction, Hom-complex fields are supplied. The later ringed-site theorems supply the two selected objectwise fields. The displayed K-projective marker remains a release-blocking interface rather than an axiom imported by notation. □

10.15 A K-flat/K-injective construction on unbounded ringed-topos derived categories

Definition 10.15.1 (Ringed-topos ambient data). Lean: `conditional-interface:def:weil-fresh-ringed-topos-ambient-v1`

Uses: `def:weil-general-unbounded-derived-localization-v1`, `def:weil-general-unbounded-k-models-v1`, `def:weil-a2-module-category`

Let

$$f : (X, \mathcal{O}_X) \longrightarrow (Y, \mathcal{O}_Y)$$

be a morphism of universe-small ringed topoi. Put

$$\mathcal{A}_X := \mathrm{Mod}(X, \mathcal{O}_X), \qquad \mathcal{A}_Y := \mathrm{Mod}(Y, \mathcal{O}_Y),$$

and let $K_X = K(\mathcal{A}_X)$, $K_Y = K(\mathcal{A}_Y)$ and $D_X = D(\mathcal{A}_X)$, $D_Y = D(\mathcal{A}_Y)$ be the all-integer homotopy categories and their quasi-isomorphism localizations from Definition 10.14.1. The termwise functors are denoted by f^*_{cw} and $f_{*,\mathrm{cw}}$. On the heart,

$$f^*M := \mathcal{O}_X \otimes_{f^{-1}\mathcal{O}_Y} f^{-1}M, \qquad f_*N := \text{the induced } \mathcal{O}_Y\text{-module direct image},$$

and the specified underived adjunction is

$$\varphi_{M,N} : \mathrm{Hom}_{\mathcal{A}_X}(f^*M, N) \xrightarrow{\sim} \mathrm{Hom}_{\mathcal{A}_Y}(M, f_*N). \tag{A0}$$

The inverse-image functor f^{-1} is exact. The symbols $Q_X : K_X \to D_X$ and $Q_Y : K_Y \to D_Y$ denote the localization functors.

Definition 10.15.2 (Universe/Grothendieck certificate). Lean: `conditional-interface:def:weil-fresh-ringed-topos-grothendieck-certificate-v1`

Uses: `def:weil-fresh-ringed-topos-ambient-v1`

A Grothendieck certificate for $(X, \mathcal{O}_X)$ consists of the assertion that $\mathcal{A}_X$ is abelian, has all small limits and colimits, filtered colimits are exact, and a generator U_X is supplied. The analogous fields are required for Y. The certificate also fixes the universes in which the sites, module sheaves, complexes, and localization roofs are formed.

Proposition 10.15.3 (Module sheaves on a small ringed site are Grothendieck). *Lean:* `conditional-interface:prop:weil-fresh-ringed-topos-module-grothendieck-v1`

Uses: `def:weil-fresh-ringed-topos-grothendieck-certificate-v1`, `def:weil-fresh-ringed-topos-ambient-v1`

For a universe-small ringed site, the category of sheaves of modules over its ring sheaf has a Grothendieck certificate. Consequently the module-sheaf categories in Definition 10.15.1 admit the unbounded K-injective theorem once their universe adapters are supplied.

Proof. Uses: def:weil-fresh-ringed-topos-grothendieck-certificate-v1

The category of presheaves of modules is AB5 and is generated by the free Yoneda modules $j_{U!}\mathcal{O}$ (one for each site object U). Module sheafification is an exact left adjoint to the inclusion of sheaves, so it preserves colimits and exact filtered colimits; the sheafified free Yoneda family has a coproduct generator. The resulting abelian category is therefore Grothendieck. This is the module-sheaf construction of Stacks, Section 21.19, with the module-sheaf inputs in Section 18.14; the only project-specific input is the fixed universe-small site. Thus the two certificates required in the preceding Definition are obtained after the universe adapter has been installed. □

Definition 10.15.4 (K-injective adapted class)**.** Lean: conditional-interface:def:weil-fresh-k injective-adapted-class-v1

Uses: def:weil-fresh-ringed-topos-ambient-v1, def:weil-general-unbounded-k-models-v1

Define

$$\mathsf{KI}_X := \{I \in \mathrm{Ch}(\mathcal{A}_X) \mid \mathrm{Hom}^\bullet_X(C, I) \text{ is acyclic for every acyclic } C\},$$

and similarly KI_Y. A K-injective resolution arrow is a quasi-isomorphism $\iota_B : B \to I$ with $I \in \mathsf{KI}_X$ (or the corresponding Y-version). This is the adapted target class for the right derived direct image.

Lemma 10.15.5 (A quasi-isomorphism between K-injectives is a homotopy equivalence)**.** *Lean: conditional-interface:lemma:weil-fresh-kinjective-quis-homotopy-v1*

Uses: def:weil-fresh-kinjective-adapted-class-v1

If $u : I \to I'$ is a quasi-isomorphism with $I, I' \in \mathsf{KI}_X$, then u is an isomorphism in K_X; equivalently, it has a cochain homotopy inverse. The same assertion holds over Y.

Proof. Uses: def:weil-fresh-kinjective-adapted-class-v1

The cone $C = \mathrm{Cone}(u)$ is acyclic. In the cone triangle $I \to I' \to C \to I[1]$, the groups $\mathrm{Hom}_{K_X}(C, I[r])$ and $\mathrm{Hom}_{K_X}(C, I'[r])$ vanish for every r by the K-injective Hom-complex condition. The two long exact Hom sequences first lift id_I to a map $s : I' \to I$ with $su = 1_I$, and then show $us = 1_{I'}$. Equalities in K_X are precisely cochain homotopies. □

Definition 10.15.6 (Selected 079P transfinite replacement data)**.** Lean: conditional-interface: def:weil-fresh-kinjective-079p-construction-v1

Uses: def:weil-fresh-ringed-topos-grothendieck-certificate-v1, def:weil-fresh-kinjective-adapted -class-v1, def:weil-general-unbounded-kinjective-replacement-v1

Given a Grothendieck certificate, fix the generator, a regular cardinal, a set of bounded-above small acyclic tests, the 079M map-killing cell functor, the 079N injective-term cell functor, and a terminal ordinal of sufficiently large cofinality. Define $I_X(B)$ to be the terminal value of their alternating transfinite tower and $\iota_X(B) : B \to I_X(B)$ to be the composite tower map. The identical selection over Y defines I_Y and ι_Y.

Proposition 10.15.7 (Functorial unbounded K-injectives)**.** *Lean: conditional-interface: prop:weil-fresh-kinjective-existence-079p-v1*

Uses: def:weil-fresh-ringed-topos-grothendieck-certificate-v1, def:weil-fresh-kinjective-adapted-class-v1, def:weil-general-unbounded-kinjective-replacement-v1, def:weil-fresh-kinjective-079p-construction-v1

Assume the Grothendieck certificate for $\mathcal{A}_X$. There are an ordinary functor $I_X : \mathrm{Ch}(\mathcal{A}_X) \to \mathrm{Ch}(\mathcal{A}_X)$ and a natural quasi-isomorphism $\iota_X : \mathrm{id} \Rightarrow I_X$ such that $I_X(B) \in \mathsf{KI}_X$ for every B. The analogous functor I_Y exists under the Y-certificate.

Constructed by: def:weil-fresh-kinjective-079p-construction-v1

Proof. Uses: def:weil-fresh-ringed-topos-grothendieck-certificate-v1, def:weil-fresh-kinjective-adapted-class-v1, def:weil-fresh-kinjective-079p-construction-v1

This is Stacks, Theorem 19.12.6. For completeness, its transfinite construction is as follows. Choose a generator U and a cardinal κ dominating the terms of the small bounded-above acyclic tests. Let $(K_i)_i$ be a set of representatives of those tests. The pushout functor $\mathbf{M}$ of Stacks Lemma 19.12.4 attaches the maps $K_i \to C$ and makes each such map null-homotopic; the functor $\mathbf{N}$ of Lemma 19.12.5 supplies a functorial termwise monomorphic quasi-isomorphism whose component maps factor through injective objects. Set

$$T_0 = \mathrm{id}, \qquad T_{\alpha+1} = \mathbf{NM}T_\alpha, \qquad T_\beta = \operatorname*{colim}_{\alpha<\beta} T_\alpha \quad (\beta \text{ limit}).$$

Every transition is a monomorphism and quasi-isomorphism; AB5 preserves these properties at limit stages. If $\mathrm{cf}(\Theta) > \max\{\kappa, |U|, \aleph_0\}$, smallness makes every map from U (and every K_i) factor through a stage $< \Theta$. The generator extension criterion then makes each term of $T_\Theta(B)$ injective, while the $\mathbf{M}$ step makes every $K_i \to T_\Theta(B)$ null-homotopic. The small-test criterion for K-injectivity gives $T_\Theta(B) \in \mathsf{KI}_X$. Taking $I_X = T_\Theta$ and composing the transition maps gives ι_X and its naturality. This is exactly the functorial replacement certificate of the generic interface; no boundedness assumption is used. □

Definition 10.15.8 (Unbounded right derived direct image from K-injectives)**.** Lean: `conditional-interface:def:weil-fresh-rfstar-kinjective-v1`

Uses: `def:weil-fresh-ringed-topos-ambient-v1`, `def:weil-fresh-kinjective-adapted-class-v1`, `def:weil-fresh-kinjective-079p-construction-v1`

Using the selected 079P datum, put

$$\widehat{f_*} := f_{*,\mathrm{cw}} I_X : \mathrm{Ch}(\mathcal{A}_X) \longrightarrow \mathrm{Ch}(\mathcal{A}_Y).$$

The unbounded right derived direct image is the localization factor

$$Rf_* : D_X \longrightarrow D_Y, \qquad Rf_* Q_X(B) := Q_Y\big(f_{*,\mathrm{cw}} I_X(B)\big), \tag{R1}$$

with its factorization isomorphism understood as part of the localization output.

Lemma 10.15.9 (The K-injective resolved direct image inverts quasi-isomorphisms)**.** Lean: `conditional-interface:lemma:weil-fresh-rfstar-inverts-qis-v1`

Uses: `def:weil-fresh-rfstar-kinjective-v1`

For $u \in W_X$, the map $\widehat{f_*}(u)$ is a quasi-isomorphism.

Proof. Uses: `def:weil-fresh-rfstar-kinjective-v1`, `prop:weil-fresh-kinjective-existence-079p-v1`, `lemma:weil-fresh-kinjective-qis-homotopy-v1`

Naturality of ι_X and two-out-of-three make $I_X(u)$ a quasi-isomorphism. Its source and target are K-injective, so the preceding Lemma gives a homotopy inverse. The additive termwise functor f_* carries that inverse and both homotopies to the corresponding data over Y; therefore $\widehat{f_*}(u)$ is a homotopy equivalence and hence a quasi-isomorphism. □

Proposition 10.15.10 (Existence of Rf_*)**.** Lean: `conditional-interface:prop:weil-fresh-rfstar-existence-v1`

Uses: `def:weil-fresh-rfstar-kinjective-v1`, `def:weil-general-unbounded-derived-localization-v1`

The formula (R1) defines a functor on the full unbounded category D_X and is a right-derived direct image. Any other output equipped with the same right-derived factorization property is uniquely naturally isomorphic to it.

Proof. Uses: `def:weil-fresh-rfstar-kinjective-v1`, `lemma:weil-fresh-rfstar-inverts-qis-v1`, `def:weil-general-unbounded-derived-localization-v1`

The preceding Lemma says that $Q_Y \widehat{f_*}$ inverts W_X; the universal property of Q_X therefore supplies the factor Rf_*. The K-injective adapted class is right adapted, so this factor is the right-derived output of the K-injective construction (Stacks, the derived-functor construction following Theorem 19.12.6). If (R', ρ') is any other right-derived output, its factorization isomorphism and the one for (R1) agree on the image of Q_X; the uniqueness part of the localization universal property extends that comparison to a unique natural isomorphism $Rf_* \simeq R'$. Thus replacement independence uses uniqueness of the derived output, not an unproved comparison between arbitrary replacement functors. □

Definition 10.15.11 (K-flat adapted source class)**.** Lean: `conditional-interface:def:weil-fresh-kflat-adapted-class-v1`

Uses: `def:weil-fresh-ringed-topos-ambient-v1`, `def:weil-general-unbounded-k-models-v1`

Let

$$\mathsf{KF}_Y^{\mathrm{ad}} := \{P \in \mathrm{Ch}(\mathcal{A}_Y) \mid P \text{ is K-flat over } \mathcal{O}_Y \text{ and each } P^n \text{ is a flat } \mathcal{O}_Y\text{-module}\}.$$

An adapted source arrow is a quasi-isomorphism $p : P \to A$ with $P \in \mathsf{KF}_Y^{\mathrm{ad}}$. This class is used only to compute the tensor/scalar-extension functor f^*; it is not a class of K-projectives.

Definition 10.15.12 (Selected 06YS K-flat resolution data)**.** Lean: `conditional-interface:def:weil-fresh-kflat-06ys-construction-v1`

Uses: `def:weil-fresh-kflat-adapted-class-v1`

For a complex A, fix the canonical truncation tower. At stage m, select a bounded-above flat resolution $P_m \to \tau_{\le m} A$, with the 06YS split-monic transition maps, and define

$$P_A := \operatorname*{colim}_m P_m, \qquad p_A : P_A \longrightarrow A$$

from the compatible stage maps. This is objectwise data; no strict functorial K-flat package is included in the definition.

Proposition 10.15.13 (Every complex has a K-flat adapted source). *Lean:* `conditional-interface:prop:weil-fresh-kflat-existence-06ys-v1`

Uses: `def:weil-fresh-kflat-adapted-class-v1`, `def:weil-fresh-kflat-06ys-construction-v1`

For every $A \in \mathrm{Ch}(\mathcal{A}_Y)$ there is a termwise surjective quasi-isomorphism $p : P \to A$ with $P \in \mathsf{KF}_Y^{\mathrm{ad}}$.

Constructed by: `def:weil-fresh-kflat-06ys-construction-v1`

Proof. Uses: `def:weil-fresh-kflat-adapted-class-v1`, `def:weil-fresh-kflat-06ys-construction-v1`

Use the free flat epimorphisms on a ringed site (Stacks, Section 19.8, using Modules on Sites, Lemma 18.28.8) to form a direct system of bounded-above complexes P_m of flat modules mapping to the truncation of A through degree m. The transition maps are chosen so that the colimit map is termwise surjective and a quasi-isomorphism, as in Stacks, Lemma 21.17.11. Each P_m is K-flat by Lemma 21.17.8; filtered colimits of K-flat complexes are K-flat by Lemma 21.17.9. Exactness of filtered colimits preserves the quasi-isomorphism, and a direct sum of flat modules is flat, so $P := \mathrm{colim}_m P_m$ has the asserted properties. This is objectwise existence; it does not assert a functorial K-flat endofunctor. □

Lemma 10.15.14 (Pullback preserves adapted-source quasi-isomorphisms). *Lean:* `conditional-interface:lemma:weil-fresh-kflat-pullback-qis-v1`

Uses: `def:weil-fresh-kflat-adapted-class-v1`, `def:weil-fresh-ringed-topos-ambient-v1`

If $u : P \to P'$ is a quasi-isomorphism between objects of $\mathsf{KF}_Y^{\mathrm{ad}}$, then

$$f_{\mathrm{cw}}^* u : \mathcal{O}_X \otimes_{f^{-1}\mathcal{O}_Y} f^{-1}P \longrightarrow \mathcal{O}_X \otimes_{f^{-1}\mathcal{O}_Y} f^{-1}P'$$

is a quasi-isomorphism.

Proof. Uses: `def:weil-fresh-kflat-adapted-class-v1`, `def:weil-fresh-ringed-topos-ambient-v1`

Exactness of f^{-1} makes $f^{-1}u$ a quasi-isomorphism. Regard $f^{-1}P$ and $f^{-1}P'$ as complexes of $f^{-1}\mathcal{O}_Y$-modules. K-flatness is preserved by this exact inverse-image operation; this is the base-change lemma used in Stacks, Lemma 21.18.1. Apply Stacks, Lemma 21.17.12 to the quasi-isomorphism $f^{-1}u$ and the complex $\mathcal{O}_X$ concentrated in degree zero. The resulting total tensor map is exactly $f_{\mathrm{cw}}^* u$. □

Definition 10.15.15 (K-flat localization output for derived pullback). Lean: `conditional-interface:def:weil-fresh-lfstar-kflat-output-v1`

Uses: `def:weil-fresh-ringed-topos-ambient-v1`, `def:weil-fresh-kflat-adapted-class-v1`, `def:weil-general-unbounded-derived-localization-v1`

Define the derived-pullback output to be a localization factor

$$Lf^* : D_Y \longrightarrow D_X$$

together with, for every adapted arrow $p : P \to A$, an isomorphism

$$\lambda_p : Lf^*(Q_Y A) \xrightarrow{\sim} Q_X(f_{\mathrm{cw}}^* P),$$

subject to the localization-factor and naturality equations. Existence and uniqueness of this output are proved next from the adapted class.

Proposition 10.15.16 (Construction of Lf^* from the adapted K-flat class). *Lean:* `conditional-interface:prop:weil-fresh-lfstar-existence-06yy-v1`

Uses: `def:weil-fresh-kflat-adapted-class-v1`, `def:weil-general-unbounded-derived-localization-v1`, `def:weil-fresh-lfstar-kflat-output-v1`

There is a functor

$$Lf^* : D_Y \longrightarrow D_X$$

and, for every adapted arrow $p : P \to A$, an isomorphism

$$\lambda_p : Lf^*(Q_Y A) \xrightarrow{\sim} Q_X(f_{\mathrm{cw}}^* P), \tag{L1}$$

natural after passage to localization. If a functorial K-flat replacement package is supplied, (L1) agrees with the generic K-flat model definition; the present construction itself uses no K-projective replacement.

Constructed by: `def:weil-fresh-lfstar-kflat-output-v1`

Proof. Uses: def:weil-fresh-kflat-adapted-class-v1,prop:weil-fresh-kflat-existence-06ys-v1,lemma:weil-fresh-kflat-pullback-qis-v1,def:weil-general-unbounded-derived-localization-v1,def:weil-fresh-lfstar-kflat-output-v1

Define H on the full subcategory of adapted complexes by $H(P) = Q_X(f_{\mathrm{cw}}^* P)$. The preceding Lemma says that H sends adapted quasi-isomorphisms to isomorphisms. The existence Proposition says that every complex A receives an adapted quasi-isomorphism $P \to A$. The adapted-object localization lemma of Derived Categories, Lemma 13.14.15, therefore extends H uniquely to D_Y; this is precisely the construction in Stacks, Lemma 21.18.2. The same lemma gives (L1) and proves independence of the adapted arrow. The extension is defined on all $\mathbf{Z}$-indexed complexes. □

Lemma 10.15.17 (The underived adjunction is an adjunction of Hom complexes)**.** Lean: conditional-interface:lemma:weil-fresh-underived-hom-complex-adjunction-v1

Uses: def:weil-fresh-ringed-topos-ambient-v1

For $P \in \mathrm{Ch}(\mathcal{A}_Y)$ and $J \in \mathrm{Ch}(\mathcal{A}_X)$, the heart adjunction (A0) induces a natural isomorphism of cochain complexes

$$\Phi_{P,J} : \underline{\mathrm{Hom}}_X^\bullet(f_{\mathrm{cw}}^* P, J) \xrightarrow{\sim} \underline{\mathrm{Hom}}_Y^\bullet(P, f_{*,\mathrm{cw}} J). \tag{H}$$

Proof. Uses: def:weil-fresh-ringed-topos-ambient-v1,def:weil-general-unbounded-k-models-v1

For a homogeneous degree-r family h, define

$$(\Phi_{P,J}^r h)^n := \varphi_{P^n, J^{n+r}}(h^n).$$

The inverse applies φ^{-1} in every degree. Naturality of (A0) in the source and target gives

$$\Phi^{r+1}(d_J h - (-1)^r h\, d_{f^* P}) = d_{f_* J}\Phi^r(h) - (-1)^r \Phi^r(h) d_P,$$

so Φ commutes with the total differentials. The inverse calculation is identical. Passing to degree-zero cocycles modulo homotopies gives the natural Hom-set bijection in K_X and K_Y. □

Definition 10.15.18 (Adapted denominator categories)**.** Lean: conditional-interface:def:weil-fresh-adapted-roof-index-v1

Uses: def:weil-fresh-ringed-topos-ambient-v1,def:weil-fresh-kinjective-adapted-class-v1,def:weil-fresh-kflat-adapted-class-v1

Let $\overline{W}_X \subseteq \mathrm{Mor}(K_X)$ and $\overline{W}_Y \subseteq \mathrm{Mor}(K_Y)$ be the images of the quasi-isomorphisms. For $B \in K_X$, define $\mathsf{KIRes}_X(B)$ as the full subcategory of $B/\overline{W}_X$ on arrows

$$s : B \longrightarrow I, \qquad I \in \mathsf{KI}_X.$$

For $A \in K_Y$, define $\mathsf{KFRes}_Y(A)$ as the full subcategory of $\overline{W}_Y/A$ on arrows

$$t : P \longrightarrow A, \qquad P \in \mathsf{KF}_Y^{\mathrm{ad}}.$$

Morphisms are commuting triangles in the homotopy categories; two-out-of-three makes their comparison arrows quasi-isomorphisms. The target denominator is indexed by $\mathsf{KIRes}_X(B)$, while the source denominator is indexed by $\mathsf{KFRes}_Y(A)^{\mathrm{op}}$. The opposite category is part of the definition: $\mathrm{Hom}_{K_Y}(P, -)$ is contravariant in P.

Lemma 10.15.19 (Cofinality of the adapted replacement indices)**.** Lean: conditional-interface:lemma:weil-fresh-adapted-roof-cofinality-v1

Uses: def:weil-fresh-adapted-roof-index-v1

The inclusion

$$i_B : \mathsf{KIRes}_X(B) \longrightarrow B/\overline{W}_X$$

is final. The opposite of the inclusion

$$j_A : \mathsf{KFRes}_Y(A) \longrightarrow \overline{W}_Y/A$$

is final:

$$j_A^{\mathrm{op}} : \mathsf{KFRes}_Y(A)^{\mathrm{op}} \longrightarrow (\overline{W}_Y/A)^{\mathrm{op}}.$$

Thus restriction along these two functors preserves the two colimits used in the localization Hom formula.

Proof. Uses: def:weil-fresh-adapted-roof-index-v1,prop:weil-fresh-kflat-existence-06ys-v1,prop:weil-fresh-kinjective-existence-079p-v1

Fix $s : B \to J$ in $B/\overline{W}_X$. Compose it with the 079P quasi-isomorphism $J \to I_X(J)$. This gives an object of $(s \downarrow i_B)$, so that comma category is nonempty. Given two objects of $(s \downarrow i_B)$, the common-cocone form of the calculus of fractions for quasi-isomorphisms in a homotopy category gives quasi-isomorphisms from their two targets to a common target, with equal composites from J. Resolving that common target by 079P gives a

common object of $(s \downarrow i_B)$. The same construction applied to a parallel pair of comparison triangles equalizes it. Hence $(s \downarrow i_B)$ is connected, and the comma-category criterion proves that i_B is final.

Now view $t : C \to A$ as an object of $(\overline{W}_Y/A)^{\mathrm{op}}$. Choose a 06YS arrow $p : P(C) \to C$ with K-flat, termwise-flat source. The composite $t \circ p : P(C) \to A$ is adapted, and p is a morphism $j_A(t \circ p) \to t$ in $\overline{W}_Y/A$. Equivalently, $p^{\mathrm{op}} : t \to j_A^{\mathrm{op}}(t \circ p)$ is an object of $(t \downarrow j_A^{\mathrm{op}})$. Two such objects admit a common-source refinement by the calculus of fractions. Resolving that common source by 06YS gives an adapted common refinement, and the parallel-pair version gives connectedness. Therefore j_A^{op} is final. Mapping cylinders provide the two common-refinement constructions, and two-out-of-three keeps every comparison arrow in $\overline{W}$. K-flatness is used only to remain in the adapted source class, never as a substitute for K-projectivity. □

Lemma 10.15.20 (Adapted roof/colimit calculation). *Lean:* `conditional-interface:lemma:weil-fresh-adapted-roof-colimit-v1`

Uses: `def:weil-fresh-adapted-roof-index-v1`, `def:weil-fresh-rfstar-kinjective-v1`, `def:weil-fresh-lfstar-kflat-output-v1`

For $A \in D_Y$ and $B \in D_X$, choose complex representatives (still denoted A, B). The localization morphisms admit the bifunctorial calculation

$$\begin{aligned} \mathrm{Hom}_{D_Y}(A, Rf_*B) &\cong \varinjlim_{s \in \mathsf{KIRes}_X(B)} \varinjlim_{t \in \mathsf{KFRes}_Y(A)^{\mathrm{op}}} \mathrm{Hom}_{K_Y}(P, f_{*,\mathrm{cw}} I) \\ &\cong \varinjlim_{s,t} \mathrm{Hom}_{K_X}(f_{\mathrm{cw}}^* P, I) \\ &\cong \mathrm{Hom}_{D_X}(Lf^*A, B). \end{aligned} \tag{Roof}$$

Here $s : B \to I$ and $t : P \to A$ are the roof denominators. The first and last isomorphisms use the adapted cofinality, not a K-projective assertion.

Proof. Uses: `def:weil-fresh-adapted-roof-index-v1`, `def:weil-fresh-rfstar-kinjective-v1`, `prop:weil-fresh-rfstar-existence-v1`, `prop:weil-fresh-lfstar-existence-06yy-v1`, `lemma:weil-fresh-adapted-roof-cofinality-v1`, `lemma:weil-fresh-underived-hom-complex-adjunction-v1`

Apply Stacks, Lemma 13.30.1, with $\mathcal{D} = K_X$, $\mathcal{D}' = K_Y$, right adjoint $F = f_{*,\mathrm{cw}}$, left adjoint $G = f_{\mathrm{cw}}^*$, and multiplicative systems $\overline{W}_X, \overline{W}_Y$. Proposition 10.15.10 and Proposition 10.15.16 verify its two hypotheses: RF is defined at every B and LG is defined at every A.

The first two lines of the 0FND proof are the colimits over $s : B \to I$ and over the opposite of the category of $t : P \to A$. Lemma 10.15.19 replaces those indexing categories by $\mathsf{KIRes}_X(B)$ and $\mathsf{KFRes}_Y(A)^{\mathrm{op}}$. For each pair (s, t), the degree-zero part of the Hom-complex adjunction gives

$$\mathrm{Hom}_{K_Y}(P, f_{*,\mathrm{cw}} I) \cong \mathrm{Hom}_{K_X}(f_{\mathrm{cw}}^* P, I).$$

Its naturality in P and I makes these bijections a natural isomorphism of the two colimit diagrams. The last two lines of 0FND identify the resulting colimit with $\mathrm{Hom}_{D_X}(Lf^*A, B)$. The same theorem proves functoriality in both A and B. Thus denominator changes, identities, and composition are handled by the localization transition maps themselves; no K-flat-to-K-projective implication is used. □

Definition 10.15.21 (Derived Hom comparison from adapted roofs). *Lean:* `conditional-interface:def:weil-fresh-derived-hom-bijection-v1`

Uses: `def:weil-fresh-ringed-topos-ambient-v1`, `def:weil-fresh-rfstar-kinjective-v1`, `def:weil-fresh-lfstar-kflat-output-v1`, `def:weil-fresh-adapted-roof-index-v1`

For $A \in D_Y$ and $B \in D_X$, let $\Psi_{A,B}$ be the composite roof map

$$\mathrm{Hom}_{D_Y}(A, Rf_*B) \longrightarrow \varinjlim_{\substack{s \in \mathsf{KIRes}_X(B) \\ t \in \mathsf{KFRes}_Y(A)^{\mathrm{op}}}} \mathrm{Hom}_{K_Y}(P, f_{*,\mathrm{cw}} I) \longrightarrow \varinjlim_{s,t} \mathrm{Hom}_{K_X}(f_{\mathrm{cw}}^* P, I) \longrightarrow \mathrm{Hom}_{D_X}(Lf^*A, B),$$

where the middle arrow is the inverse of the degree-zero underived adjunction bijection,

$$(\Phi_{P,I})^{-1} : \mathrm{Hom}_{K_Y}(P, f_{*,\mathrm{cw}} I) \xrightarrow{\sim} \mathrm{Hom}_{K_X}(f_{\mathrm{cw}}^* P, I).$$

Define $\Phi_{A,B} := \Psi_{A,B}^{-1}$.

Constructed by: `lemma:weil-fresh-adapted-roof-colimit-v1`

Theorem 10.15.22 (General unbounded ringed-topos derived adjunction). *Lean:* `blueprint-target:thm:weil-fresh-ringed-topos-unbounded-adjunction-v1`

Uses: `def:weil-fresh-rfstar-kinjective-v1`, `def:weil-fresh-kflat-adapted-class-v1`, `def:weil-fresh-derived-hom-bijection-v1`

Under the ringed-topos data and the two Grothendieck certificates, the functors

$$Lf^* : D_Y \rightleftarrows D_X : Rf_*$$

are adjoint on the full unbounded derived categories. More precisely, there is a natural bijection

$$\Phi_{A,B} : \mathrm{Hom}_{D_X}(Lf^*A, B) \xrightarrow{\sim} \mathrm{Hom}_{D_Y}(A, Rf_*B) \tag{Adj}$$

for all $A \in D_Y$ and $B \in D_X$.

Constructed by: `def:weil-fresh-derived-hom-bijection-v1`

Proof. Uses: `lemma:weil-fresh-adapted-roof-colimit-v1`, `def:weil-fresh-derived-hom-bijection-v1`
The three-line calculation (Roof) proves that $\Psi_{A,B}$ is a bijection natural in both variables; hence its inverse $\Phi_{A,B}$ has the displayed type and the same bifunctoriality. This is exactly Stacks, Lemma 21.19.1, obtained from the derived-adjoint theorem after the everywhere-defined Rf_* and Lf^* constructions have been installed. □

Definition 10.15.23 (Unit and counit)**.** Lean: `conditional-interface:def:weil-fresh-ringed-topos-unit-counit-v1`

Uses: `def:weil-fresh-ringed-topos-ambient-v1`, `def:weil-fresh-derived-hom-bijection-v1`
For the bijection (Adj), define

$$\eta_A := \Phi_{A,Lf^*A}(\mathrm{id}_{Lf^*A}) : A \longrightarrow Rf_*Lf^*A,$$

and

$$\epsilon_B := \Phi^{-1}_{Rf_*B,B}(\mathrm{id}_{Rf_*B}) : Lf^*Rf_*B \longrightarrow B.$$

Lemma 10.15.24 (Unit/counit formulas)**.** Lean: `conditional-interface:lemma:weil-fresh-ringed-topos-unit-counit-formulas-v1`

Uses: `def:weil-fresh-ringed-topos-unit-counit-v1`
For $u : Lf^*A \to B$ and $v : A \to Rf_*B$ one has

$$\Phi_{A,B}(u) = Rf_*(u) \circ \eta_A, \qquad \Phi^{-1}_{A,B}(v) = \epsilon_B \circ Lf^*(v). \tag{UC}$$

Proof. Uses: `def:weil-fresh-ringed-topos-unit-counit-v1`
Naturality of (Adj) in the second variable sends the identity of Lf^*A to the first displayed composite. Naturality in the first variable, applied to the identity of Rf_*B, gives the second composite. The definitions of η and ϵ identify the remaining factors. □

Proposition 10.15.25 (Triangle identities)**.** Lean: `conditional-interface:prop:weil-fresh-ringed-topos-triangles-v1`

Uses: `def:weil-fresh-ringed-topos-unit-counit-v1`
For every $A \in D_Y$ and $B \in D_X$,

$$\epsilon_{Lf^*A} \circ Lf^*(\eta_A) = \mathrm{id}_{Lf^*A}, \qquad Rf_*(\epsilon_B) \circ \eta_{Rf_*B} = \mathrm{id}_{Rf_*B}\,. \tag{Tri}$$

Proof. Uses: `def:weil-fresh-ringed-topos-unit-counit-v1`, `lemma:weil-fresh-ringed-topos-unit-counit-formulas-v1`
By the second formula in (UC), applied to $v = \eta_A$,

$$\epsilon_{Lf^*A} \circ Lf^*(\eta_A) = \Phi^{-1}_{A,Lf^*A}(\eta_A) = \Phi^{-1}_{A,Lf^*A}\big(\Phi_{A,Lf^*A}(\mathrm{id}_{Lf^*A})\big) = \mathrm{id}_{Lf^*A}\,.$$

By the first formula in (UC), applied to $u = \epsilon_B$,

$$Rf_*(\epsilon_B) \circ \eta_{Rf_*B} = \Phi_{Rf_*B,B}(\epsilon_B) = \Phi_{Rf_*B,B}\big(\Phi^{-1}_{Rf_*B,B}(\mathrm{id}_{Rf_*B})\big) = \mathrm{id}_{Rf_*B}\,.$$

□

Definition 10.15.26 (Same-coefficient exactness certificate)**.** Lean: `conditional-interface:def:weil-fresh-same-coefficient-exactness-v1`

Uses: `def:weil-fresh-ringed-topos-ambient-v1`
A same-coefficient exactness certificate is an isomorphism $f^{-1}\mathcal{O}_Y \simeq \mathcal{O}_X$ under which the module pullback f^* identifies with the exact inverse-image functor f^{-1}.

Definition 10.15.27 (Same-coefficient derived-pullback comparison). Lean: `conditional-interface:def:weil-fresh-same-coefficient-comparison-v1`

Uses: `def:weil-fresh-same-coefficient-exactness-v1`, `def:weil-fresh-lfstar-kflat-output-v1`

Define the comparison

$$\chi_f : Lf^* \circ Q_Y \Longrightarrow Q_X \circ f^*_{\mathrm{cw}}$$

as the unique localization-factor map induced by the exact termwise pullback and the K-flat localization output.

Proposition 10.15.28 (Exact same-coefficient simplification). Lean: *conditional-interface:prop:weil-fresh-same-coefficient-exact-simplification-v1*

Uses: *def:weil-fresh-same-coefficient-exactness-v1*, *def:weil-fresh-same-coefficient-comparison-v1*

Under a same-coefficient exactness certificate, there is a natural isomorphism

$$Lf^* \circ Q_Y \xrightarrow{\sim} Q_X \circ f^*_{\mathrm{cw}}. \tag{Exact-L}$$

Thus Lf^* may be computed termwise on every unbounded complex; no K-projective or K-flat replacement is needed for this left functor. The right functor still uses K-injectives unless f_* is separately exact.

Constructed by: *def:weil-fresh-same-coefficient-comparison-v1*

Proof. Uses: `def:weil-fresh-same-coefficient-exactness-v1`, `def:weil-fresh-same-coefficient-comparison-v1`, `prop:weil-fresh-lfstar-existence-06yy-v1`

The exact functor f^*_{cw} sends cones of quasi-isomorphisms to acyclic cones, hence sends every quasi-isomorphism to a quasi-isomorphism. The localization universal property gives the termwise factorization (Exact-L). Its comparison with the K-flat construction is the uniqueness clause in Proposition 10.15.16. Nothing in this argument produces K-projectives. □

Lemma 10.15.29 (Exact pullback makes direct-image K-injectives visible). Lean: *conditional-interface:lemma:weil-fresh-same-coefficient-rfstar-kinjective-v1*

Uses: *def:weil-fresh-same-coefficient-exactness-v1*, *def:weil-fresh-kinjective-adapted-class-v1*

If $J \in \mathsf{KI}_X$ and f^* is exact, then $f_{*,\mathrm{cw}} J \in \mathsf{KI}_Y$.

Proof. Uses: `def:weil-fresh-same-coefficient-exactness-v1`, `def:weil-fresh-kinjective-adapted-class-v1`, `lemma:weil-fresh-underived-hom-complex-adjunction-v1`

For an acyclic $C \in \mathrm{Ch}(\mathcal{A}_Y)$, exactness makes $f^*_{\mathrm{cw}} C$ acyclic. The Hom-complex isomorphism (H) gives

$$\underline{\mathrm{Hom}}^\bullet_Y(C, f_{*,\mathrm{cw}} J) \cong \underline{\mathrm{Hom}}^\bullet_X(f^*_{\mathrm{cw}} C, J),$$

whose right side is acyclic because J is K-injective. This is the K-injective condition for $f_{*,\mathrm{cw}} J$. □

10.15.1 Weil coefficient specializations

Definition 10.15.30 (Weil finite and integral ambient coefficient modes). Lean: `conditional-interface:def:weil-fresh-weil-coefficient-modes-v1`

Uses: `def:weil-ftdc-small-etale-site`, `def:weil-ftdc-small-proetale-site`, `def:weil-ftdc-coefficient-sheaf`, `def:weil-a2-integral-ring-sheaf`, `def:weil-a2-torsion-ring-sheaf`, `def:weil-a2-module-category`

For $\tau \in \{\mathrm{et}, \mathrm{proet}\}$ and $n \geq 1$, write

$$\Lambda_{X,\tau,n} := \underline{\mathbf{Z}/\ell^n}_{X_\tau}, \qquad \mathcal{A}_{X,\tau,n} := \mathrm{Mod}(X_\tau, \Lambda_{X,\tau,n}).$$

For $\tau = \mathrm{proet}$, the typed coefficient comparison identifies $\Lambda_{X,\mathrm{proet},n}$ with the A2 torsion ring sheaf $\Lambda_{n,X}$; the topology-indexed notation prevents using that A2 symbol for the distinct etale-site object. For the ordinary integral pro-etale coefficient write

$$\mathcal{A}_{X,\mathrm{proet},\infty} := \mathrm{Mod}(X_{\mathrm{proet}}, \Lambda_X),$$

where Λ_X is the continuous $\mathbf{Z}_\ell$-valued ring sheaf. These are ambient module-sheaf categories; the integral notation does not mean the derived-complete or normalized coefficient-system category.

Proposition 10.15.31 (Finite-level unbounded Weil adjunction). Lean: *conditional-interface:prop:weil-fresh-finite-level-unbounded-adjunction-v1*

Uses: *def:weil-fresh-weil-coefficient-modes-v1*, *def:weil-fresh-rfstar-kinjective-v1*, *def:weil-fresh-same-coefficient-exactness-v1*

Let $f : X \to Y$ be a morphism of schemes, $\tau \in \{\mathrm{et}, \mathrm{proet}\}$, and $n \geq 1$. The ambient all-integer categories have an adjunction

$$Lf^* : D(\mathcal{A}_{Y,\tau,n}) \rightleftarrows D(\mathcal{A}_{X,\tau,n}) : Rf_*.$$

Here Lf^* is termwise inverse image and Rf_* is computed by the functorial K-injective replacement; the unit and counit obey the triangle identities.

Proof. Uses: `lemma:weil-ftdc-pullback-exact,prop:weil-fresh-ringed-topos-module-grothendieck-v1,prop:weil-fresh-kinjective-existence-079p-v1,prop:weil-fresh-kflat-existence-06ys-v1,prop:weil-fresh-rfstar-existence-v1,prop:weil-fresh-lfstar-existence-06yy-v1,thm:weil-fresh-ringed-topos-unbounded-adjunction-v1,prop:weil-fresh-ringed-topos-triangles-v1,prop:weil-fresh-same-coefficient-exact-simplification-v1`

The finite coefficient ring sheaf pulls back isomorphically, so the existing exact inverse-image lemma supplies the same-coefficient certificate. The module-sheaf category is Grothendieck, hence 079P gives a functorial all-integer K-injective replacement. The 06YS construction gives the termwise-flat K-flat adapted sources used by the localization construction of Lf^*. The exact simplification identifies that construction with termwise inverse image. The ringed-topos Hom calculation and its triangle proposition then give the displayed adjunction. □

Proposition 10.15.32 (Ordinary integral pro-etale unbounded Weil adjunction)**.** Lean: `conditional-interface:prop:weil-fresh-integral-unbounded-adjunction-v1`

Uses: `def:weil-fresh-weil-coefficient-modes-v1,def:weil-fresh-rfstar-kinjective-v1,def:weil-fresh-same-coefficient-exactness-v1`

Let $f : X \to Y$ be a morphism of schemes. Assume a typed coefficient comparison $f^{-1}\Lambda_Y \xrightarrow{\sim} \Lambda_X$ for the ordinary continuous $\mathbf{Z}_\ell$-valued ring sheaves. Then

$$Lf^* : D(\mathcal{A}_{Y,\mathrm{proet},\infty}) \rightleftarrows D(\mathcal{A}_{X,\mathrm{proet},\infty}) : Rf_*$$

is an adjunction on the full unbounded ambient categories, with Lf^* computed termwise and Rf_* computed on K-injectives.

Proof. Uses: `prop:weil-a2-underlying-additive-comparison,prop:weil-fresh-ringed-topos-module-grothendieck-v1,prop:weil-fresh-kinjective-existence-079p-v1,prop:weil-fresh-kflat-existence-06ys-v1,prop:weil-fresh-rfstar-existence-v1,prop:weil-fresh-lfstar-existence-06yy-v1,thm:weil-fresh-ringed-topos-unbounded-adjunction-v1,prop:weil-fresh-same-coefficient-exact-simplification-v1`

The ring-valued continuous sheaf supplies the module category and its Grothendieck certificate. Apply 079P and 06YS in that ambient category. The assumed coefficient comparison makes scalar extension the exact inverse image, so the general roof/colimit adjunction specializes as stated. This does not assert that the derived-complete subcategory is preserved. □

Proposition 10.15.33 (Ambient-first boundary for later restrictions)**.** Lean: `conditional-interface:prop:weil-fresh-ambient-first-restriction-boundary-v1`

Uses: `def:weil-fresh-weil-coefficient-modes-v1`

The K-injective and K-flat replacements above are constructed in the full ambient module-sheaf categories. Restriction to derived-complete, compatible coefficient-tower, D^+, or D^b_c subcategories is a separate stability statement; in particular, independently chosen levelwise replacements are not identified with a compatible tower.

Proof. Uses: `def:weil-fresh-weil-coefficient-modes-v1`

The unbounded replacement constructions use arbitrary coproducts, filtered colimits, and injective or flat terms, so they need not preserve boundedness, constructibility, or derived completeness. The ambient localization is therefore the domain of the replacement theorem; amplitude, completion, and tower comparison are later certificates. □

Proposition 10.15.34 (Lean localization adapter remains OPEN)**.** Lean: `conditional-interface:prop:weil-fresh-ringed-topos-lean-adapter-open-v1`

Uses: `def:weil-fresh-ringed-topos-ambient-v1,def:weil-fresh-kinjective-adapted-class-v1,def:weil-fresh-kflat-adapted-class-v1`

OPEN. *To consume this mathematical module in Lean, the project must instantiate the universe-small ringed-site module categories, export the external 079P K-injective and 06YS K-flat replacement data, and identify the concrete adapted roof/colimit map with the pinned all-integer derived category. The pinned probe already supplies the generic all-$\mathbb{Z}$ Q/Q_h localization API and the exact-functor 'mapDerivedCategory' factorization; it does not yet supply the concrete sheaf-category export or the roof cofinality term. No K-projective replacement is requested and the mathematical theorem is unchanged.*

Open obligation: `OBL-FRESH-RINGED-TOPOS-LOCALIZATION-ADAPTER`

Proof. Uses: def:weil-fresh-ringed-topos-ambient-v1,def:weil-fresh-kinjective-adapted-class-v1,def:weil-fresh-kflat-adapted-class-v1

The displayed mathematical construction uses the ordinary localization universal property and the roof calculation of Stacks Lemma 13.30.1. The remaining Lean work is precisely the concrete coefficient/universe instantiation, the export of the external replacement constructions, and the natural roof comparison. Recording this narrower bridge leaves the mathematical result available without silently treating an interface as a proof. □

Remark 10.15.35 (Source ledger and insertion point). The K-injective existence input is Stacks 19.12.6; the adapted K-flat resolution input is Stacks 21.17.11, with tensor invariance from 21.17.12. The derived pullback construction is 21.18.2. The roof calculation is Derived Categories, Lemma 13.30.1, and the formal derived-adjoint conclusion is 21.19.1. The fresh module is intended to be inserted immediately after Definition 10.14.5 and its conditional construction lemmas in the frozen general-unbounded source, and before the D^+/D^b_c restriction modules. The central source and its SHA-frozen baseline are not modified by this file.

10.15.2 Selected unbounded replacement discharges

Theorem 10.15.36 (Objectwise K-injective replacement for Weil coefficient categories). Lean: blueprint-target: thm:weil-general-unbounded-kinjective-objectwise-discharge-v1

Uses: def:weil-general-unbounded-objectwise-replacement-v1, def:weil-fresh-kinjective-079p-construction-v1, def:weil-fresh-weil-coefficient-modes-v1

For every all-integer complex M in a finite etale, finite pro-etale, or ordinary integral pro-etale module-sheaf category fixed in Definition 10.15.30, there is a datum

$$\mathrm{KIRep}(M) = (I_M, M \xrightarrow{\iota_M} I_M, \kappa_M)$$

in which ι_M is a quasi-isomorphism and κ_M says that I_M is K-injective against every acyclic all-integer complex.

Constructed by: def:weil-fresh-kinjective-079p-construction-v1

Proof. Uses: prop:weil-fresh-ringed-topos-module-grothendieck-v1,prop:weil-fresh-kinjective-existence-079p-v1,prop:weil-general-unbounded-objectwise-from-package-v1

The module-sheaf Grothendieck proposition supplies the required Grothendieck certificate. Proposition 10.15.7 then gives a functor I, a natural quasi-isomorphism $\iota : \mathrm{id} \Rightarrow I$, and the assertion that every $I(M)$ is K-injective. Evaluate at M and apply Proposition 10.14.7. This gives the displayed objectwise datum. By the fixed definition of K-injectivity, its final field is precisely the acyclicity of $\underline{\mathrm{Hom}}^\bullet(A, I(M))$ for every acyclic all-integer complex A. □

Theorem 10.15.37 (Objectwise K-flat replacement for Weil coefficient categories). Lean: blueprint-target: thm:weil-general-unbounded-kflat-objectwise-discharge-v1

Uses: def:weil-general-unbounded-objectwise-replacement-v1, def:weil-fresh-kflat-06ys-construction-v1, def:weil-fresh-weil-coefficient-modes-v1

For every all-integer complex A in the selected ringed-site coefficient categories there is a datum

$$\mathrm{KFRep}(A) = (P^\flat_A, P^\flat_A \xrightarrow{p_A} A, \kappa^\flat_A)$$

such that p_A is a termwise-surjective quasi-isomorphism, every term of $P^\flat_A$ is flat, and $\kappa^\flat_A$ proves K-flatness.

Constructed by: def:weil-fresh-kflat-06ys-construction-v1

Proof. Uses: prop:weil-fresh-kflat-existence-06ys-v1

Apply Proposition 10.15.13 to A. Its construction is the 077J truncation diagram: the transition maps are termwise split injections with flat direct-sum cokernels. The colimit is termwise flat for this reason, is K-flat by the filtered-colimit theorem 06YR, and maps termwise surjectively and quasi-isomorphically to A by 06YS. These are exactly the three fields of the displayed objectwise datum. □

Theorem 10.15.38 (K-flat localization alternative for derived pullback). Lean: blueprint-target: thm:weil-general-unbounded-kflat-existence-discharge-v1

Uses: def:weil-fresh-kflat-adapted-class-v1, def:weil-fresh-lfstar-kflat-output-v1

For pullback and scalar extension on the selected ringed sites, the adapted K-flat sources determine an everywhere-defined, choice-independent functor

$$Lf^* : D(\mathcal{A}_Y) \longrightarrow D(\mathcal{A}_X)$$

with the localization-factor and naturality equations. This is the localization-level alternative allowed by the K-flat package obligation; no strict K-flat endofunctor on the complex category is asserted.

Constructed by: `def:weil-fresh-lfstar-kflat-output-v1`

Proof. Uses: `prop:weil-fresh-kflat-existence-06ys-v1,lemma:weil-fresh-kflat-pullback-qis-v1,prop:weil-fresh-lfstar-existence-06yy-v1`

The 06YS proposition gives an adapted arrow $P \to A$ for every complex. Pullback sends quasi-isomorphisms between adapted sources to quasi-isomorphisms. Therefore the adapted-object derived-functor theorem, applied in Proposition 10.15.16, constructs Lf^* on the entire localization and gives, for every adapted arrow, the canonical comparison

$$Lf^*(Q_Y A) \xrightarrow{\sim} Q_X(f^*_{\mathrm{cw}} P).$$

The same theorem supplies independence of the adapted arrow, naturality, identity, composition, and uniqueness of the localization factor. Thus the second discharge alternative is satisfied without a strict functorial K-flat replacement on complexes. □

10.16 Derived inverse limits and completion

Definition 10.16.1 (Reduction tower). Lean: `conditional-interface:def:weil-a2-reduction-tower`

Uses: `def:weil-a2-finite-reduction,def:weil-a2-reduction-transition`

Define $\mathcal{R}(K)$ to be

$$\cdots \longrightarrow \mathrm{Red}_3(K) \longrightarrow \mathrm{Red}_2(K) \longrightarrow \mathrm{Red}_1(K).$$

Definition 10.16.2 (K-injective tower package). Lean: `conditional-interface:def:weil-a2-kinjective-tower-package`

Uses: `def:weil-a2-reduction-tower,def:weil-a2-complex-category`

A K-injective tower package consists of K-injective complexes $I_n(K)$, quasi-isomorphisms $\mathrm{Red}_n(K) \to Q_X(I_n(K))$, and cochain transition maps $t_{n+1,n} : I_{n+1}(K) \to I_n(K)$ representing $r_{n+1,n}(K)$, together with the tower and naturality equations.

Proposition 10.16.3 (K-injective tower package existence). Lean: `conditional-interface:prop:weil-a2-kinjective-tower-package-existence-open`

Uses: `def:weil-a2-kinjective-tower-package`

Every reduction tower admits a functorial K-injective tower package.

Proof. Uses: `thm:weil-a2-module-category-abelian`

OPEN obligation A2-KINJECTIVE-TOWER. The exact proof must derive enough injectives from the Grothendieck property, construct functorial unbounded Spaltenstein replacements, prove K-injectivity and quasi-isomorphism, lift all transition maps, and strictify the tower and naturality homotopies. It is not a foundation leaf. □

Definition 10.16.4 (Fixed K-injective tower package). Lean: `conditional-interface:def:weil-a2-fixed-kinjective-tower-package`

Uses: `def:weil-a2-kinjective-tower-package`

Fix an explicit K-injective tower package ι_K as input to the following homotopy-limit construction.

Definition 10.16.5 (Product shift map). Lean: `conditional-interface:def:weil-a2-tower-product-shift`

Uses: `def:weil-a2-fixed-kinjective-tower-package`

Let $I(K) := \prod_{n \geq 1} I_n(K)$. Define

$$s_K : I(K) \longrightarrow I(K), \qquad s_K((x_n)_n) := (t_{n+1,n}(x_{n+1}))_n.$$

Lemma 10.16.6 (The product shift is a cochain map). Lean: `conditional-interface:lemma:weil-a2-tower-product-shift-chain-map`

Uses: `def:weil-a2-tower-product-shift`

The map s_K commutes with the product differential.

Proof. Uses: `def:weil-a2-tower-product-shift`

Its n-th component is the cochain map $t_{n+1,n}$. Equality of maps into a product is componentwise. □

Definition 10.16.7 (Explicit derived inverse limit)**.** Lean: `conditional-interface:def:weil-a2-derived-inverse-limit`

Uses: `def:weil-a2-tower-product-shift`, `def:weil-a2-derived-category`

Define

$$\mathbf{R}\varprojlim_{n} \mathrm{Red}_n(K) := Q_X\big(\mathrm{Cone}(1 - s_K)[-1]\big).$$

Proposition 10.16.8 (Derived-limit package independence)**.** Lean: `conditional-interface:prop:weil-a2-derived-limit-package-independent-open`

Uses: `def:weil-a2-derived-inverse-limit`

Two K-injective tower packages give canonically isomorphic derived-limit objects, naturally in the tower.

Proof. Uses: `lemma:weil-a2-finite-reduction-package-independent`

OPEN obligation A2-RLIM-INDEPENDENCE. Construct a common K-injective refinement of the two strict towers, prove that the induced map on countable products is a quasi-isomorphism, compare the two $1 - s$ maps, and apply the cone criterion. Product preservation of these quasi-isomorphisms is the unresolved leaf. □

Definition 10.16.9 (Derived completion)**.** Lean: `conditional-interface:def:weil-a2-derived-completion`

Uses: `def:weil-a2-derived-inverse-limit`

Define

$$\widehat{K} := \mathbf{R}\varprojlim_{n\geq 1} \mathrm{Red}_n(K).$$

Definition 10.16.10 (Completion unit)**.** Lean: `conditional-interface:def:weil-a2-completion-unit`

Uses: `def:weil-a2-derived-completion`, `def:weil-a2-reduction-tower`

The quotient map to L_n gives compatible arrows $K \to \mathrm{Red}_n(K)$. Define $\eta_K : K \to \widehat{K}$ as the induced arrow to the explicit $\mathrm{Cone}(1 - s_K)[-1]$ model.

Lemma 10.16.11 (Completion unit naturality)**.** Lean: `conditional-interface:lemma:weil-a2-completion-unit-natural`

Uses: `def:weil-a2-completion-unit`

For $f : K \to K'$, one has $\widehat{f}\,\eta_K = \eta_{K'} f$.

Proof. Uses: `lemma:weil-a2-reduction-transition-natural`

At every finite level the two maps are reduction of f after the same quotient arrow. Hence the maps to the tower product agree componentwise and induce the same map to the shifted cone. □

Definition 10.16.12 (Derived-complete object)**.** Lean: `conditional-interface:def:weil-a2-derived-complete-object`

Uses: `def:weil-a2-derived-completion`, `def:weil-a2-completion-unit`

An object K is derived complete if η_K is an isomorphism.

Definition 10.16.13 (Complete derived category)**.** Lean: `conditional-interface:def:weil-a2-complete-derived-category`

Uses: `def:weil-a2-derived-category`, `def:weil-a2-derived-complete-object`

Let $\mathcal{D}_{\mathrm{comp}}(X, \Lambda)$ be the full subcategory of derived-complete objects.

Proposition 10.16.14 (Completion idempotence)**.** Lean: `conditional-interface:prop:weil-a2-completion-idempotent-open`

Uses: `def:weil-a2-derived-completion`, `def:weil-a2-completion-unit`

The map $\eta_{\widehat{K}} : \widehat{K} \to \widehat{\widehat{K}}$ is an isomorphism.

Proof. Uses: `prop:weil-a2-derived-limit-package-independent-open`

OPEN obligation A2-COMPLETION-IDEMPOTENCE. First prove $\mathrm{Red}_n(\widehat{K}) \simeq \mathrm{Red}_n(K)$ by commuting finite reduction with the explicit homotopy limit; next prove compatibility with all transition maps; finally apply the same $\mathbf{R}\varprojlim$ cone model. The derived-limit independence obligation must be closed first. □

10.17 Lisse and constructible complete complexes

Definition 10.17.1 (Completed tensor product). Lean: `conditional-interface:def:weil-a2-completed-tensor`

Uses: `def:weil-a2-derived-completion`, `def:weil-a2-fixed-kflat-package`, `def:weil-a2-total-tensor-complex`

For complete objects K, L, define

$$K \widehat{\otimes}^{\mathbf{L}}_{\Lambda_X} L := Q_X(\mathfrak{P}(\widehat{K \otimes_{\Lambda_X}} \mathfrak{P}(L))\,.$$

Definition 10.17.2 (Completed tensor unit). Lean: `conditional-interface:def:weil-a2-completed-tensor-unit`

Uses: `def:weil-a2-derived-completion`, `def:weil-a2-integral-ring-sheaf`

Define the completed unit by

$$\mathbf{1}_X := \widehat{\Lambda_X[0]}.$$

Proposition 10.17.3 (Completed monoidal coherence). Lean: *`conditional-interface:prop:weil-a2-completed-monoidal-coherence-open`*

Uses: *`def:weil-a2-completed-tensor`*, *`def:weil-a2-completed-tensor-unit`*

The completed tensor product has natural associator, left and right unitors, and symmetry satisfying the pentagon, triangle, and hexagon equations.

Proof. Uses: `prop:weil-a2-completion-idempotent-open`, `prop:weil-a2-sheaf-tensor-universal`

OPEN obligation A2-COMPLETED-MONOIDAL. Construct the comparison $\widehat{K \otimes L} \widehat{\otimes} M \to K \widehat{\otimes} \widehat{L \otimes} M$, prove it is an isomorphism by all finite reductions, and transport the ordinary tensor associator. Repeat for the two unitors and symmetry; then verify each coherence diagram after finite reduction and use conservativity of reductions on complete objects. Both conservativity and completion idempotence remain prior obligations. □

Definition 10.17.4 (Duality datum for a complete complex). Lean: `conditional-interface:def:weil-a2-duality-datum`

Uses: `def:weil-a2-completed-tensor`, `def:weil-a2-completed-tensor-unit`

A duality datum for $K \in \mathcal{D}_{\mathrm{comp}}(X, \Lambda)$ consists of an object $K^\vee$ and arrows

$$\mathrm{coev} : \mathbf{1}_X \to K \widehat{\otimes} K^\vee, \qquad \mathrm{ev} : K^\vee \widehat{\otimes} K \to \mathbf{1}_X$$

such that the two standard triangle composites are identities.

Definition 10.17.5 (Lisse complete complex). Lean: `conditional-interface:def:weil-a2-lisse-complex`

Uses: `def:weil-a2-complete-derived-category`, `def:weil-a2-duality-datum`

A complete complex K is lisse if its class of duality data is nonempty.

Lemma 10.17.6 (A selected duality datum gives the first triangle). Lean: *`conditional-interface:lemma:weil-a2-lisse-first-triangle`*

Uses: *`def:weil-a2-duality-datum`*

For a duality datum, the composite

$$K \longrightarrow K \widehat{\otimes} K^\vee \widehat{\otimes} K \longrightarrow K$$

is id_K.

Proof. Uses: `def:weil-a2-duality-datum`

This is the first retained triangle equation in the datum. □

Lemma 10.17.7 (A selected duality datum gives the second triangle). Lean: *`conditional-interface:lemma:weil-a2-lisse-second-triangle`*

Uses: *`def:weil-a2-duality-datum`*

For a duality datum, the analogous composite on $K^\vee$ *is* $\mathrm{id}_{K^\vee}$.

Proof. Uses: `def:weil-a2-duality-datum`

This is the second retained triangle equation. □

Definition 10.17.8 (Pro-etale pullback package for a locally closed immersion)**.** Lean: `conditional-interface:def:weil-a2-locally-closed-pullback-package`

Uses: `def:weil-a2-integral-ring-sheaf`, `def:weil-a2-derived-category`

For a locally closed immersion $i : Z \hookrightarrow X$, a pullback package consists of:

$$i_{\mathrm{proet}} : Z_{\mathrm{proet}} \to X_{\mathrm{proet}}, \qquad i^{-1}\Lambda_X \to \Lambda_Z, \qquad i^* : \mathcal{D}(X, \Lambda) \to \mathcal{D}(Z, \Lambda),$$

together with the identity, composition, tensor, finite-reduction, and completion comparison isomorphisms and their cocycle equations.

Proposition 10.17.9 (Locally closed pullback package existence)**.** Lean: *`conditional-interface:prop:weil-a2-locally-closed-pullback-package-open`*

Uses: *`def:weil-a2-locally-closed-pullback-package`*

Every locally closed immersion admits a pullback package.

Proof. Uses: `prop:weil-a2-underlying-additive-comparison`, `prop:weil-a2-finite-reduction-functor`

OPEN obligation A2-LOCALLY-CLOSED-PULLBACK. The proof must define the base-change functor $V \mapsto V \times_X Z$ on pro-etale objects, prove it is continuous, construct inverse image by sheafified left Kan extension, construct the coefficient-ring morphism, define module pullback by scalar extension, derive it using K-flat models, and prove separately the identity, composition, tensor, reduction, and completion compatibilities. This is not replaced by the later proper or smooth base-change theorem. □

Definition 10.17.10 (Fixed locally closed pullback packages)**.** Lean: `conditional-interface:def:weil-a2-fixed-locally-closed-pullbacks`

Uses: `def:weil-a2-locally-closed-pullback-package`

Fix an explicit pullback package i^* for every locally closed immersion used in a stratification.

Definition 10.17.11 (Finite constructible stratification)**.** Lean: `conditional-interface:def:weil-a2-finite-stratification`

Uses: `def:weil-a2-fixed-locally-closed-pullbacks`

A finite constructible stratification of X is a finite family of locally closed immersions $i_a : X_a \hookrightarrow X$ such that the images are pairwise disjoint and their union is X. Retain the disjointness and covering equalities as fields.

Definition 10.17.12 (Constructible-stratification witness)**.** Lean: `conditional-interface:def:weil-a2-constructible-stratification-witness`

Uses: `def:weil-a2-finite-stratification`, `def:weil-a2-lisse-complex`

For $K \in \mathcal{D}_{\mathrm{comp}}(X, \Lambda)$, a constructible-stratification witness is the explicit pair of:

1. a finite constructible stratification $(i_a : X_a \hookrightarrow X)_a$;
2. for every index a, a selected duality datum exhibiting $i_a^* K$ as lisse.

Definition 10.17.13 (Constructible complete complex)**.** Lean: `conditional-interface:def:weil-a2-constructible-complex`

Uses: `def:weil-a2-constructible-stratification-witness`

A complete complex is constructible if the type of constructible-stratification witnesses is nonempty.

Lemma 10.17.14 (A constructibility witness has a finite stratification)**.** Lean: *`conditional-interface:lemma:weil-a2-constructibility-witness-stratification`*

Uses: *`def:weil-a2-constructible-stratification-witness`*

The first projection of a constructible-stratification witness is a finite constructible stratification of X.

Proof. Uses: `def:weil-a2-constructible-stratification-witness`

This is the first field of the witness. □

Lemma 10.17.15 (A constructibility witness is lisse on each stratum)**.** Lean: *`conditional-interface:lemma:weil-a2-constructibility-witness-lisse`*

Uses: *`def:weil-a2-constructible-stratification-witness`*

For every stratum $i_a : X_a \hookrightarrow X$ in a constructible-stratification witness, the restriction $i_a^* K$ is lisse.

Proof. Uses: `def:weil-a2-constructible-stratification-witness`, `def:weil-a2-lisse-complex`

The second field supplies a duality datum for $i_a^* K$. By definition, nonemptiness of the type of duality data is precisely lissity. □

Definition 10.17.16 (Common refinement of two stratifications). Lean: `conditional-interface:def:weil-a2-stratification-common-refinement`

Uses: `def:weil-a2-finite-stratification`

For stratifications $(X_a)_a$ and $(Y_b)_b$, define their common refinement to consist of the nonempty locally closed intersections

$$X_a \times_X Y_b \hookrightarrow X.$$

Lemma 10.17.17 (The intersection family is disjoint). Lean: `conditional-interface:lemma:weil-a2-stratification-refinement-disjoint`

Uses: `def:weil-a2-stratification-common-refinement`

Two distinct nonempty intersections in the common refinement have disjoint images.

Proof. Uses: `def:weil-a2-stratification-common-refinement`

If $(a,b) \neq (a',b')$, then either $a \neq a'$ or $b \neq b'$. The corresponding original strata are disjoint, so their intersections are disjoint. □

Lemma 10.17.18 (The intersection family covers). Lean: `conditional-interface:lemma:weil-a2-stratification-refinement-covers`

Uses: `def:weil-a2-stratification-common-refinement`

The common refinement covers X.

Proof. Uses: `def:weil-a2-stratification-common-refinement`

For $x \in X$, choose its unique stratum X_a in the first partition and its unique stratum Y_b in the second. Then $x \in X_a \cap Y_b$. □

Proposition 10.17.19 (Common refinement is a stratification). Lean: `conditional-interface:prop:weil-a2-stratification-common-refinement`

Uses: `def:weil-a2-stratification-common-refinement`

The nonempty intersections form a finite constructible stratification.

Proof. Uses: `lemma:weil-a2-stratification-refinement-disjoint,lemma:weil-a2-stratification-refinement-covers`

Finiteness is the finiteness of the product index set. Intersections of locally closed subsets are locally closed. The cited lemmas give disjointness and covering. □

Definition 10.17.20 (Bounded derived object). Lean: `conditional-interface:def:weil-a2-bounded-object`

Uses: `def:weil-a2-derived-category`

An object K is bounded if there exist integers $a \leq b$ such that $H^i(K) = 0$ for all $i < a$ and all $i > b$.

Definition 10.17.21 (Integral constructible bounded category). Lean: `conditional-interface:def:weil-a2-dbc-zl`

Uses: `def:weil-a2-complete-derived-category,def:weil-a2-constructible-complex,def:weil-a2-bounded-object`

Define

$$D^b_c(X_{\mathrm{proet}}, \mathbf{Z}_\ell)$$

to be the full subcategory of $\mathcal{D}_{\mathrm{comp}}(X,\Lambda)$ whose objects are bounded and constructible.

Definition 10.17.22 (Finite-torsion constructibility witness). Lean: `conditional-interface:def:weil-a2-torsion-constructibility-witness`

Uses: `def:weil-a2-finite-stratification,def:weil-a2-torsion-derived-category`

For $K \in \mathcal{D}(X,\Lambda_n)$, a finite-torsion constructibility witness consists of a finite stratification $(i_a : X_a \hookrightarrow X)_a$, a finite interval $[r,s]$, vanishing $H^j(K) = 0$ outside that interval, and for every a, j a finite locally constant Λ_n-module sheaf identified with $H^j(i_a^* K)$.

Definition 10.17.23 (Finite-torsion constructible bounded category). Lean: `conditional-interface:def:weil-a2-dbc-torsion`

Uses: `def:weil-a2-torsion-derived-category,def:weil-a2-torsion-constructibility-witness`

Define

$$D^b_c(X_{\mathrm{proet}}, \mathbf{Z}/\ell^n)$$

to be the full subcategory of objects admitting a finite-torsion constructibility witness.

Lemma 10.17.24 (A torsion constructibility witness is bounded). Lean: `conditional-interface:lemma:weil-a2-torsion-witness-bounded`

Uses: `def:weil-a2-torsion-constructibility-witness`

If $[r,s]$ is the interval in a finite-torsion constructibility witness, then $H^j(K)=0$ for $j<r$ and for $j>s$.

Proof. Uses: `def:weil-a2-torsion-constructibility-witness`

This is the vanishing field of the witness, evaluated at $j<r$ and $j>s$. □

Lemma 10.17.25 (Torsion cohomology is finite locally constant on each stratum). Lean: `conditional-interface:lemma:weil-a2-torsion-witness-cohomology`

Uses: `def:weil-a2-torsion-constructibility-witness`

For every stratum i_a and integer j, the sheaf $H^j(i_a^*K)$ is identified with a finite locally constant Λ_n-module sheaf.

Proof. Uses: `def:weil-a2-torsion-constructibility-witness`

This is the final family of fields in the witness, evaluated at (a,j). □

Proposition 10.17.26 (Independence of the witnessing stratification). Lean: `conditional-interface:prop:weil-a2-constructibility-stratification-independent`

Uses: `def:weil-a2-constructible-complex`

If K is lisse on one finite stratification, it is lisse on every refinement of that stratification.

Proof. Uses: `prop:weil-a2-stratification-common-refinement,prop:weil-a2-completed-monoidal-coherence-open,prop:weil-a2-locally-closed-pullback-package-open`

OPEN obligation A2-LISSE-PULLBACK. For each refined stratum, use composition of locally closed pullbacks to identify its restriction with pullback of a lisse object. Pull back its evaluation and coevaluation maps, use tensor compatibility to identify their targets, and pull back both triangle equations. This proof closes once the pullback and completed monoidal obligations are closed. □

Definition 10.17.27 (Locally perfect-constant trivialization datum). Lean: `conditional-interface:def:weil-a2-locally-perfect-constant-trivialization`

Uses: `def:weil-a2-complete-derived-category,def:weil-a2-fixed-locally-closed-pullbacks`

A locally perfect-constant trivialization datum for K consists of a pro-etale cover $(U_j \to X)_j$, perfect complexes P_j of $\mathbf{Z}_\ell$-modules, and isomorphisms

$$K|_{U_j} \simeq \underline{\widehat{P}_j}_{U_j}$$

whose pullbacks to every pairwise fiber product agree.

Theorem 10.17.28 (Local perfect-complex criterion). Lean: `conditional-interface:thm:weil-a2-lisse-local-perfect-open`

Uses: `def:weil-a2-lisse-complex,def:weil-a2-locally-perfect-constant-trivialization`

If X is locally noetherian, a complete complex is lisse if and only if it has a locally perfect-constant trivialization datum.

Proof. Uses: `prop:weil-a2-completed-monoidal-coherence-open,prop:weil-a2-locally-closed-pullback-package-open`

OPEN obligation A2-LISSE-LOCAL-PERFECT. On a w-contractible affine, construct the section functor and its constant-sheaf inverse, prove the unit and counit on dualizable objects, identify dualizable coefficient complexes with perfect complexes, and descend the resulting local isomorphisms through pro-etale hyperdescent. Conversely, pull back the standard duality datum of each perfect P_j, verify both triangle equations locally, and descend the datum. □

Proposition 10.17.29 (Constructible complete complexes are bounded). Lean: `conditional-interface:prop:weil-a2-constructible-bounded-open`

Uses: `def:weil-a2-constructible-complex,def:weil-a2-bounded-object`

If X is noetherian and qcqs, every constructible complete complex is bounded.

Proof. Uses: `thm:weil-a2-lisse-local-perfect-open`

OPEN obligation A2-CONSTRUCTIBLE-BOUNDED. Refine a witnessing finite stratification by finitely many w-contractible affine covers. Use the local perfect criterion to obtain a finite cohomological interval on each member, take the minimum and maximum over the finite family, and use the localization triangles for successive closed-open decompositions to propagate that interval from the strata to X. □

Proposition 10.17.30 (Constructible tensor and internal Hom closure). *Lean:* `conditional-interface:prop:weil-a2-constructible-tensor-hom-open`

Uses: `def:weil-a2-constructible-complex`, `def:weil-a2-completed-tensor`
Completed tensor products and completed internal Homs of constructible objects are constructible.

Proof. Uses: `prop:weil-a2-stratification-common-refinement`, `thm:weil-a2-lisse-local-perfect-open`

OPEN obligation A2-CONSTRUCTIBLE-TENSOR-HOM. Pass to the common refinement of the two witnessing stratifications. On each refined stratum, replace both objects by perfect coefficient complexes, compute tensor and internal Hom in perfect-complex algebra, and prove the results remain perfect. Transport back through the local equivalences and descend the local trivialization data. □

Proposition 10.17.31 (Finite reduction preserves the constructible bounded category). *Lean:* `conditional-interface:prop:weil-a2-reduction-preserves-dbc-open`

Uses: `def:weil-a2-dbc-zl`, `def:weil-a2-dbc-torsion`, `def:weil-a2-finite-reduction`
For $K \in D_c^b(X, \mathbf{Z}_\ell)$, $\mathrm{Red}_n(K) \in D_c^b(X, \mathbf{Z}/\ell^n)$.

Proof. Uses: `prop:weil-a2-constructibility-stratification-independent`

OPEN obligation A2-REDUCTION-CONSTRUCTIBLE. Boundedness follows from the two-term amplitude of L_n. On each witnessing stratum, tensor the duality datum with L_n, prove the reduced evaluation and coevaluation maps satisfy both triangle equations, and identify restriction after reduction with reduction after restriction. The last identification uses the open pullback-package compatibility. □

10.18 Rational coefficients by explicit linearization

Definition 10.18.1 (ℓ-fraction morphism representative). Lean: `conditional-interface:def:weil-a2-ql-morphism-representative`

Uses: `def:weil-a2-dbc-zl`
For $K, L \in D_c^b(X, \mathbf{Z}_\ell)$, a fraction representative from K to L is a pair (f, n) with

$$f : K \to L, \qquad n \in \mathbf{N},$$

written formally as $\ell^{-n} f$.

Definition 10.18.2 (Equality of ℓ-fraction representatives). Lean: `conditional-interface:def:weil-a2-ql-morphism-equivalence`

Uses: `def:weil-a2-ql-morphism-representative`
Define

$$(f, n) \sim (g, m) \quad \Longleftrightarrow \quad \exists r \in \mathbf{N},\ \ell^{r+m} f = \ell^{r+n} g.$$

Lemma 10.18.3 (Fraction equality is reflexive). *Lean:* `conditional-interface:lemma:weil-a2-ql-fraction-reflexive`

Uses: `def:weil-a2-ql-morphism-equivalence`
For every (f, n), $(f, n) \sim (f, n)$.

Proof. Uses: `def:weil-a2-ql-morphism-equivalence`
Take $r = 0$; both sides are $\ell^n f$. □

Lemma 10.18.4 (Fraction equality is symmetric). *Lean:* `conditional-interface:lemma:weil-a2-ql-fraction-symmetric`

Uses: `def:weil-a2-ql-morphism-equivalence`
If $(f, n) \sim (g, m)$, then $(g, m) \sim (f, n)$.

Proof. Uses: `def:weil-a2-ql-morphism-equivalence`
Reuse the same witness r and reverse the equality. □

Lemma 10.18.5 (Fraction equality is transitive). *Lean:* `conditional-interface:lemma:weil-a2-ql-fraction-transitive`

Uses: `def:weil-a2-ql-morphism-equivalence`
If $(f, n) \sim (g, m)$ and $(g, m) \sim (h, t)$, then $(f, n) \sim (h, t)$.

Proof. Uses: def:weil-a2-ql-morphism-equivalence
Choose witnesses r, s. Multiply the first equality by ℓ^{s+t} and the second by ℓ^{r+n}. Their middle terms become the same power $\ell^{r+s+n+t}g$; cancelling the common displayed rewriting gives $\ell^{r+s+m+t}f = \ell^{r+s+m+n}h$. Thus $r+s+m$ is a witness. □

Proposition 10.18.6 (Fraction equality is an equivalence relation). Lean: conditional-interface:prop:weil-a2-ql-fraction-equivalence

Uses: def:weil-a2-ql-morphism-equivalence
The relation $\sim$ is an equivalence relation.

Proof. Uses: lemma:weil-a2-ql-fraction-reflexive,lemma:weil-a2-ql-fraction-symmetric,lemma:weil-a2-ql-fraction-transitive
The three cited lemmas are reflexivity, symmetry, and transitivity. □

Definition 10.18.7 (Rationalized Hom set). Lean: conditional-interface:def:weil-a2-ql-hom

Uses: def:weil-a2-ql-morphism-representative,def:weil-a2-ql-morphism-equivalence
Define

$$\mathrm{Hom}_{\mathbf{Q}_\ell}(K, L) := \{(f, n)\}/\sim .$$

Definition 10.18.8 (Composition of rationalized morphisms). Lean: conditional-interface:def:weil-a2-ql-composition

Uses: def:weil-a2-ql-hom
For representatives $(f, n) : K \to L$ and $(g, m) : L \to M$, define

$$(g, m) \circ (f, n) := (gf, m + n).$$

Define the identity of K to be $(\mathrm{id}_K, 0)$.

Lemma 10.18.9 (Rationalized composition is representative-independent). Lean: conditional-interface:lemma:weil-a2-ql-composition-well-defined

Uses: def:weil-a2-ql-composition
Equivalent representatives give equivalent composites.

Proof. Uses: def:weil-a2-ql-morphism-equivalence,def:weil-a2-ql-composition
Suppose $\ell^{r+n'}f = \ell^{r+n}f'$ and $\ell^{s+m'}g = \ell^{s+m}g'$. Multiply the first equality on the left by $\ell^{s+m'}g$, substitute the second equality, and use centrality of the scalar action. The result is exactly the fraction-equivalence equation for $(gf, m+n)$ and $(g'f', m'+n')$, with witness $r+s$. □

Lemma 10.18.10 (Rationalized composition is associative). Lean: conditional-interface:lemma:weil-a2-ql-composition-associative

Uses: def:weil-a2-ql-composition
Composition of rationalized morphisms is associative.

Proof. Uses: def:weil-a2-ql-composition
On representatives, both bracketings have numerator hgf and denominator exponent $n+m+t$. □

Lemma 10.18.11 (Rationalized identity laws). Lean: conditional-interface:lemma:weil-a2-ql-composition-identities

Uses: def:weil-a2-ql-composition
The displayed identity representatives are left and right identities.

Proof. Uses: def:weil-a2-ql-composition
Composition with $(\mathrm{id}, 0)$ leaves both numerator and denominator exponent unchanged. □

Definition 10.18.12 (Naive rationalized category). Lean: conditional-interface:def:weil-a2-ql-linearized-category

Uses: def:weil-a2-dbc-zl,def:weil-a2-ql-hom,def:weil-a2-ql-composition
Let $D^b_c(X, \mathbf{Z}_\ell)[1/\ell]$ have the same objects as $D^b_c(X, \mathbf{Z}_\ell)$ and the rationalized Hom sets above.

Proposition 10.18.13 (The rationalized data form a category). Lean: conditional-interface:prop:weil-a2-ql-linearized-category

Uses: def:weil-a2-ql-linearized-category
The preceding data define a $\mathbf{Q}_\ell$-linear category.

Proof. Uses: lemma:weil-a2-ql-composition-well-defined,lemma:weil-a2-ql-composition-associative,lemma:weil-a2-ql-composition-identities

The cited lemmas give a well-defined category. Addition is induced from common denominators. If $q = a/\ell^n \in \mathbf{Q}_\ell$, define $q[(f,m)] = [(af, m+n)]$. Cross-multiplication proves independence of the presentation of q, and the module equations reduce to integer scalar equations on a common denominator. □

Definition 10.18.14 (Karoubi completion). Lean: conditional-interface:def:weil-a2-ql-karoubi-completion

Uses: def:weil-a2-ql-linearized-category

Define an object of the Karoubi completion to be a pair (K, e) with $e : K \to K$ and $e^2 = e$. Define a morphism $(K, e) \to (L, e')$ to be a rationalized morphism $h : K \to L$ satisfying

$$h = e' h e.$$

Composition and identities are inherited, with identity e on (K, e).

Lemma 10.18.15 (Karoubi composition closes). Lean: conditional-interface:lemma:weil-a2-ql-karoubi-composition-closes

Uses: def:weil-a2-ql-karoubi-completion

If $h = e'he$ and $k = e''ke'$, then $kh = e''(kh)e$.

Proof. Uses: def:weil-a2-ql-karoubi-completion

Substitute both equations: $kh = (e''ke')(e'he) = e''k(e'^2)he = e''ke'he = e''(kh)e$. □

Lemma 10.18.16 (Karoubi identities close). Lean: conditional-interface:lemma:weil-a2-ql-karoubi-identities

Uses: def:weil-a2-ql-karoubi-completion

The arrow e satisfies $e = eee$ and is an identity on every admissible morphism with source or target (K, e).

Proof. Uses: def:weil-a2-ql-karoubi-completion

The first equality is $e^3 = e$, obtained twice from $e^2 = e$. For an admissible $h = e'he$, one has $he = h$ and $e'h = h$. □

Definition 10.18.17 (Rational constructible category). Lean: conditional-interface:def:weil-a2-dbc-ql

Uses: def:weil-a2-ql-karoubi-completion

Define

$$D_c^b(X_{\mathrm{proet}}, \mathbf{Q}_\ell) := \mathrm{Kar}\big(D_c^b(X, \mathbf{Z}_\ell)[1/\ell]\big).$$

Definition 10.18.18 (Rationalization functor). Lean: conditional-interface:def:weil-a2-zl-to-ql

Uses: def:weil-a2-dbc-zl,def:weil-a2-dbc-ql

Define

$$(-)_{\mathbf{Q}_\ell} : D_c^b(X, \mathbf{Z}_\ell) \longrightarrow D_c^b(X, \mathbf{Q}_\ell)$$

by $K \mapsto (K, \mathrm{id}_K)$ and $f \mapsto [(f, 0)]$.

Proposition 10.18.19 (Rationalization is a functor). Lean: conditional-interface:prop:weil-a2-zl-to-ql-functor

Uses: def:weil-a2-zl-to-ql

The preceding assignment preserves identities and composition.

Proof. Uses: lemma:weil-a2-ql-composition-associative,lemma:weil-a2-ql-composition-identities,lemma:weil-a2-ql-karoubi-identities

The denominator-zero representative of a composite is the composite of the denominator-zero representatives. The denominator-zero identity becomes the Karoubi identity of (K, id_K). □

Proposition 10.18.20 (Comparison with sheaf-theoretic $\mathbf{Q}_\ell$-coefficients). Lean: conditional-interface:prop:weil-a2-ql-lattice-comparison-open

Uses: def:weil-a2-dbc-ql

The Karoubi-completed rationalized category is equivalent to the category of bounded constructible complexes of sheaves of $\Lambda_X[1/\ell]$-modules.

Proof. Uses: `prop:weil-a2-zl-to-ql-functor`

OPEN obligation A2-QL-LATTICES. Construct an integral lattice for every lisse $\mathbf{Q}_\ell$-sheaf on each stratum, prove two lattices become commensurable after powers of ℓ, glue lattice data across a finite stratification, extend from lisse sheaves to bounded complexes by truncation, and prove full faithfulness by the fraction formula for Hom. Essential surjectivity requires the Karoubi completion to split the resulting idempotents. No lattice theorem is presently in mathlib. □

10.19 Finite extensions and their transition functors

Definition 10.19.1 (Finite coefficient-extension index). Lean: `conditional-interface:def:weil-a2-finite-extension-index`

Uses: `def:weil-coefficient-data`

Let FinExt_ℓ be the category whose objects are finite extensions $E/\mathbf{Q}_\ell$ contained in the fixed $\overline{\mathbf{Q}}_\ell$, and with a unique arrow $E \to F$ when $E \subseteq F$.

Definition 10.19.2 (Compositum in the fixed algebraic closure). Lean: `conditional-interface:def:weil-a2-finite-extension-compositum`

Uses: `def:weil-a2-finite-extension-index`

For $E, F \in \mathsf{FinExt}_\ell$, define EF to be the subfield of $\overline{\mathbf{Q}}_\ell$ generated by $E \cup F$.

Lemma 10.19.3 (The compositum is finite). Lean: *conditional-interface:lemma:weil-a2-finite-extension-compositum-finite*

Uses: *def:weil-a2-finite-extension-compositum*

The extension $EF/\mathbf{Q}_\ell$ is finite.

Proof. Uses: `def:weil-a2-finite-extension-compositum`

Choose finite bases of E and F. Products of basis elements span the subring generated by $E \cup F$, and its fraction field lies in the finite extension generated by those finitely many algebraic elements. Hence $[EF : \mathbf{Q}_\ell] \le [E : \mathbf{Q}_\ell][F : \mathbf{Q}_\ell]$. □

Proposition 10.19.4 (The finite-extension index is filtered). Lean: *conditional-interface:prop:weil-a2-finite-extension-index-filtered*

Uses: *def:weil-a2-finite-extension-index*

The category FinExt_ℓ is filtered.

Proof. Uses: `lemma:weil-a2-finite-extension-compositum-finite`

It is nonempty because it contains $\mathbf{Q}_\ell$. The compositum EF receives arrows from E and F. There are no nontrivial parallel arrows in this poset category, so the parallel-arrow condition is automatic. □

Definition 10.19.5 (Finite-extension continuous ring presheaf). Lean: `conditional-interface:def:weil-a2-oe-continuous-ring-presheaf`

Uses: `def:weil-a2-finite-extension-index`

For $E \in \mathsf{FinExt}_\ell$, let $\mathcal{O}_E$ be its valuation ring and define

$$\mathcal{O}^{\mathrm{pre}}_{E,X}(U) := C(|U|, \mathcal{O}_E), \qquad v^\sharp(a) := a \circ |v|.$$

All ring operations are pointwise.

Lemma 10.19.6 (Finite-extension restrictions are ring maps). Lean: *conditional-interface:lemma:weil-a2-oe-continuous-ring-presheaf*

Uses: *def:weil-a2-oe-continuous-ring-presheaf*

The restrictions in the preceding definition preserve $0, 1, +, -, \cdot$ and satisfy identity and composition.

Proof. Uses: `def:weil-a2-oe-continuous-ring-presheaf`

Evaluate at $x \in |V|$. Every assertion is the corresponding equality in $\mathcal{O}_E$, and precomposition gives the two functor laws. □

Definition 10.19.7 (Finite-extension ring source sheaf). Lean: `conditional-interface:def:weil-a2-oe-ring-source-sheaf`

Uses: `def:weil-a2-oe-continuous-ring-presheaf`

Let $\mathcal{C}^{\mathrm{ring}}_{\mathcal{O}_E}$ be the `continuousMapPresheaf`$(\mathcal{O}_E)$ sheaf with the ring structure transported from pointwise operations.

Proposition 10.19.8 (Finite-extension source ring sheaf). Lean: `conditional-interface:prop:weil-a2-oe-ring-source-sheaf`

Uses: `def:weil-a2-oe-ring-source-sheaf`

$\mathcal{C}^{\mathrm{ring}}_{\mathcal{O}_E}$ is a ring sheaf on the source site.

Proof. Uses: `lemma:weil-a2-oe-continuous-ring-presheaf`

The additive continuous-map sheaf theorem gives locality and gluing. Pointwise multiplication and the unit preserve matching families, and uniqueness of additive gluing reflects the ring axioms. □

Definition 10.19.9 (Integral coefficient ring for E). Lean: `conditional-interface:def:weil-a2-oe-ring-sheaf`

Uses: `def:weil-a2-finite-extension-index,def:weil-a2-oe-ring-source-sheaf`

Define $(\mathcal{O}_E)_X$ as the `sheafPushforwardContinuous` image of $\mathcal{C}^{\mathrm{ring}}_{\mathcal{O}_E}$ along the same pro-etale forgetful functor used for Λ_X.

Definition 10.19.10 (Integral constructible category for E). Lean: `conditional-interface:def:weil-a2-dbc-oe`

Uses: `def:weil-a2-oe-ring-sheaf,def:weil-a2-dbc-zl`

Define $D^b_c(X, \mathcal{O}_E)$ by the same complete, lisse-on-a-finite- stratification, and bounded conditions as Definition 10.17.21, using $(\mathcal{O}_E)_X$.

Definition 10.19.11 (Uniformizer linearization for E). Lean: `conditional-interface:def:weil-a2-e-linearized-category`

Uses: `def:weil-a2-dbc-oe,def:weil-a2-ql-linearized-category,def:weil-a2-ql-morphism-representative`

Choose the least uniformizer ϖ_E in the fixed well-order of $\mathcal{O}_E$. Form

$$D^b_c(X, \mathcal{O}_E)[1/\varpi_E]$$

by the same fraction-representative construction as Definitions 10.18.1 through 10.18.12, with ℓ replaced by ϖ_E.

Definition 10.19.12 (E-coefficient constructible category). Lean: `conditional-interface:def:weil-a2-dbc-e`

Uses: `def:weil-a2-e-linearized-category`

Define

$$D^b_c(X, E) := \mathrm{Kar}\big(D^b_c(X, \mathcal{O}_E)[1/\varpi_E]\big).$$

Proposition 10.19.13 (Independence of the uniformizer). Lean: `conditional-interface:prop:weil-a2-dbc-e-uniformizer-independent`

Uses: `def:weil-a2-dbc-e`

Replacing ϖ_E by any other uniformizer produces canonically equivalent E-coefficient categories.

Proof. Uses: `prop:weil-a2-ql-fraction-equivalence`

If $\varpi'_E = u\varpi_E$ with $u \in \mathcal{O}_E^\times$, send (f, n) in the first fraction category to $(u^{-n} f, n)$ in the second. The fraction-equivalence formula shows this is well-defined. The analogous formula with u^n is inverse. Both functors act identically on objects and commute with Karoubi completion. □

Definition 10.19.14 (Integral scalar-extension complex). Lean: `conditional-interface:def:weil-a2-integral-scalar-extension`

Uses: `def:weil-a2-dbc-oe,def:weil-a2-fixed-kflat-package`

For an inclusion $E \subseteq F$ and $K \in D^b_c(X, \mathcal{O}_E)$, define

$$K_{\mathcal{O}_F} := Q_X\big(\mathfrak{P}_E(K) \otimes_{(\mathcal{O}_E)_X} (\mathcal{O}_F)_X\big).$$

Proposition 10.19.15 (Integral scalar extension preserves D^b_c). Lean: `conditional-interface:prop:weil-a2-integral-scalar-extension-dbc-open`

Uses: `def:weil-a2-integral-scalar-extension`

The object $K_{\mathcal{O}_F}$ is bounded, derived complete, and constructible over $\mathcal{O}_F$.

Proof. Uses: `prop:weil-a2-reduction-preserves-dbc-open`

OPEN obligation A2-COEFFICIENT-EXTENSION-CONSTRUCTIBLE. Prove $\mathcal{O}_F$ finite free over $\mathcal{O}_E$, use its finite flatness to preserve boundedness, tensor a local duality datum to preserve lissity, commute scalar extension with locally closed pullback, and prove derived completeness from the finite-level reduction formula. The pullback compatibility is still an open predecessor. □

Definition 10.19.16 (Transition functor for finite coefficient fields)**.** Lean: `conditional-interface:def:weil-a2-finite-extension-transition`

Uses: `def:weil-a2-dbc-e`, `def:weil-a2-integral-scalar-extension`

For $E \subseteq F$, define

$$\mathrm{Ext}_{F/E} : D_c^b(X, E) \longrightarrow D_c^b(X, F)$$

by integral scalar extension on fraction representatives, followed by Karoubi completion.

Lemma 10.19.17 (Transition preserves identities)**.** Lean: `conditional-interface:lemma:weil-a2-finite-extension-transition-identity`

Uses: `def:weil-a2-finite-extension-transition`

For $E = F$, $\mathrm{Ext}_{E/E}$ is naturally isomorphic to the identity functor.

Proof. Uses: `def:weil-a2-finite-extension-transition`

The coefficient bimodule is $\mathcal{O}_E$, and $K \otimes^{\mathbf{L}}_{\mathcal{O}_E} \mathcal{O}_E \simeq K$ by the tensor unitor. The same componentwise isomorphism respects fraction morphisms and idempotents. □

Lemma 10.19.18 (Transition composition comparison)**.** Lean: `conditional-interface:lemma:weil-a2-finite-extension-transition-composition`

Uses: `def:weil-a2-finite-extension-transition`

For $E \subseteq F \subseteq G$, there is a natural isomorphism

$$\mathrm{Ext}_{G/F} \mathrm{Ext}_{F/E} \simeq \mathrm{Ext}_{G/E}\,.$$

Proof. Uses: `prop:weil-a2-completed-monoidal-coherence-open`, `prop:weil-a2-integral-scalar-extension-dbc-open`

OPEN obligation A2-COEFFICIENT-EXTENSION-COMPOSITION. The component must be induced by

$$(K \otimes^{\mathbf{L}}_{\mathcal{O}_E} \mathcal{O}_F) \otimes^{\mathbf{L}}_{\mathcal{O}_F} \mathcal{O}_G \simeq K \otimes^{\mathbf{L}}_{\mathcal{O}_E} \mathcal{O}_G.$$

One must prove it respects completion, fraction equivalence, and Karoubi idempotents, and then verify naturality. □

Proposition 10.19.19 (Transition cocycle)**.** Lean: `conditional-interface:prop:weil-a2-finite-extension-transition-cocycle-open`

Uses: `def:weil-a2-finite-extension-transition`

For $E \subseteq F \subseteq G \subseteq H$, the two composites of transition composition comparisons from fourfold scalar extension to $\mathrm{Ext}_{H/E}$ agree.

Proof. Uses: `lemma:weil-a2-finite-extension-transition-composition`

OPEN obligation A2-COEFFICIENT-EXTENSION-COCYCLE. Expand both composites to the associator of the derived tensor product. Their equality is the pentagon equation. The completed monoidal coherence obligation must supply that pentagon before this node can close. □

10.20 The explicit filtered bicategorical colimit for algebraically closed coefficients

Definition 10.20.1 ($\overline{\mathbf{Q}}_\ell$-object representative)**.** Lean: `conditional-interface:def:weil-a2-qbar-object-representative`

Uses: `def:weil-a2-finite-extension-index`, `def:weil-a2-dbc-e`

An object representative is a pair (E, K) with $E \in \mathsf{FinExt}_\ell$ and $K \in D_c^b(X, E)$.

Definition 10.20.2 (Common-stage morphism representative). Lean: `conditional-interface:def:weil-a2-qbar-morphism-representative`

Uses: `def:weil-a2-qbar-object-representative,def:weil-a2-finite-extension-transition`

From (E, K) to (F, L), a morphism representative is a triple $(G, \iota_E, \iota_F; h)$, where G is a finite extension containing E, F and

$$h : \mathrm{Ext}_{G/E}(K) \longrightarrow \mathrm{Ext}_{G/F}(L)$$

is a morphism in $D_c^b(X, G)$.

Definition 10.20.3 (Equality of common-stage representatives). Lean: `conditional-interface:def:weil-a2-qbar-morphism-equivalence`

Uses: `def:weil-a2-qbar-morphism-representative`

Representatives (G, h) and (G', h') are equivalent if there exists a finite extension H containing G, G' such that the two arrows obtained by applying the transition functors to H, and then the fixed composition comparisons, are equal in $D_c^b(X, H)$.

Lemma 10.20.4 (Common-stage equality is reflexive). Lean: *conditional-interface:lemma:weil-a2-qbar-equivalence-reflexive*

Uses: *def:weil-a2-qbar-morphism-equivalence*

Every representative is equivalent to itself.

Proof. Uses: `lemma:weil-a2-finite-extension-transition-identity`

Take $H = G$; both transported arrows are h after the identity comparison. □

Lemma 10.20.5 (Common-stage equality is symmetric). Lean: *conditional-interface:lemma:weil-a2-qbar-equivalence-symmetric*

Uses: *def:weil-a2-qbar-morphism-equivalence*

If $(G, h) \sim (G', h')$, then $(G', h') \sim (G, h)$.

Proof. Uses: `def:weil-a2-qbar-morphism-equivalence`

Reuse the same common field H and reverse the equality. □

Lemma 10.20.6 (Common-stage equality is transitive). Lean: *conditional-interface:lemma:weil-a2-qbar-equivalence-transitive*

Uses: *def:weil-a2-qbar-morphism-equivalence*

If $(G, h) \sim (G', h')$ and $(G', h') \sim (G'', h'')$, then $(G, h) \sim (G'', h'')$.

Proof. Uses: `lemma:weil-a2-finite-extension-compositum-finite,prop:weil-a2-finite-extension-transition-cocycle-open`

OPEN until A2-COEFFICIENT-EXTENSION-COCYCLE closes. Let H_1, H_2 witness the two equalities and take their compositum H. Transport both equalities to H. The cocycle identifies each iterated transport with direct transport. Transitivity of equality in $D_c^b(X, H)$ then identifies the transports of h and h''. □

Proposition 10.20.7 (Common-stage equality is an equivalence relation). Lean: *conditional-interface:prop:weil-a2-qbar-morphism-equivalence*

Uses: *def:weil-a2-qbar-morphism-equivalence*

The common-stage relation is an equivalence relation, conditional only on the transition cocycle obligation.

Proof. Uses: `lemma:weil-a2-qbar-equivalence-reflexive,lemma:weil-a2-qbar-equivalence-symmetric,lemma:weil-a2-qbar-equivalence-transitive`

The cited lemmas give reflexivity, symmetry, and transitivity. □

Definition 10.20.8 ($\overline{\mathbf{Q}}_\ell$-Hom set). Lean: `conditional-interface:def:weil-a2-qbar-hom`

Uses: `def:weil-a2-qbar-morphism-representative,def:weil-a2-qbar-morphism-equivalence`

Define

$$\mathrm{Hom}_{\overline{\mathbf{Q}}_\ell}((E, K), (F, L)) := \{\text{common-stage representatives}\}/\sim .$$

Definition 10.20.9 (Identity representative). Lean: `conditional-interface:def:weil-a2-qbar-identity`

Uses: `def:weil-a2-qbar-object-representative,def:weil-a2-qbar-morphism-representative`

Define the identity of (E, K) to be represented at stage E by id_K.

Definition 10.20.10 (Composition representative). Lean: `conditional-interface:def:weil-a2-qbar-composition`

Uses: `def:weil-a2-qbar-morphism-representative,def:weil-a2-finite-extension-compositum`

For $(G,h):(E,K)\to(F,L)$ and $(G',k):(F,L)\to(E',M)$, take $H=GG'$, transport h,k to H, use the composition comparisons to identify their middle objects, and define the composite representative to be their composite in $D_c^b(X,H)$.

Lemma 10.20.11 (Composition is independent of the chosen common field). Lean: *`conditional-interface:lemma:weil-a2-qbar-composition-stage-independent`*

Uses: *`def:weil-a2-qbar-composition`*

Replacing H by any larger common field gives an equivalent composite representative.

Proof. Uses: `lemma:weil-a2-finite-extension-transition-composition,prop:weil-a2-finite-extension-transition-cocycle-open`

OPEN until the transition cocycle closes. Transport the composite to the larger field. Functoriality sends it to the composite of the two transported arrows. The cocycle identifies all iterated transports with the direct transports used in the larger-stage definition. □

Lemma 10.20.12 (Composition is independent of representatives). Lean: *`conditional-interface:lemma:weil-a2-qbar-composition-representative-independent`*

Uses: *`def:weil-a2-qbar-composition`*

Equivalent representatives $h\sim\tilde h$ and $k\sim\tilde k$ yield equivalent composites.

Proof. Uses: `lemma:weil-a2-finite-extension-compositum-finite,lemma:weil-a2-qbar-composition-stage-independent`

Choose composita of the two equality-witness fields and then one field containing both. At that common stage the transported arrows are equal in each factor, hence their composites are equal. Stage independence returns the required equivalence classes. □

Lemma 10.20.13 (Composition is associative). Lean: *`conditional-interface:lemma:weil-a2-qbar-composition-associative`*

Uses: *`def:weil-a2-qbar-composition`*

Composition of $\overline{\mathbf{Q}}_\ell$-morphisms is associative.

Proof. Uses: `lemma:weil-a2-finite-extension-compositum-finite,prop:weil-a2-finite-extension-transition-cocycle-open`

OPEN until the transition cocycle closes. Move three representatives to one finite compositum. The two bracketings are then the same by associativity in $D_c^b(X,H)$; the cocycle identifies the two transport paths used to reach H. □

Lemma 10.20.14 (Identity laws). Lean: *`conditional-interface:lemma:weil-a2-qbar-composition-identities`*

Uses: *`def:weil-a2-qbar-identity,def:weil-a2-qbar-composition`*

The selected identity representatives are left and right identities.

Proof. Uses: `lemma:weil-a2-finite-extension-transition-identity,lemma:weil-a2-qbar-composition-stage-independent`

Move the identity and an arbitrary representative to the representative's stage. The identity transition comparison turns the first into the actual identity there, so composition leaves the arrow unchanged. □

Definition 10.20.15 (Explicit $\overline{\mathbf{Q}}_\ell$-coefficient category). Lean: `conditional-interface:def:weil-a2-dbc-qbar`

Uses: `def:weil-a2-qbar-object-representative,def:weil-a2-qbar-hom,def:weil-a2-qbar-identity,def:weil-a2-qbar-composition`

Define

$$D_c^b(X,\overline{\mathbf{Q}}_\ell)$$

to have object representatives (E,K), the quotient Hom sets, and the preceding identities and composition.

Proposition 10.20.16 (The explicit colimit data form a category). Lean: *`conditional-interface:prop:weil-a2-dbc-qbar-category`*

Uses: *`def:weil-a2-dbc-qbar`*

Conditional on the explicit transition-cocycle obligation, the preceding data form a category.

Proof. Uses: `lemma:weil-a2-qbar-composition-representative-independent`, `lemma:weil-a2-qbar-composition-associative`, `lemma:weil-a2-qbar-composition-identities`

The first lemma makes composition well-defined on quotient Hom sets. The last two supply associativity and identities. □

Definition 10.20.17 (Finite-stage inclusion into the colimit). Lean: `conditional-interface:def:weil-a2-e-to-qbar`

Uses: `def:weil-a2-dbc-e`, `def:weil-a2-dbc-qbar`

For $E \in \mathsf{FinExt}_\ell$, define

$$\iota_E : D_c^b(X, E) \longrightarrow D_c^b(X, \overline{\mathbf{Q}}_\ell)$$

by $K \mapsto (E, K)$ and by representing a morphism at stage E.

Proposition 10.20.18 (Finite-stage inclusion is a functor). Lean: `conditional-interface:prop:weil-a2-e-to-qbar-functor`

Uses: `def:weil-a2-e-to-qbar`

The map ι_E preserves identities and composition.

Proof. Uses: `def:weil-a2-qbar-identity`, `def:weil-a2-qbar-composition`

Both assertions can be computed at the unchanged stage E, where they are the identity and composition equations in $D_c^b(X, E)$. □

Proposition 10.20.19 (Bicategorical filtered-colimit universal property). Lean: `conditional-interface:prop:weil-a2-qbar-filtered-colimit-open`

Uses: `def:weil-a2-dbc-qbar`, `def:weil-a2-e-to-qbar`

The category $D_c^b(X, \overline{\mathbf{Q}}_\ell)$, the functors ι_E, and their transition comparison isomorphisms satisfy the bicategorical filtered-colimit universal property.

Proof. Uses: `prop:weil-a2-finite-extension-index-filtered`, `prop:weil-a2-dbc-qbar-category`, `prop:weil-a2-e-to-qbar-functor`

OPEN obligation A2-QBAR-2COLIMIT-UNIVERSAL. Given a pseudocone $(F_E, \alpha_{F/E})$ to a category $\mathcal{C}$, define the induced functor on (E, K) by $F_E(K)$, and define it on a common-stage representative by transporting with α. Prove representative and stage independence using the pseudocone cocycle. Conversely restrict a functor along every ι_E. Construct the two comparison pseudonatural equivalences and verify both modifications componentwise. □

10.21 Compatible finite-level systems and reconstruction

Definition 10.21.1 (Normalized finite-level system). Lean: `conditional-interface:def:weil-a2-finite-level-system`

Uses: `def:weil-a2-finite-reduction`

A normalized finite-level system consists of objects

$$K_n \in \mathcal{D}(X, \Lambda/\ell^n) \qquad (n \geq 1)$$

and isomorphisms

$$\theta_{n+1,n} : K_{n+1} \otimes^{\mathbf{L}}_{\Lambda/\ell^{n+1}} \Lambda/\ell^n \xrightarrow{\sim} K_n$$

such that the two induced isomorphisms from level $n+2$ to level n agree.

Definition 10.21.2 (Morphism of finite-level systems). Lean: `conditional-interface:def:weil-a2-finite-level-system-morphism`

Uses: `def:weil-a2-finite-level-system`

A morphism $f_\bullet : K_\bullet \to L_\bullet$ is a family $f_n : K_n \to L_n$ such that every square with the two normalization isomorphisms commutes.

Definition 10.21.3 (Category of normalized systems). Lean: `conditional-interface:def:weil-a2-finite-level-system-category`

Uses: `def:weil-a2-finite-level-system`, `def:weil-a2-finite-level-system-morphism`

Let $\mathsf{Sys}(X, \Lambda)$ be the category of normalized systems, with componentwise identities and composition.

Lemma 10.21.4 (System identities satisfy compatibility). *Lean:* `conditional-interface:lemma:weil-a2-finite-level-system-identity`

Uses: `def:weil-a2-finite-level-system-category`
The componentwise identity is a morphism of systems.

Proof. Uses: `def:weil-a2-finite-level-system-morphism`
Every compatibility square has identity vertical arrows. □

Lemma 10.21.5 (System morphisms are closed under composition). *Lean:* `conditional-interface:lemma:weil-a2-finite-level-system-composition`

Uses: `def:weil-a2-finite-level-system-category`
The componentwise composite of two system morphisms is a system morphism.

Proof. Uses: `def:weil-a2-finite-level-system-morphism`
Paste their two compatibility squares; the outer rectangle is the required square for the composite. □

Definition 10.21.6 (Reduction-system functor). Lean: `conditional-interface:def:weil-a2-reduction-system-functor`

Uses: `def:weil-a2-complete-derived-category,def:weil-a2-finite-level-system-category,def:weil-a2-finite-reduction,def:weil-a2-reduction-transition`
Define

$$\mathscr{R} : \mathcal{D}_{\mathrm{comp}}(X, \Lambda) \longrightarrow \mathsf{Sys}(X, \Lambda)$$

by $K \mapsto (\mathrm{Red}_n(K))_n$, using the normalization maps induced by

$$L_{n+1} \otimes^{\mathbf{L}}_{\Lambda} \Lambda/\ell^n \simeq \Lambda/\ell^n.$$

Proposition 10.21.7 (The reductions form a normalized system). *Lean:* `conditional-interface:prop:weil-a2-reductions-normalized-open`

Uses: `def:weil-a2-reduction-system-functor`
For every complete K, the finite reductions satisfy the normalization cocycle.

Proof. Uses: `lemma:weil-a2-reduction-transition-chain-map,prop:weil-a2-completion-idempotent-open`
OPEN obligation A2-REDUCTION-NORMALIZATION. Compute the double tensor of L_{n+1} and L_n, contract its acyclic summand, identify the remaining summand with L_n, and verify that the two contractions from level $n+2$ to level n are chain homotopic. Then transport through K-flat replacement and completion. □

Definition 10.21.8 (System reconstruction). Lean: `conditional-interface:def:weil-a2-system-reconstruction`

Uses: `def:weil-a2-finite-level-system-category,def:weil-a2-derived-inverse-limit`
View each K_n as a Λ_X-complex by restriction of scalars and define

$$\mathscr{L}(K_\bullet) := \mathbf{R}\varprojlim_n K_n.$$

Use the induced map of product-cone models on morphisms.

Lemma 10.21.9 (Reconstruction is functorial). *Lean:* `conditional-interface:lemma:weil-a2-system-reconstruction-functorial`

Uses: `def:weil-a2-system-reconstruction`
The assignment $\mathscr{L}$ preserves identities and composition.

Proof. Uses: `def:weil-a2-derived-inverse-limit,lemma:weil-a2-tower-product-shift-chain-map`
System maps induce product-complex maps commuting with s, hence maps of the cones of $1 - s$. Cones and localization preserve identity and composition. □

Definition 10.21.10 (Etale-origin datum at level n). Lean: `conditional-interface:def:weil-a2-etale-origin-datum`

Uses: `def:weil-a2-finite-level-system`
Let $\nu_X : X_{\mathrm{proet}} \to X_{\mathrm{et}}$ be the canonical morphism of sites. An etale-origin datum for K_n consists of a bounded-below constructible complex

$$L_n \in D^+(X_{\mathrm{et}}, \Lambda/\ell^n)$$

and an isomorphism $\nu_X^* L_n \simeq K_n$.

Definition 10.21.11 (Etale-origin predicate). Lean: `conditional-interface:def:weil-a2-etale-origin-predicate`

Uses: `def:weil-a2-etale-origin-datum`

Say that K_n comes from the etale site if its type of etale-origin data is nonempty.

Definition 10.21.12 (Compatible etale-origin system). Lean: `conditional-interface:def:weil-a2-compatible-etale-origin-system`

Uses: `def:weil-a2-finite-level-system,def:weil-a2-etale-origin-datum`

A compatible etale-origin system consists of etale-origin data (L_n, α_n) for all n, transition maps $L_{n+1} \otimes^{\mathbf{L}} \Lambda/\ell^n \to L_n$, and the equality saying that their pullbacks correspond under α_{n+1}, α_n to the normalization maps of $K_\bullet$.

Definition 10.21.13 (Reconstruction counit). Lean: `conditional-interface:def:weil-a2-system-reconstruction-counit`

Uses: `def:weil-a2-system-reconstruction,def:weil-a2-finite-reduction`

For a normalized system $K_\bullet$, define

$$\epsilon_n : \mathrm{Red}_n(\mathscr{L}(K_\bullet)) \longrightarrow K_n$$

by projection of the homotopy limit to level n, reduction modulo ℓ^n, and the normalization isomorphisms.

Proposition 10.21.14 (Reconstruction counit is an isomorphism). *Lean: `conditional-interface:prop:weil-a2-system-reconstruction-counit-open`*

Uses: `def:weil-a2-system-reconstruction-counit`

For every normalized system and every n, ϵ_n is an isomorphism, compatibly in n.

Proof. Uses: `prop:weil-a2-reductions-normalized-open,prop:weil-a2-derived-limit-package-independent-open`

OPEN obligation A2-SYSTEM-COUNIT. Commute finite reduction with the product-cone model for $\mathbf{R}\varprojlim$. Use normalization to identify the reduced tower with one whose transition maps are eventually identities at level n. Contract the cone of $1 - s$ on the tail, identify the surviving factor with K_n, and verify transition compatibility. □

Definition 10.21.15 (Completion reconstruction unit). Lean: `conditional-interface:def:weil-a2-complete-reconstruction-unit`

Uses: `def:weil-a2-reduction-system-functor,def:weil-a2-system-reconstruction,def:weil-a2-completion-unit`

For complete K, define

$$u_K : K \longrightarrow \mathscr{L}\mathscr{R}(K)$$

to be the completion unit, using the identical homotopy-limit model on the right.

Proposition 10.21.16 (Completion reconstruction unit is an isomorphism). *Lean: `conditional-interface:prop:weil-a2-complete-reconstruction-unit`*

Uses: `def:weil-a2-complete-reconstruction-unit`

For every complete K, u_K is an isomorphism.

Proof. Uses: `def:weil-a2-derived-complete-object`

Derived completeness is defined to mean precisely that this completion unit is an isomorphism. □

Theorem 10.21.17 (Completion-system equivalence). *Lean: `conditional-interface:thm:weil-a2-completion-system-equivalence-open`*

Uses: `def:weil-a2-complete-derived-category,def:weil-a2-finite-level-system-category,def:weil-a2-reduction-system-functor,def:weil-a2-system-reconstruction`

The functors $\mathscr{R}$ and $\mathscr{L}$ are quasi-inverse equivalences

$$\mathcal{D}_{\mathrm{comp}}(X, \Lambda) \simeq \mathsf{Sys}(X, \Lambda).$$

Proof. Uses: `prop:weil-a2-system-reconstruction-counit-open,prop:weil-a2-complete-reconstruction-unit`

OPEN until A2-SYSTEM-COUNIT closes. The cited propositions give the counit and unit isomorphisms. Their naturality follows componentwise. The two triangle equations reduce at each finite level to the projection-normalization identity. Conservativity of all finite reductions on complete objects then gives the global equations. □

Proposition 10.21.18 (Conservativity of all finite reductions). *Lean:* `conditional-interface:prop:weil-a2-finite-reductions-conservative-open`

Uses: `def:weil-a2-complete-derived-category`, `def:weil-a2-finite-reduction`

If a morphism $f : K \to L$ between complete objects becomes an isomorphism under every Red_n, then f is an isomorphism.

Proof. Uses: `thm:weil-a2-completion-system-equivalence-open`

OPEN until the completion-system equivalence closes. Under that equivalence, f is sent to a componentwise isomorphism of systems. Its componentwise inverse is a system morphism because the compatibility squares for f can be inverted. Apply the reconstruction functor to that inverse. □

10.22 The exact proper-complete interface used by pro-etale base change

Definition 10.22.1 (Proper base-change square). Uses:

Lean: `conditional-interface:def:weil-a2-proper-base-change-square`

For schemes X, Y, Y', a proper base-change square is data

$$\begin{array}{ccc} X' & \xrightarrow{g'} & X \\ {\scriptstyle f'}\downarrow & & \downarrow{\scriptstyle f} \\ Y' & \xrightarrow{g} & Y \end{array}$$

with f proper and $X' = X \times_Y Y'$, together with the specified identification $fg' = gf'$. The square is part of the datum, so later base-change maps refer to this chosen cartesian identification.

Definition 10.22.2 (Finite-level etale-origin realization). Lean: `conditional-interface:def:weil-a2-finite-level-etale-origin-realization`

Uses: `def:weil-a2-finite-level-system`, `def:weil-a2-etale-origin-datum`

For a finite-level system $K_\bullet = (K_n, r_{n+1,n})$, a finite-level etale-origin realization is the family of explicit data

$$(L_n, \alpha_n), \qquad L_n \in D^+(X_{\mathrm{et}}, \Lambda_n), \quad \alpha_n : \nu_X^* L_n \xrightarrow{\sim} K_n,$$

where each L_n is bounded below and constructible. It also contains, for every n, the displayed transition arrow

$$\tau_n : L_{n+1} \otimes^{\mathbf{L}}_{\Lambda_{n+1}} \Lambda_n \longrightarrow L_n$$

and the equality

$$\alpha_n \circ \nu_X^*(\tau_n) = r_{n+1,n} \circ (\alpha_{n+1} \otimes \Lambda_n)$$

after inserting the normalization isomorphism of the finite-level system.

Lemma 10.22.3 (Finite-level etale-origin realization has the required source). *Lean:* `conditional-interface:lemma:weil-a2-finite-level-etale-origin-source`

Uses: `def:weil-a2-finite-level-etale-origin-realization`

The pullback of every L_n in a finite-level etale-origin realization is bounded below and constructible on X_{proetale}.

Proof. Uses: `def:weil-a2-finite-level-etale-origin-realization`

Bounded-below means that one integer a_n kills all cohomology below a_n; the same inequality is preserved by the exact inverse-image functor ν_X^*. Constructibility is part of the input datum and its finite stratification is pulled back along ν_X, so each cohomology sheaf has the displayed locally constant finite-level model. □

Definition 10.22.4 (Perfect finite-level coefficient datum). Lean: `conditional-interface:def:weil-a2-perfect-finite-level-coefficient-datum`

Uses: `def:weil-a2-finite-coefficient-complex`, `def:weil-a2-integral-ring-sheaf`

For every $n \geq 1$, define a perfect finite-level coefficient datum to be the two-term finite free resolution

$$0 \longrightarrow \Lambda_X \xrightarrow{\ell^n} \Lambda_X \longrightarrow \Lambda_{n,X} \longrightarrow 0, \qquad \Lambda_{n,X} := \Lambda_X / \ell^n \Lambda_X,$$

together with its tensorial base-change isomorphisms. The datum records that $\Lambda_{n,X}$, regarded as a complex of Λ_X-modules in degree zero, is perfect of Tor-amplitude contained in $[-1, 0]$.

Lemma 10.22.5 (The finite coefficient sheaf is perfect). Lean: `conditional-interface:lemma:weil-a2-perfect-finite-level-coefficient`

Uses: `def:weil-a2-perfect-finite-level-coefficient-datum`

The complex $\Lambda_{n,X}$ is perfect as a Λ_X-module and its derived tensor computes finite reduction.

Proof. Uses: `def:weil-a2-perfect-finite-level-coefficient-datum,def:weil-a2-finite-reduction`

The displayed two-term complex is a bounded complex of finite free Λ_X-modules. Its cokernel is $\Lambda_{n,X}$ and its kernel is zero because multiplication by the non-zero-divisor ℓ^n on $\mathbf{Z}_\ell$ is injective. Hence it is a finite locally free resolution, so its derived tensor with K is exactly $K \otimes^{\mathbf{L}}_{\Lambda_X} \Lambda_{n,X} = \mathrm{Red}_n(K)$. □

Definition 10.22.6 (Proper complete-base-change datum). Lean: `conditional-interface:def:weil-a2-proper-complete-base-change-datum`

Uses: `def:weil-a2-proper-base-change-square,def:weil-a2-derived-complete-object,def:weil-a2-finite-level-etale-origin-realization,def:weil-a2-perfect-finite-level-coefficient-datum`

For a proper base-change square and a complete object $K \in \mathcal{D}_{\mathrm{comp}}(X, \Lambda)$, a proper complete-base-change datum consists of:

1. the derived-completeness isomorphism $\eta_K : K \xrightarrow{\sim} \widehat{K}$;
2. a finite-level etale-origin realization of the reduction system $(\mathrm{Red}_n(K))_n$;
3. the perfect finite-level coefficient datum;
4. for every n, the finite-level proper-etale base-change transformation
$$\beta_n : Lg^* Rf_* K_n \longrightarrow Rf'_* L(g')^* K_n;$$
5. a proof that the β_n commute with all transition maps and with the normalization maps of the reduction system.

The object does not assert that any of these fields exists; it only exposes the exact input required by the subsequent comparison proposition.

Definition 10.22.7 (Pro-etale proper base-change morphism). Lean: `conditional-interface:def:weil-a2-proetale-proper-base-change-map`

Uses: `def:weil-a2-proper-base-change-square,def:weil-a2-derived-completion`

For a proper square and $K \in \mathcal{D}_{\mathrm{comp}}(X, \Lambda)$, define
$$\beta_K^{\mathrm{proet}} : Lg^*\, Rf_*\, K \longrightarrow Rf'_*\, L(g')^* K$$
as the composite obtained by replacing K with its explicit completion cone, applying Rf_* and Rf'_* levelwise, and taking the induced map between the two homotopy-limit cones. At level n its component is the finite-level map β_n from a proper complete-base-change datum.

Lemma 10.22.8 (The pro-etale proper-base-change map is well-defined). Lean: `conditional-interface:lemma:weil-a2-proetale-proper-base-change-well-defined`

Uses: `def:weil-a2-proetale-proper-base-change-map,def:weil-a2-derived-inverse-limit`

Changing the chosen K-injective tower or the representatives of the finite-level transformations produces the same morphism after the canonical derived-limit comparison.

Proof. Uses: `prop:weil-a2-derived-limit-package-independent-open`

For two tower packages, use the common refinement isomorphism of the derived-limit package. The transition-compatibility field in the datum says that both representatives induce the same map on the product complex and commute with $1 - s$. Therefore they induce the same cone morphism; the canonical comparison transports this equality to the derived category. □

Definition 10.22.9 (Etale-to-pro-etale pullback comparison). Lean: `conditional-interface:def:weil-a2-etale-proetale-pullback-comparison`

Uses: `def:weil-a2-proper-base-change-square`

For a morphism $g : Y' \to Y$, let $\nu_Y : Y_{\mathrm{proet}} \to Y_{\mathrm{et}}$ and $\nu_{Y'} : Y'_{\mathrm{proet}} \to Y'_{\mathrm{et}}$ be the canonical morphisms of sites. Define
$$c_g(L) : Lg^*_{\mathrm{proet}}\, \nu_Y^* L \longrightarrow \nu_{Y'}^*\, Lg^*_{\mathrm{et}} L$$
by the natural transformation associated with the commutative square of site functors $\nu_Y \circ g_{\mathrm{proet}} = g_{\mathrm{et}} \circ \nu_{Y'}$. Define $c_{g'}(L)$ by the same formula on $X' \to X$.

Lemma 10.22.10 (Pullback comparison on presheaf sections). Lean: `conditional-interface:lemma:weil-a2-etale-proetale-pullback-components`

Uses: `def:weil-a2-etale-proetale-pullback-comparison`

On a pro-etale object $U' \to Y'$, both sides of $c_g(L)$ are induced by the same diagram of sections of L over etale objects receiving a map from U'.

Proof. Uses: `def:weil-a2-etale-proetale-pullback-comparison`

Expand inverse image on presheaves as the colimit over objects mapping to the image of U'. The equality of the two composite site functors gives the same indexing category and the same transition maps. The component of $c_g(L)$ is induced by the identity functor on that indexing category. □

Proposition 10.22.11 (Etale-to-pro-etale pullback comparison is an isomorphism). Lean: `conditional-interface:prop:weil-a2-etale-proetale-pullback-comparison-open`

Uses: `def:weil-a2-etale-proetale-pullback-comparison`

For every bounded-below $L \in D^+(Y_{\mathrm{et}}, \Lambda_n)$, the maps $c_g(L)$ and $c_{g'}(L)$ are natural isomorphisms.

Proof. Uses: `lemma:weil-a2-etale-proetale-pullback-components`

OPEN obligation A2-ETALE-PROETALE-PULLBACK. Prove exactness of the inverse-image functors, identify their derived functors with termwise inverse image, and sheafify the componentwise presheaf identification. Naturality is the square induced by a morphism $L \to L'$; the identity and composition equations follow from equality of the corresponding composite site functors. □

Definition 10.22.12 (Etale-to-pro-etale direct-image comparison). Lean: `conditional-interface:def:weil-a2-etale-proetale-direct-image-comparison`

Uses: `def:weil-a2-proper-base-change-square`

For $L \in D^+(X_{\mathrm{et}}, \Lambda_n)$, define

$$d_f(L) : \nu_Y^* \, Rf_{\mathrm{et},*} L \longrightarrow Rf_{\mathrm{proet},*} \, \nu_X^* L$$

as the derived natural transformation induced by the equality of the underived direct-image functors on objects pulled back from the etale site. Define $d_{f'}(L')$ analogously.

Lemma 10.22.13 (Direct-image comparison on injective resolutions). Lean: `conditional-interface:lemma:weil-a2-etale-proetale-direct-image-resolution`

Uses: `def:weil-a2-etale-proetale-direct-image-comparison`

If $L \to I^\bullet$ is a bounded-below injective resolution on X_{et}, then $d_f(L)$ is represented by the termwise maps

$$\nu_Y^* f_{\mathrm{et},*} I^q \longrightarrow f_{\mathrm{proet},*} \nu_X^* I^q.$$

Proof. Uses: `def:weil-a2-etale-proetale-direct-image-comparison`

The derived direct image of a bounded-below complex is computed by a bounded-below injective resolution. Apply the defining underived natural transformation in every degree. Naturality of that transformation makes the degreewise maps commute with the differentials. □

Proposition 10.22.14 (Etale-to-pro-etale direct-image comparison is an isomorphism). Lean: `conditional-interface:prop:weil-a2-etale-proetale-direct-image-comparison-open`

Uses: `def:weil-a2-etale-proetale-direct-image-comparison`

For every bounded-below $L \in D^+(X_{\mathrm{et}}, \Lambda_n)$, the maps $d_f(L)$ and $d_{f'}(L)$ are natural isomorphisms.

Proof. Uses: `lemma:weil-a2-etale-proetale-direct-image-resolution`

OPEN obligation A2-ETALE-PROETALE-DIRECT-IMAGE. Prove that ν_X^* sends the chosen etale injective resolution to a complex acyclic for the pro-etale direct image, compare sections on the etale and pro-etale localization sites, and show each displayed resolution-level map is a quasi-isomorphism. Bounded-below is used to justify the injective resolution calculation and cannot be omitted. □

Definition 10.22.15 (Finite-coefficient etale proper-base-change map). Lean: `conditional-interface:def:weil-a2-finite-etale-proper-base-change-map`

Uses: `def:weil-a2-proper-base-change-square`

For $L_n \in D^+(X_{\mathrm{et}}, \Lambda_n)$, define the ordinary etale base-change transformation

$$b_n : Lg_{\mathrm{et}}^* Rf_{\mathrm{et},*} L_n \longrightarrow Rf'_{\mathrm{et},*} L(g')_{\mathrm{et}}^* L_n$$

from the adjunction unit $L_n \to R(g')_{\mathrm{et},*} L(g')_{\mathrm{et}}^* L_n$, the cartesian identity $fg' = gf'$, and the adjunction counit for g.

Proposition 10.22.16 (Finite-coefficient etale proper base change)**.** *Lean:* `conditional-interface:prop:weil-a2-finite-etale-proper-base-change-open`

Uses: `def:weil-a2-finite-etale-proper-base-change-map`

If f is proper, then b_n is an isomorphism for every bounded-below constructible L_n.

Proof. Uses: `def:weil-a2-finite-etale-proper-base-change-map`

OPEN obligation A2-FINITE-ETALE-PROPER-BC. This is the finite-coefficient etale proper-base-change theorem. It must be expanded in the finite-torsion geometric module into: reduction to a geometric stalk; strict henselian localization; comparison of the proper fiber with the localized total space; proper cohomological descent; and the final stalkwise isomorphism criterion. The present A2 segment records the exact map and hypotheses but does not treat that theorem as a mathlib leaf. □

Definition 10.22.17 (Transported finite-level pro-etale base-change map)**.** *Lean:* `conditional-interface:def:weil-a2-transported-finite-level-base-change-map`

Uses: `def:weil-a2-etale-proetale-pullback-comparison`, `def:weil-a2-etale-proetale-direct-image-comparison`, `def:weil-a2-finite-etale-proper-base-change-map`, `def:weil-a2-etale-origin-datum`

For an etale-origin isomorphism $\alpha_n : \nu_X^* L_n \xrightarrow{\sim} K_n$, define $\widetilde{\beta}_n$ as the composite

$$\begin{aligned} Lg^*_{\mathrm{proet}} Rf_{\mathrm{proet},*} K_n &\xrightarrow[\alpha_n]{\sim} Lg^*_{\mathrm{proet}} Rf_{\mathrm{proet},*} \nu_X^* L_n \\ &\xleftarrow[d_f(L_n)]{\sim} Lg^*_{\mathrm{proet}} \nu_Y^* Rf_{\mathrm{et},*} L_n \\ &\xrightarrow[c_g]{\sim} \nu_{Y'}^* Lg^*_{\mathrm{et}} Rf_{\mathrm{et},*} L_n \\ &\xrightarrow{\nu_{Y'}^* b_n} \nu_{Y'}^* Rf'_{\mathrm{et},*} L(g')^*_{\mathrm{et}} L_n \\ &\xrightarrow[d_{f'},\, c_{g'},\, \alpha_n]{\sim} Rf'_{\mathrm{proet},*} L(g')^*_{\mathrm{proet}} K_n. \end{aligned}$$

Lemma 10.22.18 (The transported map is the canonical finite-level map)**.** *Lean:* `conditional-interface:lemma:weil-a2-transported-base-change-equals-canonical`

Uses: `def:weil-a2-transported-finite-level-base-change-map`, `def:weil-a2-proper-complete-base-change-datum`

The composite $\widetilde{\beta}_n$ equals the finite-level base-change transformation β_n retained in the proper complete-base-change datum.

Proof. Uses: `def:weil-a2-etale-proetale-pullback-comparison`, `def:weil-a2-etale-proetale-direct-image-comparison`, `def:weil-a2-finite-etale-proper-base-change-map`

Expand both transformations into units, counits, and the cartesian equality $fg' = gf'$. The two composites have the same unit at L_n, the same exchange through the cartesian equality, and the same counit on Y'. Naturality of $c_g, d_f, c_{g'}, d_{f'}$ with respect to α_n makes the four comparison squares commute. Pasting these squares identifies the two outer composites. □

Proposition 10.22.19 (Finite-level proper-etale base change)**.** *Lean:* `conditional-interface:prop:weil-a2-finite-level-proper-etale-base-change-open`

Uses: `def:weil-a2-proper-base-change-square`, `def:weil-a2-finite-level-etale-origin-realization`, `def:weil-a2-perfect-finite-level-coefficient-datum`

For each n, if f is proper and K_n has a finite-level etale-origin realization, then the map

$$Lg^* Rf_* K_n \longrightarrow Rf'_* L(g')^* K_n$$

is an isomorphism.

Proof. Uses: `lemma:weil-a2-finite-level-etale-origin-source`, `prop:weil-a2-etale-proetale-pullback-comparison-open`, `prop:weil-a2-etale-proetale-direct-image-comparison-open`, `prop:weil-a2-finite-etale-proper-base-change-open`, `lemma:weil-a2-transported-base-change-equals-canonical`

The pullback and direct-image comparison propositions make every comparison arrow in the definition of $\widetilde{\beta}_n$ an isomorphism. The etale proper-base-change proposition makes its central arrow $\nu_{Y'}^* b_n$ an isomorphism. Therefore the composite $\widetilde{\beta}_n$ is an isomorphism. The equality lemma identifies this composite with β_n. □

Proposition 10.22.20 (Proper complete base change, Tag 09C9 interface)**.** Lean: `conditional-interface:prop:weil-a2-proper-complete-base-change-open`

Uses: `def:weil-a2-proper-base-change-square`, `def:weil-a2-proper-complete-base-change-datum`, `def:weil-a2-proetale-proper-base-change-map`

For a proper base-change square, the map

$$\beta_K^{\mathrm{proet}} : Lg^* Rf_* K \xrightarrow{\sim} Rf'_* L(g')^* K$$

is an isomorphism whenever K carries a proper complete-base-change datum.

Proof. Uses: `prop:weil-a2-finite-level-proper-etale-base-change-open`, `prop:weil-a2-finite-reductions-conservative-open`, `prop:weil-a2-completion-idempotent-open`

The n-th reduction of β_K^{proet} is β_n: this is the defining component equation in the proper complete-base-change datum, and finite reduction commutes with the two cone maps by the normalization equations. The preceding proposition makes every β_n an isomorphism. Thus the two sides are complete, and conservativity of all finite reductions forces β_K^{proet} to be an isomorphism. The only unresolved leaves are the explicitly named etale comparison, finite-level proper base change, and reduction conservativity obligations; this proposition does not silently import Tag 09C9 as a foundation axiom. □

10.23 Coefficient-change compatibility interfaces

Definition 10.23.1 ($\mathbf{Q}_\ell$-to-$\overline{\mathbf{Q}}_\ell$ functor)**.** Lean: `conditional-interface:def:weil-a2-ql-to-qbar`

Uses: `def:weil-a2-dbc-ql`, `def:weil-a2-e-to-qbar`

At the stage $E = \mathbf{Q}_\ell$, define

$$(-)_{\overline{\mathbf{Q}}_\ell} : D_c^b(X, \mathbf{Q}_\ell) \longrightarrow D_c^b(X, \overline{\mathbf{Q}}_\ell)$$

to be $\iota_{\mathbf{Q}_\ell}$.

Proposition 10.23.2 (The algebraic-closure extension is a functor)**.** Lean: `conditional-interface:prop:weil-a2-ql-to-qbar-functor`

Uses: `def:weil-a2-ql-to-qbar`

The preceding assignment preserves identities and composition.

Proof. Uses: `prop:weil-a2-e-to-qbar-functor`

It is the finite-stage inclusion functor at $E = \mathbf{Q}_\ell$. □

Definition 10.23.3 (Reduction-scalar-extension comparison)**.** Lean: `conditional-interface:def:weil-a2-reduction-scalar-extension-comparison`

Uses: `def:weil-a2-finite-reduction`, `def:weil-a2-integral-scalar-extension`

For $E \subseteq F$, define the comparison candidate

$$\mathrm{Red}_n(K) \otimes^{\mathbf{L}}_{\mathcal{O}_{E/\ell^n}} \mathcal{O}_F/\ell^n \longrightarrow \mathrm{Red}_n\big(K \otimes^{\mathbf{L}}_{\mathcal{O}_E} \mathcal{O}_F\big)$$

by associating the three K-flat tensor factors.

Proposition 10.23.4 (Reduction commutes with scalar extension)**.** Lean: `conditional-interface:prop:weil-a2-reduction-scalar-extension-open`

Uses: `def:weil-a2-reduction-scalar-extension-comparison`

The comparison candidate is an isomorphism and is natural in K.

Proof. Uses: `lemma:weil-a2-finite-reduction-package-independent`, `prop:weil-a2-completed-monoidal-coherence-open`

OPEN obligation A2-REDUCTION-SCALAR-EXTENSION. Choose one common K-flat model, expand both sides as a total tensor with L_n and $\mathcal{O}_F$, and use the tensor associator. Prove the associator respects the null-homotopy of ℓ^n, transport across package comparisons, and verify naturality. □

Definition 10.23.5 (Pullback-coefficient comparison)**.** Lean: `conditional-interface:def:weil-a2-pullback-coefficient-comparison`

Uses: `def:weil-a2-fixed-locally-closed-pullbacks`, `def:weil-a2-finite-extension-transition`

For a locally closed $i : Z \hookrightarrow X$ and $E \subseteq F$, define

$$i^* \operatorname{Ext}_{F/E}(K) \longrightarrow \operatorname{Ext}_{F/E}(i^* K)$$

from the tensor comparison retained in the pullback package.

Proposition 10.23.6 (Pullback commutes with coefficient extension)**.** Lean: `conditional-interface:prop:weil-a2-pullback-coefficient-extension-open`

Uses: `def:weil-a2-pullback-coefficient-comparison`

The comparison is a natural isomorphism, compatible with $E \subseteq F \subseteq G$.

Proof. Uses: `prop:weil-a2-locally-closed-pullback-package-open,prop:weil-a2-finite-extension-transition-cocycle-open`

OPEN obligation A2-PULLBACK-COEFFICIENT-COMPATIBILITY. Expand the comparison as pullback of a derived tensor, apply the pullback tensor isomorphism, and identify the pulled-back coefficient sheaf with the coefficient sheaf on Z. Naturality is the morphism-level tensor square; tower compatibility is the associator pentagon. □

Definition 10.23.7 (Six-operation coefficient-compatibility target)**.** Lean: `conditional-interface:def:weil-a2-six-operation-coefficient-target`

Uses: `def:weil-a2-finite-extension-transition,def:weil-a2-finite-reduction`

For each later operation

$$\Phi \in \{f^*, Rf_*, Rf_!, Rf^!, \otimes^{\mathbf{L}}, \mathbf{R}\mathcal{H}om\},$$

define its coefficient-compatibility target to consist of natural comparison maps

$$\mathrm{Ext}_{F/E}\,\Phi_E \longrightarrow \Phi_F\,\mathrm{Ext}_{F/E}$$

and analogous finite-reduction comparisons, together with identity, composition, adjunction, projection-formula, and base-change coherence equations.

Proposition 10.23.8 (Status of six-operation coefficient compatibility)**.** Lean: `conditional-interface:prop:weil-a2-six-operation-coefficient-status-open`

Uses: `def:weil-a2-six-operation-coefficient-target`

The target is explicit, but its fields for operations constructed in A1, A3, and A4 remain unpopulated.

Proof. Uses: `prop:weil-a2-reduction-scalar-extension-open,prop:weil-a2-pullback-coefficient-extension-open`

OPEN obligation A2-SIX-OP-COEFFICIENT-COMPATIBILITY. After each operation is constructed, define its comparison at every finite level, prove it is an isomorphism, lift through completion, extend through E-linearization and the filtered two-colimit, and verify every listed coherence equation separately. This interface is not cited as proved. □

> **Formalization note.** Pinned mathlib commit: `81a5d257`. The additive coefficient definition is `AlgebraicGeometry.Scheme.ellAdicSheaf` in `Mathlib/AlgebraicGeometry/Sites/ElladicCohomology.lean`, lines 52–59. It is `sheafPushforwardContinuous` applied to `continuousMapPresheafAb`, not a sheafification. The object and map calculations use `continuousMapPresheaf_obj`, `continuousMapPresheaf_map`, and `continuousMapPresheafAbForgetIso`. The exact abelian instance is `SheafOfModules.instAbelian` in `Mathlib/Algebra/Category/ModuleCat/Sheaf/Abelian.lean`, lines 35–42, under `HasSheafify` and `WEqualsLocallyBijective`.

> **Status.** No Artin–Rees category and no AR-null quotient occurs in this public construction. The non-mathlib existence theorems for functorial K-flat and K-injective replacements, derived-limit independence, completed monoidal coherence, locally closed pullback, constructibility under coefficient change, and the two-colimit cocycle remain explicit obligations and cannot be consumed as proved inputs by later modules.

Chapter 11

Stacks route: comparison through the strictly henselian projective line

Status. This file is the S0–S4 core of the A3 DAG `research/a3_stacks_proper_base_change_dag.md`. The coefficient sheaf is a torsion sheaf on the small etale site. A Stacks tag identifies the source paragraph to be audited; it is never used as an admitted theorem.

11.1 S0. Comparison data and injective acyclicity

Definition 11.1.1 (Cartesian square and chosen pullback identification)**.** Uses:

Lean: `conditional-interface:def:weil-pbc-cartesian-square`

For schemes and morphisms

$$\begin{array}{ccc} X' & \xrightarrow{g'} & X \\ {\scriptstyle f'}\downarrow & & \downarrow{\scriptstyle f} \\ Y' & \xrightarrow{g} & Y, \end{array}$$

we write $\square = (f, f', g, g', \alpha)$ when $\alpha : X' \xrightarrow{\sim} X \times_Y Y'$ is a specified isomorphism and $f \circ g' = g \circ f'$ as maps to Y. The notation $g'^{-1}F$ always means inverse image using this specified identification.

Definition 11.1.2 (Underived exchange map)**.** Lean: `conditional-interface:def:weil-pbc-underived-exchange`

Uses: `def:weil-pbc-cartesian-square`

Let F be a sheaf of abelian groups on X_{et}. For an etale object $V \to Y'$, put $V_X := V \times_{Y'} X'$. A section $s \in (g^{-1}f_*F)(V)$ is, by the inverse-image sheaf construction, represented after an etale cover $\{V_i \to V\}$ by sections $s_i \in (f_*F)(V_i \times_{Y'} Y)$ satisfying $s_i|_{V_i \times_V V_j} = s_j|_{V_i \times_V V_j}$ for every pair (i, j). Pulling s_i along $V_i \times_Y X \to X$ and then along $V_i \times_{Y'} X' \to V_i \times_Y X$ gives sections of $g'^{-1}F$ on $V_{i,X}$. Their equality on overlaps is preserved by inverse image, hence they glue to a section of $f'_*g'^{-1}F$ on V. This construction is independent of the representing cover because it is the composite of the adjunction unit $g^{-1}f_*F \to f'_*g'^{-1}F$ and the counit for the chosen pullback. Denote the resulting map by

$$\mathrm{Ex}_{\square,F} : g^{-1}f_*F \longrightarrow f'_*g'^{-1}F.$$

Lemma 11.1.3 (Sectionwise exchange is natural)**.** *Lean:* `conditional-interface:lemma:weil-pbc-underived-exchange-naturality`

Uses: `def:weil-pbc-underived-exchange`

For every morphism $u : F \to G$ the square

$$\begin{array}{ccc} g^{-1}f_*F & \xrightarrow{\mathrm{Ex}_{\square,F}} & f'_*g'^{-1}F \\ {\scriptstyle g^{-1}f_*u}\downarrow & & \downarrow{\scriptstyle f'_*g'^{-1}u} \\ g^{-1}f_*G & \xrightarrow{\mathrm{Ex}_{\square,G}} & f'_*g'^{-1}G \end{array}$$

commutes on every etale object of Y'.

Proof. Uses: `def:weil-pbc-underived-exchange`

On a representative section s_i, both paths first apply u and then pull back along the same two maps. Since inverse image is a functor, the two resulting sections of $g'^{-1}G$ are equal on each V_i and therefore after descent on V. Equality for all V is equality of sheaf morphisms. □

Lemma 11.1.4 (Finite refinement of a surjective etale morphism)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-pbc-finite-refinement-etale-cover`

Let $U \to T$ be surjective and etale, with T quasi-compact and quasi-separated. There is a finite, finitely presented, surjective morphism $r : T' \to T$ and a finite open covering $T' = \bigcup_a W_a$ such that every $W_a \to T$ factors through $U \to T$.

Proof. Uses:

Choose finitely many affine opens $U_a \subset U$ whose images cover T and replace U by their disjoint union. The resulting map is quasi-compact, separated, etale, and has a finite upper bound d on the cardinalities of its geometric fibres. We induct on d. If $d = 1$, the map is radicial and etale, hence an open immersion; surjectivity makes it an isomorphism.

For $d > 1$, apply the affine form of Zariski's Main Theorem to the separated quasi-finite map $U \to T$: adjoining to $\mathcal{O}_T$ the integral elements of the quasi-coherent algebra $u_*\mathcal{O}_U$ gives an integral $\pi : \widetilde{T} \to T$ and a quasi-compact open immersion $j : U \hookrightarrow \widetilde{T}$. Replace $\widetilde{T}$ by the scheme-theoretic closure of U, so U is schematically dense. The diagonal section gives an open-and-closed decomposition

$$U \times_T \widetilde{T} = U \amalg W.$$

The image V of $W \to \widetilde{T}$ is open and contains $\widetilde{T} \setminus U$. Over a point of the dense open U, the fibre of $W \to V$ is the fibre of $U \to T$ with the diagonal point removed, so it has fewer than d points. The number of points in a separated etale fibre cannot increase under specialization: after writing the map locally as a finite disjoint union of Jacobian etale algebra presentations, distinct idempotents at a specialization lift to distinct idempotents at a generization. Thus every geometric fibre of $W \to V$ has cardinality $< d$.

By induction there is an integral surjection $Z \to V$, locally factoring through W. Apply the same integral-closure construction to $Z \to \widetilde{T}$ and take the closure Z' of Z; then $Z' \times_{\widetilde{T}} V = Z$. If C is the reduced closed complement $\widetilde{T} \setminus V$, then $C \subset U$, and $Z' \amalg C \to T$ is integral and surjective. Near a point of Z it factors through $W \to U$; near a point of $Z' \setminus Z$ it factors through $C \subset U$. Hence it locally factors through the original cover.

Finally write this integral morphism as the inverse limit of its finite finitely presented T-subalgebras. Choose finitely many affine opens on which the factorization through U is defined. Each open, each map to U, the equalities over T, and surjectivity descend to a common finite stage: the first three assertions are finite-presentation data, while failure of surjectivity has quasi-compact open image and would persist after pullback to the limit. That common stage is the required T'. This is the construction located at Tag `09Z0`, with the degree induction and final finite stage made explicit. □

Lemma 11.1.5 (Degree-zero comparison over a strictly henselian base)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-pbc-proper-strict-henselian-h0`

Let $(A, \mathfrak{m})$ be strictly henselian local, let $h : T \to \operatorname{Spec} A$ be proper, and let T_0 be the closed geometric fibre. For every etale sheaf of sets E on T, restriction is a bijection

$$\Gamma(T, E) \xrightarrow{\sim} \Gamma(T_0, E|_{T_0}).$$

The bijection is functorial in E and in finite morphisms over T.

Proof. Uses: `lemma:weil-pbc-finite-refinement-etale-cover`

We first prove the corresponding assertion for Zariski sheaves, uniformly after every finite $T' \to T$. The space T' is spectral and $T' \to \operatorname{Spec} A$ is proper. For an irreducible closed reduced $C \subset T'$, Stein factorization gives

$$C \longrightarrow \operatorname{Spec} B \longrightarrow \operatorname{Spec} A,$$

where $A \to B$ is integral and the first map has geometrically connected fibres. The pair $(B, \mathfrak{m}B)$ is henselian: an idempotent of a finite B-algebra modulo $\mathfrak{m}$ satisfies $e^2 - e = 0$, and the simple roots $0, 1$ lift uniquely by the henselian property of A after restriction of scalars along the integral algebra B. Consequently the closed subset $\operatorname{Spec} B \cap V(\mathfrak{m})$ is connected and nonempty. Its inverse image $C \cap T_0'$ is connected because the map is closed, surjective onto its image, and has connected fibres.

For completeness, let G be a Zariski sheaf on T'. Every point specializes to T_0', so a global section is determined by its restriction. For existence, write G as the filtered union of subsheaves generated by finitely many sections on quasi-compact opens. Such a finitely generated subsheaf embeds into a finite product $\prod_b (i_b)_* \underline{M_b}$, where $i_b : C_b \hookrightarrow T'$ is closed and M_b is a finite set: take the finitely many values of the generators and separate their equalizer loci. Sections of $(i_b)_* \underline{M_b}$ are locally constant maps $C_b \to M_b$. Connectedness of every $C \cap T_0'$ says

that the connected components of C_b and $C_b \cap T_0'$ correspond, so such maps extend uniquely. Finite products, intersections with the embedded subsheaf, and filtered colimits preserve this equality of section sets. Thus $\Gamma(T', G) = \Gamma(T_0', G|)$ for every Zariski sheaf and every finite $T' \to T$.

Now let E be an etale sheaf. The map from the restriction to the Zariski site followed by closed-fibre inverse image to the Zariski restriction of $E|_{T_0}$ is stalkwise injective. Indeed, an elementary etale neighbourhood $V \to T$ whose closed fibre has a local section has open image $V_0 \subset T$, and $V \to V_0$ is an etale cover; hence

$$E(V_0) \longrightarrow E(V) \rightrightarrows E(V \times_{V_0} V)$$

is an equalizer. Taking the colimit over such neighbourhoods proves the stalkwise injection. The Zariski result therefore proves injectivity of $\Gamma(T, E) \to \Gamma(T_0, E|)$, and the same argument applies after every finite $T' \to T$.

For surjectivity, take $t_0 \in \Gamma(T_0, E|)$. By the definition of inverse image and sheafification there are finitely many etale maps $V_a \to T_0$ covering T_0, etale $U_a \to T$, maps $V_a \to U_a \times_T T_0$, and sections $s_a \in E(U_a)$ representing $t_0|_{V_a}$. Shrink U_a to the open image of V_a in $U_a \times_T T_0$ and then in U_a; the sheaf equalizer shows that $s_a|_{U_a \times_T T_0} = t_0|_{U_a \times_T T_0}$. Every nonempty closed subset of T meets T_0 by the connectedness argument above, so $\coprod_a U_a \to T$ is surjective.

Choose the finite surjective refinement $r : T' \to T$ of the preceding lemma and opens $W_a \subset T'$ mapping to U_a. Pulling back s_a gives local sections on W_a. On $W_a \cap W_b$ their closed-fibre restrictions both equal $r_0^{-1} t_0$; the already proved injectivity for the finite scheme $T' \to T$ makes them equal. They glue to $s' \in \Gamma(T', r^{-1}E)$. The two pullbacks of s' to $T' \times_T T'$ have the same closed-fibre restriction, and injectivity for the finite morphism $T' \times_T T' \to T$ makes them equal.

It remains to spell out descent along r, which need not be etale. At a geometric point $\bar{x} \to T$, choose a geometric $\bar{x}' \to T'$ above it. The germ of s' at $\bar{x}'$ is an element of the stalk $E_{\bar{x}}$. The equalizer equality on $T' \times_T T'$ makes this element independent of $\bar{x}'$. Represent it on an etale neighbourhood of $\bar{x}$ in T. After shrinking, its pullback equals s' near every point over $\bar{x}$; finiteness of r makes that inverse image quasi-compact, so one shrinking works for all of them. These local descended sections agree on overlaps because equality can be checked at geometric stalks, and hence glue to $s \in \Gamma(T, E)$. Its restriction is t_0 by injectivity on T_0. All pullbacks used are the functorial sheafification units, so the argument also proves functoriality in E and compatibility with finite pullback. The source locators are Tags `09ZG` and `09ZF`; neither is used as an unexpanded conclusion. □

Lemma 11.1.6 (Underived proper exchange is an isomorphism)**.** *Lean:* `conditional-interface:lemma:weil-pbc-proper-underived-exchange`

Uses: `def:weil-pbc-underived-exchange`

If f is proper, then $\mathrm{Ex}_{\square,F} : g^{-1} f_* F \to f'_* g'^{-1} F$ is an isomorphism for every sheaf F.

Proof. Uses: `lemma:weil-pbc-underived-exchange-naturality,lemma:weil-pbc-proper-strict-henselian-h0`

Fix a geometric point $\bar{y}' \to Y'$ and let $\bar{y} \to Y$ be its composite with g. Let $\mathcal{N}_{\bar{y}}^{\mathrm{aff}}$ be the category of pointed affine etale neighbourhoods $(V, \bar{v})$ of $\bar{y}$. It is cofiltered: an affine open containing the lifted point in a fibre product refines two objects, and an affine neighbourhood of the equalizer of two maps makes parallel maps equal. It is cofinal among all pointed etale neighbourhoods because every lifted point has an affine open neighbourhood. Thus

$$(f_* F)_{\bar{y}} = \varinjlim_{(V,\bar{v})} \Gamma(X \times_Y V, F|).$$

Writing $V = \operatorname{Spec} A_V$, the strict henselization is $Y_{(\bar{y})} = \operatorname{Spec}(\varinjlim A_V)$ and $X_{(\bar{y})} = X \times_Y Y_{(\bar{y})} = \varprojlim (X \times_Y V)$. The natural map from the displayed colimit to $\Gamma(X_{(\bar{y})}, F|)$ is bijective. For surjectivity, represent a section on the limit by sections on a finite cover by quasi-compact etale objects. Each object, each covering map, and each section representative descends to a stage because etale morphisms are locally of finite presentation. The finitely many equality conditions on the pairwise fibre products descend after enlarging that stage. The sheaf equalizer then glues the descended representatives. For injectivity, represent two sections at one stage; equality on the limit is witnessed on a finite etale cover, and the cover together with its finitely many equality witnesses descends to a later stage. Hence the two representatives agree in the colimit.

The base $Y_{(\bar{y})}$ is strictly henselian local and $X_{(\bar{y})}$ is proper over it. The preceding lemma identifies its sections with those on the closed geometric fibre:

$$(f_* F)_{\bar{y}} \xrightarrow{\sim} \Gamma(X_{(\bar{y})}, F|) \xrightarrow{\sim} \Gamma(X_{\bar{y}}, F|_{X_{\bar{y}}}).$$

The identical construction for f' gives the target stalk. Since $\bar{y}$ is the composite of the same $\operatorname{Spec} \Omega \to Y'$ with g, the Cartesian isomorphism identifies $X_{\bar{y}}$ and $X'_{\bar{y}'}$ with $X \times_Y \operatorname{Spec} \Omega$.

Finally, a germ represented by $s \in \Gamma(X \times_Y V, F|)$ travels around the comparison by pulling s first to $X' \times_{Y'} (V \times_Y Y')$ and then to the geometric fibre. The two stalk identifications above pull s directly to that same fibre. Hence the fibre bijection is the stalk of $\mathrm{Ex}_{\square,F}$, not an unnamed isomorphism. Geometric stalks are conservative, so the exchange map is an isomorphism. This closes the compatibility omission at Tag `0A3U`. □

Definition 11.1.7 (Derived exchange map). Lean: `conditional-interface:def:weil-pbc-derived-exchange`

Uses: `def:weil-pbc-underived-exchange`

For $K \in D^+(X_{\mathrm{et}})$ choose an injective resolution $K \xrightarrow{\simeq} I^\bullet$. Apply $\mathrm{Ex}_{\square,I^r}$ in every degree and use the naturality lemma to obtain a morphism of complexes

$$g^{-1} f_* I^\bullet \longrightarrow f'_* g'^{-1} I^\bullet.$$

Choose a K-injective replacement $g'^{-1} I^\bullet \xrightarrow{\simeq} J^\bullet$ in the Grothendieck category of etale abelian sheaves on X'. Compose the displayed map with $f'_* g'^{-1} I^\bullet \to f'_* J^\bullet$ and define

$$\mathrm{Ex}_{\square,K} : g^{-1} f_* I^\bullet \longrightarrow f'_* J^\bullet$$

to be its class in $D^+(Y'_{\mathrm{et}})$. The target complex is a model for $Rf'_* g'^{-1} K$, since $J^\bullet$ is K-injective; the source is a model for $g^{-1} Rf_* K$ because g^{-1} is exact. Its class in $D^+(Y'_{\mathrm{et}})$ is

$$\mathrm{Ex}_{\square,K} : g^{-1} Rf_* K \longrightarrow Rf'_* g'^{-1} K.$$

Derived Exchange Comparison Chain

Definition 11.1.8 (Derived-exchange presentation). Lean: `conditional-interface:def:weil-pbc-derived-exchange-presentation-v5`

Uses: `def:weil-pbc-derived-exchange`

For a bounded-below complex K, a presentation consists of an injective resolution $u : K \to I$, a K-injective complex J, a quasi-isomorphism $q : g'^{-1} I \to J$, and the complex map

$$E_I : g^{-1} f_* I \longrightarrow f'_* g'^{-1} I$$

obtained degreewise from the underived exchange. Write $E_{I,J} := f'_* q \, E_I$.

Definition 11.1.9 (Hom complex and K-injective test). Lean: `conditional-interface:def:weil-pbc-derived-exchange-hom-test-v5`

Uses: `def:weil-pbc-derived-exchange-presentation-v5`

For complexes A, B, let $\underline{\mathrm{Hom}}^r(A,B)$ be the degree-r graded maps, with differential $d(h) = d_B h - (-1)^r h d_A$. A complex B is K-injective when this Hom complex is acyclic for every acyclic A.

Lemma 11.1.10 (The Hom differential squares to zero). Lean: `blueprint-target:lemma:weil-pbc-derived-exchange-hom-differential-v5`

Uses: `def:weil-pbc-derived-exchange-hom-test-v5`

The displayed Hom differential satisfies $d^2 = 0$.

Proof. Uses: `def:weil-pbc-derived-exchange-hom-test-v5`

For homogeneous h of degree r, expanding $d(dh)$ gives $d_B^2 h - (-1)^r d_B h d_A - (-1)^{r+1} d_B h d_A - h d_A^2 = 0$. □

Lemma 11.1.11 (Bounded-below injectives are K-injective). Lean: `blueprint-target:lemma:weil-pbc-derived-exchange-bounded-injective-kinjective-v5`

Uses: `def:weil-pbc-derived-exchange-hom-test-v5`

Every bounded-below complex whose terms are injective is K-injective.

Proof. Uses: `def:weil-pbc-derived-exchange-hom-test-v5,lemma:weil-pbc-derived-exchange-hom-differential-v5`

After shifting the target, it suffices to treat a degree-zero Hom cocycle $f : C \to I$. Choose m with $I^n = 0$ for $n < m$, and construct maps $s^n : C^n \to I^{n-1}$ upward from $n = m$ so that

$$f^n = d s^n + s^{n+1} d. \tag{*}$$

Assume $s^m, \dots, s^n$ are known (with $s^m = 0$ at the initial step). On $B^{n+1} = \mathrm{im}(d : C^n \to C^{n+1}) = \ker d$, define

$$t(dx) := f^n(x) - d s^n(x) \in I^n.$$

This is well-defined: if $dx = 0$, acyclicity gives $x = dy$, and the cocycle equation together with (*) in degree $n-1$ gives $f^n(x) = d f^{n-1}(y) = d s^n(dy) = d s^n(x)$. Injectivity extends the map $t : B^{n+1} \to I^n$ along the monomorphism $B^{n+1} \hookrightarrow C^{n+1}$ to $s^{n+1} : C^{n+1} \to I^n$. Its defining equality is (*) in degree n. Thus $f = ds$, so every Hom cocycle is a coboundary. □

Definition 11.1.12 (Target-replacement comparison problem)**.** Lean: `conditional-interface:def:weil-pbc-derived-exchange-target-comparison-v5`

Uses: `def:weil-pbc-derived-exchange-presentation-v5`

For two target replacements $q : A \to J$ and $q' : A \to J'$ with J, J' K-injective, a comparison is a chain map $c : J \to J'$ together with a homotopy $cq \simeq q'$.

Lemma 11.1.13 (Target-replacement comparisons exist)**.** Lean: `blueprint-target:lemma:weil-pbc-derived-exchange-target-comparison-exists-v5`

Uses: `def:weil-pbc-derived-exchange-target-comparison-v5`, `def:weil-pbc-derived-exchange-hom-test-v5`

Every target-replacement comparison problem has a comparison.

Proof. Uses: `def:weil-pbc-derived-exchange-target-comparison-v5`, `def:weil-pbc-derived-exchange-hom-test-v5`

The cone $C(q)$ is acyclic because q is a quasi-isomorphism. K-injectivity of J' makes $\underline{\mathrm{Hom}}(C(q), J')$ acyclic. The cone triangle gives the exact segment

$$H^0\underline{\mathrm{Hom}}(J, J') \longrightarrow H^0\underline{\mathrm{Hom}}(A, J') \longrightarrow H^1\underline{\mathrm{Hom}}(C(q), J') = 0.$$

Thus $[q']$ lifts to $[c]$; equality $[cq] = [q']$ is exactly a homotopy $cq \simeq q'$. □

Lemma 11.1.14 (Target comparisons are unique up to homotopy)**.** Lean: `blueprint-target:lemma:weil-pbc-derived-exchange-target-comparison-unique-v5`

Uses: `def:weil-pbc-derived-exchange-target-comparison-v5`, `def:weil-pbc-derived-exchange-hom-test-v5`

Two comparisons $c, d : J \to J'$ are homotopic. Comparisons in both directions are inverse to one another in the homotopy category.

Proof. Uses: `def:weil-pbc-derived-exchange-target-comparison-v5`, `def:weil-pbc-derived-exchange-hom-test-v5`

The difference $(c - d)q$ is null-homotopic. Since $\underline{\mathrm{Hom}}(C(q), J')$ is acyclic, the cone exact segment

$$H^0\underline{\mathrm{Hom}}(C(q), J') = 0 \longrightarrow H^0\underline{\mathrm{Hom}}(J, J') \xrightarrow{q^*} H^0\underline{\mathrm{Hom}}(A, J')$$

makes q^* injective; hence $c - d$ is null-homotopic. If $c : J \to J'$ and $c' : J' \to J$ are comparisons, both $c'c$ and 1_J compare q with itself, and both cc' and $1_{J'}$ compare q' with itself. The first assertion therefore gives $c'c \simeq 1_J$ and $cc' \simeq 1_{J'}$. □

Definition 11.1.15 (Source-resolution comparison problem)**.** Lean: `conditional-interface:def:weil-pbc-derived-exchange-source-comparison-v5`

Uses: `def:weil-pbc-derived-exchange-presentation-v5`

For injective resolutions $u : K \to I$ and $u' : K \to I'$, a comparison is a chain map $a : I \to I'$ with $au \simeq u'$.

Lemma 11.1.16 (Source-resolution comparisons exist and invert)**.** Lean: `blueprint-target:lemma:weil-pbc-derived-exchange-source-comparison-v5`

Uses: `def:weil-pbc-derived-exchange-source-comparison-v5`, `def:weil-pbc-derived-exchange-hom-test-v5`

There are comparisons $a : I \to I'$ and $b : I' \to I$, unique up to homotopy, with $ba \simeq 1_I$ and $ab \simeq 1_{I'}$.

Proof. Uses: `def:weil-pbc-derived-exchange-source-comparison-v5`, `lemma:weil-pbc-derived-exchange-bounded-injective-kinjective-v5`, `lemma:weil-pbc-derived-exchange-target-comparison-exists-v5`, `lemma:weil-pbc-derived-exchange-target-comparison-unique-v5`

The resolutions are bounded below, so the preceding lemma makes I' and I K-injective. Apply Lemma 11.1.13 with $A = K$, $q = u$, $q' = u'$, $J = I$, and $J' = I'$ to obtain a. Interchanging the two resolutions gives b. Now apply Lemma 11.1.14: ba and 1_I are two comparisons of u with itself, while ab and $1_{I'}$ compare u' with itself. It gives uniqueness up to homotopy and the two displayed inverse equations. □

Lemma 11.1.17 (Exchange naturality is a homotopy square)**.** Lean: `blueprint-target:lemma:weil-pbc-derived-exchange-naturality-square-v5`

Uses: `def:weil-pbc-derived-exchange-presentation-v5`, `def:weil-pbc-derived-exchange-target-comparison-v5`

Let (I, J, q) and (I', J', q') be presentations and let $a : I \to I'$ be a source comparison. A target comparison $c : J \to J'$ for $q'g'^{-1}a$ and cq gives

$$f'_* c\, E_{I,J} \simeq E_{I',J'}\, g^{-1} f_* a.$$

Proof. Uses: def:weil-pbc-derived-exchange-presentation-v5,lemma:weil-pbc-underived-exchange-naturality,lemma:weil-pbc-derived-exchange-target-comparison-exists-v5

Underived naturality gives the strict degreewise equality $f'_* g'^{-1} a\, E_I = E_{I'} g^{-1} f_* a$. The defining homotopy $cq \simeq q' g'^{-1} a$ remains a homotopy after f'_*; compose it with E_I and substitute the strict equality. □

Definition 11.1.18 (Identity and pasted exchange presentations)**.** Lean: conditional-interface: def:weil-pbc-derived-exchange-pasting-presentations-v5

Uses: def:weil-pbc-derived-exchange-presentation-v5

For an identity or a horizontally or vertically pasted Cartesian square, use one injective resolution and form the degreewise exchange maps by their unit–pullback-identification–counit formula before selecting targets.

Lemma 11.1.19 (Identity exchange equation)**.** Lean: blueprint-target: lemma: weil-pbc-derived-exchange-identity-v5

Uses: def: weil-pbc-derived-exchange-pasting-presentations-v5

The exchange for an identity square is the identity in the derived category.

Proof. Uses: def:weil-pbc-derived-exchange-pasting-presentations-v5,lemma:weil-pbc-derived-exchange-target-comparison-unique-v5

The unit and counit for the identity pullback are identities on every sheaf term. Thus the degreewise exchange is 1; changing targets leaves its class unchanged by the target-comparison uniqueness argument above. □

Lemma 11.1.20 (Horizontal and vertical exchange pasting)**.** Lean: blueprint-target: lemma: weil-pbc-derived-exchange-pasting-v5

Uses: def: weil-pbc-derived-exchange-pasting-presentations-v5, def: weil-pbc-underived-exchange, def: weil-pbc-cartesian-square

The exchange for a horizontal or vertical pasted square equals the composite of the exchanges for its two constituent squares in the derived category.

Proof. Uses: def:weil-pbc-derived-exchange-pasting-presentations-v5,def:weil-pbc-underived-exchange,def:weil-pbc-cartesian-square,lemma:weil-pbc-derived-exchange-target-comparison-unique-v5

Before target replacement, expand the three degreewise maps as unit, pullback identification, and counit. Associativity of fibre products makes the two middle identifications equal, and the adjunction triangle identities cancel the adjacent unit–counit pairs. The remaining maps are equal as complex maps. Any different target replacements represent this same class by target-comparison uniqueness. □

Lemma 11.1.21 (Well-definedness certificate for def:weil-pbc-derived-exchange)**.** Lean: conditional-interface: lemma: weil-sf-well-defined-def-weil-pbc-derived-exchange

Uses: def: weil-pbc-derived-exchange

The construction in Definition 11.1.7 is a typed raw datum. Its theorem-valued naturality, comparison, descent, or coherence fields are isolated in this property lemma.

Proof. Uses: def:weil-pbc-derived-exchange-presentation-v5,lemma:weil-pbc-derived-exchange-target-comparison-unique-v5,lemma:weil-pbc-derived-exchange-source-comparison-v5,lemma:weil-pbc-derived-exchange-naturality-square-v5,lemma:weil-pbc-derived-exchange-identity-v5,lemma:weil-pbc-derived-exchange-pasting-v5

The presentation definition constructs the complex map. Source comparisons, the naturality homotopy square, and target-comparison uniqueness identify all selected presentations in the derived category. The identity and the two pasting equations are supplied by the final two lemmas. Hence the derived exchange construction is well-defined with all fields stated in the lemma. □

Lemma 11.1.22 (Derived exchange replacement independence)**.** Lean: conditional-interface: lemma: weil-pbc-derived-exchange-replacement

Uses: def: weil-pbc-derived-exchange

The K-injective replacement used in Definition 11.1.7 gives a well-defined derived exchange map, independent of all replacement choices.

Proof. Uses:

Given two K-injective replacements $J^\bullet$ and $J'^\bullet$, the identity of $g'^{-1} I^\bullet$ lifts to maps $J^\bullet \to J'^\bullet$ and $J'^\bullet \to J^\bullet$ because K-injective complexes are homotopically injective. Their composites are homotopic to the identities by lifting the homotopies of the replacement quasi-isomorphisms. Applying f'_* preserves chain homotopies. The two resulting maps from $g^{-1} f_* I^\bullet$ therefore define the same morphism in the derived category. A comparison of two injective resolutions of K is handled by the same lifting argument, and naturality of the underived exchange map commutes with the comparison maps. Thus the class is canonical. □

Lemma 11.1.23 (Resolution independence, identities, and pasting)**.** Lean: `conditional-interface:lemma:weil-pbc-derived-exchange-coherence`

Uses: `def:weil-pbc-derived-exchange`

The class $\mathrm{Ex}_{\square,K}$ is independent of the injective resolution and is compatible with identity squares, horizontal pasting, and vertical pasting.

Proof. Uses: `def:weil-pbc-derived-exchange`

Given two injective resolutions, the comparison theorem supplies chain maps each way, unique up to chain homotopy, whose composites are homotopic to the identities. Naturality of the degreewise exchange map sends these homotopies to homotopies after both direct images; localization therefore gives the same derived morphism. For the identity square, the unit followed by the counit is the identity sectionwise, hence degreewise. For pasted squares, expand each exchange as "unit, pullback identification, counit"; the two middle identifications are equal by associativity of fibre products, and the two adjunction triangle identities cancel the adjacent unit–counit pairs. The same calculation with the order reversed proves vertical pasting. □

Definition 11.1.24 (Proper base-change property)**.** Lean: `conditional-interface:def:weil-pbc-property`

Uses: `def:weil-pbc-derived-exchange`

For a proper morphism $f : X \to Y$, define $\mathsf{PBC}(f)$ to mean that for every morphism $g : Y' \to Y$, every associated Cartesian square $\square$, and every torsion sheaf F on X_{et}, the map $\mathrm{Ex}_{\square,F}$ is an isomorphism in the derived category. The assertion includes the specified map of Definition 11.1.7, not an unnamed isomorphism.

Definition 11.1.25 (Injective-acyclicity property)**.** Lean: `conditional-interface:def:weil-pbc-injective-acyclicity-property`

Uses: `def:weil-pbc-cartesian-square`

For a proper $f : X \to Y$, define $\mathsf{IA}(f)$ to mean: for every prime ℓ, every $g : Y' \to Y$, every square $\square$, and every injective sheaf I of $\mathbf{F}_\ell$-modules on X_{et}, the sheaf $g'^{-1}I$ is f'_*-acyclic, i.e. $R^q f'_* g'^{-1} I = 0$ for all $q > 0$.

Lemma 11.1.26 (Filtered torsion union)**.** Uses:

Lean: `conditional-interface:lemma:weil-pbc-torsion-filtered-union`

For every torsion sheaf F the inclusions $F[n] \hookrightarrow F[mn]$ form a filtered system and the canonical map $\varinjlim_n F[n] \to F$ is an isomorphism.

Proof. Uses:

The transition map sends a section killed by n to the same section, now regarded as killed by mn. The map to F is monic because a section that is zero in F is already zero in every $F[n]$. It is epic because every local section of F is killed by some positive integer and hence is locally in one of the $F[n]$; sheafification preserves this local epimorphism. □

Lemma 11.1.27 (Filtered colimits and higher direct image)**.** Uses:

Lean: `conditional-interface:lemma:weil-pbc-direct-image-filtered-colimit`

For a quasi-compact and quasi-separated morphism of schemes $h : W \to V$ and a filtered system of torsion sheaves (F_i), the canonical map

$$\varinjlim_i R^q h_* F_i \longrightarrow R^q h_* (\varinjlim_i F_i)$$

is an isomorphism for every q.

Proof. Uses:

For $q = 0$, sections on a quasi-compact etale object are the equalizer of a finite covering diagram after a finite quasi-compact refinement; filtered colimits commute with this finite limit. For $q > 0$, represent a class by a degree-q cocycle on a split etale hypercover with finite $(q+1)$-skeleton. Each of its finitely many sections is represented in some F_i, and the finite list of face-map equalities holds at one later index because the system is filtered. This proves surjectivity of $\varinjlim_i H^q(W, F_i) \to H^q(W, \varinjlim_i F_i)$.

If a class from F_i dies after the colimit, choose a finite hypercover refinement and a degree-$(q-1)$ null-homotopy on the limit. Its finitely many sections, refinement maps, and equations descend to one index $j \geq i$; there the class is already a coboundary. Thus the map is injective. The same argument applies after every quasi-compact etale base change $U \to V$, so the cohomology-sheaf description of $R^q h_*$ identifies the displayed sheaves. This is the derived filtered-colimit argument, with no unbounded injective replacement required. □

Lemma 11.1.28 (Prime-factor induction)**.** Uses:

Lean: `conditional-interface:lemma:weil-pbc-prime-factor-induction`

If a statement about torsion sheaves is stable under short exact sequences, then its verification for all $F[n]$ reduces by induction on the total number of prime factors of n to its verification for sheaves killed by one prime.

Proof. Uses:

Choose a prime $\ell \mid n$ and use the short exact sequence $0 \to F[\ell] \to F[n] \xrightarrow{\ell} \ell F[n] \to 0$. The kernel is killed by ℓ, while $\ell F[n]$ is killed by n/ℓ and therefore has fewer prime factors in its exponent. Let $C(F) = \mathrm{Cone}(\mathrm{Ex}_{\square,F})$ and use the cohomological cone convention $d_C(y,x) = (d_Y y + u(x), -d_X x)$. Applying the two derived functors to the short exact sequence gives a distinguished triangle of exchange maps and, by the octahedral axiom, a triangle

$$C(F[\ell]) \longrightarrow C(F[n]) \longrightarrow C(\ell F[n]) \xrightarrow{+1} C(F[\ell])[1].$$

The connecting map is the class of $(y,x) \mapsto y$ in the above cone convention; the sign in the last arrow is forced by $d_C^2 = 0$. Induct on $\nu(n) = \sum_\ell v_\ell(n)$. If the two end cones are acyclic, the long exact cohomology sequence of this triangle makes the middle cone acyclic. The base $\nu(n) = 1$ is the prime-torsion input. This displays the invariant and all connecting morphisms. □

Lemma 11.1.29 (Five-lemma transfer for exchange)**.** *Lean:* `conditional-interface:lemma:weil-pbc-five-lemma-transfer`

Uses: `def:weil-pbc-derived-exchange`

In a short exact sequence of torsion sheaves, if two of the three derived exchange maps are isomorphisms, then the third is an isomorphism.

Proof. Uses: `lemma:weil-pbc-prime-factor-induction`

Apply $g^{-1}Rf_*$ and $Rf'_*g'^{-1}$ to the short exact sequence and write the natural long exact cohomology sequences. Naturality of the exchange map gives a morphism between these sequences. In each degree the five-lemma applies to the five adjacent terms, with the boundary maps identified by the connecting-homomorphism construction from an injective resolution. Thus the middle exchange map is an isomorphism in every degree. □

Lemma 11.1.30 (Leray acyclicity criterion)**.** *Lean:* `conditional-interface:lemma:weil-pbc-leray-acyclicity`

Uses: `def:weil-pbc-injective-acyclicity-property`

If a complex $I^\bullet$ consists of f'_*-acyclic sheaves, then the natural map $\mathcal{H}^q(f'_*I^\bullet) \to R^q f'_* K$ is an isomorphism whenever $K \to I^\bullet$ is a quasi-isomorphism.

Proof. Uses:

Resolve K by a bounded-below Cartan–Eilenberg double complex $J^{r,s}$ with vertical differential $d_v : J^{r,s} \to J^{r+1,s}$ resolving I^s and horizontal differential $d_h : J^{r,s} \to J^{r,s+1}$ lifting the differential of $I^\bullet$. Set $d_{\mathrm{tot}} = d_v + (-1)^r d_h$. Applying f'_* and filtering the total complex by r gives

$$E_1^{r,s} = R^r f'_* I^s, \qquad E_2^{r,s} = R^r f'_* \mathcal{H}^s(K) \Longrightarrow \mathcal{H}^{r+s}(Rf'_* K).$$

The assumed acyclicity makes $E_1^{r,s} = 0$ for $r > 0$, so the edge map from the $r = 0$ column is the cohomology map of the ordinary complex $f'_* I^\bullet$. The sign $(-1)^r$ makes the square with the horizontal differential commute, and the edge morphism is therefore exactly the natural map $\mathcal{H}^q(f'_* I^\bullet) \to R^q f'_* K$. Collapse of the first column proves it is an isomorphism. □

Lemma 11.1.31 (Prime-torsion devissage)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-pbc-prime-torsion-devissage`

For a torsion sheaf F, write $F[n] = \ker(n : F \to F)$. Then $F = \varinjlim_n F[n]$; for $m \mid n$ the inclusion $F[m] \subseteq F[n]$ is monic, and if $n = \prod_i \ell_i^{e_i}$, repeated use of

$$0 \longrightarrow F[\ell_i] \longrightarrow F[n] \xrightarrow{\ell_i} \ell_i F[n] \longrightarrow 0$$

reduces assertions about $F[n]$ to assertions about ℓ_i-torsion.

Proof. Uses: `lemma:weil-pbc-torsion-filtered-union,lemma:weil-pbc-direct-image-filtered-colimit,lemma:weil-pbc-prime-factor-induction,lemma:weil-pbc-five-lemma-transfer`

The filtered union is Lemma 11.1.26. The exact sequence is the kernel–image sequence for multiplication by ℓ_i, since the kernel in $F[n]$ is $F[\ell_i]$. Lemma 11.1.28 and the five-lemma transfer then prove the assertion for every $F[n]$, and Lemma 11.1.27 passes to F. □

Lemma 11.1.32 (Equivalence of PBC and IA)**.** *Lean:* `conditional-interface:lemma:weil-pbc-property-iff-acyclicity`

Uses: `def:weil-pbc-property,def:weil-pbc-injective-acyclicity-property`

For proper f, $\mathsf{PBC}(f)$ holds if and only if $\mathsf{IA}(f)$ holds.

Proof. Uses: `def:weil-pbc-property`, `def:weil-pbc-injective-acyclicity-property`, `lemma:weil-pbc-prime-torsion-devissage`, `lemma:weil-pbc-proper-underived-exchange`, `lemma:weil-pbc-leray-acyclicity`

Assume $\mathsf{PBC}(f)$. For an injective $\mathbf{F}_\ell$-module sheaf I and a base change g, apply the isomorphism to the complex concentrated in degree zero. The source has no higher cohomology because I is injective, so the target has none; this is the required acyclicity. Conversely assume $\mathsf{IA}(f)$. Resolve any torsion F by injectives $I^\bullet$. The inverse image complex $g'^{-1}I^\bullet$ computes $Rf'_*g'^{-1}F$ because each term is acyclic. For an $\mathbf{F}_\ell$-module sheaf E, choose an injective resolution $E \to I^\bullet$ in the $\mathbf{F}_\ell$-module category. IA makes every $g'^{-1}I^r$ f'_*-acyclic, so Lemma 11.1.30 identifies the target of the derived exchange map with the cohomology of $f'_*g'^{-1}I^\bullet$. In each degree, the underived proper exchange lemma gives an isomorphism

$$g^{-1}f_*I^r \xrightarrow{\sim} f'_*g'^{-1}I^r,$$

and the replacement-independence lemma makes these maps a morphism of the chosen resolutions. Thus $\mathrm{Ex}_{\square,E}$ is an isomorphism. For $F[n]$, the prime-factor induction lemma and the five-lemma transfer propagate this isomorphism through the exact sequences $0 \to F[\ell] \to F[n] \xrightarrow{\ell} \ell F[n] \to 0$; the induction parameter is $\nu(n) = \sum_\ell v_\ell(n)$, with the prime-torsion case just proved. Finally $F = \varinjlim_n F[n]$ by the filtered torsion-union lemma. The filtered-colimit direct-image lemma applies after every quasi-compact etale base change, so it identifies both cohomology sheaves and carries the isomorphisms for the $F[n]$ to the specified map $\mathrm{Ex}_{\square,F}$. This proves PBC for every torsion sheaf and closes the converse implication. □

11.2 S1. Finite morphisms

Lemma 11.2.1 (Geometric stalk formula for finite direct image). Lean: `conditional-interface:lemma:weil-pbc-finite-stalk-formula`

Uses: `def:weil-pbc-cartesian-square`

Let $h : W \to V$ be finite and $\bar v \to V$ geometric. For every sheaf F on W_{et} there is a canonical bijection

$$(h_*F)_{\bar v} \;\cong\; \prod_{\bar w \mid \bar v} F_{\bar w},$$

where the product is finite and the index runs over geometric points of W above $\bar v$ up to the natural equivalence. For an abelian sheaf the same object is canonically the finite direct sum.

Proof. Uses:

The strict henselian local scheme $V_{(\bar v)}$ has the property that a finite V-scheme becomes a finite disjoint union of finite local schemes after base change to $V_{(\bar v)}$. Sections of F over that disjoint union are products of the sections over the components. Passing through the cofiltered system of etale neighbourhoods of $\bar v$ identifies the stalk with the displayed product. Finiteness of the fibre makes product and sum equal in the abelian category. This is the stalk calculation located at Stacks Proposition 59.55.2, Tag `03QP`; no exactness is inferred until the next lemma. □

Lemma 11.2.2 (Exactness of finite direct image). Uses:

Lean: `conditional-interface:lemma:weil-pbc-finite-direct-image-exact`

If h is finite, then h_* on etale abelian sheaves is exact.

Proof. Uses: `lemma:weil-pbc-finite-stalk-formula`

For a short exact sequence $0 \to F_1 \to F_2 \to F_3 \to 0$, apply the stalk formula at every geometric $\bar v$. A finite product of short exact sequences of abelian groups is exact, so the sequence of stalks of the direct images is exact. Geometric stalks are conservative for etale sheaves: injectivity and surjectivity can be checked after an etale neighbourhood of each geometric point and then sheafified. Hence the direct-image sequence is exact. Consequently $R^q h_* = 0$ for $q > 0$. □

Lemma 11.2.3 (Finite exchange isomorphism). Lean: `conditional-interface:lemma:weil-pbc-finite-exchange`

Uses: `def:weil-pbc-cartesian-square`

For a finite Cartesian square, the underived exchange map is an isomorphism; therefore all its derived exchange maps are isomorphisms.

Proof. Uses: `lemma:weil-pbc-finite-stalk-formula,lemma:weil-pbc-finite-direct-image-exact`

At a geometric point $\bar{y}' \to Y'$, the fibre of f' is the base change of the finite fibre of f along $\kappa(\bar{y}) \to \kappa(\bar{y}')$. The stalk formula identifies both sides with the same finite product indexed by these points, and the exchange map is the identity on each factor. Thus it is a stalkwise bijection and hence an isomorphism. Exactness from the preceding lemma means injective resolutions remain direct-image acyclic, so the same degreewise argument computes the derived map. □

Proposition 11.2.4 (Finite morphisms satisfy proper base change)**.** *Uses:* `def:weil-pbc-property`

Lean: `conditional-interface:prop:weil-pbc-finite-morphism`

Every finite morphism satisfies PBC.

Proof. Uses: `def:weil-pbc-property,lemma:weil-pbc-finite-exchange,lemma:weil-pbc-property-iff-acyclicity`

The finite exchange lemma supplies the required isomorphism for every base change and every torsion sheaf, which is exactly Definition 11.1.24. Equivalently, the vanishing in the exactness lemma gives IA and Lemma 11.1.32 gives PBC. □

11.3 S2. Composition and proper-surjective cancellation

Lemma 11.3.1 (Composition of proper maps)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-pbc-composition`

Suppose $X \xrightarrow{f} Y \xrightarrow{h} Z$ are proper and both satisfy IA*. Then hf satisfies* IA *and hence* PBC*.*

Proof. Uses: `lemma:weil-pbc-property-iff-acyclicity,lemma:weil-pbc-proper-underived-exchange`

Let $a : Z' \to Z$ be a base change, $Y' = Y \times_Z Z'$, $X' = X \times_Y Y'$, and I an injective $\mathbf{F}_\ell$-module sheaf on X. Since inverse image is exact, f_* (a right adjoint) sends injectives to injectives: for an epimorphism $A \twoheadrightarrow B$, the map $\mathrm{Hom}(B, f_*I) \to \mathrm{Hom}(A, f_*I)$ identifies with $\mathrm{Hom}(f^{-1}B, I) \to \mathrm{Hom}(f^{-1}A, I)$ and is surjective. The proper underived exchange lemma identifies $f'_* a_X^{-1} I \cong b^{-1} f_* I$, where $b : Y' \to Y$ is the base change. The Grothendieck spectral sequence has terms

$$E_2^{p,q} = R^p h'_* \, R^q f'_* \, a_X^{-1} I \Longrightarrow R^{p+q}(h'f')_* a_X^{-1} I,$$

where $a_X : X' \to X$ and $h' : Y' \to Z'$ are the base changes. The first-quadrant hypotheses are: this sheaf is h'_*-acyclic by the assumed IA(h), and $a_X^{-1} I$ is f'_*-acyclic by IA(f). Thus the $q > 0$ rows vanish, then the $p > 0$ column of the surviving row vanishes, so all terms with $p+q > 0$ vanish. The spectral sequence converges because each total degree has only finitely many pairs (p, q). This is the calculation located at Tag `0A4D`. □

Lemma 11.3.2 (Cancellation along a proper surjection)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-pbc-cancellation`

Let $f : X \to Y$ be proper and surjective and $h : Y \to Z$ proper. If f and hf satisfy IA*, then h satisfies* IA*.*

Proof. Uses: `lemma:weil-pbc-property-iff-acyclicity,lemma:weil-pbc-proper-underived-exchange`

Fix a base change $a : Z' \to Z$ and an injective $\mathbf{F}_\ell$-module sheaf I on Y. Choose a monomorphism $f^{-1}I \hookrightarrow J$ with J injective on X. Applying f_* gives $\iota : I \to f_*J$. If a germ $a \in I_{\bar{y}}$ maps to zero, choose a geometric $\bar{x} \to X$ above $\bar{y}$ (surjectivity). The canonical stalk identification $(f^{-1}I)_{\bar{x}} \cong I_{\bar{y}}$ sends a to the germ of a at $\bar{x}$; the monomorphism into J makes that germ zero. The definition of the direct-image stalk then says that a vanishes on an etale neighbourhood of $\bar{y}$, so $a = 0$. Thus ι is monic. Injectivity of I supplies a retraction $r : f_*J \to I$ with $r \circ \iota = \mathrm{id}_I$, making I a direct summand. After base change, proper underived exchange gives $b^{-1} f_* J \cong f'_* a_X^{-1} J$. The spectral sequence

$$R^p h'_* R^q f'_* a_Y^{-1} I \Longrightarrow R^{p+q}(h'f')_* a_X^{-1} I$$

is computed using the injective J and the assumed acyclicity of hf. The $q > 0$ terms vanish for J by IA(f), and total-degree vanishing for hf forces $R^p h'_* a_Y^{-1}(f_*J) = 0$ for $p > 0$. Derived direct image preserves finite direct sums and retracts, so the same vanishing holds for the direct summand $a_Y^{-1} I$. This is Tag `0A4C`; the stalk injectivity argument is written here because surjectivity is essential. □

11.4 S3. Reduction to the projective line

Lemma 11.4.1 (Target locality). *Lean:* `conditional-interface:lemma:weil-pbc-target-locality`

Uses: `def:weil-pbc-property`

For proper $f : X \to Y$, the property $\mathsf{PBC}(f)$ is local for the etale topology on Y.

Proof. Uses:

For an etale $V \to Y$ and a torsion sheaf F, sections of $R^q f_* F$ on V are the sheafification of $U \mapsto H^q(X \times_Y U, F|)$ over etale $U \to V$. This description is functorial in V and commutes with further etale localization. Therefore an exchange map is an isomorphism if and only if it is so after an etale cover of Y (and its pullback cover on Y'), because geometric stalks form a conservative family. The section description is the content located at Tag `03Q8`. □

Lemma 11.4.2 (Chow and projective factorization). *Uses:*

Lean: `conditional-interface:lemma:weil-pbc-chow-factorization`

After replacing an affine target by an etale affine cover, every proper $f : X \to Y$ admits a proper surjection $\pi : X' \to X$ and an integer n with $X' \hookrightarrow \mathbf{P}^n_Y$ a closed immersion. Moreover $X' \to Y$ factors as that closed immersion followed by $\mathbf{P}^n_Y \to Y$.

Proof. Uses: `lemma:weil-pbc-target-locality,lemma:weil-pbc-cancellation`

Chow's lemma in the form located at Tag `0202` supplies a proper surjective π for which $X' \to Y$ is projective. To make the source hypotheses explicit, first work over one affine member $Y = \operatorname{Spec} A$ of the selected etale cover. Write $A = \varinjlim_{\lambda \in \Lambda} A_\lambda$, where every A_λ is a finitely generated $\mathbf{Z}$-algebra and the transition maps are directed. Since a proper morphism is of finite presentation, a finite affine cover $X = \bigcup_{a=1}^{r} U_a$, the finitely presented algebras defining the U_a, the pairwise overlap maps, and their cocycle equations descend to one index λ_0. After increasing once to λ_1, the descended morphism $f_{\lambda_1} : X_{\lambda_1} \to \operatorname{Spec} A_{\lambda_1}$ is proper: finite presentation and separatedness descend from the finite affine data, and the valuative criterion for properness is stable under this inverse-limit presentation.

The ring A_{λ_1} is noetherian. Noetherian Chow's lemma produces a proper surjection $\pi_{\lambda_1} : X'_{\lambda_1} \to X_{\lambda_1}$ and an $f_{\lambda_1}\pi_{\lambda_1}$-ample invertible sheaf L_{λ_1}. Choose a finite affine cover $X'_{\lambda_1} = \bigcup_{b=1}^{s} W_b$. On each W_b, ampleness supplies an exponent $m_b > 0$ and finitely many global sections whose nonvanishing opens cover W_b. With $m = \prod_b m_b$, take the corresponding powers of these sections. There are then finitely many sections $s_0, \dots, s_n$ of $L_{\lambda_1}^{\otimes m}$ whose nonvanishing opens cover X'_{λ_1}. Increasing m by one further common multiple if necessary, the same finite affine-cover test for very ampleness separates the finitely many overlap generators; hence the map

$$[s_0 : \cdots : s_n] : X'_{\lambda_1} \longrightarrow \mathbf{P}^n_{A_{\lambda_1}}$$

is an immersion. It is proper, because X'_{λ_1} is proper over A_{λ_1} and projective space is separated over that base. A proper immersion is closed.

Base change along $A_{\lambda_1} \to A$ defines $X' = X'_{\lambda_1} \times_{A_{\lambda_1}} A$, π, and the closed immersion $X' \hookrightarrow \mathbf{P}^n_Y$. Properness and closed immersions are preserved by base change. Surjectivity is preserved because every geometric point of X lifts after base change from the surjective geometric fibre of π_{λ_1}. Thus the displayed index λ_1, exponent m, and integer n give the asserted single factorization. □

Lemma 11.4.3 (Projective embedding over X). *Uses:*

Lean: `conditional-interface:lemma:weil-pbc-chow-embedding-over-x`

Let $\iota_Y : X' \hookrightarrow \mathbf{P}^n_Y$ be the closed immersion and $\pi : X' \to X$ the Chow modification. Then

$$\iota_X = (\pi, \iota_Y) : X' \longrightarrow X \times_Y \mathbf{P}^n_Y = \mathbf{P}^n_X$$

is a closed immersion over X.

Proof. Uses: `lemma:weil-pbc-chow-factorization`

The product map is an immersion because ι_Y is an immersion and immersions are preserved by base change and composition with a graph. The map ι_X is proper over X: its source X' is proper over X, while $\mathbf{P}^n_X$ is separated over X, so the graph criterion makes every X-morphism from X' proper. A proper immersion is a closed immersion: properness makes its image closed after every base change, and an immersion identifies the source with a locally closed subscheme of that image; universal closedness forces the locally closed subset to be closed. The equality with $\mathbf{P}^n_X$ is the defining fibre-product identity. □

Lemma 11.4.4 (Symmetric product map)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-pbc-symmetric-product-p1`

For $n \geq 1$ the formula

$$((x_1 : y_1), \dots, (x_n : y_n)) \longmapsto [F_0 : \cdots : F_n], \qquad \prod_{i=1}^{n} (x_i t + y_i) = \sum_{j=0}^{n} F_j t^j,$$

defines a finite surjective S_n-invariant morphism $\sigma_n : (\mathbf{P}^1_S)^n \to \mathbf{P}^n_S$.

Proof. Uses: `prop:weil-pbc-finite-morphism,lemma:weil-pbc-composition,lemma:weil-pbc-cancellation`

Each F_j is the sum of monomials choosing j of the x_i and the remaining y_i; hence it is homogeneous of degree one in every pair (x_i, y_i). On the chart $y_1 \cdots y_n \neq 0$, the ratios $z_i = x_i/y_i$ give elementary symmetric polynomials, so the map is induced by the universal polynomial $\prod(T - z_i)$. On a general chart, the same calculation uses the coefficient ring localized at one nonzero F_j; the coefficients generate the unit ideal because if all F_j vanished then the homogeneous polynomial $\prod_i (x_i t + y_i)$ would be zero, forcing one factor to be $(0, 0)$, impossible in projective coordinates. The coordinate ring is integral over the symmetric subring: each z_i satisfies the monic equation $T^n - e_1 T^{n-1} + \cdots + (-1)^n e_n$, and the analogous statement holds on all charts after clearing the chosen y_i or x_i. Thus σ_n is finite. Permutation of the factors leaves every F_j unchanged, proving S_n-invariance. To make finiteness independent of the abbreviated chart calculation, let $[a_0 : \cdots : a_n]$ be a geometric point of $\mathbf{P}^n_S$. Its fibre is the set of ordered n-tuples of points of $\mathbf{P}^1$ whose linear forms multiply to $\sum a_j t^j$. Over an algebraically closed field, the factorization theorem for one-variable polynomials, applied after choosing the first nonzero coefficient to decide whether $t = \infty$ is a root, gives only finitely many such tuples (at most $n!$ after the multiplicities are fixed). Hence σ_n is quasi-finite.

The source $(\mathbf{P}^1_S)^n$ is projective over S, so σ_n is proper. On an affine target chart, the quasi-finite proper map is covered by finitely many affine opens on which the coordinate rings are integral over the target ring: on the chart where the last coefficient is invertible use the monic equations for the z_i, and on the chart where the first coefficient is invertible use the reversed monic equations for $w_i = y_i/x_i$; on an intermediate chart, intersect these two covers with the finitely many opens where a selected coefficient of the binary form is invertible. The same monic equations after multiplying by that selected coefficient show that each coordinate is integral. Integral algebras glue on overlaps because the coordinate identifications are induced by the common coefficients F_j. Thus the proper quasi-finite morphism is finite. Finally a geometric binary form of degree n factors into n linear forms, including the possible factor x for $t = \infty$, so every geometric fibre is nonempty; geometric surjectivity and properness give surjectivity. This closes the chart and gluing obligation. □

Proposition 11.4.5 (Reduction of proper base change to $\mathbf{P}^1$)**.** *Uses:* `def:weil-pbc-derived-exchange`

Lean: `conditional-interface:prop:weil-pbc-reduction-to-projective-line`

If every projection $\mathbf{P}^1_S \to S$ satisfies PBC *for every base scheme S, then every proper morphism satisfies* PBC.

Proof. Uses: `def:weil-pbc-derived-exchange,lemma:weil-pbc-target-locality,lemma:weil-pbc-chow-factorization,lemma:weil-pbc-chow-embedding-over-x,lemma:weil-pbc-symmetric-product-p1,prop:weil-pbc-finite-morphism,lemma:weil-pbc-composition,lemma:weil-pbc-cancellation`

Use target locality and the Chow factorization. The closed immersion $X' \hookrightarrow \mathbf{P}^n_Y$ is proper and finite onto its image, so finite base change plus composition reduces $X' \to Y$ to $\mathbf{P}^n_Y \to Y$. The finite surjective map σ_n and cancellation reduce this projection to $(\mathbf{P}^1_Y)^n \to Y$. Repeated base change and composition reduce the product projection to the one-dimensional projection. Finally cancellation along π transfers the property from X' and $X' \to Y$ to $X \to Y$. Every comparison used is the named exchange map of Definition 11.1.7. □

11.5 S4. The projective line over a strictly henselian local base

Definition 11.5.1 (Strictly henselian setup)**.** Uses:

Lean: `conditional-interface:def:weil-pbc-strict-henselian-setup`

Let $(A, \mathfrak{m})$ be strictly henselian local, $S = \operatorname{Spec} A$, $s = \operatorname{Spec}(A/\mathfrak{m})$, $X = \mathbf{P}^1_S$, and $X_0 = \mathbf{P}^1_s$. For a sheaf F on X_{et} write F_0 for its inverse image on $(X_0)_{\mathrm{et}}$.

Lemma 11.5.2 (Zariski restriction on a spectral space)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-pbc-spectral-restriction`

If T is spectral, $T_0 \subseteq T$ is closed, and every irreducible closed subset $C \subseteq T$ has $C \cap T_0$ connected and nonempty, then for every abelian sheaf F on T the restriction $\Gamma(T, F) \to \Gamma(T_0, F|)$ is a bijective map.

Proof. Uses:

Injectivity is pointwise. If $s \in \Gamma(T, F)$ restricts to zero, its germ vanishes at every point of T_0. The zero-germ locus is open and stable under generization. If its complement contained x, then the irreducible closed $C = \overline{\{x\}}$ would meet T_0; the specialization map from the germ at the generic point of C to the germ at x would force the latter to be zero, a contradiction.

For surjectivity, write F as the filtered colimit of subsheaves generated by finitely many sections on quasi-compact opens. Restriction to T_0 and both section functors commute with this filtered colimit, so it is enough to handle one finitely generated subsheaf. Such a subsheaf embeds into a finite product

$$F \hookrightarrow \prod_{b=1}^{r} (i_b)_* \underline{M_b},$$

with $i_b : C_b \hookrightarrow T$ closed and M_b finite: the finite germs of the chosen generators determine finitely many closed equalizer loci, and the corresponding finite value sets give the displayed embedding. If $F \subseteq G$, the injectivity already proved gives

$$\Gamma(T, F) = \Gamma(T_0, F|_{T_0}) \cap \Gamma(T, G) \subseteq \Gamma(T_0, G|_{T_0}).$$

Thus it suffices to extend sections of one finite constant pushforward.

For $i : C \hookrightarrow T$ closed, the condition on irreducible closures is inherited by C. A section of $\underline{M}$ on $C \cap T_0$ is constant on each connected component. Every component of C meets its closed subset, and that intersection is connected by hypothesis, so the same constant gives a unique section on the component. These componentwise sections agree on quasi-compact overlaps and glue by the sheaf equalizer; pushing forward gives the extension on T. Finite products, subobjects, and the filtered colimit then give an extension for F. The stalkwise injectivity proves uniqueness, so restriction is bijective. This is the expanded spectral argument at Tag `09ZG`; no etale lifting is used. □

Lemma 11.5.3 (Stein connectedness for $\mathbf{P}^1_A$)**.** Lean: `conditional-interface:lemma:weil-pbc-stein-connectedness`

Uses: `def:weil-pbc-strict-henselian-setup`

Every irreducible closed subset of X meets X_0 in a connected nonempty set. Consequently $\Gamma(X, F) \xrightarrow{\sim} \Gamma(X_0, F_0)$ for every Zariski sheaf F.

Proof. Uses: `lemma:weil-pbc-spectral-restriction`

Fix an irreducible closed subset and give it the reduced closed-subscheme structure $C \subseteq X$. The map $C \to S$ is proper. Its Stein factorization gives $C \xrightarrow{u} \operatorname{Spec} B \xrightarrow{v} S$, with $A \to B$ integral and u having geometrically connected fibres. Since $(A, \mathfrak{m})$ is henselian, $(B, \mathfrak{m}B)$ is a henselian pair. The image $D = v(\operatorname{Spec} B)$ is irreducible and closed because $C \to S$ is proper. Henselian connectedness makes $D \cap V(\mathfrak{m})$ connected and nonempty. The map $C \cap X_0 \to D \cap V(\mathfrak{m})$ is closed and surjective with connected fibres, so the closed-surjection connected-fibre criterion makes $C \cap X_0$ connected. This expands Tag `0A0B`; the final equality of sections is Lemma 11.5.2. □

Definition 11.5.4 (Pullback of a section along a finite morphism)**.** Uses:

Lean: `conditional-interface:def:weil-pbc-finite-section-pullback`

Let $u : T' \to T$ be finite and F an etale sheaf on T. For an etale $V \to T$ and $s \in F(V)$, define

$$u_V^\sharp(s) \in (u^{-1}F)(V \times_T T')$$

to be the image of s under the unit from the presheaf inverse image to its sheafification.

Lemma 11.5.5 (Finite pullback identity)**.** Lean: `conditional-interface:lemma:weil-pbc-finite-pullback-identity`

Uses: `def:weil-pbc-finite-section-pullback`

For $u = \mathrm{id}_T$, $u_V^\sharp(s) = s$ under the canonical identity $V \times_T T = V$.

Proof. Uses:

The presheaf inverse image along the identity is the same presheaf, and its sheafification unit is the identity on an already sheafified section. □

Lemma 11.5.6 (Finite pullback composition)**.** Lean: `conditional-interface:lemma:weil-pbc-finite-pullback-composition`

Uses: `def:weil-pbc-finite-section-pullback`

For finite $T'' \xrightarrow{v} T' \xrightarrow{u} T$, $(uv)_V^\sharp(s) = v^\sharp_{V \times_T T'}(u_V^\sharp(s))$ under the associativity isomorphism of fibre products.

Proof. Uses: `lemma:weil-pbc-finite-pullback-identity`

Both sections are the image of the same presheaf section s under the composite inverse-image functor. Naturality of sheafification and the associativity isomorphism make the two unit maps equal. □

Lemma 11.5.7 (Finite pullback and closed-fibre restriction)**.** *Lean:* `conditional-interface:lemma:weil-pbc-finite-pullback-closed-fibre`

Uses: `def:weil-pbc-finite-section-pullback`

For a closed immersion $i : Z \hookrightarrow T$ and $Z' = Z \times_T T'$, restriction of $u_V^\sharp(s)$ to Z' equals the finite pullback of the restriction of s to Z.

Proof. Uses: `lemma:weil-pbc-finite-pullback-composition`

Apply the composition lemma to the two equal composites $Z' \to T' \to T$ and $Z' \to Z \to T$. The Cartesian identification makes their inverse-image sheafification units equal. □

Lemma 11.5.8 (Finite descent compatibility)**.** *Lean:* `conditional-interface:lemma:weil-pbc-finite-pullback-descent`

Uses: `def:weil-pbc-finite-section-pullback`

Let $u : T' \to T$ be finite surjective. Suppose $s' \in (u^{-1}F)(T')$ is equipped with a descended section $s \in F(T)$, meaning $u_T^\sharp(s) = s'$. Then the same equality holds after restriction to a closed fibre.

Proof. Uses: `lemma:weil-pbc-finite-pullback-composition,lemma:weil-pbc-finite-pullback-closed-fibre`

The first equality is the descent witness. Its closed-fibre restriction is the equality supplied by Lemma 11.5.7. □

Lemma 11.5.9 (Etale henselian restriction in degree zero)**.** *Lean:* `conditional-interface:lemma:weil-pbc-etale-degree-zero`

Uses: `def:weil-pbc-strict-henselian-setup`

For every etale abelian sheaf F on X, restriction induces $\Gamma(X, F) \xrightarrow{\sim} \Gamma(X_0, F_0)$.

Proof. Uses: `lemma:weil-pbc-stein-connectedness,lemma:weil-pbc-finite-pullback-identity,lemma:weil-pbc-finite-pullback-composition,lemma:weil-pbc-finite-pullback-closed-fibre,lemma:weil-pbc-finite-pullback-descent,lemma:weil-pbc-finite-refinement-etale-cover`

The canonical map from the etale to Zariski restriction is injective on stalks because a geometric point has a strictly local etale neighbourhood. For surjectivity, lift a fibre section etale-locally; quasi-compactness of X_0 gives finitely many local lifts, and Lemma 11.1.4 gives a finite surjective refinement on which all lifts exist. The two lifts agree on the finite overlap after restriction to X_0 by the Zariski result, and the equalizer sheaf descends the lift to X. Uniqueness is stalkwise injectivity. The finite-pullback compatibility required in Stacks Tag `09ZF` is recorded as four explicit equalities: identity pullback preserves the lift, composition of finite pullbacks preserves the lift, restriction to X_0 recovers the given section, and the descended section pulls back to each local lift; these are the four preceding lemmas. The finite refinement lemma proves existence and finite-presentation stage, and its stalkwise equalizer argument proves equality of the local lifts on the self-intersection. Thus the four finite-pullback compatibilities complete the descent. □

Lemma 11.5.10 (Finite-stage killing of a class)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-pbc-finite-stage-killing`

In an affine henselian pair, if a positive-degree class c dies on an integral strictly-local cover $T_\infty \to T_0$ represented by a filtered system $\{T_\lambda\}$ of finite finitely presented covers, then there exists λ_0 such that the pullback of c to $H^q(T_{\lambda_0}, F|)$ is zero.

Proof. Uses:

Represent the chosen cochain and its null-homotopy on the limit by finitely many sections. Each section and each equality occurs at some index because the system is filtered and the etale site is quasi-compact and quasi-separated. Choose one index dominating those finitely many indices. The descended cochain and the descended null-homotopy satisfy the coboundary equation at a further finite index; replacing by that index gives λ_0. This exposes the indexing and vanishing witness requested by Tag `09ZD`. □

Lemma 11.5.11 (Constructible approximation for effacement)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-pbc-constructible-approximation`

For a torsion sheaf F on an affine henselian pair and a fixed class $c \in H^q(T_0, F_0)$, there is a quasi-compact etale neighbourhood $U \to T$ and a constructible sheaf C on U with a monomorphism $C \hookrightarrow F|_U$ through which a cocycle representing c factors.

Proof. Uses:

Choose a split etale hypercover $U_\bullet \to T_0$ whose $(q+1)$-skeleton is finite and quasi-compact and a cocycle $c \in \Gamma(U_q, F_0)$. Write its finite components as $U_{r,a}$ for $0 \le r \le q+1$ and denote all face and degeneracy maps by $\partial^i_{r,a}$ and $s^i_{r,a}$. Choose quasi-compact etale lifts $\widetilde{U}_{r,a} \to U \to T$ and sections of F representing the finitely many sections in c and in the equations $dc = 0$.

Enlarge the finite list by applying every $\partial^i_{r,a}$ and $s^i_{r,a}$ to every chosen section. The simplicial identities are now a finite list of equalities on finite fibre products. Shrink the lifts once so that all equalities hold on the closed fibre and all maps factor through one quasi-compact etale neighbourhood $U \to T$. On each lifted component let $C_{r,a}$ be the subsheaf of $F|_U$ generated by this finite list. The equalizer loci of pairs of generators give a finite stratification on which $C_{r,a}$ is locally constant with finite fibres, hence constructible.

Closure under all displayed face and degeneracy maps gives a simplicial constructible subsheaf $C_\bullet \subseteq F|_{U_\bullet}$. The cocycle lies in $\Gamma(U_q, C_q)$ and the equation $dc = 0$ remains true in $C_\bullet$. Take the image of the augmentation $C_0 \to F|_U$ and its finite iterated extensions through degree $q+1$, then sheafify; this gives a constructible subsheaf $C \hookrightarrow F|_U$ through which c factors. Monicity is checked at a geometric stalk: a finite combination of the generators maps to zero in F precisely when it is already zero in the generated subsheaf. Thus every face map, cocycle, and equality used by the selected class is contained in C. □

Lemma 11.5.12 (Pushout preserves monomorphisms)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-pbc-pushout-monicity`

Given monomorphisms $A \hookrightarrow B$ and $A \hookrightarrow C$ of sheaves, the canonical map $C \to B \amalg_A C$ is monic.

Proof. Uses:

Let $P = B \amalg_A C$ be the pushout in sheaves. It is the cokernel sheaf of $A \to B \oplus C$, $a \mapsto (i_B a, -i_C a)$. If $c \in C(V)$ maps to zero in $P(V)$, then by the definition of a sheaf cokernel there is an etale cover $\{V_\alpha \to V\}$ and $a_\alpha \in A(V_\alpha)$ with $i_C(a_\alpha) = c|_{V_\alpha}$ and $i_B(a_\alpha) = 0$. Since i_B is monic, $a_\alpha = 0$, hence $c|_{V_\alpha} = 0$. The sheaf axiom gives $c = 0$. This proves monicity on arbitrary etale objects, and the same local equalizer calculation proves naturality of the pushout map. □

Lemma 11.5.13 (Constructible sheaves reduce to finite pushforwards)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-pbc-constructible-finite-pushforward`

If U is quasi-compact and quasi-separated, every constructible torsion sheaf C admits a monomorphism

$$C \longrightarrow \bigoplus_{i=1}^{r} (h_i)_* \underline{M_i},$$

where each $h_i : V_i \to U$ is finite and finitely presented and each M_i is a finite abelian group.

Proof. Uses: `lemma:weil-pbc-constructible-approximation`

Write $U = \varprojlim U_\lambda$ with affine transition maps and noetherian, quasi-compact, quasi-separated U_λ. The finite stratification of C, its finite stalk groups, all restriction maps, and their equalizer relations descend to one stage U_λ. We prove the assertion there by noetherian induction on a closed complement of a dense stratum. Choose a dense open $j : V \hookrightarrow U_\lambda$ where C_λ is finite locally constant and a finite etale cover $e : W \to V$ trivializing it. Take the integral closure of U_λ in $e_*\mathcal{O}_W$ and discard components disjoint from V; this gives a finite map $h : W' \to U_\lambda$ extending e. The adjunction unit is a monomorphism on V,

$$C_\lambda|_V \hookrightarrow e_* \underline{M} = (h_* \underline{M})|_V,$$

for a finite group M.

Let $Z = U_\lambda \setminus V$ with its reduced closed structure. By induction, $C_\lambda|_Z$ embeds into a finite sum $\bigoplus_b (h_b)_* \underline{M_b}$ with h_b finite over Z; composing with $Z \hookrightarrow U_\lambda$ keeps the maps finite. Map C_λ to

$$h_* \underline{M} \oplus \bigoplus_b (h_b)_* \underline{M_b}$$

by the dense-open unit in the first component and restriction to Z in the remaining components. At a geometric point of V the first component is injective, and at a geometric point of Z the second block is injective, so the combined map is monic. This is the localization induction, including the boundary and overlap comparison. Pull the finite diagram and monomorphism back to U. Etale inverse image is exact, and finite direct image commutes with this pullback by the finite geometric-stalk product formula. This gives the displayed monomorphism and expands Tag `09Z7`. □

Lemma 11.5.14 (Finite-cover killing and finite pushforward monicity)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-pbc-finite-cover-killing`

For a fixed $c \in H^q(T_0, C_0)$ with $q > 0$ and constructible C, there is a finite finitely presented cover $h : V \to T$ whose closed-fibre pullback $h_0^* c$ is zero; the induced map $C \hookrightarrow h_* h^* C$ is monic.

Proof. Uses: `lemma:weil-pbc-finite-stage-killing,lemma:weil-pbc-finite-stalk-formula`

Use the integral strictly-local cover killing c and represent it by the filtered system of finite, finitely presented covers $h_\lambda : V_\lambda \to T$. Lemma 11.5.10 selects an index λ_0 at which $h_{\lambda_0,0}^* c = 0$; set $h = h_{\lambda_0}$. The map to the finite pushforward is the unit of (h^*, h_*). At a geometric point its stalk is the diagonal into the finite product of stalks from Lemma 11.2.1; the factor corresponding to the chosen point above it is the identity. Hence the unit is injective on every geometric stalk and therefore monic. □

Lemma 11.5.15 (Affine Gabber effacement)**.** Uses:

Lean: `conditional-interface:lemma:weil-pbc-gabber-effacement`

For an affine henselian pair (T, T_0) and a torsion sheaf F on T_{et}, every class $c \in H^q(T_0, F_0)$ with $q > 0$ admits a monomorphism $F \hookrightarrow F'$ such that the image of c in $H^q(T_0, F_0')$ is zero.

Proof. Uses: `lemma:weil-pbc-constructible-approximation,lemma:weil-pbc-constructible-finite-pushforward,lemma:weil-pbc-finite-cover-killing,lemma:weil-pbc-pushout-monicity`

Apply constructible approximation to obtain $C \hookrightarrow F$ through which c factors, and use finite-pushforward reduction to choose

$$C \hookrightarrow D_0 = \bigoplus_{i=1}^{r} (h_i)_* \underline{M_i}.$$

Let c_i be the image of c in $H^q(T_0, (h_i)_* \underline{M_i}|_{T_0})$. Finite direct-image exactness identifies c_i with a class on the closed fibre of h_i. Apply finite-cover killing to each c_i and take the fibre product of the resulting finite covers over T; after replacing by one finite-presentation stage, call the common finite cover $h : V \to T$. Every component of $h_0^* c$ is zero. The composite unit

$$C \hookrightarrow D_0 \longrightarrow h_* h^* D_0 =: D$$

is monic by the finite stalk product formula. Form the pushout square

$$\begin{array}{ccc} C & \longrightarrow & D \\ \downarrow & & \downarrow \\ F & \longrightarrow & F' := F \amalg_C D. \end{array}$$

Pushout monicity makes $F \to F'$ injective. The image of c in $H^q(T_0, D_0)$ is carried to zero in $H^q(T_0, D)$ because its pullback to V_0 is zero and h_* is exact. The pushout square therefore carries c to zero in $H^q(T_0, F_0')$. This displays both the common dominating cover and the null-homotopy square and closes Tag `09ZE`'s effacement step. □

Proposition 11.5.16 (Gabber affine comparison)**.** Uses:

Lean: `conditional-interface:prop:weil-pbc-gabber-affine-comparison`

For an affine henselian pair (T, T_0) and torsion F, $H^q(T, F) \xrightarrow{\sim} H^q(T_0, F_0)$ for all $q \geq 0$.

Proof. Uses:

Degree zero is the etale henselian restriction lemma applied on the affine. For $q > 0$, assume comparison in all degrees below q. To prove injectivity, take a nonzero $\xi \in H^q(T, F)$ and apply effacement with closed subscheme T itself, obtaining $F \hookrightarrow F'$ in which ξ dies. Put $Q = F'/F$. Exactness lifts ξ from $H^{q-1}(T, Q)$. The induction isomorphism in degree $q-1$ carries any such lift to T_0; if ξ restricted to zero, exactness and the commutative boundary square would force the lift to come from $H^{q-1}(T, F')$, contradicting the choice of nonzero ξ.

For surjectivity, take $\eta \in H^q(T_0, F_0)$ and use effacement with closed subscheme T_0 to choose $F \hookrightarrow F'$ killing η. Exactness lifts η from $H^{q-1}(T_0, Q_0)$. By induction this lift is the restriction of a class in $H^{q-1}(T, Q)$, whose boundary in $H^q(T, F)$ restricts to η. Thus the comparison is separately injective and surjective. This expands Tag `09ZI`; no statement is imported from that tag. □

Lemma 11.5.17 (Injective restriction acyclicity bound)**.** Uses:

Lean: `conditional-interface:lemma:weil-pbc-injective-restriction-acyclicity`

Let a separated scheme with affine diagonal be covered by $n+1$ affine opens. If I is injective on the scheme over a henselian pair, then the restriction to the closed fibre has vanishing cohomology in degrees $> n$.

Proof. Uses: `prop:weil-pbc-gabber-affine-comparison`

For $n = 0$ this is the affine comparison applied to the injective I. For $n > 0$, split the cover as U_0 and $V = \bigcup_{i>0} U_i$. The Mayer–Vietoris long exact sequence for $U_0 \cup V$ has terms on U_0, V, and $U_0 \cap V$; the last two have covers of size at most n and the affine comparison kills their groups above the inductive bound. The long exact sequence therefore kills the group above degree n. This is the dimension induction located at Tag `0A51`. □

Lemma 11.5.18 (Finite locally constant sheaves and finite-etale schemes)**.** Uses:

Lean: `conditional-interface: lemma: weil-pbc-finite-locally-constant-etale-equivalence`

For a scheme T, finite locally constant sheaves of sets on T_{et} are equivalent to finite etale T-schemes. A finite abelian group action on the sheaf corresponds to an action by finite-etale T-automorphisms.

Proof. Uses:

Given a sheaf E, choose a finite etale cover $U \to T$ trivializing it with finite fibre S. The descent isomorphism on $U \times_T U$ is a permutation of S satisfying the cocycle equation, hence descends the disjoint union $\coprod_{s \in S} U$ to a finite etale scheme P. Conversely, the sheaf of sections of a finite etale P is finite locally constant; the two constructions are inverse because both are computed after the same trivializing cover. An M-action is a compatible family of permutations, so it descends exactly when the sheaf action is compatible with the descent cocycle. □

Lemma 11.5.19 (Finite-etale lifting over a henselian pair)**.** Lean: `conditional-interface: lemma: weil-pbc-finite-etale-henselian-lifting`

Uses: `def: weil-pbc-strict-henselian-setup`

If $(A, \mathfrak{m})$ is henselian and $Y \to \operatorname{Spec} A$ is proper, restriction from finite etale Y-schemes to finite etale Y_0-schemes is an equivalence, including unique lifting of morphisms and group actions.

Proof. Uses: `lemma:weil-pbc-finite-locally-constant-etale-equivalence`

We expand the equivalence used by Tag `0A5G`; its source locator is Tag `0A48`. First suppose that A is complete noetherian and put $Y_r = Y \times_A A/\mathfrak{m}^{r+1}$. Formal etaleness and nilpotent invariance of the etale site lift a finite etale $P_0 \to Y_0$ uniquely to a compatible system $P_r \to Y_r$. Finiteness can be checked modulo the nilpotent ideal: on a finite affine cover $Y_r = \bigcup_a \operatorname{Spec} R_{r,a}$, each lifted algebra is generated by lifts of a finite set of generators of the $r = 0$ algebra, and Nakayama makes the resulting algebra finite. The transition isomorphisms on the finitely many overlaps obey the cocycle equation because their restrictions do and etale morphisms are formally unramified.

Grothendieck existence applied to the compatible finite algebras $(p_r)_* \mathcal{O}_{P_r}$ algebraizes them to a coherent $\mathcal{O}_Y$-algebra, hence to a finite morphism $P \to Y$ with special fibre P_0. The etale locus of $P \to Y$ is open and contains P_0. Every point of P specializes to the closed fibre because P is proper over the local base; therefore the complementary closed non-etale locus is empty. This proves essential surjectivity in the complete noetherian case. For full faithfulness, a map $u_0 : P_0 \to Q_0$ has graph

$$\Gamma_{u_0} : P_0 \longrightarrow P_0 \times_{Y_0} Q_0,$$

which is an open-and-closed finite etale subscheme. Algebraize this graph to $W \to P \times_Y Q$. The projection $W \to P$ is finite etale and is an isomorphism on the closed fibre; its isomorphism locus is open, and the proper closed complement would meet the closed fibre if nonempty. Hence it is empty, and $W \cong P$ gives the unique lift $u : P \to Q$.

Now suppose A is a noetherian henselian local G-ring. Complete it to $\widehat{A}$. The first step gives a finite etale cover over $Y_{\widehat{A}}$. Choose a finite affine cover of Y, write the finitely many cover algebras, overlap maps, and cocycle equations with finitely many generators and relations, and descend all of them to a finite type A-algebra $A_i = A[x_1, \dots, x_N]/(f_1, \dots, f_M)$ inside $\widehat{A}$. After increasing i, the descended map is finite etale. Approximation for henselian G-rings gives a solution $(b_1, \dots, b_N) \in A^N$ of all equations with $b_j \equiv x_j \pmod{\mathfrak{m}}$. The induced retraction $A_i \to A$ base-changes the descended cover to a finite etale cover of Y with special fibre P_0. The graph argument proves full faithfulness.

For a general henselian local A, choose noetherian local G-rings A_i from finite-type $\mathbf{Z}$-models and write $A = \varinjlim_i A_i^h$. Proper finite presentation gives an index i_0 and a proper model $Y_{i_0} \to \operatorname{Spec} A_{i_0}^h$. A finite etale cover, a morphism between two such covers, and an equality of two morphisms are each finite presentation data; using a finite affine cover and the finitely many overlap equations, they descend to one common $i \geq i_0$. Thus

$$FEt_Y = \varinjlim_i FEt_{Y_i}, \qquad FEt_{Y_0} = \varinjlim_i FEt_{(Y_i)_0},$$

including the Hom sets. The equivalence at every henselian noetherian G-ring stage gives the desired equivalence after the colimit. Finally an action of a finite group consists of finitely many morphisms and finitely many multiplication, unit, and inverse equations. Full faithfulness lifts the morphisms and their equations uniquely, so group actions lift as well. □

Lemma 11.5.20 (Finite-etale torsor comparison)**.** Lean: `conditional-interface: lemma: weil-pbc-h1-torsor-comparison`

Uses: `def: weil-pbc-strict-henselian-setup`

For a finite abelian group M and proper $Y \to \operatorname{Spec} A$, restriction induces a bijection

$$H^1(Y, \underline{M}) \xrightarrow{\sim} H^1(Y_0, \underline{M}).$$

Proof. Uses: `lemma:weil-pbc-etale-degree-zero,lemma:weil-pbc-finite-locally-constant-etale-equivalence,lemma:weil-pbc-finite-etale-henselian-lifting`

An $\underline{M}$-torsor is a finite etale Y-scheme P with a simply transitive M-action by the finite-locally-constant equivalence. Apply the henselian lifting lemma to lift P, every automorphism $m : P \to P$, and the multiplication diagrams. Uniqueness of lifted morphisms preserves the group law and simple transitivity. Restriction and lifting are inverse on isomorphism classes, which is the displayed H^1 bijection. This is the torsor subtree of Tag `0A5G`. □

Lemma 11.5.21 (Constructible representation of an H^1 class)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-pbc-h1-constructible-representation`

For an injective sheaf I of $\mathbf{F}_\ell$-modules on $\mathbf{P}^1_A$, every class $c \in H^1(X_0, I_0)$ admits finite $\mathbf{F}_\ell$-modules M_i, finite finitely presented morphisms $h_i : V_i \to X$, and a map $\bigoplus_i (h_i)_*\underline{M_i} \to I$ such that c is the image of a tuple of classes in $\bigoplus_i H^1(X_0, (h_i)_*\underline{M_i}|_{X_0})$.

Proof. Uses: `lemma:weil-pbc-constructible-approximation,lemma:weil-pbc-constructible-finite-pushforward`

View c as the isomorphism class of an I_0-torsor. It is trivial on an etale cover; quasi-compactness of X_0 selects finitely many members $U_a \to X_0$. Choosing local origins gives a Cech cocycle $z = (z_{ab})$ with

$$z_{ab} \in I_0(U_a \times_{X_0} U_b), \qquad z_{bc} - z_{ac} + z_{ab} = 0$$

on every triple overlap. There are finitely many sections and equations. Constructible approximation therefore supplies a constructible sheaf F, a map $\alpha : F \to I$, and lifts $\widetilde{z}_{ab} \in F_0(U_{ab})$ satisfying the same equations and mapping to z_{ab}.

The finite-pushforward lemma gives a monomorphism $\iota : F \hookrightarrow F' = \bigoplus_{i=1}^r (h_i)_*\underline{M_i}$. Injectivity of I extends α across ι to a map $\psi : F' \to I$. The sheaf square and the three relevant terms of the Cech complex form the commuting three-row diagram

$$\begin{array}{ccccc}
F & \xrightarrow{\iota} & F' & \longrightarrow & \operatorname{coker}(\iota) \\
{\scriptstyle\alpha}\downarrow & & {\scriptstyle\psi}\downarrow & & \downarrow \\
I & = & I & \longrightarrow & 0 \\
\downarrow & & \downarrow & & \downarrow \\
\prod_a I_0(U_a) & \xrightarrow{d^0} & \prod_{a,b} I_0(U_{ab}) & \xrightarrow{d^1} & \prod_{a,b,c} I_0(U_{abc}).
\end{array}$$

Here the bottom vertical maps mean restriction followed by the indicated Cech cochain inclusion; commutativity in degrees $0, 1, 2$ follows from functoriality of restriction. The cocycle $\iota(\widetilde{z})$ defines $c' \in H^1(X_0, F'_0)$ and $\psi_0(c') = c$.

Finite direct sums commute with H^1, while exact finite direct image and Leray give

$$H^1(X_0, F'_0) = \bigoplus_{i=1}^r H^1(X_0, (h_i)_*\underline{M_i}|_{X_0}) = \bigoplus_{i=1}^r H^1((V_i)_0, \underline{M_i}).$$

Thus c' is the required finite tuple, and the displayed map $F' \to I$ sends that tuple to the original class. □

Lemma 11.5.22 (Acyclicity of injectives on $\mathbf{P}^1_s$)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-pbc-p1-injective-acyclic`

For an injective sheaf I of $\mathbf{F}_\ell$-modules on $X = \mathbf{P}^1_A$, the restriction I_0 is $\Gamma(X_0, -)$-acyclic: $H^q(X_0, I_0) = 0$ for all $q > 0$.

Proof. Uses: `lemma:weil-pbc-injective-restriction-acyclicity,lemma:weil-pbc-h1-torsor-comparison,lemma:weil-pbc-h1-constructible-representation`

The affine-cover lemma gives vanishing for $q > 1$ because $\mathbf{P}^1$ is covered by two affines. For $q = 1$, apply the representation lemma. For each finite summand, finite-direct-image exactness and Leray identify its H^1 on X_0 with $H^1((V_i)_0, \underline{M_i})$. Torsor comparison lifts every component to $H^1(V_i, \underline{M_i})$, and Leray returns a class in $H^1(X, (h_i)_*\underline{M_i})$. Their finite sum maps to zero in $H^1(X, I)$ because I is injective. Restriction of the commuting square shows that the original class is zero. The source locator is Tag `0A5H`. □

Proposition 11.5.23 (Strictly henselian $\mathbf{P}^1$ comparison)**.** Lean: `conditional-interface:prop:weil-pbc-p1-strict-henselian-comparison`

Uses: `def:weil-pbc-strict-henselian-setup`

For every torsion sheaf F on X, restriction induces isomorphisms

$$H^q(X, F) \xrightarrow{\sim} H^q(X_0, F_0) \qquad (q \geq 0).$$

Proof. Uses: `lemma:weil-pbc-etale-degree-zero,lemma:weil-pbc-p1-injective-acyclic`

Choose an injective resolution $F \to I^\bullet$. Exactness of inverse image along the closed immersion gives an exact complex $F_0 \to I_0^\bullet$. The preceding lemma makes each I_0^r acyclic, so Leray's acyclicity lemma identifies $R\Gamma(X_0, F_0)$ with $\Gamma(X_0, I_0^\bullet)$. Degree-zero restriction identifies this complex with $\Gamma(X, I^\bullet)$ term by term. Taking cohomology gives the displayed isomorphisms, and naturality follows from the naturality of the restriction map and of injective resolutions. The source locators are Tags `0A0C` and `0A3S`. □

Proposition 11.5.24 (Strict-henselian projective-line proper base change)**.** Uses:

Lean: `conditional-interface:prop:weil-pbc-projective-line-strict-henselian`

Let $A \to B$ be a local map of strictly henselian local rings which induces an isomorphism of their separably closed residue fields. For the Cartesian square obtained from $\mathbf{P}^1_A \to \operatorname{Spec} A$, the derived exchange map for a torsion sheaf is an isomorphism on the stalk at the closed geometric point of $\operatorname{Spec} B$.

Proof. Uses: `prop:weil-pbc-p1-strict-henselian-comparison`

Write $S = \operatorname{Spec} A$, $T = \operatorname{Spec} B$, and let $\bar{t}$ and $\bar{s}$ be their closed geometric points under the chosen residue-field identification. Choose an injective resolution $F \to I^\bullet$ on $\mathbf{P}^1_A$. A strict henselian local scheme is initial in the opposite category of pointed etale neighbourhoods of its closed geometric point: the chosen point selects a unique section of the connected component of any pointed etale neighbourhood. Consequently the pointed-neighbourhood stalk colimit has canonical restriction quasi-isomorphisms to global cohomology, and the following rectangle is defined degree by degree:

$$\begin{array}{ccc}
(Rp_*F)_{\bar{s}} & \xrightarrow{(\mathrm{Ex}_{\square,F})_{\bar{t}}} & (Rp_{T,*}F_T)_{\bar{t}} \\
{\scriptstyle \Phi_A}\downarrow{\scriptstyle \sim} & & \downarrow{\scriptstyle \Phi_B} \\
R\Gamma(\mathbf{P}^1_A, F) & \xrightarrow{\mathrm{res}_{A,B}} & R\Gamma(\mathbf{P}^1_B, F_T) \\
{\scriptstyle \rho_A}\downarrow{\scriptstyle \sim} & & \downarrow{\scriptstyle \rho_B} \quad \sim V \\
R\Gamma(\mathbf{P}^1_{k_A}, F_{k_A}) & \xrightarrow{\mathrm{id}} & R\Gamma(\mathbf{P}^1_{k_B}, F_{k_B}).
\end{array}$$

Here Φ_A and Φ_B send a germ represented on a pointed neighbourhood to its pullback on the corresponding strict local base. To verify that they are quasi-isomorphisms without a stalk theorem as a leaf, compute a cohomology class on a finite split etale hypercover. Its $(q+1)$-skeleton, its cocycle, and any null-homotopy contain finitely many etale objects, face maps, sections, and equalities. The unique pointed sections pull this complete finite datum to the strict local base; conversely the datum descends to one pointed neighbourhood by finite presentation. Thus the stalk colimit and strict-local cohomology agree in every degree.

The upper square commutes because both routes apply to a representative $s \in I^r(\mathbf{P}^1_A \times_S V)$ by pulling it to $\mathbf{P}^1_B \times_T (V \times_S T)$ and then to the pointed limit. This is exactly the unit–counit formula defining the S0 exchange map. The lower square commutes because restriction of that same section through B and then k_B equals restriction through $k_A \cong k_B$; this also proves the compatibility with differentials of $I^\bullet$.

The maps ρ_A and ρ_B are quasi-isomorphisms by Proposition 11.5.23, and the bottom map is the identity under $k_A \cong k_B$. Hence the top exchange stalk is an isomorphism. Notice that no assertion is made here for a nontrivial extension of separably closed residue fields; that is precisely the S5.2 curve-invariance input. □

> **Status.** The unique stable finite-torsion export label is `prop:weil-proet-finite-proper-base-change`. It is deliberately not declared in this S0–S4 fragment: S5 (strict-henselization stalks and separably-closed field invariance) and S6 (global assembly and torsion devissage) must precede that declaration. The older A3 blueprint already occupies the stable label and must be replaced, not duplicated, at merge.

11.6 S5. The projective line over an arbitrary base

11.6.1 S5.1. Stalks through strict henselization

Definition 11.6.1 (Pointed etale neighbourhood category)**.** Uses:

Lean: `conditional-interface:def:weil-pbc-pointed-etale-neighbourhoods`

Let $\bar{s} : \operatorname{Spec} \Omega \to S$ be a geometric point. Define $\mathcal{N}_{\bar{s}}$ to have objects pairs $(V, \bar{v})$ with $V \to S$ etale and $\bar{v} : \operatorname{Spec} \Omega \to V$ above $\bar{s}$. A morphism $(V, \bar{v}) \to (W, \bar{w})$ is an S-map $V \to W$ carrying $\bar{v}$ to $\bar{w}$. We index stalk colimits by $\mathcal{N}_{\bar{s}}^{\mathrm{op}}$ so that restriction maps point forward.

Lemma 11.6.2 (Affine pointed neighbourhoods are cofinal). *Lean:* `conditional-interface:lemma:weil-pbc-affine-neighbourhood-cofinal`

Uses: `def:weil-pbc-pointed-etale-neighbourhoods`

The full subcategory $\mathcal{N}_{\bar{s}}^{\mathrm{aff}}$ of affine pointed etale neighbourhoods is initial in $\mathcal{N}_{\bar{s}}$ and hence cofinal for the opposite-indexed stalk colimit.

Proof. Uses:

Given $(V, \bar{v})$, choose an affine open $V_0 \subseteq V$ containing the image of $\bar{v}$; then $(V_0, \bar{v}) \to (V, \bar{v})$ is a morphism. Given two affine refinements, their fibre product over V contains the lifted geometric point; an affine open around that point refines both. These two facts give nonemptiness and connectedness of every comma category required for initiality. □

Definition 11.6.3 (Strict henselization from pointed neighbourhoods). *Lean:* `conditional-interface:def:weil-pbc-strict-henselization-limit`

Uses: `def:weil-pbc-pointed-etale-neighbourhoods`

Let

$$\mathcal{I}_{\bar{s}} := (\mathcal{N}_{\bar{s}}^{\mathrm{aff}})^{\mathrm{op}}.$$

For $V = \operatorname{Spec} A_V$, define $D_{\bar{s}} : \mathcal{I}_{\bar{s}} \to \mathbf{CRing}$ by $D_{\bar{s}}(V, \bar{v}) = A_V$. An arrow $(W, \bar{w}) \to (V, \bar{v})$ in $\mathcal{I}_{\bar{s}}$ represented by a pointed map $a : (V, \bar{v}) \to (W, \bar{w})$ is sent to $a^{\#} : A_W \to A_V$. Put

$$A_{S,\bar{s}}^{\mathrm{sh}} := \varinjlim_{\mathcal{I}_{\bar{s}}} D_{\bar{s}}, \qquad S_{(\bar{s})} := \operatorname{Spec} A_{S,\bar{s}}^{\mathrm{sh}}.$$

The affine scheme–ring anti-equivalence carries the colimit cocone to the canonical limit cone

$$S_{(\bar{s})} \xrightarrow{\sim} \varprojlim_{(V,\bar{v}) \in \mathcal{N}_{\bar{s}}^{\mathrm{aff}}} V.$$

Lemma 11.6.4 (Well-definedness certificate for def:weil-pbc-strict-henselization-limit). *Lean:* `conditional-interface:lemma:weil-sf-well-defined-def-weil-pbc-strict-henselization-limit`

Uses: `def:weil-pbc-strict-henselization-limit`

The coordinate-ring rule of Definition 11.6.3 is a functor $D_{\bar{s}} : \mathcal{I}_{\bar{s}} \to \mathbf{CRing}$. Applying Spec *to its colimit cocone gives the displayed limit cone, and affine cofinality gives the same strict-local object when nonaffine pointed neighbourhoods are admitted.*

Proof. Uses: `def:weil-pbc-strict-henselization-limit,lemma:weil-pbc-affine-neighbourhood-cofinal`

Coordinate rings carry an identity pointed map to an identity ring map and reverse composition. Passage to $(\mathcal{N}_{\bar{s}}^{\mathrm{aff}})^{\mathrm{op}}$ therefore makes $D_{\bar{s}}$ covariant. The fixed universe of the coefficient convention makes this a small diagram, so its colimit exists in **CRing**.

The anti-equivalence between affine schemes and commutative rings sends this colimit, together with its structure maps, to the inverse-limit cone on the affine pointed neighbourhoods. The initiality statement in Lemma 11.6.2 makes restriction to the affine subcategory limit-cofinal and opposite-colimit-cofinal. Hence the same ring and affine scheme compute the indicated strict-local object. □

Lemma 11.6.5 (Base change commutes with the neighbourhood limit). *Lean:* `conditional-interface:lemma:weil-pbc-fibre-product-limit`

Uses: `def:weil-pbc-strict-henselization-limit`

For a quasi-compact and quasi-separated $f : X \to S$,

$$X \times_S S_{(\bar{s})} \cong \varprojlim_{(V,\bar{v})} (X \times_S V).$$

Proof. Uses:

On affine charts $X_0 = \operatorname{Spec} B$ over $S_0 = \operatorname{Spec} A$, the left side has coordinate ring $B \otimes_A \varinjlim A_V$. Tensor product commutes with filtered colimits, so this is $\varinjlim (B \otimes_A A_V)$, the coordinate ring of the inverse limit on the right. Quasi-compactness gives finitely many charts and quasi-separatedness gives quasi-compact pairwise intersections; the affine identifications therefore glue. □

Lemma 11.6.6 (Finite etale hypercover skeleta descend)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-pbc-finite-hypercover-descent`

Let $X_\infty = \varprojlim_i X_i$ be as in the continuity lemma below and fix $N \geq 0$. Every etale hypercover of X_∞ admits a refinement whose N-skeleton is split, has finitely many quasi-compact and quasi-separated components in each degree, and is the pullback of an N-truncated etale hypercover of some X_j. A morphism between two such finite skeleta, and an equality between two such morphisms, also descends after increasing j.

Proof. Uses: `lemma:weil-pbc-fibre-product-limit`

Construct the refinement inductively. In degree zero, quasi-compactness of X_∞ lets us choose a finite subcover by quasi-compact etale objects. Suppose degrees $< r$ have been chosen. The rth matching object is a finite limit of lower-degree objects, because the boundary diagram of the simplex r-simplex has finitely many faces. It is therefore quasi-compact and quasi-separated. Cover it by finitely many quasi-compact etale objects and adjoin the degenerate summands prescribed by the already chosen degeneracy maps. This gives a split hypercover through degree r. Iterating to N gives the required finite skeleton.

Every chosen component is of finite presentation over X_∞, so it is the pullback of an etale component at some stage. There are only finitely many components, face maps, degeneracy maps, and augmentations through degree N; finite presentation descends them all to one common stage. The simplicial identities are finitely many equalities of morphisms between finitely presented schemes and hence hold after a further common stage. For each $r \leq N$, the map to the matching object is an etale cover after pullback to X_∞. Etaleness descends by a Jacobian etale algebra presentation $B = A[x_1, \ldots, x_m]/(f_1, \ldots, f_m)$ with an invertible Jacobian minor. Surjectivity also descends: the complement of the image is constructible and quasi-compact; if it survived at every later stage, its inverse limit would give a point outside the limiting image. Thus, after one more stage, all matching maps are covers. The same finite-presentation argument descends a morphism of skeleta. Equality of two morphisms is the factorization through their equalizer, a finitely presented closed condition, so it too holds at a later stage. □

Lemma 11.6.7 (Etale cohomology continuity)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-pbc-etale-cohomology-continuity`

Let (X_i) be a cofiltered inverse system of quasi-compact and quasi-separated schemes with affine transition maps and limit X_∞. For a torsion sheaf F_i descending to one stage, the natural map

$$\varinjlim_{j \geq i} H^q(X_j, F_j) \longrightarrow H^q(X_\infty, F_\infty)$$

is an isomorphism.

Proof. Uses: `lemma:weil-pbc-fibre-product-limit,lemma:weil-pbc-finite-hypercover-descent`

We first recall why finite hypercover data compute the required cohomology. For an etale hypercover $U_\bullet \to Z$, put

$$C^r(U_\bullet, F) = \Gamma(U_r, F), \qquad d = \sum_{a=0}^{r+1} (-1)^a \partial_a^*.$$

Order split hypercovers by common refinement and set $H^q_{\mathrm{hyp}}(Z, F) = \varinjlim_{U_\bullet} H^q(C^\bullet(U_\bullet, F))$. The sheaf equalizer gives $H^0_{\mathrm{hyp}}(Z, F) = \Gamma(Z, F)$. A short exact sequence of sheaves gives connecting maps after refining so that each of the finitely many local lifts of a cocycle exists. The resulting functors form a cohomological delta functor. They are effaceable in positive degree: given a q-cocycle, cover its finitely many degree-q components so that they lift through a chosen monomorphism into an injective sheaf, then fill the matching objects successively through degree $q+1$; on this refinement the cocycle is a coboundary. Hence this delta functor is the universal derived functor of global sections. Therefore

$$H^q_{\mathrm{hyp}}(Z, F) = H^q_{\mathrm{et}}(Z, F).$$

The construction also shows that a degree-q class and a witness that it is zero use only the $(q+1)$-skeleton of one split hypercover.

Let $\alpha \in H^q(X_\infty, F_\infty)$. Represent it by a cocycle $c \in C^q(U_\bullet, F_\infty)$ on a split hypercover with finite qcqs $(q+1)$-skeleton. By the preceding lemma, this skeleton descends to some X_j. Each component of c is, by the definition of the inverse-image sheaf, locally represented by a section at a finite stage. Refine the finite degree-q cover once and choose one stage dominating all those representatives. The equation $dc = 0$ consists of finitely many equalities on the degree-$(q+1)$ components; these equalities descend after one further increase of the stage. We obtain a cocycle c_j and hence a class $\alpha_j \in H^q(X_j, F_j)$ mapping to α. This proves surjectivity.

For injectivity, suppose $\alpha_j \in H^q(X_j, F_j)$ maps to zero on X_∞. Choose a finite split hypercover and cocycle c_j representing it. Vanishing on the limit supplies a refinement $V_\bullet \to U_\bullet \times_{X_j} X_\infty$ and a cochain $b \in C^{q-1}(V_\bullet, F_\infty)$ with $db = c_j|_V$. Only the $(q+1)$-skeleton, the refinement maps, the components of b, and the finite list of equations $db = c_j|_V$ are used. Descend that entire list to one stage X_k by the preceding lemma and the same section argument. It gives a null-homotopy for the image of c_j on X_k, so α_j is zero in the filtered colimit. The comparison map is therefore injective.

All constructions commute with restriction and with the transition maps, so the resulting bijection is the canonical map in the statement. This expands the continuity input of Tag 03Q6; neither the hypercover nor the common finite stage is suppressed. □

Proposition 11.6.8 (Strict-henselization stalk formula). *Lean:* `conditional-interface:prop:weil-pbc-strict-henselization-stalk-formula`

Uses: `def:weil-pbc-pointed-etale-neighbourhoods`, `def:weil-pbc-strict-henselization-limit`

For quasi-compact quasi-separated $f : X \to S$, torsion F, and $q \geq 0$,

$$(R^q f_* F)_{\bar{s}} \cong H^q(X \times_S S_{(\bar{s})}, F|).$$

For $K \in D^+(X_{\mathrm{et}})$ this lifts to

$$(Rf_* K)_{\bar{s}} \xrightarrow{\sim} R\Gamma(X \times_S S_{(\bar{s})}, K|)$$

in $D^+(\mathbf{Ab})$.

Proof. Uses: `lemma:weil-pbc-affine-neighbourhood-cofinal`, `lemma:weil-pbc-fibre-product-limit`, `lemma:weil-pbc-etale-cohomology-continuity`

The stalk is the colimit over $\mathcal{N}_{\bar{s}}^{\mathrm{op}}$ of $H^q(X \times_S V, F|)$. Replace the index by affine neighbourhoods, identify their inverse limit by Lemma 11.6.5, and apply cohomology continuity. For the derived statement, choose a bounded-below injective complex $J^\bullet$ representing K. The chain map from the stalk complex $(f_* J^\bullet)_{\bar{s}}$ to the sections complex on the strict henselization is obtained by restricting each representative section. Its map on every cohomology group is the preceding isomorphism, so it is a quasi-isomorphism. This expands Tag 03Q9. □

11.6.2 S5.2. Proper curves under separably closed extension

Lemma 11.6.9 (Canonical etale descent across a universal homeomorphism). *Uses:*

Lean: `conditional-interface:lemma:weil-pbc-universal-homeomorphism-descent-datum`

Let $u : X_0 \to X$ be a universal homeomorphism. For every etale $V_0 \to X_0$, the two pullbacks to $X_0 \times_X X_0$ have a unique isomorphism whose restriction along the diagonal is the identity. This isomorphism is a descent datum and is compatible with morphisms, identities, and composition.

Proof. Uses:

A universal homeomorphism is integral, surjective, and universally injective. Hence the diagonal

$$\Delta : X_0 \longrightarrow X_0 \times_X X_0$$

is a closed immersion and is surjective on points. On affine charts it is therefore defined by a nil ideal: if $X = \operatorname{Spec} A$ and $X_0 = \operatorname{Spec} B$, the multiplication $B \otimes_A B \to B$ is surjective and its kernel is contained in every prime ideal.

We first record the nil-ideal calculation used below. If $R \to R/J$ has J nil, reduction gives an equivalence between etale R-algebras and etale R/J-algebras. For a finitely presented affine etale algebra this is checked on a Jacobian presentation

$$((R/J)[T_1, \ldots, T_m]/(f_1, \ldots, f_m))_g$$

with the relevant Jacobian minor invertible. Lift the finitely many coefficients and g to R. The same minor remains invertible after localization because an element invertible modulo a nil ideal is invertible. Existence of a lift of a morphism follows by successively correcting the images of the T_i modulo the powers of the finitely generated nil ideal appearing in those coefficients; the inverse Jacobian gives the correction. The process is finite because that finitely generated subideal is nilpotent. If two lifts agree modulo J, their difference lies in J; the same linearized equations and inverse Jacobian force equality, one nilpotence power at a time. General etale schemes are glued from such affine pieces; the uniqueness just proved makes the overlap maps and their cocycle equations lift uniquely.

Apply this nil-invariance to Δ. The two pullbacks $p_1^* V_0$ and $p_2^* V_0$ restrict along Δ to the same etale X_0-scheme V_0. Thus there is a unique isomorphism

$$\alpha : p_1^* V_0 \xrightarrow{\sim} p_2^* V_0, \qquad \Delta^* \alpha = \mathrm{id}_{V_0}.$$

On $X_0 \times_X X_0 \times_X X_0$, both $p_{23}^* \alpha \circ p_{12}^* \alpha$ and $p_{13}^* \alpha$ restrict to the identity along the small diagonal. That diagonal is again a surjective closed immersion defined by a nil ideal, so uniqueness gives the cocycle equation. If $a : V_0 \to W_0$ is a morphism, the two composites $p_2^* a \circ \alpha_V$ and $\alpha_W \circ p_1^* a$ have the same diagonal restriction and hence are equal. Taking a to be an identity or a composite proves the asserted identity and composition compatibilities. □

Lemma 11.6.10 (Covering compatibility for universal homeomorphisms)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-pbc-universal-homeomorphism-coverings`

Let $u : X_0 \to X$ be a universal homeomorphism. Base change gives an equivalence between etale X-schemes and etale X_0-schemes, and a family $\{U_i \to U\}$ is jointly surjective before base change if and only if its base change is jointly surjective.

Proof. Uses: `lemma:weil-pbc-universal-homeomorphism-descent-datum`

We construct a quasi-inverse to base change. First let $X = \operatorname{Spec} A$ and let $V_0 = \operatorname{Spec} C$ be affine etale over an affine inverse image $\operatorname{Spec} B \subset X_0$. The preceding lemma gives an isomorphism between the two $B \otimes_A B$-algebras obtained from C, with its cocycle equation. Define D as the equalizer of the two resulting maps

$$C \rightrightarrows C \otimes_A B,$$

where the second arrow is followed by the descent isomorphism. On a Jacobian etale presentation of C, this equalizer uses only its finitely many generators, relations, and descent equations. The inverse-Jacobian lifting calculation from the preceding proof gives

$$B \otimes_A D \xrightarrow{\sim} C$$

and shows that D is finitely presented and etale over A: surjectivity lifts each generator of C after imposing its descent equation, injectivity uses the cocycle equation on $B \otimes_A B \otimes_A B$, and the lifted Jacobian minor is invertible. This is the affine effective-descent calculation, rather than an appeal to an unnamed descended algebra.

For a general etale $V_0 \to X_0$, choose affine opens $W_a \subset V_0$, each mapping into an affine inverse image in X_0. Their descended affine etale schemes are denoted $V_a \to X$. The intersections $W_a \cap W_b$ are open subspaces. Because $|X_0| \to |X|$ and every one of its base changes are homeomorphisms, each such open is the inverse image of a unique open $V_{ab} \subset V_a$. The two identifications of $W_a \cap W_b$ descend uniquely to $V_{ab} \cong V_{ba}$ by the morphism compatibility in the preceding lemma. On triple intersections the two composites have equal pullback and hence are equal; thus the V_a glue to an etale $V \to X$ with $V \times_X X_0 \cong V_0$. The same uniqueness shows that maps descend and that the unit and counit of the two functors are identities after pullback. Base change is therefore an equivalence.

It remains to check the topology, the omission in Tag `03SI`. For a family $\{U_i \to U\}$ of etale X-schemes, set $Z = |U| \setminus \bigcup_i \operatorname{im}(|U_i| \to |U|)$. Base change gives a homeomorphism $|U \times_X X_0| \to |U|$, and its inverse image of Z is exactly the complement of the union of the base-changed images. Hence $Z = \varnothing$ if and only if that inverse image is empty. This proves joint surjectivity in both directions. The identity and composition compatibilities already proved show that this correspondence of covering families respects refinements and composite coverings. □

Definition 11.6.11 (The eight curve-cohomology assertions)**.** Uses:

Lean: `conditional-interface:def:weil-pbc-eight-curve-assertions`

Let k be algebraically closed, let X be separated of finite type over k with $\dim X \leq 1$, and let F be a torsion sheaf on X_{et}. Write $\mathcal{C}_r(X, F)$, $1 \leq r \leq 8$, for the following assertions:

(1) $H^q(X, F) = 0$ for $q > 2$;

(2) $H^q(X, F) = 0$ for $q > 1$ when X is affine;

(3) $H^q(X, F) = 0$ for $q > 1$ when $\operatorname{char} k = p > 0$ and F is p-power torsion;

(4) every $H^q(X, F)$ is finite when F is constructible and its torsion is prime to $\operatorname{char} k$;

(5) every $H^q(X, F)$ is finite when X is proper and F is constructible;

(6) for every algebraically closed extension k'/k, the restriction $H^q(X, F) \to H^q(X_{k'}, F|_{X_{k'}})$ is an isomorphism when the torsion of F is prime to $\operatorname{char} k$;

(7) the map in (6) is an isomorphism when X is proper, without a restriction on the torsion;

(8) for every open $a : W \hookrightarrow X$, the restriction $H^2(X, F) \to H^2(W, a^{-1}F)$ is surjective.

Lemma 11.6.12 (Filtered cohomology continuity on a curve)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-pbc-curve-filtered-cohomology`

Let X be quasi-compact and quasi-separated and let (F_λ) be a filtered system of torsion etale sheaves. For every $q \geq 0$ the canonical map

$$\varinjlim_\lambda H^q(X, F_\lambda) \longrightarrow H^q(X, \varinjlim_\lambda F_\lambda)$$

is an isomorphism, naturally in X, the system, and pullback along a field extension.

Proof. Uses:

For $q = 0$, choose a finite etale cover on which a section is represented. The section and its finitely many equalizer equations on pairwise fibre products occur at one index, proving surjectivity; if two representatives become equal in the colimit, the finite cover and equality witnesses occur at one later index, proving injectivity.

For $q > 0$, represent a class on a split etale hypercover whose $(q+1)$-skeleton has finitely many quasi-compact components. A cocycle uses finitely many sections and face-map equations, so all occur at one index λ. This proves surjectivity. If the class from F_λ dies in the colimit, choose a finite refinement and a degree-$(q-1)$ cochain whose coboundary is the cocycle. The refinement maps, cochain sections, and coboundary equations occur at one $\mu \geq \lambda$, so the class already dies in $H^q(X, F_\mu)$. Pullback of the finite hypercover and all its face maps gives the same construction over an extension field, which proves the stated naturality. □

Lemma 11.6.13 (Short-exact devissage for the eight assertions)**.** *Lean:* `conditional-interface:lemma:weil-pbc-eight-curve-devissage`

Uses: `def:weil-pbc-eight-curve-assertions`

Let

$$0 \longrightarrow F_1 \xrightarrow{a} F \xrightarrow{b} F_2 \longrightarrow 0$$

be exact. If all applicable assertions $\mathcal{C}_r(X, F_i)$ hold for $i = 1, 2$, then all applicable assertions hold for F.

Proof. Uses:

Pullback to an extension field is exact, so the displayed sequence and its base change give natural long exact cohomology sequences.

For (1), (2), and (3), in every degree above the stated bound the exact segment

$$H^q(X, F_1) \xrightarrow{a_*} H^q(X, F) \xrightarrow{b_*} H^q(X, F_2)$$

has zero end terms. Hence its middle term is zero. The hypotheses on affineness and characteristic are unchanged, and in (3) an extension of two p-power torsion sheaves is again killed by a power of p.

For (4) and (5), constructible sheaves form a Serre subcategory: on a common finite stratification the stalk sequence is an exact sequence of finite groups. Prime-to-characteristic torsion is also preserved by extensions. Exactness identifies $H^q(X, F)$ as an extension of $\operatorname{im} a_*$ by $\operatorname{im} b_*$; these are respectively a quotient of the finite group $H^q(X, F_1)$ and a subgroup of the finite group $H^q(X, F_2)$. Thus $H^q(X, F)$ is finite.

For (6) and (7), write the long exact sequence for F_1, F, F_2 above its base-changed sequence. Every square commutes because restriction commutes with a, b, and the boundary map obtained from an injective resolution. For fixed q, the four outer vertical arrows in the five-term portion

$$H^{q-1}(F_2) \to H^q(F_1) \to H^q(F) \to H^q(F_2) \to H^{q+1}(F_1)$$

are isomorphisms; the five lemma makes the middle arrow an isomorphism.

For (8), let $c \in H^2(W, F|_W)$ and put $c_2 = b_*(c)$. Choose $x_2 \in H^2(X, F_2)$ restricting to c_2. Its boundary lies in $H^3(X, F_1) = 0$ by (1), so exactness gives $x \in H^2(X, F)$ with $b_*(x) = x_2$. The difference $c - x|_W$ maps to zero in $H^2(W, F_2|_W)$, hence equals $a_*(d_1)$ for some $d_1 \in H^2(W, F_1|_W)$. Choose $y_1 \in H^2(X, F_1)$ restricting to d_1. Then $x + a_*(y_1)$ restricts to c. This proves the eighth assertion and completes all eight cases compressed in Tag `0A5C`. □

Lemma 11.6.14 (Constructible and primary reduction on curves)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-pbc-curve-constructible-primary-reduction`

For a separated finite-type curve X over an algebraically closed field and a torsion sheaf F, extension-invariance of $H^q(X, F)$ reduces to the case where X is reduced and F is a constructible $\mathbf{F}_\ell$-sheaf.

Proof. Uses: `lemma:weil-pbc-universal-homeomorphism-coverings,lemma:weil-pbc-curve-filtered-cohomology,lemma:weil-pbc-eight-curve-devissage`

The reduction map $r : X_{\mathrm{red}} \to X$ is a universal homeomorphism. The site equivalence sends F to $r^{-1}F$, preserves covering families, and therefore identifies the cochain complexes computing all cohomology groups. It also commutes with the corresponding reduction after extending k.

The etale topos of the noetherian scheme X is coherent. Given finitely many local sections of F on quasi-compact etale objects and finitely many relations, their image generates a constructible subsheaf: refine by a finite stratification on which the finitely generated stalk groups are constant. Ordered by inclusion, these constructible subsheaves F_λ form a filtered system with $F = \varinjlim_\lambda F_\lambda$. The preceding continuity lemma gives the natural square

$$\begin{array}{ccc} \varinjlim_\lambda H^q(X, F_\lambda) & \xrightarrow{\sim} & H^q(X, F) \\ \downarrow & & \downarrow \\ \varinjlim_\lambda H^q(X_{k'}, F_\lambda|) & \xrightarrow{\sim} & H^q(X_{k'}, F|). \end{array}$$

Thus comparison for every constructible F_λ implies comparison for F.

Fix a constructible F. A finite stratification with finite stalks gives one exponent N annihilating every stalk, hence $NF = 0$. Write $N = \prod_{\ell | N} \ell^{r_\ell}$. Bezout supplies integers e_ℓ which are 1 modulo ℓ^{r_ℓ} and 0 modulo the other

prime powers. The mutually orthogonal endomorphisms $e_\ell : F \to F$ sum to the identity and give the explicit decomposition

$$F \xrightarrow{\sim} \bigoplus_{\ell \mid N} F[\ell^{r_\ell}], \qquad s \longmapsto (e_\ell s)_\ell,$$

whose inverse is addition. It remains to treat one summand.

If $\ell^r F = 0$, use the exact kernel–quotient sequence

$$0 \longrightarrow F[\ell] \xrightarrow{\iota} F \xrightarrow{\pi} F/F[\ell] \longrightarrow 0.$$

The quotient is killed by ℓ^{r-1} because $\ell^{r-1} s \in F[\ell]$. Pullback to $X_{k'}$ preserves this kernel, quotient, and both maps. Induction on r, with the base case $\ell F = 0$, and the short-exact devissage lemma prove all applicable eight assertions. In particular assertion (7) gives the required proper field-extension invariance, reduced to constructible $\mathbf{F}_\ell$-module sheaves. □

Lemma 11.6.15 (Dense-open localization reduction)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-pbc-curve-dense-open-reduction`

Let X be a reduced curve and F a constructible $\mathbf{F}_\ell$-sheaf. There is a dense open immersion $j : U \hookrightarrow X$ with $U = \coprod_i U_i$ irreducible and $F|_U$ finite locally constant. If $i : Z = X \setminus U \hookrightarrow X$, then

$$0 \longrightarrow j_! j^{-1} F \longrightarrow F \longrightarrow i_* i^{-1} F \longrightarrow 0$$

reduces extension-invariance to $j_! G$ and to sheaves on the zero-dimensional scheme Z.

Proof. Uses: `lemma:weil-pbc-curve-constructible-primary-reduction,lemma:weil-pbc-eight-curve-devissage`

Constructibility gives a finite stratification. Remove the finite closed strata and all intersections of distinct irreducible components. The resulting $U = \coprod_a U_a$ meets every component, each U_a is irreducible, and $G = j^{-1}F$ is finite locally constant.

Let $\epsilon : j_! G \to F$ be extension by zero of the identity of G, and let $\rho : F \to i_* i^{-1} F$ be restriction to Z. At a geometric point of U, the stalks of (ϵ, ρ) are $G_{\bar{x}} \xrightarrow{\mathrm{id}} F_{\bar{x}} \to 0$; at a geometric point of Z they are $0 \to F_{\bar{z}} \xrightarrow{\mathrm{id}} F_{\bar{z}}$. Geometric stalks are exact and conservative, so

$$0 \longrightarrow j_! G \xrightarrow{\epsilon} F \xrightarrow{\rho} i_* i^{-1} F \longrightarrow 0$$

is exact with the displayed maps.

The reduced zero-dimensional scheme Z is a finite disjoint union of k-points. Exact finite direct image and Leray give

$$H^q(X, i_* i^{-1} F) = H^q(Z, i^{-1} F) = 0 \quad (q > 0), \qquad H^0(Z, i^{-1} F) = \bigoplus_{\bar{z} \in Z} F_{\bar{z}}.$$

The last group is finite. The long exact sequence is therefore

$$\begin{aligned} 0 \to H^0(X, j_! G) \to H^0(X, F) \to \bigoplus_{\bar{z} \in Z} F_{\bar{z}} \\ \to H^1(X, j_! G) \to H^1(X, F) \to 0, \end{aligned}$$

and restriction induces $H^q(X, j_! G) \cong H^q(X, F)$ for $q > 1$.

These maps separate the eight assertions as follows. Assertions (1)–(3) and (8) use the isomorphisms in degrees at least two, also after replacing X by an open W and U by $W \cap U$. Assertions (4) and (5) use the displayed five-term sequence and finiteness of the boundary-stalk sum. For (6) and (7), base change preserves Z, its stalk groups, ϵ, ρ, and the boundary map; the two five-term sequences form a commuting diagram, and the five lemma handles degrees $0, 1$, while the higher-degree maps are already identified. Thus the eight assertions hold for F if and only if they hold for $j_! G$.

Finally $j_! G = \bigoplus_a j_{a,!}(G|_{U_a})$. The inclusions and projections split this finite sum, and repeated split exact sequences $0 \to E_1 \to E_1 \oplus E_2 \to E_2 \to 0$ together with short-exact devissage reduce to the individual irreducible components. □

Lemma 11.6.16 (Finite ℓ-group representation filtration)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-pbc-finite-l-group-filtration`

Let H be a finite ℓ-group and V a nonzero finite-dimensional $\mathbf{F}_\ell[H]$-module. Then $V^H \neq 0$ and V has a finite filtration by H-stable subspaces whose successive quotients are the trivial one-dimensional representation.

Proof. Uses:

The group algebra augmentation ideal $J \subset \mathbf{F}_\ell[H]$ is nilpotent: induct on $|H|$, choose a central subgroup C of order ℓ, and use $(c-1)^\ell = c^\ell - 1 = 0$ together with nilpotence for H/C. Choose the least r with $J^r V = 0$; then $J^{r-1} V$ is nonzero and fixed by H. Select a one-dimensional fixed subspace and induct on $\dim V$ after quotienting by it. This fills the future-reference gap in Tag `0A3R`. □

Lemma 11.6.17 (Locally constant sheaf reduction)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-pbc-curve-locally-constant-reduction`

For irreducible U and finite locally constant G of $\mathbf{F}_\ell$-vector spaces, there is a finite etale $v : V \to U$ of degree prime to ℓ such that $v^{-1}G$ has a filtration by constant $\mathbf{F}_\ell$-sheaves, and G is a direct summand of $v_*v^{-1}G$.

Proof. Uses: `lemma:weil-pbc-finite-l-group-filtration,lemma:weil-pbc-finite-exchange`

Choose a geometric point $\bar{u}$ of U. The locally constant sheaf gives a representation

$$\rho : \pi_1(U, \bar{u}) \longrightarrow \mathrm{GL}_{\mathbf{F}_\ell}(G_{\bar{u}})$$

with finite image Q. If $H \subseteq Q$ is an ℓ-Sylow subgroup, the inverse image $\rho^{-1}(H)$ defines a connected finite etale cover $v : V \to U$ of degree $d = [Q : H]$, which is prime to ℓ. On V, the monodromy factors through H.

The representation filtration gives $0 = V_0 \subset V_1 \subset \cdots \subset V_m = G_{\bar{u}}$ with $V_a/V_{a-1} \cong \mathbf{F}_\ell$ and trivial H-action. Passing from H-modules to locally constant sheaves gives, for every a, the explicit short exact sequence

$$0 \longrightarrow G_{a-1} \longrightarrow G_a \longrightarrow \underline{\mathbf{F}_\ell}_V \longrightarrow 0, \qquad G_m = v^{-1}G.$$

All inclusions, quotient maps, and connecting maps commute with extension of the algebraically closed ground field.

The adjunction unit and the fibrewise sum are

$$\eta : G \longrightarrow v_*v^{-1}G, \qquad \mathrm{tr} : v_*v^{-1}G \longrightarrow G.$$

At a geometric stalk, $\eta(s) = (s, \ldots, s)$ in the direct sum indexed by the d points of the fibre, while $\mathrm{tr}(s_1, \ldots, s_d) = \sum_a s_a$. Consequently $\mathrm{tr} \circ \eta = d\,\mathrm{id}_G$. Multiplication by $d^{-1} \in \mathbf{F}_\ell$ defines $r = d^{-1}\,\mathrm{tr}$ with $r\eta = \mathrm{id}_G$. The stalk formulas are preserved by base change, so the direct-summand decomposition and every filtration exact sequence are compatible with the cohomology comparison maps. □

Definition 11.6.18 (Finite-open comparison map)**.** Uses:

Lean: `conditional-interface:def:weil-pbc-finite-open-comparison`

Consider a Cartesian square

$$\begin{array}{ccc} U & \xrightarrow{j_X} & X \\ {\scriptstyle v}\downarrow & & \downarrow{\scriptstyle \bar{v}} \\ V & \xrightarrow{j_Y} & Y \end{array}$$

with j_X, j_Y open immersions and $\bar{v}$ finite. For an abelian sheaf G on U_{et}, define

$$\beta_{\bar{v}, j_Y, G} : (j_Y)_! v_* G \longrightarrow \bar{v}_* (j_X)_! G$$

as the morphism adjoint to

$$v_* G \xrightarrow{\sim} j_Y^{-1} \bar{v}_* (j_X)_! G.$$

The displayed isomorphism is finite base change $j_Y^{-1}\bar{v}_* \cong v_* j_X^{-1}$ followed by $j_X^{-1}(j_X)_! G \cong G$.

Lemma 11.6.19 (Section and stalk inverse for the finite-open map)**.** *Lean:* `conditional-interface:lemma:weil-pbc-finite-open-comparison-inverse`

Uses: `def:weil-pbc-finite-open-comparison`

The map $\beta_{\bar{v}, j_Y, G}$ is an isomorphism. Its inverse is obtained by restricting to U, collecting the finite-fibre tuple on V, and extending that tuple by zero to Y; both inverse equations hold on sections and on geometric stalks.

Proof. Uses:

Let $T \to Y$ be etale and put $T_V = T \times_Y V$, $T_X = T \times_Y X$, and $T_U = T \times_Y U$. A section of $(j_Y)_! v_* G$ over T is etale-locally a finite tuple

$$(s_x)_{x \in T_U / T_V}, \qquad s_x \in G$$

on T_V, extended by zero on $T \setminus T_V$. The section map $\beta(T)$ sends this tuple to the identical tuple on $T_U \subset T_X$ and extends it by zero on $T_X \setminus T_U$. This is a section of $(j_X)_! G$ because the support remains closed after the finite map $T_X \to T$. Restriction along a map $T' \to T$ pulls back every component, so these section maps define a sheaf morphism and agree with the adjunction definition.

Conversely, take $a \in \Gamma(T_X, (j_X)_! G)$. Restrict a to T_U and group its components over the finite fibres of $T_U \to T_V$. Since a vanishes at every geometric point of $T_X \setminus T_U$, the grouped tuple vanishes at every geometric point of $T \setminus T_V$ and therefore defines a section $\gamma(T)(a) \in \Gamma(T, (j_Y)_! v_* G)$. Restriction preserves the grouping, so the $\gamma(T)$ define a sheaf morphism

$$\gamma_{\bar{v}, j_Y, G} : \bar{v}_* (j_X)_! G \longrightarrow (j_Y)_! v_* G.$$

At a geometric point $\bar{y} \to Y$, the two stalks are

$$\begin{cases} \bigoplus_{\bar{x} \mapsto \bar{y}} G_{\bar{x}}, & \bar{y} \in V, \\ 0, & \bar{y} \notin V. \end{cases}$$

Here a point $\bar{x}$ above $\bar{y} \in V$ lies in U by Cartesianity. Both $\beta_{\bar{y}}$ and $\gamma_{\bar{y}}$ are the identity of the displayed finite sum; outside V both are the zero map of the zero group. Hence

$$\gamma_{\bar{v},j_Y,G} \circ \beta_{\bar{v},j_Y,G} = \mathrm{id}_{(j_Y)_! v_* G}, \qquad \beta_{\bar{v},j_Y,G} \circ \gamma_{\bar{v},j_Y,G} = \mathrm{id}_{\bar{v}_*(j_X)_! G}$$

on all geometric stalks. Conservativity of geometric stalks proves both equations as sheaf maps. □

Lemma 11.6.20 (Finite pushforward and extension by zero)**.** *Lean:* `conditional-interface: lemma: weil-pbc-finite-pushforward-extension-zero`

Uses: `def: weil-pbc-finite-open-comparison`

In a Cartesian square with j, j' open immersions and $\bar{v}$ finite,

$$j_! v_* G \xrightarrow{\sim} \bar{v}_* j'_! G$$

is the map of Definition 11.6.18. It is functorial in G, compatible with identity and composite finite maps, and commutes with arbitrary base change.

Proof. Uses: `lemma:weil-pbc-curve-locally-constant-reduction,lemma:weil-pbc-finite-open-comparison-inverse`

The preceding lemma proves that the specified map is an isomorphism and gives both inverse equations.

For a morphism $\phi : G \to H$, naturality is the commutative square

$$\begin{array}{ccc} j_! v_* G & \xrightarrow{\beta_G} & \bar{v}_* j'_! G \\ {\scriptstyle j_! v_* \phi}\downarrow & & \downarrow{\scriptstyle \bar{v}_* j'_! \phi} \\ j_! v_* H & \xrightarrow{\beta_H} & \bar{v}_* j'_! H. \end{array}$$

At $\bar{y} \in V$, both routes send $(s_{\bar{x}})_{\bar{x} \mapsto \bar{y}}$ to $(\phi_{\bar{x}}(s_{\bar{x}}))_{\bar{x} \mapsto \bar{y}}$; outside V all four stalks vanish. Thus the square commutes. The same calculation with the inverse maps γ_G, γ_H proves naturality of the inverse.

If $\bar{v} = \mathrm{id}_Y$ and $v = \mathrm{id}_V$, every fibre sum has one summand. Both the unit–base-change construction of β and the section formula are the identity of $j_! G$. This proves identity compatibility.

For composition, let $f_1 : X_1 \to Y$ and $f_2 : X_2 \to X_1$ be finite. Put $U_i = X_i \times_Y V$, let $g_1 : U_1 \to V$, $g_2 : U_2 \to U_1$, and denote the open immersions by $j_0 : V \to Y$, $j_i : U_i \to X_i$. The compatibility to prove is the outer equality in

$$\begin{array}{ccccc} (j_0)_!(g_1 g_2)_* G & \xrightarrow{\beta_{f_1, g_2, * G}} & f_{1,*}(j_1)_! g_{2,*} G & \xrightarrow{f_{1,*}\beta_{f_2,G}} & f_{1,*} f_{2,*}(j_2)_! G \\ {\scriptstyle \beta_{f_1 f_2, G}}\downarrow{\scriptstyle \sim} & & & & \downarrow{\scriptstyle =} \\ (f_1 f_2)_*(j_2)_! G & = & (f_1 f_2)_*(j_2)_! G & = & (f_1 f_2)_*(j_2)_! G. \end{array}$$

At $\bar{y} \in V$, the upper route first groups a tuple by $\bar{x}_1 \mapsto \bar{y}$ and then by $\bar{x}_2 \mapsto \bar{x}_1$:

$$\bigoplus_{\bar{x}_1 \mapsto \bar{y}} \left(\bigoplus_{\bar{x}_2 \mapsto \bar{x}_1} G_{\bar{x}_2} \right) \longrightarrow \bigoplus_{\bar{x}_2 \mapsto \bar{y}} G_{\bar{x}_2}.$$

It is the canonical flattening of the doubly indexed finite sum. The lower route is the same flattening for the composite fibre. Outside V every term is zero. This proves composition compatibility; applying the reverse grouping proves the corresponding equation for γ.

Finally let $h : \widetilde{Y} \to Y$ be arbitrary and put tildes on all four schemes and maps after base change. If h_X, h_V, h_U are the projections, open and finite base change give the vertical isomorphisms in

$$\begin{array}{ccc} h^{-1}(j_Y)_! v_* G & \xrightarrow{h^{-1}\beta_{\bar{v},j_Y,G}} & h^{-1}\bar{v}_*(j_X)_! G \\ {\scriptstyle \sim}\downarrow & & \downarrow{\scriptstyle \sim} \\ (\widetilde{j}_Y)_! \widetilde{v}_* h_U^{-1} G & \xrightarrow{\beta_{\widetilde{v},\widetilde{j}_Y,h_U^{-1}G}} & \widetilde{\bar{v}}_*(\widetilde{j}_X)_! h_U^{-1} G. \end{array}$$

The left vertical map is $h^{-1}(j_Y)_! \cong (\widetilde{j}_Y)_! h_V^{-1}$ followed by $h_V^{-1} v_* \cong \widetilde{v}_* h_U^{-1}$; the right vertical map uses the same two comparisons in the other order. At a geometric point of $\widetilde{V}$, both routes pull each component $s_{\bar{x}}$ to the corresponding component over the pulled-back geometric fibre. Off $\widetilde{V}$, both routes are zero. The rectangle therefore commutes, and the inverse rectangle follows from the two inverse equations. This supplies the functoriality omitted at Tag `0GJ1`. □

Lemma 11.6.21 (Smooth prime-to-characteristic curve calculation). *Lean:* `conditional-interface: lemma: weil-pbc-smooth-curve-kummer-localization`

Uses: `def: weil-pbc-eight-curve-assertions`

Let k be algebraically closed, let C be a smooth separated finite-type scheme of dimension at most one, and let $\ell \neq \operatorname{char} k$. All eight applicable assertions $\mathcal{C}_r(C, \underline{\mathbf{F}_\ell})$ hold, and every field-extension map and open-restriction map is the one induced by pullback of etale cocycles.

Proof. Uses:

Points have no higher etale cohomology, and finite disjoint unions commute with every map under consideration. We may therefore take C to be a smooth connected curve. Let $\overline{C}$ be its smooth proper compactification, of genus g, and let $D = \overline{C} \setminus C$ consist of r points.

Choose a primitive ℓth root in k, identifying $\underline{\mathbf{F}_\ell}$ with μ_ℓ. The Kummer sequence

$$1 \longrightarrow \mu_\ell \longrightarrow \mathbf{G}_m \xrightarrow{(\cdot)^\ell} \mathbf{G}_m \longrightarrow 1$$

gives

$$H^1(\overline{C}, \mu_\ell) = \operatorname{Pic}(\overline{C})[\ell], \qquad 0 \to \operatorname{Pic}(\overline{C})/\ell \to H^2(\overline{C}, \mu_\ell) \to \operatorname{Br}(\overline{C})[\ell] \to 0.$$

The degree sequence $0 \to \operatorname{Pic}^0(\overline{C})(k) \to \operatorname{Pic}(\overline{C}) \to \mathbf{Z} \to 0$ and the finite etale isogeny $[\ell]$ on the g-dimensional Jacobian give

$$\operatorname{Pic}(\overline{C})[\ell] \cong (\mathbf{Z}/\ell)^{2g}, \qquad \operatorname{Pic}(\overline{C})/\ell \cong \mathbf{Z}/\ell.$$

The Brauer group is zero. To expose the curve input, restriction injects it into $\operatorname{Br}(k(\overline{C}))$ by the discrete-valuation residue sequence. The latter group is zero because $k(\overline{C})$ is C_1: writing the unknowns as polynomials of degree at most N turns a homogeneous equation of degree d in more than d variables into fewer coefficient equations than unknown coefficients for N large; a nonzero solution over the algebraically closed field gives a zero of the original form. Applied to a reduced norm form, this splits every central simple algebra. The same residue sequence and ℓ-cohomological dimension one of this C_1 field give $H^q(\overline{C}, \mu_\ell) = 0$ for $q > 2$. Consequently

$$H^0(\overline{C}, \mathbf{F}_\ell) = \mathbf{F}_\ell, \quad H^1(\overline{C}, \mathbf{F}_\ell) = (\mathbf{F}_\ell)^{2g}, \quad H^2(\overline{C}, \mathbf{F}_\ell) = \mathbf{F}_\ell.$$

Purity for the finite set D is the local DVR calculation: a parameter t identifies the boundary class with the valuation of an ℓth-root Kummer torsor. It gives the exact localization segment

$$0 \longrightarrow H^1(\overline{C}, \mathbf{F}_\ell) \longrightarrow H^1(C, \mathbf{F}_\ell) \xrightarrow{\partial} H^0(D, \mathbf{F}_\ell(-1)) \xrightarrow{\sum \mathrm{tr}_x} H^2(\overline{C}, \mathbf{F}_\ell) \longrightarrow H^2(C, \mathbf{F}_\ell) \longrightarrow 0.$$

The trace sends the basis vector at each $x \in D$ to the fundamental class, so when $r > 0$ it is the summation map $(\mathbf{F}_\ell)^r \to \mathbf{F}_\ell$ and is surjective. Thus a nonproper smooth curve is affine,

$$H^1(C, \mathbf{F}_\ell) \cong (\mathbf{F}_\ell)^{2g+r-1}, \qquad H^q(C, \mathbf{F}_\ell) = 0 \quad (q > 1).$$

Under an algebraically closed extension k'/k, the compactification, genus, boundary points, Kummer sequence, valuation boundary ∂, trace, and the displayed summation map all pull back to their primed counterparts. Hence the vertical maps in the localization diagram are isomorphisms. This proves assertions (1), (2), (4), (5), and (6), and assertion (7) in the proper case. For assertion (8), an open in a proper connected curve is either the whole curve or is affine with zero H^2; opens in an affine curve are affine. The restriction $H^2(C) \to H^2(W)$ is therefore either the identity or the unique map onto zero. All claimed maps are the maps in the Kummer and localization sequences, so their naturality is fixed. □

Lemma 11.6.22 (Smooth characteristic-p Artin–Schreier calculation). *Lean:* `conditional-interface: lemma: weil-pbc-smooth-curve-artin-schreier`

Uses: `def: weil-pbc-eight-curve-assertions`

Let k be algebraically closed of characteristic $p > 0$ and let C be a smooth separated finite-type scheme of dimension at most one. All applicable assertions $\mathcal{C}_r(C, \underline{\mathbf{F}_p})$ hold. For proper C, invariance under algebraically closed extension is induced by the kernel of Frobenius minus the identity on $H^1(C, \mathcal{O}_C)$.

Proof. Uses:

The Artin–Schreier sequence on the etale site is

$$0 \longrightarrow \underline{\mathbf{F}_p} \longrightarrow \mathcal{O}_C \xrightarrow{F-1} \mathcal{O}_C \longrightarrow 0.$$

Exactness on the right is checked on an etale neighbourhood by adjoining a root of $T^p - T - a$; its derivative is -1, so that neighbourhood is etale. Etale cohomology of the quasi-coherent sheaf $\mathcal{O}_C$ equals its Zariski cohomology.

If C is affine, $H^q(C,\mathcal{O}_C)=0$ for $q>0$. The long exact sequence gives

$$H^1(C,\mathbf{F}_p)=\operatorname{coker}\bigl(F-1:\Gamma(C,\mathcal{O}_C)\to\Gamma(C,\mathcal{O}_C)\bigr),\qquad H^q(C,\mathbf{F}_p)=0\quad(q>1).$$

This proves assertions (1)–(3) and (8) in the affine case; no field-extension assertion for affine p-torsion is claimed.

Suppose C is proper and connected. Then $H^0(C,\mathcal{O}_C)=k$, $H^q(C,\mathcal{O}_C)=0$ for $q>1$, and $E=H^1(C,\mathcal{O}_C)$ has dimension g. Since $a\mapsto a^p-a$ is surjective on the algebraically closed field k, the long exact sequence reduces to

$$H^1(C,\mathbf{F}_p)=\ker(F-1:E\to E),\qquad H^2(C,\mathbf{F}_p)=\operatorname{coker}(F-1:E\to E).$$

Choose a basis of E. The additive map $F-1$ is a morphism $\mathbf{A}_k^g\to\mathbf{A}_k^g$ whose derivative is $-\mathrm{id}$. Its kernel is therefore zero-dimensional etale and finite. The image is an open subgroup of the connected additive group $\mathbf{A}_k^g$, hence is the whole group. Thus the cokernel is zero and the kernel is a finite $\mathbf{F}_p$-vector space.

Proper coherent base change identifies $E\otimes_k k'$ with $H^1(C_{k'},\mathcal{O}_{C_{k'}})$ and carries Frobenius to Frobenius. The finite etale kernel of $F-1$ is a finite disjoint union of copies of $\operatorname{Spec}k$; after an algebraically closed extension it has the same geometric points. Hence $H^1(C,\mathbf{F}_p)\to H^1(C_{k'},\mathbf{F}_p)$ is an isomorphism, while degree zero is $\mathbf{F}_p$ and all degrees above one vanish. This proves (1), (3), (5), (7), and (8) in the proper case, with the exact comparison map specified by the Artin–Schreier sequence. □

Lemma 11.6.23 (Normalization comparison exact sequence). *Lean:* `conditional-interface: lemma: weil-pbc-normalization-exact-sequence`

Uses: `def: weil-pbc-finite-open-comparison`

Let X be a reduced curve over an algebraically closed field, $\nu:X^\nu\to X$ its finite normalization, $j:U\hookrightarrow X$ open, and $j^\nu:U^\nu\hookrightarrow X^\nu$ its pullback. For every prime ℓ, the unit and the finite-open comparison define a monomorphism

$$\alpha:j_!\underline{\mathbf{F}_\ell}\longrightarrow\nu_*j^\nu_!\underline{\mathbf{F}_\ell}$$

whose cokernel Q is constructible and supported on the finite nonnormal locus. The sequence

$$0\longrightarrow j_!\underline{\mathbf{F}_\ell}\xrightarrow{\alpha}\nu_*j^\nu_!\underline{\mathbf{F}_\ell}\longrightarrow Q\longrightarrow 0$$

and all its cohomology boundary maps commute with algebraically closed field extension and restriction to opens.

Proof. Uses: `lemma:weil-pbc-finite-pushforward-extension-zero,lemma:weil-pbc-eight-curve-devissage`

Let $v:U^\nu\to U$. Define α as

$$j_!\underline{\mathbf{F}_\ell}\xrightarrow{j_!\eta}j_!v_*\underline{\mathbf{F}_\ell}\xrightarrow{\beta_{\nu,j}}\nu_*j^\nu_!\underline{\mathbf{F}_\ell},$$

where η is the unit of $v^{-1}\dashv v_*$ and β is the comparison map already constructed. At $\bar x\in U$, its stalk is the diagonal

$$\mathbf{F}_\ell\longrightarrow\bigoplus_{\bar z\in\nu^{-1}(\bar x)}\mathbf{F}_\ell,\qquad a\longmapsto(a)_{\bar z};$$

outside U, both stalks are zero. Normalization is surjective, so the diagonal is injective. Over the dense open where ν is an isomorphism it is an isomorphism. Therefore the cokernel Q is supported on the finite nonnormal locus, with stalk the quotient of the displayed finite sum by its diagonal. This proves exactness and constructibility.

After an algebraically closed extension, X^ν remains smooth and normal and is the normalization of the reduced base-changed curve. The unit stalk map remains the diagonal, while the base-change rectangle for β identifies the two pulled-back comparison maps. Thus the short exact sequence pulls back to the primed sequence. Naturality of the connecting homomorphism follows by applying compatible injective resolutions. For an open $W\subset X$, finite base change replaces ν by $\nu^{-1}W\to W$ and gives the same stalk diagonals, proving compatibility with open restriction.

Finally $H^q(X,Q)=0$ for $q>0$, and $H^0(X,Q)$ is the finite direct sum of its stalks; both statements persist after base change. Exact finite direct image and Leray identify $H^q(X,\nu_*j^\nu_!\mathbf{F}_\ell)$ with $H^q(X^\nu,j^\nu_!\mathbf{F}_\ell)$. Hence the displayed long exact sequence transfers vanishing, finiteness, field-extension isomorphisms, and open-restriction surjectivity from the normalized term to $j_!\mathbf{F}_\ell$: in degrees at least two the maps are isomorphisms, and in degrees zero and one the finite sequence

$$0\to H^0(j_!\mathbf{F}_\ell)\to H^0(\nu_*j^\nu_!\mathbf{F}_\ell)\to H^0(Q)\to H^1(j_!\mathbf{F}_\ell)\to H^1(\nu_*j^\nu_!\mathbf{F}_\ell)\to 0$$

gives the comparison by a five-lemma diagram. □

Lemma 11.6.24 (Passage between separably closed fields). *Uses:*

Lean: `conditional-interface: lemma: weil-pbc-curve-separably-closed-passage`

Let $k\subseteq k'$ be separably closed fields, let C be proper of dimension at most one over k, and let F be a torsion etale sheaf. If proper-curve cohomology is invariant for algebraically closed extensions, then the specified restriction $H^q(C,F)\to H^q(C_{k'},F|)$ is an isomorphism for every q.

Proof. Uses: `lemma:weil-pbc-universal-homeomorphism-coverings`

Choose compatible algebraic closures $\bar{k} \subseteq \bar{k}'$. Since k and k' are separably closed, $\bar{k}/k$ and $\bar{k}'/k'$ are purely inseparable. Their spectrum maps are universal homeomorphisms; the explicit site equivalence and covering compatibility give the vertical isomorphisms in

$$\begin{array}{ccc} H^q(C, F) & \longrightarrow & H^q(C_{k'}, F|) \\ {\scriptstyle\sim}\downarrow & & \downarrow{\scriptstyle\sim} \\ H^q(C_{\bar{k}}, F|) & \longrightarrow & H^q(C_{\bar{k}'}, F|). \end{array}$$

The lower arrow is an isomorphism by the algebraically closed case. The square commutes because each route pulls a representative cocycle through the same composite field map; identity and composition compatibility of the site equivalence identify the two pullbacks. The upper arrow is therefore an isomorphism. In particular spectra of separably closed fields, which are strictly henselian local schemes, introduce no additional stalk ambiguity. □

Lemma 11.6.25 (Normalization and smooth constant case)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-pbc-curve-normalization-constant`

For a reduced curve, extension-invariance for $j_!\underline{\mathbf{F}}_\ell$ reduces through the finite normalization to the corresponding calculation on a smooth curve, both for proper and affine opens and also when $\ell = \operatorname{char} k$.

Proof. Uses: `lemma:weil-pbc-smooth-curve-kummer-localization,lemma:weil-pbc-smooth-curve-artin-schreier,lemma:weil-pbc-normalization-exact-sequence,lemma:weil-pbc-curve-dense-open-reduction`

The normalization X^ν is a finite disjoint union of smooth curves and points. On a component disjoint from U^ν the extension-by-zero sheaf is zero and every assertion is immediate; on every other component U^ν is dense. The dense-open localization lemma on X^ν therefore reduces $j_!^\nu\underline{\mathbf{F}}_\ell$ to the constant sheaf on each remaining smooth component, with the five-term sequence and all comparison maps already specified there. If $\ell \neq \operatorname{char} k$, apply the Kummer and localization calculation; if $\ell = \operatorname{char} k$, apply the Artin–Schreier calculation. Thus all applicable eight assertions hold for $j_!^\nu\underline{\mathbf{F}}_\ell$.

Exact finite direct image transfers them to $\nu_* j_!^\nu\underline{\mathbf{F}}_\ell$. The normalization exact sequence then transfers them to $j_!\underline{\mathbf{F}}_\ell$, including the low-degree comparison maps and assertion (8). Properness is preserved by finite normalization; affineness is preserved because a finite morphism over an affine is affine. This completes the normalization, proper/affine localization, prime-to-characteristic, and characteristic-ℓ branches without an unnamed smooth-curve calculation. □

Proposition 11.6.26 (Proper curve invariance over separably closed fields)**.** *Uses:*

Lean: `conditional-interface:prop:weil-pbc-proper-curve-sepclosed-invariance`

Let $k \subseteq k'$ be separably closed fields, C a proper k-scheme of dimension at most one, and F a torsion etale sheaf. Then

$$H^q(C, F) \xrightarrow{\sim} H^q(C_{k'}, F|)$$

for every q.

Proof. Uses: `lemma:weil-pbc-universal-homeomorphism-coverings,lemma:weil-pbc-curve-constructible-primary-reduction,lemma:weil-pbc-curve-dense-open-reduction,lemma:weil-pbc-curve-locally-constant-reduction,lemma:weil-pbc-finite-pushforward-extension-zero,lemma:weil-pbc-curve-normalization-constant,lemma:weil-pbc-curve-separably-closed-passage`

First assume both fields algebraically closed. Apply constructible/primary reduction. The closed immersion $C_{\mathrm{red}} \to C$ is a universal homeomorphism, so the explicit site equivalence first replaces C by its reduction. Then apply dense-open localization, the prime-to-ℓ trace splitting, the finite-pushforward comparison, and the normalized smooth constant calculation in that order. Every step commutes with field extension, giving the result. For separably closed $k \subseteq k'$, apply Lemma 11.6.24 to the algebraically closed result just proved. Its commutative rectangle identifies the resulting isomorphism with the specified restriction map. This expands Tags `03SC` and `0A5E`. □

11.6.3 S5.3. Stalkwise projective-line base change

Lemma 11.6.27 (Exchange on a geometric stalk)**.** *Lean:* `conditional-interface:lemma:weil-pbc-p1-exchange-geometric-stalk`

Uses: `def:weil-pbc-derived-exchange`

For $p : \mathbf{P}^1_S \to S$, a base change $g : T \to S$, and $\bar{t} \to T$ with image $\bar{s} \to S$, the stalk of $\mathrm{Ex}_{\square,F}$ is the restriction map

$$H^q(\mathbf{P}^1_{\kappa(\bar{s})}, F|) \longrightarrow H^q(\mathbf{P}^1_{\kappa(\bar{t})}, F|).$$

Proof. Uses: prop:weil-pbc-strict-henselization-stalk-formula,prop:weil-pbc-p1-strict-henselian-comparison

Let $\bar{t} \to T$ be represented by an algebraically closed field Ω, and let $\bar{s} \to S$ be its composite. Put $S^{\mathrm{sh}} = \operatorname{Spec} \mathcal{O}^{\mathrm{sh}}_{S,\bar{s}}$ and $T^{\mathrm{sh}} = \operatorname{Spec} \mathcal{O}^{\mathrm{sh}}_{T,\bar{t}}$. The map $T \to S$ induces a local map $T^{\mathrm{sh}} \to S^{\mathrm{sh}}$ and a separably closed residue-field extension $\kappa(\bar{s}) \hookrightarrow \kappa(\bar{t})$. For an injective resolution $F \to I^\bullet$, the strict-henselization construction gives the following diagram in $D^+(\mathbf{Ab})$:

$$\begin{array}{ccc} (g^{-1}Rp_*F)_{\bar{t}} & \xrightarrow{(\mathrm{Ex}_{\square,F})_{\bar{t}}} & (Rp'_*g'^{-1}F)_{\bar{t}} \\ {\scriptstyle \Phi_S}\downarrow{\scriptstyle \sim} & & \downarrow{\scriptstyle \Phi_T} \quad \sim V \\ R\Gamma(\mathbf{P}^1_{S^{\mathrm{sh}}}, I|) & \xrightarrow{\lambda} & R\Gamma(\mathbf{P}^1_{T^{\mathrm{sh}}}, I|) \\ {\scriptstyle \rho_S}\downarrow{\scriptstyle \sim} & & \downarrow{\scriptstyle \rho_T} \quad \sim V \\ R\Gamma(\mathbf{P}^1_{\kappa(\bar{s})}, I_{\bar{s}}) & \xrightarrow{\kappa(\bar{s})\subset\kappa(\bar{t})} & R\Gamma(\mathbf{P}^1_{\kappa(\bar{t})}, I_{\bar{t}}). \end{array}$$

The maps Φ_S and Φ_T are the stalk-to-strict-local maps from the strict-henselization formula. The maps ρ_S and ρ_T are the S4 restriction quasi-isomorphisms, applied after replacing $\mathbf{P}^1$ by its base change to the indicated strict local ring. The bottom arrow is induced by the residue-field map.

To check the upper square, take a representative section of I^r on an affine pointed etale neighbourhood $V \to S$. The left route first applies the unit $I^r \to g'_*g'^{-1}I^r$ and then restricts along $\mathbf{P}^1_T \times_T (V \times_S T)$. The right route first restricts to $\mathbf{P}^1_{S^{\mathrm{sh}}}$ and then along $T^{\mathrm{sh}} \to S^{\mathrm{sh}}$. Both are the same section on $\mathbf{P}^1_{T^{\mathrm{sh}}} \times_{T^{\mathrm{sh}}} V$, so the square commutes degree by degree and hence in the derived category. The lower square is the identical calculation after passing to the closed geometric fibres. Therefore the stalk of the named S0 exchange is exactly the field extension map displayed in the bottom row, not an abstract replacement. □

Lemma 11.6.28 (Conservativity of geometric stalks)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-pbc-geometric-stalk-conservative`

A morphism of etale abelian sheaves is an isomorphism if and only if it is an isomorphism at every geometric stalk.

Proof. Uses:

The kernel and cokernel have zero geometric stalks. If a sheaf has zero stalks, every local section vanishes on an etale neighbourhood of each point; quasi-compact refinement on an etale object makes it locally zero on a cover, hence zero by the sheaf axiom. Thus kernel and cokernel vanish. □

Proposition 11.6.29 (Projective-line proper base change)**.** *Uses:*

Lean: `conditional-interface:prop:weil-pbc-projective-line`

For every scheme S, the projection $\mathbf{P}^1_S \to S$ satisfies PBC.

Proof. Uses: lemma:weil-pbc-p1-exchange-geometric-stalk,prop:weil-pbc-proper-curve-sepclosed-invariance,lemma:weil-pbc-geometric-stalk-conservative,lemma:weil-pbc-property-iff-acyclicity

At every geometric $\bar{t} \to T$, the exchange map is the field-extension map of the preceding lemma. The two residue fields are separably closed and the fibre is a proper curve, so proper-curve invariance makes this map an isomorphism. Geometric stalk conservativity gives an isomorphism of every R^q sheaf. Specializing to inverse images of injective $\mathbf{F}_\ell$-module sheaves gives IA after every base change; the S0 equivalence gives PBC. □

11.7 S6. Finite-torsion proper base change

Lemma 11.7.1 (Prime-coefficient proper base change)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-pbc-prime-proper-base-change`

Every proper morphism satisfies proper base change for sheaves of $\mathbf{F}_\ell$-modules.

Proof. Uses: prop:weil-pbc-reduction-to-projective-line,prop:weil-pbc-projective-line

The projective-line proposition supplies the sole geometric input required by the S3 reduction. Finite morphisms, composition, and cancellation in that reduction preserve the named S0 exchange map, so the resulting isomorphism is the exchange map for the original proper morphism. □

Lemma 11.7.2 (Torsion coefficient assembly)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-pbc-torsion-assembly`

For every torsion abelian sheaf F and every proper Cartesian square, the specific map $\mathrm{Ex}_{\square,F}$ is an isomorphism.

Proof. Uses: `lemma:weil-pbc-prime-proper-base-change,lemma:weil-pbc-torsion-filtered-union,lemma:weil-pbc-direct-image-filtered-colimit,lemma:weil-pbc-prime-factor-induction,lemma:weil-pbc-five-lemma-transfer`

Write $F = \varinjlim_n F[n]$ and commute both sides of exchange with the filtered colimit. Induct on the number of prime factors of n through $0 \to F[\ell] \to F[n] \to F[n]/F[\ell] \to 0$; the quotient is killed by n/ℓ. Prime coefficients are handled by the preceding lemma, and the five-lemma transfer handles the induction step. Naturality shows at every stage that the comparison is the S0 exchange map. □

Proposition 11.7.3 (Finite-torsion proper base change). *Lean:* `conditional-interface:prop:weil-pbc-proet-finite-proper-base-change`

Uses: `def:weil-pbc-derived-exchange`

Let

$$\begin{array}{ccc} X' & \xrightarrow{g'} & X \\ {\scriptstyle f'}\downarrow & & \downarrow{\scriptstyle f} \\ Y' & \xrightarrow{g} & Y \end{array}$$

be Cartesian with f proper. For every torsion sheaf F on X_{et}, the canonical exchange map

$$g^{-1}Rf_*F \xrightarrow{\sim} Rf'_*g'^{-1}F$$

is an isomorphism. These isomorphisms are natural in F, equal the identity for identity squares, and commute with horizontal and vertical pasting.

Proof. Uses: `lemma:weil-pbc-derived-exchange-coherence,lemma:weil-pbc-torsion-assembly`

The torsion assembly proves the isomorphism in every cohomological degree for the map constructed in S0. A morphism in D^+ is an isomorphism exactly when all its cohomology sheaf maps are isomorphisms. Naturality, identity, and pasting are the corresponding S0 coherence equations and survive each devissage step. This is the expanded endpoint whose source locator is Tag `095T`; that tag is not used as a proof leaf. □

> **Status.** Proposition 11.7.3 is the stable A3 finite-torsion export. It remains conditional until every OPEN in S0–S5 on which its proof depends is discharged.

11.8 S7. Bounded-below finite-torsion complexes

Definition 11.8.1 (Hypercohomology filtrations). *Lean:* `conditional-interface:def:weil-pbc-hypercohomology-filtrations`

Uses: `def:weil-pbc-derived-exchange`

Let $K \in D^+(X_{\mathrm{et}})$ have torsion cohomology sheaves and choose a with $\mathcal{H}^q(K) = 0$ for $q < a$. The canonical truncation tower $\tau_{\le q}K$ defines spectral sequences

$$E_2^{p,q} = R^p f_* \mathcal{H}^q(K) \Longrightarrow \mathcal{H}^{p+q}(Rf_*K),$$

and

$$E'^{p,q}_2 = R^p f'_* g'^{-1} \mathcal{H}^q(K) \Longrightarrow \mathcal{H}^{p+q}(Rf'_*g'^{-1}K).$$

Their filtration in total degree n is the image filtration induced by $\mathcal{H}^n(Rf_*\tau_{\le q}K) \to \mathcal{H}^n(Rf_*K)$; the second filtration is induced by $\mathcal{H}^n(Rf'_*g'^{-1}\tau_{\le q}K) \to \mathcal{H}^n(Rf'_*g'^{-1}K)$.

Lemma 11.8.2 (Half-plane convergence). *Lean:* `conditional-interface:lemma:weil-pbc-hypercohomology-convergence`

Uses: `def:weil-pbc-hypercohomology-filtrations`

The spectral sequences lie in $p \ge 0$, $q \ge a$. For fixed total degree n, only $a \le q \le n$ and $0 \le p \le n - a$ occur; hence the induced filtration is finite, exhaustive, separated, and converges to the displayed abutment.

Proof. Uses:

Right-derived direct image has no negative cohomological degree, giving $p \ge 0$. The lower bound gives $q \ge a$. On $p+q = n$ these inequalities give the finite interval. The truncation triangles show exhaustivity, and finiteness shows separatedness; no inverse-limit convergence issue occurs. □

Definition 11.8.3 (The two truncation exact couples). Lean: `conditional-interface:def:weil-pbc-truncation-exact-couples`

Uses: `def:weil-pbc-hypercohomology-filtrations,def:weil-pbc-derived-exchange`

For every $q \geq a$, set

$$A_q = g^{-1}Rf_*\tau_{\leq q}K, \quad G_q = g^{-1}Rf_*\mathcal{H}^q(K)[-q],$$
$$B_q = Rf'_*g'^{-1}\tau_{\leq q}K, \quad H_q = Rf'_*g'^{-1}\mathcal{H}^q(K)[-q].$$

The canonical truncation triangle and its exchanged image are

$$\begin{array}{ccccccc} A_{q-1} & \xrightarrow{i_q} & A_q & \xrightarrow{j_q} & G_q & \xrightarrow{k_q} & A_{q-1}[1] \\ \downarrow{\scriptstyle e_{q-1}} & & \downarrow{\scriptstyle e_q} & & \downarrow{\scriptstyle \bar{e}_q} & & \downarrow{\scriptstyle e_{q-1}[1]} \\ B_{q-1} & \xrightarrow{i'_q} & B_q & \xrightarrow{j'_q} & H_q & \xrightarrow{k'_q} & B_{q-1}[1]. \end{array}$$

For total degree n, define

$$D_A^{q,n} = \mathcal{H}^n(A_q), \quad \mathcal{E}_A^{p,q} = \mathcal{H}^{p+q}(G_q) = g^{-1}R^pf_*\mathcal{H}^q(K),$$
$$D_B^{q,n} = \mathcal{H}^n(B_q), \quad \mathcal{E}_B^{p,q} = \mathcal{H}^{p+q}(H_q) = R^pf'_*g'^{-1}\mathcal{H}^q(K).$$

The exact-couple differential is

$$d_1 = (j_{q-1})_* \circ (k_q)_* : \mathcal{E}_A^{p,q} \longrightarrow \mathcal{E}_A^{p+2,q-1},$$

and likewise $d'_1 = (j'_{q-1})_*(k'_q)_*$. This first exact-couple page is the page conventionally denoted E_2 in the hypercohomology indexing.

Lemma 11.8.4 (Hypercover model and E_1-map). Lean: `conditional-interface:lemma:weil-pbc-hypercover-exact-couple-map`

Uses: `def:weil-pbc-truncation-exact-couples`

On every split etale hypercover, the exchange maps define a morphism between the two exact couples. Its first exact-couple page is

$$\bar{e}_1^{p,q} : g^{-1}R^pf_*\mathcal{H}^q(K) \longrightarrow R^pf'_*g'^{-1}\mathcal{H}^q(K),$$

the S6 finite-torsion map, and $d'_1\bar{e}_1^{p,q} = \bar{e}_1^{p+2,q-1}d_1$ with no residual sign.

Proof. Uses: `def:weil-pbc-truncation-exact-couples,prop:weil-proet-finite-proper-base-change`

Evaluate the two rows of truncation triangles on an etale object and choose a split etale hypercover $U_\bullet$. A Cartan–Eilenberg representative gives on each side a double complex $C^{m,r}$, where m is the cosimplicial degree and r the complex degree. Write

$$d_{\mathrm{simp}} = \sum_{t=0}^{m+1}(-1)^t d_t^*, \qquad d_{\mathrm{tot}} = d_{\mathrm{simp}} + (-1)^m d_{\mathrm{int}}.$$

The simplicial identities give $d_{\mathrm{simp}}^2 = 0$; naturality of the face maps gives $d_{\mathrm{simp}}d_{\mathrm{int}} = d_{\mathrm{int}}d_{\mathrm{simp}}$, and the factor $(-1)^m$ makes $d_{\mathrm{tot}}^2 = 0$. The left and right totalizations use this same sign convention.

The sectionwise exchange map pulls a section on every U_m through the Cartesian square. It commutes with every face and degeneracy map and with d_{int}, hence is a chain map of the two total complexes. Filtering by canonical truncation gives exactly the objects A_q, G_q and B_q, H_q of Definition 11.8.3. Naturality with respect to $\tau_{\leq q-1}K \to \tau_{\leq q}K$ gives the first three squares of the displayed triangle.

For the last square, use the cohomological cone convention

$$\mathrm{Cone}(u)^n = Y^n \oplus X^{n+1}, \qquad d(y,x) = (d_Y y + u(x), -d_X x).$$

The connecting map is induced by $(y,x) \mapsto x$. Applying exchange before or after this projection gives $e_{q-1}[1](x)$ in both routes; the minus sign belongs to the shifted differential on both sides and does not change the map. Thus $k'_q\bar{e}_q = e_{q-1}[1]k_q$, with no extra sign.

Taking cohomology yields a map of exact couples. On the graded object $\mathcal{H}^q(K)[-q]$, degree $p+q$ cohomology is $g^{-1}R^pf_*\mathcal{H}^q(K)$ on the left and $R^pf'_*g'^{-1}\mathcal{H}^q(K)$ on the right. The induced map is the S0 exchange for that torsion sheaf and therefore the specified S6 map. Finally

$$\begin{aligned} d'_1\bar{e}_1 &= (j'_{q-1})_*(k'_q)_*\bar{e}_1 \\ &= (j'_{q-1})_*(e_{q-1}[1])_*(k_q)_* = \bar{e}_1(j_{q-1})_*(k_q)_* = \bar{e}_1 d_1, \end{aligned}$$

which proves differential compatibility. □

Lemma 11.8.5 (Finite rows and the abutment comparison)**.** *Lean:* `conditional-interface:lemma:weil-pbc-exact-couple-abutment`

Uses: `def:weil-pbc-hypercohomology-filtrations`, `def:weil-pbc-truncation-exact-couples`

In total degree n, the two exact couples have only the finite row range $a \le q \le n$. Their induced abutment map is the cohomology map

$$\mathcal{H}^n(g^{-1}Rf_*K) \longrightarrow \mathcal{H}^n(Rf'_*g'^{-1}K)$$

of the derived exchange morphism, with the image filtrations of Definition 11.8.1.

Proof. Uses: `lemma:weil-pbc-hypercohomology-convergence`

On $\mathcal{E}_1^{p,q}$ one has $p \ge 0$ and $q \ge a$. The equality $p+q=n$ therefore forces $a \le q \le n$. On a hypercover total complex, only these finitely many filtered quotients contribute to degree n. Consequently direct-sum and product totalization agree in that degree, the filtration is finite and separated, and passage from a finite skeleton to the full hypercover changes none of its graded terms.

The maps $e_q : A_q \to B_q$ commute with $\tau_{\le q}K \to K$. For $q \ge n$, the omitted cohomology sheaves of K have degrees greater than n; after a nonnegative right-derived functor they cannot contribute to total degree n. Hence

$$\mathcal{H}^n(A_q) \xrightarrow{\sim} \mathcal{H}^n(g^{-1}Rf_*K), \qquad \mathcal{H}^n(B_q) \xrightarrow{\sim} \mathcal{H}^n(Rf'_*g'^{-1}K).$$

Under these identifications, $\mathcal{H}^n(e_q)$ is $\mathcal{H}^n(\mathrm{Ex}_{\square,K})$ by naturality of exchange in the truncation map. Its filtration is the image of $\mathcal{H}^n(A_q)$ and maps to the image of $\mathcal{H}^n(B_q)$, exactly the two filtrations defined earlier. Thus the exact-couple abutment map is the specified derived exchange map, rather than an unnamed filtered isomorphism. □

Lemma 11.8.6 (Morphism of hypercohomology spectral sequences)**.** *Lean:* `conditional-interface:lemma:weil-pbc-hypercohomology-map`

Uses: `def:weil-pbc-hypercohomology-filtrations`, `def:weil-pbc-truncation-exact-couples`

The S0 exchange map induces a morphism from the inverse image of the first spectral sequence to the second, whose E_2 component is

$$g^{-1}R^pf_*\mathcal{H}^q(K) \longrightarrow R^pf'_*g'^{-1}\mathcal{H}^q(K),$$

the S6 finite-torsion proper-base-change map.

Proof. Uses: `def:weil-pbc-truncation-exact-couples`, `lemma:weil-pbc-hypercohomology-convergence`, `lemma:weil-pbc-hypercover-exact-couple-map`, `lemma:weil-pbc-exact-couple-abutment`, `prop:weil-proet-finite-proper-base-change`

Definition 11.8.3 displays both exact couples and every structure map. Lemma 11.8.4 constructs their morphism, identifies its first page with the S6 map, and checks the differential and connecting-map sign. Lemma 11.8.5 proves finite-row convergence and identifies the abutment map with $\mathcal{H}^n(\mathrm{Ex}_{\square,K})$. Deriving the exact couples therefore gives the asserted morphism on every page. □

Proposition 11.8.7 (Bounded-below finite derived proper base change)**.** *Uses:*

Lean: `conditional-interface:prop:weil-proet-finite-derived-proper-base-change`

For a proper Cartesian square and $K \in D^+(X_{\mathrm{et}})$ with torsion cohomology, the exchange map

$$g^{-1}Rf_*K \xrightarrow{\sim} Rf'_*g'^{-1}K$$

is an isomorphism.

Proof. Uses: `lemma:weil-pbc-hypercohomology-convergence`, `lemma:weil-pbc-hypercohomology-map`, `prop:weil-proet-finite-proper-base-change`

S6 makes every E_2 component of the spectral-sequence morphism an isomorphism. Induction on the page gives isomorphisms on E_r and E_∞. In each total degree the finite filtrations have isomorphic associated graded pieces, so induction on the filtration length gives an isomorphism of abutments. A morphism in the derived category is an isomorphism when every cohomology sheaf map is an isomorphism. This expands Tag `0DDE`, including its exact-couple comparison. □

11.9 S8. Etale-to-pro-etale finite-level transport

Definition 11.9.1 (Etale objects inside the pro-etale site). Lean: `conditional-interface:def:weil-pbc-etale-proetale-inclusion`

Uses: `def:weil-small-etale-topology`, `def:weil-small-etale-pretopology`, `def:weil-small-proetale-category`, `def:weil-small-proetale-precoverage`, `def:weil-small-proetale-topology`

For a scheme X, define the assignment

$$\iota_X : X_{\mathrm{et}} \longrightarrow X_{\mathrm{proet}}$$

as follows. An object $p : U \to X$, etale in the A2 small etale category, is sent to the same structural morphism p. It is an object of the A2 small pro-etale category because an etale morphism is weakly etale. An X-morphism $h : U \to V$ is sent to the same scheme morphism h. Thus both forgetful functors to $\mathrm{Over}(X)$ agree on this assignment.

Lemma 11.9.2 (Identity law for the etale-to-pro-etale assignment). Lean: *`conditional-interface:lemma:weil-efi-site-inclusion-identity`*

Uses: *`def:weil-pbc-etale-proetale-inclusion`*

For every etale object $U \to X$,

$$\iota_X(1_U) = 1_{\iota_X U}.$$

Proof. Uses: `def:weil-pbc-etale-proetale-inclusion`

Both arrows have underlying scheme morphism 1_U, and morphisms in the selected over-category are determined by their underlying scheme maps. □

Lemma 11.9.3 (Composition law for the etale-to-pro-etale assignment). Lean: *`conditional-interface:lemma:weil-efi-site-inclusion-composition`*

Uses: *`def:weil-pbc-etale-proetale-inclusion`*

For composable etale-site arrows $U \xrightarrow{h} V \xrightarrow{k} W$,

$$\iota_X(kh) = \iota_X(k)\iota_X(h).$$

Proof. Uses: `def:weil-pbc-etale-proetale-inclusion`

Each side has underlying scheme morphism kh. Equality follows from the faithful forgetful functor to $\mathrm{Over}(X)$. □

Definition 11.9.4 (Functor-law certificate for the site assignment). Lean: `conditional-interface:def:weil-efi-site-inclusion-functor-certificate`

Uses: `def:weil-pbc-etale-proetale-inclusion`

Let $\mathsf{FunctorLaw}(\iota_X)$ be the conjunction of the identity equation for every etale object and the composition equation for every composable pair of etale-site arrows.

Proposition 11.9.5 (The site assignment satisfies its functor laws). Lean: *`conditional-interface:prop:weil-efi-site-inclusion-functor`*

Uses: *`def:weil-efi-site-inclusion-functor-certificate`*, *`def:weil-pbc-etale-proetale-inclusion`*

The predicate $\mathsf{FunctorLaw}(\iota_X)$ holds. Hence the assignment of Definition 11.9.1 is a functor.

Proof. Uses: `lemma:weil-efi-site-inclusion-identity`, `lemma:weil-efi-site-inclusion-composition`

The first lemma supplies the identity field and the second supplies the composition field of the displayed predicate. □

Definition 11.9.6 (Image of an etale sieve in the pro-etale site). Lean: `conditional-interface:def:weil-efi-site-covering-sieve-image`

Uses: `def:weil-pbc-etale-proetale-inclusion`

For an etale object $U \to X$ and a sieve $R \subset h_U$, let $\iota^{\mathrm{sieve}}_{X,!}R$ be the sieve on $\iota_X U$ generated by the arrows $\iota_X V \to \iota_X U$ with $V \to U$ in R.

Lemma 11.9.7 (Etale covering families remain pro-etale covering). Lean: *`conditional-interface:lemma:weil-efi-site-covering-family-image`*

Uses: *`def:weil-pbc-etale-proetale-inclusion`*

The image under ι_X of an etale covering family is a pro-etale covering family.

Proof. Uses: `def:weil-pbc-etale-proetale-inclusion`, `def:weil-small-etale-pretopology`, `def:weil-small-proetale-precoverage`

Every arrow in the family is etale and hence weakly etale. The underlying scheme maps are unchanged, so their joint surjectivity is unchanged. These are exactly the membership conditions in the selected pro-etale precoverage. □

Lemma 11.9.8 (Etale covering sieves remain pro-etale covering). Lean: `conditional-interface: lemma:weil-efi-site-covering-sieve-image`

Uses: `def:weil-efi-site-covering-sieve-image`

If R is covering on U for the selected etale topology, then $\iota_{X,!}^{\mathrm{sieve}} R$ is covering for the selected pro-etale topology.

Proof. Uses: `lemma:weil-efi-site-covering-family-image`, `def:weil-small-etale-topology`, `def:weil-small-proetale-topology`

The etale topology is generated by its pretopology, so R contains the sieve generated by an etale covering family. The preceding lemma sends that family to a pro-etale covering family. Its generated sieve is contained in $\iota_{X,!}^{\mathrm{sieve}} R$, and covering sieves are upward closed. □

Lemma 11.9.9 (Pullback equation for the image-sieve operation). Lean: `conditional-interface: lemma:weil-efi-site-covering-sieve-pullback`

Uses: `def:weil-efi-site-covering-sieve-image`

For an etale arrow $f : V \to U$ and a sieve R on U,

$$(\iota_X f)^*(\iota_{X,!}^{\mathrm{sieve}} R) = \iota_{X,!}^{\mathrm{sieve}}(f^* R).$$

Proof. Uses: `def:weil-efi-site-covering-sieve-image`, `def:weil-pbc-etale-proetale-inclusion`

Both sieves are generated by the base changes $V \times_U V' \to V$ as $V' \to U$ ranges through R. The selected etale and pro-etale over-categories compute these fiber products by the same scheme $V \times_U V'$, and ι_X fixes its projections. Hence their generating arrows, and therefore the generated sieves, agree. □

Definition 11.9.10 (Continuity predicate for the site inclusion). Lean: `conditional-interface: def:weil-efi-site-continuity-certificate`

Uses: `def:weil-efi-site-covering-sieve-image`

Let $\mathsf{Continuous}(\iota_X)$ mean that the image-sieve operation sends a covering sieve to a covering sieve and satisfies the displayed pullback equation.

Proposition 11.9.11 (Continuity certificate for the site inclusion). Lean: `conditional-interface: prop:weil-efi-site-continuity-certificate`

Uses: `def:weil-efi-site-continuity-certificate`

The predicate $\mathsf{Continuous}(\iota_X)$ holds.

Proof. Uses: `lemma:weil-efi-site-covering-sieve-image`, `lemma:weil-efi-site-covering-sieve-pullback`

The first lemma supplies covering preservation and the second supplies the pullback field. □

Lemma 11.9.12 (The etale/pro-etale assignment is a functor). Lean: `conditional-interface: lemma:weil-pbc-etale-proetale-functor`

Uses: `def:weil-pbc-etale-proetale-inclusion`

The assignment ι_X preserves identity morphisms and composition.

Proof. Uses:

For an etale object $U \to X$, both $\iota_X(1_U)$ and $1_{\iota_X(U)}$ have underlying scheme morphism 1_U. For composable X-morphisms $U \xrightarrow{h} V \xrightarrow{k} W$, both $\iota_X(kh)$ and $\iota_X(k)\iota_X(h)$ have underlying scheme morphism kh. Morphisms in either A2 over-category are determined by this underlying map, which proves both equations. □

Lemma 11.9.13 (Continuity of the etale/pro-etale inclusion). Lean: `conditional-interface: lemma:weil-pbc-etale-proetale-continuous`

Uses: `def:weil-pbc-etale-proetale-inclusion`

The functor $\iota_X : X_{\mathrm{et}} \to X_{\mathrm{proet}}$ is continuous for the final A2 topologies.

Proof. Uses: `lemma:weil-pbc-etale-proetale-functor`

Let $\{U_a \to U\}_{a\in A}$ be a covering family in the A2 small etale pretopology. Each $U_a \to U$ is etale, hence weakly etale, and the underlying family of scheme maps is unchanged by ι_X. Its images are jointly surjective, so the same family belongs to the pulled-back pro-etale precoverage. Consequently the sieve that it generates is covering for the A2 pro-etale topology.

More generally, if $R \subset h_U$ is an etale covering sieve, send it to the sieve on $\iota_X U$ generated by the arrows $\iota_X V \to \iota_X U$ for $V \to U$ in R. The preceding argument shows that this image sieve is pro-etale covering. Formation of the image sieve commutes with pullback, because the underlying scheme fiber products are unchanged. Since both A2 topologies are generated by the displayed precoverages, this is the continuity criterion. □

Lemma 11.9.14 (Finite-limit preservation of the inclusion). Lean: `conditional-interface:lemma:weil-pbc-etale-proetale-left-exact`

Uses: `def:weil-pbc-etale-proetale-inclusion`

The functor ι_X preserves the terminal object and fiber products, hence is left exact.

Proof. Uses: `lemma:weil-pbc-etale-proetale-functor`

The terminal object in both A2 sites is $1_X : X \to X$, and ι_X fixes it. For a diagram $U \to W \leftarrow V$ of etale X-objects, its fiber product in both over-categories has underlying scheme $U \times_W V$ with the same two projections and structural map to X. Etale morphisms are stable under base change and composition, so this is again an etale object. Thus the canonical map

$$\iota_X(U \times_W V) \longrightarrow \iota_X U \times_{\iota_X W} \iota_X V$$

is the identity on $U \times_W V$, and is an isomorphism. Terminal objects and fiber products generate finite limits. □

Definition 11.9.15 (Etale/pro-etale comparison morphism). Uses:

Lean: `conditional-interface:def:weil-pbc-epsilon-comparison`

The continuous left-exact functor ι_X induces the geometric morphism

$$\epsilon_X : X_{\mathrm{proet}} \longrightarrow X_{\mathrm{et}}.$$

For every commutative coefficient ring A, its inverse- and direct-image functors have the typed source and target

$$\epsilon_X^{-1} : \mathrm{Sh}_A(X_{\mathrm{et}}) \longrightarrow \mathrm{Sh}_A(X_{\mathrm{proet}}), \qquad \epsilon_{X,*} : \mathrm{Sh}_A(X_{\mathrm{proet}}) \longrightarrow \mathrm{Sh}_A(X_{\mathrm{et}}).$$

For $G \in \mathrm{Sh}_A(X_{\mathrm{proet}})$ one has $(\epsilon_{X,*}G)(U) = G(\iota_X U)$. For $F \in \mathrm{Sh}_A(X_{\mathrm{et}})$, the inverse image is the pro-etale sheafification of the left Kan extension along ι_X^{op}; before sheafification its value at $W \to X$ is

$$\operatorname*{colim}_{(W \to \iota_X U)} F(U).$$

Definition 11.9.16 (Etale factorization index for a pro-etale object). Lean: `conditional-interface:def:weil-pbc-epsilon-factorization-index-v1`

Uses: `def:weil-pbc-epsilon-comparison`

For a pro-etale object $W \to X$, let I_W be the category whose objects are factorizations $W \xrightarrow{a} \iota_X U \to X$ with $U \to X$ etale. A morphism from (a, U) to (a', U') is an etale-site arrow $h : U' \to U$ such that $a = \iota_X(h)a'$. The transition map on a sheaf F is restriction $F(U) \to F(U')$.

Lemma 11.9.17 (The etale factorization index is filtered). Lean: `conditional-interface:lemma:weil-pbc-epsilon-factorization-index-filtered-v1`

Uses: `def:weil-pbc-epsilon-factorization-index-v1`

For every W, the category I_W is nonempty and filtered.

Proof. Uses: `def:weil-pbc-epsilon-factorization-index-v1,lemma:weil-pbc-etale-proetale-left-exact`

The structural map $W \to X = \iota_X X$ gives an object. Two factorizations through U_1 and U_2 are dominated by the factorization through $U_1 \times_X U_2$, since ι_X preserves this fibre product. For two parallel arrows $h_0, h_1 : U' \rightrightarrows U$ that agree after precomposition with $W \to \iota_X U'$, their equalizer is the pullback of the diagonal of the etale morphism $U \to X$. This diagonal is an open immersion, so the equalizer is an etale open subobject of U' through which W factors. The resulting object receives a morphism from (a', U') and equalizes the two arrows in I_W. These are the three filteredness axioms. □

Lemma 11.9.18 (Construction of the comparison geometric morphism). Lean: `conditional-interface:lemma:weil-pbc-epsilon-geometric-morphism-construction-v1`

Uses: `def:weil-pbc-epsilon-comparison, def:weil-pbc-epsilon-factorization-index-v1`

Restriction along ι_X^{op}, pro-etale sheafification of left Kan extension, and the displayed evaluation maps define an adjoint pair $\epsilon_X^{-1} \dashv \epsilon_{X,*}$. The adjunction bijection is natural in both variables, and ϵ_X^{-1} preserves finite limits.

Proof. Uses: lemma:weil-pbc-etale-proetale-continuous,lemma:weil-pbc-etale-proetale-left-exact,lemma:weil-pbc-epsilon-factorization-index-filtered-v1,def:weil-pbc-epsilon-comparison

Continuity makes restriction of a pro-etale sheaf an etale sheaf. Presheaf left Kan extension is left adjoint to presheaf restriction, and sheafification is left adjoint to inclusion of sheaves. Composing these two adjunctions gives

$$\mathrm{Hom}(\epsilon_X^{-1}F, G) \cong \mathrm{Hom}(F, \epsilon_{X,*}G).$$

The two universal properties make this bijection natural in F and G and make its identity and composition equations automatic.

Before sheafification, evaluation at W is $\varinjlim_{I_W} F(U)$. The preceding lemma makes I_W filtered, so this colimit of A-modules commutes with finite limits. Sheafification of A-module presheaves is left exact, and the left-exactness of ι_X identifies the transition diagrams for terminal objects and fibre products. Thus ϵ_X^{-1} preserves finite limits. All maps used are induced by the same restriction maps, so they are compatible with restriction in W and with morphisms of coefficient sheaves. □

Definition 11.9.19 (Forward Hom map for the comparison). Lean: conditional-interface:def:weil-efi-site-adjunction-hom-forward

Uses: def:weil-pbc-epsilon-comparison

For A-module sheaves F on X_{et} and G on X_{proet}, send $\alpha : \epsilon_X^{-1}F \to G$ to the map $\Phi(\alpha) : F \to \epsilon_{X,*}G$ whose component at U is

$$s \longmapsto \alpha_{\iota_X U}\big[(1_{\iota_X U}, s)\big].$$

Definition 11.9.20 (Representative formula for the reverse Hom map). Lean: conditional-interface:def:weil-efi-site-adjunction-hom-backward-presheaf

Uses: def:weil-pbc-epsilon-comparison

For $\beta : F \to \epsilon_{X,*}G$, assign to a left-Kan-extension representative $(W \xrightarrow{a} \iota_X U, s)$ the section

$$G(a)(\beta_U(s)) \in G(W).$$

Lemma 11.9.21 (Descent of the reverse Hom formula). Lean: conditional-interface:lemma:weil-efi-site-adjunction-hom-backward-descent

Uses: def:weil-efi-site-adjunction-hom-backward-presheaf

The representative formula factors uniquely through pro-etale sheafification.

Proof. Uses: def:weil-efi-site-adjunction-hom-backward-presheaf,def:weil-pbc-epsilon-factorization-index-v1

For a transition $h : U' \to U$ with $a = \iota_X(h)a'$, naturality of β and functoriality of G identify the values of (a, s) and $(a', F(h)s)$. Restriction along $b : W' \to W$ changes each formula to $G(ab)(\beta_U(s))$. Local equality descends because G is a sheaf. The universal property of sheafification gives the unique factorization. □

Definition 11.9.22 (Reverse Hom map for the comparison). Lean: conditional-interface:def:weil-efi-site-adjunction-hom-backward

Uses: def:weil-efi-site-adjunction-hom-backward-presheaf

Let $\Psi(\beta) : \epsilon_X^{-1}F \to G$ be the sheaf map induced from the representative formula by the universal property of sheafification.

Lemma 11.9.23 (Left inverse equation for the comparison Hom maps). Lean: conditional-interface:lemma:weil-efi-site-adjunction-hom-left-inverse

Uses: def:weil-efi-site-adjunction-hom-forward, def:weil-efi-site-adjunction-hom-backward

For every $\alpha : \epsilon_X^{-1}F \to G$,

$$\Psi(\Phi(\alpha)) = \alpha.$$

Proof. Uses: def:weil-efi-site-adjunction-hom-forward,def:weil-efi-site-adjunction-hom-backward

On a representative $(W \xrightarrow{a} \iota_X U, s)$, the left side is $G(a)(\alpha_{\iota_X U}[(1, s)])$. Naturality of α identifies this with $\alpha_W[(a, s)]$. Representatives are locally exhaustive after sheafification, so the sheaf maps agree. □

Lemma 11.9.24 (Right inverse equation for the comparison Hom maps). Lean: conditional-interface:lemma:weil-efi-site-adjunction-hom-right-inverse

Uses: def:weil-efi-site-adjunction-hom-forward, def:weil-efi-site-adjunction-hom-backward

For every $\beta : F \to \epsilon_{X,*}G$,

$$\Phi(\Psi(\beta)) = \beta.$$

Proof. Uses: def:weil-efi-site-adjunction-hom-forward,def:weil-efi-site-adjunction-hom-backward

At an etale object U, the left side sends s to the evaluation of $(1_{\iota_X U}, s)$, namely $G(1_{\iota_X U})(\beta_U(s)) = \beta_U(s)$. □

Lemma 11.9.25 (Source naturality of the comparison Hom correspondence). Lean: conditional-interface:lemma:weil-efi-site-adjunction-hom-source-naturality

Uses: def:weil-efi-site-adjunction-hom-forward

For $u : F' \to F$ and $\alpha : \epsilon_X^{-1} F \to G$,

$$\Phi(\alpha\, \epsilon_X^{-1} u) = \Phi(\alpha) u.$$

Proof. Uses: def:weil-efi-site-adjunction-hom-forward

At U, each side sends $s \in F'(U)$ to $\alpha_{\iota_X U}[(1_{\iota_X U}, u_U(s))]$. □

Lemma 11.9.26 (Target naturality of the comparison Hom correspondence). Lean: conditional-interface:lemma:weil-efi-site-adjunction-hom-target-naturality

Uses: def:weil-efi-site-adjunction-hom-forward

For $v : G \to G'$ and $\alpha : \epsilon_X^{-1} F \to G$,

$$\Phi(v\alpha) = \epsilon_{X,*}(v)\Phi(\alpha).$$

Proof. Uses: def:weil-efi-site-adjunction-hom-forward

At U, each side sends $s \in F(U)$ to $v_{\iota_X U}(\alpha_{\iota_X U}[(1_{\iota_X U}, s)])$. □

Definition 11.9.27 (Adjunction-certificate predicate for the comparison functors). Lean: conditional-interface:def:weil-efi-site-sheaf-adjunction-certificate

Uses: def:weil-efi-site-adjunction-hom-forward,def:weil-efi-site-adjunction-hom-backward

Let $\mathsf{AdjunctionCert}(\Phi, \Psi)$ be the conjunction of the left inverse equation, the right inverse equation, source naturality, and target naturality for the displayed Hom-map pair.

Proposition 11.9.28 (Adjunction certificate for the comparison functors). Lean: conditional-interface:prop:weil-efi-site-sheaf-adjunction-certificate

Uses: def:weil-efi-site-sheaf-adjunction-certificate

The predicate $\mathsf{AdjunctionCert}(\Phi, \Psi)$ holds.

Proof. Uses: lemma:weil-efi-site-adjunction-hom-left-inverse,lemma:weil-efi-site-adjunction-hom-right-inverse,lemma:weil-efi-site-adjunction-hom-source-naturality,lemma:weil-efi-site-adjunction-hom-target-naturality

The four cited lemmas supply the four fields in their displayed order. □

Lemma 11.9.29 (Finite-limit preservation by comparison inverse image). Lean: conditional-interface:lemma:weil-efi-site-inverse-image-finite-limits

Uses: def:weil-pbc-epsilon-comparison

The functor ϵ_X^{-1} preserves finite limits of A-module sheaves.

Proof. Uses: def:weil-pbc-epsilon-factorization-index-v1,lemma:weil-pbc-epsilon-factorization-index-filtered-v1,prop:weil-efi-site-continuity-certificate

Before sheafification, evaluation at W is the filtered colimit $\varinjlim_{I_W} F(U)$. Filtered colimits of A-modules commute with finite limits. Sheafification of A-module presheaves is left exact, and the pullback equation in the continuity certificate identifies the transition diagrams. The resulting sheaf functor therefore preserves finite limits. □

Definition 11.9.30 (Geometric-morphism certificate predicate). Lean: conditional-interface:def:weil-efi-site-geometric-morphism-certificate

Uses: def:weil-pbc-epsilon-comparison,def:weil-efi-site-sheaf-adjunction-certificate

Let $\mathsf{GeometricMorphismCert}(\epsilon_X^{-1}, \epsilon_{X,*})$ mean that the Hom correspondence satisfies the adjunction-certificate predicate and that ϵ_X^{-1} preserves finite limits.

Proposition 11.9.31 (Geometric-morphism package for the comparison). Lean: conditional-interface:prop:weil-efi-site-geometric-morphism-package

Uses: def:weil-efi-site-geometric-morphism-certificate

The predicate $\mathsf{GeometricMorphismCert}(\epsilon_X^{-1}, \epsilon_{X,*})$ holds.

Proof. Uses: prop:weil-efi-site-sheaf-adjunction-certificate,lemma:weil-efi-site-inverse-image-finite-limits
The proposition supplies the adjunction field and the lemma supplies the finite-limit field. □

Lemma 11.9.32 (Exactness of the packaged comparison inverse image). *Lean:* conditional-interface:lemma:weil-efi-site-inverse-image-exact

Uses: def:weil-pbc-epsilon-comparison
The functor ϵ_X^{-1} is exact on A-module sheaves.

Proof. Uses: prop:weil-efi-site-sheaf-adjunction-certificate,lemma:weil-efi-site-inverse-image-finite-limits
Finite-limit preservation gives preservation of kernels. The adjunction certificate makes ϵ_X^{-1} a left adjoint, so it preserves cokernels. Kernel and cokernel preservation gives exactness in the abelian category of A-module sheaves. □

Lemma 11.9.33 (Well-definedness certificate for def:weil-pbc-epsilon-comparison). *Lean:* conditional-interface:lemma:weil-sf-well-defined-def-weil-pbc-epsilon-comparison

Uses: def:weil-pbc-epsilon-comparison
The construction in Definition 11.9.15 is a typed raw datum. Its theorem-valued naturality, comparison, descent, or coherence fields are isolated in this property lemma.

Proof. Uses: lemma:weil-pbc-etale-proetale-continuous,lemma:weil-pbc-etale-proetale-left-exact,lemma:weil-pbc-epsilon-factorization-index-filtered-v1,lemma:weil-pbc-epsilon-geometric-morphism-construction-v1
The inclusion functor is continuous and left exact. The factorization-index lemma makes the displayed Kan-extension formula a filtered colimit, and the construction lemma supplies its sheafification, the adjunction, naturality, and finite-limit preservation. The formulas for direct image and inverse image are therefore independent of representatives and restrictions and define the stated geometric morphism with all comparison fields. □

Lemma 11.9.34 (The comparison functors form a geometric morphism). *Lean:* conditional-interface:lemma:weil-pbc-epsilon-geometric-morphism

Uses: def:weil-pbc-epsilon-comparison
For A-module sheaves F on X_{et} and G on X_{proet} there is a natural bijection

$$\mathrm{Hom}_{X_{\mathrm{proet}}}(\epsilon_X^{-1}F, G) \cong \mathrm{Hom}_{X_{\mathrm{et}}}(F, \epsilon_{X,*}G),$$

and ϵ_X^{-1} preserves finite limits.

Proof. Uses: lemma:weil-pbc-etale-proetale-continuous,lemma:weil-pbc-etale-proetale-left-exact
Restriction along ι_X^{op} sends pro-etale sheaves to etale sheaves by Lemma 11.9.13. The left-Kan extension adjunction, followed by the sheafification adjunction, gives the displayed bijection.

For a pro-etale object W, let I_W be the category of factorizations $W \to \iota_X U$ with U etale over X. It is filtered: the map to X gives an object, two factorizations are dominated by their fiber product over X, and two parallel arrows are equalized by their scheme-theoretic equalizer, an open etale subobject through which W factors. Filtered colimits of A-modules commute with finite limits. The pointwise Kan extension therefore preserves finite limits, and A-module sheafification does as well. This proves left exactness of ϵ_X^{-1} and hence the geometric-morphism assertion. □

Lemma 11.9.35 (Exactness of comparison inverse image). *Uses:*

Lean: conditional-interface:lemma:weil-pbc-epsilon-inverse-image-exact
For every commutative ring A, the functor $\epsilon_X^{-1} : \mathrm{Sh}_A(X_{\mathrm{et}}) \to \mathrm{Sh}_A(X_{\mathrm{proet}})$ is exact.

Proof. Uses: lemma:weil-pbc-epsilon-geometric-morphism
It preserves kernels by the finite-limit assertion in Lemma 11.9.34. It preserves cokernels because it is a left adjoint. Applying these two facts to a short exact sequence

$$0 \longrightarrow F' \longrightarrow F \longrightarrow F'' \longrightarrow 0$$

shows that its inverse-image sequence is exact at all three terms. □

Definition 11.9.36 (Unit and counit of the comparison)**.** Lean: `conditional-interface:def:weil-pbc-epsilon-unit-counit`

Uses: `def:weil-pbc-epsilon-comparison`

Write the unit and counit of the preceding adjunction as

$$\eta^X : \mathrm{id}_{\mathrm{Sh}_A(X_{\mathrm{et}})} \longrightarrow \epsilon_{X,*}\epsilon_X^{-1}, \qquad \varepsilon^X : \epsilon_X^{-1}\epsilon_{X,*} \longrightarrow \mathrm{id}_{\mathrm{Sh}_A(X_{\mathrm{proet}})}.$$

For $F \in \mathrm{Sh}_A(X_{\mathrm{et}})$, the component $\eta^X_{F,U}$ sends $s \in F(U)$ to the class represented by $(1_{\iota_X U} : \iota_X U \to \iota_X U, s)$. For $G \in \mathrm{Sh}_A(X_{\mathrm{proet}})$, the component $\varepsilon^X_{G,W}$ sends a representative $(W \xrightarrow{a} \iota_X U, s)$ to the restriction $G(a)(s) \in G(W)$. This map on the Kan-extension presheaf factors uniquely through pro-etale sheafification.

Lemma 11.9.37 (Representative descent for the comparison counit)**.** Lean: `conditional-interface:lemma:weil-pbc-epsilon-counit-representative-descent-v1`

Uses: `def:weil-pbc-epsilon-unit-counit`

The rule $(W \xrightarrow{a} \iota_X U, s) \mapsto G(a)(s)$ respects the left-Kan-extension relations, restriction in W, and local equality, hence factors uniquely through pro-etale sheafification.

Proof. Uses: `def:weil-pbc-epsilon-unit-counit,def:weil-pbc-epsilon-factorization-index-v1,lemma:weil-pbc-epsilon-geometric-morphism-construction-v1`

If $h : U' \to U$ is a transition and $a = \iota_X(h)a'$, then functoriality of G gives

$$G(a')G(\iota_X h)(s) = G(a)(s),$$

so the rule is constant on the colimit relation. For $b : W' \to W$, both restriction before evaluation and evaluation before restriction send (a, s) to $G(ab)(s)$. If two representatives agree after a pro-etale cover, their images agree on that cover; the sheaf equalizer for G makes the images equal. Thus the presheaf map is local and has a unique factor through sheafification. This factor is precisely the counit attached to the adjunction constructed above. □

Lemma 11.9.38 (Naturality equation for the comparison unit)**.** Lean: `conditional-interface:lemma:weil-efi-site-unit-naturality`

Uses: `def:weil-pbc-epsilon-unit-counit`

For $\alpha : F \to F'$ on X_{et},

$$(\epsilon_{X,*}\epsilon_X^{-1}\alpha)\eta^X_F = \eta^X_{F'}\alpha.$$

Proof. Uses: `def:weil-pbc-epsilon-unit-counit`

At an etale object U, each route sends $s \in F(U)$ to $(1_{\iota_X U}, \alpha_U(s))$. □

Lemma 11.9.39 (Naturality equation for the comparison counit)**.** Lean: `conditional-interface:lemma:weil-efi-site-counit-naturality`

Uses: `def:weil-pbc-epsilon-unit-counit`

For $\beta : G \to G'$ on X_{proet},

$$\beta\varepsilon^X_G = \varepsilon^X_{G'}(\epsilon_X^{-1}\epsilon_{X,*}\beta).$$

Proof. Uses: `def:weil-pbc-epsilon-unit-counit,lemma:weil-pbc-epsilon-counit-representative-descent-v1`

On a representative $(W \xrightarrow{a} \iota_X U, s)$, each route gives $\beta_W(G(a)(s)) = G'(a)(\beta_{\iota_X U}(s))$. The representative-descent lemma carries this equality through sheafification. □

Lemma 11.9.40 (Restriction equation for the comparison unit)**.** Lean: `conditional-interface:lemma:weil-efi-site-unit-restriction`

Uses: `def:weil-pbc-epsilon-unit-counit`

For an etale arrow $b : U' \to U$, restriction carries $\eta^X_{F,U}(s)$ to $\eta^X_{F,U'}(F(b)s)$.

Proof. Uses: `def:weil-pbc-epsilon-unit-counit`

Restriction carries $(1_{\iota_X U}, s)$ to $(\iota_X b, s)$. The left-Kan-extension relation identifies this representative with $(1_{\iota_X U'}, F(b)s)$. □

Lemma 11.9.41 (Restriction equation for the comparison counit)**.** Lean: `conditional-interface:lemma:weil-efi-site-counit-restriction`

Uses: `def:weil-pbc-epsilon-unit-counit`

For a pro-etale arrow $b : W' \to W$, restriction carries the counit value of $(W \xrightarrow{a} \iota_X U, s)$ to the counit value of $(W' \xrightarrow{ab} \iota_X U, s)$.

Proof. Uses: def:weil-pbc-epsilon-unit-counit,lemma:weil-pbc-epsilon-counit-representative-descent-v1

The first route gives $G(b)G(a)(s)$ and the second gives $G(ab)(s)$; these are equal by functoriality of G and descend through sheafification. □

Definition 11.9.42 (Naturality predicate for the comparison unit and counit). Lean: conditional-interface:def:weil-efi-site-unit-counit-certificate

Uses: def:weil-pbc-epsilon-unit-counit

Let $\mathsf{NaturalUnitCounit}(\eta^X, \varepsilon^X)$ be the conjunction of unit naturality, counit naturality, the unit restriction equation, and the counit restriction equation.

Proposition 11.9.43 (Natural unit-counit package for the comparison). Lean: conditional-interface:prop:weil-efi-site-unit-counit-package

Uses: def:weil-efi-site-unit-counit-certificate

The predicate $\mathsf{NaturalUnitCounit}(\eta^X, \varepsilon^X)$ *holds.*

Proof. Uses: lemma:weil-pbc-epsilon-counit-representative-descent-v1,lemma:weil-efi-site-unit-naturality,lemma:weil-efi-site-counit-naturality,lemma:weil-efi-site-unit-restriction,lemma:weil-efi-site-counit-restriction

Representative descent types the counit, and the four atomic equation lemmas supply the fields of the displayed predicate. □

Lemma 11.9.44 (Left triangle equation for the comparison adjunction). Lean: conditional-interface:lemma:weil-efi-site-left-triangle

Uses: def:weil-pbc-epsilon-unit-counit

For every etale A-module sheaf F,

$$\varepsilon^X_{\epsilon_X^{-1}F}\,\epsilon_X^{-1}(\eta^X_F) = 1_{\epsilon_X^{-1}F}.$$

Proof. Uses: def:weil-pbc-epsilon-unit-counit

A local section is represented by $(W \xrightarrow{a} \iota_X U, s)$. The composite replaces s by its identity representative and evaluates along a, returning the class (a, s). Local representatives detect sheaf maps. □

Lemma 11.9.45 (Right triangle equation for the comparison adjunction). Lean: conditional-interface:lemma:weil-efi-site-right-triangle

Uses: def:weil-pbc-epsilon-unit-counit

For every pro-etale A-module sheaf G,

$$\epsilon_{X,*}(\varepsilon^X_G)\,\eta^X_{\epsilon_{X,*}G} = 1_{\epsilon_{X,*}G}.$$

Proof. Uses: def:weil-pbc-epsilon-unit-counit

At an etale object U, a section $s \in G(\iota_X U)$ first becomes $(1_{\iota_X U}, s)$ and is then evaluated along the identity, returning s. □

Definition 11.9.46 (Triangle-certificate predicate for the comparison adjunction). Lean: conditional-interface:def:weil-efi-site-triangle-certificate

Uses: def:weil-pbc-epsilon-unit-counit

Let $\mathsf{TriangleCert}(\eta^X, \varepsilon^X)$ be the conjunction of the left-triangle equation and the right-triangle equation.

Proposition 11.9.47 (Triangle certificate for the comparison adjunction). Lean: conditional-interface:prop:weil-efi-site-triangle-certificate

Uses: def:weil-efi-site-triangle-certificate

The predicate $\mathsf{TriangleCert}(\eta^X, \varepsilon^X)$ *holds.*

Proof. Uses: lemma:weil-efi-site-left-triangle,lemma:weil-efi-site-right-triangle

The cited lemmas supply the left and right fields, respectively. □

Lemma 11.9.48 (Naturality and triangle equations for the comparison adjunction). Lean: conditional-interface:lemma:weil-pbc-epsilon-adjunction-equations-v1

Uses: def:weil-pbc-epsilon-unit-counit

The displayed unit and counit are natural in coefficient sheaves and satisfy the two triangle identities. Their components commute with restriction on both sites.

Proof. Uses: def:weil-pbc-epsilon-unit-counit,lemma:weil-pbc-epsilon-counit-representative-descent-v1,lemma:weil-pbc-epsilon-geometric-morphism-construction-v1

For $\alpha : F \to F'$, both routes around the unit naturality square send $s \in F(U)$ to $(1_{\iota_X U}, \alpha_U(s))$. For $\beta : G \to G'$, both routes around the counit square send (a, s) to $\beta_W(G(a)(s)) = G'(a)(\beta_{\iota_X U}(s))$. These equalities commute with restriction and descend to sheaves by the preceding lemma.

A local section of $\epsilon_X^{-1}F$ is represented by (a, s). The first triangle composite replaces s by its identity representative and evaluates along a, returning (a, s). A section $s \in (\epsilon_{X,*}G)(U) = G(\iota_X U)$ is sent by the second composite to $(1_{\iota_X U}, s)$ and then evaluated along the identity, returning s. Local representatives detect sheaf maps, so both composites are identities. □

Lemma 11.9.49 (Well-definedness certificate for def:weil-pbc-epsilon-unit-counit)**.** Lean: `conditional-interface: lemma: weil-sf-well-defined-def-weil-pbc-epsilon-unit-counit`

Uses: `def: weil-pbc-epsilon-unit-counit`

The construction in Definition 11.9.36 is a typed raw datum. Its theorem-valued naturality, comparison, descent, or coherence fields are isolated in this property lemma.

Proof. Uses: lemma:weil-pbc-epsilon-geometric-morphism-construction-v1,lemma:weil-pbc-epsilon-counit-representative-descent-v1,lemma:weil-pbc-epsilon-adjunction-equations-v1

The geometric-morphism construction supplies the adjunction. Representative descent makes the displayed counit formula a sheaf morphism, while the unit formula is the identity-factorization component of the same adjunction. The final lemma proves naturality, restriction compatibility, and both triangle identities, so every theorem-valued field of the raw unit–counit datum is well-defined. □

Lemma 11.9.50 (Naturality of the comparison unit and counit)**.** Lean: `conditional-interface: lemma: weil-pbc-epsilon-unit-counit-naturality`

Uses: `def: weil-pbc-epsilon-unit-counit`

For $\alpha : F \to F'$ on X_{et} and $\beta : G \to G'$ on X_{proet}, one has

$$(\epsilon_{X,*}\epsilon_X^{-1}\alpha)\eta_F^X = \eta_{F'}^X \alpha,$$
$$\beta\varepsilon_G^X = \varepsilon_{G'}^X(\epsilon_X^{-1}\epsilon_{X,*}\beta).$$

Proof. Uses: def:weil-pbc-epsilon-comparison,def:weil-pbc-epsilon-unit-counit

At an etale object U, both sides of the first equation send $s \in F(U)$ to the class $(1_{\iota_X U}, \alpha_U(s))$. At a pro-etale object W, apply the second equation to a representative $(W \xrightarrow{a} \iota_X U, s)$. Both sides give $\beta_W(G(a)(s)) = G'(a)(\beta_{\iota_X U}(s))$ by naturality of β. These equations survive sheafification and prove the asserted naturality. □

Lemma 11.9.51 (Triangle identities for the comparison adjunction)**.** Lean: `conditional-interface: lemma: weil-pbc-epsilon-triangle-identities`

Uses: `def: weil-pbc-epsilon-unit-counit`

The unit and counit satisfy

$$\varepsilon^X_{\epsilon_X^{-1}F}\,\epsilon_X^{-1}(\eta_F^X) = 1_{\epsilon_X^{-1}F},$$
$$\epsilon_{X,*}(\varepsilon_G^X)\,\eta^X_{\epsilon_{X,*}G} = 1_{\epsilon_{X,*}G}.$$

Proof. Uses: lemma:weil-pbc-epsilon-unit-counit-naturality

A section of $\epsilon_X^{-1}F$ is locally represented by $(W \xrightarrow{a} \iota_X U, s)$. The first composite replaces s by its identity representative at $\iota_X U$ and then restricts it along a, which returns the original class. For $s \in (\epsilon_{X,*}G)(U) = G(\iota_X U)$, the second composite forms $(1_{\iota_X U}, s)$ and evaluates it along the identity, returning s. Local representatives determine maps of sheaves, so both displayed composites are identities. □

Lemma 11.9.52 (Naturality of the comparison in the scheme)**.** Lean: `conditional-interface: lemma: weil-pbc-epsilon-scheme-naturality`

Uses: `def: weil-pbc-etale-proetale-inclusion, def: weil-pbc-epsilon-comparison, def: weil-pbc-epsilon-unit-counit`

For a scheme morphism $u : X \to Y$, base change on the two sites gives a canonical natural isomorphism

$$u^*_{\mathrm{proet}}\iota_Y \xrightarrow{\sim} \iota_X u^*_{\mathrm{et}}.$$

It is compatible with identities and composition in u, and the induced square of geometric morphisms carries η^Y, ε^Y to η^X, ε^X.

Proof. Uses: def:weil-pbc-epsilon-comparison,def:weil-pbc-epsilon-unit-counit
On an etale object $V \to Y$, both composites have underlying X-scheme $X \times_Y V \to X$; on a morphism they both use its base change. The identity map of this fiber product gives the component of the natural isomorphism. The associativity and unit isomorphisms for fiber products give its composition and identity coherence. Under this identification, the unit is still represented by an identity arrow and the counit is still restriction along $W \to \iota_X U$. The formulas in Definition 11.9.36 therefore give the two compatibility squares. □

Definition 11.9.53 (Witness for coming from the etale site)**.** Lean: conditional-interface:def:weil-pbc-comes-from-etale

Uses: def:weil-pbc-epsilon-comparison
For a coefficient ring A and $K \in D^+(X_{\mathrm{proet}}, A)$, a witness that K comes from the etale site is a pair

$$(L, \theta), \qquad L \in D^+(X_{\mathrm{et}}, A), \quad \theta : \epsilon_X^{-1} L \xrightarrow{\sim} K,$$

including the two inverse equations for θ.

Lemma 11.9.54 (Comparison inverse image commutes with pullback)**.** Lean: conditional-interface:lemma:weil-pbc-epsilon-pullback

Uses: def:weil-pbc-epsilon-comparison,def:weil-pbc-comes-from-etale
For $u : X \to Y$ and $L \in D^+(Y_{\mathrm{et}}, A)$ there is a natural isomorphism

$$Lu^* \epsilon_Y^{-1} L \xrightarrow{\sim} \epsilon_X^{-1} Lu^* L.$$

It is compatible with identities, composition, and both horizontal maps in a Cartesian square.

Proof. Uses: lemma:weil-pbc-epsilon-scheme-naturality,lemma:weil-pbc-epsilon-inverse-image-exact
Lemma 11.9.52 identifies the two composite site functors. Their inverse images are therefore sheafifications of the same presheaf left Kan extension, and the identity on that presheaf induces the displayed isomorphism. The identity and composition coherence proved there gives the corresponding equations and the two horizontal maps in a Cartesian square. Exactness of ϵ^{-1} and K-flat replacement extend these equations to derived pullback. □

Lemma 11.9.55 (The comparison unit is an isomorphism)**.** Lean: conditional-interface:lemma:weil-pbc-epsilon-unit-isomorphism

Uses: def:weil-pbc-epsilon-unit-counit,def:weil-pbc-epsilon-comparison
For every A-module sheaf F on X_{et}, the unit

$$\eta_F^X : F \longrightarrow \epsilon_{X,*} \epsilon_X^{-1} F$$

is an isomorphism.

Proof. Uses: def:weil-pbc-epsilon-comparison,def:weil-pbc-epsilon-unit-counit
Evaluate at an etale object $U \to X$. Restriction of the left-Kan-extension presheaf in Definition 11.9.15 back to $\iota_X U$ has the identity factorization $\iota_X U \to \iota_X U$. Refining any other factorization by its fiber product with U shows that the associated sheaf value is $F(U)$. Under this identification, the formula in Definition 11.9.36 sends s to the same section s. Thus every component of η_F^X is the identity. Etale objects detect maps of etale sheaves, so the unit is an isomorphism. □

Definition 11.9.56 (Derived etale-to-pro-etale cohomology comparison)**.** Lean: conditional-interface:def:weil-efi-site-derived-cohomology-comparison

Uses: def:weil-pbc-epsilon-comparison,def:weil-pbc-epsilon-unit-counit
For a commutative ring A and $L \in D^+(X_{\mathrm{et}}, A)$, let

$$u_{X,L}^R : L \longrightarrow R\epsilon_{X,*} \mathbf{L}\epsilon_X^{-1} L$$

be the derived adjunction unit. Define

$$c_{X,L} : R\Gamma(X_{\mathrm{et}}, L) \longrightarrow R\Gamma(X_{\mathrm{proet}}, \mathbf{L}\epsilon_X^{-1} L)$$

as $R\Gamma(X_{\mathrm{et}}, u_{X,L}^R)$ followed by the canonical identity $R\Gamma(X_{\mathrm{et}}, R\epsilon_{X,*}(-)) = R\Gamma(X_{\mathrm{proet}}, -)$. Write $c_{X,L}^n$ for its map on nth cohomology.

Lemma 11.9.57 (Acyclicity of pulled-back injectives)**.** Uses:

Lean: conditional-interface:lemma:weil-pbc-epsilon-injective-acyclicity
Let U be affine and etale over X, and let I be an injective A-module sheaf on U_{et}. Then

$$H^r(U_{\mathrm{proet}}, \epsilon_U^{-1} I) = 0 \qquad (r > 0).$$

Consequently $R^r \epsilon_{X,*}(\epsilon_X^{-1} J) = 0$ for every injective J on X_{et} and every $r > 0$.

Proof. Uses: lemma:weil-pbc-epsilon-unit-isomorphism,lemma:weil-pbc-epsilon-scheme-naturality

On U_{proet}, take the basis $\mathcal{B}$ of affine objects $V = \varprojlim_j V_j$ for which every $V_j \to U$ is affine etale and the transition maps are affine. Use finite pro-etale covers $\{V_a \to V\}_{a=1}^m$ inside this basis. Write each V_a as an inverse limit of affine etale $V_{a,j} \to V$. Fiber products commute with these limits, so every term of the Cech complex of the cover is the filtered colimit of the corresponding terms for the finite-stage families $\{V_{a,j} \to V_j\}$.

After increasing j, all finitely many arrows and their joint surjectivity descend, so $\{V_{a,j} \to V_j\}$ is an etale covering. The comparison unit identifies the restriction of $\epsilon_U^{-1} I$ on every finite-stage etale object with I. Hence

$$\check{H}^r(\{V_a \to V\}, \epsilon_U^{-1} I) = \varinjlim_j \check{H}^r(\{V_{a,j} \to V_j\}, I) = 0 \qquad (r > 0).$$

The last equality uses exactness of filtered colimits and Cech acyclicity of an injective sheaf. The basis is closed under fiber products, and every pro-etale object is covered by its members; the basis Cech criterion therefore gives the displayed cohomology vanishing.

For an affine etale $U \to X$, scheme naturality identifies the restriction of $\epsilon_X^{-1} J$ with $\epsilon_U^{-1}(J|_U)$. The restriction of an injective to the localization site is injective because extension by zero is its exact left adjoint. Thus the first part applies. Affine etale objects cover every etale object, so the sheaf associated to $U \mapsto H^r(U_{\mathrm{proet}}, \epsilon_U^{-1}(J|_U))$ is zero. This sheaf is $R^r \epsilon_{X,*}(\epsilon_X^{-1} J)$. □

Lemma 11.9.58 (Derived comparison for the epsilon unit)**.** *Uses:*

Lean: conditional-interface:lemma:weil-pbc-epsilon-derived-unit

For $L \in D^+(X_{\mathrm{et}}, A)$, the derived unit

$$L \longrightarrow R\epsilon_{X,*}\epsilon_X^{-1} L$$

is an isomorphism, naturally in L.

Proof. Uses: lemma:weil-pbc-epsilon-injective-acyclicity,lemma:weil-pbc-epsilon-unit-isomorphism,lemma:weil-pbc-epsilon-inverse-image-exact

Choose a bounded-below injective resolution $L \to I^\bullet$. Exactness of ϵ_X^{-1} represents the inverse image by $\epsilon_X^{-1} I^\bullet$. Lemma 11.9.57 makes every term $\epsilon_{X,*}$-acyclic. The bounded-below acyclic-resolution lemma then computes the derived direct image termwise:

$$R\epsilon_{X,*}\epsilon_X^{-1} L \simeq \epsilon_{X,*}\epsilon_X^{-1} I^\bullet.$$

The unit isomorphism identifies the last complex with $I^\bullet$, including its differentials. A morphism of bounded-below complexes lifts to the chosen injective resolutions, and two lifts are homotopic; termwise naturality of the unit makes the resulting derived isomorphism independent of the lift. □

Proposition 11.9.59 (Finite-level etale/pro-etale cohomology comparison)**.** *Lean:* conditional-interface:prop:weil-efi-site-derived-comparison-isomorphism

Uses: def:weil-efi-site-derived-cohomology-comparison

Let ℓ be prime, let $r \geq 1$, and let $\mathcal{A}$ be a constructible $\mathbf{Z}/\ell^r$-module sheaf on X_{et}. For $n \geq 0$, the map

$$c^n_{X,\mathcal{A}} : H^n(X_{\mathrm{et}}, \mathcal{A}) \longrightarrow H^n(X_{\mathrm{proet}}, \epsilon_X^{-1}\mathcal{A})$$

is an isomorphism.

Proof. Uses: lemma:weil-pbc-epsilon-derived-unit,lemma:weil-efi-site-inverse-image-exact

Exactness identifies $\mathbf{L}\epsilon_X^{-1}(\mathcal{A}[0])$ with $\epsilon_X^{-1}\mathcal{A}[0]$. The derived-unit lemma makes

$$\mathcal{A}[0] \longrightarrow R\epsilon_{X,*}\epsilon_X^{-1}\mathcal{A}[0]$$

an isomorphism. Derived global sections preserve this isomorphism, and the canonical identity $R\Gamma(X_{\mathrm{et}}, R\epsilon_{X,*}(-)) = R\Gamma(X_{\mathrm{proet}}, -)$ identifies the resulting arrow with $c_{X,\mathcal{A}[0]}$. Applying H^n gives the displayed isomorphism. □

Definition 11.9.60 (Canonical direct-image comparison)**.** Lean: conditional-interface:def:weil-pbc-epsilon-direct-image-comparison

Uses: def:weil-pbc-epsilon-unit-counit

Let $f : X \to Y$ be quasi-compact and quasi-separated. For an A-module sheaf F on X_{et}, define

$$\delta_{f,F} : \epsilon_Y^{-1} f_{\mathrm{et},*} F \longrightarrow f_{\mathrm{proet},*}\epsilon_X^{-1} F$$

to be adjoint to

$$f_{\mathrm{et},*} F \xrightarrow{f_{\mathrm{et},*}\eta_F^X} f_{\mathrm{et},*}\epsilon_{X,*}\epsilon_X^{-1} F = \epsilon_{Y,*} f_{\mathrm{proet},*}\epsilon_X^{-1} F.$$

The equality follows by evaluating both restrictions on an etale object of Y. Right deriving this natural transformation and using exactness of ϵ_Y^{-1} defines

$$D_f(L) : \epsilon_Y^{-1} R f_{\mathrm{et},*} L \longrightarrow R f_{\mathrm{proet},*}\epsilon_X^{-1} L$$

for $L \in D^+(X_{\mathrm{et}}, A)$.

Lemma 11.9.61 (Well-definedness certificate for def:weil-pbc-epsilon-direct-image-comparison)**.** *Lean:* `conditional-interface:lemma:weil-sf-well-defined-def-weil-pbc-epsilon-direct-image-comparison`

Uses: `def:weil-pbc-epsilon-direct-image-comparison`

The construction in Definition 11.9.60 is a typed raw datum. Its theorem-valued naturality, comparison, descent, or coherence fields are isolated in this property lemma.

Proof. Uses: `lemma:weil-pbc-epsilon-scheme-naturality,lemma:weil-pbc-epsilon-adjunction-equations-v1,lemma:weil-pbc-epsilon-inverse-image-exact`

The underived transformation is the adjoint of $f_*\eta^X$. Naturality of η^X proves naturality in F; the identity formula and the triangle identities prove the identity and composition equations. Scheme naturality identifies the two restrictions in the displayed equality $f_{\mathrm{et},*}\epsilon_{X,*} = \epsilon_{Y,*}f_{\mathrm{proet},*}$ and makes the construction compatible with Cartesian pullback.

For the derived map choose a bounded-below injective resolution $L \to I^\bullet$ on X_{et}, and then a bounded-below injective resolution $\epsilon_X^{-1}I^\bullet \to J^\bullet$ on X_{proet}. Exactness of ϵ_X^{-1} and ϵ_Y^{-1} identifies the two derived source representatives, and the complex map

$$\epsilon_Y^{-1}f_{\mathrm{et},*}I^\bullet \xrightarrow{\delta_{f,I^\bullet}} f_{\mathrm{proet},*}\epsilon_X^{-1}I^\bullet \longrightarrow f_{\mathrm{proet},*}J^\bullet$$

represents $D_f(L)$. Comparisons between two choices of either resolution are unique up to homotopy; naturality of δ_f carries both comparison homotopies to the target, so the localized map is independent of every choice. The underived identity, composition, and base-change equations persist after localization. Thus $D_f(L)$ and all its stated coherence fields are well-defined. □

Lemma 11.9.62 (Site-local components of direct-image comparison)**.** *Lean:* `conditional-interface:lemma:weil-pbc-epsilon-direct-image-site-comparison`

Uses: `def:weil-pbc-epsilon-direct-image-comparison`

Suppose Y is affine and let $W = \varprojlim_i W_i$ be an affine pro-etale object, with $W_i \to Y$ affine etale and affine transition maps. Put $X_i = X \times_Y W_i$ and $X_W = X \times_Y W$. Then

$$(\epsilon_Y^{-1}f_{\mathrm{et},*}F)(W) = \varinjlim_i \Gamma(X_i, F),$$

$$(f_{\mathrm{proet},*}\epsilon_X^{-1}F)(W) = \Gamma(X_W, \epsilon_{X_W}^{-1}g_{\mathrm{et}}^{-1}F),$$

where $g : X_W \to X$. Under these identifications, $\delta_{f,F}(W)$ is the map induced by pullback from the finite stages. Such W form a covering basis of Y_{proet}.

Proof. Uses: `lemma:weil-pbc-epsilon-scheme-naturality`

The first equality is the left-Kan-extension formula for ϵ_Y^{-1}: the subsystem of the displayed affine etale stages is cofinal among etale objects receiving a map from W. The second equality is the definition of pro-etale direct image, followed by the scheme-naturality identification of the restricted inverse image. Expanding the adjoint of $f_{\mathrm{et},*}\eta_F^X$ shows that a section over X_i is pulled back to X_W; hence its filtered-colimit map is exactly $\delta_{f,F}(W)$. Every pro-etale object over affine Y admits a cover by inverse limits of this form, so these evaluations detect sheaf isomorphisms. □

Lemma 11.9.63 (Higher direct-image comparison on finite sheaves)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-pbc-epsilon-direct-image-higher-sheaves`

Let A be finite, let $f : X \to Y$ be quasi-compact and quasi-separated, and let F be a constructible A-module sheaf on X_{et}. For every $r \geq 0$, the canonical map

$$\epsilon_Y^{-1}R^r f_{\mathrm{et},*}F \longrightarrow R^r f_{\mathrm{proet},*}\epsilon_X^{-1}F$$

is an isomorphism.

Proof. Uses: `lemma:weil-pbc-epsilon-direct-image-site-comparison,lemma:weil-pbc-epsilon-derived-unit`

The assertion is Zariski local on Y, so take Y affine and use the basis $W = \varprojlim_i W_i$ from Lemma 11.9.62. The left side at W is

$$\varinjlim_i H^r_{\mathrm{et}}(X_i, F).$$

For the right side, apply Lemma 11.9.58 on X_W to $g_{\mathrm{et}}^{-1}F$. It identifies its value at W with

$$H^r_{\mathrm{proet}}(X_W, \epsilon_{X_W}^{-1}g_{\mathrm{et}}^{-1}F) = H^r_{\mathrm{et}}(X_W, g_{\mathrm{et}}^{-1}F).$$

Because f is quasi-compact and quasi-separated, X_W is the inverse limit of the X_i with affine transition maps and etale cohomology is continuous for this system. Hence

$$\varinjlim_i H^r_{\mathrm{et}}(X_i, F) \xrightarrow{\sim} H^r_{\mathrm{et}}(X_W, g_{\mathrm{et}}^{-1}F).$$

The component description in the preceding lemma identifies this isomorphism with the canonical comparison map. The basis evaluations are conservative, proving the sheaf assertion. □

Lemma 11.9.64 (Finite constructibility and lower bounds). *Uses:*

Lean: `conditional-interface:lemma:weil-pbc-epsilon-direct-image-constructible-bounds`

Let A be finite and let $L \in D_c^+(X_{\mathrm{et}}, A)$ vanish in degrees $< a$. Then $\epsilon_X^{-1}L$ has constructible cohomology and also vanishes in degrees $< a$. Both $Rf_{\mathrm{et},*}L$ and $Rf_{\mathrm{proet},*}\epsilon_X^{-1}L$ vanish in degrees $< a$.

Proof. Uses: `lemma:weil-pbc-epsilon-inverse-image-exact,lemma:weil-pbc-epsilon-scheme-naturality`

Exactness gives

$$\mathcal{H}^q(\epsilon_X^{-1}L) = \epsilon_X^{-1}\mathcal{H}^q(L).$$

For each q, choose a finite stratification on which $\mathcal{H}^q(L)$ is locally constant with finite stalks. Scheme naturality identifies its inverse image on each pro-etale stratum with the same finite locally constant sheaf, so that stratification witnesses constructibility of the corresponding cohomology sheaf. The equality also preserves the lower vanishing bound. Right-derived direct image has no negative derived degree; its hypercohomology terms have $p \geq 0$ and $q \geq a$, which gives the same lower bound for both direct images. □

Lemma 11.9.65 (Naturality of direct-image comparison). *Lean:* `conditional-interface:lemma:weil-pbc-epsilon-direct-image-naturality`

Uses: `def:weil-pbc-epsilon-direct-image-comparison`

The maps $D_f(L)$ are natural in L, equal the identity for $f = 1_X$, and respect composition of quasi-compact quasi-separated morphisms. For a Cartesian square

$$\begin{array}{ccc} X' & \xrightarrow{v} & X \\ {\scriptstyle f'}\downarrow & & \downarrow{\scriptstyle f} \\ Y' & \xrightarrow{u} & Y, \end{array}$$

the square formed by $u^*D_f(L)$, $D_{f'}(v^*L)$, and the two canonical base-change transformations commutes.

Proof. Uses: `lemma:weil-pbc-epsilon-unit-counit-naturality,lemma:weil-pbc-epsilon-scheme-naturality`

The underived map is adjoint to $f_*\eta^X$. Naturality of η^X gives naturality in F, its identity formula gives the identity case, and the triangle identity shows that the adjoint for a composite is the ordered composite of the two adjoints. Deriving preserves these equations.

For the Cartesian square, expand both routes before taking adjoints. Scheme naturality identifies $v^*\eta_L^X$ with $\eta_{v^*L}^{X'}$, and both routes then consist of this unit followed by restriction along the same projection $X' \to X$ and the same base-change map. They are equal before derivation, hence the derived base-change square commutes. □

Lemma 11.9.66 (Comparison direct-image transport). *Lean:* `conditional-interface:lemma:weil-pbc-epsilon-direct-image`

Uses: `def:weil-pbc-epsilon-direct-image-comparison`

Let A be finite, let $f : X \to Y$ be quasi-compact and quasi-separated, and let $L \in D_c^+(X_{\mathrm{et}}, A)$. The canonical map

$$D_f(L) : \epsilon_Y^{-1}Rf_{\mathrm{et},*}L \longrightarrow Rf_{\mathrm{proet},*}\epsilon_X^{-1}L$$

is an isomorphism, natural in L, in f, and in Cartesian base change.

Proof. Uses: `lemma:weil-pbc-epsilon-direct-image-higher-sheaves,lemma:weil-pbc-epsilon-direct-image-constructible-bounds,lemma:weil-pbc-epsilon-direct-image-naturality,lemma:weil-pbc-hypercohomology-convergence`

Choose a below the cohomology of L. The truncation towers on both sides give morphisms of hypercohomology spectral sequences whose second pages are

$$\epsilon_Y^{-1}R^p f_{\mathrm{et},*}\mathcal{H}^q(L) \longrightarrow R^p f_{\mathrm{proet},*}\epsilon_X^{-1}\mathcal{H}^q(L).$$

Every component is an isomorphism by Lemma 11.9.63. The bounds lemma puts both spectral sequences in $p \geq 0$, $q \geq a$. Thus each total degree has a finite exhaustive separated filtration, by the numerical argument of Lemma 11.8.2. Page induction and finite-filtration induction make the abutment map an isomorphism in every degree. Hence $D_f(L)$ is an isomorphism in the derived category. Its naturality statements are Lemma 11.9.65. □

Proposition 11.9.67 (Finite-level pro-etale proper base change). Lean: `conditional-interface:prop:weil-pbc-proet-finite-level-transport`

Uses: `def:weil-pbc-comes-from-etale`

Let A be a finite torsion ring and let (L,θ) witness that $K \in D^+(X_{\mathrm{proet}}, A)$ comes from the etale site, with $L \in D_c^+(X_{\mathrm{et}}, A)$. For a proper Cartesian square, the pro-etale exchange map

$$Lg^* Rf_* K \longrightarrow Rf'_* L(g')^* K$$

is an isomorphism.

Proof. Uses: `lemma:weil-pbc-epsilon-pullback,lemma:weil-pbc-epsilon-direct-image,prop:weil-proet-finite-derived-proper-base-change`

Use θ to replace K by $\epsilon_X^{-1} L$. Commute ϵ^{-1} past the two pullbacks and two direct images using the preceding lemmas. The resulting middle arrow is $\epsilon_{Y'}^{-1}$ applied to the S7 etale exchange map and is therefore an isomorphism. Expanding all four comparison isomorphisms shows the transported arrow is the pro-etale unit–counit exchange map; naturality follows from the two comparison squares. □

11.10 S9. Derived completion and inverse limits

Definition 11.10.1 (Adic coefficient datum and reduction tower). Uses:

Lean: `conditional-interface:def:weil-pbc-complete-coefficient-datum`

Let Λ be Noetherian, let $I = (f_1, \dots, f_r) \subseteq \Lambda$ have Λ/I torsion, and suppose every $P_n := \Lambda/I^n$ is perfect over Λ. For $K \in D(X_{\mathrm{proet}}, \Lambda)$ put

$$K_n := K \otimes^L_\Lambda P_n.$$

For $b \geq a$, let $q_{b,a} : K_b \to K_a$ be induced by $P_b \to P_a$. The quotient identities give $q_{a,a} = 1$ and $q_{c,a} = q_{b,a} q_{c,b}$. Associativity also gives the normalization isomorphisms

$$K_b \otimes^L_{P_b} P_a \xrightarrow{\sim} K_a,$$

and these obey the same cocycle.

Definition 11.10.2 (Explicit adic derived inverse limit). Lean: `conditional-interface:def:weil-pbc-adic-derived-inverse-limit`

Uses: `def:weil-pbc-complete-coefficient-datum`

For a tower $M_\bullet = (M_n, t_{n+1,n})$, put

$$s((x_n)_n) = (t_{n+1,n}(x_{n+1}))_n, \qquad R\varprojlim_n M_n := \mathrm{Cone}\left(1 - s : \prod_n M_n \longrightarrow \prod_n M_n\right)[-1].$$

Thus there is a functorial distinguished triangle

$$R\varprojlim_n M_n \longrightarrow \prod_n M_n \xrightarrow{1-s} \prod_n M_n \longrightarrow .$$

Maps of towers act componentwise on the two products and hence on the cone.

Lemma 11.10.3 (Milnor sequence on pro-etale test objects). Lean: `conditional-interface:lemma:weil-pbc-adic-milnor-sequence`

Uses: `def:weil-pbc-adic-derived-inverse-limit`

Let U be a weakly contractible quasi-compact object of the pro-etale site. For every tower $M_\bullet$ and every i there is a functorial exact sequence

$$0 \longrightarrow \varprojlim{}^1_n H^{i-1}(U, M_n) \longrightarrow H^i\left(U, R\varprojlim_n M_n\right) \longrightarrow \varprojlim_n H^i(U, M_n) \longrightarrow 0.$$

Proof. Uses:

Weak contractibility makes $\Gamma(U, -)$ exact, and evaluation on U commutes with products. Apply it to the product-cone triangle. The long exact cohomology sequence identifies the kernel of $1 - s$ on $\prod_n H^i(U, M_n)$ with $\varprojlim H^i(U, M_n)$ and its cokernel in degree $i-1$ with $\varprojlim{}^1 H^{i-1}(U, M_n)$. Kernel–cokernel exactness gives the displayed sequence, and every construction is natural in a map of towers. □

Lemma 11.10.4 (Finite towers satisfy Mittag–Leffler)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-pbc-adic-mittag-leffler`

For an inverse system of finite abelian groups (A_n), the images $\mathrm{Im}(A_m \to A_n)$ are constant for all sufficiently large $m \geq n$; hence (A_n) is Mittag–Leffler.

Proof. Uses:

For fixed n these images form a descending chain of subgroups of the finite group A_n. Such a chain stabilizes. Doing this for every n is exactly the Mittag–Leffler condition. □

Lemma 11.10.5 (Mittag–Leffler kills the Milnor obstruction)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-pbc-adic-limit-one-zero`

For a countable Mittag–Leffler system (A_n) one has $\varprojlim{}^1_n A_n = 0$. In particular this holds for a tower of finite abelian groups.

Proof. Uses: `lemma:weil-pbc-adic-mittag-leffler`

Replace A_n by the stable image in A_n; this does not change either $\varprojlim$ or the cokernel description of $\varprojlim{}^1$, and the new transition maps are surjective. Given $(y_n) \in \prod_n A_n$, choose x_1 arbitrarily and then choose x_{n+1} recursively so that $x_n - t_{n+1,n}(x_{n+1}) = y_n$. Thus $1 - s$ is surjective on the product and its cokernel, namely $\varprojlim{}^1 A_n$, is zero. □

Definition 11.10.6 (Intrinsic Koszul completion)**.** Lean: `conditional-interface:def:weil-pbc-intrinsic-koszul-completion`

Uses: `def:weil-pbc-adic-derived-inverse-limit`

Let

$$Q_m := K(\Lambda; f_1^m, \dots, f_r^m)$$

be the Koszul complex in degrees $[-r, 0]$. Its map $Q_{m+1} \to Q_m$ is the tensor product, over j, of the two-term chain maps whose degree-zero component is the identity and whose degree-minus-one component is multiplication by f_j. Define

$$K^{\wedge,\mathrm{Kos}} := R\varprojlim_m (K \otimes^L_\Lambda Q_m).$$

The augmentations $\Lambda \to Q_m$ give $\kappa_K : K \to K^{\wedge,\mathrm{Kos}}$. Call K intrinsically derived I-complete when κ_K is an isomorphism; below, "derived I-complete" always means this intrinsic condition.

Definition 11.10.7 (Completion map)**.** Lean: `conditional-interface:def:weil-pbc-completion-map`

Uses: `def:weil-pbc-adic-derived-inverse-limit`

Define

$$\mathrm{comp}_K : K \longrightarrow R\varprojlim_n K_n$$

as the map induced by the compatible quotient arrows $K \to K_n$.

Lemma 11.10.8 (Koszul and quotient towers have the same limit)**.** *Lean:* `conditional-interface:lemma:weil-pbc-koszul-quotient-limit`

Uses: `def:weil-pbc-intrinsic-koszul-completion`, `def:weil-pbc-completion-map`

There is a functorial isomorphism

$$c_K : K^{\wedge,\mathrm{Kos}} \xrightarrow{\sim} R\varprojlim_n K_n$$

such that $c_K \kappa_K = \mathrm{comp}_K$.

Proof. Uses:

Put $J_m = (f_1^m, \dots, f_r^m)$. The augmentation $Q_m \to \Lambda/J_m$ has cone $\tau_{<0} Q_m$. For $p < 0$ the modules $H^p(Q_m)$ are finite and killed by J_m. The transition in degree p has entries in $(f_1^{b-a}, \dots, f_r^{b-a})$ from level b to level a. Artin–Rees, applied to the cycles inside the finite free term Q_a^p, therefore makes $H^p(Q_b) \to H^p(Q_a)$ zero for $b \gg a$. Hence $(\tau_{<0} Q_m)_m$ is pro-zero and $(Q_m)_m \to (\Lambda/J_m)_m$ is a pro-isomorphism.

The inclusions

$$I^{r(m-1)+1} \subseteq J_m \subseteq I^m$$

show that $(\Lambda/J_m)_m$ and $(P_n)_n$ are cofinal pro-systems. Tensor the comparison maps with K and apply the product-cone limit. Pro-zero cones have zero product-cone limit after a cofinal reindexing, so the result is c_K. Both units are induced by the same augmentations $\Lambda \to Q_m \to \Lambda/J_m \to P_n$; this proves the triangle equation. □

Lemma 11.10.9 (Derived-complete reconstruction). *Lean:* `conditional-interface:lemma:weil-pbc-derived-complete-reconstruction`

Uses: `def:weil-pbc-intrinsic-koszul-completion`

If K is intrinsically derived I-complete, then

$$\mathrm{comp}_K : K \xrightarrow{\sim} R\varprojlim_n K_n$$

is an isomorphism, functorially in K.

Proof. Uses: `lemma:weil-pbc-koszul-quotient-limit`

By intrinsic completeness, κ_K is an isomorphism. The preceding lemma identifies comp_K with the composite $c_K\kappa_K$, so it too is an isomorphism. For $u : K \to L$, naturality of the Koszul augmentations and of the product-cone construction gives

$$(R\varprojlim u_n)\,\mathrm{comp}_K = \mathrm{comp}_L\, u.$$

Conjugating by the two reconstruction isomorphisms proves functoriality, including functoriality of their inverses. This is the explicit Koszul-to-adic argument behind Tag `099M`. □

Lemma 11.10.10 (Finite reduction of adic completion). *Uses:*

Lean: `conditional-interface:lemma:weil-pbc-adic-completion-reduction`

For every $a \geq 1$, projection to level a induces a functorial isomorphism

$$\left(R\varprojlim_n K_n\right) \otimes^L_\Lambda P_a \xrightarrow{\sim} K_a.$$

Proof. Uses: `lemma:weil-pbc-koszul-quotient-limit`

Use the preceding comparison to replace the quotient tower by $K \otimes^L Q_m$. Since P_a is perfect, tensoring with P_a commutes with both products in the product-cone model. For $m \geq a$, the differential of $Q_m \otimes^L P_a$ is zero. Its degree-zero transition is the identity, whereas every composite of a successive transitions in a negative degree contains a factor $f_j^a \in I^a$ and is zero over P_a. Thus the negative-degree towers are pro-zero and the degree-zero tower is the constant tower P_a. After tensoring with K, its product-cone limit is $K \otimes^L P_a = K_a$. The surviving degree-zero projection is precisely the displayed map. This also exposes the finite Tor-amplitude input: the bounded finite-projective complex representing P_a is what permits it to pass through the two countable products. □

Definition 11.10.11 (Uniformly bounded constructible adic tower). *Lean:* `conditional-interface:def:weil-pbc-bounded-constructible-adic-tower`

Uses: `def:weil-pbc-complete-coefficient-datum`

Say that the reduction tower $K_\bullet$ attached to K is uniformly bounded constructible if there are integers $d_0 \leq d_1$ and one finite stratification of X, each of whose strata has finitely many connected components, such that, for every n, $H^i(K_n) = 0$ outside $[d_0, d_1]$ and each $H^i(K_n)$ is finite lisse on every stratum. The normalization isomorphisms and their cocycle are part of this datum.

Lemma 11.10.12 (Bounded constructible towers have no Milnor obstruction). *Lean:* `conditional-interface:lemma:weil-pbc-constructible-adic-lim-one-zero`

Uses: `def:weil-pbc-bounded-constructible-adic-tower`

If the reduction tower $K_\bullet$ attached to K is uniformly bounded constructible, then on every weakly contractible quasi-compact U lying over one of the displayed strata,

$$H^i\left(U, R\varprojlim_n K_n\right) \xrightarrow{\sim} \varprojlim_n H^i(U, K_n),$$

and both sides vanish outside $[d_0, d_1]$.

Proof. Uses: `lemma:weil-pbc-adic-milnor-sequence,lemma:weil-pbc-adic-lim-one-zero`

Fix n and i. As m increases, the image sheaves of $H^i(K_m) \to H^i(K_n)$ form a descending chain of finite lisse subsheaves on each displayed stratum. On each connected component their fibers are descending chains of subgroups of a finite group, so they stabilize. There are only finitely many strata and, by hypothesis, finitely many connected components on each stratum; hence one index works globally. Exactness of $\Gamma(U, -)$ identifies the image of $H^i(U, K_m) \to H^i(U, K_n)$ with the sections of that image sheaf. Consequently $(H^i(U, K_n))_n$ is Mittag–Leffler even when U has infinitely many connected components. The vanishing lemma now kills its $\varprojlim^1$. The Milnor sequence is therefore the displayed isomorphism, and uniform finite-level vanishing gives the last assertion. □

Lemma 11.10.13 (Bounded constructibility survives reconstruction)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-pbc-bounded-constructible-reconstruction`

If the reduction tower $K_\bullet$ attached to K is uniformly bounded constructible, then $\widehat{K} := R\varprojlim_n K_n$ is intrinsically derived I-complete, has finite reduction $\widehat{K} \otimes^L P_n \simeq K_n$, and retains the common cohomological interval $[d_0, d_1]$ on the conservative weakly contractible basis subordinate to the displayed stratification.

Proof. Uses: `lemma:weil-pbc-adic-completion-reduction,lemma:weil-pbc-constructible-adic-lim-one-zero,lemma:weil-pbc-koszul-quotient-limit`

Finite reduction is the preceding reduction lemma. Hence the quotient completion of $\widehat{K}$ is the limit of the same tower and its completion map is the identity under the product-cone model. The Koszul–quotient comparison then makes its intrinsic Koszul unit an isomorphism. The common cohomological bounds and the absence of a hidden degree from $\varprojlim^1$ are exactly the conclusion of the preceding lemma. Weakly contractible objects subordinate to a finite stratification form a conservative basis, so these bounds detect the bounds of $\widehat{K}$. □

Definition 11.10.14 (Completed pullback)**.** Lean: `conditional-interface:def:weil-pbc-completed-pullback`

Uses: `def:weil-pbc-complete-coefficient-datum`

For $g : Y' \to Y$ define

$$Lg^*_{\mathrm{comp}} K := R\varprojlim_n \big(Lg^* K \otimes^L_\Lambda \Lambda/I^n \big).$$

The comparison from ordinary pullback is the completion map of Lg^*K.

Lemma 11.10.15 (Reduction commutes with completed pullback)**.** *Lean:* `conditional-interface:lemma:weil-pbc-completed-pullback-reduction`

Uses: `def:weil-pbc-completed-pullback`

For every n,

$$(Lg^*_{\mathrm{comp}} K) \otimes^L_\Lambda \Lambda/I^n \xrightarrow{\sim} Lg^* K_n.$$

Proof. Uses: `lemma:weil-pbc-adic-completion-reduction`

Derived pullback commutes with derived tensor, so its defining tower is the reduction tower of Lg^*K. Apply finite reduction of adic completion to this object. Its level n is $Lg^*K \otimes^L_\Lambda P_n \simeq Lg^*(K \otimes^L_\Lambda P_n) = Lg^* K_n$. □

Definition 11.10.16 (Adic operator and base-change datum)**.** Lean: `conditional-interface:def:weil-pbc-adic-operator-base-change-datum`

Uses: `def:weil-pbc-completed-pullback`

Let $g : Y' \to Y$, let K be on Y_{proet}, and let L be on Y'_{proet}. An adic operator and base-change datum consists of compatible tower endomorphisms

$$\phi_n : K_n \to K_n, \qquad \psi_n : L_n \to L_n,$$

and compatible maps $b_n : Lg^* K_n \to L_n$ such that

$$b_n \, Lg^*(\phi_n) = \psi_n b_n$$

for every n. Compatibility includes the transition squares, the normalization squares, and their cocycles. This is a datum of ordinary endomorphisms. A genuine Frobenius or Weil structure has the different type $\mathrm{Fr}^* K_n \to K_n$ and is not identified with this datum.

Lemma 11.10.17 (Reconstruction preserves operators and base change)**.** *Lean:* `conditional-interface:lemma:weil-pbc-adic-reconstruction-compatibility`

Uses: `def:weil-pbc-adic-operator-base-change-datum`

An adic operator and base-change datum induces maps

$$\widehat{\phi} = R\varprojlim_n \phi_n, \quad \widehat{\psi} = R\varprojlim_n \psi_n, \quad \widehat{Lg^*\phi} = R\varprojlim_n Lg^* \phi_n, \quad \widehat{b} = R\varprojlim_n b_n : Lg^*_{\mathrm{comp}} K \longrightarrow \widehat{L}$$

satisfying $\widehat{b}\,\widehat{Lg^*\phi} = \widehat{\psi}\widehat{b}$. Under the canonical finite-reduction isomorphisms, reduction modulo I^n gives exactly ϕ_n, ψ_n, b_n. If K and L are intrinsically derived complete, the reconstruction isomorphisms identify these with operators and a base-change map on K and L themselves.

Proof. Uses: `lemma:weil-pbc-adic-completion-reduction`

Compatibility with transitions makes the three families maps of towers. They therefore give maps between the two product-cone diagrams. The levelwise equations give an equality on both product terms, hence on their fibres, proving the displayed completed equation. The finite-reduction lemma identifies projection to the nth factor with reduction modulo I^n, so the reduced maps are precisely the named level maps. Finally, conjugate by the functorial reconstruction isomorphisms for complete K and L; the same equality survives conjugation. □

Definition 11.10.18 (Local strict-perfect coefficient package)**.** Uses:

Lean: `conditional-interface:def:weil-pbc-strict-perfect-coefficient-package`

On a localization of a ringed topos, let Q be represented by a bounded complex $F^\bullet$ of finite projective Λ-modules. A strict-perfect package for this representation consists, for every r, of an integer N_r and maps

$$F^r \xrightarrow{i_r} \Lambda^{N_r} \xrightarrow{p_r} F^r, \qquad p_r i_r = 1.$$

Write $e_r = i_r p_r$; then $e_r^2 = e_r$, $p_r e_r = p_r$, and $e_r i_r = i_r$. Perfectness means that such packages exist after a covering. When $Q \in D(\Lambda)$ is a perfect coefficient complex, one global package and its associated constant sheaf may be used.

Definition 11.10.19 (Projection-formula arrows)**.** Lean: `conditional-interface:def:weil-pbc-projection-formula-arrows`

Uses: `def:weil-pbc-strict-perfect-coefficient-package`

For a morphism of ringed topoi $h : \mathcal{T} \to \mathcal{S}$, an object $E \in D(\mathcal{T}, \Lambda)$, and a strict-perfect package $F^\bullet$ on $\mathcal{S}$, define

$$\pi_{h,E,F} : Rh_*E \otimes^L_\Lambda F^\bullet \longrightarrow Rh_*(E \otimes^L_\Lambda Lh^*F^\bullet)$$

to be adjoint to

$$Lh^*(Rh_*E \otimes^L F^\bullet) \simeq Lh^*Rh_*E \otimes^L Lh^*F^\bullet \xrightarrow{\epsilon_E \otimes 1} E \otimes^L Lh^*F^\bullet.$$

For $F^r = \Lambda^{N_r}$, let ρ_r be the inverse of the finite-sum identification

$$(Rh_*E)^{N_r} \xrightarrow{\pi_r} Rh_*(E^{N_r}).$$

For a projective term with the displayed splitting, define

$$\rho_{F^r} := (Rh_*E \otimes p_r)\, \rho_r\, Rh_*(1_E \otimes h^*i_r).$$

The bounded totalization of the ρ_{F^r} is denoted $\rho_{h,E,F}$.

Lemma 11.10.20 (Finite-free projection formula)**.** Lean: `conditional-interface:lemma:weil-pbc-projection-formula-finite-free`

Uses: `def:weil-pbc-projection-formula-arrows`

For $F = \Lambda^N[a]$, the maps $\pi_{h,E,F}$ and $\rho_{h,E,F}$ are inverse:

$$\rho_{h,E,F}\pi_{h,E,F} = 1, \qquad \pi_{h,E,F}\rho_{h,E,F} = 1.$$

Proof. Uses:

Both source and target are the shift by a of a finite direct sum of N copies of Rh_*E. A right adjoint preserves finite products, and in an additive category finite products are finite coproducts. Under these two identifications, the counit defining π is the identity in every summand. The map ρ is the reverse finite-sum identification, so the two displayed composites are componentwise identities. □

Lemma 11.10.21 (Finite-projective summand equations)**.** Lean: `conditional-interface:lemma:weil-pbc-projection-formula-projective-summand`

Uses: `def:weil-pbc-projection-formula-arrows`

For a finite projective F^r and a displayed splitting (i_r, p_r), the arrows π_{h,E,F^r} and ρ_{F^r} are inverse and are independent of the chosen finite-free summand presentation.

Proof. Uses: `lemma:weil-pbc-projection-formula-finite-free`

Let $A(u) = Rh_*E \otimes u$ and $B(u) = Rh_*(E \otimes h^*u)$. Naturality of the counit gives

$$B(i_r)\pi_{F^r} = \pi_r A(i_r), \qquad B(p_r)\pi_r = \pi_{F^r} A(p_r).$$

Using $p_r i_r = 1$ and the finite-free inverse equations, compute

$$\rho_{F^r}\pi_{F^r} = A(p_r)\rho_r B(i_r)\pi_{F^r} = A(p_r)\rho_r\pi_r A(i_r) = 1,$$

and

$$\pi_{F^r}\rho_{F^r} = \pi_{F^r} A(p_r)\rho_r B(i_r) = B(p_r)\pi_r\rho_r B(i_r) = 1.$$

Thus ρ_{F^r} is the unique inverse of the intrinsic map π_{F^r}; uniqueness removes the dependence on (i_r, p_r). □

Lemma 11.10.22 (Signed totalization of the perfect formula). Lean: `conditional-interface:lemma:weil-pbc-projection-formula-totalization`

Uses: `def:weil-pbc-projection-formula-arrows`

For a bounded finite-projective complex $F^\bullet$, the total arrows $\pi_{h,E,F}$ and $\rho_{h,E,F}$ are inverse isomorphisms.

Proof. Uses: `lemma:weil-pbc-projection-formula-projective-summand`

Choose a complex representing Rh_*E. On a homogeneous tensor $x \otimes y$ with x of degree a, both total complexes use

$$d(x \otimes y) = d(x) \otimes y + (-1)^a x \otimes d_F(y).$$

Naturality of π_{F^r} in a map of finite projective modules and uniqueness of its inverse give naturality of ρ_{F^r}. Apply these two naturality statements to every differential $d_F^r : F^r \to F^{r+1}$. They say the term maps commute with the horizontal differentials; their degree zero says they commute with the signed vertical differentials. Hence their finite totalizations are chain maps. On every bidegree the two composites are the identities proved in the summand lemma. Finite totalization preserves these equalities, so

$$\rho_{h,E,F}\pi_{h,E,F} = 1, \qquad \pi_{h,E,F}\rho_{h,E,F} = 1.$$

□

Definition 11.10.23 (Chosen compactification and compact-support image). Uses:

Lean: `conditional-interface:def:weil-pbc-projection-compactification`

Let $f : X \to Y$ admit a chosen compactification

$$X \xrightarrow{j} \overline{X} \xrightarrow{p} Y, \qquad f = pj,$$

with j a quasi-compact open immersion and p proper. For this datum set

$$Rf_!^{(j,p)}E := Rp_*j_!E.$$

For proper f, its identity compactification is $(j,p) = (1_X, f)$.

Lemma 11.10.24 (Open-immersion tensor piece). Lean: `conditional-interface:lemma:weil-pbc-projection-formula-open-piece`

Uses: `def:weil-pbc-projection-compactification`

For a bounded finite-projective complex $F^\bullet$ on $\overline{X}$, the canonical map

$$(j_!E) \otimes^L F^\bullet \xrightarrow{\sim} j_!(E \otimes^L Lj^*F^\bullet)$$

is an isomorphism, natural in E and $F^\bullet$.

Proof. Uses:

Extension by zero along an open immersion is exact. Resolve only E; tensoring with each finite projective term remains exact. At a point of the pro-etale topos lying over X, both sides are $E_x \otimes F_x^\bullet$ and the map is the identity. At a point lying over $\overline{X} \setminus X$, both sides are zero. These points form a conservative family, so the term map is an isomorphism. It commutes with d_F and with the signed total differential, hence gives the displayed derived isomorphism. □

Lemma 11.10.25 (Proper compactification piece). Lean: `conditional-interface:lemma:weil-pbc-projection-formula-proper-piece`

Uses: `def:weil-pbc-projection-compactification`

For the proper piece p and every strict-perfect $F^\bullet$ on Y,

$$(Rp_*j_!E) \otimes^L F^\bullet \xrightarrow[\pi_{p,j_!E,F}]{\sim} Rp_*(j_!E \otimes^L Lp^*F^\bullet),$$

with inverse $\rho_{p,j_!E,F}$.

Proof. Uses: `lemma:weil-pbc-projection-formula-totalization`

Apply the signed-totalization lemma to the morphism of ringed pro-etale topoi induced by p and the object $j_!E$. □

Lemma 11.10.26 (Compactified finite-level projection formula)**.** *Uses:*

Lean: `conditional-interface: lemma: weil-pbc-projection-formula-compactified`

For a chosen compactification and a perfect Q on Y, the ordered composite

$$\begin{aligned}(Rf_!^{(j,p)}E)\otimes^L Q = (Rp_*j_!E)\otimes^L Q \\ \xrightarrow{\pi_{p,j_!E,Q}} Rp_*(j_!E\otimes^L Lp^*Q) \\ \xrightarrow{Rp_*\theta_j} Rp_*j_!(E\otimes^L Lf^*Q) = Rf_!^{(j,p)}(E\otimes^L Lf^*Q)\end{aligned}$$

is an isomorphism. Here θ_j is the open-piece isomorphism and the inverse is $\rho_{p,j_!E,Q}\,Rp_*(\theta_j^{-1})$ in reverse order.

Proof. Uses: `lemma:weil-pbc-projection-formula-open-piece,lemma:weil-pbc-projection-formula-proper-piece`

The assertion is local on Y. On a cover choose a strict-perfect package for Q. The first arrow and its displayed inverse are the proper-piece inverse pair. The second arrow and its inverse come from the open-piece inverse pair, using $Lj^*Lp^*Q = Lf^*Q$. Associativity gives

$$\rho_{p,j_!E,Q}(Rp_*\theta_j^{-1})(Rp_*\theta_j)\pi_{p,j_!E,Q} = 1$$

and

$$(Rp_*\theta_j)\pi_{p,j_!E,Q}\rho_{p,j_!E,Q}(Rp_*\theta_j^{-1}) = 1.$$

These are precisely the two inverse equations for the ordered composite. They prove that the globally defined forward map is an isomorphism; uniqueness of an inverse glues the local reverse maps. □

Lemma 11.10.27 (Identity compactification recovers direct image)**.** *Uses:*

Lean: `conditional-interface: lemma: weil-pbc-projection-formula-identity-compactification`

If f is proper, the identity compactification identifies $Rf_!^{(1,f)}$ with Rf_*, makes θ_1 the identity, and identifies the compactified formula with $\pi_{f,E,Q}$ and its inverse with $\rho_{f,E,Q}$.

Proof. Uses: `lemma:weil-pbc-projection-formula-compactified`

For $j = 1_X$, extension by zero, inverse image, and both unit constraints are identities. Substitution into the ordered composite leaves $\pi_{f,E,Q}$; substitution into its inverse leaves $\rho_{f,E,Q}$. □

Lemma 11.10.28 (Projection formulas form a coefficient tower)**.** *Lean:* `conditional-interface: lemma: weil-pbc-projection-formula-tower-compatibility`

Uses: `def: weil-pbc-complete-coefficient-datum, def: weil-pbc-projection-formula-arrows`

For every n, let $q_n : P_{n+1} \to P_n$ be the quotient. The square

$$\begin{array}{ccc}(Rf_*K)\otimes^L P_{n+1} & \xrightarrow{\pi_{n+1}} & Rf_*(K\otimes^L P_{n+1}) \\ \downarrow{\scriptstyle 1\otimes q_n} & & \downarrow{\scriptstyle Rf_*(1\otimes q_n)} \\ (Rf_*K)\otimes^L P_n & \xrightarrow{\pi_n} & Rf_*(K\otimes^L P_n)\end{array}$$

commutes, and so does the square with every π replaced by ρ and horizontal arrows reversed; its vertical arrows remain the two quotient maps.

Proof. Uses:

The two adjoints of the forward composites are both

$$Lf^*Rf_*K\otimes^L P_{n+1} \xrightarrow{\epsilon_K\otimes 1} K\otimes^L P_{n+1} \xrightarrow{1\otimes q_n} K\otimes^L P_n.$$

Thus the forward square commutes. Invert the two horizontal isomorphisms in the equality $Rf_*(1\otimes q_n)\pi_{n+1} = \pi_n(1\otimes q_n)$ to obtain the asserted compatibility for ρ. □

Lemma 11.10.29 (Operator and base-change naturality of the formula)**.** *Lean:* `conditional-interface: lemma: weil-pbc-projection-formula-base-change-naturality`

Uses: `def: weil-pbc-cartesian-square, def: weil-pbc-projection-formula-arrows`

The projection-formula arrows are natural in K; hence for every endomorphism $\phi : K \to K$ with reductions $\phi_n : K_n \to K_n$,

$$Rf_*\phi_n\,\pi_n = \pi_n((Rf_*\phi)\otimes 1), \qquad ((Rf_*\phi)\otimes 1)\rho_n = \rho_n Rf_*\phi_n.$$

For a Cartesian square, the following two routes are equal:

$$Lg^*((Rf_*K)\otimes^L Q)\xrightarrow{Lg^*\pi_f} Lg^*Rf_*(K\otimes^L Lf^*Q)\xrightarrow{\mathrm{Ex}_{K\otimes Lf^*Q}} Rf'_*L(g')^*(K\otimes^L Lf^*Q),$$
$$Lg^*((Rf_*K)\otimes^L Q)\xrightarrow{\sim}(Lg^*Rf_*K)\otimes^L Lg^*Q\xrightarrow{\mathrm{Ex}_K\otimes 1}(Rf'_*L(g')^*K)\otimes^L Lg^*Q$$
$$\xrightarrow{\pi_{f'}} Rf'_*(L(g')^*K\otimes^L L(f')^*Lg^*Q).$$

The targets are identified by the chosen Cartesian isomorphism.

Proof. Uses:
For a map $u:K\to L$, both naturality composites are adjoint to

$$Lf^*Rf_*K\otimes Lf^*Q\xrightarrow{\epsilon_K\otimes 1}K\otimes Lf^*Q\xrightarrow{u\otimes 1}L\otimes Lf^*Q.$$

This proves the forward operator equation; conjugation by $\rho_n=\pi_n^{-1}$ gives the second. For base change, expand both exchange maps as unit followed by counit. After adjunction to X', both displayed routes become

$$L(g')^*Lf^*Rf_*K\otimes L(g')^*Lf^*Q\xrightarrow{L(g')^*(\epsilon_K)\otimes 1}L(g')^*K\otimes L(g')^*Lf^*Q.$$

The Cartesian identification $L(g')^*Lf^*=L(f')^*Lg^*$ identifies the coefficient factors. Hence the adjoints, and therefore the two routes, are equal. □

Lemma 11.10.30 (Perfect projection formula for reductions)**.** *Lean:* `conditional-interface:lemma:weil-pbc-perfect-projection-formula`

Uses: `def:weil-pbc-complete-coefficient-datum`
For every n there is a natural isomorphism

$$(Rf_*K)\otimes^L_\Lambda \Lambda/I^n\xrightarrow[\pi_n]{\sim}Rf_*K_n$$

whose inverse is ρ_n. The families (π_n) and (ρ_n) commute with the reduction tower, compatible coefficient endomorphisms, and base change.

Proof. Uses: `lemma:weil-pbc-projection-formula-totalization`, `lemma:weil-pbc-projection-formula-identity-compactification`, `lemma:weil-pbc-projection-formula-tower-compatibility`, `lemma:weil-pbc-projection-formula-base-change-naturality`
Choose a strict-perfect package $F_n^\bullet\to P_n$. Signed totalization gives inverse arrows

$$(Rf_*K)\otimes^L F_n^\bullet\underset{\rho_n}{\overset{\pi_n}{\rightleftarrows}}Rf_*(K\otimes^L Lf^*F_n^\bullet).$$

Because P_n is constant, $Lf^*P_n=P_n$, and the target is Rf_*K_n. The identity-compactification lemma shows that this is also the open/proper compactification composite for proper f. The totalization lemma gives $\rho_n\pi_n=1$ and $\pi_n\rho_n=1$. Uniqueness of an inverse makes the map independent of the strict-perfect package. Tower, operator, and base-change compatibility are exactly the preceding two lemmas. This expands the finite-projective argument located at Tag `0944`. □

Lemma 11.10.31 (Direct image commutes with sequential homotopy limits)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-pbc-direct-image-R-lim`
For an inverse tower M_n on X_{proet},

$$Rf_*R\varprojlim_n M_n\xrightarrow{\sim}R\varprojlim_n Rf_*M_n.$$

Proof. Uses:
Use the product/equalizer model

$$R\varprojlim M_n\longrightarrow\prod_n M_n\xrightarrow{1-\mathrm{shift}}\prod_n M_n$$

as a distinguished triangle. Since Rf_* is a right adjoint, it commutes with products. Applying it to the triangle gives the product/equalizer model for the right-hand limit, and the identity maps on the two products give the isomorphism of fibres. □

Definition 11.10.32 (Completed tensor product). Lean: `conditional-interface:def:weil-pbc-completed-tensor-product`

Uses: `def:weil-pbc-complete-coefficient-datum,def:weil-pbc-adic-derived-inverse-limit`

For A, B on the same ringed pro-etale topos, define

$$A \widehat{\otimes}^L_\Lambda B := R\varprojlim_n (A \otimes^L_\Lambda B \otimes^L_\Lambda P_n).$$

Thus $A\widehat{\otimes}^L\Lambda$ is the derived I-completion of A.

Lemma 11.10.33 (Finite reduction of completed tensor). Lean: `conditional-interface:lemma:weil-pbc-completed-tensor-reduction`

Uses: `def:weil-pbc-completed-tensor-product`

For every a there is a functorial isomorphism

$$(A\widehat{\otimes}^L B) \otimes^L P_a \xrightarrow{\sim} A \otimes^L B \otimes^L P_a.$$

Proof. Uses: `lemma:weil-pbc-adic-completion-reduction`

The completed tensor is the adic completion of $A \otimes^L B$. Apply finite reduction of adic completion to that object and use associativity of derived tensor product. □

Lemma 11.10.34 (Completed projection-formula composite). Lean: `conditional-interface:lemma:weil-pbc-completed-projection-formula-composite`

Uses: `def:weil-pbc-completed-tensor-product`

Let B be an object on Y_{proet} such that $B_n := B \otimes^L P_n$ is perfect for every n. Locally choose a strict-perfect package for each B_n. Then the ordered composite

$$\begin{aligned}(Rf_*E)\widehat{\otimes}^L B = R\varprojlim_n ((Rf_*E) \otimes^L B_n) \\ \xrightarrow{R\varprojlim \pi_{E,B_n}} R\varprojlim_n Rf_*(E \otimes^L Lf^*B_n) \\ \xrightarrow{(Rf_*\text{-}R\varprojlim)^{-1}} Rf_*R\varprojlim_n (E \otimes^L Lf^*B \otimes^L P_n) \\ = Rf_*(E\widehat{\otimes}^L Lf^*B)\end{aligned}$$

is an isomorphism, natural in E, B, compatible endomorphisms, and base-change squares. Its inverse is the reverse composite using direct-image/limit commutation and $R\varprojlim \rho_{E,B_n}$.

Proof. Uses: `lemma:weil-pbc-projection-formula-totalization,lemma:weil-pbc-projection-formula-tower-compatibility,lemma:weil-pbc-direct-image-r-lim`

Naturality in the coefficient maps $B_{n+1} \to B_n$ makes the levelwise projection formulas a map of towers. Every π_{E,B_n} is an isomorphism locally by signed totalization, with inverse ρ_{E,B_n}; uniqueness of an inverse glues the local inverses. The second displayed arrow is an isomorphism because direct image commutes with the product-cone limit. The forward composite followed by the stated reverse composite is

$$R\varprojlim(\rho_{E,B_n}\pi_{E,B_n}) = R\varprojlim 1 = 1;$$

the reverse order uses $R\varprojlim(\pi_{E,B_n}\rho_{E,B_n}) = 1$ and the two inverse equations for direct-image/limit commutation. Levelwise operator and base-change naturality passes through products and the product-cone fibre, proving the last assertion. □

Lemma 11.10.35 (Direct image commutes with derived completion). Uses:

Lean: `conditional-interface:lemma:weil-pbc-direct-image-derived-completion`

There is a functorial isomorphism

$$\widehat{Rf_*E} \xrightarrow{\sim} Rf_*\widehat{E}.$$

If E is intrinsically derived I-complete, then Rf_*E is intrinsically derived I-complete. Both statements commute with compatible endomorphisms and base change.

Proof. Uses: `lemma:weil-pbc-completed-projection-formula-composite,lemma:weil-pbc-derived-complete-reconstruction`

Apply the completed projection-formula composite with $B = \Lambda$; its finite reductions are P_n, which are perfect by the coefficient datum. This gives the displayed map and its explicit inverse. If E is complete, reconstruction identifies $\widehat{E}$ with E. Naturality of the composite identifies the resulting inverse $Rf_*E \to \widehat{Rf_*E}$ with the completion unit, so that unit is an isomorphism. The naturality assertion is inherited from the completed composite. □

Lemma 11.10.36 (Limits preserve compatible levelwise isomorphisms)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-pbc-r-lim-levelwise-isomorphism`

If $u_n : M_n \to N_n$ is a compatible map of towers and every u_n is an isomorphism, then $R\varprojlim u_n$ is an isomorphism.

Proof. Uses:

The maps $\prod_n u_n$ are isomorphisms and commute with $1-$shift. Taking fibres in the homotopy-limit triangles preserves the isomorphism. □

Lemma 11.10.37 (Complete cone detected by reductions)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-pbc-complete-cone-zero`

If C is derived complete and $C \otimes^L \Lambda/I^n = 0$ for every n, then $C = 0$.

Proof. Uses: `lemma:weil-pbc-derived-complete-reconstruction`

Reconstruction identifies C with the derived limit of its zero reductions; the product/equalizer model of the zero tower is zero. □

Lemma 11.10.38 (Perfectness of $\mathbf{Z}/\ell^n$)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-pbc-zl-reduction-perfect`

For $\Lambda = \mathbf{Z}_\ell$ and $I = (\ell)$, every Λ/I^n is perfect over Λ.

Proof. Uses:

The two-term complex

$$[\mathbf{Z}_\ell \xrightarrow{\ell^n} \mathbf{Z}_\ell]$$

in degrees $-1, 0$ consists of finite free modules. Multiplication by ℓ^n is injective and its cokernel is $\mathbf{Z}/\ell^n$, so the complex is a finite projective resolution. □

Definition 11.10.39 (Canonical finite reduction tower over $\mathbf{Z}_\ell$)**.** Lean: `conditional-interface:def:weil-efi-coeff-zl-reduction-tower`

Uses: `def:weil-a2-derived-category,def:weil-pbc-complete-coefficient-datum`

Constructed by: `lemma:weil-pbc-zl-reduction-perfect`

Specialize the adic coefficient construction to $\Lambda = \mathbf{Z}_\ell$, $I = (\ell)$, and $P_a = \mathbf{Z}_\ell/\ell^a$. For $K \in \mathcal{D}(X, \Lambda)$ set

$$K_a := K \otimes^L_\Lambda P_a, \qquad q_{b,a} : K_b \longrightarrow K_a \quad (b \geq a)$$

using the quotient map $P_b \to P_a$.

Lemma 11.10.40 (Identity equation for the canonical reduction tower)**.** *Lean:* `conditional-interface:lemma:weil-efi-coeff-zl-reduction-identity`

Uses: `def:weil-efi-coeff-zl-reduction-tower`

For every a, $q_{a,a} = \mathrm{id}_{K_a}$.

Proof. Uses: `def:weil-efi-coeff-zl-reduction-tower`

The coefficient map $P_a \to P_a$ is the identity, and derived tensor sends it to the identity of $K \otimes^L_\Lambda P_a$. □

Lemma 11.10.41 (Composition equation for the canonical reduction tower)**.** *Lean:* `conditional-interface:lemma:weil-efi-coeff-zl-reduction-composition`

Uses: `def:weil-efi-coeff-zl-reduction-tower`

For $c \geq b \geq a$, $q_{c,a} = q_{b,a} q_{c,b}$.

Proof. Uses: `def:weil-efi-coeff-zl-reduction-tower`

The two coefficient maps are induced by the equal quotient homomorphisms $P_c \to P_a$. Functoriality of derived tensor preserves that equality. □

Definition 11.10.42 (Normalization comparison for the canonical reduction tower)**.** Lean: `conditional-interface:def:weil-efi-coeff-zl-reduction-normalization`

Uses: `def:weil-efi-coeff-zl-reduction-tower`

For $b \geq a$, let

$$\nu_{b,a,K} : K_b \otimes^L_{P_b} P_a \longrightarrow K_a$$

be the derived-tensor associator followed by the multiplication isomorphism $P_b \otimes^L_{P_b} P_a \simeq P_a$.

Lemma 11.10.43 (The normalization comparison is invertible). Lean: `conditional-interface:lemma:weil-efi-coeff-zl-reduction-normalization-iso`

Uses: `def:weil-efi-coeff-zl-reduction-normalization`

The map $\nu_{b,a,K}$ is an isomorphism.

Proof. Uses: `lemma:weil-pbc-zl-reduction-perfect`

The derived associator is an isomorphism. The second factor is the standard derived unitor $P_b \otimes^L_{P_b} P_a \simeq P_a$; composing the two isomorphisms gives the claim. □

Lemma 11.10.44 (Cocycle equation for reduction normalization). Lean: `conditional-interface:lemma:weil-efi-coeff-zl-reduction-normalization-cocycle`

Uses: `def:weil-efi-coeff-zl-reduction-normalization`

For $c \geq b \geq a$, the direct normalization from level c to level a equals the composite through level b.

Proof. Uses: `lemma:weil-efi-coeff-zl-reduction-composition`

Expand the two maps into derived-tensor associators and quotient maps. The associator pentagon makes the reassociations equal, and the cited quotient composition equation identifies the remaining coefficient map. □

Definition 11.10.45 (Intrinsic constructible integral category candidate). Lean: `conditional-interface:def:weil-efi-coeff-intrinsic-constructible-zl-category`

Uses: `def:weil-a2-derived-category`, `def:weil-pbc-intrinsic-koszul-completion`, `def:weil-pbc-bounded-constructible-adic-tower`, `def:weil-efi-coeff-zl-reduction-tower`

Define $\mathscr{A}^{\mathrm{intr}}_{\mathbf{Z}_\ell}(X)$ to be the full subcategory of $\mathcal{D}(X, \mathbf{Z}_\ell)$ on those K for which the intrinsic Koszul completion unit is an isomorphism and the canonical finite reduction tower is uniformly bounded constructible. This definition has no K-flat or K-injective package parameter.

Lemma 11.10.46 (Adic completion lands in the intrinsic candidate). Lean: `conditional-interface:lemma:weil-efi-coeff-completion-in-intrinsic-category`

Uses: `def:weil-efi-coeff-intrinsic-constructible-zl-category`

If the canonical reduction tower of K is uniformly bounded constructible, then its explicit adic completion is an object of $\mathscr{A}^{\mathrm{intr}}_{\mathbf{Z}_\ell}(X)$.

Proof. Uses: `lemma:weil-pbc-bounded-constructible-reconstruction`, `lemma:weil-pbc-adic-completion-reduction`

The reconstruction lemma proves intrinsic completeness and preserves the common constructible bounds. The finite-reduction lemma identifies the reductions of the completion with the original tower, so the same constructibility witness applies. □

11.11 S10. Complete pro-etale proper base change

Definition 11.11.1 (Four finite-level rectangle corners). Lean: `conditional-interface:def:weil-pbc-level-map-corners`

Uses: `def:weil-pbc-cartesian-square`, `def:weil-pbc-complete-coefficient-datum`

For the proper Cartesian square □, put $E = L(g')^* K$ and, for every $n \geq 1$, define

$$A_n := Lg^*((Rf_*K) \otimes^L P_n), \quad B_n := Lg^* Rf_* K_n,$$
$$C_n := (Rf'_* E) \otimes^L P_n, \quad D_n := Rf'_* E_n,$$

where $E_n = L(g')^* K_n = E \otimes^L P_n$. The chosen Cartesian identification supplies $L(g')^* Lf^* = L(f')^* Lg^*$ throughout.

Definition 11.11.2 (The four finite-level arrows). Lean: `conditional-interface:def:weil-pbc-level-map-arrows`

Uses: `def:weil-pbc-level-map-corners`

Let $\xi_K : Lg^* Rf_* K \to Rf'_* E$ be the unit–counit exchange map

$$Lg^* Rf_* K \longrightarrow Rf'_* L(f')^* Lg^* Rf_* K \simeq Rf'_* L(g')^* Lf^* Rf_* K \xrightarrow{Rf'_* L(g')^* \epsilon_K} Rf'_* E.$$

Define the four edges

$$h_n := Lg^*(\pi_{f,K,n}) : A_n \longrightarrow B_n,$$
$$v_n := \xi_K \otimes^L 1_{P_n} : A_n \longrightarrow C_n,$$
$$h'_n := \pi_{f',E,n} : C_n \longrightarrow D_n,$$
$$v'_n := \xi_{K_n} : B_n \longrightarrow D_n.$$

For v_n use $Lg^*((Rf_*K) \otimes P_n) \simeq (Lg^*Rf_*K) \otimes P_n$. The transported S8 map is v'_n, because its construction was identified there with this unit–counit exchange map.

Lemma 11.11.3 (Well-definedness certificate for def:weil-pbc-level-map-arrows)**.** *Lean:* `conditional-interface:lemma:weil-sf-well-defined-def-weil-pbc-level-map-arrows`

Uses: `def:weil-pbc-level-map-arrows`

For $n \geq 1$, the tuple (h_n, v_n, h'_n, v'_n) belongs to

$$\mathrm{Hom}(A_n, B_n) \times \mathrm{Hom}(A_n, C_n) \times \mathrm{Hom}(C_n, D_n) \times \mathrm{Hom}(B_n, D_n),$$

and h_n, h'_n are isomorphisms. Under finite-level pro-etale transport, the fourth component is the unit–counit exchange map ξ_{K_n}.

Proof. Uses: `lemma:weil-pbc-perfect-projection-formula,prop:weil-pbc-proet-finite-level-transport`

Perfectness of P_n makes $\pi_{f,K,n} : (Rf_*K) \otimes^L P_n \to Rf_*K_n$ and $\pi_{f',E,n} : (Rf'_*E) \otimes^L P_n \to Rf'_*E_n$ isomorphisms. Derived pullback preserves the first isomorphism, giving the stated type and invertibility of h_n; the second gives the same facts for h'_n. Symmetric monoidality of derived pullback gives

$$Lg^*((Rf_*K) \otimes^L P_n) \simeq (Lg^*Rf_*K) \otimes^L P_n,$$

so tensoring ξ_K with 1_{P_n} has type $A_n \to C_n$. Proposition 11.9.67 transports the finite etale-origin exchange to a map $B_n \to D_n$ and identifies that map with the unit–counit formula ξ_{K_n}. These are exactly the four components asserted above; no rectangle or tower equation is used here. □

Lemma 11.11.4 (The finite-level exchange rectangle)**.** *Lean:* `conditional-interface:lemma:weil-pbc-level-map-rectangle`

Uses: `def:weil-pbc-level-map-arrows`

For every n, the diagram

$$\begin{array}{ccc} A_n & \xrightarrow{h_n} & B_n \\ {\scriptstyle v_n}\downarrow & & \downarrow{\scriptstyle v'_n} \\ C_n & \xrightarrow{h'_n} & D_n \end{array}$$

commutes; equivalently, its two composites satisfy

$$v'_n h_n = h'_n v_n : A_n \longrightarrow D_n.$$

Proof. Uses: `lemma:weil-pbc-projection-formula-base-change-naturality`

This is the base-change naturality rectangle for the projection-formula map with coefficient P_n. To expose the equality, adjoint both composites to X'. Their common adjoint is

$$L(g')^*Lf^*Rf_*K \otimes^L P_n \xrightarrow{L(g')^*(\epsilon_K)\otimes 1} L(g')^*K \otimes^L P_n = E_n.$$

Thus every unit, counit, tensor associator, and Cartesian comparison occurs in the same order in the two adjoints. Equality after adjunction proves the displayed component equality. □

Lemma 11.11.5 (Transition compatibility of all four edges)**.** *Lean:* `conditional-interface:lemma:weil-pbc-level-map-tower-compatibility`

Uses: `def:weil-pbc-level-map-arrows`

The four corner transitions from level $n+1$ to level n are

$$a_{n+1,n} = Lg^*(1 \otimes q_n) : A_{n+1} \to A_n,$$
$$b_{n+1,n} = Lg^*Rf_*(K_{n+1} \to K_n) : B_{n+1} \to B_n,$$
$$c_{n+1,n} = 1 \otimes q_n : C_{n+1} \to C_n,$$
$$d_{n+1,n} = Rf'_*L(g')^*(K_{n+1} \to K_n) : D_{n+1} \to D_n.$$

Every edge is a map of towers:

$$b_{n+1,n}h_{n+1} = h_n a_{n+1,n}, \quad c_{n+1,n}v_{n+1} = v_n a_{n+1,n},$$
$$d_{n+1,n}h'_{n+1} = h'_n c_{n+1,n}, \quad d_{n+1,n}v'_{n+1} = v'_n b_{n+1,n}.$$

Proof. Uses: `lemma:weil-pbc-projection-formula-tower-compatibility`

The first and third equations are the coefficient-tower equations for the two projection formulas. The second follows by tensor naturality of ξ_K with respect to q_n. The last is naturality of the unit–counit exchange map with respect to $K_{n+1} \to K_n$. Expanded after adjunction, both sides of the last equation are the counit of K followed by the same coefficient quotient. Hence all four equations hold componentwise. □

Lemma 11.11.6 (Operator naturality of the finite rectangle). Lean: `conditional-interface:lemma:weil-pbc-level-map-operator-naturality`

Uses: `def:weil-pbc-level-map-arrows`

Let $\phi : K \to K$ be a tower-compatible endomorphism. Give A_n, B_n, C_n, D_n the endomorphisms induced by $Rf_*\phi$, $Rf_*\phi_n$, $Rf'_*L(g')^*\phi$, and $Rf'_*L(g')^*\phi_n$, respectively. Then h_n, v_n, h'_n, v'_n intertwine these endomorphisms, and the equality $v'_n h_n = h'_n v_n$ is an equality of operator-equivariant maps.

Proof. Uses: `lemma:weil-pbc-projection-formula-base-change-naturality`

The horizontal statements are operator naturality of the projection formula. The vertical statements follow by naturality of the unit and counit defining exchange. Composing either pair of equivariant edges gives an equivariant map; the preceding rectangle identifies those two maps. □

Definition 11.11.7 (Frobenius-equivariant square and Weil tower). Lean: `conditional-interface:def:weil-pbc-frobenius-tower-datum`

Uses: `def:weil-pbc-level-map-corners`, `def:weil-pbc-level-map-arrows`

A Frobenius-equivariant datum for the proper Cartesian square □ and the tower $K_\bullet$ includes displayed witnesses

$$\epsilon_X^{-1} L_n \xrightarrow{\sim} K_n, \qquad L_n \in D_c^+(X_{\mathrm{et}}, P_n),$$

compatible with the tower transitions. Pullback supplies the corresponding etale-origin witnesses for $E_n = L(g')^* K_n$. The datum then consists of automorphisms

$$\mathrm{Fr}_X, \quad \mathrm{Fr}_Y, \quad \mathrm{Fr}_{X'}, \quad \mathrm{Fr}_{Y'}$$

such that

$$f \,\mathrm{Fr}_X = \mathrm{Fr}_Y \, f, \qquad g \,\mathrm{Fr}_{Y'} = \mathrm{Fr}_Y \, g,$$
$$g' \,\mathrm{Fr}_{X'} = \mathrm{Fr}_X \, g', \quad f' \,\mathrm{Fr}_{X'} = \mathrm{Fr}_{Y'} \, f'.$$

The four resulting Frobenius faces are equipped with their Cartesian identifications, and these identifications make the resulting cube commute with the chosen Cartesian identification for □. Put $\Phi_Z = L \,\mathrm{Fr}_Z^*$. Since every Fr_Z is an automorphism, every Φ_Z is an exact equivalence and preserves products and fibres.

The pullback faces give canonical isomorphisms

$$\kappa_g : \Phi_{Y'} L g^* \xrightarrow{\sim} L g^* \Phi_Y, \qquad \kappa_{g'} : \Phi_{X'} L(g')^* \xrightarrow{\sim} L(g')^* \Phi_X.$$

The proper Frobenius faces give finite-level exchange isomorphisms

$$\beta_{f,n} : \Phi_Y R f_* K_n \xrightarrow{\sim} R f_* \Phi_X K_n, \qquad \beta_{f',M} : \Phi_{Y'} R f'_* M \xrightarrow{\sim} R f'_* \Phi_{X'} M.$$

Finally the datum contains isomorphisms

$$\vartheta_n : \Phi_X K_n \xrightarrow{\sim} K_n$$

such that, for $r_{n+1,n} : K_{n+1} \to K_n$,

$$r_{n+1,n} \vartheta_{n+1} = \vartheta_n \Phi_X(r_{n+1,n}).$$

The constant coefficient quotients carry their identity Frobenius structures, and all displayed comparisons are required to respect their normalizations and cocycles.

Lemma 11.11.8 (Well-definedness certificate for def:weil-pbc-frobenius-tower-datum). Lean: `conditional-interface:lemma:weil-sf-well-defined-def-weil-pbc-frobenius-tower-datum`

Uses: `def:weil-pbc-frobenius-tower-datum`

The displayed Frobenius record has fields

$$\kappa_g : \Phi_{Y'} L g^* \xrightarrow{\sim} L g^* \Phi_Y, \qquad \kappa_{g'} : \Phi_{X'} L(g')^* \xrightarrow{\sim} L(g')^* \Phi_X,$$
$$\beta_{f,n} : \Phi_Y R f_* K_n \xrightarrow{\sim} R f_* \Phi_X K_n, \qquad \beta_{f',E_n} : \Phi_{Y'} R f'_* E_n \xrightarrow{\sim} R f'_* \Phi_{X'} E_n,$$
$$\vartheta_n : \Phi_X K_n \xrightarrow{\sim} K_n.$$

The two sides of the tower relation belong to $\mathrm{Hom}(\Phi_X K_{n+1}, K_n)$*, so the relation is a typed field of the record.*

Proof. Uses: `lemma:weil-pbc-derived-exchange-coherence,prop:weil-pbc-proet-finite-level-transport`

Pullback by the inverse of Fr_Z is a quasi-inverse to $\Phi_Z = L\,\mathrm{Fr}_Z^*$; hence Φ_Z is an exact equivalence. The two pullback faces in the Frobenius cube give κ_g and $\kappa_{g'}$ with the displayed endpoints. Applied to the two proper faces, finite-level pro-etale transport gives the displayed exchange isomorphisms because K_n and E_n carry their etale-origin witnesses. The identity and pasting equations required of these exchanges are those of Lemma 11.1.23.

Finally, $\Phi_X(r_{n+1,n})$ has source $\Phi_X K_{n+1}$ and target $\Phi_X K_n$. Thus $r_{n+1,n}\vartheta_{n+1}$ and $\vartheta_n \Phi_X(r_{n+1,n})$ have the common source and target stated in the lemma. This verifies the record typing without asserting a new Frobenius-equivariance theorem for the finite rectangle. □

Definition 11.11.9 (Weil structures on the four finite corners)**.** Lean: `conditional-interface:def:weil-pbc-frobenius-corner-structures`

Uses: `def:weil-pbc-frobenius-tower-datum`

For $E_n = L(g')^* K_n$, define

$$\vartheta_{E,n} := L(g')^*(\vartheta_n)\,\kappa_{g',K_n} : \Phi_{X'} E_n \xrightarrow{\sim} E_n.$$

The Weil structures on B_n and D_n are the typed composites

$$\Theta_{B,n} : \Phi_{Y'} B_n \xrightarrow{\kappa_g} Lg^*\Phi_Y Rf_* K_n \xrightarrow{Lg^*\beta_{f,n}} Lg^* Rf_* \Phi_X K_n \xrightarrow{Lg^* Rf_* \vartheta_n} B_n,$$
$$\Theta_{D,n} : \Phi_{Y'} D_n \xrightarrow{\beta_{f',E_n}} Rf'_*\Phi_{X'} E_n \xrightarrow{Rf'_*\vartheta_{E,n}} D_n.$$

Transport these structures across the two projection-formula isomorphisms:

$$\Theta_{A,n} := h_n^{-1}\Theta_{B,n}\Phi_{Y'}(h_n), \qquad \Theta_{C,n} := (h'_n)^{-1}\Theta_{D,n}\Phi_{Y'}(h'_n).$$

Thus every $\Theta_{Z,n}$ has the genuine Weil type $\Phi_{Y'} Z_n \xrightarrow{\sim} Z_n$ for $Z = A, B, C, D$.

Lemma 11.11.10 (Well-definedness certificate for def:weil-pbc-frobenius-corner-structures)**.** Lean: `conditional-interface:lemma:weil-sf-well-defined-def-weil-pbc-frobenius-corner-structures`

Uses: `def:weil-pbc-frobenius-corner-structures`

The tuple $(\Theta_{A,n}, \Theta_{B,n}, \Theta_{C,n}, \Theta_{D,n})$ belongs to

$$\mathrm{Iso}(\Phi_{Y'} A_n, A_n) \times \mathrm{Iso}(\Phi_{Y'} B_n, B_n) \times \mathrm{Iso}(\Phi_{Y'} C_n, C_n) \times \mathrm{Iso}(\Phi_{Y'} D_n, D_n).$$

This assertion records the four corner types and does not assert edge equivariance.

Proof. Uses: `lemma:weil-sf-well-defined-def-weil-pbc-level-map-arrows,lemma:weil-sf-well-defined-def-weil-pbc-frobenius-tower-datum,lemma:weil-pbc-perfect-projection-formula`

The certified tower datum makes $\kappa_g, \kappa_{g'}, \beta_{f,n}, \beta_{f',E_n}$ and ϑ_n isomorphisms with the displayed endpoints. Their ordered composites therefore give $\vartheta_{E,n}$, $\Theta_{B,n}$ and $\Theta_{D,n}$ with the asserted types. The level-arrow certificate and the perfect projection formula make h_n and h'_n invertible. Conjugating $\Theta_{B,n}$ by h_n and $\Theta_{D,n}$ by h'_n gives the asserted types for $\Theta_{A,n}$ and $\Theta_{C,n}$. Edge equivariance remains the next lemma. □

Lemma 11.11.11 (The finite rectangle is a Weil rectangle)**.** Lean: `conditional-interface:lemma:weil-pbc-frobenius-level-rectangle`

Uses: `def:weil-pbc-frobenius-corner-structures`

All four finite-level edges are Weil-equivariant:

$$h_n\Theta_{A,n} = \Theta_{B,n}\Phi_{Y'}(h_n), \quad v_n\Theta_{A,n} = \Theta_{C,n}\Phi_{Y'}(v_n),$$
$$h'_n\Theta_{C,n} = \Theta_{D,n}\Phi_{Y'}(h'_n), \quad v'_n\Theta_{B,n} = \Theta_{D,n}\Phi_{Y'}(v'_n).$$

In particular the two explicitly typed composites

$$\mathcal{P}_n := \Phi_{Y'} A_n \xrightarrow{\Theta_{A,n}} A_n \xrightarrow{v'_n h_n} D_n,$$
$$\mathcal{Q}_n := \Phi_{Y'} A_n \xrightarrow{\Phi_{Y'}(v'_n h_n)} \Phi_{Y'} D_n \xrightarrow{\Theta_{D,n}} D_n$$

are equal. The same common map is obtained after replacing $v'_n h_n$ by $h'_n v_n$.

Proof. Uses: `lemma:weil-pbc-level-map-rectangle,lemma:weil-pbc-derived-exchange-coherence`

The equations for h_n and h'_n are the two transport definitions. For v'_n, expand the upper-right and lower-left routes in the Frobenius cube. Pasting coherence for exchange and naturality in ϑ_n identify both with

$$\Phi_{Y'}Lg^*Rf_*K_n \xrightarrow{\kappa_g} Lg^*\Phi_Y Rf_*K_n \xrightarrow{Lg^*\beta_{f,n}} Lg^*Rf_*\Phi_X K_n \xrightarrow{\xi_{\Phi_X K_n}} Rf'_*L(g')^*\Phi_X K_n \xrightarrow{Rf'_*L(g')^*\vartheta_n} D_n.$$

This proves the fourth equation with every pullback and exchange map typed. For the remaining vertical edge, use the ordinary rectangle and the three equations already proved:

$$\begin{aligned} h'_n v_n \Theta_{A,n} &= v'_n h_n \Theta_{A,n} = v'_n \Theta_{B,n}\Phi_{Y'}(h_n) \\ &= \Theta_{D,n}\Phi_{Y'}(v'_n h_n) = h'_n \Theta_{C,n}\Phi_{Y'}(v_n). \end{aligned}$$

Cancel the isomorphism h'_n. Composing either pair of edge equations and using $v'_n h_n = h'_n v_n$ gives $\mathcal{P}_n = \mathcal{Q}_n$ and the final assertion. □

Lemma 11.11.12 (Frobenius compatibility with tower transitions). Lean: `conditional-interface:lemma:weil-pbc-frobenius-tower-compatibility`

Uses: `def:weil-pbc-frobenius-corner-structures`

For the four transitions $a_{n+1,n}, b_{n+1,n}, c_{n+1,n}, d_{n+1,n}$,

$$\begin{aligned} a_{n+1,n}\Theta_{A,n+1} &= \Theta_{A,n}\Phi_{Y'}(a_{n+1,n}), \\ b_{n+1,n}\Theta_{B,n+1} &= \Theta_{B,n}\Phi_{Y'}(b_{n+1,n}), \\ c_{n+1,n}\Theta_{C,n+1} &= \Theta_{C,n}\Phi_{Y'}(c_{n+1,n}), \\ d_{n+1,n}\Theta_{D,n+1} &= \Theta_{D,n}\Phi_{Y'}(d_{n+1,n}). \end{aligned}$$

Hence the four corner Weil structures and all four edges are maps of towers.

Proof. Uses: `lemma:weil-pbc-frobenius-level-rectangle,lemma:weil-pbc-level-map-tower-compatibility`

The equation for B follows by naturality of κ_g and β_f and the defining equation $r_{n+1,n}\vartheta_{n+1} = \vartheta_n\Phi_X(r_{n+1,n})$. The same argument for $\kappa_{g'}$, $\beta_{f'}$, and $\vartheta_{E,n}$ proves the equation for D. For A, combine $b_{n+1,n}h_{n+1} = h_n a_{n+1,n}$ with the transport formula for Θ_A and the equation for B; cancelling h_n gives the first displayed equation. The equation for C follows identically from $d_{n+1,n}h'_{n+1} = h'_n c_{n+1,n}$ and the equation for D. The finite Weil rectangle and the four transition equations now say precisely that every edge is a morphism of Weil towers. □

Lemma 11.11.13 (The rectangle passes to the derived inverse limit). Lean: `conditional-interface:lemma:weil-pbc-level-map-derived-limit`

Uses: `def:weil-pbc-adic-derived-inverse-limit`

Write $A_\infty = R\varprojlim A_n$ and likewise for the other three towers, and write H, V, H', V' for the limits of h, v, h', v'. Then

$$V'H = H'V : A_\infty \longrightarrow D_\infty.$$

For every n, both maps have the same nth component

$$\mathrm{pr}_{D,n}\, V'H = v'_n h_n\, \mathrm{pr}_{A,n} = h'_n v_n\, \mathrm{pr}_{A,n} = \mathrm{pr}_{D,n}\, H'V.$$

The equality is natural for compatible tower endomorphisms.

Proof. Uses: `lemma:weil-pbc-level-map-rectangle,lemma:weil-pbc-level-map-tower-compatibility,lemma:weil-pbc-level-map-operator-naturality`

Transition compatibility makes each family a map between the product-cone diagrams. On both product terms, the nth coordinate of $V'H$ is $v'_n h_n$, while that of $H'V$ is $h'_n v_n$; these are equal by the finite-level rectangle. Hence the product maps, their $1-$shift squares, and the induced maps on the fibres are equal. This proves both the limit equality and the three displayed component equations. Operator naturality holds on every product coordinate, so it also holds on the fibre. □

Lemma 11.11.14 (Frobenius structures pass through the product cone). Lean: `conditional-interface:lemma:weil-pbc-frobenius-product-cone-limit`

Uses: `def:weil-pbc-adic-derived-inverse-limit`

For $Z = A, B, C, D$, the product-cone limit carries the Weil structure

$$\Theta_{Z,\infty} : \Phi_{Y'}Z_\infty \xrightarrow{\sim} R\varprojlim_n \Phi_{Y'}Z_n \xrightarrow{R\varprojlim_n \Theta_{Z,n}} Z_\infty.$$

The first arrow is the canonical comparison, which is an isomorphism because $\Phi_{Y'}$ is an equivalence and therefore preserves the two products and the fibre in the product-cone model. Each of H, V, H', V' is Weil-equivariant. For the common limit map $W := V'H = H'V : A_\infty \to D_\infty$, the two typed composites

$$\Phi_{Y'}A_\infty \xrightarrow{\Theta_{A,\infty}} A_\infty \xrightarrow{W} D_\infty,$$
$$\Phi_{Y'}A_\infty \xrightarrow{\Phi_{Y'}(W)} \Phi_{Y'}D_\infty \xrightarrow{\Theta_{D,\infty}} D_\infty$$

are equal.

Proof. Uses: `lemma:weil-pbc-frobenius-level-rectangle,lemma:weil-pbc-frobenius-tower-compatibility,lemma:weil-pbc-level-map-derived-limit`

Apply $\Phi_{Y'}$ to the product-cone triangle for each tower. Preservation of products identifies the two product terms with the products of $\Phi_{Y'}Z_n$, and transition compatibility identifies the two $1-$shift arrows. Thus the displayed comparison is an isomorphism of fibres. On its nth product coordinate, equivariance of H, V, H', V' is exactly one of the four finite edge equations. Hence the product maps and their maps on fibres are equivariant. Composing the equations for H, V' or for V, H' gives

$$W\Theta_{A,\infty} = \Theta_{D,\infty}\Phi_{Y'}(W).$$

The equality $V'H = H'V$ is already an equality of maps of product-cone diagrams, so both factorizations give the same typed Weil morphism. □

Definition 11.11.15 (Completed base-change map)**.** Lean: `conditional-interface:def:weil-pbc-completed-base-change-map`

Uses: `def:weil-pbc-completed-pullback`

Let K be derived complete and let every K_n have a displayed witness $\epsilon_X^{-1}L_n \simeq K_n$ with $L_n \in D_c^+(X_{\mathrm{et}}, \Lambda/I^n)$. Define

$$\mathrm{Ex}^{\mathrm{comp}}_{\square,K} : Lg^*_{\mathrm{comp}}Rf_*K \longrightarrow Rf'_*L(g')^*_{\mathrm{comp}}K$$

to be the ordered composite

$$\begin{aligned} Lg^*_{\mathrm{comp}}Rf_*K &\xrightarrow{\sim} R\varprojlim_n Lg^*Rf_*K_n \\ &\xrightarrow{R\varprojlim_n \mathrm{Ex}_{\square,K_n}} R\varprojlim_n Rf'_*L(g')^*K_n \\ &\xrightarrow{\sim} Rf'_*R\varprojlim_n L(g')^*K_n = Rf'_*L(g')^*_{\mathrm{comp}}K. \end{aligned}$$

In the notation above, the first two arrows are H followed by V'. The first isomorphism uses reconstruction and the perfect projection formula; the second displayed isomorphism is the inverse of direct-image/limit commutation. This definition fills the map-construction omission in Tag `09C9`.

Lemma 11.11.16 (Well-definedness certificate for def:weil-pbc-completed-base-change-map)**.** Lean: `conditional-interface:lemma:weil-sf-well-defined-def-weil-pbc-completed-base-change-map`

Uses: `def:weil-pbc-completed-base-change-map`

With $S = Lg^*_{\mathrm{comp}}Rf_*K$ and $T = Rf'_*L(g')^*_{\mathrm{comp}}K$, the ordered expression

$$S \xrightarrow{s} A_\infty \xrightarrow{H} B_\infty \xrightarrow{V'} D_\infty \xrightarrow{t} T$$

is a morphism $S \to T$ and is the map $\mathrm{Ex}^{\mathrm{comp}}_{\square,K}$ of Definition 11.11.15. The endpoint comparisons s, t and the limit projection-formula map H are isomorphisms.

Proof. Uses: `lemma:weil-pbc-perfect-projection-formula,lemma:weil-pbc-level-map-derived-limit,lemma:weil-pbc-direct-image-r-lim`

Symmetric monoidality of Lg^* identifies the defining completed-pullback tower with the A_n tower. The perfect projection formula then gives the map $H : A_\infty \to B_\infty$, while finite exchange gives the compatible tower map $V' : B_\infty \to D_\infty$ recorded by Lemma 11.11.13. Direct image commutation with the product-cone limit gives $t : D_\infty \xrightarrow{\sim} T$. The target of each factor is the source of the next one.

At level n, h_n is invertible by the perfect projection formula. From $b_{n+1,n}h_{n+1} = h_n a_{n+1,n}$, multiplication by the two inverses gives $a_{n+1,n}h_{n+1}^{-1} = h_n^{-1}b_{n+1,n}$. Hence the inverse family is a map of towers, and its derived limit is inverse to H. Thus s, H, t are isomorphisms. Expanding the ordered definition gives the displayed composite. No invertibility or Frobenius assertion for the whole composite is made in this certificate. □

Definition 11.11.17 (Weil structures on the completed source and target)**.** Lean: `conditional-interface:def:weil-pbc-completed-frobenius-structures`

Uses: `def:weil-pbc-completed-base-change-map`

Write

$$S = Lg^*_{\mathrm{comp}} Rf_* K, \qquad T = Rf'_* L(g')^*_{\mathrm{comp}} K.$$

Let $s : S \xrightarrow{\sim} A_\infty$ be the pullback/tensor comparison in the first part of the completed construction, and let $t : D_\infty \xrightarrow{\sim} T$ be the inverse of direct-image/limit commutation. Define genuine completed Weil structures by

$$\Theta_S := s^{-1}\Theta_{A,\infty}\Phi_{Y'}(s) : \Phi_{Y'}S \xrightarrow{\sim} S,$$
$$\Theta_T := t\Theta_{D,\infty}\Phi_{Y'}(t^{-1}) : \Phi_{Y'}T \xrightarrow{\sim} T.$$

Under these identifications the completed base-change map is the ordered composite

$$\mathrm{Ex}^{\mathrm{comp}}_{\square,K} = tWs, \qquad W = V'H = H'V.$$

Lemma 11.11.18 (Well-definedness certificate for def:weil-pbc-completed-frobenius-structures)**.** Lean: `conditional-interface:lemma:weil-sf-well-defined-def-weil-pbc-completed-frobenius-structures`

Uses: `def:weil-pbc-completed-frobenius-structures`

The pair (Θ_S, Θ_T) *belongs to*

$$\mathrm{Iso}(\Phi_{Y'}S, S) \times \mathrm{Iso}(\Phi_{Y'}T, T),$$

and tWs *belongs to* $\mathrm{Hom}(S, T)$*. This certificate records the completed types but not the equivariance equation for* tWs*.*

Proof. Uses: `lemma:weil-sf-well-defined-def-weil-pbc-completed-base-change-map,lemma:weil-pbc-frobenius-product-cone-limit`

The product-cone lemma supplies isomorphisms $\Theta_{A,\infty} : \Phi_{Y'}A_\infty \to A_\infty$ and $\Theta_{D,\infty} : \Phi_{Y'}D_\infty \to D_\infty$, together with $W : A_\infty \to D_\infty$. The preceding certificate gives the isomorphisms $s : S \to A_\infty$ and $t : D_\infty \to T$. Conjugating the two limit structures by s and t yields the displayed pair of isomorphism types, and the endpoints of s, W, t give $tWs : S \to T$. The next lemma proves the equivariance equation. □

Lemma 11.11.19 (Completed proper base change is Frobenius-equivariant)**.** Lean: `conditional-interface:lemma:weil-pbc-completed-frobenius-equivariance`

Uses: `def:weil-pbc-completed-frobenius-structures`

For every Frobenius-equivariant square and Weil tower datum,

$$\mathrm{Ex}^{\mathrm{comp}}_{\square,K}\,\Theta_S = \Theta_T\Phi_{Y'}(\mathrm{Ex}^{\mathrm{comp}}_{\square,K}) : \Phi_{Y'}S \longrightarrow T.$$

If all four Frobenius automorphisms and all pullback/exchange comparisons are identities, then ϑ_n *is an ordinary compatible endomorphism and this equation specializes exactly to the operator naturality of the finite rectangle and its product-cone limit. For nonidentity Frobenius, no identification of* $\Phi_X K_n \to K_n$ *with an endomorphism* $K_n \to K_n$ *is made.*

Proof. Uses: `lemma:weil-pbc-frobenius-product-cone-limit,lemma:weil-pbc-level-map-operator-naturality`

Using the displayed definitions and cancelling $s^{-1}s$ and $t^{-1}t$, the left side is

$$tW\Theta_{A,\infty}\Phi_{Y'}(s),$$

whereas the right side is

$$t\Theta_{D,\infty}\Phi_{Y'}(W)\Phi_{Y'}(s).$$

They are equal by the product-cone Frobenius equation. When every Φ_Z is the identity, the four formulas defining $\Theta_{Z,n}$ become the endomorphisms induced from ϑ_n and the equation just proved is the fibrewise limit of the equations in operator naturality. This proves the specialization statement without using it to type the genuine Frobenius case. □

Definition 11.11.20 (The four edges of the completed reduction square)**.** Lean: `conditional-interface:def:weil-pbc-completed-reduction-square`

Uses: `def:weil-pbc-completed-base-change-map`

Put

$$S = Lg^*_{\mathrm{comp}} Rf_* K, \qquad T = Rf'_* L(g')^*_{\mathrm{comp}} K.$$

Let

$$\sigma_n : S \otimes^L P_n \xrightarrow{\sim} A_n$$

be finite reduction of completed pullback. For $E = L(g')^*K$, let

$$\tau_n : T \otimes^L P_n \xrightarrow{\sim} C_n$$

be the inverse of $\widehat{Rf'_*E} \xrightarrow{\sim} Rf'_*\widehat{E} = T$, tensored with P_n, followed by finite reduction of $\widehat{Rf'_*E}$. Define

$$\alpha_n = h_n\sigma_n : S \otimes^L P_n \to B_n, \qquad \delta_n = h'_n\tau_n : T \otimes^L P_n \to D_n,$$

and let

$$\overline{\xi}_n = \mathrm{Ex}^{\mathrm{comp}}_{\square,K} \otimes^L 1_{P_n} : S \otimes^L P_n \longrightarrow T \otimes^L P_n.$$

Thus every arrow in the completed reduction square is named, typed, and expressed as an ordered composite of reduction, direct-image, projection- formula, or base-change maps.

Lemma 11.11.21 (Well-definedness certificate for def:weil-pbc-completed-reduction-square)**.** *Lean:* `conditional-interface:lemma:weil-sf-well-defined-def-weil-pbc-completed-reduction-square`

Uses: `def:weil-pbc-completed-reduction-square`

For $n \geq 1$, $\sigma_n : S \otimes^L P_n \to A_n$ and $\tau_n : T \otimes^L P_n \to C_n$ are isomorphisms, while

$$(\alpha_n, \delta_n, \overline{\xi}_n) \in \mathrm{Hom}(S \otimes^L P_n, B_n) \times \mathrm{Hom}(T \otimes^L P_n, D_n) \times \mathrm{Hom}(S \otimes^L P_n, T \otimes^L P_n).$$

The first two components of this triple are isomorphisms. No commutativity claim for the reduction square is part of this certificate.

Proof. Uses: `lemma:weil-sf-well-defined-def-weil-pbc-level-map-arrows,lemma:weil-sf-well-defined-def-weil-pbc-completed-base-change-map,lemma:weil-pbc-completed-pullback-reduction,lemma:weil-pbc-direct-image-derived-completion,lemma:weil-pbc-completed-tensor-reduction`

Finite reduction of completed pullback gives σ_n with the stated source and target. Direct-image completion gives $\widehat{Rf'_*E} \xrightarrow{\sim} Rf'_*(\widehat{E}) = T$; tensoring its inverse with P_n and applying finite reduction of completed tensor gives $\tau_n : T \otimes^L P_n \xrightarrow{\sim} C_n$.

The level-arrow certificate makes $h_n : A_n \to B_n$ and $h'_n : C_n \to D_n$ isomorphisms. Hence $\alpha_n = h_n\sigma_n$ and $\delta_n = h'_n\tau_n$ have the asserted types and are invertible. Finally the completed-map certificate gives $\mathrm{Ex}^{\mathrm{comp}}_{\square,K} : S \to T$, so derived tensor with 1_{P_n} gives the asserted type of $\overline{\xi}_n$. The next lemma contains the square's commutativity and its compatibility fields. □

Lemma 11.11.22 (Finite reduction of the completed map)**.** *Lean:* `conditional-interface:lemma:weil-pbc-completed-map-reduction`

Uses: `def:weil-pbc-completed-reduction-square,def:weil-pbc-completed-frobenius-structures`

For every n, the diagram

$$\begin{array}{ccc} S \otimes^L P_n & \xrightarrow{\overline{\xi}_n} & T \otimes^L P_n \\ {\scriptstyle\alpha_n}\downarrow{\scriptstyle\sim} & & \downarrow{\scriptstyle\delta_n}\ {\scriptstyle\sim} \\ B_n & \xrightarrow{v'_n} & D_n \end{array}$$

commutes. Hence reduction of $\mathrm{Ex}^{\mathrm{comp}}_{\square,K}$ modulo I^n, conjugated by the displayed canonical reduction isomorphisms, is exactly the S8 transported finite-level exchange map $v'_n = \mathrm{Ex}_{\square,K_n}$. The squares for $n+1 \to n$ commute, and the assertion is natural for compatible tower endomorphisms. Under a Frobenius-equivariant datum, use $\Phi_{Y'}P_n = P_n$ to give the two reductions their induced Weil structures. Then

$$\alpha_n(\Theta_S \otimes 1) = \Theta_{B,n}\Phi_{Y'}(\alpha_n),$$
$$\delta_n(\Theta_T \otimes 1) = \Theta_{D,n}\Phi_{Y'}(\delta_n),$$

and the displayed reduction square is a square of Weil objects.

Proof. Uses: `lemma:weil-pbc-level-map-derived-limit,lemma:weil-pbc-frobenius-tower-compatibility,lemma:weil-pbc-completed-frobenius-equivariance`

Under $S = A_\infty$ and the direct-image completion identification $T \simeq C_\infty$, the completed exchange map is both limit composites

$$V'H = H'V : A_\infty \longrightarrow D_\infty,$$

followed by the inverse of direct-image/limit commutation. More explicitly, if $d : T \to D_\infty$ is direct-image/limit commutation and $c : C_\infty \to T$ is direct-image completion, then its construction gives $dc = H'$. Functorial finite reduction and the component equations of the limit rectangle give

$$\tau_n\overline{\xi}_n = v_n\sigma_n.$$

Consequently the two composites around the completed reduction square are

$$\delta_n \overline{\xi}_n = h'_n \tau_n \overline{\xi}_n = h'_n v_n \sigma_n$$

and

$$v'_n \alpha_n = v'_n h_n \sigma_n.$$

They are equal term by term because $h'_n v_n = v'_n h_n$ in the finite-level rectangle. Naturality of finite reduction makes σ_n and τ_n commute with $n+1 \to n$, and functoriality of tensor reduction does the same for $\overline{\xi}_n$. Together with the four transition equations, this shows that the completed squares commute with $K_{n+1} \to K_n$ on every corner. The limit construction and all four reduction edges are operator-equivariant, so the conjugated equality is operator-natural as well.

For Frobenius, the definition of Θ_S transports $\Theta_{A,\infty}$ across s, and the nth product-cone component followed by h_n is α_n. This gives the first displayed Weil equation. The definition of Θ_T transports $\Theta_{D,\infty}$ across t, and its nth component is δ_n, giving the second. The completed Frobenius equation for $\overline{\xi}_n$, together with finite-level equivariance of v'_n, now says that every arrow of the reduction square is a Weil morphism. □

Lemma 11.11.23 (Completed exchange coherence). *Lean:* `conditional-interface:lemma:weil-pbc-completed-exchange-coherence`

Uses: `def:weil-pbc-completed-base-change-map`

The completed exchange map is natural in K, is the identity for identity squares, and commutes with horizontal and vertical pasting.

Proof. Uses: `lemma:weil-pbc-completed-map-reduction,lemma:weil-pbc-r-lim-levelwise-isomorphism`

At every finite level these equations are the S0/S8 equations. They commute with the tower transitions by naturality of coefficient reduction. Apply $R\varprojlim$ to each equality and conjugate by the same reconstruction, projection-formula, and direct-image/limit isomorphisms used in the definition. The resulting equations are precisely the completed equations. □

Proposition 11.11.24 (Complete pro-etale proper base change). *Lean:* `conditional-interface:prop:weil-pbc-proet-complete-proper-base-change`

Uses: `def:weil-pbc-coefficient-datum,def:weil-pbc-completed-base-change-map`

Let $f : X \to Y$ be proper and let $\square$ be any base change. Suppose $K \in D(X_{\mathrm{proet}}, \Lambda)$ is derived I-complete, each K_n is equipped with a displayed witness $\epsilon_X^{-1} L_n \simeq K_n$ for some $L_n \in D_c^+(X_{\mathrm{et}}, \Lambda/I^n)$, and every Λ/I^n is perfect. Then

$$Lg^*_{\mathrm{comp}} Rf_* K \xrightarrow{\sim} Rf'_* L(g')^*_{\mathrm{comp}} K.$$

The isomorphism is natural and satisfies identity and pasting coherence. If the square and the reductions carry a Frobenius-equivariant square and Weil tower datum, this is an isomorphism of the completed Weil objects (S, Θ_S) and (T, Θ_T).

Proof. Uses: `def:weil-pbc-completed-base-change-map,prop:weil-pbc-proet-finite-level-transport,lemma:weil-pbc-r-lim-levelwise-isomorphism,lemma:weil-pbc-completed-exchange-coherence,lemma:weil-pbc-completed-frobenius-equivariance`

Every finite-level middle arrow in Definition 11.11.15 is an isomorphism by S8. The derived limit preserves this compatible levelwise isomorphism. The outer arrows in the defining composite are reconstruction, perfect projection formula, and direct-image/limit isomorphisms. Hence the completed map is an isomorphism. Coherence is the preceding lemma. This proves the mathematical content located at Tag `09C9` without using that theorem as a leaf. The completed Frobenius-equivariance lemma gives the last assertion with the genuine pullback type $\Phi_{Y'} S \to T$. □

Definition 11.11.25 (A2 constructible rebinding certificate). *Lean:* `conditional-interface:def:weil-pbc-a2-constructible-rebind`

Uses: `def:weil-a2-dbc-zl,def:weil-a2-dbc-torsion,def:weil-a2-proper-complete-base-change-datum,def:weil-a2-finite-level-etale-origin-realization,def:weil-a2-torsion-constructibility-witness,def:weil-a2-constructible-stratification-witness,def:weil-a1-torsion-derived-category,def:weil-a1-rfshriek-selected,def:weil-pbc-level-map-corners,def:weil-pbc-completed-reduction-square`

Specialize A2 to $\Lambda = \mathbf{Z}_\ell$, $I = (\ell)$, and $P_n = \Lambda/\ell^n$. For a scheme Z write

$$\mathsf{C}(Z) := D_c^b(Z_{\mathrm{proet}}, \mathbf{Z}_\ell), \qquad \mathsf{C}_n(Z) := D_c^b(Z_{\mathrm{proet}}, \mathbf{Z}/\ell^n)$$

for the full subcategories defined, respectively, by `def:weil-a2-dbc-zl` and `def:weil-a2-dbc-torsion`.

Fix a proper Cartesian square $\square$ as in S10. An A2 constructible rebinding certificate for $K \in \mathsf{C}(X)$ consists of the following displayed data.

1. An A2 proper complete-base-change datum, hence compatible etale-origin objects

$$L_n \in D_c^b(X_{\mathrm{et}}, \mathbf{Z}/\ell^n), \qquad \alpha_n : \epsilon_X^{-1} L_n \xrightarrow{\sim} K_n,$$

including their transition equations. The superscript b is part of the certificate: one interval $[d_0, d_1]$ works for every L_n and every K_n.

2. On each of X, X', Y, Y', one finite stratification, each stratum having finitely many connected components, and A2 finite-torsion constructibility witnesses, uniform in n, for every finite object occurring in S7–S10. On Y' these are, in particular, the four objects A_n, B_n, C_n, D_n of Definition 11.11.1; their witnesses use one common interval and one common refinement.

3. The finite exceptional-direct-image values for the identity compactifications of the proper maps,

$$Rf_!^{(1,f)} K_n \in \mathsf{C}_n(Y), \qquad Rf'^{(1,f')}_! E_n \in \mathsf{C}_n(Y'),$$

with the A2 torsion witnesses retained by the typed A1 input and with their transition squares. The certificate records their displayed identifications with $Rf_* K_n$ and $Rf'_* E_n$.

4. A2 constructible-stratification witnesses for

$$S(K) := Lg^*_{\mathrm{comp}} Rf_* K, \qquad T(K) := Rf'_* L(g')^*_{\mathrm{comp}} K$$

in $\mathsf{C}(Y')$. Their finite reductions are required to be the objects identified by the maps α_n and δ_n of Definition 11.11.20.

Let $\mathsf{C}_\square(X)$ be the full subcategory of $\mathsf{C}(X)$ on objects for which the type of such certificates is nonempty. Certificates are membership witnesses only; morphisms in $\mathsf{C}_\square(X)$ are the ambient A2 morphisms, so no selected witness enters their equality.

Lemma 11.11.26 (Reduction and completed-tensor transport to A2). *Lean:* `conditional-interface:lemma:weil-pbc-a2-reduction-tensor-transport`

Uses: `def:weil-pbc-a2-constructible-rebind, def:weil-a2-finite-reduction, def:weil-a2-reduction-transition, def:weil-a2-completed-tensor, def:weil-pbc-completed-coefficient-datum, def:weil-pbc-completed-tensor-product`

For every K and n, there is a canonical natural isomorphism

$$\chi_{K,n} : \mathrm{Red}_n^{\mathrm{A2}}(K) \xrightarrow{\sim} K \otimes^L_{\mathbf{Z}_\ell} \mathbf{Z}/\ell^n = K_n.$$

It intertwines the A2 transition with the quotient transition $q_{n+1,n}$ of S9. For complete A, B there is likewise a canonical natural isomorphism

$$\omega_{A,B} : A \widehat{\otimes}^{L,\mathrm{A2}} B \xrightarrow{\sim} A \widehat{\otimes}^{L,\mathrm{S9}} B,$$

and its n-th reduction is the identity of $A \otimes^L B \otimes^L \mathbf{Z}/\ell^n$ under χ. Both isomorphisms are independent of the K-flat, quotient-complex, and completion aliases.

Proof. Uses: `lemma:weil-a2-finite-reduction-package-independent,prop:weil-a2-finite-reduction-functor,lemma:weil-pbc-zl-reduction-perfect,lemma:weil-pbc-completed-tensor-reduction`

A2 represents $\mathbf{Z}/\ell^n$ by the two-term complex

$$L_n = [\mathbf{Z}_\ell \xrightarrow{\ell^n} \mathbf{Z}_\ell].$$

The augmentation $L_n \to \mathbf{Z}/\ell^n$ is a quasi-isomorphism and S9 uses the same complex as its strict perfect resolution. Tensor it with the A2 K-flat model of K and localize. This gives $\chi_{K,n}$. If the K-flat model is changed, the two maps are obtained from a common comparison roof; A2 package independence says that they are equal. Thus χ is canonical, and A2 functoriality makes it natural.

The A2 transition is represented by multiplication by ℓ in degree -1 and the identity in degree 0. After the augmentation it is the quotient map $\mathbf{Z}/\ell^{n+1} \to \mathbf{Z}/\ell^n$. Therefore

$$\chi_{K,n}\, r^{\mathrm{A2}}_{n+1,n} = q_{n+1,n}\, \chi_{K,n+1}.$$

Finally, A2 completes the localized K-flat tensor of A and B, whereas S9 takes the product-cone limit of its reductions. Apply χ levelwise to these two identical towers. The transition equation just proved gives a map of product-cone models and hence $\omega_{A,B}$. Reduction of completion and the S9 completed-tensor reduction identify its n-th component with the identity of the displayed triple tensor. Common K-flat roofs give identical level maps, and equality into the two products is componentwise; consequently the induced cone map is independent of every alias. □

Lemma 11.11.27 (Site-comparison and exceptional-image transport to A2). *Lean:* `conditional-interface:lemma:weil-pbc-a2-comparison-rfshriek-transport`

Uses: `def:weil-pbc-a2-constructible-rebind`, `def:weil-a2-etale-origin-datum`, `def:weil-a2-etale-proetale-pullback-comparison`, `def:weil-a2-etale-proetale-direct-image-comparison`, `def:weil-a2-six-operation-coefficient-target`, `def:weil-a1-torsion-derived-category`, `def:weil-a1-rfshriek-selected`, `def:weil-pbc-epsilon-comparison`, `def:weil-pbc-projection-compactification`

Under the identity-on-objects site functor of S8, the A2 notation ν_Z^* is the functor ϵ_Z^{-1}. The A2 pullback comparison c_g, direct-image comparison d_f, and etale-origin isomorphism therefore transport to the S8 maps with the same components. Moreover the A1 typed input category $\mathcal{D}_n(Z)$ transports by the identity on objects and morphisms to $\mathsf{C}_n(Z)$, and on a proper map

$$Rf_!^{(1,f)} = Rf_* : \mathsf{C}_n(X) \longrightarrow \mathsf{C}_n(Y).$$

Thus the $Rf_!$ field of the A2 six-operation target has exactly the displayed source and target on $\mathsf{C}_\square(X)$. These transports are independent of whether the A1, A2, S8, or S9 alias is used.

Proof. Uses: `lemma:weil-pbc-epsilon-pullback`, `lemma:weil-pbc-epsilon-direct-image`, `lemma:weil-pbc-projection-formula-identity-compactification`

Both ν_Z and ϵ_Z are the geometric morphism induced by the functor that sends an etale object to the same object of the pro-etale site. Their inverse images are the same sheafified left Kan extension, so the identity of the indexing categories identifies them. Expanding c_g gives the S8 pullback square, and expanding d_f gives the S8 unit followed by the same derived direct image. The component formulas and their naturality squares are therefore literally equal, not merely abstractly isomorphic.

A1 denoted by $\mathcal{D}_n(Z)$ the typed finite torsion constructible category whose construction was assigned to A2. A2 realizes that input as the full subcategory defined by its finite-torsion constructibility witness. Consequently the transport functor and its inverse are both the identity on underlying complexes and maps; their two composites are definitional identities. The selected A1 exceptional image is $Rp_*j_!$. This is the S9 definition of $Rf_!^{(j,p)}$. When f is proper, the selected compactification is $(1_X, f)$, so $j_! = 1$ and both expressions are Rf_*. The identity-compactification lemma shows that the projection-formula arrows also agree. Since all transports are identities on representatives, changing only the displayed alias cannot change an object, a morphism, or a coherence equation. □

Lemma 11.11.28 (A2 compatibility with every S7–S10 endpoint). *Lean:* `conditional-interface:lemma:weil-pbc-a2-s7-s10-endpoint-compatibility`

Uses: `def:weil-a2-finite-etale-proper-base-change-map`, `def:weil-a2-transported-finite-level-base-change-map`, `def:weil-a2-proetale-proper-base-change-map`

Let $K \in \mathsf{C}_\square(X)$. After the transports χ and ω:

1. *the A2 finite etale map b_n is the S7 unit–counit exchange map;*
2. *the A2 transported pro-etale map $\widetilde{\beta}_n$ is the S8 map v'_n;*
3. *the A2 product-cone map β_K^{proet} is the S10 completed map ξ_K;*
4. *the n-th reduction of either completed alias is exactly $\widetilde{\beta}_n$, and the equations are compatible with every tower transition, compatible tower endomorphism, identity square, and pasted square.*

All four equalities are independent of the rebinding certificate chosen for K.

Proof. Uses: `lemma:weil-pbc-a2-reduction-tensor-transport`, `lemma:weil-pbc-a2-comparison-rfshriek-transport`, `prop:weil-proet-finite-derived-proper-base-change`, `prop:weil-pbc-proet-finite-level-transport`, `lemma:weil-pbc-bounded-constructible-reconstruction`, `lemma:weil-pbc-completed-pullback-reduction`, `lemma:weil-pbc-completed-tensor-reduction`, `lemma:weil-pbc-projection-formula-identity-compactification`, `lemma:weil-pbc-completed-map-reduction`, `lemma:weil-pbc-completed-exchange-coherence`

The A2 definition of b_n and the S7 definition both use the adjunction unit, the chosen Cartesian equality $fg' = gf'$, and the adjunction counit, in that order. Hence their resolved representatives are the same chain map. By the preceding lemma, the four A2 etale/pro-etale comparison arrows are the four S8 arrows. Pasting the same comparison squares around the common middle map b_n therefore makes $\widetilde{\beta}_n = v'_n$.

The A2 completed map and the S10 completed map are both induced from the tower $(v'_n)_n$ on the product-cone model. The reduction and tensor transports commute with the transitions, so the induced maps on both products are componentwise equal and their cone maps are equal. The completed reduction square then gives

$$\delta_n \, \xi_K = v'_n \, \alpha_n,$$

which is precisely the fourth assertion under χ. S7 naturality, S8 comparison naturality, the S9 tower equations, and S10 completed coherence give the transition, operator, identity, and pasting equations. Finally, the certificate

supplies only membership and bound witnesses: none occurs in the unit–counit formula or in the product-cone map. Changing it therefore leaves every displayed morphism unchanged. □

Lemma 11.11.29 (Constructible-subcategory restriction). *Lean:* `conditional-interface: lemma: weil-pbc-complete-constructible-restriction`

Uses: `def: weil-pbc-a2-constructible-rebind`

For every proper Cartesian square, the complete base-change isomorphism restricts to a natural isomorphism on the selected A2 full subcategory:

$$\xi_K : S(K) = Lg^*_{\mathrm{comp}} Rf_* K \xrightarrow{\sim} T(K) = Rf'_* L(g')^*_{\mathrm{comp}} K, \qquad K \in \mathsf{C}_{\square}(X),$$

where both endpoints lie in $\mathsf{C}(Y')$. Its finite reductions, $Rf_!$ proper aliases, etale/pro-etale transports, compatible tower endomorphisms, and identity and pasting equations are the A2 transports of the S7–S10 endpoints. The restricted isomorphism is independent of all certificate and notation choices.

Proof. Uses: `lemma:weil-pbc-a2-reduction-tensor-transport,lemma:weil-pbc-a2-comparison-rfshriek-transport,lemma:weil-pbc-a2-s7-s10-endpoint-compatibility,prop:weil-proet-complete-proper-base-change`

A rebinding certificate supplies the compatible bounded constructible etale witnesses required in the complete proposition. Its uniform intervals and stratifications are exactly the input of S9 bounded reconstruction, while the perfect coefficient is the two-term resolution used in the reduction transport lemma. Thus Proposition 11.11.24 gives an ambient isomorphism $\xi_K : S(K) \to T(K)$ and its explicit ambient inverse.

The last field of the certificate exhibits $S(K)$ and $T(K)$ as objects of the A2 full subcategory $\mathsf{C}(Y')$. Fullness puts ξ_K and its inverse in that subcategory, and faithfulness preserves their two inverse equations. The three transport lemmas identify all A2 and A3 aliases and prove every finite-reduction and coherence equation. Since the ambient map is constructed without consulting a membership witness, replacing the certificate changes neither the map nor its inverse. This proves the claimed restriction with no additional external hypothesis. □

Lemma 11.11.30 (Extension by zero preserves the zero object). *Lean:* `blueprint-target: lemma: weil-a1-extension-zero-object-v1`

Uses: `def: weil-a1-extension-zero-presheaf, def: weil-a1-extension-zero-sheaf`

For an open immersion $j : U \hookrightarrow X$, the canonical morphism

$$j_! 0_U \longrightarrow 0_X$$

is an isomorphism. Equivalently, $j_! 0_U$ is a zero object in the category of Λ_n-module sheaves on $X_{\mathrm{pro\acute{e}t}}$.

Proof. Uses: `def:weil-a1-extension-zero-presheaf,def:weil-a1-extension-zero-sheaf`

For every test object $V \to X$, the zero sheaf on U has only its zero section on $V \times_X U$. That section has empty support, so each of the three clauses defining $P_{j,0_U}(V)$ gives the zero module. Thus $P_{j,0_U}$ is the zero presheaf. Sheafification, as a left adjoint, preserves the initial object; in the category of module sheaves the initial object is also terminal. Hence its sheafification $j_! 0_U$ is a zero object, and the displayed canonical morphism is an isomorphism. □

Lemma 11.11.31 (Extension by zero preserves zero morphisms). *Lean:* `blueprint-target: lemma: weil-a1-extension-zero-morphism-v1`

Uses: `def: weil-a1-extension-zero-sheaf`

For Λ_n-module sheaves F, G on U, one has

$$j_!(0_{F,G}) = 0_{j_! F, j_! G}.$$

Proof. Uses: `def:weil-a1-extension-zero-sheaf`

By definition, $j_!(0_{F,G})$ is induced by applying $0_{F,G}$ to every support-closed local representative. Every such representative is sent to the zero section. The induced presheaf morphism is therefore the zero morphism, and the morphism obtained from it by sheafification is the zero sheaf morphism. This proves the displayed equality without using exactness of $j_!$. □

Lemma 11.11.32 (Square-zero transport for open extension). *Lean:* `blueprint-target: lemma: weil-a1-open-extension-square-zero-v1`

Uses: `def: weil-a1-extension-zero-sheaf`

If $a : F \to G$ and $b : G \to H$ satisfy $b \circ a = 0$, then

$$j_! b \circ j_! a = 0.$$

Proof. Uses: lemma:weil-a1-extension-zero-functorial,lemma:weil-a1-extension-zero-morphism-v1
Functoriality and preservation of the zero morphism give

$$j_!b \circ j_!a = j_!(b \circ a) = j_!0 = 0.$$

The order of composition agrees with the cochain convention: a is applied first and b second. □

Lemma 11.11.33 (Boundedness of termwise open extension). *Lean: blueprint-target:lemma:weil-a1-open-extension-bounded-v1*

Uses: def:weil-a1-extension-zero-sheaf
If a graded family $(K^r)_{r\in\mathbf{Z}}$ is zero outside $[a,b]$, then $(j_!K^r)_{r\in\mathbf{Z}}$ is zero outside the same interval.

Proof. Uses: lemma:weil-a1-extension-zero-object-v1
For $r < a$ or $r > b$, the term K^r is a zero object. The preceding zero-object lemma makes $j_!K^r$ a zero object as well. No exactness or cohomological-amplitude assertion is used. □

Definition 11.11.34 (Bridge termwise open-extension complex). Lean: blueprint-target:def:weil-a1-open-extension-complex

Uses: def:weil-a1-extension-zero-sheaf
For an open immersion $j : U \hookrightarrow X$ and a bounded complex $K = (K^r, d^r)$ of finite-torsion constructible sheaves on U, define $j_!^{\mathrm{cx}}K$ degreewise by

$$(j_!^{\mathrm{cx}}K)^r := j_!K^r, \qquad d^r_{j_!K} := j_!(d^r).$$

For every r, the square-zero transport lemma gives

$$d^{r+1}_{j_!K} \circ d^r_{j_!K} = j_!(d^{r+1}) \circ j_!(d^r) = 0,$$

and the boundedness lemma retains any interval outside which the terms of K vanish. Thus this is a bounded complex in the ambient category of Λ_n-module sheaves on $X_{\mathrm{pro\acute{e}t}}$.

Constructed by: lemma:weil-a1-open-extension-square-zero-v1,lemma:weil-a1-open-extension-bounded-v1
This bridge does not assert preservation of constructibility, exactness, functoriality on chain maps, or descent through quasi-isomorphisms; those remain in their separately indexed A1/A4 interfaces.

Definition 11.11.35 (Bridge underived compactification refinement map). Lean: conditional-interface:def:weil-a1-refinement-underived-map

Uses: def:weil-a1-compactification-morphism,def:weil-a1-extension-zero-sheaf
Let $q : \overline{X}_D \to \overline{X}_C$ be a compactification morphism over Y, with $q \circ j_D = j_C$, and let K be a finite-torsion constructible sheaf on X. Define

$$\gamma^0_{q,K} : q_* j_{D!}K \longrightarrow j_{C!}K$$

on a local support-closed section by transporting its support along (q), using $q\circ j_D = j_C$, and applying the identity on the coefficient module. Define the reverse local transport on the corresponding factorization representatives and denote the two local maps by $\Theta_{q,V}$ and $\Xi_{q,V}$. Sheafification gives the displayed maps; their inverse equations are recorded separately.

Open obligation: OBL-SF-BRIDGE-UNDERIVED-REFINEMENT

Lemma 11.11.36 (Bridge refinement factorization left inverse). *Lean: conditional-interface:lemma:weil-a1-refinement-factorization-left*

Uses: def:weil-a1-refinement-underived-map
For every Cartesian source square and every local test object V, the factorization maps in the preceding bridge should satisfy

$$\Theta_{q,V}\,\Xi_{q,V} = \mathrm{id}\,.$$

Proof. Uses: def:weil-a1-refinement-underived-map,lemma:weil-a2-finite-reduction-package-independent,prop:weil-a2-finite-reduction-functor,lemma:weil-pbc-zl-reduction-perfect,lemma:weil-pbc-completed-tensor-reduction
The assertion is the explicit local-index inverse calculation required to identify the two transported support representatives. Its geometric factorization and sheafification verification remains an independent leaf.

Open obligation: OBL-SF-BRIDGE-FACTOR-LEFT

□

Lemma 11.11.37 (Bridge refinement factorization right inverse). *Lean:* `conditional-interface:lemma:weil-a1-refinement-factorization-right`

Uses: `def:weil-a1-refinement-underived-map`

For every Cartesian source square and every local test object (V), the factorization maps in the preceding bridge should satisfy

$$\Xi_{q,V}\,\Theta_{q,V} = \mathrm{id}\,.$$

Proof. Uses: `def:weil-a1-refinement-underived-map`

The assertion is the reverse local-index inverse calculation. The fiber-product uniqueness and compatibility with sheafification are kept as the independent obligation named below.

Open obligation: `OBL-SF-BRIDGE-FACTOR-RIGHT`

□

Proposition 11.11.38 (Bridge open-extension adjunction). *Lean:* `blueprint-target:prop:weil-a1-open-extension-adjunction`

Uses: `def:weil-a1-extension-zero-sheaf`, `def:weil-a1-extension-adjunction-forward`, `def:weil-a1-extension-adjunction-backward`

For finite-torsion sheaves F on U and G on X, the forward map is

$$\mathrm{res}_{F,G} : \mathrm{Hom}_X(j_!F, G) \longrightarrow \mathrm{Hom}_U(F, j^*G),$$

and its inverse is

$$\mathrm{ext}_{F,G} : \mathrm{Hom}_U(F, j^*G) \longrightarrow \mathrm{Hom}_X(j_!F, G).$$

They are natural contravariantly in F and covariantly in G; hence they give the underived adjunction $j_! \dashv j^*$.

Constructed by: `prop:weil-a1-extension-zero-adjunction`

Proof. Uses: `prop:weil-a1-extension-zero-adjunction`

These are exactly the maps denoted res and ext in Proposition 10.1.13, which proves that they are mutually inverse and natural. Explicitly, for $u : F' \to F$, $v : G \to G'$, $h : j_!F \to G$, and $k : F \to j^*G$, that naturality has variance

$$\begin{aligned}
\mathrm{res}_{F',G}(h \circ j_!u) &= \mathrm{res}_{F,G}(h) \circ u,\\
\mathrm{res}_{F,G'}(v \circ h) &= j^*v \circ \mathrm{res}_{F,G}(h),\\
\mathrm{ext}_{F',G}(k \circ u) &= \mathrm{ext}_{F,G}(k) \circ j_!u,\\
\mathrm{ext}_{F,G'}(j^*v \circ k) &= v \circ \mathrm{ext}_{F,G}(k).
\end{aligned}$$

Thus the displayed Hom bijection has the asserted direction. This is only the underived sheaf adjunction and uses no complex, exactness, or derived descent statement. □

11.12 Exceptional inverse image and Verdier duality

> **Status.** This section is downstream from A1–A3. It never upgrades an OPEN predecessor to a theorem. A displayed result carrying OPEN records the exact remaining proof leaf and may be used only in a final theorem whose hypotheses explicitly include that leaf.

11.12.1 Coefficient, Frobenius, and twist conventions

Definition 11.12.1 (A4 coefficient index). *Lean:* `conditional-interface:def:weil-a4-coefficient-index`

Uses: `def:weil-a2-dbc-zl`, `def:weil-a2-dbc-e`, `def:weil-a2-dbc-qbar`

An A4 coefficient index is one of

$$\Lambda_n = \mathbf{Z}/\ell^n\mathbf{Z}, \qquad \Lambda = \mathbf{Z}_\ell, \qquad E/\mathbf{Q}_\ell \text{ finite}, \qquad \overline{\mathbf{Q}}_\ell.$$

For a finite-type scheme X, write $D^b_c(X, A)$ for the corresponding A2 category. At $A = \Lambda_n$ this means the finite-torsion category; at the other three indices it means the A2 integral, rationalized, or filtered-colimit category.

Definition 11.12.2 (Structural morphism and coefficient unit). *Lean:* `conditional-interface:def:weil-a4-structural-unit`

Uses: `def:weil-a4-coefficient-index`

For X of finite type over $k = \mathbf{F}_q$, let $a_X : X \to \mathrm{Spec}\,k$ be the structural morphism. Denote by A_X the pullback of the coefficient ring A, regarded as the tensor unit in $D^b_c(X, A)$.

Definition 11.12.3 (Geometric Frobenius convention). Lean: `conditional-interface:def:weil-a4-geometric-frobenius-convention`

Uses: `def:weil-a4-structural-unit`

Let $F_{X,\mathrm{ar}}$ be the endomorphism induced by $x \mapsto x^q$ on functions and let $\Phi_X = F_{X,\mathrm{ar}}^{-1}$ on geometric points. Every Frobenius action in A4 is the pullback action of Φ_X, called geometric Frobenius. No later declaration changes this convention.

Definition 11.12.4 (Tate object and Tate twist). Lean: `conditional-interface:def:weil-a4-tate-twist`

Uses: `def:weil-a4-coefficient-index,def:weil-a4-geometric-frobenius-convention`

Define $\mathbf{Z}_\ell(1) := \varprojlim_r \mu_{\ell^r}$. At finite level define $\Lambda_n(1) = \mathbf{Z}_\ell(1) \otimes^L_{\mathbf{Z}_\ell} \Lambda_n$ on the pro-etale site and define $\Lambda_n(m) = \Lambda_n(1)^{\otimes m}$ for $m \geq 0$; for $m < 0$ use the tensor dual. The integral and characteristic-zero twists are the A2 reconstruction and scalar extensions of this tower. For $K \in D^b_c(X, A)$, put $K(m) = K \otimes^L_A A_X(m)$. Our normalization is

$$\Phi_{\operatorname{Spec} k}|_{A(1)} = q^{-1}, \qquad \Phi_{\operatorname{Spec} k}|_{A(-1)} = q.$$

Lemma 11.12.5 (Additivity of Tate twists). Lean: `conditional-interface:lemma:weil-a4-tate-twist-addition`

Uses: `def:weil-a4-tate-twist`

For all integers r, s, the multiplication of tensor powers defines

$$(K(r))(s) \longrightarrow K(r+s).$$

Its inverse is induced by inserting the coevaluation of $A_X(1)$ and its tensor dual, and the two composites are identities.

Proof. Uses: `def:weil-a4-tate-twist`

For nonnegative r, s, reassociate the $r+s$ tensor factors; the inverse uses the opposite reassociation. If one exponent is negative, cancel each adjacent pair $A_X(1) \otimes A_X(-1)$ by evaluation. In the reverse composite, the evaluation–coevaluation triangle leaves every uncancelled factor unchanged. This proves both inverse equations. □

11.12.2 Finite-level construction of the exceptional inverse image

Definition 11.12.6 (Separated finite-type exceptional index). Lean: `conditional-interface:def:weil-a4-exceptional-index`

Uses: `def:weil-a4-coefficient-index,def:weil-a1-rfshriek-selected`

An exceptional index is a separated finite-type morphism $f : X \to Y$, a finite coefficient ring Λ_n, and the A1 selected compactification $X \xrightarrow{j_f} \overline{X}_f \xrightarrow{p_f} Y$, together with its constructible bounded compact-support model $Rf_! : D^b_c(X, \Lambda_n) \to D^b_c(Y, \Lambda_n)$. All A1 compactification-independence and composition obligations remain attached to this model.

Definition 11.12.7 (Ambient compact-support candidate). Lean: `conditional-interface:def:weil-a4-ambient-rfshriek-candidate`

Uses: `def:weil-a4-exceptional-index,def:weil-a2-derived-category`

For $K \in D(X, \Lambda_n)$, choose a complex representative $K^\bullet$, apply the sectionwise open extension $j_{f!}K^\bullet$, choose a K-injective replacement $I^\bullet$ on $\overline{X}_f$, and define

$$Rf_{!,\mathrm{amb}}K := p_{f*}I^\bullet \in D(Y, \Lambda_n).$$

On a derived roof, lift both arrows to the selected K-injective replacements and apply p_{f*}. In the Brown block, $Rf_!$ denotes this ambient candidate; on D^b_c it is compared with the A1 model below.

Definition 11.12.8 (Finite-origin compactification input). Lean: `conditional-interface:def:weil-a4-finite-origin-compactification-input`

Uses: `def:weil-a4-exceptional-index,def:weil-pbc-comes-from-etale,def:weil-a1-open-extension-complex`

Fix integers $a \leq b$. A finite-origin compactification input consists of $K \in D_c^{[a,b]}(X_{\mathrm{pro\acute{e}t}}, \Lambda_n)$, an object $L \in D_c^{[a,b]}(X_{\acute{e}t}, \Lambda_n)$, and an isomorphism

$$\theta : \epsilon_X^{-1} L \xrightarrow{\sim} K.$$

Put

$$J := j_{f,\text{ét},!}L, \qquad \overline{K} := j_{f,\text{proét},!}K.$$

The inverse equations for θ, the interval $[a,b]$, and the two chosen complex representatives are fields of the input.

Definition 11.12.9 (Open-extension site-comparison map). Lean: `conditional-interface:def:weil-a4-open-extension-site-comparison`

Uses: `def:weil-a4-finite-origin-compactification-input,def:weil-pbc-epsilon-comparison`

For a finite-origin compactification input, define

$$c_{j,L} : \epsilon_{\overline{X}_f}^{-1} j_{f,\text{ét},!}L \longrightarrow j_{f,\text{proét},!}\epsilon_X^{-1}L$$

degreewise as follows. On a pro-étale object $V \to \overline{X}_f$, a local section represented on $V_i \to \overline{X}_f$ at a finite étale stage is sent to the same section on $V_i \times_{\overline{X}_f} X$, extended by zero on the complement. Define the comparison

$$\overline{\theta}_j : \epsilon_{\overline{X}_f}^{-1} J \longrightarrow \overline{K}$$

to be $j_{f,!}(\theta)c_{j,L}$.

Lemma 11.12.10 (Open-extension site comparison is an isomorphism). Lean: `conditional-interface:lemma:weil-a4-open-extension-site-comparison-iso`

Uses: `def:weil-a4-open-extension-site-comparison`

The maps $c_{j,L}$ and $\overline{\theta}_j$ are isomorphisms, natural in the finite-origin input and compatible with identity and composition of quasi-compact open immersions.

Proof. Uses: `def:weil-a4-open-extension-site-comparison,lemma:weil-pbc-epsilon-scheme-naturality,lemma:weil-a1-refinement-factorization-left,lemma:weil-a1-refinement-factorization-right`

Both sites compute extension by zero from the same dichotomy. If a test object factors through X, both values are the inverse image of the value of L on that factorization. If it does not, both supported values are zero after the covering used in the sheafification. Define the reverse map by the identity on the first value and the zero map on the second. The two composites are identities on every local representative; the two factorization inverse equations identify their indices. Sheafification preserves these equations. Applying $j_!(\theta)$ and $j_!(\theta^{-1})$ gives the two inverse equations for $\overline{\theta}_j$. Scheme naturality of ϵ^{-1} and associativity of the factorization map give naturality, identity, and composition. □

Lemma 11.12.11 (Open extension has cohomological amplitude zero). Lean: `conditional-interface:lemma:weil-a4-open-extension-amplitude-zero`

Uses: `def:weil-a4-finite-origin-compactification-input`

If $K \in D^{[a,b]}(X_{\text{proét}}, \Lambda_n)$, then

$$j_{f,!}K \in D^{[a,b]}(\overline{X}_{f,\text{proét}}, \Lambda_n), \qquad \mathcal{H}^r(j_{f,!}K) \simeq j_{f,!}\mathcal{H}^r(K).$$

Proof. Uses: `def:weil-a4-finite-origin-compactification-input`

For a short exact sequence of sheaves, restrict to a pro-étale object over X and to one over the closed complement. On the first restriction $j_!$ is the identity, and on the second it is the zero functor. These objects form a covering basis of $\overline{X}_f$, so kernels and cokernels of the extended sequence agree locally with the extensions of the original kernels and cokernels. Hence $j_!$ is exact. Applying it degreewise to

$$\ker d^r / \operatorname{im} d^{r-1}$$

gives the displayed cohomology-sheaf formula. The formula is zero for $r < a$ and $r > b$, which is the asserted amplitude. □

Lemma 11.12.12 (Open extension preserves finite constructibility). Lean: `conditional-interface:lemma:weil-a4-open-extension-constructible`

Uses: `def:weil-a4-finite-origin-compactification-input`

For a finite-origin compactification input, J and $\overline{K}$ are bounded constructible in the interval $[a,b]$, and $\overline{\theta}_j : \epsilon^{-1}J \xrightarrow{\sim} \overline{K}$ identifies their cohomology sheaves.

Proof. Uses: lemma:weil-a4-open-extension-amplitude-zero,lemma:weil-a4-open-extension-site-comparison-iso,lemma:weil-pbc-epsilon-direct-image-constructible-bounds

Refine the finite constructible stratification of L by its intersections with X and adjoin the locally closed complement $\overline{X}_f \setminus X$. On an old stratum, $j_!\mathcal{H}^r(L)$ is the same finite locally constant sheaf; on the complement it is zero. Therefore J is constructible. The amplitude-zero lemma retains exactly $[a,b]$. Exactness of ϵ^{-1} transports each cohomology sheaf, and the site-comparison isomorphism identifies it with the corresponding cohomology sheaf of $\overline{K}$. □

Definition 11.12.13 (Relative-dimension certificate for the proper piece)**.** Lean: conditional-interface:def:weil-a4-proper-relative-dimension-certificate

Uses: def:weil-a4-exceptional-index

A relative-dimension certificate for $p_f : \overline{X}_f \to Y$ is a natural number d together with, for every geometric point $\bar{y} \to Y$, the inequality

$$\dim(\overline{X}_f \times_Y \bar{y}) \le d.$$

The integer and the family of inequalities are data; no global bound is silently selected for a non-quasi-compact base.

Definition 11.12.14 (Finite Artin–constructibility predicate)**.** Lean: conditional-interface:def:weil-a4-finite-artin-constructibility-predicate

Uses: def:weil-a4-proper-relative-dimension-certificate,def:weil-a4-finite-origin-compactification-input

For a relative-dimension certificate d, define $\mathrm{ArtFin}(p_f, \Lambda_n; d)$ to mean that for every constructible sheaf F of finite Λ_n-modules on $\overline{X}_{f,\text{ét}}$,

$$R^r p_{f,\text{ét},*}F = 0 \quad (r > 2d), \qquad R^r p_{f,\text{ét},*}F \text{ is constructible} \quad (0 \le r \le 2d).$$

The predicate concerns the small étale site and finite-origin sheaves; it does not quantify over arbitrary sheaves on the full pro-étale site.

Lemma 11.12.15 (Artin–constructibility gives the proper amplitude)**.** Lean: conditional-interface:lemma:weil-a4-artin-constructibility-amplitude

Uses: def:weil-a4-finite-artin-constructibility-predicate,def:weil-a4-finite-origin-compactification-input

Assume $\mathrm{ArtFin}(p_f, \Lambda_n; d)$. *Then*

$$J \in D_c^{[a,b]}(\overline{X}_{f,\text{ét}}, \Lambda_n) \quad \Longrightarrow \quad Rp_{f,\text{ét},*}J \in D_c^{[a,b+2d]}(Y_{\text{ét}}, \Lambda_n).$$

Proof. Uses: def:weil-a4-finite-artin-constructibility-predicate

Use the bounded hypercohomology spectral sequence

$$E_2^{r,s} = R^r p_{f,\text{ét},*}\mathcal{H}^s(J) \Longrightarrow \mathcal{H}^{r+s}(Rp_{f,\text{ét},*}J).$$

The input complex kills $s < a$ and $s > b$, while ArtFin kills $r < 0$ and $r > 2d$. Thus a term of total degree m can occur only for $a \le m \le b + 2d$, and every total degree has finitely many terms. In the remaining rectangle each $E_2^{r,s}$ is constructible. Kernels, images, and cokernels of maps of constructible finite sheaves are constructible on a common finite stratification, so every later page is constructible. The finite abutment filtration expresses each cohomology sheaf as finitely many extensions of constructible graded pieces. This proves both the interval and constructibility. □

Definition 11.12.16 (A3 proper-piece comparison)**.** Lean: conditional-interface:def:weil-a4-a3-proper-piece-comparison

Uses: def:weil-a4-finite-origin-compactification-input,def:weil-pbc-epsilon-direct-image-comparison

For the proper map p_f and the étale complex J, define

$$d_{p_f,J} : \epsilon_Y^{-1} Rp_{f,\text{ét},*}J \longrightarrow Rp_{f,\text{proét},*}\epsilon_{\overline{X}_f}^{-1} J$$

to be the A3 direct-image comparison. Compose its target with $Rp_{f,*}(\overline{\theta}_j)$ to obtain

$$\widetilde{d}_{f,K} : \epsilon_Y^{-1} Rp_{f,\text{ét},*}J \longrightarrow Rp_{f,\text{proét},*}\overline{K}.$$

Lemma 11.12.17 (The A3 proper-piece comparison is an isomorphism). *Lean:* `conditional-interface:lemma:weil-a4-a3-proper-piece-comparison-iso`

Uses: `def:weil-a4-a3-proper-piece-comparison`

For every finite-origin compactification input, $d_{p_f,J}$ and $\widetilde{d}_{f,K}$ are natural isomorphisms in $D^+(Y_{\mathrm{pro\acute{e}t}},\Lambda_n)$.

Proof. Uses: `lemma:weil-pbc-epsilon-direct-image,lemma:weil-a4-open-extension-site-comparison-iso`

The complex J is bounded below and constructible by the open-extension lemma. Instantiate A3 direct-image transport with p_f, J, and the finite ring Λ_n; it makes $d_{p_f,J}$ an isomorphism. The second map is its composite with $Rp_{f,*}(\overline{\theta}_j)$, which is an isomorphism because $\overline{\theta}_j$ is. Naturality is the naturality of the two factors. □

Lemma 11.12.18 (A3 comparison respects composition and refinement). *Lean:* `conditional-interface:lemma:weil-a4-a3-proper-piece-coherence`

Uses: `def:weil-a4-a3-proper-piece-comparison,def:weil-a1-refinement-underived-map`

The comparison $\widetilde{d}_{f,K}$ respects identity and composition of proper maps. For a compactification refinement $q:\overline{X}_D\to\overline{X}_C$ over Y, its square with the degreewise refinement map $q_*j_{D!}K\to j_{C!}K$ commutes in D^+.

Proof. Uses: `lemma:weil-pbc-epsilon-direct-image-naturality,lemma:weil-pbc-derived-exchange-coherence,def:weil-a1-refinement-underived-map,lemma:weil-a4-open-extension-site-comparison-iso`

For identity and composition, expand $d_{p,J}$ as the unit of the étale/pro-étale comparison followed by direct image. A3 naturality and composition coherence identify the two expansions term by term. For a refinement, evaluate both paths on a local section represented by a factorization through X. Each path keeps its module value unchanged, changes its index by the same map to the fiber product defining q, and then applies the same direct image. They are therefore equal before sheafification. The site-comparison isomorphism and A3 replacement independence transport this equality to D^+. □

Proposition 11.12.19 (Finite-origin bounded-below cohomologicality). *Lean:* `conditional-interface:prop:weil-a4-finite-origin-compact-support-cohomological`

Uses: `def:weil-a4-finite-origin-compactification-input,def:weil-a4-a3-proper-piece-comparison`

On the full subcategory of bounded-below finite-origin objects, the functor

$$K\longmapsto Rp_{f,\mathrm{pro\acute{e}t},*}j_{f,!}K$$

sends shifts and distinguished triangles to shifts and distinguished triangles. Consequently, for every $L_Y\in D^+(Y,\Lambda_n)$, the functor

$$K\longmapsto \mathrm{Hom}_{D^+(Y,\Lambda_n)}\big(Rp_{f,*}j_{f,!}K,L_Y\big)$$

is contravariantly cohomological on this subcategory.

Proof. Uses: `lemma:weil-a4-open-extension-amplitude-zero,lemma:weil-pbc-derived-exchange-replacement,lemma:weil-pbc-derived-exchange-coherence,prop:weil-a2-derived-category-triangulated`

Every distinguished triangle is isomorphic to the mapping-cone triangle of a map $u:K'\to K$. The amplitude-zero lemma makes termwise $j_{f,!}$ exact, so it identifies $j_{f,!}\,\mathrm{Cone}(u)$ with $\mathrm{Cone}(j_{f,!}u)$, including the differential signs. Choose bounded-below injective resolutions I' and I of the first two open extensions and lift $j_{f,!}u$ to a chain map $\widetilde{u}:I'\to I$. The complex $\mathrm{Cone}(\widetilde{u})$ is bounded below and each term is a finite direct sum of injectives, hence injective. It therefore computes the derived image of the third object. Since $p_{f,*}$ is additive and acts termwise,

$$p_{f,*}\,\mathrm{Cone}(\widetilde{u})=\mathrm{Cone}(p_{f,*}\widetilde{u})$$

with the same signed differential. Thus the image triangle is a mapping-cone triangle. A3 replacement independence shows that changing the resolutions or the lift gives the same derived triangle; its coherence gives shift and rotation compatibility. Finally, apply the triangulated-category Hom exactness axiom to the resulting mapping-cone triangle; this gives the displayed contravariant long exact sequence. □

Proposition 11.12.20 (Finite-origin compact-support proper base change). *Lean:* `conditional-interface:prop:weil-a4-finite-origin-compact-support-base-change`

Uses: `def:weil-a4-finite-origin-compactification-input,def:weil-a4-a3-proper-piece-comparison`

For every Cartesian base change of the selected compactification, the exchange map for $Rp_{f,*}j_{f,!}K$ is an isomorphism on finite-origin bounded-below inputs. It is compatible with identity, pasting, and the refinement square of the preceding lemma.

Proof. Uses: prop:weil-proet-finite-derived-proper-base-change,prop:weil-pbc-proet-finite-level-transport,lemma:weil-a4-a3-proper-piece-coherence,lemma:weil-a4-open-extension-site-comparison-iso

Replace $j_{f,!}K$ by $\epsilon_{\overline{X}_f}^{-1} J$ using the open-extension comparison. A3 finite derived proper base change makes the small-site exchange map an isomorphism, and A3 finite-level transport identifies its inverse image with the pro-étale exchange map. Identity and pasting are the A3 coherence equations. The refinement compatibility is the preceding refinement square applied before and after base change. □

Proposition 11.12.21 (Conditional finite-origin compact-support amplitude). Lean: conditional-interface:prop:weil-a4-finite-origin-compact-support-amplitude

Uses: def:weil-a4-finite-artin-constructibility-predicate, def:weil-a4-a3-proper-piece-comparison

Assume $\mathrm{ArtFin}(p_f, \Lambda_n; d)$. *For an input in the interval* $[a, b]$,

$$Rp_{f,\mathrm{pro\acute{e}t},*} j_{f,!} K \in D_c^{[a,b+2d]}(Y_{\mathrm{pro\acute{e}t}}, \Lambda_n).$$

The resulting bounded constructible object is naturally the same complex as the A1 selected compactification candidate.

Proof. Uses: lemma:weil-a4-artin-constructibility-amplitude,lemma:weil-a4-a3-proper-piece-comparison-iso,lemma:weil-pbc-epsilon-direct-image-constructible-bounds,def:weil-a1-rfshriek-selected,def:weil-a1-compactification-candidate

The Artin-amplitude lemma places $Rp_{f,\acute{e}t,*}J$ in the displayed interval with constructible cohomology. Exact inverse image along ϵ_Y preserves the interval and the finite stratification. The A3 proper-piece isomorphism identifies this inverse image with $Rp_{f,\mathrm{pro\acute{e}t},*} j_{f,!} K$. Finally, expand the A1 selected candidate and its compactification candidate: on the selected compactification it is definitionally the same ordered composite $Rp_{f,*} j_{f,!}$, on the same representatives and arrows. □

Definition 11.12.22 (Selected finite Artin-bound candidate). Lean: conditional-interface:def:weil-a4-selected-finite-artin-bound-candidate

Uses: def:weil-a4-finite-artin-constructibility-predicate,def:weil-a4-proper-relative-dimension-certificate

Let $S_f \subseteq \mathbf{N}$ consist of the integers d for which the fiber-dimension inequalities and $\mathrm{ArtFin}(p_f, \Lambda_n; d)$ both hold. Define

$$d_f^{\mathrm{fin}} := \begin{cases} \min S_f, & S_f \neq \varnothing, \\ 0, & S_f = \varnothing. \end{cases}$$

This is a total definition and does not assert that S_f is inhabited.

Definition 11.12.23 (Total-dimension relative-bound candidate). Lean: conditional-interface:def:weil-a4-total-dimension-relative-bound

Uses: def:weil-a4-exceptional-index

Put

$$D_f := \dim(\overline{X}_f).$$

The dimension is the Krull dimension of the noetherian finite-dimensional scheme $\overline{X}_f$. The candidate consists of the integer D_f and the intended inequalities

$$\dim(\overline{X}_f \times_Y \bar{y}) \leq D_f$$

for all geometric points $\bar{y} \to Y$.

Lemma 11.12.24 (Total dimension bounds every geometric fiber). Lean: conditional-interface:lemma:weil-a4-total-dimension-relative-bound

Uses: def:weil-a4-total-dimension-relative-bound, def:weil-a4-proper-relative-dimension-certificate

The integer D_f *with the displayed inequalities is a relative-dimension certificate for* p_f.

Proof. Uses: def:weil-a4-total-dimension-relative-bound,def:weil-a4-proper-relative-dimension-certificate

Fix $\bar{y} = \operatorname{Spec}\Omega \to Y$, with image $y \in Y$. The ordinary fiber $\overline{X}_{f,y}$ is a locally closed subscheme of $\overline{X}_f$: it is obtained by first restricting to the inverse image of $\overline{\{y\}}$ and then taking the open fiber over y. Every strict chain of irreducible closed subsets of $\overline{X}_{f,y}$ is therefore a strict chain of irreducible locally closed subsets of $\overline{X}_f$, so

$$\dim(\overline{X}_{f,y}) \leq \dim(\overline{X}_f) = D_f.$$

The fiber is of finite type over $\kappa(y)$. For a finite-type scheme over a field, extension of the ground field preserves Krull dimension: after passing to affine charts this is the equality $\dim(A \otimes_{\kappa(y)} \Omega) = \dim A$, proved by contracting and extending chains of prime ideals and applying the dimension formula for finitely generated algebras over fields. Hence

$$\dim(\overline{X}_f \times_Y \bar{y}) = \dim(\overline{X}_{f,y}) \le D_f.$$

This is exactly the family of inequalities required by the certificate definition. □

Definition 11.12.25 (Affine Artin-vanishing certificate)**.** Lean: `conditional-interface:def:weil-a4-affine-artin-vanishing-certificate`

Uses: `def:weil-a4-exceptional-index`

An affine Artin-vanishing certificate at Λ_n assigns to every separably closed field k of characteristic different from ℓ, every affine finite-type k-scheme U of dimension at most e, every constructible sheaf G of finite Λ_n-modules on $U_{\acute{e}t}$, and every integer $r > e$, an equality

$$H^r(U_{\acute{e}t}, G) = 0.$$

The assignments include compatibility with restriction to an affine open, finite filtrations of G, extension between separably closed base fields, and pullback by an automorphism of k. When $k = \overline{\mathbf{F}}_q$, the last compatibility applies in particular to geometric Frobenius.

Definition 11.12.26 (Field torsion-cohomological-dimension predicate)**.** Lean: `conditional-interface:def:weil-a4-field-torsion-cd-predicate`

Uses: `def:weil-a4-exceptional-index`

For a field K and $c \in \mathbf{N}$, let FieldCD$(K; c)$ mean that for every torsion abelian sheaf M on $(\operatorname{Spec} K)_{\acute{e}t}$,

$$H^r(\operatorname{Spec} K, M) = 0 \qquad (r > c).$$

The predicate includes the same equality after replacing M by a torsion Λ_n-module sheaf.

Definition 11.12.27 (Strengthened affine cohomological-dimension predicate)**.** Lean: `conditional-interface:def:weil-a4-strong-affine-cd-predicate`

Uses: `def:weil-a4-field-torsion-cd-predicate`

For a field K, integers $c, d \ge 0$, define AffCD$(K; c, d)$ to mean:

> if FieldCD$(K; c)$, X is affine of finite type over K with $\dim X \le d$, and F is any torsion abelian sheaf on $X_{\acute{e}t}$, then
>
> $$H^r(X, F) = 0 \qquad (r > d + c).$$

The quantification over arbitrary torsion sheaves is intentional. In the induction, the sheaves $R^q f_* F$ need not yet be known constructible.

Definition 11.12.28 (Ordinary and compact-support affine complexes)**.** Lean: `conditional-interface:def:weil-a4-affine-ordinary-compact-support`

Uses: `def:weil-a4-exceptional-index`

Let $u : U \hookrightarrow \overline{U}$ be an open immersion with $\overline{U}$ proper over k, let $z : Z \hookrightarrow \overline{U}$ be the reduced closed complement, and let F be a torsion sheaf on U. Define

$$R\Gamma(U, F) := R\Gamma(\overline{U}, Ru_* F), \qquad R\Gamma_c(U, F) := R\Gamma(\overline{U}, u_! F).$$

Define the comparison $R\Gamma_c(U, F) \to R\Gamma(U, F)$ by the sheaf map $u_! F \to Ru_* F$. For a sheaf $\overline{F}$ on $\overline{U}$, define the open–closed localization triangle to be

$$u_! u^* \overline{F} \longrightarrow \overline{F} \longrightarrow z_* z^* \overline{F} \longrightarrow .$$

Lemma 11.12.29 (Ordinary and compact-support bounds are distinct)**.** *Lean:* `conditional-interface:lemma:weil-a4-affine-ordinary-compact-support-distinction`

Uses: `def:weil-a4-affine-ordinary-compact-support`

The affine Artin assertion concerns $R\Gamma(U, F)$, not $R\Gamma_c(U, F)$. The localization triangle yields the functorial long exact sequence

$$\cdots \to H^r_c(U, u^* \overline{F}) \to H^r(\overline{U}, \overline{F}) \to H^r(Z, z^* \overline{F}) \to H^{r+1}_c(U, u^* \overline{F}) \to \cdots .$$

No vanishing $H^r_c(U, F) = 0$ for $r > \dim U$ is asserted.

Proof. Uses: def:weil-a4-affine-ordinary-compact-support

On an etale object over $\overline{U}$, the first map is extension by zero followed by inclusion and the second is restriction to Z. At a geometric point of U the stalk sequence is

$$F_{\bar{u}} \xrightarrow{\mathrm{id}} F_{\bar{u}} \longrightarrow 0,$$

and at a geometric point of Z it is

$$0 \longrightarrow \overline{F}_{\bar{z}} \xrightarrow{\mathrm{id}} \overline{F}_{\bar{z}}.$$

Thus the displayed three sheaves form a short exact sequence. Applying an injective resolution of $\overline{F}$ and global sections gives the stated connecting maps and long exact sequence. By definition, the first complex is compactly supported and the middle one is cohomology of the proper compactification. The ordinary affine complex instead uses Ru_*F. Hence the two bounds are different statements. □

Lemma 11.12.30 (A separably closed field has torsion cd zero)**.** Lean: conditional-interface:lemma:weil-a4-sep closed-field-cd-zero

Uses: def:weil-a4-field-torsion-cd-predicate

If k is separably closed, then FieldCD$(k;0)$.

Proof. Uses: def:weil-a4-field-torsion-cd-predicate

Every connected finite etale k-algebra is a finite separable field extension of k, hence is k itself. Every finite etale cover of $\operatorname{Spec} k$ therefore refines to a finite disjoint union of identity maps. Evaluation at the unique geometric point identifies the category of etale sheaves with the category of sets, and abelian sheaves with abelian groups. Under this equivalence, global sections is the identity functor and is exact. Its positive right-derived functors vanish, which is the predicate with $c = 0$. □

Definition 11.12.31 (Noether-normalization datum)**.** Lean: conditional-interface:def:weil-a4-noether-normalization-datum

Uses: def:weil-a4-strong-affine-cd-predicate

Let A be a finitely generated K-algebra of Krull dimension d. A Noether-normalization datum consists of elements $y_1, \dots, y_d \in A$, an injective map

$$\iota : K[T_1, \dots, T_d] \longrightarrow A, \qquad T_i \longmapsto y_i,$$

and a finite list $a_1, \dots, a_N \in A$ which generates A as a module over the image of ι. The injectivity proof and every module expression for products of the generators are fields of the datum.

Lemma 11.12.32 (Triangular substitution makes a relation monic)**.** Lean: conditional-interface:lemma:weil-a4-noether-triangular-monic

Uses: def:weil-a4-noether-normalization-datum

Let $0 \neq P \in K[X_1, \dots, X_m]$. There are positive integers $N_1 < \cdots < N_{m-1}$ such that after the automorphism

$$X_i \longmapsto X_i + X_m^{N_i} \quad (i < m), \qquad X_m \longmapsto X_m,$$

the polynomial P has a unique highest power of X_m, with nonzero coefficient in K. After multiplying by its inverse, it is monic in X_m.

Proof. Uses: def:weil-a4-noether-normalization-datum

Write the finite support of P as exponent vectors $\alpha = (\alpha_1, \dots, \alpha_m)$. Choose an integer $B > 1 + \max_{\alpha,i} \alpha_i$ and put $N_i = B^i$. Then the integer

$$w(\alpha) = \alpha_m + \sum_{i<m} \alpha_i N_i$$

is different for distinct exponent vectors: it is the base-B numeral whose successive digits are $\alpha_m, \alpha_1, \dots, \alpha_{m-1}$, and every digit is less than B. In the substituted monomial X^α, the largest X_m-power is $X_m^{w(\alpha)}$ and has coefficient 1. The exponent vector with maximal w is unique, so its original nonzero coefficient is the nonzero leading coefficient of the substituted polynomial. Scaling by that coefficient gives a monic polynomial. □

Lemma 11.12.33 (A monic relation gives finite module generation)**.** Lean: conditional-interface:lemma:weil-a4-noether-monic-finite

Uses: def:weil-a4-noether-normalization-datum

Let A be generated over K by $x_1, \dots, x_m$. If x_m satisfies a monic polynomial of degree s over the subalgebra $B = K[x_1, \dots, x_{m-1}]$, then

$$A = \sum_{r=0}^{s-1} B x_m^r.$$

In particular A is a finite B-module and is integral over B.

Proof. Uses: `def:weil-a4-noether-normalization-datum`

Write the relation as

$$x_m^s + b_{s-1}x_m^{s-1} + \cdots + b_0 = 0, \qquad b_i \in B.$$

It rewrites x_m^s as a B-linear combination of lower powers. Multiplying the equality by x_m^t and inducting on t rewrites every x_m^{s+t} as a combination of $1, x_m, \ldots, x_m^{s-1}$. Every polynomial in the generators therefore has the displayed form. The same monic relation is the definition that x_m is integral over B, and a finite algebra generated by integral elements is integral. □

Proposition 11.12.34 (Existence of Noether-normalization data). Lean: `conditional-interface:prop:weil-a4-noether-normalization-datum`

Uses: `def:weil-a4-noether-normalization-datum`

Every finitely generated K-algebra A of finite Krull dimension d has a Noether-normalization datum.

Proof. Uses: `lemma:weil-a4-noether-triangular-monic,lemma:weil-a4-noether-monic-finite`

Choose a surjection

$$K[X_1, \ldots, X_m] \twoheadrightarrow A$$

with m minimal among the current induction stage. If its kernel is zero, then A is a polynomial ring, $m = d$, and the images of the variables give the datum with the single module generator 1.

If the kernel is nonzero, choose $0 \neq P$ in it. Apply the triangular automorphism from the preceding lemma and rename the transformed generators. The relation is monic in X_m; hence its image makes A finite over the subalgebra B generated by the first $m-1$ images. Integral extensions preserve Krull dimension: contraction sends a strict chain of primes of A to a chain in B, incomparability keeps it strict, and going-up lifts every chain in B. Thus $\dim B = \dim A = d$.

Apply induction on m to the finitely generated algebra B. It is finite over a polynomial subalgebra $K[y_1, \ldots, y_d]$. If (b_i) generate B over that subalgebra and $(1, x_m, \ldots, x_m^{s-1})$ generate A over B, then the finite list

$$\{\, b_i x_m^r : 1 \le i \le t,\ 0 \le r < s \,\}$$

generates A over $K[y_1, \ldots, y_d]$. The inclusion of that polynomial algebra in $B \subseteq A$ is injective by the induction datum. These elements and the multiplication formulas obtained by the two finite reductions form the required datum. □

Lemma 11.12.35 (Filtered colimits of torsion sheaves commute with cohomology). Lean: `conditional-interface:lemma:weil-a4-torsion-filtered-cohomology`

Uses: `def:weil-a4-strong-affine-cd-predicate`

Let X be a noetherian scheme and let (F_λ) be a filtered system of torsion abelian sheaves on $X_{\text{ét}}$. For every q,

$$\varinjlim_\lambda H^q(X, F_\lambda) \xrightarrow{\sim} H^q(X, \varinjlim_\lambda F_\lambda).$$

The isomorphism is natural in the system and in pullback.

Proof. Uses: `lemma:weil-pbc-finite-hypercover-descent`

Represent etale cohomology by the filtered colimit of cochain cohomology over split etale hypercovers. A class in degree q uses only finitely many sections and the face equations through degree $q+1$. Noetherian quasi-compactness refines the relevant hypercover skeleton so that it has finitely many quasi-compact components. Every section of $\varinjlim F_\lambda$ on those finitely many components is represented at one common index, and the finitely many cocycle equations hold at one later common index. This proves surjectivity.

If a class represented at λ becomes zero, choose a finite refinement and a degree-$(q-1)$ cochain whose coboundary is the cocycle. The refinement maps, its finitely many sections, and its finitely many coboundary equations occur at one $\mu \ge \lambda$. The class is already zero at μ, proving injectivity. Pullback carries the selected finite skeleton, sections, and equations to their pullbacks, which proves naturality. □

Lemma 11.12.36 (Cohomological dimension under affine inverse limits). Lean: `conditional-interface:lemma:weil-a4-affine-limit-cd`

Uses: `def:weil-a4-field-torsion-cd-predicate`

Let $X = \varprojlim_i X_i$ be a cofiltered inverse limit of qcqs schemes with affine transition maps. If every X_i has torsion cohomological dimension at most N, then X has torsion cohomological dimension at most N.

Proof. Uses: `lemma:weil-pbc-etale-cohomology-continuity,lemma:weil-a4-torsion-filtered-cohomology`

For the projection $p_i : X \to X_i$ and a torsion sheaf F on X, the adjunction maps form a filtered system

$$p_i^{-1}p_{i,*}F \longrightarrow F.$$

It is an isomorphism after taking the filtered colimit: a section and an equality of two sections on a finitely presented etale object descend to a finite stage, so the stalk maps are respectively surjective and injective. Thus

$$F = \varinjlim_i p_i^{-1} p_{i,*} F.$$

Filtered-cohomology continuity gives

$$H^q(X, F) = \varinjlim_i H^q(X, p_i^{-1} p_{i,*} F).$$

For a fixed i, the inverse-limit continuity theorem identifies the right-hand group with the filtered colimit of the cohomology groups of the pullback of $p_{i,*}F$ at later stages. These vanish for $q > N$ by the hypothesis on the stages. Both filtered colimits are therefore zero in degree $q > N$. □

Lemma 11.12.37 (Finite morphisms do not increase torsion cd)**.** *Lean:* `conditional-interface:lemma:weil-a4-finite-morphism-cd`

Uses: `def:weil-a4-strong-affine-cd-predicate`

Let $v : X \to Y$ be finite and F any torsion abelian sheaf on $X_{\acute{e}t}$. Then v_* is exact,

$$R^r v_* F = 0 \quad (r > 0), \qquad H^q(X, F) \xrightarrow{\sim} H^q(Y, v_* F).$$

Proof. Uses: `prop:weil-pbc-strict-henselization-stalk-formula,lemma:weil-pbc-geometric-stalk-conservative`

Fix a geometric point $\bar{y} \to Y$. After base change to $\operatorname{Spec} \mathcal{O}^{\mathrm{sh}}_{Y,\bar{y}}$, the finite scheme splits as a finite disjoint union of spectra of strictly henselian local rings: idempotents lift from the finite closed fiber, and each local factor is henselian with separably closed residue field. The etale topos of each factor has exact global sections, because every finite etale cover splits. Hence the stalk of $R^r v_* F$, computed by the strict-henselization stalk formula, is zero for $r > 0$. Conservativity of geometric stalks gives $R^r v_* F = 0$.

The Leray spectral sequence

$$H^p(Y, R^q v_* F) \Longrightarrow H^{p+q}(X, F)$$

has only the row $q = 0$. Its edge maps are the displayed isomorphisms. Exactness of v_* is the degree-zero form of the same stalk calculation: on every strict local fiber it is a finite product of exact section functors. □

Lemma 11.12.38 (Affine cd in dimension zero)**.** *Lean:* `conditional-interface:lemma:weil-a4-affine-cd-zero`

Uses: `def:weil-a4-strong-affine-cd-predicate`

For every field K and c, $\mathrm{AffCD}(K; c, 0)$ holds.

Proof. Uses: `lemma:weil-a4-finite-morphism-cd,def:weil-a4-strong-affine-cd-predicate,def:weil-a4-field-torsion-cd-predicate`

Let X be affine of finite type over K and dimension zero. Its coordinate ring is an artinian finitely generated K-algebra, hence is finite-dimensional over K: each residue field is a finite extension and the nilradical has a finite filtration by finite-dimensional quotients. Thus $a : X \to \operatorname{Spec} K$ is finite. The finite-morphism lemma identifies

$$H^r(X, F) = H^r(\operatorname{Spec} K, a_* F).$$

The right side is zero for $r > c$ by $\mathrm{FieldCD}(K; c)$, which is the required $d = 0$ bound. □

Lemma 11.12.39 (Affine curves over a separably closed field)**.** *Lean:* `conditional-interface:lemma:weil-a4-affine-curve-sep closed`

Uses: `def:weil-a4-strong-affine-cd-predicate`

Let k be separably closed, let C be affine of finite type over k with $\dim C \le 1$, and let F be a torsion sheaf. Then

$$H^q(C, F) = 0 \qquad (q > 1).$$

The zero identifications commute with extension between separably closed fields and with automorphisms of k.

Proof. Uses: `lemma:weil-pbc-universal-homeomorphism-coverings,def:weil-pbc-eight-curve-assertions,lemma:weil-pbc-eight-curve-devissage,lemma:weil-pbc-curve-constructible-primary-reduction,lemma:weil-pbc-curve-dense-open-reduction,lemma:weil-pbc-curve-locally-constant-reduction,lemma:weil-pbc-finite-pushforward-extension-zero,lemma:weil-pbc-curve-normalization-constant`

Choose an algebraic closure $\bar{k}$ of k. Since k is separably closed, $\bar{k}/k$ is purely inseparable. The projection $C_{\bar{k}} \to C$ is a universal homeomorphism, and the A3 explicit site equivalence identifies F with its pullback and identifies every cohomology group.

The A3 eight-assertion curve proof, including constructible approximation, primary devissage, normalization, and the Kummer/Artin–Schreier calculation, gives

$$H^q(C_{\bar{k}}, F|) = 0 \qquad (q > 1)$$

for the affine curve over the algebraically closed field. Transporting the zero group through the site equivalence proves the assertion over k. All maps in the construction are pullbacks of etale cocycles and the site equivalence is compatible with identity and composition. Therefore field extension and automorphism pullback commute with the zero identifications. □

Lemma 11.12.40 (Affine cd in dimension one). *Lean:* `conditional-interface:lemma:weil-a4-affine-cd-one`

Uses: `def:weil-a4-strong-affine-cd-predicate`

For every field K and c, $\mathrm{AffCD}(K; c, 1)$ *holds.*

Proof. Uses: `prop:weil-pbc-strict-henselization-stalk-formula,lemma:weil-a4-affine-curve-sepclosed,def:weil-a4-strong-affine-cd-predicate,def:weil-a4-field-torsion-cd-predicate`

Let $a : C \to \operatorname{Spec} K$ be affine of finite type with $\dim C \leq 1$. The strict-henselization stalk formula at a geometric point $\operatorname{Spec} K^{\mathrm{sep}} \to \operatorname{Spec} K$ identifies

$$(R^q a_* F)_{\bar{\eta}} = H^q(C_{K^{\mathrm{sep}}}, F|).$$

The affine-curve lemma kills this group for $q > 1$. Since a sheaf on the etale topos of a field is determined by its geometric stalk, $R^q a_* F = 0$ for $q > 1$.

In the Leray spectral sequence

$$E_2^{p,q} = H^p(\operatorname{Spec} K, R^q a_* F) \Longrightarrow H^{p+q}(C, F),$$

only $q = 0, 1$ occur. The field-cd predicate kills every term with $p > c$. Thus $p + q > c + 1$ implies $E_2^{p,q} = 0$, and the finite two-row filtration makes $H^r(C, F) = 0$ for $r > c + 1$. □

Lemma 11.12.41 (Transcendence degree raises field cd by at most its degree). *Lean:* `conditional-interface:lemma:weil-a4-field-transcendence-cd`

Uses: `def:weil-a4-field-torsion-cd-predicate`

Assume $\mathrm{FieldCD}(K; c)$ *and let L/K have finite transcendence degree s. Then* $\mathrm{FieldCD}(L; c + s)$.

Proof. Uses: `lemma:weil-a4-affine-cd-one,lemma:weil-a4-affine-limit-cd`

It is enough to treat $s = 1$ and then iterate along a transcendence basis. Write L as the filtered union of its finitely generated K-subalgebras A_i. After replacing A_i by its image in the field L, it is a domain of dimension at most one. Its spectrum is an affine finite-type K-scheme of dimension at most one, so the dimension-one lemma gives torsion cd at most $c + 1$. The maps $\operatorname{Spec} L \to \operatorname{Spec} A_i$ exhibit the former as the inverse limit with affine transition maps. The affine-limit lemma gives $\mathrm{FieldCD}(L; c+1)$. Repeating for a tower of s transcendence-degree-one extensions gives $c + s$. □

Definition 11.12.42 (Punctured strict-local dimension-drop datum). *Lean:* `conditional-interface:def:weil-a4-punctured-strict-local-datum`

Uses: `def:weil-a4-strong-affine-cd-predicate`

Let Y be a smooth d-dimensional finite-type K-scheme, let $D \subset Y$ be an effective Cartier divisor, let $y \in D$, put

$$a = \operatorname{trdeg}_K \kappa(y),$$

and choose a geometric point $\bar{y}$ and a local equation $f \in \mathfrak{m}_y$ for D. A punctured strict-local dimension-drop datum consists of:

1. a field $L = K(t_1, \ldots, t_a)^{\mathrm{sep}}$, a map $L \to A := \mathcal{O}^{\mathrm{sh}}_{Y,\bar{y}}$, and a purely inseparable residue-field extension $L \to \kappa(A)$;
2. an injective map
$$L(x) = L[x]_{(x)}[1/x] \longrightarrow A[1/f], \qquad x \longmapsto f;$$
3. a filtered system of finite-type $L(x)$-algebras B_λ, each of dimension at most $d - a - 1$, with affine transition maps and an isomorphism
$$A[1/f] \xrightarrow{\sim} \varinjlim_\lambda B_\lambda;$$
4. the inverse-limit scheme isomorphism, the pullback maps for torsion sheaves, and the cohomology-continuity isomorphisms in every degree.

Lemma 11.12.43 (Strict-local coefficient-field coordinates). *Lean:* `conditional-interface:lemma:weil-a4-strict-local-coefficient-field`

Uses: `def:weil-a4-punctured-strict-local-datum, def:weil-a4-noether-normalization-datum`

In the setup of the preceding definition there is a map

$$K(t_1,\dots,t_a)^{\mathrm{sep}} \longrightarrow \mathcal{O}^{\mathrm{sh}}_{Y,\bar{y}}$$

whose residue-field extension is purely inseparable.

Proof. Uses: `prop:weil-a4-noether-normalization-datum,def:weil-pbc-strict-henselization-limit`

Shrink Y to an affine neighbourhood of y. Apply the Noether-normalization proposition to its coordinate algebra and localize the resulting finite map around y. If z is the image of y in $\mathbf{A}^d_K$, then $\kappa(y)/\kappa(z)$ is finite, so $\operatorname{trdeg}_K \kappa(z) = a$.

Apply the same normalization construction to the domain $K[X_1,\dots,X_d]/\mathfrak{p}_z$. It selects a algebraically independent residue functions $\bar{t}_1,\dots,\bar{t}_a$ over which $\kappa(z)$ is algebraic. Lift these functions to polynomials t_i on $\mathbf{A}^d_K$. The triangular monic construction applied successively to the remaining residue functions produces a finite polynomial endomorphism of $\mathbf{A}^d_K$ which sends z to the generic point η of the coordinate subspace $\mathbf{A}^a_K$. After replacing the original finite map by its composite with this endomorphism, projection to the first a coordinates maps y to η.

The strict henselization at a geometric point over η is the separable closure

$$\mathcal{O}^{\mathrm{sh}}_{\mathbf{A}^a_K,\bar{\eta}} = K(t_1,\dots,t_a)^{\mathrm{sep}} = L.$$

Functoriality of the pointed etale-neighbourhood colimit gives $L \to \mathcal{O}^{\mathrm{sh}}_{Y,\bar{y}}$. The residue field of the target is algebraic over the separably closed field L. Its maximal separable subextension is therefore L, so the residue extension is purely inseparable. □

Lemma 11.12.44 (Finite-type stages of a strict henselization). *Lean:* `conditional-interface:lemma:weil-a4-strict-local-finite-type-stages`

Uses: `def:weil-a4-punctured-strict-local-datum`

With $L \to A = \mathcal{O}^{\mathrm{sh}}_{Y,\bar{y}}$ as above, A is a filtered colimit of finite-type L-algebras C_λ of dimension at most $d-a$.

Proof. Uses: `def:weil-pbc-strict-henselization-limit,lemma:weil-pbc-finite-hypercover-descent`

Write the affine neighbourhood as $\operatorname{Spec} R$. The strict henselization is the filtered colimit of pointed etale R-algebras R_j. The ring $L = K(t_1,\dots,t_a)^{\mathrm{sep}}$ is itself the filtered colimit of finite etale algebras A_i over localizations of $K[t_1,\dots,t_a]$. A map from the finitely presented algebra A_i to the colimit A factors through some R_j.

Index the resulting factorizations by triples $(i,j,\psi : A_i \to R_j)$. Given two triples, filteredness of the pointed neighbourhoods and finite presentation of both source algebras give a third triple receiving both; equality of the two induced maps holds after one further stage because it is a finite list of polynomial equalities. Therefore the triples form a filtered category and

$$A = \varinjlim_{(i,j,\psi)} (R_j \otimes_{A_i} L)\,.$$

The chosen finite map to $\mathbf{A}^d_K$ makes R_j quasi-finite over a localization of $K[t_1,\dots,t_d]$. After tensoring the first a coordinates with L, the algebra $C_{i,j} = R_j \otimes_{A_i} L$ is quasi-finite over

$$L[t_{a+1},\dots,t_d].$$

A chain of primes in a quasi-finite algebra contracts to a strict chain in the polynomial algebra, so its length is at most $d-a$. Thus every $C_{i,j}$ has dimension at most $d-a$, as required. □

Lemma 11.12.45 (Inverting the divisor drops the relative dimension). *Lean:* `conditional-interface:lemma:weil-a4-strict-local-puncture-dimension-drop`

Uses: `def:weil-a4-punctured-strict-local-datum`

The map $L \to A$ extends to an injection

$$L(x) \longrightarrow A[1/f], \qquad x \longmapsto f,$$

and $A[1/f]$ is a filtered colimit of finite-type $L(x)$-algebras of dimension at most $d-a-1$.

Proof. Uses: `lemma:weil-a4-strict-local-finite-type-stages,lemma:weil-pbc-etale-cohomology-continuity`

The smooth scheme Y is regular. Hence the local ring at y, its strict henselization A, and every localization of A are domains on the connected component selected by $\bar{y}$. The Cartier equation f is nonzero and lies in

the maximal ideal. If a nonzero polynomial $\sum_{i=0}^{n} c_i f^i$ with $c_i \in L$ vanished, choose its least nonzero exponent and divide by that power of f. Reduction modulo the maximal ideal would give a nonzero constant in the residue field equal to zero. Thus f is transcendental over L, giving the injection $L[x]_{(x)} \to A$, and inversion gives $L(x) \to A[1/f]$.

Choose the finite-type stages C_λ so that f is represented by an element f_λ. Base change along $L[x]_{(x)} \to L(x)$ gives

$$B_\lambda = C_\lambda \otimes_{L[x]_{(x)}} L(x), \qquad A[1/f] = \varinjlim_{\lambda} B_\lambda.$$

An irreducible component of B_λ comes from an irreducible component of C_λ which dominates $\operatorname{Spec} L[x]_{(x)}$; the other components disappear after tensoring with $L(x)$. If Q is the corresponding minimal prime, the finite-type dimension formula gives

$$\begin{aligned} \dim(C_\lambda/Q) &= \operatorname{trdeg}_L \operatorname{Frac}(C_\lambda/Q), \\ \dim\big((C_\lambda/Q) \otimes_{L[x]_{(x)}} L(x)\big) &= \operatorname{trdeg}_{L(x)} \operatorname{Frac}(C_\lambda/Q) \\ &= \dim(C_\lambda/Q) - 1. \end{aligned}$$

The last equality uses the algebraic independence of x over L. Since every component of C_λ has dimension at most $d-a$, every component of B_λ has dimension at most $d-a-1$.

The affine scheme identity reverses the colimit into an inverse limit with affine transition maps. The A3 cohomology-continuity isomorphism supplies the final cohomology field of the datum for every torsion sheaf descending to a stage; arbitrary torsion sheaves are handled by filtered sheaf continuity. □

Proposition 11.12.46 (Existence of punctured strict-local dimension-drop data). *Lean:* `conditional-interface:prop:weil-a4-punctured-strict-local-datum`

Uses: `def:weil-a4-punctured-strict-local-datum`

Every smooth pair (Y, D), point $y \in D$, geometric lift $\bar{y}$, and Cartier equation f as in the definition has a punctured strict-local dimension-drop datum.

Proof. Uses: `lemma:weil-a4-strict-local-coefficient-field`, `lemma:weil-a4-strict-local-finite-type-stages`, `lemma:weil-a4-strict-local-puncture-dimension-drop`, `lemma:weil-pbc-etale-cohomology-continuity`

Take the field and coefficient map from the coefficient-field lemma. Take the filtered finite-type stages from the second lemma and invert the Cartier equation by the third. The third lemma gives the injection, the dimension bound, and the colimit identity. Passing to spectra gives the inverse-limit identity. Pullback of sheaves is functorial for its transition maps, and A3 cohomology continuity gives the specified cohomology isomorphisms. These are exactly the four fields of the datum. □

Definition 11.12.47 (Transcendence-degree support filtration). Lean: `conditional-interface:def:weil-a4-transcendence-support-set`

Uses: `def:weil-a4-strong-affine-cd-predicate`

Let Z be a finite-type scheme over a field K. For an integer a, define

$$E_a(Z/K) := \{z \in Z : \operatorname{trdeg}_K \kappa(z) \le a\}.$$

For $a < 0$, this definition gives $E_a(Z/K) = \varnothing$.

Lemma 11.12.48 (The transcendence support sets are specialization closed). *Lean:* `conditional-interface:lemma:weil-a4-transcendence-support-specialization`

Uses: `def:weil-a4-transcendence-support-set`

For every integer a, the subset $E_a(Z/K)$ is closed under specialization.

Proof. Uses: `def:weil-a4-transcendence-support-set`

It is enough to treat an immediate specialization $z' \in \overline{\{z\}}$. Choose an affine neighbourhood $\operatorname{Spec} A$ of z' meeting z, and let $\mathfrak{p} \subseteq \mathfrak{q}$ be the corresponding prime ideals. The quotient map $A/\mathfrak{p} \twoheadrightarrow A/\mathfrak{q}$ shows that a family in $A/\mathfrak{q}$ algebraically independent over K lifts to a family in $A/\mathfrak{p}$ algebraically independent over K: a polynomial relation among the lifts would descend to one among their images. Passing to fraction fields gives

$$\operatorname{trdeg}_K \kappa(z') \le \operatorname{trdeg}_K \kappa(z).$$

Thus membership of z in E_a implies membership of z'. □

Definition 11.12.49 (Support-approximation datum)**.** Lean: `conditional-interface:def:weil-a4-support-approximation-datum`

Uses: `def:weil-a4-transcendence-support-set`

Let Z be noetherian, let $E \subseteq Z$ be specialization closed, and let G be a torsion abelian sheaf supported on E. A support-approximation datum consists of a filtered category I, constructible torsion subsheaves $G_i \subseteq G$, and reduced closed subschemes $i_i : Z_i \hookrightarrow Z$, such that

$$G = \varinjlim_{i \in I} G_i, \qquad \mathrm{Supp}(G_i) \subseteq |Z_i| \subseteq E.$$

It also contains the transition inclusions, the colimit cocone, and the two displayed support inclusions.

Definition 11.12.50 (Canonical finite-generator image system)**.** Lean: `conditional-interface:def:weil-a4-finite-generator-image-system`

Uses: `def:weil-a4-support-approximation-datum`

Let Z be noetherian and G a torsion abelian sheaf. An index α is a finite list

$$\alpha = ((V_j, m_j, s_j))_{1 \le j \le N},$$

where $V_j \to Z$ is affine etale, $m_j > 0$, and $s_j \in G(V_j)$ is killed by m_j. If h_{V_j} is the representable sheaf, define

$$C_\alpha = \bigoplus_{j=1}^{N} (\mathbf{Z}/m_j\mathbf{Z})[h_{V_j}], \qquad \phi_\alpha : C_\alpha \longrightarrow G$$

by sending the basis element represented by id_{V_j} to s_j, and define $G_\alpha = \mathrm{im}(\phi_\alpha) \subseteq G$. Order indices by inclusion of finite lists. The transition map is the induced inclusion $G_\alpha \subseteq G_\beta$.

Lemma 11.12.51 (Representable finite generators are constructible)**.** Lean: `conditional-interface:lemma:weil-a4-representable-generator-constructible`

Uses: `def:weil-a4-finite-generator-image-system`

For every affine etale $V \to Z$ and $m > 0$, the sheaf $(\mathbf{Z}/m\mathbf{Z})[h_V]$ is constructible torsion.

Proof. Uses: `def:weil-a4-finite-generator-image-system`

The morphism $V \to Z$ is affine, etale, and of finite presentation; in particular it is separated and quasi-finite. We construct a finite stratification of Z by noetherian induction. Over the generic points of each irreducible component, quasi-finiteness gives a finite set of points, and after shrinking to one dense open the morphism is finite etale: write the affine algebra by finitely many generators, invert the finitely many leading coefficients making those generators integral, and invert the discriminant which is nonzero at the etale generic fibers. Remove that dense open and repeat on the proper closed complement. Noetherianity makes the process finite.

On each stratum S, the pullback $V_S \to S$ is finite etale. Hence $h_V|_S$ is a finite locally constant sheaf of sets: an etale cover splitting the finite etale algebra identifies it with a fixed finite set. The free $\mathbf{Z}/m\mathbf{Z}$-module on that finite set is finite locally constant. Thus the displayed sheaf is finite locally constant on every member of a finite stratification, which is constructibility. □

Lemma 11.12.52 (Constructible finite-torsion sheaves form a Serre class)**.** Lean: `conditional-interface:lemma:weil-a4-constructible-torsion-serre`

Uses: `def:weil-a4-finite-generator-image-system`

On a noetherian scheme, kernels, cokernels, images, extensions, and finite direct sums of constructible torsion sheaves are constructible torsion.

Proof. Uses: `def:weil-a4-finite-generator-image-system`

For finitely many input sheaves and maps, intersect their finite constructibility stratifications. On a connected member of the common refinement, every sheaf is represented by a finite module with a continuous action of the profinite etale fundamental group, and every map is an equivariant module homomorphism. Kernel, cokernel, image, finite direct sum, and an extension middle module are finite modules preserved by that action. They therefore give finite locally constant sheaves on the stratum. Repeating over the finite common refinement proves each assertion. □

Lemma 11.12.53 (The finite-generator indices are filtered)**.** Lean: `conditional-interface:lemma:weil-a4-finite-generator-filtered`

Uses: `def:weil-a4-finite-generator-image-system`

The index preorder in the canonical finite-generator image system is filtered, and every transition map is a monomorphism.

Proof. Uses: `def:weil-a4-finite-generator-image-system`

The empty list is an index. The concatenation of two finite lists contains both and therefore is a common upper bound. If two inclusions between indices are parallel, they induce the same inclusion between the image subsheaves of G, so the equalization axiom holds without enlarging the target. By definition a transition map is the inclusion of one image subsheaf of G into another, hence is a monomorphism. □

Lemma 11.12.54 (Constructible images inside a torsion sheaf). Lean: *conditional-interface:lemma:weil-a4-constructible-image-subsheaf*

Uses: *def:weil-a4-finite-generator-image-system*

In the canonical finite-generator image system, every C_α and G_α is constructible torsion and the canonical map

$$\varinjlim_{\alpha} G_\alpha \longrightarrow G$$

is an isomorphism.

Proof. Uses: `lemma:weil-a4-representable-generator-constructible,lemma:weil-a4-constructible-torsion-serre,lemma:weil-a4-finite-generator-filtered,lemma:weil-pbc-geometric-stalk-conservative`

Each C_α is a finite direct sum of the constructible generators from the first lemma. The Serre lemma makes its image G_α constructible torsion. The index system is filtered by the preceding lemma.

Fix a geometric point $\bar{z} \to Z$ and an element $t \in G_{\bar{z}}$. Represent t by a section $s \in G(V)$ on an etale neighbourhood $V \to Z$. Shrinking to an affine open neighbourhood of the chosen lift of $\bar{z}$ makes $V \to Z$ affine etale. Since G is torsion, shrink once more so that one integer $m > 0$ kills s. The one-term index $((V, m, s))$ contains t in the stalk of its image. Thus the colimit map is surjective on every geometric stalk. It is injective because all transition maps and all maps $G_\alpha \to G$ are literal subsheaf inclusions. Geometric-stalk conservativity proves that the map is an isomorphism. □

Lemma 11.12.55 (Closure of a constructible support). Lean: *conditional-interface:lemma:weil-a4-constructible-support-closure*

Uses: *def:weil-a4-support-approximation-datum*

If $E \subseteq Z$ is specialization closed and a constructible sheaf C has support contained in E, then $\overline{\mathrm{Supp}(C)} \subseteq E$.

Proof. Uses: `lemma:weil-a4-transcendence-support-specialization`

The support of a constructible sheaf is a constructible subset: on a finite stratification where the sheaf is finite locally constant, its intersection with each stratum is a union of connected components. Let x lie in the closure of this support. In a noetherian spectral space, a constructible subset contains a point u whose specialization is x: restrict to an irreducible component of its closure containing x, and choose the generic point of a locally closed piece dense in that component. Since $u \in E$ and E is specialization closed, $x \in E$. □

Proposition 11.12.56 (Existence of support-approximation data). Lean: *conditional-interface:prop:weil-a4-support-approximation-datum*

Uses: *def:weil-a4-support-approximation-datum*

Every noetherian Z, specialization-closed E, and torsion sheaf G supported on E has a support-approximation datum.

Proof. Uses: `lemma:weil-a4-constructible-image-subsheaf,lemma:weil-a4-constructible-support-closure,lemma:weil-a4-finite-generator-filtered`

Take the canonical filtered constructible image subsheaves $G_\alpha \subseteq G$ from the first lemma. Their supports lie in the support of G, hence in E. Let Z_α be the reduced closed subscheme whose underlying closed subset is $\overline{\mathrm{Supp}(G_\alpha)}$. The closure lemma gives $|Z_\alpha| \subseteq E$. The transition inclusions and the colimit cocone are those of the image system, and the support inclusions are the just-proved inclusions. This supplies every field of the datum. □

Lemma 11.12.57 (Dimension of a closed subset in E_a). Lean: *conditional-interface:lemma:weil-a4-support-dimension-bound*

Uses: *def:weil-a4-transcendence-support-set*

Let Z be finite type over K. If a reduced closed subscheme $W \subseteq Z$ satisfies $|W| \subseteq E_a(Z/K)$, then $\dim W \leq a$.

Proof. Uses: prop:weil-a4-noether-normalization-datum, def:weil-a4-transcendence-support-set
For every irreducible component W_ν with generic point η_ν, finite type over a field gives

$$\dim W_\nu = \operatorname{trdeg}_K \kappa(\eta_\nu).$$

Indeed, after choosing an affine open in W_ν, a transcendence basis of its fraction field gives a finite integral extension after the Noether-normalization construction, and contraction, incomparability, and going-up identify the lengths of maximal prime chains. Since $\eta_\nu \in |W| \subseteq E_a$, every component has dimension at most a. The dimension of the finite union of components is their maximum. □

Lemma 11.12.58 (Closed-immersion reconstruction and exact pushforward). Lean: conditional-interface:lemma:weil-a4-closed-immersion-pushforward

Uses: def:weil-a4-transcendence-support-set
Let $i : W \hookrightarrow Z$ be a closed immersion. The functor i_* on etale abelian sheaves is exact. If a sheaf C on Z is supported on $|W|$, then the counit

$$i_* i^{-1} C \longrightarrow C$$

is an isomorphism. Consequently

$$H^r(Z, C) \cong H^r(W, i^{-1} C)$$

naturally in C.

Proof. Uses: prop:weil-pbc-strict-henselization-stalk-formula, lemma:weil-pbc-geometric-stalk-conservative
At a geometric point $\bar{z} \to Z$, the strict-local pullback $W \times_Z \operatorname{Spec} \mathcal{O}^{\mathrm{sh}}_{Z,\bar{z}}$ is empty when $z \notin W$, and has one closed strictly local component through $\bar{z}$ when $z \in W$. Therefore

$$(i_* A)_{\bar{z}} = \begin{cases} A_{\bar{z}}, & z \in W, \\ 0, & z \notin W. \end{cases}$$

This formula is computed degreewise and preserves kernels and cokernels, so i_* is exact. Applied to $A = i^{-1}C$, the counit is the identity at points of W; outside W, both stalks vanish by the support hypothesis. Geometric-stalk conservativity proves that it is an isomorphism. Since i_* is exact and global sections satisfy $\Gamma(Z, i_* A) = \Gamma(W, A)$, applying injective resolutions gives the cohomology isomorphism. □

Lemma 11.12.59 (Cohomology of sheaves with bounded transcendence support). Lean: conditional-interface:lemma:weil-a4-supported-sheaf-cohomology

Uses: def:weil-a4-transcendence-support-set, def:weil-a4-strong-affine-cd-predicate
Let Z be affine, smooth, and finite type over K. Suppose G is a torsion sheaf supported on $E_a(Z/K)$, $\mathrm{FieldCD}(K; c)$, *and* $\mathrm{AffCD}(K; c, e)$ *is known for every $e < d$, where $a < d$. Then*

$$H^b(Z, G) = 0 \qquad (b > a + c).$$

Proof. Uses: prop:weil-a4-support-approximation-datum, lemma:weil-a4-support-dimension-bound, lemma:weil-a4-closed-immersion-pushforward, lemma:weil-a4-torsion-filtered-cohomology, def:weil-a4-strong-affine-cd-predicate
Take a support-approximation datum $G = \varinjlim_i G_i$, with G_i supported on a reduced closed affine subscheme $i_i : Z_i \hookrightarrow Z$ contained in E_a. The support dimension lemma gives $\dim Z_i \le a < d$. Closed-immersion reconstruction writes

$$G_i = i_{i,*} i_i^{-1} G_i, \qquad H^b(Z, G_i) = H^b(Z_i, i_i^{-1} G_i).$$

The induction hypothesis $\mathrm{AffCD}(K; c, a)$ kills the last group for $b > a + c$. Filtered cohomology continuity gives

$$H^b(Z, G) = \varinjlim_i H^b(Z, G_i) = 0.$$

Every equality is induced by a transition map of the chosen filtered system, so it is natural in maps of supported sheaves. □

Definition 11.12.60 (Standard affine-line compactification). Lean: conditional-interface:def:weil-a4-standard-affine-line-compactification

Uses: def:weil-a4-transcendence-support-set
For $d > 1$, put $B = \mathbf{A}^{d-1}_K$ and define

$$\begin{array}{ccc} \mathbf{A}^d_K = \mathbf{A}^1_K \times_K B & \xrightarrow{\ j\ } & \mathbf{P}^1_K \times_K B \\ {\scriptstyle f}\downarrow & & \downarrow{\scriptstyle g} \\ B & = & B. \end{array}$$

Here j is induced by the chart $[X_0 : X_1] = [1 : t]$, f and g are the second projections, and

$$D = (\{[0 : 1]\} \times_K B)_{\mathrm{red}}$$

is the complement. The inverse of $g|_D : D \to B$ is $b \mapsto ([0 : 1], b)$.

Lemma 11.12.61 (Geometry of the standard compactification). *Lean:* `conditional-interface:lemma:weil-a4-standard-compactification-geometry`

Uses: `def:weil-a4-standard-affine-line-compactification`

The morphism j is an open immersion, D is an effective Cartier divisor, both its source and target are smooth of dimension d over K, g is proper of relative dimension one, and $g|_D : D \xrightarrow{\sim} B$.

Proof. Uses: `def:weil-a4-standard-affine-line-compactification`

On the chart $X_0 \neq 0$, the coordinate $t = X_1/X_0$ identifies the source with $\mathbf{A}^1 \times B$. Its complement is the vanishing of the homogeneous coordinate X_0; on the chart $X_1 \neq 0$, it is cut out by the non-zero-divisor X_0/X_1. Hence the complement is an effective Cartier divisor and is the displayed section. Polynomial rings over K, their localizations, and the two standard charts of $\mathbf{P}^1_K$ are smooth, and dimensions add under their products, giving dimension d. Projective space is proper over K, and proper morphisms and relative dimension are preserved by base change, so g is proper of relative dimension one. The two explicit maps for D and B are inverse on coordinates. □

Lemma 11.12.62 (The relative projective line has torsion cd at most two). *Lean:* `conditional-interface:lemma:weil-a4-relative-p1-cd-two`

Uses: `def:weil-a4-standard-affine-line-compactification`

For every torsion sheaf H on $(\mathbf{P}^1_K \times_K B)_{\acute{e}t}$,

$$R^p g_* H = 0 \qquad (p > 2).$$

Proof. Uses: `prop:weil-pbc-p1-strict-henselian-comparison,prop:weil-pbc-strict-henselization-stalk-formula,lemma:weil-pbc-geometric-stalk-conservative,lemma:weil-pbc-universal-homeomorphism-coverings,def:weil-pbc-eight-curve-assertions,lemma:weil-pbc-eight-curve-devissage,lemma:weil-pbc-curve-constructible-primary-reduction,lemma:weil-pbc-curve-dense-open-reduction,lemma:weil-pbc-curve-locally-constant-reduction,lemma:weil-pbc-finite-pushforward-extension-zero,lemma:weil-pbc-curve-normalization-constant`

Fix a geometric point $\bar{b} \to B$, and put $S = \operatorname{Spec} \mathcal{O}^{\mathrm{sh}}_{B,\bar{b}}$. The strict-henselization stalk formula first identifies

$$(R^p g_* H)_{\bar{b}} = H^p(\mathbf{P}^1_S, H|).$$

The A3 strictly-henselian $\mathbf{P}^1$ comparison then identifies the right side with $H^p(\mathbf{P}^1_{\kappa(\bar{b})}, H|)$. The residue field $\kappa(\bar{b})$ is separably closed. Passing to its algebraic closure is a universal homeomorphism and hence does not change the etale topos. The A3 eight-assertion curve calculation, including arbitrary torsion approximation, gives zero in degrees $p > 2$ for this proper curve. Thus every geometric stalk of $R^p g_* H$ is zero for $p > 2$, and geometric-stalk conservativity proves the asserted sheaf equality. □

Lemma 11.12.63 (Higher direct images of the open immersion live at infinity). *Lean:* `conditional-interface:lemma:weil-a4-open-immersion-higher-support`

Uses: `def:weil-a4-standard-affine-line-compactification`

For a torsion sheaf F on $\mathbf{A}^d_K$ and every $q > 0$, $R^q j_* F$ is supported on D.

Proof. Uses: `prop:weil-pbc-strict-henselization-stalk-formula,lemma:weil-pbc-geometric-stalk-conservative`

Let $\bar{y}$ be a geometric point in the open image of j. Base change to the strict henselization at $\bar{y}$ leaves j equal to the identity on an etale neighbourhood of the closed point. The stalk formula therefore gives

$$(R^q j_* F)_{\bar{y}} = H^q(\operatorname{Spec} \mathcal{O}^{\mathrm{sh}}_{Y,\bar{y}}, F|) = 0 \qquad (q > 0),$$

because a strictly henselian local scheme has exact global sections on its etale site. All stalks outside D vanish, which is the stated support condition. □

Lemma 11.12.64 (Punctured strict-local Artin bound). *Lean:* `conditional-interface:lemma:weil-a4-punctured-strict-local-artin-bound`

Uses: `def:weil-a4-punctured-strict-local-datum,def:weil-a4-strong-affine-cd-predicate`

Assume the affine cd theorem over every field in dimensions less than d. In the punctured strict-local datum at a point of transcendence degree a, every torsion sheaf F satisfies

$$H^q(\operatorname{Spec} A[1/f], F) = 0 \qquad (q > d - a).$$

Proof. Uses: prop:weil-a4-punctured-strict-local-datum, lemma:weil-a4-sepclosed-field-cd-zero, lemma:weil-a4-field-transcendence-cd, lemma:weil-a4-affine-limit-cd, def:weil-a4-strong-affine-cd-predicate

Write $L = K(t_1, \dots, t_a)^{\mathrm{sep}}$ and $M = L(x)$. The separably closed field lemma gives $\mathrm{FieldCD}(L; 0)$, and the transcendence-degree-one lemma gives $\mathrm{FieldCD}(M; 1)$. The datum writes

$$\operatorname{Spec} A[1/f] = \varprojlim_{\lambda} \operatorname{Spec} B_\lambda, \qquad \dim B_\lambda \le d - a - 1.$$

Since $d - a - 1 < d$, the induction hypothesis over the field M gives torsion cd at most $(d - a - 1) + 1 = d - a$ for every stage. The affine inverse limit lemma preserves this bound. Hence the displayed cohomology vanishes for $q > d - a$. □

Lemma 11.12.65 (Interlude I: support of $R^q j_*$). Lean: conditional-interface:lemma:weil-a4-interlude-one-support

Uses: def:weil-a4-standard-affine-line-compactification, def:weil-a4-transcendence-support-set

Assume the affine cd theorem over every field in dimensions less than d. For every $q > 0$,

$$\operatorname{Supp}(R^q j_* F) \subseteq D \cap E_{d-q}(\mathbf{P}^1_K \times_K B/K).$$

In particular, $R^q j_* F = 0$ for $q > d$.

Proof. Uses: lemma:weil-a4-open-immersion-higher-support, prop:weil-pbc-strict-henselization-stalk-formula, prop:weil-a4-punctured-strict-local-datum, lemma:weil-a4-punctured-strict-local-artin-bound, lemma:weil-pbc-geometric-stalk-conservative

The open-immersion support lemma restricts attention to $y \in D$. Put $a = \operatorname{trdeg}_K \kappa(y)$, choose a geometric lift $\bar{y}$, and choose the local Cartier equation f. The strict-henselization stalk formula and the equality between the punctured strict-local scheme and the pullback of the open give

$$(R^q j_* F)_{\bar{y}} = H^q(\operatorname{Spec} \mathcal{O}^{\mathrm{sh}}_{Y,\bar{y}}[1/f], F|).$$

The punctured strict-local Artin bound kills this group when $q > d - a$, equivalently when $a > d - q$. Hence a nonzero stalk can occur only at a point of E_{d-q}. If $q > d$, then $d - q < 0$ and this set is empty; conservativity of geometric stalks gives $R^q j_* F = 0$. □

Lemma 11.12.66 (A sheaf supported on the infinity section is g-acyclic). Lean: conditional-interface:lemma:weil-a4-infinity-section-acyclic

Uses: def:weil-a4-standard-affine-line-compactification

If a torsion sheaf C on $\mathbf{P}^1_K \times_K B$ is supported on D, then

$$R^p g_* C = 0 \qquad (p > 0), \qquad g_* C \cong (g|_D)_* i_D^{-1} C.$$

Proof. Uses: lemma:weil-a4-closed-immersion-pushforward, lemma:weil-a4-standard-compactification-geometry

Closed-immersion reconstruction gives $C = i_{D,*} i_D^{-1} C$. Since $g \circ i_D = g|_D$ is an isomorphism, its direct image is an exact equivalence of abelian sheaf categories. The composite $g_* i_{D,*} = (g|_D)_*$ is exact, so all its positive derived functors vanish, and degree zero is the displayed identification. □

Lemma 11.12.67 (Relative Leray row calculation). Lean: conditional-interface:lemma:weil-a4-relative-leray-row-shape

Uses: def:weil-a4-standard-affine-line-compactification

Assume the affine cd theorem over every field in dimensions less than d. For the standard compactification and every torsion F,

$$R^q f_* F \cong g_* R^q j_* F \quad (q > 2), \qquad R^q f_* F = 0 \quad (q > d).$$

The isomorphism is the edge map of the Grothendieck spectral sequence for $g_* \circ j_*$.

Proof. Uses: lemma:weil-a4-interlude-one-support, lemma:weil-a4-infinity-section-acyclic, lemma:weil-a4-relative-p1-cd-two

The composite equality $f = g \circ j$ gives the first-quadrant spectral sequence

$$E_2^{p,q} = R^p g_* R^q j_* F \Longrightarrow R^{p+q} f_* F.$$

For $q > 0$, Interlude I places $R^q j_* F$ on D, and the infinity section lemma makes every $E_2^{p,q}$ with $p > 0$ zero. For $q = 0$, the relative projective-line lemma makes $E_2^{p,0} = 0$ for $p > 2$. Therefore, in total degree $q > 2$, the only nonzero filtration quotient is $E_2^{0,q} = g_* R^q j_* F$; no incoming or outgoing differential can meet it because every positive column in a positive row is zero. This gives the edge isomorphism. Interlude I gives $R^q j_* F = 0$ for $q > d$, and the edge isomorphism then gives $R^q f_* F = 0$ in those degrees. □

Lemma 11.12.68 (Support of the higher affine-line direct images). *Lean:* `conditional-interface:lemma:weil-a4-affine-line-higher-support`

Uses: `def:weil-a4-standard-affine-line-compactification`, `def:weil-a4-transcendence-support-set`

Assume the affine cd theorem over every field in dimensions less than d. Then

$$\operatorname{Supp}(R^q f_* F) \subseteq E_{d-q}(B/K) \quad (q \geq 2), \qquad R^q f_* F = 0 \quad (q > d).$$

Proof. Uses: `lemma:weil-a4-relative-leray-row-shape`, `lemma:weil-a4-interlude-one-support`, `lemma:weil-a4-standard-compactification-geometry`, `prop:weil-pbc-strict-henselization-stalk-formula`, `lemma:weil-a4-affine-curve-sepclosed`, `lemma:weil-pbc-geometric-stalk-conservative`

For $q > 2$, the relative Leray edge isomorphism identifies $R^q f_* F$ with the pushforward of $R^q j_* F$ along the isomorphism $D \to B$. Under this isomorphism, residue fields over K are equal, so Interlude I gives support in $E_{d-q}(B/K)$.

For $q = 2$, let η be the generic point of the irreducible scheme $B = \mathbf{A}^{d-1}_K$, and choose a geometric point $\bar{\eta} \to B$. The strict-henselization stalk formula gives

$$(R^2 f_* F)_{\bar{\eta}} = H^2(\mathbf{A}^1_{\kappa(\bar{\eta})}, F|) = 0$$

by affine-curve vanishing over the separably closed residue field. A sheaf with zero geometric generic stalk has support disjoint from the generic point. Every non-generic point b of B satisfies

$$\operatorname{trdeg}_K \kappa(b) \leq \dim B - 1 = d - 2 :$$

the closure of b is a proper closed subset of the integral $(d-1)$-dimensional affine space, and its dimension equals that transcendence degree. Thus the support lies in E_{d-2}. The final vanishing for $q > d$ is already part of the row-shape lemma. □

Proposition 11.12.69 (Affine-space induction step). *Lean:* `conditional-interface:prop:weil-a4-affine-space-induction-step`

Uses: `def:weil-a4-strong-affine-cd-predicate`, `def:weil-a4-standard-affine-line-compactification`

Let $d > 1$. If $\mathrm{AffCD}(L; c', e)$ holds for every field L, every field-cd bound c', and every $e < d$, then for every field K satisfying $\mathrm{FieldCD}(K; c)$, every torsion sheaf F on $\mathbf{A}^d_K$, and every $r > d + c$,

$$H^r(\mathbf{A}^d_K, F) = 0.$$

Proof. Uses: `lemma:weil-a4-affine-line-higher-support`, `lemma:weil-a4-supported-sheaf-cohomology`, `def:weil-a4-strong-affine-cd-predicate`

Use the Leray spectral sequence for the affine-line projection:

$$E_2^{p,q} = H^p(B, R^q f_* F) \Longrightarrow H^{p+q}(\mathbf{A}^d_K, F).$$

For $q = 0, 1$, the induction hypothesis on the affine $(d-1)$-dimensional scheme B gives $E_2^{p,q} = 0$ for $p > d - 1 + c$. If $p + q > d + c$, this inequality holds for $q = 0, 1$.

For $q \geq 2$, the support lemma places $R^q f_* F$ in $E_{d-q}(B/K)$, and it is zero for $q > d$. When $2 \leq q \leq d$, the supported-sheaf lemma gives

$$E_2^{p,q} = 0 \qquad (p > d - q + c).$$

Again $p + q > d + c$ implies this inequality. Hence every E_2-term of total degree greater than $d + c$ is zero. The spectral sequence has only the rows $0 \leq q \leq d$, so each abutment has a finite filtration, and all its graded pieces are zero in those total degrees. This proves the displayed vanishing. □

Proposition 11.12.70 (Strengthened affine torsion cohomological dimension). *Lean:* `conditional-interface:prop:weil-a4-strong-affine-cd`

Uses: `def:weil-a4-strong-affine-cd-predicate`

For every field K, every $c, d \geq 0$, $\mathrm{AffCD}(K; c, d)$ holds.

Proof. Uses: `lemma:weil-a4-affine-cd-zero`, `lemma:weil-a4-affine-cd-one`, `prop:weil-a4-noether-normalization-datum`, `lemma:weil-a4-finite-morphism-cd`, `prop:weil-a4-affine-space-induction-step`, `def:weil-a4-strong-affine-cd-predicate`

Induct simultaneously on d for all fields and all field-cd bounds. The cases $d = 0$ and $d = 1$ are the two preceding dimension lemmas. Let $d > 1$, and assume the assertion in every smaller dimension.

Let $X = \operatorname{Spec} A$ be affine of finite type over K, with $\dim X = e \leq d$, and let F be torsion. If $e < d$, the induction hypothesis applies directly. Suppose $e = d$. A Noether-normalization datum gives a finite morphism

$$h : X \longrightarrow \mathbf{A}^d_K.$$

The finite-morphism lemma gives

$$H^r(X, F) = H^r(\mathbf{A}^d_K, h_*F),$$

and h_*F is torsion because each of its sections is a section of F over a finite etale pullback and is locally killed by an integer. The affine-space induction step, whose hypotheses are precisely the simultaneous induction hypotheses, kills the right-hand side for $r > d + c$. This is AffCD$(K; c, d)$, closing the induction. □

Lemma 11.12.71 (Specialization to finite Λ_n-coefficients). *Lean:* `conditional-interface:lemma:weil-a4-strong-affine-to-certificate`

Uses: `def:weil-a4-affine-artin-vanishing-certificate`

For every $n \geq 1$, the strengthened affine cd theorem supplies all fields of an affine Artin-vanishing certificate at Λ_n.

Proof. Uses: `prop:weil-a4-strong-affine-cd,lemma:weil-a4-sepclosed-field-cd-zero,def:weil-a4-affine-artin-vanishing-certificate`

Let k be separably closed, U affine of dimension at most e, and G a constructible sheaf of finite Λ_n-modules. It is a torsion abelian sheaf. The separably closed field lemma gives FieldCD$(k; 0)$, and the strengthened theorem gives

$$H^r(U, G) = 0 \qquad (r > e).$$

Restriction to an affine open, passage through a finite filtration, base extension between separably closed fields, and pullback by a field automorphism all act by functorial pullback on the cochain complexes and on the Leray spectral sequences used above. The induced maps between zero groups are the unique maps, so identity and composition compatibilities hold. For $k = \overline{\mathbf{F}}_q$, geometric Frobenius is one of these field automorphisms. These equalities provide the compatibility fields required by the certificate definition. □

Proposition 11.12.72 (Affine Artin vanishing). *Lean:* `conditional-interface:prop:weil-a4-affine-artin-vanishing`

Uses: `def:weil-a4-affine-artin-vanishing-certificate`

An affine Artin-vanishing certificate exists for every finite coefficient ring Λ_n.

Proof. Uses: `lemma:weil-a4-strong-affine-to-certificate`

For the chosen n, take the certificate whose fields are supplied by the specialization lemma. Its vanishing, restriction, filtration, field extension, automorphism, and geometric-Frobenius fields are exactly those listed there. □

Definition 11.12.73 (Strong geometric proper cohomological-dimension predicate). Uses:

Lean: `conditional-interface:def:weil-a4-strong-geometric-proper-cd`

For $d \geq 0$, let ProperCD(d) mean that for every algebraically closed field Ω of characteristic different from ℓ, every proper finite-type Ω-scheme X with $\dim X \leq d$, and every torsion abelian sheaf F on $X_{\acute{e}t}$,

$$H^r(X, F) = 0 \qquad (r > 2d).$$

The quantification over arbitrary torsion sheaves is intentional: the cohomology sheaves of the graph cone need not yet be known constructible.

Definition 11.12.74 (Normalization-descent datum). Uses:

Lean: `conditional-interface:def:weil-a4-normalization-descent-datum`

Let X be a reduced proper finite-type scheme over an algebraically closed field and let F be a torsion sheaf. A normalization-descent datum consists of:

1. a finite disjoint union X^ν of normal integral schemes and a finite surjective morphism $\nu : X^\nu \to X$;
2. a dense open $j : U \hookrightarrow X$ over which ν is an isomorphism, its reduced closed complement $i : Z \hookrightarrow X$, and the inequality $\dim Z < \dim X$;
3. the adjunction map $c_F : F \to \nu_*\nu^{-1}F$, its kernel, cokernel C_F, and explicit maps exhibiting

$$0 \longrightarrow F \xrightarrow{c_F} \nu_*\nu^{-1}F \longrightarrow i_*i^{-1}C_F \longrightarrow 0$$

as a short exact sequence;

4. the finite-morphism cohomology identifications

$$H^r(X, \nu_*\nu^{-1}F) \xrightarrow{\sim} H^r(X^\nu, \nu^{-1}F)$$

and the long exact cohomology sequence, including all boundary maps.

Lemma 11.12.75 (Reduction does not change the small etale topos)**.** *Lean:* `conditional-interface:lemma:weil-a4-reduction-etale-equivalence`

Uses: `def:weil-a4-normalization-descent-datum`

For the nilpotent closed immersion $r : X_{\mathrm{red}} \hookrightarrow X$, pullback and pushforward give inverse exact equivalences of small etale topoi. Consequently

$$H^q(X, F) \xrightarrow{\sim} H^q(X_{\mathrm{red}}, r^{-1}F)$$

for every torsion sheaf F, naturally in F.

Proof. Uses: `lemma:weil-pbc-universal-homeomorphism-coverings`

The immersion is integral, surjective, and radicial, hence a universal homeomorphism. The A3 explicit site equivalence sends an etale X-scheme V to $V \times_X X_{\mathrm{red}}$. Its inverse lifts an etale algebra uniquely across a nilpotent ideal: a presentation by an etale Jacobian matrix lifts because invertibility lifts modulo a nilpotent ideal, and two lifts are uniquely isomorphic by formal etaleness. The two unit and counit maps are identities after these presentations. The induced equivalence on abelian sheaves is exact, sends an injective resolution to an injective resolution, and commutes with global sections, giving the stated cohomology isomorphism. □

Lemma 11.12.76 (Integral closure over a polynomial algebra is finite)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-a4-polynomial-integral-closure-finite`

Let K be algebraically closed, let $R = K[T_1, \dots, T_d]$, and let $L/\operatorname{Frac}(R)$ be a finite field extension. The integral closure S of R in L is a finite R-module.

Proof. Uses: `def:weil-a4-normalization-descent-datum`

First suppose the extension is separable. Choose a field basis $v_1, \dots, v_m$ of $L/\operatorname{Frac}(R)$ and multiply its members by nonzero elements of R so that every v_i is integral over R. The trace pairing is nondegenerate, so there is a trace-dual basis $w_1, \dots, w_m$ satisfying $\operatorname{Tr}(v_i w_j) = \delta_{ij}$. If $x \in S$, then xv_i is integral over R. Its field trace is integral over R, because it is the negative of a coefficient of the characteristic polynomial of multiplication by xv_i, and it belongs to $\operatorname{Frac}(R)$. The polynomial ring R is integrally closed: induction on the variables uses Gauss's lemma for a UFD. Hence $\operatorname{Tr}(xv_i) \in R$, and the dual-basis formula gives

$$x = \sum_{i=1}^{m} \operatorname{Tr}(xv_i) w_i \in \sum_{i=1}^{m} R w_i.$$

Thus S is an R-submodule of a finite free module. Since R is noetherian, S is finite.

For an arbitrary finite extension, let L_s be its maximal separable subextension. In characteristic zero it equals L. In characteristic $p > 0$, choose e with $L^{p^e} \subseteq L_s$. The separable case makes the integral closure S_s of R in L_s finite; choose generators $s_1, \dots, s_N$. For $x \in S$, the element x^{p^e} lies in S_s, so $x^{p^e} = \sum_j r_j s_j$ with $r_j \in R$. The algebraically closed base field is perfect. In a fixed algebraic closure, uniqueness and additivity of p^e-th roots give

$$x = \sum_j r_j^{1/p^e} s_j^{1/p^e}.$$

The ring $R^{1/p^e} = K^{1/p^e}[T_1^{1/p^e}, \dots, T_d^{1/p^e}]$ is finite free over R, with basis the monomials whose exponents lie in $\{0, \dots, p^e - 1\}/p^e$, since $K^{1/p^e} = K$. Therefore all $x \in S$ lie in the finite R-module generated by those monomials times the finitely many s_j^{1/p^e}. Noetherianity again makes the submodule S finite. □

Lemma 11.12.77 (Finite normalization of a reduced finite-type scheme)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-a4-finite-normalization`

The normalization of a reduced finite-type scheme over an algebraically closed field is a finite surjective morphism from the finite disjoint union of the normalizations of its irreducible components. It is an isomorphism over a dense open, and the closed complement of that open has dimension strictly smaller than the source scheme.

Proof. Uses: `prop:weil-a4-noether-normalization-datum,lemma:weil-a4-polynomial-integral-closure-finite`

On an affine open $\operatorname{Spec} A$, write the reduced ring as a subring of the finite product of the fraction fields of $A/\mathfrak{p}_\alpha$ over its finitely many minimal primes. Each $A/\mathfrak{p}_\alpha$ is a finite-type domain over the field. A Noether-normalization datum makes it finite over a polynomial ring. Its fraction field is a finite extension of the polynomial ring's fraction field. The preceding integral-closure lemma makes the integral closure in that field finite over the polynomial ring. Since it contains $A/\mathfrak{p}_\alpha$, the same finite generating list also generates it as an $A/\mathfrak{p}_\alpha$-module. These finite integral closures agree on overlaps by their common universal property and glue to a finite morphism $\nu : X^\nu \to X$.

Every prime of A contains a minimal prime, and lying-over for the integral closure supplies a prime above it, so ν is surjective. On each component the finite birational map is an isomorphism after inverting the finitely

many denominators of a module generating set. Intersect these dense opens and remove component intersections to obtain one dense open $U \subseteq X$ over which ν is an isomorphism. Every irreducible component of the closed complement omits the generic point of its ambient component. A strict prime-chain argument therefore gives dimension at most $\dim X - 1$. □

Lemma 11.12.78 (Normalization adjunction is injective). Lean: `conditional-interface:lemma:weil-a4-normalization-unit-injective`

Uses: `def:weil-a4-normalization-descent-datum`

For a finite surjective $\nu : X^\nu \to X$ and every etale sheaf F, the adjunction map $F \to \nu_*\nu^{-1}F$ is injective. If ν is an isomorphism over U, its cokernel is supported on $X \setminus U$.

Proof. Uses: `prop:weil-pbc-strict-henselization-stalk-formula,lemma:weil-pbc-geometric-stalk-conservative`

Fix a geometric point $\bar{x} \to X$. Surjectivity and integrality give a geometric point $\bar{x}^\nu \to X^\nu$ above it. On stalks, the adjunction map sends a germ to its pullback at every geometric lift. Pullback followed by the chosen lift's forgetful map is the identity on the original germ, so the stalk map is injective. Geometric-stalk conservativity makes the sheaf map injective. Over U, the adjunction is the unit of an isomorphism and is itself an isomorphism; hence the cokernel has zero stalks on U and is supported on its closed complement. □

Proposition 11.12.79 (Existence of normalization-descent data). Lean: `conditional-interface:prop:weil-a4-normalization-descent-datum`

Uses: `def:weil-a4-normalization-descent-datum`

Every reduced proper finite-type X and torsion sheaf F has a normalization-descent datum.

Proof. Uses: `lemma:weil-a4-finite-normalization,lemma:weil-a4-normalization-unit-injective,lemma:weil-a4-closed-immersion-pushforward,lemma:weil-a4-finite-morphism-cd`

Take X^ν, ν, U, Z from the finite-normalization lemma and take c_F and its cokernel C_F from adjunction. The unit lemma gives injectivity and support of C_F on Z. Closed-immersion reconstruction identifies $C_F = i_* i^{-1} C_F$, producing the short exact sequence. Finite-morphism exactness identifies cohomology on X with cohomology on X^ν. Applying global sections to an injective resolution of the short exact sequence supplies its functorial long exact sequence and boundary maps. These are all fields of the datum. □

Definition 11.12.80 (Nonconstant rational-function datum). Uses:

Lean: `conditional-interface:def:weil-a4-rational-function-datum`

For an integral finite-type d-dimensional scheme X over a field, with $d > 0$, a rational-function datum is an element $t \in K(X)$ together with two points of X at which its images in the corresponding residue fields are different. In particular $t \notin K$.

Proposition 11.12.81 (Existence of a nonconstant rational-function datum). Lean: `conditional-interface:prop:weil-a4-rational-function-datum`

Uses: `def:weil-a4-rational-function-datum`

Every positive-dimensional integral finite-type scheme has a nonconstant rational-function datum.

Proof. Uses: `prop:weil-a4-noether-normalization-datum`

Choose a nonempty affine open $V = \operatorname{Spec} A \subseteq X$. Its dimension is $d > 0$. Noether normalization provides an injective map $K[T_1, \dots, T_d] \to A$. Let t be the image of T_1 in $A \subseteq K(X)$. Injectivity shows that t is transcendental over K, hence nonconstant. The morphism $V \to \mathbf{A}^1_K$ induced by t is dominant; the inverse images of the disjoint closed subsets $T_1 = 0$ and $T_1 = 1$ are nonempty after passing to closed points because the finite normalization map to affine space is surjective. Choose one point over each value. Their residue images of t are 0 and 1, providing the two witnesses in the datum. □

Definition 11.12.82 (Graph-alteration datum). Lean: `conditional-interface:def:weil-a4-graph-alteration-datum`

Uses: `def:weil-a4-rational-function-datum`

Let X be normal, integral, proper, and d-dimensional over an algebraically closed field, with $d > 1$, and let t be a rational-function datum. A graph-alteration datum consists of:

1. the scheme X_t, defined as the reduced closure in $X \times \mathbf{P}^1$ of the graph of $t : \mathrm{Dom}(t) \to \mathbf{P}^1$, and the restrictions
$$X \xleftarrow{b_t} X_t \xrightarrow{f_t} \mathbf{P}^1;$$
2. an open $U_t \subseteq X$ containing every codimension-one point, over which b_t is an isomorphism, and its reduced complement $i_t : Z_t \hookrightarrow X$, with $\dim Z_t \leq d - 2$;

3. proofs that b_t is proper, surjective, and has fibers of dimension at most one, and that f_t is proper with fibers of dimension at most $d-1$;
4. for every torsion sheaf F, the adjunction map $F \to Rb_{t,*}b_t^{-1}F$ and its cone

$$Q_{t,F} := \mathrm{Cone}(F \to Rb_{t,*}b_t^{-1}F).$$

Lemma 11.12.83 (A normal rational map to $\mathbf{P}^1$ is defined in codimension one). Lean: `conditional-interface:lemma:weil-a4-rational-map-codimension-one`

Uses: `def:weil-a4-graph-alteration-datum`

For normal integral X and $t \in K(X)$, the rational map $[1:t] : X \dashrightarrow \mathbf{P}^1$ is defined at every codimension-one point. Its indeterminacy locus has dimension at most $\dim X - 2$.

Proof. Uses: `def:weil-a4-graph-alteration-datum`

At a codimension-one point x, the noetherian normal local domain $\mathcal{O}_{X,x}$ is a discrete valuation ring. If the valuation of t is nonnegative, then $t \in \mathcal{O}_{X,x}$ and the chart $[1:t]$ defines the map. If it is negative, then $t^{-1} \in \mathcal{O}_{X,x}$ and the chart $[t^{-1}:1]$ defines it. These two formulas agree on their common domain. Thus every point omitted from the maximal domain has codimension at least two. In a noetherian d-dimensional scheme, a closed subset all of whose generic points have codimension at least two has dimension at most $d-2$. □

Lemma 11.12.84 (Geometry of the graph alteration). Lean: `conditional-interface:lemma:weil-a4-graph-alteration-geometry`

Uses: `def:weil-a4-graph-alteration-datum`

The graph closure has all the properness, surjectivity, isomorphism-locus, and fiber-dimension properties listed in a graph-alteration datum.

Proof. Uses: `lemma:weil-a4-rational-map-codimension-one`

The product $X \times \mathbf{P}^1$ is proper over both X and the base field. The reduced graph closure is closed in this product, so b_t and f_t are proper. Over the domain of the rational map, the graph projection is an isomorphism. Its image under the proper map b_t is closed and contains this dense open; hence it is all of X, proving surjectivity. The codimension-one lemma supplies the chosen open U_t and the dimension bound on its complement.

For $x \in X$, the fiber of b_t is a closed subscheme of $\{x\} \times \mathbf{P}^1_{\kappa(x)}$, hence has dimension at most one. For $y \in \mathbf{P}^1$, the fiber of f_t is a proper closed subscheme of X. It cannot contain the generic point of X_t: the function t is nonconstant, so the generic point maps to the generic point of $\mathbf{P}^1$. Every irreducible component of the fiber therefore has dimension at most $d-1$. □

Proposition 11.12.85 (Existence of graph-alteration data). Lean: `conditional-interface:prop:weil-a4-graph-alteration-datum`

Uses: `def:weil-a4-graph-alteration-datum`

Every normal integral proper X of dimension $d > 1$ and every rational-function datum on X has a graph-alteration datum.

Proof. Uses: `lemma:weil-a4-graph-alteration-geometry`

Define X_t as the reduced graph closure and define b_t, f_t by the two product projections. Take the maximal domain and complement from the codimension-one calculation, and take every geometric property from the graph-geometry lemma. For a torsion sheaf F, the unit of the derived adjunction $b_t^{-1} \dashv Rb_{t,*}$ is canonical; its mapping cone with the displayed cochain convention defines $Q_{t,F}$. These choices supply all fields of the datum. □

Definition 11.12.86 (Common refinement of two graph alterations). Lean: `conditional-interface:def:weil-a4-graph-common-refinement`

Uses: `def:weil-a4-graph-alteration-datum`

For rational functions t_1, t_2 on the same normal integral proper X, define X_{12} to be the reduced closure of their joint graph in $X \times \mathbf{P}^1 \times \mathbf{P}^1$. A graph common-refinement datum consists of the three projections

$$X_{12} \longrightarrow X_{t_1}, \qquad X_{12} \longrightarrow X_{t_2}, \qquad X_{12} \longrightarrow X,$$

their properness, their equality over the common domain of t_1, t_2, and the induced pullback maps on adjunction triangles and Leray spectral sequences.

Proposition 11.12.87 (Existence and cocycle for graph common refinements). Lean: `conditional-interface:prop:weil-a4-graph-common-refinement`

Uses: `def:weil-a4-graph-common-refinement`

Every pair of graph-alteration data has a graph common-refinement datum. For three rational functions, the two iterated common refinements are dominated by the closure of the joint graph in $X \times (\mathbf{P}^1)^3$, and the induced comparison maps satisfy the cocycle equation after pullback to that domination.

Proof. Uses: lemma:weil-a4-graph-alteration-geometry

The closure of the joint graph maps to each one-function graph closure by a coordinate projection. These maps are restrictions of projections with projective fibers, hence proper. On the intersection of the two rational maps' domains all graph closures equal the corresponding literal graphs, so the projections agree with the identity over X. Functoriality of inverse image, derived direct image, adjunction units, mapping cones, and the Leray exact couple gives the comparison maps.

For three functions, form the reduced closure X_{123} of the joint graph. Forgetting coordinates gives proper maps from X_{123} to both parenthesized iterated closures. Every composite comparison is induced by the same coordinate projection from X_{123}. Equality of scheme maps gives equality of pullback functors and of every naturality square; hence the two composites on triangles and spectral sequences are equal. This is the cocycle field. □

Lemma 11.12.88 (Proper relative curves have direct-image amplitude two)**.** *Lean:* conditional-interface:lemma:weil-a4-proper-relative-curve-cd-two

Uses: def:weil-a4-strong-geometric-proper-cd

Let $h : Y \to S$ be proper with every geometric fiber of dimension at most one. For every torsion sheaf F on $Y_{ét}$,

$$R^q h_* F = 0 \qquad (q > 2).$$

Proof. Uses: prop:weil-proet-finite-proper-base-change,lemma:weil-pbc-finite-stalk-formula,lemma:weil-pbc-geometric-stalk-conservative,lemma:weil-pbc-universal-homeomorphism-coverings,def:weil-pbc-eight-curve-assertions,lemma:weil-pbc-eight-curve-devissage,lemma:weil-pbc-curve-constructible-primary-reduction,lemma:weil-pbc-curve-dense-open-reduction,lemma:weil-pbc-curve-locally-constant-reduction,lemma:weil-pbc-finite-pushforward-extension-zero,lemma:weil-pbc-curve-normalization-constant

At a geometric point $\bar{s} \to S$, finite-torsion proper base change and the stalk formula give

$$(R^q h_* F)_{\bar{s}} = H^q(Y_{\bar{s}}, F|).$$

The fiber is separated, finite type, and of dimension at most one over a separably closed field. Passing to its algebraic closure is a universal homeomorphism and does not change its etale topos. The A3 eight-assertion curve proof kills its torsion cohomology in degrees greater than two. Every geometric stalk is therefore zero, and stalk conservativity gives the claimed sheaf equality. □

Lemma 11.12.89 (Graph-cone cohomology range and support)**.** *Lean:* conditional-interface:lemma:weil-a4-graph-cone-range-support

Uses: def:weil-a4-graph-alteration-datum

For a graph-alteration datum and torsion F,

$$\mathcal{H}^q(Q_{t,F}) = 0 \quad (q \notin \{0,1,2\}), \qquad \operatorname{Supp} \mathcal{H}^q(Q_{t,F}) \subseteq Z_t.$$

Proof. Uses: lemma:weil-a4-proper-relative-curve-cd-two,lemma:weil-a4-normalization-unit-injective,lemma:weil-pbc-geometric-stalk-conservative

The graph projection b_t is proper, surjective, and has fibers of dimension at most one. The relative-curve amplitude lemma gives $R^q b_{t,*} b_t^{-1} F = 0$ for $q > 2$, while a right-derived functor has no negative cohomology. Surjectivity makes $F \to b_{t,*} b_t^{-1} F$ injective by the same geometric-stalk argument as for normalization. The long exact cohomology sequence of

$$F \longrightarrow R b_{t,*} b_t^{-1} F \longrightarrow Q_{t,F} \longrightarrow F[1]$$

therefore identifies

$$\mathcal{H}^0(Q_{t,F}) = \operatorname{Coker}(F \to b_{t,*} b_t^{-1} F), \quad \mathcal{H}^q(Q_{t,F}) = R^q b_{t,*} b_t^{-1} F \ (q = 1, 2),$$

and makes every other cohomology sheaf zero. Over U_t, the graph map is an isomorphism, so the unit and all positive direct images restrict to the identity and zero respectively. Thus all three sheaves are supported on Z_t. □

Lemma 11.12.90 (Finite-row hypercohomology bound for the graph cone)**.** *Lean:* conditional-interface:lemma:weil-a4-graph-cone-hypercohomology

Uses: def:weil-a4-graph-alteration-datum, def:weil-a4-strong-geometric-proper-cd

Assume ProperCD(e) *for every $e < d$. Then*

$$H^r(X, Q_{t,F}) = 0 \qquad (r > 2d - 2).$$

The hypercohomology filtration has exactly the three possible rows $q = 0, 1, 2$.

Proof. Uses: `lemma:weil-a4-graph-cone-range-support,lemma:weil-a4-closed-immersion-pushforward,def:weil-a4-strong-geometric-proper-cd`
Use the canonical truncation spectral sequence

$$E_2^{p,q} = H^p(X, \mathcal{H}^q(Q_{t,F})) \Longrightarrow H^{p+q}(X, Q_{t,F}).$$

The range lemma leaves only $q = 0, 1, 2$. Each surviving cohomology sheaf is supported on Z_t, so closed-immersion reconstruction writes it as $i_{t,*}A_q$ for a torsion sheaf A_q on Z_t, and

$$E_2^{p,q} = H^p(Z_t, A_q).$$

The graph datum gives $\dim Z_t \leq d-2$; properness of Z_t follows from its closed immersion into proper X. The induction hypothesis kills the last group for $p > 2(d-2)$. Hence a nonzero term satisfies $p+q \leq 2(d-2)+2 = 2d-2$. In each total degree there are only the three rows, so the abutment filtration is finite and the stated higher groups are zero. □

Lemma 11.12.91 (The graph triangle reduces cohomology to the alteration). *Lean:* `conditional-interface:lemma:weil-a4-graph-triangle-reduction`

Uses: `def:weil-a4-graph-alteration-datum,def:weil-a4-strong-geometric-proper-cd`
Under the hypotheses of the graph-cone hypercohomology lemma, the unit map induces an isomorphism

$$H^r(X, F) \xrightarrow{\sim} H^r(X_t, b_t^{-1}F) \qquad (r > 2d).$$

Proof. Uses: `lemma:weil-a4-graph-cone-hypercohomology,lemma:weil-a4-finite-morphism-cd`
The adjunction triangle gives the exact segment

$$H^{r-1}(X, Q_{t,F}) \longrightarrow H^r(X, F) \longrightarrow H^r(X, Rb_{t,*}b_t^{-1}F) \longrightarrow H^r(X, Q_{t,F}).$$

For $r > 2d$, both outer groups vanish because $r-1 > 2d-2$ and $r > 2d-2$. The middle right group equals $H^r(X_t, b_t^{-1}F)$ by the defining property of derived direct image. The middle arrow is therefore an isomorphism. □

Lemma 11.12.92 (Higher direct-image bound for the graph projection). *Lean:* `conditional-interface:lemma:weil-a4-graph-projection-direct-image-bound`

Uses: `def:weil-a4-graph-alteration-datum,def:weil-a4-strong-geometric-proper-cd`
Assume ProperCD(e) *for every* $e < d$. *Then*

$$R^q f_{t,*} b_t^{-1} F = 0 \qquad (q > 2d-2).$$

Proof. Uses: `prop:weil-proet-finite-proper-base-change,lemma:weil-pbc-finite-stalk-formula,lemma:weil-pbc-geometric-stalk-conservative,def:weil-a4-strong-geometric-proper-cd`
Fix a geometric point $\bar{y} \to \mathbf{P}^1$. Proper base change gives

$$(R^q f_{t,*} b_t^{-1} F)_{\bar{y}} = H^q((X_t)_{\bar{y}}, b_t^{-1}F|).$$

The graph datum gives $\dim(X_t)_{\bar{y}} \leq d-1$, and the fiber is proper over an algebraically closed field. The induction hypothesis ProperCD$(d-1)$ kills the right side for $q > 2(d-1)$. This holds at every geometric point, so stalk conservativity proves the direct-image vanishing. □

Lemma 11.12.93 (Finite-row Leray bound for the graph projection). *Lean:* `conditional-interface:lemma:weil-a4-graph-projection-leray-bound`

Uses: `def:weil-a4-graph-alteration-datum,def:weil-a4-strong-geometric-proper-cd`
Under the preceding induction hypotheses,

$$H^r(X_t, b_t^{-1}F) = 0 \qquad (r > 2d).$$

Proof. Uses: `lemma:weil-a4-graph-projection-direct-image-bound,lemma:weil-a4-relative-p1-cd-two`
Use the Leray spectral sequence

$$E_2^{p,q} = H^p(\mathbf{P}^1, R^q f_{t,*} b_t^{-1} F) \Longrightarrow H^{p+q}(X_t, b_t^{-1}F).$$

The direct-image lemma gives $0 \leq q \leq 2d-2$. Projective-line torsion cohomology vanishes for $p > 2$, so a nonzero term has $p+q \leq 2 + (2d-2) = 2d$. Both indices are bounded, making the abutment filtration finite. Thus its groups in total degree greater than $2d$ vanish. □

Proposition 11.12.94 (Strong geometric proper-cd induction step). *Lean:* `conditional-interface:prop:weil-a4-strong-proper-cd-induction-step`

Uses: `def:weil-a4-strong-geometric-proper-cd`

Let $d > 1$. If ProperCD(e) *holds for every $e < d$, then* ProperCD(d) *holds.*

Proof. Uses: `lemma:weil-a4-reduction-etale-equivalence,prop:weil-a4-normalization-descent-datum,prop:weil-a4-rational-function-datum,prop:weil-a4-graph-alteration-datum,lemma:weil-a4-graph-triangle-reduction,lemma:weil-a4-graph-projection-leray-bound,def:weil-a4-strong-geometric-proper-cd`

Let X be proper of dimension at most d over an algebraically closed field and F torsion. If $\dim X < d$, apply the induction hypothesis. Suppose $\dim X = d$. Reduction invariance replaces X by X_{red}.

Take the normalization-descent datum. Its exact sequence and the induction hypothesis on Z, whose dimension is at most $d-1$, show that for $r > 2d$ the map

$$H^r(X, F) \longrightarrow H^r(X^\nu, \nu^{-1}F)$$

is an isomorphism: the adjacent cokernel-cohomology groups occur in degrees $r-1, r$, both greater than $2(d-1)$. The scheme X^ν is a finite disjoint union of normal integral proper components, and cohomology over a finite disjoint union is their finite product. It is enough to treat one normal integral component, still denoted X.

Take a rational-function datum and its graph-alteration datum. The graph triangle reduction identifies $H^r(X, F)$ with $H^r(X_t, b_t^{-1}F)$ for $r > 2d$, and the graph-projection Leray lemma makes the latter group zero. This proves every instance of ProperCD(d). □

Lemma 11.12.95 (Strong geometric proper cd in dimensions zero and one). *Lean:* `conditional-interface:lemma:weil-a4-strong-proper-cd-base`

Uses: `def:weil-a4-strong-geometric-proper-cd`

The predicates ProperCD(0) *and* ProperCD(1) *hold.*

Proof. Uses: `lemma:weil-a4-finite-morphism-cd,lemma:weil-a4-sepclosed-field-cd-zero,lemma:weil-pbc-universal-homeomorphism-coverings,def:weil-pbc-eight-curve-assertions,lemma:weil-pbc-eight-curve-devissage,lemma:weil-pbc-curve-constructible-primary-reduction,lemma:weil-pbc-curve-dense-open-reduction,lemma:weil-pbc-curve-locally-constant-reduction,lemma:weil-pbc-finite-pushforward-extension-zero,lemma:weil-pbc-curve-normalization-constant,def:weil-a4-strong-geometric-proper-cd`

In dimension zero, the structure morphism to the algebraically closed field is finite. Finite-morphism exactness and field cd zero kill every positive cohomology group.

In dimension at most one, the A3 eight-assertion curve proof applies to every separated finite-type curve and arbitrary torsion sheaf. It gives $H^q(X, F) = 0$ for $q > 2$. Zero-dimensional components satisfy the stronger preceding bound, and the finite open–closed devissage in A3 combines the components. This is ProperCD(1). □

Proposition 11.12.96 (Strong geometric proper cohomological dimension). *Lean:* `conditional-interface:prop:weil-a4-strong-geometric-proper-cd`

Uses: `def:weil-a4-strong-geometric-proper-cd`

The predicate ProperCD(d) *holds for every $d \geq 0$.*

Proof. Uses: `lemma:weil-a4-strong-proper-cd-base,prop:weil-a4-strong-proper-cd-induction-step`

Use induction on d. The preceding base lemma handles $d = 0, 1$. For $d > 1$, the induction hypotheses in all smaller dimensions are exactly the assumptions of the strong induction-step proposition, which yields the next predicate. □

Lemma 11.12.97 (Base-change and Frobenius compatibility of the graph induction). *Lean:* `conditional-interface:lemma:weil-a4-proper-induction-base-change`

Uses: `def:weil-a4-normalization-descent-datum,def:weil-a4-graph-common-refinement`

The normalization and graph induction commute with extensions of algebraically closed base fields and with base-field automorphisms. For two rational-function choices, the induced zero identifications agree after the graph common refinement. Over $\overline{\mathbf{F}}_q$, these statements apply to geometric Frobenius.

Proof. Uses: `prop:weil-a4-graph-common-refinement,prop:weil-proet-finite-proper-base-change,lemma:weil-pbc-hypercohomology-map`

An extension between algebraically closed fields is faithfully flat and geometrically regular. For a finite-type normal scheme over the perfect base field, normality is geometric; hence normalization commutes with this base change by the universal property of integral closure. Reduced graph closure commutes with flat base change

because schematic closure is the kernel of restriction to the dense graph and flat tensor product preserves that kernel. The open domains, codimension bounds, and fiber-dimension bounds pull back to the corresponding data.

Finite proper base change identifies every derived direct-image term after base extension. Naturality of adjunction gives a morphism of graph triangles; the A3 hypercohomology exact-couple map identifies the induced maps on every truncation page and on the finite filtered abutment. The same argument applies to a base-field automorphism.

For two rational functions, the common-refinement proposition supplies a third graph alteration dominating both and its cocycle. Naturality makes the two diagrams from the original adjunction triangles commute after this pullback. Each proof chain concludes that the same group $H^r(X, F)$ is the zero group; an isomorphism from a zero group to the zero group is unique. Thus the final zero identification is independent of the graph choice even though no comparison map between the two graph cohomology groups is asserted to be invertible. Geometric Frobenius is a base-field automorphism of $\overline{\mathbf{F}}_q$, giving the last assertion. □

Definition 11.12.98 (Dimension-truncated proper-descent certificate)**.** Lean: `conditional-interface:def:weil-a4-proper-descent-certificate`

Uses: `def:weil-a4-affine-artin-vanishing-certificate,def:weil-a4-normalization-descent-datum,def:weil-a4-graph-alteration-datum,def:weil-a4-graph-common-refinement,def:weil-a4-exceptional-index`

Fix an affine Artin-vanishing certificate. A dimension-truncated proper-descent certificate assigns to every algebraically closed Ω, proper finite-type X/Ω of dimension at most d, and constructible finite Λ_n-sheaf G, a well-founded proof tree of height at most $d+1$, with finite data at each node and the following fields.

1. A reduction equivalence and, at every reduced node, a normalization-descent datum. Closed-complement children have strictly smaller dimension; normal components are the remaining children.
2. At every positive-dimensional normal integral node, a rational-function datum and graph-alteration datum. Its closed support child has dimension at most two less, while every fiber child over $\mathbf{P}^1$ has dimension at most one less.
3. The normalization long exact sequence, the graph-cone truncation spectral sequence
$$H^p(X, \mathcal{H}^q Q) \Longrightarrow H^{p+q}(X, Q), \qquad q \in \{0, 1, 2\},$$
and the graph-projection Leray spectral sequence
$$H^p(\mathbf{P}^1, R^q f_* b^{-1} G) \Longrightarrow H^{p+q}(X_t, b^{-1} G),$$
with every differential, edge map, finite row bound, and abutment filtration recorded.
4. The zero identifications supplied by the dimension induction, the comparison maps from both finite filtered abutments, and the final equality $H^r(X, G) = 0$ for $r > 2d$.
5. Common graph refinements, the cocycle for three choices, and the base-extension, automorphism, and geometric-Frobenius compatibility maps.

This is the normalization-and-graph proper alteration implementation of Stacks Tag `095U`. It does not posit a proper-hypercover descent theorem as an additional leaf.

Lemma 11.12.99 (The strong proper-cd proof supplies the descent certificate)**.** Lean: `conditional-interface:lemma:weil-a4-strong-proper-cd-to-certificate`

Uses: `def:weil-a4-proper-descent-certificate`

For every affine Artin-vanishing certificate, the strong geometric proper-cd proof supplies every field of a dimension-truncated proper-descent certificate.

Proof. Uses: `prop:weil-a4-strong-geometric-proper-cd,lemma:weil-a4-reduction-etale-equivalence,prop:weil-a4-normalization-descent-datum,prop:weil-a4-rational-function-datum,prop:weil-a4-graph-alteration-datum,lemma:weil-a4-graph-cone-hypercohomology,lemma:weil-a4-graph-projection-leray-bound,prop:weil-a4-graph-common-refinement,lemma:weil-a4-proper-induction-base-change`

Build the tree by strong induction on dimension. At a node, attach the reduction equivalence and normalization datum. Its closed-complement child has smaller dimension. Split the normalization into its finite list of normal integral components. In dimensions zero and one, attach the base proof and stop. In larger dimension, attach a rational-function and graph datum. The graph support child drops dimension by two and every geometric fiber child drops dimension by one, so every branch terminates.

Attach to each graph node the three-row cone spectral sequence and the bounded graph Leray spectral sequence from the two named lemmas. Their proofs specify the zero entries, all edge comparisons, and finite abutment filtrations. The strong proper-cd proposition supplies the final zero at the root. Attach common refinements and their cocycles for alternative rational functions, and attach the base-change and Frobenius maps from the compatibility lemma. These are exactly the five fields of the certificate definition. □

Proposition 11.12.100 (Dimension-truncated proper descent). Lean: `conditional-interface:prop:weil-a4-proper-descent-certificate`

Uses: `def:weil-a4-proper-descent-certificate`

Every affine Artin-vanishing certificate extends to a dimension-truncated proper-descent certificate.

Proof. Uses: `lemma:weil-a4-strong-proper-cd-to-certificate`

Apply the preceding lemma to the fixed affine Artin-vanishing certificate and take the proof tree, alteration data, spectral sequences, zero identifications, refinements, and compatibility maps constructed there. □

Definition 11.12.101 (Geometric proper Artin predicate). Lean: `conditional-interface:def:weil-a4-geometric-proper-artin-predicate`

Uses: `def:weil-a4-exceptional-index`

For $d \in \mathbf{N}$, let $\mathrm{GeomArtin}(\Lambda_n; d)$ mean that for every algebraically closed field Ω of characteristic different from ℓ, every proper finite-type Ω-scheme Z of dimension at most d, and every constructible finite Λ_n-sheaf G,

$$H^r(Z_{\acute{e}t}, G) = 0 \qquad (r > 2d).$$

Lemma 11.12.102 (Proper descent gives geometric Artin vanishing). Lean: `conditional-interface:lemma:weil-a4-proper-descent-geometric-artin`

Uses: `def:weil-a4-proper-descent-certificate`, `def:weil-a4-geometric-proper-artin-predicate`

An affine Artin-vanishing certificate and a dimension-truncated proper-descent certificate imply $\mathrm{GeomArtin}(\Lambda_n; d)$ *for every* d.

Proof. Uses: `def:weil-a4-proper-descent-certificate`, `def:weil-a4-geometric-proper-artin-predicate`

Fix Z, G, d as in the geometric predicate and take the descent data assigned by the certificate. If $r > 2d$, every bidegree (a, b) with $a + b = r$ has a finite filtration whose graded pieces carry the certificate's zero identification. Induction through the finite filtration gives $E_1^{a,b} = 0$. The differentials preserve total degree, so every later page in total degree r is zero. The finite-row field makes the filtration on the abutment finite. Each associated graded piece of $H^r(Z, G)$ is therefore zero, hence $H^r(Z, G) = 0$. This is the required instance of $\mathrm{GeomArtin}(\Lambda_n; d)$. □

Lemma 11.12.103 (Dimension-zero geometric Artin base case). Lean: `conditional-interface:lemma:weil-a4-geometric-artin-dimension-zero`

Uses: `def:weil-a4-geometric-proper-artin-predicate`

$\mathrm{GeomArtin}(\Lambda_n; 0)$ *holds.*

Proof. Uses: `def:weil-a4-geometric-proper-artin-predicate`

Let Z be proper of dimension zero over an algebraically closed field. Its reduction is a finite disjoint union of spectra of finite field extensions, hence a finite disjoint union of geometric points. The small etale topoi of Z and Z_{red} are equivalent because a nilpotent closed immersion induces an equivalence on etale schemes. Sections of a sheaf on a finite disjoint union of points are a finite product of its stalks. Finite products of Λ_n-modules are exact, so their right-derived functors vanish in every positive degree. Thus $H^r(Z, G) = 0$ for $r > 0 = 2 \cdot 0$. □

Lemma 11.12.104 (Proper-curve geometric Artin base case). Lean: `conditional-interface:lemma:weil-a4-geometric-artin-curve`

Uses: `def:weil-a4-geometric-proper-artin-predicate`

$\mathrm{GeomArtin}(\Lambda_n; 1)$ *holds.*

Proof. Uses: `def:weil-pbc-eight-curve-assertions`, `prop:weil-pbc-proper-curve-sepclosed-invariance`, `def:weil-a4-geometric-proper-artin-predicate`

Fix a proper curve Z over an algebraically closed field and a finite constructible sheaf G. The A3 curve package first reduces G, by a finite constructible filtration and the normalization/open–closed exact sequence, to the locally constant normal-curve case. Its proper separably-closed invariance proposition gives

$$H^r(Z, G) = 0 \qquad (r > 2).$$

The long exact sequences used in the reduction preserve the same upper bound. Since $2 = 2 \dim Z$, this is the $d = 1$ instance of the geometric predicate. □

Lemma 11.12.105 (Geometric vanishing gives relative higher-direct-image vanishing). *Lean:* *conditional-interface:lemma:weil-a4-geometric-to-relative-artin*

Uses: *def:weil-a4-geometric-proper-artin-predicate, def:weil-a4-proper-relative-dimension-certificate, def:weil-a4-finite-origin-compactification-input*

Assume d is a relative-dimension certificate for p_f and $\mathrm{GeomArtin}(\Lambda_n; d)$. *Then every constructible finite Λ_n-sheaf F on $\overline{X}_{f,\acute{e}t}$ satisfies*

$$R^r p_{f,\acute{e}t,*} F = 0 \qquad (r > 2d).$$

Proof. Uses: `prop:weil-proet-finite-proper-base-change,lemma:weil-pbc-finite-stalk-formula,lemma:weil-pbc-geometric-stalk-conservative,def:weil-a4-geometric-proper-artin-predicate,def:weil-a4-proper-relative-dimension-certificate`

Fix $r > 2d$ and a geometric point $\bar{y} \to Y$. Finite proper base change identifies the geometric stalk with

$$(R^r p_{f,\acute{e}t,*} F)_{\bar{y}} \simeq H^r((\overline{X}_f)_{\bar{y},\acute{e}t}, F|_{(\overline{X}_f)_{\bar{y}}}).$$

The relative-dimension certificate gives $\dim(\overline{X}_f)_{\bar{y}} \le d$, and the geometric Artin predicate makes the group on the right zero. This holds at every geometric point. Geometric stalks are conservative for sheaves on the small etale site, so $R^r p_{f,\acute{e}t,*} F = 0$. □

Definition 11.12.106 (Proper smooth curve specialization certificate). *Lean:* `conditional-interface:def:weil-a4-proper-smooth-curve-specialization-certificate`

Uses: `def:weil-a4-exceptional-index`

A proper smooth curve specialization certificate assigns to every noetherian finite-dimensional scheme T on which ℓ is invertible, every proper smooth morphism $h : C \to T$ of relative dimension one with geometrically connected fibers, and every finite locally constant Λ_n-sheaf G on $C_{\acute{e}t}$, the following data:

1. finite Λ_n-sheaves $\mathcal{V}^r$ on $T_{\acute{e}t}$ for $0 \le r \le 2$, with $\mathcal{V}^r = 0$ outside this interval;
2. maps $\mathcal{V}^r \to R^r h_* G$ and $R^r h_* G \to \mathcal{V}^r$ whose composites are identities;
3. a finite stratification $T = \coprod_\alpha T_\alpha$ on which every $\mathcal{V}^r$ is finite locally constant;
4. for every geometric $\bar{t} \to T$, inverse comparison maps

$$(\mathcal{V}^r)_{\bar{t}} \;\rightleftarrows\; H^r(C_{\bar{t},\acute{e}t}, G|_{C_{\bar{t}}})$$

compatible with specialization, finite base change, and the two identifications in item 2.

Definition 11.12.107 (Affine-line constructible devissage tree). *Lean:* `conditional-interface:def:weil-a4-affine-line-devissage-tree`

Uses: `def:weil-a4-proper-smooth-curve-specialization-certificate,def:weil-a4-exceptional-index`

Fix a proper smooth curve specialization certificate. An affine-line constructible devissage tree assigns to every noetherian finite-dimensional T, the projection $\pi : \mathbf{A}^1_T \to T$, and every constructible finite Λ_n-sheaf F, a finite rooted tree with the following data.

1. The root is $(T, \mathbf{A}^1_T, F)$. A non-leaf vertex is equipped with a nonempty open $V \subseteq T'$, its reduced closed complement T'', and the two base changes of the vertex to V and T''. The closed child has strictly smaller noetherian closed support.
2. Over V, a finite stratification of $\mathbf{A}^1_V$ is given. Each stratum is either finite over V, or is a smooth relative curve carrying a finite filtration of the restricted sheaf whose graded pieces are finite locally constant.
3. Every smooth relative-curve leaf has a finite surjective base change $V' \to V$, and for each finite locally constant graded sheaf G, a finite etale cover $a : U' \to U_{V'}$ for which a^*G is constant. The data include a normal proper smooth geometrically connected compactification $U' \hookrightarrow C \to V'$, its finite boundary, the extension of the constant sheaf to C, and maps exhibiting $G|_{U_{V'}}$ as a subquotient of $a_* a^* G$. They also include descent maps, the two descent composites, and the cocycle on $V' \times_V V' \times_V V'$.
4. Every internal sheaf filtration is accompanied by its short exact sequences. Every open–closed split is accompanied by the exact triangle
$$j_! j^* F \longrightarrow F \longrightarrow i_* i^* F \longrightarrow$$
and every finite cover by its equalizer and cocycle maps.
5. Compact-support composition maps for finite morphisms, closed immersions, affine-line projections, and the displayed curve compactifications are included, together with their inverse equations. Applying these maps to a leaf reduces its compact-support direct image either to finite pushforward or to the proper smooth curve specialization certificate.

The well-founded rank of a closed child, all face maps, and all compatibility equations are fields of the tree.

Definition 11.12.108 (Relative-curve constructibility seed). Lean: `conditional-interface:def:weil-a4-relative-curve-constructibility-seed`

Uses: `def:weil-a4-proper-smooth-curve-specialization-certificate`, `def:weil-a4-affine-line-devissage-tree`

A relative-curve constructibility seed is a proper smooth curve specialization certificate together with an affine-line constructible devissage tree for that certificate.

Definition 11.12.109 (Relative-curve specialization candidate). Lean: `conditional-interface:def:weil-a4-relative-curve-specialization-candidate`

Uses: `def:weil-a4-proper-smooth-curve-specialization-certificate`

For a noetherian finite-dimensional T, a proper smooth geometrically connected curve $h : C \to T$, and a finite locally constant Λ_n-sheaf G on $C_{\acute{e}t}$, a specialization candidate consists of sheaves $\mathcal{V}^0, \mathcal{V}^1, \mathcal{V}^2$ on $T_{\acute{e}t}$, maps in both directions

$$\mathcal{V}^r \rightleftarrows R^r h_* G,$$

whose two composites are identities, and for every geometric point $\bar{t} \to T$ maps in both directions

$$(\mathcal{V}^r)_{\bar{t}} \rightleftarrows H^r(C_{\bar{t},\acute{e}t}, G|_{C_{\bar{t}}})$$

with the specialization and finite-base-change squares as explicit fields.

Proposition 11.12.110 (Relative-curve specialization candidate exists – OPEN). Lean: `conditional-interface:prop:weil-a4-relative-curve-specialization-candidate-open`

Uses: `def:weil-a4-relative-curve-specialization-candidate`

OPEN. *For every T, h, G as in the candidate definition, a specialization candidate exists with finite constructible $\mathcal{V}^r$ and with all displayed inverse comparison maps functorial in G.*

Open obligation: `OBL-A4-RC-01`

Proof. Uses: `def:weil-a4-relative-curve-specialization-candidate`

The closed A3 curve computation supplies the three fiber groups only after the base is separably closed. The missing relative assertion is the construction of the sheaves $\mathcal{V}^r$, their finite stratification, and the specialization and finite-base-change maps. The exact source interface is the proper-smooth specialization part of Stacks Tags $0GKU$–$0GKW$; it is not a theorem in the current foundation boundary. □

Definition 11.12.111 (Relative-curve finite-monodromy candidate). Lean: `conditional-interface:def:weil-a4-relative-curve-finite-monodromy-candidate`

Uses: `def:weil-a4-relative-curve-specialization-candidate`

For a smooth relative-curve stratum $U \to V$ and a finite locally constant Λ_n-sheaf G on $U_{\acute{e}t}$, a finite-monodromy candidate consists of a finite surjective morphism $V' \to V$, a finite etale cover $a : U' \to U \times_V V'$, a specified isomorphism from a^*G to a constant finite Λ_n-sheaf, and the pullback maps on the two projections from $V' \times_V V'$.

Proposition 11.12.112 (Relative-curve finite monodromy exists – OPEN). Lean: `conditional-interface:prop:weil-a4-relative-curve-finite-monodromy-open`

Uses: `def:weil-a4-relative-curve-finite-monodromy-candidate`

OPEN. *Every finite locally constant coefficient on a smooth relative-curve stratum admits a finite-monodromy candidate, functorially after refinement and compatibly on double and triple overlaps.*

Open obligation: `OBL-A4-RC-02`

Proof. Uses: `def:weil-a4-relative-curve-finite-monodromy-candidate`

One must construct a finite quotient of the geometric fundamental-group representation, spread it over a finite surjective base change, and prove that the chosen trivialization agrees on the two projections and satisfies the triple-overlap cocycle. Stacks Tags $0GKU$–$0GKV$ contain the geometric finite-monodromy input but do not provide these functorial fields inside the present A1–A3 interface, so this construction remains OPEN. □

Definition 11.12.113 (Relative-curve compactification candidate). Lean: `conditional-interface:def:weil-a4-relative-curve-compactification-candidate`

Uses: `def:weil-a4-relative-curve-finite-monodromy-candidate`

For the finite-monodromy candidate above, a compactification candidate consists of a normal proper smooth geometrically connected curve $U' \hookrightarrow C \xrightarrow{\bar{h}} V'$, a finite relative boundary $B = C \setminus U'$, an extension of the constant coefficient to C, and the open–closed exact triangle identifying the coefficient on U' with the restriction of that extension and the boundary term supported on B. It also includes the degree-zero, degree-one, and degree-two comparison maps to the specialization candidate.

Proposition 11.12.114 (Relative-curve compactification exists – OPEN)**.** *Lean:* `conditional-interface:prop:weil-a4-relative-curve-compactification-open`

Uses: `def:weil-a4-relative-curve-compactification-candidate`

OPEN. *Every finite-monodromy candidate admits a compactification candidate after a further finite surjective base change, with finite boundary, proper smoothness, and all restriction and comparison maps displayed in the definition.*

Open obligation: `OBL-A4-RC-03`

Proof. Uses: `def:weil-a4-relative-curve-compactification-candidate`

The unresolved steps are relative compactification of the smooth affine curve, preservation of smoothness and geometric connectedness after the chosen base change, finiteness of the boundary, and extension of the coefficient together with the exact open–closed triangle. Stacks Tags $0GKV$–$0GKY$ organize these operations, but their descent and compatibility assertions are not foundation leaves here. □

Definition 11.12.115 (Relative-curve descent and cocycle candidate)**.** Lean: `conditional-interface:def:weil-a4-relative-curve-descent-candidate`

Uses: `def:weil-a4-relative-curve-compactification-candidate`

A descent candidate for a compactification candidate consists of comparison maps on $V' \times_V V'$ between the two pullbacks of every coefficient, maps on $V' \times_V V' \times_V V'$, the two descent-composite identities, the triple-overlap cocycle, and commutative squares for restriction, open–closed triangles, specialization maps, and compact-support direct images. Its descended kernel/equalizer is part of the data.

Proposition 11.12.116 (Relative-curve descent candidate exists – OPEN)**.** *Lean:* `conditional-interface:prop:weil-a4-relative-curve-descent-open`

Uses: `def:weil-a4-relative-curve-descent-candidate`

OPEN. *Every compactification candidate admits a descent candidate, and its descended equalizer is a finite constructible coefficient whose proper-curve comparison maps are independent of all finite refinements.*

Open obligation: `OBL-A4-RC-04`

Proof. Uses: `def:weil-a4-relative-curve-descent-candidate`

The missing proof must construct the two projection maps, verify both descent composites and the triple cocycle, then descend the coefficient and each comparison map through the equalizer. It must additionally prove independence under a common finite refinement. These are the descent and coherence portions of Stacks Tags $0GKY$–$0GKZ$, not currently available in mathlib or in the closed A1–A3 declarations. □

Definition 11.12.117 (Affine-line devissage existence candidate)**.** Lean: `conditional-interface:def:weil-a4-affine-line-devissage-existence-candidate`

Uses: `def:weil-a4-relative-curve-descent-candidate`

For each noetherian finite-dimensional T and constructible finite sheaf on $\mathbf{A}^1_T$, a devissage-existence candidate consists of a finite rooted tree, a strictly descending closed-support rank, finite strata on every open child, finite leaves, smooth relative-curve leaves carrying the four previous candidates, and all short exact sequences, exact triangles, equalizer maps, compact-support composition maps, and inverse equations required by the affine-line devissage tree.

Proposition 11.12.118 (Affine-line devissage tree exists – OPEN)**.** *Lean:* `conditional-interface:prop:weil-a4-affine-line-devissage-existence-open`

Uses: `def:weil-a4-affine-line-devissage-existence-candidate`

OPEN. *Every constructible finite coefficient on $\mathbf{A}^1_T$ admits an affine-line devissage-existence candidate, with finite height and all face, descent, and compact-support compatibility equations.*

Open obligation: `OBL-A4-RC-05`

Proof. Uses: `def:weil-a4-affine-line-devissage-existence-candidate`

The missing construction is noetherian induction on the support, followed by finite stratification into finite and smooth relative-curve pieces. One must prove termination, construct the finite covers and compactifications on every smooth leaf, and glue the local exact triangles and composition maps into a finite tree. Stacks Tags $0GKZ$ and the adjacent constructibility lemmas state this route but do not supply the explicit tree fields required here. □

Proposition 11.12.119 (Missing relative-curve constructibility seed). *Lean:* `conditional-interface:prop:weil-a4-relative-curve-constructibility-seed-open`

Uses: `def:weil-a4-relative-curve-constructibility-seed`, `def:weil-a4-relative-curve-specialization-candidate`, `def:weil-a4-relative-curve-finite-monodromy-candidate`, `def:weil-a4-relative-curve-compactification-candidate`, `def:weil-a4-relative-curve-descent-candidate`, `def:weil-a4-affine-line-devissage-existence-candidate`

OPEN. *A relative-curve constructibility seed exists for finite Λ_n-coefficients over noetherian finite-dimensional schemes on which ℓ is invertible.*

Open obligation: `OBL-A4-EX-01`

Proof. Uses: `prop:weil-a4-relative-curve-specialization-candidate-open`, `prop:weil-a4-relative-curve-finite-monodromy-open`, `prop:weil-a4-relative-curve-compactification-open`, `prop:weil-a4-relative-curve-descent-open`, `prop:weil-a4-affine-line-devissage-existence-open`, `def:weil-a4-relative-curve-constructibility-seed`

The seed is exactly the conjunction of the five preceding construction propositions: the specialization candidate supplies the finite degree sheaves and fiber comparisons; the finite-monodromy candidate supplies the trivializing finite cover; the compactification candidate supplies the proper smooth curve and finite boundary; the descent candidate supplies the equalizer, cocycle, and refinement independence; and the devissage candidate assembles these leaves into a finite tree. Since each of the five required existence propositions is OPEN, the aggregate seed remains OPEN, with no unstated existence step. □

Lemma 11.12.120 (Finite constructible sheaves form a Serre subcategory). *Lean:* `conditional-interface:lemma:weil-a4-finite-constructible-serre`

Uses: `def:weil-a4-exceptional-index`

On the small etale site of a noetherian scheme, kernels, cokernels, images, finite direct sums, and extensions of constructible sheaves of finite Λ_n-modules are constructible.

Proof. Uses: `def:weil-a4-exceptional-index`

For finitely many input sheaves, intersect their finite constructible stratifications. On a connected stratum each restriction is represented by a finite continuous Λ_n-module for the etale fundamental group. The category of finite modules with continuous action is abelian: kernels, images, and cokernels are finite invariant submodules or quotients, and an extension has the diagonal continuous action on a finite module. Thus each listed operation is finite locally constant on every common stratum. The common stratification is finite, proving constructibility. □

Lemma 11.12.121 (Finite direct image is exact and constructible). *Lean:* `conditional-interface:lemma:weil-a4-finite-direct-image-constructible`

Uses: `def:weil-a4-exceptional-index`

Let $v : Z \to T$ be finite and let G be a constructible finite Λ_n-sheaf on $Z_{\acute{e}t}$. Then v_*G is constructible and

$$R^r v_* G = 0 \qquad (r > 0).$$

Proof. Uses: `lemma:weil-a4-finite-constructible-serre`

After a finite stratification of T, the finite morphism decomposes into finite radicial and finite etale pieces and G is finite locally constant on a finite stratification of their sources. A radicial finite morphism induces an equivalence of small etale topoi. On a finite etale piece, the geometric stalk of v_*G is the finite product of the stalks over the finite geometric fiber. These products are finite locally constant as the fiber is permuted by finite monodromy. Hence v_*G is constructible. For a short exact sequence, the stalk sequence of these finite products is exact. Geometric stalks are conservative, so v_* is exact and its positive right-derived functors vanish. □

Lemma 11.12.122 (A relative-curve seed computes every leaf). *Lean:* `conditional-interface:lemma:weil-a4-relative-curve-seed-leaves`

Uses: `def:weil-a4-relative-curve-constructibility-seed`

For every leaf in the affine-line devissage tree, all compact-support higher direct images of every displayed graded sheaf are constructible and vanish outside a finite interval.

Proof. Uses: `def:weil-a4-relative-curve-constructibility-seed`, `lemma:weil-a4-finite-direct-image-constructible`, `lemma:weil-a4-finite-constructible-serre`

For a finite leaf, use exact finite pushforward; its only nonzero direct image is in degree zero. For a smooth curve leaf, pass to V', then to the stored finite etale cover $a : U' \to U_{V'}$. The chosen isomorphism makes a^*G constant, and the stored subquotient maps reduce G to the finite pushforward a_*a^*G. Extend the constant sheaf from U' to the proper smooth compactification C. The open–closed triangle has a finite-boundary term, handled by finite pushforward, and a constant proper smooth term, identified degree by degree with the constructible

sheaves in the specialization certificate. Apply the subquotient presentation and the finite filtration one short exact sequence at a time. The resulting long exact sequence has constructible neighboring terms, so the Serre lemma makes the middle kernels and cokernels constructible. The descent equalizer and cocycle stored at the leaf recover the object on the original base; their kernel is constructible by the same Serre lemma. The finite term, the degrees $0, 1, 2$ of the proper curve, and the one connecting morphism give a finite cohomological interval. □

Lemma 11.12.123 (The devissage tree computes affine-line compact support). *Lean:* `conditional-interface: lemma: weil-a4-affine-line-constructibility`

Uses: `def: weil-a4-relative-curve-constructibility-seed`

Given a relative-curve constructibility seed, for every noetherian finite-dimensional T, every constructible finite Λ_n-sheaf F on $\mathbf{A}^1_T$, and every r,

$$R^r\pi_!F$$

is constructible on T, and it is zero outside a finite interval depending on the finite devissage tree.

Proof. Uses: `lemma:weil-a4-relative-curve-seed-leaves,lemma:weil-a4-finite-constructible-serre,def:weil-a4-affine-line-devissage-tree`

Induct upward from the leaves of the finite tree. The preceding lemma settles each leaf. At a filtration vertex, apply $R\pi_!$ to one stored short exact sequence. In every degree, the long exact sequence expresses the unknown term as an extension of a cokernel by a kernel of maps between the two already computed terms; the Serre lemma preserves constructibility. At an open–closed vertex, use the stored exact triangle and the same kernel–cokernel calculation. At a finite-descent vertex, use the stored equalizer and cocycle, whose terms have already been computed after base change. Each operation enlarges the finite cohomological interval by at most one. Since the tree has finite height, the root has a finite interval and constructible cohomology in every degree. The root is $R\pi_!F$. □

Lemma 11.12.124 (Affine-space compact-support constructibility). *Lean:* `conditional-interface: lemma: weil-a4-affine-space-constructibility`

Uses: `def: weil-a4-relative-curve-constructibility-seed`

Given a relative-curve constructibility seed, let $\pi_N : \mathbf{A}^N_T \to T$. For every constructible finite Λ_n-sheaf F, all $R^r\pi_{N,!}F$ are constructible and vanish outside a finite interval.

Proof. Uses: `lemma:weil-a4-affine-line-constructibility,lemma:weil-a4-finite-constructible-serre,def:weil-a4-affine-line-devissage-tree`

Induct on N. For $N = 0$, π_0 is the identity. For $N + 1$, factor

$$\mathbf{A}^{N+1}_T \xrightarrow{\rho} \mathbf{A}^N_T \xrightarrow{\pi_N} T$$

with ρ an affine-line projection. The composition maps stored in the seed identify $R\pi_{N+1,!}$ with $R\pi_{N,!}R\rho_!$. The affine-line lemma gives a bounded complex with constructible cohomology for $R\rho_!F$. Apply the bounded hypercohomology spectral sequence

$$R^a\pi_{N,!}\mathcal{H}^b(R\rho_!F) \Longrightarrow R^{a+b}\pi_{N+1,!}F.$$

The induction hypothesis makes every E_2-term constructible and leaves only finitely many bidegrees. Kernels, cokernels, and the finite abutment filtration are constructible by the Serre lemma. The finite rectangle also gives a finite vanishing interval. □

Lemma 11.12.125 (Closed immersion preserves constructibility). *Lean:* `conditional-interface: lemma: weil-a4-closed-immersion-constructible`

Uses: `def: weil-a4-exceptional-index`

For a closed immersion $i : Z \hookrightarrow W$, the direct image i_* is exact, sends constructible finite sheaves to constructible finite sheaves, and agrees with $i_!$.

Proof. Uses: `lemma:weil-a4-finite-direct-image-constructible`

A closed immersion of finite-type noetherian schemes is finite. Apply the finite-direct-image lemma for exactness and constructibility. A section extended from Z has support contained in the closed subset Z, so the support condition defining $i_!$ imposes no additional restriction. Thus the sectionwise maps $i_!G \rightleftarrows i_*G$ are both identities on objects over Z and both zero off Z; they are inverse. □

Lemma 11.12.126 (Affine finite-presentation compact-support constructibility). *Lean:* `conditional-interface: lemma: weil-a4-affine-morphism-constructibility`

Uses: `def: weil-a4-relative-curve-constructibility-seed`

Given a relative-curve constructibility seed, let $g : U \to T$ be affine of finite presentation between noetherian finite-dimensional schemes. For every constructible finite Λ_n-sheaf G, all $R^r g_!G$ are constructible and vanish outside a finite interval.

Proof. Uses: `lemma:weil-a4-affine-space-constructibility,lemma:weil-a4-closed-immersion-constructible,def:weil-a4-affine-line-devissage-tree`

Choose finitely many algebra generators for the finite-presentation $\mathcal{O}_T$-algebra defining U. They give a closed immersion

$$i : U \hookrightarrow \mathbf{A}^N_T$$

whose composite with π_N is g. The closed-immersion lemma makes $i_!G = i_*G$ a constructible sheaf with no positive direct images. The compact-support composition map stored in the seed identifies

$$Rg_!G \simeq R\pi_{N,!}i_*G.$$

Apply affine-space constructibility to the right side. Its finite cohomological interval is the required interval for g. □

Lemma 11.12.127 (Finite affine Cech devissage for a proper map)**.** *Lean:* `conditional-interface:lemma:weil-a4-proper-finite-affine-cech`

Uses: `def:weil-a4-relative-curve-constructibility-seed,def:weil-a4-finite-origin-compactification-input`

Given a relative-curve constructibility seed, let $p : Z \to T$ be proper of finite presentation between noetherian finite-dimensional schemes. For every constructible finite Λ_n-sheaf G, all $R^r p_*G$ are constructible.

Proof. Uses: `lemma:weil-a4-affine-morphism-constructibility,lemma:weil-a4-finite-constructible-serre`

Constructibility is local on T, so replace T by an affine open. Quasi-compactness of Z gives a finite affine open cover $Z = \bigcup_{\nu=1}^m U_\nu$. Since Z is separated over T, every intersection

$$U_I = \bigcap_{\nu \in I} U_\nu$$

is affine; its morphism $g_I : U_I \to T$ is affine of finite presentation. For the open immersion $j_I : U_I \hookrightarrow Z$, the ordered Cech complex

$$0 \longrightarrow G \longrightarrow \bigoplus_{|I|=1} j_{I,!}G|_{U_I} \longrightarrow \cdots \longrightarrow \bigoplus_{|I|=m} j_{I,!}G|_{U_I} \longrightarrow 0$$

is exact on every geometric stalk: the summands containing the point form the augmented simplex on a nonempty finite set, and its alternating contraction is insertion of the least index containing that point. Because p is proper, the compactification $U_I \xrightarrow{j_I} Z \xrightarrow{p} T$ gives the identity

$$Rp_*j_{I,!} = Rg_{I,!}$$

on the same degreewise complex. The affine-morphism lemma makes the cohomology of each term constructible. The spectral sequence of the finite Cech complex has only m columns; its kernels, cokernels, and finite abutment filtration are constructible by the Serre lemma. Its abutment is $R^r p_*G$, proving the claim. □

Proposition 11.12.128 (Conditional finite Artin–constructibility package)**.** *Lean:* `conditional-interface:prop:weil-a4-artfin-package-conditional`

Uses: `def:weil-a4-affine-artin-vanishing-certificate,def:weil-a4-proper-descent-certificate,def:weil-a4-relative-curve-constructibility-seed,def:weil-a4-finite-artin-constructibility-predicate,def:weil-a4-proper-relative-dimension-certificate`

Assume an affine Artin-vanishing certificate, a dimension-truncated proper-descent certificate, and a relative-curve constructibility seed. For every relative-dimension certificate d for p_f,

$$\mathrm{ArtFin}(p_f, \Lambda_n; d)$$

holds.

Proof. Uses: `lemma:weil-a4-proper-descent-geometric-artin,lemma:weil-a4-geometric-to-relative-artin,lemma:weil-a4-proper-finite-affine-cech,def:weil-a4-finite-artin-constructibility-predicate`

The proper-descent lemma converts the first two certificates into $\mathrm{GeomArtin}(\Lambda_n; d)$. The relative proper-base-change lemma then gives

$$R^r p_{f,*}F = 0 \qquad (r > 2d)$$

for every constructible finite sheaf F. Apply the finite affine Cech lemma to the proper morphism p_f; it makes every $R^r p_{f,*}F$ constructible, in particular for $0 \le r \le 2d$. These are precisely the two clauses of $\mathrm{ArtFin}(p_f, \Lambda_n; d)$. □

Lemma 11.12.129 (Conditional inhabitedness of the selected Artin set). *Lean:* `conditional-interface:lemma:weil-a4-selected-artin-set-inhabited-conditional`

Uses: `def:weil-a4-selected-finite-artin-bound-candidate`, `def:weil-a4-affine-artin-vanishing-certificate`, `def:weil-a4-proper-descent-certificate`, `def:weil-a4-relative-curve-constructibility-seed`, `def:weil-a4-total-dimension-relative-bound`

Assume the three certificates in the preceding proposition. Then $D_f \in S_f$, so S_f is inhabited and

$$d_f^{\mathrm{fin}} \in S_f.$$

Proof. Uses: `lemma:weil-a4-total-dimension-relative-bound`, `prop:weil-a4-artfin-package-conditional`, `def:weil-a4-selected-finite-artin-bound-candidate`

The total-dimension lemma supplies the fiber inequalities for D_f. Instantiate the conditional ArtFin package with this relative-dimension certificate; it supplies $\mathrm{ArtFin}(p_f, \Lambda_n; D_f)$. Hence $D_f \in S_f$. By the definition of d_f^{fin}, the inhabited branch is active and $d_f^{\mathrm{fin}} = \min S_f$. The minimum of an inhabited subset of $\mathbf{N}$ belongs to that subset, giving $d_f^{\mathrm{fin}} \in S_f$. □

Lemma 11.12.130 (Uniform amplitude is compatible with compactification refinement). *Lean:* `conditional-interface:lemma:weil-a4-artfin-refinement-uniform-amplitude`

Uses: `def:weil-a4-finite-artin-constructibility-predicate`, `def:weil-a4-finite-origin-compactification-input`, `def:weil-a1-refinement-underived-map`

Let

$$q : \overline{X}_D \longrightarrow \overline{X}_C$$

be a proper refinement of two compactifications of f. Assume $\mathrm{ArtFin}(p_C, \Lambda_n; d_C)$ *and* $\mathrm{ArtFin}(p_D, \Lambda_n; d_D)$. *For $K \in D_c^{[a,b]}(X, \Lambda_n)$, the two compactification models lie in*

$$D_c^{[a,b+2d_C]}(Y, \Lambda_n) \quad \text{and} \quad D_c^{[a,b+2d_D]}(Y, \Lambda_n),$$

and the refinement comparison commutes with their etale/pro-etale comparison maps.

Proof. Uses: `lemma:weil-a4-artin-constructibility-amplitude`, `lemma:weil-a4-a3-proper-piece-coherence`, `lemma:weil-a4-a3-proper-piece-comparison-iso`, `def:weil-a1-refinement-underived-map`

Apply the Artin-amplitude lemma separately to $j_{C,!}K$ and $j_{D,!}K$. This gives the two displayed intervals. The underived refinement map

$$q_* j_{D,!} K \longrightarrow j_{C,!} K$$

is the identity on a local section over X and zero on both sides over the complement. The A3 proper-piece coherence lemma states that applying the two proper direct images and then the site-comparison maps gives the same morphism as first applying the site-comparison maps and then the refinement. The proper-piece comparison maps are isomorphisms, so this is the required commuting refinement square. No maximum of d_C, d_D is needed; if one uniform interval is desired, both intervals embed in $[a, b + 2\max(d_C, d_D)]$. □

Proposition 11.12.131 (Missing finite Artin bound and constructibility). *Lean:* `conditional-interface:prop:weil-a4-selected-finite-artin-bound-open`

Uses: `def:weil-a4-selected-finite-artin-bound-candidate`

OPEN. *The selected integer belongs to S_f; equivalently it is a relative-dimension certificate and satisfies* $\mathrm{ArtFin}(p_f, \Lambda_n; d_f^{\mathrm{fin}})$.

Open obligation: `OBL-A4-EX-01`

Proof. Uses: `prop:weil-a4-affine-artin-vanishing`, `prop:weil-a4-proper-descent-certificate`, `prop:weil-a4-relative-curve-constructibility-seed-open`, `lemma:weil-a4-selected-artin-set-inhabited-conditional`

The first OPEN proposition would supply the affine Artin-vanishing certificate. The second would extend it to the dimension-truncated proper descent certificate. The third would supply the relative-curve constructibility seed. With exactly these three inputs, the conditional inhabitedness lemma proves

$$D_f \in S_f, \qquad d_f^{\mathrm{fin}} = \min S_f \in S_f.$$

Thus the selected proposition has been reduced to three named geometric leaves. The total-dimension bound, passage from geometric cohomology to higher direct images, constructible devissage, finite affine Cech calculation, uniform amplitude, and refinement compatibility are already proved above. None of the three existence propositions is in mathlib or in the closed A1–A3 boundary, so the aggregate proposition remains OPEN. □

Definition 11.12.132 (Unbounded direct-image resolution datum). Lean: `conditional-interface:def:weil-a4-unbounded-direct-image-resolution-datum`

Uses: `def:weil-a4-exceptional-index,def:weil-a2-complex-category,def:weil-a2-derived-category`

An unbounded direct-image resolution datum on $\overline{X}_{f,\mathrm{proét}}$ consists of a functor I on unbounded complexes and a natural quasi-isomorphism $\eta_K : K \to I(K)$ such that:

1. every $I(K)$ is K-injective;
2. $I(1) = 1$ and $I(vu) = I(v)I(u)$;
3. homotopic maps have homotopic images, and the induced map on every localization roof depends only on its equivalence class;
4. the natural comparison $I(\operatorname{Cone} u) \to \operatorname{Cone}(Iu)$ is a homotopy equivalence, compatible with shifts and rotations.

All comparison maps, homotopies, and inverse equations are fields of the datum.

Proposition 11.12.133 (Missing unbounded resolution datum). Lean: *conditional-interface:prop:weil-a4-unbounded-direct-image-resolution-open*

Uses: *def:weil-a4-unbounded-direct-image-resolution-datum*

OPEN. *An unbounded direct-image resolution datum exists for the full pro-étale module-sheaf category used by the ambient candidate.*

Open obligation: *OBL-A4-EX-01*

Proof. Uses: `def:weil-a4-unbounded-direct-image-resolution-datum`

The A3 constructions select bounded-below injective resolutions in D^+; they do not construct a functorial unbounded Spaltenstein replacement with the four displayed strict coherence fields. A2 records the analogous tower construction as 'prop:weil-a2-kinjective-tower-package-existence-open', so it also cannot be consumed under the present boundary. Thus the required construction is not available from A3 and remains the second exact leaf of 'OBL-A4-EX-01'. □

Lemma 11.12.134 (Resolution datum gives ambient descent and exactness). Lean: *conditional-interface:lemma:weil-a4-resolution-datum-ambient-exactness*

Uses: *def:weil-a4-unbounded-direct-image-resolution-datum,def:weil-a4-ambient-rfshriek-candidate*

Given an unbounded direct-image resolution datum, the formula $K \mapsto p_{f*}I(j_{f,!}K)$ descends to a triangulated functor on $D(X, \Lambda_n)$, independently of representatives and roofs.

Proof. Uses: `def:weil-a4-unbounded-direct-image-resolution-datum,def:weil-a4-ambient-rfshriek-candidate,prop:weil-a2-derived-category-triangulated`

Exactness of $j_{f,!}$ sends quasi-isomorphisms to quasi-isomorphisms. The roof field of the datum makes the image of a localized morphism independent of its roof and gives identity and composition. Two representatives are connected by a quasi-isomorphism; K-injectivity makes its comparison a homotopy equivalence, which remains a homotopy equivalence after applying p_{f*}. Finally, apply the cone field to a chain map. Termwise $j_{f,!}$ and p_{f*} commute with the direct sums and differentials in its cone, so the image cone is isomorphic to the cone of the image. Shifts and rotations agree by the remaining fields. Hence mapping-cone triangles map to distinguished triangles. □

Lemma 11.12.135 (Finite and ambient compactification models agree conditionally). Lean: *conditional-interface:lemma:weil-a4-finite-ambient-agreement*

Uses: *def:weil-a4-unbounded-direct-image-resolution-datum,def:weil-a4-selected-finite-artin-bound-candidate,def:weil-a4-ambient-rfshriek-candidate*

Assume the unbounded resolution datum and $d_f^{\mathrm{fin}} \in S_f$. On every finite-origin bounded constructible input, the ambient compactification model is naturally isomorphic to the A1 selected model, has amplitude $[a, b + 2d_f^{\mathrm{fin}}]$, and the isomorphism respects proper composition and compactification refinement.

Proof. Uses: `lemma:weil-a4-resolution-datum-ambient-exactness,prop:weil-a4-finite-origin-compact-support-amplitude,lemma:weil-a4-a3-proper-piece-coherence,def:weil-a1-rfshriek-selected`

The resolution datum identifies the ambient value with the right-derived proper direct image of the termwise open extension. The conditional finite-origin amplitude proposition identifies that object, on the same representative, with the bounded constructible A1 candidate and gives the stated interval. For a proper composite, both comparison maps are induced by the same underived equality of iterated direct images; functoriality of the resolution datum identifies their derived maps. For a refinement, the A3 coherence lemma identifies both paths with the same degreewise refinement map. These equalities give the naturality, composition, and refinement assertions. □

Proposition 11.12.136 (Ambient compact-support descent and exactness). Lean: `conditional-interface:prop:weil-a4-ambient-rfshriek-exact-open`

Uses: `def:weil-a4-ambient-rfshriek-candidate`

OPEN. *The candidate is independent of all representatives, defines a triangulated functor*

$$Rf_{!,\mathrm{amb}} : D(X, \Lambda_n) \longrightarrow D(Y, \Lambda_n),$$

and its restriction to $D^b_c(X, \Lambda_n)$ is naturally isomorphic to the A1 selected compact-support functor.

Open obligation: `OBL-A4-EX-01`

Proof. Uses: `prop:weil-a4-unbounded-direct-image-resolution-open,lemma:weil-a4-resolution-datum-ambient-exactness,prop:weil-a4-selected-finite-artin-bound-open,lemma:weil-a4-finite-ambient-agreement`

The first OPEN proposition supplies the exact unbounded resolution datum. The resolution-datum lemma then proves representative independence, roof-independence, functoriality, and preservation of mapping-cone triangles. The second OPEN proposition supplies $d_f^{\mathrm{fin}} \in S_f$. Instantiate the finite/ambient agreement lemma with these two inputs. It gives the natural identification with the A1 selected candidate, including proper-composition and refinement compatibility. All bounded-below site comparisons, finite proper base change, and the conditional amplitude calculation used in this last step were proved in the preceding closed nodes. Thus this aggregate proposition carries no uncertainty beyond the two displayed leaves of 'OBL-A4-EX-01'. □

Definition 11.12.137 (Exceptional Hom functor). Lean: `conditional-interface:def:weil-a4-exceptional-hom-functor`

Uses: `def:weil-a4-exceptional-index,def:weil-a4-ambient-rfshriek-candidate`

For $L \in D^b_c(Y, \Lambda_n)$, define the contravariant set-valued functor

$$H_{f,L}(K) := \mathrm{Hom}_{D(Y,\Lambda_n)}(Rf_{!,\mathrm{amb}}K, L)$$

on the ambient category $D(X, \Lambda_n)$. For $u : K' \to K$, set $H_{f,L}(u)(a) = a \circ Rf_{!,\mathrm{amb}}(u)$.

Lemma 11.12.138 (Identity law for the exceptional Hom functor). Lean: `conditional-interface:lemma:weil-a4-exceptional-hom-identity`

Uses: `def:weil-a4-exceptional-hom-functor`

For every K, $H_{f,L}(\mathrm{id}_K)$ is the identity function.

Proof. Uses: `def:weil-a4-exceptional-hom-functor`

For $a : Rf_{!,\mathrm{amb}}K \to L$, functoriality of the ambient candidate gives $Rf_{!,\mathrm{amb}}\,\mathrm{id}_K = \mathrm{id}_{Rf_{!,\mathrm{amb}}K}$. Hence the image is $a \circ \mathrm{id} = a$. □

Lemma 11.12.139 (Composition law for the exceptional Hom functor). Lean: `conditional-interface:lemma:weil-a4-exceptional-hom-composition`

Uses: `def:weil-a4-exceptional-hom-functor`

For $K'' \xrightarrow{v} K' \xrightarrow{u} K$, $H_{f,L}(uv) = H_{f,L}(v) \circ H_{f,L}(u)$.

Proof. Uses: `def:weil-a4-exceptional-hom-functor`

For $a : Rf_{!,\mathrm{amb}}K \to L$, the left side is $a \circ Rf_{!,\mathrm{amb}}(uv)$. The right side is $a \circ Rf_{!,\mathrm{amb}}(u) \circ Rf_{!,\mathrm{amb}}(v)$. Functoriality of the ambient candidate and associativity of composition identify these expressions. □

Proposition 11.12.140 (Cohomologicality of the exceptional Hom functor). Lean: `conditional-interface:prop:weil-a4-exceptional-hom-cohomological-open`

Uses: `def:weil-a4-exceptional-hom-functor`

OPEN. *For every distinguished triangle $K' \to K \to K'' \to K'[1]$, the shifted values of $H_{f,L}$ form the associated long exact Hom sequence.*

Open obligation: `OBL-A4-EX-01`

Proof. Uses: `prop:weil-a4-ambient-rfshriek-exact-open,def:weil-a4-exceptional-hom-functor,prop:weil-a2-derived-category-triangulated`

The ambient exactness proposition sends the given triangle and all its shifts to a distinguished triangle. In a triangulated category, applying $\mathrm{Hom}(-, L)$ to a distinguished triangle gives the exact sequence

$$\begin{aligned}\cdots \longrightarrow \mathrm{Hom}(Rf_{!,\mathrm{amb}}K'[r+1], L) &\longrightarrow \mathrm{Hom}(Rf_{!,\mathrm{amb}}K''[r], L)\\ \longrightarrow \mathrm{Hom}(Rf_{!,\mathrm{amb}}K[r], L) &\longrightarrow \mathrm{Hom}(Rf_{!,\mathrm{amb}}K'[r], L) \longrightarrow \cdots .\end{aligned}$$

The shift compatibility of the ambient functor identifies these groups with the shifted values of $H_{f,L}$. Hence exactness of this sequence is the claimed cohomologicality. Its sole unresolved construction is the unbounded resolution leaf already carried by the cited aggregate proposition. □

Universe-controlled weakly contractible affine covers

Definition 11.12.141 (Pro-etale smallness envelope)**.** Lean: `conditional-interface:def:weil-a4-proetale-smallness-envelope`

Uses: `def:weil-a4-exceptional-index`

Fix cumulative universes $\mathcal{U} \in \mathcal{V}$, and let Θ be the strongly inaccessible cardinal bounding $\mathcal{V}$. All schemes occurring as objects of the selected small pro-etale site are represented by $\mathcal{U}$-small schemes. For an infinite cardinal $\kappa < \Theta$, put

$$B(\kappa) := \max\{\kappa^{2^{2^{2^\kappa}}}, \kappa^{\aleph_0}, \kappa^+\}.$$

For a scheme T, let $\mathrm{size}(T)$ be the maximum of $\aleph_0$, the cardinality of $|T|$, the cardinality of a chosen affine open basis, and the cardinalities of the rings of that basis. The object type of X_{proet}, its morphism types, and every covering family used below are taken in $\mathcal{V}$.

Lemma 11.12.142 (The envelope stays inside the selected universe)**.** *Lean:* `conditional-interface:lemma:weil-a4-proetale-envelope-closed`

Uses: `def:weil-a4-proetale-smallness-envelope`

If $\kappa < \Theta$, then $B(\kappa) < \Theta$. A union indexed by at most κ sets, each of cardinal at most $B(\kappa)$, also has cardinal less than Θ.

Proof. Uses: `def:weil-a4-proetale-smallness-envelope`

Strong inaccessibility says that Θ is regular and a strong limit. Starting with $\kappa < \Theta$, apply the strong-limit inequality three times to obtain

$$2^\kappa < \Theta, \qquad 2^{2^\kappa} < \Theta, \qquad 2^{2^{2^\kappa}} < \Theta.$$

For cardinals $\lambda, \mu < \Theta$, regularity and the strong-limit property give $\lambda^\mu < \Theta$: the set of functions $\mu \to \lambda$ injects into the power set of $\mu \times \lambda$, whose cardinality is below Θ. This treats the first and second terms in $B(\kappa)$; inaccessibility gives $\kappa^+ < \Theta$. A finite maximum of cardinals below Θ is below Θ. Finally, $\kappa \cdot B(\kappa) = B(\kappa) < \Theta$, proving the union assertion. □

Definition 11.12.143 (Ring-theoretic and site-theoretic contractibility)**.** Lean: `conditional-interface:def:weil-a4-wcontractible-notions`

Uses: `def:weil-a4-proetale-smallness-envelope`

A spectral space is w-local if its closed points form a closed subspace and every point specializes to a unique closed point. A compact Hausdorff space is extremally disconnected if the closure of every open subset is open. A ring R is w-contractible if every faithfully flat weakly etale map $R \to R'$ has an R-algebra retraction.

An object U of X_{proet} is weakly contractible if, for every epimorphism $F \to G$ of sheaves of sets, the map $F(U) \to G(U)$ is surjective.

Definition 11.12.144 (Localization attached to a locally closed stratum)**.** Lean: `conditional-interface:def:weil-a4-stratum-localization`

Uses: `def:weil-a4-wcontractible-notions`

Let A be a ring and let

$$Z = D(f) \cap V(I) \subset \operatorname{Spec} A.$$

Define $S_Z \subset A$ to be the inverse image of the unit group of $(A/I)_f$, and define

$$A_Z^\sim := S_Z^{-1} A.$$

Lemma 11.12.145 (Points of a stratum localization)**.** *Lean:* `conditional-interface:lemma:weil-a4-stratum-localization-points`

Uses: `def:weil-a4-stratum-localization`

The image of $\operatorname{Spec}(A_Z^\sim) \to \operatorname{Spec} A$ is exactly the set of primes specializing to a point of Z. The subscheme Z is closed in $\operatorname{Spec}(A_Z^\sim)$.

Proof. Uses: `def:weil-a4-stratum-localization`

Let $\mathfrak{p} \subset A$. If $\mathfrak{p}$ does not specialize to Z, its extension to $(A/I)_f$ is the unit ideal. Hence there are $n \geq 0$, $g \in \mathfrak{p}$, and $h \in I$ with $f^n = g + h$. The image of g is the unit f^n in $(A/I)_f$, so $g \in S_Z \cap \mathfrak{p}$; thus $\mathfrak{p}$ is not in the image of the localization.

Conversely, if $\mathfrak{p} \subset \mathfrak{q}$, $I \subset \mathfrak{q}$, and $f \notin \mathfrak{q}$, then every element of S_Z avoids $\mathfrak{q}$, hence avoids $\mathfrak{p}$. The prime $\mathfrak{p}$ therefore extends to $S_Z^{-1}A$. The quotient map

$$S_Z^{-1} A \longrightarrow (A/I)_f$$

is surjective, so its spectrum identifies Z with a closed subscheme. □

Definition 11.12.146 (Finite vanishing-pattern algebra). Lean: `conditional-interface:def:weil-a4-vanishing-pattern-algebra`

Uses: `def:weil-a4-stratum-localization`

For a finite set $E \subset A$ and a decomposition $E = E' \amalg E''$, put

$$Z(E', E'') = D\left(\prod_{e \in E'} e\right) \cap V\left(\sum_{e \in E''} eA\right).$$

Define

$$A_E := \prod_{E = E' \amalg E''} A^{\sim}_{Z(E', E'')}.$$

If $E_1 \subset E_2$, restriction of a vanishing pattern and the localization maps define $A_{E_1} \to A_{E_2}$.

Lemma 11.12.147 (The vanishing-pattern pieces partition the spectrum). Lean: `conditional-interface:lemma:weil-a4-vanishing-pattern-partition`

Uses: `def:weil-a4-vanishing-pattern-algebra`

For every finite $E \subset A$, the locally closed subsets $Z(E', E'')$, indexed by decompositions $E = E' \amalg E''$, are pairwise disjoint and have union $\operatorname{Spec} A$. *Every point of* $\operatorname{Spec} A_E$ *specializes to a point in the corresponding stratum.*

Proof. Uses: `lemma:weil-a4-stratum-localization-points,def:weil-a4-vanishing-pattern-algebra`

For a prime $\mathfrak{p}$, set $E'' = E \cap \mathfrak{p}$ and $E' = E \setminus E''$. Then $\mathfrak{p} \in Z(E', E'')$. Membership in a second stratum would assign some $e \in E$ both to the vanishing and nonvanishing parts, which is impossible. This proves the partition. The second assertion is the point-description of the stratum localization applied to each factor of A_E. □

Definition 11.12.148 (w-localization of a ring). Lean: `conditional-interface:def:weil-a4-wlocalization-ring`

Uses: `def:weil-a4-vanishing-pattern-algebra`

Let $\mathrm{Fin}(A)$ be the directed poset of finite subsets of A. Define

$$A_w := \varinjlim_{E \in \mathrm{Fin}(A)} A_E.$$

Write $X_w = \operatorname{Spec}(A_w)$. For each E, let Z_E be the disjoint union of the strata inside $\operatorname{Spec}(A_E)$, and put $Z_w := \varprojlim_E Z_E \subset X_w$.

Proposition 11.12.149 (Properties of the w-localization). Lean: `conditional-interface:prop:weil-a4-wlocalization-properties`

Uses: `def:weil-a4-wlocalization-ring,def:weil-a4-wcontractible-notions`

The map $A \to A_w$ is faithfully flat and ind-Zariski, $\operatorname{Spec}(A_w)$ *is w-local, and $Z_w \to \operatorname{Spec} A$ is a bijection identifying Z_w with the closed points of X_w. If $|A| \le \kappa$ for an infinite cardinal, then $|A_w| \le \kappa$.*

Proof. Uses: `lemma:weil-a4-vanishing-pattern-partition,lemma:weil-a4-stratum-localization-points,def:weil-a4-wlocalization-ring`

Each factor in A_E is a localization of A; finite products along the partition are flat, and their spectra cover $\operatorname{Spec} A$. Thus $A \to A_E$ is faithfully flat and ind-Zariski. Filtered colimits preserve flatness, and surjectivity on spectra follows from the compatible closed subset Z_w, so $A \to A_w$ is faithfully flat.

The partition gives a bijection $Z_E \to \operatorname{Spec} A$ for every E, compatible with transition maps; hence $Z_w \to \operatorname{Spec} A$ is bijective. Let $y \in X_w \setminus Z_w$. At some finite stage its image is outside Z_E. The closure of that image meets Z_E, because each component of $\operatorname{Spec} A_E$ consists of points specializing to its stratum. Enlarging E preserves these nonempty closed intersections. Quasi-compactness of inverse limits of nonempty closed subsets supplies a point of Z_w in the closure of y; hence y is not closed.

If y specialized to distinct $z, z' \in Z_w$, their images $\mathfrak{p}, \mathfrak{p}'$ in $\operatorname{Spec} A$ would be distinct. Choose $f \in \mathfrak{p} \setminus \mathfrak{p}'$, or interchange the two. At the stage $E = \{f\}$, the points z, z' lie in different open-and-closed factors, so no point can specialize to both. Thus each point has a unique closed specialization. The inverse limit Z_w is closed by construction, which proves w-locality.

There are at most κ finite subsets of A. Every localization and every finite product used in A_E has cardinal at most κ. The filtered colimit therefore has cardinal at most $\kappa \cdot \kappa = \kappa$. □

Definition 11.12.150 (One-step etale splitting algebra). Lean: `conditional-interface:def:weil-a4-etale-splitting-algebra`

Uses: `def:weil-a4-wlocalization-ring`

Let $\mathcal{S}(A)$ be the set of faithfully flat standard etale A-algebras

$$A[x_1, \dots, x_m]/(f_1, \dots, f_m)$$

for which the Jacobian determinant is invertible. For a finite $E = \{B_1, \dots, B_s\} \subset \mathcal{S}(A)$, put $B_E = B_1 \otimes_A \cdots \otimes_A B_s$, with $B_\varnothing = A$, and define

$$T(A) := \varinjlim_{E \subset \mathcal{S}(A),\ E \text{ finite}} B_E.$$

Lemma 11.12.151 (Properties of the one-step splitting algebra). *Lean:* `conditional-interface:lemma:weil-a4-etale-splitting-properties`

Uses: `def:weil-a4-etale-splitting-algebra`

The map $A \to T(A)$ is faithfully flat and ind-etale. Every faithfully flat etale A-algebra admits an A-algebra map to $T(A)$. If $|A| \le \kappa$ with κ infinite, then $|T(A)| \le \kappa$.

Proof. *Uses:* `def:weil-a4-etale-splitting-algebra`

Finite tensor products of faithfully flat etale algebras are faithfully flat and etale. The transition $B_E \to B_{E \cup \{B\}}$ is tensoring with B and sends b to $b \otimes 1$. Filtered colimits preserve flatness; every prime of A lifts at each finite stage and hence to the limit, proving faithful flatness. The displayed filtered presentation is the definition of ind-etale.

Every etale A-algebra is isomorphic to a standard etale presentation. For a faithfully flat one, choose its representative $B_0 \in \mathcal{S}(A)$; the map through the singleton stage $B \simeq B_0 \to B_{\{B_0\}} \to T(A)$ is the required map. There are at most κ finite standard presentations over A; each B_E has cardinal at most κ. The same $\kappa \cdot \kappa = \kappa$ calculation bounds the colimit. □

Definition 11.12.152 (Iterated etale splitting saturation). *Lean:* `conditional-interface:def:weil-a4-etale-splitting-saturation`

Uses: `def:weil-a4-etale-splitting-algebra`

Define $T^0(A) = A$, $T^{m+1}(A) = T(T^m(A))$, and

$$C_{\mathrm{et}}(A) := \varinjlim_{m \in \mathbf{N}} T^m(A).$$

Proposition 11.12.153 (Finite etale maps split after saturation). *Lean:* `conditional-interface:prop:weil-a4-etale-saturation-splitting`

Uses: `def:weil-a4-etale-splitting-saturation`

The map $A \to C_{\mathrm{et}}(A)$ is faithfully flat and ind-etale. Every faithfully flat etale $C_{\mathrm{et}}(A)$-algebra has a retraction. If $|A| \le \kappa$, then $|C_{\mathrm{et}}(A)| \le \kappa$.

Proof. *Uses:* `lemma:weil-a4-etale-splitting-properties,def:weil-a4-etale-splitting-saturation`

The first and third assertions follow by induction on m and then by the filtered-colimit flatness and cardinal calculations from the preceding lemma.

Let $C = C_{\mathrm{et}}(A)$ and let $C \to B$ be faithfully flat etale. Finite presentation descends through the sequential filtered colimit: there are m, an etale $T^m(A)$-algebra B_m, and an isomorphism

$$B \simeq C \otimes_{T^m(A)} B_m.$$

Surjectivity of $\operatorname{Spec} B \to \operatorname{Spec} C$, followed by $\operatorname{Spec} C \to \operatorname{Spec} T^m(A)$, implies $\operatorname{Spec} B_m \to \operatorname{Spec} T^m(A)$ is surjective. Thus B_m is faithfully flat. The one-step property gives $B_m \to T^{m+1}(A)$; tensoring its composite with $T^{m+1}(A) \to C$ produces a C-algebra map $B \to C$, which is a retraction. □

Lemma 11.12.154 (Strict henselian local rings after etale saturation). *Lean:* `conditional-interface:lemma:weil-a4-etale-saturation-strict-henselian`

Uses: `def:weil-a4-etale-splitting-saturation`

Every localization of $C_{\mathrm{et}}(A)$ at a maximal ideal is strictly henselian.

Proof. *Uses:* `prop:weil-a4-etale-saturation-splitting,def:weil-a4-etale-splitting-saturation`

Put $C = C_{\mathrm{et}}(A)$ and let $\mathfrak{m}$ be maximal. It is enough to show that for every etale C-algebra B and prime $\mathfrak{q}$ above $\mathfrak{m}$, the local etale map $C_{\mathfrak{m}} \to B_{\mathfrak{q}}$ is an isomorphism. There are finitely many primes of B above $\mathfrak{m}$. Localize B at an element belonging to all the others but not to $\mathfrak{q}$; then $\mathfrak{q}$ is the sole prime above $\mathfrak{m}$. The image of $\operatorname{Spec} B$ is open. Choose $f \notin \mathfrak{m}$ such that this image together with $D(f)$ covers $\operatorname{Spec} C$. The map

$$C \longrightarrow B \times C_f$$

is faithfully flat etale, hence has a retraction by Proposition 11.12.153. Localizing the retraction at $\mathfrak{m}$ gives inverse flat local maps

$$C_{\mathfrak{m}} \longrightarrow B_{\mathfrak{q}} \longrightarrow C_{\mathfrak{m}}.$$

A flat local ring map is injective; applying this to both maps makes both injective, while their composite is the identity. They are therefore inverse isomorphisms. This is the etale lifting characterization of a strictly henselian local ring. □

Definition 11.12.155 (Closed-point preserving w-localization)**.** Lean: `conditional-interface:def:weil-a4-relative-wlocalization`

Uses: `def:weil-a4-wlocalization-ring`, `def:weil-a4-stratum-localization`

Let R be w-local and let $I \subset R$ cut out its closed points. For a map $R \to S$ inducing algebraic residue-field extensions, first form S_w. Let T be the inverse image in the closed-point subspace of $\operatorname{Spec}(S_w)$ of $V(IS)$, and define

$$\mathrm{WLoc}_I(S) := (S_w)^{\sim}_T.$$

Proposition 11.12.156 (Properties of relative w-localization)**.** Lean: `conditional-interface:prop:weil-a4-relative-wlocalization-properties`

Uses: `def:weil-a4-relative-wlocalization`, `def:weil-a4-wcontractible-notions`

The map $S \to \mathrm{WLoc}_I(S)$ is ind-Zariski, the target spectrum is w-local, its closed points are precisely the inverse image of the closed points of $\operatorname{Spec} R$*, and*

$$S/IS \longrightarrow \mathrm{WLoc}_I(S)/I\,\mathrm{WLoc}_I(S)$$

is an isomorphism. All local rings are unchanged by the ind-Zariski map.

Proof. Uses: `prop:weil-a4-wlocalization-properties`, `lemma:weil-a4-stratum-localization-points`, `def:weil-a4-relative-wlocalization`

Because R/I has no nontrivial specializations and the residue extensions are algebraic, every prime of S/IS is maximal. Its spectrum is therefore profinite. In the w-localization S_w, the inverse image T of this profinite subspace is closed inside the closed-point subspace and maps homeomorphically to $\operatorname{Spec}(S/IS)$. The stratum-localization point calculation says that localizing along T retains exactly the points specializing to T. Hence the resulting spectrum is w-local and has T as its closed points.

The composite $S/IS \to \mathrm{WLoc}_I(S)/I$ is ind-Zariski and induces a homeomorphism on spectra. On every prime localization it is the identity, because every transition is a localization. An ind-Zariski map which is bijective on primes and identifies all local rings is an isomorphism: injectivity and surjectivity can be checked after localization at every prime. The same localization calculation proves the final assertion. □

Definition 11.12.157 (The w-local strictly henselian stage)**.** Lean: `conditional-interface:def:weil-a4-wlocal-strict-stage`

Uses: `def:weil-a4-wlocalization-ring`, `def:weil-a4-etale-splitting-saturation`, `def:weil-a4-relative-wlocalization`

Starting from A, put $A' = A_w$, let $I \subset A'$ cut out the closed points of $\operatorname{Spec} A'$, put

$$C' = C_{\mathrm{et}}(A'), \qquad C = \mathrm{WLoc}_I(C').$$

Proposition 11.12.158 (Properties of the strictly henselian stage)**.** Lean: `conditional-interface:prop:weil-a4-wlocal-strict-stage-properties`

Uses: `def:weil-a4-wlocal-strict-stage`, `def:weil-a4-wcontractible-notions`

The map $A \to C$ is faithfully flat and ind-etale, $\operatorname{Spec} C$ *is w-local, and every local ring $C_{\mathfrak{m}}$ at a maximal ideal is strictly henselian. If $|A| \le \kappa$, then $|C| \le \kappa$.*

Proof. Uses: `prop:weil-a4-wlocalization-properties`, `prop:weil-a4-etale-saturation-splitting`, `lemma:weil-a4-etale-saturation-strict-henselian`, `prop:weil-a4-relative-wlocalization-properties`, `def:weil-a4-wlocal-strict-stage`

The maps $A \to A'$ and $A' \to C'$ are faithfully flat and respectively ind-Zariski and ind-etale. The map $C' \to C$ is ind-Zariski, hence flat and ind-etale. Its closed-point quotient agrees with that of C', which surjects onto the closed points of A'; those closed points surject onto $\operatorname{Spec} A$. Thus the composite is surjective on spectra and faithfully flat. The relative localization proposition gives w-locality. It also identifies local rings of C with local rings of C', so strict henselianity follows from Lemma 11.12.154. Each of the three stages preserves the infinite cardinal bound κ, by the cardinal parts of the preceding propositions and because a localization has at most $\max(\kappa, \aleph_0) = \kappa$ elements. □

Definition 11.12.159 (Ultrafilter component cover). Lean: `conditional-interface:def:weil-a4-ultrafilter-component-cover`

Uses: `def:weil-a4-wlocal-strict-stage`

Let $P = \pi_0(\operatorname{Spec} C)$, viewed as a profinite space. Let $\beta(P_{\mathrm{disc}})$ be the set of ultrafilters on the underlying set of P, with basic clopen subsets

$$\widehat{E} = \{\mathcal{F} : E \in \mathcal{F}\} \qquad (E \subset P).$$

For $\mathcal{F}$, define $\lambda(\mathcal{F}) \in P$ to be the unique limit of the pushforward ultrafilter on the compact Hausdorff space P. Write $Q(A) = \beta(P_{\mathrm{disc}})$ and $\lambda_A : Q(A) \to P$.

Lemma 11.12.160 (The component cover is extremally disconnected and surjective). Lean: `conditional-interface:lemma:weil-a4-ultrafilter-component-properties`

Uses: `def:weil-a4-ultrafilter-component-cover`, `def:weil-a4-wcontractible-notions`

The space $Q(A)$ is compact Hausdorff and extremally disconnected, and $\lambda_A : Q(A) \to P$ is continuous and surjective. If $|P| \le 2^\kappa$, then

$$|Q(A)| \le 2^{2^{2^\kappa}}.$$

Proof. Uses: `def:weil-a4-ultrafilter-component-cover`

The map $\mathcal{F} \mapsto (\mathbf{1}_{E\in\mathcal{F}})_{E\subset P}$ embeds $Q(A)$ into the product $\{0,1\}^{\mathcal{P}(P)}$. The ultrafilter axioms are closed coordinate conditions, so $Q(A)$ is a closed subspace of this compact Hausdorff product and is compact Hausdorff.

Let $O \subset Q(A)$ be open, and let

$$E_O = \{p \in P : \text{the principal ultrafilter at } p \text{ belongs to } O\}.$$

Then $\overline{O} = \widehat{E_O}$. Indeed, every basic neighborhood $\widehat{B}$ of an ultrafilter containing E_O has $B \cap E_O \neq \varnothing$, and a principal ultrafilter at a point of that intersection lies in $O \cap \widehat{B}$. Conversely, a point outside $\widehat{E_O}$ has the disjoint basic neighborhood $\widehat{P \setminus E_O}$. Thus $\overline{O}$ is clopen, proving extremal disconnectedness.

An ultrafilter on a compact space has a cluster point: otherwise finitely many complements of neighborhoods would belong to the ultrafilter and have empty intersection. Hausdorffness makes the cluster point unique. This defines λ_A. The inverse image of a clopen $V \subset P$ is $\widehat{V}$, proving continuity because clopens form a basis of the profinite topology. Principal ultrafilters map to their defining points, so λ_A is surjective. Finally, the set of ultrafilters is a subset of $\mathcal{P}(\mathcal{P}(P))$; substituting $|P| \le 2^\kappa$ gives the displayed cardinal bound. □

Lemma 11.12.161 (The ultrafilter component cover lifts surjections). Lean: `conditional-interface:lemma:weil-a4-ultrafilter-component-lifting`

Uses: `def:weil-a4-ultrafilter-component-cover`

Every continuous surjection $q : Y \to Q(A)$ from a profinite space admits a continuous section.

Proof. Uses: `def:weil-a4-ultrafilter-component-cover`

For each $p \in P$, choose $y_p \in q^{-1}(\mathcal{U}_p)$, where $\mathcal{U}_p$ is the principal ultrafilter at p. The map $p \mapsto y_p$ is continuous when P has the discrete topology. Extend it to $s : \beta(P_{\mathrm{disc}}) \to Y$ by sending an ultrafilter $\mathcal{F}$ to the unique limit in the compact Hausdorff space Y of the pushforward ultrafilter $(y_{(-)})_*\mathcal{F}$.

For a clopen $V \subset Y$, the inverse image $s^{-1}(V)$ is $\{p : \widehat{y_p \in V}\}$, hence is clopen. Since clopens form a basis of Y, the map s is continuous. The maps $q \circ s$ and $\mathrm{id}_{Q(A)}$ agree on every principal ultrafilter. Principal ultrafilters are dense: every nonempty basic open $\widehat{E}$ contains one. As $Q(A)$ is compact and Hausdorff, two continuous maps agreeing on this dense subset agree everywhere. Thus $q \circ s = \mathrm{id}$. □

Definition 11.12.162 (Component pullback ring). Lean: `conditional-interface:def:weil-a4-component-pullback-ring`

Uses: `def:weil-a4-ultrafilter-component-cover`, `def:weil-a4-wlocal-strict-stage`

Let $\mathcal{P}(Q(A))$ be the directed set of finite clopen partitions of $Q(A)$. For a partition Π, let Q_Π be its finite set of parts and let

$$Z_\Pi = \operatorname{im}\big(Q(A) \longrightarrow P \times Q_\Pi\big).$$

The inverse image of Z_Π in $\operatorname{Spec} C \times Q_\Pi = \operatorname{Spec}(\prod_{Q_\Pi} C)$ is a union of connected components. Define D_Π to be the filtered colimit of the localizations of $\prod_{Q_\Pi} C$ at all clopen neighborhoods of that union. For a refinement $\Pi' \ge \Pi$, pullback of clopen neighborhoods defines $D_\Pi \to D_{\Pi'}$. Define

$$D(A) := \varinjlim_{\Pi} D_\Pi.$$

Proposition 11.12.163 (Properties of the component pullback ring)**.** *Lean:* `conditional-interface:prop:weil-a4-component-pullback-properties`

Uses: `def:weil-a4-component-pullback-ring`

The map $C \to D(A)$ is faithfully flat and ind-Zariski, identifies local rings, and induces a cartesian square

$$\begin{array}{ccc} \operatorname{Spec} D(A) & \longrightarrow & Q(A) \\ \downarrow & & \downarrow \lambda_A \\ \operatorname{Spec} C & \longrightarrow & \pi_0(\operatorname{Spec} C). \end{array}$$

Consequently $\operatorname{Spec} D(A)$ is w-local, $\pi_0(\operatorname{Spec} D(A)) = Q(A)$, and maximal local rings of $D(A)$ are strictly henselian.

Proof. Uses: `prop:weil-a4-wlocal-strict-stage-properties,lemma:weil-a4-ultrafilter-component-properties,def:weil-a4-component-pullback-ring`

At a finite partition, Z_Π is closed in the finite product $P \times Q_\Pi$. The filtered intersection of its clopen neighborhoods is Z_Π. Passing to spectra turns the filtered colimit of the corresponding localizations into the inverse image of Z_Π. Refining all finite clopen partitions and using $Q(A) = \varprojlim_\Pi Q_\Pi$ gives the displayed fiber product.

Every transition is a localization, hence flat and identifies local rings. The projection of the fiber product onto $\operatorname{Spec} C$ is surjective because λ_A is surjective. Flatness plus surjectivity on spectra gives faithful flatness. A fiber of $\operatorname{Spec} C \to P$ contains exactly the points specializing to one closed point, because $\operatorname{Spec} C$ is w-local. The fiber product therefore has one closed point over each point of $Q(A)$, and every point specializes to that closed point. Thus it is w-local and its component space is $Q(A)$. Local-ring identification transfers strict henselianity from C. □

Lemma 11.12.164 (Weakly etale fibers over fields)**.** *Lean:* `conditional-interface:lemma:weil-a4-weakly-etale-fiber-idempotent-test`

Uses: `def:weil-a4-wcontractible-notions`

Let $k \to R$ be weakly etale. If $\operatorname{Spec} R$ has two distinct points, or if some residue extension $k \to \kappa(\mathfrak{q})$ is nontrivial and separable, then for some separable algebraic extension L/k, the ring $R \otimes_k L$ has a nontrivial idempotent.

Proof. Uses: `def:weil-a4-wcontractible-notions`

A weakly etale algebra over a field is a filtered colimit of etale subalgebras: every finite tuple of elements and every finite set of relations descends to a finitely presented subalgebra, and flatness of the algebra and of its diagonal permits the presentation to be refined until its module of differentials vanishes; a finitely presented flat algebra with vanishing differentials is etale. Distinct primes, or a nontrivial separable residue element, are witnessed at one finite etale stage R_0.

After a finite separable extension L/k splitting the residue fields of that stage, the finite etale quotient containing the two witnesses is a product with at least two nonzero factors. Its factor projector is a nontrivial idempotent. Idempotents descend and persist under filtered colimits because the equation $e^2 = e$, together with $e \neq 0, 1$, is witnessed at a finite stage. Its image gives the required idempotent of $R \otimes_k L$. □

Lemma 11.12.165 (Olivier rigidity in the local case)**.** *Lean:* `conditional-interface:lemma:weil-a4-weakly-etale-local-rigidity`

Uses: `def:weil-a4-wcontractible-notions`

Let $R \to S$ be a local weakly etale map of local rings. If R is strictly henselian, then $R \to S$ is an isomorphism.

Proof. Uses: `lemma:weil-a4-weakly-etale-fiber-idempotent-test,def:weil-a4-wcontractible-notions`

Fix $\mathfrak{p} \subset R$. Suppose that the fiber $S \otimes_R \kappa(\mathfrak{p})$ has two primes, or has a nontrivial separable residue extension. The preceding lemma supplies a separable algebraic $L/\kappa(\mathfrak{p})$ for which $S \otimes_R L$ has a nontrivial idempotent.

Let R' be the integral closure of $R/\mathfrak{p}$ in L. Hensel lifting along finite subextensions and passage to the filtered union show that R' is a strictly henselian local domain; its residue field is a purely inseparable extension of the separably closed residue field of R. The tensor product $S \otimes_R R'$ is local: every finite integral subextension is local by the henselian idempotent-lifting decomposition, and a filtered union of compatible local rings is local. It is integrally closed inside $S \otimes_R L$, because weak etaleness makes the diagonal flat and integral closure can be checked after the flat base change to each finite subextension. A nontrivial idempotent of $S \otimes_R L$ is integral over $S \otimes_R R'$, so it would lie in this local ring, a contradiction.

Thus every fiber has one prime and unchanged residue field. The multiplication $\mu : S \otimes_R S \to S$ is flat by weak etaleness, surjective, and bijective on spectra with identical residue fields. Its kernel is a pure ideal and has zero localization at every prime, hence is zero; therefore μ is an isomorphism. So $R \to S$ is a faithfully flat epimorphism. For $s \in S$, the equality $s \otimes 1 = 1 \otimes s$ and faithful flat descent give a unique $r \in R$ mapping to s. This constructs the inverse $S \to R$. □

Proposition 11.12.166 (The component pullback ring is w-contractible). Lean: `conditional-interface:prop:weil-a4-component-pullback-wcontractible`

Uses: `def:weil-a4-component-pullback-ring`, `def:weil-a4-wcontractible-notions`

The ring $D(A)$ is w-contractible.

Proof. Uses: `prop:weil-a4-component-pullback-properties`, `lemma:weil-a4-ultrafilter-component-lifting`, `lemma:weil-a4-ultrafilter-component-properties`, `prop:weil-a4-relative-wlocalization-properties`, `lemma:weil-a4-weakly-etale-local-rigidity`, `def:weil-a4-wcontractible-notions`

Write $D = D(A)$, and let $D \to B$ be faithfully flat and weakly etale. Let $I \subset D$ cut out the closed points. Apply the relative w-localization construction to obtain $B \to B'$ such that $\operatorname{Spec} B'$ is w-local and its closed points are $V(IB')$. The map on closed-point spaces

$$V(IB') \longrightarrow V(I) = \pi_0(\operatorname{Spec} D)$$

is a continuous surjection: faithful flatness lifts every maximal ideal. The target is the ultrafilter component cover $Q(A)$, so Lemma 11.12.161 supplies a continuous section.

Its image is a closed union of components of $\operatorname{Spec} B'$. Localize B' along this image, using the stratum-localization construction. The resulting map $D \to B''$ is faithfully flat, induces a bijection on component spaces, and identifies local rings: at a maximal ideal this is Lemma 11.12.165, because the local ring of D is strictly henselian. Away from the closed point both local rings are localizations of these maximal local rings.

For an ind-Zariski map inducing a bijection on components and identifying local rings, the inverse local maps glue. On overlaps they agree after localization at every prime, hence agree as ring maps. This gives a retraction $B'' \to D$. Composing

$$B \longrightarrow B' \longrightarrow B'' \longrightarrow D$$

is a D-algebra retraction of $D \to B$. This is precisely w-contractibility. □

Lemma 11.12.167 (w-contractible enlargement cardinality). Lean: `conditional-interface:lemma:weil-a4-wcontractible-enlargement-cardinality`

Uses: `def:weil-a4-component-pullback-ring`, `def:weil-a4-proetale-smallness-envelope`

If $|A| \le \kappa$ for an infinite cardinal, then

$$|D(A)| \le \kappa^{2^{2^{2^\kappa}}} \le B(\kappa).$$

Proof. Uses: `prop:weil-a4-wlocal-strict-stage-properties`, `lemma:weil-a4-ultrafilter-component-properties`, `def:weil-a4-component-pullback-ring`

The ring C has cardinal at most κ, so it has at most 2^κ prime ideals and components. The ultrafilter component cover has cardinal at most $2^{2^{2^\kappa}}$. The finite clopen partitions and their clopen neighborhoods form a set of cardinal at most that number. Each D_Π is made from finite products and localizations of C, so each of its local rings has cardinal at most κ. The canonical map

$$D(A) \longrightarrow \prod_{\mathfrak{m} \in \operatorname{Max} D(A)} D(A)_{\mathfrak{m}}$$

is injective: an element vanishing in every maximal localization has annihilator not contained in any maximal ideal, hence has unit annihilator. There are at most $2^{2^{2^\kappa}}$ maximal ideals, giving the stated power bound. The last inequality is one of the terms defining $B(\kappa)$. □

Definition 11.12.168 (Selected affine weakly contractible cover). Lean: `conditional-interface:def:weil-a4-selected-affine-wcontractible-cover`

Uses: `def:weil-a4-component-pullback-ring`, `def:weil-a4-proetale-smallness-envelope`

For an affine object $U = \operatorname{Spec} A \to X$ of the small pro-etale site, define

$$W(U) := \operatorname{Spec} D(A)$$

with its composite map $W(U) \to U \to X$.

Proposition 11.12.169 (The selected affine cover lies in the small site). Lean: `conditional-interface:prop:weil-a4-selected-affine-cover-small`

Uses: `def:weil-a4-selected-affine-wcontractible-cover`

The object $W(U) \to X$ belongs to the selected small pro-etale site. The scheme $W(U)$ is affine and quasi-compact, and $\{W(U) \to U\}$ is a pro-etale covering.

Proof. Uses: prop:weil-a4-wlocal-strict-stage-properties,prop:weil-a4-component-pullback-properties,lemma:weil-a4-wcontractible-enlargement-cardinality,lemma:weil-a4-proetale-envelope-closed,def:weil-a4-selected-affine-wcontractible-cover

The composite $A \to C \to D(A)$ is faithfully flat and ind-etale; an ind-Zariski map is ind-etale, and compositions of ind-etale maps are ind-etale. Hence $\operatorname{Spec} D(A) \to \operatorname{Spec} A$ is weakly etale and fpqc, which is exactly a one-member pro-etale covering. Its source is affine, hence quasi-compact. If $\kappa = \operatorname{size}(U)$, the cardinality lemma bounds its coordinate ring by $B(\kappa) < \Theta$. The smallness envelope therefore contains an isomorphic representative of $W(U)$; replacing by that representative gives an actual object of the selected site. Composition with the weakly etale map $U \to X$ is weakly etale. □

Proposition 11.12.170 (The selected affine cover is weakly contractible). Lean: *conditional-interface:prop:weil-a4-selected-affine-cover-contractible*

Uses: *def:weil-a4-selected-affine-wcontractible-cover, def:weil-a4-wcontractible-notions*

The object $W(U)$ is weakly contractible in the selected small pro-etale site.

Proof. Uses: prop:weil-a4-component-pullback-wcontractible,prop:weil-a4-selected-affine-cover-small,def:weil-a4-wcontractible-notions

Let $F \to G$ be an epimorphism of sheaves of sets and $s \in G(W(U))$. There is a pro-etale covering $\{V_i \to W(U)\}$ on which s lifts to F. Quasi-compactness replaces it by finitely many quasi-compact pieces. Refining each piece by an affine open cover gives a finite affine pro-etale cover. Its disjoint union is affine, say $\operatorname{Spec} B$, and $D(A) \to B$ is faithfully flat and weakly etale. The ring retraction from Proposition 11.12.166 gives a section $W(U) \to \operatorname{Spec} B$ over $W(U)$. Pulling one of the local lifts along the induced open-and-closed partition of $W(U)$, and gluing over that finite partition, produces a section of $F(W(U))$ mapping to s. Thus evaluation on $W(U)$ preserves epimorphisms. □

Definition 11.12.171 (Selected weakly contractible covering family). Lean: conditional-interface:def:weil-a4-selected-wcontractible-covering-family

Uses: def:weil-a4-selected-affine-wcontractible-cover

For $T \to X$ in the small pro-etale site, choose an affine open cover $(U_i)_{i \in J_T}$ indexed by a subset of the chosen affine basis of T. Define

$$\mathfrak{W}(T) := \{W(U_i) \longrightarrow U_i \longrightarrow T\}_{i \in J_T}.$$

Proposition 11.12.172 (Properties of the selected covering family). Lean: *conditional-interface:prop:weil-a4-selected-wcontractible-covering-properties*

Uses: *def:weil-a4-selected-wcontractible-covering-family*

The family $\mathfrak{W}(T)$ is a $\mathcal{V}$-small pro-etale covering of T by affine quasi-compact weakly contractible objects of the selected small site.

Proof. Uses: prop:weil-a4-selected-affine-cover-small,prop:weil-a4-selected-affine-cover-contractible,lemma:weil-a4-proetale-envelope-closed,def:weil-a4-selected-wcontractible-covering-family

The affine open family covers T, and each singleton $\{W(U_i) \to U_i\}$ is a pro-etale covering. Composition of covering families gives a pro-etale covering of T. The preceding two propositions give affineness, quasi-compactness, weak contractibility, and membership in the small site. The index set has cardinal at most $\operatorname{size}(T) < \Theta$; every member has size at most $B(\operatorname{size}(T)) < \Theta$. The union clause of Lemma 11.12.142 proves that the entire family is $\mathcal{V}$-small. □

Definition 11.12.173 (Refinement preorder of selected covers). Lean: conditional-interface:def:weil-a4-wcontractible-cover-refinement

Uses: def:weil-a4-selected-wcontractible-covering-family

For two affine quasi-compact weakly contractible covering families $\mathcal{U} = (U_i \to T)_{i \in I}$ and $\mathcal{V} = (V_j \to T)_{j \in J}$, write $\mathcal{U} \preceq_T \mathcal{V}$ if for every i a chosen index $\rho(i) \in J$ and a map $U_i \to V_{\rho(i)}$ over T are given.

Definition 11.12.174 (Pairwise-fiber-product common refinement). Lean: conditional-interface:def:weil-a4-wcontractible-common-refinement

Uses: def:weil-a4-wcontractible-cover-refinement,def:weil-a4-selected-wcontractible-covering-family

For $\mathcal{U}, \mathcal{V}$ as above, apply $\mathfrak{W}(-)$ to every fiber product $U_i \times_T V_j$. Let

$$\mathcal{U} \star_T \mathcal{V} = \coprod_{(i,j) \in I \times J} \mathfrak{W}(U_i \times_T V_j),$$

with its maps to U_i, V_j, and T.

Proposition 11.12.175 (Common refinement). Lean: `conditional-interface:prop:weil-a4-wcontractible-common-refinement`

Uses: `def:weil-a4-wcontractible-common-refinement`

The family $\mathcal{U} \star_T \mathcal{V}$ is an affine quasi-compact weakly contractible covering family and refines both $\mathcal{U}$ and $\mathcal{V}$.

Proof. Uses: `prop:weil-a4-selected-wcontractible-covering-properties,def:weil-a4-wcontractible-common-refinement,def:weil-a4-wcontractible-cover-refinement`

Base change of $\mathcal{U}$ along each $V_j \to T$ covers V_j; as the V_j cover T, all fiber products together cover T. Each selected family $\mathfrak{W}(U_i \times_T V_j)$ covers its fiber product, so composition shows that the star family covers T. Its members have the required four properties by Proposition 11.12.172. The two projections of the fiber product, composed with each member of its selected cover, are the specified refinement maps. □

Definition 11.12.176 (Pullback refinement along a site morphism). Lean: `conditional-interface:def:weil-a4-wcontractible-pullback-refinement`

Uses: `def:weil-a4-selected-wcontractible-covering-family`

For a morphism $g : T' \to T$, define $g^\diamond \mathfrak{W}(T)$ by applying $\mathfrak{W}(-)$ to every

$$T' \times_T W \qquad (W \in \mathfrak{W}(T))$$

and composing the resulting members with the projection to T'.

Proposition 11.12.177 (Functoriality up to common refinement). Lean: `conditional-interface:prop:weil-a4-wcontractible-cover-functoriality`

Uses: `def:weil-a4-wcontractible-pullback-refinement,def:weil-a4-wcontractible-common-refinement`

The family $g^\diamond \mathfrak{W}(T)$ covers T' and refines the base-change of $\mathfrak{W}(T)$. It has a common refinement with $\mathfrak{W}(T')$. For $T'' \xrightarrow{h} T' \xrightarrow{g} T$, the direct family $(gh)^\diamond \mathfrak{W}(T)$ and the iterated family $h^\diamond(g^\diamond \mathfrak{W}(T))$ have a common refinement. For $g = \mathrm{id}_T$, the same assertion compares $g^\diamond \mathfrak{W}(T)$ with $\mathfrak{W}(T)$.

Proof. Uses: `prop:weil-a4-selected-wcontractible-covering-properties,prop:weil-a4-wcontractible-common-refinement,def:weil-a4-wcontractible-pullback-refinement`

Base change preserves covering families, and the selected family covers each fiber product, proving the first assertion. Apply Proposition 11.12.175 to this cover and $\mathfrak{W}(T')$ for the second.

For a composite, both displayed families are affine quasi-compact weakly contractible covers of T'': the first by one base change and the second by two. Their star construction is a common refinement. The identity case is the same star construction for the two covers of T. Hence no strictly functorial choice is asserted; every comparison required for descent is supplied in the directed refinement preorder. □

Definition 11.12.178 (Weakly contractible pro-etale generators). Lean: `conditional-interface:def:weil-a4-compact-generator-family`

Uses: `def:weil-a4-exceptional-index,def:weil-a4-selected-wcontractible-covering-family`

Let I_X be the $\mathcal{V}$-small set of affine, quasi-compact, weakly contractible objects $u_U : U \to X$ in the selected small pro-etale site. For $U \in I_X$ and $r \in \mathbf{Z}$, define

$$G_{U,r} := u_{U!}\Lambda_{n,U}[r],$$

where $u_{U!}\Lambda_{n,U}$ is the sheafification of

$$V \longmapsto \bigoplus_{\alpha \in \mathrm{Hom}_X(V,U)} \Lambda_n.$$

Let $\mathcal{G}_X = \{G_{U,r} : U \in I_X,\ r \in \mathbf{Z}\}$. Thus $\mathcal{G}_X$ is closed under shifts by replacing r with $r+1$.

Lemma 11.12.179 (Generator sheaf represents sections). Lean: `conditional-interface:lemma:weil-a4-generator-sheaf-represents-sections`

Uses: `def:weil-a4-compact-generator-family`

For every sheaf F of Λ_n-modules,

$$\mathrm{Hom}(u_{U!}\Lambda_{n,U}, F) \longrightarrow F(U), \qquad \varphi \longmapsto \varphi_U([\mathrm{id}_U])$$

is a bijection.

Proof. Uses: def:weil-a4-compact-generator-family

Given $s \in F(U)$, define a morphism on the displayed presheaf by sending the basis vector indexed by $\alpha : V \to U$ to $F(\alpha)(s)$. Restriction along $V' \to V$ replaces α by its composite, so these maps commute with restriction. The sheafification universal property gives a sheaf morphism. Its value on $[\mathrm{id}_U]$ is s. Conversely, naturality forces the image of every basis vector $[\alpha]$ to be the restriction of the image of $[\mathrm{id}_U]$. The two constructions are inverse. □

Lemma 11.12.180 (Sections on a weakly contractible object are exact)**.** *Lean:* conditional-interface:lemma:weil-a4-weakly-contractible-sections-exact

Uses: def:weil-a4-compact-generator-family

For $U \in I_X$, the functor $F \mapsto F(U)$ on sheaves of Λ_n-modules is exact.

Proof. Uses: def:weil-a4-compact-generator-family

The section functor preserves kernels because kernels of sheaves are calculated sectionwise. Let $F \to F''$ be an epimorphism and let $s'' \in F''(U)$. By the definition of a sheaf epimorphism, a covering family $\{U_a \to U\}$ exists on which s'' has lifts $s_a \in F(U_a)$. The family $(s_a)_a$ defines a morphism $\coprod_a h^\#_{U_a} \to F$. Weak contractibility supplies a right inverse

$$h^\#_U \longrightarrow \coprod_a h^\#_{U_a}$$

to the sheafified covering map. Composing this right inverse with the morphism defined by $(s_a)_a$ gives a map $h^\#_U \to F$, hence a section $s \in F(U)$. Its image in $F''(U)$ is s'', because the covering-map composite is the identity of $h^\#_U$. Thus sections preserve epimorphisms and are exact. □

Lemma 11.12.181 (Quasi-compact sections commute with direct sums)**.** *Lean:* conditional-interface:lemma:weil-a4-qc-sections-direct-sums

Uses: def:weil-a4-compact-generator-family

For $U \in I_X$ and a set-indexed family $(F_j)_{j\in J}$, the map

$$\bigoplus_{j\in J} F_j(U) \longrightarrow \left(\bigoplus_{j\in J} F_j\right)(U)$$

induced by the coproduct inclusions is bijective.

Proof. Uses: def:weil-a4-compact-generator-family

The sheaf coproduct is the sheafification of the presheaf direct sum. A section of the sheafification is locally represented by sections having finite support in J. Quasi-compactness of U replaces the local cover by a finite subcover; the union of the resulting finite supports is a finite subset $J_0 \subset J$. The local representatives therefore glue inside $\bigoplus_{j\in J_0} F_j$, proving surjectivity. If a finite-support family maps to zero after sheafification, it is locally zero. A finite subcover witnesses that every one of its finitely many components is zero, which proves injectivity. □

Lemma 11.12.182 (Derived Hom from a generator is cohomology of sections)**.** *Lean:* conditional-interface:lemma:weil-a4-generator-derived-hom-sections

Uses: def:weil-a4-compact-generator-family

For $K \in D(X, \Lambda_n)$,

$$\mathrm{Hom}_{D(X,\Lambda_n)}(G_{U,r}, K) \simeq H^{-r}(K(U)).$$

Proof. Uses: lemma:weil-a4-generator-sheaf-represents-sections,lemma:weil-a4-weakly-contractible-sections-exact,prop:weil-a2-derived-category-triangulated

Represent K by a complex $K^\bullet$. The sheaf-level representing bijection identifies the Hom complex from $u_{U!}\Lambda_{n,U}$ to $K^\bullet$ with the section complex $K^\bullet(U)$, degree by degree, and the differentials agree by naturality. Exactness of sections makes this calculation invariant under quasi-isomorphisms. Shifting the source by r shifts the Hom complex by $-r$; its degree-zero cohomology is $H^{-r}(K(U))$. □

Lemma 11.12.183 (Each selected generator is compact)**.** *Lean:* conditional-interface:lemma:weil-a4-selected-generator-compact

Uses: def:weil-a4-compact-generator-family

For every $G_{U,r} \in \mathcal{G}_X$ and every family $(K_j)_{j\in J}$,

$$\bigoplus_j \mathrm{Hom}(G_{U,r}, K_j) \longrightarrow \mathrm{Hom}\left(G_{U,r}, \bigoplus_j K_j\right)$$

is bijective.

Proof. Uses: lemma:weil-a4-generator-derived-hom-sections,lemma:weil-a4-qc-sections-direct-sums

Replace both sides by degree $-r$ cohomology of section complexes. The section lemma identifies the target complex with $\bigoplus_j K_j(U)$. Direct sums of Λ_n-modules are exact, so degree $-r$ cohomology commutes with this direct sum. The resulting map is the displayed source-to-target map, hence is bijective. □

Lemma 11.12.184 (Weakly contractible affine objects cover). Lean: conditional-interface:lemma:weil-a4-weakly-contractible-affine-cover

Uses: def:weil-a4-compact-generator-family, def:weil-a4-selected-wcontractible-covering-family

Every object of the selected small pro-etale site is covered by objects belonging to I_X.

Proof. Uses: prop:weil-a4-selected-wcontractible-covering-properties,def:weil-a4-compact-generator-family

Apply $\mathfrak{W}(T)$ to the given object T. Every member is affine, quasi-compact, weakly contractible, and an object of the selected site, so it belongs to I_X. The same proposition says that these members cover T. □

Lemma 11.12.185 (The selected generators detect zero objects). Lean: conditional-interface:lemma:weil-a4-selected-generators-detect-zero

Uses: def:weil-a4-compact-generator-family

If $K \in D(X, \Lambda_n)$ and $\mathrm{Hom}(G, K) = 0$ for every $G \in \mathcal{G}_X$, then $K \simeq 0$.

Proof. Uses: lemma:weil-a4-generator-derived-hom-sections,lemma:weil-a4-weakly-contractible-affine-cover

If $K \not\simeq 0$, choose a with $\mathcal{H}^a(K) \neq 0$, an object T, and a nonzero section $x \in \mathcal{H}^a(K)(T)$. Pull x to a cover of T by selected weakly contractible affines. Since a sheaf section is zero exactly when it is zero on a cover, at least one pullback $x|_U$ is nonzero. The derived-Hom calculation identifies it with a nonzero map $G_{U,-a} \to K$, contradicting the hypothesis. □

11.12.3 The bounded-below coproduct calculation and the unbounded gap

Definition 11.12.186 (Uniformly bounded-below family). Lean: conditional-interface:def:weil-a4-uniformly-bounded-below-family

Uses: def:weil-a2-derived-category

Let T be a scheme and let $(K_s)_{s\in S}$ be a set-indexed family in $D(T_{\mathrm{pro\acute{e}t}}, \Lambda_n)$. Define $\mathrm{UBelow}_a((K_s)_s)$ to mean

$$\mathcal{H}^r(K_s) = 0 \qquad \text{for every } s \in S \text{ and every } r < a.$$

Define $\mathrm{UBelow}((K_s)_s)$ to mean that $\mathrm{UBelow}_a((K_s)_s)$ holds for some $a \in \mathbf{Z}$. The integer a is part of the datum whenever a total complex is formed.

Lemma 11.12.187 (Finite coproduct of weakly contractible affines). Lean: conditional-interface:lemma:weil-a4-finite-coproduct-wcontractible

Uses: def:weil-a4-selected-wcontractible-covering-family

Let $V_1, \dots, V_m$ be affine weakly contractible schemes. Then $V = \coprod_{i=1}^m V_i$ is affine, quasi-compact, and weakly contractible.

Proof. Uses: def:weil-a4-selected-wcontractible-covering-family

Write $V_i = \mathrm{Spec}(A_i)$. The finite coproduct is $\mathrm{Spec}(\prod_i A_i)$, hence affine and quasi-compact. Let $W \to V$ be a pro-étale covering. Base change to each open-and-closed summand gives a covering $W_i \to V_i$, and weak contractibility supplies a section $s_i : V_i \to W_i$. The coproduct of the s_i is a section of $W \to V$. Thus every pro-étale covering of V splits. □

Definition 11.12.188 (Selected one-member weakly contractible cover). Lean: conditional-interface:def:weil-a4-selected-qcqs-one-cover

Uses: def:weil-a4-selected-wcontractible-covering-family

For every quasi-compact quasi-separated object T of the pro-étale site, let $\mathfrak{W}(T) = \{V_i \to T\}_{i=1}^m$ be the selected finite affine weakly contractible covering family. Define

$$Q(T) := \coprod_{i=1}^m V_i, \qquad q_T : Q(T) \longrightarrow T$$

as the coproduct scheme and the map induced by the covering maps.

Proposition 11.12.189 (Properties of the selected one-member cover). *Lean:* `conditional-interface:prop:weil-a4-selected-qcqs-one-cover-properties`

Uses: `def:weil-a4-selected-qcqs-one-cover`

The object $Q(T)$ is affine, quasi-compact, and weakly contractible; the map q_T is quasi-compact and is a covering of the pro-étale site.

Proof. Uses: `lemma:weil-a4-finite-coproduct-wcontractible,prop:weil-a4-selected-wcontractible-covering-properties,def:weil-a4-selected-qcqs-one-cover`

The first three assertions follow from the finite-coproduct lemma. The family $\{V_i \to T\}$ is a covering by the selected-covering proposition, so the induced single map $\coprod_i V_i \to T$ is a covering. Every $V_i \to T$ is quasi-compact, and a finite coproduct of quasi-compact maps is quasi-compact. This proves the last assertion. □

Definition 11.12.190 (Recursively selected weakly contractible hypercover). Lean: `conditional-interface:def:weil-a4-selected-wcontractible-hypercover`

Uses: `def:weil-a4-selected-qcqs-one-cover`

Let T be quasi-compact and quasi-separated. Define an augmented simplicial pro-étale object $U_\bullet(T) \to T$ recursively. Put $N_0(T) = U_0(T) = Q(T)$. Suppose a split truncated simplicial object has been defined below degree r, with nondegenerate pieces $N_k(T)$. Let

$$M_r(T) := (\mathrm{cosk}^T_{r-1}\,\mathrm{sk}_{r-1}\,U_\bullet(T))_r$$

be the r-th matching object and define the new nondegenerate piece by

$$N_r(T) := Q(M_r(T)).$$

Define

$$U_r(T) := N_r(T) \amalg \coprod_{\substack{\sigma:[r]\twoheadrightarrow[k]\\ 0\le k<r}} N_k(T).$$

There are only finitely many displayed surjections. Map the first summand to $M_r(T)$ by $q_{M_r(T)}$; map the summand indexed by σ by the matching tuple of already-defined faces of the corresponding degenerate k-simplex. These maps define all faces from degree r.

The degeneracy belonging to a surjection $\sigma : [r] \twoheadrightarrow [k]$ is the inclusion of its displayed $N_k(T)$-summand. More generally, for a simplicial operator $\theta : [m] \to [r]$, factor the composite $\sigma\theta : [m] \to [k]$ uniquely as a surjection followed by an injection. Apply the already-defined face attached to the injection and then the summand inclusion attached to the surjection. The uniqueness of the epi–mono factorization in the simplex category gives the identity and composition equations for these operators. This completes the split simplicial recursion.

Lemma 11.12.191 (Matching objects remain quasi-compact and quasi-separated). *Lean:* `conditional-interface:lemma:weil-a4-hypercover-matching-qcqs`

Uses: `def:weil-a4-selected-wcontractible-hypercover`

For every $r \ge 0$, the matching object $M_r(T)$, the nondegenerate piece $N_r(T)$, and the selected term $U_r(T)$ are quasi-compact and quasi-separated.

Proof. Uses: `prop:weil-a4-selected-qcqs-one-cover-properties,def:weil-a4-selected-wcontractible-hypercover`

Proceed by induction on r. For $r = 0$, the matching object is T, which is quasi-compact and quasi-separated by hypothesis. Suppose this is known in lower degrees. The degree-r matching object is a finite limit of lower-degree $U_i(T)$ over T: only the finitely many injective maps between finite ordinals of size at most r occur. Finite fiber products of quasi-compact quasi-separated schemes over a quasi-separated base are quasi-compact and quasi-separated, and equalizers are inverse images of the diagonal of the target, which is quasi-compact because the target is quasi-separated. Thus $M_r(T)$ is quasi-compact and quasi-separated. The selected-cover proposition makes $N_r(T) = Q(M_r(T))$ affine and quasi-compact. The term $U_r(T)$ is a finite coproduct of $N_r(T)$ and lower-degree nondegenerate pieces, so it is quasi-compact and quasi-separated. □

Proposition 11.12.192 (The selected object is a weakly contractible hypercover). *Lean:* `conditional-interface:prop:weil-a4-selected-wcontractible-hypercover-properties`

Uses: `def:weil-a4-selected-wcontractible-hypercover`

For every quasi-compact quasi-separated T, the augmentation $U_\bullet(T) \to T$ is a hypercover and every $U_r(T)$ is a single affine quasi-compact weakly contractible object.

Proof. Uses: lemma:weil-a4-hypercover-matching-qcqs,prop:weil-a4-selected-qcqs-one-cover-properties,def:weil-a4-selected-wcontractible-hypercover

For each r, the preceding lemma permits application of the selected one-member cover to $M_r(T)$. Its property proposition says that $N_r(T) = Q(M_r(T)) \to M_r(T)$ is a cover and that $N_r(T)$ is affine, quasi-compact, and weakly contractible. Since the matching map $U_r(T) \to M_r(T)$ contains this cover as a coproduct summand, it is a cover. The defining condition for an augmented simplicial object to be a hypercover is precisely that every matching map be a cover.

Every $U_r(T)$ is a finite coproduct of the affine weakly contractible schemes $N_k(T)$. The finite-coproduct lemma makes it affine, quasi-compact, and weakly contractible. Hence both asserted properties hold. □

Definition 11.12.193 (Hypercover section double complex)**.** Lean: conditional-interface:def:weil-a4-hypercover-section-double-complex

Uses: def:weil-a4-selected-wcontractible-hypercover

Let $A^\bullet$ be a bounded-below complex of Λ_n-module sheaves on $T_{\text{proét}}$. Define the double complex

$$C^{r,s}(T, A) := \Gamma(U_r(T), A^s), \qquad r \geq 0,\ s \in \mathbf{Z}.$$

The vertical differential is that of $A^\bullet$, and the horizontal differential is

$$d_h = \sum_{a=0}^{r+1} (-1)^a (d_a)^* : C^{r,s} \longrightarrow C^{r+1,s}.$$

Define $\mathrm{Tot}^m C = \bigoplus_{r+s=m} C^{r,s}$ with differential $d_h + (-1)^r d_v$. Because $A^s = 0$ below a fixed degree, this direct sum has only finitely many nonzero summands in every total degree.

Definition 11.12.194 (Augmented free-sheaf complex of the selected hypercover)**.** Lean: conditional-interface:def:weil-a4-hypercover-augmented-free-sheaf-complex

Uses: def:weil-a4-selected-wcontractible-hypercover

For an object V of the pro-étale site, let $\Lambda_n[h_V]^\#$ denote the sheafification of the free Λ_n-module presheaf on the representable presheaf h_V. Define

$$P_{-1}(T) := \Lambda_n[h_T]^\#, \qquad P_r(T) := \Lambda_n[h_{U_r(T)}]^\# \quad (r \geq 0).$$

The augmentation $P_0(T) \to P_{-1}(T)$ is induced by $U_0(T) \to T$, and

$$\partial_r := \sum_{a=0}^{r} (-1)^a (d_a)_* : P_r(T) \longrightarrow P_{r-1}(T).$$

The simplicial identities give $\partial_{r-1}\partial_r = 0$, so $P_\bullet(T) \to P_{-1}(T)$ is an augmented chain complex of sheaves.

Lemma 11.12.195 (The augmented hypercover chain complex is locally contractible)**.** Lean: conditional-interface:lemma:weil-a4-hypercover-local-contraction

Uses: def:weil-a4-hypercover-augmented-free-sheaf-complex

The homology sheaf of the augmented complex $P_\bullet(T) \to P_{-1}(T)$ is zero in every degree $r \geq -1$.

Proof. Uses: prop:weil-a4-selected-wcontractible-hypercover-properties,def:weil-a4-hypercover-augmented-free-sheaf-complex

Fix $r \geq -1$, an object $V \to T$, and a section z of the kernel of ∂_r over V; for $r = -1$, read this as a section of P_{-1} whose pullback to P_0 is being sought. By the definition of sheafification, after replacing V by a covering family, z is a finite Λ_n-linear combination of maps $V \to U_r(T)$.

For each of the finitely many simplices occurring in z, collect its faces and all simplicial identities through degree $r+1$. These data define a map from V to the corresponding matching object $M_{r+1}(T)$. Since $U_{r+1}(T) \to M_{r+1}(T)$ is a cover, refine the covering of V so that each of these finitely many matching maps lifts to $U_{r+1}(T)$. Take a common finite refinement. Repeating the same argument downward for the finitely many faces supplies, on this refinement, maps s_{-1} satisfying

$$d_0 s_{-1} = \mathrm{id}, \qquad d_{a+1} s_{-1} = s_{-1} d_a \quad (0 \leq a \leq r).$$

These are precisely the extra-degeneracy identities. On the free module chain complex they imply, by an explicit alternating-sum calculation,

$$\partial s_{-1} + s_{-1}\partial = \mathrm{id}\,.$$

Indeed, the $d_0 s_{-1}$ term is the identity and, for each $a \geq 0$, the term $(-1)^{a+1} d_{a+1} s_{-1}$ cancels $(-1)^a s_{-1} d_a$. Since $\partial z = 0$, the equality gives $z = \partial(s_{-1} z)$ locally. Thus every kernel section is locally a boundary. Kernels, images, and homology in the category of sheaves are computed after sheafifying these local conditions, so every homology sheaf vanishes. □

Lemma 11.12.196 (Free representable sheaves represent sections)**.** *Lean:* `conditional-interface:lemma:weil-a4-free-representable-represents-sections`

Uses: `def:weil-a4-hypercover-augmented-free-sheaf-complex`

For every sheaf I of Λ_n-modules and every pro-étale object V, evaluation at the universal basis section gives a natural isomorphism

$$\mathrm{Hom}(\Lambda_n[h_V]^{\#}, I) \xrightarrow{\sim} \Gamma(V, I).$$

Proof. Uses: `def:weil-a4-hypercover-augmented-free-sheaf-complex`

A map from the free module presheaf $\Lambda_n[h_V]$ to the underlying presheaf of I is determined by the image of the basis vector $\mathrm{id}_V \in h_V(V)$. Naturality forces the basis vector attached to $a : W \to V$ to map to the restriction a^*s of that image. Conversely, every $s \in I(V)$ gives this natural transformation. The two constructions are inverse. The sheafification adjunction identifies maps from $\Lambda_n[h_V]^{\#}$ with maps from the presheaf $\Lambda_n[h_V]$, giving the displayed isomorphism. □

Lemma 11.12.197 (The selected hypercover gives a projective resolution)**.** *Lean:* `conditional-interface:lemma:weil-a4-hypercover-row-resolution`

Uses: `def:weil-a4-hypercover-augmented-free-sheaf-complex`

The augmented complex

$$\cdots \longrightarrow P_1(T) \longrightarrow P_0(T) \longrightarrow P_{-1}(T) \longrightarrow 0$$

is exact, and every $P_r(T)$ for $r \geq 0$ is projective in the category of Λ_n-module sheaves.

Proof. Uses: `lemma:weil-a4-hypercover-local-contraction,lemma:weil-a4-free-representable-represents-sections,prop:weil-a4-selected-wcontractible-hypercover-properties,lemma:weil-a4-weakly-contractible-sections-exact`

Exactness is the local-contraction lemma. Fix $r \geq 0$. The free-representable lemma identifies

$$\mathrm{Hom}(P_r(T), F) \cong \Gamma(U_r(T), F)$$

naturally in F. The selected-hypercover proposition says that $U_r(T)$ is weakly contractible. Evaluation on a weakly contractible object is exact by the section-exactness lemma. Hence $\mathrm{Hom}(P_r(T), -)$ is exact, which is the definition that $P_r(T)$ is projective. □

Lemma 11.12.198 (Bounded-below hypercover totalization)**.** *Lean:* `conditional-interface:lemma:weil-a4-bounded-below-hypercover-totalization`

Uses: `def:weil-a4-hypercover-section-double-complex`

Let $A^\bullet$ be a bounded-below complex representing $K \in D^+(T, \Lambda_n)$. Then $\mathrm{Tot}\, C^{\bullet,\bullet}(T, A)$ represents $R\Gamma(T, K)$. A quasi-isomorphism between bounded-below representatives induces a quasi-isomorphism between the corresponding total complexes.

Proof. Uses: `lemma:weil-a4-hypercover-row-resolution,lemma:weil-a4-free-representable-represents-sections,def:weil-a4-hypercover-section-double-complex`

Place $P_r(T)$ in cohomological degree $-r$. The preceding lemma makes the resulting bounded-above complex $P^\bullet(T)$ a complex of projective objects and makes its augmentation $P^\bullet(T) \to P_{-1}(T)[0]$ a quasi-isomorphism. A bounded-above complex of projectives is K-projective: if $B^\bullet$ is acyclic, solve the equation $dh + hd = u$ for a cochain map $u : P^\bullet(T) \to B^\bullet$ recursively from the largest degree downward, lifting each cycle through the preceding differential by projectivity. Thus

$$R\,\mathrm{Hom}(P_{-1}(T), K) \simeq \mathrm{Hom}^\bullet(P^\bullet(T), A^\bullet).$$

The free-representable lemma identifies the right-hand double complex with $C^{r,s}(T, A)$, including both differentials and their signs. The left side is $R\Gamma(T, K)$, because $P_{-1}(T) = \Lambda_n[h_T]^{\#}$ represents the section functor.

For the final assertion, apply $\mathrm{Hom}(P_r(T), -)$ to a quasi-isomorphism. Projectivity makes the induced column map a quasi-isomorphism. Filtering the total complexes by r, the bounded-below hypothesis leaves finitely many entries on every total diagonal, so the columnwise quasi-isomorphism induces a quasi-isomorphism on total complexes. □

Proposition 11.12.199 (Quasi-compact pro-etale sections preserve uniformly bounded-below coproducts)**.** *Lean:* `conditional-interface:prop:weil-a4-qcqs-rgamma-uniform-coproducts`

Uses: `def:weil-a4-uniformly-bounded-below-family`

Let T be quasi-compact and quasi-separated. If $\mathrm{UBelow}_a((K_s)_{s\in S})$, then the canonical map

$$\bigoplus_{s\in S} R\Gamma(T, K_s) \longrightarrow R\Gamma\left(T, \bigoplus_{s\in S} K_s\right)$$

is an isomorphism.

Proof. Uses: `lemma:weil-a4-bounded-below-hypercover-totalization,lemma:weil-a4-qc-sections-direct-sums,prop:weil-a4-selected-wcontractible-hypercover-properties,def:weil-a4-uniformly-bounded-below-family`

Choose an arbitrary complex representative for K_s and apply its good truncation $\tau_{\geq a}$. The common cohomological lower bound makes the truncation map a quasi-isomorphism. Denote the resulting bounded-below representative by $A_s^\bullet$. Direct sums of Λ_n-module sheaves are exact, so the degreewise complex $A^\bullet = \bigoplus_s A_s^\bullet$ represents $\bigoplus_s K_s$. In bidegree (r,t), quasi-compactness of the weakly contractible affine $U_r(T)$ and the section-coproduct lemma give

$$\Gamma\left(U_r(T), \bigoplus_s A_s^t\right) \cong \bigoplus_s \Gamma(U_r(T), A_s^t).$$

For fixed total degree m, the constraints $r \geq 0$, $t \geq a$, and $r + t = m$ leave only $0 \leq r \leq m - a$. Therefore totalization uses a finite sum in degree m, and finite sums commute with the outer direct sum over s. The induced isomorphism of total complexes is the canonical coproduct comparison under the bounded-below hypercover-totalization lemma. □

Definition 11.12.200 (Proper-pushforward evaluation comparison)**.** Lean: `conditional-interface:def:weil-a4-proper-pushforward-evaluation-comparison`

Uses: `def:weil-a2-derived-category`

Let $p : Z \to Y$ be a proper morphism and $U \to Y$ a quasi-compact weakly contractible affine pro-étale object. Write $Z_U = Z \times_Y U$. For $K \in D^+(Z, \Lambda_n)$, define

$$\beta_{p,U,K} : R\Gamma(U, Rp_*K) \longrightarrow R\Gamma(Z_U, K|_{Z_U})$$

as the derived comparison induced by the equality of underived composite functors $\Gamma(U, p_*(-)) = \Gamma(Z_U, -)$.

Lemma 11.12.201 (Proper-pushforward evaluation comparison is an isomorphism)**.** Lean: `conditional-interface:lemma:weil-a4-proper-pushforward-evaluation-isomorphism`

Uses: `def:weil-a4-proper-pushforward-evaluation-comparison`

For every $K \in D^+(Z, \Lambda_n)$, the map $\beta_{p,U,K}$ is an isomorphism.

Proof. Uses: `lemma:weil-a4-weakly-contractible-sections-exact,def:weil-a4-proper-pushforward-evaluation-comparison`

Choose a bounded-below injective resolution $K \to I^\bullet$. The direct image p_* is right adjoint to the exact inverse-image functor p^{-1}, so each p_*I^t is injective. Since sections on U are exact, $R\Gamma(U, Rp_*K)$ is represented by $\Gamma(U, p_*I^\bullet)$. By the defining equality of underived composites, this is termwise equal to $\Gamma(Z_U, I^\bullet|_{Z_U})$, which represents $R\Gamma(Z_U, K|_{Z_U})$. Under these representatives $\beta_{p,U,K}$ is the identity complex. □

Proposition 11.12.202 (Proper pushforward preserves uniformly bounded-below coproducts)**.** Lean: `conditional-interface:prop:weil-a4-proper-pushforward-uniform-coproducts`

Uses: `def:weil-a4-uniformly-bounded-below-family,def:weil-a4-proper-pushforward-evaluation-comparison`

Let $p : Z \to Y$ be proper and let $(K_s)_{s \in S}$ satisfy $\mathrm{UBelow}_a((K_s)_s)$*. Then*

$$\bigoplus_s Rp_*K_s \longrightarrow Rp_*\left(\bigoplus_s K_s\right)$$

is an isomorphism in $D(Y, \Lambda_n)$.

Proof. Uses: `lemma:weil-a4-proper-pushforward-evaluation-isomorphism,prop:weil-a4-qcqs-rgamma-uniform-coproducts,lemma:weil-a4-selected-generators-detect-zero,lemma:weil-a4-generator-derived-hom-sections`

Let C be the cone of the displayed comparison. It suffices to show $C \simeq 0$. Evaluate on a selected weakly contractible affine $U \to Y$. The proper base change $Z_U \to U$ is proper, hence Z_U is quasi-compact and quasi-separated. The evaluation-isomorphism lemma identifies the evaluated comparison with

$$\bigoplus_s R\Gamma(Z_U, K_s|_{Z_U}) \longrightarrow R\Gamma\left(Z_U, \bigoplus_s K_s|_{Z_U}\right),$$

which is an isomorphism by the quasi-compact section proposition. Therefore $H^r(C(U)) = 0$ for every r. The generator-derived-Hom formula says that every map from every selected shifted generator to C is zero. The generator-detection lemma gives $C \simeq 0$, so the comparison is an isomorphism. □

Definition 11.12.203 (Open-extension coproduct comparison)**.** Lean: `conditional-interface: def:weil-a4-open-extension-coproduct-comparison`

Uses: `def:weil-a2-derived-category`

For an open immersion $j : X \hookrightarrow \overline{X}$ and a family $(K_s)_s$ in $D(X, \Lambda_n)$, define

$$\alpha_{j,S} : \bigoplus_s j_! K_s \longrightarrow j_! \left(\bigoplus_s K_s \right)$$

by applying $j_!$ to each coproduct inclusion and using the coproduct universal property.

Lemma 11.12.204 (Open extension preserves arbitrary coproducts)**.** Lean: `conditional-interface: lemma: weil-a4-open-extension-coproduct-isomorphism`

Uses: `def: weil-a4-open-extension-coproduct-comparison`

The map $\alpha_{j,S}$ is an isomorphism for every set S, with no boundedness hypothesis.

Proof. Uses: `prop:weil-a1-open-extension-adjunction,def:weil-a1-open-extension-complex,def:weil-a4-open-extension-coproduct-comparison`

First let the K_s be sheaves F_s. For every sheaf G on $\overline{X}$, the extension adjunction and the coproduct universal property give natural bijections

$$\begin{aligned} \mathrm{Hom}\left(j_!(\bigoplus_s F_s), G \right) &\cong \mathrm{Hom}\left(\bigoplus_s F_s, j^* G \right) \\ &\cong \prod_s \mathrm{Hom}(F_s, j^* G) \\ &\cong \prod_s \mathrm{Hom}(j_! F_s, G) \\ &\cong \mathrm{Hom}\left(\bigoplus_s j_! F_s, G \right). \end{aligned}$$

Tracing a morphism through the four bijections shows that the resulting precomposition map is precomposition by $\alpha_{j,S}$. Since this map is bijective naturally in G, Yoneda says that $\alpha_{j,S}$ is an isomorphism on sheaves.

The selected ambient construction applies the A1 complex functor $j_!^{\mathrm{cx}}$ termwise. Coproducts of complexes of Λ_n-module sheaves are degreewise. The sheaf-level comparison is therefore an isomorphism in every cochain degree, hence an isomorphism of complexes. Its image under localization is the asserted comparison in the ambient derived category. This calculation concerns the coproduct map itself; descent of the ambient functor remains separately indexed by `OBL-A4-EX-01`. □

Proposition 11.12.205 (A fixed compactification preserves uniformly bounded-below coproducts)**.** Lean: `conditional-interface: prop: weil-a4-fixed-compactification-uniform-coproducts`

Uses: `def: weil-a4-ambient-rfshriek-candidate, def: weil-a4-uniformly-bounded-below-family`

Let $f = pj$ be the A1-selected compactification, with j open and p proper. If $\mathrm{UBelow}_a((K_s)_s)$, then the comparison

$$\bigoplus_s Rp_* j_! K_s \longrightarrow Rp_* j_! \left(\bigoplus_s K_s \right)$$

is an isomorphism.

Proof. Uses: `lemma:weil-a4-open-extension-coproduct-isomorphism,prop:weil-a4-proper-pushforward-uniform-coproducts,def:weil-a4-ambient-rfshriek-candidate`

Exactness of $j_!$ preserves the common lower bound a. Factor the displayed comparison as

$$\bigoplus_s Rp_* j_! K_s \longrightarrow Rp_* \left(\bigoplus_s j_! K_s \right) \xrightarrow{Rp_*(\alpha_{j,S})} Rp_* j_! \left(\bigoplus_s K_s \right).$$

The first arrow is an isomorphism by the proper-pushforward proposition and the second by the open-extension lemma. Their composite is the canonical comparison, hence is an isomorphism. □

Definition 11.12.206 (Ambient refinement-comparison datum). Lean: `conditional-interface:def:weil-a4-ambient-refinement-comparison-datum`

Uses: `def:weil-a4-ambient-rfshriek-candidate`

For two compactifications $f = p_1 j_1 = p_2 j_2$, an ambient refinement-comparison datum consists of a common refinement (j_3, p_3), proper maps $r_i : \overline{X}_3 \to \overline{X}_i$ satisfying $r_i j_3 = j_i$ and $p_i r_i = p_3$, and natural isomorphisms

$$\rho_i : Rp_{3*} j_{3!} \xrightarrow{\sim} Rp_{i*} j_{i!} \quad (i = 1, 2)$$

on the selected ambient derived category. The datum additionally requires that ρ_i be induced by the unit $j_{3!} \to Rr_{i*} r_i^* j_{3!} = Rr_{i*} j_{i!}$ and the equality $Rp_{i*} Rr_{i*} = Rp_{3*}$.

Lemma 11.12.207 (Refinement comparison respects bounded-below coproducts). Lean: `conditional-interface:lemma:weil-a4-refinement-uniform-coproduct-rectangle`

Uses: `def:weil-a4-ambient-refinement-comparison-datum`, `def:weil-a4-uniformly-bounded-below-family`

Given an ambient refinement-comparison datum and a uniformly bounded-below family $(K_s)_s$, the square

$$\begin{array}{ccc} \bigoplus_s Rp_{3*} j_{3!} K_s & \longrightarrow & Rp_{3*} j_{3!} (\bigoplus_s K_s) \\ {\scriptstyle \oplus \rho_i} \downarrow & & \downarrow {\scriptstyle \rho_i} \\ \bigoplus_s Rp_{i*} j_{i!} K_s & \longrightarrow & Rp_{i*} j_{i!} (\bigoplus_s K_s) \end{array}$$

commutes for $i = 1, 2$.

Proof. Uses: `def:weil-a4-ambient-refinement-comparison-datum`

Both routes from the upper-left corner are determined by their restrictions to each coproduct summand. On the s-th summand, the upper route is the functoriality square of the natural transformation ρ_i applied to the coproduct inclusion $K_s \to \bigoplus_t K_t$. Naturality says that this equals the lower route. The coproduct universal property then gives equality of the two arrows. □

Proposition 11.12.208 (Bounded-below coproduct compatibility is compactification-independent). Lean: `conditional-interface:prop:weil-a4-uniform-coproduct-compactification-independent`

Uses: `def:weil-a4-ambient-refinement-comparison-datum`, `def:weil-a4-uniformly-bounded-below-family`

Given an ambient refinement-comparison datum, the coproduct comparison is an isomorphism for one of the three compactifications if and only if it is an isomorphism for each of the other two.

Proof. Uses: `lemma:weil-a4-refinement-uniform-coproduct-rectangle`, `prop:weil-a4-fixed-compactification-uniform-coproducts`

In the commuting square of the preceding lemma, both vertical maps are isomorphisms by the refinement datum. Therefore the upper horizontal map is an isomorphism if and only if the lower horizontal map is an isomorphism. Apply this once for $i = 1$ and once for $i = 2$. The fixed-compactification proposition supplies the isomorphism for the selected compactification, so all comparisons related to it by the datum agree. □

Definition 11.12.209 (Left Postnikov reconstruction map). Lean: `conditional-interface:def:weil-a4-left-postnikov-reconstruction`

Uses: `def:weil-a2-derived-category`

For $K \in D(T, \Lambda_n)$, let $\tau_{\geq -m} K \to \tau_{\geq -(m-1)} K$ be the canonical truncation maps. Define

$$P_m(K) := \tau_{\geq -m} K, \qquad P_\infty(K) := \operatorname*{holim}_{m \geq 0} P_m(K),$$

where the homotopy inverse limit is the fiber of

$$1 - \mathrm{shift} : \prod_{m \geq 0} P_m(K) \longrightarrow \prod_{m \geq 0} P_m(K).$$

Define the reconstruction map $\lambda_K : K \to P_\infty(K)$ by the compatible family of truncation maps $K \to P_m(K)$.

Lemma 11.12.210 (The pro-etale derived category is left complete). Lean: `conditional-interface:lemma:weil-a4-proetale-left-complete`

Uses: `def:weil-a4-left-postnikov-reconstruction`

For every $K \in D(T, \Lambda_n)$, the reconstruction map λ_K is an isomorphism.

Proof. Uses: lemma:weil-a4-selected-generators-detect-zero,lemma:weil-a4-generator-derived-hom-sections,def:weil-a4-left-postnikov-reconstruction

Let $U \to T$ be a selected weakly contractible affine. Exactness of sections makes evaluation on U t-exact and identifies $P_m(K)(U)$ with $\tau_{\geq -m}(K(U))$. Fix an integer r. For every $m > -r$, the truncation map induces an equality

$$H^r(K(U)) \xrightarrow{\sim} H^r(P_m(K)(U)),$$

and all subsequent transition maps are identities under this identification. The inverse system of r-th cohomology groups is thus eventually constant; its $\varprojlim^1$ vanishes. The Milnor exact sequence for the defining fiber of the homotopy inverse limit gives

$$H^r(P_\infty(K)(U)) \cong \varprojlim_m H^r(P_m(K)(U)) \cong H^r(K(U)).$$

Under this isomorphism $H^r(\lambda_K(U))$ is the identity. Hence the cone of λ_K has zero cohomology on every selected weakly contractible affine. The generator-derived-Hom formula makes it right-orthogonal to all selected shifted generators, and generator detection makes the cone zero. Thus λ_K is an isomorphism. □

Definition 11.12.211 (Full pro-etale cohomological-dimension predicate)**.** Lean: conditional-interface:def:weil-a4-full-proetale-cd-predicate

Uses: def:weil-a2-derived-category

Let $p : Z \to Y$ be a proper morphism. Define $\mathrm{FullCD}_{\mathrm{pro\acute{e}t}}(p, \Lambda_n; d)$ to mean: for every quasi-compact weakly contractible affine pro-étale object $U \to Y$, every sheaf F of Λ_n-modules on $(Z \times_Y U)_{\mathrm{pro\acute{e}t}}$, and every $r > d$,

$$H^r(Z \times_Y U, F) = 0.$$

The quantifier ranges over all finite-coefficient sheaves on the full pro-étale site, not only sheaves pulled back from the small étale site and not only constructible sheaves.

Lemma 11.12.212 (Full cohomological dimension controls truncation)**.** Lean: conditional-interface:lemma:weil-a4-full-cd-truncation-control

Uses: def:weil-a4-full-proetale-cd-predicate

Assume $\mathrm{FullCD}_{\mathrm{pro\acute{e}t}}(p, \Lambda_n; d)$*. For* $K \in D(Z, \Lambda_n)$*, a selected weakly contractible affine* $U \to Y$*, and* $r \in \mathbf{Z}$*, the map*

$$H^r R\Gamma(Z_U, K|_{Z_U}) \longrightarrow H^r R\Gamma(Z_U, \tau_{\geq r-d} K|_{Z_U})$$

is an isomorphism.

Proof. Uses: def:weil-a4-full-proetale-cd-predicate

Put $L = \tau_{<r-d} K|_{Z_U}$. The hypercohomology spectral sequence for the bounded-above complex L is

$$E_2^{a,b} = H^a(Z_U, \mathcal{H}^b(L)) \Longrightarrow H^{a+b} R\Gamma(Z_U, L).$$

The full cohomological-dimension hypothesis kills all terms with $a > d$, while the truncation kills all terms with $b \geq r - d$. Hence every nonzero term satisfies $a + b \leq d + (r - d - 1) = r - 1$. It follows directly from the filtered complex that $H^r R\Gamma(Z_U, L) = H^{r+1} R\Gamma(Z_U, L) = 0$. Apply $R\Gamma(Z_U, -)$ to the truncation triangle $L \to K|_{Z_U} \to \tau_{\geq r-d} K|_{Z_U} \to L[1]$. The two displayed vanishings make the middle map on H^r an isomorphism. □

Proposition 11.12.213 (Full cohomological dimension implies arbitrary proper coproducts)**.** Lean: conditional-interface:prop:weil-a4-full-cd-implies-proper-coproducts

Uses: def:weil-a4-full-proetale-cd-predicate

If $\mathrm{FullCD}_{\mathrm{pro\acute{e}t}}(p, \Lambda_n; d)$*, then for every set-indexed family* $(K_s)_{s \in S}$ *in the unbounded category* $D(Z, \Lambda_n)$*, the canonical map*

$$\bigoplus_s Rp_* K_s \longrightarrow Rp_* \left(\bigoplus_s K_s \right)$$

is an isomorphism.

Proof. Uses: lemma:weil-a4-full-cd-truncation-control,prop:weil-a4-proper-pushforward-uniform-coproducts,lemma:weil-a4-proetale-left-complete,lemma:weil-a4-selected-generators-detect-zero,lemma:weil-a4-generator-derived-hom-sections

Let C be the cone of the comparison. Fix a selected weakly contractible affine $U \to Y$ and a degree r. By truncation control, the r-th cohomology of each of the two evaluated sides is unchanged if every K_s is replaced by $\tau_{\geq r-d} K_s$. Exactness of direct sums of Λ_n-modules gives

$$\tau_{\geq r-d} \left(\bigoplus_s K_s \right) \simeq \bigoplus_s \tau_{\geq r-d} K_s.$$

The truncated family has the common lower bound $r - d$, so the uniformly bounded-below proper-pushforward proposition says that its coproduct comparison is an isomorphism. Therefore $H^r(C(U)) = 0$. Since U and r were arbitrary, generator-derived Hom and generator detection imply $C \simeq 0$. Left completeness guarantees that the argument applies to the original unbounded objects rather than only to their Postnikov towers. □

Definition 11.12.214 (Candidate full pro-etale bound for the selected proper map). Lean: `conditional-interface:def:weil-a4-selected-proper-full-cd-candidate`

Uses: `def:weil-a4-full-proetale-cd-predicate,def:weil-a4-ambient-rfshriek-candidate`

Let

$$S_f := \{d \in \mathbf{N} : \mathrm{FullCD}_{\mathrm{pro\acute{e}t}}(p_f, \Lambda_n; d)\}.$$

Define $d_f^{\mathrm{full}} = \min S_f$ when $S_f \neq \varnothing$, and define $d_f^{\mathrm{full}} = 0$ when $S_f = \varnothing$. This total definition does not assert that S_f is inhabited.

Proposition 11.12.215 (The missing full pro-etale bound for the selected proper map). Lean: `conditional-interface:prop:weil-a4-selected-proper-full-cd-open`

Uses: `def:weil-a4-selected-proper-full-cd-candidate`

OPEN. *For the proper map $p_f : \overline{X}_f \to Y$ in the selected compactification, the candidate integer satisfies*

$$\mathrm{FullCD}_{\mathrm{pro\acute{e}t}}(p_f, \Lambda_n; d_f^{\mathrm{full}})$$

Open obligation: `OBL-A4-EX-02`

Proof. Uses: `def:weil-a4-selected-proper-full-cd-candidate,def:weil-a4-full-proetale-cd-predicate`

No proof is currently available in the selected ambient category. The classical bound $H^r_{\acute{e}t}(V, F) = 0$ for $r > 2 \dim V$ is a theorem on the small étale site. Its hypotheses quantify over étale sheaves and do not imply the displayed predicate, which quantifies over every sheaf on the full pro-étale site. Likewise, the pro-étale hypercover calculation proved above commutes with coproducts only for families with a common lower bound; in the unbounded case infinitely many cochain degrees can contribute to one total degree. Thus importing the étale $2d$ estimate here would change the site and would not prove the statement. This proposition is the precise remaining endpoint of 'OBL-A4-EX-02'. □

Definition 11.12.216 (Compact-support coproduct comparison). Lean: `conditional-interface:def:weil-a4-rfshriek-coproduct-comparison`

Uses: `def:weil-a4-exceptional-index,def:weil-a4-ambient-rfshriek-candidate`

For a family $(K_j)_{j \in J}$ in $D(X, \Lambda_n)$, define

$$c^!_{f,J} : \bigoplus_{j \in J} Rf_{!,\mathrm{amb}} K_j \longrightarrow Rf_{!,\mathrm{amb}} \left(\bigoplus_{j \in J} K_j \right)$$

as the coproduct map whose j-component is $Rf_{!,\mathrm{amb}}$ of the j-th coproduct inclusion.

Proposition 11.12.217 (Compact-support pushforward preserves direct sums). Lean: `conditional-interface:prop:weil-a4-rfshriek-preserves-coproducts-open`

Uses: `def:weil-a4-rfshriek-coproduct-comparison`

OPEN. *The map $c^!_{f,J}$ is an isomorphism for every set J.*

Open obligation: `OBL-A4-EX-02`

Proof. Uses: `prop:weil-a4-selected-proper-full-cd-open,prop:weil-a4-full-cd-implies-proper-coproducts,lemma:weil-a4-open-extension-coproduct-isomorphism,prop:weil-a4-fixed-compactification-uniform-coproducts,def:weil-a4-rfshriek-coproduct-comparison,def:weil-a4-ambient-rfshriek-candidate`

Expand the selected compactification as $Rf_{!,\mathrm{amb}} = Rp_{f*} j_{f!}$. The open-extension comparison is an isomorphism for every family. The OPEN full-cohomological-dimension proposition, instantiated with d_f^{full}, and the conditional unbounded proper-coproduct proposition make the proper-pushforward comparison an isomorphism. Their composite is $c^!_{f,J}$.

Without the OPEN input, the fixed-compactification proposition already proves this argument for every family with a common lower bound. What is not proved is the passage from those families to arbitrary unbounded families in the full pro-étale ambient category. Thus this proposition carries exactly 'OBL-A4-EX-02', represented by the single upstream statement $\mathrm{FullCD}_{\mathrm{pro\acute{e}t}}(p_f, \Lambda_n; d_f^{\mathrm{full}})$. □

Definition 11.12.218 (Exceptional-Hom coproduct comparison)**.** Lean: `conditional-interface:def:weil-a4-exceptional-hom-coproduct-comparison`

Uses: `def:weil-a4-exceptional-hom-functor`, `def:weil-a4-rfshriek-coproduct-comparison`

Define

$$\delta_{J,(K_j)} : H_{f,L}\left(\bigoplus_j K_j\right) \longrightarrow \prod_j H_{f,L}(K_j)$$

by $\delta(a)_j = a \circ Rf_{!,\mathrm{amb}}(\iota_j)$, where $\iota_j : K_j \to \bigoplus_j K_j$ is the coproduct inclusion.

Proposition 11.12.219 (Exceptional Hom converts coproducts to products)**.** Lean: *`conditional-interface:prop:weil-a4-exceptional-hom-coproduct-product-open`*

Uses: *`def:weil-a4-exceptional-hom-coproduct-comparison`*

OPEN. *The map $\delta_{J,(K_j)}$ is bijective.*

Open obligation: *`OBL-A4-EX-02`*

Proof. Uses: `prop:weil-a4-rfshriek-preserves-coproducts-open`, `def:weil-a4-exceptional-hom-coproduct-comparison`

Use $(c^!_{f,J})^{-1}$ to replace a map $Rf_{!,\mathrm{amb}}(\bigoplus_j K_j) \to L$ by a map $\bigoplus_j Rf_{!,\mathrm{amb}}K_j \to L$. The coproduct universal property identifies the latter map with its family of restrictions to $Rf_{!,\mathrm{amb}}K_j$. The two inverse constructions are composition with $c^!_{f,J}$ and $(c^!_{f,J})^{-1}$; their inverse equations are associativity followed by the two inverse equations for $c^!_{f,J}$. Thus the only unresolved input is 'OBL-A4-EX-02'. □

Proposition 11.12.220 (Small compact generating family)**.** Lean: *`conditional-interface:prop:weil-a4-generator-small-compact-open`*

Uses: *`def:weil-a4-compact-generator-family`*

OPEN. *The collection $\mathcal{G}_X$ is a set of compact objects, is stable under shifts, and its localizing closure is all of $D(X,\Lambda_n)$, hence contains $D^b_c(X,\Lambda_n)$.*

Open obligation: *`OBL-A4-EX-02`*

Proof. Uses: `lemma:weil-a4-selected-generator-compact`, `lemma:weil-a4-selected-generators-detect-zero`

The chosen small-site skeleton and $\mathbf{Z}$ are sets, so $\mathcal{G}_X$ is a set; its definition gives shift stability. The compactness lemma treats every member. Let $\mathcal{C}$ be the localizing closure of $\mathcal{G}_X$ and let K be right-orthogonal to $\mathcal{C}$. Then it is right-orthogonal to every generator and is zero by detection. Hence the right orthogonal of $\mathcal{C}$ is zero, which is the generator criterion for $\mathcal{C} = D(X,\Lambda_n)$. Detection still carries exactly 'OBL-A4-EX-02'. □

Definition 11.12.221 (Generator element index)**.** Lean: `conditional-interface:def:weil-a4-generator-element-index`

Uses: `def:weil-a4-exceptional-hom-functor`, `def:weil-a4-compact-generator-family`

Define the set

$$S_0(f,L) = \{(G,\xi) : G \in \mathcal{G}_X,\ \xi \in H_{f,L}(G)\}.$$

For $s = (G,\xi)$, write $G_s = G$ and $\xi_s = \xi$.

Definition 11.12.222 (Zeroth Brown object)**.** Lean: `conditional-interface:def:weil-a4-brown-zero-object`

Uses: `def:weil-a4-generator-element-index`

Define

$$B_0(f,L) := \bigoplus_{s \in S_0(f,L)} G_s,$$

and write $i_s : G_s \to B_0(f,L)$ for the s-th inclusion.

Definition 11.12.223 (Zeroth evaluation fiber)**.** Lean: `conditional-interface:def:weil-a4-brown-zero-evaluation-fiber`

Uses: `def:weil-a4-brown-zero-object`, `def:weil-a4-exceptional-hom-coproduct-comparison`

Define

$$\mathcal{E}_0(f,L) := \{e \in H_{f,L}(B_0) : H_{f,L}(i_s)(e) = \xi_s \text{ for every } s \in S_0(f,L)\}.$$

Lemma 11.12.224 (The zeroth evaluation fiber is inhabited). *Lean:* `conditional-interface:lemma:weil-a4-brown-zero-evaluation-fiber-inhabited-open`

Uses: `def:weil-a4-brown-zero-evaluation-fiber`
OPEN. *The set $\mathcal{E}_0(f, L)$ is nonempty.*

Open obligation: `OBL-A4-EX-02`

Proof. Uses: `prop:weil-a4-exceptional-hom-coproduct-product-open,def:weil-a4-brown-zero-evaluation-fiber`
The tuple $(\xi_s)_s$ belongs to $\prod_s H_{f,L}(G_s)$. Surjectivity of $\delta_{S_0,(G_s)}$ supplies $e \in H_{f,L}(B_0)$ whose s-th restriction is ξ_s. This is exactly membership in $\mathcal{E}_0(f, L)$; the product comparison carries 'OBL-A4-EX-02'. □

Definition 11.12.225 (Chosen zeroth Brown evaluation). Lean: `conditional-interface:def:weil-a4-brown-zero-evaluation`

Uses: `def:weil-a4-brown-zero-evaluation-fiber`
After fixing the nonemptiness witness indexed by 'OBL-A4-EX-02', define

$$e_0(f, L) := \text{Classical.choose}(\mathcal{E}_0(f, L)) \in H_{f,L}(B_0).$$

Lemma 11.12.226 (Zeroth evaluation component equation). *Lean:* `conditional-interface:lemma:weil-a4-brown-zero-evaluation-components-open`

Uses: `def:weil-a4-brown-zero-evaluation,def:weil-a4-brown-zero-evaluation-fiber`
OPEN. *For every $s \in S_0(f, L)$, $H_{f,L}(i_s)(e_0) = \xi_s$.*

Open obligation: `OBL-A4-EX-02`

Proof. Uses: `lemma:weil-a4-brown-zero-evaluation-fiber-inhabited-open,def:weil-a4-brown-zero-evaluation`
The choice operator returns an element of the fiber to which it is applied. Expanding membership in $\mathcal{E}_0(f, L)$ gives the displayed component equation. Its only unresolved premise is the fiber nonemptiness under 'OBL-A4-EX-02'. □

Definition 11.12.227 (Evaluated Brown stage datum). Lean: `conditional-interface:def:weil-a4-brown-stage-datum`

Uses: `def:weil-a4-exceptional-hom-functor`
An evaluated Brown stage datum is a pair $D = (B, e)$ with $B \in D(X, \Lambda_n)$ and $e \in H_{f,L}(B)$. For a map $u : K \to B$, define

$$\theta_{D,K}(u) := H_{f,L}(u)(e).$$

Lemma 11.12.228 (Stage evaluation is natural). *Lean:* `conditional-interface:lemma:weil-a4-brown-stage-evaluation-natural`

Uses: `def:weil-a4-brown-stage-datum`
For $v : K' \to K$ and $u : K \to B$, $\theta_{D,K'}(uv) = H_{f,L}(v)(\theta_{D,K}(u))$.

Proof. Uses: `def:weil-a4-brown-stage-datum,lemma:weil-a4-exceptional-hom-composition`
The left side is $H_{f,L}(uv)(e)$. The composition law rewrites this as $H_{f,L}(v)(H_{f,L}(u)(e))$, which is the right side. □

Definition 11.12.229 (Zeroth evaluated stage). Lean: `conditional-interface:def:weil-a4-brown-zero-stage`

Uses: `def:weil-a4-brown-stage-datum,def:weil-a4-brown-zero-object,def:weil-a4-brown-zero-evaluation`
Define $D_0(f, L) := (B_0(f, L), e_0(f, L))$.

Definition 11.12.230 (Relation index of an evaluated stage). Lean: `conditional-interface:def:weil-a4-brown-relation-index`

Uses: `def:weil-a4-brown-stage-datum,def:weil-a4-compact-generator-family`
For $D = (B, e)$, define

$$R(D) := \{(G, u) : G \in \mathcal{G}_X,\ u : G \to B,\ \theta_{D,G}(u) = 0\}.$$

Lemma 11.12.231 (The relation index is a set). *Lean:* `conditional-interface:lemma:weil-a4-brown-relation-index-small`

Uses: `def:weil-a4-brown-relation-index`
The collection $R(D)$ is a set.

Proof. Uses: `def:weil-a4-brown-relation-index,def:weil-a2-derived-category`
The generator index is a set. For each generator, the localization defining the derived category is locally small, so $\mathrm{Hom}(G, B)$ is a set. The disjoint union of these Hom sets is a set, and $R(D)$ is the subset cut out by the equality $\theta_{D,G}(u) = 0$. □

Definition 11.12.232 (Relation object and relation-killing map). Lean: `conditional-interface:def:weil-a4-brown-relation-map`

Uses: `def:weil-a4-brown-relation-index`
Define

$$A(D) := \bigoplus_{(G,u)\in R(D)} G, \qquad \rho_D : A(D) \longrightarrow B$$

by the coproduct rule $\rho_D \iota_{(G,u)} = u$.

Lemma 11.12.233 (Relation-map component equation). *Lean:* `conditional-interface:lemma:weil-a4-brown-relation-map-component`

Uses: `def:weil-a4-brown-relation-map`
For $(G, u) \in R(D)$, $\rho_D \iota_{(G,u)} = u$.

Proof. Uses: `def:weil-a4-brown-relation-map`
This is the component equation in the coproduct universal property used to define ρ_D. □

Definition 11.12.234 (Selected successor mapping cone). Lean: `conditional-interface:def:weil-a4-brown-successor-object`

Uses: `def:weil-a4-brown-relation-map,def:weil-a2-derived-category`
Choose a roof of cochain maps representing ρ_D, replace its left quasi-isomorphism by its mapping-cylinder factorization, and let $B^+(D)$ be the image in the derived category of the ordinary cochain mapping cone of the resulting cochain map. Denote the induced maps by

$$A(D) \xrightarrow{\rho_D} B \xrightarrow{b_D} B^+(D) \xrightarrow{c_D} A(D)[1].$$

Lemma 11.12.235 (The selected successor triangle is distinguished). *Lean:* `conditional-interface:lemma:weil-a4-brown-successor-triangle-distinguished`

Uses: `def:weil-a4-brown-successor-object`
The triangle $A(D) \xrightarrow{\rho_D} B \xrightarrow{b_D} B^+(D) \xrightarrow{c_D} A(D)[1]$ is distinguished.

Proof. Uses: `def:weil-a4-brown-successor-object,prop:weil-a2-derived-category-triangulated`
The A2 triangulation declares the localization of every cochain mapping-cone triangle to be distinguished. The selected successor is defined to be exactly that localization. □

Lemma 11.12.236 (The stage evaluation vanishes on the relation object). *Lean:* `conditional-interface:lemma:weil-a4-brown-relation-evaluation-zero-open`

Uses: `def:weil-a4-brown-relation-map, def:weil-a4-brown-stage-datum`
OPEN. *$H_{f,L}(\rho_D)(e) = 0$ in $H_{f,L}(A(D))$.*

Open obligation: `OBL-A4-EX-02`

Proof. Uses: `prop:weil-a4-exceptional-hom-coproduct-product-open,lemma:weil-a4-brown-relation-map-component,def:weil-a4-brown-relation-index`
Apply the product comparison to $H_{f,L}(\rho_D)(e)$. Its coordinate at (G, u) is

$$H_{f,L}(\iota_{(G,u)})(H_{f,L}(\rho_D)(e)) = H_{f,L}(u)(e) = \theta_{D,G}(u) = 0.$$

Injectivity of the product comparison makes the original element zero. That injectivity carries 'OBL-A4-EX-02'. □

Definition 11.12.237 (Successor evaluation fiber). Lean: `conditional-interface:def:weil-a4-brown-successor-evaluation-fiber`

Uses: `def:weil-a4-brown-successor-object`, `def:weil-a4-brown-stage-datum`

Define

$$\mathcal{E}^+(D) := \{e^+ \in H_{f,L}(B^+(D)) : H_{f,L}(b_D)(e^+) = e\}.$$

Lemma 11.12.238 (The successor evaluation fiber is inhabited). Lean: *conditional-interface:lemma:weil-a4-brown-successor-evaluation-inhabited-open*

Uses: *def:weil-a4-brown-successor-evaluation-fiber*

OPEN. *The set $\mathcal{E}^+(D)$ is nonempty.*

Open obligation: *OBL-A4-EX-01*

Open obligation: *OBL-A4-EX-02*

Proof. Uses: `prop:weil-a4-exceptional-hom-cohomological-open`, `lemma:weil-a4-brown-successor-triangle-distinguished`, `lemma:weil-a4-brown-relation-evaluation-zero-open`

Apply the contravariant cohomological functor to the successor triangle. The relevant exact segment is

$$H_{f,L}(B^+(D)) \xrightarrow{H(b_D)} H_{f,L}(B) \xrightarrow{H(\rho_D)} H_{f,L}(A(D)).$$

The relation-evaluation lemma puts e in the kernel of the second arrow. Exactness supplies e^+ mapping to e, which is an element of the displayed fiber. Exactness and relation vanishing carry precisely 'OBL-A4-EX-01' and 'OBL-A4-EX-02'. □

Definition 11.12.239 (Chosen successor evaluation). Lean: `conditional-interface:def:weil-a4-brown-successor-evaluation`

Uses: `def:weil-a4-brown-successor-evaluation-fiber`

After fixing the nonemptiness witness, define

$$e^+(D) := \mathrm{Classical.choose}(\mathcal{E}^+(D)).$$

No uniqueness is asserted or required.

Lemma 11.12.240 (Successor evaluation compatibility). Lean: *conditional-interface:lemma:weil-a4-brown-successor-evaluation-compatible-open*

Uses: *def:weil-a4-brown-successor-evaluation*, *def:weil-a4-brown-stage-datum*

OPEN. $H_{f,L}(b_D)(e^+(D)) = e$.

Open obligation: *OBL-A4-EX-01*

Open obligation: *OBL-A4-EX-02*

Proof. Uses: `lemma:weil-a4-brown-successor-evaluation-inhabited-open`, `def:weil-a4-brown-successor-evaluation-fiber`

The choice lies in $\mathcal{E}^+(D)$. Expanding membership in that fiber is the displayed equation. Its inherited premises are the two listed obligations. □

Definition 11.12.241 (Successor evaluated stage). Lean: `conditional-interface:def:weil-a4-brown-successor-stage`

Uses: `def:weil-a4-brown-stage-datum`, `def:weil-a4-brown-successor-object`, `def:weil-a4-brown-successor-evaluation`

Define $D^+ := (B^+(D), e^+(D))$.

Definition 11.12.242 (Recursive Brown stages). Lean: `conditional-interface:def:weil-a4-brown-stage-recursion`

Uses: `def:weil-a4-brown-zero-stage`, `def:weil-a4-brown-successor-stage`

Define recursively

$$D_0 = (B_0, e_0), \qquad D_{m+1} = D_m^+.$$

Write $D_m = (B_m, e_m)$, and write $b_m : B_m \to B_{m+1}$ for b_{D_m}.

Lemma 11.12.243 (Stage evaluation compatibility). Lean: *conditional-interface:lemma:weil-a4-brown-stage-evaluation-compatible-open*

Uses: *def:weil-a4-brown-stage-recursion*

OPEN. *For every m, $H_{f,L}(b_m)(e_{m+1}) = e_m$.*

Open obligation: *OBL-A4-EX-01*

Open obligation: *OBL-A4-EX-02*

Proof. Uses: lemma:weil-a4-brown-successor-evaluation-compatible-open,def:weil-a4-brown-stage-recursion

Substitute $D = D_m$ in the successor compatibility equation and expand the recursive definitions of B_{m+1}, e_{m+1}, and b_m. □

Lemma 11.12.244 (Every indexed relation is killed at the successor stage). Lean: conditional-interface:lemma:weil-a4-brown-indexed-relation-killed

Uses: def:weil-a4-brown-stage-recursion,def:weil-a4-brown-relation-index

If $(G,u) \in R(D_m)$, then $b_m u = 0$.

Proof. Uses: lemma:weil-a4-brown-relation-map-component,lemma:weil-a4-brown-successor-triangle-distinguished,def:weil-a4-brown-stage-recursion

The component equation gives $u = \rho_{D_m}\iota_{(G,u)}$. Consecutive arrows in the distinguished successor triangle compose to zero, so $b_m\rho_{D_m} = 0$. Therefore $b_m u = b_m\rho_{D_m}\iota_{(G,u)} = 0$. □

Definition 11.12.245 (Brown telescope shift). Lean: conditional-interface:def:weil-a4-brown-telescope-shift

Uses: def:weil-a4-brown-stage-recursion

Let $P := \bigoplus_{m\geq 0} B_m$, with inclusions $j_m : B_m \to P$. Define $s : P \to P$ by the component equations

$$sj_m = j_{m+1}b_m,$$

and define $d := \mathrm{id}_P - s$.

Lemma 11.12.246 (Telescope differential component). Lean: conditional-interface:lemma:weil-a4-brown-telescope-differential-component

Uses: def:weil-a4-brown-telescope-shift

For every m, $dj_m = j_m - j_{m+1}b_m$.

Proof. Uses: def:weil-a4-brown-telescope-shift

Expand $d = \mathrm{id}_P - s$, compose with j_m, and replace sj_m by its defining component $j_{m+1}b_m$. □

Definition 11.12.247 (Brown telescope). Lean: conditional-interface:def:weil-a4-brown-telescope

Uses: def:weil-a4-brown-telescope-shift,def:weil-a2-derived-category

Choose the A2 mapping cone of d and define $B_\infty(f,L) := \mathrm{Cone}(d)$. Write its triangle as

$$P \xrightarrow{d} P \xrightarrow{q} B_\infty \xrightarrow{\partial} P[1],$$

and define $t_m := qj_m : B_m \to B_\infty$.

Lemma 11.12.248 (The Brown telescope triangle is distinguished). Lean: conditional-interface:lemma:weil-a4-brown-telescope-triangle-distinguished

Uses: def:weil-a4-brown-telescope

The displayed telescope triangle is distinguished.

Proof. Uses: def:weil-a4-brown-telescope,prop:weil-a2-derived-category-triangulated

The object and arrows were defined by the A2 mapping-cone triangle for d, which is distinguished by the A2 triangulation. □

Lemma 11.12.249 (Telescope structure maps are compatible). Lean: conditional-interface:lemma:weil-a4-brown-telescope-structure-compatible

Uses: def:weil-a4-brown-telescope,def:weil-a4-brown-stage-recursion

For every m, $t_{m+1}b_m = t_m$.

Proof. Uses: lemma:weil-a4-brown-telescope-differential-component,lemma:weil-a4-brown-telescope-triangle-distinguished

Consecutive arrows in the telescope triangle give $qd = 0$. Composing with j_m and using the differential component equation yields $0 = qj_m - qj_{m+1}b_m = t_m - t_{m+1}b_m$. □

Definition 11.12.250 (Telescope evaluation fiber). Lean: `conditional-interface:def:weil-a4-brown-telescope-evaluation-fiber`

Uses: `def:weil-a4-brown-telescope`, `def:weil-a4-brown-stage-recursion`

Define

$$\mathcal{E}_\infty(f,L) := \{e_\infty \in H_{f,L}(B_\infty) : H_{f,L}(t_m)(e_\infty) = e_m \text{ for every } m\}.$$

Lemma 11.12.251 (The telescope evaluation fiber is inhabited). Lean: *conditional-interface:lemma:weil-a4-brown-telescope-evaluation-inhabited-open*

Uses: *def:weil-a4-brown-telescope-evaluation-fiber*

OPEN. *The set $\mathcal{E}_\infty(f,L)$ is nonempty.*

Open obligation: *OBL-A4-EX-01*

Open obligation: *OBL-A4-EX-02*

Proof. Uses: `prop:weil-a4-exceptional-hom-cohomological-open`, `prop:weil-a4-exceptional-hom-coproduct-product-open`, `lemma:weil-a4-brown-stage-evaluation-compatible-open`, `lemma:weil-a4-brown-telescope-differential-component`, `lemma:weil-a4-brown-telescope-triangle-distinguished`

Use the coproduct-to-product bijection to let $\widehat{e} \in H_{f,L}(P)$ be the element with coordinates $(e_m)_m$. The m-th coordinate of $H_{f,L}(d)(\widehat{e})$ is

$$H(j_m)(\widehat{e}) - H(j_{m+1}b_m)(\widehat{e}) = e_m - H(b_m)(e_{m+1}) = 0.$$

Injectivity of the product comparison gives $H(d)(\widehat{e}) = 0$. Applying $H_{f,L}$ to the telescope triangle gives the exact segment

$$H_{f,L}(B_\infty) \xrightarrow{H(q)} H_{f,L}(P) \xrightarrow{H(d)} H_{f,L}(P).$$

Choose a preimage e_∞ of $\widehat{e}$. Since $t_m = qj_m$, contravariant functoriality gives $H(t_m)(e_\infty) = e_m$. Product preservation and cohomological exactness carry precisely the two listed obligations. □

Definition 11.12.252 (Chosen telescope evaluation). Lean: `conditional-interface:def:weil-a4-brown-telescope-evaluation`

Uses: `def:weil-a4-brown-telescope-evaluation-fiber`

After fixing its nonemptiness witness, define

$$e_\infty(f,L) := \mathrm{Classical.choose}(\mathcal{E}_\infty(f,L)).$$

Lemma 11.12.253 (Telescope evaluation restriction). Lean: *conditional-interface:lemma:weil-a4-brown-telescope-evaluation-restriction-open*

Uses: *def:weil-a4-brown-telescope-evaluation*, *def:weil-a4-brown-stage-recursion*

OPEN. *For every m, $H_{f,L}(t_m)(e_\infty) = e_m$.*

Open obligation: *OBL-A4-EX-01*

Open obligation: *OBL-A4-EX-02*

Proof. Uses: `lemma:weil-a4-brown-telescope-evaluation-inhabited-open`, `def:weil-a4-brown-telescope-evaluation-fiber`

The chosen element belongs to $\mathcal{E}_\infty(f,L)$. Expanding that membership gives all displayed restriction equations. □

Definition 11.12.254 (Brown universal transformation). Lean: `conditional-interface:def:weil-a4-brown-universal-transformation`

Uses: `def:weil-a4-brown-telescope-evaluation`, `def:weil-a4-exceptional-hom-functor`

For every $K \in D(X,\Lambda_n)$, define

$$\Theta_{f,L,K} : \mathrm{Hom}_X(K, B_\infty(f,L)) \longrightarrow H_{f,L}(K), \qquad u \longmapsto H_{f,L}(u)(e_\infty).$$

Lemma 11.12.255 (The Brown transformation is natural). Lean: *conditional-interface:lemma:weil-a4-brown-universal-transformation-natural*

Uses: *def:weil-a4-brown-universal-transformation*

For $v : K' \to K$, $\Theta_{K'}(uv) = H_{f,L}(v)(\Theta_K(u))$.

Proof. Uses: `def:weil-a4-brown-universal-transformation`, `lemma:weil-a4-exceptional-hom-composition`

Both sides expand to $H_{f,L}(v)(H_{f,L}(u)(e_\infty))$ by the composition law of the exceptional Hom functor. □

Definition 11.12.256 (Compact-source telescope comparison). Lean: `conditional-interface: def:weil-a4-brown-compact-telescope-comparison`

Uses: `def:weil-a4-brown-telescope`, `def:weil-a4-brown-stage-recursion`

For $G \in \mathcal{G}_X$, define

$$\gamma_G : \operatorname*{colim}_m \mathrm{Hom}(G, B_m) \longrightarrow \mathrm{Hom}(G, B_\infty)$$

by sending the class of $u : G \to B_m$ to $t_m u$.

Lemma 11.12.257 (The compact-source telescope comparison is well-defined). Lean: *conditional-interface: lemma: weil-a4-brown-compact-telescope-comparison-well-defined*

Uses: *def: weil-a4-brown-compact-telescope-comparison*

If $u : G \to B_m$ and $u' : G \to B_{m'}$ define the same colimit class, then $t_m u = t_{m'} u'$.

Proof. Uses: `def:weil-a4-brown-compact-telescope-comparison`, `lemma:weil-a4-brown-telescope-structure-compatible`

Equality in the sequential colimit means that after composing both maps to some B_N, their images agree. Iterating $t_{a+1} b_a = t_a$ identifies each of $t_m u$ and $t_{m'} u'$ with the composite of its image in B_N followed by t_N. Hence they agree. □

Proposition 11.12.258 (Compact-source telescope comparison is bijective). Lean: *conditional-interface: prop: weil-a4-brown-compact-telescope-comparison-open*

Uses: *def: weil-a4-brown-compact-telescope-comparison*, *def: weil-a4-compact-generator-family*

OPEN. *The map γ_G is bijective for every $G \in \mathcal{G}_X$.*

Open obligation: *OBL-A4-EX-02*

Proof. Uses: `lemma:weil-a4-selected-generator-compact`, `lemma:weil-a4-brown-telescope-triangle-distinguished`, `def:weil-a4-brown-telescope-shift`

Apply $\mathrm{Hom}(G, -)$ to the telescope triangle. Compactness identifies each Hom into $P = \bigoplus_m B_m$ with the direct sum of the Hom groups. The resulting exact segment identifies $\mathrm{Hom}(G, B_\infty)$ with the cokernel of

$$\mathrm{id} - \mathrm{shift} : \bigoplus_m \mathrm{Hom}(G, B_m) \to \bigoplus_m \mathrm{Hom}(G, B_m),$$

because the kernel of the same map on the shifted Hom groups is zero. If $x = \mathrm{shift}(x)$, its coordinate zero is $x_0 = 0$, and the coordinate $m + 1$ equation is $x_{m+1} = (b_m)_*(x_m)$. Induction gives $x_m = 0$ for every m. The displayed cokernel is the sequential colimit by its generators-and-relations presentation, and the induced map is γ_G. The compactness input remains indexed by 'OBL-A4-EX-02'. □

Lemma 11.12.259 (Brown comparison is surjective on generators). Lean: *conditional-interface: lemma: weil-a4-brown-generator-surjective-open*

Uses: *def: weil-a4-brown-universal-transformation*, *def: weil-a4-compact-generator-family*

OPEN. *For every $G \in \mathcal{G}_X$, $\Theta_{f,L,G}$ is surjective.*

Open obligation: *OBL-A4-EX-01*

Open obligation: *OBL-A4-EX-02*

Proof. Uses: `lemma:weil-a4-brown-zero-evaluation-components-open`, `lemma:weil-a4-brown-telescope-evaluation-restriction-open`, `def:weil-a4-generator-element-index`, `def:weil-a4-brown-universal-transformation`

Given $\xi \in H_{f,L}(G)$, let $s = (G, \xi) \in S_0(f, L)$ and set $u = t_0 i_s : G \to B_\infty$. Then

$$\Theta_G(u) = H(i_s)(H(t_0)(e_\infty)) = H(i_s)(e_0) = \xi.$$

The restriction equations inherit 'OBL-A4-EX-01' and 'OBL-A4-EX-02'; no additional existence assertion is used. □

Lemma 11.12.260 (Brown comparison is injective on generators). Lean: *conditional-interface: lemma: weil-a4-brown-generator-injective-open*

Uses: *def: weil-a4-brown-universal-transformation*, *def: weil-a4-compact-generator-family*

OPEN. *For every $G \in \mathcal{G}_X$, $\Theta_{f,L,G}$ is injective.*

Open obligation: *OBL-A4-EX-01*

Open obligation: *OBL-A4-EX-02*

Proof. Uses: `prop:weil-a4-brown-compact-telescope-comparison-open,lemma:weil-a4-brown-telescope-evaluation-restriction-open,lemma:weil-a4-brown-indexed-relation-killed,def:weil-a4-brown-relation-index`

Let $u : G \to B_\infty$ satisfy $\Theta_G(u) = 0$. Surjectivity of γ_G gives a stage m and $u_m : G \to B_m$ with $u = t_m u_m$. The stage restriction equation gives

$$\theta_{D_m,G}(u_m) = H(u_m)(e_m) = H(t_m u_m)(e_\infty) = \Theta_G(u) = 0.$$

Thus $(G, u_m) \in R(D_m)$, and the relation-killing lemma gives $b_m u_m = 0$. Hence the class of u_m in the sequential colimit is zero. Injectivity of γ_G gives $u = 0$. The only inherited premises are the two listed obligations. □

Proposition 11.12.261 (Brown comparison on generators). Lean: `conditional-interface:prop:weil-a4-brown-generator-bijection-open`

Uses: `def:weil-a4-brown-universal-transformation,def:weil-a4-compact-generator-family`

OPEN. *For every $G \in \mathcal{G}_X$, $\Theta_{f,L,G}$ is bijective.*

Open obligation: `OBL-A4-EX-01`

Open obligation: `OBL-A4-EX-02`

Proof. Uses: `lemma:weil-a4-brown-generator-surjective-open,lemma:weil-a4-brown-generator-injective-open`

The two preceding lemmas give surjectivity and injectivity for the same map. Their conjunction is bijectivity. Thus the former independent generator-comparison obligation has been reduced completely to ‘OBL-A4-EX-01‘ and ‘OBL-A4-EX-02‘. □

Definition 11.12.262 (Brown comparison locus). Lean: `conditional-interface:def:weil-a4-brown-comparison-locus`

Uses: `def:weil-a4-brown-universal-transformation`

Let $\mathcal{L}_{f,L}$ be the full subcategory of objects K such that $\Theta_{f,L,K[r]}$ is bijective for every $r \in \mathbf{Z}$.

Lemma 11.12.263 (The Brown locus contains zero and is shift-stable). Lean: `conditional-interface:lemma:weil-a4-brown-locus-zero-shift-open`

Uses: `def:weil-a4-brown-comparison-locus`

OPEN. *The zero object belongs to $\mathcal{L}_{f,L}$, and $K \in \mathcal{L}_{f,L}$ implies $K[a] \in \mathcal{L}_{f,L}$ for every integer a.*

Open obligation: `OBL-A4-EX-01`

Open obligation: `OBL-A4-EX-02`

Proof. Uses: `def:weil-a4-brown-comparison-locus,def:weil-a4-brown-universal-transformation`

Both Hom functors take the zero object to the one-element zero group, so its comparison is bijective. For the shift assertion, the family of maps tested for $K[a]$ is $\{\Theta_{K[a+r]} : r \in \mathbf{Z}\}$, which is the same family tested for K after reindexing by $r \mapsto a + r$. The declaration remains OPEN only because Θ uses the conditional telescope evaluation attached to the two listed obligations. □

Lemma 11.12.264 (The Brown locus is closed under cones). Lean: `conditional-interface:lemma:weil-a4-brown-locus-cones-open`

Uses: `def:weil-a4-brown-comparison-locus`

OPEN. *In a distinguished triangle $K' \to K \to K'' \to K'[1]$, if two objects belong to $\mathcal{L}_{f,L}$, then the third belongs to $\mathcal{L}_{f,L}$.*

Open obligation: `OBL-A4-EX-01`

Open obligation: `OBL-A4-EX-02`

Proof. Uses: `prop:weil-a4-exceptional-hom-cohomological-open,lemma:weil-a4-brown-universal-transformation-natural,def:weil-a4-brown-comparison-locus,prop:weil-a2-derived-category-triangulated`

Apply $\mathrm{Hom}(-, B_\infty)$ and $H_{f,L}$ to every shift of the triangle. Naturality of Θ makes the vertical maps into a map of long exact sequences. Suppose, for example, that the comparisons for K' and K are bijective. In the five-term segment centered at the comparison for $K''[r]$, the other four vertical maps are comparisons for shifts of K' and K, hence are bijective. For injectivity of the middle map, chase an element first to the right, lift its zero image across the preceding bijection, and use exactness to subtract a preimage; for surjectivity, lift the image to the adjacent term, use the next bijection to correct its boundary, and apply exactness once more. These two chases prove the middle map bijective. Rotating the triangle treats the other choices of two objects. Cohomologicality carries ‘OBL-A4-EX-01‘, and construction of Θ carries ‘OBL-A4-EX-02‘. □

Lemma 11.12.265 (The Brown locus is closed under direct sums). *Lean:* `conditional-interface:lemma:weil-a4-brown-locus-coproducts-open`

Uses: `def:weil-a4-brown-comparison-locus`

OPEN. *If $K_j \in \mathcal{L}_{f,L}$ for all $j \in J$, then $\bigoplus_j K_j \in \mathcal{L}_{f,L}$.*

Open obligation: `OBL-A4-EX-01`

Open obligation: `OBL-A4-EX-02`

Proof. Uses: `prop:weil-a4-exceptional-hom-coproduct-product-open,lemma:weil-a4-brown-universal-transformation-natural,def:weil-a4-brown-comparison-locus`

For every shift r, the coproduct universal property gives a bijection

$$\mathrm{Hom}\left(\bigoplus_j K_j[r], B_\infty\right) \simeq \prod_j \mathrm{Hom}(K_j[r], B_\infty).$$

The exceptional-Hom coproduct comparison gives $H_{f,L}(\bigoplus_j K_j[r]) \simeq \prod_j H_{f,L}(K_j[r])$. Naturality identifies $\Theta_{\oplus K_j[r]}$ under these two bijections with $\prod_j \Theta_{K_j[r]}$. A product of bijections is bijective, so every shift of the coproduct lies in the locus. The two listed obligations are inherited from construction and product preservation. □

Proposition 11.12.266 (The Brown comparison locus is localizing). *Lean:* `conditional-interface:prop:weil-a4-brown-locus-localizing-open`

Uses: `def:weil-a4-brown-comparison-locus`

OPEN. *The subcategory $\mathcal{L}_{f,L}$ contains zero, is closed under shifts and cones, and is closed under set-indexed coproducts.*

Open obligation: `OBL-A4-EX-01`

Open obligation: `OBL-A4-EX-02`

Proof. Uses: `lemma:weil-a4-brown-locus-zero-shift-open,lemma:weil-a4-brown-locus-cones-open,lemma:weil-a4-brown-locus-coproducts-open`

The three preceding lemmas establish the four defining closure properties of a localizing subcategory. They leave only the two inherited input obligations. □

Proposition 11.12.267 (Brown representability on constructible objects). *Lean:* `conditional-interface:prop:weil-a4-brown-representability-open`

Uses: `def:weil-a4-brown-universal-transformation,def:weil-a4-brown-comparison-locus`

OPEN. *For every $K \in D^b_c(X, \Lambda_n)$, $\Theta_{f,L,K}$ is a bijection.*

Open obligation: `OBL-A4-EX-01`

Open obligation: `OBL-A4-EX-02`

Proof. Uses: `prop:weil-a4-generator-small-compact-open,prop:weil-a4-brown-generator-bijection-open,prop:weil-a4-brown-locus-localizing-open`

The generator bijection puts every member of $\mathcal{G}_X$ in $\mathcal{L}_{f,L}$. Since the locus is localizing, it contains the localizing closure of $\mathcal{G}_X$. The generator proposition identifies that closure with the ambient derived category, hence it contains every bounded constructible object. The formal solution-object, relation-killing, telescope, generator-bijection, and localizing-locus arguments introduce no obligation beyond 'OBL-A4-EX-01' and 'OBL-A4-EX-02'. □

Proposition 11.12.268 (Constructibility and amplitude of the Brown object). *Lean:* `conditional-interface:prop:weil-a4-brown-object-constructible-open`

Uses: `def:weil-a4-brown-telescope, def:weil-a4-exceptional-index`

OPEN. *If L is bounded constructible, then $B_\infty(f, L)$ is bounded constructible. More precisely, for an A4-fixed cohomological dimension bound c_f,*

$$L \in D^{[a,b]} \quad \Longrightarrow \quad B_\infty(f, L) \in D^{[a-c_f,\, b]}.$$

Open obligation: `OBL-A4-EX-06`

Proof. Uses: `def:weil-a4-brown-telescope`

The telescope construction alone gives no finite amplitude. The missing proof must calculate stalks after a finite constructible stratification, apply the finite cohomological-dimension bound for compact support, and show the bound is uniform across the Brown stages. It must then show each cohomology sheaf is finite and locally constant on the chosen strata. This is exactly 'OBL-A4-EX-06'. □

Definition 11.12.269 (Finite-level exceptional inverse image on objects)**.** Lean: `conditional-interface:def:weil-a4-finite-exceptional-pullback-object`

Uses: `def:weil-a4-brown-telescope`, `def:weil-a4-exceptional-index`

For $L \in D_c^b(Y, \Lambda_n)$, define

$$Rf_n^! L := B_\infty(f, L).$$

Its membership in $D_c^b(X, \Lambda_n)$ is conditional precisely on 'OBL-A4-EX-06'.

Definition 11.12.270 (Finite-level exceptional counit)**.** Lean: `conditional-interface:def:weil-a4-finite-exceptional-counit`

Uses: `def:weil-a4-finite-exceptional-pullback-object`, `def:weil-a4-brown-universal-transformation`

Define

$$\epsilon^!_{f,n,L} : Rf_! Rf_n^! L \longrightarrow L$$

to be the element $\Theta_{f,L,Rf_n^! L}(\mathrm{id}_{Rf_n^! L})$.

Definition 11.12.271 (Backward representing map)**.** Lean: `conditional-interface:def:weil-a4-finite-exceptional-backward-map`

Uses: `def:weil-a4-finite-exceptional-counit`

For $b : K \to Rf_n^! L$, define

$$\Psi_{f,n,K,L}(b) := \epsilon^!_{f,n,L} \circ Rf_!(b).$$

Definition 11.12.272 (Forward representing map)**.** Lean: `conditional-interface:def:weil-a4-finite-exceptional-forward-map`

Uses: `def:weil-a4-brown-universal-transformation`, `def:weil-a4-finite-exceptional-pullback-object`

When the Brown representability proposition is discharged, define

$$\Phi_{f,n,K,L}(a) := \Theta^{-1}_{f,L,K}(a)$$

for $a : Rf_! K \to L$.

Proposition 11.12.273 (First representing inverse equation)**.** Lean: *`conditional-interface:prop:weil-a4-finite-representing-left-inverse-open`*

Uses: *`def:weil-a4-finite-exceptional-forward-map`*, *`def:weil-a4-finite-exceptional-backward-map`*

OPEN. *For every* $a : Rf_! K \to L$, $\Psi_{f,n,K,L}(\Phi_{f,n,K,L}(a)) = a$.

Open obligation: *`OBL-A4-EX-01`*

Open obligation: *`OBL-A4-EX-02`*

Proof. Uses: `def:weil-a4-finite-exceptional-forward-map`, `def:weil-a4-finite-exceptional-backward-map`, `def:weil-a4-finite-exceptional-counit`

By the definition of the counit, the backward map is the Brown transformation $\Theta_{f,L,K}$. The left side is therefore $\Theta_{f,L,K}(\Theta^{-1}_{f,L,K}(a))$. It equals a once the Brown comparison, now conditional only on 'OBL-A4-EX-01' and 'OBL-A4-EX-02', is bijective. □

Proposition 11.12.274 (Second representing inverse equation)**.** Lean: *`conditional-interface:prop:weil-a4-finite-representing-right-inverse-open`*

Uses: *`def:weil-a4-finite-exceptional-forward-map`*, *`def:weil-a4-finite-exceptional-backward-map`*

OPEN. *For every* $b : K \to Rf_n^! L$, $\Phi_{f,n,K,L}(\Psi_{f,n,K,L}(b)) = b$.

Open obligation: *`OBL-A4-EX-01`*

Open obligation: *`OBL-A4-EX-02`*

Proof. Uses: `def:weil-a4-finite-exceptional-forward-map`, `def:weil-a4-finite-exceptional-backward-map`, `def:weil-a4-finite-exceptional-counit`

As in the previous proof, $\Psi_{f,n,K,L}(b) = \Theta_{f,L,K}(b)$. Thus the left side is $\Theta^{-1}_{f,L,K}(\Theta_{f,L,K}(b))$, equal to b after discharge of 'OBL-A4-EX-01' and 'OBL-A4-EX-02'. □

Definition 11.12.275 (Finite-level exceptional pullback on arrows)**.** Lean: `conditional-interface:def:weil-a4-finite-exceptional-pullback-arrow`

Uses: `def:weil-a4-finite-exceptional-forward-map`, `def:weil-a4-finite-exceptional-counit`

For $v : L \to L'$, define

$$Rf_n^!(v) := \Phi_{f,n,Rf_n^! L,L'}(v \circ \epsilon^!_{f,n,L}).$$

Proposition 11.12.276 (Exceptional pullback preserves identities). *Lean:* `conditional-interface:prop:weil-a4-finite-exceptional-pullback-identity-open`

Uses: `def:weil-a4-finite-exceptional-pullback-arrow`, `def:weil-a4-finite-exceptional-counit`

OPEN. $Rf_n^!(\mathrm{id}_L) = \mathrm{id}_{Rf_n^! L}$.

Open obligation: `OBL-A4-EX-01`

Open obligation: `OBL-A4-EX-02`

Proof. Uses: `prop:weil-a4-finite-representing-right-inverse-open`, `def:weil-a4-finite-exceptional-pullback-arrow`, `def:weil-a4-finite-exceptional-backward-map`

Expanding the arrow definition gives $\Phi(\epsilon^!_{f,n,L})$. The counit is $\Psi(\mathrm{id}_{Rf_n^! L})$, so the second representing inverse equation gives the claimed identity. The result inherits its OPEN status. □

Proposition 11.12.277 (Exceptional counit naturality). *Lean:* `conditional-interface:prop:weil-a4-finite-exceptional-counit-naturality-open`

Uses: `def:weil-a4-finite-exceptional-counit`, `def:weil-a4-finite-exceptional-pullback-arrow`

OPEN. *For* $v : L \to L'$,

$$v \circ \epsilon^!_{f,n,L} = \epsilon^!_{f,n,L'} \circ Rf_! Rf_n^!(v).$$

Open obligation: `OBL-A4-EX-01`

Open obligation: `OBL-A4-EX-02`

Proof. Uses: `prop:weil-a4-finite-representing-left-inverse-open`, `def:weil-a4-finite-exceptional-pullback-arrow`, `def:weil-a4-finite-exceptional-backward-map`

The right side is the backward representing map applied to $Rf_n^!(v)$. Substitute its definition as the forward image of $v \circ \epsilon^!_{f,n,L}$; the first representing inverse equation reduces the result to the left side. □

Proposition 11.12.278 (Exceptional pullback preserves composition). *Lean:* `conditional-interface:prop:weil-a4-finite-exceptional-pullback-composition-open`

Uses: `def:weil-a4-finite-exceptional-pullback-arrow`, `def:weil-a4-finite-exceptional-counit`

OPEN. *For* $L \xrightarrow{v} L' \xrightarrow{w} L''$, $Rf_n^!(wv) = Rf_n^!(w) Rf_n^!(v)$.

Open obligation: `OBL-A4-EX-01`

Open obligation: `OBL-A4-EX-02`

Proof. Uses: `prop:weil-a4-finite-exceptional-counit-naturality-open`, `prop:weil-a4-finite-representing-right-inverse-open`, `def:weil-a4-finite-exceptional-pullback-arrow`

Apply the backward representing map to the proposed equality. The left side becomes $wv\epsilon^!_L$. Counit naturality first for v and then for w turns the right side into the same composite. The backward map is injective by the second inverse equation, proving equality conditionally on the same OPEN obligation. □

Definition 11.12.279 (Finite-level exceptional unit). Lean: `conditional-interface:def:weil-a4-finite-exceptional-unit`

Uses: `def:weil-a4-finite-exceptional-forward-map`

For $K \in D^b_c(X, \Lambda_n)$, define

$$\eta^!_{f,n,K} : K \longrightarrow Rf_n^! Rf_! K, \qquad \eta^!_{f,n,K} := \Phi_{f,n,K,Rf_! K}(\mathrm{id}_{Rf_! K}).$$

Proposition 11.12.280 (First exceptional triangle identity). *Lean:* `conditional-interface:prop:weil-a4-finite-exceptional-first-triangle-open`

Uses: `def:weil-a4-finite-exceptional-unit`, `def:weil-a4-finite-exceptional-counit`

OPEN. *The composite*

$$Rf_! K \xrightarrow{Rf_!(\eta^!_{f,n,K})} Rf_! Rf_n^! Rf_! K \xrightarrow{\epsilon^!_{f,n,Rf_! K}} Rf_! K$$

is the identity.

Open obligation: `OBL-A4-EX-01`

Open obligation: `OBL-A4-EX-02`

Proof. Uses: `def:weil-a4-finite-exceptional-unit`, `def:weil-a4-finite-exceptional-backward-map`, `prop:weil-a4-finite-representing-left-inverse-open`

The displayed composite is $\Psi(\Phi(\mathrm{id}_{Rf_!K}))$ after expanding the unit and the backward map. The first representing inverse equation gives the identity. □

Proposition 11.12.281 (Second exceptional triangle identity). Lean: `conditional-interface:prop:weil-a4-finite-exceptional-second-triangle-open`

Uses: `def:weil-a4-finite-exceptional-unit`, `def:weil-a4-finite-exceptional-counit`, `def:weil-a4-finite-exceptional-pullback-arrow`

OPEN. *The composite*

$$Rf_n^! L \xrightarrow{\eta^!_{f,n,Rf_n^!L}} Rf_n^! Rf_! Rf_n^! L \xrightarrow{Rf_n^!(\epsilon^!_{f,n,L})} Rf_n^! L$$

is the identity.

Open obligation: `OBL-A4-EX-01`

Open obligation: `OBL-A4-EX-02`

Proof. Uses: `prop:weil-a4-finite-representing-right-inverse-open`, `prop:weil-a4-finite-exceptional-counit-naturality-open`, `def:weil-a4-finite-exceptional-unit`, `def:weil-a4-finite-exceptional-pullback-arrow`

Apply the backward representing map. By counit naturality and the first triangle calculation, the image is the counit $\epsilon^!_{f,n,L}$, which is also the backward image of the identity. The second representing inverse equation makes the backward map injective, so the original composite is the identity. □

Definition 11.12.282 (Open-restriction comparison for finite exceptional pullback). Lean: `conditional-interface:def:weil-a4-finite-exceptional-open-restriction`

Uses: `def:weil-a4-finite-exceptional-pullback-object`, `def:weil-a4-finite-exceptional-forward-map`

For an open immersion $j : U \hookrightarrow X$ and $f : X \to Y$, define

$$\beta_{j,f,L} : j^* Rf_n^! L \longrightarrow R(fj)_n^! L$$

as the forward representing image of the composite obtained from the A1 identity $R(fj)_! = Rf_! j_!$ and the finite counit of f.

Definition 11.12.283 (Closed-restriction comparison for finite exceptional pullback). Lean: `conditional-interface:def:weil-a4-finite-exceptional-closed-restriction`

Uses: `def:weil-a4-finite-exceptional-pullback-object`, `def:weil-a4-finite-exceptional-forward-map`

For a closed immersion $i : Z \hookrightarrow X$, define

$$\beta_{i,f,L} : Ri_n^! Rf_n^! L \longrightarrow R(fi)_n^! L$$

as the forward representing image of the two-step counit $R(fi)_! Ri_n^! Rf_n^! L \to L$, using the A1 comparison $R(fi)_! = Rf_! Ri_*$.

Proposition 11.12.284 (Finite exceptional pullback localization formulas). Lean: `conditional-interface:prop:weil-a4-finite-exceptional-localization-open`

Uses: `def:weil-a4-finite-exceptional-open-restriction`, `def:weil-a4-finite-exceptional-closed-restriction`

OPEN. *The maps $\beta_{j,f,L}$ and $\beta_{i,f,L}$ are isomorphisms, natural in L, compatible with compositions of open and closed immersions, and compatible with finite coefficient reduction.*

Open obligation: `OBL-A4-EX-07`

Proof. Uses: `def:weil-a4-finite-exceptional-open-restriction`, `def:weil-a4-finite-exceptional-closed-restriction`, `prop:weil-a4-finite-representing-left-inverse-open`, `prop:weil-a4-finite-representing-right-inverse-open`

For the open case, apply successively the four Hom identifications

$$\mathrm{Hom}_U(K, j^* Rf_n^! L) \simeq \mathrm{Hom}_X(j_! K, Rf_n^! L) \simeq \mathrm{Hom}_Y(Rf_! j_! K, L) \simeq \mathrm{Hom}_Y(R(fj)_! K, L) \simeq \mathrm{Hom}_U(K, R(fj)_n^! L).$$

For the closed case, replace $j_! \dashv j^*$ by $Ri_* \dashv Ri^!$ and use $R(fi)_! = Rf_! Ri_*$. Yoneda gives inverse maps. Naturality and composition follow by applying the same Hom identifications to both routes. The unresolved A1 comparison and reduction rectangles are exactly ‘OBL-A4-EX-07‘. □

Lemma 11.12.285 (Geometric Frobenius on a Tate twist). Lean: `conditional-interface:lemma:weil-a4-tate-frobenius-scalar`

Uses: `def:weil-a4-tate-twist`, `def:weil-a4-geometric-frobenius-convention`

Geometric Frobenius acts on $A(m)$ by q^{-m}, and on $A(-m)$ by q^{m}.

Proof. Uses: `def:weil-a4-tate-twist`

The assertion for $m = 1$ is the normalization in the Definition. Tensoring endomorphisms multiplies their scalars, giving q^{-m} for $m \geq 0$. The evaluation $A(1) \otimes A(-1) \to A$ is Frobenius equivariant, so the scalar on $A(-1)$ is the inverse q. Tensor powers give the negative case. □

11.12.4 Underived and derived internal Hom

Definition 11.12.286 (Sectionwise Hom presheaf). Lean: `conditional-interface:def:weil-a4-sectionwise-hom-presheaf`

Uses: `def:weil-a4-coefficient-index`

Let M, N be sheaves of A-modules on X_{proet}. Define the presheaf

$$\underline{\mathrm{Hom}}_A^{\mathrm{pre}}(M,N)(U) := \mathrm{Hom}_{A|_U}(M|_U, N|_U).$$

For $v : V \to U$, restriction sends h to $h|_V$.

Lemma 11.12.287 (Identity law for restrictions of sectionwise Hom). Lean: `conditional-interface:lemma:weil-a4-sectionwise-hom-restriction-identity`

Uses: `def:weil-a4-sectionwise-hom-presheaf`

Restriction along id_U is the identity.

Proof. Uses: `def:weil-a4-sectionwise-hom-presheaf`

Both the source and target restriction functors along id_U are identity functors, so $h|_U = h$. □

Lemma 11.12.288 (Composition law for restrictions of sectionwise Hom). Lean: `conditional-interface:lemma:weil-a4-sectionwise-hom-restriction-composition`

Uses: `def:weil-a4-sectionwise-hom-presheaf`

For $W \xrightarrow{w} V \xrightarrow{v} U$, restriction along vw equals restriction along v followed by restriction along w.

Proof. Uses: `def:weil-a4-sectionwise-hom-presheaf`

For a morphism $h : M|_U \to N|_U$, both maps are the morphism obtained by applying the restriction functor from U to W. Their components on every object over W are therefore equal. □

Definition 11.12.289 (Sheaf internal Hom). Lean: `conditional-interface:def:weil-a4-sheaf-internal-hom`

Uses: `def:weil-a4-sectionwise-hom-presheaf`

Define

$$\underline{\mathrm{Hom}}_A(M,N) := a\big(\underline{\mathrm{Hom}}_A^{\mathrm{pre}}(M,N)\big),$$

where a is module-sheafification. The unit of sheafification is denoted

$$\iota_{M,N} : \underline{\mathrm{Hom}}_A^{\mathrm{pre}}(M,N) \to \underline{\mathrm{Hom}}_A(M,N).$$

This constructs an A-module sheaf because module-sheafification is the left adjoint to the inclusion of module sheaves into module presheaves; no existence datum is fixed as an input.

Definition 11.12.290 (Internal evaluation morphism). Lean: `conditional-interface:def:weil-a4-internal-evaluation`

Uses: `def:weil-a4-sheaf-internal-hom`

Define

$$\mathrm{ev}_{M,N} : \underline{\mathrm{Hom}}_A(M,N) \otimes_A M \longrightarrow N$$

as the sheafification of the sectionwise map $h \otimes m \mapsto h(m)$.

Lemma 11.12.291 (Sectionwise evaluation respects restrictions). Lean: `conditional-interface:lemma:weil-a4-evaluation-restriction`

Uses: `def:weil-a4-sectionwise-hom-presheaf`, `def:weil-a4-internal-evaluation`

For $v : V \to U$, h over U, and $m \in M(U)$, one has

$$(h(m))|_V = (h|_V)(m|_V).$$

Proof. Uses: def:weil-a4-sectionwise-hom-presheaf,def:weil-a4-internal-evaluation
The restriction $h|_V$ is the natural transformation obtained by restricting h. Its naturality square at $V \to U$ is exactly the displayed equality. □

Definition 11.12.292 (Currying map). Lean: conditional-interface:def:weil-a4-currying-map

Uses: def:weil-a4-sheaf-internal-hom,def:weil-a4-internal-evaluation
For $u : P \otimes_A M \to N$, define

$$\mathrm{curry}(u) : P \to \underline{\mathrm{Hom}}_A(M, N)$$

by sending a local section p to the local morphism $m \mapsto u(p \otimes m)$, followed by $\iota_{M,N}$.

Definition 11.12.293 (Uncurrying map). Lean: conditional-interface:def:weil-a4-uncurrying-map

Uses: def:weil-a4-internal-evaluation
For $v : P \to \underline{\mathrm{Hom}}_A(M, N)$, define

$$\mathrm{uncurry}(v) := \mathrm{ev}_{M,N} \circ (v \otimes \mathrm{id}_M).$$

Lemma 11.12.294 (Uncurrying after currying). Lean: conditional-interface:lemma:weil-a4-uncurry-curry

Uses: def:weil-a4-currying-map, def:weil-a4-uncurrying-map
For every $u : P \otimes_A M \to N$, $\mathrm{uncurry}(\mathrm{curry}(u)) = u$.

Proof. Uses: def:weil-a4-currying-map,def:weil-a4-uncurrying-map,def:weil-a4-internal-evaluation
Evaluate both maps on local pure tensors $p \otimes m$. The left side is the value at m of the morphism $m' \mapsto u(p \otimes m')$, hence is $u(p \otimes m)$. Pure tensors generate the sheaf tensor product, so the two sheaf morphisms agree. □

Lemma 11.12.295 (Currying after uncurrying). Lean: conditional-interface:lemma:weil-a4-curry-uncurry

Uses: def:weil-a4-currying-map, def:weil-a4-uncurrying-map
For every $v : P \to \underline{\mathrm{Hom}}_A(M, N)$, $\mathrm{curry}(\mathrm{uncurry}(v)) = v$.

Proof. Uses: def:weil-a4-currying-map,def:weil-a4-uncurrying-map,def:weil-a4-internal-evaluation
After passage to a covering, a local section $v(p)$ is represented by a local morphism $h : M \to N$. Currying the evaluation sends p to the morphism $m \mapsto h(m)$, which represents the same section $v(p)$. Equality is local for sheaves, so the two maps coincide globally. □

Proposition 11.12.296 (Underived tensor–Hom adjunction). Lean: conditional-interface:prop:weil-a4-underived-tensor-hom-adjunction

Uses: def:weil-a4-currying-map, def:weil-a4-uncurrying-map
The displayed currying and uncurrying maps give mutually inverse bijections

$$\mathrm{Hom}(P \otimes_A M, N) \simeq \mathrm{Hom}(P, \underline{\mathrm{Hom}}_A(M, N)),$$

natural in P, M, N.

Proof. Uses: lemma:weil-a4-uncurry-curry,lemma:weil-a4-curry-uncurry,def:weil-a4-currying-map,def:weil-a4-uncurrying-map
The two inverse equations are the preceding two Lemmas. Naturality in each variable follows by substituting a precomposition or postcomposition into the two formulas: both resulting maps evaluate a local pair p, m by the same ordered composite. □

Definition 11.12.297 (Internal Hom complex). Lean: conditional-interface:def:weil-a4-internal-hom-complex

Uses: def:weil-a4-sheaf-internal-hom
For cochain complexes $P^\bullet, I^\bullet$, define

$$\underline{\mathrm{Hom}}^r_A(P, I) = \prod_{a \in \mathbf{Z}} \underline{\mathrm{Hom}}_A(P^a, I^{a+r}),$$

with differential

$$(dh)^a = d_I^{a+r} h^a - (-1)^r h^{a+1} d_P^a.$$

Lemma 11.12.298 (The internal Hom differential squares to zero). *Lean:* `conditional-interface:lemma:weil-a4-internal-hom-differential-square`

Uses: `def:weil-a4-internal-hom-complex`
For every homogeneous h of degree r, one has $d(dh) = 0$.

Proof. Uses: `def:weil-a4-internal-hom-complex`
The component at a is

$$d_I^2 h^a - (-1)^r d_I h^{a+1} d_P^a - (-1)^{r+1} d_I h^{a+1} d_P^a + (-1)^{2r+1} h^{a+2} d_P^2.$$

The first and last terms vanish because $d_I^2 = d_P^2 = 0$; the middle two terms have opposite coefficients and cancel. □

Definition 11.12.299 (K-injective replacement datum). *Lean:* `conditional-interface:def:weil-a4-kinjective-replacement-datum`

Uses: `def:weil-a2-complex-category,def:weil-a2-derived-category`
A K-injective replacement datum for a complex L is a complex $I(L)$, a quasi-isomorphism $\iota_L : L \to I(L)$, and, for every acyclic complex C, the assertion that $\mathrm{Hom}^\bullet(C, I(L))$ is acyclic. A functorial datum also specifies $I(v)$ for each cochain map v, with identity and composition equalities and $I(v)\iota_L = \iota_{L'} v$.

Definition 11.12.300 (Selected 079P endofunctor). *Lean:* `conditional-interface:def:weil-a4-kinjective-selected-functor`

Uses: `def:weil-fresh-kinjective-079p-construction-v1`
For each A2 module-sheaf category, let

$$I : \mathrm{Ch}(A) \longrightarrow \mathrm{Ch}(A)$$

be the terminal transfinite-tower functor selected in the 079P construction.

Definition 11.12.301 (Selected 079P unit). *Lean:* `conditional-interface:def:weil-a4-kinjective-selected-unit`

Uses: `def:weil-a4-kinjective-selected-functor,def:weil-fresh-kinjective-079p-construction-v1`
Let

$$\iota : \mathrm{id} \Longrightarrow I$$

be the composite tower transformation supplied by the selected 079P construction.

Proposition 11.12.302 (The selected unit is a quasi-isomorphism). *Lean:* `conditional-interface:prop:weil-a4-kinjective-selected-unit-qis`

Uses: `def:weil-a4-kinjective-selected-unit`
For every complex L, the component $\iota_L : L \to I(L)$ is a quasi-isomorphism.

Proof. Uses: `prop:weil-fresh-ringed-topos-module-grothendieck-v1,prop:weil-fresh-kinjective-existence-079p-v1,def:weil-a4-kinjective-selected-unit`
The A2 module-sheaf category is Grothendieck, so the component statement is the quasi-isomorphism clause of the selected 079P tower. □

Proposition 11.12.303 (The selected values are K-injective). *Lean:* `conditional-interface:prop:weil-a4-kinjective-selected-value`

Uses: `def:weil-a4-kinjective-selected-functor`
For every complex L, the complex $I(L)$ is K-injective.

Proof. Uses: `prop:weil-fresh-ringed-topos-module-grothendieck-v1,prop:weil-fresh-kinjective-existence-079p-v1,def:weil-a4-kinjective-selected-functor`
The A2 Grothendieck certificate allows the 079P small-test criterion, whose conclusion for the selected terminal tower is precisely this assertion. □

Proposition 11.12.304 (Identity field of the selected endofunctor). *Lean:* `conditional-interface:prop:weil-a4-kinjective-selected-identity`

Uses: `def:weil-a4-kinjective-selected-functor`
For every complex L, one has $I(1_L) = 1_{I(L)}$.

Proof. Uses: prop:weil-fresh-kinjective-existence-079p-v1,def:weil-a4-kinjective-selected-functor
This is the identity law of the ordinary functor selected by 079P. □

Proposition 11.12.305 (Composition field of the selected endofunctor). *Lean:* conditional-interface:prop:weil-a4-kinjective-selected-composition

Uses: def:weil-a4-kinjective-selected-functor
For composable cochain maps $L \xrightarrow{v} L' \xrightarrow{w} L''$, one has

$$I(wv) = I(w)I(v).$$

Proof. Uses: prop:weil-fresh-kinjective-existence-079p-v1,def:weil-a4-kinjective-selected-functor
This is the composition law of the ordinary functor selected by 079P. □

Proposition 11.12.306 (Naturality field of the selected unit). *Lean:* conditional-interface:prop:weil-a4-kinjective-selected-naturality

Uses: def:weil-a4-kinjective-selected-functor, def:weil-a4-kinjective-selected-unit
For every cochain map $v : L \to L'$, one has

$$I(v)\iota_L = \iota_{L'}v.$$

Proof. Uses: prop:weil-fresh-kinjective-existence-079p-v1,def:weil-a4-kinjective-selected-functor,def:weil-a4-kinjective-selected-unit
This is the naturality component of the transformation selected by 079P. □

Proposition 11.12.307 (Typed A4 K-injective replacement record). *Lean:* conditional-interface:prop:weil-a4-kinjective-replacement-existence-open

Uses: def:weil-a4-kinjective-replacement-datum
The preceding certified field tuple inhabits the A4 K-injective replacement datum type.

Constructed by: def:weil-fresh-kinjective-079p-construction-v1

Proof. Uses: def:weil-a4-kinjective-selected-functor,def:weil-a4-kinjective-selected-unit,prop:weil-a4-kinjective-selected-unit-qis,prop:weil-a4-kinjective-selected-value,prop:weil-a4-kinjective-selected-identity,prop:weil-a4-kinjective-selected-composition,prop:weil-a4-kinjective-selected-naturality,def:weil-a4-kinjective-replacement-datum
Use I, ι, and their certified fields in the corresponding slots of the replacement-datum record. This is a blueprint-level typed-record assembly. It does not assert that the record has already been exported as a Lean term through the still-open pinned universe/localization adapter. □

Lemma 11.12.308 (K-injective targets compute localized morphisms). *Lean:* blueprint-target:lemma:weil-a4-kinjective-localization-hom

Uses: def:weil-general-unbounded-derived-localization-v1, def:weil-general-unbounded-k-models-v1
For every complex C and K-injective complex J, localization induces a natural bijection

$$\mathrm{Hom}_K(C, J) \xrightarrow{\sim} \mathrm{Hom}_D(C, J).$$

Proof. Uses: def:weil-general-unbounded-derived-localization-v1,def:weil-general-unbounded-k-models-v1
Represent a localized arrow by a roof $C \xleftarrow{q} C' \xrightarrow{a} J$, where q is a quasi-isomorphism. Its cone is acyclic, so the long exact Hom sequence into the shifts of J says that $q^* : \mathrm{Hom}_K(C, J) \to \mathrm{Hom}_K(C', J)$ is a bijection. There is consequently a unique $b : C \to J$ in K with $bq = a$. Common refinement of roofs shows that this assignment is independent of the roof, and the identity roof shows that it is inverse to localization. The construction commutes with precomposition and with maps of K-injective targets. □

Definition 11.12.309 (Selected K-injective replacement). Lean: conditional-interface:def:weil-a4-fixed-kinjective-replacement

Uses: def:weil-a4-kinjective-replacement-datum,def:weil-fresh-kinjective-079p-construction-v1
For every A2 module-sheaf complex L, define $I(L)$, $\iota_L : L \to I(L)$, and $I(v)$ to be the corresponding values of the selected 079P transfinite functor and its natural transformation. Thus the replacement used below is the output of a prior construction, rather than an assumed inhabitant of the replacement-datum predicate.

Definition 11.12.310 (Derived internal Hom candidate). Lean: `conditional-interface:def:weil-a4-derived-internal-hom`

Uses: `def:weil-a4-internal-hom-complex`, `def:weil-a4-fixed-kinjective-replacement`, `def:weil-fresh-kflat-adapted-class-v1`

For derived objects K, L, select a termwise-flat K-flat arrow $p_K : P_K \to K$ from the 06YS construction and put

$$R\underline{\mathrm{Hom}}_X(K, L) := \underline{\mathrm{Hom}}_A^\bullet(P_K, I(L))$$

in the ambient derived category. Source and target comparison maps are the ones induced by common K-flat roofs and by the functorial 079P replacement; their independence is proved below rather than included as choice data.

Constructed by: `thm:weil-general-unbounded-kflat-objectwise-discharge-v1`

Definition 11.12.311 (Derived internal Hom comparison for source resolutions). Lean: `conditional-interface:def:weil-a4-derived-hom-source-comparison`

Uses: `def:weil-a4-derived-internal-hom`

For a quasi-isomorphism $u : P' \to P$, define

$$u^* : \underline{\mathrm{Hom}}_A^\bullet(P, I(L)) \longrightarrow \underline{\mathrm{Hom}}_A^\bullet(P', I(L))$$

by $h \mapsto h \circ u$.

Lemma 11.12.312 (The source-resolution comparison is a cochain map). Lean: `conditional-interface:lemma:weil-a4-derived-hom-source-comparison-chain`

Uses: `def:weil-a4-derived-hom-source-comparison`, `def:weil-a4-internal-hom-complex`

The map u^* commutes with the internal Hom differential.

Proof. Uses: `def:weil-a4-derived-hom-source-comparison`, `def:weil-a4-internal-hom-complex`

For homogeneous h of degree r, expand both sides. Since u has degree zero and $d_P u = u d_{P'}$,

$$d(hu) = d_I hu - (-1)^r h u d_{P'} = (d_I h - (-1)^r h d_P)u = (dh)u.$$

□

Lemma 11.12.313 (A K-flat source and a K-injective target give a K-injective internal Hom). Lean: `blueprint-target:lemma:weil-a4-kflat-kinjective-internal-hom`

Uses: `def:weil-a4-internal-hom-complex`, `def:weil-general-unbounded-k-models-v1`

If P is K-flat and I is K-injective, then $\underline{\mathrm{Hom}}_A^\bullet(P, I)$ is K-injective.

Proof. Uses: `prop:weil-a4-underived-tensor-hom-adjunction`, `def:weil-general-unbounded-k-models-v1`

Let C be acyclic. Total currying, with the same Koszul convention as the internal Hom differential, gives an isomorphism of complexes of groups

$$\mathrm{Hom}^\bullet\big(C, \underline{\mathrm{Hom}}_A^\bullet(P, I)\big) \cong \mathrm{Hom}^\bullet(C \otimes_A P, I).$$

The tensor complex on the right is acyclic because P is K-flat, and its Hom complex into I is acyclic because I is K-injective. This is exactly the K-injective condition for the displayed internal Hom. □

Proposition 11.12.314 (Source-resolution independence of derived internal Hom). Lean: `conditional-interface:prop:weil-a4-derived-hom-source-independence-open`

Uses: `def:weil-a4-derived-hom-source-comparison`, `def:weil-a4-fixed-kinjective-replacement`

If $u : P' \to P$ is a quasi-isomorphism between K-flat source models, then

$$u^* : \underline{\mathrm{Hom}}_A^\bullet(P, I(L)) \longrightarrow \underline{\mathrm{Hom}}_A^\bullet(P', I(L))$$

is a quasi-isomorphism. Two roofs connecting source representatives induce the same morphism in the derived category.

Proof. Uses: `lemma:weil-a4-derived-hom-source-comparison-chain,prop:weil-a4-kinjective-replacement-existence-open,lemma:weil-a4-kflat-kinjective-internal-hom,thm:weil-general-unbounded-kflat-objectwise-discharge-v1,lemma:weil-general-unbounded-kflat-tensor-qis-v1,lemma:weil-a4-kinjective-localization-hom,lemma:weil-fresh-adapted-roof-cofinality-v1`

Write $H = \underline{\mathrm{Hom}}_A^\bullet(P, I(L))$ and define H' similarly. The preceding lemma makes both H and H' K-injective. For an arbitrary derived object C, choose a 06YS K-flat arrow $R \to C$. The K-injective localization comparison and total currying identify the map induced by u^* with

$$\mathrm{Hom}_D(R \otimes_A P, I(L)) \longrightarrow \mathrm{Hom}_D(R \otimes_A P', I(L)).$$

This is a bijection: K-flatness of R makes $R \otimes u$ a quasi-isomorphism. Thus u^* induces a bijection on $\mathrm{Hom}_D(C, -)$ for every C. Yoneda makes it an isomorphism in the derived category, and the cohomology functors then make the displayed cochain map a quasi-isomorphism.

For roof independence, refine two roofs to a common K-flat roof. Their maps induce the same transformation of the functors just displayed, since both are precomposition by the same localized source isomorphism. Yoneda therefore identifies the two derived morphisms. Induction gives the same conclusion for a finite zigzag. □

Definition 11.12.315 (Constructible derived internal Hom restriction)**.** Lean: `conditional-interface:def:weil-a4-constructible-derived-internal-hom`

Uses: `def:weil-a4-derived-internal-hom,def:weil-a2-dbc-zl`

When the ambient object $R\underline{\mathrm{Hom}}_X(K, L)$ is bounded constructible, denote its restriction to $D_c^b(X, A)$ by $R\mathcal{H}om_X(K, L)$.

Proposition 11.12.316 (Constructible closure of internal Hom)**.** Lean: `conditional-interface:prop:weil-a4-constructible-internal-hom-open`

Uses: `def:weil-a4-constructible-derived-internal-hom`

OPEN. *For the dualizable/finite Tor-amplitude first arguments and constructible bounded second arguments used below, $R\mathcal{H}om_X(K, L)$ is bounded constructible.*

Open obligation: `OBL-A4-IH-03`

Proof. Uses: `def:weil-a4-constructible-derived-internal-hom`

The proof must first identify the internal Hom on a stratum where K is a bounded complex of finite free local systems with $K^\vee \otimes L$. It must then pass through one open–closed distinguished triangle at a time and record the amplitude after each step. This devissage is not in the current foundation and is exactly 'OBL-A4-IH-03'. □

Definition 11.12.317 (Derived evaluation)**.** Lean: `conditional-interface:def:weil-a4-derived-evaluation`

Uses: `def:weil-a4-derived-internal-hom,def:weil-a4-internal-evaluation,def:weil-a2-completed-tensor`

On K-flat and K-injective representatives, totalize the degreewise evaluation with Koszul signs to define

$$\mathrm{ev}_{K,L} : R\underline{\mathrm{Hom}}_X(K, L) \otimes_X^L K \longrightarrow L.$$

Definition 11.12.318 (Derived currying map)**.** Lean: `conditional-interface:def:weil-a4-derived-currying`

Uses: `def:weil-a4-derived-internal-hom,def:weil-a4-derived-evaluation`

For $u : P \otimes_X^L K \to L$, define $\mathrm{Curry}(u) : P \to R\underline{\mathrm{Hom}}_X(K, L)$ by applying underived currying on the chosen total complexes.

Definition 11.12.319 (Derived uncurrying map)**.** Lean: `conditional-interface:def:weil-a4-derived-uncurrying`

Uses: `def:weil-a4-derived-evaluation`

For $v : P \to R\underline{\mathrm{Hom}}_X(K, L)$, define

$$\mathrm{Uncurry}(v) := \mathrm{ev}_{K,L} \circ (v \otimes \mathrm{id}_K).$$

Proposition 11.12.320 (Derived tensor–Hom adjunction)**.** Lean: `conditional-interface:prop:weil-a4-derived-tensor-hom-adjunction-open`

Uses: `def:weil-a4-derived-currying,def:weil-a4-derived-uncurrying`

OPEN. *The derived currying and uncurrying maps are mutually inverse, natural in all variables, independent of replacements, and their defining rectangles commute with finite reduction and scalar extension. The ordinary ambient module-sheaf calculation and replacement independence are proved below. Passing that calculation through completed tensor products, linearized/Karoubi coefficient categories, and the filtered $\overline{\mathbf{Q}}_\ell$-coefficient category remains open.*

Open obligation: `OBL-A4-IH-04`

Proof. Uses: prop:weil-a4-underived-tensor-hom-adjunction,prop:weil-a4-kinjective-replacement-existence-open,thm:weil-general-unbounded-kflat-objectwise-discharge-v1,lemma:weil-a4-kflat-kinjective-internal-hom,lemma:weil-a4-kinjective-localization-hom,prop:weil-a4-derived-hom-source-independence-open,def:weil-a4-derived-evaluation,prop:weil-a2-completed-monoidal-coherence-open

Resolve the two tensor inputs by the constructed K-flat models P, Q and the target by the constructed K-injective model I. The preceding lemma makes $\underline{\mathrm{Hom}}^\bullet(Q, I)$ K-injective. Hence the two K-injective localization comparisons reduce the desired derived Hom sets to homotopy classes, where the underived adjunction is the degree-zero part of the isomorphism of total Hom complexes

$$\underline{\mathrm{Hom}}^\bullet(P \otimes Q, I) \xrightarrow{\sim} \underline{\mathrm{Hom}}^\bullet(P, \underline{\mathrm{Hom}}^\bullet(Q, I)), \qquad h \longmapsto \big(p \mapsto (q \mapsto h(p \otimes q))\big),$$

with the Koszul sign fixed by the displayed total differentials. Expanding one homogeneous component shows that this map commutes with differentials. Its inverse evaluates the curried cochain. The two composites are literally the identity on homogeneous tensors, so they remain inverse after taking degree-zero cocycles modulo homotopy and localizing.

A change of K-flat source is carried to a common roof; source-resolution independence cancels the roof arrows. For two K-injective resolutions $L \to I$ and $L \to I'$, the localized zigzag gives an isomorphism $I \to I'$. The K-injective localization lemma represents it by a unique homotopy class of cochain maps; it is a quasi-isomorphism because it is an isomorphism in the derived category. Postcomposition with this map gives the target-model comparison, and a common third resolution proves its cocycle and naturality. This proves replacement independence without assuming a comparison datum between packages. Naturality follows by inserting a cochain map in the displayed formula: both routes evaluate the same ordered composite on $p \otimes q$.

This proves the tensor–Hom adjunction and its model independence in the ordinary unbounded derived category of modules on the ringed site. It does not prove the coefficient part of the proposition. At the integral A4 index, Definition 11.12.317 uses the completed tensor product, whose associator, unitors, symmetry, and compatibility with reduction are still the content of Proposition 10.17.3. At finite extensions and at $\overline{\mathbf{Q}}_\ell$, preservation of D_c^b, the scalar-extension composition comparison, and its transition cocycle are likewise still open. The finite-reduction functor by itself does not supply the strong-monoidal comparison needed to form the mate of evaluation. Consequently the reduction and scalar-extension rectangles cannot yet be obtained from the ordinary calculation, and OBL-A4-IH-04 remains open. □

11.12.5 Adic reconstruction and coefficient change

Definition 11.12.321 (Reduction of the exceptional Hom problem)**.** Lean: conditional-interface:def:weil-a4-exceptional-reduction-problem

Uses: def:weil-a4-exceptional-index,def:weil-a2-finite-reduction,def:weil-a4-finite-exceptional-pullback-object

For $L \in D_c^b(Y, \Lambda)$, let $L_n = L \otimes_A^L \Lambda_n$. The level-$n$ reduction problem is the pair

$$Rf_!(-) \dashv Rf_n^!(-) : D_c^b(X, \Lambda_n) \rightleftarrows D_c^b(Y, \Lambda_n)$$

together with the reduction comparison maps $\rho_{n+1,n} : L_{n+1} \otimes^L \Lambda_n \to L_n$.

Definition 11.12.322 (Level transition candidate for exceptional pullback)**.** Lean: conditional-interface:def:weil-a4-exceptional-level-transition

Uses: def:weil-a4-exceptional-reduction-problem,def:weil-a4-finite-exceptional-pullback-object

For $L \in D_c^b(Y, \Lambda_{n+1})$, define

$$\tau_{n+1,n,L} : Rf_{n+1}^! L \otimes_{\Lambda_{n+1}}^L \Lambda_n \longrightarrow Rf_n^!(L \otimes_{\Lambda_{n+1}}^L \Lambda_n)$$

to be the unique map whose image under the level-n Hom comparison is the composite obtained by reducing the counit $Rf_! Rf_{n+1}^! L \to L$.

Lemma 11.12.323 (Type correctness of the level transition)**.** Lean: conditional-interface:lemma:weil-a4-exceptional-level-transition-type

Uses: def:weil-a4-exceptional-level-transition,def:weil-a4-finite-exceptional-counit

The source and target of $\tau_{n+1,n,L}$ lie in $D_c^b(X, \Lambda_n)$, and the defining reduced counit has type

$$Rf_!(Rf_{n+1}^! L \otimes^L \Lambda_n) \longrightarrow L \otimes^L \Lambda_n.$$

Proof. Uses: def:weil-a4-exceptional-level-transition,def:weil-a4-finite-exceptional-counit

The source type is the A2 finite reduction of a constructible object; the target type is the definition of L_n. Apply the coefficient-change map for $Rf_!$ from A1 to the counit at level $n+1$; its target is exactly the displayed reduced counit. No inverse assertion is used. □

Proposition 11.12.324 (Level transition is uniquely determined). *Lean:* `conditional-interface:prop:weil-a4-exceptional-level-transition-unique-open`

Uses: `def:weil-a4-exceptional-level-transition`, `def:weil-a4-finite-exceptional-forward-map`

OPEN. *The map $\tau_{n+1,n,L}$ exists and is unique.*

Open obligation: `OBL-A4-AD-01`

Proof. Uses: `lemma:weil-a4-exceptional-level-transition-type`, `prop:weil-a4-derived-tensor-hom-adjunction-open`

Existence is the inverse image of the reduced counit under the level-n representing bijection. Uniqueness follows by injectivity of that bijection. The required comparison between reduction of the source and the source of the level-n representing problem uses the derived tensor–Hom reduction rectangle, which is included in 'OBL-A4-AD-01'. □

Lemma 11.12.325 (Identity level transition). *Lean:* `conditional-interface:lemma:weil-a4-exceptional-level-transition-identity`

Uses: `def:weil-a4-exceptional-level-transition`

OPEN. *For L at level n, the transition from level n to itself is the identity.*

Open obligation: `OBL-A4-AD-02`

Proof. Uses: `def:weil-a4-exceptional-level-transition`, `def:weil-a4-finite-exceptional-counit`, `prop:weil-a4-exceptional-level-transition-unique-open`

The reduced counit is unchanged by the identity reduction map. The identity map on $Rf_n^! L$ has the same image under the representing bijection as the defining transition. Uniqueness gives equality. □

Lemma 11.12.326 (Two-step level transition coherence). *Lean:* `conditional-interface:lemma:weil-a4-exceptional-level-transition-composition`

Uses: `def:weil-a4-exceptional-level-transition`

OPEN. *For $m \geq n \geq r$,*

$$\tau_{n,r,L_n} \circ (\tau_{m,n,L_m} \otimes^L \Lambda_r) = \tau_{m,r,L_m}.$$

Open obligation: `OBL-A4-AD-02`

Proof. Uses: `def:weil-a4-exceptional-level-transition`, `lemma:weil-a4-exceptional-level-transition-identity`, `prop:weil-a4-exceptional-level-transition-unique-open`

Reduce the three maps' counits to level r. Each composite becomes the same level-r reduction of the counit at level m, because A2 reduction maps compose and A1's coefficient comparison is transitive. Injectivity of the level-r representing bijection gives the equality. □

Definition 11.12.327 (Exceptional inverse-image tower). *Lean:* `conditional-interface:def:weil-a4-exceptional-inverse-image-tower`

Uses: `def:weil-a4-exceptional-level-transition`

For $L \in D_c^b(Y, \Lambda)$, define the inverse system

$$\mathcal{T}_f(L) = \left(Rf_n^! L_n, \tau_{n+1,n,L}\right)_{n \geq 1}.$$

The preceding identity and two-step lemmas are its normalization and cocycle equations.

Definition 11.12.328 (Adic exceptional inverse image candidate). *Lean:* `conditional-interface:def:weil-a4-adic-exceptional-pullback`

Uses: `def:weil-a4-exceptional-inverse-image-tower`, `def:weil-a2-derived-completion`

Define

$$Rf_\Lambda^! L := R\varprojlim_n Rf_n^! L_n$$

using the explicit derived inverse-limit model from A2. The comparison map $Rf_\Lambda^! L \otimes^L \Lambda_n \to Rf_n^! L_n$ is the canonical projection from the derived-limit triangle.

Lemma 11.12.329 (Adic exceptional reduction comparison). *Lean:* `conditional-interface:lemma:weil-a4-adic-exceptional-reduction-comparison`

Uses: `def:weil-a4-adic-exceptional-pullback`, `def:weil-a2-derived-completion`

The reduction comparison map has the displayed type and is natural in L and n.

Proof. Uses: def:weil-a4-adic-exceptional-pullback,def:weil-a4-exceptional-inverse-image-tower
The map is the n-th projection of the inverse-limit cone. Naturality follows because a morphism of towers is a commuting family of level maps and the cone construction is functorial. □

Proposition 11.12.330 (Adic exceptional reduction is an isomorphism). Lean: conditional-interface:prop:weil-a4-adic-exceptional-reduction-open

Uses: def:weil-a4-adic-exceptional-pullback
OPEN. *For every n,*

$$Rf^!_\Lambda L \otimes^L \Lambda_n \xrightarrow{\sim} Rf^!_n L_n.$$

Open obligation: OBL-A4-AD-03

Proof. Uses: def:weil-a4-adic-exceptional-pullback,lemma:weil-a4-adic-exceptional-reduction-comparison
The cone of the projection is computed by the derived inverse-limit triangle. Vanishing of the higher derived-limit contribution and exactness of reduction identify that cone with zero. The required Mittag–Leffler vanishing and finite-amplitude bound are not in the foundation; they are the two explicit subclaims of 'OBL-A4-AD-03'. □

Proposition 11.12.331 (Adic exceptional Hom adjunction). Lean: conditional-interface:prop:weil-a4-adic-exceptional-adjunction-open

Uses: def:weil-a4-adic-exceptional-pullback, def:weil-a4-finite-exceptional-counit
OPEN. *There are natural mutually inverse bijections*

$$\mathrm{Hom}_Y(Rf_! K, L) \simeq \mathrm{Hom}_X(K, Rf^!_\Lambda L).$$

Open obligation: OBL-A4-AD-04

Proof. Uses: def:weil-a4-adic-exceptional-pullback,def:weil-a4-exceptional-inverse-image-tower,prop:weil-a4-finite-representing-left-inverse-open,prop:weil-a4-finite-representing-right-inverse-open
Reduce a prospective map to each n. The finite-level representing bijections give a compatible inverse-system of maps. The inverse-limit universal property produces the adic map, and reducing it recovers every finite-level map. Conversely, a map to the inverse limit is determined by its projections. The missing commutation of Hom with this inverse limit, including the $R^1\varprojlim$ term, is 'OBL-A4-AD-04'. □

Definition 11.12.332 (Exceptional unit and counit at adic coefficients). Lean: conditional-interface:def:weil-a4-adic-unit-counit

Uses: def:weil-a4-adic-exceptional-pullback,def:weil-a4-finite-exceptional-unit,def:weil-a4-finite-exceptional-counit
Define $\eta^!_{f,\Lambda,K}$ as the unique map to the derived inverse limit whose level-n projection is the finite unit $\eta^!_{f,n,K_n}$. Define $\epsilon^!_{f,\Lambda,L}$ as the unique map whose reduction at every level is the finite counit $\epsilon^!_{f,n,L_n}$:

$$\eta^!_{f,\Lambda,K} : K \to Rf^!_\Lambda Rf_! K, \qquad \epsilon^!_{f,\Lambda,L} : Rf_! Rf^!_\Lambda L \to L.$$

Existence and uniqueness of both compatible maps are exactly the comparison content recorded in 'OBL-A4-AD-04'.

Proposition 11.12.333 (Adic triangle identities). Lean: conditional-interface:prop:weil-a4-adic-triangle-identities-open

Uses: def:weil-a4-adic-unit-counit
OPEN. *The adic unit and counit satisfy both adjunction triangle identities.*

Open obligation: OBL-A4-AD-04

Proof. Uses: def:weil-a4-adic-unit-counit,prop:weil-a4-adic-exceptional-adjunction-open
Apply either inverse Hom bijection to the composite in the corresponding triangle. It becomes an identity morphism, and the inverse bijection then returns the identity. The only nonformal point is that the adic bijection is indeed inverse, which is 'OBL-A4-AD-04'. □

Definition 11.12.334 (Scalar-extension exceptional inverse image). Lean: conditional-interface:def:weil-a4-scalar-exceptional-pullback

Uses: def:weil-a4-adic-exceptional-pullback,def:weil-a2-integral-scalar-extension,def:weil-a2-ql-to-qbar
For E or $\overline{\mathbf{Q}}_\ell$, define

$$Rf^!_E L_E := Rf^!_\Lambda(L_E^{\mathrm{int}}) \otimes^L_\Lambda E,$$

where L_E^{int} is an integral lattice representative supplied by A2; for $\overline{\mathbf{Q}}_\ell$ take the filtered colimit over finite extensions.

Proposition 11.12.335 (Independence of the scalar-extension lattice). *Lean:* `conditional-interface:prop:weil-a4-scalar-exceptional-lattice-independent-open`

Uses: `def:weil-a4-scalar-exceptional-pullback`

OPEN. *The scalar-extension object and its unit/counit are independent of the chosen lattice and finite coefficient field.*

Open obligation: `OBL-A4-AD-05`

Proof. Uses: `def:weil-a4-scalar-exceptional-pullback,prop:weil-a4-adic-exceptional-adjunction-open`

Given two lattices, choose a common lattice inside their intersection after multiplying by a power of ℓ. The two reduction towers agree from that stage onward by the finite-level representing uniqueness. Tensoring the resulting comparison with E gives an isomorphism; for algebraic closure, the same argument on a common finite extension gives a filtered-colimit map. The assertion that scalar extension preserves the reduction tower is 'OBL-A4-AD-05'. □

11.12.6 Composition, units, counits, and projection

Definition 11.12.336 (Composable exceptional indices). *Lean:* `conditional-interface:def:weil-a4-composable-exceptional-indices`

Uses: `def:weil-a4-exceptional-index,def:weil-a1-rfshriek-composition-map`

For separated finite-type maps $X \xrightarrow{f} Y \xrightarrow{g} Z$, let $h = g \circ f$. The composable index consists of the three A1 functors $Rf_!, Rg_!, Rh_!$, their A1 composition map

$$\alpha_{g,f} : Rg_!Rf_! \longrightarrow Rh_!,$$

and the adic exceptional objects $Rf^!, Rg^!, Rh^!$ when the preceding adic OPENs are discharged.

Definition 11.12.337 (Exceptional composition comparison). *Lean:* `conditional-interface:def:weil-a4-exceptional-composition-comparison`

Uses: `def:weil-a4-composable-exceptional-indices,def:weil-a4-adic-unit-counit`

For $M \in D_c^b(Z, A)$, define

$$c_{g,f,M} : Rf^!Rg^!M \longrightarrow R(gf)^!M$$

as the unique morphism whose image under the adjunction bijection for $g \circ f$ is the two-step counit. Explicitly, it is the unique map for which

$$\epsilon^!_{gf,M} \circ R(gf)_!(c_{g,f,M}) = \epsilon^!_{g,M} \circ Rg_!(\epsilon^!_{f,Rg^!M}) \circ \alpha^{-1}_{g,f,Rf^!Rg^!M}.$$

Lemma 11.12.338 (Type of exceptional composition comparison). *Lean:* `conditional-interface:lemma:weil-a4-exceptional-composition-comparison-type`

Uses: `def:weil-a4-exceptional-composition-comparison`

Every composite in the defining equation has source $R(gf)_!Rf^!Rg^!M$ and target M, so the equation is well typed.

Proof. Uses: `def:weil-a4-exceptional-composition-comparison`

The A1 map $\alpha_{g,f}$ changes $Rg_!Rf_!$ to $R(gf)_!$, and its inverse changes $R(gf)_!$ back to $Rg_!Rf_!$. The first counit changes $Rf_!Rf^!Rg^!M$ to $Rg^!M$, the second changes that object to M. Each adjacent codomain is the next domain, proving the type assertion. □

Definition 11.12.339 (Exceptional composition inverse candidate). *Lean:* `conditional-interface:def:weil-a4-exceptional-composition-inverse`

Uses: `def:weil-a4-composable-exceptional-indices,def:weil-a4-exceptional-composition-comparison`

Define

$$d_{g,f,M} : R(gf)^!M \longrightarrow Rf^!Rg^!M$$

as the unique map whose image under the representing bijection for (gf) is the counit composite

$$Rg_!Rf_!R(gf)^!M \xrightarrow{\alpha_{g,f}} R(gf)_!R(gf)^!M \xrightarrow{\epsilon^!_{gf,M}} M.$$

Lemma 11.12.340 (Composition comparison inverse equations). Lean: `conditional-interface:lemma:weil-a4-exceptional-composition-inverse-equations-open`

Uses: `def:weil-a4-exceptional-composition-comparison`, `def:weil-a4-exceptional-composition-inverse`

OPEN. *For every M,*

$$d_{g,f,M}c_{g,f,M} = \mathrm{id}, \qquad c_{g,f,M}d_{g,f,M} = \mathrm{id}\,.$$

Open obligation: `OBL-A4-PS-01`

Proof. Uses: `def:weil-a4-exceptional-composition-comparison`, `def:weil-a4-exceptional-composition-inverse`, `prop:weil-a4-adic-triangle-identities-open`

Apply the adjunction Hom bijection to each composite. The first composite reduces by the f- and g-counit equations to the identity on the two-step counit; the second reduces by the gf-counit equation to the identity. The A1 comparison $\alpha_{g,f}$ must be inserted in both orders; its inverse equations and composition compatibility are the missing part 'OBL-A4-PS-01'. □

Proposition 11.12.341 (Exceptional composition isomorphism). Lean: `conditional-interface:prop:weil-a4-exceptional-composition-isomorphism-open`

Uses: `def:weil-a4-exceptional-composition-comparison`, `def:weil-a4-exceptional-composition-inverse`

OPEN. *$c_{g,f,M}$ is an isomorphism with inverse $d_{g,f,M}$, natural in M.*

Open obligation: `OBL-A4-PS-01`

Proof. Uses: `lemma:weil-a4-exceptional-composition-inverse-equations-open`, `def:weil-a4-exceptional-composition-comparison`, `def:weil-a4-exceptional-composition-inverse`

The two equations are exactly the definition of a two-sided inverse. For naturality, apply the representing bijection to a morphism $M \to M'$; both paths become the same composite of counits and $\alpha_{g,f}$, and injectivity of the bijection gives equality. □

Proposition 11.12.342 (Exceptional composition associativity). Lean: `conditional-interface:prop:weil-a4-exceptional-composition-associativity-open`

Uses: `def:weil-a4-composable-exceptional-indices`, `def:weil-a4-exceptional-composition-comparison`

OPEN. *For $X \xrightarrow{f} Y \xrightarrow{g} Z \xrightarrow{h} W$, the two composites from $Rf^!Rg^!Rh^!M$ to $R(hgf)^!M$ agree.*

Open obligation: `OBL-A4-PS-02`

Proof. Uses: `prop:weil-a4-exceptional-composition-isomorphism-open`, `def:weil-a4-exceptional-composition-comparison`

Apply the hgf-representing bijection. Both paths become the composite $Rh_!Rg_!Rf_! \to M$ formed from three counits and the A1 associativity rectangle. The two parenthesizations are equal after the A1 pentagon, so the representing bijection's injectivity gives the result. □

Proposition 11.12.343 (Exceptional identity unit laws). Lean: `conditional-interface:prop:weil-a4-exceptional-identity-unit-laws-open`

Uses: `def:weil-a4-exceptional-composition-comparison`, `def:weil-a4-finite-exceptional-unit`

OPEN. *The composition comparisons for $\mathrm{id}_X \circ f$ and $f \circ \mathrm{id}_X$ agree with the unit comparison of A1 and the identity transformation of $Rf^!$.*

Open obligation: `OBL-A4-PS-03`

Proof. Uses: `prop:weil-a4-exceptional-composition-isomorphism-open`, `prop:weil-a4-finite-exceptional-first-triangle-open`, `prop:weil-a4-finite-exceptional-second-triangle-open`

Apply the relevant representing bijection. The two composites reduce to a triangle identity with the A1 identity compactification map. The first and second triangle identities then identify them with identity maps. The A1 unit coherence remains the leaf 'OBL-A4-PS-03'. □

Definition 11.12.344 (Exceptional trace morphism). Lean: `conditional-interface:def:weil-a4-exceptional-trace`

Uses: `def:weil-a4-adic-unit-counit`, `def:weil-a4-exceptional-index`

For every separated finite-type $f : X \to Y$, define the trace (counit)

$$\mathrm{Tr}_f := \epsilon_f^! : Rf_!Rf^! \longrightarrow \mathrm{id}_{D_c^b(Y,A)}\,.$$

Lemma 11.12.345 (Trace naturality in the target). *Lean:* `conditional-interface:lemma:weil-a4-exceptional-trace-target-naturality-open`

Uses: `def:weil-a4-exceptional-trace`, `def:weil-a4-adic-unit-counit`

OPEN. *For $v : L \to L'$,*

$$v \circ \mathrm{Tr}_{f,L} = \mathrm{Tr}_{f,L'} \circ Rf_!(Rf^!(v)).$$

Open obligation: `OBL-A4-TR-01`

Proof. Uses: `def:weil-a4-exceptional-trace,prop:weil-a4-adic-triangle-identities-open`

This is the naturality equation for the counit of the adjunction. Expand both sides as images of identity morphisms under the two inverse Hom bijections; both are the image of v. Its validity at adic and scalar coefficients is the remaining obligation 'OBL-A4-TR-01'. □

Lemma 11.12.346 (Trace transitivity). *Lean:* `conditional-interface:lemma:weil-a4-exceptional-trace-transitivity-open`

Uses: `def:weil-a4-exceptional-trace`, `def:weil-a4-exceptional-composition-comparison`

OPEN. *For $X \xrightarrow{f} Y \xrightarrow{g} Z$,*

$$\mathrm{Tr}_{gf} \circ R(gf)_!(c_{g,f,-}) \circ \alpha_{g,f,Rf^!Rg^!(-)} = \mathrm{Tr}_g \circ Rg_!(\mathrm{Tr}_f).$$

Open obligation: `OBL-A4-TR-02`

Proof. Uses: `prop:weil-a4-exceptional-composition-isomorphism-open,def:weil-a4-exceptional-trace`

Apply the representing bijection of gf to both maps. The left side is the counit of gf; the right side is the two-step counit. The defining equation of $c_{g,f,-}$ identifies them. The unresolved coefficient and A1 coherence checks are 'OBL-A4-TR-02'. □

Definition 11.12.347 (Projection formula comparison). Lean: `conditional-interface:def:weil-a4-projection-formula-comparison`

Uses: `def:weil-a4-exceptional-index,def:weil-a4-derived-evaluation,def:weil-a4-exceptional-trace`

For $K \in D_c^b(X, A)$, $L \in D_c^b(Y, A)$, define

$$\pi_{f,K,L} : Rf_!K \otimes_Y^L L \longrightarrow Rf_!\big(K \otimes_X^L Lf^*L\big)$$

by deriving the sheaf-level map which, on a compactly supported local section, sends $f_!k \otimes l$ to $f_!(k \otimes f^*l)$. For the A1 model $Rf_! = R\bar{f}_* j_!$, use the tensor comparison for $j_!$, followed by the proper pushforward tensor comparison for $R\bar{f}_*$.

Lemma 11.12.348 (Projection formula comparison is well typed). *Lean:* `conditional-interface:lemma:weil-a4-projection-formula-type`

Uses: `def:weil-a4-projection-formula-comparison`

The source and target of $\pi_{f,K,L}$ are objects of $D_c^b(Y, A)$.

Proof. Uses: `def:weil-a4-projection-formula-comparison,def:weil-a4-exceptional-index`

The tensor $K \otimes Lf^*L$ is on X, so $Rf_!$ has target Y. The second expression is the tensor of two objects on Y. Constructible closure of tensor and pullback is an A2 input; compact-support functoriality is the A1 input. □

Definition 11.12.349 (Projection formula inverse candidate). Lean: `conditional-interface:def:weil-a4-projection-formula-inverse`

Uses: `def:weil-a4-projection-formula-comparison`

Define $\pi_{f,K,L}^{-1}$ by deriving the reverse sheaf-level comparison: on representatives it sends $f_!(k \otimes f^*l)$ to $f_!k \otimes l$.

Proposition 11.12.350 (Projection formula). *Lean:* `conditional-interface:prop:weil-a4-projection-formula-open`

Uses: `def:weil-a4-projection-formula-comparison`, `def:weil-a4-projection-formula-inverse`

OPEN. *$\pi_{f,K,L}$ is an isomorphism with inverse $\pi_{f,K,L}^{-1}$, natural in K, L, associative for $L_1 \otimes L_2$, unital for A_Y, compatible with composition and with A1/A3 base change.*

Open obligation: `OBL-A4-PF-01`

Open obligation: `OBL-A4-PF-02`

Open obligation: `OBL-A4-PF-03`

Open obligation: `OBL-A4-PF-04`

Open obligation: `OBL-A4-PF-05`

Proof. Uses: `def:weil-a4-projection-formula-comparison,def:weil-a4-projection-formula-inverse,prop:weil-a4-derived-tensor-hom-adjunction-open`

On K-flat representatives, both composites send a pure tensor $f_!k \otimes l$ back to itself, and the same is true for $f_!(k \otimes f^*l)$. The two inverse equations therefore reduce to the explicit tensor identities. Replacement independence, open/proper devissage, compactification independence, and all coherence rectangles are the five separately listed OPEN obligations; they cannot be replaced by the phrase "projection formula". □

Definition 11.12.351 (Dualizable coefficient object)**.** Lean: `conditional-interface:def:weil-a4-dualizable-coefficient-object`

Uses: `def:weil-a4-constructible-derived-internal-hom,def:weil-a4-derived-evaluation`

A coefficient object $K \in D_c^b(Y, A)$ is dualizable if, with $K^\vee := R\mathcal{H}om_Y(K, A_Y)$, it is equipped with evaluation and coevaluation maps

$$K^\vee \otimes^L K \to A_Y, \qquad A_Y \to K \otimes^L K^\vee,$$

whose two evaluation–coevaluation composites are the identity maps of K and $K^\vee$.

Definition 11.12.352 (Exceptional tensor–Hom exchange comparison)**.** Lean: `conditional-interface:def:weil-a4-exceptional-tensor-hom-exchange`

Uses: `def:weil-a4-projection-formula-inverse,def:weil-a4-exceptional-trace,def:weil-a4-derived-currying,def:weil-a4-constructible-derived-internal-hom`

For $K, L \in D_c^b(Y, A)$, define

$$\mathrm{Ex}_{f,K,L} : Rf^! R\mathcal{H}om_Y(K, L) longrightarrow R\mathcal{H}om_X(Lf^*K, Rf^!L)$$

by currying the morphism whose adjoint under $Rf_! \dashv Rf^!$ is

$$\begin{aligned} Rf_!\big(Lf^*K \otimes^L Rf^! R\mathcal{H}om_Y(K, L)\big) &\xrightarrow{\pi^{-1}} K \otimes^L Rf_! Rf^! R\mathcal{H}om_Y(K, L) \\ &\xrightarrow{\mathrm{id} \otimes \mathrm{Tr}_f} K \otimes^L R\mathcal{H}om_Y(K, L) \xrightarrow{\mathrm{ev}} L. \end{aligned}$$

Definition 11.12.353 (Exceptional tensor–Hom exchange inverse candidate)**.** Lean: `conditional-interface:def:weil-a4-exceptional-tensor-hom-inverse`

Uses: `def:weil-a4-exceptional-tensor-hom-exchange,def:weil-a4-dualizable-coefficient-object,def:weil-a4-projection-formula-inverse`

For dualizable K, define

$$\mathrm{Ex}^{-1}_{f,K,L} : R\mathcal{H}om_X(Lf^*K, Rf^!L) longrightarrow Rf^! R\mathcal{H}om_Y(K, L)$$

by first using the dual-basis identification of the source with $Lf^*K^\vee \otimes^L Rf^!L$, then applying the adjoint of

$$Rf_!\big(Lf^*K^\vee \otimes^L Rf^!L\big) \xrightarrow{\pi^{-1}} K^\vee \otimes^L Rf_! Rf^!L \xrightarrow{\mathrm{id} \otimes \mathrm{Tr}_f} K^\vee \otimes^L L,$$

and finally identifying $K^\vee \otimes^L L$ with $R\mathcal{H}om_Y(K, L)$.

Proposition 11.12.354 (Exceptional tensor–Hom exchange)**.** *Lean:* `conditional-interface:prop:weil-a4-exceptional-tensor-hom-exchange-open`

Uses: `def:weil-a4-exceptional-tensor-hom-exchange, def:weil-a4-exceptional-tensor-hom-inverse, def:weil-a4-dualizable-coefficient-object`

OPEN. *For dualizable K, the two exchange maps are inverse, natural in K, L, and compatible with composition, reduction, scalar extension, and A3 base change.*

Open obligation: `OBL-A4-TH-01`

Proof. Uses: `def:weil-a4-exceptional-tensor-hom-exchange,def:weil-a4-exceptional-tensor-hom-inverse,prop:weil-a4-projection-formula-open,prop:weil-a4-derived-tensor-hom-adjunction-open`

On a stratum where K is finite free, choose a basis e_i with dual basis $e_i^\vee$. After expanding currying, the first composite sends a functional φ to $\sum_i \varphi(e_i)e_i^\vee$ and then evaluates the same dual-basis sum, so it is φ. The reverse composite performs the other evaluation–coevaluation triangle and is also the identity. Extending this calculation through constructible devissage and verifying every coefficient, composition, and base-change rectangle is exactly 'OBL-A4-TH-01'. □

11.12.7 Smooth purity and the trace normalization

Definition 11.12.355 (Smooth morphism with relative dimension)**.** Lean: `conditional-interface:def:weil-a4-smooth-relative-dimension`

Uses: `def:weil-a4-exceptional-index`

A smooth-purity index is a smooth separated finite-type morphism $f : X \to Y$ together with an integer d and a chosen finite etale cover of X by diagrams

$$X_i \xrightarrow{u_i} \mathbf{A}^d_Y \xrightarrow{p} Y, \qquad f|_{X_i} = p \circ u_i,$$

where each u_i is etale and the relative dimension of f is d.

Definition 11.12.356 (Affine-line zero-section purity class)**.** Lean: `conditional-interface:def:weil-a4-affine-line-purity-class`

Uses: `def:weil-a4-tate-twist,def:weil-a4-smooth-relative-dimension`

Let $s : Y \hookrightarrow \mathbf{A}^1_Y$ be the zero section and $p : \mathbf{A}^1_Y \to Y$ the projection. The finite-level purity class is the local-cohomology class

$$\mathrm{pur}_s : \Lambda_n \longrightarrow s^!\Lambda_n(1)[2]$$

whose restriction to every geometric fiber is the orientation class with residue $1 \in \Lambda_n$.

Open obligation: `OBL-A4-SP-01`

Lemma 11.12.357 (Affine-line purity class has the required support)**.** *Lean:* `conditional-interface:lemma:weil-a4-affine-line-purity-support`

Uses: `def:weil-a4-affine-line-purity-class`

The source of pur_s *is supported on the zero section and its restriction to* $\mathbf{A}^1_Y \setminus s(Y)$ *is zero.*

Proof. Uses: `def:weil-a4-affine-line-purity-class`

By definition the class lies in cohomology with support on $s(Y)$. The localization triangle for the open complement sends every supported class to zero after restriction, which is the claimed vanishing. □

Lemma 11.12.358 (Affine-line purity class is normalized on geometric fibers)**.** *Lean:* `conditional-interface:lemma:weil-a4-affine-line-purity-fiber-normalization`

Uses: `def:weil-a4-affine-line-purity-class`

For every geometric point $\bar{y} \to Y$*, the pullback of* pur_s *to* $\mathbf{A}^1_{\bar{y}}$ *is the unique class whose residue at* 0 *is* 1*.*

Proof. Uses: `def:weil-a4-affine-line-purity-class`

The residue condition is part of the defining fiberwise normalization. Uniqueness is the assertion that the relevant supported cohomology group of the strict henselian affine line is free of rank one; this rank-one calculation is the unresolved content of 'OBL-A4-SP-01'. □

Definition 11.12.359 (Affine-line purity comparison)**.** Lean: `conditional-interface:def:weil-a4-affine-line-purity-comparison`

Uses: `def:weil-a4-affine-line-purity-class,def:weil-a4-finite-exceptional-pullback-object`

For $K \in D^b_c(Y, \Lambda_n)$, define

$$\gamma_{p,K} : p^*K(1)[2] \longrightarrow p^!K$$

by tensoring pur_s with K, applying the smooth base-change identification for the zero section, and composing with the unit/counit maps of $p_! \dashv p^!$.

Lemma 11.12.360 (Affine-line trace calculation)**.** *Lean:* `conditional-interface:lemma:weil-a4-affine-line-trace-calculation-open`

Uses: `def:weil-a4-affine-line-purity-comparison,def:weil-a4-exceptional-trace`

OPEN. *The composite*

$$p_!p^*K(1)[2] \xrightarrow{p_!(\gamma_{p,K})} p_!p^!K \xrightarrow{\mathrm{Tr}_p} K$$

is the identity.

Open obligation: `OBL-A4-SP-02`

Proof. Uses: `def:weil-a4-affine-line-purity-class,def:weil-a4-exceptional-trace`

After geometric-stalk pullback, the assertion is the equality between the orientation class and the compactly supported fundamental class of the affine line. The fiber calculation is reduced to $R\Gamma_c(\mathbf{A}^1_{\bar{y}}, \Lambda_n) = \Lambda_n(-1)[-2]$, and its compatibility with the residue generator. This is the explicit finite-level calculation required by 'OBL-A4-SP-02'. □

Definition 11.12.361 (Affine-space purity comparison). Lean: `conditional-interface:def:weil-a4-affine-space-purity-comparison`

Uses: `def:weil-a4-affine-line-purity-comparison,def:weil-a4-tate-twist`

For $p_d : \mathbf{A}^d_Y \to Y$, define

$$\gamma_{p_d,K} : p_d^* K(d)[2d] \longrightarrow p_d^! K$$

as the ordered tensor product of the d affine-line maps, with the Koszul sign determined by the order $(1, \dots, d)$.

Lemma 11.12.362 (Affine-space purity composition). Lean: *conditional-interface:lemma:weil-a4-affine-space-purity-composition-open*

Uses: *def:weil-a4-affine-space-purity-comparison*

OPEN. *The affine-space maps satisfy*

$$\gamma_{p_{d+e},K} = \gamma_{p_d,p_e^! K} \circ (\gamma_{p_e,K}(e)[2e])$$

after the canonical reorder of twists and shifts.

Open obligation: *OBL-A4-SP-03*

Proof. Uses: `def:weil-a4-affine-space-purity-comparison,lemma:weil-a4-tate-twist-addition`

Expand both sides into $d+e$ affine-line factors. The factors occur in the same order after the displayed tensor reassociation. The only signs are the Koszul signs from moving an e-fold degree-$2e$ block past a d-fold degree-$2d$ block; since $2de$ is even, the sign is $+1$. The residual geometric input is the affine-line calculation 'OBL-A4-SP-03'. □

Definition 11.12.363 (Etale-coordinate purity map). Lean: `conditional-interface:def:weil-a4-etale-coordinate-purity-map`

Uses: `def:weil-a4-smooth-relative-dimension,def:weil-a4-affine-space-purity-comparison`

For a chosen etale coordinate $u_i : X_i \to \mathbf{A}^d_Y$, define

$$\gamma_i : u_i^* p_d^* K(d)[2d] \longrightarrow u_i^! p_d^! K$$

by applying $u_i^!$ to $\gamma_{p_d,K}$ and the etale purity identification $u_i^! = u_i^*$.

Lemma 11.12.364 (Etale-coordinate map is well typed). Lean: *conditional-interface:lemma:weil-a4-etale-coordinate-purity-type*

Uses: *def:weil-a4-etale-coordinate-purity-map*

The source of γ_i is $f|_{X_i}^* K(d)[2d]$ and the target is $f|_{X_i}^! K$.

Proof. Uses: `def:weil-a4-etale-coordinate-purity-map,def:weil-a4-smooth-relative-dimension`

The factorization $f|_{X_i} = p_d \circ u_i$ gives $u_i^* p_d^* = f|_{X_i}^*$. The etale purity identification replaces $u_i^! p_d^!$ by $f|_{X_i}^!$, proving the two type equalities. □

Lemma 11.12.365 (Overlap equality of etale-coordinate purity maps). Lean: *conditional-interface:lemma:weil-a4-etale-purity-overlap-equality-open*

Uses: *def:weil-a4-etale-coordinate-purity-map, def:weil-a4-exceptional-trace*

OPEN. *On $X_i \times_X X_j$, the maps γ_i and γ_j agree.*

Open obligation: *OBL-A4-SP-04*

Open obligation: *OBL-A4-SP-05*

Proof. Uses: `def:weil-a4-etale-coordinate-purity-map,lemma:weil-a4-affine-line-trace-calculation-open`

Pull both maps back to a geometric point of the overlap. They are maps from the same rank-one orientation object to the same exceptional pullback. Their composites with the affine trace are both the identity by the affine-line calculation. To conclude equality one must prove the trace map is injective on this rank-one summand and that etale coordinate changes preserve the orientation generator. These are the two stated OPEN obligations. □

Definition 11.12.366 (Glued smooth-purity comparison). Lean: `conditional-interface:def:weil-a4-smooth-purity-comparison`

Uses: `def:weil-a4-etale-coordinate-purity-map,def:weil-a4-smooth-relative-dimension`

Define

$$\gamma_{f,K} : f^* K(d)[2d] \longrightarrow f^! K$$

as the unique morphism whose restriction to each coordinate open X_i is γ_i.

Open obligation: `OBL-A4-SP-05`

Proposition 11.12.367 (Smooth purity at finite coefficients). Lean: `conditional-interface:prop:weil-a4-smooth-purity-finite-open`

Uses: `def:weil-a4-smooth-purity-comparison`, `def:weil-a4-smooth-relative-dimension`

OPEN. *The glued map $\gamma_{f,K}$ is an isomorphism, natural in K, stable under smooth base change and composition, and independent of the chosen etale coordinates.*

Open obligation: `OBL-A4-SP-04`

Open obligation: `OBL-A4-SP-05`

Open obligation: `OBL-A4-SP-06`

Proof. Uses: `lemma:weil-a4-etale-purity-overlap-equality-open`, `def:weil-a4-smooth-purity-comparison`

Overlap equality gives a glued map. Its inverse is glued from the local affine-space inverses. Local inverse equations imply global inverse equations because the coordinate opens cover X. Base change, composition, and independence reduce to the corresponding affine-space statements after pullback to the common etale refinement; the exact comparison rectangles are the three OPEN leaves named in the statement. □

Proposition 11.12.368 (Smooth purity at adic and scalar coefficients). Lean: `conditional-interface:prop:weil-a4-smooth-purity-all-coefficients-open`

Uses: `def:weil-a4-smooth-purity-comparison`, `def:weil-a4-tate-twist`, `def:weil-a4-adic-exceptional-pullback`

OPEN. *The finite-level maps form a compatible tower and induce*

$$f^*K(d)[2d] \xrightarrow{\sim} f^!K$$

for $A = \mathbf{Z}_\ell, E, \overline{\mathbf{Q}}_\ell$, with the fixed Tate normalization.

Open obligation: `OBL-A4-SP-07`

Proof. Uses: `prop:weil-a4-smooth-purity-finite-open`, `lemma:weil-a4-tate-frobenius-scalar`

Reduce the candidate and its local inverse to every Λ_n. The finite purity maps agree by the coefficient transition uniqueness, so they define a map of inverse systems. The adic reconstruction lemma then gives the integral isomorphism; tensoring with E and taking finite-stage filtered colimits gives the remaining coefficients. Compatibility of twists and shifts is the exact content of 'OBL-A4-SP-07'. □

11.12.8 Verdier duality and the proper case

Definition 11.12.369 (Dualizing object). Lean: `conditional-interface:def:weil-a4-dualizing-object`

Uses: `def:weil-a4-adic-exceptional-pullback`, `def:weil-a4-structural-unit`

For a finite-type X/k, define

$$\omega_{X,A} := Ra^!_X A_{\mathrm{Spec}\, k}.$$

The notation ω_X always includes the coefficient index A when the latter is not clear.

Proposition 11.12.370 (Bounded constructibility of the dualizing object). Lean: `conditional-interface:prop:weil-a4-dualizing-object-constructible-open`

Uses: `def:weil-a4-dualizing-object`, `def:weil-a4-coefficient-index`

OPEN. *The object $\omega_{X,A}$ belongs to $D^b_c(X, A)$, is compatible with finite reduction and scalar extension, and has amplitude bounded in terms of the dimension of X.*

Open obligation: `OBL-A4-VD-01`

Proof. Uses: `def:weil-a4-dualizing-object`, `prop:weil-a4-brown-object-constructible-open`, `prop:weil-a4-adic-exceptional-reduction-open`, `prop:weil-a4-scalar-exceptional-lattice-independent-open`

Apply the exceptional-pullback amplitude statement to the structural map a_X and the coefficient unit. This gives finite-level boundedness and constructibility; the adic reduction and scalar-extension comparisons give the remaining coefficient cases. The uniform dimension bound and its compatibility with those passages are exactly 'OBL-A4-VD-01', so this proposition remains OPEN. □

Definition 11.12.371 (Verdier dual functor). Lean: `conditional-interface:def:weil-a4-verdier-dual-functor`

Uses: `def:weil-a4-dualizing-object`, `def:weil-a4-constructible-derived-internal-hom`

For $K \in D^b_c(X, A)$, define

$$\mathbb{D}_X K := R\mathcal{H}om_X(K, \omega_{X,A}).$$

For $u : K \to L$, define $\mathbb{D}_X(u)$ by precomposition with u.

Lemma 11.12.372 (Verdier dual preserves identity). *Lean:* `conditional-interface:lemma:weil-a4-verdier-dual-identity`

Uses: `def:weil-a4-verdier-dual-functor`

$\mathbb{D}_X\,\mathrm{id}_K) = \mathrm{id}_{\mathbb{D}_X K}$.

Proof. Uses: `def:weil-a4-verdier-dual-functor`

Precomposition by the identity morphism is the identity map on every Hom complex, hence also on its derived object. □

Lemma 11.12.373 (Verdier dual reverses composition). *Lean:* `conditional-interface:lemma:weil-a4-verdier-dual-composition`

Uses: `def:weil-a4-verdier-dual-functor`

For $K \xrightarrow{u} L \xrightarrow{v} M$, $\mathbb{D}_X(vu) = \mathbb{D}_X(u)\mathbb{D}_X(v)$.

Proof. Uses: `def:weil-a4-verdier-dual-functor`

For a local functional $\varphi : M \to \omega_X$, both sides send φ to $\varphi \circ v \circ u$. Equality of the induced internal-Hom maps is therefore immediate from the definition. □

Definition 11.12.374 (Verdier bidual evaluation map). Lean: `conditional-interface:def:weil-a4-verdier-bidual-map`

Uses: `def:weil-a4-verdier-dual-functor,def:weil-a4-derived-evaluation`

Define

$$\delta_{X,K} : K \longrightarrow \mathbb{D}_X\mathbb{D}_X K$$

as the adjoint of the evaluation composite

$$K \otimes^L_X R\mathcal{H}om_X(K,\omega_X) \xrightarrow{\mathrm{ev}} \omega_X.$$

The order is fixed by the symmetry constraint before applying the adjunction.

Lemma 11.12.375 (Bidual map is natural). *Lean:* `conditional-interface:lemma:weil-a4-verdier-bidual-naturality`

Uses: `def:weil-a4-verdier-bidual-map, def:weil-a4-verdier-dual-functor`

For $u : K \to L$,

$$\mathbb{D}_X\mathbb{D}_X(u) \circ \delta_{X,K} = \delta_{X,L} \circ u.$$

Proof. Uses: `def:weil-a4-verdier-bidual-map,def:weil-a4-verdier-dual-functor`

Apply the tensor–Hom bijection to both sides. Evaluation of either side on a local functional $\varphi : L \to \omega_X$ is $\varphi \circ u$, so the two maps have equal adjoints and are equal. □

Definition 11.12.376 (Lisse perfect bidual inverse). Lean: `conditional-interface:def:weil-a4-lisse-bidual-inverse`

Uses: `def:weil-a4-verdier-bidual-map,def:weil-a4-smooth-purity-comparison`

Let S be a smooth stratum of dimension e on which smooth purity identifies ω_S with $A_S(e)[2e]$. For a lisse perfect K on S, cancel the evaluation pair $\omega_S^\vee \otimes \omega_S \to A_S$ in $\mathbb{D}_S\mathbb{D}_S K$, then define $\delta^{-1}_{S,K}$ by the finite-rank dual-basis formula. Locally choose a dual basis $e_i, e_i^\vee$ of K and set

$$\mathbb{D}_X\mathbb{D}_X K \longrightarrow K, \qquad \varphi \longmapsto \sum_i \varphi(e_i^\vee)e_i.$$

Lemma 11.12.377 (Lisse bidual inverse is basis independent). *Lean:* `conditional-interface:lemma:weil-a4-lisse-bidual-inverse-basis`

Uses: `def:weil-a4-lisse-bidual-inverse`

The dual-basis formula is independent of the chosen local basis.

Proof. Uses: `def:weil-a4-lisse-bidual-inverse`

If $e'_j = \sum_i a_{ji}e_i$, then $e_i^\vee = \sum_j a_{ji}e_j'^\vee$. Substituting these two expressions into the sum gives $\sum_i \varphi(e_i^\vee)e_i = \sum_j \varphi(e_j'^\vee)e'_j$. Thus the local formulas agree on overlaps and glue. □

Lemma 11.12.378 (Lisse bidual inverse equations). *Lean:* `conditional-interface:lemma:weil-a4-lisse-bidual-inverse-equations`

Uses: `def:weil-a4-verdier-bidual-map`, `def:weil-a4-lisse-bidual-inverse`

OPEN. *For lisse perfect K on a smooth stratum equipped with the purity comparison,*

$$\delta_{X,K}^{-1}\delta_{X,K} = \mathrm{id}_K, \qquad \delta_{X,K}\delta_{X,K}^{-1} = \mathrm{id}_{\mathbb{D}_X\mathbb{D}_X K}.$$

Open obligation: `OBL-A4-SP-07`

Proof. Uses: `def:weil-a4-lisse-bidual-inverse`, `def:weil-a4-verdier-bidual-map`, `prop:weil-a4-smooth-purity-all-coefficients-open`

Evaluate the first composite on each basis vector e_j; the dual-basis relation $e_i^\vee(e_j) = \delta_{ij}$ leaves e_j. Evaluate the second composite on a functional and use the expansion $\varphi = \sum_i \varphi(e_i^\vee)e_i^\vee$. Basis independence makes both computations global. □

Definition 11.12.379 (Constructible bidual stratification). *Lean:* `conditional-interface:def:weil-a4-bidual-stratification`

Uses: `def:weil-a2-constructible-complex`, `def:weil-a4-lisse-bidual-inverse`

For a constructible K, let $\mathcal{S}_K$ be the least element, in a fixed well-order of finite constructible stratifications, among those on which every stratum is smooth over k and every cohomology sheaf of K is lisse perfect. Write $X = \coprod_{i=1}^{r} X_i$ for $\mathcal{S}_K$, order its strata so that $U_j = \bigcup_{i\le j} X_i$ is open, and let $j_j : U_j \hookrightarrow U_{j+1}$, $i_j : X_{j+1} \hookrightarrow U_{j+1}$ be the open and closed maps.

Open obligation: `OBL-A4-VD-02`

Lemma 11.12.380 (Bidual map on a one-stratum complex). *Lean:* `conditional-interface:lemma:weil-a4-bidual-one-stratum`

Uses: `def:weil-a4-bidual-stratification`, `def:weil-a4-lisse-bidual-inverse`, `def:weil-a4-verdier-bidual-map`

OPEN. *For K supported on one smooth lisse-perfect stratum, $\delta_{X,K}$ is an isomorphism with the explicitly constructed inverse.*

Open obligation: `OBL-A4-VD-02`

Open obligation: `OBL-A4-SP-07`

Proof. Uses: `lemma:weil-a4-lisse-bidual-inverse-equations`, `def:weil-a4-verdier-bidual-map`

The object is lisse perfect on its support, so the two dual-basis equations apply on the stratum. Transporting those maps through extension by zero and identifying the ambient Verdier dual with the stratum Verdier dual are the localization clauses of 'OBL-A4-VD-02'. □

Lemma 11.12.381 (Bidual map and an open–closed triangle). *Lean:* `conditional-interface:lemma:weil-a4-bidual-open-closed-morphism`

Uses: `def:weil-a4-verdier-bidual-map`, `def:weil-a4-bidual-stratification`

OPEN. *For $j : U \hookrightarrow X$, $i : Z \hookrightarrow X$, the maps $\delta_{j_!K}$, δ_{i_*L}, and δ_K form a morphism of the open–closed distinguished triangles.*

Open obligation: `OBL-A4-VD-02`

Proof. Uses: `def:weil-a4-verdier-bidual-map`, `lemma:weil-a4-verdier-bidual-naturality`

On the fixed functorial cone model of the open–closed triangle, the first two squares are naturality squares of δ. The connecting square must be checked on the actual cone differential; triangulated axiom TR3 alone does not give uniqueness. That cone calculation is a clause of 'OBL-A4-VD-02'. □

Proposition 11.12.382 (Verdier biduality for constructible complexes). *Lean:* `conditional-interface:prop:weil-a4-verdier-biduality-open`

Uses: `def:weil-a4-verdier-bidual-map`, `def:weil-a4-bidual-stratification`

OPEN. *For every $K \in D_c^b(X, A)$, $\delta_{X,K}$ is an isomorphism, natural in K, and compatible with reduction and scalar extension.*

Open obligation: `OBL-A4-VD-02`

Open obligation: `OBL-A4-IH-04`

Proof. Uses: `lemma:weil-a4-bidual-one-stratum,lemma:weil-a4-bidual-open-closed-morphism,def:weil-a4-bidual-stratification`

Induct on the number of strata. The one-stratum case is the preceding lemma. The open–closed morphism gives a morphism of triangles; by induction the first two vertical maps are isomorphisms, hence the third is an isomorphism by the five lemma applied to cohomology. Independence of the stratification and coefficient rectangles are the explicit obligations ‘OBL-A4-VD-02‘ and ‘OBL-A4-IH-04‘. □

Definition 11.12.383 (Ordinary pullback–pushforward adjunction data). Lean: `conditional-interface:def:weil-a4-ordinary-adjunction-data`

Uses: `def:weil-a4-coefficient-index`

For a morphism $f : X \to Y$, the ordinary adjunction data are the A2 functors $Lf^* : D_c^b(Y, A) \to D_c^b(X, A)$ and $Rf_* : D_c^b(X, A) \to D_c^b(Y, A)$, their unit $\eta^*_{f,L} : L \to Rf_*Lf^*L$, their counit $\epsilon^*_{f,K} : Lf^*Rf_*K \to K$, and the two triangle identities.

Definition 11.12.384 (Mixed ordinary–compact tensor map). Lean: `conditional-interface:def:weil-a4-mixed-pushforward-tensor-map`

Uses: `def:weil-a4-ordinary-adjunction-data,def:weil-a4-projection-formula-comparison`

For $M, N \in D_c^b(X, A)$, define

$$\mu_{f,M,N} : Rf_*M \otimes^L_Y Rf_!N \longrightarrow Rf_!(M \otimes^L_X N)$$

as the composite

$$\begin{aligned} Rf_*M \otimes Rf_!N &\xrightarrow{\mathrm{sym}} Rf_!N \otimes Rf_*M \\ &\xrightarrow{\pi_{f,N,Rf_*M}} Rf_!\big(N \otimes Lf^*Rf_*M\big) \\ &\xrightarrow{Rf_!(\mathrm{id}\otimes\epsilon^*_{f,M})} Rf_!(N \otimes M) \xrightarrow{Rf_!(\mathrm{sym})} Rf_!(M \otimes N). \end{aligned}$$

Definition 11.12.385 (Verdier exchange comparison). Lean: `conditional-interface:def:weil-a4-verdier-exchange-comparison`

Uses: `def:weil-a4-verdier-dual-functor,def:weil-a4-mixed-pushforward-tensor-map,def:weil-a4-exceptional-composition-inverse,def:weil-a4-exceptional-trace`

For $f : X \to Y$, define

$$\chi_f(K) : Rf_*\mathbb{D}_X K \longrightarrow \mathbb{D}_Y Rf_!K$$

as the tensor–Hom adjoint of

$$\begin{aligned} Rf_*\mathbb{D}_X K \otimes Rf_!K &\xrightarrow{\mu_{f,\mathbb{D}_X K,K}} Rf_!(\mathbb{D}_X K \otimes K) \xrightarrow{Rf_!(\mathrm{ev})} Rf_!\omega_X \\ &\xrightarrow{Rf_!(d_{a_Y,f,A})} Rf_!Rf^!\omega_Y \xrightarrow{\mathrm{Tr}_{f,\omega_Y}} \omega_Y, \end{aligned}$$

where $d_{a_Y,f,A} : \omega_X = R(a_Y f)^! A \to Rf^!\omega_Y$ is the previously constructed exceptional-composition inverse. For a proper f, identify $Rf_! = Rf_*$ only after the proper-support comparison below has been constructed.

Definition 11.12.386 (Proper support comparison). Lean: `conditional-interface:def:weil-a4-proper-support-comparison`

Uses: `def:weil-a4-exceptional-index,def:weil-a4-ordinary-adjunction-data`

For proper f, the identity factorization $X \xrightarrow{\mathrm{id}_X} X \xrightarrow{f} Y$ is an admissible A1 compactification. Define

$$\sigma_f : Rf_! \longrightarrow Rf_*$$

as the A1 compactification-independence map from the selected model of $Rf_!$ to this identity model, followed by the equality of the identity model with Rf_*. Define σ_f^{-1} using the comparison in the opposite direction.

Lemma 11.12.387 (Proper support comparison inverse equations). Lean: `conditional-interface:lemma:weil-a4-proper-support-comparison-inverse`

Uses: `def:weil-a4-proper-support-comparison`

OPEN. *The two maps σ_f, σ_f^{-1} are inverse.*

Open obligation: `OBL-A4-VD-04`

Proof. Uses: `def:weil-a4-proper-support-comparison`

Both composites are the two A1 compactification-independence cocycles between the selected compactification and the identity compactification. Proving that those cocycles reduce to the identity, including derived-resolution independence, is ‘OBL-A4-VD-04‘. □

Proposition 11.12.388 (Proper pushforward equals compact-support pushforward). Lean: `conditional-interface:prop:weil-a4-proper-pushforward-equality-open`

Uses: `def:weil-a4-proper-support-comparison`

OPEN. *If f is proper, $\sigma_f : Rf_! \xrightarrow{\sim} Rf_*$.*

Open obligation: `OBL-A4-VD-04`

Proof. Uses: `lemma:weil-a4-proper-support-comparison-inverse`

The preceding inverse equations exhibit σ_f^{-1} as a two-sided inverse. Hence σ_f is an isomorphism. □

Definition 11.12.389 (Proper Verdier exchange map). Lean: `conditional-interface:def:weil-a4-proper-verdier-exchange-map`

Uses: `def:weil-a4-verdier-exchange-comparison,def:weil-a4-proper-support-comparison`

For proper f, define

$$\chi_f^{\mathrm{pr}}(K) : Rf_*\mathbb{D}_X K \xrightarrow{\sigma_f^{-1}} Rf_!\mathbb{D}_X K \xrightarrow{\chi_f(K)} \mathbb{D}_Y Rf_! K \xrightarrow{\mathbb{D}_Y(\sigma_f^{-1})} \mathbb{D}_Y Rf_* K.$$

Proposition 11.12.390 (Proper Verdier duality exchange). Lean: `conditional-interface:prop:weil-a4-proper-verdier-exchange-open`

Uses: `def:weil-a4-proper-verdier-exchange-map,def:weil-a4-verdier-exchange-comparison`

OPEN. *The proper exchange map is an isomorphism, natural in K, and compatible with proper base change.*

Open obligation: `OBL-A4-VD-03`

Open obligation: `OBL-A4-VD-05`

Proof. Uses: `def:weil-a4-verdier-exchange-comparison,prop:weil-a4-proper-pushforward-equality-open,prop:weil-a4-projection-formula-open,prop:weil-a4-verdier-biduality-open`

Construct the inverse by reversing the adjunction and projection maps. The two composites become identities after applying the bidual Hom comparison and the projection inverse. Naturalities are equalities of the same adjoint maps. The proper-base-change rectangle is reduced to A3 on each finite coefficient level and then to the adic reconstruction; this is exactly 'OBL-A4-VD-03' plus 'OBL-A4-VD-05'. □

11.12.9 Poincare pairing and Frobenius similitude

Definition 11.12.391 (Smooth proper pure-dimensional index). Lean: `conditional-interface:def:weil-a4-smooth-proper-pure-index`

Uses: `def:weil-a4-smooth-relative-dimension,def:weil-a4-exceptional-index`

Fix X/k smooth, proper, geometrically connected, of pure dimension d. Fix an algebraic closure $\bar{k}/k$, put $\bar{X} = X \times_k \bar{k}$, and write $\bar{a} : \bar{X} \to \operatorname{Spec} \bar{k}$. For a coefficient object K on X, write $\bar{K}$ for its pullback to $\bar{X}$, and in this subsection abbreviate

$$H^i(X, K) := H^i(\bar{X}, \bar{K}).$$

Use the smooth-purity comparison to identify

$$\omega_{\bar{X},A} = \bar{a}^! A \simeq A_{\bar{X}}(d)[2d].$$

The identification is not assumed; it is the output of the purity and structural-map comparisons below.

Definition 11.12.392 (Top-degree trace target). Lean: `conditional-interface:def:weil-a4-top-degree-trace-target`

Uses: `def:weil-a4-smooth-proper-pure-index,def:weil-a4-tate-twist`

Define first the twisted trace in cohomological degree $2d$ by

$$\mathrm{Tr}_X^{\mathrm{tw}} : H^{2d}(\bar{X}, A_{\bar{X}}(d)) \longrightarrow A$$

as the degree-$2d$ component of the counit $R\bar{a}_!\bar{a}^! A \to A$ after substituting $\bar{a}^! A \simeq A_{\bar{X}}(d)[2d]$. Define the untwisted trace

$$\mathrm{Tr}_X : H^{2d}(\bar{X}, A_{\bar{X}}) \longrightarrow A(-d)$$

by tensoring $\mathrm{Tr}_X^{\mathrm{tw}}$ with $A(-d)$ and using $A_{\bar{X}}(d) \otimes A_{\bar{X}}(-d) \simeq A_{\bar{X}}$.

Lemma 11.12.393 (Top-degree trace is degree preserving). Lean: `conditional-interface:lemma:weil-a4-top-degree-trace-degree`

Uses: `def:weil-a4-top-degree-trace-target`

The twisted trace receives only $H^{2d}(\bar{X}, A_{\bar{X}}(d))$, and the untwisted trace receives only $H^{2d}(\bar{X}, A_{\bar{X}})$; all other degrees map to zero in the corresponding degree-zero coefficient object.

Proof. Uses: `def:weil-a4-top-degree-trace-target`

The shift $[2d]$ identifies the degree-zero component of $R\bar{a}_!\bar{a}^!A$ with the degree-$2d$ component of $R\bar{a}_!A_{\bar{X}}(d)$. Taking cohomology in degree zero therefore gives exactly the twisted trace; tensoring with $A(-d)$ gives the untwisted trace. A degree $i \neq 2d$ class lands in $H^{i-2d}(\operatorname{Spec}\bar{k}, A) = 0$ in the corresponding graded component. □

Definition 11.12.394 (Degreewise cup component). Lean: `conditional-interface:def:weil-a4-degreewise-cup-component`

Uses: `def:weil-a4-smooth-proper-pure-index`, `def:weil-a2-completed-tensor`

For $x \in H^i(X, K)$ and $y \in H^{2d-i}(X, L)$, define

$$x \smile y \in H^{2d}(X, K \otimes_X^L L)$$

by the derived tensor product followed by the cohomology multiplication map. For homogeneous classes the order is x then y, and the Koszul sign is $(-1)^{i(2d-i)}$ when the order is reversed.

Lemma 11.12.395 (Cup component is bilinear). Lean: `conditional-interface:lemma:weil-a4-degreewise-cup-bilinear`

Uses: `def:weil-a4-degreewise-cup-component`

The degreewise cup component is additive and A-linear in each argument.

Proof. Uses: `def:weil-a4-degreewise-cup-component`

The derived tensor product is biadditive on cochain representatives. The cohomology multiplication map is linear, so passing to cohomology preserves addition and scalar multiplication in both entries. □

Definition 11.12.396 (Compact Poincare pairing). Lean: `conditional-interface:def:weil-a4-compact-poincare-pairing`

Uses: `def:weil-a4-degreewise-cup-component`, `def:weil-a4-top-degree-trace-target`

For $K \in D_c^b(X, A)$, put $K^\vee := R\mathcal{H}om_{\bar{X}}(\bar{K}, A_{\bar{X}})$ and define

$$\langle -, - \rangle_{X,i,K} : H^i(X, K) \times H^{2d-i}(X, K^\vee) \longrightarrow A(-d)$$

as cup product, followed by the cohomology map induced by evaluation $K \otimes^L K^\vee \to A_X$, followed by the untwisted trace:

$$\langle x, y \rangle := \operatorname{Tr}_X(x \smile y).$$

Lemma 11.12.397 (Pairing is graded symmetric). Lean: `conditional-interface:lemma:weil-a4-compact-poincare-graded-symmetry`

Uses: `def:weil-a4-compact-poincare-pairing`, `def:weil-a4-degreewise-cup-component`

For homogeneous $x \in H^i(X, A_X)$ and $y \in H^{2d-i}(X, A_X)$,

$$\langle x, y \rangle = (-1)^{i(2d-i)} \langle y, x \rangle$$

after interchanging the tensor factors and the coefficient twists.

Proof. Uses: `def:weil-a4-compact-poincare-pairing`, `def:weil-a4-degreewise-cup-component`

The symmetry constraint of the derived tensor product contributes the Koszul sign $(-1)^{i(2d-i)}$. The trace is a morphism out of the tensor product, so applying it preserves this equality. □

Definition 11.12.398 (Global Verdier-to-cohomology comparison). Lean: `conditional-interface:def:weil-a4-global-verdier-cohomology-comparison`

Uses: `def:weil-a4-compact-poincare-pairing`, `def:weil-a4-verdier-dual-functor`, `def:weil-a4-smooth-proper-pure-index`

For every i, define

$$\kappa_i : H^{2d-i}(X, K^\vee) \longrightarrow \operatorname{Hom}_A(H^i(X, K), A(-d))$$

by sending y to the functional $x \mapsto \operatorname{Tr}_X(x \smile y)$.

Definition 11.12.399 (Cohomological inverse candidate). Lean: `conditional-interface:def:weil-a4-cohomological-inverse-candidate`

Uses: `def:weil-a4-global-verdier-cohomology-comparison`, `def:weil-a4-verdier-bidual-map`

Define λ_i by taking the degree-i cohomology component of the Verdier bidual inverse

$$R\Gamma(\bar{X}, \mathbb{D}_{\bar{X}}\bar{K}) \longrightarrow R\operatorname{Hom}_A(R\Gamma(\bar{X}, \bar{K}), A)$$

and then identifying $\mathbb{D}_{\bar{X}}\bar{K}$ with $K^\vee(d)[2d]$ by smooth purity and tensoring the resulting degreewise map with $A(-d)$.

Lemma 11.12.400 (Inverse candidate is adjoint to the cup pairing). Lean: *`conditional-interface:lemma:weil-a4-cohomological-inverse-adjoint`*

Uses: *`def:weil-a4-cohomological-inverse-candidate`*, *`def:weil-a4-compact-poincare-pairing`*

The composite obtained by evaluating $\lambda_i(y)$ at x equals $\langle x, y\rangle$.

Proof. Uses: `def:weil-a4-cohomological-inverse-candidate`, `def:weil-a4-compact-poincare-pairing`, `def:weil-a4-top-degree-trace-target`

By construction, the global Verdier map is adjoint to evaluation into $\omega_{\bar{X}}$. Replacing $\omega_{\bar{X}}$ by $A_{\bar{X}}(d)[2d]$, tensoring by $A(-d)$, and taking the degree-zero component is exactly cup product followed by the untwisted top trace. Thus the resulting scalar is the defined pairing. □

Proposition 11.12.401 (Poincare pairing is perfect). Lean: *`conditional-interface:prop:weil-a4-poincare-pairing-perfect-open`*

Uses: *`def:weil-a4-compact-poincare-pairing`*, *`def:weil-a4-global-verdier-cohomology-comparison`*, *`def:weil-a4-cohomological-inverse-candidate`*

OPEN. *The map κ_i is an isomorphism with inverse λ_i, for every i. In particular the pairing is perfect and the cohomology groups are finite-dimensional over A after rationalizing.*

Open obligation: `OBL-A4-PD-01`

Open obligation: `OBL-A4-PD-02`

Open obligation: `OBL-A4-PD-03`

Proof. Uses: `lemma:weil-a4-cohomological-inverse-adjoint`, `prop:weil-a4-verdier-biduality-open`, `def:weil-a4-cohomological-inverse-candidate`

The global Verdier bidual isomorphism gives mutually inverse maps before taking cohomology. The adjointness lemma identifies those maps with κ_i and λ_i. Finite-dimensionality and passage from the complex-level inverse to degreewise inverse require the explicit global finiteness and totalization calculations recorded in 'OBL-A4-PD-01'– 'OBL-A4-PD-03'. □

Definition 11.12.402 (Frobenius action on cohomology). Lean: `conditional-interface:def:weil-a4-frobenius-cohomology-action`

Uses: `def:weil-a4-geometric-frobenius-convention`, `def:weil-a4-compact-poincare-pairing`

Let $\Phi_X : \bar{X} \to \bar{X}$ be geometric Frobenius. For a descended coefficient object K, define

$$\Phi_{H^i(X,K)} := H^i(\Phi_X^*) : H^i(X, K) \to H^i(X, K).$$

For a Weil object, include the chosen descent isomorphism $\Phi_X^* K \simeq K$ in this formula.

Lemma 11.12.403 (Frobenius commutes with cup product). Lean: *`conditional-interface:lemma:weil-a4-frobenius-cup-compatibility`*

Uses: *`def:weil-a4-frobenius-cohomology-action`*, *`def:weil-a4-degreewise-cup-component`*

For x, y as in the pairing,

$$\Phi_X(x \smile y) = \Phi_X(x) \smile \Phi_X(y).$$

Proof. Uses: `def:weil-a4-frobenius-cohomology-action`, `def:weil-a4-degreewise-cup-component`

Pullback is monoidal, so the square comparing the two tensor products commutes on representatives. Taking cohomology gives the displayed equality. □

Definition 11.12.404 (Frobenius trace-similitude predicate). Lean: `conditional-interface:def:weil-a4-frobenius-trace-similitude-predicate`

Uses: `def:weil-a4-frobenius-cohomology-action`, `def:weil-a4-top-degree-trace-target`

Define FrobTrace$_d$ to be the assertion

$$\operatorname{Tr}_X(\Phi_X z) = q^d \operatorname{Tr}_X(z) \qquad (z \in H^{2d}(X, A_X)).$$

Lemma 11.12.405 (Frobenius trace multiplier). *Lean:* `conditional-interface:lemma:weil-a4-frobenius-trace-multiplier-open`

Uses: `def:weil-a4-frobenius-trace-similitude-predicate`, `def:weil-a4-tate-twist`

OPEN. FrobTrace$_d$ *holds.*

Open obligation: `OBL-A4-PD-05`

Proof. Uses: `def:weil-a4-frobenius-trace-similitude-predicate`,`lemma:weil-a4-tate-frobenius-scalar`

The structural trace is Frobenius equivariant as a map to the coefficient unit. Under the smooth-purity identification the source contributes the twist $A_X(d)$, while the shift contributes no scalar. Passing to the untwisted trace tensors the target by $A(-d)$, on which geometric Frobenius acts by q^d. Proving that the structural trace identification and the Frobenius descent square are the same map is the precise obligation 'OBL-A4-PD-05'. □

Lemma 11.12.406 (Frobenius similitude of the Poincare pairing). *Lean:* `conditional-interface:lemma:weil-a4-frobenius-poincare-similitude-open`

Uses: `def:weil-a4-compact-poincare-pairing`, `def:weil-a4-frobenius-cohomology-action`

OPEN. *For $x \in H^i(X,K)$ and $y \in H^{2d-i}(X,K^\vee)$,*

$$\langle \Phi_X x, \Phi_X y\rangle = q^d \langle x, y\rangle.$$

Open obligation: `OBL-A4-PD-04`

Open obligation: `OBL-A4-PD-05`

Proof. Uses: `def:weil-a4-compact-poincare-pairing`,`lemma:weil-a4-frobenius-cup-compatibility`,`lemma:weil-a4-frobenius-trace-multiplier-open`

Use cup compatibility to replace the left side by $\mathrm{Tr}_X(\Phi_X(x \smile y))$, then apply the trace multiplier lemma. The Frobenius descent map on $K^\vee$ must be the dual of the descent map on K; that comparison is 'OBL-A4-PD-04'. □

Proposition 11.12.407 (Reciprocal Frobenius eigenvalues). *Lean:* `conditional-interface:prop:weil-a4-frobenius-reciprocal-eigenvalues-open`

Uses: `def:weil-a4-compact-poincare-pairing`, `def:weil-a4-frobenius-cohomology-action`, `def:weil-a4-global-verdier-cohomology-comparison`

OPEN. *Let $A = E$ be a characteristic-zero coefficient field and let $K = A_X$. Under the Poincare comparison*

$$H^{2d-i}(X,E) \simeq H^i(X,E)^\vee(-d),$$

geometric Frobenius on the left is carried to $q^d(\Phi_{H^i}^{-1})^\vee$. Hence, with algebraic multiplicity, an eigenvalue α on $H^i(X,E)$ is paired with q^d/α on $H^{2d-i}(X,E)$, and

$$\det(1 - T\Phi \mid H^{2d-i}(X,E)) = \prod_{\alpha \in \mathrm{Eig}(H^i)} (1 - q^d \alpha^{-1} T).$$

Open obligation: `OBL-A4-PD-01`

Open obligation: `OBL-A4-PD-02`

Open obligation: `OBL-A4-PD-04`

Open obligation: `OBL-A4-PD-05`

Proof. Uses: `prop:weil-a4-poincare-pairing-perfect-open`,`lemma:weil-a4-frobenius-poincare-similitude-open`

For $y \in H^{2d-i}(X,E)$, let $\varphi_y(x) = \langle x, y\rangle$. Substituting $x = \Phi z$ in the similitude equation gives

$$\varphi_{\Phi y}(\Phi z) = q^d \varphi_y(z), \qquad \varphi_{\Phi y} = q^d \varphi_y \circ \Phi^{-1}.$$

Perfectness identifies $y \mapsto \varphi_y$ with the displayed dual space, so the conjugated operator is $q^d(\Phi^{-1})^\vee$. Extending scalars to an algebraic closure and triangularizing Φ gives the reciprocal eigenvalue multiset and the determinant formula. The statement inherits exactly the four listed Poincare/Frobenius obligations. □

11.12.10 Coefficient and base-change compatibility rectangles

Definition 11.12.408 (Proper base-change square for duality)**.** Lean: `conditional-interface:def:weil-a4-duality-proper-base-change-square`

Uses: `def:weil-a4-exceptional-index,def:weil-a4-proper-support-comparison`

For a Cartesian square

$$\begin{array}{ccc} X' & \xrightarrow{g'} & X \\ {\scriptstyle f'}\downarrow & & \downarrow{\scriptstyle f} \\ Y' & \xrightarrow[g]{} & Y \end{array}$$

with f proper, define the duality comparison rectangle to have top edge the A3 map $g^{*}Rf_{*} \to Rf'_{*}g'^{*}$, bottom edge the corresponding map for $Rf_{!}$, and vertical edges the proper Verdier exchange maps.

Lemma 11.12.409 (Proper duality base-change rectangle)**.** Lean: `conditional-interface:lemma:weil-a4-proper-duality-base-change-rectangle-open`

Uses: `def:weil-a4-duality-proper-base-change-square,def:weil-a4-proper-verdier-exchange-map`

OPEN. *The proper duality rectangle commutes at every finite coefficient level and after adic completion.*

Open obligation: `OBL-A4-VD-05`

Proof. Uses: `def:weil-a4-duality-proper-base-change-square,prop:weil-a4-proper-verdier-exchange-open,prop:weil-a2-proper-complete-base-change-open`

Expand both routes into evaluation, projection, trace, and the A3 base-change map. Proper base change commutes with tensor and trace at finite level, so the routes agree there. The adic equality follows only after proving that the A2 derived-completion comparison preserves this rectangle; this is the remaining clause of 'OBL-A4-VD-05'. □

Definition 11.12.410 (Reduction rectangle for exceptional pullback)**.** Lean: `conditional-interface:def:weil-a4-exceptional-reduction-rectangle`

Uses: `def:weil-a4-adic-exceptional-pullback,def:weil-a4-finite-exceptional-pullback-object,def:weil-a2-finite-reduction`

For every n, define the square

$$\begin{array}{ccc} Rf^{!}_{\Lambda}L \otimes^{L} \Lambda_n & \longrightarrow & Rf^{!}_{n}(L_n) \\ \downarrow & & \downarrow \\ Rf^{!}_{\Lambda}(L') \otimes^{L} \Lambda_n & \longrightarrow & Rf^{!}_{n}(L'_n) \end{array}$$

for a morphism $L \to L'$, with horizontal arrows the reduction comparisons and vertical arrows induced by $f^{!}$.

Lemma 11.12.411 (Exceptional reduction rectangle commutes)**.** Lean: `conditional-interface:lemma:weil-a4-exceptional-reduction-rectangle-open`

Uses: `def:weil-a4-exceptional-reduction-rectangle,def:weil-a4-adic-exceptional-pullback`

OPEN. *The reduction rectangle commutes and its horizontal maps are isomorphisms.*

Open obligation: `OBL-A4-AD-01`

Open obligation: `OBL-A4-AD-03`

Proof. Uses: `def:weil-a4-exceptional-reduction-rectangle,lemma:weil-a4-exceptional-level-transition-composition,prop:weil-a4-adic-exceptional-reduction-open`

Commutativity follows by projecting the inverse-limit cone to level n; both routes are the same level transition followed by $Rf^{!}_{n}$ on the morphism. Isomorphism of the horizontal maps is the adic reduction proposition. Compatibility of the transition maps is 'OBL-A4-AD-01'. □

Definition 11.12.412 (Scalar-extension rectangle for exceptional pullback)**.** Lean: `conditional-interface:def:weil-a4-exceptional-scalar-rectangle`

Uses: `def:weil-a4-scalar-exceptional-pullback,def:weil-a4-adic-exceptional-pullback,def:weil-a2-integral-scalar-extension`

For a finite extension $E/\mathbf{Q}_{\ell}$, define the square comparing $Rf^{!}_{\Lambda}$ followed by tensor with E to $Rf^{!}_{E}$. For $\overline{\mathbf{Q}}_{\ell}$, define the analogous square at each finite stage and take the filtered colimit.

Lemma 11.12.413 (Scalar-extension rectangle commutes). *Lean:* `conditional-interface: lemma: weil-a4-exceptional-scalar-rectangle-open`

Uses: `def: weil-a4-exceptional-scalar-rectangle, def: weil-a4-scalar-exceptional-pullback`

OPEN. *The scalar-extension square commutes, is natural in f, and agrees with finite reduction.*

Open obligation: `OBL-A4-AD-05`

Proof. Uses: `def:weil-a4-exceptional-scalar-rectangle,prop:weil-a4-scalar-exceptional-lattice-independent-open`

At a common finite coefficient stage, both paths are the unique map whose reduced counit is the scalar extension of the finite-level counit. Uniqueness of the representing map gives equality. Passing through the inverse limit and filtered colimit requires the lattice-independence and continuity part of 'OBL-A4-AD-05'. □

Proposition 11.12.414 (A4 conditional duality endpoint). *Lean:* `conditional-interface: prop: weil-a4-duality-extraordinary-endpoint-open`

Uses: `def: weil-a4-verdier-dual-functor, def: weil-a4-smooth-purity-comparison, def: weil-a4-compact-poincare-pairing`

OPEN. *After discharging the explicitly listed A4 obligations, the constructions above provide $Rf^!$, its adjunction to $Rf_!$, trace, projection formula, smooth purity, proper Verdier exchange, Poincare pairing, perfectness, Frobenius similitude, reciprocal Frobenius eigenvalue pairing, and their A1–A3 coefficient/base-change rectangles.*

Proof. Uses: `prop:weil-a4-adic-exceptional-adjunction-open,prop:weil-a4-exceptional-composition-isomorphism-open,prop:weil-a4-projection-formula-open,prop:weil-a4-smooth-purity-all-coefficients-open,prop:weil-a4-proper-verdier-exchange-open,prop:weil-a4-poincare-pairing-perfect-open,lemma:weil-a4-frobenius-poincare-similitude-open,prop:weil-a4-frobenius-reciprocal-eigenvalues-open,lemma:weil-a4-proper-duality-base-change-rectangle-open,lemma:weil-a4-exceptional-scalar-rectangle-open`

This is a dependency assembly, not a claim that the obligations have been proved. Each listed component supplies exactly one named part of the endpoint; no component is replaced by the words "by duality" or "by six functors". □

Chapter 12

A5. Smooth base change and compact-support transport

Status. This fragment is downstream of the finite-torsion derived category and the A3 proper-base-change interface. Stacks Project locators are source locators only. No statement below treats a Stacks theorem or a six-functor package as a foundation leaf. Every step whose proof is not reduced to a previous label is recorded as an explicit OPEN obligation.

12.1 A5.1. Smooth squares and coefficient data

Definition 12.1.1 (Finite-torsion smooth Cartesian square)**.** Lean: `conditional-interface:def:weil-a5-smooth-cartesian-square`

Uses: `def:weil-pbc-cartesian-square`

Let Λ be a finite commutative ring annihilated by an integer invertible on the schemes below. A smooth Cartesian square is a tuple

$$\begin{array}{ccc} X' & \xrightarrow{g'} & X \\ {\scriptstyle f'}\downarrow & & \downarrow{\scriptstyle f} \\ Y' & \xrightarrow{g} & Y \end{array} \qquad \square = (f, f', g, g', \alpha)$$

as in Definition 11.1.1, together with the assertion that g is smooth. The displayed identification α is part of the datum, so all exchange maps below use the same pullback identification.

Definition 12.1.2 (Finite-torsion constructible input)**.** Lean: `conditional-interface:def:weil-a5-constructible-input`

Uses: `def:weil-ftdc-finite-torsion-coefficient`, `def:weil-ftdc-constructible-derived-category`

For a noetherian scheme T write

$$\mathsf{C}_\Lambda(T) = D^b_c(T_{\mathrm{proet}}, \Lambda)$$

for the finite-torsion constructible derived category already constructed in the A2 module. An input package is $(T, K, [a, b])$ with $K \in \mathsf{C}_\Lambda(T)$ and $H^i(K) = 0$ for $i \notin [a, b]$.

Definition 12.1.3 (Smooth exchange map in degree q)**.** Lean: `conditional-interface:def:weil-a5-smooth-exchange-degree`

Uses: `def:weil-a5-smooth-cartesian-square`, `def:weil-a5-constructible-input`

For a sheaf F on X_{proet} and a smooth square, define

$$\mathrm{Ex}^q_{\square, F} : g^* R^q f_* F \longrightarrow R^q f'_* g'^* F$$

to be the qth cohomology sheaf of the derived exchange morphism obtained from the unit of $g^* \dashv g_*$, the chosen pullback square, and the counit of $g'^* \dashv g'_*$. For a complex K, use the same notation for the derived morphism $Lg^* Rf_* K \to Rf'_* Lg'^* K$.

Lemma 12.1.4 (Pullback preserves the chosen constructible bounds)**.** Lean: `conditional-interface:lemma:weil-a5-pullback-bounded-constructible`

Uses: def:weil-a5-constructible-input

If $(X, K, [a, b])$ is an input package and $g' : X' \to X$ is any morphism, then $Lg'^{*}K$ belongs to $D^b_c(X'_{\mathrm{proet}}, \Lambda)$ and has vanishing cohomology outside $[a, b]$.

Proof. Uses: def:weil-a5-constructible-input, lemma:weil-ftdc-derived-pullback-constructible

Represent K by a bounded complex of constructible sheaves in degrees $[a, b]$. The A2 pullback is exact on the heart and preserves each constructible term by the cited lemma. The termwise pulled-back complex has the same degree interval; its cohomology is constructible by the A2 cone and truncation closure lemmas. It represents $Lg'^{*}K$ by the termwise derived pullback definition. □

12.2 A5.2. Audit repair for finite-level etale/pro-etale comparison

Definition 12.2.1 (Finite-lisse pro-etale descent datum)**.** Lean: conditional-interface:def:weil-a5-finite-lisse-descent-datum

Uses: def:weil-ftdc-etale-proetale-morphism, def:weil-ftdc-finite-locally-constant

Let $\epsilon_X : X_{\mathrm{proet}} \to X_{\mathrm{et}}$ be the comparison geometric morphism. A finite-lisse descent datum for a finite locally constant pro-etale Λ-sheaf L consists of a finite jointly surjective pro-etale cover $(U_a \to X)_a$, finite etale maps $V_a \to X$, factorizations $U_a \to V_a \to X$, constant finite Λ-modules M_a on V_a, isomorphisms $L|_{U_a} \simeq M_a|_{U_a}$, and transition isomorphisms on all pairwise overlaps satisfying identity, inverse, and triple-overlap cocycle equations.

Proposition 12.2.2 (Finite-lisse descent is an explicit open leaf)**.** Lean: conditional-interface:prop:weil-a5-finite-lisse-descent-open

Uses: def:weil-a5-finite-lisse-descent-datum

Every finite locally constant pro-etale Λ-sheaf admits a finite-lisse descent datum, and the glued etale sheaf is independent of the selected datum, is the assertion

Open obligation: OBL-A5-FINITE-LISSE-PROETALE-DESCENT

.

Proof. Uses: def:weil-a5-finite-lisse-descent-datum

The datum separates cover selection, finite-stage factorization, overlap maps, cocycle, gluing, and choice independence. None of these existence assertions is a mathlib leaf or a proved A1–A3 label. Stacks Project, Lemma 61.19.2 (Tag 09AJ) is the source locator, but is not imported as a theorem; the entire displayed assertion therefore remains one named external proof obligation. □

Definition 12.2.3 (Hom comparison for finite-lisse sheaves)**.** Lean: conditional-interface:def:weil-a5-epsilon-hom-comparison

Uses: def:weil-ftdc-etale-proetale-morphism, def:weil-ftdc-finite-locally-constant

For finite locally constant etale sheaves A, B, define

$$\eta_{A,B} : \mathrm{Hom}_{X_{\mathrm{et}}}(A, B) \longrightarrow \mathrm{Hom}_{X_{\mathrm{proet}}}(\epsilon_X^{-1}A, \epsilon_X^{-1}B)$$

by applying the inverse-image functor to a morphism.

Proposition 12.2.4 (Finite-lisse full faithfulness leaf)**.** Lean: conditional-interface:prop:weil-a5-epsilon-finite-lisse-full-faithfulness-open

Uses: def:weil-a5-epsilon-hom-comparison

The assertion that $\eta_{A,B}$ is bijective for every pair (A, B), with the inverse compatible with composition and identities, is

Open obligation: OBL-A5-EPSILON-FINITE-LISSE-FULL-FAITHFULNESS

.

Proof. Uses: def:weil-a5-epsilon-hom-comparison, prop:weil-a5-finite-lisse-descent-open

Faithfulness reduces to equality after a jointly surjective finite-lisse cover. Fullness requires descending a pro-etale morphism to finite etale trivializations and checking the pairwise and triple-overlap equations. Those steps are consequences of the preceding descent obligation but are not definition reductions; they are therefore retained as a separate leaf rather than hidden in 'epsilon is fully faithful'. □

Definition 12.2.5 (Open and closed comparison arrows). Lean: `conditional-interface:def:weil-a5-epsilon-open-closed-comparison`

Uses: `def:weil-ftdc-etale-proetale-morphism`, `def:weil-a1-extension-zero-sheaf`

For a complementary open–closed pair $j : U \hookrightarrow X$, $i : Z \hookrightarrow X$, define the natural arrows

$$\epsilon_X^{-1} j_! A \longrightarrow j_! \epsilon_U^{-1} A, \qquad \epsilon_X^{-1} i_* B \longrightarrow i_* \epsilon_Z^{-1} B$$

by the unit–counit formulas and the identity identifications of the base-changed open and closed subspaces.

Proposition 12.2.6 (Open comparison leaf). Lean: *conditional-interface:prop:weil-a5-epsilon-open-comparison-open*

Uses: *def:weil-a5-epsilon-open-closed-comparison*

The first comparison arrow is an isomorphism for every constructible finite torsion A, naturally in A and compatibly with composition of open immersions, is

Open obligation: *OBL-A5-EPSILON-JSHRIEK-COMPARISON*

.

Proof. Uses: `def:weil-a5-epsilon-open-closed-comparison`

On the open locus both sides restrict to $\epsilon_U^{-1} A$, and on the closed complement both have zero stalks. Turning this stalk calculation into an isomorphism uses exactness and enough geometric points for the selected pro-etale module-sheaf category, precisely the unclosed A1 stalk criterion. Thus the remaining input is recorded rather than silently assumed. □

Proposition 12.2.7 (Closed comparison leaf). Lean: *conditional-interface:prop:weil-a5-epsilon-closed-comparison-open*

Uses: *def:weil-a5-epsilon-open-closed-comparison*

The second comparison arrow is an isomorphism for every constructible finite torsion B, naturally in B and compatibly with composition of closed immersions, is

Open obligation: *OBL-A5-EPSILON-ISTAR-COMPARISON*

.

Proof. Uses: `def:weil-a5-epsilon-open-closed-comparison`

Both sides have value B on objects supported on Z and zero off Z. The missing proof is the sheafification and geometric-stalk comparison in the chosen small pro-etale site, not an elementary equality in mathlib. It is therefore retained as the stated leaf. □

Lemma 12.2.8 (Constructible-heart comparison by stratum induction). Lean: *conditional-interface:lemma:weil-a5-epsilon-constructible-heart-induction*

Uses: *def:weil-a5-finite-lisse-descent-datum*, *def:weil-a5-epsilon-open-closed-comparison*, *def:weil-ftdc-constructible-sheaf*

Assuming the four preceding comparison obligations, inverse image along ϵ_X induces an equivalence between constructible finite-torsion etale and pro-etale sheaves.

Proof. Uses: `prop:weil-a5-finite-lisse-descent-open`, `prop:weil-a5-epsilon-finite-lisse-full-faithfulness-open`, `prop:weil-a5-epsilon-open-comparison-open`, `prop:weil-a5-epsilon-closed-comparison-open`

Induct on the number of strata. On one stratum, essential surjectivity is finite-lisse descent and full faithfulness is the Hom comparison. For more strata, choose one open stratum $j : U \hookrightarrow X$ and its closed complement $i : Z \hookrightarrow X$. The open–closed exact sequence expresses each constructible sheaf as an extension of $j_! j^* F$ by $i_* i^* F$. Induction descends the two restrictions; the two comparison arrows descend the outer terms. Full faithfulness identifies the extension class, and the five lemma identifies the glued middle term. Refining two choices to the intersection stratification and applying the same argument proves choice independence. □

Definition 12.2.9 (Derived comparison by cohomological amplitude). Lean: `conditional-interface:def:weil-a5-derived-site-comparison`

Uses: `def:weil-ftdc-constructible-derived-category`, `def:weil-ftdc-etale-proetale-morphism`

For $K \in D_c^b(X_{\mathrm{et}}, \Lambda)$, define $\epsilon_X^{-1} K$ termwise. Its amplitude is the least length of an interval $[a, b]$ containing all nonzero cohomology sheaves. The derived comparison functor is the localization of this termwise functor at quasi-isomorphisms.

Proposition 12.2.10 (Derived site comparison by amplitude induction). *Lean:* `conditional-interface:prop:weil-a5-derived-site-comparison-conditional`

Uses: `def:weil-a5-derived-site-comparison`

Assuming the four finite-heart obligations, the derived comparison functor is an equivalence on D_c^b, is exact, and commutes with truncations and pullbacks.

Proof. Uses: `def:weil-a5-derived-site-comparison,lemma:weil-a5-epsilon-constructible-heart-induction`

For amplitude zero this is the constructible-heart equivalence. For positive amplitude, use the truncation triangle

$$\tau_{\le b-1}K \to K \to \mathcal{H}^b(K)[-b] \to .$$

The first and third terms have smaller amplitude. Induction gives their preimages and the morphism defining the extension; full faithfulness on the smaller-amplitude terms gives a unique middle object. Applying the inverse image functor identifies its triangle with the original one. The same induction on morphism cones proves full faithfulness. Exactness, truncation compatibility, and pullback compatibility follow from the termwise formulas. No additional unbounded or enhancement argument is used. □

Proposition 12.2.11 (Promotion barrier for A3 and A5). *Lean:* `conditional-interface:prop:weil-a5-site-comparison-promotion-barrier`

Uses: `def:weil-a5-derived-site-comparison`

Until the four finite-heart obligations are discharged, neither A3's etale-to-pro-etale finite-level transport nor the pro-etale forms of the A5 smooth and compact-support base-change maps may be marked closed.

Proof. Uses: `prop:weil-a5-derived-site-comparison-conditional`

Each named transport conjugates an etale morphism by the derived comparison equivalence. Without essential surjectivity and full faithfulness, this conjugation is neither defined on every constructible pro-etale object nor independent of a selected etale representative. Hence the four obligations are genuine ancestors of every such transported output. □

12.3 A5.3. Local systems and the smooth stalk calculation

Definition 12.3.1 (Finite locally constant stratum datum). Lean: `conditional-interface:def:weil-a5-locally-constant-stratum`

Uses: `def:weil-ftdc-constructible-sheaf`

For a locally closed subscheme $j : U \hookrightarrow X$, a stratum datum for a sheaf F is a finite locally constant Λ-module sheaf L on U and an isomorphism $j^*F \simeq L$. A finite stratum datum is a finite list of such pairs whose underlying locally closed subsets partition X.

Lemma 12.3.2 (Smooth pullback of a finite locally constant sheaf). *Lean:* `conditional-interface:lemma:weil-a5-smooth-pullback-locally-constant`

Uses: `def:weil-a5-smooth-cartesian-square,def:weil-a5-locally-constant-stratum`

If L is finite locally constant on a stratum $U \subset X$, then its pullback to $U' = U \times_X X'$ is finite locally constant and the canonical map $g'^* j_* L \to j'_* g_U^* L$ is an isomorphism whenever $j : U \hookrightarrow X$ and $j' : U' \hookrightarrow X'$ are the corresponding locally closed immersions.

Proof. Uses: `def:weil-a5-smooth-cartesian-square,def:weil-a5-locally-constant-stratum`

Choose a finite etale cover $V \to U$ trivializing L. Its base change $V \times_X X' \to U'$ is finite etale and trivializes $g_U^* L$, proving the first assertion by the finite-cover definition of local constancy. On an etale object $W \to X'$, both sheaves are obtained by sections on $W \times_X U$; the Cartesian identification identifies these section sets. The restriction maps and descent equalizers are identical, hence the map is an isomorphism sectionwise. □

Definition 12.3.3 (Affine smooth base-change test). Lean: `conditional-interface:def:weil-a5-affine-smooth-test`

Uses: `def:weil-a5-smooth-cartesian-square`

For an affine smooth morphism $u : V \to S$, a quasi-compact and quasi-separated $h : T \to S$, and a finite-torsion sheaf F on T, form $T_V = T \times_S V$ and let $h_V : T_V \to V$. Define $\mathsf{AS}(u, h, F, q)$ to be the assertion that

$$u^{-1} R^q h_* F \longrightarrow R^q h_{V,*}(F|_{T_V})$$

is an isomorphism on V_{et}.

Lemma 12.3.4 (Finite affine smooth test reduction). *Lean:* `conditional-interface:lemma:weil-a5-finite-affine-smooth-test-reduction`

Uses: `def:weil-a5-affine-smooth-test`

The assertion $\mathsf{AS}(u, h, F, q)$ *is local on* V *for the etale topology, local on* S *for the Zariski topology, and compatible with composition of two smooth base-change morphisms.*

Proof. Uses: `def:weil-a5-affine-smooth-test`

For an etale cover $(V_a \to V)_a$, inverse image to V_a commutes with the localization morphisms of the two sites, so the restriction of the exchange map is the exchange map for V_a. Isomorphy is local because geometric stalks are conservative. The same calculation after a Zariski restriction of S proves target locality. For $V_2 \to V_1 \to S$, expand the three unit–counit exchange maps: the exchange for the composite is the composite of the two exchanges, because the two intervening adjunction unit–counit pairs cancel by the triangle identities. □

Lemma 12.3.5 (Smooth strict-henselian stalk comparison). *Lean:* `conditional-interface:lemma:weil-a5-smooth-strict-henselian-stalk`

Uses: `def:weil-a5-smooth-cartesian-square`

Let $\bar{y}'$ *be a geometric point of* Y' *and let* $\bar{y}$ *be its image in* Y*. For a finite locally constant* L *on* X*, the stalk of* $\mathrm{Ex}^q_{\square,L}$ *at* $\bar{y}'$ *is the map*

$$H^q(X^{\mathrm{sh}}_{\bar{y}}, L) \longrightarrow H^q(X'^{\mathrm{sh}}_{\bar{y}'}, L|).$$

Proof. Uses: `def:weil-a5-smooth-cartesian-square,prop:weil-pbc-strict-henselization-stalk-formula`

Take the filtered colimit over pointed etale neighbourhoods of $\bar{y}'$. The A3 strict-henselization stalk formula identifies the stalk of $R^q f_* L$ with cohomology of the strict-henselian fibre. The Cartesian square identifies the corresponding fibre after base change, and the unit–counit definition of the exchange map is exactly restriction along this identification. Passing to the colimit gives the displayed map. □

Definition 12.3.6 (Universal affine-curve test datum). Lean: `conditional-interface:def:weil-a5-universal-affine-curve-test`

Uses: `def:weil-a5-affine-smooth-test`

A universal affine-curve test datum consists of a normal domain A whose fraction field L is algebraically closed, an extension of algebraically closed fields K/L, an affine smooth morphism $u : V \to \operatorname{Spec} A$ of relative dimension one, a finite constant coefficient sheaf $\mathbf{Z}/d\mathbf{Z}$ with d invertible in A, and the base change $V_K = V \times_A \operatorname{Spec} K$.

Proposition 12.3.7 (Etale-local standard-smooth presentation leaf). *Lean:* `conditional-interface:prop:weil-a5-standard-smooth-presentation-open`

Uses: `def:weil-a5-affine-smooth-test`

Every smooth base-change morphism is etale locally a morphism admitting an etale factorization through an affine space, with the factorization, relative dimension, Cartesian squares, and overlap equalities descending to one common neighbourhood, is

Open obligation: `OBL-A5-SBC-STANDARD-SMOOTH-PRESENTATION`

.

Proof. Uses: `def:weil-a5-affine-smooth-test`

This is the precise local-structure input used at the first reduction. The source locator is Stacks Project, Lemma 29.37.21; the project foundation contract contains only the smooth predicate and base-change stability, not this presentation theorem, so the leaf remains open. □

Lemma 12.3.8 (Reduction from affine space to relative dimension one). *Lean:* `conditional-interface:lemma:weil-a5-smooth-relative-dimension-one-reduction`

Uses: `def:weil-a5-affine-smooth-test`

Assume the standard-smooth presentation leaf. If smooth base change holds for affine smooth maps of relative dimension one, then it holds for every smooth base-change morphism.

Proof. Uses: `prop:weil-a5-standard-smooth-presentation-open,lemma:weil-a5-finite-affine-smooth-test-reduction`

Work on an etale neighbourhood where the smooth map factors etale over $\mathbf{A}^d_S$. The etale factor contributes an identity localization comparison. Factor $\mathbf{A}^d_S \to S$ into d successive projections with one-dimensional affine fibres. Composition compatibility from the preceding lemma composes the d one-dimensional exchange isomorphisms. Finally descend the local isomorphisms along the etale cover using geometric stalk conservativity. □

Proposition 12.3.9 (Universal-curve reduction leaf). *Lean:* `conditional-interface:prop:weil-a5-universal-curve-reduction-open`

Uses: `def:weil-a5-universal-affine-curve-test`

For finite torsion coefficients, the relative-dimension-one affine smooth base-change assertion reduces to vanishing of the sheaf associated to

$$U \longmapsto H^q(U_K, \mathbf{Z}/d\mathbf{Z})$$

for every universal affine-curve test datum and every $q > 0$, is

Open obligation: `OBL-A5-SBC-UNIVERSAL-CURVE-REDUCTION`

.

Proof. Uses: `def:weil-a5-universal-affine-curve-test`

The reduction includes the injective-sheaf degree induction, finite-type approximation, normalization, reduction to algebraically closed fraction fields, and the identification of $R^q h_*$ with the sheafification of the displayed presheaf. These are the separate results collected in Stacks Project, Lemma 59.88.7. They are not prior project labels, so the reduction is retained as an explicit leaf rather than abbreviated as 'standard'. □

Proposition 12.3.10 (Affine-curve higher vanishing leaf). *Lean:* `conditional-interface:prop:weil-a5-affine-curve-higher-vanishing-open`

Uses: `def:weil-a5-universal-affine-curve-test`

For a smooth affine curve C over an algebraically closed field of characteristic prime to d, $H^q(C, \mathbf{Z}/d\mathbf{Z}) = 0$ for every $q > 1$, is

Open obligation: `OBL-A5-SBC-AFFINE-CURVE-HIGHER-VANISHING`

.

Proof. Uses: `def:weil-a5-universal-affine-curve-test`

The source locator is Stacks Project, Theorem 59.83.10. This cohomological dimension theorem is absent from mathlib and A1–A3 and is not proved in the present bounded turn, so it remains exactly the displayed open leaf. □

Proposition 12.3.11 (Affine-curve H^1 field invariance leaf). *Lean:* `conditional-interface:prop:weil-a5-affine-curve-h1-invariance-open`

Uses: `def:weil-a5-universal-affine-curve-test`

For algebraically closed fields K/L and a smooth affine curve C/L, the pullback map

$$H^1(C, \mathbf{Z}/d\mathbf{Z}) \longrightarrow H^1(C_K, \mathbf{Z}/d\mathbf{Z})$$

is an isomorphism, is

Open obligation: `OBL-A5-SBC-AFFINE-CURVE-H1-INVARIANCE`

.

Proof. Uses: `def:weil-a5-universal-affine-curve-test`

The source locator is Stacks Project, Lemma 59.83.12. Its proof uses the classification of finite etale covers of smooth curves and is not admitted by the foundation contract; hence the assertion remains open. □

Proposition 12.3.12 (Finite-etale spreading leaf). *Lean:* `conditional-interface:prop:weil-a5-finite-etale-spreading-open`

Uses: `def:weil-a5-universal-affine-curve-test`

Every finite etale cover of the generic fibre of the universal affine-curve test descends, after an etale localization of the base, to a finite etale cover of the family, with its Galois action, selected section, and equality on the generic fibre, is

Open obligation: `OBL-A5-SBC-FINITE-ETALE-SPREADING`

.

Proof. Uses: `def:weil-a5-universal-affine-curve-test`

The source locators are Stacks Project, Lemmas 58.31.8 and 58.31.9. The statement lists the cover, localization, action, section and comparison separately; none is a current mathlib leaf, so the combined finite-presentation descent assertion is kept open. □

Lemma 12.3.13 (Universal affine-curve terminal vanishing). *Lean:* `conditional-interface:lemma:weil-a5-universal-affine-curve-vanishing`

Uses: `def:weil-a5-universal-affine-curve-test`

Assuming the three affine-curve leaves, every local section of the presheaf $U \mapsto H^q(U_K, \mathbf{Z}/d\mathbf{Z})$ dies after an etale cover for $q > 0$; hence its associated sheaf is zero.

Proof. Uses: prop:weil-a5-affine-curve-higher-vanishing-open,prop:weil-a5-affine-curve-h1-invariance-open,prop:weil-a5-finite-etale-spreading-open

For $q > 1$ the cohomology group is zero by higher vanishing. For $q = 1$, field invariance represents the class over the algebraically closed field L. View it as a finite $\mathbf{Z}/d\mathbf{Z}$-torsor and choose its finite etale cover; after pullback the class is zero. Spread this cover to an etale neighbourhood of the family by the spreading leaf. The resulting etale cover of U kills the original section. Since every local section dies locally, the associated sheaf is zero. □

Proposition 12.3.14 (Smooth stalk comparison from the atomic leaves). *Lean:* `conditional-interface:prop:weil-a5-smooth-stalk-comparison-assembly`

Uses: `def:weil-a5-smooth-cartesian-square`, `def:weil-a5-universal-affine-curve-test`

Assuming the five atomic smooth-base-change leaves above, every strict-henselian comparison map constructed in the preceding stalk-comparison lemma is an isomorphism.

Proof. Uses: lemma:weil-a5-smooth-relative-dimension-one-reduction,prop:weil-a5-universal-curve-reduction-open,lemma:weil-a5-universal-affine-curve-vanishing

The standard presentation and composition lemma reduce the smooth morphism to relative dimension one. The universal-curve reduction identifies the failure sheaf with the sheafification of the terminal presheaf. The terminal vanishing lemma makes this sheaf zero in every positive degree; degree zero is the equality of section equalizers under base change. Thus the cone of the stalk comparison has zero cohomology in every degree, so the comparison is an isomorphism. □

Proposition 12.3.15 (Finite locally constant smooth base change). *Lean:* `conditional-interface:prop:weil-a5-smooth-base-change-locally-constant`

Uses: `def:weil-a5-smooth-cartesian-square`, `def:weil-a5-locally-constant-stratum`

For a finite locally constant Λ-module sheaf L on X, the map

$$g^* R^q f_* L \xrightarrow{\sim} R^q f'_* g'^* L$$

is an isomorphism for every q provided the smooth strict-henselian stalk comparison constructed in the preceding stalk-comparison lemma is an isomorphism for every geometric point.

Proof. Uses: lemma:weil-a5-smooth-strict-henselian-stalk

By the stalk formula, the stalk of the displayed map at every geometric point is the comparison map in Lemma 12.3.5. The hypothesis makes each stalk map an isomorphism. A morphism of sheaves of Λ-modules is an isomorphism exactly when all geometric stalk maps are isomorphisms; therefore the displayed map is an isomorphism. □

Proposition 12.3.16 (Smooth base change for finite locally constant sheaves). *Lean:* `conditional-interface:prop:weil-a5-smooth-base-change-locally-constant-open`

Uses: `def:weil-a5-smooth-cartesian-square`, `def:weil-a5-locally-constant-stratum`

Assuming the five atomic smooth-base-change leaves, smooth base change holds for every finite locally constant sheaf.

Proof. Uses: prop:weil-a5-smooth-stalk-comparison-assembly,prop:weil-a5-smooth-base-change-locally-constant

The assembly proposition discharges the stalk hypothesis of the preceding global criterion. Applying that criterion gives the required isomorphism. □

12.4 A5.4. Constructible devissage and derived smooth base change

Lemma 12.4.1 (Finite stratification filtration). *Lean:* `conditional-interface:lemma:weil-a5-stratification-filtration`

Uses: `def:weil-ftdc-constructible-sheaf`, `def:weil-a5-locally-constant-stratum`

Let F be a constructible finite-torsion sheaf on X and fix a finite stratification. There is a finite filtration

$$0 = F_{-1} \subset F_0 \subset \cdots \subset F_r = F$$

whose successive quotients are extensions by zero of finite locally constant sheaves on individual strata.

Proof. Uses: def:weil-ftdc-constructible-sheaf,def:weil-a5-locally-constant-stratum

Order the finitely many strata so that the union of the first i strata is closed in the union of the first $i+1$. Let F_i be the kernel of the restriction of F to the complement of the first $i+1$ strata. Exactness of restriction gives $F_{i-1} \subset F_i$, and the quotient is the extension by zero of $F|_{U_i}$ because the two restriction kernels coincide on the complement and the open–closed sheaf equalizer identifies the quotient there. The finite locally constant identification is the stratum datum. □

Lemma 12.4.2 (Smooth exchange is compatible with short exact sequences)**.** Lean: conditional-interface:lemma:weil-a5-smooth-exchange-short-exact

Uses: def:weil-a5-smooth-exchange-degree

A short exact sequence $0 \to F_1 \to F_2 \to F_3 \to 0$ induces a morphism between the long exact cohomology sequences for the two sides of $\mathrm{Ex}^q_{\square,-}$, and the connecting morphisms commute.

Proof. Uses: def:weil-a5-smooth-exchange-degree

Represent the short exact sequence by its canonical distinguished triangle in the ambient derived category. Apply Lg^*Rf_* and $Rf'_*Lg'^*$; each is a triangulated functor, and the exchange maps form a morphism of triangles by their unit–counit construction. Taking cohomology gives the two long exact sequences and naturality gives commutation with the connecting maps. □

Proposition 12.4.3 (Smooth base change for constructible sheaves)**.** Lean: conditional-interface:prop:weil-a5-smooth-base-change-constructible

Uses: def:weil-a5-smooth-cartesian-square, def:weil-a5-constructible-input

For every constructible finite-torsion sheaf F on X, the map

$$g^* R^q f_* F \longrightarrow R^q f'_* g'^* F$$

is an isomorphism for every q, assuming the five atomic smooth-base-change obligations in Section A5.3.

Proof. Uses: lemma:weil-a5-stratification-filtration,lemma:weil-a5-smooth-exchange-short-exact,prop:weil-a5-smooth-base-change-locally-constant-open

Use the finite filtration of Lemma 12.4.1. For each quotient, the locally closed extension is reduced to its stratum by the open–closed exact triangle; the finite locally constant case is the preceding proposition, and extension by zero commutes with the restriction maps in the same triangle. Induct on the filtration length. At an induction step the two outer terms of the morphism of long exact sequences are isomorphisms; the five lemma gives the middle term in every degree. The connecting maps commute by Lemma 12.4.2. □

Definition 12.4.4 (Bounded constructible hypercohomology spectral sequences)**.** Lean: conditional-interface:def:weil-a5-hypercohomology-spectral-sequences

Uses: def:weil-a5-constructible-input

For $K \in D^b_c(X_{\mathrm{proet}}, \Lambda)$ with cohomology in $[a,b]$, define

$$E_2^{p,q} = R^p f_* \mathcal{H}^q(K) \Longrightarrow \mathcal{H}^{p+q}(Rf_*K), \qquad E_2'^{p,q} = R^p f'_* \mathcal{H}^q(Lg'^*K) \Longrightarrow \mathcal{H}^{p+q}(Rf'_*Lg'^*K).$$

The exchange map induces a morphism of these two spectral sequences.

Lemma 12.4.5 (Finite-range convergence and five-lemma criterion)**.** Lean: conditional-interface:lemma:weil-a5-hypercohomology-convergence-criterion

Uses: def:weil-a5-hypercohomology-spectral-sequences

If the E_2 comparison map is an isomorphism for every (p,q), then the abutment comparison on $\mathcal{H}^n$ is an isomorphism for every n.

Proof. Uses: def:weil-a5-hypercohomology-spectral-sequences

Only finitely many $q \in [a,b]$ occur in a fixed total degree. Filter each abutment by the finite filtration supplied by the exact couple. Induct on the filtration length; at each step the short exact sequence of successive filtration quotients has isomorphic outer terms, so the middle term is an isomorphism. This proves the criterion without invoking an unbounded convergence theorem. □

Proposition 12.4.6 (Derived smooth base change for finite-torsion complexes)**.** Lean: conditional-interface:prop:weil-a5-derived-smooth-base-change

Uses: def:weil-a5-smooth-cartesian-square, def:weil-a5-constructible-input, def:weil-a5-hypercohomology-spectral-sequences

For $K \in D^b_c(X_{\mathrm{proet}}, \Lambda)$, the exchange map

$$Lg^* Rf_* K \xrightarrow{\sim} Rf'_* Lg'^* K$$

is an isomorphism, assuming the locally constant stalk obligation.

Proof. Uses: def:weil-a5-hypercohomology-spectral-sequences,lemma:weil-a5-hypercohomology-convergence-criterion,prop:weil-a5-smooth-base-change-constructible,lemma:weil-a5-pullback-bounded-constructible

By the pullback-boundedness lemma, the cohomology sheaves of $Lg'^{*}K$ are constructible and lie in a finite interval. The E_2 comparison map is the constructible-sheaf smooth base-change map, hence an isomorphism in every entry. The finite-range criterion gives an isomorphism on every abutment cohomology sheaf. A map of bounded derived objects inducing isomorphisms on all cohomology sheaves is a quasi-isomorphism, which is the asserted derived isomorphism. □

12.5 A5.5. Compact-support base change at finite level

Definition 12.5.1 (Compactification datum for A5). Lean: conditional-interface:def:weil-a5-compactification-datum

Uses: def:weil-a5-smooth-cartesian-square,def:weil-a5-constructible-input

For a separated finite-type morphism $f : X \to Y$, a compactification datum is an open immersion $j : X \hookrightarrow \overline{X}$ followed by a proper map $p : \overline{X} \to Y$ with $f = pj$, together with the base-changed datum $j' : X' \hookrightarrow \overline{X}'$ and $p' : \overline{X}' \to Y'$.

Definition 12.5.2 (Underived open-extension exchange on sections). Lean: conditional-interface:def:weil-a5-open-extension-exchange-sections

Uses: def:weil-a5-compactification-datum,def:weil-a1-extension-zero-presheaf

For the Cartesian square of open immersions induced by a compactification and for a pro-etale object $W \to \overline{X}'$, define

$$\beta^{\mathrm{pre}}_{j,g}(W) : (gbar^{-1} j_!^{\mathrm{pre}} F)(W) \longrightarrow (j'^{\mathrm{pre}}_! g'^{-1} F)(W)$$

as follows. Represent a source section on a covering $(W_a \to W)_a$ by sections of F on $W_a \times_{\overline{X}} X$. Use the Cartesian equality

$$W_a \times_{\overline{X}} X \cong W_a \times_{\overline{X}'} X'$$

to regard the same family as a target section. Define $\gamma^{\mathrm{pre}}_{j,g}(W)$ by the inverse identification.

Lemma 12.5.3 (The open-extension section map is well-defined). Lean: conditional-interface:lemma:weil-a5-open-extension-section-well-defined

Uses: def:weil-a5-open-extension-exchange-sections

The maps $\beta^{\mathrm{pre}}_{j,g}(W)$ and $\gamma^{\mathrm{pre}}_{j,g}(W)$ are independent of the representing cover, preserve the support-closed condition, and commute with restriction in W.

Proof. Uses: def:weil-a5-open-extension-exchange-sections

Refine two representing covers by their fibre product. On that common refinement both constructions use the same Cartesian isomorphism, so the two images agree. The support of a representing section is carried to the same closed subset under the homeomorphism of the two fibre products; hence it is closed before the identification exactly when it is closed after it. For a map $W_1 \to W$, both restriction composites pull the family back along $W_1 \times_W W_a \to W_a$ and then apply the same Cartesian identification. Thus the two orders agree. □

Lemma 12.5.4 (The two section maps are inverse). Lean: conditional-interface:lemma:weil-a5-open-extension-section-inverses

Uses: def:weil-a5-open-extension-exchange-sections

For every W, the composites $\gamma^{\mathrm{pre}}\beta^{\mathrm{pre}}$ and $\beta^{\mathrm{pre}}\gamma^{\mathrm{pre}}$ are identity maps.

Proof. Uses: def:weil-a5-open-extension-exchange-sections,lemma:weil-a5-open-extension-section-well-defined

On a representing family the first composite applies a Cartesian identification and its inverse; every component is unchanged. The second composite is the same calculation in the opposite order. Independence of the representing family was proved in the preceding lemma, so the two sectionwise identities descend to the presheaves. □

Proposition 12.5.5 (Underived open-extension base change). Lean: conditional-interface:prop:weil-a5-underived-open-extension-base-change

Uses: def:weil-a5-open-extension-exchange-sections,def:weil-a1-extension-zero-sheaf

Sheafification of $\beta^{\mathrm{pre}}_{j,g}$ is a natural isomorphism

$$gbar^{*} j_! F \xrightarrow{\sim} j'_! g'^{*} F,$$

whose inverse is the sheafification of $\gamma^{\mathrm{pre}}_{j,g}$.

Proof. Uses: lemma:weil-a5-open-extension-section-well-defined,lemma:weil-a5-open-extension-section-inverses

Restriction compatibility makes the section maps morphisms of presheaves. Sheafification is functorial, so it carries the two presheaf composites to the two corresponding sheaf composites. Since the presheaf composites are identities, the sheaf composites are identities. Naturality in F is checked on a representing family, where both orders apply the sheaf morphism to the same sections. □

Proposition 12.5.6 (Derived open-extension base change). Lean: conditional-interface:prop:weil-a5-derived-open-extension-base-change

Uses: def:weil-a5-open-extension-exchange-sections, def:weil-a1-derived-extension-zero

The termwise underived comparison induces a natural isomorphism

$$Lgbar^* Rj_! K \xrightarrow{\sim} Rj'_! Lg'^* K$$

on bounded constructible finite-torsion complexes.

Proof. Uses: prop:weil-a5-underived-open-extension-base-change,lemma:weil-a1-extension-zero-exact,lemma:weil-ftdc-derived-pullback-termwise

By exactness, both derived extension-by-zero functors are represented by applying $j_!$ termwise. A2 pullback is also represented termwise. Apply the underived comparison in each degree; its naturality makes the squares with the differentials commute. The inverse is obtained degreewise from the underived inverse, and the two composites are identity chain maps. Localization therefore gives the asserted derived isomorphism. □

Definition 12.5.7 (Finite-level compact-support exchange). Lean: conditional-interface:def:weil-a5-finite-compact-support-exchange

Uses: def:weil-a5-compactification-datum,def:weil-a1-derived-extension-zero,def:weil-pbc-derived-exchange

For $K \in D_c^b(X_{\mathrm{proet}}, \Lambda)$ define

$$Rf_!^{(j,p)} K := Rp_* Rj_! K,$$

and define the exchange morphism by the composite

$$Lg^* Rp_* Rj_! K \to Rp'_* Lgbar^* Rj_! K \xrightarrow{Rp'_* \mathrm{Ex}_j} Rp'_* Rj'_! Lg'^* K.$$

Here the first arrow is the A3 proper exchange and Ex_j is the open-extension exchange map supplied by the chosen A1 compactification data.

Lemma 12.5.8 (Proper part of compact-support exchange is invertible). Lean: conditional-interface:lemma:weil-a5-compact-support-proper-part

Uses: def:weil-a5-finite-compact-support-exchange

The first arrow in the compact-support exchange is an isomorphism whenever p is proper.

Proof. Uses: prop:weil-proet-finite-derived-proper-base-change

Apply the finite-level A3 proper base-change proposition to p, the base-changed square, and the constructible complex $Rj_! K$. Its hypotheses are satisfied because p is proper and the A1 extension-by-zero output is constructible by the compactification input certificate. □

Lemma 12.5.9 (Open-extension exchange criterion). Lean: conditional-interface:lemma:weil-a5-open-extension-exchange-criterion

Uses: def:weil-a5-finite-compact-support-exchange

The compact-support exchange is an isomorphism if the open-extension exchange $Lgbar^* Rj_! K \to Rj'_! Lg'^* K$ is an isomorphism.

Proof. Uses: def:weil-a5-finite-compact-support-exchange,lemma:weil-a5-compact-support-proper-part

The compact-support exchange is a composite of the proper part and the open-extension part. Both factors are isomorphisms by the hypothesis and the preceding lemma, hence their composite is an isomorphism. □

Proposition 12.5.10 (Finite-level compact-support base change). Lean: conditional-interface:prop:weil-a5-finite-compact-support-base-change

Uses: def:weil-a5-finite-compact-support-exchange

For every compactification datum and every finite-torsion constructible K, the compact-support exchange is an isomorphism.

Proof. Uses: `lemma:weil-a5-open-extension-exchange-criterion,prop:weil-a5-derived-open-extension-base-change`

The derived open-extension proposition supplies the hypothesis of the criterion. The criterion then combines it with A3 proper base change. No independence of compactification is asserted at this stage. □

12.6 A5.6. Compactification refinement

Definition 12.6.1 (A5 compactification comparison datum)**.** Lean: `conditional-interface:def:weil-a5-compactification-comparison`

Uses: `def:weil-a5-compactification-datum`

For two compactification data (j_1, p_1) and (j_2, p_2) of f, a comparison datum is a proper scheme $\overline{X}_{12}$ with proper maps $r_i : \overline{X}_{12} \to \overline{X}_i$ restricting to the identity on X, together with the two displayed proper exchange isomorphisms and the two open-extension comparison maps.

Lemma 12.6.2 (Refinement comparison gives a commuting square)**.** Lean: `conditional-interface:lemma:weil-a5-refinement-comparison-square`

Uses: `def:weil-a5-compactification-comparison,def:weil-a5-finite-compact-support-exchange`

The two compact-support exchange maps associated with compactifications 1 *and* 2 *agree after transport through a comparison datum, provided the proper and open-extension comparison maps satisfy their displayed cocycle equations.*

Proof. Uses: `def:weil-a5-compactification-comparison,def:weil-a5-finite-compact-support-exchange`

Expand each exchange as proper exchange followed by open-extension exchange. Insert the two comparison maps from $\overline{X}_{12}$ and use the assumed proper cocycle and open-extension cocycle. The resulting rectangle has equal top and bottom composites by functoriality of Rp_* and $Rj_!$; hence the two maps agree after transport. □

Proposition 12.6.3 (Canonical finite-level compact-support base change)**.** Lean: `conditional-interface:prop:weil-a5-canonical-finite-compact-support-base-change`

Uses: `def:weil-a5-compactification-comparison,def:weil-a5-finite-compact-support-exchange`

Assume the A1 cofilteredness, cancellation, and refinement-coherence obligations listed below. Then the finite-level compact-support exchange descends to a compactification-independent natural isomorphism

$$Lg^* Rf_! K \xrightarrow{\sim} Rf'_! Lg'^* K.$$

Proof. Uses: `def:weil-a5-compactification-comparison,lemma:weil-a5-refinement-comparison-square,prop:weil-a5-finite-compact-support-base-change`

Choose a compactification datum only in the proof of the local representative and use the common refinement to compare any two choices. The comparison square shows equality after refinement; the refinement maps are proper, so A3 proper base change transports equality back. Thus the family of maps is independent of the datum and defines the displayed canonical isomorphism. Naturality follows by expanding the same rectangle for a morphism $K \to K'$. □

12.7 A5.7. Constructibility preservation for $Rf_!$

Definition 12.7.1 (Finite-level constructibility-preservation datum)**.** Lean: `conditional-interface:def:weil-a5-rfshriek-constructibility-datum`

Uses: `def:weil-a5-constructible-input,def:weil-a5-compactification-datum`

A constructibility-preservation datum consists of a separated morphism of finite presentation $f : X \to Y$ between quasi-compact quasi-separated schemes, a finite torsion coefficient ring Λ invertible on Y, a bounded constructible complex K on X_{et}, and a selected finite cohomological interval for K.

Proposition 12.7.2 (Quasi-finite exceptional-image seed)**.** Lean: `conditional-interface:prop:weil-a5-quasi-finite-constructibility-seed`

Uses: `def:weil-a5-rfshriek-constructibility-datum`

If f is locally quasi-finite, then $f_!$ is exact and sends constructible finite-torsion sheaves to constructible sheaves, assuming the A1 exact-stalk criterion.

Proof. Uses: lemma:weil-a1-extension-zero-exact,lemma:weil-pbc-constructible-finite-pushforward

The assertion is local on Y and a finite affine cover reduces to the separated case. Zariski's Main Theorem factors a separated quasi-finite finite-presentation map as an open immersion $j : X \hookrightarrow \overline{X}$ followed by a finite map $p : \overline{X} \to Y$. The A1 stalk formula makes $j_!$ exact and constructible, subject to its named exact-stalk obligation. Finite direct image is exact and constructible by the A3 finite-pushforward lemma. Therefore $f_! = p_* j_!$ is exact and constructible. This is the expanded dependency chain of Stacks Project, Lemma 63.14.1. □

Definition 12.7.3 (Affine-line constructibility test)**.** Lean: conditional-interface:def:weil-a5-affine-line-constructibility-test

Uses: def:weil-a5-rfshriek-constructibility-datum

For S affine and $p : \mathbf{A}^1_S \to S$, define $\mathsf{AC}(S, F)$ to be the assertion that every cohomology sheaf of $Rp_!F$ is constructible, for a constructible finite-torsion sheaf F on $\mathbf{A}^1_S$.

Proposition 12.7.4 (Compactly supported fibre finiteness leaf)**.** Lean: conditional-interface:prop:weil-a5-compact-support-fibre-finiteness-open

Uses: def:weil-a5-affine-line-constructibility-test

For a finite-type curve C over an algebraically closed field and a finite constructible coefficient sheaf F, every group $H^q_c(C, F)$ is finite and vanishes outside a finite range, is

Open obligation: OBL-A5-CONSTR-CURVE-COMPACT-SUPPORT-FINITENESS

.

Proof. Uses: def:weil-a5-affine-line-constructibility-test

The source locator is Stacks Project, Lemma 63.12.4. This finiteness theorem is absent from mathlib and A1–A3, so it is isolated rather than bundled into constructibility of direct image. □

Proposition 12.7.5 (Relative-curve constant-sheaf seed leaf)**.** Lean: conditional-interface:prop:weil-a5-relative-curve-constant-seed-open

Uses: def:weil-a5-affine-line-constructibility-test

For a noetherian affine finite-dimensional S, a separated affine smooth relative curve $u : C \to S$, and a finite constant module M, the complex $Ru_!\underline{M}$ has constructible cohomology, is

Open obligation: OBL-A5-CONSTR-RELATIVE-CURVE-CONSTANT-SEED

.

Proof. Uses: def:weil-a5-affine-line-constructibility-test,prop:weil-a5-compact-support-fibre-finiteness-open

The proof required by Stacks Project, Lemma 63.14.2, is an induction on $\dim S$ using compact-support base change, fibre finiteness, finite normalizations, open–closed triangles, and a quasi-finite boundary. These ingredients are not all prior closed labels, so the exact seed remains open.

Open obligation: OBL-A5-CONSTR-RFSHRIEK-BASE-CHANGE-RESTRICTION

Open obligation: OBL-A5-CONSTR-THICKENING-INVARIANCE

□

Proposition 12.7.6 (Finite-etale lisse filtration leaf)**.** Lean: conditional-interface:prop:weil-a5-lisse-filtration-open

Uses: def:weil-a5-affine-line-constructibility-test

For a finite-type locally constant Λ-sheaf L on a connected noetherian scheme, after a finite etale cover there is a finite filtration of the pullback whose quotients are constant finite modules, together with splitting maps whose composite on L is the identity, is

Open obligation: OBL-A5-CONSTR-LISSE-FINITE-ETALE-FILTRATION

.

Proof. Uses: def:weil-a5-affine-line-constructibility-test

The source locator is Stacks Project, Lemma 59.66.4 and Section 59.66. The existence of the finite cover, filtration, and splitting maps is not a foundation leaf; all three are explicitly included in the obligation. □

Lemma 12.7.7 (Affine-line lisse constructibility)**.** Lean: conditional-interface:lemma:weil-a5-affine-line-lisse-constructibility

Uses: def:weil-a5-affine-line-constructibility-test

Assuming the relative-curve seed and lisse-filtration leaves, $Rp_!L$ has constructible cohomology for every finite locally constant L on an open subscheme of $\mathbf{A}^1_S$.

Proof. Uses: `prop:weil-a5-relative-curve-constant-seed-open`, `prop:weil-a5-lisse-filtration-open`, `prop:weil-a5-quasi-finite-constructibility-seed`

Use the finite etale cover and splitting to make L a direct summand of the finite pushforward of a filtered constant sheaf. Composition for $Rf_!$ identifies its exceptional image with the image after the finite cover. Apply the constant relative-curve seed to each filtration quotient and use the long exact cohomology sequence to preserve constructibility through the finite filtration. Constructible sheaves are closed under direct summands, so the original exceptional image is constructible. □

Proposition 12.7.8 (Affine-line constructible-sheaf induction leaf). Lean: `conditional-interface:prop:weil-a5-affine-line-constructible-induction-open`

Uses: `def:weil-a5-affine-line-constructibility-test`

The assertion $\mathsf{AC}(S, F)$ *for arbitrary constructible* F*, including finite-stage descent of* F*, selection of a dense lisse open, finiteness of the boundary after shrinking* S*, dimension induction, and the open–closed triangle, is*

Open obligation: `OBL-A5-CONSTR-AFFINE-LINE-DEVISSAGE`

.

Proof. Uses: `def:weil-a5-affine-line-constructibility-test`, `lemma:weil-a5-affine-line-lisse-constructibility`, `prop:weil-a5-quasi-finite-constructibility-seed`

The lisse open is handled by the preceding lemma and the finite boundary by the quasi-finite seed. What remains is the simultaneous finite-stage and dimension induction of Stacks Project, Lemma 63.14.4. Because those choices and termination are not prior labels, the complete devissage is retained as the displayed obligation. The dense-open restriction step uses `OBL-A5-CONSTR-RFSHRIEK-BASE-CHANGE-RESTRICTION`, and replacing a nilpotent thickening by its reduction uses `OBL-A5-CONSTR-THICKENING-INVARIANCE`. □

Proposition 12.7.9 (Finite cohomological dimension of $Rf_!$ leaf). Lean: `conditional-interface:prop:weil-a5-rfshriek-finite-cd-open`

Uses: `def:weil-a5-rfshriek-constructibility-datum`

For separated finite-presentation f*, the functor* $Rf_!$ *has a finite cohomological-dimension bound locally on the target, stable under base change and composition, is*

Open obligation: `OBL-A5-CONSTR-RFSHRIEK-FINITE-COHOMOLOGICAL-DIMENSION`

.

Proof. Uses: `def:weil-a5-rfshriek-constructibility-datum`

The source locator is Stacks Project, Lemma 63.10.2. This bound is required to pass from a sheaf to a bounded constructible complex without an unbounded convergence shortcut and is absent from the project foundation contract. □

Proposition 12.7.10 ($Rf_!$ preserves constructibility). Lean: `conditional-interface:prop:weil-a5-rfshriek-preserves-constructibility`

Uses: `def:weil-a5-rfshriek-constructibility-datum`, `def:weil-a5-affine-line-constructibility-test`

Assuming the affine-line devissage and finite-cohomological-dimension leaves, $Rf_!K$ *has bounded constructible cohomology for every finite-level constructibility-preservation datum.*

Proof. Uses: `prop:weil-a5-affine-line-constructible-induction-open`, `prop:weil-a5-rfshriek-finite-cd-open`, `prop:weil-a5-quasi-finite-constructibility-seed`

The question is local on Y; a finite affine cover and the Mayer–Vietoris triangle reduce to X, Y affine. Choose a finite-presentation closed immersion $i : X \hookrightarrow \mathbf{A}^n_Y$. Composition gives $Rf_! = Rp_! i_!$, and the quasi-finite seed preserves constructibility under $i_!$. Factor p into n affine-line projections and apply the affine-line devissage successively. For a bounded complex, use its finite truncation filtration. The finite cohomological-dimension bound makes every total-degree filtration finite, so the long exact sequences preserve constructibility and boundedness. This is the expanded assembly of Stacks Project, Theorem 63.14.5 (Tag 0GL0). □

Proposition 12.7.11 (Proper direct image preserves constructibility). Lean: `conditional-interface:prop:weil-a5-proper-direct-image-constructible`

Uses: `def:weil-a5-rfshriek-constructibility-datum`

For proper f*, assuming the preceding constructibility leaves,* Rf_*K *has bounded constructible cohomology.*

Proof. Uses: `prop:weil-a5-rfshriek-preserves-constructibility`, `lemma:weil-pbc-a2-comparison-rfshriek-transport`

For a proper morphism use the identity compactification $(1_X, f)$. Extension by zero for the identity is the identity functor, so the A1/A3 transport identifies $Rf_!$ with Rf_*. Apply the preceding preservation proposition and transport its constructible output through this identity. □

12.8 A5.8. Inherited obligations and coefficient passage

Proposition 12.8.1 (Inherited A1 compactification obligations). *Lean:* `conditional-interface:prop:weil-a5-compactification-obligations`

Uses: `def:weil-a5-compactification-comparison`

The remaining finite-level choice-independence prerequisites are the existing A1 obligations OBL-A1-COMPACTIFICATIONS-COFILTERED, OBL-A1-COMMON-REFINEMENT-CANCELLATION, OBL-A1-BASE-CHANGE-REFINEMENT-COHERENCE, and OBL-A1-RFSHRIEK-INDEPENDENCE. The derived open-extension step introduces no further A1 compactification obligation.

Proof. Uses: `def:weil-a5-compactification-comparison,prop:weil-a5-canonical-finite-compact-support-base-change,lemma:weil-a1-extension-zero-exact`

Common refinement is needed to compare choices, cancellation to descend the comparison, and refinement coherence to identify the two exchange maps. The A1 $Rf_!$ independence node packages precisely those consequences. The open-extension map itself is constructed and inverted above by the exactness theorem. □

Definition 12.8.2 (Compatible adic reduction tower). *Lean:* `conditional-interface:def:weil-a5-adic-reduction-tower`

Uses: `def:weil-a5-constructible-input`

For $\Lambda = \mathbf{Z}_\ell$ and $K \in D_c^b(X_{\mathrm{proet}}, \Lambda)$, an adic reduction tower is a family

$$K_n = K \otimes_\Lambda^L \Lambda/\ell^n, \qquad r_{n+1,n} : K_{n+1} \to K_n,$$

with the canonical quotient transition equation and a common cohomological bound.

Lemma 12.8.3 (Finite-level exchange is compatible with reduction). *Lean:* `conditional-interface:lemma:weil-a5-reduction-compatibility`

Uses: `def:weil-a5-adic-reduction-tower, def:weil-a5-finite-compact-support-exchange`

The reduction of the adic compact-support exchange at level n equals the finite-torsion compact-support exchange for K_n, and these equalities commute with $r_{n+1,n}$.

Proof. Uses: `def:weil-a5-adic-reduction-tower,prop:weil-a5-canonical-finite-compact-support-base-change`

Expand both maps using the same compactification. Derived pullback, derived tensor with Λ/ℓ^n, $Rj_!$, and Rp_* are functorial, and the A3 proper exchange is natural in the coefficient object. Applying reduction to each component therefore gives the finite-level map. Applying the quotient transition to the same componentwise equality gives compatibility with $r_{n+1,n}$. □

Proposition 12.8.4 (A5 adic compact-support base change interface). *Lean:* `conditional-interface:prop:weil-a5-adic-compact-support-base-change`

Uses: `def:weil-a5-adic-reduction-tower, def:weil-a5-finite-compact-support-exchange`

Assume the inherited A1 finite-level obligations and the A2 derived-complete reconstruction theorem. Then the compatible finite-level maps induce a natural isomorphism

$$Lg^* Rf_! K \xrightarrow{\sim} Rf'_! Lg'^* K$$

in the bounded constructible $\mathbf{Z}_\ell$ category; its reduction at every n is the finite-level map.

Proof. Uses: `def:weil-a5-adic-reduction-tower,lemma:weil-a5-reduction-compatibility,prop:weil-a5-canonical-finite-compact-support-base-change`

The finite-level maps form a morphism of compatible towers by the reduction lemma. Apply the A2 derived-complete reconstruction to the two towers and the tower morphism. Each level is an isomorphism, so the reconstructed map is an isomorphism; the reconstruction projections give the stated reductions. Naturality is inherited levelwise and reflected by the jointly monic product of reduction projections. □

Proposition 12.8.5 (A5 exported interface). *Lean:* `conditional-interface:prop:weil-a5-smooth-constructibility-interface`

Uses: `def:weil-a5-smooth-cartesian-square, def:weil-a5-constructible-input, def:weil-a5-adic-reduction-tower`

The A5 interface exports the following conditional results:

(a) smooth pullback preserves bounded finite-torsion constructibility;

(b) *smooth base change holds for $D_c^b(-,\Lambda)$ once the five atomic smooth-base-change obligations in Section A5.3 are discharged;*

(c) *compact-support base change holds at finite and adic levels once the listed A1 compactification and exact-stalk obligations are discharged.*

Proof. Uses: `prop:weil-a5-derived-smooth-base-change,prop:weil-a5-adic-compact-support-base-change,lemma:weil-a5-pullback-bounded-constructible`

Item (a) is the pullback-boundedness lemma. Item (b) is the derived smooth base-change proposition and its explicitly named local-system leaf. Item (c) is the finite-level canonical proposition followed by the adic reconstruction proposition. The list is conditional exactly at the named A1, site-comparison, and smooth-stalk obligations and does not assert their existence. □

Chapter 13

Trace and arithmetic core

Status. All Frobenius operators in this fragment are geometric. If φ_q denotes arithmetic Frobenius, then the operator used below is $\Phi_q = \varphi_q^{-1}$. The Tate normalization is $\Phi_q|E(1) = q^{-1}$, and every characteristic polynomial is $P_i(T) = \det(1 - T\Phi_i)$. Thus a Frobenius eigenvalue is the reciprocal of an ordinary polynomial root. No geometric fixed-point theorem, trace formula, finiteness theorem, or six-functor theorem is silently imported.

13.1 Arithmetic input and Frobenius conventions

Definition 13.1.1 (Trace-core arithmetic convention)**.** Uses: `def:weil-prime-power-data`, `def:weil-finite-field-data`, `def:weil-coefficient-data`, `def:weil-arithmetic-field-frobenius`, `def:weil-geometric-field-frobenius`, `def:weil-tate-line`, `def:weil-tate-twist`

Lean: `Weil.TraceCore.ArithmeticConventions`

For a prime-power datum (p, a, q), a finite-field datum $k_0 = \mathbf{F}_q \subset k = \overline{\mathbf{F}}_q$, and a coefficient datum $E = \mathbf{Q}_\ell \subset \overline{E}$, define the trace-core convention $\mathcal{C}_{q,\ell}$ to consist of the following fixed choices:

(i) arithmetic field Frobenius $\varphi_q(x) = x^q$;

(ii) geometric field Frobenius $\Phi_q = \varphi_q^{-1}$;

(iii) the Tate line $E(1)$ on which Φ_q is multiplication by q^{-1}, and hence $E(-r)$ has scalar q^r;

(iv) for a finite-dimensional Frobenius module (V, F_V), the factor $P_V(T) = \det(1 - TF_V) \in E[T]$.

No scheme, sheaf, or cohomology object is part of this Definition.

Definition 13.1.2 (Periodic geometric-point input)**.** Uses: `def:tracecore-arithmetic-conventions`

Lean: `Weil.TraceCore.PeriodicPointInput`

A periodic geometric-point input is a tuple

$$\mathcal{D} = (\Omega, \Phi, \mathrm{per}, \mathrm{perpos}),$$

where Ω is a finite type, $\Phi : \Omega \simeq \Omega$ is a permutation, $\mathrm{per} : \Omega \to \mathbf{N}$, and $\mathrm{perpos}(x) : 1 \le \mathrm{per}(x)$ together with the equation $\Phi^{\mathrm{per}(x)}(x) = x$ for every $x \in \Omega$. The intended geometric instance is $\Omega = X_0(k)$ with geometric Frobenius induced by Φ_q, but this intended instance is not asserted.

Definition 13.1.3 (Finite-extension point input)**.** Uses: `def:tracecore-periodic-point-input`

Lean: `Weil.TraceCore.FiniteExtensionPointInput`

For a periodic input $\mathcal{D}$, a finite-extension point input consists of finite types R_n for $n \ge 1$, maps

$$s_n : R_n \longrightarrow \Omega, \qquad r_n : \Omega \longrightarrow R_n,$$

and a natural number sequence $N_n := \#R_n$. The maps are candidates; the equations saying that they are inverse and that R_n equals $\mathrm{Fix}(\Phi^n)$ are not fields of this Definition.

Definition 13.1.4 (Geometric Frobenius fixed-point finset)**.** Uses: `def:tracecore-periodic-point-input`

Lean: `Weil.TraceCore.fixedPointFinset`

For $n \ge 1$, define

$$\mathrm{Fix}_{\mathcal{D}}(n) := \{x \in \Omega \mid \Phi^n(x) = x\}.$$

It is regarded as a finite subtype because Ω is a finite type.

Lemma 13.1.5 (Fixed-point finiteness). *Uses:* def:tracecore-fixed-point-finset

Lean: Weil.TraceCore.fixedPointFinset_finite

For every $n \geq 1$, $\mathrm{Fix}_{\mathcal{D}}(n)$ is finite.

Proof. Uses: def:tracecore-periodic-point-input, def:tracecore-fixed-point-finset

The carrier of $\mathrm{Fix}_{\mathcal{D}}(n)$ is a subtype of the finite type Ω. The restriction of the finite-type instance to a subtype supplies a finite-type instance, and the cardinality of a finite type is therefore defined. No assertion about a scheme is used. □

Lemma 13.1.6 (Membership in the fixed-point finset). *Uses:* def:tracecore-fixed-point-finset

Lean: Weil.TraceCore.mem_fixedPointFinset

For $x \in \Omega$,

$$x \in \mathrm{Fix}_{\mathcal{D}}(n) \quad \Longleftrightarrow \quad \Phi^{n}(x) = x.$$

Proof. Uses: def:tracecore-fixed-point-finset

Unfold the subtype predicate in the Definition. Its membership proposition is exactly the displayed equality, so both implications are the identity function on proofs of that equality. □

Definition 13.1.7 (Rational–fixed-point comparison candidate). Uses: def:tracecore-finite-extension-point-input, def:tracecore-fixed-point-finset

Lean: Weil.TraceCore.RationalFixedComparison

A rational–fixed-point comparison candidate is the family of maps

$$u_n : R_n \to \mathrm{Fix}_{\mathcal{D}}(n), \qquad v_n : \mathrm{Fix}_{\mathcal{D}}(n) \to R_n,$$

obtained by factoring s_n and r_n through the fixed-point subtype. The bare candidate contains no inverse equation. Whenever an inverse law is used in this legacy abstract layer, it is an explicit hypothesis.

Lemma 13.1.8 (Rational–fixed left inverse from a supplied certificate). *Uses:* def:tracecore-rational-fixed-comparison

Lean: Weil.TraceCore.rationalFixed_leftInverse

For $n \geq 1$, suppose that the comparison candidate is supplied with a pointwise certificate

$$h_n^{\mathrm{L}} : \quad \forall y \in R_n, \qquad v_n(u_n(y)) = y.$$

Then $v_n \circ u_n = \mathrm{id}_{R_n}$.

Proof. Uses: def:tracecore-rational-fixed-comparison

Apply function extensionality. At $y \in R_n$, the required equality is exactly $h_n^{\mathrm{L}}(y)$. Thus this abstract lemma consumes an inverse certificate and does not manufacture finite-field descent from the candidate maps alone. □

Lemma 13.1.9 (Fixed–rational right inverse from a supplied certificate). *Uses:* def:tracecore-rational-fixed-comparison

Lean: Weil.TraceCore.rationalFixed_rightInverse

For $n \geq 1$, suppose that the comparison candidate is supplied with a pointwise certificate

$$h_n^{\mathrm{R}} : \quad \forall z \in \mathrm{Fix}_{\mathcal{D}}(n), \qquad u_n(v_n(z)) = z.$$

Then $u_n \circ v_n = \mathrm{id}_{\mathrm{Fix}_{\mathcal{D}}(n)}$.

Proof. Uses: def:tracecore-rational-fixed-comparison

Apply function extensionality and then h_n^{R} at the chosen fixed point. No descent equation is inferred from the bare candidate package. □

Proposition 13.1.10 (Finite-extension cardinality from inverse certificates). *Uses:* def:tracecore-finite-extension-point-input, def:tracecore-fixed-point-finset, def:tracecore-rational-fixed-comparison

Lean: Weil.TraceCore.finiteExtension_card_eq_fixed_card

For $n \geq 1$, assume pointwise certificates h_n^{L} and h_n^{R} as in the preceding two lemmas. Then

$$\#R_n = \#\,\mathrm{Fix}_{\mathcal{D}}(n).$$

Proof. Uses: `lemma:tracecore-rational-fixed-left-inverse,lemma:tracecore-rational-fixed-right-inverse,lemma:tracecore-fixed-point-finite`

The two cited lemmas turn the pointwise certificates into inverse equations for u_n and v_n. They therefore define an equivalence between the two finite types. Apply finite-cardinality invariance to this equivalence. □

Definition 13.1.11 (Closed-point degree family). Uses: `def:tracecore-periodic-point-input`

Lean: `Weil.TraceCore.ClosedPointDegreeFamily`

A closed-point degree family is a type C, a degree map $\deg : C \to \mathbf{N}_{>0}$, and, for each $m \geq 1$, a proof that $\{c \in C \mid \deg(c) \leq m\}$ is finite. It is an arithmetic index family; no identification with scheme closed points is assumed.

Definition 13.1.12 (Closed-point–orbit comparison candidate). Uses: `def:tracecore-closed-point-degree-family,def:tracecore-periodic-point-input,def:tracecore-fixed-point-finset`

Lean: `Weil.TraceCore.ClosedPointOrbitComparison`

A closed-point–orbit comparison candidate consists of maps

$$\alpha : C \to \Omega/\langle\Phi\rangle, \qquad \beta : \Omega/\langle\Phi\rangle \to C,$$

and a degree comparison candidate $\deg(c) = |\alpha(c)|$. The maps, their inverse equations, and degree equation are not asserted by this Definition.

Lemma 13.1.13 (Closed-point degree compatibility from a supplied certificate). *Uses:* `def:tracecore-closed-point-orbit-comparison`

Lean: `Weil.TraceCore.closedPoint_degree_compatibility`

Suppose that a closed-point–orbit comparison candidate is supplied with the pointwise certificate

$$h_{\deg} : \quad \forall c \in C, \qquad \deg(c) = |\alpha(c)|.$$

Then the candidate is degree compatible.

Proof. Uses: `def:tracecore-closed-point-orbit-comparison`

The degree-compatibility field is exactly the family $h_{\deg}$. This abstract lemma consumes that family; it does not infer it from the two candidate maps. □

13.2 Closed-point products and point-counting series

Definition 13.2.1 (Point-counting sequence and logarithm). Uses: `def:tracecore-finite-extension-point-input,def:tracecore-arithmetic-conventions`

Lean: `Weil.TraceCore.CountingSeries`

Define

$$L_R(T) := \sum_{n \geq 1} \frac{\#R_n}{n} T^n \in E[[T]], \qquad Z_R(T) := \mathrm{Exp}(L_R(T)).$$

The inclusion $\mathbf{Q} \hookrightarrow E$ is used for the rational coefficients $1/n$. This is a formal-power-series Definition; no analytic convergence is asserted.

Definition 13.2.2 (Closed-point local factor). Uses: `def:tracecore-closed-point-degree-family`

Lean: `Weil.TraceCore.closedPointLocalFactor`

For $c \in C$, define

$$e_c(T) := (1 - T^{\deg(c)})^{-1} \in E[[T]],$$

where the inverse is the formal inverse of the constant-one series.

Definition 13.2.3 (Truncated closed-point Euler product). Uses: `def:tracecore-closed-point-degree-family,def:tracecore-closed-point-local-factor`

Lean: `Weil.TraceCore.truncatedEulerProduct`

For $m \geq 1$, define the finite product

$$Z_{C,\leq m}(T) := \prod_{\substack{c \in C \\ \deg(c) \leq m}} e_c(T).$$

The product is finite by the local-finiteness field of the degree family.

Lemma 13.2.4 (Geometric-series expansion of one local factor). *Uses:* *def:tracecore-closed-point-local-factor*

Lean: *Weil.TraceCore.localFactor_geometricSeries*

For $d \geq 1$,

$$(1 - T^d)^{-1} = \sum_{j \geq 0} T^{jd} \quad \text{in } E[[T]].$$

Proof. Uses: def:tracecore-closed-point-local-factor

Multiply the right-hand side by $1 - T^d$. The coefficient of degree zero is one. At positive degree r, the coefficient is one from the term T^{rd} exactly when $d \mid r$, and is cancelled by the shifted term from $T^{(r/d-1)d}$; if $d \nmid r$, both coefficients are zero. Thus the product is the constant-one series, which is the defining inverse. □

Lemma 13.2.5 (Low-degree stability of truncated products). *Uses:* *def:tracecore-truncated-euler-product*, *def:tracecore-closed-point-local-factor*

Lean: *Weil.TraceCore.truncatedEulerProduct_stable*

If $m \geq r$, then the coefficients of degrees $0, 1, \ldots, r$ in $Z_{C,\leq m}(T)$ and $Z_{C,\leq r}(T)$ agree.

Proof. Uses: def:tracecore-truncated-euler-product, lemma:tracecore-local-factor-geometric-series

Factor the larger product as the product for $\deg(c) \leq r$ times the product over $r < \deg(c) \leq m$. Each factor in the second product is $1 + T^{\deg(c)} + T^{2\deg(c)} + \cdots$, hence is congruent to one modulo T^{r+1}. A finite product of such series is also congruent to one modulo T^{r+1}, so the two products have equal coefficients through degree r. □

Definition 13.2.6 (Locally finite closed-point Euler product). Uses: def:tracecore-truncated-euler-product, def:tracecore-closed-point-degree-family

Lean: Weil.TraceCore.closedPointEulerProduct

Define $Z_C(T) \in E[[T]]$ coefficientwise by

$$[T^r] Z_C := [T^r] Z_{C,\leq r}(T) \qquad (r \geq 0).$$

The preceding stability lemma proves that this coefficient is unchanged if any cutoff at least r is used.

Lemma 13.2.7 (Coefficient formula for the logarithm of the Euler product). *Uses:* *def:tracecore-closed-point-euler-product*, *def:tracecore-closed-point-degree-family*

Lean: *Weil.TraceCore.eulerProduct_logCoefficient*

For every $n \geq 1$,

$$[T^n] \operatorname{Log}(Z_C(T)) = \sum_{\substack{c \in C \\ \deg(c) \mid n}} \frac{\deg(c)}{n}.$$

The sum is finite by local finiteness.

Proof. Uses: def:tracecore-closed-point-euler-product, def:tracecore-closed-point-local-factor, lemma:tracecore-local-factor-geometric-series

Fix a cutoff $m \geq n$. By low-degree stability, degree n of the locally finite product equals degree n of the finite product $Z_{C,\leq m}$. For a finite product of constant-one series, the formal logarithm is additive. For one factor, the coefficient of T^n in $-\operatorname{Log}(1 - T^d)$ is $1/j$ when $n = jd$, and zero otherwise, by the formal derivative identity $\frac{d}{dT} \operatorname{Log}(1 - U) = -U'/(1 - U)$ and the geometric-series expansion. Therefore each c with $d = \deg(c) \mid n$ contributes $1/(n/d) = d/n$, and all other factors contribute zero. The set of such c is contained in $\{\deg(c) \leq n\}$, hence finite. □

Definition 13.2.8 (Orbit-counting compatibility predicate). Uses: def:tracecore-finite-extension-point-input, def:tracecore-closed-point-degree-family

Lean: Weil.TraceCore.OrbitCountingCompatibility

Define $\operatorname{OrbitCountCompat}(R, C)$ to be the family of equalities

$$\#R_n = \sum_{\substack{c \in C \\ \deg(c) \mid n}} \deg(c) \qquad (n \geq 1).$$

It is a predicate on the already supplied point and degree families; it does not assert that an instance exists.

Proposition 13.2.9 (Euler product equals the point-counting exponential). *Uses:* *def:tracecore-counting-series, def:tracecore-closed-point-euler-product, def:tracecore-orbit-counting-compatibility*

Lean: *Weil.TraceCore.eulerProduct_exponential*

If OrbitCountCompat(R, C), *then*

$$Z_R(T) = Z_C(T) \quad \text{in } E[[T]].$$

Proof. Uses: `def:tracecore-counting-series`, `def:tracecore-closed-point-euler-product`, `lemma:tracecore-euler-log-coefficient`, `def:tracecore-orbit-counting-compatibility`

The coefficient of T^n in $\mathrm{Log}(Z_R)$ is $\#R_n/n$ by the Definition of L_R. The preceding logarithm coefficient lemma gives the coefficient of $\mathrm{Log}(Z_C)$ as $\sum_{\deg(c)|n} \deg(c)/n$. The compatibility predicate identifies these coefficients for every positive degree, and both logarithms have zero constant coefficient. Formal exponentiation is inverse to formal logarithm on constant-one series, so the two exponentials are equal. □

Proposition 13.2.10 (Orbit-counting compatibility from explicit certificates). *Uses:* *def:tracecore-closed-point-degree-family, def:tracecore-periodic-point-input, def:tracecore-finite-extension-point-input, def:tracecore-rational-fixed-comparison, def:tracecore-closed-point-orbit-comparison*

Lean: *Weil.TraceCore.schemeOrbit_compatibility*

Let $\mathcal{D}$, R, C, (u_n, v_n), *and* (α, β) *be the displayed abstract data. Suppose that*

(i) h_n^{L} *and* h_n^{R} *give the two rational–fixed inverse equations for every* $n \geq 1$*;*

(ii) $\beta\alpha = \mathrm{id}_C$ *and* $\alpha\beta = \mathrm{id}_{\Omega/\langle\Phi\rangle}$*;*

(iii) $h_{\deg}(c) : \deg(c) = |\alpha(c)|$ *for every* $c \in C$.

Then OrbitCountCompat(R, C).

Proof. Uses: `prop:tracecore-finite-extension-fixed-cardinality`, `lemma:tracecore-closed-point-degree-compatibility`, `def:tracecore-orbit-counting-compatibility`

Fix $n \geq 1$. The supplied rational–fixed certificates and the first cited proposition replace $\#R_n$ by the cardinality of $\mathrm{Fix}_{\mathcal{D}}(n)$. Partition this finite fixed-point type by the finite orbits of the permutation Φ. An orbit O contributes all of its $|O|$ elements exactly when $|O| \mid n$, and contributes none otherwise. Hence

$$\#\,\mathrm{Fix}_{\mathcal{D}}(n) = \sum_{\substack{O \in \Omega/\langle\Phi\rangle \\ |O| \mid n}} |O|.$$

The two inverse equations for α, β reindex this finite sum by $c \in C$. The supplied degree certificate, exposed by the degree lemma, replaces both the divisibility condition and the summand, giving

$$\#R_n = \sum_{\substack{c \in C \\ \deg(c) \mid n}} \deg(c).$$

Since n was arbitrary, this is the required compatibility family. This proposition is abstract finite-permutation algebra; it makes no claim that the globally finite carrier Ω is the geometric-point set of a positive-dimensional scheme. □

13.3 Local terms and the cohomological trace formula

Definition 13.3.1 (Finite cohomological Weil-complex input). Uses: `def:tracecore-arithmetic-conventions`, `def:tracecore-periodic-point-input`

Lean: `Weil.TraceCore.WeilComplexInput`

A finite cohomological Weil-complex input over X_0 consists of a constructible Weil complex K_0, its geometric pullback K to $X = X_0 \times_{\mathbf{F}_q} k$, a bound B, finite-dimensional E-spaces V_i and automorphisms F_i for $0 \leq i \leq B$, and identifications $V_i \cong H_c^i(X, K)$. It also includes a local stalk $K_{\bar{x}}$ and its Frobenius operator for every rational geometric fixed point. The input stores no trace identity.

Definition 13.3.2 (Local fixed-point term). Uses: `def:tracecore-weil-complex-input`, `def:tracecore-fixed-point-finset`

Lean: `Weil.TraceCore.localFixedTerm`

For $n \geq 1$, define

$$\mathrm{Loc}_n(K_0) := \sum_{x \in \mathrm{Fix}_{\mathcal{D}}(n)} \mathrm{Tr}\big(F_x^{;n} \mid K_{\bar{x}}\big) \in E.$$

The sum is finite by the fixed-point finiteness lemma. If the fixed point is not isolated or the correspondence is not transversal, the displayed term is replaced by the graph–diagonal local intersection trace; that replacement is encoded in the next correspondence Definition.

Definition 13.3.3 (Graph–diagonal correspondence datum)**.** Uses: `def:tracecore-weil-complex-input`, `def:tracecore-fixed-point-finset`

Lean: `Weil.TraceCore.GraphDiagonalCorrespondence`

A graph–diagonal correspondence datum consists of the graph of $\Phi^n : X \to X$, the diagonal Δ_X, their derived intersection, and a local trace morphism

$$\lambda_x^n : \mathrm{FixLoc}_x(\Gamma_{\Phi^n}, \Delta_X, K) \longrightarrow E$$

for each fixed geometric point. The datum stores the support and the restriction maps to stalks. It does not assert that the local intersection is transverse or that λ_x^n equals an ordinary stalk trace.

Definition 13.3.4 (Global categorical trace)**.** Uses: `def:tracecore-weil-complex-input`

Lean: `Weil.TraceCore.globalCategoricalTrace`

For the perfect compact-support complex representing $Rf_!K$, define

$$\mathrm{Glob}_n(K_0) := \sum_{i=0}^{B} (-1)^i \,\mathrm{Tr}(F_i^n \mid V_i) \in E.$$

The finite bound is part of the input; terms above B are zero by the stored vanishing proof field. This Definition is only an alternating cohomological trace and does not assert equality with a local sum.

Lemma 13.3.5 (Local normalization at an isolated fixed point)**.** *Uses:* *def:tracecore-graph-diagonal-correspondence*, *def:tracecore-local-fixed-term*

Lean: *Weil.TraceCore.local_normalization*

At an isolated fixed point where the graph of Φ^n and the diagonal meet with the stored multiplicity-one orientation, the local intersection trace is the stalk trace in $\mathrm{Loc}_n(K_0)$. The construction of the orientation, multiplicity, and equality is

Open obligation: *OBL-TRACEC-LOCAL-TERM*

.

Proof. Uses: `def:tracecore-graph-diagonal-correspondence`, `def:tracecore-local-fixed-term`

The graph–diagonal datum supplies a local complex and a map to the stalk. To prove that this map is an isomorphism in the isolated multiplicity-one case one must construct the local intersection orientation, prove that the derived tensor is the residue field, and check that the induced endomorphism is the declared Frobenius action. These are not consequences of the finite dimensionality fields, so the exact missing comparison is recorded as OBL-TRACEC-LOCAL-TERM. □

Definition 13.3.6 (Triangle additivity datum)**.** Uses: `def:tracecore-weil-complex-input`

Lean: `Weil.TraceCore.TriangleAdditivityDatum`

A triangle additivity datum for $K' \to K \to K'' \to K'[1]$ consists of the three compact-support complexes, the induced Frobenius endomorphisms, the long exact cohomology sequence, and the local stalk triangles at every fixed point. The connecting morphisms and Frobenius commutative squares are stored as data to be checked, not inferred.

Lemma 13.3.7 (Additivity of local and global traces)**.** *Uses:* *def:tracecore-triangle-additivity-datum*, *def:tracecore-local-fixed-term*, *def:tracecore-global-categorical-trace*

Lean: *Weil.TraceCore.trace_additivity*

For every distinguished triangle as above,

$$\mathrm{Loc}_n(K_0) = \mathrm{Loc}_n(K_0') + \mathrm{Loc}_n(K_0''), \qquad \mathrm{Glob}_n(K_0) = \mathrm{Glob}_n(K_0') + \mathrm{Glob}_n(K_0'')$$

are the obligation

Open obligation: *OBL-TRACEC-TRACE-ADDITIVITY*

.

Proof. Uses: def:tracecore-triangle-additivity-datum,def:tracecore-local-fixed-term,def:tracecore-global-categorical-trace

For the global equality, one must split the long exact cohomology sequence into finite short exact sequences, apply additivity of ordinary traces to each short exact sequence, and cancel adjacent connecting-image terms with opposite signs. For the local equality, one must perform the same argument on the stalk triangles and then sum over the finite fixed set. The required exactness, Frobenius squares, and finite cancellation are not fields of the input; the complete construction is therefore the named obligation. □

Definition 13.3.8 (Proper-pushforward trace datum). Uses: def:tracecore-weil-complex-input,def:tracecore-graph-diagonal-correspondence

Lean: Weil.TraceCore.ProperPushforwardTraceDatum

For a proper map $f : X_0 \to Y_0$, a raw proper-pushforward trace datum consists of a candidate exchange map between the graph–diagonal fixed complexes on X and Y, a reverse candidate, the canonical candidate comparison $Rf_!K \to Rf_*K$, and the induced candidate maps on local supports and cohomology. It records the sources, targets, and the two Frobenius actions. It does not assert either inverse equation, the fibrewise local-trace formula, compact-support composition, or any Frobenius commutative square.

Lemma 13.3.9 (Finite-derived proper exchange is invertible). Lean: blueprint-target:lemma:tracecore-proper-pushforward-finite-derived-exchange

Uses: def:tracecore-proper-pushforward-datum,def:weil-pbc-cartesian-square,def:weil-pbc-derived-exchange

For a proper Cartesian square and a bounded-below finite-torsion complex K, the cohomological exchange edge used by the TraceCore datum is the natural isomorphism

$$g^{-1}Rf_*K \xrightarrow{\sim} Rf'_*g'^{-1}K.$$

This assertion concerns the finite derived direct-image edge only.

Proof. Uses: prop:weil-proet-finite-derived-proper-base-change

The displayed arrow is the derived unit–counit exchange map of the cited proper-base-change proposition. That proposition proves it is an isomorphism by finite truncation and the bounded-below hypercohomology comparison, and its construction is natural in K. No graph–diagonal trace class or compact-support comparison occurs in this specialization. □

Lemma 13.3.10 (Finite-derived proper exchange respects Frobenius). Lean: blueprint-target:lemma:tracecore-proper-pushforward-finite-derived-frobenius

Uses: def:tracecore-proper-pushforward-datum,def:weil-pbc-level-map-arrows,def:weil-pbc-frobenius-corner-structures

For a finite-level Frobenius tower datum, the proper exchange edge $v'_n : B_n \to D_n$ obeys

$$v'_n\Theta_{B,n} = \Theta_{D,n}\Phi_{Y'}(v'_n) : \Phi_{Y'}B_n \longrightarrow D_n.$$

This is the finite cohomological exchange square, not a graph-local trace-class square.

Proof. Uses: lemma:weil-pbc-frobenius-level-rectangle

The cited Weil-rectangle lemma proves equivariance of all four typed finite-level edges. Its fourth equation is exactly the displayed equation for v'_n, which the level-arrow certificate identifies with the finite derived proper exchange. Taking that component proves the assertion. □

Lemma 13.3.11 (A proper identity compactification recovers direct image). Lean: blueprint-target:lemma:tracecore-proper-pushforward-identity-compactification

Uses: def:tracecore-proper-pushforward-datum,def:weil-pbc-projection-compactification

For a proper map $f : X \to Y$, the finite-torsion candidate attached to the identity compactification

$$X \xrightarrow{1_X} X \xrightarrow{f} Y$$

agrees naturally with direct image:

$$Rf_!^{(1_X,f)} = Rf_*.$$

This does not compare an independently selected compactification with the identity compactification.

Proof. Uses: `lemma:weil-pbc-projection-formula-identity-compactification`

For $j = 1_X$, extension by zero, inverse image, and the two unit constraints are identities. The cited identity-compactification lemma therefore identifies the candidate $Rf_!^{(1_X,f)}$ with Rf_*, naturally in the finite coefficient complex. Compactification-independence and its inverse equations are not used and are not consequences of this calculation. □

Lemma 13.3.12 (Completed proper base change respects Frobenius). *Uses:* `def:trace-core-proper-pushforward-datum`, `def:weil-pbc-completed-frobenius-structures`

Lean: `Weil.TraceCore.properPushforward_completedFrobenius`

For a completed proper Cartesian square equipped with the Frobenius tower datum of the A3 construction, its completed exchange map satisfies

$$\mathrm{Ex}^{\mathrm{comp}}_{\square,K}\,\Theta_S = \Theta_T\Phi_{Y'}(\mathrm{Ex}^{\mathrm{comp}}_{\square,K}).$$

This is the cohomological proper-base-change square only; it does not assert compatibility of graph–diagonal trace classes with proper direct image.

Proof. Uses: `lemma:weil-pbc-completed-frobenius-equivariance`

The displayed equality is exactly the completed Frobenius-equivariance equation, with the source and target Weil structures transported through the product-cone construction. No local trace class occurs in that theorem, so the final boundary sentence is part of the type of this specialization. □

Lemma 13.3.13 (Trace of an iterate is invariant under equivariant conjugacy). *Uses:* `def:trace-core-global-categorical-trace`

Lean: `Weil.TraceCore.equivariantConjugacy_trace_iterate`

Let V, W be finite-dimensional E-spaces, let F and G be endomorphisms, and let $a : V \xrightarrow{\sim} W$ satisfy $aF = Ga$. Then for every $n \geq 0$,

$$\mathrm{Tr}(F^n \mid V) = \mathrm{Tr}(G^n \mid W).$$

Proof. Uses: `def:tracecore-global-categorical-trace`

Induction on n gives $aF^n = G^n a$, hence $G^n = aF^n a^{-1}$. Choose a basis of V and transport it through a to a basis of W. The two displayed endomorphisms then have the same matrix, so their ordinary traces are equal. □

Lemma 13.3.14 (Degreewise equivariant transport preserves global supertrace). *Uses:* `def:trace-core-global-categorical-trace`

Lean: `Weil.TraceCore.globalSupertrace_transport`

Suppose that for $0 \leq i \leq B$ there are isomorphisms $a_i : V_i \xrightarrow{\sim} W_i$ satisfying $a_i F_i = G_i a_i$. Then for every $n \geq 1$,

$$\sum_{i=0}^{B}(-1)^i\,\mathrm{Tr}(F_i^n \mid V_i) = \sum_{i=0}^{B}(-1)^i\,\mathrm{Tr}(G_i^n \mid W_i).$$

Proof. Uses: `lemma:tracecore-equivariant-conjugacy-trace`

Apply the preceding conjugacy lemma in each degree. Multiplication by the fixed sign $(-1)^i$ preserves the equality, and summing the resulting finite family proves the displayed identity. □

Lemma 13.3.15 (Trace compatibility with proper pushforward). *Uses:* `def:trace-core-proper-pushforward-datum`, `def:trace-core-local-fixed-term`, `def:trace-core-global-categorical-trace`

Lean: `Weil.TraceCore.properPushforward_compatibility`

The local sum and global alternating trace are preserved under proper pushforward, including the fibrewise sum of local terms, both inverse laws, compact-support composition, and Frobenius equivariance. The remaining geometric and six-functor constructions are

Open obligation: `OBL-TRACE-PROPER-PUSHFORWARD`

.

Proof. Uses: `def:tracecore-proper-pushforward-datum`, `lemma:tracecore-proper-pushforward-finite-derived-exchange`, `lemma:tracecore-proper-pushforward-finite-derived-frobenius`, `lemma:tracecore-proper-pushforward-identity-compactification`, `lemma:tracecore-proper-pushforward-completed-frobenius`, `lemma:tracecore-global-supertrace-transport`

The finite derived proper exchange is invertible and Frobenius-equivariant, and its completed A3 realization has the corresponding equivariance square. For a proper map, the particular identity-compactification candidate is literally Rf_*. These facts neither compare that candidate with an independently selected $Rf_!$ nor construct a graph–diagonal trace-class pushforward. The degreewise conjugacy lemma transports the global alternating

trace only after an equivariant compact-support comparison has been supplied. The graph-trace map and inverse equations, fibrewise local sum, selected-$Rf_!$ coherence, remaining Frobenius squares, and E-coefficient transport are therefore still the named assembly obligation. □

Definition 13.3.16 (Open extension-by-zero trace datum). Uses: `def:tracecore-weil-complex-input`, `def:tracecore-graph-diagonal-correspondence`

Lean: `Weil.TraceCore.ExtensionByZeroTraceDatum`

For an open immersion $j : U_0 \hookrightarrow X_0$ with closed complement $i : Z_0 \hookrightarrow X_0$, a raw extension-by-zero trace datum consists of the three candidate arrows

$$j_!j^*K \longrightarrow K \longrightarrow i_*i^*K \longrightarrow,$$

candidate comparison maps in both directions between $Rf_!j_!$ and $R(fj)_!$, and the induced candidate maps on local terms. It records the Frobenius actions and endpoints but does not assert distinguishedness, inverse equations, zero stalks, trace additivity, or Frobenius commutativity.

Lemma 13.3.17 (Finite-level extension by zero has zero complement stalk). Uses: *def:tracecore-extension-zero-datum*, *def:weil-a1-extension-zero-sheaf*

Lean: *Weil.TraceCore.extensionZero_finiteComplementStalk*

Let $L^\bullet$ be a bounded complex of constructible finite-coefficient sheaves on U. For every geometric point $\bar{z} \to Z$, the stalk complex $(j_!L^\bullet)_{\bar{z}}$ is degreewise zero.

Proof. Uses: `lemma:weil-a1-extension-zero-stalks`

In every degree r, the finite-level stalk formula gives $(j_!L^r)_{\bar{z}} = 0$. The stalk differential is therefore the unique map between zero modules. Thus the whole bounded stalk complex is zero. This argument is finite-level and does not by itself construct derived-complete or E-coefficient extension by zero. □

Lemma 13.3.18 (Fixed points split across a stable open–closed pair). Uses: *def:tracecore-fixed-point-finset*

Lean: *Weil.TraceCore.fixedSupport_decomposition*

Let $U, Z \subseteq \Omega$ be complementary Φ-stable predicates. For every $n \geq 1$, restriction of the two inclusions gives a canonical equivalence of finite types

$$\mathrm{Fix}_{\mathcal{D}}(n) \simeq \mathrm{Fix}_U(n) \amalg \mathrm{Fix}_Z(n).$$

Proof. Uses: `def:tracecore-fixed-point-finset`

Send a fixed point to the left or right summand according to the unique side of the complementary pair containing it. The inverse forgets the side. Complementarity proves the two inverse equations, and stability ensures that the restricted point remains fixed by Φ^n. Finiteness is inherited from the ambient fixed-point finite type. □

Lemma 13.3.19 (A fixed-point sum splits across stable supports). Uses: *def:tracecore-fixed-point-finset*

Lean: *Weil.TraceCore.fixedSupport_sum*

For any function $a : \mathrm{Fix}_{\mathcal{D}}(n) \to E$,

$$\sum_{x \in \mathrm{Fix}_{\mathcal{D}}(n)} a(x) = \sum_{x \in \mathrm{Fix}_U(n)} a(x) + \sum_{x \in \mathrm{Fix}_Z(n)} a(x).$$

Proof. Uses: `lemma:tracecore-fixed-support-decomposition`

Reindex the finite sum along the displayed equivalence. A sum over a disjoint union is the sum of the two component sums, which is the asserted identity. □

Lemma 13.3.20 (Underived extension by zero respects Frobenius pullback). Uses: *def:tracecore-extension-zero-datum*, *def:weil-a5-open-extension-exchange-sections*, *def:weil-a1-extension-zero-sheaf*

Lean: *Weil.TraceCore.extensionZero_underivedFrobenius*

For a Frobenius-stable open immersion and a constructible finite-coefficient sheaf L, the Cartesian Frobenius square induces a natural isomorphism

$$\Phi_X^* j_! L \xrightarrow{\sim} j_! \Phi_U^* L.$$

Proof. Uses: prop:weil-a5-underived-open-extension-base-change

Apply underived open-extension base change to the Cartesian square obtained from Frobenius and the stable open immersion. Its sectionwise inverse sheafifies to the inverse of the displayed comparison. This proves only the finite underived square; passage to derived-complete and E-coefficients requires the still separate exactness and coefficient-completion leaves. □

Lemma 13.3.21 (Finite-derived extension by zero has zero complement stalk). Lean: blueprint-target: lemma: tracecore-extension-zero-finite-derived-complement-stalk

Uses: def: tracecore-extension-zero-datum, def: weil-a1-derived-extension-zero, def: weil-ftdc-derived-pullback

Let $K \in \mathcal{D}_n(U)$. For the closed complement $i : Z \hookrightarrow X$, there is a natural isomorphism

$$Li^* Rj_! K \xrightarrow{\sim} 0$$

in $\mathcal{D}_n(Z)$. Consequently, the derived stalk of $Rj_!K$ at every geometric point factoring through Z is the zero object.

Proof. Uses: lemma:weil-sf-well-defined-def-weil-a1-derived-extension-zero,lemma:weil-a1-extension-zero-closed-restriction,lemma:tracecore-extension-zero-finite-complement-stalk,lemma:weil-ftdc-derived-pullback-termwise

Choose a bounded constructible representative $L^\bullet$ of K. The well-definedness certificate represents $Rj_!K$ by the termwise complex $j_!L^\bullet$. The closed-restriction lemma makes $i^* j_! L^r$ the zero sheaf in every degree r; the finite complement stalk lemma records the corresponding pointwise calculation. Derived pullback is represented termwise, so localization identifies this zero complex with $Li^*Rj_!K$, naturally in K. □

Lemma 13.3.22 (Finite-derived extension by zero respects Frobenius pullback). Lean: blueprint-target: lemma: tracecore-extension-zero-finite-derived-frobenius

Uses: def: tracecore-extension-zero-datum, def: weil-a1-derived-extension-zero, def: weil-ftdc-derived-pullback

For a Frobenius-stable open immersion and $K \in \mathcal{D}_n(U)$, the Cartesian Frobenius square induces a natural isomorphism

$$L\Phi_X^* Rj_! K \xrightarrow{\sim} Rj_! L\Phi_U^* K.$$

Its naturality is the finite-torsion derived Frobenius square for every morphism of coefficient complexes.

Proof. Uses: prop:weil-a5-derived-open-extension-base-change

Specialize the finite derived open-extension base-change isomorphism to the Cartesian square defined by the Frobenius endomorphisms and the stable open. That proposition constructs the comparison degreewise from the underived isomorphism and its degreewise inverse, then descends both through Verdier localization. Its asserted naturality is exactly the stated square. □

Lemma 13.3.23 (Extension-by-zero and localization compatibility). Uses: def: tracecore-extension-zero-datum, def: tracecore-local-fixed-term, def: tracecore-global-categorical-trace

Lean: Weil.TraceCore.extensionZero_compatibility

The local and global trace additivity needed to pass from U_0 and Z_0 to X_0, together with the derived E-coefficient comparison $Rf_!j_! \simeq R(fj)_!$, is

Open obligation: OBL-TRACEC-EXTENSION-ZERO

.

Proof. Uses: def:tracecore-extension-zero-datum,lemma:tracecore-trace-additivity,lemma:tracecore-extension-zero-finite-derived-complement-stalk,lemma:tracecore-fixed-support-sum,lemma:tracecore-extension-zero-finite-derived-frobenius

At finite-torsion derived level, extension by zero has zero complement stalks, the fixed set and every finite local sum split over the stable open–closed pair, and derived extension by zero commutes naturally with Frobenius pullback. The finite localization triangle is discharged separately later in the blueprint. What remains here begins with the normalization/completion and scalar-extension passage to E; it also requires graph-local excision, both compact-support composition inverse equations, and local/global trace additivity. None of those assertions is supplied by the finite derived lemmas above. □

Definition 13.3.24 (Grothendieck–Lefschetz trace-formula predicate). Uses: def:tracecore-local-fixed-term,def:tracecore-global-categorical-trace,def:tracecore-graph-diagonal-correspondence,def:tracecore-triangle-additivity-datum,def:tracecore-proper-pushforward-datum,def:tracecore-extension-zero-datum

Lean: Weil.TraceCore.IsGrothendieckLefschetzTraceFormula

Define $\mathrm{GLTrace}(K_0, n)$ to be the conjunction of:

(i) the graph–diagonal local sum is defined and finite;

(ii) every isolated multiplicity-one point has the stalk normalization;

(iii) local and global traces are additive in distinguished triangles;

(iv) proper pushforward transports both sides;

(v) extension by zero and localization transport both sides; and

(vi) $\mathrm{Loc}_n(K_0) = \mathrm{Glob}_n(K_0)$.

This is a predicate, not an existence assertion.

Proposition 13.3.25 (Grothendieck–Lefschetz formula at every iterate). *Uses:* *def:tracecore-trace-formula-predicate, def:tracecore-local-fixed-term, def:tracecore-global-categorical-trace*

Lean: *Weil.TraceCore.grothendieckLefschetz*

For every separated finite-type $X_0/\mathbf{F}_q$, constructible Weil complex K_0, and $n \geq 1$,

$$\sum_{x \in X_0(\mathbf{F}_{q^n})} \mathrm{Tr}(F_x^n \mid K_{\bar{x}}) = \sum_i (-1)^i \,\mathrm{Tr}(F_i^n \mid H_c^i(X, K))$$

is the obligation

Open obligation: *OBL-TRACEC-GLOBAL-TRACE*

.

Proof. Uses: def:tracecore-trace-formula-predicate,lemma:tracecore-local-normalization,lemma:tracecore-trace-additivity,lemma:tracecore-proper-pushforward-compatibility,lemma:tracecore-extension-zero-compatibility

The induction on a finite stratification starts with a proper smooth stratum, where graph–diagonal intersection gives the local sum, and uses the extension-by-zero triangle for the complement. Each induction step needs the five displayed proof fields in the predicate: local finiteness and normalization, triangle additivity, proper-pushforward transport, and extension-by-zero transport. Since the corresponding construction is not an earlier leaf, the complete formula remains the named global-trace obligation. □

13.4 Determinants, trace–log identities, and rationality

Definition 13.4.1 (Bounded Frobenius factor family). Uses: def:tracecore-weil-complex-input,def:tracecore-arithmetic-conventions

Lean: Weil.TraceCore.BoundedFrobeniusFactorFamily

For a bound B, define

$$P_i(T) := \det(1 - TF_i \mid V_i) \in E[T] \qquad (0 \leq i \leq B),$$

and set $P_i(T) = 1$ for $i > B$. The finite product

$$\Delta(T) := \prod_{i=0}^{B} P_i(T)^{(-1)^{i+1}}$$

is interpreted in $E[[T]]$, with negative exponent the formal inverse of the constant-one series.

Lemma 13.4.2 (Determinant factors are normalized polynomials). *Uses:* *def:tracecore-bounded-frobenius-factor-family*

Lean: *Weil.TraceCore.factor_normalization*

Every P_i is a polynomial with $P_i(0) = 1$, and its degree is at most $\dim_E V_i$.

Proof. Uses: def:tracecore-bounded-frobenius-factor-family,def:weil-based-reciprocal-determinant,lemma:weil-based-reciprocal-determinant-constant

The determinant of a finite square matrix is a finite sum of products, hence is a polynomial. Substitution $T = 0$ leaves $\det(I) = 1$. Every term in the determinant contains at most one entry from each of the $\dim_E V_i$ rows, so its degree is at most that dimension. The cited finite-dimensional determinant lemma identifies this coordinate expression with the basis-independent determinant in the Definition. □

Lemma 13.4.3 (Degreewise trace–log–determinant identity). *Uses:* def:tracecore-bounded-frobenius-factor-family

Lean: Weil.TraceCore.degreewiseTraceLogDet

For every $0 \le i \le B$,

$$-\operatorname{Log}(P_i(T)) = \sum_{n\ge 1} \operatorname{Tr}(F_i^n)\frac{T^n}{n} \quad \text{in } E[[T]].$$

Proof. Uses: def:tracecore-bounded-frobenius-factor-family, thm:weil-based-trace-log-det

Apply the previously audited finite-dimensional trace–log–determinant theorem to the based endomorphism datum underlying (V_i, F_i). Its reciprocal determinant is exactly P_i by the factor Definition, and its matrix trace is the ordinary trace of F_i^n. Substitution of these two identifications gives the displayed equality coefficient by coefficient. □

Lemma 13.4.4 (Alternating determinant logarithm). *Uses:* def:tracecore-bounded-frobenius-factor-family

Lean: Weil.TraceCore.alternatingTraceLog

$$-\operatorname{Log}(\Delta(T)) = \sum_{n\ge 1}\left(\sum_{i=0}^{B}(-1)^i \operatorname{Tr}(F_i^n)\right)\frac{T^n}{n}.$$

Proof. Uses: lemma:tracecore-degreewise-trace-log-det, def:tracecore-bounded-frobenius-factor-family

The product defining Δ is finite. Formal logarithm is additive on finite products of constant-one series. For exponent $(-1)^{i+1}$, the negative logarithm contributes $(-1)^i$ times the degreewise series from the preceding lemma. Summing over $0 \le i \le B$ and interchanging two finite sums gives the displayed coefficient formula. □

Definition 13.4.5 (Trace-datum compatibility predicate). Uses: def:tracecore-counting-series, def:tracecore-bounded-frobenius-factor-family

Lean: Weil.TraceCore.CountingTraceCompatibility

Define CountTraceCompat$(R, (V_i, F_i))$ to be the family of equalities

$$\#R_n = \sum_{i=0}^{B}(-1)^i \operatorname{Tr}(F_i^n) \qquad (n \ge 1).$$

It is a predicate and does not supply cohomology or a trace proof.

Proposition 13.4.6 (Cohomological determinant factorization). *Uses:* def:tracecore-counting-series, def:tracecore-bounded-frobenius-factor-family, def:tracecore-counting-trace-compatibility

Lean: Weil.TraceCore.zeta_determinant_factorization

If CountTraceCompat$(R, (V_i, F_i))$, *then*

$$Z_R(T) = \prod_{i=0}^{B} P_i(T)^{(-1)^{i+1}} = \Delta(T).$$

Proof. Uses: def:tracecore-counting-series, def:tracecore-counting-trace-compatibility, lemma:tracecore-alternating-trace-log, def:tracecore-bounded-frobenius-factor-family

The coefficient of T^n in $\operatorname{Log}(Z_R)$ is $\#R_n/n$. Replace $\#R_n$ using the compatibility predicate. The result is the right side of the alternating trace–log identity. Both series have constant coefficient one, so exponentiating and using $\operatorname{Exp}(-\operatorname{Log}(\Delta)) = \Delta$ yields the displayed equality. □

Definition 13.4.7 (Trace-core rational-function candidate). Uses: def:tracecore-bounded-frobenius-factor-family

Lean: Weil.TraceCore.RationalFunctionCandidate

Define

$$R_{\mathrm{tr}}(T) := \frac{\prod_{i\le B,\ i \text{ ă}odd} P_i(T)}{\prod_{i\le B,\ i \text{ ă}even} P_i(T)} \in E(T),$$

where the numerator and denominator are finite polynomial products and the denominator has value one at $T = 0$. Its Taylor germ is therefore defined at zero.

Proposition 13.4.8 (Rationality conditional on the geometric trace formula). *Uses:* *def:tracecore-counting-series, def:tracecore-rational-function-candidate, def:tracecore-counting-trace-compatibility, def:tracecore-trace-formula-predicate*

Lean: *Weil.TraceCore.rationality*

For the geometric realization of the finite cohomological input, the Taylor series of $R_{\mathrm{tr}}(T)$ equals $Z_R(T)$ once the every-iterate Grothendieck–Lefschetz equality has been constructed. The missing geometric construction is

Open obligation: *OBL-TRACEC-GLOBAL-TRACE*

.

Proof. Uses: prop:tracecore-grothendieck-lefschetz,prop:tracecore-zeta-determinant-factorization,def:tracecore-rational-function-candidate

The geometric trace formula supplies CountTraceCompat$(R,(V_i,F_i))$ for every positive iterate. The determinant factorization proposition then identifies Z_R with Δ. Splitting the finite product according to parity identifies Δ with the Taylor germ of R_{tr}. The first step is exactly the unresolved global-trace obligation, so this proposition is conditional and does not promote that obligation to a theorem. □

13.5 Weight route from the supplied PDF

Route decision and DAG. The PRIMARY route is

$$W0 \to BASE \to AS/FT/PERV/REAL,$$
$$REAL \to PUR + MON + LFT \to AHC1 \to CURVE \to HC,$$
$$HC \to TAME \to DEV6 \to RFI \to DER \to PD \to RH.$$

The earlier W3 square-improvement branch is FALLBACK historical material only and is not an allowed premise of any PRIMARY proposition.

13.5.1 Conventions and the foundational boundary

Definition 13.5.1 (Weight datum). Uses:

Lean: conditional-interface:def:weil-wpdf-weight-datum

Fix $k = \mathbf{F}_q$, $\ell \neq \mathrm{char}\, k$, an embedding $\iota : \overline{\mathbf{Q}}_\ell \hookrightarrow \mathbf{C}$, and geometric Frobenius. An eigenvalue a has weight w when $|\iota(a)| = q^{w/2}$. A Tate twist (r) multiplies a by q^{-r}.

Lemma 13.5.2 (Weight algebra). *Uses:*

Lean: *conditional-interface:lemma:weil-wpdf-weight-algebra*

If a has weight w, then $q^{-r}a$ has weight $w-2r$, and $a^{-1}q^d$ has weight $-w+2d$.

Proof. Uses:

The two moduli are $q^{-r}q^{w/2} = q^{(w-2r)/2}$ and $q^d q^{-w/2} = q^{(-w+2d)/2}$. □

W0 (mathlib target, not yet marked closed). The preceding real-exponent calculation, finite direct-sum maximum rule, and the reciprocal-spectrum calculation are to be closed only by exact Mathlib declarations recorded in the manifest. No geometric weight result is asserted by W0.

Definition 13.5.3 (Required six-operation datum). Uses:

Lean: conditional-interface:def:weil-wpdf-required-six-operation-datum

For each finite-type map used below, require typed Weil categories, f^*, Rf_*, $Rf_!$, $f^!$, tensor product, localization triangles, base-change maps, projection-formula maps, duality maps, Frobenius squares, and the trace and L-function maps. The datum includes a map and an inverse with both inverse equations for every comparison invoked below.

Proposition 13.5.4 (Why the datum cannot be an imported BASE interface). *Uses:*

Lean: *conditional-interface:prop:weil-wpdf-why-the-datum-cannot-be-an-imported-base-interface*

The desired conclusion that $Rf_!$ preserves upper weights is not a field of the required datum.

Proof. Uses:
For $f : X \to \operatorname{Spec} k$, that conclusion bounds the Frobenius weights of $H^i_c(X, -)$, which is the target conclusion of this route. Including it would make the final argument circular. □

Open obligation: `OBL-WPDF-BASE`

OPEN WPDF-BASE. Construct the required datum from the present Lean libraries, recording exact imports and declaration names for each map, inverse, inverse equation, and Frobenius square. The PDF's B1–B6 are not available before this construction.

13.5.2 W0: Frobenius modules and derived weights

Definition 13.5.5 (Finite Frobenius module). Uses:

Lean: `conditional-interface:def:weil-wpdf-finite-frobenius-module`
A finite Frobenius module is a finite-dimensional $\overline{\mathbf{Q}}_\ell$-vector space V with an invertible endomorphism Φ_V. Its spectrum is the multiset of roots in $\mathbf{C}$ obtained by applying ι to the characteristic polynomial of Φ_V.

Definition 13.5.6 (Pure and mixed Frobenius modules). Uses:

Lean: `conditional-interface:def:weil-wpdf-pure-and-mixed-frobenius-modules`
A finite Frobenius module is pure of weight w when every member a of its spectrum satisfies $|a| = q^{w/2}$. It is mixed of weights at most w when it has a finite Frobenius-stable filtration whose nonzero graded factors are pure of weights at most w. "At least w" is defined by reversing the last inequality.

Definition 13.5.7 (Derived upper and lower weights). Uses:

Lean: `conditional-interface:def:weil-wpdf-derived-upper-and-lower-weights`
For a bounded Weil complex K, write $K \in D_{\le w}$ when $H^a(K)$ is mixed of weights at most $w + a$ for every a. Define $K \in D_{\ge w}$ by the corresponding lower inequality, and call K pure of derived weight w when both conditions hold.

Definition 13.5.8 (Frobenius-stable exact sequence). Uses:

Lean: `conditional-interface:def:weil-wpdf-frobenius-stable-exact-sequence`
An exact Frobenius sequence is a short exact sequence $0 \to V' \to V \to V'' \to 0$ whose maps commute with Frobenius.

Lemma 13.5.9 (Characteristic polynomial in an exact Frobenius sequence). *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-characteristic-polynomial-in-an-exact-frobenius-sequence`
For an exact Frobenius sequence, $\chi_V(T) = \chi_{V'}(T)\chi_{V''}(T)$.

Proof. Uses:
Choose a basis of V' and extend it to one of V. Frobenius has a block upper-triangular matrix with diagonal blocks the matrices on V' and V''. The determinant of $TI - \Phi_V$ is the product of the diagonal-block determinants. □

Proposition 13.5.10 (Subquotient and extension bounds). *Uses:*

Lean: `conditional-interface:prop:weil-wpdf-subquotient-and-extension-bounds`
If V is mixed of weights at most w, then every Frobenius-stable subquotient of V is mixed of weights at most w. If V' and V'' are mixed of weights at most w in an exact Frobenius sequence, then V is mixed of weights at most w.

Proof. Uses:
Intersect a mixed filtration of V with a stable subspace and pass it to a quotient. Each new graded piece is a subquotient of a pure graded piece; the characteristic-polynomial lemma shows that its spectrum is a submultiset and so has the same weight. For the extension assertion, lift a mixed filtration of V'' to V and place a mixed filtration of V' below it. The graded pieces are precisely graded pieces of V' or V''. □

Definition 13.5.11 (Tensor, dual, and Tate operations). Uses:

Lean: `conditional-interface:def:weil-wpdf-tensor-dual-and-tate-operations`
For finite Frobenius modules set $\Phi_{V\otimes W} = \Phi_V \otimes \Phi_W$, $\Phi_{V^\vee} = (\Phi_V^{-1})^\vee$, and $\Phi_{V(r)} = q^{-r}\Phi_V$.

Lemma 13.5.12 (Tensor, dual, and twist weights). *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-tensor-dual-and-twist-weights`

If V and W are pure of weights v and w, then $V \otimes W$ is pure of weight $v+w$, $V^\vee$ is pure of weight $-v$, and $V(r)$ is pure of weight $v-2r$.

Proof. Uses:

Over $\mathbf{C}$, triangularize the two Frobenius operators. The diagonal entries on the tensor product are products ab, on the dual are a^{-1}, and on the twist are $q^{-r}a$. Their moduli are respectively $q^{(v+w)/2}$, $q^{-v/2}$, and $q^{(v-2r)/2}$. □

Proposition 13.5.13 (Mixed tensor, dual, and twist bounds). *Uses:*

Lean: `conditional-interface:prop:weil-wpdf-mixed-tensor-dual-and-twist-bounds`

If V and W have upper weights v and w, then $V \otimes W$ has upper weight $v+w$, $V^\vee$ has lower weight $-v$, and $V(r)$ has upper weight $v-2r$.

Proof. Uses:

Tensor the two finite stable filtrations and filter the tensor product by the sum of filtration indices. Its graded factors are tensor products of pure factors, so the preceding lemma bounds them by $v+w$. Dualizing reverses a stable filtration and changes the pure weights to their negatives. Twisting the given filtration changes every pure weight by $-2r$. □

Lemma 13.5.14 (Weight separation). *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-weight-separation`

If V is pure of weight v, W is pure of weight w, and $v \neq w$, every Frobenius-equivariant map $V \to W$ is zero.

Proof. Uses:

The two spectra have disjoint complex moduli. Their minimal polynomials are therefore coprime. Bezout gives polynomials A, B with $A\mu_V + B\mu_W = 1$. For a Frobenius-equivariant map u, the relations $\mu_V(\Phi_V) = 0$ and $\mu_W(\Phi_W) = 0$ imply $u = A(\Phi_W)\mu_V(\Phi_W)u + B(\Phi_W)\mu_W(\Phi_W)u = 0$. □

Definition 13.5.15 (Bounded Frobenius spectral sequence). Uses:

Lean: `conditional-interface:def:weil-wpdf-bounded-frobenius-spectral-sequence`

A bounded Frobenius spectral sequence is a finite-page spectral sequence $E_r^{a,b}$ of finite Frobenius modules, with Frobenius-equivariant differentials and a finite Frobenius-stable filtration on each abutment H^n whose graded factors are $E_\infty^{a,n-a}$.

Proposition 13.5.16 (Spectral-sequence upper bound). *Uses:*

Lean: `conditional-interface:prop:weil-wpdf-spectral-sequence-upper-bound`

In a bounded Frobenius spectral sequence, if every $E_1^{a,b}$ has weights at most $w+a+b$, then H^n has weights at most $w+n$.

Proof. Uses:

For each page, $E_{r+1}^{a,b}$ is the quotient of the kernel of a Frobenius-equivariant differential on $E_r^{a,b}$; the subquotient part of the extension proposition preserves its upper bound. Thus every $E_\infty^{a,n-a}$ has upper weight $w+n$. The abutment filtration and the extension part of that proposition give the same bound for H^n. □

Proposition 13.5.17 (Derived truncation devissage). *Uses:*

Lean: `conditional-interface:prop:weil-wpdf-derived-truncation-devissage`

If every cohomology module $H^a(K)$ is mixed of weights at most $w+a$, then $K \in D_{\leq w}$; conversely, $K \in D_{\leq w}$ gives these bounds.

Proof. Uses:

Both assertions are the two directions of the defining predicate $K \in D_{\leq w}$. The point of naming the statement is that later use of a Postnikov tower changes a derived bound into these individual, indexed Frobenius-module bounds without changing the shift convention. □

13.5.3 BASE ownership and unresolved interface

Definition 13.5.18 (A1–A4 availability table)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-a1-a4-availability-table`

The following table records only ownership, not unconditional availability.

owner	available construction or conditional export
A1	$j_!$, Rp_*, $Rf_! = Rp_* j_!$, compactification comparison, and the compact-support exchange map
A2	ℓ-adic coefficient categories, derived completion, tensor construction, finite reductions, and scalar-extension constructions
A3	typed finite/complete proper, smooth, and compact base-change maps; their isomorphism statements remain conditional on listed leaves
A4	$Rf^!$, Verdier duality, projection formula, trace, and Poincare comparison maps; all global isomorphism claims remain OPEN.

Open obligation: `OBL-WPDF-BASE-A1`

OPEN WPDF-BASE-A1. A1's constructibility, derived-resolution, proper-pushforward, and rational-lift leaves must be closed before its compact-support construction can serve as a Fourier input.

Open obligation: `OBL-WPDF-BASE-A2`

OPEN WPDF-BASE-A2. The complete coefficient rebinding, monoidal coherence, constructibility, and scalar-extension leaves remain open; a derived category name alone does not supply the needed comparisons.

Open obligation: `OBL-WPDF-BASE-A3`

OPEN WPDF-BASE-A3. Proper base change, smooth base change, compact base change, direct-image boundedness, and constructibility have named comparison maps but require their exact finite-level and completion leaves before they may be used as isomorphisms.

Open obligation: `OBL-WPDF-BASE-A4`

OPEN WPDF-BASE-A4. Extraordinary pullback, Verdier biduality and exchange, proper-support comparison, projection formula, global finiteness, Poincare perfectness, and Frobenius-duality normalization remain construction targets. Therefore the curve duality and trace portions of PDF-BASE are not available in this module.

13.5.4 Fourier line

Definition 13.5.19 (Additive character over finite extensions)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-additive-character-over-finite-extensions`

Fix a nontrivial homomorphism $\psi : (\mathbf{F}_p, +) \to \overline{\mathbf{Q}}_\ell^\times$. For $n \geq 1$ put $\psi_n(a) = \psi(\mathrm{Tr}_{\mathbf{F}_{q^n}/\mathbf{F}_p}(a))$.

Definition 13.5.20 (Artin–Schreier projector)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-artin-schreier-projector`

Let $\pi : \mathbf{A}^1_z \to \mathbf{A}^1_t$ be $z \mapsto z^p - z$. Translation by $a \in \mathbf{F}_p$ acts on $\pi_*\overline{\mathbf{Q}}_\ell$. Define

$$e_\psi = p^{-1} \sum_{a \in \mathbf{F}_p} \psi(a)^{-1}[a] \in \mathrm{End}(\pi_*\overline{\mathbf{Q}}_\ell).$$

Lemma 13.5.21 (Artin–Schreier projector idempotence)**.** *Uses:*

Lean: *`conditional-interface:lemma:weil-wpdf-artin-schreier-projector-idempotence`*

The endomorphism e_ψ satisfies $e_\psi^2 = e_\psi$.

Proof. Uses:

The coefficient of $[c]$ in e_ψ^2 is $p^{-2}\sum_a \psi(a)^{-1}\psi(c-a)^{-1}$. Additivity gives $\psi(a)^{-1}\psi(c-a)^{-1} = \psi(c)^{-1}$, and the p summands yield the coefficient $p^{-1}\psi(c)^{-1}$ of $[c]$ in e_ψ. □

Definition 13.5.22 (Finite Artin–Schreier character pairing). Uses: def:weil-wpdf-additive-character-over-finite-extensions

Lean: blueprint-target:def:weil-wpdf-as-character-pairing

For additive characters $\psi, \chi : (\mathbf{F}_p, +) \to \overline{\mathbf{Q}}_\ell^\times$, define

$$\langle \psi, \chi \rangle_p := p^{-1} \sum_{a \in \mathbf{F}_p} \psi(a)^{-1} \chi(a).$$

This definition involves only the finite translation algebra; it makes no claim about the existence, lissity, or Frobenius structure of an Artin–Schreier sheaf.

Lemma 13.5.23 (Finite Artin–Schreier character orthogonality). *Uses: def:weil-wpdf-as-character-pairing*

Lean: blueprint-target:lemma:weil-wpdf-as-character-pairing-orthogonality

For additive characters ψ, χ,

$$\langle \psi, \chi \rangle_p = \begin{cases} 1, & \psi = \chi, \\ 0, & \psi \neq \chi. \end{cases}$$

Proof. Uses: def:weil-wpdf-as-character-pairing

If $\psi = \chi$, every summand is one, so the sum is p and its normalized value is one. Suppose $\psi \neq \chi$ and put $\theta = \psi^{-1}\chi$. Then θ is a nontrivial additive character, so there is $b \in \mathbf{F}_p$ with $\theta(b) \neq 1$. If $S = \sum_a \theta(a)$, translation by b permutes $\mathbf{F}_p$ and gives

$$S = \sum_a \theta(a + b) = \theta(b) S.$$

Thus $(1 - \theta(b))S = 0$. The coefficient field has characteristic zero and $1 - \theta(b) \neq 0$, hence $S = 0$. Dividing by the nonzero scalar p proves the second case. □

Lemma 13.5.24 (Artin–Schreier projector orthogonality and translation). *Uses: def:weil-wpdf-artin-schreier-projector*

Lean: blueprint-target:lemma:weil-wpdf-as-projector-orthogonality

For $b \in \mathbf{F}_p$ and additive characters ψ, χ,

$$[b] e_\psi = e_\psi [b] = \psi(b) e_\psi, \qquad e_\psi e_\chi = \begin{cases} e_\psi, & \psi = \chi, \\ 0, & \psi \neq \chi. \end{cases}$$

Proof. Uses: def:weil-wpdf-artin-schreier-projector,lemma:weil-wpdf-as-character-pairing-orthogonality

The coefficient of $[c]$ in $[b]e_\psi$ is $p^{-1}\psi(c - b)^{-1} = p^{-1}\psi(b)\psi(c)^{-1}$; hence $[b]e_\psi = \psi(b)e_\psi$. The translation algebra of the additive group is commutative, so the same equation holds for right translation. The coefficient of $[c]$ in the product is

$$p^{-2} \sum_a \psi(a)^{-1} \chi(c - a)^{-1} = p^{-2} \chi(c)^{-1} \sum_a \psi(a)^{-1} \chi(a).$$

The preceding orthogonality lemma makes this coefficient $p^{-1}\chi(c)^{-1}$ when $\psi = \chi$ and zero otherwise. These are exactly the coefficients of e_ψ and of the zero endomorphism, respectively. □

13.6 The Artin–Schreier finite etale cover

Definition 13.6.1 (Artin–Schreier coordinate datum). Uses:

Lean: blueprint-target:def:weil-as-cover-coordinate-datum

Let $k = \mathbf{F}_q$ have characteristic p, where $q = p^r$. Put

$$A = k[t], \qquad B = k[z], \qquad \iota : A \longrightarrow B, \quad t \longmapsto z^p - z,$$

and write

$$Y = \operatorname{Spec} A = \mathbf{A}^1_{t,k}, \qquad X = \operatorname{Spec} B = \mathbf{A}^1_{z,k}, \qquad \pi : X \longrightarrow Y.$$

Let $G = (\mathbf{F}_p)_k = \coprod_{a \in \mathbf{F}_p} \operatorname{Spec} k$ be the constant finite group scheme. No sheaf summand or Fourier transform is part of this datum. For each $a \in \mathbf{F}_p$, define the k-algebra automorphism $\tau_a^\sharp : B \to B$ by $z \mapsto z + a$, and denote its contravariant affine morphism by $\tau_a : X \to X$.

Lemma 13.6.2 (Coordinate presentation and finite freeness). *Uses:* def:weil-as-cover-coordinate-datum

Lean: blueprint-target:lemma:weil-as-cover-coordinate-free

There is an A-algebra isomorphism

$$B \cong A[Z]/(Z^p - Z - t),$$

under which B is finite free of rank p with ordered A-basis $1, z, \dots, z^{p-1}$. In particular, π is finite and faithfully flat.

Proof. Uses: def:weil-as-cover-coordinate-datum

Define

$$\Phi : A[Z]/(Z^p - Z - t) \longrightarrow B, \qquad \overline{Z} \longmapsto z, \quad t \longmapsto z^p - z.$$

Conversely, the k-algebra map $B \to A[Z]/(Z^p - Z - t)$ sending z to $\overline{Z}$ is A-linear because $\overline{Z}^p - \overline{Z} = t$. The two composites fix the generators z and $\overline{Z}, t$, so they are identities.

The polynomial $Z^p - Z - t$ is monic of degree p. Euclidean division by this polynomial writes every class uniquely as

$$c_0(t) + c_1(t)\overline{Z} + \cdots + c_{p-1}(t)\overline{Z}^{p-1}.$$

Thus the displayed powers form an A-basis. A finite free algebra is finite and flat, and a nonzero free module is faithful, proving the final assertion. □

Definition 13.6.3 (Raw standard-etale data for the Artin–Schreier polynomial). Uses: def:weil-as-cover-coordinate-datum

Lean: blueprint-target:def:weil-as-cover-standard-etale-raw-data

In $A[Z]$ define

$$f(Z) = Z^p - Z - t, \qquad g(Z) = 1, \qquad u(Z) = -1, \qquad v(Z) = 0, \qquad n = 0.$$

These are only the polynomial and Bezout candidates; the standard-etale identity is not part of the Definition.

Lemma 13.6.4 (Standard-etale Bezout identity). *Uses:* def:weil-as-cover-standard-etale-raw-data

Lean: blueprint-target:lemma:weil-as-cover-standard-etale-bezout

The raw data satisfy

$$f'(Z)u(Z) + f(Z)v(Z) = g(Z)^n = 1.$$

Proof. Uses: def:weil-as-cover-coordinate-datum,def:weil-as-cover-standard-etale-raw-data

Characteristic p gives

$$f'(Z) = pZ^{p-1} - 1 = -1.$$

Substitution of $u = -1$, $v = 0$, $g = 1$, and $n = 0$ gives $(-1)(-1) + f \cdot 0 = 1 = 1^0$. □

Lemma 13.6.5 (Standard-etale transport and scheme etaleness). *Uses:* def:weil-as-cover-coordinate-datum

Lean: blueprint-target:lemma:weil-as-cover-etale

The morphism π is finite etale of constant degree p.

Proof. Uses: def:weil-as-cover-coordinate-datum,def:weil-as-cover-standard-etale-raw-data,lemma:weil-as-cover-coordinate-free,lemma:weil-as-cover-standard-etale-bezout

Apply the pinned mathlib structure `StandardEtalePair` to the monic polynomial f, the polynomial $g = 1$, and the witnesses u, v, n of Lemma 13.6.4. Its associated algebra $\mathcal{P}$.`Ring` has an `Algebra.Etale` instance. The exact comparison `StandardEtalePair.equivAwayAdjoinRoot` identifies it with

$$\text{Localization.Away}(\text{AdjoinRoot.mk}(f, g)).$$

Because $g = 1$, the displayed localization is canonically $A[Z]/(f)$. Compose this equivalence with the explicit coordinate isomorphism of Lemma 13.6.2. One application of `Algebra.Etale.of_equiv` transports the etale-algebra instance to B. Finally instantiate `HasRingHomProperty.Spec_iff` in the `AlgebraicGeometry` namespace with $P =$ `AlgebraicGeometry.Etale` and $Q =$ `RingHom.Etale`; its right-hand side is precisely the etaleness of $A \to B$, and its left-hand side is the etaleness of π. The finite and constant-degree assertions are the finite-free rank-p conclusion of Lemma 13.6.2. □

Lemma 13.6.6 (Translation action over the quotient). *Uses:* def:weil-as-cover-coordinate-datum
Lean: blueprint-target:lemma:weil-as-cover-translation-action
For $a \in \mathbf{F}_p$, the formula

$$\tau_a : X \longrightarrow X, \qquad z \longmapsto z + a,$$

defines a Y-automorphism. The equations $\tau_0 = \mathrm{id}_X$ and $\tau_a \circ \tau_b = \tau_{a+b}$ define a G-action on X over Y.

Proof. Uses: def:weil-as-cover-coordinate-datum
On coordinate rings, substitution gives

$$(z + a)^p - (z + a) = z^p - z + (a^p - a) = z^p - z,$$

because $a^p = a$ in $\mathbf{F}_p$. Hence τ_a fixes the image of A and is a Y-map. Translation by $-a$ is its inverse. The unit and composition equations follow by substituting z, then $z+b$, then $z+b+a$; addition in $\mathbf{F}_p$ is commutative. □

Lemma 13.6.7 (Simple transitivity on geometric fibres). *Uses:* def:weil-as-cover-coordinate-datum
Lean: blueprint-target:lemma:weil-as-cover-geometric-fibres
For every algebraically closed extension Ω/k and every $t_0 \in \Omega$, the group $\mathbf{F}_p$ acts freely and transitively on the geometric fibre

$$X_{t_0}(\Omega) = \{z \in \Omega : z^p - z = t_0\}.$$

The fibre has exactly p points.

Proof. Uses: def:weil-as-cover-coordinate-datum, lemma:weil-as-cover-translation-action
The polynomial $Z^p - Z - t_0$ has a root in Ω and has derivative -1, so all its roots are distinct. If z, z' are two roots, then $d = z' - z$ satisfies $d^p - d = 0$. In characteristic p,

$$T^p - T = \prod_{a \in \mathbf{F}_p} (T - a),$$

and the factors are distinct. Therefore there is a unique $a \in \mathbf{F}_p$ with $z' = z + a$. Existence gives transitivity, uniqueness gives freeness, and the p translations of one root give all p points. □

Lemma 13.6.8 (Invariant ring). *Uses:* def:weil-as-cover-coordinate-datum
Lean: blueprint-target:lemma:weil-as-cover-invariants
The invariant ring is

$$B^G = k[z]^{\mathbf{F}_p} = k[z^p - z] = A.$$

Proof. Uses: def:weil-as-cover-coordinate-datum, lemma:weil-as-cover-coordinate-free, lemma:weil-as-cover-translation-action
The inclusion $A \subseteq B^G$ follows from Lemma 13.6.6. For the reverse inclusion, use the free basis of Lemma 13.6.2 to write an invariant $h \in B$ uniquely as

$$h = \sum_{i=0}^{p-1} c_i(t) z^i, \qquad c_i(t) \in A.$$

Put $K = k(t)$ and

$$P(T) = \sum_{i=0}^{p-1} c_i(t) T^i \in K[T].$$

Let α be a root of $Z^p - Z - t$ in a separable closure of K. For each $a \in \mathbf{F}_p$, the equality $h(z + a) = h(z)$ in B, followed by the K-algebra evaluation $z \mapsto \alpha$, gives

$$P(\alpha + a) = P(\alpha).$$

Thus

$$Q(T) = P(T) - P(\alpha) \in K(\alpha)[T]$$

has the p roots $\alpha + a$ for $a \in \mathbf{F}_p$. These roots are pairwise distinct, whereas $\deg Q \leq p - 1 < p$. The root bound forces $Q = 0$. Comparing the coefficients of T^i in $K(\alpha)[T]$ for $1 \leq i \leq p - 1$ gives $c_i(t) = 0$. Hence $h = c_0(t) \in A$. □

Lemma 13.6.9 (Affine quotient universal property). *Uses:* def:weil-as-cover-coordinate-datum
Lean: blueprint-target:lemma:weil-as-cover-affine-quotient
The morphism $\pi : X \to Y$ is the affine categorical quotient of X by G.

Proof. Uses: def:weil-as-cover-coordinate-datum,lemma:weil-as-cover-invariants,lemma:weil-as-cover-translation-action

Let $Z = \operatorname{Spec} R$ be affine. If a morphism $u : X \to Z$ is G-invariant, its coordinate-ring map $u^\sharp : R \to B$ has image in $B^G = A$ by Lemma 13.6.8. It therefore has a unique factorization $R \to A \to B$, and reversing arrows gives a unique factorization $X \xrightarrow{\pi} Y \to Z$. Conversely, every morphism that factors through π is G-invariant because every $\tau_a^\sharp$ fixes A. This proves the required bijection of morphism sets, natural in the affine target Z. □

Definition 13.6.10 (Artin–Schreier torsor comparison candidates). Uses: def:weil-as-cover-coordinate-datum

Lean: blueprint-target:def:weil-as-cover-torsor-candidates

On $X \times_Y X$ write z_1, z_2 for the two coordinate functions and define $d = z_2 - z_1$. Define

$$C = k[z_1, z_2]/(z_1^p - z_1 - z_2^p + z_2) \cong k[z_1, d]/(d^p - d).$$

For $a \in \mathbf{F}_p$, define

$$e_a(d) = \prod_{b \in \mathbf{F}_p,\ b \neq a} \frac{d - b}{a - b} \quad \text{in } k[d]/(d^p - d).$$

Define the candidate coordinate maps

$$\alpha^* : C \longrightarrow \prod_{a \in \mathbf{F}_p} k[z], \qquad z_1 \longmapsto (z)_a, \qquad d \longmapsto (a)_a,$$

and

$$\beta^* : \prod_{a \in \mathbf{F}_p} k[z] \longrightarrow C, \qquad (f_a(z))_a \longmapsto \sum_a e_a(d) f_a(z_1).$$

No inverse equation is included in this Definition.

Proposition 13.6.11 (Explicit torsor cocycle isomorphism). Uses: def:weil-as-cover-coordinate-datum, def:weil-as-cover-torsor-candidates

Lean: blueprint-target:prop:weil-as-cover-torsor-map

The action map induces an isomorphism

$$\alpha : G \times_k X \longrightarrow X \times_Y X, \qquad (a, z) \longmapsto (z, z + a).$$

The candidate β^* is the inverse of α^*; both inverse equations and the component-idempotent equations are proved below.

Proof. Uses: def:weil-as-cover-coordinate-datum,def:weil-as-cover-torsor-candidates

Since $d^p - d = \prod_{a \in \mathbf{F}_p}(d - a)$ has pairwise coprime factors, the Chinese remainder map is the α^* of Definition 13.6.10. The denominators are nonzero elements of $\mathbf{F}_p$. The equations $e_a(b) = \delta_{ab}$ imply $e_a e_b = 0$ for $a \neq b$, $e_a^2 = e_a$, and $\sum_a e_a = 1$. Hence the candidate β^* is a ring homomorphism. Evaluation gives

$$\big(\alpha^* \beta^*((f_a)_a)\big)_b = \sum_a e_a(b) f_a(z) = f_b(z),$$

so $\alpha^* \beta^*$ is the identity. Conversely,

$$\beta^* \alpha^*(z_1) = \sum_a e_a(d) z_1 = z_1, \qquad \beta^* \alpha^*(d) = \sum_a e_a(d) a = d,$$

where the second equality is the degree-one Lagrange interpolation formula. Since z_1, d generate C, $\beta^* \alpha^*$ is also the identity. Thus α is an isomorphism with the asserted explicit inverse. □

Lemma 13.6.12 (Torsor cocycle equations). Uses: def:weil-as-cover-coordinate-datum, def:weil-as-cover-torsor-candidates

Lean: blueprint-target:lemma:weil-as-cover-cocycle

On $X \times_Y X \times_Y X$, let $d_{ij} = z_j - z_i$. The transition labels satisfy

$$d_{13} = d_{12} + d_{23}.$$

The diagonal has label 0, swapping the two factors negates the label, and on four factors the two iterated sums agree. These are the unit, inverse, and cocycle equations for the G-torsor.

Proof. Uses: def:weil-as-cover-coordinate-datum, def:weil-as-cover-torsor-candidates, lemma:weil-as-cover-geometric-fibres, prop:weil-as-cover-torsor-map

All equations hold already in the polynomial ring:

$$z_3 - z_1 = (z_2 - z_1) + (z_3 - z_2), \qquad z_i - z_i = 0, \qquad z_i - z_j = -(z_j - z_i).$$

On a geometric fibre each difference is the unique element of $\mathbf{F}_p$ supplied by Lemma 13.6.7; the explicit inverse in Proposition 13.6.11 recovers exactly that label. For four factors, both parenthesizations equal $d_{12} + d_{23} + d_{34}$ by associativity in $\mathbf{F}_p$. Thus none of the cocycle or inverse equations is left implicit. □

Lemma 13.6.13 (Compatibility with geometric Frobenius). *Uses:* def:weil-as-cover-coordinate-datum, def:weil-as-cover-torsor-candidates

Lean: blueprint-target:lemma:weil-as-cover-frobenius

The morphism π, the G-action, the quotient identification, the torsor map α, its explicit inverse, and all cocycle equations commute with geometric Frobenius over $k = \mathbf{F}_q$.

Proof. Uses: def:weil-as-cover-coordinate-datum, def:weil-as-cover-torsor-candidates, lemma:weil-as-cover-translation-action, lemma:weil-as-cover-invariants, lemma:weil-as-cover-affine-quotient, prop:weil-as-cover-torsor-map, lemma:weil-as-cover-cocycle

First use arithmetic Frobenius on $\overline{k}$-points. It fixes every $a \in \mathbf{F}_p$ because $a^q = a$, and

$$(z + a)^q = z^q + a, \qquad (z^p - z)^q = (z^q)^p - z^q.$$

Thus it commutes with every translation and with π. On the torsor fibre product it sends $d = z_2 - z_1$ to d^q; each component has $d = a \in \mathbf{F}_p$, hence is fixed. The Lagrange polynomials $e_a(d)$ have coefficients in $\mathbf{F}_p$, so the displayed inverse β^* is fixed as well. The identities $d_{13} = d_{12} + d_{23}$, $d_{ii} = 0$, and $d_{ji} = -d_{ij}$ are preserved term by term. Geometric Frobenius is the inverse of arithmetic Frobenius on geometric points, so the same commutative squares hold for geometric Frobenius. □

Theorem 13.6.14 (Artin–Schreier finite-etale torsor package). *Uses:* def:weil-as-cover-coordinate-datum, def:weil-as-cover-standard-etale-raw-data, def:weil-as-cover-torsor-candidates

Lean: blueprint-target:thm:weil-as-cover-closure

The map $\pi : z \mapsto z^p - z$ is a finite etale, finite free rank-p $G = (\mathbf{F}_p)_k$-torsor. Its geometric fibres are simply transitive G-sets, its quotient is $Y = \operatorname{Spec} B^G$, its torsor map has the explicit inverse and cocycle equations above, and all data commute with geometric Frobenius.

Proof. Uses: def:weil-as-cover-coordinate-datum, def:weil-as-cover-standard-etale-raw-data, def:weil-as-cover-torsor-candidates, lemma:weil-as-cover-coordinate-free, lemma:weil-as-cover-standard-etale-bezout, lemma:weil-as-cover-etale, lemma:weil-as-cover-translation-action, lemma:weil-as-cover-geometric-fibres, lemma:weil-as-cover-invariants, lemma:weil-as-cover-affine-quotient, prop:weil-as-cover-torsor-map, lemma:weil-as-cover-cocycle, lemma:weil-as-cover-frobenius

Finite freeness is Lemma 13.6.2, etaleness is Lemma 13.6.5, and the action and geometric fibres are Lemmas 13.6.6 and 13.6.7. The invariant-ring computation is Lemma 13.6.8, and the quotient universal property is Lemma 13.6.9. The explicit torsor inverse and both inverse equations are Proposition 13.6.11, the cocycle is Lemma 13.6.12, and Frobenius compatibility is Lemma 13.6.13. These prove all assertions in the theorem. □

13.7 A conditional Artin–Schreier character summand

This staging cluster retains the additive character fixed in the existing additive-character Definition. It does not choose a second character. The finite projector algebra is proved below; the eight categorical construction interfaces that are not yet available in the master are recorded as exact atomic OPEN leaves.

13.7.1 Objects and arrow candidates

Definition 13.7.1 (Adic coefficient context candidate). Lean: conditional-interface:def:weil-as-summand-adic-context

Uses: def:weil-wpdf-additive-character-over-finite-extensions, def:weil-as-cover-coordinate-datum

Retain the nontrivial character $\psi : (\mathbf{F}_p, +) \to \overline{\mathbf{Q}}_\ell^\times$ and the characters ψ_n already fixed in the existing additive-character Definition. Write $\mathcal{A}(Y)$ and $\mathcal{A}(X)$ for the intended pro-etale/adic $\overline{\mathbf{Q}}_\ell$ coefficient categories, and use f^*, f_*, tensor products, finite sums, images, and duals as notation for the corresponding candidate operations. No existence, exactness, or coherence assertion is included in this notation or inferred by any later Definition in this staging cluster.

Definition 13.7.2 (Finite pushforward and translation candidates). Lean: `conditional-interface:def:weil-as-summand-pushforward-translations`

Uses: `def:weil-as-summand-adic-context,def:weil-as-cover-coordinate-datum`

For the Artin–Schreier cover $\pi : X \to Y$, put

$$\mathcal{P} := \pi_* \underline{\overline{\mathbf{Q}}_\ell}_X.$$

For $a \in \mathbf{F}_p$, let $T_a : \mathcal{P} \to \mathcal{P}$ denote the candidate map induced by $\tau_a(z) = z + a$. On a finite geometric fibre the intended convention is $(T_a f)(x) = f(x + a)$.

Definition 13.7.3 (Character projector family). Lean: `conditional-interface:def:weil-as-summand-projector-family`

Uses: `def:weil-as-summand-adic-context,def:weil-as-summand-pushforward-translations`

Put

$$\widehat{G} := \mathrm{Hom}((\mathbf{F}_p, +), \overline{\mathbf{Q}}_\ell^\times).$$

For $\chi \in \widehat{G}$, define the finite expressions

$$e_\chi := \frac{1}{p} \sum_{a \in \mathbf{F}_p} \chi(a)^{-1}[a], \qquad E_\chi := \frac{1}{p} \sum_{a \in \mathbf{F}_p} \chi(a)^{-1} T_a.$$

The symbol E_ψ uses the previously fixed ψ.

Definition 13.7.4 (Image object and its two arrows). Lean: `conditional-interface:def:weil-wpdf-artin-schreier-summand-and-its-required-maps`

Uses: `def:weil-as-summand-projector-family`

For $\chi \in \widehat{G}$, write

$$\mathcal{L}_\chi := \mathrm{im}(E_\chi), \qquad \mathcal{P} \xrightarrow{r_\chi} \mathcal{L}_\chi \xrightarrow{s_\chi} \mathcal{P}$$

for the candidate image object and its candidate quotient and inclusion arrows. This Definition records no factorization, splitting, lissity, trace, or Frobenius assertion.

Definition 13.7.5 (Ambient Frobenius arrow). Lean: `conditional-interface:def:weil-as-summand-ambient-frobenius-forward`

Uses: `def:weil-as-summand-pushforward-translations`

Let F_Y be geometric Frobenius on Y. Write

$$\Phi_{\mathcal{P}} : F_Y^* \mathcal{P} \longrightarrow \mathcal{P}$$

for the candidate forward Frobenius arrow on the finite pushforward.

Definition 13.7.6 (Ambient reverse Frobenius arrow). Lean: `conditional-interface:def:weil-as-summand-ambient-frobenius-reverse`

Uses: `def:weil-as-summand-pushforward-translations,def:weil-as-summand-ambient-frobenius-forward`

Write

$$\Psi_{\mathcal{P}} : \mathcal{P} \longrightarrow F_Y^* \mathcal{P}$$

for the candidate reverse ambient Frobenius arrow.

Definition 13.7.7 (Induced image Frobenius arrow). Lean: `conditional-interface:def:weil-as-summand-image-frobenius-forward`

Uses: `def:weil-wpdf-artin-schreier-summand-and-its-required-maps,def:weil-as-summand-ambient-frobenius-forward`

Define the candidate composite

$$\Phi_\chi := r_\chi \Phi_{\mathcal{P}} F_Y^* s_\chi : F_Y^* \mathcal{L}_\chi \longrightarrow \mathcal{L}_\chi.$$

Definition 13.7.8 (Reverse image Frobenius arrow). Lean: `conditional-interface:def:weil-as-summand-image-frobenius-reverse`

Uses: `def:weil-wpdf-artin-schreier-summand-and-its-required-maps,def:weil-as-summand-ambient-frobenius-reverse`

Define the candidate composite

$$\Psi_\chi := F_Y^* r_\chi \Psi_{\mathcal{P}} s_\chi : \mathcal{L}_\chi \longrightarrow F_Y^* \mathcal{L}_\chi.$$

Definition 13.7.9 (Torsor trivialization arrow). Lean: `conditional-interface:def:weil-as-summand-torsor-forward`

Uses: `def:weil-as-summand-pushforward-translations`, `def:weil-as-cover-torsor-candidates`

Put

$$\mathcal{Q} := \bigoplus_{a \in \mathbf{F}_p} \underline{\overline{\mathbf{Q}}_\ell}_X \, \delta_a.$$

Write $\theta : \pi^*\mathcal{P} \to \mathcal{Q}$ for the candidate finite-fibre function map indexed by the points $z + a$ of the pulled-back torsor.

Definition 13.7.10 (Reverse torsor trivialization arrow). Lean: `conditional-interface:def:weil-as-summand-torsor-reverse`

Uses: `def:weil-as-summand-torsor-forward`

Write $\upsilon : \mathcal{Q} \to \pi^*\mathcal{P}$ for the candidate arrow sending δ_a to the delta function at $z + a$.

Definition 13.7.11 (Direct-sum forward arrow). Lean: `conditional-interface:def:weil-as-summand-direct-sum-forward`

Uses: `def:weil-wpdf-artin-schreier-summand-and-its-required-maps`

Define

$$I := \bigoplus_{\chi \in \widehat{G}} s_\chi : \bigoplus_{\chi \in \widehat{G}} \mathcal{L}_\chi \longrightarrow \mathcal{P}.$$

Definition 13.7.12 (Direct-sum reverse arrow). Lean: `conditional-interface:def:weil-as-summand-direct-sum-reverse`

Uses: `def:weil-wpdf-artin-schreier-summand-and-its-required-maps`

Define

$$R := (r_\chi)_\chi : \mathcal{P} \longrightarrow \bigoplus_{\chi \in \widehat{G}} \mathcal{L}_\chi.$$

Definition 13.7.13 (Local character vectors and sections). Lean: `conditional-interface:def:weil-as-summand-local-character-sections`

Uses: `def:weil-as-summand-projector-family`, `def:weil-wpdf-artin-schreier-summand-and-its-required-maps`, `def:weil-as-summand-torsor-forward`, `def:weil-as-summand-torsor-reverse`

For $\chi \in \widehat{G}$, put

$$v_\chi := \sum_{a \in \mathbf{F}_p} \chi(a)\delta_a \in \mathcal{Q}, \qquad u_\chi := \pi^* r_\chi \, \upsilon(v_\chi).$$

These are candidate sections on the torsor cover.

Definition 13.7.14 (Zero-stalk rigidification arrow). Lean: `conditional-interface:def:weil-as-summand-rigidification-forward`

Uses: `def:weil-as-summand-local-character-sections`

Using the lift $z = 0$ above $t = 0$, define the candidate arrow

$$\epsilon_\chi : \overline{\mathbf{Q}}_\ell \longrightarrow 0^*\mathcal{L}_\chi, \qquad 1 \longmapsto u_\chi(0).$$

Definition 13.7.15 (Reverse zero-stalk arrow). Lean: `conditional-interface:def:weil-as-summand-rigidification-reverse`

Uses: `def:weil-as-summand-rigidification-forward`

Write $\eta_\chi : 0^*\mathcal{L}_\chi \to \overline{\mathbf{Q}}_\ell$ for the candidate coordinate arrow specified by $\eta_\chi(u_\chi(0)) = 1$.

Definition 13.7.16 (Raw addition arrow). Lean: `conditional-interface:def:weil-as-summand-addition-raw`

Uses: `def:weil-as-summand-pushforward-translations`

For $S = Y \times_k Y$, let $m(t_1, t_2) = t_1 + t_2$ and let $p_1, p_2 : S \to Y$ be the projections. Write

$$\mathsf{A}^* : m^*\mathcal{P} \longrightarrow p_1^*\mathcal{P} \otimes p_2^*\mathcal{P}$$

for the candidate pullback-of-functions arrow associated with $(z_1, z_2) \mapsto z_1 + z_2$.

Definition 13.7.17 (Additive comparison arrow). Lean: `conditional-interface:def:weil-as-summand-addition-forward`

Uses: `def:weil-wpdf-artin-schreier-summand-and-its-required-maps`, `def:weil-as-summand-addition-raw`

Define the candidate composite

$$c_\chi := (p_1^* r_\chi \otimes p_2^* r_\chi)\mathsf{A}^* m^* s_\chi : m^*\mathcal{L}_\chi \longrightarrow p_1^*\mathcal{L}_\chi \otimes p_2^*\mathcal{L}_\chi.$$

Definition 13.7.18 (Reverse additive arrow). Lean: `conditional-interface:def:weil-as-summand-addition-reverse`

Uses: `def:weil-as-summand-local-character-sections`, `def:weil-as-summand-addition-forward`

Write

$$d_\chi : p_1^*\mathcal{L}_\chi \otimes p_2^*\mathcal{L}_\chi \longrightarrow m^*\mathcal{L}_\chi$$

for the candidate arrow with local prescription $d_\chi(u_\chi(z_1) \otimes u_\chi(z_2)) = u_\chi(z_1 + z_2)$.

Definition 13.7.19 (Ambient fibre pairing). Lean: `conditional-interface:def:weil-as-summand-ambient-pairing`

Uses: `def:weil-as-summand-pushforward-translations`

Write

$$B : \mathcal{P} \otimes \mathcal{P} \longrightarrow \underline{\overline{\mathbf{Q}}_\ell}_Y$$

for the candidate normalized fibre pairing whose intended geometric-stalk formula is $B_y(f,g) = p^{-1}\sum_{x \in X_{\bar{y}}} f(x)g(x)$.

Definition 13.7.20 (Restricted inverse-character pairing). Lean: `conditional-interface:def:weil-as-summand-restricted-pairing`

Uses: `def:weil-wpdf-artin-schreier-summand-and-its-required-maps`, `def:weil-as-summand-ambient-pairing`

Define the candidate composite

$$b_\chi := B(s_\chi \otimes s_{\chi^{-1}}) : \mathcal{L}_\chi \otimes \mathcal{L}_{\chi^{-1}} \longrightarrow \underline{\overline{\mathbf{Q}}_\ell}_Y.$$

Definition 13.7.21 (Duality arrow). Lean: `conditional-interface:def:weil-as-summand-dual-forward`

Uses: `def:weil-as-summand-restricted-pairing`

Write

$$D_\chi : \mathcal{L}_{\chi^{-1}} \longrightarrow \mathcal{L}_\chi^\vee$$

for the candidate curried arrow associated with b_χ.

Definition 13.7.22 (Reverse duality arrow). Lean: `conditional-interface:def:weil-as-summand-dual-reverse`

Uses: `def:weil-as-summand-local-character-sections`, `def:weil-as-summand-dual-forward`

Write $C_\chi : \mathcal{L}_\chi^\vee \to \mathcal{L}_{\chi^{-1}}$ for the candidate arrow with local prescription $C_\chi(u_\chi(z)^\vee) = u_{\chi^{-1}}(z)$.

13.7.2 Finite algebra

Lemma 13.7.23 (Translation unit). Lean: `conditional-interface:lemma:weil-as-summand-translation-unit`

Uses: `def:weil-as-summand-pushforward-translations`

Assuming the finite-pushforward function-model leaf, $T_0 = \mathrm{id}_{\mathcal{P}}$.

Proof. Uses: `def:weil-as-summand-pushforward-translations`, `lemma:weil-as-cover-translation-action`

On every finite geometric fibre, $(T_0 f)(x) = f(x)$. The cited atomic leaf asserts that this fibre formula represents the sheaf arrow. □

Lemma 13.7.24 (Translation composition). Lean: `conditional-interface:lemma:weil-as-summand-translation-composition`

Uses: `def:weil-as-summand-pushforward-translations`

Assuming the finite-pushforward function-model leaf, $T_a T_b = T_{a+b}$ for $a, b \in \mathbf{F}_p$.

Proof. Uses: `def:weil-as-summand-pushforward-translations`, `lemma:weil-as-cover-translation-action`

For a fibre function f, $(T_a T_b f)(x) = f(x+a+b)$. The finite-pushforward leaf promotes this pointwise identity to the asserted arrow identity. □

Lemma 13.7.25 (Nontrivial character witness). *Lean:* `conditional-interface:lemma:weil-as-summand-character-witness`

Uses: `def:weil-as-summand-projector-family`

If $\rho \in \widehat{G}$ and $\rho \neq 1$, then there is $b \in \mathbf{F}_p$ with $\rho(b) \neq 1$.

Proof. Uses: `def:weil-as-summand-projector-family`

If $\rho(b) = 1$ for every b, then ρ is the trivial function by function extensionality. This contradicts $\rho \neq 1$, so the asserted witness exists. □

Lemma 13.7.26 (Witness translation multiplier). *Lean:* `conditional-interface:lemma:weil-as-summand-witness-translation`

Uses: `def:weil-as-summand-projector-family`

For $\rho \in \widehat{G}$ and $b \in \mathbf{F}_p$ with $\rho(b) \neq 1$,

$$\sum_{a \in \mathbf{F}_p} \rho(a) = \rho(b) \sum_{a \in \mathbf{F}_p} \rho(a).$$

Proof. Uses: `def:weil-as-summand-projector-family`

Translation of the finite index by $a \mapsto a + b$ gives $\sum_a \rho(a+b) = \rho(b) \sum_a \rho(a)$, while translation permutes the same finite index set. □

Lemma 13.7.27 (Finite character enumeration). *Lean:* `conditional-interface:lemma:weil-as-summand-character-enumeration`

Uses: `def:weil-as-summand-projector-family`

The character group $\widehat{G}$ has p elements.

Proof. Uses: `def:weil-as-summand-projector-family`

The additive group of $\mathbf{F}_p$ is cyclic, generated by 1. A character is determined by its value at 1, which is a pth root of unity; the p roots ζ^j give the distinct characters $a \mapsto \zeta^{ja}$ for $0 \leq j < p$. □

Lemma 13.7.28 (Finite character sum). *Lean:* `conditional-interface:lemma:weil-as-summand-character-sum`

Uses: `def:weil-as-summand-projector-family`

For a character $\rho \in \widehat{G}$,

$$\sum_{a \in \mathbf{F}_p} \rho(a) = \begin{cases} p, & \rho = 1, \\ 0, & \rho \neq 1. \end{cases}$$

Proof. Uses: `def:weil-as-summand-projector-family,lemma:weil-as-summand-character-witness,lemma:weil-as-summand-witness-translation`

For $\rho \neq 1$, the character-witness lemma supplies a pair (b, h_b) with $h_b : \rho(b) \neq 1$. The witness-translation lemma says $(1-\rho(b)) \sum_a \rho(a) = 0$, and $1 - \rho(b) \neq 0$, so the sum is zero. For the trivial character, every summand is 1. □

Lemma 13.7.29 (Character separation at a nonzero point). *Lean:* `conditional-interface:lemma:weil-as-summand-point-character-witness`

Uses: `def:weil-as-summand-projector-family`

If $a \in \mathbf{F}_p$ is nonzero, then some $\chi \in \widehat{G}$ satisfies $\chi(a) \neq 1$.

Proof. Uses: `def:weil-as-summand-projector-family,lemma:weil-as-summand-character-enumeration`

The explicit cyclic enumeration in the preceding lemma contains the characters $\chi_j(a) = \zeta^{ja}$. Taking the nonzero index $j = 1$ gives $\chi_1(a) \neq 1$, because $a \neq 0$ in the cyclic group of order p. □

Lemma 13.7.30 (Dual finite character sum). *Lean:* `conditional-interface:lemma:weil-as-summand-dual-character-sum`

Uses: `def:weil-as-summand-projector-family`

For $a \in \mathbf{F}_p$,

$$\sum_{\chi \in \widehat{G}} \chi(a) = \begin{cases} p, & a = 0, \\ 0, & a \neq 0. \end{cases}$$

Proof. Uses: def:weil-as-summand-projector-family,lemma:weil-as-summand-character-enumeration,lemma:weil-as-summand-point-character-witness

For $a \neq 0$, the point-character witness supplies a character χ_0 with $\chi_0(a) \neq 1$. Multiplication by χ_0 permutes the finite character index set and multiplies the sum by $\chi_0(a)$, forcing the sum to vanish. At $a = 0$, the p-element enumeration makes every summand 1. □

Lemma 13.7.31 (Group-algebra projector product). *Lean:* conditional-interface:lemma:weil-as-summand-group-projector-product

Uses: def:weil-as-summand-projector-family

For $\chi, \eta \in \widehat{G}$, $e_\chi e_\eta = \delta_{\chi,\eta} e_\chi$.

Proof. Uses: def:weil-as-summand-projector-family,lemma:weil-as-summand-character-sum

The coefficient of $[c]$ is $p^{-2}\eta(c)^{-1}\sum_a (\eta\chi^{-1})(a)$. The character-sum lemma makes this 0 if $\eta \neq \chi$ and $p^{-1}\chi(c)^{-1}$ otherwise. □

Lemma 13.7.32 (Group-algebra projector completeness). *Lean:* conditional-interface:lemma:weil-as-summand-group-projector-completeness

Uses: def:weil-as-summand-projector-family

$\sum_{\chi \in \widehat{G}} e_\chi = [0]$.

Proof. Uses: def:weil-as-summand-projector-family,lemma:weil-as-summand-dual-character-sum

The coefficient of $[a]$ is $p^{-1}\sum_\chi \chi(a)^{-1}$, which is 1 for $a = 0$ and 0 otherwise. □

Lemma 13.7.33 (Sheaf projector product). *Lean:* conditional-interface:lemma:weil-as-summand-sheaf-projector-product

Uses: def:weil-as-summand-projector-family

Assuming the finite-pushforward function-model leaf, $E_\chi E_\eta = \delta_{\chi,\eta} E_\chi$.

Proof. Uses: def:weil-as-summand-projector-family,lemma:weil-as-summand-translation-composition,lemma:weil-as-summand-character-sum,lemma:weil-as-summand-group-projector-product

Expand the product and use $T_a T_b = T_{a+b}$. The coefficient calculation is exactly the one in Lemma 13.7.31. □

Lemma 13.7.34 (Sheaf projector completeness). *Lean:* conditional-interface:lemma:weil-as-summand-sheaf-projector-completeness

Uses: def:weil-as-summand-projector-family

Assuming the finite-pushforward function-model leaf, $\sum_\chi E_\chi = \mathrm{id}_{\mathcal{P}}$.

Proof. Uses: def:weil-as-summand-projector-family,lemma:weil-as-summand-translation-unit,lemma:weil-as-summand-dual-character-sum

Interchange the two finite sums. Dual character orthogonality leaves only T_0, which is the identity. □

Lemma 13.7.35 (Left translation eigen equation). *Lean:* conditional-interface:lemma:weil-as-summand-projector-left-eigen

Uses: def:weil-as-summand-projector-family

Assuming the finite-pushforward function-model leaf, $T_b E_\chi = \chi(b) E_\chi$.

Proof. Uses: def:weil-as-summand-projector-family,lemma:weil-as-summand-translation-composition

In the defining sum for $T_b E_\chi$, substitute $c = a + b$ and use $\chi(c-b)^{-1} = \chi(b)\chi(c)^{-1}$. □

Lemma 13.7.36 (Right translation eigen equation). *Lean:* conditional-interface:lemma:weil-as-summand-projector-right-eigen

Uses: def:weil-as-summand-projector-family

Assuming the finite-pushforward function-model leaf, $E_\chi T_b = \chi(b) E_\chi$.

Proof. Uses: def:weil-as-summand-projector-family,lemma:weil-as-summand-translation-composition,lemma:weil-as-summand-projector-left-eigen

The translation composition law is commutative, so the computation in Lemma 13.7.35 applies on the right. □

13.7.3 Image and direct sum

Lemma 13.7.37 (Image factorization). *Lean: conditional-interface:lemma:weil-as-summand-image-factorization*

Uses: def:weil-wpdf-artin-schreier-summand-and-its-required-maps, def:weil-as-summand-projector-family

Assuming the categorical-image leaf, $s_\chi r_\chi = E_\chi$.

Proof. Uses: def:weil-wpdf-artin-schreier-summand-and-its-required-maps,def:weil-as-summand-projector-family

This equation is the factorization field required by the named image leaf. □

Lemma 13.7.38 (Image retraction). *Lean: conditional-interface:lemma:weil-as-summand-image-retraction*

Uses: def:weil-wpdf-artin-schreier-summand-and-its-required-maps

Assuming the categorical-image leaf, $r_\chi s_\chi = \mathrm{id}_{\mathcal{L}_\chi}$.

Proof. Uses: lemma:weil-as-summand-image-factorization,lemma:weil-as-summand-sheaf-projector-product

Idempotence gives $srsr = sr$. The image leaf supplies that s is monic and r is epic, so cancellation gives $rs = 1$. □

Lemma 13.7.39 (Inclusion translation equation). *Lean: conditional-interface:lemma:weil-as-summand-inclusion-translation*

Uses: def:weil-wpdf-artin-schreier-summand-and-its-required-maps, def:weil-as-summand-pushforward-translations

Assuming the finite-pushforward and image leaves, $T_b s_\chi = \chi(b) s_\chi$.

Proof. Uses: lemma:weil-as-summand-image-factorization,lemma:weil-as-summand-image-retraction,lemma:weil-as-summand-projector-left-eigen

$T_b s = T_b E s = \chi(b) E s = \chi(b) s$. □

Lemma 13.7.40 (Retraction translation equation). *Lean: conditional-interface:lemma:weil-as-summand-retraction-translation*

Uses: def:weil-wpdf-artin-schreier-summand-and-its-required-maps, def:weil-as-summand-pushforward-translations

Assuming the finite-pushforward and image leaves, $r_\chi T_b = \chi(b) r_\chi$.

Proof. Uses: lemma:weil-as-summand-image-factorization,lemma:weil-as-summand-image-retraction,lemma:weil-as-summand-projector-right-eigen

$r T_b = r E T_b = \chi(b) r E = \chi(b) r$. □

Lemma 13.7.41 (Off-diagonal image composite). *Lean: conditional-interface:lemma:weil-as-summand-cross-term*

Uses: def:weil-wpdf-artin-schreier-summand-and-its-required-maps

Assuming the finite-pushforward and image leaves, if $\chi \neq \eta$ then $r_\chi s_\eta = 0$.

Proof. Uses: lemma:weil-as-summand-image-factorization,lemma:weil-as-summand-image-retraction,lemma:weil-as-summand-sheaf-projector-product

$s_\chi r_\chi s_\eta = E_\chi E_\eta s_\eta = 0$; monicity of s_χ gives the assertion. □

Lemma 13.7.42 (Direct-sum left inverse). *Lean: conditional-interface:lemma:weil-as-summand-direct-sum-left-inverse*

Uses: def:weil-as-summand-direct-sum-forward, def:weil-as-summand-direct-sum-reverse

Assuming the finite-pushforward and image leaves, $RI = \mathrm{id}_{\oplus_\chi \mathcal{L}_\chi}$.

Proof. Uses: lemma:weil-as-summand-image-retraction,lemma:weil-as-summand-cross-term

The (χ, η) entry is $r_\chi s_\eta$, identity on the diagonal and zero off the diagonal. □

Lemma 13.7.43 (Direct-sum right inverse). *Lean:* `conditional-interface:lemma:weil-as-summand-direct-sum-right-inverse`

Uses: `def:weil-as-summand-direct-sum-forward`, `def:weil-as-summand-direct-sum-reverse`

Assuming the finite-pushforward and image leaves, $IR = \mathrm{id}_{\mathcal{P}}$.

Proof. Uses: `lemma:weil-as-summand-image-factorization`, `lemma:weil-as-summand-sheaf-projector-completeness`

$IR = \sum_\chi s_\chi r_\chi = \sum_\chi E_\chi = 1$. □

13.7.4 Frobenius arrows

Lemma 13.7.44 (Ambient Frobenius left inverse). *Lean:* `conditional-interface:lemma:weil-as-summand-ambient-frobenius-left-inverse`

Uses: `def:weil-as-summand-ambient-frobenius-forward`, `def:weil-as-summand-ambient-frobenius-reverse`

Assuming the Frobenius-action leaf, $\Phi_{\mathcal{P}}\Psi_{\mathcal{P}} = \mathrm{id}_{\mathcal{P}}$.

Proof. Uses: `def:weil-as-summand-ambient-frobenius-forward`, `def:weil-as-summand-ambient-frobenius-reverse`

This is the forward–reverse equation required by the Frobenius leaf. □

Lemma 13.7.45 (Ambient Frobenius right inverse). *Lean:* `conditional-interface:lemma:weil-as-summand-ambient-frobenius-right-inverse`

Uses: `def:weil-as-summand-ambient-frobenius-forward`, `def:weil-as-summand-ambient-frobenius-reverse`

Assuming the Frobenius-action leaf, $\Psi_{\mathcal{P}}\Phi_{\mathcal{P}} = \mathrm{id}_{F_Y^*\mathcal{P}}$.

Proof. Uses: `def:weil-as-summand-ambient-frobenius-forward`, `def:weil-as-summand-ambient-frobenius-reverse`

This is the reverse–forward equation required by the Frobenius leaf. □

Lemma 13.7.46 (Image inclusion Frobenius square). *Lean:* `conditional-interface:lemma:weil-as-summand-inclusion-frobenius-square`

Uses: `def:weil-wpdf-artin-schreier-summand-and-its-required-maps`, `def:weil-as-summand-ambient-frobenius-forward`, `def:weil-as-summand-image-frobenius-forward`

Assuming the image and Frobenius leaves, $s_\chi\Phi_\chi = \Phi_{\mathcal{P}}F_Y^* s_\chi$.

Proof. Uses: `lemma:weil-as-summand-image-factorization`, `lemma:weil-as-summand-image-retraction`, `lemma:weil-as-summand-sheaf-projector-product`

Insert the definition of Φ_χ. The Frobenius leaf supplies $E_\chi\Phi_{\mathcal{P}} = \Phi_{\mathcal{P}}F_Y^*E_\chi$; simplify with the two image equations. □

Lemma 13.7.47 (Image retraction Frobenius square). *Lean:* `conditional-interface:lemma:weil-as-summand-retraction-frobenius-square`

Uses: `def:weil-wpdf-artin-schreier-summand-and-its-required-maps`, `def:weil-as-summand-ambient-frobenius-forward`, `def:weil-as-summand-image-frobenius-forward`

Assuming the image and Frobenius leaves, $\Phi_\chi F_Y^* r_\chi = r_\chi \Phi_{\mathcal{P}}$.

Proof. Uses: `lemma:weil-as-summand-image-factorization`, `lemma:weil-as-summand-image-retraction`

Expand Φ_χ, replace $F_Y^*(sr)$ by $F_Y^*E_\chi$, and use the projector commutation field of the Frobenius leaf. □

Lemma 13.7.48 (Image Frobenius left inverse). *Lean:* `conditional-interface:lemma:weil-as-summand-image-frobenius-left-inverse`

Uses: `def:weil-as-summand-image-frobenius-forward`, `def:weil-as-summand-image-frobenius-reverse`

Assuming the image, exact-pullback, and Frobenius leaves, $\Phi_\chi\Psi_\chi = \mathrm{id}_{\mathcal{L}_\chi}$.

Proof. Uses: `lemma:weil-as-summand-image-retraction`, `lemma:weil-as-summand-ambient-frobenius-left-inverse`, `lemma:weil-as-summand-inclusion-frobenius-square`

Expand the composites, use $F_Y^*(rs) = 1$, and then use $\Phi_{\mathcal{P}}\Psi_{\mathcal{P}} = 1$ and $rs = 1$. □

Lemma 13.7.49 (Image Frobenius right inverse). *Lean:* `conditional-interface:lemma:weil-as-summand-image-frobenius-right-inverse`

Uses: `def:weil-as-summand-image-frobenius-forward`, `def:weil-as-summand-image-frobenius-reverse`

Assuming the image, exact-pullback, and Frobenius leaves, $\Psi_\chi \Phi_\chi = \mathrm{id}_{F_Y^* \mathcal{L}_\chi}$.

Proof. *Uses:* `lemma:weil-as-summand-image-retraction`, `lemma:weil-as-summand-ambient-frobenius-right-inverse`, `lemma:weil-as-summand-retraction-frobenius-square`

Expand the composites, use $rs = 1$ and $\Psi_\mathcal{P} \Phi_\mathcal{P} = 1$, and finish with $F_Y^*(rs) = 1$. □

Lemma 13.7.50 (Uniqueness of the image Frobenius arrow). *Lean:* `conditional-interface:lemma:weil-as-summand-image-frobenius-uniqueness`

Uses: `def:weil-wpdf-artin-schreier-summand-and-its-required-maps`, `def:weil-as-summand-ambient-frobenius-forward`, `def:weil-as-summand-image-frobenius-forward`

Assuming the image and Frobenius leaves, Φ_χ *is the unique arrow* $h : F_Y^* \mathcal{L}_\chi \to \mathcal{L}_\chi$ *satisfying* $s_\chi h = \Phi_\mathcal{P} F_Y^* s_\chi$.

Proof. *Uses:* `lemma:weil-as-summand-image-retraction`, `lemma:weil-as-summand-inclusion-frobenius-square`

If h satisfies the square, then $h = r_\chi s_\chi h = r_\chi \Phi_\mathcal{P} F_Y^* s_\chi = \Phi_\chi$. □

Lemma 13.7.51 (Direct-sum forward Frobenius square). *Lean:* `conditional-interface:lemma:weil-as-summand-direct-sum-forward-frobenius`

Uses: `def:weil-as-summand-direct-sum-forward`, `def:weil-as-summand-ambient-frobenius-forward`, `def:weil-as-summand-image-frobenius-forward`

Assuming the image and Frobenius leaves, $I(\oplus_\chi \Phi_\chi) = \Phi_\mathcal{P} F_Y^* I$.

Proof. *Uses:* `lemma:weil-as-summand-inclusion-frobenius-square`

The χ component is precisely the inclusion Frobenius square. □

Lemma 13.7.52 (Direct-sum reverse Frobenius square). *Lean:* `conditional-interface:lemma:weil-as-summand-direct-sum-reverse-frobenius`

Uses: `def:weil-as-summand-direct-sum-reverse`, `def:weil-as-summand-ambient-frobenius-forward`, `def:weil-as-summand-image-frobenius-forward`

Assuming the image and Frobenius leaves, $(\oplus_\chi \Phi_\chi) F_Y^* R = R \Phi_\mathcal{P}$.

Proof. *Uses:* `lemma:weil-as-summand-retraction-frobenius-square`

The χ component is precisely the retraction Frobenius square. □

13.7.5 Torsor model, rank, and trace

Lemma 13.7.53 (Torsor trivialization left inverse). *Lean:* `conditional-interface:lemma:weil-as-summand-torsor-left-inverse`

Uses: `def:weil-as-summand-torsor-forward`, `def:weil-as-summand-torsor-reverse`

Assuming the finite-pushforward function-model leaf, $\theta \upsilon = \mathrm{id}_\mathcal{Q}$.

Proof. *Uses:* `def:weil-as-summand-torsor-forward`, `def:weil-as-summand-torsor-reverse`, `prop:weil-as-cover-torsor-map`

The composite fixes every delta basis vector; the function-model leaf states that these basis calculations determine the arrow. □

Lemma 13.7.54 (Torsor trivialization right inverse). *Lean:* `conditional-interface:lemma:weil-as-summand-torsor-right-inverse`

Uses: `def:weil-as-summand-torsor-forward`, `def:weil-as-summand-torsor-reverse`

Assuming the finite-pushforward function-model leaf, $\upsilon \theta = \mathrm{id}_{\pi^* \mathcal{P}}$.

Proof. *Uses:* `def:weil-as-summand-torsor-forward`, `def:weil-as-summand-torsor-reverse`, `prop:weil-as-cover-torsor-map`

On every split fibre the composite fixes each delta function. The function-model leaf promotes the fibre calculation to this arrow equation. □

Lemma 13.7.55 (Translation on delta functions). *Lean:* `conditional-interface:lemma:weil-as-summand-translation-delta`

Uses: `def:weil-as-summand-pushforward-translations`, `def:weil-as-summand-torsor-forward`

Assuming the finite-pushforward function-model leaf, $T_b\delta_a = \delta_{a-b}$ in the torsor coordinates.

Proof. Uses: `def:weil-as-summand-pushforward-translations`, `prop:weil-as-cover-torsor-map`

The function $(T_b\delta_a)(x) = \delta_a(x+b)$ is supported at the point labelled $a-b$. □

Lemma 13.7.56 (Inverse finite Fourier expansion). *Lean:* `conditional-interface:lemma:weil-as-summand-fourier-basis-inverse`

Uses: `def:weil-as-summand-local-character-sections`

For $a \in \mathbf{F}_p$,

$$\delta_a = \frac{1}{p}\sum_{\chi\in\widehat{G}} \chi(a)^{-1} v_\chi.$$

Proof. Uses: `def:weil-as-summand-local-character-sections`, `lemma:weil-as-summand-dual-character-sum`

The coefficient of δ_b on the right is $p^{-1}\sum_\chi \chi(b-a)$, equal to $\delta_{a,b}$. □

Lemma 13.7.57 (Projector on a character vector). *Lean:* `conditional-interface:lemma:weil-as-summand-projector-character-vector`

Uses: `def:weil-as-summand-projector-family`, `def:weil-as-summand-local-character-sections`

Assuming the finite-pushforward function-model leaf, $E_\eta v_\chi = \delta_{\eta,\chi} v_\chi$.

Proof. Uses: `lemma:weil-as-summand-translation-delta`, `lemma:weil-as-summand-character-sum`

Reindexing the delta sum gives $T_b v_\chi = \chi(b) v_\chi$; substituting this into E_η leaves the character sum for $\chi\eta^{-1}$. □

Lemma 13.7.58 (Local inclusion formula). *Lean:* `conditional-interface:lemma:weil-as-summand-local-inclusion`

Uses: `def:weil-wpdf-artin-schreier-summand-and-its-required-maps`, `def:weil-as-summand-local-character-sections`

Assuming the finite-pushforward and image leaves, $\pi^* s_\chi(u_\chi) = v(v_\chi)$.

Proof. Uses: `lemma:weil-as-summand-image-factorization`, `lemma:weil-as-summand-projector-character-vector`

By definition of u_χ, its image is $\pi^*(s_\chi r_\chi)v(v_\chi) = \pi^* E_\chi v(v_\chi)$, which equals $v(v_\chi)$. □

Lemma 13.7.59 (Local retraction formula). *Lean:* `conditional-interface:lemma:weil-as-summand-local-retraction`

Uses: `def:weil-wpdf-artin-schreier-summand-and-its-required-maps`, `def:weil-as-summand-local-character-sections`

Assuming the finite-pushforward and image leaves,

$$\pi^* r_\chi\, v(\delta_a) = \frac{\chi(a)^{-1}}{p}, u_\chi.$$

Proof. Uses: `lemma:weil-as-summand-fourier-basis-inverse`, `lemma:weil-as-summand-projector-character-vector`, `lemma:weil-as-summand-image-retraction`

Apply $\pi^* r_\chi$ to the inverse Fourier expansion. All terms except v_χ vanish after inserting E_χ, and the remaining coefficient is $p^{-1}\chi(a)^{-1}$. □

Lemma 13.7.60 (Pullback preserves the image). *Lean:* `conditional-interface:lemma:weil-as-summand-exact-pullback-image`

Uses: `def:weil-wpdf-artin-schreier-summand-and-its-required-maps`, `def:weil-as-summand-projector-family`

Assuming the exact-pullback leaf, $\pi^*\mathcal{L}_\chi$ with its pulled-back arrows is the image of $\pi^* E_\chi$.

Proof. Uses: `lemma:weil-as-summand-image-factorization`

This is the image-preservation field required by the exact-pullback leaf. □

Lemma 13.7.61 (Local constancy on the torsor). *Lean:* `conditional-interface:lemma:weil-as-summand-local-constancy`

Uses: `def:weil-as-summand-local-character-sections`, `def:weil-wpdf-artin-schreier-summand-and-its-required-maps`

Assuming the finite-pushforward, image, and exact-pullback leaves, $\pi^*\mathcal{L}_\chi$ is the constant subobject generated by u_χ.

Proof. Uses: `lemma:weil-as-summand-projector-character-vector`, `lemma:weil-as-summand-fourier-basis-inverse`, `lemma:weil-as-summand-exact-pullback-image`

The vectors v_η form a basis and π^*E_χ is identity on v_χ and zero on the other basis vectors. Its image is therefore the constant subobject generated by u_χ. □

Lemma 13.7.62 (Rank one on the torsor). *Lean:* `conditional-interface:lemma:weil-as-summand-rank-one-on-cover`

Uses: `def:weil-as-summand-local-character-sections`, `def:weil-wpdf-artin-schreier-summand-and-its-required-maps`

Assuming the finite-pushforward, image, and exact-pullback leaves, $\pi^*\mathcal{L}_\chi$ has rank one.

Proof. Uses: `lemma:weil-as-summand-local-constancy`

It is the constant line with basis u_χ. □

Lemma 13.7.63 (Lisse descent). *Lean:* `conditional-interface:lemma:weil-as-summand-lisse-descent`

Uses: `def:weil-wpdf-artin-schreier-summand-and-its-required-maps`

Assuming the lisse-descent leaf, $\mathcal{L}_\chi$ is lisse of rank one on Y.

Proof. Uses: `lemma:weil-as-summand-rank-one-on-cover`, `lemma:weil-as-cover-etale`

The lisse-descent leaf applies to the rank-one constant pullback along the finite etale surjection π. □

Lemma 13.7.64 (Constructibility). *Lean:* `conditional-interface:lemma:weil-as-summand-constructible`

Uses: `def:weil-wpdf-artin-schreier-summand-and-its-required-maps`

Assuming the lisse-descent leaf, $\mathcal{L}_\chi$ is constructible.

Proof. Uses: `lemma:weil-as-summand-lisse-descent`

The constructibility clause of the lisse-descent leaf applies to a finite-rank lisse adic sheaf. □

Lemma 13.7.65 (Artin–Schreier telescope). *Lean:* `conditional-interface:lemma:weil-as-summand-trace-telescope`

Uses: `def:weil-as-summand-adic-context`

If $z^p - z = t$ with $t \in \mathbf{F}_{q^n}$, then

$$z^{q^n} - z = \mathrm{Tr}_{\mathbf{F}_{q^n}/\mathbf{F}_p}(t).$$

Proof. Uses: `def:weil-as-summand-adic-context`, `lemma:weil-as-cover-frobenius`

Writing $q = p^r$, telescope $z^{p^{rn}} - z = \sum_{j=0}^{rn-1}(z^p - z)^{p^j}$; the right side is the displayed field trace. □

Lemma 13.7.66 (Geometric Frobenius on fibre points). *Lean:* `conditional-interface:lemma:weil-as-summand-geometric-frobenius-points`

Uses: `def:weil-as-summand-ambient-frobenius-forward`

Assuming the Frobenius-action leaf, geometric Frobenius at a degree-n point of coordinate t sends a lift z to $z - \mathrm{Tr}_{\mathbf{F}_{q^n}/\mathbf{F}_p}(t)$.

Proof. Uses: `lemma:weil-as-summand-trace-telescope`, `lemma:weil-as-cover-frobenius`

The telescope describes arithmetic Frobenius by translation by the trace; the geometric action is its inverse. □

Lemma 13.7.67 (Frobenius eigenvalue on the character vector). *Lean:* `conditional-interface:lemma:weil-as-summand-character-vector-frobenius`

Uses: `def:weil-as-summand-local-character-sections`, `def:weil-as-summand-image-frobenius-forward`

Assuming the finite-pushforward and Frobenius leaves, at a degree-n point of coordinate t the line generated by v_χ has geometric-Frobenius eigenvalue $\chi(\mathrm{Tr}_{\mathbf{F}_{q^n}/\mathbf{F}_p}(t))$.

Proof. Uses: lemma:weil-as-summand-geometric-frobenius-points
If c is the trace, Frobenius sends the delta vector at $z+a$ to the one at $z+a-c$. Reindexing v_χ multiplies it by $\chi(c)$. □

Lemma 13.7.68 (Closed-point stalk trace). Lean: conditional-interface:lemma:weil-as-summand-stalk-trace

Uses: def:weil-wpdf-artin-schreier-summand-and-its-required-maps, def:weil-as-summand-image-frobenius-forward

Assuming the finite-pushforward, image, exact-pullback, Frobenius, and lisse-descent leaves, a degree-n point of coordinate t satisfies

$$\mathrm{Tr}(F_t \mid (\mathcal{L}_\chi)_{\bar{t}}) = \chi(\mathrm{Tr}_{\mathbf{F}_{q^n}/\mathbf{F}_p}(t)).$$

Proof. Uses: lemma:weil-as-summand-local-inclusion, lemma:weil-as-summand-rank-one-on-cover, lemma:weil-as-summand-character-vector-frobenius
The inclusion identifies the rank-one stalk with the character line. Its single Frobenius eigenvalue is the one computed in the preceding lemma. □

13.7.6 Rigidification and addition

Lemma 13.7.69 (Rigidification left inverse). Lean: conditional-interface:lemma:weil-as-summand-rigidification-left-inverse

Uses: def:weil-as-summand-rigidification-forward, def:weil-as-summand-rigidification-reverse

Assuming the image, exact-pullback, and lisse-descent leaves, $\eta_\chi \epsilon_\chi = \mathrm{id}_{\overline{\mathbf{Q}}_\ell}$.

Proof. Uses: def:weil-as-summand-rigidification-forward, def:weil-as-summand-rigidification-reverse
The composite sends 1 to 1 by the defining coordinate prescription. □

Lemma 13.7.70 (Rigidification right inverse). Lean: conditional-interface:lemma:weil-as-summand-rigidification-right-inverse

Uses: def:weil-as-summand-rigidification-forward, def:weil-as-summand-rigidification-reverse

Assuming the image, exact-pullback, and lisse-descent leaves, $\epsilon_\chi \eta_\chi = \mathrm{id}_{0^*\mathcal{L}_\chi}$.

Proof. Uses: lemma:weil-as-summand-rank-one-on-cover
The zero stalk is the line generated by $u_\chi(0)$, and the composite fixes that generator. □

Lemma 13.7.71 (Rigidification Frobenius square). Lean: conditional-interface:lemma:weil-as-summand-rigidification-frobenius

Uses: def:weil-as-summand-rigidification-forward, def:weil-as-summand-image-frobenius-forward

Assuming the Frobenius and lisse-descent leaves, ϵ_χ commutes with geometric Frobenius.

Proof. Uses: lemma:weil-as-summand-stalk-trace
At $t=0$ the computed eigenvalue is $\chi(0)=1$, which is also the Frobenius action on the coefficient line. □

Lemma 13.7.72 (Raw addition translation equation). Lean: conditional-interface:lemma:weil-as-summand-raw-addition-translation

Uses: def:weil-as-summand-addition-raw, def:weil-as-summand-pushforward-translations

Assuming the addition leaf and the finite-pushforward leaf,

$$(T_a \otimes T_b)\mathsf{A}^* = \mathsf{A}^* T_{a+b}.$$

Proof. Uses: def:weil-as-summand-addition-raw
On a fibre function f, both sides evaluate at (z_1, z_2) as $f(z_1+z_2+a+b)$. The addition leaf asserts that these fibre formulas represent the sheaf maps. □

Lemma 13.7.73 (Raw addition Frobenius square). Lean: conditional-interface:lemma:weil-as-summand-raw-addition-frobenius

Uses: def:weil-as-summand-addition-raw, def:weil-as-summand-ambient-frobenius-forward

Assuming the addition and Frobenius leaves, A^* commutes with the ambient geometric-Frobenius arrows.

Proof. Uses: def:weil-as-summand-addition-raw
This is the Frobenius compatibility field required for the addition arrow by the two named leaves. □

Lemma 13.7.74 (Change of lift for local sections). Lean: conditional-interface:lemma:weil-as-summand-lift-change

Uses: def:weil-as-summand-local-character-sections
Assuming the finite-pushforward and image leaves, $u_\chi(z+d) = \chi(d)^{-1}u_\chi(z)$ for $d \in \mathbf{F}_p$.

Proof. Uses: lemma:weil-as-summand-translation-delta,lemma:weil-as-summand-local-retraction
Changing the origin by d relabels the character vector by $v_\chi(z+d) = \chi(d)^{-1}v_\chi(z)$; applying the retraction gives the displayed formula. □

Lemma 13.7.75 (Independence of lifts for the reverse additive arrow). Lean: conditional-interface:lemma:weil-as-summand-addition-reverse-lift-independent

Uses: def:weil-as-summand-addition-reverse, def:weil-as-summand-local-character-sections
Assuming the addition, finite-pushforward, and image leaves, the local prescription for d_χ is unchanged when either lift is translated by an element of $\mathbf{F}_p$.

Proof. Uses: lemma:weil-as-summand-lift-change
Translations by d_1, d_2 scale the source by $\chi(d_1)^{-1}\chi(d_2)^{-1}$ and the target by $\chi(d_1+d_2)^{-1}$; these scalars are equal. □

Lemma 13.7.76 (Additive left inverse). Lean: conditional-interface:lemma:weil-as-summand-addition-left-inverse

Uses: def:weil-as-summand-addition-forward, def:weil-as-summand-addition-reverse
Assuming the addition, finite-pushforward, image, exact-pullback, and lisse-descent leaves, $d_\chi c_\chi = \mathrm{id}_{m^*\mathcal{L}_\chi}$.

Proof. Uses: lemma:weil-as-summand-local-inclusion,lemma:weil-as-summand-addition-reverse-lift-independent,lemma:weil-as-summand-lisse-descent
The raw addition formula sends $u_\chi(z_1+z_2)$ to $u_\chi(z_1)\otimes u_\chi(z_2)$; d_χ sends it back. The lisse-descent leaf makes equality on these local bases detect equality of arrows. □

Lemma 13.7.77 (Additive right inverse). Lean: conditional-interface:lemma:weil-as-summand-addition-right-inverse

Uses: def:weil-as-summand-addition-forward, def:weil-as-summand-addition-reverse
Assuming the addition, finite-pushforward, image, exact-pullback, and lisse-descent leaves, $c_\chi d_\chi$ is the identity on $p_1^*\mathcal{L}_\chi \otimes p_2^*\mathcal{L}_\chi$.

Proof. Uses: lemma:weil-as-summand-addition-reverse-lift-independent,lemma:weil-as-summand-lisse-descent
The composite fixes each local tensor basis $u_\chi(z_1)\otimes u_\chi(z_2)$. The lisse-descent leaf makes those local bases jointly detect the arrow. □

Lemma 13.7.78 (Additive comparison Frobenius square). Lean: conditional-interface:lemma:weil-as-summand-addition-frobenius

Uses: def:weil-as-summand-addition-forward, def:weil-as-summand-image-frobenius-forward
Assuming the image, exact-pullback, addition, and Frobenius leaves, c_χ commutes with geometric Frobenius.

Proof. Uses: lemma:weil-as-summand-inclusion-frobenius-square,lemma:weil-as-summand-retraction-frobenius-square,lemma:weil-as-summand-raw-addition-frobenius
The Frobenius square for the defining composite is the composite of the two retraction squares, the raw-addition square, and the inclusion square. □

Lemma 13.7.79 (Left unit equation). Lean: conditional-interface:lemma:weil-as-summand-addition-left-unit

Uses: def:weil-as-summand-rigidification-forward, def:weil-as-summand-rigidification-reverse, def:weil-as-summand-addition-forward
Assuming all leaves needed for the rigidification and additive comparison, the restriction of $(\eta_\chi \otimes 1)c_\chi$ to $\{0\}\times Y$ is the identity.

Proof. Uses: lemma:weil-as-summand-rigidification-left-inverse,lemma:weil-as-summand-addition-left-inverse
On the local basis, c_χ sends $u_\chi(z)$ to $u_\chi(0) \otimes u_\chi(z)$, and η_χ sends the first factor to 1. □

Lemma 13.7.80 (Right unit equation). Lean: conditional-interface:lemma:weil-as-summand-addition-right-unit

Uses: def:weil-as-summand-rigidification-forward, def:weil-as-summand-rigidification-reverse, def:weil-as-summand-addition-forward

Assuming all leaves needed for the rigidification and additive comparison, the restriction of $(1 \otimes \eta_\chi)c_\chi$ *to* $Y \times \{0\}$ *is the identity.*

Proof. Uses: lemma:weil-as-summand-rigidification-left-inverse,lemma:weil-as-summand-addition-left-inverse
On the local basis, c_χ sends $u_\chi(z)$ to $u_\chi(z) \otimes u_\chi(0)$, and η_χ sends the second factor to 1. □

Lemma 13.7.81 (Symmetry equation). Lean: conditional-interface:lemma:weil-as-summand-addition-symmetry

Uses: def:weil-as-summand-addition-forward

Assuming the leaves needed for the additive comparison, swap $c_\chi = \sigma^* c_\chi$ *for* $\sigma(t_1, t_2) = (t_2, t_1)$.

Proof. Uses: lemma:weil-as-summand-addition-left-inverse,lemma:weil-as-summand-lisse-descent
Both arrows send the local source basis to $u_\chi(z_2) \otimes u_\chi(z_1)$. Local basis detection is supplied by the lisse-descent leaf. □

Lemma 13.7.82 (Associativity equation). Lean: conditional-interface:lemma:weil-as-summand-addition-associativity

Uses: def:weil-as-summand-addition-forward

Assuming the leaves needed for the additive comparison, the two iterates of c_χ *on* Y^3 *are equal.*

Proof. Uses: lemma:weil-as-summand-addition-left-inverse,lemma:weil-as-summand-lisse-descent
Each iterate sends $u_\chi(z_1 + z_2 + z_3)$ to $u_\chi(z_1) \otimes u_\chi(z_2) \otimes u_\chi(z_3)$. Local basis detection is supplied by the lisse-descent leaf. □

13.7.7 Pairing and inverse-character duality

Lemma 13.7.83 (Pairing independent of fibre labelling). Lean: conditional-interface:lemma:weil-as-summand-pairing-relabel

Uses: def:weil-as-summand-ambient-pairing

Assuming the pairing-dual leaf, the geometric-stalk formula for B *is unchanged by a permutation of the finite fibre.*

Proof. Uses: def:weil-as-summand-ambient-pairing
Relabelling only permutes the p terms in the finite sum. □

Lemma 13.7.84 (Ambient pairing Frobenius square). Lean: conditional-interface:lemma:weil-as-summand-ambient-pairing-frobenius

Uses: def:weil-as-summand-ambient-pairing, def:weil-as-summand-ambient-frobenius-forward

Assuming the pairing-dual and Frobenius leaves, B *commutes with geometric Frobenius.*

Proof. Uses: lemma:weil-as-summand-pairing-relabel
Frobenius permutes the finite fibre, and the normalized sum is invariant under that permutation. □

Lemma 13.7.85 (Restricted character pairing). Lean: conditional-interface:lemma:weil-as-summand-restricted-character-pairing

Uses: def:weil-as-summand-restricted-pairing, def:weil-as-summand-local-character-sections

Assuming the finite-pushforward, image, exact-pullback, and pairing-dual leaves, $b_\chi(u_\chi(z), u_{\chi^{-1}}(z)) = 1$.

Proof. Uses: lemma:weil-as-summand-local-inclusion,lemma:weil-as-summand-character-sum
After applying the two inclusions, the value is $p^{-1} \sum_a \chi(a)\chi(a)^{-1} = 1$. □

Lemma 13.7.86 (Perfectness of the restricted pairing). *Lean:* `conditional-interface:lemma:weil-as-summand-pairing-perfect`

Uses: `def:weil-as-summand-restricted-pairing`

Assuming the leaves needed for rank-one lissity and pairing-dual, b_χ is perfect.

Proof. Uses: `lemma:weil-as-summand-lisse-descent,lemma:weil-as-summand-restricted-character-pairing`

On the torsor cover its matrix in the two rank-one bases is (1). The pairing-dual leaf makes this local perfectness detect perfectness in the adic category. □

Lemma 13.7.87 (Duality left inverse). *Lean:* `conditional-interface:lemma:weil-as-summand-dual-left-inverse`

Uses: `def:weil-as-summand-dual-forward,def:weil-as-summand-dual-reverse`

Assuming the leaves needed for rank-one lissity and pairing-dual, $C_\chi D_\chi = \mathrm{id}_{\mathcal{L}_{\chi^{-1}}}$.

Proof. Uses: `lemma:weil-as-summand-pairing-perfect,lemma:weil-as-summand-restricted-character-pairing`

The composite fixes the local basis $u_{\chi^{-1}}(z)$; the pairing-dual leaf makes this local calculation detect the arrow. □

Lemma 13.7.88 (Duality right inverse). *Lean:* `conditional-interface:lemma:weil-as-summand-dual-right-inverse`

Uses: `def:weil-as-summand-dual-forward,def:weil-as-summand-dual-reverse`

Assuming the leaves needed for rank-one lissity and pairing-dual, $D_\chi C_\chi = \mathrm{id}_{\mathcal{L}_\chi^\vee}$.

Proof. Uses: `lemma:weil-as-summand-pairing-perfect,lemma:weil-as-summand-restricted-character-pairing`

The composite fixes the local dual basis $u_\chi(z)^\vee$; the pairing-dual leaf makes this local calculation detect the arrow. □

Lemma 13.7.89 (Duality Frobenius square). *Lean:* `conditional-interface:lemma:weil-as-summand-dual-frobenius`

Uses: `def:weil-as-summand-dual-forward,def:weil-as-summand-image-frobenius-forward`

Assuming the image, Frobenius, and pairing-dual leaves, D_χ commutes with the induced geometric-Frobenius arrows.

Proof. Uses: `lemma:weil-as-summand-inclusion-frobenius-square,lemma:weil-as-summand-ambient-pairing-frobenius`

Currying the Frobenius square for b_χ gives the displayed square; the pairing-dual leaf supplies the required currying comparison. □

13.7.8 Conditional closure and exact OPEN boundary

Theorem 13.7.90 (Conditional closure of the Artin–Schreier summand cluster). *Lean:* `conditional-interface:thm:weil-as-summand-conditional-closure`

Uses: `def:weil-wpdf-artin-schreier-summand-and-its-required-maps,def:weil-as-summand-direct-sum-forward,def:weil-as-summand-direct-sum-reverse,def:weil-as-summand-image-frobenius-forward,def:weil-as-summand-image-frobenius-reverse,def:weil-as-summand-rigidification-forward,def:weil-as-summand-rigidification-reverse,def:weil-as-summand-addition-forward,def:weil-as-summand-addition-reverse,def:weil-as-summand-restricted-pairing,def:weil-as-summand-dual-forward,def:weil-as-summand-dual-reverse`

If the eight atomic OPEN leaves listed below are discharged, then the objects and arrows in this cluster supply the two image splitting equations, the explicit character direct sum, an invertible Frobenius structure and its squares, rank-one lissity and constructibility, the closed-point trace, the two rigidification equations, the two additive inverse equations and their unit, symmetry, and associativity equations, and the perfect inverse-character pairing with both duality inverse equations. Under those hypotheses all properties formerly grouped into one aggregate summand obligation are proved.

Proof. Uses: lemma:weil-as-summand-image-factorization,lemma:weil-as-summand-image-retraction,lemma:weil-as-summand-direct-sum-left-inverse,lemma:weil-as-summand-direct-sum-right-inverse,lemma:weil-as-summand-image-frobenius-left-inverse,lemma:weil-as-summand-image-frobenius-right-inverse,lemma:weil-as-summand-inclusion-frobenius-square,lemma:weil-as-summand-retraction-frobenius-square,lemma:weil-as-summand-lisse-descent,lemma:weil-as-summand-constructible,lemma:weil-as-summand-stalk-trace,lemma:weil-as-summand-rigidification-left-inverse,lemma:weil-as-summand-rigidification-right-inverse,lemma:weil-as-summand-rigidification-frobenius,lemma:weil-as-summand-addition-left-inverse,lemma:weil-as-summand-addition-right-inverse,lemma:weil-as-summand-addition-frobenius,lemma:weil-as-summand-addition-left-unit,lemma:weil-as-summand-addition-right-unit,lemma:weil-as-summand-addition-symmetry,lemma:weil-as-summand-addition-associativity,lemma:weil-as-summand-pairing-perfect,lemma:weil-as-summand-dual-left-inverse,lemma:weil-as-summand-dual-right-inverse,lemma:weil-as-summand-dual-frobenius

Each field in the conclusion is exactly one of the cited declarations. The theorem is conditional because every cited categorical declaration retains the hypotheses of its named atomic leaf. □

Atomic OPEN leaf: coefficient rebinding. Open obligation: OBL-WPDF-AS-SUMMAND-COEFFICIENT-REBINDING

Construct the intended pro-etale/adic $\overline{\mathbf{Q}}_\ell$ categories and the pullback, finite pushforward, tensor, finite-sum, image, and dual operations used above; bind their scalars to the previously fixed ψ and ψ_n. This is the local specialization of the coefficient and coherence gap already recorded in A2.

Atomic OPEN leaf: finite-pushforward function model. Open obligation: OBL-WPDF-AS-SUMMAND-FINITE-PUSHFORWARD-FUNCTION-MODEL

Construct $\mathcal{P}$, every T_a, the strict-henselian/geometric-stalk Map model, its base-change comparison, and the delta-basis arrows θ, υ in the adic category.

Atomic OPEN leaf: categorical image. Open obligation: OBL-WPDF-AS-SUMMAND-IMAGE

Construct each image $\mathcal{L}_\chi$, the factorization arrows r_χ, s_χ, the required epi/mono cancellation interfaces, and the finite direct sums of these images in the adic category.

Atomic OPEN leaf: exact pullback. Open obligation: OBL-WPDF-AS-SUMMAND-EXACT-PULLBACK

Prove exactness for every pullback used here, preservation of the image factorizations, and the finite-sum and tensor pullback comparisons used in the displayed formulas.

Atomic OPEN leaf: Frobenius action. Open obligation: OBL-WPDF-AS-SUMMAND-FROBENIUS

Construct the ambient forward and reverse Frobenius arrows, their action on finite-fibre functions and delta vectors, commutation with all T_a, and the induced image maps and pullback comparisons required for every Frobenius square above.

Atomic OPEN leaf: lisse descent. Open obligation: OBL-WPDF-AS-SUMMAND-LISSE-DESCENT

Prove that the constant rank-one pullback on the Artin–Schreier torsor descends to a rank-one lisse adic sheaf, that local basis computations detect arrow equalities, and that the resulting sheaf is constructible.

Atomic OPEN leaf: addition arrow. Open obligation: OBL-WPDF-AS-SUMMAND-ADDITION

Construct the raw addition pullback-of-functions map, its adic base-change and tensor comparisons, and the sheaf arrow represented by the local prescription for d_χ.

Atomic OPEN leaf: pairing and duality. Open obligation: OBL-WPDF-AS-SUMMAND-PAIRING-DUAL

Construct the normalized fibre pairing as an adic sheaf map, the dual objects, evaluation and currying comparisons, and the local-to-sheaf detection used for perfectness and for the candidate reverse duality arrow.

No Fourier transform, Fourier inversion, fibre-cohomology calculation, or weight estimate is defined or asserted in this cluster.

Definition 13.7.91 (Fourier kernel and transform)**.** Lean: conditional-interface:def:weil-wpdf-fourier-kernel-and-transform

Uses: def:weil-wpdf-artin-schreier-summand-and-its-required-maps

Let A_x, A_y be typed copies of $\mathbf{A}^1_k$, let $\mu(x,y) = xy$, and set $\mathcal{K}_\psi = \mu^*\mathcal{L}_\psi$. Define

$$\mathrm{FT}_\psi(K) = Rp_{y!}(p_x^*K \otimes \mu^*\mathcal{L}_\psi)[1].$$

Definition 13.7.92 (Fourier stalk map). Lean: `conditional-interface:def:weil-wpdf-fourier-stalk-map`

Uses: `def:weil-wpdf-artin-schreier-summand-and-its-required-maps`

For a geometric point $\bar{a} \to A_y$, let

$$\beta_{K,a} : (\mathrm{FT}_\psi\, K)_{\bar{a}} \longrightarrow R\Gamma_c(A_{x,\bar{k}}, K \otimes \mathcal{L}_{\psi_a})[1]$$

be compact-support base change followed by restriction of the kernel, where $\mathcal{L}_{\psi_a}$ is pullback along $x \mapsto ax$.

Proposition 13.7.93 (Fourier stalk formula from named maps). *Lean: `conditional-interface:prop:weil-wpdf-fourier-stalk-formula-from-named-maps`*

Uses: `def:weil-wpdf-fourier-kernel-and-transform`, `def:weil-wpdf-fourier-stalk-map`

If an inverse to every $\beta_{K,a}$, its two inverse equations, naturality, extension-of-geometric-points compatibility, and the Frobenius square have been constructed, then $\beta_{K,a}$ is a natural Frobenius-equivariant isomorphism.

Proof. Uses: `def:weil-wpdf-fourier-kernel-and-transform`, `def:weil-wpdf-fourier-stalk-map`

The two displayed inverse equations establish invertibility. Naturality and Frobenius equivariance are the two asserted compatibility maps. □

Lemma 13.7.94 (Finite Fourier orthogonality). *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-finite-fourier-orthogonality`

For $x, x' \in \mathbf{F}_{q^n}$,

$$\sum_y \psi(\mathrm{Tr}((x-x')y)) = \begin{cases} q^n & x = x', \\ 0 & x \neq x'. \end{cases}$$

Proof. Uses:

If $x = x'$ all summands equal one. Otherwise multiplication by $x - x'$ permutes the field. Translation by an element on which the nontrivial character is not one multiplies the sum by a scalar different from one. Subtracting the two expressions for the sum and using that the coefficient field has characteristic zero gives that the sum is zero. □

Proposition 13.7.95 (Fourier inversion, conditional on constructed maps). *Uses:*

Lean: `conditional-interface:prop:weil-wpdf-fourier-inversion-conditional-on-constructed-maps`

If compact-support composition, base change, projection formula, and the difference-phase calculation provide the diagonal identification

$$Rr_!\mathcal{L}_\psi(y(x-z))[2] \simeq \Delta_!\overline{\mathbf{Q}}_\ell(-1),$$

with named inverses and Frobenius squares, then $\mathrm{FT}_{\psi^{-1}}\,\mathrm{FT}_\psi(K) \simeq K(-1)$.

Proof. Uses:

Expand the two definitions and apply the named composition and base-change maps to integrate first in y. Replace that integral by the displayed diagonal kernel. The diagonal pushforward returns K, and its Tate factor returns $K(-1)$. Reversing these named isomorphisms supplies the inverse. □

Proposition 13.7.96 (Perverse exactness, conditional on constructed maps). *Uses:*

Lean: `conditional-interface:prop:weil-wpdf-perverse-exactness-conditional-on-constructed-maps`

If smooth pullback with relative shift, tensoring by a lisse rank-one object, and $Rp_{y!}$ for the affine projection are left perverse exact, then FT_ψ is perverse exact.

Proof. Uses:

The three stated maps send ${}^pD^{\geq 0}$ to itself. Apply the same argument to ψ^{-1}. Fourier inversion identifies the composites with the t-exact twist (-1); a functor and quasi-inverse which are both left exact are exact, since $F(P) \in {}^pD^{\geq 0}$ and $G(F(P)) \simeq P(-1)$ force the opposite truncation inequality. □

Definition 13.7.97 (Paired real envelope). Uses:

Lean: `conditional-interface:def:weil-wpdf-paired-real-envelope`

For an admissible middle extension K, define

$$\mathcal{G} = \mathrm{FT}_\psi(K) \oplus \mathrm{FT}_{\psi^{-1}}(K^\dagger).$$

Here $K^\dagger$ includes a specified conjugate-dual comparison. Reality means equality of each local characteristic polynomial with its conjugate partner polynomial.

Lemma 13.7.98 (Power sums recover a local characteristic polynomial)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-power-sums-recover-a-local-characteristic-polynomial`

For an r-dimensional Frobenius module, its first r power sums determine the characteristic polynomial.

Proof. Uses:

Newton's recurrence gives $me_m = \sum_{a=1}^{m} (-1)^{a-1} e_{m-a} p_a$ for $1 \le m \le r$. Characteristic zero permits division by m, and the coefficients are $(-1)^m e_m$. □

Proposition 13.7.99 (Paired-envelope reality, conditional on Fourier trace maps)**.** *Uses:*

Lean: `conditional-interface:prop:weil-wpdf-paired-envelope-reality-conditional-on-fourier-trace-maps`

If the two Fourier trace formulas identify the power sums of the two summands as complex conjugates, then $\mathcal{G}$ is real.

Proof. Uses:

At each closed point, apply the preceding Newton recurrence to the paired power sums. Each elementary symmetric coefficient is conjugate to the corresponding coefficient, so the displayed characteristic polynomials have the required paired reality. □

Open obligation: `OBL-WPDF-FT-DIFFERENCE`

OPEN WPDF-FT-DIFFERENCE. Construct the compactly supported cohomology of each nonzero linear Artin–Schreier phase, the zero-phase calculation, restriction maps on and off the diagonal, their gluing map to $\Delta_! \overline{\mathbf{Q}}_\ell(-1)$, its inverse, both inverse equations, and its Frobenius square.

Open obligation: `OBL-WPDF-FT-INVERSION`

OPEN WPDF-FT-INVERSION. Construct compact-support composition, base change, projection formula, the expansion map for the two transforms, the inverse to the resulting η_K, both inverse equations, naturality, and both triangle identities. The same-character calculation must use the sum phase and produce the antidiagonal, not the diagonal.

Open obligation: `OBL-WPDF-PERVERSE`

OPEN WPDF-PERVERSE. Construct the smooth-pullback shift map, lisse tensor preservation, affine left perverse exactness, and exactness of the Tate twist; apply these separately to ψ and ψ^{-1}.

Open obligation: `OBL-WPDF-PAIRED-REALITY`

OPEN WPDF-PAIRED-REALITY. Construct the conjugate-dual input, the two Fourier trace identities over every finite extension, direct-summand maps, and the Frobenius-local characteristic-polynomial comparison.

13.7.9 The affine H^1_c estimate

Definition 13.7.100 (Affine Fourier-admissible datum)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-affine-fourier-admissible-datum`

An affine Fourier-admissible datum consists of a dense open $j : U \hookrightarrow \mathbf{A}^1$, an irreducible lisse Weil sheaf L on U, and a lisse sheaf L' on $\mathbf{A}^1 \setminus \{0\}$. It includes the hypotheses that L is non-geometrically-constant and unramified at infinity, and that Fourier shape identifies $\mathrm{FT}_\psi(j_* L[1])$ with $(j'_* L')[1]$.

Definition 13.7.101 (Real lisse sheaf and PUR clause)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-real-lisse-sheaf-and-pur-clause`

A lisse Weil sheaf G is real when every local Frobenius characteristic polynomial is fixed by the conjugate-dual comparison specified in the paired real envelope. The PUR clause for G says: each irreducible lisse subquotient S of G has a real number $\mathrm{wt}(S)$ for which S is pure of that weight.

Definition 13.7.102 (Local Fourier representation)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-local-fourier-representation`

For L' on the punctured Fourier line, let $V = L'_{\bar{\eta}_0}$ be its geometric-generic stalk at 0. A local Fourier representation consists of the inertia action $I \curvearrowright V$, a geometric Frobenius map F_V, and the relation $F_V I F_V^{-1} = I$. Put $V^I = \{v \in V : gv = v \text{ for all } g \in I\}$.

Definition 13.7.103 (MON(M1) invariant-weight clause)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-mon-m1-invariant-weight-clause`

For a pure local Fourier representation V of weight t, MON(M1) is the assertion that V^I is mixed of weights at most t, together with the Frobenius-stability map of $V^I \subset V$.

Definition 13.7.104 (MON(M2) filtration clause)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-mon-m2-filtration-clause`

MON(M2) is a nilpotent map $N : V \to V(-1)$, a finite increasing Frobenius-stable filtration $M_\bullet V$, the relation $F_V N F_V^{-1} = q^{-1}N$, and isomorphisms $N^r : \mathrm{Gr}^M_{m+r} V \xrightarrow{\sim} \mathrm{Gr}^M_{m-r} V(-r)$ for every required $r \geq 0$.

Definition 13.7.105 (MON(M3) quotient-weight clause)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-mon-m3-quotient-weight-clause`

For an admissible L, MON(M3) consists of a Frobenius module Q_L, a Frobenius-equivariant surjection $c : V \twoheadrightarrow Q_L$ with kernel V^I, and the statement that, if V is pure of weight t, then the nonzero module Q_L is pure of weight $t+1$.

Definition 13.7.106 (Stationary-phase maps)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-stationary-phase-maps`

Let $H = H^1_c(\mathbf{A}^1, j_*L)$. Stationary-phase maps are Frobenius-equivariant maps

$$a : H \longrightarrow V, \qquad c : V \longrightarrow Q_L, \qquad b : Q_L \longrightarrow L_\infty(-1),$$

where a identifies H with V^I, c is the MON(M3) quotient map, and b is an isomorphism. Exactness means a is injective, $\ker c = \operatorname{im} a$, c is surjective, and the three asserted maps commute with geometric Frobenius.

Lemma 13.7.107 (Stationary exact sequence)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-stationary-exact-sequence`

Stationary-phase maps give a Frobenius-equivariant short exact sequence

$$0 \longrightarrow H^1_c(\mathbf{A}^1, j_*L) \xrightarrow{a} V \xrightarrow{c} Q_L \longrightarrow 0$$

and a Frobenius-equivariant identification $Q_L \simeq L_\infty(-1)$.

Proof. Uses:

The three exactness equations in the definition prove the short exact sequence. The map b and its inverse give the displayed quotient identification; its Frobenius commutative square gives equivariance. □

Proposition 13.7.108 (Paired reality supplies a weight for L')**.** *Uses:*

Lean: `conditional-interface:prop:weil-wpdf-paired-reality-supplies-a-weight-for-l`

Assume the paired Fourier envelope $\mathcal{G}$ is real, L' is an irreducible lisse subquotient of $\mathcal{G}$, and the PUR clause holds for $\mathcal{G}$. Then L' is pure of some real weight w'.

Proof. Uses:

The PUR clause applied to the specified irreducible subquotient L' supplies a real number w' and the assertion that L' is pure of weight w'. □

Lemma 13.7.109 (A nonzero pure module has a unique weight)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-a-nonzero-pure-module-has-a-unique-weight`

If a nonzero finite Frobenius module is pure of both weights u and v, then $u = v$.

Proof. Uses:

Choose an eigenvalue a. The two purity conditions give $q^{u/2} = |\iota(a)| = q^{v/2}$. Since $q > 1$, injectivity of the real exponential function gives $u = v$. □

Lemma 13.7.110 (Stationary-phase numerical shift)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-stationary-phase-numerical-shift`

Assume L is pure of weight w, V is pure of weight w', stationary-phase maps exist, and MON(M3) holds. Then $w' = w + 1$.

Proof. Uses:

The Tate-twist calculation makes $L_\infty(-1)$ pure of weight $w+2$. The stationary quotient identification makes Q_L pure of weight $w+2$. MON(M3) makes the same nonzero Q_L pure of weight $w'+1$. Uniqueness of the weight of a nonzero pure module gives $w'+1 = w+2$. □

Proposition 13.7.111 (Affine-line compact-support degree-one estimate)**.** *Uses:*

Lean: `conditional-interface:prop:weil-wpdf-affine-line-compact-support-degree-one-estimate`

Let (j, L, L') be Fourier-admissible, with L pure of weight w. Assume paired-envelope reality, the PUR clause, a pure local representation V, MON(M1), MON(M3), and stationary-phase maps. Then

$$H^1_c(\mathbf{A}^1, j_*L) \text{ is mixed of weights at most } w+1.$$

Proof. Uses:

Paired reality and the PUR proposition make L' pure of some weight w'; thus its local representation V is pure of weight w'. The numerical shift lemma gives $w' = w+1$. MON(M1) gives an upper bound $w+1$ for V^I. The stationary map a identifies $H^1_c(\mathbf{A}^1, j_*L)$ with V^I and commutes with Frobenius, so that bound transfers to compact-support cohomology. □

Open obligation: `OBL-WPDF-PUR-REAL`

OPEN WPDF-PUR-REAL. Construct the conjugate-dual maps and local polynomial comparison defining a real lisse sheaf, with all finite-extension and Frobenius compatibility maps.

Open obligation: `OBL-WPDF-PUR-CRITERION`

OPEN WPDF-PUR-CRITERION. Prove the PUR clause for every real lisse sheaf by constructing scalar-weight witnesses for every irreducible lisse subquotient. No global purity theorem may be substituted for this predicate.

Open obligation: `OBL-WPDF-MON-LOCAL`

OPEN WPDF-MON-LOCAL. Construct inertia, geometric Frobenius, quasi-unipotent reduction, the nilpotent logarithm, and the relation $FNF^{-1} = q^{-1}N$ for the local Fourier representation.

Open obligation: `OBL-WPDF-MON-M1`

OPEN WPDF-MON-M1. Construct the invariant submodule and prove its Frobenius stability and upper-weight inequality.

Open obligation: `OBL-WPDF-MON-M2`

OPEN WPDF-MON-M2. Construct the monodromy filtration, every graded Frobenius module, the maps induced by N^r, their inverses, and both inverse equations.

Open obligation: `OBL-WPDF-MON-M3`

OPEN WPDF-MON-M3. Construct Q_L, the quotient map, kernel identity, nonzeroness witness, purity statement of weight $t+1$, and every Frobenius square.

Open obligation: `OBL-WPDF-LFT-SHAPE`

OPEN WPDF-LFT-SHAPE. Construct the Fourier-shape isomorphism $\mathrm{FT}_\psi(j_*L[1]) \simeq (j'_*L')[1]$, its inverse, and its Frobenius square.

Open obligation: `OBL-WPDF-LFT-STATIONARY`

OPEN WPDF-LFT-STATIONARY. Construct a, c, their exactness witnesses, the invariant-image comparison, and all Frobenius squares.

Open obligation: `OBL-WPDF-LFT-QUOTIENT`

OPEN WPDF-LFT-QUOTIENT. Construct $b : Q_L \simeq L_\infty(-1)$, its inverse, both inverse equations, and its Frobenius-equivariance.

13.7.10 Projective and compact-support curve layers

Definition 13.7.112 (Projective curve upper-bound predicate)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-projective-curve-upper-bound-predicate`

For a proper curve C, a dense open $j : U \hookrightarrow C$, and a lisse sheaf L pure of weight w on U, write

$$\mathrm{CurveUpper}(C, j, L, w) \quad :\Longleftrightarrow \quad H^i(C, j_*L) \text{ is mixed of weights at most } w + i \quad \text{for every } i.$$

Definition 13.7.113 (Finite-map reduction data)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-finite-map-reduction-data`

For a proper geometrically connected curve C, finite-map reduction data are a finite map $h : C \to \mathbf{P}^1$, the exact Frobenius-equivariant pushforward h_*, a cohomology comparison

$$R\Gamma(C, j_*L) \xrightarrow{\sim} R\Gamma(\mathbf{P}^1, h_*j_*L),$$

its inverse, both inverse equations, and a finite stratification of $\mathbf{P}^1$ on whose dense open the pushforward is lisse. It also records that h_* preserves the input upper weight and that the complementary punctual quotient has the same upper bound.

Proposition 13.7.114 (Finite-map reduction to the projective line)**.** *Uses:*

Lean: `conditional-interface:prop:weil-wpdf-finite-map-reduction-to-the-projective-line`

If finite-map reduction data exist, then proving $\mathrm{CurveUpper}(\mathbf{P}^1, j', h_*L, w)$ *for the lisse dense-open part and its punctual quotient proves* $\mathrm{CurveUpper}(C, j, L, w)$*.*

Proof. Uses:

The cohomology comparison transfers the desired bound from the pushforward to C. The finite stratification gives an exact sequence from the lisse dense-open part and the punctual quotient to h_*j_*L. The long exact cohomology sequence and the extension bound for Frobenius modules transfer the two upper bounds to every cohomology group of the pushforward. □

Definition 13.7.115 (Normalization reduction data)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-normalization-reduction-data`

For a reduced proper curve C, normalization data are a finite normalization $\nu : \widetilde{C} \to C$, a Frobenius-equivariant map $L \to \nu_*\nu^*L$, and a punctual cokernel P giving an exact sequence

$$0 \longrightarrow L \longrightarrow \nu_*\nu^*L \longrightarrow P \longrightarrow 0.$$

The data include the finite-pushforward cohomology comparison, its inverse, both inverse equations, and the upper weight of P.

Proposition 13.7.116 (Normalization reduction)**.** *Uses:*

Lean: `conditional-interface:prop:weil-wpdf-normalization-reduction`

Assume normalization reduction data and the projective curve upper bound for every connected component of $\widetilde{C}$*. If the punctual cokernel has weights at most* w*, then* $\mathrm{CurveUpper}(C, j, L, w)$ *holds.*

Proof. Uses:

Finite pushforward identifies cohomology of $\widetilde{C}$ with that of $\nu_*\nu^*L$, so the assumed component bounds give upper weights $w + i$. The punctual sheaf has cohomology only in degree zero and weight at most w. The long exact sequence of the displayed normalization sequence gives the same upper bounds for L. □

Definition 13.7.117 (Irreducible-filtration reduction)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-irreducible-filtration-reduction`

An irreducible-filtration reduction of a lisse sheaf L is a finite Frobenius-stable filtration with irreducible lisse nonzero graded factors. For a pure L of weight w, it includes the proof that every graded factor is pure of weight w.

Proposition 13.7.118 (Irreducible reduction)**.** *Uses:*

Lean: `conditional-interface:prop:weil-wpdf-irreducible-reduction`

If the projective curve upper bound holds for every irreducible graded factor of an irreducible-filtration reduction of L*, then it holds for* L*.*

Proof. Uses:
Apply cohomology to each exact sequence between two consecutive filtration terms. The long exact sequence expresses each group for the larger term as an extension of a subquotient of groups already bounded by $w+i$. The subquotient and extension bound for Frobenius modules closes the induction. □

Definition 13.7.119 (Constant curve cohomology data)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-constant-curve-cohomology-data`

For a smooth projective geometrically connected curve C, constant curve cohomology data are Frobenius-equivariant Kunneth maps

$$H^i(C,\overline{\mathbf{Q}}_\ell)\otimes M \xrightarrow{\sim} H^i(C,\underline{M}),$$

their inverses and inverse equations, together with the upper bound $H^i(C,\overline{\mathbf{Q}}_\ell)$ of weight at most i.

Proposition 13.7.120 (Geometrically constant case)**.** *Uses:*

Lean: `conditional-interface:prop:weil-wpdf-geometrically-constant-case`

If M is pure of weight w and constant curve cohomology data exist, then $\mathrm{CurveUpper}(C,\mathrm{id},\underline{M},w)$ *holds.*

Proof. Uses:
The tensor-weight lemma bounds $H^i(C,\overline{\mathbf{Q}}_\ell)\otimes M$ by $i+w$. The Kunneth map and its Frobenius square transfer this bound to $H^i(C,\underline{M})$. □

Definition 13.7.121 (Open–closed curve localization data)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-open-closed-curve-localization-data`

For $j:U\hookrightarrow C$ with closed complement $i:D\hookrightarrow C$, localization data are the Frobenius-equivariant triangle

$$j_!L\longrightarrow j_*L\longrightarrow i_*B\longrightarrow j_!L[1],\qquad B=i^*j_*L,$$

the resulting Frobenius-equivariant long exact cohomology sequence, and identifications $H^r(C,i_*B)=0$ for $r\neq 0$ and $H^0(C,i_*B)=H^0(D,B)$.

Lemma 13.7.122 (Punctual term in the localization sequence)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-punctual-term-in-the-localization-sequence`

If B is mixed of weights at most w, then $H^r(C,i_*B)$ is mixed of weights at most $w+r$ for every r.

Proof. Uses:
The localization data make the group zero for $r\neq 0$. The zero module has every upper bound. At $r=0$, the specified identification preserves Frobenius and gives the assumed upper bound w. □

Proposition 13.7.123 (Compact-support upper estimate)**.** *Uses:*

Lean: `conditional-interface:prop:weil-wpdf-compact-support-upper-estimate`

Assume $\mathrm{CurveUpper}(C,j,L,w)$*, open–closed curve localization data, and that the boundary sheaf $B=i^*j_*L$ is mixed of weights at most w. Then $H^i_c(U,L)$ is mixed of weights at most $w+i$ for every i.*

Proof. Uses:
The localization triangle identifies $H^i_c(U,L)$ with $H^i(C,j_!L)$. The preceding lemma bounds the punctual term by $w+i$, and the curve upper predicate gives the same bound for $H^i(C,j_*L)$. Exactness of the long cohomology sequence expresses $H^i(C,j_!L)$ as an extension of a subquotient of those two terms, so the Frobenius-module extension lemma proves the claim. □

Definition 13.7.124 (Curve duality data)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-curve-duality-data`

For a smooth projective curve, curve duality data are perfect Frobenius-equivariant pairings

$$H^i(C,j_*L)\otimes H^{2-i}(C,j_*L^\vee)\longrightarrow \overline{\mathbf{Q}}_\ell(-1),$$

with explicit adjoint maps, their inverses, and both inverse equations.

Proposition 13.7.125 (Why an upper bound suffices)**.** *Uses:*

Lean: `conditional-interface:prop:weil-wpdf-why-an-upper-bound-suffices`

Assume curve duality data and that the projective curve upper predicate holds for both L of weight w and $L^\vee$ of weight $-w$. Then $H^i(C,j_*L)$ is pure of weight $w+i$.

Proof. Uses:

The upper predicate gives the upper bound $w+i$. Let α be an eigenvalue in degree i. Perfect Frobenius-equivariant duality pairs it with an eigenvalue β in degree $2-i$ for $L^\vee$ satisfying $\alpha\beta = q$. The dual upper bound gives $|\iota(\beta)| \le q^{(-w+2-i)/2}$, hence $|\iota(\alpha)| \ge q^{(w+i)/2}$. The two inequalities give equality for every α. □

Definition 13.7.126 (Mixed constructible curve devissage)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-mixed-constructible-curve-devissage`

A mixed constructible sheaf F on a curve has a finite Frobenius-stable filtration whose graded pieces are either pure lisse sheaves on a dense open extended by zero, or punctual sheaves. The devissage data include every short exact sequence between filtration terms and the open–closed localization sequence for each lisse graded piece.

Proposition 13.7.127 (Mixed constructible compact-support devissage)**.** *Uses:*

Lean: `conditional-interface:prop:weil-wpdf-mixed-constructible-compact-support-devissage`

Suppose every pure lisse graded piece of a mixed constructible sheaf has the compact-support upper estimate with its stated weight, and every punctual graded piece has that bound in degree zero. Then $H^i_c(C,F)$ has the upper weight prescribed by the filtration bound of F.

Proof. Uses:

For a punctual graded piece the conclusion is the assumed degree-zero bound. For a lisse graded piece it is the compact-support upper estimate. Induct on the finite filtration. Each long exact compact-support sequence yields a subquotient and an extension of already bounded Frobenius modules, and the weight-extension lemma preserves the requested upper bound. □

Open obligation: `OBL-WPDF-CURVE-FINITE-MAP`

OPEN WPDF-CURVE-FINITE-MAP. Construct a finite map to $\mathbf{P}^1$ after the permitted finite-field extension, exact finite pushforward, the cohomology comparison and inverse, weight preservation, stratification, and the punctual quotient comparison.

Open obligation: `OBL-WPDF-CURVE-NORMALIZATION`

OPEN WPDF-CURVE-NORMALIZATION. Construct finite normalization, the map to its pushforward, the punctual cokernel, its weight calculation, and all Frobenius/cohomology comparison maps.

Open obligation: `OBL-WPDF-CURVE-IRREDUCIBLE`

OPEN WPDF-CURVE-IRREDUCIBLE. Construct finite Frobenius-stable irreducible filtrations and prove purity of their graded factors without invoking semisimplicity of pure perverse sheaves.

Open obligation: `OBL-WPDF-CURVE-CONSTANT`

OPEN WPDF-CURVE-CONSTANT. Construct the constant-sheaf Kunneth maps and prove the upper weights of constant curve cohomology.

Open obligation: `OBL-WPDF-CURVE-LOCALIZATION`

OPEN WPDF-CURVE-LOCALIZATION. Construct $j_! \to j_* \to i_*B$, the Frobenius-equivariant long exact sequence, the punctual cohomology identifications, and the boundary upper weight supplied by MON(M1).

Open obligation: `OBL-WPDF-CURVE-DUALITY`

OPEN WPDF-CURVE-DUALITY. Construct the curve duality pairing, its adjoint inverses, their equations, and the geometric-Frobenius multiplier q on $\overline{\mathbf{Q}}_\ell(-1)$.

Open obligation: `OBL-WPDF-CURVE-MIXED-DEVISSAGE`

OPEN WPDF-CURVE-MIXED-DEVISSAGE. Construct the finite weight filtration, lisse/punctual stratification, and each compact-support exact sequence used by the mixed constructible induction.

13.7.11 Controlled tame relative curves

Definition 13.7.128 (Relative-curve and compactification data). Uses:

Lean: `conditional-interface:def:weil-wpdf-relative-curve-and-compactification-data`

A relative curve is a separated finite-type map $f : X \to Y$ of relative dimension one. A relative compactification consists of an open immersion $j : X \hookrightarrow \bar{X}$, a smooth projective relative curve $\bar{f} : \bar{X} \to Y$, a closed immersion $i : D \hookrightarrow \bar{X}$, and a finite etale map $g : D \to Y$, with $X = \bar{X} \setminus D$ and $f = \bar{f}j$.

Definition 13.7.129 (Tame boundary monodromy data). Uses:

Lean: `conditional-interface:def:weil-wpdf-tame-boundary-monodromy-data`

Let L be pure lisse of weight w on X. Tame boundary monodromy data are the inertia representation at every geometric point of D, a tame inertia subgroup acting through a finite prime-to-characteristic quotient, the Frobenius-stable invariant module, and a boundary sheaf $B = i^*j_*L$ whose stalk is identified with that invariant module. The data include the boundary upper-weight statement $B \in \mathrm{Mixed}_{\le w}$ and its compatibility with geometric fibres.

Definition 13.7.130 (Relative boundary localization data). Uses:

Lean: `conditional-interface:def:weil-wpdf-relative-boundary-localization-data`

Relative boundary localization data are the Frobenius-equivariant triangle

$$j_!L \longrightarrow j_*L \longrightarrow i_*B \longrightarrow j_!L[1],$$

its image under $R\bar{f}_*$, the identification $R\bar{f}_*i_*B \simeq Rg_*B$, and the resulting long exact sequence of cohomology sheaves. They also include $R^a g_*B = 0$ for $a \neq 0$ and a Frobenius-equivariant finite-etale pushforward map for g_*B.

Definition 13.7.131 (Projective fibre comparison data). Uses:

Lean: `conditional-interface:def:weil-wpdf-projective-fibre-comparison-data`

For $P^r = R^r\bar{f}_*j_*L$, projective fibre comparison data consist of, for each geometric $\bar{y} \to Y$, maps

$$(P^r)_{\bar{y}} \xrightarrow{\mathrm{bc}_{r,\bar{y}}} H^r(\bar{X}_{\bar{y}}, j_{\bar{y},*}L_{\bar{y}})$$

and inverses with both inverse equations. They include compatibility with geometric Frobenius and extension of geometric points, and the assertion that after the declared shrinking each P^r is lisse and pure of weight $w + r$.

Definition 13.7.132 (Controlled tame reduction data). Uses:

Lean: `conditional-interface:def:weil-wpdf-controlled-tame-reduction-data`

Controlled tame reduction data for a relative curve and lisse sheaf consist of a dense shrinking of Y, a finite radicial base change, a finite cover of the resulting X, a direct-summand map recovering L, and a relative compactification with tame boundary monodromy for the covering sheaf. Every map commutes with geometric Frobenius and base change to a geometric fibre.

Lemma 13.7.133 (Finite etale boundary preserves the upper bound). *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-finite-etale-boundary-preserves-the-upper-bound`

If B is mixed of weights at most w and $g : D \to Y$ is finite etale with the stated stalk comparison, then g_*B is mixed of weights at most w.

Proof. Uses:

At a geometric point $\bar{y}$, the stalk of g_*B is the finite direct sum of stalks of B over points above $\bar{y}$. Frobenius permutes these summands and its characteristic polynomial is obtained from finitely many Frobenius powers on them. Every resulting eigenvalue has modulus at most the bound for the corresponding stalk of B, so the direct-sum filtration gives the upper bound w. □

Proposition 13.7.134 (Fibrewise projective curve weights). *Uses:*

Lean: `conditional-interface:prop:weil-wpdf-fibrewise-projective-curve-weights`

Assume projective fibre comparison data and the projective curve purity result on every geometric fibre. Then P^r is lisse and pure of weight $w + r$ after the declared shrinking.

Proof. Uses:

For each geometric $\bar{y}$, the comparison map and its inverse identify the stalk of P^r with the projective-curve cohomology of the fibre. Fibrewise projective purity gives its weight $w + r$. The lissity and family purity clauses in the comparison data promote these compatible stalk statements to the asserted lisse sheaf statement. □

Definition 13.7.135 (Relative compact-support cohomology sheaves)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-relative-compact-support-cohomology-sheaves`

For the controlled relative curve, set

$$A^r = R^r f_! L, \qquad P^r = R^r \bar{f}_* j_* L, \qquad E = g_* B.$$

The relative boundary localization data provide the connecting maps among these Frobenius-equivariant sheaves.

Lemma 13.7.136 (Low-degree relative localization sequence)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-low-degree-relative-localization-sequence`

Relative boundary localization data give exact Frobenius-equivariant sequences

$$0 \to A^0 \to P^0 \to E \to A^1 \to P^1 \to 0, \qquad A^2 \xrightarrow{\sim} P^2,$$

provided the declared relative-curve amplitude makes $P^r = A^r = 0$ outside $0 \le r \le 2$.

Proof. Uses:

Take cohomology sheaves of the relative localization triangle. The vanishing of higher $R^a g_* B$ removes all boundary terms except E in degree zero. The stated amplitude removes the outer terms, leaving the displayed exact sequence and the degree-two isomorphism. □

Proposition 13.7.137 (Degree-zero controlled relative-curve bound)**.** *Uses:*

Lean: `conditional-interface:prop:weil-wpdf-degree-zero-controlled-relative-curve-bound`

Assume P^0 is pure of weight w and the low-degree relative localization sequence. Then A^0 is mixed of weights at most w.

Proof. Uses:

The first arrow identifies A^0 with a Frobenius-stable submodule of P^0. The subquotient weight lemma transfers the upper bound w. □

Proposition 13.7.138 (Degree-one controlled relative-curve bound)**.** *Uses:*

Lean: `conditional-interface:prop:weil-wpdf-degree-one-controlled-relative-curve-bound`

Assume P^0 is pure of weight w, E is mixed of weights at most w, P^1 is pure of weight $w+1$, and the low-degree relative localization sequence. Then A^1 is mixed of weights at most $w+1$.

Proof. Uses:

The cokernel of $P^0 \to E$ has upper weight w. Exactness makes it a Frobenius-stable submodule of A^1, while the quotient of A^1 by this submodule is a Frobenius-stable submodule of P^1 and has upper weight $w+1$. The extension bound gives A^1 upper weight $w+1$. □

Proposition 13.7.139 (Degree-two controlled relative-curve bound)**.** *Uses:*

Lean: `conditional-interface:prop:weil-wpdf-degree-two-controlled-relative-curve-bound`

Assume P^2 is pure of weight $w+2$ and the low-degree relative localization sequence. Then A^2 is pure, hence mixed of weights at most, $w+2$.

Proof. Uses:

The degree-two isomorphism identifies A^2 with P^2 as a Frobenius module. Purity and the asserted upper bound transfer across the isomorphism. □

Proposition 13.7.140 (Controlled tame relative-curve estimate)**.** *Uses:*

Lean: `conditional-interface:prop:weil-wpdf-tame-relative-curve`

Assume a relative compactification, tame boundary monodromy, relative boundary localization, projective fibre comparison, projective fibre purity, finite etale boundary pushforward, and the relative-curve amplitude. Then

$$R^r f_! L \text{ is mixed of weights at most } w + r \quad (0 \le r \le 2).$$

Proof. Uses:

The fibrewise proposition gives P^r pure of weight $w+r$. Tame boundary monodromy and finite-etale pushforward give E upper weight w. Apply the degree-zero, degree-one, and degree-two propositions in turn. □

Proposition 13.7.141 (Direct-summand descent for a controlled cover)**.** *Uses:*

Lean: `conditional-interface:prop:weil-wpdf-tame-summand-descent`

If L is a Frobenius-equivariant direct summand of a covering sheaf M and the controlled tame relative-curve estimate holds for M, then it holds for L.

Proof. Uses:

Functoriality of $Rf_!$ carries the splitting maps of L and M to splitting maps of every $R^r f_!$. Thus $R^r f_! L$ is a Frobenius-stable direct summand, hence a subquotient, of $R^r f_! M$. The subquotient weight lemma transfers the upper bound. □

Open obligation: `OBL-WPDF-TAME-SHRINK`

OPEN WPDF-TAME-SHRINK. Construct the dense shrinking and prove that its complement is handled only by a later target localization devissage.

Open obligation: `OBL-WPDF-TAME-RADICIAL`

OPEN WPDF-TAME-RADICIAL. Construct the finite radicial base-change comparison, its inverse, both inverse equations, and all Frobenius squares.

Open obligation: `OBL-WPDF-TAME-COVER`

OPEN WPDF-TAME-COVER. Construct the finite cover, its direct- summand maps, splitting equations, and fibrewise/base-change compatibility.

Open obligation: `OBL-WPDF-TAME-COMPACTIFICATION`

OPEN WPDF-TAME-COMPACTIFICATION. Construct j, $\bar{f}$, i, the equality $X = \bar{X} \setminus D$, smooth projectivity of $\bar{f}$, and finite etaleness of $D \to Y$.

Open obligation: `OBL-WPDF-TAME-MONODROMY`

OPEN WPDF-TAME-MONODROMY. Construct tame inertia, invariant modules, the comparison with $B = i^* j_* L$, the boundary upper weight, and compatibility with each geometric fibre. This leaf may use only the explicit MON(M1) statement after it is constructed.

Open obligation: `OBL-WPDF-TAME-BASE-CHANGE`

OPEN WPDF-TAME-BASE-CHANGE. Construct the projective fibre maps, their inverses, inverse equations, geometric-point extension maps, and Frobenius squares; prove lissity and purity of P^r after shrinking.

Open obligation: `OBL-WPDF-TAME-LOCALIZATION`

OPEN WPDF-TAME-LOCALIZATION. Construct the relative localization triangle, its $R\bar{f}_*$ image, the comparison to $Rg_* B$, the long exact sequence, and the finite-etale pushforward stalk comparison.

Open obligation: `OBL-WPDF-TAME-AMPLITUDE`

OPEN WPDF-TAME-AMPLITUDE. Prove $R^r f_! L = R^r \bar{f}_* j_* L = 0$ outside degrees $0, 1, 2$ in this controlled family.

13.7.12 D1–D2: support, normalization, and radicial descent

Definition 13.7.142 (D1 fixed cohomological output)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-d1-fixed-output`

For one fixed map $f : X \to Y$ and one fixed bounded Weil input K, a D1 cohomological output is a bounded sequence of finite Frobenius modules $T_f^n(K)$, functorial maps induced by a specified localization triangle, and the resulting Frobenius-equivariant long exact sequence. This definition makes no assertion for arbitrary maps or arbitrary inputs.

Definition 13.7.143 (D1 upper-bound target)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-d1-upper-target`

For a D1 cohomological output and real w, define

$$\mathrm{DOneUpper}(f, K, w) \quad :\Longleftrightarrow \quad T_f^n(K) \text{ is mixed of weights at most } w + n \text{ for all } n.$$

Definition 13.7.144 (D1 open–closed localization data)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-d1-open-closed-data`

For $j : U \hookrightarrow X$ open with closed complement $i : Z \hookrightarrow X$, D1 data are a Frobenius-equivariant triangle

$$j_! j^* K \longrightarrow K \longrightarrow i_* i^* K \longrightarrow j_! j^* K[1],$$

maps from its three terms to the three fixed cohomological outputs, and a Frobenius-equivariant long exact sequence

$$\cdots \to T^n_{fj}(j^*K) \to T^n_f(K) \to T^n_{fi}(i^*K) \to T^{n+1}_{fj}(j^*K) \to \cdots .$$

Lemma 13.7.145 (D1 one-step support bound)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-d1-one-step`

If D1 open–closed localization data exist and both $\mathrm{DOneUpper}(fj, j^*K, w)$ *and* $\mathrm{DOneUpper}(fi, i^*K, w)$ *hold, then* $\mathrm{DOneUpper}(f, K, w)$ *holds.*

Proof. Uses:

For each n, exactness makes $T^n_f(K)$ an extension of the kernel of the outgoing map to $T^n_{fi}(i^*K)$ by a quotient of $T^n_{fj}(j^*K)$. Both pieces have upper weight $w+n$ by the two hypotheses. The subquotient and extension weight lemma gives the same bound for $T^n_f(K)$. Every map commutes with Frobenius by the D1 data. □

Definition 13.7.146 (D1 finite support stratification)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-d1-finite-stratification`

A D1 support stratification of X is a finite ordered family of locally closed strata $S_1, \dots, S_m$ with disjoint union X, such that $X_r = \bigcup_{a \le r} S_a$ is closed in X_{r+1} for every $r < m$. It includes the open–closed localization data for $X_r \hookrightarrow X_{r+1} \hookleftarrow S_{r+1}$ and the restriction of K to every stratum.

Definition 13.7.147 (D1 termination measure)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-d1-termination`

For a D1 support stratification, define its termination measure to be the nonnegative integer m, the number of strata. Removing the final open stratum changes the measure from m to $m-1$.

Proposition 13.7.148 (D1 finite support devissage)**.** *Uses:*

Lean: `conditional-interface:prop:weil-wpdf-d1-finite-support`

Assume a D1 support stratification and, for every S_a*, the upper-bound target for the restricted fixed output with bound* w*. Then* $\mathrm{DOneUpper}(f, K, w)$ *holds.*

Proof. Uses:

Induct on the termination measure. For $m = 1$ the conclusion is the assumed stratum statement. For $m > 1$, apply the induction hypothesis to X_{m-1} and the assumed statement to S_m. The D1 one-step lemma glues the two bounds. The measure decreases by one, so the induction terminates. □

Definition 13.7.149 (D2 finite normalization data)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-d2-normalization-data`

Let X be a fixed reduced finite-type curve with finite normalization $\nu : \widetilde{X} \to X$. D2 normalization data are a Frobenius-equivariant triangle

$$K \longrightarrow \nu_* \nu^* K \longrightarrow P \longrightarrow K[1],$$

where P is supported on the finite non-normal locus, together with maps, inverses, and both inverse equations comparing the fixed outputs of $f\nu$ and of f applied to $\nu_*\nu^*K$.

Definition 13.7.150 (D2 normalization defect)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-d2-normalization-defect`

For D2 normalization data, the termination measure is

$$\delta(X) = \mathrm{length}_{\mathcal{O}_X}(\nu_* \mathcal{O}_{\widetilde{X}} / \mathcal{O}_X).$$

It is zero exactly when the displayed normalization map is an isomorphism; otherwise the defect term P has finite support and is handled by D1.

Proposition 13.7.151 (D2 normalization descent)**.** *Uses:*

Lean: `conditional-interface:prop:weil-wpdf-d2-normalization-descent`

Assume D2 normalization data, upper bounds for the fixed output of $f\nu$ *on* ν^*K*, and the D1 upper-bound target for the finite-support term* P*. Then* $\mathrm{DOneUpper}(f, K, w)$ *holds.*

Proof. Uses:

The finite-output comparison transfers the upper bound from $f\nu$ to the middle term $\nu_*\nu^*K$. The normalization triangle gives a long exact Frobenius sequence with the finite-support term P. Apply the D1 one-step argument to the two bounded terms. If $\delta(X) > 0$, the only new support is finite; D1 handles it. If $\delta(X) = 0$, the normalization comparison is an isomorphism and the conclusion is immediate. □

Definition 13.7.152 (D2 finite and radicial transport data)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-d2-radicial-data`

For a fixed finite map $p : X' \to X$ or finite radicial map $u : X' \to X$, D2 transport data are pullback and pushforward comparison maps on the chosen input and fixed cohomological outputs, inverse maps, both inverse equations, and Frobenius-commutative squares. In the radicial case, the maps identify the chosen Weil categories and their geometric stalks.

Proposition 13.7.153 (D2 finite and radicial descent)**.** *Uses:*

Lean: `conditional-interface:prop:weil-wpdf-d2-finite-radicial-descent`

If D2 finite and radicial transport data identify the fixed output for K with the output after the declared finite or radicial transport, then the upper-bound target holds before transport if and only if it holds after transport.

Proof. Uses:

The two output comparison maps are Frobenius-equivariant inverses. They preserve every characteristic polynomial and hence every upper weight bound in every degree. Apply the same argument to the inverse comparison for the converse. □

Open obligation: `OBL-WPDF-D1-TRIANGLE`

OPEN WPDF-D1-TRIANGLE. Construct the open–closed triangle, its Frobenius structure, and the maps to the fixed cohomological outputs.

Open obligation: `OBL-WPDF-D1-LES`

OPEN WPDF-D1-LES. Construct the connecting maps, exactness proof, and every Frobenius square of the D1 long exact sequence.

Open obligation: `OBL-WPDF-D1-STRATIFICATION`

OPEN WPDF-D1-STRATIFICATION. Construct finite ordered support stratifications, their restriction maps, and the closed/open decomposition at every induction step.

Open obligation: `OBL-WPDF-D1-TERMINATION`

OPEN WPDF-D1-TERMINATION. Prove finiteness of the support stratification and the strict decrease of the stated integer measure.

Open obligation: `OBL-WPDF-D2-NORMALIZATION`

OPEN WPDF-D2-NORMALIZATION. Construct finite normalization, the normalization triangle, the finite-support defect term, and all Frobenius compatibility maps.

Open obligation: `OBL-WPDF-D2-FINITE-COMPARISON`

OPEN WPDF-D2-FINITE-COMPARISON. Construct the finite-pushforward cohomological comparison, inverse, both inverse equations, and its Frobenius-equivariance.

Open obligation: `OBL-WPDF-D2-RADICIAL`

OPEN WPDF-D2-RADICIAL. Construct the radicial Weil-category and geometric-stalk comparisons, inverses, inverse equations, and output transport squares.

Open obligation: `OBL-WPDF-D2-DEFECT`

OPEN WPDF-D2-DEFECT. Prove finiteness of $\delta(X)$, the zero-defect normality criterion, and finite support of the non-normal defect.

13.7.13 D3–D4: coefficients and target localization

Definition 13.7.154 (D3 coefficient exact-sequence data)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-d3-exact-data`

For the fixed map and fixed cohomological output of D1, D3 data consist of a Frobenius-equivariant short exact sequence

$$0 \longrightarrow K' \longrightarrow K \longrightarrow K'' \longrightarrow 0,$$

its associated triangle $K' \to K \to K'' \to K'[1]$, and the Frobenius-equivariant long exact sequence

$$\cdots \to T_f^n(K') \to T_f^n(K) \to T_f^n(K'') \to T_f^{n+1}(K') \to \cdots .$$

Every comparison includes the connecting map and its Frobenius commutative square.

Definition 13.7.155 (D3 coefficient-filtration length)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-d3-length`

A D3 coefficient filtration is a finite Frobenius-stable filtration $0 = K_0 \subset K_1 \subset \cdots \subset K_m = K$ with graded objects $G_a = K_a/K_{a-1}$. Its termination measure is m; removing the final graded object changes the measure to $m - 1$.

Lemma 13.7.156 (D3 exact-sequence one-step bound)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-d3-one-step`

Given D3 exact-sequence data, if $\mathrm{DOneUpper}(f, K', w)$ *and* $\mathrm{DOneUpper}(f, K'', w)$ *hold, then* $\mathrm{DOneUpper}(f, K, w)$ *holds.*

Proof. Uses:

For each n, exactness identifies $T_f^n(K)$ as an extension of the image from $T_f^n(K')$ by the kernel of the connecting map out of $T_f^n(K'')$. The first piece is a quotient and the second a submodule of modules with upper weight $w + n$. Subquotient stability and the extension lemma give the same bound for $T_f^n(K)$. Frobenius compatibility follows from the commutative squares included in the D3 data. □

Proposition 13.7.157 (D3 finite coefficient devissage)**.** *Uses:*

Lean: `conditional-interface:prop:weil-wpdf-d3-filtration`

If a D3 coefficient filtration is given and $\mathrm{DOneUpper}(f, G_a, w)$ *holds for every graded object, then* $\mathrm{DOneUpper}(f, K, w)$ *holds.*

Proof. Uses:

Induct on the filtration length. The zero-length case is the zero object. For positive length, the exact sequence $0 \to K_{m-1} \to K_m \to G_m \to 0$ has D3 data. Apply the induction hypothesis to K_{m-1} and the assumed bound to G_m, then use the D3 one-step lemma. The length decreases by one at each induction step. □

Definition 13.7.158 (D4 target localization data)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-d4-target-data`

Fix one bounded constructible Weil output $G = T_f(K)$ on Y. D4 target data consist of an open immersion $v : V \hookrightarrow Y$, its closed complement $t : T \hookrightarrow Y$, fixed base-changed outputs G_V, G_T, and Frobenius-equivariant comparison isomorphisms

$$v^*G \xrightarrow{\sim} G_V, \qquad t^*G \xrightarrow{\sim} G_T$$

with inverses and both inverse equations. They also include the target localization triangle

$$v_!v^*G \longrightarrow G \longrightarrow t_*t^*G \longrightarrow v_!v^*G[1]$$

and its Frobenius-equivariant long exact sequence of cohomology sheaves.

Definition 13.7.159 (D4 target-stratification complexity)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-d4-target-complexity`

A D4 target stratification is a finite ordered decomposition $Y = Y_1 \sqcup \cdots \sqcup Y_s$ such that the union of the first a strata is closed in the union of the first $a + 1$. It includes D4 target localization data for every successive open–closed pair. Its termination measure is the integer s, decreased to $s - 1$ after removing the last open stratum.

Lemma 13.7.160 (D4 target stalk gluing)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-d4-stalk-gluing`

Assume D4 target localization data and upper derived-weight bounds w *for both fixed outputs* G_V *and* G_T*. Then the fixed output* G *has upper derived weight* w*.*

Proof. Uses:

For each cohomological degree n, the localization long exact sequence makes $\mathcal{H}^n(G)$ an extension of a quotient of $\mathcal{H}^n(v_!v^*G)$ by a subobject of $\mathcal{H}^n(t_*t^*G)$. At a geometric point of V, the first term is identified with the corresponding stalk of G_V and the closed term is zero; at a geometric point of T, the roles reverse and the comparison is with G_T. The two comparison isomorphisms commute with Frobenius. Subquotient and extension stability therefore give upper weight $w + n$ at every stalk. □

Proposition 13.7.161 (D4 finite target localization devissage)**.** *Uses:*

Lean: `conditional-interface:prop:weil-wpdf-d4-target-localization`

Assume a D4 target stratification and an upper derived-weight bound w for the restriction of the fixed output to every stratum. Then G has upper derived weight w on Y.

Proof. Uses:

Induct on the target-stratification complexity. A single stratum is the assumed restricted bound. For $s > 1$, the induction hypothesis handles the closed union of the first $s - 1$ strata, and the hypothesis handles the last open stratum. Apply the D4 target stalk-gluing lemma to the recorded open–closed pair. The integer complexity decreases strictly, so the induction terminates. □

Open obligation: `OBL-WPDF-D3-TRIANGLE`

OPEN WPDF-D3-TRIANGLE. Construct the triangle associated with a Frobenius-equivariant coefficient short exact sequence and its connecting map.

Open obligation: `OBL-WPDF-D3-LES`

OPEN WPDF-D3-LES. Construct the fixed-output long exact sequence, its exactness proof, and every Frobenius commutative square.

Open obligation: `OBL-WPDF-D3-FILTRATION`

OPEN WPDF-D3-FILTRATION. Construct finite Frobenius-stable coefficient filtrations and their graded short exact sequences.

Open obligation: `OBL-WPDF-D3-TERMINATION`

OPEN WPDF-D3-TERMINATION. Prove finiteness and strict decrease of the coefficient-filtration length.

Open obligation: `OBL-WPDF-D4-BASE-CHANGE`

OPEN WPDF-D4-BASE-CHANGE. Construct the two target base-change comparisons, their inverses, both inverse equations, and Frobenius squares.

Open obligation: `OBL-WPDF-D4-TRIANGLE`

OPEN WPDF-D4-TRIANGLE. Construct the target open–closed triangle, its cohomology-sheaf long exact sequence, exactness, and Frobenius squares.

Open obligation: `OBL-WPDF-D4-STRATIFICATION`

OPEN WPDF-D4-STRATIFICATION. Construct the finite target stratification and D4 data on every successive pair.

Open obligation: `OBL-WPDF-D4-TERMINATION`

OPEN WPDF-D4-TERMINATION. Prove finiteness and strict decrease of the target-stratum count.

13.7.14 D5–D6: composition and relative dimension zero

Definition 13.7.162 (D5 composition data)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-d5-composition-data`

Fix maps $h : X \to Z$ and $g : Z \to Y$, their composite $f = gh$, and one fixed input K. D5 composition data consist of fixed Frobenius-equivariant outputs $T_h^b(K)$, $T_g^a(T_h^b(K))$, and $T_f^n(K)$, together with a bounded spectral sequence

$$E_2^{a,b} = T_g^a(T_h^b(K)) \Longrightarrow T_f^{a+b}(K).$$

Every differential commutes with Frobenius. The data include the finite abutment filtration, its graded comparisons with $E_\infty^{a,b}$, inverse maps and both inverse equations, and the filtration triangles and long exact sequences.

Definition 13.7.163 (D5 filtration length)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-d5-filtration-length`

For bounded D5 data, choose integers $b_{\min} \le b_{\max}$ outside which $T_h^b(K) = 0$. The termination measure is $N = b_{\max} - b_{\min} + 1$. Removing the final nonzero filtration grade changes N to $N - 1$.

Lemma 13.7.164 (D5 page and graded-weight bound)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-d5-page-bound`

Assume $T_h^b(K)$ has upper weight $w+b$ and the fixed g-output applied to it has upper weight $(w+b)+a$. Then every $E_r^{a,b}$ for $r \ge 2$, and every corresponding $E_\infty^{a,b}$, has upper weight $w+a+b$.

Proof. Uses:

The stated g-output hypothesis gives the bound on $E_2^{a,b}$. For each r, the next page is the quotient of the kernel of the outgoing Frobenius-equivariant differential by the image of the incoming one. Subquotient stability preserves the bound from page r to page $r+1$. Boundedness makes the pages stationary, giving the same bound on $E_\infty^{a,b}$. □

Proposition 13.7.165 (D5 composition devissage)**.** *Uses:*

Lean: `conditional-interface:prop:weil-wpdf-d5-composition`

Under D5 composition data and the hypotheses of the D5 page-bound lemma for every a, b, the fixed composite output $T_f^n(K)$ has upper weight $w+n$ for every n.

Proof. Uses:

For fixed n, each abutment graded factor is an $E_\infty^{a,n-a}$ and therefore has upper weight $w+n$. Use the filtration triangles from the D5 data one grade at a time. Their Frobenius-equivariant long exact sequences express each filtration term as an extension of subquotients of the preceding term and the new graded factor. The extension lemma propagates the bound through all N grades. The filtration terminates because N decreases by one. □

Definition 13.7.166 (D6 quasi-finite factorization data)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-d6-quasifinite-data`

Fix a separated relative-dimension-zero map $f : X \to Y$ and one fixed input K. D6 data consist of a finite locally closed stratification $X = \bigsqcup_{a=1}^m X_a$ and, for each stratum, a factorization

$$X_a \xrightarrow{j_a} \overline{X}_a \xrightarrow{p_a} Y,$$

where j_a is open, p_a is finite, and $f|_{X_a} = p_a j_a$. They include Frobenius-equivariant comparisons between the fixed output on X_a and the finite output of $j_{a,!}K|_{X_a}$, with inverses, both inverse equations, and compatibility with every target base change and geometric stalk.

Definition 13.7.167 (D6 stratum complexity)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-d6-complexity`

The D6 termination measure is the number m of quasi-finite strata. Removing the final open stratum leaves the closed union of the first $m-1$ strata and decreases the measure by one. The measure is zero only for the empty source.

Lemma 13.7.168 (D6 finite-map weight bound)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-d6-finite-bound`

Let $p : W \to Y$ be one of the fixed finite maps and let M have upper weight w. If the finite-stalk and Frobenius comparisons exist, then the fixed finite output is concentrated in degree zero and has upper weight w.

Proof. Uses:

At a geometric point $\bar{y}$, the degree-zero stalk is the finite direct sum of stalks of M over the fibre. Frobenius permutes the summands and acts by the recorded stalk Frobenius maps, so the induced orbit characteristic polynomials preserve the upper weight w. Exactness of finite pushforward gives zero in nonzero degrees. The base-change comparison transfers this calculation to every geometric stalk of the fixed output. □

Proposition 13.7.169 (D6 relative-dimension-zero devissage)**.** *Uses:*

Lean: `conditional-interface:prop:weil-wpdf-d6-relative-zero`

Assume D6 quasi-finite factorization data, that open extension by zero preserves the input upper weight, and the D6 finite-map bound on every stratum. Then the fixed output for f has upper weight w in degree zero and vanishes in nonzero degrees.

Proof. Uses:

For each stratum, $j_{a,!}$ preserves stalk weights and the finite-map lemma gives the asserted concentrated output after p_a. Induct on the D6 stratum complexity. The D1 localization triangle and Frobenius long exact sequence glue the last open stratum to the closed union of the preceding strata. Subquotient and extension stability preserve the degree-zero bound; exactness and the zero outer terms preserve vanishing in nonzero degrees. The measure decreases from m to $m-1$, so the induction terminates. □

Open obligation: `OBL-WPDF-D5-SPECTRAL-SEQUENCE`

OPEN WPDF-D5-SPECTRAL-SEQUENCE. Construct the fixed composition spectral sequence, its differentials, convergence map, and every Frobenius commutative square.

Open obligation: `OBL-WPDF-D5-BOUNDEDNESS`

OPEN WPDF-D5-BOUNDEDNESS. Construct finite amplitude bounds for the fixed h- and g-outputs and prove eventual page stability.

Open obligation: `OBL-WPDF-D5-FILTRATION`

OPEN WPDF-D5-FILTRATION. Construct the abutment filtration, graded comparisons and inverses, both inverse equations, filtration triangles, and long exact sequences.

Open obligation: `OBL-WPDF-D5-TERMINATION`

OPEN WPDF-D5-TERMINATION. Prove finiteness and strict decrease of the D5 filtration-length measure.

Open obligation: `OBL-WPDF-D6-QUASIFINITE`

OPEN WPDF-D6-QUASIFINITE. Construct the finite locally closed quasi-finite stratification and each open-into-finite factorization.

Open obligation: `OBL-WPDF-D6-FINITE-OUTPUT`

OPEN WPDF-D6-FINITE-OUTPUT. Construct finite-pushforward exactness, finite-stalk comparisons, inverse equations, and Frobenius actions.

Open obligation: `OBL-WPDF-D6-OPEN-EXTENSION`

OPEN WPDF-D6-OPEN-EXTENSION. Construct open extension by zero, prove weight preservation, and give its target base-change and Frobenius compatibilities.

Open obligation: `OBL-WPDF-D6-TERMINATION`

OPEN WPDF-D6-TERMINATION. Prove finiteness and strict decrease of the D6 stratum-count measure.

13.7.15 Fixed-degree direct-image assembly

Definition 13.7.170 (Fixed-degree direct-image goal)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-rfi-fixed-goal`

Fix one separated finite-type map $f : X \to Y$, one constructible Weil sheaf K over a coefficient field E with upper weight w, and one integer i. A fixed-degree goal consists of one chosen sheaf

$$A(f, K, i) = T_f^i(K)$$

and, at every geometric point $\bar{y} \to Y$, its finite Frobenius module $A(f, K, i)_{\bar{y}}$. The target assertion is that these modules have upper weight $w + i$.

For every target base change used below, the data include a comparison from the pullback of $A(f, K, i)$ to the corresponding fixed output, its inverse, both inverse equations, compatibility with extension of geometric points, and every Frobenius square. For every finite coefficient extension E'/E used below, they similarly include

$$A(f, K, i) \otimes_E E' \longleftrightarrow A(f, K \otimes_E E', i)$$

with inverse equations and Frobenius compatibility. This definition fixes a single degree and does not construct a derived object $Rf_!K$.

Definition 13.7.171 (Fixed local factorization data)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-rfi-local-factorization-data`

Local factorization data for the fixed goal consist of a finite ordered affine target refinement $Y = \bigsqcup_t V_t$ and, over each V_t, a finite ordered affine source refinement $X \times_Y V_t = \bigsqcup_u U_{tu}$. The refinements carry all D4 and D1 successive open–closed data. On every U_{tu}, the restriction of K has a finite D3 filtration with pure lisse graded pieces. For every such grade there are an integer $N_{tu} \geq 0$ and a factorization

$$U_{tu} \xrightarrow{j_{tu}} \mathbf{A}^{N_{tu}}_{V_t} \xrightarrow{p_{N_{tu}}} \cdots \longrightarrow \mathbf{A}^1_{V_t} \xrightarrow{p_1} V_t,$$

where j_{tu} is a closed immersion and every p_a forgets one coordinate. For $N_{tu} = 0$, this display means only the closed immersion $j_{tu} : U_{tu} \hookrightarrow \mathbf{A}^0_{V_t} = V_t$, with no projection. The datum records the finite lists, restriction maps, D1/D3/D4 labels, and the designated D2 finite/radicial transports and TAME shrinking data used on the eventual relative-one pieces.

The datum is a finite factorization atlas, not only root data: for every prefix $p_a \cdots p_1 j_{tu}$ and every intermediate fixed D5 output degree in its declared finite amplitude, it contains the corresponding finite target and source refinements and pure-lisse D3 grades used when that output becomes the coefficient of p_{a+1}. Projection prefixes, amplitude ranges, and all three refinement lists are finite, so the atlas has finitely many entries.

Every displayed map is defined over k. For each designated target base change $V' \to V_t$, the datum includes the base-changed factorization, the canonical comparisons $(\mathbf{A}^a_{V_t})_{V'} \leftrightarrow \mathbf{A}^a_{V'}$, their inverses and both inverse equations, and the Frobenius squares for the immersion, projections, restrictions, and coefficient grades.

Lemma 13.7.172 (Affine generator factorization and projection chain)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-rfi-affine-factorization`

Let $U = \operatorname{Spec} A \to V = \operatorname{Spec} B$ be a fixed finite-type map. A finite set of B-algebra generators of A gives a closed immersion $U \hookrightarrow \mathbf{A}^N_V$, and the affine projection to V is the composite of the N coordinate-forgetting relative-one projections. This factorization commutes with every target base change and geometric Frobenius.

Proof. Uses:

Generators give a surjection $B[x_1, \ldots, x_N] \twoheadrightarrow A$, hence the closed immersion. The structural map from the polynomial algebra is the successive omission of $x_N, \ldots, x_1$. Tensoring the surjection with a B-algebra B' gives the same construction over $V' = \operatorname{Spec} B'$, with canonical inverse affine-space comparisons. All rings and maps are over k, so the induced squares commute with the Galois action and geometric Frobenius. □

Definition 13.7.173 (Target affine-cover atlas)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-rfi-target-cover-atlas`

For each root target request Y_τ in the fixed factorization atlas, choose a finite ordered affine open cover $\mathcal{U}_\tau = (U_{\tau,1}, \ldots, U_{\tau,s_\tau})$. Close this list under the intermediate D5 targets: for an affine cover member V and every coordinate prefix, the target $\mathbf{A}^a_V$ receives its singleton affine cover. For a designated base change $g : Y' \to Y_\tau$, choose a finite affine cover $\mathcal{U}'_\tau$ and a refinement function c such that

$$U'_{\tau,b} \subseteq g^{-1}(U_{\tau,c(b)}).$$

The atlas also contains finite affine common refinements of every two cover choices and of their designated pullbacks. All opens and refinement maps are defined over k.

Definition 13.7.174 (Ordered affine partition from a cover)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-rfi-target-affine-partition`

For one cover $(U_1, \dots, U_s)$, put

$$S_a = U_a \setminus \bigcup_{b<a} U_b$$

and discard the empty S_a. Order the remaining strata in decreasing a. Writing them as $T_1, \dots, T_t$, require the D4 data for

$$Z_r = \bigcup_{c \le r} T_c \hookrightarrow Z_{r+1} \hookleftarrow T_{r+1}.$$

Thus Z_r is the closed part and T_{r+1} the open complement at the $(r+1)$st gluing step. Coordinate-space singleton covers have $t = 1$.

Definition 13.7.175 (Open–closed and Cech target descent data)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-rfi-target-descent-data`

Let G be a fixed root or intermediate D5 output on a target Y. The open–closed datum is the D4 triangle and long exact sequence at every $Z_r \hookrightarrow Z_{r+1} \hookleftarrow T_{r+1}$, with comparisons, inverses, both equations, and Frobenius squares.

For a nonempty index set $I \subseteq \{1, \dots, s\}$, put $U_I = \bigcap_{a \in I} U_a$ and let $j_I : U_I \to Y$. The Cech datum is the augmented finite complex

$$G \longrightarrow C^0(G) \longrightarrow C^1(G) \longrightarrow \cdots, \qquad C^p(G) = \bigoplus_{|I|=p+1} j_{I*} j_I^* G,$$

with alternating restriction maps, an exactness witness, refinement comparisons and inverses, the cocycle equations for iterated refinements, and all geometric-Frobenius and designated-base-change squares. For compatible local Frobenius-stable weight filtrations, the datum includes descended subobjects on G, their finite filtration, and restriction isomorphisms back to every U_I.

Definition 13.7.176 (Target-atlas complexity and choice equivalence)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-rfi-target-complexity`

For the finite target-cover atlas, define

$$\kappa = \sum_{\tau} \max(t_\tau - 1, 0),$$

where t_τ is the number of nonempty ordered strata for that entry, including the singleton coordinate targets, which contribute zero. Thus κ is the number of nontrivial D4 gluing steps still present in the finite atlas. Two cover atlases are choice-equivalent when a finite affine common refinement carries Cech comparison maps to both, satisfying both inverse equations and the refinement cocycle. Choice equivalence concerns the resulting fixed output and its upper-weight predicate; it does not assert equality of two arbitrarily chosen filtration witnesses.

Lemma 13.7.177 (A finite affine cover gives the required D4 partition)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-rfi-target-cover-to-partition`

The reverse ordered nonempty S_a form a finite affine D4 target stratification of Y.

Proof. Uses:

S_a is closed in the affine scheme U_a, hence affine. The strata are disjoint and cover Y. Before empty strata are discarded, the union of the reverse first r strata is

$$Y \setminus \bigcup_{b<s-r+1} U_b,$$

which is closed in Y and in the next partial union. Removing empty terms does not change any partial union. This gives exactly the ordered closed–open pairs required by the D4 target stratification. □

Lemma 13.7.178 (Open–closed and Cech weight-filtration gluing)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-rfi-target-weight-gluing`

Assume local upper-weight filtrations for the fixed output on every affine cover member, compatible on all intersections. The D4 iteration and the Cech descent datum each produce a finite Frobenius-stable filtration on G with the same upper bound.

Proof. Uses:

For one D4 step, the long exact sequence expresses each cohomology sheaf as an extension of a quotient of the open term by a subobject of the closed term. Intersect and quotient the two finite filtrations, then concatenate them; subquotient and extension stability preserve the bound and Frobenius stability. Induction through $Z_1 \subset \cdots \subset Z_t = Y$ gives the first filtration. For the Cech construction, effective filtration descent supplies the descended subobjects, and exactness of the augmented complex identifies their final term with G. Its restriction to every U_I is the prescribed local filtration, so its graded bounds are the same. The two witnesses need not be identical, but both establish the intrinsic upper-weight predicate. □

Lemma 13.7.179 (Target Frobenius and base-change compatibility)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-rfi-target-base-change`

The target-cover construction, both descent procedures, and their weight filtrations commute with geometric Frobenius and every designated target base change, after passing to the recorded affine refinement of the pullback cover.

Proof. Uses:

Every open and coordinate projection is defined over k, so restriction and the alternating Cech maps commute with geometric Frobenius. The refinement function places each chosen affine of Y' inside a pulled-back cover member. Apply the D4 base-change comparisons and the Cech refinement/base-change comparisons, together with their inverses and cocycles. Their commutative squares identify every restricted filtration, connecting map, and descended subobject. Coordinate targets use $(\mathbf{A}^a_V)_{V'} \simeq \mathbf{A}^a_{V'}$, so no additional intermediate target case occurs. □

Lemma 13.7.180 (Strict target-complexity decrease)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-rfi-target-strict-decrease`

At every nontrivial D4 branch, each recursive child has smaller target stratum count than its parent, and processing one D4 gluing step decreases the global complexity κ by one while leaving all other entries fixed.

Proof. Uses:

If an entry has $t > 1$ strata, the closed child contains $t-1$ and the last open child contains one, both strictly fewer than t. In the deterministic schedule each one-stratum child leaves row D4, while the closed child repeats it. Replacing this entry by the two children changes its contribution from $t-1$ to $(t-2)+0$ and changes no other finite-atlas summand. Thus κ decreases by exactly one, and the recursion is finite with both local and global strict descent. □

Lemma 13.7.181 (Independence of the affine-cover choice)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-rfi-target-choice-independence`

Two finite affine-cover atlases satisfying the displayed data yield canonically identified fixed outputs and the same upper-weight assertion, compatibly with Frobenius and designated base change.

Proof. Uses:

Choose the recorded finite affine common refinement. Cech refinement maps in both directions, their inverses, and the cocycle equations identify both augmentations with the one over the common refinement. The Frobenius and base-change squares identify the fixed outputs and restricted filtrations. Alternatively, each ordered open–closed induction proves the same intrinsic stalkwise upper-weight predicate on G. Hence cover order, redundant cover members, and the choice of common refinement cannot change the output or its bound, although the chosen filtration witnesses may differ. □

Proposition 13.7.182 (Realization of target affine refinement)**.** *Uses:*

Lean: `conditional-interface:prop:weil-wpdf-rfi-target-affine-realization`

Assume the three exact target leaves displayed below and the existing D4 localization/base-change data. Then every root and intermediate D5 target in the fixed factorization atlas has finite affine target-refinement data, strictly decreasing complexity, Frobenius/base-change-compatible weight gluing, and choice-independent output.

Proof. Uses:

Use the affine-basis leaf to choose each finite root cover and all common or base-changed refinements. Close it under coordinate affine spaces by singleton covers. The cover-to-partition lemma constructs the finite ordered D4 strata. The two descent leaves and the D4 data construct the open–closed and Cech witnesses and their compatible weight filtrations. Apply the base-change, strict-decrease, and choice-independence lemmas. This constructs all target-side fields of the finite factorization atlas and uses no source affine or pure-lisse refinement. □

Open obligation: `OBL-WPDF-RFI-TARGET-AFFINE-BASIS`

OPEN WPDF-RFI-TARGET-AFFINE-BASIS. For each fixed root target and designated pullback, prove noetherian quasi-compactness, construct finite affine subcovers of the required quasi-compact opens, and construct finite affine common refinements of two choices, all over k. This leaf contains no sheaf or weight assertion.

Open obligation: `OBL-WPDF-RFI-TARGET-CECH-DESCENT`

OPEN WPDF-RFI-TARGET-CECH-DESCENT. Construct the augmented finite Cech complex for the fixed constructible Weil output, prove its exactness, and construct refinement and target-base-change comparisons, inverses, both inverse equations, cocycles, and Frobenius squares. This leaf contains no filtration-effectivity assertion.

Open obligation: `OBL-WPDF-RFI-TARGET-FILTRATION-DESCENT`

OPEN WPDF-RFI-TARGET-FILTRATION-DESCENT. Prove effective descent of a finite compatible family of Frobenius-stable subobjects and weight filtrations through the fixed Cech datum, including restriction isomorphisms, graded identifications, refinement cocycles, and base-change/Frobenius compatibility. D4 continues to own open–closed filtration gluing.

Closed assembly label WPDF-RFI-TARGET-AFFINE-REFINEMENT. Proposition 13.7.182 constructs the former target-refinement package from the three exact target leaves and D4. The closed name is no longer an OPEN dependency and closes neither D4 nor the three inputs.

Definition 13.7.183 (Source affine-cover atlas)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-rfi-source-cover-atlas`

Let $h_\sigma : X_\sigma \to V_\sigma$ range over the finite list of source requests in the factorization atlas: the root source over every fixed target affine and the source of every coordinate-prefix map in every intermediate D5 degree. Write its chosen D1 support stratification as

$$\varnothing = X_{\sigma,0} \subset X_{\sigma,1} \subset \cdots \subset X_{\sigma,m_\sigma} = X_\sigma, \qquad S_{\sigma,e} = X_{\sigma,e} \setminus X_{\sigma,e-1}.$$

For each nonempty $S_{\sigma,e}$ choose a finite ordered affine open cover $\mathcal{W}_{\sigma,e} = (W_{\sigma,e,1}, \ldots, W_{\sigma,e,s_{\sigma,e}})$. A coordinate-affine source $\mathbf{A}^a_V$ with no nontrivial D1 support split receives its singleton cover.

For every designated target base change $V' \to V_\sigma$, choose finite affine covers of the pulled-back source strata and refinement functions into the inverse images of the $W_{\sigma,e,a}$. The atlas also contains finite affine common refinements of every two source-cover choices and of their designated pullbacks. All source strata, opens, and refinement maps are defined over k.

Definition 13.7.184 (Ordered affine D1 refinement)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-rfi-source-affine-partition`

Inside one $S_{\sigma,e}$ put

$$R_{\sigma,e,a} = W_{\sigma,e,a} \setminus \bigcup_{b<a} W_{\sigma,e,b},$$

discard the empty terms, and order the remainder in decreasing a. Concatenate these lists in increasing e and denote the resulting affine locally closed strata by $P_{\sigma,1}, \ldots, P_{\sigma,r_\sigma}$. Put

$$H_{\sigma,c} = \bigcup_{d \le c} P_{\sigma,d}.$$

The source-refinement datum requires the D1 triangle and long exact sequence for every $H_{\sigma,c} \hookleftarrow H_{\sigma,c+1} hookleftarrow P_{\sigma,c+1}$.

Definition 13.7.185 (Source Cech and compact-support descent data)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-rfi-source-descent-data`

Fix one old D1 stratum $s : S \to X_\sigma$ and its restricted coefficient K_S. For a nonempty index set I, put $W_I = \bigcap_{a \in I} W_a$, let $u_I : W_I \to S$, and write $\ell_I = s u_I$. The source Cech datum is the finite augmented extension-by-zero complex

$$\cdots \longrightarrow C^{-2}(K_S) \longrightarrow C^{-1}(K_S) \longrightarrow C^0(K_S) \longrightarrow s_! K_S,$$
$$C^{-p}(K_S) = \bigoplus_{|I|=p+1} \ell_{I!} u_I^* K_S.$$

with alternating maps, an exactness witness, refinement comparisons and inverses, both inverse equations, iterated-refinement cocycles, and all Frobenius and designated-base-change squares.

The compact-support datum contains the composition comparisons

$$Rh_{\sigma!}\ell_{I!}u_I^*K_S \longleftrightarrow R(h_\sigma\ell_I)_!u_I^*K_S,$$

their inverses and equations, the comparison from the image of the finite Cech totalization to $R(h_\sigma s)_!K_S$, and the induced comparisons on every fixed cohomological output. These maps are coherent with augmentation, refinement, D1 localization, geometric Frobenius, and designated target base change. This datum reconstructs the compactly supported output; it does not assert that termwise weights on the shifted Cech complex imply the desired cohomological weight slope.

Definition 13.7.186 (Source-atlas complexity and choice equivalence)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-rfi-source-complexity`

Define

$$\kappa_X = \sum_\sigma \max(r_\sigma - 1, 0).$$

Thus a singleton coordinate source contributes zero, and κ_X counts the nontrivial D1 gluing steps in the finite source atlas. Two source atlases are choice-equivalent when finite affine common refinements carry compatible input Cech and compact-support comparisons to both, with inverses, equations, refinement cocycles, Frobenius squares, and base-change squares. Equivalence concerns the fixed compactly supported output and its upper-weight predicate, not literal equality of two chosen Cech models.

Lemma 13.7.187 (Affine covers refine the D1 support stratification)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-rfi-source-cover-to-partition`

The concatenated reverse ordered differences form a finite affine D1 support stratification of every root or intermediate D5 source object.

Proof. Uses:

Each $R_{\sigma,e,a}$ is closed in the affine $W_{\sigma,e,a}$ and hence is affine. Within $S_{\sigma,e}$, every reverse partial union is closed by the same complement calculation as for the target cover. If A is such a partial union, then $X_{\sigma,e-1} \cup A$ is closed in $X_{\sigma,e}$: its complement is $S_{\sigma,e} \setminus A$, open first in $S_{\sigma,e}$ and then in $X_{\sigma,e}$. Concatenating in increasing e therefore gives exactly the successive closed–open pairs required by D1. Finiteness follows from the finite old stratification and its finite covers. □

Lemma 13.7.188 (D1 gluing and Cech compact-support reconstruction)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-rfi-source-compact-gluing`

Assume the fixed upper-weight target on every refined affine source stratum. The D1 iteration proves the same bound for the source request, while the source Cech and compact-support data reconstruct its fixed compactly supported derived output and every selected cohomological output.

Proof. Uses: `prop:weil-wpdf-d1-finite-support`

Apply Proposition 13.7.148 to the refined finite D1 stratification. Its one-step proof uses only same-degree kernels, quotients, and extensions, so the upper bound is preserved through all $H_{\sigma,c}$. Separately, exactness of the augmented extension-by-zero Cech complex identifies its finite totalization with $s_!K_S$. Apply $Rh_{\sigma!}$ and the recorded composition comparisons, then glue the old D1 strata. The totalization comparison and its inverse identify the result with $R(h_\sigma s)_!K_S$ and hence with the fixed output. No weight estimate is read from the shifted Cech E_1 terms; the weight assertion is exactly the D1 open–closed induction just given. □

Lemma 13.7.189 (Source Frobenius and base-change compatibility)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-rfi-source-base-change`

The refined D1 triangles, source Cech resolutions, compact-support comparisons, and fixed outputs commute with geometric Frobenius and every designated target base change, after passage to the recorded affine refinement of the pulled-back source cover.

Proof. Uses:

All strata and locally closed immersions are defined over k, so restriction, extension by zero, augmentation, and alternating maps commute with geometric Frobenius. The chosen refinement function places every affine of the base-changed source inside the inverse image of an original cover member. Apply the recorded input-refinement maps and the compact-support composition and base-change comparisons. Their inverses, equations, cocycles, and commutative squares identify the D1 connecting maps, totalizations, and every fixed cohomological output. For a coordinate source use $(\mathbf{A}^a_V)_{V'} \simeq \mathbf{A}^a_{V'}$ and its singleton cover. □

Lemma 13.7.190 (Strict source-complexity decrease)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-rfi-source-strict-decrease`

At every nontrivial refined D1 branch, each recursive child has fewer affine source strata than its parent, and processing one gluing step decreases κ_X by exactly one.

Proof. Uses:

An entry with $r > 1$ strata has a closed child with $r - 1$ strata and a last open child with one. Both are strictly smaller than the parent, and the one-stratum child leaves the D1 row. Replacing the parent contribution $r - 1$ by $(r - 2) + 0$ decreases κ_X by one without changing another atlas entry. Thus both the local recursion and the finite global work list descend strictly. □

Lemma 13.7.191 (Independence of the affine source choice)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-rfi-source-choice-independence`

Two source affine-cover atlases satisfying the displayed data give canonically identified compactly supported fixed outputs and the same upper-weight assertion, compatibly with Frobenius and designated base change.

Proof. Uses:

Pass to the recorded finite affine common refinement on every old D1 stratum. The Cech refinement maps, their inverses, and cocycles identify both extension-by-zero resolutions with the common one. Compact-support composition and base-change comparisons then identify their totalizations and fixed outputs, including every Frobenius square. The refined D1 stratifications prove the same intrinsic upper-weight predicate. Hence cover order, redundant members, and common-refinement choice do not alter the output or bound, although the chosen Cech complexes need not be literally equal. □

Proposition 13.7.192 (Realization of source affine refinement)**.** *Uses:*

Lean: `conditional-interface:prop:weil-wpdf-rfi-source-affine-realization`

Assume the three exact source leaves displayed below and the existing D1 localization data. Then every root and intermediate D5 source object in the fixed factorization atlas has finite affine D1 refinement, Cech and compact-support descent, strictly decreasing complexity, Frobenius/base-change compatibility, and choice-independent fixed output.

Proof. Uses:

Use the source affine-basis leaf on every old D1 stratum, including all root and intermediate source requests, and use singleton covers for the unsplit coordinate sources. The cover-to-partition lemma gives the finite affine D1 stratification. The Cech-resolution and compact-support leaves give the augmentation, totalization, output comparisons, inverses, equations, cocycles, and compatibility squares. Apply the compact-gluing, base-change, strict-decrease, and choice-independence lemmas. This constructs every source-side field of the finite factorization atlas. It uses no pure-lisse refinement and neither invokes nor closes any of the three target input leaves. □

Open obligation: `OBL-WPDF-RFI-SOURCE-AFFINE-BASIS`

OPEN WPDF-RFI-SOURCE-AFFINE-BASIS.

For each old D1 stratum in every fixed root or intermediate source request and designated pullback, prove noetherian quasi-compactness, construct finite affine subcovers and finite affine common refinements over k, and verify the singleton coordinate-source cases. This leaf contains no sheaf, compact-support, or weight assertion.

Open obligation: `OBL-WPDF-RFI-SOURCE-CECH-RESOLUTION`

OPEN WPDF-RFI-SOURCE-CECH-RESOLUTION.

Construct the exact finite augmented extension-by-zero Cech resolution on each old D1 stratum, together with refinement maps, inverses, both equations, iterated-refinement cocycles, and Frobenius/base-change squares. This leaf contains no $Rf_!$ assertion.

Open obligation: `OBL-WPDF-RFI-SOURCE-RF-SHRIEK`

OPEN WPDF-RFI-SOURCE-RF-SHRIEK.

Construct $Rf_!\ell_! \leftrightarrow R(f\ell)_!$, its inverse and equations, the image of the finite Cech totalization, the fixed-output comparisons, compatibility with D1 localization and augmentation, refinement cocycles, and every Frobenius square and base-change square. This leaf contains no purity assertion and no termwise Cech weight inference.

Closed assembly label.

WPDF-RFI-SOURCE-AFFINE-REFINEMENT.

Proposition 13.7.192 constructs the former source-refinement package from the three exact source leaves and D1. The closed name is no longer an OPEN dependency and closes neither D1 nor the three inputs.

Definition 13.7.193 (Pure-lisse request atlas)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-rfi-pure-lisse-atlas`

Let $(S_\eta \to V_\eta, L_\eta, w_\eta)$ range over the finite list of coefficient requests already present on the root source pieces and on every intermediate D5 source degree. The list also records each designated target base change, finite coefficient extension, normalization pullback, and finite radicial pullback used by the reduction tree.

For every irreducible locally closed piece of the reduced support of L_η, choose a descending sequence of reduced closed subsets

$$Z_{\eta,0} \supset Z_{\eta,1} \supset \cdots \supset Z_{\eta,q_\eta} = \varnothing, \qquad U_{\eta,c} = Z_{\eta,c} \setminus Z_{\eta,c+1}.$$

Each $U_{\eta,c}$ is a finite disjoint union of smooth affine k-schemes, $L_\eta|_{U_{\eta,c}}$ is lisse, and every component of $Z_{\eta,c+1}$ has smaller dimension than the irreducible closure being processed. Irreducible components are first made disjoint by ordered locally closed differences. For a one-dimensional closure C, the chosen dense open is contained in the smooth normalization-isomorphism locus, and $C \setminus U$ is a finite reduced union of closed points.

Each designated pullback has its own such sequence and a finite common locally closed refinement with the pulled-back sequence. Refinement maps, strata, and their order are defined over k. No claim is made that normalization itself commutes with an arbitrary target base change.

Definition 13.7.194 (Normalization, punctual, and radicial transport data)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-rfi-pure-lisse-transport`

For every one-dimensional irreducible closure C in the coefficient atlas, the data contain its finite normalization $\nu : \widetilde{C} \to C$, a dense smooth lisse open $j : U \to C$ on which ν is an isomorphism, and the finite punctual complement $i : P = C \setminus U \to C$. They contain the D2 normalization triangle

$$L_C \longrightarrow \nu_*\nu^* L_C \longrightarrow Q_C \longrightarrow L_C[1],$$

where Q_C is supported on P, together with the finite-output comparisons, inverses, inverse equations, and Frobenius squares.

For every declared finite map or finite radicial universal homeomorphism $u : C' \to C$, the data contain the D2 pullback/pushforward comparisons on coefficients, geometric stalks, and fixed outputs, their inverses and both equations, and all Frobenius squares. After a target base change, comparisons are made through the recorded common refined dense open and the chosen normalization of the base-changed curve, rather than by asserting a false unconditional base-change identity for normalization. A punctual coefficient means the finite family of Frobenius modules on the closed points of P.

Definition 13.7.195 (Maximal normalized smooth-lisse locus)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-rfi-locus-maximal-data`

For every coefficient request and every one-dimensional irreducible closure C in its reduced support, let $\nu_C : \widetilde{C} \to C$ be the recorded finite normalization and put $\widetilde{L}_C = \nu_C^* L_C$. Write $\mathrm{Lis}(\widetilde{L}_C)$ for the union of all opens on which $\widetilde{L}_C$ is lisse, and let $I_{\nu_C} \subset \widetilde{C}$ be the inverse image of the locus on which ν_C is an isomorphism. Define

$$\widetilde{U}_C^{\max} = \mathrm{Sm}(\widetilde{C}/k) \cap \mathrm{Lis}(\widetilde{L}_C) \cap I_{\nu_C}, \qquad U_C^{\max} = \nu_C(\widetilde{U}_C^{\max}).$$

The second expression uses the isomorphism on I_{ν_C}. Put $P_C = C \setminus U_C^{\max}$ with its reduced structure and choose a finite affine cover of $U_C^{\max}$; reverse ordered differences give the affine locally closed pieces used in the coefficient atlas. The finite list includes every root and intermediate D5 coefficient request.

Definition 13.7.196 (Renormalized target-base-change comparison)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-rfi-locus-renormalization-data`

Let $g : V' \to V$ be a designated target base change and let D be a one-dimensional irreducible component of $(C \times_V V')_{\mathrm{red}}$, with normalization $\nu_D : \widetilde{D} \to D$ and induced map $g_C : D \to C$. If g_C is dominant, the datum contains the normalization lift $r_D : \widetilde{D} \to \widetilde{C}$ and the common dense open

$$\widetilde{W}_D = \widetilde{U}_D^{\max} \cap r_D^{-1}(\widetilde{U}_C^{\max}).$$

On $\widetilde{W}_D$ it contains the canonical restriction comparison

$$r_D^* \widetilde{L}_C\big|_{\widetilde{W}_D} \longleftrightarrow \nu_D^* g_C^* L_C\big|_{\widetilde{W}_D},$$

its inverse and both equations. For two successive target base changes the data include the corresponding comparison cocycle, and every map has its geometric-Frobenius commutative square. A component not dominating C is a separate coefficient request with its own maximal locus; no generic comparison to C is asserted. In particular, the datum re-normalizes after base change and never identifies $\widetilde{C} \times_V V'$ with $\widetilde{D}$ globally.

Definition 13.7.197 (Locus-refinement complexity)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-rfi-locus-complexity`

For a locus state Q, define

$$\rho_{\mathrm{loc}}(Q) = (d(Q), e(Q), b(Q)) \in \mathbf{N}^3,$$

in lexicographic order. Here $d = 1 + \dim \operatorname{Supp} Q$ for a nonempty state and $d = 0$ for the empty state; $e = 2, 1, 0$ means respectively an unnormalized curve, a normalized curve whose maximal locus is unresolved, or a finished smooth-lisse/punctual piece. A higher-dimensional unresolved support uses $e = 1$, and b counts unprocessed points in a finite punctual complement. The global locus work measure is the finite multiset of these ranks, ordered by the multiset extension. Designated target base changes and common refinements are entries fixed in the finite atlas, not dynamically generated recursive children.

Definition 13.7.198 (Pure and mixed coefficient-filtration data)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-rfi-pure-lisse-filtration`

On every smooth lisse stratum U, a filtered mixed coefficient consists of monomorphisms in the lisse Weil coefficient category

$$0 = W_0 L_U \subset W_1 L_U \subset \cdots \subset W_m L_U = L_U$$

commuting with Frobenius. Each grade is supplied by an exact sequence

$$0 \longrightarrow W_{a-1} L_U \longrightarrow W_a L_U \longrightarrow G_a \longrightarrow 0,$$

and every nonzero G_a is lisse and pure of a recorded weight $v_a \leq w_\eta$. A pure lisse coefficient of weight v has the canonical one-step filtration $0 \subset L_U$ with grade L_U and weight v. On a punctual stratum the same display is a finite filtration of its Frobenius module, viewed as a pure lisse coefficient on that zero-dimensional stratum. The term/grade exactness and all transported filtrations are the coefficient data defined next; D3 still owns coefficient triangles, connecting maps, and fixed-output long exact sequences.

Definition 13.7.199 (Weight subobject and quotient data)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-rfi-weight-exact-data`

For a Frobenius-equivariant exact sequence $0 \to A \to L \to B \to 0$ and a supplied filtration $W_\bullet L$, put

$$W_a A = A \times_L W_a L, \qquad W_a B = \operatorname{im}(W_a L \longrightarrow B).$$

The data contain the monomorphisms, epimorphisms, kernel and cokernel witnesses, and exact sequences

$$0 \to W_a A \to W_a L \to W_a B \to 0, \qquad 0 \to \operatorname{Gr}_a A \to \operatorname{Gr}_a L \to \operatorname{Gr}_a B \to 0.$$

They identify each grade with the displayed successive quotient, with maps, inverses, both equations, and Frobenius squares. Every nonzero subobject or quotient grade is lisse and pure of the weight of $\operatorname{Gr}_a L$. The same data are required for punctual Frobenius modules.

Definition 13.7.200 (Strict filtered morphism and closure certificate)**.** Uses: `def:weil-wpdf-rfi-pure-lisse-filtration`, `def:weil-wpdf-rfi-weight-exact-data`

Lean: `conditional-interface:def:weil-wpdf-rfi-weight-strictness-data`

Let $f : L \to M$ be a Frobenius-equivariant morphism between coefficients with finite Frobenius-stable filtrations $W_\bullet L$ and $W_\bullet M$. Put

$$K = \ker f, \qquad I = \operatorname{im} f, \qquad C = \operatorname{coker} f,$$

and define the induced terms

$$W_a K = K \times_L W_a L, \qquad W_a I = I \times_M W_a M, \qquad W_a C = \operatorname{im}(W_a M \longrightarrow C).$$

A strict filtered-morphism certificate consists of Frobenius-equivariant kernel, image, and cokernel witnesses; finite exhaustive filtrations on K, I, C; and, for every a, the equality

$$f(W_a L) = I \cap W_a M.$$

It also contains lisse or punctual closure witnesses for (K,I,C) and for every finite Frobenius-stable intersection or sum used in these filtrations, together with the comparison maps, inverse equations, and Frobenius squares. This Definition specifies an input package. It does not assert the existence of such a package for every Frobenius-equivariant morphism.

Lemma 13.7.201 (A strictness certificate yields exact filtered terms)**.** Uses: def:weil-wpdf-rfi-weight-strictness-data, def:weil-wpdf-rfi-weight-exact-data

Lean: conditional-interface:lemma:weil-wpdf-rfi-weight-strictness-exact

Assume a Frobenius-equivariant exact sequence $0 \to A \xrightarrow{a} L \xrightarrow{p} B \to 0$*, a supplied filtration* $W_\bullet L$*, and strict filtered-morphism certificates for* a *and* p*. Then*

$$W_r A = A \times_L W_r L, \qquad W_r B = \mathrm{im}(W_r L \longrightarrow B)$$

form a termwise exact sequence for every r*, and the successive quotients form an exact sequence*

$$0 \longrightarrow \mathrm{Gr}_r\, A \longrightarrow \mathrm{Gr}_r\, L \longrightarrow \mathrm{Gr}_r\, B \longrightarrow 0.$$

The successive-quotient comparisons have specified inverses satisfying both inverse equations, all maps commute with Frobenius, and every nonzero end grade is lisse and pure of the same weight as $\mathrm{Gr}_r\, L$*.*

Proof. Uses: def:weil-wpdf-rfi-weight-strictness-data, def:weil-wpdf-rfi-weight-exact-data, lemma:weil-weight-subquotient-at-most, lemma:weil-weight-subquotient-at-least

The strictness equality for p identifies the image of $W_r L$ with $W_r B$. Exactness of the original sequence identifies the kernel of $W_r L \to W_r B$ with $A \cap W_r L = W_r A$. Therefore

$$0 \longrightarrow W_r A \longrightarrow W_r L \longrightarrow W_r B \longrightarrow 0$$

is exact. Write this sequence for (r-1) and for (r), and use the three inclusion maps to form the commutative diagram of short exact sequences. Apply the kernel–cokernel calculation to that diagram. Its three cokernels are $W_r A/W_{r-1}A$, $W_r L/W_{r-1}L$, and $W_r B/W_{r-1}B$, so the resulting cokernel sequence is the displayed graded exact sequence.

The comparison from each cokernel to the corresponding grade and its inverse are the maps supplied by the two quotient universal properties. Substituting one universal factorization into the other gives the identity on the quotient map; quotient epimorphy gives both inverse equations. Every subobject, quotient, and comparison in the certificate is Frobenius-stable, so applying Frobenius to either universal factorization gives the same factorization; uniqueness gives all Frobenius squares. Finally, each end grade is a certified subquotient of $\mathrm{Gr}_r\, L$. The two cited subquotient lemmas give respectively the upper and lower inequalities for its weight; equality of the bounds gives purity of the same weight. The closure field of the certificate supplies lissity or punctuality. □

Definition 13.7.202 (Transported weight-filtration data)**.** Uses:

Lean: conditional-interface:def:weil-wpdf-rfi-weight-transport-data

Let T range over restriction to a recorded common refinement, finite scalar extension E'/E, pullback to an already recorded target base change, normalization pullback, and the recorded finite-radicial equivalence. For every filtered coefficient the data contain exact comparisons

$$T(W_a L) \longleftrightarrow W_a(TL), \qquad T(G_a) \longleftrightarrow \mathrm{Gr}_a(TL),$$

their inverses and both equations, compatibility with the term and grade exact sequences, iterated-transport and common-refinement cocycles, and all geometric-Frobenius squares. The transported nonzero grade has the same recorded weight. Target base change here means only coefficient pullback on an already supplied lisse/common-refined stratum; this datum constructs no maximal locus or re-normalization comparison.

Definition 13.7.203 (Weight-filtration complexity)**.** Uses:

Lean: conditional-interface:def:weil-wpdf-rfi-weight-complexity

For one unresolved filtered coefficient state Q, let $\rho_{\mathrm{wt}}(Q) = m(Q) \in \mathbf{N}$ be the number of its remaining nonzero grades; a terminal pure grade has rank zero. The global measure is the finite multiset of these ranks. Subobject/quotient diagrams, designated transports, and common-refinement comparisons are fixed records in the finite coefficient atlas, not dynamically generated recursive children.

Definition 13.7.204 (Pure-lisse refinement rank)**.** Uses:

Lean: conditional-interface:def:weil-wpdf-rfi-pure-lisse-complexity

For one unresolved coefficient state Q, put

$$\rho_{\mathrm{PL}}(Q) = (d(Q), p(Q), \delta(Q), m(Q)) \in \mathbf{N}^4,$$

ordered lexicographically, where $d(Q) = 1 + \dim \mathrm{Supp}\, Q$ for a nonzero state and $d(0) = 0$. The phase p is respectively $4, 3, 2, 1, 0$ for an unresolved dense-lisse split, curve normalization, finite or radicial transport, mixed-filtration step, or pure-lisse/punctual leaf. The third coordinate is the D2 normalization defect when phase 3 is active and zero otherwise; m is the remaining D3 filtration length when phase 1 is active and zero otherwise. The global work measure is the finite multiset of these ranks, ordered by the multiset extension of the lexicographic order. Choice equivalence identifies the fixed coefficient/output and its intrinsic upper weight predicate, not literal equality of two stratifications or filtrations.

Lemma 13.7.205 (Maximality and punctuality of the normalized lisse locus)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-rfi-locus-maximality`

Assume the exact maximal-locus leaf below and the existing D2 normalization and defect leaves. Then $\widetilde{U}_C^{\max}$ and $U_C^{\max}$ are dense opens defined over k, $U_C^{\max}$ is maximal among opens on which C is smooth, L_C is lisse, and ν_C is an isomorphism, and P_C is finite and punctual.

Proof. Uses:

The maximal-locus leaf makes the smooth, lisse, and normalization-isomorphism loci open and dense. Their finite intersection is therefore dense. On I_{ν_C} the finite normalization is an isomorphism, so its image is the claimed open of C. Any open with the three stated properties pulls back inside each of the three factors and hence inside $\widetilde{U}_C^{\max}$, proving maximality. The complement of a dense open in a noetherian irreducible curve has dimension zero and finitely many closed points. The D2 defect leaf identifies the non-normal part among these points, while the excluded non-lisse locus is zero-dimensional as well; hence the whole reduced complement is a finite punctual support. □

Lemma 13.7.206 (Dense smooth-lisse stratification and punctual curve boundary)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-rfi-pure-lisse-stratification`

Assume the exact maximal-locus leaf displayed below and the existing D2 normalization and defect leaves. Every coefficient request in the finite atlas has a finite ordered smooth affine lisse stratification; on a one-dimensional irreducible closure, the complement of its dense stratum is punctual.

Proof. Uses:

On a curve use $U_C^{\max}$ from the preceding lemma and its finite affine cover; reverse ordered differences form finitely many affine lisse pieces, and P_C is punctual. On an irreducible support of dimension above one, use the maximal-locus leaf to choose a dense affine open in the simultaneous smooth and lisse locus. Its reduced closed complement has strictly smaller dimension. Split its finitely many irreducible components by ordered locally closed differences and repeat. Noetherianity and strict dimension descent terminate the process after finitely many stages. The root/intermediate request list and all designated pullbacks are finite, so the total atlas is finite. □

Lemma 13.7.207 (Common refinement and locus-choice independence)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-rfi-locus-choice-independence`

Any two admissible dense smooth-lisse opens for the same normalized curve have a finite affine common refinement inside $\widetilde{U}_C^{\max}$, and they give the same coefficient restriction data independently of the choice and ordering of affine pieces.

Proof. Uses:

Maximality places both opens inside $\widetilde{U}_C^{\max}$. Their intersection is again dense and open because the curve is irreducible. It is quasi-compact because the curve is noetherian, hence has a finite affine cover. Cover every intersection of old cover members by finitely many affine opens and take reverse ordered differences. Restriction of $\widetilde{L}_C$ along the two composites is the same functorial restriction, giving canonical inverse comparisons and the usual refinement cocycle. Thus the geometric locus and restricted coefficient are independent of cover, order, or common-refinement choice; no filtration or weight assertion is used. □

Lemma 13.7.208 (Comparison after target base change and re-normalization)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-rfi-locus-base-change`

Assume the exact re-normalization leaf below. For every designated target base change and every dominating one-dimensional component D, the old and newly normalized maximum loci have the recorded dense common refinement, and their coefficient restrictions are canonically identified with inverse equations and iterated-base-change cocycles.

Proof. Uses:

The universal property of normalization gives r_D from the normal curve $\widetilde{D}$ to $\widetilde{C}$. Pullback of the old dense lisse open is dense on a dominating component, and pullback preserves lissity there. Intersect it with $\widetilde{U}_D^{\max}$ to obtain $\widetilde{W}_D$. The two composites from $\widetilde{W}_D$ to C agree, so pseudofunctoriality of coefficient pullback gives the displayed comparison and its inverse. The two inverse equations and the cocycle for a second base change are exactly those supplied by the leaf. Non-dominating components enter the already finite request list separately, so no missing comparison or recursive child is introduced. □

Lemma 13.7.209 (Frobenius stability of maximum loci and comparisons)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-rfi-locus-frobenius`

Every maximal locus, punctual complement, common refinement, normalization lift, and re-normalized base-change comparison is stable under geometric Frobenius.

Proof. Uses:

Smoothness, lissity, and the isomorphism locus are intrinsic properties of data defined over k. Geometric Frobenius therefore preserves each factor in the definition of $\widetilde{U}_C^{\max}$ and hence preserves its intersection and P_C; on the finite complement it merely permutes closed points. Intersections defining common refinements are preserved as well. The normalization lifts are maps over k, and the supplied coefficient comparison squares commute with Frobenius. Composition gives the same statement for every iterated target base change. □

Lemma 13.7.210 (Strict locus descent)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-rfi-locus-strict-decrease`

Normalization, maximum-locus splitting, and punctual processing strictly decrease the local locus rank on every recursive child and hence strictly decrease the finite global multiset.

Proof. Uses:

Normalizing a curve preserves d and changes phase $e = 2$ to $e = 1$; its D2 defect children are punctual and have smaller d. Splitting a normalized curve into $\widetilde{U}_C^{\max}$ and P_C changes the dense child from $e = 1$ to $e = 0$, while every punctual child again has smaller d. Processing one of the finitely many points decreases b by one. On a higher-dimensional support, the dense smooth-lisse child changes $e = 1$ to $e = 0$ and every complement child has smaller d. Thus every child is lexicographically smaller. Replacing one multiset entry by finitely many smaller entries is a strict decrease in the well-founded multiset extension, so the locus recursion terminates. □

Proposition 13.7.211 (Realization of the pure-lisse locus refinement)**.** *Uses:*

Lean: `conditional-interface:prop:weil-wpdf-rfi-locus-realization`

Assume the two exact locus leaves below and the existing D2 normalization and defect leaves. Every root and intermediate D5 coefficient request has finite smooth affine lisse locus data, maximum normalized curve opens, finite punctual complements, common refinements, target-base-change re-normalization comparisons, Frobenius stability, choice-independent restrictions, and strictly decreasing locus complexity.

Proof. Uses:

The maximality lemma constructs the canonical dense curve opens and punctual complements; the stratification lemma handles every support dimension in the finite request atlas. Apply the common-refinement lemma to all pairs of choices and the base-change lemma to every designated pullback component. The Frobenius lemma supplies all stability squares, and the strict-decrease lemma terminates the finite work multiset. This realizes every field of the former locus aggregate. It uses no weight filtration and neither invokes nor closes any target or source affine leaf. □

Definition 13.7.212 (Jordan–Holder and semisimplification data)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-rfi-weight-jh-data`

For a lisse Weil coefficient L of rank r on a connected recorded stratum, Jordan–Holder data consist of a chain

$$0 = J_0 \subset J_1 \subset \cdots \subset J_n = L, \qquad S_i = J_i/J_{i-1},$$

of Frobenius-stable lisse subobjects with simple nonzero lisse grades, their term and quotient maps, and the grade identifications with inverse equations. Put $L^{\mathrm{ss}} = \bigoplus_{i=1}^{n} S_i$. Its datum records only the isomorphism class and constituent multiset supplied by Jordan–Holder; it does not assert an isomorphism $L \simeq L^{\mathrm{ss}}$. Each S_i carries a recorded pure weight $v_i \le w$, and $n \le r$.

Definition 13.7.213 (Single-weight certificate for a simple lisse sheaf)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-rfi-weight-simple-certificate`

For a nonzero simple lisse Weil coefficient S on a recorded stratum, a single-weight certificate is a real number v and, for every closed point x, the geometric-Frobenius action on $S_{\bar{x}}$ together with the claim that every root α of its characteristic polynomial satisfies

$$|\iota(\alpha)| = q_x^{v/2}$$

for every chosen complex embedding ι. The certificate records one weight and the Frobenius polynomial comparisons only; it contains no further filtration or extension-ordering choice.

Definition 13.7.214 (Extension ordering and grouped weight layers)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-rfi-weight-order-data`

For a weighted Jordan–Holder chain, let

$$I(J_\bullet) = \#\{(i,j) : i < j \text{ and } v_i > v_j\}$$

be its inversion count. An adjacent inversion $v_i > v_{i+1}$ determines

$$0 \to S_i \to J_{i+1}/J_{i-1} \to S_{i+1} \to 0.$$

An adjacent-swap datum is a Frobenius-equivariant splitting of this sequence, with section, retraction, both equations, and the replacement intermediate subobject that exchanges S_i, S_{i+1} while leaving J_{i-1} and J_{i+1} fixed.

After all inversions are removed, write the distinct weights as $u_1 < \cdots < u_t$ and let b_j be the last index with $v_{b_j} \le u_j$. The grouped layers are

$$W_{u_j} L = J_{b_j}, \qquad G_{u_j} = W_{u_j} L / W_{u_{j-1}} L, \quad W_{u_0} L = 0.$$

All inclusion and quotient maps, grade inverses and equations, and Frobenius squares are recorded.

Definition 13.7.215 (Existence transport for Jordan–Holder data)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-rfi-weight-existence-transport-data`

Let T be finite scalar extension or pullback to an already recorded target base change. Transport the grouped filtration termwise. Each pure $T(S_i)$ may cease to be simple, so choose a Jordan–Holder refinement of it and concatenate these refinements in the old weight order. The data contain

$$T(W_u L) \longleftrightarrow W_u(TL), \qquad T(G_u) \longleftrightarrow \mathrm{Gr}_u(TL),$$

their inverses and both equations, the induced semisimplification comparison, iterated scalar/base-change cocycles, and Frobenius squares. Compatibility means this transported choice; no independence from another filtration and no weight-strictness assertion is included.

Definition 13.7.216 (Existence rank)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-rfi-weight-existence-complexity`

For an ordered-construction state put

$$\rho_{\mathrm{ex}} = (I, t) \in \mathbf{N}^2$$

in lexicographic order, where I is the inversion count and t is the number of unprocessed distinct grouped weights after $I = 0$. The Jordan–Holder length is bounded separately by the rank. Scalar and target-base-change comparisons are fixed finite atlas records and do not create recursive children.

Lemma 13.7.217 (Finite length and Jordan–Holder semisimplification)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-rfi-weight-jh-finite`

Assume the exact lisse coefficient-category portion of A2. Every rank-r lisse Weil coefficient has Jordan–Holder data of length at most r, and the isomorphism class of its semisimplification and constituent multiset are independent of the chosen chain.

Proof. Uses:

On a connected stratum, a strict inclusion of lisse subobjects strictly increases generic rank: equal rank would give a rank-zero lisse quotient and hence an equality. Thus every ascending or descending chain has at most r strict steps. Choose a maximal proper subobject and induct on rank to obtain a finite chain with simple quotients. The usual butterfly refinement of two finite chains gives isomorphic refinements; simplicity forces their nonzero factors to agree up to permutation. Hence $n \le r$ and the direct sum of the simple factors is well defined up to isomorphism. All subobjects and quotient maps live in the Weil category, so they commute with Frobenius. □

Lemma 13.7.218 (Existence of a simple single-weight certificate)**.** *Uses:* *def:weil-wpdf-rfi-pure-lisse-filtration*

Lean: `conditional-interface:lemma:weil-wpdf-rfi-weight-simple-certificate-existence`

Assume the exact lisse coefficient-category portion of A2 and the supplied mixed filtration data of Definition 13.7.198. Every nonzero simple Jordan–Holder constituent of a declared mixed lisse coefficient has a single-weight certificate.

Proof. Uses: `def:weil-wpdf-rfi-pure-lisse-filtration`

Induct on the finite length m of the supplied mixed filtration and, inside the terminal pure grade, on rank. When $m = 1$, the coefficient itself is pure, and every stable subquotient has characteristic polynomial obtained by factoring the characteristic-polynomial identity for an exact Frobenius sequence; hence every simple factor has the same weight. For $m > 1$, use the last exact sequence

$$0 \longrightarrow W_{m-1} L \longrightarrow L \longrightarrow G_m \longrightarrow 0.$$

The finite butterfly refinement of Jordan–Holder chains identifies the constituent multiset of L with the union of those of $W_{m-1} L$ and G_m. The induction hypothesis handles the first term, while the preceding pure grade argument handles the second. The resulting polynomial equalities are the required Frobenius-stalk certificates. □

Lemma 13.7.219 (Well-defined and unique simple weight)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-rfi-weight-simple-certificate-well-defined`

Any two single-weight certificates for the same nonzero simple lisse Weil coefficient have the same weight. Consequently the weight assigned to a Jordan–Holder constituent is independent of the chosen mixed witness and Jordan–Holder chain.

Proof. Uses:

Choose a closed point where the nonzero lisse stalk has an eigenvalue α. Two certificates of weights v and v' give $q_x^{v/2} = |\iota(\alpha)| = q_x^{v'/2}$. Since $q_x > 1$, injectivity of the real exponential gives $v = v'$. Jordan–Holder identifies the same simple constituent up to isomorphism, and an isomorphism preserves every Frobenius polynomial, so the conclusion is independent of both choices. □

Lemma 13.7.220 (Simple-certificate descent)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-rfi-weight-simple-certificate-no-cycle`

The simple-certificate construction is finite and creates no recursive child. Its filtration induction decreases the supplied filtration length, and every Jordan–Holder refinement has length at most the coefficient rank.

Proof. Uses:

For a filtration of length $m > 1$, the construction calls itself only on the proper term $W_{m-1}L$ of length $m-1$; the terminal pure grade is discharged by the finite rank induction. At each rank induction step a proper lisse subobject has strictly smaller generic rank. Thus the lexicographic pair (m, r) decreases whenever an induction call is made. The finite atlas and the recorded Frobenius polynomial comparisons are passive output records, so no step re-enters an earlier state. □

Lemma 13.7.221 (Pure weights of simple constituents)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-rfi-weight-simple-constituents`

Every simple factor in every recorded mixed lisse Jordan–Holder datum is punctually pure of its unique recorded weight $v_i \le w$; a pure input of weight v has all its simple factors of weight v.

Proof. Uses:

Apply the simple-certificate existence and well-definedness lemmas to the declared mixed witness. Its grade weights are at most w, so the resulting constituent weights inherit that bound. For a pure input, take a geometric stalk. Every stable subquotient has characteristic polynomial dividing that of the pure module, so all of its eigenvalues retain modulus $q_x^{v/2}$ at every closed point x. Thus every nonzero simple factor is punctually pure of weight v. Finiteness follows from the Jordan–Holder lemma. □

Definition 13.7.222 (Descending adjacent split datum)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-rfi-weight-adjacent-split-data`

For a descending adjacent extension, the input is the Frobenius-equivariant exact sequence

$$0 \longrightarrow S_i \xrightarrow{a} E \xrightarrow{p} S_{i+1} \longrightarrow 0, \qquad v_i > v_{i+1},$$

where $E = J_{i+1}/J_{i-1}$. The output is a Frobenius-equivariant section $s : S_{i+1} \to E$ and retraction $r : E \to S_i$ satisfying

$$ps = \mathrm{id}_{S_{i+1}}, \qquad ra = \mathrm{id}_{S_i}, \qquad rs = 0, \qquad ar + sp = \mathrm{id}_E .$$

The replacement term is the inverse image in J_{i+1} of $s(S_{i+1}) \subset E$. The datum asserts only this local split exact sequence and its equations; it contains no global semisimplicity, weight-strictness, or locus assertion.

Lemma 13.7.223 (Why an adjacent extension swap is a separate input)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-rfi-weight-swap-necessity`

For $v_i > v_{i+1}$, exchanging the two adjacent Jordan–Holder factors is equivalent to producing a Frobenius-equivariant section of $J_{i+1}/J_{i-1} \twoheadrightarrow S_{i+1}$; the existing weight-separation lemma does not construct such a section.

Proof. Uses:

A replacement intermediate object between J_{i-1} and J_{i+1} with quotient S_{i+1} is precisely the inverse image of a subobject $S_{i+1} \hookrightarrow J_{i+1}/J_{i-1}$ complementary to S_i. This is equivalent to a section of the displayed epimorphism, and a section gives the replacement by inverse image. Weight separation only proves that morphisms between distinct pure weights vanish; it is a Hom statement and gives no vanishing or chosen zero class in $\mathrm{Ext}^1(S_{i+1}, S_i)$. Therefore the local adjacent swap cannot be inferred from separation alone. □

Lemma 13.7.224 (A section determines the split identities). *Uses:* `def:weil-wpdf-rfi-weight-adjacent-split-data`

Lean: `conditional-interface:lemma:weil-wpdf-rfi-weight-adjacent-split-identities`

For a descending adjacent extension, a Frobenius-equivariant section s with $ps = \mathrm{id}$ uniquely determines a Frobenius-equivariant retraction r and all four equations in Definition 13.7.222. Conversely, any descending adjacent split datum contains such a section.

Proof. Uses: `def:weil-wpdf-rfi-weight-adjacent-split-data`

The endomorphism $d = \mathrm{id}_E - sp$ satisfies $pd = 0$. Exactness identifies $\ker p$ with $\operatorname{im} a$, so the kernel universal property gives a unique map $r : E \to S_i$ with $ar = d$. Since a is monic,

$$ara = (\mathrm{id}_E - sp)a = a \quad \Longrightarrow \quad ra = \mathrm{id}_{S_i},$$

and similarly

$$ars = (\mathrm{id}_E - sp)s = 0 \quad \Longrightarrow \quad rs = 0.$$

The defining equality for r is exactly $ar + sp = \mathrm{id}_E$, while $ps = \mathrm{id}$ is the section equation. All maps defining d commute with Frobenius; uniqueness through the Frobenius-stable kernel makes r Frobenius equivariant. The converse is the first displayed equation of the split datum. □

Lemma 13.7.225 (Well-defined and unique adjacent replacement). *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-rfi-weight-adjacent-swap-well-defined`

Let $E = J_{i+1}/J_{i-1}$ and $0 \to S_i \xrightarrow{a} E \xrightarrow{p} S_{i+1} \to 0$ have $v_i > v_{i+1}$. If a Frobenius-equivariant section $s : S_{i+1} \to E$ exists, then its image has a unique inverse-image replacement J_i' between J_{i-1} and J_{i+1}, with successive factors S_{i+1}, S_i. The section, retraction, and replacement are unique whenever they exist.

Proof. Uses:

Apply the split-identities lemma to obtain the unique retraction and the decomposition equations. The image $K = s(S_{i+1})$ is Frobenius stable and $p|_K$ is an isomorphism. The inverse image of K in J_{i+1} is therefore a Frobenius-stable subobject J_i' with $J_i'/J_{i-1} \simeq S_{i+1}$ and $J_{i+1}/J_i' \simeq S_i$; the inverse equations are the section and quotient equations. If s' is another section, then $s - s'$ factors through a as a map $S_{i+1} \to S_i$, which is zero by weight separation. Thus $s = s'$ and K, J_i' are unique. The retraction is already unique from $ar = \mathrm{id}_E - sp$ and monicity of a. □

Lemma 13.7.226 (Non-cyclic adjacent-swap descent). *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-rfi-weight-adjacent-swap-no-cycle`

Whenever the adjacent-swap section exists, the replacement is non-cyclic: it lowers the inversion count by exactly one and preserves every other factor and its weight.

Proof. Uses:

The replacement lemma exchanges only the consecutive weights $v_i > v_{i+1}$, leaving all other factors fixed. Their mutual inversion is removed, and comparisons with factors outside the two positions are unchanged because the two entries are merely exchanged as an adjacent descending pair. Hence $I(J_\bullet)$ decreases by one. Since I is a nonnegative integer, no finite sequence of such replacements can return to a previous chain. □

Lemma 13.7.227 (Adjacent-swap termination). *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-rfi-weight-extension-ordering`

Assume the exact adjacent-swap section-existence leaf below. Repeated swaps terminate after exactly the initial inversion count and produce a Frobenius-equivariant Jordan–Holder chain with nondecreasing factor weights.

Proof. Uses:

If $I = 0$, the chain is already ordered. If $I > 0$, the finite weight sequence has an adjacent inversion. Apply the leaf to its two-step subquotient and replace the intermediate subobject by the supplied complementary inverse image. The endpoints and all other simple factors stay fixed, while the adjacent weights are exchanged. This lowers I by exactly one. Induction on the finite inversion count terminates with a nondecreasing chain, and the recorded splitting equations make every replacement Frobenius equivariant. □

Lemma 13.7.228 (Existence, exhaustion, and bounded indices of weight layers). *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-rfi-weight-layer-exhaustion`

For a nondecreasing weighted Jordan–Holder chain, the grouped objects $W_{u_j}L$ form a finite exhaustive Frobenius-stable filtration, and every nonzero G_{u_j} is lisse and punctually pure of weight $u_j \le w$. Its indices lie in the finite interval $[\min_i v_i, \max_i v_i]$.

Proof. Uses:

The definition of b_j makes the grouped terms increasing, with first term zero before u_1 and final term $J_n = L$ at u_t. The quotient G_{u_j} has a finite filtration whose simple factors are exactly the consecutive S_i of weight u_j. At each closed-point stalk, repeated use of the characteristic-polynomial identity for exact Frobenius sequences shows that all eigenvalues of the quotient have modulus $q_x^{u_j/2}$. Hence the grade is punctually pure of weight u_j. The finite list has at most $n \le r$ entries, every $u_j \le w$, and its minimum and maximum bound all indices. □

Lemma 13.7.229 (Scalar and target-base-change compatibility of existence data)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-rfi-weight-existence-transport`

Assume the exact scalar-extension portion of A2 and the recorded target pullback comparisons of A3. Jordan–Holder, semisimplification, ordered weight-layer, exhaustion, and index-bound data transport compatibly under finite scalar extension and every designated target base change.

Proof. Uses:

Exactness sends each grouped term sequence to a grouped term sequence and gives the term and quotient comparisons with inverse equations. A transported simple factor can split, but it has finite rank; apply the finite-length lemma on each connected component and concatenate its simple factors inside the old weight block. Scalar extension preserves the eigenvalues, and target pullback uses the recorded geometric-stalk Frobenius comparison, so every new simple factor remains pure of the old weight. Jordan–Holder uniqueness identifies the resulting semisimplification with the direct sum of these refined transported factors. Functoriality supplies both equations, iterated-transport cocycles, and Frobenius squares. This makes a compatible transported choice and asserts no choice independence or strictness. □

Lemma 13.7.230 (Strict descent of the existence construction)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-rfi-weight-existence-strict-decrease`

Every adjacent swap or grouped-layer processing step strictly decreases ρ_{ex}, so the finite existence construction terminates.

Proof. Uses:

An adjacent swap lowers the first coordinate I by one and leaves the finite weight set unchanged. Once $I = 0$, removing the final unprocessed grouped weight lowers t by one. Jordan–Holder construction is already finite by $n \le r$, and designated transports are fixed records rather than recursive children. Thus every recursive step is lexicographically smaller in $\mathbf{N}^2$, proving termination. □

Proposition 13.7.231 (Realization of pure-lisse weight existence)**.** *Uses:*

Lean: `conditional-interface:prop:weil-wpdf-rfi-weight-existence-realization`

Assume the exact adjacent-swap existence leaf below, the exact lisse coefficient-category and scalar-extension portions of A2, and the recorded target-pullback comparisons of A3. Every declared pure or mixed lisse coefficient, and every punctual coefficient, has a finite exhaustive Frobenius-stable weight filtration with pure bounded grades, fixed Jordan–Holder and semisimplification data, compatible scalar/base-change choices, and strictly decreasing existence complexity.

Proof. Uses:

Choose a finite Jordan–Holder chain. The simple-constituent lemma labels its factors. Use the adjacent-swap lemma finitely many times to order the weights, then group equal adjacent weights. The layer-exhaustion lemma proves existence, purity, exhaustion, finiteness, and the index bounds. The transport lemma supplies all designated scalar and target-base-change comparisons, and the strict-decrease lemma terminates the construction. A pure coefficient may instead use its canonical one-step filtration. On a punctual coefficient the same proof is finite-dimensional Frobenius linear algebra; distinct-weight primary summands split by coprime polynomials. This realizes the former existence aggregate without invoking weight strictness, either locus leaf, or any target/source affine leaf. □

Lemma 13.7.232 (Pure and mixed weight-filtration construction)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-rfi-weight-pure-and-mixed`

Assume the previously constructed weight-existence package. Every pure lisse coefficient has its canonical one-step filtration, and every declared mixed lisse or punctual coefficient has finite Frobenius-stable filtration data with pure lisse grades of recorded weights at most its input bound.

Proof. Uses: `prop:weil-wpdf-rfi-weight-existence-realization`

For a pure lisse coefficient L of weight v, use $0 = W_0L \subset W_1L = L$; its only grade is L, so all assertions are immediate. For a mixed lisse coefficient, apply the existence realization: its grouped Jordan–Holder construction is finite, exhaustive, Frobenius stable, and has pure lisse grades with bounded recorded weights. On a punctual support apply the finite-dimensional part point by point. Thus every requested filtration and grade has the stated properties. □

Lemma 13.7.233 (Exact induced filtrations on subobjects and quotients)**.** *Uses:* `def:weil-wpdf-rfi-weight-exact-data`

Lean: `conditional-interface:lemma:weil-wpdf-rfi-weight-subquotient-exactness`

Assume the exact weight-strictness leaf below. The pullback/image formulas in Definition 13.7.199 construct finite induced filtrations on every Frobenius-stable lisse or punctual subobject and quotient, and the term and grade sequences displayed there are exact with pure lisse nonzero grades of the same recorded weights.

Proof. Uses: `def:weil-wpdf-rfi-weight-exact-data`

Strictness identifies the filtration on a subobject with intersection and the filtration on a quotient with image. Hence the first displayed sequence is exact by the pullback and image universal properties. Apply the kernel–cokernel lemma to two successive term sequences; it gives the exact grade sequence and the canonical successive-quotient comparison, including its inverse equations. The strictness leaf keeps kernels, images, cokernels, and their finite intersections and sums inside the lisse Weil category. At every geometric stalk, each nonzero induced grade is a Frobenius-stable subobject or quotient of a pure grade. The characteristic-polynomial and subquotient lemmas therefore preserve its weight. The punctual assertion is the same finite-dimensional argument, and all maps commute with Frobenius. □

Lemma 13.7.234 (Scalar and geometric transport of weight filtrations)**.** *Uses:* `def:weil-wpdf-rfi-weight-transport-data`

Lean: `conditional-interface:lemma:weil-wpdf-rfi-weight-transport`

Assume the scalar-extension exactness of A2, the recorded target-pullback comparisons of A3, and the D2 normalization and radicial coefficient comparisons. Every filtration, term exact sequence, and grade transports along each functor of Definition 13.7.202, with inverse equations, iterated cocycles, unchanged weights, and Frobenius squares.

Proof. Uses: `def:weil-wpdf-rfi-weight-transport-data`

An exact coefficient functor sends the short exact sequence defining G_a to a short exact sequence. Its cokernel universal property gives $T(G_a) \simeq T(W_aL)/T(W_{a-1}L)$ and the comparison with the transported grade; functoriality gives the inverse equations and compatibility with every term diagram. Finite scalar extension changes neither the Frobenius eigenvalues nor their complex moduli. Restriction, target pullback, and normalization pullback preserve lissity and identify the new geometric stalk Frobenius with the recorded pullback action. The D2 radicial equivalence identifies geometric stalks and Frobenius in both directions. Thus every transported grade is pure of the same weight. The A2, A3, and D2 comparison equations and their Frobenius squares restrict functorially to all terms and grades; composing them gives the iterated cocycles. Only the already recorded strata and normalization/radicial maps are used, so no locus leaf enters. □

Lemma 13.7.235 (Common refinement and filtration-choice independence)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-rfi-weight-choice-independence`

Assume the exact weight-strictness leaf. Any two finite pure-grade filtrations of the same mixed lisse or punctual coefficient have a finite Frobenius-stable common refinement. Their underlying coefficient, transported term/grade diagrams, and intrinsic upper-weight predicate are independent of the filtration and of the chosen common refinement.

Proof. Uses:

Write the two filtrations as $W_\bullet L$ and $W'_\bullet L$. For $1 \le a \le m$ and $0 \le b \le n$, set

$$R_{a,b} = W_{a-1}L + \big(W_aL \cap W'_bL\big).$$

For fixed a these form an increasing chain from $W_{a-1}L$ to W_aL; concatenate the finitely many chains. The strictness leaf makes every intersection, sum, and quotient lisse and Frobenius stable. Each nonzero new grade is a subquotient of both $\mathrm{Gr}_a^W L$ and some $\mathrm{Gr}_b^{W'} L$, hence is pure of both recorded weights. Stalkwise weight separation forces those weights to agree. Thus the grid is a common pure-grade refinement. Both forgetful composites are the identity of L, whose identity inverse gives both comparison equations. Repeating the construction with three filtrations gives the same finite triple grid, so the refinement comparisons satisfy the cocycle. All operations commute with Frobenius, and the set of nonzero grade weights, hence the upper-weight predicate, is unchanged. □

Lemma 13.7.236 (Strict weight-filtration descent)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-rfi-weight-strict-decrease`

Removing the final grade of a nontrivial weight filtration strictly decreases ρ_{wt} on the remaining recursive child, while the removed pure grade is terminal; hence the finite global multiset decreases strictly.

Proof. Uses:

If the filtration has $m > 0$ remaining grades, its final exact sequence is $0 \to W_{m-1}L \to L \to G_m \to 0$. The first child has $m - 1$ remaining grades and the pure grade G_m has rank zero. Both are smaller than m. The term/grade, transport, and common-refinement diagrams are fixed finite records and create no recursive child. Replacing one multiset entry by these finitely many smaller entries is a strict decrease in the well-founded multiset extension of $\mathbf{N}$, so the filtration recursion terminates. □

Proposition 13.7.237 (Realization of pure-lisse weight filtrations)**.** *Uses:*

Lean: `conditional-interface:prop:weil-wpdf-rfi-weight-filtration-realization`

Assume the previously constructed weight-existence package, the exact weight-strictness leaf below, the scalar-extension portion of A2, the target-pullback portion of A3, and the D2 normalization and radicial coefficient comparisons. Every lisse or punctual coefficient in every root and intermediate D5 request has finite Frobenius-stable pure-grade filtration data, exact induced subobject/quotient filtrations, all designated transports, choice independence, and strictly decreasing weight complexity.

Proof. Uses: `prop:weil-wpdf-rfi-weight-existence-realization,lemma:weil-wpdf-rfi-weight-pure-and-mixed,lemma:weil-wpdf-rfi-weight-subquotient-exactness,lemma:weil-wpdf-rfi-weight-transport`

Apply the pure/mixed construction lemma to the finite coefficient atlas and the exactness lemma to every filtration term, subobject, and quotient used by coefficient devissage. The transport lemma supplies scalar extension, normalization/radicial pullback, target pullback, inverse equations, cocycles, and Frobenius squares on every term and grade. Apply the common-refinement lemma to every pair of recorded choices and the strict-decrease lemma to the finite work multiset. These constructions realize every field of the former weight-filtration aggregate. They construct no D1/D3 triangle or fixed-output comparison and neither invoke nor close a maximal-locus, re-normalization, target-affine, or source-affine leaf. □

Lemma 13.7.238 (Normalization and finite-radicial transport)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-rfi-pure-lisse-transport`

Assume the existing four D2 leaves. The normalization and transport data separate every curve request into its normalized dense lisse coefficient and a punctual defect, and finite or radicial transport preserves the fixed upper-weight assertion.

Proof. Uses:

The D2 normalization and defect leaves give the finite map, the displayed triangle, and finite support of Q_C. The finite-comparison leaf identifies the middle fixed output with the output after normalization; D1 owns the punctual open–closed contribution. For a finite radicial universal homeomorphism, the D2 radicial comparisons identify Weil categories, geometric stalks, and Frobenius actions, so lissity, purity, and every upper weight predicate transport in both directions. The same inverse equations and Frobenius squares give the declared finite transport. On a target pullback use the chosen normalization and common refined open on that curve; only the recorded D2 comparisons are used, not commutation of normalization with arbitrary base change. □

Lemma 13.7.239 (Mixed coefficients give pure-lisse D3 grades)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-rfi-pure-lisse-d3`

Assume the previously constructed weight-filtration package and the existing D3 data. On every positive-dimensional lisse stratum, the restricted coefficient has a finite D3 filtration with pure lisse grades; every punctual remainder has the analogous finite pure filtration on its zero-dimensional support.

Proof. Uses: `prop:weil-wpdf-rfi-weight-filtration-realization`

Use the realized Frobenius-stable filtration and its exact term/grade sequences. Each successive quotient is pure of its stated weight and lisse. The D3 triangle, long-exact, filtration, and termination leaves provide the coefficient triangles and fixed-output maps required for induction. On a finite punctual support the same construction is a finite filtration of Frobenius modules; D1 or D6, not a relative-curve TAME leaf, handles those grades. □

Lemma 13.7.240 (Pure-lisse Frobenius and base-change compatibility)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-rfi-pure-lisse-base-change`

Assume locus refinement and weight filtrations as realized above, together with the recorded A2, A3, and D2 comparisons. The stratification, normalization/radicial transports, and pure/mixed filtrations commute with geometric Frobenius, finite coefficient extension, and every designated target base change through those comparisons.

Proof. Uses:

All strata and immersions are defined over k, so their restriction maps commute with geometric Frobenius. The weight-transport lemma supplies the filtration maps, inverses, equations, and cocycles on every term and grade. The A2 scalar-extension and A3 target-base-change comparisons identify the chosen coefficients; the D2 comparisons do the same after normalization or finite-radicial transport. Composing the recorded commutative squares gives the required comparison on every root and intermediate D5 request. This asserts compatibility of the chosen data and never asserts literal equality of two choices. □

Lemma 13.7.241 (Strict pure-lisse complexity decrease)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-rfi-pure-lisse-strict-decrease`

Every recursive child in the pure-lisse refinement schedule has smaller ρ_{PL} than its parent, so the finite global multiset decreases strictly at every step.

Proof. Uses:

At phase 4, the dense open keeps the support dimension but enters a smaller phase, while every closed-complement child has smaller d. At phase 3, the normalized child enters phase 2 and the defect is punctual, hence has smaller d; any repeated normalization comparison with the same phase uses the strictly smaller D2 defect. Phase 2 transport enters phase 1. At phase 1, the strict weight-filtration descent lemma changes the remainder length from m to $m-1$, while the final pure grade enters phase 0. Thus each replacement consists of finitely many lexicographically smaller ranks. The multiset extension of a well-founded order is well founded, proving strict global descent and finite termination. □

Proposition 13.7.242 (Realization of pure-lisse refinement)**.** *Uses:*

Lean: `conditional-interface:prop:weil-wpdf-rfi-pure-lisse-realization`

Assume the previously constructed locus-refinement and weight-filtration packages, the existing D1–D3 leaves, the four D2 leaves, and the scalar- and target-base-change portions of A2 and A3. Then every root and intermediate D5 coefficient request in the fixed atlas has a finite smooth affine source refinement with pure lisse D3 grades, punctual curve complements, normalization/finite-radicial transport, Frobenius/base-change compatibility, and strictly decreasing complexity.

Proof. Uses: `prop:weil-wpdf-rfi-locus-realization,prop:weil-wpdf-rfi-weight-filtration-realization`

Apply Proposition 13.7.211 to the finite request list and all designated pullbacks. On curve closures apply the normalization/transport lemma; send the punctual complement to its D1 or D6 branch. Apply the weight-filtration realization and the D3 lemma on every lisse or punctual stratum. The compatibility lemma supplies restriction, coefficient-extension, Frobenius, and target-base-change squares, and the strict-decrease lemma terminates the finite work multiset. This constructs all pure-lisse fields of the local factorization atlas. It neither invokes nor closes any of the three target or three source affine leaves, and it leaves controlled relative-curve shrinking to TAME. □

Open obligation: `OBL-WPDF-RFI-PURE-LISSE-MAXIMAL-LOCUS`

OPEN WPDF-RFI-PURE-LISSE-MAXIMAL-LOCUS.

For every irreducible support in every root/intermediate D5 coefficient request, construct the largest open lissity locus and a dense affine smooth subopen over k. On a normalized curve, prove density of the intersection with the smooth and normalization-isomorphism loci and construct its finite affine cover. This leaf contains no target-base-change comparison, weight filtration, D2 output comparison, or D3 exact sequence.

Open obligation: `OBL-WPDF-RFI-PURE-LISSE-RENORMALIZATION`

OPEN WPDF-RFI-PURE-LISSE-RENORMALIZATION.

After every designated target base change, normalize each reduced one-dimensional component, construct the normalization lift for every dominating component and the dense common open with the old maximal locus, and construct the coefficient restriction comparison, inverse, both equations, iterated-base-change cocycle, and Frobenius squares. Treat every non-dominating component as a separately indexed request; do not assert that normalization commutes globally with base change. This leaf contains no weight filtration or fixed-output comparison.

Closed assembly label.

WPDF-RFI-PURE-LISSE-LOCUS.

Proposition 13.7.211 constructs the former locus package from the two exact geometric leaves and the D2 normalization and defect inputs. The closed name is no longer an OPEN dependency and closes neither D2, the three weight leaves, nor any target/source leaf.

Closed assembly label.
WPDF-RFI-PURE-LISSE-WEIGHT-SIMPLE.

Lemmas 13.7.218, 13.7.219, and 13.7.220 construct the single-weight certificate for every simple constituent of a declared mixed lisse coefficient and prove its unique weight and finite non-cyclic construction. The closed name is no longer an OPEN dependency and closes no extension ordering, strictness, transport, D3, locus, or target/source leaf.

Open obligation: `OBL-WPDF-RFI-PURE-LISSE-WEIGHT-ADJACENT-SWAP`

OPEN WPDF-RFI-PURE-LISSE-WEIGHT-ADJACENT-SWAP.

For each descending extension, construct an equivariant section

$$s\colon S_{i+1} \longrightarrow E, \qquad p \circ s = \mathrm{id}_{S_{i+1}}\,.$$

The preceding lemma then gives the unique retraction and split identities. Replacement, strict inversion descent, and termination are proved; only the local Ext^1 zero class remains OPEN. Supplied-PDF Definition 1.3 assumes an already filtered mixed object and Proposition 5.8 uses that witness; neither proves this assertion. Do not replace it by pure perverse semisimplicity, which the supplied PDF explicitly forbids.

Closed assembly label.
WPDF-RFI-PURE-LISSE-WEIGHT-EXISTENCE.

Proposition 13.7.231 constructs the former package from adjacent-swap existence, the closed simple-certificate package, and exact lisse/scalar/target-pullback interfaces. The closed name is no longer an OPEN dependency and closes neither those inputs, weight strictness, the two locus leaves, nor any target/source affine leaf.

Open obligation: `OBL-WPDF-RFI-PURE-LISSE-WEIGHT-STRICTNESS`

OPEN WPDF-RFI-PURE-LISSE-WEIGHT-STRICTNESS.

Prove that Frobenius-equivariant morphisms of mixed lisse coefficients are strict for the supplied weight filtrations and that stable kernels, images, cokernels, finite intersections, and finite sums remain lisse. Construct the induced subobject/quotient term and grade exact sequences and their successive-quotient identifications. This leaf contains no scalar extension, target-base-change, normalization/radicial comparison, D3 triangle, or fixed output.

Closed assembly label.
WPDF-RFI-PURE-LISSE-WEIGHT-FILTRATION.

Proposition 13.7.237 constructs the former weight-filtration package from the weight-existence realization, the exact strictness leaf, and the recorded A2, A3, and D2 coefficient transports. The closed name is no longer an OPEN dependency and closes neither those inputs, the two locus leaves, nor any target/source affine leaf.

Closed assembly label.
WPDF-RFI-PURE-LISSE-REFINEMENT.

Proposition 13.7.242 constructs the former pure-lisse package from the locus realization, the weight-filtration realization, and the already displayed D1–D3, D2, A2, and A3 inputs. The closed name is no longer an OPEN dependency and closes none of those inputs or any target/source affine leaf.

Proposition 13.7.243 (Realization of the fixed local factorization)**.** *Uses:*

Lean: `conditional-interface:prop:weil-wpdf-rfi-local-factorization-realization`

Assume the target-affine, source-affine, and pure-lisse refinement packages constructed above, together with the D1, D3, D4, D2, and TAME construction data they cite. Then fixed local factorization data exist.

Proof. Uses: `prop:weil-wpdf-rfi-target-affine-realization,prop:weil-wpdf-rfi-source-affine-realization,prop:weil-wpdf-rfi-pure-lisse-realization`

Take the finite ordered affine target refinement supplied by Proposition 13.7.182, then the finite ordered affine source refinements supplied by Proposition 13.7.192, and finally the finite pure-lisse D3 filtrations supplied by Proposition 13.7.242. For each resulting affine map, apply the affine-generator lemma and concatenate its coordinate projections. Repeat the same three finite refinements for each projection prefix and each degree in the fixed finite D5 amplitude. There are finitely many prefixes, degrees, strata, coefficient grades, and algebra generators, so their finite nested union is the required atlas. The D1/D3/D4 data supply the restriction and

gluing maps; D2 and TAME supply only their already labelled transports and controlled shrinking data. The base-change comparisons and Frobenius squares are those of the target- and source-affine realization propositions, the pure-lisse realization proposition, the affine-generator lemma, and the cited upstream labels. Thus no unlabelled geometric operation remains. □

Closed assembly label.

WPDF-RFI-LOCAL-FACTORIZATION.

The former assembly package follows from Proposition 13.7.243, using the target- and source-affine realizations, the pure-lisse realization proposition, and the already displayed D1–D4/TAME labels. The closed name is no longer an OPEN dependency and does not close any of its inputs.

Definition 13.7.244 (Fixed-degree reduction tree)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-rfi-reduction-tree`

A reduction tree for the fixed goal is a finitely branching rooted tree built from fixed local factorization data. A vertex v carries one fixed tuple $(f_v, K_v, n_v, w_v, A_v)$ and the budget $B_v = w_v + n_v$. The root carries $(f, K, i, w, A(f, K, i))$. Every internal vertex is tagged by exactly one of the eight devissage rules listed in the preceding assembly discussion; the tag records which fixed construction and its proved termination decrease are applied. Base-change and coefficient-extension transports are recorded on the adjacent edge, with comparison, inverse, both equations, and Frobenius square, but do not count as unlabelled devissage rules or recursive branches.

Every leaf is explicitly of one of two types: a relative-dimension-zero leaf discharged conditionally by the D6 rule, or a pure-lisse controlled relative-curve leaf discharged conditionally by the TAME rule, followed when needed by its direct-summand descent. Thus no leaf is labelled merely "known" or "geometric case."

Definition 13.7.245 (Reduction-tree termination rank)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-rfi-termination-rank`

Each non-leaf vertex has a rank

$$\rho(v) = (N_v, e_v, s_v, r_v, m_v, \delta_v, \lambda_v, q_v) \in \mathbf{N}^8,$$

ordered lexicographically. The entries count, respectively, unresolved relative-one projection factors, the unresolved relative-zero immersion ($e_v \in \{0, 1\}$), target strata, source strata, coefficient-filtration length, normalization defect, D5 abutment-filtration length, and D6 quasi-finite strata. The immersion flag precedes all stratification counters so that the factorization of one projection after one immersion changes $(N, e) = (1, 1)$ to $(1, 0)$ on the projection child. Every recursive edge to a child is required to decrease this rank strictly; transport comparisons do not create children.

Definition 13.7.246 (Deterministic RFI branch schedule)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-rfi-branch-schedule`

At a vertex, apply the first applicable row of the following schedule. The two displayed children in a devissage row include all kernel, quotient, or spectral grades required by its cited label.

row	condition and label	strictly decreasing data
T	$s > 1$, D4	$s - 1$ and 1
S	$s = 1$, $r > 1$, D1	$r - 1$ and 1
C	$s = r = 1$, $m > 1$, D3	$m - 1$ and 1
N	$\delta > 0$, D2 normalization	$\delta \mapsto 0$; defect has $N = 0$
$F_{>1}$	$N > 1$, D5 composition	$N - 1$ and 1
F_1	$(N, e) = (1, 1)$, D5 composition	$(0, 0)$ and $(1, 0)$
Z	$N = 0$	D6 leaf
R	$(N, e) = (1, 0)$	controlled TAME leaf.

In row N, the normalized child has the same earlier coordinates and defect zero; the finite-support defect child is sent directly to its relative-zero D6 factorization. In row $F_{>1}$, split $p_N \cdots p_1 j$ as $p_N \circ (p_{N-1} \cdots p_1 j)$; in row F_1, split $p_1 j$ as $p_1 \circ j$. D5 filtration grades are assembled at that vertex by their finite λ-filtration. If D6 is expanded internally, its strata decrease q one at a time. Finite/radicial, base-change, and coefficient transports are decorations on the corresponding edge and create no new row.

Lemma 13.7.247 (Strict decrease for every scheduled branch)**.** Uses:

Lean: `conditional-interface:lemma:weil-wpdf-rfi-scheduled-rank-decrease`

Every recursive child in the deterministic branch schedule has rank strictly smaller than its parent, including both children of every binary row.

Proof. Uses:

Rows T, S, C preserve all preceding coordinates and decrease respectively s, r, m. The normalized child in row N preserves preceding coordinates and decreases δ, while its finite-support child has $N = 0$. In row $F_{>1}$, both children have fewer than N projection factors. In row F_1, the immersion child has $N = 0$ and the projection child preserves $N = 1$ but changes the next coordinate from $e = 1$ to $e = 0$. Any internal D5 or D6 filtration step preserves preceding coordinates and decreases λ or q. These are exactly the lexicographic comparisons asserted. □

Lemma 13.7.248 (Exhaustive reduced-leaf coverage)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-rfi-leaf-coverage`

Every branch generated by the schedule ends at exactly one displayed leaf type: a D6 relative-dimension-zero leaf or a pure lisse controlled TAME relative-curve leaf, with summand descent when recorded.

Proof. Uses:

The rank is well-founded and decreases on every branch, so a branch reaches a vertex to which no internal row applies. The target/source/coefficient and normalization rows then have completed their finite data. If $N > 1$, row $F_{>1}$ would apply; if $(N, e) = (1, 1)$, row F_1 would apply. The remaining possibilities are $N = 0$, which is row Z and has the D6 label, or $(N, e) = (1, 0)$, which is row R and has the controlled TAME label. The local factorization datum supplies the exact D2/TAME shrinking and summand decorations, so no third or unlabelled leaf is possible. □

Lemma 13.7.249 (Termination of a ranked fixed reduction tree)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-rfi-tree-termination`

No branch of a fixed reduction tree with the displayed rank is infinite.

Proof. Uses:

The lexicographic order on a finite product of copies of $\mathbf{N}$ is well-founded. The rank decreases strictly on every edge, so an infinite branch would give an infinite descending chain in this order. This is impossible. Since the tree is finitely branching, König's lemma then shows that the rooted reduction object has only finitely many vertices. □

Proposition 13.7.250 (Realization of the finite ranked RFI tree)**.** *Uses:*

Lean: `conditional-interface:prop:weil-wpdf-rfi-tree-realization`

Assume fixed local factorization data and every cited D1–D6/TAME construction datum. The deterministic schedule constructs a finite ranked reduction tree, all Frobenius assembly filtrations, and all declared base-change and coefficient-extension edge comparisons. Every leaf has exactly one of the two reduced-case labels.

Proof. Uses:

Starting at the root, create the finitely many children prescribed by the first applicable schedule row. D4, D1, and D3 use their finite ordered lists; D2 has its normalized and finite-support outputs; D5 has the finitely many bounded spectral grades; D6 and TAME provide the two leaf constructors. Thus every vertex is finitely branching. The strict-decrease lemma makes this a well-founded recursion, and the ranked-tree termination lemma makes the resulting tree finite. The leaf-coverage lemma proves exhaustiveness and uniqueness of the leaf tag.

At each internal vertex, use the already displayed D1/D3/D4 long-exact kernel–image–cokernel filtration, D5 abutment filtration, or D2 transport comparison. The local factorization data and the cited upstream leaves give every inclusion, quotient, connecting map, inverse, and inverse equation. Their Frobenius squares make each filtration stable. On a target base change, the affine-generator factorization base changes coordinate by coordinate and the D1–D6/TAME comparisons identify the corresponding outputs; composition of these squares gives the edge square. Coefficient extension is identical stage by stage by the fixed comparison in the root goal. Hence the realized tree has exactly the filtration and compatibility data required by the fixed-degree assembly proposition. □

Definition 13.7.251 (Frobenius assembly filtration)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-rfi-assembly-filtration`

At a geometric target point, a Frobenius assembly filtration at an internal vertex v is a finite Frobenius-stable filtration

$$0 = F_0 A_v \subset F_1 A_v \subset \cdots \subset F_t A_v = A_v$$

such that every graded module is a Frobenius-stable subquotient of a child output with allocated upper bound at most B_v. For D1, D3, and D4 vertices these are the kernel/image/cokernel pieces of the displayed localization or coefficient long exact sequence. For D5 they are the E_∞^{a,n_v-a} grades of the displayed abutment filtration. D2 and base-change or coefficient-extension transport vertices instead carry the displayed Frobenius-equivariant comparison and inverse. All filtration inclusions, quotient maps, comparisons, and connecting maps commute with Frobenius.

Lemma 13.7.252 (Finite coefficient extension preserves a fixed upper bound). *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-rfi-coefficient-extension`

Under the coefficient comparison in the fixed-degree goal, $A(f,K,i)$ has upper weight $w+i$ if and only if its fixed output after a finite coefficient extension E'/E has that upper weight.

Proof. Uses:

Scalar extension does not change the roots of the Frobenius characteristic polynomial after embedding the coefficient fields into $\overline{\mathbf{Q}}_\ell$. The comparison and its inverse are Frobenius-equivariant, so they identify the characteristic polynomials at every geometric stalk. The same modulus inequalities therefore hold in both directions. □

Lemma 13.7.253 (One-node fixed-degree weight assembly). *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-rfi-one-node`

Let v be an internal vertex. If every child output has its allocated upper weight and the Frobenius assembly filtration or transport comparison at v is given, then A_v has upper weight B_v.

Proof. Uses:

At each geometric stalk, subquotient stability transfers the child bounds to every graded module of the assembly filtration. Repeated extension stability then transfers the bound through the finite filtration to A_v. At a D2, base-change, or coefficient-extension transport vertex, the two inverse Frobenius comparisons transfer the characteristic polynomial and the bound directly. The D5 grading has $a+b=n_v$, so its child budget is exactly $w_v+a+b=B_v$. All transfers are Frobenius-equivariant by the vertex data. □

Proposition 13.7.254 (Conditional fixed-degree direct-image assembly). *Uses:*

Lean: `conditional-interface:prop:weil-wpdf-rfi-fixed-degree-assembly`

Assume a ranked reduction tree for the fixed triple (f,K,i), every assembly filtration and comparison in that tree, and every hypothesis of the exact leaf label attached to each leaf. Then the single fixed output $A(f,K,i)$ has upper weight $w+i$.

Proof. Uses:

Use well-founded induction on the termination rank. At a leaf, apply its displayed D6 or controlled relative-curve label, including direct-summand descent when recorded. At an internal vertex, the induction hypotheses give the allocated child bounds; apply the one-node assembly lemma. Target base-change identifies the needed geometric stalk, and finite coefficient extension may be removed using the coefficient-extension lemma. The root budget is $w+i$, which is the asserted fixed-degree bound. □

Closed assembly label WPDF-RFI-TREE-REALIZATION. Proposition 13.7.250 constructs the finite ranked tree, its exhaustive leaf tags, every assembly filtration, and all Frobenius/base-change comparisons. The former aggregate name is no longer an OPEN dependency.

Existing OPEN dependencies. Target and radicial base change remain exactly `WPDF-D4-BASE-CHANGE`, `WPDF-D2-RADICIAL`, `WPDF-TAME-RADICIAL`, `WPDF-TAME-BASE-CHANGE`, and `WPDF-BASE-A3`. Coefficient extension remains part of `WPDF-BASE-A2`. The six affine-refinement leaves, two locus leaves, and three weight leaves remain OPEN, as do all earlier D1–D6 and TAME inputs. Closing the assembly packages closes none of those leaves.

13.7.16 Derived upper bound for one bounded constructible input

Definition 13.7.255 (Fixed bounded input and cohomological amplitude). Uses:

Lean: `conditional-interface:def:weil-wpdf-der-amplitude`

Fix one map $f: X \to Y$, one bounded constructible Weil complex K, and one weight bound w. Choose integers $a_- \le a_+$ such that $\mathcal{H}^a(K)=0$ outside $[a_-,a_+]$ and $\mathcal{H}^a(K)$ has upper weight $w+a$ inside this interval. Choose also a fixed sheaf-degree amplitude $[b_-,b_+]$ for the already constructed outputs, so $T_f^b(M)=0$ outside that interval for every cohomology sheaf M occurring here.

The fixed derived output is one chosen object $D=Rf_!K$ with cohomology sheaves $G^n=\mathcal{H}^n(D)$, required to vanish outside $[a_-+b_-,a_++b_+]$. This is data for one f and one K, not a theorem for all bounded complexes.

Definition 13.7.256 (Frobenius truncation-tower data). Uses:

Lean: `conditional-interface:def:weil-wpdf-der-truncation-tower`

For $a_- \le a \le a_+$, put $K_a=\tau_{\le a}K$, with $K_{a_--1}=0$ and $K_{a_+}=K$. Frobenius truncation-tower data consist of the Frobenius-equivariant triangles

$$K_{a-1} \longrightarrow K_a \longrightarrow \mathcal{H}^a(K)[-a] \longrightarrow K_{a-1}[1]$$

and their fixed images

$$Rf_!K_{a-1} \longrightarrow Rf_!K_a \longrightarrow Rf_!\mathcal{H}^a(K)[-a] \longrightarrow Rf_!K_{a-1}[1].$$

The data include every cohomology-sheaf connecting map, exactness witness, and Frobenius commutative square. They identify the third term in degree n with the fixed sheaf output $T_f^{n-a}(\mathcal{H}^a(K))$.

Definition 13.7.257 (Finite Postnikov termination measure)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-der-postnikov-length`

The Postnikov length of the fixed input is

$$L = a_+ - a_- + 1.$$

At stage a, the number of unassembled nonzero cohomology sheaves is $a_+ - a$. Passing from K_a to K_{a+1} decreases this number by one, and the tower terminates at $K_{a_+} = K$ after exactly L steps.

Definition 13.7.258 (Derived cohomology-sheaf filtration)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-der-output-filtration`

For every output degree n, the truncation tower supplies a finite Frobenius-stable filtration

$$0 = F_{a_- - 1}G^n \subset F_{a_-}G^n \subset \cdots \subset F_{a_+}G^n = G^n.$$

Its grade $Q_a^n = F_aG^n/F_{a-1}G^n$ is a Frobenius-stable subquotient of

$$T_f^{n-a}(\mathcal{H}^a(K)).$$

The inclusions, quotient maps, comparison maps, and the recursive kernel/image/cokernel descriptions obtained from the truncation long exact sequences all commute with Frobenius. A grade is zero whenever $a \notin [a_-, a_+]$ or $n - a \notin [b_-, b_+]$.

Definition 13.7.259 (Derived base-change and coefficient-extension data)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-der-transport-data`

For every target base change $u : Y' \to Y$ used for the fixed input, derived transport data contain maps

$$u^*Rf_!K_a \longleftrightarrow Rf'_!u_X^*K_a$$

for every truncation stage, their inverses and both inverse equations, and compatibility with the truncation triangles, cohomology-sheaf filtrations, geometric-point extension, and Frobenius. For every finite coefficient extension E'/E, they contain the analogous comparisons between $Rf_!K_a \otimes_E E'$ and $Rf_!(K_a \otimes_E E')$, with the same equations and compatibilities.

Lemma 13.7.260 (Fixed-degree index conversion)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-der-index-conversion`

Let $a \in [a_-, a_+]$ and put $b = n - a$. If the fixed-degree upper-bound assembly applies to $(f, \mathcal{H}^a(K), b)$, then $T_f^{n-a}(\mathcal{H}^a(K))$ has upper weight $w + n$.

Proof. Uses: `prop:weil-wpdf-rfi-fixed-degree-assembly`

The input $\mathcal{H}^a(K)$ has upper weight $w + a$. The fixed-degree bound in degree $b = n - a$ is therefore

$$(w + a) + b = (w + a) + (n - a) = w + n.$$

This is only an index substitution in the conclusion of 13.7.254; it does not close any hypothesis of that proposition. □

Lemma 13.7.261 (One-step Frobenius Postnikov assembly)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-der-one-step`

Fix a and n. If $\mathcal{H}^n(Rf_!K_{a-1})$ has upper weight $w + n$ and the fixed-degree bound for $T_f^{n-a}(\mathcal{H}^a(K))$ has been supplied, then $\mathcal{H}^n(Rf_!K_a)$ has upper weight $w + n$.

Proof. Uses:

The cohomology long exact sequence of the a-th truncation triangle gives a short exact sequence whose subobject is the image of $\mathcal{H}^n(Rf_!K_{a-1})$ in $\mathcal{H}^n(Rf_!K_a)$ and whose quotient is the kernel of

$$T_f^{n-a}(\mathcal{H}^a(K)) \longrightarrow \mathcal{H}^{n+1}(Rf_!K_{a-1}).$$

Thus the subobject is a quotient of the preceding stage and the quotient is a subobject of the fixed sheaf-degree output. The induction hypothesis and the index-conversion lemma bound these two terms by $w + n$. Frobenius-equivariant extension stability gives the same bound for the middle term. □

Lemma 13.7.262 (Derived upper bounds transport by target base change)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-der-base-change`

Under the displayed derived base-change comparisons, the bound on the stalk $G^n_{\bar{y}}$ is equivalent to the bound on the compared stalk over every chosen $\bar{y}' \to \bar{y}$. Consequently the global stalkwise bound may be checked after a fixed family of base changes only when their images cover all geometric points of Y.

Proof. Uses:

The comparison and inverse identify the two corresponding geometric stalks and their filtration grades. Every square commutes with Frobenius, so the characteristic polynomials and the subquotient/extension filtration are unchanged. This proves the pointwise equivalence. A covering family reaches every geometric stalk, giving the final assertion. □

Lemma 13.7.263 (Derived upper bounds transport by coefficient extension)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-der-coefficient-extension`

Under the displayed finite coefficient-extension comparisons, G^n has upper weight $w+n$ over E if and only if the extended fixed output has that upper weight over E'.

Proof. Uses:

The comparisons identify every extended truncation triangle and filtration grade. Scalar extension does not change the Frobenius roots in $\overline{\mathbf{Q}}_\ell$, and the inverse comparison gives the converse. Thus the same modulus inequalities hold over both coefficient fields. □

Proposition 13.7.264 (Conditional derived upper bound for the fixed input)**.** *Uses:*

Lean: `conditional-interface:prop:weil-wpdf-der-fixed-derived-upper`

Assume the fixed amplitude, truncation tower, output filtrations, and transport data above, and the conditional fixed-degree upper bound for every $T_f^b(\mathcal{H}^a(K))$ occurring in their finite ranges. Then

$$\mathcal{H}^n(Rf_!K) \text{ has upper weight } w+n \quad \text{for every } n,$$

equivalently the fixed output $Rf_!K$ lies in $D_{\le w}(Y)$.

Proof. Uses:

Induct through the finite Postnikov tower. The initial stage is zero. At stage a, apply the index-conversion lemma to every relevant degree and then the one-step Postnikov lemma. The termination measure decreases until $K_{a_+} = K$. Equivalently, the resulting filtration on each G^n has grades that are subquotients of fixed-degree outputs of upper weight $w+n$; repeated extension stability gives the same bound for G^n. The amplitude makes both the tower and every output filtration finite. Base change and coefficient extension may be inserted or removed by the two transport lemmas, with all Frobenius structures preserved. □

Open obligation: `OBL-WPDF-DERIVED`

OPEN WPDF-DERIVED. Construct the fixed truncation triangles and their $Rf_!$ images, cohomology-sheaf connecting maps and exactness, cohomological amplitude, finite Frobenius Postnikov filtration, and all base-change and coefficient-extension comparisons, inverses, equations, and Frobenius squares. This obligation remains OPEN after the conditional assembly above.

13.7.17 Fixed Poincare–Verdier upper-to-lower argument

Definition 13.7.265 (Fixed duality input and upper bounds)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-pd-fixed-input`

Fix one smooth proper k-scheme X of pure dimension d, one lisse Weil sheaf L pure of weight w, its chosen dual $L^\vee$ pure of weight $-w$, and one degree i. Put

$$V_i = H^i(X_{\bar{k}}, L), \qquad W_{2d-i} = H^{2d-i}(X_{\bar{k}}, L^\vee).$$

The input includes Frobenius-equivariant proper-support comparisons $H^r_c \leftrightarrow H^r$ with inverses and both inverse equations. The previous upper route is assumed only in the two fixed degrees, giving upper weight $w+i$ for V_i and upper weight $-w+2d-i$ for W_{2d-i}.

Definition 13.7.266 (Verdier dual and dimension-shift normalization)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-pd-verdier-normalization`

Let $a : X \to \operatorname{Spec} k$. The fixed normalization data are Frobenius-equivariant comparisons

$$a^! \overline{\mathbf{Q}}_\ell \longleftrightarrow \overline{\mathbf{Q}}_\ell(d)[2d], \qquad \mathbb{D}_X(L) \longleftrightarrow L^\vee(d)[2d],$$

their inverses, both inverse equations, the evaluation map $L \otimes L^\vee \to \overline{\mathbf{Q}}_\ell$, and the trace map in degree $2d$. The shift $[2d]$ pairs degrees i and $2d - i$; rewriting the trace without twisting the second coefficient places the pairing target in $\overline{\mathbf{Q}}_\ell(-d)$.

Definition 13.7.267 (Frobenius-equivariant perfect pairing)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-pd-perfect-pairing`

Perfect pairing data consist of

$$\langle -, - \rangle : V_i \otimes W_{2d-i} \longrightarrow \overline{\mathbf{Q}}_\ell(-d)$$

and the adjoint comparison

$$\theta_i : V_i \longleftrightarrow W_{2d-i}^\vee(-d),$$

with inverse maps, both inverse equations, and Frobenius commutative squares. For geometric Frobenius the target has multiplier q^d, so the square says

$$\langle \Phi_V v, \Phi_W x \rangle = q^d \langle v, x \rangle.$$

Definition 13.7.268 (Fixed lower-bound and purity output)**.** Uses:

Lean: `conditional-interface:def:weil-wpdf-pd-purity-output`

The lower-bound output is that every eigenvalue of V_i has weight at least $w + i$. The purity output is that V_i is pure of weight $w + i$, obtained only after combining this lower bound with the already assumed upper bound $w + i$. No zeta function, trace-formula product, or RH statement is part of this output.

Lemma 13.7.269 (Tate target and dimension multiplier)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-pd-tate-multiplier`

The one-dimensional target $\overline{\mathbf{Q}}_\ell(-d)$ is pure of weight $2d$ and geometric Frobenius acts on it by q^d.

Proof. Uses:

By the global convention, a twist (r) multiplies geometric Frobenius by q^{-r} and changes weight by $-2r$. Taking $r = -d$ gives multiplier q^d and weight $2d$. □

Lemma 13.7.270 (Reciprocal eigenvalue relation from perfect duality)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-pd-reciprocal-spectrum`

Under the perfect pairing data, the spectrum of V_i is the multiset

$$\{q^d \beta^{-1} : \beta \in \operatorname{Spec}(W_{2d-i})\}.$$

Equivalently, eigenvalues paired with multiplicity satisfy $\alpha\beta = q^d$.

Proof. Uses:

The adjoint comparison is a Frobenius-equivariant isomorphism $V_i \simeq W_{2d-i}^\vee(-d)$. Frobenius on the dual has spectrum $\{\beta^{-1}\}$, and the twist $(-d)$ multiplies it by q^d. The comparison and inverse preserve characteristic polynomials, proving the multiset identity and its product form. □

Lemma 13.7.271 (Dual upper bound gives the fixed lower bound)**.** *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-pd-upper-to-lower`

If W_{2d-i} has upper weight $-w + 2d - i$, then V_i has lower weight $w + i$.

Proof. Uses:

For an eigenvalue α of V_i, choose the corresponding β from the reciprocal-spectrum lemma. The dual upper bound gives

$$|\iota(\beta)| \le q^{(-w+2d-i)/2}.$$

Since $\alpha = q^d \beta^{-1}$,

$$|\iota(\alpha)| \ge q^d q^{-(-w+2d-i)/2} = q^{(w+i)/2}.$$

This is precisely the lower-weight inequality for every eigenvalue of V_i. □

Proposition 13.7.272 (Conditional fixed Poincare purity). *Uses:*

Lean: `conditional-interface:prop:weil-wpdf-pd-fixed-purity`

Assume the fixed duality input, Verdier normalization, perfect pairing data, and the two fixed upper bounds. Then V_i is pure of weight $w+i$.

Proof. Uses:

The upper route bounds every eigenvalue of V_i by $w+i$. Apply the upper-to-lower lemma to the fixed dual degree to obtain the opposite inequality $w+i$. Hence every eigenvalue has weight exactly $w+i$, which is the stated purity output. No trace-formula or zeta-function step is used. □

Open obligation: `OBL-WPDF-PD`

OPEN WPDF-PD. Construct the displayed fixed Verdier comparisons, proper-support comparison, evaluation and trace maps, perfect pairing, adjoint maps and inverses, both inverse equations, the $[2d]$ shift, the $(-d)$ twist, and every geometric-Frobenius normalization square.

Existing OPEN dependencies. General duality construction, biduality, trace, projection formula, global finiteness, and Poincare perfectness remain exactly `WPDF-BASE-A4`. The two upper bounds still depend on `WPDF-DERIVED`, the fixed-degree RFI tree, and all of their earlier OPEN leaves. None is closed by the reciprocal-weight argument.

13.7.18 Strict conditional WPDF atoms

Lemma 13.7.273 (Inverse-character convolution from supplied comparisons). *Uses:*

Lean: `blueprint-target:lemma:weil-wpdf-cond-ft-inverse-character-transport`

Fix a Weil complex K. Suppose that the Fubini, base-change, and projection-formula arrows have been supplied as a natural Weil isomorphism to convolution by the two-variable kernel

$$H^- := R\pi_{xz!}\mathcal{L}_\psi\big(y(x-z)\big)[2],$$

where π_{xz} forgets y, p_x, p_z are the two projections from the (x,z)-plane, and the displayed shift contains both Fourier normalization shifts. Suppose also that a Weil kernel isomorphism $d^- : H^- \xrightarrow{\sim} \Delta_! E(-1)$ and the diagonal evaluation isomorphism have been supplied. Then the typed chain

$$\mathrm{FT}_{\psi^{-1}}\,\mathrm{FT}_\psi\,K \xrightarrow{\sim} Rp_{z!}(p_x^*K \otimes H^-) \xrightarrow[\sim]{Rp_{z!}(p_x^*K\otimes d^-)} Rp_{z!}(p_x^*K \otimes \Delta_! E(-1)) \xrightarrow{\sim} K(-1)$$

is a natural Weil isomorphism η_K^-.

Proof. Uses:

Write the first and last supplied isomorphisms as u_K^- and e_K^-. Apply the functor $Rp_{z!}(p_x^*K \otimes -)$ to d^-, and denote the resulting isomorphism by D_K^-. Set $\eta_K^- = e_K^- \circ D_K^- \circ u_K^-$. Its inverse is $(u_K^-)^{-1} \circ (D_K^-)^{-1} \circ (e_K^-)^{-1}$. Associativity of composition and the inverse equations for the supplied isomorphisms reduce the two composites to identity arrows. Naturality and the Frobenius square are preserved under composition. □

Lemma 13.7.274 (Same-character convolution from supplied comparisons). *Uses:*

Lean: `blueprint-target:lemma:weil-wpdf-cond-ft-same-character-transport`

Fix a Weil complex K and put

$$H^+ := R\pi_{xz!}\mathcal{L}_\psi\big(y(x+z)\big)[2].$$

Here π_{xz}, p_x, p_z are as above and $[2]$ contains both Fourier normalization shifts. Suppose that the Fubini and projection-formula comparison to convolution by H^+ is supplied, together with a Weil kernel isomorphism $d^+ : H^+ \xrightarrow{\sim} (\Gamma_{-1})_! E(-1)$ for the graph $\Gamma_{-1} : x \mapsto (x,-x)$ and the graph-evaluation isomorphism. Then the typed chain

$$\mathrm{FT}_\psi\,\mathrm{FT}_\psi\,K \xrightarrow{\sim} Rp_{z!}(p_x^*K \otimes H^+) \xrightarrow[\sim]{Rp_{z!}(p_x^*K\otimes d^+)} Rp_{z!}(p_x^*K \otimes (\Gamma_{-1})_! E(-1)) \xrightarrow{\sim} [-1]^*K(-1)$$

is a natural Weil isomorphism η_K^+.

Proof. Uses:

Apply $Rp_{z!}(p_x^*K \otimes -)$ to d^+ and compose the resulting isomorphism between the supplied convolution and graph-evaluation arrows. Reverse the three inverses to obtain the inverse composite. The inverse equations, naturality, and Frobenius equivariance follow component by component. The support is the graph $z=-x$, so this composite cannot be identified with the diagonal composite of the inverse-character lemma. □

Lemma 13.7.275 (Stationary quotient as a first-isomorphism object)**.** *Uses:*

Lean: `blueprint-target: lemma:weil-wpdf-cond-stationary-quotient-equivalence`

Let V and Q be finite-dimensional Weil modules, let $V^I \subset V$ be Frobenius stable, and let $c : V \twoheadrightarrow Q$ be Frobenius equivariant with $\ker(c) = V^I$. Then

$$\bar{c} : V/V^I \xrightarrow{\sim} Q, \qquad [v] \longmapsto c(v),$$

is a Frobenius-equivariant linear equivalence with a uniquely determined inverse.

Proof. Uses:

The kernel equation makes the displayed formula independent of the representative. Surjectivity of c makes $\bar{c}$ surjective, and the same kernel equation makes it injective. Define the inverse as the inverse of this bijective linear map. Equivalently, a lift of $q \in Q$ determines a class in V/V^I, and two lifts differ by an element of V^I. These two descriptions give the identity composites. Frobenius stability of V^I and equivariance of c give the Frobenius square for $\bar{c}$; uniqueness of the inverse gives the square for its inverse. □

Lemma 13.7.276 (Nonzeroness transported across a supplied stationary isomorphism)**.** *Uses:*

Lean: `blueprint-target: lemma:weil-wpdf-cond-stationary-nonzero-transfer`

Let $b : Q \xrightarrow{\sim} T(-1)$ be a linear equivalence. For $0 \neq t \in T(-1)$, the element $b^{-1}(t)$ is a nonzero witness in Q.

Proof. Uses:

If $b^{-1}(t) = 0$, applying b and using $b\,b^{-1} = \mathrm{id}$ gives $t = 0$, contrary to the hypothesis. □

Lemma 13.7.277 (Weight transported across a supplied stationary isomorphism)**.** *Uses:*

Lean: `blueprint-target: lemma:weil-wpdf-cond-stationary-weight-transfer`

Let $b : Q \xrightarrow{\sim} T(-1)$ be Frobenius equivariant. If T is pure of weight s, then Q is pure of weight $s + 2$. In particular, $s = t - 1$ gives weight $t + 1$ on Q.

Proof. Uses:

The twist (-1) multiplies geometric Frobenius eigenvalues by q and raises their weights by two. Thus $T(-1)$ is pure of weight $s + 2$. The Frobenius-equivariant equivalence b preserves the eigenvalue multiset, so the same purity statement holds for Q. □

Lemma 13.7.278 (Maximality of a supplied low-weight truncation)**.** *Uses:*

Lean: `blueprint-target: lemma:weil-wpdf-cond-weight-truncation-maximal`

Let $\mathcal{A}$ be a finite-length abelian category and assign a real weight to every isomorphism class of simple objects. Let $W_{\le a}L \subset L$ be a supplied subobject whose composition factors have weight at most a, and suppose every composition factor of $L/W_{\le a}L$ has weight greater than a. Then $W_{\le a}L$ is the unique maximal low-weight subobject of L.

Proof. Uses:

Let $U \subset L$ have composition factors of weight at most a. The image of the composite $U \to L \to L/W_{\le a}L$ is a quotient of U and a subobject of $L/W_{\le a}L$. Its composition factors would therefore have weight both at most and greater than a, so the image is zero. Hence $U \subset W_{\le a}L$. This proves maximality, and two subobjects with the same specification contain one another. □

Lemma 13.7.279 (Functoriality of intrinsic weight truncation)**.** *Uses:*

Lean: `blueprint-target: lemma:weil-wpdf-cond-weight-truncation-functorial`

Under the finite-length and simple-weight hypotheses, let $W_{\le a}L \subset L$ and $W_{\le a}M \subset M$ be supplied subobjects whose factors have weight at most a and whose quotients have factors of weight greater than a. Every morphism $f : L \to M$ satisfies

$$f(W_{\le a}L) \subseteq W_{\le a}M.$$

Proof. Uses: `lemma:weil-wpdf-cond-weight-truncation-maximal`

The image of $W_{\le a}L$ is a quotient of an object whose composition factors have weight at most a. Its composition factors therefore have the same upper bound. Maximality of $W_{\le a}M$ gives the inclusion. □

Lemma 13.7.280 (Strictness of intrinsic weight truncation)**.** *Uses:*

Lean: `blueprint-target: lemma:weil-wpdf-cond-weight-truncation-strict`

Under the finite-length and simple-weight hypotheses, take supplied subobjects $W_{\le a}L$ and $W_{\le a}M$ with the low-subobject and high-quotient specifications. Every morphism $f : L \to M$ satisfies

$$\mathrm{im}(f) \cap W_{\le a}M = f(W_{\le a}L).$$

Proof. Uses: `lemma:weil-wpdf-cond-weight-truncation-maximal`,`lemma:weil-wpdf-cond-weight-truncation-functorial`

Functoriality gives the inclusion from right to left. Put $I = \mathrm{im}(f)$ and $J = f(W_{\le a}L)$. The quotient I/J is a quotient of $L/W_{\le a}L$, so all its composition factors have weight greater than a. On the other hand,

$$(I \cap W_{\le a}M)/J$$

is a subquotient of $W_{\le a}M$, so all its composition factors have weight at most a. As it is also a subobject of I/J, finite length forces it to be zero. The displayed equality follows. □

Lemma 13.7.281 (Relative-curve amplitude from a supplied full exact sequence)**.** *Uses:*

Lean: `blueprint-target:lemma:weil-wpdf-cond-tame-amplitude`

Let A^r, P^r, E^r be Weil modules in a long exact sequence

$$\cdots \longrightarrow P^{r-1} \longrightarrow E^{r-1} \longrightarrow A^r \longrightarrow P^r \longrightarrow E^r \longrightarrow \cdots .$$

If $E^r = 0$ for $r \neq 0$ and $P^r = 0$ for $r \notin \{0,1,2\}$, then $A^r = 0$ for $r \notin \{0,1,2\}$.

Proof. Uses:

For $r < 0$ or $r > 2$, both E^{r-1} and P^r vanish. Exactness of $E^{r-1} \to A^r \to P^r$ then says that the zero map into A^r is surjective, so $A^r = 0$. □

Lemma 13.7.282 (Low-degree row after the amplitude argument)**.** *Uses:*

Lean: `blueprint-target:lemma:weil-wpdf-cond-tame-low-degree-row`

Under the exact-sequence and vanishing hypotheses of the relative-curve amplitude argument, the degree-zero and degree-one terms form an exact row

$$0 \longrightarrow A^0 \longrightarrow P^0 \longrightarrow E^0 \longrightarrow A^1 \longrightarrow P^1 \longrightarrow 0.$$

Proof. Uses: `lemma:weil-wpdf-cond-tame-amplitude`

Insert the vanishing of E^r outside degree zero, the supplied vanishing of P^r outside degrees zero through two, and the preceding vanishing of A^r into the full long exact sequence. The segment from degree minus one through degree one is the displayed row. □

Lemma 13.7.283 (Reciprocal lower bound from a supplied perfect pairing)**.** *Uses:*

Lean: `blueprint-target:lemma:weil-wpdf-cond-pd-reciprocity`

Let V, W be finite-dimensional Weil modules with a perfect Frobenius-equivariant pairing b

$$b : V \otimes W \longrightarrow E(-d)$$

for which geometric Frobenius on the target is q^d; explicitly, $b(F_V v, F_W w) = q^d b(v,w)$. If W has upper weight $-w + 2d - i$, then V has lower weight $w + i$.

Proof. Uses:

The adjoint of the perfect pairing is a Frobenius-equivariant isomorphism $V \simeq W^\vee(-d)$. Hence the eigenvalues, with algebraic multiplicity, can be paired as $\alpha\beta = q^d$. The assumed upper bound gives

$$|\iota(\beta)| \le q^{(-w+2d-i)/2},$$

so

$$|\iota(\alpha)| = q^d |\iota(\beta)|^{-1} \ge q^{(w+i)/2}.$$

This is the asserted lower bound. □

Lemma 13.7.284 (Parabolic curve j_*–j_* reciprocity from supplied data)**.** *Uses:*

Lean: `blueprint-target:lemma:weil-wpdf-cond-curve-jstar-jstar-reciprocity`

Let $j : U \hookrightarrow C$ be a dense open in a smooth proper curve. A supplied perfect Frobenius-equivariant Weil pairing b

$$b : H^i(C, j_*L) \otimes H^{2-i}(C, j_*L^\vee) \longrightarrow E(-1)$$

with $b(Fx, Fy) = qb(x,y)$, together with upper weights $w + i$ and $-w + 2 - i$ on the two displayed factors, implies that $H^i(C, j_*L)$ is pure of weight $w + i$.

Proof. Uses: `lemma:weil-wpdf-cond-pd-reciprocity`

Apply the reciprocal-pairing lemma with $d = 1$, $V = H^i(C, j_*L)$, and $W = H^{2-i}(C, j_*L^\vee)$. Its lower bound agrees with the supplied upper bound, which gives purity. The second factor is $j_*L^\vee$, exactly as in the supplied parabolic pairing. This input is not deduced by replacing ordinary or middle-extension j_* with derived Rj_*: Verdier duality pairs Rj_*L with $Rj_!L^\vee$, which is a different statement. □

13.7.19 Fixed RH endpoint: eigenvalues and reciprocal-root coordinates

Definition 13.7.285 (Arithmetic and geometric Frobenius conventions). Uses:

Lean: `conditional-interface:def:weil-wpdf-rh-frobenius-conventions`

Specialize to $L = \overline{\mathbf{Q}}_\ell$, $w = 0$, and $V_i = H^i(X_{\bar{k}}, \overline{\mathbf{Q}}_\ell)$ for one fixed $0 \le i \le 2d$. Arithmetic Frobenius on $\bar{k}$ is $F_{\mathrm{ar}} : x \mapsto x^q$, geometric Frobenius is $F_{\mathrm{geo}} = F_{\mathrm{ar}}^{-1}$, and their chosen cohomological operators satisfy

$$\Phi_{\mathrm{ar}} = \Phi_{\mathrm{geo}}^{-1}.$$

The route's Tate convention is that (r) multiplies Φ_{geo} by q^{-r} and Φ_{ar} by q^r.

Definition 13.7.286 (Algebraic realization and embedding data). Uses:

Lean: `conditional-interface:def:weil-wpdf-rh-algebraic-realization`

Fix a number field $E \subset \overline{\mathbf{Q}}$, an embedding $j_\ell : E \hookrightarrow \overline{\mathbf{Q}}_\ell$, and a multiset $A_i = \{\widetilde{\alpha}_1, \dots, \widetilde{\alpha}_m\} \subset E^\times$ whose image under j_ℓ is, with multiplicity, the spectrum of Φ_{geo} on V_i. For every complex embedding $\sigma : E \hookrightarrow \mathbf{C}$, fix a compatible field embedding $\iota_\sigma : \overline{\mathbf{Q}}_\ell \hookrightarrow \mathbf{C}$ satisfying $\iota_\sigma j_\ell = \sigma$ on E. The fixed-purity proposition is used with each such ι_σ; no independence of E, j_ℓ, or a Betti realization is asserted.

Definition 13.7.287 (Characteristic-polynomial and root variables). Uses:

Lean: `conditional-interface:def:weil-wpdf-rh-polynomial-variables`

Let T be a formal variable and set

$$P_i^{\mathrm{geo}}(T) = \det(1 - T\Phi_{\mathrm{geo}}) = \prod_j (1 - j_\ell(\widetilde{\alpha}_j)T),$$

$$P_i^{\mathrm{ar}}(T) = \det(1 - T\Phi_{\mathrm{ar}}) = \prod_j (1 - j_\ell(\widetilde{\alpha}_j)^{-1}T).$$

After applying σ, write $\rho_{j,\sigma}^{\mathrm{geo}} = \sigma(\widetilde{\alpha}_j)^{-1}$ for the zero in the T-plane of the jth geometric factor, and write $r_{j,\sigma}^{\mathrm{geo}} = (\rho_{j,\sigma}^{\mathrm{geo}})^{-1} = \sigma(\widetilde{\alpha}_j)$ for its reciprocal-root coordinate. If the single factor is inverted, its pole variable is the same $\rho_{j,\sigma}^{\mathrm{geo}}$; this is only a variable convention, not a zeta-function construction.

Definition 13.7.288 (Bounded RH endpoint output). Uses:

Lean: `conditional-interface:def:weil-wpdf-rh-endpoint-output`

The endpoint output is the collection, for every j and every $\sigma : E \hookrightarrow \mathbf{C}$, of the four modulus statements for the geometric eigenvalue, arithmetic eigenvalue, geometric-polynomial zero, and geometric reciprocal root. It contains no product over cohomological degrees, closed points, or Euler factors.

Lemma 13.7.289 (Frobenius inversion and Tate normalization). *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-rh-frobenius-inversion`

The arithmetic spectrum is $\{j_\ell(\widetilde{\alpha}_j)^{-1}\}_j$. On $\overline{\mathbf{Q}}_\ell(r)$, geometric Frobenius has multiplier q^{-r} and arithmetic Frobenius has multiplier q^r.

Proof. Uses:

The inverse of an invertible operator has the reciprocal spectrum, with multiplicity. The geometric multiplier is the fixed Tate convention, and inverting it gives the arithmetic multiplier. □

Lemma 13.7.290 (Purity gives every-embedding absolute values). *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-rh-embedding-absolute-value`

Assume the fixed Poincare-purity proposition specialized to the constant sheaf for every compatible ι_σ. Then

$$|\sigma(\widetilde{\alpha}_j)| = q^{i/2} \qquad \text{and} \qquad |\sigma(\widetilde{\alpha}_j)^{-1}| = q^{-i/2}$$

for every j and every complex embedding $\sigma : E \hookrightarrow \mathbf{C}$.

Proof. Uses:

For $L = \overline{\mathbf{Q}}_\ell$ and $w = 0$, fixed Poincare purity gives weight i. Apply it with ι_σ to $j_\ell(\widetilde{\alpha}_j)$ and use $\iota_\sigma j_\ell = \sigma$. Taking reciprocals gives the second equality. □

Lemma 13.7.291 (Eigenvalue, zero, pole, and reciprocal-root conversion). *Uses:*

Lean: `conditional-interface:lemma:weil-wpdf-rh-root-conversion`

For each j, σ,

quantity	*value*	*absolute value*
geometric eigenvalue	$\sigma(\widetilde{\alpha}_j)$	$q^{i/2}$
arithmetic eigenvalue	$\sigma(\widetilde{\alpha}_j)^{-1}$	$q^{-i/2}$
zero/pole variable $\rho^{\mathrm{geo}}_{j,\sigma}$	$\sigma(\widetilde{\alpha}_j)^{-1}$	$q^{-i/2}$
reciprocal root $r^{\mathrm{geo}}_{j,\sigma}$	$\sigma(\widetilde{\alpha}_j)$	$q^{i/2}$

The zero of the corresponding arithmetic factor is instead $\rho^{\mathrm{ar}}_{j,\sigma} = \sigma(\widetilde{\alpha}_j)$ and has modulus $q^{i/2}$.

Proof. Uses:
The factor $1 - aT$ vanishes exactly at $T = a^{-1}$, with the same multiplicity as a in the spectrum. Substitute the geometric and arithmetic eigenvalues from the preceding lemmas and take reciprocals. □

Proposition 13.7.292 (Conditional fixed RH endpoint)**.** *Uses:*

Lean: `conditional-interface:prop:weil-wpdf-rh-fixed-endpoint`

Under the fixed algebraic realization, compatible embeddings, and the fixed Poincare-purity proposition for the constant sheaf, every geometric Frobenius eigenvalue on $H^i(X_{\bar{k}}, \overline{\mathbf{Q}}_\ell)$ has absolute value $q^{i/2}$ under every $\sigma : E \hookrightarrow \mathbf{C}$. Equivalently, every reciprocal root of $P_i^{\mathrm{geo}}(T)$ has that absolute value, while every actual zero in the T-plane has absolute value $q^{-i/2}$.

Proof. Uses:
The embedding statement is the every-embedding lemma. The exact conversion to polynomial variables is the root-conversion lemma. No trace formula, Euler product, or rationality input is used. □

Endpoint dependency boundary. This endpoint closes no construction leaf. All 64 earlier `OPEN WPDF-*` obligations remain visible above and remain required: in particular `WPDF-PD`, `WPDF-BASE-A4`, `WPDF-DERIVED`, the fixed-degree RFI tree, and their upstream leaves. The algebraic realization and compatible embeddings are explicit fixed data; the endpoint does not promote them to a Betti-independence theorem.

13.7.20 Zeta and global Weil extensions not entered

This route forms no alternating product of the $P_i^{\mathrm{geo}}(T)$, no closed-point Euler product, and no zeta function. It proves no trace-formula rationality statement, functional equation, Betti-number independence, coefficient-field independence, or total Weil theorem.

13.7.21 Forbidden branch

No node in this primary route may depend on a product surface, Lefschetz pencil, blowup, critical-value calculation, global vanishing-cycle square, Kunneth defect halving, or any W3 square-improvement statement.

Chapter 14

Trace-core follow-up: fixed points and determinant series

Status. The Frobenius convention is inherited without change: arithmetic $\varphi_q(x) = x^q$, geometric $\Phi_q = \varphi_q^{-1}$, $\Phi_q|E(1) = q^{-1}$, and $P_i(T) = \det(1 - TF_i)$. The legacy fixed-point lemmas below are abstract implications from explicit certificates. Their finite-type-scheme instance is supplied later by the TraceCore3/ZPC package. The determinant and formal power-series part is proved by finite algebra and the previously audited trace–log–determinant leaf.

14.1 Finite-field Frobenius and fixed loci

Definition 14.1.1 (Follow-up Frobenius iterate convention). Uses: `def:weil-arithmetic-field-frobenius`, `def:weil-geometric-field-frobenius`, `def:tracecore-arithmetic-conventions`

Lean: `Weil.TraceCore2.FrobeniusIterateConvention`

For $n \geq 1$, define

$$\varphi_{q,n} := \varphi_q^n, \qquad \Phi_{q,n} := \Phi_q^n.$$

The fixed-point predicate on a geometric-point type Ω is $\mathrm{Fix}_n(x) :\Longleftrightarrow \Phi_{q,n}(x) = x$. This Definition fixes the iterate before any point-counting comparison is made.

Lemma 14.1.2 (Arithmetic and geometric iterates are inverse). *Uses:* *def: tracecore2-frobenius-iterate-convention*

Lean: *Weil.TraceCore2.frobeniusIterate_inverse*

For every $n \geq 1$,

$$\Phi_{q,n} \circ \varphi_{q,n} = \mathrm{id}, \qquad \varphi_{q,n} \circ \Phi_{q,n} = \mathrm{id}\,.$$

Proof. Uses: `def:tracecore2-frobenius-iterate-convention`, `lemma:weil-frobenius-inverse-identities`

Expand both iterates as products of n copies of the two inverse automorphisms. Associate the product from the left. At each adjacent pair, replace $\Phi_q \circ \varphi_q$ or $\varphi_q \circ \Phi_q$ by the identity using the two inverse equations. Induction on n removes all pairs and leaves the identity map. □

Definition 14.1.3 (Follow-up fixed-locus subtype). Uses: `def:tracecore2-frobenius-iterate-convention`, `def:tracecore-periodic-point-input`

Lean: `Weil.TraceCore2.FixedLocusSubtype`

For a periodic geometric-point input $(\Omega, \Phi, \mathrm{per})$, define

$$\Omega_n^{\mathrm{fix}} := \{x : \Omega \mid \Phi^n(x) = x\}.$$

The subtype projection is written $\pi_n : \Omega_n^{\mathrm{fix}} \to \Omega$.

Lemma 14.1.4 (Finite fixed-locus subtype). *Uses:* *def: tracecore2-fixed-locus-subtype*

Lean: *Weil.TraceCore2.fixedLocus_finite*

If Ω is finite, then Ω_n^{fix} is finite for every $n \geq 1$.

Proof. Uses: def:tracecore2-fixed-locus-subtype
The subtype carrier embeds into Ω by π_n. Restrict the finite enumeration of Ω to those entries satisfying $\Phi^n(x) = x$. This is a finite enumeration of the subtype, and hence gives its finite-type and finite-cardinality structures. □

Definition 14.1.5 (Follow-up rational–fixed map candidates). Uses: def:tracecore-finite-extension-point-input, def:tracecore2-fixed-locus-subtype

Lean: Weil.TraceCore2.RationalFixedMapCandidates
For each $n \geq 1$, define a candidate package to consist of maps

$$u_n : R_n \to \Omega_n^{\mathrm{fix}}, \qquad v_n : \Omega_n^{\mathrm{fix}} \to R_n,$$

and the two factorization propositions

$$\pi_n u_n = s_n, \qquad v_n \pi_n = r_n|_{\Omega_n^{\mathrm{fix}}}.$$

The maps and propositions are fields of the candidate package; no proof of either proposition or of an inverse equation is included.

Lemma 14.1.6 (Forward map has the required fixed-point condition). Uses: def:tracecore2-rational-fixed-map-candidates, def:tracecore2-fixed-locus-subtype

Lean: Weil.TraceCore2.forwardFixedCondition
For every $y \in R_n$,

$$\Phi_q^n(\pi_n(u_n(y))) = \pi_n(u_n(y)).$$

Proof. Uses: def:tracecore2-rational-fixed-map-candidates, def:tracecore2-fixed-locus-subtype
The value $u_n(y)$ has codomain Ω_n^{fix}. Its subtype membership field is, after unfolding, exactly the displayed equation. □

Lemma 14.1.7 (Two inverse equations from supplied rational–fixed certificates). Uses: def:tracecore2-rational-fixed-map-candidates

Lean: Weil.TraceCore2.rationalFixed_inverseEquations
Suppose that the candidate package is supplied with pointwise certificates

$$h_n^{\mathrm{L}} : \forall y \in R_n,\ v_n(u_n(y)) = y, \qquad h_n^{\mathrm{R}} : \forall z \in \Omega_n^{\mathrm{fix}},\ u_n(v_n(z)) = z.$$

Then

$$v_n \circ u_n = \mathrm{id}_{R_n}, \qquad u_n \circ v_n = \mathrm{id}_{\Omega_n^{\mathrm{fix}}}.$$

Proof. Uses: def:tracecore2-rational-fixed-map-candidates
Apply function extensionality to the two composites. The resulting pointwise equations are h_n^{L} and h_n^{R}, respectively. Thus no inverse law is extracted from the factorization fields alone. □

Proposition 14.1.8 (Point-count/fixed-locus equality from inverse certificates). Uses: def:tracecore-finite-extension-point-input, def:tracecore2-fixed-locus-subtype, def:tracecore2-rational-fixed-map-candidates

Lean: Weil.TraceCore2.pointCount_fixedLocus
If the two pointwise inverse certificates in the preceding lemma are supplied, then for every $n \geq 1$,

$$\#R_n = \#\Omega_n^{\mathrm{fix}}.$$

Proof. Uses: lemma:tracecore2-rational-fixed-inverse-equations, lemma:tracecore2-fixed-locus-finite
The inverse certificates make u_n and v_n an equivalence. The finite-extension input makes R_n finite, and the fixed-locus lemma makes the target finite. Transport finite cardinality along the equivalence. □

14.2 One factor: finite matrix algebra and formal series

Definition 14.2.1 (Single finite-dimensional Frobenius datum). Uses: `def:weil-frobenius-module`, `def:tracecore-arithmetic-conventions`

Lean: `Weil.TraceCore2.SingleFrobeniusDatum`

A single finite-dimensional Frobenius datum is (V, F, β, r), where V is an E-vector space of dimension r, $F : V \simeq_E V$, and β is a basis indexed by $\mathrm{Fin}(r)$. Define the coordinate matrix $A := \mathrm{toMatrix}(\beta, \beta, F)$ and

$$P_{V,F}(T) := \det(I_r - TA) \in E[T].$$

Lemma 14.2.2 (Constant term of the single factor). Uses: *def:tracecore2-single-frobenius-datum*

Lean: *Weil.TraceCore2.singleFactor_constant*

$$P_{V,F}(0) = 1.$$

Proof. Uses: `def:tracecore2-single-frobenius-datum`, `lemma:weil-based-reciprocal-determinant-constant`

At $T = 0$, the matrix $I_r - TA$ is I_r. The determinant of the identity matrix is one by the finite permutation-sum Definition of determinant. The cited audited determinant lemma gives the same statement for the given basis datum. □

Lemma 14.2.3 (Degree bound and invertible leading coefficient). Uses: *def:tracecore2-single-frobenius-datum*

Lean: *Weil.TraceCore2.singleFactor_degree*

The degree of $P_{V,F}$ is at most r; if F is an equivalence, its coefficient of T^r is $(-1)^r \det(A) \neq 0$, so the degree is exactly r.

Proof. Uses: `def:tracecore2-single-frobenius-datum`, `lemma:weil-based-reciprocal-determinant-constant`

Each determinant summand contains at most one factor of T from each of the r rows, proving the degree bound. The only degree-r term uses $-TA$ in every row; its sign is $(-1)^r$ and its coefficient is $\det(A)$. Since F is an equivalence, A has an inverse and its determinant is nonzero. Therefore this coefficient does not vanish. □

Definition 14.2.4 (Single-factor trace logarithm). Uses: `def:tracecore2-single-frobenius-datum`

Lean: `Weil.TraceCore2.SingleTraceLog`

Define

$$S_{V,F}(T) := \sum_{n \geq 1} \mathrm{tr}(A^n) \frac{T^n}{n} \in E[[T]].$$

The sum is a formal series: for a fixed degree m, only the term $n = m$ contributes.

Lemma 14.2.5 (Single-factor trace–log–det identity). Uses: *def:tracecore2-single-frobenius-datum*, *def:tracecore2-single-trace-log*

Lean: *Weil.TraceCore2.singleTraceLogDet*

$$-\mathrm{Log}(P_{V,F}(T)) = S_{V,F}(T).$$

Proof. Uses: `def:tracecore2-single-frobenius-datum`, `def:tracecore2-single-trace-log`, `thm:weil-based-trace-log-det`

Instantiate the audited finite-dimensional trace–log–determinant theorem with the based endomorphism datum (V, β, F). The theorem's reciprocal determinant is the polynomial $P_{V,F}$ by Definition, and its degree-n trace is $\mathrm{tr}(A^n)$. Substitution gives the coefficient equality for every n, while both sides have zero constant coefficient. □

Lemma 14.2.6 (Exponential determinant identity). Uses: *def:tracecore2-single-frobenius-datum*, *def:tracecore2-single-trace-log*

Lean: *Weil.TraceCore2.singleExpTraceDet*

$$\mathrm{Exp}(S_{V,F}(T)) = P_{V,F}(T)^{-1} \quad \text{in } E[[T]].$$

Proof. Uses: `lemma:tracecore2-single-trace-log-det`, `def:tracecore2-single-frobenius-datum`

Replace $S_{V,F}$ by $-\mathrm{Log}(P_{V,F})$. The constant coefficient of $P_{V,F}$ is one by the preceding constant-term lemma, so formal exponential and logarithm are inverse on this series. Thus $\mathrm{Exp}(-\mathrm{Log}(P_{V,F})) = P_{V,F}^{-1}$. □

Definition 14.2.7 (Finite alternating factor product). Uses: def:tracecore2-single-frobenius-datum

Lean: Weil.TraceCore2.AlternatingFactorProduct

For a finite family $(V_i, F_i)_{0\le i\le B}$, define

$$\Delta_B(T) := \prod_{i=0}^{B} P_{V_i,F_i}(T)^{(-1)^{i+1}}.$$

All factors have constant term one, so the negative powers are formal inverses and the product is finite.

Lemma 14.2.8 (Alternating finite trace logarithm). Uses: def:tracecore2-alternating-factor-product, def:tracecore2-single-trace-log

Lean: Weil.TraceCore2.alternatingTraceLog

$$-\operatorname{Log}(\Delta_B(T)) = \sum_{n\ge 1}\left(\sum_{i=0}^{B}(-1)^i \operatorname{tr}(A_i^n)\right)\frac{T^n}{n}.$$

Proof. Uses: def:tracecore2-alternating-factor-product,lemma:tracecore2-single-trace-log-det,def:tracecore2-single-trace-log

Use additivity of the formal logarithm for the finite product. The exponent $(-1)^{i+1}$ changes the negative logarithm of the i-th factor into $(-1)^i S_{V_i,F_i}$. Sum the finitely many resulting series and then collect the coefficient of T^n; this yields the displayed finite inner sum. □

14.3 A4 adic exceptional-transition cluster

> **Status.** This cluster assumes the finite exceptional Hom construction and the A2 reduction/completion interfaces already declared in the A4 source. It does not discharge Brown representability, compactification independence, or the general adic adjunction. Those dependencies remain explicit OPEN leaves below.

14.3.1 Finite-level transition datum

Definition 14.3.1 (Typed finite transition input). Lean: conditional-interface:def:weil-a4-adc-transition-input

Uses: def:weil-a4-exceptional-reduction-problem,def:weil-a4-finite-exceptional-pullback-object,def:weil-a4-finite-exceptional-counit

Fix integers $m > n \ge 1$, a separated finite-type map $f : X \to Y$, and finite coefficient rings $\Lambda_m \twoheadrightarrow \Lambda_n$. A typed finite transition input consists of

$$L_m \in D_c^b(Y, \Lambda_m), \qquad L_n := L_m \otimes^L_{\Lambda_m} \Lambda_n,$$

the objects

$$E_m := Rf_m^! L_m \in D_c^b(X, \Lambda_m), \qquad E_n := Rf_n^! L_n \in D_c^b(X, \Lambda_n),$$

the finite counit $\epsilon_m : Rf_! E_m \to L_m$, and the coefficient-change isomorphism

$$\rho_{m,n} : Rf_! E_m \otimes^L_{\Lambda_m} \Lambda_n \xrightarrow{\sim} Rf_!(E_m \otimes^L_{\Lambda_m} \Lambda_n).$$

The input also records the reduction morphism $r_{m,n} : L_m \otimes^L \Lambda_n \to L_n$, its identity and composition equations, and the chosen finite-level Hom representing bijection

$$\Theta_n(K, E_n) : \operatorname{Hom}_X(K, E_n) \xrightarrow{\sim} \operatorname{Hom}_Y(Rf_! K, L_n).$$

No transition map $E_m \otimes^L \Lambda_n \to E_n$ is part of this definition; it is constructed below. The displayed fields have the indicated source and target by the coefficient reduction definition and the finite exceptional-pullback definition. The map $\rho_{m,n}$ is the coefficient-change comparison for $Rf_!$, while the equations for $r_{m,n}$ are the equations in the coefficient tower. The last displayed bijection is the finite representing map already attached to the exceptional counit. Since the definition merely packages these typed fields and does not assert a new existence theorem, the construction is well-formed.

Definition 14.3.2 (Reduced counit at a finite transition). Lean: conditional-interface:def:weil-a4-adc-reduced-counit

Uses: def:weil-a4-adc-transition-input

For the input of Definition 14.3.1, define

$$c_{m,n} : Rf_!(E_m \otimes^L_{\Lambda_m} \Lambda_n) \longrightarrow L_n$$

by the composite

$$Rf_!(E_m \otimes^L \Lambda_n) \xleftarrow{\rho_{m,n}} Rf_!E_m \otimes^L \Lambda_n \xrightarrow{\epsilon_m \otimes 1} L_m \otimes^L \Lambda_n \xrightarrow{r_{m,n}} L_n.$$

Here the first arrow is $\rho^{-1}_{m,n}$, so the displayed composite is read from left to right with the inverse coefficient-change isomorphism. The source of $\rho^{-1}_{m,n}$ is exactly $Rf_!(E_m \otimes^L \Lambda_n)$, and its target is $Rf_!E_m \otimes^L \Lambda_n$. The next arrow is obtained by tensoring the finite counit with Λ_n, and the last arrow is the reduction map in the input. The codomain is therefore L_n, so the composite has the declared type.

Lemma 14.3.3 (Type of the reduced counit). *Lean:* `conditional-interface:lemma:weil-a4-adc-reduced-counit-type`

Uses: `def:weil-a4-adc-reduced-counit`

The reduced counit has the typed form

$$c_{m,n} : Rf_!(E_m \otimes^L_{\Lambda_m} \Lambda_n) \longrightarrow L_n,$$

and its restriction along a morphism $u : L_m \to L'_m$ is the reduced counit obtained from u after tensoring with Λ_n.

Proof. Uses: `def:weil-a4-adc-transition-input`

The first assertion is the source-target calculation in the preceding definition. For the second, tensor the naturality square $u\epsilon_m = \epsilon'_m Rf_!(Rf^!_m u)$ with Λ_n, then compose with $r_{m,n}$. Functoriality of tensor and the coefficient-change square identifies the resulting composite with the reduced counit for u. □

Definition 14.3.4 (Finite transition candidate). Lean: `conditional-interface:def:weil-a4-adc-transition-candidate`

Uses: `def:weil-a4-adc-transition-input`, `def:weil-a4-adc-reduced-counit`, `def:weil-a4-finite-exceptional-forward-map`

Define the finite transition candidate

$$\tau^{\mathrm{cand}}_{m,n} : E_m \otimes^L_{\Lambda_m} \Lambda_n \longrightarrow E_n$$

to be the forward representing image under Θ_n of the reduced counit:

$$\tau^{\mathrm{cand}}_{m,n} := \Theta_n^{-1}(c_{m,n}).$$

The symbol Θ_n^{-1} denotes the inverse supplied by the finite representing map; its existence and independence are isolated in the next lemmas. The source of $c_{m,n}$ is the object $E_m \otimes^L \Lambda_n$, so it is an admissible first argument of Θ_n. Applying the displayed inverse produces a morphism with codomain E_n, which is the declared type. No claim that this inverse is independent of all representatives is made in the definition.

Lemma 14.3.5 (Representing equation for the transition candidate). *Lean:* `conditional-interface:lemma:weil-a4-adc-transition-representing-equation`

Uses: `def:weil-a4-adc-transition-candidate`

The candidate satisfies

$$\Theta_n(\tau^{\mathrm{cand}}_{m,n}) = c_{m,n}.$$

Proof. Uses: `prop:weil-a4-finite-representing-left-inverse-open`, `def:weil-a4-adc-reduced-counit`

Apply the left inverse equation for the finite representing maps to the element $c_{m,n}$. Substitution of the definition of $\tau^{\mathrm{cand}}_{m,n}$ gives exactly the displayed equality.

Open obligation: `OBL-A4-ADCLUSTER-REPRESENTING`

□

Lemma 14.3.6 (Well-definedness of the finite transition candidate). *Lean:* `conditional-interface:lemma:weil-a4-adc-transition-well-defined`

Uses: `def:weil-a4-adc-transition-candidate`

Changing the representatives of L_m, the coefficient resolution, or the finite Hom representing roof leaves $\tau^{\mathrm{cand}}_{m,n}$ unchanged in the derived category.

Proof. Uses: `lemma:weil-a4-adc-transition-representing-equation,prop:weil-a4-finite-representing-right-inverse-open`

Two choices produce maps with the same image under Θ_n, because the coefficient-change square and the reduced-counit equation are natural under comparison roofs. Apply injectivity of the finite representing bijection to the equality of these images. The comparison of arbitrary roofs and the independence of the inverse representing map are the unresolved leaf `OBL-A4-ADCLUSTER-REPRESENTING`.

Open obligation: `OBL-A4-ADCLUSTER-WELLDEFINED`

□

Lemma 14.3.7 (Naturality of finite transition candidates). *Lean:* `conditional-interface:lemma:weil-a4-adc-transition-naturality`

Uses: `def:weil-a4-adc-transition-candidate`

For a morphism $u : L_m \to L'_m$, with reduction $u_n : L_n \to L'_n$,

$$(Rf^!_n u_n)\,\tau^{\mathrm{cand}}_{m,n,L_m} = \tau^{\mathrm{cand}}_{m,n,L'_m}(Rf^!_m u \otimes^L \Lambda_n).$$

Proof. Uses: `lemma:weil-a4-adc-reduced-counit-type,lemma:weil-a4-adc-transition-representing-equation,prop:weil-a4-finite-exceptional-counit-naturality-open`

Apply Θ_n to both sides. The left side becomes the reduced counit for u_n by counit naturality; the right side becomes the same reduced counit after tensoring u, by the preceding type lemma. Equality of the two images and injectivity of Θ_n give the formula. The injectivity step is conditional on the finite representing leaf `OBL-A4-ADCLUSTER-REPRESENTING`.

Open obligation: `OBL-A4-ADCLUSTER-NATURALITY`

□

Lemma 14.3.8 (Identity transition candidate). *Lean:* `conditional-interface:lemma:weil-a4-adc-transition-identity`

Uses: `def:weil-a4-adc-transition-candidate`

For $m = n$, the candidate is the identity of E_n.

Proof. Uses: `lemma:weil-a4-adc-transition-representing-equation,prop:weil-a4-finite-representing-right-inverse-open`

At $m = n$, the coefficient-change map and reduction map are identities, so the reduced counit is ϵ_n. The identity of E_n has image ϵ_n under Θ_n, by the definition of the finite counit. Injectivity of Θ_n therefore identifies the candidate with the identity. This is conditional on `OBL-A4-ADCLUSTER-REPRESENTING`.

Open obligation: `OBL-A4-ADCLUSTER-IDENTITY`

□

Lemma 14.3.9 (Two-step cocycle for transition candidates). *Lean:* `conditional-interface:lemma:weil-a4-adc-transition-cocycle`

Uses: `def:weil-a4-adc-transition-candidate`

For $r < n < m$,

$$\tau^{\mathrm{cand}}_{n,r} \circ (\tau^{\mathrm{cand}}_{m,n} \otimes^L \Lambda_r) = \tau^{\mathrm{cand}}_{m,r}.$$

Proof. Uses: `lemma:weil-a4-adc-transition-representing-equation,lemma:weil-a4-adc-transition-naturality,lemma:weil-a4-adc-transition-identity`

Apply Θ_r to the two sides. The image of the left side is obtained by first reducing the level-m counit to level n, then reducing it to level r. The coefficient reductions satisfy $r_{m,r} = r_{n,r} r_{m,n}$, and the coefficient-change maps for $Rf_!$ compose, so this image equals the reduced level-r counit. The image of the right side is the same by its representing equation. Injectivity of Θ_r proves the equality, conditional on the representing and naturality leaves.

Open obligation: `OBL-A4-ADCLUSTER-COCYCLE`

□

14.3.2 Tower and Frobenius structure

Definition 14.3.10 (Exceptional inverse-image transition tower)**.** Lean: `conditional-interface:def:weil-a4-adc-transition-tower`

Uses: `def:weil-a4-adc-transition-candidate`

For a compatible coefficient tower $(L_n)_n$, define

$$\mathcal{T}_f(L) := \left(E_n, \tau^{\mathrm{cand}}_{n+1,n}\right)_{n\geq 1}, \qquad E_n := Rf_n^! L_n.$$

The transition maps are the candidates of the preceding definition. The object at level n is the finite exceptional pullback of L_n, and the arrow from level $n+1$ to level n is the candidate with $m = n+1$. Thus every displayed component has the required type. The identity and cocycle equations are not silently asserted here; they are the two preceding lemmas.

Lemma 14.3.11 (Tower identity and cocycle)**.** Lean: *`conditional-interface:lemma:weil-a4-adc-transition-tower-coherence`*

Uses: *`def:weil-a4-adc-transition-tower`*

The tower transitions satisfy the identity equation at every level and the two-step cocycle equation at every triple of levels.

Proof. Uses: `lemma:weil-a4-adc-transition-identity,lemma:weil-a4-adc-transition-cocycle`

The identity equation is the identity-transition lemma with $m = n$. The cocycle equation is the two-step cocycle lemma applied to the three selected levels. No additional limit or completion argument occurs at this stage. □

Definition 14.3.12 (Frobenius-equivariant transition tower datum)**.** Lean: `conditional-interface:def:weil-a4-adc-frobenius-tower-datum`

Uses: `def:weil-a4-adc-transition-tower,def:weil-a4-geometric-frobenius-convention`

For each level n, suppose a Weil descent map

$$\varphi_n : \Phi_Y^* L_n \xrightarrow{\sim} L_n$$

is given, and let $\psi_n : \Phi_X^* E_n \xrightarrow{\sim} E_n$ be the finite exceptional-pullback descent map obtained by applying the finite representing construction to φ_n. A Frobenius-equivariant tower datum consists of these maps, their reduction equations, and the requirement that the squares with the transition candidates commute. The geometric Frobenius convention fixes Φ_X and Φ_Y. Each φ_n is an isomorphism in the coefficient category, and the finite representing construction gives a map on E_n with the displayed source and target. The reduction equations and commutative transition squares are recorded as fields rather than inferred, so the datum is well-typed.

Lemma 14.3.13 (Frobenius compatibility of transition candidates)**.** Lean: *`conditional-interface:lemma:weil-a4-adc-transition-frobenius`*

Uses: *`def:weil-a4-adc-frobenius-tower-datum`*

For $m > n$, the square

$$\begin{array}{ccc} \Phi_X^*(E_m \otimes^L \Lambda_n) & \xrightarrow{\Phi_X^* \tau^{\mathrm{cand}}_{m,n}} & \Phi_X^* E_n \\ {\scriptstyle \psi_m \otimes 1}\downarrow & & \downarrow{\scriptstyle \psi_n} \\ E_m \otimes^L \Lambda_n & \xrightarrow[\tau^{\mathrm{cand}}_{m,n}]{} & E_n \end{array}$$

commutes.

Proof. Uses: `lemma:weil-a4-adc-transition-representing-equation,lemma:weil-a4-adc-transition-naturality`

Apply the level-(n) representing bijection to the two composites. The Frobenius-equivariance of the finite counit identifies both images with the reduced counit of φ_m, followed by the reduction equation for φ_n. The two images are therefore equal; injectivity of the representing bijection gives commutativity. The Frobenius-equivariant counit square is not in the current foundation and is recorded as `OBL-A4-ADCLUSTER-FROBENIUS`.

Open obligation: `OBL-A4-ADCLUSTER-FROBENIUS`

□

14.3.3 Adic inverse limit

Definition 14.3.14 (Adic exceptional inverse-image candidate)**.** Lean: `conditional-interface:def:weil-a4-adc-derived-limit-object`

Uses: `def:weil-a4-adc-transition-tower,def:weil-a2-derived-completion`

For a complete coefficient object L, define

$$Rf^!_{\Lambda,\mathrm{adc}}L := R\varprojlim_n E_n$$

using the explicit A2 derived inverse-limit model, with cone projections $\pi_n : Rf^!_{\Lambda,\mathrm{adc}}L \otimes^L \Lambda_n \to E_n$. The tower coherence lemma supplies an inverse system in the category of finite-level complexes. Applying the A2 derived-completion construction to that system gives the displayed derived inverse limit and its canonical cone projections. The definition does not assert that any projection is an isomorphism.

Lemma 14.3.15 (Naturality of adic cone projections)**.** *Lean:* `conditional-interface:lemma:weil-a4-adc-derived-limit-projection-naturality`

Uses: `def:weil-a4-adc-derived-limit-object`

For a morphism $u : L \to L'$, the induced map of towers commutes with every projection π_n.

Proof. Uses: `lemma:weil-a4-adc-transition-naturality,def:weil-a4-adc-transition-tower`

At level n, the map of towers is $Rf^!_n(u_n)$. Naturality of the finite transition candidates says that these maps commute with every transition arrow. The universal property of the derived inverse-limit cone then gives a unique map of limits whose composite with π_n is the displayed level map. This is a componentwise verification of the cone equations. □

Proposition 14.3.16 (Adic reduction comparison for the cluster)**.** *Lean:* `conditional-interface:prop:weil-a4-adc-derived-limit-reduction-open`

Uses: `def:weil-a4-adc-derived-limit-object`

For every n, the canonical projection

$$\pi_n : Rf^!_{\Lambda,\mathrm{adc}}L \otimes^L \Lambda_n \longrightarrow Rf^!_n L_n$$

is an isomorphism whenever the finite-level transition tower has bounded constructible amplitude and satisfies the Mittag–Leffler vanishing required by the A2 derived-completion model.

Proof. Uses: `lemma:weil-a4-adc-derived-limit-projection-naturality`

The cone of π_n is the shifted higher derived-limit term of the reduction tower. Under the stated bounded-amplitude hypothesis, only finitely many cohomological degrees contribute. The Mittag–Leffler vanishing kills the $R^1\varprojlim$ term in each such degree, so the long exact cohomology sequence of the derived-limit triangle makes the cone acyclic. A morphism with acyclic cone is an isomorphism in the derived category. Establishing the bounded amplitude and the required Mittag–Leffler vanishing is the exact unresolved leaf `OBL-A4-ADCLUSTER-REDUCTION`.

Open obligation: `OBL-A4-ADCLUSTER-REDUCTION`

□

14.4 A4 finite exceptional-pullback arrow cluster

> **Status.** The cluster refines only the finite-level arrow $Rf^!_n(v)$. It assumes the finite Brown/representing interfaces already declared in the A4 source. No representability, duality, Frobenius-counit, or coefficient-reduction theorem is silently discharged.

14.4.1 Arrow construction and formal identities

Definition 14.4.1 (Typed finite exceptional-arrow input)**.** Lean: `conditional-interface:def:weil-a4-fl-arrow-input`

Uses: `def:weil-a4-exceptional-index,def:weil-a4-finite-exceptional-pullback-object,def:weil-a4-finite-exceptional-counit`

Fix a finite coefficient ring Λ_n, a separated finite-type map $f : X \to Y$, objects $L, L' \in D^b_c(Y, \Lambda_n)$, and a morphism $v : L \to L'$. Put

$$E := Rf^!_n L, \qquad E' := Rf^!_n L',$$

and retain the finite counits

$$\epsilon_L : Rf_!E \to L, \qquad \epsilon_{L'} : Rf_!E' \to L'.$$

This is only typed object-and-map data. In particular, the input contains neither an inverse equation for the upstream maps Φ and Ψ, nor a map $E \to E'$, nor any naturality or Frobenius certificate.

Definition 14.4.2 (Finite exceptional-arrow candidate)**.** Lean: `conditional-interface:def:weil-a4-fl-arrow-candidate`

Uses: `def:weil-a4-fl-arrow-input`, `def:weil-a4-finite-exceptional-forward-map`

Define

$$a_v := v \circ \epsilon_L : Rf_!E \longrightarrow L', \qquad Rf^!_{n,\mathrm{fl}}(v) := \Phi_{E,L'}(a_v) : E \longrightarrow E'.$$

The notation $Rf^!_{n,\mathrm{fl}}(v)$ denotes this candidate only; its independence and functorial identities are separate lemmas.

Lemma 14.4.3 (Type correctness of the finite exceptional arrow)**.** Lean: *`conditional-interface:lemma:weil-a4-fl-arrow-typing`*

Uses: *`def:weil-a4-fl-arrow-candidate`*

The composite a_v has source $Rf_!E$ and target L', and $Rf^!_{n,\mathrm{fl}}(v)$ has source E and target E'.

Proof. Uses: `def:weil-a4-fl-arrow-input`, `def:weil-a4-finite-exceptional-forward-map`

The counit has type $Rf_!E \to L$, and postcomposition with $v : L \to L'$ gives $a_v : Rf_!E \to L'$. The forward representing map $\Phi_{E,L'}$ takes precisely such a morphism to a map $E \to Rf^!_n L' = E'$. These are the two claimed source-target calculations. □

Lemma 14.4.4 (Representing equation for the finite exceptional arrow)**.** Lean: *`conditional-interface:lemma:weil-a4-fl-arrow-representing-equation`*

Uses: *`def:weil-a4-fl-arrow-candidate`*, *`def:weil-a4-finite-exceptional-backward-map`*

The candidate satisfies

$$\Psi_{E,L'}\big(Rf^!_{n,\mathrm{fl}}(v)\big) = v \circ \epsilon_L.$$

Proof. Uses: `prop:weil-a4-finite-representing-left-inverse-open`, `def:weil-a4-fl-arrow-input`, `def:weil-a4-fl-arrow-candidate`, `def:weil-a4-finite-exceptional-backward-map`

Expand the candidate as $\Phi_{E,L'}(a_v)$. The left inverse equation for the finite forward and backward representing maps sends this element back to a_v, which is $v\epsilon_L$ by Definition 14.4.2. The inverse step is conditional on `OBL-A4-FL-REPRESENTING`.

Open obligation: `OBL-A4-FL-REPRESENTING`

□

Lemma 14.4.5 (Well-definedness of the finite exceptional arrow)**.** Lean: *`conditional-interface:lemma:weil-a4-fl-arrow-well-defined`*

Uses: *`def:weil-a4-fl-arrow-candidate`*

Changing the representing roof, the chosen finite replacements, or the counit representative leaves $Rf^!_{n,\mathrm{fl}}(v)$ unchanged in the finite derived category.

Proof. Uses: `lemma:weil-a4-fl-arrow-representing-equation`, `prop:weil-a4-finite-representing-right-inverse-open`, `def:weil-a4-finite-exceptional-backward-map`

Comparison roofs transport the counit square and therefore transport a_v to the same morphism in $\mathrm{Hom}_Y(Rf_!E, L')$. The two candidate maps consequently have equal images under $\Psi_{E,L'}$. The upstream right-inverse proposition makes the backward map injective and therefore identifies the candidates. The roof-localization and inverse-representing assertions remain the explicit leaves `OBL-A4-FL-REPRESENTING` and `OBL-A4-FL-WELLDEFINED`.

Open obligation: `OBL-A4-FL-WELLDEFINED`

□

Lemma 14.4.6 (Identity law for the finite exceptional arrow)**.** Lean: *`conditional-interface:lemma:weil-a4-fl-arrow-identity`*

Uses: *`def:weil-a4-fl-arrow-candidate`*

For $v = \mathrm{id}_L$,

$$Rf^!_{n,\mathrm{fl}}(\mathrm{id}_L) = \mathrm{id}_E\,.$$

Proof. Uses: `lemma:weil-a4-fl-arrow-representing-equation`, `def:weil-a4-fl-arrow-input`, `def:weil-a4-finite-exceptional-backward-map`, `prop:weil-a4-finite-representing-right-inverse-open`

For the identity morphism, the defining composite is the counit ϵ_L. The identity of E has the same image under the backward representing map, by the definition of the finite counit. The upstream right-inverse proposition makes that map injective and therefore gives equality. This use remains the explicit unresolved input `OBL-A4-FL-IDENTITY`.

Open obligation: `OBL-A4-FL-IDENTITY`

□

Lemma 14.4.7 (Composition law for finite exceptional arrows). *Lean:* `conditional-interface:lemma:weil-a4-fl-arrow-composition`

Uses: `def:weil-a4-fl-arrow-candidate`
For $L \xrightarrow{v} L' \xrightarrow{w} L''$,

$$Rf^{!}_{n,\mathrm{fl}}(wv) = Rf^{!}_{n,\mathrm{fl}}(w) \circ Rf^{!}_{n,\mathrm{fl}}(v).$$

Proof. Uses: `lemma:weil-a4-fl-arrow-representing-equation,lemma:weil-a4-fl-arrow-well-defined,prop:weil-a4-finite-representing-right-inverse-open,def:weil-a4-fl-arrow-input,def:weil-a4-finite-exceptional-backward-map`
Apply $\Psi_{E,L''}$ to both sides. The left image is $wv\epsilon_L$. For the right image, first use the representing equation for $Rf^{!}_{n,\mathrm{fl}}(w)$, then substitute the equation for $Rf^{!}_{n,\mathrm{fl}}(v)$; counit naturality identifies the result with $wv\epsilon_L$. The upstream right-inverse equation makes $\Psi_{E,L''}$ injective, proving the equality conditional on `OBL-A4-FL-COMPOSITION` and the representing leaf.

Open obligation: `OBL-A4-FL-COMPOSITION`

□

14.4.2 Frobenius and coefficient reduction

Definition 14.4.8 (Raw Frobenius maps for a finite exceptional arrow). *Lean:* `conditional-interface:def:weil-a4-fl-arrow-weil-data`

Uses: `def:weil-a4-fl-arrow-input,def:weil-a4-geometric-frobenius-convention`
In addition to the input of Definition 14.4.1, fix the six typed morphisms

$$\varphi_L : \Phi_Y^* L \longrightarrow L, \quad \varphi_{L'} : \Phi_Y^* L' \longrightarrow L', \quad \psi_E : \Phi_X^* E \longrightarrow E, \quad \psi_{E'} : \Phi_X^* E' \longrightarrow E',$$

and

$$\chi_E : \Phi_Y^* Rf_! E \longrightarrow Rf_! \Phi_X^* E, \qquad \chi_{E'} : \Phi_Y^* Rf_! E' \longrightarrow Rf_! \Phi_X^* E'.$$

No isomorphism, commutative square, inverse, or equivariance equation is part of this definition.

Lemma 14.4.9 (Frobenius equation for the input arrow). *Lean:* `conditional-interface:lemma:weil-a4-fl-input-arrow-frobenius-open`

Uses: `def:weil-a4-fl-arrow-weil-data`
OPEN. *The typed Frobenius maps on* L, L' *satisfy*

$$\varphi_{L'} \circ \Phi_Y^*(v) = v \circ \varphi_L.$$

Proof. Uses: `def:weil-a4-fl-arrow-weil-data,def:weil-a4-fl-arrow-input,def:weil-a4-geometric-frobenius-convention`
This is not a field of Definition 14.4.8. It is exactly the assertion that v is a morphism between the two chosen finite Weil objects. Its construction from descent data and its equality in the finite derived category remain `OBL-A4-FL-WEIL-ARROW`.

Open obligation: `OBL-A4-FL-WEIL-ARROW`

□

Lemma 14.4.10 (Frobenius equation for the source counit). *Lean:* `conditional-interface:lemma:weil-a4-fl-source-counit-frobenius-open`

Uses: `def:weil-a4-fl-arrow-weil-data`
OPEN. *The source counit satisfies*

$$\varphi_L \circ \Phi_Y^*(\epsilon_L) = \epsilon_L \circ Rf_!(\psi_E) \circ \chi_E.$$

Proof. Uses: `def:weil-a4-finite-exceptional-counit,def:weil-a4-geometric-frobenius-convention`
Both sides have source $\Phi_Y^* Rf_! E$ and target L. Identifying the Brown counit with its pullback and proving that the base-change map χ_E transports the identity of E to ψ_E are precisely the unresolved Frobenius-counit calculation `OBL-A4-FL-WEIL-COUNIT-SOURCE`.

Open obligation: `OBL-A4-FL-WEIL-COUNIT-SOURCE`

□

Lemma 14.4.11 (Frobenius equation for the target counit). *Lean:* `conditional-interface:lemma:weil-a4-fl-target-counit-frobenius-open`

Uses: `def:weil-a4-fl-arrow-weil-data`
OPEN. *The target counit satisfies*

$$\varphi_{L'} \circ \Phi_Y^*(\epsilon_{L'}) = \epsilon_{L'} \circ Rf_!(\psi_{E'}) \circ \chi_{E'}.$$

Proof. Uses: def:weil-a4-finite-exceptional-counit,def:weil-a4-geometric-frobenius-convention
This is the preceding source-counit calculation instantiated with L' and $E' = Rf_n^! L'$. It is recorded separately so that no compatibility is hidden in the raw data; its unresolved content is OBL-A4-FL-WEIL-COUNIT-TARGET.

Open obligation: OBL-A4-FL-WEIL-COUNIT-TARGET

□

Lemma 14.4.12 (Frobenius compatibility of the finite exceptional arrow). Lean: conditional-interface:lemma:weil-a4-fl-arrow-frobenius

Uses: def:weil-a4-fl-arrow-weil-data
The square

$$\begin{array}{ccc} \Phi_X^* E & \xrightarrow{\Phi_X^* Rf^!_{n,\mathrm{fl}}(v)} & \Phi_X^* E' \\ {\scriptstyle \psi_E}\downarrow & & \downarrow{\scriptstyle \psi_{E'}} \\ E & \xrightarrow[Rf^!_{n,\mathrm{fl}}(v)]{} & E' \end{array}$$

commutes.

Proof. Uses: lemma:weil-a4-fl-arrow-representing-equation,lemma:weil-a4-fl-input-arrow-frobenius-open,lemma:weil-a4-fl-source-counit-frobenius-open,lemma:weil-a4-fl-target-counit-frobenius-open,prop:weil-a4-finite-representing-right-inverse-open,def:weil-a4-finite-exceptional-backward-map
Apply the backward representing map to the two composites. Substitute the source- and target-counit equations, and then the Frobenius equation for v. Both images become the Frobenius transport of $v\epsilon_L$. The upstream right-inverse equation makes the backward map injective, yielding the square. The exchange of Frobenius pullback with the representing construction, beyond the three displayed component equations, remains OBL-A4-FL-FROBENIUS.

Open obligation: OBL-A4-FL-FROBENIUS

□

Definition 14.4.13 (Raw finite reductions for exceptional arrows). Lean: conditional-interface:def:weil-a4-fl-arrow-reduction-data

Uses: def:weil-a4-fl-arrow-input,def:weil-a2-finite-reduction
For every $m \geq n$, fix a level-m and a level-n instance of Definition 14.4.1, and fix only the four typed maps

$$r^L_{m,n} : L_m \otimes^L_{\Lambda_m} \Lambda_n \longrightarrow L_n, \quad r^{L'}_{m,n} : L'_m \otimes^L_{\Lambda_m} \Lambda_n \longrightarrow L'_n,$$

$$r^E_{m,n} : E_m \otimes^L_{\Lambda_m} \Lambda_n \longrightarrow E_n, \quad r^{E'}_{m,n} : E'_m \otimes^L_{\Lambda_m} \Lambda_n \longrightarrow E'_n.$$

Using the already defined coefficient-change morphism for $Rf_!$, define the reduced-counit maps

$$\bar{\epsilon}^L_{m,n} : Rf_!(E_m \otimes^L \Lambda_n) \longrightarrow L_m \otimes^L \Lambda_n, \quad \bar{\epsilon}^{L'}_{m,n} : Rf_!(E'_m \otimes^L \Lambda_n) \longrightarrow L'_m \otimes^L \Lambda_n$$

by reducing ϵ_{L_m} and $\epsilon_{L'_m}$, respectively. No normalization, transitivity, arrow square, or counit square is part of this raw datum.

Lemma 14.4.14 (Typing of the finite reduction maps). Lean: conditional-interface:lemma:weil-a4-fl-reduction-typing

Uses: def:weil-a4-fl-arrow-reduction-data
Each $r^L_{m,n}, r^{L'}_{m,n}$ lies in $D^b_c(Y, \Lambda_n)$, each $r^E_{m,n}, r^{E'}_{m,n}$ lies in $D^b_c(X, \Lambda_n)$, and the two reduced counits have the displayed source and target in $D^b_c(Y, \Lambda_n)$.

Proof. Uses: def:weil-a2-finite-reduction,def:weil-a4-finite-exceptional-counit
Finite reduction sends a level-m constructible complex to a level-n constructible complex. Applying this once to each of L_m, L'_m, E_m, E'_m gives the four arrow types. Applying the fixed coefficient-change morphism for $Rf_!$ to the two finite counits gives the two reduced-counit types. No equation between these maps is used. □

Lemma 14.4.15 (Identity normalization of the raw reduction maps). Lean: conditional-interface:lemma:weil-a4-fl-reduction-identity-open

Uses: def:weil-a4-fl-arrow-reduction-data
OPEN. *For each $A \in \{L, L', E, E'\}$, the map $r^A_{n,n} : A_n \otimes^L_{\Lambda_n} \Lambda_n \to A_n$ is the canonical tensor-unit identification.*

Proof. Uses: `lemma:weil-a4-fl-reduction-typing`, `def:weil-a2-finite-reduction`

After rewriting the source by the tensor-unit isomorphism, both maps have type $A_n \to A_n$. Showing that the chosen raw reduction equals the identity reduction is not definitional for the present data and remains `OBL-A4-FL-REDUCTION-IDENTITY`.

Open obligation: `OBL-A4-FL-REDUCTION-IDENTITY`

□

Lemma 14.4.16 (Transitivity of the raw reduction maps). *Lean:* `conditional-interface:lemma:weil-a4-fl-reduction-transitivity-open`

Uses: `def:weil-a4-fl-arrow-reduction-data`

OPEN. *For $m \geq n \geq r$ and $A \in \{L, L', E, E'\}$, after inserting the canonical derived-tensor associator,*

$$r^A_{n,r} \circ (r^A_{m,n} \otimes^L_{\Lambda_n} \Lambda_r) = r^A_{m,r}.$$

Proof. Uses: `lemma:weil-a4-fl-reduction-typing`, `def:weil-a2-finite-reduction`

Both sides have source $A_m \otimes^L_{\Lambda_m} \Lambda_r$ and target A_r. Equality is the two-step coefficient-reduction coherence, which is not a field of the raw datum and remains `OBL-A4-FL-REDUCTION-TRANSITIVITY`.

Open obligation: `OBL-A4-FL-REDUCTION-TRANSITIVITY`

□

Lemma 14.4.17 (Reduction equation for the input arrow). *Lean:* `conditional-interface:lemma:weil-a4-fl-input-arrow-reduction-open`

Uses: `def:weil-a4-fl-arrow-reduction-data`

OPEN. *For $m \geq n$,*

$$r^{L'}_{m,n} \circ (v_m \otimes^L \Lambda_n) = v_n \circ r^L_{m,n}.$$

Proof. Uses: `lemma:weil-a4-fl-reduction-typing`, `def:weil-a4-fl-arrow-input`

The two composites are typed maps $L_m \otimes^L_{\Lambda_m} \Lambda_n \to L'_n$. Their equality is the assertion that $(v_n)_n$ is a morphism of coefficient systems; it remains `OBL-A4-FL-REDUCTION-ARROW`.

Open obligation: `OBL-A4-FL-REDUCTION-ARROW`

□

Lemma 14.4.18 (Reduction equation for the source counit). *Lean:* `conditional-interface:lemma:weil-a4-fl-source-counit-reduction-open`

Uses: `def:weil-a4-fl-arrow-reduction-data`

OPEN. *For $m \geq n$,*

$$r^L_{m,n} \circ \bar{\epsilon}^L_{m,n} = \epsilon_{L_n} \circ Rf_!(r^E_{m,n}).$$

Proof. Uses: `def:weil-a4-fl-arrow-reduction-data`, `lemma:weil-a4-fl-reduction-typing`, `def:weil-a4-finite-exceptional-counit`

Both sides are maps from $Rf_!(E_m \otimes^L \Lambda_n)$ to L_n. Their equality is the coefficient-change square for the Brown counit at L, not part of the raw datum; it remains `OBL-A4-FL-REDUCTION-COUNIT-SOURCE`.

Open obligation: `OBL-A4-FL-REDUCTION-COUNIT-SOURCE`

□

Lemma 14.4.19 (Reduction equation for the target counit). *Lean:* `conditional-interface:lemma:weil-a4-fl-target-counit-reduction-open`

Uses: `def:weil-a4-fl-arrow-reduction-data`

OPEN. *For $m \geq n$,*

$$r^{L'}_{m,n} \circ \bar{\epsilon}^{L'}_{m,n} = \epsilon_{L'_n} \circ Rf_!(r^{E'}_{m,n}).$$

Proof. Uses: `def:weil-a4-fl-arrow-reduction-data`, `lemma:weil-a4-fl-reduction-typing`, `def:weil-a4-finite-exceptional-counit`

This is the source-counit reduction equation instantiated with L' and E'. It is stated separately so that no target-counit certificate is hidden in Definition 14.4.13; its unresolved content is `OBL-A4-FL-REDUCTION-COUNIT-TARGET`.

Open obligation: `OBL-A4-FL-REDUCTION-COUNIT-TARGET`

□

Lemma 14.4.20 (Reduction compatibility of the finite exceptional arrow). Lean: `conditional-interface:lemma:weil-a4-fl-arrow-reduction`

Uses: `def:weil-a4-fl-arrow-reduction-data`

OPEN. *The square*

$$\begin{array}{ccc} E_{n+1} \otimes^L \Lambda_n & \xrightarrow{Rf^!_{n+1,\mathrm{fl}}(v_{n+1})\otimes 1} & E'_{n+1} \otimes^L \Lambda_n \\ \downarrow{\scriptstyle r^E_{n+1,n}} & & \downarrow{\scriptstyle r^{E'}_{n+1,n}} \\ E_n & \xrightarrow[Rf^!_{n,\mathrm{fl}}(v_n)]{} & E'_n \end{array}$$

commutes.

Proof. Uses: `lemma:weil-a4-fl-arrow-representing-equation,lemma:weil-a4-fl-input-arrow-reduction-open,lemma:weil-a4-fl-source-counit-reduction-open,lemma:weil-a4-fl-target-counit-reduction-open,prop:weil-a4-finite-representing-right-inverse-open,def:weil-a4-finite-exceptional-backward-map`

Apply the level-n backward representing map to the two composites. Substitute the source- and target-counit reduction equations and then the reduction equation for v. Both images become the reduction of $v_{n+1}\epsilon_{L_{n+1}}$. The upstream right-inverse equation makes the backward map injective. The remaining interchange between reduction, $Rf_!$, and the forward representing construction is the explicit leaf `OBL-A4-FL-REDUCTION`.

Open obligation: `OBL-A4-FL-REDUCTION`

□

14.5 Compact-support pushforward from a dense compactification

Status. All coefficients in this section are $\Lambda_n = \mathbf{Z}/\ell^n\mathbf{Z}$. The A3 finite proper-base-change statement is imported at its existing label and is not reproved here. Consequently every OPEN ancestor of that A3 statement, and the existing A1 leaves for the finite constructible category, exact open extension, proper derived pushforward, and selected-resolution composition, remain inherited hypotheses. The new OPEN leaves introduced below concern only the explicitly identified replacement comparisons and localization naturality. No extraordinary pullback or duality assertion is used.

Definition 14.5.1 (Selected dense compactification). Lean: `conditional-interface:def:weil-a1c-selected-dense-compactification`

Uses: `def:weil-a1-selected-compactification,def:nagata-dense-refinement`

Let (f, C_f^0) be the selected compactification index, with

$$C_f^0 = (X \xrightarrow{j_f^0} \overline{X}_f^0 \xrightarrow{p_f^0} Y).$$

Define C_f to be the dense-refinement construction applied to C_f^0. Write

$$C_f = (X \xrightarrow{j_f} \overline{X}_f \xrightarrow{p_f} Y), \qquad d_f : \overline{X}_f \longrightarrow \overline{X}_f^0$$

for its source open immersion, proper structure map, and refinement map.

Lemma 14.5.2 (Properties of the selected dense compactification). Lean: `conditional-interface:lemma:weil-a1c-selected-dense-compactification`

Uses: `def:weil-a1c-selected-dense-compactification`

The map j_f is a quasi-compact open immersion and is scheme-theoretically dense, p_f is proper, $p_f j_f = f$, and d_f is a proper compactification morphism satisfying $d_f j_f = j_f^0$ and $p_f^0 d_f = p_f$.

Proof. Uses: `def:weil-a1c-selected-dense-compactification,prop:nagata-dense-refinement`

Apply the dense-refinement proposition to C_f^0. Its open-immersion, density, properness, factorization, and refinement fields are respectively the five assertions in the statement. □

Definition 14.5.3 (Compact-support candidate at a dense compactification). Lean: `conditional-interface:def:weil-a1c-compact-support-candidate`

Uses: `def:weil-a1-compactification-candidate`

For a dense compactification $C = (X \xrightarrow{j_C} \overline{X}_C \xrightarrow{p_C} Y)$, define

$$\mathsf{F}_C := Rp_{C*}\, Rj_{C!} : \mathcal{D}_n(X) \longrightarrow \mathcal{D}_n(Y).$$

On a morphism $a : K \to L$, put $\mathsf{F}_C(a) = Rp_{C*}(Rj_{C!}(a))$.

Lemma 14.5.4 (The dense compactification candidate is a functor). Lean: `conditional-interface:lemma:weil-a1c-compact-support-candidate-functor`

Uses: `def:weil-a1c-compact-support-candidate`

For every dense compactification C, the assignment F_C preserves identity morphisms and binary composition.

Proof. Uses: `def:weil-a1c-compact-support-candidate,lemma:weil-a1-compactification-candidate-functorial`

The displayed object and morphism formulas are those of the existing compactification candidate. Apply its identity equation to id_K and its composition equation to $K \xrightarrow{a} L \xrightarrow{b} M$. □

14.5.1 A refinement is the identity over the source open

Definition 14.5.5 (Dense refinement datum). Lean: `conditional-interface:def:weil-a1c-dense-refinement-datum`

Uses: `def:weil-a1-compactification-morphism,def:weil-a1c-compact-support-candidate`

A dense refinement datum $q : D \to C$ consists of dense compactifications D, C of the same morphism $f : X \to Y$, a proper map $q : \overline{X}_D \to \overline{X}_C$, and stored equalities

$$qj_D = j_C, \qquad p_C q = p_D.$$

Definition 14.5.6 (Source-open pullback of a refinement). Lean: `conditional-interface:def:weil-a1c-refinement-source-pullback`

Uses: `def:weil-a1c-dense-refinement-datum`

For $q : D \to C$, define

$$P_q := \overline{X}_D \times_{\overline{X}_C} X.$$

Let $\pi_q : P_q \to X$ and $r_q : P_q \to \overline{X}_D$ be the projections. Define $s_q : X \to P_q$ by the pair (j_D, id_X); the equality field is $qj_D = j_C$.

Lemma 14.5.7 (The source-open pullback projection is proper). Lean: `conditional-interface:lemma:weil-a1c-refinement-source-proper`

Uses: `def:weil-a1c-refinement-source-pullback`

The projection $\pi_q : P_q \to X$ is proper.

Proof. Uses: `def:weil-a1c-refinement-source-pullback,def:weil-a1c-dense-refinement-datum`

The defining square for P_q is Cartesian, and π_q is the base change of the proper map q along j_C. Proper morphisms are stable under base change. □

Lemma 14.5.8 (The source section is an open immersion). Lean: `conditional-interface:lemma:weil-a1c-refinement-source-open`

Uses: `def:weil-a1c-refinement-source-pullback`

The map $s_q : X \to P_q$ is an open immersion.

Proof. Uses: `def:weil-a1c-refinement-source-pullback,def:weil-a1c-dense-refinement-datum`

The map $r_q s_q = j_D$ is an open immersion. The map s_q identifies X with the inverse image under r_q of the open subscheme $j_D(X)$, because a point of P_q over that inverse image is uniquely determined by its X-coordinate and the monomorphism j_D. Hence s_q is the corresponding open immersion. □

Lemma 14.5.9 (The source section is a closed immersion). Lean: `conditional-interface:lemma:weil-a1c-refinement-source-closed`

Uses: `def:weil-a1c-refinement-source-pullback`

The map $s_q : X \to P_q$ is a closed immersion.

Proof. Uses: `lemma:weil-a1c-refinement-source-proper,def:weil-a1c-refinement-source-pullback`

The equality $\pi_q s_q = \mathrm{id}_X$ makes s_q a section of the proper morphism π_q. A proper morphism is separated, and a section of a separated morphism is a closed immersion. □

Lemma 14.5.10 (The source section is scheme-theoretically dense). Lean: `conditional-interface:lemma:weil-a1c-refinement-source-dense`

Uses: `def:weil-a1c-refinement-source-pullback`

The map $s_q : X \to P_q$ is scheme-theoretically dense.

Proof. Uses: `def:weil-a1c-refinement-source-pullback,def:weil-a1c-dense-refinement-datum`

The map $r_q : P_q \to \overline{X}_D$ is an open immersion, since it is the base change of j_C. Restrict the injection $\mathcal{O}_{\overline{X}_D} \to j_{D*}\mathcal{O}_X$, supplied by the scheme-theoretic density of j_D, to the open subscheme P_q. The resulting map is $\mathcal{O}_{P_q} \to s_{q*}\mathcal{O}_X$, so it is injective. □

Proposition 14.5.11 (A dense refinement is an isomorphism over the source). Lean: `conditional-interface:prop:weil-a1c-refinement-source-isomorphism`

Uses: `def:weil-a1c-refinement-source-pullback`

The section $s_q : X \to P_q$ and projection $\pi_q : P_q \to X$ are mutually inverse isomorphisms.

Proof. Uses: `lemma:weil-a1c-refinement-source-open,lemma:weil-a1c-refinement-source-closed,lemma:weil-a1c-refinement-source-dense,def:weil-a1c-refinement-source-pullback`

The open immersion s_q has open image, and the closed-immersion lemma makes the same image closed. Let $I \subseteq \mathcal{O}_{P_q}$ be its defining ideal. Scheme-theoretic density says that $\mathcal{O}_{P_q} \to s_{q*}\mathcal{O}_X = \mathcal{O}_{P_q}/I$ is injective, so $I = 0$. Thus the closed image is all of P_q, and s_q is an isomorphism. The equality $\pi_q s_q = \mathrm{id}_X$ identifies π_q with its inverse. □

14.5.2 Comparison along a dense refinement

Definition 14.5.12 (Forward refinement comparison). Lean: `conditional-interface:def:weil-a1c-refinement-forward`

Uses: `def:weil-a1c-dense-refinement-datum,def:weil-a1-compactification-comparison-map`

For $q : D \to C$, define

$$\rho_q : \mathsf{F}_D \longrightarrow \mathsf{F}_C$$

to be the existing compactification comparison: first use the derived composition map $Rp_{C*}Rq_* \to R(p_C q)_* = Rp_{D*}$, in the direction fixed by that comparison datum, and then apply Rp_{C*} to the support-extension map $Rq_*Rj_{D!} \to Rj_{C!}$.

Definition 14.5.13 (Raw reverse support transport). Lean: `conditional-interface:def:weil-a1c-reverse-support-raw`

Uses: `def:weil-a1c-dense-refinement-datum,def:weil-a1c-refinement-source-pullback,def:weil-a1-derived-extension-zero`

Let $K^\bullet$ be a bounded complex representing K, and let $V \to \overline{X}_C$ be a pro-etale object. For a local representative t of a section of $j_{C!}K^r$ on V, define $\bar{\gamma}^r_{q,V}(t)$ by pulling t to $V_D := V \times_{\overline{X}_C} \overline{X}_D$, transporting its coefficient through

$$V_D \times_{\overline{X}_D} X \simeq V \times_{\overline{X}_C} X,$$

and using the same coefficient section on this open subscheme of V_D, with zero value on its complement.

Lemma 14.5.14 (Raw reverse support has the required target). Lean: `conditional-interface:lemma:weil-a1c-reverse-support-target`

Uses: `def:weil-a1c-reverse-support-raw`

The section $\bar{\gamma}^r_{q,V}(t)$ has support closed in V_D and contained in $V_D \times_{\overline{X}_D} X$; hence it is a section of $j_{D!}K^r$ on V_D.

Proof. Uses: `def:weil-a1c-reverse-support-raw,prop:weil-a1c-refinement-source-isomorphism`

The support of t is closed in V and contained in $V \times_{\overline{X}_C} X$. Its pullback to V_D is closed. The source-isomorphism proposition identifies the pullback of the source open with $V_D \times_{\overline{X}_D} X$, so the pulled-back support is contained in that open. This is exactly the support condition in the definition of extension by zero. □

Lemma 14.5.15 (Raw reverse support is independent of a local representative). Lean: `conditional-interface:lemma:weil-a1c-reverse-support-representative`

Uses: `def:weil-a1c-reverse-support-raw`

If two local representatives determine the same section of $j_{C!}K^r(V)$, their raw reverse transports determine the same section of $j_{D!}K^r(V_D)$.

Proof. Uses: def:weil-a1c-reverse-support-raw

There is a pro-etale covering of V on which the two representatives are equal. Pulling that covering back to V_D gives a covering on which the two transported coefficient sections are equal. Both are zero off the same source-open pullback. The separatedness axiom for the sheaf $j_{D!}K^r$ identifies the two resulting sections. □

Lemma 14.5.16 (Raw reverse support commutes with restriction). Lean: conditional-interface:lemma:weil-a1c-reverse-support-restriction

Uses: def:weil-a1c-reverse-support-raw

For an arrow $a : V' \to V$,

$$a_D^* \bar{\gamma}^r_{q,V}(t) = \bar{\gamma}^r_{q,V'}(a^* t), \qquad a_D : V'_D \to V_D.$$

Proof. Uses: def:weil-a1c-reverse-support-raw,lemma:weil-a1c-reverse-support-representative

Both sides pull the coefficient of t through the same iterated fiber product $V' \times_{\overline{X}_C} \overline{X}_D \times_{\overline{X}_D} X$. Associativity of fiber products identifies the source opens, and restriction of a zero section is zero. The representative-independence lemma then gives equality of the sheaf sections. □

Lemma 14.5.17 (Raw reverse support commutes with the differential). Lean: conditional-interface:lemma:weil-a1c-reverse-support-differential

Uses: def:weil-a1c-reverse-support-raw

For every degree r,

$$j_{D!}(d^r_K)\, \bar{\gamma}^r_{q,V} = \bar{\gamma}^{r+1}_{q,V}\, j_{C!}(d^r_K).$$

Proof. Uses: def:weil-a1c-reverse-support-raw

On the source-open pullback, both sides apply the coefficient differential d^r_K to the same pulled-back section. On the complement, both sides are zero. These two formulas agree on every local representative, hence give equal sheaf morphisms. □

Definition 14.5.18 (Reverse support chain map). Lean: conditional-interface:def:weil-a1c-reverse-support-chain-map

Uses: def:weil-a1c-reverse-support-raw,def:weil-a1-derived-extension-zero

Define

$$\bar{\gamma}^{\mathrm{cx}}_q : Rj_{C!}K \longrightarrow q_* Rj_{D!}K$$

degreewise by the maps $\bar{\gamma}^r_{q,V}$, and define its value on a chain map $K^\bullet \to L^\bullet$ by applying the same pullback formula to each component.

Lemma 14.5.19 (Typing and naturality of the reverse support chain map). Lean: conditional-interface:lemma:weil-a1c-reverse-support-chain-map

Uses: def:weil-a1c-reverse-support-chain-map

The displayed degreewise maps form a chain map of sheaf complexes and are natural in $K^\bullet$.

Proof. Uses: lemma:weil-a1c-reverse-support-target,lemma:weil-a1c-reverse-support-representative,lemma:weil-a1c-reverse-support-restriction,lemma:weil-a1c-reverse-support-differential,def:weil-a1c-reverse-support-chain-map

The target lemma types each component. Representative independence and restriction compatibility make it a sheaf morphism. Differential compatibility is the chain-map equation. For a chain map $a : K \to L$, both naturality paths pull back a local coefficient and then apply a^r, so they are equal in every degree. □

Definition 14.5.20 (Proper-composition normal form). Lean: conditional-interface:def:weil-a1c-proper-composition-normal-form

Uses: def:weil-a1c-dense-refinement-datum,def:weil-a1-proper-direct-image-input

For the proper maps $\overline{X}_D \xrightarrow{q} \overline{X}_C \xrightarrow{p_C} Y$ and a bounded complex A on $\overline{X}_D$, let $I_D(A)$ be the fixed K-injective replacement. Define

$$\mathsf{N}_{p_C,q}(A) := p_{C*} q_* I_D(A).$$

The equality of underived functors $p_{C*}q_* = (p_C q)_*$ identifies the same complex with $(p_C q)_* I_D(A)$.

Definition 14.5.21 (One-step raw normal-form roofs). Lean: conditional-interface:def:weil-a1c-proper-one-step-raw-roofs

Uses: def:weil-a1c-proper-composition-normal-form,def:weil-a1-proper-direct-image-input

The one-step derived image $R(p_C q)_* A$ and $\mathsf{N}_{p_C,q}(A)$ are both represented by the same complex $(p_C q)_* I_D(A) = p_{C*} q_* I_D(A)$. Define

$$\eta^{\mathrm{one}}_{p_C,q,A} : R(p_C q)_* A \longrightarrow \mathsf{N}_{p_C,q}(A), \qquad \theta^{\mathrm{one}}_{p_C,q,A} : \mathsf{N}_{p_C,q}(A) \longrightarrow R(p_C q)_* A$$

to be the identity chain map on this common representative, written with the two source and target aliases.

Lemma 14.5.22 (Proper pushforward preserves K-injective complexes). *Lean:* `conditional-interface:lemma:weil-a1c-proper-pushforward-kinjective`

Uses: `def:weil-a1c-proper-composition-normal-form`

If I is K-injective on $\overline{X}_D$, then q_*I is K-injective on $\overline{X}_C$.

Proof. Uses: `lemma:weil-ftdc-pullback-exact,def:weil-a1c-proper-composition-normal-form`

For an acyclic complex B on $\overline{X}_C$, adjunction gives

$$\mathrm{Hom}(B, q_*I) = \mathrm{Hom}(q^*B, I).$$

Exactness of q^* makes q^*B acyclic. K-injectivity of I makes the right Hom complex acyclic, hence so is the left Hom complex. This is the defining K-injectivity test for q_*I. □

Definition 14.5.23 (Two-step replacement arrow in the homotopy category). Lean: `conditional-interface:def:weil-a1c-proper-two-step-replacement-arrow`

Uses: `def:weil-a1c-proper-composition-normal-form,def:weil-a1-proper-direct-image-input`

Put

$$I_q(A) := q_*I_D(A), \qquad J_{p,q}(A) := I_C(I_q(A)).$$

Let $u_{p,q,A} : I_q(A) \to J_{p,q}(A)$ be the replacement arrow supplied by the fixed functorial K-injective resolution on $\overline{X}_C$. If Q_h denotes passage to the homotopy category, define

$$v_{p,q,A} := Q_h(u_{p,q,A}) : Q_hI_q(A) \longrightarrow Q_hJ_{p,q}(A).$$

No inverse representative or homotopy is selected here.

Definition 14.5.24 (Precomposition on homotopy classes). Lean: `conditional-interface:def:weil-a1c-kinjective-precomposition-map`

Uses: `def:weil-a1c-proper-two-step-replacement-arrow`

For a quasi-isomorphism $s : M \to N$ of complexes and a complex T, define

$$s_T^* : \mathrm{Hom}_K(N, T) \longrightarrow \mathrm{Hom}_K(M, T), \qquad [a] \longmapsto [a]\,[s].$$

Lemma 14.5.25 (K-injective precomposition is bijective). *Lean:* `conditional-interface:lemma:weil-a1c-kinjective-precomposition-bijective`

Uses: `def:weil-a1c-kinjective-precomposition-map`

If T is K-injective, then s_T^* is bijective.

Proof. Uses: `def:weil-a1c-kinjective-precomposition-map,def:weil-a1-proper-direct-image-input`

The cone $C(s)$ is acyclic because s is a quasi-isomorphism. Apply $\mathrm{Hom}_K(-, T)$ to the triangle

$$M \xrightarrow{s} N \longrightarrow C(s) \longrightarrow M[1].$$

K-injectivity gives $\mathrm{Hom}_K(C(s), T) = 0$ and $\mathrm{Hom}_K(C(s)[-1], T) = 0$. The exact segment between these two zero groups is

$$0 \longrightarrow \mathrm{Hom}_K(N, T) \xrightarrow{s_T^*} \mathrm{Hom}_K(M, T) \longrightarrow 0.$$

Thus s_T^* is injective and surjective. □

Lemma 14.5.26 (The two-step source is K-injective). *Lean:* `conditional-interface:lemma:weil-a1c-proper-two-step-source-kinjective`

Uses: `def:weil-a1c-proper-two-step-replacement-arrow`

The complex $I_q(A) = q_*I_D(A)$ is K-injective on $\overline{X}_C$.

Proof. Uses: `lemma:weil-a1c-proper-pushforward-kinjective,def:weil-a1c-proper-two-step-replacement-arrow`

The complex $I_D(A)$ is K-injective by its definition as the fixed replacement of A. Apply the proper-pushforward K-injectivity lemma to q and $I_D(A)$. □

Definition 14.5.27 (Left inverse homotopy class of the replacement arrow). Lean: `conditional-interface:def:weil-a1c-proper-two-step-left-inverse-class`

Uses: `def:weil-a1c-kinjective-precomposition-map,def:weil-a1c-proper-two-step-replacement-arrow`

Use the inverse function of $(u_{p,q,A})^*_{I_q(A)}$ to define

$$\lambda_{p,q,A} := \big((u_{p,q,A})^*_{I_q(A)}\big)^{-1}\big(\mathrm{id}_{Q_hI_q(A)}\big) \in \mathrm{Hom}_{K(\overline{X}_C)}\big(Q_hJ_{p,q}(A), Q_hI_q(A)\big).$$

This Definition selects a morphism of the homotopy category, not a chain map or a homotopy.

Lemma 14.5.28 (Left inverse equation in the homotopy category)**.** Lean: `conditional-interface:lemma:weil-a1c-proper-two-step-homotopy-left-inverse`

Uses: `def:weil-a1c-proper-two-step-left-inverse-class`, `def:weil-a1c-proper-two-step-replacement-arrow`

One has

$$\lambda_{p,q,A}\upsilon_{p,q,A} = \mathrm{id}_{Q_h I_q(A)}\,.$$

Proof. Uses: `lemma:weil-a1c-kinjective-precomposition-bijective`, `lemma:weil-a1c-proper-two-step-source-kinjective`, `def:weil-a1c-proper-two-step-left-inverse-class`, `def:weil-a1c-proper-two-step-replacement-arrow`

The left-hand side is $(u_{p,q,A})^*_{I_q(A)}(\lambda_{p,q,A})$. The defining inverse-function evaluation gives the displayed identity. □

Lemma 14.5.29 (Right inverse equation in the homotopy category)**.** Lean: `conditional-interface:lemma:weil-a1c-proper-two-step-homotopy-right-inverse`

Uses: `def:weil-a1c-proper-two-step-left-inverse-class`, `def:weil-a1c-proper-two-step-replacement-arrow`

One has

$$\upsilon_{p,q,A}\lambda_{p,q,A} = \mathrm{id}_{Q_h J_{p,q}(A)}\,.$$

Proof. Uses: `lemma:weil-a1c-kinjective-precomposition-bijective`, `lemma:weil-a1c-proper-two-step-source-kinjective`, `lemma:weil-a1c-proper-two-step-homotopy-left-inverse`, `def:weil-a1c-proper-two-step-left-inverse-class`, `def:weil-a1c-proper-two-step-replacement-arrow`

The source $I_q(A) = q_* I_D(A)$ is K-injective by the source lemma, and $J_{p,q}(A)$ is K-injective by construction. Apply the bijectivity lemma to $u_{p,q,A}$ with target $J_{p,q}(A)$. It suffices to compare the two endomorphisms after precomposition with $\upsilon_{p,q,A}$. For the first one,

$$(\upsilon\lambda)\upsilon = \upsilon(\lambda\upsilon) = \upsilon$$

by the left-inverse lemma; for the identity endomorphism the result is also υ. Injectivity of the precomposition map gives the equation. □

Lemma 14.5.30 (The replacement arrow is invertible in the homotopy category)**.** Lean: `conditional-interface:lemma:weil-a1c-proper-two-step-homotopy-isomorphism`

Uses: `def:weil-a1c-proper-two-step-replacement-arrow`

The morphism $\upsilon_{p,q,A}$ is an isomorphism in $K(\overline{X}_C)$.

Proof. Uses: `lemma:weil-a1c-proper-two-step-homotopy-left-inverse`, `lemma:weil-a1c-proper-two-step-homotopy-right-inverse`, `def:weil-a1c-proper-two-step-left-inverse-class`, `def:weil-a1c-proper-two-step-replacement-arrow`

Use $\lambda_{p,q,A}$ as inverse. The preceding two lemmas are its two inverse equations. □

Definition 14.5.31 (Proper pushforward on homotopy categories)**.** Lean: `conditional-interface:def:weil-a1c-proper-homotopy-pushforward`

Uses: `def:weil-a1-proper-direct-image-input`

For a proper map $p_C : \overline{X}_C \to Y$, define

$$K(p_{C*}) : K(\overline{X}_C) \longrightarrow K(Y)$$

by applying p_{C*} to every term, differential, and component of a chain map.

Lemma 14.5.32 (Proper pushforward respects chain homotopy)**.** Lean: `conditional-interface:lemma:weil-a1c-proper-homotopy-pushforward`

Uses: `def:weil-a1c-proper-homotopy-pushforward`

The displayed assignment is a functor on homotopy categories.

Proof. Uses: `def:weil-a1c-proper-homotopy-pushforward`

If $a - b = dh + hd$, additivity of p_{C*} gives

$$p_{C*}(a) - p_{C*}(b) = d\,p_{C*}(h) + p_{C*}(h)d.$$

Thus homotopic chain maps have homotopic images. Termwise application preserves identity maps and composition, which proves functoriality. □

Definition 14.5.33 (Localized two-step comparison arrow)**.** Lean: `conditional-interface:def:weil-a1c-proper-two-step-localized-arrow`

Uses: `def:weil-a1c-proper-composition-normal-form`, `def:weil-a1c-proper-two-step-replacement-arrow`, `def:weil-a1c-proper-homotopy-pushforward`

Let $Q_Y : K(Y) \to D(Y)$ be the localization functor. Define

$$\mathcal{U}_{p,q,A} := Q_Y\big(K(p_{C*})(\upsilon_{p,q,A})\big) : \mathsf{N}_{p_C,q}(A) \longrightarrow Rp_{C*}Rq_*A,$$

where the target is represented by $p_{C*}J_{p,q}(A)$.

Lemma 14.5.34 (The localized two-step arrow is an isomorphism)**.** Lean: `conditional-interface:lemma:weil-a1c-proper-two-step-localized-isomorphism`

Uses: `def:weil-a1c-proper-two-step-localized-arrow`

The morphism $\mathcal{U}_{p,q,A}$ is an isomorphism in $D(Y)$.

Proof. Uses: `lemma:weil-a1c-proper-two-step-homotopy-isomorphism`, `lemma:weil-a1c-proper-homotopy-pushforward`, `def:weil-a1c-proper-two-step-localized-arrow`

The homotopy-category morphism $\upsilon_{p,q,A}$ is an isomorphism. The functor $K(p_{C*})$ sends it to an isomorphism in $K(Y)$, and the localization functor Q_Y sends that isomorphism to an isomorphism in $D(Y)$. This image is $\mathcal{U}_{p,q,A}$. □

Definition 14.5.35 (Inverse of the localized two-step arrow)**.** Lean: `conditional-interface:def:weil-a1c-proper-two-step-localized-inverse`

Uses: `def:weil-a1c-proper-two-step-localized-arrow`

Define

$$\mathcal{V}_{p,q,A} := \mathcal{U}_{p,q,A}^{-1} : Rp_{C*}Rq_*A \longrightarrow \mathsf{N}_{p_C,q}(A)$$

using the inverse operation for isomorphisms in $D(Y)$.

Definition 14.5.36 (Four normal-form comparison maps)**.** Lean: `conditional-interface:def:weil-a1c-proper-normal-form-comparison-maps`

Uses: `def:weil-a1c-proper-one-step-raw-roofs`, `def:weil-a1c-proper-two-step-localized-arrow`, `def:weil-a1c-proper-two-step-localized-inverse`

Define

$$\eta^{\mathrm{one}} := \eta^{\mathrm{one}}_{p_C,q,A}, \qquad \theta^{\mathrm{one}} := \theta^{\mathrm{one}}_{p_C,q,A},$$
$$\eta^{\mathrm{two}} := \mathcal{V}_{p,q,A}, \qquad \theta^{\mathrm{two}} := \mathcal{U}_{p,q,A}.$$

Lemma 14.5.37 (One-step left inverse)**.** Lean: `conditional-interface:lemma:weil-a1c-proper-one-step-left-inverse`

Uses: `def:weil-a1c-proper-one-step-raw-roofs`

One has $\theta^{\mathrm{one}}\eta^{\mathrm{one}} = \mathrm{id}_{R(p_Cq)_*A}$.

Proof. Uses: `def:weil-a1c-proper-one-step-raw-roofs`

Both factors are the identity chain map of $(p_Cq)_*I_D(A) = p_{C*}q_*I_D(A)$. Their composite is the identity chain map, and localization preserves that equation. □

Lemma 14.5.38 (One-step right inverse)**.** Lean: `conditional-interface:lemma:weil-a1c-proper-one-step-right-inverse`

Uses: `def:weil-a1c-proper-one-step-raw-roofs`

One has $\eta^{\mathrm{one}}\theta^{\mathrm{one}} = \mathrm{id}_{\mathsf{N}_{p_C,q}(A)}$.

Proof. Uses: `def:weil-a1c-proper-one-step-raw-roofs`

This is the opposite composite of the same identity chain map with itself. □

Lemma 14.5.39 (Two-step left inverse)**.** Lean: `conditional-interface:lemma:weil-a1c-proper-two-step-left-inverse`

Uses: `def:weil-a1c-proper-two-step-localized-arrow`, `def:weil-a1c-proper-two-step-localized-inverse`

One has

$$\mathcal{V}_{p,q,A}\mathcal{U}_{p,q,A} = \mathrm{id}_{\mathsf{N}_{p_C,q}(A)}.$$

Proof. Uses: lemma:weil-a1c-proper-two-step-localized-isomorphism,def:weil-a1c-proper-two-step-localized-arrow,def:weil-a1c-proper-two-step-localized-inverse
This is the inverse-followed-by-map equation for the isomorphism $\mathcal{U}_{p,q,A}$ in $D(Y)$. □

Lemma 14.5.40 (Two-step right inverse). Lean: conditional-interface:lemma:weil-a1c-proper-two-step-right-inverse

Uses: def:weil-a1c-proper-two-step-localized-arrow, def:weil-a1c-proper-two-step-localized-inverse
One has

$$\mathcal{U}_{p,q,A}\mathcal{V}_{p,q,A} = \mathrm{id}_{Rp_{C*}Rq_*A}.$$

Proof. Uses: lemma:weil-a1c-proper-two-step-localized-isomorphism,def:weil-a1c-proper-two-step-localized-arrow,def:weil-a1c-proper-two-step-localized-inverse
This is the map-followed-by-inverse equation for $\mathcal{U}_{p,q,A}$. □

Lemma 14.5.41 (Naturality of the normal-form comparison maps). Lean: conditional-interface:lemma:weil-a1c-proper-normal-form-naturality

Uses: def:weil-a1c-proper-normal-form-comparison-maps
For a chain map $a : A \to A'$, the four normal-form maps commute with the morphisms induced by a in the derived category.

Proof. Uses: lemma:weil-a1c-proper-two-step-left-inverse,lemma:weil-a1c-proper-two-step-right-inverse,lemma:weil-a1c-proper-homotopy-pushforward,def:weil-a1c-proper-normal-form-comparison-maps
The one-step maps are identities. Functoriality of the resolution arrow gives

$$B(a)\mathcal{U}_A = \mathcal{U}_{A'}N(a).$$

For the inverse maps, insert the two inverse equations:

$$N(a)\mathcal{V}_A = \mathcal{V}_{A'}\mathcal{U}_{A'}N(a)\mathcal{V}_A = \mathcal{V}_{A'}B(a)\mathcal{U}_A\mathcal{V}_A = \mathcal{V}_{A'}B(a).$$

This is the required square for $\mathcal{V}$. □

Lemma 14.5.42 (Uniqueness of the two-step inverse). Lean: conditional-interface:lemma:weil-a1c-proper-normal-form-lift-independent

Uses: def:weil-a1c-proper-two-step-localized-arrow, def:weil-a1c-proper-two-step-localized-inverse
If $\xi : Rp_{C*}Rq_*A \to \mathsf{N}_{p_C,q}(A)$ satisfies $\xi\mathcal{U}_{p,q,A} = \mathrm{id}$, then $\xi = \mathcal{V}_{p,q,A}$.

Proof. Uses: lemma:weil-a1c-proper-two-step-right-inverse,def:weil-a1c-proper-two-step-localized-arrow,def:weil-a1c-proper-two-step-localized-inverse
Multiply the assumed equation on the right by $\mathcal{V}_{p,q,A}$:

$$\xi = \xi\mathcal{U}\mathcal{V} = \mathcal{V}.$$

The first equality uses $\mathcal{U}\mathcal{V} = \mathrm{id}$. □

Lemma 14.5.43 (The normal form computes both proper derived images). Lean: conditional-interface:lemma:weil-a1c-proper-composition-normal-form-computes

Uses: def:weil-a1c-proper-composition-normal-form, def:weil-a1c-proper-normal-form-comparison-maps
The four comparison maps satisfy

$$\theta^{\mathrm{one}}\eta^{\mathrm{one}} = \mathrm{id}, \quad \eta^{\mathrm{one}}\theta^{\mathrm{one}} = \mathrm{id}, \quad \theta^{\mathrm{two}}\eta^{\mathrm{two}} = \mathrm{id}, \quad \eta^{\mathrm{two}}\theta^{\mathrm{two}} = \mathrm{id},$$

and all four maps are natural in A. Thus either pair identifies its derived direct image with $\mathsf{N}_{p_C,q}(A)$.

Proof. Uses: lemma:weil-a1c-proper-one-step-left-inverse,lemma:weil-a1c-proper-one-step-right-inverse,lemma:weil-a1c-proper-two-step-left-inverse,lemma:weil-a1c-proper-two-step-right-inverse,lemma:weil-a1c-proper-normal-form-naturality,lemma:weil-a1c-proper-normal-form-lift-independent
The first four cited lemmas are the four displayed inverse equations. The fifth is the naturality equation. The sixth is uniqueness of the inverse of $\mathcal{U}$. These statements give each clause of the conclusion. □

Definition 14.5.44 (Forward and reverse proper-composition maps). Lean: `conditional-interface:def:weil-a1c-proper-composition-two-way`

Uses: `def:weil-a1c-proper-normal-form-comparison-maps`

Using the two maps to the common normal form, define

$$\Sigma_{p_C,q} := \theta^{\mathrm{one}}\eta^{\mathrm{two}} : Rp_{C*}Rq_* \longrightarrow R(p_Cq)_*, \qquad T_{p_C,q} := \theta^{\mathrm{two}}\eta^{\mathrm{one}}.$$

Lemma 14.5.45 (Proper-composition inverse equations). Lean: *`conditional-interface:lemma:weil-a1c-proper-composition-inverses`*

Uses: *`def:weil-a1c-proper-composition-two-way`*

The maps satisfy $T_{p_C,q}\Sigma_{p_C,q} = \mathrm{id}$ *and* $\Sigma_{p_C,q}T_{p_C,q} = \mathrm{id}$.

Proof. Uses: `def:weil-a1c-proper-composition-two-way,lemma:weil-a1c-proper-composition-normal-form-computes`

Expand both definitions. In the first composite, cancel $\eta^{\mathrm{one}}\theta^{\mathrm{one}}$ and then $\theta^{\mathrm{two}}\eta^{\mathrm{two}}$. The second composite uses $\eta^{\mathrm{two}}\theta^{\mathrm{two}}$ and $\theta^{\mathrm{one}}\eta^{\mathrm{one}}$. □

Lemma 14.5.46 (The forward refinement comparison is an isomorphism). Lean: *`conditional-interface:lemma:weil-a1c-refinement-forward-isomorphism`*

Uses: *`def:weil-a1c-refinement-forward`*

For every K, *the morphism* $\rho_q(K) : \mathsf{F}_D(K) \to \mathsf{F}_C(K)$ *is an isomorphism.*

Proof. Uses: `lemma:weil-a1-compactification-comparison-isomorphism,def:weil-a1c-refinement-forward`

The definition of ρ_q is the compactification comparison associated to the compactification morphism $q : D \to C$. Apply the cited compactification-comparison isomorphism to q and K. □

Definition 14.5.47 (Derived reverse refinement map). Lean: `conditional-interface:def:weil-a1c-refinement-reverse`

Uses: `def:weil-a1c-refinement-forward`

For a dense refinement $q : D \to C$, define

$$\bar{\rho}_q(K) := \rho_q(K)^{-1} : \mathsf{F}_C(K) \longrightarrow \mathsf{F}_D(K)$$

in the finite constructible derived category.

Lemma 14.5.48 (Naturality and resolution independence of the reverse refinement). Lean: *`conditional-interface:lemma:weil-a1c-refinement-reverse-natural`*

Uses: *`def:weil-a1c-refinement-reverse`*

For every morphism $a : K \to L$,

$$\mathsf{F}_D(a)\bar{\rho}_q(K) = \bar{\rho}_q(L)\mathsf{F}_C(a).$$

Consequently $\bar{\rho}_q$ *depends only on the derived morphism* a, *not on a complex representative or resolution map.*

Proof. Uses: `lemma:weil-a1c-refinement-forward-isomorphism,lemma:weil-a1-compactification-candidate-functorial,def:weil-a1c-refinement-forward,def:weil-a1c-refinement-reverse`

Naturality of ρ_q gives

$$\mathsf{F}_C(a)\rho_q(K) = \rho_q(L)\mathsf{F}_D(a).$$

Compose on the left with $\bar{\rho}_q(L)$ and on the right with $\bar{\rho}_q(K)$. The two inverse equations reduce the result to the displayed equality. Every operation in this calculation occurs in the derived category, so changing a chain representative or a resolution roof does not change either side. □

Lemma 14.5.49 (Interior stalk equation for refinement comparison). Lean: *`conditional-interface:lemma:weil-a1c-refinement-interior-stalk`*

Uses: *`def:weil-a1c-refinement-forward`*, *`def:weil-a1c-refinement-reverse`*

For a geometric point $\bar{x} \to \overline{X}_C$ *factoring through* $j_C(X)$, *the stalks of the support-extension factors in* $\rho_q\bar{\rho}_q$ *and* $\bar{\rho}_q\rho_q$ *are identity maps.*

Proof. Uses: `prop:weil-a1c-refinement-source-isomorphism,lemma:weil-a1-extension-zero-open-restriction,def:weil-a1c-refinement-forward,def:weil-a1c-refinement-reverse`

Pull the refinement square to X. The source-isomorphism proposition identifies it with the identity square of X. Both extensions by zero restrict to the original complex by the open-restriction lemma. Under these identifications the forward and reverse local formulas both act as the identity on the same stalk complex, so both composites are identities. □

Definition 14.5.50 (Boundary fiber of a refinement). Lean: `conditional-interface:def:weil-a1c-refinement-boundary-fiber`

Uses: `def:weil-a1c-dense-refinement-datum`

For a geometric point $\bar{z} \to \overline{X}_C \setminus j_C(X)$, define

$$D_{\bar{z}} := \overline{X}_D \times_{\overline{X}_C} \bar{z}$$

and let $q_{\bar{z}} : D_{\bar{z}} \to \bar{z}$ be its structure map.

Lemma 14.5.51 (The boundary fiber misses the source open). Lean: `conditional-interface:lemma:weil-a1c-refinement-boundary-disjoint`

Uses: `def:weil-a1c-refinement-boundary-fiber`

The fiber product $D_{\bar{z}} \times_{\overline{X}_D} X$ is empty.

Proof. Uses: `def:weil-a1c-refinement-boundary-fiber,def:weil-a1c-dense-refinement-datum`

If a morphism factored through this fiber product, its composite with $qj_D = j_C$ would factor the boundary point $\bar{z}$ through $j_C(X)$, contrary to the definition of the complement. The universal property of the fiber product therefore identifies it with the empty scheme. □

Lemma 14.5.52 (The extended coefficient vanishes on a boundary fiber). Lean: `conditional-interface:lemma:weil-a1c-refinement-boundary-coefficient-zero`

Uses: `def:weil-a1c-refinement-boundary-fiber,def:weil-a1-derived-extension-zero`

For $K \in \mathcal{D}_n(X)$, the pullback of $Rj_{D!}K$ to $D_{\bar{z}}$ is the zero complex.

Proof. Uses: `lemma:weil-a1c-refinement-boundary-disjoint,lemma:weil-a1-extension-zero-closed-restriction,def:weil-a1-derived-extension-zero`

The preceding lemma puts the whole fiber in the closed complement of the source open. Termwise extension by zero has zero restriction to that complement. Applying this degree by degree to a representative of K gives the zero complex on $D_{\bar{z}}$. □

Lemma 14.5.53 (Proper-base-change boundary stalk). Lean: `conditional-interface:lemma:weil-a1c-refinement-boundary-stalk-zero`

Uses: `def:weil-a1c-refinement-boundary-fiber,def:weil-a1c-refinement-forward`

For a boundary geometric point $\bar{z}$, the stalk of $Rq_*Rj_{D!}K$ at $\bar{z}$ is zero.

Proof. Uses: `prop:weil-proet-finite-proper-base-change,lemma:weil-a1c-refinement-boundary-coefficient-zero,def:weil-a1c-refinement-boundary-fiber`

Apply finite proper base change to the Cartesian square defining $D_{\bar{z}}$. It identifies the stalk with $R\Gamma(D_{\bar{z}}, Rj_{D!}K|_{D_{\bar{z}}})$. The coefficient is the zero complex by the preceding lemma, and derived global sections of the zero complex are zero. □

Lemma 14.5.54 (Forward after reverse is the identity). Lean: `conditional-interface:lemma:weil-a1c-refinement-resolution-left-inverse`

Uses: `def:weil-a1c-refinement-forward,def:weil-a1c-refinement-reverse`

For every K,

$$\rho_q(K)\bar{\rho}_q(K) = \mathrm{id}_{\mathsf{F}_{C(K)}}.$$

Proof. Uses: `lemma:weil-a1c-refinement-forward-isomorphism,def:weil-a1c-refinement-reverse`

By definition, $\bar{\rho}_q(K)$ is the inverse of $\rho_q(K)$. The displayed equality is the map-followed-by-inverse equation. □

Lemma 14.5.55 (Reverse after forward is the identity). Lean: `conditional-interface:lemma:weil-a1c-refinement-resolution-right-inverse`

Uses: `def:weil-a1c-refinement-forward,def:weil-a1c-refinement-reverse`

For every K,

$$\bar{\rho}_q(K)\rho_q(K) = \mathrm{id}_{\mathsf{F}_{D(K)}}.$$

Proof. Uses: `lemma:weil-a1c-refinement-forward-isomorphism,def:weil-a1c-refinement-reverse`

This is the inverse-followed-by-map equation for $\rho_q(K)$. □

Lemma 14.5.56 (Forward and reverse refinement maps are inverse). Lean: *conditional-interface:lemma:weil-a1c-refinement-inverse-equations*

Uses: *def:weil-a1c-refinement-forward*, *def:weil-a1c-refinement-reverse*

For every K,

$$\rho_q(K)\bar{\rho}_q(K) = \mathrm{id}_{\mathsf{F}_C(K)}, \qquad \bar{\rho}_q(K)\rho_q(K) = \mathrm{id}_{\mathsf{F}_D(K)}.$$

Proof. Uses: `lemma:weil-a1c-refinement-resolution-left-inverse`, `lemma:weil-a1c-refinement-resolution-right-inverse`

The first cited lemma is the first displayed equation, and the second cited lemma is the second displayed equation. □

Proposition 14.5.57 (Refinement comparison isomorphism). Lean: *conditional-interface:prop:weil-a1c-refinement-isomorphism*

Uses: *def:weil-a1c-refinement-forward*, *def:weil-a1c-refinement-reverse*

The pair $(\rho_q, \bar{\rho}_q)$ defines a natural isomorphism $\mathsf{F}_D \simeq \mathsf{F}_C$.

Proof. Uses: `lemma:weil-a1c-refinement-inverse-equations`

The two equations in the preceding lemma are the left- and right-inverse axioms. Naturality holds componentwise because both transformations were defined from functorial pullback, extension, pushforward, and totalization maps. □

14.5.3 Common refinements and compactification independence

Definition 14.5.58 (Selected common dense refinement). Lean: `conditional-interface:def:weil-a1c-common-refinement`

Uses: `def:weil-a1-product-closure-refinement`, `def:nagata-dense-refinement`

For dense compactifications C_1, C_2 of f, first take the explicit product-closure refinement and then its dense refinement. Denote the result by D_{12}, and denote the two composite projections by $q_a : D_{12} \to C_a$, $a = 1, 2$.

Lemma 14.5.59 (Properties of the selected common refinement). Lean: *conditional-interface:lemma:weil-a1c-common-refinement*

Uses: *def:weil-a1c-common-refinement*

The object D_{12} is a dense compactification of f, and each q_a is a dense refinement datum.

Proof. Uses: `lemma:weil-a1-product-closure-source`, `lemma:weil-a1-product-closure-properties`, `prop:nagata-dense-refinement`, `def:weil-a1c-common-refinement`

The product-closure source lemma supplies the source map and its factorization. The product-closure property lemma supplies the proper map to Y and the two proper projections. Applying dense refinement preserves the factorization and composing its proper refinement with each projection gives the maps q_1, q_2. □

Definition 14.5.60 (Comparison between two compactifications). Lean: `conditional-interface:def:weil-a1c-common-comparison`

Uses: `def:weil-a1c-common-refinement`, `def:weil-a1c-refinement-forward`, `def:weil-a1c-refinement-reverse`

Define

$$\kappa_{12} := \rho_{q_2}\bar{\rho}_{q_1} : \mathsf{F}_{C_1} \longrightarrow \mathsf{F}_{C_2}, \qquad \bar{\kappa}_{12} := \rho_{q_1}\bar{\rho}_{q_2}.$$

Lemma 14.5.61 (Comparison inverse equations). Lean: *conditional-interface:lemma:weil-a1c-common-comparison-inverses*

Uses: *def:weil-a1c-common-comparison*

One has $\bar{\kappa}_{12}\kappa_{12} = \mathrm{id}$ and $\kappa_{12}\bar{\kappa}_{12} = \mathrm{id}$.

Proof. Uses: `def:weil-a1c-common-comparison`, `lemma:weil-a1c-refinement-inverse-equations`

Expand the first composite. The middle factors $\bar{\rho}_{q_2}\rho_{q_2}$ cancel, followed by $\rho_{q_1}\bar{\rho}_{q_1}$. Expanding the second composite gives the same two cancellations with q_1, q_2 interchanged. □

Definition 14.5.62 (Triple common dense refinement). Lean: `conditional-interface:def:weil-a1c-triple-common-refinement`

Uses: `def:weil-a1c-common-refinement`

For C_1, C_2, C_3, define D_{123} by applying the selected common refinement construction first to C_1, C_2, and then to its result and C_3. Let $r_a : D_{123} \to C_a$ be the three composite refinement maps.

Definition 14.5.63 (Two refinement-composition paths). Lean: `conditional-interface:def:weil-a1c-refinement-composition-paths`

Uses: `def:weil-a1c-refinement-forward,def:weil-a1-compactification-morphism`

For dense refinements $E \xrightarrow{r} D \xrightarrow{q} C$, define

$$P_{q,r} := \rho_q \rho_r, \qquad Q_{q,r} := \rho_{q\circ r}.$$

These are natural transformations $\mathsf{F}_E \to \mathsf{F}_C$.

Lemma 14.5.64 (Composition of refinement comparisons). Lean: *conditional-interface:lemma:weil-a1c-refinement-composition*

Uses: *def:weil-a1c-refinement-composition-paths*

The two transformations satisfy $P_{q,r} = Q_{q,r}$.

Proof. Uses: `lemma:weil-a1-compactification-comparison-composition,def:weil-a1c-refinement-forward,def:weil-a1c-refinement-composition-paths`

Apply the compactification-comparison composition lemma to $E \xrightarrow{r} D \xrightarrow{q} C$. Under the definition $\rho_s = c_s$, its equation $c_{qr} = c_q c_r$ reads $Q_{q,r} = P_{q,r}$. Symmetry of equality gives the stated orientation. □

Lemma 14.5.65 (Identity refinement comparison). Lean: *conditional-interface:lemma:weil-a1c-refinement-identity*

Uses: *def:weil-a1c-refinement-forward*

For the identity dense refinement $1_C : C \to C$, one has $\rho_{1_C} = \mathrm{id}_{\mathsf{F}_C}$.

Proof. Uses: `def:weil-a1c-refinement-forward,lemma:weil-a1-proper-direct-image-functorial`

The source pullback is X, the support transport is the identity on every local section, and the proper direct-image comparison for an identity map is the identity on every resolved term. Totalization preserves this equality. □

Lemma 14.5.66 (Cocycle for compactification comparisons). Lean: *conditional-interface:lemma:weil-a1c-common-comparison-cocycle*

Uses: *def:weil-a1c-common-comparison, def:weil-a1c-triple-common-refinement*

For three dense compactifications,

$$\kappa_{23}\kappa_{12} = \kappa_{13}, \qquad \kappa_{11} = \mathrm{id}\,.$$

Proof. Uses: `lemma:weil-a1c-refinement-composition,lemma:weil-a1c-refinement-inverse-equations,lemma:weil-a1c-refinement-identity,def:weil-a1c-triple-common-refinement`

Rewrite every binary common-refinement map through D_{123} by the refinement-composition lemma. In the first equation, the two adjacent maps to and from C_2 cancel by the refinement inverse equations. The remaining two maps are the definition of κ_{13}. For $C_1 = C_1$, the same expression reduces to the identity refinement comparison. □

Proposition 14.5.67 (Compactification independence of the finite-level candidate). Lean: *conditional-interface:prop:weil-a1c-compactification-independence*

Uses: *def:weil-a1c-compact-support-candidate, def:weil-a1c-common-comparison*

The functors F_C, indexed by dense compactifications of f, are related by natural isomorphisms κ_{12} satisfying the identity and cocycle equations. If another common refinement is inserted, the resulting comparison equals κ_{12}.

Proof. Uses: `lemma:weil-a1c-common-comparison-inverses,lemma:weil-a1c-common-comparison-cocycle,lemma:weil-a1c-refinement-composition`

The first cited lemma supplies an explicit inverse. The second supplies the identity and cocycle. Given another common refinement E, refine it together with D_{12} and rewrite both comparison paths through that further refinement. Refinement composition identifies the rewritten paths, and the adjacent inverse pairs cancel. Hence the original paths are equal. □

14.6 Units, composition, and associativity of compact support

Definition 14.6.1 (Finite-level exceptional direct image)**.** Lean: `conditional-interface:def:weil-a1c-rfshriek`

Uses: `def:weil-a1c-selected-dense-compactification,def:weil-a1c-compact-support-candidate`

For a separated finite-type morphism $f : X \to Y$, define

$$Rf_!^{(n)} := \mathsf{F}_{C_f} = Rp_{f*}Rj_{f!}.$$

Definition 14.6.2 (Identity compactification and unit maps)**.** Lean: `conditional-interface:def:weil-a1c-identity-unit-maps`

Uses: `def:weil-a1c-rfshriek,def:weil-a1c-common-comparison`

For $1_X : X \to X$, let $I_X = (X \xrightarrow{1_X} X \xrightarrow{1_X} X)$. Define

$$u_X : \mathrm{id} = \mathsf{F}_{I_X} \longrightarrow R(1_X)_!^{(n)} \quad \text{by } \kappa_{I_X,C_{1_X}}, \qquad v_X := \bar{\kappa}_{I_X,C_{1_X}}.$$

Lemma 14.6.3 (Unit inverse equations)**.** Lean: *conditional-interface:lemma:weil-a1c-unit-inverse-equations*

Uses: *def:weil-a1c-identity-unit-maps*

One has $v_X u_X = \mathrm{id}$ and $u_X v_X = \mathrm{id}$.

Proof. Uses: `def:weil-a1c-identity-unit-maps,lemma:weil-a1c-common-comparison-inverses`

These are the two comparison inverse equations with $(C_1, C_2) = (I_X, C_{1_X})$. □

Proposition 14.6.4 (Identity law for compact support)**.** Lean: *conditional-interface:prop:weil-a1c-identity-isomorphism*

Uses: *def:weil-a1c-identity-unit-maps*

The maps u_X, v_X define a natural isomorphism $R(1_X)_!^{(n)} \simeq \mathrm{id}_{\mathcal{D}_n(X)}$.

Proof. Uses: `lemma:weil-a1c-unit-inverse-equations`

Package v_X with the two inverse equations. Naturality is inherited from the common-refinement comparison maps. □

Definition 14.6.5 (Composition maps for compact support)**.** Lean: `conditional-interface:def:weil-a1c-composition-maps`

Uses: `def:weil-a1c-rfshriek,def:weil-a1-rfshriek-composition-map,def:weil-a1-rfshriek-composition-inverse-candidate`

For $X \xrightarrow{f} Y \xrightarrow{g} Z$, define

$$c_{g,f} : Rg_!^{(n)} Rf_!^{(n)} \longrightarrow R(gf)_!^{(n)}$$

by the existing successive-compactification composition map, with every selected compactification replaced by its dense core. Define $d_{g,f} : R(gf)_!^{(n)} \to Rg_!^{(n)} Rf_!^{(n)}$ by the displayed reverse successive-compactification map and reverse common-refinement comparison.

Lemma 14.6.6 (Composition inverse equations)**.** Lean: *conditional-interface:lemma:weil-a1c-composition-inverse-equations*

Uses: *def:weil-a1c-composition-maps*

The equations

$$d_{g,f} c_{g,f} = \mathrm{id}, \qquad c_{g,f} d_{g,f} = \mathrm{id}$$

hold as natural transformations.

Proof. Uses: `def:weil-a1c-composition-maps,lemma:weil-a1-rfshriek-composition-left-inverse,lemma:weil-a1-rfshriek-composition-right-inverse,lemma:weil-a1c-common-comparison-inverses`

Expand $c_{g,f}$ and $d_{g,f}$. In the first composite, cancel the successive-compactification map with its reverse by the left-inverse lemma, then cancel the remaining compactification comparison. In the second composite, apply the right-inverse lemma and then the other common-comparison inverse equation. Each cancellation is an equality of natural transformations, so the resulting equations are natural in the coefficient complex. □

Proposition 14.6.7 (Composition isomorphism for compact support)**.** Lean: *conditional-interface:prop:weil-a1c-composition-isomorphism*

Uses: *def:weil-a1c-composition-maps*

The pair $(c_{g,f}, d_{g,f})$ defines a natural isomorphism

$$R(gf)_!^{(n)} \simeq Rg_!^{(n)} Rf_!^{(n)}.$$

Proof. Uses: lemma:weil-a1c-composition-inverse-equations
The two displayed equations are precisely the inverse axioms for the pair. □

Definition 14.6.8 (Two triple-composition paths). Lean: conditional-interface:def:weil-a1c-associativity-paths

Uses: def:weil-a1c-composition-maps
For $X \xrightarrow{f} Y \xrightarrow{g} Z \xrightarrow{h} W$, define

$$A^{\mathrm{L}}_{h,g,f} := c_{h,gf} \circ Rh_!(c_{g,f}), \qquad A^{\mathrm{R}}_{h,g,f} := c_{hg,f} \circ (c_{h,g} Rf_!).$$

Both maps have source $Rh_!Rg_!Rf_!$ and target $R(hgf)_!$.

Lemma 14.6.9 (Associativity of compact-support composition). Lean: conditional-interface:lemma:weil-a1c-associativity

Uses: def:weil-a1c-associativity-paths
The two paths satisfy $A^{\mathrm{L}}_{h,g,f} = A^{\mathrm{R}}_{h,g,f}$.

Proof. Uses: def:weil-a1c-associativity-paths,lemma:weil-a1-rfshriek-associativity,lemma:weil-a1c-common-comparison-cocycle
Expand both paths through the two successive compactifications. The existing associativity lemma identifies their open-extension and proper-totalization parts. Both remaining paths are changes from the same triple successive compactification to C_{hgf}; the common-comparison cocycle identifies them. □

Lemma 14.6.10 (Left unit law for compact support). Lean: conditional-interface:lemma:weil-a1c-left-unit

Uses: def:weil-a1c-identity-unit-maps, def:weil-a1c-composition-maps
The composite

$$Rf_! \xrightarrow{u_Y Rf_!} R(1_Y)_! Rf_! \xrightarrow{c_{1_Y,f}} Rf_!$$

is the identity.

Proof. Uses: lemma:weil-a1c-refinement-identity,lemma:weil-a1c-common-comparison-cocycle,def:weil-a1c-identity-unit-maps,def:weil-a1c-composition-maps
Use I_Y for the identity compactification. The successive compactification is related to C_f by identity refinements. Their maps are identities, and the two remaining comparison maps compose to the direct comparison by the cocycle; that direct comparison is the identity. □

Lemma 14.6.11 (Right unit law for compact support). Lean: conditional-interface:lemma:weil-a1c-right-unit

Uses: def:weil-a1c-identity-unit-maps, def:weil-a1c-composition-maps
The composite

$$Rf_! \xrightarrow{Rf_! u_X} Rf_! R(1_X)_! \xrightarrow{c_{f,1_X}} Rf_!$$

is the identity.

Proof. Uses: lemma:weil-a1c-refinement-identity,lemma:weil-a1c-common-comparison-cocycle,def:weil-a1c-identity-unit-maps,def:weil-a1c-composition-maps
For I_X, extension by zero and pushforward along the identity are identity functors. The successive compactification differs from C_f only by identity refinements. Remove them by the refinement-identity lemma and combine the remaining comparisons with the cocycle equation. □

14.7 Open–closed localization and its compact-support image

Definition 14.7.1 (Closed direct image and restriction). Lean: conditional-interface:def:weil-a1c-closed-direct-image

Uses: def:weil-a1-open-closed-complement,def:weil-a1-torsion-derived-category
For an open–closed pair $j : U \hookrightarrow X$, $i : Z \hookrightarrow X$, define i_* on a sheaf G by $(i_*G)(V) = G(V \times_X Z)$, with restriction maps induced by fiber products. Define i^* by sheafified pullback.

Lemma 14.7.2 (Closed direct image stalks)**.** Lean: `conditional-interface:lemma:weil-a1c-closed-direct-image-stalks`

Uses: `def:weil-a1c-closed-direct-image`

For a geometric point $\bar{x} \to X$,

$$(i_*G)_{\bar{x}} \cong \begin{cases} G_{\bar{x}}, & \bar{x} \text{ factors through } Z, \\ 0, & \bar{x} \text{ factors through } U. \end{cases}$$

Proof. Uses: `def:weil-a1c-closed-direct-image`

If $\bar{x}$ factors through Z, pullback of a geometric neighborhood to Z is a cofinal geometric neighborhood in Z, so the filtered colimit is $G_{\bar{x}}$. If it factors through U, shrink inside the open complement of Z; its pullback to Z is empty and the section module is zero. □

Lemma 14.7.3 (Exactness of closed direct image)**.** Lean: `conditional-interface:lemma:weil-a1c-closed-direct-image-exact`

Uses: `def:weil-a1c-closed-direct-image`

For every short exact sequence of finite constructible sheaves on Z, its image under i_* is a short exact sequence on X.

Proof. Uses: `lemma:weil-a1c-closed-direct-image-stalks`

Take a geometric point of X. On Z, the induced stalk sequence is the given exact sequence after applying the exact filtered-colimit stalk functor. On U, all three stalks are zero. Thus the image sequence is exact on every geometric stalk. Kernels and cokernels of sheaves are detected on geometric stalks, so the sheaf sequence is exact. □

Definition 14.7.4 (Termwise closed direct image on complexes)**.** Lean: `conditional-interface:def:weil-a1c-derived-closed-direct-image`

Uses: `def:weil-a1c-closed-direct-image`

For a bounded complex $G = (G^r, d^r)$ on Z, define

$$(i_*^{\mathrm{cx}}G)^r := i_*G^r, \qquad d^r_{i_*G} := i_*(d^r).$$

On a chain map, apply i_* in every degree. Denote the induced functor on the localization at quasi-isomorphisms by Ri_*.

Lemma 14.7.5 (Termwise closed direct image computes the derived functor)**.** Lean: `conditional-interface:lemma:weil-a1c-derived-closed-direct-image-computes`

Uses: `def:weil-a1c-derived-closed-direct-image`

The termwise functor preserves quasi-isomorphisms and represents the right derived functor of i_*. In particular, $Ri_* = i_*^{\mathrm{cx}}$ on the finite constructible derived category.

Proof. Uses: `lemma:weil-a1c-closed-direct-image-exact,def:weil-a1c-derived-closed-direct-image`

Exactness gives $\mathcal{H}^r(i_*^{\mathrm{cx}}G) = i_*\mathcal{H}^r(G)$. Hence an acyclic complex is sent to an acyclic complex and a quasi-isomorphism is sent to a quasi-isomorphism. The termwise functor therefore descends to the derived localization. An exact functor has no positive right-derived corrections, so this descended functor is Ri_*. □

Definition 14.7.6 (Open localization counit)**.** Lean: `conditional-interface:def:weil-a1c-open-localization-counit`

Uses: `def:weil-a1-derived-extension-zero,def:weil-a1-extension-adjunction-backward`

For $K \in \mathcal{D}_n(X)$, define

$$e_{j,K} : Rj_!j^*K \longrightarrow K$$

by applying the extension-backward map to id_{j^*K} degreewise on a bounded representative and then passing to the derived category.

Definition 14.7.7 (Closed localization restriction)**.** Lean: `conditional-interface:def:weil-a1c-closed-localization-restriction`

Uses: `def:weil-a1c-closed-direct-image,def:weil-a1c-derived-closed-direct-image`

For $K \in \mathcal{D}_n(X)$, define

$$r_{i,K} : K \longrightarrow Ri_*i^*K$$

on a sheaf in each degree by restricting a section on $V \to X$ to $V \times_X Z$, and use the same formula on the differentials.

Lemma 14.7.8 (The localization composite is zero). Lean: `conditional-interface:lemma:weil-a1c-localization-composite-zero`

Uses: `def:weil-a1c-open-localization-counit`, `def:weil-a1c-closed-localization-restriction`

For every K, one has $r_{i,K} e_{j,K} = 0$.

Proof. Uses: `def:weil-a1c-open-localization-counit`, `def:weil-a1c-closed-localization-restriction`, `lemma:weil-a1-extension-zero-closed-restriction`

Restrict the source of $e_{j,K}$ to Z. Extension by zero has zero restriction to Z, so the resulting component is zero in every degree. The composite chain map is zero, and localization preserves this equation. □

Lemma 14.7.9 (Naturality of the open localization counit). Lean: `conditional-interface:lemma:weil-a1c-open-localization-counit-natural`

Uses: `def:weil-a1c-open-localization-counit`

For every chain map $a : K \to L$,

$$a\, e_{j,K} = e_{j,L}\, Rj_!j^*(a).$$

Proof. Uses: `def:weil-a1c-open-localization-counit`, `lemma:weil-a1-extension-zero-functorial`

Fix a degree r and a local section represented by a coefficient section on the source open. The left side first extends that coefficient by zero and then applies a^r. The right side first applies a^r to the coefficient and then extends it by zero. Both give a^r of the same coefficient on the source open and zero on its complement. Hence the chain maps agree degree by degree. □

Lemma 14.7.10 (Naturality of the closed localization restriction). Lean: `conditional-interface:lemma:weil-a1c-closed-localization-restriction-natural`

Uses: `def:weil-a1c-closed-localization-restriction`

For every chain map $a : K \to L$,

$$Ri_*i^*(a)\, r_{i,K} = r_{i,L}\, a.$$

Proof. Uses: `def:weil-a1c-closed-localization-restriction`

On an object $V \to X$, both composites restrict a section to $V \times_X Z$ and then apply the same component a^r. Restriction of sections commutes with a sheaf morphism, so the two degree-r maps are equal. This holds for every r. □

Definition 14.7.11 (Localization cone comparison). Lean: `conditional-interface:def:weil-a1c-localization-cone-comparison`

Uses: `def:weil-a1c-open-localization-counit`, `def:weil-a1c-closed-localization-restriction`

Form $\mathrm{Cone}(e_{j,K})$. Define

$$\theta_K : \mathrm{Cone}(e_{j,K}) \longrightarrow Ri_*i^*K$$

to have component $r_{i,K}$ on the K-summand and zero component on the shifted $Rj_!j^*K$-summand.

Lemma 14.7.12 (The localization cone comparison is a chain map). Lean: `conditional-interface:lemma:weil-a1c-localization-cone-comparison-chain-map`

Uses: `def:weil-a1c-localization-cone-comparison`

The two components defining θ_K satisfy the cone chain-map equation.

Proof. Uses: `lemma:weil-a1c-localization-composite-zero`, `def:weil-a1c-localization-cone-comparison`

On the K-summand the equation is the chain-map equation for $r_{i,K}$. On the shifted summand the only additional term is $r_{i,K} e_{j,K}$, which is zero by the cited lemma. Thus the two components define a chain map. □

Definition 14.7.13 (Map between localization cones). Lean: `conditional-interface:def:weil-a1c-localization-cone-map-on-morphism`

Uses: `def:weil-a1c-open-localization-counit`

For a chain map $a : K \to L$, define

$$\mathrm{Cone}_j(a) : \mathrm{Cone}(e_{j,K}) \longrightarrow \mathrm{Cone}(e_{j,L})$$

to have component a on the unshifted summand and component $Rj_!j^*(a)[1]$ on the shifted summand.

Lemma 14.7.14 (The map between localization cones is a chain map). *Lean:* `conditional-interface:lemma:weil-a1c-localization-cone-map-on-morphism-chain-map`

Uses: `def:weil-a1c-localization-cone-map-on-morphism`

The two components defining $\mathrm{Cone}_j(a)$ *satisfy the cone chain-map equation.*

Proof. Uses: `lemma:weil-a1c-open-localization-counit-natural,def:weil-a1c-localization-cone-map-on-morphism`

The diagonal parts are chain-map equations for a and $Rj_!j^*(a)[1]$. Equality of the off-diagonal parts is precisely $ae_{j,K} = e_{j,L}Rj_!j^*(a)$, proved by the counit-naturality lemma. □

Lemma 14.7.15 (Localization cone on the open). *Lean:* `conditional-interface:lemma:weil-a1c-localization-cone-open-stalk`

Uses: `def:weil-a1c-localization-cone-comparison`

At a geometric point $\bar{u} \to U$*, the stalk of* θ_K *is an isomorphism from the zero complex to the zero complex.*

Proof. Uses: `def:weil-a1c-localization-cone-comparison,lemma:weil-a1-extension-zero-open-restriction,lemma:weil-a1c-closed-direct-image-stalks`

At $\bar{u}$, the counit $e_{j,K}$ is the identity $K_{\bar{u}} \to K_{\bar{u}}$. Its cone is contractible. The closed direct-image stalk is zero. Thus $\theta_{K,\bar{u}}$ is the unique map between zero objects and is an isomorphism. □

Lemma 14.7.16 (Localization cone on the closed complement). *Lean:* `conditional-interface:lemma:weil-a1c-localization-cone-closed-stalk`

Uses: `def:weil-a1c-localization-cone-comparison`

At a geometric point $\bar{z} \to Z$*, the stalk of* θ_K *is the identity of* $K_{\bar{z}}$*.*

Proof. Uses: `def:weil-a1c-localization-cone-comparison,lemma:weil-a1-extension-zero-closed-restriction,lemma:weil-a1c-closed-direct-image-stalks`

The stalk of $Rj_!j^*K$ at $\bar{z}$ is zero, so the cone of $0 \to K_{\bar{z}}$ is $K_{\bar{z}}$. The target stalk is also $K_{\bar{z}}$, and the restriction map is the identity under this identification. This is the stalk of θ_K. □

Lemma 14.7.17 (Naturality of the localization cone comparison). *Lean:* `conditional-interface:lemma:weil-a1c-localization-cone-comparison-natural`

Uses: `def:weil-a1c-localization-cone-comparison,def:weil-a1c-localization-cone-map-on-morphism`

For every chain map $a : K \to L$*,*

$$Ri_*i^*(a)\,\theta_K = \theta_L\ \mathrm{Cone}_j(a).$$

Proof. Uses: `lemma:weil-a1c-closed-localization-restriction-natural,def:weil-a1c-localization-cone-comparison,def:weil-a1c-localization-cone-map-on-morphism`

On the unshifted summand the equation is $Ri_*i^*(a)r_{i,K} = r_{i,L}a$, which is the restriction-naturality lemma. On the shifted summand both composites are zero. Equality on the two direct summands gives equality of the cone maps. □

Proposition 14.7.18 (Open–closed localization triangle). *Lean:* `conditional-interface:prop:weil-a1c-open-closed-localization`

Uses: `def:weil-a1c-open-localization-counit,def:weil-a1c-closed-localization-restriction,def:weil-a1c-localization-cone-comparison`

For every $K \in \mathcal{D}_n(X)$*, the sequence*

$$Rj_!j^*K \xrightarrow{e_{j,K}} K \xrightarrow{r_{i,K}} Ri_*i^*K \longrightarrow Rj_!j^*K[1]$$

is a distinguished triangle, natural in K*.*

Proof. Uses: `lemma:weil-a1c-localization-cone-open-stalk,lemma:weil-a1c-localization-cone-closed-stalk,lemma:weil-a1c-localization-cone-comparison-chain-map,lemma:weil-a1c-localization-cone-map-on-morphism-chain-map,lemma:weil-a1c-localization-cone-comparison-natural,lemma:weil-pbc-geometric-stalk-conservative,def:weil-a1c-localization-cone-comparison`

The two stalk lemmas show that θ_K is an isomorphism at every geometric point. Geometric-stalk conservativity makes θ_K an isomorphism in the finite constructible derived category. Transport the third object of the mapping-cone triangle for $e_{j,K}$ through this isomorphism. The cone-map lemma and the naturality equation for θ show that this transport commutes with every morphism $K \to L$. □

Theorem 14.7.19 (Finite-torsion TraceCore localization discharge). *Lean:* `blueprint-target:thm:trace core-extension-zero-derived-localization-v1`

Uses: `def:trace core-extension-zero-datum`, `def:weil-a1-torsion-derived-category`, `def:weil-a1c-open-localization-counit`, `def:weil-a1c-closed-localization-restriction`, `def:weil-a1c-localization-cone-comparison`

Let $j : U \hookrightarrow X$ and $i : Z \hookrightarrow X$ be the open–closed pair underlying the finite-torsion realization of the raw TraceCore datum. For every $K \in \mathcal{D}_n(X)$, take its three derived candidates to be $e_{j,K}$, $r_{i,K}$, and the boundary ∂_K obtained by transporting the mapping-cone boundary through $\theta_K : \mathrm{Cone}(e_{j,K}) \xrightarrow{\sim} Ri_*i^*K$. Then

$$Rj_!j^*K \xrightarrow{e_{j,K}} K \xrightarrow{r_{i,K}} Ri_*i^*K \xrightarrow{\partial_K} Rj_!j^*K[1]$$

is a distinguished triangle, naturally in K.

Proof. Uses: `prop:weil-a1c-open-closed-localization`, `lemma:weil-sf-well-defined-def-weil-a1-derived-extension-zero`, `lemma:weil-a1-extension-zero-exact`, `lemma:weil-a1c-derived-closed-direct-image-computes`

The derived-extension certificate identifies $Rj_!$ with the exact termwise localization of $j_!$, while the closed-image computation identifies Ri_* with termwise i_*. Thus the first two arrows are precisely the finite derived realizations of the corresponding raw TraceCore candidates. The A1C localization proposition identifies θ_K as a natural isomorphism and transports the mapping-cone triangle of $e_{j,K}$ through it. Its third arrow is therefore the stated ∂_K, proving distinguishedness and naturality. No completion, scalar inversion, E-coefficient Frobenius structure, local trace class, or compact-support trace assertion is used. □

Definition 14.7.20 (Standard compactification of an open immersion). *Lean:* `conditional-interface:def:weil-a1c-open-standard-compactification`

Uses: `def:weil-a1c-compact-support-candidate`, `def:weil-a1-open-closed-complement`

For $j : U \hookrightarrow X$, define

$$C_j^{\mathrm{op}} := \big(U \xrightarrow{j} X \xrightarrow{\mathrm{id}_X} X\big).$$

Its compactification candidate is $\mathsf{F}_{C_j^{\mathrm{op}}} = R(\mathrm{id}_X)_*Rj_!$.

Lemma 14.7.21 (The open standard tuple is a compactification). *Lean:* `conditional-interface:lemma:weil-a1c-open-standard-compactification`

Uses: `def:weil-a1c-open-standard-compactification`

The source map of C_j^{op} is a quasi-compact open immersion, the structure map is proper, and their composite is j.

Proof. Uses: `def:weil-a1c-open-standard-compactification`, `def:weil-a1-open-closed-complement`

The source map is the given quasi-compact open immersion. The identity of X is proper, and $\mathrm{id}_X \circ j = j$ by the identity law for composition. □

Definition 14.7.22 (Open-extension compact-support bridge). Lean: `conditional-interface:def:weil-a1c-open-shriek-bridge`

Uses: `def:weil-a1c-open-standard-compactification`, `def:weil-a1c-common-comparison`, `def:weil-a1c-rfshriek`

Define

$$b_j^+ : Rj_! \longrightarrow Rj_!^{(n)}$$

as the identity-direct-image comparison $Rj_! \simeq \mathsf{F}_{C_j^{\mathrm{op}}}$, followed by $\kappa_{C_j^{\mathrm{op}},C_j}$. Define b_j^- by the reverse common comparison followed by the reverse identity-direct-image map.

Proposition 14.7.23 (Open extension agrees with compact-support pushforward). *Lean:* `conditional-interface:prop:weil-a1c-open-shriek-bridge`

Uses: `def:weil-a1c-open-shriek-bridge`

The maps b_j^+, b_j^- are mutually inverse natural isomorphisms $Rj_! \simeq Rj_!^{(n)}$.

Proof. Uses: `lemma:weil-a1c-common-comparison-inverses`, `lemma:weil-a1-proper-direct-image-functorial`, `def:weil-a1c-open-shriek-bridge`

Direct image along id_X is the identity on the resolved complex, so its forward and reverse comparison maps cancel. The remaining two factors are the common compactification comparison and its reverse; they cancel in both orders by the comparison inverse equations. □

Definition 14.7.24 (Standard compactification of a closed immersion). Lean: `conditional-interface:def:weil-a1c-closed-standard-compactification`

Uses: `def:weil-a1c-compact-support-candidate,def:weil-a1c-closed-direct-image`

For $i : Z \hookrightarrow X$, define

$$C_i^{\mathrm{cl}} := \left(Z \xrightarrow{\mathrm{id}_Z} Z \xrightarrow{i} X\right).$$

Its compactification candidate is $\mathsf{F}_{C_i^{\mathrm{cl}}} = Ri_* R(\mathrm{id}_Z)_!$.

Lemma 14.7.25 (The closed standard tuple is a compactification). Lean: *conditional-interface:lemma:weil-a1c-closed-standard-compactification*

Uses: *def:weil-a1c-closed-standard-compactification*

The source map of C_i^{cl} is a quasi-compact open immersion, the structure map is proper, and their composite is i.

Proof. Uses: `def:weil-a1c-closed-standard-compactification`

The identity of Z is a quasi-compact open immersion. A closed immersion is proper. The composite $i \circ \mathrm{id}_Z$ is i. □

Definition 14.7.26 (Closed-star compact-support bridge). Lean: `conditional-interface:def:weil-a1c-closed-shriek-star-bridge`

Uses: `def:weil-a1c-closed-standard-compactification,def:weil-a1c-common-comparison,def:weil-a1c-rfshriek,def:weil-a1c-derived-closed-direct-image`

Define

$$b_i^+ : Ri_* \longrightarrow Ri_!^{(n)}$$

as the identity-open-extension comparison $Ri_* \simeq \mathsf{F}_{C_i^{\mathrm{cl}}}$, followed by $\kappa_{C_i^{\mathrm{cl}}, C_i}$. Define b_i^- by the reverse common comparison followed by the reverse identity-open-extension map.

Proposition 14.7.27 (Closed exceptional image agrees with direct image). Lean: *conditional-interface:prop:weil-a1c-closed-shriek-star-bridge*

Uses: *def:weil-a1c-closed-shriek-star-bridge*

The maps b_i^+, b_i^- are mutually inverse natural isomorphisms $Ri_* \simeq Ri_!^{(n)}$.

Proof. Uses: `lemma:weil-a1c-common-comparison-inverses,lemma:weil-a1-extension-zero-open-restriction,lemma:weil-a1c-derived-closed-direct-image-computes,def:weil-a1c-closed-shriek-star-bridge`

Extension by zero along id_Z is the identity degreewise. The termwise closed direct image computes Ri_*. Thus the two bridge composites reduce to the common compactification comparisons in the two orders, and those cancel by their inverse equations. □

Definition 14.7.28 (Compact-support localization triangle). Lean: `conditional-interface:def:weil-a1c-rfshriek-localization-triangle`

Uses: `def:weil-a1c-rfshriek,def:weil-a1c-composition-maps,def:weil-a1c-open-localization-counit,def:weil-a1c-closed-localization-restriction,def:weil-a1c-open-shriek-bridge,def:weil-a1c-closed-shriek-star-bridge`

For $f : X \to Y$ and an open–closed pair (j, i) in X, apply $Rf_!$ to the three maps of the localization triangle and use the composition maps to define

$$R(fj)_! j^* K \longrightarrow Rf_! K \longrightarrow R(fi)_! i^* K \longrightarrow R(fj)_! j^* K[1].$$

Proposition 14.7.29 (Compact-support localization is distinguished). Lean: *conditional-interface:prop:weil-a1c-rfshriek-localization-distinguished*

Uses: *def:weil-a1c-rfshriek-localization-triangle*

The compact-support localization sequence is a distinguished triangle, natural in K, and compatible with a second open–closed decomposition by the associativity of the composition maps.

Proof. Uses: `prop:weil-a1c-open-closed-localization,prop:weil-a1c-open-shriek-bridge,prop:weil-a1c-closed-shriek-star-bridge,prop:weil-a1c-composition-isomorphism,lemma:weil-a1c-associativity,def:weil-a1c-rfshriek-localization-triangle`

The functor $Rf_! = Rp_* Rj_!$ is a composite of derived functors and hence sends the distinguished localization triangle to a distinguished triangle. Replace its first and third terms by $R(fj)_! j^* K$ and $R(fi)_! i^* K$ using the composition isomorphisms. Transport of a distinguished triangle across isomorphisms remains distinguished. For an iterated decomposition, the two transports differ only by the two parenthesizations of a triple compact-support composite, which agree by the associativity lemma. □

Chapter 15

Locally finite Frobenius orbit dictionary

Status. The geometric Frobenius convention is unchanged. The type of geometric points may be infinite. Finiteness is required only for each iterate fixed locus. The full orbit quotient is never declared finite.

15.1 Locally finite periodic input

Definition 15.1.1 (Locally finite periodic Frobenius input). Uses: `def:tracecore-arithmetic-conventions`

Lean: `Weil.TraceCore3.LocallyFinitePeriodicInput`

A locally finite periodic Frobenius input is a tuple

$$\mathcal{D} = (\Omega, \Phi, r, r_{>0}, r_{\mathrm{fix}}, f_{\mathrm{fin}}),$$

where Ω is an arbitrary type, $\Phi : \Omega \simeq \Omega$ is a bijection, $r : \Omega \to \mathbf{N}$, and, for every $x \in \Omega$,

$$1 \le r(x), \qquad \Phi^{r(x)}(x) = x.$$

For every $n \ge 1$, the field $f_{\mathrm{fin}}(n)$ supplies finiteness of the subtype $\{x : \Omega \mid \Phi^n(x) = x\}$. No finiteness assumption is placed on Ω.

Definition 15.1.2 (Locally finite fixed-locus subtype). Uses: `def:tracecore3-locally-finite-periodic-input`

Lean: `Weil.TraceCore3.FixedLocusSubtype`

For $n \ge 1$, define

$$\mathrm{Fix}_{\mathcal{D}}(n) := \{x : \Omega \mid \Phi^n(x) = x\}.$$

Write $\pi_n : \mathrm{Fix}_{\mathcal{D}}(n) \to \Omega$ for the subtype projection.

Lemma 15.1.3 (Finiteness of each fixed locus). Uses: `def:tracecore3-fixed-locus-subtype`

Lean: `Weil.TraceCore3.fixedLocus_finite`

For every $n \ge 1$, $\mathrm{Fix}_{\mathcal{D}}(n)$ is finite.

Proof. Uses: `def:tracecore3-locally-finite-periodic-input`, `def:tracecore3-fixed-locus-subtype`

Project the field $f_{\mathrm{fin}}(n)$ from $\mathcal{D}$. Its target subtype is definitionally $\mathrm{Fix}_{\mathcal{D}}(n)$. □

Lemma 15.1.4 (Membership in a locally finite fixed locus). Uses: `def:tracecore3-fixed-locus-subtype`

Lean: `Weil.TraceCore3.mem_fixedLocus`

For $x \in \Omega$,

$$x \in \mathrm{Fix}_{\mathcal{D}}(n) \quad \Longleftrightarrow \quad \Phi^n(x) = x.$$

Proof. Uses: `def:tracecore3-fixed-locus-subtype`

Unfold the subtype predicate. Both sides reduce to the same equality. □

Definition 15.1.5 (Corrected finite-extension point family). Uses: def:tracecore3-locally-finite-periodic-input

Lean: Weil.TraceCore3.FiniteExtensionPointFamily

A finite-extension point family consists, for each $n \geq 1$, of a finite type R_n and its cardinality $N_n := \#R_n$. In the scheme instance, $R_n = X_0(\mathbf{F}_{q^n})$. No map between R_n and a fixed locus is part of this Definition.

Definition 15.1.6 (Corrected closed-point degree family). Uses: def:tracecore-arithmetic-conventions

Lean: Weil.TraceCore3.ClosedPointDegreeFamily

A closed-point degree family consists of a type C, a map $\deg_C : C \to \mathbf{N}_{>0}$, and, for every $m \geq 1$, finiteness of

$$\{c : C \mid \deg_C(c) \leq m\}.$$

No finiteness assumption is placed on the full type C.

15.2 The unrestricted orbit quotient

Definition 15.2.1 (Frobenius orbit relation). Uses: def:tracecore3-locally-finite-periodic-input

Lean: Weil.TraceCore3.FrobeniusOrbitRelation

For $x, y \in \Omega$, define

$$x \sim_\Phi y \quad \Longleftrightarrow \quad \exists m \in \mathbf{Z},\ y = \Phi^m(x).$$

Negative powers use the inverse bijection.

Lemma 15.2.2 (Orbit relation is reflexive). *Uses: def:tracecore3-frobenius-orbit-relation*

Lean: Weil.TraceCore3.orbitRelation_refl

For every $x \in \Omega$, $x \sim_\Phi x$.

Proof. Uses: def:tracecore3-frobenius-orbit-relation

Use the exponent zero and the identity $\Phi^0 = \mathrm{id}$. □

Lemma 15.2.3 (Orbit relation is symmetric). *Uses: def:tracecore3-frobenius-orbit-relation*

Lean: Weil.TraceCore3.orbitRelation_symm

If $x \sim_\Phi y$, then $y \sim_\Phi x$.

Proof. Uses: def:tracecore3-frobenius-orbit-relation

Write $y = \Phi^m(x)$. Applying Φ^{-m} gives $x = \Phi^{-m}(y)$, which is the relation in the reverse direction. □

Lemma 15.2.4 (Orbit relation is transitive). *Uses: def:tracecore3-frobenius-orbit-relation*

Lean: Weil.TraceCore3.orbitRelation_trans

If $x \sim_\Phi y$ and $y \sim_\Phi z$, then $x \sim_\Phi z$.

Proof. Uses: def:tracecore3-frobenius-orbit-relation

Write $y = \Phi^m(x)$ and $z = \Phi^s(y)$. The integer-power law gives $z = \Phi^{s+m}(x)$. □

Definition 15.2.5 (Unrestricted Frobenius orbit quotient). Uses: def:tracecore3-frobenius-orbit-relation

Constructed by: lemma:tracecore3-orbit-relation-reflexive,lemma:tracecore3-orbit-relation-symmetric,lemma:tracecore3-orbit-relation-transitive

Lean: Weil.TraceCore3.FrobeniusOrbitClass

Define

$$\mathrm{Orb}_\Phi(\Omega) := \Omega/\!\sim_\Phi.$$

The quotient may be infinite.

Definition 15.2.6 (Orbit quotient map). Uses: def:tracecore3-frobenius-orbit-class

Lean: Weil.TraceCore3.orbitQuotientMap

Define

$$q_\Phi : \Omega \longrightarrow \mathrm{Orb}_\Phi(\Omega), \qquad q_\Phi(x) := [x]_\Phi.$$

Lemma 15.2.7 (Equality in the orbit quotient). *Uses: def:tracecore3-orbit-quotient-map, def:tracecore3-frobenius-orbit-relation*

Lean: Weil.TraceCore3.orbitQuotientMap_eq_iff

For $x, y \in \Omega$,

$$q_\Phi(x) = q_\Phi(y) \quad \Longleftrightarrow \quad x \sim_\Phi y.$$

Proof. Uses: def:tracecore3-orbit-quotient-map,def:tracecore3-frobenius-orbit-relation,def:tracecore3-frobenius-orbit-class
The forward implication is quotient exactness for the equivalence relation $\sim_\Phi$. The reverse implication is the quotient soundness map. □

Definition 15.2.8 (Canonical representative of an orbit class). Uses: def:tracecore3-frobenius-orbit-class
Lean: Weil.TraceCore3.orbitRepresentative
For $O \in \mathrm{Orb}_\Phi(\Omega)$, define $\rho(O) \in \Omega$ by the quotient representative operation.

Lemma 15.2.9 (Class of the canonical representative). Uses: def:tracecore3-orbit-representative, def:tracecore3-orbit-quotient-map
Lean: Weil.TraceCore3.orbitRepresentative_class
For every orbit class O,

$$q_\Phi(\rho(O)) = O.$$

Proof. Uses: def:tracecore3-orbit-representative,def:tracecore3-orbit-quotient-map
This is the quotient representative equation for the quotient constructor and representative operation. □

15.3 Minimal periods and orbit degree

Definition 15.3.1 (Return-time finset). Uses: def:tracecore3-locally-finite-periodic-input
Lean: Weil.TraceCore3.ReturnTimeFinset
For $x \in \Omega$, define

$$R_x := \{m \in \mathbf{N} \mid 1 \le m \le r(x),\ \Phi^m(x) = x\}.$$

The set is finite by the upper bound and contains $r(x)$.

Definition 15.3.2 (Minimal Frobenius period). Uses: def:tracecore3-return-time-finset
Lean: Weil.TraceCore3.minimalPeriod
Define $p_\Phi(x) := \min R_x$.

Lemma 15.3.3 (Minimal period is a return time). Uses: def:tracecore3-minimal-period, def:tracecore3-return-time-finset
Lean: Weil.TraceCore3.minimalPeriod_mem_returnTime
For every $x \in \Omega$,

$$p_\Phi(x) \in R_x, \qquad \Phi^{p_\Phi(x)}(x) = x.$$

Proof. Uses: def:tracecore3-minimal-period,def:tracecore3-return-time-finset
The element $r(x)$ proves that R_x is nonempty. Apply membership of the minimum of a nonempty finite set, then unfold membership in R_x. □

Lemma 15.3.4 (Minimal period is positive). Uses: def:tracecore3-minimal-period
Lean: Weil.TraceCore3.minimalPeriod_positive
For every x, $1 \le p_\Phi(x)$.

Proof. Uses: def:tracecore3-return-time-finset,lemma:tracecore3-minimal-period-return
Every element of R_x is at least one. Apply this defining inequality to the element $p_\Phi(x) \in R_x$. □

Lemma 15.3.5 (Minimal period divides every return time). Uses: def:tracecore3-minimal-period, def:tracecore3-locally-finite-periodic-input
Lean: Weil.TraceCore3.minimalPeriod_dvd_return
If $m \ge 1$ and $\Phi^m(x) = x$, then $p_\Phi(x) \mid m$.

Proof. Uses: lemma:tracecore3-minimal-period-positive,lemma:tracecore3-minimal-period-return,def:tracecore3-minimal-period
Write $m = ap + s$ with $0 \le s < p := p_\Phi(x)$. Since $\Phi^p(x) = x$, cancellation of the a blocks gives $\Phi^s(x) = x$. If $s > 0$, this is a positive return time smaller than p, contrary to minimality. Hence $s = 0$. □

Lemma 15.3.6 (Minimal period is constant on an orbit). Uses: def:tracecore3-minimal-period, def:tracecore3-frobenius-orbit-relation
Lean: Weil.TraceCore3.minimalPeriod_orbitInvariant
If $x \sim_\Phi y$, then $p_\Phi(x) = p_\Phi(y)$.

Proof. Uses: def:tracecore3-frobenius-orbit-relation,def:tracecore3-minimal-period
Write $y = \Phi^s(x)$. For every m, applying Φ^s and its inverse shows

$$\Phi^m(x) = x \quad \Longleftrightarrow \quad \Phi^m(y) = y.$$

Thus the two positive return-time sets agree, so their minima agree. □

Definition 15.3.7 (Raw orbit-degree formula). Uses: def:tracecore3-minimal-period

Lean: Weil.TraceCore3.rawOrbitDegree
Define $d_{\mathrm{raw}}(x) := p_\Phi(x)$.

Lemma 15.3.8 (Raw orbit degree is constant on classes). *Uses: def:tracecore3-raw-orbit-degree, def:tracecore3-frobenius-orbit-relation*

Lean: Weil.TraceCore3.rawOrbitDegree_classInvariant
If $x \sim_\Phi y$, then $d_{\mathrm{raw}}(x) = d_{\mathrm{raw}}(y)$.

Proof. Uses: def:tracecore3-raw-orbit-degree,lemma:tracecore3-minimal-period-orbit-invariant
Unfold d_{raw} and apply minimal-period orbit invariance. □

Definition 15.3.9 (Orbit degree). Uses: def:tracecore3-frobenius-orbit-class,def:tracecore3-raw-orbit-degree

Constructed by: lemma:tracecore3-raw-orbit-degree-class-invariant,lemma:tracecore3-minimal-period-positive

Lean: Weil.TraceCore3.orbitDegree
Define the quotient lift

$$d_\Phi : \mathrm{Orb}_\Phi(\Omega) \to \mathbf{N}_{>0}, \qquad d_\Phi([x]_\Phi) := p_\Phi(x).$$

Lemma 15.3.10 (Orbit degree on a representative). *Uses: def:tracecore3-orbit-degree, def:tracecore3-minimal-period, def:tracecore3-orbit-quotient-map*

Lean: Weil.TraceCore3.orbitDegree_representative
For every $x \in \Omega$,

$$d_\Phi(q_\Phi(x)) = p_\Phi(x).$$

Proof. Uses: def:tracecore3-orbit-degree,def:tracecore3-raw-orbit-degree,def:tracecore3-orbit-quotient-map
Unfold the quotient lift, the quotient map, and the raw degree formula. □

15.4 Explicit enumeration of one orbit

Definition 15.4.1 (Carrier of a representative orbit). Uses: def:tracecore3-frobenius-orbit-relation

Lean: Weil.TraceCore3.representativeOrbitCarrier
For $x \in \Omega$, define

$$\mathcal{O}_x := \{y : \Omega \mid x \sim_\Phi y\}.$$

Definition 15.4.2 (Representative orbit enumeration). Uses: def:tracecore3-representative-orbit-carrier,def:tracecore3-minimal-period

Lean: Weil.TraceCore3.representativeOrbitEnumeration
Define

$$\epsilon_x : \mathrm{Fin}(p_\Phi(x)) \to \mathcal{O}_x, \qquad \epsilon_x(j) := \Phi^j(x).$$

The integer j is the orbit-relation witness for codomain membership.

Lemma 15.4.3 (Representative orbit enumeration is injective). *Uses: def:tracecore3-representative-orbit-enumeration*

Lean: Weil.TraceCore3.representativeOrbitEnumeration_injective
The map ϵ_x is injective.

Proof. Uses: def:tracecore3-representative-orbit-enumeration,lemma:tracecore3-minimal-period-divides
Suppose $\Phi^j(x) = \Phi^k(x)$. If $j < k$, applying Φ^{-j} gives the positive return time $k - j < p_\Phi(x)$, contradicting minimal-period divisibility. Interchanging j, k excludes $k < j$. Thus $j = k$. □

Lemma 15.4.4 (Representative orbit enumeration is surjective). *Uses: def:tracecore3-representative-orbit-enumeration, def:tracecore3-representative-orbit-carrier*

Lean: Weil.TraceCore3.representativeOrbitEnumeration_surjective

The map ϵ_x is surjective.

Proof. Uses: def:tracecore3-representative-orbit-enumeration, def:tracecore3-representative-orbit-carrier, lemma:tracecore3-minimal-period-return, lemma:tracecore3-minimal-period-positive

For $y \in \mathcal{O}_x$, write $y = \Phi^m(x)$ with $m \in \mathbf{Z}$. Euclidean division gives $m = ap + s$, where $p = p_\Phi(x)$ and $0 \le s < p$. The equality $\Phi^p(x) = x$ reduces $\Phi^m(x)$ to $\Phi^s(x)$, and s defines the required element of $\mathrm{Fin}(p)$. □

Proposition 15.4.5 (Cardinality of one orbit). *Uses: def:tracecore3-representative-orbit-carrier, def:tracecore3-orbit-degree, def:tracecore3-orbit-quotient-map*

Lean: Weil.TraceCore3.representativeOrbit_cardinality

For every $x \in \Omega$,

$$\#\mathcal{O}_x = d_\Phi(q_\Phi(x)).$$

Proof. Uses: lemma:tracecore3-representative-orbit-enumeration-injective, lemma:tracecore3-representative-orbit-enumeration-surjective, lemma:tracecore3-orbit-degree-representative

The two enumeration lemmas give a bijection $\mathrm{Fin}(p_\Phi(x)) \simeq \mathcal{O}_x$. Transport cardinality across it and use $\#\,\mathrm{Fin}(p) = p$, followed by the representative formula for orbit degree. □

Lemma 15.4.6 (Fixed by an iterate exactly at divisible orbit degree). *Uses: def:tracecore3-orbit-degree, def:tracecore3-fixed-locus-subtype, def:tracecore3-orbit-quotient-map*

Lean: Weil.TraceCore3.orbitFixed_iff_degreeDvd

For $x \in \Omega$ and $n \ge 1$,

$$\Phi^n(x) = x \quad \Longleftrightarrow \quad d_\Phi(q_\Phi(x)) \mid n.$$

Proof. Uses: lemma:tracecore3-orbit-degree-representative, lemma:tracecore3-minimal-period-divides, lemma:tracecore3-minimal-period-return

The forward implication is minimal-period divisibility. For the reverse implication write $n = ap_\Phi(x)$; iterating $\Phi^{p_\Phi(x)}(x) = x$ exactly a times gives $\Phi^n(x) = x$. □

15.5 Finite orbit sets attached to one iterate

Definition 15.5.1 (Degree-dividing orbit type). Uses: def:tracecore3-fixed-locus-subtype, def:tracecore3-orbit-quotient-map

Lean: Weil.TraceCore3.DegreeDividingOrbitType

For $n \ge 1$, define the image subtype

$$\mathcal{I}_n := \{O : \mathrm{Orb}_\Phi(\Omega) \mid \exists z : \mathrm{Fix}_{\mathcal{D}}(n),\ q_\Phi(\pi_n(z)) = O\}.$$

Thus $\mathcal{I}_n$ is the image of the finite fixed locus, not the full orbit quotient.

Lemma 15.5.2 (Degree-dividing orbit type is finite). *Uses: def:tracecore3-degree-dividing-orbit-type*

Lean: Weil.TraceCore3.degreeDividingOrbit_finite

For every $n \ge 1$, the type $\mathcal{I}_n$ is finite.

Proof. Uses: def:tracecore3-degree-dividing-orbit-type, lemma:tracecore3-fixed-locus-finite

The defining map from $\mathrm{Fix}_{\mathcal{D}}(n)$ onto its image is surjective. A surjective image of a finite type is finite. □

Lemma 15.5.3 (Membership in the degree-dividing orbit type). *Uses: def:tracecore3-degree-dividing-orbit-type, def:tracecore3-orbit-degree*

Lean: Weil.TraceCore3.mem_degreeDividingOrbit_iff

For $O \in \mathrm{Orb}_\Phi(\Omega)$,

$$O \in \mathcal{I}_n \quad \Longleftrightarrow \quad d_\Phi(O) \mid n.$$

Proof. Uses: def:tracecore3-degree-dividing-orbit-type, lemma:tracecore3-orbit-fixed-iff-degree-divides, lemma:tracecore3-orbit-representative-class

If $O \in \mathcal{I}_n$, its image witness z is fixed by Φ^n, so the fixed/divisibility lemma gives $d_\Phi(O) \mid n$. Conversely, apply the same lemma to $\rho(O)$. The representative-class equation turns the divisibility hypothesis into $\Phi^n(\rho(O)) = \rho(O)$, so this fixed point is an image witness for $O \in \mathcal{I}_n$. □

Definition 15.5.4 (Fixed-point to finite-orbit map). Uses: def:tracecore3-degree-dividing-orbit-type, def:tracecore3-fixed-locus-subtype

Lean: Weil.TraceCore3.fixedToDegreeOrbitMap

Define

$$Q_n : \mathrm{Fix}_{\mathcal{D}}(n) \to \mathcal{I}_n, \qquad Q_n(z) := q_\Phi(\pi_n(z)),$$

with image membership witnessed by z.

Lemma 15.5.5 (Fixed-point to finite-orbit map is surjective). *Uses: def:tracecore3-fixed-to-degree-orbit-map*

Lean: Weil.TraceCore3.fixedToDegreeOrbitMap_surjective

The map Q_n is surjective.

Proof. Uses: def:tracecore3-fixed-to-degree-orbit-map, def:tracecore3-degree-dividing-orbit-type

An element of $\mathcal{I}_n$ contains an image witness z and the equation $q_\Phi(\pi_n(z)) = O$. Subtype extensionality turns this equation into $Q_n(z) = O$. □

Definition 15.5.6 (Fiber over a degree-dividing orbit). Uses: def:tracecore3-fixed-to-degree-orbit-map

Lean: Weil.TraceCore3.FixedOrbitFiber

For $O : \mathcal{I}_n$, define

$$\mathcal{F}_{n,O} := \{z : \mathrm{Fix}_{\mathcal{D}}(n) \mid Q_n(z) = O\}.$$

Definition 15.5.7 (Fiber-to-orbit carrier map). Uses: def:tracecore3-fixed-orbit-fiber, def:tracecore3-representative-orbit-carrier, def:tracecore3-orbit-representative

Constructed by: lemma:tracecore3-orbit-quotient-equality, lemma:tracecore3-orbit-representative-class

Lean: Weil.TraceCore3.fixedFiberToOrbitMap

For $O : \mathcal{I}_n$, define

$$A_{n,O} : \mathcal{F}_{n,O} \to \mathcal{O}_{\rho(O)}, \qquad A_{n,O}(z) := \pi_n(z).$$

The fiber equation and the representative-class equation imply the required orbit-carrier membership.

Definition 15.5.8 (Orbit-carrier-to-fiber map). Uses: def:tracecore3-fixed-orbit-fiber, def:tracecore3-representative-orbit-carrier, def:tracecore3-orbit-representative

Constructed by: lemma:tracecore3-degree-dividing-orbit-membership, lemma:tracecore3-orbit-fixed-iff-degree-divides, lemma:tracecore3-orbit-representative-class, lemma:tracecore3-orbit-quotient-equality

Lean: Weil.TraceCore3.orbitToFixedFiberMap

For $O : \mathcal{I}_n$, define

$$B_{n,O} : \mathcal{O}_{\rho(O)} \to \mathcal{F}_{n,O}, \qquad B_{n,O}(y) := y.$$

Membership of O in $\mathcal{I}_n$ gives $d_\Phi(O) \mid n$. Orbit membership gives $q_\Phi(y) = O$; hence the fixed/divisibility criterion supplies $\Phi^n(y) = y$ and the fiber equation.

Lemma 15.5.9 (The fiber/orbit carrier maps are inverse). *Uses: def:tracecore3-fixed-fiber-to-orbit-map, def:tracecore3-orbit-to-fixed-fiber-map*

Lean: Weil.TraceCore3.fixedFiberOrbitMaps_inverse

For every $O : \mathcal{I}_n$,

$$B_{n,O} A_{n,O} = \mathrm{id}_{\mathcal{F}_{n,O}}, \qquad A_{n,O} B_{n,O} = \mathrm{id}_{\mathcal{O}_{\rho(O)}}.$$

Proof. Uses: def:tracecore3-fixed-fiber-to-orbit-map, def:tracecore3-orbit-to-fixed-fiber-map

Both composites preserve the underlying element of Ω. Apply subtype extensionality in each codomain. □

Proposition 15.5.10 (Cardinality of a fixed-locus orbit fiber). *Uses: def:tracecore3-fixed-orbit-fiber, def:tracecore3-orbit-degree*

Lean: Weil.TraceCore3.fixedOrbitFiber_cardinality

For every $O : \mathcal{I}_n$,

$$\#\mathcal{F}_{n,O} = d_\Phi(O).$$

Proof. Uses: lemma:tracecore3-fixed-fiber-orbit-maps-inverse, prop:tracecore3-representative-orbit-cardinality, lemma:tracecore3-orbit-representative-class

The inverse maps give $\#\mathcal{F}_{n,O} = \#\mathcal{O}_{\rho(O)}$. The orbit-cardinality proposition changes the right side to $d_\Phi(q_\Phi(\rho(O)))$, and the representative-class equation changes this to $d_\Phi(O)$. □

Proposition 15.5.11 (Locally finite fixed-locus orbit sum). *Uses: def: tracecore3-fixed-locus-subtype, def: tracecore3-degree-dividing-orbit-type, def: tracecore3-orbit-degree*

Lean: Weil.TraceCore3.fixedLocus_orbitSum

For every $n \geq 1$,

$$\#\operatorname{Fix}_{\mathcal{D}}(n) = \sum_{O:\mathcal{I}_n} d_\Phi(O).$$

Equivalently, this is the sum over all orbit classes whose degree divides n.

Proof. Uses: lemma:tracecore3-fixed-locus-finite, lemma:tracecore3-degree-dividing-orbit-finite, lemma:tracecore3-fixed-to-degree-orbit-surjective, def:tracecore3-fixed-orbit-fiber, prop:tracecore3-fixed-orbit-fiber-cardinality, lemma:tracecore3-degree-dividing-orbit-membership

Partition the finite type $\operatorname{Fix}_{\mathcal{D}}(n)$ into the fibers of Q_n. Surjectivity makes every $O : \mathcal{I}_n$ an index of a nonempty fiber. Finite additivity gives

$$\#\operatorname{Fix}_{\mathcal{D}}(n) = \sum_{O:\mathcal{I}_n} \#\mathcal{F}_{n,O}.$$

Substitute the fiber-cardinality proposition term by term. The membership lemma gives the equivalent degree-divisibility description of the index. □

15.6 The finite-type scheme instance

Definition 15.6.1 (Finite-type scheme over the fixed finite field). Uses: def:tracecore-arithmetic-conventions

Lean: Weil.TraceCore3.FiniteTypeSchemeData

Fix a scheme X_0 together with its structure morphism $a_0 : X_0 \to \operatorname{Spec}(k_0)$, and assume that a_0 is of finite type. Denote this data by $\mathcal{X}_0$. No geometric point, Frobenius map, closed-point map, or finiteness certificate is a field of this Definition.

Definition 15.6.2 (Geometric-point type and support). Uses: def:tracecore3-finite-type-scheme-data

Lean: Weil.TraceCore3.SchemeGeometricPointData

Define

$$\Omega_{X_0} := \operatorname{Hom}_{k_0}(\operatorname{Spec} k, X_0).$$

For $\bar{x} \in \Omega_{X_0}$, define its support $\operatorname{sp}(\bar{x}) \in |X_0|$ to be the image of the unique point of $\operatorname{Spec} k$ under $\bar{x}$.

Definition 15.6.3 (Raw geometric and arithmetic actions on geometric points). Uses: def:tracecore3-scheme-geometric-point-data, def:tracecore-arithmetic-conventions

Lean: Weil.TraceCore3.rawSchemeFrobeniusActions

For $\bar{x} : \operatorname{Spec} k \to X_0$, define

$$\Phi^{\mathrm{raw}}_{X_0}(\bar{x}) := \bar{x} \circ \operatorname{Spec}(\Phi_q), \qquad \varphi^{\mathrm{raw}}_{X_0}(\bar{x}) := \bar{x} \circ \operatorname{Spec}(\varphi_q).$$

The order is forced by contravariance of Spec. Thus the first map is the action induced by geometric field Frobenius.

Lemma 15.6.4 (The two raw point actions are inverse). *Uses: def: tracecore3-scheme-raw-frobenius-actions*

Lean: Weil.TraceCore3.rawSchemeFrobeniusActions_inverse

The two composites on Ω_{X_0} are the identity.

Proof. Uses: def:tracecore3-scheme-raw-frobenius-actions, lemma:weil-frobenius-inverse-identities

For $\bar{x}$, associativity of composition changes the first composite to $\bar{x} \circ \operatorname{Spec}(\Phi_q \circ \varphi_q)$, and the second to $\bar{x} \circ \operatorname{Spec}(\varphi_q \circ \Phi_q)$. Apply the two field-Frobenius inverse equations and then the identity law for composition. □

Definition 15.6.5 (Geometric Frobenius permutation on scheme points). Uses: def:tracecore3-scheme-geometric-point-data, def:tracecore3-scheme-raw-frobenius-actions

Constructed by: lemma:tracecore3-scheme-raw-frobenius-inverse

Lean: Weil.TraceCore3.schemeGeometricFrobenius

Define the bijection

$$\Phi_{X_0} : \Omega_{X_0} \simeq \Omega_{X_0}$$

with forward map $\Phi^{\mathrm{raw}}_{X_0}$ and inverse map $\varphi^{\mathrm{raw}}_{X_0}$.

Definition 15.6.6 (Positive return-time carrier of a geometric point). Uses: def:tracecore3-scheme-geometric-frobenius

Lean: Weil.TraceCore3.SchemeReturnTimeCarrier

For $\bar{x} \in \Omega_{X_0}$, define

$$\mathcal{T}_{\bar{x}} := \{n : \mathbf{N}_{>0} \mid \Phi_{X_0}^{n}(\bar{x}) = \bar{x}\}.$$

Lemma 15.6.7 (The positive return-time carrier is nonempty). Uses: def:tracecore3-scheme-return-time-carrier

Lean: Weil.TraceCore3.schemeReturnTimeCarrier_nonempty

For every $\bar{x} \in \Omega_{X_0}$, the type $\mathcal{T}_{\bar{x}}$ is nonempty.

Proof. Uses: def:tracecore3-scheme-return-time-carrier,def:tracecore3-scheme-geometric-point-data,def:tracecore3-scheme-geometric-frobenius,lemma:weil-arithmetic-frobenius-power-formula,lemma:weil-frobenius-inverse-identities

Let $x \in |X_0|$ be the support of $\bar{x}$, choose an affine open $U = \operatorname{Spec} A$ containing x, and factor $\bar{x}$ through U. Since X_0 is of finite type over k_0, the k_0-algebra A is generated by finitely many elements $a_1, \dots, a_r$. Write $\theta : A \to k$ for the coordinate homomorphism of the factorization. Each $\theta(a_i)$ is algebraic over k_0, and therefore

$$E := k_0\big(\theta(a_1), \dots, \theta(a_r)\big) \subseteq k$$

is a finite extension of k_0. If $m = [E : k_0]$, then $\#E = q^m$, so every $u \in E$ satisfies $u^{q^m} = u$. The arithmetic-Frobenius power formula gives $\varphi_q^m|_E = \mathrm{id}_E$; its inverse geometric Frobenius consequently also satisfies $\Phi_q^m|_E = \mathrm{id}_E$.

The image of θ lies in E, because the chosen generators do and every element of A is a polynomial in them with coefficients in k_0. Hence $\Phi_q^m \circ \theta = \theta$. Under the affine functor-of-points correspondence this is exactly $\Phi_{X_0}^m(\bar{x}) = \bar{x}$. The field extension E/k_0 is nonzero, so $m \geq 1$; thus m, together with the displayed return equation, is an element of $\mathcal{T}_{\bar{x}}$. □

Definition 15.6.8 (Selected scheme return time). Uses: def:tracecore3-scheme-return-time-carrier

Constructed by: lemma:tracecore3-scheme-return-time-nonempty

Lean: Weil.TraceCore3.schemeReturnTime

For each $\bar{x}$, define $r_{X_0}(\bar{x})$ to be the least natural number represented in the nonempty subtype $\mathcal{T}_{\bar{x}}$.

Lemma 15.6.9 (The selected scheme return time is positive). Uses: def:tracecore3-scheme-return-time

Lean: Weil.TraceCore3.schemeReturnTime_positive

For every $\bar{x}$, $1 \leq r_{X_0}(\bar{x})$.

Proof. Uses: def:tracecore3-scheme-return-time,def:tracecore3-scheme-return-time-carrier

The selected element belongs to a subtype of $\mathbf{N}_{>0}$; its underlying natural number is therefore at least one. □

Lemma 15.6.10 (The selected scheme return time fixes the point). Uses: def:tracecore3-scheme-return-time

Lean: Weil.TraceCore3.schemeReturnTime_fixes

For every $\bar{x}$,

$$\Phi_{X_0}^{r_{X_0}(\bar{x})}(\bar{x}) = \bar{x}.$$

Proof. Uses: def:tracecore3-scheme-return-time,def:tracecore3-scheme-return-time-carrier

The displayed equation is the predicate carried by the selected element of $\mathcal{T}_{\bar{x}}$. □

Definition 15.6.11 (Raw scheme fixed-locus subtype). Uses: def:tracecore3-scheme-geometric-frobenius

Lean: Weil.TraceCore3.SchemeFixedLocusSubtype

For $n \geq 1$, define

$$\mathrm{Fix}_{X_0}(n) := \{\bar{x} : \Omega_{X_0} \mid \Phi_{X_0}^{n}(\bar{x}) = \bar{x}\}.$$

Lemma 15.6.12 (Finiteness of the raw scheme fixed locus). Uses: def:tracecore3-scheme-fixed-locus-subtype

Lean: Weil.TraceCore3.schemeFixedLocus_finite

For every $n \geq 1$, $\mathrm{Fix}_{X_0}(n)$ is finite.

Proof. Uses: def:tracecore3-scheme-fixed-locus-subtype,def:tracecore3-finite-type-scheme-data, def:tracecore3-scheme-geometric-frobenius,lemma:weil-arithmetic-frobenius-power-formula,lemma:weil-frobenius-inverse-identities

Choose a finite affine cover $X_0 = \bigcup_{i=1}^{s} U_i$, with $U_i = \operatorname{Spec} A_i$. Such a cover exists because a finite-type morphism is quasi-compact. Choose a finite list of k_0-algebra generators of every A_i.

Put

$$S_n := \{a \in k \mid \Phi_q^n(a) = a\}.$$

The inverse relation between Φ_q and φ_q identifies this set with $\{a \mid \varphi_q^n(a) = a\}$. Iterating the arithmetic power formula therefore embeds S_n in the root set of $T^{q^n} - T$. This polynomial is nonzero of degree q^n, so the polynomial root bound shows that S_n is finite.

A point of $\mathrm{Fix}_{X_0}(n)$ whose support lies in U_i factors through U_i, and its coordinate map $\theta : A_i \to k$ satisfies $\Phi_q^n \circ \theta = \theta$. Thus the images of all selected generators lie in the finite set S_n. A k_0-algebra homomorphism out of A_i is determined by those generator images, so the fixed points supported in U_i inject into a finite power of S_n and form a finite set. The full fixed locus is the union of these sets over the finite affine cover, hence is finite. □

Definition 15.6.13 (Locally finite periodic input constructed from the scheme). Uses: def:tracecore3-locally-finite-periodic-input,def:tracecore3-scheme-geometric-point-data,def:tracecore3-scheme-geometric-frobenius,def:tracecore3-scheme-return-time,def:tracecore3-scheme-fixed-locus-subtype

Constructed by: lemma:tracecore3-scheme-return-time-positive,lemma:tracecore3-scheme-return-time-fixes,lemma:tracecore3-scheme-fixed-locus-finite

Lean: Weil.TraceCore3.schemeLocallyFinitePeriodicInput

Define $\mathcal{D}_{X_0}$ by substituting Ω_{X_0}, Φ_{X_0}, and r_{X_0} into the locally finite periodic-input constructor. Its positivity, return, and fixed-locus finiteness arguments are the three preceding lemmas.

Lemma 15.6.14 (The abstract and raw scheme fixed loci agree). *Uses: def:tracecore3-scheme-locally-finite-periodic-input, def:tracecore3-scheme-fixed-locus-subtype, def:tracecore3-fixed-locus-subtype*

Lean: Weil.TraceCore3.schemeFixedLocus_agreement

For every $n \geq 1$, the identity on underlying geometric points defines an equivalence

$$\mathrm{Fix}_{X_0}(n) \simeq \mathrm{Fix}_{\mathcal{D}_{X_0}}(n).$$

Proof. Uses: def:tracecore3-scheme-locally-finite-periodic-input,def:tracecore3-scheme-fixed-locus-subtype,def:tracecore3-fixed-locus-subtype

Unfold the two subtype predicates. Both are the equation $\Phi_{X_0}^n(\bar{x}) = \bar{x}$, so the identity map and itself are the two inverse maps. □

15.7 Closed points, residue embeddings, and the two explicit maps

Definition 15.7.1 (Closed-point type of the finite-type scheme). Uses: def:tracecore3-finite-type-scheme-data

Lean: Weil.TraceCore3.SchemeClosedPointType

Define

$$C_{X_0} := \{c : |X_0| \mid \{c\} \text{ is closed in } |X_0|\}.$$

For $c \in C_{X_0}$, write $\kappa(c)$ for its residue field and $i_c : \operatorname{Spec} \kappa(c) \to X_0$ for its canonical point morphism.

Lemma 15.7.2 (Residue field of a closed point is finite over the base). *Uses: def:tracecore3-scheme-closed-point-type*

Lean: Weil.TraceCore3.closedPointResidue_finite

For every $c \in C_{X_0}$, the extension $\kappa(c)/k_0$ is finite.

Proof. Uses: def:tracecore3-scheme-closed-point-type,def:tracecore3-finite-type-scheme-data

Choose an affine neighbourhood $U = \operatorname{Spec} A$ of c. Finite type of X_0/k_0 makes A a finitely generated k_0-algebra. Since c is closed in X_0, it is closed in U, so the corresponding prime $\mathfrak{m} \subset A$ is maximal and

$$\kappa(c) \cong A/\mathfrak{m}.$$

Consequently $\kappa(c)$ is both a field and a finitely generated k_0-algebra.

For completeness, apply Zariski's lemma in this precise form. If a field $L = k_0[x_1, \ldots, x_r]$ had a nonempty transcendence basis among the x_i, then, after taking a maximal such subfamily and clearing the finitely many

denominators in the algebraic equations for the remaining generators, the rational function field generated by that subfamily would be finite as a module over a localization of a polynomial ring. The unit lemma for polynomial rings rules this out: infinitely many distinct irreducible polynomials would all have to become units after inverting one fixed denominator. Hence every x_i is algebraic over k_0. Adjoining the generators one at a time then shows that L/k_0 has finite degree. Taking $L = A/\mathfrak{m}$ proves that $\kappa(c)/k_0$ is finite. □

Definition 15.7.3 (Scheme closed-point degree). Uses: `def:tracecore3-scheme-closed-point-type`

Constructed by: `lemma:tracecore3-closed-point-residue-finite`

Lean: `Weil.TraceCore3.schemeClosedPointDegree`

For $c \in C_{X_0}$, define

$$\deg_{X_0}(c) := [\kappa(c) : k_0] \in \mathbf{N}_{>0}.$$

The positive value is the dimension of the nonzero finite-dimensional k_0-vector space $\kappa(c)$.

Lemma 15.7.4 (Finiteness of degree-bounded closed points). *Uses:* `def:tracecore3-scheme-closed-point-degree`

Lean: `Weil.TraceCore3.degreeBoundedClosedPoints_finite`

For every $m \geq 1$, the subtype

$$\{c : C_{X_0} \mid \deg_{X_0}(c) \leq m\}$$

is finite.

Proof. Uses: `def:tracecore3-scheme-closed-point-degree`, `def:tracecore3-finite-type-scheme-data`, `lemma:tracecore3-closed-point-residue-finite`, `def:tracecore-arithmetic-conventions`, `lemma:weil-arithmetic-frobenius-power-formula`

Choose a finite affine cover $X_0 = \bigcup_{i=1}^{s} U_i$, with $U_i = \operatorname{Spec} A_i$, and choose finitely many k_0-algebra generators for each A_i. Fix $m \geq 1$ and put

$$S_m := \{a \in k \mid a^{q^{m!}} = a\}.$$

It is the root set of the nonzero polynomial $T^{q^{m!}} - T$, so the polynomial root bound makes S_m finite.

Let $c \in U_i$ have degree $e \leq m$. Its residue field is the finite field with q^e elements. Because finite fields are perfect, the finite extension $\kappa(c)/k_0$ is simple. By classical choice, for every such c select a root in the algebraic closure k of a primitive element's minimal polynomial; this gives a k_0-embedding $\iota_c : \kappa(c) \hookrightarrow k$. Since $e \mid m!$, every element of its image is fixed by the $q^{m!}$-power map, so $\iota_c(\kappa(c)) \subseteq S_m$.

Compose $A_i \to \kappa(c)$ with ι_c. The resulting homomorphism is determined by the images of the selected generators, hence by a tuple in a finite power of S_m. If two closed points give the same tuple, the two homomorphisms are equal; their kernels are therefore equal maximal ideals of A_i, so the points coincide. Thus the degree-at-most- m closed points lying in U_i inject into a finite set. Taking the finite union over the affine cover proves the assertion. □

Definition 15.7.5 (Closed-point degree family constructed from the scheme). Uses: `def:tracecore3-closed-point-degree-family`, `def:tracecore3-scheme-closed-point-type`, `def:tracecore3-scheme-closed-point-degree`

Constructed by: `lemma:tracecore3-degree-bounded-closed-points-finite`

Lean: `Weil.TraceCore3.schemeClosedPointDegreeFamily`

Define the closed-point degree family $\mathcal{C}_{X_0}$ with carrier C_{X_0}, degree map $\deg_{X_0}$, and the preceding bounded-degree finiteness proof.

Definition 15.7.6 (Residue-field embedding carrier). Uses: `def:tracecore3-scheme-closed-point-type`, `def:tracecore-arithmetic-conventions`

Lean: `Weil.TraceCore3.ResidueEmbeddingCarrier`

For $c \in C_{X_0}$, define

$$\operatorname{Emb}(c) := \operatorname{Hom}_{k_0\text{-alg}}(\kappa(c), k).$$

Lemma 15.7.7 (Existence of a residue-field embedding). *Uses:* `def:tracecore3-residue-embedding-carrier`

Lean: `Weil.TraceCore3.residueEmbeddingCarrier_nonempty`

For every $c \in C_{X_0}$, $\operatorname{Emb}(c)$ is nonempty.

Proof. Uses: def:tracecore3-residue-embedding-carrier,lemma:tracecore3-closed-point-residue-finite,def:tracecore-arithmetic-conventions

Put $L = \kappa(c)$. The preceding lemma makes L/k_0 finite. Since k_0 is finite it is perfect, so this extension is separable. The primitive-element theorem therefore supplies $\alpha \in L$ with $L = k_0(\alpha)$. Let $m_\alpha(T) \in k_0[T]$ be its monic minimal polynomial. It is irreducible and nonconstant.

The selected field k is algebraically closed, hence m_α has a root $\beta \in k$. Evaluation at β gives a k_0-algebra map

$$k_0[T] \longrightarrow k, \qquad P \longmapsto P(\beta),$$

whose kernel contains (m_α). Because m_α is irreducible and 1 does not vanish at β, the kernel is exactly (m_α). It therefore induces an injective map

$$k_0[T]/(m_\alpha) \hookrightarrow k.$$

The evaluation isomorphism $k_0[T]/(m_\alpha) \xrightarrow{\sim} k_0(\alpha) = L$ sends the class of T to α. Composing its inverse with the displayed injection produces an injective k_0-algebra homomorphism $\kappa(c) = L \hookrightarrow k$, which is an element of $\mathrm{Emb}(c)$. □

Definition 15.7.8 (Geometric lift attached to a residue embedding). Uses: def:tracecore3-residue-embedding-carrier,def:tracecore3-scheme-geometric-point-data,def:tracecore3-scheme-closed-point-type

Lean: Weil.TraceCore3.geometricLiftFromEmbedding

For $\iota \in \mathrm{Emb}(c)$, define

$$\bar{c}_\iota := i_c \circ \mathrm{Spec}(\iota) : \mathrm{Spec}\, k \longrightarrow X_0.$$

Here $\mathrm{Spec}(\iota) : \mathrm{Spec}\, k \to \mathrm{Spec}\, \kappa(c)$ is the morphism contravariantly induced by ι.

Lemma 15.7.9 (The embedding lift lies over the closed point). *Uses:* def:tracecore3-geometric-lift-from-embedding

Lean: Weil.TraceCore3.geometricLiftFromEmbedding_support

For $\iota \in \mathrm{Emb}(c)$,

$$\mathrm{sp}(\bar{c}_\iota) = c.$$

Proof. Uses: def:tracecore3-geometric-lift-from-embedding,def:tracecore3-scheme-geometric-point-data,def:tracecore3-scheme-closed-point-type

The unique point of $\mathrm{Spec}\, k$ maps to the unique point of $\mathrm{Spec}\, \kappa(c)$, and the canonical point morphism i_c maps that point to c. This is the definition of the support of the composite. □

Definition 15.7.10 (Geometric-lift carrier over a closed point). Uses: def:tracecore3-scheme-geometric-point-data,def:tracecore3-scheme-closed-point-type

Lean: Weil.TraceCore3.GeometricLiftCarrier

For $c \in C_{X_0}$, define

$$\mathrm{Lift}(c) := \{\bar{x} : \Omega_{X_0} \mid \mathrm{sp}(\bar{x}) = c\}.$$

Definition 15.7.11 (Embedding-to-lift map). Uses: def:tracecore3-residue-embedding-carrier,def:tracecore3-geometric-lift-from-embedding,def:tracecore3-geometric-lift-carrier

Constructed by: lemma:tracecore3-geometric-lift-support

Lean: Weil.TraceCore3.embeddingToLiftMap

Define

$$\lambda_c : \mathrm{Emb}(c) \longrightarrow \mathrm{Lift}(c), \qquad \iota \longmapsto \bar{c}_\iota,$$

with subtype membership supplied by the support lemma.

Definition 15.7.12 (Lift-to-residue-embedding map). Uses: def:tracecore3-geometric-lift-carrier,def:tracecore3-residue-embedding-carrier,def:tracecore3-scheme-closed-point-type

Lean: Weil.TraceCore3.liftToEmbeddingMap

For $\bar{x} \in \mathrm{Lift}(c)$, the pointed morphism $\bar{x} : \mathrm{Spec}\, k \to X_0$ induces the canonical local-ring homomorphism $\mathcal{O}_{X_0,c} \to k$. Its inverse image of the zero maximal ideal is $\mathfrak{m}_c$, because the support is c. Passing to residue fields therefore gives the canonical injective map

$$\mu_c(\bar{x}) : \kappa(c) = \mathcal{O}_{X_0,c}/\mathfrak{m}_c \hookrightarrow k.$$

It respects k_0 because $\bar{x}$ is a k_0-morphism. Define $\mu_c : \mathrm{Lift}(c) \to \mathrm{Emb}(c)$ by this canonical residue-field map; no affine-chart choice enters the Definition.

Lemma 15.7.13 (Residue embeddings and geometric lifts are inverse). *Uses:* *def: tracecore3-embedding-to-lift-map*, *def: tracecore3-lift-to-embedding-map*

Lean: *Weil.TraceCore3.embeddingLift_inverse*

For every $\iota \in \mathrm{Emb}(c)$ and $\bar{x} \in \mathrm{Lift}(c)$,

$$\mu_c(\lambda_c(\iota)) = \iota, \qquad \lambda_c(\mu_c(\bar{x})) = \bar{x}.$$

Proof. Uses: def:tracecore3-embedding-to-lift-map, def:tracecore3-lift-to-embedding-map, def:tracecore3-geometric-lift-from-embedding

On an affine neighbourhood $\operatorname{Spec} A$ of c, the coordinate map of $\lambda_c(\iota)$ is the quotient $A \to \kappa(c)$ followed by ι; uniqueness of factorization through the quotient gives the first identity. Conversely, the coordinate map of $\bar{x}$ is by definition the quotient followed by $\mu_c(\bar{x})$; the affine functor-of-points correspondence gives the second identity. Both identities are independent of the chosen chart by the residue-field universal property. □

Lemma 15.7.14 (Frobenius equivariance of residue lifts). *Uses:* *def: tracecore3-embedding-to-lift-map*, *def: tracecore3-scheme-geometric-frobenius*

Lean: *Weil.TraceCore3.embeddingLift_frobeniusEquivariant*

For $j \in \mathbf{N}$ and $\iota \in \mathrm{Emb}(c)$,

$$\Phi^j_{X_0}(|\lambda_c(\iota)|) = |\lambda_c(\Phi_q^j \circ \iota)|.$$

Proof. Uses: def:tracecore3-embedding-to-lift-map, def:tracecore3-geometric-lift-from-embedding, def:tracecore3-scheme-raw-frobenius-actions

On any affine neighbourhood of c, the coordinate map on the left is $\Phi_q^j \circ \iota$ after the residue quotient. This is exactly the coordinate map defining the lift on the right. The affine functor-of-points correspondence gives the equality. □

Lemma 15.7.15 (The geometric-lift carrier is nonempty). *Uses:* *def: tracecore3-geometric-lift-carrier*

Lean: *Weil.TraceCore3.geometricLiftCarrier_nonempty*

For every $c \in C_{X_0}$, $\mathrm{Lift}(c)$ is nonempty.

Proof. Uses: lemma:tracecore3-residue-embedding-nonempty, def:tracecore3-embedding-to-lift-map

Take an element ι of the nonempty embedding carrier and apply λ_c. Its value is an element of $\mathrm{Lift}(c)$. □

Definition 15.7.16 (Selected geometric lift of a closed point). Uses: def:tracecore3-geometric-lift-carrier

Constructed by: lemma:tracecore3-geometric-lift-carrier-nonempty

Lean: Weil.TraceCore3.selectedGeometricLift

For every $c \in C_{X_0}$, define $\widetilde{c} \in \mathrm{Lift}(c)$ to be the selected element obtained from the preceding nonemptiness lemma. Write $|\widetilde{c}| \in \Omega_{X_0}$ for its underlying geometric point.

Lemma 15.7.17 (Support of the selected geometric lift). *Uses:* *def: tracecore3-selected-geometric-lift*

Lean: *Weil.TraceCore3.selectedGeometricLift_support*

For every c, $\mathrm{sp}(|\widetilde{c}|) = c$.

Proof. Uses: def:tracecore3-selected-geometric-lift, def:tracecore3-geometric-lift-carrier

This is the subtype predicate carried by $\widetilde{c}$. □

Lemma 15.7.18 (Independence of the selected lift at the orbit-class level). *Uses:* *def: tracecore3-selected-geometric-lift*, *def: tracecore3-geometric-lift-carrier*, *def: tracecore3-scheme-locally-finite-periodic-input*, *def: tracecore3-orbit-quotient-map*

Lean: *Weil.TraceCore3.selectedLiftClass_independent*

For $c \in C_{X_0}$ and every $\bar{x} \in \mathrm{Lift}(c)$,

$$q_{X_0}(|\bar{x}|) = q_{X_0}(|\widetilde{c}|).$$

Proof. Uses: def:tracecore3-selected-geometric-lift, def:tracecore3-residue-embedding-carrier, def:tracecore3-geometric-lift-from-embedding, def:tracecore3-lift-to-embedding-map, def:tracecore3-scheme-geometric-frobenius, lemma:tracecore3-embedding-lift-inverse, lemma:tracecore3-embedding-lift-frobenius-equivariant, lemma:tracecore3-orbit-quotient-equality, lemma:tracecore3-closed-point-residue-finite, lemma:weil-arithmetic-frobenius-power-formula, lemma:weil-frobenius-inverse-identities

Put $L = \kappa(c)$ and $d = [L : k_0]$. On an affine neighbourhood of c, a geometric point with support c has a coordinate map whose kernel is the prime of c; passing to the fraction field of the quotient therefore gives a unique k_0-embedding $L \hookrightarrow k$. Apply this to $|\bar{x}|$ and $|\widetilde{c}|$, obtaining embeddings ι and ι_0.

The field L has q^d elements. For either embedding, its image is contained in the roots of $T^{q^d} - T$ and has q^d elements. The polynomial root bound therefore shows that both images are the same subfield

$$K_d := \{a \in k \mid a^{q^d} = a\}.$$

Corestricting the two embeddings to their common image gives field isomorphisms $\bar{\iota}, \bar{\iota}_0 : L \simeq K_d$. Thus

$$\tau := \bar{\iota} \circ \bar{\iota}_0^{-1} : K_d \simeq K_d$$

is a well-typed k_0-automorphism.

The restrictions $1, \varphi_q, \dots, \varphi_q^{d-1}$ to K_d are distinct: if $0 < r < d$ and φ_q^r were the identity, all q^d elements of K_d would be roots of $T^{q^r} - T$, contradicting the root bound. There are at most d k_0-embeddings of the degree-d separable extension K_d/k_0, so these d Frobenius powers exhaust its automorphisms. Hence $\iota = \varphi_q^j \circ \iota_0$ for some $0 \le j < d$. On K_d one has the explicit restricted identity $\Phi_q^{d-j}|_{K_d} = \varphi_q^j|_{K_d}$. The equivariance lemma therefore gives the compatibility

$$\Phi_{X_0}^{d-j}(\bar{c}_{\iota_0}) = \bar{c}_{\varphi_q^j \circ \iota_0} = \bar{c}_{\iota};$$

the exponent is read modulo the common period d. The embedding–lift inverse lemma identifies $|\bar{x}|$ and $|\widetilde{c}|$ with the two lifts in this display. They therefore lie in one geometric-Frobenius orbit, and the orbit-quotient equality criterion gives the displayed equality of classes. □

Definition 15.7.19 (Closed-point-to-orbit map). Uses: `def:tracecore3-selected-geometric-lift`, `def:tracecore3-scheme-locally-finite-periodic-input`, `def:tracecore3-orbit-quotient-map`

Lean: `Weil.TraceCore3.closedPointToOrbitMap`

Define the actual map

$$\alpha_{X_0} : C_{X_0} \longrightarrow \mathrm{Orb}_{\Phi_{X_0}}(\Omega_{X_0}), \qquad \alpha_{X_0}(c) := q_{X_0}(|\widetilde{c}|).$$

Definition 15.7.20 (Representative geometric point of a scheme orbit). Uses: `def:tracecore3-scheme-locally-finite-periodic-input`, `def:tracecore3-orbit-representative`

Lean: `Weil.TraceCore3.schemeOrbitRepresentative`

For an orbit $O \in \mathrm{Orb}_{\Phi_{X_0}}(\Omega_{X_0})$, define

$$\bar{x}_O := \rho(O) \in \Omega_{X_0}.$$

This is the specialization of the earlier quotient representative, not an unexplained map field.

Lemma 15.7.21 (The scheme representative has class O). Uses: `def:tracecore3-scheme-orbit-representative`, `def:tracecore3-orbit-quotient-map`

Lean: `Weil.TraceCore3.schemeOrbitRepresentative_class`

For every O, $q_{X_0}(\bar{x}_O) = O$.

Proof. Uses: `def:tracecore3-scheme-orbit-representative`, `lemma:tracecore3-orbit-representative-class`

This is the representative-class equation instantiated at the scheme input $\mathcal{D}_{X_0}$. □

Definition 15.7.22 (Scheme-theoretic image of an orbit representative). Uses: `def:tracecore3-scheme-orbit-representative`, `def:tracecore3-finite-type-scheme-data`

Lean: `Weil.TraceCore3.orbitRepresentativeSchemeImage`

For O, define

$$Z_O := \mathrm{SIm}_{X_0}(\bar{x}_O : \operatorname{Spec} k \to X_0),$$

the scheme-theoretic image of the displayed morphism. Also write $c_O := \mathrm{sp}(\bar{x}_O) \in |X_0|$ for its topological image.

Lemma 15.7.23 (The representative image point is closed). Uses: `def:tracecore3-orbit-representative-scheme-image`

Lean: `Weil.TraceCore3.orbitRepresentativeImage_closed`

For every O, the singleton $\{c_O\}$ is closed in $|X_0|$.

Proof. Uses: def:tracecore3-orbit-representative-scheme-image,lemma:tracecore3-scheme-return-time-fixes,def:tracecore3-finite-type-scheme-data,def:tracecore3-scheme-geometric-frobenius,lemma:weil-arithmetic-frobenius-power-formula,lemma:weil-frobenius-inverse-identities

Choose an affine neighbourhood $U = \operatorname{Spec} A$ of c_O and write $\theta : A \to k$ for the coordinate map of $\bar{x}_O$. The selected return-time equation gives an $r \geq 1$ with $\Phi_q^r \circ \theta = \theta$, hence also $\varphi_q^r \circ \theta = \theta$. Therefore every element of $\theta(A)$ is a root of $T^{q^r} - T$. The root bound makes this image finite. As a finite subring of the field k, it is a finite integral domain, hence a field. The first isomorphism theorem identifies $A/\ker\theta$ with $\theta(A)$, so $\ker\theta$ is maximal and the residue field at c_O is finite over k_0. The closed-point criterion for a finite-type scheme over a field now shows that c_O is closed. □

Lemma 15.7.24 (Underlying support of the representative scheme image). Uses: def:tracecore3-orbit-representative-scheme-image

Lean: Weil.TraceCore3.orbitRepresentativeSchemeImage_support

For every O, the underlying topological space of the scheme-theoretic image is

$$|Z_O| = \{c_O\}.$$

Proof. Uses: def:tracecore3-orbit-representative-scheme-image,lemma:tracecore3-orbit-representative-image-closed,def:tracecore3-scheme-closed-point-type

The point morphism $\bar{x}_O : \operatorname{Spec} k \to X_0$ induces a residue-field embedding $\kappa(c_O) \hookrightarrow k$, so it factors through the canonical point morphism $i_{c_O} : \operatorname{Spec}\kappa(c_O) \to X_0$. Since c_O is closed, i_{c_O} is a closed immersion. By the minimality of the scheme-theoretic image, Z_O is therefore a closed subscheme of this one-point closed subscheme, and hence $|Z_O| \subseteq \{c_O\}$. On the other hand $\bar{x}_O$ factors through Z_O, so its image point c_O belongs to $|Z_O|$. The two inclusions give the displayed equality. □

Definition 15.7.25 (Orbit-to-closed-point map). Uses: def:tracecore3-orbit-representative-scheme-image,def:tracecore3-scheme-closed-point-type

Constructed by: lemma:tracecore3-orbit-representative-image-closed

Lean: Weil.TraceCore3.orbitToClosedPointMap

Define

$$\beta_{X_0} : \operatorname{Orb}_{\Phi_{X_0}}(\Omega_{X_0}) \longrightarrow C_{X_0}, \qquad \beta_{X_0}(O) := (c_O, \{c_O\} \text{ closed}).$$

The closedness proof is supplied by the preceding lemma; β_{X_0} is not an input field.

Lemma 15.7.26 (Scheme-theoretic image is independent of an orbit representative). Uses: def:tracecore3-orbit-representative-scheme-image, def:tracecore3-orbit-quotient-map

Lean: Weil.TraceCore3.orbitSchemeImage_independent

If $q_{X_0}(\bar{y}) = O$, then

$$\operatorname{SIm}_{X_0}(\bar{y}) = Z_O.$$

Proof. Uses: def:tracecore3-orbit-representative-scheme-image,lemma:tracecore3-scheme-orbit-representative-class,lemma:tracecore3-orbit-quotient-equality,def:tracecore3-scheme-geometric-frobenius

The two quotient equations make $\bar{y}$ a Frobenius translate of $\bar{x}_O$. Thus, for a field automorphism $\sigma : k \simeq k$ given by an integer power of either of the inverse Frobenius automorphisms,

$$\bar{y} = \bar{x}_O \circ \operatorname{Spec}(\sigma).$$

The morphism $\operatorname{Spec}(\sigma)$ is an automorphism. Hence, for every closed subscheme $W \hookrightarrow X_0$, the point $\bar{x}_O$ factors through W if and only if $\bar{y}$ does: precompose a factorization with $\operatorname{Spec}(\sigma)$ in one direction and with its inverse in the other. The two point morphisms therefore have exactly the same closed-subscheme factorizations. Their minimal such closed subschemes, namely their scheme-theoretic images, are equal. The image of $\bar{x}_O$ is Z_O by definition. □

Lemma 15.7.27 (Beta is independent of an orbit representative). Uses: def:tracecore3-orbit-to-closed-point-map, def:tracecore3-orbit-quotient-map

Lean: Weil.TraceCore3.orbitToClosedPoint_wellDefined

If $q_{X_0}(\bar{y}) = O$, then

$$\operatorname{sp}(\bar{y}) = |\beta_{X_0}(O)|.$$

Proof. Uses: lemma:tracecore3-orbit-scheme-image-independent, lemma:tracecore3-orbit-representative-scheme-image-support, def:tracecore3-orbit-to-closed-point-map, def:tracecore3-orbit-representative-scheme-image

The scheme-image equality identifies the scheme-theoretic image of $\bar{y}$ with Z_O. By the representative-image lemma, the underlying space of Z_O is the singleton $\{c_O\}$. The image of the unique point of $\operatorname{Spec} k$ must therefore be c_O, so $\operatorname{sp}(\bar{y}) = c_O$. The underlying point of $\beta_{X_0}(O)$ is definitionally c_O, proving the claim. □

Lemma 15.7.28 (Closed-point/orbit left-inverse equation). *Uses:* def: tracecore3-closed-point-to-orbit-map, def: tracecore3-orbit-to-closed-point-map

Lean: Weil.TraceCore3.closedPointOrbit_leftInverse

For every $c \in C_{X_0}$,

$$\beta_{X_0}(\alpha_{X_0}(c)) = c.$$

Proof. Uses: def:tracecore3-closed-point-to-orbit-map, lemma:tracecore3-selected-geometric-lift-support, lemma:tracecore3-orbit-to-closed-point-well-defined

Apply representative independence for β_{X_0} to the representative $|\widetilde{c}|$ of $\alpha_{X_0}(c)$, then substitute the support equation $\operatorname{sp}(|\widetilde{c}|) = c$. Equality of the underlying points gives equality in the closed-point subtype. □

Lemma 15.7.29 (Closed-point/orbit right-inverse equation). *Uses:* def: tracecore3-closed-point-to-orbit-map, def: tracecore3-orbit-to-closed-point-map

Lean: Weil.TraceCore3.closedPointOrbit_rightInverse

For every $O \in \operatorname{Orb}_{\Phi_{X_0}}(\Omega_{X_0})$,

$$\alpha_{X_0}(\beta_{X_0}(O)) = O.$$

Proof. Uses: def:tracecore3-closed-point-to-orbit-map, lemma:tracecore3-selected-lift-class-independent, lemma:tracecore3-orbit-to-closed-point-well-defined, lemma:tracecore3-scheme-orbit-representative-class

The well-definedness equation puts $\bar{x}_O$ in $\operatorname{Lift}(\beta_{X_0}(O))$. Selected-lift class independence identifies its orbit with the selected lift used by α_{X_0}, and the representative-class equation identifies that orbit with O. Transitivity of equality gives the displayed inverse equation. □

Proposition 15.7.30 (Closed-point/orbit inverse pair). *Uses:* def: tracecore3-closed-point-to-orbit-map, def: tracecore3-orbit-to-closed-point-map

Lean: Weil.TraceCore3.closedPointOrbit_inverse

The maps α_{X_0} and β_{X_0} are mutually inverse.

Proof. Uses: lemma:tracecore3-closed-point-orbit-left-inverse, lemma:tracecore3-closed-point-orbit-right-inverse

The two preceding pointwise equations are the left- and right-inverse laws. □

Lemma 15.7.31 (Residue degree equals orbit degree). *Uses:* def: tracecore3-closed-point-to-orbit-map, def: tracecore3-scheme-closed-point-degree, def: tracecore3-orbit-degree

Lean: Weil.TraceCore3.closedPointOrbit_degree

For every $c \in C_{X_0}$,

$$\deg_{X_0}(c) = d_{\Phi_{X_0}}(\alpha_{X_0}(c)).$$

Proof. Uses: def:tracecore3-closed-point-to-orbit-map, def:tracecore3-residue-embedding-carrier, lemma:tracecore3-orbit-degree-representative, lemma:tracecore3-closed-point-residue-finite, def:tracecore3-selected-geometric-lift, def:tracecore3-lift-to-embedding-map, lemma:tracecore3-embedding-lift-inverse, lemma:tracecore3-embedding-lift-frobenius-equivariant, def:tracecore3-scheme-geometric-frobenius, lemma:weil-arithmetic-frobenius-power-formula, lemma:weil-frobenius-inverse-identities

Put $L = \kappa(c)$ and $d = [L : k_0]$, and let $\iota := \mu_c(\widetilde{c}) : L \hookrightarrow k$ be the residue-field embedding obtained by the explicit lift-to-embedding map. The inverse lemma identifies $|\widetilde{c}|$ with $|\lambda_c(\iota)|$. The finite field L has q^d elements, so $u^{q^d} = u$ for every $u \in L$. The arithmetic power formula shows that φ_q^d fixes $\iota(L)$, and the inverse identity shows the same for Φ_q^d. Frobenius equivariance of λ_c therefore shows that the selected lift returns after d geometric-Frobenius steps.

Suppose that it returned after an integer r with $0 < r < d$. Then Φ_q^r, and hence also φ_q^r, would fix every element of $\iota(L)$. All q^d elements of this image would consequently be roots of $T^{q^r} - T$, a nonzero polynomial of degree $q^r < q^d$, contradicting the polynomial root bound. Thus the least positive period of $|\widetilde{c}|$ is exactly d. The representative formula for orbit degree identifies this least period with $d_{\Phi_{X_0}}(q_{X_0}(|\widetilde{c}|))$, which is $d_{\Phi_{X_0}}(\alpha_{X_0}(c))$ by definition. □

Definition 15.7.32 (Degree-dividing closed-point type). Uses: def:tracecore3-closed-point-degree-family

Lean: Weil.TraceCore3.DegreeDividingClosedPointType

For $n \geq 1$, define

$$C_n := \{c : C \mid \deg_C(c) \mid n\}.$$

Lemma 15.7.33 (Degree-dividing closed-point type is finite). Uses: def:tracecore3-degree-dividing-closed-point-type

Lean: Weil.TraceCore3.degreeDividingClosedPoint_finite

For every $n \geq 1$, the type C_n is finite.

Proof. Uses: def:tracecore3-degree-dividing-closed-point-type,def:tracecore3-closed-point-degree-family

If $\deg_C(c) \mid n$, positivity gives $\deg_C(c) \leq n$. Thus C_n is a subtype of the finite degree-bounded set stored at the bound n, and is finite. □

15.8 Finite-extension points and the fixed-locus equivalence

Definition 15.8.1 (Canonical degree-n subfield of the algebraic closure). Uses: def:tracecore-arithmetic-conventions

Lean: Weil.TraceCore3.degreeNFixedSubfield

For $n \geq 1$, define

$$k_n := \{a \in k \mid \varphi_q^n(a) = a\}.$$

It is a subfield of k, with inclusion $j_n : k_n \hookrightarrow k$.

Lemma 15.8.2 (Cardinality of the canonical degree-n subfield). Uses: def:tracecore3-degree-n-fixed-subfield

Lean: Weil.TraceCore3.degreeNFixedSubfield_cardinality

For every $n \geq 1$, the field k_n is finite and

$$\#k_n = q^n.$$

Proof. Uses: def:tracecore3-degree-n-fixed-subfield,def:tracecore-arithmetic-conventions,lemma:weil-arithmetic-frobenius-power-formula

Iterating the arithmetic-Frobenius power formula gives

$$\varphi_q^n(a) = a^{q^n} \qquad (a \in k).$$

Thus the underlying set of k_n is exactly the root set in k of

$$f_n(T) := T^{q^n} - T \in k_0[T].$$

The selected field k is algebraically closed, so the monic polynomial f_n splits completely in $k[T]$. Moreover, because $q = p^a$ and k has characteristic p,

$$f_n'(T) = q^n T^{q^n - 1} - 1 = -1.$$

Hence f_n is squarefree. A split squarefree polynomial of degree q^n has exactly q^n distinct roots, proving both finiteness and the displayed cardinality. The root set is closed under the field operations because it is the fixed set of the field automorphism φ_q^n; therefore it is precisely the canonical subfield of k with q^n elements. □

Definition 15.8.3 (Finite-extension rational-point type). Uses: def:tracecore3-degree-n-fixed-subfield, def:tracecore3-finite-type-scheme-data

Constructed by: lemma:tracecore3-degree-n-fixed-subfield-cardinality

Lean: Weil.TraceCore3.SchemeFiniteExtensionPointType

For $n \geq 1$, define

$$R_{X_0,n} := \mathrm{Hom}_{k_0}(\mathrm{Spec}\, k_n, X_0).$$

This is the literal type denoted by $X_0(\mathbf{F}_{q^n})$ in the counting formula.

Lemma 15.8.4 (Finiteness of the finite-extension rational-point type). Uses: def:tracecore3-scheme-finite-extension-point-type

Lean: Weil.TraceCore3.schemeFiniteExtensionPoint_finite

For every $n \geq 1$, $R_{X_0,n}$ is finite.

Proof. Uses: def:tracecore3-scheme-finite-extension-point-type,def:tracecore3-finite-type-scheme-data,lemma:tracecore3-degree-n-fixed-subfield-cardinality

Choose a finite affine cover $X_0 = \bigcup_{i=1}^{s} \operatorname{Spec} A_i$ and, for every i, a finite list of k_0-algebra generators of A_i. A morphism $y : \operatorname{Spec} k_n \to X_0$ has a unique image point, which lies in some member of this cover; because the inverse image of that open is all of $\operatorname{Spec} k_n$, the morphism factors through the chart. On the i-th chart it is therefore determined by a k_0-algebra homomorphism $A_i \to k_n$, and such a homomorphism is determined by the images of the selected generators. The preceding lemma makes k_n a finite set, so only finitely many generator tuples, and hence only finitely many homomorphisms, occur on each chart. A finite union over the chosen affine cover proves that $R_{X_0,n}$ is finite. □

Definition 15.8.5 (Finite-extension family constructed from the scheme). Uses: def:tracecore3-finite-extension-point-family,def:tracecore3-scheme-finite-extension-point-type,def:tracecore3-scheme-locally-finite-periodic-input

Constructed by: lemma:tracecore3-scheme-finite-extension-point-finite

Lean: Weil.TraceCore3.schemeFiniteExtensionPointFamily

Define $\mathcal{R}_{X_0}$ by taking its degree- n type to be $R_{X_0,n}$ and its point number to be $N_n(X_0) := \#R_{X_0,n}$.

Definition 15.8.6 (Scalar extension of a finite-extension point). Uses: def:tracecore3-scheme-finite-extension-point-type,def:tracecore3-scheme-geometric-point-data

Lean: Weil.TraceCore3.rationalPointScalarExtension

For $y : \operatorname{Spec} k_n \to X_0$, define

$$s_n(y) := y \circ \operatorname{Spec}(j_n) : \operatorname{Spec} k \to X_0.$$

Definition 15.8.7 (A finite-extension point lies over a closed point). Uses: def:tracecore3-rational-point-scalar-extension,def:tracecore3-scheme-geometric-point-data,def:tracecore3-scheme-closed-point-type

Lean: Weil.Etale.Local09PointLiesOverClosedPoint

For $n \geq 1$, $y \in R_{X_0,n}$, and $c \in C_{X_0}$, define

$$\operatorname{LiesOver}_{X_0,n}(y, c) \quad :\Longleftrightarrow \quad \operatorname{sp}(s_n(y)) = c,$$

where the equality is in the underlying point space of X_0.

Lemma 15.8.8 (Scalar extension is fixed by the nth geometric iterate). Uses: def:tracecore3-rational-point-scalar-extension,def:tracecore3-scheme-fixed-locus-subtype

Lean: Weil.TraceCore3.rationalPointScalarExtension_fixed

For every $y \in R_{X_0,n}$,

$$\Phi^n_{X_0}(s_n(y)) = s_n(y).$$

Proof. Uses: def:tracecore3-rational-point-scalar-extension,def:tracecore3-degree-n-fixed-subfield,def:tracecore3-scheme-geometric-frobenius,lemma:weil-frobenius-inverse-identities

Choose an affine open $U = \operatorname{Spec} A$ containing the image of y, and write $\eta : A \to k_n$ for its coordinate map. The coordinate map of $s_n(y)$ is $j_n \circ \eta$. By definition of the geometric point action, the coordinate map of its n-th iterate is

$$\Phi_q^n \circ j_n \circ \eta.$$

For $a \in k_n$, the defining predicate gives $\varphi_q^n(j_n(a)) = j_n(a)$. Since Φ_q^n is inverse to φ_q^n, applying Φ_q^n to this equality gives $\Phi_q^n(j_n(a)) = j_n(a)$. Hence $\Phi_q^n \circ j_n \circ \eta = j_n \circ \eta$. The affine functor-of-points correspondence, followed by the inclusion $U \hookrightarrow X_0$, gives the required equality of geometric points. □

Theorem 15.8.9 (A rational point can lie only over a degree dividing the extension degree). Uses: def:weil-efi-local09-point-lies-over-closed-point,def:tracecore3-scheme-closed-point-degree

Lean: Weil.Etale.local09Degree_dvd

Let $n \geq 1$, $y \in R_{X_0,n}$, *and* $c \in C_{X_0}$. *If* $\operatorname{LiesOver}_{X_0,n}(y, c)$, *then*

$$\deg_{X_0}(c) \mid n.$$

Equivalently, there is $r \in \mathbf{N}$ *such that* $n = \deg_{X_0}(c) r$.

Proof. Uses: def:weil-efi-local09-point-lies-over-closed-point,def:tracecore3-geometric-lift-carrier,def:tracecore3-closed-point-to-orbit-map,lemma:tracecore3-rational-point-scalar-extension-fixed,lemma:tracecore3-selected-lift-class-independent,lemma:tracecore3-orbit-fixed-iff-degree-divides,lemma:tracecore3-closed-point-orbit-degree

Put $\bar{y} = s_n(y)$. The lies-over equation makes $\widehat{y} := (\bar{y}, \mathrm{sp}(\bar{y}) = c)$ an element of $\mathrm{Lift}(c)$. Selected-lift independence therefore gives

$$q_{X_0}(\bar{y}) = q_{X_0}(|\widetilde{c}|) = \alpha_{X_0}(c).$$

The scalar-extension fixed-point lemma gives $\Phi^n_{X_0}(\bar{y}) = \bar{y}$. Applying the fixed-iterate/orbit-degree criterion to $\bar{y}$ yields

$$d_{\Phi_{X_0}}(q_{X_0}(\bar{y})) \mid n.$$

Replace its orbit class by $\alpha_{X_0}(c)$, and then replace that orbit degree by $\deg_{X_0}(c)$ using the closed-point/orbit degree theorem. This proves the divisibility statement; unfolding divisibility supplies the displayed witness r. □

Definition 15.8.10 (Forward rational-point-to-fixed-point map). Uses: `def:tracecore3-rational-point-scalar-extension`, `def:tracecore3-scheme-fixed-locus-subtype`

Constructed by: `lemma:tracecore3-rational-point-scalar-extension-fixed`

Lean: `Weil.TraceCore3.rationalToFixedMap`

Define

$$u_n : R_{X_0,n} \longrightarrow \mathrm{Fix}_{X_0}(n), \qquad u_n(y) := (s_n(y), \Phi^n_{X_0} s_n(y) = s_n(y)).$$

Lemma 15.8.11 (Existence of descent for a fixed geometric point). *Uses:* `def: tracecore3-rational-point-scalar-extension`, `def: tracecore3-scheme-fixed-locus-subtype`, `def: tracecore3-scheme-finite-extension-point-type`

Lean: `Weil.TraceCore3. fixedPointDescent_ exists`

For every $z \in \mathrm{Fix}_{X_0}(n)$, there is $y \in R_{X_0,n}$ such that

$$s_n(y) = |z|.$$

Proof. Uses: `def:tracecore3-rational-point-scalar-extension`, `def:tracecore3-degree-n-fixed-subfield`, `def:tracecore3-scheme-fixed-locus-subtype`, `def:tracecore3-scheme-geometric-frobenius`, `lemma:weil-frobenius-inverse-identities`

Let x be the support of $|z|$, choose one affine neighbourhood $U = \operatorname{Spec} A$ of x, and factor $|z|$ through U. This uses only that $\operatorname{Spec} k$ has one point: the inverse image of U is then the whole source. Write $\theta : A \to k$ for the coordinate map. The Frobenius translate has the same support and also factors through U. Since the open immersion $U \hookrightarrow X_0$ is a monomorphism, the fixed-point equality in X_0 lifts to the equality of the two U-valued factorizations. On coordinate rings this is $\Phi^n_q \circ \theta = \theta$. Composing with φ^n_q and using the inverse identity yields $\varphi^n_q \circ \theta = \theta$. Therefore $\theta(a) \in k_n$ for every $a \in A$.

Define

$$\eta : A \longrightarrow k_n, \qquad a \longmapsto \big(\theta(a),\ \varphi^n_q(\theta(a)) = \theta(a)\big).$$

Because the operations on k_n are inherited from k, the ring and k_0-algebra identities for η follow componentwise from those for θ. Thus η defines a morphism $y_U : \operatorname{Spec} k_n \to U$; compose it with $U \hookrightarrow X_0$ to obtain $y \in R_{X_0,n}$. By construction $j_n \circ \eta = \theta$, so scalar extension of y has exactly the coordinate map of $|z|$. Hence $s_n(y) = |z|$. No affine gluing is needed, since the source spectrum has a single point. □

Lemma 15.8.12 (Uniqueness of descent for a fixed geometric point). *Uses:* `def: tracecore3-rational-point-scalar-extension`, `def: tracecore3-scheme-finite-extension-point-type`

Lean: `Weil.TraceCore3. fixedPointDescent_ unique`

If $y_1, y_2 \in R_{X_0,n}$ and $s_n(y_1) = s_n(y_2)$, then $y_1 = y_2$.

Proof. Uses: `def:tracecore3-rational-point-scalar-extension`, `def:tracecore3-degree-n-fixed-subfield`

The equality after scalar extension first implies that y_1 and y_2 have the same image point $x \in |X_0|$. Choose an affine neighbourhood $U = \operatorname{Spec} A$ of x. Since $\operatorname{Spec} k_n$ has one point, both morphisms factor through U; write their coordinate maps as $\eta_1, \eta_2 : A \to k_n$. Their scalar extensions also factor through U. Because the open immersion $U \hookrightarrow X_0$ is a monomorphism, the equality $s_n(y_1) = s_n(y_2)$ lifts to equality of these two U-valued scalar extensions. On coordinate rings this is

$$j_n \circ \eta_1 = j_n \circ \eta_2 : A \longrightarrow k.$$

The inclusion $j_n : k_n \hookrightarrow k$ is injective, hence $\eta_1(a) = \eta_2(a)$ for every $a \in A$. Function extensionality gives $\eta_1 = \eta_2$, and the affine functor-of-points correspondence gives equality of the two factorizations through U. Composing with the open immersion gives $y_1 = y_2$. □

Definition 15.8.13 (Reverse fixed-point-to-rational-point map). Uses: def:tracecore3-rational-point-scalar-extension,def:tracecore3-scheme-fixed-locus-subtype,def:tracecore3-scheme-finite-extension-point-type

Constructed by: lemma:tracecore3-fixed-point-descent-exists

Lean: Weil.TraceCore3.fixedToRationalMap

For $z \in \mathrm{Fix}_{X_0}(n)$, define $v_n(z)$ to be the selected $y \in R_{X_0,n}$ satisfying $s_n(y) = |z|$, supplied by the descent-existence lemma.

Lemma 15.8.14 (Specification of the selected descended point). Uses: def:tracecore3-fixed-to-rational-map

Lean: Weil.TraceCore3.fixedToRationalMap_spec

For every z, $s_n(v_n(z)) = |z|$.

Proof. Uses: def:tracecore3-fixed-to-rational-map,lemma:tracecore3-fixed-point-descent-exists

This is the property of the selected witness in the definition of v_n. □

Lemma 15.8.15 (Rational-to-fixed-to-rational inverse equation). Uses: def:tracecore3-rational-to-fixed-map,def:tracecore3-fixed-to-rational-map

Lean: Weil.TraceCore3.rationalFixed_leftInverse

For every $y \in R_{X_0,n}$,

$$v_n(u_n(y)) = y.$$

Proof. Uses: lemma:tracecore3-fixed-to-rational-map-spec,lemma:tracecore3-fixed-point-descent-unique,def:tracecore3-rational-to-fixed-map

Both $v_n(u_n(y))$ and y have scalar extension $s_n(y)$. Apply the descent-uniqueness lemma to obtain the displayed equality. □

Lemma 15.8.16 (Fixed-to-rational-to-fixed inverse equation). Uses: def:tracecore3-rational-to-fixed-map,def:tracecore3-fixed-to-rational-map

Lean: Weil.TraceCore3.rationalFixed_rightInverse

For every $z \in \mathrm{Fix}_{X_0}(n)$,

$$u_n(v_n(z)) = z.$$

Proof. Uses: lemma:tracecore3-fixed-to-rational-map-spec,def:tracecore3-rational-to-fixed-map,def:tracecore3-scheme-fixed-locus-subtype

The two subtype elements have equal underlying geometric points by the specification of v_n. Subtype extensionality reduces their equality to that underlying equality and proves the claim. □

Definition 15.8.17 (Finite-extension points equivalent to the fixed locus). Uses: def:tracecore3-rational-to-fixed-map,def:tracecore3-fixed-to-rational-map

Constructed by: lemma:tracecore3-rational-fixed-left-inverse,lemma:tracecore3-rational-fixed-right-inverse

Lean: Weil.TraceCore3.rationalFixedEquivalence

Define

$$e_n : R_{X_0,n} \simeq \mathrm{Fix}_{X_0}(n)$$

with forward map u_n, inverse map v_n, and the two preceding inverse equations.

15.9 Scheme Frobenius after finite base-field extension

Status. The construction of the degree-one scheme Frobenius and its descended-affine formula remain owned by SCHFROB–05 and SCHFROB–06. This section proves the two consequences requested by SCHFROB–13 and SCHFROB–14. The fixed-point proof reuses the closed TraceCore3 finite-field descent theorem and adds the comparison between the scheme-map convention and the geometric point-action convention.

Definition 15.9.1 (Degree-m base-extension input). Uses: def:weil-prime-power-data,def:weil-finite-field-data,def:weil-descended-scheme-data,def:weil-scheme-frobenius-candidate,def:weil-scheme-frobenius-affine-formula,def:tracecore3-degree-n-fixed-subfield

Constructed by: lemma:tracecore3-degree-n-fixed-subfield-cardinality

Lean: conditional-interface:Weil.Etale.SchFrob13BaseExtensionInput

Fix a positive integer m, and put

$$k_m := \{c \in k \mid \varphi_q^m(c) = c\}.$$

Thus k_m is the selected copy of $\mathbf{F}_{q^m}$ inside k. Put

$$X_{0,m} := X_0 \times_{\operatorname{Spec} k_0} \operatorname{Spec} k_m.$$

The cartesian square in the descended scheme datum gives the canonical associativity isomorphism

$$\iota_m : X_{0,m} \times_{\operatorname{Spec} k_m} \operatorname{Spec} k \xrightarrow{\sim} X.$$

A degree-m base-extension input consists of a scheme Frobenius candidate $F_X^{\mathrm{sch}} : X \to X$, its descended-affine q-power certificate, and an endomorphism

$$F_{X,m}^{\mathrm{bc}} : X \longrightarrow X$$

obtained, through ι_m, by base-changing the relative q^m-power endomorphism of $X_{0,m}/k_m$. The input records its affine formula: for every $U_0 = \operatorname{Spec} A_0 \subseteq X_0$, the restrictions of the two endomorphisms to $U = U_0 \times_{k_0} k = \operatorname{Spec}(A_0 \otimes_{k_0} k)$ have inverse-image maps

$$a \otimes c \longmapsto a^q \otimes c, \qquad a \otimes c \longmapsto a^{q^m} \otimes c,$$

respectively. This is a conditional input whose existence fields are the outputs of the preceding scheme-Frobenius construction obligations.

Definition 15.9.2 (Affine scalar-fixed Frobenius power). Uses: def:weil-schfrob13-base-extension-input

Lean: conditional-interface:Weil.Etale.schFrob13AffineScalarFixedPower

For a descended affine $U = \operatorname{Spec}(A_0 \otimes_{k_0} k)$ and $r \in \mathbf{N}$, define the k-algebra endomorphism

$$\gamma_{U,r} : A_0 \otimes_{k_0} k \longrightarrow A_0 \otimes_{k_0} k, \qquad \gamma_{U,r}(a \otimes c) = a^{q^r} \otimes c.$$

The map $a \mapsto a^{q^r}$ fixes k_0, and the tensor-product universal property gives the displayed k-algebra endomorphism.

Lemma 15.9.3 (Composition law for affine scalar-fixed powers). Uses: def:weil-schfrob13-affine-scalar-fixed-power

Lean: conditional-interface:Weil.Etale.schFrob13AffineScalarFixedPower_comp

For $r, s \in \mathbf{N}$,

$$\gamma_{U,r} \circ \gamma_{U,s} = \gamma_{U,r+s}.$$

Proof. Uses: def:weil-schfrob13-affine-scalar-fixed-power

On a pure tensor the left side is

$$\gamma_{U,r}(a^{q^s} \otimes c) = (a^{q^s})^{q^r} \otimes c = a^{q^{r+s}} \otimes c.$$

This is the pure-tensor formula for $\gamma_{U,r+s}$. Tensor-product extensionality gives equality of the two k-algebra maps. □

Lemma 15.9.4 (Ordinary scheme powers on descended affine charts). Uses: def:weil-schfrob13-base-extension-input, def:weil-scheme-endomorphism-ordinary-power, def:weil-schfrob13-affine-scalar-fixed-power

Lean: conditional-interface:Weil.Etale.schFrob13OrdinaryPower_affine

The inverse-image map of $(F_X^{\mathrm{sch}})^{\langle r \rangle}|_U$ is $\gamma_{U,r}$ for every $r \in \mathbf{N}$.

Proof. Uses: def:weil-schfrob13-base-extension-input,lemma:weil-scheme-endomorphism-ordinary-power-zero,lemma:weil-scheme-endomorphism-ordinary-power-succ,lemma:weil-schfrob13-affine-scalar-fixed-power-composition

Induct on r. In degree zero, the empty scheme-map power is the identity, whose inverse-image map is $\gamma_{U,0}$. For the successor, the ordinary power recursion is

$$(F_X^{\mathrm{sch}})^{\langle r+1 \rangle} = F_X^{\mathrm{sch}} \circ (F_X^{\mathrm{sch}})^{\langle r \rangle}.$$

Contravariance reverses this composite on coordinate rings. The induction hypothesis and the degree-one affine certificate give

$$((F_X^{\mathrm{sch}})^{\langle r+1 \rangle}|_U)^{\#} = \gamma_{U,r} \circ \gamma_{U,1} = \gamma_{U,r+1}.$$

The last equality is the affine composition lemma. □

Lemma 15.9.5 (Chartwise base-extension comparison). *Uses:* *def: weil-schfrob13-base-extension-input*, *def: weil-schfrob13-affine-scalar-fixed-power*, *def: weil-scheme-endomorphism-ordinary-power*

Lean: *conditional-interface: Weil.Etale.schFrob13BaseExtension_chart*

For every descended affine $U \subseteq X$,

$$F^{\mathrm{bc}}_{X,m}|_U = (F^{\mathrm{sch}}_X)^{\langle m \rangle}|_U.$$

Proof. Uses: def:weil-schfrob13-base-extension-input, lemma:weil-schfrob13-ordinary-power-affine-formula

The base-extension certificate identifies the inverse-image map on U with $\gamma_{U,m}$. The ordinary-power affine lemma gives the same map for the right side. The affine scheme–ring anti-equivalence turns equality of these coordinate maps into equality of the restricted scheme morphisms. □

Proposition 15.9.6 (Global base-extension comparison). *Uses:* *def: weil-schfrob13-base-extension-input*, *def: weil-scheme-endomorphism-ordinary-power*

Lean: *conditional-interface: Weil.Etale.schFrob13BaseExtension_global*

For every degree-m base-extension input,

$$F^{\mathrm{bc}}_{X,m} = (F^{\mathrm{sch}}_X)^{\langle m \rangle}.$$

Proof. Uses: lemma:weil-schfrob13-base-extension-chart-comparison, def:weil-descended-scheme-data

Take an affine open cover $X_0 = \bigcup_i U_{0,i}$. Base change gives the open cover $X = \bigcup_i (U_{0,i} \times_{k_0} k)$. The chartwise comparison identifies the restrictions of the two global morphisms on every member of this source cover. Equality of scheme morphisms is local on the source, so the two global endomorphisms are equal. □

Definition 15.9.7 (SCHFROB–13 comparison predicate). Uses: def:weil-schfrob13-base-extension-input, def:weil-scheme-frobenius-iterate

Lean: conditional-interface:Weil.Etale.IsSchFrob13BaseExtensionIterate

Define the degree-m base-extension comparison predicate to be

$$F^{\mathrm{bc}}_{X,m} = (F^{\mathrm{sch}}_X)^{[m]}.$$

Theorem 15.9.8 (SCHFROB–13 discharge). *Uses:* *def: weil-schfrob13-base-extension-iterate-predicate*

Lean: *conditional-interface: Weil.Etale.schFrob13*

The degree-m base-extension comparison predicate holds for every degree-m base-extension input.

Proof. Uses: prop:weil-schfrob13-base-extension-global-comparison, prop:weil-scheme-frobenius-iterate-ordinary-power

The global comparison identifies the base-extension map with the ordinary categorical m-fold power. The bracket-to-ordinary-power proposition identifies that power with $(F^{\mathrm{sch}}_X)^{[m]}$. Substitution is the defining equality of the comparison predicate. □

15.10 Scheme-map fixed points and finite-extension points

Definition 15.10.1 (SCHFROB–14 fixed-point input). Uses: def:weil-descended-scheme-data, def:weil-scheme-frobenius-candidate, def:weil-scheme-frobenius-affine-formula, def:tracecore3-finite-type-scheme-data

Lean: conditional-interface:Weil.Etale.SchFrob14FixedPointInput

A SCHFROB–14 input consists of a descended scheme datum, a finite-type certificate for $X_0 \to \operatorname{Spec} k_0$, a scheme Frobenius candidate F^{sch}_X, and its descended-affine q-power certificate. The finite-type certificate makes the same X_0 an instance of the TraceCore3 finite-type scheme datum.

Definition 15.10.2 (Geometric scheme-point type). Uses: def:weil-schfrob14-fixed-point-input

Lean: conditional-interface:Weil.Etale.SchFrob14GeometricSchemePoint

Define

$$X(k) := \operatorname{Hom}_k(\operatorname{Spec} k, X).$$

Definition 15.10.3 (Projection to a descended geometric point). Uses: def:weil-schfrob14-geometric-scheme-point-type,def:tracecore3-scheme-geometric-point-data

Lean: conditional-interface:Weil.Etale.schFrob14SchemeToDescendedPoint

For $x : \operatorname{Spec} k \to X$, define

$$b(x) := \pi \circ x : \operatorname{Spec} k \longrightarrow X_0.$$

The composite is a k_0-morphism and hence belongs to the TraceCore3 geometric-point type Ω_{X_0}.

Definition 15.10.4 (Cartesian lift of a descended geometric point). Uses: def:weil-schfrob14-fixed-point-input,def:tracecore3-scheme-geometric-point-data

Lean: conditional-interface:Weil.Etale.schFrob14DescendedToSchemePoint

For $\bar{x} : \operatorname{Spec} k \to X_0$, define

$$a(\bar{x}) : \operatorname{Spec} k \longrightarrow X$$

as the unique arrow supplied by the cartesian square for the compatible pair $(\bar{x}, \mathrm{id}_{\operatorname{Spec} k})$. It is a k-point of X.

Lemma 15.10.5 (Projection after cartesian lift). Uses: def:weil-schfrob14-scheme-to-descended-point, def:weil-schfrob14-descended-to-scheme-point

Lean: conditional-interface:Weil.Etale.schFrob14DescendedPoint_leftInverse

For every $\bar{x} \in \Omega_{X_0}$,

$$b(a(\bar{x})) = \bar{x}.$$

Proof. Uses: def:weil-schfrob14-scheme-to-descended-point,def:weil-schfrob14-descended-to-scheme-point

The first projection equation is a field of the cartesian lift. It is the displayed equality after unfolding b. □

Lemma 15.10.6 (Cartesian lift after projection). Uses: def:weil-schfrob14-scheme-to-descended-point, def:weil-schfrob14-descended-to-scheme-point

Lean: conditional-interface:Weil.Etale.schFrob14DescendedPoint_rightInverse

For every $x \in X(k)$,

$$a(b(x)) = x.$$

Proof. Uses: def:weil-schfrob14-scheme-to-descended-point,def:weil-schfrob14-descended-to-scheme-point,def:weil-descended-scheme-data

Both arrows to X have first projection $\pi \circ x$ and second projection $\mathrm{id}_{\operatorname{Spec} k}$. Uniqueness in the cartesian square identifies them. □

Definition 15.10.7 (Descended-point equivalence). Uses: def:weil-schfrob14-scheme-to-descended-point,def:weil-schfrob14-descended-to-scheme-point

Constructed by: lemma:weil-schfrob14-descended-point-left-inverse,lemma:weil-schfrob14-descended-point-right-inverse

Lean: conditional-interface:Weil.Etale.schFrob14DescendedPointEquiv

Define

$$b_X : X(k) \xrightarrow{\sim} \Omega_{X_0}$$

with forward map b, inverse map a, and the two cartesian inverse equations.

Definition 15.10.8 (Scheme-map fixed-point subtype). Uses: def:weil-schfrob14-geometric-scheme-point-type,def:weil-scheme-frobenius-iterate

Lean: conditional-interface:Weil.Etale.SchFrob14SchemeFixedPointSubtype

For $m \geq 1$, define

$$\mathrm{Fix}_X^{\mathrm{sch}}(m) := \{x \in X(k) \mid (F_X^{\mathrm{sch}})^{[m]} \circ x = x\}.$$

Lemma 15.10.9 (One-step scheme action is arithmetic on descended points). Uses: def:weil-schfrob14-descended-point-equivalence, def:tracecore3-scheme-raw-frobenius-actions, def:weil-schfrob14-fixed-point-input

Lean: conditional-interface:Weil.Etale.schFrob14OneStepActionComparison

For $x \in X(k)$,

$$b_X(F_X^{\mathrm{sch}} \circ x) = \varphi_{X_0}^{\mathrm{raw}}(b_X(x)).$$

Proof. Uses: def:weil-schfrob14-fixed-point-input,def:weil-schfrob14-descended-point-equivalence,def:tracecore3-scheme-raw-frobenius-actions,lemma:weil-arithmetic-frobenius-power-formula

Let $U_0 = \operatorname{Spec} A_0$ be an affine neighbourhood of the image of $b_X(x)$. The point factors through U_0; write its coordinate map as $\theta : A_0 \to k$. The affine certificate for the scheme Frobenius shows that the left side has coordinate map

$$a \longmapsto \theta(a^q) = \theta(a)^q.$$

The arithmetic-Frobenius power formula identifies this with $\varphi_q \circ \theta$, which is the coordinate map of the right side by the raw-action definition. The affine functor-of-points correspondence and the open immersion $U_0 \hookrightarrow X_0$ give the asserted equality. □

Lemma 15.10.10 (Iterated scheme action is iterated arithmetic action). Uses: def:weil-schfrob14-descended-point-equivalence, def:weil-scheme-frobenius-iterate, def:tracecore3-scheme-raw-frobenius-actions

Lean: conditional-interface:Weil.Etale.schFrob14IterateActionComparison

For $r \in \mathbf{N}$ and $x \in X(k)$,

$$b_X((F_X^{\mathrm{sch}})^{[r]} \circ x) = (\varphi_{X_0}^{\mathrm{raw}})^r(b_X(x)).$$

Proof. Uses: lemma:weil-scheme-frobenius-iterate-zero,lemma:weil-scheme-frobenius-iterate-succ,lemma:weil-schfrob14-one-step-action-comparison

Induct on r. The zero clause reduces both sides to $b_X(x)$. In the successor step, associativity rewrites the scheme side as

$$b_X\big(F_X^{\mathrm{sch}} \circ ((F_X^{\mathrm{sch}})^{[r]} \circ x)\big).$$

Apply the one-step comparison to the inner k-point and then substitute the induction hypothesis. This is the successor recursion for the iterate of $\varphi_{X_0}^{\mathrm{raw}}$. □

Lemma 15.10.11 (Arithmetic and geometric fixed predicates agree). Uses: def:tracecore3-scheme-raw-frobenius-actions

Lean: conditional-interface:Weil.Etale.schFrob14ArithmeticGeometricFixedIff

For $r \in \mathbf{N}$ and $\bar{x} \in \Omega_{X_0}$,

$$(\varphi_{X_0}^{\mathrm{raw}})^r(\bar{x}) = \bar{x} \quad \Longleftrightarrow \quad (\Phi_{X_0}^{\mathrm{raw}})^r(\bar{x}) = \bar{x}.$$

Proof. Uses: lemma:tracecore3-scheme-raw-frobenius-inverse

The r-fold iterates of the two raw actions are inverse maps. If the arithmetic iterate fixes $\bar{x}$, apply the geometric iterate to that equation; cancellation gives $\bar{x} = (\Phi_{X_0}^{\mathrm{raw}})^r(\bar{x})$, and symmetry gives the displayed orientation. The reverse implication is obtained by applying the arithmetic iterate and using the other inverse equation. □

Definition 15.10.12 (Scheme-fixed to TraceCore-fixed map). Uses: def:weil-schfrob14-scheme-fixed-point-subtype,def:tracecore3-scheme-fixed-locus-subtype,def:weil-schfrob14-descended-point-equivalence

Constructed by: lemma:weil-schfrob14-iterate-action-comparison,lemma:weil-schfrob14-arithmetic-geometric-fixed-iff

Lean: conditional-interface:Weil.Etale.schFrob14SchemeFixedToTracecoreFixed

For $z = (x, h) \in \mathrm{Fix}_X^{\mathrm{sch}}(m)$, define

$$B_m(z) := \big(b_X(x), (\Phi_{X_0}^{\mathrm{raw}})^m(b_X(x)) = b_X(x)\big).$$

The fixedness proof is obtained by transporting h through the iterated action comparison and then through the arithmetic–geometric fixed-predicate equivalence.

Definition 15.10.13 (TraceCore-fixed to scheme-fixed map). Uses: def:weil-schfrob14-scheme-fixed-point-subtype,def:tracecore3-scheme-fixed-locus-subtype,def:weil-schfrob14-descended-point-equivalence

Constructed by: lemma:weil-schfrob14-iterate-action-comparison,lemma:weil-schfrob14-arithmetic-geometric-fixed-iff

Lean: conditional-interface:Weil.Etale.schFrob14TracecoreFixedToSchemeFixed

For $w = (\bar{x}, h) \in \mathrm{Fix}_{X_0}(m)$, define

$$A_m(w) := \big(b_X^{-1}(\bar{x}), (F_X^{\mathrm{sch}})^{[m]} \circ b_X^{-1}(\bar{x}) = b_X^{-1}(\bar{x})\big).$$

The fixedness proof applies injectivity of b_X, the iterated action comparison, and the reverse direction of the fixed-predicate equivalence.

Lemma 15.10.14 (Scheme-fixed bridge left inverse). *Uses:* *def:weil-schfrob14-scheme-fixed-to-tracecore-fixed*, *def:weil-schfrob14-tracecore-fixed-to-scheme-fixed*

Lean: *conditional-interface:Weil.Etale.schFrob14FixedBridge_leftInverse*

For $z \in \mathrm{Fix}_X^{\mathrm{sch}}(m)$,

$$A_m(B_m(z)) = z.$$

Proof. Uses: def:weil-schfrob14-scheme-fixed-to-tracecore-fixed,def:weil-schfrob14-tracecore-fixed-to-scheme-fixed,lemma:weil-schfrob14-descended-point-right-inverse

Subtype extensionality reduces the assertion to the underlying k-point. That equality is $b_X^{-1}(b_X(x)) = x$, the right inverse equation for the descended-point equivalence. □

Lemma 15.10.15 (Scheme-fixed bridge right inverse). *Uses:* *def:weil-schfrob14-scheme-fixed-to-tracecore-fixed*, *def:weil-schfrob14-tracecore-fixed-to-scheme-fixed*

Lean: *conditional-interface:Weil.Etale.schFrob14FixedBridge_rightInverse*

For $w \in \mathrm{Fix}_{X_0}(m)$,

$$B_m(A_m(w)) = w.$$

Proof. Uses: def:weil-schfrob14-scheme-fixed-to-tracecore-fixed,def:weil-schfrob14-tracecore-fixed-to-scheme-fixed,lemma:weil-schfrob14-descended-point-left-inverse

Subtype extensionality reduces the assertion to the underlying descended geometric point. That equality is $b_X(b_X^{-1}(\bar{x})) = \bar{x}$, the left inverse equation for the descended-point equivalence. □

Definition 15.10.16 (Scheme-fixed and TraceCore-fixed equivalence). Uses: def:weil-schfrob14-scheme-fixed-to-tracecore-fixed,def:weil-schfrob14-tracecore-fixed-to-scheme-fixed

Constructed by: lemma:weil-schfrob14-fixed-bridge-left-inverse,lemma:weil-schfrob14-fixed-bridge-right-inverse

Lean: conditional-interface:Weil.Etale.schFrob14FixedBridgeEquiv

Define

$$\beta_m : \mathrm{Fix}_X^{\mathrm{sch}}(m) \xrightarrow{\sim} \mathrm{Fix}_{X_0}(m)$$

with forward map B_m, inverse map A_m, and the two preceding inverse equations.

Definition 15.10.17 (Finite-extension points versus scheme-map fixed points). Uses: def:tracecore3-rational-fixed-equivalence,def:weil-schfrob14-fixed-bridge-equivalence

Lean: conditional-interface:Weil.Etale.schFrob14RationalSchemeFixedEquiv

Define

$$e_m^{\mathrm{sch}} : X_0(k_m) \xrightarrow{\sim} \mathrm{Fix}_X^{\mathrm{sch}}(m)$$

as the TraceCore3 equivalence $e_m : X_0(k_m) \simeq \mathrm{Fix}_{X_0}(m)$ followed by β_m^{-1}.

Definition 15.10.18 (Finite-extension point to scheme fixed point). Uses: def:weil-schfrob14-rational-scheme-fixed-equivalence

Lean: conditional-interface:Weil.Etale.schFrob14RationalToSchemeFixed

Define u_m^{sch} to be the forward map of e_m^{sch}.

Definition 15.10.19 (Scheme fixed point to finite-extension point). Uses: def:weil-schfrob14-rational-scheme-fixed-equivalence

Lean: conditional-interface:Weil.Etale.schFrob14SchemeFixedToRational

Define v_m^{sch} to be the inverse map of e_m^{sch}.

Lemma 15.10.20 (Finite-extension/scheme-fixed left inverse). *Uses:* *def:weil-schfrob14-rational-to-scheme-fixed-map*, *def:weil-schfrob14-scheme-fixed-to-rational-map*

Lean: *conditional-interface:Weil.Etale.schFrob14RationalSchemeFixed_leftInverse*

For $y \in X_0(k_m)$,

$$v_m^{\mathrm{sch}}(u_m^{\mathrm{sch}}(y)) = y.$$

Proof. Uses: def:weil-schfrob14-rational-scheme-fixed-equivalence,def:weil-schfrob14-rational-to-scheme-fixed-map,def:weil-schfrob14-scheme-fixed-to-rational-map

This is the left inverse field of the equivalence e_m^{sch}. □

Lemma 15.10.21 (Finite-extension/scheme-fixed right inverse). *Uses:* def:weil-schfrob14-rational-to-scheme-fixed-map, def:weil-schfrob14-scheme-fixed-to-rational-map

Lean: conditional-interface:Weil.Etale.schFrob14RationalSchemeFixed_rightInverse

For $z \in \mathrm{Fix}_X^{\mathrm{sch}}(m)$,

$$u_m^{\mathrm{sch}}(v_m^{\mathrm{sch}}(z)) = z.$$

Proof. Uses: def:weil-schfrob14-rational-scheme-fixed-equivalence, def:weil-schfrob14-rational-to-scheme-fixed-map, def:weil-schfrob14-scheme-fixed-to-rational-map

This is the right inverse field of the equivalence e_m^{sch}. □

Lemma 15.10.22 (Compatibility with the geometric-Frobenius convention). *Uses:* def:weil-schfrob14-rational-to-scheme-fixed-map, def:weil-schfrob14-scheme-fixed-to-tracecore-fixed, def:tracecore3-rational-to-fixed-map

Lean: conditional-interface:Weil.Etale.schFrob14GeometricConventionCompatibility

For $y \in X_0(k_m)$,

$$B_m(u_m^{\mathrm{sch}}(y)) = u_m(y) \quad \textit{in } \mathrm{Fix}_{X_0}(m).$$

Proof. Uses: def:weil-schfrob14-rational-scheme-fixed-equivalence, def:weil-schfrob14-fixed-bridge-equivalence, def:tracecore3-rational-fixed-equivalence, def:tracecore3-rational-to-fixed-map

Unfold the composite equivalence. Its forward map first applies the TraceCore3 rational-to-fixed map and then applies β_m^{-1}. Applying B_m, the forward map of β_m, cancels this last inverse. The remaining subtype element is exactly $u_m(y)$. Thus the bridge changes only the proof that the point is fixed: the underlying descended geometric point is the scalar extension used by the geometric-Frobenius convention. □

Theorem 15.10.23 (SCHFROB–14 discharge). *Uses:* def:weil-schfrob14-rational-scheme-fixed-equivalence

Lean: conditional-interface:Weil.Etale.schFrob14

For every SCHFROB–14 fixed-point input and every $m \geq 1$, the equivalence e_m^{sch} is the finite-extension-point dictionary for the fixed locus of the m-th scheme Frobenius iterate.

Proof. Uses: def:weil-schfrob14-rational-to-scheme-fixed-map, def:weil-schfrob14-scheme-fixed-to-rational-map, lemma:weil-schfrob14-rational-scheme-fixed-left-inverse, lemma:weil-schfrob14-rational-scheme-fixed-right-inverse, lemma:weil-schfrob14-geometric-convention-compatibility, prop:weil-scheme-frobenius-iterate-ordinary-power

The two named maps are the forward and inverse maps of e_m^{sch}, and the two preceding lemmas give their inverse equations. The convention lemma identifies their image in the TraceCore3 fixed subtype with the geometric-Frobenius rational-point map. Finally, the bracket-to-ordinary-power proposition identifies the bracketed scheme iterate with the categorical m-fold endomorphism. These statements are the fixed-point dictionary asserted by the theorem. □

15.11 EFI closed-point finite-field and geometric-point adapters

Definition 15.11.1 (Selected residue embedding for the EFI local adapter). Lean: blueprint-target:def:weil-efi-local-selected-residue-embedding

Uses: def:tracecore3-scheme-closed-point-degree, def:tracecore3-selected-geometric-lift, def:tracecore3-lift-to-embedding-map

For $c \in C_{X_0}$, write

$$e_c := \deg_{X_0}(c) > 0, \qquad \iota_c := \mu_c(\widetilde{c}) : \kappa(c) \hookrightarrow_{k_0} k.$$

Thus ι_c is the selected $k_0 = \mathbf{F}_q$-algebra embedding obtained from the selected geometric lift, rather than a second independent choice. Write $I_c := \iota_c(\kappa(c)) \subseteq k$ for its image subfield.

Lemma 15.11.2 (Cardinality of the selected closed-point residue field). *Lean:* blueprint-target:lemma:weil-efi-local02-residue-cardinality

Uses: def:weil-efi-local-selected-residue-embedding

The residue field is finite and

$$\#\kappa(c) = q^{e_c}.$$

Proof. Uses: lemma:tracecore3-closed-point-residue-finite, def:tracecore3-scheme-closed-point-degree, def:weil-finite-field-data

The finite extension lemma makes $\kappa(c)$ a finite-dimensional k_0-vector space, and the degree Definition identifies its dimension with e_c. Choose a basis. The resulting additive equivalence with $k_0^{e_c}$ transports finite cardinality, so $\#\kappa(c) = (\#k_0)^{e_c} = q^{e_c}$, using the specified equality $\#k_0 = q$. □

Lemma 15.11.3 (The selected residue image lies in the canonical fixed subfield). Lean: `blueprint-target:lemma:weil-efi-local02-image-contained-fixed-subfield`

Uses: `def:weil-efi-local-selected-residue-embedding`, `def:tracecore3-degree-n-fixed-subfield`

Inside k,

$$I_c \subseteq j_{e_c}(k_{e_c}).$$

Proof. Uses: `lemma:weil-efi-local02-residue-cardinality`, `lemma:weil-arithmetic-frobenius-power-formula`, `def:tracecore3-degree-n-fixed-subfield`

Let $u \in \kappa(c)$. If $u = 0$, then $u^{q^{e_c}} = u$. Otherwise, Lagrange's theorem in the multiplicative group of the finite field, whose order is $q^{e_c} - 1$, gives the same equality. Applying the algebra map ι_c and the arithmetic-Frobenius power formula yields

$$\varphi_q^{e_c}(\iota_c(u)) = \iota_c(u)^{q^{e_c}} = \iota_c(u).$$

This is precisely the defining predicate of k_{e_c}. □

Lemma 15.11.4 (The residue image and fixed subfield have the same cardinality). Lean: `blueprint-target:lemma:weil-efi-local02-image-fixed-subfield-cardinality`

Uses: `def:weil-efi-local-selected-residue-embedding`, `def:tracecore3-degree-n-fixed-subfield`

Both finite fields have cardinality q^{e_c}:

$$\#I_c = \#k_{e_c} = q^{e_c}.$$

Proof. Uses: `lemma:weil-efi-local02-residue-cardinality`, `lemma:tracecore3-degree-n-fixed-subfield-cardinality`

The map ι_c is injective because it is a unital homomorphism between fields, hence it is a bijection from $\kappa(c)$ to its image I_c. The first equality follows from the residue-cardinality lemma. The second is the TraceCore3 cardinality theorem for the canonical fixed subfield at the positive integer e_c. □

Lemma 15.11.5 (The selected residue image is the canonical fixed subfield). Lean: `blueprint-target:lemma:weil-efi-local02-image-eq-fixed-subfield`

Uses: `def:weil-efi-local-selected-residue-embedding`, `def:tracecore3-degree-n-fixed-subfield`

As subfields of the selected algebraic closure k,

$$I_c = j_{e_c}(k_{e_c}).$$

Proof. Uses: `lemma:weil-efi-local02-image-contained-fixed-subfield`, `lemma:weil-efi-local02-image-fixed-subfield-cardinality`

The preceding containment is an inclusion of finite sets, and the cardinality lemma says that its source and target have equal finite cardinality. The inclusion is therefore surjective, which is equality of the two subfields. □

Definition 15.11.6 (Corestricted selected residue embedding). Lean: `blueprint-target:def:weil-efi-local02-corestricted-residue-map`

Uses: `def:weil-efi-local-selected-residue-embedding`, `def:tracecore3-degree-n-fixed-subfield`

Constructed by: `lemma:weil-efi-local02-image-contained-fixed-subfield`

Corestrict ι_c along the proved containment and define the k_0-algebra map

$$\bar{\iota}_c : \kappa(c) \longrightarrow k_{e_c}, \qquad u \longmapsto \big(\iota_c(u),\ \varphi_q^{e_c}(\iota_c(u)) = \iota_c(u)\big).$$

Lemma 15.11.7 (The corestricted selected residue map is surjective). Lean: `blueprint-target:lemma:weil-efi-local02-corestricted-residue-surjective`

Uses: `def:weil-efi-local02-corestricted-residue-map`

The underlying map of $\bar{\iota}_c$ is surjective.

Proof. Uses: `def:weil-efi-local02-corestricted-residue-map`, `lemma:weil-efi-local02-image-eq-fixed-subfield`

For $v \in k_{e_c}$, the image-equality lemma places $j_{e_c}(v)$ in I_c. Hence there is $u \in \kappa(c)$ with $\iota_c(u) = j_{e_c}(v)$. Equality in the fixed-subfield subtype then gives $\bar{\iota}_c(u) = v$. □

Definition 15.11.8 (Canonical finite-field identification at a closed point). Lean: `blueprint-target:def:weil-closed-point-finite-field-identification`

Uses: `def:weil-efi-local02-corestricted-residue-map`

Constructed by: `lemma:weil-efi-local02-corestricted-residue-surjective`

The map $\bar{\iota}_c$ is injective as a unital map of fields and is surjective by the preceding lemma. Package it with its selected inverse as the explicit $k_0 = \mathbf{F}_q$-algebra equivalence

$$\varepsilon_c : \kappa(c) \xrightarrow[k_0\text{-alg}]{\sim} k_{e_c}.$$

This fixes the realization $\mathbf{F}_{q^{e_c}} := k_{e_c}$ inside the already selected algebraic closure k.

Theorem 15.11.9 (EFI LOCAL–02 discharge). Lean: *blueprint-target:thm:weil-efi-local02-discharge*

Uses: *def:weil-closed-point-finite-field-identification*

For every $c \in C_{X_0}$, ε_c is the required explicit $\mathbf{F}_q$-algebra isomorphism from $\kappa(c)$ to the canonical degree-e_c finite field k_{e_c}.

Proof. Uses: `def:weil-closed-point-finite-field-identification`, `lemma:weil-efi-local02-corestricted-residue-surjective`

The Definition packages the corestricted algebra homomorphism. Its injectivity is the field-homomorphism kernel argument and its surjectivity is the named leaf, so both inverse equations of the resulting algebra equivalence hold. □

Lemma 15.11.10 (The selected residue embedding reconstructs the selected lift). Lean: *blueprint-target:lemma:weil-efi-local03-selected-embedding-lift*

Uses: *def:weil-efi-local-selected-residue-embedding*, *def:tracecore3-embedding-to-lift-map*

One has

$$\lambda_c(\iota_c) = \widetilde{c} \quad \text{in } \mathrm{Lift}(c).$$

Proof. Uses: `def:weil-efi-local-selected-residue-embedding`, `lemma:tracecore3-embedding-lift-inverse`

Substitute $\iota_c = \mu_c(\widetilde{c})$ into the second inverse equation $\lambda_c(\mu_c(\bar{x})) = \bar{x}$. □

Lemma 15.11.11 (The selected closure is algebraic over the residue field). Lean: *blueprint-target:lemma:weil-efi-local03-closure-algebraic*

Uses: *def:weil-efi-local-selected-residue-embedding*, *def:weil-finite-field-data*

Give k its $\kappa(c)$-algebra structure through ι_c. Then

$$\mathrm{Algebra.IsAlgebraic}(\kappa(c), k).$$

Proof. Uses: `def:weil-efi-local-selected-residue-embedding`, `def:weil-finite-field-data`

Let $z \in k$. The finite-field datum says that z is algebraic over k_0, so a nonzero polynomial $P \in k_0[T]$ annihilates it. Map the coefficients of P along the structural embedding $k_0 \hookrightarrow \kappa(c)$. This coefficient map is injective because both rings are fields, so the resulting polynomial is still nonzero. The algebra-tower identity $k_0 \to \kappa(c) \xrightarrow{\iota_c} k$ shows that its evaluation at z is the image of $P(z) = 0$. Thus every z is algebraic over $\kappa(c)$. □

Definition 15.11.12 (Selected algebraic closure of the closed-point residue field). Lean: `blueprint-target:def:weil-closed-point-algebraic-closure`

Uses: `def:weil-efi-local-selected-residue-embedding`, `def:weil-finite-field-data`

Constructed by: `lemma:weil-efi-local03-closure-algebraic`

Define

$$\overline{\kappa(c)}^{\mathrm{EFI}} := k$$

with $\kappa(c)$-algebra structure ι_c. The preceding lemma gives algebraicity, and the selected algebraically closed field structure on k gives the other algebraic-closure axiom. Hence this is the selected algebraic closure of $\kappa(c)$, using the same carrier k as the global finite-field datum.

Definition 15.11.13 (Closed-point geometric-point morphism). Lean: `blueprint-target:def:weil-closed-point-geometric-point-map`

Uses: `def:weil-efi-local-selected-residue-embedding`, `def:tracecore3-scheme-closed-point-type`, `def:tracecore3-scheme-geometric-point-data`

Define

$$\bar{c}^{\mathrm{EFI}} := i_c \circ \mathrm{Spec}(\iota_c) : \mathrm{Spec}\, k \longrightarrow X_0.$$

The variance is contravariant: the algebra embedding $\kappa(c) \to k$ induces $\mathrm{Spec}\, k \to \mathrm{Spec}\, \kappa(c)$.

Lemma 15.11.14 (The EFI geometric point has the selected closed-point support). *Lean:* `blueprint-target:lemma:weil-efi-local03-geometric-point-support`

Uses: `def:weil-closed-point-geometric-point-map`

The morphism $\bar{c}^{\mathrm{EFI}}$ is a k_0-geometric point and

$$\mathrm{sp}(\bar{c}^{\mathrm{EFI}}) = c.$$

Proof. Uses: `def:weil-closed-point-geometric-point-map,def:tracecore3-geometric-lift-from-embedding,lemma:tracecore3-geometric-lift-support,lemma:weil-efi-local03-selected-embedding-lift,lemma:tracecore3-selected-geometric-lift-support`

The displayed composite is exactly the embedding lift $\bar{c}_{\iota_c}$. The TraceCore3 support lemma therefore gives support c. Equivalently, the selected-embedding leaf identifies this lift with $\widetilde{c}$, whose subtype predicate is the same support equation. □

Definition 15.11.15 (Closed-point geometric-point package). Lean: `blueprint-target:def:weil-closed-point-geometric-point`

Uses: `def:weil-closed-point-algebraic-closure,def:weil-closed-point-geometric-point-map`

Constructed by: `lemma:weil-efi-local03-closure-algebraic,lemma:weil-efi-local03-geometric-point-support`

Define $\mathscr{G}_c$ to consist of the algebraic closure $\overline{\kappa(c)}^{\mathrm{EFI}} = k$, its selected residue embedding ι_c, the geometric-point morphism $\bar{c}^{\mathrm{EFI}}$, and the proved support equation $\mathrm{sp}(\bar{c}^{\mathrm{EFI}}) = c$. All four fields use the same selected lift and the same global closure k.

Theorem 15.11.16 (EFI LOCAL–03 discharge). *Lean:* `blueprint-target:thm:weil-efi-local03-discharge`

Uses: `def:weil-closed-point-geometric-point`

For every $c \in C_{X_0}$, $\mathscr{G}_c$ is a geometric-point package above c whose closure field is k and whose residue embedding is the selected map $\mu_c(\widetilde{c})$.

Proof. Uses: `def:weil-closed-point-geometric-point,lemma:weil-efi-local03-selected-embedding-lift,lemma:weil-efi-local03-closure-algebraic,lemma:weil-efi-local03-geometric-point-support`

The three named leaves supply, respectively, the selected-embedding coherence, algebraicity over the residue field, and the required support. Algebraic closedness is the existing structure on k. These are exactly the fields of $\mathscr{G}_c$. □

Definition 15.11.17 (TraceCore3 inhabitant of the closed-point arithmetic datum). Lean: `blueprint-target:def:weil-tracecore3-closed-point-arithmetic-data`

Uses: `def:weil-closed-point-arithmetic-data,def:tracecore3-scheme-closed-point-degree,def:weil-closed-point-finite-field-identification,def:weil-closed-point-algebraic-closure`

For a descended scheme datum whose X_0 carries the TraceCore3 finite-type scheme structure, define $\mathcal{A}_c$ by the fields

$$x = c, \quad e = e_c, \quad \mathbf{F}_{q^e} = k_{e_c}, \quad \kappa(c) \xrightarrow{\varepsilon_c}{\sim} k_{e_c}, \quad \overline{\kappa(c)} = k, \quad \kappa(c) \xrightarrow{\iota_c} k.$$

The positivity of e_c comes from the degree Definition. This is an actual inhabitant of the earlier closed-point arithmetic input type.

Theorem 15.11.18 (Typed LOCAL–02/03 adapter). *Lean:* `blueprint-target:thm:weil-efi-local02-local03-typed-adapter`

Uses: `def:weil-tracecore3-closed-point-arithmetic-data,def:weil-closed-point-geometric-point`

The pair $(\mathcal{A}_c, \mathscr{G}_c)$ simultaneously supplies the finite-field-identification and geometric-point fields required by the EFI closed-point interface, with one common degree, closure, and residue embedding.

Proof. Uses: `thm:weil-efi-local02-discharge,thm:weil-efi-local03-discharge,def:weil-tracecore3-closed-point-arithmetic-data,def:weil-closed-point-geometric-point`

The LOCAL–02 theorem supplies ε_c, and the LOCAL–03 theorem supplies the closure k, embedding ι_c, point $\bar{c}^{\mathrm{EFI}}$, and support. Both Definitions refer to e_c, $\iota_c = \mu_c(\widetilde{c})$, and the same selected k, so the pair is type-compatible without a transport or a second choice. □

Proposition 15.11.19 (Finite-extension point count equals the fixed-locus count). *Uses:* *def: tracecore3-rational-fixed-equivalence, def: tracecore3-scheme-fixed-locus-subtype, def: tracecore3-scheme-finite-extension-point-type*

Lean: *Weil.TraceCore3.schemePointCount_fixedLocus*

For every $n \geq 1$,

$$\#R_{X_0,n} = \#\operatorname{Fix}_{X_0}(n).$$

Proof. Uses: def:tracecore3-rational-fixed-equivalence,lemma:tracecore3-scheme-finite-extension-point-finite,lemma:tracecore3-scheme-fixed-locus-finite

Transport finite cardinality along the equivalence e_n. The two finiteness lemmas supply the finite-cardinality structures on its source and target. □

Proposition 15.11.20 (Closed-point counting relation). *Uses:* *def: tracecore3-scheme-finite-extension-point-type, def: tracecore3-scheme-closed-point-degree-family, def: tracecore3-degree-dividing-closed-point-type*

Lean: *Weil.TraceCore3.closedPointCountingRelation*

For every $n \geq 1$,

$$\#R_{X_0,n} = \sum_{\substack{c : C_{X_0} \\ \deg_{X_0}(c) \mid n}} \deg_{X_0}(c).$$

Proof. Uses: prop:tracecore3-scheme-point-count-fixed-locus,lemma:tracecore3-scheme-fixed-locus-agreement,prop:tracecore3-fixed-locus-orbit-sum,prop:tracecore3-closed-point-orbit-inverse,lemma:tracecore3-closed-point-orbit-degree,lemma:tracecore3-degree-dividing-orbit-membership,lemma:tracecore3-degree-dividing-closed-point-finite

First replace $\#R_{X_0,n}$ by the raw fixed-locus cardinality and then by the abstract fixed-locus cardinality using the identity equivalence. Apply the fixed-locus orbit sum. Reindex the finite orbit sum along the inverse pair $\alpha_{X_0}, \beta_{X_0}$; the orbit divisibility condition becomes $\deg_{X_0}(c) \mid n$ by the degree equality. Finally replace every summand $d_{\Phi_{X_0}}(\alpha_{X_0}(c))$ by $\deg_{X_0}(c)$. □

15.12 Exact ZPC interface instances supplied by TraceCore3

Definition 15.12.1 (Finite-orbit datum attached to the scheme). Uses: def:weil-finite-orbit-frobenius-datum,def:tracecore3-scheme-geometric-point-data,def:tracecore3-scheme-geometric-frobenius,def:tracecore3-scheme-return-time,def:tracecore3-scheme-fixed-locus-subtype

Constructed by: lemma:tracecore3-scheme-return-time-positive,lemma:tracecore3-scheme-return-time-fixes,lemma:tracecore3-scheme-fixed-locus-finite

Lean: Weil.TraceCore3.zpcFiniteOrbitFrobeniusData

For $n \geq 1$, use the finiteness of $\operatorname{Fix}_{X_0}(n)$ to define the literal finset

$$\widehat{\operatorname{Fix}}_{X_0}(n) := \big(\text{Finset.univ} : \text{Finset}(\operatorname{Fix}_{X_0}(n))\big).\text{map}(\text{the subtype embedding}) \subseteq \Omega_{X_0}.$$

Then define $D_{X_0}^{\mathrm{ZPC}}$ as the finite-orbit geometric-Frobenius datum whose carrier is Ω_{X_0}, whose permutation is Φ_{X_0}, whose selected return time is r_{X_0}, and whose degree-n fixed finset is the displayed finset. The return-time positivity and iterate equation are the two selected-time lemmas. Subtype membership gives, without a cardinality argument,

$$\bar{x} \in \widehat{\operatorname{Fix}}_{X_0}(n) \quad \Longleftrightarrow \quad \Phi_{X_0}^n(\bar{x}) = \bar{x}.$$

Thus every occurrence of Frobenius in this datum is the geometric permutation; arithmetic Frobenius occurs only in the affine proofs that construct its inverse and its finite fixed loci.

Proposition 15.12.2 (ZPC-01 geometric-Frobenius package). *Uses:* *def: tracecore3-zpc01-finite-orbit-datum*

Lean: *Weil.TraceCore3.zpcFiniteOrbitFrobeniusData_spec*

The datum $D_{X_0}^{\mathrm{ZPC}}$ has carrier $X_0(k)$, geometric-Frobenius permutation Φ_{X_0} with its arithmetic inverse, a positive selected period fixing every geometric point, and for each $n \geq 1$ a finite fixed-point finset whose membership predicate is exactly $\Phi_{X_0}^n(\bar{x}) = \bar{x}$.

Proof. Uses: def:tracecore3-zpc01-finite-orbit-datum,lemma:tracecore3-scheme-raw-frobenius-inverse,lemma:tracecore3-scheme-return-time-positive,lemma:tracecore3-scheme-return-time-fixes,lemma:tracecore3-scheme-fixed-locus-finite

Unfold $D_{X_0}^{\mathrm{ZPC}}$. Its first four assertions are the corresponding fields of the construction. For the last assertion, membership in the mapped universal finset is equivalent to the existence of an element $(\bar{x}, h)$ of the raw fixed-locus subtype with underlying point $\bar{x}$; this is equivalent to $h : \Phi_{X_0}^n(\bar{x}) = \bar{x}$. □

Definition 15.12.3 (Finite-extension point interface attached to the scheme). Uses: def:weil-finite-extension-point-interface,def:tracecore3-zpc01-finite-orbit-datum,def:tracecore3-scheme-finite-extension-point-type,def:tracecore3-rational-fixed-equivalence,def:tracecore3-scheme-fixed-locus-subtype

Constructed by: lemma:tracecore3-scheme-finite-extension-point-finite,lemma:tracecore3-rational-fixed-left-inverse,lemma:tracecore3-rational-fixed-right-inverse

Lean: Weil.TraceCore3.zpcFiniteExtensionPointInterface

For each $n \geq 1$, let h_n be the equivalence

$$h_n : \mathrm{Fix}_{X_0}(n) \simeq \{\bar{x} : \Omega_{X_0} \mid \bar{x} \in \widehat{\mathrm{Fix}}_{X_0}(n)\}$$

that is the identity on underlying points. Its forward membership proof and its inverse fixed-point proof are the two directions of the membership equivalence in Definition 15.12.1. Define $P_{X_0}^{\mathrm{ZPC}}$ over $D_{X_0}^{\mathrm{ZPC}}$ by

$$\mathrm{Rat}_{P_{X_0}^{\mathrm{ZPC}}}(n) = R_{X_0,n}, \qquad \rho_n = e_n \,\mathrm{trans}\, h_n.$$

The finite structure on $R_{X_0,n}$ is supplied only by the direct finite affine-cover proof.

Proposition 15.12.4 (ZPC-02 fixed-point descent package). Uses: def:tracecore3-zpc02-finite-extension-point-interface

Lean: Weil.TraceCore3.zpcFiniteExtensionPointInterface_spec

For every $n \geq 1$, the forward map of ρ_n is scalar extension s_n together with its geometric-Frobenius fixedness certificate, the inverse map is the selected affine descent to k_n, and the two composites are identities. In particular,

$$X_0(k_n) \simeq \{\bar{x} \in X_0(k) \mid \Phi_{X_0}^n(\bar{x}) = \bar{x}\},$$

and both sides are finite for reasons proved before this package.

Proof. Uses: def:tracecore3-zpc02-finite-extension-point-interface,lemma:tracecore3-rational-point-scalar-extension-fixed,lemma:tracecore3-fixed-point-descent-exists,lemma:tracecore3-fixed-point-descent-unique,lemma:tracecore3-rational-fixed-left-inverse,lemma:tracecore3-rational-fixed-right-inverse,lemma:tracecore3-scheme-finite-extension-point-finite,lemma:tracecore3-scheme-fixed-locus-finite

Unfold $\rho_n = e_n \,\mathrm{trans}\, h_n$. The equivalence h_n changes only the proof component, so the underlying forward and inverse maps are those of e_n. Their descriptions and inverse laws are exactly the five cited scalar-extension and descent lemmas. The last two cited lemmas install finiteness independently on the two displayed types. □

15.12.1 The concrete finite-extension point interface for Module 03

The TraceCore3 ZPC-02 object already has the exact carrier, finite structure, and geometric fixed-point equivalence required by the abstract Module 03 interface. The following declarations expose those fields under a single Module 03 adapter and record the two cardinality projections explicitly.

Definition 15.12.5 (TRR-02 concrete point-interface adapter). Uses: def:tracecore3-zpc02-finite-extension-point-interface

Lean: Weil.TraceCore3.trr02ConcretePointInterface

For the finite-type scheme datum X_0/k_0, define

$$P_{X_0}^{\mathrm{TRR}} := P_{X_0}^{\mathrm{ZPC}}.$$

This is a literal rebind of the already constructed ZPC-02 interface, not an isomorphic replacement and not a new choice of finite fields.

Lemma 15.12.6 (Exact carrier of the TRR-02 point interface). Uses: def:tracecore3-trr02-concrete-point-interface, def:tracecore3-scheme-finite-extension-point-type

Lean: Weil.TraceCore3.trr02ConcretePointInterface_rat

For every $n \geq 1$, the carrier is definitionally

$$\mathrm{Rat}_{P_{X_0}^{\mathrm{TRR}}}(n) = R_{X_0,n} = X_0(k_n) = \mathrm{Hom}_{k_0}(\mathrm{Spec}\, k_n, X_0).$$

Here $k_n = \{a \in k \mid \varphi_q^n(a) = a\}$ is the canonical field denoted by $\mathbf{F}_{q^n}$ in the counting formula.

Proof. Uses: def:tracecore3-trr02-concrete-point-interface,def:tracecore3-zpc02-finite-extension-point-interface,def:tracecore3-scheme-finite-extension-point-type

Unfold the adapter and the ZPC-02 interface. Its degree-n rational type was defined to be the literal scheme point type $R_{X_0,n}$, whose definition is the displayed Hom type. All equalities are therefore reflexivity after unfolding. □

Lemma 15.12.7 (Finiteness stored by the TRR-02 point interface). *Uses:* *def: tracecore3-trr02-concrete-point-interface*

Lean: *Weil.TraceCore3.trr02ConcretePointInterface_finite*

For every $n \geq 1$, the type $\mathrm{Rat}_{P^{\mathrm{TRR}}_{X_0}}(n)$ is finite.

Proof. Uses: lemma:tracecore3-trr02-concrete-point-interface-carrier,lemma:tracecore3-scheme-finite-extension-point-finite

Rewrite the carrier by the preceding literal equality and use the direct finite affine-cover theorem for $R_{X_0,n}$. This is the same finite structure installed in ZPC-02. □

Lemma 15.12.8 (Stored point count equals the finite-extension rational count). *Uses:* *def: tracecore3-trr02-concrete-point-interface*, *def: weil-finite-extension-point-count*, *def: tracecore3-scheme-finite-extension-point-type*

Lean: *Weil.TraceCore3.trr02PointCount_eq_rational*

For every $n \geq 1$,

$$N_n(P^{\mathrm{TRR}}_{X_0}) = \#X_0(k_n) = \#X_0(\mathbf{F}_{q^n}).$$

The last expression is notation for the same canonical fixed-subfield point type, not a comparison with an external finite-field presentation.

Proof. Uses: lemma:tracecore3-trr02-concrete-point-interface-carrier,lemma:tracecore3-trr02-concrete-point-interface-finite,def:weil-finite-extension-point-count

By definition, the left side is the finite cardinality of $\mathrm{Rat}_{P^{\mathrm{TRR}}_{X_0}}(n)$. Transport cardinality along the identity equivalence supplied by the exact-carrier lemma. This is `Fintype.card_congr`; it also shows that the equality is independent of the particular finite-structure instance used to enumerate the literal carrier. □

Lemma 15.12.9 (Stored point count equals the geometric fixed-point count). *Uses:* *def: tracecore3-trr02-concrete-point-interface*, *def: tracecore3-scheme-fixed-locus-subtype*, *def: weil-finite-extension-point-count*

Lean: *Weil.TraceCore3.trr02PointCount_eq_fixed*

For every $n \geq 1$,

$$N_n(P^{\mathrm{TRR}}_{X_0}) = \#\{\bar{x} \in X_0(k) \mid \Phi^n_{X_0}(\bar{x}) = \bar{x}\}.$$

The endomorphism Φ_{X_0} is geometric Frobenius.

Proof. Uses: lemma:tracecore3-trr02-point-count-rational,prop:tracecore3-scheme-point-count-fixed-locus

The preceding lemma identifies the stored count with $\#R_{X_0,n}$. Proposition 15.11.19 transports this finite cardinality along the TraceCore3 equivalence $e_n : R_{X_0,n} \simeq \mathrm{Fix}_{X_0}(n)$. Unfolding the fixed subtype gives exactly the displayed geometric-Frobenius predicate. □

Definition 15.12.10 (Exact TRR-02 interface predicate). Uses: def:weil-finite-extension-point-interface,def:weil-finite-extension-point-count,def:tracecore3-scheme-finite-extension-point-type,def:tracecore3-scheme-fixed-locus-subtype

Lean: `Weil.TraceCore3.IsTRR02ConcretePointInterface`

For a finite-extension point interface P over $D^{\mathrm{ZPC}}_{X_0}$, let IsTRR02ConcretePointInterface(P) mean that for every $n \geq 1$: its rational carrier is literally $R_{X_0,n} = X_0(k_n)$, that carrier is finite, and its stored count is equal both to $\#X_0(k_n)$ and to $\#\,\mathrm{Fix}_{X_0}(n)$.

Proposition 15.12.11 (The TraceCore3 adapter satisfies the exact TRR-02 predicate). *Uses:* *def: tracecore3-trr02-concrete-point-interface*, *def: tracecore3-trr02-concrete-point-interface-predicate*

Lean: *Weil.TraceCore3.trr02ConcretePointInterface_spec*

One has

$$\mathrm{IsTRR02ConcretePointInterface}(P^{\mathrm{TRR}}_{X_0}).$$

Proof. Uses: lemma:tracecore3-trr02-concrete-point-interface-carrier,lemma:tracecore3-trr02-concrete-point-interface-finite,lemma:tracecore3-trr02-point-count-rational,lemma:tracecore3-trr02-point-count-fixed

Fix $n \geq 1$. The four cited lemmas supply, in the order stated by the predicate, the literal carrier equality, its finite instance, the rational point-cardinality equality, and the geometric fixed-point-cardinality equality. Pair these four proof terms and abstract over n. □

Definition 15.12.12 (TRR-02 concrete point-interface certificate type). Uses: def:weil-finite-extension-point-interface, def:tracecore3-zpc01-finite-orbit-datum, def:tracecore3-trr02-concrete-point-interface-predicate

Lean: Weil.TraceCore3.TRR02ConcretePointInterfaceCertificate

Define the certificate type to consist of a finite-extension point interface P over $D^{\mathrm{ZPC}}_{X_0}$, together with a proof of IsTRR02ConcretePointInterface(P).

Definition 15.12.13 (Concrete finite-extension point-interface certificate). Uses: def:tracecore3-trr02-concrete-point-interface, def:tracecore3-trr02-concrete-point-interface-certificate-type

Constructed by: prop:tracecore3-trr02-concrete-point-interface-predicate

Lean: Weil.TraceCore3.trr02ConcretePointInterfaceCertificate

Define the required certificate by

$$\left(P^{\mathrm{TRR}}_{X_0}, \mathrm{IsTRR02ConcretePointInterface}(P^{\mathrm{TRR}}_{X_0})\right),$$

using the preceding proposition for the second component. This is the exact concrete finite-extension point interface required by Module 03.

Definition 15.12.14 (Explicit finset of degree-m scheme closed points). Uses: def:tracecore3-scheme-closed-point-type, def:tracecore3-scheme-closed-point-degree

Constructed by: lemma:tracecore3-degree-bounded-closed-points-finite

Lean: Weil.TraceCore3.zpcDegreeClosedPointFinset

For $m \geq 1$, put $B_m := \{c : C_{X_0} \mid \deg_{X_0}(c) \leq m\}$. Give B_m the finite structure proved above, filter its universal finset by the equality $\deg_{X_0}(c) = m$, and map the result along the injective subtype projection $B_m \hookrightarrow C_{X_0}$. Denote the resulting finset by $\mathcal{C}^{\mathrm{geom}}_{X_0,m}$.

Lemma 15.12.15 (Exact membership in the scheme degree finset). *Uses: def:tracecore3-zpc03-degree-closed-point-finset*

Lean: Weil.TraceCore3.mem_zpcDegreeClosedPointFinset

For $m \geq 1$ and $c \in C_{X_0}$,

$$c \in \mathcal{C}^{\mathrm{geom}}_{X_0,m} \quad \Longleftrightarrow \quad \deg_{X_0}(c) = m.$$

Proof. Uses: def:tracecore3-zpc03-degree-closed-point-finset

Membership in the mapped filtered universal finset is equivalent to the existence of the unique bounded subtype element over c satisfying the filter equality. The forward implication returns that equality. Conversely, if $\deg_{X_0}(c) = m$, then $\deg_{X_0}(c) \leq m$, so c defines an element of B_m retained by the filter and hence belongs to the image. □

Proposition 15.12.16 (ZPC-03 closed-point degree package). *Uses: def:tracecore3-scheme-closed-point-type, def:tracecore3-scheme-closed-point-degree, def:tracecore3-zpc03-degree-closed-point-finset*

Lean: Weil.TraceCore3.zpcClosedPointDegreePackage

The type C_{X_0} is the subtype of closed points of X_0; every $c \in C_{X_0}$ has a finite residue extension over k_0 and positive degree $[\kappa(c) : k_0]$; and for every $m \geq 1$, the explicit finset $\mathcal{C}^{\mathrm{geom}}_{X_0,m}$ is finite with membership exactly $\deg_{X_0}(c) = m$. No Euler-product identity is used in any of these finiteness assertions.

Proof. Uses: lemma:tracecore3-closed-point-residue-finite, def:tracecore3-scheme-closed-point-degree, lemma:tracecore3-degree-bounded-closed-points-finite, lemma:tracecore3-zpc03-degree-closed-point-finset-mem

The first assertion is definitional. Zariski's lemma supplies residue-field finiteness, and the degree is the positive dimension of the nonzero finite extension. The degree-m finset was cut out of the independently finite degree-at-most-m subtype; its exact membership is the preceding lemma. □

Definition 15.12.17 (Bridge between the two Frobenius orbit quotients). Uses: def:tracecore3-frobenius-orbit-class, def:tracecore3-zpc01-finite-orbit-datum, def:weil-frobenius-orbit-type

Constructed by: lemma:tracecore3-orbit-quotient-equality

Lean: Weil.TraceCore3.zpcOrbitQuotientBridge

Define

$$b_{X_0} : \mathrm{Orb}_{\Phi_{X_0}}(\Omega_{X_0}) \simeq \mathrm{Orb}_{D^{\mathrm{ZPC}}_{X_0}}(\Omega_{X_0})$$

by sending the TraceCore3 class of $\bar{x}$ to the generic class of the same $\bar{x}$, and conversely. These quotient lifts are well defined: the TraceCore3 relation says $\bar{y} = \Phi^j_{X_0}(\bar{x})$ for some $j \in \mathbf{Z}$, while the generic orbit relation says that $\bar{y}$ is obtained from $\bar{x}$ by an element of Subgroup.zpowers(Φ_{X_0}); unfolding membership in zpowers makes the two predicates identical. Both composites are the identity by quotient induction.

Lemma 15.12.18 (The orbit bridge preserves degree). *Uses:* *def: tracecore3-zpc04-orbit-quotient-bridge, def: tracecore3-orbit-degree, def: weil-frobenius-orbit-degree*

Lean: *Weil.TraceCore3.zpcOrbitQuotientBridge_degree*

For every TraceCore3 orbit O,

$$d_{\Phi_{X_0}}(O) = \deg_{D_{X_0}^{\mathrm{ZPC}}}(b_{X_0}(O)).$$

Proof. Uses: def:tracecore3-zpc04-orbit-quotient-bridge, lemma:tracecore3-orbit-degree-representative, lemma:weil-frobenius-orbit-degree-point

Use quotient induction and write $O = q_{X_0}(\bar{x})$. The bridge sends it to the generic class $[\bar{x}]$. The two cited representative formulas identify both displayed degrees with the same least positive natural period of the same permutation Φ_{X_0} at $\bar{x}$. Hence they are equal. □

Definition 15.12.19 (Closed-point–orbit interface attached to the scheme). Uses: def:weil-closed-point-orbit-interface, def:tracecore3-zpc01-finite-orbit-datum, def:tracecore3-scheme-closed-point-type, def:tracecore3-scheme-closed-point-degree, def:tracecore3-closed-point-to-orbit-map, def:tracecore3-orbit-to-closed-point-map, def:tracecore3-zpc04-orbit-quotient-bridge

Constructed by: prop:tracecore3-closed-point-orbit-inverse, lemma:tracecore3-closed-point-orbit-degree, lemma:tracecore3-zpc04-orbit-bridge-degree

Lean: Weil.TraceCore3.zpcClosedPointOrbitInterface

Define $C_{X_0}^{\mathrm{ZPC}}$ over $D_{X_0}^{\mathrm{ZPC}}$ with closed-point carrier C_{X_0}, degree $\deg_{X_0}$, and equivalence

$$\theta_{X_0} := b_{X_0} \circ \alpha_{X_0} : C_{X_0} \simeq \mathrm{Orb}_{D_{X_0}^{\mathrm{ZPC}}}(\Omega_{X_0}).$$

Its inverse is $\beta_{X_0} \circ b_{X_0}^{-1}$. The two inverse laws come from the proved α-β inverse pair, positivity comes from $\deg_{X_0}(c) \in \mathbf{N}_{>0}$, and degree compatibility is the composite of the TraceCore3 closed-point degree theorem and the bridge degree lemma.

Definition 15.12.20 (Degreewise closed-point–orbit equivalence). Uses: def:tracecore3-zpc03-degree-closed-point-finset, def:tracecore3-zpc04-closed-point-orbit-interface, def:weil-degree-frobenius-orbit-finset

Constructed by: lemma:tracecore3-zpc03-degree-closed-point-finset-mem, lemma:weil-degree-frobenius-orbit-finset-mem, lemma:tracecore3-closed-point-orbit-degree, lemma:tracecore3-zpc04-orbit-bridge-degree

Lean: Weil.TraceCore3.zpcDegreeClosedPointOrbitEquiv

For every $m \geq 1$, restrict θ_{X_0} and its inverse to obtain

$$\{c \mid c \in \mathcal{C}_{X_0,m}^{\mathrm{geom}}\} \simeq \{O \mid O \in \mathcal{O}_m(D_{X_0}^{\mathrm{ZPC}})\}.$$

The forward and inverse landing certificates follow by rewriting the two finset-membership predicates as degree equalities and then applying the degree compatibility stored in $C_{X_0}^{\mathrm{ZPC}}$.

Proposition 15.12.21 (ZPC-04 closed-point–orbit package). *Uses:* *def: tracecore3-zpc04-closed-point-orbit-interface, def: tracecore3-zpc04-degree-orbit-equivalence*

Lean: *Weil.TraceCore3.zpcClosedPointOrbitPackage*

The maps underlying θ_{X_0} and $\theta_{X_0}^{-1}$ are the explicit selected-residue-lift and scheme-image maps. Residue embeddings exist; the selected lift has orbit class independent of every choice; orbit representative images are closed and scheme-theoretically independent of the representative; α_{X_0} and β_{X_0} are inverse; residue degree equals orbit degree; and the equivalence restricts to the explicit degree-m finsets for every $m \geq 1$.

Proof. Uses: lemma:tracecore3-residue-embedding-nonempty, lemma:tracecore3-selected-lift-class-independent, lemma:tracecore3-orbit-representative-image-closed, lemma:tracecore3-orbit-scheme-image-independent, lemma:tracecore3-orbit-to-closed-point-well-defined, lemma:tracecore3-closed-point-orbit-left-inverse, lemma:tracecore3-closed-point-orbit-right-inverse, lemma:tracecore3-closed-point-orbit-degree, def:tracecore3-zpc04-orbit-quotient-bridge, lemma:tracecore3-zpc04-orbit-bridge-degree, def:tracecore3-zpc04-degree-orbit-equivalence

The clauses occur in the order of the cited construction and property lemmas. The quotient bridge changes only the presentation of the same integer-power orbit relation. The last clause is precisely the restricted equivalence just constructed, whose landing proofs use the already proved degree equality rather than an Euler-product count. □

Theorem 15.12.22 (TraceCore finite-extension/fixed-point discharge for schemes). *Lean:* `blueprint-target: thm: tracecore3-tracec-point-fixed-discharge-v1`

Uses: `def: tracecore3-zpc02-finite-extension-point-interface`

For every finite-type scheme $X_0/\mathbf{F}_q$ and every $n \geq 1$, the TraceCore3 maps

$$u_n : R_{X_0,n} \longrightarrow \mathrm{Fix}_{X_0}(n), \qquad v_n : \mathrm{Fix}_{X_0}(n) \longrightarrow R_{X_0,n}$$

have the following properties: u_n is scalar extension together with its n-th geometric-Frobenius fixedness certificate; v_n is affine descent to k_n; both composites are identities; both displayed types are finite; and

$$\#R_{X_0,n} = \#\,\mathrm{Fix}_{X_0}(n).$$

Thus these maps supply the required finite-type-scheme point/fixed comparison.

Proof. Uses: `def:tracecore3-zpc02-finite-extension-point-interface`, `prop:tracecore3-zpc02-finite-extension-fixed-package`, `lemma:weil-finite-extension-point-count-eq-fixed`, `prop:tracecore3-scheme-point-count-fixed-locus`

The ZPC-02 package identifies the forward and inverse maps of ρ_n with the previously constructed scalar-extension and selected affine-descent maps. The same package supplies their fixedness, inverse equations, and the independent finiteness proofs. Applying the generic cardinality lemma to ρ_n, or equivalently applying the cited direct TraceCore3 proposition, gives the displayed equality. No globally finite structure on $X_0(\overline{\mathbf{F}}_q)$ is used. □

Theorem 15.12.23 (TraceCore closed-point/orbit discharge for schemes). *Lean:* `blueprint-target: thm: tracecore3-tracec-closed-orbit-discharge-v1`

Uses: `def: tracecore3-zpc02-finite-extension-point-interface`, `def: tracecore3-zpc03-degree-closed-point-finset`, `def: tracecore3-zpc04-closed-point-orbit-interface`

For every finite-type scheme $X_0/\mathbf{F}_q$, the selected-residue-lift map from closed points to geometric-Frobenius orbits and the scheme-image map in the reverse direction are mutually inverse. The orbit class of a lift is independent of the residue embedding, the scheme image is independent of the orbit representative, and

$$\deg_{X_0}(c) = d_{\Phi_{X_0}}(\alpha_{X_0}(c)).$$

Every fixed-degree closed-point fiber is finite, and for every $n \geq 1$,

$$\#R_{X_0,n} = \sum_{\substack{c : C_{X_0} \\ \deg_{X_0}(c) \mid n}} \deg_{X_0}(c).$$

These statements give the required closed-point/orbit comparison without using the Euler-product identity as an input.

Proof. Uses: `thm:tracecore3-tracec-point-fixed-discharge-v1`, `prop:tracecore3-zpc03-closed-point-degree-package`, `prop:tracecore3-zpc04-closed-point-orbit-package`, `prop:tracecore3-closed-point-counting-relation`

The ZPC-04 package supplies the two explicit maps, both inverse equations, both independence statements, and residue-degree/orbit-degree compatibility. The ZPC-03 package supplies the finite degree fibers independently of any counting series. The final displayed equality is the direct TraceCore3 closed-point counting relation; its fixed-point input is the scalar-extension and descent result summarized in the preceding discharge theorem. Hence the argument is coefficientwise orbit partitioning, not a circular appeal to an Euler product. □

Theorem 15.12.24 (TraceCore2 fixed-point descent rebind for schemes). *Lean:* `blueprint-target: thm: tracecore3-tracec2-point-fixed-discharge-v1`

Uses: `def: tracecore3-zpc02-finite-extension-point-interface`

For a finite-type scheme $X_0/\mathbf{F}_q$, identify the follow-up scheme fixed-locus notation with

$$\mathrm{Fix}_{X_0}(n) = \{\bar{x} : X_0(\overline{\mathbf{F}}_q) \mid \Phi^n_{X_0}(\bar{x}) = \bar{x}\}$$

and take its forward and reverse candidates to be the TraceCore3 maps u_n and v_n. Then the forward fixed equation, both inverse equations, finiteness of both types, and equality of their cardinalities hold for every $n \geq 1$. This is the required scheme-level follow-up statement.

Proof. Uses: `thm:tracecore3-tracec-point-fixed-discharge-v1`

After the displayed notational identification, every clause is one of the clauses of the preceding TraceCore discharge theorem. This is a rebind of the scheme-specific fixed-locus construction, not an attempted instance of the legacy record whose carrier Ω was required to be globally finite. □

15.13 Canonical finite-field towers and the final point realization

Status. This section uses only the canonical fixed subfields $k_n \subset k$ already constructed by TraceCore3. It supplies the missing divisibility tower, then rebinds the TraceCore3 ZPC-02, ZPC-03, and ZPC-04 scheme instances to the literal point-realization types used by the final chapter. No arbitrary presentation of a field with q^n elements is chosen. Naturality under replacement by such arbitrary presentations is isolated at the end as the still-open `OBL-FINAL-POINT-FIELD`.

15.13.1 The canonical fixed-subfield tower

Definition 15.13.1 (Final scheme as a TraceCore3 finite-type input). Uses: `def:weil-final-classical-scheme-data`, `def:tracecore3-finite-type-scheme-data`

Lean: `Weil.Final.traceCore3SchemeData`

For a final classical scheme datum $\mathcal{X}$, let $\mathcal{T}(\mathcal{X})$ be the TraceCore3 finite-type scheme datum whose scheme is the same X_0, whose structure morphism is the same $a_0 : X_0 \to \operatorname{Spec} k_0$, and whose finite-type certificate is the certificate projected from the classical core. Thus every type and map constructed by TraceCore3 below is indexed by the literal final scheme, not by an isomorphic replacement.

Lemma 15.13.2 (Monotonicity of the canonical fixed subfields). *Uses: def: tracecore3-degree-n-fixed-subfield*

Lean: Weil.TraceCore3.degreeNFixedSubfield_mono

Let $n, m \geq 1$ and let $h : n \mid m$. Then the underlying subsets of k satisfy

$$k_n \subseteq k_m.$$

Proof. Uses: `def:tracecore3-degree-n-fixed-subfield`

Write $m = nd$. If $a \in k_n$, then $\varphi_q^n(a) = a$. Induction on d gives $(\varphi_q^n)^d(a) = a$: the successor step applies φ_q^n to the preceding equality and then uses $\varphi_q^n(a) = a$. The standard iterate-multiplication identity $(\varphi_q^n)^d = \varphi_q^{nd}$ now gives $\varphi_q^m(a) = a$, which is precisely membership in k_m. □

Definition 15.13.3 (Canonical fixed-subfield inclusion). Uses: `def:tracecore3-degree-n-fixed-subfield`

Constructed by: `lemma:tracecore3-degree-fixed-subfield-mono`

Lean: `Weil.TraceCore3.degreeNFixedSubfieldInclusion`

For $h : n \mid m$, define

$$\iota_h = \iota_{n,m} : k_n \longrightarrow k_m$$

to be the k_0-algebra homomorphism induced by the preceding subset inclusion. Both fixed subfields carry the inherited k_0-embedding, so the inclusion commutes with the base maps. Its underlying function sends the subtype element $(a, \varphi_q^n(a) = a)$ to the subtype element with the same $a \in k$ and the newly proved m-fixedness certificate.

Lemma 15.13.4 (The canonical inclusions commute with the algebraic-closure maps). *Uses: def: tracecore3-degree-n-fixed-subfield-inclusion, def : tracecore3-degree-n-fixed-subfield*

Lean: Weil.TraceCore3.degreeNFixedSubfieldInclusion_toClosure

For $h : n \mid m$,

$$j_m \circ \iota_h = j_n : k_n \longrightarrow k.$$

Proof. Uses: `def:tracecore3-degree-fixed-subfield-inclusion`

Both k_0-algebra homomorphisms send a subtype element of k_n to its underlying element of k. Algebra-homomorphism extensionality proves the equality. □

Lemma 15.13.5 (Identity law for the canonical fixed-subfield inclusions). *Uses: def: tracecore3-degree-n-fixed-subfield-inclusion*

Lean: Weil.TraceCore3.degreeNFixedSubfieldInclusion_id

For the reflexive divisibility witness at n,

$$\iota_{n,n} = \mathrm{id}_{k_n}.$$

Proof. Uses: `def:tracecore3-degree-fixed-subfield-inclusion`

For every $a \in k_n$, both maps have underlying value $a \in k_n$. Algebra-homomorphism extensionality therefore reduces the goal to reflexivity on the underlying field element; proof irrelevance identifies the proposition-valued membership certificates. □

Lemma 15.13.6 (Composition law for the canonical fixed-subfield inclusions). *Uses:* def:tracecore3-degree-fixed-subfield-inclusion

Lean: Weil.TraceCore3.degreeNFixedSubfieldInclusion_comp

If $n \mid m$ and $m \mid \ell$, then

$$\iota_{m,\ell} \circ \iota_{n,m} = \iota_{n,\ell} : k_n \longrightarrow k_\ell.$$

Proof. Uses: def:tracecore3-degree-fixed-subfield-inclusion

Each side sends an element of k_n to the same underlying element of k, now regarded as an element of k_ℓ. Extensionality and proof irrelevance give the displayed equality. □

15.13.2 Point transitions and fixed-locus transitions

In this subsection write σ_n for the TraceCore3 scalar-extension map called s_n in Definition 15.8.6. This keeps it distinct from the inverse map named s_n in the earlier Final candidate record.

Definition 15.13.7 (Transition of rational points in a divisibility tower). Uses: def:tracecore3-scheme-finite-extension-point-type, def:tracecore3-degree-fixed-subfield-inclusion

Lean: Weil.TraceCore3.schemePointTransition

For $h : n \mid m$, define

$$t_h = t_{n,m} : R_{X_0,n} \longrightarrow R_{X_0,m}, \qquad t_h(y) := y \circ \mathrm{Spec}(\iota_h).$$

The morphism $\mathrm{Spec}(\iota_h) : \mathrm{Spec}\, k_m \to \mathrm{Spec}\, k_n$ is over $\mathrm{Spec}\, k_0$, so the composite is again a k_0-point.

Lemma 15.13.8 (Identity law for rational-point transitions). *Uses:* def:tracecore3-scheme-point-transition

Lean: Weil.TraceCore3.schemePointTransition_id

For $y \in R_{X_0,n}$,

$$t_{n,n}(y) = y.$$

Proof. Uses: def:tracecore3-scheme-point-transition, lemma:tracecore3-degree-fixed-subfield-inclusion-identity

Replace $\iota_{n,n}$ by the identity field homomorphism. The spectrum of an identity is an identity and composition with it leaves y unchanged. □

Lemma 15.13.9 (Composition law for rational-point transitions). *Uses:* def:tracecore3-scheme-point-transition

Lean: Weil.TraceCore3.schemePointTransition_comp

If $n \mid m$, $m \mid \ell$, and $y \in R_{X_0,n}$, then

$$t_{m,\ell}(t_{n,m}(y)) = t_{n,\ell}(y).$$

Proof. Uses: def:tracecore3-scheme-point-transition, lemma:tracecore3-degree-fixed-subfield-inclusion-composition

Expand the two transitions. Contravariant functoriality of Spec turns the composite of the two source maps into the spectrum of $\iota_{m,\ell} \circ \iota_{n,m} = \iota_{n,\ell}$. Associativity of scheme-morphism composition gives the equality. □

Lemma 15.13.10 (Scalar extension is constant along the rational-point tower). *Uses:* def:tracecore3-scheme-point-transition, def:tracecore3-rational-point-scalar-extension

Lean: Weil.TraceCore3.schemePointTransition_scalarExtension

For $h : n \mid m$ and $y \in R_{X_0,n}$,

$$\sigma_m(t_h(y)) = \sigma_n(y) : \mathrm{Spec}\, k \longrightarrow X_0.$$

Proof. Uses: def:tracecore3-scheme-point-transition, def:tracecore3-rational-point-scalar-extension, lemma:tracecore3-degree-fixed-subfield-inclusion-to-closure

The left side is $y \circ \mathrm{Spec}(\iota_h) \circ \mathrm{Spec}(j_m)$. Contravariant functoriality and $j_m \circ \iota_h = j_n$ identify the two source maps with $\mathrm{Spec}(j_n)$. The result is the defining composite for $\sigma_n(y)$. □

Lemma 15.13.11 (Monotonicity of geometric fixed loci). *Uses:* def:tracecore3-scheme-fixed-locus-subtype

Lean: Weil.TraceCore3.schemeFixedLocus_mono

If $h : n \mid m$ and $z \in \mathrm{Fix}_{X_0}(n)$, then its underlying geometric point satisfies

$$\Phi_{X_0}^m(|z|) = |z|.$$

Proof. Uses: def:tracecore3-scheme-fixed-locus-subtype
Write $m = nd$. The certificate of z is $\Phi_{X_0}^n(|z|) = |z|$. Induction on d, followed by the iterate-multiplication identity, gives $(\Phi_{X_0}^n)^d(|z|) = |z|$, hence the displayed m-fixedness. □

Definition 15.13.12 (Transition of geometric fixed points). Uses: def:tracecore3-scheme-fixed-locus-subtype

Constructed by: lemma:tracecore3-scheme-fixed-locus-mono

Lean: Weil.TraceCore3.schemeFixedPointTransition
For $h : n \mid m$, define

$$T_h^{\mathrm{fix}} : \mathrm{Fix}_{X_0}(n) \longrightarrow \mathrm{Fix}_{X_0}(m)$$

by retaining the underlying geometric point and replacing its fixedness proof by the preceding lemma.

Lemma 15.13.13 (Identity law for geometric fixed-point transitions). Uses: def:tracecore3-scheme-fixed-point-transition

Lean: Weil.TraceCore3.schemeFixedPointTransition_id
For $z \in \mathrm{Fix}_{X_0}(n)$,

$$T_{n,n}^{\mathrm{fix}}(z) = z.$$

Proof. Uses: def:tracecore3-scheme-fixed-point-transition
Both subtype elements have the same underlying geometric point. Subtype extensionality proves the equality. □

Lemma 15.13.14 (Composition law for geometric fixed-point transitions). Uses: def:tracecore3-scheme-fixed-point-transition

Lean: Weil.TraceCore3.schemeFixedPointTransition_comp
If $n \mid m$, $m \mid \ell$, *and* $z \in \mathrm{Fix}_{X_0}(n)$, *then*

$$T_{m,\ell}^{\mathrm{fix}}(T_{n,m}^{\mathrm{fix}}(z)) = T_{n,\ell}^{\mathrm{fix}}(z).$$

Proof. Uses: def:tracecore3-scheme-fixed-point-transition
Both sides retain the same underlying geometric point. Subtype extensionality discards the proposition-valued fixedness certificates. □

Lemma 15.13.15 (Naturality of the TraceCore3 rational-fixed equivalence). Uses: def:tracecore3-rational-fixed-equivalence, def:tracecore3-scheme-point-transition, def:tracecore3-scheme-fixed-point-transition

Lean: Weil.TraceCore3.rationalFixedEquivalence_towerNaturality
For $h : n \mid m$ *and* $y \in R_{X_0,n}$,

$$e_m(t_h(y)) = T_h^{\mathrm{fix}}(e_n(y)).$$

Proof. Uses: def:tracecore3-rational-fixed-equivalence, def:tracecore3-rational-to-fixed-map, lemma:tracecore3-scheme-point-transition-basechange, def:tracecore3-scheme-fixed-point-transition
The forward map of e_r is u_r, whose underlying point is σ_r. The base-change lemma identifies the underlying point on the left with $\sigma_n(y)$, which is also the underlying point on the right. Subtype extensionality finishes the proof. □

15.13.3 The two literal Final equivalences

Definition 15.13.16 (Canonical finite-extension point type for the final scheme). Uses: def:weil-final-tracecore3-scheme-data, def:tracecore3-scheme-finite-extension-point-type

Lean: Weil.Final.traceCore3SchemePointType
For $n \geq 1$, interpret the point type appearing as $X_0(\mathbf{F}_{q^n})$ in the final counting formula by the canonical type

$$X_0(k_n) = R_{\mathcal{T}(\mathcal{X}),n} = \mathrm{Hom}_{k_0}(\mathrm{Spec}\, k_n, X_0).$$

This is a type abbreviation using the fixed subfield inside k, not a choice of an external finite-field presentation.

Definition 15.13.17 (Final rational-point equivalence)**.** Uses: `def:tracecore3-zpc02-finite-extension-point-interface,def:weil-final-tracecore3-scheme-point-type`

Lean: `Weil.Final.traceCore3RationalPointEquiv`

Let $P = P^{\mathrm{ZPC}}_{\mathcal{T}(\mathcal{X})}$. For every $n \geq 1$, define

$$\epsilon_n : \mathrm{Rat}_P(n) \xrightarrow{\sim} X_0(k_n)$$

to be the identity equivalence after unfolding the two sides. Indeed, ZPC-02 defines $\mathrm{Rat}_P(n)$ to be the literal TraceCore3 scheme-point type, and the preceding Definition uses that same type for the final notation.

Lemma 15.13.18 (Left inverse for the Final rational-point equivalence)**.** Uses: *def:weil-final-tracecore3-rational-point-equivalence*

Lean: *Weil.Final.traceCore3RationalPointEquiv_leftInverse*

For $p \in \mathrm{Rat}_P(n)$,

$$\epsilon_n^{-1}(\epsilon_n(p)) = p.$$

Proof. Uses: `def:weil-final-tracecore3-rational-point-equivalence`

This is the left-inverse field of ϵ_n; after unfolding it is $p = p$. □

Lemma 15.13.19 (Right inverse for the Final rational-point equivalence)**.** Uses: *def:weil-final-tracecore3-rational-point-equivalence*

Lean: *Weil.Final.traceCore3RationalPointEquiv_rightInverse*

For $y \in X_0(k_n)$,

$$\epsilon_n(\epsilon_n^{-1}(y)) = y.$$

Proof. Uses: `def:weil-final-tracecore3-rational-point-equivalence`

This is the right-inverse field of ϵ_n; after unfolding it is $y = y$. □

Definition 15.13.20 (Tower transition on the Final finite-extension interface)**.** Uses: `def:weil-final-tracecore3-rational-point-equivalence,def:tracecore3-scheme-point-transition`

Lean: `Weil.Final.traceCore3InterfacePointTransition`

For $h : n \mid m$, define

$$\tau_h^P : operatorname{Rat}_P(n) \longrightarrow \mathrm{Rat}_P(m), \qquad \tau_h^P := \epsilon_m^{-1} \circ t_h \circ \epsilon_n.$$

Lemma 15.13.21 (Identity law for Final interface point transitions)**.** Uses: *def:weil-final-tracecore3-interface-point-transition*

Lean: *Weil.Final.traceCore3InterfacePointTransition_id*

For $p \in \mathrm{Rat}_P(n)$,

$$\tau^P_{n,n}(p) = p.$$

Proof. Uses: `def:weil-final-tracecore3-interface-point-transition,lemma:tracecore3-scheme-point-transition-identity,lemma:weil-final-tracecore3-rational-point-left-inverse`

Expand $\tau^P_{n,n}$, replace $t_{n,n}$ by the identity, and use $\epsilon_n^{-1}\epsilon_n = \mathrm{id}$. □

Lemma 15.13.22 (Composition law for Final interface point transitions)**.** Uses: *def:weil-final-tracecore3-interface-point-transition*

Lean: *Weil.Final.traceCore3InterfacePointTransition_comp*

If $n \mid m$, $m \mid \ell$, *and* $p \in \mathrm{Rat}_P(n)$, *then*

$$\tau^P_{m,\ell}(\tau^P_{n,m}(p)) = \tau^P_{n,\ell}(p).$$

Proof. Uses: `def:weil-final-tracecore3-interface-point-transition,lemma:tracecore3-scheme-point-transition-composition,lemma:weil-final-tracecore3-rational-point-right-inverse`

Expand both transitions. The middle pair $\epsilon_m\epsilon_m^{-1}$ cancels, and the TraceCore3 composition law replaces $t_{m,\ell}t_{n,m}$ by $t_{n,\ell}$. The remaining composite is the definition of $\tau^P_{n,\ell}$. □

Lemma 15.13.23 (Naturality of the Final rational-point equivalence in towers)**.** Uses: *def:weil-final-tracecore3-interface-point-transition, def:weil-final-tracecore3-rational-point-equivalence*

Lean: *Weil.Final.traceCore3RationalPointEquiv_towerNaturality*

For $h : n \mid m$ *and* $p \in \mathrm{Rat}_P(n)$,

$$\epsilon_m(\tau_h^P(p)) = t_h(\epsilon_n(p)).$$

Proof. Uses: def:weil-final-tracecore3-interface-point-transition,lemma:weil-final-tracecore3-rational-point-right-inverse
Unfold τ_h^P and cancel $\epsilon_m \epsilon_m^{-1}$. □

Lemma 15.13.24 (Base change of Final interface points is tower-compatible). *Uses: def:weil-final-tracecore3-interface-point-transition, def:tracecore3-rational-point-scalar-extension*

Lean: Weil.Final.traceCore3InterfacePointTransition_scalarExtension
For $h : n \mid m$ and $p \in \mathrm{Rat}_P(n)$,

$$\sigma_m\big(\epsilon_m(\tau_h^P(p))\big) = \sigma_n(\epsilon_n(p)).$$

Proof. Uses: lemma:weil-final-tracecore3-rational-point-tower-naturality,lemma:tracecore3-scheme-point-transition-basechange
Rewrite the argument on the left by rational-point tower naturality and then apply the TraceCore3 scalar-extension compatibility equation. □

Lemma 15.13.25 (Geometric fixed-point naturality of Final interface points). *Uses: def:weil-final-tracecore3-interface-point-transition, def:tracecore3-rational-fixed-equivalence, def:tracecore3-scheme-fixed-point-transition*

Lean: Weil.Final.traceCore3InterfacePointTransition_fixedNaturality
For $h : n \mid m$ and $p \in \mathrm{Rat}_P(n)$,

$$e_m\big(\epsilon_m(\tau_h^P(p))\big) = T_h^{\mathrm{fix}}\big(e_n(\epsilon_n(p))\big).$$

Proof. Uses: lemma:weil-final-tracecore3-rational-point-tower-naturality,lemma:tracecore3-rational-fixed-equivalence-tower-naturality
Rewrite the left argument by the first cited naturality equation and apply the second one. □

Lemma 15.13.26 (Compatibility with the selected ZPC-02 fixed-point map). *Uses: def:weil-final-tracecore3-rational-point-equivalence, def:tracecore3-zpc02-finite-extension-point-interface*

Lean: Weil.Final.traceCore3Zpc02FixedMapCompatibility
Let h_n denote the identity-on-carriers bridge used in ZPC-02. For $p \in \mathrm{Rat}_P(n)$,

$$h_n^{-1}(\rho_n(p)) = e_n(\epsilon_n(p)) \quad \textit{in } \mathrm{Fix}_{X_0}(n).$$

Proof. Uses: def:tracecore3-zpc02-finite-extension-point-interface,prop:tracecore3-zpc02-finite-extension-fixed-package,def:weil-final-tracecore3-rational-point-equivalence
By definition $\rho_n = e_n \,\mathrm{trans}\, h_n$. Apply h_n^{-1} and cancel the bridge. The equivalence ϵ_n is the identity rebind of the source, so the remaining term is the displayed TraceCore3 fixed point. □

Lemma 15.13.27 (Compatibility with the SCHFROB–14 geometric convention). *Uses: def:weil-final-tracecore3-rational-point-equivalence, def:weil-schfrob14-rational-scheme-fixed-equivalence*

Lean: conditional-interface: Weil.Final.traceCore3SchFrob14Compatibility
For a SCHFROB–14 input whose underlying finite-type datum is $\mathcal{T}(\mathcal{X})$, and for $p \in \mathrm{Rat}_P(n)$,

$$B_n\big(e_n^{\mathrm{sch}}(\epsilon_n(p))\big) = e_n(\epsilon_n(p)) \quad \textit{in } \mathrm{Fix}_{X_0}(n).$$

Proof. Uses: lemma:weil-schfrob14-geometric-convention-compatibility,thm:weil-efi-schfrob14-discharge,def:tracecore3-rational-fixed-equivalence,def:weil-final-tracecore3-rational-point-equivalence
The SCHFROB–14 compatibility lemma says that applying B_n to the scheme-fixed image of a point gives the TraceCore3 map u_n on that same point. The forward map of e_n is u_n. Substitute $\epsilon_n(p)$ for the point. The discharge theorem guarantees that the scheme-fixed map used here is the certified SCHFROB–14 dictionary, not a second fixed-point convention. □

Theorem 15.13.28 (Final rational-point/fixed-point compatibility assembly). *Uses: def:weil-final-tracecore3-rational-point-equivalence*

Lean: Weil.Final.traceCore3PointFixedDischarge
For the canonical fields k_n, the maps ϵ_n and ϵ_n^{-1}, the divisibility transitions τ_h^P, scalar extension to k, and the geometric fixed-locus transition form the point/fixed compatibility package required by the final construction.

Proof. Uses: thm:tracecore3-tracec-point-fixed-discharge-v1, thm:tracecore3-tracec2-point-fixed-discharge-v1, lemma:weil-final-tracecore3-rational-point-left-inverse, lemma:weil-final-tracecore3-rational-point-right-inverse, lemma:weil-final-tracecore3-interface-point-transition-identity, lemma:weil-final-tracecore3-interface-point-transition-composition, lemma:weil-final-tracecore3-interface-point-basechange, lemma:weil-final-tracecore3-interface-point-fixed-naturality, lemma:weil-final-tracecore3-zpc02-fixed-map-compatibility

The two TraceCore scheme discharges supply the finite-extension/fixed-point equivalence, geometric fixedness, descent, and the two inverse equations in the notation used by the final chapter. The separately named tower lemmas supply identity, composition, scalar-base-change, and fixed-locus naturality. The last compatibility lemma identifies the selected ZPC-02 fixed subtype with that same geometric fixed point. The separately stated SCHFROB–14 lemma above is an additional comparison for a supplied scheme-Frobenius input; its inhabitation is not used by this package. These are exactly the fields of the stated package. □

15.13.4 Closed points and finite structures

Definition 15.13.29 (Final closed-point equivalence). Uses: def:tracecore3-zpc04-closed-point-orbit-interface, def:tracecore3-scheme-closed-point-type, def:weil-final-scheme-closed-point-type, def:weil-final-tracecore3-scheme-data

Lean: Weil.Final.traceCore3ClosedPointEquiv

Let $C = C^{\mathrm{ZPC}}_{\mathcal{T}(\mathcal{X})}$. Define

$$\delta : \mathrm{ClPt}(C) \xrightarrow{\sim} \mathrm{ClPt}(X_0)$$

to be the identity equivalence after unfolding. ZPC-04 takes its closed carrier to be the TraceCore3 scheme closed-point subtype, while the final closed-point type is the identical subtype of the identical scheme X_0. Concretely, the forward map sends (x, h_x) to (x, h_x), and the reverse map has the same formula; subtype extensionality gives the inverse equations even if the two record projections are not definitionally reduced. This equivalence is independent of the rational-point equivalence ϵ_n.

Lemma 15.13.30 (Left inverse for the Final closed-point equivalence). Uses: def:weil-final-tracecore3-closed-point-equivalence

Lean: Weil.Final.traceCore3ClosedPointEquiv_leftInverse

For $c \in \mathrm{ClPt}(C)$,

$$\delta^{-1}(\delta(c)) = c.$$

Proof. Uses: def:weil-final-tracecore3-closed-point-equivalence

This is the left-inverse field of δ; after unfolding it is reflexivity on the closed-point subtype. □

Lemma 15.13.31 (Right inverse for the Final closed-point equivalence). Uses: def:weil-final-tracecore3-closed-point-equivalence

Lean: Weil.Final.traceCore3ClosedPointEquiv_rightInverse

For $x \in \mathrm{ClPt}(X_0)$,

$$\delta(\delta^{-1}(x)) = x.$$

Proof. Uses: def:weil-final-tracecore3-closed-point-equivalence

This is the right-inverse field of δ; after unfolding it is reflexivity on the closed-point subtype. □

Lemma 15.13.32 (Residue degree under the Final closed-point equivalence). Uses: def:weil-final-tracecore3-closed-point-equivalence, def:tracecore3-zpc04-closed-point-orbit-interface

Lean: Weil.Final.traceCore3ClosedPointEquiv_degree

For $c \in \mathrm{ClPt}(C)$,

$$\deg_C(c) = [\kappa(\delta(c)) : k_0].$$

Proof. Uses: def:tracecore3-zpc04-closed-point-orbit-interface, def:tracecore3-scheme-closed-point-degree, def:weil-final-tracecore3-closed-point-equivalence

The interface degree is definitionally $\deg_{X_0}(c)$, and that degree was defined as $[\kappa(c) : k_0]$. The equivalence δ retains the same closed point, so unfolding proves the equality. □

Lemma 15.13.33 (Orbit representative compatibility of the Final closed-point map). Uses: def:weil-final-tracecore3-closed-point-equivalence, def:tracecore3-zpc04-closed-point-orbit-interface

Lean: Weil.Final.traceCore3ClosedPointEquiv_representativeCompatibility

For $c \in \mathrm{ClPt}(C)$, *the orbit* $\theta_{X_0}(c)$ *is the geometric-Frobenius orbit of every geometric lift of the final closed point* $\delta(c)$.

Proof. Uses: prop:tracecore3-zpc04-closed-point-orbit-package,lemma:tracecore3-selected-lift-class-independent,def:weil-final-tracecore3-closed-point-equivalence

The ZPC-04 package identifies θ_{X_0} with the selected-residue-lift map followed by the quotient bridge. Selected-lift class independence says that replacing the selected residue embedding by any other geometric lift does not change its orbit class. Since δ retains the closed point, this is the asserted compatibility. □

Definition 15.13.34 (Final fixed-degree closed-point finset). Uses: def:weil-final-tracecore3-closed-point-equivalence,def:tracecore3-zpc03-degree-closed-point-finset

Lean: Weil.Final.traceCore3DegreeClosedPointFinset

For $m \geq 1$, define the final scheme finset

$$\mathcal{C}^{\mathrm{final}}_{X_0,m} := \mathcal{C}^{\mathrm{geom}}_{X_0,m}$$

through the literal closed-point rebind δ.

Lemma 15.13.35 (Degree-finset compatibility of the Final closed-point equivalence). *Uses: def:weil-final-tracecore3-degree-closed-point-finset, def:weil-degree-closed-point-finset*

Lean: Weil.Final.traceCore3ClosedPointEquiv_degreeFinsetCompatibility

For $m \geq 1$ and $c \in \mathrm{ClPt}(C)$,

$$c \in \mathcal{C}_m(C) \quad \Longleftrightarrow \quad \delta(c) \in \mathcal{C}^{\mathrm{final}}_{X_0,m}.$$

Proof. Uses: lemma:weil-degree-closed-point-finset-mem,lemma:tracecore3-zpc03-degree-closed-point-finset-mem,lemma:weil-final-tracecore3-closed-point-degree,def:weil-final-tracecore3-degree-closed-point-finset

Generic finset membership is $\deg_C(c) = m$. The TraceCore3 final finset membership is $[\kappa(\delta(c)) : k_0] = m$. The preceding degree equality identifies these predicates. □

Theorem 15.13.36 (Final closed-point/orbit compatibility assembly). *Uses: def:weil-final-tracecore3-closed-point-equivalence*

Lean: Weil.Final.traceCore3ClosedOrbitDischarge

The equivalence δ, the ZPC-04 orbit equivalence, and the explicit degree finsets form the closed-point/orbit compatibility package required by the final construction.

Proof. Uses: thm:tracecore3-tracec-closed-orbit-discharge-v1,prop:tracecore3-zpc03-closed-point-degree-package,prop:tracecore3-zpc04-closed-point-orbit-package,lemma:weil-final-tracecore3-closed-point-left-inverse,lemma:weil-final-tracecore3-closed-point-right-inverse,lemma:weil-final-tracecore3-closed-point-degree,lemma:weil-final-tracecore3-closed-point-representative-compatibility,lemma:weil-final-tracecore3-closed-point-degree-finset-compatibility

The TraceCore closed-orbit discharge supplies the explicit orbit arrows, representative independence, residue-degree equality, and fixed-degree finiteness. The two inverse lemmas rebind the carrier to the final closed-point subtype. The last three cited lemmas identify the rebind with the residue degree, representative class, and degree finsets required in the final notation. □

Lemma 15.13.37 (Finiteness of the Final finite-extension point type). *Uses: def:weil-final-tracecore3-scheme-point-type*

Lean: Weil.Final.traceCore3SchemePoint_finite

For every $n \geq 1$, the type $X_0(k_n)$ used by the final adapter is finite.

Proof. Uses: def:weil-final-tracecore3-scheme-point-type,lemma:tracecore3-scheme-finite-extension-point-finite

Unfold the final type abbreviation and apply the TraceCore3 finite affine cover theorem. □

Lemma 15.13.38 (Transport of the rational-point finite structure). *Uses: def:weil-final-tracecore3-rational-point-equivalence*

Lean: Weil.Final.traceCore3RationalPoint_finTypeTransport

For $n \geq 1$, the finite structure on $X_0(k_n)$ transported along ϵ_n is equal to the finite structure stored by $P^{\mathrm{ZPC}}_{X_0}$.

Proof. Uses: def:tracecore3-zpc02-finite-extension-point-interface,lemma:tracecore3-scheme-finite-extension-point-finite,def:weil-final-tracecore3-rational-point-equivalence,lemma:weil-final-tracecore3-scheme-point-finite

After unfolding ϵ_n, source and target are the same type and both finite structures enumerate its universal finite set. Apply `Fintype.ext`; membership in each stored universal finset is true for exactly the same elements. Proof irrelevance identifies the completeness fields. □

Lemma 15.13.39 (Finiteness of each Final fixed-degree closed-point stratum). *Uses:* *def:weil-final-tracecore3-degree-closed-point-finset*

Lean: *Weil.Final.traceCore3DegreeClosedPoint_finite*

For every $m \geq 1$, the subtype cut out by $\mathcal{C}^{\mathrm{final}}_{X_0,m}$ is finite.

Proof. Uses: def:weil-final-tracecore3-degree-closed-point-finset,prop:tracecore3-zpc03-closed-point-degree-package

The final finset is the ZPC-03 explicit finset under the identity rebind. The finite structure of its membership subtype is the standard `Fintype` attached to a finset. □

Lemma 15.13.40 (Transport of the fixed-degree closed-point finite structure). *Uses:* *def:weil-final-tracecore3-closed-point-equivalence*, *def:weil-final-tracecore3-degree-closed-point-finset*

Lean: *Weil.Final.traceCore3ClosedPoint_fintypeTransport*

For $m \geq 1$, transport along the restriction of δ identifies the finite structure used by $b_m(C)$ with the finite structure on the final degree-m closed-point stratum.

Proof. Uses: lemma:weil-final-tracecore3-closed-point-degree-finset-compatibility,def:weil-degree-closed-point-finset,lemma:weil-final-tracecore3-degree-closed-point-finite

The membership equivalence supplies a restriction of δ between the two finset subtypes. Transport the target finite structure through that equivalence. The resulting universal finset and the one used in the generic definition have identical membership predicates, so `Fintype.ext` identifies the structures. □

Lemma 15.13.41 (Local finiteness of the Final closed-point Euler family). *Uses:* *def:weil-final-tracecore3-closed-point-equivalence*, *def:weil-closed-point-euler-factor-family*

Lean: *Weil.Final.traceCore3ClosedEulerLocallyFinite*

For every truncation order N, only finitely many final scheme closed points have a nontrivial Euler component through order N.

Proof. Uses: lemma:weil-closed-point-euler-factor-family-locally-finite,lemma:weil-final-tracecore3-closed-point-degree,def:weil-final-tracecore3-closed-point-equivalence

The generic locally finite family for $C^{\mathrm{ZPC}}_{X_0}$ has component indexed by c equal to $(1-T^{\deg_C(c)})^{-1}-1$. Reindex its finite active set along δ. The degree equality identifies each component with the final scheme component, and equivalence injectivity transports finiteness of the active set. □

15.13.5 The actual Final candidate and certificate

Definition 15.13.42 (TraceCore3 candidate for the Final point realization). Uses: def:weil-final-point-realization-candidate,def:tracecore3-zpc01-finite-orbit-datum,def:tracecore3-zpc02-finite-extension-point-interface,def:tracecore3-zpc04-closed-point-orbit-interface,def:weil-final-tracecore3-rational-point-equivalence,def:weil-final-tracecore3-closed-point-equivalence

Lean: `Weil.Final.traceCore3PointRealizationCandidate`

For $\mathcal{T} = \mathcal{T}(\mathcal{X})$, define $\mathcal{P}^{\mathrm{TC3}}_{\mathcal{X}}$ by the literal fields

$$D := D^{\mathrm{ZPC}}_{\mathcal{T}}, \qquad P := P^{\mathrm{ZPC}}_{\mathcal{T}}, \qquad C := C^{\mathrm{ZPC}}_{\mathcal{T}},$$

$$r_n := \epsilon_n, \qquad s_n := \epsilon_n^{-1}, \qquad r_{\mathrm{cl}} := \delta, \qquad s_{\mathrm{cl}} := \delta^{-1}.$$

These are the two separately constructed equivalences above; no wrapper or assumed point-realization certificate occurs among the fields.

Proposition 15.13.43 (Predicate for the TraceCore3 Final point candidate). *Uses:* *def:weil-final-tracecore3-point-realization-candidate*, *def:weil-final-point-realization-predicate*

Lean: *Weil.Final.traceCore3PointRealizationCandidate_spec*

The candidate $\mathcal{P}^{\mathrm{TC3}}_{\mathcal{X}}$ satisfies

$$\mathrm{IsPointRealization}(\mathcal{P}^{\mathrm{TC3}}_{\mathcal{X}}).$$

Proof. Uses: lemma:weil-final-tracecore3-rational-point-left-inverse,lemma:weil-final-tracecore3-rational-point-right-inverse,lemma:weil-final-tracecore3-closed-point-left-inverse,lemma:weil-final-tracecore3-closed-point-right-inverse,lemma:weil-final-tracecore3-closed-point-degree,lemma:weil-final-tracecore3-rational-fintype-transport,lemma:weil-final-tracecore3-closed-fintype-transport

Unfold the candidate and the predicate. The first predicate clause is the two rational inverse lemmas. The second is the two closed-point inverse lemmas. The third is the residue-degree lemma. The fourth is the pair of finite-structure transport lemmas. Each cited result has exactly the source and target obtained by unfolding the corresponding candidate field. □

Definition 15.13.44 (Certified TraceCore3 Final point realization). Uses: `def:weil-final-point-realization-certificate`,`def:weil-final-tracecore3-point-realization-candidate`

Constructed by: `prop:weil-final-tracecore3-point-realization-predicate`

Lean: `Weil.Final.traceCore3PointRealizationCertificate`

Define

$$\mathbf{P}_{\mathcal{X}}^{\mathrm{TC3}} : \mathrm{PointRealizationCertificate}(\mathcal{X})$$

to have candidate $\mathcal{P}_{\mathcal{X}}^{\mathrm{TC3}}$ and predicate field the preceding proposition.

Theorem 15.13.45 (Final point-realization discharge). Uses: `def:weil-final-tracecore3-point-realization-certificate`

Lean: `Weil.Final.traceCore3PointRealizationDischarge`

Every final classical scheme datum has the canonical certified point realization $\mathbf{P}_{\mathcal{X}}^{\mathrm{TC3}}$; its point, closed, degree, finite-structure, tower, base-change, and geometric-Frobenius compatibilities are the named results above.

Proof. Uses: `def:weil-final-tracecore3-point-realization-certificate`,`thm:weil-final-tracecore3-point-fixed-discharge`,`thm:weil-final-tracecore3-closed-orbit-discharge`,`lemma:weil-final-tracecore3-rational-fintype-transport`,`lemma:weil-final-tracecore3-closed-fintype-transport`,`lemma:weil-final-tracecore3-closed-euler-locally-finite`

The first cited Definition is the required certificate. Its predicate field contains the inverse, degree, and finite-transport clauses. The two assembly theorems add the external tower, base-change, fixed-point, and orbit coherences required by the corresponding historical ledger statements. The last lemma supplies the local finiteness used by the formal Euler product. □

15.13.6 The remaining finite-field presentation boundary

15.13.7 External finite-field towers and the precise independence boundary

The canonical point-realization certificate uses the literal fixed subfields $k_n \subset k$, so it does not require any of the comparisons in this subsection. We nevertheless give the strongest comparison which is both canonical and true. An external presentation must remember its embeddings into the selected algebraic closure. Once those embeddings are part of the object, the comparison with k_n is unique. Independence then means naturality under the unique reindexing which commutes with the selected embeddings; it cannot mean equality after changing an embedding while fixing the carrier pointwise.

Definition 15.13.46 (Embedded external finite-field tower presentation). Uses: `def:tracecore-arithmetic-conventions`

Lean: `Weil.Final.ExternalFiniteFieldTowerPresentation`

An embedded external presentation $\mathcal{E}$ consists, for every $n \geq 1$, of a finite field $K_n^{\mathcal{E}}$, a k_0-algebra structure, an equality

$$\#K_n^{\mathcal{E}} = q^n,$$

and a selected injective k_0-algebra homomorphism

$$b_n^{\mathcal{E}} : K_n^{\mathcal{E}} \longrightarrow k.$$

For every divisibility witness $h : n \mid m$, it also contains a k_0-algebra homomorphism

$$u_h^{\mathcal{E}} : K_n^{\mathcal{E}} \longrightarrow K_m^{\mathcal{E}}$$

and the triangle equation

$$b_m^{\mathcal{E}} \circ u_h^{\mathcal{E}} = b_n^{\mathcal{E}}.$$

No identity, composition, or proof-independence axiom is stored: those laws will follow from injectivity of the selected embeddings.

Lemma 15.13.47 (Fermat identity in an external degree-n field). Uses: `def:weil-final-external-field-tower-presentation`

Lean: `Weil.Final.externalFiniteField_pow_card`

For $x \in K_n^{\mathcal{E}}$,

$$x^{q^n} = x.$$

Proof. Uses: def:weil-final-external-field-tower-presentation

Put $N = q^n = \#K_n^{\mathcal{E}}$. The assertion is immediate for $x = 0$. If $x \neq 0$, multiplication by x permutes the finite set $(K_n^{\mathcal{E}})^\times$. Taking the product of all its elements gives

$$\prod_{a \neq 0} a = \prod_{a \neq 0} xa = x^{N-1} \prod_{a \neq 0} a.$$

The displayed product is nonzero, so cancellation gives $x^{N-1} = 1$, and multiplication by x gives $x^N = x$. Replacing N by q^n proves the claim. □

Lemma 15.13.48 (The external embedding lands in the arithmetic fixed field). Uses: def:weil-final-external-field-tower-presentation, def:trace-core-arithmetic-conventions

Lean: Weil.Final.externalFiniteFieldEmbedding_arithmeticFixed

For $x \in K_n^{\mathcal{E}}$,

$$\varphi_q^n\left(b_n^{\mathcal{E}}(x)\right) = b_n^{\mathcal{E}}(x).$$

Proof. Uses: lemma:weil-final-external-field-fermat, lemma:weil-arithmetic-frobenius-power-formula

The arithmetic-Frobenius power formula and multiplicativity of $b_n^{\mathcal{E}}$ give

$$\varphi_q^n(b_n^{\mathcal{E}}(x)) = (b_n^{\mathcal{E}}(x))^{q^n} = b_n^{\mathcal{E}}(x^{q^n}).$$

The preceding Fermat identity turns the last expression into $b_n^{\mathcal{E}}(x)$. □

Lemma 15.13.49 (The external embedding is also geometrically n-fixed). Uses: def:weil-final-external-field-tower-presentation, def:trace-core-arithmetic-conventions

Lean: Weil.Final.externalFiniteFieldEmbedding_geometricFixed

For $x \in K_n^{\mathcal{E}}$,

$$F_k^n\left(b_n^{\mathcal{E}}(x)\right) = b_n^{\mathcal{E}}(x).$$

Proof. Uses: lemma:weil-final-external-field-embedding-arithmetic-fixed, lemma:weil-frobenius-inverse-identities

The iterates F_k^n and φ_q^n are inverse. Apply F_k^n to the arithmetic fixedness equation and cancel the composite $F_k^n \varphi_q^n$. □

Definition 15.13.50 (Comparison homomorphism to the canonical fixed subfield). Uses: def:weil-final-external-field-tower-presentation, def:tracecore3-degree-n-fixed-subfield

Constructed by: lemma:weil-final-external-field-embedding-arithmetic-fixed

Lean: Weil.Final.externalFiniteFieldComparisonHom

For $n \geq 1$, define the k_0-algebra homomorphism

$$e_n^{\mathcal{E}} : K_n^{\mathcal{E}} \longrightarrow k_n, \qquad e_n^{\mathcal{E}}(x) := \left(b_n^{\mathcal{E}}(x), \varphi_q^n(b_n^{\mathcal{E}}(x)) = b_n^{\mathcal{E}}(x)\right).$$

The second component is the preceding arithmetic fixedness lemma.

Lemma 15.13.51 (Comparison triangle in the algebraic closure). Uses: def:weil-final-external-field-comparison-hom, def:tracecore3-degree-n-fixed-subfield

Lean: Weil.Final.externalFiniteFieldComparisonHom_toClosure

One has

$$j_n \circ e_n^{\mathcal{E}} = b_n^{\mathcal{E}}.$$

Proof. Uses: def:weil-final-external-field-comparison-hom

Both sides send x to the same element $b_n^{\mathcal{E}}(x) \in k$. Algebra-homomorphism extensionality proves the equality. □

Lemma 15.13.52 (Injectivity of the comparison homomorphism). Uses: def:weil-final-external-field-comparison-hom

Lean: Weil.Final.externalFiniteFieldComparisonHom_injective

The map $e_n^{\mathcal{E}}$ is injective.

Proof. Uses: lemma:weil-final-external-field-comparison-to-closure

If $e_n^{\mathcal{E}}(x) = e_n^{\mathcal{E}}(y)$, apply j_n and use the comparison triangle to obtain $b_n^{\mathcal{E}}(x) = b_n^{\mathcal{E}}(y)$. The selected embedding $b_n^{\mathcal{E}}$ is injective, hence $x = y$. □

Lemma 15.13.53 (Surjectivity of the comparison homomorphism). *Uses:* def:weil-final-external-field-comparison-hom

Lean: Weil.Final.externalFiniteFieldComparisonHom_surjective

The map $e_n^{\mathcal{E}}$ is surjective.

Proof. Uses: lemma:weil-final-external-field-comparison-injective, lemma:tracecore3-degree-n-fixed-subfield-cardinality

The source and target are finite and both have cardinality q^n: this is a field of the external presentation for the source and the TraceCore3 fixed-field cardinality lemma for the target. An injective map between two finite sets of equal cardinality is surjective. □

Definition 15.13.54 (Canonical equivalence attached to an embedded presentation). Uses: def:weil-final-external-field-comparison-hom, def:weil-final-external-field-tower-presentation, def:tracecore3-degree-n-fixed-subfield

Constructed by: lemma:weil-final-external-field-comparison-injective, lemma:weil-final-external-field-comparison-surjective

Lean: Weil.Final.externalFiniteFieldComparisonEquiv

Let

$$\varepsilon_n^{\mathcal{E}} : K_n^{\mathcal{E}} \xrightarrow{\sim} k_n$$

be the k_0-algebra equivalence whose forward homomorphism is $e_n^{\mathcal{E}}$ and whose inverse sends $a \in k_n$ to the unique $x \in K_n^{\mathcal{E}}$ satisfying $e_n^{\mathcal{E}}(x) = a$.

Lemma 15.13.55 (First inverse formula for the external comparison). *Uses:* def:weil-final-external-field-comparison-equiv

Lean: Weil.Final.externalFiniteFieldComparisonEquiv_leftInverse

For $x \in K_n^{\mathcal{E}}$,

$$(\varepsilon_n^{\mathcal{E}})^{-1}(\varepsilon_n^{\mathcal{E}}(x)) = x.$$

Proof. Uses: def:weil-final-external-field-comparison-equiv

This is the left-inverse field of the equivalence. Equivalently, uniqueness of the preimage identifies the selected preimage of $e_n^{\mathcal{E}}(x)$ with x. □

Lemma 15.13.56 (Second inverse formula for the external comparison). *Uses:* def:weil-final-external-field-comparison-equiv

Lean: Weil.Final.externalFiniteFieldComparisonEquiv_rightInverse

For $a \in k_n$,

$$\varepsilon_n^{\mathcal{E}}((\varepsilon_n^{\mathcal{E}})^{-1}(a)) = a.$$

Proof. Uses: def:weil-final-external-field-comparison-equiv

This is the right-inverse field of the equivalence, or directly the defining property of the unique preimage selected in the preceding Definition. □

Lemma 15.13.57 (The equivalence has the prescribed embedding triangle). *Uses:* def:weil-final-external-field-comparison-equiv, def:tracecore3-degree-n-fixed-subfield

Lean: Weil.Final.externalFiniteFieldComparisonEquiv_toClosure

The forward map of the equivalence satisfies

$$j_n \circ \varepsilon_n^{\mathcal{E}} = b_n^{\mathcal{E}}.$$

Proof. Uses: lemma:weil-final-external-field-comparison-to-closure

The forward homomorphism of $\varepsilon_n^{\mathcal{E}}$ is definitionally $e_n^{\mathcal{E}}$, so this is the comparison triangle already proved. □

Lemma 15.13.58 (Arithmetic Frobenius preserves the canonical fixed subfield). *Uses:* def:tracecore3-degree-n-fixed-subfield, def:tracecore-arithmetic-conventions

Lean: Weil.Final.degreeNFixedSubfield_arithmeticFrobenius_stable

If $a \in k_n$, then $\varphi_q(j_n(a))$ again belongs to k_n.

Proof. Uses: def:tracecore3-degree-n-fixed-subfield

The iterates of one map commute. Hence

$$\varphi_q^n(\varphi_q(j_n(a))) = \varphi_q(\varphi_q^n(j_n(a))) = \varphi_q(j_n(a)),$$

where the last equality is the membership certificate of a. □

Definition 15.13.59 (Arithmetic Frobenius on a canonical fixed subfield). Uses: def:tracecore3-degree-n-fixed-subfield,def:tracecore-arithmetic-conventions

Constructed by: lemma:weil-final-canonical-field-arithmetic-frobenius-stable

Lean: Weil.Final.degreeNFixedSubfieldArithmeticFrobenius

Define $\alpha_n : k_n \simeq k_n$ by restricting φ_q to k_n. Its underlying function sends a to the subtype represented by $\varphi_q(j_n(a))$.

Definition 15.13.60 (Geometric Frobenius on a canonical fixed subfield). Uses: def:weil-final-canonical-field-arithmetic-frobenius

Lean: Weil.Final.degreeNFixedSubfieldGeometricFrobenius

Define

$$\gamma_n := \alpha_n^{-1} : k_n \xrightarrow{\sim} k_n.$$

It is the restriction of geometric field Frobenius $F_k = \varphi_q^{-1}$.

Lemma 15.13.61 (Arithmetic Frobenius has n-th iterate equal to the identity). Uses: def:weil-final-canonical-field-arithmetic-frobenius

Lean: Weil.Final.degreeNFixedSubfieldArithmeticFrobenius_pow

One has $\alpha_n^n = \mathrm{id}_{k_n}$.

Proof. Uses: def:weil-final-canonical-field-arithmetic-frobenius,def:tracecore3-degree-n-fixed-subfield

For $a \in k_n$, the underlying element of $\alpha_n^n(a)$ is $\varphi_q^n(j_n(a)) = j_n(a)$ by the defining membership certificate. Subtype extensionality proves the result. □

Lemma 15.13.62 (Geometric Frobenius has n-th iterate equal to the identity). Uses: def:weil-final-canonical-field-geometric-frobenius

Lean: Weil.Final.degreeNFixedSubfieldGeometricFrobenius_pow

One has $\gamma_n^n = \mathrm{id}_{k_n}$.

Proof. Uses: def:weil-final-canonical-field-geometric-frobenius,lemma:weil-final-canonical-field-arithmetic-frobenius-fixed

The inverse of α_n^n is $(\alpha_n^{-1})^n = \gamma_n^n$. The preceding lemma identifies α_n^n with the identity, whose inverse is itself. □

Lemma 15.13.63 (Arithmetic Frobenius is natural in the canonical tower). Uses: def:weil-final-canonical-field-arithmetic-frobenius, def:tracecore3-degree-fixed-subfield-inclusion

Lean: Weil.Final.degreeNFixedSubfieldArithmeticFrobenius_natural

For $h : n \mid m$,

$$\alpha_m \circ \iota_h = \iota_h \circ \alpha_n.$$

Proof. Uses: def:weil-final-canonical-field-arithmetic-frobenius,def:tracecore3-degree-fixed-subfield-inclusion

Both sides send a subtype element with underlying value $a \in k$ to the subtype element of k_m with underlying value $\varphi_q(a)$. Extensionality and proof irrelevance prove the equality. □

Lemma 15.13.64 (Geometric Frobenius is natural in the canonical tower). Uses: def:weil-final-canonical-field-geometric-frobenius, def:tracecore3-degree-fixed-subfield-inclusion

Lean: Weil.Final.degreeNFixedSubfieldGeometricFrobenius_natural

For $h : n \mid m$,

$$\gamma_m \circ \iota_h = \iota_h \circ \gamma_n.$$

Proof. Uses: def:weil-final-canonical-field-geometric-frobenius,lemma:weil-final-canonical-field-arithmetic-frobenius-transition

Compose the arithmetic naturality square on the left and right with the appropriate inverses α_m^{-1} and α_n^{-1}. The inverse equations cancel the two arithmetic Frobenius factors, leaving the displayed geometric square. □

Definition 15.13.65 (Arithmetic Frobenius in an external presentation). Uses: def:weil-final-external-field-comparison-equiv,def:weil-final-canonical-field-arithmetic-frobenius

Lean: Weil.Final.externalFiniteFieldArithmeticFrobenius

Transport canonical arithmetic Frobenius across the comparison:

$$\alpha_n^{\mathcal{E}} := (\varepsilon_n^{\mathcal{E}})^{-1} \circ \alpha_n \circ \varepsilon_n^{\mathcal{E}}.$$

Definition 15.13.66 (Geometric Frobenius in an external presentation). Uses: def:weil-final-external-field-comparison-equiv,def:weil-final-canonical-field-geometric-frobenius

Lean: Weil.Final.externalFiniteFieldGeometricFrobenius

Transport canonical geometric Frobenius across the comparison:

$$\gamma_n^{\mathcal{E}} := (\varepsilon_n^{\mathcal{E}})^{-1} \circ \gamma_n \circ \varepsilon_n^{\mathcal{E}}.$$

Lemma 15.13.67 (Arithmetic Frobenius comparison square). Uses: def:weil-final-external-field-arithmetic-frobenius, def:weil-final-external-field-comparison-equiv, def:weil-final-canonical-field-arithmetic-frobenius

Lean: Weil.Final.externalFiniteFieldArithmeticFrobenius_comparison

One has

$$\varepsilon_n^{\mathcal{E}} \circ \alpha_n^{\mathcal{E}} = \alpha_n \circ \varepsilon_n^{\mathcal{E}}.$$

Proof. Uses: def:weil-final-external-field-arithmetic-frobenius,lemma:weil-final-external-field-comparison-right-inverse

Expand $\alpha_n^{\mathcal{E}}$, reassociate, and cancel $\varepsilon_n^{\mathcal{E}}(\varepsilon_n^{\mathcal{E}})^{-1}$. □

Lemma 15.13.68 (Geometric Frobenius comparison square). Uses: def:weil-final-external-field-geometric-frobenius, def:weil-final-external-field-comparison-equiv, def:weil-final-canonical-field-geometric-frobenius

Lean: Weil.Final.externalFiniteFieldGeometricFrobenius_comparison

One has

$$\varepsilon_n^{\mathcal{E}} \circ \gamma_n^{\mathcal{E}} = \gamma_n \circ \varepsilon_n^{\mathcal{E}}.$$

Proof. Uses: def:weil-final-external-field-geometric-frobenius,lemma:weil-final-external-field-comparison-right-inverse

Expand $\gamma_n^{\mathcal{E}}$, reassociate, and cancel the adjacent comparison equivalence and its inverse. □

Lemma 15.13.69 (Arithmetic fixedness in an external presentation). Uses: def:weil-final-external-field-arithmetic-frobenius

Lean: Weil.Final.externalFiniteFieldArithmeticFrobenius_pow

One has $(\alpha_n^{\mathcal{E}})^n = \mathrm{id}$.

Proof. Uses: def:weil-final-external-field-arithmetic-frobenius,lemma:weil-final-canonical-field-arithmetic-frobenius-fixed,lemma:weil-final-external-field-comparison-left-inverse,lemma:weil-final-external-field-comparison-right-inverse

Iterating the conjugation formula cancels every adjacent $\varepsilon_n^{\mathcal{E}}(\varepsilon_n^{\mathcal{E}})^{-1}$, leaving $(\varepsilon_n^{\mathcal{E}})^{-1}\alpha_n^n\varepsilon_n^{\mathcal{E}}$. Replace α_n^n by the identity and use the two inverse formulas. □

Lemma 15.13.70 (Geometric fixedness in an external presentation). Uses: def:weil-final-external-field-geometric-frobenius

Lean: Weil.Final.externalFiniteFieldGeometricFrobenius_pow

One has $(\gamma_n^{\mathcal{E}})^n = \mathrm{id}$.

Proof. Uses: def:weil-final-external-field-geometric-frobenius,lemma:weil-final-canonical-field-geometric-frobenius-fixed,lemma:weil-final-external-field-comparison-left-inverse,lemma:weil-final-external-field-comparison-right-inverse

The same conjugation calculation as in the arithmetic case reduces the iterate to $(\varepsilon_n^{\mathcal{E}})^{-1}\gamma_n^n\varepsilon_n^{\mathcal{E}}$. Canonical geometric fixedness and the inverse formulas reduce this composite to the identity. □

Lemma 15.13.71 (Uniqueness of an external transition from its closure triangle). Uses: def:weil-final-external-field-tower-presentation

Lean: Weil.Final.externalFiniteFieldTransition_unique

Let $v, w : K_n^{\mathcal{E}} \to K_m^{\mathcal{E}}$ *be* k_0*-algebra homomorphisms. If* $b_m^{\mathcal{E}} v = b_n^{\mathcal{E}} = b_m^{\mathcal{E}} w$, *then* $v = w$.

Proof. Uses: def:weil-final-external-field-tower-presentation

For every x, the two triangle equations give $b_m^{\mathcal{E}}(v(x)) = b_m^{\mathcal{E}}(w(x))$. Injectivity of the selected embedding $b_m^{\mathcal{E}}$ gives $v(x) = w(x)$, and extensionality gives $v = w$. □

Lemma 15.13.72 (External transitions do not depend on divisibility proofs). *Uses:* def:weil-final-external-field-tower-presentation

Lean: Weil.Final.externalFiniteFieldTransition_proofIrrel

If $h, h' : n \mid m$, then

$$u^{\mathcal{E}}_h = u^{\mathcal{E}}_{h'}.$$

Proof. Uses: lemma:weil-final-external-field-transition-unique

Both transition maps satisfy the stored triangle with $b^{\mathcal{E}}_m$ and $b^{\mathcal{E}}_n$. Transition uniqueness identifies them. □

Lemma 15.13.73 (Identity law in every external tower). *Uses:* def:weil-final-external-field-tower-presentation

Lean: Weil.Final.externalFiniteFieldTransition_id

For the reflexive divisibility witness at n,

$$u^{\mathcal{E}}_{n,n} = \mathrm{id}_{K^{\mathcal{E}}_n}.$$

Proof. Uses: lemma:weil-final-external-field-transition-unique

The stored transition and the identity both become $b^{\mathcal{E}}_n$ after composition with $b^{\mathcal{E}}_n$. Apply transition uniqueness. □

Lemma 15.13.74 (Composition law in every external tower). *Uses:* def:weil-final-external-field-tower-presentation

Lean: Weil.Final.externalFiniteFieldTransition_comp

For $n \mid m \mid \ell$,

$$u^{\mathcal{E}}_{m,\ell} \circ u^{\mathcal{E}}_{n,m} = u^{\mathcal{E}}_{n,\ell}.$$

Proof. Uses: lemma:weil-final-external-field-transition-unique

After composition with $b^{\mathcal{E}}_\ell$, the left side becomes $b^{\mathcal{E}}_m u^{\mathcal{E}}_{n,m} = b^{\mathcal{E}}_n$; the right side has the same stored triangle. Apply transition uniqueness. □

Lemma 15.13.75 (The canonical comparison is natural in divisibility towers). *Uses:* def:weil-final-external-field-comparison-equiv, def:tracecore3-degree-fixed-subfield-inclusion

Lean: Weil.Final.externalFiniteFieldComparisonEquiv_natural

For $h : n \mid m$,

$$\varepsilon^{\mathcal{E}}_m \circ u^{\mathcal{E}}_h = \iota_h \circ \varepsilon^{\mathcal{E}}_n.$$

Proof. Uses: lemma:weil-final-external-field-comparison-equiv-to-closure, lemma:tracecore3-degree-fixed-subfield-inclusion-to-closure

Compose both sides with j_m. The left side becomes $b^{\mathcal{E}}_m u^{\mathcal{E}}_h = b^{\mathcal{E}}_n$. The right side becomes $j_n \varepsilon^{\mathcal{E}}_n = b^{\mathcal{E}}_n$. Since j_m is injective, the two maps are equal. □

Lemma 15.13.76 (The inverse comparison is natural in divisibility towers). *Uses:* def:weil-final-external-field-comparison-equiv, def:tracecore3-degree-fixed-subfield-inclusion

Lean: Weil.Final.externalFiniteFieldComparisonEquiv_symm_natural

For $h : n \mid m$,

$$u^{\mathcal{E}}_h \circ (\varepsilon^{\mathcal{E}}_n)^{-1} = (\varepsilon^{\mathcal{E}}_m)^{-1} \circ \iota_h.$$

Proof. Uses: lemma:weil-final-external-field-comparison-transition, lemma:weil-final-external-field-comparison-left-inverse, lemma:weil-final-external-field-comparison-right-inverse

Precompose the comparison-transition square with $(\varepsilon^{\mathcal{E}}_n)^{-1}$, then postcompose with $(\varepsilon^{\mathcal{E}}_m)^{-1}$. The two inverse equations cancel the comparison factors and give the displayed equality. □

Lemma 15.13.77 (Arithmetic Frobenius is natural in every external tower). *Uses:* def:weil-final-external-field-arithmetic-frobenius, def:weil-final-external-field-tower-presentation

Lean: Weil.Final.externalFiniteFieldArithmeticFrobenius_natural

For $h : n \mid m$,

$$\alpha^{\mathcal{E}}_m \circ u^{\mathcal{E}}_h = u^{\mathcal{E}}_h \circ \alpha^{\mathcal{E}}_n.$$

Proof. Uses: lemma:weil-final-external-field-arithmetic-frobenius-comparison,lemma:weil-final-external-field-comparison-transition,lemma:weil-final-canonical-field-arithmetic-frobenius-transition,lemma:weil-final-external-field-comparison-injective

Postcompose both sides with the injective equivalence $\varepsilon_m^{\mathcal{E}}$. The comparison squares turn the desired equality into canonical arithmetic-Frobenius naturality, with the two occurrences of the external transition replaced by the comparison-transition square. □

Lemma 15.13.78 (Geometric Frobenius is natural in every external tower). Uses: def:weil-final-external-field-geometric-frobenius, def:weil-final-external-field-tower-presentation

Lean: Weil.Final.externalFiniteFieldGeometricFrobenius_natural

For $h : n \mid m$,

$$\gamma_m^{\mathcal{E}} \circ u_h^{\mathcal{E}} = u_h^{\mathcal{E}} \circ \gamma_n^{\mathcal{E}}.$$

Proof. Uses: lemma:weil-final-external-field-geometric-frobenius-comparison,lemma:weil-final-external-field-comparison-transition,lemma:weil-final-canonical-field-geometric-frobenius-transition,lemma:weil-final-external-field-comparison-injective

Apply $\varepsilon_m^{\mathcal{E}}$ to both sides. Geometric comparison naturality and the tower comparison square reduce the result to the canonical geometric-Frobenius transition square. Injectivity of the equivalence gives the original equality. □

Definition 15.13.79 (Canonical change of embedded presentation). Uses: def:weil-final-external-field-comparison-equiv

Lean: Weil.Final.externalFiniteFieldPresentationChange

For two embedded presentations $\mathcal{E}, \mathcal{E}'$, define

$$c_n^{\mathcal{E},\mathcal{E}'} := (\varepsilon_n^{\mathcal{E}'})^{-1} \circ \varepsilon_n^{\mathcal{E}} : K_n^{\mathcal{E}} \xrightarrow{\sim} K_n^{\mathcal{E}'}.$$

Lemma 15.13.80 (A change of presentation commutes with closure embeddings). Uses: def:weil-final-external-field-presentation-change, def:weil-final-external-field-tower-presentation

Lean: Weil.Final.externalFiniteFieldPresentationChange_toClosure

One has

$$b_n^{\mathcal{E}'} \circ c_n^{\mathcal{E},\mathcal{E}'} = b_n^{\mathcal{E}}.$$

Proof. Uses: def:weil-final-external-field-presentation-change,lemma:weil-final-external-field-comparison-equiv-to-closure,lemma:weil-final-external-field-comparison-left-inverse,lemma:weil-final-external-field-comparison-right-inverse

Replace both embeddings by j_n followed by their comparison equivalences. The adjacent $\varepsilon_n^{\mathcal{E}'}(\varepsilon_n^{\mathcal{E}'})^{-1}$ cancels, leaving $j_n \varepsilon_n^{\mathcal{E}} = b_n^{\mathcal{E}}$. □

Lemma 15.13.81 (Uniqueness of a change of embedded presentation). Uses: def:weil-final-external-field-presentation-change, def:weil-final-external-field-tower-presentation

Lean: Weil.Final.externalFiniteFieldPresentationChange_unique

If $a : K_n^{\mathcal{E}} \to K_n^{\mathcal{E}'}$ is a k_0-algebra homomorphism satisfying $b_n^{\mathcal{E}'} a = b_n^{\mathcal{E}}$, then its underlying homomorphism equals that of $c_n^{\mathcal{E},\mathcal{E}'}$.

Proof. Uses: lemma:weil-final-external-field-presentation-change-to-closure

For every x, applying the two closure triangles gives

$$b_n^{\mathcal{E}'}(a(x)) = b_n^{\mathcal{E}}(x) = b_n^{\mathcal{E}'}(c_n^{\mathcal{E},\mathcal{E}'}(x)).$$

Injectivity of $b_n^{\mathcal{E}'}$, followed by extensionality, gives the claimed equality. □

Lemma 15.13.82 (Comparison naturality under change of presentation). Uses: def:weil-final-external-field-presentation-change, def:weil-final-external-field-comparison-equiv

Lean: Weil.Final.externalFiniteFieldPresentationChange_comparison

One has

$$\varepsilon_n^{\mathcal{E}'} \circ c_n^{\mathcal{E},\mathcal{E}'} = \varepsilon_n^{\mathcal{E}}.$$

Proof. Uses: def:weil-final-external-field-presentation-change,lemma:weil-final-external-field-comparison-right-inverse

Expand the change map and cancel $\varepsilon_n^{\mathcal{E}'}(\varepsilon_n^{\mathcal{E}'})^{-1}$. □

Lemma 15.13.83 (Identity law for changes of presentation). *Uses:* def:weil-final-external-field-presentation-change

Lean: Weil.Final.externalFiniteFieldPresentationChange_id

One has $c_n^{\mathcal{E},\mathcal{E}} = \mathrm{id}$.

Proof. Uses: lemma:weil-final-external-field-presentation-change-unique

The identity has the required closure triangle. Uniqueness identifies it with the canonical change map. □

Lemma 15.13.84 (Composition law for changes of presentation). *Uses:* def:weil-final-external-field-presentation-change

Lean: Weil.Final.externalFiniteFieldPresentationChange_comp

For three presentations,

$$c_n^{\mathcal{E}',\mathcal{E}''} \circ c_n^{\mathcal{E},\mathcal{E}'} = c_n^{\mathcal{E},\mathcal{E}''}.$$

Proof. Uses: lemma:weil-final-external-field-presentation-change-unique,lemma:weil-final-external-field-presentation-change-to-closure

The composite has closure triangle $b_n^{\mathcal{E}''} c_n^{\mathcal{E}',\mathcal{E}''} c_n^{\mathcal{E},\mathcal{E}'} = b_n^{\mathcal{E}}$. It is therefore the unique change map from $\mathcal{E}$ to $\mathcal{E}''$. □

Lemma 15.13.85 (Inverse law for changes of presentation). *Uses:* def:weil-final-external-field-presentation-change

Lean: Weil.Final.externalFiniteFieldPresentationChange_inv

The inverse of $c_n^{\mathcal{E},\mathcal{E}'}$ *is* $c_n^{\mathcal{E}',\mathcal{E}}$.

Proof. Uses: lemma:weil-final-external-field-presentation-change-composition,lemma:weil-final-external-field-presentation-change-identity

Apply the composition law in the two orders and then the identity law. The two resulting equations are the inverse equations. □

Lemma 15.13.86 (Tower naturality of changes of presentation). *Uses:* def:weil-final-external-field-presentation-change,def:weil-final-external-field-tower-presentation

Lean: Weil.Final.externalFiniteFieldPresentationChange_natural

For $h : n \mid m$,

$$u_h^{\mathcal{E}'} \circ c_n^{\mathcal{E},\mathcal{E}'} = c_m^{\mathcal{E},\mathcal{E}'} \circ u_h^{\mathcal{E}}.$$

Proof. Uses: lemma:weil-final-external-field-transition-unique,lemma:weil-final-external-field-presentation-change-to-closure

After applying $b_m^{\mathcal{E}'}$, each side becomes $b_n^{\mathcal{E}}$: on the left use the transition triangle in $\mathcal{E}'$ and the degree-n change triangle; on the right use the degree-m change triangle and the transition triangle in $\mathcal{E}$. Injectivity of $b_m^{\mathcal{E}'}$ proves the equality. □

Lemma 15.13.87 (Arithmetic Frobenius naturality under presentation change). *Uses:* def:weil-final-external-field-presentation-change,def:weil-final-external-field-arithmetic-frobenius

Lean: Weil.Final.externalFiniteFieldPresentationChange_arithmeticFrobenius

One has

$$\alpha_n^{\mathcal{E}'} \circ c_n^{\mathcal{E},\mathcal{E}'} = c_n^{\mathcal{E},\mathcal{E}'} \circ \alpha_n^{\mathcal{E}}.$$

Proof. Uses: lemma:weil-final-external-field-arithmetic-frobenius-comparison,lemma:weil-final-external-field-presentation-change-comparison,lemma:weil-final-external-field-comparison-injective

Apply $\varepsilon_n^{\mathcal{E}'}$ to both sides. The comparison square for arithmetic Frobenius and change-of-presentation comparison naturality reduce both sides to $\alpha_n \varepsilon_n^{\mathcal{E}}$. Injectivity gives the result. □

Lemma 15.13.88 (Geometric Frobenius naturality under presentation change). *Uses:* def:weil-final-external-field-presentation-change,def:weil-final-external-field-geometric-frobenius

Lean: Weil.Final.externalFiniteFieldPresentationChange_geometricFrobenius

One has

$$\gamma_n^{\mathcal{E}'} \circ c_n^{\mathcal{E},\mathcal{E}'} = c_n^{\mathcal{E},\mathcal{E}'} \circ \gamma_n^{\mathcal{E}}.$$

Proof. Uses: lemma:weil-final-external-field-geometric-frobenius-comparison, lemma:weil-final-external-field-presentation-change-comparison, lemma:weil-final-external-field-comparison-injective

Apply $\varepsilon_n^{\mathcal{E}'}$. The geometric comparison squares reduce both sides to $\gamma_n \varepsilon_n^{\mathcal{E}}$, and injectivity of the equivalence finishes the proof. □

Theorem 15.13.89 (Pointed presentation-independence theorem). *Uses:* *def:weil-final-external-field-tower-presentation, def:weil-final-external-field-comparison-equiv, def:weil-final-external-field-presentation-change*

Lean: *Weil.Final.externalFiniteFieldPointedPresentationIndependence*

Every embedded external finite-field tower admits a unique comparison with the canonical fixed-subfield tower over k. The comparisons have both inverse formulas, commute with all divisibility transitions and with arithmetic and geometric Frobenius, and are natural under the unique changes of embedded presentation. Those changes satisfy identity, composition, and inverse laws. Thus the groupoid of presentations carrying selected compatible embeddings into k is canonically equivalent to the canonical fixed-subfield presentation.

Proof. Uses: lemma:weil-final-external-field-comparison-left-inverse, lemma:weil-final-external-field-comparison-right-inverse, lemma:weil-final-external-field-comparison-equiv-to-closure, lemma:weil-final-external-field-comparison-transition, lemma:weil-final-external-field-comparison-inverse-transition, lemma:weil-final-external-field-arithmetic-frobenius-fixed, lemma:weil-final-external-field-geometric-frobenius-fixed, lemma:weil-final-external-field-arithmetic-frobenius-transition, lemma:weil-final-external-field-geometric-frobenius-transition, lemma:weil-final-external-field-presentation-change-unique, lemma:weil-final-external-field-presentation-change-comparison, lemma:weil-final-external-field-presentation-change-identity, lemma:weil-final-external-field-presentation-change-composition, lemma:weil-final-external-field-presentation-change-inverse, lemma:weil-final-external-field-presentation-change-transition, lemma:weil-final-external-field-presentation-change-arithmetic-frobenius, lemma:weil-final-external-field-presentation-change-geometric-frobenius

The cited comparison lemmas give existence, uniqueness, the two inverse formulas, closure compatibility, tower naturality, and both Frobenius compatibilities. The presentation-change lemmas give the unique reindexing between any two embedded models and prove its identity, composition, inverse, tower, and Frobenius laws. These are exactly the object, morphism, and coherence fields asserted in the statement. □

Definition 15.13.90 (Frobenius-twisted canonical presentation). Uses: def:weil-final-external-field-tower-presentation, def:weil-final-canonical-field-arithmetic-frobenius, def:tracecore3-degree-fixed-subfield-inclusion

Constructed by: lemma:weil-final-canonical-field-arithmetic-frobenius-transition, lemma:tracecore3-degree-fixed-subfield-inclusion-to-closure, lemma:tracecore3-degree-n-fixed-subfield-cardinality

Lean: Weil.Final.canonicalFiniteFieldFrobeniusTwistPresentation

For $r \geq 0$, define an embedded presentation $\mathcal{K}^{(r)}$ by

$$K_n^{\mathcal{K}^{(r)}} := k_n, \qquad b_n^{\mathcal{K}^{(r)}} := j_n \circ \alpha_n^r, \qquad u_h^{\mathcal{K}^{(r)}} := \iota_h.$$

The cardinality field is the canonical fixed-subfield cardinality lemma. The transition triangle follows by iterating arithmetic-Frobenius naturality and then using $j_m \iota_h = j_n$.

Lemma 15.13.91 (Comparison for a Frobenius-twisted presentation). *Uses:* *def:weil-final-canonical-field-frobenius-twist-presentation, def:weil-final-external-field-comparison-equiv*

Lean: *Weil.Final.canonicalFiniteFieldFrobeniusTwistComparison*

For every $r \geq 0$,

$$\varepsilon_n^{\mathcal{K}^{(r)}} = \alpha_n^r.$$

Proof. Uses: def:weil-final-external-field-comparison-hom, def:weil-final-canonical-field-frobenius-twist-presentation

The forward map of the left side sends $a \in k_n$ to the subtype whose underlying element of k is $j_n(\alpha_n^r(a))$. This is exactly the forward map of α_n^r. Extensionality identifies the algebra equivalences. □

Lemma 15.13.92 (Arithmetic Frobenius is nontrivial in degree two). *Uses:* *def:weil-final-canonical-field-arithmetic-frobenius, def:tracecore3-degree-n-fixed-subfield*

Lean: *Weil.Final.degreeTwoFixedSubfieldArithmeticFrobenius_ne_id*

The automorphism $\alpha_2 : k_2 \simeq k_2$ is not the identity.

Proof. Uses: `lemma:tracecore3-degree-n-fixed-subfield-cardinality`

Suppose α_2 were the identity. Then every $a \in k_2$ would satisfy $\varphi_q(j_2(a)) = j_2(a)$, hence would determine an element of k_1. This gives an injection $k_2 \hookrightarrow k_1$. The fixed-subfield cardinality lemma would then give $q^2 \leq q$, contrary to $q \geq 2$. Therefore $\alpha_2 \neq \mathrm{id}$. □

Proposition 15.13.93 (Absolute equality under a change of embedding is false). *Uses: def:weil-final-canonical-field-frobenius-twist-presentation, def:weil-final-external-field-comparison-equiv*

Lean: Weil.Final.externalFiniteFieldAbsoluteChoiceIndependence_false

There are two compatible embedded presentations with the same fields and the same divisibility transitions whose comparison equivalences with the canonical tower are unequal in degree two.

Proof. Uses: `lemma:weil-final-canonical-field-frobenius-twist-comparison`, `lemma:weil-final-canonical-field-degree-two-frobenius-nontrivial`

Take $\mathcal{K}^{(0)}$ and $\mathcal{K}^{(1)}$. They have the same carrier k_n and transition ι_h; only the selected closure embedding is changed from j_n to $j_n\alpha_n$. Their degree-two comparison maps are, respectively, id_{k_2} and α_2, which are unequal by the preceding lemma. □

Proposition 15.13.94 (Presentation independence for external finite-field models). *Uses: def:tracecore3-degree-n-fixed-subfield, def:tracecore3-degree-fixed-subfield-inclusion, def:weil-final-external-field-tower-presentation*

Lean: blueprint-target: Weil.Final.finiteFieldPresentationIndependence

For arbitrary selected finite fields K_n of cardinality q^n, with compatible k_0-embeddings and transition maps, construct equivalences $K_n \simeq k_n$ commuting with the embeddings into k and with every divisibility transition, and prove literal independence from all choices without reindexing the external presentations.

Open obligation: OBL-FINAL-POINT-FIELD

Proof. Uses: `thm:weil-final-external-field-pointed-presentation-independence`, `prop:weil-final-external-field-absolute-choice-independence-false`

The pointed, reindexing-natural form is proved by Theorem 15.13.89. The stronger literal equality requested here is false: the two globally compatible Frobenius-twisted presentations in Proposition 15.13.93 have unequal degree-two comparison maps. Consequently this historical row cannot be promoted without replacing "independence" by naturality under the unique embedding-compatible reindexing, or by passing to invariants under that reindexing. □

Chapter 16

Atomic terminal assembly of the Weil conjectures

Status. All Frobenius operators in this chapter are geometric. Geometric Frobenius acts on $E(1)$ by q^{-1}, hence on $E(-d)$ by q^d. The degree-i factor is always $P_i(T) = \det(1 - TF_i)$. Thus a Frobenius eigenvalue is a reciprocal root, whereas an ordinary polynomial root is its inverse. No six-functor, trace-formula, Poincare-duality, comparison, or weight theorem is introduced as a foundation axiom here: the chapter only projects and combines the individually proved outputs of the preceding modules.

16.1 The already constructed geometric input

Status. This section is a development-only conditional wrapper. Its input datum is not an existence theorem and none of its fields may discharge an upstream obligation. Existence remains blocked, in particular, by `OBL-FINAL-COH-TRACE`, `OBL-FINAL-TOP-PAIRING`, `OBL-FINAL-WEIGHT-REALIZATION`, `OBL-TRACE-FORMULA`, `OBL-POINCARE-DUALITY`, `OBL-SMOOTH-PROPER-PURITY`, and `OBL-REALIZATION-INDEPENDENCE`. Every theorem below therefore proves only an implication from the displayed input until those precise ancestors are proved and the wrapper is replaced by their constructed outputs.

Definition 16.1.1 (Terminal assembly input). Uses: `def:weil-final-master-input-data`

Lean: `conditional-interface:def:phase4-weil-assembly-input`

A terminal assembly input $\mathcal{A}$ is a final Weil master input datum for a smooth, projective, geometrically connected $X_0/\mathbf{F}_q$ of pure dimension d. In particular, the datum has fields whose intended realizations are the point realization, compactly supported and ordinary cohomology, geometric Frobenius, every-iterate trace formula, proper Poincare pairings, compatible coefficient realizations, and the smooth-proper purity output. This Definition constructs none of those fields, asserts no instance of their existence, and cannot be cited to close any of the obligations listed above.

Definition 16.1.2 (Assembly point number). Uses: `def:phase4-weil-assembly-input,def:weil-final-scheme-point-number`

Lean: `conditional-interface:def:phase4-weil-point-number`

For $n \geq 1$, define

$$N_n(X_0) := \#\,\mathrm{Rat}_{P_X}(n),$$

where P_X is the finite-extension point interface in $\mathcal{A}$.

Lemma 16.1.3 (Assembly point number is the geometric point count). Uses: `def:phase4-weil-point-number`

Lean: `conditional-interface:lemma:phase4-weil-point-number-cardinality`

For every $n \geq 1$,

$$N_n(X_0) = \#X_0(\mathbf{F}_{q^n}).$$

Proof. Uses: `def:phase4-weil-point-number,lemma:weil-final-scheme-point-number-cardinality`

Instantiate the preceding scheme point-realization lemma with the point realization projected from $\mathcal{A}$, then unfold the assembly alias. □

Definition 16.1.4 (Assembly counting logarithm). Uses: def:phase4-weil-point-number

Lean: conditional-interface:def:phase4-weil-counting-logarithm

Define $L_X(T) \in \mathbf{Q}[[T]]$ by

$$[T^0]L_X = 0, \qquad [T^n]L_X = \frac{N_n(X_0)}{n} \quad (n \geq 1).$$

Definition 16.1.5 (Assembly zeta series). Uses: def:phase4-weil-counting-logarithm,def:weil-formal-exp

Lean: conditional-interface:def:phase4-weil-zeta-series

Define

$$Z_X(T) := \mathrm{Exp}(L_X(T)) \in \mathbf{Q}[[T]].$$

Lemma 16.1.6 (Exponential point-count formula). Uses: def:phase4-weil-point-number,def:phase4-weil-zeta-series

Lean: conditional-interface:lemma:phase4-weil-zeta-exponential

One has

$$Z_X(T) = \exp\left(\sum_{n\geq 1} \#X_0(\mathbf{F}_{q^n})\frac{T^n}{n}\right).$$

Proof. Uses: def:phase4-weil-counting-logarithm,def:phase4-weil-zeta-series,lemma:phase4-weil-point-number-cardinality

Unfold the zeta and logarithm Definitions. Replace the coefficient $N_n(X_0)$ by the cardinality from the preceding lemma in each positive degree. The constant coefficient remains zero, so coefficient extensionality gives the displayed equality. □

Definition 16.1.7 (Assembly closed-point Euler product). Uses: def:phase4-weil-assembly-input,def:weil-final-scheme-closed-euler-product

Lean: conditional-interface:def:phase4-weil-closed-point-product

Define the locally finite formal product

$$Z_{X,\mathrm{cl}}(T) := \prod_{x\in \mathrm{ClPt}(X_0)}^{\mathrm{lf}} (1 - T^{\deg(x)})^{-1} \in \mathbf{Z}[[T]].$$

The degree is $[\kappa(x) : \mathbf{F}_q]$.

Proposition 16.1.8 (Euler product equals the exponential zeta series). Uses: def:phase4-weil-zeta-series,def:phase4-weil-closed-point-product

Lean: conditional-interface:prop:phase4-weil-euler-product

One has

$$Z_X(T) = Z_{X,\mathrm{cl}}(T) = \prod_{x\in \mathrm{ClPt}(X_0)}^{\mathrm{lf}} (1 - T^{\deg(x)})^{-1}.$$

Proof. Uses: def:phase4-weil-zeta-series,def:phase4-weil-closed-point-product,prop:weil-final-zeta-euler-product

Apply the preceding closed-orbit Euler-product proposition to the certified point realization in $\mathcal{A}$. Its two series are exactly the two assembly Definitions. □

Corollary 16.1.9 (Integral coefficients of the zeta series). Uses: def:phase4-weil-zeta-series

Lean: conditional-interface:cor:phase4-weil-zeta-integral-coefficients

For every $m \geq 0$, the coefficient $[T^m]Z_X(T)$ lies in $\mathbf{Z} \subset \mathbf{Q}$.

Proof. Uses: prop:phase4-weil-euler-product,cor:weil-final-zeta-integral-coefficients

Instantiate the preceding integral-coefficient corollary for the point interface of $\mathcal{A}$, and rewrite its zeta series by the Euler-product equality. □

16.2 Frobenius factors and the trace formula

Definition 16.2.1 (Assembly cohomology space). Uses: def:phase4-weil-assembly-input

Lean: conditional-interface:def:phase4-weil-cohomology-space

For a realization (lambda) and $0 \le i \le 2d$, define

$$V_{\lambda,i} := H^i(X, E_\lambda), \qquad X = X_0 \times_{\mathbf{F}_q} \overline{\mathbf{F}}_q,$$

using the ordinary cohomology object already constructed in $\mathcal{A}$.

Definition 16.2.2 (Assembly geometric Frobenius). Uses: def:phase4-weil-assembly-input, def:phase4-weil-cohomology-space

Lean: conditional-interface:def:phase4-weil-geometric-frobenius

Define

$$F_{\lambda,i} : V_{\lambda,i} \longrightarrow V_{\lambda,i}$$

to be the geometric-Frobenius automorphism projected from the constructed cohomological Frobenius module.

Lemma 16.2.3 (Finite-dimensionality of assembly cohomology). *Uses: def:phase4-weil-cohomology-space*

Lean: conditional-interface:lemma:phase4-weil-cohomology-finite-dimensional

The E_λ-vector space $V_{\lambda,i}$ is finite-dimensional.

Proof. Uses: def:phase4-weil-cohomology-space, lemma:weil-conditional-cohomology-frobenius-module-carrier

Apply the carrier projection lemma to the degree-i finite-dimensional Frobenius module in the cohomological core of $\mathcal{A}$. □

Lemma 16.2.4 (Invertibility of assembly Frobenius). *Uses: def:phase4-weil-geometric-frobenius*

Lean: conditional-interface:lemma:phase4-weil-geometric-frobenius-invertible

The map $F_{\lambda,i}$ is an automorphism.

Proof. Uses: def:phase4-weil-geometric-frobenius, lemma:weil-conditional-cohomology-frobenius-module-operator

The projected Frobenius module contains an inverse and the two inverse equations. The operator projection lemma identifies its forward map with $F_{\lambda,i}$. □

Definition 16.2.5 (Coefficient-field Frobenius factor). Uses: def:phase4-weil-cohomology-space, def:phase4-weil-geometric-frobenius, def:weil-based-reciprocal-determinant

Lean: conditional-interface:def:phase4-weil-coefficient-factor

Define

$$P_{\lambda,i}(T) := \det(1 - TF_{\lambda,i} \mid V_{\lambda,i}) \in E_\lambda[T].$$

Lemma 16.2.6 (Constant coefficient of a Frobenius factor). *Uses: def:phase4-weil-coefficient-factor*

Lean: conditional-interface:lemma:phase4-weil-factor-constant

For every (lambda,i), $P_{\lambda,i}(0) = 1$.

Proof. Uses: def:phase4-weil-coefficient-factor, lemma:weil-based-reciprocal-determinant-constant

Apply the reciprocal-determinant constant-coefficient lemma to the based endomorphism datum underlying $F_{\lambda,i}$. □

Lemma 16.2.7 (Degree of a Frobenius factor). *Uses: def:phase4-weil-cohomology-space, def:phase4-weil-coefficient-factor*

Lean: conditional-interface:lemma:phase4-weil-factor-degree-over-coefficient-field

One has

$$\deg P_{\lambda,i} = \dim_{E_\lambda} V_{\lambda,i}.$$

Proof. Uses: lemma:phase4-weil-geometric-frobenius-invertible, prop:weil-final-factor-degree-and-betti-independence

The determinant factor has degree at most the dimension. Its coefficient in that degree is $(-1)^r \det(F_{\lambda,i})$, where $r = \dim V_{\lambda,i}$; invertibility makes this coefficient nonzero. This is exactly the degree calculation instantiated in the cited degree–Betti proposition. □

Definition 16.2.8 (Assembly trace datum). Uses: def:phase4-weil-assembly-input, def:weil-finite-cohomological-trace-datum

Lean: conditional-interface:def:phase4-weil-trace-datum

Define $\mathcal{T}_\lambda$ to be the finite cohomological trace datum projected from the recovery input in realization (lambda). Its graded family is $(V_{\lambda,i}, F_{\lambda,i})_{0\le i\le 2d}$, and its point interface is the one used to define $N_n(X_0)$.

Lemma 16.2.9 (Trace formula at one iterate). *Uses:* def:phase4-weil-point-number, def:phase4-weil-trace-datum

Lean: conditional-interface:lemma:phase4-weil-trace-formula-at-iterate

For every $n \ge 1$,

$$N_n(X_0) = \sum_{i=0}^{2d} (-1)^i \operatorname{Tr}(F_{\lambda,i}^n).$$

Proof. Uses: def:phase4-weil-point-number, def:phase4-weil-trace-datum, lemma:weil-finite-cohomological-trace-at-iterate

Instantiate the every-iterate projection lemma for $\mathcal{T}_\lambda$. The point interface and each degree-i based automorphism are the assembly objects by Definition, so the projected equality is the displayed formula. □

Definition 16.2.10 (Alternating Frobenius determinant). Uses: def:phase4-weil-coefficient-factor

Lean: conditional-interface:def:phase4-weil-alternating-determinant

Define the regular formal series

$$\Delta_\lambda(T) := \prod_{i=0}^{2d} P_{\lambda,i}(T)^{(-1)^{i+1}} \in E_\lambda[[T]],$$

where negative exponents denote the formal inverse of a constant-one series.

Lemma 16.2.11 (Counting logarithm equals the alternating trace logarithm). *Uses:* def:phase4-weil-counting-logarithm, def:phase4-weil-trace-datum

Lean: conditional-interface:lemma:phase4-weil-counting-log-equals-trace-log

After coefficient extension from (mathbf Q) to E_λ,

$$L_X(T) = \sum_{n\ge 1} \left(\sum_{i=0}^{2d} (-1)^i \operatorname{Tr}(F_{\lambda,i}^n) \right) \frac{T^n}{n}.$$

Proof. Uses: def:phase4-weil-counting-logarithm, lemma:phase4-weil-trace-formula-at-iterate

The constant coefficients on both sides are zero. In positive degree n, divide the trace-formula equality by the nonzero scalar $n \in E_\lambda$. Coefficient extensionality proves the series identity. □

Proposition 16.2.12 (Cohomological determinant factorization). *Uses:* def:phase4-weil-zeta-series, def:phase4-weil-alternating-determinant

Lean: conditional-interface:prop:phase4-weil-cohomological-factorization

Coefficient extension to $E_\lambda[[T]]$ sends $Z_X(T)$ to

$$\Delta_\lambda(T) = \prod_{i=0}^{2d} P_{\lambda,i}(T)^{(-1)^{i+1}}.$$

Proof. Uses: def:phase4-weil-zeta-series, def:phase4-weil-alternating-determinant, def:phase4-weil-trace-datum, lemma:weil-zeta-eq-alternating-determinant-from-trace-datum

Apply the trace-datum determinant theorem once to $\mathcal{T}_\lambda$. Its counting zeta and alternating determinant are precisely the two assembly Definitions. □

Definition 16.2.13 (Assembly rational zeta function). Uses: def:phase4-weil-assembly-input, def:weil-final-rational-zeta-function

Lean: conditional-interface:def:phase4-weil-rational-zeta

Define $Z_X^{\mathrm{rat}}(T) \in \mathbf{Q}(T)$ to be the rational function constructed from the common integral Frobenius factors in $\mathcal{A}$.

Proposition 16.2.14 (Rationality over the counting field). *Uses:* *def: phase4-weil-zeta-series*, *def: phase4-weil-rational-zeta*

Lean: *conditional-interface: prop: phase4-weil-rationality*

The Taylor expansion at $T = 0$ of Z_X^{rat} is Z_X.

Proof. Uses: def:phase4-weil-rational-zeta, cor:weil-final-zeta-rationality

Instantiate the preceding rationality corollary with $\mathcal{A}$. Its rational function and formal series are the two assembly aliases. □

16.3 Integral factors, degrees, and realization independence

Definition 16.3.1 (Common integral Frobenius factor). Uses: def:phase4-weil-assembly-input, def:weil-final-common-frobenius-factor

Lean: conditional-interface:def:phase4-weil-integral-factor

For $0 \le i \le 2d$, define

$$P_i(T) \in \mathbf{Z}[T]$$

to be the common integral factor already constructed from the compatible realization family in $\mathcal{A}$.

Lemma 16.3.2 (Integral factor realizes every coefficient factor). *Uses:* *def: phase4-weil-integral-factor*, *def: phase4-weil-coefficient-factor*

Lean: *conditional-interface: lemma: phase4-weil-integral-factor-realization*

For every realization (lambda), coefficient extension sends

$$P_i(T) \longmapsto P_{\lambda,i}(T).$$

Proof. Uses: def:phase4-weil-integral-factor, def:phase4-weil-coefficient-factor, lemma:weil-final-common-factor-realization

Apply the common-factor realization lemma at (λ, i), then unfold the two assembly aliases. □

Lemma 16.3.3 (Normalization of the integral factor). *Uses:* *def: phase4-weil-integral-factor*

Lean: *conditional-interface: lemma: phase4-weil-integral-factor-constant*

One has $P_i(0) = 1$.

Proof. Uses: lemma:phase4-weil-integral-factor-realization, lemma:phase4-weil-factor-constant

Map the desired equality to one coefficient field E_λ. The realization lemma identifies the image with $P_{\lambda,i}(0)$, which is (1). Injectivity of $\mathbf{Z} \hookrightarrow E_\lambda$ reflects the equality. □

Proposition 16.3.4 (Integral cohomological factorization). *Uses:* *def: phase4-weil-zeta-series*, *def: phase4-weil-integral-factor*

Lean: *conditional-interface: prop: phase4-weil-integral-factorization*

In $\mathbf{Q}[[T]]$,

$$Z_X(T) = \prod_{i=0}^{2d} P_i(T)^{(-1)^{i+1}}.$$

Proof. Uses: lemma:phase4-weil-integral-factor-realization, prop:phase4-weil-cohomological-factorization

Map both sides coefficientwise to one $E_\lambda[[T]]$. The cohomological factorization gives the image of the left side, and the factor-realization lemma identifies every factor on the right. Injectivity of coefficient extension reflects the formal-series equality. □

Definition 16.3.5 (Common Betti number). Uses: def:phase4-weil-integral-factor

Lean: conditional-interface:def:phase4-weil-betti-number

For $0 \le i \le 2d$, define

$$b_i := \deg P_i.$$

For $i > 2d$, define $b_i := 0$.

Lemma 16.3.6 (Coefficient extension preserves the factor degree). *Uses:* *def: phase4-weil-integral-factor*, *def: phase4-weil-coefficient-factor*

Lean: *conditional-interface: lemma: phase4-weil-factor-degree-preserved*

For every (lambda),

$$\deg P_i = \deg P_{\lambda,i}.$$

Proof. Uses: lemma:phase4-weil-integral-factor-constant,lemma:phase4-weil-integral-factor-realization

The constant coefficient (1) makes P_i nonzero. An injective coefficient map preserves the highest nonzero coefficient, hence preserves the degree. Apply this finite coefficient calculation to $\mathbf{Z} \hookrightarrow E_\lambda$. □

Proposition 16.3.7 (Betti number equals every ell-adic dimension). *Uses: def:phase4-weil-betti-number, def:phase4-weil-cohomology-space*

Lean: conditional-interface:prop:phase4-weil-betti-realization-independence

For every realization (lambda) and $0 \le i \le 2d$,

$$b_i = \dim_{E_\lambda} H^i(X, E_\lambda).$$

Consequently the dimension is independent of ell$_\lambda$.

Proof. Uses: def:phase4-weil-betti-number,lemma:phase4-weil-factor-degree-preserved,lemma:phase4-weil-factor-degree-over-coefficient-field

The three terms in the chain

$$b_i = \deg P_i = \deg P_{\lambda,i} = \dim_{E_\lambda} V_{\lambda,i}$$

are identified by the Definition and the two cited degree lemmas. The first term does not depend on (lambda). □

Lemma 16.3.8 (Poincare symmetry of Betti numbers). *Uses: def:phase4-weil-betti-number*

Lean: conditional-interface:lemma:phase4-weil-betti-symmetry

For $0 \le i \le 2d$, $b_i = b_{2d-i}$.

Proof. Uses: prop:phase4-weil-betti-realization-independence,lemma:weil-final-common-betti-symmetry

Apply the Poincare-duality dimension symmetry in one realization and replace both dimensions by the common Betti numbers. □

Lemma 16.3.9 (Bottom factor). *Uses: def:phase4-weil-integral-factor*

Lean: conditional-interface:lemma:phase4-weil-bottom-factor

One has $P_0(T) = 1 - T$.

Proof. Uses: def:phase4-weil-integral-factor,lemma:weil-final-common-bottom-factor

This is the preceding computation of geometric Frobenius on the one-dimensional space of constant sections, rewritten through the assembly factor alias. □

Lemma 16.3.10 (Top factor). *Uses: def:phase4-weil-integral-factor*

Lean: conditional-interface:lemma:phase4-weil-top-factor

One has $P_{2d}(T) = 1 - q^d T$.

Proof. Uses: def:phase4-weil-integral-factor,lemma:weil-final-common-top-factor

This is the preceding top-trace and Tate-normalization computation, rewritten through the assembly factor alias. □

Lemma 16.3.11 (Endpoint Betti numbers). *Uses: def:phase4-weil-betti-number*

Lean: conditional-interface:lemma:phase4-weil-endpoint-betti

One has $b_0 = b_{2d} = 1$.

Proof. Uses: def:phase4-weil-betti-number,lemma:phase4-weil-bottom-factor,lemma:phase4-weil-top-factor

Both displayed endpoint factors are nonconstant linear polynomials. Their degrees are therefore one, which is the Definition of the two Betti numbers. □

16.4 Poincare duality and the functional equation

Definition 16.4.1 (Assembly Euler characteristic). Uses: def:phase4-weil-betti-number

Lean: conditional-interface:def:phase4-weil-euler-characteristic

Define

$$\chi := \sum_{i=0}^{2d} (-1)^i b_i \in \mathbf{Z}.$$

Definition 16.4.2 (Assembly half exponent). Uses: def:phase4-weil-assembly-input,def:weil-final-half-functional-equation-exponent

Lean: conditional-interface:def:phase4-weil-half-exponent

Define $h \in \mathbf{Z}$ to be the half functional-equation exponent constructed from the compatible duality family in $\mathcal{A}$.

Definition 16.4.3 (Assembly functional-equation sign). Uses: def:phase4-weil-assembly-input,def:weil-final-functional-equation-sign

Lean: conditional-interface:def:phase4-weil-functional-equation-sign

Define $\varepsilon \in \mathbf{Q}^\times$ to be the normalized sign constructed from the alternating Frobenius determinant in $\mathcal{A}$.

Lemma 16.4.4 (Integrality of the half exponent). Uses: def:phase4-weil-euler-characteristic, def:phase4-weil-half-exponent

Lean: conditional-interface:lemma:phase4-weil-half-exponent-equation

One has $2h = d\chi$. In particular $d\chi$ is even.

Proof. Uses: def:phase4-weil-euler-characteristic,def:phase4-weil-half-exponent,lemma:weil-final-functional-equation-exponent

Apply the preceding parity and half-exponent computation and rewrite its Euler characteristic and exponent by the assembly aliases. □

Lemma 16.4.5 (The functional-equation scalar is a sign). Uses: def:phase4-weil-functional-equation-sign

Lean: conditional-interface:lemma:phase4-weil-sign-dichotomy

One has $\varepsilon = 1$ or $\varepsilon = -1$.

Proof. Uses: def:phase4-weil-functional-equation-sign,lemma:weil-final-functional-equation-sign-dichotomy

Apply the preceding sign-dichotomy lemma and unfold the assembly alias. □

Proposition 16.4.6 (Functional equation). Uses: def:phase4-weil-rational-zeta, def:phase4-weil-euler-characteristic, def:phase4-weil-half-exponent, def:phase4-weil-functional-equation-sign

Lean: conditional-interface:prop:phase4-weil-functional-equation

In $\mathbf{Q}(T)$,

$$Z_X^{\mathrm{rat}}(q^{-d}T^{-1}) = \varepsilon q^h T^\chi Z_X^{\mathrm{rat}}(T), \qquad \varepsilon \in \{1, -1\}, \qquad 2h = d\chi.$$

Proof. Uses: def:phase4-weil-rational-zeta,lemma:phase4-weil-half-exponent-equation,lemma:phase4-weil-sign-dichotomy,prop:weil-final-zeta-functional-equation

Instantiate the preceding normalized functional equation with $\mathcal{A}$. Rewrite its rational zeta function, Euler characteristic, exponent, and sign by the four assembly Definitions. The last two assertions are the two preceding lemmas. □

16.5 Purity, reciprocal roots, and the Riemann hypothesis

Definition 16.5.1 (Assembly reciprocal-root datum). Uses: def:phase4-weil-integral-factor

Lean: conditional-interface:def:phase4-weil-reciprocal-root-data

For $0 \le i \le 2d$, a reciprocal-root datum is a nonzero $\alpha \in \overline{\mathbf{Q}}$ together with

$$P_i(\alpha^{-1}) = 0.$$

Definition 16.5.2 (Assembly ordinary-root datum). Uses: def:phase4-weil-integral-factor

Lean: conditional-interface:def:phase4-weil-ordinary-root-data

For $0 \le i \le 2d$, an ordinary-root datum is $\beta \in \overline{\mathbf{Q}}$ together with

$$P_i(\beta) = 0.$$

Lemma 16.5.3 (An ordinary root is nonzero). Uses: def:phase4-weil-ordinary-root-data

Lean: conditional-interface:lemma:phase4-weil-ordinary-root-nonzero

Every ordinary root β is nonzero.

Proof. Uses: def:phase4-weil-ordinary-root-data,lemma:phase4-weil-integral-factor-constant

If $\beta = 0$, the root equation gives $P_i(0) = 0$, whereas factor normalization gives $P_i(0) = 1$. □

Definition 16.5.4 (Reciprocal scalar of an ordinary root). Uses: def:phase4-weil-ordinary-root-data

Lean: conditional-interface:def:phase4-weil-ordinary-root-reciprocal

Define

$$\alpha_\beta := \beta^{-1}.$$

Lemma 16.5.5 (The reciprocal scalar is a reciprocal root). Uses: def:phase4-weil-ordinary-root-data, def:phase4-weil-ordinary-root-reciprocal, def:phase4-weil-reciprocal-root-data

Lean: conditional-interface:lemma:phase4-weil-ordinary-root-gives-reciprocal-root

The scalar α_β, with the nonzero proof from the preceding lemma, defines a reciprocal-root datum.

Proof. Uses: def:phase4-weil-ordinary-root-data,def:phase4-weil-ordinary-root-reciprocal,lemma:phase4-weil-ordinary-root-nonzero

Nonvanishing gives $(\beta^{-1})^{-1} = \beta$. Substitute this equality in the stored ordinary-root equation. □

Lemma 16.5.6 (Reciprocal roots are algebraic integers). Uses: def:phase4-weil-reciprocal-root-data

Lean: conditional-interface:lemma:phase4-weil-reciprocal-root-integral

Every reciprocal root α is an algebraic integer.

Proof. Uses: def:phase4-weil-reciprocal-root-data,lemma:weil-final-reciprocal-root-algebraic-integer

Apply the preceding integral-factor root theorem to the common factor P_i and its stored reciprocal-root equation. □

Lemma 16.5.7 (Reciprocal roots occur as Frobenius eigenvalues). Uses: def:phase4-weil-reciprocal-root-data, def:phase4-weil-cohomology-space, def:phase4-weil-geometric-frobenius

Lean: conditional-interface:lemma:phase4-weil-reciprocal-root-eigenvalue

After extension to an algebraic closure of one coefficient field, the α-eigenspace of $F_{\lambda,i}$ is nonzero.

Proof. Uses: lemma:phase4-weil-integral-factor-realization,thm:weil-root-reciprocal-conversion-after-scalar-extension

Transport $P_i(\alpha^{-1}) = 0$ to the coefficient field by the factor realization lemma. Apply the scalar-extension root–eigenvalue theorem once; its conclusion is the asserted nonzero eigenspace at $(\alpha^{-1})^{-1} = \alpha$. □

Lemma 16.5.8 (Weight of a reciprocal root). Uses: def:phase4-weil-reciprocal-root-data

Lean: conditional-interface:lemma:phase4-weil-reciprocal-root-modulus

For every embedding $\sigma : \mathbf{Q}(\alpha) \hookrightarrow \mathbf{C}$,

$$|\sigma(\alpha)| = q^{i/2}.$$

Proof. Uses: lemma:phase4-weil-reciprocal-root-eigenvalue,lemma:weil-final-reciprocal-root-classical-weight

The preceding lemma realizes α as a degree-i geometric-Frobenius eigenvalue. Apply the already proved smooth-proper universal-purity theorem, in its classical all-embeddings form. □

Lemma 16.5.9 (Modulus of an ordinary root). Uses: def:phase4-weil-ordinary-root-data

Lean: conditional-interface:lemma:phase4-weil-ordinary-root-modulus

For every embedding $\sigma : \mathbf{Q}(\beta) \hookrightarrow \mathbf{C}$,

$$|\sigma(\beta)| = q^{-i/2}.$$

Proof. Uses: def:phase4-weil-ordinary-root-reciprocal,lemma:phase4-weil-ordinary-root-nonzero,lemma:phase4-weil-ordinary-root-gives-reciprocal-root,lemma:phase4-weil-reciprocal-root-modulus
The reciprocal-root lemma gives $|\sigma(\beta^{-1})| = q^{i/2}$. Since $\beta \neq 0$, the left side is $|\sigma(\beta)|^{-1}$. Taking positive real inverses gives the formula. □

Proposition 16.5.10 (Riemann hypothesis for the assembly factors). *Uses: def:phase4-weil-reciprocal-root-data, def:phase4-weil-ordinary-root-data*

Lean: conditional-interface:prop:phase4-weil-riemann-hypothesis

Every reciprocal root of P_i is an algebraic integer, occurs as a geometric-Frobenius eigenvalue, and has modulus $q^{i/2}$ under every complex embedding. Every ordinary root has modulus $q^{-i/2}$.

Proof. Uses: lemma:phase4-weil-reciprocal-root-integral,lemma:phase4-weil-reciprocal-root-eigenvalue,lemma:phase4-weil-reciprocal-root-modulus,lemma:phase4-weil-ordinary-root-modulus
The four assertions are exactly the four cited lemmas, each instantiated in degree i. □

16.6 Optional singular comparison and terminal theorem

Definition 16.6.1 (Assembly lift enhancement). Uses: def:phase4-weil-assembly-input,def:weil-final-lift-enhancement-data

Lean: conditional-interface:def:phase4-weil-lift-enhancement

A lift enhancement is a supplied characteristic-zero lift comparison datum for the same special fiber and compatible realization family.

Theorem 16.6.2 (Singular Betti interpretation for a supplied lift). *Uses: def:phase4-weil-lift-enhancement, def:phase4-weil-betti-number*

Lean: conditional-interface:thm:phase4-weil-singular-betti-comparison

For every supplied lift enhancement,

$$b_i = \dim_{\mathbf{Q}} H^i_{\mathrm{sing}}(X_{\mathbf{C}}(\mathbf{C}), \mathbf{Q}).$$

Proof. Uses: def:phase4-weil-lift-enhancement,thm:weil-final-singular-betti-interpretation
Apply the preceding comparison theorem to the supplied enhancement and rewrite its common Betti number by the assembly Definition. □

Theorem 16.6.3 (Weil conjectures from the proved six-functor and weight interfaces). *Uses: def:phase4-weil-assembly-input, def:phase4-weil-point-number, def:phase4-weil-zeta-series, def:phase4-weil-closed-point-product, def:phase4-weil-integral-factor, def:phase4-weil-betti-number, def:phase4-weil-rational-zeta, def:phase4-weil-euler-characteristic, def:phase4-weil-half-exponent, def:phase4-weil-functional-equation-sign, def:phase4-weil-lift-enhancement*

Lean: conditional-interface:thm:phase4-weil-conjectures-from-six-functors-and-weights

For every terminal assembly input attached to a smooth, projective, geometrically connected $X_0/\mathbf{F}_q$ of pure dimension d:

i

$$Z_X(T) = \exp\left(\sum_{n\geq 1} \#X_0(\mathbf{F}_{q^n})\frac{T^n}{n}\right) = \prod_{x\in \mathrm{ClPt}(X_0)}^{\mathrm{lf}} (1 - T^{\deg x})^{-1}.$$

ii There are explicitly defined $P_i \in \mathbf{Z}[T]$, $0 \leq i \leq 2d$, with $P_i(0) = 1$, such that

$$Z_X(T) = \prod_{i=0}^{2d} P_i(T)^{(-1)^{i+1}},$$

and Z_X is the Taylor expansion of $Z_X^{\mathrm{rat}} \in \mathbf{Q}(T)$.

iii The image of P_i in every $E_\lambda[T]$ is $\det(1 - TF_{\lambda,i} \mid H^i(X, E_\lambda))$, and

$$\deg P_i = b_i = \dim_{E_\lambda} H^i(X, E_\lambda).$$

Thus both factors and Betti dimensions are independent of ell$_\lambda$, and $b_i = b_{2d-i}$.

iv

$$P_0 = 1 - T, \quad P_{2d} = 1 - q^d T, \quad b_0 = b_{2d} = 1.$$

v *Every reciprocal root of P_i is an algebraic integer and has absolute value $q^{i/2}$ under every complex embedding; every ordinary root has absolute value $q^{-i/2}$.*

vi
$$Z_X^{\mathrm{rat}}(q^{-d}T^{-1}) = \varepsilon q^{h} T^{\chi} Z_X^{\mathrm{rat}}(T), \qquad \varepsilon \in \{1,-1\}, \qquad 2h = d\chi.$$

vii *For every supplied characteristic-zero lift enhancement, b_i equals the corresponding singular Betti number.*

Proof. Uses: `lemma:phase4-weil-zeta-exponential`, `prop:phase4-weil-euler-product`, `cor:phase4-weil-zeta-integral-coefficients`, `lemma:phase4-weil-integral-factor-constant`, `prop:phase4-weil-integral-factorization`, `prop:phase4-weil-rationality`, `lemma:phase4-weil-integral-factor-realization`, `prop:phase4-weil-betti-realization-independence`, `lemma:phase4-weil-betti-symmetry`, `lemma:phase4-weil-bottom-factor`, `lemma:phase4-weil-top-factor`, `lemma:phase4-weil-endpoint-betti`, `prop:phase4-weil-riemann-hypothesis`, `prop:phase4-weil-functional-equation`, `thm:phase4-weil-singular-betti-comparison`

Clause *i* is the exponential lemma followed by the Euler-product proposition. Clause (ii) is factor normalization, integral factorization, and rationality. Clause (iii) is factor realization, the degree–dimension proposition, and Betti symmetry. Clause (iv) is the two endpoint-factor lemmas and the endpoint-degree lemma. Clause (v) is the Riemann-hypothesis proposition. Clause (vi) is the normalized functional equation. Clause (vii) is the singular comparison theorem instantiated with the supplied enhancement. Each clause is therefore a direct combination of declarations that occur earlier in this chapter. □

16.7 Typed construction of scheme Frobenius

16.7.1 Affine construction and localization coherence

Definition 16.7.1 (Affine absolute q-power scheme morphism). Lean: `conditional-interface:def:weil-absolute-q-power-affine-morphism`

Uses: `def:weil-affine-q-power-ring-endomorphism`

For an $\mathbf{F}_q$-algebra A, define the affine scheme endomorphism
$$\mathrm{Fr}^{\mathrm{abs}}_{\mathrm{Spec}\,A,q} := \mathrm{Spec}(\varphi_{A,q}) : \mathrm{Spec}\,A \longrightarrow \mathrm{Spec}\,A.$$

Thus the ring homomorphism is already typed before the contravariant Spec construction is invoked.

Definition 16.7.2 (Principal-localized q-power endomorphism). Lean: `conditional-interface:def:weil-affine-q-power-principal-localization`

Uses: `def:weil-affine-q-power-ring-endomorphism`

For $f \in A$, define the bundled localization endomorphism
$$\varphi_{A,q,f} : A_f \longrightarrow + * A_f, \qquad \frac{a}{f^m} \longmapsto \frac{a^q}{f^{qm}}.$$

It is the localization lift of $\varphi_{A,q}$; the image of f is the unit f^q in A_f.

Lemma 16.7.3 (Principal-open inverse image). Lean: `blueprint-target:lemma:weil-affine-frobenius-principal-preimage`

Uses: `def:weil-absolute-q-power-affine-morphism`

For $f \in A$, the inverse image of $D(f)$ under $\mathrm{Fr}^{\mathrm{abs}}_{\mathrm{Spec}\,A,q}$ is $D(f)$.

Proof. Uses: `def:weil-affine-q-power-ring-endomorphism`

For a prime $\mathfrak{p}$, one has $f^q \in \mathfrak{p}$ exactly when $f \in \mathfrak{p}$, because prime ideals are radical. Hence $D(f^q) = D(f)$. □

Lemma 16.7.4 (Principal-open restriction formula). Lean: `blueprint-target:lemma:weil-affine-frobenius-principal-restriction`

Uses: `def:weil-affine-q-power-principal-localization`, `def:weil-absolute-q-power-affine-morphism`

Under the standard identification $D(f) = \mathrm{Spec}(A_f)$, the restriction of the affine absolute Frobenius is induced by $\varphi_{A,q,f}$.

Proof. Uses: `lemma:weil-affine-frobenius-principal-preimage`

The universal property of A_f gives a unique lift of $\varphi_{A,q}$ whose value on a/f^m is a^q/f^{qm}. This is precisely the ring map defining the restricted affine morphism. □

Lemma 16.7.5 (Pairwise localization comparison). *Lean:* `blueprint-target: lemma: weil-affine-frobenius-pairwise-localization`

Uses: `def: weil-affine-q-power-principal-localization`

For $f, g \in A$, let $\lambda_f : A_f \to A_{fg}$ and $\lambda_g : A_g \to A_{fg}$ be the canonical localization maps. Then

$$\lambda_f \circ \varphi_{A,q,f} = \varphi_{A,q,fg} \circ \lambda_f, \qquad \lambda_g \circ \varphi_{A,q,g} = \varphi_{A,q,fg} \circ \lambda_g.$$

Proof. Uses: `lemma:weil-affine-frobenius-principal-restriction`

Each side of the first equality sends

$$\frac{a}{f^m} \longmapsto \frac{a^q}{f^{qm}} \quad \text{in } A_{fg},$$

and the analogous calculation gives the second equality. Uniqueness in the localization universal property proves both commuting squares; no literal identification of an iterated localization with A_{fg} is used. □

Lemma 16.7.6 (Triple localization cocycle). *Lean:* `blueprint-target: lemma: weil-affine-frobenius-triple-localization`

Uses: `def: weil-affine-q-power-principal-localization`

For $f, g, h \in A$, each route of canonical localization maps from a principal localization into A_{fgh} intertwines its q-power endomorphism with $\varphi_{A,q,fgh}$. The resulting pairwise comparisons obey the triple cocycle in the common target A_{fgh}.

Proof. Uses: `lemma:weil-affine-frobenius-pairwise-localization`

After passage along the canonical maps to A_{fgh}, each route is the unique localization of $\varphi_{A,q}$. It sends the image of $a/(f^m g^n h^r)$ to $a^q/(f^{qm} g^{qn} h^{qr})$. Uniqueness identifies the routes and gives the cocycle without treating the intermediate localization rings as definitionally equal. □

Lemma 16.7.7 (Affine structure-sheaf formula). *Lean:* `blueprint-target: lemma: weil-affine-frobenius-structure-sheaf-formula`

Uses: `def: weil-absolute-q-power-affine-morphism`

On every open $W \subseteq \operatorname{Spec} A$, the structure-sheaf map of the affine absolute Frobenius sends a section s to s^q.

Proof. Uses: `lemma:weil-affine-frobenius-principal-restriction,lemma:weil-affine-frobenius-pairwise-localization`

Principal opens form a basis. On $D(f)$ the assertion is the displayed localization formula. Those formulas commute with further restriction, so the sheaf equality follows by checking on a principal-open cover of W. □

Definition 16.7.8 (Affine localization-overlap certificate). Lean: `conditional-interface:def:weil-affine-frobenius-overlap-certificate`

Uses: `def:weil-affine-q-power-principal-localization,def:weil-absolute-q-power-affine-morphism`

Constructed by: `lemma:weil-affine-frobenius-principal-preimage,lemma:weil-affine-frobenius-principal-restriction,lemma:weil-affine-frobenius-pairwise-localization,lemma:weil-affine-frobenius-triple-localization`

Define the localization-overlap certificate to consist of the selected principal restrictions together with their pairwise comparison and triple cocycle fields. Each field is the separately labeled certificate above.

16.7.2 Global gluing and the descended model

Definition 16.7.9 (Affine absolute-Frobenius gluing datum). Lean: `conditional-interface:def:weil-absolute-q-power-affine-gluing-datum`

Uses: `def:weil-absolute-q-power-affine-morphism,def:weil-affine-frobenius-overlap-certificate`

For an $\mathbf{F}_q$-scheme Z, define the gluing datum indexed by affine opens $U = \operatorname{Spec} A \subseteq Z$. Its component on U is $\mathrm{Fr}^{\mathrm{abs}}_{U,q}$, and its restriction fields are the localization-overlap certificates on principal-open refinements.

Lemma 16.7.10 (Arbitrary affine-overlap refinement). *Lean:* `blueprint-target: lemma: weil-absolute-q-power-arbitrary-affine-overlap`

Uses: `def: weil-absolute-q-power-affine-gluing-datum`

For affine opens $U, V \subseteq Z$, the two components of the gluing datum restrict to the same locally ringed-space morphism on $U \cap V$.

Proof. Uses: lemma:weil-affine-frobenius-principal-preimage,lemma:weil-affine-frobenius-structure-sheaf-formula,lemma:weil-affine-frobenius-pairwise-localization,lemma:weil-affine-frobenius-triple-localization

Fix $x \in U \cap V$. Choose a principal neighborhood $D_U(f)$ of x inside $U \cap V$, then cover it by principal opens of V. As x varies, these opens cover $U \cap V$. The principal-preimage lemma fixes the inverse image of each basic open; since the basis separates points, both underlying maps are the identity. On each member of the refinement, both structure-sheaf maps are $s \mapsto s^q$. The pairwise localization comparison identifies further refinements, and the triple equation makes these identifications coherent. Equality is local on the source, so the two restrictions agree on $U \cap V$. □

Definition 16.7.11 (Global structure-sheaf q-power map). Lean: conditional-interface:def:weil-global-structure-sheaf-q-power-map

Uses: def:weil-prime-power-data,def:weil-finite-field-data

For an $\mathbf{F}_q$-scheme Z, define an endomorphism of its structure sheaf by

$$\mathcal{O}_Z(W) \longrightarrow \mathcal{O}_Z(W), \qquad s \longmapsto s^q$$

on every open W. The restriction squares commute because restriction maps are ring homomorphisms.

Lemma 16.7.12 (Locality on stalks). Lean: blueprint-target:lemma:weil-global-q-power-stalk-locality

Uses: def:weil-global-structure-sheaf-q-power-map

At each $z \in Z$, the induced endomorphism of the local ring $\mathcal{O}_{Z,z}$ is local.

Proof. Uses: def:weil-affine-q-power-ring-endomorphism

In a local ring with maximal ideal $\mathfrak{m}$, an element u lies in $\mathfrak{m}$ exactly when u^q does: the forward implication is closure of $\mathfrak{m}$ under multiplication, and the reverse implication uses that a unit has a unit q-th power. Thus the inverse image of $\mathfrak{m}$ is $\mathfrak{m}$. □

Definition 16.7.13 (Selected global absolute q-power morphism). Lean: conditional-interface:def:weil-absolute-q-power-scheme-morphism

Uses: def:weil-global-structure-sheaf-q-power-map,def:weil-absolute-q-power-affine-gluing-datum

Constructed by: lemma:weil-global-q-power-stalk-locality,lemma:weil-absolute-q-power-arbitrary-affine-overlap

Define directly

$$\mathrm{Fr}^{\mathrm{abs}}_{Z,q} : Z \longrightarrow Z$$

to have the identity continuous map and the selected structure-sheaf q-power map. Stalk locality makes this a scheme morphism. Its restrictions are the prescribed affine components, so this direct locally ringed-space morphism realizes the gluing datum independently of the affine cover.

Proposition 16.7.14 (Affine specification of the global morphism). Lean: blueprint-target:prop:weil-absolute-q-power-scheme-morphism

Uses: def:weil-absolute-q-power-scheme-morphism

On an affine open $U = \operatorname{Spec} A$, the restriction of $\mathrm{Fr}^{\mathrm{abs}}_{Z,q}$ is $\operatorname{Spec}(\varphi_{A,q})$. This specification is unchanged after replacing an affine cover by a refinement.

Proof. Uses: lemma:weil-affine-frobenius-structure-sheaf-formula,lemma:weil-absolute-q-power-arbitrary-affine-overlap

The selected global morphism and the affine morphism have the identity map on U and the same sheaf map $s \mapsto s^q$. Hence they agree as locally ringed-space morphisms. The arbitrary-overlap lemma gives the same equality on any common refinement, proving cover independence. □

Definition 16.7.15 (Underlying descended q-Frobenius). Lean: conditional-interface:def:weil-descended-q-frobenius-underlying

Uses: def:weil-descended-scheme-data,def:weil-absolute-q-power-scheme-morphism

For the descended scheme X_0, define the underlying endomorphism

$$F^{\mathrm{sch}}_{X_0} := \mathrm{Fr}^{\mathrm{abs}}_{X_0,q} : X_0 \longrightarrow X_0.$$

Lemma 16.7.16 ($\mathbf{F}_q$-structural compatibility). Lean: blueprint-target:lemma:weil-descended-q-frobenius-structural-square

Uses: def:weil-descended-q-frobenius-underlying

The underlying descended endomorphism commutes with the structural map $X_0 \to \operatorname{Spec} \mathbf{F}_q$.

Proof. Uses: def:weil-affine-q-power-ring-endomorphism,prop:weil-absolute-q-power-scheme-morphism

On each affine chart, the structural ring homomorphism is followed by $c \mapsto c^q = c$ on $\mathbf{F}_q$. The two composites therefore agree on an affine cover and hence agree globally. □

Definition 16.7.17 ($\mathbf{F}_q$-linear descended Frobenius)**.** Lean: conditional-interface:def:weil-descended-q-frobenius-over-fq

Uses: def:weil-descended-q-frobenius-underlying,def:weil-descended-scheme-data

Constructed by: lemma:weil-descended-q-frobenius-structural-square

Define the $\mathbf{F}_q$-scheme endomorphism of X_0 whose underlying scheme map is $F_{X_0}^{\mathrm{sch}}$ and whose structural commutativity field is the preceding certificate.

Definition 16.7.18 (Frobenius on the canonical pullback)**.** Lean: conditional-interface:def:weil-canonical-pullback-descended-frobenius

Uses: def:weil-descended-q-frobenius-over-fq,def:weil-descended-canonical-pullback-model

Define

$$F_{P_X} := F_{X_0}^{\mathrm{sch}} \times \mathrm{id}_{\operatorname{Spec} k} : P_X \longrightarrow P_X$$

by the pullback universal property. Its first projection is followed by $F_{X_0}^{\mathrm{sch}}$ and its second projection is unchanged; explicitly,

$$\mathrm{pr}_1 \circ F_{P_X} = F_{X_0}^{\mathrm{sch}} \circ \mathrm{pr}_1, \qquad \mathrm{pr}_2 \circ F_{P_X} = \mathrm{pr}_2 .$$

Definition 16.7.19 (Selected Frobenius candidate on the chosen X)**.** Lean: conditional-interface:def:weil-basechanged-descended-scheme-frobenius

Uses: def:weil-scheme-frobenius-candidate,def:weil-descended-scheme-pullback-comparison,def:weil-canonical-pullback-descended-frobenius

Define the scheme-Frobenius candidate on the supplied scheme X by transport across the selected comparison:

$$F_X^{\mathrm{sch}} := \chi_X^{-1} \circ F_{P_X} \circ \chi_X : X \longrightarrow X.$$

This is a map on the chosen object X, not a definitional identification of X with a literal fiber product.

Lemma 16.7.20 (Transport and projection coherence)**.** *Lean:* blueprint-target:lemma:weil-basechanged-descended-frobenius-coherence

Uses: def:weil-basechanged-descended-scheme-frobenius

The comparison intertwines the two endomorphisms, $\chi_X \circ F_X^{\mathrm{sch}} = F_{P_X} \circ \chi_X$, and the supplied projection satisfies $\pi \circ F_X^{\mathrm{sch}} = F_{X_0}^{\mathrm{sch}} \circ \pi$. If $s_X : X \to \operatorname{Spec} k$ is the second structural map in the supplied cartesian square, then $s_X \circ F_X^{\mathrm{sch}} = s_X$.

Proof. Uses: def:weil-descended-scheme-pullback-comparison,def:weil-canonical-pullback-descended-frobenius

The first equality follows by cancelling $\chi_X \circ \chi_X^{-1}$ in the definition. Compose it with the first pullback projection and use the corresponding projection equations for F_{P_X} and χ_X; this gives the equality with π. Composition with the second projection gives the equality with s_X, because F_{P_X} uses the identity on the k-factor. No Frobenius square is asserted to be cartesian. □

16.7.3 Affine formula, uniqueness, and functoriality

Lemma 16.7.21 (Pure-tensor formula)**.** *Lean:* blueprint-target:lemma:weil-descended-affine-frobenius-pure-tensor

Uses: def:weil-descended-affine-frobenius-tensor-map

For $a \in A_0$ and $c \in k$,

$$\psi_{A_0,k}(a \otimes c) = a^q \otimes c.$$

Proof. Uses: def:weil-affine-q-power-ring-endomorphism

This is the pure-tensor computation rule for the tensor-product map. Its first factor is $a \mapsto a^q$ and its second factor is the identity of k. □

Lemma 16.7.22 (Canonical pullback affine stability)**.** *Lean:* blueprint-target:lemma:weil-canonical-pullback-frobenius-affine-stable

Uses: def:weil-canonical-pullback-descended-frobenius, def:weil-descended-canonical-pullback-model

For $U_0 = \operatorname{Spec} A_0 \subseteq X_0$, put $P_U = \mathrm{pr}_1^{-1}(U_0)$. Then $F_{P_X}^{-1}(P_U) = P_U$, so the restriction $F_{P_U} : P_U \to P_U$ is typed.

Proof. Uses: prop:weil-absolute-q-power-scheme-morphism

The first projection equation gives $\mathrm{pr}_1 \circ F_{P_X} = F^{\mathrm{sch}}_{X_0} \circ \mathrm{pr}_1$. The underlying map of $F^{\mathrm{sch}}_{X_0}$ is the identity, so inverse image of U_0 under the two sides gives the asserted equality. □

Definition 16.7.23 (Selected canonical-pullback affine restriction)**.** Lean: conditional-interface:def:weil-canonical-pullback-frobenius-affine-restriction

Uses: def:weil-canonical-pullback-descended-frobenius, def:weil-descended-canonical-pullback-model

Constructed by: lemma:weil-canonical-pullback-frobenius-affine-stable

For $P_U = \mathrm{pr}_1^{-1}(U_0)$, define $F_{P_U} : P_U \to P_U$ to be the restriction selected by the preceding stability equality. Its square with the open immersion into P_X is part of the selected restriction datum.

Lemma 16.7.24 (Canonical pullback affine ring map)**.** *Lean: blueprint-target:lemma:weil-canonical-pullback-frobenius-affine-ring-map*

Uses: def:weil-canonical-pullback-frobenius-affine-restriction, def:weil-descended-affine-frobenius-tensor-map

Under the canonical identification $P_U \simeq \mathrm{Spec}(A_0 \otimes_{\mathbf{F}_q} k)$, the pullback of F_{P_U} is exactly $\psi_{A_0,k}$.

Proof. Uses: lemma:weil-canonical-pullback-frobenius-affine-stable, lemma:weil-descended-affine-frobenius-pure-tensor, prop:weil-absolute-q-power-scheme-morphism

The two projection equations for F_{P_X} say that its affine pullback is the tensor-product map of $a \mapsto a^q$ with id_k. The tensor universal property gives the corresponding algebra homomorphism, and its pure-tensor rule is $a \otimes c \mapsto a^q \otimes c$. Tensor-product induction, or algebra-homomorphism extensionality on pure tensors, identifies it with $\psi_{A_0,k}$. □

Lemma 16.7.25 (Stability of descended affine inverse images)**.** *Lean: blueprint-target:lemma:weil-basechanged-frobenius-descended-affine-stable*

Uses: def:weil-basechanged-descended-scheme-frobenius, def:weil-descended-affine-pullback-coordinate, def:weil-descended-affine-coordinate-ring-equivalence

For a descended affine $U_0 \subseteq X_0$ and $U = \pi^{-1}(U_0)$, one has $(F^{\mathrm{sch}}_X)^{-1}(U) = U$. Hence the restriction $F^{\mathrm{sch}}_X|_U : U \to U$ is a typed scheme endomorphism.

Proof. Uses: lemma:weil-basechanged-descended-frobenius-coherence, prop:weil-absolute-q-power-scheme-morphism

Projection coherence gives $\pi \circ F^{\mathrm{sch}}_X = F^{\mathrm{sch}}_{X_0} \circ \pi$. The underlying continuous map of $F^{\mathrm{sch}}_{X_0}$ is the identity by the global absolute-Frobenius specification. Taking inverse images of U_0 in the displayed equality gives the asserted equality of opens. □

Definition 16.7.26 (Selected descended-affine restriction on X)**.** Lean: conditional-interface:def:weil-basechanged-frobenius-descended-affine-restriction

Uses: def:weil-basechanged-descended-scheme-frobenius, def:weil-descended-affine-pullback-coordinate

Constructed by: lemma:weil-basechanged-frobenius-descended-affine-stable

For $U = \pi^{-1}(U_0)$, define $F^{\mathrm{sch}}_U : U \to U$ to be the restriction selected by the stability equality. Its defining square is $j_U \circ F^{\mathrm{sch}}_U = F^{\mathrm{sch}}_X \circ j_U$.

Lemma 16.7.27 (Transported affine pullback formula)**.** *Lean: blueprint-target:lemma:weil-basechanged-frobenius-affine-coordinate-formula*

Uses: def:weil-basechanged-frobenius-descended-affine-restriction, def:weil-descended-affine-coordinate-ring-equivalence, def:weil-descended-affine-frobenius-tensor-map

In the selected section coordinate Θ_{U_0}, the pullback homomorphism of F^{sch}_U satisfies the typed conjugation equation

$$\Theta_{U_0} \circ (F^{\mathrm{sch}}_U)^{\#} \circ \Theta_{U_0}^{-1} = \psi_{A_0,k}.$$

Proof. Uses: lemma:weil-basechanged-frobenius-descended-affine-stable, lemma:weil-basechanged-descended-frobenius-coherence, lemma:weil-canonical-pullback-frobenius-affine-ring-map

The comparison equation for χ_X restricts to the selected isomorphism $\theta_{U_0} : U \simeq P_U$. Conjugating the canonical affine ring-map identity by the induced global-section equivalence Θ_{U_0} gives the displayed formula on the chosen inverse image U. □

Proposition 16.7.28 (Descended affine-formula certificate). *Lean:* `blueprint-target:prop:weil-descended-scheme-frobenius-affine-formula`

Uses: `def:weil-basechanged-descended-scheme-frobenius`, `def:weil-scheme-frobenius-affine-formula`

The selected candidate F_X^{sch} *satisfies* IsDescendedAffineFrobenius.

Proof. Uses: `lemma:weil-basechanged-frobenius-descended-affine-stable`, `lemma:weil-basechanged-frobenius-affine-coordinate-formula`

For each descended affine, the stability lemma selects the typed restriction F_U and its square with the inclusion j_U. The coordinate formula is the required conjugacy field. These data fill every field of IsTypedDescendedAffineFrobenius, which is definitionally the legacy affine-formula predicate. □

Lemma 16.7.29 (Affine scheme-morphism extensionality in the chosen coordinate). *Lean:* `blueprint-target:lemma:weil-descended-affine-coordinate-morphism-ext`

Uses: `def:weil-descended-affine-coordinate-ring-equivalence`

Let $g, h : U \to U$ *be scheme morphisms. If*

$$\Theta_{U_0} \circ g^{\#} \circ \Theta_{U_0}^{-1} = \Theta_{U_0} \circ h^{\#} \circ \Theta_{U_0}^{-1},$$

then $g = h$.

Proof. Uses: `def:weil-descended-affine-pullback-coordinate`

Cancel the algebra equivalence Θ_{U_0} to obtain $g^{\#} = h^{\#}$. Under θ_{U_0}, both are morphisms between the same affine scheme. Full faithfulness of contravariant Spec gives $g = h$. □

Lemma 16.7.30 (Descended affine-cover morphism extensionality). *Lean:* `blueprint-target:lemma:weil-descended-affine-cover-morphism-ext`

Uses: `def:weil-descended-scheme-data`, `def:weil-typed-descended-affine-frobenius`

Suppose candidates $G, H : X \to X$ *carry typed descended affine Frobenius certificates. If their selected restrictions agree on each inverse image of a descended affine open, then* $G = H$.

Proof. Uses: `def:weil-descended-affine-pullback-coordinate`

Descended affines cover X_0, so their inverse images cover X. The restriction squares in the typed certificates identify the restrictions with the global maps after composing with the open immersions. Equality on the cover therefore implies equality of the two scheme morphisms. □

Proposition 16.7.31 (Uniqueness from the descended affine formula). *Lean:* `blueprint-target:prop:weil-descended-scheme-frobenius-unique`

Uses: `def:weil-descended-scheme-data`, `def:weil-scheme-frobenius-candidate`, `def:weil-scheme-frobenius-affine-formula`

If two scheme-Frobenius candidates $G, H : X \to X$ *satisfy* IsDescendedAffineFrobenius, *then* $G = H$.

Proof. Uses: `prop:weil-descended-scheme-frobenius-affine-formula`, `lemma:weil-descended-affine-coordinate-morphism-ext`, `lemma:weil-descended-affine-cover-morphism-ext`

Let $F = F_X^{\mathrm{sch}}$ be the selected candidate. The cited proposition gives its typed descended affine record. The records for G and F provide typed restrictions to the same U; both conjugated pullback maps equal $\psi_{A_0,k}$. Chosen-coordinate affine extensionality makes those restrictions equal, and cover extensionality gives $G = F$. The same argument gives $H = F$, whence $G = H$. The statement proves only uniqueness; the selected Definition supplies existence. □

Definition 16.7.32 (Canonical base change of a descended scheme map). Lean: `conditional-interface:def:weil-canonical-basechange-descended-scheme-map`

Uses: `def:weil-descended-scheme-data`, `def:weil-descended-canonical-pullback-model`

For descended data X_0, X and Y_0, Y and an $\mathbf{F}_q$-scheme morphism $f_0 : X_0 \to Y_0$, define

$$f_P := f_0 \times \mathrm{id}_{\mathrm{Spec}\, k} : P_X \longrightarrow P_Y$$

by the pullback universal property.

Definition 16.7.33 (Transported descended scheme map)**.** Lean: `conditional-interface:def:weil-transported-descended-scheme-map`

Uses: `def:weil-canonical-basechange-descended-scheme-map`, `def:weil-descended-scheme-pullback-comparison`

Define the map between the chosen models by

$$f := \chi_Y^{-1} \circ f_P \circ \chi_X : X \longrightarrow Y.$$

This is the meaning of the notation $f_0 \times \mathrm{id}$ on supplied cartesian models.

Definition 16.7.34 (Compatibility predicate for an arbitrary descended map)**.** Lean: `conditional-interface:def:weil-descended-scheme-map-compatibility`

Uses: `def:weil-canonical-basechange-descended-scheme-map`, `def:weil-descended-scheme-pullback-comparison`

For an $\mathbf{F}_q$-scheme map $f_0 : X_0 \to Y_0$ and a proposed map $f : X \to Y$, define IsDescendedMap(f_0, f) to be the typed comparison equation

$$\chi_Y \circ f = f_P \circ \chi_X.$$

Its two projection consequences are the structural compatibilities with the stored cartesian squares.

Lemma 16.7.35 (Compatibility of the selected transported map)**.** *Lean:* `blueprint-target:lemma:weil-transported-descended-scheme-map-compatible`

Uses: `def:weil-transported-descended-scheme-map`, `def:weil-descended-scheme-map-compatibility`

The selected transport of f_0 satisfies IsDescendedMap(f_0, f).

Proof. Uses: `def:weil-descended-scheme-pullback-comparison`

Substitute $f = \chi_Y^{-1} \circ f_P \circ \chi_X$ and cancel $\chi_Y \circ \chi_Y^{-1}$. □

Proposition 16.7.36 (Uniqueness of a compatible descended map)**.** *Lean:* `blueprint-target:prop:weil-descended-scheme-map-compatible-unique`

Uses: `def:weil-transported-descended-scheme-map`, `def:weil-descended-scheme-map-compatibility`

If $f : X \to Y$ satisfies IsDescendedMap(f_0, f)*, then it is equal to the selected transport of f_0.*

Proof. Uses: `def:weil-descended-scheme-pullback-comparison`

The compatibility equation is $\chi_Y \circ f = f_P \circ \chi_X$. Compose on the left with χ_Y^{-1}; the result is exactly $f = \chi_Y^{-1} \circ f_P \circ \chi_X$. □

Lemma 16.7.37 (Identity law for transported descended maps)**.** *Lean:* `blueprint-target:lemma:weil-transported-descended-scheme-map-identity`

Uses: `def:weil-transported-descended-scheme-map`

Transport of id_{X_0} *is* id_X.

Proof. Uses: `def:weil-canonical-basechange-descended-scheme-map`, `def:weil-descended-scheme-pullback-comparison`

The canonical base change of an identity is the identity of P_X. The transport formula reduces to $\chi_X^{-1} \circ \mathrm{id}_{P_X} \circ \chi_X$. □

Lemma 16.7.38 (Composition law for transported descended maps)**.** *Lean:* `blueprint-target:lemma:weil-transported-descended-scheme-map-composition`

Uses: `def:weil-transported-descended-scheme-map`

For composable $f_0 : X_0 \to Y_0$ and $g_0 : Y_0 \to Z_0$, transport of $g_0 \circ f_0$ equals the composite of their transported maps.

Proof. Uses: `def:weil-canonical-basechange-descended-scheme-map`, `def:weil-descended-scheme-pullback-comparison`

Pullback formation sends the composite to $g_P \circ f_P$. Substitute the three conjugation formulas and cancel the adjacent factor $\chi_Y \circ \chi_Y^{-1}$. □

Lemma 16.7.39 (Naturality of global absolute q-power Frobenius). *Lean:* `blueprint-target:lemma:weil-absolute-q-power-scheme-morphism-natural`

Uses: `def:weil-absolute-q-power-scheme-morphism`

For an $\mathbf{F}_q$-scheme morphism $f_0 : X_0 \to Y_0$,

$$f_0 \circ \mathrm{Fr}^{\mathrm{abs}}_{X_0,q} = \mathrm{Fr}^{\mathrm{abs}}_{Y_0,q} \circ f_0.$$

Proof. Uses: `prop:weil-absolute-q-power-scheme-morphism,def:weil-affine-q-power-ring-endomorphism`

On affine charts the reversed ring-map equality is $f_0^{\#}(b^q) = f_0^{\#}(b)^q$. It follows from multiplicativity of $f_0^{\#}$, and the affine specification glues these equalities globally. □

Lemma 16.7.40 (Naturality on canonical pullbacks). *Lean:* `blueprint-target:lemma:weil-canonical-pullback-frobenius-natural`

Uses: `def:weil-canonical-basechange-descended-scheme-map,def:weil-canonical-pullback-descended-frobenius`

For $f_0 : X_0 \to Y_0$, the canonical maps satisfy

$$f_P \circ F_{P_X} = F_{P_Y} \circ f_P.$$

Proof. Uses: `lemma:weil-absolute-q-power-scheme-morphism-natural`

After composing with the first pullback projection, the equality is the absolute-Frobenius naturality square for f_0. After composing with the second projection, both sides are the unchanged k-projection. The pullback homomorphism extensionality principle gives equality of the maps to P_Y. □

Proposition 16.7.41 (Naturality in descended scheme maps). *Lean:* `blueprint-target:prop:weil-descended-scheme-frobenius-functorial`

Uses: `def:weil-basechanged-descended-scheme-frobenius,def:weil-descended-scheme-map-compatibility`

For an $\mathbf{F}_q$-scheme map $f_0 : X_0 \to Y_0$ and an arbitrary $f : X \to Y$ satisfying $\mathrm{IsDescendedMap}(f_0, f)$,

$$f \circ F_X^{\mathrm{sch}} = F_Y^{\mathrm{sch}} \circ f.$$

Proof. Uses: `prop:weil-descended-scheme-map-compatible-unique,lemma:weil-canonical-pullback-frobenius-natural,lemma:weil-basechanged-descended-frobenius-coherence`

The uniqueness proposition identifies the proposed f with the selected transport. The canonical naturality equality gives the required square on P_X and P_Y. Conjugating it by χ_X and χ_Y gives the displayed equality for f on the chosen models. The separate identity and composition lemmas record the functor laws for the selected transports. □

16.7.4 Absolute and semilinear factorization

Definition 16.7.42 (Absolute q-power morphism on the geometric scheme). Lean: `conditional-interface:def:weil-absolute-q-power-on-geometric-scheme`

Uses: `def:weil-absolute-q-power-scheme-morphism,def:weil-descended-scheme-data`

View X as an $\mathbf{F}_q$-scheme through k. Define $\mathrm{Fr}^{\mathrm{abs}}_{X,q}$ to be its selected global absolute q-power morphism.

Lemma 16.7.43 (Absolute q-power stability on descended affines). *Lean:* `blueprint-target:lemma:weil-absolute-q-power-geometric-affine-stable`

Uses: `def:weil-absolute-q-power-on-geometric-scheme,def:weil-descended-affine-pullback-coordinate`

For $U = \pi^{-1}(U_0)$, $(\mathrm{Fr}^{\mathrm{abs}}_{X,q})^{-1}(U) = U$.

Proof. Uses: `prop:weil-absolute-q-power-scheme-morphism`

The selected global absolute map has identity underlying continuous map, so it fixes the inverse image of every open subset of X_0. □

Definition 16.7.44 (Absolute q-power restriction to a descended affine). Lean: `conditional-interface:def:weil-absolute-q-power-geometric-affine-restriction`

Uses: `def:weil-absolute-q-power-on-geometric-scheme,def:weil-descended-affine-pullback-coordinate`

Constructed by: `lemma:weil-absolute-q-power-geometric-affine-stable`

Define $\mathrm{Fr}^{\mathrm{abs}}_{U,q} : U \to U$ to be the restriction selected by the preceding stability equality.

Lemma 16.7.45 (Absolute affine-coordinate formula). Lean: `blueprint-target:lemma:weil-absolute-q-power-geometric-affine-formula`

Uses: `def:weil-absolute-q-power-geometric-affine-restriction`, `def:weil-descended-affine-coordinate-ring-equivalence`

In the coordinate Θ_{U_0}, pullback by $\mathrm{Fr}^{\mathrm{abs}}_{U,q}$ sends $a \otimes c$ to $a^q \otimes c^q$.

Proof. Uses: `prop:weil-absolute-q-power-scheme-morphism`, `lemma:weil-arithmetic-frobenius-power-formula`

The global affine specification applies the q-power endomorphism to the whole tensor-product ring. On a pure tensor this is $(a \otimes c)^q = a^q \otimes c^q$; the cited field formula identifies the second exponent with arithmetic field Frobenius. □

Lemma 16.7.46 (Semilinear stability on descended affines). Lean: `blueprint-target:lemma:weil-semilinear-field-morphism-affine-stable`

Uses: `def:weil-semilinear-field-morphism-on-geometric-scheme`, `def:weil-descended-affine-pullback-coordinate`

For $U = \pi^{-1}(U_0)$, one has $S_X^{-1}(U) = U$.

Proof. Uses: `def:weil-canonical-semilinear-field-morphism`, `def:weil-descended-scheme-pullback-comparison`

The canonical map S_{P_X} is the identity on the first pullback projection, so it preserves $P_U = \mathrm{pr}_1^{-1}(U_0)$. Conjugation by χ_X transports this equality of inverse images to U. □

Definition 16.7.47 (Semilinear restriction to a descended affine). Lean: `conditional-interface:def:weil-semilinear-field-morphism-affine-restriction`

Uses: `def:weil-semilinear-field-morphism-on-geometric-scheme`, `def:weil-descended-affine-pullback-coordinate`

Constructed by: `lemma:weil-semilinear-field-morphism-affine-stable`

Define $S_U : U \to U$ to be the restriction selected by the preceding stability equality.

Lemma 16.7.48 (Semilinear affine-coordinate formula). Lean: `blueprint-target:lemma:weil-semilinear-field-morphism-affine-formula`

Uses: `def:weil-semilinear-field-morphism-affine-restriction`, `def:weil-descended-affine-coordinate-ring-equivalence`

In the coordinate Θ_{U_0}, pullback by S_U sends $a \otimes c$ to $a \otimes F_k(c)$.

Proof. Uses: `def:weil-canonical-semilinear-field-morphism`, `def:weil-descended-scheme-pullback-comparison`

The canonical product map has pullback $\mathrm{id}_{A_0} \otimes F_k$. Conjugation by the selected coordinate comparison transports this exact formula to the chosen inverse image in X. □

Definition 16.7.49 (Absolute–semilinear composite candidate). Lean: `conditional-interface:def:weil-absolute-semilinear-composite-candidate`

Uses: `def:weil-scheme-frobenius-candidate`, `def:weil-absolute-q-power-on-geometric-scheme`, `def:weil-semilinear-field-morphism-on-geometric-scheme`

Define the candidate

$$C_X := \mathrm{Fr}^{\mathrm{abs}}_{X,q} \circ S_X : X \longrightarrow X.$$

Proposition 16.7.50 (Typed affine formula for the absolute–semilinear composite). Lean: `blueprint-target:prop:weil-absolute-semilinear-composite-affine-formula`

Uses: `def:weil-absolute-semilinear-composite-candidate`, `def:weil-scheme-frobenius-affine-formula`

The candidate C_X satisfies IsDescendedAffineFrobenius.

Proof. Uses: `lemma:weil-absolute-q-power-geometric-affine-stable`, `lemma:weil-semilinear-field-morphism-affine-stable`, `def:weil-absolute-q-power-geometric-affine-restriction`, `def:weil-semilinear-field-morphism-affine-restriction`, `lemma:weil-absolute-q-power-geometric-affine-formula`, `lemma:weil-semilinear-field-morphism-affine-formula`, `lemma:weil-arithmetic-frobenius-power-formula`, `lemma:weil-frobenius-inverse-identities`

Both factors preserve U, so their restricted composite is a typed map $C_U : U \to U$ and its square with j_U is the required restriction field. Pullback reverses composition. In the chosen coordinate it sends

$$a \otimes c \longmapsto a^q \otimes c^q \longmapsto a^q \otimes F_k(c^q) = a^q \otimes c.$$

The last equality is the inverse identity for field Frobenius. Tensor-product induction on zero, pure tensors, and addition upgrades this formula to the algebra-homomorphism equality with $\psi_{A_0,k}$. These are exactly the stability, restriction, and conjugacy fields of the typed predicate. □

Proposition 16.7.51 (Absolute/semilinear factorization). *Lean:* `blueprint-target:prop:weil-field-scheme-frobenius-factorization`

Uses: `def:weil-base-changed-descended-scheme-frobenius`, `def:weil-absolute-q-power-on-geometric-scheme`, `def:weil-semilinear-field-morphism-on-geometric-scheme`, `def:weil-field-scheme-frobenius-comparison`

With the fixed scheme-map order,

$$F_X^{\mathrm{sch}} = \mathrm{Fr}_{X,q}^{\mathrm{abs}} \circ S_X.$$

Consequently the selected maps satisfy the field–scheme Frobenius comparison predicate.

Proof. Uses: `prop:weil-descended-scheme-frobenius-affine-formula`, `prop:weil-descended-scheme-frobenius-unique`, `prop:weil-absolute-semilinear-composite-affine-formula`

The selected F_X^{sch} and the composite candidate C_X carry the same typed affine-formula predicate. The uniqueness proposition gives $F_X^{\mathrm{sch}} = C_X$. Unfolding C_X gives the displayed equality, and the typed comparison Definition says that this equality is exactly the requested predicate. □

16.7.5 Universal-homeomorphism certificates

Definition 16.7.52 (Radicial morphism certificate). Lean: `conditional-interface:def:weil-radicial-morphism-certificate`

Uses:

For a scheme morphism $h : T \to S$, define IsRadicial(h) to be the certificate that, after each base change $S' \to S$, the map $|T \times_S S'| \to |S'|$ is injective.

Definition 16.7.53 (Integral morphism certificate). Lean: `conditional-interface:def:weil-integral-morphism-certificate`

Uses:

For a scheme morphism $h : T \to S$, define IsIntegral(h) to mean that for each affine $V = \operatorname{Spec} A \subseteq S$, the inverse image is affine $\operatorname{Spec} B$ and every element of B is integral over the image of A.

Definition 16.7.54 (Universally surjective morphism certificate). Lean: `conditional-interface:def:weil-universally-surjective-morphism-certificate`

Uses:

For $h : T \to S$, define IsUniversallySurjective(h) to be the certificate that every base change of h is surjective on points.

Definition 16.7.55 (Universal-homeomorphism certificate). Lean: `conditional-interface:def:weil-universal-homeomorphism-certificate`

Uses:

For $h : T \to S$, define IsUniversalHomeomorphism(h) to be the certificate that every base change induces a homeomorphism of underlying topological spaces.

Lemma 16.7.56 (Radiciality of absolute q-power Frobenius). *Lean:* `blueprint-target:lemma:weil-absolute-frobenius-radicial`

Uses: `def:weil-absolute-q-power-scheme-morphism`, `def:weil-radicial-morphism-certificate`

For an $\mathbf{F}_q$-scheme Z, IsRadicial($\mathrm{Fr}_{Z,q}^{\mathrm{abs}}$).

Proof. Uses: `def:weil-affine-q-power-ring-endomorphism`, `prop:weil-absolute-q-power-scheme-morphism`

It suffices to check that each geometric fiber has at most one point. On an affine chart, after a field-valued base change $A \to K$, the fiber algebra is

$$B = A \otimes_{A,\varphi_{A,q}} K.$$

For each $a \in A$, the element $a \otimes 1$ has q-th power equal to the scalar $1 \otimes \overline{a}$. Consequently every element of B has a q-power in K: this follows from the freshman's-dream identity and the fact that the displayed elements generate B as a K-algebra. Such an element is either nilpotent or a unit. Every prime ideal of B, if one exists, therefore contains precisely the nonunits and equals the nilradical. Thus $\operatorname{Spec} B$ has at most one point. The same calculation applies after an arbitrary base change, giving the specified radicial certificate. □

Lemma 16.7.57 (Integrality of absolute q-power Frobenius). *Lean:* `blueprint-target:lemma:weil-absolute-frobenius-integral`

Uses: `def:weil-absolute-q-power-scheme-morphism`, `def:weil-integral-morphism-certificate`

For an $\mathbf{F}_q$-scheme Z, $\mathrm{IsIntegral}(\mathrm{Fr}^{\mathrm{abs}}_{Z,q})$.

Proof. Uses: `def:weil-affine-q-power-ring-endomorphism`, `prop:weil-absolute-q-power-scheme-morphism`

On $\operatorname{Spec} A$, regard the source A as an algebra over the target through $\varphi_{A,q}$. Each $x \in A$ satisfies the monic equation

$$T^q - x^q = 0,$$

whose coefficient $x^q = \varphi_{A,q}(x)$ lies in the target image. Thus the map is integral on the affine chart. For a ring base change $A \to A'$, the source becomes $A \otimes_{A,\varphi_{A,q}} A'$. It is generated over A' by the elements $x \otimes 1$, each satisfying $T^q - (1 \otimes \overline{x})$. The same monic-equation argument proves integrality after base change, and it also shows directly that inverse images of affine opens remain affine. □

Lemma 16.7.58 (Universal surjectivity of absolute q-power Frobenius). *Lean:* `blueprint-target:lemma:weil-absolute-frobenius-universally-surjective`

Uses: `def:weil-absolute-q-power-scheme-morphism`, `def:weil-universally-surjective-morphism-certificate`

For an $\mathbf{F}_q$-scheme Z, $\mathrm{IsUniversallySurjective}(\mathrm{Fr}^{\mathrm{abs}}_{Z,q})$.

Proof. Uses: `lemma:weil-absolute-frobenius-integral`, `def:weil-affine-q-power-ring-endomorphism`

Consider a field-valued point $A \to K$ of an affine base change and its fiber $B = A \otimes_{A,\varphi_{A,q}} K$. Choose an algebraic closure Ω of K. Frobenius is an automorphism of Ω, so

$$A \longrightarrow \Omega, \qquad a \longmapsto \left(\overline{a}\right)^{1/q}$$

is a ring homomorphism and, together with $K \hookrightarrow \Omega$, defines a unital map $B \to \Omega$. Thus B is nonzero and has a prime ideal. Every geometric fiber is nonempty. Hence each base-changed map is surjective on points. Equivalently, one may apply lying over to the integral algebra from the preceding lemma; the explicit map to Ω verifies the needed nonzero-fiber hypothesis. □

Lemma 16.7.59 (Integral, radicial, and universally surjective criterion). *Lean:* `blueprint-target:lemma:weil-integral-radicial-surjective-universal-homeomorphism`

Uses: `def:weil-radicial-morphism-certificate`, `def:weil-integral-morphism-certificate`, `def:weil-universally-surjective-morphism-certificate`, `def:weil-universal-homeomorphism-certificate`

If $h : T \to S$ has the first three displayed certificates, then it has the universal-homeomorphism certificate.

Proof. Uses: `def:weil-integral-morphism-certificate`

After any base change, integrality is preserved by the tensor-algebra argument in its definition. An integral affine map is closed: for an ideal $I \subseteq B$, the image of $V(I)$ is $V(I \cap A)$ by lying over for the integral extension $A/(I \cap A) \to B/I$. Therefore every base change of h is closed. The other two certificates make its map on points injective and surjective. A continuous closed bijection is a homeomorphism, which is the required certificate after each base change. □

Proposition 16.7.60 (Universal-homeomorphism package for descended Frobenius). *Lean:* `blueprint-target:prop:weil-descended-scheme-frobenius-universal-homeomorphism`

Uses: `def:weil-descended-q-frobenius-over-fq`, `def:weil-base-changed-descended-scheme-frobenius`, `def:weil-universal-homeomorphism-certificate`

Both $F^{\mathrm{sch}}_{X_0}$ and F^{sch}_{X} carry the selected universal-homeomorphism certificate.

Proof. Uses: `lemma:weil-absolute-frobenius-radicial`, `lemma:weil-absolute-frobenius-integral`, `lemma:weil-absolute-frobenius-universally-surjective`, `lemma:weil-integral-radicial-surjective-universal-homeomorphism`, `def:weil-descended-scheme-pullback-comparison`

The three property lemmas apply to the absolute q-power map on X_0, and the criterion gives its universal-homeomorphism certificate. The canonical map on P_X is its base change along $\operatorname{Spec} k \to \operatorname{Spec} \mathbf{F}_q$, so the universal certificate applies there. The selected map on X is conjugate to that map by χ_X; pre- and post-composition by an isomorphism preserve each base-changed homeomorphism. □

16.7.6 Finite-type finiteness and properness

Definition 16.7.61 (Finite morphism certificate)**.** Lean: `conditional-interface:def:weil-finite-morphism-certificate`

Uses:

For a scheme morphism $h : T \to S$, define IsFinite(h) to be the certificate that for each affine $V = \operatorname{Spec} A \subseteq S$, the inverse image is affine $\operatorname{Spec} B$ and B is a finite A-module through $h^{\#}$.

Definition 16.7.62 (Proper morphism certificate)**.** Lean: `conditional-interface:def:weil-proper-morphism-certificate`

Uses:

For $h : T \to S$, define IsProper(h) to be the standard certificate consisting of finite type, separatedness, and universal closedness.

Lemma 16.7.63 (Finite module over the q-power image)**.** Lean: *`blueprint-target:lemma:weil-frobenius-finite-type-affine-module-finite`*

Uses: *`def:weil-affine-q-power-ring-endomorphism`*

If A is generated over $\mathbf{F}_q$ by $x_1, \dots, x_d$, then A is a finite module over $\varphi_{A,q}(A)$.

Proof. Uses: `def:weil-finite-field-data`

The q^d residue monomials

$$x_1^{r_1} \cdots x_d^{r_d}, \qquad 0 \le r_i < q,$$

span. Indeed, write each exponent as $qm_i + r_i$ and move the factor $(x_1^{m_1} \cdots x_d^{m_d})^q$ into the image. Coefficients in $\mathbf{F}_q$ are fixed by q-power and hence also lie in the image. Expanding a polynomial in the selected generators proves the spanning claim; passing to a quotient preserves it. □

Lemma 16.7.64 (Finite module after scalar base change)**.** Lean: *`blueprint-target:lemma:weil-frobenius-basechanged-affine-module-finite`*

Uses: *`def:weil-descended-affine-frobenius-tensor-map`*

If A_0 is a finite-type $\mathbf{F}_q$-algebra, then $A_0 \otimes_{\mathbf{F}_q} k$ is a finite module over the image of $\psi_{A_0,k}$.

Proof. Uses: `lemma:weil-frobenius-finite-type-affine-module-finite,lemma:weil-descended-affine-frobenius-pure-tensor`

Choose residue monomials spanning A_0 over A_0^q. Their tensors with 1 span $A_0 \otimes k$ over the image of $\psi_{A_0,k}$: the image contains $A_0^q \otimes 1$ and contains $1 \otimes k$ because $\psi_{A_0,k}$ fixes the scalar factor. Thus the same finite family is a module-spanning family after scalar extension. □

Lemma 16.7.65 (Finite-morphism descent from an affine target cover)**.** Lean: *`blueprint-target:lemma:weil-finite-morphism-affine-cover-descent`*

Uses: *`def:weil-finite-morphism-certificate`*

Let $h : T \to S$ and let $(V_i = \operatorname{Spec} A_i)$ be an affine open cover of S. Suppose each inverse image is $h^{-1}(V_i) = \operatorname{Spec} B_i$ and B_i is finite as an A_i-module. Then IsFinite(h).

Proof. Uses: `def:weil-finite-morphism-certificate`

Let $V = \operatorname{Spec} A \subseteq S$ be an arbitrary affine target open. The intersections $V \cap V_i$ admit a cover by principal opens $D(f)$ of V lying in some V_i. Quasi-compactness of V selects a finite such cover. Over $D(f)$, the inverse image is the localization of $\operatorname{Spec} B_i$ and its coordinate ring is a finite A_f-module. On double intersections these finite algebras are identified by canonical localization maps. Glue them to the quasi-coherent finite $\mathcal{O}_V$-algebra $\mathcal{B} = h_*\mathcal{O}_{h^{-1}(V)}$. Finite module generators descend over the finite principal cover. If $B = \Gamma(V, \mathcal{B})$, the canonical map $h^{-1}(V) \to \operatorname{Spec} B$ is an isomorphism after restriction to each $D(f)$ and hence globally. Therefore $h^{-1}(V)$ is affine and B is a finite A-module. Since V was arbitrary, these are all fields of the finite-morphism certificate. □

Lemma 16.7.66 (Finite-morphism transport across isomorphisms)**.** Lean: *`blueprint-target:lemma:weil-finite-morphism-isomorphism-transport`*

Uses: *`def:weil-finite-morphism-certificate`*

Let $h : T \to S$ be finite, let $\alpha : T' \xrightarrow{\sim} T$ and $\beta : S' \xrightarrow{\sim} S$, and define $h' = \beta^{-1} \circ h \circ \alpha$. Then h' is finite.

Proof. Uses: `def:weil-finite-morphism-certificate`

For an arbitrary affine $V' \subseteq S'$, the image $\beta(V')$ is affine. Its inverse image under h is affine with a finite coordinate module. The isomorphism α identifies that inverse image with $(h')^{-1}(V')$, and β identifies the two target coordinate rings. Transport of a finite generating family through these algebra equivalences gives the required finite module over $\Gamma(V', \mathcal{O}_{S'})$. □

Proposition 16.7.67 (Global finiteness on the descended model). *Lean:* `blueprint-target:prop:weil-descended-scheme-frobenius-finite-x0`

Uses: `def:weil-descended-q-frobenius-over-fq`, `def:weil-finite-morphism-certificate`

If X_0 is finite type over $\mathbf{F}_q$, then IsFinite($F_{X_0}^{\mathrm{sch}}$).

Proof. Uses: `lemma:weil-frobenius-finite-type-affine-module-finite`, `prop:weil-absolute-q-power-scheme-morphism`, `lemma:weil-finite-morphism-affine-cover-descent`

Cover X_0 by finite-type affines $\operatorname{Spec} A$. Absolute Frobenius has the same affine as inverse image, and its coordinate map makes the source ring finite over the target by the preceding module lemma. The affine-cover descent lemma starts with an arbitrary affine $V = \operatorname{Spec} R \subseteq X_0$ and supplies an affine inverse image with a finite R-module. Thus the certificate is quantified over every affine target open, not merely the initially selected cover. □

Proposition 16.7.68 (Finiteness on the canonical scalar pullback). *Lean:* `blueprint-target:prop:weil-canonical-pullback-descended-frobenius-finite`

Uses: `def:weil-canonical-pullback-descended-frobenius`, `def:weil-finite-morphism-certificate`

If X_0 is finite type over $\mathbf{F}_q$, then IsFinite(F_{P_X}).

Proof. Uses: `lemma:weil-frobenius-basechanged-affine-module-finite`, `lemma:weil-canonical-pullback-frobenius-affine-ring-map`, `lemma:weil-finite-morphism-affine-cover-descent`

Pull back a finite-type affine cover of X_0 to an affine cover of P_X. On each member the ring map is $\psi_{A_0,k}$, and the scalar-base-change module lemma gives a finite module. Affine-cover descent supplies the finite-morphism certificate for every affine target open of P_X. □

Proposition 16.7.69 (Finiteness on the chosen base-changed model). *Lean:* `blueprint-target:prop:weil-descended-scheme-frobenius-finite-x`

Uses: `def:weil-basechanged-descended-scheme-frobenius`, `def:weil-finite-morphism-certificate`

If X_0 is finite type over $\mathbf{F}_q$, then IsFinite(F_X^{sch}).

Proof. Uses: `prop:weil-canonical-pullback-descended-frobenius-finite`, `lemma:weil-finite-morphism-isomorphism-transport`, `def:weil-descended-scheme-pullback-comparison`

The canonical pullback proposition gives finiteness of F_{P_X}. Apply the isomorphism-transport lemma with the source and target comparison isomorphisms both equal to χ_X; the transported morphism is $\chi_X^{-1} \circ F_{P_X} \circ \chi_X = F_X^{\mathrm{sch}}$. □

Lemma 16.7.70 (A finite scheme morphism is proper). *Lean:* `blueprint-target:lemma:weil-finite-morphism-proper-contract`

Uses: `def:weil-finite-morphism-certificate`, `def:weil-proper-morphism-certificate`

Every morphism carrying the finite-morphism certificate carries the proper morphism certificate.

Proof. Uses: `def:weil-integral-morphism-certificate`

On an affine chart $A \to B$, module finiteness gives algebra finiteness and integrality. The diagonal is a closed immersion because the multiplication map $B \otimes_A B \to B$ is surjective, so the morphism is separated. After any base change, the tensors of a finite A-module generating family still generate; hence the base change is finite and integral. For $I \subseteq B$, lying over for $A/(I \cap A) \to B/I$ identifies the image of $V(I)$ with $V(I \cap A)$. Thus every base change is closed. The morphism is finite type, separated, and universally closed, which supplies the proper certificate. □

Proposition 16.7.71 (Properness on the descended model). *Lean:* `blueprint-target:prop:weil-descended-scheme-frobenius-proper-x0`

Uses: `def:weil-descended-q-frobenius-over-fq`, `def:weil-proper-morphism-certificate`

If X_0 is finite type over $\mathbf{F}_q$, then $F_{X_0}^{\mathrm{sch}}$ carries the proper-morphism certificate.

Proof. Uses: `prop:weil-descended-scheme-frobenius-finite-x0`, `lemma:weil-finite-morphism-proper-contract`

Apply the finite-implies-proper lemma to the finite certificate on X_0. □

Proposition 16.7.72 (Properness on the chosen scalar extension). *Lean:* `blueprint-target:prop:weil-descended-scheme-frobenius-proper-x`

Uses: `def:weil-basechanged-descended-scheme-frobenius`, `def:weil-proper-morphism-certificate`

If X_0 is finite type over $\mathbf{F}_q$, then F_X^{sch} carries the proper-morphism certificate.

Proof. Uses: `prop:weil-descended-scheme-frobenius-finite-x`,`lemma:weil-finite-morphism-proper-contract`

Apply the finite-implies-proper lemma to the finite certificate on the chosen X. □

Proposition 16.7.73 (Finiteness and properness assembly). *Lean:* `blueprint-target:prop:weil-descended-scheme-frobenius-finite-proper`

Uses: `def:weil-descended-q-frobenius-over-fq`, `def:weil-basechanged-descended-scheme-frobenius`, `def:weil-finite-morphism-certificate`, `def:weil-proper-morphism-certificate`

In the finite-type setting, the selected Frobenius on X_0 and its transported base extension on X have the finite and proper certificates.

Proof. Uses: `prop:weil-descended-scheme-frobenius-finite-x0`,`prop:weil-descended-scheme-frobenius-finite-x`,`prop:weil-descended-scheme-frobenius-proper-x0`,`prop:weil-descended-scheme-frobenius-proper-x`

The first two propositions supply the finite certificates. The last two propositions supply the proper certificates. No additional construction is introduced here. □

Bibliography

[1] The Stacks Project Authors, *The Stacks Project*, https://stacks.math.columbia.edu/.